

FC7300 SDK User Guide

Rev. 2.0

Table of Contents

FC7300 SDK 2.0 Driver cluster	20
Description.....	20
History	20
Module Index.....	21
Class Index.....	26
File Index	41
Module Documentation.....	46
HwA_adc	46
HwA_aontimer	88
HwA_cmp	98
HwA_cmu	126
HwA_cordic.....	132
HwA_cpm	137
HwA_crc	143
HwA_csc	151
HwA_dma.....	174
HwA_dmamux	218
HwA_eim	221
HwA_enet.....	225
ENET system control	435
ENET MAC Configuration and Status	435
HwA_erm.....	652
HwA_fciic.....	654
HwA_fcpit.....	691
HwA_fcsmu	703
HwA_fcuart	724
HwA_flexcan	749
HwA_freqm	815
HwA_ftu.....	819
HwA_gpio	877
HwA_hsadc	881
HwA_ism	943
HwA_lu	965
HwA_mam.....	972
HwA_mb.....	976

HwA_msc.....	990
HwA_ospi	1011
HwA_overlay	1037
HwA_pmc	1046
HwA_port	1048
HwA_ptimer	1056
HwA_rgm.....	1077
HwA_rtc.....	1092
HwA_scg.....	1102
HwA_scm.....	1134
HwA_sdadc	1138
HwA_sddf	1301
HwA_sec.....	1383
HwA_sent	1399
HwA_smc.....	1442
HwA_tmu.....	1448
HwA_trgsel.....	1464
HwA_tstmp	1466
HwA_wdog	1469
HwA_wku	1472
Module_driver_adc	1474
Module_driver_aontimer	1488
Module_driver_cmp	1493
Module_driver_cmu	1502
Module_driver_cordic	1515
Module_driver_cpm	1522
Module_driver_crc	1528
Module_driver_csc	1535
Module_driver_dma	1538
Module_driver_eim	1556
Module_driver_enet.....	1560
Module_driver_erm.....	1628
Module_driver_fciic.....	1632
Module_driver_fcpi.....	1681
Module_driver_fcsmu	1689
Module_driver_fcspi.....	1704
Module_driver_fcuart.....	1716
Module_driver_flash	1737

Module_driver_flexcan	1751
Module_driver_freqm	1764
Module_driver_ftu	1770
Module_driver_gpio	1806
Module_driver_hsadc.....	1814
Fc7xxx_driver_hsm.....	1828
Module_driver_hsm_Level1_LevelN.....	1865
Module_driver_intm.....	1923
Module_driver_ism.....	1930
Module_driver_lin	1938
Module_driver_lu	1956
Module_driver_mam.....	1961
Module_driver_mb.....	1963
Module_driver_msc.....	1971
Module_driver_ospi	1990
Module_driver_overlay	2001
Module_driver_pcc.....	2005
Module_driver_pmc	2013
Module_driver_port	2021
Module_driver_ptimer	2034
Module_driver_rgm.....	2043
Module_driver_rtc.....	2051
Module_driver_scg.....	2060
Module_driver_scm.....	2087
Module_driver_scst.....	2090
Module_driver_sdadc.....	2097
Module_driver_sddf	2137
Module_driver_sec	2150
Module_driver_sent	2165
Module_driver_smc.....	2187
Module_driver_tmu.....	2191
Module_driver_trgsel	2196
Module_driver_tstmp.....	2201
Module_driver_wdog	2208
Module_driver_wku	2214
Class Documentation	2217
_ADC_HandleType.....	2217
_AONTIMER_HandleType.....	2219

_CMP_HandleType	2220
_CMU_HandleType.....	2222
_CPM_HandleType	2223
_DMA_HandleType	2224
_EIM_HandleType.....	2225
_ERM_HandleType	2226
_FCIIC_DmaConfig.....	2227
_FCIIC_MasterHandle	2228
_FCIIC_SlaveHandle	2232
_FCPIT_HandleType.....	2236
_FCSMU_HandleType.....	2238
_FCSPI_HandleType.....	2240
_FCUART_HandleType	2246
_FLASH_HandleType.....	2249
_FLEXCAN_HandleType	2251
_flexspi_transfer.....	2255
_FREQM_HandleType.....	2257
_FTU_HandleType	2259
_HSADC_HandleType.....	2261
_INTM_HandleType.....	2264
_ISM_HandleType.....	2265
_LIN_HandleType	2269
_MB_HandleType	2270
_MSC_HandleType	2271
_PMC_HandleType	2272
_PORT_HandleType	2273
_PTIMER_HandleType.....	2274
_RGM_HandleType.....	2276
_RTC_HandleType.....	2277
_SCG_HandleType.....	2279
_SDADC_HandleType	2281
_SDDF_HandleType.....	2283
_Sent_HandleType.....	2284
_TMU_HandleType.....	2286
_TSTMP_HandleType.....	2288
_WDOG_HandleType.....	2290
ADC_ChannelCfgType	2291
ADC_CompareType	2292

ADC_InitType	2294
AONTIMER_InitType	2297
CMP_ChannelScanType	2299
CMP_ComparatorType.....	2300
CMP_CSChannelCfgType	2303
CMP_DacType	2304
CMP_InitType.....	2305
CMP_InterruptType	2307
CMP_MuxType	2309
CMU_CfgType	2311
CORDIC_SinCos_Type	2313
CORDIC_SinhCosh_Type.....	2314
CPM_InterruptType	2315
Crc_ConfigType	2316
Crc_HandleType	2317
CSC0_AONCLKSRType	2318
CSC0_ClkoutType.....	2319
DMA_ChainTransferType	2320
DMA_ChannelCfgType.....	2321
DMA_CircularBufferType.....	2325
DMA_InitType.....	2326
DMA_InstanceHandleType.....	2327
EIM_ChCfgType.....	2329
ENET_Buffer	2331
ENET_Config	2333
ENET_DescConfig	2336
ENET_DescList	2338
ENET_DMACHannelConfig.....	2340
ENET_DMAConfig.....	2342
ENET_DMADescType	2344
ENET_DrvHandle	2345
ENET_EstConfig	2348
ENET_FlexPPSConfigType	2350
ENET_FlowControl	2352
ENET_FlowControlRxQ	2354
ENET_GateCtrl	2355
ENET_GateCtrlList	2356
ENET_L3L4Filter	2358

ENET_MACConfig.....	2361
ENET_MACFilter	2365
ENET_MiiConfigType	2367
ENET_MMCCConfig	2368
ENET_MTLConfig	2370
ENET_MTLQueueConfig	2372
ENET_OuterVlanCBTI	2375
ENET_PreemptionConfig	2376
Enet_PTPConfig.....	2378
ENET_RxInfo	2381
ENET_TimestampType.....	2382
ENET_TxOption	2383
ENET_TxVlanConfig.....	2385
ENET_VlanFilterConfig	2386
ENET_VlanPerfectFilter	2388
ERM_CfgType	2390
FCIIC_InterruptType.....	2391
FCIIC_MasterConfigType.....	2393
FCIIC_MasterTransferType.....	2395
FCIIC_RxDataType	2397
FCIIC_SlaveConfigType	2398
FCIIC_SlaveTransferType	2401
FCIIC_TxDataType	2403
FCPIT_ChannelInitType	2404
FCPIT_CommonInitType.....	2405
FCPIT_IntType	2406
FCPIT_TriggerType	2407
FCSMU_CrcCfgType.....	2408
FCSMU_InitCfgType.....	2409
FCSMU_StatusOutputCfgType	2411
FCUART_DataType	2413
FCUART_InitType.....	2414
FCUART_InterruptType	2418
FCUART_WakeupType.....	2421
FLASH_DRIVER_ParamType	2422
FLASH_Lock_ParamType.....	2424
FLEXCAN_BaudRegType	2426
FLEXCAN_EnhancedFifoConfigType	2428

FLEXCAN_ErrorInfoType	2430
FLEXCAN_FifoConfigType	2432
FLEXCAN_FifoFilterType	2434
FLEXCAN_FifoIssueType.....	2435
FLEXCAN_InitType	2437
FLEXCAN_LegacyFifoConfigType	2440
FLEXCAN_MbConfigType	2442
FLEXCAN_MbSettingType	2444
FLEXCAN_PNET_ConfigType	2446
FLEXCAN_PNET_IDConfigType.....	2448
FLEXCAN_PNET_IdSettingType.....	2450
FLEXCAN_PNET_PayloadConfigType.....	2451
FLEXCAN_RxMsgListType.....	2453
FLEXCAN_RxMsgType.....	2454
FLEXCAN_TxMsgType	2456
FLEXCAN_WakeupMsgType	2458
FPC_Type	2460
FREQM_CfgType	2461
FTU_CommonType	2462
FTU_CounterModeType	2465
FTU_EdgeNumberMeasureType.....	2466
FTU_ExpectEdgeNumberMeasureType	2468
FTU_ExpectEdgeNumberResultType	2469
FTU_FaultControlType.....	2470
FTU_FaultInitType.....	2471
FTU_InputChannelType	2473
FTU_OutputCompareModeType	2474
FTU_PwmChannelType.....	2475
FTU_PwmDutyUpdateType	2477
FTU_PwmModeType	2478
FTU_QuadratureInitType	2480
FTU_SignalMeasureType	2482
FTU_SignalMeasureValueType	2483
HostNVRPermission	2484
HostUKPermission.....	2485
HSADC_ChannelCfgType.....	2486
HSADC_CmpType	2487
HSADC_FastCmpType	2489

HSADC_InitType	2492
HSADC_SeqGroupType	2496
HSM_AesEnDecryptType.....	2497
HSM_AESM_CFG_KEY_CFG_T.....	2499
HSM_AESM_ENTRY_CFG_T.....	2500
HSM_AesmRawApiType	2502
HSM_AesNewKeyInfType	2503
HSM_AesVendorKeyInfType	2504
HSM_BigNumberCalcExtType	2505
HSM_BigNumberCalcType.....	2507
HSM_CcmDecryptType	2509
HSM_CcmEncryptType.....	2511
HSM_CmdType	2514
Hsm_configuration.....	2515
HSM_DrvAesDecryptInfType.....	2517
HSM_DrvAesEncryptInfType.....	2518
HSM_DrvBnCalcExtInfType.....	2520
HSM_DrvBnCalcInfType	2522
HSM_DrvCMacInfType.....	2524
HSM_DrvEccCalcInfType.....	2526
HSM_DrvEccCurveParamType.....	2529
HSM_DrvEccDecryptInfType.....	2531
HSM_DrvEccSignInfType.....	2534
HSM_DrvEccVerifyInfType.....	2537
HSM_DrvKeyManagelInfType	2540
HSM_DrvKeySpaceInfType	2541
HSM_DrvLoadFirmwareInfType	2542
HSM_DrvMd5InfType.....	2543
HSM_DrvRandPoolType.....	2545
HSM_DrvRsaInfType.....	2546
HSM_DrvScatterCMacInfType.....	2548
HSM_DrvScatterHashInfType	2551
HSM_DrvShaInfType	2555
HSM_DrvSm2GenZaInfType	2557
HSM_DrvSm2VerifyInfType.....	2558
HSM_DrvSm3InfType	2560
HSM_DrvSm4DecryptInfType	2562
HSM_DrvSm4EncryptInfType	2563

HSM_DrvTrngInfType	2565
HSM_EccCalcType	2567
HSM_EccEasyDecryType	2569
HSM_EccEasyEncryType	2571
HSM_EccSignType	2573
HSM_EccVerifyType	2575
HSM_FlexAesEnDecryptType	2577
HSM_GcmDecryptType	2580
HSM_GcmEncryptType	2582
HSM_KeyManagerCheckUserKeyStatusExtType	2584
HSM_KeyManagerExportUserKeyType	2585
HSM_KeyManagerGenUserKeyType	2587
HSM_KeyManagerImportUserKeyType	2588
HSM_L1_BnCalcType	2590
HSM_L1_DrvRSASsaPkcs1V15SignType	2593
HSM_L1_DrvRSASsaPkcs1V15VerifyType	2594
HSM_L1_DrvRSASsaPssSignType	2595
HSM_L1_DrvRSASsaPssVerifyType	2596
HSM_L1_EccCurveParamType	2597
HSM_L1_EccVerifyType	2599
HSM_L1_InitParmType	2602
HSM_L1_PKCS8_PrivateKeyInfType	2604
HSM_L1_PKCS_EccPrivateKeyInfType	2605
HSM_L1_PKCS_EccPublicKeyInfType	2606
HSM_L1_PKCS_RsaPrivateKeyInfType	2607
HSM_L1_PKCS_RsaPublicKeyInfType	2608
HSM_L1_PublicKeyInfType	2609
HSM_L1_RsaType	2610
HSM_L1_RtCtxType_tag	2613
HSM_L1_Sm2VerifyType	2618
HSM_L2_DrvCMacEmptyInfType	2621
HSM_L2_DrvEccVerifyInfType	2623
HSM_L2_DrvRevokeKeyInfType	2624
HSM_L2_DrvSm2VerifyInfType	2625
HSM_L2_DrvUpdateKeyInfType	2626
HSM_L2_InitParmType	2627
HSM_L2_RtCtxType_tag	2628
HSM_L3_DrvScatterCMacInfType	2630

HSM_L3_DrvUpdateKeyInfType	2631
HSM_L3_InitParmType	2632
HSM_L3_RtCtxType_tag.....	2633
HSM_Ln_AesDecryptType.....	2635
HSM_Ln_AesEncryptType.....	2638
HSM_Ln_CMacCfgParmsType	2641
HSM_Ln_CMacType.....	2642
HSM_Ln_CopyUserKeyType	2645
HSM_Ln_EccDecryptType.....	2646
HSM_Ln_EccKPTYPE	2649
HSM_Ln_EccSignType.....	2651
HSM_Ln_EccVerifyType.....	2653
HSM_Ln_HashMacCfgType	2654
HSM_Ln_ImportUserKeyExtType	2656
HSM_Ln_InitParmType	2658
HSM_Ln_KeySpaceStatusType.....	2660
HSM_Ln_Md5Type.....	2661
HSM_Ln_PlainKeyImport.....	2663
HSM_Ln_ResultBufInfoType.....	2664
HSM_Ln_RngType	2665
HSM_Ln_RsaSsaPkcs1V15SignDataType.....	2666
HSM_Ln_RsaSsaPkcs1V15SignHashType	2668
HSM_Ln_RsaSsaPkcs1v15VerifyType.....	2670
HSM_Ln_RsaSsaPssSignDataType	2673
HSM_Ln_RsaSsaPssSignHashType	2675
HSM_Ln_RsaSsaPssVerifyType.....	2677
HSM_Ln_RtCtxType.....	2680
HSM_Ln_ScatterCMacType.....	2681
HSM_Ln_ScatterHashType	2682
HSM_Ln_ShaType.....	2683
HSM_LN_Sm2GenZaType	2685
HSM_Ln_Sm2VerifyType.....	2687
HSM_Ln_Sm3Type	2688
HSM_Md5ExType	2690
HSM_Md5Type.....	2691
HSM_RsaSsaPkcs1V15SignType.....	2692
HSM_RsaSsaPkcs1V15VerifyType.....	2695
HSM_RsaSsaPssSignType	2698

HSM_RsaSsaPssVerifyType	2701
HSM_RsaType	2704
HSM_ScatterHashType	2706
HSM_Sha2Type	2707
HSM_ShaExType	2708
HSM_ShaResultBufType	2710
HSM_ShaType	2711
HSM_Sm2DecryptType	2712
HSM_Sm2EncryptType	2714
HSM_Sm2GenHashType	2716
HSM_Sm2GenKeyPairType	2717
HSM_Sm2GenZaType	2718
HSM_Sm2SignType	2720
HSM_Sm2VerifyType	2722
HSM_Sm3ExType	2724
HSM_Sm3Type	2725
HSM_Sm4EnDecryptType	2726
HSM_SmsCfgType	2728
HSM_TrnglInitCfgType	2730
HSM_XCMacExType	2732
HSM_XCMacType	2734
HSMCom_AesDecryptType	2736
HSMCom_AesEncryptType	2737
HSMCom_AesmRawApiType	2738
HSMCom_AuthCheckType	2739
HSMCom_BigNumberCalcExtType	2741
HSMCom_BigNumberCalcType	2743
HSMCom_CancelJobType	2745
HSMCom_CcmDecryptType	2746
HSMCom_CcmEncryptType	2747
HSMCom_CMacExType	2748
HSMCom_CMacType	2749
HSMCom_EccCalcType	2750
HSMCom_EccEasyDecryType	2752
HSMCom_EccEasyEncryType	2754
HSMCom_EccSignType	2756
HSMCom_EccVerifyType	2758
HSMCom_FastTrueRandType	2760

HSMCom_FlexAesDecryptType	2761
HSMCom_FlexAesEncryptType	2762
HSMCom_GcmDecryptType	2763
HSMCom_GcmEncryptType	2764
HSMCom_GenUserKeyType.....	2765
HSMCom_GetRomHashType	2766
HSMCom_KeySpaceStatusType	2767
HSMCom_LifeCycleChangeType	2768
HSMCom_LoadFirmwareType.....	2770
HSMCom_Md5ExType	2771
HSMCom_Md5Type	2772
HSMCom_MonCountType.....	2773
HSMCom_NvrOtpType	2774
HSMCom_RequestAuthType	2775
HSMCom_RsaSsaPkcs1v15SignType	2776
HSMCom_RsaSsaPkcs1V15VerifyType	2777
HSMCom_RsaSsaPssSignType	2779
HSMCom_RsaSsaPssVerifyType	2780
HSMCom_RsaType	2782
HSMCom_ScatterHashType	2784
HSMCom_SelfTestType.....	2785
HSMCom_Sha2Type	2787
HSMCom_ShaExType	2788
HSMCom_ShaType.....	2789
HSMCom_Sm2DecryptType	2790
HSMCom_Sm2EncryptType.....	2792
HSMCom_Sm2GenHashType	2794
HSMCom_Sm2GenKeyPairType	2795
HSMCom_Sm2GenZaType.....	2796
HSMCom_Sm2SignType	2798
HSMCom_Sm2VerifyType	2800
HSMCom_Sm3ExType.....	2802
HSMCom_Sm3Type.....	2803
HSMCom_Sm4EnDecryptType	2804
HSMCom_TrueRandExType.....	2805
HSMCom_TrueRandType.....	2806
HSMCom_UserKeyManageType	2807
HSMCom_XMacType	2809

HSMWrap_Sm4EncryptExType.....	2810
HSMWrap_Sm4EncryptType.....	2811
INTM_InitType	2812
INTM_MonitorType.....	2814
ISM_FpcCfgType	2815
ISM_InitCfgType.....	2816
ISM_LamCfgType.....	2817
LAM_Type.....	2819
LIN_BaudRateCfgType	2820
LIN_ConfigType	2821
LIN_PduType	2823
LIN_XferStateType.....	2825
LU_AoiConfigType.....	2828
LU_ConfigType	2830
LU_InConfigType	2832
LU_InputsSyncCtrlType	2833
MAM_Inf_Type	2834
MB_InitType.....	2835
MB_ReceiveType.....	2836
MB_RequestType.....	2838
MSC_InitCfgType	2839
MSC_InterruptCfgType.....	2841
MSC_IOControlInitType	2843
NVR_Per	2845
OSPI_ConfigType.....	2846
OSPI_DeviceConfigType.....	2847
OVERLAY_ErrorInfoType	2849
OVERLAY_FARInitType	2852
OVERLAY_InterruptType	2853
OVERLAY_OverlayRegionInitType	2854
PCC_CtrlType.....	2856
PMC_ConfigType	2857
PORT_DigitalFilterType.....	2858
PORT_InitType.....	2859
PORT_InterruptType	2861
PTIMER_ChannelCfgType	2862
PTIMER_InitType	2864
PTIMER_PulseOutType.....	2866

ReEnDebug_Keys.....	2867
RGM_ConfigType	2869
RTC_Compensation.....	2871
RTC_InitType	2872
RTC_TimeDate	2874
SCG_ClockCtrlType	2875
SCG_FircType.....	2877
SCG_FoscType	2879
SCG_InterruptType.....	2881
SCG_PllType.....	2883
SCG_Sirc32kType	2885
SCG_SircType	2886
SCG_SoscType	2888
SDADC_ChannelInitType.....	2890
SDADC_ChannelRegType	2894
SDADC_DetectCfgType.....	2896
SDADC_FilterChainCfgType	2900
SDADC_InitType.....	2903
SDADC_IntegrationCfgType.....	2906
SDADC_IntInitType	2908
SDADC_ModulatorCfgType	2910
SDADC_PWMInitType	2912
SDADC_RectCfgType	2913
SDDF_AuxFilterConfigType.....	2915
SDDF_ChannelConfigType.....	2916
SDDF_ChannelDMAConfigType	2918
SDDF_ChannelInterruptConfigType	2920
SDDF_ChannelNotificationType.....	2922
SDDF_ClockAbsenceDetectConfigType	2924
SDDF_ClockOutConfigType	2926
SDDF_InitConfigType	2928
SDDF_LimitCheckConfigType.....	2929
SDDF_MainFilterConfigType.....	2931
SDDF_ShortCircuitDetectConfigType	2933
SDDF_ZeroCrossDetectConfigType.....	2935
SEC_BootInfo.....	2936
SEC_IsplInfo	2938
Sent_ChannelConfigType	2940

Sent_ChannelInterruptType.....	2943
Sent_ConfigType	2945
Sent_DMAConfigType.....	2947
Sent_FastMessageDataType.....	2948
Sent_SlowMessageDataType	2949
Sent_SpcConfigType	2951
STCU_ConfigType.....	2953
STCU_InitRamConfigType	2955
TMU_InitType.....	2956
TSTMP_InitType.....	2958
TSTMP_IntType.....	2959
TSTMP_UpdateType.....	2960
WDOG_CfgType	2961
File Documentation	2963
HwA_adc.h	2963
HwA_aontimer.h.....	2974
HwA_cm7.h	2978
HwA_cmp.h.....	2980
HwA_cmu.h.....	2988
HwA_cordic.h.....	2991
HwA_cpm.h.....	2994
HwA_crc.h	2997
HwA_csc.h.....	3000
HwA_dma.h.....	3008
HwA_dmamux.h	3021
HwA_eftu_ccm.h	3023
HwA_eftu_cmu.h.....	3024
HwA_eftu_dtm.h	3025
HwA_eftu_tbu.h.....	3026
HwA_eftu_tim.h.....	3027
HwA_eftu_tom.h.....	3028
HwA_eim.h.....	3029
HwA_elu.h	3031
HwA_enet.h.....	3032
HwA_erm.h	3143
HwA_fciic.h	3145
HwA_fcpit.h.....	3154
HwA_fcsmu.h.....	3159

HwA_fcspi.h	3165
HwA_fcuart.h	3177
HwA_flexcan.h	3185
HwA_freqm.h	3207
HwA_ftu.h	3209
HwA_gpio.h	3226
HwA_hrpwm.h	3228
HwA_hsadc.h	3229
HwA_intm.h	3245
HwA_ism.h	3248
HwA_ldi.h	3254
HwA_lu.h	3255
HwA_mam.h	3258
HwA_mb.h	3260
HwA_msc.h	3265
HwA_ospic.h	3272
HwA_overlay.h	3280
HwA_pmc.h	3284
HwA_port.h	3286
HwA_ptimer.h	3289
HwA_qdt.h	3296
HwA_rgm.h	3297
HwA_rtc.h	3302
HwA_scg.h	3306
HwA_scm.h	3319
HwA_sdadc.h	3321
HwA_sddf.h	3369
HwA_sec.h	3391
HwA_sent.h	3396
HwA_smc.h	3410
HwA_ssi.h	3412
HwA_stcu.h	3413
HwA_tmu.h	3421
HwA_tpue.h	3426
HwA_tpuh.h	3427
HwA_trgsel.h	3428
HwA_tstmp.h	3430
HwA_wdog.h	3432

HwA_wku.h	3434
module_driver_adc.h.....	3436
module_driver_aontimer.h	3441
module_driver_cmp.h.....	3444
module_driver_cmu.h.....	3448
module_driver_cordic.h	3452
module_driver_cpm.h.....	3455
module_driver_crc.h.....	3457
module_driver_csc.h.....	3460
module_driver_dma.h	3462
module_driver_dsp.h.....	3467
module_driver_eftu.h	3469
module_driver_eftu_ccm.h	3470
module_driver_eftu_cmu.h.....	3471
module_driver_eftu_dtm.h	3472
module_driver_eftu_tbu.h	3473
module_driver_eftu_tim.h	3474
module_driver_eftu_tom.h	3475
module_driver_eim.h.....	3476
module_driver_elu.h.....	3478
module_driver_enet.h	3479
module_driver_erm.h	3500
module_driver_fciic.h	3502
module_driver_fcpi.h	3514
module_driver_fcsmu.h.....	3518
module_driver_fcspi.h	3524
module_driver_fcuart.h.....	3528
module_driver_flash.h.....	3534
module_driver_flexcan.h	3539
module_driver_freqm.h.....	3544
module_driver_ftu.h	3546
module_driver_gpio.h	3556
module_driver_hrpwm.h.....	3559
module_driver_hsadc.h.....	3560
module_driver_hsm.h.....	3565
module_driver_hsm_level1_leveln.h.....	3622
module_driver_intm.h	3642
module_driver_ism.h.....	3645

module_driver_ldi.h.....	3649
module_driver_lin.h.....	3650
module_driver_lu.h.....	3655
module_driver_mam.h	3658
module_driver_mb.h	3660
module_driver_msc.h	3663
module_driver_ospi.h.....	3668
module_driver_overlay.h.....	3672
module_driver_pcc.h	3674
module_driver_pmc.h.....	3677
module_driver_port.h.....	3680
module_driver_ptimer.h.....	3685
module_driver_qdt.h	3689
module_driver_rgm.h	3690
module_driver_rtc.h	3693
module_driver_scg.h	3697
module_driver_scm.h	3704
module_driver_scst.h	3706
module_driver_sdadc.h.....	3708
module_driver_sddf.h.....	3718
module_driver_sec.h	3723
module_driver_sema.h.....	3728
module_driver_sent.h.....	3729
module_driver_smc.h	3736
module_driver_ssi.h.....	3738
module_driver_stcu.h.....	3739
module_driver_tmu.h.....	3748
module_driver_tpu.h	3750
module_driver_trgsel.h	3751
module_driver_tstmp.h.....	3754
module_driver_wdog.h.....	3757
module_driver_wku.h.....	3760

FC7300 SDK 2.4 Driver cluster

Project	FC7300 SDK 2.4
Author	Flagchip
Source	Refer to tools.flagchip.com.cn

Description

New SDK for user and export more API and parameters

Notice

This Software Development Kit (SDK) is provided solely for functional validation and reference purposes only. It includes example code and configurations to assist in prototyping and evaluation. The SDK is not optimized for mass production. Unverified performance, edge cases, or compatibility issues may exist.

It is not recommended for direct use of SDK in production environments. Flagchip assumes no liability for issues arising from direct or indirect SDK usage in production, including functional defects or financial losses.

History

Date	Version	Author	Description
2024-11-26	1.0	Flagchip	First version for SDK 2.0
2025-07-24	2.0	Flagchip	Updated for SDK 2.4

Module Index

Modules

Here is a list of all modules:

Module_driver_adc	1474
HwA_adc.....	46
Module_driver_aontimer.....	1488
HwA_aontimer	88
Module_driver_cmp.....	1493
HwA_cmp	98
Module_driver_cmu.....	1502
HwA_cmu	126
Module_driver_cordic.....	1515
HwA_cordic	132
Module_driver_cpm.....	1522
HwA_cpm	137
Module_driver_crc	1528
HwA_crc.....	143
Module_driver_csc	1535
HwA_csc	151
Module_driver_dma.....	1538
HwA_dma	174
HwA_dmamux.....	218
Module_driver_eim	1556

HwA_eim	221
Module_driver_enet.....	1560
HwA_enet	225
Module_driver_erm.....	1628
HwA_erm	652
Module_driver_fciic	1632
HwA_fciic.....	654
Module_driver_fcpit.....	1681
HwA_fcpit	691
Module_driver_fcsmu	1689
HwA_fcsmu	703
Module_driver_fcspi	1704
Module_driver_fcuart	1716
HwA_fcuart.....	724
Module_driver_flash	1737
Module_driver_flexcan	1751
HwA_flexcan.....	749
Module_driver_freqm	1764
HwA_freqm	815
Module_driver_ftu.....	1770
HwA_ftu	819
Module_driver_gpio.....	1806
HwA_gpio	877

Module_driver_hsadc	1814
HwA_hsadc.....	881
Fc7xxx_driver_hsm	1828
Module_driver_hsm_Level1_LevelN.....	1865
Module_driver_intm	1923
Module_driver_ism	1930
HwA_ism.....	943
Module_driver_lin	1938
Module_driver_lu	1956
HwA_lu	965
Module_driver_mam.....	1961
HwA_mam.....	972
Module_driver_mb.....	1963
HwA_mb	976
Module_driver_msc	1971
HwA_msc.....	990
Module_driver_ospi	1990
HwA_ospi.....	1011
Module_driver_overlay	2001
HwA_overlay	1037
Module_driver_pcc	2005
Module_driver_pmc.....	2013
HwA_pmc	1046
Module_driver_port	2021
HwA_port	1048

Module_driver_ptimer	2034
HwA_ptimer	1056
Module_driver_rgm.....	2043
HwA_rgm	1077
Module_driver_rtc.....	2051
HwA_rtc	1092
Module_driver_scg.....	2060
HwA_scg	1102
Module_driver_scm	2087
HwA_scm	1134
Module_driver_scst.....	2090
Module_driver_sdadc	2097
HwA_sdadc.....	1138
Module_driver_sddf.....	2137
HwA_sddf	1301
Module_driver_sec.....	2150
HwA_sec	1383
Module_driver_sent	2165
HwA_sent	1399
Module_driver_smc	2187
HwA_smc.....	1442
Module_driver_tmu	2191
HwA_tmu.....	1448

Module_driver_trgsel.....	2196
HwA_trgsel	1464
Module_driver_tstmp	2201
HwA_tstmp.....	1466
Module_driver_wdog.....	2208
HwA_wdog	1469
Module_driver_wku	2214
HwA_wku.....	1472

Class Index

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

_ADC_HandleType (The structure of the ADC processing handle)	2217
_AONTIMER_HandleType (The structure of the AONTIMER processing handle)	2219
_CMP_HandleType (The structure of the CMP processing handle)	2220
_CMU_HandleType (The structure of the CMU processing handle)	2222
_CPM_HandleType (The structure of the CPM processing handle)	2223
_DMA_HandleType	2224
_EIM_HandleType (The structure of the EIM processing handle)	2225
_ERM_HandleType (The structure of the ERM processing handle)	2226
_FCIIC_DmaConfig (Structure for configuring DMA for I2C Master and Slave operations)	2227
_FCIIC_MasterHandle (Structure for handling the FCIIC Master operations. Implements : _FCIIC_MasterHandle)	2228
_FCIIC_SlaveHandle (Structure for handling the FCIIC Slave operations. Implements : _FCIIC_SlaveHandle)	2232
_FCPIT_HandleType	2236
_FCSMU_HandleType (The structure of the FCSMU processing handle)	2238
_FCSPI_HandleType (The structure of the FCSPI processing handle)	2240
_FCUART_HandleType (The structure of the UART processing handle)	2246
_FLASH_HandleType (The structure of the FLASH processing handle)	2249
_FLEXCAN_HandleType (The structure of the CAN processing handle)	2251
_flexspi_transfer (FlexSPI Transfer structure definition)	2255
_FREQM_HandleType (The structure of the FREQM processing handle)	2257
_FTU_HandleType (The structure of the FTU processing handle)	2259
_HSADC_HandleType (The structure of the CMU processing handle Implements : _HSADC_HandleType)	2261
_INTM_HandleType (The structure of the INTM handle)	2264
_ISM_HandleType	2265
_LIN_HandleType (LIN driver handle structure (entry point for all API operations))	2269
_MB_HandleType	2270
_MSC_HandleType	2271

_PMC_HandleType (The structure of the SCG processing handle)	2272
_PORT_HandleType (The structure of the PORT processing handle)	2273
_PTIMER_HandleType (Ptimer process handler)	2274
_RGM_HandleType (The structure of the SCG processing handle)	2276
_RTC_HandleType (The structure of the RTC processing handle)	2277
_SCG_HandleType (The structure of the SCG processing handle)	2279
_SDADC_HandleType	2281
_SDDF_HandleType	2283
_Sent_HandleType (The structure of the SENT processing handle)	2284
_TMU_HandleType (The structure of the TMU processing handle)	2286
_TSTMP_HandleType (The structure of the Tstmp processing handle)	2288
_WDOG_HandleType (The structure of the WDOG processing handle)	2290
ADC_ChannelCfgType (The configuration option for the ADC channel)	2291
ADC_CompareType (Defines the hardware compare configuration)	2292
ADC_InitType (Defines the converter configuration)	2294
AONTIMER_InitType (Aontimer Initialization struct type)	2297
CMP_ChannelScanType (Defines the comparator channel scan configuration)	2299
CMP_ComparatorType (The instance index of the CMP Comparator parts)	2300
CMP_CSChannelCfgType (Defines the comparator channel selected in channel scan mode)	2303
CMP_DacType (The instance index of the CMP Dac parts)	2304
CMP_InitType (Defines the comparator configuration)	2305
CMP_InterruptType (Defines the comparator interrupt configuration)	2307
CMP_MuxType (The instance index of the CMP Mux parts)	2309
CMU_CfgType (CMU configure structure)	2311
CORDIC_SinCos_Type (Circular sine and cosine type)	2313
CORDIC_SinhCosh_Type (Circular hyperbolic sine and cosine type)	2314
CPM_InterruptType (Cpm interrupt configuration type)	2315
Crc_ConfigType (Defines the crc configuration)	2316
Crc_HandleType (The structure of the SENT processing handle)	2317
CSC0_AONCLKSRType (CSC0_AONCLKSR clock source info)	2318
CSC0_ClkoutType (Data type for register CSC0_CLKOUT_CTRL,mainly focus on CLKOUT setting)	2319
DMA_ChainTransferType (The paarameters to configure the DMA chained transfer)	2320
DMA_ChannelCfgType (The configutation parameters of the DMA channel)	2321

DMA_CircularBufferType (The parameters to configure the DMA channel as a circular buffer engine)	2325
DMA_InitType (The configuration parameters of the DMA engine)	2326
DMA_InstanceHandleType (The structure of the DMA processing handle)	2327
EIM_ChCfgType (Define the max EIM channels)	2329
ENET_Buffer (Defines the buffer type for ENET driver)	2331
ENET_Config (ENET configuration type)	2333
ENET_DescConfig (ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA.It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache)	2336
ENET_DescList (DMA descriptor list type for internal use by ENET driver)	2338
ENET_DMACHannelConfig (DMA channel configuration type)	2340
ENET_DMAConfig (ENET DMA configuration type)	2342
ENET_DMADescType (ENET DMA descriptor type)	2344
ENET_DrvHandle (ENET handle is internal used by ENET driver)	2345
ENET_EstConfig (ENET EST configuration type)	2348
ENET_FlexPPSConfigType (IEEE1588 flexible PPS configuration type)	2350
ENET_FlowControl (ENET flow control configuration type)	2352
ENET_FlowControlRxQ (ENET flow control receive queue configuration type)	2354
ENET_GateCtrl (The ENET EST Gate control type)	2355
ENET_GateCtrlList (The ENET EST Gate control List type)	2356
ENET_L3L4Filter (ENET Layer3 Layer4 filter configuration type)	2358
ENET_MACConfig (ENET MAC configuration type)	2361
ENET_MACFilter (ENET MAC filter configuration type)	2365
ENET_MiiConfigType (Mdio configuration type)	2367
ENET_MMConfig (Defines MMC counter configuration)	2368
ENET_MTLConfig (ENET MTL configuration type)	2370
ENET_MTLQueueConfig (ENET MTL Queue configuration type)	2372
ENET_OuterVlanCBTI (ENET Channel based VLAN insertion type)	2375
ENET_PreemptionConfig (The ENET EST Frame Preemption configuration type)	2376
Enet_PTPConfig (IEEE1588 time stamp configuration type)	2378
ENET_RxInfo (Defines the received buffer information)	2381
ENET_TimestampType (IEEE1588 time stamp type)	2382
ENET_TxOption (Defines the options for the Tx packet)	2383

ENET_TxVlanConfig (ENET transmit VLAN configuration type)	2385
ENET_VlanFilterConfig (ENET VLAN filter configuration type)	2386
ENET_VlanPerfectFilter (ENET VLAN perfect filter type)	2388
ERM_CfgType (The structure of the ERM Init configuration)	2390
FCIIC_InterruptType (Structure for configuring FCIIC interrupts)	2391
FCIIC_MasterConfigType (Structure for configuring the FCIIC Master)	2393
FCIIC_MasterTransferType (Structure for defining a master transfer in the FCIIC)	2395
FCIIC_RxDataType (Structure for holding received data from FCIIC operations)	2397
FCIIC_SlaveConfigType (Configuration structure for initializing the FCIIC (Fast Communication Interface IC) Slave module)	2398
FCIIC_SlaveTransferType (Structure for defining a slave transfer in the FCIIC)	2401
FCIIC_TxDataType (Structure for holding transmit data for FCIIC operations)	2403
FCPIT_ChannellInitType (Fcpit initialization type)	2404
FCPIT_CommonInitType	2405
FCPIT_IntType (Fcpit interrupt structure)	2406
FCPIT_TriggerType	2407
FCSMU_CrcCfgType (FCSMU CRC configure structure)	2408
FCSMU_InitCfgType (FCSMU init configure structure)	2409
FCSMU_StatusOutputCfgType (FCSMU status output pin configure structure)	2411
FCUART_DataType (UART transmit and receive data define)	2413
FCUART_InitType (UART initial data define)	2414
FCUART_InterruptType (UART callback functions data define)	2418
FCUART_WakeupType (UART WakeUp functions data define)	2421
FLASH_DRIVER_ParamType (Flash driver parameter define)	2422
FLASH_Lock_ParamType (Unlock parameter define)	2424
FLEXCAN_BaudRegType (Baud-rate clock and divider)	2426
FLEXCAN_EnhancedFifoConfigType (FLEXCAN enhanced fifo configuration structure definition)	2428
FLEXCAN_ErrorInfoType (FLEXCAN Error Information)	2430
FLEXCAN_FifoConfigType (FLEXCAN fifo configuration structure definition)	2432
FLEXCAN_FifoFilterType (Receive Message Buffer Configuration)	2434
FLEXCAN_FifoIssueType (FIFO error response type)	2435
FLEXCAN_InitType (FLEXCAN init structure definition)	2437
FLEXCAN_LegacyFifoConfigType (FLEXCAN legacy fifo configuration structure definition)	2440

FLEXCAN_MbConfigType (FLEXCAN mb configuration structure definition)	2442
FLEXCAN_MbSettingType (Message Buffer Configuration)	2444
FLEXCAN_PNET_ConfigType (CAN Pretended network configuration)	2446
FLEXCAN_PNET_IDConfigType (CAN Pretended network id filter setting)	2448
FLEXCAN_PNET_IdSettingType (CAN ID filter setting)	2450
FLEXCAN_PNET_PayloadConfigType	2451
FLEXCAN_RxMsgListType (FLEXCAN polling receive message)	2453
FLEXCAN_RxMsgType (FLEXCAN receive message)	2454
FLEXCAN_TxMsgType (FLEXCAN transmit message)	2456
FLEXCAN_WakeupMsgType (FLEXCAN wake-up receive message)	2458
FPC_Type	2460
FREQM_CfgType (FREQM configure structure)	2461
FTU_CommonType (The basic configuration option for the FTU peripheral)	2462
FTU_CounterModeType (The configuration option for the time counter mode)	2465
FTU_EdgeNumberMeasureType (FTU edge number measurement configuration type)	2466
FTU_ExpectEdgeNumberMeasureType (FTU expect edge number measurement configuration type)	2468
FTU_ExpectEdgeNumberResultType (Expect edge number result time)	2469
FTU_FaultControlType (The configuration option for enabling a FTU fault)	2470
FTU_FaultInitType (The configuration option for initializing FTU fault)	2471
FTU_InputChannelType (The configuration option for a input capture channel)	2473
FTU_OutputCompareModeType (The configuration option for the output compare mode)	2474
FTU_PwmChannelType (The configuration for the Pwm channel)	2475
FTU_PwmDutyUpdateType (The configuration option for update PWM Duty)	2477
FTU_PwmModeType (The configuration option for the Pwm Mode)	2478
FTU_QuadratureInitType (The configuration option for the quadrature decoder mode)	2480
FTU_SignalMeasureType (FTU signal measurement configuration type)	2482
FTU_SignalMeasureValueType (FTU signal measurement result time)	2483
HostNVRPermission (Indicate the Host NVR Read/Write/Erase Protection)	2484
HostUKPermission (Indicate the Host User Key Read/Write/Erase Protection)	2485
HSADC_ChannelCfgType (The configuration option for the HSADC channel)	2486
HSADC_CmpType (Defines the hardware compare configuration)	2487
HSADC_FastCmpType (Defines the hardware fast compare configuration)	2489
HSADC_InitType (Defines the converter configuration)	2492

HSADC_SeqGroupType (The configuration option for the HSADC sequence group)	2496
HSM_AesEnDecryptType (AES encrypt/decrypt information used by driver when call specific API)	
.....	2497
HSM_AESM_CFG_KEY_CFG_T	2499
HSM_AESM_ENTRY_CFG_T	2500
HSM_AesmRawApiType	2502
HSM_AesNewKeyInfType (Key information used by AES when use one key in software)	2503
HSM_AesVendorKeyInfType (Key information used by AES when use one key in nvr flash)	2504
HSM_BigNumberCalcExtType	2505
HSM_BigNumberCalcType (RSA information used by driver when call specific API)	2507
HSM_CcmDecryptType (CCM Decrypt information used by driver when call specific API)	2509
HSM_CcmEncryptType (CCM Encrypt information used by driver when call specific API)	2511
HSM_CmdType (HSM cmd type definition)	2514
Hsm_configuration (Indicate the HSM configuration)	2515
HSM_DrvAesDecryptInfType (Aes decrypt information type)	2517
HSM_DrvAesEncryptInfType (Aes encrypt information type)	2518
HSM_DrvBnCalcExtInfType	2520
HSM_DrvBnCalcInfType (Big number calculator information type)	2522
HSM_DrvCMacInfType (Cmac information type)	2524
HSM_DrvEccCalcInfType (ECC calculate info type)	2526
HSM_DrvEccCurveParamType (ECC curve information used by driver when call specific API)	
.....	2529
HSM_DrvEccDecryptInfType (Ecc decrypt information type)	2531
HSM_DrvEccSignInfType (Ecc sign information type)	2534
HSM_DrvEccVerifyInfType (Ecc verify information type)	2537
HSM_DrvKeyManageInfType	2540
HSM_DrvKeySpaceInfType	2541
HSM_DrvLoadFirmwareInfType	2542
HSM_DrvMd5InfType (Md5 information type)	2543
HSM_DrvRandPoolType (Drive rand pool type)	2545
HSM_DrvRsaInfType (Rsa information type)	2546
HSM_DrvScatterCMacInfType (Scatter SM CMac information type)	2548
HSM_DrvScatterHashInfType (Scatter hash information type)	2551
HSM_DrvShaInfType (Sha information type)	2555

HSM_DrvSm2GenZaInfType	2557
HSM_DrvSm2VerifyInfType	2558
HSM_DrvSm3InfType (Sm3 information type)	2560
HSM_DrvSm4DecryptInfType (Sm4 decrypt information type)	2562
HSM_DrvSm4EncryptInfType (Sm4 encrypt information type)	2563
HSM_DrvTrngInfType (Trng information type)	2565
HSM_EccCalcType (ECC point calculate used by driver when call specific API) ..	2567
HSM_EccEasyDecryType (ECC decrypt information used by driver when call specific API)	2569
HSM_EccEasyEncryType (ECC encrypt information used by driver when call specific API)	2571
HSM_EccSignType (ECC sign information used by driver when call specific API)	2573
HSM_EccVerifyType (ECC verify information used by driver when call specific API)	2575
HSM_FlexAesEnDecryptType (Flex AES encrypt/decrypt information used by driver when call specific API)	2577
HSM_GcmDecryptType (GCM Decrypt information used by driver when call specific API)	2580
HSM_GcmEncryptType (GCM Encrypt information used by driver when call specific API)	2582
HSM_KeyManagerCheckUserKeyStatusExtType	2584
HSM_KeyManagerExportUserKeyType	2585
HSM_KeyManagerGenUserKeyType	2587
HSM_KeyManagerImportUserKeyType	2588
HSM_L1_BnCalcType	2590
HSM_L1_DrvRSASsaPkcs1V15SignType	2593
HSM_L1_DrvRSASsaPkcs1V15VerifyType (Level 1 RSAssa Pkcs15 verify type) ...	2594
HSM_L1_DrvRSASsaPssSignType	2595
HSM_L1_DrvRSASsaPssVerifyType (Level 1 RSAssa Pss verify type)	2596
HSM_L1_EccCurveParamType (Ecc curve type)	2597
HSM_L1_EccVerifyType	2599
HSM_L1_InitParmType (Ecc level 1 init type)	2602
HSM_L1_PKCS8_PrivateKeyInfType	2604
HSM_L1_PKCS_EccPrivateKeyInfType	2605
HSM_L1_PKCS_EccPublicKeyInfType	2606
HSM_L1_PKCS_RsaPrivateKeyInfType	2607
HSM_L1_PKCS_RsaPublicKeyInfType	2608
HSM_L1_PublicKeyInfType	2609
HSM_L1_RsaType (L1 Rsa config type)	2610

HSM_L1_RtCtxType_tag (The structure of the HSM level 1 processing handle) ..	2613
HSM_L1_Sm2VerifyType	2618
HSM_L2_DrvCMacEmptyInfType (The structure of the HSM level 2 processing handle)	2621
HSM_L2_DrvEccVerifyInfType	2623
HSM_L2_DrvRevokeKeyInfType	2624
HSM_L2_DrvSm2VerifyInfType	2625
HSM_L2_DrvUpdateKeyInfType	2626
HSM_L2_InitParmType (Ecc level 2 init type)	2627
HSM_L2_RtCtxType_tag (The structure of the HSM level 2 processing handle) ..	2628
HSM_L3_DrvScatterCMacInfType (The structure of the level 3 scatter cmac information type)	
.....	2630
HSM_L3_DrvUpdateKeyInfType	2631
HSM_L3_InitParmType (Ecc level 3 init type)	2632
HSM_L3_RtCtxType_tag (The structure of the HSM level 3 processing handle) ..	2633
HSM_Ln_AesDecryptType (The structure of the aes decrypt type)	2635
HSM_Ln_AesEncryptType (The structure of the aes encrypt type)	2638
HSM_Ln_CMacCfgParmsType (The structure of the level 3 cmac config type)	2641
HSM_Ln_CMacType (CMAC information used by driver when call specific API) ..	2642
HSM_Ln_CopyUserKeyType	2645
HSM_Ln_EccDecryptType (Ecc decrypt type)	2646
HSM_Ln_EccKPTType	2649
HSM_Ln_EccSignType (Ecc sign config type)	2651
HSM_Ln_EccVerifyType (Ecc verify type)	2653
HSM_Ln_HashMacCfgType (Ln hash mac config type)	2654
HSM_Ln_ImportUserKeyExtType	2656
HSM_Ln_InitParmType (Init information for HSM_Ln_Init)	2658
HSM_Ln_KeySpaceStatusType	2660
HSM_Ln_Md5Type (MD5 information used by driver when call specific API)	2661
HSM_Ln_PlainKeyImport	2663
HSM_Ln_ResultBufInfoType (HSM result buffer type)	2664
HSM_Ln_RngType (Rng information used by driver when call specific API)	2665
HSM_Ln_RsaSsaPkcs1V15SignDataType	2666
HSM_Ln_RsaSsaPkcs1V15SignHashType	2668

HSM_Ln_RsaSsaPkcs1v15VerifyType (RSA PKCS1 V1.5 signature verify information used by driver when call specific API)	2670
HSM_Ln_RsaSsaPssSignDataType	2673
HSM_Ln_RsaSsaPssSignHashType	2675
HSM_Ln_RsaSsaPssVerifyType (RSA PSS signature verify information used by driver when call specific API)	2677
HSM_Ln_RtCtxType (The structure of the HSM processing handle)	2680
HSM_Ln_ScatterCMacType (Scatter cmac information used by driver when call specific API)	2681
HSM_Ln_ScatterHashType (Scatter hash level n type)	2682
HSM_Ln_ShaType (Sha information used by driver when call specific API)	2683
HSM_LN_Sm2GenZaType	2685
HSM_Ln_Sm2VerifyType	2687
HSM_Ln_Sm3Type (Sm3 information used by driver when call specific API)	2688
HSM_Md5ExType (MD5 information used by driver when call specific API)	2690
HSM_Md5Type (MD5 information used by driver when call specific API)	2691
HSM_RsaSsaPkcs1V15SignType (RSA PKCS1 V1.5 signature information used by driver when call specific API)	2692
HSM_RsaSsaPkcs1V15VerifyType (RSA PKCS1V1.5 signature verify information used by driver when call specific API)	2695
HSM_RsaSsaPssSignType	2698
HSM_RsaSsaPssVerifyType (RSA PSS signature verify information used by driver when call specific API)	2701
HSM_RsaType (RSA information used by driver when call specific API)	2704
HSM_ScatterHashType (Scatter Hash information used by driver when call specific API)	2706
HSM_Sha2Type (SHA information used by driver when call specific API)	2707
HSM_ShaExType (SHA extend information used by driver when call specific API)	2708
HSM_ShaResultBufType (Type definition for driver to store information)	2710
HSM_ShaType (SHA information used by driver when call specific API)	2711
HSM_Sm2DecryptType (SM2 decrypt information used by driver when call specific API)	2712
HSM_Sm2EncryptType (SM2 encrypt information used by driver when call specific API)	2714
HSM_Sm2GenHashType (SM2 HASH generation information used by driver when call specific API)	2716
HSM_Sm2GenKeyPairType (SM2 key pair generation information used by driver when call specific API)	2717

HSM_Sm2GenZaType (SM2 ZA generation information used by driver when call specific API)	
.....	2718
HSM_Sm2SignType (SM2 sign information used by driver when call specific API)	2720
HSM_Sm2VerifyType (SM2 verify information used by driver when call specific API)	2722
HSM_Sm3ExType	2724
HSM_Sm3Type (SM3 information used by driver when call specific API)	2725
HSM_Sm4EnDecryptType (SM4 encrypt/decrypt information used by driver when call specific API)	
.....	2726
HSM_SmsCfgType (SHA/MD5/SM3 context information used by driver when call specific API)	
.....	2728
HSM_TrngInitCfgType	2730
HSM_XCMacExType	2732
HSM_XCMacType (CMAC/XMAC information used by driver when call specific API)	2734
HSMCom_AesDecryptType	2736
HSMCom_AesEncryptType	2737
HSMCom_AesmRawApiType	2738
HSMCom_AuthCheckType	2739
HSMCom_BigNumberCalcExtType	2741
HSMCom_BigNumberCalcType	2743
HSMCom_CancelJobType (Cancel job used by driver when call specific API)	2745
HSMCom_CcmDecryptType	2746
HSMCom_CcmEncryptType	2747
HSMCom_CMacExType	2748
HSMCom_CMacType	2749
HSMCom_EccCalcType (ECC point calculate used by driver when call specific API)	2750
HSMCom_EccEasyDecryType (ECC decrypt information used by driver when call specific API)	
.....	2752
HSMCom_EccEasyEncryType (ECC encrypt information used by driver when call specific API)	
.....	2754
HSMCom_EccSignType (ECC sign information used by driver when call specific API)	2756
HSMCom_EccVerifyType (ECC verify information used by driver when call specific API)	2758
HSMCom_FastTrueRandType	2760
HSMCom_FlexAesDecryptType	2761
HSMCom_FlexAesEncryptType	2762

HSMCom_GcmDecryptType	2763
HSMCom_GcmEncryptType	2764
HSMCom_GenUserKeyType	2765
HSMCom_GetRomHashType	2766
HSMCom_KeySpaceStatusType	2767
HSMCom_LifeCycleChangeType (Change lifecycle used by driver when call specific API)	2768
HSMCom_LoadFirmwareType	2770
HSMCom_Md5ExType	2771
HSMCom_Md5Type	2772
HSMCom_MonCountType	2773
HSMCom_NvrOtpType	2774
HSMCom_RequestAuthType (Request Auth used by driver when call specific API)	2775
HSMCom_RsaSsaPkcs1v15SignType	2776
HSMCom_RsaSsaPkcs1V15VerifyType	2777
HSMCom_RsaSsaPssSignType	2779
HSMCom_RsaSsaPssVerifyType	2780
HSMCom_RsaType	2782
HSMCom_ScatterHashType (Scatter Hash information used by driver when call specific API)	2784
HSMCom_SelfTestType	2785
HSMCom_Sha2Type (SHA information used by driver when call specific API)	2787
HSMCom_ShaExType (SHA extend information used by driver when call specific API)	2788
HSMCom_ShaType (SHA information used by driver when call specific API)	2789
HSMCom_Sm2DecryptType (SM2 decrypt information used by driver when call specific API)	2790
HSMCom_Sm2EncryptType (SM2 encrypt information used by driver when call specific API)	2792
HSMCom_Sm2GenHashType (SM2 HASH generation information used by driver when call specific API)	2794
HSMCom_Sm2GenKeyPairType (SM2 key pair generation information used by driver when call specific API)	2795
HSMCom_Sm2GenZaType (SM2 ZA generation information used by driver when call specific API)	2796
HSMCom_Sm2SignType (SM2 sign information used by driver when call specific API)	2798

HSMCom_Sm2VerifyType (SM2 verify information used by driver when call specific API)	2800
HSMCom_Sm3ExType	2802
HSMCom_Sm3Type	2803
HSMCom_Sm4EnDecryptType	2804
HSMCom_TrueRandExType (True random extend information passed to HSM core by mailbox)	2805
HSMCom_TrueRandType (True random information passed to HSM core by mailbox)	2806
HSMCom_UserKeyManageType	2807
HSMCom_XMacType	2809
HSMWrap_Sm4EncryptExType	2810
HSMWrap_Sm4EncryptType	2811
INTM_InitType (The structure of the INTM init)	2812
INTM_MonitorType	2814
ISM_FpcCfgType	2815
ISM_InitCfgType	2816
ISM_LamCfgType	2817
LAM_Type	2819
LIN_BaudRateCfgType (Baudrate configuration parameters)	2820
LIN_ConfigType (LIN controller hardware configuration parameters)	2821
LIN_PduType (LIN Protocol Data Unit (PDU) structure)	2823
LIN_XferStateType (Runtime state of the LIN driver (maintained during operation))	2825
LU_AoiConfigType (LU AOI IN(n) configuration type)	2828
LU_ConfigType (LU initialization type)	2830
LU_InConfigType (LU IN(n) configuration register type)	2832
LU_InputsSyncCtrlType (LU LG inputs synchronous control)	2833
MAM_Inf_Type (MAM slave information)	2834
MB_InitType (Configuration of the Mailbox)	2835
MB_ReceiveType (Configuration of a receiving request)	2836
MB_RequestType (Configuration of a Mailbox request)	2838
MSC_InitCfgType	2839
MSC_InterruptCfgType (MSC interrupt configure structure)	2841
MSC_IOControlInitType	2843
NVR_Per (NVR write and read Permissions. @)	2845
OSPI_ConfigType (The basic configuration option for the OSPI peripheral)	2846

OSPI_DeviceConfigType (Define the structure for OSPI device configuration) ...	2847
OVERLAY_ErrorInfoType (Error Info)	2849
OVERLAY_FARInitType (FAR parameter)	2852
OVERLAY_InterruptType (Interrupt Parameter)	2853
OVERLAY_OverlayRegionInitType (Overlay Region parameters)	2854
PCC_CtrlType (Define the PCC module initialization structure)	2856
PMC_ConfigType (Pmc control type)	2857
PORT_DigitalFilterType (Port digital filter configuration structure)	2858
PORT_InitType (Port initialization structure)	2859
PORT_InterruptType (Port interrupt configuration structure)	2861
PTIMER_ChannelCfgType (Ptimer channel configuration parameters)	2862
PTIMER_InitType (Ptimer instance initialization parameters)	2864
PTIMER_PulseOutType (Ptimer pulse out configgurations)	2866
ReEnDebug_Keys (Debug Re-Enable Key)	2867
RGM_ConfigType (Rgm configure type)	2869
RTC_Compensation	2871
RTC_InitType (Rtc initialization type)	2872
RTC_TimeDate	2874
SCG_ClockCtrlType (Current system clock definition, include register SCG_CCR)	2875
SCG_FircType (FIRC initial definition, include register SCG_FIRCCSR/ SCG_FIRCDIV/ SCG_FIRCFG)	2877
SCG_FoscType (FOSC initial definition, include register SCG_FOSCCSR/SCG_FOSCDIV/SCG_FOSCCFG)	2879
SCG_InterruptType (The configuration option for the SCG interrupt)	2881
SCG_PllType (PLL definition, include register SCG_PLLCSR/ SCG_PLLDIV/SCG_PLL0CFG)	2883
SCG_Sirc32kType	2885
SCG_SircType (SIRC initial definition, include register SCG_SIRCCSR/ SCG_SIRCDIV/ SCG_SIRCTCCFG)	2886
SCG_SoscType (SOSC definition, include register SCG_SOSCCFG)	2888
SDADC_ChannelInitType (SDADC Channel Initialization Structure)	2890
SDADC_ChannelRegType	2894
SDADC_DetectCfgType	2896
SDADC_FilterChainCfgType	2900
SDADC_InitType (SDADC Initialization Structure)	2903

SDADC_IntegrationCfgType	2906
SDADC_IntInitType	2908
SDADC_ModulatorCfgType	2910
SDADC_PWMInitType	2912
SDADC_RectCfgType	2913
SDDF_AuxFilterConfigType (The configuration for auxiliary filter)	2915
SDDF_ChannelConfigType (The configuration of the SDDF channel)	2916
SDDF_ChannelDMAConfigType (The DMA configuration of the SDDF channel) ...	2918
SDDF_ChannelInterruptConfigType (The result related interrupt configuraion of the SDDF channel)	2920
SDDF_ChannelNotificationType	2922
SDDF_ClockAbsenceDetectConfigType (The configuration of the clock absence detect)	2924
SDDF_ClockOutConfigType (The clock out configuration for SDDF)	2926
SDDF_InitConfigType (The init configuration for SDDF instance)	2928
SDDF_LimitCheckConfigType (The configuration of the limit detect)	2929
SDDF_MainFilterConfigType (The configuration for main filter)	2931
SDDF_ShortCircuitDetectConfigType (The configuration of the short circuit detect)	2933
SDDF_ZeroCrossDetectConfigType (The configuration of the zero cross detect)	2935
SEC_BootInfo (Indicate the boot information)	2936
SEC_IspInfo (Indicate the ISP information)	2938
Sent_ChannelConfigType (Structure to configure the SENT channel)	2940
Sent_ChannelInterruptType (Structure to enable or disable the channel interrupts)	2943
Sent_ConfigType (Structure to configure the SENT)	2945
Sent_DMAConfigType (Structure to configure the SENT channel DMA)	2947
Sent_FastMessageDataType (Structure for fast message buffer)	2948
Sent_SlowMessageDataType (Structure for slow message buffer)	2949
Sent_SpcConfigType (Structure to configure the SENT channel SPC)	2951
STCU_ConfigType	2953
STCU_InitRamConfigType	2955
TMU_InitType (Defines the temperature sensor configuration)	2956
TSTMP_InitType (TSTMP initialization type)	2958
TSTMP_IntType (TSTMP interrupt type)	2959
TSTMP_UpdateType (TSTMP update type)	2960

WD OG_CfgType (WD OG init module type, include register setting for WD OG_CS/WD OG_TIMEOUT/WD OG_WINDOW)	2961
--	------

File Index

File List

Here is a list of all files with brief descriptions:

HwA_adc.h (Hardware access layer for ADC)	2963
HwA_aontimer.h (Aontimer hardware access layer)	2974
HwA_cm7.h (Cortex m4 hardware access layer)	2978
HwA_cmp.h (CMP driver type definition and API)	2980
HwA_cmu.h (CMU Module Register API)	2988
HwA_cordic.h (CORDIC hardware access layer)	2991
HwA_cpm.h (CPM register API)	2994
HwA_crc.h (CRC hardware access layer)	2997
HwA_csc.h (Hardware access layer for CSC)	3000
HwA_dma.h (Hardware access layer for DMA)	3008
HwA_dmamux.h (Hardware access layer for DMAMUX)	3021
HwA_eftu_ccm.h (Hardware access layer for EFTU CCM)	3023
HwA_eftu_cmu.h (Hardware access layer for EFTU CMU)	3024
HwA_eftu_dtm.h (Hardware access layer for EFTU DTM)	3025
HwA_eftu_tbu.h (Hardware access layer for EFTU TBU)	3026
HwA_eftu_tim.h (Hardware access layer for EFTU TIM)	3027
HwA_eftu_tom.h (Hardware access layer for EFTU TOM)	3028
HwA_eim.h (EIM Module Register API)	3029
HwA_elu.h (EsLU hardware access layer)	3031
HwA_enet.h (ENET hardware access layer)	3032
HwA_erm.h (ERM Module Register API)	3143
HwA_fciic.h (FCIIC hardware access layer)	3145
HwA_fcpi.h (FCPIT hardware access layer)	3154
HwA_fcsmu.h (FCSMU module register API)	3159
HwA_fcspi.h	3165
HwA_fcuart.h (FCUart register API)	3177
HwA_flexcan.h (FLEXCAN hardware access layer)	3185
HwA_freqm.h (FREQM Module Register API)	3207
HwA_ftu.h (FTU hardware access layer)	3209

HwA_gpio.h (GPIO hardware access layer)	3226
HwA_hrpwm.h (HRPWM hardware access layer)	3228
HwA_hsadc.h (Hardware access layer for HSADC)	3229
HwA_intm.h (Interrupt monitor)	3245
HwA_ism.h (ISM hardware access layer)	3248
HwA_ldi.h (LDI hardware access layer)	3254
HwA_lu.h (LU hardware access layer)	3255
HwA_mam.h (Hardware access layer for MAM)	3258
HwA_mb.h (Mailbox hardware access layer)	3260
HwA_msc.h (MSC hardware access layer)	3265
HwA_ospi.h (Ospi hardware access layer)	3272
HwA_overlay.h (FC4xxx Overlay hardware access layer)	3280
HwA_pmc.h (PMC register API)	3284
HwA_port.h (PORT hardware access layer)	3286
HwA_ptimer.h (Hardware access layer for PTIMER)	3289
HwA_qdt.h (Hardware access layer for QDT)	3296
HwA_rgm.h (RGM hardware access layer)	3297
HwA_rtc.h (Rtc hardware access layer)	3302
HwA_scg.h (SCG hardware access layer)	3306
HwA_scm.h (Hardware access layer for SCM)	3319
HwA_sdadc.h (Hardware access layer for SDADC)	3321
HwA_sddf.h (Hardware access layer for SDDF)	3369
HwA_sec.h (Sec hardware access layer)	3391
HwA_sent.h (Hardware access layer for SENT)	3396
HwA_smc.h (SMC hardware access layer)	3410
HwA_ssi.h (Ssi hardware access layer)	3412
HwA_stcu.h (Safety Test and Control Unit)	3413
HwA_tmu.h (Hardware access layer for TMU)	3421
HwA_tpue.h (FC7xxx TPUE hardware access layer)	3426
HwA_tpuh.h (FC7xxx TPUH hardware access layer)	3427
HwA_trgsel.h (Hardware access layer for TrgSel)	3428
HwA_tstmp.h (TSTMP hardware access layer)	3430
HwA_wdog.h (Wdog hardware access layer)	3432
HwA_wku.h (WKU hardware access layer)	3434

module_driver_adc.h (ADC driver type definition and API)	3436
module_driver_aontimer.h (Aontimer driver type definition and API)	3441
module_driver_cmp.h (CMP driver type definition and API)	3444
module_driver_cmu.h (CMU Module driver type definition and API)	3448
module_driver_cordic.h (Cordic driver type definition and API)	3452
module_driver_cpm.h (CPM driver type definition and API)	3455
module_driver_crc.h (CRC driver type definition and API)	3457
module_driver_csc.h (Csc driver type definition and API)	3460
module_driver_dma.h (DMA driver type definition and API)	3462
module_driver_dsp.h (DSP driver type definition and API)	3467
module_driver_eftu.h (EFTU driver type definition and API)	3469
module_driver_eftu_ccm.h (EFTU ccm driver type definition and API)	3470
module_driver_eftu_cmu.h (EFTU cmu driver type definition and API)	3471
module_driver_eftu_dtm.h (EFTU dtm driver type definition and API)	3472
module_driver_eftu_tbu.h (EFTU tbu driver type definition and API)	3473
module_driver_eftu_tim.h (EFTU tim driver type definition and API)	3474
module_driver_eftu_tom.h (EFTU tom driver type definition and API)	3475
module_driver_eim.h (FC7xxx EIM driver type definition and API)	3476
module_driver_elu.h	3478
module_driver_enet.h (ENET driver type definition and API)	3479
module_driver_erm.h (ERM Module driver type definition and API)	3500
module_driver_fciic.h (FCIIC driver type definition and API)	3502
module_driver_fcpiit.h (FCPIT driver type definition and API)	3514
module_driver_fcsmu.h (FCSMU module driver type definition and API)	3518
module_driver_fcspi.h (ADC driver type definition and API)	3524
module_driver_fcuart.h (FCUart driver type definition and API)	3528
module_driver_flash.h (Flash driver type definition and API)	3534
module_driver_flexcan.h (FLEXCAN driver type definition and API)	3539
module_driver_freqm.h (FREQM module driver type definition and API)	3544
module_driver_ftu.h (FTU driver type definition and API)	3546
module_driver_gpio.h (GPIO driver type definition and API)	3556
module_driver_hrpwm.h (HRPWM driver type definition and API)	3559
module_driver_hsadc.h (HSADC driver type definition and API)	3560
module_driver_hsm.h (HSM driver type definition and API)	3565

module_driver_hsm_level1_leveln.h (HSM driver type definition and API)	3622
module_driver_intm.h (INTM driver type definition and API)	3642
module_driver_ism.h (ISM driver type definition and API)	3645
module_driver_ldi.h (LDI driver type definition and API)	3649
module_driver_lin.h (LIN driver type definition and API)	3650
module_driver_lu.h (LU driver type definition and API)	3655
module_driver_mam.h (MAM driver type definition and API)	3658
module_driver_mb.h (Mailbox driver type definition and API)	3660
module_driver_msc.h (MSC driver type definition and API)	3663
module_driver_ospi.h (OSPI driver type definition and API)	3668
module_driver_overlay.h (FC7xxx overlay driver type definition and API)	3672
module_driver_pcc.h (PCC driver type definition and API)	3674
module_driver_pmc.h (FC7xxx PMC driver type definition and API)	3677
module_driver_port.h (PORT driver type definition and API)	3680
module_driver_ptimer.h (PTIMER driver type definition and API)	3685
module_driver_qdt.h (FC7xxx QDT driver type definition and API)	3689
module_driver_rgm.h (RGM driver type definition and API)	3690
module_driver_rtc.h (Rtc driver type definition and API)	3693
module_driver_scg.h (SCG driver type definition and API)	3697
module_driver_scm.h (SCM driver type definition and API)	3704
module_driver_scst.h (Scst driver type definition and API)	3706
module_driver_sdadc.h (SDADC driver source code)	3708
module_driver_sddf.h (Sigma Delta Digital Filter (SDDF) driver type definition and API)	3718
module_driver_sec.h (Sec driver type definition and API)	3723
module_driver_sema.h	3728
module_driver_sent.h (SENT driver type definition and API)	3729
module_driver_smc.h (SMC driver type definition and API)	3736
module_driver_ssi.h (Ssi driver type definition and API)	3738
module_driver_stcu.h	3739
module_driver_tmu.h (TMU driver type definition and API)	3748
module_driver_tpu.h	3750
module_driver_trgsel.h (TRGSEL driver type definition and API)	3751
module_driver_tstmp.h (TSTMP driver type definition and API)	3754
module_driver_wdog.h (WDOG driver type definition and API)	3757

module_driver_wku.h (WKU driver type definition and API)	3760
--	-------------

Module Documentation

HwA_adc

Collaboration diagram for HwA_adc:Enumerations

```
enum ADC_ResolutionType { ADC_RESOLUTION_12_BIT = 0U, ADC_RESOLUTION_10_BIT = 1U,  
ADC_RESOLUTION_8_BIT = 2U }
```

Select the ADC Resolution.

```
enum ADC_AlignType { ADC_ALIGN_RIGHT = 0U, ADC_ALIGN_LEFT = 1U }
```

Select the ADC result alignment.

```
enum ADC_TriggerModeType { ADC_TRGMODE_SW = 0U, ADC_TRGMODE_RISING_EDGE = 1U,  
ADC_TRGMODE_FALLING_EDGE = 2U, ADC_TRGMODE_BOTH_EDGE = 3U,  
ADC_TRGMODE_HIGH_VOLTAGE = 4U, ADC_TRGMODE_LOW_VOLTAGE = 5U }
```

Select the ADC trigger mode.

```
enum ADC_SequenceModeType { ADC_SEQMODE_SINGLE = 0U, ADC_SEQMODE_CONTINUOUS = 1U,  
ADC_SEQMODE_DISCONTINUOUS_0 = 2U, ADC_SEQMODE_DISCONTINUOUS_1 = 3U }
```

Select the ADC sequence mode.

```
enum ADC_OvrModeType { ADC_OVERRUN_MODE_PRESERVE = 0U, ADC_OVERRUN_MODE_OVERWRITE  
= 1U }
```

Select the ADC overrun management mode.

```
enum ADC_RefType { ADC_REF_INTERNAL = 0U, ADC_REF_EXTERNAL = 1U }
```

Select the ADC voltage reference source.

```
enum ADC_AverageLenType { ADC_AVERAGE_LEN_4 = 0U, ADC_AVERAGE_LEN_8 = 1U,  
ADC_AVERAGE_LEN_16 = 2U, ADC_AVERAGE_LEN_32 = 3U }
```

Select the ADC hardware average samples.

```
enum ADC_ClockDivideType { ADC_CLOCK_DIV_1 = 0U, ADC_CLOCK_DIV_2 = 1U, ADC_CLOCK_DIV_4 =  
2U, ADC_CLOCK_DIV_8 = 3U }
```

Set the ADC clock divider.

```
enum ADC_TriggerSrcType { ADC_TRGSRC_PTIMER = 0U, ADC_TRGSRC_TRGSEL = 2U }
```

The trigger source of the ADC instance.

```
enum ADC_CmpChannelType { ADC_CMP_CHANNEL_ALL = 0U, ADC_CMP_CHANNEL_SINGLE = 1U }
```

Select the channel compare mode.

Functions

LOCAL_INLINE bool **ADC_HWA_GetCompareFlag** (const ADC_Type *const pAdc)

Check whether the conversion result is in the comparing range.

LOCAL_INLINE void **ADC_HWA_ClearCompareFlag** (ADC_Type *const pAdc)

Clear the compare flag.

LOCAL_INLINE bool **ADC_HWA_GetOverRunFlag** (const ADC_Type *const pAdc)

Get the overrun status of the ADC instance.

LOCAL_INLINE void **ADC_HWA_ClearOverRunFlag** (ADC_Type *const pAdc)

Clear the overrun flag of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSequenceFlag** (const ADC_Type *const pAdc)

Check whether the ADC conversion sequence is finished.

LOCAL_INLINE void **ADC_HWA_ClearEndOfSequenceFlag** (ADC_Type *const pAdc)

Clear the ADC conversion sequence complete flag.

LOCAL_INLINE bool **ADC_HWA_GetEndOfConversionFlag** (const ADC_Type *const pAdc)

Check whether the current ADC conversion is finished.

LOCAL_INLINE void **ADC_HWA_ClearEndOfConversionFlag** (ADC_Type *const pAdc)

Clear the ADC single conversion complete flag.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSampleFlag** (const ADC_Type *const pAdc)

Check whether the sampling phase of the current ADC conversion is finished.

LOCAL_INLINE void **ADC_HWA_ClearEndOfSampleFlag** (ADC_Type *const pAdc)

Clear the ADC sampling complete flag.

LOCAL_INLINE bool **ADC_HWA_GetFIFOReadyFlag** (const ADC_Type *const pAdc)

Check whether the data quantity in the FIFO is greater than watermark.

LOCAL_INLINE bool **ADC_HWA_GetFIFOFullFlag** (const ADC_Type *const pAdc)

Check whether the ADC FIFO is full.

LOCAL_INLINE bool **ADC_HWA_GetFIFOEmptyFlag** (const ADC_Type *const pAdc)

Check whether the ADC FIFO is empty.

LOCAL_INLINE bool **ADC_HWA_GetReady** (const ADC_Type *const pAdc)

Check whether the ADC instance is ready to operate.

LOCAL_INLINE void **ADC_HWA_ClearReady** (ADC_Type *const pAdc)

Clear the ready flag of the ADC instance.

LOCAL_INLINE void **ADC_HWA_ClearFlags** (ADC_Type *const pAdc, uint32_t u32Flags)

Clear the ready flag of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetFIFOReadyInterruptFlag** (const ADC_Type *const pAdc)

Get the FIFO Ready interrupt flag If enabled, ADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE void **ADC_HWA_SetFIFOReadyInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the FIFO Ready interrupt flag If enabled, ADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE bool **ADC_HWA_GetCompareInterruptFlag** (const ADC_Type *const pAdc)

Get the Compare interrupt flag If enabled, ADC interrupt is generated when the ADC conversion result is not within the compare threshold.

LOCAL_INLINE void **ADC_HWA_SetCompareInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the Compare interrupt flag If enabled, ADC interrupt is generated when the ADC conversion result is not within the compare threshold.

LOCAL_INLINE bool **ADC_HWA_GetOverrunInterruptFlag** (const ADC_Type *const pAdc)

Get the Overrun interrupt flag If enabled, ADC interrupt is generated when the ADC instance is overrun.

LOCAL_INLINE void **ADC_HWA_SetOverrunInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the Overrun interrupt flag If enabled, ADC interrupt is generated when the ADC instance is overrun.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSequenceInterruptFlag** (const ADC_Type *const pAdc)

Get the End of Sequence interrupt enable flag If enabled, ADC interrupt is generated when the ADC sequence conversion is completed.

LOCAL_INLINE void **ADC_HWA_SetEndOfSequenceInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the End of Sequence interrupt enable flag If enabled, ADC interrupt is generated when the ADC sequence conversion is completed.

LOCAL_INLINE bool **ADC_HWA_GetEndOfConversionInterruptFlag** (const ADC_Type *const pAdc)

Get the conversion complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion is completed.

LOCAL_INLINE void **ADC_HWA_SetEndOfConversionInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the conversion complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion is completed.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSampleInterruptFlag** (const ADC_Type *const pAdc)

Get the sample complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion finished the sampling phase.

LOCAL_INLINE void **ADC_HWA_SetEndOfSampleInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the sample complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion finished the sampling phase.

LOCAL_INLINE bool **ADC_HWA_GetADCReadyInterruptFlag** (const ADC_Type *const pAdc)

Get the ADC ready interrupt enable flag If enabled, ADC interrupt is generated when the ADC module is ready for conversion.

LOCAL_INLINE void **ADC_HWA_SetADCReadyInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the ADC ready interrupt enable flag If enabled, ADC interrupt is generated when the ADC module is ready for conversion.

LOCAL_INLINE uint32_t **ADC_HWA_GetInterruptEnable** (const ADC_Type *const pAdc)

Get the interrupt enable config.

LOCAL_INLINE void **ADC_HWA_SetInterruptEnable** (ADC_Type *const pAdc, uint32_t u32InterruptCfg)

Set the interrupt enable.

LOCAL_INLINE void **ADC_HWA_Reset** (ADC_Type *const pAdc)

Reset the ADC hardware.

LOCAL_INLINE bool **ADC_HWA_GetEnable** (const ADC_Type *const pAdc)

Get the enable status of the ADC instance.

LOCAL_INLINE void **ADC_HWA_Enable** (ADC_Type *const pAdc)

Enable the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetDisable** (const ADC_Type *const pAdc)

Get whether the ADC instance is in disable status.

LOCAL_INLINE void **ADC_HWA_Disable** (ADC_Type *const pAdc)

Disable the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetStart** (const ADC_Type *const pAdc)

Get the conversion start status of the ADC instance.

LOCAL_INLINE void **ADC_HWA_Start** (ADC_Type *const pAdc)

Start the ADC conversion.

LOCAL_INLINE bool **ADC_HWA_GetStop** (const ADC_Type *const pAdc)

Get whether ADC is in stopping status.

LOCAL_INLINE void **ADC_HWA_Stop** (ADC_Type *const pAdc)

Stop the ADC conversion.

LOCAL_INLINE **ADC_OvrModeType** **ADC_HWA_GetOverrunManagementMode** (const ADC_Type *const pAdc)

Get the overrun management mode.

LOCAL_INLINE void **ADC_HWA_SetOverrunManagementMode** (ADC_Type *const pAdc, **ADC_OvrModeType** eOvrMode)

Set the overrun management mode.

LOCAL_INLINE uint8_t **ADC_HWA_GetSequenceLength** (const ADC_Type *const pAdc)

Get the sequence length of the ADC conversion sequence.

LOCAL_INLINE void **ADC_HWA_SetSequenceLength** (ADC_Type *const pAdc, uint8_t u8SequenceLength)

Set the sequence length of the ADC conversion sequence.

LOCAL_INLINE **ADC_SeqModeType** **ADC_HWA_GetSequenceMode** (const ADC_Type *const pAdc)

Get the ADC sequence mode.

LOCAL_INLINE void **ADC_HWA_SetSequenceMode** (ADC_Type *const pAdc, **ADC_SeqModeType** eSequenceMode)

Set the ADC sequence mode.

LOCAL_INLINE bool **ADC_HWA_GetAutoDisableModeEnableFlag** (const ADC_Type *const pAdc)

Get whether auto disable is enabled.

LOCAL_INLINE void **ADC_HWA_SetAutoDisableModeEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Set whether to enable auto disable mode.

LOCAL_INLINE bool **ADC_HWA_GetWaitConversionModeFlag** (const ADC_Type *const pAdc)

Get whether the wait conversion mode is enabled.

LOCAL_INLINE void **ADC_HWA_SetWaitConversionModeFlag** (ADC_Type *const pAdc, bool bEnable)

Set the wait conversion mode.

LOCAL_INLINE **ADC_TrgSrcType** **ADC_HWA_GetTriggerSource** (const ADC_Type *const pAdc)

Get the trigger source the the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetTriggerSource** (ADC_Type *const pAdc, **ADC_TrgSrcType** eTrgSource)

Set the trigger source the the ADC instance.

LOCAL_INLINE **ADC_TrgModeType** **ADC_HWA_GetTriggerMode** (const ADC_Type *const pAdc)

Get the trigger mode of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetTriggerMode** (ADC_Type *const pAdc, **ADC_TrgModeType** eTrgMode)

Set the trigger mode of the ADC instance.

LOCAL_INLINE **ADC_AlignType** **ADC_HWA_GetDataAlignment** (const ADC_Type *const pAdc)

Get the data align mode.

LOCAL_INLINE void **ADC_HWA_SetDataAlignment** (ADC_Type *const pAdc, **ADC_AlignType** eAlign)

Set the data align mode.

LOCAL_INLINE **ADC_ResolutionType** **ADC_HWA_GetDataResolution** (const ADC_Type *const pAdc)

Get the ADC resolution of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetDataResolution** (ADC_Type *const pAdc, **ADC_ResolutionType** eResolution)

Set the resolution of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetDMAEnableFlag** (const ADC_Type *const pAdc)

Get whether DMA for the ADC instance is enabled.

LOCAL_INLINE void **ADC_HWA_SetDMAEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Enable or disable the DMA for the ADC instance.

LOCAL_INLINE uint32_t **ADC_HWA_GetConfig1** (const ADC_Type *const pAdc)

Get the ADC_CFG1 config.

LOCAL_INLINE void **ADC_HWA_SetConfig1** (ADC_Type *const pAdc, uint32_t u32Config)

Set the ADC_CFG1 config.

LOCAL_INLINE uint8_t **ADC_HWA_GetFIFOWaterMark** (const ADC_Type *const pAdc)

Get the FIFO water mark settings for the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetFIFOWaterMark** (ADC_Type *const pAdc, uint8_t u8WaterMark)

Set the FIFO water mark for the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetAverageEnableFlag** (const ADC_Type *const pAdc)

Get whether hardware average is enabled.

LOCAL_INLINE void **ADC_HWA_SetAverageEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Enable or disable hardware average for the ADC instance.

LOCAL_INLINE ADC_AverageLenType **ADC_HWA_GetAverageNumber** (const ADC_Type *const pAdc)

Get the hardware average number.

LOCAL_INLINE void **ADC_HWA_SetAverageNumber** (ADC_Type *const pAdc, ADC_AverageLenType eAverageNumber)

Set the hardware average number.

LOCAL_INLINE bool **ADC_HWA_GetClockGatingAck** (const ADC_Type *const pAdc)

Whether clock gating is acknowledged.

LOCAL_INLINE bool **ADC_HWA_GetClockGatingEnableFlag** (const ADC_Type *const pAdc)

Get whether clock gating is enabled.

LOCAL_INLINE void **ADC_HWA_SetClockGatingEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Set whether to enable ADC clock gating.

LOCAL_INLINE **ADC_RefType** **ADC_HWA_GetVoltageReference** (const ADC_Type *const pAdc)

Get the voltage reference of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetVoltageReference** (ADC_Type *const pAdc, **ADC_RefType** eRefSel)

Set the voltage reference of the ADC instance.

LOCAL_INLINE **ADC_ClockDivideType** **ADC_HWA_GetClockDivider** (const ADC_Type *const pAdc)

Get the ADC clock divider.

LOCAL_INLINE void **ADC_HWA_SetClockDivider** (ADC_Type *const pAdc, **ADC_ClockDivideType** eDivider)

Set the ADC clock divider.

LOCAL_INLINE uint8_t **ADC_HWA_GetStartupCnt** (const ADC_Type *const pAdc)

Get the ADC start up count.

LOCAL_INLINE void **ADC_HWA_SetStartupCnt** (ADC_Type *const pAdc, uint8_t u8StartupCnt)

Set the ADC start up count.

LOCAL_INLINE uint32_t **ADC_HWA_GetConfig2** (const ADC_Type *const pAdc)

Get the ADC_CFG2 config.

LOCAL_INLINE void **ADC_HWA_SetConfig2** (ADC_Type *const pAdc, uint32_t u32Config)

Set the ADC_CFG2 config.

LOCAL_INLINE uint8_t **ADC_HWA_GetSampleTime** (ADC_Type *const pAdc, uint8_t u8Selection)

Get the sample time of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetSampleTime** (ADC_Type *const pAdc, uint8_t u8Selection, uint8_t u8SampleTime)

Set the sample time of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetHwCompareChannel** (ADC_Type *const pAdc, **ADC_CmpChannelType** eChannelType, uint8_t u8ChannelNum)

Set the ADC hardware compare channel.

LOCAL_INLINE bool **ADC_HWA_GetHwCompareEnableFlag** (const ADC_Type *const pAdc)

Get whether hardware compare is enabled.

LOCAL_INLINE void **ADC_HWA_SetHwCompareEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Set to enable or disable the hardware compare.

LOCAL_INLINE void **ADC_HWA_SetHwCompareThreshold** (ADC_Type *const pAdc, uint16_t u16LowThres, uint16_t u16HighThres)

Set the ADC hardware compare threshold.

LOCAL_INLINE uint32_t **ADC_HWA_GetFIFOData** (const ADC_Type *const pAdc)

Get the conversion result FIFO data of the ADC instance.

LOCAL_INLINE uint8_t **ADC_HWA_GetChannelSampleTimeIndex** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Get the sample time index of the ADC channel.

LOCAL_INLINE void **ADC_HWA_SetChannelSampleTimeIndex** (ADC_Type *const pAdc, const uint8_t u8ChnIdx, uint8_t u8SmpSel)

Set the sample time index of the ADC channel.

LOCAL_INLINE bool **ADC_HWA_GetChannelConvertComplete** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Get the channel conversion complete status of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetChannelInterruptEnable** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Check whether interrupt is enabled on the selected ADC channel.

LOCAL_INLINE void **ADC_HWA_SetChannelInterruptEnable** (ADC_Type *const pAdc, const uint8_t u8ChnIdx, bool bEnable)

Set to enable or disable interrupt on the selected ADC channel.

LOCAL_INLINE uint8_t **ADC_HWA_GetChannelInput** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)
Get the input channel of the selected ADC channel.

LOCAL_INLINE void **ADC_HWA_SetChannelInput** (ADC_Type *const pAdc, const uint8_t u8ChnIdx, uint8_t u8InputChann)
Set the input channel to the selected ADC channel.

LOCAL_INLINE uint32_t **ADC_HWA_GetChannelData** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)
Get the conversion result data of the ADC instance.

Detailed Description

Enumeration Type Documentation

enum ADC_AlignType

Select the ADC result alignment.

Enumerator:

ADC_ALIGN_RIG HT	ADC result is aligned in right
ADC_ALIGN_LEF T	ADC result is aligned in left

enum ADC_AverageLenType

Select the ADC hardware average samples.

Enumerator:

ADC_AVERAGE_ LEN_4	result average by 4 samples
ADC_AVERAGE_ LEN_8	result average by 8 samples
ADC_AVERAGE_ LEN_16	result average by 16 samples
ADC_AVERAGE_ LEN_32	result average by 32 samples

enum ADC_ClockDivideType

Set the ADC clock divider.

Note

ADC clock divider is not available in FC7300F512K

Enumerator:

ADC_CLOCK_DI V_1	
ADC_CLOCK_DI V_2	
ADC_CLOCK_DI V_4	
ADC_CLOCK_DI V_8	

enum ADC_CmpChannelType

Select the channel compare mode.

Select whether the channel compare is enabled on all channels or on the single selected channel

Enumerator:

ADC_CMP_CHA NNEL_ALL	Compare enabled in all channels
ADC_CMP_CHA NNEL_SINGLE	Compare enabled in the specified channel

enum ADC_OvrModeType

Select the ADC overrun management mode.

To select whether the old data are preserved or overwritten by the new data when ADC is overrun
(The FIFO is full when new conversion result comes)

Enumerator:

ADC_OVERRUN_ MODE_PRESERV E	old data are preserved when ADC is overrun
ADC_OVERRUN_ MODE_OVERWR ITE	old data are overwritten when ADC is overrun

enum ADC_RefType

Select the ADC voltage reference source.

Enumerator:

ADC_REF_INTER NAL	select the internal voltage reference
ADC_REF_EXTE RNAL	select the external voltage reference

enum ADC_ResolutionType

Select the ADC Resolution.

Enumerator:

ADC_RESOLUTION_12_BIT	12 bit resolution
ADC_RESOLUTION_10_BIT	10 bit resolution
ADC_RESOLUTION_8_BIT	8 bit resolution

enum ADC_SeqModeType

Select the ADC sequence mode.

Enumerator:

ADC_SEQMODE_SINGLE	ADC single sequence mode
ADC_SEQMODE_CONTINUOUS	ADC continuous mode
ADC_SEQMODE_DISCONTINUOUS_0	ADC discontinuous 0 mode
ADC_SEQMODE_DISCONTINUOUS_1	ADC discontinuous 1 mode

enum ADC_TrgModeType

Select the ADC trigger mode.

Note

This option is only valid in ADC single sequence mode and ADC continuous mode. In ADC discontinuous mode, the ADC trigger mode is fixed as ADC_TRIGMODE_RISING_EDGE

Enumerator:

ADC_TRGMODE _SW	ADC is triggered by software
ADC_TRGMODE _RISING_EDGE	ADC is triggered by hardware trigger on rising edge
ADC_TRGMODE _FALLING_EDGE	ADC is triggered by hardware trigger on falling edge
ADC_TRGMODE _BOTH_EDGE	ADC is triggered by hardware trigger on both edges
ADC_TRGMODE _HIGH_VOLTAGE	ADC is triggered when hardware trigger is high voltage
ADC_TRGMODE _LOW_VOLTAGE	ADC is triggered when hardware trigger is low voltage

enum ADC_TrgSrcType

The trigger source of the ADC instance.

Note

In ADC discontinuous 1 mode, the trigger source is from Ptimer In ADC single and continuous mode, if hardware trigger is enabled, the trigger source is from TRGSEL

Enumerator:

ADC_TRGSRP TIMER	Trigger source from PTIMER/TRGSEL with Ptimer
ADC_TRGSRG SEL	Trigger source from TRGSEL

Function Documentation

LOCAL_INLINE void ADC_HWA_ClearCompareFlag (ADC_Type *const *pAdc*)

Clear the compare flag.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_ClearEndOfConversionFlag (ADC_Type *const *pAdc*)

Clear the ADC single conversion complete flag.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_ClearEndOfSampleFlag (ADC_Type *const *pAdc*)

Clear the ADC sampling complete flag.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_ClearEndOfSequenceFlag (ADC_Type *const *pAdc*)

Clear the ADC conversion sequence complete flag.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_ClearFlags (ADC_Type *const *pAdc*, uint32_t *u32Flags*)

Clear the ready flag of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_ClearOverRunFlag (ADC_Type *const *pAdc*)

Clear the overrun flag of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_ClearReady (ADC_Type *const *pAdc*)

Clear the ready flag of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_Disable (ADC_Type *const *pAdc*)

Disable the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_Enable (ADC_Type *const *pAdc*)

Enable the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE bool ADC_HWA_GetADCReadyInterruptFlag (const ADC_Type *const *pAdc*)

Get the ADC ready interrupt enable flag If enabled, ADC interrupt is generated when the ADC module is ready for conversion.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC ready interrupt is enabled

false the ADC ready interrupt is disabled

LOCAL_INLINE bool ADC_HWA_GetAutoDisableModeEnableFlag (const ADC_Type *const *pAdc*)

Get whether auto disable is enabled.

Note

Auto disable mode is only available in FC7300F2M

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true auto disable mode is enabled

false auto disable mode is disabled

LOCAL_INLINE bool ADC_HWA_GetAverageEnableFlag (const ADC_Type *const *pAdc*)

Get whether hardware average is enabled.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true hardware average is enabled

false hardware average is disabled

LOCAL_INLINE ADC_AverageLenType ADC_HWA_GetAverageNumber (const ADC_Type *const *pAdc*)

Get the hardware average number.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_AverageType the hardware average number

LOCAL_INLINE bool ADC_HWA_GetChannelConvertComplete (const ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*)

Get the channel conversion complete status of the ADC instance.

Note

this function is used only in ADC discontinuous mode to get the channel complete status

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

true the ADC conversion on the selected channel is completed

false the ADC conversion on the selected channel is not completed

LOCAL_INLINE uint32_t ADC_HWA_GetChannelData (const ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*)

Get the conversion result data of the ADC instance.

Note

only result data of ADC discontinuous mode will be stored in RESULTn register.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the ADC channel

Returns

uint32_t the ADC conversion result

LOCAL_INLINE uint8_t ADC_HWA_GetChannelInput (const ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*)

Get the input channel of the selected ADC channel.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

uint8_t the hardware input channel

LOCAL_INLINE bool ADC_HWA_GetChannelInterruptEnable (const ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*)

Check whether interrupt is enabled on the selected ADC channel.

Note

this function is used only in ADC discontinuous mode to get the channel interrupt settings

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

true interrupt is enabled on the selected channel

false interrupt is disabled on the selected channel

LOCAL_INLINE uint8_t ADC_HWA_GetChannelSampleTimeIndex (const ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*)

Get the sample time index of the ADC channel.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

uint8_t the sample time index of the ADC channel

LOCAL_INLINE ADC_ClockDivideType ADC_HWA_GetClockDivider (const ADC_Type *const *pAdc*)

Get the ADC clock divider.

Note

This feature is only available in FC7300F2M

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_ClockDivideType the ADC clock divider

LOCAL_INLINE bool ADC_HWA_GetClockGatingAck (const ADC_Type *const *pAdc*)

Whether clock gating is acknowledged.

Note

This feature is only available in FC7300F2M

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true ADC clock source is off after setting Clock Gating Enable Flag

false ADC clock source is on after clearing Clock Gating Enable Flag

LOCAL_INLINE bool ADC_HWA_GetClockGatingEnableFlag (const ADC_Type *const *pAdc*)

Get whether clock gating is enabled.

Note

This feature is only available in FC7300F2M

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true Clock gating is enabled, ADC clock is off

false Clock gating is disabled, ADC clock is on

LOCAL_INLINE bool ADC_HWA_GetCompareFlag (const ADC_Type *const *pAdc*)

Check whether the conversion result is in the comparing range.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the conversion result is in the comparing range

false the conversion result is not in the comparing range

LOCAL_INLINE bool ADC_HWA_GetCompareInterruptFlag (const ADC_Type *const *pAdc*)

Get the Compare interrupt flag If enabled, ADC interrupt is generated when the ADC conversion result is not within the compare threshold.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true ADC Compare interrupt is enabled

false ADC Compare interrupt is disabled

LOCAL_INLINE uint32_t ADC_HWA_GetConfig1 (const ADC_Type *const *pAdc*)

Get the ADC_CFG1 config.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

uint32_t the ADC_CFG1 config

LOCAL_INLINE uint32_t ADC_HWA_GetConfig2 (const ADC_Type *const *pAdc*)

Get the ADC_CFG2 config.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

uint32_t the ADC_CFG2 config

LOCAL_INLINE ADC_AlignType ADC_HWA_GetDataAlignment (const ADC_Type *const *pAdc*)

Get the data align mode.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_ALIGN_RIGHT the conversion data is aligned right

ADC_ALIGN_LEFT the conversion is aligned left

LOCAL_INLINE ADC_ResolutionType ADC_HWA_GetDataResolution (const ADC_Type *const *pAdc*)

Get the ADC resolution of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_ResolutionType the resolution of the ADC instance

LOCAL_INLINE bool ADC_HWA_GetDisable (const ADC_Type *const *pAdc*)

Get whether the ADC instance is in disable status.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC instance is in disable status

false the ADC instance is not in disable status

LOCAL_INLINE bool ADC_HWA_GetDMAEnableFlag (const ADC_Type *const *pAdc*)

Get whether DMA for the ADC instance is enabled.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true DMA is enabled for the ADC instance

false DMA is disabled for the ADC instance

LOCAL_INLINE bool ADC_HWA_GetEnable (const ADC_Type *const *pAdc*)

Get the enable status of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC instance is enabled

false the ADC instance has not been enabled

LOCAL_INLINE bool ADC_HWA_GetEndOfConversionFlag (const ADC_Type *const *pAdc*)

Check whether the current ADC conversion is finished.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the current ADC conversion is finished

false the current ADC conversion is unfinished

LOCAL_INLINE bool ADC_HWA_GetEndOfSampleFlag (const ADC_Type *const *pAdc*)

Check whether the sampling phase of the current ADC conversion is finished.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the sampling phase of the ADC conversion is finished

false the sampling phase of the ADC conversion is unfinished

LOCAL_INLINE bool ADC_HWA_GetEndOfSampleInterruptFlag (const ADC_Type *const *pAdc*)

Get the sample complete interrupt enable flag. If enabled, ADC interrupt is generated when each ADC conversion finished the sampling phase.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the sample complete interrupt is enabled

false the sample complete interrupt is disabled

LOCAL_INLINE bool ADC_HWA_GetEndOfSequenceFlag (const ADC_Type *const *pAdc*)

Check whether the ADC conversion sequence is finished.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC conversion sequence is finished

false the ADC conversion sequence is unfinished

LOCAL_INLINE bool ADC_HWA_GetEndOfSequenceInterruptFlag (const ADC_Type *const *pAdc*)

Get the End of Sequence interrupt enable flag. If enabled, ADC interrupt is generated when the ADC sequence conversion is completed.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true ADC End of Sequence interrupt is enabled

false ADC End of Sequence interrupt is disabled

LOCAL_INLINE uint32_t ADC_HWA_GetFIFOData (const ADC_Type *const *pAdc*)

Get the conversion result FIFO data of the ADC instance.

Note

only result data of ADC single mode and continuous mode will be stored in FIFO register.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

uint32_t the ADC conversion result

LOCAL_INLINE bool ADC_HWA_GetFIFOEmptyFlag (const ADC_Type *const *pAdc*)

Check whether the ADC FIFO is empty.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC FIFO is empty

false the ADC FIFO is not empty

LOCAL_INLINE bool ADC_HWA_GetFIFOFullFlag (const ADC_Type *const *pAdc*)

Check whether the ADC FIFO is full.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC FIFO is full

false the ADC FIFO is not full

LOCAL_INLINE bool ADC_HWA_GetFIFOReadyFlag (const ADC_Type *const *pAdc*)

Check whether the data quantity in the FIFO is greater than watermark.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC FIFO is ready

false the ADC FIFO is unready

LOCAL_INLINE bool ADC_HWA_GetFIFOReadyInterruptFlag (const ADC_Type *const *pAdc*)

Get the FIFO Ready interrupt flag If enabled, ADC interrupt is generated when the FIFO water mark is greater than FWMARK.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true ADC FIFO Ready interrupt is enabled

false ADC FIFO Ready interrupt is disabled

LOCAL_INLINE uint8_t ADC_HWA_GetFIFOWaterMark (const ADC_Type *const *pAdc*)

Get the FIFO water mark settings for the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

uint8_t the ADC FIFO water mark setting

LOCAL_INLINE bool ADC_HWA_GetHwCompareEnableFlag (const ADC_Type *const *pAdc*)

Get whether hardware compare is enabled.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true hardware compare is enabled

false hardware compare is disabled

LOCAL_INLINE uint32_t ADC_HWA_GetInterruptEnable (const ADC_Type *const *pAdc*)

Get the interrupt enable config.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

uint32_t the interrupt enable config

LOCAL_INLINE bool ADC_HWA_GetOverRunFlag (const ADC_Type *const *pAdc*)

Get the overrun status of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC is in overrun status

false the ADC is not overrun

LOCAL_INLINE bool ADC_HWA_GetOverrunInterruptFlag (const ADC_Type *const *pAdc*)

Get the Overrun interrupt flag If enabled, ADC interrupt is generated when the ADC instance is overrun.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true ADC Overrun interrupt is enabled

false ADC Overrun interrupt is disabled

LOCAL_INLINE ADC_OvrModeType ADC_HWA_GetOverrunManagementMode (const ADC_Type *const *pAdc*)

Get the overrun management mode.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_OVERRUN_MODE_PRESERVE the conversion data is preserved when ADC is overrun

ADC_OVERRUN_MODE_OVERWRITE the conversion data is overwritten when ADC is overrun

LOCAL_INLINE bool ADC_HWA_GetReady (const ADC_Type *const *pAdc*)

Check whether the ADC instance is ready to operate.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC instance is ready for a new conversion

false the ADC instance is unready

LOCAL_INLINE uint8_t ADC_HWA_GetSampleTime (ADC_Type *const *pAdc*, uint8_t *u8Selection*)

Get the sample time of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8Selection</i>	the sample time index, range 0~3

Returns

uint8_t the sample time of the selected index

LOCAL_INLINE uint8_t ADC_HWA_GetSequenceLength (const ADC_Type *const *pAdc*)

Get the sequence length of the ADC conversion sequence.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

uint8_t the sequence length of the ADC conversion sequence

LOCAL_INLINE ADC_SeqModeType ADC_HWA_GetSequenceMode (const ADC_Type *const *pAdc*)

Get the ADC sequence mode.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_SeqModeType the sequence mode the the ADC instance

LOCAL_INLINE bool ADC_HWA_GetStart (const ADC_Type *const *pAdc*)

Get the conversion start status of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the conversion of the ADC instance is started

false the conversion of the ADC instance has not been started

LOCAL_INLINE uint8_t ADC_HWA_GetStartupCnt (const ADC_Type *const *pAdc*)

Get the ADC start up count.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

uint8_t the start count of the ADC instance

LOCAL_INLINE bool ADC_HWA_GetStop (const ADC_Type *const *pAdc*)

Get whether ADC is in stopping status.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC instance is stopping

false the ADC instance is not in stopping status

LOCAL_INLINE ADC_TriggerModeType ADC_HWA_GetTriggerMode (const ADC_Type *const *pAdc*)

Get the trigger mode of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_TrigModeType the trigger mode if the ADC instance

LOCAL_INLINE ADC_TrigSrcType ADC_HWA_GetTriggerSource (const ADC_Type *const *pAdc*)

Get the trigger source the the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_TrigSrcType the trigger source of the ADC instance

LOCAL_INLINE ADC_RefType ADC_HWA_GetVoltageReference (const ADC_Type *const *pAdc*)

Get the voltage reference of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

ADC_REF_INTERNAL the ADC instance uses internal voltage reference

ADC_REF_EXTERNAL the ADC instance uses external voltage reference

LOCAL_INLINE bool ADC_HWA_GetWaitConversionModeFlag (const ADC_Type *const *pAdc*)

Get whether the wait conversion mode is enabled.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the wait conversion mode is enabled

false the wait conversion mode is disabled

LOCAL_INLINE void ADC_HWA_Reset (ADC_Type *const *pAdc*)

Reset the ADC hardware.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_SetADCReadyInterruptFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the ADC ready interrupt enable flag If enabled, ADC interrupt is generated when the ADC module is ready for conversion.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	Whether to enable the ADC ready interrupt

LOCAL_INLINE void ADC_HWA_SetAutoDisableModeEnableFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set whether to enable auto disable mode.

Note

Auto disable mode is only available in FC7300F2M

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable auto disable mode

LOCAL_INLINE void ADC_HWA_SetAverageEnableFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Enable or disable hardware average for the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the ADC hardware average

LOCAL_INLINE void ADC_HWA_SetAverageNumber (ADC_Type *const *pAdc*, ADC_AverageLenType *eAverageNumber*)

Set the hardware average number.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eAverageNumber</i>	the hardware average number to set

LOCAL_INLINE void ADC_HWA_SetChannelInput (ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*, uint8_t *u8InputChann*)

Set the input channel to the selected ADC channel.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the channel
<i>u8InputChann</i>	the hardware input channel

LOCAL_INLINE void ADC_HWA_SetChannelInterruptEnable (ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*, bool *bEnable*)

Set to enable or disable interrupt on the selected ADC channel.

Note

this function is used only in ADC discontinuous mode to get the channel interrupt settings

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the channel
<i>bEnable</i>	whether to enable interrupt on the selected ADC channel

LOCAL_INLINE void ADC_HWA_SetChannelSampleTimeIndex (ADC_Type *const *pAdc*, const uint8_t *u8ChnIdx*, uint8_t *u8SmpSel*)

Set the sample time index of the ADC channel.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8ChnIdx</i>	the index of the channel
<i>u8SmpSel</i>	the sample time index of the ADC channel

LOCAL_INLINE void ADC_HWA_SetClockDivider (ADC_Type *const *pAdc*, ADC_ClockDivideType *eDivider*)

Set the ADC clock divider.

Note

Before configuring DIV, must set CG and wait for CG_ACK=1. After configuring DIV, must clear CG and wait for CG_ACK=0.

This feature is only available in FC7300F2M

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eDivider</i>	the ADC clock divider to set

LOCAL_INLINE void ADC_HWA_SetClockGatingEnableFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set whether to enable ADC clock gating.

Note

This feature is only available in FC7300F2M

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable ADC clock gating

LOCAL_INLINE void ADC_HWA_SetCompareInterruptFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the Compare interrupt flag If enabled, ADC interrupt is generated when the ADC conversion result is not within the compare threshold.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the ADC Compare interrupt

LOCAL_INLINE void ADC_HWA_SetConfig1 (ADC_Type *const *pAdc*, uint32_t *u32Config*)

Set the ADC_CFG1 config.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u32Config</i>	the ADC_CFG1 config

LOCAL_INLINE void ADC_HWA_SetConfig2 (ADC_Type *const *pAdc*, uint32_t *u32Config*)

Set the ADC_CFG2 config.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u32Config</i>	the ADC_CFG2 config

LOCAL_INLINE void ADC_HWA_SetDataAlignment (ADC_Type *const *pAdc*, ADC_AlignType *eAlign*)

Set the data align mode.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eAlign</i>	the data align mode

LOCAL_INLINE void ADC_HWA_SetDataResolution (ADC_Type *const *pAdc*, ADC_ResolutionType *eResolution*)

Set the resolution of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eResolution</i>	the resolution of the ADC instance

LOCAL_INLINE void ADC_HWA_SetDMAEnableFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Enable or disable the DMA for the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the DMA for the ADC instance

LOCAL_INLINE void ADC_HWA_SetEndOfConversionInterruptFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the conversion complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion is completed.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the conversion complete interrupt

LOCAL_INLINE void ADC_HWA_SetEndOfSampleInterruptFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the sample complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion finished the sampling phase.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the sample complete interrupt

LOCAL_INLINE void ADC_HWA_SetEndOfSequenceInterruptFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the End of Sequence interrupt enable flag If enabled, ADC interrupt is generated when the ADC sequence conversion is completed.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the ADC End of Sequence interrupt

LOCAL_INLINE void ADC_HWA_SetFIFOReadyInterruptFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the FIFO Ready interrupt flag If enabled, ADC interrupt is generated when the FIFO water mark is greater than FWMARK.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the ADC FIFO Ready interrupt

LOCAL_INLINE void ADC_HWA_SetFIFOWaterMark (ADC_Type *const *pAdc*, uint8_t *u8WaterMark*)

Set the FIFO water mark for the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8WaterMark</i>	the ADC FIFO water mark setting

LOCAL_INLINE void ADC_HWA_SetHwCompareChannel (ADC_Type *const *pAdc*, ADC_CmpChannelType *eChannelType*, uint8_t *u8ChannalNum*)

Set the ADC hardware compare channel.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eChannelType</i>	whether the hardware compare enabled on single channel or all channels
<i>u8ChannalNum</i>	if hardware compare is enabled on single channel, this specifies the channel number

LOCAL_INLINE void ADC_HWA_SetHwCompareEnableFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set to enable or disable the hardware compare.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether the ADC hardware compare is enabled

LOCAL_INLINE void ADC_HWA_SetHwCompareThreshold (ADC_Type *const *pAdc*, uint16_t *u16LowThres*, uint16_t *u16HighThres*)

Set the ADC hardware compare threshold.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u16LowThres</i>	the lower threshold
<i>u16HighThres</i>	the higher threshold

LOCAL_INLINE void ADC_HWA_SetInterruptEnable (ADC_Type *const *pAdc*, uint32_t *u32InterruptCfg*)

Set the interrupt enable.

Parameters

<i>pAdc</i>	adc address point
<i>u32InterruptCfg</i>	the interrupt enable config

LOCAL_INLINE void ADC_HWA_SetOverrunInterruptFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the Overrun interrupt flag If enabled, ADC interrupt is generated when the ADC instance is overrun.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the ADC Overrun interrupt

**LOCAL_INLINE void ADC_HWA_SetOverrunManagementMode (ADC_Type *const *pAdc*,
ADC_OvrModeType *eOvrMode*)**

Set the overrun management mode.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eOvrMode</i>	the overrun management for the ADC instance

**LOCAL_INLINE void ADC_HWA_SetSampleTime (ADC_Type *const *pAdc*, uint8_t *u8Selection*, uint8_t
u8SampleTime)**

Set the sample time of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8Selection</i>	the sample time index, range 0~3
<i>u8SampleTime</i>	the sample time of the selected index

**LOCAL_INLINE void ADC_HWA_SetSequenceLength (ADC_Type *const *pAdc*, uint8_t
u8SequenceLength)**

Set the sequence length of the ADC conversion sequence.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8SequenceLength</i>	the sequence length of the ADC conversion sequence

**LOCAL_INLINE void ADC_HWA_SetSequenceMode (ADC_Type *const *pAdc*, ADC_SeqModeType
eSequenceMode)**

Set the ADC sequence mode.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eSequenceMod</i> <i>e</i>	the sequence mode the the ADC instance

LOCAL_INLINE void ADC_HWA_SetStartupCnt (ADC_Type *const *pAdc*, uint8_t *u8StartupCnt*)

Set the ADC start up count.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>u8StartupCnt</i>	the start count of the ADC instance

LOCAL_INLINE void ADC_HWA_SetTriggerMode (ADC_Type *const *pAdc*, ADC_TrgModeType *eTrgMode*)

Set the trigger mode of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eTriggerMode</i>	the trigger mode if the ADC instance

LOCAL_INLINE void ADC_HWA_SetTriggerSource (ADC_Type *const *pAdc*, ADC_TrgSrcType *eTrgSource*)

Set the trigger source the the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eTriggerSource</i>	the trigger source of the ADC instance

LOCAL_INLINE void ADC_HWA_SetVoltageReference (ADC_Type *const *pAdc*, ADC_RefType *eRefSel*)

Set the voltage reference of the ADC instance.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>eRefSel</i>	the voltage reference of the ADC instance

LOCAL_INLINE void ADC_HWA_SetWaitConversionModeFlag (ADC_Type *const *pAdc*, bool *bEnable*)

Set the wait conversion mode.

Parameters

<i>pAdc</i>	the base address of the ADC instance
<i>bEnable</i>	whether to enable the wait conversion mode

LOCAL_INLINE void ADC_HWA_Start (ADC_Type *const *pAdc*)

Start the ADC conversion.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE void ADC_HWA_Stop (ADC_Type *const *pAdc*)

Stop the ADC conversion.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

LOCAL_INLINE bool ADC_HWAGetEndOfConversionInterruptFlag (const ADC_Type *const *pAdc*)

Get the conversion complete interrupt enable flag. If enabled, ADC interrupt is generated when each ADC conversion is completed.

Parameters

<i>pAdc</i>	the base address of the ADC instance
-------------	--------------------------------------

Returns

true the ADC End of Conversion interrupt is enabled

false the ADC End of Conversion interrupt is disabled

HwA_aontimer

Collaboration diagram for HwA_aontimer:Enumerations

```
enum AONTIMER_PulseClkSrcType { AONTIMER_CLK0_PIN = 0, AONTIMER_CLK1_PIN,  
AONTIMER_CLK2_PIN, AONTIMER_TRGSEL_OUTPUT }
```

The clock source of the pulse mode.

```
enum AONTIMER_ClkSrcType { AONTIMER_SIRC_1MHZ = 0U, AONTIMER_RTC_CLK = 2U,  
AONTIMER_IRC_CLK = 3U }
```

Aontimer clock source, please refer to Reference Manual chapter8.Aontimer for details.

```
enum AONTIMER_PulsePolarityType { AONTIMER_PulsePolarityType_HIGH = 0,  
AONTIMER_PulsePolarityType_LOW }
```

The polarity of pulse mode.

Functions

```
LOCAL_INLINE void AONTIMER_HWA_ConfigModule (AONTIMER_Type *const pAontimer, uint32_t  
u32RegValue)
```

Configure AONTIMER module.

```
LOCAL_INLINE void AONTIMER_HWA_ConfigModulePrescale (AONTIMER_Type *const pAontimer, uint32_t  
u32RegValue)
```

Configure AONTIMER module prescale.

```
LOCAL_INLINE void AONTIMER_HWA_SetModuleCompareValue (AONTIMER_Type *const pAontimer,  
uint32_t u32RegValue)
```

Set AONTIMER compare value.

```
LOCAL_INLINE void AONTIMER_HWA_SetModuleCounterValue (AONTIMER_Type *const pAontimer,  
uint32_t u32RegValue)
```

Set AONTIMER current counter value.

LOCAL_INLINE void **AONTIMER_HWA_SetModuleRunOnDebug** (AONTIMER_Type *const pAontimer)

Set AONTIMER module running on debug mode.

LOCAL_INLINE void **AONTIMER_HWA_EnableModuleInterrupt** (AONTIMER_Type *const pAontimer)

Enable AONTIMER module interrupt.

LOCAL_INLINE void **AONTIMER_HWA_SelectClkSrcOnPulseMode** (AONTIMER_Type *const pAontimer,
AONTIMER_PulseClkSrcType eClk)

Select AONTIMER module external clock source when timer configured to pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_ClearInterruptFlag** (AONTIMER_Type *const pAontimer)

Clear AONTIMER interrupt flags.

LOCAL_INLINE void **AONTIMER_HWA_SetModulePolarity** (AONTIMER_Type *const pAontimer)

Set AONTIMER module polarity. Pulse counter input source is active-low, and the CNR increments on falling-edge.

LOCAL_INLINE void **AONTIMER_HWA_ConfigModulePolarity** (AONTIMER_Type *const pAontimer,
AONTIMER_PulsePolarityType ePol)

Configure AONTIMER module polarity. If ePol is 0:Pulse counter input source is active-high, and the CNR increments on rising-edge. If ePol is 1:Pulse counter input source is active-low, and the CNR increments on falling-edge.

LOCAL_INLINE void **AONTIMER_HWA_EnablePulseMode** (AONTIMER_Type *const pAontimer)

Enable AONTIMER module pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_EnableTimer** (AONTIMER_Type *const pAontimer)

Enable AONTIMER timer.

LOCAL_INLINE void **AONTIMER_HWA_SetPrescale** (AONTIMER_Type *const pAontimer, uint8_t
u8PrescalerValue)

Set AONTIMER prescale.

LOCAL_INLINE void **AONTIMER_HWA_EnableBypassMode** (AONTIMER_Type *const pAontimer)

If enable bypass mode, the timer will bypass the prescaler in timer counter mode or glitch filter in pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_SelectModuleClkSrc** (AONTIMER_Type *const pAontimer,
AONTIMER_ClkSrcType eClk)

Select AONTIMER module clock source.

LOCAL_INLINE void **AONTIMER_HWA_SetModuleStopOnDebug** (AONTIMER_Type *const pAontimer)

Set AONTIMER module stop on debug mode.

LOCAL_INLINE void **AONTIMER_HWA_DisableModuleInterrupt** (AONTIMER_Type *const pAontimer)

Disable AONTIMER module interrupt.

LOCAL_INLINE void **AONTIMER_HWA_ClearModuleMode** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module mode.

LOCAL_INLINE void **AONTIMER_HWA_ClearModulePolarity** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module polarity. Pulse counter input source is active-high, and the CNR increments on rising-edge.

LOCAL_INLINE void **AONTIMER_HWA_DisablePulseMode** (AONTIMER_Type *const pAontimer)

Disable AONTIMER module pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_DisableTimer** (AONTIMER_Type *const pAontimer)

Disable AONTIMER module timer.

LOCAL_INLINE void **AONTIMER_HWA_ClearPrescale** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module prescaler.

LOCAL_INLINE void **AONTIMER_HWA_DisableBypassMode** (AONTIMER_Type *const pAontimer)

If disable bypass mode, the timer will enable the prescaler in timer counter mode or glitch filter in pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_ClearModuleClkSrc** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module clock source.

Detailed Description

Enumeration Type Documentation

enum AONTIMER_ClkSrcType

Aontimer clock source, please refer to Reference Manual chapter8.Aontimer for details.

Enumerator:

AONTIMER_SIRC _1MHZ	AONTIMER SIRC 1mhz clock
AONTIMER_RTC _CLK	AONTIMER RTC clock
AONTIMER_IRC_ CLK	AONTIMER internal clock, which comes from PCC

enum AONTIMER_PulseClkSrcType

The clock source of the pulse mode.

Enumerator:

AONTIMER_CLK 0_PIN	select the Aontimer_clk0 pin as pulse source
AONTIMER_CLK 1_PIN	select the Aontimer_clk1 pin as pulse source

AONTIMER_CLK 2_PIN	select the Aontimer_clk2 pin as pulse source
AONTIMER_TRG SEL_OUTPUT	select the tresel as pulse source

enum AONTIMER_PulsePolarityType

The polarity of pulse mode.

Enumerator:

AONTIMER_Puls ePolarityType_ HIGH	select the high polarity
AONTIMER_Puls ePolarityType_ LOW	select the low polarity

Function Documentation

LOCAL_INLINE void AONTIMER_HWA_ClearInterruptFlag (AONTIMER_Type *const *pAontimer*)

Clear AONTIMER interrupt flags.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_ClearModuleClkSrc (AONTIMER_Type *const *pAontimer*)

Clear AONTIMER module clock source.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_ClearModuleMode (AONTIMER_Type *const *pAontimer*)

Clear AONTIMER module mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_ClearModulePolarity (AONTIMER_Type *const *pAontimer*)

Clear AONTIMER module polarity. Pulse counter input source is active-high, and the CNR increments on rising-edge.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_ClearPrescale (AONTIMER_Type *const *pAontimer*)

Clear AONTIMER module prescaler.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_ConfigModule (AONTIMER_Type *const *pAontimer*, uint32_t *u32RegValue*)

Configure AONTIMER module.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>u32RegValue</i>	CSR register value

LOCAL_INLINE void AONTIMER_HWA_ConfigModulePolarity (AONTIMER_Type *const *pAontimer*, AONTIMER_PulsePolarityType *ePol*)

Configure AONTIMER module polarity. If ePol is 0:Pulse counter input source is active-high, and the CNR increments on rising-edge. If ePol is 1:Pulse counter input source is active-low, and the CNR increments on falling-edge.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>ePol</i>	Polarity enumeration

**LOCAL_INLINE void AONTIMER_HWA_ConfigModulePrescale (AONTIMER_Type *const *pAontimer*,
uint32_t *u32RegValue*)**

Configure AONTIMER module prescale.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>u32RegValue</i>	PSR register value

LOCAL_INLINE void AONTIMER_HWA_DisableBypassMode (AONTIMER_Type *const *pAontimer*)

If disable bypass mode, the timer will enable the prescaler in timer counter mode or glitch filter in pulse mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_DisableModuleInterrupt (AONTIMER_Type *const *pAontimer*)

Disable AONTIMER module interrupt.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_DisablePulseMode (AONTIMER_Type *const *pAontimer*)

Disable AONTIMER module pulse mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_DisableTimer (AONTIMER_Type *const *pAontimer*)

Disable AONTIMER module timer.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_EnableBypassMode (AONTIMER_Type *const *pAontimer*)

If enable bypass mode, the timer will bypass the prescaler in timer counter mode or glitch filter in pulse mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_EnableModuleInterrupt (AONTIMER_Type *const *pAontimer*)

Enable AONTIMER module interrupt.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_EnablePulseMode (AONTIMER_Type *const *pAontimer*)

Enable AONTIMER module pulse mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_EnableTimer (AONTIMER_Type *const *pAontimer*)

Enable AONTIMER timer.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

**LOCAL_INLINE void AONTIMER_HWA_SelectClkSrcOnPulseMode (AONTIMER_Type *const *pAontimer*,
AONTIMER_PulseClkSrcType *eClk*)**

Select AONTIMER module external clock source when timer configured to pulse mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>eClk</i>	Input counter clock source

**LOCAL_INLINE void AONTIMER_HWA_SelectModuleClkSrc (AONTIMER_Type *const *pAontimer*,
AONTIMER_ClkSrcType *eClk*)**

Select AONTIMER module clock source.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>eClk</i>	Aontimer clock source

**LOCAL_INLINE void AONTIMER_HWA_SetModuleCompareValue (AONTIMER_Type *const *pAontimer*,
uint32_t *u32RegValue*)**

Set AONTIMER compare value.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>u32RegValue</i>	CMR register value

**LOCAL_INLINE void AONTIMER_HWA_SetModuleCounterValue (AONTIMER_Type *const *pAontimer*,
uint32_t *u32RegValue*)**

Set AONTIMER current counter value.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>u32RegValue</i>	CNR register value

LOCAL_INLINE void AONTIMER_HWA_SetModulePolarity (AONTIMER_Type *const *pAontimer*)

Set AONTIMER module polarity. Pulse counter input source is active-low, and the CNR increments on falling-edge.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_SetModuleRunOnDebug (AONTIMER_Type *const *pAontimer*)

Set AONTIMER module running on debug mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_SetModuleStopOnDebug (AONTIMER_Type *const *pAontimer*)

Set AONTIMER module stop on debug mode.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
------------------	---

LOCAL_INLINE void AONTIMER_HWA_SetPrescale (AONTIMER_Type *const *pAontimer*, uint8_t *u8PrescalerValue*)

Set AONTIMER prescale.

Parameters

<i>pAontimer</i>	the base address of the pAontimer instance.
<i>u8PrescalerValue</i>	Prescaler value, the range of the input value is :0~15, and the range of prescaler is :2 ¹ ~ 2 ¹⁶ .

HwA_cmp

Collaboration diagram for HwA_cmp:Enumerations

```
enum CMP_ModeSelType { CMP_MODE_DISABLE = 0U, CMP_MODE_CONTINUOUS = 1U,  
CMP_MODE_SAMPLE_NONFILTER_EXTCLK = 2U, CMP_MODE_SAMPLE_NONFILTER_INTCLK = 3U,  
CMP_MODE_SAMPLE_FILTER_EXTCLK = 4U, CMP_MODE_SAMPLE_FILTER_INTCLK = 5U,  
CMP_MODE_WINDOW = 6U, CMP_MODE_WINDOW_RESAMPLE = 7U, CMP_MODE_WINDOW_FILTER = 8U,  
CMP_MODE_CHANNEL_SCAN = 9U }
```

The instance index of the CMP mode.

```
enum CMP_InstanceType { CMP_INSTANCE_0 = 0U, CMP_INSTANCE_1 = 1U, CMP_INSTANCE_2 = 2U }
```

The instance index of the CMP peripheral.

```
enum CMP_DacEnableSrcType { CMP_DACENABLE_DCR = 0U, CMP_DACENABLE_CCR0 = 1U }
```

The instance index of the CMP DAC enable select.

```
enum CMP_InvertType { CMP_NON_INVERT = 0U, CMP_INVERT = 1U }
```

The instance index of the CMP output invert.

```
enum CMP_OutSelectType { CMP_FILTEROUT = 0U, CMP_UNFILTEROUT = 1U }
```

The instance index of the CMP output select.

```
enum CMP_OutWinLevelType { CMP_HOLD = 0U, CMP_USERDEF = 1U }
```

The instance index of the CMP window level.

```
enum CMP_OutWinLevel_UserDefType { CMP_OUTWIN_0 = 0U, CMP_OUTWIN_1 = 1U }
```

The instance index of the window output under userdefine CMP window level.

```
enum CMP_EventType { CMP_RISINGEDGE = 0U, CMP_FALLINGEDGE = 1U, CMP_BOTHEDGES = 2U }
```

The instance index of the CMP Event caused window close.

```
enum CMP_FilterCntType { CMP_FILTERCNT_0 = 0U, CMP_FILTERCNT_1 = 1U, CMP_FILTERCNT_2 = 2U,  
CMP_FILTERCNT_3 = 3U, CMP_FILTERCNT_4 = 4U }
```

The instance index of the CMP filter count numbers.

```
enum CMP_HystCtrlType { CMP_HYSTCTRL_0 = 0U, CMP_HYSTCTRL_1 = 1U, CMP_HYSTCTRL_2 = 2U,  
CMP_HYSTCTRL_3 = 3U }
```

The instance index of the CMP hysteresis control.

enum **CMP_PortSelType** { **CMP_PORTSEL_MUX_P** = 0U, **CMP_PORTSEL_MUX_N** = 1U }

The fixed CMP port for reference in channel scan mode.

enum **CMP_INSrcSelType** { **CMP_INSRCSEL_DAC** = 0U, **CMP_INSRCSEL_MUX** = 1U }

The source of the CMP input.

enum **CMP_MuxSelType** { **CMP_INSEL_MUX_IN0** = 0U, **CMP_INSEL_MUX_IN1** = 1U, **CMP_INSEL_MUX_IN2** = 2U, **CMP_INSEL_MUX_IN3** = 3U, **CMP_INSEL_MUX_IN4** = 4U, **CMP_INSEL_MUX_IN5** = 5U, **CMP_INSEL_MUX_IN6** = 6U, **CMP_INSEL_MUX_IN7** = 7U }

The instance index of the CMP input mux.

enum **CMP_DacVinRefSelType** { **CMP_DACVINREF_H0** = 0U, **CMP_DACVINREF_H1** = 1U }

The instance index of DAC high voltage reference select.

enum **CMP_SpeedModeSelType** { **CMP_LOWSPEEDMODE** = 0U, **CMP_HIGHSPEEDMODE** = 1U }

The instance index of high power mode select.

enum **CMP_OutStatus** { **CMP_OUT_FALLING_EDGE** = 0U, **CMP_OUT_RISING_EDGE** = 1U, **CMP_OUT_NONE** = 2U }

Defines CMP out status.

Functions

LOCAL_INLINE void **CMP_HWA_SetEn** (CMP_Type *const pCmp, bool bEnable)

set CMP enable

LOCAL_INLINE void **CMP_HWA_SetEnLPMode** (CMP_Type *const pCmp, bool bEnable)

STOP mode enable.

LOCAL_INLINE void **CMP_HWA_SetDacEnableSrc** (CMP_Type *const pCmp, **CMP_DacEnableSrcType** eType)

set DAC enable selection

LOCAL_INLINE void **CMP_HWA_SetDma** (CMP_Type *const pCmp, bool bEnable)

set CMP DMA enable

LOCAL_INLINE void **CMP_HWA_SetComparatorMode** (CMP_Type *const pCmp, **CMP_ModeSelType** eMode, uint8_t u8FilterPrd, **CMP_FilterCntType** eFilterCnt)

set CMP mode

LOCAL_INLINE void **CMP_HWA_SetCmpOutInvert** (CMP_Type *const pCmp, **CMP_InvertType** eType)

set CPM output invert

LOCAL_INLINE void **CMP_HWA_SetCmpOutSel** (CMP_Type *const pCmp, **CMP_OutSelectType** eType)

set CPM output filter/unfilter selection

LOCAL_INLINE void **CMP_HWA_SetEnCmpOutPack** (CMP_Type *const pCmp, bool bEnable)

set comparator output pin enable

LOCAL_INLINE void **CMP_HWA_SetCmpOutWinLevel** (CMP_Type *const pCmp, **CMP_OutWinLevelType** eType)

set CMPOUT_WIN level, when window is closed

LOCAL_INLINE void **CMP_HWA_SetCmpOutWin** (CMP_Type *const pCmp, **CMP_OutWinLevel_UserDefType** eType)

set CMPOUT_WIN level in user-define mode, when window is closed

LOCAL_INLINE void **CMP_HWA_SetEnWinSampleInvert** (CMP_Type *const pCmp, bool bEnable)

set invert the WINDOW/SAMPLE signal enable or not

LOCAL_INLINE void **CMP_HWA_SetEnEventCloseWin** (CMP_Type *const pCmp, bool bEnable)

WINDOW signal can or not be closed by CMPO event when window mode.

LOCAL_INLINE void **CMP_HWA_SetEventCloseWin** (CMP_Type *const pCmp, **CMP_EventType** eType)

set which CMPO event causes window close

LOCAL_INLINE void **CMP_HWA_SetCCR1** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CCR1 register

LOCAL_INLINE void **CMP_HWA_SetSpeedMode** (CMP_Type *const pCmp, **CMP_SpeedModeSelType** eMode)

set CMP power mode select

LOCAL_INLINE void **CMP_HWA_SetHystCtrl** (CMP_Type *const pCmp, **CMP_HystCtrlType** eType)

set Comparator hard block hysteresis control

LOCAL_INLINE void **CMP_HWA_SetPSelMux** (CMP_Type *const pCmp, **CMP_MuxSelType** eType)
set which input is selected for the positive mux

LOCAL_INLINE void **CMP_HWA_SetNSelMux** (CMP_Type *const pCmp, **CMP_MuxSelType** eType)
set which input is selected for the negative mux

LOCAL_INLINE void **CMP_HWA_SetINPSel** (CMP_Type *const pCmp, **CMP_INSrcSelType** eType)
set the input to the positive port of the comparator

LOCAL_INLINE void **CMP_HWA_SetINNSel** (CMP_Type *const pCmp, **CMP_INSrcSelType** eType)
set the input to the negative port of the comparator

LOCAL_INLINE void **CMP_HWA_SetCCR2** (CMP_Type *const pCmp, uint32_t u32RegVal)
set CCR2 register

LOCAL_INLINE void **CMP_HWA_SetEnDac** (CMP_Type *const pCmp, bool bEnable)
set CMP DAC enable

LOCAL_INLINE void **CMP_HWA_SetVinRefSel** (CMP_Type *const pCmp, **CMP_DacVinRefSelType** eType)
set DAC reference voltage source

LOCAL_INLINE void **CMP_HWA_SetDacData** (CMP_Type *const pCmp, uint8_t u8Data)
set CMP Dac output

LOCAL_INLINE void **CMP_HWA_SetDCR** (CMP_Type *const pCmp, uint32_t u32RegVal)
set DCR register

LOCAL_INLINE void **CMP_HWA_SetIntEn_Rising** (CMP_Type *const pCmp, bool bEnable)
set comparator rising interrupt enable

LOCAL_INLINE bool **CMP_HWA_GetIntEn_Rising** (CMP_Type *const pCmp)
get comparator rising interrupt enable status

LOCAL_INLINE void **CMP_HWA_SetIntEn_Falling** (CMP_Type *const pCmp, bool bEnable)
set comparator falling interrupt enable

LOCAL_INLINE bool **CMP_HWA_GetIntEn_Falling** (CMP_Type *const pCmp)
get comparator falling interrupt enable status

LOCAL_INLINE void **CMP_HWA_SetIER** (CMP_Type *const pCmp, uint32_t u32RegVal)
set IER register

LOCAL_INLINE bool **CMP_HWA_GetIntFlag_Rising** (CMP_Type *const pCmp)
get CMP output rising edge status

LOCAL_INLINE bool **CMP_HWA_GetIntFlag_Falling** (CMP_Type *const pCmp)
get CMP output falling edge status

LOCAL_INLINE void **CMP_HWA_ClearIntFlag_Rising** (CMP_Type *const pCmp)
clear rising interrupt flag

LOCAL_INLINE void **CMP_HWA_ClearIntFlag_Falling** (CMP_Type *const pCmp)
clear falling interrupt flag

LOCAL_INLINE uint32_t **CMP_HWA_GetCmpOut** (CMP_Type *const pCmp)
get CMP filtered output

LOCAL_INLINE void **CMP_HWA_SetCSR** (CMP_Type *const pCmp, uint32_t u32RegVal)
set CSR register

LOCAL_INLINE void **CMP_HWA_SetIntEn_ChannelScan** (CMP_Type *const pCmp, bool bEnable)
set comparator channel scan interrupt enable

LOCAL_INLINE bool **CMP_HWA_GetIntEn_ChannelScan** (CMP_Type *const pCmp)
get comparator channel scan interrupt enable status

LOCAL_INLINE void **CMP_HWA_ClearIntFlag_ChannelScan** (CMP_Type *const pCmp)

clear channel scan interrupt flag

LOCAL_INLINE void **CMP_HWA_SetCSInitModulus** (CMP_Type *const pCmp, uint8_t u8Modulus)

set CMP and DAC initialization delay modulus

LOCAL_INLINE void **CMP_HWA_SetCSNSAM** (CMP_Type *const pCmp, uint8_t u8Nsam)

set number of clock cycles for sampling

LOCAL_INLINE void **CMP_HWA_SetCSEn** (CMP_Type *const pCmp, bool bEnable)

set CMP channel scan enable

LOCAL_INLINE void **CMP_HWA_SetCSCR0** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSCR0 register

LOCAL_INLINE void **CMP_HWA_SetCSFixedChannel** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel)

set channel scan fixed channel

LOCAL_INLINE void **CMP_HWA_SetCSFixedPort** (CMP_Type *const pCmp, **CMP_PortSelType** ePort)

set channel scan fixed port

LOCAL_INLINE void **CMP_HWA_SetCSChannelEn** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel, bool bEnable)

set channel scan channel enabled

LOCAL_INLINE void **CMP_HWA_SetCSCR1** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSCR1 register

LOCAL_INLINE bool **CMP_HWA_GetIntFlag_ChannelScan** (CMP_Type *const pCmp)

get CMP output channel scan status

LOCAL_INLINE void **CMP_HWA_SetCSChannelPresetstate** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel, bool bPresetstate)

set channel scan channel enabled

LOCAL_INLINE bool **CMP_HWA_GetCSChannelOut** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel)

get channel scan channel current state

LOCAL_INLINE void **CMP_HWA_ClearCSCompRes** (CMP_Type *const pCmp)

software clear channel scan comparison results

LOCAL_INLINE void **CMP_HWA_SetCSCompareResultACEn** (CMP_Type *const pCmp, bool bEnable)

software clear channel scan comparison results

LOCAL_INLINE void **CMP_HWA_SetCSCSR** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSCSR register

LOCAL_INLINE bool **CMP_HWA_GetCSActive** (CMP_Type *const pCmp)

get channel scan active status

LOCAL_INLINE bool **CMP_HWA_GetCSComparisonResultFlag** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel)

get channel scan comparison result changed flag

LOCAL_INLINE void **CMP_HWA_SetCSSR** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSSR register

Detailed Description

Enumeration Type Documentation

enum CMP_DacEnableSrcType

The instance index of the CMP DAC enable select.

Enumerator:

CMP_DACENABL E_DCR	CMP Dac is enabled by DCR[DAC_EN]
CMP_DACENABL E_CCR0	CMP Dac is enabled by CCR0[EN]

enum CMP_DacVinRefSelType

The instance index of DAC high voltage reference select.

Enumerator:

CMP_DACVINRE F_H0	CMP DAC high voltage input reference vrefh0
CMP_DACVINRE F_H1	CMP DAC high voltage input reference vrefh1

enum CMP_EventType

The instance index of the CMP Event caused window close.

Enumerator:

CMP_RISINGED GE	CMP output event RisingEdge causes window close
CMP_FALLINGE DGE	CMP output event FallingEdge causes window close

CMP_BOTHEDGES	CMP output event bothEdges causes window close
---------------	--

enum CMP_FilterCntType

The instance index of the CMP filter count numbers.

Enumerator:

CMP_FILTERCNT_0	CMP filter is bypassed
CMP_FILTERCNT_1	CMP filter is 1 consecutive sample
CMP_FILTERCNT_2	CMP filter is 2 consecutive sample
CMP_FILTERCNT_3	CMP filter is 3 consecutive sample
CMP_FILTERCNT_4	CMP filter is 4 consecutive sample

enum CMP_HystCtrlType

The instance index of the CMP hysteresis control.

Enumerator:

CMP_HYSTCTRL_0	CMP 0 hysteresis internal
----------------	---------------------------

CMP_HYSTCTRL _1	CMP 1 hysteresis internal
CMP_HYSTCTRL _2	CMP 2 hysteresis internal
CMP_HYSTCTRL _3	CMP 3 hysteresis internal

enum CMP_INSrcSelType

The source of the CMP input.

Enumerator:

CMP_INSRCSEL _DAC	CMP input source is DAC
CMP_INSRCSEL _MUX	CMP input source is analog 1-8 mux

enum CMP_InstanceType

The instance index of the CMP peripheral.

Enumerator:

CMP_INSTANCE _0	CMP instance 0 is selected
CMP_INSTANCE _1	CMP instance 1 is selected
CMP_INSTANCE _2	CMP instance 2 is selected

enum CMP_InvertType

The instance index of the CMP output invert.

Enumerator:

CMP_NON_INVE RT	CMP output do not Invert
CMP_INVERT	CMP output Invert

enum CMP_ModeSelType

The instance index of the CMP mode.

Enumerator:

CMP_MODE_DIS ABLE	CMP function mode is disable
CMP_MODE_CO NTINUOUS	CMP function mode is continuous
CMP_MODE_SA MPLE_NONFILT ER_EXTCLK	CMP function mode is sampled,non-filtered mode 1
CMP_MODE_SA MPLE_NONFILT ER_INTCLK	CMP function mode is sampled,non-filtered mode 2
CMP_MODE_SA MPLE_FILTER_E XTCLK	CMP function mode is sampled,filtered mode 1
CMP_MODE_SA MPLE_FILTER_I NTCLK	CMP function mode is sampled,filtered mode 2
CMP_MODE_WI NDOW	CMP function mode is windowed mode

CMP_MODE_WINDOW_RESAMPLE	CMP function mode is windowed, re-sampled mode
CMP_MODE_WINDOW_FILTER	CMP function mode is windowed, filtered mode
CMP_MODE_CHANNEL_SCAN	CMP channel scan mode

enum CMP_MuxSelType

The instance index of the CMP input mux.

Enumerator:

CMP_INSEL_MUX_IN0	CMP input mux from IN0(CMP0,CMP1,CMP2)
CMP_INSEL_MUX_IN1	CMP input mux from IN1(CMP0,CMP1,CMP2)
CMP_INSEL_MUX_IN2	CMP input mux from IN2(CMP0,CMP1,CMP2)
CMP_INSEL_MUX_IN3	CMP input mux from IN3(CMP0,CMP1,CMP2)
CMP_INSEL_MUX_IN4	CMP input mux from IN4(CMP0,CMP1,CMP2)
CMP_INSEL_MUX_IN5	CMP input mux from IN5(CMP1,CMP2)
CMP_INSEL_MUX_IN6	CMP input mux from IN6(CMP1,CMP2)
CMP_INSEL_MUX_IN7	CMP input mux from IN7(CMP1,CMP2)

enum CMP_OutSelectType

The instance index of the CMP output select.

Enumerator:

CMP_FILTEROUT	CMP filter output
CMP_UNFILTEROUT	CMP Unfilter output

enum CMP_OutStatus

Defines CMP out status.

Enumerator:

CMP_OUT_FALLING_EDGE	CMP out detect falling edge
CMP_OUT_RISING_EDGE	CMP out detect rising edge
CMP_OUT_NONE	CMP out detect none

enum CMP_OutWinLevel_UserDefType

The instance index of the window output under userdefine CMP window level.

Enumerator:

CMP_OUTWIN_0	CMP window output is 0
CMP_OUTWIN_1	CMP window output is 1

--	--

enum CMP_OutWinLevelType

The instance index of the CMP window level.

Enumerator:

CMP_HOLD	CMP output hold when window close
CMP_USERDEF	CMP output userdefine when window close

enum CMP_PortSelType

The fixed CMP port for reference in channel scan mode.

Enumerator:

CMP_PORTSEL_ MUX_P	CMP Positive Port
CMP_PORTSEL_ MUX_N	CMP Negative Port

enum CMP_SpeedModeSelType

The instance index of high power mode select.

Enumerator:

CMP_LOWSPEE DMODE	CMP low speed mode
----------------------	--------------------

CMP_HIGHSPEEDMODE	CMP high speed mode
-------------------	---------------------

Function Documentation

LOCAL_INLINE void CMP_HWA_ClearCSCompRes (CMP_Type *const *pCmp*)

software clear channel scan comparison results

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

LOCAL_INLINE void CMP_HWA_ClearIntFlag_ChannelScan (CMP_Type *const *pCmp*)

clear channel scan interrupt flag

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

LOCAL_INLINE void CMP_HWA_ClearIntFlag_Falling (CMP_Type *const *pCmp*)

clear falling interrupt flag

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

LOCAL_INLINE void CMP_HWA_ClearIntFlag_Rising (CMP_Type *const *pCmp*)

clear rising interrupt flag

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

LOCAL_INLINE uint32_t CMP_HWA_GetCmpOut (CMP_Type *const *pCmp*)

get CMP filtered output

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

CMP filtered output

LOCAL_INLINE bool CMP_HWA_GetCSActive (CMP_Type *const *pCmp*)

get channel scan active status

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

status cmp channel whether is active

LOCAL_INLINE bool CMP_HWA_GetCSChannelOut (CMP_Type *const *pCmp*, CMP_MuxSelType *eChannel*)

get channel scan channel current state

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eChannel</i>	the channel that want to get state

LOCAL_INLINE bool CMP_HWA_GetCSComparisonResultFlag (CMP_Type *const *pCmp*, CMP_MuxSelType *eChannel*)

get channel scan comparison result changed flag

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eChannel</i>	the channel to get comparison result changed flag

Returns

the comparison result changed flag for a given channel

LOCAL_INLINE bool CMP_HWA_GetIntEn_ChannelScan (CMP_Type *const *pCmp*)

get comparator channel scan interrupt enable status

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

comparator channel scan interrupt status

LOCAL_INLINE bool CMP_HWA_GetIntEn_Falling (CMP_Type *const *pCmp*)

get comparator falling interrupt enable status

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

comparator falling interrupt status

LOCAL_INLINE bool CMP_HWA_GetIntEn_Rising (CMP_Type *const *pCmp*)

get comparator rising interrupt enable status

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

comparator rising interrupt status

LOCAL_INLINE bool CMP_HWA_GetIntFlag_ChannelScan (CMP_Type *const *pCmp*)

get CMP output channel scan status

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

CMP channel scan status

LOCAL_INLINE bool CMP_HWA_GetIntFlag_Falling (CMP_Type *const *pCmp*)

get CMP output falling edge status

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

CMP falling edge status

LOCAL_INLINE bool CMP_HWA_GetIntFlag_Rising (CMP_Type *const *pCmp*)

get CMP output rising edge status

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

Returns

CMP rising edge status

LOCAL_INLINE void CMP_HWA_SetCCR1 (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set CCR1 register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetCCR2 (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set CCR2 register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetCmpOutInvert (CMP_Type *const *pCmp*, CMP_InvertType *eType*)

set CPM output invert

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CPM output invert type

LOCAL_INLINE void CMP_HWA_SetCmpOutSel (CMP_Type *const *pCmp*, CMP_OutSelectType *eType*)

set CPM output filter/unfilter selection

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CPM output filter/unfilter type

**LOCAL_INLINE void CMP_HWA_SetCmpOutWin (CMP_Type *const *pCmp*,
CMP_OutWinLevel_UserDefType *eType*)**

set CMPOUT_WIN level in user-define mode, when window is closed

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	user-define CMPOUT_WIN level type

LOCAL_INLINE void CMP_HWA_SetCmpOutWinLevel (CMP_Type *const *pCmp*, CMP_OutWinLevelType *eType*)

set CMPOUT_WIN level, when window is closed

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMPOUT_WIN level type

LOCAL_INLINE void CMP_HWA_SetComparatorMode (CMP_Type *const *pCmp*, CMP_ModeSelType *eMode*, uint8_t *u8FilterPrd*, CMP_FilterCntType *eFilterCnt*)

set CMP mode

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eMod</i>	the CMP mode to use
<i>u8FilterPrd</i>	the CMP filter period
<i>eFilterCnt</i>	the CMP filter sample count

LOCAL_INLINE void CMP_HWA_SetCSChannelEn (CMP_Type *const *pCmp*, CMP_MuxSelType *eChannel*, bool *bEnable*)

set channel scan channel enabled

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eChannel</i>	the enabled channel
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetCSChannelPresetstate (CMP_Type *const *pCmp*, CMP_MuxSelType *eChannel*, bool *bPresetstate*)

set channel scan channel enabled

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eChannel</i>	the enabled channel
<i>bPresetstate</i>	preset state for channel

LOCAL_INLINE void CMP_HWA_SetCSCompareResultACEn (CMP_Type *const *pCmp*, bool *bEnable*)

software clear channel scan comparison results

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetCSCR0 (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set CSCR0 register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetCSCR1 (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set CSCR1 register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetCSCSR (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set CSCSR register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetCSEn (CMP_Type *const *pCmp*, bool *bEnable*)

set CMP channel scan enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetCSFixedChannel (CMP_Type *const *pCmp*, CMP_MuxSelType *eChannel*)

set channel scan fixed channel

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eChannel</i>	the fixed channel

LOCAL_INLINE void CMP_HWA_SetCSFixedPort (CMP_Type *const *pCmp*, CMP_PortSelType *ePort*)

set channel scan fixed port

Parameters

<i>pCmp</i>	the CMP instance to use
<i>ePort</i>	the fixed port

LOCAL_INLINE void CMP_HWA_SetCSInitModulus (CMP_Type *const *pCmp*, uint8_t *u8Modulus*)

set CMP and DAC initialization delay modulus

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

LOCAL_INLINE void CMP_HWA_SetCSNSAM (CMP_Type *const *pCmp*, uint8_t *u8Nsam*)

set number of clock cycles for sampling

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u8Nsam</i>	the sampling clocks value

LOCAL_INLINE void CMP_HWA_SetCSR (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set CSR register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetCSSR (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set CSSR register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetDacData (CMP_Type *const *pCmp*, uint8_t *u8Data*)

set CMP Dac output

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u8Data</i>	the Dac data

Note

output = (VinRef / 256) * (u8Data + 1)

LOCAL_INLINE void CMP_HWA_SetDacEnableSrc (CMP_Type *const *pCmp*, CMP_DacEnableSrcType *eType*)

set DAC enable selection

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	Dac enable source

LOCAL_INLINE void CMP_HWA_SetDCR (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set DCR register

Parameters

<i>pCmp</i>	the CMP instance to use
<i>u32RegVal</i>	Register value to set

LOCAL_INLINE void CMP_HWA_SetDma (CMP_Type *const *pCmp*, bool *bEnable*)

set CMP DMA enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetEn (CMP_Type *const *pCmp*, bool *bEnable*)

set CMP enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetEnCmpOutPack (CMP_Type *const *pCmp*, bool *bEnable*)

set comparator output pin enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetEnDac (CMP_Type *const *pCmp*, bool *bEnable*)

set CMP DAC enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetEnEventCloseWin (CMP_Type *const *pCmp*, bool *bEnable*)

WINDOW signal can or not be closed by CMPO event when window mode.

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetEnLPMode (CMP_Type *const *pCmp*, bool *bEnable*)

STOP mode enable.

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetEnWinSampleInvert (CMP_Type *const *pCmp*, bool *bEnable*)

set invert the WINDOW/SAMPLE signal enable or not

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetEventCloseWin (CMP_Type *const *pCmp*, CMP_EventType *eType*)

set which CMPO event causes window close

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMPO event type

LOCAL_INLINE void CMP_HWA_SetHystCtrl (CMP_Type *const *pCmp*, CMP_HystCtrlType *eType*)

set Comparator hard block hysteresis control

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMP hysteresis control type

LOCAL_INLINE void CMP_HWA_SetIER (CMP_Type *const *pCmp*, uint32_t *u32RegVal*)

set IER register

Parameters

<i>pCmp</i>	the CMP instance to use
-------------	-------------------------

<i>u32RegVal</i>	Register value to set
------------------	-----------------------

LOCAL_INLINE void CMP_HWA_SetINNSel (CMP_Type *const *pCmp*, CMP_INSrcSelType *eType*)

set the input to the negative port of the comparator

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMP negative input source type(analog mux,dac)

LOCAL_INLINE void CMP_HWA_SetINPSel (CMP_Type *const *pCmp*, CMP_INSrcSelType *eType*)

set the input to the positive port of the comparator

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMP positive input source type(analog mux,dac)

LOCAL_INLINE void CMP_HWA_SetIntEn_ChannelScan (CMP_Type *const *pCmp*, bool *bEnable*)

set comparator channel scan interrupt enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetIntEn_Falling (CMP_Type *const *pCmp*, bool *bEnable*)

set comparator falling interrupt enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetIntEn_Rising (CMP_Type *const *pCmp*, bool *bEnable*)

set comparator rising interrupt enable

Parameters

<i>pCmp</i>	the CMP instance to use
<i>bEnable</i>	enable/disable flag

LOCAL_INLINE void CMP_HWA_SetNSelMux (CMP_Type *const *pCmp*, CMP_MuxSelType *eType*)

set which input is selected for the negative mux

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMP negative mux type

LOCAL_INLINE void CMP_HWA_SetPSelMux (CMP_Type *const *pCmp*, CMP_MuxSelType *eType*)

set which input is selected for the positive mux

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMP positive mux type

LOCAL_INLINE void CMP_HWA_SetSpeedMode (CMP_Type *const *pCmp*, CMP_SpeedModeSelType *eMode*)

set CMP power mode select

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eMod</i>	CMP power mode

LOCAL_INLINE void CMP_HWA_SetVinRefSel (CMP_Type *const *pCmp*, CMP_DacVinRefSelType *eType*)

set DAC reference voltage source

Parameters

<i>pCmp</i>	the CMP instance to use
<i>eType</i>	CMP reference voltage source type

HwA_cmu

Collaboration diagram for HwA_cmu:Functions

LOCAL_INLINE void **CMU_HWA_SetRefWindow** (CMU_Type *const pCmu, uint32_t u32Temp)

Set Reference Window value.

LOCAL_INLINE void **CMU_HWA_SetMinCnts** (CMU_Type *const pCmu, uint32_t u32Temp)

Set Minimum Counter value.

LOCAL_INLINE uint32_t **CMU_HWA_GetMinCnts** (const CMU_Type *const pCmu)

Get Minimum Counter value.

LOCAL_INLINE void **CMU_HWA_SetMaxCnts** (CMU_Type *const pCmu, uint32_t u32Temp)

Set Maximum Counter value.

LOCAL_INLINE uint32_t **CMU_HWA_GetMaxCnts** (const CMU_Type *const pCmu)

Get Maximum Counter value.

LOCAL_INLINE uint32_t **CMU_HWA_GetCount** (const CMU_Type *const pCmu)

Get Counter value.

LOCAL_INLINE void **CMU_HWA_SetPeriodWindow** (CMU_Type *const pCmu, uint32_t u32Temp)

Set period window Counter.

LOCAL_INLINE void **CMU_HWA_SetPeriodEnable** (CMU_Type *const pCmu, bool bEnable)

Set period enable bit.

LOCAL_INLINE bool **CMU_HWA_GetPeriodEnable** (const CMU_Type *const pCmu)

Get period enable bit.

LOCAL_INLINE void **CMU_HWA_SetCtrl** (CMU_Type *const pCmu, uint32_t u32Temp)

Set control register value.

LOCAL_INLINE uint32_t **CMU_HWA_GetCtrl** (const CMU_Type *const pCmu)

Get control register value.

LOCAL_INLINE uint32_t **CMU_HWA_GetST** (const CMU_Type *const pCmu)

Get status register value.

LOCAL_INLINE void **CMU_HWA_ClsST** (CMU_Type *const pCmu)

Clear clock monitor status.

LOCAL_INLINE void **CMU_HWA_StopModeEnable** (CMU_Type *const pCmu, bool bEnable)

Enable Stop mode.

LOCAL_INLINE void **CMU_HWA_SoftwareRST** (CMU_Type *const pCmu)

Enable Software Reset.

LOCAL_INLINE bool **CMU_HWA_GetSoftwareRST** (const CMU_Type *const pCmu)

Enable Software Reset.

LOCAL_INLINE void **CMU_HWA_LPRestartEnable** (CMU_Type *const pCmu, bool bEnable)

Enable low power restart mode.

Detailed Description

Function Documentation

LOCAL_INLINE void CMU_HWA_ClsST (CMU_Type *const *pCmu*)

Clear clock monitor status.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

LOCAL_INLINE uint32_t CMU_HWA_GetCount (const CMU_Type *const *pCmu*)

Get Counter value.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

Returns

Counter value.

LOCAL_INLINE uint32_t CMU_HWA_GetCtrl (const CMU_Type *const *pCmu*)

Get control register value.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

Returns

Control register value.

LOCAL_INLINE uint32_t CMU_HWA_GetMaxCnts (const CMU_Type *const *pCmu*)

Get Maximun Counter value.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

Returns

Max count value.

LOCAL_INLINE uint32_t CMU_HWA_GetMinCnts (const CMU_Type *const *pCmu*)

Get Minimun Counter value.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

Returns

Min count value.

LOCAL_INLINE bool CMU_HWA_GetPeriodEnable (const CMU_Type *const *pCmu*)

Get period enble bit.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

Returns

Period mode enable bit.

LOCAL_INLINE bool CMU_HWA_GetSoftwareRST (const CMU_Type *const *pCmu*)

Enable Software Reset.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

Returns

Software reset is done.

LOCAL_INLINE uint32_t CMU_HWA_GetST (const CMU_Type *const *pCmu*)

Get status register value.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

Returns

Status register value.

LOCAL_INLINE void CMU_HWA_LPRestartEnable (CMU_Type *const *pCmu*, bool *bEnable*)

Enable low power restart mode.

Parameters

<i>pCmu</i>	CMU Instance.
<i>bEnable</i>	Enable mode.

LOCAL_INLINE void CMU_HWA_SetCtrl (CMU_Type *const *pCmu*, uint32_t *u32Temp*)

Set control register value.

Parameters

<i>pCmu</i>	CMU Instance.
<i>u32Temp</i>	Control value.

LOCAL_INLINE void CMU_HWA_SetMaxCnts (CMU_Type *const *pCmu*, uint32_t *u32Temp*)

Set Maximun Counter value.

Parameters

<i>pCmu</i>	CMU Instance.
<i>u32Temp</i>	Max count value.

LOCAL_INLINE void CMU_HWA_SetMinCnts (CMU_Type *const *pCmu*, uint32_t *u32Temp*)

Set Minimum Counter value.

Parameters

<i>pCmu</i>	CMU Instance.
<i>u32Temp</i>	Min Count value.

LOCAL_INLINE void CMU_HWA_SetPeriodEnable (CMU_Type *const *pCmu*, bool *bEnable*)

Set period enable bit.

Parameters

<i>pCmu</i>	CMU Instance.
<i>bEnable</i>	Set enable bit.

LOCAL_INLINE void CMU_HWA_SetPeriodWindow (CMU_Type *const *pCmu*, uint32_t *u32Temp*)

Set period window Counter.

Parameters

<i>pCmu</i>	CMU Instance.
<i>u32Temp</i>	Period value.

LOCAL_INLINE void CMU_HWA_SetRefWindow (CMU_Type *const *pCmu*, uint32_t *u32Temp*)

Set Reference Window value.

Parameters

<i>pCmu</i>	CMU Instance.
<i>u32Temp</i>	Ref Window value.

LOCAL_INLINE void CMU_HWA_SoftwareRST (CMU_Type *const *pCmu*)

Enable Software Reset.

Parameters

<i>pCmu</i>	CMU Instance.
-------------	---------------

LOCAL_INLINE void CMU_HWA_StopModeEnable (CMU_Type *const *pCmu*, bool *bEnable*)

Enable Stop mode.

Parameters

<i>pCmu</i>	CMU Instance.
<i>bEnable</i>	Enable mode.

HwA_cordic

Collaboration diagram for HwA_cordic:Macros

```
#define CORDIC_CTR_VAL(s, a, b, c, d) (CORDIC_CTRL_SCALE(s) | CORDIC_CTRL_IE(a) |  
CORDIC_CTRL_ITER(b) | CORDIC_CTRL_OS(c) | CORDIC_CTRL_MODE(d))
```

Enumerations

enum **CORDIC_IterationType** { **CORDIC_Iteration_8** = 0, **CORDIC_Iteration_16**, **CORDIC_Iteration_24** }
CORDIC Iteration Type.

enum **CORDIC_SystemType** { **CORDIC_Trigonometric** = 0, **CORDIC_Hyperbolic**, **CORDIC_Linear** }
CORDIC System Type.

enum **CORDIC_ModeType** { **CORDIC_Rotate** = 0, **CORDIC_Vector** }
CORDIC Mode Type.

Functions

LOCAL_INLINE void Cordic_HWA_SetCtrl (CORDIC_Type *const pCordic, uint32_t u32Value)
Set CORDIC module Control register.

LOCAL_INLINE uint32_t Cordic_HWA_GetCtrl (CORDIC_Type *const pCordic)
Read CORDIC module Control register.

LOCAL_INLINE void **Cordic_HWA_Set_XInput** (CORDIC_Type *const pCordic, int32_t u32Value)

Set CORDIC module X Input register.

LOCAL_INLINE void **Cordic_HWA_Set_YInput** (CORDIC_Type *const pCordic, int32_t u32Value)

Set CORDIC module Y Input register.

LOCAL_INLINE void **Cordic_HWA_Set_ZInput** (CORDIC_Type *const pCordic, int32_t u32Value)

Set CORDIC module Z Input register.

LOCAL_INLINE int32_t **Cordic_HWA_Get_XOutput** (CORDIC_Type *const pCordic)

Read CORDIC module X Output register.

LOCAL_INLINE int32_t **Cordic_HWA_Get_YOutput** (CORDIC_Type *const pCordic)

Read CORDIC module Y Output register.

LOCAL_INLINE int32_t **Cordic_HWA_Get_ZOutput** (CORDIC_Type *const pCordic)

Read CORDIC module Z Output register.

LOCAL_INLINE bool **Cordic_HWA_Get_Stat** (CORDIC_Type *const pCordic)

Read CORDIC module State.

LOCAL_INLINE void **Cordic_HWA_Clear_Stat** (CORDIC_Type *const pCordic)

Clear CORDIC module done Flag.

Detailed Description

Macro Definition Documentation

```
#define CORDIC_CTRL_VAL(s, a, b, c, d) (CORDIC_CTRL_SCALE(s) | CORDIC_CTRL_IE(a) |  
CORDIC_CTRL_ITER(b) | CORDIC_CTRL_OS(c) | CORDIC_CTRL_MODE(d))
```

Enumeration Type Documentation

enum CORDIC_IterationType

CORDIC Iteration Type.

Defines the number of iterations for the CORDIC algorithm.

Enumerator:

CORDIC_Iteratio n_8	
CORDIC_Iteratio n_16	
CORDIC_Iteratio n_24	

enum CORDIC_ModeType

CORDIC Mode Type.

Specifies the operation mode for the CORDIC algorithm.

Enumerator:

CORDIC_Rotate	
CORDIC_Vector	

enum CORDIC_SystemType

CORDIC System Type.

Specifies the system type for the CORDIC algorithm.

Enumerator:

CORDIC_Trigonometric	
CORDIC_Hyperbolic	
CORDIC_Linear	

Function Documentation

LOCAL_INLINE void Cordic_HWA_Clear_Stat (CORDIC_Type *const *pCordic*)

Clear CORDIC module done Flag.

Reads the current state of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
----------------	-------------------------------

LOCAL_INLINE bool Cordic_HWA_Get_Stat (CORDIC_Type *const *pCordic*)

Read CORDIC module State.

Reads the current state of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
----------------	-------------------------------

Returns

Current state of the CORDIC module.

LOCAL_INLINE int32_t Cordic_HWA_Get_XOutput (CORDIC_Type *const *pCordic*)

Read CORDIC module X Output register.

Reads the X output register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
----------------	-------------------------------

Returns

X output register value.

LOCAL_INLINE int32_t Cordic_HWA_Get_YOutput (CORDIC_Type *const *pCordic*)

Read CORDIC module Y Output register.

Reads the Y output register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
----------------	-------------------------------

Returns

Y output register value.

LOCAL_INLINE int32_t Cordic_HWA_Get_ZOutput (CORDIC_Type *const *pCordic*)

Read CORDIC module Z Output register.

Reads the Z output register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
----------------	-------------------------------

Returns

Z output register value.

LOCAL_INLINE uint32_t Cordic_HWA_GetCtrl (CORDIC_Type *const *pCordic*)

Read CORDIC module Control register.

Reads the control register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
----------------	-------------------------------

Returns

Control register value.

LOCAL_INLINE void Cordic_HWA_Set_XInput (CORDIC_Type *const *pCordic*, int32_t *u32Value*)

Set CORDIC module X Input register.

Sets the X input register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
<i>u32Value</i>	X input register value to set.

LOCAL_INLINE void Cordic_HWA_Set_YInput (CORDIC_Type *const *pCordic*, int32_t *u32Value*)

Set CORDIC module Y Input register.

Sets the Y input register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
<i>u32Value</i>	Y input register value to set.

LOCAL_INLINE void Cordic_HWA_Set_ZInput (CORDIC_Type *const *pCordic*, int32_t *u32Value*)

Set CORDIC module Z Input register.

Sets the Z input register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
<i>u32Value</i>	Z input register value to set.

LOCAL_INLINE void Cordic_HWA_SetCtrl (CORDIC_Type *const *pCordic*, uint32_t *u32Value*)

Set CORDIC module Control register.

Sets the control register value of the CORDIC module.

Parameters

<i>pCordic</i>	Pointer to the CORDIC module.
<i>u32Value</i>	Control register value to set.

HwA_cpm

Collaboration diagram for HwA_cpm:Functions

LOCAL_INLINE uint32_t CPM_HWA_GetFiscr (CPM_Type *const pCpm)

Get the value of CPM FISCER.

LOCAL_INLINE bool **CPM_HWA_GetFpuFiocFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FIOC value.

LOCAL_INLINE bool **CPM_HWA_GetFpuFdzcFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FDZC value.

LOCAL_INLINE bool **CPM_HWA_GetFpuFofcFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FOFC value.

LOCAL_INLINE bool **CPM_HWA_GetFpuFufcFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FUFC value.

LOCAL_INLINE bool **CPM_HWA_GetFpuFixcFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FIXC value.

LOCAL_INLINE bool **CPM_HWA_GetFpuFidcFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FIDC value.

LOCAL_INLINE void **CPM_HWA_SetFiocInt** (CPM_Type *const pCpm, bool bEnable)

Set FIOCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFdzcInt** (CPM_Type *const pCpm, bool bEnable)

Set FDZCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFofcInt** (CPM_Type *const pCpm, bool bEnable)

Set FOFCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFufcInt** (CPM_Type *const pCpm, bool bEnable)

Set FUFCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFixcInt** (CPM_Type *const pCpm, bool bEnable)

Set FIXCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFidceInt** (CPM_Type *const pCpm, bool bEnable)
Set FIDCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFiscr** (CPM_Type *const pCpm, uint32_t u32Val)
Set FISCR value.

LOCAL_INLINE uint32_t **CPM_HWA_GetCoreIDValve** (CPM_Type *const pCpm)
Return CPM_CoreID value.

Detailed Description

Function Documentation

LOCAL_INLINE uint32_t **CPM_HWA_GetCoreIDValve** (CPM_Type *const *pCpm*)

Return CPM_CoreID value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

uint32_t the core ID value

LOCAL_INLINE uint32_t **CPM_HWA_GetFiscr** (CPM_Type *const *pCpm*)

Get the value of CPM FISCR.

This function returns FISCR value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

uint32_t the value of the FISCRegister.

LOCAL_INLINE bool CPM_HWA_GetFpuFdzcFlag (CPM_Type *const pCpm)

Return CPM_FISCRegister FDZC value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

0: No interrupt; 1: Interrupt occurred

LOCAL_INLINE bool CPM_HWA_GetFpuFidcFlag (CPM_Type *const pCpm)

Return CPM_FISCRegister FIDC value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

0: No interrupt; 1: Interrupt occurred

LOCAL_INLINE bool CPM_HWA_GetFpuFiocFlag (CPM_Type *const pCpm)

Return CPM_FISCRegister FIOC value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

0: No interrupt; 1: Interrupt occurred

LOCAL_INLINE bool CPM_HWA_GetFpuFixcFlag (CPM_Type *const pCpm)

Return CPM_FISCRegister FIXC value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

0: No interrupt; 1: Interrupt occurred

LOCAL_INLINE bool CPM_HWA_GetFpuFofcFlag (CPM_Type *const *pCpm*)

Return CPM_FISCR FOFC value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

0: No interrupt; 1: Interrupt occurred

LOCAL_INLINE bool CPM_HWA_GetFpuFufcFlag (CPM_Type *const *pCpm*)

Return CPM_FISCR FUFc value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
-------------	---------------------------------------

Returns

0: No interrupt; 1: Interrupt occurred

LOCAL_INLINE void CPM_HWA_SetFdzcInt (CPM_Type *const *pCpm*, bool *bEnable*)

Set FDZCE interrupt.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
<i>bEnable</i>	1: enable interrupt 0: disable interrupt

LOCAL_INLINE void CPM_HWA_SetFidceInt (CPM_Type *const *pCpm*, bool *bEnable*)

Set FIDCE interrupt.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
<i>bEnable</i>	1: enable interrupt 0: disable interrupt

LOCAL_INLINE void CPM_HWA_SetFioceInt (CPM_Type *const *pCpm*, bool *bEnable*)

Set FIOCE interrupt.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
<i>bEnable</i>	1: enable interrupt 0: disable interrupt

LOCAL_INLINE void CPM_HWA_SetFiscr (CPM_Type *const *pCpm*, uint32_t *u32Val*)

Set FISCR value.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
<i>u32Val</i>	the value want to set the register

LOCAL_INLINE void CPM_HWA_SetFixceInt (CPM_Type *const *pCpm*, bool *bEnable*)

Set FIXCE interrupt.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
<i>bEnable</i>	1: enable interrupt 0: disable interrupt

LOCAL_INLINE void CPM_HWA_SetFofceInt (CPM_Type *const *pCpm*, bool *bEnable*)

Set FOFCE interrupt.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
<i>bEnable</i>	1: enable interrupt 0: disable interrupt

LOCAL_INLINE void CPM_HWA_SetFufcelnt (CPM_Type *const *pCpm*, bool *bEnable*)

Set FUFCE interrupt.

Parameters

<i>pCpm</i>	the base address of the CPM instance.
<i>bEnable</i>	1: enable interrupt 0: disable interrupt

HwA_crc

Collaboration diagram for HwA_crc:Enumerations

enum **CRC_WriteDataSwapType** { **WRITE_DATASWAP_NONE** = 0U, **WRITE_DATASWAP_BIT** = 1U, **WRITE_DATASWAP_BIT_BYTE** = 2U, **WRITE_DATASWAP_BYTE** = 3U }

The data swap for write

enum **CRC_ReadDataSwapType** { **READ_DATASWAP_NONE** = 0U, **READ_DATASWAP_BIT** = 1U, **READ_DATASWAP_BIT_BYTE** = 2U, **READ_DATASWAP_BYTE** = 3U }

The data swap for read.

enum **CRC_ReadDataFXORType** { **READ_DATA_NORMAL** = 0U, **READ_DATA_FXOR** = 1U }

The complement of reading crc data.

enum **CRC_WriteCommondType** { **WRITE_COMMAND_DATA** = 0U, **WRITE_COMMAND_SEED** = 1U }

The command type of write crc data or seed value.

enum **CRC_BitWidthType** { **CRC_BIT_16** = 0U, **CRC_BIT_32** = 1U, **CRC_BIT_8** = 2U, **CRC_BIT_INVALID** = 3U }

The crc mode select.

Functions

LOCAL_INLINE void **CRC_HWA_SetDataOrSeed** (CRC_Type *const pCrc, **CRC_WriteCommondType** u32Mod)

set CRC CR register for writing data or seed

LOCAL_INLINE void **CRC_HWA_SetPolyVal** (CRC_Type *const pCrc, uint32_t u32Poly)

set polynomial value

LOCAL_INLINE void **CRC_HWA_SetData_U32** (CRC_Type *const pCrc, uint32_t u32Data)

set data register(32 bits)

LOCAL_INLINE void **CRC_HWA_SetData_U16** (CRC_Type *const pCrc, uint16_t u16Data)

set data register(16 bits)

LOCAL_INLINE void **CRC_HWA_SetData_U8** (CRC_Type *const pCrc, uint8_t u8Data)

set data register(low 8 bits)

LOCAL_INLINE uint32_t **CRC_HWA_GetData_U32** (CRC_Type *const pCrc)

get data register(32 bits)

LOCAL_INLINE uint16_t **CRC_HWA_GetData_U16_H** (CRC_Type *const pCrc)

get data register(high 16 bits)

LOCAL_INLINE uint16_t **CRC_HWA_GetData_U16_L** (CRC_Type *const pCrc)

get data register(low 16 bits)

LOCAL_INLINE void **CRC_HWA_SetWriteDataSwap** (CRC_Type *const pCrc, **CRC_WriteDataSwapType** eWrDataSwap)

set data swap for writes

LOCAL_INLINE void **CRC_HWA_SetReadDataSwap** (CRC_Type *const pCrc, **CRC_ReadDataSwapType** eRdDataSwap)

set data swap for read

LOCAL_INLINE void **CRC_HWA_SetReadDataFXOR** (CRC_Type *const pCrc, **CRC_ReadDataFXORType** eRdDataFXOR)

set complement read Of CRC data

LOCAL_INLINE void **CRC_HWA_SetBitWidth** (CRC_Type *const pCrc, **CRC_BitWidthType** eWidth)

set width of CRC protocol

LOCAL_INLINE **CRC_BitWidthType** **CRC_HWA_GetBitWidth** (CRC_Type *const pCrc)

get width of CRC protocol

LOCAL_INLINE **CRC_ReadDataSwapType** **CRC_HWA_GetReadDataSwap** (CRC_Type *const pCrc)

get data swap type for read

Detailed Description

Enumeration Type Documentation

enum CRC_BitWidthType

The crc mode select.

Enumerator:

CRC_BIT_16	crc 16 bit is selected
CRC_BIT_32	crc 32 bit is selected
CRC_BIT_8	crc 8 bit is selected

CRC_BIT_INVALID	
-----------------	--

enum CRC_ReadDataFXORType

The complement of reading crc data.

Enumerator:

READ_DATA_NO RMAL	none complement of reading crc data
READ_DATA_FX OR	Invert or complement with 0xFFFFFFFF or 0xFFFF of crc data

enum CRC_ReadDataSwapType

The data swap for read.

Enumerator:

READ_DATASWA P_NONE	none data swap for read
READ_DATASWA P_BIT	only bits data swap for read
READ_DATASWA P_BIT_BYTE	both bits and bytes data swap for read
READ_DATASWA P_BYTE	only bytes data swap for read

enum CRC_WriteCommandType

The command type of write crc data or seed value.

Enumerator:

WRITE_COMMAND DATA	write crc data
-----------------------	----------------

WRITE_COMMA ND_SEED	write seed value(used to initialization crc calculation)

enum CRC_WriteDataSwapType

The data swap for write

Enumerator:

WRITE_DATASW AP_NONE	none data swap for write
WRITE_DATASW AP_BIT	only bits data swap for write
WRITE_DATASW AP_BIT_BYTE	both bits and bytes data swap for write
WRITE_DATASW AP_BYTE	only bytes data swap for write

Function Documentation

LOCAL_INLINE CRC_BitWidthType CRC_HWA_GetBitWidth (CRC_Type *const *pCrc*)

get width of CRC protocol

Parameters

<i>pCrc</i>	CRC instance
-------------	--------------

LOCAL_INLINE uint16_t CRC_HWA_GetData_U16_H (CRC_Type *const *pCrc*)

get data register(high 16 bits)

Parameters

<i>pCrc</i>	CRC instance
-------------	--------------

Returns

high 16-bit value

LOCAL_INLINE uint16_t CRC_HWA_GetData_U16_L (CRC_Type *const *pCrc*)

get data register(low 16 bits)

Parameters

<i>pCrc</i>	CRC instance
-------------	--------------

Returns

low 16-bit value

LOCAL_INLINE uint32_t CRC_HWA_GetData_U32 (CRC_Type *const *pCrc*)

get data register(32 bits)

Parameters

<i>pCrc</i>	CRC instance
-------------	--------------

Returns

32-bit value

LOCAL_INLINE CRC_ReadDataSwapType CRC_HWA_GetReadDataSwap (CRC_Type *const *pCrc*)

get data swap type for read

Parameters

<i>pCrc</i>	CRC instance
-------------	--------------

LOCAL_INLINE void CRC_HWA_SetBitWidth (CRC_Type *const *pCrc*, CRC_BitWidthType *eWidth*)

set width of CRC protocol

Parameters

<i>pCrc</i>	CRC instance
<i>eWidth</i>	the CRC_BitWidthType type

LOCAL_INLINE void CRC_HWA_SetData_U16 (CRC_Type *const *pCrc*, uint16_t *u16Data*)

set data register(16 bits)

Parameters

<i>pCrc</i>	CRC instance
<i>u16Data</i>	data to be set

LOCAL_INLINE void CRC_HWA_SetData_U32 (CRC_Type *const *pCrc*, uint32_t *u32Data*)

set data register(32 bits)

Parameters

<i>pCrc</i>	CRC instance
<i>u32Data</i>	data to be set

LOCAL_INLINE void CRC_HWA_SetData_U8 (CRC_Type *const *pCrc*, uint8_t *u8Data*)

set data register(low 8 bits)

Parameters

<i>pCrc</i>	CRC instance
<i>u8Data</i>	data to be set

LOCAL_INLINE void CRC_HWA_SetDataOrSeed (CRC_Type *const *pCrc*, CRC_WriteCommondType *u32Mod*)

set CRC CR register for writing data or seed

Parameters

<i>pCrc</i>	CRC instance
<i>u32Mod</i>	WAS mode

LOCAL_INLINE void CRC_HWA_SetPolyVal (CRC_Type *const *pCrc*, uint32_t *u32Poly*)

set polynomial value

Parameters

<i>pCrc</i>	CRC instance
<i>u32Poly</i>	the polynomial value

LOCAL_INLINE void CRC_HWA_SetReadDataFXOR (CRC_Type *const *pCrc*, CRC_ReadDataFXORType *eRdDataFXOR*)

set complement read Of CRC data

Parameters

<i>pCrc</i>	CRC instance
<i>eRdDataFXOR</i>	the CRC_ReadDataFXORType type

LOCAL_INLINE void CRC_HWA_SetReadDataSwap (CRC_Type *const *pCrc*, CRC_ReadDataSwapType *eRdDataSwap*)

set data swap for read

Parameters

<i>pCrc</i>	CRC instance
<i>eRdDataSwap</i>	the eRdDataSwap type

LOCAL_INLINE void CRC_HWA_SetWriteDataSwap (CRC_Type *const *pCrc*, CRC_WriteDataSwapType *eWrDataSwap*)

set data swap for writes

Parameters

<i>pCrc</i>	CRC instance
<i>eWrDataSwap</i>	the CRC_WriteDataSwapType type

HwA_csc

Collaboration diagram for HwA_csc:Enumerations

enum **CSC0_ClockOutDivType** { **CSC0_CLKOUT_DIV_BY1** = 0U, **CSC0_CLKOUT_DIV_BY2** = 1U, **CSC0_CLKOUT_DIV_BY3** = 2U, **CSC0_CLKOUT_DIV_BY4** = 3U, **CSC0_CLKOUT_DIV_BY5** = 4U, **CSC0_CLKOUT_DIV_BY6** = 5U, **CSC0_CLKOUT_DIV_BY7** = 6U, **CSC0_CLKOUT_DIV_BY8** = 7U }

CSC0 clock out divide ratio type.

enum **CSC0_ClockOutSrcType** { **CSC0_CLKOUT_SCG_CLKOUT** = 0U, **CSC0_CLKOUT_FOSC_DIVM_CLK** = 2U, **CSC0_CLKOUT_SLOW_CLK** = 3U, **CSC0_CLKOUT_SIRC_DIVM_CLK** = 4U, **CSC0_CLKOUT_PLL1_DIVM_CLK** = 5U, **CSC0_CLKOUT_FIRC_DIVM_CLK** = 6U, **CSC0_CLKOUT_CORE_CLK** = 7U, **CSC0_CLKOUT_PLL0_DIVM_CLK** = 8U, **CSC0_CLKOUT_BUS_CLK** = 9U, **CSC0_CLKOUT_SIRC_128K_CLK** = 10U, **CSC0_CLKOUT_AON_CLK** = 12U, **CSC0_CLKOUT_RTC_CLK** = 14U }

CSC0 clock out source type.

enum **CSC0_AON32KClkSrcType** { **CSC0_AON32K_SIRCDIV_32K_CLK** = 1U, **CSC0_AON32K_SOSC32K_CLK** = 2U, **CSC0_AON32K_SIRC32K_CLK** = 3U }

Data type for CSC0_AONCLKSR[32KAONCLKSEL], set AON32KCLK source clock.

enum **CSC0_RTCClkSrcType** { **CSC0_RTC_FOSCDIVL_CLK** = 0U, **CSC0_RTC_SIRCDIV_32K_CLK** = 1U, **CSC0_RTC_SOSC_CLK** = 2U, **CSC0_RTC_SIRC32K_CLK** = 3U }

Data type for CSC0_RTCCLKSEL[RTCCLKSEL], set RTCCLK source clock.

enum **CSC0_AONClkSrcType** { **CSC0_AON_SIRCDIV_128K_CLK** = 0U, **CSC0_AON_SIRC32K_CLK** = 1U, **CSC0_AON_SIRCDIV_32K_CLK** = 2U, **CSC0_AON_SIRC32_1K_CLK** = 3U }

Data type for CSC0_AONCLKSEL[AONCLKSEL], set AONCLK source clock.

Functions

LOCAL_INLINE void **CSC0_HWA_Set_STOP_MODER0** (uint32_t u32val)

Set stop mode register 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_MODER0** (void)

Get stop mode register 0.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_REQR0** (uint32_t u32val)

Set stop request register 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_REQR0** (void)

Get stop request register 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_ACKR0** (void)

Get stop ack status in register 0.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_MODER1** (uint32_t u32val)

Set stop mode register 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_MODER1** (void)

Get stop mode register 1.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_REQR1** (uint32_t u32val)

Set stop request register 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_REQR1** (void)

Get stop request register 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_ACKR1** (void)

Get stop ack status in register 1.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_MODER2** (uint32_t u32val)

Set stop mode register 2.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_MODER2** (void)

Get stop mode register 2.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_REQR2** (uint32_t u32val)

Set stop request register 2.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_REQR2** (void)

Get stop request register 2.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_ACKR2** (void)

Get stop ack status in register 2.

LOCAL_INLINE void **CSC0_HWA_Set_CCM0_CFG** (uint32_t u32val)

Set CCM0 configure register.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_CCM0_CFG** (void)

Get CCM0 configure.

LOCAL_INLINE void **CSC0_HWA_Set_SCG_MAM_STALL** (uint32_t u32val)

Set SCG MAM stall request register.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SCG_MAM_STALL** (void)

Get SCG MAM stall request register.

LOCAL_INLINE void **CSC0_HWA_Set_CPU0_INT** (uint32_t u32val)

Set cpu0 software interrupt register.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_CPU0_INT** (void)

Get cpu0 software interrupt register.

LOCAL_INLINE void **CSC0_HWA_EnableCpu0SWInterrupt** (void)

Enable cpu0 software interrupt.

LOCAL_INLINE void **CSC0_HWA_DisableCpu0SWInterrupt** (void)

Disable cpu0 software interrupt.

LOCAL_INLINE void **CSC0_HWA_Set_SMU_CTRL0** (uint32_t u32Value)

Set multi CSC_SMU control group 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SMU_CTRL0** (void)

Get CSC0_SMU_CTRL0 register status.

LOCAL_INLINE void **CSC0_HWA_Set_SMU_CTRL1** (uint32_t u32Value)

Set multi CSC_SMU control group 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SMU_CTRL1** (void)

Get CSC0_SMU_CTRL1 register status.

LOCAL_INLINE void **CSC0_HWA_Set_SMU_CTRL4** (uint32_t u32Value)

Set multi CSC_SMU control group 4.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SMU_CTRL4** (void)

Get CSC0_SMU_CTRL4 register status.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_MODER0** (uint32_t u32val)

Set stop mode register 0.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_MODER0** (void)

Get stop mode register 0.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_REQR0** (uint32_t u32val)

Set stop request register 0.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_REQR0** (void)

Get stop request register 0.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_ACKR0** (void)

Get stop ack status in register 0.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_MODER1** (uint32_t u32val)

Set stop mode register 1.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_MODER1** (void)

Get stop mode register 1.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_REQR1** (uint32_t u32val)

Set stop request register 1.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_REQR1** (void)

Get stop request register 1.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_ACKR1** (void)

Get stop ack status in register 1.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_MODER2** (uint32_t u32val)

Set stop mode register 2.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_MODER2** (void)

Get stop mode register 2.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_REQR2** (uint32_t u32val)

Set stop request register 2.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_REQR2** (void)

Get stop request register 2.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_ACKR2** (void)

Get stop ack status in register 2.

LOCAL_INLINE void **CSC1_HWA_Set_CCM1_CFG** (uint32_t u32val)

Set CCM0 configure register.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_CCM1_CFG** (void)

Get CCM0 configure.

LOCAL_INLINE void **CSC1_HWA_Set_CPU1_INT** (uint32_t u32val)

Set cpu0 software interrupt register.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_CPU1_INT** (void)

Get cpu1 software interrupt register.

LOCAL_INLINE void **CSC1_HWA_EnableCpu1SWInterrupt** (void)

Enable cpu1 software interrupt.

LOCAL_INLINE void **CSC1_HWA_DisableCpu1SWInterrupt** (void)

Disable cpu1 software interrupt.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_MODER0** (uint32_t u32val)

Set stop mode register 0.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_MODER0** (void)

Get stop mode register 0.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_REQR0** (uint32_t u32val)

Set stop request register 0.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_REQR0** (void)

Get stop request register 0.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_ACKR0** (void)

Get stop ack status in register 0.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_MODER1** (uint32_t u32val)

Set stop mode register 1.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_MODER1** (void)

Get stop mode register 1.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_REQR1** (uint32_t u32val)

Set stop request register 1.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_REQR1** (void)

Get stop request register 1.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_ACKR1** (void)

Get stop ack status in register 1.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_MODER2** (uint32_t u32val)

Set stop mode register 2.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_MODER2** (void)

Get stop mode register 2.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_REQR2** (uint32_t u32val)

Set stop request register 2.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_REQR2** (void)

Get stop request register 2.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_ACKR2** (void)

Get stop ack status in register 2.

LOCAL_INLINE void **CSC2_HWA_Set_CCM2_CFG** (uint32_t u32val)

Set CCM0 configure register.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_CCM2_CFG** (void)

Get CCM0 configure.

LOCAL_INLINE void **CSC2_HWA_Set_CPU2_INT** (uint32_t u32val)

Set cpu0 software interrupt register.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_CPU2_INT** (void)

Get cpu0 software interrupt register.

LOCAL_INLINE void **CSC2_HWA_EnableCpu2SWInterrupt** (void)

Enable cpu0 software interrupt.

LOCAL_INLINE void **CSC2_HWA_DisableCpu2SWInterrupt** (void)

Disable cpu0 software interrupt.

Detailed Description

Enumeration Type Documentation

enum CSC0_AON32KClkSrcType

Data type for CSC0_AONCLKSR[32KAONCLKSEL], set AON32CLK source clock.

Enumerator:

CSC0_AON32K_ SIRCDIV_32K_C LK	CSC0_AONCLKSR[32KAONCLKSEL], SIRCDIV_32K
CSC0_AON32K_ SOSC32K_CLK	CSC0_AONCLKSR[32KAONCLKSEL], SOSC32K
CSC0_AON32K_ SIRC32K_CLK	CSC0_AONCLKSR[32KAONCLKSEL], SIRC32K

enum CSC0_AONClkSrcType

Data type for CSC0_AONCLKSEL[AONCLKSEL], set AONCLK source clock.

Enumerator:

CSC0_AON_SIR CDIV_128K_CLK	CSC0_AONCLKSEL[AONCLKSEL], SIRCDIV_128K
CSC0_AON_SIR C32K_CLK	CSC0_AONCLKSEL[AONCLKSEL], SIR32K
CSC0_AON_SIR CDIV_32K_CLK	CSC0_AONCLKSEL[AONCLKSEL], SIRCDIV_32K
CSC0_AON_SIR C32_1K_CLK	CSC0_AONCLKSEL[AONCLKSEL], SIR32_1K

enum CSC0_ClockOutDivType

CSC0 clock out divide ratio type.

Enumerator:

CSC0_CLKOUT_ DIV_BY1	Divided by 1
CSC0_CLKOUT_ DIV_BY2	Divided by 2
CSC0_CLKOUT_ DIV_BY3	Divided by 3
CSC0_CLKOUT_ DIV_BY4	Divided by 4
CSC0_CLKOUT_ DIV_BY5	Divided by 5
CSC0_CLKOUT_ DIV_BY6	Divided by 6
CSC0_CLKOUT_ DIV_BY7	Divided by 7

CSC0_CLKOUT_ DIV_BY8	Divided by 8
-------------------------	--------------

enum CSC0_ClockOutSrcType

CSC0 clock out source type.

Enumerator:

CSC0_CLKOUT_ SCG_CLKOUT	SCG CLKOUT
CSC0_CLKOUT_ FOSC_DIVM_CLK K	FOSC DIVM CLK
CSC0_CLKOUT_ SLOW_CLK	SLOW CLK
CSC0_CLKOUT_ SIRC_DIVM_CLK	SIRC DIVM CLK
CSC0_CLKOUT_ PLL1_DIVM_CLK	PLL1 DIVM CLK
CSC0_CLKOUT_ FIRC_DIVM_CLK	FIRC DIVM CLK
CSC0_CLKOUT_ CORE_CLK	CORE CLK
CSC0_CLKOUT_ PLL0_DIVM_CLK	PLL0 DIVM CLK
CSC0_CLKOUT_ BUS_CLK	BUS CLK
CSC0_CLKOUT_ SIRC_128K_CLK	SIRC 128K CLK
CSC0_CLKOUT_ AON_CLK	AON CLK

CSC0_CLKOUT_ RTC_CLK	AON32K CLK
-------------------------	------------

enum CSC0_RTCClkSrcType

Data type for CSC0_RTCCLKSEL[RTCCLKSEL], set RTCCLK source clock.

Enumerator:

CSC0_RTC_FOS CDIVL_CLK	CSC0_RTCCLKSEL[RTCCLKSEL], FOSC_DIVL
CSC0_RTC_SIRC DIV_32K_CLK	CSC0_RTCCLKSEL[RTCCLKSEL], SIRC_DIV
CSC0_RTC_SOSC C_CLK	CSC0_RTCCLKSEL[RTCCLKSEL], SOSC
CSC0_RTC_SIRC 32K_CLK	CSC0_RTCCLKSEL[RTCCLKSEL], SIRC_32K

Function Documentation

LOCAL_INLINE void CSC0_HWA_DisableCpu0SWInterrupt (void)

Disable cpu0 software interrupt.

LOCAL_INLINE void CSC0_HWA_EnableCpu0SWInterrupt (void)

Enable cpu0 software interrupt.

LOCAL_INLINE uint32_t CSC0_HWA_Get_CCM0_CFG (void)

Get CCM0 configure.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_CPU0_INT (void)

Get cpu0 software interrupt register.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_SCG_MAM_STALL (void)

Get SCG MAM stall request register.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_SMU_CTRL0 (void)

Get CSC0_SMU_CTRL0 register status.

Returns

register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_SMU_CTRL1 (void)

Get CSC0_SMU_CTRL1 register status.

Returns

register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_SMU_CTRL4 (void)

Get CSC0_SMU_CTRL4 register status.

Returns

register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_ACKR0 (void)

Get stop ack status in register 0.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_ACKR1 (void)

Get stop ack status in register 1.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_ACKR2 (void)

Get stop ack status in register 2.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_MODER0 (void)

Get stop mode register 0.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_MODER1 (void)

Get stop mode register 1.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_MODER2 (void)

Get stop mode register 2.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_REQR0 (void)

Get stop request register 0.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_REQR1 (void)

Get stop request register 1.

Returns

Register status

LOCAL_INLINE uint32_t CSC0_HWA_Get_STOP_REQR2 (void)

Get stop request register 2.

Returns

Register status

LOCAL_INLINE void CSC0_HWA_Set_CCM0_CFG (uint32_t *u32val*)

Set CCM0 configure register.

LOCAL_INLINE void CSC0_HWA_Set_CPU0_INT (uint32_t *u32val*)

Set cpu0 software interrupt register.

LOCAL_INLINE void CSC0_HWA_Set_SCG_MAM_STALL (uint32_t *u32val*)

Set SCG MAM stall request register.

LOCAL_INLINE void CSC0_HWA_Set_SMU_CTRL0 (uint32_t *u32Value*)

Set multi CSC_SMU control group 0.

Parameters

<i>u32Value</i>	Value to be set
-----------------	-----------------

LOCAL_INLINE void CSC0_HWA_Set_SMU_CTRL1 (uint32_t *u32Value*)

Set multi CSC_SMU control group 1.

Parameters

<i>u32Value</i>	Value to be set
-----------------	-----------------

LOCAL_INLINE void CSC0_HWA_Set_SMU_CTRL4 (uint32_t *u32Value*)

Set multi CSC_SMU control group 4.

Parameters

<i>u32Value</i>	Value to be set
-----------------	-----------------

LOCAL_INLINE void CSC0_HWA_Set_STOP_MODER0 (uint32_t *u32val*)

Set stop mode register 0.

LOCAL_INLINE void CSC0_HWA_Set_STOP_MODER1 (uint32_t *u32val*)

Set stop mode register 1.

LOCAL_INLINE void CSC0_HWA_Set_STOP_MODER2 (uint32_t *u32val*)

Set stop mode register 2.

LOCAL_INLINE void CSC0_HWA_Set_STOP_REQR0 (uint32_t *u32val*)

Set stop request register 0.

LOCAL_INLINE void CSC0_HWA_Set_STOP_REQR1 (uint32_t *u32val*)

Set stop request register 1.

LOCAL_INLINE void CSC0_HWA_Set_STOP_REQR2 (uint32_t *u32val*)

Set stop request register 2.

LOCAL_INLINE void CSC1_HWA_DisableCpu1SWInterrupt (void)

Disable cpu1 software interrupt.

LOCAL_INLINE void CSC1_HWA_EnableCpu1SWInterrupt (void)

Enable cpu1 software interrupt.

LOCAL_INLINE uint32_t CSC1_HWA_Get_CCM1_CFG (void)

Get CCM0 configure.

Returns

Register status

LOCAL_INLINE uint32_t CSC1_HWA_Get_CPU1_INT (void)

Get cpu1 software interrupt register.

Returns

Register status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_ACKR0 (void)

Get stop ack status in register 0.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_ACKR1 (void)

Get stop ack status in register 1.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_ACKR2 (void)

Get stop ack status in register 2.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_MODER0 (void)

Get stop mode register 0.

Returns

Register status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_MODER1 (void)

Get stop mode register 1.

Returns

Register status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_MODER2 (void)

Get stop mode register 2.

Returns

Register status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_REQR0 (void)

Get stop request register 0.

Returns

Register status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_REQR1 (void)

Get stop request register 1.

Returns

Register status

LOCAL_INLINE uint32_t CSC1_HWA_Get_STOP_REQR2 (void)

Get stop request register 2.

Returns

Register status

LOCAL_INLINE void CSC1_HWA_Set_CCM1_CFG (uint32_t *u32val*)

Set CCM0 configure register.

LOCAL_INLINE void CSC1_HWA_Set_CPU1_INT (uint32_t *u32val*)

Set cpu0 software interrupt register.

LOCAL_INLINE void CSC1_HWA_Set_STOP_MODER0 (uint32_t *u32val*)

Set stop mode register 0.

LOCAL_INLINE void CSC1_HWA_Set_STOP_MODER1 (uint32_t *u32val*)

Set stop mode register 1.

LOCAL_INLINE void CSC1_HWA_Set_STOP_MODER2 (uint32_t *u32val*)

Set stop mode register 2.

LOCAL_INLINE void CSC1_HWA_Set_STOP_REQR0 (uint32_t *u32val*)

Set stop request register 0.

LOCAL_INLINE void CSC1_HWA_Set_STOP_REQR1 (uint32_t *u32val*)

Set stop request register 1.

LOCAL_INLINE void CSC1_HWA_Set_STOP_REQR2 (uint32_t *u32val*)

Set stop request register 2.

LOCAL_INLINE void CSC2_HWA_DisableCpu2SWInterrupt (void)

Disable cpu0 software interrupt.

LOCAL_INLINE void CSC2_HWA_EnableCpu2SWInterrupt (void)

Enable cpu0 software interrupt.

LOCAL_INLINE uint32_t CSC2_HWA_Get_CCM2_CFG (void)

Get CCM0 configure.

Returns

Register status

LOCAL_INLINE uint32_t CSC2_HWA_Get_CPU2_INT (void)

Get cpu0 software interrupt register.

Returns

Register status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_ACKR0 (void)

Get stop ack status in register 0.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_ACKR1 (void)

Get stop ack status in register 1.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_ACKR2 (void)

Get stop ack status in register 2.

Returns

Register ack status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_MODER0 (void)

Get stop mode register 0.

Returns

Register status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_MODER1 (void)

Get stop mode register 1.

Returns

Register status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_MODER2 (void)

Get stop mode register 2.

Returns

Register status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_REQR0 (void)

Get stop request register 0.

Returns

Register status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_REQR1 (void)

Get stop request register 1.

Returns

Register status

LOCAL_INLINE uint32_t CSC2_HWA_Get_STOP_REQR2 (void)

Get stop request register 2.

Returns

Register status

LOCAL_INLINE void CSC2_HWA_Set_CCM2_CFG (uint32_t *u32val*)

Set CCM0 configure register.

LOCAL_INLINE void CSC2_HWA_Set_CPU2_INT (uint32_t *u32val*)

Set cpu0 software interrupt register.

LOCAL_INLINE void CSC2_HWA_Set_STOP_MODER0 (uint32_t *u32val*)

Set stop mode register 0.

LOCAL_INLINE void CSC2_HWA_Set_STOP_MODER1 (uint32_t *u32val*)

Set stop mode register 1.

```
LOCAL_INLINE void CSC2_HWA_Set_STOP_MODER2 (uint32_t  u32val)
```

Set stop mode register 2.

```
LOCAL_INLINE void CSC2_HWA_Set_STOP_REQR0 (uint32_t  u32val)
```

Set stop request register 0.

```
LOCAL_INLINE void CSC2_HWA_Set_STOP_REQR1 (uint32_t  u32val)
```

Set stop request register 1.

```
LOCAL_INLINE void CSC2_HWA_Set_STOP_REQR2 (uint32_t  u32val)
```

Set stop request register 2.

HwA_dma

Collaboration diagram for HwA_dma:Enumerations

```
enum DMA_RunningStatusType { DMA_RUNNING_STATUS_IDLE = 0x0U,  
DMA_RUNNING_STATUS_ACTIVE = 0x1U }
```

DMA running status.

```
enum DMA_ArbitrationAlgorithmType { DMA_ARBITRATION_ALGORITHM_FIXED_PRIORITY = 0U,  
DMA_ARBITRATION_ALGORITHM_ROUND_ROBIN = 1U }
```

DMA channel arbitration algorithm used in the channel arbitration phase.

enum **DMA_TransferSizeType** { **DMA_TRANSFER_SIZE_1B** = 0x0U, **DMA_TRANSFER_SIZE_2B** = 0x1U, **DMA_TRANSFER_SIZE_4B** = 0x2U, **DMA_TRANSFER_SIZE_8B** = 0x3U, **DMA_TRANSFER_SIZE_32B** = 0x5U }

Defines the size of data in one transfer.

Functions

LOCAL_INLINE bool **DMA_HWA_GetCancelTransferStatus** (const DMA_Type *const pDma)

Get whether the DMA instance is cancelling.

LOCAL_INLINE void **DMA_HWA_CancelTransfer** (DMA_Type *const pDma)

Cancel the remaining transfer of the DMA.

LOCAL_INLINE bool **DMA_HWA_GetErrorCancelTransferStatus** (const DMA_Type *const pDma)

Get whether the DMA instance is cancelling with error.

LOCAL_INLINE void **DMA_HWA_ErrorCancelTransfer** (DMA_Type *const pDma)

Cancel the remaining transfer of the DMA and generate an error after finished cancelling.

LOCAL_INLINE bool **DMA_HWA_GetInnerLoopMappingEnableFlag** (const DMA_Type *const pDma)

Get whether inner loop mapping is enabled.

LOCAL_INLINE void **DMA_HWA_SetInnerLoopMappingEnableFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to enable inner loop mapping.

LOCAL_INLINE bool **DMA_HWA_GetContinuousTrigModeEnableFlag** (const DMA_Type *const pDma)

Get whether the continuous trig mode is enabled When continuous trig mode is enabled, channel arbitration is not used for a inner loop channel trig before being activated again. Upon inner loop completion, the channel activates again if that channel has a inner loop channel trig enabled and the trig channel is itself. This effectively applies the inner loop offsets and restarts the next inner loop.

LOCAL_INLINE void **DMA_HWA_SetContinuousTrigModeEnableFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to enable the continuous trig mode.

LOCAL_INLINE bool **DMA_HWA_GetHatIStatus** (const DMA_Type *const pDma)

Get whether the DMA is halted When the DMA is halted, it will ignore all service requests.

LOCAL_INLINE void **DMA_HWA_HaltOperations** (DMA_Type *const pDma)

Halt the DMA operations.

LOCAL_INLINE void **DMA_HWA_ClearHaltFlag** (DMA_Type *const pDma)

Clear the halt flag of the DMA instance After HALT is cleared, the DMA could continue to operate.

LOCAL_INLINE bool **DMA_HWA_GetHaltOnErrorFlag** (const DMA_Type *const pDma)

Get whether halt on error is enabled on the DMA instance.

LOCAL_INLINE void **DMA_HWA_SetHaltOnErrorFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to enable halt on error on the DMA instance.

LOCAL_INLINE **DMA_ArbitrationAlgorithmType** **DMA_HWA_GetArbitrationAlgorithm** (const DMA_Type *const pDma)

Get the DMA arbitration algorithm.

LOCAL_INLINE void **DMA_HWA_SetArbitrationAlgorithm** (DMA_Type *const pDma, **DMA_ArbitrationAlgorithmType** eAlgorithm)

Set the DMA arbitration algorithm.

LOCAL_INLINE bool **DMA_HWA_GetDebugModeStopFlag** (const DMA_Type *const pDma)

Get whether the DMA is configured to stop under debug mode.

LOCAL_INLINE uint32_t **DMA_HWA_GetControlRegister** (const DMA_Type *const pDma)

Get the control register of the DMA instance.

LOCAL_INLINE void **DMA_HWA_SetControlRegister** (DMA_Type *const pDma, uint32_t u32Settings)

Set the control register of the DMA instance.

LOCAL_INLINE void **DMA_HWA_SetDebugModeStopFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to stop DMA under debug mode.

LOCAL_INLINE uint32_t **DMA_HWA_GetErrorStatus** (const DMA_Type *const pDma)

Get the DMA error status.

LOCAL_INLINE bool **DMA_HWA_GetChannelErrorFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether there is error occurred on the specified channel.

LOCAL_INLINE uint16_t **DMA_HWA_GetAllChannelErrorFlag** (const DMA_Type *const pDma)

Get the channel error status, each bit represents a channel.

LOCAL_INLINE bool **DMA_HWA_GetChannelErrorInterruptEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether error interrupt is enabled on the specified channel.

LOCAL_INLINE uint16_t **DMA_HWA_GetAllChannelErrorInterruptEnableFlag** (const DMA_Type *const pDma)

Get channel error interrupt enable status.

LOCAL_INLINE void **DMA_HWA_EnableChannelErrorInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Enable error interrupt for the specified channel.

LOCAL_INLINE void **DMA_HWA_EnableAllChannelErrorInterrupt** (DMA_Type *const pDma)

Enable error interrupt for all channels.

LOCAL_INLINE void **DMA_HWA_DisableChannelErrorInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Disable error interrupt for the specified channel.

LOCAL_INLINE void **DMA_HWA_DisableAllChannelErrorInterrupt** (DMA_Type *const pDma)

Disable error interrupt for all channels.

LOCAL_INLINE void **DMA_HWA_EnableChannelRequest** (DMA_Type *const pDma, uint8_t u8Channel)

Enable channel request for the specified channel.

LOCAL_INLINE void **DMA_HWA_EnableAllChannelRequest** (DMA_Type *const pDma)

Enable channel request for all channels.

LOCAL_INLINE void **DMA_HWA_DisableChannelRequest** (DMA_Type *const pDma, uint8_t u8Channel)

Disable channel request for the specified channel.

LOCAL_INLINE void **DMA_HWA_DisableAllChannelRequest** (DMA_Type *const pDma)

Disable channel request for all channels.

LOCAL_INLINE void **DMA_HWA_ClearChannelDoneStatus** (DMA_Type *const pDma, uint8_t u8Channel)

Clear DONE bit for the specified channel.

LOCAL_INLINE void **DMA_HWA_ClearAllChannelDoneStatus** (DMA_Type *const pDma)

Clear DONE bit for the all channels.

LOCAL_INLINE void **DMA_HWA_ClearChannelErrorFlag** (DMA_Type *const pDma, uint8_t u8Channel)

Clear ERR bit for the specified channel.

LOCAL_INLINE void **DMA_HWA_ClearAllChannelErrorFlag** (DMA_Type *const pDma)

Clear ERR bit for the all channels.

LOCAL_INLINE void **DMA_HWA_ClearChannelInterruptFlag** (DMA_Type *const pDma, uint8_t u8Channel)

Clear interrupt flag for the specified channel.

LOCAL_INLINE void **DMA_HWA_ClearAllChannelInterruptFlag** (DMA_Type *const pDma)

Clear interrupt flag for the all channels.

LOCAL_INLINE void **DMA_HWA_SetChannelStart** (DMA_Type *const pDma, uint8_t u8Channel)

Set start for the specified channel.

LOCAL_INLINE void **DMA_HWA_SetAllChannelStart** (DMA_Type *const pDma)

Set start for the all channels.

LOCAL_INLINE bool **DMA_HWA_GetSrcUnalignModuloEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether source unalign modulo is enabled for the specified channel.

LOCAL_INLINE bool **DMA_HWA_GetDestUnalignModuloEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether destination unalign modulo is enabled for the specified channel.

LOCAL_INLINE void **DMA_HWA_SetUnalignModuloEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnableSrcModulo, bool bEnableDestModulo)

Get whether to enable unalign modulo for the specified channel.

LOCAL_INLINE uint16_t **DMA_HWA_GetSrcUnalignModulo** (const DMA_Type *const pDma, uint8_t u8Selection)

Get the source unalign modulo of the index.

LOCAL_INLINE uint16_t **DMA_HWA_GetDestUnalignModulo** (const DMA_Type *const pDma, uint8_t u8Selection)

Get the destination unalign modulo of the index.

LOCAL_INLINE void **DMA_HWA_SetUnalignModulo** (DMA_Type *const pDma, uint8_t u8Selection, uint16_t u16SrcModulo, uint16_t u16DestModulo)

Set unalign modulo of the index.

LOCAL_INLINE uint8_t **DMA_HWA_GetPriority** (const DMA_Type *const pDma, uint8_t u8Channel)

Get priority of the channel.

LOCAL_INLINE void **DMA_HWA_SetPriority** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8Priority)

Set priority of the channel.

LOCAL_INLINE uint32_t **DMA_HWA_GetSrcAddr** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the source address of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetSrcAddr** (DMA_Type *const pDma, uint8_t u8Channel, uint32_t u32SrcAddr)

Set the source address of the DMA channel.

LOCAL_INLINE uint32_t **DMA_HWA_GetDestAddr** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the destination address of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetDestAddr** (DMA_Type *const pDma, uint8_t u8Channel, uint32_t u32DestAddr)

Set the destination address of the DMA channel.

LOCAL_INLINE int16_t **DMA_HWA_GetSrcOffset** (const DMA_Type *const pDma, uint8_t u8Channel)

Get source data offset of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetSrcOffset** (DMA_Type *const pDma, uint8_t u8Channel, int16_t s16Offset)

Set source data offset of the DMA channel.

LOCAL_INLINE int16_t **DMA_HWA_GetDestOffset** (const DMA_Type *const pDma, uint8_t u8Channel)

Get destination data offset of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetDestOffset** (DMA_Type *const pDma, uint8_t u8Channel, int16_t s16Offset)

Set destination data offset of the DMA channel.

LOCAL_INLINE **DMA_TransferSizeType** **DMA_HWA_GetSrcDataSize** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the source data size of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetSrcDataSize** (DMA_Type *const pDma, uint8_t u8Channel, **DMA_TransferSizeType** eDataSize)

Set the source data size of the DMA channel.

LOCAL_INLINE **DMA_TransferSizeType DMA_HWA_GetDestDataSize** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the destination data size of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetDestDataSize** (DMA_Type *const pDma, uint8_t u8Channel, **DMA_TransferSizeType** eDataSize)

Set the destination data size of the DMA channel.

LOCAL_INLINE uint8_t **DMA_HWA_GetSrcModulo** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the source address aligned modulo of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetSrcModulo** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8SrcModulo)

Set the source address aligned modulo of the DMA channel.

LOCAL_INLINE uint8_t **DMA_HWA_GetDestModulo** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the destination address aligned modulo of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetDestModulo** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8DestModulo)

Set the destination address aligned modulo of the DMA channel.

LOCAL_INLINE int32_t **DMA_HWA_GetSrcLastAddrAdjustment** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the address adjustment applied to the source address after major loop finished.

LOCAL_INLINE void **DMA_HWA_SetSrcLastAddrAdjustment** (DMA_Type *const pDma, uint8_t u8Channel, int32_t s32LastAdj)

Set the address adjustment applied to the source address after major loop finished.

LOCAL_INLINE int32_t **DMA_HWA_GetDestLastAddrAdjustment** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the address adjustment applied to the destination address after major loop finished.

LOCAL_INLINE void **DMA_HWA_SetDestLastAddrAdjustment** (DMA_Type *const pDma, uint8_t u8Channel, int32_t s32LastAdj)

Set the address adjustment applied to the destination address after major loop finished.

LOCAL_INLINE bool **DMA_HWA_GetInnerLoopSrcOffsetEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether source address inner loop offset is enabled.

LOCAL_INLINE bool **DMA_HWA_GetInnerLoopDestOffsetEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether destination address inner loop offset is enabled.

LOCAL_INLINE int32_t **DMA_HWA_GetInnerLoopOffset** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the inner loop offset of the DMA channel.

LOCAL_INLINE uint32_t **DMA_HWA_GetInnerLoopSize** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the transfer data size of the inner loop.

LOCAL_INLINE void **DMA_HWA_SetInnerLoopOffset** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnableSrcOffset, bool bEnableDestOffset, int32_t s32Offset)

Set the inner loop offset of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetInnerLoopSize** (DMA_Type *const pDma, uint8_t u8Channel, uint32_t u32Size)

Set the transfer data size of the inner loop.

LOCAL_INLINE bool **DMA_HWA_GetCurrentChannelToChannelTrigEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether current channel to channel trig is enabled when inner loop complete.

LOCAL_INLINE uint8_t **DMA_HWA_GetCurrentChannelToChannelTrigChannel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the target channel when current channel to channel trig is enabled.

LOCAL_INLINE uint16_t **DMA_HWA_GetCurrentLoopCount** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the current loop count The current loop count is the same as the beginning loop count initially, and it will decrement each time a inner loop finishes.

LOCAL_INLINE bool **DMA_HWA_GetBeginningChannelToChannelTrigEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether beginning channel to channel trig is enabled when inner loop complete.

LOCAL_INLINE uint8_t **DMA_HWA_GetBeginningChannelToChannelTrigChannel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the target channel when beginning channel to channel trig is enabled.

LOCAL_INLINE uint16_t **DMA_HWA_GetBeginningLoopCount** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the beginning loop count This field specifies how many inner loops will be executed in a DMA transfer.

LOCAL_INLINE void **DMA_HWA_SetChannelToChannelTrig** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable, uint8_t u8TrigChannel)

Set channel to channel trig when inner loop complete.

LOCAL_INLINE void **DMA_HWA_SetLoopCount** (DMA_Type *const pDma, uint8_t u8Channel, uint16_t u16LoopCnt)

Set the loop count of the DMA transfer This field specifies how many inner loops will be executed in a DMA transfer.

LOCAL_INLINE uint8_t **DMA_HWA_GetUnalignModuloSel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the unalign modulo index selection.

LOCAL_INLINE void **DMA_HWA_SetUnalignModuloSel** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8UnalignModuloSel)

Set the unalign modulo index selection.

LOCAL_INLINE uint8_t **DMA_HWA_GetOuterLoopTrigChannel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the target channel to trig when outer loop is completed.

LOCAL_INLINE void **DMA_HWA_SetOuterLoopTrigChannel** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8TrigChannel)

Set the target channel to trig when outer loop is completed.

LOCAL_INLINE bool **DMA_HWA_GetChannelDoneStatus** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether the transfer is done on the selected channel.

LOCAL_INLINE DMA_RunningStatusType **DMA_HWA_GetChannelActiveStatus** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the active status of the DMA channel This flag signals the channel is currently in execution. It is set when channel service begins, and is cleared by the DMA as the inner loop completes or when any error condition is detected.

LOCAL_INLINE bool **DMA_HWA_GetOuterLoopTrigEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get channel to channel trig is enabled when outer loop is completed.

LOCAL_INLINE void **DMA_HWA_SetOuterLoopTrigEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable)

Set whether to enable channel to channel trig when outer loop is completed.

LOCAL_INLINE bool **DMA_HWA_GetAutoDisableReuquestEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether DMA request will be disabled automatically when outer loop is completed.

LOCAL_INLINE void **DMA_HWA_SetAutoDisableReuquestEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable)

Set whether to disable DMA request automatically when outer loop is completed.

LOCAL_INLINE bool **DMA_HWA_GetHalfCompleteInterruptEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether DMA interrupt will be generated when the outer loop is half done.

LOCAL_INLINE void **DMA_HWA_SetHalfCompleteInterruptEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable)

Set whether to enable DMA interrupt when the outer loop is half done.

LOCAL_INLINE bool **DMA_HWA_GetTransferCompleteInterruptEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether DMA interrupt is enabled when DMA transfer is completed.

LOCAL_INLINE void **DMA_HWA_EnableTransferCompleteInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Enable DMA transfer complete interrupt.

LOCAL_INLINE void **DMA_HWA_DisableTransferCompleteInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Disable DMA transfer complete interrupt.

LOCAL_INLINE bool **DMA_HWA_GetWaitStartFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether the DMA start is requested but has not been executed.

LOCAL_INLINE uint16_t **DMA_HWA_GetChannelControlStatus** (const DMA_Type *const pDma, uint8_t u8Channel)

Get DMA channel control and status.

LOCAL_INLINE void **DMA_HWA_SetChannelControlStatus** (DMA_Type *const pDma, uint8_t u8Channel, uint16_t u16Setting)

Set DMA channel control and status.

Detailed Description

Enumeration Type Documentation

enum DMA_ArbitrationAlgorithmType

DMA channel arbitration algorithm used in the channel arbitration phase.

Enumerator:

DMA_ARBITRATION_ALGORITHM_FIXED_PRIORITY	Use the fixed priority for arbitration
DMA_ARBITRATION_ALGORITHM_ROUND_ROBIN	Use the channel number for arbitration, higher channel number has higher priority

enum DMA_RunningStatusType

DMA running status.

Enumerator:

DMA_RUNNING_STATUS_IDLE	The DMA engine is idle
DMA_RUNNING_STATUS_ACTIVE	The DMA engine is transferring data

enum DMA_TransferSizeType

Defines the size of data in one transfer.

One transfer can contain multiple block, and one block may contain multiple data, this parameter specifies the size of data which the DMA engine will access one time in the memory.

Enumerator:

DMA_TRANSFER _SIZE_1B	
DMA_TRANSFER _SIZE_2B	
DMA_TRANSFER _SIZE_4B	
DMA_TRANSFER _SIZE_8B	
DMA_TRANSFER _SIZE_32B	

Function Documentation

LOCAL_INLINE void DMA_HWA_CancelTransfer (DMA_Type *const *pDma*)

Cancel the remaining transfer of the DMA.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_ClearAllChannelDoneStatus (DMA_Type *const *pDma*)

Clear DONE bit for the all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_ClearAllChannelErrorFlag (DMA_Type *const *pDma*)

Clear ERR bit for the all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_ClearAllChannelInterruptFlag (DMA_Type *const *pDma*)

Clear interrupt flag for the all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_ClearChannelDoneStatus (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Clear DONE bit for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_ClearChannelErrorFlag (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Clear ERR bit for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_ClearChannelInterruptFlag (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Clear interrupt flag for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_ClearHaltFlag (DMA_Type *const *pDma*)

Clear the halt flag of the DMA instance After HALT is cleared, the DMA could continue to operate.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_DisableAllChannelErrorInterrupt (DMA_Type *const *pDma*)

Disable error interrupt for all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_DisableAllChannelRequest (DMA_Type *const *pDma*)

Disable channel request for all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_DisableChannelErrorInterrupt (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Disable error interrupt for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_DisableChannelRequest (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Disable channel request for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_DisableTransferCompleteInterrupt (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Disable DMA transfer complete interrupt.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_EnableAllChannelErrorInterrupt (DMA_Type *const *pDma*)

Enable error interrupt for all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_EnableAllChannelRequest (DMA_Type *const *pDma*)

Enable channel request for all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_EnableChannelErrorInterrupt (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Enable error interrupt for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_EnableChannelRequest (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Enable channel request for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_EnableTransferCompleteInterrupt (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Enable DMA transfer complete interrupt.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_ErrorCancelTransfer (DMA_Type *const *pDma*)

Cancel the remaining transfer of the DMA and generate an error after finished cancelling.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE uint16_t DMA_HWA_GetAllChannelErrorFlag (const DMA_Type *const *pDma*)

Get the channel error status, each bit represents a channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

uint16_t the channel error status

LOCAL_INLINE uint16_t DMA_HWA_GetAllChannelErrorInterruptEnableFlag (const DMA_Type *const *pDma*)

Get channel error interrupt enable status.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

uint16_t the channel error interrupt enable status

LOCAL_INLINE DMA_ArbitrationAlgorithmType DMA_HWA_GetArbitrationAlgorithm (const DMA_Type *const *pDma*)

Get the DMA arbitration algorithm.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

DMA_ArbitrationAlgorithmType the DMA arbitration algorithm

LOCAL_INLINE bool DMA_HWA_GetAutoDisableReuquestEnableFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether DMA request will be disabled automatically when outer loop is completed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true DMA request will be disabled automatically when outer loop is completed

false DMA request will not be cleared when outer loop is completed

LOCAL_INLINE uint8_t DMA_HWA_GetBeginningChannelToChannelTrigChannel (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the target channel when beginning channel to channel trig is enabled.

Note

this field shall always be same with the current target channel trig field

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint8_t the target channel

Here is the call graph for this function:

IMAGE

LOCAL_INLINE bool DMA_HWA_GetBeginningChannelToChannelTrigEnableFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether beginning channel to channel trig is enabled when inner loop complete.

Note

this field shall always be same with the current channel to channel trig enable flag

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true channel to channel trig is enabled when inner loop complete

false channel to channel trig is disabled when inner loop complete

LOCAL_INLINE uint16_t DMA_HWA_GetBeginningLoopCount (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the beginning loop count This field specifies how many inner loops will be executed in a DMA transfer.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint16_t the beginning loop count

Here is the call graph for this function:

IMAGE

LOCAL_INLINE bool DMA_HWA_GetCancelTransferStatus (const DMA_Type *const *pDma*)

Get whether the DMA instance is cancelling.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

true cancel operation is requested and has not been finished

false DMA is under normal operation

LOCAL_INLINE DMA_RunningStatusType DMA_HWA_GetChannelActiveStatus (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the active status of the DMA channel This flag signals the channel is currently in execution. It is set when channel service begins, and is cleared by the DMA as the inner loop completes or when any error condition is detected.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true the channel is in execution

false the channel is idle

LOCAL_INLINE uint16_t DMA_HWA_GetChannelControlStatus (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get DMA channel control and status.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint16_t control and status of the selected channel

LOCAL_INLINE bool DMA_HWA_GetChannelDoneStatus (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether the transfer is done on the selected channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true the transfer is done

false the transfer has not done

LOCAL_INLINE bool DMA_HWA_GetChannelErrorFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether there is error occurred on the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

<i>u8Channel</i>	the selected channel
------------------	----------------------

Returns

true there is error on the specified channel

false there is no error on the specified channel

**LOCAL_INLINE bool DMA_HWA_GetChannelErrorInterruptEnableFlag (const DMA_Type *const *pDma*,
uint8_t *u8Channel*)**

Get whether error interrupt is enabled on the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true error interrupt is enabled on the specified channel

false error interrupt is disabled on the specified channel

LOCAL_INLINE bool DMA_HWA_GetContinuousTrigModeEnableFlag (const DMA_Type *const *pDma*)

Get whether the continuous trig mode is enabled When continuous trig mode is enabled, channel arbitration is not used for a inner loop channel trig before being activated again. Upon inner loop completion, the channel activates again if that channel has a inner loop channel trig enabled and the trig channel is itself. This effectively applies the inner loop offsets and restarts the next inner loop.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

true continuous trig mode is enabled

false continuous trig mode is disabled

LOCAL_INLINE uint32_t DMA_HWA_GetControlRegister (const DMA_Type *const *pDma*)

Get the control register of the DMA instance.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

uint32_t the control register settings

LOCAL_INLINE uint8_t DMA_HWA_GetCurrentChannelToChannelTrigChannel (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the target channel when current channel to channel trig is enabled.

Note

this field shall always be same with the beginning traget channel trig field

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint8_t the target channel

Here is the call graph for this function:

IMAGE

LOCAL_INLINE bool DMA_HWA_GetCurrentChannelToChannelTrigEnableFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether current channel to channel trig is enabled when inner loop complete.

Note

this field shall always be same with the beginning channel to channel trig enable flag

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true channel to channel trig is enabled when inner loop complete

false channel to channel trig is disabled when inner loop complete

LOCAL_INLINE uint16_t DMA_HWA_GetCurrentLoopCount (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the current loop count The current loop count is the same as the beginning loop count initially, and it will decrement each time a inner loop finishes.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint16_t the current loop count

Here is the call graph for this function:

IMAGE

LOCAL_INLINE bool DMA_HWA_GetDebugModeStopFlag (const DMA_Type *const *pDma*)

Get whether the DMA is configured to stop under debug mode.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

true the DMA is will stop under debug mode

true the DMA is will continue to operate under debug mode

LOCAL_INLINE uint32_t DMA_HWA_GetDestAddr (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the destination address of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint32_t the destination address of the DMA channel

**LOCAL_INLINE DMA_TransferSizeType DMA_HWA_GetDestDataSize (const DMA_Type *const *pDma*,
uint8_t *u8Channel*)**

Get the destination data size of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

DMA_TransferSizeType the destination data size of the DMA channel

**LOCAL_INLINE int32_t DMA_HWA_GetDestLastAddrAdjustment (const DMA_Type *const *pDma*, uint8_t
u8Channel)**

Get the address adjustment applied to the destination address after major loop finished.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

int32_t the address adjustment applied to the destination address after major loop finished

LOCAL_INLINE uint8_t DMA_HWA_GetDestModulo (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the destination address aligned modulo of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint8_t the destination address aligned modulo

LOCAL_INLINE int16_t DMA_HWA_GetDestOffset (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get destination data offset of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

int16_t the destination data offset

LOCAL_INLINE uint16_t DMA_HWA_GetDestUnalignModulo (const DMA_Type *const *pDma*, uint8_t *u8Selection*)

Get the destination unalign modulo of the index.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Selection</i>	the index of the unalign modulo

Returns

uint16_t the destination unalign modulo

LOCAL_INLINE bool DMA_HWA_GetDestUnalignModuloEnableFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether destination unalign modulo is enabled for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE bool DMA_HWA_GetErrorCancelTransferStatus (const DMA_Type *const *pDma*)

Get whether the DMA instance is cancelling with error.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

true cancel operation is requested and has not been finished

false DMA is under normal operation

LOCAL_INLINE uint32_t DMA_HWA_GetErrorStatus (const DMA_Type *const *pDma*)

Get the DMA error status.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

uint32_t the DMA error status

**LOCAL_INLINE bool DMA_HWA_GetHalfCompleteInterruptEnableFlag (const DMA_Type *const *pDma*,
uint8_t *u8Channel*)**

Get whether DMA interrupt will be generated when the outer loop is half done.

Note

When outer loop count is 1, do not enable half complete interrupt

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true DMA will generate interrupt when outer loop is half done

false DMA will not generate interrupt when outer loop is half done

LOCAL_INLINE bool DMA_HWA_GetHaltOnErrorFlag (const DMA_Type *const *pDma*)

Get whether halt on error is enabled on the DMA instance.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

true halt on error is enabled, any error will cause HALT flag to be set

false halt on error is disabled

LOCAL_INLINE bool DMA_HWA_GetHaltStatus (const DMA_Type *const *pDma*)

Get whether the DMA is halted When the DMA is halted, it will ignore all service requests.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

true the DMA is halted

false the DMA is not halted

**LOCAL_INLINE bool DMA_HWA_GetInnerLoopDestOffsetEnableFlag (const DMA_Type *const *pDma*,
uint8_t *u8Channel*)**

Get whether destination address inner loop offset is enabled.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true destination address inner loop offset is enabled

false destination address inner loop offset is disabled

Here is the call graph for this function:

IMAGE

LOCAL_INLINE bool DMA_HWA_GetInnerLoopMappingEnableFlag (const DMA_Type *const *pDma*)

Get whether inner loop mapping is enabled.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

Returns

true inner loop mapping is enabled

false inner loop mapping is disabled

LOCAL_INLINE int32_t DMA_HWA_GetInnerLoopOffset (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the inner loop offset of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

int32_t the signed inner loop offset

Here is the call graph for this function:

IMAGE

LOCAL_INLINE uint32_t DMA_HWA_GetInnerLoopSize (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the transfer data size of the inner loop.

Note

if inner loop mapping is disabled, the range is 0~2³¹-1 if inner loop mapping is enabled and inner loop offset is disabled, the range is 0~2³⁰-1 if inner loop mapping is enabled and inner loop offset is enabled, the range is 0~2¹⁰-1 0 means 2³²

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint32_t the transfer data size of the inner loop

Here is the call graph for this function:

IMAGE

**LOCAL_INLINE bool DMA_HWA_GetInnerLoopSrcOffsetEnableFlag (const DMA_Type *const *pDma*,
uint8_t *u8Channel*)**

Get whether source address inner loop offset is enabled.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true source address inner loop offset is enabled

false source address inner loop offset is disabled

Here is the call graph for this function:

IMAGE

**LOCAL_INLINE uint8_t DMA_HWA_GetOuterLoopTrigChannel (const DMA_Type *const *pDma*, uint8_t
u8Channel)**

Get the target channel to trig when outer loop is completed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint8_t the target channel

**LOCAL_INLINE bool DMA_HWA_GetOuterLoopTrigEnableFlag (const DMA_Type *const *pDma*, uint8_t
u8Channel)**

Get channel to channel trig is enabled when outer loop is completed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true the target channel will be trigged when outer loop is completed

false the channel to channel trig when outer loop is completed is disabled

LOCAL_INLINE uint8_t DMA_HWA_GetPriority (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get priority of the channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint8_t the priority of the channel

LOCAL_INLINE uint32_t DMA_HWA_GetSrcAddr (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the source address of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint32_t the source address of the DMA channel

**LOCAL_INLINE DMA_TransferSizeType DMA_HWA_GetSrcDataSize (const DMA_Type *const *pDma*,
uint8_t *u8Channel*)**

Get the source data size of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

DMA_TransferSizeType the source data size of the DMA channel

LOCAL_INLINE int32_t DMA_HWA_GetSrcLastAddrAdjustment (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the address adjustment applied to the source address after major loop finished.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

int32_t the address adjustment applied to the source address after major loop finished

LOCAL_INLINE uint8_t DMA_HWA_GetSrcModulo (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the source address aligned modulo of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint8_t the source address aligned modulo

LOCAL_INLINE int16_t DMA_HWA_GetSrcOffset (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get source data offset of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

int16_t the source data offset

LOCAL_INLINE uint16_t DMA_HWA_GetSrcUnalignModulo (const DMA_Type *const *pDma*, uint8_t *u8Selection*)

Get the source unalign modulo of the index.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Selection</i>	the index of the unalign modulo

Returns

uint16_t the source unalign modulo

LOCAL_INLINE bool DMA_HWA_GetSrcUnalignModuloEnableFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether source unalign modulo is enabled for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE bool DMA_HWA_GetTransferCompleteInterruptEnableFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether DMA interrupt is enabled when DMA transfer is completed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true DMA will generate interrupt when outer loop is completed

false DMA will not generate interrupt when outer loop is completed

LOCAL_INLINE uint8_t DMA_HWA_GetUnalignModuloSel (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get the unalign modulo index selection.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

uint8_t the unalign modulo index selection

LOCAL_INLINE bool DMA_HWA_GetWaitStartFlag (const DMA_Type *const *pDma*, uint8_t *u8Channel*)

Get whether the DMA start is requested but has not been executed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

Returns

true the DMA start is requested but has not been executed

false the DMA start is not requested or it has started execution

LOCAL_INLINE void DMA_HWA_HaltOperations (DMA_Type *const *pDma*)

Halt the DMA operations.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

LOCAL_INLINE void DMA_HWA_SetAllChannelStart (DMA_Type *const *pDma*)

Set start for the all channels.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

**LOCAL_INLINE void DMA_HWA_SetArbitrationAlgorithm (DMA_Type *const *pDma*,
DMA_ArbitrationAlgorithmType *eAlgorithm*)**

Set the DMA arbitration algorithm.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>eAlgorithm</i>	the DMA arbitration algorithm

**LOCAL_INLINE void DMA_HWA_SetAutoDisableReuquestEnableFlag (DMA_Type *const *pDma*, uint8_t
u8Channel, bool *bEnable*)**

Set whether to disable DMA request automatically when outer loop is completed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>bEnable</i>	whether to disable DMA request automatically when outer loop is completed

**LOCAL_INLINE void DMA_HWA_SetChannelControlStatus (DMA_Type *const *pDma*, uint8_t *u8Channel*,
uint16_t *u16Setting*)**

Set DMA channel control and status.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u16Setting</i>	control settings of the selected channel

LOCAL_INLINE void DMA_HWA_SetChannelStart (DMA_Type *const *pDma*, uint8_t *u8Channel*)

Set start for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

LOCAL_INLINE void DMA_HWA_SetChannelToChannelTrig (DMA_Type *const *pDma*, uint8_t *u8Channel*, bool *bEnable*, uint8_t *u8TrigChannel*)

Set channel to channel trig when inner loop complete.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>bEnable</i>	whether to enable channel to channel trig when inner loop complete
<i>u8TrigChannel</i>	the target channel

Here is the call graph for this function:

IMAGE

LOCAL_INLINE void DMA_HWA_SetContinuousTrigModeEnableFlag (DMA_Type *const *pDma*, bool *bEnable*)

Set whether to enable the continuous trig mode.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>bEnable</i>	whether to enable the continuous trig mode

LOCAL_INLINE void DMA_HWA_SetControlRegister (DMA_Type *const *pDma*, uint32_t *u32Settings*)

Set the control register of the DMA instance.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u32Settings</i>	the settings of the DMA control register

LOCAL_INLINE void DMA_HWA_SetDebugModeStopFlag (DMA_Type *const *pDma*, bool *bEnable*)

Set whether to stop DMA under debug mode.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>bEnable</i>	whether to stop DMA under debug mode

LOCAL_INLINE void DMA_HWA_SetDestAddr (DMA_Type *const *pDma*, uint8_t *u8Channel*, uint32_t *u32DestAddr*)

Set the destination address of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u32DestAddr</i>	the destination address of the DMA channel

LOCAL_INLINE void DMA_HWA_SetDestDataSize (DMA_Type *const *pDma*, uint8_t *u8Channel*, DMA_TransferSizeType *eDataSize*)

Set the destination data size of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>eDataSize</i>	the destination data size of the DMA channel

LOCAL_INLINE void DMA_HWA_SetDestLastAddrAdjustment (DMA_Type *const *pDma*, uint8_t *u8Channel*, int32_t *s32LastAdj*)

Set the address adjustment applied to the destination address after major loop finished.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>s32LastAdj</i>	the address adjustment applied to the destination address after major loop finished

LOCAL_INLINE void DMA_HWA_SetDestModulo (DMA_Type *const *pDma*, uint8_t *u8Channel*, uint8_t *u8DestModulo*)

Set the destination address aligned modulo of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u8DestModulo</i>	the destination address aligned modulo

LOCAL_INLINE void DMA_HWA_SetDestOffset (DMA_Type *const *pDma*, uint8_t *u8Channel*, int16_t *s16Offset*)

Set destination data offset of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>s16Offset</i>	the destination data offset

LOCAL_INLINE void DMA_HWA_SetHalfCompleteInterruptEnableFlag (DMA_Type *const *pDma*, uint8_t *u8Channel*, bool *bEnable*)

Set whether to enable DMA interrupt when the outer loop is half done.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>bEnable</i>	whether to enable DMA interrupt when the outer loop is half done

LOCAL_INLINE void DMA_HWA_SetHaltOnErrorFlag (DMA_Type *const *pDma*, bool *bEnable*)

Set whether to enable halt on error on the DMA instance.

Note

If halt on error is enabled, any error will cause HALT flag to be set

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>bEnable</i>	whether to enable halt on error

LOCAL_INLINE void DMA_HWA_SetInnerLoopMappingEnableFlag (DMA_Type *const *pDma*, bool *bEnable*)

Set whether to enable inner loop mapping.

Note

Only after enabling inner loop mapping, you can apply an offset to the source and/or destination address after the inner loop finishes.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>bEnable</i>	whether to enable the inner loop mapping

LOCAL_INLINE void DMA_HWA_SetInnerLoopOffset (DMA_Type *const *pDma*, uint8_t *u8Channel*, bool *bEnableSrcOffset*, bool *bEnableDestOffset*, int32_t *s32Offset*)

Set the inner loop offset of the DMA channel.

Note

the inner loop offset can only be enabled when inner loop mapping is enabled

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>bEnableSrcOffset</i>	whether to enable source address inner loop offset
<i>bEnableDestOffset</i>	whether to enable destination address inner loop offset
<i>s32Offset</i>	the inner loop offset

Here is the call graph for this function:

IMAGE

```
LOCAL_INLINE void DMA_HWA_SetInnerLoopSize (DMA_Type *const pDma, uint8_t u8Channel,
uint32_t u32Size)
```

Set the transfer data size of the inner loop.

Note

if inner loop mapping is disabled, the range is $0 \sim 2^{31}-1$ if inner loop mapping is enabled and inner loop offset is disabled, the range is $0 \sim 2^{30}-1$ if inner loop mapping is enabled and inner loop offset is enabled, the range is $0 \sim 2^{10}-1$ 0 means 2^{32}

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u32Size</i>	the transfer data size of the inner loop

Here is the call graph for this function:

IMAGE

```
LOCAL_INLINE void DMA_HWA_SetLoopCount (DMA_Type *const pDma, uint8_t u8Channel, uint16_t
u16LoopCnt)
```

Set the loop count of the DMA transfer This field specifies how many inner loops will be executed in a DMA transfer.

Parameters

<i>pDma</i>	the base address of the DMA instance
-------------	--------------------------------------

<i>u8Channel</i>	the selected channel
<i>u16LoopCnt</i>	the inner loop count in a DMA transfer

Here is the call graph for this function:

IMAGE

LOCAL_INLINE void DMA_HWA_SetOuterLoopTrigChannel (DMA_Type *const *pDma*, uint8_t *u8Channel*, uint8_t *u8TrigChannel*)

Set the target channel to trig when outer loop is completed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u8TrigChannel</i>	the target channel

LOCAL_INLINE void DMA_HWA_SetOuterLoopTrigEnableFlag (DMA_Type *const *pDma*, uint8_t *u8Channel*, bool *bEnable*)

Set whether to enable channel to channel trig when outer loop is completed.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>bEnable</i>	whether to enable channel to channel trig when outer loop is completed

LOCAL_INLINE void DMA_HWA_SetPriority (DMA_Type *const *pDma*, uint8_t *u8Channel*, uint8_t *u8Priority*)

Set priority of the channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u8Priority</i>	the priority of the channel

LOCAL_INLINE void DMA_HWA_SetSrcAddr (DMA_Type *const *pDma*, uint8_t *u8Channel*, uint32_t *u32SrcAddr*)

Set the source address of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u32SrcAddr</i>	the source address of the DMA channel

LOCAL_INLINE void DMA_HWA_SetSrcDataSize (DMA_Type *const *pDma*, uint8_t *u8Channel*, DMA_TransferSizeType *eDataSize*)

Set the source data size of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>eDataSize</i>	the source data size of the DMA channel

LOCAL_INLINE void DMA_HWA_SetSrcLastAddrAdjustment (DMA_Type *const *pDma*, uint8_t *u8Channel*, int32_t *s32LastAdj*)

Set the address adjustment applied to the source address after major loop finished.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>s32LastAdj</i>	the address adjustment applied to the source address after major loop finished

LOCAL_INLINE void DMA_HWA_SetSrcModulo (DMA_Type *const *pDma*, uint8_t *u8Channel*, uint8_t *u8SrcModulo*)

Set the source address aligned modulo of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u8SrcModulo</i>	the source address aligned modulo

LOCAL_INLINE void DMA_HWA_SetSrcOffset (DMA_Type *const *pDma*, uint8_t *u8Channel*, int16_t *s16Offset*)

Set source data offset of the DMA channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>s16Offset</i>	the source data offset

LOCAL_INLINE void DMA_HWA_SetUnalignModulo (DMA_Type *const *pDma*, uint8_t *u8Selection*, uint16_t *u16SrcModulo*, uint16_t *u16DestModulo*)

Set unalign modulo of the index.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Selection</i>	the index of the unalign modulo
<i>u16SrcModulo</i>	the source unalign modulo
<i>u16DestModulo</i>	the destination unalign modulo

LOCAL_INLINE void DMA_HWA_SetUnalignModuloEnableFlag (DMA_Type *const *pDma*, uint8_t *u8Channel*, bool *bEnableSrcModulo*, bool *bEnableDestModulo*)

Get whether to enable unalign modulo for the specified channel.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel

<i>bEnableSrcModulo</i>	whether source unalign modulo is to be enabled
<i>bEnableDestModulo</i>	whether destination unalign modulo is to be enabled

LOCAL_INLINE void DMA_HWA_SetUnalignModuloSel (DMA_Type *const *pDma*, uint8_t *u8Channel*, uint8_t *u8UnalignModuloSel*)

Set the unalign modulo index selection.

Parameters

<i>pDma</i>	the base address of the DMA instance
<i>u8Channel</i>	the selected channel
<i>u8UnalignModuloSel</i>	the unalign modulo index selection

HwA_dmamux

Collaboration diagram for HwA_dmamux:Functions

LOCAL_INLINE bool **DMAMUX_HWA_GetEnableFlag** (const DMAMUX_Type *const pDmamux, uint8_t u8Channel)

Get whether DMAMUX is enabled for the specified DMA channel.

LOCAL_INLINE DMA_RequestSourceType **DMAMUX_HWA_GetRequestSource** (const DMAMUX_Type *const pDmamux, uint8_t u8Channel)

Get the request source for the specified DMA channel.

LOCAL_INLINE void **DMAMUX_HWA_SetRequestSource** (DMAMUX_Type *const pDmamux, uint8_t u8Channel, bool bEnable, DMA_RequestSourceType eReqSrc)

Set the request source for the specified DMA channel.

LOCAL_INLINE bool **DMAMUX_HWA_GetPeriodicTrigFlag** (const DMAMUX_Type *const pDmamux, uint8_t u8Channel)

Get whether periodic trig is enabled for the specified DMA channel.

LOCAL_INLINE void **DMAMUX_HWA_SetPeriodicTrigFlag** (DMAMUX_Type *const pDmamux, uint8_t u8Channel, bool bEnable)

Set whether to enable periodic trig for the specified DMA channel.

Detailed Description

Function Documentation

LOCAL_INLINE bool **DMAMUX_HWA_GetEnableFlag** (const DMAMUX_Type *const *pDmamux*, uint8_t *u8Channel*)

Get whether DMAMUX is enabled for the specified DMA channel.

Parameters

<i>pDmamux</i>	the base address of the DMAMUX instance
<i>u8Channel</i>	the selected DMA channel

Returns

true DMAMUX is enabled for the specified DMA channel

false DMAMUX is disabled for the specified DMA channel

LOCAL_INLINE bool **DMAMUX_HWA_GetPeriodicTrigFlag** (const DMAMUX_Type *const *pDmamux*, uint8_t *u8Channel*)

Get whether periodic trig is enabled for the specified DMA channel.

Note

Only DMA channel 0~3 supports periodic trig

Parameters

<i>pDmamux</i>	the base address of the DMAMUX instance
----------------	---

<i>u8Channel</i>	the selected DMA channel
------------------	--------------------------

Returns

true periodic trig is enabled for the specified DMA channel

true periodic trig is disabled for the specified DMA channel

LOCAL_INLINE DMA_RequestSourceType DMAMUX_HWA_GetRequestSource (const DMAMUX_Type *const *pDmamux*, uint8_t *u8Channel*)

Get the request source for the specified DMA channel.

Parameters

<i>pDmamux</i>	the base address of the DMAMUX instance
<i>u8Channel</i>	the selected DMA channel

Returns

DMA_RequestSourceType the request source of the specified DMA channel

LOCAL_INLINE void DMAMUX_HWA_SetPeriodicTrigFlag (DMAMUX_Type *const *pDmamux*, uint8_t *u8Channel*, bool *bEnable*)

Set whether to enable periodic trig for the specified DMA channel.

Note

Only DMA channel 0~3 supports periodic trig

Parameters

<i>pDmamux</i>	the base address of the DMAMUX instance
<i>u8Channel</i>	the selected DMA channel
<i>bEnable</i>	whether to enable periodic trig for the specified DMA channel

LOCAL_INLINE void DMAMUX_HWA_SetRequestSource (DMAMUX_Type *const *pDmamux*, uint8_t *u8Channel*, bool *bEnable*, DMA_RequestSourceType *eReqSrc*)

Set the request source for the specified DMA channel.

Parameters

<i>pDmamux</i>	the base address of the DMAMUX instance
<i>u8Channel</i>	the selected DMA channel
<i>bEnable</i>	whether to enable DMAMUX for the specified DMA channel
<i>eReqSrc</i>	the request source to set for the specified DMA channel

HwA_eim

Collaboration diagram for HwA_eim:Functions

LOCAL_INLINE void **EIM_HWA_EnableGlobalErrorInjection** (EIM_Type *const pEim)

Enable EIM Global Error Injection.

LOCAL_INLINE void **EIM_HWA_DisableGlobalErrorInjection** (EIM_Type *const pEim)

Disable EIM Global Error Injection.

LOCAL_INLINE uint32_t **EIM_HWA_GetCtrlRegn** (const EIM_Type *const pEim, uint8_t u8Idx)

Get EIM channel N control register value.

LOCAL_INLINE void **EIM_HWA_SetCtrlRegn** (EIM_Type *const pEim, uint8_t u8Idx, uint32_t u32Value)

Set EIM channel N control register value.

LOCAL_INLINE bool **EIM_HWA_GetCtrlRegnDwpLockStatus** (const EIM_Type *const pEim, uint8_t u8Idx)

Get EIM_CTRL_REGn register DWP lock status.

LOCAL_INLINE void **EIM_HWA_SetCtrlRegnDwpLock** (EIM_Type *const pEim, uint8_t u8Idx, bool bEnable)

Set EIM_CTRL_REGn register DWP lock status.

LOCAL_INLINE void **EIM_HWA_SetCtrlRegnDwpMode** (EIM_Type *const pEim, uint8_t u8Idx, uint8_t u8DwpMode)

Set EIM_CTRL_REGn register DWP mode.

LOCAL_INLINE void **EIM_HWA_SetLockstep** (EIM_Type *const pEim, uint8_t u8Idx, uint32_t u32Value)

Set EIM LOCKSTEP error injection register.

LOCAL_INLINE uint32_t **EIM_HWA_GetBusRegn** (const EIM_Type *const pEim, uint8_t u8Idx)
Get EIM bus n register value.

LOCAL_INLINE void **EIM_HWA_SetBusRegn** (EIM_Type *const pEim, uint8_t u8Idx, uint32_t u32Value)
Set EIM bus n register value.

Detailed Description

Function Documentation

LOCAL_INLINE void **EIM_HWA_DisableGlobalErrorInjection** (EIM_Type *const *pEim*)

Disable EIM Global Error Injection.

Parameters

<i>pEim</i>	EIM Instance.
-------------	---------------

LOCAL_INLINE void **EIM_HWA_EnableGlobalErrorInjection** (EIM_Type *const *pEim*)

Enable EIM Global Error Injection.

Parameters

<i>pEim</i>	EIM Instance.
-------------	---------------

LOCAL_INLINE uint32_t **EIM_HWA_GetBusRegn** (const EIM_Type *const *pEim*, uint8_t *u8Idx*)

Get EIM bus n register value.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Bus register index.

Returns

Register value.

LOCAL_INLINE uint32_t EIM_HWA_GetCtrlRegn (const EIM_Type *const *pEim*, uint8_t *u8Idx*)

Get EIM channel N control register value.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Control register index.

Returns

EIM channel N control register value.

LOCAL_INLINE bool EIM_HWA_GetCtrlRegnDwpLockStatus (const EIM_Type *const *pEim*, uint8_t *u8Idx*)

Get EIM_CTRL_REGn register DWP lock status.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Control register index.

Returns

Lock status.

LOCAL_INLINE void EIM_HWA_SetBusRegn (EIM_Type *const *pEim*, uint8_t *u8Idx*, uint32_t *u32Value*)

Set EIM bus n register value.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Control register index.

<i>u32Value</i>	Register value.
-----------------	-----------------

LOCAL_INLINE void EIM_HWA_SetCtrlRegn (EIM_Type *const *pEim*, uint8_t *u8Idx*, uint32_t *u32Value*)

Set EIM channel N control register value.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Control register index.
<i>u32Value</i>	Register value.

LOCAL_INLINE void EIM_HWA_SetCtrlRegnDwpLock (EIM_Type *const *pEim*, uint8_t *u8Idx*, bool *bEnable*)

Set EIM_CTRL_REGn register DWP lock status.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Control register index.
<i>bEnable</i>	Enable or disable.

LOCAL_INLINE void EIM_HWA_SetCtrlRegnDwpMode (EIM_Type *const *pEim*, uint8_t *u8Idx*, uint8_t *u8DwpMode*)

Set EIM_CTRL_REGn register DWP mode.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Control register index.
<i>u8DwpMode</i>	DWP mode.

LOCAL_INLINE void EIM_HWA_SetLockstep (EIM_Type *const *pEim*, uint8_t *u8Idx*, uint32_t *u32Value*)

Set EIM LOCKSTEP error injection register.

Parameters

<i>pEim</i>	EIM Instance.
<i>u8Idx</i>	Control register index.
<i>u32Value</i>	Register value.

HwA_enet

Collaboration diagram for HwA_enet:Enumerations

```
enum ENET_MAC_StatusType { ENET_MAC_STATUS_IDLE = 0u, ENET_MAC_STATUS_BUSY = 1u }
```

ENE MAC status type definition.

```
enum ENET_MAC_SrcAddrCtrlType { ENET_MAC_SRC_ADDR_CTRL_BY_INPUT_SIGNAL = 0u,  
ENET_MAC_SRC_ADDR_INSERT_BY_MAC_ADDR_IDX_0 = 2u,  
ENET_MAC_SRC_ADDR_REPLACE_BY_MAC_ADDR_IDX_0 = 3u,  
ENET_MAC_SRC_ADDR_INSERT_BY_MAC_ADDR_IDX_1 = 6u,  
ENET_MAC_SRC_ADDR_REPLACE_BY_MAC_ADDR_IDX_1 = 7u }
```

ENET MAC source address insertion control type definition.

```
enum ENET_MAC_InterPacketGapType { ENET_MAC_INTER_PACKET_GAP_96_BIT = 0u,  
ENET_MAC_INTER_PACKET_GAP_88_BIT = 1u, ENET_MAC_INTER_PACKET_GAP_80_BIT = 2u,  
ENET_MAC_INTER_PACKET_GAP_72_BIT = 3u, ENET_MAC_INTER_PACKET_GAP_64_BIT = 4u,  
ENET_MAC_INTER_PACKET_GAP_56_BIT = 5u, ENET_MAC_INTER_PACKET_GAP_48_BIT = 6u,  
ENET_MAC_INTER_PACKET_GAP_40_BIT = 7u }
```

MAC inter packet gap type definition.

```
enum ENET_MAC_ExtInterPacketGapType { ENET_MAC_EXT_INTER_PACKET_GAP_104_BIT = 0u,  
ENET_MAC_EXT_INTER_PACKET_GAP_112_BIT = 1u, ENET_MAC_EXT_INTER_PACKET_GAP_120_BIT =  
2u, ENET_MAC_EXT_INTER_PACKET_GAP_128_BIT = 3u, ENET_MAC_EXT_INTER_PACKET_GAP_136_BIT  
= 4u, ENET_MAC_EXT_INTER_PACKET_GAP_144_BIT = 5u,  
ENET_MAC_EXT_INTER_PACKET_GAP_152_BIT = 6u, ENET_MAC_EXT_INTER_PACKET_GAP_160_BIT =  
7u, ENET_MAC_EXT_INTER_PACKET_GAP_168_BIT = 8u, ENET_MAC_EXT_INTER_PACKET_GAP_176_BIT  
= 9u, ENET_MAC_EXT_INTER_PACKET_GAP_184_BIT = 10u,  
ENET_MAC_EXT_INTER_PACKET_GAP_192_BIT = 11u, ENET_MAC_EXT_INTER_PACKET_GAP_200_BIT =  
12u, ENET_MAC_EXT_INTER_PACKET_GAP_208_BIT = 13u,  
ENET_MAC_EXT_INTER_PACKET_GAP_216_BIT = 14u, ENET_MAC_EXT_INTER_PACKET_GAP_224_BIT =  
15u, ENET_MAC_EXT_INTER_PACKET_GAP_232_BIT = 16u,  
ENET_MAC_EXT_INTER_PACKET_GAP_240_BIT = 17u, ENET_MAC_EXT_INTER_PACKET_GAP_248_BIT =  
18u, ENET_MAC_EXT_INTER_PACKET_GAP_256_BIT = 19u,  
ENET_MAC_EXT_INTER_PACKET_GAP_264_BIT = 20u, ENET_MAC_EXT_INTER_PACKET_GAP_272_BIT =
```

21u, ENET_MAC_EXT_INTER_PACKET_GAP_280_BIT = 22u,
 ENET_MAC_EXT_INTER_PACKET_GAP_288_BIT = 23u, ENET_MAC_EXT_INTER_PACKET_GAP_296_BIT =
 24u, ENET_MAC_EXT_INTER_PACKET_GAP_304_BIT = 25u,
 ENET_MAC_EXT_INTER_PACKET_GAP_312_BIT = 26u, ENET_MAC_EXT_INTER_PACKET_GAP_320_BIT =
 27u, ENET_MAC_EXT_INTER_PACKET_GAP_328_BIT = 28u,
 ENET_MAC_EXT_INTER_PACKET_GAP_336_BIT = 29u, ENET_MAC_EXT_INTER_PACKET_GAP_344_BIT =
 30u, ENET_MAC_EXT_INTER_PACKET_GAP_352_BIT = 31u }

MAC extend inner packet gap type definition.

enum ENET_MAC_SpeedType { ENET_MAC_SPEED_1000MBPS = 0u, ENET_MAC_SPEED_2500MBPS = 1u,
 ENET_MAC_SPEED_10MBPS = 2u, ENET_MAC_SPEED_100MBPS = 3u }

MAC speed mode type definition.

enum ENET_MAC_DuplexModeType { ENET_MAC_HALF_DUPLEX = 0u, ENET_MAC_FULL_DUPLEX = 1u }

MAC duplex mode type definition.

enum ENET_MAC_BackOffLimitType { ENET_MAC_BACK_OFF_LIMIT_10_SLOT = 0u,
 ENET_MAC_BACK_OFF_LIMIT_8_SLOT = 1u, ENET_MAC_BACK_OFF_LIMIT_4_SLOT = 2u,
 ENET_MAC_BACK_OFF_LIMIT_1_SLOT = 3u }

MAC back off limit type definition.

enum ENET_MAC_SlowProtocolDetectType { ENET_MAC_SLOW_PROTOCOL_DETECT_STANDARD = 0u,
 ENET_MAC_SLOW_PROTOCOL_DETECT_EXTENDED = 1u }

MAC slow protocol detect type definition.

enum ENET_MAC_AddrFilterModeType { ENET_MAC_ADDR_FILTER_DIRECT = 0u,
 ENET_MAC_ADDR_FILTER_INVERSE = 1u }

MAC address filter mode type definition.

enum ENET_MAC_CtrlPacketFilterModeType { ENET_MAC_CTRL_PACKET_FILTER_ALL = 0u,
 ENET_MAC_CTRL_PACKET_FILTER_PAUSE_PACKET = 1u, ENET_MAC_CTRL_PACKET_FORWARD_ALL =
 2u, ENET_MAC_CTRL_PACKET_ADDR_FILTER = 3u }

MAC control packets filter mode type definition.

enum ENET_MAC_HashOrPerfectModeType { ENET_MAC_USE_PERFECT_FILTER = 0u,
 ENET_MAC_USE_HASH_FILTER = 1u }

MAC perfect or hash filter mode type definition.

enum ENET_MAC_WDGTimerSizeType { ENET_MAC_WDG_TIMER_SIZE_2KB = 0u,
 ENET_MAC_WDG_TIMER_SIZE_3KB = 1u, ENET_MAC_WDG_TIMER_SIZE_4KB = 2u,
 ENET_MAC_WDG_TIMER_SIZE_5KB = 3u, ENET_MAC_WDG_TIMER_SIZE_6KB = 4u,
 ENET_MAC_WDG_TIMER_SIZE_7KB = 5u, ENET_MAC_WDG_TIMER_SIZE_8KB = 6u,
 ENET_MAC_WDG_TIMER_SIZE_9KB = 7u, ENET_MAC_WDG_TIMER_SIZE_10KB = 8u,
 ENET_MAC_WDG_TIMER_SIZE_11KB = 9u, ENET_MAC_WDG_TIMER_SIZE_12KB = 10u,

ENET_MAC_WDG_TIMER_SIZE_13KB = 11u, **ENET_MAC_WDG_TIMER_SIZE_14KB** = 12u,
ENET_MAC_WDG_TIMER_SIZE_15KB = 13u }

MAC watchdog timer type definition.

enum **ENET_MAC_VLANTagStripModeType** { **ENET_MAC_VLAN_TAG_STRIP_NEVER** = 0u,
ENET_MAC_VLAN_TAG_STRIP_FILTER_PASS = 1u, **ENET_MAC_VLAN_TAG_STRIP_FILTER_FAIL** = 2u,
ENET_MAC_VLAN_TAG_STRIP_ALWAYS = 3u }

MAC VLAN tag strip type.

enum **ENET_MAC_VLANCommandType** { **ENET_MAC_VLAN_COMMAND_WRITE** = 0u,
ENET_MAC_VLAN_COMMAND_READ = 1u }

MAC VLAN indirect registers access command type.

enum **ENET_MAC_VLANInclCommandType** { **ENET_MAC_VLAN_INCL_COMMAND_READ** = 0u,
ENET_MAC_VLAN_INCL_COMMAND_WRITE = 1u }

MAC VLAN incl indirect registers access command type.

enum **ENET_MAC_VLANTagCtrlType** { **ENET_MAC_VLAN_TAG_NO_OPERATION** = 0u,
ENET_MAC_VLAN_TAG_DELETION = 1u, **ENET_MAC_VLAN_TAG_INSERTION** = 2u,
ENET_MAC_VLAN_TAG_REPLACEMENT = 3u }

MAC VLAN tag control type.

enum **ENET_MAC_VLANType** { **ENET_MAC_VLAN_TYPE_C_VLAN** = 0u, **ENET_MAC_VLAN_TYPE_S_VLAN** =
1u }

MAC VLAN tag type S-VLAN IEEE802.1ad (Q-in-Q) C-VLAN IEEE802.1Q.

enum **ENET_MAC_VLANCompareType** { **ENET_MAC_VLAN_16BITS_COMPARE** = 0u,
ENET_MAC_VLAN_12BITS_COMPARE = 1u }

MAC VLAN compare mode type.

enum **ENET_MAC_VLANFilterGroupType** { **ENET_MAC_VLAN_FILTER_GROUP_0** = 0u,
ENET_MAC_VLAN_FILTER_GROUP_1 = 1u, **ENET_MAC_VLAN_FILTER_GROUP_2** = 2u,
ENET_MAC_VLAN_FILTER_GROUP_3 = 3u }

MAC VLAN perfect filter group index type.

enum **ENET_MAC_PauseThresholdType** { **ENET_MAC_PAUSE_THRESHOLD_4SLOT_TIMES** = 0u,
ENET_MAC_PAUSE_THRESHOLD_28SLOT_TIMES = 1u, **ENET_MAC_PAUSE_THRESHOLD_36SLOT_TIMES**
= 2u, **ENET_MAC_PAUSE_THRESHOLD_144SLOT_TIMES** = 3u,
ENET_MAC_PAUSE_THRESHOLD_256SLOT_TIMES = 4u,
ENET_MAC_PAUSE_THRESHOLD_512SLOT_TIMES = 5u }

ENET flow control pause threshold type.

enum **ENET_MAC_TxPacketControllerStatusType** { **ENET_MAC_TX_PACKET_CONTROLLER_IDLE** = 0u,
ENET_MAC_TX_PACKET_CONTROLLER_WAIT = 1u,

ENET_MAC_TX_PACKET_CONTROLLER_TRANSFER_PAUSE_PACKET = 2u,
ENET_MAC_TX_PACKET_CONTROLLER_TRANSFER_INPUT_PACKET = 3u }

MAC TX packet controller status type.

enum ENET_MTL_QueueEnableType { ENET_MTL_QUEUE_DISABLE = 0u,
ENET_MTL_QUEUE_ENABLE_AV_MODE = 1u, ENET_MTL_QUEUE_ENABLE_GENERIC = 2u }

MTL queue enable type.

enum ENET_MAC_PTPOEQueueCtrlType { ENET_MAC_VLAN_PTPOE_ROUTED_AS_GENERIC = 0u,
ENET_MAC_VLAN_PTPOE_ROUTED_TO_PTP_QUEUE = 1u,
ENET_MAC_VLAN_PTPOE_ROUTED_BY_PRIORITY = 2u }

Tagged PTP over Ethernet packets queuing control type.

enum ENET_MAC_RGMIIclkSelType { ENET_MAC_RGMII_CLK_SEL_DISABLE = 0u,
ENET_MAC_RGMII_CLK_SEL_ENET_REF_CLK = 1u, ENET_MAC_RGMII_CLK_SEL_PLL0DIVH = 2u,
ENET_MAC_RGMII_CLK_SEL_PLL1DIVH = 3u }

MAC RGMII clock select type.

enum ENET_MAC_TxClkSelType { ENET_MAC_TX_CLK_SEL_DISABLE = 0u,
ENET_MAC_TX_CLK_SEL_PLL0DIVL = 1u, ENET_MAC_TX_CLK_SEL_FOSC = 2u,
ENET_MAC_TX_CLK_SEL_ENET_TX_CLK = 3u }

MAC TX clock select type.

enum ENET_MAC_MiiModeType { ENET_MAC_PHY_MII = 0u, ENET_MAC_PHY_RGMII = 1u,
ENET_MAC_PHY_RMII = 4u }

MAC PHY interface mode type.

enum ENET_MAC_CSRClockRangeType { ENET_MAC_CSR_CLOCK_RANGE_60_100MHZ = 0u,
ENET_MAC_CSR_CLOCK_RANGE_100_150MHZ = 1u, ENET_MAC_CSR_CLOCK_RANGE_20_35MHZ = 2u,
ENET_MAC_CSR_CLOCK_RANGE_35_60MHZ = 3u, ENET_MAC_CSR_CLOCK_RANGE_150_250MHZ = 4u,
ENET_MAC_CSR_CLOCK_RANGE_250_300MHZ = 5u }

MAC MDIO access clock source range type.

enum ENET_MAC_PhyOperationCmdType { ENET_MAC_PHY_OPERATION_WRITE = 1u,
ENET_MAC_PHY_OPERATION_POST_READ_INCR_ADDR_FOR_CLAUSE_45 = 2u,
ENET_MAC_PHY_OPERATION_READ = 3u }

MAC MDIO bus operation command type.

enum ENET_MAC_PhyClauseVersionType { ENET_MAC_PHY_CLAUSE_22 = 0u,
ENET_MAC_PHY_CLAUSE_45 = 1u }

MAC MDIO bus clause type.

enum ENET_MAC_AddrType { ENET_MAC_ADDR_TYPE_DEST = 0u, ENET_MAC_ADDR_TYPE_SRC = 1u }

MAC address destination or source type.

enum **ENET_MAC_MmcCntPresetLevelType** { **ENET_MAC_MMC_COUNTER_PRESET_LEVEL_HALF** = 0u,
ENET_MAC_MMC_COUNTER_PRESET_LEVEL_FULL = 1u }

MMC counter present level type.

enum **ENET_MAC_Layer4ProtocolType** { **ENET_MAC_LAYER_4_PROTOCOL_TCP** = 0u,
ENET_MAC_LAYER_4_PROTOCOL_UDP = 1u }

MAC layer 4 filter protocol type.

enum **ENET_MAC_Layer3ProtocolType** { **ENET_MAC_LAYER_3_PROTOCOL_IPV4** = 0u,
ENET_MAC_LAYER_3_PROTOCOL_IPV6 = 1u }

MAC layer 3 filter protocol type.

enum **ENET_MAC_L3L4FilterGroupType** { **ENET_MAC_L3_L4_FILTER_GROUP_0** = 0u,
ENET_MAC_L3_L4_FILTER_GROUP_1 = 1u, **ENET_MAC_L3_L4_FILTER_GROUP_2** = 2u,
ENET_MAC_L3_L4_FILTER_GROUP_3 = 3u }

MAC layer3 layer 4 filter group index type.

enum **Enet_MMCTxCounterType** { **ENET_CTR_TX_OCTET_COUNT_GOOD_BAD** = 0u,
ENET_CTR_TX_PACKET_COUNT_GOOD_BAD, **ENET_CTR_TX_BROADCAST_PACKETS_GOOD**,
ENET_CTR_TX_MULTICAST_PACKETS_GOOD, **ENET_CTR_TX_64OCTETS_PACKETS_GOOD_BAD**,
ENET_CTR_TX_65TO127OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_128TO255OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_256TO511OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_512TO1023OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_1024TOMAXOCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_UNICAST_PACKETS_GOOD_BAD, **ENET_CTR_TX_MULTICAST_PACKETS_GOOD_BAD**,
ENET_CTR_TX_BROADCAST_PACKETS_GOOD_BAD, **ENET_CTR_TX_UNDERFLOW_ERROR_PACKETS**,
ENET_CTR_TX_SINGLE_COLLISION_GOOD_PACKETS,
ENET_CTR_TX_MULTIPLE_COLLISION_GOOD_PACKETS, **ENET_CTR_TX_DEFERRED_PACKETS**,
ENET_CTR_TX_LATE_COLLISION_PACKETS, **ENET_CTR_TX_EXCESSIVE_COLLISION_PACKETS**,
ENET_CTR_TX_CARRIER_ERROR_PACKETS, **ENET_CTR_TX_OCTET_COUNT_GOOD**,
ENET_CTR_TX_PACKET_COUNT_GOOD, **ENET_CTR_TX_EXCESSIVE_DEFERRAL_ERROR**,
ENET_CTR_TX_PAUSE_PACKETS, **ENET_CTR_TX_VLAN_PACKETS_GOOD**,
ENET_CTR_TX_OSIZE_PACKETS_GOOD }

MAC MMC transmit statistics counter type.

enum **Enet_MMCRxCounterType** { **ENET_CTR_RX_PACKETS_COUNT_GOOD_BAD** = 0u,
ENET_CTR_RX_OCTET_COUNT_GOOD_BAD, **ENET_CTR_RX_OCTET_COUNT_GOOD**,
ENET_CTR_RX_BROADCAST_PACKETS_GOOD, **ENET_CTR_RX_MULTICAST_PACKETS_GOOD**,
ENET_CTR_RX_CRC_ERROR_PACKETS, **ENET_CTR_RX_ALIGNMENT_ERROR_PACKETS**,
ENET_CTR_RX_RUNT_ERROR_PACKETS, **ENET_CTR_RX_JABBER_ERROR_PACKETS**,
ENET_CTR_RX_UNDERSIZE_PACKETS_GOOD, **ENET_CTR_RX_OVERSIZE_PACKETS_GOOD**,

ENET_CTR_RX_64OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_65TO127OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_128TO255OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_256TO511OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_512TO1023OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_1024TOMAXOCTETS_PACKETS_GOOD_BAD, ENET_CTR_RX_UNICAST_PACKETS_GOOD,
ENET_CTR_RX_LENGTH_ERROR_PACKETS, ENET_CTR_RX_OUT_OF_RANGE_TYPE_PACKETS,
ENET_CTR_RX_PAUSE_PACKETS, ENET_CTR_RX_FIFO_OVERFLOW_PACKETS,
ENET_CTR_RX_VLAN_PACKETS_GOOD_BAD, ENET_CTR_RX_WATCHDOG_ERROR_PACKETS,
ENET_CTR_RX_RECEIVE_ERROR_PACKETS, ENET_CTR_RX_CONTROL_PACKETS_GOOD }

MAC receive statistics counter type.

enum ENET_MAC_TxTimestampStatusModeType
{ ENET_MAC_TX_TIMESTAMP_STATUS_MODE_PRESERVE = 0u,
ENET_MAC_TX_TIMESTAMP_STATUS_MODE_OVERWRITE = 1u }

PTP TX status mode type.

enum ENET_MAC_SystemTimeInputType { ENET_MAC_SYSTEM_TIME_INPUT_INTERNAL = 0u,
ENET_MAC_SYSTEM_TIME_INPUT_EXTERNAL = 1u }

PTP system time input source type.

enum ENET_MAC_PTPVersionType { ENET_MAC_PTP_IEEE_1588_V1 = 0u,
ENET_MAC_PTP_IEEE_1588_V2 = 1u }

PTP IEEE1588 version type.

enum ENET_MAC_TimestampRolloverType { ENET_MAC_TIMESTAMP_ROLLOVER_BINARY = 0u,
ENET_MAC_TIMESTAMP_ROLLOVER_DIGITAL = 1u }

PTP counter rollover mode type.

enum ENET_MAC_TimestampUpdateMethodType { ENET_MAC_TIMESTAMP_UPDATE_COARSE = 0u,
ENET_MAC_TIMESTAMP_UPDATE_FINE = 1u }

PTP timestamp update mode type.

enum ENET_MAC_SystemTimeOperationType { ENET_MAC_SYSTEM_TIME_ADD_TIME = 0u,
ENET_MAC_SYSTEM_TIME_SUBTRACT_TIME = 1u }

PTP system time offset update operation type.

enum ENET_MAC_PPSChannelType { ENET_MAC_PPS_CHANNEL_0 = 0u, ENET_MAC_PPS_CHANNEL_1 =
1u, ENET_MAC_PPS_CHANNEL_2 = 2u, ENET_MAC_PPS_CHANNEL_3 = 3u }

PTP PPS channel type.

enum ENET_MAC_PPSModeType { ENET_MAC_PPS_MODE_PPS = 0u, ENET_MAC_PPS_MODE_MCGR =
1u }

PTP PPS channel operation mode type.

```
enum ENET_MAC_PPSTargetTimeModeType { ENET_MAC_PPS_TARGET_TIME_ONLY_INT = 0u,
ENET_MAC_PPS_TARGET_TIME_MCGR = 1u, ENET_MAC_PPS_TARGET_TIME_INT_ST = 2u,
ENET_MAC_PPS_TARGET_TIME_ONLY_ST = 3u }
```

PPS target time mode type.

```
enum ENET_MAC_PPSFreqCtrlType { ENET_MAC_PPS_BINARY_1HZ_DIGITAL_NONE = 0u,
ENET_MAC_PPS_BINARY_2HZ_DIGITAL_1HZ = 1u, ENET_MAC_PPS_BINARY_4HZ_DIGITAL_2HZ = 2u,
ENET_MAC_PPS_BINARY_8HZ_DIGITAL_4HZ = 3u, ENET_MAC_PPS_BINARY_16HZ_DIGITAL_8HZ = 4u,
ENET_MAC_PPS_BINARY_32HZ_DIGITAL_16HZ = 5u, ENET_MAC_PPS_BINARY_64HZ_DIGITAL_32HZ =
6u, ENET_MAC_PPS_BINARY_128HZ_DIGITAL_64HZ = 7u,
ENET_MAC_PPS_BINARY_256HZ_DIGITAL_128HZ = 8u,
ENET_MAC_PPS_BINARY_512HZ_DIGITAL_256HZ = 9u,
ENET_MAC_PPS_BINARY_1024HZ_DIGITAL_512HZ = 10u,
ENET_MAC_PPS_BINARY_2048HZ_DIGITAL_1024HZ = 11u,
ENET_MAC_PPS_BINARY_4096HZ_DIGITAL_2048HZ = 12u,
ENET_MAC_PPS_BINARY_8192HZ_DIGITAL_4096HZ = 13u,
ENET_MAC_PPS_BINARY_16384HZ_DIGITAL_8192HZ = 14u,
ENET_MAC_PPS_BINARY_32768HZ_DIGITAL_16384HZ = 15u }
```

PTP fixed PPS output frequency control type.

```
enum ENET_MAC_PPSCmdType { ENET_MAC_PPS_NO_COMMAND = 0u,
ENET_MAC_PPS_START_SINGLE_PULSE = 1u, ENET_MAC_PPS_START_PULSE_TRAIN = 2u,
ENET_MAC_PPS_CANCEL_START = 3u, ENET_MAC_PPS_STOP_PULSE_TRAIN_AT_TIME = 4u,
ENET_MAC_PPS_STOP_PULSE_TRAIN_IMMEDIATELY = 5u,
ENET_MAC_PPS_CANCEL_STOP_PULSE_TRAIN = 6u }
```

PTP flexible PPS output command type.

```
enum ENET_MAC_MCGRCmdType { ENET_MAC_MCGR_OPERATION_NOT_CARRIED_OUT = 0u,
ENET_MAC_MCGR_CAPTURE_RISING_EDGE = 1u, ENET_MAC_MCGR_CAPTURE_FALLING_EDGE = 2u,
ENET_MAC_MCGR_CAPTURE_BOTH_EDGE = 3u, ENET_MAC_MCGR_TOGGLE_OUTPUT_ON_COMPARE =
9u, ENET_MAC_MCGR_PULSE_OUTPUT_LOW_ON_COMPARE = 10u,
ENET_MAC_MCGR_PULSE_OUTPUT_HIGH_ON_COMPARE = 11u }
```

PTP flexible MCGR command type.

```
enum ENET_MAC_PTPSnapshotMsgType { ENET_MAC_PTP_SNAPSHOT_FOR_E2E = (0x0u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_SYNC = (0x1u << 14u), ENET_MAC_PTP_SNAPSHOT_FOR_DELAYREQ =
(0x3u << 14u), ENET_MAC_PTP_SNAPSHOT_FOR_ALL_PTP = (0x4u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_SLAVE_NODE_EVENT = (0x5u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_MASTER_NODE_EVENT = (0x7u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_E2E_EVENT = (0x8u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_P2P_EVENT = (0xCu << 14u) }
```

PTP snapshot packet select type.

```
enum ENET_MAC_PTTPacketLayerType { ENET_MAC_PTP_OVER_ETHERNET =  
ENET_MAC_TIMESTAMP_CONTROL_TSIPENA_MASK, ENET_MAC_PTP_OVER_IPV4 =  
ENET_MAC_TIMESTAMP_CONTROL_TSIPV4ENA_MASK, ENET_MAC_PTP_OVER_IPV6 =  
ENET_MAC_TIMESTAMP_CONTROL_TSIPV6ENA_MASK }
```

PTP snapshot network layer select type.

```
enum ENET_MAC_PTPClkSelType { ENET_MAC_PTP_CLK_SEL_PCC = 0u,  
ENET_MAC_PTP_CLK_SEL_TX_CLK = 1u }
```

MAC PTP clock select type.

```
enum ENET_MAC_PPSPinDirType { ENET_PTP_PPS_DIR_INPUT = 0u, ENET_PTP_PPS_DIR_OUTPUT =  
1u }
```

PTP PPS channel direction type.

```
enum ENET_MTL_QueueType { ENET_MTL_QUEUE_0 = 0u, ENET_MTL_QUEUE_1 = 1u }
```

ENET MTL queue type.

```
enum ENET_MTL_TxScheduleAlgorithmType { ENET_MTL_SCHED_ALGO_WRR = 0u,  
ENET_MTL_SCHED_ALGO_SP = 3u }
```

ENET TX schedule algorithm type.

```
enum ENET_MTL_RxArbitrationAlgorithmType { ENET_MTL_RX_ARBITRATION_STRICT_POLICY = 0u,  
ENET_MTL_RX_ARBITRATION_WEIGHTED_STRICT_POLICY = 1u }
```

ENET RX arbitration type.

```
enum ENET_MTL_DebugAccessFIFOType { ENET_MTL_DEBUG_ACCESS_TX_FIFO = 0u,  
ENET_MTL_DEBUG_ACCESS_TX_STATUS_FIFO = 1u, ENET_MTL_DEBUG_ACCESS_TSO_FIFO = 2u,  
ENET_MTL_DEBUG_ACCESS_RX_FIFO = 3u }
```

MTL debug access type.

```
enum ENET_MTL_FIFOPacketStatusType { ENET_MTL_PACKET_STATE_PACKET_DATA = 0u,  
ENET_MTL_PACKET_STATE_TX_CTRL_WORD_RX_NORMAL_STATUS = 1u,  
ENET_MTL_PACKET_STATE_TX_SOP_DATA_RX_LAST_STATUS = 2u,  
ENET_MTL_PACKET_STATE_TX_EOP_DATA_RX_EOP = 3u }
```

MTL fifo packet status type.

```
enum ENET_MTL_ByteEnableType { ENET_MTL_BYTE_0_VALID = 0u, ENET_MTL_BYTE_0_1_VALID = 1u,  
ENET_MTL_BYTE_0_1_2_VALID = 2u, ENET_MTL_BYTE_ALL_VALID = 3u }
```

MTL byte enable type.

```
enum ENET_MTL_FIFODebugAccessType { ENET_MTL_FIFO_DEBUG_ACCESS_RESTRICTED = 0u,  
ENET_MTL_FIFO_DEBUG_ACCESS_UNRESTRICTED = 1u }
```

MTL fifo debug access type.

```
enum ENET_MTL_TxThresholdType { ENET_MTL_TX_THRESHOLD_32B = 0u,
ENET_MTL_TX_THRESHOLD_64B = 1u, ENET_MTL_TX_THRESHOLD_96B = 2u,
ENET_MTL_TX_THRESHOLD_128B = 3u, ENET_MTL_TX_THRESHOLD_192B = 4u,
ENET_MTL_TX_THRESHOLD_256B = 5u, ENET_MTL_TX_STORE_FORWARD = (1u <<
ENET_MTL_TXQn_OPERATION_MODE_TTC_WIDTH) }
```

MTL TX threshold type.

```
enum ENET_MTL_WriteControllerStatusType { ENET_MTL_WRITE_CONTROLLER_IDLE = 0u,
ENET_MTL_WRITE_CONTROLLER_ACTIVE = 1u }
```

MTL write controller status type.

```
enum ENET_MTL_ReadControllerStatusType { ENET_MTL_READ_CONTROLLER_IDLE = 0u,
ENET_MTL_READ_CONTROLLER_READ = 1u, ENET_MTL_READ_CONTROLLER_WAIT = 2u,
ENET_MTL_READ_CONTROLLER_FLUSH = 3u }
```

MTL read controller status type.

```
enum ENET_MTL_RxFlowCtrlThresholdType { ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_1_KB =
0u, ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_1_5_KB = 1u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_2_KB = 2u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_2_5_KB = 3u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_3_KB = 4u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_3_5_KB = 5u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_4_KB = 6u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_4_5_KB = 7u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_5_KB = 8u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_5_5_KB = 9u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_6_KB = 10u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_6_5_KB = 11u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_7_KB = 12u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_7_5_KB = 13u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_8_KB = 14u,
ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_8_5_KB = 15u }
```

MTL RX flow control threshold type.

```
enum ENET_MTL_RxThresholdType { ENET_MTL_RX_THRESHOLD_64B = 0u,
ENET_MTL_RX_THRESHOLD_32B = 1u, ENET_MTL_RX_THRESHOLD_96B = 2u,
ENET_MTL_RX_THRESHOLD_128B = 3u, ENET_MTL_RX_STORE_FORWARD = (1u <<
ENET_MTL_RXQn_OPERATION_MODE_RTC_WIDTH) }
```

MTL RX threshold type.

```
enum ENET_MTL_RxQueueStatusType { ENET_MTL_RX_QUEUE_EMPTY = 0u,  
ENET_MTL_RX_QUEUE_BELOW_FLOW_CTRL_DEACTIVATE_THRES = 1u,  
ENET_MTL_RX_QUEUE_ABOVE_FLOW_CTRL_ACTIVATE_THRES = 2u, ENET_MTL_RX_QUEUE_FULL = 3u }
```

MTL RX queue status type.

```
enum ENET_MTL_GCLIntervalMultipleType { ENET_MTL_GCL_INTERVAL_X_1 = 0x0u,  
ENET_MTL_GCL_INTERVAL_X_2 = 0x1u, ENET_MTL_GCL_INTERVAL_X_4 = 0x2u,  
ENET_MTL_GCL_INTERVAL_X_8 = 0x3u, ENET_MTL_GCL_INTERVAL_X_16 = 0x4u,  
ENET_MTL_GCL_INTERVAL_X_32 = 0x5u, ENET_MTL_GCL_INTERVAL_X_64 = 0x6u,  
ENET_MTL_GCL_INTERVAL_X_128 = 0x7u }
```

Multiple of gate control list interval time.

```
enum ENET_MTL_ScheduleErrorLoopCountType { ENET_MTL_GCL_4_ITERATIONS = 0x0u,  
ENET_MTL_GCL_8_ITERATIONS = 0x1u, ENET_MTL_GCL_16_ITERATIONS = 0x2u,  
ENET_MTL_GCL_32_ITERATIONS = 0x3u }
```

Loop count to report Scheduling Error.

```
enum ENET_MTL_GclRegistersType { ENET_MTL_GCL_BTR_LOW = 0x0u, ENET_MTL_GCL_BTR_HIGH =  
0x1u, ENET_MTL_GCL_CTR_LOW = 0x2u, ENET_MTL_GCL_CTR_HIGH = 0x3u, ENET_MTL_GCL_TER = 0x4u,  
ENET_MTL_GCL_LLR = 0x5 }
```

Gcl related registers.

```
enum ENET_MTL_GclOperationType { ENET_MTL_GCL_WRITE = 0x0u, ENET_MTL_GCL_READ = 0x1u }
```

Gcl operation type.

```
enum ENET_MTL_GateControlListType { ENET_MTL_GCL_NUMBER_0 = 0x0u,  
ENET_MTL_GCL_NUMBER_1 = 0x1u }
```

MTL gate control list index type.

```
enum ENET_DMA_ChannelType { ENET_DMA_CHANNEL_0 = 0x0u, ENET_DMA_CHANNEL_1 = 0x1u }
```

ENET DMA channel index type.

```
enum ENET_DMA_StatusType { ENET_DMA_STATUS_IDLE = 0x0u, ENET_DMA_STATUS_ACTIVE = 0x1u }
```

ENET DMA running status.

```
enum ENET_DMA_IntModeType { ENET_DMA_INTM_SBD_PERCH_PULSE = 0u,  
ENET_DMA_INTM_SBD_PERCH_LEVEL = 1u, ENET_DMA_INTM_SBD_PERCH_LEVEL_REENTRANT = 2u }
```

The interrupt mode of ENET DMA.

```
enum ENET_DMA_PriorityRatioType { ENET_DMA_PRIORITY_RATIO_1_1 = 0u,  
ENET_DMA_PRIORITY_RATIO_2_1 = 1u, ENET_DMA_PRIORITY_RATIO_3_1 = 2u,  
ENET_DMA_PRIORITY_RATIO_4_1 = 3u, ENET_DMA_PRIORITY_RATIO_5_1 = 4u,  
ENET_DMA_PRIORITY_RATIO_6_1 = 5u, ENET_DMA_PRIORITY_RATIO_7_1 = 6u,  
ENET_DMA_PRIORITY_RATIO_8_1 = 7u }
```

ENET DMA priority ratio type.

```
enum ENET_DMA_PriorityModeType { ENET_DMA_PRIORITY_MODE_RX_TX = 0u,  
ENET_DMA_PRIORITY_MODE_TX_RX = 1u }
```

ENET DMA priority mode type.

```
enum ENET_DMA_TxArbitrationAlgorithmType  
{ ENET_DMA_TX_ARBITRATION_ALGORITHM_FIXED_PRIORITY = 0u,  
ENET_DMA_TX_ARBITRATION_ALGORITHM_WEIGHTED_FIXED_PRIORITY = 1u,  
ENET_DMA_TX_ARBITRATION_ALGORITHM_WEIGHTED_ROUND_ROBIN = 2u }
```

ENET DMA arbitration algorithm between transmit channels.

```
enum ENET_DMA_ArbitrationSchemeType  
{ ENET_DMA_TXRX_ARBITRATION_SCHEME_WEIGHTED_ROUND_ROBIN = 0u,  
ENET_DMA_TXRX_ARBITRATION_SCHEME_FIXED_PRIORITY = 1u }
```

ENET DMA arbitration scheme between the transmit and receive paths.

```
enum ENET_DMA_BurstModeType { ENET_BURSTLENGTH_FIXED =  
ENET_DMA_SYSBUS_MODE_FB_MASK, ENET_BURSTLENGTH_MIXED =  
ENET_DMA_SYSBUS_MODE_MB_MASK }
```

ENET DMA burst mode type.

```
enum ENET_DMA_InterruptFlagType { ENET_DMA_TX_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_TIE_MASK, ENET_DMA_TX_STOPPED_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_TXSE_MASK, ENET_DMA_TX_BUFFER_UNAVAILABLE_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_TBUE_MASK, ENET_DMA_RX_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_RIE_MASK, ENET_DMA_RX_BUFFER_UNAVAILABLE_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_RBUE_MASK, ENET_DMA_RX_STOPPED_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_RSE_MASK, ENET_DMA_RX_WATCHDOG_TIMEOUT_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_RWTE_MASK, ENET_DMA_EARLY_TX_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_ETIE_MASK, ENET_DMA_EARLY_RX_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_ERIE_MASK, ENET_DMA_FATAL_BUS_ERROR_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_FBEE_MASK,  
ENET_DMA_CONTEXT_DESCRIPTOR_ERROR_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_CDEE_MASK, ENET_DMA_ABNORMAL_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_AIE_MASK, ENET_DMA_NORMAL_INTERRUPT =  
ENET_DMA_CHn_INTERRUPT_ENABLE_NIE_MASK }
```

ENET DMA interrupt flag type.

```
enum ENET_DMA_WDGCounterUnitType { ENET_DMA_WDG_COUNTER_UNIT_256_CYCLE = 0x0u,  
ENET_DMA_WDG_COUNTER_UNIT_512_CYCLE = 0x1u, ENET_DMA_WDG_COUNTER_UNIT_1024_CYCLE =  
0x2u, ENET_DMA_WDG_COUNTER_UNIT_2048_CYCLE = 0x3u }
```

ENET DMA watchdog counter unit type.

```
enum ENET_DMA_RxErrorType { ENET_DMA_RX_NO_ERROR = 0x0u,  
ENET_DMA_RX_READ_DESCRIPTOR_ERROR = 0x7u, ENET_DMA_RX_WRITE_DESCRIPTOR_ERROR = 0x6u,  
ENET_DMA_RX_READ_BUFFER_ERROR = 0x5u, ENET_DMA_RX_WRITE_BUFFER_ERROR = 0x4u }
```

ENET DMA RX error type.

```
enum ENET_DMA_TxErrorType { ENET_DMA_TX_NO_ERROR = 0x0u,  
ENET_DMA_TX_READ_DESCRIPTOR_ERROR = 0x7u, ENET_DMA_TX_WRITE_DESCRIPTOR_ERROR = 0x6u,  
ENET_DMA_TX_READ_BUFFER_ERROR = 0x5u, ENET_DMA_TX_WRITE_BUFFER_ERROR = 0x4u }
```

ENET DMA TX error type.

Functions

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetConfiguration (const ENET_Type *const pEnet)
```

Get the MAC Configuration Register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetConfiguration (ENET_Type *const pEnet, uint32_t u32Config)
```

Set the MAC Configuration Register value.

```
LOCAL_INLINE ENET_MAC_SrcAddrCtrlType ENET_HWA_MAC_GetSrcAddrInsertReplaceCtrl (const  
ENET_Type *const pEnet)
```

Get the source address insertion or replacement control type.

```
LOCAL_INLINE void ENET_HWA_MAC_SetSrcAddrInsertReplaceCtrl (ENET_Type *const pEnet,  
ENET_MAC_SrcAddrCtrlType eInsertReplaceCtrl)
```

Set the source address insertion or replacement control type.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetIPChecksumOffloadEnFlag (const ENET_Type *const pEnet)
```

Get checksum offload function enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetIPChecksumOffloadEnFlag (ENET_Type *const pEnet, bool  
bEnable)
```

Set checksum offload function enable flag.

```
LOCAL_INLINE ENET_MAC_InterPacketGapType ENET_HWA_MAC_GetMinInterPacketGap (const  
ENET_Type *const pEnet)
```

Get the minimum Inter-Packet Gap.

LOCAL_INLINE void **ENET_HWA_MAC_SetMinInterPacketGap** (ENET_Type *const pEnet,
ENET_MAC_InterPacketGapType eGap)
Set the minimum Inter-Packet Gap value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetGiantPacketSizeLimitCtrlEnFlag** (const ENET_Type *const
pEnet)
Get giant packet size Limit control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetGiantPacketSizeLimitCtrlEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set giant packet size Limit control enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_Get2KPacketSupportFlag** (const ENET_Type *const pEnet)
Get 2K packet support enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_Set2KPacketSupportFlag** (ENET_Type *const pEnet, bool bEnable)
Set 2K packet support enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetStripTypePacketCRCEnFlag** (const ENET_Type *const pEnet)
Get CRC stripping for type packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetStripTypePacketCRCEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set CRC stripping for type packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetAutoStripPadOrCRCEnFlag** (const ENET_Type *const pEnet)
Get automatic Pad or CRC stripping enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetAutoStripPadOrCRCEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set automatic Pad or CRC stripping enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetWatchdogDisableFlag** (const ENET_Type *const pEnet)
Get receiver watchdog timer disabled flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetWatchdogDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set receiver watchdog timer disabled flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetJabberTimerDisableFlag** (const ENET_Type *const pEnet)

Get transmitter jabber timer disabled flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetJabberTimerDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set transmitter jabber timer disabled flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetJumboPacketEnFlag** (const ENET_Type *const pEnet)

Get jumbo packet enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetJumboPacketEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set jumbo packet enable flag.

LOCAL_INLINE **ENET_MAC_SpeedType** **ENET_HWA_MAC_GetSpeed** (const ENET_Type *const pEnet)

Get ethernet line speed.

LOCAL_INLINE void **ENET_HWA_MAC_SetSpeed** (ENET_Type *const pEnet, **ENET_MAC_SpeedType** eSpeed)

Set ethernet line speed.

LOCAL_INLINE **ENET_MAC_DuplexModeType** **ENET_HWA_MAC_GetDuplexMode** (const ENET_Type *const pEnet)

Get ethernet duplex mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetDuplexMode** (ENET_Type *const pEnet, **ENET_MAC_DuplexModeType** eMode)

Set ethernet duplex mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLoopbackModeEnFlag** (const ENET_Type *const pEnet)

Get loopback mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLoopbackModeEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set loopback mode enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetFullDuplexCRSEnFlag** (const ENET_Type *const pEnet)
Get carrier sense before transmission enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetFullDuplexCRSEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set carrier sense before transmission enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetReceiveOwnDisableFlag** (const ENET_Type *const pEnet)
Get receive own disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetReceiveOwnDisableFlag** (ENET_Type *const pEnet, bool bDisable)
Set receive own disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetHalfDuplexCRSDisableFlag** (const ENET_Type *const pEnet)
Get carrier sense during transmission disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetHalfDuplexCRSDisableFlag** (ENET_Type *const pEnet, bool bDisable)
Set carrier sense during transmission disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRetryDisableFlag** (const ENET_Type *const pEnet)
Get transmission retry disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRetryDisableFlag** (ENET_Type *const pEnet, bool bDisable)
Set transmission retry disable flag.

LOCAL_INLINE **ENET_MAC_BackOffLimitType** **ENET_HWA_MAC_GetBackOffLimit** (const ENET_Type *const pEnet)
Get back-off limit.

LOCAL_INLINE void **ENET_HWA_MAC_SetBackOffLimit** (ENET_Type *const pEnet,
ENET_MAC_BackOffLimitType eLimit)

Set back-off limit.

LOCAL_INLINE bool **ENET_HWA_MAC_GetDeferralCheckEnFlag** (const ENET_Type *const pEnet)

Get deferral check enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetDeferralCheckEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set deferral check enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxEnFlag** (const ENET_Type *const pEnet)

Get transmitter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set transmitter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxEnFlag** (const ENET_Type *const pEnet)

Get receiver enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set receiver enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetExtConfiguration** (const ENET_Type *const pEnet)

Get the MAC extended configuration register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetExtConfiguration** (ENET_Type *const pEnet, uint32_t
u32Configuration)

Set the MAC extended configuration register value.

LOCAL_INLINE **ENET_MAC_ExtInterPacketGapType** **ENET_HWA_MAC_GetExtInterPacketGap** (const
ENET_Type *const pEnet)

Get the MAC extended inter-packet gap.

LOCAL_INLINE void **ENET_HWA_MAC_SetExtInterPacketGap** (ENET_Type *const pEnet,
ENET_MAC_ExtInterPacketGapType eGap)
Set the MAC extended inter-packet gap.

LOCAL_INLINE bool **ENET_HWA_MAC_GetExtInterPacketGapEnFlag** (const ENET_Type *const pEnet)
Get extended inter-packet gap enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetExtInterPacketGapEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set extended inter-packet gap enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPacketDuplicationCtrlEnFlag** (const ENET_Type *const pEnet)
Get packet duplication control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPacketDuplicationCtrlEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set packet duplication control enable flag.

LOCAL_INLINE **ENET_MAC_SlowProtocolDetectType**
ENET_HWA_MAC_GetUnicastSlowProtocolPacketDetectMethod (const ENET_Type *const pEnet)
Get unicast slow protocol packet detect method.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastSlowProtocolPacketDetectMethod** (ENET_Type *const
pEnet, **ENET_MAC_SlowProtocolDetectType** eMethod)
Set unicast slow protocol packet detect method.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSlowDetectEnFlag** (const ENET_Type *const pEnet)
Get slow protocol detection enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSlowDetectEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set slow protocol detection enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxCRCCheckDisableFlag** (const ENET_Type *const pEnet)
Get CRC checking for Received packets disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxCRCCheckDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set CRC checking for rReceived packets disable flag.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetGiantPacketSizeLimit** (const ENET_Type *const pEnet)

Get giant packet size limit value.

LOCAL_INLINE void **ENET_HWA_MAC_SetGiantPacketSizeLimit** (ENET_Type *const pEnet, uint16_t u16SizeLimit)

Set giant packet size limit value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPacketFilter** (const ENET_Type *const pEnet)

Get the MAC packet filter register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetPacketFilter** (ENET_Type *const pEnet, uint32_t u32FilterCtrl)

Set the MAC packet filter register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetReceiveAllEnFlag** (const ENET_Type *const pEnet)

Get receive all packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetReceiveAllEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set receive all packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetDropNonTCPUDPOverIPEnFlag** (const ENET_Type *const pEnet)

Get drop non-TCP/UDP over IP packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetDropNonTCPUDPOverIPEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set drop non-TCP/UDP over IP packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3Layer4FilterEnFlag** (const ENET_Type *const pEnet)

Get layer 3 and layer 4 filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Layer4FilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set layer 3 and layer 4 filter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagFilterEnFlag** (const ENET_Type *const pEnet)

Get VLAN tag filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterFilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag filter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetHashOrPerfectFilterEnFlag** (const ENET_Type *const pEnet)

Get hash or perfect filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetHashOrPerfectFilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set hash or perfect filter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSourceAddressFilterEnFlag** (const ENET_Type *const pEnet)

Get source address filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSourceAddressFilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set source address filter enable flag.

LOCAL_INLINE **ENET_MAC_AddrFilterModeType** **ENET_HWA_MAC_GetSrcAddrFilterMode** (const ENET_Type *const pEnet)

Get source address filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetSrcAddrFilterMode** (ENET_Type *const pEnet, **ENET_MAC_AddrFilterModeType** eFilterMode)

Set source address filter mode.

LOCAL_INLINE **ENET_MAC_CtrlPacketFilterModeType** **ENET_HWA_MAC_GetCtrlPacketFilterMode** (const ENET_Type *const pEnet)

Get control packets filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetCtrlPacketFilterMode** (ENET_Type *const pEnet, **ENET_MAC_CtrlPacketFilterModeType** eFilterMode)

Set control packets filter mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetBroadcastPacketDisableFlag** (const ENET_Type *const pEnet)

Get broadcast packets disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetBroadcastPacketDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set broadcast packets disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPassAllMulticastEnFlag** (const ENET_Type *const pEnet)

Get pass all the received multicast packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPassAllMulticastEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set pass all the received multicast packets enable flag.

LOCAL_INLINE **ENET_MAC_AddrFilterModeType** **ENET_HWA_MAC_GetDestAddrFilterMode** (const ENET_Type *const pEnet)

Get destination address filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetDestAddrFilterMode** (ENET_Type *const pEnet, **ENET_MAC_AddrFilterModeType** eFilterMode)

Set destination address filter mode.

LOCAL_INLINE **ENET_MAC_HashOrPerfectModeType**

ENET_HWA_MAC_GetHashOrPerfectMulticastMode (const ENET_Type *const pEnet)

Get hash or perfect multicast filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetHashOrPerfectMulticastMode** (ENET_Type *const pEnet, **ENET_MAC_HashOrPerfectModeType** eMode)

Set hash or perfect multicast filter mode.

LOCAL_INLINE **ENET_MAC_HashOrPerfectModeType** **ENET_HWA_MAC_GetHashOrPerfectUnicastMode** (const ENET_Type *const pEnet)

Get hash or perfect unicast filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetHashOrPerfectUnicastMode** (ENET_Type *const pEnet, **ENET_MAC_HashOrPerfectModeType** eMode)

Set hash or perfect unicast filter mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPromiscuousModeEnFlag** (const ENET_Type *const pEnet)

Get promiscuous mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPromiscuousModeEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set promiscuous mode enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetWatchdogTimeout** (const ENET_Type *const pEnet)

Get programmable watchdog timeout register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetWatchdogTimeout** (ENET_Type *const pEnet, uint32_t u32Settings)

Set programmable watchdog timeout register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetWatchdogEnFlag** (const ENET_Type *const pEnet)

Get programmable watchdog enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetWatchdogEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set programmable watchdog enable flag.

LOCAL_INLINE **ENET_MAC_WDGTimerSizeType** **ENET_HWA_MAC_GetWatchdogTimerSize** (const ENET_Type *const pEnet)

Get programmable watchdog timer size value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetWatchdogTimerSize (ENET_Type *const pEnet,  
ENET_MAC_WDGTimerSizeType eSize)
```

Set programmable watchdog timer size.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHashTableFirst32Bits (const ENET_Type *const pEnet)
```

Get the hash table first 32 bits value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetHashTableFirst32Bits (ENET_Type *const pEnet, uint32_t  
u32Hash0)
```

Set the hash table first 32 bits value.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHashTableSecond32Bits (const ENET_Type *const pEnet)
```

Get the hash table second 32 bits value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetHashTableSecond32Bits (ENET_Type *const pEnet, uint32_t  
u32Hash1)
```

Set the hash table second 32 bits value.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetVLANTagCtrl (const ENET_Type *const pEnet)
```

Get VLAN tag control register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagCtrl (ENET_Type *const pEnet, uint32_t u32VlanCtrl)
```

Set VLAN tag control register value.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetRxStatusInnerVLANTagEnFlag (const ENET_Type *const pEnet)
```

Get inner VLAN tag enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetRxStatusInnerVLANTagEnFlag (ENET_Type *const pEnet, bool  
bEnable)
```

Set inner VLAN tag enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagStripModeType** **ENET_HWA_MAC_GetInnerVLANTagStripMode**
(const ENET_Type *const pEnet)
Get inner VLAN tag stripping mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagStripMode** (ENET_Type *const pEnet,
ENET_MAC_VLANTagStripModeType eMode)
Set inner VLAN tag stripping mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetInnerVLANTagCmpEnFlag** (const ENET_Type *const pEnet)
Get inner VLAN tag comparison enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagCmpEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set inner VLAN tag comparison enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetDoubleVLANProcessEnFlag** (const ENET_Type *const pEnet)
Get double VLAN processing enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetDoubleVLANProcessEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set double VLAN processing enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagHashTableMatchEnFlag** (const ENET_Type *const
pEnet)
Get VLAN tag hash table match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagHashTableMatchEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set VLAN tag hash table match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxStatusVLANTagEnFlag** (const ENET_Type *const pEnet)
Get VLAN tag in rx status enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxStatusVLANTagEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag in rx status enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagStripModeType** **ENET_HWA_MAC_GetVLANTagStripMode** (const ENET_Type *const pEnet)

Get VLAN tag stripping mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagStripMode** (ENET_Type *const pEnet, **ENET_MAC_VLANTagStripModeType** eMode)

Set VLAN tag stripping mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTypeCheckDisableFlag** (const ENET_Type *const pEnet)

Get VLAN type check disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTypeCheckDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set VLAN type check disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxSVLANMatchEnFlag** (const ENET_Type *const pEnet)

Get Receive S-VLAN match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxSVLANMatchEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set Receive S-VLAN match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSVLANEnFlag** (const ENET_Type *const pEnet)

Get S-VLAN enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSVLANEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set S-VLAN enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANInverseMatchEnFlag** (const ENET_Type *const pEnet)

Get VLAN tag inverse match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInverseMatchEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag inverse match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_Get12BitVLANTagEnFlag** (const ENET_Type *const pEnet)

Get 12-Bit VLAN tag comparison enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_Set12BitVLANTagEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set 12-Bit VLAN tag comparison enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetVLANTagFilterRegOffset** (const ENET_Type *const pEnet)

Get VLAN tag filter register address offset.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterRegOffset** (ENET_Type *const pEnet, uint8_t u8Offset)

Set VLAN tag filter register address offset.

LOCAL_INLINE **ENET_MAC_VLANCommandType** **ENET_HWA_MAC_GetVLANCommandType** (const ENET_Type *const pEnet)

Get VLAN tag filter register command type.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANCommandType** (ENET_Type *const pEnet, **ENET_MAC_VLANCommandType** eCommand)

Set VLAN tag filter register command type.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetVLANOperationStatus** (const ENET_Type *const pEnet)

Get VLAN tag filter register operation status.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANOperationBusy** (ENET_Type *const pEnet)

Set VLAN tag filter register operation busy status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetVLANTagData** (const ENET_Type *const pEnet)

Get VLAN tag data registers value.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagData** (ENET_Type *const pEnet, uint32_t u32VLANData)

Set VLAN tag data registers value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetVLANHashTable** (const ENET_Type *const pEnet)

Get VLAN hash table registers value.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANHashTable** (ENET_Type *const pEnet, uint16_t u16HashTable)

Set VLAN hash table registers value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetVLANInclusion** (const ENET_Type *const pEnet)

Get VLAN tag inclusion register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInclusion** (ENET_Type *const pEnet, uint32_t u32Inclusion)

Set VLAN tag inclusion register value.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetVLANInclusionOperationStatus** (const ENET_Type *const pEnet)

Get VLAN inclusion operation status.

LOCAL_INLINE **ENET_MAC_VLANInclCommandType** **ENET_HWA_MAC_GetVLANInclusionCommandType** (const ENET_Type *const pEnet)

Set VLAN inclusion operation status.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInclusionCommandType** (ENET_Type *const pEnet, **ENET_MAC_VLANInclCommandType** eCommand)

Set VLAN tag inclusion command.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetVLANInclusionAddr** (const ENET_Type *const pEnet)

Get VLAN inclusion address.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInclusionAddr** (ENET_Type *const pEnet, uint8_t u8Addr)

Set VLAN inclusion address.

LOCAL_INLINE bool **ENET_HWA_MAC_GetChannelBasedVLANTagInsertEnFlag** (const ENET_Type *const pEnet)

Get channel based tag insertion enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetChannelBasedVLANTagInsertEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set channel based tag insertion enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagInputEnFlag** (const ENET_Type *const pEnet)

Get VLAN tag input enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagInputEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag input enable flag.

LOCAL_INLINE **ENET_MAC_VLANType** **ENET_HWA_MAC_GetVLANType** (const ENET_Type *const pEnet)

Get VLAN type.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANType** (ENET_Type *const pEnet, **ENET_MAC_VLANType** eType)

Set VLAN type.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANPriorityCtrlEnFlag** (const ENET_Type *const pEnet)

Get VLAN priority control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANPriorityCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag input enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagCtrlType** **ENET_HWA_MAC_GetVLANTagCtrlOperation** (const ENET_Type *const pEnet)

Get VLAN tag control mode in transmit packets.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagCtrlOperation** (ENET_Type *const pEnet, **ENET_MAC_VLANTagCtrlType** eCtrl)

Set VLAN tag control mode in transmit packets.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetTxVLANTag** (const ENET_Type *const pEnet)

Get Tx VLAN tag value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxVLANTag** (ENET_Type *const pEnet, uint16_t u16VLANTag)

Set Tx VLAN tag value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetInnerVLANInclusion** (const ENET_Type *const pEnet)

Get inner VLAN tag inclusion or replacement register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANInclusion** (ENET_Type *const pEnet, uint32_t u32Inclusion)

Set inner VLAN tag inclusion or replacement register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetInnerVLANTagInputEnFlag** (const ENET_Type *const pEnet)

Get inner VLAN tag input enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagInputEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set inner VLAN tag input enable flag.

LOCAL_INLINE **ENET_MAC_VLANType** **ENET_HWA_MAC_GetInnerVLANType** (const ENET_Type *const pEnet)

Get inner VLAN type.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANType** (ENET_Type *const pEnet, **ENET_MAC_VLANType** eType)

Set inner VLAN type.

LOCAL_INLINE bool **ENET_HWA_MAC_GetInnerVLANPriorityCtrlEnFlag** (const ENET_Type *const pEnet)

Get inner VLAN priority control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANPriorityCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set inner VLAN priority control enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagCtrlType** **ENET_HWA_MAC_GetInnerVLANTagCtrl** (const ENET_Type *const pEnet)

Get inner VLAN tag control mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagCtrl** (ENET_Type *const pEnet, **ENET_MAC_VLANTagCtrlType** eCtrl)

Set inner VLAN tag control mode.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetTxInnerVLANTag** (const ENET_Type *const pEnet)

Get inner VLAN tag value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxInnerVLANTag** (ENET_Type *const pEnet, uint16_t u16VLANTag)

Set inner VLAN tag value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetQnTxFlowCtrl** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx flow control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxFlowCtrl** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint32_t u32FlowCtrl)

Set Queue n tx flow control register value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetQnTxPauseTime** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx pause time value.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxPauseTime** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint16_t u16PauseTime)

Set Queue n tx pause time value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetQnTxZeroQuantaPauseDisableFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx zero quanta pause disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxZeroQuantaPauseDisableFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bDisable)

Set Queue n tx zero quanta pause disable flag.

LOCAL_INLINE **ENET_MAC_PauseThresholdType** **ENET_HWA_MAC_GetQnTxPauseLowThreshold** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx pause low threshold.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxPauseLowThreshold** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MAC_PauseThresholdType** ePauseThreshold)

Set Queue n tx pause low threshold.

LOCAL_INLINE bool **ENET_HWA_MAC_GetQnTxFlowCtrlEnableFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx flow control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxFlowCtrlEnableFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set Queue n tx flow control enable flag.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetFlowCtrlOrBackpressureStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get flow control busy or backpressure status of Queue n.

LOCAL_INLINE void **ENET_HWA_MAC_InitiatePausePacketOrBackpressure** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Initiate flow control busy or backpressure of Queue n.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxFlowCtrl** (const ENET_Type *const pEnet)

Get receive flow control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxFlowCtrl** (ENET_Type *const pEnet, uint32_t u32FlowCtrl)

Set receive flow control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUnicastPausePacketDetectEnFlag** (const ENET_Type *const pEnet)

Get unicast pause packet detect enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastPausePacketDetectEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set unicast pause packet detect enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxFlowCtrlEnFlag** (const ENET_Type *const pEnet)

Get receive flow control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxFlowCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set receive flow control enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl0** (const ENET_Type *const pEnet)

Get Receive queue control 0 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl0** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Receive queue control 0 register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl1** (const ENET_Type *const pEnet)

Get Receive queue control 1 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl1** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Receive queue control 1 register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl2** (const ENET_Type *const pEnet)

Get Receive queue control 2 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl2** (ENET_Type *const pEnet, uint32_t u32Ctrl)
Set Receive queue control 2 register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl4** (const ENET_Type *const pEnet)
Get Receive queue control 4 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl4** (ENET_Type *const pEnet, uint32_t u32Ctrl)
Set Receive queue control 4 register value.

LOCAL_INLINE **ENET_MTL_QueueEnableType** **ENET_HWA_MTL_GetRxQueueEnStatus** (const ENET_Type
*const pEnet, **ENET_MTL_QueueType** eQueue)
Get Receive queue enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueueEnStatus** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, **ENET_MTL_QueueEnableType** eEnable)
Set Receive queue enable flag.

LOCAL_INLINE **ENET_MAC_PTPoEQueueCtrlType** **ENET_HWA_MAC_GetTaggedPTPoEQueueCtrl** (const
ENET_Type *const pEnet)
Get tagged PTP over ethernet packets queuing control.

LOCAL_INLINE void **ENET_HWA_MAC_SetTaggedPTPoEQueueCtrl** (ENET_Type *const pEnet,
ENET_MAC_PTPoEQueueCtrlType eCtrl)
Set tagged PTP over ethernet packets queuing control.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTaggedAVCtrlPacketQueueEnFlag** (const ENET_Type *const
pEnet)
Get tagged AV control packets queuing enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTaggedAVCtrlPacketQueueEnFlag** (ENET_Type *const pEnet,
bool bEnable)
Set tagged AV control packets queuing enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMulticastAndBroadcastQueueEnFlag** (const ENET_Type *const pEnet)

Get multicast and broadcast queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAndBroadcastQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set multicast and broadcast queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetMulticastAndBroadcastQueue** (const ENET_Type *const pEnet)

Get multicast and broadcast queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAndBroadcastQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set multicast and broadcast queue id.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetUntaggedPacketQueue** (const ENET_Type *const pEnet)

Get untagged packet queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetUntaggedPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set untagged packet queue id.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetPTPoEQueue** (const ENET_Type *const pEnet)

Get PTP over ethernet packet queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetPTPoEQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set PTP over ethernet packet queue id.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetAVCtrlPacketQueue** (const ENET_Type *const pEnet)

Get AV untagged control packets queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetAVCtrlPacketQueue** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Set AV untagged control packets queue id.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetRxQueueSelectedPriorities** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue priorities.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueSelectedPriorities** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint8_t u8SelPrio)

Set receive queue priorities.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagFilterFailPacketQueueEnFlag** (const ENET_Type
*const pEnet)

Get VLAN tag filter fail packets queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterFailPacketQueueEnFlag** (ENET_Type *const
pEnet, bool bEnable)

Set VLAN tag filter fail packets queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetVLANTagFilterFailPacketQueue** (const
ENET_Type *const pEnet)

Get VLAN tag filter fail packets queue.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterFailPacketQueue** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Set VLAN tag filter fail packets queue.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMulticastAddrFilterFailPacketQueueEnFlag** (const ENET_Type
*const pEnet)

Get multicast address filter fail packets queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAddrFilterFailPacketQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set multicast address filter fail packets queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetMulticastAddrFilterFailPacketQueue** (const ENET_Type *const pEnet)

Get multicast address filter fail packets queue.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAddrFilterFailPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set multicast address filter fail packets queue.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUnicastAddrFilterFailPacketQueueEnFlag** (const ENET_Type *const pEnet)

Get unicast address filter fail packets queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastAddrFilterFailPacketQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set unicast address filter fail packets queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetUnicastAddrFilterFailPacketQueue** (const ENET_Type *const pEnet)

Get unicast address filter fail packets queue.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastAddrFilterFailPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set unicast address filter fail packets queue.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetInterruptStatus** (const ENET_Type *const pEnet)

Get the interrupt status of ENET MAC.

LOCAL_INLINE void **ENET_HWA_MAC_ClearInterruptStatus** (ENET_Type *const pEnet, uint32_t u32Interrupt)

Clear the interrupt flags of MAC.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMDIOInterruptStatus** (const ENET_Type *const pEnet)

Get the MDIO interrupt status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearMDIOInterruptStatus** (ENET_Type *const pEnet)

Clear the MDIO interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxInterruptStatus** (const ENET_Type *const pEnet)

Get the receive interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxInterruptStatus** (const ENET_Type *const pEnet)

Get the transmit interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampInterruptStatus** (const ENET_Type *const pEnet)

Get the timestamp interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMMCTxInterruptStatus** (const ENET_Type *const pEnet)

Get the MMC transmit interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMMCRxInterruptStatus** (const ENET_Type *const pEnet)

Get the MMC receive interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMMCInterruptStatus** (const ENET_Type *const pEnet)

Get the MMC interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPhyInterruptStatus** (const ENET_Type *const pEnet)

Get the PHY interrupt status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearPhyInterruptStatus** (ENET_Type *const pEnet)

Clear the PHY interrupt status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetInterruptEnable** (const ENET_Type *const pEnet)

Get the MAC interrupts enable register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetInterruptEnable** (ENET_Type *const pEnet, uint32_t u32IntEnable)

Set the MAC interrupts enable register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMDIOInterruptEnFlag** (const ENET_Type *const pEnet)

Get MDIO interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMDIOInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MDIO interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxStatusInterruptEnFlag** (const ENET_Type *const pEnet)

Get receive status interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxStatusInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set receive status interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxStatusInterruptEnFlag** (const ENET_Type *const pEnet)

Get transmit status interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxStatusInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set transmit status interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampInterruptEnFlag** (const ENET_Type *const pEnet)

Get timestamp interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set timestamp interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPhyInterruptEnFlag** (const ENET_Type *const pEnet)

Get phy interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set phy interrupt enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxTxStatus** (const ENET_Type *const pEnet)
Get the MAC receive and transmit status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearRxTxStatus** (ENET_Type *const pEnet, uint32_t u32Status)
Clear the MAC receive and transmit status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxWDGTimeoutStatus** (const ENET_Type *const pEnet)
Get the receive watchdog timeout status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearRxWDGTimeoutStatus** (ENET_Type *const pEnet)
Clear the receive watchdog timeout status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxExcessiveCollisionStatus** (const ENET_Type *const pEnet)
Get the excessive collision status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxExcessiveCollisionStatus** (ENET_Type *const pEnet)
Clear the excessive collision status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxLateCollisionStatus** (const ENET_Type *const pEnet)
Get the late collision status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxLateCollisionStatus** (ENET_Type *const pEnet)
Clear the late collision status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxExcessiveDeferralStatus** (const ENET_Type *const pEnet)
Get the excessive deferral status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearExcessiveDeferralStatus** (ENET_Type *const pEnet)
Clear the excessive deferral status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxLossOfCarrierStatus** (const ENET_Type *const pEnet)

Indicates the status of carrier signal.

LOCAL_INLINE void **ENET_HWA_MAC_ClearLossOfCarrierStatus** (ENET_Type *const pEnet)

Clear the loss of carrier status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxNoCarrierStatus** (const ENET_Type *const pEnet)

Indicates whether the carrier signal is present from the PHY at the end of preamble transmission.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxNoCarrierStatus** (ENET_Type *const pEnet)

Clear the no carrier status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxJabberTimeoutStatus** (const ENET_Type *const pEnet)

Indicates whether a transmit jabber timer timeout occurred.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxJabberTimeoutStatus** (ENET_Type *const pEnet)

Clear the transmit jabber timer timeout status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMACVersion** (const ENET_Type *const pEnet)

Get MAC version.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetDebugStatus** (const ENET_Type *const pEnet)

Get MAC debug status.

LOCAL_INLINE **ENET_MAC_TxPacketControllerStatusType**

ENET_HWA_MAC_GetTxPacketControllerStatus (const ENET_Type *const pEnet)

Get MAC transmit packet controller status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxPacketEngineStatus** (const ENET_Type *const pEnet)

Get MAC GMII or MII transmit protocol engine status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxPacketControllerFIFOStatus** (const ENET_Type *const pEnet)

Get MAC receive packet controller FIFO status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxProtocolEngineStatus** (const ENET_Type *const pEnet)

Get MAC GMII or MII receive protocol engine status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature0** (const ENET_Type *const pEnet)

Get MAC hardware features 0.

LOCAL_INLINE **ENET_MAC_MiiModeType** **ENET_HWA_MAC_GetPhyMiiMode** (const ENET_Type *const pEnet)

Get phy interface mii mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature1** (const ENET_Type *const pEnet)

Get MAC hardware features 1.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature2** (const ENET_Type *const pEnet)

Get MAC hardware features 2.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature3** (const ENET_Type *const pEnet)

Get MAC hardware features 3.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMDIOAddress** (const ENET_Type *const pEnet)

Get MAC MDIO address register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMDIOAddress** (ENET_Type *const pEnet, uint32_t u32MDIOAddr)

Set MAC MDIO address register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPreambleSuppressionEnFlag** (const ENET_Type *const pEnet)

Get preamble suppression enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPreambleSuppressionEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MDIO preamble suppression enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetBackToBackTransactionEnFlag** (const ENET_Type *const pEnet)

Get MDIO back to back transactions enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetBackToBackTransactionEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MDIO back to back transactions enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetPhysicalLayerAddr** (const ENET_Type *const pEnet)

Get MDIO physical layer address.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhysicalLayerAddr** (ENET_Type *const pEnet, uint8_t u8Addr)

Set MDIO physical layer address.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetPhyRegisterOrDeviceAddr** (const ENET_Type *const pEnet)

Get MDIO phy register(clause22) or MMD device(clause 45)

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyRegisterOrDeviceAddr** (ENET_Type *const pEnet, uint8_t u8Addr)

Set MDIO phy register(clause22) or MMD device(clause 45)

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetTrailingClockCnt** (const ENET_Type *const pEnet)

Get the number of trailing clock cycles.

LOCAL_INLINE void **ENET_HWA_MAC_SetTrailingClockCnt** (ENET_Type *const pEnet, uint8_t u8Cnt)

Set MDIO trailing clock cycles.

LOCAL_INLINE **ENET_MAC_CSRClockRangeType** **ENET_HWA_MAC_GetCSRClockRange** (const ENET_Type *const pEnet)

Get the MDC clock range.

LOCAL_INLINE void **ENET_HWA_MAC_SetCSRClockRange** (ENET_Type *const pEnet,

ENET_MAC_CSRClockRangeType eRange)

Set the MDC clock range.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSkipAddrPacketEnFlag** (const ENET_Type *const pEnet)

Get skip address packet enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSkipAddrPacketEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set skip address packet enable flag.

LOCAL_INLINE **ENET_MAC_PhyOperationCmdType** **ENET_HWA_MAC_GetPhyOperationCommand** (const ENET_Type *const pEnet)

Get phy operation command.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyOperationCommand** (ENET_Type *const pEnet, **ENET_MAC_PhyOperationCmdType** eCmd)

Set phy operation command.

LOCAL_INLINE **ENET_MAC_PhyClauseVersionType** **ENET_HWA_MAC_GetPhyClauseVersion** (const ENET_Type *const pEnet)

Get MDIO clause version.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyClauseVersion** (ENET_Type *const pEnet, **ENET_MAC_PhyClauseVersionType** eVersion)

Set MDIO clause version.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetGMIIStatus** (const ENET_Type *const pEnet)

Get MDIO busy status.

LOCAL_INLINE void **ENET_HWA_MAC_SetGMIIBusy** (ENET_Type *const pEnet)

Set MDIO busy to initiate a read or write access.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMDIOData** (const ENET_Type *const pEnet)

Get MDIO data register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMDIOData** (ENET_Type *const pEnet, uint32_t u32Data)

Set MDIO data register value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetPhyRegisterAddr** (const ENET_Type *const pEnet)

Get phy register address.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyRegisterAddr** (ENET_Type *const pEnet, uint16_t u16Addr)

Set phy register address.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetGMIIData** (const ENET_Type *const pEnet)

Get MDIO data.

LOCAL_INLINE void **ENET_HWA_MAC_SetGMIIData** (ENET_Type *const pEnet, uint16_t u16Data)

Set MDIO data.

LOCAL_INLINE void **ENET_HWA_MAC_SetFramePreemptionEnFlag** (ENET_Type *const pEnet, uint8
bEnable)

Enable TX frame preemption.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPresentationTime** (const ENET_Type *const pEnet)

Get presentation time in ns.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPresentationTimeUpdata** (const ENET_Type *const pEnet)

Get presentation time update.

LOCAL_INLINE void **ENET_HWA_MAC_SetPresentationTimeUpdata** (ENET_Type *const pEnet, uint32_t
u32NanoSeconds)

Set presentation time update.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr0High** (const ENET_Type *const pEnet)

Get MAC address0 high register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetAddr0High** (ENET_Type *const pEnet, uint32_t u32AddrHigh)

Set MAC address0 high register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr0Low** (const ENET_Type *const pEnet)

Get MAC address0 low register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr0Low (ENET_Type *const pEnet, uint32_t u32AddrLow)
```

Set MAC address0 low register value.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetMacAddr0EnFlag (const ENET_Type *const pEnet)
```

Get MAC address0 enable flag.

```
LOCAL_INLINE uint8_t ENET_HWA_MAC_GetMacAddr0DMAChannelSelect (const ENET_Type *const pEnet)
```

Get MAC address0 DMA channel select.

```
LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr0DMAChannelSelect (ENET_Type *const pEnet, uint8_t u8DMAChannelSel)
```

Set MAC address0 DMA channel select.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr1High (const ENET_Type *const pEnet)
```

Get MAC address1 high register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetAddr1High (ENET_Type *const pEnet, uint32_t u32AddrHigh)
```

Set MAC address1 high register value.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr1Low (const ENET_Type *const pEnet)
```

Get MAC address1 low register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr1Low (ENET_Type *const pEnet, uint32_t u32AddrLow)
```

Set MAC address1 low register value.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetMacAddr1EnFlag (const ENET_Type *const pEnet)
```

Get MAC address1 enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr1EnFlag (ENET_Type *const pEnet, bool bEnable)
```

Set MAC address1 enable flag.

LOCAL_INLINE **ENET_MAC_AddrType** **ENET_HWA_MAC_GetMacAddr1Type** (const ENET_Type *const pEnet)

Get MAC address1 compare mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1Type** (ENET_Type *const pEnet, **ENET_MAC_AddrType** eAddrType)

Set MAC address1 compare mode.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr1MaskByteCtrl** (const ENET_Type *const pEnet)

Get MAC address1 mask byte control.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1MaskByteCtrl** (ENET_Type *const pEnet, uint8_t u8MaskCtrl)

Set MAC address1 mask byte control.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr1DMAChannelSelect** (const ENET_Type *const pEnet)

Get MAC address1 DMA channel select.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1DMAChannelSelect** (ENET_Type *const pEnet, uint8_t u8DMAChannelSel)

Set MAC address1 DMA channel select.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr2High** (const ENET_Type *const pEnet)

Get MAC address2 high register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetAddr2High** (ENET_Type *const pEnet, uint32_t u32AddrHigh)

Set MAC address2 high register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr2Low** (const ENET_Type *const pEnet)

Get MAC address2 low register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2Low** (ENET_Type *const pEnet, uint32_t u32AddrLow)

Set MAC address2 low register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMacAddr2EnFlag** (const ENET_Type *const pEnet)

Get MAC address2 enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2EnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MAC address2 enable flag.

LOCAL_INLINE **ENET_MAC_AddrType** **ENET_HWA_MAC_GetMacAddr2Type** (const ENET_Type *const pEnet)

Get MAC address2 compare mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2Type** (ENET_Type *const pEnet, **ENET_MAC_AddrType** eAddrType)

Set MAC address2 compare mode.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr2MaskByteCtrl** (const ENET_Type *const pEnet)

Get MAC address2 mask byte control.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2MaskByteCtrl** (ENET_Type *const pEnet, uint8_t u8MaskCtrl)

Set MAC address2 mask byte control.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr2DMAChannelSelect** (const ENET_Type *const pEnet)

Get MAC address2 DMA channel select.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2DMAChannelSelect** (ENET_Type *const pEnet, uint8_t u8DMAChannelSel)

Set MAC address2 DMA channel select.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcControl** (const ENET_Type *const pEnet)

Get MMC control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcControl** (ENET_Type *const pEnet, uint32_t u32Ctrl)
Set MMC control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUpdateMmcCntForDroppedBroadcastPktEnFlag** (const ENET_Type *const pEnet)
Get update MMC counters for dropped broadcast packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUpdateMmcCntForDroppedBroadcastPktEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set update MMC counters for dropped broadcast packets enable flag.

LOCAL_INLINE **ENET_MAC_MmcCntPresetLevelType** **ENET_HWA_MAC_GetMmcCounterPresetLevel** (const ENET_Type *const pEnet)
Get MMC counters full-half preset level.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterPresetLevel** (ENET_Type *const pEnet, **ENET_MAC_MmcCntPresetLevelType** eLevel)
Set MMC counters full-half preset level.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterPresetStatus** (const ENET_Type *const pEnet)
Get MMC counters preset status.

LOCAL_INLINE void **ENET_HWA_MAC_PresetMmcCounters** (ENET_Type *const pEnet)
Set MMC counters preset.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterFreezeStatus** (const ENET_Type *const pEnet)
Get MMC counters freeze status.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterFreezeStatus** (ENET_Type *const pEnet, bool bFreeze)
Set MMC counters freeze.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterResetOnReadEnFlag** (const ENET_Type *const pEnet)

Get MMC counters reset on read enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterResetOnReadEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MMC counters reset on read enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterStopRolloverEnFlag** (const ENET_Type *const pEnet)

Get MMC counters stop rollover enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterStopRolloverEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MMC counters stop rollover enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterResetStatus** (const ENET_Type *const pEnet)

Get MMC counters reset status.

LOCAL_INLINE void **ENET_HWA_MAC_ResetMmcCounters** (ENET_Type *const pEnet)

Reset all MMC counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcRxInterruptStatus** (const ENET_Type *const pEnet)

Get MMC interrupts of receive statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcTxInterruptStatus** (const ENET_Type *const pEnet)

Get MMC interrupts of transmit statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcRxInterruptMask** (const ENET_Type *const pEnet)

Get MMC interrupts mask of receive statistics counters.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcRxInterruptMask** (ENET_Type *const pEnet, uint32_t u32Mask)

Set MMC interrupts mask of receive statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcTxInterruptMask** (const ENET_Type *const pEnet)

Get MMC interrupts mask of transmit statistics counters.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcTxInterruptMask** (ENET_Type *const pEnet, uint32_t u32Mask)

Set MMC interrupts mask of transmit statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxOctetCntGoodBad** (const ENET_Type *const pEnet)

Get transmitted bytes of good and bad packets.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxBroadcastPacketGood** (const ENET_Type *const pEnet)

Get the number of good broadcast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxMulticastPacketGood** (const ENET_Type *const pEnet)

Get the number of good multicast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx64OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length 64 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx65To127OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 65 and 127 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx128To255OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 128 and 255 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx256To511OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 256 and 511 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx512To1023OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 512 and 1023 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx1024ToMaxOctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length 1024 to maxsize bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxUnicastPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad unicast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxMulticastPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad multicast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxBroadcastPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad broadcast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxUnderflowErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit underflow error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxSingleCollisionGoodPacketCnt** (const ENET_Type *const pEnet)

Get the number of successfully transmitted packets after a single collision in half-duplex mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxMultipleCollisionGoodPacketCnt** (const ENET_Type *const pEnet)

Get the number of successfully transmitted packets after a multiple collision in half-duplex mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxDeferredPacketCnt** (const ENET_Type *const pEnet)

Get the number of successfully transmitted packets after a deferral in half-duplex mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxLateCollisionPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit late collision error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxExcessiveCollisionPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit excessive collision errors.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxCarrierErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit carrier sense error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxOctetCntGood** (const ENET_Type *const pEnet)

Get the number of bytes of good packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxPacketCntGood** (const ENET_Type *const pEnet)

Get the number of good packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxExcessiveDeferralErrorCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit excessive deferral error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxPausePacketCnt** (const ENET_Type *const pEnet)

Get the number of good pause packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxVLANPacketGood** (const ENET_Type *const pEnet)

Get the number of good VLAN packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxOverSizePacketGood** (const ENET_Type *const pEnet)

Get the number of transmitted packets without errors and with length greater than the maxsize.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxPacketCntGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOctetCntGoodBad** (const ENET_Type *const pEnet)

Get the number of bytes received in good and bad packets.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOctetCntGood** (const ENET_Type *const pEnet)

Get the number of bytes received in good packets.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxBroadcastPacketGood** (const ENET_Type *const pEnet)

Get the number of good broadcast packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxMulticastPacketGood** (const ENET_Type *const pEnet)

Get the number of good multicast packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxCRCErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with CRC error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxAlignmentErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with alignment error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxRunErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with runt error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxJabberErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with jabber error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxUndersizePacketGood** (const ENET_Type *const pEnet)

Get the number of packets received with length less than 64 bytes, without any errors.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOverSizeGood** (const ENET_Type *const pEnet)

Get the number of packets received without errors and with length greater than maxsize.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx64OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 64 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx65To127OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 65 to 127 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx128To255OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 128 to 255 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx256To511OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 256 to 511 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx512To1023OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 512 to 1023 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx1024ToMaxOctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 1024 to maxsize.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetUnicastPacketGood** (const ENET_Type *const pEnet)

Get the number of good unicast packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxLengthErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with length error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOutOfRangeTypePacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with length greater than 1500 but less than 1536.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxPausePacketCnt** (const ENET_Type *const pEnet)

Get the number of good and valid pause packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxFIFOOverflowPacketCnt** (const ENET_Type *const pEnet)

Get the number of missed received packets because of FIFO overflow.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxVLANPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad VLAN packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxWatchdogErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with error because of watchdog timeout error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxReceiveErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with receive error or Packet Extension error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxControlPacketGood** (const ENET_Type *const pEnet)

Get the number of good control packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3Layer4Ctrl** (const ENET_Type *const pEnet, uint8_t u8Index)

Get the layer 3 and layer 4 control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Layer4Ctrl** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Ctrl)

Set the layer 3 and layer 4 control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3Layer4DMASelectEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 and layer 4 control DMA channel select enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Layer4DMASelectEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 3 and layer 4 control DMA channel select enable flag.

LOCAL_INLINE **ENET_DMA_ChannelType** **ENET_HWA_MAC_GetLayer3Layer4DMAChannel** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 and layer 4 control DMA channel number.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Layer4DMAChannel** (ENET_Type *const pEnet, uint8_t u8Index, **ENET_DMA_ChannelType** eChannel)

Set layer 3 and layer 4 control DMA channel number.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer4DestPortInvMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 destination port inverse match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4DestPortInvMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 4 destination port inverse match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer4DestPortMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 destination port match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4DestPortMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 4 destination port match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer4SrcPortInvMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 source port inverse match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4SrcPortInvMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 4 source port inverse match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer4SrcPortMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 source port match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4SrcPortMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 4 source port match enable flag.

LOCAL_INLINE **ENET_MAC_Layer4ProtocolType** **ENET_HWA_MAC_GetLayer4ProtocolType** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 protocol type (UDP or TCP)

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4ProtocolType** (ENET_Type *const pEnet, uint8_t u8Index, **ENET_MAC_Layer4ProtocolType** eProtocol)

Set layer 4 protocol type (UDP or TCP)

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetLayer3IPv4DestAddrBitsMatch** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer3 IPv4 destination address bits match.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3IPv4DestAddrBitsMatch** (ENET_Type *const pEnet, uint8_t u8Index, uint8_t u8MaskBits)

Set layer3 IPv4 destination address bits match.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetLayer3IPv4SrcAddrBitsMatch** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer3 IPv4 source address bits match.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3IPv4SrcAddrBitsMatch** (ENET_Type *const pEnet, uint8_t u8Index, uint8_t u8MaskBits)

Set layer3 IPv4 source address bits match.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetLayer3IPv6AddrBitsMatch** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer3 IPv6 address bits match.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3IPv6AddrBitsMatch** (ENET_Type *const pEnet, uint8_t u8Index, uint8_t u8MaskBits)

Set layer3 IPv6 address bits match.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3DestAddrInvMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 destination address inverse match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3DestAddrInvMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 3 destination address inverse match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3DestAddrMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 destination address match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3DestAddrMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 3 destination address match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3SrcAddrInvMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 source address inverse match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3SrcAddrInvMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 3 source address inverse match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3SrcAddrMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 source address match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3SrcAddrMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 3 source address match enable flag.

LOCAL_INLINE **ENET_MAC_Layer3ProtocolType** **ENET_HWA_MAC_GetLayer3ProtocolType** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 protocol type (IPV4 or IPV6)

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3ProtocolType** (ENET_Type *const pEnet, uint8_t u8Index, **ENET_MAC_Layer3ProtocolType** eProtocol)

Set layer 3 protocol type (IPv4 or IPv6)

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer4Addr** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 address.

LOCAL_INLINE void **ENET_MAC_SetLayer4Addr** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)

Set layer 4 address.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetLayer4DestPort** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 destination port number.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4DestPort** (ENET_Type *const pEnet, uint8_t u8Index, uint16_t u16DestPort)

Set layer 4 destination port number.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetLayer4SrcPort** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 source port number.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4SrcPort** (ENET_Type *const pEnet, uint8_t u8Index, uint16_t u16SrcPort)

Set layer 4 source port number.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3Addr0** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 address0.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Addr0** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)

Set layer 3 address0.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Addr1 (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 address1.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Addr1 (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)
```

Set layer 3 address1.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Addr2 (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 address2.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Addr2 (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)
```

Set layer 3 address2.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Addr3 (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 address3.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Addr3 (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)
```

Set layer 3 address3.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3SrcIPv4Addr (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 source IPv4 address.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3SrcIPv4Addr (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32SrcAddr)
```

Set layer 3 source IPv4 address.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3DestIPv4Addr** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 destination IPv4 address.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3DestIPv4Addr** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32DestAddr)

Set layer 3 destination IPv4 address.

LOCAL_INLINE void **ENET_HWA_MAC_GetLayer3IPv6Addr** (const ENET_Type *const pEnet, uint8_t u8Index, uint8_t aIPv6Addr[16u])

Get layer 3 IPv6 address.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3IPv6Addr** (ENET_Type *const pEnet, uint8_t u8Index, const uint8_t aIPv6Addr[16])

Set layer 3 IPv6 address.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampControl** (const ENET_Type *const pEnet)

Get time stamp control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampControl** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set time stamp control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetAV802_1ASModeEnFlag** (const ENET_Type *const pEnet)

Get time stamp AV802_1AS mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetAV802_1ASModeEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set time stamp AV802_1AS mode enable flag.

LOCAL_INLINE **ENET_MAC_TxTimestampStatusModeType**

ENET_HWA_MAC_GetTxTimestampStatusMode (const ENET_Type *const pEnet)

Get transmit timestamp status mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxTimestampStatusMode** (ENET_Type *const pEnet, **ENET_MAC_TxTimestampStatusModeType** eMode)

Set transmit timestamp status mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUseMACAddrForPTPPacketFilterEnFlag** (const ENET_Type *const pEnet)

Get MAC address for PTP packet filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUseMACAddrForPTPPacketFilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MAC address for PTP packet filter enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetSnapshotPTPPacketType** (const ENET_Type *const pEnet)

Get PTP packets type for taking snapshots.

LOCAL_INLINE void **ENET_HWA_MAC_SetSnapshotPTPPacketType** (ENET_Type *const pEnet, uint8_t u8Type)

Set PTP packets type for taking snapshots.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMasterMsgSanpshotEnFlag** (const ENET_Type *const pEnet)

Get snapshot for messages relevant to master enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMasterMsgSanpshotEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set snapshot for messages relevant to master enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetEventMsgSnapshotEnFlag** (const ENET_Type *const pEnet)

Get snapshot for event messages enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetEventMsgSanpshotEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set snapshot for event messages enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetIPv4UDPPTPPacketProcessEnFlag** (const ENET_Type *const pEnet)

Get processing of PTP packets over IPv4-UDP enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetIPv4UDPPTPPacketProcessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set processing of PTP packets over IPv4-UDP enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetIPv6UDPPTPPacketProcessEnFlag** (const ENET_Type *const pEnet)

Get processing of PTP packets over IPv6-UDP enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetIPv6UDPPTPPacketProcessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set processing of PTP packets over IPv6-UDP enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetEthernetPTPPacketProcessEnFlag** (const ENET_Type *const pEnet)

Get processing of PTP packets over ethernet enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetEthernetPTPPacketProcessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set processing of PTP packets over ethernet enable flag.

LOCAL_INLINE **ENET_MAC_PTPVersionType** **ENET_HWA_MAC_GetPTPPacketVersion** (const ENET_Type *const pEnet)

Get PTP packets processing format version.

LOCAL_INLINE void **ENET_HWA_MAC_SetPTPPacketVersion** (ENET_Type *const pEnet, **ENET_MAC_PTPVersionType** eVersion)

Set PTP packets processing format version.

LOCAL_INLINE **ENET_MAC_TimestampRolloverType** **ENET_HWA_MAC_GetTimestampRolloverCtrl** (const ENET_Type *const pEnet)

Get PTP timestamp rollover mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetlimestamprolloverCtrl** (ENET_Type *const pEnet,
ENET_MAC_TimestampRolloverType eCtrl)
Set PTP timestamp rollover mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampForAllPacketEnFlag** (const ENET_Type *const pEnet)
Get timestamp for all packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampForAllPacketEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set timestamp for all packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPresentationTimeGenerationEnFlag** (const ENET_Type *const
pEnet)
Get presentation time generation enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPresentationTimeGenerationEnFlag** (ENET_Type *const pEnet,
bool bEnable)
Set presentation time generation enable flag.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetAddendRegisterUpdateStatus** (const
ENET_Type *const pEnet)
Get PTP addend register update status.

LOCAL_INLINE void **ENET_HWA_MAC_UpdateAddendRegister** (ENET_Type *const pEnet)
Initiate the addend register update operation.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetTimestampUpdateStatus** (const ENET_Type
*const pEnet)
Get timestamp update status.

LOCAL_INLINE void **ENET_HWA_MAC_UpdateTimestamp** (ENET_Type *const pEnet)
Initiate the timestamp update operation.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetTimestampInitStatus** (const ENET_Type *const pEnet)

Get timestamp initialize status.

LOCAL_INLINE void **ENET_HWA_MAC_InitTimestamp** (ENET_Type *const pEnet)

Initialize PTP timestamp.

LOCAL_INLINE **ENET_MAC_TimestampUpdateMethodType**

ENET_HWA_MAC_GetTimestampUpdateMethod (const ENET_Type *const pEnet)

Get timestamp update method (fine or coarse)

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampUpdateMethod** (ENET_Type *const pEnet, **ENET_MAC_TimestampUpdateMethodType** eMethod)

Set timestamp update method (fine or coarse)

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampEnFlag** (const ENET_Type *const pEnet)

Get timestamp enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set timestamp processing enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSubSecondIncrementReg** (const ENET_Type *const pEnet)

Get PTP sub-second increment register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSubSecondIncrementReg** (ENET_Type *const pEnet, uint32_t u32Increment)

Set PTP sub-second increment register value.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetSubSecondIncrementValue** (const ENET_Type *const pEnet)

Get PTP sub-second increment value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSubSecondIncrementValue** (ENET_Type *const pEnet, uint8_t u8Increment)

Set PTP sub-second increment value.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetSubNanoSecondIncrementValue** (const ENET_Type *const pEnet)

Get PTP sub-nanosecond increment value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSubNanoSecondIncrementValue** (ENET_Type *const pEnet, uint8_t u8Increment)

Set PTP sub-nanosecond increment value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeSeconds** (const ENET_Type *const pEnet)

Get PTP system time seconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeNanoSeconds** (const ENET_Type *const pEnet)

Get PTP system time nanoseconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeSecondsUpdateValue** (const ENET_Type *const pEnet)

Get PTP system time seconds update register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeSecondsUpdateValue** (ENET_Type *const pEnet, uint32_t u32Seconds)

Set PTP system time seconds update register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeNanoSecondsUpdateReg** (const ENET_Type *const pEnet)

Get PTP system time nanoseconds update register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeNanoSecondsUpdateReg** (ENET_Type *const pEnet, uint32_t u32Reg)

Set PTP system time nanoseconds update register value.

LOCAL_INLINE **ENET_MAC_SystemTimeOperationType** **ENET_HWA_MAC_GetTimeUpdateOperation** (const ENET_Type *const pEnet)

Get PTP system time update operation, add or subtract time.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimeUpdateOperation** (ENET_Type *const pEnet,
ENET_MAC_SystemTimeOperationType eOperation)

Set PTP system time update operation, add or subtract time.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeNanoSecondsUpdateValue** (const ENET_Type
*const pEnet)

Get PTP system time nanoseconds update value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeNanoSecondsUpdateValue** (ENET_Type *const
pEnet, uint32_t u32NanoSeconds)

Set PTP system time nanoseconds update value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampAddend** (const ENET_Type *const pEnet)

Get PTP timestamp addend register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampAddend** (ENET_Type *const pEnet, uint32_t u32Time)

Set PTP timestamp addend register value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetSystemTimeHigherWordSeconds** (const ENET_Type *const
pEnet)

Get PTP system time seconds bits[47:32] value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeHigherWordSeconds** (ENET_Type *const pEnet,
uint16_t u16Seconds)

Set PTP system time seconds bits[47:32] value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampStatus** (const ENET_Type *const pEnet)

Get the timestamp status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxTimestampInterruptStatus** (const ENET_Type *const pEnet)

Indicates whether the transmit timestamp interrupt status is detected.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxTimestampInterruptStatus** (ENET_Type *const pEnet)

Clear the transmit timestamp interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampTargetTimeError** (const ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Indicates whether the timestamp target time error status of the PPS channel is detected.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTimestampTargetTimeError** (ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Clear the timestamp target time error status of the PPS channel.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampTargetTimeReachedStatus** (const ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Indicates whether the timestamp target time reached for target time of the PPS channel.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTimestampTargetTimeReachedStatus** (ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Clear the status of the timestamp target time reached for target time of the PPS channel.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampSecondsOverflowStatus** (const ENET_Type *const pEnet)

Indicates whether the timestamp seconds overflow status is detected.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTimestampSecondsOverflowStatus** (ENET_Type *const pEnet)

Clear the timestamp seconds overflow status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxTimestampStatusInNanoseconds** (const ENET_Type *const pEnet)

Contains the nanosecond part of timestamp captured for transmit packets when tx status is disabled.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxTimestampStatusInSeconds** (const ENET_Type *const pEnet)

Contains the second part of timestamp captured for transmit packets when tx status is disabled.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampIngressAsymmetryCorrection** (const ENET_Type *const pEnet)

Get timestamp ingress asymmetry correction, used for PDelay_Resp PTP messages.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampIngressAsymmetryCorrection** (ENET_Type *const pEnet, int32_t s32Correction)

Set timestamp ingress asymmetry correction, used for PDelay_Resp PTP messages.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampEgressAsymmetryCorrection** (const ENET_Type *const pEnet)

Get timestamp egress asymmetry correction, used for PDelay_Req PTP messages.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEgressAsymmetryCorrection** (ENET_Type *const pEnet, int32_t s32Correction)

Set timestamp egress asymmetry correction, used for PDelay_Req PTP messages.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampIngressCorrectionInNanoseconds** (const ENET_Type *const pEnet)

Get timestamp ingress correction in nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampIngressCorrectionInNanoseconds** (ENET_Type *const pEnet, int32_t s32NanoSec)

Set timestamp ingress correction in nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampEgressCorrectionInNanoseconds** (const ENET_Type *const pEnet)

Get timestamp egress correction in nanoseconds, used for timestamp in egress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEgressCorrectionInNanoseconds** (ENET_Type *const pEnet, int32_t s32NanoSec)

Set timestamp egress correction in nanoseconds, used for timestamp in egress path.

LOCAL_INLINE int8_t **ENET_HWA_MAC_GetTimestampIngressCorrectionInSubNanoseconds** (const ENET_Type *const pEnet)

Get timestamp ingress correction in sub-nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampIngressCorrectionInSubNanoseconds** (ENET_Type *const pEnet, int8_t s8SubNanoSec)

Set timestamp ingress correction in sub-nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE int8_t **ENET_HWA_MAC_GetTimestampEgressCorrectionInSubNanoseconds** (const ENET_Type *const pEnet)

Get timestamp egress correction in sub-nanoseconds, used for timestamp in egress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEgressCorrectionInSubNanoseconds** (ENET_Type *const pEnet, int8_t s8SubNanoSec)

Set timestamp egress correction in sub-nanoseconds, used for timestamp in egress path.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampIngressLatency** (const ENET_Type *const pEnet)

Get the ingress MAC latency.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetTimestampIngressLatencyInNanoseconds** (const ENET_Type *const pEnet)

Get the ingress MAC latency in nanoseconds.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetTimestampIngressLatencyInSubNanoseconds** (const ENET_Type *const pEnet)

Get the ingress MAC latency in sub-nanoseconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampEgressLatency** (const ENET_Type *const pEnet)

Get the egress MAC latency.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetTimestampEgressLatencyInNanoseconds** (const ENET_Type *const pEnet)

Get the egress MAC latency in nanoseconds.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetTimestampEgressLatencyInSubNanoseconds** (const ENET_Type *const pEnet)

Get the egress MAC latency in sub-nanoseconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPPSControl** (const ENET_Type *const pEnet)

Get PPS control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetPPSControl** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set PPS control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPPSMCGRModeEnFlag** (const ENET_Type *const pEnet,
ENET_MAC_PPSSChannelType ePPSChann)

Get PPS MCGR Mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPPSMCGRModeEnFlag** (ENET_Type *const pEnet,
ENET_MAC_PPSSChannelType ePPSChann, bool bEnable)

Set PPS MCGR Mode enable flag.

LOCAL_INLINE **ENET_MAC_PPSTargetTimeModeType**
ENET_HWA_MAC_GetChannelPPSTargetTimeModeSelect (const ENET_Type *const pEnet,
ENET_MAC_PPSSChannelType ePPSChann)

Get PPS channel target time register operating mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetChannelPPSTargetTimeModeSelect** (ENET_Type *const pEnet,
ENET_MAC_PPSSChannelType ePPSChann, **ENET_MAC_PPSTargetTimeModeType** eTrgtMode)

Set PPS channel target time register operating mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetChannelFlexiblePPSOutputEnFlag** (const ENET_Type *const
pEnet)

Get PPS flexible mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetChannelFlexiblePPSOutputEnFlag** (ENET_Type *const pEnet,
bool bEnable)

Set PPS flexible mode enable flag.

LOCAL_INLINE **ENET_MAC_MCGRCmdType** **ENET_HWA_MAC_GetPPSMCGRCommand** (const ENET_Type
*const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Get PPS MCGR command.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSMCGRCommand (ENET_Type *const pEnet,  
ENET_MAC_PPSSChannelType ePPSChann, ENET_MAC_MCGRCmdType eCmd)
```

Set PPS MCGR command.

```
LOCAL_INLINE ENET_MAC_PPSCmdType ENET_HWA_MAC_GetFlexiblePPSCommand (ENET_Type *const  
pEnet, ENET_MAC_PPSSChannelType ePPSChann)
```

Get flexible PPS command.

```
LOCAL_INLINE void ENET_HWA_MAC_SetFlexiblePPSCommand (ENET_Type *const pEnet,  
ENET_MAC_PPSSChannelType ePPSChann, ENET_MAC_PPSCmdType eCmd)
```

Set flexible PPS command.

```
LOCAL_INLINE ENET_MAC_PPSFreqCtrlType ENET_HWA_MAC_GetFixedPPSOutputFrequency  
(ENET_Type *const pEnet)
```

Get fixed PPS output frequency.

```
LOCAL_INLINE void ENET_HWA_MAC_SetFixedPPSOutputFrequency (ENET_Type *const pEnet,  
ENET_MAC_PPSFreqCtrlType eFreq)
```

Set fixed PPS output frequency.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSTargetTimeInSeconds (const ENET_Type *const pEnet,  
ENET_MAC_PPSSChannelType ePPSChann)
```

Get PPS target time seconds.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSTargetTimeInSeconds (ENET_Type *const pEnet,  
ENET_MAC_PPSSChannelType ePPSChann, uint32_t u32Seconds)
```

Set PPS target time seconds.

```
LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetPPSTargetTimeBusyStatus (const  
ENET_Type *const pEnet, ENET_MAC_PPSSChannelType ePPSChann)
```

Get PPS target time busy status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPPSTargetTimeInNanoseconds** (const ENET_Type *const pEnet, **ENET_MAC_PPSTChannelType** ePPSChann)

Get PPS target time nanoseconds.

LOCAL_INLINE void **ENET_HWA_MAC_SetPPSTargetTimeInNanoseconds** (ENET_Type *const pEnet, **ENET_MAC_PPSTChannelType** ePPSChann, uint32_t u32NanoSeconds)

Set PPS target time nanoseconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPPSInterval** (const ENET_Type *const pEnet, **ENET_MAC_PPSTChannelType** ePPSChann)

Get PPS pulse interval.

LOCAL_INLINE void **ENET_HWA_MAC_SetPPSInterval** (ENET_Type *const pEnet, **ENET_MAC_PPSTChannelType** ePPSChann, uint32_t u32Interval)

Set PPS pulse interval.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPPSWidth** (const ENET_Type *const pEnet, **ENET_MAC_PPSTChannelType** ePPSChann)

Get PPS pulse width.

LOCAL_INLINE void **ENET_HWA_MAC_SetPPSWidth** (ENET_Type *const pEnet, **ENET_MAC_PPSTChannelType** ePPSChann, uint32_t u32Width)

Set PPS pulse width.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetOperationMode** (const ENET_Type *const pEnet)

Get MTL operation mode register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetOperationMode** (ENET_Type *const pEnet, uint32_t u32OpMode)

Set MTL operation mode register value.

LOCAL_INLINE bool **ENET_HWA_MTL_GetCounterResetStatus** (const ENET_Type *const pEnet)

Get MTL counter reset status.

LOCAL_INLINE void **ENET_HWA_MTL_ResetCounters** (ENET_Type *const pEnet)

Reset MTL counters.

LOCAL_INLINE bool **ENET_HWA_MTL_GetPresetCounterStatus** (const ENET_Type *const pEnet)

Get MTL counter preset status.

LOCAL_INLINE void **ENET_HWA_MTL_InitPresetCounters** (ENET_Type *const pEnet)

Preset MTL counters.

LOCAL_INLINE **ENET_MTL_TxScheduleAlgorithmType** **ENET_HWA_MTL_GetTxSchedulingAlgorithm**
(const ENET_Type *const pEnet)

Get MTL algorithm for transmit scheduling.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxSchedulingAlgorithm** (ENET_Type *const pEnet,
ENET_MTL_TxScheduleAlgorithmType eScheduleAlgorithm)

Set MTL algorithm for transmit scheduling.

LOCAL_INLINE **ENET_MTL_RxArbitrationAlgorithmType** **ENET_HWA_MTL_GetRxArbitrationAlgorithm**
(const ENET_Type *const pEnet)

Get MTL algorithm for receive arbitration.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxArbitrationAlgorithm** (ENET_Type *const pEnet,
ENET_MTL_RxArbitrationAlgorithmType eArbitrationAlgorithm)

Set MTL algorithm for receive arbitration.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDropTxPacketStatusEnFlag** (const ENET_Type *const pEnet)

Get whether tx packet status received from the MAC is dropped or not.

LOCAL_INLINE void **ENET_HWA_MTL_SetDropTxPacketStatusEnFlag** (ENET_Type *const pEnet, bool
bEnable)

Set drop tx packet status enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetDebugCtrl** (const ENET_Type *const pEnet)

Get MTL debug control register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetDebugCtrl** (ENET_Type *const pEnet, uint32_t u32DebugCtrl)
Set MTL debug control register value.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxStatusAvailableIntEnFlag** (const ENET_Type *const pEnet)
Get transmit status available interrupt status enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxStatusAvailableIntEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set transmit status available interrupt status enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxPacketAvailableIntEnFlag** (const ENET_Type *const pEnet)
Get receive packet available interrupt status enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxPacketAvailableIntEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set receive packet available interrupt status enable flag.

LOCAL_INLINE **ENET_MTL_DebugAccessFIFOType** **ENET_HWA_MTL_GetDebugAccessFIFO** (const ENET_Type *const pEnet)
Get the FIFO selected for debug access.

LOCAL_INLINE void **ENET_HWA_MTL_SetDebugAccessFIFO** (ENET_Type *const pEnet, **ENET_MTL_DebugAccessFIFOType** eFIFOType)
Set the FIFO selected for debug access.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFIFOWriteEnFlag** (const ENET_Type *const pEnet)
Get write operation on selected FIFO enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFOWriteEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set write operation on selected FIFO enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSelectedFIFOPointerResetStatus** (const ENET_Type *const pEnet)
Get pointers of selected FIFO reset status.

LOCAL_INLINE void **ENET_HWA_MTL_SetSelectedFIFOPointerResetFlag** (ENET_Type *const pEnet, bool bEnable)

Reset pointers of selected FIFO.

LOCAL_INLINE bool **ENET_HWA_MTL_GetAllFIFOPointersResetStatus** (const ENET_Type *const pEnet)

Get pointers of all FIFOs reset status.

LOCAL_INLINE void **ENET_HWA_MTL_SetAllFIFOPointerResetFlag** (ENET_Type *const pEnet, bool bEnable)

Reset pointers of all FIFOs.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFOPacketState** (ENET_Type *const pEnet, **ENET_MTL_FIFOPacketStatusType** eState)

Encoded packet state, the control information to the Tx FIFO or Rx FIFO.

LOCAL_INLINE void **ENET_HWA_MTL_SetByteEnable** (ENET_Type *const pEnet, **ENET_MTL_ByteEnableType** eByteValid)

Indicates the number of data bytes valid in the data register during Write operation.

LOCAL_INLINE **ENET_MTL_FIFODebugAccessType** **ENET_HWA_MTL_GetFIFODebugModeAccessType** (const ENET_Type *const pEnet)

Get pointers of all FIFOs reset status.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFODebugModeAccessType** (ENET_Type *const pEnet, **ENET_MTL_FIFODebugAccessType** eAccessType)

Set the debug mode of accessing to fifo.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFIFODebugAccessEnFlag** (const ENET_Type *const pEnet)

Get FIFO debug access enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFODebugAccessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set FIFO debug access enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetDebugStatus** (const ENET_Type *const pEnet)

Get MTL debug status register value.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetFIFORemainingLocation** (const ENET_Type *const pEnet)

Get MTL remaining locations in the FIFO.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxStatusAvailableIntStatus** (const ENET_Type *const pEnet)

Get MTL transmit status available interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearTxStatusAvailableIntStatus** (ENET_Type *const pEnet)

Clear MTL transmit status available interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxPacketAvailableIntStatus** (const ENET_Type *const pEnet)

Get MTL receive packet available interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearRxPacketAvailableIntStatus** (ENET_Type *const pEnet)

Clear MTL receive packet available interrupt status.

LOCAL_INLINE **ENET_MTL_ByteEnableType** **ENET_HWA_MTL_GetByteEnable** (const ENET_Type *const pEnet)

Get the number of data bytes valid in the data register during Read operation.

LOCAL_INLINE **ENET_MTL_FIFOPacketStatusType** **ENET_HWA_MTL_GetFIFOPacketState** (const ENET_Type *const pEnet)

Get the control or status information of the selected FIFO.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFIFOBusy** (const ENET_Type *const pEnet)

Get MTL FIFO busy status.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetFIFODebugData** (const ENET_Type *const pEnet)

Get MTL FIFO debug data.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFODebugData** (ENET_Type *const pEnet, uint32_t u32Data)

Set MTL FIFO debug data.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetIntStatus** (const ENET_Type *const pEnet)

Get MTL interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDebugIntStatus** (const ENET_Type *const pEnet)

Get debug interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetQueue1IntStatus** (const ENET_Type *const pEnet)

Get Queue 1 interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetQueue0IntStatus** (const ENET_Type *const pEnet)

Get Queue 0 interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxDABasedDMAChannelSelectionEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get DA-based DMA channel selection enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxDABasedDMAChannelSelectionEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set DA-based DMA channel selection enable flag.

LOCAL_INLINE **ENET_DMA_ChannelType** **ENET_HWA_MTL_GetRxDMAChannelMap** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get DMA channel mapped to eQueue.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxDMAChannelMap** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_DMA_ChannelType** eDMAChannel)

Set DMA channel mapped to eQueue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTBSCtrl** (const ENET_Type *const pEnet)

Get Time Based Scheduling control register.

LOCAL_INLINE void **ENET_HWA_MTL_SetTBSCtrl** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Time Based Scheduling control register.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetLaunchExpiryOffset** (const ENET_Type *const pEnet)

Get launch expiry offset.

LOCAL_INLINE void **ENET_HWA_MTL_SetLaunchExpiryOffset** (ENET_Type *const pEnet, uint32_t u32ExpiryOffset)

Set launch expiry offset.

LOCAL_INLINE bool **ENET_HWA_MTL_GetLaunchExpiryOffsetEnFlag** (const ENET_Type *const pEnet)

Get launch expiry offset enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetLaunchExpiryOffsetEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set launch expiry offset enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetEstControl** (const ENET_Type *const pEnet)

Get enhancements to scheduled transmission control register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstControl** (ENET_Type *const pEnet, uint32_t u32EstControl)

Set enhancements to scheduled transmission control register value.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetEstPtpTimeOffset** (const ENET_Type *const pEnet)

Get enhancements to scheduled transmission PTP time offset.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstPtpTimeOffset** (ENET_Type *const pEnet, uint8_t u8Value)

Set enhancements to scheduled transmission PTP time offset.

LOCAL_INLINE uint16_t **ENET_HWA_MTL_GetEstCurrentTimeOffset** (const ENET_Type *const pEnet)

Get enhancements to scheduled transmission current time offset.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstCurrentTimeOffset** (ENET_Type *const pEnet, uint16_t u16Value)

Set enhancements to scheduled transmission current time offset.

LOCAL_INLINE **ENET_MTL_GCLIntervalMultipleType** **ENET_HWA_MTL_GetEstTimeIntervalLeftShift**
(const ENET_Type *const pEnet)
Get EST time interval left shift amount.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstTimeIntervalLeftShift** (ENET_Type *const pEnet,
ENET_MTL_GCLIntervalMultipleType eValue)
Set EST time interval left shift amount.

LOCAL_INLINE **ENET_MTL_ScheduleErrorLoopCountType**
ENET_HWA_MTL_GetEstScheduleErrorLoopCount (const ENET_Type *const pEnet)
Get the loop count to report scheduling error.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstScheduleErrorLoopCount** (ENET_Type *const pEnet,
ENET_MTL_ScheduleErrorLoopCountType eValue)
Set the loop count to report scheduling error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDropSchedulingErrorFramesEnFlag** (const ENET_Type *const
pEnet)
Drop frames causing scheduling error.

LOCAL_INLINE void **ENET_HWA_MTL_SetDropSchedulingErrorFramesEnFlag** (ENET_Type *const pEnet,
bool bEnable)
Enable dropping frames causing scheduling error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDropFrameSizeErrorFramesDisableFlag** (const ENET_Type
*const pEnet)
Drop frames during frame size error.

LOCAL_INLINE void **ENET_HWA_MTL_SetDropFrameSizeErrorFramesDisableFlag** (ENET_Type *const
pEnet, bool bDisable)
Enable dropping frames causing scheduling error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSwitchGateControlListEnFlag** (const ENET_Type *const pEnet)

Get switch gate control list enable flag.

```
LOCAL_INLINE void ENET_HWA_MTL_SetSwitchGateControlListEnFlag (ENET_Type *const pEnet, bool bEnable)
```

Switch gate control list.

```
LOCAL_INLINE bool ENET_HWA_MTL_GetEstEnableFlag (const ENET_Type *const pEnet)
```

Get EST enable flag.

```
LOCAL_INLINE void ENET_HWA_MTL_SetEstEnableFlag (ENET_Type *const pEnet, bool bEnable)
```

Enable EST.

```
LOCAL_INLINE uint8_t ENET_HWA_MTL_GetEstOverheadBytes (const ENET_Type *const pEnet)
```

Get EST overhead bytes value.

```
LOCAL_INLINE void ENET_HWA_MTL_SetEstOverheadBytes (ENET_Type *const pEnet, uint8_t u8Value)
```

Set EST overhead bytes value.

```
LOCAL_INLINE uint32_t ENET_HWA_MTL_GetEstStatus (const ENET_Type *const pEnet)
```

Get EST status register value.

```
LOCAL_INLINE uint8_t ENET_HWA_MTL_GetCurrentGCLSlotNumber (const ENET_Type *const pEnet)
```

Get current GCL slot number.

```
LOCAL_INLINE uint8_t ENET_HWA_MTL_GetBTRErrorLoopCount (const ENET_Type *const pEnet)
```

Get BTR error loop count.

```
LOCAL_INLINE ENET_MTL_GateControlListType ENET_HWA_MTL_GetSoftwareOwnedList (const ENET_Type *const pEnet)
```

Get the gate control list owned by software.

```
LOCAL_INLINE bool ENET_HWA_MTL_GetConstantGateControlErrorFlag (const ENET_Type *const pEnet)
```

Get constant gate control error.

LOCAL_INLINE void **ENET_HWA_MTL_ClearConstantGateControlErrorFlag** (ENET_Type *const pEnet)
Clear constant gate control error flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadOfLineBlockingDueToSchedulingFlag** (const ENET_Type *const pEnet)
Get Head-Of-Line blocking due to scheduling.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadOfLineBlockingDueToFrameSizeFlag** (const ENET_Type *const pEnet)
Get Head-Of-Line blocking due to frame size.

LOCAL_INLINE bool **ENET_HWA_MTL_GetBTRErrorFlag** (const ENET_Type *const pEnet)
Get BTR error flag.

LOCAL_INLINE void **ENET_HWA_MTL_ClearBTRErrorFlag** (ENET_Type *const pEnet)
Clear BTR error flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSwitchGateControlListCompleteFlag** (const ENET_Type *const pEnet)
Get switch gate control list complete status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearSwitchGateControlListCompleteFlag** (ENET_Type *const pEnet)
Clear switch gate control list complete status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetScheduleErrorQueueNumber** (const ENET_Type *const pEnet)
Get the One Hot encoded queue numbers that are having the scheduling related error.

LOCAL_INLINE void **ENET_HWA_MTL_ClearScheduleErrorQueueNumber** (ENET_Type *const pEnet)
Clear the value of schedule error queue number register.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFrameSizeErrorQueueNumber** (const ENET_Type *const pEnet)
Get the One Hot encoded queue numbers that are having the frame size error.

LOCAL_INLINE void **ENET_HWA_MTL_ClearFrameSizeErrorQueueNumber** (ENET_Type *const pEnet)
Clear the value of frame size error queue number register.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFrameSizeErrorFirstQueueNumber** (const ENET_Type *const pEnet)
Get the queue numbers that are having the first frame size error.

LOCAL_INLINE uint16_t **ENET_HWA_MTL_GetFrameSizeErrorFirstFrameSize** (ENET_Type *const pEnet)
Get the frame size of the first frame size error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetConstantGateControlErrorInterruptEnFlag** (ENET_Type *const pEnet)
Get enable flag of constant gate control error interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetConstantGateControlErrorInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set enable flag of constant gate control error interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadofLineBlockingDueToSchedulingInterruptEnFlag** (ENET_Type *const pEnet)
Get enable flag of head of line blocking due to scheduling interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetHeadofLineBlockingDueToSchedulingInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set enable flag of head of line blocking due to scheduling interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadofLineBlockingDueToFrameSizeInterruptEnFlag** (ENET_Type *const pEnet)
Get enable flag of head of line blocking due to frame size interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetHeadofLineBlockingDueToFrameSizeInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set enable flag of head of line blocking due to frame size interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetBaseTimeRegisterErrorInterruptEnFlag** (ENET_Type *const pEnet)

Get enable flag of base time register error interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetBaseTimeRegisterErrorInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set enable flag of base time register error interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSwitchListInterruptEnFlag** (ENET_Type *const pEnet)

Get enable flag of switch GCL completion interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetSwitchListInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set enable flag of base time switch GCL completion interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetEstEcInjectErrorControl** (ENET_Type *const pEnet)

Get the mode of EST ECC inject error control for EST memory.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstEcInjectErrorControl** (ENET_Type *const pEnet, bool bEnable)

Set the mode of EST ECC inject error control for EST memory.

LOCAL_INLINE bool **ENET_HWA_MTL_GetEstEcInjectAddressErrorEnFlag** (ENET_Type *const pEnet)

Get enable flag of EST ECC address error injection.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstEcInjectAddressErrorEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set enable flag of EST ECC address error injection.

LOCAL_INLINE bool **ENET_HWA_MTL_GetEstEcInjectErrorEnFlag** (ENET_Type *const pEnet)

Get enable flag of EST ECC error injection.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstEccInjectErrorEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set enable flag of EST ECC error injection.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetEstGateControlListAddress** (ENET_Type *const pEnet)

Get GCL address or GCL related registers address depend on GCRR bit.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstGateControlListAddress** (ENET_Type *const pEnet, uint8_t eAddress)

Set GCL address or GCL related registers address depend on GCRR bit.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetGclDebugModeBankSelect** (ENET_Type *const pEnet)

Get GCL debug mode bank selected.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclDebugModeBankSelect** (ENET_Type *const pEnet, uint8_t eBank)

Set GCL debug mode bank.

LOCAL_INLINE bool **ENET_HWA_MTL_GetGclDebugModeEnFlag** (ENET_Type *const pEnet)

Get GCL debug mode enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclDebugModeEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set GCL debug mode enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetGateControlReleatedRegistersEnFlag** (ENET_Type *const pEnet)

Get gate control releated registers enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetGateControlReleatedRegistersEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set gate control releated registers enable flag.

LOCAL_INLINE **ENET_MTL_GclOperationType** **ENET_HWA_MTL_GetGclOperation** (ENET_Type *const pEnet)

Get GCL operation mode (read or write)

LOCAL_INLINE void **ENET_HWA_MTL_SetGclOperation** (ENET_Type *const pEnet,
ENET_MTL_GclOperationType eOperation)

Set GCL operation mode (read or write)

LOCAL_INLINE bool **ENET_HWA_MTL_GetGclOperationBusyState** (ENET_Type *const pEnet)

Get GCL operation busy status.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclOperationStartFlag** (ENET_Type *const pEnet, bool bEnable)

Start GCL Read/Write operation.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetGclData** (ENET_Type *const pEnet)

Get GCL data.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclData** (ENET_Type *const pEnet, uint32_t eData)

Set GCL data register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetPreemptionClassification** (ENET_Type *const pEnet, uint8
u8value)

Set the corresponding Queue classified as express or preemptable.

LOCAL_INLINE void **ENET_HWA_MTL_SetPreemptionReleaseAdvance** (ENET_Type *const pEnet, uint16
u16value)

Set the maximum time in nanoseconds that can elapse between issuing a RELEASE to the MAC and the MAC being ready to resume transmission of preemptable frames.

LOCAL_INLINE void **ENET_HWA_MTL_SetPreemptionHoldAdvance** (ENET_Type *const pEnet, uint16
u16value)

Set the maximum time in nanoseconds that can elapse between issuing a HOLD to the MAC and the MAC ceasing to transmit any preemptable frame.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTxOperationMode** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get MTL queue Tx operation mode register value.

```
LOCAL_INLINE void ENET_HWA_MTL_SetTxOperationMode (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue, uint32_t u32OpMode)
```

Set MTL queue Tx operation mode register value.

```
LOCAL_INLINE uint8_t ENET_HWA_MTL_GetTxQueueSize (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get MTL transmit queue size.

```
LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueSize (ENET_Type *const pEnet, ENET_MTL_QueueType  
eQueue, uint8_t u8QueueSize)
```

Set MTL transmit queue size.

```
LOCAL_INLINE ENET_MTL_TxThresholdType ENET_HWA_MTL_GetTxThreshold (const ENET_Type *const  
pEnet, ENET_MTL_QueueType eQueue)
```

Get the threshold level of the MTL Tx Queue.

```
LOCAL_INLINE void ENET_HWA_MTL_SetTxThreshold (ENET_Type *const pEnet, ENET_MTL_QueueType  
eQueue, ENET_MTL_TxThresholdType eThreshold)
```

Set the threshold level of the MTL Tx Queue.

```
LOCAL_INLINE ENET_MTL_QueueEnableType ENET_HWA_MTL_GetTxQueueEnStatus (const ENET_Type  
*const pEnet, ENET_MTL_QueueType eQueue)
```

Get the transmit queue enable mode.

```
LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueEnStatus (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue, ENET_MTL_QueueEnableType eEnable)
```

Set the transmit queue enable mode.

```
LOCAL_INLINE bool ENET_HWA_MTL_GetTxStoreForwardModeEnFlag (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get transmit queue store and forward enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxStoreForwardModeEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)
Set transmit queue store and forward enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxFlushState** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)
Get transmit queue flush state.

LOCAL_INLINE void **ENET_HWA_MTL_FlushTxQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)
Flush transmit queue.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxUnderflowPacketCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint16_t *pPacketCnt)
Get the underflow counter register of transmit queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTxDebugStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)
Get transmit queue debug status.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetTxStatusWordCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)
Get the current number of status in the transmit Status FIFO of this queue.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetTxPacketCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)
Get the current number of packets in the transmit queue.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxStatusFIFOFullStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)
Get the full flag of transmit Status FIFO.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxQueueNotEmptyStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)
Get the not empty flag of transmit queue data.

LOCAL_INLINE ENET_MTL_WriteControllerStatusType

ENET_HWA_MTL_GetTxQueueWriteControllerStatus (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the transmit queue write controller status, indicates that the MTL Tx Queue Write controller is active, and it is transferring the data to the transmit queue.

LOCAL_INLINE ENET_MTL_ReadControllerStatusType

ENET_HWA_MTL_GetTxQueueReadControllerStatus (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the transmit queue read controller status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxQueuePausedStatus** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the transmit queue paused status.

LOCAL_INLINE void **ENET_HWA_MTL_SetAVAlgorithmCBSEnFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint8 bEnable)

Set AV algorithm for strict priority or credit based shaper.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetSlotAverageBits** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the average bits per slot, when the DCB operation is enabled for the queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTxQueueQuantumOrWeight** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the quantum or weights for transmit algorithm (DWRR WRR WFQ)

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueQuantumOrWeight** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32_t u32QuantumWeight)

Set the quantum or weights for transmit algorithm (DWRR WRR WFQ)

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueSendSlopeCredit** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32 u32SendSlopeCredit)

Set the sendSlope credit value required for the credit-based shaper algorithm for the Queue.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueHiCredit** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32 u32hiCredit)

Set hiCredit value required for the credit-based shaper algorithm for the Queue.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueLoCredit** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32 u32LoCredit)

Set the loCredit value required for the credit-based shaper algorithm for the Queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetInterruptCtrlStatus** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the interrupt enable and status for the queue interrupts.

LOCAL_INLINE void **ENET_HWA_MTL_SetInterruptCtrlStatus** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32_t u32CtrlStatus)

Set the interrupt enable and status for the queue interrupts.

LOCAL_INLINE void **ENET_HWA_MTL_ClearAllIntStatus** (ENET_Type *const pEnet, **ENET_MTL_QueueType**
eQueue)

Clear all the interrupts of the MTL queue.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxQueueOverflowIntEnFlag** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue overflow interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueueOverflowIntEnFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)

Set receive queue overflow interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxQueueOverflowIntStatus** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue overflow interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearRxQueueOverflowIntStatus** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Clear receive queue overflow interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSlotAverageBitsIntEnFlag** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get average bits per slot interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetSlotAverageBitsIntEnFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)

Set average bits per slot interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxQueueUnderflowIntEnFlag** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get transmit queue underflow interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueUnderflowIntEnFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)

Set transmit queue underflow interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSlotAverageBitsIntStatus** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get average bits per slot interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearSlotAverageBitsIntStatus** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Clear average bits per slot interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxQueueUnderflowIntStatus** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get transmit queue underflow interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearTxQueueUnderflowIntStatus** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Clear transmit queue underflow interrupt status.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetRxOperationMode** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive operation mode of the queue.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxOperationMode** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint32_t u32OpMode)

Set receive operation mode of the queue.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetRxQueueSize** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue size.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueueSize** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint8_t u8QueueSize)

Set receive queue size.

LOCAL_INLINE **ENET_MTL_RxFlowCtrlThresholdType** **ENET_HWA_MTL_GetRxDeactivateFlowCtrlThres** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the threshold of receive queue at which the flow control is deactivated.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxDeactivateFlowCtrlThres** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_RxFlowCtrlThresholdType** eThreshold)

Set the threshold of receive queue at which the flow control is deactivated.

LOCAL_INLINE **ENET_MTL_RxFlowCtrlThresholdType** **ENET_HWA_MTL_GetRxActivateFlowCtrlThres** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the threshold of receive queue at which the flow control is activated.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxActivateFlowCtrlThres** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_RxFlowCtrlThresholdType** eThreshold)

Set the threshold of receive queue at which the flow control is activated.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxHwFlowCtrlEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue hardware flow control enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxHwFlowCtrlEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set receive queue hardware flow control enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetForwardTCPChecksumErrorPacketEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get forwarding of TCP/IP checksum error packets enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetForwardTCPChecksumErrorPacketEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set forwarding of TCP/IP checksum error packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxStoreForwardModeEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue store and forward enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxStoreForwardModeEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set receive queue store and forward enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetForwardErrorPacketEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get forward error packets enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetForwardErrorPacketEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set forward error packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetForwardUndersizedPacketEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get forward undersized good packets enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetForwardUndersizedPacketEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set forward undersized good packets enable flag.

LOCAL_INLINE **ENET_MTL_RxThresholdType** **ENET_HWA_MTL_GetRxThreshold** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the receive queue threshold control.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxThreshold** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_RxThresholdType** eThreshold)

Set the receive queue threshold.

LOCAL_INLINE void **ENET_HWA_MTL_GetRxMissedPacketOverflowCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool *pMissedPacketCntOverflowFlag, uint16_t *pMissedPacketCnt, bool *pOverflowPacketCntOverflowFlag, uint16_t *pOverflowPacketCnt)

Get the missed packet and overflow counters of the queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetRxDebugStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the receive queue debug status.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetRxPacketCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the number of packets in receive queue.

LOCAL_INLINE **ENET_MTL_RxQueueStatusType** **ENET_HWA_MTL_GetRxFillLevelStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue Fill-Level status.

LOCAL_INLINE **ENET_MTL_ReadControllerStatusType**
ENET_HWA_MTL_GetRxQueueReadControllerStatus (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue read controller state.

LOCAL_INLINE **ENET_MTL_WriteControllerStatusType**

ENET_HWA_MTL_GetRxQueueWriteControllerStatus (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue write controller state.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetRxControl** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get MTL queue receive control register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxControl** (ENET_Type *const pEnet, **ENET_MTL_QueueType**
eQueue, uint32_t u32Ctrl)

Set MTL queue receive control register value.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxQueuePacketArbitrationEnFlag** (const ENET_Type *const
pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue packet arbitration enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueuePacketArbitrationEnFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)

Set receive queue packet arbitration enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetRxQueueWeight** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue weight.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueueWeight** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint8_t u8Weight)

Set receive queue weight.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetMode** (const ENET_Type *const pEnet)

Get DMA bus mode register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetMode** (ENET_Type *const pEnet, uint32_t u32Mode)

Set DMA bus mode register value.

LOCAL_INLINE **ENET_DMA_StatusType** **ENET_HWA_DMA_GetSwResetStatus** (const ENET_Type *const pEnet)

Get DMA software reset status.

LOCAL_INLINE void **ENET_HWA_DMA_SwReset** (ENET_Type *const pEnet)

Initiate DMA software reset.

LOCAL_INLINE **ENET_DMA_IntModeType** **ENET_HWA_DMA_GetInterruptMode** (const ENET_Type *const pEnet)

Get DMA interrupt mode.

LOCAL_INLINE void **ENET_HWA_DMA_SetInterruptMode** (ENET_Type *const pEnet,
ENET_DMA_IntModeType eIntMode)

Set DMA interrupt mode.

LOCAL_INLINE **ENET_DMA_PriorityRatioType** **ENET_HWA_DMA_GetPriorityRatio** (const ENET_Type *const pEnet)

Get DMA Rx:Tx or Tx:Rx priority ratio.

LOCAL_INLINE void **ENET_HWA_DMA_SetPriorityRatio** (ENET_Type *const pEnet,
ENET_DMA_PriorityRatioType ePriorityRatio)

Set DMA Rx:Tx or Tx:Rx priority ratio.

LOCAL_INLINE **ENET_DMA_PriorityModeType** **ENET_HWA_DMA_GetPriorityMode** (const ENET_Type *const pEnet)

Get DMA priority mode.

LOCAL_INLINE void **ENET_HWA_DMA_SetPriorityMode** (ENET_Type *const pEnet,
ENET_DMA_PriorityModeType ePriorityMode)

Set DMA priority mode, Rx:Tx or Tx:Rx.

LOCAL_INLINE **ENET_DMA_TxArbitrationAlgorithmType** **ENET_HWA_DMA_GetTxArbitrationAlgorithm**
(const ENET_Type *const pEnet)

Get DMA arbitration algorithm between transmit channels.

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxArbitrationAlgorithm (ENET_Type *const pEnet,  
ENET_DMA_TxArbitrationAlgorithmType eArbitrationAlgorithm)
```

Set DMA arbitration algorithm between transmit channels.

```
LOCAL_INLINE ENET_DMA_ArbitrationSchemeType ENET_HWA_DMA_GetTxRxArbitrationScheme (const  
ENET_Type *const pEnet)
```

Get DMA arbitration scheme between the transmit and receive paths.

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxRxArbitrationScheme (ENET_Type *const pEnet,  
ENET_DMA_ArbitrationSchemeType eArbitrationScheme)
```

Set DMA arbitration scheme between the transmit and receive paths.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetRebuildINCRxBurstEnFlag (const ENET_Type *const pEnet)
```

Get DMA rebuild INCRx burst enable flag.

```
LOCAL_INLINE void ENET_HWA_DMA_SetRebuildINCRxBurstEnFlag (ENET_Type *const pEnet, bool  
bEnable)
```

Set DMA rebuild INCRx burst enable flag.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetMixedBurstEnFlag (const ENET_Type *const pEnet)
```

Get DMA mixed burst length enable flag.

```
LOCAL_INLINE void ENET_HWA_DMA_SetMixedBurstEnFlag (ENET_Type *const pEnet, bool bEnable)
```

Set DMA mixed burst length enable flag.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetAddressAlignedBeatsEnFlag (const ENET_Type *const pEnet)
```

Get DMA Address-Aligned Beats enable flag.

```
LOCAL_INLINE void ENET_HWA_DMA_SetAddressAlignedBeatsEnFlag (ENET_Type *const pEnet, bool  
bEnable)
```

Set DMA Address-Aligned Beats enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetFixedBurstLengthEnFlag** (const ENET_Type *const pEnet)
Get DMA fixed burst length enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetFixedBurstLengthEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set DMA fixed burst length enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetDMAIntStatus** (const ENET_Type *const pEnet)
Get DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetMACIntStatus** (const ENET_Type *const pEnet)
Get MAC interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetMTLIntStatus** (const ENET_Type *const pEnet)
Get MTL interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetDMAChannel0IntStatus** (const ENET_Type *const pEnet)
Get DMA channel0 interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetDMAChannel1IntStatus** (const ENET_Type *const pEnet)
Get DMA channel1 interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetDMAChannelIntStatus** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eChannel)
Get DMA channel interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetAHBMasterBusyStatus** (const ENET_Type *const pEnet)
Get AXI Master Read Channel Status.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetFetchTimeOffset** (const ENET_Type *const pEnet, uint8_t
eCtrlIndex)
Get DMA TBS fetch time offset.

LOCAL_INLINE void **ENET_HWA_DMA_SetFetchTimeOffset** (ENET_Type *const pEnet, uint8_t eCtrlIndex, uint32_t u32FetchTimeOffset)

Set DMA TBS fetch time offset.

LOCAL_INLINE bool **ENET_HWA_DMA_GetFetchTimeOffsetValid** (const ENET_Type *const pEnet, uint8_t eCtrlIndex)

Get DMA TBS fetch time offset valid.

LOCAL_INLINE void **ENET_HWA_DMA_SetFetchTimeOffsetValid** (ENET_Type *const pEnet, uint8_t eCtrlIndex, bool bEnable)

Set DMA TBS fetch time offset valid.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetChannelCtrl** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA channel control register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetChannelCtrl** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint32_t u32ChannelCtrl)

Set DMA channel control register value.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetDescriptorSkipLength** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA descriptor skip length.

LOCAL_INLINE void **ENET_HWA_DMA_SetDescriptorSkipLength** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint8_t u8Length)

Set DMA descriptor skip length.

LOCAL_INLINE bool **ENET_HWA_DMA_Get8xPBLModeEnFlag** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA burst length multiply by 8 mode.

LOCAL_INLINE void **ENET_HWA_DMA_Set8xPBLModeEnFlag** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, bool bEnable)

Set DMA burst length multiply by 8 mode.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetChannelTxCtrl** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit channel control register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetChannelTxCtrl** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32TxCtrl)

Set DMA transmit channel control register value.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEnhancedDescriptorEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA enhanced descriptor enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetEnhancedDescriptorEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set DMA enhanced descriptor enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEarlyTxIntEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA early transmit interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetEarlyTxIntEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set DMA early transmit interrupt enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetTxBurstLength** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit programmable burst length.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxBurstLength** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8Length)

Set DMA transmit programmable burst length.

LOCAL_INLINE bool **ENET_HWA_DMA_GetOperateOnSecondPacketEnFlag** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA operate on second packet enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetOperateOnSecondPacketEnFlag** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, bool bEnable)

Set DMA operate on second packet enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetTxChannelWeight** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA transmit channel weight.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxChannelWeight** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint8_t u8Weight)

Set DMA transmit channel weight.

LOCAL_INLINE **ENET_DMA_StatusType** **ENET_HWA_DMA_GetTxActiveStatus** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA transmit channel active status.

LOCAL_INLINE void **ENET_HWA_DMA_StartTx** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Start DMA transmission.

LOCAL_INLINE void **ENET_HWA_DMA_StopTx** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Stop DMA transmission.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetChannelRxCtrl** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA receive channel control register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetChannelRxCtrl** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint32_t u32RxCtrl)

Set DMA receive channel control register value.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxPacketFlushEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive packet flush enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxPacketFlushEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set DMA receive packet flush enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEarlyRxIntEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA early receive interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetEarlyRxIntEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set DMA early receive interrupt enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetRxBurstLength** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive programmable burst length.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxBurstLength** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8Length)

Set DMA receive programmable burst length.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetRxBufferSize** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive buffer size.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxBufferSize** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t u16BufferSize)

Set DMA receive buffer size.

LOCAL_INLINE **ENET_DMA_StatusType** **ENET_HWA_DMA_GetRxActiveStatus** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA receive channel active status.

LOCAL_INLINE void **ENET_HWA_DMA_StartRx** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Start DMA reception.

LOCAL_INLINE void **ENET_HWA_DMA_StopRx** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Stop DMA reception.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetTxDescListHeadAddr** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA channel Tx descriptor list header address.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxDescListHeadAddr** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint32_t u32TxAddr)

Set DMA channel Tx descriptor list header address.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetRxDescListHeadAddr** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA channel Rx descriptor list header address.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxDescListHeadAddr** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint32_t u32RxAddr)

Set DMA channel Rx descriptor list header address.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetTxDescListTailAddr** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA channel Tx descriptor list tail address.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxDescListTailAddr** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint32_t u32TxAddr)

Set DMA channel Tx descriptor list tail address.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetRxDescListTailAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Rx descriptor list tail address.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxDescListTailAddr** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32RxAddr)

Set DMA channel Rx descriptor list tail address.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetTxDescRingLength** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Tx descriptor ring length.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxDescRingLength** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t u16Length)

Set DMA channel Tx descriptor ring length.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetRxDescRingLength** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Rx descriptor ring length.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxDescRingLength** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t u16Length)

Set DMA channel Rx descriptor ring length.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetEnabledInterrupts** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel enabled interrupts.

LOCAL_INLINE void **ENET_HWA_DMA_SetEnabledInterrupts** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32IntEnable)

Set DMA channel interrupts enable state.

LOCAL_INLINE void **ENET_HWA_DMA_EnableInterrupt** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, **ENET_DMA_InterruptFlagType** eIntFlag)

Enable one of the DMA channel interrupts.

LOCAL_INLINE void **ENET_HWA_DMA_DisableInterrupt** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, **ENET_DMA_InterruptFlagType** eIntFlag)

Disable one of the DMA channel interrupts.

LOCAL_INLINE **ENET_DMA_WDGCounterUnitType** **ENET_HWA_DMA_GetRxWDGCounterUnit** (const
ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get receive interrupt watchdog timer count unit.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxWDGCounterUnit** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, **ENET_DMA_WDGCounterUnitType** eCounterUnit)

Set receive interrupt watchdog timer count unit.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetRxWDGCount** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get receive interrupt watchdog timer count.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxWDGCount** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8WDGCnt)

Set receive interrupt watchdog timer count.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetRefSlotNumber** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get reference slot number in the DMA.

LOCAL_INLINE void **ENET_HWA_DMA_SetRefSlotNumber** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8SlotNumber)

Set reference slot number in the DMA.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetSlotInterval** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get slot interval value.

LOCAL_INLINE void **ENET_HWA_DMA_SetSlotInterval** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t u16SlotInterval)

Set slot interval value.

LOCAL_INLINE bool **ENET_HWA_DMA_GetAdvanceSlotCheckEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get advance slot check enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetAdvanceSlotCheckEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set advance slot check enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetSlotComparisonEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get slot comparison enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetSlotComparisonEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set slot comparison enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentTxDescAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Tx descriptor address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentRxDescAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Rx descriptor address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentTxBufAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Tx buffer address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentRxBufAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Rx buffer address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentStatus** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current status.

LOCAL_INLINE void **ENET_HWA_DMA_SetCurrentStatus** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32Status)

Set DMA current status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearAllAbnormalInterruptFlags** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear all DMA abnormal interrupts.

LOCAL_INLINE void **ENET_HWA_DMA_ClearAllNormalInterruptFlags** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear all DMA normal interrupts.

LOCAL_INLINE **ENET_DMA_RxErrorType** **ENET_HWA_DMA_GetRxError** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA Rx error status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxError** (ENET_Type *const pEnet, **ENET_DMA_ChannelType**
eDMAChannel)

Clear DMA Rx error status.

LOCAL_INLINE **ENET_DMA_TxErrorType** **ENET_HWA_DMA_GetTxError** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA Tx error status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxError** (ENET_Type *const pEnet, **ENET_DMA_ChannelType**
eDMAChannel)

Clear DMA Tx error status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetNormalInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA normal interrupt summary.

LOCAL_INLINE void **ENET_HWA_DMA_ClearNormalInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA normal interrupt summary.

LOCAL_INLINE bool **ENET_HWA_DMA_GetAbnormalInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA abnormal interrupt summary.

LOCAL_INLINE void **ENET_HWA_DMA_ClearAbnormalInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA abnormal interrupt summary.

LOCAL_INLINE bool **ENET_HWA_DMA_GetContextDescriptorErrorFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA context descriptor error.

LOCAL_INLINE void **ENET_HWA_DMA_ClearContextDescriptorErrorFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA context descriptor error.

LOCAL_INLINE bool **ENET_HWA_DMA_GetFatalBusErrorFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA fatal bus error.

LOCAL_INLINE void **ENET_HWA_DMA_ClearFatalBusErrorFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA fatal bus error.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEarlyRxInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA early receive interrupt.

LOCAL_INLINE void **ENET_HWA_DMA_ClearEarlyRxInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA early receive interrupt.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEarlyTxInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA early transmit interrupt.

LOCAL_INLINE void **ENET_HWA_DMA_ClearEarlyTxInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA early transmit interrupt.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxWDGTimeoutFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive watchdog timeout status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxWDGTimeoutFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive watchdog timeout status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxProcessStoppedFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive process stopped status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxProcessStoppedFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive process stopped status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxBufferUnavailableFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive buffer unavailable status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxBufferUnavailableFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive buffer unavailable status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive interrupt status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetTxBufferUnavailableFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit buffer unavailable status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxBufferUnavailableFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA transmit buffer unavailable status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetTxProcessStoppedFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit process stopped status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxProcessStoppedFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA transmit process stopped status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetTxInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit interrupt status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)
Clear DMA transmit interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetMissFrameCount** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t *pMissPacketCnt)
Get the miss frame count and overflow status of the Ethernet DMA channel.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetEarlyRxInterruptCount** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)
Get the count of the number of times Early Receive Interrupt was asserted.

Detailed Description

Enumeration Type Documentation

enum ENET_DMA_ArbitrationSchemeType

ENET DMA arbitration scheme between the transmit and receive paths.

Enumerator:

ENET_DMA_TXR X_ARBITRATION _SCHEME_WEIG HTED_ROUND_ ROBIN	Use the weighted round-robin for arbitration
ENET_DMA_TXR X_ARBITRATION _SCHEME_FIXE D_PRIORITY	Use the fixed priority for arbitration

enum ENET_DMA_BurstModeType

ENET DMA burst mode type.

Enumerator:

ENET_BURSTLE NGTH_FIXED	Use the fixed burst
ENET_BURSTLE NGTH_MIXED	Use the mixed burst Length

enum ENET_DMA_ChannelType

ENET DMA channel index type.

Enumerator:

ENET_DMA_CHA NNEL_0	
ENET_DMA_CHA NNEL_1	

enum ENET_DMA_InterruptFlagType

ENET DMA interrupt flag type.

Enumerator:

ENET_DMA_TX_ INTERRUPT	
---------------------------	--

ENET_DMA_TX_STOPPED_INTERRUPT	
ENET_DMA_TX_BUFFER_UNAVAILABLE_INTERRUPT	
ENET_DMA_RX_INTERRUPT	
ENET_DMA_RX_BUFFER_UNAVAILABLE_INTERRUPT	
ENET_DMA_RX_STOPPED_INTERRUPT	
ENET_DMA_RX_WATCHDOG_TIMEOUT_INTERRUPT	
ENET_DMA_EARLY_TX_INTERRUPT	
ENET_DMA_EARLY_RX_INTERRUPT	
ENET_DMA_FATAL_BUS_ERROR_INTERRUPT	
ENET_DMA_CONTEXT_DESCRIPTOR_ERROR_INTERRUPT	
ENET_DMA_ABNORMAL_INTERRUPT	

ENET_DMA_NO RMAL_INTERRUPT PT	
-------------------------------------	--

enum ENET_DMA_IntModeType

The interrupt mode of ENET DMA.

Enumerator:

ENET_DMA_INTERRUPT_SBD_PERCH_PULSE	sbd_perch_* are pulse signals and sbd_inter_o will assert
ENET_DMA_INTERRUPT_SBD_PERCH_LEVEL	sbd_perch_* are level signals and sbd_inter_o will not assert
ENET_DMA_INTERRUPT_SBD_PERCH_LEVEL_REENTERANT	sbd_perch_* are level signals and will regenerate if the same event occur again before the signals cleared. sbd_inter_o will not assert

enum ENET_DMA_PriorityModeType

ENET DMA priority mode type.

Enumerator:

ENET_DMA_PRIORITY_MODE_RX_TX	The priority ratio is RX:TX
ENET_DMA_PRIORITY_MODE_TX_RX	The priority ratio is TX:RX

enum ENET_DMA_PriorityRatioType

ENET DMA priority ratio type.

Enumerator:

ENET_DMA_PRIORITY_RATIO_1	The priority ratio is 1:1
ENET_DMA_PRIORITY_RATIO_2	The priority ratio is 2:1
ENET_DMA_PRIORITY_RATIO_3	The priority ratio is 3:1
ENET_DMA_PRIORITY_RATIO_4	The priority ratio is 4:1
ENET_DMA_PRIORITY_RATIO_5	The priority ratio is 5:1
ENET_DMA_PRIORITY_RATIO_6	The priority ratio is 6:1
ENET_DMA_PRIORITY_RATIO_7	The priority ratio is 7:1
ENET_DMA_PRIORITY_RATIO_8	The priority ratio is 8:1

enum ENET_DMA_RxErrorType

ENET DMA RX error type.

Enumerator:

ENET_DMA_RX_ NO_ERROR	
ENET_DMA_RX_ READ_DESCRIP TER_ERROR	
ENET_DMA_RX_ WRITE_DESCRIP TER_ERROR	
ENET_DMA_RX_ READ_BUFFER_ ERROR	
ENET_DMA_RX_ WRITE_BUFFER _ERROR	

enum ENET_DMA_StatusType

ENET DMA running status.

Enumerator:

ENET_DMA_STA TUS_IDLE	The ENET DMA engine is stoppeed
ENET_DMA_STA TUS_ACTIVE	The ENET DMA engine is running

enum ENET_DMA_TxArbitrationAlgorithmType

ENET DMA arbitration algorithm between transmit channels.

Enumerator:

ENET_DMA_TX_ ARBITRATION_A	Use the fixed priority for arbitration
-------------------------------	--

ENET_DMA_TX_ALGORITHM_FIXED_PRIORITY	
ENET_DMA_TX_ALGORITHM_WEIGHTED_FIXED_PRIORITY	Use the weighted fixed priority for arbitration
ENET_DMA_TX_ALGORITHM_WEIGHTED_ROUND_ROBIN	Use the weighted round-robin for arbitration

enum ENET_DMA_TxErrorType

ENET DMA TX error type.

Enumerator:

ENET_DMA_TX_NO_ERROR	
ENET_DMA_TX_READ_DESCRIPTOR_ERROR	
ENET_DMA_TX_WRITE_DESCRIPTOR_ERROR	
ENET_DMA_TX_READ_BUFFER_ERROR	
ENET_DMA_TX_WRITE_BUFFER_ERROR	

enum ENET_DMA_WDGCounterUnitType

ENET DMA watchdog counter unit type.

Enumerator:

ENET_DMA_WD G_COUNTER_U NIT_256_CYCLE	
ENET_DMA_WD G_COUNTER_U NIT_512_CYCLE	
ENET_DMA_WD G_COUNTER_U NIT_1024_CYCL E	
ENET_DMA_WD G_COUNTER_U NIT_2048_CYCL E	

enum ENET_MAC_AddrFilterModeType

MAC address filter mode type definition.

Enumerator:

ENET_MAC_ADD R_FILTER_DIRE CT	
ENET_MAC_ADD R_FILTER_INVE RSE	

enum ENET_MAC_AddrType

MAC address destination or source type.

Enumerator:

ENET_MAC_ADD R_TYPE_DEST	
ENET_MAC_ADD R_TYPE_SRC	

enum ENET_MAC_BackOffLimitType

MAC back off limit type definition.

Enumerator:

ENET_MAC_BAC K_OFF_LIMIT_1 0_SLOT	
ENET_MAC_BAC K_OFF_LIMIT_8 _SLOT	
ENET_MAC_BAC K_OFF_LIMIT_4 _SLOT	
ENET_MAC_BAC K_OFF_LIMIT_1 _SLOT	

enum ENET_MAC_CSRClockRangeType

MAC MDIO access clock source range type.

Enumerator:

ENET_MAC_CSR _CLOCK_RANGE _60_100MHZ	
--	--

ENET_MAC_CSR _CLOCK_RANGE _100_150MHZ	
ENET_MAC_CSR _CLOCK_RANGE _20_35MHZ	
ENET_MAC_CSR _CLOCK_RANGE _35_60MHZ	
ENET_MAC_CSR _CLOCK_RANGE _150_250MHZ	
ENET_MAC_CSR _CLOCK_RANGE _250_300MHZ	

enum ENET_MAC_CtrlPacketFilterModeType

MAC control packets filter mode type definition.

Enumerator:

ENET_MAC_CTR L_PACKET_FILT ER_ALL	
ENET_MAC_CTR L_PACKET_FILT ER_PAUSE_PAC KET	
ENET_MAC_CTR L_PACKET_FOR WARD_ALL	
ENET_MAC_CTR L_PACKET_ADD R_FILTER	

enum ENET_MAC_DuplexModeType

MAC duplex mode type definition.

Enumerator:

ENET_MAC_HAL F_DUPLEX	
ENET_MAC_FUL L_DUPLEX	

enum ENET_MAC_ExtInterPacketGapType

MAC extend inner packet gap type definition.

Enumerator:

ENET_MAC_EXT _INTER_PACKET _GAP_104_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_112_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_120_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_128_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_136_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_144_BIT	

ENET_MAC_EXT _INTER_PACKET _GAP_152_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_160_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_168_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_176_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_184_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_192_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_200_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_208_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_216_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_224_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_232_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_240_BIT	

ENET_MAC_EXT _INTER_PACKET _GAP_248_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_256_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_264_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_272_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_280_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_288_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_296_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_304_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_312_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_320_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_328_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_336_BIT	

ENET_MAC_EXT _INTER_PACKET _GAP_344_BIT	
ENET_MAC_EXT _INTER_PACKET _GAP_352_BIT	

enum ENET_MAC_HashOrPerfectModeType

MAC perfect or hash filter mode type definition.

Enumerator:

ENET_MAC_USE _PERFECT_FILTER	
ENET_MAC_USE _HASH_FILTER	

enum ENET_MAC_InterPacketGapType

MAC inter packet gap type definition.

Enumerator:

ENET_MAC_INTER_PACKET_GAP P_96_BIT	
ENET_MAC_INTER_PACKET_GAP P_88_BIT	
ENET_MAC_INTER_PACKET_GAP P_80_BIT	

ENET_MAC_INT ER_PACKET_GA P_72_BIT	
ENET_MAC_INT ER_PACKET_GA P_64_BIT	
ENET_MAC_INT ER_PACKET_GA P_56_BIT	
ENET_MAC_INT ER_PACKET_GA P_48_BIT	
ENET_MAC_INT ER_PACKET_GA P_40_BIT	

enum ENET_MAC_L3L4FilterGroupType

MAC layer3 layer 4 filter group index type.

Enumerator:

ENET_MAC_L3_ L4_FILTER_GRO UP_0	
ENET_MAC_L3_ L4_FILTER_GRO UP_1	
ENET_MAC_L3_ L4_FILTER_GRO UP_2	
ENET_MAC_L3_ L4_FILTER_GRO UP_3	

enum ENET_MAC_Layer3ProtocolType

MAC layer 3 filter protocol type.

Enumerator:

ENET_MAC_LAY ER_3_PROTOCO L_IPV4	
ENET_MAC_LAY ER_3_PROTOCO L_IPV6	

enum ENET_MAC_Layer4ProtocolType

MAC layer 4 filter protocol type.

Enumerator:

ENET_MAC_LAY ER_4_PROTOCO L_TCP	
ENET_MAC_LAY ER_4_PROTOCO L_UDP	

enum ENET_MAC_MCGRCmdType

PTP flexible MCGR command type.

Enumerator:

ENET_MAC_MC GR_OPERATION _NOT_CARRIED _OUT	
---	--

ENET_MAC_MC GR_CAPTURE_R ISING_EDGE	
ENET_MAC_MC GR_CAPTURE_F ALLING_EDGE	
ENET_MAC_MC GR_CAPTURE_B OTH_EDGE	
ENET_MAC_MC GR_TOGGLE_O UTPUT_ON_CO MPARE	
ENET_MAC_MC GR_PULSE_OUT PUT_LOW_ON_ COMPARE	
ENET_MAC_MC GR_PULSE_OUT PUT_HIGH_ON_ COMPARE	

enum ENET_MAC_MiiModeType

MAC PHY interface mode type.

Enumerator:

ENET_MAC_PHY _MII	
ENET_MAC_PHY _RGMII	
ENET_MAC_PHY _RMII	

enum ENET_MAC_MmcCntPresetLevelType

MMC counter present level type.

Enumerator:

ENET_MAC_MM C_COUNTER_PR ESET_LEVEL_HA LF	
ENET_MAC_MM C_COUNTER_PR ESET_LEVEL_FU LL	

enum ENET_MAC_PauseThresholdType

ENET flow control pause threshold type.

Enumerator:

ENET_MAC_PAU SE_THRESHOLD _4SLOT_TIMES	
ENET_MAC_PAU SE_THRESHOLD _28SLOT_TIMES	
ENET_MAC_PAU SE_THRESHOLD _36SLOT_TIMES	
ENET_MAC_PAU SE_THRESHOLD _144SLOT_TIME S	
ENET_MAC_PAU SE_THRESHOLD _256SLOT_TIME S	

ENET_MAC_PAUSE_THRESHOLD_512SLOT_TIMES	
--	--

enum ENET_MAC_PhyClauseVersionType

MAC MDIO bus clause type.

Enumerator:

ENET_MAC_PHY_CLAUSE_22	
ENET_MAC_PHY_CLAUSE_45	

enum ENET_MAC_PhyOperationCmdType

MAC MDIO bus operation command type.

Enumerator:

ENET_MAC_PHY_OPERATION_WRITE	
ENET_MAC_PHY_OPERATION_POST_READ_INCREMENT_ADDR_FOR_CLAUSE_45	
ENET_MAC_PHY_OPERATION_READ	

enum ENET_MAC_PPSChannelType

PTP PPS channel type.

Enumerator:

ENET_MAC_PPS _CHANNEL_0	
ENET_MAC_PPS _CHANNEL_1	
ENET_MAC_PPS _CHANNEL_2	
ENET_MAC_PPS _CHANNEL_3	

enum ENET_MAC_PPSCmdType

PTP flexible PPS output command type.

Enumerator:

ENET_MAC_PPS _NO_COMMAND	
ENET_MAC_PPS _START_SINGLE _PULSE	
ENET_MAC_PPS _START_PULSE _TRAIN	
ENET_MAC_PPS _CANCEL_STAR T	
ENET_MAC_PPS _STOP_PULSE_ TRAIN_AT_TIME	
ENET_MAC_PPS _STOP_PULSE_	

TRAIN_IMMEDIATELY	
ENET_MAC_PPS_CANCEL_STOP_PULSE_TRAIN	

enum ENET_MAC_PPSFreqCtrlType

PTP fixed PPS output frequency control type.

Enumerator:

ENET_MAC_PPS_BINARY_1HZ_DIGITAL_NONE	Fixed PPS output 1HZ in binary rollover mode, no output in digital mode.
ENET_MAC_PPS_BINARY_2HZ_DIGITAL_1HZ	Fixed PPS output 2HZ in binary rollover mode, 1HZ in digital mode.
ENET_MAC_PPS_BINARY_4HZ_DIGITAL_2HZ	Fixed PPS output 4HZ in binary rollover mode, 2HZ in digital mode.
ENET_MAC_PPS_BINARY_8HZ_DIGITAL_4HZ	Fixed PPS output 8HZ in binary rollover mode, 4HZ in digital mode.
ENET_MAC_PPS_BINARY_16HZ_DIGITAL_8HZ	Fixed PPS output 16HZ in binary rollover mode, 8HZ in digital mode.
ENET_MAC_PPS_BINARY_32HZ_DIGITAL_16HZ	Fixed PPS output 32HZ in binary rollover mode, 16HZ in digital mode.
ENET_MAC_PPS_BINARY_64HZ_DIGITAL_32HZ	Fixed PPS output 64HZ in binary rollover mode, 32HZ in digital mode.

ENET_MAC_PPS _BINARY_128HZ _DIGITAL_64HZ	Fixed PPS output 128HZ in binary rollover mode, 64HZ in digital mode.
ENET_MAC_PPS _BINARY_256HZ _DIGITAL_128H Z	Fixed PPS output 256HZ in binary rollover mode, 128HZ in digital mode.
ENET_MAC_PPS _BINARY_512HZ _DIGITAL_256H Z	Fixed PPS output 512HZ in binary rollover mode, 256HZ in digital mode.
ENET_MAC_PPS _BINARY_1024H Z_DIGITAL_512 HZ	Fixed PPS output 1024HZ in binary rollover mode, 512HZ in digital mode.
ENET_MAC_PPS _BINARY_2048H Z_DIGITAL_1024 HZ	Fixed PPS output 2048HZ in binary rollover mode, 1024HZ in digital mode.
ENET_MAC_PPS _BINARY_4096H Z_DIGITAL_2048 HZ	Fixed PPS output 4096HZ in binary rollover mode, 2048HZ in digital mode.
ENET_MAC_PPS _BINARY_8192H Z_DIGITAL_4096 HZ	Fixed PPS output 8192HZ in binary rollover mode, 4096HZ in digital mode.
ENET_MAC_PPS _BINARY_16384 HZ_DIGITAL_81 92HZ	Fixed PPS output 16384HZ in binary rollover mode, 8192HZ in digital mode.
ENET_MAC_PPS _BINARY_32768 HZ_DIGITAL_16 384HZ	Fixed PPS output 32768HZ in binary rollover mode, 16384HZ in digital mode.

enum ENET_MAC_PPSModeType

PTP PPS channel operation mode type.

Enumerator:

ENET_MAC_PPS _MODE_PPS	
ENET_MAC_PPS _MODE_MCGR	

enum ENET_MAC_PPSPinDirType

PTP PPS channel direction type.

Enumerator:

ENET_PTP_PPS _DIR_INPUT	
ENET_PTP_PPS _DIR_OUTPUT	

enum ENET_MAC_PPSTargetTimeModeType

PPS target time mode type.

Enumerator:

ENET_MAC_PPS _TARGET_TIME_ ONLY_INT	
ENET_MAC_PPS _TARGET_TIME_ MCGR	
ENET_MAC_PPS _TARGET_TIME_ INT_ST	

ENET_MAC_PPS _TARGET_TIME_ ONLY_ST	
--	--

enum ENET_MAC_PTPClkSelType

MAC PTP clock select type.

Enumerator:

ENET_MAC_PTP _CLK_SEL_PCC	
ENET_MAC_PTP _CLK_SEL_TX_C LK	

enum ENET_MAC_PTPoEQueueCtrlType

Tagged PTP over Ethernet packets queuing control type.

Enumerator:

ENET_MAC_VLA N_PTPOE_ROU TED_AS_GENERI C	
ENET_MAC_VLA N_PTPOE_ROU TED_TO_PTP_Q UEUE	
ENET_MAC_VLA N_PTPOE_ROU TED_BY_PRIORI TY	

enum ENET_MAC_PTPPacketLayerType

PTP snapshot network layer select type.

Enumerator:

ENET_MAC_PTP _OVER_ETHERNET	
ENET_MAC_PTP _OVER_IPV4	
ENET_MAC_PTP _OVER_IPV6	

enum ENET_MAC_PTPSnapshotMsgType

PTP snapshot packet select type.

Enumerator:

ENET_MAC_PTP _SNAPSHOT_FO R_E2E	
ENET_MAC_PTP _SNAPSHOT_FO R_SYNC	
ENET_MAC_PTP _SNAPSHOT_FO R_DELAYREQ	
ENET_MAC_PTP _SNAPSHOT_FO R_ALL_PTP	
ENET_MAC_PTP _SNAPSHOT_FO	

R_SLAVE_NODE _EVENT	
ENET_MAC_PTP _SNAPSHOT_FO R_MASTER_NO DE_EVENT	
ENET_MAC_PTP _SNAPSHOT_FO R_E2E_EVENT	
ENET_MAC_PTP _SNAPSHOT_FO R_P2P_EVENT	

enum ENET_MAC_PTPVersionType

PTP IEEE1588 version type.

Enumerator:

ENET_MAC_PTP _IEEE_1588_V1	
ENET_MAC_PTP _IEEE_1588_V2	

enum ENET_MAC_RGMIIClkSelType

MAC RGMII clock select type.

Enumerator:

ENET_MAC_RG MII_CLK_SEL_DI SABLE	
ENET_MAC_RG MII_CLK_SEL_E NET_REF_CLK	

ENET_MAC_RG MII_CLK_SEL_P LL0DIVH	
ENET_MAC_RG MII_CLK_SEL_P LL1DIVH	

enum ENET_MAC_SlowProtocolDetectType

MAC slow protocol detect type definition.

Enumerator:

ENET_MAC_SLO W_PROTOCOL_ DETECT_STAND ARD	
ENET_MAC_SLO W_PROTOCOL_ DETECT_EXTEN DED	

enum ENET_MAC_SpeedType

MAC speed mode type definition.

Enumerator:

ENET_MAC_SPE ED_1000MBPS	
ENET_MAC_SPE ED_2500MBPS	
ENET_MAC_SPE ED_10MBPS	
ENET_MAC_SPE ED_100MBPS	

enum ENET_MAC_SrcAddrCtrlType

ENET MAC source address insertion control type definition.

Enumerator:

ENET_MAC_SRC _ADDR_CTRL_B Y_INPUT_SIGNA L	
ENET_MAC_SRC _ADDR_INSERT _BY_MAC_ADDR _IDX_0	
ENET_MAC_SRC _ADDR_REPLAC E_BY_MAC_ADD R_IDX_0	
ENET_MAC_SRC _ADDR_INSERT _BY_MAC_ADDR _IDX_1	
ENET_MAC_SRC _ADDR_REPLAC E_BY_MAC_ADD R_IDX_1	

enum ENET_MAC_StatusType

ENE MAC status type definition.

Enumerator:

ENET_MAC_STA TUS_IDLE	
--------------------------	--

ENET_MAC_STA TUS_BUSY	
--------------------------	--

enum ENET_MAC_SystemTimeInputType

PTP system time input source type.

Enumerator:

ENET_MAC_SYS TEM_TIME_INP UT_INTERNAL	
ENET_MAC_SYS TEM_TIME_INP UT_EXTERNAL	

enum ENET_MAC_SystemTimeOperationType

PTP system time offset update operation type.

Enumerator:

ENET_MAC_SYS TEM_TIME_ADD _TIME	
ENET_MAC_SYS TEM_TIME_SUB TRACT_TIME	

enum ENET_MAC_TimestampRolloverType

PTP counter rollover mode type.

Enumerator:

ENET_MAC_TIM ESTAMP_ROLLO VER_BINARY	
ENET_MAC_TIM ESTAMP_ROLLO VER_DIGITAL	

enum ENET_MAC_TimestampUpdateMethodType

PTP timestamp update mode type.

Enumerator:

ENET_MAC_TIM ESTAMP_UPDAT E_COARSE	
ENET_MAC_TIM ESTAMP_UPDAT E_FINE	

enum ENET_MAC_TxClkSelType

MAC TX clock select type.

Enumerator:

ENET_MAC_TX_ CLK_SEL_DISAB LE	
ENET_MAC_TX_ CLK_SEL_PLL0D IVL	
ENET_MAC_TX_ CLK_SEL_FOSC	

ENET_MAC_TX_ CLK_SEL_ENET _TX_CLK	
---	--

enum ENET_MAC_TxPacketControllerStatusType

MAC TX packet controller status type.

Enumerator:

ENET_MAC_TX_ PACKET_CONTR OLLER_IDLE	
ENET_MAC_TX_ PACKET_CONTR OLLER_WAIT	
ENET_MAC_TX_ PACKET_CONTR OLLER_TRANSF ER_PAUSE_PAC KET	
ENET_MAC_TX_ PACKET_CONTR OLLER_TRANSF ER_INPUT_PAC KET	

enum ENET_MAC_TxTimestampStatusModeType

PTP TX status mode type.

Enumerator:

ENET_MAC_TX_ TIMESTAMP_ST	
------------------------------	--

ATUS_MODE_P RESERVE	
ENET_MAC_TX_ TIMESTAMP_ST ATUS_MODE_O VERWRITE	

enum ENET_MAC_VLANCommandType

MAC VLAN indirect registers access command type.

Enumerator:

ENET_MAC_VLA N_COMMAND_ WRITE	
ENET_MAC_VLA N_COMMAND_R EAD	

enum ENET_MAC_VLANCompareType

MAC VLAN compare mode type.

Enumerator:

ENET_MAC_VLA N_16BITS_COM PARE	
ENET_MAC_VLA N_12BITS_COM PARE	

enum ENET_MAC_VLANFilterGroupType

MAC VLAN perfect filter group index type.

Enumerator:

ENET_MAC_VLA N_FILTER_GRO UP_0	
ENET_MAC_VLA N_FILTER_GRO UP_1	
ENET_MAC_VLA N_FILTER_GRO UP_2	
ENET_MAC_VLA N_FILTER_GRO UP_3	

enum ENET_MAC_VLANInclCommandType

MAC VLAN incl indirect registers access command type.

Enumerator:

ENET_MAC_VLA N_INCL_COMMA ND_READ	
ENET_MAC_VLA N_INCL_COMMA ND_WRITE	

enum ENET_MAC_VLANTagCtrlType

MAC VLAN tag control type.

Enumerator:

ENET_MAC_VLAN_TAG_NO_OPERATION	
ENET_MAC_VLAN_TAG_DELETION	
ENET_MAC_VLAN_TAG_INSERTION	
ENET_MAC_VLAN_TAG_REPLACEMENT	

enum ENET_MAC_VLANTagStripModeType

MAC VLAN tag strip type.

Enumerator:

ENET_MAC_VLAN_TAG_STRIP_NEVER	
ENET_MAC_VLAN_TAG_STRIP_FILTER_PASS	
ENET_MAC_VLAN_TAG_STRIP_FILTER_FAIL	
ENET_MAC_VLAN_TAG_STRIP_ALWAYS	

enum ENET_MAC_VLANType

MAC VLAN tag type S-VLAN IEEE802.1ad (Q-in-Q) C-VLAN IEEE802.1Q.

Enumerator:

ENET_MAC_VLA N_TYPE_C_VLA N	
ENET_MAC_VLA N_TYPE_S_VLA N	

enum ENET_MAC_WDGTimerSizeType

MAC watchdog timer type definition.

Enumerator:

ENET_MAC_WD G_TIMER_SIZE_ 2KB	
ENET_MAC_WD G_TIMER_SIZE_ 3KB	
ENET_MAC_WD G_TIMER_SIZE_ 4KB	
ENET_MAC_WD G_TIMER_SIZE_ 5KB	
ENET_MAC_WD G_TIMER_SIZE_ 6KB	
ENET_MAC_WD G_TIMER_SIZE_ 7KB	
ENET_MAC_WD G_TIMER_SIZE_ 8KB	

ENET_MAC_WD G_TIMER_SIZE_ 9KB	
ENET_MAC_WD G_TIMER_SIZE_ 10KB	
ENET_MAC_WD G_TIMER_SIZE_ 11KB	
ENET_MAC_WD G_TIMER_SIZE_ 12KB	
ENET_MAC_WD G_TIMER_SIZE_ 13KB	
ENET_MAC_WD G_TIMER_SIZE_ 14KB	
ENET_MAC_WD G_TIMER_SIZE_ 15KB	

enum Enet_MMCRxCounterType

MAC receive statistics counter type.

Enumerator:

ENET_CTR_RX_ PACKETS_COUN T_GOOD_BAD	
ENET_CTR_RX_ OCTET_COUNT_ GOOD_BAD	
ENET_CTR_RX_ OCTET_COUNT_ GOOD	

ENET_CTR_RX_ BROADCAST_PA CKETS_GOOD	
ENET_CTR_RX_ MULTICAST_PA CKETS_GOOD	
ENET_CTR_RX_ CRC_ERROR_PA CKETS	
ENET_CTR_RX_ ALIGNMENT_ER ROR_PACKETS	
ENET_CTR_RX_ RUNT_ERROR_P ACKETS	
ENET_CTR_RX_ JABBER_ERROR _PACKETS	
ENET_CTR_RX_ UNDERSIZE_PA CKETS_GOOD	
ENET_CTR_RX_ OVERSIZE_PACK ETS_GOOD	
ENET_CTR_RX_ 64OCTETS_PAC KETS_GOOD_BA D	
ENET_CTR_RX_ 65TO127OCTET S_PACKETS_GO OD_BAD	
ENET_CTR_RX_ 128TO255OCTE TS_PACKETS_G OOD_BAD	
ENET_CTR_RX_ 256TO511OCTE	

TS_PACKETS_GOOD_BAD	
ENET_CTR_RX_512TO1023OCTETS_PACKETS_GOOD_BAD	
ENET_CTR_RX_1024TOMAXOCTETS_PACKETS_GOOD_BAD	
ENET_CTR_RX_UNICAST_PACKETS_ETS_GOOD	
ENET_CTR_RX_LENGTH_ERROR_PACKETS	
ENET_CTR_RX_OUT_OF_RANGE_TYPE_PACKETS	
ENET_CTR_RX_PAUSE_PACKETS	
ENET_CTR_RX_FIFO_OVERFLOW_PACKETS	
ENET_CTR_RX_VLAN_PACKETS_GOOD_BAD	
ENET_CTR_RX_WATCHDOG_ERROR_PACKETS	
ENET_CTR_RX_RECEIVE_ERROR_PACKETS	
ENET_CTR_RX_CONTROL_PACKETS_ETS_GOOD	

enum Enet_MMCTxCounterType

MAC MMC transmit statistics counter type.

Enumerator:

ENET_CTR_TX_ OCTET_COUNT_ GOOD_BAD	
ENET_CTR_TX_ PACKET_COUNT _GOOD_BAD	
ENET_CTR_TX_ BROADCAST_PA CKETS_GOOD	
ENET_CTR_TX_ MULTICAST_PA CKETS_GOOD	
ENET_CTR_TX_ 64OCTETS_PAC KETS_GOOD_BA D	
ENET_CTR_TX_ 65TO127OCTET S_PACKETS_GO OD_BAD	
ENET_CTR_TX_ 128TO255OCTE TS_PACKETS_G OOD_BAD	
ENET_CTR_TX_ 256TO511OCTE TS_PACKETS_G OOD_BAD	
ENET_CTR_TX_ 512TO1023OCT	

ETS_PACKETS_ GOOD_BAD	
ENET_CTR_TX_ 1024TOMAXOCT ETS_PACKETS_ GOOD_BAD	
ENET_CTR_TX_ UNICAST_PACK ETS_GOOD_BAD	
ENET_CTR_TX_ MULTICAST_PA CKETS_GOOD_B AD	
ENET_CTR_TX_ BROADCAST_PA CKETS_GOOD_B AD	
ENET_CTR_TX_ UNDERFLOW_E RROR_PACKETS	
ENET_CTR_TX_ SINGLE_COLLISI ON_GOOD_PAC KETS	
ENET_CTR_TX_ MULTIPLE_COL LISION_GOOD_ PACKETS	
ENET_CTR_TX_ DEFERRED_PAC KETS	
ENET_CTR_TX_ LATE_COLLISIO N_PACKETS	
ENET_CTR_TX_ EXCESSIVE_COL LISION_PACKET S	

ENET_CTR_TX_ CARRIER_ERRO R_PACKETS	
ENET_CTR_TX_ OCTET_COUNT_ GOOD	
ENET_CTR_TX_ PACKET_COUNT _GOOD	
ENET_CTR_TX_ EXCESSIVE_DEF ERRAL_ERROR	
ENET_CTR_TX_ PAUSE_PACKET S	
ENET_CTR_TX_ VLAN_PACKETS _GOOD	
ENET_CTR_TX_ OSIZE_PACKETS _GOOD	

enum ENET_MTL_ByteEnableType

MTL byte enable type.

Enumerator:

ENET_MTL_BYT E_0_VALID	
ENET_MTL_BYT E_0_1_VALID	
ENET_MTL_BYT E_0_1_2_VALID	
ENET_MTL_BYT E_ALL_VALID	

enum ENET_MTL_DebugAccessFIFOType

MTL debug access type.

Enumerator:

ENET_MTL_DEB UG_ACCESS_TX _FIFO	
ENET_MTL_DEB UG_ACCESS_TX _STATUS_FIFO	
ENET_MTL_DEB UG_ACCESS_TS O_FIFO	
ENET_MTL_DEB UG_ACCESS_RX _FIFO	

enum ENET_MTL_FIFODebugAccessType

MTL fifo debug access type.

Enumerator:

ENET_MTL_FIFO _DEBUG_ACCES S_RESTRICTED	
ENET_MTL_FIFO _DEBUG_ACCES S_UNRESTRICT ED	

enum ENET_MTL_FIFOPacketStatusType

MTL fifo packet status type.

Enumerator:

ENET_MTL_PAC KET_STATE_PA CKET_DATA	
ENET_MTL_PAC KET_STATE_TX_ CTRL_WORD_RX _NORMAL_STAT US	
ENET_MTL_PAC KET_STATE_TX_ SOP_DATA_RX_ LAST_STATUS	
ENET_MTL_PAC KET_STATE_TX_ EOP_DATA_RX_ EOP	

enum ENET_MTL_GateControlListType

MTL gate control list index type.

Enumerator:

ENET_MTL_GCL _NUMBER_0	
ENET_MTL_GCL _NUMBER_1	

enum ENET_MTL_GCLIntervalMultipleType

Multiple of gate control list interval time.

Enumerator:

ENET_MTL_GCL _INTERVAL_X_1	
ENET_MTL_GCL _INTERVAL_X_2	
ENET_MTL_GCL _INTERVAL_X_4	
ENET_MTL_GCL _INTERVAL_X_8	
ENET_MTL_GCL _INTERVAL_X_1 6	
ENET_MTL_GCL _INTERVAL_X_3 2	
ENET_MTL_GCL _INTERVAL_X_6 4	
ENET_MTL_GCL _INTERVAL_X_1 28	

enum ENET_MTL_GclOperationType

Gcl operation type.

Enumerator:

ENET_MTL_GCL _WRITE	
ENET_MTL_GCL _READ	

enum ENET_MTL_GclRegistersType

Gcl related registers.

Enumerator:

ENET_MTL_GCL _BTR_LOW	
ENET_MTL_GCL _BTR_HIGH	
ENET_MTL_GCL _CTR_LOW	
ENET_MTL_GCL _CTR_HIGH	
ENET_MTL_GCL _TER	
ENET_MTL_GCL _LLR	

enum ENET_MTL_QueueEnableType

MTL queue enable type.

Enumerator:

ENET_MTL_QUE UE_DISABLE	
ENET_MTL_QUE UE_ENABLE_AV _MODE	
ENET_MTL_QUE UE_ENABLE_GE NERIC	

enum ENET_MTL_QueueType

ENET MTL queue type.

Enumerator:

ENET_MTL_QUE UE_0	
ENET_MTL_QUE UE_1	

enum ENET_MTL_ReadControllerStatusType

MTL read controller status type.

Enumerator:

ENET_MTL_READ_CONTROLLER _IDLE	
ENET_MTL_READ_CONTROLLER _READ	
ENET_MTL_READ_CONTROLLER _WAIT	
ENET_MTL_READ_CONTROLLER _FLUSH	

enum ENET_MTL_RxArbitrationAlgorithmType

ENET RX arbitration type.

Enumerator:

ENET_MTL_RX_ ARBITRATION_ TRICT_POLICY	
ENET_MTL_RX_ ARBITRATION_ WEIGHTED_STR ICT_POLICY	

enum ENET_MTL_RxFlowCtrlThresholdType

MTL RX flow control threshold type.

Enumerator:

ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 1_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 1_5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 2_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 2_5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 3_KB	
ENET_MTL_RX_ FLOWCTRL_THR	

ESHOLD_FULL_ 3_5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 4_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 4_5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 5_5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 6_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 6_5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 7_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 7_5_KB	
ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 8_KB	

ENET_MTL_RX_ FLOWCTRL_THR ESHOLD_FULL_ 8_5_KB	
--	--

enum ENET_MTL_RxQueueStatusType

MTL RX queue status type.

Enumerator:

ENET_MTL_RX_ QUEUE_EMPTY	
ENET_MTL_RX_ QUEUE_BELOW _FLOW_CTRL_D EACTIVATE_THR ES	
ENET_MTL_RX_ QUEUE_ABOVE_ FLOW_CTRL_AC TIVATE_THRES	
ENET_MTL_RX_ QUEUE_FULL	

enum ENET_MTL_RxThresholdType

MTL RX threshold type.

Enumerator:

ENET_MTL_RX_ THRESHOLD_64 B	
-----------------------------------	--

ENET_MTL_RX_ THRESHOLD_32 B	
ENET_MTL_RX_ THRESHOLD_96 B	
ENET_MTL_RX_ THRESHOLD_12 8B	
ENET_MTL_RX_ STORE_FORWA RD	

enum ENET_MTL_ScheduleErrorLoopCountType

Loop count to report Scheduling Error.

Enumerator:

ENET_MTL_GCL _4_ITERATIONS	
ENET_MTL_GCL _8_ITERATIONS	
ENET_MTL_GCL _16_ITERATION S	
ENET_MTL_GCL _32_ITERATION S	

enum ENET_MTL_TxScheduleAlgorithmType

ENET TX schedule algorithm type.

Enumerator:

ENET_MTL_SCH ED_ALGO_WRR	Weighted round robin algorithm.
ENET_MTL_SCH ED_ALGO_SP	Strict priority algorithm.

enum ENET_MTL_TxThresholdType

MTL TX threshold type.

Enumerator:

ENET_MTL_TX_ THRESHOLD_32 B	
ENET_MTL_TX_ THRESHOLD_64 B	
ENET_MTL_TX_ THRESHOLD_96 B	
ENET_MTL_TX_ THRESHOLD_12 8B	
ENET_MTL_TX_ THRESHOLD_19 2B	
ENET_MTL_TX_ THRESHOLD_25 6B	
ENET_MTL_TX_ STORE_FORWA RD	

enum ENET_MTL_WriteControllerStatusType

MTL write controller status type.

Enumerator:

ENET_MTL_WRI TE_CONTROLLE R_IDLE	
ENET_MTL_WRI TE_CONTROLLE R_ACTIVE	

Function Documentation

```
LOCAL_INLINE void ENET_HWA_DMA_ClearAbnormalInterruptFlag (ENET_Type *const  pEnet,  
ENET_DMA_ChannelType  eDMAChannel)
```

Clear DMA abnormal interrupt summary.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

```
LOCAL_INLINE void ENET_HWA_DMA_ClearAllAbnormalInterruptFlags (ENET_Type *const  pEnet,  
ENET_DMA_ChannelType  eDMAChannel)
```

Clear all DMA abnormal interrupts.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

```
LOCAL_INLINE void ENET_HWA_DMA_ClearAllNormalInterruptFlags (ENET_Type *const  pEnet,  
ENET_DMA_ChannelType  eDMAChannel)
```

Clear all DMA normal interrupts.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

**LOCAL_INLINE void ENET_HWA_DMA_ClearContextDescriptorErrorFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Clear DMA context descriptor error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

**LOCAL_INLINE void ENET_HWA_DMA_ClearEarlyRxInterruptFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Clear DMA early receive interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

**LOCAL_INLINE void ENET_HWA_DMA_ClearEarlyTxInterruptFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Clear DMA early transmit interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

```
LOCAL_INLINE void ENET_HWA_DMA_ClearFatalBusErrorFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Clear DMA fatal bus error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

```
LOCAL_INLINE void ENET_HWA_DMA_ClearNormalInterruptFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Clear DMA normal interrupt summary.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

```
LOCAL_INLINE void ENET_HWA_DMA_ClearRxBufferUnavailableFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Clear DMA receive buffer unavailable status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

```
LOCAL_INLINE void ENET_HWA_DMA_ClearRxError (ENET_Type *const pEnet, ENET_DMA_ChannelType  
eDMAChannel)
```

Clear DMA Rx error status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>eDMAChannel</i>	DMA channel
----	--------------------	-------------

**LOCAL_INLINE void ENET_HWA_DMA_ClearRxInterruptFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Clear DMA receive interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

**LOCAL_INLINE void ENET_HWA_DMA_ClearRxProcessStoppedFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Clear DMA receive process stopped status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

**LOCAL_INLINE void ENET_HWA_DMA_ClearRxWDGTimeoutFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Clear DMA receive watchdog timeout status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

**LOCAL_INLINE void ENET_HWA_DMA_ClearTxBufferUnavailableFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Clear DMA transmit buffer unavailable status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_DMA_ClearTxError (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)

Clear DMA Tx error status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_DMA_ClearTxInterruptFlag (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)

Clear DMA transmit interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_DMA_ClearTxProcessStoppedFlag (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)

Clear DMA transmit process stopped status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_DMA_DisableInterrupt (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*, ENET_DMA_InterruptFlagType *eIntFlag*)

Disable one of the DMA channel interrupts.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>eIntFlag</i>	interrupt to be disabled

**LOCAL_INLINE void ENET_HWA_DMA_EnableInterrupt (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, ENET_DMA_InterruptFlagType *eIntFlag*)**

Enable one of the DMA channel interrupts.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>eIntFlag</i>	interrupt to be enabled

**LOCAL_INLINE bool ENET_HWA_DMA_Get8xPBLModeEnFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA burst length multiply by 8 mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true DMA Tx and Rx burst length is multiplied by 8

false use DMA Tx and Rx burst length

**LOCAL_INLINE bool ENET_HWA_DMA_GetAbnormalInterruptFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA abnormal interrupt summary.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true abnormal interrupt detected

false abnormal interrupt not detected

LOCAL_INLINE bool ENET_HWA_DMA_GetAddressAlignedBeatsEnFlag (const ENET_Type *const *pEnet*)

Get DMA Address-Aligned Beats enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Address-Aligned Beats is enabled

false Address-Aligned Beats is disabled

**LOCAL_INLINE bool ENET_HWA_DMA_GetAdvanceSlotCheckEnFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get advance slot check enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true advance slot check is enabled

false advance slot check is disabled

LOCAL_INLINE bool ENET_HWA_DMA_GetAHBMasterBusyStatus (const ENET_Type *const *pEnet*)

Get AXI Master Read Channel Status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true AXI Master Read Channel detected

false AXI Master Read Channel not detected

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetChannelCtrl (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t register value

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetChannelRxCtrl (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive channel control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t register value

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetChannelTxCtrl (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA transmit channel control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t register value

**LOCAL_INLINE bool ENET_HWA_DMA_GetContextDescriptorErrorFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA context descriptor error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true context descriptor error detected

false context descriptor error not detected

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetCurrentRxBufAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA current Rx buffer address pointer.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t current Rx buffer

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetCurrentRxDescAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA current Rx descriptor address pointer.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t current Rx descriptor

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetCurrentStatus (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA current status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t DMA status

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetCurrentTxBufAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA current Tx buffer address pointer.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t current Tx buffer

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetCurrentTxDescAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA current Tx descriptor address pointer.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t current Tx descriptor

**LOCAL_INLINE uint8_t ENET_HWA_DMA_GetDescriptorSkipLength (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA descriptor skip length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint8_t descriptor skip length

LOCAL_INLINE bool ENET_HWA_DMA_GetDMAChannel0IntStatus (const ENET_Type *const *pEnet*)

Get DMA channel0 interrupt event in DMA interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true DMA channel0 interrupt event detected

false DMA channel0 interrupt event not detected

LOCAL_INLINE bool ENET_HWA_DMA_GetDMAChannel1IntStatus (const ENET_Type *const *pEnet*)

Get DMA channel1 interrupt event in DMA interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true DMA channel1 interrupt event detected

false DMA channel1 interrupt event not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetDMAChannelIntStatus (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eChannel*)**

Get DMA channel interrupt event in DMA interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eChannel</i>	DMA channel

Returns

true eChannel interrupt event detected

false eChannel interrupt event not detected

LOCAL_INLINE uint32_t ENET_HWA_DMA_GetDMAIntStatus (const ENET_Type *const *pEnet*)

Get DMA interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t DMA interrupt status value

**LOCAL_INLINE bool ENET_HWA_DMA_GetEarlyRxIntEnFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA early receive interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

- true early receive interrupt is enabled
- false early receive interrupt is disabled

**LOCAL_INLINE uint16_t ENET_HWA_DMA_GetEarlyRxInterruptCount (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get the count of the number of times Early Receive Interrupt was asserted.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eDMAChannel</i>	the DMA channel of the Ethernet instance

Returns

The count value

**LOCAL_INLINE bool ENET_HWA_DMA_GetEarlyRxInterruptFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA early receive interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

- true early receive interrupt detected
- false early receive interrupt not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetEarlyTxIntEnFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA early transmit interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>eDMAChannel</i>	DMA channel
----	--------------------	-------------

Returns

true early transmit interrupt is enabled

false early transmit interrupt is disabled

**LOCAL_INLINE bool ENET_HWA_DMA_GetEarlyTxInterruptFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA early transmit interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true early transmit interrupt detected

false early transmit interrupt not detected

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetEnabledInterrupts (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel enabled interrupts.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t enabled interrupts

**LOCAL_INLINE bool ENET_HWA_DMA_GetEnhancedDescriptorEnFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA enhanced descriptor enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true enhanced descriptor is enabled

false enhanced descriptor is disabled

**LOCAL_INLINE bool ENET_HWA_DMA_GetFatalBusErrorFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA fatal bus error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true fatal bus error detected

false fatal bus error not detected

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetFetchTimeOffset (const ENET_Type *const *pEnet*, uint8_t
eCtrlIndex)**

Get DMA TBS fetch time offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eCtrlIndex</i>	DMA TBS control index

Returns

uint32_t DMA TBS fetch time offset

**LOCAL_INLINE bool ENET_HWA_DMA_GetFetchTimeOffsetValid (const ENET_Type *const *pEnet*, uint8_t
eCtrlIndex)**

Get DMA TBS fetch time offset valid.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eCtrlIndex</i>	DMA TBS control index

Returns

true TBS fetch time offset is valid

false TBS fetch time offset is invalid

LOCAL_INLINE bool ENET_HWA_DMA_GetFixedBurstLengthEnFlag (const ENET_Type *const *pEnet*)

Get DMA fixed burst length enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true fixed burst length is enabled

false fixed burst length is disabled

LOCAL_INLINE ENET_DMA_IntModeType ENET_HWA_DMA_GetInterruptMode (const ENET_Type *const *pEnet*)

Get DMA interrupt mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_DMA_IntModeType

LOCAL_INLINE bool ENET_HWA_DMA_GetMACIntStatus (const ENET_Type *const *pEnet*)

Get MAC interrupt event in DMA interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MAC interrupt event detected

false MAC interrupt event not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetMissFrameCount (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint16_t * *pMissPacketCnt*)**

Get the miss frame count and overflow status of the Ethernet DMA channel.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eDMAChannel</i>	the DMA channel of the Ethernet instance
out	<i>pMissPacketCnt</i>	the miss frame count of the channel

Returns

true The miss frame count is overflow

false the miss frame count is not overflow

LOCAL_INLINE bool ENET_HWA_DMA_GetMixedBurstEnFlag (const ENET_Type *const *pEnet*)

Get DMA mixed burst length enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true mixed burst is enabled

false mixed burst is disabled

LOCAL_INLINE uint32_t ENET_HWA_DMA_GetMode (const ENET_Type *const *pEnet*)

Get DMA bus mode register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t DMA bus mode register value

LOCAL_INLINE bool ENET_HWA_DMA_GetMTLIntStatus (const ENET_Type *const *pEnet*)

Get MTL interrupt event in DMA interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MTL interrupt event detected

false MTL interrupt event not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetNormalInterruptFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA normal interrupt summary.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true normal interrupt detected

false normal interrupt not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetOperateOnSecondPacketEnFlag (const ENET_Type *const
pEnet, ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA operate on second packet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true operate on second packet is enabled

false operate on second packet is disabled

LOCAL_INLINE ENET_DMA_PriorityModeType ENET_HWA_DMA_GetPriorityMode (const ENET_Type *const *pEnet*)

Get DMA priority mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_DMA_PRIORITY_MODE_RX_TX the priority ratio is RX:TX

ENET_DMA_PRIORITY_MODE_TX_RX the priority ratio is TX:RX

LOCAL_INLINE ENET_DMA_PriorityRatioType ENET_HWA_DMA_GetPriorityRatio (const ENET_Type *const *pEnet*)

Get DMA Rx:Tx or Tx:Rx priority ratio.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_DMA_PriorityRatioType

LOCAL_INLINE bool ENET_HWA_DMA_GetRebuildINCRxBurstEnFlag (const ENET_Type *const *pEnet*)

Get DMA rebuild INCRx burst enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true rebuild INCRx burst is enabled

false rebuild INCRx burst is disabled

**LOCAL_INLINE uint8_t ENET_HWA_DMA_GetRefSlotNumber (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get reference slot number in the DMA.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint8_t reference slot number

**LOCAL_INLINE ENET_DMA_StatusType ENET_HWA_DMA_GetRxActiveStatus (const ENET_Type *const
pEnet, ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive channel active status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

ENET_DMA_STATUS_IDLE DMA reception is stopped

ENET_DMA_STATUS_ACTIVE DMA reception is running

**LOCAL_INLINE uint16_t ENET_HWA_DMA_GetRxBufferSize (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive buffer size.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint16_t receive buffer size

**LOCAL_INLINE bool ENET_HWA_DMA_GetRxBufferUnavailableFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive buffer unavailable status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true receive buffer unavailable status detected

false receive buffer unavailable status not detected

**LOCAL_INLINE uint8_t ENET_HWA_DMA_GetRxBurstLength (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive programmable burst length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint8_t receive burst length

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetRxDescListHeadAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel Rx descriptor list header address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t Rx descriptor list header address

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetRxDescListTailAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel Rx descriptor list tail address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t Rx descriptor list tail address

**LOCAL_INLINE uint16_t ENET_HWA_DMA_GetRxDescRingLength (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel Rx descriptor ring length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t Rx descriptor ring length

**LOCAL_INLINE ENET_DMA_RxErrorType ENET_HWA_DMA_GetRxError (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA Rx error status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

ENET_DMA_RxErrorType

**LOCAL_INLINE bool ENET_HWA_DMA_GetRxInterruptFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true receive interrupt detected

false receive interrupt not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetRxPacketFlushEnFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive packet flush enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true Rx packet flush is enabled

false Rx packet flush is disabled

**LOCAL_INLINE bool ENET_HWA_DMA_GetRxProcessStoppedFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive process stopped status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true receive process stopped status detected

false receive process stopped status not detected

**LOCAL_INLINE uint8_t ENET_HWA_DMA_GetRxWDGCount (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get receive interrupt watchdog timer count.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint8_t watchdog timer count value

**LOCAL_INLINE ENET_DMA_WDGCounterUnitType ENET_HWA_DMA_GetRxWDGCounterUnit (const
ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)**

Get receive interrupt watchdog timer count unit.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

ENET_DMA_WDGCounterUnitType

**LOCAL_INLINE bool ENET_HWA_DMA_GetRxWDGTimeoutFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA receive watchdog timeout status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true receive watchdog timeout status detected

false receive watchdog timeout status not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetSlotComparisonEnFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get slot comparison enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true slot comparison is enabled

false slot comparison is disabled

**LOCAL_INLINE uint16_t ENET_HWA_DMA_GetSlotInterval (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get slot interval value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint16_t slot interval

**LOCAL_INLINE ENET_DMA_StatusType ENET_HWA_DMA_GetSwResetStatus (const ENET_Type *const
pEnet)**

Get DMA software reset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_DMA_StatusType

LOCAL_INLINE ENET_DMA_StatusType ENET_HWA_DMA_GetTxActiveStatus (const ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)

Get DMA transmit channel active status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

ENET_DMA_STATUS_IDLE DMA transmission is stopped

ENET_DMA_STATUS_ACTIVE DMA transmission is running

LOCAL_INLINE ENET_DMA_TxArbitrationAlgorithmType ENET_HWA_DMA_GetTxArbitrationAlgorithm (const ENET_Type *const *pEnet*)

Get DMA arbitration algorithm between transmit channels.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_DMA_TxArbitrationAlgorithmType

LOCAL_INLINE bool ENET_HWA_DMA_GetTxBufferUnavailableFlag (const ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)

Get DMA transmit buffer unavailable status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true transmit buffer unavailable status detected

false transmit buffer unavailable status not detected

**LOCAL_INLINE uint8_t ENET_HWA_DMA_GetTxBurstLength (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA transmit programmable burst length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint8_t transmit burst length

**LOCAL_INLINE uint8_t ENET_HWA_DMA_GetTxChannelWeight (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA transmit channel weight.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint8_t transmit channel weight

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetTxDescListHeadAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel Tx descriptor list header address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t Tx descriptor list header address

**LOCAL_INLINE uint32_t ENET_HWA_DMA_GetTxDescListTailAddr (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel Tx descriptor list tail address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint32_t Tx descriptor list tail address

**LOCAL_INLINE uint16_t ENET_HWA_DMA_GetTxDescRingLength (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA channel Tx descriptor ring length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

uint16_t Tx descriptor ring length

**LOCAL_INLINE ENET_DMA_TxErrorType ENET_HWA_DMA_GetTxError (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA Tx error status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

ENET_DMA_TxErrorType

**LOCAL_INLINE bool ENET_HWA_DMA_GetTxInterruptFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA transmit interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true transmit interrupt detected

false transmit interrupt not detected

**LOCAL_INLINE bool ENET_HWA_DMA_GetTxProcessStoppedFlag (const ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*)**

Get DMA transmit process stopped status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

Returns

true transmit process stopped status detected

false transmit process stopped status not detected

**LOCAL_INLINE ENET_DMA_ArbitrationSchemeType ENET_HWA_DMA_GetTxRxArbitrationScheme (const
ENET_Type *const *pEnet*)**

Get DMA arbitration scheme between the transmit and receive paths.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_DMA_ArbitrationSchemeType

**LOCAL_INLINE void ENET_HWA_DMA_Set8xPBLModeEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set DMA burst length multiply by 8 mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	enable x8 mode

**LOCAL_INLINE void ENET_HWA_DMA_SetAddressAlignedBeatsEnFlag (ENET_Type *const *pEnet*, bool
bEnable)**

Set DMA Address-Aligned Beats enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_DMA_SetAdvanceSlotCheckEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set advance slot check enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	

```
LOCAL_INLINE void ENET_HWA_DMA_SetChannelCtrl (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32ChannelCtrl)
```

Set DMA channel control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32ChannelCtrl</i>	control value

```
LOCAL_INLINE void ENET_HWA_DMA_SetChannelRxCtrl (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32RxCtrl)
```

Set DMA receive channel control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32RxCtrl</i>	control value

```
LOCAL_INLINE void ENET_HWA_DMA_SetChannelTxCtrl (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32TxCtrl)
```

Set DMA transmit channel control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32TxCtrl</i>	control value

```
LOCAL_INLINE void ENET_HWA_DMA_SetCurrentStatus (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32Status)
```

Set DMA current status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32Status</i>	DMA status

**LOCAL_INLINE void ENET_HWA_DMA_SetDescriptorSkipLength (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint8_t *u8Length*)**

Set DMA descriptor skip length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u8Length</i>	descriptor skip length

**LOCAL_INLINE void ENET_HWA_DMA_SetEarlyRxIntEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set DMA early receive interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_DMA_SetEarlyTxIntEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set DMA early transmit interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_DMA_SetEnabledInterrupts (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint32_t *u32IntEnable*)**

Set DMA channel interrupts enable state.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32IntEnable</i>	interrupts to be enabled

**LOCAL_INLINE void ENET_HWA_DMA_SetEnhancedDescriptorEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set DMA enhanced descriptor enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_DMA_SetFetchTimeOffset (ENET_Type *const *pEnet*, uint8_t
eCtrlIndex, uint32_t *u32FetchTimeOffset*)**

Set DMA TBS fetch time offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eCtrlIndex</i>	DMA TBS control index
in	<i>u32FetchTimeOffset</i> <i>set</i>	DMA TBS fetch time offset

**LOCAL_INLINE void ENET_HWA_DMA_SetFetchTimeOffsetValid (ENET_Type *const *pEnet*, uint8_t
eCtrlIndex, bool *bEnable*)**

Set DMA TBS fetch time offset valid.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eCtrlIndex</i>	DMA TBS control index
in	<i>bEnable</i>	set valid

LOCAL_INLINE void ENET_HWA_DMA_SetFixedBurstLengthEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set DMA fixed burst length enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	

LOCAL_INLINE void ENET_HWA_DMA_SetInterruptMode (ENET_Type *const *pEnet*, ENET_DMA_IntModeType *eIntMode*)

Set DMA interrupt mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eIntMode</i>	DMA interrupt mode

LOCAL_INLINE void ENET_HWA_DMA_SetMixedBurstEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set DMA mixed burst length enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	

LOCAL_INLINE void ENET_HWA_DMA_SetMode (ENET_Type *const *pEnet*, uint32_t *u32Mode*)

Set DMA bus mode register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Mode</i>	DMA bus mode value

**LOCAL_INLINE void ENET_HWA_DMA_SetOperateOnSecondPacketEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set DMA operate on second packet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_DMA_SetPriorityMode (ENET_Type *const *pEnet*,
ENET_DMA_PriorityModeType *ePriorityMode*)**

Set DMA priority mode,Rx:Tx or Tx:Rx.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePriorityMode</i>	priority mode

**LOCAL_INLINE void ENET_HWA_DMA_SetPriorityRatio (ENET_Type *const *pEnet*,
ENET_DMA_PriorityRatioType *ePriorityRatio*)**

Set DMA Rx:Tx or Tx:Rx priority ratio.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePriorityRatio</i>	priority ratio

LOCAL_INLINE void ENET_HWA_DMA_SetRebuildINCRxBurstEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set DMA rebuild INCRx burst enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	

LOCAL_INLINE void ENET_HWA_DMA_SetRefSlotNumber (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*, uint8_t *u8SlotNumber*)

Set reference slot number in the DMA.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u8SlotNumber</i>	reference slot number

LOCAL_INLINE void ENET_HWA_DMA_SetRxBufferSize (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*, uint16_t *u16BufferSize*)

Set DMA receive buffer size.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u16BufferSize</i>	receive buffer size

LOCAL_INLINE void ENET_HWA_DMA_SetRxBurstLength (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*, uint8_t *u8Length*)

Set DMA receive programmable burst length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u8Length</i>	receive burst length

**LOCAL_INLINE void ENET_HWA_DMA_SetRxDescListHeadAddr (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint32_t *u32RxAddr*)**

Set DMA channel Rx descriptor list header address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32RxAddr</i>	Rx descriptor list header address

**LOCAL_INLINE void ENET_HWA_DMA_SetRxDescListTailAddr (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint32_t *u32RxAddr*)**

Set DMA channel Rx descriptor list tail address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32RxAddr</i>	Rx descriptor list tail address

**LOCAL_INLINE void ENET_HWA_DMA_SetRxDescRingLength (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint16_t *u16Length*)**

Set DMA channel Rx descriptor ring length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u16Length</i>	Rx descriptor ring length

**LOCAL_INLINE void ENET_HWA_DMA_SetRxPacketFlushEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set DMA receive packet flush enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_DMA_SetRxWDGCount (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint8_t *u8WDGCnt*)**

Set receive interrupt watchdog timer count.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u8WDGCnt</i>	watchdog timer count

**LOCAL_INLINE void ENET_HWA_DMA_SetRxWDGCounterUnit (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, ENET_DMA_WDGCounterUnitType *eCounterUnit*)**

Set receive interrupt watchdog timer count unit.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>eCounterUnit</i>	watchdog timer count unit

**LOCAL_INLINE void ENET_HWA_DMA_SetSlotComparisonEnFlag (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, bool *bEnable*)**

Set slot comparison enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_DMA_SetSlotInterval (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint16_t *u16SlotInterval*)**

Set slot interval value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u16SlotInterval</i>	slot interval

**LOCAL_INLINE void ENET_HWA_DMA_SetTxArbitrationAlgorithm (ENET_Type *const *pEnet*,
ENET_DMA_TxArbitrationAlgorithmType *eArbitrationAlgorithm*)**

Set DMA arbitration algorithm between transmit channels.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eArbitrationAlgorithm</i>	DMA arbitration algorithm

**LOCAL_INLINE void ENET_HWA_DMA_SetTxBurstLength (ENET_Type *const *pEnet*,
ENET_DMA_ChannelType *eDMAChannel*, uint8_t *u8Length*)**

Set DMA transmit programmable burst length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u8Length</i>	transmit burst length

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxChannelWeight (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8Weight)
```

Set DMA transmit channel weight.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u8Weight</i>	transmit channel weight

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxDescListHeadAddr (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32TxAddr)
```

Set DMA channel Tx descriptor list header address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32TxAddr</i>	Tx descriptor list header address

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxDescListTailAddr (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32TxAddr)
```

Set DMA channel Tx descriptor list tail address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u32TxAddr</i>	Tx descriptor list tail address

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxDescRingLength (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t u16Length)
```

Set DMA channel Tx descriptor ring length.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel
in	<i>u16Length</i>	Tx descriptor ring length

**LOCAL_INLINE void ENET_HWA_DMA_SetTxRxArbitrationScheme (ENET_Type *const *pEnet*,
ENET_DMA_ArbitrationSchemeType *eArbitrationScheme*)**

Set DMA arbitration scheme between the transmit and receive paths.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eArbitrationScheme</i>	DMA arbitration algorithm

**LOCAL_INLINE void ENET_HWA_DMA_StartRx (ENET_Type *const *pEnet*, ENET_DMA_ChannelType
eDMAChannel)**

Start DMA reception.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

**LOCAL_INLINE void ENET_HWA_DMA_StartTx (ENET_Type *const *pEnet*, ENET_DMA_ChannelType
eDMAChannel)**

Start DMA transmission.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_DMA_StopRx (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)

Stop DMA reception.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_DMA_StopTx (ENET_Type *const *pEnet*, ENET_DMA_ChannelType *eDMAChannel*)

Stop DMA transmission.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_DMA_SwReset (ENET_Type *const *pEnet*)

Initiate DMA software reset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearExcessiveDeferralStatus (ENET_Type *const *pEnet*)

Clear the excessive deferral status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearInterruptStatus (ENET_Type *const *pEnet*, uint32_t *u32Interrupt*)

Clear the interrupt flags of MAC.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Interrupt</i>	the interrupt events to be cleared

LOCAL_INLINE void ENET_HWA_MAC_ClearLossOfCarrierStatus (ENET_Type *const *pEnet*)

Clear the loss of carrier status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearMDIOInterruptStatus (ENET_Type *const *pEnet*)

Clear the MDIO interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearPhyInterruptStatus (ENET_Type *const *pEnet*)

Clear the PHY interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearRxTxStatus (ENET_Type *const *pEnet*, uint32_t *u32Status*)

Clear the MAC receive and transmit status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Status</i>	the status flags to be cleared

LOCAL_INLINE void ENET_HWA_MAC_ClearRxWDGTimeoutStatus (ENET_Type *const *pEnet*)

Clear the receive watchdog timeout status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearTimestampSecondsOverflowStatus (ENET_Type *const *pEnet*)

Clear the timestamp seconds overflow status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearTimestampTargetTimeError (ENET_Type *const *pEnet*, ENET_MAC_PPSChannelType *ePPSChann*)

Clear the timestamp target time error status of the PPS channel.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	the channel number of PPS

LOCAL_INLINE void ENET_HWA_MAC_ClearTimestampTargetTimeReachedStatus (ENET_Type *const *pEnet*, ENET_MAC_PPSChannelType *ePPSChann*)

Clear the status of the timestamp target time reached for target time of the PPS channel.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	the channel number of PPS

LOCAL_INLINE void ENET_HWA_MAC_ClearTxExcessiveCollisionStatus (ENET_Type *const *pEnet*)

Clear the excessive collision status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearTxJabberTimeoutStatus (ENET_Type *const *pEnet*)

Clear the transmit jabber timer timeout status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearTxLateCollisionStatus (ENET_Type *const *pEnet*)

Clear the late collision status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearTxNoCarrierStatus (ENET_Type *const *pEnet*)

Clear the no carrier status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ClearTxTimestampInterruptStatus (ENET_Type *const *pEnet*)

Clear the transmit timestamp interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE bool ENET_HWA_MAC_Get12BitVLANTagEnFlag (const ENET_Type *const *pEnet*)

Get 12-Bit VLAN tag comparison enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true 12-Bit VLAN tag comparison is enabled

false 12-Bit VLAN tag comparison is disabled

LOCAL_INLINE bool ENET_HWA_MAC_Get2KPacketSupportFlag (const ENET_Type *const *pEnet*)

Get 2K packet support enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true 2K packet support is enabled

false 2K packet support is disabled

LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetAddendRegisterUpdateStatus (const ENET_Type *const *pEnet*)

Get PTP addend register update status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_StatusType

LOCAL_INLINE bool ENET_HWA_MAC_GetAutoStripPadOrCRCEnFlag (const ENET_Type *const *pEnet*)

Get automatic Pad or CRC stripping enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Automatic Pad or CRC stripping is enabled

false Automatic Pad or CRC stripping is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetAV802_1ASModeEnFlag (const ENET_Type *const *pEnet*)

Get time stamp AV802_1AS mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true time stamp AV802_1AS mode is enabled

false time stamp AV802_1AS mode is disabled

LOCAL_INLINE ENET_MTL_QueueType ENET_HWA_MAC_GetAVCtrlPacketQueue (const ENET_Type *const *pEnet*)

Get AV untagged control packets queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MTL_QueueType

LOCAL_INLINE ENET_MAC_BackOffLimitType ENET_HWA_MAC_GetBackOffLimit (const ENET_Type *const *pEnet*)

Get back-off limit.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_BackOffLimitType

LOCAL_INLINE bool ENET_HWA_MAC_GetBackToBackTransactionEnFlag (const ENET_Type *const *pEnet*)

Get MDIO back to back transactions enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MDIO back to back transactions is enabled

false MDIO back to back transactions is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetBroadcastPacketDisableFlag (const ENET_Type *const *pEnet*)

Get broadcast packets disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Drop broadcast packets

false receive broadcast packets

LOCAL_INLINE bool ENET_HWA_MAC_GetChannelBasedVLANTagInsertEnFlag (const ENET_Type *const *pEnet*)

Get channel based tag insertion enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Channel based tag insertion is enabled

false Channel based tag insertion is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetChannelFlexiblePPSOutputEnFlag (const ENET_Type *const *pEnet*)

Get PPS flexible mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true PPS work in flexible mode

false PPS work in fixed mode

**LOCAL_INLINE ENET_MAC_PPSTargetTimeModeType
ENET_HWA_MAC_GetChannelPPSTargetTimeModeSelect (const ENET_Type *const *pEnet*,
ENET_MAC_PPSTargetTimeModeType *ePPSChann*)**

Get PPS channel target time register operating mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

ENET_MAC_PPSTargetTimeModeType

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetConfiguration (const ENET_Type *const *pEnet*)

Get the MAC Configuration Register value.

===== *

ENET system control

===== *

ENET MAC Configuration and Status *

Parameters

in	<i>pEnet</i>	The base address of the ENET instance
----	--------------	---------------------------------------

Returns

The Register value

```
LOCAL_INLINE ENET_MAC_CSRClockRangeType ENET_HWA_MAC_GetCSRClockRange (const ENET_Type *const pEnet)
```

Get the MDC clock range.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_CSRClockRangeType

```
LOCAL_INLINE ENET_MAC_CtrlPacketFilterModeType ENET_HWA_MAC_GetCtrlPacketFilterMode (const ENET_Type *const pEnet)
```

Get control packets filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_CtrlPacketFilterModeType

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetDebugStatus (const ENET_Type *const pEnet)
```

Get MAC debug status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC debug status

LOCAL_INLINE bool ENET_HWA_MAC_GetDeferralCheckEnFlag (const ENET_Type *const *pEnet*)

Get deferral check enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Deferral check is enabled

false Deferral check is disabled

LOCAL_INLINE ENET_MAC_AddrFilterModeType ENET_HWA_MAC_GetDestAddrFilterMode (const ENET_Type *const *pEnet*)

Get destination address filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_AddrFilterModeType

LOCAL_INLINE bool ENET_HWA_MAC_GetDoubleVLANProcessEnFlag (const ENET_Type *const *pEnet*)

Get double VLAN processing enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Double VLAN processing is enabled

false Double VLAN processing is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetDropNonTCPUDPOverIPEnFlag (const ENET_Type *const *pEnet*)

Get drop non-TCP/UDP over IP packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Drop non-TCP/UDP over IP packets

false Forward non-TCP/UDP over IP packets

LOCAL_INLINE ENET_MAC_DuplexModeType ENET_HWA_MAC_GetDuplexMode (const ENET_Type *const *pEnet*)

Get ethernet duplex mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_DuplexModeType

LOCAL_INLINE bool ENET_HWA_MAC_GetEthernetPTPPacketProcessEnFlag (const ENET_Type *const *pEnet*)

Get processing of PTP packets over ethernet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true processing of PTP packets over ethernet is enabled

false processing of PTP packets over ethernet is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetEventMsgSnapshotEnFlag (const ENET_Type *const *pEnet*)

Get snapshot for event messages enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true snapshot for event messages is enabled

false snapshot for event messages is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetExtConfiguration (const ENET_Type *const *pEnet*)

Get the MAC extended configuration register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

The register value

LOCAL_INLINE ENET_MAC_ExtInterPacketGapType ENET_HWA_MAC_GetExtInterPacketGap (const ENET_Type *const *pEnet*)

Get the MAC extended inter-packet gap.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_ExtInterPacketGapType

LOCAL_INLINE bool ENET_HWA_MAC_GetExtInterPacketGapEnFlag (const ENET_Type *const *pEnet*)

Get extended inter-packet gap enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Extended inter-packet gap is enabled

false Extended inter-packet gap is disabled

LOCAL_INLINE ENET_MAC_PPSCtrlType ENET_HWA_MAC_GetFixedPPSOutputFrequency (ENET_Type *const *pEnet*)

Get fixed PPS output frequency.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_PPSCtrlType

LOCAL_INLINE ENET_MAC_PPSCmdType ENET_HWA_MAC_GetFlexiblePPSCommand (ENET_Type *const *pEnet*, ENET_MAC_PPSCchannelType *ePPSChann*)

Get flexible PPS command.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

ENET_MAC_PPSCmdType

LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetFlowCtrlOrBackpressureStatus (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get flow control busy or backpressure status of Queue n.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_StatusType

LOCAL_INLINE bool ENET_HWA_MAC_GetFullDuplexCRSEnFlag (const ENET_Type *const *pEnet*)

Get carrier sense before transmission enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Carrier sense before transmission is enabled

false Carrier sense before transmission is disabled

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetGiantPacketSizeLimit (const ENET_Type *const *pEnet*)

Get giant packet size limit value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Giant packet size limit value

LOCAL_INLINE bool ENET_HWA_MAC_GetGiantPacketSizeLimitCtrlEnFlag (const ENET_Type *const *pEnet*)

Get giant packet size Limit control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Giant packet size Limit control is enabled

false Giant packet size Limit control is disabled

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetGMIIData (const ENET_Type *const *pEnet*)

Get MDIO data.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

data value

LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetGMIIStatus (const ENET_Type *const *pEnet*)

Get MDIO busy status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_STATUS_IDLE GMII status is idle

ENET_MAC_STATUS_BUSY GMII status is busy,

LOCAL_INLINE bool ENET_HWA_MAC_GetHalfDuplexCRSDisableFlag (const ENET_Type *const *pEnet*)

Get carrier sense during transmission disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Carrier sense during transmission is disabled

false Carrier sense during transmission is enabled

LOCAL_INLINE bool ENET_HWA_MAC_GetHashOrPerfectFilterEnFlag (const ENET_Type *const *pEnet*)

Get hash or perfect filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Enable hash or pPerfect filter

false Disable hash or pPerfect filter

LOCAL_INLINE ENET_MAC_HashOrPerfectModeType ENET_HWA_MAC_GetHashOrPerfectMulticastMode (const ENET_Type *const *pEnet*)

Get hash or perfect multicast filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_HashOrPerfectModeType

LOCAL_INLINE ENET_MAC_HashOrPerfectModeType ENET_HWA_MAC_GetHashOrPerfectUnicastMode (const ENET_Type *const *pEnet*)

Get hash or perfect unicast filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_HashOrPerfectModeType

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHashTableFirst32Bits (const ENET_Type *const *pEnet*)

Get the hash table first 32 bits value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Hash table first 32 bits value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHashTableSecond32Bits (const ENET_Type *const *pEnet*)

Get the hash table second 32 bits value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Hash table second 32 bits value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHwFeature0 (const ENET_Type *const *pEnet*)

Get MAC hardware features 0.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC hardware features 0

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHwFeature1 (const ENET_Type *const *pEnet*)

Get MAC hardware features 1.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC hardware features 1

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHwFeature2 (const ENET_Type *const *pEnet*)

Get MAC hardware features 2.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC hardware features 2

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHwFeature3 (const ENET_Type *const *pEnet*)

Get MAC hardware features 3.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC hardware features 3

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetInnerVLANInclusion (const ENET_Type *const *pEnet*)

Get inner VLAN tag inclusion or replacement register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

The register value

LOCAL_INLINE bool ENET_HWA_MAC_GetInnerVLANPriorityCtrlEnFlag (const ENET_Type *const *pEnet*)

Get inner VLAN priority control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Inner VLAN priority control is enabled

false Inner VLAN priority control is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetInnerVLANTagCmpEnFlag (const ENET_Type *const *pEnet*)

Get inner VLAN tag comparison enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Inner VLAN tag comparison is enabled

false Inner VLAN tag comparison is disabled

LOCAL_INLINE ENET_MAC_VLANTagCtrlType ENET_HWA_MAC_GetInnerVLANTagCtrl (const ENET_Type *const *pEnet*)

Get inner VLAN tag control mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_VLANTagCtrlType

LOCAL_INLINE bool ENET_HWA_MAC_GetInnerVLANTagInputEnFlag (const ENET_Type *const *pEnet*)

Get inner VLAN tag input enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Inner VLAN tag input is enabled

false Inner VLAN tag input is disabled

LOCAL_INLINE ENET_MAC_VLANTagStripModeType ENET_HWA_MAC_GetInnerVLANTagStripMode (const ENET_Type *const *pEnet*)

Get inner VLAN tag stripping mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_VLANTagStripModeType

LOCAL_INLINE ENET_MAC_VLANType ENET_HWA_MAC_GetInnerVLANType (const ENET_Type *const *pEnet*)

Get inner VLAN type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_VLANType

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetInterruptEnable (const ENET_Type *const *pEnet*)

Get the MAC interrupts enable register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC interrupts enable register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetInterruptStatus (const ENET_Type *const *pEnet*)

Get the interrupt status of ENET MAC.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t the interrupt status

LOCAL_INLINE bool ENET_HWA_MAC_GetIPChecksumOffloadEnFlag (const ENET_Type *const *pEnet*)

Get checksum offload function enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Checksum offload function is enabled

false Checksum offload function is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetIPv4UDPPTPPacketProcessEnFlag (const ENET_Type *const *pEnet*)

Get processing of PTP packets over IPv4-UDP enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true processing of PTP packets over IPv4-UDP is enabled

false processing of PTP packets over IPv4-UDP is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetIPv6UDPPTPPacketProcessEnFlag (const ENET_Type *const *pEnet*)

Get processing of PTP packets over IPv6-UDP enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true processing of PTP packets over IPv6-UDP is enabled

false processing of PTP packets over IPv6-UDP is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetJabberTimerDisableFlag (const ENET_Type *const *pEnet*)

Get transmitter jabber timer disabled flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Transmitter jabber timer is disabled

false Transmitter jabber timer is enabled

LOCAL_INLINE bool ENET_HWA_MAC_GetJumboPacketEnFlag (const ENET_Type *const *pEnet*)

Get jumbo packet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Jumbo packet is enabled

false Jumbo packet is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Addr0 (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 address0.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 3 address0 value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Addr1 (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 address1.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 3 address1 value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Addr2 (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 address2.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 3 address2 value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Addr3 (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 address3.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 3 address3 value

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3DestAddrInvMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 destination address inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

true layer 3 destination address inverse match is enabled

false layer 3 destination address inverse match is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3DestAddrMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 destination address match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

true layer 3 destination address match is enabled

false layer 3 destination address match is disabled

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3DestIPv4Addr (const ENET_Type *const pEnet,  
uint8_t u8Index)
```

Get layer 3 destination IPv4 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 3 destination IPv4 address value

```
LOCAL_INLINE uint8_t ENET_HWA_MAC_GetLayer3IPv4DestAddrBitsMatch (const ENET_Type *const  
pEnet, uint8_t u8Index)
```

Get layer3 IPv4 destination address bits match.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

lower retval bits are masked

```
LOCAL_INLINE uint8_t ENET_HWA_MAC_GetLayer3IPv4SrcAddrBitsMatch (const ENET_Type *const  
pEnet, uint8_t u8Index)
```

Get layer3 IPv4 source address bits match.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

lower retval bits are masked

```
LOCAL_INLINE void ENET_HWA_MAC_GetLayer3IPv6Addr (const ENET_Type *const pEnet, uint8_t u8Index, uint8_t aIPv6Addr[16u])
```

Get layer 3 IPv6 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
out	<i>aIPv6Addr</i>	layer 3 IPv6 address value

```
LOCAL_INLINE uint8_t ENET_HWA_MAC_GetLayer3IPv6AddrBitsMatch (const ENET_Type *const pEnet,  
uint8_t u8Index)
```

Get layer3 IPv6 address bits match.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

lower retval bits are masked

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3Layer4Ctrl (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get the layer 3 and layer 4 control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 3 and layer 4 control register value

LOCAL_INLINE ENET_DMA_ChannelType ENET_HWA_MAC_GetLayer3Layer4DMAChannel (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 and layer 4 control DMA channel number.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

ENET_DMA_ChannelType

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3Layer4DMASelectEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 and layer 4 control DMA channel select enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

true DMA channel select is enabled

false DMA channel select is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3Layer4FilterEnFlag (const ENET_Type *const *pEnet*)

Get layer 3 and layer 4 filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Enable layer 3 and layer 4 filter

false Disable layer 3 and layer 4 filter

LOCAL_INLINE ENET_MAC_Layer3ProtocolType ENET_HWA_MAC_GetLayer3ProtocolType (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 protocol type (IPv4 or IPv6)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

ENET_MAC_Layer3ProtocolType (IPv4 or IPv6)

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3SrcAddrInvMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 source address inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

true layer 3 source address inverse match is enabled

false layer 3 source address inverse match is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3SrcAddrMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 3 source address match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

true layer 3 source address match is enabled

false layer 3 source address match is disabled

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer3SrcIPv4Addr (const ENET_Type *const pEnet,  
uint8_t u8Index)
```

Get layer 3 source IPv4 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 3 source IPv4 address value

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetLayer4Addr (const ENET_Type *const pEnet, uint8_t  
u8Index)
```

Get layer 4 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 4 address value

```
LOCAL_INLINE uint16_t ENET_HWA_MAC_GetLayer4DestPort (const ENET_Type *const pEnet, uint8_t  
u8Index)
```

Get layer 4 destination port number.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 4 destination port number

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer4DestPortInvMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 4 destination port inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

- true layer 4 destination port inverse match is enabled
- false layer 4 destination port inverse match is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer4DestPortMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 4 destination port match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

- true layer 4 destination port match is enabled
- false layer 4 destination port match is disabled

LOCAL_INLINE ENET_MAC_Layer4ProtocolType ENET_HWA_MAC_GetLayer4ProtocalType (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 4 protocol type (UDP or TCP)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

ENET_MAC_Layer4ProtocolType

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetLayer4SrcPort (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 4 source port number.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

layer 4 source port number

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer4SrcPortInvMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 4 source port inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

true layer 4 source port inverse match is enabled

false layer 4 source port inverse match is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetLayer4SrcPortMatchEnFlag (const ENET_Type *const *pEnet*, uint8_t *u8Index*)

Get layer 4 source port match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index

Returns

true layer 4 source port match is enabled

false layer 4 source port match is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetLoopbackModeEnFlag (const ENET_Type *const *pEnet*)

Get loopback mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Loopback mode is enabled

false Loopback mode is disabled

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetMacAddr0DMAChannelSelect (const ENET_Type *const *pEnet*)

Get MAC address0 DMA channel select.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address0 DMA channel select

LOCAL_INLINE bool ENET_HWA_MAC_GetMacAddr0EnFlag (const ENET_Type *const *pEnet*)

Get MAC address0 enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MAC address0 is enabled

false MAC address0 is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr0High (const ENET_Type *const *pEnet*)

Get MAC address0 high register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address0 high register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr0Low (const ENET_Type *const *pEnet*)

Get MAC address0 low register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address0 low register value

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetMacAddr1DMAChannelSelect (const ENET_Type *const *pEnet*)

Get MAC address1 DMA channel select.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address1 DMA channel select

LOCAL_INLINE bool ENET_HWA_MAC_GetMacAddr1EnFlag (const ENET_Type *const *pEnet*)

Get MAC address1 enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MAC address1 is enabled

false MAC address1 is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr1High (const ENET_Type *const *pEnet*)

Get MAC address1 high register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address1 high register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr1Low (const ENET_Type *const *pEnet*)

Get MAC address1 low register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address1 low register value

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetMacAddr1MaskByteCtrl (const ENET_Type *const *pEnet*)

Get MAC address1 mask byte control.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address1 mask byte value

LOCAL_INLINE ENET_MAC_AddrType ENET_HWA_MAC_GetMacAddr1Type (const ENET_Type *const *pEnet*)

Get MAC address1 compare mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_ADDR_TYPE_DEST MAC Address1 is used to compare with the DA fields

ENET_MAC_ADDR_TYPE_SRC MAC Address1 is used to compare with the SA fields

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetMacAddr2DMAChannelSelect (const ENET_Type *const *pEnet*)

Get MAC address2 DMA channel select.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address2 DMA channel select

LOCAL_INLINE bool ENET_HWA_MAC_GetMacAddr2EnFlag (const ENET_Type *const *pEnet*)

Get MAC address2 enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MAC address2 is enabled

false MAC address2 is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr2High (const ENET_Type *const *pEnet*)

Get MAC address2 high register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address2 high register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMacAddr2Low (const ENET_Type *const *pEnet*)

Get MAC address2 low register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address2 low register value

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetMacAddr2MaskByteCtrl (const ENET_Type *const *pEnet*)

Get MAC address2 mask byte control.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC address2 mask byte value

LOCAL_INLINE ENET_MAC_AddrType ENET_HWA_MAC_GetMacAddr2Type (const ENET_Type *const *pEnet*)

Get MAC address2 compare mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_ADDR_TYPE_DEST MAC Address2 is used to compare with the DA fields

ENET_MAC_ADDR_TYPE_SRC MAC Address2 is used to compare with the SA fields

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMACVersion (const ENET_Type *const *pEnet*)

Get MAC version.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MAC version number

LOCAL_INLINE bool ENET_HWA_MAC_GetMasterMsgSanpshotEnFlag (const ENET_Type *const *pEnet*)

Get snapshot for messages relevant to master enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true snapshot for messages relevant to master

false snapshot for messages relevant to slave

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMDIOAddress (const ENET_Type *const *pEnet*)

Get MAC MDIO address register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MDIO address register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMDIOData (const ENET_Type *const *pEnet*)

Get MDIO data register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE bool ENET_HWA_MAC_GetMDIOInterruptEnFlag (const ENET_Type *const *pEnet*)

Get MDIO interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MDIO interrupt is enabled

false MDIO interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetMDIOInterruptStatus (const ENET_Type *const *pEnet*)

Get the MDIO interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true the MDIO operation is complete

false there is no MDIO operation completion event

LOCAL_INLINE ENET_MAC_InterPacketGapType ENET_HWA_MAC_GetMinInterPacketGap (const ENET_Type *const *pEnet*)

Get the minimum Inter-Packet Gap.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_InterPacketGapType.

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMmcControl (const ENET_Type *const *pEnet*)

Get MMC control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC control register value

LOCAL_INLINE bool ENET_HWA_MAC_GetMmcCounterFreezeStatus (const ENET_Type *const *pEnet*)

Get MMC counters freeze status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MMC counter freeze is enabled

false MMC counter freeze is disabled

LOCAL_INLINE ENET_MAC_MmcCntPresetLevelType ENET_HWA_MAC_GetMmcCounterPresetLevel (const ENET_Type *const *pEnet*)

Get MMC counters full-half preset level.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_MMC_COUNTER_PRESET_LEVEL_FULL MMC counter preset to almost-full value

ENET_MAC_MMC_COUNTER_PRESET_LEVEL_HALF MMC counter preset to almost-half value

LOCAL_INLINE bool ENET_HWA_MAC_GetMmcCounterPresetStatus (const ENET_Type *const *pEnet*)

Get MMC counters preset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MMC counter preset is enabled

false MMC counter preset is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetMmcCounterResetOnReadEnFlag (const ENET_Type *const *pEnet*)

Get MMC counters reset on read enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MMC counters are reset to zero after read

false MMC counters are not reset to zero after read

LOCAL_INLINE bool ENET_HWA_MAC_GetMmcCounterResetStatus (const ENET_Type *const *pEnet*)

Get MMC counters reset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true All MMC counters are reset

false MMC counters are not reset

LOCAL_INLINE bool ENET_HWA_MAC_GetMmcCounterStopRolloverEnFlag (const ENET_Type *const *pEnet*)

Get MMC counters stop rollover enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MMC counters does not roll over to zero after reaching the maximum value

false MMC counters roll over to zero after reaching the maximum value

LOCAL_INLINE bool ENET_HWA_MAC_GetMMCInterruptStatus (const ENET_Type *const *pEnet*)

Get the MMC interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

This field is valid only when you enable MAC Management Counters(MMC)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

bool indicate the status of MMC interrupt

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMmcRxInterruptMask (const ENET_Type *const *pEnet*)

Get MMC interrupts mask of receive statistics counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC interrupts mask

LOCAL_INLINE bool ENET_HWA_MAC_GetMMCRxInterruptStatus (const ENET_Type *const *pEnet*)

Get the MMC receive interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

This field is valid only when you enable MAC Management Counters(MMC)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

bool indicate the status of MMC receive

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMmcRxInterruptStatus (const ENET_Type *const *pEnet*)

Get MMC interrupts of receive statistics counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC interrupts status

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMmcTxInterruptMask (const ENET_Type *const *pEnet*)

Get MMC interrupts mask of transmit statistics counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC interrupts mask

LOCAL_INLINE bool ENET_HWA_MAC_GetMMCTxInterruptStatus (const ENET_Type *const *pEnet*)

Get the MMC transmit interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

This field is valid only when you enable MAC Management Counters(MMC)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

bool indicate the status of MMC transmit

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetMmcTxInterruptStatus (const ENET_Type *const *pEnet*)

Get MMC interrupts of transmit statistics counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC interrupts status

LOCAL_INLINE ENET_MTL_QueueType ENET_HWA_MAC_GetMulticastAddrFilterFailPacketQueue (const ENET_Type *const *pEnet*)

Get multicast address filter fail packets queue.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MTL_QueueType

LOCAL_INLINE bool ENET_HWA_MAC_GetMulticastAddrFilterFailPacketQueueEnFlag (const ENET_Type *const *pEnet*)

Get multicast address filter fail packets queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true multicast address filter fail packets queue is enabled

false multicast address filter fail packets queue is disabled

LOCAL_INLINE ENET_MTL_QueueType ENET_HWA_MAC_GetMulticastAndBroadcastQueue (const ENET_Type *const *pEnet*)

Get multicast and broadcast queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MTL_QueueType

LOCAL_INLINE bool ENET_HWA_MAC_GetMulticastAndBroadcastQueueEnFlag (const ENET_Type *const *pEnet*)

Get multicast and broadcast queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true multicast and broadcast queue is enabled

false multicast and broadcast queue is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetPacketDuplicationCtrlEnFlag (const ENET_Type *const *pEnet*)

Get packet duplication control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Packet duplication control enabled

false Packet duplication control is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPacketFilter (const ENET_Type *const *pEnet*)

Get the MAC packet filter register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

The register value

LOCAL_INLINE bool ENET_HWA_MAC_GetPassAllMulticastEnFlag (const ENET_Type *const *pEnet*)

Get pass all the received multicast packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Pass all the received multicast packets

false Pass multicast packets that match the multicast filter

LOCAL_INLINE ENET_MAC_PhyClauseVersionType ENET_HWA_MAC_GetPhyClauseVersion (const ENET_Type *const *pEnet*)

Get MDIO clause version.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_PHY_CLAUSE_22 access phy use clause 22 mode

ENET_MAC_PHY_CLAUSE_45 access phy use clause 45 mode

LOCAL_INLINE bool ENET_HWA_MAC_GetPhyInterruptEnFlag (const ENET_Type *const *pEnet*)

Get phy interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true phy interrupt is enabled

false phy interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetPhyInterruptStatus (const ENET_Type *const *pEnet*)

Get the PHY interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

bool indicate whether the PHY interrupt is detected

LOCAL_INLINE ENET_MAC_MiiModeType ENET_HWA_MAC_GetPhyMiiMode (const ENET_Type *const *pEnet*)

Get phy interface mii mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_MiiModeType

LOCAL_INLINE ENET_MAC_PhyOperationCmdType ENET_HWA_MAC_GetPhyOperationCommand (const ENET_Type *const *pEnet*)

Get phy operation command.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

Commands defined in ENET_MAC_PhyOperationCmdType

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetPhyRegisterAddr (const ENET_Type *const *pEnet*)

Get phy register address.

Note

This field is valid only in clause 45 mode

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

phy register address

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetPhyRegisterOrDeviceAddr (const ENET_Type *const *pEnet*)

Get MDIO phy register(clause22) or MMD device(clause 45)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

phy register or device address

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetPhysicalLayerAddr (const ENET_Type *const *pEnet*)

Get MDIO physical layer address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

phy address

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSControl (const ENET_Type *const *pEnet*)

Get PPS control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t PPS control value

**LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSInterval (const ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*)**

Get PPS pulse interval.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

uint32_t PPS interval value

**LOCAL_INLINE ENET_MAC_MCGRCmdType ENET_HWA_MAC_GetPPSMCGRCommand (const ENET_Type
*const *pEnet*, ENET_MAC_PPSChannelType *ePPSChann*)**

Get PPS MCGR command.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>ePPSChann</i>	PPS channel
----	------------------	-------------

Returns

ENET_MAC_MCGRCmdType

**LOCAL_INLINE bool ENET_HWA_MAC_GetPPSMCGRModeEnFlag (const ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*)**

Get PPS MCGR Mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

true PPS MCGR Mode is enabled

false PPS MCGR Mode is disabled

**LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetPPSTargetTimeBusyStatus (const ENET_Type
*const *pEnet*, ENET_MAC_PPSChannelType *ePPSChann*)**

Get PPS target time busy status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

ENET_MAC_StatusType

**LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSTargetTimeInNanoseconds (const ENET_Type *const
pEnet, ENET_MAC_PPSChannelType *ePPSChann*)**

Get PPS target time nanoseconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

uint32_t target time nanoseconds

**LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSTargetTimeInSeconds (const ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*)**

Get PPS target time seconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

uint32_t target time seconds

**LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSWidth (const ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*)**

Get PPS pulse width.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel

Returns

uint32_t PPS pulse width

LOCAL_INLINE bool ENET_HWA_MAC_GetPreambleSuppressionEnFlag (const ENET_Type *const *pEnet*)

Get preamble suppression enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MDIO preamble suppression is enabled
false MDIO preamble suppression is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPresentationTime (const ENET_Type *const *pEnet*)

Get presentation time in ns.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

presentation time

LOCAL_INLINE bool ENET_HWA_MAC_GetPresentationTimeGenerationEnFlag (const ENET_Type *const *pEnet*)

Get presentation time generation enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true presentation time generation is enabled
false presentation time generation is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPresentationTimeUpdata (const ENET_Type *const *pEnet*)

Get presentation time update.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

update time value

LOCAL_INLINE bool ENET_HWA_MAC_GetPromiscuousModeEnFlag (const ENET_Type *const *pEnet*)

Get promiscuous mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Enable promiscuous mode

false Disable promiscuous mode

LOCAL_INLINE ENET_MTL_QueueType ENET_HWA_MAC_GetPTPoEQueue (const ENET_Type *const *pEnet*)

Get PTP over ethernet packet queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MTL_QueueType

LOCAL_INLINE ENET_MAC_PTPVersionType ENET_HWA_MAC_GetPTPPacketVersion (const ENET_Type *const *pEnet*)

Get PTP packets processing format version.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_PTPVersionType

**LOCAL_INLINE uint32_t ENET_HWA_MAC_GetQnTxFlowCtrl (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get Queue n tx flow control register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

The register value

**LOCAL_INLINE bool ENET_HWA_MAC_GetQnTxFlowCtrlEnableFlag (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get Queue n tx flow control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Return values

<i>true</i>	Queue0 tx flow control is enabled
<i>false</i>	Queue0 tx flow control is disabled

**LOCAL_INLINE ENET_MAC_PauseThresholdType ENET_HWA_MAC_GetQnTxPauseLowThreshold (const
ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)**

Get Queue n tx pause low threshold.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_PauseThresholdType

**LOCAL_INLINE uint16_t ENET_HWA_MAC_GetQnTxPauseTime (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get Queue n tx pause time value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Pause time value

LOCAL_INLINE bool ENET_HWA_MAC_GetQnTxZeroQuantaPauseDisableFlag (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get Queue n tx zero quanta pause disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Queue0 tx zero quanta pause is disabled

false is Queue0 tx zero quanta pause enabled

LOCAL_INLINE bool ENET_HWA_MAC_GetReceiveAllEnFlag (const ENET_Type *const *pEnet*)

Get receive all packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Receive all packets enabled

false Receive all packets is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetReceiveOwnDisableFlag (const ENET_Type *const *pEnet*)

Get receive own disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Receive own is disabled

false Receive own is enabled

LOCAL_INLINE bool ENET_HWA_MAC_GetRetryDisableFlag (const ENET_Type *const *pEnet*)

Get transmission retry disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Ttransmission retry is disabled

false Transmission retry is enabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRx1024ToMaxOctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of packets received with length 1024 to maxsize.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRx128To255OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of packets received with length 128 to 255 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRx256To511OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of packets received with length 256 to 511 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRx512To1023OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of packets received with length 512 to 1023 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRx64OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of packets received with length 64 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRx65To127OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of packets received with length 65 to 127 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxAlignmentErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with alignment error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxBroadcastPacketGood (const ENET_Type *const *pEnet*)

Get the number of good broadcast packets received.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxControlPacketGood (const ENET_Type *const *pEnet*)

Get the number of good control packets received.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetRxCRCCheckDisableFlag (const ENET_Type *const *pEnet*)

Get CRC checking for Received packets disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true CRC checking for rReceived packets is disabled

false CRC checking for rReceived packets is enabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxCRCErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with CRC error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetRxEnFlag (const ENET_Type *const *pEnet*)

Get receiver enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Receiver is enabled

false Receiver is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxFIFOOverflowPacketCnt (const ENET_Type *const *pEnet*)

Get the number of missed received packets because of FIFO overflow.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxFlowCtrl (const ENET_Type *const *pEnet*)

Get receive flow control register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Register value

LOCAL_INLINE bool ENET_HWA_MAC_GetRxFlowCtrlEnFlag (const ENET_Type *const *pEnet*)

Get receive flow control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Return values

<i>true</i>	Receive flow control is enabled
<i>false</i>	Receive flow control is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetRxInterruptStatus (const ENET_Type *const *pEnet*)

Get the receive interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

bool indicate the status of received packets

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxJabberErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with jabber error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxLengthErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with length error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxMulticastPacketGood (const ENET_Type *const *pEnet*)

Get the number of good multicast packets received.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxOctetCntGood (const ENET_Type *const *pEnet*)

Get the number of bytes received in good packets.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxOctetCntGoodBad (const ENET_Type *const *pEnet*)

Get the number of bytes received in good and bad packets.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxOutOfRangeTypePacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with length greater than 1500 but less than 1536.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxOverSizeGood (const ENET_Type *const *pEnet*)

Get the number of packets received without errors and with length greater than maxsize.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxPacketCntGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets received.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetRxPacketControllerFIFOStatus (const ENET_Type *const *pEnet*)

Get MAC receive packet controller FIFO status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true receive packet controller FIFO is active

false receive packet controller FIFO is inactive

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxPausePacketCnt (const ENET_Type *const *pEnet*)

Get the number of good and valid pause packets received.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetRxProtocalEngineStatus (const ENET_Type *const *pEnet*)

Get MAC GMII or MII receive protocol engine status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true receive protocol engine is active

false receive protocol engine is inactive

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxQueueCtrl0 (const ENET_Type *const *pEnet*)

Get Receive queue control 0 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxQueueCtrl1 (const ENET_Type *const *pEnet*)

Get Receive queue control 1 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxQueueCtrl2 (const ENET_Type *const *pEnet*)

Get Receive queue control 2 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxQueueCtrl4 (const ENET_Type *const *pEnet*)

Get Receive queue control 4 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Register value

**LOCAL_INLINE uint8_t ENET_HWA_MAC_GetRxQueueSelectedPriorities (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get receive queue priorities.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	queue id

Returns

eQueue priorities value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxReceiveErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with receive error or Packet Extension error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxRunErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with runt error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetRxStatusInnerVLANTagEnFlag (const ENET_Type *const *pEnet*)

Get inner VLAN tag enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Inner VLAN tag is enabled

false Inner VLAN tag is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetRxStatusInterruptEnFlag (const ENET_Type *const *pEnet*)

Get receive status interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true receive status interrupt is enabled

false receive status interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetRxStatusVLANTagEnFlag (const ENET_Type *const *pEnet*)

Get VLAN tag in rx status enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true VLAN tag in rx status is enabled

false VLAN tag in rx status is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetRxSVLANMatchEnFlag (const ENET_Type *const *pEnet*)

Get Receive S-VLAN match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Receive S-VLAN match is enabled

false Receive S-VLAN match is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxtxStatus (const ENET_Type *const *pEnet*)

Get the MAC receive and transmit status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t the MAC receive and transmit status

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxUndersizePacketGood (const ENET_Type *const *pEnet*)

Get the number of packets received with length less than 64 bytes, without any errors.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxVLANPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad VLAN packets received.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetRxWatchdogErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets received with error because of watchdog timeout error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetRxWDGTimeoutStatus (const ENET_Type *const *pEnet*)

Get the receive watchdog timeout status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true receive watchdog timeout

false no receive watchdog timeout

LOCAL_INLINE bool ENET_HWA_MAC_GetSkipAddrPacketEnFlag (const ENET_Type *const *pEnet*)

Get skip address packet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true skip address packet is enabled

false skip address packet is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetSlowDetectEnFlag (const ENET_Type *const *pEnet*)

Get slow protocol detection enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Slow protocol detection enabled

false Slow protocol detection is disabled

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetSnapshotPTPPacketType (const ENET_Type *const *pEnet*)

Get PTP packets type for taking snapshots.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

PTP packets type

LOCAL_INLINE bool ENET_HWA_MAC_GetSourceAddressFilterEnFlag (const ENET_Type *const *pEnet*)

Get source address filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Enable source address filter

false Disable source address fFilter

LOCAL_INLINE ENET_MAC_SpeedType ENET_HWA_MAC_GetSpeed (const ENET_Type *const *pEnet*)

Get ethernet line speed.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_SpeedType

LOCAL_INLINE ENET_MAC_AddrFilterModeType ENET_HWA_MAC_GetSrcAddrFilterMode (const ENET_Type *const *pEnet*)

Get source address filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_AddrFilterModeType

LOCAL_INLINE ENET_MAC_SrcAddrCtrlType ENET_HWA_MAC_GetSrcAddrInsertReplaceCtrl (const ENET_Type *const *pEnet*)

Get the source address insertion or replacement control type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

source address insertion or replacement control type

LOCAL_INLINE bool ENET_HWA_MAC_GetStripTypePacketCRCEnFlag (const ENET_Type *const *pEnet*)

Get CRC stripping for type packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

- true CRC stripping for type packets is enabled
- false CRC stripping for type packets is disabled

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetSubNanoSecondIncrementValue (const ENET_Type *const *pEnet*)

Get PTP sub-nanosecond increment value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

- sub-nanosecond increment value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetSubSecondIncrementReg (const ENET_Type *const *pEnet*)

Get PTP sub-second increment register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

- sub-second increment register value

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetSubSecondIncrementValue (const ENET_Type *const *pEnet*)

Get PTP sub-second increment value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

sub-second increment value

LOCAL_INLINE bool ENET_HWA_MAC_GetSVLANEnFlag (const ENET_Type *const *pEnet*)

Get S-VLAN enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true S-VLAN is enabled

false S-VLAN is disabled

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetSystemTimeHigherWordSeconds (const ENET_Type *const *pEnet*)

Get PTP system time seconds bits[47:32] value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

seconds bits[47:32] value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetSystemTimeNanoSeconds (const ENET_Type *const *pEnet*)

Get PTP system time nanoseconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

nanoseconds value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetSystemTimeNanoSecondsUpdateReg (const ENET_Type *const *pEnet*)

Get PTP system time nanoseconds update register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

nanoseconds update register value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetSystemTimeNanoSecondsUpdateValue (const ENET_Type *const *pEnet*)

Get PTP system time nanoseconds update value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

nanoseconds update value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetSystemTimeSeconds (const ENET_Type *const *pEnet*)

Get PTP system time seconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

seconds value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetSystemTimeSecondsUpdateValue (const ENET_Type *const *pEnet*)

Get PTP system time seconds update register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

seconds update register value

LOCAL_INLINE bool ENET_HWA_MAC_GetTaggedAVCtrlPacketQueueEnFlag (const ENET_Type *const *pEnet*)

Get tagged AV control packets queuing enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true tagged AV control packets queue is enabled

false tagged AV control packets queue is disabled

LOCAL_INLINE ENET_MAC_PTPoEQueueCtrlType ENET_HWA_MAC_GetTaggedPTPoEQueueCtrl (const ENET_Type *const *pEnet*)

Get tagged PTP over ethernet packets queuing control.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_PTPoEQueueCtrlType

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTimestampAddend (const ENET_Type *const *pEnet*)

Get PTP timestamp addend register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

timestamp addend value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTimestampControl (const ENET_Type *const *pEnet*)

Get time stamp control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

time stamp control value

LOCAL_INLINE int32_t ENET_HWA_MAC_GetTimestampEgressAsymmetryCorrection (const ENET_Type *const *pEnet*)

Get timestamp egress asymmetry correction, used for PDelay_Req PTP messages.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t correction value

LOCAL_INLINE int32_t ENET_HWA_MAC_GetTimestampEgressCorrectionInNanoseconds (const ENET_Type *const *pEnet*)

Get timestamp egress correction in nanoseconds, used for timestamp in egress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t correction value

LOCAL_INLINE int8_t ENET_HWA_MAC_GetTimestampEgressCorrectionInSubNanoseconds (const ENET_Type *const *pEnet*)

Get timestamp egress correction in sub-nanoseconds, used for timestamp in egress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t correction value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTimestampEgressLatency (const ENET_Type *const *pEnet*)

Get the egress MAC latency.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t egress latency

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetTimestampEgressLatencyInNanoseconds (const ENET_Type *const *pEnet*)

Get the egress MAC latency in nanoseconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t egress latency in nanoseconds

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetTimestampEgressLatencyInSubNanoseconds (const ENET_Type *const *pEnet*)

Get the egress MAC latency in sub-nanoseconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t egress latency in sub-nanoseconds

LOCAL_INLINE bool ENET_HWA_MAC_GetTimestampEnFlag (const ENET_Type *const *pEnet*)

Get timestamp enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true timestamp processing is enabled

false timestamp processing is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetTimestampForAllPacketEnFlag (const ENET_Type *const *pEnet*)

Get timestamp for all packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true timestamp for all packets is enabled

false timestamp for all packets is disabled

LOCAL_INLINE int32_t ENET_HWA_MAC_GetTimestampIngressAsymmetryCorrection (const ENET_Type *const *pEnet*)

Get timestamp ingress asymmetry correction, used for PDelay_Resp PTP messages.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t correction value

LOCAL_INLINE int32_t ENET_HWA_MAC_GetTimestampIngressCorrectionInNanoseconds (const ENET_Type *const *pEnet*)

Get timestamp ingress correction in nanoseconds, used for timestamp in ingress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t correction value

LOCAL_INLINE int8_t ENET_HWA_MAC_GetTimestampIngressCorrectionInSubNanoseconds (const ENET_Type *const *pEnet*)

Get timestamp ingress correction in sub-nanoseconds, used for timestamp in ingress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t correction value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTimestampIngressLatency (const ENET_Type *const *pEnet*)

Get the ingress MAC latency.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t ingress latency

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetTimestampIngressLatencyInNanoseconds (const ENET_Type *const *pEnet*)

Get the ingress MAC latency in nanoseconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t ingress latency in nanoseconds

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetTimestampIngressLatencyInSubNanoseconds (const ENET_Type *const *pEnet*)

Get the ingress MAC latency in sub-nanoseconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t ingress latency in sub-nanoseconds

LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetTimestampInitStatus (const ENET_Type *const *pEnet*)

Get timestamp initialize status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_StatusType

LOCAL_INLINE bool ENET_HWA_MAC_GetTimestampInterruptEnFlag (const ENET_Type *const *pEnet*)

Get timestamp interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true timestamp interrupt is enabled

false timestamp interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetTimestampInterruptStatus (const ENET_Type *const *pEnet*)

Get the timestamp interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

bool indicate the status of timestamp interrupt

LOCAL_INLINE ENET_MAC_TimestampRolloverType ENET_HWA_MAC_GetTimestampRolloverCtrl (const ENET_Type *const *pEnet*)

Get PTP timestamp rollover mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_TimestampRolloverType

LOCAL_INLINE bool ENET_HWA_MAC_GetTimestampSecondsOverflowStatus (const ENET_Type *const *pEnet*)

Indicates whether the timestamp seconds overflow status is detected.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true timestamp seconds overflow status is detected

false timestamp seconds overflow status is not detected

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTimestampStatus (const ENET_Type *const *pEnet*)

Get the timestamp status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t the timestamp status

**LOCAL_INLINE bool ENET_HWA_MAC_GetTimestampTargetTimeError (const ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*)**

Indicates whether the timestamp target time error status of the PPS channel is detected.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	the channel number of PPS

Returns

true timestamp target time error is detected

false timestamp target time error is not detected

LOCAL_INLINE bool ENET_HWA_MAC_GetTimestampTargetTimeReachedStatus (const ENET_Type *const *pEnet*, ENET_MAC_PPSChannelType *ePPSChann*)

Indicates whether the timestamp target time reached for target time of the PPS channel.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	the channel number of PPS

Returns

true timestamp target time is reached

false timestamp target time is not reached

LOCAL_INLINE ENET_MAC_TimestampUpdateMethodType

ENET_HWA_MAC_GetTimestampUpdateMethod (const ENET_Type *const *pEnet*)

Get timestamp update method (fine or coarse)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_TimestampUpdateMethodType

LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetTimestampUpdateStatus (const ENET_Type *const *pEnet*)

Get timestamp update status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_StatusType

LOCAL_INLINE ENET_MAC_SystemTimeOperationType ENET_HWA_MAC_GetTimeUpdateOperation (const ENET_Type *const *pEnet*)

Get PTP system time update operation,add or subtract time.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_SystemTimeOperationType

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetTrailingClockCnt (const ENET_Type *const *pEnet*)

Get the number of trailing clock cycles.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

trailing clock cycles

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTx1024ToMaxOctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets transmitted with length 1024 to maxsize bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTx128To255OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets transmitted with length between 128 and 255 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTx256To511OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets transmitted with length between 256 and 511 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTx512To1023OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets transmitted with length between 512 and 1023 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTx64OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets transmitted with length 64 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTx65To127OctetPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets transmitted with length between 65 and 127 bytes.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxBroadcastPacketGood (const ENET_Type *const *pEnet*)

Get the number of good broadcast packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxBroadcastPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad broadcast packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxCarrierErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets with transmit carrier sense error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxDeferredPacketCnt (const ENET_Type *const *pEnet*)

Get the number of successfully transmitted packets after a deferral in half-duplex mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetTxEnFlag (const ENET_Type *const *pEnet*)

Get transmitter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Transmitter is enabled

false Transmitter is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxExcessiveCollisionPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets with transmit excessive collision errors.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetTxExcessiveCollisionStatus (const ENET_Type *const *pEnet*)

Get the excessive collision status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true excessive collision is sensed

false no collision

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxExcessiveDeferralErrorCnt (const ENET_Type *const *pEnet*)

Get the number of packets with transmit excessive deferral error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetTxExcessiveDeferralStatus (const ENET_Type *const *pEnet*)

Get the excessive deferral status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true excessive deferral is sensed

false no excessive deferral

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetTxInnerVLANTag (const ENET_Type *const *pEnet*)

Get inner VLAN tag value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Inner VLAN tag value

LOCAL_INLINE bool ENET_HWA_MAC_GetTxInterruptStatus (const ENET_Type *const *pEnet*)

Get the transmit interrupt status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

bool indicate the status of transmit packets

LOCAL_INLINE bool ENET_HWA_MAC_GetTxJabberTimeoutStatus (const ENET_Type *const *pEnet*)

Indicates whether a transmit jabber timer timeout occurred.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true transmit jabber timeout occurred
false no transmit jabber timeout occurred

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxLateCollisionPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets with transmit late collision error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetTxLateCollisionStatus (const ENET_Type *const *pEnet*)

Get the late collision status.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true late collision is sensed
false no collision

LOCAL_INLINE bool ENET_HWA_MAC_GetTxLossOfCarrierStatus (const ENET_Type *const *pEnet*)

Indicates the status of carrier signal.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true loss of carrier

false carrier is present

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxMulticastPacketGood (const ENET_Type *const *pEnet*)

Get the number of good multicast packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxMulticastPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad multicast packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxMultipleCollisionGoodPacketCnt (const ENET_Type *const *pEnet*)

Get the number of successfully transmitted packets after a multiple collision in half-duplex mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetTxNoCarrierStatus (const ENET_Type *const *pEnet*)

Indicates whether the carrier signal is present from the PHY at the end of preamble transmission.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true carrier is absent

false carrier is present

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxOctetCntGood (const ENET_Type *const *pEnet*)

Get the number of bytes of good packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxOctetCntGoodBad (const ENET_Type *const *pEnet*)

Get transmitted bytes of good and bad packets.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxOverSizePacketGood (const ENET_Type *const *pEnet*)

Get the number of transmitted packets without errors and with length greater than the maxsize.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxPacketCntGood (const ENET_Type *const *pEnet*)

Get the number of good packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE ENET_MAC_TxPacketControllerStatusType ENET_HWA_MAC_GetTxPacketControllerStatus (const ENET_Type *const *pEnet*)

Get MAC transmit packet controller status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_TxPacketControllerStatusType

LOCAL_INLINE bool ENET_HWA_MAC_GetTxPacketEngineStatus (const ENET_Type *const *pEnet*)

Get MAC GMII or MII transmit protocol engine status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true transmit protocol engine is active

false transmit protocol engine is inactive

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxPausePacketCnt (const ENET_Type *const *pEnet*)

Get the number of good pause packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxSingleCollisionGoodPacketCnt (const ENET_Type *const *pEnet*)

Get the number of successfully transmitted packets after a single collision in half-duplex mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetTxStatusInterruptEnFlag (const ENET_Type *const *pEnet*)

Get transmit status interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true transmit status interrupt is enabled

false transmit status interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetTxTimestampInterruptStatus (const ENET_Type *const *pEnet*)

Indicates whether the transmit timestamp interrupt status is detected.

Note

This function shall only be called when MAC_CSR_SW_CTRL[RCWE] = 1

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true transmit timestamp interrupt is detected

false transmit timestamp interrupt is not detected

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxTimestampStatusInNanoseconds (const ENET_Type *const *pEnet*)

Contains the nanosecond part of timestamp captured for transmit packets when tx status is disabled.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t transmit timestamp status in nanoseconds

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxTimestampStatusInSeconds (const ENET_Type *const *pEnet*)

Contains the second part of timestamp captured for transmit packets when tx status is disabled.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t transmit timestamp status in seconds

LOCAL_INLINE ENET_MAC_TxTimestampStatusModeType ENET_HWA_MAC_GetTxTimestampStatusMode (const ENET_Type *const *pEnet*)

Get transmit timestamp status mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MAC_TxTimestampStatusModeType

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxUnderflowErrorPacketCnt (const ENET_Type *const *pEnet*)

Get the number of packets with transmit underflow error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxUnicastPacketGoodBad (const ENET_Type *const *pEnet*)

Get the number of good and bad unicast packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetTxVLANPacketGood (const ENET_Type *const *pEnet*)

Get the number of good VLAN packets transmitted.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetTxVLANTag (const ENET_Type *const *pEnet*)

Get Tx VLAN tag value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Tx VLAN tag value

LOCAL_INLINE ENET_MTL_QueueType ENET_HWA_MAC_GetUnicastAddrFilterFailPacketQueue (const ENET_Type *const *pEnet*)

Get unicast address filter fail packets queue.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MTL_QueueType

LOCAL_INLINE bool ENET_HWA_MAC_GetUnicastAddrFilterFailPacketQueueEnFlag (const ENET_Type *const *pEnet*)

Get unicast address filter fail packets queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true unicast address filter fail packets queue is enabled

false unicast address filter fail packets queue is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetUnicastPacketGood (const ENET_Type *const *pEnet*)

Get the number of good unicast packets received.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

MMC counter value

LOCAL_INLINE bool ENET_HWA_MAC_GetUnicastPausePacketDetectEnFlag (const ENET_Type *const *pEnet*)

Get unicast pause packet detect enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Return values

<i>true</i>	Unicast pause packet detect is enabled
<i>false</i>	Unicast pause packet detect is disabled

LOCAL_INLINE ENET_MAC_SlowProtocolDetectType

ENET_HWA_MAC_GetUnicastSlowProtocolPacketDetectMethod (const ENET_Type *const *pEnet*)

Get unicast slow protocol packet detect method.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_SlowProtocolDetectType

LOCAL_INLINE ENET_MTL_QueueType ENET_HWA_MAC_GetUntaggedPacketQueue (const ENET_Type *const *pEnet*)

Get untagged packet queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MTL_QueueType

LOCAL_INLINE bool ENET_HWA_MAC_GetUpdateMmcCntForDroppedBroadcastPktEnFlag (const ENET_Type *const *pEnet*)

Get update MMC counters for dropped broadcast packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true update MMC counters for dropped broadcast packets

false do not update MMC counters for dropped broadcast packets

LOCAL_INLINE bool ENET_HWA_MAC_GetUseMACAddrForPTPPacketFilterEnFlag (const ENET_Type *const *pEnet*)

Get MAC address for PTP packet filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true MAC address for PTP packet filter is enabled

false MAC address for PTP packet filter is disabled

LOCAL_INLINE ENET_MAC_VLANCommandType ENET_HWA_MAC_GetVLANCommandType (const ENET_Type *const *pEnet*)

Get VLAN tag filter register command type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_VLANCommandType

LOCAL_INLINE uint16_t ENET_HWA_MAC_GetVLANHashTable (const ENET_Type *const *pEnet*)

Get VLAN hash table registers value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

VLAN hash tTable registers value

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetVLANInclusion (const ENET_Type *const *pEnet*)

Get VLAN tag inclusion register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

VLAN tag inclusion registers value

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetVLANInclusionAddr (const ENET_Type *const *pEnet*)

Get VLAN inclusion address.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

VLAN inclusion address value

**LOCAL_INLINE ENET_MAC_VLANInclCommandType ENET_HWA_MAC_GetVLANInclusionCommandType
(const ENET_Type *const *pEnet*)**

Set VLAN inclusion operation status.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

**LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetVLANInclusionOperationStatus (const
ENET_Type *const *pEnet*)**

Get VLAN inclusion operation status.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_StatusType

LOCAL_INLINE bool ENET_HWA_MAC_GetVLANInverseMatchEnFlag (const ENET_Type *const *pEnet*)

Get VLAN tag inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true VLAN tag inverse match is enabled

false VLAN tag inverse match is disabled

LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetVLANOperationStatus (const ENET_Type *const *pEnet*)

Get VLAN tag filter register operation status.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_StatusType

LOCAL_INLINE bool ENET_HWA_MAC_GetVLANPriorityCtrlEnFlag (const ENET_Type *const *pEnet*)

Get VLAN priority control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true VLAN priority control is enabled

false VLAN priority control is disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetVLANTagCtrl (const ENET_Type *const *pEnet*)

Get VLAN tag control register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

VLAN tag control register value

LOCAL_INLINE ENET_MAC_VLANTagCtrlType ENET_HWA_MAC_GetVLANTagCtrlOperation (const ENET_Type *const *pEnet*)

Get VLAN tag control mode in transmit packets.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_VLANTagCtrlType

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetVLANTagData (const ENET_Type *const *pEnet*)

Get VLAN tag data registers value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

VLAN tag data registers value

LOCAL_INLINE bool ENET_HWA_MAC_GetVLANTagFilterEnFlag (const ENET_Type *const *pEnet*)

Get VLAN tag filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Enable VLAN tag filter

false Disable VLAN tag filter

LOCAL_INLINE ENET_MTL_QueueType ENET_HWA_MAC_GetVLANTagFilterFailPacketQueue (const ENET_Type *const *pEnet*)

Get VLAN tag filter fail packets queue.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MTL_QueueType

LOCAL_INLINE bool ENET_HWA_MAC_GetVLANTagFilterFailPacketQueueEnFlag (const ENET_Type *const *pEnet*)

Get VLAN tag filter fail packets queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true VLAN tag filter fail packets queue is enabled

false VLAN tag filter fail packets queue is disabled

LOCAL_INLINE uint8_t ENET_HWA_MAC_GetVLANTagFilterRegOffset (const ENET_Type *const *pEnet*)

Get VLAN tag filter register address offset.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

VLAN tag filter register address offset

LOCAL_INLINE bool ENET_HWA_MAC_GetVLANTagHashTableMatchEnFlag (const ENET_Type *const *pEnet*)

Get VLAN tag hash table match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true VLAN tag hash table match is enabled

false VLAN tag hash table match is disabled

LOCAL_INLINE bool ENET_HWA_MAC_GetVLANTagInputEnFlag (const ENET_Type *const *pEnet*)

Get VLAN tag input enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true VLAN tag input is enabled

false VLAN tag input is disabled

LOCAL_INLINE ENET_MAC_VLANTagStripModeType ENET_HWA_MAC_GetVLANTagStripMode (const ENET_Type *const *pEnet*)

Get VLAN tag stripping mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_VLANTagStripModeType

LOCAL_INLINE ENET_MAC_VLANType ENET_HWA_MAC_GetVLANType (const ENET_Type *const *pEnet*)

Get VLAN type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_VLANType

LOCAL_INLINE bool ENET_HWA_MAC_GetVLANTypeCheckDisableFlag (const ENET_Type *const *pEnet*)

Get VLAN type check disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true VLAN type check is disabled

false VLAN type check is enabled

LOCAL_INLINE bool ENET_HWA_MAC_GetWatchdogDisableFlag (const ENET_Type *const *pEnet*)

Get receiver watchdog timer disabled flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Receiver watchdog timer is disabled

false Receiver watchdog timer is enabled

LOCAL_INLINE bool ENET_HWA_MAC_GetWatchdogEnFlag (const ENET_Type *const *pEnet*)

Get programmable watchdog enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

true Programmable watchdog enabled

false Programmable watchdog disabled

LOCAL_INLINE uint32_t ENET_HWA_MAC_GetWatchdogTimeout (const ENET_Type *const *pEnet*)

Get programmable watchdog timeout register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

Watchdog timeout register value

LOCAL_INLINE ENET_MAC_WDGTimerSizeType ENET_HWA_MAC_GetWatchdogTimerSize (const ENET_Type *const *pEnet*)

Get programmable watchdog timer size value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

Returns

ENET_MAC_WDGTimerSizeType

LOCAL_INLINE void ENET_HWA_MAC_InitiatePausePacketOrBackpressure (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Initiate flow control busy or backpressure of Queue n.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

LOCAL_INLINE void ENET_HWA_MAC_InitTimestamp (ENET_Type *const *pEnet*)

Initialize PTP timestamp.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_PresetMmcCounters (ENET_Type *const *pEnet*)

Set MMC counters preset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_ResetMmcCounters (ENET_Type *const *pEnet*)

Reset all MMC counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_Set12BitVLANTagEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set 12-Bit VLAN tag comparison enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_Set2KPacketSupportFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set 2K packet support enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetAddr0High (ENET_Type *const *pEnet*, uint32_t *u32AddrHigh*)

Set MAC address0 high register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>u32AddrHigh</i>	MAC address0 high register value
----	--------------------	----------------------------------

LOCAL_INLINE void ENET_HWA_MAC_SetAddr1High (ENET_Type *const *pEnet*, uint32_t *u32AddrHigh*)

Set MAC address1 high register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32AddrHigh</i>	MAC address1 high register value

LOCAL_INLINE void ENET_HWA_MAC_SetAddr2High (ENET_Type *const *pEnet*, uint32_t *u32AddrHigh*)

Set MAC address2 high register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32AddrHigh</i>	MAC address2 high register value

LOCAL_INLINE void ENET_HWA_MAC_SetAutoStripPadOrCRCEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set automatic Pad or CRC stripping enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetAV802_1ASModeEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set time stamp AV802_1AS mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>bEnable</i>	enable or disable
----	----------------	-------------------

**LOCAL_INLINE void ENET_HWA_MAC_SetAVCtrlPacketQueue (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Set AV untagged control packets queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	the queue id

**LOCAL_INLINE void ENET_HWA_MAC_SetBackOffLimit (ENET_Type *const *pEnet*,
ENET_MAC_BackOffLimitType *eLimit*)**

Set back-off limit.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eLimit</i>	Back-off limit type value

**LOCAL_INLINE void ENET_HWA_MAC_SetBackToBackTransactionEnFlag (ENET_Type *const *pEnet*, bool
bEnable)**

Set MDIO back to back transactions enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetBroadcastPacketDisableFlag (ENET_Type *const *pEnet*, bool
bDisable)**

Set broadcast packets disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetChannelBasedVLANTagInsertEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set channel based tag insertion enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetChannelFlexiblePPSOutputEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set PPS flexible mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	PPS flexible or fixed mode

LOCAL_INLINE void ENET_HWA_MAC_SetChannelPPSTargetTimeModeSelect (ENET_Type *const *pEnet*, ENET_MAC_PPSTChannelType *ePPSChann*, ENET_MAC_PPSTargetTimeModeType *eTrgtMode*)

Set PPS channel target time register operating mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>eTrgtMode</i>	target time operating mode

LOCAL_INLINE void ENET_HWA_MAC_SetConfiguration (ENET_Type *const *pEnet*, uint32_t *u32Config*)

Set the MAC Configuration Register value.

Parameters

in	<i>pEnet</i>	The base address of the ENET instance
in	<i>u32Config</i>	The Configuration Register value

**LOCAL_INLINE void ENET_HWA_MAC_SetCSRClockRange (ENET_Type *const *pEnet*,
ENET_MAC_CSRClockRangeType *eRange*)**

Set the MDC clock range.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eRange</i>	values defined in ENET_MAC_CSRClockRangeType

**LOCAL_INLINE void ENET_HWA_MAC_SetCtrlPacketFilterMode (ENET_Type *const *pEnet*,
ENET_MAC_CtrlPacketFilterModeType *eFilterMode*)**

Set control packets filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eFilterMode</i>	Control packets filter mode

LOCAL_INLINE void ENET_HWA_MAC_SetDeferralCheckEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set deferral check enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetDestAddrFilterMode (ENET_Type *const *pEnet*,
ENET_MAC_AddrFilterModeType *eFilterMode*)**

Set destination address filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
	<i>eFilterMode</i>	Destination address filter mode

LOCAL_INLINE void ENET_HWA_MAC_SetDoubleVLANProcessEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set double VLAN processing enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetDropNonTCPUDPOverIPEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set drop non-TCP/UDP over IP packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Drop or forward non-TCP/UDP over IP packets

LOCAL_INLINE void ENET_HWA_MAC_SetDuplexMode (ENET_Type *const *pEnet*, ENET_MAC_DuplexModeType *eMode*)

Set ethernet duplex mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eMode</i>	Ethernet duplex mode

LOCAL_INLINE void ENET_HWA_MAC_SetEthernetPTPPacketProcessEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set processing of PTP packets over ethernet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetEventMsgSanpshotEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set snapshot for event messages enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetExtConfiguration (ENET_Type *const *pEnet*, uint32_t *u32Configuration*)

Set the MAC extended configuration register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Configuration</i> <i>n</i>	The configuration register value

LOCAL_INLINE void ENET_HWA_MAC_SetExtInterPacketGap (ENET_Type *const *pEnet*, ENET_MAC_ExtInterPacketGapType *eGap*)

Set the MAC extended inter-packet gap.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eGap</i>	The MAC extended inter-packet gap value

LOCAL_INLINE void ENET_HWA_MAC_SetExtInterPacketGapEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set extended inter-packet gap enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetFixedPPSOutputFrequency (ENET_Type *const *pEnet*, ENET_MAC_PPSFreqCtrlType *eFreq*)

Set fixed PPS output frequency.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eFreq</i>	frequency value defined in ENET_MAC_PPSFreqCtrlType

LOCAL_INLINE void ENET_HWA_MAC_SetFlexiblePPSCommand (ENET_Type *const *pEnet*, ENET_MAC_PPSChannelType *ePPSChann*, ENET_MAC_PPSCmdType *eCmd*)

Set flexible PPS command.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>eCmd</i>	flexible PPS command

LOCAL_INLINE void ENET_HWA_MAC_SetFramePreemptionEnFlag (ENET_Type *const *pEnet*, uint8 *bEnable*)

Enable TX frame preemption.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	1U enable or 0U disable

LOCAL_INLINE void ENET_HWA_MAC_SetFullDuplexCRSEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set carrier sense before transmission enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetGiantPacketSizeLimit (ENET_Type *const *pEnet*, uint16_t *u16SizeLimit*)

Set giant packet size limit value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u16SizeLimit</i>	Giant packet size limit value

LOCAL_INLINE void ENET_HWA_MAC_SetGiantPacketSizeLimitCtrlEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set giant packet size Limit control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetGMIIIBusy (ENET_Type *const *pEnet*)

Set MDIO busy to initiate a read or write access.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_SetGMIIData (ENET_Type *const *pEnet*, uint16_t *u16Data*)

Set MDIO data.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u16Data</i>	MDIO data

LOCAL_INLINE void ENET_HWA_MAC_SetHalfDuplexCRSDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Set carrier sense during transmission disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetHashOrPerfectFilterEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set hash or perfect filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetHashOrPerfectMulticastMode (ENET_Type *const *pEnet*, ENET_MAC_HashOrPerfectModeType *eMode*)

Set hash or perfect multicast filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
	<i>eMode</i>	Hash or perfect multicast filter mode

**LOCAL_INLINE void ENET_HWA_MAC_SetHashOrPerfectUnicastMode (ENET_Type *const *pEnet*,
ENET_MAC_HashOrPerfectModeType *eMode*)**

Set hash or perfect unicast filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
	<i>eMode</i>	Hash or perfect unicast filter mode

**LOCAL_INLINE void ENET_HWA_MAC_SetHashTableFirst32Bits (ENET_Type *const *pEnet*, uint32_t
u32Hash0)**

Set the hash table first 32 bits value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
	<i>u32Hash0</i>	Hash table first 32 bits value

**LOCAL_INLINE void ENET_HWA_MAC_SetHashTableSecond32Bits (ENET_Type *const *pEnet*, uint32_t
u32Hash1)**

Set the hash table second 32 bits value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Hash1</i>	Hash table second 32 bits value

**LOCAL_INLINE void ENET_HWA_MAC_SetlimestamprolloverCtrl (ENET_Type *const *pEnet*,
ENET_MAC_TimestampRolloverType *eCtrl*)**

Set PTP timestamp rollover mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eCtrl</i>	timestamp rollover mode

LOCAL_INLINE void ENET_HWA_MAC_SetInnerVLANInclusion (ENET_Type *const *pEnet*, uint32_t *u32Inclusion*)

Set inner VLAN tag inclusion or replacement register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Inclusion</i>	Register value

LOCAL_INLINE void ENET_HWA_MAC_SetInnerVLANPriorityCtrlEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set inner VLAN priority control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetInnerVLANTagCmpEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set inner VLAN tag comparison enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetInnerVLANTagCtrl (ENET_Type *const *pEnet*, ENET_MAC_VLANTagCtrlType *eCtrl*)

Set inner VLAN tag control mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eCtrl</i>	Inner VLAN tag control mode

LOCAL_INLINE void ENET_HWA_MAC_SetInnerVLANTagInputEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set inner VLAN tag input enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetInnerVLANTagStripMode (ENET_Type *const *pEnet*, ENET_MAC_VLANTagStripModeType *eMode*)

Set inner VLAN tag stripping mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eMode</i>	Inner VLAN tag stripping mode

LOCAL_INLINE void ENET_HWA_MAC_SetInnerVLANType (ENET_Type *const *pEnet*, ENET_MAC_VLANType *eType*)

Set inner VLAN type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eType</i>	Inner VLAN type

LOCAL_INLINE void ENET_HWA_MAC_SetInterruptEnable (ENET_Type *const *pEnet*, uint32_t *u32IntEnable*)

Set the MAC interrupts enable register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32IntEnable</i>	interrupts enable value

LOCAL_INLINE void ENET_HWA_MAC_SetIPChecksumOffloadEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set checksum offload function enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetIPv4UDPPTPPacketProcessEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set processing of PTP packets over IPv4-UDP enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetIPv6UDPPTPPacketProcessEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set processing of PTP packets over IPv6-UDP enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>bEnable</i>	enable or disable
----	----------------	-------------------

LOCAL_INLINE void ENET_HWA_MAC_SetJabberTimerDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Set transmitter jabber timer disabled flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetJumboPacketEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set jumbo packet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Addr0 (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint32_t *u32Addr*)

Set layer 3 address0.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32Addr</i>	layer 3 address0 value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Addr1 (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint32_t *u32Addr*)

Set layer 3 address1.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32Addr</i>	layer 3 address1 value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Addr2 (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint32_t *u32Addr*)

Set layer 3 address2.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32Addr</i>	layer 3 address2 value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Addr3 (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint32_t *u32Addr*)

Set layer 3 address3.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32Addr</i>	layer 3 address3 value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3DestAddrInvMatchEnFlag (ENET_Type *const *pEnet*, uint8_t *u8Index*, bool *bEnable*)

Set layer 3 destination address inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3DestAddrMatchEnFlag (ENET_Type *const *pEnet*, uint8_t *u8Index*, bool *bEnable*)

Set layer 3 destination address match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3DestIPv4Addr (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint32_t *u32DestAddr*)

Set layer 3 destination IPv4 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32DestAddr</i>	layer 3 destination IPv4 address value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3IPv4DestAddrBitsMatch (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint8_t *u8MaskBits*)

Set layer3 IPv4 destination address bits match.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u8MaskBits</i>	lower u8MaskBits bits are masked

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3IPv4SrcAddrBitsMatch (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint8_t *u8MaskBits*)

Set layer3 IPv4 source address bits match.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u8MaskBits</i>	lower u8MaskBits bits are masked

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3IPv6Addr (ENET_Type *const *pEnet*, uint8_t *u8Index*, const uint8_t *aIPv6Addr*[16])

Set layer 3 IPv6 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>aIPv6Addr</i>	layer 3 IPv6 address value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3IPv6AddrBitsMatch (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint8_t *u8MaskBits*)

Set layer3 IPv6 address bits match.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u8MaskBits</i>	lower u8MaskBits bits are masked

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Layer4Ctrl (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint32_t *u32Ctrl*)

Set the layer 3 and layer 4 control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32Ctrl</i>	control value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Layer4DMAChannel (ENET_Type *const *pEnet*, uint8_t *u8Index*, ENET_DMA_ChannelType *eChannel*)

Set layer 3 and layer 4 control DMA channel number.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>eChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Layer4DMASelectEnFlag (ENET_Type *const *pEnet*, uint8_t *u8Index*, bool *bEnable*)

Set layer 3 and layer 4 control DMA channel select enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Layer4FilterEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set layer 3 and layer 4 filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer3ProtocolType (ENET_Type *const *pEnet*, uint8_t *u8Index*, ENET_MAC_Layer3ProtocolType *eProtocol*)

Set layer 3 protocol type (IPV4 or IPV6)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>eProtocol</i>	IPV4 or IPV6

**LOCAL_INLINE void ENET_HWA_MAC_SetLayer3SrcAddrInvMatchEnFlag (ENET_Type *const *pEnet*,
uint8_t *u8Index*, bool *bEnable*)**

Set layer 3 source address inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetLayer3SrcAddrMatchEnFlag (ENET_Type *const *pEnet*, uint8_t
u8Index, bool *bEnable*)**

Set layer 3 source address match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetLayer3SrcIPv4Addr (ENET_Type *const *pEnet*, uint8_t
u8Index, uint32_t *u32SrcAddr*)**

Set layer 3 source IPv4 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32SrcAddr</i>	layer 3 source IPv4 address value

LOCAL_INLINE void ENET_HWA_MAC_SetLayer4DestPort (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint16_t *u16DestPort*)

Set layer 4 destination port number.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u16DestPort</i>	port number

LOCAL_INLINE void ENET_HWA_MAC_SetLayer4DestPortInvMatchEnFlag (ENET_Type *const *pEnet*, uint8_t *u8Index*, bool *bEnable*)

Set layer 4 destination port inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer4DestPortMatchEnFlag (ENET_Type *const *pEnet*, uint8_t *u8Index*, bool *bEnable*)

Set layer 4 destination port match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer4ProtocolType (ENET_Type *const *pEnet*, uint8_t *u8Index*, ENET_MAC_Layer4ProtocolType *eProtocol*)

Set layer 4 protocol type (UDP or TCP)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>eProtocol</i>	protocol type (UDP or TCP)

LOCAL_INLINE void ENET_HWA_MAC_SetLayer4SrcPort (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint16_t *u16SrcPort*)

Set layer 4 source port number.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u16SrcPort</i>	port number

LOCAL_INLINE void ENET_HWA_MAC_SetLayer4SrcPortInvMatchEnFlag (ENET_Type *const *pEnet*, uint8_t *u8Index*, bool *bEnable*)

Set layer 4 source port inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLayer4SrcPortMatchEnFlag (ENET_Type *const *pEnet*, uint8_t *u8Index*, bool *bEnable*)

Set layer 4 source port match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>bEnable</i>	enabled or disable

LOCAL_INLINE void ENET_HWA_MAC_SetLoopbackModeEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set loopback mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr0DMAChannelSelect (ENET_Type *const *pEnet*, uint8_t *u8DMAChannelSel*)

Set MAC address0 DMA channel select.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8DMAChannelSel</i>	MAC address0 DMA channel select

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr0Low (ENET_Type *const *pEnet*, uint32_t *u32AddrLow*)

Set MAC address0 low register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32AddrLow</i>	MAC address0 low register value

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr1DMAChannelSelect (ENET_Type *const *pEnet*, uint8_t *u8DMAChannelSel*)

Set MAC address1 DMA channel select.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8DMAChannelSel</i>	MAC address1 DMA channel select

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr1EnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set MAC address1 enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr1Low (ENET_Type *const *pEnet*, uint32_t *u32AddrLow*)

Set MAC address1 low register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32AddrLow</i>	MAC address1 low register value

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr1MaskByteCtrl (ENET_Type *const *pEnet*, uint8_t *u8MaskCtrl*)

Set MAC address1 mask byte control.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8MaskCtrl</i>	MAC address1 mask byte value

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr1Type (ENET_Type *const *pEnet*, ENET_MAC_AddrType *eAddrType*)

Set MAC address1 compare mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eAddrType</i>	ENET_MAC_ADDR_TYPE_DEST or ENET_MAC_ADDR_TYPE_SRC

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr2DMAChannelSelect (ENET_Type *const *pEnet*, uint8_t *u8DMAChannelSel*)

Set MAC address2 DMA channel select.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8DMAChannelSel</i>	MAC address2 DMA channel select

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr2EnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set MAC address2 enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr2Low (ENET_Type *const *pEnet*, uint32_t *u32AddrLow*)

Set MAC address2 low register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32AddrLow</i>	MAC address2 low register value

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr2MaskByteCtrl (ENET_Type *const *pEnet*, uint8_t *u8MaskCtrl*)

Set MAC address2 mask byte control.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8MaskCtrl</i>	MAC address2 mask byte value

LOCAL_INLINE void ENET_HWA_MAC_SetMacAddr2Type (ENET_Type *const *pEnet*, ENET_MAC_AddrType *eAddrType*)

Set MAC address2 compare mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eAddrType</i>	ENET_MAC_ADDR_TYPE_DEST or ENET_MAC_ADDR_TYPE_SRC

LOCAL_INLINE void ENET_HWA_MAC_SetMasterMsgSanpshotEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set snapshot for messages relevant to master enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMDIOAddress (ENET_Type *const *pEnet*, uint32_t *u32MDIOAddr*)

Set MAC MDIO address register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32MDIOAddr</i>	MDIO address register value

LOCAL_INLINE void ENET_HWA_MAC_SetMDIOData (ENET_Type *const *pEnet*, uint32_t *u32Data*)

Set MDIO data register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Data</i>	data register value

LOCAL_INLINE void ENET_HWA_MAC_SetMDIOInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set MDIO interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMinInterPacketGap (ENET_Type *const *pEnet*, ENET_MAC_InterPacketGapType *eGap*)

Set the minimum Inter-Packet Gap value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eGap</i>	Inter-Packet Gap value

LOCAL_INLINE void ENET_HWA_MAC_SetMmcControl (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set MMC control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Ctrl</i>	MMC control register value

LOCAL_INLINE void ENET_HWA_MAC_SetMmcCounterFreezeStatus (ENET_Type *const *pEnet*, bool *bFreeze*)

Set MMC counters freeze.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bFreeze</i>	enable or disable freeze

LOCAL_INLINE void ENET_HWA_MAC_SetMmcCounterPresetLevel (ENET_Type *const *pEnet*, ENET_MAC_MmcCntPresetLevelType *eLevel*)

Set MMC counters full-half preset level.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eLevel</i>	ENET_MAC_MMC_COUNTER_PRESET_LEVEL_FULL or ENET_MAC_MMC_COUNTER_PRESET_LEVEL_HALF

LOCAL_INLINE void ENET_HWA_MAC_SetMmcCounterResetOnReadEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set MMC counters reset on read enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMmcCounterStopRolloverEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set MMC counters stop rollover enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMmcRxInterruptMask (ENET_Type *const *pEnet*, uint32_t *u32Mask*)

Set MMC interrupts mask of receive statistics counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Mask</i>	MMC interrupts mask value

LOCAL_INLINE void ENET_HWA_MAC_SetMmcTxInterruptMask (ENET_Type *const *pEnet*, uint32_t *u32Mask*)

Set MMC interrupts mask of transmit statistics counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Mask</i>	MMC interrupts mask value

LOCAL_INLINE void ENET_HWA_MAC_SetMulticastAddrFilterFailPacketQueue (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Set multicast address filter fail packets queue.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	queue id

LOCAL_INLINE void ENET_HWA_MAC_SetMulticastAddrFilterFailPacketQueueEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set multicast address filter fail packets queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetMulticastAndBroadcastQueue (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Set multicast and broadcast queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	the queue id

LOCAL_INLINE void ENET_HWA_MAC_SetMulticastAndBroadcastQueueEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set multicast and broadcast queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetPacketDuplicationCtrlEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set packet duplication control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

in	<i>bEnable</i>	Enable or disable
----	----------------	-------------------

LOCAL_INLINE void ENET_HWA_MAC_SetPacketFilter (ENET_Type *const *pEnet*, uint32_t *u32FilterCtrl*)

Set the MAC packet filter register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32FilterCtrl</i>	The filter control value

LOCAL_INLINE void ENET_HWA_MAC_SetPassAllMulticastEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set pass all the received multicast packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetPhyClauseVersion (ENET_Type *const *pEnet*, ENET_MAC_PhyClauseVersionType *eVersion*)

Set MDIO clause version.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eVersion</i>	ENET_MAC_PHY_CLAUSE_22 or ENET_MAC_PHY_CLAUSE_45

LOCAL_INLINE void ENET_HWA_MAC_SetPhyInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set phy interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetPhyOperationCommand (ENET_Type *const *pEnet*,
ENET_MAC_PhyOperationCmdType *eCmd*)**

Set phy operation command.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eCmd</i>	Commands defined in ENET_MAC_PhyOperationCmdType

**LOCAL_INLINE void ENET_HWA_MAC_SetPhyRegisterAddr (ENET_Type *const *pEnet*, uint16_t
u16Addr)**

Set phy register address.

Note

This field is valid only in clause 45 mode

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u16Addr</i>	phy register address

**LOCAL_INLINE void ENET_HWA_MAC_SetPhyRegisterOrDeviceAddr (ENET_Type *const *pEnet*, uint8_t
u8Addr)**

Set MDIO phy register(clause22) or MMD device(clause 45)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Addr</i>	phy register or device address

LOCAL_INLINE void ENET_HWA_MAC_SetPhysicalLayerAddr (ENET_Type *const *pEnet*, uint8_t *u8Addr*)

Set MDIO physical layer address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Addr</i>	phy address

LOCAL_INLINE void ENET_HWA_MAC_SetPPSControl (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set PPS control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Ctrl</i>	PPS control value

**LOCAL_INLINE void ENET_HWA_MAC_SetPPSInterval (ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*, uint32_t *u32Interval*)**

Set PPS pulse interval.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>u32Interval</i>	PPS pulse interval

**LOCAL_INLINE void ENET_HWA_MAC_SetPPSMCGRCommand (ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*, ENET_MAC_MCGRCmdType *eCmd*)**

Set PPS MCGR command.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>eCmd</i>	MCGR command

**LOCAL_INLINE void ENET_HWA_MAC_SetPPSMCGRModeEnFlag (ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*, bool *bEnable*)**

Set PPS MCGR Mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetPPSTargetTimeInNanoseconds (ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*, uint32_t *u32NanoSeconds*)**

Set PPS target time nanoseconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>u32NanoSecond s</i>	target time nanoseconds

**LOCAL_INLINE void ENET_HWA_MAC_SetPPSTargetTimeInSeconds (ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*, uint32_t *u32Seconds*)**

Set PPS target time seconds.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>u32Seconds</i>	target time seconds

**LOCAL_INLINE void ENET_HWA_MAC_SetPPSWidth (ENET_Type *const *pEnet*,
ENET_MAC_PPSChannelType *ePPSChann*, uint32_t *u32Width*)**

Set PPS pulse width.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>ePPSChann</i>	PPS channel
in	<i>u32Width</i>	PPS pulse width

LOCAL_INLINE void ENET_HWA_MAC_SetPreambleSuppressionEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set MDIO preamble suppression enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetPresentationTimeGenerationEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set presentation time generation enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetPresentationTimeUpdata (ENET_Type *const *pEnet*, uint32_t *u32NanoSeconds*)

Set presentation time update.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32NanoSecond</i> <i>s</i>	update time value

LOCAL_INLINE void ENET_HWA_MAC_SetPromiscuousModeEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set promiscuous mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetPTPoEQueue (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Set PTP over ethernet packet queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	the queue id

LOCAL_INLINE void ENET_HWA_MAC_SetPTPPacketVersion (ENET_Type *const *pEnet*, ENET_MAC_PTPVersionType *eVersion*)

Set PTP packets processing format version.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eVersion</i>	PTP version

LOCAL_INLINE void ENET_HWA_MAC_SetQnTxFlowCtrl (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, uint32_t *u32FlowCtrl*)

Set Queue n tx flow control register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

in	<i>u32FlowCtrl</i>	The register value
----	--------------------	--------------------

**LOCAL_INLINE void ENET_HWA_MAC_SetQnTxFlowCtrlEnableFlag (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, bool *bEnable*)**

Set Queue n tx flow control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetQnTxPauseLowThreshold (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, ENET_MAC_PauseThresholdType *ePauseThreshold*)**

Set Queue n tx pause low threshold.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>ePauseThreshol d</i>	Threshold value

**LOCAL_INLINE void ENET_HWA_MAC_SetQnTxPauseTime (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, uint16_t *u16PauseTime*)**

Set Queue n tx pause time value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u16PauseTime</i>	Pause time value

**LOCAL_INLINE void ENET_HWA_MAC_SetQnTxZeroQuantaPauseDisableFlag (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, bool *bDisable*)**

Set Queue n tx zero quanta pause disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetReceiveAllEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set receive all packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetReceiveOwnDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Set receive own disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetRetryDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Set transmission retry disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetRxCRCCheckDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Set CRC checking for rReceived packets disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetRxEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set receiver enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetRxFlowCtrl (ENET_Type *const *pEnet*, uint32_t *u32FlowCtrl*)

Set receive flow control register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32FlowCtrl</i>	Register value

LOCAL_INLINE void ENET_HWA_MAC_SetRxFlowCtrlEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set receive flow control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetRxQueueCtrl0 (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set Receive queue control 0 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

in	<i>u32Ctrl</i>	Register value
----	----------------	----------------

LOCAL_INLINE void ENET_HWA_MAC_SetRxQueueCtrl1 (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set Receive queue control 1 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Ctrl</i>	Register value

LOCAL_INLINE void ENET_HWA_MAC_SetRxQueueCtrl2 (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set Receive queue control 2 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Ctrl</i>	Register value

LOCAL_INLINE void ENET_HWA_MAC_SetRxQueueCtrl4 (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set Receive queue control 4 register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Ctrl</i>	Register value

**LOCAL_INLINE void ENET_HWA_MAC_SetRxQueueSelectedPriorities (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, uint8_t *u8SelPrio*)**

Set receive queue priorities.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	queue id

in	<i>u8SelPrio</i>	priorities value
----	------------------	------------------

LOCAL_INLINE void ENET_HWA_MAC_SetRxStatusInnerVLANTagEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set inner VLAN tag enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetRxStatusInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set receive status interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetRxStatusVLANTagEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set VLAN tag in rx status enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetRxSVLANMatchEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set Receive S-VLAN match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetSkipAddrPacketEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set skip address packet enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetSlowDetectEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set slow protocol detection enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetSnapshotPTPPacketType (ENET_Type *const *pEnet*, uint8_t *u8Type*)

Set PTP packets type for taking snapshots.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Type</i>	PTP packets type

LOCAL_INLINE void ENET_HWA_MAC_SetSourceAddressFilterEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set source address filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetSpeed (ENET_Type *const *pEnet*, ENET_MAC_SpeedType *eSpeed*)

Set ethernet line speed.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eSpeed</i>	Ethernet line speed

LOCAL_INLINE void ENET_HWA_MAC_SetSrcAddrFilterMode (ENET_Type *const *pEnet*, ENET_MAC_AddrFilterModeType *eFilterMode*)

Set source address filter mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eFilterMode</i>	Source address filter mode

LOCAL_INLINE void ENET_HWA_MAC_SetSrcAddrInsertReplaceCtrl (ENET_Type *const *pEnet*, ENET_MAC_SrcAddrCtrlType *eInsertReplaceCtrl*)

Set the source address insertion or replacement control type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eInsertReplaceCtrl</i>	source address insertion or replacement control type

LOCAL_INLINE void ENET_HWA_MAC_SetStripTypePacketCRCEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set CRC stripping for type packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetSubNanoSecondIncrementValue (ENET_Type *const *pEnet*,
uint8_t *u8Increment*)**

Set PTP sub-nanosecond increment value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Increment</i>	sub-nanosecond increment value

**LOCAL_INLINE void ENET_HWA_MAC_SetSubSecondIncrementReg (ENET_Type *const *pEnet*, uint32_t
u32Increment)**

Set PTP sub-second increment register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Increment</i>	increment value

**LOCAL_INLINE void ENET_HWA_MAC_SetSubSecondIncrementValue (ENET_Type *const *pEnet*, uint8_t
u8Increment)**

Set PTP sub-second increment value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Increment</i>	sub-second increment value

LOCAL_INLINE void ENET_HWA_MAC_SetSVLANEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set S-VLAN enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetSystemTimeHigherWordSeconds (ENET_Type *const *pEnet*,
uint16_t *u16Seconds*)**

Set PTP system time seconds bits[47:32] value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u16Seconds</i>	seconds bits[47:32] value

**LOCAL_INLINE void ENET_HWA_MAC_SetSystemTimeNanoSecondsUpdateReg (ENET_Type *const
pEnet, uint32_t *u32Reg*)**

Set PTP system time nanoseconds update register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Reg</i>	nanoseconds update register value

**LOCAL_INLINE void ENET_HWA_MAC_SetSystemTimeNanoSecondsUpdateValue (ENET_Type *const
pEnet, uint32_t *u32NanoSeconds*)**

Set PTP system time nanoseconds update value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32NanoSecond s</i>	nanoseconds update value

**LOCAL_INLINE void ENET_HWA_MAC_SetSystemTimeSecondsUpdateValue (ENET_Type *const *pEnet*,
uint32_t *u32Seconds*)**

Set PTP system time seconds update register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Seconds</i>	seconds update register value

**LOCAL_INLINE void ENET_HWA_MAC_SetTaggedAVCtrlPacketQueueEnFlag (ENET_Type *const *pEnet*,
bool *bEnable*)**

Set tagged AV control packets queuing enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetTaggedPTPoEQueueCtrl (ENET_Type *const *pEnet*,
ENET_MAC_PTPoEQueueCtrlType *eCtrl*)**

Set tagged PTP over ethernet packets queuing control.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eCtrl</i>	the control mode

**LOCAL_INLINE void ENET_HWA_MAC_SetTimestampAddend (ENET_Type *const *pEnet*, uint32_t
u32Time)**

Set PTP timestamp addend register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>u32Time</i>	timestamp addend value
----	----------------	------------------------

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampControl (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set time stamp control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Ctrl</i>	time stamp control value

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampEgressAsymmetryCorrection (ENET_Type *const *pEnet*, int32_t *s32Correction*)

Set timestamp egress asymmetry correction, used for PDelay_Req PTP messages.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>s32Correction</i>	correction value

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampEgressCorrectionInNanoseconds (ENET_Type *const *pEnet*, int32_t *s32NanoSec*)

Set timestamp egress correction in nanoseconds, used for timestamp in egress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>s32NanoSec</i>	correction value

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampEgressCorrectionInSubNanoseconds (ENET_Type *const *pEnet*, int8_t *s8SubNanoSec*)

Set timestamp egress correction in sub-nanoseconds, used for timestamp in egress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>s8SubNanoSec</i>	correction value

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set timestamp processing enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable timestamp processing

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampForAllPacketEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set timestamp for all packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampIngressAsymmetryCorrection (ENET_Type *const *pEnet*, int32_t *s32Correction*)

Set timestamp ingress asymmetry correction, used for PDelay_Resp PTP messages.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>s32Correction</i>	correction value

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampIngressCorrectionInNanoseconds (ENET_Type *const *pEnet*, int32_t *s32NanoSec*)

Set timestamp ingress correction in nanoseconds, used for timestamp in ingress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>s32NanoSec</i>	correction value

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampIngressCorrectionInSubNanoseconds (ENET_Type *const *pEnet*, int8_t *s8SubNanoSec*)

Set timestamp ingress correction in sub-nanoseconds, used for timestamp in ingress path.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>s8SubNanoSec</i>	correction value

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set timestamp interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetTimestampUpdateMethod (ENET_Type *const *pEnet*, ENET_MAC_TimestampUpdateMethodType *eMethod*)

Set timestamp update method (fine or coarse)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eMethod</i>	(fine update or coarse update)

LOCAL_INLINE void ENET_HWA_MAC_SetTimeUpdateOperation (ENET_Type *const *pEnet*, ENET_MAC_SystemTimeOperationType *eOperation*)

Set PTP system time update operation, add or subtract time.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eOperation</i>	add or subtract time

LOCAL_INLINE void ENET_HWA_MAC_SetTrailingClockCnt (ENET_Type *const *pEnet*, uint8_t *u8Cnt*)

Set MDIO trailing clock cycles.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Cnt</i>	trailing clock cycles(range 0~7)

LOCAL_INLINE void ENET_HWA_MAC_SetTxEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set transmitter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetTxInnerVLANTag (ENET_Type *const *pEnet*, uint16_t *u16VLANTag*)

Set inner VLAN tag value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u16VLANTag</i>	Inner VLAN tag value

LOCAL_INLINE void ENET_HWA_MAC_SetTxStatusInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set transmit status interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MAC_SetTxTimestampStatusMode (ENET_Type *const *pEnet*,
ENET_MAC_TxTimestampStatusModeType *eMode*)**

Set transmit timestamp status mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eMode</i>	transmit timestamp status mode

LOCAL_INLINE void ENET_HWA_MAC_SetTxVLANTag (ENET_Type *const *pEnet*, uint16_t *u16VLANTag*)

Set Tx VLAN tag value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u16VLANTag</i>	VLAN tag value

**LOCAL_INLINE void ENET_HWA_MAC_SetUnicastAddrFilterFailPacketQueue (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Set unicast address filter fail packets queue.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	queue id

**LOCAL_INLINE void ENET_HWA_MAC_SetUnicastAddrFilterFailPacketQueueEnFlag (ENET_Type *const
pEnet, bool *bEnable*)**

Set unicast address filter fail packets queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetUnicastPausePacketDetectEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set unicast pause packet detect enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetUnicastSlowProtocolPacketDetectMethod (ENET_Type *const *pEnet*, ENET_MAC_SlowProtocolDetectType *eMethod*)

Set unicast slow protocol packet detect method.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eMethod</i>	Unicast slow protocol packet detect method value

LOCAL_INLINE void ENET_HWA_MAC_SetUntaggedPacketQueue (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Set untagged packet queue id.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	the queue id

LOCAL_INLINE void ENET_HWA_MAC_SetUpdateMmcCntForDroppedBroadcastPktEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set update MMC counters for dropped broadcast packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetUseMACAddrForPTPPacketFilterEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set MAC address for PTP packet filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetVLANCommandType (ENET_Type *const *pEnet*, ENET_MAC_VLANCommandType *eCommand*)

Set VLAN tag filter register command type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eCommand</i>	Command type

LOCAL_INLINE void ENET_HWA_MAC_SetVLANHashTable (ENET_Type *const *pEnet*, uint16_t *u16HashTable*)

Set VLAN hash table registers value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u16HashTable</i>	VLAN hash table registers value

LOCAL_INLINE void ENET_HWA_MAC_SetVLANInclusion (ENET_Type *const *pEnet*, uint32_t *u32Inclusion*)

Set VLAN tag inclusion register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Inclusion</i>	VLAN tag inclusion register value

LOCAL_INLINE void ENET_HWA_MAC_SetVLANInclusionAddr (ENET_Type *const *pEnet*, uint8_t *u8Addr*)

Set VLAN inclusion address.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u8Addr</i>	VLAN tag inclusion address value

LOCAL_INLINE void ENET_HWA_MAC_SetVLANInclusionCommandType (ENET_Type *const *pEnet*, ENET_MAC_VLANInclCommandType *eCommand*)

Set VLAN tag inclusion command.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eCommand</i>	VLAN tag inclusion command

LOCAL_INLINE void ENET_HWA_MAC_SetVLANInverseMatchEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set VLAN tag inverse match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetVLANOperationBusy (ENET_Type *const *pEnet*)

Set VLAN tag filter register operation busy status.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

LOCAL_INLINE void ENET_HWA_MAC_SetVLANPriorityCtrlEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set VLAN tag input enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagCtrl (ENET_Type *const *pEnet*, uint32_t *u32VlanCtrl*)

Set VLAN tag control register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32VlanCtrl</i>	Register value

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagCtrlOperation (ENET_Type *const *pEnet*, ENET_MAC_VLANTagCtrlType *eCtrl*)

Set VLAN tag control mode in transmit packets.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eCtrl</i>	VLAN tag control mode

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagData (ENET_Type *const *pEnet*, uint32_t *u32VLANData*)

Set VLAN tag data registers value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32VLANData</i>	Data value

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagFilterFailPacketQueue (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Set VLAN tag filter fail packets queue.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	queue id

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagFilterFailPacketQueueEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set VLAN tag filter fail packets queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagFilterFilterEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set VLAN tag filter enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
----	--------------	---

in	<i>bEnable</i>	Enable or disable
----	----------------	-------------------

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagFilterRegOffset (ENET_Type *const *pEnet*, uint8_t *u8Offset*)

Set VLAN tag filter register address offset.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u8Offset</i>	VLAN tag filter register address offset

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagHashTableMatchEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set VLAN tag hash table match enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagInputEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set VLAN tag input enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagStripMode (ENET_Type *const *pEnet*, ENET_MAC_VLANTagStripModeType *eMode*)

Set VLAN tag stripping mode.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eMode</i>	VLAN tag stripping mode

LOCAL_INLINE void ENET_HWA_MAC_SetVLANType (ENET_Type *const *pEnet*, ENET_MAC_VLANType *eType*)

Set VLAN type.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eType</i>	VLAN type

LOCAL_INLINE void ENET_HWA_MAC_SetVLANTypeCheckDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Set VLAN type check disable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetWatchdogDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Set receiver watchdog timer disabled flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bDisable</i>	Disable or enable

LOCAL_INLINE void ENET_HWA_MAC_SetWatchdogEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set programmable watchdog enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>bEnable</i>	Enable or disable

LOCAL_INLINE void ENET_HWA_MAC_SetWatchdogTimeout (ENET_Type *const *pEnet*, uint32_t *u32Settings*)

Set programmable watchdog timeout register value.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>u32Settings</i>	Watchdog timeout register value

LOCAL_INLINE void ENET_HWA_MAC_SetWatchdogTimerSize (ENET_Type *const *pEnet*, ENET_MAC_WDGTimerSizeType *eSize*)

Set programmable watchdog timer size.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eSize</i>	Watchdog timer size

LOCAL_INLINE void ENET_HWA_MAC_UpdateAddendRegister (ENET_Type *const *pEnet*)

Initiate the addend register update operation.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MAC_UpdateTimestamp (ENET_Type *const *pEnet*)

Initiate the timestamp update operation.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

**LOCAL_INLINE void ENET_HWA_MTL_ClearAllIntStatus (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Clear all the interrupts of the MTL queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

LOCAL_INLINE void ENET_HWA_MTL_ClearBTRErrorFlag (ENET_Type *const *pEnet*)

Clear BTR error flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MTL_ClearConstantGateControlErrorFlag (ENET_Type *const *pEnet*)

Clear constant gate control error flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MTL_ClearFrameSizeErrorQueueNumber (ENET_Type *const *pEnet*)

Clear the value of frame size error queue number register.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MTL_ClearRxPacketAvailableIntStatus (ENET_Type *const *pEnet*)

Clear MTL receive packet available interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

**LOCAL_INLINE void ENET_HWA_MTL_ClearRxQueueOverflowIntStatus (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Clear receive queue overflow interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

LOCAL_INLINE void ENET_HWA_MTL_ClearScheduleErrorQueueNumber (ENET_Type *const *pEnet*)

Clear the value of schedule error queue number register.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

**LOCAL_INLINE void ENET_HWA_MTL_ClearSlotAverageBitsIntStatus (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Clear average bits per slot interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

LOCAL_INLINE void ENET_HWA_MTL_ClearSwitchGateControlListCompleteFlag (ENET_Type *const *pEnet*)

Clear switch gate control list complete status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MTL_ClearTxQueueUnderflowIntStatus (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Clear transmit queue underflow interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

LOCAL_INLINE void ENET_HWA_MTL_ClearTxStatusAvailableIntStatus (ENET_Type *const *pEnet*)

Clear MTL transmit status available interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MTL_FlushTxQueue (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Flush transmit queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

LOCAL_INLINE bool ENET_HWA_MTL_GetAllFIFOPointersResetStatus (const ENET_Type *const *pEnet*)

Get pointers of all FIFOs reset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true all FIFOs reset is enabled

false all FIFOs reset is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetBaseTimeRegisterErrorInterruptEnFlag (ENET_Type *const *pEnet*)

Get enable flag of base time register error interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true BTR Error interrupt is enabled

false BTR Error interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetBTRErrorFlag (const ENET_Type *const *pEnet*)

Get BTR error flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true BTR error detected

false BTR error not detected

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetBTRErrorLoopCount (const ENET_Type *const *pEnet*)

Get BTR error loop count.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint8_t BTR error loop count

LOCAL_INLINE ENET_MTL_ByteEnableType ENET_HWA_MTL_GetByteEnable (const ENET_Type *const *pEnet*)

Get the number of data bytes valid in the data register during Read operation.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_ByteEnableType

LOCAL_INLINE bool ENET_HWA_MTL_GetConstantGateControlErrorFlag (const ENET_Type *const *pEnet*)

Get constant gate control error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true constant gate control error detected

false constant gate control error not detected

LOCAL_INLINE bool ENET_HWA_MTL_GetConstantGateControlErrorInterruptEnFlag (ENET_Type *const *pEnet*)

Get enable flag of constant gate control error interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true CGCE interrupt is enabled

false CGCE interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetCounterResetStatus (const ENET_Type *const *pEnet*)

Get MTL counter reset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t counter reset status

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetCurrentGCLSlotNumber (const ENET_Type *const *pEnet*)

Get current GCL slot number.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint8_t current GCL slot number

LOCAL_INLINE ENET_MTL_DebugAccessFIFOType ENET_HWA_MTL_GetDebugAccessFIFO (const ENET_Type *const *pEnet*)

Get the FIFO selected for debug access.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_DebugAccessFIFOType

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetDebugCtrl (const ENET_Type *const *pEnet*)

Get MTL debug control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t MTL debug control value

LOCAL_INLINE bool ENET_HWA_MTL_GetDebugIntStatus (const ENET_Type *const *pEnet*)

Get debug interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Debug interrupt status detected

false Debug interrupt status not detected

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetDebugStatus (const ENET_Type *const *pEnet*)

Get MTL debug status register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t debug status

LOCAL_INLINE bool ENET_HWA_MTL_GetDropFrameSizeErrorFramesDisableFlag (const ENET_Type *const *pEnet*)

Drop frames during frame size error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Do not drop frames during frame size error

false Drop frames during frame size error

LOCAL_INLINE bool ENET_HWA_MTL_GetDropSchedulingErrorFramesEnFlag (const ENET_Type *const *pEnet*)

Drop frames causing scheduling error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Drop frames causing scheduling error

false Do not drop frames causing scheduling error

LOCAL_INLINE bool ENET_HWA_MTL_GetDropTxPacketStatusEnFlag (const ENET_Type *const *pEnet*)

Get whether tx packet status received from the MAC is dropped or not.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true tx packet status is dropped

false tx packet status is forwarded to application

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetEstControl (const ENET_Type *const *pEnet*)

Get enhancements to scheduled transmission control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t EST control value

LOCAL_INLINE uint16_t ENET_HWA_MTL_GetEstCurrentTimeOffset (const ENET_Type *const *pEnet*)

Get enhancements to scheduled transmission current time offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint16_t current time offset

LOCAL_INLINE bool ENET_HWA_MTL_GetEstEccInjectAddressErrorEnFlag (ENET_Type *const *pEnet*)

Get enable flag of EST ECC address error injection.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true inject 2 bit errors

false inject 1 bit error

LOCAL_INLINE bool ENET_HWA_MTL_GetEstEccInjectErrorControl (ENET_Type *const *pEnet*)

Get the mode of EST ECC inject error control for EST memory.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true inject 2 bit errors

false inject 1 bit error

LOCAL_INLINE bool ENET_HWA_MTL_GetEstEccInjectErrorEnFlag (ENET_Type *const *pEnet*)

Get enable flag of EST ECC error injection.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true EST ECC inject error is enabled

false EST ECC inject error is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetEstEnableFlag (const ENET_Type *const *pEnet*)

Get EST enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true EST is enabled

false EST is disabled

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetEstGateControlListAddress (ENET_Type *const *pEnet*)

Get GCL address or GCL related registers address depend on GCRR bit.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint8_t GCL address or GCL related registers address

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetEstOverheadBytes (const ENET_Type *const *pEnet*)

Get EST overhead bytes value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint8_t EST overhead bytes value

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetEstPtpTimeOffset (const ENET_Type *const *pEnet*)

Get enhancements to scheduled transmission PTP time offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint8_t PTP time offset

LOCAL_INLINE ENET_MTL_ScheduleErrorLoopCountType

ENET_HWA_MTL_GetEstScheduleErrorLoopCount (const ENET_Type *const *pEnet*)

Get the loop count to report scheduling error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_ScheduleErrorLoopCountType

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetEstStatus (const ENET_Type *const *pEnet*)

Get EST status register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t EST status register value

LOCAL_INLINE ENET_MTL_GCLIntervalMultipleType ENET_HWA_MTL_GetEstTimeIntervalLeftShift (const ENET_Type *const *pEnet*)

Get EST time interval left shift amount.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_GCLIntervalMultipleType

LOCAL_INLINE bool ENET_HWA_MTL_GetFIFOBusy (const ENET_Type *const *pEnet*)

Get MTL FIFO busy status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true FIFO is busy

false FIFO is not busy

LOCAL_INLINE bool ENET_HWA_MTL_GetFIFODebugAccessEnFlag (const ENET_Type *const *pEnet*)

Get FIFO debug access enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true the debug mode access to the FIFO is enabled

false the debug mode access to the FIFO is disabled

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetFIFODebugData (const ENET_Type *const *pEnet*)

Get MTL FIFO debug data.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t FIFO debug data

LOCAL_INLINE ENET_MTL_FIFODebugAccessType ENET_HWA_MTL_GetFIFODebugModeAccessType (const ENET_Type *const *pEnet*)

Get pointers of all FIFOs reset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true all FIFOs reset is enabled

false all FIFOs reset is disabled

LOCAL_INLINE ENET_MTL_FIFOPacketStatusType ENET_HWA_MTL_GetFIFOPacketState (const ENET_Type *const *pEnet*)

Get the control or status information of the selected FIFO.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_FIFOPacketStatusType

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetFIFORemainingLocation (const ENET_Type *const *pEnet*)

Get MTL remaining locations in the FIFO.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t

LOCAL_INLINE bool ENET_HWA_MTL_GetFIFOWriteEnFlag (const ENET_Type *const *pEnet*)

Get write operation on selected FIFO enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true write operation on selected FIFO is enabled

false write operation on selected FIFO is disabled

**LOCAL_INLINE bool ENET_HWA_MTL_GetForwardErrorPacketEnFlag (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get forward error packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true forward error packets to application

false drop error packets

**LOCAL_INLINE bool ENET_HWA_MTL_GetForwardTCPChecksumErrorPacketEnFlag (const ENET_Type
*const *pEnet*, ENET_MTL_QueueType *eQueue*)**

Get forwarding of TCP/IP checksum error packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true forward TCP/IP checksum error packets to application

false drop TCP/IP checksum error packets

LOCAL_INLINE bool ENET_HWA_MTL_GetForwardUndersizedPacketEnFlag (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get forward undersized good packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true forward undersized good packets to application

false drop undersized good packets

LOCAL_INLINE uint16_t ENET_HWA_MTL_GetFrameSizeErrorFirstFrameSize (ENET_Type *const *pEnet*)

Get the frame size of the first frame size error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint16_t frame size of the first frame size error

LOCAL_INLINE bool ENET_HWA_MTL_GetFrameSizeErrorFirstQueueNumber (const ENET_Type *const *pEnet*)

Get the queue numbers that are having the first frame size error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

- true queue 1 caused the first frame size error
- false queue 0 caused the first frame size error

LOCAL_INLINE bool ENET_HWA_MTL_GetFrameSizeErrorQueueNumber (const ENET_Type *const *pEnet*)

Get the One Hot encoded queue numbers that are having the frame size error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

- true queue 1 caused frame size error
- false queue 0 caused frame size error

LOCAL_INLINE bool ENET_HWA_MTL_GetGateControlReleatedRegistersEnFlag (ENET_Type *const *pEnet*)

Get gate control releated registers enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

- true ADDR fields is used for gate control releated registers
- false ADDR fields is used for gate control list address

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetGclData (ENET_Type *const *pEnet*)

Get GCL data.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

- uint32_t GCL data value

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetGclDebugModeBankSelect (ENET_Type *const *pEnet*)

Get GCL debug mode bank selected.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint8_t debug mode bank

LOCAL_INLINE bool ENET_HWA_MTL_GetGclDebugModeEnFlag (ENET_Type *const *pEnet*)

Get GCL debug mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true debug mode is enabled

false debug mode is disabled

LOCAL_INLINE ENET_MTL_GclOperationType ENET_HWA_MTL_GetGclOperation (ENET_Type *const *pEnet*)

Get GCL operation mode (read or write)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_GclOperationType (read or write)

LOCAL_INLINE bool ENET_HWA_MTL_GetGclOperationBusyState (ENET_Type *const *pEnet*)

Get GCL operation busy status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Start Read/Write operation is enabled,status is busy

false Start Read/Write operation is disabled,status is idle

LOCAL_INLINE bool ENET_HWA_MTL_GetHeadOfLineBlockingDueToFrameSizeFlag (const ENET_Type *const *pEnet*)

Get Head-Of-Line blocking due to frame size.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Head-Of-Line blocking due to frame size detected

false Head-Of-Line blocking due to frame size not detected

LOCAL_INLINE bool ENET_HWA_MTL_GetHeadofLineBlockingDueToFrameSizeInterruptEnFlag (ENET_Type *const *pEnet*)

Get enable flag of head of line blocking due to frame size interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true HLBF interrupt is enabled

false HLBF interrupt is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetHeadOfLineBlockingDueToSchedulingFlag (const ENET_Type *const *pEnet*)

Get Head-Of-Line blocking due to scheduling.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Head-Of-Line blocking due to scheduling detected

false Head-Of-Line blocking due to scheduling not detected

LOCAL_INLINE bool ENET_HWA_MTL_GetHeadofLineBlockingDueToSchedulingInterruptEnFlag
(ENET_Type *const *pEnet*)

Get enable flag of head of line blocking due to scheduling interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true HLBS interrupt is enabled

false HLBS interrupt is disabled

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetInterruptCtrlStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)

Get the interrupt enable and status for the queue interrupts.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t interrupt enable and status

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetIntStatus (const ENET_Type *const *pEnet*)

Get MTL interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t MTL interrupt status

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetLaunchExpiryOffset (const ENET_Type *const *pEnet*)

Get launch expiry offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t launch expiry offset

LOCAL_INLINE bool ENET_HWA_MTL_GetLaunchExpiryOffsetEnFlag (const ENET_Type *const *pEnet*)

Get launch expiry offset enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true launch expiry offset is valid

false launch expiry offset is invalid

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetOperationMode (const ENET_Type *const *pEnet*)

Get MTL operation mode register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t MTL operation mode

LOCAL_INLINE bool ENET_HWA_MTL_GetPresetCounterStatus (const ENET_Type *const *pEnet*)

Get MTL counter preset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t counter preset status

LOCAL_INLINE bool ENET_HWA_MTL_GetQueue0IntStatus (const ENET_Type *const *pEnet*)

Get Queue 0 interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Queue 0 interrupt status detected

false Queue 0 interrupt status not detected

LOCAL_INLINE bool ENET_HWA_MTL_GetQueue1IntStatus (const ENET_Type *const *pEnet*)

Get Queue 1 interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Queue 1 interrupt status detected

false Queue 1 interrupt status not detected

LOCAL_INLINE ENET_MTL_RxFlowCtrlThresholdType ENET_HWA_MTL_GetRxActivateFlowCtrlThres (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get the threshold of receive queue at which the flow control is activated.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_RxFlowCtrlThresholdType flow control activating threshold

LOCAL_INLINE ENET_MTL_RxArbitrationAlgorithmType ENET_HWA_MTL_GetRxArbitrationAlgorithm
(const ENET_Type *const *pEnet*)

Get MTL algorithm for receive arbitration.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_RxArbitrationAlgorithmType

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetRxControl (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)

Get MTL queue receive control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t receive control value

LOCAL_INLINE bool ENET_HWA_MTL_GetRxDABasedDMAChannelSelectionEnFlag (const ENET_Type
const *pEnet*, ENET_MTL_QueueType *eQueue)

Get DA-based DMA channel selection enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true eQueue is enabled for DA-based DMA channel selection

false eQueue is disabled for DA-based DMA channel selection

LOCAL_INLINE ENET_MTL_RxFlowCtrlThresholdType ENET_HWA_MTL_GetRxDeactivateFlowCtrlThres
(const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get the threshold of receive queue at which the flow control is deactivated.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_RxFlowCtrlThresholdType flow control deactivating threshold

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetRxDebugStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)

Get the receive queue debug status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t receive queue debug status

LOCAL_INLINE ENET_DMA_ChannelType ENET_HWA_MTL_GetRxDMAChannelMap (const ENET_Type
***const *pEnet*, ENET_MTL_QueueType *eQueue*)**

Get DMA channel mapped to eQueue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_DMA_ChannelType

LOCAL_INLINE ENET_MTL_RxQueueStatusType ENET_HWA_MTL_GetRxFillLevelStatus (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get receive queue Fill-Level status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_RxQueueStatusType (empty,below flow-control deactivate threshold,above flow-control activate threshold,full)

LOCAL_INLINE bool ENET_HWA_MTL_GetRxHwFlowCtrlEnFlag (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get receive queue hardware flow control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true receive queue hardware flow control is enabled

false receive queue hardware flow control is disabled

LOCAL_INLINE void ENET_HWA_MTL_GetRxMissedPacketOverflowCount (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, bool * *pMissedPacketCntOverflowFlag*, uint16_t * *pMissedPacketCnt*, bool * *pOverflowPacketCntOverflowFlag*, uint16_t * *pOverflowPacketCnt*)

Get the missed packet and overflow counters of the queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
out	<i>pMissedPacketCntOverflowFlag</i>	missed packet counter overflow detected
out	<i>pMissedPacketCnt</i>	missed pPacket counter
out	<i>pOverflowPacketCntOverflowFlag</i>	overflow packet counter overflow detected
out	<i>pOverflowPacketCnt</i>	overflow packet counter

```
LOCAL_INLINE uint32_t ENET_HWA_MTL_GetRxOperationMode (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get receive operation mode of the queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t receive operation mode

```
LOCAL_INLINE bool ENET_HWA_MTL_GetRxPacketAvailableIntEnFlag (const ENET_Type *const pEnet)
```

Get receive packet available interrupt status enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true receive packet available interrupt status is enabled

false receive packet available interrupt status is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetRxPacketAvailableIntStatus (const ENET_Type *const *pEnet*)

Get MTL receive packet available interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true receive packet available interrupt not detected

false receive packet available interrupt detected

**LOCAL_INLINE uint8_t ENET_HWA_MTL_GetRxPacketCount (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get the number of packets in receive queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint8_t packets number

**LOCAL_INLINE ENET_MTL_QueueEnableType ENET_HWA_MTL_GetRxQueueEnStatus (const ENET_Type
*const *pEnet*, ENET_MTL_QueueType *eQueue*)**

Get Receive queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	the queue number

Returns

ENET_MTL_QueueEnableType

**LOCAL_INLINE bool ENET_HWA_MTL_GetRxQueueOverflowIntEnFlag (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get receive queue overflow interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true receive queue overflow interrupt is enabled

false receive queue overflow interrupt is disabled

**LOCAL_INLINE bool ENET_HWA_MTL_GetRxQueueOverflowIntStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get receive queue overflow interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true receive queue overflow interrupt detected

false receive queue overflow interrupt not detected

LOCAL_INLINE bool ENET_HWA_MTL_GetRxQueuePacketArbitrationEnFlag (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get receive queue packet arbitration enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true receive queue packet arbitration is enabled

false receive queue packet arbitration is disabled

LOCAL_INLINE ENET_MTL_ReadControllerStatusType ENET_HWA_MTL_GetRxQueueReadControllerStatus (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get receive queue read controller state.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_ReadControllerStatusType

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetRxQueueSize (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get receive queue size.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint8_t receive queues size in blocks of 256 bytes

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetRxQueueWeight (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get receive queue weight.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint8_t receive queue weight

LOCAL_INLINE ENET_MTL_WriteControllerStatusType

ENET_HWA_MTL_GetRxQueueWriteControllerStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)

Get receive queue write controller state.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_WriteControllerStatusType

LOCAL_INLINE bool ENET_HWA_MTL_GetRxStoreForwardModeEnFlag (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)

Get receive queue store and forward enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true store and forward is enabled

false store and forward is disabled

LOCAL_INLINE ENET_MTL_RxThresholdType ENET_HWA_MTL_GetRxThreshold (const ENET_Type *const
pEnet, ENET_MTL_QueueType *eQueue*)

Get the receive queue threshold control.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>eQueue</i>	MTL queue id
----	---------------	--------------

Returns

ENET_MTL_RxThresholdType

LOCAL_INLINE bool ENET_HWA_MTL_GetScheduleErrorQueueNumber (const ENET_Type *const *pEnet*)

Get the One Hot encoded queue numbers that are having the scheduling related error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true queue 1 caused scheduling error

false queue 0 caused scheduling error

LOCAL_INLINE bool ENET_HWA_MTL_GetSelectedFIFOPointerResetStatus (const ENET_Type *const *pEnet*)

Get pointers of selected FIFO reset status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true FIFO reset is enabled

false FIFO reset is disabled

**LOCAL_INLINE uint32_t ENET_HWA_MTL_GetSlotAverageBits (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get the average bits per slot,when the DCB operation is enabled for the queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t average bits per slot

**LOCAL_INLINE bool ENET_HWA_MTL_GetSlotAverageBitsIntEnFlag (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get average bits per slot interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true average bits per slot interrupt is enabled

false average bits per slot interrupt is disabled

**LOCAL_INLINE bool ENET_HWA_MTL_GetSlotAverageBitsIntStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get average bits per slot interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true average bits per slot interrupt detected

false average bits per slot interrupt not detected

**LOCAL_INLINE ENET_MTL_GateControlListType ENET_HWA_MTL_GetSoftwareOwnedList (const
ENET_Type *const *pEnet*)**

Get the gate control list owned by software.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_GateControlListType

LOCAL_INLINE bool ENET_HWA_MTL_GetSwitchGateControlListCompleteFlag (const ENET_Type *const *pEnet*)

Get switch gate control list complete status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true Switching to S/W owned list completed

false Switching to S/W owned list not completed

LOCAL_INLINE bool ENET_HWA_MTL_GetSwitchGateControlListEnFlag (const ENET_Type *const *pEnet*)

Get switch gate control list enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true switch gate control list is enabled

false switch gate control list is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetSwitchListInterruptEnFlag (ENET_Type *const *pEnet*)

Get enable flag of switch GCL completion interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true BTR Error interrupt is enabled

false BTR Error interrupt is disabled

LOCAL_INLINE uint32_t ENET_HWA_MTL_GetTBSCtrl (const ENET_Type *const *pEnet*)

Get Time Based Scheduling control register.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

uint32_t Time Based Scheduling control

**LOCAL_INLINE uint32_t ENET_HWA_MTL_GetTxDebugStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get transmit queue debug status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t transmit queue debug status register value

**LOCAL_INLINE bool ENET_HWA_MTL_GetTxFlushState (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get transmit queue flush state.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true flush transmit queue is enabled

false flush transmit queue is disabled

```
LOCAL_INLINE uint32_t ENET_HWA_MTL_GetTxOperationMode (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)
```

Get MTL queue Tx operation mode register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t MTL queue Tx operation mode

```
LOCAL_INLINE uint8_t ENET_HWA_MTL_GetTxPacketCount (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)
```

Get the current number of packets in the transmit queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint8_t number of packets

```
LOCAL_INLINE ENET_MTL_QueueEnableType ENET_HWA_MTL_GetTxQueueEnStatus (const ENET_Type
*const pEnet, ENET_MTL_QueueType eQueue)
```

Get the transmit queue enable mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_QueueEnableType queue enable mode

**LOCAL_INLINE bool ENET_HWA_MTL_GetTxQueueNotEmptyStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get the not empty flag of transmit queue data.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true transmit queue data is empty

false transmit queue data is not empty

**LOCAL_INLINE bool ENET_HWA_MTL_GetTxQueuePausedStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get the transmit queue paused status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true transmit queue is paused

false transmit queue is not paused

**LOCAL_INLINE uint32_t ENET_HWA_MTL_GetTxQueueQuantumOrWeight (const ENET_Type *const
pEnet, ENET_MTL_QueueType *eQueue*)**

Get the quantum or weights for transmit algorithm (DWRR WRR WFQ)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint32_t quantum or weights

LOCAL_INLINE ENET_MTL_ReadControllerStatusType ENET_HWA_MTL_GetTxQueueReadControllerStatus (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get the transmit queue read controller status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_ReadControllerStatusType

LOCAL_INLINE uint8_t ENET_HWA_MTL_GetTxQueueSize (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get MTL transmit queue size.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint8_t queue size in blocks of 256 bytes

LOCAL_INLINE bool ENET_HWA_MTL_GetTxQueueUnderflowIntEnFlag (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*)

Get transmit queue underflow interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Return values

<i>true</i>	transmit queue underflow interrupt is enabled
<i>false</i>	transmit queue underflow interrupt is disabled

**LOCAL_INLINE bool ENET_HWA_MTL_GetTxQueueUnderflowIntStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get transmit queue underflow interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true transmit queue underflow interrupt detected

false transmit queue underflow interrupt not detected

**LOCAL_INLINE ENET_MTL_WriteControllerStatusType
ENET_HWA_MTL_GetTxQueueWriteControllerStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get the transmit queue write controller status, indicates that the MTL Tx Queue Write controller is active, and it is transferring the data to the transmit queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

ENET_MTL_WriteControllerStatusType

**LOCAL_INLINE ENET_MTL_TxScheduleAlgorithmType ENET_HWA_MTL_GetTxSchedulingAlgorithm (const
ENET_Type *const *pEnet*)**

Get MTL algorithm for transmit scheduling.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

ENET_MTL_TxScheduleAlgorithmType

LOCAL_INLINE bool ENET_HWA_MTL_GetTxStatusAvailableIntEnFlag (const ENET_Type *const *pEnet*)

Get transmit status available interrupt status enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true transmit status available interrupt status is enabled

false transmit status available interrupt status is disabled

LOCAL_INLINE bool ENET_HWA_MTL_GetTxStatusAvailableIntStatus (const ENET_Type *const *pEnet*)

Get MTL transmit status available interrupt status.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

Returns

true transmit status available interrupt not detected

false transmit status available interrupt detected

**LOCAL_INLINE bool ENET_HWA_MTL_GetTxStatusFIFOFullStatus (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get the full flag of transmit Status FIFO.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true transmit Status FIFO is full

false transmit Status FIFO is not full

**LOCAL_INLINE uint8_t ENET_HWA_MTL_GetTxStatusWordCount (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get the current number of status in the transmit Status FIFO of this queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

uint8_t number of status

**LOCAL_INLINE bool ENET_HWA_MTL_GetTxStoreForwardModeEnFlag (const ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*)**

Get transmit queue store and forward enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id

Returns

true store and forward is enabled

false store and forward is disabled

**LOCAL_INLINE ENET_MTL_TxThresholdType ENET_HWA_MTL_GetTxThreshold (const ENET_Type *const
pEnet, ENET_MTL_QueueType *eQueue*)**

Get the threshold level of the MTL Tx Queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>eQueue</i>	MTL queue id
----	---------------	--------------

Returns

ENET_MTL_TxThresholdType_t threshold level

LOCAL_INLINE bool ENET_HWA_MTL_GetTxUnderflowPacketCount (const ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, uint16_t * *pPacketCnt*)

Get the underflow counter register of transmit queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
out	<i>pPacketCnt</i>	packet counter value

Return values

<i>true</i>	Underflow packet counter overflow and rollover happened
<i>false</i>	Underflow packet counter is not overflowed

LOCAL_INLINE void ENET_HWA_MTL_InitPresetCounters (ENET_Type *const *pEnet*)

Preset MTL counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MTL_ResetCounters (ENET_Type *const *pEnet*)

Reset MTL counters.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

LOCAL_INLINE void ENET_HWA_MTL_SetAllFIFOPointerResetFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Reset pointers of all FIFOs.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MTL_SetAVAlgorithmCBSEnFlag (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, uint8 *bEnable*)**

Set AV algorithm for strict priority or credit based shaper.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	1U use credit based shaper or 0U use strict priority.

**LOCAL_INLINE void ENET_HWA_MTL_SetBaseTimeRegisterErrorInterruptEnFlag (ENET_Type *const
pEnet, bool *bEnable*)**

Set enable flag of base time register error interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MTL_SetByteEnable (ENET_Type *const *pEnet*,
ENET_MTL_ByteEnableType *eByteValid*)**

Indicates the number of data bytes valid in the data register during Write operation.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eByteValid</i>	the number of data bytes valid

LOCAL_INLINE void ENET_HWA_MTL_SetConstantGateControlErrorInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set enable flag of constant gate control error interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetDebugAccessFIFO (ENET_Type *const *pEnet*, ENET_MTL_DebugAccessFIFOType *eFIFOType*)

Set the FIFO selected for debug access.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eFIFOType</i>	debug FIFO defined in ENET_MTL_DebugAccessFIFOType

LOCAL_INLINE void ENET_HWA_MTL_SetDebugCtrl (ENET_Type *const *pEnet*, uint32_t *u32DebugCtrl*)

Set MTL debug control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32DebugCtrl</i>	control value

LOCAL_INLINE void ENET_HWA_MTL_SetDropFrameSizeErrorFramesDisableFlag (ENET_Type *const *pEnet*, bool *bDisable*)

Enable dropping frames causing scheduling error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bDisable</i>	disable or enable

LOCAL_INLINE void ENET_HWA_MTL_SetDropSchedulingErrorFramesEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Enable dropping frames causing scheduling error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetDropTxPacketStatusEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set drop tx packet status enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	drop or forward tx packet status

LOCAL_INLINE void ENET_HWA_MTL_SetEstControl (ENET_Type *const *pEnet*, uint32_t *u32EstControl*)

Set enhancements to scheduled transmission control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32EstControl</i>	EST control value

LOCAL_INLINE void ENET_HWA_MTL_SetEstCurrentTimeOffset (ENET_Type *const *pEnet*, uint16_t *u16Value*)

Set enhancements to scheduled transmission current time offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u16Value</i>	current time offset

LOCAL_INLINE void ENET_HWA_MTL_SetEstEccInjectAddressErrorEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set enable flag of EST ECC address error injection.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetEstEccInjectErrorControl (ENET_Type *const *pEnet*, bool *bEnable*)

Set the mode of EST ECC inject error control for EST memory.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	true inject 2 bit errors,false inject 1 bit error

LOCAL_INLINE void ENET_HWA_MTL_SetEstEccInjectErrorEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set enable flag of EST ECC error injection.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetEstEnableFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Enable EST.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetEstGateControlListAddress (ENET_Type *const *pEnet*, uint8_t *eAddress*)

Set GCL address or GCL related registers address depend on GCRR bit.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eAddress</i>	GCL address or GCL related registers address

LOCAL_INLINE void ENET_HWA_MTL_SetEstOverheadBytes (ENET_Type *const *pEnet*, uint8_t *u8Value*)

Set EST overhead bytes value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Value</i>	EST overhead bytes value

LOCAL_INLINE void ENET_HWA_MTL_SetEstPtpTimeOffset (ENET_Type *const *pEnet*, uint8_t *u8Value*)

Set enhancements to scheduled transmission PTP time offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Value</i>	PTP time offset

LOCAL_INLINE void ENET_HWA_MTL_SetEstScheduleErrorLoopCount (ENET_Type *const *pEnet*, ENET_MTL_ScheduleErrorLoopCountType *eValue*)

Set the loop count to report scheduling error.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eValue</i>	loop count

**LOCAL_INLINE void ENET_HWA_MTL_SetEstTimeIntervalLeftShift (ENET_Type *const *pEnet*,
ENET_MTL_GCLIntervalMultipleType *eValue*)**

Set EST time interval left shift amount.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eValue</i>	left shift amount

**LOCAL_INLINE void ENET_HWA_MTL_SetFIFODebugAccessEnFlag (ENET_Type *const *pEnet*, bool
bEnable)**

Set FIFO debug access enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetFIFODebugData (ENET_Type *const *pEnet*, uint32_t *u32Data*)

Set MTL FIFO debug data.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Data</i>	debug data to set

**LOCAL_INLINE void ENET_HWA_MTL_SetFIFODebugModeAccessType (ENET_Type *const *pEnet*,
ENET_MTL_FIFODebugAccessType *eAccessType*)**

Set the debug mode of accessing to fifo.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eAccessType</i>	accessing to fifo is restricted or not

**LOCAL_INLINE void ENET_HWA_MTL_SetFIFOPacketState (ENET_Type *const *pEnet*,
ENET_MTL_FIFOPacketStatusType *eState*)**

Encoded packet state,the control information to the Tx FIFO or Rx FIFO.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eState</i>	

LOCAL_INLINE void ENET_HWA_MTL_SetFIFOWriteEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set write operation on selected FIFO enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

**LOCAL_INLINE void ENET_HWA_MTL_SetForwardErrorPacketEnFlag (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, bool *bEnable*)**

Set forward error packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_MTL_SetForwardTCPChecksumErrorPacketEnFlag (ENET_Type *const
pEnet, ENET_MTL_QueueType *eQueue*, bool *bEnable*)**

Set forwarding of TCP/IP checksum error packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
----	--------------	---------------------------------------

in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	forward TCP/IP checksum error packets

LOCAL_INLINE void ENET_HWA_MTL_SetForwardUndersizedPacketEnFlag (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, bool *bEnable*)

Set forward undersized good packets enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	

LOCAL_INLINE void ENET_HWA_MTL_SetGateControlReleatedRegistersEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set gate control releated registers enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetGclData (ENET_Type *const *pEnet*, uint32_t *eData*)

Set GCL data register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eData</i>	data to set

LOCAL_INLINE void ENET_HWA_MTL_SetGclDebugModeBankSelect (ENET_Type *const *pEnet*, uint8_t *eBank*)

Set GCL debug mode bank.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eBank</i>	debug mode bank

LOCAL_INLINE void ENET_HWA_MTL_SetGclDebugModeEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set GCL debug mode enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetGclOperation (ENET_Type *const *pEnet*, ENET_MTL_GclOperationType *eOperation*)

Set GCL operation mode (read or write)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eOperation</i>	(read or write)

LOCAL_INLINE void ENET_HWA_MTL_SetGclOperationStartFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Start GCL Read/Write operation.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	

LOCAL_INLINE void ENET_HWA_MTL_SetHeadofLineBlockingDueToFrameSizeInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set enable flag of head of line blocking due to frame size interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetHeadofLineBlockingDueToSchedulingInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set enable flag of head of line blocking due to scheduling interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetInterruptCtrlStatus (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, uint32_t *u32CtrlStatus*)

Set the interrupt enable and status for the queue interrupts.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32CtrlStatus</i>	interrupt enable and status

LOCAL_INLINE void ENET_HWA_MTL_SetLaunchExpiryOffset (ENET_Type *const *pEnet*, uint32_t *u32ExpiryOffset*)

Set launch expiry offset.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32ExpiryOffset</i>	launch expiry offset value

LOCAL_INLINE void ENET_HWA_MTL_SetLaunchExpiryOffsetEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set launch expiry offset enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable launch expiry offset

LOCAL_INLINE void ENET_HWA_MTL_SetOperationMode (ENET_Type *const *pEnet*, uint32_t *u32OpMode*)

Set MTL operation mode register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32OpMode</i>	MTL operation mode

LOCAL_INLINE void ENET_HWA_MTL_SetPreemptionClassification (ENET_Type *const *pEnet*, uint8_t *u8value*)

Set the corresponding Queue classified as express or preemptable.

Set the Classification of Queues. A bit indicates a queue, when '1' indicates the corresponding Queue must be classified as preemptable, when '0' Queue is classified as express.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8value</i>	Preemption classification value

LOCAL_INLINE void ENET_HWA_MTL_SetPreemptionHoldAdvance (ENET_Type *const *pEnet*, uint16_t *u16value*)

Set the maximum time in nanoseconds that can elapse between issuing a HOLD to the MAC and the MAC ceasing to transmit any preemptable frame.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u16value</i>	Hold Advance time value

LOCAL_INLINE void ENET_HWA_MTL_SetPreemptionReleaseAdvance (ENET_Type *const *pEnet*, uint16 *u16value*)

Set the maximum time in nanoseconds that can elapse between issuing a RELEASE to the MAC and the MAC being ready to resume transmission of preemptable frames.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u16value</i>	Release Advance time value

LOCAL_INLINE void ENET_HWA_MTL_SetRxActivateFlowCtrlThres (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, ENET_MTL_RxFlowCtrlThresholdType *eThreshold*)

Set the threshold of receive queue at which the flow control is activated.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>eThreshold</i>	flow control activating threshold

LOCAL_INLINE void ENET_HWA_MTL_SetRxArbitrationAlgorithm (ENET_Type *const *pEnet*, ENET_MTL_RxArbitrationAlgorithmType *eArbitrationAlgorithm*)

Set MTL algorithm for receive arbitration.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eArbitrationAlgorithm</i>	arbitration algorithm mode

LOCAL_INLINE void ENET_HWA_MTL_SetRxControl (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, uint32_t *u32Ctrl*)

Set MTL queue receive control register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32Ctrl</i>	receive control value

LOCAL_INLINE void ENET_HWA_MTL_SetRxDABasedDMAChannelSelectionEnFlag (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, bool *bEnable*)

Set DA-based DMA channel selection enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetRxDeactivateFlowCtrlThres (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, ENET_MTL_RxFlowCtrlThresholdType *eThreshold*)

Set the threshold of receive queue at which the flow control is deactivated.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>eThreshold</i>	flow control deactivating threshold

LOCAL_INLINE void ENET_HWA_MTL_SetRxDMAChannelMap (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, ENET_DMA_ChannelType *eDMAChannel*)

Set DMA channel mapped to eQueue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>eDMAChannel</i>	DMA channel

LOCAL_INLINE void ENET_HWA_MTL_SetRxHwFlowCtrlEnFlag (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, bool *bEnable*)

Set receive queue hardware flow control enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	enable hardware flow control

LOCAL_INLINE void ENET_HWA_MTL_SetRxOperationMode (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, uint32_t *u32OpMode*)

Set receive operation mode of the queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32OpMode</i>	receive operation mode

LOCAL_INLINE void ENET_HWA_MTL_SetRxPacketAvailableIntEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set receive packet available interrupt status enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable


```
LOCAL_INLINE void ENET_HWA_MTL_SetRxQueueEnStatus (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, ENET_MTL_QueueEnableType eEnable)
```

Set Receive queue enable flag.

Parameters

in	<i>pEnet</i>	the base address of the Ethernet instance
in	<i>eQueue</i>	the queue number
in	<i>eEnable</i>	receive queue enable mode

```
LOCAL_INLINE void ENET_HWA_MTL_SetRxQueueOverflowIntEnFlag (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)
```

Set receive queue overflow interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	enable receive queue overflow interrupt

```
LOCAL_INLINE void ENET_HWA_MTL_SetRxQueuePacketArbitrationEnFlag (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)
```

Set receive queue packet arbitration enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	enable or disable

```
LOCAL_INLINE void ENET_HWA_MTL_SetRxQueueSize (ENET_Type *const pEnet, ENET_MTL_QueueType
eQueue, uint8_t u8QueueSize)
```

Set receive queue size.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u8QueueSize</i>	receive queues size in blocks of 256 bytes, range(0~15)

**LOCAL_INLINE void ENET_HWA_MTL_SetRxQueueWeight (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, uint8_t *u8Weight*)**

Set receive queue weight.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u8Weight</i>	receive queue weight

**LOCAL_INLINE void ENET_HWA_MTL_SetRxStoreForwardModeEnFlag (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, bool *bEnable*)**

Set receive queue store and forward enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	

**LOCAL_INLINE void ENET_HWA_MTL_SetRxThreshold (ENET_Type *const *pEnet*, ENET_MTL_QueueType
eQueue, ENET_MTL_RxThresholdType *eThreshold*)**

Set the receive queue threshold.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>eThreshold</i>	receive queue threshold

LOCAL_INLINE void ENET_HWA_MTL_SetSelectedFIFOPointerResetFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Reset pointers of selected FIFO.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetSlotAverageBitsIntEnFlag (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, bool *bEnable*)

Set average bits per slot interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	enable average bits per slot interrupt

LOCAL_INLINE void ENET_HWA_MTL_SetSwitchGateControlListEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Switch gate control list.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetSwitchListInterruptEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set enable flag of base time switch GCL completion interrupt.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetTBSCtrl (ENET_Type *const *pEnet*, uint32_t *u32Ctrl*)

Set Time Based Scheduling control register.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u32Ctrl</i>	control value

**LOCAL_INLINE void ENET_HWA_MTL_SetTxOperationMode (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, uint32_t *u32OpMode*)**

Set MTL queue Tx operation mode register value.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32OpMode</i>	MTL queue Tx operation mode

**LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueEnStatus (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, ENET_MTL_QueueEnableType *eEnable*)**

Set the transmit queue enable mode.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>eEnable</i>	queue enable mode

**LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueHiCredit (ENET_Type *const *pEnet*,
ENET_MTL_QueueType *eQueue*, uint32_t *u32HiCredit*)**

Set hiCredit value required for the credit-based shaper algorithm for the Queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32hiCredit</i>	hiCredit credit based shaper.

```
LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueLoCredit (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue, uint32 u32LoCredit)
```

Set the loCredit value required for the credit-based shaper algorithm for the Queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32LoCredit</i>	loCredit credit based shaper.

```
LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueQuantumOrWeight (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue, uint32_t u32QuantumWeight)
```

Set the quantum or weights for transmit algorithm (DWRR WRR WFQ)

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32QuantumWeight</i>	quantum or weights

```
LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueSendSlopeCredit (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue, uint32 u32SendSlopeCredit)
```

Set the sendSlope credit value required for the credit-based shaper algorithm for the Queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u32SendSlopeCredit</i>	sendSlope credit credit based shaper.

LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueSize (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, uint8_t *u8QueueSize*)

Set MTL transmit queue size.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>u8QueueSize</i>	queue size in blocks of 256 bytes,range(0~15)

LOCAL_INLINE void ENET_HWA_MTL_SetTxQueueUnderflowIntEnFlag (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, bool *bEnable*)

Set transmit queue underflow interrupt enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	enable transmit queue underflow interrupt

LOCAL_INLINE void ENET_HWA_MTL_SetTxSchedulingAlgorithm (ENET_Type *const *pEnet*, ENET_MTL_TxScheduleAlgorithmType *eScheduleAlgorithm*)

Set MTL algorithm for transmit scheduling.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eScheduleAlgorithm</i>	algorithm mode

LOCAL_INLINE void ENET_HWA_MTL_SetTxStatusAvailableIntEnFlag (ENET_Type *const *pEnet*, bool *bEnable*)

Set transmit status available interrupt status enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>bEnable</i>	enable or disable

LOCAL_INLINE void ENET_HWA_MTL_SetTxStoreForwardModeEnFlag (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, bool *bEnable*)

Set transmit queue store and forward enable flag.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>bEnable</i>	

LOCAL_INLINE void ENET_HWA_MTL_SetTxThreshold (ENET_Type *const *pEnet*, ENET_MTL_QueueType *eQueue*, ENET_MTL_TxThresholdType *eThreshold*)

Set the threshold level of the MTL Tx Queue.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>eQueue</i>	MTL queue id
in	<i>eThreshold</i>	threshold level

LOCAL_INLINE void ENET_MAC_SetLayer4Addr (ENET_Type *const *pEnet*, uint8_t *u8Index*, uint32_t *u32Addr*)

Set layer 4 address.

Parameters

in	<i>pEnet</i>	the base address of the ENET instance
in	<i>u8Index</i>	layer 3 and layer 4 filter group index
in	<i>u32Addr</i>	layer 4 address value

HwA_erm

Collaboration diagram for HwA_erm:Functions

LOCAL_INLINE void **ERM_HWA_SetCRn** (ERM_Type *const pErm, uint8_t u8Idx, uint32_t u32RegValue)
set erm configuration register.

LOCAL_INLINE uint32_t **ERM_HWA_GetCRn** (const ERM_Type *const pErm, uint8_t u8Idx)
get ERM configuration register.

LOCAL_INLINE void **ERM_HWA_SetSRn** (ERM_Type *const pErm, uint8_t u8Idx, uint32_t u32RegValue)
set erm status register.

LOCAL_INLINE uint32_t **ERM_HWA_GetSRn** (const ERM_Type *const pErm, uint8_t u8Idx)
get ERM status register.

LOCAL_INLINE uint32_t **ERM_HWA_GetEARn** (const ERM_Type *const pErm, uint8_t u8Idx)
get ERM error address register.

Detailed Description

Function Documentation

LOCAL_INLINE uint32_t **ERM_HWA_GetCRn** (const ERM_Type *const *pErm*, uint8_t *u8Idx*)

get ERM configuration register.

Parameters

<i>pErm</i>	ERM Instance.
<i>u8Idx</i>	Control register index.

Returns

ERM CRn value.

```
LOCAL_INLINE uint32_t ERM_HWA_GetEARn (const ERM_Type *const pErm, uint8_t u8Idx)
```

get ERM error address register.

Parameters

<i>pErm</i>	ERM Instance.
<i>u8Idx</i>	Error address register index.

Returns

ERM error address register value.

```
LOCAL_INLINE uint32_t ERM_HWA_GetSRn (const ERM_Type *const pErm, uint8_t u8Idx)
```

get ERM status register.

Parameters

<i>pErm</i>	ERM Instance.
<i>u8Idx</i>	Status register index.

Returns

ERM SRn value.

```
LOCAL_INLINE void ERM_HWA_SetCRn (ERM_Type *const pErm, uint8_t u8Idx, uint32_t u32RegValue)
```

set erm configuration register.

Parameters

<i>pErm</i>	ERM Instance.
<i>u8Idx</i>	Control register index.
<i>u32RegValue</i>	access control value.

LOCAL_INLINE void ERM_HWA_SetSRn (ERM_Type *const *pErm*, uint8_t *u8Idx*, uint32_t *u32RegValue*)

set erm status register.

Parameters

<i>pErm</i>	ERM Instance.
<i>u8Idx</i>	Status register index.
<i>u32RegValue</i>	access control value.

HwA_fciic

Collaboration diagram for HwA_fciic:Classes

struct **INTM_MonitorType**

Typedefs

typedef struct **INTM_MonitorType** * **INTM_MonitorMemMapPtr**

Enumerations

enum **FCIIC_MsrType** { **FCIIC_MSR_TDF** = FCIIC_MSR_TDF_MASK, **FCIIC_MSR_RDF** = FCIIC_MSR_RDF_MASK, **FCIIC_MSR_EPF** = FCIIC_MSR_EPF_MASK, **FCIIC_MSR_SDF** = FCIIC_MSR_SDF_MASK, **FCIIC_MSR_NDF** = FCIIC_MSR_NDF_MASK, **FCIIC_MSR_ALF** = FCIIC_MSR_ALF_MASK, **FCIIC_MSR_FEF** = FCIIC_MSR_FEF_MASK, **FCIIC_MSR_PLTF** = FCIIC_MSR_PLTF_MASK, **FCIIC_MSR_DMF** = FCIIC_MSR_DMF_MASK, **FCIIC_MSR_MBF** = FCIIC_MSR_MBF_MASK, **FCIIC_MSR_BBF** = FCIIC_MSR_BBF_MASK, **FCIIC_MasterClearFlags**, **FCIIC_MasterErrorFlags** }

FCIIC MSR flags.

enum **FCIIC_MierType** { **FCIIC_MIER_TDIE** = FCIIC_MIER_TDIE_MASK, **FCIIC_MIER_RDIE** = FCIIC_MIER_RDIE_MASK, **FCIIC_MIER_EPIE** = FCIIC_MIER_EPIE_MASK, **FCIIC_MIER_SDIE** = FCIIC_MIER_SDIE_MASK, **FCIIC_MIER_NDIE** = FCIIC_MIER_NDIE_MASK, **FCIIC_MIER_ALIE** = FCIIC_MIER_ALIE_MASK, **FCIIC_MIER_FEIE** = FCIIC_MIER_FEIE_MASK, **FCIIC_MIER_PLTIE** =

FCIIC_MIER_PLTIE_MASK, **FCIIC_MIER_DMIE** = FCIIC_MIER_DMIE_MASK, **FCIIC_MasterIrqErrorFlags** = FCIIC_MIER_ALIE | FCIIC_MIER_FEIE | FCIIC_MIER_PLTIE, **FCIIC_MasterIrqFlags** }

FCIIC MIER flags.

enum **FCIIC_TX_CMDType** { **FCIIC_TX_CMD_TRANSMIT** = 0, **FCIIC_TX_CMD_RECEIVE**, **FCIIC_TX_CMD_STOP**, **FCIIC_TX_CMD_RECANDDISCARD**, **FCIIC_TX_CMD_STARTANDTRANSMIT**, **FCIIC_TX_CMD_STARTANDTRANSMIT_WITHNAK** }

IIC Command Type Enumeration.

enum **FCIIC_MasterStatusType** { **FCIIC_MSR_BBF_STATUS** = 25U, **FCIIC_MSR_MBF_STATUS** = 24U, **FCIIC_MSR_DMF_STATUS** = 14U, **FCIIC_MSR_PLTF_STATUS** = 13U, **FCIIC_MSR_FEF_STATUS** = 12U, **FCIIC_MSR_ALF_STATUS** = 11U, **FCIIC_MSR_NDF_STATUS** = 10U, **FCIIC_MSR_SDF_STATUS** = 9U, **FCIIC_MSR_EPF_STATUS** = 8U, **FCIIC_MSR_RDF_STATUS** = 1U, **FCIIC_MSR_TDF_STATUS** = 0U }

IIC Master status type enumeration.

enum **FCIIC_SsrType** { **FCIIC_SSR_TDF** = FCIIC_SSR_TDF_MASK, **FCIIC_SSR_RDF** = FCIIC_SSR_RDF_MASK, **FCIIC_SSR_AVF** = FCIIC_SSR_AVF_MASK, **FCIIC_SSR_TAF** = FCIIC_SSR_TAF_MASK, **FCIIC_SSR_RSF** = FCIIC_SSR_RSF_MASK, **FCIIC_SSR_SDF** = FCIIC_SSR_SDF_MASK, **FCIIC_SSR_BEf** = FCIIC_SSR_BEf_MASK, **FCIIC_SSR_TREF** = FCIIC_SSR_TREF_MASK, **FCIIC_SSR_AM0F** = FCIIC_SSR_AM0F_MASK, **FCIIC_SSR_AM1F** = FCIIC_SSR_AM1F_MASK, **FCIIC_SSR_GCF** = FCIIC_SSR_GCF_MASK, **FCIIC_SSR_SARF** = FCIIC_SSR_SARF_MASK, **FCIIC_SSR_SBF** = FCIIC_SSR_SBF_MASK, **FCIIC_SSR_BBF** = FCIIC_SSR_BBF_MASK, **FCIIC_SlaveClearFlags**, **FCIIC_SlaveErrorFlags** = FCIIC_SSR_BEf | FCIIC_SSR_TREF }

FCIIC SSR flags.

enum **FCIIC_SierType** { **FCIIC_SIER_TDIE** = FCIIC_SIER_TDIE_MASK, **FCIIC_SIER_RDIE** = FCIIC_SIER_RDIE_MASK, **FCIIC_SIER_AVIE** = FCIIC_SIER_AVIE_MASK, **FCIIC_SIER_TAIE** = FCIIC_SIER_TAIE_MASK, **FCIIC_SIER_RSIE** = FCIIC_SIER_RSIE_MASK, **FCIIC_SIER_SDIE** = FCIIC_SIER_SDIE_MASK, **FCIIC_SIER_BEIE** = FCIIC_SIER_BEIE_MASK, **FCIIC_SIER_TREIE** = FCIIC_SIER_TREIE_MASK, **FCIIC_SIER_AM0IE** = FCIIC_SIER_AM0IE_MASK, **FCIIC_SIER_AM1IE** = FCIIC_SIER_AM1IE_MASK, **FCIIC_SIER_GCIE** = FCIIC_SIER_GCIE_MASK, **FCIIC_SIER_SARIE** = FCIIC_SIER_SARIE_MASK, **FCIIC_SlaveIrqFlags** }

FCIIC MIER flags.

enum **FCIIC_SlaveStatusType** { **FCIIC_SSR_BBF_STATUS** = 25U, **FCIIC_SSR_SBF_STATUS** = 24U, **FCIIC_SSR_SARF_STATUS** = 15U, **FCIIC_SSR_GCF_STATUS** = 14U, **FCIIC_SSR_AM1F_STATUS** = 13U, **FCIIC_SSR_AM0F_STATUS** = 12U, **FCIIC_SSR_TREF_STATUS** = 11U, **FCIIC_SSR_BEf_STATUS** = 10U, **FCIIC_SSR_SDF_STATUS** = 9U, **FCIIC_SSR_RSf_STATUS** = 8U, **FCIIC_SSR_TAF_STATUS** = 3U, **FCIIC_SSR_AVF_STATUS** = 2U, **FCIIC_SSR_RDF_STATUS** = 1U, **FCIIC_SSR_TDF_STATUS** = 0U }

IIC Slave status type enumeration.

Functions

LOCAL_INLINE void **FCIIC_HwA_MasterReset** (FCIIC_Type *const base)

Restores the FCIIC master peripheral to reset conditions.

LOCAL_INLINE void **FCIIC_HwA_MasterEnable** (FCIIC_Type *const base, bool enable)

Enables or disables the FCIIC module as master.

LOCAL_INLINE void **FCIIC_HWA_MasterResetFIFO** (FCIIC_Type *const base)

Resets the FCIIC module's transmit and receive FIFOs.

LOCAL_INLINE uint8_t **FCIIC_HWA_MasterGetEnable** (FCIIC_Type *const base)

Retrieves the current status of the FCIIC master enable bit.

LOCAL_INLINE uint32_t **FCIIC_HWA_MasterGetStatusFlags** (const FCIIC_Type *const base)

Gets the FCIIC master status flags.

LOCAL_INLINE void **FCIIC_HWA_MasterClearStatusFlags** (FCIIC_Type *const base, uint32_t statusMask)

Clears the FCIIC master status flag state.

LOCAL_INLINE void **FCIIC_HWA_MasterEnableInterrupts** (FCIIC_Type *const base, uint32_t interruptMask)

Enables the FCIIC master interrupt requests.

LOCAL_INLINE void **FCIIC_HWA_MasterDisableInterrupts** (FCIIC_Type *const base, uint32_t interruptMask)

Disables the FCIIC master interrupt requests.

LOCAL_INLINE uint32_t **FCIIC_HwA_MasterGetEnabledInterrupts** (const FCIIC_Type *const base)

Returns the set of currently enabled FCIIC master interrupt requests.

LOCAL_INLINE void **FCIIC_HWA_MasterEnableDMA** (FCIIC_Type *const base, bool enableTx, bool enableRx)

Enables or disables FCIIC master DMA requests.

LOCAL_INLINE uint32_t **FCIIC_HwA_MasterGetTxFifoAddress** (const FCIIC_Type *const base)

Gets FCIIC master transmit data register address for DMA transfer.

LOCAL_INLINE uint32_t **FCIIC_HwA_MasterGetRxFifoAddress** (const FCIIC_Type *const base)

Gets FCIIC master receive data register address for DMA transfer.

```
LOCAL_INLINE void FCIIC_HwA_MasterSetWatermarks (FCIIC_Type *const base, uint16_t txWords,  
uint16_t rxWords)
```

Sets the watermarks for FCIIC master FIFOs.

```
LOCAL_INLINE void FCIIC_Master_HWA_Transmit (FCIIC_Type *pFciic, FCIIC_TX_CMDType tCmdType,  
uint8_t u8Data)
```

Master Transmit Data.

```
LOCAL_INLINE uint32 FCIIC_Master_HWA_GetTxCount (FCIIC_Type *pFciic)
```

Get the number of bytes transmitted by the master.

```
LOCAL_INLINE uint8_t FCIIC_Master_HWA_Receive (FCIIC_Type *pFciic)
```

Master Receive Data.

```
LOCAL_INLINE void FCIIC_Master_HwA_CheckClearStatus (FCIIC_Type *pFciic, FCIIC_MasterStatusType  
eStatus)
```

Get and clear IIC master status.

```
LOCAL_INLINE uint8_t FCIIC_Master_HWA_GetStatus (FCIIC_Type *pFciic, FCIIC_MasterStatusType  
eStatus)
```

Get master status.

```
LOCAL_INLINE void FCIIC_Master_HWA_ClrStatus (FCIIC_Type *pFciic, uint32_t u32Value)
```

Clear master status.

```
LOCAL_INLINE void FCIIC_Master_HWA_EnableReceiveInterrupt (FCIIC_Type *pFciic)
```

Enable Receive Interrupt.

```
LOCAL_INLINE void FCIIC_Master_HWA_DisableReceiveInterrupt (FCIIC_Type *pFciic)
```

Disable Receive Interrupt.

```
LOCAL_INLINE void FCIIC_Master_HWA_EnableErrorInterrupt (FCIIC_Type *pFciic)
```

Enable Error Interrupt.

LOCAL_INLINE void **FCIIC_Master_HWA_DisableErrorInterrupt** (FCIIC_Type *pFciic)

Disable Error Interrupt.

LOCAL_INLINE uint32_t **FCIIC_Master_HWA_GetErrorFlag** (FCIIC_Type *pFciic)

Get Error Flag.

LOCAL_INLINE void **FCIIC_Master_HWA_EnableTransmitInterrupt** (FCIIC_Type *pFciic)

Enable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_Master_HWA_DisableTransmitInterrupt** (FCIIC_Type *pFciic)

Disable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableInterrupt** (FCIIC_Type *const pFciic, uint32 u32Mask)

Enable FCIIC SIER register bit.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableInterrupt** (FCIIC_Type *const pFciic, uint32 u32Mask)

Disable FCIIC SIER register bit.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableReceiveInterrupt** (FCIIC_Type *pFciic)

Enable Receive Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableReceiveInterrupt** (FCIIC_Type *pFciic)

Disable Receive Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableErrorInterrupt** (FCIIC_Type *pFciic)

Enable Error Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableErrorInterrupt** (FCIIC_Type *pFciic)

Disable Error Interrupt.

LOCAL_INLINE uint32_t **FCIIC_Slave_HWA_GetErrorFlag** (FCIIC_Type *pFciic)

Get Error Flag.

LOCAL_INLINE void **FCIIC_Slave_HWA_ClrErrorFlag** (FCIIC_Type *pFciic)

Clear Error Flag.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableTransmitInterrupt** (FCIIC_Type *pFciic)

Enable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableTransmitInterrupt** (FCIIC_Type *pFciic)

Disable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_HWA_SlaveEnableDMA** (FCIIC_Type *const pFciic, bool enableTx, bool enableRx)

Enables or disables FCIIC Slave DMA requests.

LOCAL_INLINE uint8_t **FCIIC_HWA_SlaveGetEnable** (FCIIC_Type *const pFciic)

Checks if the slave enable flag is set in the SCR register.

LOCAL_INLINE uint8_t **FCIIC_Slave_HWA_GetStatus** (FCIIC_Type *pFciic, **FCIIC_SlaveStatusType** eStatus)

This Function is used to get slave status.

LOCAL_INLINE void **FCIIC_Slave_HWA_ClrStatus** (FCIIC_Type *pFciic, uint32_t eStatus)

This Function is used to clear slave status.

LOCAL_INLINE uint32 **FCIIC_HWA_GetSIER** (const FCIIC_Type *const pFciic)

get FCIIC SIER register.

LOCAL_INLINE void **FCIIC_Slave_HWA_Transmit** (FCIIC_Type *pFciic, uint8_t u8Data)

Slave Transmit Data.

LOCAL_INLINE uint8_t **FCIIC_Slave_HWA_Receive** (FCIIC_Type *pFciic)

Slave Get Data.

LOCAL_INLINE void **FCIIC_Slave_HWA_NACK** (FCIIC_Type *const pFciic, uint32 u32Value)

Set FCIIC STAR register.

LOCAL_INLINE void **FCIIC_HWA_SetMcr** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCR register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetMSR** (FCIIC_Type *pFciic)

Set FCIIC SSR register.

LOCAL_INLINE void **FCIIC_HWA_AttachMcr** (FCIIC_Type *pFciic, uint32_t u32Value)

Attach FCIIC MCR register.

LOCAL_INLINE void **FCIIC_HWA_SetMder** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MDER register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetMIER** (const FCIIC_Type *const pFciic)

get FCIIC MIER register.

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR0** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR0 register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetMCFGR1** (FCIIC_Type *pFciic)

get FCIIC MCFGR1 register

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR1** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR1 register.

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR2** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR2 register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetMCFGR2** (FCIIC_Type *pFciic)

Set FCIIC MCFGR2 register.

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR3** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR3 register.

LOCAL_INLINE void **FCIIC_HWA_SetMFCR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MFCR register.

LOCAL_INLINE void **FCIIC_HWA_SetMCCR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCCR register.

LOCAL_INLINE void **FCIIC_HWA_SetSDER** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SDER register.

LOCAL_INLINE void **FCIIC_HWA_SetSCFGR1** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SCFGR1 register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetSCFGR1** (FCIIC_Type *pFciic)

Get FCIIC SCFGR1 register.

LOCAL_INLINE void **FCIIC_HWA_SetSCFGR2** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SCFGR2 register.

LOCAL_INLINE void **FCIIC_HWA_SetSAMR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SAMR register.

LOCAL_INLINE void **FCIIC_HWA_SetSCR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SCR register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetSSR** (FCIIC_Type *pFciic)

Set FCIIC SSR register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetSDER** (FCIIC_Type *const pFciic)

Set FCIIC SDER register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetSASR** (const FCIIC_Type *const pFciic)

Get FCIIIC slave received address.

LOCAL_INLINE void **INTM_HWA_Enable** (INTM_Type *const plntm, bool bEnable)

Enable the INTM.

LOCAL_INLINE **INTM_MonitorType** * **INTM_HWA_GetIrqMonitor** (INTM_Type *const plntm, uint8_t u8IrqMonitorIndex)

Get the instance of interrupt monitor.

LOCAL_INLINE void **INTM_HWA_SetIACKR** (INTM_Type *const plntm, uint16_t u16IrqNum)

Set the interrupt acknowledge.

LOCAL_INLINE void **INTM_HWA_SetIRQReqNum** (**INTM_MonitorType** *const plrqMon, uint16_t u16IrqNum)

Set which interrupt to be monitored.

LOCAL_INLINE void **INTM_HWA_EnableReset** (**INTM_MonitorType** *const plrqMon, bool bEnable)

Enable reset when interrupt delays overtime.

LOCAL_INLINE void **INTM_HWA_EnableInterrupt** (**INTM_MonitorType** *const plrqMon, bool bEnable)

Enable interrupt when monitored interrupt delays overtime.

LOCAL_INLINE void **INTM_HWA_EnableInactiveMode** (**INTM_MonitorType** *const plrqMon, bool bEnable)

Enable inactive mode.

LOCAL_INLINE void **INTM_HWA_StartInactiveMode** (**INTM_MonitorType** *const plrqMon)

Start the inactive mode.

LOCAL_INLINE void **INTM_HWA_StopInactiveMode** (**INTM_MonitorType** *const plrqMon)

Stop the inactive mode.

LOCAL_INLINE void **INTM_HWA_SetLatency** (**INTM_MonitorType** *const plrqMon, uint32_t u32Latency)

Set the timeout value of interrupt.

LOCAL_INLINE uint32_t **INTM_HWA_GetTimerCounter** (INTM_MonitorType *const plrqMon)
Get the value of timer.

LOCAL_INLINE void **INTM_HWA_SetTimerCounter** (INTM_MonitorType *const plrqMon, uint32_t
u32Value)
Set the value of timer.

LOCAL_INLINE bool **INTM_HWA_ReadStatus** (INTM_MonitorType *const plrqMon)
Read interrupt status.

Detailed Description

Typedef Documentation

typedef struct INTM_MonitorType * INTM_MonitorMemMapPtr

Enumeration Type Documentation

enum FCIIC_MasterStatusType

IIC Master status type enumeration.

Enumerator:

FCIIC_MSR_BBF _STATUS	FCIIC_MSR_BBF_STATUS Bus Busy Flag
--------------------------	------------------------------------

FCIIC_MSR_MBF_STATUS	FCIIC_MSR_MBF_STATUS Master Busy Flag
FCIIC_MSR_DMF_STATUS	FCIIC_MSR_DMF_STATUS Data Match Flag
FCIIC_MSR_PLTF_STATUS	FCIIC_MSR_PLTF_STATUS Pin Low Timeout Flag
FCIIC_MSR_FEF_STATUS	FCIIC_MSR_FEF_STATUS FIFO Error Flag
FCIIC_MSR_ALF_STATUS	FCIIC_MSR_ALF_STATUS Arbitration Lost Flag
FCIIC_MSR_NDF_STATUS	FCIIC_MSR_NDF_STATUS NACK Detect Flag
FCIIC_MSR_SDF_STATUS	FCIIC_MSR_SDF_STATUS STOP Detect Flag
FCIIC_MSR_EPF_STATUS	FCIIC_MSR_EPF_STATUS End Packet Flag
FCIIC_MSR_RDF_STATUS	FCIIC_MSR_RDF_STATUS Receive Data Flag
FCIIC_MSR_TDF_STATUS	FCIIC_MSR_TDF_STATUS Transmit Data Flag

enum FCIIC_MierType

FCIIC MIER flags.

Enumerator:

FCIIC_MIER_TDI E	Transmit data interrupt
FCIIC_MIER_RDI E	Receive data interrupt
FCIIC_MIER_EPI E	End Packet interrupt
FCIIC_MIER_SDI E	Stop detect interrupt
FCIIC_MIER_NDI E	NACK detect interrupt
FCIIC_MIER_ALI E	Arbitration lost interrupt
FCIIC_MIER_FEI E	FIFO error interrupt
FCIIC_MIER_PLT IE	Pin low timeout interrupt
FCIIC_MIER_DMI E	Data match interrupt
FCIIC_MasterIrq ErrorFlags	IRQ sources enabled for error.
FCIIC_MasterIrq Flags	IRQ sources enabled by the non-blocking transactional API.

enum FCIIC_MsrType

FCIIC MSR flags.

Enumerator:

FCIIC_MSR_TDF	Transmit data flag
FCIIC_MSR_RDF	Receive data flag
FCIIC_MSR_EPF	End Packet flag
FCIIC_MSR_SDF	Stop detect flag
FCIIC_MSR_NDF	NACK detect flag
FCIIC_MSR_ALF	Arbitration lost flag
FCIIC_MSR_FEF	FIFO error flag
FCIIC_MSR_PLTF	Pin low timeout flag
FCIIC_MSR_DMF	Data match flag
FCIIC_MSR_MBF	Master busy flag
FCIIC_MSR_BBF	Bus busy flag
FCIIC_MasterClearFlags	All flags which are cleared by the driver upon starting a transfer.
FCIIC_MasterErrorFlags	Errors to check for.

enum FCIIC_SierType

FCIIC MIER flags.

Enumerator:

FCIIC_SIER_TDI E	Transmit data interrupt
FCIIC_SIER_RDI E	Receive data interrupt
FCIIC_SIER_AVIE	Address Valid Interrupt Enable
FCIIC_SIER_TAI E	Transmit ACK Interrupt Enable
FCIIC_SIER_RSI E	Repeated Start Interrupt Enable
FCIIC_SIER_SDI E	STOP Detect Interrupt Enable
FCIIC_SIER_BEI E	Bit Error Interrupt Enable
FCIIC_SIER_TRE IE	Transmit or Receive Error Interrupt Enable
FCIIC_SIER_AM0 IE	Address Match 0 Interrupt Enable
FCIIC_SIER_AM1 IE	Address Match 1 Interrupt Enable
FCIIC_SIER_GCI E	General Call Interrupt Enable
FCIIC_SIER_SAR IE	SMBus Alert Response Interrupt Enable
FCIIC_SlaveIrqFlags	IRQ sources enabled by the non-blocking transactional API.

enum FCIIC_SlaveStatusType

IIC Slave status type enumeration.

Enumerator:

FCIIC_SSR_BBF_STATUS	FCIIC_SSR_BBF_STATUS Bus Busy Flag
FCIIC_SSR_SBF_STATUS	FCIIC_SSR_SBF_STATUS Slave Busy Flag
FCIIC_SSR_SARF_STATUS	FCIIC_SSR_SARF_STATUS SMBus Alert Response Flag
FCIIC_SSR_GCF_STATUS	FCIIC_SSR_GCF_STATUS General Call Flag
FCIIC_SSR_AM1F_STATUS	FCIIC_SSR_AM1F_STATUS Address Match 1 Flag
FCIIC_SSR_AM0F_STATUS	FCIIC_SSR_AM0F_STATUS Address Match 0 Flag
FCIIC_SSR_TREF_STATUS	FCIIC_SSR_TREF_STATUS FIFO Error Flag
FCIIC_SSR_BEFS_STATUS	FCIIC_SSR_BEFS_STATUS Bit Error Flag
FCIIC_SSR_SDFS_STATUS	FCIIC_SSR_SDFS_STATUS STOP Detect Flag

FCIIC_SSR_RSF_ STATUS	FCIIC_SSR_RSF_STATUS Repeated Start Flag
FCIIC_SSR_TAF_ STATUS	FCIIC_SSR_TAF_STATUS Transmit ACK Flag
FCIIC_SSR_AVF_ STATUS	FCIIC_SSR_AVF_STATUS Address Valid Flag
FCIIC_SSR_RDF_ _STATUS	FCIIC_SSR_RDF_STATUS Receive Data Flag
FCIIC_SSR_TDF_ _STATUS	FCIIC_SSR_TDF_STATUS Transmit Data Flag

enum FCIIC_SsrType

FCIIC SSR flags.

Enumerator:

FCIIC_SSR_TDF	Transmit data flag
FCIIC_SSR_RDF	Receive data flag
FCIIC_SSR_AVF	Address Valid Flag
FCIIC_SSR_TAF	Transmit ACK Flag
FCIIC_SSR_RSF	Repeated Start Flag

FCIIC_SSR_SDF	STOP Detect Flag
FCIIC_SSR_BEF	Bit Error Flag
FCIIC_SSR_TRE F	Transmit or Receive Error Flag
FCIIC_SSR_AM0 F	Address Match 0 Flag
FCIIC_SSR_AM1 F	Address Match 1 Flag
FCIIC_SSR_GCF	General Call Flag
FCIIC_SSR_SAR F	SMBus Alert Response Flag
FCIIC_SSR_SBF	Slave Busy Flag
FCIIC_SSR_BBF	Bus Busy Flag
FCIIC_SlaveClearFlags	All flags which are cleared by the driver upon starting a transfer.
FCIIC_SlaveErrorFlags	Errors to check for.

enum FCIIC_TX_CMDType

IIC Command Type Enumeration.

Enumerator:

FCIIC_TX_CMD_TRANSMIT	FCIIC_TX_CMD_TRANSMIT transmit command
-----------------------	--

FCIIC_TX_CMD_RECEIVE	FCIIC_TX_CMD_RECEIVE receive command
FCIIC_TX_CMD_STOP	FCIIC_TX_CMD_STOP stop command
FCIIC_TX_CMD_RECANDDISCARD	FCIIC_TX_CMD_RECANDDISCARD receive and discard data command
FCIIC_TX_CMD_STARTANDTRANSMIT	FCIIC_TX_CMD_STARTANDTRANSMIT start and then transmit data
FCIIC_TX_CMD_STARTANDTRANSMIT_WITHNAK	FCIIC_TX_CMD_STARTANDTRANSMIT_WITHNAK start and then transmit and don't care ACK

Function Documentation

LOCAL_INLINE void FCIIC_HWA_AttachMcr (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Attach FCIIC MCR register.

Attaches a value to the MCR register.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
<i>u32Value</i>	Value to attach.

LOCAL_INLINE uint32_t FCIIC_HWA_GetMCFGR1 (FCIIC_Type * *pFciic*)

get FCIIC MCFGR1 register

Sets the MCFGR1 register value.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
---------------	--------------------------------------

LOCAL_INLINE uint32_t FCIIC_HWA_GetMCFGR2 (FCIIC_Type * *pFciic*)

Set FCIIC MCFGR2 register.

Parameters

<i>pFciic</i>	FCIIC instance value
---------------	----------------------

LOCAL_INLINE uint32 FCIIC_HWA_GetMIER (const FCIIC_Type *const *pFciic*)

get FCIIC MIER register.

Reads the MIER register value.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
---------------	--------------------------------------

Returns

MIER register value.

LOCAL_INLINE uint32_t FCIIC_HWA_GetMSR (FCIIC_Type * *pFciic*)

Set FCIIC SSR register.

Reads the MSR register value.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
---------------	--------------------------------------

Returns

MSR register value.

LOCAL_INLINE uint32 FCIIC_HWA_GetSASR (const FCIIC_Type *const *pFciic*)

Get FCIIC slave received address.

Parameters

in	<i>pFciic</i>	The FCIIC instance value.
----	---------------	---------------------------

Returns

uint32 The received address.

LOCAL_INLINE uint32_t FCIIC_HWA_GetSCFGR1 (FCIIC_Type * *pFciic*)

Get FCIIC SCFGR1 register.

Parameters

<i>pFciic</i>	FCIIC instance value
---------------	----------------------

LOCAL_INLINE uint32 FCIIC_HWA_GetSDER (FCIIC_Type *const *pFciic*)

Set FCIIC SDER register.

Parameters

in	<i>pFciic</i>	The FCIIC instance value.
----	---------------	---------------------------

Returns

void.

LOCAL_INLINE uint32 FCIIC_HWA_GetSIER (const FCIIC_Type *const *pFciic*)

get FCIIC SIER register.

Reads the SIER register value in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
---------------	--------------------------------------

Returns

SIER register value.

LOCAL_INLINE uint32_t FCIIC_HWA_GetSSR (FCIIC_Type * *pFciic*)

Set FCIIC SSR register.

Parameters

<i>pFciic</i>	FCIIC instance value
---------------	----------------------

LOCAL_INLINE void FCIIC_HWA_MasterClearStatusFlags (FCIIC_Type *const *base*, uint32_t *statusMask*)

Clears the FCIIC master status flag state.

This function clears specific status flags by writing to the MSR register.

Parameters

<i>base</i>	The FCIIC peripheral base address.
<i>statusMask</i>	A bitmask of status flags that are to be cleared.

LOCAL_INLINE void FCIIC_HWA_MasterDisableInterrupts (FCIIC_Type *const *base*, uint32_t *interruptMask*)

Disables the FCIIC master interrupt requests.

Parameters

<i>base</i>	The FCIIC peripheral base address.
<i>interruptMask</i>	Bit mask of interrupts to disable.

LOCAL_INLINE void FCIIC_HWA_MasterEnable (FCIIC_Type *const *base*, bool *enable*)

Enables or disables the FCIIC module as master.

This function sets or clears the MEN bit in the MCR register to enable or disable the FCIIC module as master.

Parameters

<i>base</i>	The FCIIC peripheral base address.
<i>enable</i>	Pass true to enable or false to disable the specified FCIIC as master.

LOCAL_INLINE void FCIIC_HWA_MasterEnableDMA (FCIIC_Type *const *base*, bool *enableTx*, bool *enableRx*)

Enables or disables FCIIC master DMA requests.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

<i>enableTx</i>	Enable flag for transmit DMA request. Pass true for enable, false for disable.
<i>enableRx</i>	Enable flag for receive DMA request. Pass true for enable, false for disable.

LOCAL_INLINE void FCIIC_HWA_MasterEnableInterrupts (FCIIC_Type *const *base*, uint32_t *interruptMask*)

Enables the FCIIC master interrupt requests.

This function enables specific interrupt requests by setting bits in the MIER register.

Parameters

<i>base</i>	The FCIIC peripheral base address.
<i>interruptMask</i>	Bit mask of interrupts to enable.

LOCAL_INLINE uint8_t FCIIC_HWA_MasterGetEnable (FCIIC_Type *const *base*)

Retrieves the current status of the FCIIC master enable bit.

This function checks the MEN bit in the MCR register to determine if the FCIIC is enabled as master.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

Returns

State of the master enable bit: 0x01 if enabled, 0x00 if disabled.

LOCAL_INLINE uint32_t FCIIC_HWA_MasterGetEnabledInterrupts (const FCIIC_Type *const *base*)

Returns the set of currently enabled FCIIC master interrupt requests.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

Returns

A bitmask composed of FCIIC_MierType enumerators

LOCAL_INLINE uint32_t FCIIC_HWA_MasterGetRxFifoAddress (const FCIIC_Type *const *base*)

Gets FCIIC master receive data register address for DMA transfer.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

Returns

The FCIIC Master Receive Data Register address.

LOCAL_INLINE uint32_t FCIIC_HWA_MasterGetStatusFlags (const FCIIC_Type *const *base*)

Gets the FCIIC master status flags.

This function reads the MSR register to get the current status flags.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

Returns

The value of the MSR register.

LOCAL_INLINE uint32_t FCIIC_HWA_MasterGetTxFifoAddress (const FCIIC_Type *const *base*)

Gets FCIIC master transmit data register address for DMA transfer.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

Returns

The FCIIC Master Transmit Data Register address.

LOCAL_INLINE void FCIIC_HWA_MasterReset (FCIIC_Type *const *base*)

Restores the FCIIC master peripheral to reset conditions.

This function resets the FCIIC master peripheral by setting and then clearing the RST bit in the MCR register.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

LOCAL_INLINE void FCIIC_HWA_MasterResetFIFO (FCIIC_Type *const *base*)

Resets the FCIIC module's transmit and receive FIFOs.

This function sets the RRF and RTF bits in the MCR register to reset the transmit and receive FIFOs.

Parameters

<i>base</i>	The FCIIC peripheral base address.
-------------	------------------------------------

LOCAL_INLINE void FCIIC_HWA_MasterSetWatermarks (FCIIC_Type *const *base*, uint16_t *txWords*, uint16_t *rxWords*)

Sets the watermarks for FCIIC master FIFOs.

Parameters

<i>base</i>	The FCIIC peripheral base address.
<i>txWords</i>	Transmit FIFO watermark value.
<i>rxWords</i>	Receive FIFO watermark value.

LOCAL_INLINE void FCIIC_HWA_SetMCCR (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MCCR register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetMCFGR0 (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MCFGR0 register.

Sets the MCFGR0 register value.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
<i>u32Value</i>	Register value to set.

LOCAL_INLINE void FCIIC_HWA_SetMCFGR1 (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MCFGR1 register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetMCFGR2 (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MCFGR2 register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetMCFGR3 (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MCFGR3 register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetMcr (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MCR register.

Sets the MCR register value.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
<i>u32Value</i>	Register value to set.

LOCAL_INLINE void FCIIC_HWA_SetMder (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MDER register.

Sets the MDER register value.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
<i>u32Value</i>	Register value to set.

LOCAL_INLINE void FCIIC_HWA_SetMFCR (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC MFCR register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetSAMR (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC SAMR register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetSCFGR1 (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC SCFGR1 register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetSCFGR2 (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC SCFGR2 register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetSCR (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC SCR register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SetSDER (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Set FCIIC SDER register.

Parameters

<i>pFciic</i>	FCIIC instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCIIC_HWA_SlaveEnableDMA (FCIIC_Type *const *pFciic*, bool *enableTx*, bool *enableRx*)

Enables or disables FCIIC Slave DMA requests.

Enables or disables DMA requests for transmit and receive operations in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the FCIIC peripheral base address.
<i>enableTx</i>	Enable flag for transmit DMA request. Pass true for enable, false for disable.
<i>enableRx</i>	Enable flag for receive DMA request. Pass true for enable, false for disable.

LOCAL_INLINE uint8_t FCIIC_HWA_SlaveGetEnable (FCIIC_Type *const *pFciic*)

Checks if the slave enable flag is set in the SCR register.

This function checks if the slave is enabled by reading the SCR register.

Parameters

<i>pFciic</i>	Pointer to the FCIIC module instance.
---------------	---------------------------------------

Returns

0x01 if the slave is enabled, 0x00 otherwise.

LOCAL_INLINE void FCIIC_Master_HwA_CheckClearStatus (FCIIC_Type * *pFciic*, FCIIC_MasterStatusType *eStatus*)

Get and clear IIC master status.

Checks and clears a specific status flag in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
<i>eStatus</i>	Status type to check and clear.

LOCAL_INLINE void FCIIC_Master_HWA_ClrStatus (FCIIC_Type * *pFciic*, uint32_t *u32Value*)

Clear master status.

Clears specific status flags in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
<i>u32Value</i>	Bitmask of the status flags to clear.

LOCAL_INLINE void FCIIC_Master_HWA_DisableErrorInterrupt (FCIIC_Type * *pFciic*)

Disable Error Interrupt.

Disables the error interrupt in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Master_HWA_DisableReceiveInterrupt (FCIIC_Type * *pFciic*)

Disable Receive Interrupt.

Disables the receive interrupt in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Master_HWA_DisableTransmitInterrupt (FCIIC_Type * *pFciic*)

Disable Transmit Interrupt.

Disables the transmit interrupt in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Master_HWA_EnableErrorInterrupt (FCIIC_Type * *pFciic*)

Enable Error Interrupt.

Enables the error interrupt in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Master_HWA_EnableReceiveInterrupt (FCIIC_Type * *pFciic*)

Enable Receive Interrupt.

Enables the receive interrupt in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Master_HWA_EnableTransmitInterrupt (FCIIC_Type * *pFciic*)

Enable Transmit Interrupt.

Enables the transmit interrupt in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE uint32_t FCIIC_Master_HWA_GetErrorFlag (FCIIC_Type * *pFciic*)

Get Error Flag.

Gets the error flag in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

Returns

Error flag value.

LOCAL_INLINE uint8_t FCIIC_Master_HWA_GetStatus (FCIIC_Type * *pFciic*, FCIIC_MasterStatusType *eStatus*)

Get master status.

Reads a specific status flag in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
<i>eStatus</i>	Status type to read.

Returns

Status value (0 or 1), where 1 indicates the status is set.

LOCAL_INLINE uint32 FCIIC_Master_HWA_GetTxCount (FCIIC_Type * *pFciic*)

Get the number of bytes transmitted by the master.

Returns the count of bytes transmitted by the master.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

Returns

Number of bytes transmitted.

LOCAL_INLINE uint8_t FCIIC_Master_HWA_Receive (FCIIC_Type * *pFciic*)

Master Receive Data.

Receives a data byte in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

Returns

Received data byte.

LOCAL_INLINE void FCIIC_Master_HWA_Transmit (FCIIC_Type * *pFciic*, FCIIC_TX_CMDType *tCmdType*, uint8_t *u8Data*)

Master Transmit Data.

Transmits data in the master mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
<i>tCmdType</i>	Command type to be sent.
<i>u8Data</i>	Data byte to be transmitted.

LOCAL_INLINE void FCIIC_Slave_HWA_ClrErrorFlag (FCIIC_Type * *pFciic*)

Clear Error Flag.

Clears the error flag in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Slave_HWA_ClrStatus (FCIIC_Type * *pFciic*, uint32_t *eStatus*)

This Function is used to clear slave status.

Clears specific status flags in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
<i>u32Value</i>	Bitmask of the status flags to clear.

LOCAL_INLINE void FCIIC_Slave_HWA_DisableErrorInterrupt (FCIIC_Type * *pFciic*)

Disable Error Interrupt.

Disables the error interrupt in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Slave_HWA_DisableInterrupt (FCIIC_Type *const *pFciic*, uint32 *u32Mask*)

Disable FCIIC SIER register bit.

Disables a specific bit in the SIER register of the slave mode.

Parameters

in	<i>pFciic</i>	Pointer to the FCIIC instance value.
in	<i>u32Mask</i>	The disable bit mask.

LOCAL_INLINE void FCIIC_Slave_HWA_DisableReceiveInterrupt (FCIIC_Type * *pFciic*)

Disable Receive Interrupt.

Disables the receive interrupt in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Slave_HWA_DisableTransmitInterrupt (FCIIC_Type * *pFciic*)

Disable Transmit Interrupt.

Disables the transmit interrupt in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Slave_HWA_EnableErrorInterrupt (FCIIC_Type * *pFciic*)

Enable Error Interrupt.

Enables the error interrupt in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Slave_HWA_EnableInterrupt (FCIIC_Type *const *pFciic*, uint32 *u32Mask*)

Enable FCIIC SIER register bit.

Enables a specific bit in the SIER register of the slave mode.

Parameters

in	<i>pFciic</i>	Pointer to the FCIIC instance value.
in	<i>u32Mask</i>	The enable bit mask.

LOCAL_INLINE void FCIIC_Slave_HWA_EnableReceiveInterrupt (FCIIC_Type * *pFciic*)

Enable Receive Interrupt.

Enables the receive interrupt in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE void FCIIC_Slave_HWA_EnableTransmitInterrupt (FCIIC_Type * *pFciic*)

Enable Transmit Interrupt.

Enables the transmit interrupt in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

LOCAL_INLINE uint32_t FCIIC_Slave_HWA_GetErrorFlag (FCIIC_Type * *pFciic*)

Get Error Flag.

Gets the error flag in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

Returns

Error flag value.

LOCAL_INLINE uint8_t FCIIC_Slave_HWA_GetStatus (FCIIC_Type * *pFciic*, FCIIC_SlaveStatusType *eStatus*)

This Function is used to get slave status.

Reads a specific status flag in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

<i>eStatus</i>	Status type to read.
----------------	----------------------

Returns

Status value (0 or 1), where 1 indicates the status is set.

LOCAL_INLINE void FCIIC_Slave_HWA_NACK (FCIIC_Type *const *pFciic*, uint32_t *u32Value*)

Set FCIIC STAR register.

Sets the STAR register value in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the FCIIC instance value.
<i>u32Value</i>	Register value to set.

LOCAL_INLINE uint8_t FCIIC_Slave_HWA_Receive (FCIIC_Type * *pFciic*)

Slave Get Data.

Receives a data byte in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
---------------	------------------------------------

Returns

Received data byte.

LOCAL_INLINE void FCIIC_Slave_HWA_Transmit (FCIIC_Type * *pFciic*, uint8_t *u8Data*)

Slave Transmit Data.

Transmits data in the slave mode.

Parameters

<i>pFciic</i>	Pointer to the IIC instance value.
<i>u8Data</i>	Data byte to be transmitted.

LOCAL_INLINE void INTM_HWA_Enable (INTM_Type *const *pIntm*, bool *bEnable*)

Enable the INTM.

Parameters

<i>plntm</i>	INTM instance. INTM instance.
<i>bEnable</i>	Enable the ITNM.

LOCAL_INLINE void INTM_HWA_EnableInactiveMode (INTM_MonitorType *const *plrqMon*, bool *bEnable*)

Enable inactive mode.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
<i>bEnable</i>	Enable inactive mode.

LOCAL_INLINE void INTM_HWA_EnableInterrupt (INTM_MonitorType *const *plrqMon*, bool *bEnable*)

Enable interrupt when monitored interrupt delays overtime.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
<i>bEnable</i>	Enabel the interrupt.

LOCAL_INLINE void INTM_HWA_EnableReset (INTM_MonitorType *const *plrqMon*, bool *bEnable*)

Enable reset when interrupt delays overtime.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
<i>bEnable</i>	Enable reset.

LOCAL_INLINE INTM_MonitorType* INTM_HWA_GetIrqMonitor (INTM_Type *const *plntm*, uint8_t *u8IrqMonitorIndex*)

Get the instance of interrupt monitor.

Parameters

<i>pIntm</i>	INTM instance.
<i>u8IrqMonitorIndex</i>	Interrupt monitor index.

Returns

Monitor Instance.

LOCAL_INLINE uint32_t INTM_HWA_GetTimerCounter (INTM_MonitorType *const *plrqMon*)

Get the value of timer.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
----------------	-----------------------------

Returns

Timer value

LOCAL_INLINE bool INTM_HWA_ReadStatus (INTM_MonitorType *const *plrqMon*)

Read interrupt status.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
----------------	-----------------------------

Returns

Interrupt status.

LOCAL_INLINE void INTM_HWA_SetIACKR (INTM_Type *const *pIntm*, uint16_t *u16IrqNum*)

Set the interrupt acknowledge.

Parameters

<i>pIntm</i>	INTM instance.
<i>u16IrqNum</i>	Interrupt number to be monitored.

LOCAL_INLINE void INTM_HWA_SetIRQReqNum (INTM_MonitorType *const *plrqMon*, uint16_t *u16IrqNum*)

Set which interrupt to be monitored.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
<i>u16IrqNum</i>	Interrupt number to be monitored.

LOCAL_INLINE void INTM_HWA_SetLatency (INTM_MonitorType *const *plrqMon*, uint32_t *u32Latency*)

Set the timeout value of interrupt.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
<i>u32Latency</i>	the timeout value of interrupt monitor.

LOCAL_INLINE void INTM_HWA_SetTimerCounter (INTM_MonitorType *const *plrqMon*, uint32_t *u32Value*)

Set the value of timer.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
<i>u32Value</i>	The value to be set.

LOCAL_INLINE void INTM_HWA_StartInactiveMode (INTM_MonitorType *const *plrqMon*)

Start the inactive mode.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
----------------	-----------------------------

LOCAL_INLINE void INTM_HWA_StopInactiveMode (INTM_MonitorType *const *plrqMon*)

Stop the inactive mode.

Parameters

<i>plrqMon</i>	Interrupt monitor instance.
----------------	-----------------------------

HwA_fcpit

Collaboration diagram for HwA_fcpit:Enumerations

enum **FCPIT_TimerModeType** { **FCPIT_32PERIODIC_COUNTER** = 0, **FCPIT_DUAL_16PERIODIC_COUNTER**, **FCPIT_ACCUMULATOR**, **FCPIT_INPUT_CAPTURE**, **FCPIT_CHANNEL_NOMODE** }

Fcpit counter mode, the default mode is 32bit periodic count mode.

enum **FCPIT_ChannelType** { **FCPIT_CHANNEL_0** = 0U, **FCPIT_CHANNEL_1**, **FCPIT_CHANNEL_2**, **FCPIT_CHANNEL_3**, **FCPIT_CHANNEL_MAX** }

Fcpit channel number.

Functions

LOCAL_INLINE void **FCPIT_HWA_SetChannelValue** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel, uint32_t u32RegValue)

Set FCPIT channel value.

LOCAL_INLINE void **FCPIT_HWA_ConfigChannel** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel, uint32_t u32RegValue)

Configure FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_ConfigModule** (FCPIT_Type *pFcpit, uint32_t u32RegValue)

Configure FCPIT module.

LOCAL_INLINE uint32_t **FCPIT_HWA_ReadModuleEnable** (FCPIT_Type *pFcpit)

Read FCPIT module enable.

LOCAL_INLINE uint32_t **FCPIT_HWA_ReadChannelEnable** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Read FCPIT channel.

LOCAL_INLINE uint32_t **FCPIT_HWA_ReadInterruptFlag** (FCPIT_Type *pFcpit)

Read FCPIT active interrupt flag.

LOCAL_INLINE uint32_t **FCPIT_HWA_ReadEnableInterruptFlag** (FCPIT_Type *pFcpit)

Read FCPIT enable interrupt flag.

LOCAL_INLINE void **FCPIT_HWA_SetChannelRunOnDebug** (FCPIT_Type *pFcpit)

Set FCPIT channel running on debug mode.

LOCAL_INLINE void **FCPIT_HWA_SetChannelRunOnLpm** (FCPIT_Type *pFcpit)

Set FCPIT channel running on low power mode.

LOCAL_INLINE void **FCPIT_HWA_EnableModule** (FCPIT_Type *pFcpit)

Enable FCPIT module.

LOCAL_INLINE void **FCPIT_HWA_EnableChannelsInterrupt** (FCPIT_Type *pFcpit, uint32_t u32RegValue)

Enable FCPIT channel(n) interrupt.

LOCAL_INLINE void **FCPIT_HWA_EnableChannel** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Enable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_SetTimerEnable** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Enable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_ClearTimerEnable** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Disable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_EnableChannelChainMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Enable FCPIT channel(n) chain mode.

LOCAL_INLINE void **FCPIT_HWA_ConfigChannelMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel, **FCPIT_TimerModeType** eMode)

Configure FCPIT channel operation mode.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStartOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel start on trigger.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStopOnInterrupt** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel stop on interrupt.

LOCAL_INLINE void **FCPIT_HWA_SetChannelReloadOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel reload on trigger.

LOCAL_INLINE void **FCPIT_HWA_SetChannelTriggerSrc** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel trigger source.

LOCAL_INLINE void **FCPIT_HWA_SelectChannelTrigger** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel, uint8_t u8SelChannel)

Select FCPIT channel trigger.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStopOnDebug** (FCPIT_Type *pFcpit)

Set FCPIT channel stop on debug mode.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStopOnLpm** (FCPIT_Type *pFcpit)

Set FCPIT channel stop on low power mode.

LOCAL_INLINE void **FCPIT_HWA_DisableModule** (FCPIT_Type *pFcpit)

Disable FCPIT module.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelsInterruptFlag** (FCPIT_Type *pFcpit, uint32_t u32RegValue)

Clear FCPIT channel(n) interrupt flag.

LOCAL_INLINE void **FCPIT_HWA_DisableChannelsInterrupt** (FCPIT_Type *pFcpit, uint32_t u32RegValue)

Disable FCPIT channel(n) interrupt.

LOCAL_INLINE void **FCPIT_HWA_DisableChannel** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Disable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_DisableChannelChainMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Disable FCPIT channel chain mode.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Clear FCPIT channel operation mode.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelStartOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Clear FCPIT channel start on trigger.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelStopOnInterrupt** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Clear FCPIT channel stop on interrupt.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelReloadOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Clear FCPIT channel reload on trigger.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelTriggerSrc** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Clear FCPIT channel trigger source.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelTriggerSelect** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Clear FCPIT channel trigger select.

Detailed Description

Enumeration Type Documentation

enum FCPIT_ChannelType

Fcpit channel number.

Enumerator:

FCPIT_CHANNE L_0	
FCPIT_CHANNE L_1	
FCPIT_CHANNE L_2	
FCPIT_CHANNE L_3	
FCPIT_CHANNE L_MAX	

enum FCPIT_TimerModeType

Fcpit counter mode, the default mode is 32bit periodic count mode.

Enumerator:

FCPIT_32PERIO DIC_COUNTER	
------------------------------	--

FCPIT_DUAL_16 PERIODIC_COU NTER	
FCPIT_ACCUMU LATOR	
FCPIT_INPUT_C APTURE	
FCPIT_CHANNE L_NOMODE	

Function Documentation

LOCAL_INLINE void FCPIT_HWA_ClearChannelMode (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Clear FCPIT channel operation mode.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_ClearChannelReloadOnTrig (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Clear FCPIT channel reload on trigger.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_ClearChannelsInterruptFlag (FCPIT_Type * *pFcpit*, uint32_t *u32RegValue*)

Clear FCPIT channel(n) interrupt flag.

Parameters

<i>u32RegValue</i>	0-3 bit indicate TIF0-TIF3
--------------------	----------------------------

LOCAL_INLINE void FCPIT_HWA_ClearChannelStartOnTrig (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Clear FCPIT channel start on trigger.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_ClearChannelStopOnInterrupt (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Clear FCPIT channel stop on interrupt.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_ClearChannelTriggerSelect (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Clear FCPIT channel trigger select.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_ClearChannelTriggerSrc (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Clear FCPIT channel trigger source.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_ClearTimerEnable (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Disable FCPIT channel.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_ConfigChannel (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*, uint32_t *u32RegValue*)

Configure FCPIT channel.

Parameters

<i>eChannel</i>	FCPIT channel number
<i>u32RegValue</i>	TCTRL register value

LOCAL_INLINE void FCPIT_HWA_ConfigChannelMode (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*, FCPIT_TimerModeType *eMode*)

Configure FCPIT channel operation mode.

Parameters

<i>eChannel</i>	FCPIT channel number
<i>eMode</i>	FCPIT operation mode

LOCAL_INLINE void FCPIT_HWA_ConfigModule (FCPIT_Type * *pFcpit*, uint32_t *u32RegValue*)

Configure FCPIT module.

Parameters

<i>u32RegValue</i>	MCR register value
--------------------	--------------------

LOCAL_INLINE void FCPIT_HWA_DisableChannel (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Disable FCPIT channel.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_DisableChannelChainMode (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Disable FCPIT channel chain mode.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_DisableChannelsInterrupt (FCPIT_Type * *pFcpit*, uint32_t *u32RegValue*)

Disable FCPIT channel(n) interrupt.

Parameters

<i>u32RegValue</i>	u32RegValue 0-3 bit indicate TIE0-TIE3
--------------------	--

LOCAL_INLINE void FCPIT_HWA_DisableModule (FCPIT_Type * *pFcpit*)

Disable FCPIT module.

LOCAL_INLINE void FCPIT_HWA_EnableChannel (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Enable FCPIT channel.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_EnableChannelChainMode (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Enable FCPIT channel(n) chain mode.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_EnableChannelsInterrupt (FCPIT_Type * *pFcpit*, uint32_t *u32RegValue*)

Enable FCPIT channel(n) interrupt.

Parameters

<i>u32RegValue</i>	u32RegValue 0-3 bit indicate TIE0-TIE3
--------------------	--

LOCAL_INLINE void FCPIT_HWA_EnableModule (FCPIT_Type * *pFcpit*)

Enable FCPIT module.

LOCAL_INLINE uint32_t FCPIT_HWA_ReadChannelEnable (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Read FCPIT channel.

Parameters

<i>eChannel</i>	Channel number
-----------------	----------------

Returns

TCTRL register with FCPIT_TCTRL_T_EN_MASK

LOCAL_INLINE uint32_t FCPIT_HWA_ReadEnableInterruptFlag (FCPIT_Type * *pFcpit*)

Read FCPIT enable interrupt flag.

Returns

FCPIT enable interrupt flag

LOCAL_INLINE uint32_t FCPIT_HWA_ReadInterruptFlag (FCPIT_Type * *pFcpit*)

Read FCPIT active interrupt flag.

Returns

FCPIT active interrupt flag

LOCAL_INLINE uint32_t FCPIT_HWA_ReadModuleEnable (FCPIT_Type * *pFcpit*)

Read FCPIT module enable.

Returns

MCR register with FCPIT_MCR_M_CEN_MASK

LOCAL_INLINE void FCPIT_HWA_SelectChannelTrigger (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*, uint8_t *u8SelChannel*)

Select FCPIT channel trigger.

Parameters

<i>eChannel</i>	FCPIT channel number
<i>u8SelChannel</i>	Select channel, range is 0-3

LOCAL_INLINE void FCPIT_HWA_SetChannelReloadOnTrig (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Set FCPIT channel reload on trigger.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_SetChannelRunOnDebug (FCPIT_Type * *pFcpit*)

Set FCPIT channel running on debug mode.

LOCAL_INLINE void FCPIT_HWA_SetChannelRunOnLpm (FCPIT_Type * *pFcpit*)

Set FCPIT channel running on low power mode.

LOCAL_INLINE void FCPIT_HWA_SetChannelStartOnTrig (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Set FCPIT channel start on trigger.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_SetChannelStopOnDebug (FCPIT_Type * *pFcpit*)

Set FCPIT channel stop on debug mode.

LOCAL_INLINE void FCPIT_HWA_SetChannelStopOnInterrupt (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Set FCPIT channel stop on interrupt.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_SetChannelStopOnLpm (FCPIT_Type * *pFcpit*)

Set FCPIT channel stop on low power mode.

LOCAL_INLINE void FCPIT_HWA_SetChannelTriggerSrc (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Set FCPIT channel trigger source.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

LOCAL_INLINE void FCPIT_HWA_SetChannelValue (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*, uint32_t *u32RegValue*)

Set FCPIT channel value.

Parameters

<i>eChannel</i>	FCPIT channel number
<i>u32RegValue</i>	Timer value

LOCAL_INLINE void FCPIT_HWA_SetTimerEnable (FCPIT_Type * *pFcpit*, FCPIT_ChannelType *eChannel*)

Enable FCPIT channel.

Parameters

<i>eChannel</i>	FCPIT channel number
-----------------	----------------------

HwA_fcsmu

Collaboration diagram for HwA_fcsmu:Functions

LOCAL_INLINE void FCSMU_HWA_SetCtrl (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu CTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCtrl** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCtrlOps** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CTRL register OPS.

LOCAL_INLINE void **FCSMU_HWA_SetOprk** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu OPRK register.

LOCAL_INLINE void **FCSMU_HWA_SetSoctrl** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu SOCTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSoctrl** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SOCTRL register.

LOCAL_INLINE void **FCSMU_HWA_SetFccr0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FCCR0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFccr0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FCCR0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFrst0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FRST0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFrst0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FRST0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFst0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FST0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFst0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FST0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFstUnlk** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FST_UNLK register.

LOCAL_INLINE void **FCSMU_HWA_SetFe0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FE0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFe0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FE0 register.

LOCAL_INLINE void **FCSMU_HWA_SetWarningEn0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu WARNING_EN0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWarningEn0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WARNING_EN0 register.

LOCAL_INLINE void **FCSMU_HWA_SetWarningTo** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu WARNING_TO register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWarningTo** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WARNING_TO register.

LOCAL_INLINE void **FCSMU_HWA_SetCfgTo** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu CFG_TO register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCfgTo** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CFG_TO register.

LOCAL_INLINE void **FCSMU_HWA_SetSoutDiag** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu SOUT_DIAG register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSoutDiag** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SOUT_DIAG register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetStatus** (const FCSMU_Type *const pFcsmu)

Get the fcsmu STATUS register.

LOCAL_INLINE bool **FCSMU_HWA_GetFaultState** (const FCSMU_Type *const pFcsmu)

Get the fcsmu STATUS register FIF value.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFcsmuState** (const FCSMU_Type *const pFcsmu)

Get the fcsmu STATUS register STAT value.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetNtw** (const FCSMU_Type *const pFcsmu)

Get the fcsmu NTW register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWtf** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WTF register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetNtf** (const FCSMU_Type *const pFcsmu)

Get the fcsmu NTF register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFtw** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FTW register.

LOCAL_INLINE void **FCSMU_HWA_SetInject** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu INJECT register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetIrqStat** (const FCSMU_Type *const pFcsmu)

Get the fcsmu IRQ_STAT register.

LOCAL_INLINE void **FCSMU_HWA_ClearCfgToIrq** (FCSMU_Type *const pFcsmu)

Clear the fcsmu IRQ_STAT register.

LOCAL_INLINE void **FCSMU_HWA_EnableCfgToIrq** (FCSMU_Type *const pFcsmu, bool bEnable)

Set the fcsmu IRQ_EN register.

LOCAL_INLINE void **FCSMU_HWA_SetTempUnlk** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu TEMP_UNLK register.

LOCAL_INLINE void **FCSMU_HWA_SetPermntLock** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu PERMNT_LOCK register.

LOCAL_INLINE void **FCSMU_HWA_SetStmr** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu STMR register.

LOCAL_INLINE void **FCSMU_HWA_SetWarninglen0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu WARNING_IEN0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFaultlen0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FAULT_IEN0 register.

LOCAL_INLINE void **FCSMU_HWA_SetSoutEn0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu SOUT_EN0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWarningTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WARNING_TMR register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSafeModeTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SM_TMR register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCfgTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CFG_TMR register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSoutTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SOUT_TMR register.

LOCAL_INLINE void **FCSMU_HWA_SetCrcCtrl** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcBusy** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register BUSY value.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcErrorFlag** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register EF value.

LOCAL_INLINE void **FCSMU_HWA_ClearCrcErrorFlag** (FCSMU_Type *const pFcsmu)

Clear the fcsmu CRC_CTRL register EF.

LOCAL_INLINE void **FCSMU_HWA_EnableErrorOutput** (FCSMU_Type *const pFcsmu, bool bEnable)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcErrorOutputEnable** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register EOEN value.

LOCAL_INLINE void **FCSMU_HWA_EnableCrcChecker** (FCSMU_Type *const pFcsmu, bool bEnable)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcCheckerEnable** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register CHKEN value.

LOCAL_INLINE void **FCSMU_HWA_EnableTrigger** (FCSMU_Type *const pFcsmu, bool bEnable)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetTriggerEnable** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register TRGEN value.

LOCAL_INLINE void **FCSMU_HWA_GenerateCrc** (FCSMU_Type *const pFcsmu)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCrcResult** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_RES register.

Detailed Description

Function Documentation

LOCAL_INLINE void FCSMU_HWA_ClearCfgToIrq (FCSMU_Type *const *pFcsmu*)

Clear the fcsmu IRQ_STAT register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

LOCAL_INLINE void FCSMU_HWA_ClearCrcErrorFlag (FCSMU_Type *const *pFcsmu*)

Clear the fcsmu CRC_CTRL register EF.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

LOCAL_INLINE void FCSMU_HWA_EnableCfgToIrq (FCSMU_Type *const *pFcsmu*, bool *bEnable*)

Set the fcsmu IRQ_EN register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>bEnable</i>	IRQ_EN register value

LOCAL_INLINE void FCSMU_HWA_EnableCrcChecker (FCSMU_Type *const *pFcsmu*, bool *bEnable*)

Set the fcsmu CRC_CTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>bEnable</i>	CRC_CTRL register value

LOCAL_INLINE void FCSMU_HWA_EnableErrorOutput (FCSMU_Type *const *pFcsmu*, bool *bEnable*)

Set the fcsmu CRC_CTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>bEnable</i>	Enable or Disable CRC_CTRL register EOEN bit value

LOCAL_INLINE void FCSMU_HWA_EnableTrigger (FCSMU_Type *const *pFcsmu*, bool *bEnable*)

Set the fcsmu CRC_CTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>bEnable</i>	CRC_CTRL register value

LOCAL_INLINE void FCSMU_HWA_GenerateCrc (FCSMU_Type *const *pFcsmu*)

Set the fcsmu CRC_CTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

LOCAL_INLINE uint32_t FCSMU_HWA_GetCfgTmr (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CFG_TMR register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

CFG_TMR register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetCfgTo (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CFG_TO register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

CFG_TO register value

LOCAL_INLINE bool FCSMU_HWA_GetCrcBusy (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CRC_CTRL register BUSY value.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

BUSY value

LOCAL_INLINE bool FCSMU_HWA_GetCrcCheckerEnable (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CRC_CTRL register CHKEN value.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

CHKEN value

LOCAL_INLINE bool FCSMU_HWA_GetCrcErrorFlag (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CRC_CTRL register EF value.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

EF value

LOCAL_INLINE bool FCSMU_HWA_GetCrcErrorOutputEnable (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CRC_CTRL register EOEN value.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

EOEN value

LOCAL_INLINE uint32_t FCSMU_HWA_GetCrcResult (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CRC_RES register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

CRC_RES register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetCtrl (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

CTRL register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetCtrlOps (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CTRL register OPS.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

CTRL register OPS value

LOCAL_INLINE bool FCSMU_HWA_GetFaultState (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu STATUS register FIF value.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

STATUS register FIF value

LOCAL_INLINE uint32_t FCSMU_HWA_GetFccr0 (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu FCCR0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

FCCR0 register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetFcsmuState (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu STATUS register STAT value.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

STATUS register STAT value

LOCAL_INLINE uint32_t FCSMU_HWA_GetFe0 (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu FE0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

FE0 register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetFrst0 (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu FRST0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

FRST0 register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetFst0 (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu FST0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

FST0 register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetFtw (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu FTW register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

FTW register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetIrqStat (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu IRQ_STAT register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

IRQ_STAT register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetNtf (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu NTF register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

NTF register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetNtw (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu NTW register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

NTW register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetSafeModeTmr (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu SM_TMR register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

SM_TMR register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetSoctrl (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu SOCTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

SOCTRL register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetSoutDiag (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu SOUT_DIAG register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

SOUT_DIAG register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetSoutTmr (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu SOUT_TMR register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

SOUT_TMR register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetStatus (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu STATUS register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

STATUS register value

LOCAL_INLINE bool FCSMU_HWA_GetTriggerEnable (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu CRC_CTRL register TRGEN value.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

TRGEN value

LOCAL_INLINE uint32_t FCSMU_HWA_GetWarningEn0 (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu WARNING_EN0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

WARNING_EN0 register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetWarningTmr (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu WARNING_TMR register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

WARNING_TMR register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetWarningTo (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu WARNING_TO register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

WARNING_TO register value

LOCAL_INLINE uint32_t FCSMU_HWA_GetWtf (const FCSMU_Type *const *pFcsmu*)

Get the fcsmu WTF register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

Returns

WTF register value

LOCAL_INLINE void FCSMU_HWA_SetCfgTo (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu CFG_TO register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	CFG_TO register value

LOCAL_INLINE void FCSMU_HWA_SetCrcCtrl (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu CRC_CTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	CRC_CTRL register value

LOCAL_INLINE void FCSMU_HWA_SetCtrl (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu CTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	The register value

LOCAL_INLINE void FCSMU_HWA_SetFaultIen0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu FAULT_IEN0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	FAULT_IEN0 register value

LOCAL_INLINE void FCSMU_HWA_SetFccr0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu FCCR0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	FCCR0 register value

LOCAL_INLINE void FCSMU_HWA_SetFe0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu FE0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	FE0 register value

LOCAL_INLINE void FCSMU_HWA_SetFrst0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu FRST0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	FRST0 register value

LOCAL_INLINE void FCSMU_HWA_SetFst0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu FST0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
---------------	----------------

<i>u32Value</i>	FST0 register value
-----------------	---------------------

LOCAL_INLINE void FCSMU_HWA_SetFstUnlk (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu FST_UNLK register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	FST_UNLK register value

LOCAL_INLINE void FCSMU_HWA_SetInject (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu INJECT register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	INJECT register value

LOCAL_INLINE void FCSMU_HWA_SetOprk (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu OPRK register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	The register value

LOCAL_INLINE void FCSMU_HWA_SetPermntLock (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu PERMNT_LOCK register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	PERMNT_LOCK register value

LOCAL_INLINE void FCSMU_HWA_SetSoctrl (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu SOCTRL register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	The register value

LOCAL_INLINE void FCSMU_HWA_SetSoutDiag (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu SOUT_DIAG register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	SOUT_DIAG register value

LOCAL_INLINE void FCSMU_HWA_SetSoutEn0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu SOUT_EN0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	SOUT_EN0 register value

LOCAL_INLINE void FCSMU_HWA_SetStmr (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu STMR register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	STMR register value

LOCAL_INLINE void FCSMU_HWA_SetTempUnlk (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu TEMP_UNLK register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	TEMP_UNLK register value

LOCAL_INLINE void FCSMU_HWA_SetWarningEn0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu WARNING_EN0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	WARNING_EN0 register value

LOCAL_INLINE void FCSMU_HWA_SetWarningIen0 (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu WARNING_IEN0 register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	WARNING_IEN0 register value

LOCAL_INLINE void FCSMU_HWA_SetWarningTo (FCSMU_Type *const *pFcsmu*, uint32_t *u32Value*)

Set the fcsmu WARNING_TO register.

Parameters

<i>pFcsmu</i>	FCSMU Instance
<i>u32Value</i>	WARNING_TO register value

HwA_fcuart

Collaboration diagram for HwA_fcuart:Enumerations

```
enum FCUART_StatType { FCUART_STAT_LBKDIF = FCUART_STAT_LBKDIF_MASK, FCUART_STAT_RPAEIF = FCUART_STAT_RPAEIF_MASK, FCUART_STAT_MSBF = FCUART_STAT_MSBF_MASK, FCUART_STAT_RXINV = FCUART_STAT_RXINV_MASK, FCUART_STAT_RWUID = FCUART_STAT_RWUID_MASK, FCUART_STAT_BCGL = FCUART_STAT_BCGL_MASK, FCUART_STAT_LBKDE = FCUART_STAT_LBKDE_MASK, FCUART_STAT_RAF = FCUART_STAT_RAF_MASK, FCUART_STAT_TDREF = FCUART_STAT_TDREF_MASK, FCUART_STAT_TCF = FCUART_STAT_TCF_MASK, FCUART_STAT_RDRFF = FCUART_STAT_RDRFF_MASK, FCUART_STAT_IDLEF = FCUART_STAT_IDLEF_MASK, FCUART_STAT_RORF = FCUART_STAT_RORF_MASK, FCUART_STAT_NF = FCUART_STAT_NF_MASK, FCUART_STAT_FEF = FCUART_STAT_FEF_MASK, FCUART_STAT_PEF = FCUART_STAT_PEF_MASK, FCUART_STAT_M0F = FCUART_STAT_M0F_MASK, FCUART_STAT_M1F = FCUART_STAT_M1F_MASK, FCUART_STAT_RPEF = FCUART_STAT_RPEF_MASK, FCUART_STAT_TPEF = FCUART_STAT_TPEF_MASK }
```

UART STAT register flag.

```
enum FCUART_BitModeType { UART_BITMODE_8 = 0, UART_BITMODE_9 }
```

UART data bit length mode.

```
enum FCUART_StopBitNumType { UART_STOPBIT_NUM_1 = 0, UART_STOPBIT_NUM_2 }
```

UART stop bits number.

```
enum FCUART_ParityType { UART_PARITY_EVEN = 0, UART_PARITY_ODD }
```

UART parity check type.

Functions

```
LOCAL_INLINE void FCUART_HWA_SetSoftWareReset (FCUART_Type *pUart)
```

Get Stat Flag.

```
LOCAL_INLINE uint32_t FCUART_HWA_GetStatus (FCUART_Type *pUart, FCUART_StatType eStatusType)
```

Get Stat Flag.

```
LOCAL_INLINE void FCUART_HWA_ClearStatus (FCUART_Type *pUart, uint32_t u32StatusType)
```

Clear Stat Flag.

```
LOCAL_INLINE void FCUART_HWA_EnableInterrupt (FCUART_Type *pUart, uint32_t u32Value)
```

Enable Interrupt.

LOCAL_INLINE void **FCUART_HWA_DisableInterrupt** (FCUART_Type *pUart, uint32 u32Value)

Disable Interrupt.

LOCAL_INLINE void **FCUART_HWA_EnableReceiveInterrupt** (FCUART_Type *pUart)

Enable Receive Interrupt.

LOCAL_INLINE void **FCUART_HWA_DisableReceiveInterrupt** (FCUART_Type *pUart)

Disable Receive Interrupt.

LOCAL_INLINE void **FCUART_HWA_EnableErrorInterrupt** (FCUART_Type *pUart)

Enable Error Interrupt.

LOCAL_INLINE void **FCUART_HWA_DisableErrorInterrupt** (FCUART_Type *pUart)

Disable Error Interrupt.

LOCAL_INLINE void **FCUART_HWA_SetCtrl** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART Ctrl register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetCtrl** (FCUART_Type *pUart)

Get FCUART Ctrl register.

LOCAL_INLINE void **FCUART_HWA_AttachCtrl** (FCUART_Type *pUart, uint32_t u32Value)

Attach FCUART Ctrl register.

LOCAL_INLINE void **FCUART_HWA_SetBaud** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART Baud register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetBaud** (FCUART_Type *pUart)

Get FCUART baud register.

LOCAL_INLINE void **FCUART_HWA_AttachBaud** (FCUART_Type *pUart, uint32_t u32Value)

Attach FCUART Baud register.

LOCAL_INLINE void **FCUART_HWA_SetFifo** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART Fifo register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetFifo** (FCUART_Type *pUart)

Get FCUART fifo register.

LOCAL_INLINE void **FCUART_HWA_FlushTxRxFifo** (FCUART_Type *pUart)

Flush FCUART Tx Rx Fifo register.

LOCAL_INLINE bool **FCUART_HWA_GetEnStatusRxFifo** (FCUART_Type *pUart)

Get FCUART fifo register.

LOCAL_INLINE bool **FCUART_HWA_GetEnStatusTxFifo** (FCUART_Type *pUart)

Get FCUART fifo register.

LOCAL_INLINE void **FCUART_HWA_AttachFifo** (FCUART_Type *pUart, uint32_t u32Value)

Attach FCUART Fifo register.

LOCAL_INLINE void **FCUART_HWA_SetWaterMark** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART WaterMark register.

LOCAL_INLINE uint8_t **FCUART_HWA_GetRxWaterMark** (FCUART_Type *pUart)

Get FCUART Rx WaterMark.

LOCAL_INLINE uint8_t **FCUART_HWA_GetTxWaterMark** (FCUART_Type *pUart)

Get FCUART Tx WaterMark.

LOCAL_INLINE uint8_t **FCUART_HWA_GetFifoRxCount** (FCUART_Type *pUart)

Get FCUART WaterMark Rxcount.

LOCAL_INLINE uint8_t **FCUART_HWA_GetFifoTxCount** (FCUART_Type *pUart)

Get FCUART WaterMark Txcount.

LOCAL_INLINE void **FCUART_HWA_AttachWaterMark** (FCUART_Type *pUart, uint32_t u32Value)
Attach FCUART WaterMark register.

LOCAL_INLINE void **FCUART_HWA_SetMatch** (FCUART_Type *pUart, uint32_t u32Value)
Set FCUART Match register.

LOCAL_INLINE void **FCUART_HWA_AttachMatch** (FCUART_Type *pUart, uint32_t u32Value)
Attach FCUART Match register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetMatch** (FCUART_Type *pUart)
Get FCUART Match register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetSTAT** (FCUART_Type *pUart)
Read FCUART STAT register.

LOCAL_INLINE void **FCUART_HWA_WriteClearSTAT** (FCUART_Type *pUart, uint32_t u32Value)
Write 1 Clear FCUART STAT register.

LOCAL_INLINE void **FCUART_HWA_SetBitMode** (FCUART_Type *pUart, **FCUART_BitModeType** eBitMode)
Set Bit Mode and Parity.

LOCAL_INLINE void **FCUART_HWA_SetParity** (FCUART_Type *pUart, **FCUART_ParityType** eParityType,
bool bParityEnable)
Set Bit Mode and Parity.

LOCAL_INLINE void **FCUART_HWA_SetStopBit** (FCUART_Type *pUart, **FCUART_StopBitNumType**
eStopBit)
Set Bit Mode and Parity.

LOCAL_INLINE void **FCUART_HWA_EnableReceiveDMA** (FCUART_Type *pUart)
Enable Receive DMA.

LOCAL_INLINE void **FCUART_HWA_DisableReceiveDMA** (FCUART_Type *pUart)

Disable Receive DMA.

LOCAL_INLINE void **FCUART_HWA_EnableReceiveFIFO** (FCUART_Type *pUart)

Enable Receive FIFO.

LOCAL_INLINE void **FCUART_HWA_DisableReceiveFIFO** (FCUART_Type *pUart)

Disable Receive FIFO.

LOCAL_INLINE void **FCUART_HWA_ClearFIFOErrorFlag** (FCUART_Type *pUart)

Clear Fifo Overflow/Underflow flag.

LOCAL_INLINE void **FCUART_HWA_SetTxWaterMark** (FCUART_Type *pUart, uint8_t u8Value)

Set FCUART Tx WaterMark.

LOCAL_INLINE void **FCUART_HWA_SetRxWaterMark** (FCUART_Type *pUart, uint8_t u8Value)

Set FCUART Rx WaterMark.

LOCAL_INLINE void **FCUART_HWA_SetData** (FCUART_Type *pUart, uint32_t u32Data)

Set Data Value.

LOCAL_INLINE uint8_t **FCUART_HWA_GetData** (FCUART_Type *pUart)

Get Data Value.

LOCAL_INLINE void **FCUART_HWA_SetR8T9** (FCUART_Type *pUart, uint8_t u8Data)

Set R8T9 bit.

LOCAL_INLINE uint8_t **FCUART_HWA_GetR8T9** (FCUART_Type *pUart)

Get R8T9 bit.

LOCAL_INLINE void **FCUART_HWA_SetR9T8** (FCUART_Type *pUart, uint8_t u8Data)

Set R9T8 bit.

LOCAL_INLINE uint8_t **FCUART_HWA_GetR9T8** (FCUART_Type *pUart)

Get R9T8 bit.

LOCAL_INLINE void **FCUART_HWA_SoftwareReset** (FCUART_Type *pUart)

Reset the instance by software.

LOCAL_INLINE void **FCUART_HWA_SetTxTransfer** (FCUART_Type *pUart, bool bEnable)

Set fcuart TX Transfer enable or disable.

LOCAL_INLINE void **FCUART_HWA_StartTransmit** (FCUART_Type *pUart)

Start transmit.

LOCAL_INLINE void **FCUART_HWA_SetRxTransfer** (FCUART_Type *pUart, bool bEnable)

Set fcuart RX Transfer enable or disable.

LOCAL_INLINE void **FCUART_HWA_StartTransfer** (FCUART_Type *pUart)

Set fcuart start transfer.

LOCAL_INLINE void **FCUART_HWA_StopTransfer** (FCUART_Type *pUart)

Set fcuart stop transfer.

LOCAL_INLINE void **FCUART_HWA_SetLinBreakDetectInterrupt** (FCUART_Type *pUart, bool bEnable)

Set lin break detect interrupt.

LOCAL_INLINE void **FCUART_HWA_SetLinBreakDetectEnable** (FCUART_Type *pUart, bool bEnable)

Set lin break detect feature enable.

LOCAL_INLINE void **FCUART_HWA_SendBreakField** (FCUART_Type *pUart)

Send a lin break field.

LOCAL_INLINE void **FCUART_HWA_SetReceiveActiveInterrupt** (FCUART_Type *pUart, bool bEnable)

Set uart receive active interrupt.

LOCAL_INLINE bool **FCUART_HWA_GetReceiveActiveInterrupt** (FCUART_Type *pUart)

Set uart receive active interrupt.

LOCAL_INLINE void **FCUART_HWA_SetReceiveDataInverse** (FCUART_Type *pUart, bool bEnable)

Set FCUART inverse feature.

LOCAL_INLINE bool **FCUART_HWA_GetReceiveDataInverse** (FCUART_Type *pUart)

Get the FCUART inverse bit value.

LOCAL_INLINE void **FCUART_HWA_SetFrameErrorInterrupt** (FCUART_Type *pUart, bool bEnable)

Set the FCUART frame error interrupt.

LOCAL_INLINE void **FCUART_HWA_EnableLoopMode** (FCUART_Type *pUart)

Enable the FCUART loop mode.

LOCAL_INLINE void **FCUART_HWA_DisableLoopMode** (FCUART_Type *pUart)

Disable the FCUART loop mode.

LOCAL_INLINE void **FCUART_HWA_SetModir** (FCUART_Type *pUart, uint32_t u32Data)

Set FCUART MODIR value.

LOCAL_INLINE uint32_t **FCUART_HWA_GetModir** (FCUART_Type *pUart)

Get FCUART MODIR value.

Detailed Description

Enumeration Type Documentation

enum FCUART_BitModeType

UART data bit length mode.

Enumerator:

UART_BITMODE _8	UART_BITMODE_8
UART_BITMODE _9	UART_BITMODE_9

enum FCUART_ParityType

UART parity check type.

Enumerator:

UART_PARITY_E VEN	UART_PARITY_EVEN
UART_PARITY_O DD	UART_PARITY_ODD

enum FCUART_StatType

UART STAT register flag.

Enumerator:

FCUART_STAT_ LBKDIF	FCUART_STAT_LBKDIF LIN Break Detect Interrupt Flag, w1c
FCUART_STAT_ RPAEIF	FCUART_STAT_RPAEIF RXD Pin Active Edge Interrupt Flag, w1c
FCUART_STAT_ MSBF	FCUART_STAT_MSBF MSB First, RW

FCUART_STAT_ RXINV	FCUART_STAT_RXINV Receive Data Inversion, RW
FCUART_STAT_ RWUID	FCUART_STAT_RWUID Receive Wake Up Idle Detect, RW
FCUART_STAT_ BCGL	FCUART_STAT_BCGL Break Character Generation Length, RW
FCUART_STAT_ LBKDE	FCUART_STAT_LBKDE LIN Break Detection Enable, RW
FCUART_STAT_ RAF	FCUART_STAT_RAF Receiver Active Flag, RO
FCUART_STAT_ TDREF	FCUART_STAT_TDREF Transmit Data Register Empty Flag, RO
FCUART_STAT_ TCF	FCUART_STAT_TCF Transmission Complete Flag, RO
FCUART_STAT_ RDRFF	FCUART_STAT_RDRFF Receive Data Register Full Flag, RO
FCUART_STAT_ IDLEF	FCUART_STAT_IDLEF Idle Line Flag, w1c
FCUART_STAT_ RORF	FCUART_STAT_RORF Receiver Overrun Flag, w1c
FCUART_STAT_ NF	FCUART_STAT_NF Noise Flag, w1c

FCUART_STAT_ FEF	FCUART_STAT_FEF Frame Error Flag, w1c
FCUART_STAT_ PEF	FCUART_STAT_PEF Parity Error Flag, w1c
FCUART_STAT_ M0F	FCUART_STAT_M0F Match address 0 Flag, w1c
FCUART_STAT_ M1F	FCUART_STAT_M1F Match address 1 Flag, w1c
FCUART_STAT_ RPEF	FCUART_STAT_RPEF Receive Data Parity Error Flag, w1c
FCUART_STAT_ TPEF	FCUART_STAT_TPEF Transmit Data Parity Error Flag, w1c

enum FCUART_StopBitNumType

UART stop bits number.

Enumerator:

UART_STOPBIT _NUM_1	UART_STOPBIT_NUM_1
UART_STOPBIT _NUM_2	UART_STOPBIT_NUM_2

Function Documentation

LOCAL_INLINE void FCUART_HWA_AttachBaud (FCUART_Type * *pUart*, uint32_t *u32Value*)

Attach FCUART Baud register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

<i>u32Value</i>	written value
-----------------	---------------

LOCAL_INLINE void FCUART_HWA_AttachCtrl (FCUART_Type * *pUart*, uint32_t *u32Value*)

Attach FCUART Ctrl register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_AttachFifo (FCUART_Type * *pUart*, uint32_t *u32Value*)

Attach FCUART Fifo register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_AttachMatch (FCUART_Type * *pUart*, uint32_t *u32Value*)

Attach FCUART Match register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_AttachWaterMark (FCUART_Type * *pUart*, uint32_t *u32Value*)

Attach FCUART WaterMark register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_ClearFIFOErrorFlag (FCUART_Type * *pUart*)

Clear Fifo Overflow/Underflow flag.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_ClearStatus (FCUART_Type * *pUart*, uint32_t *u32StatusType*)

Clear Stat Flag.

Parameters

<i>pUart</i>	UART instance value
<i>u32StatusType</i>	stat type

LOCAL_INLINE void FCUART_HWA_DisableErrorInterrupt (FCUART_Type * *pUart*)

Disable Error Interrupt.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_DisableInterrupt (FCUART_Type * *pUart*, uint32_t *u32Value*)

Disable Interrupt.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_DisableLoopMode (FCUART_Type * *pUart*)

Disable the FCUART loop mode.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_DisableReceiveDMA (FCUART_Type * *pUart*)

Disable Receive DMA.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_DisableReceiveFIFO (FCUART_Type * *pUart*)

Disable Receive FIFO.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_DisableReceiveInterrupt (FCUART_Type * *pUart*)

Disable Receive Interrupt.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_EnableErrorInterrupt (FCUART_Type * *pUart*)

Enable Error Interrupt.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_EnableInterrupt (FCUART_Type * *pUart*, uint32 *u32Value*)

Enable Interrupt.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_EnableLoopMode (FCUART_Type * *pUart*)

Enable the FCUART loop mode.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_EnableReceiveDMA (FCUART_Type * *pUart*)

Enable Receive DMA.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_EnableReceiveFIFO (FCUART_Type * *pUart*)

Enable Receive FIFO.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_EnableReceiveInterrupt (FCUART_Type * *pUart*)

Enable Receive Interrupt.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_FlushTxRxFifo (FCUART_Type * *pUart*)

Flush FCUART Tx Rx Fifo register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE uint32_t FCUART_HWA_GetBaud (FCUART_Type * *pUart*)

Get FCUART baud register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Register value

LOCAL_INLINE uint32_t FCUART_HWA_GetCtrl (FCUART_Type * *pUart*)

Get FCUART Ctrl register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Register value

LOCAL_INLINE uint8_t FCUART_HWA_GetData (FCUART_Type * *pUart*)

Get Data Value.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

the data value

LOCAL_INLINE bool FCUART_HWA_GetEnStatusRxFifo (FCUART_Type * *pUart*)

Get FCUART fifo register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Register value

LOCAL_INLINE bool FCUART_HWA_GetEnStatusTxFifo (FCUART_Type * *pUart*)

Get FCUART fifo register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Register value

LOCAL_INLINE uint32_t FCUART_HWA_GetFifo (FCUART_Type * *pUart*)

Get FCUART fifo register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Register value

LOCAL_INLINE uint8_t FCUART_HWA_GetFifoRxCount (FCUART_Type * *pUart*)

Get FCUART WaterMark Rxcount.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Rxcount value

LOCAL_INLINE uint8_t FCUART_HWA_GetFifoTxCount (FCUART_Type * *pUart*)

Get FCUART WaterMark Txcount.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Rxcount value

LOCAL_INLINE uint32_t FCUART_HWA_GetMatch (FCUART_Type * *pUart*)

Get FCUART Match register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Register value

LOCAL_INLINE uint32_t FCUART_HWA_GetModir (FCUART_Type * *pUart*)

Get FCUART MODIR value.

Parameters

<i>pUart</i>	UART instance value
<i>u32Data</i>	Get data

LOCAL_INLINE uint8_t FCUART_HWA_GetR8T9 (FCUART_Type * *pUart*)

Get R8T9 bit.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

the data value

LOCAL_INLINE uint8_t FCUART_HWA_GetR9T8 (FCUART_Type * *pUart*)

Get R9T8 bit.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

the data value

LOCAL_INLINE bool FCUART_HWA_GetReceiveActiveInterrupt (FCUART_Type * *pUart*)

Set uart receive active interrupt.

Parameters

<i>pUart</i>	UART instance value
<i>return</i>	false for disable, true for enable.

LOCAL_INLINE bool FCUART_HWA_GetReceiveDataInverse (FCUART_Type * *pUart*)

Get the FCUART inverse bit value.

Parameters

<i>pUart</i>	UART instance value
<i>return</i>	false for disable, true for enable.

LOCAL_INLINE uint8_t FCUART_HWA_GetRxWaterMark (FCUART_Type * *pUart*)

Get FCUART Rx WaterMark.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Rxcount value

LOCAL_INLINE uint32_t FCUART_HWA_GetSTAT (FCUART_Type * *pUart*)

Read FCUART STAT register.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

STAT read value

LOCAL_INLINE uint32_t FCUART_HWA_GetStatus (FCUART_Type * *pUart*, FCUART_StatType *eStatusType*)

Get Stat Flag.

Parameters

<i>pUart</i>	UART instance value
<i>eStatusType</i>	stat type

Returns

FCUART STAT status flag

LOCAL_INLINE uint8_t FCUART_HWA_GetTxWaterMark (FCUART_Type * *pUart*)

Get FCUART Tx WaterMark.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

Rxcount value

LOCAL_INLINE void FCUART_HWA_SendBreakField (FCUART_Type * *pUart*)

Send a lin break field.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_SetBaud (FCUART_Type * *pUart*, uint32_t *u32Value*)

Set FCUART Baud register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_SetBitMode (FCUART_Type * *pUart*, FCUART_BitModeType *eBitMode*)

Set Bit Mode and Parity.

Parameters

<i>pUart</i>	UART instance value
<i>eBitMode</i>	is bit mode, 8 or 9 bits

LOCAL_INLINE void FCUART_HWA_SetCtrl (FCUART_Type * *pUart*, uint32_t *u32Value*)

Set FCUART Ctrl register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_SetData (FCUART_Type * *pUart*, uint32_t *u32Data*)

Set Data Value.

Parameters

<i>pUart</i>	UART instance value
<i>u32Data</i>	Set data

LOCAL_INLINE void FCUART_HWA_SetFifo (FCUART_Type * *pUart*, uint32_t *u32Value*)

Set FCUART Fifo register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_SetFrameErrorInterrupt (FCUART_Type * *pUart*, bool *bEnable*)

Set the FCUART frame error interrupt.

Parameters

<i>pUart</i>	UART instance value
<i>bEnable</i>	false for disable, true for enable.

LOCAL_INLINE void FCUART_HWA_SetLinBreakDetectEnable (FCUART_Type * *pUart*, bool *bEnable*)

Set lin break detect feature enable.

Parameters

<i>pUart</i>	UART instance value
<i>bEnable</i>	Enable cmd, false for disable, true for enable.

LOCAL_INLINE void FCUART_HWA_SetLinBreakDetectInterrupt (FCUART_Type * *pUart*, bool *bEnable*)

Set lin break detect interrupt.

Parameters

<i>pUart</i>	UART instance value
<i>bEnable</i>	Enable cmd, false for disable, true for enable.

LOCAL_INLINE void FCUART_HWA_SetMatch (FCUART_Type * *pUart*, uint32_t *u32Value*)

Set FCUART Match register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_SetModir (FCUART_Type * *pUart*, uint32_t *u32Data*)

Set FCUART MODIR value.

Parameters

<i>pUart</i>	UART instance value
<i>u32Data</i>	Set data

LOCAL_INLINE void FCUART_HWA_SetParity (FCUART_Type * *pUart*, FCUART_ParityType *eParityType*, bool *bParityEnable*)

Set Bit Mode and Parity.

Parameters

<i>pUart</i>	UART instance value
<i>bParityEnable</i>	If enable Parity, set 1U, or set 0U
<i>eParityType</i>	Parity type, odd-even

LOCAL_INLINE void FCUART_HWA_SetR8T9 (FCUART_Type * *pUart*, uint8_t *u8Data*)

Set R8T9 bit.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

the data value

LOCAL_INLINE void FCUART_HWA_SetR9T8 (FCUART_Type * *pUart*, uint8_t *u8Data*)

Set R9T8 bit.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

Returns

the data value

LOCAL_INLINE void FCUART_HWA_SetReceiveActiveInterrupt (FCUART_Type * *pUart*, bool *bEnable*)

Set uart receive active interrupt.

Parameters

<i>pUart</i>	UART instance value
<i>bEnable</i>	Enable cmd, false for disable, true for enable.

LOCAL_INLINE void FCUART_HWA_SetReceiveDataInverse (FCUART_Type * *pUart*, bool *bEnable*)

Set FCUART inverse feature.

Parameters

<i>pUart</i>	UART instance value
<i>bEnable</i>	false for disable, true for enable.

LOCAL_INLINE void FCUART_HWA_SetRxTransfer (FCUART_Type * *pUart*, bool *bEnable*)

Set fcuart RX Transfer enable or disable.

Parameters

<i>pUart</i>	UART instance value
<i>bEnable</i>	Enable cmd, false for disable, true for enable.

LOCAL_INLINE void FCUART_HWA_SetRxWaterMark (FCUART_Type * *pUart*, uint8_t *u8Value*)

Set FCUART Rx WaterMark.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_SetSoftWareReset (FCUART_Type * *pUart*)

Get Stat Flag.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_SetStopBit (FCUART_Type * *pUart*, FCUART_StopBitNumType *eStopBit*)

Set Bit Mode and Parity.

Parameters

<i>pUart</i>	UART instance value
<i>eStopBit</i>	stop bits number 1 or 2 bits

LOCAL_INLINE void FCUART_HWA_SetTxTransfer (FCUART_Type * *pUart*, bool *bEnable*)

Set fcuart TX Transfer enable or disable.

Parameters

<i>pUart</i>	UART instance value
<i>bEnable</i>	Enable cmd, false for disable, true for enable.

LOCAL_INLINE void FCUART_HWA_SetTxWaterMark (FCUART_Type * *pUart*, uint8_t *u8Value*)

Set FCUART Tx WaterMark.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_SetWaterMark (FCUART_Type * *pUart*, uint32_t *u32Value*)

Set FCUART WaterMark register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FCUART_HWA_SoftwareReset (FCUART_Type * *pUart*)

Reset the instance by software.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_StartTransfer (FCUART_Type * *pUart*)

Set fcuart start transfer.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_StartTransmit (FCUART_Type * *pUart*)

Start transmit.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_StopTransfer (FCUART_Type * *pUart*)

Set fcuart stop transfer.

Parameters

<i>pUart</i>	UART instance value
--------------	---------------------

LOCAL_INLINE void FCUART_HWA_WriteClearSTAT (FCUART_Type * *pUart*, uint32_t *u32Value*)

Write 1 Clear FCUART STAT register.

Parameters

<i>pUart</i>	UART instance value
<i>u32Value</i>	written value

HwA_flexcan

Collaboration diagram for HwA_flexcan:Macros

```
#define FLEXCAN_REGISTER_WITHOFFSET2(reg, offset1) ((uint32_t)(reg)+(uint32_t)(offset1))
#define FLEXCAN_REGISTER_WITHOFFSET(reg, offset1,
offset2) ((uint32_t)(reg)+(uint32_t)(offset1)+(uint32_t)(offset2))
#define FLEXCAN_REG32_CONTENT(reg) (*((uint32_t*)(reg)))
#define FLEXCAN_U32MACRO(var) ((uint32_t)(var))
U32 value around.

#define FLEXCAN_U32SHIFT(var, mask, shift) ((uint32_t)((((uint32_t)(var))&(mask))<<(shift)))
```

U32 value shift.

```
#define FLEXCAN_U32NOTSHIFT(mask, shift) (((uint32_t)(~(((uint32_t)(mask))<<(shift))))
```

U32 value shift and revert.

```
#define FLEXCAN_MB_SRR_MASK_U32 0x00400000U
```

```
#define FLEXCAN_ONE_REGION_SIZE 0x200U
```

```
#define FLEXCAN_MB_HEADER_BYTELEN 8UL
```

Legacy/Enhanced Message Buffer Header.

```
#define FLEXCAN_MB_ADDR(mbOffset, mbIndex, dataLen)
```

Legacy Message Buffer Address.

```
#define FLEXCAN_MB_WORDN_ADDR(mbOffset, mbIndex, dataLen,  
offset) (FLEXCAN_MB_ADDR((mbOffset), (mbIndex), (dataLen))+FLEXCAN_U32MACRO(offset))
```

Legacy Message Buffer word n Address.

```
#define FLEXCAN_MB_GET_DATALEN(mbindex, mbNum, datalenNum,  
datalenarray) ((uint8_t)(datalenarray[((mbindex)*(datalenNum)/(mbNum)) ]]))
```

check mb data length

```
#define FLEXCAN_MB_EDL_GET(edlAddr) ((FLEXCAN_REG32_CONTENT(edlAddr)>>31)&0x01U)
```

Legacy Message Buffer Data Address.

```
#define FLEXCAN_MB_EDL_CLR(edlAddr) (FLEXCAN_REG32_CONTENT(edlAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,31))
```

Legacy Message Buffer EDL Clear.

```
#define FLEXCAN_MB_EDL_ATTACH(edlAddr, var) (FLEXCAN_REG32_CONTENT(edlAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,31))
```

Legacy Message Buffer EDL Attach.

```
#define FLEXCAN_MB_EDL_SET(edlAddr, var) FLEXCAN_REG32_CONTENT(edlAddr) =  
((FLEXCAN_REG32_CONTENT(edlAddr) & FLEXCAN_U32NOTSHIFT(0x01U,31)) |  
FLEXCAN_U32SHIFT(var,0x01U,31))
```

Legacy Message Buffer EDL Set.

```
#define FLEXCAN_MB_BRS_GET(brsAddr) (((FLEXCAN_REG32_CONTENT(brsAddr))>>30)&0x01U)
```

Legacy Message Buffer BRS Get.

```
#define FLEXCAN_MB_BRS_CLR(brsAddr) (FLEXCAN_REG32_CONTENT(brsAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,30))
```

Legacy Message Buffer BRS Clear.

```
#define FLEXCAN_MB_BRS_ATTACH(brsAddr, var) (FLEXCAN_REG32_CONTENT(brsAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,30))
```

Legacy Message Buffer BRS Attach.

```
#define FLEXCAN_MB_BRS_SET(brsAddr, var) FLEXCAN_REG32_CONTENT(brsAddr) =  
((FLEXCAN_REG32_CONTENT(brsAddr) & FLEXCAN_U32NOTSHIFT(0x01U,30)) |  
FLEXCAN_U32SHIFT(var,0x01U,30))
```

Legacy Message Buffer BRS set.

```
#define FLEXCAN_MB_ESI_GET(esiAddr) (((FLEXCAN_REG32_CONTENT(esiAddr))>>29)&0x01U)
```

Legacy Message Buffer ESI Get.

```
#define FLEXCAN_MB_ESI_CLR(esiAddr) (FLEXCAN_REG32_CONTENT(esiAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,29))
```

Legacy Message Buffer ESI Clear.

```
#define FLEXCAN_MB_ESI_ATTACH(esiAddr, var) (FLEXCAN_REG32_CONTENT(esiAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,29))
```

Legacy Message Buffer ESI Attach.

```
#define FLEXCAN_MB_ESI_SET(esiAddr, var) FLEXCAN_REG32_CONTENT(esiAddr) =  
((FLEXCAN_REG32_CONTENT(esiAddr) & FLEXCAN_U32NOTSHIFT(0x01U,29)) |  
FLEXCAN_U32SHIFT(var,0x01U,29))
```

Legacy Message Buffer ESI Set.

```
#define FLEXCAN_MB_CODE_GET(codeAddr) (((FLEXCAN_REG32_CONTENT(codeAddr))>>24)&0x0EU)
```

Legacy Message Buffer CODE Get.

```
#define FLEXCAN_MB_CODE_CLR(codeAddr) (FLEXCAN_REG32_CONTENT(codeAddr) &=  
FLEXCAN_U32NOTSHIFT(0x0FU,24))
```

Legacy Message Buffer CODE Clear.

```
#define FLEXCAN_MB_CODE_ATTACH(codeAddr, var) (FLEXCAN_REG32_CONTENT(codeAddr) |=  
FLEXCAN_U32SHIFT(var,0x0FU,24))
```

Legacy Message Buffer CODE Attach.

```
#define FLEXCAN_MB_CODE_SET(codeAddr, var) FLEXCAN_REG32_CONTENT(codeAddr) =  
(((FLEXCAN_REG32_CONTENT(codeAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,24)) |  
FLEXCAN_U32SHIFT(var,0x0FU,24))
```

Legacy Message Buffer CODE set.

```
#define FLEXCAN_MB_SRR_GET(srrAddr) (((FLEXCAN_REG32_CONTENT(srrAddr)) >>22)&0x01U)
```

Legacy Message Buffer SRR Get.

```
#define FLEXCAN_MB_SRR_CLR(srrAddr) (FLEXCAN_REG32_CONTENT(srrAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,22))
```

Legacy Message Buffer SRR Clear.

```
#define FLEXCAN_MB_SRR_ATTACH(srrAddr, var) (FLEXCAN_REG32_CONTENT(srrAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,22))
```

Legacy Message Buffer SRR Attach.

```
#define FLEXCAN_MB_SRR_SET(srrAddr, var) FLEXCAN_REG32_CONTENT(srrAddr) =  
(((FLEXCAN_REG32_CONTENT(srrAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,22)) |  
FLEXCAN_U32SHIFT(var,0x0FU,22))
```

Legacy Message Buffer SRR set.

```
#define FLEXCAN_MB_IDE_GET(ideAddr) (((FLEXCAN_REG32_CONTENT(ideAddr))>>21)&0x01U)
```

Legacy Message Buffer IDE Get.

```
#define FLEXCAN_MB_IDE_CLR(ideAddr) (FLEXCAN_REG32_CONTENT(ideAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,21))
```

Legacy Message Buffer IDE Clear.

```
#define FLEXCAN_MB_IDE_ATTACH(ideAddr, var) (FLEXCAN_REG32_CONTENT(ideAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,21))
```

Legacy Message Buffer IDE Attach.

```
#define FLEXCAN_MB_IDE_SRR_SET(ideAddr, var) FLEXCAN_REG32_CONTENT(ideAddr) =  
((FLEXCAN_REG32_CONTENT(ideAddr) & FLEXCAN_U32NOTSHIFT(0x03U,21)) |  
FLEXCAN_U32SHIFT(var,0x03U,21))
```

Legacy Message Buffer IDE set with SRR.

```
#define FLEXCAN_MB_RTR_GET(rtrAddr) (((FLEXCAN_REG32_CONTENT(rtrAddr))>>20)&0x01U)
```

Legacy Message Buffer RTR Get.

```
#define FLEXCAN_MB_RTR_CLR(rtrAddr) (FLEXCAN_REG32_CONTENT(rtrAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,20))
```

Legacy Message Buffer RTR Clear.

```
#define FLEXCAN_MB_RTR_ATTACH(rtrAddr, var) (FLEXCAN_REG32_CONTENT(rtrAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,20))
```

Legacy Message Buffer RTR Attach.

```
#define FLEXCAN_MB_RTR_SET(rtrAddr, var) FLEXCAN_REG32_CONTENT(rtrAddr) =  
((FLEXCAN_REG32_CONTENT(rtrAddr) & FLEXCAN_U32NOTSHIFT(0x01U,20)) |  
FLEXCAN_U32SHIFT(var,0x01U,20))
```

Legacy Message Buffer RTR set.

```
#define FLEXCAN_MB_DLC_GET(dlcAddr) (((FLEXCAN_REG32_CONTENT(dlcAddr))>>16)&0x0FU)
```

Legacy Message Buffer DLC Get.

```
#define FLEXCAN_MB_DLC_CLR(dlcAddr) (FLEXCAN_REG32_CONTENT(dlcAddr) &=  
FLEXCAN_U32NOTSHIFT(0x0FU,16))
```

Legacy Message Buffer DLC Clear.

```
#define FLEXCAN_MB_DLC_ATTACH(dlcAddr, var) (FLEXCAN_REG32_CONTENT(dlcAddr) |=  
FLEXCAN_U32SHIFT(var,0x0FU,16))
```

Legacy Message Buffer DLC Attach.

```
#define FLEXCAN_MB_DLC_SET(dlcAddr, var) FLEXCAN_REG32_CONTENT(dlcAddr) =  
((FLEXCAN_REG32_CONTENT(dlcAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,16)) |  
FLEXCAN_U32SHIFT(var,0x0FU,16))
```

Legacy Message Buffer DLC Set.

```
#define  
FLEXCAN_MB_TIMESTAMP_GET(timestampAddr) (FLEXCAN_REG32_CONTENT(timestampAddr)&0xFFFF  
U)
```

Legacy Message Buffer TIME STAMP Get.

```
#define FLEXCAN_MB_TIMESTAMP_CLR(timestampAddr) (FLEXCAN_REG32_CONTENT(timestampAddr)  
&= FLEXCAN_U32NOTSHIFT(0xFFFFU,0))
```

Legacy Message Buffer TIME STAMP Clear.

```
#define FLEXCAN_MB_TIMESTAMP_ATTACH(timestampAddr,  
var) (FLEXCAN_REG32_CONTENT(timestampAddr) |= FLEXCAN_U32SHIFT(var,0xFFFFU,0))
```

Legacy Message Buffer TIME STAMP Attach.

```
#define FLEXCAN_MB_TIMESTAMP_SET(timestampAddr,  
var) FLEXCAN_REG32_CONTENT(timestampAddr) = ((FLEXCAN_REG32_CONTENT(timestampAddr) &  
FLEXCAN_U32NOTSHIFT(0xFFFFU,0)) | FLEXCAN_U32SHIFT(var,0xFFFFU,0))
```

Legacy Message Buffer TIME STAMP Set.

```
#define FLEXCAN_MB_PRIO_GET(prioAddr) (((FLEXCAN_REG32_CONTENT(prioAddr))>>29)&0x07U)
```

Legacy Message Buffer PRIO Get.

```
#define FLEXCAN_MB_PRIO_CLR(prioAddr) (FLEXCAN_REG32_CONTENT(prioAddr) &=  
FLEXCAN_U32NOTSHIFT(0x07U,29))
```

Legacy Message Buffer PRIO Clear.

```
#define FLEXCAN_MB_PRIO_ATTACH(prioAddr, var) (FLEXCAN_REG32_CONTENT(prioAddr) |=  
FLEXCAN_U32SHIFT(var,0x07U,29))
```

Legacy Message Buffer PRIO Attach.

```
#define FLEXCAN_MB_PRIO_SET(prioAddr, var) FLEXCAN_REG32_CONTENT(prioAddr) =  
((FLEXCAN_REG32_CONTENT(prioAddr) & FLEXCAN_U32NOTSHIFT(0x07U,29)) |  
FLEXCAN_U32SHIFT(var,0x07U,29))
```

Legacy Message Buffer PRIO Set.

```
#define FLEXCAN_MB_STDID_GET(stdidAddr) (((FLEXCAN_REG32_CONTENT(stdidAddr))>>18)&0xFFU)
```

Legacy Message Buffer STD ID Get.

```
#define FLEXCAN_MB_STDID_CLR(stdidAddr) (FLEXCAN_REG32_CONTENT(stdidAddr) &=  
FLEXCAN_U32NOTSHIFT(0xFFU,18))
```

Legacy Message Buffer STD ID CLR.

```
#define FLEXCAN_MB_STDID_ATTACH(stdidAddr, var) (FLEXCAN_REG32_CONTENT(stdidAddr) |=  
FLEXCAN_U32SHIFT(var,0xFFU,18))
```

Legacy Message Buffer STD ID Attach.

```
#define FLEXCAN_MB_STDID_SET(stdidAddr, var) FLEXCAN_REG32_CONTENT(stdidAddr) =  
((FLEXCAN_REG32_CONTENT(stdidAddr) & FLEXCAN_U32NOTSHIFT(0xFFU,18)) |  
FLEXCAN_U32SHIFT(var,0xFFU,18))
```

Legacy Message Buffer STD ID set.

```
#define FLEXCAN_MB_EXTID_GET(extidAddr) ((FLEXCAN_REG32_CONTENT(extidAddr))&0xFFFFFFFF)
```

Legacy Message Buffer EXT ID Get.

```
#define FLEXCAN_MB_EXTID_CLR(extidAddr) (FLEXCAN_REG32_CONTENT(extidAddr) &=  
FLEXCAN_U32NOTSHIFT(0xFFFFFFFF,0))
```

Legacy Message Buffer EXT ID Clear.

```
#define FLEXCAN_MB_EXTID_ATTACH(extidAddr, var) (FLEXCAN_REG32_CONTENT(extidAddr) |=  
FLEXCAN_U32SHIFT(var,0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID Attach.

```
#define FLEXCAN_MB_EXTID_SET(extidAddr, var) FLEXCAN_REG32_CONTENT(extidAddr) =  
((FLEXCAN_REG32_CONTENT(extidAddr) & FLEXCAN_U32NOTSHIFT(0x1FFFFFFF,0)) |  
FLEXCAN_U32SHIFT(var,0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID set.

```
#define FLEXCAN_EFIFOMB_EDL_GET() ((pCan->ERX_FIFO[0U]>>31)&0x01U)
```

Enhanced FIFO Message Buffer EDL Get.

```
#define FLEXCAN_EFIFOMB_BRS_GET() ((pCan->ERX_FIFO[0U]>>30)&0x01U)
```

Enhanced FIFO Message Buffer BRS Get.

```
#define FLEXCAN_EFIFOMB_ESI_GET() ((pCan->ERX_FIFO[0U]>>29)&0x01U)
```

Enhanced FIFO Message Buffer ESI Get.

```
#define FLEXCAN_EFIFOMB_SRR_GET() ((pCan->ERX_FIFO[0U]>>22)&0x01U)
```

Enhanced FIFO Message Buffer SRR Get.

```
#define FLEXCAN_EFIFOMB_IDE_GET() ((pCan->ERX_FIFO[0U]>>21)&0x01U)
```

Enhanced FIFO Message Buffer IDE Get.

```
#define FLEXCAN_EFIFOMB_RTR_GET() ((pCan->ERX_FIFO[0U]>>20)&0x01U)
```

Enhanced FIFO Message Buffer RTR Get.

```
#define FLEXCAN_EFIFOMB_DLC_GET() ((pCan->ERX_FIFO[0U]>>16)&0x0FU)
```

Enhanced FIFO Message Buffer DLC Get.

```
#define FLEXCAN_EFIFOMB_TIMESTAMP_GET() (pCan->ERX_FIFO[0U]&0xFFU)
```

Enhanced FIFO Message Buffer TIME STAMP Get.

#define FLEXCAN_EFIFOMB_STDID_GET() ((pCan->ERX_FIFO[1U]>>18)&0x7FFU)

Enhanced FIFO Message Buffer STD ID Get.

#define FLEXCAN_EFIFOMB_EXTID_GET() (pCan->ERX_FIFO[1U]&0x1FFFFFFF)

Enhanced FIFO Message Buffer EXT ID Get.

#define FLEXCAN_EFIFOMB_DATAADDR_GET(wordIndex) (&(pCan->ERX_FIFO[2U+(wordIndex)]))

Enhanced FIFO Message Buffer Data Get.

#define FLEXCAN_EFIFOMB_IDHIT_GET(dataWordLen) ((pCan->ERX_FIFO[2U+(dataWordLen)])&0x1FFU)

Enhanced FIFO Message Buffer IDHIT Get.

#define FLEXCAN_EFIFOMB_HRTIMESTAMP_GET(dataWordLen) (pCan->ERX_FIFO[3U+(dataWordLen)])

Enhanced FIFO Message Buffer HR Time Stamp Get.

#define FLEXCAN_EFIFOMB_EDL_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>31)&0x01U)

Enhanced FIFO Message Buffer EDL Get.

#define FLEXCAN_EFIFOMB_BRS_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>30)&0x01U)

Enhanced FIFO Message Buffer BRS Get.

#define FLEXCAN_EFIFOMB_ESI_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>29)&0x01U)

Enhanced FIFO Message Buffer ESI Get.

#define FLEXCAN_EFIFOMB_SRR_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>22)&0x01U)

Enhanced FIFO Message Buffer SRR Get.

#define FLEXCAN_EFIFOMB_IDE_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>21)&0x01U)

Enhanced FIFO Message Buffer IDE Get.

#define FLEXCAN_EFIFOMB_RTR_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>20)&0x01U)

Enhanced FIFO Message Buffer RTR Get.

```
#define FLEXCAN_EFIFOMB_DLC_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>16)&0x0FU)
```

Enhanced FIFO Message Buffer DLC Get.

```
#define FLEXCAN_EFIFOMB_TIMESTAMP_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]&0xFFU)
```

Enhanced FIFO Message Buffer TIME STAMP Get.

```
#define FLEXCAN_EFIFOMB_STDID_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[1U]>>18)&0x7FFU)
```

Enhanced FIFO Message Buffer STD ID Get.

```
#define FLEXCAN_EFIFOMB_EXTID_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[1U]&0x1FFFFFFF)
```

Enhanced FIFO Message Buffer EXT ID Get.

```
#define FLEXCAN_EFIFOMB_DATAADDR_GET_FROM_BUFFER(Addr,  
wordIndex) (&(((uint32_t*)Addr)[2U+(wordIndex)]))
```

Enhanced FIFO Message Buffer Data Get.

```
#define FLEXCAN_EFIFOMB_IDHIT_GET_FROM_BUFFER(Addr,  
dataWordLen) (((uint32_t*)Addr)[2U+(dataWordLen)])&0x1FFU)
```

Enhanced FIFO Message Buffer IDHIT Get.

```
#define FLEXCAN_EFIFOMB_HRTIMESTAMP_GET_FROM_BUFFER(Addr,  
dataWordLen) (((uint32_t*)Addr)[3U+(dataWordLen)])
```

Enhanced FIFO Message Buffer HR Time Stamp Get.

Enumerations

```
enum FLEXCAN_ClockSrcType { FLEXCAN_CLOCK_CANPE = 0x00U, FLEXCAN_CLOCK_INTERFACE =  
0x01U }
```

FLEXCAN Clock Source Select Enumeration.

```
enum FLEXCAN_DataWidthType { FLEXCAN_DATAWIDTH_8 = 8U, FLEXCAN_DATAWIDTH_16 = 16U,  
FLEXCAN_DATAWIDTH_32 = 32U, FLEXCAN_DATAWIDTH_64 = 64U }
```

FLEXCAN data length type.

```
enum FLEXCAN_PNET_FilterType { FLEXCAN_PNET_ID_FILTER = 0U,  
FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER = 1U, FLEXCAN_PNET_ID_FILTER_WITH_NCOUNT = 2U,  
FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER_WITH_NCOUNT = 3U }
```

CAN Pretended Network Filter type.

```
enum FLEXCAN_PNET_IdFilterType { FLEXCAN_PNET_ID_FILTER_SAME = 0U,  
FLEXCAN_PNET_ID_FILTER_GREATER_EQUAL = 1U, FLEXCAN_PNET_ID_FILTER_SMALLER_EQUAL = 2U,  
FLEXCAN_PNET_ID_FILTER_RANGE = 3U }
```

CAN Pretended Network ID Filter type.

```
enum FLEXCAN_PNET_PayloadFilterType { FLEXCAN_PNET_PAYLOAD_FILTER_SAME = 0U,  
FLEXCAN_PNET_PAYLOAD_FILTER_GREATER_EQUAL = 1U,  
FLEXCAN_PNET_PAYLOAD_FILTER_SMALLER_EQUAL = 2U, FLEXCAN_PNET_PAYLOAD_FILTER_RANGE =  
3U }
```

CAN Pretended Network Payload Filter type.

Functions

```
LOCAL_INLINE void FLEXCAN_HWA_SetMcrDisable (FLEXCAN_Type *const pCan, uint8_t bDisable)
```

Set Module Disable.

```
LOCAL_INLINE void FLEXCAN_HWA_SetMcrFreeze (FLEXCAN_Type *const pCan, uint8_t bFreeze)
```

Set Module Freeze.

```
LOCAL_INLINE void FLEXCAN_HWA_SetHaltFreeze (FLEXCAN_Type *const pCan)
```

Set Module Halt and Freeze.

```
LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrFreezen (FLEXCAN_Type *const pCan, uint32_t  
u32TimeoutTick)
```

Wait FLEXCAN Module Enter Freezen mode.

```
LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrExitFreezen (FLEXCAN_Type *const pCan, uint32_t  
u32TimeoutTick)
```

Wait FLEXCAN Module Exit Freezen mode.

```
LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrReady (FLEXCAN_Type *const pCan, uint32_t  
u32TimeoutTick)
```

Wait FLEXCAN Module Ready.

```
LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrNoReady (FLEXCAN_Type *const pCan, uint32_t  
u32TimeoutTick)
```

Wait FLEXCAN Module Not Ready.

LOCAL_INLINE void **FLEXCAN_HWA_SetMCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FLEXCAN MCR value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetMCR** (FLEXCAN_Type *const pCan)

Get FLEXCAN MCR value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachMCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attach FLEXCAN MCR value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoEn** (FLEXCAN_Type *const pCan)

Check FLEXCAN LegacyFifo Enabled.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckFifoDma** (FLEXCAN_Type *const pCan)

Check if fifo dma enable.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckPNETAck** (FLEXCAN_Type *const pCan, uint8_t bEntryPNET,
uint32_t u32TimeoutTick)

Check if it is in pretended mode.

LOCAL_INLINE void **FLEXCAN_HWA_SetCtrl1BaudSrc** (FLEXCAN_Type *const pCan,
FLEXCAN_ClockSrcType tBaudClk)

Set FLEXCAN baudrate clock source.

LOCAL_INLINE void **FLEXCAN_HWA_SetCtrl1** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FLEXCAN ctrl1 value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachCtrl1** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attach FLEXCAN ctrl1 value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetCtrl1** (FLEXCAN_Type *const pCan)

Get FLEXCAN ctrl1 value.

LOCAL_INLINE void **FLEXCAN_HWA_SetCtrl2** (FLEXCAN_Type *const pCan, uint32_t u32Value)
Set CTRL2 value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachCtrl2** (FLEXCAN_Type *const pCan, uint32_t u32Value)
Attach FLEXCAN ctrl2 value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetCtrl2** (FLEXCAN_Type *const pCan)
Get CTRL2 value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckErrorInterrupt** (FLEXCAN_Type *const pCan)
Check Interrupt Mask.

LOCAL_INLINE void **FLEXCAN_HWA_SetFlag1** (FLEXCAN_Type *const pCan, uint32_t u32W1cFlag)
Clear IFLAG1 Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetFlag1** (FLEXCAN_Type *const pCan)
Get IFLAG1 Flag.

LOCAL_INLINE void **FLEXCAN_HWA_UnlockMbNoFifoFlag** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex)
Clear Rx Flag and unlock MB.

LOCAL_INLINE void **FLEXCAN_HWA_W1cFlag1NoFifoFlag** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex)
Clear MB Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetFlag1NoFifoFlag** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex)
Get MB Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoData** (FLEXCAN_Type *const pCan)
Check if legacy fifo contains at least one data.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoWarning** (FLEXCAN_Type *const pCan)

Check if legacy fifo contains 5 data and trigger warning.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoDataLost** (FLEXCAN_Type *const pCan)

Check if legacy fifo data lost.

LOCAL_INLINE void **FLEXCAN_HWA_ClearIFLAG1FifoFlag** (FLEXCAN_Type *const pCan)

Clear Legacy Fifo Rx data available Flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClearIFLAG1FifoWarningFlag** (FLEXCAN_Type *const pCan)

Clear Legacy Fifo Rx warning Flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClearIFLAG1FifoErrorFlag** (FLEXCAN_Type *const pCan)

Clear Legacy Fifo Rx error Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoDataPolling** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data Valid Polling Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoWarningPolling** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data warning Polling Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoErrorPolling** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data lost Polling Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoDataInterrupt** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data Valid Interrupt Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoWarningInterrupt** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data warning interrupt Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoErrorInterrupt** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data lost interrupt Flag.

LOCAL_INLINE void **FLEXCAN_HWA_SetMask1MbInterrupt** (FLEXCAN_Type *const pCan, uint32_t u32Mask)

Set special message buffer interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_AttachMask1MbInterrupt** (FLEXCAN_Type *const pCan, uint32_t u32Mask)

Attach special message buffer interrupt MASK1.

LOCAL_INLINE void **FLEXCAN_HWA_AttachMbIndexInterrupt** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex)

Attach special message buffer interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetMaskFifoInterrupt** (FLEXCAN_Type *const pCan)

Set Legacy Fifo Rx interrupt.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetMask1** (FLEXCAN_Type *const pCan)

Get special message buffer interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set CBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetFDCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FDCBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetFDCTRL** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FDCTRL value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CalDataWidth** (uint8_t *pDataWidthArr)

Set MB data width in MBDSR.

LOCAL_INLINE void **FLEXCAN_HWA_SetEPRS** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set EPRS value.

LOCAL_INLINE void **FLEXCAN_HWA_SetENCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set ENCBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetEDCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set EDCBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetETDC** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set ETDC value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetTimeStamp** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN rx timestamp.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetErrorInfo** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN error.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetErrorIntInfo** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN error with interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_ClrErrorInfo** (FLEXCAN_Type *const pCan, uint32_t u32ErrorEsr1)

Clear Error Info.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetErrorCount** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN error count.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetERFSR** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN enhanced fifo error count.

LOCAL_INLINE void **FLEXCAN_HWA_EnableLegacyFifo** (FLEXCAN_Type *const pCan)

Enable FLEXCAN Legacy Fifo.

LOCAL_INLINE void **FLEXCAN_HWA_DisableLegacyFifo** (FLEXCAN_Type *const pCan)

Disable FLEXCAN Legacy Fifo.

LOCAL_INLINE void **FLEXCAN_HWA_AttachERFCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attach FLEXCAN ERFCR value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFIFOEn** (FLEXCAN_Type *const pCan)

Check Enhanced FIFO Enable.

LOCAL_INLINE void **FLEXCAN_HWA_EnableFDFifo** (FLEXCAN_Type *const pCan)

Enable FLEXCAN FD Fifo.

LOCAL_INLINE void **FLEXCAN_HWA_DisableFDFifo** (FLEXCAN_Type *const pCan)

Disable FLEXCAN FD Fifo.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckLegacyFifoEnabled** (FLEXCAN_Type *const pCan)

Check FLEXCAN legacy fifo if enabled.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckFdFifoEnabled** (FLEXCAN_Type *const pCan)

Check FLEXCAN FD fifo if enabled.

LOCAL_INLINE void **FLEXCAN_HWA_SetLegacyFifoFilterNum** (FLEXCAN_Type *const pCan, uint32_t u32RFFN)

Set legacy fifo filter number number.

LOCAL_INLINE void **FLEXCAN_HWA_SetLegacyFifoFilterNum1** (FLEXCAN_Type *const pCan, uint32_t u32FilterNum)

Set legacy fifo filter number number.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckFd** (FLEXCAN_Type *const pCan)

Check FLEXCAN FD enable.

LOCAL_INLINE void **FLEXCAN_HWA_SetIndividualMask** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex, uint32_t u32Mask)

Set RXIMR for FLEXCAN.

LOCAL_INLINE void **FLEXCAN_HWA_SetMBGlobalMask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)

Set MB global mask value.

LOCAL_INLINE void **FLEXCAN_HWA_SetFifoGlobalMask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)
Set Fifo global mask value.

LOCAL_INLINE void **FLEXCAN_HWA_SetRx14Mask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)
Set rx14 mask value.

LOCAL_INLINE void **FLEXCAN_HWA_SetRx15Mask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)
Set rx15 mask value.

LOCAL_INLINE void **FLEXCAN_HWA_MbRam** (FLEXCAN_Type *const pCan, uint32_t u32RamIndex, uint32_t u32Value)
Set value to FLEXCAN RAM.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_CTRL1_PN** (FLEXCAN_Type *const pCan)
Clear PNET CTRL1_PN.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_TimeoutInterrupt** (FLEXCAN_Type *const pCan)
Set PNET timeout Interrupt.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ChkPNET_TimeoutInterrupt** (FLEXCAN_Type *const pCan)
Check PNET timeout Interrupt.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ChkPNET_MatchInterrupt** (FLEXCAN_Type *const pCan)
Check PNET match Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_TimeoutInterrupt** (FLEXCAN_Type *const pCan)
Attach PNET timeout Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_MatchInterrupt** (FLEXCAN_Type *const pCan)
Set PNET match Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_MatchInterrupt** (FLEXCAN_Type *const pCan)

Attach PNET match Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_MatchCount** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Set PNET Match Count value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_MatchCount** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Attach PNET Match Count value.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_FilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Set PNET Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_FilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Attach PNET Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_IDFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Set PNET ID Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_IDFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Attach PNET ID Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_PayloadFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Set PNET Payload Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_PayloadFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Attach PNET Payload Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_ID1** (FLEXCAN_Type *const pCan, uint8_t u8IDE, uint8_t u8RTR, uint32_t u32ID1)

Set PNET ID 1.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_ID2** (FLEXCAN_Type *const pCan, uint8_t u8IDE, uint8_t u8RTR, uint32_t u32ID2)

Set PNET ID 2.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_DlcRange** (FLEXCAN_Type *const pCan, uint8_t u8DLC1, uint8_t u8DLC2)

Set PNET DLC.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_Payload1Content** (FLEXCAN_Type *const pCan, uint32_t u32High, uint32_t u32Low)

Set PNET Payload 1 data content.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_Payload2Content** (FLEXCAN_Type *const pCan, uint32_t u32High, uint32_t u32Low)

Set PNET Payload 2 data content.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_Timeout** (FLEXCAN_Type *const pCan, uint16_t u16Value)

Set PNET Timeout value.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_IntFlag** (FLEXCAN_Type *const pCan)

Clear PNET Interrupt flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_TimeoutFlag** (FLEXCAN_Type *const pCan)

Clear PNET timeout Interrupt flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_MatchFlag** (FLEXCAN_Type *const pCan)

Clear PNET match Interrupt flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetPNET_MatchCount** (FLEXCAN_Type *const pCan)

Get PNET Match Counter.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_PNETRamAddr** (FLEXCAN_Type *const pCan, uint32_t u32MsgIndex)

Get value from FLEXCAN Wake-up message index.

LOCAL_INLINE void **FLEXCAN_HWA_SetERFCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)
Set ERFCR value.

LOCAL_INLINE void **FLEXCAN_HWA_SetERFIER** (FLEXCAN_Type *const pCan, uint32_t u32Value)
Set ERFIER value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachERFIER** (FLEXCAN_Type *const pCan, uint32_t u32Value)
Attache ERFIER value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoPollingFlag** (FLEXCAN_Type *const pCan)
Check Enhanced FIFO Polling flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoDataPolling** (FLEXCAN_Type *const pCan)
Check Enhanced FIFO data polling flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoErrorPolling** (FLEXCAN_Type *const pCan)
Check Enhanced FIFO error polling flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoDataInterrupt** (FLEXCAN_Type *const pCan)
Check Enhanced FIFO data interrupt flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoErrorInterrupt** (FLEXCAN_Type *const pCan)
Check Enhanced FIFO error interrupt flag.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRResetEnhancedFifo** (FLEXCAN_Type *const pCan)
Reset Enhanced FIFO Engine.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetEnhancedFifoFlag** (FLEXCAN_Type *const pCan,
uint32_t u32Shift, uint32_t u32Mask)
Get Enhanced Fifo Flag with mask.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetDataValid** (FLEXCAN_Type *const pCan)

Get Enhanced Fifo Data valid.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRClrUnderflow** (FLEXCAN_Type *const pCan)

Clear Enhanced Fifo underflow.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRClrOverflow** (FLEXCAN_Type *const pCan)

Clear Enhanced Fifo overflow.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetUnderflow** (FLEXCAN_Type *const pCan)

Get Enhanced Fifo underflow.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetOverflow** (FLEXCAN_Type *const pCan)

Get Enhanced Fifo overflow.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRClearEnhancedFifoFlag** (FLEXCAN_Type *const pCan, uint32_t u32ClearMask)

Clear Enhanced Fifo status.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRClearError** (FLEXCAN_Type *const pCan, uint32_t u32ClearMask)

Clear Enhanced Fifo Error.

LOCAL_INLINE void **FLEXCAN_HWA_AttachERFIERDataInterrupt** (FLEXCAN_Type *const pCan)

Set Enhance Fifo interrupt value.

LOCAL_INLINE void **FLEXCAN_HWA_EnhancedFifoFilter** (FLEXCAN_Type *const pCan, uint32_t u32FilterIndex, uint32_t u32Value)

Set Enhanced Fifo filter value.

Detailed Description

Macro Definition Documentation

```
#define FLEXCAN_EFIFOMB_BRS_GET() ((pCan->ERX_FIFO[0U]>>30)&0x01U)
```

Enhanced FIFO Message Buffer BRS Get.

```
#define FLEXCAN_EFIFOMB_BRS_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]>>30)&0x01U)
```

Enhanced FIFO Message Buffer BRS Get.

```
#define FLEXCAN_EFIFOMB_DATAADDR_GET( wordIndex) (&(pCan->ERX_FIFO[2U+(wordIndex)]))
```

Enhanced FIFO Message Buffer Data Get.

```
#define FLEXCAN_EFIFOMB_DATAADDR_GET_FROM_BUFFER( Addr,  
wordIndex) (&(((uint32_t*)Addr)[2U+(wordIndex)]))
```

Enhanced FIFO Message Buffer Data Get.

```
#define FLEXCAN_EFIFOMB_DLC_GET() ((pCan->ERX_FIFO[0U]>>16)&0x0FU)
```

Enhanced FIFO Message Buffer DLC Get.

```
#define FLEXCAN_EFIFOMB_DLC_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]>>16)&0x0FU)
```

Enhanced FIFO Message Buffer DLC Get.

```
#define FLEXCAN_EFIFOMB_EDL_GET() ((pCan->ERX_FIFO[0U]>>31)&0x01U)
```

Enhanced FIFO Message Buffer EDL Get.

```
#define FLEXCAN_EFIFOMB_EDL_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]>>31)&0x01U)
```

Enhanced FIFO Message Buffer EDL Get.

```
#define FLEXCAN_EFIFOMB_ESI_GET() ((pCan->ERX_FIFO[0U]>>29)&0x01U)
```

Enhanced FIFO Message Buffer ESI Get.

```
#define FLEXCAN_EFIFOMB_ESI_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]>>29)&0x01U)
```

Enhanced FIFO Message Buffer ESI Get.

```
#define FLEXCAN_EFIFOMB_EXTID_GET() (pCan->ERX_FIFO[1U]&0x1FFFFFFF)
```

Enhanced FIFO Message Buffer EXT ID Get.

```
#define FLEXCAN_EFIFOMB_EXTID_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[1U]&0x1FFFFFFF)
```

Enhanced FIFO Message Buffer EXT ID Get.

```
#define FLEXCAN_EFIFOMB_HRTIMESTAMP_GET( dataWordLen) (pCan->ERX_FIFO[3U+(dataWordLen)])
```

Enhanced FIFO Message Buffer HR Time Stamp Get.

```
#define FLEXCAN_EFIFOMB_HRTIMESTAMP_GET_FROM_BUFFER( Addr,  
dataWordLen) (((uint32_t*)Addr)[3U+(dataWordLen)])
```

Enhanced FIFO Message Buffer HR Time Stamp Get.

```
#define FLEXCAN_EFIFOMB_IDE_GET() ((pCan->ERX_FIFO[0U]>>21)&0x01U)
```

Enhanced FIFO Message Buffer IDE Get.

```
#define FLEXCAN_EFIFOMB_IDE_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]>>21)&0x01U)
```

Enhanced FIFO Message Buffer IDE Get.

```
#define FLEXCAN_EFIFOMB_IDHIT_GET( dataWordLen) ((pCan->ERX_FIFO[2U+(dataWordLen)])&0x1FFU)
```

Enhanced FIFO Message Buffer IDHIT Get.


```
#define FLEXCAN_EFIFOMB_IDHIT_GET_FROM_BUFFER( Addr,  
dataWordLen) (((uint32_t*)Addr)[2U+(dataWordLen)]&0x1FFU)
```

Enhanced FIFO Message Buffer IDHIT Get.

```
#define FLEXCAN_EFIFOMB_RTR_GET() ((pCan->ERX_FIFO[0U]>>20)&0x01U)
```

Enhanced FIFO Message Buffer RTR Get.

```
#define FLEXCAN_EFIFOMB_RTR_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]>>20)&0x01U)
```

Enhanced FIFO Message Buffer RTR Get.

```
#define FLEXCAN_EFIFOMB_SRR_GET() ((pCan->ERX_FIFO[0U]>>22)&0x01U)
```

Enhanced FIFO Message Buffer SRR Get.

```
#define FLEXCAN_EFIFOMB_SRR_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]>>22)&0x01U)
```

Enhanced FIFO Message Buffer SRR Get.

```
#define FLEXCAN_EFIFOMB_STDID_GET() ((pCan->ERX_FIFO[1U]>>18)&0x7FFU)
```

Enhanced FIFO Message Buffer STD ID Get.

```
#define FLEXCAN_EFIFOMB_STDID_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[1U]>>18)&0x7FFU)
```

Enhanced FIFO Message Buffer STD ID Get.

```
#define FLEXCAN_EFIFOMB_TIMESTAMP_GET() (pCan->ERX_FIFO[0U]&0xFFU)
```

Enhanced FIFO Message Buffer TIME STAMP Get.

```
#define FLEXCAN_EFIFOMB_TIMESTAMP_GET_FROM_BUFFER( Addr) (((uint32_t*)Addr)[0U]&0xFFU)
```

Enhanced FIFO Message Buffer TIME STAMP Get.

```
#define FLEXCAN_MB_ADDR( mbOffset, mbIndex, dataLen)
```

Legacy Message Buffer Address.

```
#define FLEXCAN_MB_BRS_ATTACH( brsAddr, var) (FLEXCAN_REG32_CONTENT(brsAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,30))
```

Legacy Message Buffer BRS Attach.

```
#define FLEXCAN_MB_BRS_CLR( brsAddr) (FLEXCAN_REG32_CONTENT(brsAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,30))
```

Legacy Message Buffer BRS Clear.

```
#define FLEXCAN_MB_BRS_GET( brsAddr) (((FLEXCAN_REG32_CONTENT(brsAddr))>>30)&0x01U)
```

Legacy Message Buffer BRS Get.

```
#define FLEXCAN_MB_BRS_SET( brsAddr, var) FLEXCAN_REG32_CONTENT(brsAddr) =  
((FLEXCAN_REG32_CONTENT(brsAddr) & FLEXCAN_U32NOTSHIFT(0x01U,30)) |  
FLEXCAN_U32SHIFT(var,0x01U,30))
```

Legacy Message Buffer BRS set.

```
#define FLEXCAN_MB_CODE_ATTACH( codeAddr, var) (FLEXCAN_REG32_CONTENT(codeAddr) |=  
FLEXCAN_U32SHIFT(var,0x0FU,24))
```

Legacy Message Buffer CODE Attach.

```
#define FLEXCAN_MB_CODE_CLR( codeAddr) (FLEXCAN_REG32_CONTENT(codeAddr) &=  
FLEXCAN_U32NOTSHIFT(0x0FU,24))
```

Legacy Message Buffer CODE Clear.

```
#define FLEXCAN_MB_CODE_GET( codeAddr) (((FLEXCAN_REG32_CONTENT(codeAddr))>>24)&0x0EU)
```

Legacy Message Buffer CODE Get.

```
#define FLEXCAN_MB_CODE_SET( codeAddr,  var)  FLEXCAN_REG32_CONTENT(codeAddr) =  
((FLEXCAN_REG32_CONTENT(codeAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,24)) |  
FLEXCAN_U32SHIFT(var,0x0FU,24))
```

Legacy Message Buffer CODE set.

```
#define FLEXCAN_MB_DLC_ATTACH( dlcAddr,  var)  (FLEXCAN_REG32_CONTENT(dlcAddr) |=  
FLEXCAN_U32SHIFT(var,0x0FU,16))
```

Legacy Message Buffer DLC Attach.

```
#define FLEXCAN_MB_DLC_CLR( dlcAddr)  (FLEXCAN_REG32_CONTENT(dlcAddr) &=  
FLEXCAN_U32NOTSHIFT(0x0FU,16))
```

Legacy Message Buffer DLC Clear.

```
#define FLEXCAN_MB_DLC_GET( dlcAddr)  (((FLEXCAN_REG32_CONTENT(dlcAddr))>>16)&0x0FU)
```

Legacy Message Buffer DLC Get.

```
#define FLEXCAN_MB_DLC_SET( dlcAddr,  var)  FLEXCAN_REG32_CONTENT(dlcAddr) =  
((FLEXCAN_REG32_CONTENT(dlcAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,16)) |  
FLEXCAN_U32SHIFT(var,0x0FU,16))
```

Legacy Message Buffer DLC Set.

```
#define FLEXCAN_MB_EDL_ATTACH( edlAddr,  var)  (FLEXCAN_REG32_CONTENT(edlAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,31))
```

Legacy Message Buffer EDL Attach.

```
#define FLEXCAN_MB_EDL_CLR( edlAddr)  (FLEXCAN_REG32_CONTENT(edlAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,31))
```

Legacy Message Buffer EDL Clear.

```
#define FLEXCAN_MB_EDL_GET( edlAddr)  ((FLEXCAN_REG32_CONTENT(edlAddr)>>31)&0x01U)
```

Legacy Message Buffer Data Address.

Legacy Message Buffer EDL Get

```
#define FLEXCAN_MB_EDL_SET( edlAddr,  var)  FLEXCAN_REG32_CONTENT(edlAddr) =  
((FLEXCAN_REG32_CONTENT(edlAddr) & FLEXCAN_U32NOTSHIFT(0x01U,31)) |  
FLEXCAN_U32SHIFT(var,0x01U,31))
```

Legacy Message Buffer EDL Set.

```
#define FLEXCAN_MB_ESI_ATTACH( esiAddr,  var)  (FLEXCAN_REG32_CONTENT(esiAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,29))
```

Legacy Message Buffer ESI Attach.

```
#define FLEXCAN_MB_ESI_CLR( esiAddr)  (FLEXCAN_REG32_CONTENT(esiAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,29))
```

Legacy Message Buffer ESI Clear.

```
#define FLEXCAN_MB_ESI_GET( esiAddr)  (((FLEXCAN_REG32_CONTENT(esiAddr))>>29)&0x01U)
```

Legacy Message Buffer ESI Get.

```
#define FLEXCAN_MB_ESI_SET( esiAddr,  var)  FLEXCAN_REG32_CONTENT(esiAddr) =  
((FLEXCAN_REG32_CONTENT(esiAddr) & FLEXCAN_U32NOTSHIFT(0x01U,29)) |  
FLEXCAN_U32SHIFT(var,0x01U,29))
```

Legacy Message Buffer ESI Set.

```
#define FLEXCAN_MB_EXTID_ATTACH( extidAddr,  var)  (FLEXCAN_REG32_CONTENT(extidAddr) |=  
FLEXCAN_U32SHIFT(var,0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID Attach.

```
#define FLEXCAN_MB_EXTID_CLR( extidAddr)  (FLEXCAN_REG32_CONTENT(extidAddr) &=  
FLEXCAN_U32NOTSHIFT(0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID Clear.

```
#define FLEXCAN_MB_EXTID_GET( extidAddr)  ((FLEXCAN_REG32_CONTENT(extidAddr))&0x1FFFFFFF)
```

Legacy Message Buffer EXT ID Get.

```
#define FLEXCAN_MB_EXTID_SET( extidAddr,  var)  FLEXCAN_REG32_CONTENT(extidAddr) =  
((FLEXCAN_REG32_CONTENT(extidAddr) & FLEXCAN_U32NOTSHIFT(0x1FFFFFFF,0)) |  
FLEXCAN_U32SHIFT(var,0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID set.

```
#define FLEXCAN_MB_GET_DATALEN( mbindex,  mbNum,  datalenNum,  
datalenarray)  ((uint8_t)(datalenarray[((mbindex)*(datalenNum))/(mbNum)) ]))
```

check mb data length

```
#define FLEXCAN_MB_HEADER_BYTELEN  8UL
```

Legacy/Enhanced Message Buffer Header.

```
#define FLEXCAN_MB_IDE_ATTACH( ideAddr,  var)  (FLEXCAN_REG32_CONTENT(ideAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,21))
```

Legacy Message Buffer IDE Attach.

```
#define FLEXCAN_MB_IDE_CLR( ideAddr)  (FLEXCAN_REG32_CONTENT(ideAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,21))
```

Legacy Message Buffer IDE Clear.

```
#define FLEXCAN_MB_IDE_GET( ideAddr)  (((FLEXCAN_REG32_CONTENT(ideAddr))>>21)&0x01U)
```

Legacy Message Buffer IDE Get.

```
#define FLEXCAN_MB_IDE_SRR_SET( ideAddr,  var)  FLEXCAN_REG32_CONTENT(ideAddr) =  
((FLEXCAN_REG32_CONTENT(ideAddr) & FLEXCAN_U32NOTSHIFT(0x03U,21)) |  
FLEXCAN_U32SHIFT(var,0x03U,21))
```

Legacy Message Buffer IDE set with SRR.

```
#define FLEXCAN_MB_PRIO_ATTACH( prioAddr,  var)  (FLEXCAN_REG32_CONTENT(prioAddr) |=  
FLEXCAN_U32SHIFT(var,0x07U,29))
```

Legacy Message Buffer PRIO Attach.

```
#define FLEXCAN_MB_PRIO_CLR( prioAddr)  (FLEXCAN_REG32_CONTENT(prioAddr) &=  
FLEXCAN_U32NOTSHIFT(0x07U,29))
```

Legacy Message Buffer PRIO Clear.

```
#define FLEXCAN_MB_PRIO_GET( prioAddr)  (((FLEXCAN_REG32_CONTENT(prioAddr))>>29)&0x07U)
```

Legacy Message Buffer PRIO Get.

```
#define FLEXCAN_MB_PRIO_SET( prioAddr,  var)  FLEXCAN_REG32_CONTENT(prioAddr) =  
((FLEXCAN_REG32_CONTENT(prioAddr) & FLEXCAN_U32NOTSHIFT(0x07U,29)) |  
FLEXCAN_U32SHIFT(var,0x07U,29))
```

Legacy Message Buffer PRIO Set.

```
#define FLEXCAN_MB_RTR_ATTACH( rtrAddr,  var)  (FLEXCAN_REG32_CONTENT(rtrAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,20))
```

Legacy Message Buffer RTR Attach.

```
#define FLEXCAN_MB_RTR_CLR( rtrAddr)  (FLEXCAN_REG32_CONTENT(rtrAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,20))
```

Legacy Message Buffer RTR Clear.

```
#define FLEXCAN_MB_RTR_GET( rtrAddr) (((FLEXCAN_REG32_CONTENT(rtrAddr))>>20)&0x01U)
```

Legacy Message Buffer RTR Get.

```
#define FLEXCAN_MB_RTR_SET( rtrAddr, var) FLEXCAN_REG32_CONTENT(rtrAddr) =  
((FLEXCAN_REG32_CONTENT(rtrAddr) & FLEXCAN_U32NOTSHIFT(0x01U,20)) |  
FLEXCAN_U32SHIFT(var,0x01U,20))
```

Legacy Message Buffer RTR set.

```
#define FLEXCAN_MB_SRR_ATTACH( srrAddr, var) (FLEXCAN_REG32_CONTENT(srrAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,22))
```

Legacy Message Buffer SRR Attach.

```
#define FLEXCAN_MB_SRR_CLR( srrAddr) (FLEXCAN_REG32_CONTENT(srrAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,22))
```

Legacy Message Buffer SRR Clear.

```
#define FLEXCAN_MB_SRR_GET( srrAddr) (((FLEXCAN_REG32_CONTENT(srrAddr)) >>22)&0x01U)
```

Legacy Message Buffer SRR Get.

```
#define FLEXCAN_MB_SRR_MASK_U32 0x00400000U
```

@breif Flexcan SRR bit mask

```
#define FLEXCAN_MB_SRR_SET( srrAddr, var) FLEXCAN_REG32_CONTENT(srrAddr) =  
((FLEXCAN_REG32_CONTENT(srrAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,22)) |  
FLEXCAN_U32SHIFT(var,0x0FU,22))
```

Legacy Message Buffer SRR set.

```
#define FLEXCAN_MB_STDID_ATTACH( stdidAddr, var) (FLEXCAN_REG32_CONTENT(stdidAddr) |=  
FLEXCAN_U32SHIFT(var,0x7FFU,18))
```

Legacy Message Buffer STD ID Attach.

```
#define FLEXCAN_MB_STDID_CLR( stdidAddr)  (FLEXCAN_REG32_CONTENT(stdidAddr) &=  
FLEXCAN_U32NOTSHIFT(0x7FFU,18))
```

Legacy Message Buffer STD ID CLR.

```
#define FLEXCAN_MB_STDID_GET( stdidAddr)  (((FLEXCAN_REG32_CONTENT(stdidAddr))>>18)&0x7FFU)
```

Legacy Message Buffer STD ID Get.

```
#define FLEXCAN_MB_STDID_SET( stdidAddr,  var)  FLEXCAN_REG32_CONTENT(stdidAddr) =  
((FLEXCAN_REG32_CONTENT(stdidAddr) & FLEXCAN_U32NOTSHIFT(0x7FFU,18)) |  
FLEXCAN_U32SHIFT(var,0x7FFU,18))
```

Legacy Message Buffer STD ID set.

```
#define FLEXCAN_MB_TIMESTAMP_ATTACH( timestampAddr,  
var)  (FLEXCAN_REG32_CONTENT(timestampAddr) |= FLEXCAN_U32SHIFT(var,0xFFFFU,0))
```

Legacy Message Buffer TIME STAMP Attach.

```
#define FLEXCAN_MB_TIMESTAMP_CLR( timestampAddr)  (FLEXCAN_REG32_CONTENT(timestampAddr)  
&= FLEXCAN_U32NOTSHIFT(0xFFFFU,0))
```

Legacy Message Buffer TIME STAMP Clear.

```
#define  
FLEXCAN_MB_TIMESTAMP_GET( timestampAddr)  (FLEXCAN_REG32_CONTENT(timestampAddr)&0xFFFF  
U)
```

Legacy Message Buffer TIME STAMP Get.

```
#define FLEXCAN_MB_TIMESTAMP_SET( timestampAddr,  
var)  FLEXCAN_REG32_CONTENT(timestampAddr) = ((FLEXCAN_REG32_CONTENT(timestampAddr) &  
FLEXCAN_U32NOTSHIFT(0xFFFFU,0)) | FLEXCAN_U32SHIFT(var,0xFFFFU,0))
```


Legacy Message Buffer TIME STAMP Set.

```
#define FLEXCAN_MB_WORDN_ADDR( mbOffset, mbIndex, dataLen,  
offset) (FLEXCAN_MB_ADDR((mbOffset), (mbIndex), (dataLen))+FLEXCAN_U32MACRO(offset))
```

Legacy Message Buffer word n Address.

```
#define FLEXCAN_ONE_REGION_SIZE 0x200U
```

one region contains 32MBs

```
#define FLEXCAN_REG32_CONTENT( reg) (*((uint32_t*)(reg)))
```

```
#define FLEXCAN_REGISTER_WITHOFFSET( reg, offset1,  
offset2) ((uint32_t)(reg)+(uint32_t)(offset1)+(uint32_t)(offset2))
```

```
#define FLEXCAN_REGISTER_WITHOFFSET2( reg, offset1) ((uint32_t)(reg)+(uint32_t)(offset1))
```

```
#define FLEXCAN_U32MACRO( var) ((uint32_t)(var))
```

U32 value around.

```
#define FLEXCAN_U32NOTSHIFT( mask, shift) ((uint32_t)(~(((uint32_t)(mask))<<(shift))))
```

U32 value shift and revert.

```
#define FLEXCAN_U32SHIFT( var, mask, shift) ((uint32_t)((((uint32_t)(var))&(mask))<<(shift)))
```

U32 value shift.

Enumeration Type Documentation

enum FLEXCAN_ClockSrcType

FLEXCAN Clock Source Select Enumeration.

Enumerator:

FLEXCAN_CLOCK_CANPE K_CANPE	FLEXCAN_CLOCK_CANPE clock from can peripheral clock
FLEXCAN_CLOCK_INTERFACE K_INTERFACE	FLEXCAN_CLOCK_INTERFACE clock from can interface clock

enum FLEXCAN_DataWidthType

FLEXCAN data length type.

Enumerator:

FLEXCAN_DATA_WIDTH_8	FLEXCAN_DATALEN_8 8 bytes data width
FLEXCAN_DATA_WIDTH_16	FLEXCAN_DATALEN_16 16 bytes data width
FLEXCAN_DATA_WIDTH_32	FLEXCAN_DATALEN_32 32 bytes data width
FLEXCAN_DATA_WIDTH_64	FLEXCAN_DATALEN_64 64 bytes data width

enum FLEXCAN_PNET_FilterType

CAN Pretended Network Filter type.

Enumerator:

FLEXCAN_PNET_ID_FILTER	FLEXCAN_PNET_ID_FILTER only ID filter
------------------------	---------------------------------------

FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER	FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER id and payload filter
FLEXCAN_PNET_ID_FILTER_WITH_NCOUNT	FLEXCAN_PNET_ID_FILTER_WITH_NCOUNT id with count
FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER_WITH_NCOUNT	FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER_WITH_NCOUNT id and payload with count

enum FLEXCAN_PNET_IdFilterType

CAN Pretended Network ID Filter type.

Enumerator:

FLEXCAN_PNET_ID_FILTER_SAME	FLEXCAN_PNET_ID_FILTER_SAME id with mask check match
FLEXCAN_PNET_ID_FILTER_GREATER_EQUAL	FLEXCAN_PNET_ID_FILTER_GREATER_EQUAL only match greater and equal ID1
FLEXCAN_PNET_ID_FILTER_SMALLER_EQUAL	FLEXCAN_PNET_ID_FILTER_SMALLER_EQUAL only match less and equal ID1
FLEXCAN_PNET_ID_FILTER_RANGE	FLEXCAN_PNET_ID_FILTER_RANGE greater and equal ID1 and less and equal ID2

enum FLEXCAN_PNET_PayloadFilterType

CAN Pretended Network Payload Filter type.

Enumerator:

FLEXCAN_PNET_PAYLOAD_FILTER_SAME	FLEXCAN_PNET_PAYLOAD_FILTER_SAME payload with mask check match, FLT_DLC_LO must equal to FLT_DLC_HI
FLEXCAN_PNET_PAYLOAD_FILTER_GREATER_EQUAL	FLEXCAN_PNET_PAYLOAD_FILTER_GREATER_EQUAL payload greater and equal than PL1_HI+PL1_LO
FLEXCAN_PNET_PAYLOAD_FILTER_SMALLER_EQUAL	FLEXCAN_PNET_PAYLOAD_FILTER_SMALLER_EQUAL payload less and equal than PL1_HI+PL1_LO
FLEXCAN_PNET_PAYLOAD_FILTER_RANGE	FLEXCAN_PNET_PAYLOAD_FILTER_RANGE payload greater and equal than PL1_HI+PL1_LO and less and equal than PL2_HI+PL2_LO

Function Documentation

LOCAL_INLINE void FLEXCAN_HWA_AttachCtrl1 (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Attach FLEXCAN ctrl1 value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	attached value

LOCAL_INLINE void FLEXCAN_HWA_AttachCtrl2 (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Attach FLEXCAN ctrl2 value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	attached value

LOCAL_INLINE void FLEXCAN_HWA_AttachERFCR (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Attach FLEXCAN ERFCR value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	attached value

LOCAL_INLINE void FLEXCAN_HWA_AttachERFIER (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Attache ERFIER value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FLEXCAN_HWA_AttachERFIERDataInterrupt (FLEXCAN_Type *const *pCan*)

Set Enhance Fifo interrupt value.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_AttachMask1MbInterrupt (FLEXCAN_Type *const *pCan*, uint32_t *u32Mask*)

Attach special message buffer interrupt MASK1.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_AttachMbIndexInterrupt (FLEXCAN_Type *const *pCan*, uint32_t *u32MbIndex*)

Attach special message buffer interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32MbIndex</i>	Message Buffer index

LOCAL_INLINE void FLEXCAN_HWA_AttachMCR (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Attach FLEXCAN MCR value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	attached value

LOCAL_INLINE void FLEXCAN_HWA_AttachPNET_FilterType (FLEXCAN_Type *const *pCan*, uint8_t *u8Value*)

Attach PNET Filter Type.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, Filter Type

LOCAL_INLINE void FLEXCAN_HWA_AttachPNET_IDFilterType (FLEXCAN_Type *const *pCan*, uint8_t *u8Value*)

Attach PNET ID Filter Type.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, ID Filter Type

LOCAL_INLINE void FLEXCAN_HWA_AttachPNET_MatchCount (FLEXCAN_Type *const *pCan*, uint8_t *u8Value*)

Attach PNET Match Count value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, match count

LOCAL_INLINE void FLEXCAN_HWA_AttachPNET_MatchInterrupt (FLEXCAN_Type *const *pCan*)

Attach PNET match Interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

**LOCAL_INLINE void FLEXCAN_HWA_AttachPNET_PayloadFilterType (FLEXCAN_Type *const *pCan*,
uint8_t *u8Value*)**

Attach PNET Payload Filter Type.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, Payload Filter Type

LOCAL_INLINE void FLEXCAN_HWA_AttachPNET_TimeoutInterrupt (FLEXCAN_Type *const *pCan*)

Attach PNET timeout Interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_CalDataWidth (uint8_t * *pDataWidthArr*)

Set MB data width in MBDSR.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckEnhancedFifoDataInterrupt (FLEXCAN_Type *const *pCan*)

Check Enhanced FIFO data interrupt flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckEnhancedFifoDataPolling (FLEXCAN_Type *const *pCan*)

Check Enhanced FIFO data polling flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckEnhancedFIFOEn (FLEXCAN_Type *const *pCan*)

Check Enhanced FIFO Enable.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckEnhancedFifoErrorInterrupt (FLEXCAN_Type *const *pCan*)

Check Enhanced FIFO error interrupt flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckEnhancedFifoErrorPolling (FLEXCAN_Type *const *pCan*)

Check Enhanced FIFO error polling flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckEnhancedFifoPollingFlag (FLEXCAN_Type *const *pCan*)

Check Enhanced FIFO Polling flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckErrorInterrupt (FLEXCAN_Type *const *pCan*)

Check Interrupt Mask.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is disable, others is enable

Here is the call graph for this function:

IMAGE

LOCAL_INLINE uint8_t FLEXCAN_HWA_CheckFd (FLEXCAN_Type *const *pCan*)

Check FLEXCAN FD enable.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

1 is fd enabled, 0 is disabled

LOCAL_INLINE uint8_t FLEXCAN_HWA_CheckFdFifoEnabled (FLEXCAN_Type *const *pCan*)

Check FLEXCAN FD fifo if enabled.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

1 is legacy fifo enabled, 0 is disabled

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckFifoDma (FLEXCAN_Type *const *pCan*)

Check if fifo dma enable.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is none, others is dma enabled

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoData (FLEXCAN_Type *const *pCan*)

Check if legacy fifo contains at least one data.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is none, others is data available

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoDataInterrupt (FLEXCAN_Type *const *pCan*)

Check Legacy Fifo Data Valid Interrupt Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

data valid interrupt flag, 0 is not ok, 1 is ok

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoDataLost (FLEXCAN_Type *const *pCan*)

Check if legacy fifo data lost.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is none, others are lost

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoDataPolling (FLEXCAN_Type *const *pCan*)

Check Legacy Fifo Data Valid Polling Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

data valid polling flag, 0 is not ok, 1 is ok

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoEn (FLEXCAN_Type *const *pCan*)

Check FLEXCAN LegacyFifo Enabled.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is disable, others are enabled

LOCAL_INLINE uint8_t FLEXCAN_HWA_CheckLegacyFifoEnabled (FLEXCAN_Type *const *pCan*)

Check FLEXCAN legacy fifo if enabled.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

1 is legacy fifo enabled, 0 is disabled

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoErrorInterrupt (FLEXCAN_Type *const *pCan*)

Check Legacy Fifo Data lost interrupt Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

data lost interrupt flag, 0 is not ok, 1 is ok

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoErrorPolling (FLEXCAN_Type *const *pCan*)

Check Legacy Fifo Data lost Polling Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

data lost polling flag, 0 is not ok, 1 is ok

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoWarning (FLEXCAN_Type *const *pCan*)

Check if legacy fifo contains 5 data and trigger warning.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is none, others are warning

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoWarningInterrupt (FLEXCAN_Type *const *pCan*)

Check Legacy Fifo Data warning interrupt Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

data warning interrupt flag, 0 is not ok, 1 is ok

LOCAL_INLINE uint32_t FLEXCAN_HWA_CheckLegacyFifoWarningPolling (FLEXCAN_Type *const *pCan*)

Check Legacy Fifo Data warning Polling Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

data warning polling flag, 0 is not ok, 1 is ok

LOCAL_INLINE uint8_t FLEXCAN_HWA_CheckPNETAck (FLEXCAN_Type *const *pCan*, uint8_t *bEntryPNET*, uint32_t *u32TimeoutTick*)

Check if it is in pretended mode.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>bEntryPNET</i>	if entry or out of pretended network
<i>u32TimeoutTick</i>	timeout counter of tick

Returns

0 is not ok, other is ok

LOCAL_INLINE uint32_t FLEXCAN_HWA_ChkPNET_MatchInterrupt (FLEXCAN_Type *const *pCan*)

Check PNET match Interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is no timeout interrupt, others means contains interrupt

LOCAL_INLINE uint32_t FLEXCAN_HWA_ChkPNET_TimeoutInterrupt (FLEXCAN_Type *const *pCan*)

Check PNET timeout Interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

0 is no timeout interrupt, others means contains interrupt

LOCAL_INLINE void FLEXCAN_HWA_ClearIFLAG1FifoErrorFlag (FLEXCAN_Type *const *pCan*)

Clear Legacy Fifo Rx error Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_ClearIFLAG1FifoFlag (FLEXCAN_Type *const *pCan*)

Clear Legacy Fifo Rx data available Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_ClearIFLAG1FifoWarningFlag (FLEXCAN_Type *const *pCan*)

Clear Legacy Fifo Rx warning Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_ClrErrorInfo (FLEXCAN_Type *const *pCan*, uint32_t *u32ErrorEsr1*)

Clear Error Info.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32ErrorEsr1</i>	is read from FLEXCAN_GetErrorInfo

LOCAL_INLINE void FLEXCAN_HWA_ClrPNET_CTRL1_PN (FLEXCAN_Type *const *pCan*)

Clear PNET CTRL1_PN.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_ClrPNET_IntFlag (FLEXCAN_Type *const *pCan*)

Clear PNET Interrupt flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_ClrPNET_MatchFlag (FLEXCAN_Type *const *pCan*)

Clear PNET match Interrupt flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_ClrPNET_TimeoutFlag (FLEXCAN_Type *const *pCan*)

Clear PNET timeout Interrupt flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_DisableFDFifo (FLEXCAN_Type *const *pCan*)

Disable FLEXCAN FD Fifo.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_DisableLegacyFifo (FLEXCAN_Type *const *pCan*)

Disable FLEXCAN Legacy Fifo.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_EnableFDFifo (FLEXCAN_Type *const *pCan*)

Enable FLEXCAN FD Fifo.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_EnableLegacyFifo (FLEXCAN_Type *const *pCan*)

Enable FLEXCAN Legacy Fifo.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_EnhancedFifoFilter (FLEXCAN_Type *const *pCan*, uint32_t *u32FilterIndex*, uint32_t *u32Value*)

Set Enhanced Fifo filter value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32FilterIndex</i>	Filter index
<i>u32Value</i>	filter value

LOCAL_INLINE void FLEXCAN_HWA_ERFSRClearEnhancedFifoFlag (FLEXCAN_Type *const *pCan*, uint32_t *u32ClearMask*)

Clear Enhanced Fifo status.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32ClearMask</i>	clear mask value

LOCAL_INLINE void FLEXCAN_HWA_ERFSRClearError (FLEXCAN_Type *const *pCan*, uint32_t *u32ClearMask*)

Clear Enhanced Fifo Error.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32ClearMask</i>	clear mask value

LOCAL_INLINE void FLEXCAN_HWA_ERFSRClrOverflow (FLEXCAN_Type *const *pCan*)

Clear Enhanced Fifo overflow.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_ERFSRClrUnderflow (FLEXCAN_Type *const *pCan*)

Clear Enhanced Fifo underflow.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_ERFSRGetDataValid (FLEXCAN_Type *const *pCan*)

Get Enhanced Fifo Data valid.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

flag value

**LOCAL_INLINE uint32_t FLEXCAN_HWA_ERFSRGetEnhancedFifoFlag (FLEXCAN_Type *const *pCan*,
uint32_t *u32Shift*, uint32_t *u32Mask*)**

Get Enhanced Fifo Flag with mask.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Shift</i>	flag shift
<i>u32Mask</i>	flag mask

Returns

flag value

LOCAL_INLINE uint32_t FLEXCAN_HWA_ERFSRGetOverflow (FLEXCAN_Type *const *pCan*)

Get Enhanced Fifo overflow.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

flag value

LOCAL_INLINE uint32_t FLEXCAN_HWA_ERFSRGetUnderflow (FLEXCAN_Type *const *pCan*)

Get Enhanced Fifo underflow.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

flag value

LOCAL_INLINE void FLEXCAN_HWA_ERFSRResetEnhancedFifo (FLEXCAN_Type *const *pCan*)

Reset Enhanced FIFO Engine.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetCtrl1 (FLEXCAN_Type *const *pCan*)

Get FLEXCAN ctrl1 value.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

CTRL1 value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetCtrl2 (FLEXCAN_Type *const *pCan*)

Get CTRL2 value.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

CTRL2 value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetERFSR (FLEXCAN_Type *const *pCan*)

This Function is used to get FLEXCAN enhanced fifo error count.

Parameters

<i>pCan</i>	is FLEXCAN instance value
-------------	---------------------------

Returns

ERFSR Register Value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetErrorCount (FLEXCAN_Type *const *pCan*)

This Function is used to get FLEXCAN error count.

Parameters

<i>pCan</i>	is FLEXCAN instance value
-------------	---------------------------

Returns

ECR Register Value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetErrorInfo (FLEXCAN_Type *const *pCan*)

This Function is used to get FLEXCAN error.

Parameters

<i>pCan</i>	is FLEXCAN instance value
-------------	---------------------------

Returns

ESR1 Register Value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetErrorIntInfo (FLEXCAN_Type *const *pCan*)

This Function is used to get FLEXCAN error with interrupt.

Parameters

<i>pCan</i>	is FLEXCAN instance value
-------------	---------------------------

Returns

ESR1 Register Value with interrupt mask

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetFlag1 (FLEXCAN_Type *const *pCan*)

Get IFLAG1 Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

flag1 value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetFlag1NoFifoFlag (FLEXCAN_Type *const *pCan*, uint32_t *u32MbIndex*)

Get MB Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32MbIndex</i>	Message Buffer Index

Returns

flag value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetMask1 (FLEXCAN_Type *const *pCan*)

Get special message buffer interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

IMASK1 value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetMCR (FLEXCAN_Type *const *pCan*)

Get FLEXCAN MCR value.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

MCR value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetPNET_MatchCount (FLEXCAN_Type *const *pCan*)

Get PNET Match Counter.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

Returns

count value

LOCAL_INLINE uint32_t FLEXCAN_HWA_GetTimeStamp (FLEXCAN_Type *const *pCan*)

This Function is used to get FLEXCAN rx timestamp.

Parameters

<i>pCan</i>	is FLEXCAN instance value
-------------	---------------------------

Returns

timestamp Register Value

**LOCAL_INLINE void FLEXCAN_HWA_MbRam (FLEXCAN_Type *const *pCan*, uint32_t *u32RamIndex*,
uint32_t *u32Value*)**

Set value to FLEXCAN RAM.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32RamIndex</i>	RAM index for FLEXCAN
<i>u32Value</i>	written value

**LOCAL_INLINE uint32_t FLEXCAN_HWA_PNETRamAddr (FLEXCAN_Type *const *pCan*, uint32_t
u32MsgIndex)**

Get value from FLEXCAN Wake-up message index.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32MsgIndex</i>	Wake-up Message index for FLEXCAN

Returns

ram address

LOCAL_INLINE void FLEXCAN_HWA_SetCBT (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set CBT value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetCtrl1 (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set FLEXCAN ctrl1 value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

**LOCAL_INLINE void FLEXCAN_HWA_SetCtrl1BaudSrc (FLEXCAN_Type *const *pCan*,
FLEXCAN_ClockSrcType *tBaudClk*)**

Set FLEXCAN baudrate clock source.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>tBaudClk</i>	Baudrate clock source

LOCAL_INLINE void FLEXCAN_HWA_SetCtrl2 (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set CTRL2 value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetEDCBT (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set EDCBT value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetENCBT (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set ENCBT value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetEPRS (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set EPRS value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetERFCR (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set ERFCR value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	written value

LOCAL_INLINE void FLEXCAN_HWA_SetERFIER (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set ERFIER value.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

<i>u32Value</i>	written value
-----------------	---------------

LOCAL_INLINE void FLEXCAN_HWA_SetETDC (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set ETDC value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetFDCBT (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set FDCBT value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetFDCTRL (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set FDCTRL value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetFifoGlobalMask (FLEXCAN_Type *const *pCan*, uint32_t *u32Mask*)

Set Fifo global mask value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Mask</i>	mask value

LOCAL_INLINE void FLEXCAN_HWA_SetFlag1 (FLEXCAN_Type *const *pCan*, uint32_t *u32W1cFlag*)

Clear IFLAG1 Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32W1cFlag</i>	write clear value

LOCAL_INLINE void FLEXCAN_HWA_SetHaltFreeze (FLEXCAN_Type *const *pCan*)

Set Module Halt and Freeze.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_SetIndividualMask (FLEXCAN_Type *const *pCan*, uint32_t *u32MbIndex*, uint32_t *u32Mask*)

Set RXIMR for FLEXCAN.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32MbIndex</i>	Message Buffer Index
<i>u32Mask</i>	ID Mask

LOCAL_INLINE void FLEXCAN_HWA_SetLegacyFifoFilterNum (FLEXCAN_Type *const *pCan*, uint32_t *u32RFFN*)

Set legacy fifo filter number number.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32RFFN</i>	is total filter number

LOCAL_INLINE void FLEXCAN_HWA_SetLegacyFifoFilterNum1 (FLEXCAN_Type *const *pCan*, uint32_t *u32FilterNum*)

Set legacy fifo filter number number.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32FilterNum</i>	is total filter number

LOCAL_INLINE void FLEXCAN_HWA_SetMask1MbInterrupt (FLEXCAN_Type *const *pCan*, uint32_t *u32Mask*)

Set special message buffer interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Mask</i>	Message Buffer interrupt mask value

LOCAL_INLINE void FLEXCAN_HWA_SetMaskFifoInterrupt (FLEXCAN_Type *const *pCan*)

Set Legacy Fifo Rx interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_SetMBGlobalMask (FLEXCAN_Type *const *pCan*, uint32_t *u32Mask*)

Set MB global mask value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Mask</i>	mask value

LOCAL_INLINE void FLEXCAN_HWA_SetMCR (FLEXCAN_Type *const *pCan*, uint32_t *u32Value*)

Set FLEXCAN MCR value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Value</i>	write value

LOCAL_INLINE void FLEXCAN_HWA_SetMcrDisable (FLEXCAN_Type *const *pCan*, uint8_t *bDisable*)

Set Module Disable.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>bDisable</i>	set whether disable

LOCAL_INLINE void FLEXCAN_HWA_SetMcrFreeze (FLEXCAN_Type *const *pCan*, uint8_t *bFreeze*)

Set Module Freeze.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>bFreeze</i>	set whether freeze

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_DlcRange (FLEXCAN_Type *const *pCan*, uint8_t *u8DLC1*, uint8_t *u8DLC2*)

Set PNET DLC.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8DLC1</i>	Low value
<i>u8DLC2</i>	High value

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_FilterType (FLEXCAN_Type *const *pCan*, uint8_t *u8Value*)

Set PNET Filter Type.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, Filter Type

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_ID1 (FLEXCAN_Type *const *pCan*, uint8_t *u8IDE*, uint8_t *u8RTR*, uint32_t *u32ID1*)

Set PNET ID 1.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8IDE</i>	0-standard, 1-extended
<i>u8RTR</i>	0-data, 1-remote
<i>u32ID1</i>	id value

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_ID2 (FLEXCAN_Type *const *pCan*, uint8_t *u8IDE*, uint8_t *u8RTR*, uint32_t *u32ID2*)

Set PNET ID 2.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8IDE</i>	0-standard, 1-extended
<i>u8RTR</i>	0-data, 1-remote
<i>u32ID2</i>	id value

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_IDFilterType (FLEXCAN_Type *const *pCan*, uint8_t *u8Value*)

Set PNET ID Filter Type.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, ID Filter Type

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_MatchCount (FLEXCAN_Type *const *pCan*, uint8_t *u8Value*)

Set PNET Match Count value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, match count

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_MatchInterrupt (FLEXCAN_Type *const *pCan*)

Set PNET match Interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_Payload1Content (FLEXCAN_Type *const *pCan*, uint32_t *u32High*, uint32_t *u32Low*)

Set PNET Payload 1 data content.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32High</i>	High value
<i>u32Low</i>	Low value

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_Payload2Content (FLEXCAN_Type *const *pCan*, uint32_t *u32High*, uint32_t *u32Low*)

Set PNET Payload 2 data content.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32High</i>	High value
<i>u32Low</i>	Low value

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_PayloadFilterType (FLEXCAN_Type *const *pCan*, uint8_t *u8Value*)

Set PNET Payload Filter Type.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u8Value</i>	written value, Payload Filter Type

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_Timeout (FLEXCAN_Type *const *pCan*, uint16_t *u16Value*)

Set PNET Timeout value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u16Value</i>	written value, period is 64 times of can bit time

LOCAL_INLINE void FLEXCAN_HWA_SetPNET_TimeoutInterrupt (FLEXCAN_Type *const *pCan*)

Set PNET timeout Interrupt.

Parameters

<i>pCan</i>	FLEXCAN instance value
-------------	------------------------

LOCAL_INLINE void FLEXCAN_HWA_SetRx14Mask (FLEXCAN_Type *const *pCan*, uint32_t *u32Mask*)

Set rx14 mask value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Mask</i>	mask value

LOCAL_INLINE void FLEXCAN_HWA_SetRx15Mask (FLEXCAN_Type *const *pCan*, uint32_t *u32Mask*)

Set rx15 mask value.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32Mask</i>	mask value

LOCAL_INLINE void FLEXCAN_HWA_UnlockMbNoFifoFlag (FLEXCAN_Type *const *pCan*, uint32_t *u32MbIndex*)

Clear Rx Flag and unlock MB.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32MbIndex</i>	Message Buffer Index

LOCAL_INLINE void FLEXCAN_HWA_W1cFlag1NoFifoFlag (FLEXCAN_Type *const *pCan*, uint32_t *u32MbIndex*)

Clear MB Flag.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32MbIndex</i>	Message Buffer Index

LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrExitFreezen (FLEXCAN_Type *const *pCan*, uint32_t *u32TimeoutTick*)

Wait FLEXCAN Module Exit Freezen mode.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32TimeoutTick</i>	Timeout tick

Returns

1U is really in frozen mode

LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrFreezen (FLEXCAN_Type *const *pCan*, uint32_t *u32TimeoutTick*)

Wait FLEXCAN Module Enter Freezen mode.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32TimeoutTick</i>	Timeout tick

Returns

0U is really in frozen mode

LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrNoReady (FLEXCAN_Type *const *pCan*, uint32_t *u32TimeoutTick*)

Wait FLEXCAN Module Not Ready.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32TimeoutTick</i>	Timeout tick

Returns

1U is really not ready

LOCAL_INLINE uint32_t FLEXCAN_HWA_WaitMcrReady (FLEXCAN_Type *const *pCan*, uint32_t *u32TimeoutTick*)

Wait FLEXCAN Module Ready.

Parameters

<i>pCan</i>	FLEXCAN instance value
<i>u32TimeoutTick</i>	Timeout tick

Returns

1U is really ready

HwA_freqm

Collaboration diagram for HwA_freqm:Functions

LOCAL_INLINE void **FREQM_HWA_MesClkSel** (FREQM_Type *const pFreqm, uint8_t u8ClkSel)

Set the measured clock selection.

LOCAL_INLINE void **FREQM_HWA_MesClk_PreDiv** (FREQM_Type *const pFreqm, uint8_t u8PredivVal)

Set the clock selection.

LOCAL_INLINE void **FREQM_HWA_EnableCntEventInterrupt** (FREQM_Type *const pFreqm)

Enable count event interrupt.

LOCAL_INLINE void **FREQM_HWA_DisableCntEventInterrupt** (FREQM_Type *const pFreqm)

Disable count event interrupt.

LOCAL_INLINE void **FREQM_HWA_SetMesLength** (FREQM_Type *const pFreqm, uint32_t u32MesLen)

Set counting length of measure counter.

LOCAL_INLINE void **FREQM_HWA_SetRefTimeout** (FREQM_Type *const pFreqm, uint32_t u32RefTo)

Set timeout value of reference counter.

LOCAL_INLINE void **FREQM_HWA_SetRefCnt** (FREQM_Type *const pFreqm, uint32_t u32RefCnt)

Set value of reference counter.

LOCAL_INLINE void **FREQM_HWA_ClearInterruptFlag** (FREQM_Type *const pFreqm)

Clear counter event interrupt flag.

LOCAL_INLINE bool **FREQM_HWA_GetInterruptFlag** (const FREQM_Type *const pFreqm)
Get counter event interrupt flag.

LOCAL_INLINE uint32_t **FREQM_HWA_GetCntStatus** (const FREQM_Type *const pFreqm)
Get counter status.

LOCAL_INLINE void **FREQM_HWA_SetMesCnt** (FREQM_Type *const pFreqm, uint32_t u32MesCnt)
Set value of measure counter.

LOCAL_INLINE uint32_t **FREQM_HWA_GetRefCntSave** (const FREQM_Type *const pFreqm)
Get saved reference counter value.

Detailed Description

Function Documentation

LOCAL_INLINE void **FREQM_HWA_ClearInterruptFlag** (FREQM_Type *const *pFreqm*)

Clear counter event interrupt flag.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
---------------	--------------------------------

LOCAL_INLINE void **FREQM_HWA_DisableCntEventInterrupt** (FREQM_Type *const *pFreqm*)

Disable count event interrupt.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
---------------	--------------------------------

LOCAL_INLINE void FREQM_HWA_EnableCntEventInterrupt (FREQM_Type *const *pFreqm*)

Enable count event interrupt.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
---------------	--------------------------------

LOCAL_INLINE uint32_t FREQM_HWA_GetCntStatus (const FREQM_Type *const *pFreqm*)

Get counter status.

Parameters

<i>pFreqm</i>	the base address of the FREQM
---------------	-------------------------------

Returns

Counter status.

LOCAL_INLINE bool FREQM_HWA_GetInterruptFlag (const FREQM_Type *const *pFreqm*)

Get counter event interrupt flag.

Parameters

<i>pFreqm</i>	the base address of the FREQM
---------------	-------------------------------

Returns

false for disable, true for enable.

LOCAL_INLINE uint32_t FREQM_HWA_GetRefCntSave (const FREQM_Type *const *pFreqm*)

Get saved reference counter value.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
---------------	--------------------------------

Returns

saved reference counter value.

LOCAL_INLINE void FREQM_HWA_MesClk_PreDiv (FREQM_Type *const *pFreqm*, uint8_t *u8PredivVal*)

Set the clock selection.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
<i>u8PredivVal</i>	the measure clock prediv value.

LOCAL_INLINE void FREQM_HWA_MesClkSel (FREQM_Type *const *pFreqm*, uint8_t *u8ClkSel*)

Set the measured clock selection.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
<i>u8ClkSel</i>	the clock selection index.

LOCAL_INLINE void FREQM_HWA_SetMesCnt (FREQM_Type *const *pFreqm*, uint32_t *u32MesCnt*)

Set value of measure counter.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
<i>u32MesCnt</i>	value of measure counter.

LOCAL_INLINE void FREQM_HWA_SetMesLength (FREQM_Type *const *pFreqm*, uint32_t *u32MesLen*)

Set counting length of measure counter.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
<i>u32MesLen</i>	counting length of measure counter.

LOCAL_INLINE void FREQM_HWA_SetRefCnt (FREQM_Type *const *pFreqm*, uint32_t *u32RefCnt*)

Set value of reference counter.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
<i>u32RefCnt</i>	value of reference counter.

LOCAL_INLINE void FREQM_HWA_SetRefTimeout (FREQM_Type *const *pFreqm*, uint32_t *u32RefTo*)

Set timeout value of reference counter.

Parameters

<i>pFreqm</i>	the base address of the FREQM.
<i>u32RefTo</i>	timeout value of reference counter.

HwA_ftu

Collaboration diagram for HwA_ftu:Macros

```
#define FTU_CHCTRL_DEADTIME_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 4U))
```

The definition of enable the deadtime of channel and channel+1.

```
#define FTU_CHCTRL_COMPLEMENT_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 1U))
```

The definition of enable the complement of channel.

```
#define FTU_CHCTRL_SYNCEN_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 5U))
```

The definition of enable the synchronization for channel and channel+1.

```
#define FTU_CHCTRL_PHASE_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U))
```

The definition of enable the Phase shift mode for channel and channel+1.

```
#define FTU_CHCTRL_EPHASE_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 7U))
```

The definition of enable the Enhanced Phase shift mode for channel and channel+1.

```
#define FTU_CHCTRL_FAULT_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 6U))
```

The definition of enable the fault control in channel and channel+1.

Enumerations

```
enum FTU_DebugModeType { FTU_DBG_COUNTER_STOP_CHN_WORKS = 0U,  
FTU_DBG_COUNTER_STOP_CHN_FORCE_SAFE = 1U, FTU_DBG_COUNTER_STOP_CHN_FROZEN = 2U,  
FTU_DBG_COUNTER_WORKS_CHN_WORKS = 3U }
```

Select the FTU Debug Mode.

```
enum FTU_ChannelModeType { FTU_CHANNEL_MODE_INPUT = 0u,  
FTU_CHANNEL_MODE_OUTPUT_COMPARE = 1u, FTU_CHANNEL_MODE_EDGE_ALIGN_PWM = 2u,  
FTU_CHANNEL_MODE_DO_NOT_CARE = 3u }  
  
enum FTU_ChannelEdgeLevelType { FTU_CHANNEL_EDGE_NOT_USED = 0u,  
FTU_CHANNEL_EDGE_RISING = 1u, FTU_CHANNEL_EDGE_FALLING = 2u, FTU_CHANNEL_EDGE_BOTH =  
3u, FTU_CHANNEL_OC_TOGGLE = 1u, FTU_CHANNEL_OC_CLEAR = 2u, FTU_CHANNEL_OC_SET = 3u,  
FTU_CHANNEL_PWM_HIGH_TRUE = 2u, FTU_CHANNEL_PWM_LOW_TRUE = 3u }
```

```
enum FTU_PrescalerType { FTU_DIV_1 = 0U, FTU_DIV_2, FTU_DIV_4, FTU_DIV_8, FTU_DIV_16,  
FTU_DIV_32, FTU_DIV_64, FTU_DIV_128 }
```

Select the prescaler of the FTU.

```
enum FTU_FilterPrescalerType { FTU_FLT_DIV_1 = 0U, FTU_FLT_DIV_2, FTU_FLT_DIV_3,  
FTU_FLT_DIV_4, FTU_FLT_DIV_5, FTU_FLT_DIV_6, FTU_FLT_DIV_7, FTU_FLT_DIV_8, FTU_FLT_DIV_9,  
FTU_FLT_DIV_10, FTU_FLT_DIV_11, FTU_FLT_DIV_12, FTU_FLT_DIV_13, FTU_FLT_DIV_14,  
FTU_FLT_DIV_15, FTU_FLT_DIV_16 }
```

Select the prescaler of the FTU filter.

```
enum FTU_QuadratureModeType { FTU_QUADMODE_PHA_PHB_ENCODING_MODE = 0U,  
FTU_QUADMODE_COUNT_DIR_ENCODING_MODE }
```

selects the encoding mode used in the Quadrature Decoder mode

```
enum FTU_QuadratureDirectionType { FTU_QUADIR_DECREMENT = 0U, FTU_QUADIR_INCREMENT }
```

FTU Counter Direction In Quadrature Decoder Mode.

```
enum FTU_BitStatusType { FTU_BIT_LOW = 0U, FTU_BIT_HIGH }
```

register bit status

enum **FTU_ModuleClkSrcType** { **FTU_MODULE_NO_CLK** = 0U, **FTU_MODULE_INTERNAL_CLK** = 1U, **FTU_MODULE_EXTERNAL_CLK** = 3U }

Ftu module clock source.

enum **FTU_UpDownDisableType** { **FTU_DISABLE_TRIG_INTR_NONE** = 0U, **FTU_DISABLE_TRIG_INTR_CNT_DOWN** = 1U, **FTU_DISABLE_TRIG_INTR_CNT_UP** = 2U, **FTU_DISABLE_TRIG_INTR_CNT_UP_DOWN** = 3U }

Disable channel match trigger/interrupt when count-up/down in CPWM/QUAD mode.

enum **FTU_TrigModeType** { **FTU_CLEARS_TRIG_WHEN_DETECTED** = 0x0U, **FTU_NOT_CLEARS_TRIG_WHEN_DETECTED** = 0x01U }

hardware trig(n) mode

enum **FTU_FaultModeType** { **FTU_FAULT_MODE_DISABLED** = 0x00U, **FTU_FAULT_MODE_EVEN_CHANNEL** = 0x01U, **FTU_FAULT_MODE_CHANNEL_ALL** = 0x02U, **FTU_FAULT_MODE_AUTO** = 0x03U }

fault mode enumeration

enum **FTU_DeadTimePrescalerType** { **FTU_DEADTIME_PRESCALER_DIV_1** = 0U, **FTU_DEADTIME_PRESCALER_DIV_4** = 2U, **FTU_DEADTIME_PRESCALER_DIV_16** = 3U }

Deadtime Prescaler Value.

enum **FTU_TrigOutputModeType** { **FTU_TRIG_OUTPUT_AS_CHANNEL_MODE** = 0U, **FTU_TRIG_OUTPUT_AS_TRIGGER** = 1U }

Trigger output control.

enum **FTU_FaultDisableDelayType** { **FTU_FAULT_DISABLE_DELAY_NO_DELAY** = 0u, **FTU_FAULT_DISABLE_DELAY_VALUE_0** = 1u, **FTU_FAULT_DISABLE_DELAY_VALUE_1** = 2u }

Fault disable channel output delay value selection.

enum **FTU_MeasurementModeType** { **FTU_MEASURE_MODE_OFF** = 0u, **FTU_MEASURE_MODE_DUTY_CYCLE** = 1u, **FTU_MEASURE_MODE_PERIOD** = 2u, **FTU_MEASURE_EDGE_NUMBER** = 3u, **FTU_MEASURE_EXPECT_EDGE_NUMBER** = 4u, **FTU_MEASURE_ICENM_WIND_WRITE** = 7u }

Input Capture Measurement Mode.

Functions

LOCAL_INLINE void **FTU_HWA_ClearModuleRegister** (FTU_Type *pFtu)

Clear FTU module registers.

LOCAL_INLINE void **FTU_HWA_DisableGlobalTimeBase** (FTU_Type *pFtu)

Disable Global Time Base Enable.

LOCAL_INLINE void **FTU_HWA_EnableGlobalTimeBase** (FTU_Type *pFtu)

Enable Global Time Base Enable.

LOCAL_INLINE void **FTU_HWA_SetPwmLoadEnable** (FTU_Type *pFtu)

Set the LDOK flag.

LOCAL_INLINE void **FTU_HWA_ClearPwmLoadEnable** (FTU_Type *pFtu)

Clear the LDOK flag.

LOCAL_INLINE void **FTU_HWA_SetCntinSync** (FTU_Type *pFtu)

Set the CNTINC flag.

LOCAL_INLINE void **FTU_HWA_ClearCntinSync** (FTU_Type *pFtu)

Clear the CNTINC flag.

LOCAL_INLINE void **FTU_HWA_SetChannelSyncEnable** (FTU_Type *pFtu, uint32_t u32Index)

Set the SYNCEN flag.

LOCAL_INLINE void **FTU_HWA_ClearChannelSyncEnable** (FTU_Type *pFtu, uint32_t u32Index)

Clear the SYNCEN flag.

LOCAL_INLINE void **FTU_HWA_SetPwmSyncMode** (FTU_Type *pFtu)

Set the PWMSYNC flag.

LOCAL_INLINE void **FTU_HWA_ClearPwmSyncMode** (FTU_Type *pFtu)

Clear the PWMSYNC flag.

LOCAL_INLINE void **FTU_HWA_SetReinitBySync** (FTU_Type *pFtu)

Set the REINIT flag.

LOCAL_INLINE void **FTU_HWA_ClearReinitBySync** (FTU_Type *pFtu)

Clear the REINIT flag.

LOCAL_INLINE void **FTU_HWA_EnableMaxLoadPoint** (FTU_Type *pFtu)

Set the CNTMAX flag.

LOCAL_INLINE void **FTU_HWA_DisableMaxLoadPoint** (FTU_Type *pFtu)

Clear the CNTMAX flag.

LOCAL_INLINE void **FTU_HWA_EnableMinLoadPoint** (FTU_Type *pFtu)

Set the CNTMIN flag.

LOCAL_INLINE void **FTU_HWA_DisableMinLoadPoint** (FTU_Type *pFtu)

Clear the CNTMIN flag.

LOCAL_INLINE void **FTU_HWA_EnableChannelMatchReload** (FTU_Type *pFtu, uint32_t u32Channel)

Set the CHNSEL flag of a channel.

LOCAL_INLINE void **FTU_HWA_DisableChannelMatchReload** (FTU_Type *pFtu, uint32_t u32Channel)

Clear the CHNSEL flag of a channel.

LOCAL_INLINE void **FTU_HWA_ConfigFreqOfReloadOp** (FTU_Type *pFtu, uint8_t u8Freq)

configure the frequency of the reload opportunities

LOCAL_INLINE void **FTU_HWA_ConfigDebugMode** (FTU_Type *pFtu, uint32_t u32Mode)

Selects the FTU behavior in Debug mode.

LOCAL_INLINE void **FTU_HWA_ConfigUpDownDisable** (FTU_Type *pFtu, **FTU_UpDownDisableType** eUpdownDisable)

Disable channel match trigger/interrupt when count-up/down in CPWM/QUAD mode.

LOCAL_INLINE void **FTU_HWA_EnableReloadPointInterrupt** (FTU_Type *pFtu)

Enable reload point interrupt.

LOCAL_INLINE void **FTU_HWA_DisableReloadPointInterrupt** (FTU_Type *pFtu)

Disable reload point interrupt.

LOCAL_INLINE void **FTU_HWA_EnableSWOControl** (FTU_Type *pFtu, uint32_t u32Mask)

Enable software output control of channels.

LOCAL_INLINE void **FTU_HWA_DisalbeSWOControl** (FTU_Type *pFtu, uint32_t u32Mask)

Disable software output control of channels.

LOCAL_INLINE void **FTU_HWA_SetSWOValue** (FTU_Type *pFtu, uint32_t u32Mask)

Set the software output control value.

LOCAL_INLINE void **FTU_HWA_ClearSWOValue** (FTU_Type *pFtu, uint32_t u32Mask)

Clear the software output control value.

LOCAL_INLINE void **FTU_HWA_ConfigPairDeadtimePrescaler** (FTU_Type *pFtu, uint32_t u32Channel, **FTU_DeadTimePrescalerType** ePrescaler)

Configure the pair deadtime prescaler value.

LOCAL_INLINE void **FTU_HWA_ConfigPairDeadtimeValue** (FTU_Type *pFtu, uint32_t u32Channel, uint32_t u32Value)

Configure the deadtime value.

LOCAL_INLINE void **FTU_HWA_ConfigDeadtimePrescaler** (FTU_Type *pFtu, **FTU_DeadTimePrescalerType** ePrescaler)

Configure the deadtime prescaler value.

LOCAL_INLINE void **FTU_HWA_ConfigDeadtimeValue** (FTU_Type *pFtu, uint32_t u32Value)

Configure the deadtime value.

LOCAL_INLINE uint32_t **FTU_HWA_GetCounterValue** (FTU_Type *pFtu)

Get the counter value of the selected FTU module.

LOCAL_INLINE void **FTU_HWA_ClearModuleCounter** (FTU_Type *pFtu, uint32_t u32RegValue)

Clear FTU counter.

LOCAL_INLINE void **FTU_HWA_ConfigChannel** (FTU_Type *pFtu, uint8_t u8Channel, uint32_t u32RegValue)

Configure FTU channel.

LOCAL_INLINE uint32_t **FTU_HWA_GetChannelValue** (FTU_Type *pFtu, uint8_t u8Channel)

Get FTU channel value.

LOCAL_INLINE void **FTU_HWA_SetChannelValue** (FTU_Type *pFtu, uint8_t u8Channel, uint32_t u32RegValue)

Set FTU channel value.

LOCAL_INLINE void **FTU_HWA_SetModuleCompareValue** (FTU_Type *pFtu, uint32_t u32RegValue)

Set FTU counter compare val.

LOCAL_INLINE void **FTU_HWA_SetCounterInitialValue** (FTU_Type *pFtu, uint32_t u32RegValue)

Set FTU counter initial value.

LOCAL_INLINE void **FTU_HWA_ConfigModuleMode** (FTU_Type *pFtu, uint32_t u32RegValue)

Configure FTU mode.

LOCAL_INLINE void **FTU_HWA_ConfigDeadtime** (FTU_Type *pFtu, uint32_t u32RegValue)

Configure FTU deadtime.

LOCAL_INLINE void **FTU_HWA_ConfigInputCaptureFilter** (FTU_Type *pFtu, uint32_t u32Channel, uint32_t u32FilterValue)

Configure input capture filter.

LOCAL_INLINE void **FTU_HWA_EnableModuleFault** (FTU_Type *pFtu, uint8_t u8InputMask)

Enable FTU module fault.

LOCAL_INLINE uint32_t **FTU_HWA_GetFaultEnable** (FTU_Type *pFtu)

get the fault input enable status

LOCAL_INLINE void **FTU_HWA_DisableModuleFault** (FTU_Type *pFtu, uint8_t u8Input)

Disable FTU module fault.

LOCAL_INLINE void **FTU_HWA_EnableModuleFaultGlitchFilter** (FTU_Type *pFtu, uint8_t u8Input)

Enable FTU module fault glitch filter.

LOCAL_INLINE void **FTU_HWA_DisableModuleFaultGlitchFilter** (FTU_Type *pFtu, uint8_t u8Input)

Disable FTU module fault glitch filter.

LOCAL_INLINE void **FTU_HWA_SetModuleFaultFilterValue** (FTU_Type *pFtu, uint8_t u8Value)

Set FTU module fault filter value.

LOCAL_INLINE void **FTU_HWA_SetFaultDelay0** (FTU_Type *pFtu, uint8_t u8Value)

Set FTU module fault disable channel output delay value0.

LOCAL_INLINE void **FTU_HWA_SetFaultDelay1** (FTU_Type *pFtu, uint8_t u8Value)

Set FTU module fault disable channel output delay value.

LOCAL_INLINE void **FTU_HWA_ConfigFaultPolarity** (FTU_Type *pFtu, uint32_t u32FaultIndex, uint8_t u8InputPol)

Configure FTU module fault polarity.

LOCAL_INLINE void **FTU_HWA_SetModulePrescale** (FTU_Type *pFtu, **FTU_PrescalerType** ePs)

Set FTU module prescale.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleOverflowFlag** (FTU_Type *pFtu)

Read FTU module overflow flag.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleOverflowIntrEnable** (FTU_Type *pFtu)

Read FTU module overflow interrupt enable.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleReloadFlag** (FTU_Type *pFtu)

Read FTU module reload flag.

LOCAL_INLINE uint8_t **FTU_HWA_ReadReloadIntrEnable** (FTU_Type *pFtu)

Read FTU module reload interrupt enable.

LOCAL_INLINE bool **FTU_HWA_ReadChannelInterruptFlag** (FTU_Type *pFtu, uint8_t u8Channel)

Read FTU channel interrupt flag.

LOCAL_INLINE void **FTU_HWA_ConfigTrigOutputMode** (FTU_Type *pFtu, uint8_t u8Channel,
FTU_TrigOutputModeType eMode)

Configure the channel output mode.

LOCAL_INLINE void **FTU_HWA_SelectFaultDelay** (FTU_Type *pFtu, uint8_t u8Channel,
FTU_FaultDisableDelayType eSelection)

Select fault disable channel output delay value.

LOCAL_INLINE uint8_t **FTU_HWA_ReadChannelInterruptEnableFlag** (FTU_Type *pFtu, uint8_t u8Channel)

Read FTU channel interrupt enable flag.

LOCAL_INLINE uint32_t **FTU_HWA_ReadModuleFaultFlag** (FTU_Type *pFtu)

Read FTU module fault(n) flag.

LOCAL_INLINE uint8_t **FTU_HWA_ReadFaultIntrEnable** (FTU_Type *pFtu)

Read FTU module fault interrupt enable.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleFaultDetectionFlag** (FTU_Type *pFtu)

Read FTU module fault detection flag.

LOCAL_INLINE void **FTU_HWA_EnableChannelsOutput** (FTU_Type *pFtu, uint8_t u8ChannelMask)

Enable FTU module channel(n) output.

LOCAL_INLINE uint8_t **FTU_HWA_GetModuleCpwmMode** (FTU_Type *pFtu)

Get cpwm mode of the FTU module.

LOCAL_INLINE void **FTU_HWA_EnableModuleCpwmMode** (FTU_Type *pFtu)

Enable FTU module cpwm mode.

LOCAL_INLINE void **FTU_HWA_DisableModuleCpwmMode** (FTU_Type *pFtu)

Disable FTU module cpwm mode.

LOCAL_INLINE void **FTU_HWA_ConfigModuleFilterPrescale** (FTU_Type *pFtu, **FTU_FilterPrescalerType** eFilterPs)

Set FTU module filter prescale.

LOCAL_INLINE void **FTU_HWA_ConfigChannelMode** (FTU_Type *pFtu, uint8_t u8Channel, **FTU_ChannelModeType** eMode)

LOCAL_INLINE void **FTU_HWA_ConfigChannelEdgeLevel** (FTU_Type *pFtu, uint8_t u8Channel, **FTU_ChannelEdgeLevelType** eEdgeLevel)

LOCAL_INLINE uint8_t **FTU_HWA_GetEdgeNumberCount** (FTU_Type *pFtu, uint8_t u8Channel)

LOCAL_INLINE void **FTU_HWA_ConfigEdgeNumber** (FTU_Type *pFtu, uint8_t u8Channel, uint8_t u8EdgeNumber)

LOCAL_INLINE void **FTU_HWA_SingleMeasurement** (FTU_Type *pFtu, uint8_t u8Channel)

Re-start measurement when in single mode.

LOCAL_INLINE void **FTU_HWA_EnableMeasureContinuous** (FTU_Type *pFtu, uint8_t u8Channel)

Set the measurement channel to continuous mode.

LOCAL_INLINE void **FTU_HWA_DisableMeasureContinuous** (FTU_Type *pFtu, uint8_t u8Channel)

Set the measurement channel to single mode.

LOCAL_INLINE void **FTU_HWA_EnableMeasureStartImmd** (FTU_Type *pFtu, uint8_t u8Channel)

Set the channel starts measuring after the first edge is detected.

LOCAL_INLINE void **FTU_HWA_ConfigInputCaptureMeasureMode** (FTU_Type *pFtu, uint8_t u8Channel, **FTU_MeasurementModeType** eMode)

LOCAL_INLINE void **FTU_HWA_DisableMeasureStartImmd** (FTU_Type *pFtu, uint8_t u8Channel)

Set the measurement starts immediately after activating the channel.

LOCAL_INLINE **FTU_MeasurementModeType** **FTU_HWA_GetMeasureMode** (FTU_Type *pFtu, uint8_t u8Channel)

Get channel measure mode.

LOCAL_INLINE void **FTU_HWA_SetChannelEpwmHighTrue** (FTU_Type *pFtu, uint8_t u8Channel)
Configure Edge-Aligned pwm mode channel to High-true pulses(clear output on match)

LOCAL_INLINE void **FTU_HWA_SetChannelEpwmLowTrue** (FTU_Type *pFtu, uint8_t u8Channel)
Configure Edge-Aligned pwm mode channel to Low-true pulses(set output on match)

LOCAL_INLINE void **FTU_HWA_ConfigChannelPwmLinkMode** (FTU_Type *pFtu, uint8_t u8Channel)
Configure pwm link mode channel.

LOCAL_INLINE void **FTU_HWA_EnableChannelInterrupt** (FTU_Type *pFtu, uint8_t u8Channel)
Enable FTU channel interrupt.

LOCAL_INLINE void **FTU_HWA_SetModuleUpdateRegBySync** (FTU_Type *pFtu)
Set MODE[FTUEN], this field define different free running counter and synchronization behavior.

LOCAL_INLINE void **FTU_HWA_ClearModuleUpdateRegBySync** (FTU_Type *pFtu)
Clear MODE[FTUEN], this field define different free running counter and synchronization behavior.

LOCAL_INLINE void **FTU_HWA_EnableModuleFaultInterrupt** (FTU_Type *pFtu)
Enable FTU module fault interrupt.

LOCAL_INLINE void **FTU_HWA_EnableChannelDma** (FTU_Type *pFtu, uint8_t u8Channel)
Enable FTU channel dma.

LOCAL_INLINE void **FTU_HWA_ConfigModuleFaultMode** (FTU_Type *pFtu, **FTU_FaultModeType** eMode)
Configure FTU module fault mode.

LOCAL_INLINE void **FTU_HWA_EnableTrigger0Sync** (FTU_Type *pFtu)
Enable synchronization hardware trigger 0.

LOCAL_INLINE void **FTU_HWA_EnableTrigger1Sync** (FTU_Type *pFtu)
Enable synchronization hardware trigger 1.

LOCAL_INLINE void **FTU_HWA_EnableTrigger2Sync** (FTU_Type *pFtu)

Enable synchronization hardware trigger 2.

LOCAL_INLINE void **FTU_HWA_EnableOutputMaskBySync** (FTU_Type *pFtu)

Enable FTU module output mask synchronization.

LOCAL_INLINE void **FTU_HWA_DisableOutputMaskBySync** (FTU_Type *pFtu)

Disable FTU module output mask synchronization.

LOCAL_INLINE void **FTU_HWA_GenerateSwSync** (FTU_Type *pFtu)

Generate FTU software trigger.

LOCAL_INLINE void **FTU_HWA_ConfigTrigMode** (FTU_Type *pFtu, **FTU_TrigModeType** eTrigMode)

Configure FTU trigger mode.

LOCAL_INLINE void **FTU_HWA_EnableChannelDeadtime** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) deadtime functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelComplement** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) complement functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelSync** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) synchronization functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelPhase** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelPhase** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel($n/n+1$) Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelEnhancedPhase** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) Enhanced Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelEnhancedPhase** (FTU_Type *pFtu, uint8_t u8Channel)
Disable channel(n/n+1) Enhanced Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelFault** (FTU_Type *pFtu, uint8_t u8Channel)
Enable channel(n/n+1) fault functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelTriggerOut** (FTU_Type *pFtu, uint8_t u8ChannelMask)
Enable channel trigger out.

LOCAL_INLINE void **FTU_HWA_DisableChannelTriggerOut** (FTU_Type *pFtu, uint8_t u8ChannelMask)
Disable channel trigger out.

LOCAL_INLINE void **FTU_HWA_ClearChannelTriggerFlag** (FTU_Type *pFtu)
clear the channel trigger flag

LOCAL_INLINE void **FTU_HWA_EnableReloadTrigger** (FTU_Type *pFtu)
Enable reload trigger.

LOCAL_INLINE void **FTU_HWA_SetChannelPolarity** (FTU_Type *pFtu, uint8_t u8Mask)
Set channel polarity.

LOCAL_INLINE void **FTU_HWA_ConfigModuleOutmask** (FTU_Type *pFtu, uint8_t u8Channel)
Set FTU module outmask.

LOCAL_INLINE void **FTU_HWA_ConfigChannelFilterValue** (FTU_Type *pFtu, uint8_t u8Channel, uint8_t u8FilterVal)
Set channel filter value.

LOCAL_INLINE void **FTU_HWA_ConfigQuadratureMode** (FTU_Type *pFtu, **FTU_QuadratureModeType** eMode)
Configure quadrature mode.

LOCAL_INLINE void **FTU_HWA_EnablePhbInv** (FTU_Type *pFtu)

inverted polarity of phase B input signal

LOCAL_INLINE void **FTU_HWA_DisablePhbInv** (FTU_Type *pFtu)

do not inverted polarity of phase B input signal

LOCAL_INLINE void **FTU_HWA_EnablePhaInv** (FTU_Type *pFtu)

inverted polarity of phase A input signal

LOCAL_INLINE void **FTU_HWA_DisablePhaInv** (FTU_Type *pFtu)

do not inverted polarity of phase B input signal

LOCAL_INLINE void **FTU_HWA_EnablePhaGlitchFilter** (FTU_Type *pFtu)

Enable phase A glitch filter.

LOCAL_INLINE void **FTU_HWA_DisablePhaGlitchFilter** (FTU_Type *pFtu)

Disable phase A glitch filter.

LOCAL_INLINE void **FTU_HWA_EnablePhbGlitchFilter** (FTU_Type *pFtu)

Enable phase B glitch filter.

LOCAL_INLINE void **FTU_HWA_DisablePhbGlitchFilter** (FTU_Type *pFtu)

Disable phase B glitch filter.

LOCAL_INLINE void **FTU_HWA_EnableQuadratureMode** (FTU_Type *pFtu)

Enable FTU module quadrature mode.

LOCAL_INLINE void **FTU_HWA_DisableQuadratureMode** (FTU_Type *pFtu)

Disable FTU module quadrature mode.

LOCAL_INLINE void **FTU_HWA_ConfigModuleDebugMode** (FTU_Type *pFtu, **FTU_DebugModeType** eDbgMode)

Configure FTU module debug mode.

LOCAL_INLINE void **FTU_HWA_ConfigModuleClkSrc** (FTU_Type *pFtu, **FTU_ModuleClkSrcType** eClkSrc)
Set FTU module clock source.

LOCAL_INLINE void **FTU_HWA_ClearOverflowFlag** (FTU_Type *pFtu)
Clear FTU module overflow flag.

LOCAL_INLINE void **FTU_HWA_ClearReloadFlag** (FTU_Type *pFtu)
Clear FTU module reload flag.

LOCAL_INLINE void **FTU_HWA_EnableOverflowInterrupt** (FTU_Type *pFtu)
Enable FTU module interrupt.

LOCAL_INLINE void **FTU_HWA_DisableOverflowInterrupt** (FTU_Type *pFtu)
Disable FTU module overflow interrupt.

LOCAL_INLINE void **FTU_HWA_ClearChannelInterruptFlag** (FTU_Type *pFtu, uint8_t u8Channel)
Clear FTU channel interrupt flag.

LOCAL_INLINE void **FTU_HWA_DisableChannelInterrupt** (FTU_Type *pFtu, uint8_t u8Channel)
Disable channel interrupt.

LOCAL_INLINE void **FTU_HWA_DisableWriteProtection** (FTU_Type *pFtu)
Disable write protection.

LOCAL_INLINE void **FTU_HWA_DisableModuleFaultInterrupt** (FTU_Type *pFtu)
Disable FTU module fault interrupt.

LOCAL_INLINE void **FTU_HWA_DisableTrigger0Sync** (FTU_Type *pFtu)
Disable synchronization hardware trigger 0.

LOCAL_INLINE void **FTU_HWA_DisableTrigger1Sync** (FTU_Type *pFtu)
Disable synchronization hardware trigger 1.

LOCAL_INLINE void **FTU_HWA_DisableTrigger2Sync** (FTU_Type *pFtu)

Disable synchronization hardware trigger 2.

LOCAL_INLINE void **FTU_HWA_DisableGenerateSwSync** (FTU_Type *pFtu)

Disable generate FTU software trigger.

LOCAL_INLINE void **FTU_HWA_DisableReloadTrigger** (FTU_Type *pFtu)

Disable reload trigger.

LOCAL_INLINE void **FTU_HWA_DisableChannelDeadtime** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) deadtime functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelComplement** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) complement functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelSync** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) synchronization functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelFault** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) fault functionality.

LOCAL_INLINE void **FTU_HWA_ClearChannelPolarity** (FTU_Type *pFtu, uint8_t u8Channel)

Clear channel polarity.

LOCAL_INLINE uint32_t **FTU_HWA_GetChannelPolarity** (FTU_Type *pFtu, uint8_t u8Channel)

Get channel polarity.

LOCAL_INLINE void **FTU_HWA_ClearModuleFaultFlag** (FTU_Type *pFtu, uint8_t u8Flag)

Clear FTU module fault flag.

LOCAL_INLINE **FTU_QuadratureDirectionType** **FTU_HWA_GetQuadDirection** (FTU_Type *pFtu)

Get FTU Counter Direction In Quadrature Decoder Mode.

LOCAL_INLINE void **FTU_HWA_ClearGlobalFaultFlag** (FTU_Type *pFtu)

Clear FTU global fault flag.

Detailed Description

Macro Definition Documentation

```
#define FTU_CHCTRL_COMPLEMENT_CHANNEL( channel)  (uint32_t)((uint32_t)1U <<  
(((uint32_t)(channel) >> 1U) << 3U) + 1U))
```

The definition of enable the complement of channel.

```
#define FTU_CHCTRL_DEADTIME_CHANNEL( channel)  (uint32_t)((uint32_t)1U <<  
(((uint32_t)(channel) >> 1U) << 3U) + 4U))
```

The definition of enable the deadtime of channel and channel+1.

```
#define FTU_CHCTRL_EPHASE_CHANNEL( channel)  (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >>  
1U) << 3U) + 7U))
```

The definition of enable the Enhanced Phase shift mode for channel and channel+1.

```
#define FTU_CHCTRL_FAULT_CHANNEL( channel)  (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >>  
1U) << 3U) + 6U))
```

The definition of enable the fault control in channel and channel+1.

```
#define FTU_CHCTRL_PHASE_CHANNEL( channel)  (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U))
```

The definition of enable the Phase shift mode for channel and channel+1.

```
#define FTU_CHCTRL_SYNCEN_CHANNEL( channel)  (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 5U))
```

The definition of enable the synchronization for channel and channel+1.

Enumeration Type Documentation

enum FTU_BitStatusType

register bit status

Enumerator:

FTU_BIT_LOW	
FTU_BIT_HIGH	

enum FTU_ChannelEdgeLevelType

Enumerator:

FTU_CHANNEL_EDGE_NOT_USED	
FTU_CHANNEL_EDGE_RISING	
FTU_CHANNEL_EDGE_FALLING	

FTU_CHANNEL_ EDGE_BOTH	
FTU_CHANNEL_ OC_TOGGLE	
FTU_CHANNEL_ OC_CLEAR	
FTU_CHANNEL_ OC_SET	
FTU_CHANNEL_ PWM_HIGH_TR UE	
FTU_CHANNEL_ PWM_LOW_TRU E	

enum FTU_ChannelModeType

Enumerator:

FTU_CHANNEL_ MODE_INPUT	
FTU_CHANNEL_ MODE_OUTPUT _COMPARE	
FTU_CHANNEL_ MODE_EDGE_AL IGN_PWM	
FTU_CHANNEL_ MODE_DO_NOT _CARE	

enum FTU_DeadTimePrescalerType

Deadtime Prescaler Value.

Enumerator:

FTU_DEADTIME _PRESCALER_DI V_1	Divide the FTU input clock by 1
FTU_DEADTIME _PRESCALER_DI V_4	Divide the FTU input clock by 4
FTU_DEADTIME _PRESCALER_DI V_16	Divide the FTU input clock by 16

enum FTU_DebugModeType

Select the FTU Debug Mode.

Enumerator:

FTU_DBG_COU NTER_STOP_CH N_WORKS	FTU counter stopped, channel works as function mode.
FTU_DBG_COU NTER_STOP_CH N_FORCE_SAFE	FTU counter stopped, channel output force to safe state.
FTU_DBG_COU NTER_STOP_CH N_FROZEN	FTU counter stopped, channel output is frozen.
FTU_DBG_COU NTER_WORKS_ CHN_WORKS	FTU counter works as function mode, channel works as function mode.

enum FTU_FaultDisableDelayType

Fault disable channel output delay value selection.

Enumerator:

FTU_FAULT_DISABLE_DELAY_NO_DELAY	Select no delay
FTU_FAULT_DISABLE_DELAY_VALUE_0	Select delay value 0
FTU_FAULT_DISABLE_DELAY_VALUE_1	Select delay value 1

enum FTU_FaultModeType

fault mode enumeration

Enumerator:

FTU_FAULT_MODE_DISABLED	Fault control is disabled for all channels
FTU_FAULT_MODE_EVEN_CHANNELS_MANUAL_CLEARING	Fault control is enabled for even channels only (channels 0, 2, 4, and 6), and the selected mode is the manual fault clearing
FTU_FAULT_MODE_ALL_CHANNELS_MANUAL_CLEARING	Fault control is enabled for all channels, and the selected mode is the manual fault clearing
FTU_FAULT_MODE_ALL_CHANNELS_AUTO_CLEARING	Fault control is enabled for all channels, and the selected mode is the automatic fault clearing

enum FTU_FilterPrescalerType

Select the prescaler of the FTU filter.

Enumerator:

FTU_FLT_DIV_1	
FTU_FLT_DIV_2	
FTU_FLT_DIV_3	
FTU_FLT_DIV_4	
FTU_FLT_DIV_5	
FTU_FLT_DIV_6	
FTU_FLT_DIV_7	
FTU_FLT_DIV_8	
FTU_FLT_DIV_9	
FTU_FLT_DIV_1 0	
FTU_FLT_DIV_1 1	
FTU_FLT_DIV_1 2	
FTU_FLT_DIV_1 3	
FTU_FLT_DIV_1 4	
FTU_FLT_DIV_1 5	
FTU_FLT_DIV_1 6	

enum FTU_MeasurementModeType

Input Capture Measurement Mode.

Enumerator:

FTU_MEASURE_ MODE_OFF	Do not use measurement mode
FTU_MEASURE_ MODE_DUTY_CYCLE	Duty-cycle Measure

FTU_MEASURE_ MODE_PERIOD	Period Measure
FTU_MEASURE_ EDGE_NUMBER	Edge Number Measure
FTU_MEASURE_ EXPECT_EDGE_ NUMBER	Expect Edge Number Measure
FTU_MEASURE_ ICENM_WIND_W RITE	Configure ICENM Window

enum FTU_ModuleClkSrcType

Ftu module clock source.

Enumerator:

FTU_MODULE_ NO_CLK	No clock selected
FTU_MODULE_I NTERNAL_CLK	FTU input clock
FTU_MODULE_E XTERNAL_CLK	External pin input clock

enum FTU_PrescalerType

Select the prescaler of the FTU.

Enumerator:

FTU_DIV_1	
FTU_DIV_2	
FTU_DIV_4	
FTU_DIV_8	

FTU_DIV_16	
FTU_DIV_32	
FTU_DIV_64	
FTU_DIV_128	

enum FTU_QuadratureDirectionType

FTU Counter Direction In Quadrature Decoder Mode.

Enumerator:

FTU_QUADIR_D ECREMENT	FTU counter decrement
FTU_QUADIR_IN CREMENT	FTU counter increment

enum FTU_QuadratureModeType

selects the encoding mode used in the Quadrature Decoder mode

Enumerator:

FTU_QUADMOD E_PHA_PHB_EN CODING_MODE	Phase A and phase B encoding mode
FTU_QUADMOD E_COUNT_DIR_ ENCODING_MO DE	Count and direction encoding mode

enum FTU_TrigModeType

hardware trig(n) mode

Enumerator:

FTU_CLEARS_TRIG_WHEN_DETECTED	FTU clears the TRIG(n) bit when the hardware trigger j is detected, where n = 0, 1,2.
FTU_NOT_CLEAR_TRIG_WHEN_DETECTED	FTU does not clear the TRIG(n) bit when the hardware trigger j is detected, where n = 0, 1,2.

enum FTU_TrigOutputModeType

Trigger output control.

Enumerator:

FTU_TRIG_OUTPUT_AS_CHANNEL_MODE	channel outputs will be controlled by channel mode
FTU_TRIG_OUTPUT_AS_TRIGGER	If a match in the channel occurs, channel output will be a trigger

enum FTU_UpDownDisableType

Disable channel match trigger/interrupt when count-up/down in CPWM/QUAD mode.

Enumerator:

FTU_DISABLE_TRIG_INTR_NONE	No effect
FTU_DISABLE_TRIG_INTR_COUNT_DOWN	Disable trigger/interrupt when count-down

FTU_DISABLE_T RIG_INTR_CNT_ UP	Disable trigger/interrupt when count-up
FTU_DISABLE_T RIG_INTR_CNT_ UP_DOWN	Disable trigger/interrupt when count-up/down

Function Documentation

LOCAL_INLINE void FTU_HWA_ClearChannelInterruptFlag (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Clear FTU channel interrupt flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_ClearChannelPolarity (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Clear channel polarity.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel, bit 0-7 indicate channel 0-7

LOCAL_INLINE void FTU_HWA_ClearChannelSyncEnable (FTU_Type * *pFtu*, uint32_t *u32Index*)

Clear the SYNCEN flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Index</i>	the index of SYNCEN

LOCAL_INLINE void FTU_HWA_ClearChannelTriggerFlag (FTU_Type * *pFtu*)

clear the channel trigger flag

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearCntinSync (FTU_Type * *pFtu*)

Clear the CNTINC flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearGlobalFaultFlag (FTU_Type * *pFtu*)

Clear FTU global fault flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearModuleCounter (FTU_Type * *pFtu*, uint32_t *u32RegValue*)

Clear FTU counter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32RegValue</i>	CNT register value

LOCAL_INLINE void FTU_HWA_ClearModuleFaultFlag (FTU_Type * *pFtu*, uint8_t *u8Flag*)

Clear FTU module fault flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Flag</i>	bit 0-3 indicate fault flag 0-3.

LOCAL_INLINE void FTU_HWA_ClearModuleRegister (FTU_Type * *pFtu*)

Clear FTU module registers.

Parameters

<i>pFtu</i>	FTU instance
-------------	--------------

LOCAL_INLINE void FTU_HWA_ClearModuleUpdateRegBySync (FTU_Type * *pFtu*)

Clear MODE[FTUEN], this field define different free running counter and synchronization behavior.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearOverflowFlag (FTU_Type * *pFtu*)

Clear FTU module overflow flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearPwmLoadEnable (FTU_Type * *pFtu*)

Clear the LDOK flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearPwmSyncMode (FTU_Type * *pFtu*)

Clear the PWMSYNC flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearReinitBySync (FTU_Type * *pFtu*)

Clear the REINIT flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearReloadFlag (FTU_Type * *pFtu*)

Clear FTU module reload flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_ClearSWOValue (FTU_Type * *pFtu*, uint32_t *u32Mask*)

Clear the software output control value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Mask</i>	channel mask

LOCAL_INLINE void FTU_HWA_ConfigChannel (FTU_Type * *pFtu*, uint8_t *u8Channel*, uint32_t *u32RegValue*)

Configure FTU channel.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

<i>u8Channel</i>	channel number, range is 0-7
<i>u32RegValue</i>	the value to write to CSC register

LOCAL_INLINE void FTU_HWA_ConfigChannelEdgeLevel (FTU_Type * *pFtu*, uint8_t *u8Channel*, FTU_ChannelEdgeLevelType *eEdgeLevel*)

LOCAL_INLINE void FTU_HWA_ConfigChannelFilterValue (FTU_Type * *pFtu*, uint8_t *u8Channel*, uint8_t *u8FilterVal*)

Set channel filter value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-3
<i>u8FilterVal</i>	FTU filter value

LOCAL_INLINE void FTU_HWA_ConfigChannelMode (FTU_Type * *pFtu*, uint8_t *u8Channel*, FTU_ChannelModeType *eMode*)

LOCAL_INLINE void FTU_HWA_ConfigChannelPwmLinkMode (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Configure pwm link mode channel.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_ConfigDeadtime (FTU_Type * *pFtu*, uint32_t *u32RegValue*)

Configure FTU deadtime.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32RegValue</i>	DEADTIME register value

```
LOCAL_INLINE void FTU_HWA_ConfigDeadtimePrescaler (FTU_Type * pFtu,
FTU_DeadTimePrescalerType ePrescaler)
```

Configure the deadtime prescaler value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>ePrescaler</i>	prescaler value

```
LOCAL_INLINE void FTU_HWA_ConfigDeadtimeValue (FTU_Type * pFtu, uint32_t u32Value)
```

Configure the deadtime value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Value</i>	deadtime value

```
LOCAL_INLINE void FTU_HWA_ConfigDebugMode (FTU_Type * pFtu, uint32_t u32Mode)
```

Selects the FTU behavior in Debug mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Mode</i>	debug mode

```
LOCAL_INLINE void FTU_HWA_ConfigEdgeNumber (FTU_Type * pFtu, uint8_t u8Channel, uint8_t
u8EdgeNumber)
```

```
LOCAL_INLINE void FTU_HWA_ConfigFaultPolarity (FTU_Type * pFtu, uint32_t u32FaultIndex, uint8_t
u8InputPol)
```

Configure FTU module fault polarity.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32FaultIndex</i>	index of the fault
<i>u8InputPol</i>	the polarity of the fault input

LOCAL_INLINE void FTU_HWA_ConfigFreqOfReloadOp (FTU_Type * *pFtu*, uint8_t *u8Freq*)

configure the frequency of the reload opportunities

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Freq</i>	the number of enabled reload opportunities that should happen until an enabled reload opportunity becomes a reload point

LOCAL_INLINE void FTU_HWA_ConfigInputCaptureFilter (FTU_Type * *pFtu*, uint32_t *u32Channel*, uint32_t *u32FilterValue*)

Configure input capture filter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Channel</i>	Channel of FTU instance
<i>u32FilterValue</i>	Filter value

LOCAL_INLINE void FTU_HWA_ConfigInputCaptureMeasureMode (FTU_Type * *pFtu*, uint8_t *u8Channel*, FTU_MeasurementModeType *eMode*)

LOCAL_INLINE void FTU_HWA_ConfigModuleClkSrc (FTU_Type * *pFtu*, FTU_ModuleClkSrcType *eClkSrc*)

Set FTU module clock source.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>eClkSrc</i>	FTU module clock source type

LOCAL_INLINE void FTU_HWA_ConfigModuleDebugMode (FTU_Type * *pFtu*, FTU_DebugModeType *eDbgMode*)

Configure FTU module debug mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>eDbgMode</i>	debug mode enumeration

LOCAL_INLINE void FTU_HWA_ConfigModuleFaultMode (FTU_Type * *pFtu*, FTU_FaultModeType *eMode*)

Configure FTU module fault mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>eMode</i>	Fault mode type

LOCAL_INLINE void FTU_HWA_ConfigModuleFilterPrescale (FTU_Type * *pFtu*, FTU_FilterPrescalerType *eFilterPs*)

Set FTU module filter prescale.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>eFilterPs</i>	FTU clock filter prescaler enumeration

LOCAL_INLINE void FTU_HWA_ConfigModuleMode (FTU_Type * *pFtu*, uint32_t *u32RegValue*)

Configure FTU mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32RegValue</i>	MODE register value

LOCAL_INLINE void FTU_HWA_ConfigModuleOutmask (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Set FTU module outmask.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	Bit of channel indicate channel number, range: 0~7 bit

LOCAL_INLINE void FTU_HWA_ConfigPairDeadtimePrescaler (FTU_Type * *pFtu*, uint32_t *u32Channel*, FTU_DeadTimePrescalerType *ePrescaler*)

Configure the pair deadtime prescaler value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Channel</i>	channel index
<i>ePrescaler</i>	prescaler value

LOCAL_INLINE void FTU_HWA_ConfigPairDeadtimeValue (FTU_Type * *pFtu*, uint32_t *u32Channel*, uint32_t *u32Value*)

Configure the deadtime value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Channel</i>	channel index
<i>u32Value</i>	deadtime value

LOCAL_INLINE void FTU_HWA_ConfigQuadratureMode (FTU_Type * *pFtu*, FTU_QuadratureModeType *eMode*)

Configure quadrature mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>eMode</i>	FTU quadrature mode

LOCAL_INLINE void FTU_HWA_ConfigTrigMode (FTU_Type * *pFtu*, FTU_TrigModeType *eTrigMode*)

Configure FTU trigger mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>eTrigMode</i>	Trigger mode type

LOCAL_INLINE void FTU_HWA_ConfigTrigOutputMode (FTU_Type * *pFtu*, uint8_t *u8Channel*, FTU_TrigOutputModeType *eMode*)

Configure the channel output mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.
<i>eMode</i>	output mode of the selected channel.

LOCAL_INLINE void FTU_HWA_ConfigUpDownDisable (FTU_Type * *pFtu*, FTU_UpDownDisableType *eUpDownDisable*)

Disable channel match trigger/interrupt when count-up/down in CPWM/QUAD mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>eUpDownDisable</i>	up-down disable mode

LOCAL_INLINE void FTU_HWA_DisableChannelComplement (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Disable channel(n/n+1) complement functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_DisableChannelDeadtime (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Disable channel(n/n+1) deadtime functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_DisableChannelEnhancedPhase (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Disable channel(n/n+1) Enhanced Phase Shift Mode functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_DisableChannelFault (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Disable channel(n/n+1) fault functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_DisableChannelInterrupt (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Disable channel interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_DisableChannelMatchReload (FTU_Type * *pFtu*, uint32_t *u32Channel*)

Clear the CHNSEL flag of a channel.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Channel</i>	channel index

LOCAL_INLINE void FTU_HWA_DisableChannelPhase (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Disable channel(n/n+1) Phase Shift Mode functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_DisableChannelSync (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Disable channel(n/n+1) synchronization functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_DisableChannelTriggerOut (FTU_Type * *pFtu*, uint8_t *u8ChannelMask*)

Disable channel trigger out.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

<i>u8ChannelMask</i>	FTU channel mask, bit 0-7 indicate channel 0-7
----------------------	--

LOCAL_INLINE void FTU_HWA_DisableGenerateSwSync (FTU_Type * *pFtu*)

Disable generate FTU software trigger.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableGlobalTimeBase (FTU_Type * *pFtu*)

Disable Global Time Base Enable.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableMaxLoadPoint (FTU_Type * *pFtu*)

Clear the CNTMAX flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableMeasureContinuous (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Set the measurement channel to single mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_DisableMeasureStartImmd (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Set the measurement starts immediately after activating the channel.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_DisableMinLoadPoint (FTU_Type * *pFtu*)

Clear the CNTMIN flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableModuleCpwmMode (FTU_Type * *pFtu*)

Disable FTU module cpwm mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableModuleFault (FTU_Type * *pFtu*, uint8_t *u8Input*)

Disable FTU module fault.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Input</i>	Fault input number, 0-3 bit indicate fault 0-3

LOCAL_INLINE void FTU_HWA_DisableModuleFaultGlitchFilter (FTU_Type * *pFtu*, uint8_t *u8Input*)

Disable FTU module fault glitch filter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Input</i>	Fault input number, 0-3 bit indicate fault 0-3

LOCAL_INLINE void FTU_HWA_DisableModuleFaultInterrupt (FTU_Type * *pFtu*)

Disable FTU module fault interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableOutputMaskBySync (FTU_Type * *pFtu*)

Disable FTU module output mask synchronization.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableOverflowInterrupt (FTU_Type * *pFtu*)

Disable FTU module overflow interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisablePhaGlitchFilter (FTU_Type * *pFtu*)

Disable phase A glitch filter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisablePhaInv (FTU_Type * *pFtu*)

do not inverted polarity of phase B input signal

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisablePhbGlitchFilter (FTU_Type * *pFtu*)

Disable phase B glitch filter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisablePhbInv (FTU_Type * *pFtu*)

do not inverted polarity of phase B input signal

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableQuadratureMode (FTU_Type * *pFtu*)

Disable FTU module quadrature mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableReloadPointInterrupt (FTU_Type * *pFtu*)

Disable reload point interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableReloadTrigger (FTU_Type * *pFtu*)

Disable reload trigger.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableTrigger0Sync (FTU_Type * *pFtu*)

Disable synchronization hardware trigger 0.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableTrigger1Sync (FTU_Type * *pFtu*)

Disable synchronization hardware trigger 1.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableTrigger2Sync (FTU_Type * *pFtu*)

Disable synchronization hardware trigger 2.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisableWriteProtection (FTU_Type * *pFtu*)

Disable write protection.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_DisalbeSWOControl (FTU_Type * *pFtu*, uint32_t *u32Mask*)

Disable software output control of channels.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Mask</i>	channel mask

LOCAL_INLINE void FTU_HWA_EnableChannelComplement (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable channel(n/n+1) complement functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_EnableChannelDeadtime (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable channel(n/n+1) deadtime functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_EnableChannelDma (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable FTU channel dma.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_EnableChannelEnhancedPhase (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable channel(n/n+1) Enhanced Phase Shift Mode functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_EnableChannelFault (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable channel(n/n+1) fault functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_EnableChannelInterrupt (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable FTU channel interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_EnableChannelMatchReload (FTU_Type * *pFtu*, uint32_t *u32Channel*)

Set the CHNSEL flag of a channel.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Channel</i>	channel index

LOCAL_INLINE void FTU_HWA_EnableChannelPhase (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable channel(n/n+1) Phase Shift Mode functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

<i>u8Channel</i>	channel number, range is 0-7
------------------	------------------------------

LOCAL_INLINE void FTU_HWA_EnableChannelsOutput (FTU_Type * *pFtu*, uint8_t *u8ChannelMask*)

Enable FTU module channel(n) output.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8ChannelMask</i>	0-7 bit indicate 0-7 channel

LOCAL_INLINE void FTU_HWA_EnableChannelSync (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Enable channel(n/n+1) synchronization functionality.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

LOCAL_INLINE void FTU_HWA_EnableChannelTriggerOut (FTU_Type * *pFtu*, uint8_t *u8ChannelMask*)

Enable channel trigger out.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8ChannelMask</i>	FTU channel mask, bit 0-7 indicate channel 0-7

LOCAL_INLINE void FTU_HWA_EnableGlobalTimeBase (FTU_Type * *pFtu*)

Enable Global Time Base Enable.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableMaxLoadPoint (FTU_Type * *pFtu*)

Set the CNTMAX flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableMeasureContinous (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Set the measurement channel to continuous mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_EnableMeasureStartImmd (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Set the channel starts measuring after the first edge is detected.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_EnableMinLoadPoint (FTU_Type * *pFtu*)

Set the CNTMIN flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableModuleCpwmMode (FTU_Type * *pFtu*)

Enable FTU module cpwm mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableModuleFault (FTU_Type * *pFtu*, uint8_t *u8InputMask*)

Enable FTU module fault.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8InputMask</i>	Fault input mask, 0-3 bit indicate fault 0-3

LOCAL_INLINE void FTU_HWA_EnableModuleFaultGlitchFilter (FTU_Type * *pFtu*, uint8_t *u8Input*)

Enable FTU module fault glitch filter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Input</i>	Fault input number, 0-3 bit indicate fault 0-3

LOCAL_INLINE void FTU_HWA_EnableModuleFaultInterrupt (FTU_Type * *pFtu*)

Enable FTU module fault interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableOutputMaskBySync (FTU_Type * *pFtu*)

Enable FTU module output mask synchronization.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableOverflowInterrupt (FTU_Type * *pFtu*)

Enable FTU module interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnablePhaGlitchFilter (FTU_Type * *pFtu*)

Enable phase A glitch filter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnablePhaInv (FTU_Type * *pFtu*)

inverted polarity of phase A input signal

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnablePhbGlitchFilter (FTU_Type * *pFtu*)

Enable phase B glitch filter.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnablePhbInv (FTU_Type * *pFtu*)

inverted polarity of phase B input signal

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableQuadratureMode (FTU_Type * *pFtu*)

Enable FTU module quadrature mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableReloadPointInterrupt (FTU_Type * *pFtu*)

Enable reload point interrupt.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableReloadTrigger (FTU_Type * *pFtu*)

Enable reload trigger.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableTrigger0Sync (FTU_Type * *pFtu*)

Enable synchronization hardware trigger 0.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableTrigger1Sync (FTU_Type * *pFtu*)

Enable synchronization hardware trigger 1.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableTrigger2Sync (FTU_Type * *pFtu*)

Enable synchronization hardware trigger 2.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_EnableSWOControl (FTU_Type * *pFtu*, uint32_t *u32Mask*)

Enable software output control of channels.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Mask</i>	channel mask

LOCAL_INLINE void FTU_HWA_GenerateSwSync (FTU_Type * *pFtu*)

Generate FTU software trigger.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE uint32_t FTU_HWA_GetChannelPolarity (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Get channel polarity.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel, bit 0-7 indicate channel 0-7

Returns

polarity of the channels

LOCAL_INLINE uint32_t FTU_HWA_GetChannelValue (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Get FTU channel value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7

Returns

CVn register value

LOCAL_INLINE uint32_t FTU_HWA_GetCounterValue (FTU_Type * *pFtu*)

Get the counter value of the selected FTU module.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE uint8_t FTU_HWA_GetEdgeNumberCount (FTU_Type * *pFtu*, uint8_t *u8Channel*)

LOCAL_INLINE uint32_t FTU_HWA_GetFaultEnable (FTU_Type * *pFtu*)

get the fault input enable status

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE FTU_MeasurementModeType FTU_HWA_GetMeasureMode (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Get channel measure mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel

Returns

measure mode

LOCAL_INLINE uint8_t FTU_HWA_GetModuleCpwmMode (FTU_Type * *pFtu*)

Get cpwm mode of the FTU module.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

cpwm mode bit value of the FTU module

LOCAL_INLINE FTU_QuadratureDirectionType FTU_HWA_GetQuadDirection (FTU_Type * *pFtu*)

Get FTU Counter Direction In Quadrature Decoder Mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

FTU Counter Direction In Quadrature Decoder Mode

LOCAL_INLINE uint8_t FTU_HWA_ReadChannelInterruptEnableFlag (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Read FTU channel interrupt enable flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

Returns

FTU channel interrupt flag

LOCAL_INLINE bool FTU_HWA_ReadChannelInterruptFlag (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Read FTU channel interrupt flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

Returns

FTU channel interrupt flag

LOCAL_INLINE uint8_t FTU_HWA_ReadFaultIntrEnable (FTU_Type * *pFtu*)

Read FTU module fault interrupt enable.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

FTU fault interrupt enable

LOCAL_INLINE uint8_t FTU_HWA_ReadModuleFaultDetectionFlag (FTU_Type * *pFtu*)

Read FTU module fault detection flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

FTU fault detection flag

LOCAL_INLINE uint32_t FTU_HWA_ReadModuleFaultFlag (FTU_Type * *pFtu*)

Read FTU module fault(n) flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

FTU fault flag

LOCAL_INLINE uint8_t FTU_HWA_ReadModuleOverflowFlag (FTU_Type * *pFtu*)

Read FTU module overflow flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

FTU overflow flag

LOCAL_INLINE uint8_t FTU_HWA_ReadModuleOverflowIntrEnable (FTU_Type * *pFtu*)

Read FTU module overflow interrupt enable.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

FTU overflow interrupt enable

LOCAL_INLINE uint8_t FTU_HWA_ReadModuleReloadFlag (FTU_Type * *pFtu*)

Read FTU module reload flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

FTU reload flag

LOCAL_INLINE uint8_t FTU_HWA_ReadReloadIntrEnable (FTU_Type * *pFtu*)

Read FTU module reload interrupt enable.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

Returns

reload interrupt enable

LOCAL_INLINE void FTU_HWA_SelectFaultDelay (FTU_Type * *pFtu*, uint8_t *u8Channel*, FTU_FaultDisableDelayType *eSelection*)

Select fault disable channel output delay value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.
<i>eSelection</i>	Fault disable channel output delay value selection.

LOCAL_INLINE void FTU_HWA_SetChannelEpwmHighTrue (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Configure Edge-Aligned pwm mode channel to High-true pulses(clear output on match)

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

LOCAL_INLINE void FTU_HWA_SetChannelEpwmLowTrue (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Configure Edge-Aligned pwm mode channel to Low-true pulses(set output on match)

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

<i>u8Channel</i>	FTU channel number, range is 0-7.
------------------	-----------------------------------

LOCAL_INLINE void FTU_HWA_SetChannelPolarity (FTU_Type * *pFtu*, uint8_t *u8Mask*)

Set channel polarity.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Mask</i>	FTU channel mask, bit 0-7 indicate channel 0-7

LOCAL_INLINE void FTU_HWA_SetChannelSyncEnable (FTU_Type * *pFtu*, uint32_t *u32Index*)

Set the SYNCEN flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Index</i>	the index of SYNCEN

LOCAL_INLINE void FTU_HWA_SetChannelValue (FTU_Type * *pFtu*, uint8_t *u8Channel*, uint32_t *u32RegValue*)

Set FTU channel value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	channel number, range is 0-7
<i>u32RegValue</i>	CVn register value

LOCAL_INLINE void FTU_HWA_SetCntinSync (FTU_Type * *pFtu*)

Set the CNTINC flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_SetCounterInitialValue (FTU_Type * *pFtu*, uint32_t *u32RegValue*)

Set FTU counter initial value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32RegValue</i>	CNTIN register value

LOCAL_INLINE void FTU_HWA_SetFaultDelay0 (FTU_Type * *pFtu*, uint8_t *u8Value*)

Set FTU module fault disable channel output delay value0.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Value</i>	FTU fault delay value0

LOCAL_INLINE void FTU_HWA_SetFaultDelay1 (FTU_Type * *pFtu*, uint8_t *u8Value*)

Set FTU module fault disable channel output delay value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Value</i>	FTU fault delay value1

LOCAL_INLINE void FTU_HWA_SetModuleCompareValue (FTU_Type * *pFtu*, uint32_t *u32RegValue*)

Set FTU counter compare val.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32RegValue</i>	MOD register value

LOCAL_INLINE void FTU_HWA_SetModuleFaultFilterValue (FTU_Type * *pFtu*, uint8_t *u8Value*)

Set FTU module fault filter value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Value</i>	FTU fault filter value

LOCAL_INLINE void FTU_HWA_SetModulePrescale (FTU_Type * *pFtu*, FTU_PrescalerType *ePs*)

Set FTU module prescale.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>ePs</i>	FTU clock prescaler enumeration

LOCAL_INLINE void FTU_HWA_SetModuleUpdateRegBySync (FTU_Type * *pFtu*)

Set MODE[FTUEN], this field define different free running counter and synchronization behavior.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_SetPwmLoadEnable (FTU_Type * *pFtu*)

Set the LDOK flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_SetPwmSyncMode (FTU_Type * *pFtu*)

Set the PWMSYNC flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_SetReinitBySync (FTU_Type * *pFtu*)

Set the REINIT flag.

Parameters

<i>pFtu</i>	the base address of the FTU instance
-------------	--------------------------------------

LOCAL_INLINE void FTU_HWA_SetSWOValue (FTU_Type * *pFtu*, uint32_t *u32Mask*)

Set the software output control value.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u32Mask</i>	channel mask

LOCAL_INLINE void FTU_HWA_SingleMeasurement (FTU_Type * *pFtu*, uint8_t *u8Channel*)

Re-start measurement when in single mode.

Parameters

<i>pFtu</i>	the base address of the FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.

HwA_gpio

Collaboration diagram for HwA_gpio:Functions

LOCAL_INLINE void **GPIO_HWA_SetPortOutput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port output to 1.

LOCAL_INLINE void **GPIO_HWA_ClearPortOutput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port output to 0.

LOCAL_INLINE void **GPIO_HWA_TogglePort** (GPIO_Type *pGpio, uint32_t u32Pins)

Toggle port output.

LOCAL_INLINE void **GPIO_HWA_SetPortDirectionInput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port direction Input.

LOCAL_INLINE void **GPIO_HWA_SetPortDirectionOutput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port direction output.

LOCAL_INLINE uint32_t **GPIO_HWA_ReadPortDataInput** (GPIO_Type *pGpio)

Read port data input, this register indicate data on pad.

LOCAL_INLINE void **GPIO_HWA_SetPinOutput** (GPIO_Type *pGpio, uint8_t u8Pin)

Set pin output to 1.

LOCAL_INLINE void **GPIO_HWA_ClearPinOutput** (GPIO_Type *pGpio, uint8_t u8Pin)

Set pin output to 0.

LOCAL_INLINE void **GPIO_HWA_SetPinDirection** (GPIO_Type *pGpio, uint8_t u8Pin)

Set pin direction.

LOCAL_INLINE void **GPIO_HWA_ClearPinDirection** (GPIO_Type *pGpio, uint8_t u8Pin)

Clear pin direction.

Detailed Description

Function Documentation

LOCAL_INLINE void **GPIO_HWA_ClearPinDirection** (GPIO_Type * *pGpio*, uint8_t *u8Pin*)

Clear pin direction.

Parameters

<i>pGpio</i>	Gpio instance
<i>u8Pin</i>	Pin number

LOCAL_INLINE void GPIO_HWA_ClearPinOutput (GPIO_Type * *pGpio*, uint8_t *u8Pin*)

Set pin output to 0.

Parameters

<i>pGpio</i>	Gpio instance
<i>u8Pin</i>	Pin number

LOCAL_INLINE void GPIO_HWA_ClearPortOutput (GPIO_Type * *pGpio*, uint32_t *u32Pins*)

Set port output to 0.

Parameters

<i>pGpio</i>	Gpio instance
<i>u32Pins</i>	Pin numbers

LOCAL_INLINE uint32_t GPIO_HWA_ReadPortDataInput (GPIO_Type * *pGpio*)

Read port data input, this register indicate data on pad.

Parameters

<i>pGpio</i>	Gpio instance
--------------	---------------

Returns

PDIR register value

LOCAL_INLINE void GPIO_HWA_SetPinDirection (GPIO_Type * *pGpio*, uint8_t *u8Pin*)

Set pin direction.

Parameters

<i>pGpio</i>	Gpio instance
<i>u8Pin</i>	Pin number

LOCAL_INLINE void GPIO_HWA_SetPinOutput (GPIO_Type * *pGpio*, uint8_t *u8Pin*)

Set pin output to 1.

Parameters

<i>pGpio</i>	Gpio instance
<i>u8Pin</i>	Pin number

LOCAL_INLINE void GPIO_HWA_SetPortDirectionInput (GPIO_Type * *pGpio*, uint32_t *u32Pins*)

Set port direction Input.

Parameters

<i>pGpio</i>	Gpio instance
<i>u32Pins</i>	Pin numbers

LOCAL_INLINE void GPIO_HWA_SetPortDirectionOutput (GPIO_Type * *pGpio*, uint32_t *u32Pins*)

Set port direction output.

Parameters

<i>pGpio</i>	Gpio instance
<i>u32Pins</i>	Pin numbers

LOCAL_INLINE void GPIO_HWA_SetPortOutput (GPIO_Type * *pGpio*, uint32_t *u32Pins*)

Set port output to 1.

Parameters

<i>pGpio</i>	Gpio instance
<i>u32Pins</i>	Pin numbers

LOCAL_INLINE void GPIO_HWA_TogglePort (GPIO_Type * *pGpio*, uint32_t *u32Pins*)

Toggle port output.

Parameters

<i>pGpio</i>	Gpio instance
<i>u32Pins</i>	Pin numbers

HwA_hsadc

Collaboration diagram for HwA_hsadc:Enumerations

enum **HSADC_AlignType** { **HSADC_ALIGN_RIGHT** = 0U, **HSADC_ALIGN_LEFT** = 1U }

Select the HSADC result alignment.

enum **HSADC_TriggerModeType** { **HSADC_TRGMODE_SW** = 0U, **HSADC_TRGMODE_RISING_EDGE** = 1U, **HSADC_TRGMODE_FALLING_EDGE** = 2U, **HSADC_TRGMODE_BOTH_EDGE** = 3U, **HSADC_TRGMODE_HIGH_VOLTAGE** = 4U, **HSADC_TRGMODE_LOW_VOLTAGE** = 5U, **HSADC_TRGMODE_INTERNAL_PERIODIC** = 6U }

Select the HSADC trigger mode.

enum **HSADC_SeqModeType** { **HSADC_SEQMODE_SINGLE** = 0U, **HSADC_SEQMODE_CONTINUOUS** = 1U, **HSADC_SEQMODE_DISCONTINUOUS** = 2U, **HSADC_SEQMODE_GROUP** = 4U }

Select the HSADC sequence mode.

enum **HSADC_FastCmpRefMode** { **HSADC_FAST_CMP_REFMODE_SW** = 0U, **HSADC_FAST_CMP_REFMODE_RAMP** = 1U, **HSADC_FAST_CMP_REFMODE_ALT** = 2U }

enum **HSADC_FastCmpRampDir** { **HSADC_FAST_CMP_RAMPDIR_UP** = 0U, **HSADC_FAST_CMP_RAMPDIR_DOWN** = 1U }

enum **HSADC_FastCmpRampTrgMode** { **HSADC_FAST_CMP_RAMPTRGMODE_SW_AUTO** = 0U, **HSADC_FAST_CMP_RAMPTRGMODE_SW_EXT** = 1U, **HSADC_FAST_CMP_RAMPTRGMODE_EXT_AUTO** = 2U }

enum **HSADC_FastCmpRampTrgPolarity** { **HSADC_FAST_CMP_RAMPTRGPOL_RISING_OR_HIGH** = 0U, **HSADC_FAST_CMP_RAMPTRGPOL_FALLING_OR_LOW** = 1U }

enum **HSADC_FCmpBFLAction** { **HSADC_FAST_CMP_BFA_A1B0** = 0U, **HSADC_FAST_CMP_BFA_A0B1** = 1U }

enum **HSADC_OvrModeType** { **HSADC_OVERRUN_MODE_PRESERVE** = 0U, **HSADC_OVERRUN_MODE_OVERWRITE** = 1U }

Select the HSADC overrun management mode.

enum **HSADC_TrgLatchUnitPri** { **TRG_LATCH_UNIT_PRI_ROUND_ROBIN** = 0U, **TRG_LATCH_UNIT_PRI_FIX** = 1U }

Select the priority of Trigger Latch Unit.

enum **HSADC_AverageLenType** { **HSADC_AVERAGE_LEN_4** = 0U, **HSADC_AVERAGE_LEN_8** = 1U, **HSADC_AVERAGE_LEN_16** = 2U, **HSADC_AVERAGE_LEN_32** = 3U }

Select the HSADC hardware average samples.

enum **HSADC_ClockDivideType** { **HSADC_CLOCK_DIV_1** = 0U, **HSADC_CLOCK_DIV_2** = 1U, **HSADC_CLOCK_DIV_4** = 2U, **HSADC_CLOCK_DIV_8** = 3U }

Set the HSADC clock divider.

enum **HSADC_TrgSrcType** { **HSADC_TRGSRC_TRGSEL** = 2U, **HSADC_TRGSRC_TRIG_LATCH_UNIT** = 3U }

The trigger source of the HSADC instance.

enum **HSADC_CmpModeType** { **HSADC_CMP_ALL_CHANNEL** = 0U, **HSADC_CMP_SINGLE_CHANNEL** = 1U }

Select the channel compare mode.

enum **HSADC_FunctionClockType** { **FUNCTION_CLOCK_FROM_PCC** = 0U, **FUNCTION_CLOCK_FROM_PAD** = 1U }

enum **HSADC_SeqGroupIndex** { **HSADC_SEQ_GROUP_0** = 0U, **HSADC_SEQ_GROUP_1** = 1U }

The HSADC Sequence Group Index.

Functions

LOCAL_INLINE bool **HSADC_HWA_GetFIFOReady** (const HSADC_Type *const pHsadc)

Check whether the data quantity in the FIFO is greater than watermark.

LOCAL_INLINE void **HSADC_HWA_ClearFIFOReady** (HSADC_Type *const pHsadc)

Clear the FIFO ready flag.

LOCAL_INLINE bool **HSADC_HWA_GetCmpFlag** (const HSADC_Type *const pHsadc)

Check whether the conversion result is in the comparing range.

LOCAL_INLINE void **HSADC_HWA_ClearCmpFlag** (HSADC_Type *const pHsadc)

Clear the compare flag.

LOCAL_INLINE bool **HSADC_HWA_GetEmpty** (const HSADC_Type *const pHsadc)

Check whether the HSADC FIFO is empty.

LOCAL_INLINE bool **HSADC_HWA_GetFull** (const HSADC_Type *const pHsadc)

Check whether the HSADC FIFO is full.

LOCAL_INLINE bool **HSADC_HWA_GetOverRun** (const HSADC_Type *const pHsadc)

Get the overrun status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_ClearOverRun** (HSADC_Type *const pHsadc)

Clear the overrun flag of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSequence** (const HSADC_Type *const pHsadc)

Check whether the HSADC conversion sequence is finished.

LOCAL_INLINE void **HSADC_HWA_ClearEndOfSequence** (HSADC_Type *const pHsadc)

Clear the HSADC conversion sequence complete flag.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfConversion** (const HSADC_Type *const pHsadc)

Check whether the current HSADC conversion is finished.

LOCAL_INLINE void **HSADC_HWA_ClearEndOfConversion** (HSADC_Type *const pHsadc)

Clear the HSADC single conversion complete flag.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSample** (const HSADC_Type *const pHsadc)

Check whether the sampling phase of the current HSADC conversion is finished.

LOCAL_INLINE void **HSADC_HWA_ClearEndOfSample** (HSADC_Type *const pHsadc)

Clear the HSADC sampling complete flag.

LOCAL_INLINE bool **HSADC_HWA_GetReady** (const HSADC_Type *const pHsadc)

Check whether the HSADC instance is ready to operate.

LOCAL_INLINE void **HSADC_HWA_ClearReady** (HSADC_Type *const pHsadc)

Clear the ready flag of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetFIFOReadyInterruptFlag** (const HSADC_Type *const pHsadc)

Get the FIFO Ready interrupt flag If enabled, HSADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE void **HSADC_HWA_SetFIFOReadyInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the FIFO Ready interrupt flag If enabled, HSADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE bool **HSADC_HWA_GetCmplInterruptFlag** (const HSADC_Type *const pHsadc)

Get the Compare interrupt flag If enabled, HSADC interrupt is generated when the HSADC conversion result is not within the compare threshold.

LOCAL_INLINE void **HSADC_HWA_SetCmplInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the Compare interrupt flag If enabled, HSADC interrupt is generated when the HSADC conversion result is not within the compare threshold.

LOCAL_INLINE bool **HSADC_HWA_GetOverrunInterruptFlag** (const HSADC_Type *const pHsadc)

Get the Overrun interrupt flag If enabled, HSADC interrupt is generated when the HSADC instance is overrun.

LOCAL_INLINE void **HSADC_HWA_SetOverrunInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the Overrun interrupt flag If enabled, HSADC interrupt is generated when the HSADC instance is overrun.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSequenceInterruptFlag** (const HSADC_Type *const pHsadc)

Get the End of Sequence interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC sequence conversion is completed.

LOCAL_INLINE void **HSADC_HWA_SetEndOfSequenceInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the End of Sequence interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC sequence conversion is completed.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfConversionInterruptFlag** (const HSADC_Type *const pHsadc)
Get the conversion complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion is completed.

LOCAL_INLINE void **HSADC_HWA_SetEndOfConversionInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)
Set the conversion complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion is completed.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSampleInterruptFlag** (const HSADC_Type *const pHsadc)
Get the sample complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion finished the sampling phase.

LOCAL_INLINE void **HSADC_HWA_SetEndOfSampleInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)
Set the sample complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion finished the sampling phase.

LOCAL_INLINE bool **HSADC_HWA_GetHSADCReadyInterruptFlag** (const HSADC_Type *const pHsadc)
Get the HSADC ready interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC module is ready for conversion.

LOCAL_INLINE void **HSADC_HWA_SetHSADCReadyInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)
Set the HSADC ready interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC module is ready for conversion.

LOCAL_INLINE uint32_t **HSADC_HWA_GetInterruptEnable** (const HSADC_Type *const pHsadc)
Get the interrupt enable config.

LOCAL_INLINE void **HSADC_HWA_SetInterruptEnable** (HSADC_Type *const pHsadc, uint32_t u32InterruptCfg)
Set the interrupt enable.

LOCAL_INLINE void **HSADC_HWA_SetFClkSel** (HSADC_Type *const pHsadc, const **HSADC_FunctionClockType** Type)

Select HSADC function clock.

LOCAL_INLINE void **HSADC_HWA_Reset** (HSADC_Type *const pHsadc)

Assert the HSADC hardware Reset bit.

LOCAL_INLINE bool **HSADC_HWA_GetStop** (const HSADC_Type *const pHsadc)

Get whether HSADC is in stopping status.

LOCAL_INLINE void **HSADC_HWA_Stop** (HSADC_Type *const pHsadc)

Stop the HSADC conversion.

LOCAL_INLINE bool **HSADC_HWA_GetStart** (const HSADC_Type *const pHsadc)

Get the conversion start status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_Start** (HSADC_Type *const pHsadc)

Start the HSADC conversion.

LOCAL_INLINE bool **HSADC_HWA_GetDisable** (const HSADC_Type *const pHsadc)

Get whether the HSADC instance is in disable status.

LOCAL_INLINE void **HSADC_HWA_Disable** (HSADC_Type *const pHsadc)

Disable the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetEnable** (const HSADC_Type *const pHsadc)

Get the enable status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_Enable** (HSADC_Type *const pHsadc)

Enable the HSADC instance.

LOCAL_INLINE **HSADC_OvrModeType** **HSADC_HWA_GetOverrunMode** (const HSADC_Type *const pHsadc)

Get the overrun mode.

LOCAL_INLINE void **HSADC_HWA_SetOverrunMode** (HSADC_Type *const pHsadc, **HSADC_OvrModeType** eOvrMode)

Set the overrun mode.

LOCAL_INLINE bool **HSADC_HWA_GetSeqGpEn** (const HSADC_Type *const pHsadc)

Get the sequence group mode state.

LOCAL_INLINE void **HSADC_HWA_SetSeqGpEn** (HSADC_Type *const pHsadc, bool bEnable)

Set the sequence group mode state.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSeqLength** (const HSADC_Type *const pHsadc)

Get the sequence length of the HSADC conversion sequence.

LOCAL_INLINE void **HSADC_HWA_SetSeqLength** (HSADC_Type *const pHsadc, uint8_t u8SequenceLength)

Set the sequence length of the HSADC conversion sequence.

LOCAL_INLINE **HSADC_SeqModeType** **HSADC_HWA_GetSeqMode** (const HSADC_Type *const pHsadc)

Get the HSADC sequence mode.

LOCAL_INLINE void **HSADC_HWA_SetSeqMode** (HSADC_Type *const pHsadc, **HSADC_SeqModeType** eSequenceMode)

Set the HSADC sequence mode.

LOCAL_INLINE bool **HSADC_HWA_GetAutoDisableMode** (const HSADC_Type *const pHsadc)

Get whether auto disable is enabled.

LOCAL_INLINE void **HSADC_HWA_SetAutoDisableMode** (HSADC_Type *const pHsadc, bool bEnable)

Set whether to enable auto disable mode.

LOCAL_INLINE bool **HSADC_HWA_GetWaitMode** (const HSADC_Type *const pHsadc)

Get whether the wait conversion mode is enabled.

LOCAL_INLINE void **HSADC_HWA_SetWaitMode** (HSADC_Type *const pHsadc, bool bEnable)

Set the wait conversion mode.

LOCAL_INLINE **HSADC_TrgSrcType** **HSADC_HWA_GetTriggerSource** (const HSADC_Type *const pHsadc)

Get the trigger source the the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetTriggerSource** (HSADC_Type *const pHsadc, **HSADC_TrgSrcType** eTrgSource)

Set the trigger source the the HSADC instance.

LOCAL_INLINE **HSADC_TrgModeType** **HSADC_HWA_GetTriggerMode** (const HSADC_Type *const pHsadc)

Get the trigger mode of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetTriggerMode** (HSADC_Type *const pHsadc, **HSADC_TrgModeType** eTrgMode)

Set the trigger mode of the HSADC instance.

LOCAL_INLINE **HSADC_AlignType** **HSADC_HWA_GetDataAlignment** (const HSADC_Type *const pHsadc)

Get the data align mode.

LOCAL_INLINE void **HSADC_HWA_SetDataAlignment** (HSADC_Type *const pHsadc, **HSADC_AlignType** eAlign)

Set the data align mode.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSGDMASEL** (const HSADC_Type *const pHsadc)

Get SGDMA SEL for the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetSGDMASEL** (HSADC_Type *const pHsadc, uint8_t index)

Set the SGDMA SEL for the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetSGDMAEnableFlag** (const HSADC_Type *const pHsadc)

Get whether SGDMA for the HSADC instance is enabled.

LOCAL_INLINE void **HSADC_HWA_SetSGDMAEnableFlag** (HSADC_Type *const pHsadc, bool bEnable)

Enable or disable the SGDMA for the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetDMAEnableFlag** (const HSADC_Type *const pHsadc)

Get whether DMA for the HSADC instance is enabled.

LOCAL_INLINE void **HSADC_HWA_SetDMAEnableFlag** (HSADC_Type *const pHsadc, bool bEnable)

Enable or disable the DMA for the HSADC instance.

LOCAL_INLINE uint32_t **HSADC_HWA_GetConfig1** (const HSADC_Type *const pHsadc)

Get the HSADC_CFG1 config.

LOCAL_INLINE void **HSADC_HWA_SetConfig1** (HSADC_Type *const pHsadc, uint32_t u32Config)

Set the HSADC_CFG1 config.

LOCAL_INLINE bool **HSADC_HWA_GetExtChEnable** (const HSADC_Type *const pHsadc)

Get the extend channel enable settings for the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetExtChEnable** (HSADC_Type *const pHsadc, bool bEnable)

Set the extend channel feature for the HSADC instance.

LOCAL_INLINE uint8_t **HSADC_HWA_GetFIFOWaterMark** (const HSADC_Type *const pHsadc)

Get the FIFO water mark settings for the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFIFOWaterMark** (HSADC_Type *const pHsadc, uint8_t u8WaterMark)

Set the FIFO water mark for the HSADC instance.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSmpSpare** (const HSADC_Type *const pHsadc)

LOCAL_INLINE void **HSADC_HWA_SetSmpSpare** (HSADC_Type *const pHsadc, bool bEnable)

LOCAL_INLINE uint8_t **HSADC_HWA_GetTriggerLatchUnitPriority** (const HSADC_Type *const pHsadc)

Get the priority of Trigger Latch.

LOCAL_INLINE void **HSADC_HWA_SetTriggerLatchUnitPriority** (HSADC_Type *const pHsadc,

HSADC_TrgLatchUnitPri eTrgLatchUnitPri)

Set the priority of Trigger Latch.

LOCAL_INLINE void **HSADC_HWA_ClearLatchTrigger** (HSADC_Type *const pHsadc)

Clear Latch Trigger in Trigger Latch Unit.

LOCAL_INLINE bool **HSADC_HWA_GetAverageEnableFlag** (const HSADC_Type *const pHsadc)

Get whether hardware average is enabled.

LOCAL_INLINE void **HSADC_HWA_SetAverageEnableFlag** (HSADC_Type *const pHsadc, bool bEnable)

Enable or disable hardware average for the HSADC instance.

LOCAL_INLINE **HSADC_AverageLenType** **HSADC_HWA_GetAverageNumber** (const HSADC_Type *const pHsadc)

Get the hardware average number.

LOCAL_INLINE void **HSADC_HWA_SetAverageNumber** (HSADC_Type *const pHsadc, **HSADC_AverageLenType** eAverageNumber)

Set the hardware average number.

LOCAL_INLINE bool **HSADC_HWA_GetClockGatingAck** (const HSADC_Type *const pHsadc)

Whether clock gating is acknowledged.

LOCAL_INLINE bool **HSADC_HWA_GetClockGatingEnableFlag** (const HSADC_Type *const pHsadc)

Get whether clock gating is enabled.

LOCAL_INLINE void **HSADC_HWA_SetClockGatingEnableFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set whether to enable HSADC clock gating.

LOCAL_INLINE **HSADC_ClockDivideType** **HSADC_HWA_GetClockDivider** (const HSADC_Type *const pHsadc)

Get the HSADC clock divider.

LOCAL_INLINE void **HSADC_HWA_SetClockDivider** (HSADC_Type *const pHsadc, **HSADC_ClockDivideType** eDivider)

Set the HSADC clock divider.

LOCAL_INLINE uint8_t **HSADC_HWA_GetStartupCnt** (const HSADC_Type *const pHsadc)

Get the HSADC start up count.

LOCAL_INLINE void **HSADC_HWA_SetStartupCnt** (HSADC_Type *const pHsadc, uint8_t u8StartupCnt)

Set the HSADC start up count.

LOCAL_INLINE uint32_t **HSADC_HWA_GetConfig2** (const HSADC_Type *const pHsadc)

Get the HSADC_CFG2 config.

LOCAL_INLINE void **HSADC_HWA_SetConfig2** (HSADC_Type *const pHsadc, uint32_t u32Config)

Set the HSADC_CFG2 config.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSampleTime** (HSADC_Type *const pHsadc, uint8_t u8Selection)

Get the sample time of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetSampleTime** (HSADC_Type *const pHsadc, uint8_t u8Selection,
uint8_t u8SampleTime)

Set the sample time of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetCmpEnable** (const HSADC_Type *const pHsadc)

Get whether hardware compare is enabled.

LOCAL_INLINE void **HSADC_HWA_SetCmpEnable** (HSADC_Type *const pHsadc, bool bEnable)

Set to enable or disable the hardware compare.

LOCAL_INLINE void **HSADC_HWA_SetCmpChannel** (HSADC_Type *const pHsadc, **HSADC_CmpModeType**
eMode, uint8_t u8ChannalNum)

Set the HSADC hardware compare channel.

LOCAL_INLINE void **HSADC_HWA_SetCmpThreshold** (HSADC_Type *const pHsadc, uint16_t u16LowThres,
uint16_t u16HighThres)

Set the HSADC hardware compare threshold.

LOCAL_INLINE uint32_t **HSADC_HWA_GetConfig3** (const HSADC_Type *const pHsadc)

Get the HSADC_CFG3 config.

```
LOCAL_INLINE void HSADC_HWA_SetConfig3 (HSADC_Type *const pHsadc, uint32_t u32Config)
```

Set the HSADC_CFG3 config.

```
LOCAL_INLINE bool HSADC_HWA_GetEndOfSeqGroupInterruptFlag (HSADC_Type *const pHsadc, const  
uint8_t u8SeqGroupIndex)
```

Get the end of sequence group flag.

```
LOCAL_INLINE void HSADC_HWA_ClearEndOfSeqGroupInterruptFlag (HSADC_Type *const pHsadc, const  
uint8_t u8SeqGroupIndex)
```

Clear the end of sequence group flag.

```
LOCAL_INLINE void HSADC_HWA_SetEndOfSeqGroupInterruptEnable (HSADC_Type *const pHsadc, const  
uint8_t u8SeqGroupIndex, const bool bEnable)
```

Set the sequence group end of sequence interrupt enable.

```
LOCAL_INLINE uint8_t HSADC_HWA_GetSeqGroupStartPoint (HSADC_Type *const pHsadc, const uint8_t  
u8SeqGroupIndex)
```

Get the sequence group start point.

```
LOCAL_INLINE uint8_t HSADC_HWA_GetSeqGroupEndPoint (HSADC_Type *const pHsadc, const uint8_t  
u8SeqGroupIndex)
```

Get the sequence group end point.

```
LOCAL_INLINE void HSADC_HWA_SetSeqGroupStartEndPoint (HSADC_Type *const pHsadc, const uint8_t  
u8SeqGroupIndex, const uint8_t u8Start, const uint8_t u8End)
```

Set the sequence group start and end point.

```
LOCAL_INLINE void HSADC_HWA_SetOGCOffset (HSADC_Type *const pHsadc, uint16_t val)
```

```
LOCAL_INLINE void HSADC_HWA_SetOGCGain (HSADC_Type *const pHsadc, uint16_t val)
```

```
LOCAL_INLINE void HSADC_HWA_SetOGCEnable (HSADC_Type *const pHsadc, bool bEnable)
```

```
LOCAL_INLINE void HSADC_HWA_SetCCal (HSADC_Type *const pHsadc, uint32_t u32CCal)
```

Set the HSADC_CCAL config.

LOCAL_INLINE uint32_t **HSADC_HWA_GetFIFOData** (const HSADC_Type *const pHsadc)

Get the conversion result FIFO data of the HSADC instance.

LOCAL_INLINE uint8_t **HSADC_HWA_GetChannelSampleTimeIndex** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Get the sample time index of the HSADC channel.

LOCAL_INLINE void **HSADC_HWA_SetChannelSampleTimeIndex** (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, uint8_t u8SmpSel)

Set the sample time index of the HSADC channel.

LOCAL_INLINE bool **HSADC_HWA_GetChannelConvComplete** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Get the channel conversion complete status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_ClearChannelConvComplete** (HSADC_Type *const pHsadc, uint8_t u8ChnIdx)

Clear the channel conversion complete status of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetChannelInterruptEnable** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Check whether interrupt is enabled on the selected HSADC channel.

LOCAL_INLINE void **HSADC_HWA_SetChannelInterruptEnable** (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, bool bEnable)

Set to enable or disable interrupt on the selected HSADC channel.

LOCAL_INLINE uint8_t **HSADC_HWA_GetChannelInput** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Get the input channel of the selected HSADC channel.

LOCAL_INLINE void **HSADC_HWA_SetChannelInput** (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, uint8_t u8InputChann)

Set the input channel to the selected HSADC channel.

```
LOCAL_INLINE void HSADC_HWA_SetSCCfg (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, uint32_t val)
```

```
LOCAL_INLINE uint32_t HSADC_HWA_GetChannelData (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)
```

Get the conversion result data of the HSADC instance.

```
LOCAL_INLINE uint16_t HSADC_HWA_GetResultCal (HSADC_Type *const pHsadc)
```

```
LOCAL_INLINE bool HSADC_HWA_GetFCEn (const HSADC_Type *const pHsadc)
```

Get the fast compare enable.

```
LOCAL_INLINE void HSADC_HWA_SetFCEn (HSADC_Type *const pHsadc, bool enable)
```

Set the fast compare reference of the HSADC instance.

```
LOCAL_INLINE void HSADC_HWA_SetFCtrl (HSADC_Type *const pHsadc, uint32_t value)
```

Set the fast compare control register.

```
LOCAL_INLINE int16_t HSADC_HWA_GetFCRef (const HSADC_Type *const pHsadc)
```

Get the fast compare reference of the HSADC instance.

```
LOCAL_INLINE void HSADC_HWA_SetFCRef (HSADC_Type *const pHsadc, uint16_t ref)
```

Set the fast compare reference of the HSADC instance.

```
LOCAL_INLINE int16_t HSADC_HWA_GetFCRampRefA (const HSADC_Type *const pHsadc)
```

Get the fast compare ramp reference A of the HSADC instance.

```
LOCAL_INLINE void HSADC_HWA_SetFCRampRefA (HSADC_Type *const pHsadc, uint16_t ref)
```

Set the fast compare ramp reference A of the HSADC instance.

```
LOCAL_INLINE uint8_t HSADC_HWA_GetFCRampStep (const HSADC_Type *const pHsadc)
```

Get the fast compare ramp step of the HSADC instance.

```
LOCAL_INLINE void HSADC_HWA_SetFCRampStep (HSADC_Type *const pHsadc, uint16_t step)
```

Set the fast compare ramp step.

LOCAL_INLINE void **HSADC_HWA_SetFCRamp0** (HSADC_Type *const pHsadc, uint32_t value)

Set the fast compare ramp0 register.

LOCAL_INLINE int16_t **HSADC_HWA_GetFCRampRefB** (const HSADC_Type *const pHsadc)

Get the fast compare ramp reference B of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFCRampRefB** (HSADC_Type *const pHsadc, uint16_t ref)

Set the fast compare ramp reference B of the HSADC instance.

LOCAL_INLINE **HSADC_FastCmpRampDir** **HSADC_HWA_GetFCRampDir** (const HSADC_Type *const pHsadc)

Get the fast compare ramp dir.

LOCAL_INLINE void **HSADC_HWA_SetFCRampDir** (HSADC_Type *const pHsadc, **HSADC_FastCmpRampDir** dir)

Set the fast compare ramp dir.

LOCAL_INLINE void **HSADC_HWA_SetFCRamp1** (HSADC_Type *const pHsadc, uint32_t value)

Set the fast compare ramp1 register.

LOCAL_INLINE uint16_t **HSADC_HWA_GetFCRefDeltaP** (const HSADC_Type *const pHsadc)

Get the fast compare reference upper delta.

LOCAL_INLINE void **HSADC_HWA_SetFCRefDeltaP** (HSADC_Type *const pHsadc, uint16_t delta)

Set the fast compare reference upper delta.

LOCAL_INLINE uint16_t **HSADC_HWA_GetFCRefDeltaN** (const HSADC_Type *const pHsadc)

Get the fast compare reference lower delta.

LOCAL_INLINE void **HSADC_HWA_SetFCRefDeltaN** (HSADC_Type *const pHsadc, uint16_t delta)

Set the fast compare reference lower delta.

LOCAL_INLINE void **HSADC_HWA_SetFCHysteresis** (HSADC_Type *const pHsadc, uint32_t u32Hyst)
Set the fast compare reference hysteresis.

LOCAL_INLINE bool **HSADC_HWA_GetFCResult** (HSADC_Type *const pHsadc)
Get the fast compare result.

LOCAL_INLINE bool **HSADC_HWA_GetFCBoundaryFlag** (HSADC_Type *const pHsadc)
Get the fast compare boundary flag.

Detailed Description

Enumeration Type Documentation

enum HSADC_AlignType

Select the HSADC result alignment.

Enumerator:

HSADC_ALIGN_ RIGHT	HSADC result is aligned in right
HSADC_ALIGN_ LEFT	HSADC result is aligned in left

enum HSADC_AverageLenType

Select the HSADC hardware average samples.

Enumerator:

HSADC_AVERAG E_LEN_4	result average by 4 samples
HSADC_AVERAG E_LEN_8	result average by 8 samples
HSADC_AVERAG E_LEN_16	result average by 16 samples
HSADC_AVERAG E_LEN_32	result average by 32 samples

enum HSADC_ClockDivideType

Set the HSADC clock divider.

Note

HSADC clock divider is not available in FC7300F512K

Enumerator:

HSADC_CLOCK_ DIV_1	
HSADC_CLOCK_ DIV_2	
HSADC_CLOCK_ DIV_4	
HSADC_CLOCK_ DIV_8	

enum HSADC_CmpModeType

Select the channel compare mode.

Select whether the channel compare is enabled on all channels or on the single selected channel

Enumerator:

HSADC_CMP_AL L_CHANNEL	Compare enabled in all channels
HSADC_CMP_SI NGLE_CHANNEL	Compare enabled in the specified channel

enum HSADC_FastCmpRampDir**Enumerator:**

HSADC_FAST_C MP_RAMPDIR_U P	
HSADC_FAST_C MP_RAMPDIR_D OWN	

enum HSADC_FastCmpRampTrgMode**Enumerator:**

HSADC_FAST_C MP_RAMPTRGM ODE_SW_AUTO	
HSADC_FAST_C MP_RAMPTRGM ODE_SW_EXT	
HSADC_FAST_C MP_RAMPTRGM ODE_EXT_AUTO	

enum HSADC_FastCmpRampTrgPolarity**Enumerator:**

HSADC_FAST_C MP_RAMPTRGP OL_RISING_OR_ HIGH	
--	--

HSADC_FAST_C	
MP_RAMPTRGP	
OL_FALLING_O	
R_LOW	

enum HSADC_FastCmpRefMode

Enumerator:

HSADC_FAST_C	
MP_REFMODE_	
SW	
HSADC_FAST_C	
MP_REFMODE_	
RAMP	
HSADC_FAST_C	
MP_REFMODE_	
ALT	

enum HSADC_FCmpBFLAction

Enumerator:

HSADC_FAST_C	
MP_BFA_A1B0	
HSADC_FAST_C	
MP_BFA_A0B1	

enum HSADC_FunctionClockType

Enumerator:

FUNCTION_CLO	
CK_FROM_PCC	
FUNCTION_CLO	
CK_FROM_PAD	

enum HSADC_OvrModeType

Select the HSADC overrun management mode.

To select whether the old data are preserved or overwritten by the new data when HSADC is overrun
(The FIFO is full when new conversion result comes)

Enumerator:

HSADC_OVERRUN_MODE_PRESERVE	old data are preserved when HSADC is overrun
HSADC_OVERRUN_MODE_OVERWRITE	old data are overwritten when HSADC is overrun

enum HSADC_SeqGroupIndex

The HSADC Sequence Group Index.

Enumerator:

HSADC_SEQ_GROUP_0	
HSADC_SEQ_GROUP_1	

enum HSADC_SeqModeType

Select the HSADC sequence mode.

Enumerator:

HSADC_SEQ_MODE_SINGLE	HSADC single sequence mode
HSADC_SEQ_MODE_CONTINUOUS	HSADC continuous mode

HSADC_SEQMODE_DISCONTINUOUS	HSADC discontinuous mode 3 is reserved
HSADC_SEQMODE_GROUP	HSADC sequence group mode

enum HSADC_TrgLatchUnitPri

Select the priority of Trigger Latch Unit.

Enumerator:

TRG_LATCH_UNIT_PRIORITY_ROUND_ROBIN	select the round robin scheduling priority
TRG_LATCH_UNIT_PRIORITY_FIXED	select the fixed priority(0>1>2>3)

enum HSADC_TrgModeType

Select the HSADC trigger mode.

Note

This option is only valid in HSADC single sequence mode and HSADC continuous mode. In HSADC discontinuous mode, the HSADC trigger mode is fixed as HSADC_TRIGMODE_RISING_EDGE

Enumerator:

HSADC_TRIGMODE_SW	HSADC is triggered by software
HSADC_TRIGMODE_RISING_EDGE	HSADC is triggered by hardware trigger on rising edge

HSADC_TRGMO DE_FALLING_EDGE	HSADC is triggered by hardware trigger on falling edge
HSADC_TRGMO DE_BOTH_EDGE	HSADC is triggered by hardware trigger on both edges
HSADC_TRGMO DE_HIGH_VOLTAGE	HSADC is triggered when hardware trigger is high voltage
HSADC_TRGMO DE_LOW_VOLTAGE	HSADC is triggered when hardware trigger is low voltage
HSADC_TRGMO DE_INTERNAL_PERIODIC	HSADC is triggered by internal generated periodic trigger

enum HSADC_TrgSrcType

The trigger source of the HSADC instance.

Note

In HSADC discontinuous 1 mode, the trigger source is from Ptimer In HSADC single and continuous mode, if hardware trigger is enabled, the trigger source is from TRGSEL

Enumerator:

HSADC_TRGSR C_TRGSEL	Trigger source from TRGSEL
HSADC_TRGSR C_TRIG_LATCH_UNIT	Trigger source from LATCH UNIT

Function Documentation

LOCAL_INLINE void HSADC_HWA_ClearChannelConvComplete (HSADC_Type *const *pHadc*, uint8_t *u8ChnlIdx*)

Clear the channel conversion complete status of the HSADC instance.

Note

this function is used only in HSADC sequence group mode to clear the channel complete flag

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

true the HSADC conversion on the selected channel is completed

false the HSADC conversion on the selected channel is not completed

LOCAL_INLINE void HSADC_HWA_ClearCmpFlag (HSADC_Type *const *pHsadc*)

Clear the compare flag.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_ClearEndOfConversion (HSADC_Type *const *pHsadc*)

Clear the HSADC single conversion complete flag.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_ClearEndOfSample (HSADC_Type *const *pHsadc*)

Clear the HSADC sampling complete flag.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_ClearEndOfSeqGroupInterruptFlag (HSADC_Type *const *pHsadc*, const uint8_t *u8SeqGroupIndex*)

Clear the end of sequence group flag.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8SeqGroupIndex</i>	the index of the sequence group

LOCAL_INLINE void HSADC_HWA_ClearEndOfSequence (HSADC_Type *const *pHsadc*)

Clear the HSADC conversion sequence complete flag.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_ClearFIFOReady (HSADC_Type *const *pHsadc*)

Clear the FIFO ready flag.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_ClearLatchTrigger (HSADC_Type *const *pHsadc*)

Clear Latch Trigger in Trigger Latch Unit.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_ClearOverRun (HSADC_Type *const *pHsadc*)

Clear the overrun flag of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_ClearReady (HSADC_Type *const *pHsadc*)

Clear the ready flag of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_Disable (HSADC_Type *const *pHsadc*)

Disable the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_Enable (HSADC_Type *const *pHsadc*)

Enable the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE bool HSADC_HWA_GetAutoDisableMode (const HSADC_Type *const *pHsadc*)

Get whether auto disable is enabled.

Note

Auto disable mode is only available in FC7300F2M

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true auto disable mode is enabled

false auto disable mode is disabled

LOCAL_INLINE bool HSADC_HWA_GetAverageEnableFlag (const HSADC_Type *const *pHsadc*)

Get whether hardware average is enabled.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true hardware average is enabled

false hardware average is disabled

LOCAL_INLINE HSADC_AverageLenType HSADC_HWA_GetAverageNumber (const HSADC_Type *const *pHsadc*)

Get the hardware average number.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_AverageType the hardware average number

LOCAL_INLINE bool HSADC_HWA_GetChannelConvComplete (const HSADC_Type *const *pHsadc*, const uint8_t *u8ChnIdx*)

Get the channel conversion complete status of the HSADC instance.

Note

this function is used only in HSADC discontinuous mode to get the channel complete status

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

<i>u8ChnIdx</i>	the index of the channel
-----------------	--------------------------

Returns

true the HSADC conversion on the selected channel is completed

false the HSADC conversion on the selected channel is not completed

LOCAL_INLINE uint32_t HSADC_HWA_GetChannelData (const HSADC_Type *const *pHsadc*, const uint8_t *u8ChnIdx*)

Get the conversion result data of the HSADC instance.

Note

only result data of HSADC discontinuous mode will be stored in RESULTn register.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8ChnIdx</i>	the index of the HSADC channel

Returns

uint32_t the HSADC conversion result

LOCAL_INLINE uint8_t HSADC_HWA_GetChannelInput (const HSADC_Type *const *pHsadc*, const uint8_t *u8ChnIdx*)

Get the input channel of the selected HSADC channel.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

uint8_t the hardware input channel

LOCAL_INLINE bool HSADC_HWA_GetChannelInterruptEnable (const HSADC_Type *const *pHsadc*, const uint8_t *u8ChnIdx*)

Check whether interrupt is enabled on the selected HSADC channel.

Note

this function is used only in HSADC sequence group mode to get the channel interrupt settings

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

true interrupt is enabled on the selected channel

false interrupt is disabled on the selected channel

```
LOCAL_INLINE uint8_t HSADC_HWA_GetChannelSampleTimeIndex (const HSADC_Type *const pHsadc,  
const uint8_t u8ChnIdx)
```

Get the sample time index of the HSADC channel.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8ChnIdx</i>	the index of the channel

Returns

uint8_t the sample time index of the HSADC channel

```
LOCAL_INLINE HSADC_ClockDivideType HSADC_HWA_GetClockDivider (const HSADC_Type *const  
pHsadc)
```

Get the HSADC clock divider.

Note

This feature is only available in FC7300F2M

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_ClockDivideType the HSADC clock divider

LOCAL_INLINE bool HSADC_HWA_GetClockGatingAck (const HSADC_Type *const *pHsadc*)

Whether clock gating is acknowledged.

Note

This feature is only available in FC7300F2M

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true HSADC clock source is off after setting Clock Gating Enable Flag

false HSADC clock source is on after clearing Clock Gating Enable Flag

LOCAL_INLINE bool HSADC_HWA_GetClockGatingEnableFlag (const HSADC_Type *const *pHsadc*)

Get whether clock gating is enabled.

Note

This feature is only available in FC7300F2M

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true Clock gating is enabled, HSADC clock is off

false Clock gating is disabled, HSADC clock is on

LOCAL_INLINE bool HSADC_HWA_GetCmpEnable (const HSADC_Type *const *pHsadc*)

Get whether hardware compare is enabled.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true hardware compare is enabled

false hardware compare is disabled

LOCAL_INLINE bool HSADC_HWA_GetCmpFlag (const HSADC_Type *const *pHsadc*)

Check whether the conversion result is in the comparing range.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the conversion result is in the comparing range

false the conversion result is not in the comparing range

LOCAL_INLINE bool HSADC_HWA_GetCmpInterruptFlag (const HSADC_Type *const *pHsadc*)

Get the Compare interrupt flag If enabled, HSADC interrupt is generated when the HSADC conversion result is not within the compare threshold.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true HSADC Compare interrupt is enabled

false HSADC Compare interrupt is disabled

LOCAL_INLINE uint32_t HSADC_HWA_GetConfig1 (const HSADC_Type *const *pHsadc*)

Get the HSADC_CFG1 config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint32_t the HSADC_CFG1 config

LOCAL_INLINE uint32_t HSADC_HWA_GetConfig2 (const HSADC_Type *const *pHsadc*)

Get the HSADC_CFG2 config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint32_t the HSADC_CFG2 config

LOCAL_INLINE uint32_t HSADC_HWA_GetConfig3 (const HSADC_Type *const *pHsadc*)

Get the HSADC_CFG3 config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint32_t the HSADC_CFG3 config

LOCAL_INLINE HSADC_AlignType HSADC_HWA_GetDataAlignment (const HSADC_Type *const *pHsadc*)

Get the data align mode.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_ALIGN_RIGHT the conversion data is aligned right

HSADC_ALIGN_LEFT the conversion is aligned left

LOCAL_INLINE bool HSADC_HWA_GetDisable (const HSADC_Type *const *pHsadc*)

Get whether the HSADC instance is in disable status.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC instance is in disable status

false the HSADC instance is not in disable status

LOCAL_INLINE bool HSADC_HWA_GetDMAEnableFlag (const HSADC_Type *const *pHsadc*)

Get whether DMA for the HSADC instance is enabled.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true DMA is enabled for the HSADC instance

false DMA is disabled for the HSADC instance

LOCAL_INLINE bool HSADC_HWA_GetEmpty (const HSADC_Type *const *pHsadc*)

Check whether the HSADC FIFO is empty.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC FIFO is empty

false the HSADC FIFO is not empty

LOCAL_INLINE bool HSADC_HWA_GetEnable (const HSADC_Type *const *pHsadc*)

Get the enable status of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC instance is enabled
false the HSADC instance has not been enabled

LOCAL_INLINE bool HSADC_HWA_GetEndOfConversion (const HSADC_Type *const *pHsadc*)

Check whether the current HSADC conversion is finished.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the current HSADC conversion is finished
false the current HSADC conversion is unfinished

LOCAL_INLINE bool HSADC_HWA_GetEndOfConversionInterruptFlag (const HSADC_Type *const *pHsadc*)

Get the conversion complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion is completed.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC End of Conversion interrupt is enabled
false the HSADC End of Conversion interrupt is disabled

LOCAL_INLINE bool HSADC_HWA_GetEndOfSample (const HSADC_Type *const *pHsadc*)

Check whether the sampling phase of the current HSADC conversion is finished.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the sampling phase of the HSADC conversion is finished

false the sampling phase of the HSADC conversion is unfinished

LOCAL_INLINE bool HSADC_HWA_GetEndOfSampleInterruptFlag (const HSADC_Type *const *pHsadc*)

Get the sample complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion finished the sampling phase.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the sample complete interrupt is enabled

false the sample complete interrupt is disabled

LOCAL_INLINE bool HSADC_HWA_GetEndOfSeqGroupInterruptFlag (HSADC_Type *const *pHsadc*, const uint8_t *u8SeqGroupIndex*)

Get the end of sequence group flag.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8SeqGroupIndex</i>	the index of the sequence group
<i>bool</i>	the sequence group interrupt flag

LOCAL_INLINE bool HSADC_HWA_GetEndOfSequence (const HSADC_Type *const *pHsadc*)

Check whether the HSADC conversion sequence is finished.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC conversion sequence is finished

false the HSADC conversion sequence is unfinished

LOCAL_INLINE bool HSADC_HWA_GetEndOfSequenceInterruptFlag (const HSADC_Type *const *pHsadc*)

Get the End of Sequence interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC sequence conversion is completed.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true HSADC End of Sequence interrupt is enabled

false HSADC End of Sequence interrupt is disabled

LOCAL_INLINE bool HSADC_HWA_GetExtChEnable (const HSADC_Type *const *pHsadc*)

Get the extend channel enable settings for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

bool the HSADC extend channel enable setting

LOCAL_INLINE bool HSADC_HWA_GetFCBoundaryFlag (HSADC_Type *const *pHsadc*)

Get the fast compare boundary flag.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

bool fast compare boundary flag

LOCAL_INLINE bool HSADC_HWA_GetFCEn (const HSADC_Type *const *pHsadc*)

Get the fast compare enable.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

bool whether the HSADC fast compare enable

LOCAL_INLINE HSADC_FastCmpRampDir HSADC_HWA_GetFCRampDir (const HSADC_Type *const *pHsadc*)

Get the fast compare ramp dir.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_FCmpRampDir the HSADC fast compare ramp dir

LOCAL_INLINE int16_t HSADC_HWA_GetFCRampRefA (const HSADC_Type *const *pHsadc*)

Get the fast compare ramp reference A of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

int16_t the HSADC fast compare ramp reference A

LOCAL_INLINE int16_t HSADC_HWA_GetFCRampRefB (const HSADC_Type *const *pHsadc*)

Get the fast compare ramp reference B of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

int16_t the HSADC fast compare ramp reference B

LOCAL_INLINE uint8_t HSADC_HWA_GetFCRampStep (const HSADC_Type *const *pHsadc*)

Get the fast compare ramp step of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint8_t the HSADC fast compare ramp step

LOCAL_INLINE int16_t HSADC_HWA_GetFCRef (const HSADC_Type *const *pHsadc*)

Get the fast compare reference of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

int16_t the HSADC fast compare reference

LOCAL_INLINE uint16_t HSADC_HWA_GetFCRefDeltaN (const HSADC_Type *const *pHsadc*)

Get the fast compare reference lower delta.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint16_t the HSADC fast compare ref lower delta

LOCAL_INLINE uint16_t HSADC_HWA_GetFCRefDeltaP (const HSADC_Type *const *pHsadc*)

Get the fast compare reference upper delta.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint16_t the HSADC fast compare ref upper delta

LOCAL_INLINE bool HSADC_HWA_GetFCResult (HSADC_Type *const *pHsadc*)

Get the fast compare result.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

bool fast compare result

LOCAL_INLINE uint32_t HSADC_HWA_GetFIFOData (const HSADC_Type *const *pHsadc*)

Get the conversion result FIFO data of the HSADC instance.

Note

only reslut data of HSADC single mode and continuous mode will be stored in FIFO register.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint32_t the HSADC conversion result

LOCAL_INLINE bool HSADC_HWA_GetFIFOReady (const HSADC_Type *const *pHsadc*)

Check whether the data quantity in the FIFO is greater than watermark.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC FIFO is ready

false the HSADC FIFO is unready

LOCAL_INLINE bool HSADC_HWA_GetFIFOReadyInterruptFlag (const HSADC_Type *const *pHsadc*)

Get the FIFO Ready interrupt flag If enabled, HSADC interrupt is generated when the FIFO water mark is greater than FWMARK.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true HSADC FIFO Ready interrupt is enabled

false HSADC FIFO Ready interrupt is disabled

LOCAL_INLINE uint8_t HSADC_HWA_GetFIFOWaterMark (const HSADC_Type *const *pHsadc*)

Get the FIFO water mark settings for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint8_t the HSADC FIFO water mark setting

LOCAL_INLINE bool HSADC_HWA_GetFull (const HSADC_Type *const *pHsadc*)

Check whether the HSADC FIFO is full.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC FIFO is full

false the HSADC FIFO is not full

LOCAL_INLINE bool HSADC_HWA_GetHSADCReadyInterruptFlag (const HSADC_Type *const *pHsadc*)

Get the HSADC ready interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC module is ready for conversion.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC ready interrupt is enabled

false the HSADC ready interrupt is disabled

LOCAL_INLINE uint32_t HSADC_HWA_GetInterruptEnable (const HSADC_Type *const *pHsadc*)

Get the interrupt enable config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint32_t the interrupt enable config

LOCAL_INLINE bool HSADC_HWA_GetOverRun (const HSADC_Type *const *pHsadc*)

Get the overrun status of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC is in overrun status

false the HSADC is not overrun

LOCAL_INLINE bool HSADC_HWA_GetOverrunInterruptFlag (const HSADC_Type *const *pHsadc*)

Get the Overrun interrupt flag If enabled, HSADC interrupt is generated when the HSADC instance is overrun.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true HSADC Overrun interrupt is enabled

false HSADC Overrun interrupt is disabled

LOCAL_INLINE HSADC_OvrModeType HSADC_HWA_GetOverrunMode (const HSADC_Type *const *pHsadc*)

Get the overrun mode.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_OVERRUN_MODE_PRESERVE the conversion data is preserved when HSADC is overrun

HSADC_OVERRUN_MODE_OVERWRITE the conversion data is overwritten when HSADC is overrun

LOCAL_INLINE bool HSADC_HWA_GetReady (const HSADC_Type *const *pHsadc*)

Check whether the HSADC instance is ready to operate.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC instance is ready for a new conversion

false the HSADC instance is unready

LOCAL_INLINE uint16_t HSADC_HWA_GetResultCal (HSADC_Type *const *pHsadc*)

LOCAL_INLINE uint8_t HSADC_HWA_GetSampleTime (HSADC_Type *const *pHsadc*, uint8_t *u8Selection*)

Get the sample time of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8Selection</i>	the sample time index, range 0~3

Returns

uint8_t the sample time of the selected index

LOCAL_INLINE bool HSADC_HWA_GetSeqGpEn (const HSADC_Type *const *pHsadc*)

Get the sequence group mode state.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

the sequence group mode state

LOCAL_INLINE uint8_t HSADC_HWA_GetSeqGroupEndPoint (HSADC_Type *const *pHsadc*, const uint8_t *u8SeqGroupIndex*)

Get the sequence group end point.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8SeqGroupIndex</i>	the index of the sequence group

LOCAL_INLINE uint8_t HSADC_HWA_GetSeqGroupStartPoint (HSADC_Type *const *pHsadc*, const uint8_t *u8SeqGroupIndex*)

Get the sequence group start point.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8SeqGroupIndex</i>	the index of the sequence group

LOCAL_INLINE uint8_t HSADC_HWA_GetSeqLength (const HSADC_Type *const *pHsadc*)

Get the sequence length of the HSADC conversion sequence.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint8_t the sequence length of the HSADC conversion sequence

LOCAL_INLINE HSADC_SeqModeType HSADC_HWA_GetSeqMode (const HSADC_Type *const *pHsadc*)

Get the HSADC sequence mode.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_SeqModeType the sequence mode the the HSADC instance

LOCAL_INLINE bool HSADC_HWA_GetSGDMAEnableFlag (const HSADC_Type *const *pHsadc*)

Get whether SGDMA for the HSADC instance is enabled.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true SGDMA is enabled for the HSADC instance

false SGDMA is disabled for the HSADC instance

LOCAL_INLINE uint8_t HSADC_HWA_GetSGDMASEL (const HSADC_Type *const *pHsadc*)

Get SGDMA SEL for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

the SGDMA SEL for the HSADC instance

LOCAL_INLINE uint8_t HSADC_HWA_GetSmpSpare (const HSADC_Type *const *pHsadc*)

LOCAL_INLINE bool HSADC_HWA_GetStart (const HSADC_Type *const *pHsadc*)

Get the conversion start status of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the conversion of the HSADC instance is started

false the conversion of the HSADC instance has not been started

LOCAL_INLINE uint8_t HSADC_HWA_GetStartupCnt (const HSADC_Type *const *pHsadc*)

Get the HSADC start up count.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

uint8_t the start count of the HSADC instance

LOCAL_INLINE bool HSADC_HWA_GetStop (const HSADC_Type *const *pHsadc*)

Get whether HSADC is in stopping status.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the HSADC instance is stopping

false the HSADC instance is not in stopping status

LOCAL_INLINE uint8_t HSADC_HWA_GetTriggerLatchUnitPriority (const HSADC_Type *const *pHsadc*)

Get the priority of Trigger Latch.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE HSADC_TrgModeType HSADC_HWA_GetTriggerMode (const HSADC_Type *const *pHsadc*)

Get the trigger mode of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_TrgModeType the trigger mode if the HSADC instance

LOCAL_INLINE HSADC_TrgSrcType HSADC_HWA_GetTriggerSource (const HSADC_Type *const *pHsadc*)

Get the trigger source the the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

HSADC_TrgSrcType the trigger source of the HSADC instance

LOCAL_INLINE bool HSADC_HWA_GetWaitMode (const HSADC_Type *const *pHsadc*)

Get whether the wait conversion mode is enabled.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

Returns

true the wait conversion mode is enabled

false the wait conversion mode is disabled

LOCAL_INLINE void HSADC_HWA_Reset (HSADC_Type *const *pHsadc*)

Assert the HSADC hardware Reset bit.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_SetAutoDisableMode (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set whether to enable auto disable mode.

Note

Auto disable mode is only available in FC7300F2M

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable auto disable mode

LOCAL_INLINE void HSADC_HWA_SetAverageEnableFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Enable or disable hardware average for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the HSADC hardware average

**LOCAL_INLINE void HSADC_HWA_SetAverageNumber (HSADC_Type *const *pHsadc*,
HSADC_AverageLenType *eAverageNumber*)**

Set the hardware average number.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eAverageNumber</i>	the hardware average number to set

LOCAL_INLINE void HSADC_HWA_SetCCal (HSADC_Type *const *pHsadc*, uint32_t *u32CCal*)

Set the HSADC_CCAL config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u32Cal</i>	the HSADC_CCAL config

LOCAL_INLINE void HSADC_HWA_SetChannelInput (HSADC_Type *const *pHsadc*, const uint8_t *u8ChnIdx*, uint8_t *u8InputChann*)

Set the input channel to the selected HSADC channel.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8ChnIdx</i>	the index of the channel
<i>u8InputChann</i>	the hardware input channel

LOCAL_INLINE void HSADC_HWA_SetChannelInterruptEnable (HSADC_Type *const *pHsadc*, const uint8_t *u8ChnIdx*, bool *bEnable*)

Set to enable or disable interrupt on the selected HSADC channel.

Note

this function is used only in HSADC sequence group mode to get the channel interrupt settings

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8ChnIdx</i>	the index of the channel
<i>bEnable</i>	whether to enable interrupt on the selected HSADC channel

LOCAL_INLINE void HSADC_HWA_SetChannelSampleTimeIndex (HSADC_Type *const *pHsadc*, const uint8_t *u8ChnIdx*, uint8_t *u8SmpSel*)

Set the sample time index of the HSADC channel.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

<i>u8ChnIdx</i>	the index of the channel
<i>u8SmpSel</i>	the sample time index of the HSADC channel

**LOCAL_INLINE void HSADC_HWA_SetClockDivider (HSADC_Type *const *pHsadc*,
HSADC_ClockDivideType *eDivider*)**

Set the HSADC clock divider.

Note

Before configuring DIV, must set CG and wait for CG_ACK=1. After configuring DIV, must clear CG and wait for CG_ACK=0.

This feature is only available in FC7300F2M

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eDivider</i>	the HSADC clock divider to set

**LOCAL_INLINE void HSADC_HWA_SetClockGatingEnableFlag (HSADC_Type *const *pHsadc*, bool
bEnable)**

Set whether to enable HSADC clock gating.

Note

This feature is only available in FC7300F2M

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable HSADC clock gating

**LOCAL_INLINE void HSADC_HWA_SetCmpChannel (HSADC_Type *const *pHsadc*, HSADC_CmpModeType
eMode, uint8_t *u8ChannalNum*)**

Set the HSADC hardware compare channel.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

<i>eMode</i>	whether the hardware compare enabled on single channel or all channels
<i>u8ChannalNum</i>	if hardware compare is enabled on single channel, this specifies the channel number

LOCAL_INLINE void HSADC_HWA_SetCmpEnable (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set to enable or disable the hardware compare.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether the HSADC hardware compare is enabled

LOCAL_INLINE void HSADC_HWA_SetCmpInterruptFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the Compare interrupt flag If enabled, HSADC interrupt is generated when the HSADC conversion result is not within the compare threshold.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the HSADC Compare interrupt

LOCAL_INLINE void HSADC_HWA_SetCmpThreshold (HSADC_Type *const *pHsadc*, uint16_t *u16LowThres*, uint16_t *u16HighThres*)

Set the HSADC hardware compare threshold.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u16LowThres</i>	the lower threshold
<i>u16HighThres</i>	the higher threshold

LOCAL_INLINE void HSADC_HWA_SetConfig1 (HSADC_Type *const *pHsadc*, uint32_t *u32Config*)

Set the HSADC_CFG1 config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u32Config</i>	the HSADC_CFG1 config

LOCAL_INLINE void HSADC_HWA_SetConfig2 (HSADC_Type *const *pHsadc*, uint32_t *u32Config*)

Set the HSADC_CFG2 config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u32Config</i>	the HSADC_CFG2 config

LOCAL_INLINE void HSADC_HWA_SetConfig3 (HSADC_Type *const *pHsadc*, uint32_t *u32Config*)

Set the HSADC_CFG3 config.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u32Config</i>	the HSADC_CFG3 config

LOCAL_INLINE void HSADC_HWA_SetDataAlignment (HSADC_Type *const *pHsadc*, HSADC_AlignType *eAlign*)

Set the data align mode.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eAlign</i>	the data align mode

LOCAL_INLINE void HSADC_HWA_SetDMAEnableFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Enable or disable the DMA for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the DMA for the HSADC instance

LOCAL_INLINE void HSADC_HWA_SetEndOfConversionInterruptFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the conversion complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion is completed.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the conversion complete interrupt

LOCAL_INLINE void HSADC_HWA_SetEndOfSampleInterruptFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the sample complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion finished the sampling phase.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the sample complete interrupt

LOCAL_INLINE void HSADC_HWA_SetEndOfSeqGroupInterruptEnable (HSADC_Type *const *pHsadc*, const uint8_t *u8SeqGroupIndex*, const bool *bEnable*)

Set the sequence group end of sequence interrupt enable.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8SeqGroupIndex</i>	the index of the sequence group
<i>bEnable</i>	the sequence group interrupt enable or disable

LOCAL_INLINE void HSADC_HWA_SetEndOfSequenceInterruptFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the End of Sequence interrupt enable flag. If enabled, HSADC interrupt is generated when the HSADC sequence conversion is completed.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the HSADC End of Sequence interrupt

LOCAL_INLINE void HSADC_HWA_SetExtChEnable (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the extend channel feature for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	the enable/disable flag

LOCAL_INLINE void HSADC_HWA_SetFCtrl (HSADC_Type *const *pHsadc*, uint32_t *value*)

Set the fast compare control register.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>value</i>	control register value

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCEn (HSADC_Type *const *pHsadc*, bool *enable*)

Set the fast compare reference of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>enable</i>	whether the HSADC fast compare enable

Returns

none

```
LOCAL_INLINE void HSADC_HWA_SetFCHysteresis (HSADC_Type *const pHsadc, uint32_t u32Hyst)
```

Set the fast compare reference hysteresis.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u32Hyst</i>	the HSADC fast compare reference hysteresis

Returns

none

```
LOCAL_INLINE void HSADC_HWA_SetFCIkSel (HSADC_Type *const pHsadc, const  
HSADC_FunctionClockType Type)
```

Select HSADC function clock.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

```
LOCAL_INLINE void HSADC_HWA_SetFCRamp0 (HSADC_Type *const pHsadc, uint32_t value)
```

Set the fast compare ramp0 register.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>value</i>	the HSADC fast compare ramp0 register value

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRamp1 (HSADC_Type *const *pHsadc*, uint32_t *value*)

Set the fast compare ramp1 register.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>value</i>	the HSADC fast compare ramp1 register value

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRampDir (HSADC_Type *const *pHsadc*, HSADC_FastCmpRampDir *dir*)

Set the fast compare ramp dir.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>dir</i>	the HSADC fast compare ramp dir

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRampRefA (HSADC_Type *const *pHsadc*, uint16_t *ref*)

Set the fast compare ramp reference A of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>ref</i>	the HSADC fast compare ramp reference A

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRampRefB (HSADC_Type *const *pHsadc*, uint16_t *ref*)

Set the fast compare ramp reference B of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>ref</i>	the HSADC fast compare ramp reference B

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRampStep (HSADC_Type *const *pHsadc*, uint16_t *step*)

Set the fast compare ramp step.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

<i>step</i>	the HSADC fast compare ramp step
-------------	----------------------------------

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRef (HSADC_Type *const *pHsadc*, uint16_t *ref*)

Set the fast compare reference of the HSADC instance.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>ref</i>	the index of the HSADC channel

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRefDeltaN (HSADC_Type *const *pHsadc*, uint16_t *delta*)

Set the fast compare reference lower delta.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>delta</i>	the HSADC fast compare ref lower delta

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFCRefDeltaP (HSADC_Type *const *pHsadc*, uint16_t *delta*)

Set the fast compare reference upper delta.

Note

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>delta</i>	the HSADC fast compare ref upper delta

Returns

none

LOCAL_INLINE void HSADC_HWA_SetFIFOReadyInterruptFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the FIFO Ready interrupt flag If enabled, HSADC interrupt is generated when the FIFO water mark is greater than FWMARK.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the HSADC FIFO Ready interrupt

LOCAL_INLINE void HSADC_HWA_SetFIFOWaterMark (HSADC_Type *const *pHsadc*, uint8_t *u8WaterMark*)

Set the FIFO water mark for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8WaterMark</i>	the HSADC FIFO water mark setting

LOCAL_INLINE void HSADC_HWA_SetHSADCReadyInterruptFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the HSADC ready interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC module is ready for conversion.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	Whether to enable the HSADC ready interrupt

LOCAL_INLINE void HSADC_HWA_SetInterruptEnable (HSADC_Type *const *pHsadc*, uint32_t *u32InterruptCfg*)

Set the interrupt enable.

Parameters

<i>u32InterruptCfg</i>	the interrupt enable config
------------------------	-----------------------------

LOCAL_INLINE void HSADC_HWA_SetOGCEnable (HSADC_Type *const *pHsadc*, bool *bEnable*)

LOCAL_INLINE void HSADC_HWA_SetOGCGain (HSADC_Type *const *pHsadc*, uint16_t *val*)

LOCAL_INLINE void HSADC_HWA_SetOGCOffset (HSADC_Type *const *pHsadc*, uint16_t *val*)

LOCAL_INLINE void HSADC_HWA_SetOverrunInterruptFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the Overrun interrupt flag If enabled, HSADC interrupt is generated when the HSADC instance is overrun.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the HSADC Overrun interrupt

LOCAL_INLINE void HSADC_HWA_SetOverrunMode (HSADC_Type *const *pHsadc*, HSADC_OvrModeType *eOvrMode*)

Set the overrun mode.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eOvrMode</i>	the overrun mode for the HSADC instance

LOCAL_INLINE void HSADC_HWA_SetSampleTime (HSADC_Type *const *pHadc*, uint8_t *u8Selection*, uint8_t *u8SampleTime*)

Set the sample time of the HSADC instance.

Parameters

<i>pHadc</i>	the base address of the HSADC instance
<i>u8Selection</i>	the sample time index, range 0~3
<i>u8SampleTime</i>	the sample time of the selected index

LOCAL_INLINE void HSADC_HWA_SetSCCfg (HSADC_Type *const *pHadc*, const uint8_t *u8ChnlIdx*, uint32_t *val*)

LOCAL_INLINE void HSADC_HWA_SetSeqGpEn (HSADC_Type *const *pHadc*, bool *bEnable*)

Set the sequence group mode state.

Parameters

<i>pHadc</i>	the base address of the HSADC instance
<i>the</i>	sequence group mode state

LOCAL_INLINE void HSADC_HWA_SetSeqGroupStartEndPoint (HSADC_Type *const *pHadc*, const uint8_t *u8SeqGroupIndex*, const uint8_t *u8Start*, const uint8_t *u8End*)

Set the sequence group start and end point.

Parameters

<i>pHadc</i>	the base address of the HSADC instance
<i>u8SeqGroupIndex</i>	the index of the sequence group
<i>u8Start</i>	the sequence group start point
<i>u8End</i>	the sequence group end point

LOCAL_INLINE void HSADC_HWA_SetSeqLength (HSADC_Type *const *pHsadc*, uint8_t *u8SequenceLength*)

Set the sequence length of the HSADC conversion sequence.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8SequenceLength</i>	the sequence length of the HSADC conversion sequence

LOCAL_INLINE void HSADC_HWA_SetSeqMode (HSADC_Type *const *pHsadc*, HSADC_SeqModeType *eSequenceMode*)

Set the HSADC sequence mode.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eSequenceMode</i>	the sequence mode the the HSADC instance

LOCAL_INLINE void HSADC_HWA_SetSGDMAEnableFlag (HSADC_Type *const *pHsadc*, bool *bEnable*)

Enable or disable the SGDMA for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the SGDMA for the HSADC instance

LOCAL_INLINE void HSADC_HWA_SetSGDMASEL (HSADC_Type *const *pHsadc*, uint8_t *index*)

Set the SGDMA SEL for the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

<i>bEnable</i>	the SGDMA SEL for the HSADC instance
----------------	--------------------------------------

LOCAL_INLINE void HSADC_HWA_SetSmpSpare (HSADC_Type *const *pHsadc*, bool *bEnable*)

LOCAL_INLINE void HSADC_HWA_SetStartupCnt (HSADC_Type *const *pHsadc*, uint8_t *u8StartupCnt*)

Set the HSADC start up count.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>u8StartupCnt</i>	the start count of the HSADC instance

LOCAL_INLINE void HSADC_HWA_SetTriggerLatchUnitPriority (HSADC_Type *const *pHsadc*, HSADC_TriggerLatchUnitPri *eTriggerLatchUnitPri*)

Set the priority of Trigger Latch.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eTriggerLatchUnitPri</i>	the priority of Trigger Latch Unit setting

LOCAL_INLINE void HSADC_HWA_SetTriggerMode (HSADC_Type *const *pHsadc*, HSADC_TriggerModeType *eTriggerMode*)

Set the trigger mode of the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eTriggerMode</i>	the trigger mode if the HSADC instance

LOCAL_INLINE void HSADC_HWA_SetTriggerSource (HSADC_Type *const *pHsadc*, HSADC_TriggerSrcType *eTriggerSource*)

Set the trigger source the the HSADC instance.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>eTriggerSource</i>	the trigger source of the HSADC instance

LOCAL_INLINE void HSADC_HWA_SetWaitMode (HSADC_Type *const *pHsadc*, bool *bEnable*)

Set the wait conversion mode.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
<i>bEnable</i>	whether to enable the wait conversion mode

LOCAL_INLINE void HSADC_HWA_Start (HSADC_Type *const *pHsadc*)

Start the HSADC conversion.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

LOCAL_INLINE void HSADC_HWA_Stop (HSADC_Type *const *pHsadc*)

Stop the HSADC conversion.

Parameters

<i>pHsadc</i>	the base address of the HSADC instance
---------------	--

HwA_ism

Collaboration diagram for HwA_ism:Classes

struct **FPC_Type**

struct **LAM_Type**

Typedefs

```
typedef struct FPC_Type * FPC_MemMapPtr  
typedef struct LAM_Type * LAM_MemMapPtr
```

Enumerations

```
enum ISM_FPC_EdgeDetectModeType { ISM_FPC_DETECT_NO_FILTER = 0U,  
ISM_FPC_DETECT_IMMI_FILTER = 1U, ISM_FPC_DETECT_DELAY_MODE = 2U,  
ISM_FPC_DETECT_PRESCALER_MODE = 3U }  
enum ISM_FPC_EdgeDelayModeType { ISM_FPC_DELAY_FIXED0 = 0U, ISM_FPC_DELAY_FIXED1 = 1U,  
ISM_FPC_DELAY_SMART_DELAY0 = 2U, ISM_FPC_DELAY_SMART_DELAY1 = 3U }  
enum ISM_LAM_InvertEventWindowType { ISM_LAM_EVT_WIN_NON_INVERT = 0U,  
ISM_LAM_EVT_WIN_INVERT = 1U }  
enum ISM_LAM_EventWindowEdgeType { ISM_LAM_NTR_CLEAR_NTR_GATE = 0U,  
ISM_LAM_POS_CLEAR_NTR_GATE = 1U, ISM_LAM_NEG_CLEAR_NTR_GATE = 2U,  
ISM_LAM_ETR_CLEAR_NTR_GATE = 3U, ISM_LAM_NTR_CLEAR_POS_GATE = 4U,  
ISM_LAM_POS_CLEAR_POS_GATE = 5U, ISM_LAM_NEG_CLEAR_POS_GATE = 6U,  
ISM_LAM_ETR_CLEAR_POS_GATE = 7U, ISM_LAM_NTR_CLEAR_NEG_GATE = 8U,  
ISM_LAM_POS_CLEAR_NEG_GATE = 9U, ISM_LAM_NEG_CLEAR_NEG_GATE = 10U,  
ISM_LAM_ETR_CLEAR_NEG_GATE = 11U, ISM_LAM_NTR_CLEAR_ETR_GATE = 12U,  
ISM_LAM_POS_CLEAR_ETR_GATE = 13U, ISM_LAM_NEG_CLEAR_ETR_GATE = 14U,  
ISM_LAM_ETR_CLEAR_ETR_GATE = 15U }  
enum ISM_LAM_EventWindowSelectType { ISM_LAM_EVT_WIN_SEL_REF = 0U,  
ISM_LAM_EVT_WIN_SEL_MON = 1U }  
enum ISM_LAM_RunModeSelectType { ISM_LAM_RUN_FREE = 0U, ISM_LAM_RUN_GATED = 1U }  
enum ISM_LAM_MonitorSourceType { ISM_LAM_SRC_FPC_MON = 0U, ISM_LAM_SRC_EXORD_FPC_REF =  
1U }  
enum ISM_LAM_InvertMonitorType { ISM_LAM_FPC_MON_NON_INVERT = 0U,  
ISM_LAM_FPC_MON_INVERT = 1U }  
enum ISM_LAM_InvertReferenceType { ISM_LAM_FPC_REF_NON_INVERT = 0U,  
ISM_LAM_FPC_REF_INVERT = 1U }
```

Functions

```
LOCAL_INLINE uint8_t ISM_HWA_PARAM_ECMC (ISM_Type *const plsm)  
Get ISM ECM count.
```

```
LOCAL_INLINE uint8_t ISM_HWA_PARAM_FPC (ISM_Type *const plsm)  
Get ISM FPC count.
```

LOCAL_INLINE uint8_t **ISM_HWA_PARAM_LAM** (ISM_Type *const plsm)

Get ISM LAM count.

LOCAL_INLINE void **ISM_HWA_Enable** (ISM_Type *const plsm, bool bEnable)

Enable ISM.

LOCAL_INLINE void **ISM_HWA_InterruptEnable** (ISM_Type *const plsm, bool bEnable)

Enable ISM Interrupt.

LOCAL_INLINE uint8_t **ISM_HWA_GetEcs** (ISM_Type *const plsm)

Get ECS of ISM_E_STATUS register.

LOCAL_INLINE void **ISM_HWA_ClearEcs** (ISM_Type *const plsm, uint32_t u32Channels)

Clear ECS of ISM_E_STATUS register.

LOCAL_INLINE uint16_t **ISM_HWA_GetEs** (ISM_Type *const plsm)

Get ES of ISM_E_STATUS register.

LOCAL_INLINE void **ISM_HWA_ClearEs** (ISM_Type *const plsm, uint32_t u32Channels)

Clear ES of ISM_E_STATUS register.

LOCAL_INLINE void **ISM_HWA_EnableEcmSystemEvent** (ISM_Type *const plsm, uint32_t u32Channels, bool bEnable)

Enable ISM ECM channels system event.

LOCAL_INLINE uint8_t **ISM_HWA_GetEnabledEcmSystemEvent** (ISM_Type *const plsm)

Get Enabled ISM ECM channels system event.

LOCAL_INLINE void **ISM_HWA_EnableLamSystemEvent** (ISM_Type *const plsm, uint32_t u32Channels, bool bEnable)

Enable ISM LAM channels system event.

LOCAL_INLINE uint16_t **ISM_HWA_GetEnabledLamSystemEvent** (ISM_Type *const plsm)

Get Enabled ISM LAM channels system event.

LOCAL_INLINE void **ISM_HWA_SetEcCtrl** (ISM_Type *const plsm, uint32_t u32Value)

Set the EC_CTRL register.

LOCAL_INLINE void **ISM_HWA_SetEcm0EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM0.

LOCAL_INLINE void **ISM_HWA_SetEcm1EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM1.

LOCAL_INLINE void **ISM_HWA_SetEcm2EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM2.

LOCAL_INLINE void **ISM_HWA_SetEcm3EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM3.

LOCAL_INLINE bool **ISM_HWA_GetFpcRgd** (FPC_Type *const pFpc)

Get the RGD value of FPC_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearFpcRgd** (FPC_Type *const pFpc)

Clear the RGD value of FPC_STATUS.

LOCAL_INLINE bool **ISM_HWA_GetFpcFgd** (FPC_Type *const pFpc)

Get the FGD value of FPC_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearFpcFgd** (FPC_Type *const pFpc)

Clear the FGD value of FPC_STATUS.

LOCAL_INLINE void **ISM_HWA_SetFpcCtrlEn** (FPC_Type *const pFpc, bool bEnable)

Set the EN of FPC_CTRL.

LOCAL_INLINE void **ISM_HWA_SetFpcCtrlIen** (FPC_Type *const pFpc, bool bEnable)

Set the IEN of FPC_CTRL.

LOCAL_INLINE bool **ISM_HWA_GetFpcCtrlIen** (FPC_Type *const pFpc)

Get the IEN of FPC_CTRL.

LOCAL_INLINE void **ISM_HWA_SetFpcFallingDetectMode** (FPC_Type *const pFpc,
ISM_FPC_EdgeDetectModeType eMode)

Set the FEG of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcFallingDelayMode** (FPC_Type *const pFpc,
ISM_FPC_EdgeDelayModeType eMode)

Set the FED of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcRisingDetectMode** (FPC_Type *const pFpc,
ISM_FPC_EdgeDetectModeType eMode)

Set the REG of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcRisingDelayMode** (FPC_Type *const pFpc,
ISM_FPC_EdgeDelayModeType eMode)

Set the RED of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcConfigCmp** (FPC_Type *const pFpc, uint32_t u32Value)

Set the CMP of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcConfig** (FPC_Type *const pFpc, uint32_t u32Value)

Set the value of FPC_CONFIG.

LOCAL_INLINE uint16_t **ISM_HWA_GetFpcTimer** (FPC_Type *const pFpc)

Get the TIM of FPC_TIMER.

LOCAL_INLINE void **ISM_HWA_ClearFpcTimer** (FPC_Type *const pFpc)

Clear the TIM of FPC_TIMER.

LOCAL_INLINE uint32_t **ISM_HWA_GetLamStatusCounter** (LAM_Type *const pLam)

Get the COUNT of LAM_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearLamStatusCounter** (LAM_Type *const pLam)

Clear the COUNT of LAM_STATUS.

LOCAL_INLINE bool **ISM_HWA_GetLamStatusOvfl** (LAM_Type *const pLam)

Get the OVFL of LAM_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearLamStatusOvfl** (LAM_Type *const pLam)

Clear the OVFL of LAM_STATUS.

LOCAL_INLINE void **ISM_HWA_SetLamCtrlEn** (LAM_Type *const pLam, bool bEnable)

Set the IEN of LAM_CTRL.

LOCAL_INLINE bool **ISM_HWA_GetLamCtrlEn** (LAM_Type *const pLam)

Get the IEN of LAM_CTRL.

LOCAL_INLINE void **ISM_HWA_SetLamCtrEn** (LAM_Type *const pLam, bool bEnable)

Set the EN of LAM_CTRL.

LOCAL_INLINE void **ISM_HWA_SetLamConfig** (LAM_Type *const pLam, uint32_t u32Value)

Set the value of LAM_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetLamCounter** (LAM_Type *const pLam, uint32_t u32Value)

Set the value of LAM_COUNTER.

LOCAL_INLINE FPC_Type * **ISM_HWA_GetFpc** (ISM_Type *const plsm, uint8_t u8FpcIndex)

Get the FPC instance.

LOCAL_INLINE LAM_Type * **ISM_HWA_GetLam** (ISM_Type *const plsm, uint8_t u8LamIndex)

Get the LAM instance.

Detailed Description

Typedef Documentation

typedef struct FPC_Type * FPC_MemMapPtr

typedef struct LAM_Type * LAM_MemMapPtr

Enumeration Type Documentation

enum ISM_FPC_EdgeDelayModeType

Enumerator:

ISM_FPC_DELAY _FIXED0	Fixed delay mode.
ISM_FPC_DELAY _FIXED1	Fixed delay mode.
ISM_FPC_DELAY _SMART_DELAY 0	Smart delay mode. The counter is decremented when a glitch happens.
ISM_FPC_DELAY _SMART_DELAY 1	Smart delay mode. The counter is reset when a glitch happens.

enum ISM_FPC_EdgeDetectModeType

Enumerator:

ISM_FPC_DETE CT_NO_FILTER	No filter mode. Copy input to SOUT directly, which is LAM input.
ISM_FPC_DETE CT_IMMI_FILTER	Immediate filter mode.
ISM_FPC_DETE CT_DELAY_MODE	Delay mode.
ISM_FPC_DETE CT_PRESCALER_MODE	Prescaler mode.

enum ISM_LAM_EventWindowEdgeType**Enumerator:**

ISM_LAM_NTR_ CLEAR_NTR_GATE	Neither edge used to clear the event window counter. Neither edge used to gate event generation.
ISM_LAM_POS_ CLEAR_NTR_GATE	Positive edge used to clear the event window counter. Neither edge used to gate event generation.
ISM_LAM_NEG_ CLEAR_NTR_GATE	Negative edge used to clear the event window counter. Neither edge used to gate event generation.
ISM_LAM_ETR_ CLEAR_NTR_GATE	Either edge used to clear the event window counter. Neither edge used to gate event generation.
ISM_LAM_NTR_ CLEAR_POS_GATE	Neither edge used to clear the event window counter. Positive edge used to gate event generation.
ISM_LAM_POS_ CLEAR_POS_GATE	Positive edge used to clear the event window counter. Positive edge used to gate event generation.

ISM_LAM_NEG_ CLEAR_POS_GATE	Negative edge used to clear the event window counter. Positive edge used to gate event generation.
ISM_LAM_ETR_ CLEAR_POS_GATE	Either edge used to clear the event window counter. Positive edge used to gate event generation.
ISM_LAM_NTR_ CLEAR_NEG_GATE	Neither edge used to clear the event window counter. Negative edge used to gate event generation.
ISM_LAM_POS_ CLEAR_NEG_GATE	Positive edge used to clear the event window counter. Negative edge used to gate event generation.
ISM_LAM_NEG_ CLEAR_NEG_GATE	Negative edge used to clear the event window counter. Negative edge used to gate event generation.
ISM_LAM_ETR_ CLEAR_NEG_GATE	Either edge used to clear the event window counter. Negative edge used to gate event generation.
ISM_LAM_NTR_ CLEAR_ETR_GATE	Neither edge used to clear the event window counter. Either edge used to gate event generation.
ISM_LAM_POS_ CLEAR_ETR_GATE	Positive edge used to clear the event window counter. Either edge used to gate event generation.
ISM_LAM_NEG_ CLEAR_ETR_GATE	Negative edge used to clear the event window counter. Either edge used to gate event generation.
ISM_LAM_ETR_ CLEAR_ETR_GATE	Either edge used to clear the event window counter. Either edge used to gate event generation.

enum ISM_LAM_EventWindowSelectType

Enumerator:

ISM_LAM_EVT_ WIN_SEL_REF	Event window generation is determined from the reference signal.
ISM_LAM_EVT_ WIN_SEL_MON	Event window generation is determined from the monitor signal.

enum ISM_LAM_InvertEventWindowType**Enumerator:**

ISM_LAM_EVT_ WIN_NON_INVE RT	Event window non-inverted.
ISM_LAM_EVT_ WIN_INVERT	Event window inverted.

enum ISM_LAM_InvertMonitorType**Enumerator:**

ISM_LAM_FPC_ MON_NON_INV ERT	Do not invert the monitor signal from FPC.
ISM_LAM_FPC_ MON_INVERT	Invert the monitor signal from FPC.

enum ISM_LAM_InvertReferenceType**Enumerator:**

ISM_LAM_FPC_ REF_NON_INVE RT	Do not invert the reference signal from FPC.
ISM_LAM_FPC_ REF_INVERT	Invert the reference signal from FPC.

enum ISM_LAM_MonitorSourceType

Enumerator:

ISM_LAM_SRC_ FPC_MON	Monitor signal is sourced directly from FPC monitor channel.
ISM_LAM_SRC_ EXORD_FPC_RE F	Monitor signal is EXOR'd with FPC reference channel.

enum ISM_LAM_RunModeSelectType

Enumerator:

ISM_LAM_RUN_ FREE	Event window generation is free-running.
ISM_LAM_RUN_ GATED	Event window generation is gated with the monitor or reference signal.

Function Documentation

LOCAL_INLINE void ISM_HWA_ClearEcs (ISM_Type *const *pIsM*, uint32_t *u32Channels*)

Clear ECS of ISM_E_STATUS register.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Channels</i>	ECM channels

LOCAL_INLINE void ISM_HWA_ClearEs (ISM_Type *const *pIsM*, uint32_t *u32Channels*)

Clear ES of ISM_E_STATUS register.

Parameters

<i>plsm</i>	ISM Instance
<i>u32Channels</i>	ECM channels

LOCAL_INLINE void ISM_HWA_ClearFpcFgd (FPC_Type *const *pFpc*)

Clear the FGD value of FPC_STATUS.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

LOCAL_INLINE void ISM_HWA_ClearFpcRgd (FPC_Type *const *pFpc*)

Clear the RGD value of FPC_STATUS.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

LOCAL_INLINE void ISM_HWA_ClearFpcTimer (FPC_Type *const *pFpc*)

Clear the TIM of FPC_TIMER.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

LOCAL_INLINE void ISM_HWA_ClearLamStatusCounter (LAM_Type *const *pLam*)

Clear the COUNT of LAM_STATUS.

Parameters

<i>pLam</i>	LAM Instance
-------------	--------------

LOCAL_INLINE void ISM_HWA_ClearLamStatusOvfl (LAM_Type *const *pLam*)

Clear the OVFL of LAM_STATUS.

Parameters

<i>pLam</i>	LAM Instance
-------------	--------------

LOCAL_INLINE void ISM_HWA_Enable (ISM_Type *const *pIsM*, bool *bEnable*)

Enable ISM.

Parameters

<i>pIsM</i>	ISM Instance
<i>bEnable</i>	EN value

LOCAL_INLINE void ISM_HWA_EnableEcmSystemEvent (ISM_Type *const *pIsM*, uint32_t *u32Channels*, bool *bEnable*)

Enable ISM ECM channels system event.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Channels</i>	ECM Channels
<i>bEnable</i>	enable value

LOCAL_INLINE void ISM_HWA_EnableLamSystemEvent (ISM_Type *const *pIsM*, uint32_t *u32Channels*, bool *bEnable*)

Enable ISM LAM channels system event.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Channels</i>	LAM Channels
<i>bEnable</i>	enable value

LOCAL_INLINE uint8_t ISM_HWA_GetEcs (ISM_Type *const *pIsM*)

Get ECS of ISM_E_STATUS register.

Parameters

<i>plsm</i>	ISM Instance
-------------	--------------

Returns

ECS value

LOCAL_INLINE uint8_t ISM_HWA_GetEnabledEcmSystemEvent (ISM_Type *const *plsm*)

Get Enabled ISM ECM channels system event.

Parameters

<i>plsm</i>	ISM Instance
-------------	--------------

Returns

Enabled ECM event

LOCAL_INLINE uint16_t ISM_HWA_GetEnabledLamSystemEvent (ISM_Type *const *plsm*)

Get Enabled ISM LAM channels system event.

Parameters

<i>plsm</i>	ISM Instance
-------------	--------------

Returns

Enabled LAM event

LOCAL_INLINE uint16_t ISM_HWA_GetEs (ISM_Type *const *plsm*)

Get ES of ISM_E_STATUS register.

Parameters

<i>plsm</i>	ISM Instance
-------------	--------------

Returns

ES value

LOCAL_INLINE FPC_Type* ISM_HWA_GetFpc (ISM_Type *const *pIsM*, uint8_t *u8FpcIndex*)

Get the FPC instance.

Parameters

<i>pIsM</i>	ISM Instance
<i>u8FpcIndex</i>	FPC index

Returns

FPC Instance

LOCAL_INLINE bool ISM_HWA_GetFpcCtrlIen (FPC_Type *const *pFpc*)

Get the IEN of FPC_CTRL.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

Returns

Interrupt enable bit

LOCAL_INLINE bool ISM_HWA_GetFpcFgd (FPC_Type *const *pFpc*)

Get the FGD value of FPC_STATUS.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

Returns

FGD value

LOCAL_INLINE bool ISM_HWA_GetFpcRgd (FPC_Type *const *pFpc*)

Get the RGD value of FPC_STATUS.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

Returns

RGD value

LOCAL_INLINE uint16_t ISM_HWA_GetFpcTimer (FPC_Type *const *pFpc*)

Get the TIM of FPC_TIMER.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

Returns

TIM value

LOCAL_INLINE LAM_Type* ISM_HWA_GetLam (ISM_Type *const *pIsM*, uint8_t *u8LamIndex*)

Get the LAM instance.

Parameters

<i>pIsM</i>	ISM Instance
<i>u8FpcIndex</i>	LAM index

Returns

LAM Instance

LOCAL_INLINE bool ISM_HWA_GetLamCtrlEn (LAM_Type *const *pLam*)

Get the IEN of LAM_CTRL.

Parameters

<i>pLam</i>	LAM Instance
-------------	--------------

Returns

LAM Channel Overflow Interrupt Enable bit

LOCAL_INLINE uint32_t ISM_HWA_GetLamStatusCounter (LAM_Type *const *pLam*)

Get the COUNT of LAM_STATUS.

Parameters

<i>pLam</i>	LAM Instance
-------------	--------------

Returns

COUNT value

LOCAL_INLINE bool ISM_HWA_GetLamStatusOvfl (LAM_Type *const *pLam*)

Get the OVFL of LAM_STATUS.

Parameters

<i>pLam</i>	LAM Instance
-------------	--------------

Returns

OVFL value

LOCAL_INLINE void ISM_HWA_InterruptEnable (ISM_Type *const *pIsM*, bool *bEnable*)

Enable ISM Interrupt.

Parameters

<i>pIsM</i>	ISM Instance
<i>bEnable</i>	IEN value

LOCAL_INLINE uint8_t ISM_HWA_PARAM_ECMC (ISM_Type *const *pIsM*)

Get ISM ECM count.

Parameters

<i>pIsM</i>	ISM Instance
-------------	--------------

Returns

ECM count

LOCAL_INLINE uint8_t ISM_HWA_PARAM_FPC (ISM_Type *const *pIsM*)

Get ISM FPC count.

Parameters

<i>pIsM</i>	ISM Instance
-------------	--------------

Returns

FPC count

LOCAL_INLINE uint8_t ISM_HWA_PARAM_LAM (ISM_Type *const *pIsM*)

Get ISM LAM count.

Parameters

<i>pIsM</i>	ISM Instance
-------------	--------------

Returns

LAM count

LOCAL_INLINE void ISM_HWA_SetEcCtrl (ISM_Type *const *pIsM*, uint32_t *u32Value*)

Set the EC_CTRL register.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Value</i>	register Value

LOCAL_INLINE void ISM_HWA_SetEcm0EcCtrl (ISM_Type *const *pIsM*, uint32_t *u32Value*, uint32_t *u32LamChannel*)

Set THRL and LAM channel of ECM0.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Value</i>	THRL value
<i>u32LamChannel</i>	Lam channel

LOCAL_INLINE void ISM_HWA_SetEcm1EcCtrl (ISM_Type *const *pIsM*, uint32_t *u32Value*, uint32_t *u32LamChannel*)

Set THRL and LAM channel of ECM1.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Value</i>	THRL value
<i>u32LamChannel</i>	Lam channel

LOCAL_INLINE void ISM_HWA_SetEcm2EcCtrl (ISM_Type *const *pIsM*, uint32_t *u32Value*, uint32_t *u32LamChannel*)

Set THRL and LAM channel of ECM2.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Value</i>	THRL value
<i>u32LamChannel</i>	Lam channel

LOCAL_INLINE void ISM_HWA_SetEcm3EcCtrl (ISM_Type *const *pIsM*, uint32_t *u32Value*, uint32_t *u32LamChannel*)

Set THRL and LAM channel of ECM3.

Parameters

<i>pIsM</i>	ISM Instance
<i>u32Value</i>	THRL value
<i>u32LamChannel</i>	Lam channel

LOCAL_INLINE void ISM_HWA_SetFpcConfig (FPC_Type *const *pFpc*, uint32_t *u32Value*)

Set the value of FPC_CONFIG.

Parameters

<i>pFpc</i>	FPC Instance
<i>u32Value</i>	Register value

LOCAL_INLINE void ISM_HWA_SetFpcConfigCmp (FPC_Type *const *pFpc*, uint32_t *u32Value*)

Set the CMP of FPC_CONFIG.

Parameters

<i>pFpc</i>	FPC Instance
<i>u32Value</i>	FPC threshold value that is compared with the 16-bit timer.

LOCAL_INLINE void ISM_HWA_SetFpcCtrlEn (FPC_Type *const *pFpc*, bool *bEnable*)

Set the EN of FPC_CTRL.

Parameters

<i>pFpc</i>	FPC Instance
<i>bEnable</i>	EN value

LOCAL_INLINE void ISM_HWA_SetFpcCtrlIen (FPC_Type *const *pFpc*, bool *bEnable*)

Set the IEN of FPC_CTRL.

Parameters

<i>pFpc</i>	FPC Instance
<i>bEnable</i>	IEN value

**LOCAL_INLINE void ISM_HWA_SetFpcFallingDelayMode (FPC_Type *const *pFpc*,
ISM_FPC_EdgeDelayModeType *eMode*)**

Set the FED of FPC_CONFIG.

Parameters

<i>pFpc</i>	FPC Instance
<i>eMode</i>	FED mode

**LOCAL_INLINE void ISM_HWA_SetFpcFallingDetectMode (FPC_Type *const *pFpc*,
ISM_FPC_EdgeDetectModeType *eMode*)**

Set the FEG of FPC_CONFIG.

Parameters

<i>pFpc</i>	FPC Instance
<i>eMode</i>	FEG mode

**LOCAL_INLINE void ISM_HWA_SetFpcRisingDelayMode (FPC_Type *const *pFpc*,
ISM_FPC_EdgeDelayModeType *eMode*)**

Set the RED of FPC_CONFIG.

Parameters

<i>pFpc</i>	FPC Instance
<i>eMode</i>	RED mode

**LOCAL_INLINE void ISM_HWA_SetFpcRisingDetectMode (FPC_Type *const *pFpc*,
ISM_FPC_EdgeDetectModeType *eMode*)**

Set the REG of FPC_CONFIG.

Parameters

<i>pFpc</i>	FPC Instance
-------------	--------------

<i>eMode</i>	REG mode
--------------	----------

LOCAL_INLINE void ISM_HWA_SetLamConfig (LAM_Type *const *pLam*, uint32_t *u32Value*)

Set the value of LAM_CONFIG.

Parameters

<i>pLam</i>	LAM Instance
<i>u32Value</i>	Reigster value

LOCAL_INLINE void ISM_HWA_SetLamCounter (LAM_Type *const *pLam*, uint32_t *u32Value*)

Set the value of LAM_COUNTER.

Parameters

<i>pLam</i>	LAM Instance
<i>u32Value</i>	Reigster value

LOCAL_INLINE void ISM_HWA_SetLamCtrEn (LAM_Type *const *pLam*, bool *bEnable*)

Set the EN of LAM_CTRL.

Parameters

<i>pLam</i>	LAM Instance
<i>bEnable</i>	EN value, LAM Channel Enable

LOCAL_INLINE void ISM_HWA_SetLamCtrlEn (LAM_Type *const *pLam*, bool *bEnable*)

Set the IEN of LAM_CTRL.

Parameters

<i>pLam</i>	LAM Instance
<i>bEnable</i>	IEN value, LAM Channel Overflow Interrupt Enable

HwA_lu

Collaboration diagram for HwA_lu:Enumerations

```
enum LU_LgType { LU_LG_0 = 0U, LU_LG_1, LU_LG_2, LU_LG_3 }
```

LU LG instance.

```
enum LU_BypassModeType { LU_NO_BYPASS = 0U, LU_AOI0_BYPASS, LU_AOI1_BYPASS,  
LU_AOI0_AOI1_BYPASS }
```

LU AOI mode.

```
enum LU_ConfigModeType { LU_BYPASS_MODE0 = 0U, LU_RS_MODE, LU_TFF_MODE, LU_DFF_MODE,  
LU_JKFF_MODE, LU_LATCH_MODE }
```

LU FF mode.

```
enum LU_InputNType { LU_INPUT_N_A = 0U, LU_INPUT_N_B, LU_INPUT_N_C, LU_INPUT_N_D }
```

LU Input(n) type.

Functions

```
LOCAL_INLINE void LU_HWA_ConfigAOI0 (LU_Type *pLu, LU_LgType eLg, uint32_t u32RegValue)
```

Configure LU LG(n) AOI0 configuration.

```
LOCAL_INLINE void LU_HWA_ConfigAOI1 (LU_Type *pLu, LU_LgType eLg, uint32_t u32RegValue)
```

Configure LU LG(n) AOI1 configuration.

```
LOCAL_INLINE void LU_HWA_ConfigCtrl (LU_Type *pLu, LU_LgType eLg, uint32_t u32RegValue)
```

Configure LU LG(n) ctrl configuration.

```
LOCAL_INLINE void LU_HWA_ConfigFilter (LU_Type *pLu, LU_LgType eLg, uint32_t u32RegValue)
```

Configure LU LG(n) filter configuration.

```
LOCAL_INLINE void LU_HWA_SetLgBypassControl (LU_Type *pLu, LU_LgType eLg, LU_BypassModeType  
eMode)
```

Set LG bypass control.

```
LOCAL_INLINE void LU_HWA_SetLgFlipFlopMode (LU_Type *pLu, LU_LgType eLg, LU_ConfigModeType  
eMode)
```

Set LG Flip-Flop mode.

LOCAL_INLINE void **LU_HWA_SetLgInputsSyncCtrl** (LU_Type *pLu, **LU_LgType** eLg, uint32_t u32Value)

Set LG inputs synchronous control.

LOCAL_INLINE void **LU_HWA_SetLgFeedbackOverrideCtrl** (LU_Type *pLu, **LU_LgType** eLg,
LU_InputNType eInput)

Set LG output feedback override control.

LOCAL_INLINE void **LU_HWA_ConfigFlipFlopTo1** (LU_Type *pLu, **LU_LgType** eLg)

Configure the output of flip-flop as "1".

LOCAL_INLINE void **LU_HWA_EnableControlFlipFlopInitOutput** (LU_Type *pLu, **LU_LgType** eLg)

Generate enable pulse.

LOCAL_INLINE void **LU_HWA_SetAOI0InputFilterSampleCount** (LU_Type *pLu, **LU_LgType** eLg, uint32_t
u32Value)

Input filter sample count for AOI0.

LOCAL_INLINE void **LU_HWA_SetAOI0InputFilterSamplePeriod** (LU_Type *pLu, **LU_LgType** eLg, uint32_t
u32Value)

Input filter sample period for AOI0.

LOCAL_INLINE void **LU_HWA_SetAOI1InputFilterSampleCount** (LU_Type *pLu, **LU_LgType** eLg, uint32_t
u32Value)

Input filter sample count for AOI1.

LOCAL_INLINE void **LU_HWA_SetAOI1InputFilterSamplePeriod** (LU_Type *pLu, **LU_LgType** eLg, uint32_t
u32Value)

Input filter sample period for AOI1.

LOCAL_INLINE void **LU_HWA_ConfigFlipFlopTo0** (LU_Type *pLu, **LU_LgType** eLg)

Configure the output of flip-flop as "0".

Detailed Description

Enumeration Type Documentation

enum LU_BypassModeType

LU AOI mode.

Enumerator:

LU_NO_BYPASS	
LU_AOI0_BYPAS S	
LU_AOI1_BYPAS S	
LU_AOI0_AOI1_ BYPASS	

enum LU_ConfigModeType

LU FF mode.

Enumerator:

LU_BYPASS_MO DE0	
LU_RS_MODE	
LU_TFF_MODE	
LU_DFF_MODE	
LU_JKFF_MODE	
LU_LATCH_MO DE	

enum LU_InputNType

LU Input(n) type.

Enumerator:

LU_INPUT_N_A	
LU_INPUT_N_B	
LU_INPUT_N_C	
LU_INPUT_N_D	

enum LU_LgType

LU LG instance.

Enumerator:

LU_LG_0	
LU_LG_1	
LU_LG_2	
LU_LG_3	

Function Documentation

LOCAL_INLINE void LU_HWA_ConfigAOI0 (LU_Type * *pLu*, LU_LgType *eLg*, uint32_t *u32RegValue*)

Configure LU LG(n) AOI0 configuration.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32RegValue</i>	AOI0 register value

LOCAL_INLINE void LU_HWA_ConfigAOI1 (LU_Type * *pLu*, LU_LgType *eLg*, uint32_t *u32RegValue*)

Configure LU LG(n) AOI1 configuration.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32RegValue</i>	AOI1 register value

LOCAL_INLINE void LU_HWA_ConfigCtrl (LU_Type * *pLu*, LU_LgType *eLg*, uint32_t *u32RegValue*)

Configure LU LG(n) ctrl configuration.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32RegValue</i>	CTRL register value

LOCAL_INLINE void LU_HWA_ConfigFilter (LU_Type * *pLu*, LU_LgType *eLg*, uint32_t *u32RegValue*)

Configure LU LG(n) filter configuration.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32RegValue</i>	FILT register value

LOCAL_INLINE void LU_HWA_ConfigFlipFlopTo0 (LU_Type * *pLu*, LU_LgType *eLg*)

Configure the output of flip-flop as "0".

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance

LOCAL_INLINE void LU_HWA_ConfigFlipFlopTo1 (LU_Type * *pLu*, LU_LgType *eLg*)

Configure the output of flip-flop as "1".

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance

LOCAL_INLINE void LU_HWA_EnableControlFlipFlopInitOutput (LU_Type * *pLu*, LU_LgType *eLg*)

Generate enable pulse.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance

**LOCAL_INLINE void LU_HWA_SetAOI0InputFilterSampleCount (LU_Type * *pLu*, LU_LgType *eLg*,
uint32_t *u32Value*)**

Input filter sample count for AOI0.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32Value</i>	Sample count value

**LOCAL_INLINE void LU_HWA_SetAOI0InputFilterSamplePeriod (LU_Type * *pLu*, LU_LgType *eLg*,
uint32_t *u32Value*)**

Input filter sample period for AOI0.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32Value</i>	Sample period value

LOCAL_INLINE void LU_HWA_SetAOI1InputFilterSampleCount (LU_Type * *pLu*, LU_LgType *eLg*, uint32_t *u32Value*)

Input filter sample count for AOI1.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32Value</i>	Sample count value

LOCAL_INLINE void LU_HWA_SetAOI1InputFilterSamplePeriod (LU_Type * *pLu*, LU_LgType *eLg*, uint32_t *u32Value*)

Input filter sample period for AOI1.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32Value</i>	Sample period value

LOCAL_INLINE void LU_HWA_SetLgBypassControl (LU_Type * *pLu*, LU_LgType *eLg*, LU_BypassModeType *eMode*)

Set LG bypass control.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>eMode</i>	LG bypass control mode

LOCAL_INLINE void LU_HWA_SetLgFeedbackOverrideCtrl (LU_Type * *pLu*, LU_LgType *eLg*, LU_InputNType *eInput*)

Set LG output feedback override control.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>eInput</i>	Feedback to LG input

LOCAL_INLINE void LU_HWA_SetLgFlipFlopMode (LU_Type * *pLu*, LU_LgType *eLg*, LU_ConfigModeType *eMode*)

Set LG Flip-Flop mode.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>eMode</i>	Flip-Flop mode

LOCAL_INLINE void LU_HWA_SetLgInputsSyncCtrl (LU_Type * *pLu*, LU_LgType *eLg*, uint32_t *u32Value*)

Set LG inputs synchronous control.

Parameters

<i>pLu</i>	LU instance
<i>eLg</i>	LG instance
<i>u32Value</i>	LG input bit,0-3 bit indicate INPUT(n)A/INPUT(n)B/INPUT(n)C/INPUT(n)D

HwA_mam

Collaboration diagram for HwA_mam:Functions

LOCAL_INLINE void **Mam_HWA_SetMatrixCfg** (MAM_Type *MAM, uint32_t *u32Value*)

Set Mam module Matrix Configure register.

LOCAL_INLINE uint32_t **Mam_HWA_GetMatrixCfg** (MAM_Type *MAM)

Read Mam module Ctrl register.

LOCAL_INLINE void **Mam_HWA_Set_Wdgctr** (MAM_Type *MAM, uint32_t u32Value)

Set Mam module Wdgctr register.

LOCAL_INLINE uint32 **Mam_HWA_Get_Wdgctr** (MAM_Type *MAM)

Read Mam module Wdgctr register.

LOCAL_INLINE void **Mam_HWA_Set_ACR** (MAM_Type *MAM, uint32_t idx, uint32_t u32Value)

Set Mam module ACR register.

LOCAL_INLINE uint32_t **Mam_HWA_Get_ACR** (MAM_Type *MAM, uint32_t idx)

Read Mam module ACR register.

LOCAL_INLINE void **Mam_HWA_Set_ACLR** (MAM_Type *MAM, uint32_t idx, uint32_t u32Value)

Set Mam module ACLR register.

LOCAL_INLINE uint32_t **Mam_HWA_Get_ACLR** (MAM_Type *MAM, uint32_t idx)

Read Mam module ACLR register.

Detailed Description

Function Documentation

LOCAL_INLINE uint32_t **Mam_HWA_Get_ACLR** (MAM_Type * *MAM*, uint32_t *idx*)

Read Mam module ACLR register.

Parameters

<i>1</i>	MAM base pointer
<i>2</i>	idx ACLR register index

Returns

ACLR register value

LOCAL_INLINE uint32_t Mam_HWA_Get_ACR (MAM_Type * *MAM*, uint32_t *idx*)

Read Mam module ACR register.

Parameters

1	MAM base pointer
2	idx ACR register index

Returns

ACR register value

LOCAL_INLINE uint32_t Mam_HWA_Get_Wdgctr (MAM_Type * *MAM*)

Read Mam module Wdgctr register.

Parameters

<i>MAM</i>	base pointer
------------	--------------

Returns

Wdgctr register value

LOCAL_INLINE uint32_t Mam_HWA_GetMatrixCfg (MAM_Type * *MAM*)

Read Mam module Ctrl register.

Parameters

<i>MAM</i>	base pointer
------------	--------------

Returns

Matrix Configure register value

LOCAL_INLINE void Mam_HWA_Set_ACLR (MAM_Type * *MAM*, uint32_t *idx*, uint32_t *u32Value*)

Set Mam module ACLR register.

Parameters

1	MAM base pointer
2	idx ACLR register index
3	u32Value ACLR register value

```
LOCAL_INLINE void Mam_HWA_Set_ACR (MAM_Type * MAM, uint32_t idx, uint32_t u32Value)
```

Set Mam module ACR register.

Parameters

1	MAM base pointer
2	idx ACR register index
3	u32Value ACR register value

```
LOCAL_INLINE void Mam_HWA_Set_Wdgctr (MAM_Type * MAM, uint32_t u32Value)
```

Set Mam module Wdgctr register.

Parameters

1	MAM base pointer
2	u32Value Wdgctr register value

```
LOCAL_INLINE void Mam_HWA_SetMatrixCfg (MAM_Type * MAM, uint32_t u32Value)
```

Set Mam module Matrix Configure register.

Parameters

1	MAM base pointer
2	u32Value Matrix Configure register value

HwA_mb

Collaboration diagram for HwA_mb:Macros

#define **MB_EVENT_NONE** 0u

The definition of mask all events.

#define **MB_EVENT_ALL** 0xFFFFFFFFu

The definition of receiving all events.

#define **MB_EVENT_ALL_REQ** 0x0000FFFFu

The definition of receiving all request events.

#define **MB_EVENT_ALL_DONE** 0xFFFF0000u

The definition of receiving all done events.

#define **MB_EVENT_REQ**(ch) (uint32_t)((uint32_t)1u << (ch))

The definition of request events on ch.

#define **MB_EVENT_DONE**(ch) (uint32_t)((uint32_t)1u << ((ch) +
MB_INTn_FLG_MASK_DONE_FLAG_MASK_SHIFT))

The definition of done events on ch.

#define **MB_CORE_MASK_CORE_NONE** 0u

The definition of issue to no core.

#define **MB_CORE_MASK_CORE_0** 1u

The definition of issue to core 0.

#define **MB_CORE_MASK_CORE_1** 2u

The definition of issue to core 1.

#define **MB_CORE_MASK_CORE_2** 4u

The definition of issue to core 2.

```
#define MB_CORE_MASK_HSM 8u
```

The definition of issue to HSM.

```
#define MB_CORE_MASK_ALL 0xFu
```

The definition of issue to all cores.

```
#define MB_CORE_MASK(core) (uint32_t)(1ul << (core))
```

The definition of issue to core.

Functions

```
LOCAL_INLINE void MB_HWA_LockIntrReg (uint8_t u8CoreIndex, uint32_t u32Mask)
```

Lock the MB_INTn registers.

```
LOCAL_INLINE void MB_HWA_UnlockIntrReg (uint8_t u8CoreIndex, uint32_t u32Mask)
```

Unlock the MB_INTn registers.

```
LOCAL_INLINE void MB_HWA_LockFlag (uint8_t u8CoreIndex)
```

Lock the MB_INTn_FLG.

```
LOCAL_INLINE void MB_HWA_UnlockFlag (uint8_t u8CoreIndex)
```

Unlock the MB_INTn_FLG.

```
LOCAL_INLINE void MB_HWA_LockFlagMask (uint8_t u8CoreIndex)
```

Lock the MB_INTn_FLG_MASK.

```
LOCAL_INLINE void MB_HWA_UnlockFlagMask (uint8_t u8CoreIndex)
```

Unlock the MB_INTn_FLG_MASK.

```
LOCAL_INLINE void MB_HWA_LockInten (uint8_t u8CoreIndex)
```

Lock the MB_INTn_INTEN.

LOCAL_INLINE void **MB_HWA_UnlockInten** (uint8_t u8CoreIndex)

Unlock the MB_INTn_INTEN.

LOCAL_INLINE void **MB_HWA_ConfigFlagMask** (uint8_t u8CoreIndex, uint32_t u32Mask)

Configure receive events of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_EnableEvent** (uint8_t u8CoreIndex, uint32_t u32Mask)

Enable receive events of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_DisableEvent** (uint8_t u8CoreIndex, uint32_t u32Mask)

Disable receive events of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_ConfigIntrEnable** (uint8_t u8CoreIndex, uint32_t u32Mask)

Configure the interrupt of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_EnableIntr** (uint8_t u8CoreIndex, uint32_t u32Mask)

Enable the interrupt of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_DisableIntr** (uint8_t u8CoreIndex, uint32_t u32Mask)

Disable the interrupt of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_ClearFlag** (uint8_t u8CoreIndex, uint32_t u32Mask)

Clear the flag of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetFlagMask** (uint8_t u8CoreIndex, uint32_t u32Mask)

Get the flag masks of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetFlag** (uint8_t u8CoreIndex, uint32_t u32Mask)

Get the flags of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetFlagStat** (uint8_t u8CoreIndex, uint32_t u32Mask)

Get the result of flag & inten of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetMasterID** (uint32_t u32Channel)

Get the master ID of the currently obtained channel.

LOCAL_INLINE void **MB_HWA_WriteData** (uint32_t u32Channel, uint32_t *pData)

Send data to the mailbox channel.

LOCAL_INLINE void **MB_HWA_GetData** (uint32_t u32Channel, uint32_t *pData)

Receive data from the mailbox channel.

LOCAL_INLINE uint32_t **MB_HWA_GetAutoClear** (uint32_t u32Channel, uint8_t u8CoreIndex)

Get the automatically clear status of the mailbox channel.

LOCAL_INLINE void **MB_HWA_ReleaseChannel** (uint32_t u32Channel)

Release the mailbox channel.

LOCAL_INLINE void **MB_HWA_UnlockChannel** (uint32_t u32Channel)

Software clears channel lock.

LOCAL_INLINE void **MB_HWA_SetDone** (uint32_t u32Channel, uint32_t u32DoneMask)

Issue a done event.

LOCAL_INLINE uint32_t **MB_HWA_GetSemaphore** (uint32_t u32Channel, uint32_t u32Mask)

Get the Communication Channel Semaphore Register.

LOCAL_INLINE void **MB_HWA_ConfigDoneMasterId** (uint32_t u32Channel, uint32_t u32MasterId)

Configure the master ID of the core that generates a done event.

LOCAL_INLINE uint32_t **MB_HWA_GetDoneMasterId** (uint32_t u32Channel)

Get the master ID of the core that generates a done event.

LOCAL_INLINE void **MB_HWA_ConfigDoneMask** (uint32_t u32Channel, uint32_t u32DoneMask)

Configure the mask of the done events.

LOCAL_INLINE uint32_t **MB_HWA_GetDoneMask** (uint32_t u32Channel)

Get the mask of the done events.

LOCAL_INLINE void **MB_HWA_ConfigAutoUnlock** (uint32_t u32Channel, uint32_t u32AutoUnlockMask)

Configure the automatically clear of the lock enable bit.

LOCAL_INLINE void **MB_HWA_ConfigRequest** (uint32_t u32Channel, uint32_t u32RequestMask)

Issue request events.

Detailed Description

Macro Definition Documentation

```
#define MB_CORE_MASK( core)  (uint32_t)(1ul << (core))
```

The definition of issue to core.

```
#define MB_CORE_MASK_ALL  0xFu
```

The definition of issue to all cores.

```
#define MB_CORE_MASK_CORE_0  1u
```

The definition of issue to core 0.

#define MB_CORE_MASK_CORE_1 2u

The definition of issue to core 1.

#define MB_CORE_MASK_CORE_2 4u

The definition of issue to core 2.

#define MB_CORE_MASK_CORE_NONE 0u

The definition of issue to no core.

#define MB_CORE_MASK_HSM 8u

The definition of issue to HSM.

#define MB_EVENT_ALL 0xFFFFFFFFu

The definition of receiving all events.

#define MB_EVENT_ALL_DONE 0xFFFF0000u

The definition of receiving all done events.

#define MB_EVENT_ALL_REQ 0x0000FFFFu

The definition of receiving all request events.

```
#define MB_EVENT_DONE( ch)  (uint32_t)((uint32_t)1u << ((ch) +
MB_INTn_FLG_MASK_DONE_FLAG_MASK_SHIFT))
```

The definition of done events on ch.

```
#define MB_EVENT_NONE  0u
```

The definition of mask all events.

```
#define MB_EVENT_REQ( ch)  (uint32_t)((uint32_t)1u << (ch))
```

The definition of request events on ch.

Function Documentation

```
LOCAL_INLINE void MB_HWA_ClearFlag (uint8_t  u8CoreIndex, uint32_t  u32Mask)
```

Clear the flag of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	indicates the flags to be cleared

```
LOCAL_INLINE void MB_HWA_ConfigAutoUnlock (uint32_t  u32Channel, uint32_t  u32AutoUnlockMask)
```

Configure the automatically clear of the lock enable bit.

Parameters

<i>u32Channel</i>	the index of the channel
-------------------	--------------------------

<i>u32AutoUnlockMask</i>	the automatically clear of the lock enable bit
--------------------------	--

LOCAL_INLINE void MB_HWA_ConfigDoneMask (uint32_t *u32Channel*, uint32_t *u32DoneMask*)

Configure the mask of the done events.

Parameters

<i>u32Channel</i>	the index of the channel
<i>u32DoneMask</i>	the cores to issue

LOCAL_INLINE void MB_HWA_ConfigDoneMasterId (uint32_t *u32Channel*, uint32_t *u32MasterId*)

Configure the master ID of the core that generates a done event.

Parameters

<i>u32Channel</i>	the index of the channel
<i>u32MasterId</i>	master ID

LOCAL_INLINE void MB_HWA_ConfigFlagMask (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Configure receive events of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	configuration for receiving events

LOCAL_INLINE void MB_HWA_ConfigIntrEnable (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Configure the interrupt of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	configuration for interrupts

LOCAL_INLINE void MB_HWA_ConfigRequest (uint32_t *u32Channel*, uint32_t *u32RequestMask*)

Issue request events.

Parameters

<i>u32Channel</i>	the index of the channel
<i>u32RequestMask</i>	the cores to issue

LOCAL_INLINE void MB_HWA_DisableEvent (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Disable receive events of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	indicates the events to be enabled

LOCAL_INLINE void MB_HWA_DisableIntr (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Disable the interrupt of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	indicates the interrupts to be disabled

LOCAL_INLINE void MB_HWA_EnableEvent (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Enable receive events of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	indicates the events to be enabled

LOCAL_INLINE void MB_HWA_EnableIntr (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Enable the interrupt of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	indicates the interrupts to be enabled

LOCAL_INLINE uint32_t MB_HWA_GetAutoClear (uint32_t *u32Channel*, uint8_t *u8CoreIndex*)

Get the automatically clear status of the mailbox channel.

Parameters

<i>u32Channel</i>	the index of the channel
<i>u8CoreIndex</i>	the index of the core

Returns

automatically clear the channel lock enable bit

LOCAL_INLINE void MB_HWA_GetData (uint32_t *u32Channel*, uint32_t * *pData*)

Receive data from the mailbox channel.

Parameters

<i>u32Channel</i>	the index of the channel
<i>pData</i>	the buffer to receive data

LOCAL_INLINE uint32_t MB_HWA_GetDoneMask (uint32_t *u32Channel*)

Get the mask of the done events.

Parameters

<i>u32Channel</i>	the index of the channel
-------------------	--------------------------

Returns

the mask of the done events

LOCAL_INLINE uint32_t MB_HWA_GetDoneMasterId (uint32_t *u32Channel*)

Get the master ID of the core that generates a done event.

Parameters

<i>u32Channel</i>	the index of the channel
-------------------	--------------------------

Returns

the master ID

LOCAL_INLINE uint32_t MB_HWA_GetFlag (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Get the flags of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	the mask to get

Returns

the flags of the mailbox interrupt channel

LOCAL_INLINE uint32_t MB_HWA_GetFlagMask (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Get the flag masks of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	the mask to get

Returns

the flag masks of the mailbox interrupt channel

LOCAL_INLINE uint32_t MB_HWA_GetFlagStat (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Get the result of flag & inten of mailbox interrupt channel.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	the mask to get

Returns

the result of flag & inten of the mailbox interrupt channel

LOCAL_INLINE uint32_t MB_HWA_GetMasterID (uint32_t *u32Channel*)

Get the master ID of the currently obtained channel.

Parameters

<i>u32Channel</i>	the index of the channel
-------------------	--------------------------

Returns

the master ID

LOCAL_INLINE uint32_t MB_HWA_GetSemaphore (uint32_t *u32Channel*, uint32_t *u32Mask*)

Get the Communication Channel Semaphore Register.

Parameters

<i>u32Channel</i>	the index of the channel
-------------------	--------------------------

Returns

Channel Lock Acquisition

LOCAL_INLINE void MB_HWA_LockFlag (uint8_t *u8CoreIndex*)

Lock the MB_INTn_FLG.

Parameters

<i>u8CoreIndex</i>	the index of the core
--------------------	-----------------------

LOCAL_INLINE void MB_HWA_LockFlagMask (uint8_t *u8CoreIndex*)

Lock the MB_INTn_FLG_MASK.

Parameters

<i>u8CoreIndex</i>	the index of the core
--------------------	-----------------------

LOCAL_INLINE void MB_HWA_LockInten (uint8_t *u8CoreIndex*)

Lock the MB_INTn_INTEN.

Parameters

<i>u8CoreIndex</i>	the index of the core
--------------------	-----------------------

LOCAL_INLINE void MB_HWA_LockIntrReg (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Lock the MB_INTn registers.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	the lock bits

LOCAL_INLINE void MB_HWA_ReleaseChannel (uint32_t *u32Channel*)

Release the mailbox channel.

Parameters

<i>u32Channel</i>	the index of the channel
-------------------	--------------------------

LOCAL_INLINE void MB_HWA_SetDone (uint32_t *u32Channel*, uint32_t *u32DoneMask*)

Issue a done event.

Parameters

<i>u32Channel</i>	the index of the channel
<i>u32DoneMask</i>	the cores to issue

LOCAL_INLINE void MB_HWA_UnlockChannel (uint32_t *u32Channel*)

Software clears channel lock.

Parameters

<i>u32Channel</i>	the index of the channel
-------------------	--------------------------

LOCAL_INLINE void MB_HWA_UnlockFlag (uint8_t *u8CoreIndex*)

Unlock the MB_INTn_FLG.

Parameters

<i>u8CoreIndex</i>	the index of the core
--------------------	-----------------------

LOCAL_INLINE void MB_HWA_UnlockFlagMask (uint8_t *u8CoreIndex*)

Unlock the MB_INTn_FLG_MASK.

Parameters

<i>u8CoreIndex</i>	the index of the core
--------------------	-----------------------

LOCAL_INLINE void MB_HWA_UnlockInten (uint8_t *u8CoreIndex*)

Unlock the MB_INTn_INTEN.

Parameters

<i>u8CoreIndex</i>	the index of the core
--------------------	-----------------------

LOCAL_INLINE void MB_HWA_UnlockIntrReg (uint8_t *u8CoreIndex*, uint32_t *u32Mask*)

Unlock the MB_INTn registers.

Parameters

<i>u8CoreIndex</i>	the index of the core
<i>u32Mask</i>	the unlock bits

LOCAL_INLINE void MB_HWA_WriteData (uint32_t *u32Channel*, uint32_t * *pData*)

Send data to the mailbox channel.

Parameters

<i>u32Channel</i>	the index of the channel
<i>pData</i>	the buffer to be written

HwA_msc

Collaboration diagram for HwA_msc:Enumerations

```
enum MSC_ReceiveStatusType { MSC_RSV_SUCCESS = 0x0U, MSC_RSV_PARITY_ERROR = 0x1U,  
MSC_RSV_STOP_ERROR = 0x2U, MSC_RSV_ERROR = 0x4U }
```

Functions

LOCAL_INLINE void MSC_HWA_SetTcctr (MSC_Type *const pMsc, uint32_t u32Value)

Get the msc TCCTR register.

LOCAL_INLINE bool MSC_HWA_GetDataNeedSend (MSC_Type *const pMsc)

Get the msc TCCTR register DTS bit.

LOCAL_INLINE void MSC_HWA_SetDataNeedSend (MSC_Type *const pMsc)

Set the msc TCCTR register DTS bit.

LOCAL_INLINE bool MSC_HWA_GetCmdNeedSend (MSC_Type *const pMsc)

Get the msc TCCTR register CTS bit.

LOCAL_INLINE bool **MSC_HWA_GetCfb** (MSC_Type *const pMsc)

Get the CFB value of TCSTR register.

LOCAL_INLINE bool **MSC_HWA_GetDfb** (MSC_Type *const pMsc)

Get the DFB value of TCSTR register.

LOCAL_INLINE void **MSC_HWA_SetNp** (MSC_Type *const pMsc, uint8_t u8Np)

Set the NP value of TCSTR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetNp** (MSC_Type *const pMsc)

Get the NP value of TCSTR register.

LOCAL_INLINE void **MSC_HWA_SetTcdar** (MSC_Type *const pMsc, uint32_t u32Value)

Set the msc TCDAR register.

LOCAL_INLINE void **MSC_HWA_SetTcdarTdh** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TDH of TCDAR register.

LOCAL_INLINE void **MSC_HWA_SetTcdarTdl** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TDL of TCDAR register.

LOCAL_INLINE void **MSC_HWA_SetTccor** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCCOR register.

LOCAL_INLINE void **MSC_HWA_SetTccorTch** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TCH of TCCOR register.

LOCAL_INLINE void **MSC_HWA_SetTccorTcl** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TCL of TCCOR register.

LOCAL_INLINE void **MSC_HWA_SetTcslr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCCLR register.

LOCAL_INLINE void **MSC_HWA_SetTcshr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCSHR register.

LOCAL_INLINE void **MSC_HWA_SetTcelr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCELR register.

LOCAL_INLINE void **MSC_HWA_Setloclr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the IOCR register.

LOCAL_INLINE void **MSC_HWA_Setlscr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the ISCR register.

LOCAL_INLINE void **MSC_HWA_SetTcdis** (MSC_Type *const pMsc)

Set the msc TCDIS.

LOCAL_INLINE void **MSC_HWA_ClearTcdis** (MSC_Type *const pMsc)

Clear the msc TCDIS.

LOCAL_INLINE void **MSC_HWA_ClearCrfi** (MSC_Type *const pMsc)

Clear the msc CRFI.

LOCAL_INLINE void **MSC_HWA_ClearCtfi** (MSC_Type *const pMsc)

Clear the msc CTFI.

LOCAL_INLINE void **MSC_HWA_ClearCcfi** (MSC_Type *const pMsc)

Clear the msc CCFI.

LOCAL_INLINE void **MSC_HWA_ClearCdfi** (MSC_Type *const pMsc)

Clear the msc CDFI.

LOCAL_INLINE uint32_t **MSC_HWA_GetInsr** (MSC_Type *const pMsc)

Get the msc INSR register.

LOCAL_INLINE void **MSC_HWA_SetIncr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the INCR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetIncr** (MSC_Type *const pMsc)

Get the msc INCR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetRccsr** (MSC_Type *const pMsc)

Get the msc RCCSR register.

LOCAL_INLINE void **MSC_HWA_SetRccsr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the RCCSR register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr0Data** (MSC_Type *const pMsc)

Get the DATA of RDR0 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr0Addr** (MSC_Type *const pMsc)

Get the LAF of RDR0 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr0Vld** (MSC_Type *const pMsc)

Get the VLD of RDR0 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr0Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR0 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr0Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr1Data** (MSC_Type *const pMsc)

Get the DATA of RDR1 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr1Addr** (MSC_Type *const pMsc)

Get the LAF of RDR1 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr1Vld** (MSC_Type *const pMsc)

Get the VLD of RDR1 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr1Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR1 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr1Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr2Data** (MSC_Type *const pMsc)

Get the DATA of RDR2 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr2Addr** (MSC_Type *const pMsc)

Get the LAF of RDR2 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr2Vld** (MSC_Type *const pMsc)

Get the VLD of RDR2 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr2Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR2 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr2Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr3Data** (MSC_Type *const pMsc)

Get the DATA of RDR3 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr3Addr** (MSC_Type *const pMsc)

Get the LAF of RDR3 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr3Vld** (MSC_Type *const pMsc)

Get the VLD of RDR3 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr3Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR3 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr3Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE void **MSC_HWA_SetMsrRst** (MSC_Type *const pMsc)

Reset the msc.

LOCAL_INLINE bool **MSC_HWA_GetMsrRdone** (MSC_Type *const pMsc)

Get the reset status.

LOCAL_INLINE void **MSC_HWA_ClearMsrDone** (MSC_Type *const pMsc)

clear reset status

LOCAL_INLINE uint32_t **MSC_HWA_GetRtor** (MSC_Type *const pMsc)

Get the msc RTOR register.

LOCAL_INLINE void **MSC_HWA_SetRtor** (MSC_Type *const pMsc, uint32_t u32Value)

Set the msc RTOR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetTcctr1** (MSC_Type *const pMsc)

Get the msc TCCTR1 register.

LOCAL_INLINE void **MSC_HWA_SetTcctr1** (MSC_Type *const pMsc, uint32_t u32Value)

Set the msc TCCTR1 register.

LOCAL_INLINE void **MSC_HWA_SetMscEnable** (MSC_Type *const pMsc, bool bEnable)

Set the EN msc GCR register.

LOCAL_INLINE void **MSC_HWA_SetMscWriteProtection** (MSC_Type *const pMsc, bool bEnable)

Set the WP_EN of msc GCR register.

LOCAL_INLINE void **MSC_HWA_Unlock** (MSC_Type *const pMsc)

Unlock the msc CCULR register.

Detailed Description

Enumeration Type Documentation

enum MSC_ReceiveStatusType

Enumerator:

MSC_RSV_SUCCESS	MSC receive status is successful.
MSC_RSV_PARITY_ERROR	MSC receive has parity error.
MSC_RSV_STOP_ERROR	MSC receive has stop error.
MSC_RSV_ERROR	MSC receive status is not successful.

Function Documentation

LOCAL_INLINE void **MSC_HWA_ClearCcfi** (MSC_Type *const *pMsc*)

Clear the msc CCFI.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearCdfi (MSC_Type *const *pMsc*)

Clear the msc CDFI.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearCrfi (MSC_Type *const *pMsc*)

Clear the msc CRFI.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearCtfi (MSC_Type *const *pMsc*)

Clear the msc CTFI.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearMsrDone (MSC_Type *const *pMsc*)

clear reset status

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearRdr0Vld (MSC_Type *const *pMsc*)

Clear the VLD of register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearRdr1Vld (MSC_Type *const *pMsc*)

Clear the VLD of register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearRdr2Vld (MSC_Type *const *pMsc*)

Clear the VLD of register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearRdr3Vld (MSC_Type *const *pMsc*)

Clear the VLD of register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_ClearTcdis (MSC_Type *const *pMsc*)

Clear the msc TCDIS.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE bool MSC_HWA_GetCfb (MSC_Type *const *pMsc*)

Get the CFB value of TCSTR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

CFB value

LOCAL_INLINE bool MSC_HWA_GetCmdNeedSend (MSC_Type *const *pMsc*)

Get the msc TCCTR register CTS bit.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

CTS value

LOCAL_INLINE bool MSC_HWA_GetDataNeedSend (MSC_Type *const *pMsc*)

Get the msc TCCTR register DTS bit.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

DTS value

LOCAL_INLINE bool MSC_HWA_GetDfb (MSC_Type *const *pMsc*)

Get the DFB value of TCSTR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

DFB value

LOCAL_INLINE uint32_t MSC_HWA_GetIncr (MSC_Type *const *pMsc*)

Get the msc INCR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

INCR register value

LOCAL_INLINE uint32_t MSC_HWA_GetInsr (MSC_Type *const *pMsc*)

Get the msc INSR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

INSR register value

LOCAL_INLINE bool MSC_HWA_GetMsrRdone (MSC_Type *const *pMsc*)

Get the reset status.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

reset status

LOCAL_INLINE uint32_t MSC_HWA_GetNp (MSC_Type *const *pMsc*)

Get the NP value of TCSTR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

NP value

LOCAL_INLINE uint32_t MSC_HWA_GetRccsr (MSC_Type *const *pMsc*)

Get the msc RCCSR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RCCSR register value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr0Addr (MSC_Type *const *pMsc*)

Get the LAF of RDR0 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

LAF value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr0Data (MSC_Type *const *pMsc*)

Get the DATA of RDR0 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RDATA value

LOCAL_INLINE MSC_ReceiveStatusType MSC_HWA_GetRdr0Rerr (MSC_Type *const *pMsc*)

Get the RERR of RDR0 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RERR value

LOCAL_INLINE bool MSC_HWA_GetRdr0Vld (MSC_Type *const *pMsc*)

Get the VLD of RDR0 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

VLD value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr1Addr (MSC_Type *const *pMsc*)

Get the LAF of RDR1 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

LAF value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr1Data (MSC_Type *const *pMsc*)

Get the DATA of RDR1 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RDATA value

LOCAL_INLINE MSC_ReceiveStatusType MSC_HWA_GetRdr1Rerr (MSC_Type *const *pMsc*)

Get the RERR of RDR1 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RERR value

LOCAL_INLINE bool MSC_HWA_GetRdr1Vld (MSC_Type *const *pMsc*)

Get the VLD of RDR1 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

VLD value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr2Addr (MSC_Type *const *pMsc*)

Get the LAF of RDR2 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

LAF value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr2Data (MSC_Type *const *pMsc*)

Get the DATA of RDR2 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RDATA value

LOCAL_INLINE MSC_ReceiveStatusType MSC_HWA_GetRdr2Rerr (MSC_Type *const *pMsc*)

Get the RERR of RDR2 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RERR value

LOCAL_INLINE bool MSC_HWA_GetRdr2Vld (MSC_Type *const *pMsc*)

Get the VLD of RDR2 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

VLD value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr3Addr (MSC_Type *const *pMsc*)

Get the LAF of RDR3 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

LAF value

LOCAL_INLINE uint8_t MSC_HWA_GetRdr3Data (MSC_Type *const *pMsc*)

Get the DATA of RDR3 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RDATA value

LOCAL_INLINE MSC_ReceiveStatusType MSC_HWA_GetRdr3Rerr (MSC_Type *const *pMsc*)

Get the RERR of RDR3 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RERR value

LOCAL_INLINE bool MSC_HWA_GetRdr3Vld (MSC_Type *const *pMsc*)

Get the VLD of RDR3 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

VLD value

LOCAL_INLINE uint32_t MSC_HWA_GetRtor (MSC_Type *const *pMsc*)

Get the msc RTOR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

RTOR register value

LOCAL_INLINE uint32_t MSC_HWA_GetTcctr1 (MSC_Type *const *pMsc*)

Get the msc TCCTR1 register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

Returns

TCCTR1 register value

LOCAL_INLINE void MSC_HWA_SetDataNeedSend (MSC_Type *const *pMsc*)

Set the msc TCCTR register DTS bit.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_SetIncr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the INCR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	INCR value

LOCAL_INLINE void MSC_HWA_SetIocr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the IOCR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	IOCR value

LOCAL_INLINE void MSC_HWA_SetIscr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the ISCR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	ISCR value

LOCAL_INLINE void MSC_HWA_SetMscEnable (MSC_Type *const *pMsc*, bool *bEnable*)

Set the EN msc GCR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>bEnable</i>	EN value

LOCAL_INLINE void MSC_HWA_SetMscWriteProtection (MSC_Type *const *pMsc*, bool *bEnable*)

Set the WP_EN of msc GCR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>bEnable</i>	WP_EN value

LOCAL_INLINE void MSC_HWA_SetMsrRst (MSC_Type *const *pMsc*)

Reset the msc.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_SetNp (MSC_Type *const *pMsc*, uint8_t *u8Np*)

Set the NP value of TCSTR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_SetRccsr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the RCCSR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	RCCSR value

LOCAL_INLINE void MSC_HWA_SetRtor (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the msc RTOR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	RTOR value

LOCAL_INLINE void MSC_HWA_SetTccor (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the TCCOR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

<i>u32Value</i>	TCCOR value
-----------------	-------------

LOCAL_INLINE void MSC_HWA_SetTccorTch (MSC_Type *const *pMsc*, uint16_t *u16Value*)

Set the TCH of TCCOR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u16Value</i>	TCH value

LOCAL_INLINE void MSC_HWA_SetTccorTcl (MSC_Type *const *pMsc*, uint16_t *u16Value*)

Set the TCL of TCCOR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u16Value</i>	TCL value

LOCAL_INLINE void MSC_HWA_SetTcctr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Get the msc TCCTR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	TCCTR register value

LOCAL_INLINE void MSC_HWA_SetTcctr1 (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the msc TCCTR1 register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	TCCTR1 value

LOCAL_INLINE void MSC_HWA_SetTcdar (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the msc TCDAR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	TCDAR value

LOCAL_INLINE void MSC_HWA_SetTcdarTdh (MSC_Type *const *pMsc*, uint16_t *u16Value*)

Set the TDH of TCDAR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u16Value</i>	TDH value

LOCAL_INLINE void MSC_HWA_SetTcdarTdl (MSC_Type *const *pMsc*, uint16_t *u16Value*)

Set the TDL of TCDAR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u16Value</i>	TDL value

LOCAL_INLINE void MSC_HWA_SetTcdis (MSC_Type *const *pMsc*)

Set the msc TCDIS.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

LOCAL_INLINE void MSC_HWA_SetTcelr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the TCELR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	TCELR value

LOCAL_INLINE void MSC_HWA_SetTcshr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the TCSHR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	TCSHR value

LOCAL_INLINE void MSC_HWA_SetTcslr (MSC_Type *const *pMsc*, uint32_t *u32Value*)

Set the TCCLR register.

Parameters

<i>pMsc</i>	MSCInstance
<i>u32Value</i>	TCCLR value

LOCAL_INLINE void MSC_HWA_Unlock (MSC_Type *const *pMsc*)

Unlock the msc CCCLR register.

Parameters

<i>pMsc</i>	MSCInstance
-------------	-------------

HWA_ospi

Collaboration diagram for HWA_ospi:Macros

#define CTRL_RST_VALUE 0x34000

Enumerations

```
enum OSPI_DqsSrcSelType { DQS_INTER_LOOPBACK = 0, DQS_PAD_LOOPBACK,  
DQS_EXTERNAL_PADINPUT }  
enum OSPI_ClockDivideType { OSPI_CLOCK_DIV_1 = 0U, OSPI_CLOCK_DIV_2 = 1U, OSPI_CLOCK_DIV_3 =  
2U, OSPI_CLOCK_DIV_4 = 3U, OSPI_CLOCK_DIV_5 = 4U, OSPI_CLOCK_DIV_6 = 5U, OSPI_CLOCK_DIV_7 =  
6U, OSPI_CLOCK_DIV_8 = 7U }  
enum OSPI_ClockMuxType { OSPI_MUX_PLL0 = 0, OSPI_MUX_FIRC, OSPI_MUX_PLL1 }  
enum OSPI_EndianType { OSPI_BIG_ENDIAN = 0, OSPI_LITTLE_ENDIAN }
```

Functions

```
LOCAL_INLINE void OSPI_HWA_Dio3DefHigh (OSPI_Type *pOSPI)  
OSPI DIO3 Default High.
```

```
LOCAL_INLINE void OSPI_HWA_Dio3DefLow (OSPI_Type *pOSPI)  
OSPI DIO3 Default Low.
```

```
LOCAL_INLINE void OSPI_HWA_Dio2DefHigh (OSPI_Type *pOSPI)  
OSPI DIO2 Default High.
```

```
LOCAL_INLINE void OSPI_HWA_Dio2DefLow (OSPI_Type *pOSPI)  
OSPI DIO2 Default Low.
```

```
LOCAL_INLINE void OSPI_HWA_ModuleEnable (OSPI_Type *pOSPI)  
OSPI Module Enable.
```

```
LOCAL_INLINE void OSPI_HWA_ModuleDisable (OSPI_Type *pOSPI)  
OSPI Module Disable.
```

```
LOCAL_INLINE void OSPI_HWA_ClearTxFifo (OSPI_Type *pOSPI)  
OSPI Clear Tx fifo.
```

```
LOCAL_INLINE void OSPI_HWA_ClearRxFifo (OSPI_Type *pOSPI)  
OSPI Clear Rx fifo.
```


LOCAL_INLINE void **OSPI_HWA_DdrModeEnable** (OSPI_Type *pOSPI)
OSPI DDR Mode Enable.

LOCAL_INLINE void **OSPI_HWA_DdrModeDisable** (OSPI_Type *pOSPI)
OSPI DDR Mode Disable.

LOCAL_INLINE void **OSPI_HWA_DqsModeEnable** (OSPI_Type *pOSPI)
OSPI DQS Mode Enable.

LOCAL_INLINE void **OSPI_HWA_DqsModeDisable** (OSPI_Type *pOSPI)
OSPI DQS Mode Disable.

LOCAL_INLINE void **OSPI_HWA_DqsLatEnable** (OSPI_Type *pOSPI)
OSPI DQS Lantency Enable.

LOCAL_INLINE void **OSPI_HWA_DqsLatDisable** (OSPI_Type *pOSPI)
OSPI DQS Lantency Disable.

LOCAL_INLINE void **OSPI_HWA_DqsOutEnable** (OSPI_Type *pOSPI)
OSPI DQS Out Enable.

LOCAL_INLINE void **OSPI_HWA_DqsOutDisable** (OSPI_Type *pOSPI)
OSPI DQS Out Disable.

LOCAL_INLINE void **OSPI_HWA_EndianSelect** (OSPI_Type *pOSPI, **OSPI_EndianType** eEndian)
Set OSPI Endian value.

LOCAL_INLINE void **OSPI_HWA_SetCtrlValue** (OSPI_Type *pOSPI, uint32_t u32Value)
Set OSPI CTRL value, users should write the whole value to this register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetCtrlValue** (OSPI_Type *pOSPI)
Read the OSPI CTRL register value for all.

LOCAL_INLINE void **OSPI_HWA_SetCmdId** (OSPI_Type *pOSPI, uint16_t u16Value)

Set CMD ID.

LOCAL_INLINE void **OSPI_HWA_SetCmdSize** (OSPI_Type *pOSPI, uint16_t u16Value)

Set CMD SIZE.

LOCAL_INLINE void **OSPI_HWA_SetCmdIdSize** (OSPI_Type *pOSPI, uint8_t u8SeqId, uint16_t u16Size)

Set CMD id and size.

LOCAL_INLINE uint32_t **OSPI_HWA_GetCmdcValue** (OSPI_Type *pOSPI)

Read the OSPI CMDC register value for all.

LOCAL_INLINE void **OSPI_HWA_SetDataHoldTime** (OSPI_Type *pOSPI, uint8_t u8Value)

Set Data Hold Time, This bit should only be valid in DDR mode.

LOCAL_INLINE void **OSPI_HWA_SetCsHoldTime** (OSPI_Type *pOSPI, uint8_t u8Value)

Set CS Hold Time.

LOCAL_INLINE void **OSPI_HWA_SetCsSetupTime** (OSPI_Type *pOSPI, uint8_t u8Value)

Set CS Setup Time.

LOCAL_INLINE uint32_t **OSPI_HWA_GetFlashCfgValue** (OSPI_Type *pOSPI)

Read the OSPI CMDC register value for all.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkEnable** (OSPI_Type *pOSPI)

OSPI Enable internal reference clock.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkDisable** (OSPI_Type *pOSPI)

OSPI Disable internal reference clock.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkDiv** (OSPI_Type *pOSPI, **OSPI_ClockDivideType** eClkDiv)

OSPI Set Internal Reference Clock Divider.

LOCAL_INLINE void **OSPI_HWA_PadIbeEnable** (OSPI_Type *pOSPI)

OSPI PAD IBE Enable.

LOCAL_INLINE void **OSPI_HWA_PadIbeDisable** (OSPI_Type *pOSPI)

OSPI PAD IBE Disable.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkSource** (OSPI_Type *pOSPI, **OSPI_ClockMuxType** u8Value)

OSPI Select Internal Reference Clock Source.

LOCAL_INLINE void **OSPI_HWA_DqsInvertedEnable** (OSPI_Type *pOSPI)

OSPI Use inverted DQS.

LOCAL_INLINE void **OSPI_HWA_DqsInvertedDisable** (OSPI_Type *pOSPI)

OSPI Not use inverted DQS.

LOCAL_INLINE void **OSPI_HWA_DqsSource** (OSPI_Type *pOSPI, **OSPI_DqsSrcSelType** u8Value)

OSPI Select DQS Source.

LOCAL_INLINE void **OSPI_HWA_ObeTimRelaxEnable** (OSPI_Type *pOSPI)

OSPI OBE timing relax enable.

LOCAL_INLINE void **OSPI_HWA_ObeTimRelaxDisable** (OSPI_Type *pOSPI)

OSPI OBE timing relax disable.

LOCAL_INLINE void **OSPI_HWA_DdrClkEnable** (OSPI_Type *pOSPI)

OSPI DDR Clock Enable.

LOCAL_INLINE void **OSPI_HWA_DdrClkDisable** (OSPI_Type *pOSPI)

OSPI DDR Clock Disable.

LOCAL_INLINE void **OSPI_HWA_CfgDelayLine** (OSPI_Type *pOSPI, uint8_t u8Value)

*OSPI Switch Delay Line into DQS for timing. DQS will delay for DLLINE_CFG * Tcell.*

LOCAL_INLINE void **OSPI_HWA_SetSocCfgValue** (OSPI_Type *pOSPI, uint32_t u32Value)

Write the OSPI SOC CFG register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetSocCfgValue** (OSPI_Type *pOSPI)

Read the OSPI SOC CFG register value for all.

LOCAL_INLINE void **OSPI_HWA_SetFlashAddr** (OSPI_Type *pOSPI, uint32_t u32Value)

Set flash address, users should write the whole value to this register.

LOCAL_INLINE void **OSPI_HWA_SetFlashAddrMode** (OSPI_Type *pOSPI, uint8_t u8Value)

Set flash addressable mode.

LOCAL_INLINE void **OSPI_HWA_SetColAddrSpace** (OSPI_Type *pOSPI, uint8_t u8Value)

Set Column Address Space.

LOCAL_INLINE void **OSPI_HWA_SetSampleDlyClkCycle** (OSPI_Type *pOSPI, uint8_t u8Value)

Set Sample delay clock cycle.

LOCAL_INLINE void **OSPI_HWA_SelectRefClkEdge** (OSPI_Type *pOSPI, uint8_t u8Value)

Selects reference clock edge for valid sampling phase.

LOCAL_INLINE uint16_t **OSPI_HWA_GetRxFifoCnt** (OSPI_Type *pOSPI)

Read RX FIFO Counter, Indicates the current RX FIFO read pointer; Automatically increases when RX FIFO pops an event.

LOCAL_INLINE uint8_t **OSPI_HWA_GetRxFifoFillLevel** (OSPI_Type *pOSPI)

Read RX FIFO Fill Level, Indicates how many words are available in RX FIFO.

LOCAL_INLINE void **OSPI_HWA_SetRxFifoWaterMark** (OSPI_Type *pOSPI, uint8_t u8Value)

Set RX FIFO Watermark, Indicates how many valid entries will trigger a readout action.

LOCAL_INLINE uint16_t **OSPI_HWA_GetTxFifoCnt** (OSPI_Type *pOSPI)

Read TX FIFO Counter, Indicates the current TX FIFO stored data pointer; Automatically increases when TX FIFO pops an event.

LOCAL_INLINE uint8_t **OSPI_HWA_GetTxFifoFillLevel** (OSPI_Type *pOSPI)

Read TX FIFO Fill Level, Indicates how many words are available in TX FIFO.

LOCAL_INLINE uint32_t **OSPI_HWA_GetTFDRAddr** (OSPI_Type *pOSPI)

Get transmit fifo register address.

LOCAL_INLINE void **OSPI_HWA_WriteTxData** (OSPI_Type *pOSPI, uint32_t u32Value)

Writing this register will trigger data entry of TX FIFO.

LOCAL_INLINE void **OSPI_HWA_SetTxFifoWaterMark** (OSPI_Type *pOSPI, uint8_t u8Value)

Set TX FIFO Watermark, Indicates how many valid entries will trigger a transmit action.

LOCAL_INLINE uint32_t **OSPI_HWA_GetStatus** (OSPI_Type *pOSPI)

Read status register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetFlag** (OSPI_Type *pOSPI)

Read flag register.

LOCAL_INLINE void **OSPI_HWA_ClearFlag** (OSPI_Type *pOSPI, uint32_t u32Value)

Writing this register will clear ospi flag.

LOCAL_INLINE void **OSPI_HWA_SetIntDmasEnableReg** (OSPI_Type *pOSPI, uint32_t u32Value)

Set the OSPI IND_EN register value for enable or disable interrupts&DMA.

LOCAL_INLINE void **OSPI_HWA_SetRxDMA** (OSPI_Type *pOSPI, uint8_t u8Enable)

Set the OSPI Rx DMA mode.

LOCAL_INLINE void **OSPI_HWA_SetTxDMA** (OSPI_Type *pOSPI, uint8_t u8Enable)

Set the OSPI Tx DMA mode.

LOCAL_INLINE void **OSPI_HWA_SetFlashTopAddr** (OSPI_Type *pOSPI, uint32_t u32Value)

Set flash top address register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetRFDRAAddr** (OSPI_Type *pOSPI)

Get receive fifo register address.

LOCAL_INLINE uint32_t **OSPI_HWA_ReadRcvFifoReg** (OSPI_Type *pOSPI, uint8_t u8Idx)

Read receive fifo data register.

LOCAL_INLINE uint32_t **OSPI_HWA_ReadSecondaryRcvFifoReg** (OSPI_Type *pOSPI)

Read secondary receive fifo data register.

LOCAL_INLINE void **OSPI_HWA_WriteLutKey** (OSPI_Type *pOSPI, uint32_t u32Value)

Write LUT Key register.

LOCAL_INLINE uint32_t **OSPI_HWA_ReadLutKey** (OSPI_Type *pOSPI)

Read LUT Key register.

LOCAL_INLINE void **OSPI_HWA_LockLut** (OSPI_Type *pOSPI)

Lock the LUT.

LOCAL_INLINE void **OSPI_HWA_UnlockLut** (OSPI_Type *pOSPI)

Unlock the LUT.

LOCAL_INLINE void **OSPI_HWA_WriteLut** (OSPI_Type *pOSPI, uint8_t u8Idx, uint32_t u32Value)

Write lookup table register.

Detailed Description

Macro Definition Documentation

```
#define CTRL_RST_VALUE 0x34000
```

Enumeration Type Documentation

enum OSPI_ClockDivideType

Enumerator:

OSPI_CLOCK_DI V_1	
OSPI_CLOCK_DI V_2	
OSPI_CLOCK_DI V_3	
OSPI_CLOCK_DI V_4	
OSPI_CLOCK_DI V_5	
OSPI_CLOCK_DI V_6	
OSPI_CLOCK_DI V_7	
OSPI_CLOCK_DI V_8	

enum OSPI_ClockMuxType

Enumerator:

OSPI_MUX_PLL 0	
OSPI_MUX_FIRC	
OSPI_MUX_PLL 1	

enum OSPI_DqsSrcSelType

Enumerator:

DQS_INTER_LO OPBACK	
DQS_PAD_LOO PBACK	
DQS_EXTERNAL _PADINPUT	

enum OSPI_EndianType

Enumerator:

OSPI_BIG_ENDI AN	
OSPI_LITTLE_E NDIAN	

Function Documentation

LOCAL_INLINE void OSPI_HWA_CfgDelayLine (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

OSPI Switch Delay Line into DQS for timing. DQS will delay for DLLINE_CFG * Tcell.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	DQS Delay Line Cfg Value.

LOCAL_INLINE void OSPI_HWA_ClearFlag (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Writing this register will clear ospi flag.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

<i>u8Value</i>	the value of flag to be cleared.
----------------	----------------------------------

LOCAL_INLINE void OSPI_HWA_ClearRxFifo (OSPI_Type * *pOSPI*)

OSPI Clear Rx fifo.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_ClearTxFifo (OSPI_Type * *pOSPI*)

OSPI Clear Tx fifo.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DdrClkDisable (OSPI_Type * *pOSPI*)

OSPI DDR Clock Disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DdrClkEnable (OSPI_Type * *pOSPI*)

OSPI DDR Clock Enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DdrModeDisable (OSPI_Type * *pOSPI*)

OSPI DDR Mode Disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DdrModeEnable (OSPI_Type * *pOSPI*)

OSPI DDR Mode Enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_Dio2DefHigh (OSPI_Type * *pOSPI*)

OSPI DIO2 Default High.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_Dio2DefLow (OSPI_Type * *pOSPI*)

OSPI DIO2 Default Low.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_Dio3DefHigh (OSPI_Type * *pOSPI*)

OSPI DIO3 Default High.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_Dio3DefLow (OSPI_Type * *pOSPI*)

OSPI DIO3 Default Low.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsInvertedDisable (OSPI_Type * *pOSPI*)

OSPI Not use inverted DQS.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsInvertedEnable (OSPI_Type * *pOSPI*)

OSPI Use inverted DQS.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsLatDisable (OSPI_Type * *pOSPI*)

OSPI DQS Lantency Disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsLatEnable (OSPI_Type * *pOSPI*)

OSPI DQS Lantency Enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsModeDisable (OSPI_Type * *pOSPI*)

OSPI DQS Mode Disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsModeEnable (OSPI_Type * *pOSPI*)

OSPI DQS Mode Enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsOutDisable (OSPI_Type * *pOSPI*)

OSPI DQS Out Disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsOutEnable (OSPI_Type * *pOSPI*)

OSPI DQS Out Enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_DqsSource (OSPI_Type * *pOSPI*, OSPI_DqsSrcSelType *u8Value*)

OSPI Select DQS Source.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	DQS Source.

LOCAL_INLINE void OSPI_HWA_EndianSelect (OSPI_Type * *pOSPI*, OSPI_EndianType *eEndian*)

Set OSPI Endian value.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>eEndian</i>	Endian select value.

LOCAL_INLINE uint32_t OSPI_HWA_GetCmdcValue (OSPI_Type * *pOSPI*)

Read the OSPI CMDC register value for all.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

OSPI CMDC regsiter value.

LOCAL_INLINE uint32_t OSPI_HWA_GetCtrlValue (OSPI_Type * *pOSPI*)

Read the OSPI CTRL register value for all.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

OSPI CTRL regsiter value.

LOCAL_INLINE uint32_t OSPI_HWA_GetFlag (OSPI_Type * *pOSPI*)

Read flag register.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

flag register.

LOCAL_INLINE uint32_t OSPI_HWA_GetFlashCfgValue (OSPI_Type * *pOSPI*)

Read the OSPI CMDC register value for all.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

OSPI CMDC regisiter value.

LOCAL_INLINE uint32_t OSPI_HWA_GetRFDRAAddr (OSPI_Type * *pOSPI*)

Get receive fifo register address.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

Returns

Receive fifo register address.

LOCAL_INLINE uint16_t OSPI_HWA_GetRxFifoCnt (OSPI_Type * *pOSPI*)

Read RX FIFO Counter,Indicates the current RX FIFO read pointer; Automatically increases when RX FIFO pops an event.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

RX FIFO Counter.

LOCAL_INLINE uint8_t OSPI_HWA_GetRxFifoFillLevel (OSPI_Type * *pOSPI*)

Read RX FIFO Fill Level,Indicates how many words are available in RX FIFO.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

RX FIFO Fill Level.

LOCAL_INLINE uint32_t OSPI_HWA_GetSocCfgValue (OSPI_Type * *pOSPI*)

Read the OSPI SOC CFG register value for all.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

OSPI SOC_CFG regsiter value.

LOCAL_INLINE uint32_t OSPI_HWA_GetStatus (OSPI_Type * *pOSPI*)

Read status register.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

Status register.

LOCAL_INLINE uint32_t OSPI_HWA_GetTFDRAddr (OSPI_Type * *pOSPI*)

Get transmit fifo register address.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

Returns

Transmit fifo register address.

LOCAL_INLINE uint16_t OSPI_HWA_GetTxFifoCnt (OSPI_Type * *pOSPI*)

Read TX FIFO Counter, Indicates the current TX FIFO stored data pointer; Automatically increases when TX FIFO pops an event.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

TX FIFO Counter.

LOCAL_INLINE uint8_t OSPI_HWA_GetTxFifoFillLevel (OSPI_Type * *pOSPI*)

Read TX FIFO Fill Level, Indicates how many words are available in TX FIFO.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
--------------	-------------------------------------

Returns

TX FIFO Fill Level.

LOCAL_INLINE void OSPI_HWA_InternalRefclkDisable (OSPI_Type * *pOSPI*)

OSPI Disable internal reference clock.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_InternalRefclkDiv (OSPI_Type * *pOSPI*, OSPI_ClockDivideType *eClkDiv*)

OSPI Set Internal Reference Clock Divider.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	Internal Reference Clock Divider.

LOCAL_INLINE void OSPI_HWA_InternalRefclkEnable (OSPI_Type * *pOSPI*)

OSPI Enable internal reference clock.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_InternalRefclkSource (OSPI_Type * *pOSPI*, OSPI_ClockMuxType *u8Value*)

OSPI Select Internal Reference Clock Source.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	Internal Reference Clock Source.

LOCAL_INLINE void OSPI_HWA_LockLut (OSPI_Type * *pOSPI*)

Lock the LUT.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_ModuleDisable (OSPI_Type * *pOSPI*)

OSPI Module Disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_ModuleEnable (OSPI_Type * *pOSPI*)

OSPI Module Enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_ObeTimRelaxDisable (OSPI_Type * *pOSPI*)

OSPI OBE timing relax disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_ObeTimRelaxEnable (OSPI_Type * *pOSPI*)

OSPI OBE timing relax enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_PadIbeDisable (OSPI_Type * *pOSPI*)

OSPI PAD IBE Disable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_PadIbeEnable (OSPI_Type * *pOSPI*)

OSPI PAD IBE Enable.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE uint32_t OSPI_HWA_ReadLutKey (OSPI_Type * *pOSPI*)

Read LUT Key register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

Returns

LUT Key Value.

LOCAL_INLINE uint32_t OSPI_HWA_ReadRcvFifoReg (OSPI_Type * *pOSPI*, uint8_t *u8Idx*)

Read receive fifo data register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Idx</i>	Fifo index to be read.

Returns

Receive fifo data register.

LOCAL_INLINE uint32_t OSPI_HWA_ReadSecondaryRcvFifoReg (OSPI_Type * *pOSPI*)

Read secondary receive fifo data register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

Returns

Secondary receive fifo data register.

LOCAL_INLINE void OSPI_HWA_SelectRefClkEdge (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Selects reference clock edge for valid sampling phase.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	the value of reference clock edge.

LOCAL_INLINE void OSPI_HWA_SetCmdId (OSPI_Type * *pOSPI*, uint16_t *u16Value*)

Set CMD ID.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	Pointers of current CMD stored in LUT registers.

LOCAL_INLINE void OSPI_HWA_SetCmdIdSize (OSPI_Type * *pOSPI*, uint8_t *u8SeqId*, uint16_t *u16Size*)

Set CMD id and size.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8SeqId</i>	Pointers of current CMD stored in LUT registers.
<i>u16Size</i>	Transfer sizes of current CMD.

LOCAL_INLINE void OSPI_HWA_SetCmdSize (OSPI_Type * *pOSPI*, uint16_t *u16Value*)

Set CMD SIZE.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u16Value</i>	Transfer sizes of current CMD.

LOCAL_INLINE void OSPI_HWA_SetColAddrSpace (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set Column Address Space.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	the value of Column Address Space.

LOCAL_INLINE void OSPI_HWA_SetCsHoldTime (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set CS Hold Time.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	CS Hold Time.

LOCAL_INLINE void OSPI_HWA_SetCsSetupTime (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set CS Setup Time.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	CS Setup Time.

LOCAL_INLINE void OSPI_HWA_SetCtrlValue (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Set OSPI CTRL value, users should write the whole value to this register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u32Value</i>	the value which will be written to the CTRL register.

LOCAL_INLINE void OSPI_HWA_SetDataHoldTime (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set Data Hold Time, This bit should only be valid in DDR mode.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	Data Hold Time.

LOCAL_INLINE void OSPI_HWA_SetFlashAddr (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Set flash address, users should write the whole value to this register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u32Value</i>	the value which will be written to the FLS_AR register.

LOCAL_INLINE void OSPI_HWA_SetFlashAddrMode (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set flash addressable mode.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	the value of addressable mode.

LOCAL_INLINE void OSPI_HWA_SetFlashTopAddr (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Set flash top address register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>Top</i>	Address of Flash Device.

LOCAL_INLINE void OSPI_HWA_SetIntDmasEnableReg (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Set the OSPI IND_EN register value for enable or disable interrupts&DMA.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u32Value</i>	the value write to the register.

LOCAL_INLINE void OSPI_HWA_SetRxDMA (OSPI_Type * *pOSPI*, uint8_t *u8Enable*)

Set the OSPI Rx DMA mode.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Enable</i>	open or close rx dma mode.

LOCAL_INLINE void OSPI_HWA_SetRxFifoWaterMark (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set RX FIFO Watermark, Indicates how many valid entries will trigger a readout action.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	the value of RX FIFO Watermark.

LOCAL_INLINE void OSPI_HWA_SetSampleDlyClkCycle (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set Sample delay clock cycle.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	the value of Sample Delay clock cycle.

LOCAL_INLINE void OSPI_HWA_SetSocCfgValue (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Write the OSPI SOC CFG register.

Parameters

<i>pOSPI</i>	pOSPI the base address of the OSPI.
<i>u32Value</i>	the value to be written.

LOCAL_INLINE void OSPI_HWA_SetTxDMA (OSPI_Type * *pOSPI*, uint8_t *u8Enable*)

Set the OSPI Tx DMA mode.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

<i>u8Enable</i>	open or close tx dma mode.
-----------------	----------------------------

LOCAL_INLINE void OSPI_HWA_SetTxFifoWaterMark (OSPI_Type * *pOSPI*, uint8_t *u8Value*)

Set TX FIFO Watermark, Indicates how many valid entries will trigger a transmit action.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	the value of TX FIFO Watermark.

LOCAL_INLINE void OSPI_HWA_UnlockLut (OSPI_Type * *pOSPI*)

Unlock the LUT.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
--------------	-------------------------------

LOCAL_INLINE void OSPI_HWA_WriteLut (OSPI_Type * *pOSPI*, uint8_t *u8Idx*, uint32_t *u32Value*)

Write lookup table register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Idx</i>	Index of LUT.
<i>u32Value</i>	LUT Value to be writed.

LOCAL_INLINE void OSPI_HWA_WriteLutKey (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Write LUT Key register.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u32Value</i>	LUT Key Value to be writed.

LOCAL_INLINE void OSPI_HWA_WriteTxData (OSPI_Type * *pOSPI*, uint32_t *u32Value*)

Writing this register will trigger data entry of TX FIFO.

Parameters

<i>pOSPI</i>	the base address of the OSPI.
<i>u8Value</i>	the value of tx data.

HwA_overlay

Collaboration diagram for HwA_overlay:Macros

```
#define OVERLYA_REGION_CNT 3U
```

Overlay Region Count.

```
#define OVERLAY_FAR_SRC 0x08000000U
```

FAR source address.

```
#define OVERLAY_FAR_SIZE_MAX 0x2000000U
```

FAR max size.

```
#define OVERLAY_FAR_SIZE_ALIGN 0x10000U
```

FAR size align.

```
#define OVERLAY_FAR_SIZE_MASK 0xFFFFU
```

FAR size max mask.

Enumerations

```
enum OVERLAY_FARSizeType { OVERLAY_FARUINTSIZE_64KB = 0x10000 }
```

FAR Size.

Functions

LOCAL_INLINE void **OVERLAY_HWA_OverlayEnable** (void)

enable overlay global switch

LOCAL_INLINE void **OVERLAY_HWA_OverlayDisable** (void)

disable overlay global switch

LOCAL_INLINE void **OVERLAY_HWA_FAREnable** (void)

enable far global switch

LOCAL_INLINE void **OVERLAY_HWA_FARDisable** (void)

disable far global switch

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion0Enable** (void)

enable overlay region 0

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion0Disable** (void)

disable overlay region 0

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion0Src** (uint32_t u32Src)

Set Overlay Region 0 Source.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion0Dst** (uint32_t u32Dst)

Set Overlay Region 0 Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion0Size** (OVERLAY_OverlaySizeType eSize)

Set Overlay Region 0 Size.

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion1Enable** (void)

enable overlay region 1

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion1Disable** (void)

disable overlay region 1

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion1Src** (uint32_t u32Src)

Set Overlay Region 1 Source.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion1Dst** (uint32_t u32Dst)

Set Overlay Region 1 Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion1Size** (OVERLAY_OverlaySizeType eSize)

Set Overlay Region 1 Size.

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion2Enable** (void)

enable overlay region 2

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion2Disable** (void)

disable overlay region 2

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion2Src** (uint32_t u32Src)

Set Overlay Region 2 Source.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion2Dst** (uint32_t u32Dst)

Set Overlay Region 2 Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion2Size** (OVERLAY_OverlaySizeType eSize)

Set Overlay Region 2 Size.

LOCAL_INLINE void **OVERLAY_HWA_SetFarDst** (uint32_t u32Dst)

Set Far Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetFarSize** (uint32_t u32Size)

Set Far Size.

LOCAL_INLINE void **OVERLAY_HWA_ErrorInterruptEnable** (void)

enable error interrupt

LOCAL_INLINE void **OVERLAY_HWA_ErrorInterruptDisable** (void)

disable error interrupt

LOCAL_INLINE uint32 **OVERLAY_HWA_GetErrorFlag** (void)

get error flag

LOCAL_INLINE uint32 **OVERLAY_HWA_ClrErrorFlag** (uint32_t u32ClrBits)

clear error flag

Detailed Description

Macro Definition Documentation

#define OVERLAY_FAR_SIZE_ALIGN 0x10000U

FAR size align.

#define OVERLAY_FAR_SIZE_MASK 0xFFFFU

FAR size max mask.

#define OVERLAY_FAR_SIZE_MAX 0x2000000U

FAR max size.

#define OVERLAY_FAR_SRC 0x08000000U

FAR source address.

```
#define OVERLYA_REGION_CNT 3U
```

Overlay Region Count.

Enumeration Type Documentation

```
enum OVERLAY_FARSizeType
```

FAR Size.

Enumerator:

OVERLAY_FARU INTSIZE_64KB	OVERLAY_FARUINTSIZE_64KB, remapping far uint size to 64KB
------------------------------	---

Function Documentation

```
LOCAL_INLINE uint32 OVERLAY_HWA_ClrErrorFlag (uint32_t  u32ClrBits)
```

clear error flag

```
LOCAL_INLINE void OVERLAY_HWA_ErrorInterruptDisable (void )
```

disable error interrupt

```
LOCAL_INLINE void OVERLAY_HWA_ErrorInterruptEnable (void )
```

enable error interrupt

LOCAL_INLINE void OVERLAY_HWA_FARDisable (void)

disable far global switch

LOCAL_INLINE void OVERLAY_HWA_FAREnable (void)

enable far global switch

LOCAL_INLINE uint32 OVERLAY_HWA_GetErrorFlag (void)

get error flag

LOCAL_INLINE void OVERLAY_HWA_OverlayDisable (void)

disable overlay global switch

LOCAL_INLINE void OVERLAY_HWA_OverlayEnable (void)

enable overlay global switch

LOCAL_INLINE void OVERLAY_HWA_OverlayRegion0Disable (void)

disable overlay region 0

LOCAL_INLINE void OVERLAY_HWA_OverlayRegion0Enable (void)

enable overlay region 0

LOCAL_INLINE void OVERLAY_HWA_OverlayRegion1Disable (void)

disable overlay region 1

LOCAL_INLINE void OVERLAY_HWA_OverlayRegion1Enable (void)

enable overlay region 1

LOCAL_INLINE void OVERLAY_HWA_OverlayRegion2Disable (void)

disable overlay region 2

LOCAL_INLINE void OVERLAY_HWA_OverlayRegion2Enable (void)

enable overlay region 2

LOCAL_INLINE void OVERLAY_HWA_SetFarDst (uint32_t *u32Dst*)

Set Far Destination.

Parameters

<i>u32Dst</i>	Destination address, must pflash
---------------	----------------------------------

LOCAL_INLINE void OVERLAY_HWA_SetFarSize (uint32_t *u32Size*)

Set Far Size.

Parameters

<i>u32Size</i>	size=(u32Size+1)*64KB
----------------	-----------------------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion0Dst (uint32_t *u32Dst*)

Set Overlay Region 0 Destination.

Parameters

<i>u32Dst</i>	Destination address
---------------	---------------------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion0Size (OVERLAY_OverlaySizeType *eSize*)

Set Overlay Region 0 Size.

Parameters

<i>eSize</i>	size
--------------	------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion0Src (uint32_t *u32Src*)

Set Overlay Region 0 Source.

Parameters

<i>u32Src</i>	Source address
---------------	----------------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion1Dst (uint32_t *u32Dst*)

Set Overlay Region 1 Destination.

Parameters

<i>u32Dst</i>	Destination address
---------------	---------------------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion1Size (OVERLAY_OverlaySizeType *eSize*)

Set Overlay Region 1 Size.

Parameters

<i>eSize</i>	size
--------------	------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion1Src (uint32_t *u32Src*)

Set Overlay Region 1 Source.

Parameters

<i>u32Src</i>	Source address
---------------	----------------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion2Dst (uint32_t *u32Dst*)

Set Overlay Region 2 Destination.

Parameters

<i>u32Dst</i>	Destination address
---------------	---------------------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion2Size (OVERLAY_OverlaySizeType *eSize*)

Set Overlay Region 2 Size.

Parameters

<i>eSize</i>	size
--------------	------

LOCAL_INLINE void OVERLAY_HWA_SetOverlayRegion2Src (uint32_t *u32Src*)

Set Overlay Region 2 Source.

Parameters

<i>u32Src</i>	Source address
---------------	----------------

HwA_pmc

Collaboration diagram for HwA_pmc:Functions

LOCAL_INLINE uint32_t **PMC_HWA_GetLVSCR** (void)

get PMC LVCSR register

LOCAL_INLINE void **PMC_HWA_SetLVSCR** (uint32_t u32Val)

set PMC LVCSR register.

LOCAL_INLINE uint32_t **PMC_HWA_GetCONFIG** (void)

get PMC CONFIG register

LOCAL_INLINE void **PMC_HWA_SetCONFIG** (uint32_t u32Val)

set PMC CONFIG register.

LOCAL_INLINE void **PMC_HWA_DisableInterrupt** (uint32_t u32Val)

HWA Disable PMC interrupt.

LOCAL_INLINE void **Pmc_HWA_EnableInterrupt** (uint32_t u32Val)

HWA Enable PMC interrupt.

LOCAL_INLINE bool **PMC_ReadLVDInterruptFlag** (void)

Return Low Voltage Detect Interrupt Enable Flag.

LOCAL_INLINE bool **PMC_ReadHVDInterruptFlag** (void)

Return High Voltage Detect Interrupt Enable Flag.

Detailed Description

Function Documentation

LOCAL_INLINE void PMC_HWA_DisableInterrupt (uint32_t *u32Val*)

HWA Disable PMC interrupt.

Parameters

<i>the</i>	parameter of interrupt flag
------------	-----------------------------

LOCAL_INLINE void Pmc_HWA_EnableInterrupt (uint32_t *u32Val*)

HWA Enable PMC interrupt.

Parameters

<i>the</i>	parameter of interrupt flag
------------	-----------------------------

LOCAL_INLINE uint32_t PMC_HWA_GetCONFIG (void)

get PMC CONFIG register

Returns

uint32_t CONFIG register value

LOCAL_INLINE uint32_t PMC_HWA_GetLVCSR (void)

get PMC LVCSR register

Returns

uint32_t LVCSR register value

LOCAL_INLINE void PMC_HWA_SetCONFIG (uint32_t *u32Val*)

set PMC CONFIG register.

This function configures the PMC CONFIG registe.

Parameters

<i>u32LVSCRValue</i>	Set PMC CONFIG register value.
----------------------	--------------------------------

LOCAL_INLINE void PMC_HWA_SetLVSCR (uint32_t *u32Val*)

set PMC LVCSR register.

This function configures the PMC LVCSR registe.

Parameters

<i>u32Val</i>	Set PMC LVCSR register value.
---------------	-------------------------------

LOCAL_INLINE bool PMC_ReadHVDInterruptFlag (void)

Return High Voltage Detect Interrupt Enable Flag.

Returns

0: High Voltage Detect Interrupt Disabled; 1: High Voltage Detect Interrupt Enabled

LOCAL_INLINE bool PMC_ReadLVDInterruptFlag (void)

Return Low Voltage Detect Interrupt Enable Flag.

Returns

0: Low Voltage Detect Interrupt Disabled; 1: Low Voltage Detect Interrupt Enabled

HwA_port

Collaboration diagram for HwA_port:Functions

LOCAL_INLINE void **PORT_HWA_ConfigPin** (PORT_Type *pPort, uint8_t u8Pin, uint32_t u32PcrReg)
Configure pin.

LOCAL_INLINE uint32_t **PORT_HWA_ReadPinInterruptFlag** (PORT_Type *pPort, uint8_t u8Pin)

Read pin interrupt flag.

LOCAL_INLINE void **PORT_HWA_SetPinMux** (PORT_Type *pPort, uint8_t u8Pin, Port_PinMuxType u32PinMux)

Set pin multiplexing.

LOCAL_INLINE void **PORT_HWA_SetPinEmgcyStop** (PORT_Type *pPort, uint8_t u8Pin, bool bEnable)

Set Pin Emergency Stop.

LOCAL_INLINE void **PORT_HWA_SetPinInterruptMode** (PORT_Type *pPort, uint8_t u8Pin, uint32_t u32PinIrqc)

Set pin interrupt mode.

LOCAL_INLINE void **PORT_HWA_SetPinPullEnable** (PORT_Type *pPort, uint8_t u8Pin, uint8_t u8PeValue)

Set pin pull enable.

LOCAL_INLINE void **PORT_HWA_SetPinPullMode** (PORT_Type *pPort, uint8_t u8Pin, uint8_t u8PsValue)

Set pin pull mode.

LOCAL_INLINE void **PORT_HWA_SetPinDriveStrength** (PORT_Type *pPort, uint8_t u8Pin)

Set pin drive strength.

LOCAL_INLINE void **PORT_HWA_SetPinPassiveFilterEnable** (PORT_Type *pPort, uint8_t u8Pin, uint8_t u8PfeValue)

Set pin passive filter enable.

LOCAL_INLINE void **PORT_HWA_ConfigDigitalFilter** (PORT_Type *pPort, uint32_t u32RegValue)

Configure port digital filter.

LOCAL_INLINE void **PORT_HWA_SetDigitalFilterEnable** (PORT_Type *pPort, uint8_t u8RegBit)

Set port digital filter enable.

LOCAL_INLINE void **PORT_HWA_SetDigitalFilterClkSrc** (PORT_Type *pPort)

Set port digital filter clock source.

LOCAL_INLINE void **PORT_HWA_ConfigDigitalFilterWidth** (PORT_Type *pPort, uint32_t u32FilterWidth)

Configure digital filter width.

LOCAL_INLINE void **PORT_HWA_ClearPinInterruptMode** (PORT_Type *pPort, uint8_t u8Pin)

Clear pin interrupt mode.

LOCAL_INLINE void **PORT_HWA_ClearPinInterruptFlag** (PORT_Type *pPort, uint8_t u8Pin)

Clear pin interrupt flag.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterEnable** (PORT_Type *pPort)

Clear port digital filter enable.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterPin** (PORT_Type *pPort, uint8_t u8RegBit)

Clear port digital filter enable for specific pin.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterClkSrc** (PORT_Type *pPort)

Clear port digital filter clock source.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterWidth** (PORT_Type *pPort)

Clear port digital filter width.

Detailed Description

Function Documentation

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterClkSrc** (PORT_Type * *pPort*)

Clear port digital filter clock source.

Parameters

<i>pPort</i>	Port instance
--------------	---------------

LOCAL_INLINE void PORT_HWA_ClearDigitalFilterEnable (PORT_Type * *pPort*)

Clear port digital filter enable.

Parameters

<i>pPort</i>	Port instance
--------------	---------------

LOCAL_INLINE void PORT_HWA_ClearDigitalFilterPin (PORT_Type * *pPort*, uint8_t *u8RegBit*)

Clear port digital filter enable for specific pin.

Parameters

<i>pPort</i>	Port instance
<i>u8RegBit</i>	DFER register bit

LOCAL_INLINE void PORT_HWA_ClearDigitalFilterWidth (PORT_Type * *pPort*)

Clear port digital filter width.

Parameters

<i>pPort</i>	Port instance
--------------	---------------

LOCAL_INLINE void PORT_HWA_ClearPinInterruptFlag (PORT_Type * *pPort*, uint8_t *u8Pin*)

Clear pin interrupt flag.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number

LOCAL_INLINE void PORT_HWA_ClearPinInterruptMode (PORT_Type * *pPort*, uint8_t *u8Pin*)

Clear pin interrupt mode.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number

LOCAL_INLINE void PORT_HWA_ConfigDigitalFilter (PORT_Type * *pPort*, uint32_t *u32RegValue*)

Configure port digital filter.

Parameters

<i>pPort</i>	Port instance
<i>u32RegValue</i>	Enable bit,0-31 bit indicate pin 0-31

LOCAL_INLINE void PORT_HWA_ConfigDigitalFilterWidth (PORT_Type * *pPort*, uint32_t *u32FilterWidth*)

Configure digital filter width.

Parameters

<i>pPort</i>	Port instance
<i>u32FilterWidth</i>	Digital filter length value,range:0-31

LOCAL_INLINE void PORT_HWA_ConfigPin (PORT_Type * *pPort*, uint8_t *u8Pin*, uint32_t *u32PcrReg*)

Configure pin.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number
<i>u32PcrReg</i>	Pin PCR register value

LOCAL_INLINE uint32_t PORT_HWA_ReadPinInterruptFlag (PORT_Type * *pPort*, uint8_t *u8Pin*)

Read pin interrupt flag.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number

Returns

interrupt flag value

LOCAL_INLINE void PORT_HWA_SetDigitalFilterClkSrc (PORT_Type * *pPort*)

Set port digital filter clock source.

Parameters

<i>pPort</i>	Port instance
<i>eClkSrc</i>	Digital filter clock source

LOCAL_INLINE void PORT_HWA_SetDigitalFilterEnable (PORT_Type * *pPort*, uint8_t *u8RegBit*)

Set port digital filter enable.

Parameters

<i>pPort</i>	Port instance
<i>u8RegBit</i>	DFER register bit

LOCAL_INLINE void PORT_HWA_SetPinDriveStrength (PORT_Type * *pPort*, uint8_t *u8Pin*)

Set pin drive strength.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number

Note

only hs PAD use this bit

LOCAL_INLINE void PORT_HWA_SetPinEmgcyStop (PORT_Type * *pPort*, uint8_t *u8Pin*, bool *bEnable*)

Set Pin Emergency Stop.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number
<i>bEnable</i>	Enable or Disable Emergency Stop of this Pin

LOCAL_INLINE void PORT_HWA_SetPinInterruptMode (PORT_Type * *pPort*, uint8_t *u8Pin*, uint32_t *u32PinIrqc*)

Set pin interrupt mode.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number
<i>u32PinIrqc</i>	Pin IRQC configuration value

LOCAL_INLINE void PORT_HWA_SetPinMux (PORT_Type * *pPort*, uint8_t *u8Pin*, Port_PinMuxType *u32PinMux*)

Set pin multiplexing.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number
<i>Port_PinMuxType</i> <i>pe</i>	Pin MXU configuration value

LOCAL_INLINE void PORT_HWA_SetPinPassiveFilterEnable (PORT_Type * *pPort*, uint8_t *u8Pin*, uint8_t *u8PfeValue*)

Set pin passive filter enable.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number
<i>u8PfeValue</i>	Pin PFE configuration value(1/0)

LOCAL_INLINE void PORT_HWA_SetPinPullEnable (PORT_Type * *pPort*, uint8_t *u8Pin*, uint8_t *u8PeValue*)

Set pin pull enable.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number
<i>u8PeValue</i>	Pin PE configuration value(1/0)

LOCAL_INLINE void PORT_HWA_SetPinPullMode (PORT_Type * *pPort*, uint8_t *u8Pin*, uint8_t *u8PsValue*)

Set pin pull mode.

Parameters

<i>pPort</i>	Port instance
<i>u8Pin</i>	Pin number
<i>u8PsValue</i>	Pin PS configuration value(1/0)

HwA_ptimer

Collaboration diagram for HwA_ptimer:Enumerations

```
enum PTIMER_LoadValueModeType { PTIMER_LOAD_VAL_IMMEDIATELY = 0U,  
PTIMER_LOAD_VAL_AT_MODULO_COUNTER = 1U, PTIMER_LOAD_VAL_AT_NEXT_TRIGGER = 2U,  
PTIMER_LOAD_VAL_AT_MODULO_COUNTER_OR_NEXT_TRIGGER = 3U }
```

Ptimer value load mode.

```
enum PTIMER_ClockPreDividerType { PTIMER_PRE_DIVIDE_BY_1 = 0U, PTIMER_PRE_DIVIDE_BY_2 = 1U,  
PTIMER_PRE_DIVIDE_BY_4 = 2U, PTIMER_PRE_DIVIDE_BY_8 = 3U, PTIMER_PRE_DIVIDE_BY_16 = 4U,  
PTIMER_PRE_DIVIDE_BY_32 = 5U, PTIMER_PRE_DIVIDE_BY_64 = 6U, PTIMER_PRE_DIVIDE_BY_128 = 7U }
```

Ptimer clock pre-divider factor.

```
enum PTIMER_ClockPreDivMultiplyFactorType { PTIMER_PRE_DIVIDER_MULTIPLY_BY_1 = 0U,  
PTIMER_PRE_DIVIDER_MULTIPLY_BY_10 = 1U, PTIMER_PRE_DIVIDER_MULTIPLY_BY_20 = 2U,  
PTIMER_PRE_DIVIDER_MULTIPLY_BY_40 = 3U }
```

Ptimer clock divider multiplication factor.

```
enum PTIMER_TrgSrcType { PTIMER_TRGSRC_TRGSEL = 0x00U, PTIMER_TRGSRC_SW = 0x0FU }
```

Ptimer trigger source.

Functions

```
LOCAL_INLINE uint32_t PTIMER_HWA_GetStatusCtrl (const PTIMER_Type *const pPtimer)
```

Get the STATUS_CTRL register value.

```
LOCAL_INLINE void PTIMER_HWA_SetStatusCtrl (PTIMER_Type *const pPtimer, uint32_t u32CfgValue)
```

Set the STATUS_CTRL register value.

```
LOCAL_INLINE PTIMER_LoadValueModeType PTIMER_HWA_GetLoadMode (const PTIMER_Type *const  
pPtimer)
```

Get the load mode of the Ptimer instance.

```
LOCAL_INLINE void PTIMER_HWA_SetLoadMode (PTIMER_Type *const pPtimer,  
PTIMER_LoadValueModeType eLoadMode)
```

Set the load mode of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetSeqErrIntEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether sequence error interrupt is enabled.

LOCAL_INLINE void **PTIMER_HWA_SetSeqErrIntEnableFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable sequence error interrupt.

LOCAL_INLINE void **PTIMER_HWA_GenerateSwTrigger** (PTIMER_Type *const pPtimer)

Generate software trigger signal for Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetDMAEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether DMA is enabled for the Ptimer insstance.

LOCAL_INLINE void **PTIMER_HWA_SetDMAEnableFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable Ptimer DMA.

LOCAL_INLINE **PTIMER_ClockPreDividerType** **PTIMER_HWA_GetDivPrescaler** (const PTIMER_Type *const pPtimer)

Get the predivider value of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetDivPrescaler** (PTIMER_Type *const pPtimer, **PTIMER_ClockPreDividerType** eDivPrescaler)

Set the predivider value of the Ptimer instance.

LOCAL_INLINE **PTIMER_TrgSrcType** **PTIMER_HWA_GetTriggerSource** (const PTIMER_Type *const pPtimer)

Get the trigger source of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetTriggerSource** (PTIMER_Type *const pPtimer, **PTIMER_TrgSrcType** eTriggerSource)

Set the trigger source of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether the Ptimer instance is enabled.

LOCAL_INLINE void **PTIMER_HWA_Enable** (PTIMER_Type *const pPtimer)

Enable the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_Disable** (PTIMER_Type *const pPtimer)

Disable the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetInterruptFlag** (const PTIMER_Type *const pPtimer)

Get the delay interrupt flag of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_ClearInterruptFlag** (PTIMER_Type *const pPtimer)

Clear the delay interrupt flag of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetInterruptEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether delay interrupt is enabled for the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetInterruptEnableFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable delay interrupt for the Ptimer instance.

LOCAL_INLINE **PTIMER_ClockPreDivMultiplyFactorType** **PTIMER_HWA_GetDivMultiply** (const PTIMER_Type *const pPtimer)

Get the multiply factor of the predivider of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetDivMultiply** (PTIMER_Type *const pPtimer, **PTIMER_ClockPreDivMultiplyFactorType** eMultFactor)

Set the multiply factor of the predivider of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetContinuousModeFlag** (const PTIMER_Type *const pPtimer)

Get whether continuous mode is enabled for the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetContinuousModeFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable continuous mode for the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetValueLoadStatus** (const PTIMER_Type *const pPtimer)

Get the config value loading status.

LOCAL_INLINE void **PTIMER_HWA_LoadValue** (PTIMER_Type *const pPtimer)

Load the buffered values into register.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetMaxCount** (const PTIMER_Type *const pPtimer)

Get the Ptimer max counter period When the Ptimer counter reaches the period, it will return to zero.

LOCAL_INLINE void **PTIMER_HWA_SetMaxCount** (PTIMER_Type *const pPtimer, uint16_t u16MaxCnt)

Set the Ptimer max counter period When the Ptimer counter reaches the period, it will return to zero.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetCounterValue** (const PTIMER_Type *const pPtimer)

Get the Ptimer current count value.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetInterruptDelay** (const PTIMER_Type *const pPtimer)

Get the ptimer interrupt period.

LOCAL_INLINE void **PTIMER_HWA_SetInterruptDelay** (PTIMER_Type *const pPtimer, uint16_t u16InterruptDelay)

Set the ptimer interrupt period.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelBackToBackFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get whether back to back trigger is enabled for the Ptimer channel.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelPretriggerOutputFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get whether pre-trigger output is enabled for the Ptimer channel.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelPretriggerEnableFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get whether pre-trigger is enabled for the Ptimer channel.

LOCAL_INLINE void **PTIMER_HWA_SetChannelControl** (PTIMER_Type *const pPtimer, uint8_t u8Channel, bool bEnablePretrigger, bool bEnablePretriggerOutput, bool bEnableBackToBack)

Set the Ptimer channel control flags.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelCounterFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get whether the Ptimer counter matches the channel counter.

LOCAL_INLINE void **PTIMER_HWA_ClearChannelCounterFlag** (PTIMER_Type *const pPtimer, uint8_t u8Channel)

Clear the Ptimer channel counter match flag.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelSequenceErrorFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get the sequence error status of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_ClearChannelSequenceErrorFlag** (PTIMER_Type *const pPtimer, uint8_t u8Channel)

Clear the sequence error flag of the selected Ptimer channel.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetChannelDelay** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get the channel delay value.

LOCAL_INLINE void **PTIMER_HWA_SetChannelDelay** (PTIMER_Type *const pPtimer, uint8_t u8Channel, uint16_t u16Delay)

Set the channel delay value.

LOCAL_INLINE bool **PTIMER_HWA_GetPulseOutEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether pulse-out is enabled.

LOCAL_INLINE void **PTIMER_HWA_EnablePulseOut** (PTIMER_Type *const pPtimer)

Enable pulse-out for the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_DisablePulseOut** (PTIMER_Type *const pPtimer)

Disable pulse-out for the Ptimer instance.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetPulseOutDelayHigh** (const PTIMER_Type *const pPtimer)
Get the delay high value for the pulse-out function When the Ptimer counter reach the delay high value, the pulse output goes high.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetPulseOutDelayLow** (const PTIMER_Type *const pPtimer)
Get the delay low value for the pulse-out function When the Ptimer counter reach the delay low value, the pulse output goes low.

LOCAL_INLINE void **PTIMER_HWA_SetPulseOutDelay** (PTIMER_Type *const pPtimer, uint16_t u16DelayHigh, uint16_t u16DelayLow)
Set the pulse out delay value When the Ptimer counter reach the delay high value, the pulse output goes high When the Ptimer counter reach the delay low value, the pulse output goes low The delay high value can be either greater or less than the delay low value.

Detailed Description

Enumeration Type Documentation

enum PTIMER_ClockPreDividerType

Ptimer clock pre-divider factor.

The Ptimer clock source is from core clock and the divider is a multiplication of PTIMER_ClockPreDividerType and PTIMER_ClockPreDivMultiplyFactorType, and thus: Freq = Core_Freq / (PTIMER_ClockPreDividerType * PTIMER_ClockPreDivMultiplyFactorType)

Enumerator:

PTIMER_PRE_DIVIDE_BY_1	
PTIMER_PRE_DIVIDE_BY_2	

PTIMER_PRE_DI VIDE_BY_4	
PTIMER_PRE_DI VIDE_BY_8	
PTIMER_PRE_DI VIDE_BY_16	
PTIMER_PRE_DI VIDE_BY_32	
PTIMER_PRE_DI VIDE_BY_64	
PTIMER_PRE_DI VIDE_BY_128	

enum PTIMER_ClockPreDivMultiplyFactorType

Ptimer clock divider multiplication factor.

The Ptimer clock source is from core clock and the divider is a multiplication of PTIMER_ClockPreDividerType and PTIMER_ClockPreDivMultiplyFactorType, and thus: $\text{Freq} = \text{Core_Freq} / (\text{PTIMER_ClockPreDividerType} * \text{PTIMER_ClockPreDivMultiplyFactorType})$

Enumerator:

PTIMER_PRE_DI VIDER_MULTIPL Y_BY_1	
PTIMER_PRE_DI VIDER_MULTIPL Y_BY_10	
PTIMER_PRE_DI VIDER_MULTIPL Y_BY_20	
PTIMER_PRE_DI VIDER_MULTIPL Y_BY_40	

enum PTIMER_LoadValueModeType

Ptimer value load mode.

Some Ptimer registers are buffered and will only take effect after called **PTIMER_LoadValue()** function, and this option selects when the buffered configurations will tack effect after **PTIMER_LoadValue()** is called.

Enumerator:

PTIMER_LOAD_VAL_IMMEDIATELY	Loaded immediately after load operation.
PTIMER_LOAD_VAL_AT_MODULE_COUNTER	Loaded when counter hits the max count after load operation.
PTIMER_LOAD_VAL_AT_NEXT_TRIGGER	Loaded when detecting an input trigger after load operation.
PTIMER_LOAD_VAL_AT_MODULE_COUNTER_OR_NEXT_TRIGGER	Loaded when counter hits the max count or detecting an input trigger after load operation.

enum PTIMER_TrgSrcType

Ptimer trigger source.

Enumerator:

PTIMER_TRGSR_C_TRGSEL	Ptimer trigger source from TrgSel
PTIMER_TRGSR_C_SW	Ptimer trigger source from software trigger

Function Documentation

LOCAL_INLINE void PTIMER_HWA_ClearChannelCounterFlag (PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*)

Clear the Ptimer channel counter match flag.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel

LOCAL_INLINE void PTIMER_HWA_ClearChannelSequenceErrorFlag (PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*)

Clear the sequence error flag of the selected Ptimer channel.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel to clear the sequence error flag

LOCAL_INLINE void PTIMER_HWA_ClearInterruptFlag (PTIMER_Type *const *pPtimer*)

Clear the delay interrupt flag of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE void PTIMER_HWA_Disable (PTIMER_Type *const *pPtimer*)

Disable the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE void PTIMER_HWA_DisablePulseOut (PTIMER_Type *const *pPtimer*)

Disable pulse-out for the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE void PTIMER_HWA_Enable (PTIMER_Type *const *pPtimer*)

Enable the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE void PTIMER_HWA_EnablePulseOut (PTIMER_Type *const *pPtimer*)

Enable pulse-out for the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE void PTIMER_HWA_GenerateSwTrigger (PTIMER_Type *const *pPtimer*)

Generate software trigger signal for Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

**LOCAL_INLINE bool PTIMER_HWA_GetChannelBackToBackFlag (const PTIMER_Type *const *pPtimer*,
uint8_t *u8Channel*)**

Get whether back to back trigger is enabled for the Ptimer channel.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel

Returns

true back to back trigger is enabled for the Ptimer channel

false back to back trigger is disabled for the Ptimer channel

LOCAL_INLINE bool PTIMER_HWA_GetChannelCounterFlag (const PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*)

Get whether the Ptimer counter matches the channel counter.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel

Returns

true the Ptimer counter matches the channel counter

false the Ptimer counter has not reached the channel counter

LOCAL_INLINE uint16_t PTIMER_HWA_GetChannelDelay (const PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*)

Get the channel delay value.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel

Returns

uint16_t the channel delay value

LOCAL_INLINE bool PTIMER_HWA_GetChannelPretriggerEnableFlag (const PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*)

Get whether pre-trigger is enabled for the Ptimer channel.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel

Returns

true pre-trigger is enabled for the Ptimer channel

false pre-trigger is disabled for the Ptimer channel

LOCAL_INLINE bool PTIMER_HWA_GetChannelPretriggerOutputFlag (const PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*)

Get whether pre-trigger output is enabled for the Ptimer channel.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel

Returns

true pre-trigger output is enabled for the Ptimer channel

false pre-trigger output is disabled for the Ptimer channel

LOCAL_INLINE bool PTIMER_HWA_GetChannelSequenceErrorFlag (const PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*)

Get the sequence error status of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the channel related to the sequence error

Returns

true the selected channel has sequence error

false the selected channel does not have sequence error

LOCAL_INLINE bool PTIMER_HWA_GetContinuousModeFlag (const PTIMER_Type *const *pPtimer*)

Get whether continuous mode is enabled for the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

true continuous mode is enabled for the Ptimer instance

false continuous mode is disabled for the Ptimer instance

LOCAL_INLINE uint16_t PTIMER_HWA_GetCounterValue (const PTIMER_Type *const *pPtimer*)

Get the Ptimer current count value.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

uint16_t the Ptimer current count value

LOCAL_INLINE PTIMER_ClockPreDivMultiplyFactorType PTIMER_HWA_GetDivMultiply (const PTIMER_Type *const *pPtimer*)

Get the multiply factor of the predivider of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

PTIMER_ClockPreDivMultiplyFactorType the multiply factor of the Ptimer instance

LOCAL_INLINE PTIMER_ClockPreDividerType PTIMER_HWA_GetDivPrescaler (const PTIMER_Type *const *pPtimer*)

Get the predivider value of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

PTIMER_ClockPreDividerType the predivider of the Ptimer instance

LOCAL_INLINE bool PTIMER_HWA_GetDMAEnableFlag (const PTIMER_Type *const *pPtimer*)

Get whether DMA is enabled for the Ptimer insstance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

true DMA is enabled

false DMA is disabled

LOCAL_INLINE bool PTIMER_HWA_GetEnableFlag (const PTIMER_Type *const *pPtimer*)

Get whether the Ptimer instance is enabled.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

true the Ptimer instance is enabled

false the Ptimer instance is disabled

LOCAL_INLINE uint16_t PTIMER_HWA_GetInterruptDelay (const PTIMER_Type *const *pPtimer*)

Get the ptimer interrupt period.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

uint16_t the Ptimer interrupt period

LOCAL_INLINE bool PTIMER_HWA_GetInterruptEnableFlag (const PTIMER_Type *const *pPtimer*)

Get whether delay interrupt is enabled for the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE bool PTIMER_HWA_GetInterruptFlag (const PTIMER_Type *const *pPtimer*)

Get the delay interrupt flag of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

true the delay interrupt flag of the Ptimer instance is generated

false the delay interrupt flag of the Ptimer instance is not generated

LOCAL_INLINE PTIMER_LoadValueModeType PTIMER_HWA_GetLoadMode (const PTIMER_Type *const *pPtimer*)

Get the load mode of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

PTIMER_LoadValueModeType the load mode of the Ptimer instance

LOCAL_INLINE uint16_t PTIMER_HWA_GetMaxCount (const PTIMER_Type *const *pPtimer*)

Get the Ptimer max counter period When the Ptimer counter reaches the period, it will return to zero.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

uint16_t the Ptimer max count

LOCAL_INLINE uint16_t PTIMER_HWA_GetPulseOutDelayHigh (const PTIMER_Type *const *pPtimer*)

Get the delay high value for the pulse-out function When the Ptimer counter reach the delay high value, the pulse output goes high.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE uint16_t PTIMER_HWA_GetPulseOutDelayLow (const PTIMER_Type *const *pPtimer*)

Get the delay low value for the pulse-out function When the Ptimer counter reach the delay low value, the pulse output goes low.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE bool PTIMER_HWA_GetPulseOutEnableFlag (const PTIMER_Type *const *pPtimer*)

Get whether pulse-out is enabled.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

true pulse-out is enabled

false pulse-out is disabled

LOCAL_INLINE bool PTIMER_HWA_GetSeqErrIntEnableFlag (const PTIMER_Type *const *pPtimer*)

Get whether sequence error interrupt is enabled.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

true sequence error interrupt is enabled

false sequence error interrupt is disabled

LOCAL_INLINE uint32_t PTIMER_HWA_GetStatusCtrl (const PTIMER_Type *const *pPtimer*)

Get the STATUS_CTRL register value.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE PTIMER_TrgSrcType PTIMER_HWA_GetTriggerSource (const PTIMER_Type *const *pPtimer*)

Get the trigger source of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

PTIMER_TRGSRG_TRGSEL the trigger source is from TrgSel

PTIMER_TRGSRG_SW the trigger source is from software

LOCAL_INLINE bool PTIMER_HWA_GetValueLoadStatus (const PTIMER_Type *const *pPtimer*)

Get the config value loading status.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

Returns

true the config values are in loading status

false the config values are loaded

LOCAL_INLINE void PTIMER_HWA_LoadValue (PTIMER_Type *const *pPtimer*)

Load the buffered values into register.

Note

Some Ptimer registers are buffered and will only take effect after called this function

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
----------------	---

LOCAL_INLINE void PTIMER_HWA_SetChannelControl (PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*, bool *bEnablePretrigger*, bool *bEnablePretriggerOutput*, bool *bEnableBackToBack*)

Set the Ptimer channel control flags.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel
<i>bEnablePretrigger</i>	whether to enable pre-trigger
<i>bEnablePretriggerOutput</i>	whether to enable pre-trigger output
<i>bEnableBackToBack</i>	whether to enable back to back trigger

LOCAL_INLINE void PTIMER_HWA_SetChannelDelay (PTIMER_Type *const *pPtimer*, uint8_t *u8Channel*, uint16_t *u16Delay*)

Set the channel delay value.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u8Channel</i>	the Ptimer channel
<i>u16Delay</i>	the channel delay value

LOCAL_INLINE void PTIMER_HWA_SetContinuousModeFlag (PTIMER_Type *const *pPtimer*, bool *bEnable*)

Set whether to enable continuous mode for the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>bEnable</i>	whether to enable continuous mode for the Ptimer instance

LOCAL_INLINE void PTIMER_HWA_SetDivMultiply (PTIMER_Type *const *pPtimer*, PTIMER_ClockPreDivMultiplyFactorType *eMultFactor*)

Set the multiply factor of the predivider of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>eMultFactor</i>	the multiply factor of the Ptimer instance

LOCAL_INLINE void PTIMER_HWA_SetDivPrescaler (PTIMER_Type *const *pPtimer*, PTIMER_ClockPreDividerType *eDivPrescaler*)

Set the predivider value of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>eDivPrescaler</i>	the predivider of the Ptimer instance

LOCAL_INLINE void PTIMER_HWA_SetDMAEnableFlag (PTIMER_Type *const *pPtimer*, bool *bEnable*)

Set whether to enable Ptimer DMA.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>bEnable</i>	whether to enable DMA for the Ptimer instance

LOCAL_INLINE void PTIMER_HWA_SetInterruptDelay (PTIMER_Type *const *pPtimer*, uint16_t *u16InterruptDelay*)

Set the ptimer interrupt period.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u16InterruptDelay</i>	the Ptimer interrupt period

LOCAL_INLINE void PTIMER_HWA_SetInterruptEnableFlag (PTIMER_Type *const *pPtimer*, bool *bEnable*)

Set whether to enable delay interrupt for the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>bEnable</i>	whether to enable delay interrupt for the Ptimer instance

LOCAL_INLINE void PTIMER_HWA_SetLoadMode (PTIMER_Type *const *pPtimer*, PTIMER_LoadValueModeType *eLoadMode*)

Set the load mode of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>eLoadMode</i>	the load mode of the Ptimer instance

LOCAL_INLINE void PTIMER_HWA_SetMaxCount (PTIMER_Type *const *pPtimer*, uint16_t *u16MaxCnt*)

Set the Ptimer max counter period When the Ptimer counter reaches the period, it will return to zero.

Note

the period parameter is buffered and will take effect only after called **PTIMER_LoadValue()** function.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u16MaxCnt</i>	the Ptimer max count

LOCAL_INLINE void PTIMER_HWA_SetPulseOutDelay (PTIMER_Type *const *pPtimer*, uint16_t *u16DelayHigh*, uint16_t *u16DelayLow*)

Set the pulse out delay value When the Ptimer counter reach the delay high value, the pulse output goes high When the Ptimer counter reach the delay low value, the pulse output goes low The delay high value can be either greater or less than the delay low value.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u16DelayHigh</i>	the delay high value
<i>u16DelayLow</i>	the delay low value

LOCAL_INLINE void PTIMER_HWA_SetSeqErrIntEnableFlag (PTIMER_Type *const *pPtimer*, bool *bEnable*)

Set whether to enable sequence error interrupt.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>bEnable</i>	whether to enable sequence error interrupt

LOCAL_INLINE void PTIMER_HWA_SetStatusCtrl (PTIMER_Type *const *pPtimer*, uint32_t *u32CfgValue*)

Set the STATUS_CTRL register value.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>u32CfgValue</i>	the register value to set

LOCAL_INLINE void PTIMER_HWA_SetTriggerSource (PTIMER_Type *const *pPtimer*, PTIMER_TrgSrcType *eTriggerSource*)

Set the trigger source of the Ptimer instance.

Parameters

<i>pPtimer</i>	the base address of the Ptimer instance
<i>eTriggerSource</i>	the trigger source of the Ptimer instance

HwA_rgm

Collaboration diagram for HwA_rgm:Enumerations

```
enum RGM_ResetEventType { RGM_WAKEUP = RGM_SRS_WAKEUP_MASK, RGM_LVD =
RGM_SRS_LVR_MASK, RGM_CLKERR1 = RGM_SRS_CLKERR1_MASK, RGM_CLKERR0 =
RGM_SRS_CLKERR0_MASK, RGM_FSCMU = RGM_SRS_FSCMU_MASK, RGM_HSMWDOG =
RGM_SRS_HSM_WDOG_MASK, RGM_PIN = RGM_SRS_PIN_MASK, RGM_POR = RGM_SRS_POR_MASK,
RGM_JTAG = RGM_SRS_JTAG_MASK, RGM_SYSAP = RGM_SRS_SYSAP_MASK, RGM_SACKERR =
RGM_SRS_SACKERR_MASK, RGM_CMU3 = RGM_SRS_CMU3_MASK, RGM_LBIST = RGM_SRS_LBIST_MASK,
RGM_CPU_LOCKUP = RGM_C0_SRS_C0_LOCKUP_MASK, RGM_CPU_SYSRST =
RGM_C0_SRS_C0_SYSRST_MASK, RGM_CPU_WDOG = RGM_C0_SRS_C0_WDOG_MASK, RGM_CPU_INTM =
RGM_C0_SRS_C0_INTM_MASK, RGM_CPU_SWRST = RGM_C0_SRS_C0_SWRST_MASK, RGM_FSMERR =
RGM_SRS_FSM_ERR_MASK, RGM_PINRSTTOUT = RGM_SRS_PINRST_TOUT_MASK, RGM_SYSRSTTOUT =
RGM_SRS_SYSRST_TOUT_MASK }
```

Rgm reset event.

```
enum RGM_ResetDelayType { RGM_8_CLOCK_CYCLES = 0U, RGM_32_CLOCK_CYCLES,
RGM_128_CLOCK_CYCLES, RGM_512_CLOCK_CYCLES }
```

Rgm reset interrupt delay cycles type.

```
enum RGM_SysResetIntMangerType { RGM_INT_CLKERR0 = RGM_SRIE_CLKERR0_RIE_MASK,
RGM_INT_WDG = RGM_SRIE_WDG_RIE_MASK, RGM_INT_JTAG = RGM_SRIE_JTAG_RIE_MASK,
RGM_INT_CPULOC = RGM_SRIE_CPULOC_RIE_MASK, RGM_INT_SW = RGM_SRIE_SW_RIE_MASK,
RGM_INT_SYSAP = RGM_SRIE_SYSAP_RIE_MASK, RGM_INT_SACKERR = RGM_SRIE_SACKERR_RIE_MASK,
RGM_INT_ALL = RGM_SRIE_ALL_RIE_MASK }
```

Rgm reset interrupt event manger.

```
enum RGM_CoreResetIntMangerType { RGM_CPU_INT_LOCKUP = RGM_C0_CFG_C0_LOCKUP_IE_MASK,
RGM_CPU_INT_SYSRST = RGM_C0_CFG_C0_SYSRST_IE_MASK, RGM_CPU_INT_WDOG =
RGM_C0_CFG_C0_WDOG_IE_MASK, RGM_CPU_INT_INTM = RGM_C0_CFG_C0_INTM_IE_MASK,
RGM_CPU_INT_SWRST = RGM_C0_CFG_C0_SWRST_IE_MASK }
```

Rgm CPU0,1,2,3 related interrupt event manger.

```
enum RGM_CoreResetStatusType { RGM_REEST_STATUS_UNDER_RESET = 0U,
RGM_REEST_STATUS_OUT_RESET }
```

Rgm CPU0,1,2,3 software reset status.

```
enum RGM_CoreRstTrigSysRstMangerType { RGM_CPU_EN_LOCKUP =
RGM_C0_CFG_C0_LOCKUP_EN_MASK, RGM_CPU_EN_SYSRST = RGM_C0_CFG_C0_SYSRST_EN_MASK,
RGM_CPU_EN_WDOG = RGM_C0_CFG_C0_WDOG_EN_MASK, RGM_CPU_EN_INTM =
RGM_C0_CFG_C0_INTM_EN_MASK, RGM_CPU_EN_SWRST = RGM_C0_CFG_C0_SWRST_EN_MASK,
RGM_CPU_EN_ALL = 0x1F0000 }
```

Rgm CPU1,2,3 system reset enable flag.

Functions

```
LOCAL_INLINE uint32_t RGM_HWA_ReadLastResetFlag (void)
```

Read last reset flag.

```
LOCAL_INLINE uint32_t RGM_HWA_ReadAllResetFlagBeforePOR (void)
```

Read all reset flag before POR,SSRS register is reset on POR only.

```
LOCAL_INLINE void RGM_HWA_SetSrsReg (uint32_t u32Value)
```

Set SSRS register.

```
LOCAL_INLINE void RGM_HWA_ClearAllResetFlagAfterPOR (void)
```

This api can clear all reset flag of SSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

```
LOCAL_INLINE uint32_t RGM_HWA_ReadResetPinFilterEnable (void)
```

Read reset pin filter register.

```
LOCAL_INLINE void RGM_HWA_SetRstfltReg (uint32_t u32Value)
```

Set RSTFLT register.

LOCAL_INLINE void **RGM_HWA_SetBusClockFilterWidth** (uint8_t u8Value)

Set reset pin filter bus clock filter width.

LOCAL_INLINE void **RGM_HWA_EnableBusClockFilter** (void)

Enable reset pin filter bus clock.

LOCAL_INLINE void **RGM_HWA_EnableAon32kClockFilter** (void)

Enable reset pin filter AON32K clock.

LOCAL_INLINE void **RGM_HWA_EnableAon32kLPClockFilter** (void)

Enable reset pin filter AON32K low power clock.

LOCAL_INLINE void **RGM_HWA_ClearBusClockFilterWidth** (void)

Clear reset pin filter bus clock filter width.

LOCAL_INLINE void **RGM_HWA_DisableBusClockFilter** (void)

Disable reset pin filter bus clock.

LOCAL_INLINE void **RGM_HWA_DisableAon32kClockFilter** (void)

Disable reset pin filter AON32K clock.

LOCAL_INLINE void **RGM_HWA_DisableAon32kLPClockFilter** (void)

Disable reset pin filter AON32K low power clock.

LOCAL_INLINE void **RGM_HWA_SetSrieReg** (uint32_t u32Value)

Set SRIE register.

LOCAL_INLINE uint32_t **RGM_HWA_GetSrieReg** (void)

Get SRIE register.

LOCAL_INLINE void **RGM_HWA_SetResetDelay** (RGM_ResetDelayType eDelay)

Set Reset delay.

LOCAL_INLINE void **RGM_HWA_EnableGlobalResetInterrupt** (void)

Enable global reset interrupt.

LOCAL_INLINE void **RGM_HWA_EnableResetInterrupt** (RGM_SysResetIntMangerType eResetInterrupt)

Enable reset interrupt.

LOCAL_INLINE void **RGM_HWA_DisableResetInterrupt** (RGM_SysResetIntMangerType eResetInterrupt)

Disable reset interrupt.

LOCAL_INLINE uint32_t **RGM_HWA_GetResetInterrupt** (void)

Get enabling status of system reset.

LOCAL_INLINE void **CM7_HWA_SystemCoreReset** (void)

Perform system core reset.

LOCAL_INLINE void **RGM_HWA_SetC0CfgReg** (uint32_t u32Value)

Set C0_CFG register.

LOCAL_INLINE uint32_t **RGM_HWA_GetC0CfgReg** (void)

Get C0_CFG register.

LOCAL_INLINE void **RGM_HWA_EnableCPU0InterruptFlag** (uint32_t u32CPU0ResetInterrupt)

Enable CPU0 interrupt.

LOCAL_INLINE void **RGM_HWA_DisableCPU0InterruptFlag** (uint32_t u32CPU0ResetInterrupt)

Disable CPU0 interrupt.

LOCAL_INLINE uint32_t **RGM_HWA_GetCPU0ResetInterrupt** (void)

Get enabling status of CPU0 core related reset.

LOCAL_INLINE RGM_CoreResetStatusType **RGM_HWA_GetCPU0OutResetFlag** (void)

Get the CPU0 exit reset flag.

LOCAL_INLINE void **RGM_HWA_CPU0_SystemSoftWareReset** (void)

Issue a CPU0 system software reset.

LOCAL_INLINE uint32_t **RGM_HWA_ReadCPU0LastResetFlag** (void)

Read CPU0 last reset flag.

LOCAL_INLINE uint32_t **RGM_HWA_ReadCPU0AllResetFlagBeforePOR** (void)

Read CPU0 all reset flag before POR,SSRS register is reset on POR only.

LOCAL_INLINE void **RGM_HWA_SetC0SsrsReg** (uint32_t u32Value)

Set C0_SSRS register.

LOCAL_INLINE void **RGM_HWA_ClearC0AllResetFlagAfterPOR** (void)

This api can clear all reset flag of SSRGM_C0_SSRSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

Detailed Description

Enumeration Type Documentation

enum RGM_CoreResetIntMangerType

Rgm CPU0,1,2,3 related interrupt event manger.

Enumerator:

RGM_CPU_INT_ LOCKUP	CPU lockup reset
RGM_CPU_INT_ SYSRST	CPU system core reset

RGM_CPU_INT_WDOG	Watchdog reset
RGM_CPU_INT_INTM	Interrupt monitor reset
RGM_CPU_INT_SWRST	Software reset

enum RGM_CoreResetStatusType

Rgm CPU0,1,2,3 software reset status.

Enumerator:

RGM_REEST_STATUS_UNDER_RESET	Under reset
RGM_REEST_STATUS_OUT_OF_RESET	Out of reset

enum RGM_CoreRstTrigSysRstMangerType

Rgm CPU1,2,3 system reset enable flag.

Enumerator:

RGM_CPU_ENABLE_LOCKUP	Core lockup reset trigger system reset
RGM_CPU_ENABLE_SYSRST	Core system core reset trigger system reset
RGM_CPU_ENABLE_WDOG	Core watchdog reset trigger system reset
RGM_CPU_ENABLE_INTM	Core interrupt monitor reset trigger system reset
RGM_CPU_ENABLE_SWRST	Core software reset trigger system reset

RGM_CPU_EN_A LL	Core All reset trigger system reset
--------------------	-------------------------------------

enum RGM_ResetDelayType

Rgm reset interrupt delay cycles type.

Enumerator:

RGM_8_CLOCK_ CYCLES	8 clock cycles
RGM_32_CLOCK_ _CYCLES	32 clock cycles
RGM_128_CLOC K_CYCLES	128 clock cycles
RGM_512_CLOC K_CYCLES	512 clock cycles

enum RGM_ResetEventType

Rgm reset event.

Enumerator:

RGM_WAKEUP	Wakeup reset
RGM_LVD	Low voltage reset
RGM_CLKERR1	Clock error 1 reset
RGM_CLKERR0	Clock error 0 reset
RGM_FSCMU	FCSMU reset

RGM_HSMWDO G	HSM watchdog reset
RGM_PIN	Pin reset
RGM_POR	Power on reset
RGM_JTAG	JTAG reset
RGM_SYSAP	System AP reset
RGM_SACKERR	SACKERR reset
RGM_CMU3	CMU3 reset
RGM_LBIST	LBIST reset
RGM_CPU_LOC KUP	CPU lockup reset
RGM_CPU_SYSR ST	CPU system core reset
RGM_CPU_WDO G	Watchdog 0 reset
RGM_CPU_INTM	CPU interrupt monitor reset
RGM_CPU_SWR ST	System software reset
RGM_FSMERR	FSM error reset
RGM_PINRSTTO UT	Pin reset timeout reset
RGM_SYSRSTTO UT	System reset timeout reset

enum RGM_SysResetIntMangerType

Rgm reset interrupt event manger.

Enumerator:

RGM_INT_CLKE RRO	Clock error 0 reset interrupt
RGM_INT_WDG	Watchdog reset interrupt
RGM_INT_JTAG	JTAG reset interrupt
RGM_INT_CPUL OC	CPU lockup reset interrupt
RGM_INT_SW	Software reset interrupt
RGM_INT_SYSA P	System AP reset interrupt
RGM_INT_SACK ERR	SACKERR reset interrupt
RGM_INT_ALL	All reset interrupt

Function Documentation

LOCAL_INLINE void CM7_HWA_SystemCoreReset (void)

Perform system core reset.

LOCAL_INLINE void RGM_HWA_ClearAllResetFlagAfterPOR (void)

This api can clear all reset flag of SSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

LOCAL_INLINE void RGM_HWA_ClearBusClockFilterWidth (void)

Clear reset pin filter bus clock filter width.

LOCAL_INLINE void RGM_HWA_ClearC0AllResetFlagAfterPOR (void)

This api can clear all reset flag of SSRGM_C0_SSRSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

LOCAL_INLINE void RGM_HWA_CPU0_SystemSoftWareReset (void)

Issue a CPU0 system software reset.

LOCAL_INLINE void RGM_HWA_DisableAon32kClockFilter (void)

Disable reset pin filter AON32K clock.

LOCAL_INLINE void RGM_HWA_DisableAon32kLPClockFilter (void)

Disable reset pin filter AON32K low power clock.

LOCAL_INLINE void RGM_HWA_DisableBusClockFilter (void)

Disable reset pin filter bus clock.

LOCAL_INLINE void RGM_HWA_DisableCPU0InterruptFlag (uint32_t *u32CPU0ResetInterrupt*)

Disable CPU0 interrupt.

Parameters

<i>u32CPU0ResetInterrupt</i>	CPU0 related interrupt type
------------------------------	-----------------------------

LOCAL_INLINE void RGM_HWA_DisableResetInterrupt (RGM_SysResetIntMangerType *eResetInterrupt*)

Disable reset interrupt.

Parameters

<i>eResetInterrupt</i>	Reset interrupt type
------------------------	----------------------

LOCAL_INLINE void RGM_HWA_EnableAon32kClockFilter (void)

Enable reset pin filter AON32K clock.

LOCAL_INLINE void RGM_HWA_EnableAon32kLPClockFilter (void)

Enable reset pin filter AON32K low power clock.

LOCAL_INLINE void RGM_HWA_EnableBusClockFilter (void)

Enable reset pin filter bus clock.

LOCAL_INLINE void RGM_HWA_EnableCPU0InterruptFlag (uint32_t *u32CPU0ResetInterrupt*)

Enable CPU0 interrupt.

Parameters

<i>u32CPU0ResetInterrupt</i>	CPU0 related interrupt type
------------------------------	-----------------------------

LOCAL_INLINE void RGM_HWA_EnableGlobalResetInterrupt (void)

Enable global reset interrupt.

LOCAL_INLINE void RGM_HWA_EnableResetInterrupt (RGM_SysResetIntMangerType *eResetInterrupt*)

Enable reset interrupt.

Parameters

<i>eResetInterrupt</i>	Reset interrupt type
------------------------	----------------------

LOCAL_INLINE uint32_t RGM_HWA_GetC0CfgReg (void)

Get C0_CFG register.

Returns

C0_CFG register value

LOCAL_INLINE RGM_CoreResetStatusType RGM_HWA_GetCPU0OutResetFlag (void)

Get the CPU0 exit reset flag.

Returns

RGM_CPU_OUT_RST_UNDER CPU0 is under reset

RGM_CPU_OUT_RST_OUT CPU0 is out of reset

LOCAL_INLINE uint32_t RGM_HWA_GetCPU0ResetInterrupt (void)

Get enabling status of CPU0 core related reset.

Returns

Interrupt enabling status of CPU0 core related reset

LOCAL_INLINE uint32_t RGM_HWA_GetResetInterrupt (void)

Get enabling status of system reset.

Returns

Interrupt enabling status of system reset

LOCAL_INLINE uint32_t RGM_HWA_GetSrieReg (void)

Get SRIE register.

Returns

SRIE register value

LOCAL_INLINE uint32_t RGM_HWA_ReadAllResetFlagBeforePOR (void)

Read all reset flag before POR,SSRS register is reset on POR only.

Returns

All reset flag before POR

LOCAL_INLINE uint32_t RGM_HWA_ReadCPU0AllResetFlagBeforePOR (void)

Read CPU0 all reset flag before POR,SSRS register is reset on POR only.

Returns

CPU0 all reset flag before POR

LOCAL_INLINE uint32_t RGM_HWA_ReadCPU0LastResetFlag (void)

Read CPU0 last reset flag.

Returns

CPU0 last reset flag

LOCAL_INLINE uint32_t RGM_HWA_ReadLastResetFlag (void)

Read last reset flag.

Returns

Last reset flag

LOCAL_INLINE uint32_t RGM_HWA_ReadResetPinFilterEnable (void)

Read reset pin filter register.

Returns

Reset pin filter register

LOCAL_INLINE void RGM_HWA_SetBusClockFilterWidth (uint8_t *u8Value*)

Set reset pin filter bus clock filter width.

Parameters

<i>u8Value</i>	Bus clock filter width value
----------------	------------------------------

LOCAL_INLINE void RGM_HWA_SetC0CfgReg (uint32_t *u32Value*)

Set C0_CFG register.

Parameters

<i>u32Value</i>	C0_CFG register value
-----------------	-----------------------

LOCAL_INLINE void RGM_HWA_SetC0SrsReg (uint32_t *u32Value*)

Set C0_SSRS register.

Parameters

<i>u32Value</i>	C0_SSRS register value
-----------------	------------------------

LOCAL_INLINE void RGM_HWA_SetResetDelay (RGM_ResetDelayType *eDelay*)

Set Reset delay.

Parameters

<i>eDelay</i>	Reset delay type
---------------	------------------

LOCAL_INLINE void RGM_HWA_SetRstfltReg (uint32_t *u32Value*)

Set RSTFLT register.

Parameters

<i>u32Value</i>	RSTFLT register value
-----------------	-----------------------

LOCAL_INLINE void RGM_HWA_SetSrieReg (uint32_t *u32Value*)

Set SRIE register.

Parameters

<i>u32Value</i>	SRIE register value
-----------------	---------------------

LOCAL_INLINE void RGM_HWA_SetSrsReg (uint32_t *u32Value*)

Set SSRS register.

Parameters

<i>u32Value</i>	SSRS register value
-----------------	---------------------

HwA_rtc

Collaboration diagram for HwA_rtc:Enumerations

```
enum RTC_ClkoutSecIntFreqType { RTC_FREQ_1HZ = 0, RTC_FREQ_2HZ, RTC_FREQ_4HZ,  
RTC_FREQ_8HZ, RTC_FREQ_16HZ, RTC_FREQ_32hZ, RTC_FREQ_64HZ, RTC_FREQ_128HZ }
```

in the second interrupt mode, this type indicates the interrupt frequency. in the clkout mode , this type indicates the clkout frequency.

Functions

```
LOCAL_INLINE uint32_t RTC_HWA_ReadSecondValue (RTC_Type *const pRtc)
```

Read second value.

```
LOCAL_INLINE bool RTC_HWA_ReadOverflowFlag (RTC_Type *const pRtc)
```

Read RTC SR overflow flag.

```
LOCAL_INLINE bool RTC_HWA_ReadOverflowEnable (RTC_Type *const pRtc)
```

Read RTC IER overflow enable bit.

```
LOCAL_INLINE bool RTC_HWA_ReadAlarmFlag (RTC_Type *const pRtc)
```

Read RTC alarm flag.

```
LOCAL_INLINE bool RTC_HWA_ReadAlarmEnable (RTC_Type *const pRtc)
```

Read RTC alarm Enable.

```
LOCAL_INLINE bool RTC_HWA_ReadSecondEnable (RTC_Type *const pRtc)
```

Read RTC second interrupt Enable.

```
LOCAL_INLINE uint8_t RTC_HWA_ReadCompInterval (RTC_Type *const pRtc)
```

Read RTC Compensation Interbal.

LOCAL_INLINE uint8_t **RTC_HWA_ReadCompValue** (RTC_Type *const pRtc)

Read RTC Compensation Value.

LOCAL_INLINE void **RTC_HWA_SetCompInterval** (RTC_Type *const pRtc, uint8_t u8Interval)

Sets the RTC compare interval.

LOCAL_INLINE void **RTC_HWA_SetCompValue** (RTC_Type *const pRtc, uint8_t u8Value)

Sets the RTC compare value.

LOCAL_INLINE void **RTC_HWA_SetPrescalerCounterValue** (RTC_Type *const pRtc, uint16_t u16Value)

Set RTC prescaler register.

LOCAL_INLINE void **RTC_HWA_SetSecondCounterValue** (RTC_Type *const pRtc, uint32_t u32Value)

Set RTC seconds register.

LOCAL_INLINE void **RTC_HWA_SetAlarmCounterValue** (RTC_Type *const pRtc, uint32_t u32Value)

Set RTC alarm value.

LOCAL_INLINE void **RTC_HWA_SetInterruptValue** (RTC_Type *const pRtc, uint32_t u32Value)

Set RTC interrupt value.

LOCAL_INLINE void **RTC_HWA_ConfigControl** (RTC_Type *const pRtc, uint32_t u32Value)

Configure control register.

LOCAL_INLINE void **RTC_HWA_EnableRtcCounter** (RTC_Type *const pRtc)

Enable RTC time counter.

LOCAL_INLINE void **RTC_HWA_SetClkoutFreqStable** (RTC_Type *const pRtc)

Set RTC_CLKOUT is from the 32.768 khz clock.

LOCAL_INLINE void **RTC_HWA_EnableAlarmInterrupt** (RTC_Type *const pRtc, bool bEnable)

Enable alarm interrupt.

LOCAL_INLINE void **RTC_HWA_EnableSecondInterrupt** (RTC_Type *const pRtc, bool bEnable)

Enable second interrupt.

LOCAL_INLINE void **RTC_HWA_EnableOverflowInterrupt** (RTC_Type *const pRtc, bool bEnable)

Enable overflow interrupt.

LOCAL_INLINE void **RTC_HWA_UnlockStatusControlCompensationReg** (RTC_Type *const pRtc)

Unlock lock/status/control/compensation register.

LOCAL_INLINE void **RTC_HWA_SetSecondAndClkoutFreq** (RTC_Type *const pRtc,

RTC_ClkoutSecIntFreqType eFreq)

Set second interrupt and RTC_CLKOUT frequency.

LOCAL_INLINE void **RTC_HWA_DisableRtcCounter** (RTC_Type *const pRtc)

Disable RTC time counter.

LOCAL_INLINE void **RTC_HWA_SetClkoutFromSelectFreq** (RTC_Type *const pRtc)

Set RTC_CLKOUT is from the prescaler output clock selected by IER[TSIC].

LOCAL_INLINE void **RTC_HWA_DisableAlarmInterrupt** (RTC_Type *const pRtc)

Disable alarm interrupt.

LOCAL_INLINE void **RTC_HWA_DisableSecondInterrupt** (RTC_Type *const pRtc)

Disable second interrupt.

LOCAL_INLINE void **RTC_HWA_DisableOverflowInterrupt** (RTC_Type *const pRtc)

Disable overflow interrupt.

Detailed Description

Enumeration Type Documentation

enum RTC_ClkoutSecIntFreqType

in the second interrupt mode, this type indicates the interrupt frequency. in the clkout mode , this type indicates the clkout frequency .

Enumerator:

RTC_FREQ_1HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 1Hz
RTC_FREQ_2HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 2Hz
RTC_FREQ_4HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 4Hz
RTC_FREQ_8HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 8Hz
RTC_FREQ_16HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 16Hz
RTC_FREQ_32HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 32Hz
RTC_FREQ_64HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 64Hz

RTC_FREQ_128 HZ	Set the RTC Seconds interrupt and the RTC_CLKOUT prescale output frequency to 128Hz
--------------------	---

Function Documentation

LOCAL_INLINE void RTC_HWA_ConfigControl (RTC_Type *const *pRtc*, uint32_t *u32Value*)

Configure control register.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>u32Value</i>	Control value

LOCAL_INLINE void RTC_HWA_DisableAlarmInterrupt (RTC_Type *const *pRtc*)

Disable alarm interrupt.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

LOCAL_INLINE void RTC_HWA_DisableOverflowInterrupt (RTC_Type *const *pRtc*)

Disable overflow interrupt.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

LOCAL_INLINE void RTC_HWA_DisableRtcCounter (RTC_Type *const *pRtc*)

Disable RTC time counter.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

LOCAL_INLINE void RTC_HWA_DisableSecondInterrupt (RTC_Type *const *pRtc*)

Disable second interrupt.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

LOCAL_INLINE void RTC_HWA_EnableAlarmInterrupt (RTC_Type *const *pRtc*, bool *bEnable*)

Enable alarm interrupt.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>bEnable</i>	Indicates whether to enable the interrupt. true: Enable interrupt, false: Disable interrupt.

LOCAL_INLINE void RTC_HWA_EnableOverflowInterrupt (RTC_Type *const *pRtc*, bool *bEnable*)

Enable overflow interrupt.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>bEnable</i>	Boolean value indicating whether to enable (true) or disable (false) the interrupt.

LOCAL_INLINE void RTC_HWA_EnableRtcCounter (RTC_Type *const *pRtc*)

Enable RTC time counter.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

LOCAL_INLINE void RTC_HWA_EnableSecondInterrupt (RTC_Type *const *pRtc*, bool *bEnable*)

Enable second interrupt.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>bEnable</i>	A boolean indicating whether to enable (true) or disable (false) the interrupt.

LOCAL_INLINE bool RTC_HWA_ReadAlarmEnable (RTC_Type *const *pRtc*)

Read RTC alarm Enable.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

true means enable alarm Interrupt ;false means disable alarm Interrupt

LOCAL_INLINE bool RTC_HWA_ReadAlarmFlag (RTC_Type *const *pRtc*)

Read RTC alarm flag.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

true means alarm flag is 1 ;false means alarm flag is 0.

LOCAL_INLINE uint8_t RTC_HWA_ReadCompInterval (RTC_Type *const *pRtc*)

Read RTC Compensation Interbal.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

Compensation Interbal

LOCAL_INLINE uint8_t RTC_HWA_ReadCompValue (RTC_Type *const *pRtc*)

Read RTC Compensation Value.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

Compensation Value

LOCAL_INLINE bool RTC_HWA_ReadOverflowEnable (RTC_Type *const *pRtc*)

Read RTC IER overflow enable bit.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

true means enable Overflow Interrupt ;false means disable Overflow Interrupt

LOCAL_INLINE bool RTC_HWA_ReadOverflowFlag (RTC_Type *const *pRtc*)

Read RTC SR overflow flag.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

true means Overflow flag is 1 ;false means Overflow flag is 0.

LOCAL_INLINE bool RTC_HWA_ReadSecondEnable (RTC_Type *const *pRtc*)

Read RTC second interrupt Enable.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

true means enable second Interrupt ;false means disable second Interrupt

LOCAL_INLINE uint32_t RTC_HWA_ReadSecondValue (RTC_Type *const *pRtc*)

Read second value.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

Returns

Second value

LOCAL_INLINE void RTC_HWA_SetAlarmCounterValue (RTC_Type *const *pRtc*, uint32_t *u32Value*)

Set RTC alarm value.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>u32Value</i>	TAR register value

LOCAL_INLINE void RTC_HWA_SetClkoutFreqStable (RTC_Type *const *pRtc*)

Set RTC_CLKOUT is from the 32.768 khz clock.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

LOCAL_INLINE void RTC_HWA_SetClkoutFromSelectFreq (RTC_Type *const *pRtc*)

Set RTC_CLKOUT is from the prescaler output clock selected by IER[TSIC].

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

LOCAL_INLINE void RTC_HWA_SetCompInterval (RTC_Type *const *pRtc*, uint8_t *u8Interval*)

Sets the RTC compare interval.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>u8Interval</i>	The compare interval reload value, in seconds.

LOCAL_INLINE void RTC_HWA_SetCompValue (RTC_Type *const *pRtc*, uint8_t *u8Value*)

Sets the RTC compare value.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>u8Value</i>	The compare value, in ticks.

LOCAL_INLINE void RTC_HWA_SetInterruptValue (RTC_Type *const *pRtc*, uint32_t *u32Value*)

Set RTC interrupt value.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>u32Value</i>	IER register value

LOCAL_INLINE void RTC_HWA_SetPrescalerCounterValue (RTC_Type *const *pRtc*, uint16_t *u16Value*)

Set RTC prescaler register.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

<i>u16Value</i>	PR register value
-----------------	-------------------

**LOCAL_INLINE void RTC_HWA_SetSecondAndClkoutFreq (RTC_Type *const *pRtc*,
RTC_ClkoutSecIntFreqType *eFreq*)**

Set second interrupt and RTC_CLKOUT frequency.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>eFreq</i>	Frequency value

LOCAL_INLINE void RTC_HWA_SetSecondCounterValue (RTC_Type *const *pRtc*, uint32_t *u32Value*)

Set RTC seconds register.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
<i>u32Value</i>	SR register value

LOCAL_INLINE void RTC_HWA_UnlockStatusControlCompensationReg (RTC_Type *const *pRtc*)

Unlock lock/status/control/compensation register.

Parameters

<i>pRtc</i>	the base address of the pRtc instance.
-------------	--

HWA_scg

Collaboration diagram for Hwa_scg:Macros

```
#define SCG_PLLCSR_LK_MASK 0x800000u
```

PLL related.

```
#define SCG_PLLCSR_EN_MASK 0x1u
```

```

#define SCG_PLLCSR_CMRE_MASK 0x20000u
#define SCG_PLLCSR_CMRE_SHIFT 17u
#define
SCG_PLLCSR_CMRE(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCSR_CMRE_SHIFT))&SCG_PLLCSR_CMRE_
MASK)
#define SCG_PLLCSR_CM_MASK 0x10000u
#define SCG_PLLCSR_CM_SHIFT 16u
#define
SCG_PLLCSR_CM(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCSR_CM_SHIFT))&SCG_PLLCSR_CM_MASK)
#define SCG_PLLCSR_STEN_MASK 0x2u
#define SCG_PLLCSR_STEN_SHIFT 1u
#define
SCG_PLLCSR_STEN(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCSR_STEN_SHIFT))&SCG_PLLCSR_STEN_M
ASK)
#define SCG_PLLDIV_DIVL_EN_MASK 0x4000000u
#define SCG_PLLDIV_DIVL_MASK 0x70000u
#define SCG_PLLDIV_DIVL_SHIFT 16u
#define
SCG_PLLDIV_DIVL(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVL_SHIFT))&SCG_PLLDIV_DIVL_MASK)
#define SCG_PLLDIV_DIVM_EN_MASK 0x2000000u
#define SCG_PLLDIV_DIVM_MASK 0x700u
#define SCG_PLLDIV_DIVM_SHIFT 8u
#define
SCG_PLLDIV_DIVM(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVM_SHIFT))&SCG_PLLDIV_DIVM_MAS
K)
#define SCG_PLLDIV_DIVH_EN_MASK 0x1000000u
#define SCG_PLLDIV_DIVH_MASK 0x7u
#define SCG_PLLDIV_DIVH_SHIFT 0u
#define
SCG_PLLDIV_DIVH(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVH_SHIFT))&SCG_PLLDIV_DIVH_MAS
K)
#define SCG_PLLCFG_MULT_MASK 0x1FF0000u
#define SCG_PLLCFG_MULT_SHIFT 16u
#define
SCG_PLLCFG_MULT(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_MULT_SHIFT))&SCG_PLLCFG_MULT_
MASK)
#define SCG_PLLCFG_PREDIV_MASK 0x1F00u
#define SCG_PLLCFG_PREDIV_SHIFT 8u

```

```

#define
SCG_PLLCFG_PREDIV(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PREDIV_SHIFT))&SCG_PLLCFG_PREDIV_MASK)
#define SCG_PLLCFG_PSTDIV_MASK 0x30u
#define SCG_PLLCFG_PSTDIV_SHIFT 4u
#define
SCG_PLLCFG_PSTDIV(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PSTDIV_SHIFT))&SCG_PLLCFG_PSTDIV_MASK)
#define SCG_PLLCFG_SOURCE_MASK 0x1u
#define SCG_PLLCFG_SOURCE_SHIFT 0u
#define
SCG_PLLCFG_SOURCE(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_SOURCE_SHIFT))&SCG_PLLCFG_SOURCE_MASK)
#define SCG_PLLCFG_PSTDIV1_MASK 0xEu
#define SCG_PLLCFG_PSTDIV1_SHIFT 1u
#define
SCG_PLLCFG_PSTDIV1(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PSTDIV1_SHIFT))&SCG_PLLCFG_PSTDIV1_MASK)
#define _SCG_HWA_SetPllCsr(_pll_, val) SCG->_pll_##CSR = val
#define SCG_HWA_SetPllCsr(pll, val) _SCG_HWA_SetPllCsr(pll, val)
#define _SCG_HWA_UnlockPllCsr(_pll_) SCG->_pll_##CSR &= ~(uint32_t)SCG_PLLCSR_LK_MASK
#define SCG_HWA_UnlockPllCsr(pll) _SCG_HWA_UnlockPllCsr(pll)
#define _SCG_HWA_LockPllCsr(_pll_) SCG->_pll_##CSR |= (uint32_t)SCG_PLLCSR_LK_MASK
#define SCG_HWA_LockPllCsr(pll) _SCG_HWA_LockPllCsr(pll)
#define _SCG_HWA_EnablePllClockMonitor(_pll_) SCG->_pll_##CSR |= (uint32_t)SCG_PLLCSR_CM_MASK
#define SCG_HWA_EnablePllClockMonitor(pll) _SCG_HWA_EnablePllClockMonitor(pll)
#define _SCG_HWA_EnablePllClockMonitorReset(_pll_) SCG->_pll_##CSR |= (uint32_t)SCG_PLLCSR_CMRE_MASK
#define SCG_HWA_EnablePllClockMonitorReset(pll) _SCG_HWA_EnablePllClockMonitorReset(pll)
#define _SCG_HWA_SetPllCfg(_pll_, val) SCG->_pll_##CFG = val
#define SCG_HWA_SetPllCfg(pll, val) _SCG_HWA_SetPllCfg(pll, val)
#define _SCG_HWA_SetPllDiv(_pll_, val) SCG->_pll_##DIV = val
#define SCG_HWA_SetPllDiv(pll, val) _SCG_HWA_SetPllDiv(pll, val)
#define _SCG_HWA_EnablePll(_pll_) SCG->_pll_##CSR |= (uint32_t)(SCG_PLLCSR_EN_MASK)
#define SCG_HWA_EnablePll(pll) _SCG_HWA_EnablePll(pll)
#define _SCG_HWA_EnablePllDiv(_pll_) SCG->_pll_##DIV |= (uint32_t)(SCG_PLLDIV_DIVL_EN_MASK | SCG_PLLDIV_DIVM_EN_MASK | SCG_PLLDIV_DIVH_EN_MASK)

```

```

#define SCG_HWA_EnablePllDiv(pll)  _SCG_HWA_EnablePllDiv(pll)
#define _SCG_HWA_DisablePllDiv(_pll_)  SCG->_pll_ ##DIV &= (~(uint32_t)(SCG_PLLDIV_DIVL_EN_MASK |
SCG_PLLDIV_DIVM_EN_MASK | SCG_PLLDIV_DIVH_EN_MASK))
#define SCG_HWA_DisablePllDiv(pll)  _SCG_HWA_DisablePllDiv(pll)
#define _SCG_HWA_GetPllMult(_pll_)  ((uint16_t)(((uint32_t)(SCG->_pll_ ##CFG) &
SCG_PLLCFG_MULT_MASK) >> SCG_PLLCFG_MULT_SHIFT))
#define SCG_HWA_GetPllMult(pll)  _SCG_HWA_GetPllMult(pll)
#define _SCG_HWA_GetPllPrediv(_pll_)  ((uint8_t)(((uint32_t)(SCG->_pll_ ##CFG) &
SCG_PLLCFG_PREDIV_MASK) >> SCG_PLLCFG_PREDIV_SHIFT))
#define SCG_HWA_GetPllPrediv(pll)  _SCG_HWA_GetPllPrediv(pll)
#define _SCG_HWA_GetPllPstdiv(_pll_)  ((uint8_t)(((uint32_t)(SCG->_pll_ ##CFG) &
SCG_PLLCFG_PSTDIV_MASK) >> SCG_PLLCFG_PSTDIV_SHIFT))
#define SCG_HWA_GetPllPstdiv(pll)  _SCG_HWA_GetPllPstdiv(pll)
#define _SCG_HWA_GetPllPstdiv1(_pll_)  ((uint8_t)(((uint32_t)(SCG->_pll_ ##CFG) &
SCG_PLLCFG_PSTDIV1_MASK) >> SCG_PLLCFG_PSTDIV1_SHIFT))
#define SCG_HWA_GetPllPstdiv1(pll)  _SCG_HWA_GetPllPstdiv1(pll)
#define _SCG_HWA_GetPllSrc(_pll_)  ((uint8_t)(((uint32_t)(SCG->_pll_ ##CFG) &
SCG_PLLCFG_SOURCE_MASK) >> SCG_PLLCFG_SOURCE_SHIFT))
#define SCG_HWA_GetPllSrc(pll)  _SCG_HWA_GetPllSrc(pll)
#define _SCG_HWA_GetPllLocked(_clock_)  ((bool)((((uint32_t)SCG->_clock_ ##CSR &
(uint32_t)SCG_ ##_clock_ ##CSR_PLLLK_MASK) != 0U) ? true : false))
#define SCG_HWA_GetPllLocked(_clock_)  _SCG_HWA_GetPllLocked(_clock_)
#define _SCG_HWA_GetClockValid(_clock_)  ((bool)((((uint32_t)SCG->_clock_ ##CSR &
(uint32_t)SCG_ ##_clock_ ##CSR_VLD_MASK) != 0U) ? true : false))
#define SCG_HWA_GetClockValid(_clock_)  _SCG_HWA_GetClockValid(_clock_)
#define _SCG_HWA_GetClockDiv(_clock_)  (uint32_t)SCG->_clock_ ##DIV
#define SCG_HWA_GetClockDiv(_clock_)  _SCG_HWA_GetClockDiv(_clock_)
#define _SCG_HWA_GetTrimLocked(_clock_)  ((bool)((((uint32_t)SCG->_clock_ ##CSR &
(uint32_t)SCG_ ##_clock_ ##CSR_TRLK_MASK) != 0U) ? true : false))
#define SCG_HWA_GetTrimLocked(_clock_)  _SCG_HWA_GetTrimLocked(_clock_)
#define SCG_CLOCKDIV_DIVL_ACK_MASK  0x40000000u
#define SCG_CLOCKDIV_DIVM_ACK_MASK  0x20000000u
#define SCG_CLOCKDIV_DIVH_ACK_MASK  0x10000000u
#define SCG_CLOCKDIV_DIV_ACK_MASK  0x70000000u
#define SCG_CLOCKDIV_DIV_ACK_MASK_SHIFT  0x7u
#define SCG_CLOCKDIV_DIVL_ACK_SHIFT  30u
#define SCG_CLOCKDIV_DIVM_ACK_SHIFT  29u
#define SCG_CLOCKDIV_DIVH_ACK_SHIFT  28u

```

```

#define SCG_CLOCKDIV_DIV_ACK_SHIFT 28u
#define SCG_CLOCKDIV_DIVL_EN_MASK 0x4000000u
#define SCG_CLOCKDIV_DIVM_EN_MASK 0x2000000u
#define SCG_CLOCKDIV_DIVH_EN_MASK 0x1000000u
#define SCG_CLOCKDIV_DIVL_EN_SHIFT 26u
#define SCG_CLOCKDIV_DIVM_EN_SHIFT 25u
#define SCG_CLOCKDIV_DIVH_EN_SHIFT 24u
#define SCG_CLOCKDIV_DIVL_VAL_MASK 0x70000u
#define SCG_CLOCKDIV_DIVM_VAL_MASK 0x700u
#define SCG_CLOCKDIV_DIVH_VAL_MASK 0x7u
#define SCG_CLOCKDIV_DIVL_VAL_SHIFT 16u
#define SCG_CLOCKDIV_DIVM_VAL_SHIFT 8u
#define SCG_CLOCKDIV_DIVH_VAL_SHIFT 0u
#define SCG_CHECK_DIVL_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIVL_ACK_MASK) >>
SCG_CLOCKDIV_DIVL_ACK_SHIFT)
#define SCG_CHECK_DIVM_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIVM_ACK_MASK) >>
SCG_CLOCKDIV_DIVM_ACK_SHIFT)
#define SCG_CHECK_DIVH_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIVH_ACK_MASK) >>
SCG_CLOCKDIV_DIVH_ACK_SHIFT)
#define SCG_CHECK_DIV_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIV_ACK_MASK) >>
SCG_CLOCKDIV_DIV_ACK_SHIFT)
#define SCG_CHECK_DIVL_EN(reg_val) ((reg_val & SCG_CLOCKDIV_DIVL_EN_MASK) >>
SCG_CLOCKDIV_DIVL_EN_SHIFT)
#define SCG_CHECK_DIVM_EN(reg_val) ((reg_val & SCG_CLOCKDIV_DIVM_EN_MASK) >>
SCG_CLOCKDIV_DIVM_EN_SHIFT)
#define SCG_CHECK_DIVH_EN(reg_val) ((reg_val & SCG_CLOCKDIV_DIVH_EN_MASK) >>
SCG_CLOCKDIV_DIVH_EN_SHIFT)
#define SCG_GET_DIVL_VAL(reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVL_VAL_MASK) >>
SCG_CLOCKDIV_DIVL_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVL_VAL_MASK) >>
SCG_CLOCKDIV_DIVL_VAL_SHIFT))
#define SCG_GET_DIVM_VAL(reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVM_VAL_MASK) >>
SCG_CLOCKDIV_DIVM_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVM_VAL_MASK) >>
SCG_CLOCKDIV_DIVM_VAL_SHIFT))
#define SCG_GET_DIVH_VAL(reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVH_VAL_MASK) >>
SCG_CLOCKDIV_DIVH_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVH_VAL_MASK) >>
SCG_CLOCKDIV_DIVH_VAL_SHIFT))

```

```
#define SCG_CALCULATE_DIVL_FREQ(clk_freq, reg_val) (((1U == SCG_CHECK_DIVL_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVL_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)
#define SCG_CALCULATE_DIVM_FREQ(clk_freq, reg_val) (((1U == SCG_CHECK_DIVM_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVM_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)
#define SCG_CALCULATE_DIVH_FREQ(clk_freq, reg_val) (((1U == SCG_CHECK_DIVH_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVH_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)
```

Functions

LOCAL_INLINE uint8_t **SCG_HWA_GetClkOutSel** (void)

Get clock out configure register CLKOUTSEL value.

LOCAL_INLINE uint32_t **SCG_HWA_GetClkOutCfg** (void)

Get clock out configure register value.

LOCAL_INLINE void **SCG_HWA_UnlockSoscCsr** (void)

Unlock SOSC CSR register.

LOCAL_INLINE void **SCG_HWA_LockSoscCsr** (void)

Lock SOSC CSR register.

LOCAL_INLINE void **SCG_HWA_EnableSoscClockMonitor** (void)

Enable SOSC clock monitor.

LOCAL_INLINE void **SCG_HWA_EnableSoscClockMonitorReset** (void)

Enable SOSC clock monitor Reset.

LOCAL_INLINE void **SCG_HWA_EnableSosc** (void)

Set SOSC enable.

LOCAL_INLINE void **SCG_HWA_DisableSosc** (void)

Disable SOSC.

LOCAL_INLINE void **SCG_HWA_SetSoscCfg** (uint32_t u32CfgValue)

Set SOSC CFG register value.

LOCAL_INLINE void **SCG_HWA_SetSoscCsr** (uint32_t u32CsrValue)

Set SOSC CSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetSoscCsr** (void)

Get SOSC CSR register value.

LOCAL_INLINE void **SCG_HWA_EnableFosc** (void)

Enable FOSC.

LOCAL_INLINE void **SCG_HWA_DisableFosc** (void)

Disable FOSC.

LOCAL_INLINE void **SCG_HWA_SetFoscCfg** (uint32_t u32CfgValue)

Set FOSCCFG register value.

LOCAL_INLINE void **SCG_HWA_SetFoscCsr** (uint32_t u32CsrValue)

Set FOSCCSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetFoscCsr** (void)

Get FOSCCSR register value.

LOCAL_INLINE void **SCG_HWA_UnlockFoscCsr** (void)

Unlock FOSC CSR register.

LOCAL_INLINE void **SCG_HWA_LockFoscCsr** (void)

lock FOSC CSR register

LOCAL_INLINE void **SCG_HWA_EnableFoscClockMonitor** (void)

Enable FOSC clock monitor.

LOCAL_INLINE void **SCG_HWA_EnableFoscClockMonitorReset** (void)

Enable FOSC clock monitor Reset.

LOCAL_INLINE void **SCG_HWA_EnableFoscDiv** (void)

enable FOSCDIV as user manual request sequence

LOCAL_INLINE void **SCG_HWA_DiableFoscDiv** (void)

Disable FOSCDIV as user manual request sequence.

LOCAL_INLINE void **SCG_HWA_SetFoscDiv** (uint32_t u32DivValue)

Set FOSCDIV register value.

LOCAL_INLINE void **SCG_HWA_SetSircCsr** (uint32_t u32CsrValue)

Set SIRCCSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetSircCsr** (void)

Get SIRCCSR register value.

LOCAL_INLINE void **SCG_HWA_UnlockSircCsr** (void)

Unlock SIRC CSR register.

LOCAL_INLINE void **SCG_HWA_LockSircCsr** (void)

Lock SIRC CSR register.

LOCAL_INLINE void **SCG_HWA_EnableSircClockMonitor** (void)

Enable SIRC clock monitor.

LOCAL_INLINE void **SCG_HWA_SetSircDiv** (uint32_t u32DivValue)

Set SIRCDIV register value.

LOCAL_INLINE void **SCG_HWA_DiableSircDiv** (void)

Disable SIRCDIV as user manual request sequence.

LOCAL_INLINE void **SCG_HWA_EnableSircDiv** (void)

enable FIRCDIV as user manual request sequence

LOCAL_INLINE void **SCG_HWA_SetSircTcfg** (uint32_t u32TcfgValue)

Set SIRC32KCSR register value for SIRC Trim configure.

LOCAL_INLINE void **SCG_HWA_EnableSirc32kCsr** (void)

Enable SIRC32KCSR register.

LOCAL_INLINE void **SCG_HWA_DisableSirc32kCsr** (void)

Disable SIRC32KCSR register.

LOCAL_INLINE void **SCG_HWA_UnlockSirc32kCsr** (void)

Unlock SIRC32K CSR register.

LOCAL_INLINE void **SCG_HWA_LockSirc32kCsr** (void)

Lock SIRC32K CSR register.

LOCAL_INLINE uint32_t **SCG_HWA_GetFircCsr** (void)

Get FIRC CSR register value.

LOCAL_INLINE void **SCG_HWA_EnableFirc** (void)

Enable FIRC.

LOCAL_INLINE void **SCG_HWA_UnlockFircCsr** (void)

Unlock FIRC CSR register.

LOCAL_INLINE void **SCG_HWA_LockFircCsr** (void)

Lock FIRC CSR register.

LOCAL_INLINE void **SCG_HWA_EnableFircClockMonitor** (void)

Enable FIRC clock monitor.

LOCAL_INLINE void **SCG_HWA_DisableFirc** (void)

Disable FIRC.

LOCAL_INLINE void **SCG_HWA_SetFircCsr** (uint32_t u32CsrValue)

Set FIRCCSR register value.

LOCAL_INLINE void **SCG_HWA_SetFircCfg** (uint32_t u32CfgValue)

Set FIRCCFG register value.

LOCAL_INLINE void **SCG_HWA_SetFircDiv** (uint32_t u32DivValue)

Set FOSCDIV register value.

LOCAL_INLINE void **SCG_HWA_DiableFircDiv** (void)

Disable FIRCDIV as user manual request sequence.

LOCAL_INLINE void **SCG_HWA_EnableFircDiv** (void)

enable FIRCDIV as user manual request sequence

LOCAL_INLINE void **SCG_HWA_SetFircTcfg** (uint32_t u32TcfgValue)

Set FIRCTCFG register value for FIRC clock trim configure.

LOCAL_INLINE void **SCG_HWA_SetCCR** (uint32_t u32CcrValue)

Set system clock CCR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetCCR** (void)

Set system clock CCR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetPll0Csr** (void)

Get PLL0CSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetPll1Csr** (void)

Get PLL1CSR register value.

LOCAL_INLINE void **SCG_HWA_SetSystemClock** (uint8_t u8SystemClock)

Set system clock.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkSrc** (void)

Get system clock source. used to calculate system clock frequency at startup. used to check if the target clock source successfully switched.

LOCAL_INLINE bool **SCG_HWA_GetSysClkUPRD** (void)

Get system clock valid status, use this status to check system clock update finished or not.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkDivCore** (void)

Get system clock Divcore. used to calculate core clock frequency at startup.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkDivBus** (void)

Get system clock Divbus. used to calculate bus clock frequency at startup.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkDivSlow** (void)

Get system clock Divslow. used to calculate slow clock frequency at startup.

LOCAL_INLINE void **SCG_HWA_SetClkOutSel** (uint8_t u8ClkOutSel)

Select clock out source.

LOCAL_INLINE void **SCG_HWA_SetNvmClk** (uint32_t u32NvmClkMask)

Set Nvm clock source.

LOCAL_INLINE void **SCG_HWA_SetCmu4Clk** (uint32_t u32Cmu4ClkMask)

Set CMU4 clock source.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearSircClkErr** (void)

Check and clear SIRC clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearSoscClkErr** (void)

Check and clear SOSC clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearFoscClkErr** (void)

Check and clear FOSC clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearPll0ClkErr** (void)

Check and clear PLL0 clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearPll1ClkErr** (void)

Check and clear PLL1 clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearFircClkErr** (void)

Check and clear FIRC clock error.

LOCAL_INLINE void **SCG_HWA_GenCrcVal** (void)

Generate SCG register CRC value.

LOCAL_INLINE void **SCG_HWA_EnableCrcCheck** (void)

Enable SCG register CRC check.

LOCAL_INLINE void **SCG_HWA_DisableCrcCheck** (void)

Disable SCG register CRC check.

LOCAL_INLINE void **SCG_HWA_EnableCrcTrigger** (void)

Enable SCG register CRC hardware trigger generate.

LOCAL_INLINE void **SCG_HWA_DisableCrcTrigger** (void)

Disable SCG register CRC hardware trigger generate.

LOCAL_INLINE void **SCG_HWA_EnableCrcErrorOutput** (void)

Enable SCG register CRC error output.

LOCAL_INLINE void **SCG_HWA_DisableCrcErrorOutput** (void)

Disable SCG register CRC error output.

LOCAL_INLINE bool **SCG_HWA_GetCrcBusyStatus** (void)

Get CRC busy status.

LOCAL_INLINE bool **SCG_HWA_GetCrcErrorStatus** (void)

Get CRC error status.

LOCAL_INLINE void **SCG_HWA_ClearCrcErrorFlag** (void)

Clear CRC error flag.

LOCAL_INLINE uint32_t **SCG_HWA_GetCrcResult** (void)

Get CRC result.

Detailed Description

Macro Definition Documentation

```
#define _SCG_HWA_DisablePllDiv( _pll_)  SCG->_pll_##DIV &= (~(uint32_t)(SCG_PLLDIV_DIVL_EN_MASK |  
SCG_PLLDIV_DIVM_EN_MASK | SCG_PLLDIV_DIVH_EN_MASK))  
  
#define _SCG_HWA_EnablePll( _pll_)  SCG->_pll_##CSR |= (uint32_t)(SCG_PLLCSR_EN_MASK)  
  
#define _SCG_HWA_EnablePllClockMonitor( _pll_)  SCG->_pll_##CSR |= (uint32_t)SCG_PLLCSR_CM_MASK  
  
#define _SCG_HWA_EnablePllClockMonitorReset( _pll_)  SCG->_pll_##CSR |=  
(uint32_t)SCG_PLLCSR_CMRE_MASK  
  
#define _SCG_HWA_EnablePllDiv( _pll_)  SCG->_pll_##DIV |= (uint32_t)(SCG_PLLDIV_DIVL_EN_MASK |  
SCG_PLLDIV_DIVM_EN_MASK | SCG_PLLDIV_DIVH_EN_MASK)  
  
#define _SCG_HWA_GetClockDiv( _clock_)  (uint32_t)SCG->_clock_##DIV  
  
#define _SCG_HWA_GetClockValid( _clock_)  ((bool)((((uint32_t)SCG->_clock_##CSR &  
(uint32_t)SCG_##_clock_##CSR_VLD_MASK) != 0U) ? true : false))  
  
#define _SCG_HWA_GetPllLocked( _clock_)  ((bool)((((uint32_t)SCG->_clock_##CSR &  
(uint32_t)SCG_##_clock_##CSR_PLLLK_MASK) != 0U) ? true : false))  
  
#define _SCG_HWA_GetPllMult( _pll_)  ((uint16_t)((((uint32_t)(SCG->_pll_##CFG) &  
SCG_PLLCFG_MULT_MASK) >> SCG_PLLCFG_MULT_SHIFT))  
  
#define _SCG_HWA_GetPllPrediv( _pll_)  ((uint8_t)((((uint32_t)(SCG->_pll_##CFG) &  
SCG_PLLCFG_PREDIV_MASK) >> SCG_PLLCFG_PREDIV_SHIFT))  
  
#define _SCG_HWA_GetPllPstdiv( _pll_)  ((uint8_t)((((uint32_t)(SCG->_pll_##CFG) &  
SCG_PLLCFG_PSTDIV_MASK) >> SCG_PLLCFG_PSTDIV_SHIFT))  
  
#define _SCG_HWA_GetPllPstdiv1( _pll_)  ((uint8_t)((((uint32_t)(SCG->_pll_##CFG) &  
SCG_PLLCFG_PSTDIV1_MASK) >> SCG_PLLCFG_PSTDIV1_SHIFT))  
  
#define _SCG_HWA_GetPllSrc( _pll_)  ((uint8_t)((((uint32_t)(SCG->_pll_##CFG) &  
SCG_PLLCFG_SOURCE_MASK) >> SCG_PLLCFG_SOURCE_SHIFT))  
  
#define _SCG_HWA_GetTrimLocked( _clock_)  ((bool)((((uint32_t)SCG->_clock_##CSR &  
(uint32_t)SCG_##_clock_##CSR_TRLK_MASK) != 0U) ? true : false))
```

```

#define _SCG_HWA_LockPllCsr( _pll_)  SCG->_pll_##CSR |= (uint32_t)SCG_PLLCSR_LK_MASK

#define _SCG_HWA_SetPllCfg( _pll_,  val)  SCG->_pll_##CFG = val

#define _SCG_HWA_SetPllCsr( _pll_,  val)  SCG->_pll_##CSR = val

#define _SCG_HWA_SetPllDiv( _pll_,  val)  SCG->_pll_##DIV = val

#define _SCG_HWA_UnlockPllCsr( _pll_)  SCG->_pll_##CSR &= ~(uint32_t)SCG_PLLCSR_LK_MASK

#define SCG_CALCULATE_DIVH_FREQ( clk_freq,  reg_val)  (((1U == SCG_CHECK_DIVH_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVH_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)

#define SCG_CALCULATE_DIVL_FREQ( clk_freq,  reg_val)  (((1U == SCG_CHECK_DIVL_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVL_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)

#define SCG_CALCULATE_DIVM_FREQ( clk_freq,  reg_val)  (((1U == SCG_CHECK_DIVM_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVM_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)

#define SCG_CHECK_DIV_ACK( reg_val)  ((reg_val & SCG_CLOCKDIV_DIV_ACK_MASK) >>
SCG_CLOCKDIV_DIV_ACK_SHIFT)

#define SCG_CHECK_DIVH_ACK( reg_val)  ((reg_val & SCG_CLOCKDIV_DIVH_ACK_MASK) >>
SCG_CLOCKDIV_DIVH_ACK_SHIFT)

#define SCG_CHECK_DIVH_EN( reg_val)  ((reg_val & SCG_CLOCKDIV_DIVH_EN_MASK) >>
SCG_CLOCKDIV_DIVH_EN_SHIFT)

#define SCG_CHECK_DIVL_ACK( reg_val)  ((reg_val & SCG_CLOCKDIV_DIVL_ACK_MASK) >>
SCG_CLOCKDIV_DIVL_ACK_SHIFT)

#define SCG_CHECK_DIVL_EN( reg_val)  ((reg_val & SCG_CLOCKDIV_DIVL_EN_MASK) >>
SCG_CLOCKDIV_DIVL_EN_SHIFT)

#define SCG_CHECK_DIVM_ACK( reg_val)  ((reg_val & SCG_CLOCKDIV_DIVM_ACK_MASK) >>
SCG_CLOCKDIV_DIVM_ACK_SHIFT)

```



```

#define SCG_CHECK_DIVM_EN( reg_val)  ((reg_val & SCG_CLOCKDIV_DIVM_EN_MASK) >>
SCG_CLOCKDIV_DIVM_EN_SHIFT)

#define SCG_CLOCKDIV_DIV_ACK_MASK  0x70000000u

#define SCG_CLOCKDIV_DIV_ACK_MASK_SHIFT  0x7u

#define SCG_CLOCKDIV_DIV_ACK_SHIFT  28u

#define SCG_CLOCKDIV_DIVH_ACK_MASK  0x10000000u

#define SCG_CLOCKDIV_DIVH_ACK_SHIFT  28u

#define SCG_CLOCKDIV_DIVH_EN_MASK  0x1000000u

#define SCG_CLOCKDIV_DIVH_EN_SHIFT  24u

#define SCG_CLOCKDIV_DIVH_VAL_MASK  0x7u

#define SCG_CLOCKDIV_DIVH_VAL_SHIFT  0u

#define SCG_CLOCKDIV_DIVL_ACK_MASK  0x40000000u

#define SCG_CLOCKDIV_DIVL_ACK_SHIFT  30u

#define SCG_CLOCKDIV_DIVL_EN_MASK  0x4000000u

#define SCG_CLOCKDIV_DIVL_EN_SHIFT  26u

#define SCG_CLOCKDIV_DIVL_VAL_MASK  0x70000u

#define SCG_CLOCKDIV_DIVL_VAL_SHIFT  16u

#define SCG_CLOCKDIV_DIVM_ACK_MASK  0x20000000u

#define SCG_CLOCKDIV_DIVM_ACK_SHIFT  29u

#define SCG_CLOCKDIV_DIVM_EN_MASK  0x2000000u

#define SCG_CLOCKDIV_DIVM_EN_SHIFT  25u

#define SCG_CLOCKDIV_DIVM_VAL_MASK  0x700u

```

```

#define SCG_CLOCKDIV_DIVM_VAL_SHIFT 8u

#define SCG_GET_DIVH_VAL( reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVH_VAL_MASK) >>
SCG_CLOCKDIV_DIVH_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVH_VAL_MASK) >>
SCG_CLOCKDIV_DIVH_VAL_SHIFT))

#define SCG_GET_DIVL_VAL( reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVL_VAL_MASK) >>
SCG_CLOCKDIV_DIVL_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVL_VAL_MASK) >>
SCG_CLOCKDIV_DIVL_VAL_SHIFT))

#define SCG_GET_DIVM_VAL( reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVM_VAL_MASK) >>
SCG_CLOCKDIV_DIVM_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVM_VAL_MASK) >>
SCG_CLOCKDIV_DIVM_VAL_SHIFT))

#define SCG_HWA_DisablePllDiv( pll) _SCG_HWA_DisablePllDiv(pll)

#define SCG_HWA_EnablePll( pll) _SCG_HWA_EnablePll(pll)

#define SCG_HWA_EnablePllClockMonitor( pll) _SCG_HWA_EnablePllClockMonitor(pll)

#define SCG_HWA_EnablePllClockMonitorReset( pll) _SCG_HWA_EnablePllClockMonitorReset(pll)

#define SCG_HWA_EnablePllDiv( pll) _SCG_HWA_EnablePllDiv(pll)

#define SCG_HWA_GetClockDiv( _clock_) _SCG_HWA_GetClockDiv(_clock_)

#define SCG_HWA_GetClockValid( _clock_) _SCG_HWA_GetClockValid(_clock_)

#define SCG_HWA_GetPllLocked( _clock_) _SCG_HWA_GetPllLocked(_clock_)

#define SCG_HWA_GetPllMult( pll) _SCG_HWA_GetPllMult(pll)

#define SCG_HWA_GetPllPrediv( pll) _SCG_HWA_GetPllPrediv(pll)

#define SCG_HWA_GetPllPstdiv( pll) _SCG_HWA_GetPllPstdiv(pll)

#define SCG_HWA_GetPllPstdiv1( pll) _SCG_HWA_GetPllPstdiv1(pll)

#define SCG_HWA_GetPllSrc( pll) _SCG_HWA_GetPllSrc(pll)

#define SCG_HWA_GetTrimLocked( _clock_) _SCG_HWA_GetTrimLocked(_clock_)

```

```

#define SCG_HWA_LockPllCsr( pll)  _SCG_HWA_LockPllCsr(pll)

#define SCG_HWA_SetPllCfg( pll,  val)  _SCG_HWA_SetPllCfg(pll, val)

#define SCG_HWA_SetPllCsr( pll,  val)  _SCG_HWA_SetPllCsr(pll, val)

#define SCG_HWA_SetPllDiv( pll,  val)  _SCG_HWA_SetPllDiv(pll, val)

#define SCG_HWA_UnlockPllCsr( pll)  _SCG_HWA_UnlockPllCsr(pll)

#define
SCG_PLLCFG_MULT( x)  (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_MULT_SHIFT))&SCG_PLLCFG_MULT_
MASK)

#define SCG_PLLCFG_MULT_MASK  0x1FF0000u

#define SCG_PLLCFG_MULT_SHIFT  16u

#define
SCG_PLLCFG_PREDIV( x)  (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PREDIV_SHIFT))&SCG_PLLCFG_PRE
DIV_MASK)

#define SCG_PLLCFG_PREDIV_MASK  0x1F00u

#define SCG_PLLCFG_PREDIV_SHIFT  8u

#define
SCG_PLLCFG_PSTDIV( x)  (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PSTDIV_SHIFT))&SCG_PLLCFG_PSTD
IV_MASK)

#define
SCG_PLLCFG_PSTDIV1( x)  (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PSTDIV1_SHIFT))&SCG_PLLCFG_PS
TDIV1_MASK)

#define SCG_PLLCFG_PSTDIV1_MASK  0xEu

#define SCG_PLLCFG_PSTDIV1_SHIFT  1u

#define SCG_PLLCFG_PSTDIV_MASK  0x30u

#define SCG_PLLCFG_PSTDIV_SHIFT  4u

```

```

#define
SCG_PLLCFG_SOURCE(x) (((uint32_t)((((uint32_t)(x))<<SCG_PLLCFG_SOURCE_SHIFT))&SCG_PLLCFG_SOURCE_MASK)

#define SCG_PLLCFG_SOURCE_MASK 0x1u

#define SCG_PLLCFG_SOURCE_SHIFT 0u

#define
SCG_PLLCSR_CM(x) (((uint32_t)((((uint32_t)(x))<<SCG_PLLCSR_CM_SHIFT))&SCG_PLLCSR_CM_MASK)

#define SCG_PLLCSR_CM_MASK 0x10000u

#define SCG_PLLCSR_CM_SHIFT 16u

#define
SCG_PLLCSR_CMRE(x) (((uint32_t)((((uint32_t)(x))<<SCG_PLLCSR_CMRE_SHIFT))&SCG_PLLCSR_CMRE_MASK)

#define SCG_PLLCSR_CMRE_MASK 0x20000u

#define SCG_PLLCSR_CMRE_SHIFT 17u

#define SCG_PLLCSR_EN_MASK 0x1u

#define SCG_PLLCSR_LK_MASK 0x800000u

```

PLL related.

```

#define
SCG_PLLCSR_STEN( x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCSR_STEN_SHIFT))&SCG_PLLCSR_STEN_M
ASK)

#define SCG_PLLCSR_STEN_MASK 0x2u

#define SCG_PLLCSR_STEN_SHIFT 1u

#define
SCG_PLLDIV_DIVH( x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVH_SHIFT))&SCG_PLLDIV_DIVH_MASK
)

#define SCG_PLLDIV_DIVH_EN_MASK 0x1000000u

#define SCG_PLLDIV_DIVH_MASK 0x7u

#define SCG_PLLDIV_DIVH_SHIFT 0u

#define
SCG_PLLDIV_DIVL( x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVL_SHIFT))&SCG_PLLDIV_DIVL_MASK)

#define SCG_PLLDIV_DIVL_EN_MASK 0x4000000u

#define SCG_PLLDIV_DIVL_MASK 0x70000u

#define SCG_PLLDIV_DIVL_SHIFT 16u

#define
SCG_PLLDIV_DIVM( x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVM_SHIFT))&SCG_PLLDIV_DIVM_MAS
K)

#define SCG_PLLDIV_DIVM_EN_MASK 0x2000000u

#define SCG_PLLDIV_DIVM_MASK 0x700u

#define SCG_PLLDIV_DIVM_SHIFT 8u

```

Function Documentation

LOCAL_INLINE bool SCG_HWA_CheckAndClearFircClkErr (void)

Check and clear FIRC clock error.

LOCAL_INLINE bool SCG_HWA_CheckAndClearFoscClkErr (void)

Check and clear FOSC clock error.

LOCAL_INLINE bool SCG_HWA_CheckAndClearPll0ClkErr (void)

Check and clear PLL0 clock error.

LOCAL_INLINE bool SCG_HWA_CheckAndClearPll1ClkErr (void)

Check and clear PLL1 clock error.

LOCAL_INLINE bool SCG_HWA_CheckAndClearSircClkErr (void)

Check and clear SIRC clock error.

LOCAL_INLINE bool SCG_HWA_CheckAndClearSoscClkErr (void)

Check and clear SOSC clock error.

LOCAL_INLINE void SCG_HWA_ClearCrcErrorFlag (void)

Clear CRC error flag.

LOCAL_INLINE void SCG_HWA_DiableFircDiv (void)

Disable FIRCDIV as user manual request sequence.

LOCAL_INLINE void SCG_HWA_DiableFoscDiv (void)

Disable FOSCDIV as user manual request sequence.

LOCAL_INLINE void SCG_HWA_DiableSircDiv (void)

Disable SIRCDIV as user manual request sequence.

LOCAL_INLINE void SCG_HWA_DisableCrcCheck (void)

Disable SCG register CRC check.

LOCAL_INLINE void SCG_HWA_DisableCrcErrorOutput (void)

Disable SCG register CRC error output.

LOCAL_INLINE void SCG_HWA_DisableCrcTrigger (void)

Disable SCG register CRC hardware trigger generate.

LOCAL_INLINE void SCG_HWA_DisableFirc (void)

Disable FIRC.

LOCAL_INLINE void SCG_HWA_DisableFosc (void)

Disable FOSC.

LOCAL_INLINE void SCG_HWA_DisableSirc32kCsr (void)

Disable SIRC32KCSR register.

LOCAL_INLINE void SCG_HWA_DisableSosc (void)

Diabie SOSC.

LOCAL_INLINE void SCG_HWA_EnableCrcCheck (void)

Enable SCG register CRC check.

LOCAL_INLINE void SCG_HWA_EnableCrcErrorOutput (void)

Enable SCG register CRC error output.

LOCAL_INLINE void SCG_HWA_EnableCrcTrigger (void)

Enable SCG register CRC hardware trigger generate.

LOCAL_INLINE void SCG_HWA_EnableFirc (void)

Enable FIRC.

LOCAL_INLINE void SCG_HWA_EnableFircClockMonitor (void)

Enable FIRC clock monitor.

LOCAL_INLINE void SCG_HWA_EnableFircDiv (void)

enable FIRCDIV as user manual request sequence

LOCAL_INLINE void SCG_HWA_EnableFosc (void)

Enable FOSC.

LOCAL_INLINE void SCG_HWA_EnableFoscClockMonitor (void)

Enable FOSC clock monitor.

LOCAL_INLINE void SCG_HWA_EnableFoscClockMonitorReset (void)

Enable FOSC clock monitor Reset.

LOCAL_INLINE void SCG_HWA_EnableFoscDiv (void)

enable FOSCDIV as user manual request sequence

LOCAL_INLINE void SCG_HWA_EnableSirc32kCsr (void)

Enable SIRC32KCSR register.

LOCAL_INLINE void SCG_HWA_EnableSircClockMonitor (void)

Enable SIRC clock monitor.

LOCAL_INLINE void SCG_HWA_EnableSircDiv (void)

enable FIRCDIV as user manual request sequence

LOCAL_INLINE void SCG_HWA_EnableSosc (void)

Set SOSC enable.

LOCAL_INLINE void SCG_HWA_EnableSoscClockMonitor (void)

Enable SOSC clock monitor.

LOCAL_INLINE void SCG_HWA_EnableSoscClockMonitorReset (void)

Enable SOSC clock monitor Reset.

LOCAL_INLINE void SCG_HWA_GenCrcVal (void)

Generate SCG register CRC value.

LOCAL_INLINE uint32_t SCG_HWA_GetCCR (void)

Set system clock CCR register value.

Parameters

<i>u32CcrValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE uint32_t SCG_HWA_GetClkOutCfg (void)

Get clock out configure register value.

LOCAL_INLINE uint8_t SCG_HWA_GetClkOutSel (void)

Get clock out configure register CLKOUTSEL value.

LOCAL_INLINE bool SCG_HWA_GetCrcBusyStatus (void)

Get CRC busy status.

LOCAL_INLINE bool SCG_HWA_GetCrcErrorStatus (void)

Get CRC error status.

LOCAL_INLINE uint32_t SCG_HWA_GetCrcResult (void)

Get CRC result.

LOCAL_INLINE uint32_t SCG_HWA_GetFircCsr (void)

Get FIRC CSR register value.

LOCAL_INLINE uint32_t SCG_HWA_GetFoscCsr (void)

Get FOSCCSR register value.

LOCAL_INLINE uint32_t SCG_HWA_GetPll0Csr (void)

Get PLL0CSR register value.

Returns

PLL0CSR register value.

LOCAL_INLINE uint32_t SCG_HWA_GetPll1Csr (void)

Get PLL1CSR register value.

Returns

PLL1CSR register value.

LOCAL_INLINE uint32_t SCG_HWA_GetSircCsr (void)

Get SIRCCSR register value.

LOCAL_INLINE uint32_t SCG_HWA_GetSoscCsr (void)

Get SOSC CSR register value.

LOCAL_INLINE uint8_t SCG_HWA_GetSysClkDivBus (void)

Get system clock Divbus. used to calculate bus clock frequency at startup.

Returns

uint8_t. system clock Divbus.

LOCAL_INLINE uint8_t SCG_HWA_GetSysClkDivCore (void)

Get system clock Divcore. used to calculate core clock frequency at startup.

Returns

uint8_t. system clock Divcore.

LOCAL_INLINE uint8_t SCG_HWA_GetSysClkDivSlow (void)

Get system clock Divslow. used to calculate slow clock frequency at startup.

Returns

uint8_t. system clock Divslow.

LOCAL_INLINE uint8_t SCG_HWA_GetSysClkSrc (void)

Get system clock source. used to calculate system clock frequency at startup. used to check if the target clock source successfully switched.

Returns

uint8_t. system clock source.

LOCAL_INLINE bool SCG_HWA_GetSysClkUPRD (void)

Get system clock valid status, use this status to check system clock update finished or not.

Returns

bool. true as updated; false as not updated.

LOCAL_INLINE void SCG_HWA_LockFircCsr (void)

Lock FIRC CSR register.

LOCAL_INLINE void SCG_HWA_LockFoscCsr (void)

lock FOSC CSR register

LOCAL_INLINE void SCG_HWA_LockSirc32kCsr (void)

Lock SIRC32K CSR register.

LOCAL_INLINE void SCG_HWA_LockSircCsr (void)

Lock SIRC CSR register.

LOCAL_INLINE void SCG_HWA_LockSoscCsr (void)

Lock SOSC CSR register.

LOCAL_INLINE void SCG_HWA_SetCCR (uint32_t *u32CcrValue*)

Set system clock CCR register value.

Parameters

<i>u32CcrValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetClkOutSel (uint8_t *u8ClkOutSel*)

Select clock out source.

Parameters

<i>u8ClkOutSel</i>	Clock out select
--------------------	------------------

LOCAL_INLINE void SCG_HWA_SetCmu4Clk (uint32_t *u32Cmu4ClkMask*)

Set CMU4 clock source.

Parameters

<i>u32Cmu4ClkMask</i>	CMU4 clock source mask
<i>sk</i>	

LOCAL_INLINE void SCG_HWA_SetFircCfg (uint32_t *u32CfgValue*)

Set FIRCCFG register value.

Parameters

<i>u32CfgValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetFircCsr (uint32_t *u32CsrValue*)

Set FIRCCSR register value.

Parameters

<i>u32CsrValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetFircDiv (uint32_t *u32DivValue*)

Set FOSCDIV register value.

Parameters

<i>u32DivValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetFircTcfg (uint32_t *u32TcfgValue*)

Set FIRCTCFG register value for FIRC clock trim configure.

Parameters

<i>u32TcfgValue</i>	configured register value.
---------------------	----------------------------

LOCAL_INLINE void SCG_HWA_SetFoscCfg (uint32_t *u32CfgValue*)

Set FOSCCFG register value.

Parameters

<i>u32CfgValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetFoscCsr (uint32_t *u32CsrValue*)

Set FOSCCSR register value.

Parameters

<i>u32CsrValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetFoscDiv (uint32_t *u32DivValue*)

Set FOSCDIV register value.

Parameters

<i>u32DivValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetNvmClk (uint32_t *u32NvmClkMask*)

Set Nvm clock source.

Parameters

<i>u32NvmClkMask</i>	Nvm clock source mask
----------------------	-----------------------

LOCAL_INLINE void SCG_HWA_SetSircCsr (uint32_t *u32CsrValue*)

Set SIRCCSR register value.

Parameters

<i>u32CsrValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetSircDiv (uint32_t *u32DivValue*)

Set SIRCDIV register value.

Parameters

<i>u32DivValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetSircTcfg (uint32_t *u32TcfgValue*)

Set SIRCTCFG register value for SIRC Trim configure.

Parameters

<i>u32TcfgValue</i>	configured register value
---------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetSoscCfg (uint32_t *u32CfgValue*)

Set SOSC CFG register value.

Parameters

<i>u32CfgValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetSoscCsr (uint32_t *u32CsrValue*)

Set SOSC CSR register value.

Parameters

<i>u32CsrValue</i>	configured register value
--------------------	---------------------------

LOCAL_INLINE void SCG_HWA_SetSystemClock (uint8_t *u8SystemClock*)

Set system clock.

Parameters

<i>u8SystemClock</i>	System clock value
----------------------	--------------------

LOCAL_INLINE void SCG_HWA_UnlockFircCsr (void)

Unlock FIRC CSR register.

LOCAL_INLINE void SCG_HWA_UnlockFoscCsr (void)

Unlock FOSC CSR register.

LOCAL_INLINE void SCG_HWA_UnlockSirc32kCsr (void)

Unlock SIRC32K CSR register.

LOCAL_INLINE void SCG_HWA_UnlockSircCsr (void)

Unlock SIRC CSR register.

LOCAL_INLINE void SCG_HWA_UnlockSoscCsr (void)

Unlock SOSC CSR register.

HwA_scm

Collaboration diagram for HwA_scm:Functions

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDL (void)

Get UIDL data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDML (void)

Get UIDML data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDMH (void)

Get UIDMH data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDH (void)

Get UIDH data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint8_t SCM_HWA_GetData_FamilyID (void)

Get Family Identification.

LOCAL_INLINE uint8_t SCM_HWA_GetData_RevID (void)

Get Revision Identification.

LOCAL_INLINE uint32_t SCM_HWA_Get_CHIPCFG0 (void)

Get CHIPCFG0 register status.

LOCAL_INLINE void SCM_HWA_Set_SOCMISC (uint32_t u32Val)

Set the SOCMISC register value.

LOCAL_INLINE uint32_t **SCM_HWA_Get_SOCMISC** (void)

Get the SOCMISC register status.

LOCAL_INLINE uint32_t **SCM_HWA_Get_SYSAP_CTRL** (void)

Get SYSAP_CTRL register status.

LOCAL_INLINE void **SCM_HWA_Set_FTU_GTBC** (uint32_t u32Val)

Set the FTU_GTBC register value.

LOCAL_INLINE void **SCM_HWA_SUBSYS_PCC_SetEnable_TPUClock** (bool bEnable)

Set TPU clock enable/disable.

LOCAL_INLINE void **SCM_HWA_SUBSYS_PCC_SetEnable_SubSystemClock** (bool bEnable)

Set SubSystem clock enable/disable.

Detailed Description

Function Documentation

LOCAL_INLINE uint32_t **SCM_HWA_Get_CHIPCFG0** (void)

Get CHIPCFG0 register status.

Returns

return value

LOCAL_INLINE uint32_t **SCM_HWA_Get_SOCMISC** (void)

Get the SOCMISC register status.

Returns

Lock status

LOCAL_INLINE uint32_t SCM_HWA_Get_SYSAP_CTRL (void)

Get SYSAP_CTRL register status.

Returns

Register status

LOCAL_INLINE uint8_t SCM_HWA_GetData_FamilyID (void)

Get Family Identification.

Returns

return value

LOCAL_INLINE uint8_t SCM_HWA_GetData_RevID (void)

Get Revision Identification.

Returns

return value

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDH (void)

Get UIDH data(Unique identification for the chip. Loaded from NVR)

Returns

return value

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDL (void)

Get UIDL data(Unique identification for the chip. Loaded from NVR)

Returns

return value

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDMH (void)

Get UIDMH data(Unique identification for the chip. Loaded from NVR)

Returns

return value

LOCAL_INLINE uint32_t SCM_HWA_GetData_UIDML (void)

Get UIDML data(Unique identification for the chip. Loaded from NVR)

Returns

return value

LOCAL_INLINE void SCM_HWA_Set_FTU_GTBC (uint32_t *u32Val*)

Set the FTU_GTBC register value.

Parameters

<i>u32Val</i>	value to be set
---------------	-----------------

LOCAL_INLINE void SCM_HWA_Set_SOCMISC (uint32_t *u32Val*)

Set the SOCMISC register value.

Parameters

<i>u32Val</i>	value to be set
---------------	-----------------

LOCAL_INLINE void SCM_HWA_SUBSYS_PCC_SetEnable_SubSystemClock (bool *bEnable*)

Set SubSystem clock enable/disable.

Parameters

<i>bEnable</i>	Enable/Disable
----------------	----------------

LOCAL_INLINE void SCM_HWA_SUBSYS_PCC_SetEnable_TPUClock (bool *bEnable*)

Set TPU clock enable/disable.

Parameters

<i>bEnable</i>	Enable/Disable
----------------	----------------

HwA_sdadc

Collaboration diagram for HwA_sdadc:Classes

struct **SDADC_ChannelRegType**

Macros

```
#define SDADC_CHANNELCFGREG(base) ((pSDADC_ChannelRegType)((int)&((SDADC_Type *)0)->CCR0  
+ (int)(base)))
```

Typedefs

```
typedef struct SDADC_ChannelRegType * pSDADC_ChannelRegType
```

Functions

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO2_DIS** (const SDADC_Type *const pSdadc)

Get the value of CLKO2_DIS. Modulator Clock Output 2 Disable 0b - CLKO2 is enabled when MEN is asserted. 1b - CLKO2 is disabled.

LOCAL_INLINE void **SDADC_HWA_SetCLKO2_DIS** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set CLKO2_DIS. Modulator Clock Output 2 Disable 0b - CLKO2 is enabled when MEN is asserted. 1b - CLKO2 is disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO1_DIS** (const SDADC_Type *const pSadc)

Get the value of CLKO1_DIS. Modulator Clock Output 1 Disable 0b - CLKO1 is enabled when MEN is asserted. 1b - CLKO1 is disabled.

LOCAL_INLINE void **SDADC_HWA_SetCLKO1_DIS** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CLKO1_DIS. Modulator Clock Output 1 Disable 0b - CLKO1 is enabled when MEN is asserted. 1b - CLKO1 is disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO0_DIS** (const SDADC_Type *const pSadc)

Get the value of CLKO0_DIS. Modulator Clock Output 0 Disable 0b - CLKO0 is enabled when MEN is asserted. 1b - CLKO0 is disabled.

LOCAL_INLINE void **SDADC_HWA_SetCLKO0_DIS** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CLKO0_DIS. Modulator Clock Output 0 Disable 0b - CLKO0 is enabled when MEN is asserted. 1b - CLKO0 is disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPRESCALE** (const SDADC_Type *const pSadc)

Get the value of PRESCALE. PRE_DIV Divider Value 00b - No prescale. 01b - Divided by 2. 10b - Divided by 4. 11b - Divided by 8.

LOCAL_INLINE void **SDADC_HWA_SetPRESCALE** (SDADC_Type *const pSadc, uint32_t u32Value)

Set PRESCALE. PRE_DIV Divider Value 00b - No prescale. 01b - Divided by 2. 10b - Divided by 4. 11b - Divided by 8.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCLK_DIV** (const SDADC_Type *const pSadc)

Get the value of MCLK_DIV. DIV Divider Value The (MCLK_DIV+1) sets the clock divider ratio for DIV block. The minimum value MCLK_DIV is 1.

LOCAL_INLINE void **SDADC_HWA_SetMCLK_DIV** (SDADC_Type *const pSadc, uint32_t u32Value)

Set MCLK_DIV. DIV Divider Value The (MCLK_DIV+1) sets the clock divider ratio for DIV block. The minimum value MCLK_DIV is 1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMEN** (const SDADC_Type *const pSadc)

Get the value of MEN. Main Enable Setting this bit enables all Function block simultaneously if CHEN and corresponding enable bit are asserted. 0b - All function blocks are disabled. 1b - All function blocks are enabled simultaneously.

LOCAL_INLINE void **SDADC_HWA_SetMEN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set MEN. Main Enable Setting this bit enables all Function block simultaneously if CHEN and corresponding enable bit are asserted. 0b - All function blocks are disabled. 1b - All function blocks are enabled simultaneously.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMENSEL** (const SDADC_Type *const pSadc)

Get the value of MENSEL. Main Enable Selection Setting this bit to select main enable(MEN) from an enable from external of SDADC IP. By this way, all SDADC in a system can share one main enable and can be enable at the same time. 0b - Use internal MEN. 1b - Use external MEN.

LOCAL_INLINE void **SDADC_HWA_SetMENSEL** (SDADC_Type *const pSadc, uint32_t u32Value)

Set MENSEL. Main Enable Selection Setting this bit to select main enable(MEN) from an enable from external of SDADC IP. By this way, all SDADC in a system can share one main enable and can be enable at the same time. 0b - Use internal MEN. 1b - Use external MEN.

LOCAL_INLINE uint32_t **SDADC_HWA_GetRESET** (const SDADC_Type *const pSadc)

Get the value of RESET. Software Reset Used to reset all function blocks (filters, short circuit detect, etc) and all flags or status. Note, to reset all function domain, RESET must keep high at least 3 div_clk1 cycles. 0b - No reset. 1b - All function blocks are reset and disabled.

LOCAL_INLINE void **SDADC_HWA_SetRESET** (SDADC_Type *const pSadc, uint32_t u32Value)

Set RESET. Software Reset Used to reset all function blocks (filters, short circuit detect, etc) and all flags or status. Note, to reset all function domain, RESET must keep high at least 3 div_clk1 cycles. 0b - No reset. 1b - All function blocks are reset and disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDBGM** (const SDADC_Type *const pSadc)

Get the value of DBGM. Software Debug Mode Request This bit is writable only when MEN is asserted. 0b - SDADC is in Normal Mode. 1b - SDADC is in Debug Mode.

LOCAL_INLINE void **SDADC_HWA_SetDBGM** (SDADC_Type *const pSadc, uint32_t u32Value)

Set DBGM. Software Debug Mode Request This bit is writable only when MEN is asserted. 0b - SDADC is in Normal Mode. 1b - SDADC is in Debug Mode.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDBGE** (const SDADC_Type *const pSdadc)

Get the value of DBGE. Module Enable in Debug Mode Enables/disables the SDADC operation in Debug mode. 0b - SDADC is enabled in debug mode. 1b - SDADC is disabled in debug mode. Need trigger again for new conversion.

LOCAL_INLINE void **SDADC_HWA_SetDBGE** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set DBGE. Module Enable in Debug Mode Enables/disables the SDADC operation in Debug mode. 0b - SDADC is enabled in debug mode. 1b - SDADC is disabled in debug mode. Need trigger again for new conversion.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSTRIG2** (const SDADC_Type *const pSdadc)

Get the value of STRIG2. Software Trigger for Filter Channel 2.

LOCAL_INLINE void **SDADC_HWA_SetSTRIG2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set STRIG2. Software Trigger for Filter Channel 2.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSTRIG1** (const SDADC_Type *const pSdadc)

Get the value of STRIG1. Software Trigger for Filter Channel 1.

LOCAL_INLINE void **SDADC_HWA_SetSTRIG1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set STRIG1. Software Trigger for Filter Channel 1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSTRIG0** (const SDADC_Type *const pSdadc)

Get the value of STRIG0. Software Trigger for Filter Channel 0.

LOCAL_INLINE void **SDADC_HWA_SetSTRIG0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set STRIG0. Software Trigger for Filter Channel 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCDIE2** (const SDADC_Type *const pSdadc)

Get the value of ZCDIE2. Filter Channel 2 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetZCDIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set ZCDIE2. Filter Channel 2 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCDIE1** (const SDADC_Type *const pSadc)

Get the value of ZCDIE1. Filter Channel 1 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetZCDIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set ZCDIE1. Filter Channel 1 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCDIE0** (const SDADC_Type *const pSadc)

Get the value of ZCDIE0. Filter Channel 0 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetZCDIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set ZCDIE0. Filter Channel 0 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDYIE2** (const SDADC_Type *const pSadc)

Get the value of FRDYIE2. Filter Channel 2 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFRDYIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FRDYIE2. Filter Channel 2 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDYIE1** (const SDADC_Type *const pSadc)

Get the value of FRDYIE1. Filter Channel 1 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFRDYIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FRDYIE1. Filter Channel 1 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDYIE0** (const SDADC_Type *const pSadc)

Get the value of FRDYIE0. Filter Channel 0 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFRDYIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FRDYIE0. Filter Channel 0 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOCIE2** (const SDADC_Type *const pSadc)

Get the value of COCIE2. Filter Channel 2 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCOCIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set COCIE2. Filter Channel 2 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOCIE1** (const SDADC_Type *const pSadc)

Get the value of COCIE1. Filter Channel 1 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCOCIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set COCIE1. Filter Channel 1 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOCIE0** (const SDADC_Type *const pSadc)

Get the value of COCIE0. Filter Channel 0 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCOCIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set COCIE0. Filter Channel 0 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMTIE2** (const SDADC_Type *const pSadc)

Get the value of HLMTIE2. Filter Channel 2 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetHLMTIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set HLMTIE2. Filter Channel 2 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMTIE1** (const SDADC_Type *const pSadc)

Get the value of HLMTIE1. Filter Channel 1 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetHLMTIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set HLMTIE1. Filter Channel 1 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMTIE0** (const SDADC_Type *const pSadc)

Get the value of HLMTIE0. Filter Channel 0 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetHLMTIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set HLMTIE0. Filter Channel 0 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMTIE2** (const SDADC_Type *const pSadc)

Get the value of LLMTIE2. Filter Channel 2 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetLLMTIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set LLMTIE2. Filter Channel 2 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMTIE1** (const SDADC_Type *const pSadc)

Get the value of LLMTIE1. Filter Channel 1 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetLLMTIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set LLMTIE1. Filter Channel 1 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMTIE0** (const SDADC_Type *const pSdadc)

Get the value of LLMTIE0. Filter Channel 0 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetLLMTIE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set LLMTIE0. Filter Channel 0 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMTIE2** (const SDADC_Type *const pSdadc)

Get the value of WLMTIE2. Filter Channel 2 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetWLMTIE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set WLMTIE2. Filter Channel 2 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMTIE1** (const SDADC_Type *const pSdadc)

Get the value of WLMTIE1. Filter Channel 1 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetWLMTIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set WLMTIE1. Filter Channel 1 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMTIE0** (const SDADC_Type *const pSdadc)

Get the value of WLMTIE0. Filter Channel 0 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetWLMTIE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set WLMTIE0. Filter Channel 0 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDIE2** (const SDADC_Type *const pSadc)

Get the value of SCDIE2. Filter Channel 2 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSCDIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SCDIE2. Filter Channel 2 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDIE1** (const SDADC_Type *const pSadc)

Get the value of SCDIE1. Filter Channel 1 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSCDIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SCDIE1. Filter Channel 1 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDIE0** (const SDADC_Type *const pSadc)

Get the value of SCDIE0. Filter Channel 0 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSCDIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SCDIE0. Filter Channel 0 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSATIE2** (const SDADC_Type *const pSadc)

Get the value of SATIE2. Filter Channel 2 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSATIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SATIE2. Filter Channel 2 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSATIE1** (const SDADC_Type *const pSadc)

Get the value of SATIE1. Filter Channel 1 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSATIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SATIE1. Filter Channel 1 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSATIE0** (const SDADC_Type *const pSadc)

Get the value of SATIE0. Filter Channel 0 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSATIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SATIE0. Filter Channel 0 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADIE2** (const SDADC_Type *const pSadc)

Get the value of CADIE2. Filter Channel 2 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCADIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CADIE2. Filter Channel 2 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADIE1** (const SDADC_Type *const pSadc)

Get the value of CADIE1. Filter Channel 1 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCADIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CADIE1. Filter Channel 1 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADIE0** (const SDADC_Type *const pSadc)

Get the value of CADIE0. Filter Channel 0 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCADIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CADIE0. Filter Channel 0 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVFIE2** (const SDADC_Type *const pSadc)

Get the value of FOVFIE2. Filter Channel 2 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFOVFIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FOVFIE2. Filter Channel 2 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVFIE1** (const SDADC_Type *const pSadc)

Get the value of FOVFIE1. Filter Channel 1 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFOVFIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FOVFIE1. Filter Channel 1 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVFIE0** (const SDADC_Type *const pSadc)

Get the value of FOVFIE0. Filter Channel 0 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFOVFIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FOVFIE0. Filter Channel 0 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNFIE2** (const SDADC_Type *const pSadc)

Get the value of FUNFIE2. Filter Channel 2 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFUNFIE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set FUNFIE2. Filter Channel 2 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNFIE1** (const SDADC_Type *const pSdadc)

Get the value of FUNFIE1. Filter Channel 1 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFUNFIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set FUNFIE1. Filter Channel 1 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNFIE0** (const SDADC_Type *const pSdadc)

Get the value of FUNFIE0. Filter Channel 0 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFUNFIE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set FUNFIE0. Filter Channel 0 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD2** (const SDADC_Type *const pSdadc)

Get the value of ZCD2. Filter Channel 2 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearZCD2** (SDADC_Type *const pSdadc)

Clear ZCD2. Filter Channel 2 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD1** (const SDADC_Type *const pSdadc)

Get the value of ZCD1. Filter Channel 1 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearZCD1** (SDADC_Type *const pSdadc)

Clear ZCD1. Filter Channel 1 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD0** (const SDADC_Type *const pSadc)

Get the value of ZCD0. Filter Channel 0 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearZCD0** (SDADC_Type *const pSadc)

Clear ZCD0. Filter Channel 0 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDY2** (const SDADC_Type *const pSadc)

Get the value of FRDY2. Filter Channel 2 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFRDY2** (SDADC_Type *const pSadc)

Clear FRDY2. Filter Channel 2 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDY1** (const SDADC_Type *const pSadc)

Get the value of FRDY1. Filter Channel 1 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFRDY1** (SDADC_Type *const pSadc)

Clear FRDY1. Filter Channel 1 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDY0** (const SDADC_Type *const pSadc)

Get the value of FRDY0. Filter Channel 0 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFRDY0** (SDADC_Type *const pSadc)

Clear FRDY0. Filter Channel 0 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOC2** (const SDADC_Type *const pSadc)

Get the value of COC2. Filter Channel 2 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCOC2** (SDADC_Type *const pSadc)

Clear COC2. Filter Channel 2 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOC1** (const SDADC_Type *const pSdadc)

Get the value of COC1. Filter Channel 1 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCOC1** (SDADC_Type *const pSdadc)

Clear COC1. Filter Channel 1 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOC0** (const SDADC_Type *const pSdadc)

Get the value of COC0. Filter Channel 0 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCOC0** (SDADC_Type *const pSdadc)

Clear COC0. Filter Channel 0 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT2** (const SDADC_Type *const pSdadc)

Get the value of HLMT2. Filter Channel 2 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearHLMT2** (SDADC_Type *const pSdadc)

Clear HLMT2. Filter Channel 2 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT1** (const SDADC_Type *const pSdadc)

Get the value of HLMT1. Filter Channel 1 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearHLMT1** (SDADC_Type *const pSdadc)

Clear HLMT1. Filter Channel 1 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT0** (const SDADC_Type *const pSdadc)

Get the value of HLMT0. Filter Channel 0 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearHLMT0** (SDADC_Type *const pSdadc)

Clear HLMT0. Filter Channel 0 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT2** (const SDADC_Type *const pSadc)

Get the value of LLMT2. Filter Channel 2 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearLLMT2** (SDADC_Type *const pSadc)

Clear LLMT2. Filter Channel 2 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT1** (const SDADC_Type *const pSadc)

Get the value of LLMT1. Filter Channel 1 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearLLMT1** (SDADC_Type *const pSadc)

Clear LLMT1. Filter Channel 1 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT0** (const SDADC_Type *const pSadc)

Get the value of LLMT0. Filter Channel 0 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearLLMT0** (SDADC_Type *const pSadc)

Clear LLMT0. Filter Channel 0 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT2** (const SDADC_Type *const pSadc)

Get the value of WLMT2. Filter Channel 2 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearWLMT2** (SDADC_Type *const pSadc)

Clear WLMT2. Filter Channel 2 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT1** (const SDADC_Type *const pSadc)

Get the value of WLMT1. Filter Channel 1 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearWLMT1** (SDADC_Type *const pSadc)

Clear WLMT1. Filter Channel 1 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT0** (const SDADC_Type *const pSadc)

Get the value of WLMT0. Filter Channel 0 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearWLMT0** (SDADC_Type *const pSadc)

Clear WLMT0. Filter Channel 0 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD2** (const SDADC_Type *const pSadc)

Get the value of SCD2. Filter Channel 2 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSCD2** (SDADC_Type *const pSadc)

Clear SCD2. Filter Channel 2 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD1** (const SDADC_Type *const pSadc)

Get the value of SCD1. Filter Channel 1 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSCD1** (SDADC_Type *const pSadc)

Clear SCD1. Filter Channel 1 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD0** (const SDADC_Type *const pSadc)

Get the value of SCD0. Filter Channel 0 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSCD0** (SDADC_Type *const pSadc)

Clear SCD0. Filter Channel 0 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSAT2** (const SDADC_Type *const pSadc)

Get the value of SAT2. Filter Channel 2 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSAT2** (SDADC_Type *const pSadc)

Clear SAT2. Filter Channel 2 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSAT1** (const SDADC_Type *const pSadc)

Get the value of SAT1. Filter Channel 1 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSAT1** (SDADC_Type *const pSadc)

Clear SAT1. Filter Channel 1 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSAT0** (const SDADC_Type *const pSadc)

Get the value of SAT0. Filter Channel 0 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSAT0** (SDADC_Type *const pSadc)

Clear SAT0. Filter Channel 0 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD2** (const SDADC_Type *const pSadc)

Get the value of CAD2. Filter Channel 2 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCAD2** (SDADC_Type *const pSadc)

Clear CAD2. Filter Channel 2 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD1** (const SDADC_Type *const pSadc)

Get the value of CAD1. Filter Channel 1 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCAD1** (SDADC_Type *const pSadc)

Clear CAD1. Filter Channel 1 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD0** (const SDADC_Type *const pSadc)

Get the value of CAD0. Filter Channel 0 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCAD0** (SDADC_Type *const pSadc)

Clear CAD0. Filter Channel 0 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVF2** (const SDADC_Type *const pSadc)

Get the value of FOVF2. Filter Channel 2 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFOVF2** (SDADC_Type *const pSadc)

Clear FOVF2. Filter Channel 2 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVF1** (const SDADC_Type *const pSdadc)

Get the value of FOVF1. Filter Channel 1 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFOVF1** (SDADC_Type *const pSdadc)

Clear FOVF1. Filter Channel 1 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVF0** (const SDADC_Type *const pSdadc)

Get the value of FOVF0. Filter Channel 0 FIFO Overflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFOVF0** (SDADC_Type *const pSdadc)

Clear FOVF0. Filter Channel 0 FIFO Overflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNF2** (const SDADC_Type *const pSdadc)

Get the value of FUNF2. Filter Channel 2 FIFO underflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFUNF2** (SDADC_Type *const pSdadc)

Clear FUNF2. Filter Channel 2 FIFO underflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNF1** (const SDADC_Type *const pSdadc)

Get the value of FUNF1. Filter Channel 1 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFUNF1** (SDADC_Type *const pSdadc)

Clear FUNF1. Filter Channel 1 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNF0** (const SDADC_Type *const pSdadc)

Get the value of FUNF0. Filter Channel 0 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFUNF0** (SDADC_Type *const pSdadc)

Clear FUNF0. Filter Channel 0 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO2_RDY** (const SDADC_Type *const pSadc)

Get the value of CLKO2_RDY. Modulator Clock Output 2 Ready Flag Indicate the modulator clock 2 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO1_RDY** (const SDADC_Type *const pSadc)

Get the value of CLKO1_RDY. Modulator Clock Output 1 Ready Flag Indicate the modulator clock 1 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO0_RDY** (const SDADC_Type *const pSadc)

Get the value of CLKO0_RDY. Modulator Clock Output 0 Ready Flag Indicate the modulator clock 0 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFIFO_EMPTY** (const SDADC_Type *const pSadc)

Get the value of FIFO_EMPTY. FIFO Empty Flag bit 18-16 are for filter channel 2-0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHRDY** (const SDADC_Type *const pSadc)

Get the value of CHRDY. Filter Channel Ready Flag bit 10-8 are for filter channel 2-0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON2** (const SDADC_Type *const pSadc)

Get the value of CHON2. Filter Channel 2 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON1** (const SDADC_Type *const pSadc)

Get the value of CHON1. Filter Channel 1 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON0** (const SDADC_Type *const pSadc)

Get the value of CHON0. Filter Channel 0 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON** (const SDADC_Type *const pSadc)

Get the value of CHON. Filter Channel ON Flag bit 10-8 are for filter channel 2-0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCCRN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of the Channel Configuration Register (CCR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCCRN** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Configuration Register (CCR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMT_SEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LMT_SEL. Limit Check Input Selection Select the input from Main Filter or Auxiliary Filter. 0b - Auxiliary Filter. 1b - Main Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD_SEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of ZCD_SEL. Zero Cross Check Input Selection Select the input from Main Filter or Auxiliary Filter. 0b - Auxiliary Filter. 1b - Main Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDBG_SEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DBG_SEL. Debug Output Selection Select which data is output when read debug register 0100b - Instantaneous Primary Filter shifted 24bit Data. 0110b - Instantaneous HPF Output Data. 1000b - Instantaneous GAIN Output Data. 1010b - Instantaneous Secondary Filter CIC Data. 1110b - Instantaneous Primary Filter offset removed 24bit Data. others - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INTG_EN. Integration Enable Enable the integration block. 0b - Disabled and bypassed. 1b - Enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetRECT_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of RECT_EN. Rectification Enable Enable the rectification block. 0b - Disabled and bypassed. 1b - Enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFIFO_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of FIFO_EN. FIFO Enable 0b - FIFO is disabled for the channel. 1b - FIFO is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMT_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LMT_EN. Limit Enable 0b - Limit check is disabled for the Main or Auxiliary Filter. 1b - Limit check is enabled for the Main or Auxiliary Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of ZCD_EN. Zero Cross Detect Enable 0b - Zero cross detection is disabled for the Main or Auxiliary Filter. 1b - Zero cross detection is enabled for the Main or Auxiliary Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of CAD_EN. Clock Absence Detect Enable 0b - Clock absence detection is disabled for the channel. 1b - Clock absence detection is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SCD_EN. Short Circuit Detect Enable 0b - Short current detection is disabled for the channel. 1b - Short current detection is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDMA_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of DMA_EN. DMA Enable 0b - DMA is disabled for the channel. 1b - DMA is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetAF_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AF_EN. Auxiliary Filter Enable 0b - The Auxiliary Filter is disabled for the channel. 1b - The Auxiliary Filter is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMF_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MF_EN. Main Filter Enable 0b - The Main Filter is disabled for the channel. 1b - The Main Filter is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCH_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of CH_EN. Channel Enable 0b - The channel is disabled. 0b - The channel is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCH_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint8_t u8Enable)
Set the value of CH_EN. Channel Enable 0b - The channel is disabled. 0b - The channel is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetAFCM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AFCM. Auxiliary Filter Conversion Mode 0b - Always on. 1b - Follow Main Filter mode (trigger).

LOCAL_INLINE uint32_t **SDADC_HWA_GetAFORD** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AFORD. Auxiliary Filter Order 00b - Reserved. 01b - First order filter. 10b - Second order filter. 11b - Third order filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetAFOSR** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AFOSR. Auxiliary Filter Oversampling Rate The oversampling rate = (AFOSR + 1). The lower valid value of AFOSR is 3. The higher is depended on AFORD, in order to keep the internal CIC raw result not exceeding 24bit. If AFORD = 3, the maximum value is 202, else the maximum value is 511.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFCM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MFCM. Main Filter Conversion Mode 00b - Single mode. One conversion following a triggering event (edge or level). 01b - Continuous conversion mode. Multiple conversions following a triggering event, and the next trigger will cancel and restart conversion. 10b - Always conversion mode. Multiple conversions following the first trigger, and the next trigger will be ignored. 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFORD** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MFORD. Main Filter Order 000b - Reserved. 001b - First order filter. 010b - Second order filter. 011b - Third order filter. 100b - Fourth order filter. 101b - Fifth order filter. 110,111 - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFOSR** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MFOSR. Main Filter Oversampling Rate The oversampling rate = (MFOSR + 1). The lower valid of MFOSR value is 3. The higher is depended on MFORD, in order to keep the internal CIC raw result not exceeding 32bit. If MFORD = 4, the maximum value is 203. If MFORD = 5, the maximum value is 72, else the maximum value is 511.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCFR_SYNC_TRGEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SYNC_TRGEN. Synced Trigger Select Enable 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTRGLVL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of TRGLVL. Trigger Level type 0b - Trigger is edge event. Next edge event will cancel the current conversion, and re-start in single or continuous mode. 1b - Trigger is level event. De-assert cancels the current conversion, and stop filter function in single or continuous mode.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSYNC_TRGSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGSEL. Synced Trigger Select Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - select the trigger from the channel 0. 001b - select the trigger from the channel 1. 010b - select the trigger from the channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetTRGSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TRGSEL. Trigger Select 0b - Software trigger is selected. 1b - Hardware trigger is selected.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKEDG** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CLKEDG. Input Clock Edge Select This bits select the effective edge of the modulator clock recovery to generate mclk. When using on chip modulator, only posedge is supported. 01b - Posedge. 10b - Negedge. 11b - Both edge. 00b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCFRCLKSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CLKSEL. Input Clock Select This bits select the clock source for the clock recovery to generate mclk. 00b - Reserved. 01b - Internal generated function clock div_clk1 selected, used for the case where MA is using div_clk0 as its function clock. The clock recovery is worked in sync mode. 10b - Internal generated function clock div_clk2 selected, used for the case where MA is using div_clk1 as its function clock. The clock recovery is worked in sync mode. 11b - Modulator clock from external modulator selected. The clock recovery is worked in async mode.

LOCAL_INLINE uint32_t **SDADC_HWA_GetIBFMT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of IBFMT. Input Bit Format 00b - External one bit stream. 01b - External Manchester code, CLKEDG will determine rise or fall decoder. 10b - Reserved. 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFWMK** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of FWMK. FIFO Water Mark Controls the watermark of the FIFO used to set FRDYn. When the number of results in the FIFO is greater than FWMK value, the FRDYn flag will be set. A DMA request or interrupt can also be generated if related DMA_EN and FRDYIE bit-field is set.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCUTOFF** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CUTOFF. Cutoff Frequency Setting for High Pass Filter(HPF) HPF can only be enabled when RDFMT=0. 000b - HPF is disabled 001b - HPF is enabled and setting the cutoff frequency to rate 1(lowest cutoff frequency) 010b - HPF is enabled and setting the cutoff frequency to rate 2. 011b - HPF is enabled and setting the cutoff frequency to rate 3. 100b - HPF is enabled and setting the cutoff frequency to rate 4. 101b - HPF is enabled and setting the cutoff frequency to rate 5. 110b - HPF is enabled and setting the cutoff frequency to rate 6. 111b - HPF is enabled and setting the cutoff frequency to rate 7.(highest cutoff frequency)

LOCAL_INLINE uint32_t **SDADC_HWA_GetRDFMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of RDFMT. Result Data Format 0b - Left justified, signed, for the case where input bitstream is signed 1b - Left justified, unsigned, for the case where input bitstream is unsigned.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFSHIFT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of MFSHIFT. Main Filter Shift to Get Correct 24 bit Precision Main Filter CIC raw result is always 32bit width. The user should program MFSHIFT for cut-off and shift left/right to get 24bit result. MFSHIFT[4] = 1 means left shift, and MFSHIFT[4] = 0 means right shift. MFSHIFT[3:0] is the shift length.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCPRn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Protection Register (CPR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCPRn** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Protection Register (CPR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT_BKEN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of HLMT_BKEN. Break Signal Enable for High Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT_BKEN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of WLMT_BKEN. Break Signal Enabled for Window Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT_BKEN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of LLMT_BKEN. Break Signal Enabled for Low Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD_BKEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SCD_BKEN. Break Signal Enabled for Short Circuit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD_BKEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CAD_BKEN. Break Signal Enabled for Clock Absence Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTF_PROTEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LMTF_PROTEN. Protection Singal Enabled for Limit Flag The protection signal can be enabled and sent to TRGSEL for application protection function. LMTOPT can be used to select which limit flag(HLMT/LLMT/WLMT) is used as the protection source. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTR_PROTEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LMTR_PROTEN. Protection Singal Enabled for Limit Result The protection signal can be enabled and sent to TRGSEL for application protection function. LMTOPT can be used to select which limit result(high/low/window limit result) is used as the protection source. This protection signal will keep on if related condition keeps(keep high/keep low/keep in window) until the condition is not satisfied. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHYSTR_PROTEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of HYSTR_PROTEN. Protection Singal Enabled for Limit Hysteresis Result The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal is from the result of hysteresis comparator in limit detector and will keep on if related condition keeps(keep high/keep low) until the condition is not satisfied. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD_PROTEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SCD_PROTEN. Protection Singal Enabled for SCD Flag The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD_PROTEN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CAD_PROTEN. Protection Singal Enabled for CAD Flag The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTAVT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of LMTAVT. Limitation Result Activation Select The conversion results are compared with HILMTn and LOLMTn in limit detector, and HILMTn and LOLMTn can define a hysteresis range , A compare result is set/clear and sent to system with configurable activation. Note, When this hysteresis compare function is used in system, LMTOPTn need to be 00b to enable detect on exceed both high and low value. 0b - The compare result is set only when the conversion result is larger than HILMTn and clear only when the conversion result is smaller than LOLMTn. 1b - The compare result is clear only when the conversion result is larger than HILMTn and set only when the conversion result is smaller than LOLMTn.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADLMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CADLMT. Clock Absence Detector Limitation Threshold The threshold value for the clock absence counter. The counter indicates the number of cycles the modulator clock not detected. If this value is reached, a clock absence detector event will occur on the dedicated channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTOPT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of LMTOPT. Limitation Option 00b - Limitation detect on exceed both high and low value. 01b - Limitation detect on exceed high threshold value. 10b - Limitation detect on exceed low threshold value. 11b - Limitation detect on windowed value.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDOPT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SCDOPT. Short Circuit Option 00b - Count on both 0 and 1. Detect both high and low value. 01b - Count on 1. Detect high value. 10b - Count on 0. Detect low value. 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDCM** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SCDCM. Short Circuit Detect Conversion Mode 0b - Always on when CHEN and MEN are asserted. 1b - Only works when the any filter is in progress.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDLMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SCDLMT. Short Circuit Detector Limitation extension The threshold value for short circuit detector counter. The counter indicates the number of continuous received bit 0 or bit 1. If this value is reached, a short-circuit detector event will occur on the dedicated channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIAS** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of BIAS. Bias Value Bias offset for dedicated channel main filter; used for sign adjustment and compensation. It's a signed data. The main filter data after shift, will subtract this bias value.

LOCAL_INLINE void **SDADC_HWA_SetBIAS** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of BIAS. Bias Value Bias offset for dedicated channel main filter; used for sign adjustment and compensation. It's a signed data. The main filter data after shift, will subtract this bias value.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLOLMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of LOLMT. Low Limit Threshold When the data value exceeds this value, a low limit event will occur. The limit value format (signed or unsigned) depends on RDFMT.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHILMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of HILMT. High Limit Threshold When data value exceeds this value, high limit event will occur. The limit value format (signed or unsigned) is depended on RDFMT.

LOCAL_INLINE uint32_t **SDADC_HWA_GetRDATA** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of RDATA. Result Data Read this register to get the conversion results. Bit[31] is sign bit if RDFMT is signed and only support 32bit read. The data format may changed by TSRDM. When TSRDM=2'b00, read CRDATA will pop data from FIFO. RDATA[31:8] is a 24-bit conversion results and RDATA[7:0] is 8'h00. At this case, CRDATA can only be read when FIFO is enabled and not empty. When TSRDM=2'b01, read CRDATA will feedback latest conversion result and timestamp info, RDATA[31:16] is a cut and round 16-bit latest conversion result(update on last COC) and RDATA[15:0] is latest timestamp counter value. FIFO is not used. At this case, CRDATA can only be read at any time. When TSRDM=2'b10, read CRDATA will pop data from FIFO and a timestamp info captured by trigger will also be included. RDATA[31:16] is a cut and round 16-bit result from FIFO and RDATA[15:0] is captured timestamp counter value. FIFO is used and gated by timestamp trigger. At this case, CRDATA can only be read when FIFO is enabled and not empty.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMPDATA** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of MPDATA. Multi Purpose Data This register can be accessed by DBG_SEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSIGN** (const SDADC_Type *const pSadc)

Get the value of SIGN. Sign Signal for the PWM pattern 0b - Positive values 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPWM_CNT** (const SDADC_Type *const pSadc)

Get the value of PWM_CNT. PWM Counter Value Counts the 32 cycles for each PWM pattern step. (-16 to 15)

LOCAL_INLINE uint32_t **SDADC_HWA_GetPAT_CNT** (const SDADC_Type *const pSadc)

Get the value of PAT_CNT. PWM Pattern Counter Value Counts the 32 steps for each PWM pattern period.

LOCAL_INLINE uint32_t **SDADC_HWA_GetON** (const SDADC_Type *const pSadc)

Get the value of ON. ON Status 0b - PWM generator is stopped 1b - PWM generator is ON.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTRGSIGN_OEN** (const SDADC_Type *const pSadc)

Get the value of TRGSIGN_OEN. Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once SIGN falling edge is got.

LOCAL_INLINE void **SDADC_HWA_SetTRGSIGN_OEN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set TRGSIGN_OEN. Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once SIGN falling edge is got.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPWMCLKSEL** (const SDADC_Type *const pSadc)

Get the value of CLKSEL. PWM Generator Clock selection 00b - div_clk0 01b - div_clk1 10b - div_clk2 11b - off.

LOCAL_INLINE void **SDADC_HWA_SetCLKSEL** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CLKSEL. PWM Generator Clock selection 00b - div_clk0 01b - div_clk1 10b - div_clk2 11b - off.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIT_REV** (const SDADC_Type *const pSadc)

Get the value of BIT_REV. Bit Reverse Mode 0b - Normal mode, a triangle wave is used as PWM carrier. 1b - Bit reverse mode. A bit reversed sawtooth wave is used as PWM carrier.

LOCAL_INLINE void **SDADC_HWA_SetBIT_REV** (SDADC_Type *const pSadc, uint32_t u32Value)

Set BIT_REV. Bit Reverse Mode 0b - Normal mode, a triangle wave is used as PWM carrier. 1b - Bit reverse mode. A bit reversed sawtooth wave is used as PWM carrier.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPOL** (const SDADC_Type *const pSadc)

Get the value of POL. PWM Generator Polarity 0b - PWM generator output begin with +1 1b - PWM generator output begin with -1.

LOCAL_INLINE void **SDADC_HWA_SetPOL** (SDADC_Type *const pSadc, uint32_t u32Value)

Set POL. PWM Generator Polarity 0b - PWM generator output begin with +1 1b - PWM generator output begin with -1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPATTERN** (const SDADC_Type *const pSadc)

Get the value of PATTERN. PWM Generator Pattern Config this before set MEN. Clear the bits during PWM generator run will terminate the PWM generator outputs after completion of the current period(indicated by bit ON=0) 00b - Stopped 01b - Square wave 10b - Triangle 11b - Sine wave.

LOCAL_INLINE void **SDADC_HWA_SetPATTERN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set PATTERN. PWM Generator Pattern Config this before set MEN. Clear the bits during PWM generator run will terminate the PWM generator outputs after completion of the current period(indicated by bit ON=0) 00b - Stopped 01b - Square wave 10b - Triangle 11b - Sine wave.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD2** (const SDADC_Type *const pSadc)

Get the value of TSVLD2. Timestamp Valid Flag for Channel 2 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearTSVLD2** (SDADC_Type *const pSadc)

Clear TSVLD2. Timestamp Valid Flag for Channel 2 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD1** (const SDADC_Type *const pSadc)

Get the value of TSVLD1. Timestamp Valid Flag for Channel 1 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearTSVLD1** (SDADC_Type *const pSadc)

Clear TSVLD1. Timestamp Valid Flag for Channel 1 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD0** (const SDADC_Type *const pSadc)

Get the value of TSVLD0. Timestamp Valid Flag for Channel 0 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearTSVLD0** (SDADC_Type *const pSadc)

Clear TSVLD0. Timestamp Valid Flag for Channel 0 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N2** (const SDADC_Type *const pSadc)

Get the value of TSIGN_P2N2. Trigger Sign Changed from 1 to 0 for Channel 2 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE void **SDADC_HWA_ClearTSIGN_P2N2** (SDADC_Type *const pSadc)

Clear TSIGN_P2N2. Trigger Sign Changed from 1 to 0 for Channel 2 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N1** (const SDADC_Type *const pSadc)

Get the value of TSIGN_P2N1. Trigger Sign Changed from 1 to 0 for Channel 1 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE void **SDADC_HWA_ClearTSIGN_P2N1** (SDADC_Type *const pSadc)

Clear TSIGN_P2N1. Trigger Sign Changed from 1 to 0 for Channel 1 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N0** (const SDADC_Type *const pSadc)

Get the value of TSIGN_P2N0. Trigger Sign Changed from 1 to 0 for Channel 0 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE void **SDADC_HWA_ClearTSIGN_P2N0** (SDADC_Type *const pSadc)

Clear TSIGN_P2N0. Trigger Sign Changed from 1 to 0 for Channel 0 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCIC_COC2** (const SDADC_Type *const pSadc)

Get the value of MCIC_COC2. Filter Channel 2 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE void **SDADC_HWA_ClearMCIC_COC2** (SDADC_Type *const pSadc)

Clear MCIC_COC2. Filter Channel 2 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCIC_COC1** (const SDADC_Type *const pSadc)

Get the value of MCIC_COC1. Filter Channel 1 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE void **SDADC_HWA_ClearMCIC_COC1** (SDADC_Type *const pSadc)

Clear MCIC_COC1. Filter Channel 1 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCIC_COC0** (const SDADC_Type *const pSadc)

Get the value of MCIC_COC0. Filter channel 0 Conversion Complete Flag for MCIC. Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE void **SDADC_HWA_ClearMCIC_COC0** (SDADC_Type *const pSdadc)

Clear MCIC_COC0. Filter channel 0 Conversion Complete Flag for MCIC. Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD2** (const SDADC_Type *const pSdadc)

Get the value of SD_CAPVLD2. Capture Valid Flag for channel 2 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE void **SDADC_HWA_ClearSD_CAPVLD2** (SDADC_Type *const pSdadc)

Clear SD_CAPVLD2. Capture Valid Flag for channel 2 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD1** (const SDADC_Type *const pSdadc)

Get the value of SD_CAPVLD1. Capture Valid Flag for channel 1 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE void **SDADC_HWA_ClearSD_CAPVLD1** (SDADC_Type *const pSdadc)

Clear SD_CAPVLD1. Capture Valid Flag for channel 1 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD0** (const SDADC_Type *const pSdadc)

Get the value of SD_CAPVLD0. Capture Valid Flag for channel 0 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE void **SDADC_HWA_ClearSD_CAPVLD0** (SDADC_Type *const pSdadc)

Clear SD_CAPVLD0. Capture Valid Flag for channel 0 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD_IE2** (const SDADC_Type *const pSdadc)

Get the value of TSVLD_IE2. Timestamp Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSVLD_IE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSVLD_IE2. Timestamp Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD_IE1** (const SDADC_Type *const pSdadc)

Get the value of TSVLD_IE1. Timestamp Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSVLD_IE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSVLD_IE1. Timestamp Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD_IE0** (const SDADC_Type *const pSdadc)

Get the value of TSVLD_IE0. Timestamp Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSVLD_IE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSVLD_IE0. Timestamp Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N_IE2** (const SDADC_Type *const pSdadc)

Get the value of TSIGN_P2N_IE2. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSIGN_P2N_IE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSIGN_P2N_IE2. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N_IE1** (const SDADC_Type *const pSdadc)

Get the value of TSIGN_P2N_IE1. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSIGN_P2N_IE1** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set TSIGN_P2N_IE1. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N_IE0** (const SDADC_Type *const pSdadc)
Get the value of TSIGN_P2N_IE0. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSIGN_P2N_IE0** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set TSIGN_P2N_IE0. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD_IE2** (const SDADC_Type *const pSdadc)
Get the value of SD_CAPVLD_IE2. Capture Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSD_CAPVLD_IE2** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set SD_CAPVLD_IE2. Capture Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD_IE1** (const SDADC_Type *const pSdadc)
Get the value of SD_CAPVLD_IE1. Capture Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSD_CAPVLD_IE1** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set SD_CAPVLD_IE1. Capture Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD_IE0** (const SDADC_Type *const pSdadc)
Get the value of SD_CAPVLD_IE0. Capture Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSD_CAPVLD_IE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set SD_CAPVLD_IE0. Capture Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_STRIG2** (const SDADC_Type *const pSdadc)

Get the value of INTG_STRIG2. Integration Software Trigger for channel 2 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE void **SDADC_HWA_SetINTG_STRIG2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set INTG_STRIG2. Integration Software Trigger for channel 2 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_STRIG1** (const SDADC_Type *const pSdadc)

Get the value of INTG_STRIG1. Integration Software Trigger for channel 1 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE void **SDADC_HWA_SetINTG_STRIG1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set INTG_STRIG1. Integration Software Trigger for channel 1 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_STRIG0** (const SDADC_Type *const pSdadc)

Get the value of INTG_STRIG0. Integration Software Trigger for channel 0 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE void **SDADC_HWA_SetINTG_STRIG0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set INTG_STRIG0. Integration Software Trigger for channel 0 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIAS_ON** (const SDADC_Type *const pSdadc)

Get the value of BIAS_ON. Bias On Flag 0b - Bias analog is not ready for using. 1b - Bias analog is ready for using.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBSTC** (const SDADC_Type *const pSadc)

Get the value of BSTC. Bias Start Time Control Bias analog need a ~2us start time, this bits control number of div_clk1 cycles before BIAS_ON flag set after BIAS_EN set.

LOCAL_INLINE void **SDADC_HWA_SetBSTC** (SDADC_Type *const pSadc, uint32_t u32Value)

Set BSTC. Bias Start Time Control Bias analog need a ~2us start time, this bits control number of div_clk1 cycles before BIAS_ON flag set after BIAS_EN set.

LOCAL_INLINE uint32_t **SDADC_HWA_GetVCM_BEN** (const SDADC_Type *const pSadc)

Get the value of VCM_BEN. Common Mode Voltage Bias Enable Enable common mode voltage for AC input of internal modulator analog. 0b - Common mode voltage Bias is disabled. 1b - Common mode voltage Bias is enabled.

LOCAL_INLINE void **SDADC_HWA_SetVCM_BEN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set VCM_BEN. Common Mode Voltage Bias Enable Enable common mode voltage for AC input of internal modulator analog. 0b - Common mode voltage Bias is disabled. 1b - Common mode voltage Bias is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHFVREF_EN** (const SDADC_Type *const pSadc)

Get the value of HFVREF_EN. Half VREFH output Enable to Related Internal Modulator Analog. Bit 4 enable internal Half VREFH output to modulator analog for channel 0. Bit 5 enable internal Half VREFH output to modulator analog for channel 1. Bit 6 enable internal Half VREFH output to modulator analog for channel 2. Bit 7 enable internal Half VREFH output to modulator analog for channel 3. NOTE: Not all channels have internal modulator analog, if the channel has no internal modulator analog, related control bit is reserved.

LOCAL_INLINE void **SDADC_HWA_SetHFVREF_EN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set HFVREF_EN. Half VREFH output Enable to Related Internal Modulator Analog. Bit 4 enable internal Half VREFH output to modulator analog for channel 0. Bit 5 enable internal Half VREFH output to modulator analog for channel 1. Bit 6 enable internal Half VREFH output to modulator analog for channel 2. Bit 7 enable internal Half VREFH output to modulator analog for channel 3. NOTE: Not all channels have internal modulator analog, if the channel has no internal modulator analog, related control bit is reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIAS_EN** (const SDADC_Type *const pSadc)

Get the value of BIAS_EN. Bias Analog Enable 0b - Bias analog is disabled. 1b - Bias analog is enabled.

LOCAL_INLINE void **SDADC_HWA_SetBIAS_EN** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set BIAS_EN. Bias Analog Enable 0b - Bias analog is disabled. 1b - Bias analog is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDSIGN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DSIGN. Delayed Sign Signal 0b - Positive values. 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSSIGN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SSIGN. Selected Sign Signal 0b - Positive values. 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSDON** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SDON. Sign Delay Counter on Flag Indicates the activity of the sign delay counter. 0b - Sign delay counter stopped. 1b - Sign delay counter enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSIGN. Sign Signal for Channel Trigger Output This sign can be configured to some cycles before DSIGN, and can be used to generate channel trigger output, typically can be used to trigger integration so that it can start at the ZCD point 0b - Positive values. 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDLY_TRG** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DLY_TRG. Sign Delay Value for TSIGN Posedge TSIGN is clear at DLY_TRG and set at DLY_NEG in CSDRn.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCTRGSIGN_OEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CTRGSIGN_OEN. Channel Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once TSIGN falling edge is got.

LOCAL_INLINE void **SDADC_HWA_SetCTRGSIGN_OEN** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint8_t u8Enable)

Set the value of CTRGSIGN_OEN. Channel Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once TSIGN falling edge is got.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSSCH** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SSCH. Sign Signal From Channel Selection Select the channel providing the sign signal if SSRC = 01b 0000b - From channel 0. 0001b - From channel 1. 0010b - From channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetSSRC** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SSRC. Sign Source Selection Select the sign signal to be delayed. 00b - SDADC internal PWM generation block. 01b - Sign of result of channel selected by SSCH. 10b - External sign signal 0. 11b - External sign signal 1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDLY_NEG** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of DLY_NEG. Sign Delay Value for Sign Negedge Defines the content of SD_CNT to generate a positive delayed sign DSIGN.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDLY_POS** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of DLY_POS. Sign Delay Value for Sign Posedge Defines the content of SD_CNT to generate a negative delayed sign DSIGN.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SD_CAP. Sign Delay Capture Value Indicates the results value counted between the negedge of SSIGN and the first received positive value.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CNT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SD_CNT. Sign Delay Counter Counts the results values from the filter chain to delay the sign signal, this field can only be checked after MCIC_COC assert.

LOCAL_INLINE uint32_t **SDADC_HWA_GetNVALINT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of NVALINT. Number of Values to be Accumulated NVALINT+1 defines a complete integration cycle.

LOCAL_INLINE uint32_t **SDADC_HWA_GetNVALDIS** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of NVALDIS. Number of Values discarded Start the integration cycle after NVALDIS values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCICFR_SYNC_TRGEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SYNC_TRGEN. Synced Trigger Select Enable for Integration 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCICFR_SYNC_TRGSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGSEL. Synced Trigger Select for Integration Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - select the trigger from the channel 0. 001b - select the trigger from the channel 1. 010b - select the trigger from the channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetNVALREP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of NVALREP. Number of Integration Cycles NVALREP+1 defines number of integration cycles to be counted by REPCNT if activated(EXTSTP=0)

LOCAL_INLINE uint32_t **SDADC_HWA_GetITRSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of ITRSEL. Integration Trigger Select 0b - Software trigger selected. 1b - Hardware trigger dedicate to channel selected.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHTRPOL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of HTRPOL. Hardware Trigger Polarity 0b - High and rising edge effective. 1b - Low and falling edge effective.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFCRC** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of FCRC. Filter Chain Restart Control 0b - Do not restart. 1b - Restart the filter chain when an integration window starts.

LOCAL_INLINE uint32_t **SDADC_HWA_GetEXTSTP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of EXTSTP. Integration Stop Control For Hardware trigger control only. 0b - Internal control, stop integration after NVALREP+1 integration cycles, support both edge and level events. 1b - External control, stop integration upon the inverse trigger event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetIOGT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of IOGT. Integration Output Gating Setting this bit will gate conversion complete flag and FIFO push when Integration is not on-going. conversion complete flag and FIFO push can only be effective when Integration is not on-going(INTON asserted) 0b - Conversion complete flag and FIFO push always works. 1b - Conversion complete flag and FIFO push can only work when Integration is not on-going(INTON asserted)

LOCAL_INLINE uint32_t **SDADC_HWA_GetISVAL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of ISVAL. Integration Shift value 000b - Shift 0 bits 001b - Shift 1 bits, for 2 values integration 010b - Shift 2 bits, for 3-4 values integration 011b - Shift 3 bits, for 5-8 values integration 100b - Shift 4 bits, for 9-16 values integration 101b - Shift 5 bits, for 17-32 values integration 110b - Shift 6 bits, for 33-64 values integration.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTGON** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INTGON. Integration on Flag Indicates the activity of the integration. 0b - Integration stopped. 1b - Integration enabled and on-going.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTVAL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INTVAL. Results of most recent accumulation, this field can only be checked after MCIC_COC assert.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMASEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of MASEL. Modulator Analog Selection 0b - External modulator analog is selected as channel input. 1b - Internal modulator analog is selected as channel input.

LOCAL_INLINE void **SDADC_HWA_SetMASEL** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set MASEL. Modulator Analog Selection 0b - External modulator analog is selected as channel input. 1b - Internal modulator analog is selected as channel input.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMACLKSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

*Get the value of MACLKSEL. Internal Modulator Analog Clock Selection 00b - Div_clk1 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 01b - Div_clk0 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 1*b - Reserved.*

LOCAL_INLINE void **SDADC_HWA_SetMACLKSEL** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

*Set MACLKSEL. Internal Modulator Analog Clock Selection 00b - Div_clk1 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 01b - Div_clk0 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 1*b - Reserved.*

LOCAL_INLINE uint32_t **SDADC_HWA_GetRSTSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of RSTSEL. Modulator Analog Reset Cycle Selection Select RSTSEL+1 mclk cycles for modulator analog reset after a channel conversion trigger. This is needed especially when the input mux(INMUX) is changed. 0d - Modulator analog is not reset after a channel conversion trigger 1d - Modulator analog is reset for 2 mclk cycles after a channel conversion trigger. Total 7 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 2d - Modulator analog is reset for 3 mclk cycles after a channel conversion trigger. Total 8 mclk cycle cycles needed for filters start to run after a channel conversion trigger. ... 8d - Modulator analog is reset for 9 mclk cycles after a channel conversion trigger. Total 14 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 9-15d - Modulator analog is reset for 10 mclk cycles after a channel conversion trigger. Total 15 mclk cycle cycles needed for filters start to run after a channel conversion trigger.

LOCAL_INLINE void **SDADC_HWA_SetRSTSEL** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set RSTSEL. Modulator Analog Reset Cycle Selection Select RSTSEL+1 mclk cycles for modulator analog reset after a channel conversion trigger. This is needed especially when the input mux(INMUX) is changed. 0d - Modulator analog is not reset after a channel conversion trigger 1d - Modulator analog is reset for 2 mclk cycles after a channel conversion trigger. Total 7 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 2d - Modulator analog is reset for 3 mclk cycles after a channel conversion trigger. Total 8 mclk cycle cycles needed for filters start to run after a channel conversion trigger. ... 8d - Modulator analog is reset for 9 mclk cycles after a channel conversion trigger. Total 14 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 9-15d - Modulator analog is reset for 10 mclk cycles after a channel conversion trigger. Total 15 mclk cycle cycles needed for filters start to run after a channel conversion trigger.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDITHER_EN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of DITHER_EN. Dither Enable This bit enable the dither to analog circuit to remove the idle tone. 0b - Dither is disabled. 1b - Dither is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINCFGPV11** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of INCFGPV11. Configuration of Positive Input Line with V11 from System This bit have higher priority then INCFGP, this this bit is set, INCFGP is ignored. 0b - Positive input from V11 disabled. 1b - Positive input from V11 enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGAINSEL** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of GAINSEL. Gain Select of Analog Input Path 00b - 1. 01b - 2. 10b - 4. 11b - 8.

LOCAL_INLINE void **SDADC_HWA_SetGAINSEL** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set GAINSEL. Gain Select of Analog Input Path 00b - 1. 01b - 2. 10b - 4. 11b - 8.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCMENP** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CMENP. Common Voltage Configuration of Positive Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE void **SDADC_HWA_SetCMENP** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set CMENP. Common Voltage Configuration of Positive Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCMENM** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CMENM. Common Voltage Configuration of Negative Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE void **SDADC_HWA_SetCMENM** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set CMENM. Common Voltage Configuration of Negative Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINCFGP** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of INCFGP. Configuration of Positive Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE void **SDADC_HWA_SetINCFGP** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set INCFGP. Configuration of Positive Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINCFGM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INCFGM. Configuration of Negative Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE void **SDADC_HWA_SetINCFGM** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)
Set INCFGM. Configuration of Negative Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINMUX** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INMUX. Active Input Pin Selection Indicates the active input pin selection for input MUX. 00b - Input pin 0 selected. 01b - Input pin 1 selected. 10b - Input pin 2 selected. 11b - Input pin 3 selected.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INSEL. Input Pin Selection Defines the initial or permanent setting for the input MUX(bits INMUX) depending on the selected operating mode (bits INMOD).

LOCAL_INLINE void **SDADC_HWA_SetINSEL** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)
Set INSEL. Input Pin Selection Defines the initial or permanent setting for the input MUX(bits INMUX) depending on the selected operating mode (bits INMOD).

LOCAL_INLINE uint32_t **SDADC_HWA_GetINMOD** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INMOD. Input MUX Control Mode 00b - Software control mode (INMUX follows INSEL). 01b - Preset mode (load INMUX upon a channel conversion trigger). 10b - Scan mode (decrement INMUX upon a channel conversion trigger, wrap around to INSEL). 11b - Reserved.

LOCAL_INLINE void **SDADC_HWA_SetINMOD** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)
Set INMOD. Input MUX Control Mode 00b - Software control mode (INMUX follows INSEL). 01b - Preset mode (load INMUX upon a channel conversion trigger). 10b - Scan mode (decrement INMUX upon a channel conversion trigger, wrap around to INSEL). 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetOffsetOPT1** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT1. Offset Option 1 This is an offset value provided by factory test for modulator analog input path gain=2. User need read this field and left shift it by 8 bit to form a 24-bit offset and then write to related CBR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetOffsetOPT0** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT0. Offset Option 0 This is an offset value provided by factory test for modulator analog input path gain=1. User need read this field and left shift it by 8 bit to form a 24-bit offset and then write to related CBR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGainOPT1** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT1. Gain Option 1 This is a gain factor provided by factory test for modulator analog input path gain=2. User need read this field and multiply it with gain correction factor 2^m and then write to related CGNR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGainOPT0** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT0. Gain Option 0 This is a gain factor provided by factory test for modulator analog input path gain=1. User need read this field and multiply it with gain correction factor 2^m and then write to related CGNR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGAIN_FAC** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of GAIN_FAC. Gain Factor This bit field defines a gain factor($GAIN_FAC[17:0]/2^{16}$, range is from 0 to 4) which is multiplied with the data after HPF to compensate the gain error caused by analog circuits or the gain mismatch when decimation factor is not 2^N , after compensation, the total gain from analog input to data from FIFO should be 1. GAIN_FAC can be got from calibration, it should be in a suitable range, or the data output may be saturated. When $GAIN_FAC[17:0]=0$, GAIN block is bypassed.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCTSCNT_SYNC_TRGEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGEN. Synced Trigger Select Enable for Timestamp 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCTSCNT_SYNC_TRGSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGSEL. Synced Trigger Select for Timestamp Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - Select the trigger from the channel 0. 001b - Select the trigger from the channel 1. 010b - Select the trigger from the channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSRDM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSRDM. Timestamp Information Read Mode 00b - Default mode, timestamp information can only be got by reading CTSINFO register captured by trigger event. FIFO is used for conversion results only and can be pop from FIFO by reading CRDATAn.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSTRGM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSTRGM. Timestamp Counter Trigger Mode 00/11b - Trigger is ignored. 01b - Rising edge effective. Trigger level low will gating conversion results push to FIFO if TSRDM is 2'b10. Trigger level high will allow conversion results push to FIFO if TSRDM is 2'b10. 10b - Falling edge effective. Trigger level High will gating conversion results push to FIFO if TSRDM is 2'b10. Trigger level low will allow conversion results push to FIFO if TSRDM is 2'b10.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINMXCP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INMXCP. Input Mux Setting Copy Enable 0b - Disabled. 1b - Copy INMUX from CMACSR to TSTAMP[15:14].

LOCAL_INLINE uint32_t **SDADC_HWA_GetTS_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TS_EN. Timestamp Counter Enable 0b - Disabled. 1b - Counting at the rate selected by TSCLKSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSCLKSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSCLKSEL. Timestamp Counter Clock Selection 00b - Modulator clock. 01b - Modulator clock divided by 2. 10b - Modulator clock divided by 4. 11b - Modulator clock divided by 8.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSCNT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSCNT. Timestamp Counter Value TSCNT is clocked with the modulator clock and is cleared when a new result value has been generated.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSDATA** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of TSDATA. The Latest Captured Conversion Result Copied from saved conversion result.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSTAMP** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of TSTAMP. The Latest Captured Timestamp Value Copy from timestamp counter TSCNT at the trigger event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCFRn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Configuration Register (CFR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCFRn** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Configuration Register (CFR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCBRn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Bias Register (CBR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCBRn** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Bias Register (CBR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLLMTn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Low Limit Register (CLLMT) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCLLMTn** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Low Limit Register (CLLMT) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHLMTn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel High Limit Register (CHLMT) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCHLMTn** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel High Limit Register (CHLMT) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCRDATAn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Result Data Register (CRDATA) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCMPDATAn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Multi Purpose Data Register (CMPDATA) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCDRn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Data Rate Register (CDR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCDRn** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Data Rate Register (CDR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetNISR** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of NISR.

LOCAL_INLINE void **SDADC_HWA_SetABNISR0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of ABNISR0.

LOCAL_INLINE void **SDADC_HWA_SetABNISR1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of ABNISR1.

LOCAL_INLINE void **SDADC_HWA_SetEXTIS** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of EXTIS.

LOCAL_INLINE void **SDADC_HWA_SetDMA_EN** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint8_t u8Enable)

Set the value of DMA_EN. Channel Enable 0b - The channel dma is disabled. 0b - The channel dma is enabled.

Detailed Description

Macro Definition Documentation

```
#define SDADC_CHANNELCFGREG( base) ((pSDADC_ChannelRegType)((int)(amp((SDADC_Type *)0)->CCR0) + (int)(base)))
```

Typedef Documentation

```
typedef struct SDADC_ChannelRegType * pSDADC_ChannelRegType
```

Function Documentation

```
LOCAL_INLINE void SDADC_HWA_ClearCAD0 (SDADC_Type *const pSdadc)
```

Clear CAD0. Filter Channel 0 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

```
LOCAL_INLINE void SDADC_HWA_ClearCAD1 (SDADC_Type *const pSdadc)
```

Clear CAD1. Filter Channel 1 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

```
LOCAL_INLINE void SDADC_HWA_ClearCAD2 (SDADC_Type *const pSdadc)
```

Clear CAD2. Filter Channel 2 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearCOC0 (SDADC_Type *const *pSdadc*)

Clear COC0. Filter Channel 0 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearCOC1 (SDADC_Type *const *pSdadc*)

Clear COC1. Filter Channel 1 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearCOC2 (SDADC_Type *const *pSdadc*)

Clear COC2. Filter Channel 2 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFOVF0 (SDADC_Type *const *pSdadc*)

Clear FOVF0. Filter Channel 0 FIFO Overflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFOVF1 (SDADC_Type *const *pSdadc*)

Clear FOVF1. Filter Channel 1 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFOVF2 (SDADC_Type *const *pSdadc*)

Clear FOVF2. Filter Channel 2 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFRDY0 (SDADC_Type *const *pSdadc*)

Clear FRDY0. Filter Channel 0 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFRDY1 (SDADC_Type *const *pSdadc*)

Clear FRDY1. Filter Channel 1 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFRDY2 (SDADC_Type *const *pSdadc*)

Clear FRDY2. Filter Channel 2 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFUNF0 (SDADC_Type *const *pSdadc*)

Clear FUNF0. Filter Channel 0 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFUNF1 (SDADC_Type *const *pSdadc*)

Clear FUNF1. Filter Channel 1 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearFUNF2 (SDADC_Type *const *pSdadc*)

Clear FUNF2. Filter Channel 2 FIFO underflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearHLMT0 (SDADC_Type *const *pSdadc*)

Clear HLMT0. Filter Channel 0 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearHLMT1 (SDADC_Type *const *pSdadc*)

Clear HLMT1. Filter Channel 1 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearHLMT2 (SDADC_Type *const *pSdadc*)

Clear HLMT2. Filter Channel 2 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearLLMT0 (SDADC_Type *const *pSdadc*)

Clear LLMT0. Filter Channel 0 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearLLMT1 (SDADC_Type *const *pSdadc*)

Clear LLMT1. Filter Channel 1 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearLLMT2 (SDADC_Type *const *pSdadc*)

Clear LLMT2. Filter Channel 2 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearMCIC_COC0 (SDADC_Type *const *pSdadc*)

Clear MCIC_COC0. Filter channel 0 Conversion Complete Flag for MCIC. Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which is means integration output valid.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearMCIC_COC1 (SDADC_Type *const *pSdadc*)

Clear MCIC_COC1. Filter Channel 1 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which is means integration output valid.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearMCIC_COC2 (SDADC_Type *const *pSdadc*)

Clear MCIC_COC2. Filter Channel 2 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which is means integration output valid.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSAT0 (SDADC_Type *const *pSdadc*)

Clear SAT0. Filter Channel 0 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSAT1 (SDADC_Type *const *pSdadc*)

Clear SAT1. Filter Channel 1 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSAT2 (SDADC_Type *const *pSdadc*)

Clear SAT2. Filter Channel 2 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSCD0 (SDADC_Type *const *pSdadc*)

Clear SCD0. Filter Channel 0 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSCD1 (SDADC_Type *const *pSdadc*)

Clear SCD1. Filter Channel 1 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSCD2 (SDADC_Type *const *pSdadc*)

Clear SCD2. Filter Channel 2 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSD_CAPVLD0 (SDADC_Type *const *pSdadc*)

Clear SD_CAPVLD0. Capture Valid Flag for channel 0 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSD_CAPVLD1 (SDADC_Type *const *pSdadc*)

Clear SD_CAPVLD1. Capture Valid Flag for channel 1 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearSD_CAPVLD2 (SDADC_Type *const *pSdadc*)

Clear SD_CAPVLD2. Capture Valid Flag for channel 2 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearTSIGN_P2N0 (SDADC_Type *const *pSdadc*)

Clear TSIGN_P2N0. Trigger Sign Changed from 1 to 0 for Channel 0 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearTSIGN_P2N1 (SDADC_Type *const *pSdadc*)

Clear TSIGN_P2N1. Trigger Sign Changed from 1 to 0 for Channel 1 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearTSIGN_P2N2 (SDADC_Type *const *pSdadc*)

Clear TSIGN_P2N2. Trigger Sign Changed from 1 to 0 for Channel 2 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearTSVLD0 (SDADC_Type *const *pSdadc*)

Clear TSVLD0. Timestamp Valid Flag for Channel 0 Indicates timestamp information is captured, it will be cleared if CTSINFO register is read. 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearTSVLD1 (SDADC_Type *const *pSdadc*)

Clear TSVLD1. Timestamp Valid Flag for Channel 1 Indicates timestamp information is captured, it will be cleared if CTSINFO register is read. 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearTSVLD2 (SDADC_Type *const *pSdadc*)

Clear TSVLD2. Timestamp Valid Flag for Channel 2 Indicates timestamp information is captured, it will be cleared if CTSINFO register is read. 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearWLMT0 (SDADC_Type *const *pSdadc*)

Clear WLMT0. Filter Channel 0 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearWLMT1 (SDADC_Type *const *pSdadc*)

Clear WLMT1. Filter Channel 1 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearWLMT2 (SDADC_Type *const *pSdadc*)

Clear WLMT2. Filter Channel 2 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearZCD0 (SDADC_Type *const *pSdadc*)

Clear ZCD0. Filter Channel 0 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearZCD1 (SDADC_Type *const *pSdadc*)

Clear ZCD1. Filter Channel 1 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE void SDADC_HWA_ClearZCD2 (SDADC_Type *const *pSdadc*)

Clear ZCD2. Filter Channel 2 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

LOCAL_INLINE uint32_t SDADC_HWA_GetAF_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of AF_EN. Auxiliary Filter Enable 0b - The Auxiliary Filter is disabled for the channel. 1b - The Auxiliary Filter is enabled for the channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of AF_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetAFCM (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of AFCM. Auxiliary Filter Conversion Mode 0b - Always on. 1b - Follow Main Filter mode (trigger).

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>u8Channel</i>	SDADC Channel.
------------------	----------------

Returns

uint32_t the value of AFCM.

LOCAL_INLINE uint32_t SDADC_HWA_GetAFORD (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of AFORD. Auxiliary Filter Order 00b - Reserved. 01b - First order filter. 10b - Second order filter. 11b - Third order filter.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of AFORD.

LOCAL_INLINE uint32_t SDADC_HWA_GetAFOSR (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of AFOSR. Auxiliary Filter Oversampling Rate The oversampling rate = (AFOSR + 1). The lower valid value of AFOSR is 3. The higher is depended on AFORD, in order to keep the internal CIC raw result not exceeding 24bit. If AFORD = 3, the maximum value is 202, else the maximum value is 511.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of AFOSR.

LOCAL_INLINE uint32_t SDADC_HWA_GetBIAS (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of BIAS. Bias Value Bias offset for dedicated channel main filter; used for sign adjustment and compensation. It's a signed data. The main filter data after shift, will subtract this bias value.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of BIAS.

LOCAL_INLINE uint32_t SDADC_HWA_GetBIAS_EN (const SDADC_Type *const *pSdadc*)

Get the value of BIAS_EN. Bias Analog Enable 0b - Bias analog is disabled. 1b - Bias analog is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of BIAS_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetBIAS_ON (const SDADC_Type *const *pSdadc*)

Get the value of BIAS_ON. Bias On Flag 0b - Bias analog is not ready for using. 1b - Bias analog is ready for using.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of BIAS_ON.

LOCAL_INLINE uint32_t SDADC_HWA_GetBIT_REV (const SDADC_Type *const *pSdadc*)

Get the value of BIT_REV. Bit Reverse Mode 0b - Normal mode, a triangle wave is used as PWM carrier. 1b - Bit reverse mode. A bit reversed sawtooth wave is used as PWM carrier.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of BIT_REV.

LOCAL_INLINE uint32_t SDADC_HWA_GetBSTC (const SDADC_Type *const *pSdadc*)

Get the value of BSTC. Bias Start Time Control Bias analog need a ~2us start time, this bits control number of div_clk1 cycles before BIAS_ON flag set after BIAS_EN set.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of BSTC.

LOCAL_INLINE uint32_t SDADC_HWA_GetCAD0 (const SDADC_Type *const *pSdadc*)

Get the value of CAD0. Filter Channel 0 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CAD0.

LOCAL_INLINE uint32_t SDADC_HWA_GetCAD1 (const SDADC_Type *const *pSdadc*)

Get the value of CAD1. Filter Channel 1 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CAD1.

LOCAL_INLINE uint32_t SDADC_HWA_GetCAD2 (const SDADC_Type *const *pSdadc*)

Get the value of CAD2. Filter Channel 2 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CAD2.

LOCAL_INLINE uint32_t SDADC_HWA_GetCAD_BKEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CAD_BKEN. Break Signal Enabled for Clock Absence Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CAD_BKEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetCAD_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CAD_EN. Clock Absence Detect Enable 0b - Clock absence detection is disabled for the channel. 1b - Clock absence detection is enabled for the channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CAD_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetCAD_PROTEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CAD_PROTEN. Protection Singal Enabled for CAD Flag The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CAD_PROTEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetCADIE0 (const SDADC_Type *const *pSdadc*)

Get the value of CADIE0. Filter Channel 0 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CADIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetCADIE1 (const SDADC_Type *const *pSdadc*)

Get the value of CADIE1. Filter Channel 1 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CADIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetCADIE2 (const SDADC_Type *const *pSdadc*)

Get the value of CADIE2. Filter Channel 2 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CADIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetCADLMT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CADLMT. Clock Absence Detector Limitation Threshold The threshold value for the clock absence counter. The counter indicates the number of cycles the modulator clock not detected. If this value is reached, a clock absence detector event will occur on the dedicated channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CADLMT.

LOCAL_INLINE uint32_t SDADC_HWA_GetCBRn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Bias Register (CBR) for a specified channel.

This function retrieves the value of the Channel Bias Register (CBR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CBR value is to be retrieved.

Returns

uint32_t the value of the CBR for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCCRn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Configuration Register (CCR) for a specified channel.

This function retrieves the value of the Channel Configuration Register (CCR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CCR value is to be retrieved.

Returns

uint32_t the value of the CCR for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCDRn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Data Rate Register (CDR) for a specified channel.

This function retrieves the value of the Channel Data Rate Register (CDR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CDR value is to be retrieved.

Returns

uint32_t the value of the CDR for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCFR_SYNC_TRGEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SYNC_TRGEN. Synced Trigger Select Enable 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SYNC_TRGEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetCFRCLKSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CLKSEL. Input Clock Select This bits select the clock source for the clock recovery to generate mclk. 00b - Reserved. 01b - Internal generated function clock div_clk1 selected, used for the case where MA is using div_clk0 as its function clock. The clock recovery is worked in sync mode. 10b - Internal generated function clock div_clk2 selected, used for the case where MA is using div_clk1 as its function clock. The clock recovery is worked in sync mode. 11b - Modulator clock from external modulator selected. The clock recovery is worked in async mode.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CLKSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetCFRn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Configuration Register (CFR) for a specified channel.

This function retrieves the value of the Channel Configuration Register (CFR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CFR value is to be retrieved.

Returns

uint32_t the value of the CFR for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCH_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CH_EN. Channel Enable 0b - The channel is disabled. 0b - The channel is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CH_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetCHLMTn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel High Limit Register (CHLMT) for a specified channel.

This function retrieves the value of the Channel High Limit Register (CHLMT) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CHLMT value is to be retrieved.

Returns

uint32_t the value of the CHLMT for the specified channel.

LOCAL_INLINE uint32 SDADC_HWA_GetCHON (const SDADC_Type *const *pSdadc*)

Get the value of CHON. Filter Channel ON Flag bit 10-8 are for filter channel 2-0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32 the value of CHON.

LOCAL_INLINE uint32_t SDADC_HWA_GetCHON0 (const SDADC_Type *const *pSdadc*)

Get the value of CHON0. Filter Channel 0 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CHON0.

LOCAL_INLINE uint32_t SDADC_HWA_GetCHON1 (const SDADC_Type *const *pSdadc*)

Get the value of CHON1. Filter Channel 1 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of CHON1.

LOCAL_INLINE uint32_t SDADC_HWA_GetCHON2 (const SDADC_Type *const *pSdac*)

Get the value of CHON2. Filter Channel 2 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of CHON2.

LOCAL_INLINE uint32_t SDADC_HWA_GetCHRDY (const SDADC_Type *const *pSdac*)

Get the value of CHRDY. Filter Channel Ready Flag bit 10-8 are for filter channel 2-0.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of CHRDY.

**LOCAL_INLINE uint32_t SDADC_HWA_GetCICFR_SYNC_TRGEN (const SDADC_Type *const *pSdac*,
uint8_t *u8Channel*)**

Get the value of SYNC_TRGEN. Synced Trigger Select Enable for Integration 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SYNC_TRGEN.

**LOCAL_INLINE uint32_t SDADC_HWA_GetCICFR_SYNC_TRGSEL (const SDADC_Type *const *pSdadc*,
uint8_t *u8Channel*)**

Get the value of SYNC_TRGSEL. Synced Trigger Select for Integration Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - select the trigger from the channel 0. 001b - select the trigger from the channel 1. 010b - select the trigger from the channel 2. ...

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SYNC_TRGSEL.

**LOCAL_INLINE uint32_t SDADC_HWA_GetCLKEDG (const SDADC_Type *const *pSdadc*, uint8_t
u8Channel)**

Get the value of CLKEDG. Input Clock Edge Select This bits select the effective edge of the modulator clock recovery to generate mclk. When using on chip modulator, only posedge is supported. 01b - Posedge. 10b - Negedge. 11b - Both edge. 00b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CLKEDG.

LOCAL_INLINE uint32_t SDADC_HWA_GetCLKO0_DIS (const SDADC_Type *const *pSdadc*)

Get the value of CLK00_DIS. Modulator Clock Output 0 Disable 0b - CLK00 is enabled when MEN is asserted.
1b - CLK00 is disabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CLK00_DIS.

LOCAL_INLINE uint32_t SDADC_HWA_GetCLK00_RDY (const SDADC_Type *const *pSdadc*)

Get the value of CLK00_RDY. Modulator Clock Output 0 Ready Flag Indicate the modulator clock 0 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CLK00_RDY.

LOCAL_INLINE uint32_t SDADC_HWA_GetCLK01_DIS (const SDADC_Type *const *pSdadc*)

Get the value of CLK01_DIS. Modulator Clock Output 1 Disable 0b - CLK01 is enabled when MEN is asserted.
1b - CLK01 is disabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CLK01_DIS.

LOCAL_INLINE uint32_t SDADC_HWA_GetCLK01_RDY (const SDADC_Type *const *pSdadc*)

Get the value of CLK01_RDY. Modulator Clock Output 1 Ready Flag Indicate the modulator clock 1 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CLKO1_RDY.

LOCAL_INLINE uint32_t SDADC_HWA_GetCLKO2_DIS (const SDADC_Type *const *pSdadc*)

Get the value of CLKO2_DIS. Modulator Clock Output 2 Disable 0b - CLKO2 is enabled when MEN is asserted. 1b - CLKO2 is disabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CLKO2_DIS.

LOCAL_INLINE uint32_t SDADC_HWA_GetCLKO2_RDY (const SDADC_Type *const *pSdadc*)

Get the value of CLKO2_RDY. Modulator Clock Output 2 Ready Flag Indicate the modulator clock 2 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CLKO2_RDY.

LOCAL_INLINE uint32_t SDADC_HWA_GetCLLMTn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Low Limit Register (CLLMT) for a specified channel.

This function retrieves the value of the Channel Low Limit Register (CLLMT) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CLLMT value is to be retrieved.

Returns

uint32_t the value of the CLLMT for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCMENM (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CMENM. Common Voltage Configuration of Negative Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CMENM.

LOCAL_INLINE uint32_t SDADC_HWA_GetCMENP (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CMENP. Common Voltage Configuration of Positive Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CMENP.

LOCAL_INLINE uint32_t SDADC_HWA_GetCMPDATAn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Multi Purpose Data Register (CMPDATA) for a specified channel.

This function retrieves the value of the Channel Multi Purpose Data Register (CMPDATA) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CMPDATA value is to be retrieved.

Returns

uint32_t the value of the CMPDATA for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCOC0 (const SDADC_Type *const *pSdadc*)

Get the value of COC0. Filter Channel 0 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of COC0.

LOCAL_INLINE uint32_t SDADC_HWA_GetCOC1 (const SDADC_Type *const *pSdadc*)

Get the value of COC1. Filter Channel 1 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of COC1.

LOCAL_INLINE uint32_t SDADC_HWA_GetCOC2 (const SDADC_Type *const *pSdadc*)

Get the value of COC2. Filter Channel 2 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of COC2.

LOCAL_INLINE uint32_t SDADC_HWA_GetCOCIE0 (const SDADC_Type *const *pSdadc*)

Get the value of COCIE0. Filter Channel 0 Conversion Complete Interrupt Enable 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of COCIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetCOCIE1 (const SDADC_Type *const *pSdadc*)

Get the value of COCIE1. Filter Channel 1 Conversion Complete Interrupt Enable 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of COCIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetCOCIE2 (const SDADC_Type *const *pSdadc*)

Get the value of COCIE2. Filter Channel 2 Conversion Complete Interrupt Enable 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of COCIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetCPRn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Protection Register (CPR) for a specified channel.

This function retrieves the value of the Channel Protection Register (CPR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CPR value is to be retrieved.

Returns

uint32_t the value of the CPR for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCRDATAn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of the Channel Result Data Register (CRDATA) for a specified channel.

This function retrieves the value of the Channel Result Data Register (CRDATA) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CRDATA value is to be retrieved.

Returns

uint32_t the value of the CRDATA for the specified channel.

LOCAL_INLINE uint32_t SDADC_HWA_GetCTRGSIGN_OEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of CTRGSIGN_OEN. Channel Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once TSIGN falling edge is got.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>u8Channel</i>	SDADC Channel.
------------------	----------------

Returns

uint32_t the value of CTRGSIGN_OEN.

**LOCAL_INLINE uint32_t SDADC_HWA_GetCTSCNT_SYNC_TRGEN (const SDADC_Type *const *pSdad*,
uint8_t *u8Channel*)**

Get the value of SYNC_TRGEN. Synced Trigger Select Enable for Timestamp 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

Parameters

<i>pSdad</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SYNC_TRGEN.

**LOCAL_INLINE uint32_t SDADC_HWA_GetCTSCNT_SYNC_TRGSEL (const SDADC_Type *const *pSdad*,
uint8_t *u8Channel*)**

Get the value of SYNC_TRGSEL. Synced Trigger Select for Timestamp Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - Select the trigger from the channel 0. 001b - Select the trigger from the channel 1. 010b - Select the trigger from the channel 2. ...

Parameters

<i>pSdad</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SYNC_TRGSEL.

**LOCAL_INLINE uint32_t SDADC_HWA_GetCUTOFF (const SDADC_Type *const *pSdad*, uint8_t
u8Channel)**

Get the value of CUTOFF. Cutoff Frequency Setting for High Pass Filter(HPF) HPF can only be enabled when RDFMT=0. 000b - HPF is disabled 001b - HPF is enabled and setting the cutoff frequency to rate 1(lowest cutoff frequency) 010b - HPF is enabled and setting the cutoff frequency to rate 2. 011b - HPF is enabled and setting the cutoff frequency to rate 3. 100b - HPF is enabled and setting the cutoff frequency to rate 4. 101b - HPF is enabled and setting the cutoff frequency to rate 5. 110b - HPF is enabled and setting the cutoff frequency to rate 6. 111b - HPF is enabled and setting the cutoff frequency to rate 7.(highest cutoff frequency)

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of CUTOFF.

LOCAL_INLINE uint32_t SDADC_HWA_GetDBG_SEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of DBG_SEL. Debug Output Selection Select which data is output when read debug register 0100b - Instantaneous Primary Filter shifted 24bit Data. 0110b - Instantaneous HPF Output Data. 1000b - Instantaneous GAIN Output Data. 1010b - Instantaneous Secondary Filter CIC Data. 1110b - Instantaneous Primary Filter offset removed 24bit Data. others - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of DBG_SEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetDBGE (const SDADC_Type *const *pSdadc*)

Get the value of DBGE. Module Enable in Debug Mode Enables/disables the SDADC operation in Debug mode. 0b - SDADC is enabled in debug mode. 1b - SDADC is disabled in debug mode. Need trigger again for new conversion.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of DBGE.

LOCAL_INLINE uint32_t SDADC_HWA_GetDBGM (const SDADC_Type *const *pSdadc*)

Get the value of DBGM. Software Debug Mode Request This bit is writable only when MEN is asserted. 0b - SDADC is in Normal Mode. 1b - SDADC is in Debug Mode.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of DBGM.

LOCAL_INLINE uint32_t SDADC_HWA_GetDITHER_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of DITHER_EN. Dither Enable This bit enable the dither to analog circuit to remove the idle tone. 0b - Dither is disabled. 1b - Dither is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of DITHER_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetDLY_NEG (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of DLY_NEG. Sign Delay Value for Sign Negedge Defines the content of SD_CNT to generate a positive delayed sign DSIGN.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of DLY_NEG.

LOCAL_INLINE uint32_t SDADC_HWA_GetDLY_POS (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of DLY_POS. Sign Delay Value for Sign Posedge Defines the content of SD_CNT to generate a negative delayed sign DSIGN.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of DLY_POS.

LOCAL_INLINE uint32_t SDADC_HWA_GetDLY_TRG (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of DLY_TRG. Sign Delay Value for TSIGN Posedge TSIGN is clear at DLY_TRG and set at DLY_NEG in CSDRn.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of DLY_TRG.

LOCAL_INLINE uint32_t SDADC_HWA_GetDMA_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of DMA_EN. DMA Enable 0b - DMA is disabled for the channel. 1b - DMA is enabled for the channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of DMA_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetDSIGN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of DSIGN. Delayed Sign Signal 0b - Positive values. 1b - Negative values.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of DSIGN.

LOCAL_INLINE uint32_t SDADC_HWA_GetEXTSTP (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of EXTSTP. Integration Stop Control For Hardware trigger control only. 0b - Internal control, stop integration after NVALREP+1 integration cycles, support both edge and level events. 1b - External control, stop integration upon the inverse trigger event.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of EXTSTP.

LOCAL_INLINE uint32_t SDADC_HWA_GetFCRC (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of FCRC. Filter Chain Restart Control 0b - Do not restart. 1b - Restart the filter chain when an integration window starts.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of FCRC.

LOCAL_INLINE uint32_t SDADC_HWA_GetFIFO_EMPTY (const SDADC_Type *const *pSdadc*)

Get the value of FIFO_EMPTY. FIFO Empty Flag bit 18-16 are for filter channel 2-0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FIFO_EMPTY.

LOCAL_INLINE uint32_t SDADC_HWA_GetFIFO_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of FIFO_EN. FIFO Enable 0b - FIFO is disabled for the channel. 1b - FIFO is enabled for the channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of FIFO_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetFOVF0 (const SDADC_Type *const *pSdadc*)

Get the value of FOVF0. Filter Channel 0 FIFO Overflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FOVF0.

LOCAL_INLINE uint32_t SDADC_HWA_GetFOVF1 (const SDADC_Type *const *pSdadc*)

Get the value of FOVF1. Filter Channel 1 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FOVF1.

LOCAL_INLINE uint32_t SDADC_HWA_GetFOVF2 (const SDADC_Type *const *pSdadc*)

Get the value of FOVF2. Filter Channel 2 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FOVF2.

LOCAL_INLINE uint32_t SDADC_HWA_GetFOVFIE0 (const SDADC_Type *const *pSdadc*)

Get the value of FOVFIE0. Filter Channel 0 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FOVFIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetFOVFIE1 (const SDADC_Type *const *pSdadc*)

Get the value of FOVFIE1. Filter Channel 1 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FOVFIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetFOVFIE2 (const SDADC_Type *const *pSdadc*)

Get the value of FOVFIE2. Filter Channel 2 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FOVFIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetFRDY0 (const SDADC_Type *const *pSdadc*)

Get the value of FRDY0. Filter Channel 0 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FRDY0.

LOCAL_INLINE uint32_t SDADC_HWA_GetFRDY1 (const SDADC_Type *const *pSdadc*)

Get the value of FRDY1. Filter Channel 1 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FRDY1.

LOCAL_INLINE uint32_t SDADC_HWA_GetFRDY2 (const SDADC_Type *const *pSdadc*)

Get the value of FRDY2. Filter Channel 2 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of FRDY2.

LOCAL_INLINE uint32_t SDADC_HWA_GetFRDYIE0 (const SDADC_Type *const *pSdac*)

Get the value of FRDYIE0. Filter Channel 0 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of FRDYIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetFRDYIE1 (const SDADC_Type *const *pSdac*)

Get the value of FRDYIE1. Filter Channel 1 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of FRDYIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetFRDYIE2 (const SDADC_Type *const *pSdac*)

Get the value of FRDYIE2. Filter Channel 2 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FRDYIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetFUNF0 (const SDADC_Type *const *pSdadc*)

Get the value of FUNF0. Filter Channel 0 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FUNF0.

LOCAL_INLINE uint32_t SDADC_HWA_GetFUNF1 (const SDADC_Type *const *pSdadc*)

Get the value of FUNF1. Filter Channel 1 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FUNF1.

LOCAL_INLINE uint32_t SDADC_HWA_GetFUNF2 (const SDADC_Type *const *pSdadc*)

Get the value of FUNF2. Filter Channel 2 FIFO underflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FUNF2.

LOCAL_INLINE uint32_t SDADC_HWA_GetFUNFIE0 (const SDADC_Type *const *pSdadc*)

Get the value of FUNFIE0. Filter Channel 0 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FUNFIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetFUNFIE1 (const SDADC_Type *const *pSdadc*)

Get the value of FUNFIE1. Filter Channel 1 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FUNFIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetFUNFIE2 (const SDADC_Type *const *pSdadc*)

Get the value of FUNFIE2. Filter Channel 2 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of FUNFIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetFWMK (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of FWMK. FIFO Water Mark Controls the watermark of the FIFO used to set FRDYN. When the number of results in the FIFO is greater than FWMK value, the FRDYN flag will be set. A DMA request or interrupt can also be generated if related DMA_EN and FRDYIE bit-field is set.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of FWMK.

LOCAL_INLINE uint32_t SDADC_HWA_GetGAIN_FAC (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of GAIN_FAC. Gain Factor This bit field defines a gain factor($GAIN_FAC[17:0]/2^{16}$, range is from 0 to 4) which is multiplied with the data after HPF to compensate the gain error caused by analog circuits or the gain mismatch when decimation factor is not 2^N , after compensation, the total gain from analog input to data from FIFO should be 1. GAIN_FAC can be got from calibration, it should be in a suitable range, or the data output may be saturated. When $GAIN_FAC[17:0]=0$, GAIN block is bypassed.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of GAIN_FAC.

LOCAL_INLINE uint32_t SDADC_HWA_GetGainOPT0 (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of OPT0. Gain Option 0 This is a gain factor provided by factory test for modulator analog input path gain=1. User need read this field and multiply it with gain correction factor 2^m and then write to related CGNR register.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of OPT0.

LOCAL_INLINE uint32_t SDADC_HWA_GetGainOPT1 (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of OPT1. Gain Option 1 This is a gain factor provided by factory test for modulator analog input path gain=2. User need read this field and multiply it with gain correction factor 2^m and then write to related CGNR register.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of OPT1.

LOCAL_INLINE uint32_t SDADC_HWA_GetGAINSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of GAINSEL. Gain Select of Analog Input Path 00b - 1. 01b - 2. 10b - 4. 11b - 8.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of GAINSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetHFVREF_EN (const SDADC_Type *const *pSdadc*)

Get the value of HFVREF_EN. Half VREFH output Enable to Related Internal Modulator Analog. Bit 4 enable internal Half VREFH output to modulator analog for channel 0. Bit 5 enable internal Half VREFH output to

modulator analog for channel 1. Bit 6 enable internal Half VREFH output to modulator analog for channel 2. Bit 7 enable internal Half VREFH output to modulator analog for channel 3. NOTE: Not all channels have internal modulator analog, if the channel has no internal modulator analog, related control bit is reserved.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of HFVREF_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetHILMT (const SDADC_Type *const *pSdac*, uint8_t *u8Channel*)

Get the value of HILMT. High Limit Threshold When data value exceeds this value, high limit event will occur. The limit value format (signed or unsigned) is depended on RDFMT.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of HILMT.

LOCAL_INLINE uint32_t SDADC_HWA_GetHLMT0 (const SDADC_Type *const *pSdac*)

Get the value of HLMT0. Filter Channel 0 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of HLMT0.

LOCAL_INLINE uint32_t SDADC_HWA_GetHLMT1 (const SDADC_Type *const *pSdac*)

Get the value of HLMT1. Filter Channel 1 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of HLMT1.

LOCAL_INLINE uint32_t SDADC_HWA_GetHLMT2 (const SDADC_Type *const *pSdadc*)

Get the value of HLMT2. Filter Channel 2 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of HLMT2.

LOCAL_INLINE uint32_t SDADC_HWA_GetHLMT_BKEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of HLMT_BKEN. Break Signal Enable for High Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of HLMT_BKEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetHLMTIE0 (const SDADC_Type *const *pSdadc*)

Get the value of HLMTIE0. Filter Channel 0 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of HLMTIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetHLMTIE1 (const SDADC_Type *const *pSdadc*)

Get the value of HLMTIE1. Filter Channel 1 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of HLMTIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetHLMTIE2 (const SDADC_Type *const *pSdadc*)

Get the value of HLMTIE2. Filter Channel 2 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of HLMTIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetHTRPOL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of HTRPOL. Hardware Trigger Polarity 0b - High and rising edge effective. 1b - Low and falling edge effective.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of HTRPOL.

LOCAL_INLINE uint32_t SDADC_HWA_GetHYSTR_PROTEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of HYSTR_PROTEN. Protection Singal Enabled for Limit Hysteresis Result The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal is from the result of hysteresis comparator in limit detector and will keep on if related condition keeps(keep high/keep low) until the condition is not satisfied. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of HYSTR_PROTEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetIBFMT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of IBFMT. Input Bit Format 00b - External one bit stream. 01b - External Manchester code, CLKEDG will determine rise or fall decoder. 10b - Reserved. 11b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of IBFMT.

LOCAL_INLINE uint32_t SDADC_HWA_GetINCFGM (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INCFGM. Configuration of Negative Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>u8Channel</i>	SDADC Channel.
------------------	----------------

Returns

uint32_t the value of INCFGM.

LOCAL_INLINE uint32_t SDADC_HWA_GetINCFGP (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INCFGP. Configuration of Positive Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INCFGP.

LOCAL_INLINE uint32_t SDADC_HWA_GetINCFGPV11 (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INCFGPV11. Configuration of Positive Input Line with V11 from System This bit have higher priority then INCFGP, this this bit is set, INCFGP is ignored. 0b - Positive input from V11 disabled. 1b - Positive input from V11 enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INCFGPV11.

LOCAL_INLINE uint32_t SDADC_HWA_GetINMOD (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INMOD. Input MUX Control Mode 00b - Software control mode (INMUX follows INSEL). 01b - Preset mode (load INMUX upon a channel conversion trigger). 10b - Scan mode (decrement INMUX upon a channel conversion trigger, wrap around to INSEL). 11b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INMOD.

LOCAL_INLINE uint32_t SDADC_HWA_GetINMUX (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INMUX. Active Input Pin Selection Indicates the active input pin selection for input MUX. 00b - Input pin 0 selected. 01b - Input pin 1 selected. 10b - Input pin 2 selected. 11b - Input pin 3 selected.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INMUX.

LOCAL_INLINE uint32_t SDADC_HWA_GetINMXCP (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INMXCP. Input Mux Setting Copy Enable 0b - Disabled. 1b - Copy INMUX from CMACSR to TSTAMP[15:14].

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INMXCP.

LOCAL_INLINE uint32_t SDADC_HWA_GetINSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INSEL. Input Pin Selection Defines the initial or permanent setting for the input MUX(bits INMUX) depending on the selected operating mode (bits INMOD).

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetINTG_EN (const SDADC_Type *const *pSdac*, uint8_t *u8Channel*)

Get the value of INTG_EN. Integration Enable Enable the integration block. 0b - Disabled and bypassed. 1b - Enabled.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INTG_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetINTG_STRIG0 (const SDADC_Type *const *pSdac*)

Get the value of INTG_STRIG0. Integration Software Trigger for channel 0 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
--------------	---

Returns

uint32_t the value of INTG_STRIG0.

LOCAL_INLINE uint32_t SDADC_HWA_GetINTG_STRIG1 (const SDADC_Type *const *pSdac*)

Get the value of INTG_STRIG1. Integration Software Trigger for channel 1 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of INTG_STRIG1.

LOCAL_INLINE uint32_t SDADC_HWA_GetINTG_STRIG2 (const SDADC_Type *const *pSdadc*)

Get the value of INTG_STRIG2. Integration Software Trigger for channel 2 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of INTG_STRIG2.

LOCAL_INLINE uint32_t SDADC_HWA_GetINTGON (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INTGON. Integration on Flag Indicates the activity of the integration. 0b - Integration stopped. 1b - Integration enabled and on-going.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INTGON.

LOCAL_INLINE uint32_t SDADC_HWA_GetINTVAL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of INTVAL. Results of most recent accumulation, this field can only be checked after MCIC_COC assert.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of INTVAL.

LOCAL_INLINE uint32_t SDADC_HWA_GetIOGT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of IOGT. Integration Output Gating Setting this bit will gate conversion complete flag and FIFO push when Integration is not on-going. conversion complete flag and FIFO push can only be effective when Integration is not on-going (INTON asserted) 0b - Conversion complete flag and FIFO push always works. 1b - Conversion complete flag and FIFO push can only work when Integration is not on-going (INTON asserted)

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of IOGT.

LOCAL_INLINE uint32_t SDADC_HWA_GetISVAL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of ISVAL. Integration Shift value 000b - Shift 0 bits 001b - Shift 1 bits, for 2 values integration 010b - Shift 2 bits, for 3-4 values integration 011b - Shift 3 bits, for 5-8 values integration 100b - Shift 4 bits, for 9-16 values integration 101b - Shift 5 bits, for 17-32 values integration 110b - Shift 6 bits, for 33-64 values integration.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of ISVAL.

LOCAL_INLINE uint32_t SDADC_HWA_GetITRSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of ITRSEL. Integration Trigger Select 0b - Software trigger selected. 1b - Hardware trigger dedicate to channel selected.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of ITRSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetLLMT0 (const SDADC_Type *const *pSdadc*)

Get the value of LLMT0. Filter Channel 0 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of LLMT0.

LOCAL_INLINE uint32_t SDADC_HWA_GetLLMT1 (const SDADC_Type *const *pSdadc*)

Get the value of LLMT1. Filter Channel 1 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of LLMT1.

LOCAL_INLINE uint32_t SDADC_HWA_GetLLMT2 (const SDADC_Type *const *pSdadc*)

Get the value of LLMT2. Filter Channel 2 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of LLMT2.

LOCAL_INLINE uint32_t SDADC_HWA_GetLLMT_BKEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LLMT_BKEN. Break Signal Enabled for Low Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LLMT_BKEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetLLMTIE0 (const SDADC_Type *const *pSdadc*)

Get the value of LLMTIE0. Filter Channel 0 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of LLMTIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetLLMTIE1 (const SDADC_Type *const *pSdadc*)

Get the value of LLMTIE1. Filter Channel 1 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of LLMTIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetLLMTIE2 (const SDADC_Type *const *pSdadc*)

Get the value of LLMTIE2. Filter Channel 2 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of LLMTIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetLMT_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LMT_EN. Limit Enable 0b - Limit check is disabled for the Main or Auxiliary Filter. 1b - Limit check is enabled for the Main or Auxiliary Filter.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LMT_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetLMT_SEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LMT_SEL. Limit Check Input Selection Select the input from Main Filter or Auxiliary Filter. 0b - Auxiliary Filter. 1b - Main Filter.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LMT_SEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetLMTAVT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LMTAVT. Limitation Result Activation Select The conversion results are compared with HILMTn and LOLMTn in limit detector, and HILMTn and LOLMTn can define a hysteresis range , A compare result is set/clear and sent to system with configurable activation. Note, When this hysteresis compare function is used in system, LMTOPTn need to be 00b to enable detect on exceed both high and low value. 0b - The compare result is set only when the conversion result is larger than HILMTn and clear only when the conversion result is smaller than LOLMTn. 1b - The compare result is clear only when the conversion result is larger than HILMTn and set only when the conversion result is smaller than LOLMTn.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LMTAVT.

LOCAL_INLINE uint32_t SDADC_HWA_GetLMTF_PROTEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LMTF_PROTEN. Protection Singal Enabled for Limit Flag The protection signal can be enabled and sent to TRGSEL for application protection function. LMTOPT can be used to select which limit flag(HLMT/LLMT/WLMT) is used as the protection source. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LMTF_PROTEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetLMTOPT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LMTOPT. Limitation Option 00b - Limitation detect on exceed both high and low value. 01b - Limitation detect on exceed high threshold value. 10b - Limitation detect on exceed low threshold value. 11b - Limitation detect on windowed value.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LMTOPT.

LOCAL_INLINE uint32_t SDADC_HWA_GetLMTR_PROTEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LMTR_PROTEN. Protection Singal Enabled for Limit Result The protection signal can be enabled and sent to TRGSEL for application protection function. LMTOPT can be used to select which limit result(high/low/window limit result) is used as the protection source. This protection signal will keep on if related condition keeps(keep high/keep low/keep in window) until the condition is not satisfied. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LMTR_PROTEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetLOLMT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of LOLMT. Low Limit Threshold When the data value exceeds this value, a low limit event will occur. The limit value format (signed or unsigned) depends on RDFMT.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of LOLMT.

LOCAL_INLINE uint32_t SDADC_HWA_GetMACLKSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MACLKSEL. Internal Modulator Analog Clock Selection 00b - Div_clk1 is selected as the clock, output bitstream is sampled on a clock with frequency of F_div_clk1/2. 01b - Div_clk0 is selected as the clock, output bitstream is sampled on a clock with frequency of F_div_clk1/2. 1*b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MACLKSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetMASEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MASEL. Modulator Analog Selection 0b - External modulator analog is selected as channel input. 1b - Internal modulator analog is selected as channel input.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MASEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetMCIC_COC0 (const SDADC_Type *const pSdadc)

Get the value of MCIC_COC0. Filter channel 0 Conversion Complete Flag for MCIC. Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which is means integration output valid.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of MCIC_COC0.

LOCAL_INLINE uint32_t SDADC_HWA_GetMCIC_COC1 (const SDADC_Type *const pSdadc)

Get the value of MCIC_COC1. Filter Channel 1 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which is means integration output valid.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of MCIC_COC1.

LOCAL_INLINE uint32_t SDADC_HWA_GetMCIC_COC2 (const SDADC_Type *const pSdadc)

Get the value of MCIC_COC2. Filter Channel 2 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which is means integration output valid.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of MCIC_COC2.

LOCAL_INLINE uint32_t SDADC_HWA_GetMCLK_DIV (const SDADC_Type *const *pSdadc*)

Get the value of MCLK_DIV. DIV Divider Value The (MCLK_DIV+1) sets the clock divider ratio for DIV block. The minimum value MCLK_DIV is 1.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of MCLK_DIV.

LOCAL_INLINE uint32_t SDADC_HWA_GetMEN (const SDADC_Type *const *pSdadc*)

Get the value of MEN. Main Enable Setting this bit enables all Function block simultaneously if CHEN and corresponding enable bit are asserted. 0b - All function blocks are disabled. 1b - All function blocks are enabled simultaneously.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of MEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetMENSEL (const SDADC_Type *const *pSdadc*)

Get the value of MENSEL. Main Enable Selection Setting this bit to select main enable(MEN) from an enable from external of SDADC IP. By this way, all SDADC in a system can share one main enable and can be enable at the same time. 0b - Use internal MEN. 1b - Use external MEN.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of MENSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetMF_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MF_EN. Main Filter Enable 0b - The Main Filter is disabled for the channel. 1b - The Main Filter is enabled for the channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MF_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetMFCM (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MFCM. Main Filter Conversion Mode 00b - Single mode. One conversion following a triggering event (edge or level). 01b - Continuous conversion mode. Multiple conversions following a triggering event, and the next trigger will cancel and restart conversion. 10b - Always conversion mode. Multiple conversions following the first trigger, and the next trigger will be ignored. 11b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MFCM.

LOCAL_INLINE uint32_t SDADC_HWA_GetMFORD (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MFORD. Main Filter Order 000b - Reserved. 001b - First order filter. 010b - Second order filter. 011b - Third order filter. 100b - Fourth order filter. 101b - Fifth order filter. 110,111 - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MFORD.

LOCAL_INLINE uint32_t SDADC_HWA_GetMFOSR (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MFOSR. Main Filter Oversampling Rate The oversampling rate = (MFOSR + 1). The lower valid of MFOSR value is 3. The higher is depended on MFORD, in order to keep the internal CIC raw result not exceeding 32bit. If MFORD = 4, the maximum value is 203. If MFORD = 5, the maximum value is 72, else the maximum value is 511.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MFOSR.

LOCAL_INLINE uint32_t SDADC_HWA_GetMFSHIFT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MFSHIFT. Main Filter Shift to Get Correct 24 bit Precision Main Filter CIC raw result is always 32bit width. The user should program MFSHIFT for cut-off and shift left/right to get 24bit result. MFSHIFT[4] = 1 means left shift, and MFSHIFT[4] = 0 means right shift. MFSHIFT[3:0] is the shift length.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MFSHIFT.

LOCAL_INLINE uint32_t SDADC_HWA_GetMPDATA (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of MPDATA. Multi Purpose Data This register can be accessed by DBG_SEL.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of MPDATA.

LOCAL_INLINE uint32_t SDADC_HWA_GetNVALDIS (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of NVALDIS. Number of Values discarded Start the integration cycle after NVALDIS values.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of NVALDIS.

LOCAL_INLINE uint32_t SDADC_HWA_GetNVALINT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of NVALINT. Number of Values to be Accumulated NVALINT+1 defines a complete integration cycle.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of NVALINT.

LOCAL_INLINE uint32_t SDADC_HWA_GetNVALREP (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of NVALREP. Number of Integration Cycles NVALREP+1 defines number of integration cycles to be counted by REPCNT if activated(EXTSTP=0)

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of NVALREP.

LOCAL_INLINE uint32_t SDADC_HWA_GetOffsetOPT0 (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of OPT0. Offset Option 0 This is an offset value provided by factory test for modulator analog input path gain=1. User need read this field and left shift it by 8 bit to form a 24-bit offset and then write to related CBR register.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of OPT0.

LOCAL_INLINE uint32_t SDADC_HWA_GetOffsetOPT1 (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of OPT1. Offset Option 1 This is an offset value provided by factory test for modulator analog input path gain=2. User need read this field and left shift it by 8 bit to form a 24-bit offset and then write to related CBR register.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>u8Channel</i>	SDADC Channel.
------------------	----------------

Returns

uint32_t the value of OPT1.

LOCAL_INLINE uint32_t SDADC_HWA_GetON (const SDADC_Type *const *pSdadc*)

Get the value of ON. ON Status 0b - PWM generator is stopped 1b - PWM generator is ON.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of ON.

LOCAL_INLINE uint32_t SDADC_HWA_GetPAT_CNT (const SDADC_Type *const *pSdadc*)

Get the value of PAT_CNT. PWM Pattern Counter Value Counts the 32 steps for each PWM pattern period.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of PAT_CNT.

LOCAL_INLINE uint32_t SDADC_HWA_GetPATTERN (const SDADC_Type *const *pSdadc*)

Get the value of PATTERN. PWM Generator Pattern Config this before set MEN. Clear the bits during PWM generator run will terminate the PWM generator outputs after completion of the current period(indicated by bit ON=0) 00b - Stopped 01b - Square wave 10b - Triangle 11b - Sine wave.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of PATTERN.

LOCAL_INLINE uint32_t SDADC_HWA_GetPOL (const SDADC_Type *const *pSdadc*)

Get the value of POL. PWM Generator Polarity 0b - PWM generator output begin with +1 1b - PWM generator output begin with -1.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of POL.

LOCAL_INLINE uint32_t SDADC_HWA_GetPRESCALE (const SDADC_Type *const *pSdadc*)

Get the value of PRESCALE. PRE_DIV Divider Value 00b - No prescale. 01b - Divided by 2. 10b - Divided by 4. 11b - Divided by 8.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of PRESCALE.

LOCAL_INLINE uint32_t SDADC_HWA_GetPWM_CNT (const SDADC_Type *const *pSdadc*)

Get the value of PWM_CNT. PWM Counter Value Counts the 32 cycles for each PWM pattern step.(-16 to 15)

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of PWM_CNT.

LOCAL_INLINE uint32_t SDADC_HWA_GetPWMCLKSEL (const SDADC_Type *const *pSdadc*)

Get the value of CLKSEL. PWM Generator Clock selection 00b - div_clk0 01b - div_clk1 10b - div_clk2 11b - off.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of CLKSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetRDATA (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of RDATA. Result Data Read this register to get the conversion results. Bit[31] is sign bit if RDFMT is signed and only support 32bit read. The data format may changed by TSRDM. When TSRDM=2'b00, read CRDATA will pop data from FIFO. RDATA[31:8] is a 24-bit conversion results and RDATA[7:0] is 8'h00. At this case, CRDATA can only be read when FIFO is enabled and not empty. When TSRDM=2'b01, read CRDATA will feedback latest conversion result and timestamp info, RDATA[31:16] is a cut and round 16-bit latest conversion result(update on last COC) and RDATA[15:0] is latest timestamp counter value. FIFO is not used. At this case, CRDATA can only be read at any time. When TSRDM=2'b10, read CRDATA will pop data from FIFO and a timestamp info captured by trigger will also be included. RDATA[31:16] is a cut and round 16-bit result from FIFO and RDATA[15:0] is captured timestamp counter value. FIFO is used and gated by timestamp trigger. At this case, CRDATA can only be read when FIFO is enabled and not empty.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of RDATA.

LOCAL_INLINE uint32_t SDADC_HWA_GetRDFMT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of RDFMT. Result Data Format 0b - Left justified, signed, for the case where input bitstream is signed 1b - Left justified, unsigned, for the case where input bitstream is unsigned.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of RDFMT.

LOCAL_INLINE uint32_t SDADC_HWA_GetRECT_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of RECT_EN. Rectification Enable Enable the rectification block. 0b - Disabled and bypassed. 1b - Enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of RECT_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetRESET (const SDADC_Type *const *pSdadc*)

Get the value of RESET. Software Reset Used to reset all function blocks (filters, short circuit detect, etc) and all flags or status. Note, to reset all function domain, RESET must keep high at least 3 div_clk1 cycles. 0b - No reset. 1b - All function blocks are reset and disabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of RESET.

LOCAL_INLINE uint32_t SDADC_HWA_GetRSTSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of RSTSEL. Modulator Analog Reset Cycle Selection Select RSTSEL+1 mclk cycles for modulator analog reset after a channel conversion trigger. This is needed especially when the input mux(INMUX) is changed. 0d - Modulator analog is not reset after a channel conversion trigger 1d - Modulator analog is reset for 2 mclk cycles after a channel conversion trigger. Total 7 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 2d - Modulator analog is reset for 3 mclk cycles after a channel conversion trigger. Total 8 mclk cycle cycles needed for filters start to run after a channel conversion trigger. ... 8d -

Modulator analog is reset for 9 mclk cycles after a channel conversion trigger. Total 14 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 9-15d - Modulator analog is reset for 10 mclk cycles after a channel conversion trigger. Total 15 mclk cycle cycles needed for filters start to run after a channel conversion trigger.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of RSTSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetSAT0 (const SDADC_Type *const *pSdadc*)

Get the value of SAT0. Filter Channel 0 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SAT0.

LOCAL_INLINE uint32_t SDADC_HWA_GetSAT1 (const SDADC_Type *const *pSdadc*)

Get the value of SAT1. Filter Channel 1 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SAT1.

LOCAL_INLINE uint32_t SDADC_HWA_GetSAT2 (const SDADC_Type *const *pSdadc*)

Get the value of SAT2. Filter Channel 2 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SAT2.

LOCAL_INLINE uint32_t SDADC_HWA_GetSATIE0 (const SDADC_Type *const *pSdadc*)

Get the value of SATIE0. Filter Channel 0 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SATIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetSATIE1 (const SDADC_Type *const *pSdadc*)

Get the value of SATIE1. Filter Channel 1 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SATIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetSATIE2 (const SDADC_Type *const *pSdadc*)

Get the value of SATIE2. Filter Channel 2 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SATIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCD0 (const SDADC_Type *const *pSdadc*)

Get the value of SCD0. Filter Channel 0 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SCD0.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCD1 (const SDADC_Type *const *pSdadc*)

Get the value of SCD1. Filter Channel 1 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SCD1.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCD2 (const SDADC_Type *const *pSdadc*)

Get the value of SCD2. Filter Channel 2 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SCD2.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCD_BKEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SCD_BKEN. Break Signal Enabled for Short Circuit Event 0b - The Break signal is not enabled.
1b - The Break signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SCD_BKEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCD_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SCD_EN. Short Circuit Detect Enable 0b - Short current detection is disabled for the channel.
1b - Short current detection is enabled for the channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SCD_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCD_PROTEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SCD_PROTEN. Protection Singal Enabled for SCD Flag The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SCD_PROTEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCDCM (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SCDCM. Short Circuit Detect Conversion Mode 0b - Always on when CHEN and MEN are asserted. 1b - Only works when the any filter is in progress.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SCDCM.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCDIE0 (const SDADC_Type *const *pSdadc*)

Get the value of SCDIE0. Filter Channel 0 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SCDIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCDIE1 (const SDADC_Type *const *pSdadc*)

Get the value of SCDIE1. Filter Channel 1 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SCDIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCDIE2 (const SDADC_Type *const *pSdadc*)

Get the value of SCDIE2. Filter Channel 2 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SCDIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCDLMT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SCDLMT. Short Circuit Detector Limitation extension The threshold value for short circuit detector counter. The counter indicates the number of continuous received bit 0 or bit 1. If this value is reached, a short-circuit detector event will occur on the dedicated channel.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SCDLMT.

LOCAL_INLINE uint32_t SDADC_HWA_GetSCDOPT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SCDOPT. Short Circuit Option 00b - Count on both 0 and 1. Detect both high and low value.
01b - Count on 1. Detect high value. 10b - Count on 0. Detect low value. 11b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SCDOPT.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CAP (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SD_CAP. Sign Delay Capture Value Indicates the results value counted between the negedge of SSIGN and the first received positive value.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SD_CAP.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CAPVLD0 (const SDADC_Type *const *pSdadc*)

Get the value of SD_CAPVLD0. Capture Valid Flag for channel 0 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SD_CAPVLD0.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CAPVLD1 (const SDADC_Type *const *pSdadc*)

Get the value of SD_CAPVLD1. Capture Valid Flag for channel 1 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SD_CAPVLD1.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CAPVLD2 (const SDADC_Type *const *pSdadc*)

Get the value of SD_CAPVLD2. Capture Valid Flag for channel 2 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SD_CAPVLD2.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CAPVLD_IE0 (const SDADC_Type *const *pSdadc*)

Get the value of SD_CAPVLD_IE0. Capture Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SD_CAPVLD_IE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CAPVLD_IE1 (const SDADC_Type *const *pSdadc*)

Get the value of SD_CAPVLD_IE1. Capture Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SD_CAPVLD_IE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CAPVLD_IE2 (const SDADC_Type *const *pSdadc*)

Get the value of SD_CAPVLD_IE2. Capture Valid Interrupt Enable for channel 2 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SD_CAPVLD_IE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetSD_CNT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SD_CNT. Sign Delay Counter Counts the results values from the filter chain to delay the sign signal, this field can only be checked after MCIC_COC assert.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SD_CNT.

LOCAL_INLINE uint32_t SDADC_HWA_GetSDON (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SDON. Sign Delay Counter on Flag Indicates the activity of the sign delay counter. 0b - Sign delay counter stopped. 1b - Sign delay counter enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SDON.

LOCAL_INLINE uint32_t SDADC_HWA_GetSIGN (const SDADC_Type *const *pSdadc*)

Get the value of SIGN. Sign Signal for the PWM pattern 0b - Positive values 1b - Negative values.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of SIGN.

LOCAL_INLINE uint32_t SDADC_HWA_GetSSCH (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SSCH. Sign Signal From Channel Selection Select the channel providing the sign signal if SSRC = 01b 0000b - From channel 0. 0001b - From channel 1. 0010b - From channel 2. ...

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SSCH.

LOCAL_INLINE uint32_t SDADC_HWA_GetSSIGN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SSIGN. Selected Sign Signal 0b - Positive values. 1b - Negative values.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SSIGN.

LOCAL_INLINE uint32_t SDADC_HWA_GetSSRC (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SSRC. Sign Source Selection Select the sign signal to be delayed. 00b - SDADC internal PWM generation block. 01b - Sign of result of channel selected by SSCH. 10b - External sign signal 0. 11b - External sign signal 1.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SSRC.

LOCAL_INLINE uint32_t SDADC_HWA_GetSTRIG0 (const SDADC_Type *const *pSdadc*)

Get the value of STRIG0. Software Trigger for Filter Channel 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of STRIG0.

LOCAL_INLINE uint32_t SDADC_HWA_GetSTRIG1 (const SDADC_Type *const *pSdadc*)

Get the value of STRIG1. Software Trigger for Filter Channel 1.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of STRIG1.

LOCAL_INLINE uint32_t SDADC_HWA_GetSTRIG2 (const SDADC_Type *const *pSdadc*)

Get the value of STRIG2. Software Trigger for Filter Channel 2.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of STRIG2.

LOCAL_INLINE uint32_t SDADC_HWA_GetSYNC_TRGSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of SYNC_TRGSEL. Synced Trigger Select Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - select the trigger from the channel 0. 001b - select the trigger from the channel 1. 010b - select the trigger from the channel 2. ...

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of SYNC_TRGSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetTRGLVL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TRGLVL. Trigger Level type 0b - Trigger is edge event. Next edge event will cancel the current conversion, and re-start in single or continuous mode. 1b - Trigger is level event. De-assert cancels the current conversion, and stop filter function in single or continuous mode.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TRGLVL.

LOCAL_INLINE uint32_t SDADC_HWA_GetTRGSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TRGSEL. Trigger Select 0b - Software trigger is selected. 1b - Hardware trigger is selected.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>u8Channel</i>	SDADC Channel.
------------------	----------------

Returns

uint32_t the value of TRGSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetTRGSIGN_OEN (const SDADC_Type *const *pSdadc*)

Get the value of TRGSIGN_OEN. Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once SIGN falling edge is got.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TRGSIGN_OEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetTS_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TS_EN. Timestamp Counter Enable 0b - Disabled. 1b - Counting at the rate selected by TSCLKSEL.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TS_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSCLKSEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TSCLKSEL. Timestamp Counter Clock Selection 00b - Modulator clock. 01b - Modulator clock divided by 2. 10b - Modulator clock divided by 4. 11b - Modulator clock divided by 8.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TSCLKSEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSCNT (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TSCNT. Timestamp Counter Value TSCNT is clocked with the modulator clock and is cleared when a new result value has been generated.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TSCNT.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSDATA (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TSDATA. The Latest Captured Conversion Result Copied from saved conversion result.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TSDATA.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSIGN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TSIGN. Sign Signal for Channel Trigger Output This sign can be configured to some cycles before DSIGN, and can be used to generate channel trigger output, typically can be used to trigger integration so that it can start at the ZCD point 0b - Positive values. 1b - Negative values.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>u8Channel</i>	SDADC Channel.
------------------	----------------

Returns

uint32_t the value of TSIGN.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSIGN_P2N0 (const SDADC_Type *const *pSdadc*)

Get the value of TSIGN_P2N0. Trigger Sign Changed from 1 to 0 for Channel 0 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSIGN_P2N0.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSIGN_P2N1 (const SDADC_Type *const *pSdadc*)

Get the value of TSIGN_P2N1. Trigger Sign Changed from 1 to 0 for Channel 1 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSIGN_P2N1.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSIGN_P2N2 (const SDADC_Type *const *pSdadc*)

Get the value of TSIGN_P2N2. Trigger Sign Changed from 1 to 0 for Channel 2 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSIGN_P2N2.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSIGN_P2N_IE0 (const SDADC_Type *const *pSdadc*)

Get the value of TSIGN_P2N_IE0. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSIGN_P2N_IE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSIGN_P2N_IE1 (const SDADC_Type *const *pSdadc*)

Get the value of TSIGN_P2N_IE1. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSIGN_P2N_IE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSIGN_P2N_IE2 (const SDADC_Type *const *pSdadc*)

Get the value of TSIGN_P2N_IE2. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSIGN_P2N_IE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSRDM (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TSRDM. Timestamp Information Read Mode 00b - Default mode, timestamp information can only be got by reading CTSINFO register captured by trigger event. FIFO is used for conversion results only and can be pop from FIFO by reading CRDATAn.

01b - Direct mode. Timestamp information is saved in CRDATAn[15:0]. Conversion result is not saved in FIFO(FIFO pop and push are gated), it is cut, round to 16-bit and saved in CRDATAn[31:16] once conversion completes. In this mode, application can read conversion result and timestamp information at same time.

10b - FIFO gating mode. The FIFO is gated before the timestamp trigger event(none active trigger level), a timestamp trigger event(rising edge or falling edge) will capture the current timestamp and save it in TSTAMP[15:0] of CTSINFO register and CRDATAn[15:0] is bypass to TSTAMP[15:0]. This trigger will also capture last conversion result and save it in FIFO, and then following conversion results are push to FIFO one by one until trigger level is not active again, the 24 bits result out from FIFO is cut/ round to 16 bits and sent to CRDATAn[31:16]. In this mode, application read CRDATAn will get both conversion result and captured timestamp information at same time after the trigger.

11b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TSRDM.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSTAMP (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TSTAMP. The Latest Captured Timestamp Value Copy from timestamp counter TSCNT at the trigger event.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TSTAMP.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSTRGM (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of TSTRGM. Timestamp Counter Trigger Mode 00/11b - Trigger is ignored. 01b - Rising edge effective. Trigger level low will gating conversion results push to FIFO if TSRDM is 2'b10. Trigger level high will allow conversion results push to FIFO if TSRDM is 2'b10. 10b - Falling edge effective. Trigger level High will gating conversion results push to FIFO if TSRDM is 2'b10. Trigger level low will allow conversion results push to FIFO if TSRDM is 2'b10.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of TSTRGM.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSVLD0 (const SDADC_Type *const *pSdadc*)

Get the value of TSVLD0. Timestamp Valid Flag for Channel 0 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSVLD0.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSVLD1 (const SDADC_Type *const *pSdadc*)

Get the value of TSVLD1. Timestamp Valid Flag for Channel 1 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSVLD1.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSVLD2 (const SDADC_Type *const *pSdadc*)

Get the value of TSVLD2. Timestamp Valid Flag for Channel 2 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSVLD2.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSVLD_IE0 (const SDADC_Type *const *pSdadc*)

Get the value of TSVLD_IE0. Timestamp Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSVLD_IE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSVLD_IE1 (const SDADC_Type *const *pSdadc*)

Get the value of TSVLD_IE1. Timestamp Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSVLD_IE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetTSVLD_IE2 (const SDADC_Type *const *pSdadc*)

Get the value of TSVLD_IE2. Timestamp Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of TSVLD_IE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetVCM_BEN (const SDADC_Type *const *pSdadc*)

Get the value of VCM_BEN. Common Mode Voltage Bias Enable Enable common mode voltage for AC input of internal modulator analog. 0b - Common mode voltage Bias is disabled. 1b - Common mode voltage Bias is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of VCM_BEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetWLMT0 (const SDADC_Type *const *pSdadc*)

Get the value of WLMT0. Filter Channel 0 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of WLMT0.

LOCAL_INLINE uint32_t SDADC_HWA_GetWLMT1 (const SDADC_Type *const *pSdadc*)

Get the value of WLMT1. Filter Channel 1 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of WLMT1.

LOCAL_INLINE uint32_t SDADC_HWA_GetWLMT2 (const SDADC_Type *const *pSdadc*)

Get the value of WLMT2. Filter Channel 2 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of WLMT2.

LOCAL_INLINE uint32_t SDADC_HWA_GetWLMT_BKEN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of WLMT_BKEN. Break Signal Enabled for Window Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of WLMT_BKEN.

LOCAL_INLINE uint32_t SDADC_HWA_GetWLMTIE0 (const SDADC_Type *const *pSdadc*)

Get the value of WLMTIE0. Filter Channel 0 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of WLMTIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetWLMTIE1 (const SDADC_Type *const *pSdadc*)

Get the value of WLMTIE1. Filter Channel 1 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of WLMTIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetWLMTIE2 (const SDADC_Type *const *pSdadc*)

Get the value of WLMTIE2. Filter Channel 2 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of WLMTIE2.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCD0 (const SDADC_Type *const *pSdadc*)

Get the value of ZCD0. Filter Channel 0 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of ZCD0.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCD1 (const SDADC_Type *const *pSdadc*)

Get the value of ZCD1. Filter Channel 1 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of ZCD1.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCD2 (const SDADC_Type *const *pSdadc*)

Get the value of ZCD2. Filter Channel 2 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of ZCD2.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCD_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of ZCD_EN. Zero Cross Detect Enable 0b - Zero cross detection is disabled for the Main or Auxiliary Filter. 1b - Zero cross detection is enabled for the Main or Auxiliary Filter.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of ZCD_EN.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCD_SEL (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*)

Get the value of ZCD_SEL. Zero Cross Check Input Selection Select the input from Main Filter or Auxiliary Filter.
0b - Auxiliary Filter. 1b - Main Filter.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.

Returns

uint32_t the value of ZCD_SEL.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCDIE0 (const SDADC_Type *const *pSdadc*)

Get the value of ZCDIE0. Filter Channel 0 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of ZCDIE0.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCDIE1 (const SDADC_Type *const *pSdadc*)

Get the value of ZCDIE1. Filter Channel 1 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of ZCDIE1.

LOCAL_INLINE uint32_t SDADC_HWA_GetZCDIE2 (const SDADC_Type *const *pSdadc*)

Get the value of ZCDIE2. Filter Channel 2 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled.
1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

Returns

uint32_t the value of ZCDIE2.

LOCAL_INLINE void SDADC_HWA_SetABNISR0 (SDADC_Type *const *pSdadc*, uint32 *u32Value*)

Set the value of ABNISR0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u32Value</i>	Set Value.

LOCAL_INLINE void SDADC_HWA_SetABNISR1 (SDADC_Type *const *pSdadc*, uint32 *u32Value*)

Set the value of ABNISR1.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u32Value</i>	Set Value.

**LOCAL_INLINE void SDADC_HWA_SetBIAS (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*,
uint32 *u32Value*)**

Set the value of BIAS. Bias Value Bias offset for dedicated channel main filter; used for sign adjustment and compensation. It's a signed data. The main filter data after shift, will subtract this bias value.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>u8Channel</i>	SDADC Channel.
<i>u32Value</i>	Bias value.

Returns

uint32_t the value of BIAS.

LOCAL_INLINE void SDADC_HWA_SetBIAS_EN (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set BIAS_EN. Bias Analog Enable 0b - Bias analog is disabled. 1b - Bias analog is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value BIAS_EN value.

LOCAL_INLINE void SDADC_HWA_SetBIT_REV (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set BIT_REV. Bit Reverse Mode 0b - Normal mode, a triangle wave is used as PWM carrier. 1b - Bit reverse mode. A bit reversed sawtooth wave is used as PWM carrier.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value BIT_REV value.

LOCAL_INLINE void SDADC_HWA_SetBSTC (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set BSTC. Bias Start Time Control Bias analog need a ~2us start time, this bits control number of div_clk1 cycles before BIAS_ON flag set after BIAS_EN set.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value BSTC value.

LOCAL_INLINE void SDADC_HWA_SetCADIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set CADIE0. Filter Channel 0 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value CADIE0 value.

LOCAL_INLINE void SDADC_HWA_SetCADIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set CADIE1. Filter Channel 1 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value CADIE1 value.

LOCAL_INLINE void SDADC_HWA_SetCADIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set CADIE2. Filter Channel 2 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value CADIE2 value.

LOCAL_INLINE void SDADC_HWA_SetCBRn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set the value of the Channel Bias Register (CBR) for a specified channel.

This function sets the value of the Channel Bias Register (CBR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CBR value is to be set.
<i>u32Value</i>	the value to be set in the CBR register.

```
LOCAL_INLINE void SDADC_HWA_SetCCRN (const SDADC_Type *const  pSdadc, uint8_t  u8Channel,  
uint32_t  u32Value)
```

Set the value of the Channel Configuration Register (CCR) for a specified channel.

This function sets the value of the Channel Configuration Register (CCR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CCR value is to be set.
<i>u32Value</i>	the value to be set in the CCR register.

```
LOCAL_INLINE void SDADC_HWA_SetCDRn (const SDADC_Type *const  pSdadc, uint8_t  u8Channel,  
uint32_t  u32Value)
```

Set the value of the Channel Data Rate Register (CDR) for a specified channel.

This function sets the value of the Channel Data Rate Register (CDR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CDR value is to be set.
<i>u32Value</i>	the value to be set in the CDR register.

```
LOCAL_INLINE void SDADC_HWA_SetCFRn (const SDADC_Type *const  pSdadc, uint8_t  u8Channel,  
uint32_t  u32Value)
```

Set the value of the Channel Configuration Register (CFR) for a specified channel.

This function sets the value of the Channel Configuration Register (CFR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CFR value is to be set.
<i>u32Value</i>	the value to be set in the CFR register.

LOCAL_INLINE void SDADC_HWA_SetCH_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint8_t *u8Enable*)

Set the value of CH_EN. Channel Enable 0b - The channel is disabled. 0b - The channel is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>u8Enable</i>	SDADC Channel Channel Enable.

Returns

uint32_t the value of CH_EN.

LOCAL_INLINE void SDADC_HWA_SetCHLMTn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set the value of the Channel High Limit Register (CHLMT) for a specified channel.

This function sets the value of the Channel High Limit Register (CHLMT) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CHLMT value is to be set.
<i>u32Value</i>	the value to be set in the CHLMT register.

LOCAL_INLINE void SDADC_HWA_SetCLKO0_DIS (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set CLKO0_DIS. Modulator Clock Output 0 Disable 0b - CLKO0 is enabled when MEN is asserted. 1b - CLKO0 is disabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value CLKO0_DIS value.

LOCAL_INLINE void SDADC_HWA_SetCLKO1_DIS (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set CLKO1_DIS. Modulator Clock Output 1 Disable 0b - CLKO1 is enabled when MEN is asserted. 1b - CLKO1 is disabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value CLKO1_DIS value.

LOCAL_INLINE void SDADC_HWA_SetCLKO2_DIS (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set CLKO2_DIS. Modulator Clock Output 2 Disable 0b - CLKO2 is enabled when MEN is asserted. 1b - CLKO2 is disabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value CLKO2_DIS value.

LOCAL_INLINE void SDADC_HWA_SetCLKSEL (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set CLKSEL. PWM Generator Clock selection 00b - div_clk0 01b - div_clk1 10b - div_clk2 11b - off.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value CLKSEL value.

LOCAL_INLINE void SDADC_HWA_SetCLLMTn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set the value of the Channel Low Limit Register (CLLMT) for a specified channel.

This function sets the value of the Channel Low Limit Register (CLLMT) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CLLMT value is to be set.
<i>u32Value</i>	the value to be set in the CLLMT register.

LOCAL_INLINE void SDADC_HWA_SetCMENM (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set CMENM. Common Voltage Configuration of Negative Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value CMENM value.

LOCAL_INLINE void SDADC_HWA_SetCMENP (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set CMENP. Common Voltage Configuration of Positive Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value CMENP value.

LOCAL_INLINE void SDADC_HWA_SetCOCIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set COCIE0. Filter Channel 0 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value COCIE0 value.

LOCAL_INLINE void SDADC_HWA_SetCOCIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set COCIE1. Filter Channel 1 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value COCIE1 value.

LOCAL_INLINE void SDADC_HWA_SetCOCIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set COCIE2. Filter Channel 2 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value COCIE2 value.

LOCAL_INLINE void SDADC_HWA_SetCPRn (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set the value of the Channel Protection Register (CPR) for a specified channel.

This function sets the value of the Channel Protection Register (CPR) for a given channel in the SDADC instance.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	the channel number for which the CPR value is to be set.
<i>u32Value</i>	the value to be set in the CPR register.

LOCAL_INLINE void SDADC_HWA_SetCTRGSIGN_OEN (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint8_t *u8Enable*)

Set the value of CTRGSIGN_OEN. Channel Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once TSIGN falling edge is got.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>u8Enable</i>	Value.

LOCAL_INLINE void SDADC_HWA_SetDBGE (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set DBGE. Module Enable in Debug Mode Enables/disables the SDADC operation in Debug mode. 0b - SDADC is enabled in debug mode. 1b - SDADC is disabled in debug mode. Need trigger again for new conversion.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value DBGE value.

LOCAL_INLINE void SDADC_HWA_SetDBGM (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set DBGM. Software Debug Mode Request This bit is writable only when MEN is asserted. 0b - SDADC is in Normal Mode. 1b - SDADC is in Debug Mode.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value DBGM value.

LOCAL_INLINE void SDADC_HWA_SetDMA_EN (const SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint8_t *u8Enable*)

Set the value of DMA_EN. Channel Enable 0b - The channel dma is disabled. 0b - The channel dma is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>u8Enable</i>	SDADC Channel Channel Enable.

Returns

uint32_t the value of DMA_EN.

LOCAL_INLINE void SDADC_HWA_SetEXTIS (SDADC_Type *const *pSdadc*, uint32 *u32Value*)

Set the value of EXTIS.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u32Value</i>	Set Value.

LOCAL_INLINE void SDADC_HWA_SetFOVFIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FOVFIE0. Filter Channel 0 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FOVFIE0 value.

LOCAL_INLINE void SDADC_HWA_SetFOVFIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FOVFIE1. Filter Channel 1 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FOVFIE1 value.

LOCAL_INLINE void SDADC_HWA_SetFOVFIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FOVFIE2. Filter Channel 2 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FOVFIE2 value.

LOCAL_INLINE void SDADC_HWA_SetFRDYIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FRDYIE0. Filter Channel 0 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FRDYIE0 value.

LOCAL_INLINE void SDADC_HWA_SetFRDYIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FRDYIE1. Filter Channel 1 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FRDYIE1 value.

LOCAL_INLINE void SDADC_HWA_SetFRDYIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FRDYIE2. Filter Channel 2 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FRDYIE2 value.

LOCAL_INLINE void SDADC_HWA_SetFUNFIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FUNFIE0. Filter Channel 0 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>uint32_t</i>	u32Value FUNFIE0 value.
-----------------	-------------------------

LOCAL_INLINE void SDADC_HWA_SetFUNFIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FUNFIE1. Filter Channel 1 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FUNFIE1 value.

LOCAL_INLINE void SDADC_HWA_SetFUNFIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set FUNFIE2. Filter Channel 2 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value FUNFIE2 value.

LOCAL_INLINE void SDADC_HWA_SetGAINSEL (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set GAINSEL. Gain Select of Analog Input Path 00b - 1. 01b - 2. 10b - 4. 11b - 8.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value GAINSEL value.

LOCAL_INLINE void SDADC_HWA_SetHFVREF_EN (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set HFVREF_EN. Half VREFH output Enable to Related Internal Modulator Analog. Bit 4 enable internal Half VREFH output to modulator analog for channel 0. Bit 5 enable internal Half VREFH output to modulator analog for channel 1. Bit 6 enable internal Half VREFH output to modulator analog for channel 2. Bit 7 enable internal

Half VREFH output to modulator analog for channel 3. NOTE: Not all channels have internal modulator analog, if the channel has no internal modulator analog, related control bit is reserved.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value HFVREF_EN value.

LOCAL_INLINE void SDADC_HWA_SetHLMTIE0 (SDADC_Type *const *pSdac*, uint32_t *u32Value*)

Set HLMTIE0. Filter Channel 0 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value HLMTIE0 value.

LOCAL_INLINE void SDADC_HWA_SetHLMTIE1 (SDADC_Type *const *pSdac*, uint32_t *u32Value*)

Set HLMTIE1. Filter Channel 1 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value HLMTIE1 value.

LOCAL_INLINE void SDADC_HWA_SetHLMTIE2 (SDADC_Type *const *pSdac*, uint32_t *u32Value*)

Set HLMTIE2. Filter Channel 2 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value HLMTIE2 value.

LOCAL_INLINE void SDADC_HWA_SetINCFGM (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set INCFGM. Configuration of Negative Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value INCFGM value.

LOCAL_INLINE void SDADC_HWA_SetINCFGP (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set INCFGP. Configuration of Positive Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value INCFGP value.

LOCAL_INLINE void SDADC_HWA_SetINMOD (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set INMOD. Input MUX Control Mode 00b - Software control mode (INMUX follows INSEL). 01b - Preset mode (load INMUX upon a channel conversion trigger). 10b - Scan mode (decrement INMUX upon a channel conversion trigger, wrap around to INSEL). 11b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value INMOD value.

LOCAL_INLINE void SDADC_HWA_SetINSEL (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set INSEL. Input Pin Selection Defines the initial or permanent setting for the input MUX(bits INMUX) depending on the selected operating mode (bits INMOD).

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value INSEL value.

LOCAL_INLINE void SDADC_HWA_SetINTG_STRIG0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set INTG_STRIG0. Integration Software Trigger for channel 0 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value INTG_STRIG0 value.

LOCAL_INLINE void SDADC_HWA_SetINTG_STRIG1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set INTG_STRIG1. Integration Software Trigger for channel 1 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value INTG_STRIG1 value.

LOCAL_INLINE void SDADC_HWA_SetINTG_STRIG2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set INTG_STRIG2. Integration Software Trigger for channel 2 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value INTG_STRIG2 value.

LOCAL_INLINE void SDADC_HWA_SetLLMTIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set LLMTIE0. Filter Channel 0 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value LLMTIE0 value.

LOCAL_INLINE void SDADC_HWA_SetLLMTIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set LLMTIE1. Filter Channel 1 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value LLMTIE1 value.

LOCAL_INLINE void SDADC_HWA_SetLLMTIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set LLMTIE2. Filter Channel 2 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value LLMTIE2 value.

LOCAL_INLINE void SDADC_HWA_SetMACLKSEL (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set MACLKSEL. Internal Modulator Analog Clock Selection 00b - Div_clk1 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 01b - Div_clk0 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 1*b - Reserved.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value MACLKSEL value.

LOCAL_INLINE void SDADC_HWA_SetMASEL (SDADC_Type *const *pSdadc*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set MASEL. Modulator Analog Selection 0b - External modulator analog is selected as channel input. 1b - Internal modulator analog is selected as channel input.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value MASEL value.

LOCAL_INLINE void SDADC_HWA_SetMCLK_DIV (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set MCLK_DIV. DIV Divider Value The (MCLK_DIV+1) sets the clock divider ratio for DIV block. The minimum value MCLK_DIV is 1.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value MCLK_DIV value.

LOCAL_INLINE void SDADC_HWA_SetMEN (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set MEN. Main Enable Setting this bit enables all Function block simultaneously if CHEN and corresponding enable bit are asserted. 0b - All function blocks are disabled. 1b - All function blocks are enabled simultaneously.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value MEN value.

LOCAL_INLINE void SDADC_HWA_SetMENSEL (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set MENSEL. Main Enable Selection Setting this bit to select main enable(MEN) from an enable from external of SDADC IP. By this way, all SDADC in a system can share one main enable and can be enable at the same time.
0b - Use internal MEN. 1b - Use external MEN.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value MENSEL value.

LOCAL_INLINE void SDADC_HWA_SetNISR (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set the value of NISR.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u32Value</i>	Set Value.

LOCAL_INLINE void SDADC_HWA_SetPATTERN (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set PATTERN. PWM Generator Pattern Config this before set MEN. Clear the bits during PWM generator run will terminate the PWM generator outputs after completion of the current period(indicated by bit ON=0) 00b - Stopped 01b - Square wave 10b - Triangle 11b - Sine wave.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value PATTERN value.

LOCAL_INLINE void SDADC_HWA_SetPOL (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set POL. PWM Generator Polarity 0b - PWM generator output begin with +1 1b - PWM generator output begin with -1.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value POL value.

LOCAL_INLINE void SDADC_HWA_SetPRESCALE (SDADC_Type *const *pSdac*, uint32_t *u32Value*)

Set PRESCALE. PRE_DIV Divider Value 00b - No prescale. 01b - Divided by 2. 10b - Divided by 4. 11b - Divided by 8.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value PRESCALE value.

LOCAL_INLINE void SDADC_HWA_SetRESET (SDADC_Type *const *pSdac*, uint32_t *u32Value*)

Set RESET. Software Reset Used to reset all function blocks (filters, short circuit detect, etc) and all flags or status. Note, to reset all function domain, RESET must keep high at least 3 div_clk1 cycles. 0b - No reset. 1b - All function blocks are reset and disabled.

Parameters

<i>pSdac</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value RESET value.

LOCAL_INLINE void SDADC_HWA_SetRSTSEL (SDADC_Type *const *pSdac*, uint8_t *u8Channel*, uint32_t *u32Value*)

Set RSTSEL. Modulator Analog Reset Cycle Selection Select RSTSEL+1 mclk cycles for modulator analog reset after a channel conversion trigger. This is needed especially when the input mux(INMUX) is changed. 0d - Modulator analog is not reset after a channel conversion trigger 1d - Modulator analog is reset for 2 mclk cycles after a channel conversion trigger. Total 7 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 2d - Modulator analog is reset for 3 mclk cycles after a channel conversion trigger. Total 8 mclk cycle cycles needed for filters start to run after a channel conversion trigger. ... 8d - Modulator analog is

reset for 9 mclk cycles after a channel conversion trigger. Total 14 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 9-15d - Modulator analog is reset for 10 mclk cycles after a channel conversion trigger. Total 15 mclk cycle cycles needed for filters start to run after a channel conversion trigger.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>u8Channel</i>	SDADC Channel.
<i>uint32_t</i>	u32Value RSTSEL value.

LOCAL_INLINE void SDADC_HWA_SetSATIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SATIE0. Filter Channel 0 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SATIE0 value.

LOCAL_INLINE void SDADC_HWA_SetSATIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SATIE1. Filter Channel 1 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SATIE1 value.

LOCAL_INLINE void SDADC_HWA_SetSATIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SATIE2. Filter Channel 2 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
---------------	---

<i>uint32_t</i>	u32Value SATIE2 value.
-----------------	------------------------

LOCAL_INLINE void SDADC_HWA_SetSCDIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SCDIE0. Filter Channel 0 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SCDIE0 value.

LOCAL_INLINE void SDADC_HWA_SetSCDIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SCDIE1. Filter Channel 1 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SCDIE1 value.

LOCAL_INLINE void SDADC_HWA_SetSCDIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SCDIE2. Filter Channel 2 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SCDIE2 value.

LOCAL_INLINE void SDADC_HWA_SetSD_CAPVLD_IE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SD_CAPVLD_IE0. Capture Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SD_CAPVLD_IE0 value.

LOCAL_INLINE void SDADC_HWA_SetSD_CAPVLD_IE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SD_CAPVLD_IE1. Capture Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SD_CAPVLD_IE1 value.

LOCAL_INLINE void SDADC_HWA_SetSD_CAPVLD_IE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set SD_CAPVLD_IE2. Capture Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value SD_CAPVLD_IE2 value.

LOCAL_INLINE void SDADC_HWA_SetSTRIG0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set STRIG0. Software Trigger for Filter Channel 0.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value STRIG0 value.

LOCAL_INLINE void SDADC_HWA_SetSTRIG1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set STRIG1. Software Trigger for Filter Channel 1.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value STRIG1 value.

LOCAL_INLINE void SDADC_HWA_SetSTRIG2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set STRIG2. Software Trigger for Filter Channel 2.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value STRIG2 value.

LOCAL_INLINE void SDADC_HWA_SetTRGSIGN_OEN (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set TRGSIGN_OEN. Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once SIGN falling edge is got.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value TRGSIGN_OEN value.

LOCAL_INLINE void SDADC_HWA_SetTSIGN_P2N_IE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set TSIGN_P2N_IE0. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value TSIGN_P2N_IE0 value.

LOCAL_INLINE void SDADC_HWA_SetTSIGN_P2N_IE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set TSIGN_P2N_IE1. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value TSIGN_P2N_IE1 value.

LOCAL_INLINE void SDADC_HWA_SetTSIGN_P2N_IE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set TSIGN_P2N_IE2. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value TSIGN_P2N_IE2 value.

LOCAL_INLINE void SDADC_HWA_SetTSVLD_IE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set TSVLD_IE0. Timestamp Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value TSVLD_IE0 value.

LOCAL_INLINE void SDADC_HWA_SetTSVLD_IE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set TSVLD_IE1. Timestamp Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value TSVLD_IE1 value.

LOCAL_INLINE void SDADC_HWA_SetTSVLD_IE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set TSVLD_IE2. Timestamp Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value TSVLD_IE2 value.

LOCAL_INLINE void SDADC_HWA_SetVCM_BEN (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set VCM_BEN. Common Mode Voltage Bias Enable Enable common mode voltage for AC input of internal modulator analog. 0b - Common mode voltage Bias is disabled. 1b - Common mode voltage Bias is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value VCM_BEN value.

LOCAL_INLINE void SDADC_HWA_SetWLMTIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set WLMTIE0. Filter Channel 0 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value WLMTIE0 value.

LOCAL_INLINE void SDADC_HWA_SetWLMTIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set WLMTIE1. Filter Channel 1 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value WLMTIE1 value.

LOCAL_INLINE void SDADC_HWA_SetWLMTIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set WLMTIE2. Filter Channel 2 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value WLMTIE2 value.

LOCAL_INLINE void SDADC_HWA_SetZCDIE0 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set ZCDIE0. Filter Channel 0 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value ZCDIE0 value.

LOCAL_INLINE void SDADC_HWA_SetZCDIE1 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set ZCDIE1. Filter Channel 1 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value ZCDIE1 value.

LOCAL_INLINE void SDADC_HWA_SetZCDIE2 (SDADC_Type *const *pSdadc*, uint32_t *u32Value*)

Set ZCDIE2. Filter Channel 2 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

Parameters

<i>pSdadc</i>	the base address of the SDADC instance.
<i>uint32_t</i>	u32Value ZCDIE2 value.

HwA_sddf

Collaboration diagram for HwA_sddf:Enumerations

```
enum SDDF_PreDivType { SDDF_PREDIV_1 = 0U, SDDF_PREDIV_2 = 1U, SDDF_PREDIV_4 = 2U,  
SDDF_PREDIV_8 = 3U }
```

Select the SDDF function clock pre-divider ratio.

```
enum SDDF_ModeType { SDDF_MODE_NORMAL = 0U, SDDF_MODE_DEBUG = 1U }
```

Select the SDDF operation mode.

```
enum SDDF_DebugModuleEnableType { SDDF_MODULE_ENABLE_IN_DEBUG = 0U,  
SDDF_MODULE_DISABLE_IN_DEBUG = 1U }
```

Select whether the SDDF module enables in debug mode.

```
enum SDDF_InputSrcType { SDDF_INPUT_SRC_MAIN_FILTER = 0U,  
SDDF_INPUT_SRC_AUXILIARY_FILTER = 1U }
```

Select the data source for limit check and zero cross check.

```
enum SDDF_AuxFilterConvModeType { SDDF_AUX_FILTER_CONV_ALWAYS_ON = 0U,  
SDDF_AUX_FILTER_CONV_FOLLOW_MAIN = 1U }
```

Select the conversion mode for auxiliary filter.

```
enum SDDF_AuxFilterOrderType { SDDF_AUX_FILTER_FIRST_ORDER = 1U,  
SDDF_AUX_FILTER_SECOND_ORDER = 2U, SDDF_AUX_FILTER_THIRD_ORDER = 3U }
```

Selects the filter order for auxiliary filter.

```
enum SDDF_MainFilterConvModeType { SDDF_MAIN_FILTER_CONV_SINGLE = 0U,  
SDDF_MAIN_FILTER_CONV_CONTINUOUS = 1U, SDDF_MAIN_FILTER_CONV_ALWAYS = 2U }
```

Selects the conversion mode for main filter.

```
enum SDDF_MainFilterOrderType { SDDF_MAIN_FILTER_FIRST_ORDER = 1U,  
SDDF_MAIN_FILTER_SECOND_ORDER = 2U, SDDF_MAIN_FILTER_THIRD_ORDER = 3U,  
SDDF_MAIN_FILTER_FOURTH_ORDER = 4U, SDDF_MAIN_FILTER_FIFTH_ORDER = 5U }
```

Selects the filter order for main filter.

```
enum SDDF_TriggerEventType { SDDF_TRIGGER_EVENT_EDGE = 0U, SDDF_TRIGGER_EVENT_LEVEL =  
1U }
```

Select the trigger level type.

```
enum SDDF_TriggerSrcType { SDDF_TRIGGER_SRC_SOFTWARE = 0U, SDDF_TRIGGER_SRC_HARDWARE  
= 1U }
```

Select the trigger source.

enum **SDDF_ClockEdgeType** { **SDDF_CLOCK_EDGE_RISING** = 1U, **SDDF_CLOCK_EDGE_FALLING** = 2U, **SDDF_CLOCK_EDGE_BOTH** = 3U }

Select the input clock edge.

enum **SDDF_ClockSrcType** { **SDDF_CLOCK_SRC_EXTERNAL_MODULATOR_CLOCK** = 3U }

Select the input clock source.

enum **SDDF_BitFormatType** { **SDDF_BIT_FORMAT_EXTERNAL_ONE_BIT_STREAM** = 0U, **SDDF_BIT_FORMAT_EXTERNAL_MANCHSTER_CODE** = 1U }

Select the input bit format.

enum **SDDF_ResultFormatType** { **SDDF_RESULT_DATA_LEFT_ALIGNED_SIGNED** = 0U, **SDDF_RESULT_DATA_LEFT_ALIGNED_UNSIGNED** = 1U }

Select the result data format.

enum **SDDF_MainFilterShiftType** { **SDDF_MAIN_FILTER_NOT_SHIFT** = 0U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_1** = 1U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_2** = 2U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_3** = 3U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_4** = 4U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_5** = 5U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_6** = 6U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_7** = 7U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_8** = 8U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_9** = 9U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_10** = 10U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_11** = 11U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_12** = 12U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_13** = 13U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_14** = 14U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_15** = 15U, **SDDF_MAIN_FILTER_SHIFT_LEFT_1** = 17U, **SDDF_MAIN_FILTER_SHIFT_LEFT_2** = 18U, **SDDF_MAIN_FILTER_SHIFT_LEFT_3** = 19U, **SDDF_MAIN_FILTER_SHIFT_LEFT_4** = 20U, **SDDF_MAIN_FILTER_SHIFT_LEFT_5** = 21U, **SDDF_MAIN_FILTER_SHIFT_LEFT_6** = 22U, **SDDF_MAIN_FILTER_SHIFT_LEFT_7** = 23U, **SDDF_MAIN_FILTER_SHIFT_LEFT_8** = 24U, **SDDF_MAIN_FILTER_SHIFT_LEFT_9** = 25U, **SDDF_MAIN_FILTER_SHIFT_LEFT_10** = 26U, **SDDF_MAIN_FILTER_SHIFT_LEFT_11** = 27U, **SDDF_MAIN_FILTER_SHIFT_LEFT_12** = 28U, **SDDF_MAIN_FILTER_SHIFT_LEFT_13** = 29U, **SDDF_MAIN_FILTER_SHIFT_LEFT_14** = 30U, **SDDF_MAIN_FILTER_SHIFT_LEFT_15** = 31U }

Set the main filter shift to get correct 24 bit precision.

enum **SDDF_ChannelLimitOptionType** { **SDDF_CHANNEL_LIMIT_BOTH_HITH_LOW_THRES** = 0U, **SDDF_CHANNEL_LIMIT_HITH_THRES** = 1U, **SDDF_CHANNEL_LIMIT_LOW_THRES** = 2U, **SDDF_CHANNEL_LIMIT_WINDOW** = 3U }

Select the limit detection option.

enum **SDDF_ShortCircuitOptionType** { **SDDF_SHORT_CIRCUIT_COUNT_ON_BOTH** = 0U, **SDDF_SHORT_CIRCUIT_COUNT_ON_1** = 1U, **SDDF_SHORT_CIRCUIT_COUNT_ON_0** = 2U }

Select the short circuit detection option.

enum **SDDF_ShortCircuitDetectModeType** { **SDDF_SHORT_DETECT_ALWAYS_ON** = 0U, **SDDF_SHORT_DETECT_AUX_FILTER_EN** = 1U }

Select sht short circuit detect conversion mode.

Functions

LOCAL_INLINE uint32_t **SDDF_HWA_GetCtrl** (const SDDF_Type *const pSddf)

Get the global control register (SDDF_CTRL) value.

LOCAL_INLINE void **SDDF_HWA_SetCtrl** (SDDF_Type *const pSddf, uint32_t u32Ctrl)

Set the global control register (SDDF_CTRL) value.

LOCAL_INLINE bool **SDDF_HWA_GetClockOut2DisableFlag** (const SDDF_Type *const pSddf)

Get whether SDDF clock output 2 is disabled.

LOCAL_INLINE void **SDDF_HWA_SetClockOut2DisableFlag** (SDDF_Type *const pSddf, bool bDisable)

Set whether to disable clock output 2.

LOCAL_INLINE bool **SDDF_HWA_GetClockOut1DisableFlag** (const SDDF_Type *const pSddf)

Get whether SDDF clock output 1 is disabled.

LOCAL_INLINE void **SDDF_HWA_SetClockOut1DisableFlag** (SDDF_Type *const pSddf, bool bDisable)

Set whether to disable clock output 1.

LOCAL_INLINE bool **SDDF_HWA_GetClockOut0DisableFlag** (const SDDF_Type *const pSddf)

Get whether SDDF clock output 0 is disabled.

LOCAL_INLINE void **SDDF_HWA_SetClockOut0DisableFlag** (SDDF_Type *const pSddf, bool bDisable)

Set whether to disable clock output 0.

LOCAL_INLINE **SDDF_PreDivType** **SDDF_HWA_GetPreDivider** (const SDDF_Type *const pSddf)

Get the function clock pre-divider value.

LOCAL_INLINE void **SDDF_HWA_SetPreDivider** (SDDF_Type *const pSddf, **SDDF_PreDivType** eDiv)

Set the function clock pre-divider value.

LOCAL_INLINE uint32_t **SDDF_HWA_GetDivider** (const SDDF_Type *const pSddf)

Get the clock divider value.

LOCAL_INLINE void **SDDF_HWA_SetDivider** (SDDF_Type *const pSddf, uint32_t u32Div)

Set the clock divider value.

LOCAL_INLINE bool **SDDF_HWA_GetMainEnableFlag** (const SDDF_Type *const pSddf)

Get whether the SDDF function blocks are enabled.

LOCAL_INLINE void **SDDF_HWA_MainEnable** (SDDF_Type *const pSddf)

Enable the SDDF function blocks.

LOCAL_INLINE void **SDDF_HWA_MainDisable** (SDDF_Type *const pSddf)

Disable all SDDF function blocks.

LOCAL_INLINE void **SDDF_HWA_Reset** (SDDF_Type *const pSddf)

Reset all SDDF function blocks and clock configuration bits.

LOCAL_INLINE **SDDF_ModeType** **SDDF_HWA_GetOperationMode** (const SDDF_Type *const pSddf)

Get the SDDF operation mode (normal mode or debug mode)

LOCAL_INLINE void **SDDF_HWA_SetOperationMode** (SDDF_Type *const pSddf, **SDDF_ModeType** eMode)

Set the SDDF operation mode.

LOCAL_INLINE **SDDF_DebugModuleEnableType** **SDDF_HWA_GetModuleDebugEnableFlag** (const SDDF_Type *const pSddf)

Get whether SDDF module is enabled in debug mode.

LOCAL_INLINE void **SDDF_HWA_SetModuleDebugEnableFlag** (SDDF_Type *const pSddf, **SDDF_DebugModuleEnableType** eEnable)

Set whether SDDF module is enabled in debug mode.

LOCAL_INLINE void **SDDF_HWA_GenerateChannelSwTrigger** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Generate software trigger signal for the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetNormalInterruptEnable** (const SDDF_Type *const pSddf)

Get the normal interrupt (SDDF_NIER) register value.

LOCAL_INLINE void **SDDF_HWA_SetNormalInterruptEnable** (SDDF_Type *const pSddf, uint32_t u32IntEn)

Set the normal interrupt (SDDF_NIER) register value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelZeroCrossDetectIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether zero cross detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelZeroCrossDetectIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable zero cross detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFODataReadyIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel FIFO data ready interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFODataReadyIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel FIFO data ready interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelConvCompleteIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel conversion complete interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelConvCompleteIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel conversion complete interrupt.

LOCAL_INLINE uint32_t **SDDF_HWA_GetAbnormalInterruptEnable0** (const SDDF_Type *const pSddf)

Get the channel abnormal interrupt 0 (SDDF_ABNIER0) register value.

LOCAL_INLINE void **SDDF_HWA_SetAbnormalInterruptEnable0** (SDDF_Type *const pSddf, uint32_t u32IntEn)

Set the channel abnormal interrupt 0 (SDDF_ABNIER0) register value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelHighLimitIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel high limit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelHighLimitIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel high limit detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelLowLimitIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel low limit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelLowLimitIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel low limit detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelWindowLimitIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel window limit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelWindowLimitIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel window limit detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelShortCircuitDetectIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel short circuit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelShortCircuitDetectIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel short circuit detect interrupt.

LOCAL_INLINE uint32_t **SDDF_HWA_GetAbnormalInterruptEnable1** (const SDDF_Type *const pSddf)
Get the channel abnormal interrupt 1 (SDDF_ABNIER1) register value.

LOCAL_INLINE void **SDDF_HWA_SetAbnormalInterruptEnable1** (SDDF_Type *const pSddf, uint32_t
u32IntEn)
Set the channel abnormal interrupt 1 (SDDF_ABNIER1) register value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelSaturationIntEnableFlag** (const SDDF_Type *const pSddf,
uint8_t u8ChanIdx)
Get whether channel result saturation interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelSaturationIntEnableFlag** (SDDF_Type *const pSddf, uint8_t
u8ChanIdx, bool bEnable)
Set whether to enable channel result saturation interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelClockAbsenceIntEnableFlag** (const SDDF_Type *const pSddf,
uint8_t u8ChanIdx)
Get whether channel clock absence detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelClockAbsenceIntEnableFlag** (SDDF_Type *const pSddf,
uint8_t u8ChanIdx, bool bEnable)
Set whether to enable channel clock absence detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOOverflowIntEnableFlag** (const SDDF_Type *const pSddf,
uint8_t u8ChanIdx)
Get whether channel FIFO overflow interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFOOverflowIntEnableFlag** (SDDF_Type *const pSddf,
uint8_t u8ChanIdx, bool bEnable)
Set whether to enable channel FIFO overflow interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOUnderflowIntEnableFlag** (const SDDF_Type *const
pSddf, uint8_t u8ChanIdx)

Get whether channel FIFO underflow interrupt is enabled.

```
LOCAL_INLINE void SDDF_HWA_SetChannelFIFOUnderflowIntEnableFlag (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable channel FIFO underflow interrupt.

```
LOCAL_INLINE uint32_t SDDF_HWA_GetNormalInterruptStatus (const SDDF_Type *const pSddf)
```

Get the normal interrupt status register (SDDF_NISR) value.

```
LOCAL_INLINE void SDDF_HWA_ClearNormalInterruptStatus (SDDF_Type *const pSddf)
```

Clear the normal interrupt status register (SDDF_NISR) value.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelZeroCrossDetectStatus (const SDDF_Type *const pSddf,  
uint8_t u8ChanIdx)
```

Get zero cross detect status of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_ClearChannelZeroCrossDetectStatus (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Clear zero cross detect flag of the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelFIFODataReadyStatus (const SDDF_Type *const pSddf,  
uint8_t u8ChanIdx)
```

Get FIFO data ready status of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_ClearChannelFIFODataReadyStatus (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Clear FIFO data ready flag of the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelConvCompleteStatus (const SDDF_Type *const pSddf,  
uint8_t u8ChanIdx)
```

Get channel conversion complete status of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_ClearChannelConvCompleteStatus (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```


Clear channel conversion complete flag of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetAbnormalInterruptStatus0** (const SDDF_Type *const pSddf)

Get the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

LOCAL_INLINE void **SDDF_HWA_ClearAbnormalInterruptStatus0** (SDDF_Type *const pSddf)

Clear the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelHighLimitStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get out of high limit range status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelHighLimitStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear out of high limit range flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelLowLimitStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get out of low limit range status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelLowLimitStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear out of low limit range flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelWindowLimitStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get out of window limit range status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelWindowLimitStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear out of window limit range flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelShortCircuitDetectStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get short circuit detect status of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_ClearChannelShortCircuitDetectStatus (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx)
```

Clear short circuit flag of the selected channel.

```
LOCAL_INLINE uint32_t SDDF_HWA_GetAbnormalInterruptStatus1 (const SDDF_Type *const pSddf)
```

Get the abnormal interrupt status 1 register (SDDF_ABNISR1) value.

```
LOCAL_INLINE void SDDF_HWA_ClearAbnormalInterruptStatus1 (SDDF_Type *const pSddf)
```

Clear the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelSaturationStatus (const SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Get result saturation status of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_ClearChannelSaturationStatus (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Clear result saturation flag of the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelClockAbsenceStatus (const SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Get clock absence detect status of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_ClearChannelClockAbsenceStatus (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Clear clock absence flag of the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelFIFOOverflowStatus (const SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Get FIFO overflow status of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_ClearChannelFIFOOverflowStatus (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Clear FIFO overflow flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOUnderflowStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get FIFO underflow status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelFIFOUnderflowStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear FIFO underflow flag of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetMainStatus** (const SDDF_Type *const pSddf)

Get main status of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetClockOutput2ReadyFlag** (const SDDF_Type *const pSddf)

Get whether the modulator clock 2 output is ready.

LOCAL_INLINE bool **SDDF_HWA_GetClockOutput1ReadyFlag** (const SDDF_Type *const pSddf)

Get whether the modulator clock 1 output is ready.

LOCAL_INLINE bool **SDDF_HWA_GetClockOutput0ReadyFlag** (const SDDF_Type *const pSddf)

Get whether the modulator clock 0 output is ready.

LOCAL_INLINE bool **SDDF_HWA_GetChannelConvOngoingStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether conversion of the selected channel is ongoing.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelControl** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get channel control register (SDDF_CCRn) value of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelControl** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32Ctrl)

Set channel control register (SDDF_CCRn) value of the selected channel.

LOCAL_INLINE **SDDF_InputSrcType** **SDDF_HWA_GetChannelLimitCheckInputSrc** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the result input source of channel limit check.

LOCAL_INLINE void **SDDF_HWA_SetChannelLimitCheckInputSrc** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_InputSrcType** eSrc)

Set the result input source of the channel limit check.

LOCAL_INLINE **SDDF_InputSrcType** **SDDF_HWA_GetChannelZeroCrossCheckInputSrc** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the result input source of channel zero cross check.

LOCAL_INLINE void **SDDF_HWA_SetChannelZeroCrossCheckInputSrc** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_InputSrcType** eSrc)

Set the result input source of the channel zero cross check.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether FIFO is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFOEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable FIFO for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelLimitCheckEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether limit check is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelLimitCheckEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable limit check for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelZeroCrossDetectEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether zero cross detect is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelZeroCrossDetectEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable zero cross detect for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelClockAbsenceDetectEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether clock absence detect is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelClockAbsenceDetectEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable clock absence detect for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelShortCircuitDetectEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether short circuit detect is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelShortCircuitDetectEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable short circuit detect for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelDMAEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether DMA is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelDMAEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable DMA for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelAuxiliaryFilterEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether auxiliary filter is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelAuxiliaryFilterEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable auxiliary filter for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelMainFilterEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether main filter is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable main filter for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether the selected channel is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelDataRate** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the data rate register (SDDF_CDRn) value of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelDataRate** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32DataRate)

Set the data rate register (SDDF_CDRn) value of the selected channel.

LOCAL_INLINE **SDDF_AuxFilterConvModeType** **SDDF_HWA_GetChannelAuxiliaryFilterConvMode** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the auxiliary filter conversion mode of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelAuxiliaryFilterConvMode** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_AuxFilterConvModeType** eConvMode)

Set the auxiliary filter conversion mode of the selected channel.

LOCAL_INLINE **SDDF_AuxFilterOrderType SDDF_HWA_GetChannelAuxiliaryFilterOrder** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the auxiliary filter order of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelAuxiliaryFilterOrder** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_AuxFilterOrderType** eOrder)

Set the auxiliary filter order of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelAuxiliaryFilterOversampleRate** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the auxiliary filter oversampling rate of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelAuxiliaryFilterOversampleRate** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32SampleRate)

Set the auxiliary filter oversampling rate of the selected channel.

LOCAL_INLINE **SDDF_MainFilterConvModeType SDDF_HWA_GetChannelMainFilterConvMode** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the main filter conversion mode of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterConvMode** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_MainFilterConvModeType** eConvMode)

Set the main filter conversion mode of the selected channel.

LOCAL_INLINE **SDDF_MainFilterOrderType SDDF_HWA_GetChannelMainFilterOrder** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the main filter order of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterOrder** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_MainFilterOrderType** eOrder)

Set the main filter order of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelMainFilterOversampleRate** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the main filter oversampling rate of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterOversampleRate** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32SampleRate)

Set the main filter oversampling rate of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelConfig** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the configuration register (SDDF_CFRn) value of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelConfig** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32Config)

Set the configuration register (SDDF_CFRn) value of the selected channel.

LOCAL_INLINE SDDF_TriggerEventType **SDDF_HWA_GetChannelTriggerEvent** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the trigger level type of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelTriggerEvent** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, SDDF_TriggerEventType eEvent)

Set the trigger level type of the selected channel.

LOCAL_INLINE SDDF_TriggerSrcType **SDDF_HWA_GetChannelTriggerSource** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the trigger source type of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelTriggerSource** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, SDDF_TriggerSrcType eSrc)

Set the trigger source type of the selected channel.

LOCAL_INLINE SDDF_ClockEdgeType **SDDF_HWA_GetChannelInputClockEdge** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the clock edge for conversion of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelInputClockEdge** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ClockEdgeType** eClkEdge)

Set the clock edge for conversion of the selected channel.

LOCAL_INLINE **SDDF_ClockSrcType** **SDDF_HWA_GetChannelInputClockSource** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the input clock select of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelInputClockSource** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ClockSrcType** eSrc)

Set the input clock select of the selected channel.

LOCAL_INLINE **SDDF_BitFormatType** **SDDF_HWA_GetChannelInputBitFormat** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the input bit format of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelInputBitFormat** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_BitFormatType** eFormat)

Set the input bit format of the selected channel.

LOCAL_INLINE uint8_t **SDDF_HWA_GetChannelFIFOWatermark** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the FIFO watermark of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFOWatermark** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint8_t u8Watermark)

Set the FIFO watermark of the selected channel.

LOCAL_INLINE **SDDF_ResultFormatType** **SDDF_HWA_GetChannelResultDataFormat** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the result data format of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelResultDataFormat** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ResultFormatType** eFormat)

Set the result data format of the selected channel.

LOCAL_INLINE **SDDF_MainFilterShiftType** **SDDF_HWA_GetChannelMainFilterShift** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the main filter shift of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterShift** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_MainFilterShiftType** eShift)

Set the main filter shift to get correct 24 bit precision of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelProtectionConfig** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the protection settings (SDDF_CPRn) value of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelProtectionConfig** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32Config)

Set the protection settings (SDDF_CPRn) value of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelHighLimitBreakSignalEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether the high limit break signal is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelHighLimitBreakSignalEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable high limit break signal for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelWindowLimitBreakSignalEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether the window limit break signal is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelWindowLimitBreakSignalEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable window limit break signal for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelLowLimitBreakSignalEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether the low limit break signal is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelLowLimitBreakSignalEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable low limit break signal for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelShortCircuitBreakSignalEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether short circuit detect break signal is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelShortCircuitBreakSignalEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable short circuit detect break signal for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelClockAbsenceBreakSignalEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether clock absence detect break signal is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelClockAbsenceBreakSignalEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable clock absence detect break signal for the selected channel.

LOCAL_INLINE uint8_t **SDDF_HWA_GetChannelClockAbsenceLimitThreshold** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the clock absence detector limitation threshold.

LOCAL_INLINE void **SDDF_HWA_SetChannelClockAbsenceLimitThreshold** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint8_t u8Thres)

Set the clock absence detector limitation threshold.

LOCAL_INLINE **SDDF_ChannelLimitOptionType SDDF_HWA_GetChannelLimitOption** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the channel limit check option.

LOCAL_INLINE void **SDDF_HWA_SetChannelLimitOption** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ChannelLimitOptionType** eOption)

Set the channel limit check option.

LOCAL_INLINE **SDDF_ShortCircuitOptionType SDDF_HWA_GetChannelShortCircuitOption** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the channel short circuit detect option.

LOCAL_INLINE void **SDDF_HWA_SetChannelShortCircuitOption** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ShortCircuitOptionType** eOption)

Set the channel short circuit detect option.

LOCAL_INLINE **SDDF_ShortCircuitDetectModeType SDDF_HWA_GetShortCircuitDetectMode** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the channel short circuit detect conversion mode.

LOCAL_INLINE void **SDDF_HWA_SetShortCircuitDetectMode** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ShortCircuitDetectModeType** eMode)

Set the channel short circuit detect conversion mode.

LOCAL_INLINE uint8_t **SDDF_HWA_GetShortCircuitLimitThreshold** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the threshold value for short circuit detector counter of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetShortCircuitLimitThreshold** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint8_t u8Thres)

Set the threshold value for short circuit detector counter of the selected channel.

LOCAL_INLINE int32_t **SDDF_HWA_GetChannelBias** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the bias value for the selected channel main filter.

LOCAL_INLINE void **SDDF_HWA_SetChannelBias** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, int32_t s32Bias)

Set the bias value for the selected channel main filter.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelLowLimit** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the low limit threshold for limit check of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelLowLimit** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32LowLimit)

Set the low limit threshold for limit check of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelHighLimit** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the high limit threshold for limit check of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelHighLimit** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32HighLimit)

Set the high limit threshold for limit check of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelData** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the result data of the selected channel.

Detailed Description

Enumeration Type Documentation

enum SDDF_AuxFilterConvModeType

Select the conversion mode for auxiliary filter.

Enumerator:

SDDF_AUX_FILTER_CONV_ALWAYS_ON	Auxiliary filter is always on
SDDF_AUX_FILTER_CONV_FOLLOW_MAIN_FILTER	Auxiliary filter operates following the main filter

enum SDDF_AuxFilterOrderType

Selects the filter order for auxiliary filter.

Enumerator:

SDDF_AUX_FILTER_FIRST_ORDER	The auxiliary filter is 1st order
SDDF_AUX_FILTER_SECOND_ORDER	The auxiliary filter is 2nd order
SDDF_AUX_FILTER_THIRD_ORDER	The auxiliary filter is 3rd order

enum SDDF_BitFormatType

Select the input bit format.

Enumerator:

SDDF_BIT_FORMAT_EXTERNAL	External one bit stream
--------------------------	-------------------------

_ONE_BIT_STRE AM	
SDDF_BIT_FOR MAT_EXTERNAL _MANCHSTER_C ODE	External Manchester code, SDDF_ClockEdgeType will determine rise or fall decoder

enum SDDF_ChannelLimitOptionType

Select the limit detection option.

Enumerator:

SDDF_CHANNEL _LIMIT_BOTH_H ITH_LOW_THRE S	Limitation detect on exceed both high and low value
SDDF_CHANNEL _LIMIT_HITH_T HRES	Limitation detect on exceed high threshold value
SDDF_CHANNEL _LIMIT_LOW_TH RES	Limitation detect on exceed low threshold value
SDDF_CHANNEL _LIMIT_WINDO W	Limitation detect on windowed value

enum SDDF_ClockEdgeType

Select the input clock edge.

Enumerator:

SDDF_CLOCK_E DGE_RISING	Use the input clock rising edge
----------------------------	---------------------------------

SDDF_CLOCK_E DGE_FALLING	Use the input clock falling edge
SDDF_CLOCK_E DGE_BOTH	Use both edges of the input clock

enum SDDF_ClockSrcType

Select the input clock source.

Enumerator:

SDDF_CLOCK_S RC_EXTERNAL_ MODULATOR_C LOCK	The input clock is from the external modulator clock dedicated to this channel
---	--

enum SDDF_DebugModuleEnableType

Select whether the SDDF module enables in debug mode.

Enumerator:

SDDF_MODULE _ENABLE_IN_DE BUG	SDDF is enabled in debug mode
SDDF_MODULE _DISABLE_IN_D EBUG	SDDF is disabled in debug mode. Need trigger again for new conversion

enum SDDF_InputSrcType

Select the data source for limit check and zero cross check.

Enumerator:

SDDF_INPUT_SRC_MAIN_FILTER	The input source is main filter
SDDF_INPUT_SRC_AUXILIARY_FILTER	The input source is auxiliary filter

enum SDDF_MainFilterConvModeType

Selects the conversion mode for main filter.

Enumerator:

SDDF_MAIN_FILTER_CONV_SINGLE	Single mode. One conversion following a triggering event
SDDF_MAIN_FILTER_CONV_CONTINUOUS	Continuouts conversion mode. Multiple conversions following a triggering event, and the next trigger will cancel and restart conversion
SDDF_MAIN_FILTER_CONV_ALWAYS	Always conversion mode. Multiple conversions following the first trigger, and the next trigger will be ignored

enum SDDF_MainFilterOrderType

Selects the filter order for main filter.

Enumerator:

SDDF_MAIN_FILTER_FIRST_ORDER	The main filter is 1st order
------------------------------	------------------------------

SDDF_MAIN_FILTER_SECOND_ORDER	The main filter is 2nd order
SDDF_MAIN_FILTER_THIRD_ORDER	The main filter is 3rd order
SDDF_MAIN_FILTER_FOURTH_ORDER	The main filter is 4th order
SDDF_MAIN_FILTER_FIFTH_ORDER	The main filter is 5th order

enum SDDF_MainFilterShiftType

Set the main filter shift to get correct 24 bit precision.

Main Filter CIC raw result is always 32bit width. The user should program MFSHIFT for cut-off and shift left/right to get 24bit result.

Enumerator:

SDDF_MAIN_FILTER_NOT_SHIFT	Main filter no shift needed
SDDF_MAIN_FILTER_SHIFT_RIGHT_1	Main filter shift right by 1 bit
SDDF_MAIN_FILTER_SHIFT_RIGHT_2	Main filter shift right by 2 bits
SDDF_MAIN_FILTER_SHIFT_RIGHT_3	Main filter shift right by 3 bits
SDDF_MAIN_FILTER_SHIFT_RIGHT_4	Main filter shift right by 4 bits

SDDF_MAIN_FIL TER_SHIFT_RIG HT_5	Main filter shift right by 5 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_6	Main filter shift right by 6 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_7	Main filter shift right by 7 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_8	Main filter shift right by 8 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_9	Main filter shift right by 9 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_10	Main filter shift right by 10 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_11	Main filter shift right by 11 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_12	Main filter shift right by 12 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_13	Main filter shift right by 13 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_14	Main filter shift right by 14 bits
SDDF_MAIN_FIL TER_SHIFT_RIG HT_15	Main filter shift right by 15 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_1	Main filter shift left by 1 bit

SDDF_MAIN_FIL TER_SHIFT_LEF T_2	Main filter shift left by 2 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_3	Main filter shift left by 3 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_4	Main filter shift left by 4 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_5	Main filter shift left by 5 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_6	Main filter shift left by 6 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_7	Main filter shift left by 7 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_8	Main filter shift left by 8 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_9	Main filter shift left by 9 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_10	Main filter shift left by 10 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_11	Main filter shift left by 11 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_12	Main filter shift left by 12 bits
SDDF_MAIN_FIL TER_SHIFT_LEF T_13	Main filter shift left by 13 bits

SDDF_MAIN_FILTER_SHIFT_LEFT_14	Main filter shift left by 14 bits
SDDF_MAIN_FILTER_SHIFT_LEFT_15	Main filter shift left by 15 bits

enum SDDF_ModeType

Select the SDDF operation mode.

Enumerator:

SDDF_MODE_NORMAL	SDDF is in normal mode
SDDF_MODE_DEBUG	SDDF is in debug mode

enum SDDF_PreDivType

Select the SDDF function clock pre-divider ratio.

Enumerator:

SDDF_PREDIV_1	The clock is pre-divided by 1
SDDF_PREDIV_2	The clock is pre-divided by 2
SDDF_PREDIV_4	The clock is pre-divided by 4
SDDF_PREDIV_8	The clock is pre-divided by 8

enum SDDF_ResultFormatType

Select the result data format.

Enumerator:

SDDF_RESULT_DATA_LEFT_ALIGNED_SIGNED	The result is left justified, signed, for the case where input bitstream is signed
SDDF_RESULT_DATA_LEFT_ALIGNED_UNSIGNED	The result is left justified, unsigned, for the case where input bitstream is unsigned

enum SDDF_ShortCircuitDetectModeType

Select sht short circuit detect conversion mode.

Enumerator:

SDDF_SHORT_DETECT_ALWAYS_ON	Short circuit detect is always on when CHEN and MEN are asserted
SDDF_SHORT_DETECT_AUX_FILTER_ENABLED	Short circuit detect only works when the Auxiliary Filter is enabled

enum SDDF_ShortCircuitOptionType

Select the short circuit detection option.

Enumerator:

SDDF_SHORT_CIRCUIT_COUNT_ON_BOTH	Short circuit count on both 0 and 1, detect both high and low value
SDDF_SHORT_CIRCUIT_COUNT_ON_1	Short circuit count on 1, detect high value
SDDF_SHORT_CIRCUIT_COUNT_ON_0	Short circuit count on 0, detect low value

enum SDDF_TriggerEventType

Select the trigger level type.

Enumerator:

SDDF_TRIGGER_EVENT_EDGE	Trigger is edge event. Next edge event will cancel the current conversion, and re-start in single or continuous mode
SDDF_TRIGGER_EVENT_LEVEL	Trigger is level event. De-assert cancels the current conversion, and stop filter function in single or continuous mode

enum SDDF_TriggerSrcType

Select the trigger source.

Enumerator:

SDDF_TRIGGER_SRC_SOFTWARE	Software trigger is selected
---------------------------	------------------------------

SDDF_TRIGGER _SRC_HARDWA RE	Hardware trigger is selected
-----------------------------------	------------------------------

Function Documentation

LOCAL_INLINE void SDDF_HWA_ClearAbnormalInterruptStatus0 (SDDF_Type *const *pSddf*)

Clear the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

LOCAL_INLINE void SDDF_HWA_ClearAbnormalInterruptStatus1 (SDDF_Type *const *pSddf*)

Clear the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

LOCAL_INLINE void SDDF_HWA_ClearChannelClockAbsenceStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear clock absence flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelConvCompleteStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear channel conversion complete flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelFIFODataReadyStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear FIFO data ready flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelFIFOOverflowStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear FIFO overflow flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelFIFOUnderflowStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear FIFO underflow flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelHighLimitStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear out of high limit range flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelLowLimitStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear out of low limit range flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelSaturationStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear result saturation flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelShortCircuitDetectStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear short circuit flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelWindowLimitStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear out of window limit range flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearChannelZeroCrossDetectStatus (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Clear zero cross detect flag of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

LOCAL_INLINE void SDDF_HWA_ClearNormalInterruptStatus (SDDF_Type *const *pSddf*)

Clear the normal interrupt status register (SDDF_NISR) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

LOCAL_INLINE void SDDF_HWA_GenerateChannelSwTrigger (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Generate software trigger signal for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel to generate software trigger signal

LOCAL_INLINE uint32_t SDDF_HWA_GetAbnormalInterruptEnable0 (const SDDF_Type *const *pSddf*)

Get the channel abnormal interrupt 0 (SDDF_ABNIER0) register value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the SDDF_ABNIER0 register value

LOCAL_INLINE uint32_t SDDF_HWA_GetAbnormalInterruptEnable1 (const SDDF_Type *const *pSddf*)

Get the channel abnormal interrupt 1 (SDDF_ABNIER1) register value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the SDDF_ABNIER1 register value

LOCAL_INLINE uint32_t SDDF_HWA_GetAbnormalInterruptStatus0 (const SDDF_Type *const *pSddf*)

Get the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the value of SDDF_ABNISR0 register

LOCAL_INLINE uint32_t SDDF_HWA_GetAbnormalInterruptStatus1 (const SDDF_Type *const *pSddf*)

Get the abnormal interrupt status 1 register (SDDF_ABNISR1) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the value of SDDF_ABNISR1 register

LOCAL_INLINE SDDF_AuxFilterConvModeType SDDF_HWA_GetChannelAuxiliaryFilterConvMode (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the auxiliary filter conversion mode of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_AuxFilterConvModeType the auxiliary filter conversion mode

LOCAL_INLINE bool SDDF_HWA_GetChannelAuxiliaryFilterEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether auxiliary filter is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true auxiliary filter is enabled for the selected channel

false auxiliary filter is disabled for the selected channel

LOCAL_INLINE SDDF_AuxFilterOrderType SDDF_HWA_GetChannelAuxiliaryFilterOrder (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the auxiliary filter order of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_AuxFilterOrderType the auxiliary filter order

LOCAL_INLINE uint32_t SDDF_HWA_GetChannelAuxiliaryFilterOversampleRate (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the auxiliary filter oversampling rate of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint32_t the auxiliary filter oversampling rate

LOCAL_INLINE int32_t SDDF_HWA_GetChannelBias (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the bias value for the selected channel main filter.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

int32_t the bias value for result adjustment or compensation

LOCAL_INLINE bool SDDF_HWA_GetChannelClockAbsenceBreakSignalEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether clock absence detect break signal is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true clock absence detect break signal is enabled for the selected channel

false clock absence detect break signal is disabled for the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelClockAbsenceDetectEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether clock absence detect is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true clock absence detect is enabled for the selected channel

false clock absence detect is disabled for the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelClockAbsenceIntEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether channel clock absence detect interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel clock absence detect interrupt is enabled

false the channel clock absence detect interrupt is disabled

LOCAL_INLINE uint8_t SDDF_HWA_GetChannelClockAbsenceLimitThreshold (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the clock absence detector limitation threshold.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint8_t the clock absence detector limitation threshold

**LOCAL_INLINE bool SDDF_HWA_GetChannelClockAbsenceStatus (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get clock absence detect status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true clock absence is detected

false clock absence is not detected

**LOCAL_INLINE uint32_t SDDF_HWA_GetChannelConfig (const SDDF_Type *const *pSddf*, uint8_t
u8ChanIdx)**

Get the configuration register (SDDF_CFRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint32_t the channel configuration register value

**LOCAL_INLINE uint32_t SDDF_HWA_GetChannelControl (const SDDF_Type *const *pSddf*, uint8_t
u8ChanIdx)**

Get channel control register (SDDF_CCRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint32_t the SDDF_CCRn register value

LOCAL_INLINE bool SDDF_HWA_GetChannelConvCompleteIntEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether channel conversion complete interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true channel conversion complete interrupt is enabled

false channel conversion complete interrupt is disabled

LOCAL_INLINE bool SDDF_HWA_GetChannelConvCompleteStatus (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get channel conversion complete status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true channel conversion complete is detected

false channel conversion complete is not detected

LOCAL_INLINE bool SDDF_HWA_GetChannelConvOngoingStatus (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether conversion of the selected channel is ongoing.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

<i>u8ChanIdx</i>	the selected SDDF channel
------------------	---------------------------

Returns

- true the conversion is ongoing
- false the conversion is not ongoing

LOCAL_INLINE uint32_t SDDF_HWA_GetChannelData (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the result data of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

- uint32_t the result data

LOCAL_INLINE uint32_t SDDF_HWA_GetChannelDataRate (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the data rate register (SDDF_CDRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

- uint32_t the data rate register value of the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelDMAEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether DMA is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true DMA is enabled for the selected channel

false DMA is disabled for the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether the selected channel is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true the selected channel is enabled

false the selected channel is disabled

LOCAL_INLINE bool SDDF_HWA_GetChannelFIFODataReadyIntEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether channel FIFO data ready interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true channel FIFO data ready interrupt is enabled

false channel FIFO data ready interrupt is disabled

LOCAL_INLINE bool SDDF_HWA_GetChannelFIFODataReadyStatus (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get FIFO data ready status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true FIFO data ready is detected

false FIFO data ready is not detected

LOCAL_INLINE bool SDDF_HWA_GetChannelFIFOEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether FIFO is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true FIFO is enabled for the selected channel

false FIFO is disabled for the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelFIFOOverflowIntEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether channel FIFO overflow interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel FIFO overflow interrupt is enabled

false the channel FIFO overflow interrupt is disabled

**LOCAL_INLINE bool SDDF_HWA_GetChannelFIFOOverflowStatus (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get FIFO overflow status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true FIFO overflow is detected

false FIFO overflow is not detected

**LOCAL_INLINE bool SDDF_HWA_GetChannelFIFOUnderflowIntEnableFlag (const SDDF_Type *const
pSddf, uint8_t *u8ChanIdx*)**

Get whether channel FIFO underflow interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel FIFO underflow interrupt is enabled

false the channel FIFO underflow interrupt is disabled

**LOCAL_INLINE bool SDDF_HWA_GetChannelFIFOUnderflowStatus (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get FIFO underflow status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true FIFO underflow is detected

false FIFO underflow is not detected

LOCAL_INLINE uint8_t SDDF_HWA_GetChannelFIFOWatermark (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the FIFO watermark of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint8_t the FIFO watermark of the selected channel

LOCAL_INLINE uint32_t SDDF_HWA_GetChannelHighLimit (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the high limit threshold for limit check of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint32_t the high limit threshold

LOCAL_INLINE bool SDDF_HWA_GetChannelHighLimitBreakSignalEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether the high limit break signal is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true the high limit break signal is enabled for the selected channel

false the high limit break signal is disabled for the selected channel

**LOCAL_INLINE bool SDDF_HWA_GetChannelHighLimitIntEnableFlag (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get whether channel high limit detect interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel high limit detect interrupt is enabled

false the channel high limit detect interrupt is disabled

**LOCAL_INLINE bool SDDF_HWA_GetChannelHighLimitStatus (const SDDF_Type *const *pSddf*, uint8_t
u8ChanIdx)**

Get out of high limit range status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true out of high limit range is detected

false out of high limit range is not detected

**LOCAL_INLINE SDDF_BitFormatType SDDF_HWA_GetChannelInputBitFormat (const SDDF_Type *const
pSddf, uint8_t *u8ChanIdx*)**

Get the input bit format of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_BitFormatType the input bit format

LOCAL_INLINE SDDF_ClockEdgeType SDDF_HWA_GetChannelInputClockEdge (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the clock edge for conversion of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_ClockEdgeType the input clock edge

LOCAL_INLINE SDDF_ClockSrcType SDDF_HWA_GetChannelInputClockSource (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the input clock select of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_ClockSrcType the input clock select

LOCAL_INLINE bool SDDF_HWA_GetChannelLimitCheckEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether limit check is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true limit check is enabled for the selected channel

false limit check is disabled for the selected channel

LOCAL_INLINE SDDF_InputSrcType SDDF_HWA_GetChannelLimitCheckInputSrc (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the result input source of channel limit check.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_InputSrcType the result input source of channel limit check

LOCAL_INLINE SDDF_ChannelLimitOptionType SDDF_HWA_GetChannelLimitOption (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the channel limit check option.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_ChannelLimitOptionType the channel limit check option

LOCAL_INLINE uint32_t SDDF_HWA_GetChannelLowLimit (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the low limit threshold for limit check of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint32_t the low limit threshold

LOCAL_INLINE bool SDDF_HWA_GetChannelLowLimitBreakSignalEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether the low limit break signal is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true the low limit break signal is enabled for the selected channel

false the low limit break signal is disabled for the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelLowLimitIntEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether channel low limit detect interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel low limit detect interrupt is enabled

false the channel low limit detect interrupt is disabled

LOCAL_INLINE bool SDDF_HWA_GetChannelLowLimitStatus (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get out of low limit range status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true out of low limit range is detected

false out of low limit range is not detected

LOCAL_INLINE SDDF_MainFilterConvModeType SDDF_HWA_GetChannelMainFilterConvMode (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the main filter conversion mode of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_AuxFilterConvModeType the main filter conversion mode

LOCAL_INLINE bool SDDF_HWA_GetChannelMainFilterEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether main filter is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true main filter is enabled for the selected channel

false main filter is disabled for the selected channel

LOCAL_INLINE SDDF_MainFilterOrderType SDDF_HWA_GetChannelMainFilterOrder (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the main filter order of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_AuxFilterOrderType the main filter order

LOCAL_INLINE uint32_t SDDF_HWA_GetChannelMainFilterOversampleRate (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the main filter oversampling rate of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint32_t the main filter oversampling rate

LOCAL_INLINE SDDF_MainFilterShiftType SDDF_HWA_GetChannelMainFilterShift (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the main filter shift of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_MainFilterShiftType the main filter shift of the selected channel

LOCAL_INLINE uint32_t SDDF_HWA_GetChannelProtectionConfig (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the protection settings (SDDF_CPRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint32_t the protection settings (SDDF_CPRn) value

LOCAL_INLINE SDDF_ResultFormatType SDDF_HWA_GetChannelResultDataFormat (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the result data format of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_ResultFormatType the result data format of the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelSaturationIntEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether channel result saturation interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel result saturation interrupt is enabled

false the channel result saturation interrupt is disabled

LOCAL_INLINE bool SDDF_HWA_GetChannelSaturationStatus (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get result saturation status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true result saturation is detected

false result saturation is not detected

LOCAL_INLINE bool SDDF_HWA_GetChannelShortCircuitBreakSignalEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether short circuit detect break signal is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true short circuit detect break signal is enabled for the selected channel

false short circuit detect break signal is disabled for the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelShortCircuitDetectEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether short circuit detect is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true short circuit detect is enabled for the selected channel

false short circuit detect is disabled for the selected channel

LOCAL_INLINE bool SDDF_HWA_GetChannelShortCircuitDetectIntEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether channel short circuit detect interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel short circuit detect interrupt is enabled

false the channel short circuit detect interrupt is disabled

LOCAL_INLINE bool SDDF_HWA_GetChannelShortCircuitDetectStatus (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get short circuit detect status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true short circuit is detected

false short circuit is not detected

LOCAL_INLINE SDDF_ShortCircuitOptionType SDDF_HWA_GetChannelShortCircuitOption (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the channel short circuit detect option.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_ShortCircuitOptionType the channel short circuit detect option

LOCAL_INLINE SDDF_TriggerEventType SDDF_HWA_GetChannelTriggerEvent (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the trigger level type of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_TriggerEventType the trigger level type

LOCAL_INLINE SDDF_TriggerSrcType SDDF_HWA_GetChannelTriggerSource (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the trigger source type of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_TriggerSrcType the trigger source type

LOCAL_INLINE bool SDDF_HWA_GetChannelWindowLimitBreakSignalEnableFlag (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get whether the window limit break signal is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true the window limit break signal is enabled for the selected channel

false the window limit break signal is disabled for the selected channel

**LOCAL_INLINE bool SDDF_HWA_GetChannelWindowLimitIntEnableFlag (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get whether channel window limit detect interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true the channel window limit detect interrupt is enabled

false the channel window limit detect interrupt is disabled

**LOCAL_INLINE bool SDDF_HWA_GetChannelWindowLimitStatus (const SDDF_Type *const *pSddf*, uint8_t
u8ChanIdx)**

Get out of window limit range status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true out of window limit range is detected

false out of window limit range is not detected

**LOCAL_INLINE SDDF_InputSrcType SDDF_HWA_GetChannelZeroCrossCheckInputSrc (const SDDF_Type
*const *pSddf*, uint8_t *u8ChanIdx*)**

Get the result input source of channel zero cross check.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_InputSrcType the result input source of channel zero cross check

**LOCAL_INLINE bool SDDF_HWA_GetChannelZeroCrossDetectEnableFlag (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get whether zero cross detect is enabled for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true zero cross detect is enabled for the selected channel

false zero cross detect is disabled for the selected channel

**LOCAL_INLINE bool SDDF_HWA_GetChannelZeroCrossDetectIntEnableFlag (const SDDF_Type *const
pSddf, uint8_t *u8ChanIdx*)**

Get whether zero cross detect interrupt is enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index

Returns

true zero cross detect interrupt is enabled

false zero cross detect interrupt is disabled

**LOCAL_INLINE bool SDDF_HWA_GetChannelZeroCrossDetectStatus (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get zero cross detect status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

true zero cross is detected

false zero cross is not detected

LOCAL_INLINE bool SDDF_HWA_GetClockOut0DisableFlag (const SDDF_Type *const *pSddf*)

Get whether SDDF clock output 0 is disabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

true SDDF clock output 0 is disabled

false SDDF clock output 0 is enabled

LOCAL_INLINE bool SDDF_HWA_GetClockOut1DisableFlag (const SDDF_Type *const *pSddf*)

Get whether SDDF clock output 1 is disabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

true SDDF clock output 1 is disabled

false SDDF clock output 1 is enabled

LOCAL_INLINE bool SDDF_HWA_GetClockOut2DisableFlag (const SDDF_Type *const *pSddf*)

Get whether SDDF clock output 2 is disabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

true SDDF clock output 2 is disabled

false SDDF clock output 2 is enabled

LOCAL_INLINE bool SDDF_HWA_GetClockOutput0ReadyFlag (const SDDF_Type *const *pSddf*)

Get whether the modulator clock 0 output is ready.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

true the modulator clock 0 output is ready

false the modulator clock 0 output is not ready

LOCAL_INLINE bool SDDF_HWA_GetClockOutput1ReadyFlag (const SDDF_Type *const *pSddf*)

Get whether the modulator clock 1 output is ready.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

true the modulator clock 1 output is ready

false the modulator clock 1 output is not ready

LOCAL_INLINE bool SDDF_HWA_GetClockOutput2ReadyFlag (const SDDF_Type *const *pSddf*)

Get whether the modulator clock 2 output is ready.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

true the modulator clock 2 output is ready

false the modulator clock 2 output is not ready

LOCAL_INLINE uint32_t SDDF_HWA_GetCtrl (const SDDF_Type *const *pSddf*)

Get the global control register (SDDF_CTRL) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the SDDF_CTRL register value

LOCAL_INLINE uint32_t SDDF_HWA_GetDivider (const SDDF_Type *const *pSddf*)

Get the clock divider value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the clock out divider value

LOCAL_INLINE bool SDDF_HWA_GetMainEnableFlag (const SDDF_Type *const *pSddf*)

Get whether the SDDF function blocks are enabled.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

true the fuction blocks are enabled

false all function blocks are disabled

LOCAL_INLINE uint32_t SDDF_HWA_GetMainStatus (const SDDF_Type *const *pSddf*)

Get main status of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the main status of the selected channel

LOCAL_INLINE SDDF_DebugModuleEnableType SDDF_HWA_GetModuleDebugEnabledFlag (const SDDF_Type *const *pSddf*)

Get whether SDDF module is enabled in debug mode.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

SDDF_DebugModuleEnableType whether SDDF module is enabled in debug mode

LOCAL_INLINE uint32_t SDDF_HWA_GetNormalInterruptEnable (const SDDF_Type *const *pSddf*)

Get the normal interrupt (SDDF_NIER) register value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the SDDF_NIER register value

LOCAL_INLINE uint32_t SDDF_HWA_GetNormalInterruptStatus (const SDDF_Type *const *pSddf*)

Get the normal interrupt status register (SDDF_NISR) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

uint32_t the normal interrupt status (SDDF_NISR) register value

LOCAL_INLINE SDDF_ModeType SDDF_HWA_GetOperationMode (const SDDF_Type *const *pSddf*)

Get the SDDF operation mode (normal mode or debug mode)

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

SDDF_ModeType the operation mode of SDDF

LOCAL_INLINE SDDF_PreDivType SDDF_HWA_GetPreDivider (const SDDF_Type *const *pSddf*)

Get the function clock pre-divider value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

Returns

SDDF_PreDivType the function clock pre-divider value

LOCAL_INLINE SDDF_ShortCircuitDetectModeType SDDF_HWA_GetShortCircuitDetectMode (const SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*)

Get the channel short circuit detect conversion mode.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

SDDF_ShortCircuitDetectModeType the channel short circuit detect conversion mode

**LOCAL_INLINE uint8_t SDDF_HWA_GetShortCircuitLimitThreshold (const SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*)**

Get the threshold value for short circuit detector counter of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

Returns

uint8_t the threshold value for short circuit detector counter

LOCAL_INLINE void SDDF_HWA_MainDisable (SDDF_Type *const *pSddf*)

Disable all SDDF function blocks.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

LOCAL_INLINE void SDDF_HWA_MainEnable (SDDF_Type *const *pSddf*)

Enable the SDDF function blocks.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

LOCAL_INLINE void SDDF_HWA_Reset (SDDF_Type *const *pSddf*)

Reset all SDDF function blocks and clock configuration bits.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
--------------	---------------------------------------

LOCAL_INLINE void SDDF_HWA_SetAbnormalInterruptEnable0 (SDDF_Type *const *pSddf*, uint32_t *u32IntEn*)

Set the channel abnormal interrupt 0 (SDDF_ABNIER0) register value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u32IntEn</i>	the SDDF_ABNIER0 register value to set

LOCAL_INLINE void SDDF_HWA_SetAbnormalInterruptEnable1 (SDDF_Type *const *pSddf*, uint32_t *u32IntEn*)

Set the channel abnormal interrupt 1 (SDDF_ABNIER1) register value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u32IntEn</i>	the SDDF_ABNIER1 register value to set

LOCAL_INLINE void SDDF_HWA_SetChannelAuxiliaryFilterConvMode (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_AuxFilterConvModeType *eConvMode*)

Set the auxiliary filter conversion mode of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eConvMode</i>	the auxiliary filter conversion mode

LOCAL_INLINE void SDDF_HWA_SetChannelAuxiliaryFilterEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable auxiliary filter for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable auxiliary filter for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelAuxiliaryFilterOrder (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_AuxFilterOrderType *eOrder*)

Set the auxiliary filter order of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eOrder</i>	the auxiliary filter order

LOCAL_INLINE void SDDF_HWA_SetChannelAuxiliaryFilterOversampleRate (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, uint32_t *u32SampleRate*)

Set the auxiliary filter oversampling rate of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u32SampleRate</i>	the auxiliary filter oversampling rate

Here is the call graph for this function:

IMAGE

LOCAL_INLINE void SDDF_HWA_SetChannelBias (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, int32_t *s32Bias*)

Set the bias value for the selected channel main filter.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel

<i>s32Bias</i>	the bias value for result adjustment or compensation
----------------	--

LOCAL_INLINE void SDDF_HWA_SetChannelClockAbsenceBreakSignalEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable clock absence detect break signal for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable clock absence detect break signal for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelClockAbsenceDetectEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable clock absence detect for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable clock absence detect for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelClockAbsenceIntEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable channel clock absence detect interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel clock absence detect interrupt

**LOCAL_INLINE void SDDF_HWA_SetChannelClockAbsenceLimitThreshold (SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*, uint8_t *u8Thres*)**

Set the clock absence detector limitation threshold.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u8Thres</i>	the clock absence detector limitation threshold

**LOCAL_INLINE void SDDF_HWA_SetChannelConfig (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*,
uint32_t *u32Config*)**

Set the configuration register (SDDF_CFRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u32Config</i>	the channel configuration register value to set

**LOCAL_INLINE void SDDF_HWA_SetChannelControl (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*,
uint32_t *u32Ctrl*)**

Set channel control register (SDDF_CCRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u32Ctrl</i>	the SDDF_CCRn register value to set

**LOCAL_INLINE void SDDF_HWA_SetChannelConvCompleteIntEnableFlag (SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*, bool *bEnable*)**

Set whether to enable channel conversion complete interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel conversion complete interrupt

LOCAL_INLINE void SDDF_HWA_SetChannelDataRate (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, uint32_t *u32DataRate*)

Set the data rate register (SDDF_CDRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u32DataRate</i>	the data rate register value to set of the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelDMAEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable DMA for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable DMA for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable the selected channel

**LOCAL_INLINE void SDDF_HWA_SetChannelFIFODataReadyIntEnableFlag (SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*, bool *bEnable*)**

Set whether to enable channel FIFO data ready interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel FIFO data ready interrupt

**LOCAL_INLINE void SDDF_HWA_SetChannelFIFOEnableFlag (SDDF_Type *const *pSddf*, uint8_t
u8ChanIdx, bool *bEnable*)**

Set whether to enable FIFO for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable FIFO for the selected channel

**LOCAL_INLINE void SDDF_HWA_SetChannelFIFOOverflowIntEnableFlag (SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*, bool *bEnable*)**

Set whether to enable channel FIFO overflow interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel FIFO overflow interrupt

**LOCAL_INLINE void SDDF_HWA_SetChannelFIFOUnderflowIntEnableFlag (SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*, bool *bEnable*)**

Set whether to enable channel FIFO underflow interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChnIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel FIFO underflow interrupt

LOCAL_INLINE void SDDF_HWA_SetChannelFIFOWatermark (SDDF_Type *const *pSddf*, uint8_t *u8ChnIdx*, uint8_t *u8Watermark*)

Set the FIFO watermark of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChnIdx</i>	the selected SDDF channel
<i>u8Watermark</i>	the FIFO watermark of the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelHighLimit (SDDF_Type *const *pSddf*, uint8_t *u8ChnIdx*, uint32_t *u32HighLimit*)

Set the high limit threshold for limit check of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChnIdx</i>	the selected SDDF channel
<i>u32HighLimit</i>	the high limit threshold

LOCAL_INLINE void SDDF_HWA_SetChannelHighLimitBreakSignalEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChnIdx*, bool *bEnable*)

Set whether to enable high limit break signal for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChnIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable high limit break signal for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelHighLimitIntEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable channel high limit detect interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel high limit detect interrupt

LOCAL_INLINE void SDDF_HWA_SetChannelInputBitFormat (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_BitFormatType *eFormat*)

Set the input bit format of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eFormat</i>	the input bit format

LOCAL_INLINE void SDDF_HWA_SetChannelInputClockEdge (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_ClockEdgeType *eClkEdge*)

Set the clock edge for conversion of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eClkEdge</i>	the input clock edge

LOCAL_INLINE void SDDF_HWA_SetChannelInputClockSource (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_ClockSrcType *eSrc*)

Set the input clock select of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eSrc</i>	the input clock select

LOCAL_INLINE void SDDF_HWA_SetChannelLimitCheckEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable limit check for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable limit check for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelLimitCheckInputSrc (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_InputSrcType *eSrc*)

Set the result input source of the channel limit check.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eSrc</i>	the result input source of channel limit check

LOCAL_INLINE void SDDF_HWA_SetChannelLimitOption (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_ChannelLimitOptionType *eOption*)

Set the channel limit check option.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eOption</i>	the channel limit check option

LOCAL_INLINE void SDDF_HWA_SetChannelLowLimit (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, uint32_t *u32LowLimit*)

Set the low limit threshold for limit check of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u32LowLimit</i>	the low limit threshold

LOCAL_INLINE void SDDF_HWA_SetChannelLowLimitBreakSignalEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable low limit break signal for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable low limit break signal for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelLowLimitIntEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable channel low limit detect interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel low limit detect interrupt

LOCAL_INLINE void SDDF_HWA_SetChannelMainFilterConvMode (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_MainFilterConvModeType *eConvMode*)

Set the main filter conversion mode of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eConvMode</i>	the main filter conversion mode

LOCAL_INLINE void SDDF_HWA_SetChannelMainFilterEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable main filter for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable main filter for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelMainFilterOrder (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_MainFilterOrderType *eOrder*)

Set the main filter order of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eOrder</i>	the main filter order

LOCAL_INLINE void SDDF_HWA_SetChannelMainFilterOversampleRate (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, uint32_t *u32SampleRate*)

Set the main filter oversampling rate of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u32SampleRate</i>	the main filter oversampling rate

Here is the call graph for this function:

IMAGE

LOCAL_INLINE void SDDF_HWA_SetChannelMainFilterShift (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_MainFilterShiftType *eShift*)

Set the main filter shift to get correct 24 bit precision of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eShift</i>	the main filter shift to set

LOCAL_INLINE void SDDF_HWA_SetChannelProtectionConfig (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, uint32_t *u32Config*)

Set the protection settings (SDDF_CPRn) value of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u32Config</i>	the protection settings (SDDF_CPRn) value

LOCAL_INLINE void SDDF_HWA_SetChannelResultDataFormat (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_ResultFormatType *eFormat*)

Set the result data format of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eFormat</i>	the result data format of the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelSaturationIntEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable channel result saturation interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel result saturation interrupt

LOCAL_INLINE void SDDF_HWA_SetChannelShortCircuitBreakSignalEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable short circuit detect break signal for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable short circuit detect break signal for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelShortCircuitDetectEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable short circuit detect for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable short circuit detect for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelShortCircuitDetectIntEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable channel short circuit detect interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel short circuit detect interrupt

LOCAL_INLINE void SDDF_HWA_SetChannelShortCircuitOption (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_ShortCircuitOptionType *eOption*)

Set the channel short circuit detect option.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eOption</i>	the channel short circuit detect option

LOCAL_INLINE void SDDF_HWA_SetChannelTriggerEvent (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_TriggerEventType *eEvent*)

Set the trigger level type of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eEvent</i>	the trigger level type

LOCAL_INLINE void SDDF_HWA_SetChannelTriggerSource (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_TriggerSrcType *eSrc*)

Set the trigger source type of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eSrc</i>	the trigger source type

LOCAL_INLINE void SDDF_HWA_SetChannelWindowLimitBreakSignalEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable window limit break signal for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable window limit break signal for the selected channel

LOCAL_INLINE void SDDF_HWA_SetChannelWindowLimitIntEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable channel window limit detect interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable channel window limit detect interrupt

LOCAL_INLINE void SDDF_HWA_SetChannelZeroCrossCheckInputSrc (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_InputSrcType *eSrc*)

Set the result input source of the channel zero cross check.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eSrc</i>	the result input source of channel zero cross check

LOCAL_INLINE void SDDF_HWA_SetChannelZeroCrossDetectEnableFlag (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, bool *bEnable*)

Set whether to enable zero cross detect for the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>bEnable</i>	whether to enable zero cross detect for the selected channel

**LOCAL_INLINE void SDDF_HWA_SetChannelZeroCrossDetectIntEnableFlag (SDDF_Type *const *pSddf*,
uint8_t *u8ChanIdx*, bool *bEnable*)**

Set whether to enable zero cross detect interrupt.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the channel index
<i>bEnable</i>	whether to enable zero cross detect interrupt

LOCAL_INLINE void SDDF_HWA_SetClockOut0DisableFlag (SDDF_Type *const *pSddf*, bool *bDisable*)

Set whether to disable clock output 0.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>bDisable</i>	whether to disable clock output 0

LOCAL_INLINE void SDDF_HWA_SetClockOut1DisableFlag (SDDF_Type *const *pSddf*, bool *bDisable*)

Set whether to disable clock output 1.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>bDisable</i>	whether to disable clock output 1

LOCAL_INLINE void SDDF_HWA_SetClockOut2DisableFlag (SDDF_Type *const *pSddf*, bool *bDisable*)

Set whether to disable clock output 2.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>bDisable</i>	whether to disable clock output 2

LOCAL_INLINE void SDDF_HWA_SetCtrl (SDDF_Type *const *pSddf*, uint32_t *u32Ctrl*)

Set the global control register (SDDF_CTRL) value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u32Ctrl</i>	the SDDF_CTRL register value to set

LOCAL_INLINE void SDDF_HWA_SetDivider (SDDF_Type *const *pSddf*, uint32_t *u32Div*)

Set the clock divider value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u32Div</i>	the clock out divider value

LOCAL_INLINE void SDDF_HWA_SetModuleDebugEnabledFlag (SDDF_Type *const *pSddf*, SDDF_DebugModuleEnableType *eEnable*)

Set whether SDDF module is enabled in debug mode.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>eEnable</i>	whether SDDF module is enabled in debug mode

LOCAL_INLINE void SDDF_HWA_SetNormalInterruptEnable (SDDF_Type *const *pSddf*, uint32_t *u32IntEn*)

Set the normal interrupt (SDDF_NIER) register value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u32IntEn</i>	the SDDF_NIER register value to set

LOCAL_INLINE void SDDF_HWA_SetOperationMode (SDDF_Type *const *pSddf*, SDDF_ModeType *eMode*)

Set the SDDF operation mode.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>eMode</i>	the operation mode of SDDF

Here is the call graph for this function:

IMAGE

LOCAL_INLINE void SDDF_HWA_SetPreDivider (SDDF_Type *const *pSddf*, SDDF_PreDivType *eDiv*)

Set the function clock pre-divider value.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>eDiv</i>	the function clock pre-divider value

LOCAL_INLINE void SDDF_HWA_SetShortCircuitDetectMode (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, SDDF_ShortCircuitDetectModeType *eMode*)

Set the channel short circuit detect conversion mode.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>eMode</i>	the channel short circuit detect conversion mode

LOCAL_INLINE void SDDF_HWA_SetShortCircuitLimitThreshold (SDDF_Type *const *pSddf*, uint8_t *u8ChanIdx*, uint8_t *u8Thres*)

Set the threshold value for short circuit detector counter of the selected channel.

Parameters

<i>pSddf</i>	the base address of the SDDF instance
<i>u8ChanIdx</i>	the selected SDDF channel
<i>u8Thres</i>	the threshold value for short circuit detector counter

HwA_sec

Collaboration diagram for HwA_sec:Functions

LOCAL_INLINE void **SEC_HWA_EnDebug** (SEC_Type *const pSec)

Enable the debug module.

LOCAL_INLINE uint16_t **SEC_HWA_GetSScontrol0** (SEC_Type *const pSec)

Get the system security KEY0.

LOCAL_INLINE uint16_t **SEC_HWA_GetSScontrol1** (SEC_Type *const pSec)

Get the system security KEY1.

LOCAL_INLINE void **SEC_HWA_ReEnDebug** (SEC_Type *const pSec)

Re-enable the Debug mode.

LOCAL_INLINE bool **SEC_HWA_GetReEnDebug** (SEC_Type *const pSec)

Get the Re-enable Debug permission.

LOCAL_INLINE void **SEC_HWA_WriteUnlock** (SEC_Type *const pSec)

Enable the write operation for SEC register.

LOCAL_INLINE bool **SEC_HWA_GetWritePer** (SEC_Type *const pSec)

Get the sec write permission.

LOCAL_INLINE void **SEC_HWA_WriteReEnKeyn** (SEC_Type *const pSec, uint8_t count, uint32_t key)

Enable the write operation for SEC register.

LOCAL_INLINE void **SEC_HWA_EnTest** (SEC_Type *const pSec)

Enable the Test module.

LOCAL_INLINE void **SEC_HWA_ReEnTest** (SEC_Type *const pSec)

Re-Enable Test mode.

LOCAL_INLINE bool **SEC_HWA_GetReEnTest** (SEC_Type *const pSec)

Get the Re-enable Test permission.

LOCAL_INLINE void **SEC_HWA_ReEnTestKey** (SEC_Type *const pSec, uint32_t testkey)

write Test re-enable mode key

LOCAL_INLINE void **SEC_HWA_EnME** (SEC_Type *const pSec)

Enable the Mass Erase.

LOCAL_INLINE void **SEC_HWA_EnWriteB0NVR** (SEC_Type *const pSec)

Enable the Block 0 NVR write .

LOCAL_INLINE void **SEC_HWA_DisWriteB0NVR** (SEC_Type *const pSec)

Disable the Block 0 NVR write .

LOCAL_INLINE void **SEC_HWA_EnEraseB0NVR** (SEC_Type *const pSec)

Enable the Block 0 NVR erase .

LOCAL_INLINE void **SEC_HWA_DisEraseB0NVR** (SEC_Type *const pSec)

Disable the Block 0 NVR erase .

LOCAL_INLINE uint8_t **SEC_HWA_GetFastBootClock** (SEC_Type *const pSec)

Get the Fast Boot Select.

LOCAL_INLINE bool **SEC_HWA_GetNmiPin** (SEC_Type *const pSec)

NMI Function Enable/Disable.

LOCAL_INLINE bool **SEC_HWA_GetBootRom** (SEC_Type *const pSec)

Boot Rom Configuration.

LOCAL_INLINE bool **SEC_HWA_GetIsSpStatus** (SEC_Type *const pSec)

ISP Mode Status.

LOCAL_INLINE bool **SEC_HWA_GetUKAS** (SEC_Type *const pSec)

User Key Access Enable. Only valid under non-secure boot.

LOCAL_INLINE bool **SEC_HWA_GetSB** (SEC_Type *const pSec)

Secure Boot Disable (Value loaded from NVR sector)

LOCAL_INLINE bool **SEC_HWA_GetDEAUEn** (SEC_Type *const pSec)

Host Debug Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

LOCAL_INLINE bool **SEC_HWA_GetISPAU** (SEC_Type *const pSec)

ISP Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

LOCAL_INLINE uint8_t **SEC_HWA_GetUartBR** (SEC_Type *const pSec)

Get the FCUART Baud Rate for ISP.

LOCAL_INLINE uint8_t **SEC_HWA_GetCanBR** (SEC_Type *const pSec)

Get CAN Baud Rate for ISP.

LOCAL_INLINE uint8_t **SEC_HWA_GetOSCFre** (SEC_Type *const pSec)

Get OSC Frequency.

LOCAL_INLINE bool **SEC_HWA_GetOSCAvail** (SEC_Type *const pSec)

Get whether OSC Available.

LOCAL_INLINE bool **SEC_HWA_GetMBBKEN** (SEC_Type *const pSec)

Get whether Debug Mailbox Backdoor Key Enable.

LOCAL_INLINE uint32_t **SEC_HWA_GetBLMask** (SEC_Type *const pSec)

Get Bootloader Verification Mask.

LOCAL_INLINE uint8_t **SEC_HWA_GetIsplns** (SEC_Type *const pSec)

Get the ISP Instance Select.

LOCAL_INLINE uint8_t **SEC_HWA_GetBLVer** (SEC_Type *const pSec)

Get Bootloader Verification Algorithm.

LOCAL_INLINE uint32_t **SEC_HWA_GetBLAddr** (SEC_Type *const pSec)

Get the bootloader address.

LOCAL_INLINE uint8_t **SEC_HWA_GetLCStaus** (SEC_Type *const pSec)

Get the life cycle status of the chip.

LOCAL_INLINE bool **SEC_HWA_GetHUKRead** (SEC_Type *const pSec)

Get the Host User Key Read Protection.

LOCAL_INLINE bool **SEC_HWA_GetHUKWrite** (SEC_Type *const pSec)

Get the Host User Key write Protection.

LOCAL_INLINE bool **SEC_HWA_GetHUKEraser** (SEC_Type *const pSec)

Get the Host User Key erase Protection.

LOCAL_INLINE bool **SEC_HWA_GetHostNvrRead** (SEC_Type *const pSec)

Get the Host NVR Read Protection.

LOCAL_INLINE bool **SEC_HWA_GetHostNvrWrite** (SEC_Type *const pSec)

Get the Host NVR write Protection.

LOCAL_INLINE bool **SEC_HWA_GetHostNvrErase** (SEC_Type *const pSec)

Get the Host NVR erase Protection.

LOCAL_INLINE bool **SEC_HWA_GetHsmUKERase** (SEC_Type *const pSec)

Get the HSM User Key Erase Protection.

LOCAL_INLINE bool **SEC_HWA_GetHsmNvrErase** (SEC_Type *const pSec)

Get the HSM NVR Erase Protection.

Detailed Description

Function Documentation

LOCAL_INLINE void **SEC_HWA_DisEraseB0NVR** (SEC_Type *const pSec)

Disable the Block 0 NVR erase .

Parameters

pSec	A pointer to the SEC registers.
------	---------------------------------

LOCAL_INLINE void **SEC_HWA_DisWriteB0NVR** (SEC_Type *const pSec)

Disable the Block 0 NVR write .

Parameters

pSec	A pointer to the SEC registers.
------	---------------------------------

LOCAL_INLINE void SEC_HWA_EnDebug (SEC_Type *const *pSec*)

Enable the debug module.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE void SEC_HWA_EnEraseB0NVR (SEC_Type *const *pSec*)

Enable the Block 0 NVR erase .

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE void SEC_HWA_EnME (SEC_Type *const *pSec*)

Enable the Mass Erase.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE void SEC_HWA_EnTest (SEC_Type *const *pSec*)

Enable the Test module.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE void SEC_HWA_EnWriteB0NVR (SEC_Type *const *pSec*)

Enable the Block 0 NVR write .

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE uint32_t SEC_HWA_GetBLAddr (SEC_Type *const *pSec*)

Get the bootloader address.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

Bootloader Address.

LOCAL_INLINE uint32_t SEC_HWA_GetBLMask (SEC_Type *const *pSec*)

Get Bootloader Verification Mask.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

Bootloader Verification Mask.

LOCAL_INLINE uint8_t SEC_HWA_GetBLVer (SEC_Type *const *pSec*)

Get Bootloader Verification Algorithm.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

Bootloader Verification Algorithm

LOCAL_INLINE bool SEC_HWA_GetBootRom (SEC_Type *const *pSec*)

Boot Rom Configuration.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

false - Boot from GPR defined address (except ROM). true - Boot from ROM.

LOCAL_INLINE uint8_t SEC_HWA_GetCanBR (SEC_Type *const *pSec*)

Get CAN Baud Rate for ISP.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

CAN Baud Rate for ISP

LOCAL_INLINE bool SEC_HWA_GetDEAUEn (SEC_Type *const *pSec*)

Host Debug Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true means Host debug authentication enable. false means Host debug authentication disable.

LOCAL_INLINE uint8_t SEC_HWA_GetFastBootClock (SEC_Type *const *pSec*)

Get the Fast Boot Select.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

0b - Select PLL0 as the core clock source; the core clock is 300MHz 1b - Select FIRC as the core clock source; the core clock is 96MHz.

LOCAL_INLINE bool SEC_HWA_GetHostNvrErase (SEC_Type *const *pSec*)

Get the Host NVR erase Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - Host erase access to NVR region is enabled false - Host erase access to NVR region is disabled

LOCAL_INLINE bool SEC_HWA_GetHostNvrRead (SEC_Type *const *pSec*)

Get the Host NVR Read Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - Host read access to NVR region is enabled false - Host read access to NVR region is disabled

LOCAL_INLINE bool SEC_HWA_GetHostNvrWrite (SEC_Type *const *pSec*)

Get the Host NVR write Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - Host write access to NVR region is enabled false - Host write access to NVR region is disabled

LOCAL_INLINE bool SEC_HWA_GetHsmNvrErase (SEC_Type *const *pSec*)

Get the HSM NVR Erase Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - HSM erase access to NVR region is enabled false -HSM erase access to NVR region is disabled

LOCAL_INLINE bool SEC_HWA_GetHsmUKEraser (SEC_Type *const *pSec*)

Get the HSM User Key Erase Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - HSM erase access to User Key region is enabled false -HSM erase access to User Key region is disabled

LOCAL_INLINE bool SEC_HWA_GetHUKEraser (SEC_Type *const *pSec*)

Get the Host User Key erase Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - Host erase access to User Key region is enabled false - Host erase access to User Key region is disabled

LOCAL_INLINE bool SEC_HWA_GetHUKRead (SEC_Type *const *pSec*)

Get the Host User Key Read Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - Host read access to User Key region is enabled false - Host read access to User Key region is disabled

LOCAL_INLINE bool SEC_HWA_GetHUKWrite (SEC_Type *const *pSec*)

Get the Host User Key write Protection.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true - Host write access to User Key region is enabled false - Host write access to User Key region is disabled

LOCAL_INLINE bool SEC_HWA_GetISPAU (SEC_Type *const *pSec*)

ISP Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true means ISP authentication enable, false means ISP authentication disable.

LOCAL_INLINE uint8_t SEC_HWA_GetIsplns (SEC_Type *const *pSec*)

Get the ISP Instance Select.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

ISP Instance Select

LOCAL_INLINE bool SEC_HWA_GetIsplStatus (SEC_Type *const *pSec*)

ISP Mode Status.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

false - ISP mode is inactive. true - ISP mode is active.

LOCAL_INLINE uint8_t SEC_HWA_GetLCStaus (SEC_Type *const *pSec*)

Get the life cycle status of the chip.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

The lifecycle status

LOCAL_INLINE bool SEC_HWA_GetMBBKEN (SEC_Type *const *pSec*)

Get whether Debug Mailbox Backdoor Key Enable.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true means user can enable debug by writing debug backdoor key, false means user can not enable debug

LOCAL_INLINE bool SEC_HWA_GetNmiPin (SEC_Type *const *pSec*)

NMI Function Enable/Disable.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

false - NMI pin function is disabled after reset. true - NMI pin function is enabled

LOCAL_INLINE bool SEC_HWA_GetOSCAvail (SEC_Type *const *pSec*)

Get whether OSC Available.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true means OSC is available. false means -OSC is not available

LOCAL_INLINE uint8_t SEC_HWA_GetOSCFre (SEC_Type *const *pSec*)

Get OSC Frequency.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

OSC Frequency

LOCAL_INLINE bool SEC_HWA_GetReEnDebug (SEC_Type *const *pSec*)

Get the Re-enable Debug permission.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE bool SEC_HWA_GetReEnTest (SEC_Type *const *pSec*)

Get the Re-enable Test permission.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE bool SEC_HWA_GetSB (SEC_Type *const *pSec*)

Secure Boot Disable (Value loaded from NVR sector)

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true means Secure boot mode , false means Non secure boot mode.

LOCAL_INLINE uint16_t SEC_HWA_GetSScontrol0 (SEC_Type *const *pSec*)

Get the system security KEY0.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE uint16_t SEC_HWA_GetSScontrol1 (SEC_Type *const *pSec*)

Get the system security KEY1.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE uint8_t SEC_HWA_GetUartBR (SEC_Type *const *pSec*)

Get the FCUART Baud Rate for ISP.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

FCUART Baud Rate for ISP

LOCAL_INLINE bool SEC_HWA_GetUKAS (SEC_Type *const *pSec*)

User Key Access Enable. Only valid under non-secure boot.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true means User key can be read/programmed/erased by host CPU ,false means User key is not available for host cpu

LOCAL_INLINE bool SEC_HWA_GetWritePer (SEC_Type *const *pSec*)

Get the sec write permission.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

Returns

true means allow write. false means forbid write.

LOCAL_INLINE void SEC_HWA_ReEnDebug (SEC_Type *const *pSec*)

Re-enable the Debug mode.

Note

This register can only be write once. startup_fc4150.s has lock the register.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE void SEC_HWA_ReEnTest (SEC_Type *const *pSec*)

Re-Enable Test mode.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

LOCAL_INLINE void SEC_HWA_ReEnTestKey (SEC_Type *const *pSec*, uint32_t *testkey*)

write Test re-enable mode key

Parameters

<i>pSec</i>	A pointer to the SEC registers.
<i>testkey</i>	The test key value to be reloaded into the TMEK register.

LOCAL_INLINE void SEC_HWA_WriteReEnKeyn (SEC_Type *const *pSec*, uint8_t *count*, uint32_t *key*)

Enable the write operation for SEC register.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
<i>count</i>	Counter indicating the number of encryption executions
<i>key</i>	New encryption key to be written

LOCAL_INLINE void SEC_HWA_WriteUnlock (SEC_Type *const *pSec*)

Enable the write operation for SEC register.

Note

This register can only be write once. startup_fc4150.s has lock the register.

Parameters

<i>pSec</i>	A pointer to the SEC registers.
-------------	---------------------------------

HwA_sent

Collaboration diagram for HwA_sent:Enumerations

```
enum SENT_NibbleDataModeType { SENT_NIBBLE_DATA_MODE_A = 0x0U,  
SENT_NIBBLE_DATA_MODE_H1 = 0x1U, SENT_NIBBLE_DATA_MODE_H2 = 0x2U,  
SENT_NIBBLE_DATA_MODE_H3 = 0x3U, SENT_NIBBLE_DATA_MODE_H4 = 0x4U,  
SENT_NIBBLE_DATA_MODE_H5 = 0x5U, SENT_NIBBLE_DATA_MODE_H6 = 0x6U,  
SENT_NIBBLE_DATA_MODE_H7 = 0x7U }
```

sent running status

```
enum SENT_SlowMessageType { SENT_SLOW_MESSAGE_SHORT = 0x0U,  
SENT_SLOW_MESSAGE_ENHANCE_12DATA_8ID = 0x1U, SENT_SLOW_MESSAGE_ENHANCE_16DATA_4ID  
= 0x2U }
```

sent slow message type

```
enum SENT_SPCTriggerType { SENT_SPC_SW_TRIGGER = 0x0U, SENT_SPC_HW_TRIGGER = 0x1U }
```

sent SPC trigger type

```
enum SENT_SPCTickType { SENT_SPC_TICK_PRE_RECEIVED_MESSAGE = 0x0U,  
SENT_SPC_TICK_CONFIG = 0x1U }
```

sent SPC tick base

Functions

```
LOCAL_INLINE void SENT_HWA_SetGlobalPreScaler (SENT_Type *const pSent, uint8_t u8PreScaler)
```

Set SENT Global pre-scaler value.

```
LOCAL_INLINE void SENT_HWA_SetDMAWaterMark (SENT_Type *const pSent, uint8_t u8WaterMark)
```

Set SENT Global DMA watermark value.

```
LOCAL_INLINE void SENT_HWA_EnableFastMessageFIFOOverflowInterrupt (SENT_Type *const pSent)
```

Enable SENT Fast message FIFO overflow interrupt.

```
LOCAL_INLINE void SENT_HWA_DisableFastMessageFIFOOverflowInterrupt (SENT_Type *const pSent)
```

Disable SENT Fast message FIFO overflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableFastMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)

Enable SENT Fast message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_DisableFastMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)

Disable SENT Fast message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableSlowMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)

Enable SENT slow message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_DisableSlowMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)

Disable SENT slow message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableDebugMode** (SENT_Type *const pSent)

Enable SENT debug mode.

LOCAL_INLINE void **SENT_HWA_DisableDebugMode** (SENT_Type *const pSent)

Disable SENT debug mode.

LOCAL_INLINE void **SENT_HWA_EnableGlobal** (SENT_Type *const pSent)

Enable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_DisableGlobal** (SENT_Type *const pSent)

Disable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_EnableDataOverflowFlagFastClear** (SENT_Type *const pSent)

Enable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_DisableDataOverflowFlagFastClear** (SENT_Type *const pSent)

Disable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_EnableChannelReceive** (SENT_Type *const pSent, uint8_t u8Channel)
Enable SENT channel receive.

LOCAL_INLINE void **SENT_HWA_DisableChannelReceive** (SENT_Type *const pSent, uint8_t u8Channel)
Disable SENT channel receive.

LOCAL_INLINE bool **SENT_HWA_GetChannelReceive** (const SENT_Type *const pSent, uint8_t u8Channel)
Get SENT channel receive.

LOCAL_INLINE bool **SENT_HWA_GetSlowMessageUnderflowFlag** (const SENT_Type *const pSent, uint8_t u8Channel)
Get Channel slow message dma read underflow flag.

LOCAL_INLINE void **SENT_HWA_ClearSlowMessageUnderflowFlag** (SENT_Type *const pSent, uint8_t u8Channel)
Clear Channel slow message dma read underflow flag.

LOCAL_INLINE bool **SENT_HWA_GetFastMessageReadyFlag** (const SENT_Type *const pSent, uint8_t u8Channel)
Get Channel fast message ready flag.

LOCAL_INLINE void **SENT_HWA_ClearFastMessageReadyFlag** (SENT_Type *const pSent, uint8_t u8Channel)
Clear Channel fast message ready flag.

LOCAL_INLINE bool **SENT_HWA_GetSlowMessageReadyFlag** (const SENT_Type *const pSent, uint8_t u8Channel)
Get Channel slow message ready flag.

LOCAL_INLINE void **SENT_HWA_ClearSlowMessageReadyFlag** (SENT_Type *const pSent, uint8_t u8Channel)
Clear Channel slow message ready flag.

LOCAL_INLINE void **SENT_HWA_EnableChannelFIFO** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel FIFO function.

LOCAL_INLINE void **SENT_HWA_DisableChannelFIFO** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel receive function.

LOCAL_INLINE void **SENT_HWA_SetChannelNibbleNumber** (SENT_Type *const pSent, uint8_t u8Channel, uint8_t u8Number)

Set SENT channel data nibble number.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelNibbleNumber** (const SENT_Type *const pSent, uint8_t u8Channel)

Get SENT channel data nibble number.

LOCAL_INLINE bool **SENT_HWA_GetFastMessageFIFOOverflowFlag** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message FIFO overflow flag.

LOCAL_INLINE void **SENT_HWA_ClearFastMessageFIFOOverflowFlag** (SENT_Type *const pSent, uint8_t u8Channel)

Clear Channel fast message FIFO overflow flag.

LOCAL_INLINE bool **SENT_HWA_GetFastMessageDMAUnderflowFlag** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message DMA underflow flag.

LOCAL_INLINE void **SENT_HWA_ClearFastMessageDMAUnderflowFlag** (SENT_Type *const pSent, uint8_t u8Channel)

Clear Channel fast message DMA underflow flag.

LOCAL_INLINE void **SENT_HWA_SetChannelNibbleDataMode** (SENT_Type *const pSent, uint8_t u8Channel, **SENT_NibbleDataModeType** eMode)

Set SENT channel nibble data mode.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel fast message DMA request.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel fast message DMA request.

LOCAL_INLINE void **SENT_HWA_EnableChannelSlowMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel slow message DMA request.

LOCAL_INLINE void **SENT_HWA_DisableChannelSlowMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel slow message DMA request.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Fast message interrupt.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Fast message interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelSlowMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel slow message interrupt.

LOCAL_INLINE void **SENT_HWA_DisableChannelSlowMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel slow message interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCompensate** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Compensate.

LOCAL_INLINE void **SENT_HWA_DisableChannelCompensate** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Compensate.

LOCAL_INLINE void **SENT_HWA_SetChannelPreScaler** (SENT_Type *const pSent, uint8_t u8Channel, uint16_t u16PreScaler)

Set SENT channel pre-scaler.

LOCAL_INLINE uint16_t **SENT_HWA_GetChannelCompensatePreScaler** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel Prescaler value after compensate.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelStatus** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel status.

LOCAL_INLINE void **SENT_HWA_ClearChannelStatus** (SENT_Type *const pSent, uint8_t u8Channel, uint32_t u32W1CBit)

Clear Channel status.

LOCAL_INLINE void **SENT_HWA_EnableChannelAllErrorInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel all error interrupt.

LOCAL_INLINE void **SENT_HWA_DisableChannelAllErrorInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel all error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelBusIdleInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel idle flag interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalResyncInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel sync/calibration pulse error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalERRInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable Sync/calibration pulse difference exceed 1.5625% error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalFailInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Calibration 20% pass 25% fail interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelSOVFLInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Slow message overflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelFOVFLInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel fast message overflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelNibbleErrorInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Nibble value error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelPPDiagInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Previous pulse diagnosis error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCALDiagInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Calibration diagnosis over 25% error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelFCRCInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Fast message CRC check error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelSCRCInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel slow message CRC check error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelEdgeERRInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Falling edge number error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelAltCRC** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Alternative 4-bit CRC algorithm.

LOCAL_INLINE void **SENT_HWA_DisableChannelAltCRC** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Alternative 4-bit CRC algorithm.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageDataChange** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Fast message data change.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageDataChange** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Fast message data change.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalValid20To25** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel calibration valid range from 20% tp 25%.

LOCAL_INLINE void **SENT_HWA_DisableChannelCalValid20To25** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel calibration valid range from 20% tp 25%.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalValidDiagnostic** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel calibration valid diagnostic.

LOCAL_INLINE void **SENT_HWA_DisableChannelCalValidDiagnostic** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel calibration valid diagnostic.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel fast message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel fast message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageCRCWithSC** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Fast message crc with S&C.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageCRCWithSC** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Fast message crc with S&C.

LOCAL_INLINE void **SENT_HWA_EnableChannelSlowMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel slow message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_DisableChannelSlowMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel slow message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_EnableChannelPausePulse** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Pause pulse.

LOCAL_INLINE void **SENT_HWA_DisableChannelPausePulse** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Successive calibration pulses diagnostic option1.

LOCAL_INLINE void **SENT_HWA_EnableChannelSPCOption1** (SENT_Type *const pSent, uint8_t u8Channel)
Enable SENT channel Successive calibration pulses diagnostic option1.

LOCAL_INLINE void **SENT_HWA_DisableChannelSPCOption1** (SENT_Type *const pSent, uint8_t u8Channel)
Disable SENT channel Pause pulse.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageCRCCheck** (SENT_Type *const pSent, uint8_t u8Channel)
Enable SENT channel Fast message crc check.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageCRCCheck** (SENT_Type *const pSent, uint8_t u8Channel)
Disable SENT channel Fast message crc check.

LOCAL_INLINE void **SENT_HWA_SetChannelDigitalFilterCount** (SENT_Type *const pSent, uint8_t u8Channel, uint8_t u8Count)
Set SENT channel digital filter count.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelDataNibble** (const SENT_Type *const pSent, uint8_t u8Channel)
Get Channel fast message data nibble.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelFastMessageCRCNibble** (const SENT_Type *const pSent, uint8_t u8Channel)
Get Channel fast message crc nibble.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelFastMessageStatusNibble** (const SENT_Type *const pSent, uint8_t u8Channel)
Get Channel fast message status communication nibble.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelFastMessageTimeStamp** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message status timestamp.

LOCAL_INLINE **SENT_SlowMessageType SENT_HWA_GetChannelSlowMessageType** (const SENT_Type
*const pSent, uint8_t u8Channel)

Get Channel slow message type.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelBit3EnhancedID7_4_OR_ID3_0** (const SENT_Type *const
pSent, uint8_t u8Channel)

Get Channel enhanced serial message id bit 7 to bit4 or id bit 3 to id bit 0.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelBit3EnhancedID3_0_OR_DATA15_12** (const SENT_Type
*const pSent, uint8_t u8Channel)

Get Channel enhanced serial message id bit 3 to bit0 or data bit 15 to id bit 12.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelBit2CRC** (const SENT_Type *const pSent, uint8_t
u8Channel)

Get Channel slow message bit2 with CRC.

LOCAL_INLINE uint16_t **SENT_HWA_GetChannelBit2DATA** (const SENT_Type *const pSent, uint8_t
u8Channel)

Get Channel slow message bit2 with DATA.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelSlowMessageTimeStamp** (const SENT_Type *const
pSent, uint8_t u8Channel)

Get Channel slow message status timestamp.

LOCAL_INLINE uint32_t **SENT_HWA_GetChanneReceivedFrameNumber** (const SENT_Type *const pSent,
uint8_t u8Channel)

Get Channel number of received frames.

LOCAL_INLINE uint32_t **SENT_HWA_GetChanneReceivedEdgeNumber** (const SENT_Type *const pSent,
uint8_t u8Channel)

Get Channel number of received edges.

LOCAL_INLINE void **SENT_HWA_ClearChannelReceivedFrameNumber** (SENT_Type *const pSent, uint8_t u8Channel)

Clear the number of received frames.

LOCAL_INLINE void **SENT_HWA_ClearChannelReceivedEdgeNumber** (SENT_Type *const pSent, uint8_t u8Channel)

Clear the number of received edges.

LOCAL_INLINE void **SENT_HWA_EnableChannelSPCMode** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel SPC mode.

LOCAL_INLINE void **SENT_HWA_DisableChannelSPCMode** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel SPC mode.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalibrationDiag** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel calibration diagnosis in SPC mod.

LOCAL_INLINE void **SENT_HWA_DisableChannelCalibrationDiag** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel calibration diagnosis in SPC mod.

LOCAL_INLINE void **SENT_HWA_SetChannelSPCTriggerMethod** (SENT_Type *const pSent, uint8_t u8Channel, **SENT_SPCTriggerType** eTrigger)

Set the SPC pulse trigger method.

LOCAL_INLINE void **SENT_HWA_SetChannelSPCPulseDelay** (SENT_Type *const pSent, uint8_t u8Channel, uint8_t u8Delay)

Set the delay between SPC-trigger assert and finally generate a SPC pulse.

LOCAL_INLINE void **SENT_HWA_SetChannelSPCTickBase** (SENT_Type *const pSent, uint8_t u8Channel, **SENT_SPCTickType** eTick)

Set the delay between SPC-trigger assert and finally generate a SPC pulse.

LOCAL_INLINE void **SENT_HWA_StartChannelSPC** (SENT_Type *const pSent, uint8_t u8Channel)
trigger the SENT to generate a SPC pulse by software

LOCAL_INLINE void **SENT_HWA_SetChannelSPCPulseWidth** (SENT_Type *const pSent, uint8_t u8Channel, uint8_t u8Width)
Set the width of the SPC pulse.

LOCAL_INLINE void **SENT_HWA_SetChannelIdleCount** (SENT_Type *const pSent, uint8_t u8Channel, uint8_t u8Count)
Set the idle count determines how long the bus idle flag will assert when bus is idle.

LOCAL_INLINE bool **SENT_HWA_GetChanneSPCBusyFlag** (const SENT_Type *const pSent, uint8_t u8Channel)
Get the flag indicating the spc is busy.

Detailed Description

Enumeration Type Documentation

enum **SENT_NibbleDataModeType**

sent running status

Enumerator:

SENT_NIBBLE_ DATA_MODE_A	Frame format A
SENT_NIBBLE_ DATA_MODE_H 1	Frame format H1

SENT_NIBBLE_ DATA_MODE_H 2	Frame format H2
SENT_NIBBLE_ DATA_MODE_H 3	Frame format H3
SENT_NIBBLE_ DATA_MODE_H 4	Frame format H4
SENT_NIBBLE_ DATA_MODE_H 5	Frame format H5
SENT_NIBBLE_ DATA_MODE_H 6	Frame format H6
SENT_NIBBLE_ DATA_MODE_H 7	Frame format H7

enum SENT_SlowMessageType

sent slow message type

Enumerator:

SENT_SLOW_M ESSAGE_SHORT	short serial data message
SENT_SLOW_M ESSAGE_ENHAN CE_12DATA_8ID	enhanced serial data message with 12-bit data and 8-bit ID
SENT_SLOW_M ESSAGE_ENHAN CE_16DATA_4ID	enhanced serial data message with 16-bit data and 4-bit ID

enum SENT_SPCTickType

sent SPC tick base

Enumerator:

SENT_SPC_TICK _PRE_RECEIVED _MESSAGE	Previous received message tick base
SENT_SPC_TICK _CONFIG	SENT configured tick base

enum SENT_SPCTriggerType

sent SPC trigger type

Enumerator:

SENT_SPC_SW_ TRIGGER	SPC pulse triggered by software method
SENT_SPC_HW_ TRIGGER	SPC pulse triggered by external trigger

Function Documentation

LOCAL_INLINE void SENT_HWA_ClearChannelReceivedEdgeNumber (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Clear the number of received edges.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_ClearChannelReceivedFrameNumber (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Clear the number of received frames.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_ClearChannelStatus (SENT_Type *const *pSent*, uint8_t *u8Channel*, uint32_t *u32W1CBit*)

Clear Channel status.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>u32W1CBit</i>	the status bits need to be clear.

LOCAL_INLINE void SENT_HWA_ClearFastMessageDMAUnderflowFlag (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Clear Channel fast message DMA underflow flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_ClearFastMessageFIFOOverflowFlag (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Clear Channel fast message FIFO overflow flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_ClearFastMessageReadyFlag (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Clear Channel fast message ready flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_ClearSlowMessageReadyFlag (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Clear Channel slow message ready flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_ClearSlowMessageUnderflowFlag (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Clear Channel slow message dma read underflow flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelAllErrorInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel all error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelAltCRC (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel Alternative 4-bit CRC algorithm.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelCalibrationDiag (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel calibration diagnosis in SPC mod.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelCalValid20To25 (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel calibration valid range from 20% tp 25%.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelCalValidDiagnostic (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel calibration valid diagnostic.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelCompensate (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel Compensate.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelFastMessageAugmentation (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel fast message CRC with Augmentation.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelFastMessageCRCCheck (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel Fast message crc check.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelFastMessageCRCWithSC (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel Fast message crc with S&C.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelFastMessageDataChange (SENT_Type *const *pSent*,
uint8_t *u8Channel*)

Disable SENT channel Fast message data change.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelFastMessageDmaRequest (SENT_Type *const *pSent*,
uint8_t *u8Channel*)

Disable SENT channel fast message DMA request.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelFastMessageInterrupt (SENT_Type *const *pSent*, uint8_t
u8Channel)

Disable SENT channel Fast message interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelFIFO (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel receive function.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelPausePulse (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel Successive calibration pulses diagnostic option1.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelReceive (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel receive.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelSlowMessageAugmentation (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel slow message CRC with Augmentation.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableChannelSlowMessageDmaRequest (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Disable SENT channel slow message DMA request.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_DisableChannelSlowMessageInterrupt (SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Disable SENT channel slow message interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_DisableChannelSPCMode (SENT_Type *const *pSent*, uint8_t
u8Channel)**

Disable SENT channel SPC mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_DisableChannelSPCOption1 (SENT_Type *const *pSent*, uint8_t
u8Channel)**

Disable SENT channel Pause pulse.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_DisableDataOverflowFlagFastClear (SENT_Type *const *pSent*)

Disable Global SENT debug mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_DisableDebugMode (SENT_Type *const *pSent*)

Disable SENT debug mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_DisableFastMessagedmaReadUnderflowInterrupt (SENT_Type *const *pSent*)

Disable SENT Fast message read underflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_DisableFastMessageFIFOOverflowInterrupt (SENT_Type *const *pSent*)

Disable SENT Fast message FIFO overflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_DisableGlobal (SENT_Type *const *pSent*)

Disable Global SENT debug mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_DisableSlowMessagedmaReadUnderflowInterrupt (SENT_Type *const *pSent*)

Disable SENT slow message read underflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_EnableChannelAllErrorInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel all error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelAltCRC (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel Alternative 4-bit CRC algorithm.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelBusIdleInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel idle flag interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelCALDiagInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel Calibration diagnosis over 25% error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelCalERRInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable Sync/calibration pulse difference exceed 1.5625% error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelCalFailInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel Calibration 20% pass 25% fail interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelCalibrationDiag (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel calibration diagnosis in SPC mod.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelCalResyncInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel sync/calibration pulse error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelCalValid20To25 (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel calibration valid range from 20% tp 25%.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelCalValidDiagnostic (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel calibration valid diagnostic.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelCompensate (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel Compensate.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelEdgeERRInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel Falling edge number error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelFastMessageAugmentation (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel fast message CRC with Augmentation.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelFastMessageCRCCheck (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel Fast message crc check.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_EnableChannelFastMessageCRCWithSC (SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Enable SENT channel Fast message crc with S&C.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_EnableChannelFastMessageDataChange (SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Enable SENT channel Fast message data change.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_EnableChannelFastMessageDmaRequest (SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Enable SENT channel fast message DMA request.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_EnableChannelFastMessageInterrupt (SENT_Type *const *pSent*, uint8_t
u8Channel)**

Enable SENT channel Fast message interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

<i>u8Channel</i>	the index of sent channel
------------------	---------------------------

LOCAL_INLINE void SENT_HWA_EnableChannelFCRCInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel Fast message CRC check error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelFIFO (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel FIFO function.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelFOVFLInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel fast message overflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelNibbleErrorInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel Nibble value error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelPausePulse (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel Pause pulse.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelPPDiagInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel Previous pulse diagnosis error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelReceive (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel receive.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelSCRCInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel slow message CRC check error interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

**LOCAL_INLINE void SENT_HWA_EnableChannelSlowMessageAugmentation (SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Enable SENT channel slow message CRC with Augmentation.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_EnableChannelSlowMessageDmaRequest (SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Enable SENT channel slow message DMA request.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

**LOCAL_INLINE void SENT_HWA_EnableChannelSlowMessageInterrupt (SENT_Type *const *pSent*, uint8_t
u8Channel)**

Enable SENT channel slow message interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelSOVFLInterrupt (SENT_Type *const *pSent*, uint8_t *u8Channel*, bool *bEn*)

Enable/disable SENT channel Slow message overflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>bEn</i>	Enable or disable the interrupt

LOCAL_INLINE void SENT_HWA_EnableChannelSPCMode (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel SPC mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableChannelSPCOption1 (SENT_Type *const *pSent*, uint8_t *u8Channel*)

Enable SENT channel Successive calibration pulses diagnostic option1.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE void SENT_HWA_EnableDataOverflowFlagFastClear (SENT_Type *const *pSent*)

Enable Global SENT debug mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_EnableDebugMode (SENT_Type *const *pSent*)

Enable SENT debug mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_EnableFastMessagedmaReadUnderflowInterrupt (SENT_Type *const *pSent*)

Enable SENT Fast message read underflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_EnableFastMessageFIFOOverflowInterrupt (SENT_Type *const *pSent*)

Enable SENT Fast message FIFO overflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_EnableGlobal (SENT_Type *const *pSent*)

Enable Global SENT debug mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE void SENT_HWA_EnableSlowMessagedmaReadUnderflowInterrupt (SENT_Type *const *pSent*)

Enable SENT slow message read underflow interrupt.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

LOCAL_INLINE uint8_t SENT_HWA_GetChannelBit2CRC (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel slow message bit2 with CRC.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint8_t the corresponding channel slow message CRC value

LOCAL_INLINE uint16_t SENT_HWA_GetChannelBit2DATA (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel slow message bit2 with DATA.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint16_t the corresponding channel slow message DATA value

LOCAL_INLINE uint8_t SENT_HWA_GetChannelBit3EnhancedID3_0_OR_DATA15_12 (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel enhanced serial message id bit 3 to bit0 or data bit 15 to id bit 12.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint8_t the corresponding channel enhanced serial message id bit 3 to bit0 or data bit 15 to id bit 12

LOCAL_INLINE uint8_t SENT_HWA_GetChannelBit3EnhancedID7_4_OR_ID3_0 (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel enhanced serial message id bit 7 to bit4 or id bit 3 to id bit 0.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint8_t the corresponding channel enhanced serial message id bit 7 to bit4 or id bit 3 to id bit 0

LOCAL_INLINE uint16_t SENT_HWA_GetChannelCompensatePreScaler (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel Prescaler value after compensate.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

Returns

uint8_t the related Channel Prescaler value after compensate

LOCAL_INLINE uint32_t SENT_HWA_GetChannelDataNibble (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel fast message data nibble.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint32_t the corresponding channel data nibble value

**LOCAL_INLINE uint8_t SENT_HWA_GetChannelFastMessageCRCNibble (const SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Get Channel fast message crc nibble.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint8_t the corresponding channel crc nibble value

**LOCAL_INLINE uint8_t SENT_HWA_GetChannelFastMessageStatusNibble (const SENT_Type *const
pSent, uint8_t *u8Channel*)**

Get Channel fast message status communication nibble.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint8_t the corresponding channel status communication nibble value

**LOCAL_INLINE uint32_t SENT_HWA_GetChannelFastMessageTimeStamp (const SENT_Type *const
pSent, uint8_t *u8Channel*)**

Get Channel fast message status timestamp.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint32_t the corresponding channel timestamp value

LOCAL_INLINE uint8_t SENT_HWA_GetChannelNibbleNumber (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get SENT channel data nibble number.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>uint8_t</i>	the config data number

LOCAL_INLINE bool SENT_HWA_GetChannelReceive (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get SENT channel receive.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

LOCAL_INLINE uint32_t SENT_HWA_GetChannelSlowMessageTimeStamp (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel slow message status timestamp.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint32_t the corresponding channel timestamp value

LOCAL_INLINE SENT_SlowMessageType SENT_HWA_GetChannelSlowMessageType (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel slow message type.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

SENT_SlowMessageType the corresponding channel slow message type

LOCAL_INLINE uint32_t SENT_HWA_GetChannelStatus (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel status.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

Returns

uint32_t the related Channel status

LOCAL_INLINE uint32_t SENT_HWA_GetChanneReceivedEdgeNumber (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel number of received edges.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint32_t the number of received edges

LOCAL_INLINE uint32_t SENT_HWA_GetChanneReceivedFrameNumber (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel number of received frames.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

uint32_t the number of received frames

LOCAL_INLINE bool SENT_HWA_GetChanneSPCBusyFlag (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get the flag indicating the spc is busy.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel(0-3)

Returns

bool the flag indicating the spc is busy

LOCAL_INLINE bool SENT_HWA_GetFastMessageDMAUnderflowFlag (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel fast message DMA underflow flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

Returns

bool true: the related channel fast message DMA read is underflow

LOCAL_INLINE bool SENT_HWA_GetFastMessageFIFOOverflowFlag (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel fast message FIFO overflow flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

Returns

bool true: the related channel fast message FIFO is overflow

LOCAL_INLINE bool SENT_HWA_GetFastMessageReadyFlag (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel fast message ready flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

Returns

bool true: the related channel fast message is ready

LOCAL_INLINE bool SENT_HWA_GetSlowMessageReadyFlag (const SENT_Type *const *pSent*, uint8_t *u8Channel*)

Get Channel slow message ready flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

Returns

bool true: the related channel slow message is ready

**LOCAL_INLINE bool SENT_HWA_GetSlowMessageUnderflowFlag (const SENT_Type *const *pSent*,
uint8_t *u8Channel*)**

Get Channel slow message dma read underflow flag.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

Returns

bool true: the related channel slow message dma is underflow

**LOCAL_INLINE void SENT_HWA_SetChannelDigitalFilterCount (SENT_Type *const *pSent*, uint8_t
u8Channel, uint8_t *u8Count*)**

Set SENT channel digital filter count.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>u8Count</i>	the count of channel digital filter

**LOCAL_INLINE void SENT_HWA_SetChannelIdleCount (SENT_Type *const *pSent*, uint8_t *u8Channel*,
uint8_t *u8Count*)**

Set the idle count determines how long the bus idle flag will assert when bus is idle.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>u8Count</i>	the idle count

**LOCAL_INLINE void SENT_HWA_SetChannelNibbleDataMode (SENT_Type *const *pSent*, uint8_t
u8Channel, SENT_NibbleDataModeType *eMode*)**

Set SENT channel nibble data mode.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>eMode</i>	the nibble data mode

LOCAL_INLINE void SENT_HWA_SetChannelNibbleNumber (SENT_Type *const *pSent*, uint8_t *u8Channel*, uint8_t *u8Number*)

Set SENT channel data nibble number.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>u8Number</i>	the received data number

LOCAL_INLINE void SENT_HWA_SetChannelPreScaler (SENT_Type *const *pSent*, uint8_t *u8Channel*, uint16_t *u16PreScaler*)

Set SENT channel pre-scaler.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>u16PreScaler</i>	the related Channel Prescaler value

LOCAL_INLINE void SENT_HWA_SetChannelSPCPulseDelay (SENT_Type *const *pSent*, uint8_t *u8Channel*, uint8_t *u8Delay*)

Set the delay between SPC-trigger assert and finally generate a SPC pulse.

Parameters

<i>pSent</i>	the base address of the SENT instance
--------------	---------------------------------------

<i>u8Channel</i>	the index of sent channel
<i>eTrigger</i>	the delay value

LOCAL_INLINE void SENT_HWA_SetChannelSPCPulseWidth (SENT_Type *const *pSent*, uint8_t *u8Channel*, uint8_t *u8Width*)

Set the width of the SPC pulse.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>u8Width</i>	the width of the SPC pulse.

LOCAL_INLINE void SENT_HWA_SetChannelSPCTickBase (SENT_Type *const *pSent*, uint8_t *u8Channel*, SENT_SPCTickType *eTick*)

Set the delay between SPC-trigger assert and finally generate a SPC pulse.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>eTick</i>	the selected tick base of SPC

LOCAL_INLINE void SENT_HWA_SetChannelSPCTriggerMethod (SENT_Type *const *pSent*, uint8_t *u8Channel*, SENT_SPCTriggerType *eTrigger*)

Set the SPC pulse trigger method.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel
<i>eTrigger</i>	the SPC pulse trigger method

LOCAL_INLINE void SENT_HWA_SetDMAWaterMark (SENT_Type *const *pSent*, uint8_t *u8WaterMark*)

Set SENT Global DMA watermark value.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8WaterMark</i>	the value of the dma watermark.

LOCAL_INLINE void SENT_HWA_SetGlobalPreScaler (SENT_Type *const *pSent*, uint8_t *u8PreScaler*)

Set SENT Global pre-scaler value.

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8PreScaler</i>	the value of the PreScaler.

LOCAL_INLINE void SENT_HWA_StartChannelSPC (SENT_Type *const *pSent*, uint8_t *u8Channel*)

trigger the SENT to generate a SPC pulse by software

Parameters

<i>pSent</i>	the base address of the SENT instance
<i>u8Channel</i>	the index of sent channel

HwA_smc

Collaboration diagram for HwA_smc:Enumerations

enum **SMC_LpwModeCtrlType** { **SMC_STOP_MODE** = 0U, **SMC_STANDBY_MODE** = 4U }

SMC stop mode control.

enum **SMC_StandbyModeType** { **SMC_CFG_STANDBY_0** = 0U, **SMC_CFG_STANDBY_1** = 1U, **SMC_CFG_STANDBY_2** = 2U, **SMC_CFG_STANDBY_3** = 3U }

SMC standby mode.

enum **SMC_ClockOutSrcType** { **SMC_CLKOUT_SCG_CLKOUT** = 0U, **SMC_CLKOUT_FOSC_DIVM_CLK** = 2U, **SMC_CLKOUT_SLOW_CLK** = 3U, **SMC_CLKOUT_SIRC_DIVM_CLK** = 4U, **SMC_CLKOUT_PLL1_DIVM_CLK** = 5U, **SMC_CLKOUT_FIRC_DIVM_CLK** = 6U, **SMC_CLKOUT_CORE_CLK** = 7U, **SMC_CLKOUT_PLL0_DIVM_CLK**

= 8U, **SMC_CLKOUT_BUS_CLK** = 9U, **SMC_CLKOUT_SIRC_128K_CLK** = 10U, **SMC_CLKOUT_AON_CLK** = 12U, **SMC_CLKOUT_RTC_CLK** = 14U }

SMC clock out source type.

enum **SMC_ClockOutDivType** { **SMC_CLKOUT_DIV_BY1** = 0U, **SMC_CLKOUT_DIV_BY2** = 1U, **SMC_CLKOUT_DIV_BY3** = 2U, **SMC_CLKOUT_DIV_BY4** = 3U, **SMC_CLKOUT_DIV_BY5** = 4U, **SMC_CLKOUT_DIV_BY6** = 5U, **SMC_CLKOUT_DIV_BY7** = 6U, **SMC_CLKOUT_DIV_BY8** = 7U }

SMC clock out divide ratio type.

enum **SMC_AON32KClkSrcType** { **SMC_AON32K_SIRCDIV_32K_CLK** = 1U, **SMC_AON32K_SOSC32K_CLK** = 2U, **SMC_AON32K_SIRC32K_CLK** = 3U }

Data type for SMC_AONCLKSR[32KAONCLKSR], set AON32KCLK source clock.

enum **SMC_RTCClkSrcType** { **SMC_RTC_FOSCDIVL_CLK** = 0U, **SMC_RTC_SIRCDIV_32K_CLK** = 1U, **SMC_RTC_SOSC_CLK** = 2U, **SMC_RTC_SIRC32K_CLK** = 3U }

Data type for SMC_RTCCLKSEL[RTCCLKSEL], set RTCCLK source clock.

enum **SMC_AONClkSrcType** { **SMC_AON_SIRCDIV_128K_CLK** = 0U, **SMC_AON_SIRC32K_CLK** = 1U, **SMC_AON_SIRCDIV_32K_CLK** = 2U, **SMC_AON_SIRC32_1K_CLK** = 3U }

Data type for SMC_AONCLKSR[AONCLKSR], set AONCLK source clock.

Functions

LOCAL_INLINE void **SMC_HWA_ClearStopModeCtrl** (void)

Clear stop mode control value.

LOCAL_INLINE void **SMC_HWA_SetStopModeCtrl** (**SMC_LpwModeCtrlType** eMode)

Set stop mode control.

LOCAL_INLINE void **SMC_HWA_ClearStandbyMode** (void)

Clear standby mode.

LOCAL_INLINE void **SMC_HWA_SetStandbyMode** (**SMC_StandbyModeType** eMode)

Set standby mode.

Detailed Description

Enumeration Type Documentation

enum SMC_AON32KClkSrcType

Data type for SMC_AONCLKSR[32KAONCLKSR], set AON32KCLK source clock.

Enumerator:

SMC_AON32K_SIRCDIV_32K_CLK	SMC_AONCLKSR[32KAONCLKSR], SIRCDIV_32K
SMC_AON32K_SOSC32K_CLK	SMC_AONCLKSR[32KAONCLKSR], SOSC32K
SMC_AON32K_SIRC32K_CLK	SMC_AONCLKSR[32KAONCLKSR], SIRC32K

enum SMC_AONClkSrcType

Data type for SMC_AONCLKSR[AONCLKSR], set AONCLK source clock.

Enumerator:

SMC_AON_SIRCDIV_128K_CLK	SMC_AONCLKSR[AONCLKSR], SIRCDIV_128K
SMC_AON_SIRC32K_CLK	SMC_AONCLKSR[AONCLKSR], SIRC32K
SMC_AON_SIRCDIV_32K_CLK	SMC_AONCLKSR[AONCLKSR], SIRCDIV_32K
SMC_AON_SIRC32_1K_CLK	SMC_AONCLKSR[AONCLKSR], SIRC32_1K

enum SMC_ClockOutDivType

SMC clock out divide ratio type.

Enumerator:

SMC_CLKOUT_ DIV_BY1	Divided by 1
SMC_CLKOUT_ DIV_BY2	Divided by 2
SMC_CLKOUT_ DIV_BY3	Divided by 3
SMC_CLKOUT_ DIV_BY4	Divided by 4
SMC_CLKOUT_ DIV_BY5	Divided by 5
SMC_CLKOUT_ DIV_BY6	Divided by 6
SMC_CLKOUT_ DIV_BY7	Divided by 7
SMC_CLKOUT_ DIV_BY8	Divided by 8

enum SMC_ClockOutSrcType

SMC clock out source type.

Enumerator:

SMC_CLKOUT_S CG_CLKOUT	SCG CLKOUT
---------------------------	------------

SMC_CLKOUT_F OSC_DIVM_CLK	FOSC DIVM CLK
SMC_CLKOUT_S LOW_CLK	SLOW CLK
SMC_CLKOUT_S IRC_DIVM_CLK	SIRC DIVM CLK
SMC_CLKOUT_P LL1_DIVM_CLK	PLL1 DIVM CLK
SMC_CLKOUT_F IRC_DIVM_CLK	FIRC DIVM CLK
SMC_CLKOUT_C CORE_CLK	CORE CLK
SMC_CLKOUT_P LLO_DIVM_CLK	PLL0 DIVM CLK
SMC_CLKOUT_B BUS_CLK	BUS CLK
SMC_CLKOUT_S IRC_128K_CLK	SIRC 128K CLK
SMC_CLKOUT_A ON_CLK	AON CLK
SMC_CLKOUT_R TC_CLK	AON32K CLK

enum SMC_LpwModeCtrlType

SMC stop mode control.

Enumerator:

SMC_STOP_MO DE	
SMC_STANDBY_ MODE	

enum SMC_RTCClkSrcType

Data type for SMC_RTCCLKSEL[RTCCLKSEL], set RTCCLK source clock.

Enumerator:

SMC_RTC_FOSC DIVL_CLK	SMC_RTCCLKSEL[RTCCLKSEL], FOSC_DIVL
SMC_RTC_SIRC DIV_32K_CLK	SMC_RTCCLKSEL[RTCCLKSEL], SIRC_DIV
SMC_RTC_SOSC _CLK	SMC_RTCCLKSEL[RTCCLKSEL], SOSC
SMC_RTC_SIRC 32K_CLK	SMC_RTCCLKSEL[RTCCLKSEL], SIRC_32K

enum SMC_StandbyModeType

SMC standby mode.

Enumerator:

SMC_CFG_STAN DBY_0	
SMC_CFG_STAN DBY_1	
SMC_CFG_STAN DBY_2	
SMC_CFG_STAN DBY_3	

Function Documentation

LOCAL_INLINE void SMC_HWA_ClearStandbyMode (void)

Clear standby mode.

LOCAL_INLINE void SMC_HWA_ClearStopModeCtrl (void)

Clear stop mode control value.

LOCAL_INLINE void SMC_HWA_SetStandbyMode (SMC_StandbyModeType *eMode*)

Set standby mode.

Parameters

<i>eMode</i>	Standby mode type
--------------	-------------------

LOCAL_INLINE void SMC_HWA_SetStopModeCtrl (SMC_LpwModeCtrlType *eMode*)

Set stop mode control.

Parameters

<i>eMode</i>	Stop mode control type
--------------	------------------------

HwA_tmu

Collaboration diagram for HwA_tmu:Enumerations

enum **TMU_LockType** { **TMU_TF_TV_LOCK** = 0U, **TMU_TF_TV_UNLOCK** = 1U }

Select the TF_CTRL and TV_CTRL register is lock or not.

enum **TMU_HysteresisType** { **TMU_TF_HYSOFF_ON** = 0U, **TMU_TF_HYSOFF_OFF** = 1U }

Select the Flag-based temperature sensor hysteresis control.

enum **TMU_bypassType** { **TMU_TF_FILT_BYP_BYPASSED** = 0U, **TMU_TF_FILT_BYP_ENABLED** = 1U }

Select the Flag-based temperature sensor filter control.

Functions

LOCAL_INLINE **TMU_LockType** **TMU_HWA_GetTemperatureLockStatus** (const TMU_Type *const pTmu)

Get the status of whether the temperature sensor register is locked.

LOCAL_INLINE void **TMU_HWA_SetTemperatureLockStatus** (TMU_Type *const pTmu, **TMU_LockType** eLockStatus)

Set the status of whether the temperature sensor register is locked or not.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperature150InterruptFlag** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor over 150 Celsius interrupt flag.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperature150InterruptFlag** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor over 150 Celsius interrupt flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperature125InterruptFlag** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor over 125 Celsius interrupt flag.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperature125InterruptFlag** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor over 125 Celsius interrupt flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureReadyInterruptFlag** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor ready interrupt flag.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureReadyInterruptFlag** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor over ready interrupt flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureHysteresisStatus** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor hysteresis control status.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureHysteresisStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor hysteresis control status.

LOCAL_INLINE uint8_t **TMU_HWA_GetFlagTemperatureCounter** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor startup counter.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureCounter** (TMU_Type *const pTmu, uint8_t u8Counter)

Set the Flag-based temperature sensor startup counter.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureFilterBypassStatus** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor filter bypass control status.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureFilterBypassStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor filter bypass control status.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureEnableStatus** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor enable status.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureEnableStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor enable status.

LOCAL_INLINE bool **TMU_HWA_Get150Status** (TMU_Type *const pTmu)

Check whether the temperature is over 150 Celsius.

LOCAL_INLINE bool **TMU_HWA_Get125Status** (TMU_Type *const pTmu)

Check whether the temperature is over 125 Celsius.

LOCAL_INLINE bool **TMU_HWA_Get150Flag** (TMU_Type *const pTmu)

Check whether the flag for temperature over 150 Celsius is set.

LOCAL_INLINE void **TMU_HWA_Clear150Flag** (TMU_Type *const pTmu)

Clear the temperature over 150 Celsius flag.

LOCAL_INLINE bool **TMU_HWA_Get125Flag** (TMU_Type *const pTmu)

Check whether the flag for temperature over 125 Celsius is set.

LOCAL_INLINE void **TMU_HWA_Clear125Flag** (TMU_Type *const pTmu)

Clear the temperature over 125 Celsius flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureReady** (TMU_Type *const pTmu)

Check whether the Flag-based temperature sensor is ready.

LOCAL_INLINE void **TMU_HWA_ClearFlagTemperatureReady** (TMU_Type *const pTmu)

Clear the Flag-based temperature sensor ready flag.

LOCAL_INLINE bool **TMU_HWA_GetVoltageTemperatureReadyInterruptFlag** (const TMU_Type *const pTmu)

Get the Voltage-based temperature sensor ready interrupt flag.

LOCAL_INLINE void **TMU_HWA_SetVoltageTemperatureReadyInterruptFlag** (TMU_Type *const pTmu, bool bEnable)

Set the Voltage-based temperature sensor over ready interrupt flag.

LOCAL_INLINE uint8_t **TMU_HWA_GetVoltageTemperatureCounter** (const TMU_Type *const pTmu)

Get the Voltage-based temperature sensor startup counter.

LOCAL_INLINE void **TMU_HWA_SetVoltageTemperatureCounter** (TMU_Type *const pTmu, uint8_t u8Counter)

Set the Voltage-based temperature sensor startup counter.

LOCAL_INLINE bool **TMU_HWA_GetVoltageTemperatureEnableStatus** (const TMU_Type *const pTmu)

Get the Voltage-based temperature sensor enable status.

LOCAL_INLINE void **TMU_HWA_SetVoltageTemperatureEnableStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Voltage-based temperature sensor enable status.

LOCAL_INLINE bool **TMU_HWA_GetVoltageTemperatureReady** (TMU_Type *const pTmu)

Check whether the Voltage-based temperature sensor is ready.

LOCAL_INLINE void **TMU_HWA_ClearVoltageTemperatureReady** (TMU_Type *const pTmu)

Clear the Voltage-based temperature sensor ready flag.

LOCAL_INLINE uint32_t **TMU_HWA_GetFlagTempCtrl** (const TMU_Type *const pTmu)

Get the TMU_TF_CTRL config.

LOCAL_INLINE void **TMU_HWA_SetFlagTempCtrl** (TMU_Type *const pTmu, uint32_t u32Config)

Set the TMU_TF_CTRL config.

LOCAL_INLINE uint32_t **TMU_HWA_GetVoltageTempCtrl** (const TMU_Type *const pTmu)

Get the TMU_TV_CTRL config.

LOCAL_INLINE void **TMU_HWA_SetVoltageTempCtrl** (TMU_Type *const pTmu, uint32_t u32Config)

Set the TMU_TV_CTRL config.

Detailed Description

Enumeration Type Documentation

enum TMU_bypassType

Select the Flag-based temperature sensor filter control.

Enumerator:

TMU_TF_FILT_B YP_BYPASSED	Flag-based temperature sensor filter is bypassed
TMU_TF_FILT_B YP_ENABLED	Flag-based temperature sensor filter is enabled

enum TMU_HysteresisType

Select the Flag-based temperature sensor hysteresis control.

Enumerator:

TMU_TF_HYSOF F_ON	Flag-based temperature sensor hysteresis is on
TMU_TF_HYSOF F_OFF	Flag-based temperature sensor hysteresis is off

enum TMU_LockType

Select the TF_CTRL and TV_CTRL register is lock or not.

Enumerator:

TMU_TF_TV_LO CK	TF_CTRL and TV_CTRL register is lock
TMU_TF_TV_UN LOCK	TF_CTRL and TV_CTRL register is unlock

Function Documentation

LOCAL_INLINE void TMU_HWA_Clear125Flag (TMU_Type *const *pTmu*)

Clear the temperature over 125 Celsius flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

LOCAL_INLINE void TMU_HWA_Clear150Flag (TMU_Type *const *pTmu*)

Clear the temperature over 150 Celsius flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

LOCAL_INLINE void TMU_HWA_ClearFlagTemperatureReady (TMU_Type *const *pTmu*)

Clear the Flag-based temperature sensor ready flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

LOCAL_INLINE void TMU_HWA_ClearVoltageTemperatureReady (TMU_Type *const *pTmu*)

Clear the Voltage-based temperature sensor ready flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

LOCAL_INLINE bool TMU_HWA_Get125Flag (TMU_Type *const *pTmu*)

Check whether the flag for temperature over 125 Celsius is set.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true the temperature has exceeded 125 Celsius since last time W1C

false the temperature has not exceeded 125 Celsius since TF is ready

LOCAL_INLINE bool TMU_HWA_Get125Status (TMU_Type *const *pTmu*)

Check whether the temperature is over 125 Celsius.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true the temperature is over 125 Celsius

false the temperature is not over 125 Celsius

LOCAL_INLINE bool TMU_HWA_Get150Flag (TMU_Type *const *pTmu*)

Check whether the flag for temperature over 150 Celsius is set.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true the temperature has exceeded 150 Celsius since last time W1C

false the temperature has not exceeded 150 Celsius since TF is ready

LOCAL_INLINE bool TMU_HWA_Get150Status (TMU_Type *const *pTmu*)

Check whether the temperature is over 150 Celsius.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true the temperature is over 150 Celsius

false the temperature is not over 150 Celsius

LOCAL_INLINE uint32_t TMU_HWA_GetFlagTempCtrl (const TMU_Type *const *pTmu*)

Get the TMU_TF_CTRL config.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

uint32_t the TMU_TF_CTRL config

LOCAL_INLINE bool TMU_HWA_GetFlagTemperature125InterruptFlag (const TMU_Type *const *pTmu*)

Get the Flag-based temperature sensor over 125 Celsius interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature over 125 Celsius interrupt is enabled
false Temperature over 125 Celsius interrupt is disabled

LOCAL_INLINE bool TMU_HWA_GetFlagTemperature150InterruptFlag (const TMU_Type *const *pTmu*)

Get the Flag-based temperature sensor over 150 Celsius interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature over 150 Celsius interrupt is enabled
false Temperature over 150 Celsius interrupt is disabled

LOCAL_INLINE uint8_t TMU_HWA_GetFlagTemperatureCounter (const TMU_Type *const *pTmu*)

Get the Flag-based temperature sensor startup counter.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

u32TmpVal count of the startup

LOCAL_INLINE bool TMU_HWA_GetFlagTemperatureEnableStatus (const TMU_Type *const *pTmu*)

Get the Flag-based temperature sensor enable status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature sensor is on

false Temperature sensor is off

LOCAL_INLINE bool TMU_HWA_GetFlagTemperatureFilterBypassStatus (const TMU_Type *const *pTmu*)

Get the Flag-based temperature sensor filter bypass control status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature sensor Filter is enabled

false Temperature sensor Filter is bypassed

LOCAL_INLINE bool TMU_HWA_GetFlagTemperatureHysteresisStatus (const TMU_Type *const *pTmu*)

Get the Flag-based temperature sensor hysteresis control status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature sensor Hysteresis is off

false Temperature sensor Hysteresis is on

LOCAL_INLINE bool TMU_HWA_GetFlagTemperatureReady (TMU_Type *const *pTmu*)

Check whether the Flag-based temperature sensor is ready.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true the Flag-based temperature sensor is ready

false the Flag-based temperature sensor is not ready

LOCAL_INLINE bool TMU_HWA_GetFlagTemperatureReadyInterruptFlag (const TMU_Type *const *pTmu*)

Get the Flag-based temperature sensor ready interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature sensor ready interrupt is enabled

false Temperature sensor ready interrupt is disabled

LOCAL_INLINE TMU_LockType TMU_HWA_GetTemperatureLockStatus (const TMU_Type *const *pTmu*)

Get the status of whether the temperature sensor register is locked.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature register is locked(CTRL)

false Temperature register is unlocked(CTRL)

LOCAL_INLINE uint32_t TMU_HWA_GetVoltageTempCtrl (const TMU_Type *const *pTmu*)

Get the TMU_TV_CTRL config.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

uint32_t the TMU_TV_CTRL config

LOCAL_INLINE uint8_t TMU_HWA_GetVoltageTemperatureCounter (const TMU_Type *const *pTmu*)

Get the Voltage-based temperature sensor startup counter.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

u32TmpVal count of the startup

LOCAL_INLINE bool TMU_HWA_GetVoltageTemperatureEnableStatus (const TMU_Type *const *pTmu*)

Get the Voltage-based temperature sensor enable status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature sensor is on
false Temperature sensor is off

LOCAL_INLINE bool TMU_HWA_GetVoltageTemperatureReady (TMU_Type *const *pTmu*)

Check whether the Voltage-based temperature sensor is ready.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true the Voltage-based temperature sensor is ready

false the Voltage-based temperature sensor is not ready

LOCAL_INLINE bool TMU_HWA_GetVoltageTemperatureReadyInterruptFlag (const TMU_Type *const *pTmu*)

Get the Vlotage-based temperature sensor ready interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

Returns

true Temperature sensor ready interrupt is enabled

false Temperature sensor ready interrupt is disabled

LOCAL_INLINE void TMU_HWA_SetFlagTempCtrl (TMU_Type *const *pTmu*, uint32_t *u32Config*)

Set the TMU_TF_CTRL config.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>u32Config</i>	the TMU_TF_CTRL config

LOCAL_INLINE void TMU_HWA_SetFlagTemperature125InterruptFlag (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Flag-based temperature sensor over 125 Celsius interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>bEnable</i>	whether to enable the temperature over 125 Celsius interrupt

LOCAL_INLINE void TMU_HWA_SetFlagTemperature150InterruptFlag (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Flag-based temperature sensor over 150 Celsius interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>bEnable</i>	whether to enable the temperature over 150 Celsius interrupt

LOCAL_INLINE void TMU_HWA_SetFlagTemperatureCounter (TMU_Type *const *pTmu*, uint8_t *u8Counter*)

Set the Flag-based temperature sensor startup counter.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>u8Counter</i>	the startup counter

LOCAL_INLINE void TMU_HWA_SetFlagTemperatureEnableStatus (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Flag-based temperature sensor enable status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>bEnable</i>	whether the temperature sensor is on(1) or off(0)

LOCAL_INLINE void TMU_HWA_SetFlagTemperatureFilterBypassStatus (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Flag-based temperature sensor filter bypass control status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
-------------	--------------------------------------

<i>bEnable</i>	whether the temperature sensor filter is enabled(1) or bypassed(0)
----------------	--

LOCAL_INLINE void TMU_HWA_SetFlagTemperatureHysteresisStatus (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Flag-based temperature sensor hysteresis control status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>bEnable</i>	whether to off(1)/on(0) the temperature sensor hysteresis

LOCAL_INLINE void TMU_HWA_SetFlagTemperatureReadyInterruptFlag (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Flag-based temperature sensor over ready interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>bEnable</i>	whether to enable the temperature sensor ready interrupt

LOCAL_INLINE void TMU_HWA_SetTemperatureLockStatus (TMU_Type *const *pTmu*, TMU_LockType *eLockStatus*)

Set the status of whether the temperature sensor register is locked or not.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>eLockStatus</i>	Select the TF_CTRL and TV_CTRL register is lock or not

LOCAL_INLINE void TMU_HWA_SetVoltageTempCtrl (TMU_Type *const *pTmu*, uint32_t *u32Config*)

Set the TMU_TV_CTRL config.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>u32Config</i>	the TMU_TV_CTRL config

LOCAL_INLINE void TMU_HWA_SetVoltageTemperatureCounter (TMU_Type *const *pTmu*, uint8_t *u8Counter*)

Set the Voltage-based temperature sensor startup counter.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>u8Counter</i>	the startup counter

LOCAL_INLINE void TMU_HWA_SetVoltageTemperatureEnableStatus (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Voltage-based temperature sensor enable status.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>bEnable</i>	whether the temperature sensor is on(1) or off(0)

LOCAL_INLINE void TMU_HWA_SetVoltageTemperatureReadyInterruptFlag (TMU_Type *const *pTmu*, bool *bEnable*)

Set the Voltage-based temperature sensor over ready interrupt flag.

Parameters

<i>pTmu</i>	the base address of the TMU instance
<i>bEnable</i>	whether to enable the temperature sensor ready interrupt

HwA_trgsel

Collaboration diagram for HwA_trgsel:Functions

LOCAL_INLINE uint32_t **TRGSEL_HWA_GetTargetTriggerSource** (const TRGSEL_Type *const pTrgsel, uint32_t u32TargetIndex)

Get the trigger source of the selected trigger target.

LOCAL_INLINE void **TRGSEL_HWA_SetTargetTriggerSource** (TRGSEL_Type *const pTrgsel, uint32_t u32TargetIndex, uint32_t u32SourceIndex)

Set the trigger source of the selected trigger target.

LOCAL_INLINE bool **TRGSEL_HWA_GetTargetLockStatus** (const TRGSEL_Type *const pTrgsel, uint32_t u32TargetIndex)

Get whether the trigger source of the selected target is locked.

LOCAL_INLINE void **TRGSEL_HWA_LockTargetTriggerSource** (TRGSEL_Type *const pTrgsel, uint32_t u32TargetIndex)

Lock the trigger source of the selected target.

Detailed Description

Function Documentation

LOCAL_INLINE bool **TRGSEL_HWA_GetTargetLockStatus** (const TRGSEL_Type *const *pTrgsel*, uint32_t *u32TargetIndex*)

Get whether the trigger source of the selected target is locked.

Parameters

<i>pTrgsel</i>	the base address of the TrgSel instance
<i>u32TargetIndex</i>	the trigger target to get the lock status

Returns

true the trigger source of the selected target cannot be modified

false the trigger source of the selected target can be modified

**LOCAL_INLINE uint32_t TRGSEL_HWA_GetTargetTriggerSource (const TRGSEL_Type *const *pTrgsel*,
uint32_t *u32TargetIndex*)**

Get the trigger source of the selected trigger target.

Parameters

<i>pTrgsel</i>	the base address of the TrgSel instance
<i>u32TargetIndex</i>	the trigger target to get the trigger source

Returns

uint32_t the trigger source of the selected trigger target

**LOCAL_INLINE void TRGSEL_HWA_LockTargetTriggerSource (TRGSEL_Type *const *pTrgsel*, uint32_t
u32TargetIndex)**

Lock the trigger source of the selected target.

Note

The trigger target is grouped by four, so if you lock the trigger source of one target, the adjacent three trigger targets in the same register group are also be locked. So please ensure the trigger sources are not to be modified before lock the trigger target.

Parameters

<i>pTrgsel</i>	the base address of the TrgSel instance
<i>u32TargetIndex</i>	the trigger target to lock the trigger source

**LOCAL_INLINE void TRGSEL_HWA_SetTargetTriggerSource (TRGSEL_Type *const *pTrgsel*, uint32_t
u32TargetIndex, uint32_t *u32SourceIndex*)**

Set the trigger source of the selected trigger target.

Parameters

<i>pTrgsel</i>	the base address of the TrgSel instance
<i>u32TargetIndex</i>	the trigger target to set the trigger source
<i>u32SourceIndex</i>	the selected trigger source to trig the target

HwA_tstmp

Collaboration diagram for HwA_tstmp:Enumerations

```
enum TSTMP_ModChannelType { TSTMP_MODChn0 = 0U, TSTMP_MODChn1, TSTMP_MODChn2,
TSTMP_MODChn3 }
```

Tstmp modulate number.

Functions

```
LOCAL_INLINE uint64_t TSTMP_HWA_ReadTstmpValue (TSTMP_Type *pTstmp)
```

Read TSTMP value.

```
LOCAL_INLINE uint32_t TSTMP_HWA_ReadTstmpInterruptEnable (TSTMP_Type *pTstmp)
```

Read TSTMP interrupt enable bits.

```
LOCAL_INLINE uint32_t TSTMP_HWA_ReadModMatchFlag (TSTMP_Type *pTstmp)
```

Read TSTMP all MOD match flag.

```
LOCAL_INLINE void TSTMP_HWA_ClearModMatchFlag (TSTMP_Type *pTstmp, TSTMP_ModChannelType
eMod)
```

Clear TSTMP MOD match flag.

```
LOCAL_INLINE void TSTMP_HWA_SetModMatchValue (TSTMP_Type *pTstmp, TSTMP_ModChannelType
eMod, uint32_t u32ModValue)
```

Set MOD match value.

LOCAL_INLINE void **TSTMP_HWA_EnableModMatchInterrupt** (TSTMP_Type *pTstmp,
TSTMP_ModChannelType eMod)
Enable TSTMP MOD(n) match interrupt.

LOCAL_INLINE void **TSTMP_HWA_DisableModMatchInterrupt** (TSTMP_Type *pTstmp,
TSTMP_ModChannelType eMod)
Disable TSTMP MOD(n) match interrupt.

Detailed Description

Enumeration Type Documentation

enum TSTMP_ModChannelType

Tstmp modulate number.

Enumerator:

TSTMP_MODCh n0	
TSTMP_MODCh n1	
TSTMP_MODCh n2	
TSTMP_MODCh n3	

Function Documentation

LOCAL_INLINE void **TSTMP_HWA_ClearModMatchFlag** (TSTMP_Type * *pTstmp*, **TSTMP_ModChannelType** *eMod*)

Clear TSTMP MOD match flag.

Parameters

<i>pTstmp</i>	TSTMP instance
---------------	----------------

```
LOCAL_INLINE void TSTMP_HWA_DisableModMatchInterrupt (TSTMP_Type * pTstmp,  
TSTMP_ModChannelType eMod)
```

Disable TSTMP MOD(n) match interrupt.

Parameters

<i>pTstmp</i>	TSTMP instance
<i>eMod</i>	MOD number

```
LOCAL_INLINE void TSTMP_HWA_EnableModMatchInterrupt (TSTMP_Type * pTstmp,  
TSTMP_ModChannelType eMod)
```

Enable TSTMP MOD(n) match interrupt.

Parameters

<i>pTstmp</i>	TSTMP instance
<i>eMod</i>	MOD number

```
LOCAL_INLINE uint32_t TSTMP_HWA_ReadModMatchFlag (TSTMP_Type * pTstmp)
```

Read TSTMP all MOD match flag.

Parameters

<i>pTstmp</i>	TSTMP instance
---------------	----------------

Returns

TSTMP all MOD match flag

```
LOCAL_INLINE uint32_t TSTMP_HWA_ReadTstmpInterruptEnable (TSTMP_Type * pTstmp)
```


Read TSTMP interrupt enable bits.

Parameters

<i>pTstmp</i>	TSTMP instance
---------------	----------------

Returns

TSTMP interrupt enable bits

LOCAL_INLINE uint64_t TSTMP_HWA_ReadTstmpValue (TSTMP_Type * *pTstmp*)

Read TSTMP value.

Parameters

<i>pTstmp</i>	TSTMP instance
---------------	----------------

Returns

TSTMP value

LOCAL_INLINE void TSTMP_HWA_SetModMatchValue (TSTMP_Type * *pTstmp*, TSTMP_ModChannelType *eMod*, uint32_t *u32ModValue*)

Set MOD match value.

Parameters

<i>pTstmp</i>	TSTMP instance
<i>eMod</i>	MOD number
<i>u32ModValue</i>	MOD value

HwA_wdog

Collaboration diagram for HwA_wdog:Functions

LOCAL_INLINE void **WDOG_HWA_SetCs** (WDOG_Type *pWdog, uint32_t u32Cs)

Set CS register value, for Wdog working mode configuration.

LOCAL_INLINE uint32_t **WDOG_HWA_GetCs** (WDOG_Type *pWdog)

Get CS register value, for WDOG working mode configuration.

LOCAL_INLINE void **WDOG_HWA_ClearInterruptFlag** (WDOG_Type *pWdog)

Clear Wdog interrupt flag.

LOCAL_INLINE bool **WDOG_HWA_GetUnlockStatus** (WDOG_Type *pWdog)

Get WDOG unlock status, if locked, register can't be written.

LOCAL_INLINE void **WDOG_HWA_SetCounter** (WDOG_Type *pWdog, uint32_t u32Counter)

Set COUNTER register value. for Wdog unlock and refresh usage.

LOCAL_INLINE void **WDOG_HWA_SetTimeout** (WDOG_Type *pWdog, uint16_t u16Timeout)

Set TIMEOUT register value. for WDOG timeout value.

LOCAL_INLINE void **WDOG_HWA_SetWindow** (WDOG_Type *pWdog, uint16_t u16Window)

Set WINDOW register value. for windowed WDOG low threshold value.

Detailed Description

Function Documentation

LOCAL_INLINE void **WDOG_HWA_ClearInterruptFlag** (WDOG_Type * *pWdog*)

Clear Wdog interrupt flag.

Parameters

<i>WDOG_Type</i>	*pWdog. point to wdog instance base register address.
------------------	---

LOCAL_INLINE uint32_t WDOG_HWA_GetCs (WDOG_Type * *pWdog*)

Get CS register value, for WDOG working mode configuration.

Returns

uint32_t CS register value.

LOCAL_INLINE bool WDOG_HWA_GetUnlockStatus (WDOG_Type * *pWdog*)

Get WDOG unlock status, if locked, register can't be written.

Returns

bool. true as unlocked. false as locked.

LOCAL_INLINE void WDOG_HWA_SetCounter (WDOG_Type * *pWdog*, uint32_t *u32Counter*)

Set COUNTER register value. for Wdog unlock and refresh usage.

Parameters

<i>pWdog.</i>	point to wdog instance base register address.
<i>u32Counter.</i>	configured register value

LOCAL_INLINE void WDOG_HWA_SetCs (WDOG_Type * *pWdog*, uint32_t *u32Cs*)

Set CS register value, for Wdog working mode configuration.

Parameters

<i>WDOG_Type</i>	*pWdog. point to wdog instance base register address.
<i>uint32_t</i>	u32Cs. configured register value

LOCAL_INLINE void WDOG_HWA_SetTimeout (WDOG_Type * *pWdog*, uint16_t *u16Timeout*)

Set TIMEOUT register value. for WDOG timeout value.

Parameters

<i>WDOG_Type</i>	*pWdog. point to wdog instance base register address.
<i>uint16_t</i>	u16Timeout configured register value

LOCAL_INLINE void WDOG_HWA_SetWindow (WDOG_Type * pWdog, uint16_t u16Window)

Set WINDOW register value. for windowed WDOG low threshold value.

Parameters

<i>WDOG_Type</i>	*pWdog. point to wdog instance base register address.
<i>uint16_t</i>	u16Window. configured register value

HwA_wku

Collaboration diagram for HwA_wku:Macros

```
#define WKU_MWR0 0x0000100
#define WKU_MWR1 0x0010000
#define WKU_MWR2 0x1000000
```

Enumerations

```
enum WKU_WakeupInputType { WKU_INPUT_MAX = 0xFFFFFFFFU }
Wku input.
```

Functions

LOCAL_INLINE void WKU_HWA_EnableWakeupSource (const WKU_WakeupInputType eWakeup)
Enable wakeup source.

LOCAL_INLINE void WKU_HWA_DisableWakeupSource (const WKU_WakeupInputType eWakeup)
Disable wakeup source.

LOCAL_INLINE uint32_t WKU_HWA_ReadWakeupSource (void)
Read WKU wakeup source.

Detailed Description

Macro Definition Documentation

#define WKU_MWR0 0x0000100

#define WKU_MWR1 0x0010000

#define WKU_MWR2 0x1000000

Enumeration Type Documentation

enum WKU_WakeupInputType

Wku input.

Enumerator:

WKU_INPUT_M AX	
-------------------	--

Function Documentation

LOCAL_INLINE void WKU_HWA_DisableWakeupSource (const WKU_WakeupInputType *eWakeup*)

Disable wakeup source.

Parameters

<i>eWakeup</i>	Wakeup source type
----------------	--------------------

LOCAL_INLINE void WKU_HWA_EnableWakeupSource (const WKU_WakeupInputType eWakeup)

Enable wakeup source.

Parameters

<i>eWakeup</i>	Wakeup source type
----------------	--------------------

LOCAL_INLINE uint32_t WKU_HWA_ReadWakeupSource (void)

Read WKU wakeup source.

Returns

WKU wakeup source value

Module_driver_adc

Collaboration diagram for Module_driver_adc:Modules

HwA_adc

Classes

- struct **_ADC_HandleType**
The structure of the ADC processing handle.
- struct **ADC_InitType**
Defines the converter configuration.
- struct **ADC_ChannelCfgType**
The configuration option for the ADC channel.
- struct **ADC_CompareType**
Defines the hardware compare configuration.

Typedefs

typedef struct **_ADC_HandleType** **ADC_HandleType**
The structure of the ADC processing handle.

Enumerations

```
enum ADC_InstanceType { ADC_INSTANCE_0 = 0U, ADC_INSTANCE_1 = 1U, ADC_INSTANCE_2 = 2U,  
ADC_INSTANCE_3 = 3U, ADC_INSTANCE_4 = 4U, ADC_INSTANCE_5 = 5U }
```

The instance index of the ADC peripheral.

```
enum ADC_ChannelType { ADC_CHANNEL_0 = 0U, ADC_CHANNEL_1 = 1U, ADC_CHANNEL_2 = 2U,  
ADC_CHANNEL_3 = 3U, ADC_CHANNEL_4 = 4U, ADC_CHANNEL_5 = 5U, ADC_CHANNEL_6 = 6U,  
ADC_CHANNEL_7 = 7U, ADC_CHANNEL_8 = 8U, ADC_CHANNEL_9 = 9U, ADC_CHANNEL_10 = 10U,  
ADC_CHANNEL_11 = 11U, ADC_CHANNEL_12 = 12U, ADC_CHANNEL_13 = 13U, ADC_CHANNEL_14 = 14U,  
ADC_CHANNEL_15 = 15U, ADC_CHANNEL_16 = 16U, ADC_CHANNEL_17 = 17U, ADC_CHANNEL_18 = 18U,  
ADC_CHANNEL_19 = 19U, ADC_CHANNEL_20 = 20U, ADC_CHANNEL_21 = 21U, ADC_CHANNEL_22 = 22U,  
ADC_CHANNEL_23 = 23U, ADC_CHANNEL_24 = 24U, ADC_CHANNEL_25 = 25U, ADC_CHANNEL_26 = 26U,  
ADC_CHANNEL_27 = 27U, ADC_CHANNEL_28 = 28U, ADC_CHANNEL_29 = 29U, ADC_CHANNEL_30 = 30U,  
ADC_CHANNEL_31 = 31U, ADC_CHANNEL_INTERNAL_0 = 32U, ADC_CHANNEL_INTERNAL_1 = 33U,  
ADC_CHANNEL_INTERNAL_2 = 34U, ADC_CHANNEL_INTERNAL_3 = 35U, ADC_CHANNEL_INTERNAL_4 =  
36U, ADC_CHANNEL_INTERNAL_5 = 37U }
```

The channel selected for ADC conversion.

```
enum ADC_DifferentialChannelType { ADC_CHANNEL_0_4 = 0U, ADC_CHANNEL_1_5 = 1U,  
ADC_CHANNEL_2_6 = 2U, ADC_CHANNEL_3_7 = 3U, ADC_CHANNEL_TEMPSENSOR = 37U }
```

The channel selected for ADC conversion in differential mode.

```
enum ADC_SampleTimeOptionType { ADC_SAMPLE_TIME_OPTION_0 = 0U,  
ADC_SAMPLE_TIME_OPTION_1 = 1U, ADC_SAMPLE_TIME_OPTION_2 = 2U,  
ADC_SAMPLE_TIME_OPTION_3 = 3U }
```

The ADC sample time option for selection.

```
enum ADC_StatusType { ADC_STATUS_SUCCESS = 0x0U, ADC_STATUS_ERROR = 0x1U,  
ADC_STATUS_TIMEOUT = 0x2U }
```

ADC operation return values.

ADC API Service IDs

```
#define ADC_INIT_STRUCTURE_ID 0U  
#define ADC_INIT_ID 1U  
#define ADC_DEINIT_ID 2U  
#define ADC_INIT_CHANNEL_ID 3U  
#define ADC_INIT_COMPARE_ID 4U  
#define ADC_INIT_INTERRUPT_ID 5U  
#define ADC_INIT_DMA_CHANNEL_ID 6U  
#define ADC_SET_SW_PRETRIGGER_ID 7U  
#define ADC_ENABLE_ID 8U
```

```

#define ADC_DISABLE_ID 9U
#define ADC_START_ID 10U
#define ADC_STOP_ID 11U
#define ADC_RESET_ID 12U
#define ADC_GET_TRIG_ERR_STATUS_ID 13U
#define ADC_CLEAR_TRIG_ERR_ID 14U
#define ADC_CLEAR_TRIG_ERRS_ID 15U
#define ADC_GET_TRIG_LATCH_STATUS_ID 16U
#define ADC_CLEAR_LATCH_TRIGGER_ID 17U

```

ADC Dev Error Code

Error Code of calling ADC apis

```

#define ADC_E_PARAM_INSTANCE 0x01U
#define ADC_E_PARAM_CHANNEL 0x02U
#define ADC_E_PARAM_POINTER 0x03U
#define ADC_E_PARAM_COUNT 0x04U
#define ADC_E_PARAM_INVALID 0x05U
#define ADC_E_PARAM_PHASE_SHIFT 0x06U

```

ADC0 Internal Channels

Available internal ADC channels for ADC Instance 0

```

#define ADC0_CHANNEL_VBG_BUFFER ADC_CHANNEL_INTERNAL_0
#define ADC0_CHANNEL_V25 ADC_CHANNEL_INTERNAL_1
#define ADC0_CHANNEL_V11 ADC_CHANNEL_INTERNAL_2
#define ADC0_CHANNEL_CMP0_DAC ADC_CHANNEL_INTERNAL_4
#define ADC0_CHANNEL_TEMPSENSOR_OUT ADC_CHANNEL_INTERNAL_5

```

ADC1 Internal channels

Available internal ADC channels for ADC Instance 1

```

#define ADC1_CHANNEL_V11_PD0 ADC_CHANNEL_INTERNAL_0
#define ADC1_CHANNEL_VREFH ADC_CHANNEL_INTERNAL_1
#define ADC1_CHANNEL_VDDA0 ADC_CHANNEL_INTERNAL_2
#define ADC1_CHANNEL_V15 ADC_CHANNEL_INTERNAL_3
#define ADC1_CHANNEL_CMP_DAC0 ADC_CHANNEL_INTERNAL_4

```

ADC2 Internal channels

Available internal ADC channels for ADC instance 2


```
#define ADC2_CHANNEL_V11_PD0 ADC_CHANNEL_INTERNAL_0
#define ADC2_CHANNEL_VREFH0 ADC_CHANNEL_INTERNAL_1
#define ADC2_CHANNEL_VDDA0 ADC_CHANNEL_INTERNAL_2
#define ADC2_CHANNEL_CMP_DAC0 ADC_CHANNEL_INTERNAL_4
#define ADC2_CHANNEL_TEMPSENSOR_OUT ADC_CHANNEL_INTERNAL_5
#define ADC4_CHANNEL_V11 ADC_CHANNEL_INTERNAL_0
#define ADC4_CHANNEL_VREFH1 ADC_CHANNEL_INTERNAL_1
#define ADC4_CHANNEL_VDDA1 ADC_CHANNEL_INTERNAL_2
#define ADC5_CHANNEL_V11 ADC_CHANNEL_INTERNAL_0
#define ADC5_CHANNEL_VREFH1 ADC_CHANNEL_INTERNAL_1
#define ADC5_CHANNEL_VDDA1 ADC_CHANNEL_INTERNAL_2
```

ADC3 Internal channels

Available internal ADC channels for ADC instance 3

```
#define ADC3_CHANNEL_V11 ADC_CHANNEL_INTERNAL_0
#define ADC3_CHANNEL_VREFH1 ADC_CHANNEL_INTERNAL_1
#define ADC3_CHANNEL_VDDA1 ADC_CHANNEL_INTERNAL_2
```

Initial Structure

void **ADC_InitStructure** (ADC_InitType *const pInitCfg)

Provide the default values of ADC_InitType.

Initialization Functions

void **ADC_Init** (ADC_HandleType *pAdcHandle, const ADC_InitType *const pInitCfg)

Initialize the ADC instance.

void **ADC_DeInit** (ADC_HandleType *pAdcHandle)

De-initialize the ADC instance.

void **ADC_InitChannel** (ADC_HandleType *pAdcHandle, const ADC_ChannelCfgType aChannels[], const uint8_t u8ChnCnt)

Configure the ADC sample channels.

void **ADC_InitCompare** (const ADC_HandleType *pAdcHandle, const ADC_CompareType *const pCmpCfg)

Configure the hardware compare feature of ADC.

Operation functions

ADC_StatusType ADC_Enable (const **ADC_HandleType** *pAdcHandle)

Enable the ADC instance.

ADC_StatusType ADC_Disable (**ADC_HandleType** *pAdcHandle)

Disable the ADC instance.

void ADC_Start (const **ADC_HandleType** *pAdcHandle)

Start the ADC conversion.

ADC_StatusType ADC_Stop (const **ADC_HandleType** *pAdcHandle)

Stop the ADC conversion.

void ADC_Reset (const **ADC_HandleType** *pAdcHandle)

Reset the ADC hardware.

void ADCn_IRQHandler (**ADC_HandleType** *pAdcHandle)

The internal interrupt handler function for ADC instances.

Detailed Description

Macro Definition Documentation

#define ADC0_CHANNEL_CMP0_DAC ADC_CHANNEL_INTERNAL_4

#define ADC0_CHANNEL_TEMPSENSOR_OUT ADC_CHANNEL_INTERNAL_5

#define ADC0_CHANNEL_V11 ADC_CHANNEL_INTERNAL_2

#define ADC0_CHANNEL_V25 ADC_CHANNEL_INTERNAL_1

#define ADC0_CHANNEL_VBG_BUFFER ADC_CHANNEL_INTERNAL_0

#define ADC1_CHANNEL_CMP_DAC0 ADC_CHANNEL_INTERNAL_4

#define ADC1_CHANNEL_V11_PD0 ADC_CHANNEL_INTERNAL_0

#define ADC1_CHANNEL_V15 ADC_CHANNEL_INTERNAL_3

#define ADC1_CHANNEL_VDDA0 ADC_CHANNEL_INTERNAL_2

#define ADC1_CHANNEL_VREFH ADC_CHANNEL_INTERNAL_1

#define ADC2_CHANNEL_CMP_DAC0 ADC_CHANNEL_INTERNAL_4

#define ADC2_CHANNEL_TEMPSENSOR_OUT ADC_CHANNEL_INTERNAL_5

#define ADC2_CHANNEL_V11_PD0 ADC_CHANNEL_INTERNAL_0

#define ADC2_CHANNEL_VDDA0 ADC_CHANNEL_INTERNAL_2

#define ADC2_CHANNEL_VREFH0 ADC_CHANNEL_INTERNAL_1

#define ADC3_CHANNEL_V11 ADC_CHANNEL_INTERNAL_0

#define ADC3_CHANNEL_VDDA1 ADC_CHANNEL_INTERNAL_2

#define ADC3_CHANNEL_VREFH1 ADC_CHANNEL_INTERNAL_1

#define ADC4_CHANNEL_V11 ADC_CHANNEL_INTERNAL_0

#define ADC4_CHANNEL_VDDA1 ADC_CHANNEL_INTERNAL_2

#define ADC4_CHANNEL_VREFH1 ADC_CHANNEL_INTERNAL_1

```
#define ADC5_CHANNEL_V11  ADC_CHANNEL_INTERNAL_0

#define ADC5_CHANNEL_VDDA1  ADC_CHANNEL_INTERNAL_2

#define ADC5_CHANNEL_VREFH1  ADC_CHANNEL_INTERNAL_1

#define ADC_CLEAR_LATCH_TRIGGER_ID  17U

#define ADC_CLEAR_TRIG_ERR_ID  14U

#define ADC_CLEAR_TRIG_ERRS_ID  15U

#define ADC_DEINIT_ID  2U

#define ADC_DISABLE_ID  9U

#define ADC_E_PARAM_CHANNEL  0x02U

#define ADC_E_PARAM_COUNT  0x04U

#define ADC_E_PARAM_INSTANCE  0x01U

#define ADC_E_PARAM_INVALID  0x05U

#define ADC_E_PARAM_PHASE_SHIFT  0x06U

#define ADC_E_PARAM_POINTER  0x03U

#define ADC_ENABLE_ID  8U

#define ADC_GET_TRIG_ERR_STATUS_ID  13U

#define ADC_GET_TRIG_LATCH_STATUS_ID  16U

#define ADC_INIT_CHANNEL_ID  3U

#define ADC_INIT_COMPARE_ID  4U

#define ADC_INIT_DMA_CHANNEL_ID  6U

#define ADC_INIT_ID  1U

#define ADC_INIT_INTERRUPT_ID  5U
```

```
#define ADC_INIT_STRUCTURE_ID 0U
```

```
#define ADC_RESET_ID 12U
```

```
#define ADC_SET_SW_PRETRIGGER_ID 7U
```

```
#define ADC_START_ID 10U
```

```
#define ADC_STOP_ID 11U
```

Typedef Documentation

typedef struct _ADC_HandleType ADC_HandleType

The structure of the ADC processing handle.

Enumeration Type Documentation

enum ADC_ChannelType

The channel selected for ADC conversion.

Enumerator:

ADC_CHANNEL_ 0	
ADC_CHANNEL_ 1	
ADC_CHANNEL_ 2	
ADC_CHANNEL_ 3	
ADC_CHANNEL_ 4	

ADC_CHANNEL_ 5	
ADC_CHANNEL_ 6	
ADC_CHANNEL_ 7	
ADC_CHANNEL_ 8	
ADC_CHANNEL_ 9	
ADC_CHANNEL_ 10	
ADC_CHANNEL_ 11	
ADC_CHANNEL_ 12	
ADC_CHANNEL_ 13	
ADC_CHANNEL_ 14	
ADC_CHANNEL_ 15	
ADC_CHANNEL_ 16	
ADC_CHANNEL_ 17	
ADC_CHANNEL_ 18	
ADC_CHANNEL_ 19	
ADC_CHANNEL_ 20	
ADC_CHANNEL_ 21	
ADC_CHANNEL_ 22	
ADC_CHANNEL_ 23	

ADC_CHANNEL_ 24	
ADC_CHANNEL_ 25	
ADC_CHANNEL_ 26	
ADC_CHANNEL_ 27	
ADC_CHANNEL_ 28	
ADC_CHANNEL_ 29	
ADC_CHANNEL_ 30	
ADC_CHANNEL_ 31	
ADC_CHANNEL_ INTERNAL_0	
ADC_CHANNEL_ INTERNAL_1	
ADC_CHANNEL_ INTERNAL_2	
ADC_CHANNEL_ INTERNAL_3	
ADC_CHANNEL_ INTERNAL_4	
ADC_CHANNEL_ INTERNAL_5	

enum ADC_DifferentialChannelType

The channel selected for ADC conversion in differential mode.

Enumerator:

ADC_CHANNEL_ 0_4	
---------------------	--

ADC_CHANNEL_ 1_5	
ADC_CHANNEL_ 2_6	
ADC_CHANNEL_ 3_7	
ADC_CHANNEL_ TEMPSENSOR	

enum ADC_InstanceType

The instance index of the ADC peripheral.

Enumerator:

ADC_INSTANCE _0	ADC instance 0 is selected
ADC_INSTANCE _1	ADC instance 1 is selected
ADC_INSTANCE _2	ADC instance 2 is selected
ADC_INSTANCE _3	ADC instance 3 is selected
ADC_INSTANCE _4	ADC instance 4 is selected
ADC_INSTANCE _5	ADC instance 5 is selected

enum ADC_SampleTimeOptionType

The ADC sample time option for selection.

Enumerator:

ADC_SAMPLE_TI ME_OPTION_0	
ADC_SAMPLE_TI ME_OPTION_1	
ADC_SAMPLE_TI ME_OPTION_2	
ADC_SAMPLE_TI ME_OPTION_3	

enum ADC_StatusType

ADC operation return values.

Enumerator:

ADC_STATUS_S UCCESS	The ADC operation is succeed
ADC_STATUS_E RROR	The ADC operation is failed
ADC_STATUS_TI MEOUT	The ADC operation is failed because of time out

Function Documentation

void ADC_DeInit (ADC_HandleType * *pAdcHandle*)

De-initialize the ADC instance.

Restore the ADC instance to its reset state

Parameters

<i>pAdcHandle</i>	the ADC instance to de-init
-------------------	-----------------------------

ADC_StatusType ADC_Disable (ADC_HandleType * *pAdcHandle*)

Disable the ADC instance.

Parameters

<i>pAdcHandle</i>	the ADC process handler
-------------------	-------------------------

Returns

ADC_StatusType whether ADC is disabled successfully

ADC_StatusType ADC_Enable (const ADC_HandleType * *pAdcHandle*)

Enable the ADC instance.

Parameters

<i>pAdcHandle</i>	the ADC process handler
-------------------	-------------------------

Returns

ADC_StatusType whether ADC is enabled successfully

void ADC_Init (ADC_HandleType * *pAdcHandle*, const ADC_InitType *const *pInitCfg*)

Initialize the ADC instance.

Parameters

<i>pAdcHandle</i>	the ADC process handler
<i>pInitCfg</i>	the configurations of the ADC instance

void ADC_InitChannel (ADC_HandleType * *pAdcHandle*, const ADC_ChannelCfgType *aChannels*[], const uint8_t *u8ChnCnt*)

Configure the ADC sample channels.

Parameters

<i>pAdcHandle</i>	the ADC process handler
<i>aChannels</i>	the channels to use
<i>u8ChnCnt</i>	the quantity of channels

```
void ADC_InitCompare (const ADC_HandleType * pAdcHandle, const ADC_CompareType *const pCmpCfg)
```

Configure the hardware compare feature of ADC.

Parameters

<i>pAdcHandle</i>	the ADC process handler
<i>pCmpCfg</i>	the compare paremeters

```
void ADC_InitStructure (ADC_InitType *const pInitCfg)
```

Provide the default values of **ADC_InitType**.

Parameters

<i>pInitCfg</i>	the structure to initialize
-----------------	-----------------------------

```
void ADC_Reset (const ADC_HandleType * pAdcHandle)
```

Reset the ADC hardware.

Parameters

<i>pAdcHandle</i>	the ADC process handler
-------------------	-------------------------

```
void ADC_Start (const ADC_HandleType * pAdcHandle)
```

Start the ADC conversion.

If the ADC sequence mode is single or continuous, and the trigger mode is ADC_TRIGMODE_SW, the adc conversion will start immediately. Otherwise, the ADC will wait for the trigger signal to start the conversion

Parameters

<i>pAdcHandle</i>	the ADC process handler
-------------------	-------------------------

```
ADC_StatusType ADC_Stop (const ADC_HandleType * pAdcHandle)
```

Stop the ADC conversion.

If the ADC sequence mode is single, it will stop the ongoing conversion. If no ongoing conversion, it will have no effect. If the ADC sequence mode is continuous or discontinuous, it will stop the ongoing conversion and meanwhile the further conversions.

Parameters

<i>pAdcHandle</i>	the ADC process handler
-------------------	-------------------------

Returns

ADC_StatusType whether ADC is stopped successfully

void ADCn_IRQHandler (ADC_HandleType * *pAdcHandle*)

The internal interrupt handler function for ADC instances.

Parameters

<i>pAdcHandle</i>	the ADC process handler
-------------------	-------------------------

Module_driver_aontimer

Collaboration diagram for Module_driver_aontimer:Modules

HwA_aontimer

Classes

struct **_AONTIMER_HandleType**

The structure of the AONTIMER processing handle.

struct **AONTIMER_InitType**

Aontimer Initialization struct type.

Typedefs

typedef struct **_AONTIMER_HandleType** **AONTIMER_HandleType**

The structure of the AONTIMER processing handle.

Enumerations

```
enum AONTIMER_InstanceType { AONTIMER_INSTANCE_0 = 0U }
```

The instance index of the FTU peripheral.

```
enum AONTIMER_ModeType { AONTIMER_COUNTER_MODE = 0, AONTIMER_PULSE_MODE }
```

Aontimer mode.

Functions

```
void AONTIMER_Init (AONTIMER_HandleType *pAontimerHandle, const AONTIMER_InitType *const  
pInitStruct)
```

Initialize aontimer instance.

```
void AONTIMER_Deinit (AONTIMER_HandleType *pAontimerHandle)
```

De-initialize aontimer instance.

```
void AONTIMER_StartTimer (AONTIMER_HandleType *pAontimerHandle)
```

Start Aontimer.

```
void AONTIMER_StopTimer (AONTIMER_HandleType *pAontimerHandle)
```

Stop Aontimer.

```
void AONTIMER_UpdateCounterValue (AONTIMER_HandleType *pAontimerHandle, const uint16_t  
u16StartValue)
```

Update value of aontimer counter.

```
void Aontimer_CommonProcessInterrupt (AONTIMER_HandleType *pAontimerHandle)
```

Aontimer common interrupt handle function.

AONTIMER API Service IDs

```
#define AONTIMER_INIT_ID 0U
```

```
#define AONTIMER_DEINIT_ID 1U
```

```
#define AONTIMER_START_ID 2U
```

```
#define AONTIMER_STOP_ID 3U
```

```
#define AONTIMER_UPDATE_COUNT_ID 4U
```

```
#define AONTIMER_INTE_PROC_ID 5U
```

AONTIMER Dev Error Code

Error Code of calling AONTIMER apis

```
#define AONTIMER_E_PARAM_INSTANCE 0x01U
#define AONTIMER_E_PARAM_CLOCK 0x02U
#define AONTIMER_E_PARAM_POINTER 0x03U
```

Detailed Description

Macro Definition Documentation

```
#define AONTIMER_DEINIT_ID 1U

#define AONTIMER_E_PARAM_CLOCK 0x02U

#define AONTIMER_E_PARAM_INSTANCE 0x01U

#define AONTIMER_E_PARAM_POINTER 0x03U

#define AONTIMER_INIT_ID 0U

#define AONTIMER_INTE_PROC_ID 5U

#define AONTIMER_START_ID 2U

#define AONTIMER_STOP_ID 3U

#define AONTIMER_UPDATE_COUNT_ID 4U
```

Typedef Documentation

```
typedef struct _AONTIMER_HandleType AONTIMER_HandleType
```

The structure of the AONTIMER processing handle.

Enumeration Type Documentation

enum AONTIMER_InstanceType

The instance index of the FTU peripheral.

Enumerator:

AONTIMER_INSTANCE_0	
---------------------	--

enum AONTIMER_ModeType

Aontimer mode.

Enumerator:

AONTIMER_COUNTER_MODE	In this mode ,the clock source is internal.
AONTIMER_PULSE_MODE	In this mode ,the clock source is external(pin/trgsel).

Function Documentation

void Aontimer_CommonProcessInterrupt (AONTIMER_HandleType * *pAontimerHandle*)

Aontimer common interrupt handle function.

Parameters

<i>pAontimerHandle</i>	the Aontimer instance to use
------------------------	------------------------------

void AONTIMER_Deinit (AONTIMER_HandleType * *pAontimerHandle*)

De-initialize aontimer instance.

Parameters

<i>pAontimerHandle</i>	the Aontimer instance to use
------------------------	------------------------------

void AONTIMER_Init (AONTIMER_HandleType * *pAontimerHandle*, const AONTIMER_InitType *const *pInitStruct*)

Initialize aontimer instance.

Parameters

<i>pAontimerHandle</i>	the Aontimer instance to use
<i>pInitStruct</i>	Aontimer Initialize structure

void AONTIMER_StartTimer (AONTIMER_HandleType * *pAontimerHandle*)

Start Aontimer.

Parameters

<i>pAontimerHandle</i>	the Aontimer instance to use
------------------------	------------------------------

void AONTIMER_StopTimer (AONTIMER_HandleType * *pAontimerHandle*)

Stop Aontimer.

Parameters

<i>pAontimerHandle</i>	the Aontimer instance to use
------------------------	------------------------------


```
void AONTIMER_UpdateCounterValue (AONTIMER_HandleType * pAontimerHandle, const uint16_t u16StartValue)
```

Update value of aontimer counter.

Parameters

<i>pAontimerHandle</i>	the Aontimer instance to use
<i>u16StartValue</i>	input value, range : 0~65535

Module_driver_cmp

Collaboration diagram for Module_driver_cmp:Modules

HwA_cmp

Classes

struct **CMP_DacType**

The instance index of the CMP Dac parts.

struct **CMP_MuxType**

The instance index of the CMP Mux parts.

struct **CMP_ComparatorType**

The instance index of the CMP Comparator parts.

struct **CMP_InterruptType**

Defines the comparator interrupt configuration.

struct **_CMP_HandleType**

The structure of the CMP processing handle.

struct **CMP_CSChannelCfgType**

Defines the comparator channel selected in channel scan mode.

struct **CMP_ChannelScanType**

Defines the comparator channel scan configuration.

struct **CMP_InitType**

Defines the comparator configuration.

Typedefs

typedef void(* **CMP_CompleteIntCallback**) (void)

The CMP complete interrupt callback function prototype.

typedef struct **_CMP_HandleType** **CMP_HandleType**

The structure of the CMP processing handle.

Functions

void **CMP_GetDefaultInitCfg** (**CMP_InitType** *pInitCfg)

set CMP module configurations instructure to defalut value

void **CMP_Enable** (**CMP_HandleType** *pCmpHandle)

set CMP module enable

void **CMP_Disable** (**CMP_HandleType** *pCmpHandle)

set CMP module disable

void **CMP_Init** (**CMP_HandleType** *pCmpHandle, const **CMP_InitType** *const pInitCfg)

CMP init.

void **CMP_Deinit** (**CMP_HandleType** *pCmpHandle)

CMP Deinit.

void **CMP_Enter_LP_Mode** (**CMP_HandleType** *pCmpHandle)

CMP Deinit.

uint32_t **CMP_GetCmpOut** (**CMP_HandleType** *pCmpHandle)

get CMP filtered output

void **CMP_SetDacData** (**CMP_HandleType** *pCmpHandle, uint8_t u8Data)

set CMP Dac output

CMP_OutStatus CMP_GetOutFlagStatus (CMP_HandleType *pCmpHandle)

get CMP interrupter flag status

void CMPn_IRQHandler (CMP_HandleType *pCmpHandle)

CMP interrupt process function.

void CMP_CSEnable (CMP_HandleType *pCmpHandle)

set CMP module channel scan mode enable

void CMP_CSDisable (CMP_HandleType *pCmpHandle)

set CMP module channel scan mode disable

bool CMP_GetCSFlagStatus (CMP_HandleType *pCmpHandle)

get CMP channel scan flag status

void CMP_EnableCSChannel (CMP_HandleType *pCmpHandle, const CMP_MuxSelType eChannel, const bool bPreSetState)

enable CMP channel scan channels

void CMP_DisableCSChannel (CMP_HandleType *pCmpHandle, const CMP_MuxSelType eChannel)

disable CMP channel scan channels

bool CMP_GetCSChannlOut (CMP_HandleType *pCmpHandle, const CMP_MuxSelType e_Channel)

get CMP channel scan channel out

bool CMP_GetCmpCSActive (CMP_HandleType *pCmpHandle)

get CMP channel scan status

CMP API Service IDs

#define CMP_INIT_ID 0U

#define CMP_DEINIT_ID 1U

#define CMP_GETDEFAULTINITCFG_ID 2U

#define CMP_ENABLE_ID 3U

```
#define CMP_DISABLE_ID 4U
#define CMP_GETCMPOUT_ID 5U
#define CMP_SETDACDATA_ID 6U
#define CMP_GETOUTFLAGSTATUS_ID 7U
#define CMP_GETCSFLAGSTATUS_ID 8U
#define CMP_ENABLECSCHANNEL_ID 9U
#define CMP_DISABLECSCHANNEL_ID 10U
#define CMP_GETCSCHANNELOUT_ID 11U
#define CMP_GETCMPCSACTIVE_ID 12U
#define CMP_ENTERLPMODE_ID 13U
#define CMP_EXITLPMODE_ID 14U
```

CMP Dev Error Code

Error Code of calling CMP apis

```
#define CMP_E_PARAM_INSTANCE 0x01U
#define CMP_E_PARAM_CHANNEL 0x02U
#define CMP_E_PARAM_POINTER 0x03U
```

Detailed Description

Macro Definition Documentation

#define CMP_DEINIT_ID 1U

#define CMP_DISABLE_ID 4U

#define CMP_DISABLECHANNEL_ID 10U

#define CMP_E_PARAM_CHANNEL 0x02U

#define CMP_E_PARAM_INSTANCE 0x01U

#define CMP_E_PARAM_POINTER 0x03U

#define CMP_ENABLE_ID 3U

#define CMP_ENABLECHANNEL_ID 9U

#define CMP_ENTERLPMODE_ID 13U

#define CMP_EXITLPMODE_ID 14U

#define CMP_GETCMPCSACTIVE_ID 12U

#define CMP_GETCMPOUT_ID 5U

#define CMP_GETCSCHANNELOUT_ID 11U

#define CMP_GETCSFLAGSTATUS_ID 8U

#define CMP_GETDEFAULTINITCFG_ID 2U

#define CMP_GETOUTFLAGSTATUS_ID 7U

#define CMP_INIT_ID 0U

#define CMP_SETDACDATA_ID 6U

Typedef Documentation

typedef void(* CMP_CompleteIntCallback) (void)

The CMP complete interrupt callback function prototype.

typedef struct _CMP_HandleType CMP_HandleType

The structure of the CMP processing handle.

Function Documentation

void CMP_CSDisable (CMP_HandleType * *pCmpHandle*)

set CMP module channel scan mode disable

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

void CMP_CSEnable (CMP_HandleType * *pCmpHandle*)

set CMP module channel scan mode enable

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

void CMP_Deinit (CMP_HandleType * *pCmpHandle*)

CMP Deinit.

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

void CMP_Disable (CMP_HandleType * *pCmpHandle*)

set CMP module disable

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

void CMP_DisableCSChannel (CMP_HandleType * *pCmpHandle*, const CMP_MuxSelType *eChannel*)

disable CMP channel scan channels

Parameters

<i>pCmpHandle</i>	CMP processing handle
<i>e_Channel</i>	the CMP channel channel

void CMP_Enable (CMP_HandleType * *pCmpHandle*)

set CMP module enable

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

**void CMP_EnableCSChannel (CMP_HandleType * *pCmpHandle*, const CMP_MuxSelType *eChannel*,
const bool *bPreSetState*)**

enable CMP channel scan channels

Parameters

<i>pCmpHandle</i>	CMP processing handle
<i>e_Channel</i>	the CMP channel channel
<i>bPreSetState</i>	preset state for the given channel

void CMP_Enter_LP_Mode (CMP_HandleType * *pCmpHandle*)

CMP Deinit.

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

bool CMP_GetCmpCSActive (CMP_HandleType * *pCmpHandle*)

get CMP channel scan status

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

uint32_t CMP_GetCmpOut (CMP_HandleType * *pCmpHandle*)

get CMP filtered output

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

Returns

CMP filtered output

bool CMP_GetCSChannlOut (CMP_HandleType * *pCmpHandle*, const CMP_MuxSelType *e_Channel*)

get CMP channel scan channel out

Parameters

<i>pCmpHandle</i>	CMP processing handle
<i>e_Channel</i>	the CMP channel channel

Returns

the CMP channel current status

bool CMP_GetCSFlagStatus (CMP_HandleType * *pCmpHandle*)

get CMP channel scan flag status

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

Returns

CMP channel scan status

void CMP_GetDefaultInitCfg (CMP_InitType * *pInitCfg*)

set CMP module configurations instructure to defalut value

Parameters

<i>pInitCfg</i>	The basic configurations of the CMP instance
-----------------	--

CMP_OutStatus CMP_GetOutFlagStatus (CMP_HandleType * *pCmpHandle*)

get CMP interrupter flag status

Parameters

<i>pCmpHandle</i>	CMP processing handle
-------------------	-----------------------

Returns

CMP output status

void CMP_Init (CMP_HandleType * *pCmpHandle*, const CMP_InitType *const *pInitCfg*)

CMP init.

Parameters

<i>pCmpHandle</i>	CMP processing handle
<i>pInitCfg</i>	The basic configurations of the CMP instance

void CMP_SetDacData (CMP_HandleType * *pCmpHandle*, uint8_t *u8Data*)

set CMP Dac output

Parameters

<i>pCmpHandle</i>	CMP processing handle
<i>u8Data</i>	input voltage value

Note

voltage = (VinRef / 256) * (eDacData + 1)

void CMPn_IRQHandler (CMP_HandleType * *pCmpHandle*)

CMP interrupt process function.

Parameters

<i>pCmpHandle</i>	CMP processing handlee
-------------------	------------------------

Module_driver_cmu

Collaboration diagram for Module_driver_cmu:Modules

HwA_cmu

Classes

struct **_CMU_HandleType**

The structure of the CMU processing handle.

struct **CMU_CfgType**

CMU configure structure.

Typedefs

typedef struct **_CMU_HandleType** **CMU_HandleType**

The structure of the CMU processing handle.

Enumerations

enum **CMU_InstanceType** { **CMU_INSTANCE_0** = 0U, **CMU_INSTANCE_1** = 1U, **CMU_INSTANCE_2** = 2U, **CMU_INSTANCE_3** = 3U, **CMU_INSTANCE_4** = 4U }

CMU Instance type.

enum **CMU_StatusType** { **CMU_STATUS_SUCCESS** = 0x0U, **CMU_STATUS_FAIL** = 0x1U }

CMU operation return values.

enum **CMU_LowpowerModeType** { **CMU_STOP_MODE** = 0U, **CMU_STANDBY_MODE** = 1U }

CMU Lowpower Mode type.

enum **CMU_Cmu4ClkSrcType** { **CMU_CMU4_REF_CLK_NONE** = 0U, **CMU_CMU4_REF_CLK_FOSC** = 1U, **CMU_CMU4_REF_CLK_SIRC** = 2U }

CMU4 Reference clock type.

enum **CMU_InterruptType** { **CMU_INTERRUPT_NONE** = 0U, **CMU_INTERRUPT_LOC** = 1U, **CMU_INTERRUPT_MIS** = 2U }

CMU interrupt mode type.

enum **CMU_RefClockDivType** { **CMU_REFCLK_NODIV** = 0U, **CMU_REFCLK_DIV_BY2** = 1U, **CMU_REFCLK_DIV_BY4** = 2U, **CMU_REFCLK_DIV_BY8** = 3U, **CMU_REFCLK_DIV_BY16** = 4U, **CMU_REFCLK_DIV_BY32** = 5U, **CMU_REFCLK_DIV_BY64** = 6U, **CMU_REFCLK_DIV_BY128** = 7U }

CMU Reference clock div type.

Functions

void **CMU_GetDefaultConfig** (**CMU_CfgType** *const pInitCfg)

Initializes a CMU configuration structure with default values.

void **CMU_CaculateMonitorWindowsValue** (**CMU_CfgType** *const pInitCfg)

Calculates monitor window values for the CMU based on the given configuration.

CMU_StatusType **CMU_Init** (**CMU_HandleType** *const pHandle, const **CMU_CfgType** *const pInitCfg)

Initializes the CMU module with the provided configuration.

void **CMU_Enable** (**CMU_HandleType** *const pHandle)

Enables the Clock Monitor Unit (CMU) for the specified instance.

void **CMU_Disable** (**CMU_HandleType** *const pHandle)

Disables the Clock Monitor Unit (CMU) for the specified instance.

void **CMU_EnableInterrupt** (CMU_HandleType *const pHandle)
Enables the Clock Monitor Unit (CMU) interrupt for the specified instance.

void **CMU_DisableInterrupt** (CMU_HandleType *const pHandle)
Disables the Clock Monitor Unit (CMU) interrupt for the specified instance.

void **CMU_LowPowerModeEnable** (CMU_HandleType *const pHandle, CMU_LowpowerModeType eMode, bool bModeEnable, bool bRestartEnable)
Enables or disables low power mode on the CMU module.

uint32_t **CMU_GetCount** (CMU_HandleType *const pHandle)
Retrieves the current count from the CMU module.

uint32_t **CMU_GetMinCount** (CMU_HandleType *const pHandle)
Retrieves the minimum count value from the CMU module.

uint32_t **CMU_GetMaxCount** (CMU_HandleType *const pHandle)
Retrieves the maximum count value from the CMU module.

void **CMU_SetCmu4RefSrc** (CMU_Cmu4ClkSrcType eSrc)
Sets the reference clock source for CMU4.

CMU_InterruptType **CMU_GetInterruptType** (CMU_HandleType *const pHandle)
Retrieves the interrupt type of the CMU (Clock Monitor Unit) module.

void **CMU_CommonIRQHandler** (CMU_HandleType *const pHandle)
Common Clock Monitor Unit (CMU) Interrupt Service Routine (ISR)

CMU API Service IDs

```
#define CMU_INIT_ID 0x00U  
#define CMU_GET_DEFAULT_CONFIG_ID 0x01U  
#define CMU_ENABLE_ID 0x02U
```

```
#define CMU_DISABLE_ID 0x03U
#define CMU_ENABLE_IRQ_ID 0x04U
#define CMU_DISABLE_IRQ_ID 0x05U
#define CMU_COMMONIRQHANDLER_ID 0x06U
#define CMU_ENABLE_LOWPOWER_MODE_ID 0x07U
#define CMU_CAL_MONITOR_WINDOW_ID 0x08U
#define CMU_GET_CURRENTCOUNT_ID 0x09U
#define CMU_GET_MINCOUNT_ID 0x0AU
#define CMU_GET_MAXCOUNT_ID 0x0BU
#define CMU_GET_INQTYPE_ID 0x0CU
```

CMU Dev Error Code

Error Code of calling CMU apis

```
#define CMU_E_PARAM_NULLPTR 0x01U
#define CMU_E_PARAM_INSTANCE 0x02U
```

Detailed Description

Macro Definition Documentation

#define CMU_CAL_MONITOR_WINDOW_ID 0x08U

#define CMU_COMMONIRQHANDLER_ID 0x06U

#define CMU_DISABLE_ID 0x03U

#define CMU_DISABLE_IRQ_ID 0x05U

#define CMU_E_PARAM_INSTANCE 0x02U

#define CMU_E_PARAM_NULLPTR 0x01U

#define CMU_ENABLE_ID 0x02U

#define CMU_ENABLE_IRQ_ID 0x04U

#define CMU_ENABLE_LOWPOWER_MODE_ID 0x07U

#define CMU_GET_CURRENTCOUNT_ID 0x09U

#define CMU_GET_DEFAULT_CONFIG_ID 0x01U

#define CMU_GET_INQTYPE_ID 0x0CU

#define CMU_GET_MAXCOUNT_ID 0x0BU

#define CMU_GET_MINCOUNT_ID 0x0AU

#define CMU_INIT_ID 0x00U

Typedef Documentation

typedef struct _CMU_HandleType CMU_HandleType

The structure of the CMU processing handle.

Enumeration Type Documentation

enum CMU_Cmu4ClkSrcType

CMU4 Reference clock type.

Enumerator:

CMU_CMU4_RE F_CLK_NONE	Not Select Cmu4 Reference clock.
CMU_CMU4_RE F_CLK_FOSC	Select the FOSC as Cmu4 Reference clock.
CMU_CMU4_RE F_CLK_SIRC	Select the SIRC as Cmu4 Reference clock.

enum CMU_InstanceType

CMU Instance type.

Enumerator:

CMU_INSTANCE _0	
CMU_INSTANCE _1	
CMU_INSTANCE _2	
CMU_INSTANCE _3	
CMU_INSTANCE _4	

enum CMU_InterruptType

CMU interrupt mode type.

Enumerator:

CMU_INTERRUPT_ T_NONE	No CMU interrupt.
CMU_INTERRUPT_ T_LOC	CMU_ST[LOC] bit, Monitor clock is loss.
CMU_INTERRUPT_ T_MIS	CMU_ST[MIS] bit, Monitor clock is mismatched.

enum CMU_LowpowerModeType

CMU Lowpower Mode type.

Enumerator:

CMU_STOP_MO DE	CMU enable in Stop Mode.
CMU_STANDBY_ MODE	CMU enable in Standby Mode.

enum CMU_RefClockDivType

CMU Reference clock div type.

Enumerator:

CMU_REFCLK_N ODIV	Clock output is disabled.
CMU_REFCLK_D IV_BY2	Divided by 2.

CMU_REFCLK_D IV_BY4	Divided by 4.
CMU_REFCLK_D IV_BY8	Divided by 8.
CMU_REFCLK_D IV_BY16	Divided by 16.
CMU_REFCLK_D IV_BY32	Divided by 32.
CMU_REFCLK_D IV_BY64	Divided by 64.
CMU_REFCLK_D IV_BY128	Divided by 128.

enum CMU_StatusType

CMU operation return values.

Enumerator:

CMU_STATUS_S UCCESS	The CMU status success
CMU_STATUS_F AIL	The CMU status fail

Function Documentation

void CMU_CaculateMonitorWindowsValue (CMU_CfgType *const *plnitCfg*)

Calculates monitor window values for the CMU based on the given configuration.

This function calculates the reference and monitor clock window values for the specified CMU instance. It verifies input parameters, checks the clock status, and performs necessary calculations. If any input is invalid or an error occurs, appropriate error reporting may be triggered.

Parameters

<i>pInitCfg</i>	Pointer to a CMU configuration structure with user-defined settings
-----------------	---

Note

Error reporting is enabled when `CMU_DEV_ERROR_REPORT` is set to `STD_ON`.

void CMU_CommonIRQHandler (CMU_HandleType *const *pHandle*)

Common Clock Monitor Unit (CMU) Interrupt Service Routine (ISR)

This ISR handles common interrupt scenarios for the CMU module associated with the given handle. It retrieves the current interrupt status, performs error handling as needed, and calls the registered error callback function if present. Additionally, it addresses a specific hardware erratum (ERR_CMU_001). After processing, the ISR clears the status flags.

Parameters

<i>pHandle</i>	Pointer to a CMU handle structure containing the instance information
----------------	---

Note

This function assumes that the CMU interrupt has already occurred and is being serviced.

Error reporting and handling are performed based on the interrupt status.

void CMU_Disable (CMU_HandleType *const *pHandle*)

Disables the Clock Monitor Unit (CMU) for the specified instance.

This function disables the CMU module for the given handle's instance. It performs parameter validation, clears the control register flags for enabling and interrupts, and resets the status. If an error occurs or the handle is invalid, an error report will be generated.

Parameters

<i>pHandle</i>	Pointer to a CMU handle structure containing the instance information
----------------	---

Note

Error reporting is enabled when `CMU_DEV_ERROR_REPORT` is set to `STD_ON`.

void CMU_DisableInterrupt (CMU_HandleType *const *pHandle*)

Disables the Clock Monitor Unit (CMU) interrupt for the specified instance.

This function disables the interrupt for the CMU module associated with the given handle's instance. It performs parameter validation and updates the control register to disable the interrupt. If an error occurs or the handle is invalid, an error report will be generated.

Parameters

<i>pHandle</i>	Pointer to a CMU handle structure containing the instance information
----------------	---

Note

Error reporting is enabled when CMU_DEV_ERROR_REPORT is set to STD_ON .

void CMU_Enable (CMU_HandleType *const *pHandle*)

Enables the Clock Monitor Unit (CMU) for the specified instance.

This function enables the CMU module for the given handle's instance. It performs parameter validation and updates the control register to activate the CMU. If an error occurs or the handle is invalid, an error report will be generated.

Parameters

<i>pHandle</i>	Pointer to a CMU handle structure containing the instance information
----------------	---

Note

Error reporting is enabled when CMU_DEV_ERROR_REPORT is set to STD_ON .

void CMU_EnableInterrupt (CMU_HandleType *const *pHandle*)

Enables the Clock Monitor Unit (CMU) interrupt for the specified instance.

This function enables the interrupt for the CMU module associated with the given handle's instance. It performs parameter validation and updates the control register to enable the interrupt. If an error occurs or the handle is invalid, an error report will be generated.

Parameters

<i>pHandle</i>	Pointer to a CMU handle structure containing the instance information
----------------	---

Note

Error reporting is enabled when CMU_DEV_ERROR_REPORT is set to STD_ON .

uint32_t CMU_GetCount (CMU_HandleType *const *pHandle*)

Retrieves the current count from the CMU module.

This function returns the current count value from the specified CMU module instance.

Parameters

<i>pHandle</i>	Pointer to the CMU handle structure.
----------------	--------------------------------------

Returns

uint32_t The current count value from the CMU module.

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported.

void CMU_GetDefaultConfig (CMU_CfgType *const *pInitCfg*)

Initializes a CMU configuration structure with default values.

This function configures the provided CMU configuration structure with a set of default settings, including interrupt enable, low-power mode, clock divider, and more. If the passed configuration structure pointer is NULL and error reporting is enabled, an error will be reported.

Parameters

<i>pInitCfg</i>	Pointer to a CMU_CfgType structure that will be initialized with default configuration values
-----------------	--

Note

The function checks for NULL pointer only if error reporting (CMU_DEV_ERROR_REPORT) is turned on.

By default, it sets the structure fields to:

- Interrupt Enable: true
- Low Power Enable: false
- Clock Monitor Enable: false
- Module Enable: true
- Peripheral Monitor Enable: false
- Clock Divider: CMU_REFCLK_NODIV
- Error Callback: NULL

CMU_InterruptType CMU_GetInterruptType (CMU_HandleType *const *pHandle*)

Retrieves the interrupt type of the CMU (Clock Monitor Unit) module.

This function reads the status register of the CMU module to determine if a LOC (Loss of Clock) or MIS (Mismatch) interrupt has occurred.

Parameters

<i>pHandle</i>	Pointer to the CMU module handle. The handle contains information about the CMU instance.
----------------	---

Returns

CMU_InterruptType The current interrupt type, which can be one of the following:

- CMU_INTERRUPT_NONE: No interrupt has occurred.
- CMU_INTERRUPT_LOC: A LOC interrupt has occurred.
- CMU_INTERRUPT_MIS: A MIS interrupt has occurred.

Note

If device error reporting is enabled (CMU_DEV_ERROR_REPORT == STD_ON), the function will report an error if the handle pointer is NULL or if the instance number is invalid.

uint32_t CMU_GetMaxCount (CMU_HandleType *const *pHandle*)

Retrieves the maximum count value from the CMU module.

This function returns the maximum count value that the specified CMU module instance can support.

Parameters

<i>pHandle</i>	Pointer to the CMU handle structure.
----------------	--------------------------------------

Returns

uint32_t The maximum count value supported by the CMU module.

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported.

uint32_t CMU_GetMinCount (CMU_HandleType *const *pHandle*)

Retrieves the minimum count value from the CMU module.

This function returns the minimum count value that the specified CMU module instance can support.

Parameters

<i>pHandle</i>	Pointer to the CMU handle structure.
----------------	--------------------------------------

Returns

uint32_t The minimum count value supported by the CMU module.

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported.

CMU_StatusType CMU_Init (CMU_HandleType *const *pHandle*, const CMU_CfgType *const *pInitCfg*)

Initializes the CMU module with the provided configuration.

This function initializes the Clock Monitor Unit (CMU) according to the specified configuration. It performs parameter validation, initializes hardware registers, and sets up callback functions. If any input is invalid, an error will be reported and the function will return CMU_STATUS_FAIL .

Parameters

<i>pHandle</i>	Pointer to a CMU handle structure containing the instance information
<i>pInitCfg</i>	Pointer to a constant CMU configuration structure with initialization settings

Returns

Status of the initialization process

- CMU_STATUS_SUCCESS: Initialization successful
- CMU_STATUS_FAIL: Initialization failed due to an error or invalid parameter

Note

Error reporting is enabled when CMU_DEV_ERROR_REPORT is set to STD_ON .

void CMU_LowPowerModeEnable (CMU_HandleType *const *pHandle*, CMU_LowpowerModeType *eMode*, bool *bModeEnable*, bool *bRestartEnable*)

Enables or disables low power mode on the CMU module.

This function configures the CMU module to enter or exit low power mode based on the parameters provided. It supports both standby and stop modes, and can optionally enable low power restart.

Parameters

<i>pHandle</i>	Pointer to the CMU handle structure.
<i>eMode</i>	The low power mode type to be set (CMU_STANDBY_MODE or CMU_STOP_MODE).
<i>bModeEnable</i>	Boolean flag indicating whether to enable (true) or disable (false) the low power mode.
<i>bRestartEnable</i>	Boolean flag indicating whether to enable (true) or disable (false) low power restart.

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported.

void CMU_SetCmu4RefSrc (CMU_Cmu4ClkSrcType *eSrc*)

Sets the reference clock source for CMU4.

This function configures the CMU4 reference clock source to either the Fast Oscillator Clock (FOSC) or the Slow Internal RC Oscillator Clock (SIRC).

Parameters

<i>eSrc</i>	The CMU4 clock source type (CMU_CMU4_REF_CLK_FOSC or CMU_CMU4_REF_CLK_SIRC).
-------------	--

Note

The function clears the current CMU4 clock setting before applying the new source.

Module_driver_cordic

Collaboration diagram for Module_driver_cordic:Modules

HwA_cordic

Classes

struct **CORDIC_SinCos_Type**

Circular sine and cosine type.

struct **CORDIC_SinhCosh_Type**

Circular hyperbolic sine and cosine type.

Typedefs

typedef float **CORDIC_Radical_Type**

Circular radical type.

typedef float **CORDIC_Arctan_Type**

Circular arctangent type.

typedef float **CORDIC_Arctanh_Type**

Circular hyperbolic arctangent type.

typedef float **CORDIC_Ln_Type**

The arctan type.

typedef float **CORDIC_Sqrt_Type**

The arctanh type.

Enumerations

enum **CORDIC_InstanceType** { **CORDIC_INSTANCE_0** = 0U }

CORDIC instance type.

Functions

CORDIC_SinCos_Type **Cordic_Sin_Cos** (**CORDIC_InstanceType** eInstance, const float radian)

Computes sine and cosine values using the circular CORDIC algorithm.

CORDIC_Radical_Type **Cordic_Radical_Add** (**CORDIC_InstanceType** eInstance, const float x, const float y)

Computes the angle given the x and y components using the circular CORDIC algorithm.

CORDIC_Arctan_Type **Cordic_Arctan_F** (**CORDIC_InstanceType** eInstance, const float y)

Computes the arctangent of a value using the circular CORDIC algorithm.

CORDIC_Arctan_Type **Cordic_Arctan** (**CORDIC_InstanceType** eInstance, const int32_t x, const int32_t y)

Computes the arctangent of a point given its x and y coordinates using the circular CORDIC algorithm.

CORDIC_SinhCosh_Type **Cordic_Sinh_Cosh** (**CORDIC_InstanceType** eInstance, const float z)

Computes the hyperbolic sine and cosine of a value using the circular CORDIC algorithm.

CORDIC_Radical_Type **Cordic_Radical_Sub** (**CORDIC_InstanceType** eInstance, const float x, const float y)

Computes the circular radical subtraction using the CORDIC algorithm.

CORDIC_Arctanh_Type **Cordic_Arctanh** (**CORDIC_InstanceType** eInstance, const float y)

Computes the hyperbolic arctangent using the CORDIC algorithm.

CORDIC_Ln_Type Cordic_Extended_LN (CORDIC_InstanceType eInstance, const float y)

The function to calculate ln.

CORDIC_Sqrt_Type Cordic_Extended_Sqrt (CORDIC_InstanceType eInstance, const float y)

The function to calculate sqrt.

CORDIC API Service IDs

```
#define CORDIC_SIN_COS_ID 0x00
#define CORDIC_RADICAL_ADD_ID 0x01
#define CORDIC_ARCTAN_F_ID 0x02
#define CORDIC_ARCTAN_ID 0x03
#define CORDIC SINH_COSH_ID 0x04
#define CORDIC_RADICAL_SUB_ID 0x05
#define CORDIC_ARCTANH_ID 0x06
#define CORDIC_EXTENDED_LN_ID 0x07
#define CORDIC_EXTENDED_SQRT_ID 0x08
```

CORDIC Error Codes

```
#define CORDIC_E_PARAM_INSTANCE 0x01U
```

Detailed Description

Macro Definition Documentation

#define CORDIC_ARCTAN_F_ID 0x02

#define CORDIC_ARCTAN_ID 0x03

#define CORDIC_ARCTANH_ID 0x06

#define CORDIC_E_PARAM_INSTANCE 0x01U

#define CORDIC_EXTENDED_LN_ID 0x07

#define CORDIC_EXTENDED_SQRT_ID 0x08

#define CORDIC_RADICAL_ADD_ID 0x01

#define CORDIC_RADICAL_SUB_ID 0x05

#define CORDIC_SIN_COS_ID 0x00

#define CORDIC_SINH_COSH_ID 0x04

Typedef Documentation

typedef float CORDIC_Arctan_Type

Circular arctangent type.

typedef float CORDIC_Arctanh_Type

Circular hyperbolic arctangent type.

typedef float CORDIC_Ln_Type

The arctan type.

typedef float CORDIC_Radical_Type

Circular radical type.

typedef float CORDIC_Sqrt_Type

The arctanh type.

Enumeration Type Documentation

enum CORDIC_InstanceType

CORDIC instance type.

Enumerator:

CORDIC_INSTA NCE_0	CORDIC instance 0 is selected.
-----------------------	--------------------------------

Function Documentation

CORDIC_Arctan_Type Cordic_Arctan (CORDIC_InstanceType *elInstance*, const int32_t *x*, const int32_t *y*)

Computes the arctangent of a point given its x and y coordinates using the circular CORDIC algorithm.

This function computes the arctangent of a point defined by its x and y coordinates using the circular CORDIC algorithm in vector mode.

Parameters

<i>elInstance</i>	The instance of the CORDIC module to use.
<i>x</i>	The x-coordinate of the point.
<i>y</i>	The y-coordinate of the point.

Returns

A CORDIC_Arctan_Type representing the computed arctangent value.

CORDIC_Arctan_Type Cordic_Arctan_F (CORDIC_InstanceType *elInstance*, const float *y*)

Computes the arctangent of a value using the circular CORDIC algorithm.

This function computes the arctangent of a value using the circular CORDIC algorithm in vector mode. It handles special cases for large input values.

Parameters

<i>elInstance</i>	The instance of the CORDIC module to use.
<i>y</i>	The value for which to compute the arctangent.

Returns

A CORDIC_Arctan_Type representing the computed arctangent value.

CORDIC_Arctanh_Type Cordic_Arctanh (CORDIC_InstanceType *elInstance*, const float *y*)

Computes the hyperbolic arctangent using the CORDIC algorithm.

This function calculates the hyperbolic arctangent of a given input value *y* . It uses the CORDIC hardware abstraction layer (HAL) functions to perform the calculations.

Parameters

<i>elInstance</i>	The CORDIC instance identifier.
<i>y</i>	The input value for which to compute the hyperbolic arctangent.

Returns

CORDIC_Arctanh_Type The computed hyperbolic arctangent value.

CORDIC_Ln_Type Cordic_Extended_LN (CORDIC_InstanceType *elInstance*, const float *y*)

The function to calculate ln.

Parameters

<i>y</i>	the input value
----------	-----------------

CORDIC_Sqrt_Type Cordic_Extended_Sqrt (CORDIC_InstanceType *elInstance*, const float *y*)

The function to calculate sqrt.

Parameters

<i>y</i>	the input value (0.0267120594,2.3397671865)
----------	---

CORDIC_Radical_Type Cordic_Radical_Add (CORDIC_InstanceType *elInstance*, const float *x*, const float *y*)

Computes the angle given the x and y components using the circular CORDIC algorithm.

This function computes the angle given the x and y components using the circular CORDIC algorithm in vector mode.

Parameters

<i>elInstance</i>	The instance of the CORDIC module to use.
<i>x</i>	The x component.
<i>y</i>	The y component.

Returns

A CORDIC_Radical_Type representing the computed angle.

CORDIC_Radical_Type Cordic_Radical_Sub (CORDIC_InstanceType *elInstance*, const float *x*, const float *y*)

Computes the circular radical subtraction using the CORDIC algorithm.

This function computes the circular radical subtraction of two input values *x* and *y* . It uses the CORDIC hardware abstraction layer (HAL) functions to perform the calculations.

Parameters

<i>elInstance</i>	The CORDIC instance identifier.
<i>x</i>	The first input value.
<i>y</i>	The second input value.

Returns

CORDIC_Radical_Type The result of the circular radical subtraction.

CORDIC_SinCos_Type Cordic_Sin_Cos (CORDIC_InstanceType *elInstance*, const float *radian*)

Computes sine and cosine values using the circular CORDIC algorithm.

This function computes the sine and cosine of an angle specified in radians. It uses the circular CORDIC algorithm to perform the computation.

Parameters

<i>elInstance</i>	The instance of the CORDIC module to use.
<i>radian</i>	The angle in radians for which to compute the sine and cosine.

Returns

A **CORDIC_SinCos_Type** structure containing the computed sine and cosine values.

CORDIC_SinhCosh_Type Cordic_Sinh_Cosh (CORDIC_InstanceType *elInstance*, const float *z*)

Computes the hyperbolic sine and cosine of a value using the circular CORDIC algorithm.

This function computes the hyperbolic sine and cosine of a value using the circular CORDIC algorithm in rotate mode.

Parameters

<i>elInstance</i>	The instance of the CORDIC module to use.
<i>z</i>	The value for which to compute the hyperbolic sine and cosine.

Returns

A **CORDIC_SinhCosh_Type** structure containing the computed hyperbolic sine and cosine values.

Module_driver_cpm

Collaboration diagram for Module_driver_cpm:Modules

HwA_cpm

Classes

struct **_CPM_HandleType**

The structure of the CPM processing handle.

struct **CPM_InterruptType**

Cpm interrupt configuration type.

Typedefs

typedef struct **_CPM_HandleType** CPM_HandleType

The structure of the CPM processing handle.

Enumerations

enum **CPM_RetType** { **CPM_STATUS_SUCCESS** = 0U, **CPM_STATUS_PARAM_INVALID** = 1U }

Cpm return type.

enum **FPU_IntType** { **CPM_FPU_FIO** = 1U, **CPM_FPU_FDZ** = 2U, **CPM_FPU_FUF** = 4U, **CPM_FPU_FOF** = 8U, **CPM_FPU_FID** = 16U, **CPM_FPU_FIX** = 32U }

FPU INTERRUPT type.

enum **CPM_MISRCoreActiveType** { **CPM_MISCR_CPUCORE0** = 1U, **CPM_MISCR_CPUCORE1** = 2U, **CPM_MISCR_CPUCORE2** = 4U, **CPM_MISCR_HSM** = 8U }

enum **CPM_InstanceType** { **CPM_INSTANCE_0** = 0U }

The instance index of the CPM peripheral.

Functions

void **CPM_FpuIntMode** (**CPM_HandleType** *pCpmHandle, **FPU_IntType** eIntSrc, bool bEnable)

Configures the CPM module interrupts.

bool **CPM_GetFpuIntStatus** (**CPM_HandleType** *pCpmHandle, **FPU_IntType** eIntSrc)

Get CPM Interrupt occurred flag.

uint32_t **CPM_MiscrGetCoreMode** (**CPM_HandleType** *pCpmHandle)

Get core mode.

uint32_t **CPM_GetFiscr** (**CPM_HandleType** *pCpmHandle)

Gets the value of the Fiscr register.

CPM_RetType **CPM_InitInterrupt** (**CPM_HandleType** *pCpmHandle, const **CPM_InterruptType** *pIntStruct)

Cpm set interrupt.

void **CPM_CommonProcessInterrupt** (**CPM_HandleType** *pCpmHandle)

CPM interrupt function.

Detailed Description

Typedef Documentation

typedef struct _CPM_HandleType CPM_HandleType

The structure of the CPM processing handle.

Enumeration Type Documentation

enum CPM_InstanceType

The instance index of the CPM peripheral.

Enumerator:

CPM_INSTANCE	
_0	

enum CPM_MISCRCoreActiveType

Enumerator:

CPM_MISCR_CP	
UCORE0	
CPM_MISCR_CP	
UCORE1	
CPM_MISCR_CP	
UCORE2	
CPM_MISCR_HS	
M	

enum CPM_RetType

Cpm return type.

Enumerator:

CPM_STATUS_SUCCESS	CPM status success
CPM_STATUS_PARAMETER_INVALID	CPM status parameter invalid

enum FPU_IntType

FPU INTERRUPT type.

This provides constants for FPU interrupt type for use in the FPU functions. Please refer to Reference Manual chapter 14 CPM, it introduce register FISCR for details.

Enumerator:

CPM_FPU_FIO	in function CPM_FpuIntMode, set CPM_FISCR FIOCE bit; in function CPM_GetFpuIntStatus, get CPM_FISCR FIOC Status
CPM_FPU_FDZ	in function CPM_FpuIntMode, set CPM_FISCR FDZCE bit; in function CPM_GetFpuIntStatus, get CPM_FISCR FDZC Status
CPM_FPU_FUF	in function CPM_FpuIntMode, set CPM_FISCR FUFCE bit; in function CPM_GetFpuIntStatus, get CPM_FISCR FUFC Status
CPM_FPU_FOF	in function CPM_FpuIntMode, set CPM_FISCR FOFCE bit; in function CPM_GetFpuIntStatus, get CPM_FISCR FOFC Status
CPM_FPU_FID	in function CPM_FpuIntMode, set CPM_FISCR FIDCE bit; in function CPM_GetFpuIntStatus, get CPM_FISCR FIDC Status

CPM_FPU_FIX	in function CPM_FpuIntMode, set CPM_FISCR FIXCE bit; in function CPM_GetFpuIntStatus, get CPM_FISCR FIXC Status
-------------	---

Function Documentation

void CPM_CommonProcessInterrupt (CPM_HandleType * *pCpmHandle*)

CPM interrupt function.

Parameters

<i>pCpmHandle</i>	CPM handle
-------------------	------------

void CPM_FpuIntMode (CPM_HandleType * *pCpmHandle*, FPU_IntType *eIntSrc*, bool *bEnable*)

Configures the CPM module interrupts.

This function configures the CPM module interrupts to enable/disable various interrupt sources.

Parameters

<i>pCpmHandle</i>	CPM handle
<i>eIntSrc</i>	CPM FPU interrupt type. refer CPM FISCR register
<i>bEnable</i>	true: interrupt enable, false:interrupt disable.

uint32_t CPM_GetFiscr (CPM_HandleType * *pCpmHandle*)

Gets the value of the Flscr register.

This function retrieves the value of the Flscr register using the provided CPM module handle. The Flscr register contains status information about interrupts and exceptions within the CPM module. Reading this register's value provides insight into the current state of interrupts and exceptions.

Parameters

<i>pCpmHandle</i>	Pointer to the CPM module handle. This handle identifies the CPM instance to be accessed.
-------------------	---

Returns

uint32_t The value of the Flscr register.

bool CPM_GetFpuIntStatus (CPM_HandleType * *pCpmHandle*, FPU_IntType *eIntSrc*)

Get CPM Interrupt occurred flag.

This function returns the interrupt flag.

Parameters

<i>pCpmHandle</i>	CPM handle
<i>eIntSrc</i>	CPM FPU interrupt type. refer CPM FISCRegister

Returns

true interrupt occurred

false No interrupt

CPM_RetType CPM_InitInterrupt (CPM_HandleType * *pCpmHandle*, const CPM_InterruptType * *pIntStruct*)

Cpm set interrupt.

Parameters

<i>pCpmHandle</i>	CPM handle
<i>pIntStruct</i>	interrupt structure pointer

Returns

Cpm return type

uint32_t CPM_MiscrGetCoreMode (CPM_HandleType * *pCpmHandle*)

Get core mode.

This function returns the interrupt flag.

Returns

the value of core

Module_driver_crc

Collaboration diagram for Module_driver_crc:Modules

HwA_crc

Classes

struct **Crc_ConfigType**

Defines the crc configuration.

struct **Crc_HandleType**

The structure of the SENT processing handle.

Variables

CRC_WriteDataSwapType Crc_ConfigType::eWriteDataSwap

CRC_ReadDataSwapType Crc_ConfigType::eReadDataSwap

CRC_ReadDataFXORType Crc_ConfigType::eReadDataFXOR

CRC_BitWidthType Crc_ConfigType::eBitWidth

eCrctype Crc_ConfigType::eType

uint8_t Crc_ConfigType::_aligned [2]

uint32_t Crc_ConfigType::u32Polynomial

uint32_t Crc_ConfigType::u32SeedValue

Crc_InstanceType Crc_HandleType::eInstance

bool Crc_HandleType::bInitStatus

struct {

 bool Crc_HandleType::bInitStatus

} Crc_HandleType::tStatus

SSI API Service IDs

#define **CRC_INIT_ID** 1U

#define **CRC_DEINIT_ID** 2U

#define **CRC_GETRST** 3U

#define **CRC_SETCALCDATA8** 4U

#define **CRC_SETCALCDATA16** 5U

#define **CRC_SETCALCDATA32** 6U

#define **CRC_SETINPUTDATA** 7U

SSI Dev Error Code

Error Code of calling ADC apis

```
enum eCrctype { CRC_NONE = 0, CRC_16_CCITT = 1, CRC_16_CCITT_FALSE = 2, CRC_16_MODBUS = 3,  
CRC_16_USB = 4, CRC_32_USER = 5 }
```

```
typedef struct Crc_HandleType Crc_HandleType
```

The structure of the SENT processing handle.

```
void CRC_Init (Crc_HandleType *pCrcHandle, const Crc_ConfigType *const pInitCfg)
```

Initializes the CRC module with the specified configuration.

```
void CRC_DeInit (Crc_HandleType *pCrcHandle)
```

Deinitializes the CRC module and restores default configuration.

```
uint32_t CRC_GetCrcResult (Crc_HandleType *pCrcHandle)
```

get CRC result

```
void CRC_SetCalcData_U8 (Crc_HandleType *pCrcHandle, uint8_t u8Data)
```

set CRC DATA register 8bits

```
void CRC_SetCalcData_U16 (Crc_HandleType *pCrcHandle, uint16_t u16Data)
```

set CRC DATA register 16bits

```
void CRC_SetCalcData_U32 (Crc_HandleType *pCrcHandle, uint32_t u32Data)
```

set CRC low lower register

```
void CRC_SetSeed (Crc_HandleType *pCrcHandle, uint32_t u32SeedVal)
```

set CRC seed value

```
void CRC_SetInputData (Crc_HandleType *pCrcHandle, const uint8_t *pData, uint32_t u32DataSize)
```

set CRC input data

```
#define CRC_E_PARAM_INSTANCE 0x01U
```

```
#define CRC_E_PARAM_POINTER 0x02U
```

```
#define CRC_E_UNINIT 0x03U
#define CRC_E_ALREADY_INIT 0x04U
#define CRC_DEFAULT_POLY (0x1021U)
#define CRC_DEFAULT_INTVAL (0U)
```

Detailed Description

Macro Definition Documentation

```
#define CRC_DEFAULT_INTVAL (0U)
```

```
#define CRC_DEFAULT_POLY (0x1021U)
```

```
#define CRC_DEINIT_ID 2U
```

```
#define CRC_E_ALREADY_INIT 0x04U
```

```
#define CRC_E_PARAM_INSTANCE 0x01U
```

```
#define CRC_E_PARAM_POINTER 0x02U
```

```
#define CRC_E_UNINIT 0x03U
```

```
#define CRC_GETRST 3U
```

```
#define CRC_INIT_ID 1U
```

```
#define CRC_SETCALCDATA16 5U
```

```
#define CRC_SETCALCDATA32 6U
```

```
#define CRC_SETCALCDATA8 4U
```

```
#define CRC_SETINPUTDATA 7U
```

Typedef Documentation

typedef struct Crc_HandleType Crc_HandleType

The structure of the SENT processing handle.

Enumeration Type Documentation

enum eCrctype

Enumerator:

CRC_NONE	
CRC_16_CCITT	
CRC_16_CCITT_ FALSE	
CRC_16_MODB US	
CRC_16_USB	
CRC_32_USER	

Function Documentation

void CRC_DeInit (Crc_HandleType * *pCrCHandle*)

Deinitializes the CRC module and restores default configuration.

This function resets the CRC module to its default state, effectively undoing any previous initialization. It is typically used to release the CRC module before reinitialization or system shutdown.

Parameters

<i>pCrCHandle</i>	Pointer to the CRC handle structure that contains the instance information. Must not be NULL.
-------------------	---

Returns

None.

uint32_t CRC_GetCrcResult (Crc_HandleType * *pCrcHandle*)

get CRC result

Parameters

<i>pCrcHandle</i>	Pointer to the CRC handle structure.
-------------------	--------------------------------------

void CRC_Init (Crc_HandleType * *pCrcHandle*, const Crc_ConfigType *const *pInitCfg*)

Initializes the CRC module with the specified configuration.

This function configures the CRC module according to the provided configuration structure. It must be called before any other CRC operations to ensure the module is properly set up.

Parameters

<i>pCrcHandle</i>	Pointer to the CRC handle structure that contains the instance information.
<i>pInitCfg</i>	Pointer to the CRC configuration structure containing initialization data. Must not be NULL.

Returns

None.

Note

The CRC module must be uninitialized before calling this function to avoid conflicts.

void CRC_SetCalcData_U16 (Crc_HandleType * *pCrcHandle*, uint16_t *u16Data*)

set CRC DATA register 16bits

Parameters

<i>pCrcHandle</i>	Pointer to the CRC handle structure.
<i>u16Data</i>	the data to be set

void CRC_SetCalcData_U32 (Crc_HandleType * *pCrcHandle*, uint32_t *u32Data*)

set CRC low lower register

Parameters

<i>pCrcHandle</i>	Pointer to the CRC handle structure.
<i>u32Data</i>	the data to be set

void CRC_SetCalcData_U8 (Crc_HandleType * *pCrcHandle*, uint8_t *u8Data*)

set CRC DATA register 8bits

Parameters

<i>pCrcHandle</i>	Pointer to the CRC handle structure.
<i>u8Data</i>	the data to be set

void CRC_SetInputData (Crc_HandleType * *pCrcHandle*, const uint8_t * *pData*, uint32_t *u32DataSize*)

set CRC input data

Parameters

<i>pCrcHandle</i>	Pointer to the CRC handle structure.
<i>pData</i>	data pointer
<i>u32DataSize</i>	input data size

void CRC_SetSeed (Crc_HandleType * *pCrcHandle*, uint32_t *u32SeedVal*)

set CRC seed value

Parameters

<i>pCrcHandle</i>	Pointer to the CRC handle structure.
<i>u32SeedVal</i>	seed value

Variable Documentation

uint8_t Crc_ConfigType::_aligned[2]

bool Crc_HandleType::bInitStatus

Crc instance initialize status

bool { ... } ::bInitStatus

Crc instance initialize status

CRC_BitWidthType Crc_ConfigType::eBitWidth

crc bit width

Crc_InstanceType Crc_HandleType::eInstance

Crc instance

CRC_ReadDataFXORType Crc_ConfigType::eReadDataFXOR

complement of reading crc data(none or FXOR)

CRC_ReadDataSwapType Crc_ConfigType::eReadDataSwap

the data swap for read(none,bits,bytes or both)

eCrctype Crc_ConfigType::eType

crc type

CRC_WriteDataSwapType Crc_ConfigType::eWriteDataSwap

the data swap for write(none,bits,bytes or both)

struct { ... } Crc_HandleType::tStatus

uint32_t Crc_ConfigType::u32Polynomial

the polynomial value of crc

uint32_t Crc_ConfigType::u32SeedValue

the seed value of crc

Module_driver_csc

Collaboration diagram for Module_driver_csc:Modules

HwA_csc

Classes

struct **CSC0_ClkoutType**

Data type for register CSC0_CLKOUT_CTRL, mainly focus on CLKOUT setting.

struct **CSC0_AONCLKSRType**

CSC0_AONCLKSR clock source info.

Macros

```
#define CSC0_DEV_ERROR_REPORT STD_OFF
```

```
#define CSC0_ReportDevError(func, error) ReportDevError(CSC0_MODULE_ID, func, error)
```

```
#define CSC0_AONCLK_128K 128000U
```

```
#define CSC0_AONCLK_32K 32000U
```

```
#define CSC0_AONCLK_SOSC_32K 32768U
```

```
#define CSC0_AONCLK_1K 1000U
```

Enumerations

```
enum CSC0_ClockStatusDef { CSC0_CLOCK_UNDEFINE = 0U, CSC0_CLOCK_DISABLE = 1U,  
CSC0_CLOCK_VALID = 2U, CSC0_CLOCK_ERROR = 3U, CSC0_CLOCK_UNKNOWN = 4U }
```

Indicate the clock status for each clock source in clock tree list.

```
enum CSC0_ClkSrcType { CSC0_AON32K_CLK = 0U, CSC0_RTC_CLK = 1U, CSC0_AON_CLK = 2U,  
CSC0_CLKOUT_CLK = 3U, CSC0_END_OF_CLOCKS = 4U }
```

CSC0 clock source type for clock infomation query.

```
enum CSC_RetStatusType { CSC_E_OK = 0U, CSC_E_NOT_OK }
```

CSC return status.

CSC0 API Service IDs

```
#define CSC0_SET_CLOCK_OUT_ID 0x00U
```

```
#define CSC0_SET_AONCLKSRC_ID 0x01U
```

```
#define CSC0_GET_CLOCK_FREQ_ID 0x02U
```

CSC0 Dev Error Code

Error Code of calling CSC0 apis

```
#define CSC0_E_PARAM_POINTER 0x01U
#define CSC0_E_PARAM_OUT_RANGE 0x02U
```

Detailed Description

Macro Definition Documentation

```
#define CSC0_AONCLK_128K 128000U

#define CSC0_AONCLK_1K 1000U

#define CSC0_AONCLK_32K 32000U

#define CSC0_AONCLK_SOSC_32K 32768U

#define CSC0_DEV_ERROR_REPORT STD_OFF

#define CSC0_E_PARAM_OUT_RANGE 0x02U

#define CSC0_E_PARAM_POINTER 0x01U

#define CSC0_GET_CLOCK_FREQ_ID 0x02U

#define CSC0_ReportDevError( func, error) ReportDevError(CSC0_MODULE_ID, func, error)

#define CSC0_SET_AONCLKSRC_ID 0x01U

#define CSC0_SET_CLOCK_OUT_ID 0x00U
```

Enumeration Type Documentation

```
enum CSC0_ClkSrcType
```

CSC0 clock source type for clock information query.

Enumerator:

CSC0_AON32K_CLK	AON32K_CLK
CSC0_RTC_CLK	RTC_CLK
CSC0_AON_CLK	AON_CLK
CSC0_CLKOUT_CLK	CLKOUT_CLK
CSC0_END_OF_CLOCKS	End of CSC0 clocks

enum CSC0_ClockStatusDef

Indicate the clock status for each clock source in clock tree list.

Enumerator:

CSC0_CLOCK_UNDEFINE	Clock status undefined from power on
CSC0_CLOCK_DISABLE	clock source set as disabled
CSC0_CLOCK_VALID	clock set succeed
CSC0_CLOCK_ERROR	clock set failed

CSC0_CLOCK_U NKNOWN	

enum CSC_RetStatusType

CSC return status.

Enumerator:

CSC_E_OK	Return ok
CSC_E_NOT_OK	Return not ok

Module_driver_dma

Collaboration diagram for Module_driver_dma:Modules

HwA_dma

HwA_dmamux

Classes

struct **DMA_InitType**

The configuration parameters of the DMA engine.

struct **DMA_InstanceHandleType**

The structure of the DMA processing handle.

struct **_DMA_HandleType**

struct **DMA_ChannelCfgType**

The configuration parameters of the DMA channel.

struct **DMA_CircularBufferType**

The parameters to configure the DMA channel as a circular buffer engine.

struct **DMA_ChainTransferType**

The paarameters to configure the DMA chained transfer.

Macros

```
#define DMA_CHANNEL_INVALID (0xFFU)
```

Typedefs

```
typedef void(* DMA_TransferCompleteCallbackType) (void *arg)
```

DMA transfer complete callback function prototype.

```
typedef void(* DMA_TransferErrorCallbackType) (void *arg)
```

DMA transfer error callback function prototype.

```
typedef struct _DMA_HandleType DMA_HandleType
```

Enumerations

```
enum DMA_InstanceType { DMA_INSTANCE_0 = 0U, DMA_INSTANCE_1 = 1U, DMA_INSTANCE_MAX =  
DMA_INSTANCE_COUNT }
```

Available DMA Instances.

```
enum DMA_ChannelType { DMA_CHANNEL_0 = 0U, DMA_CHANNEL_1 = 1U, DMA_CHANNEL_2 = 2U,  
DMA_CHANNEL_3 = 3U, DMA_CHANNEL_4 = 4U, DMA_CHANNEL_5 = 5U, DMA_CHANNEL_6 = 6U,  
DMA_CHANNEL_7 = 7U, DMA_CHANNEL_8 = 8U, DMA_CHANNEL_9 = 9U, DMA_CHANNEL_10 = 10U,  
DMA_CHANNEL_11 = 11U, DMA_CHANNEL_12 = 12U, DMA_CHANNEL_13 = 13U, DMA_CHANNEL_14 = 14U,  
DMA_CHANNEL_15 = 15U, DMA_CHANNEL_16 = 16U, DMA_CHANNEL_17 = 17U, DMA_CHANNEL_18 = 18U,  
DMA_CHANNEL_19 = 19U, DMA_CHANNEL_20 = 20U, DMA_CHANNEL_21 = 21U, DMA_CHANNEL_22 = 22U,  
DMA_CHANNEL_23 = 23U, DMA_CHANNEL_24 = 24U, DMA_CHANNEL_25 = 25U, DMA_CHANNEL_26 = 26U,  
DMA_CHANNEL_27 = 27U, DMA_CHANNEL_28 = 28U, DMA_CHANNEL_29 = 29U, DMA_CHANNEL_30 = 30U,  
DMA_CHANNEL_31 = 31U, DMA_CHANNEL_MAX = DMA_CFG_COUNT }
```

Available DMA channels.

```
enum DMA_StatusType { DMA_STATUS_SUCCESS = 0x00U, DMA_STATUS_ERROR = 0x01U,  
DMA_STATUS_BUSY = 0x02U, DMA_STATUS_TIMEOUT = 0x03U, DMA_STATUS_UNSUPPORTED = 0x04U,  
DMA_STATUS_INVALID_ADDRESS = 0x05U, DMA_STATUS_NO_RESOURCE = 0x06U }
```

DMA operation return values.

```
enum DMA_IncrementModeType { DMA_INCREMENT_DISABLE = 0x0U, DMA_INCREMENT_DATA_SIZE =  
0x1U, DMA_INCREMENT_DATA_SIZE_4BYTE_ALIGNED = 0x2U }
```

DMA data increment size.

```
enum DMA_CircularBufferSizeType { DMA_CIRCULAR_BUFFER_SIZE_1B = 0x1U,  
DMA_CIRCULAR_BUFFER_SIZE_2B = 0x2U, DMA_CIRCULAR_BUFFER_SIZE_4B = 0x4U,  
DMA_CIRCULAR_BUFFER_SIZE_8B = 0x8U, DMA_CIRCULAR_BUFFER_SIZE_16B = 0x10U,
```

DMA_CIRCULAR_BUFFER_SIZE_32B = 0x20U, **DMA_CIRCULAR_BUFFER_SIZE_64B** = 0x40U,
DMA_CIRCULAR_BUFFER_SIZE_128B = 0x80U, **DMA_CIRCULAR_BUFFER_SIZE_256B** = 0x100U,
DMA_CIRCULAR_BUFFER_SIZE_512B = 0x200U, **DMA_CIRCULAR_BUFFER_SIZE_1KB** = 0x400U,
DMA_CIRCULAR_BUFFER_SIZE_2KB = 0x800U, **DMA_CIRCULAR_BUFFER_SIZE_4KB** = 0x1000U,
DMA_CIRCULAR_BUFFER_SIZE_8KB = 0x2000U, **DMA_CIRCULAR_BUFFER_SIZE_16KB** = 0x4000U,
DMA_CIRCULAR_BUFFER_SIZE_32KB = 0x8000U, **DMA_CIRCULAR_BUFFER_SIZE_64KB** = 0x10000U,
DMA_CIRCULAR_BUFFER_SIZE_128KB = 0x20000U, **DMA_CIRCULAR_BUFFER_SIZE_256KB** = 0x40000U,
DMA_CIRCULAR_BUFFER_SIZE_512KB = 0x80000U, **DMA_CIRCULAR_BUFFER_SIZE_1MB** = 0x100000U,
DMA_CIRCULAR_BUFFER_SIZE_2MB = 0x200000U, **DMA_CIRCULAR_BUFFER_SIZE_4MB** = 0x400000U,
DMA_CIRCULAR_BUFFER_SIZE_8MB = 0x800000U, **DMA_CIRCULAR_BUFFER_SIZE_16MB** = 0x1000000U,
DMA_CIRCULAR_BUFFER_SIZE_32MB = 0x2000000U, **DMA_CIRCULAR_BUFFER_SIZE_64MB** =
0x4000000U, **DMA_CIRCULAR_BUFFER_SIZE_128MB** = 0x8000000U,
DMA_CIRCULAR_BUFFER_SIZE_256MB = 0x10000000U, **DMA_CIRCULAR_BUFFER_SIZE_512MB** =
0x20000000U, **DMA_CIRCULAR_BUFFER_SIZE_1GB** = 0x40000000U, **DMA_CIRCULAR_BUFFER_SIZE_2GB** =
0x80000000U }

The size of the circular buffer.

enum **DMA_CircularBufferRoleType** { **DMA_CIRCULAR_BUFFER_ROLE_SOURCE** = 0x0U,
DMA_CIRCULAR_BUFFER_ROLE_DESTINATION = 0x1U }

The eBufferRole of the circular buffer.

Functions

void **DMA_Init** (**DMA_InstanceHandleType** *pDmaInstanceHandle, const **DMA_InitType** *const pInitCfg)

Initialize the DMA instance.

void **DMA_DeInit** (**DMA_InstanceHandleType** *pDmaInstanceHandle)

De-initialize the DMA instance.

DMA_StatusType **DMA_InitChannel** (**DMA_HandleType** *pDmaHandle, const **DMA_ChannelCfgType**
*const pChnCfg)

Initialize the DMA channel.

void **DMA_DeinitChannel** (**DMA_HandleType** *pDmaHandle)

De-initialize the DMA channel.

void **DMA_InitChannelInterrupt** (**DMA_HandleType** *pDmaHandle, const **DMA_ChannelCfgType** *const
pChnCfg)

Initialize the DMA channel interrupt.

void DMA_ConfigChainedTransfer (DMA_HandleType *pDmaHandle, const DMA_ChainTransferType *const pChainTransferCfg)

Config the DMA channel chained transfer.

DMA_StatusType DMA_ModifyAddress (DMA_HandleType *pDmaHandle, const volatile void *pSrcBuffer, const volatile void *pDestBuffer)

Modify the source and destination address of the DMA channel.

void DMA_StartChannel (DMA_HandleType *pDmaHandle)

Start the DMA channel transfer.

void DMA_StopChannel (DMA_HandleType *pDmaHandle)

Stop the DMA channel transfer.

DMA_StatusType DMA_CancelTransfer (DMA_InstanceHandleType *pDmaInstanceHandle, bool bGenerateErr)

Cancel the ongoing DMA transfer.

DMA_RequestSourceType DMA_GetChannelRequestSrc (DMA_HandleType *pDmaHandle)

Get the request source of the selected channel.

DMA_RunningStatusType DMA_GetStatus (DMA_InstanceHandleType *pDmaInstanceHandle)

Get the status of the DMA engine.

DMA_RunningStatusType DMA_GetChannelStatus (DMA_HandleType *pDmaHandle)

Get the status of the DMA channel.

void DMA_Transfer_Complete_IRQHandler (DMA_HandleType *pDmaHandle)

The internal interrupt handler for DMA transfer complete.

void DMA_ProcessErrorInterrupt (DMA_InstanceHandleType *pDmaInstanceHandle)

The internal interrupt handler for DMA error complete.

DMA API Service IDs

```
#define DMA_INIT_ID 0U
#define DMA_INIT_CHANNEL_ID 1U
#define DMA_DEINIT_CHANNEL_ID 2U
#define DMA_INIT_CHANNEL_INT_ID 3U
#define DMA_CONFIG_CHAINED_TRANS_ID 4U
#define DMA_CONFIG_MODIFY_ADDR_ID 5U
#define DMA_CONFIG_START_CH_ID 6U
#define DMA_CONFIG_STOP_CH_ID 7U
#define DMA_CONFIG_GET_CH_REQ_SOURCE_ID 8U
#define DMA_CONFIG_GET_CH_STATUS_ID 9U
#define DMA_CONFIG_GET_MON_ID 10U
#define DMA_CONFIG_GET_STATUS_ID 11U
#define DMA_CONFIG_CLEAR_MON_ID 12U
#define DMA_CONFIG_DISABLE_MON_ID 13U
#define DMA_CONFIG_EN_MON_ID 14U
#define DMA_CONFIG_CANCEL_TRANS_ID 15U
```

DMA Dev Error Code

Error Code of calling DMA apis

```
#define DMA_E_PARAM_CHCFG 0x01U
#define DMA_E_PARAM_BLOCKCNT 0x02U
#define DMA_E_PARAM_BLOCKSIZE 0x03U
#define DMA_E_PARAM_DESTCIRCULARBUFSIZE 0x04U
#define DMA_E_PARAM_SRCCIRCULARBUFSIZE 0x05U
#define DMA_E_PARAM_INTERRUPTCFG 0x06U
#define DMA_E_PARAM_CHANNEL 0x07U
#define DMA_E_PARAM_INSTANCE 0x08U
#define DMA_E_PARAM_INITCFG 0x09U
#define DMA_E_PARAM_BLOCKSIZE_DATASIZE 0x0AU
```

Detailed Description

Macro Definition Documentation

```
#define DMA_CHANNEL_INVALID (0xFFU)

#define DMA_CONFIG_CANCEL_TRANS_ID 15U

#define DMA_CONFIG_CHAINED_TRANS_ID 4U

#define DMA_CONFIG_CLEAR_MON_ID 12U

#define DMA_CONFIG_DISABLE_MON_ID 13U

#define DMA_CONFIG_EN_MON_ID 14U

#define DMA_CONFIG_GET_CH_REQ_SOURCE_ID 8U

#define DMA_CONFIG_GET_CH_STATUS_ID 9U

#define DMA_CONFIG_GET_MON_ID 10U

#define DMA_CONFIG_GET_STATUS_ID 11U

#define DMA_CONFIG_MODIFY_ADDR_ID 5U

#define DMA_CONFIG_START_CH_ID 6U

#define DMA_CONFIG_STOP_CH_ID 7U

#define DMA_DEINIT_CHANNEL_ID 2U

#define DMA_E_PARAM_BLOCKCNT 0x02U

#define DMA_E_PARAM_BLOCKSIZE 0x03U

#define DMA_E_PARAM_BLOCKSIZE_DATASIZE 0x0AU

#define DMA_E_PARAM_CHANNEL 0x07U

#define DMA_E_PARAM_CHCFG 0x01U

#define DMA_E_PARAM_DESTCIRCULARBUFSIZE 0x04U

#define DMA_E_PARAM_INITCFG 0x09U
```

```
#define DMA_E_PARAM_INSTANCE 0x08U
```

```
#define DMA_E_PARAM_INTERRUPTCFG 0x06U
```

```
#define DMA_E_PARAM_SRCCIRCULARBUFSIZE 0x05U
```

```
#define DMA_INIT_CHANNEL_ID 1U
```

```
#define DMA_INIT_CHANNEL_INT_ID 3U
```

```
#define DMA_INIT_ID 0U
```

Typedef Documentation

```
typedef struct _DMA_HandleType DMA_HandleType
```

```
typedef void(* DMA_TransferCompleteCallbackType) (void *arg)
```

DMA transfer complete callback function prototype.

```
typedef void(* DMA_TransferErrorCallbackType) (void *arg)
```

DMA transfer error callback function prototype.

Enumeration Type Documentation

```
enum DMA_ChannelType
```

Available DMA channels.

Enumerator:

DMA_CHANNEL _0	
DMA_CHANNEL _1	
DMA_CHANNEL _2	
DMA_CHANNEL _3	
DMA_CHANNEL _4	
DMA_CHANNEL _5	
DMA_CHANNEL _6	
DMA_CHANNEL _7	
DMA_CHANNEL _8	
DMA_CHANNEL _9	
DMA_CHANNEL _10	
DMA_CHANNEL _11	
DMA_CHANNEL _12	
DMA_CHANNEL _13	
DMA_CHANNEL _14	
DMA_CHANNEL _15	
DMA_CHANNEL _16	
DMA_CHANNEL _17	

DMA_CHANNEL _18	
DMA_CHANNEL _19	
DMA_CHANNEL _20	
DMA_CHANNEL _21	
DMA_CHANNEL _22	
DMA_CHANNEL _23	
DMA_CHANNEL _24	
DMA_CHANNEL _25	
DMA_CHANNEL _26	
DMA_CHANNEL _27	
DMA_CHANNEL _28	
DMA_CHANNEL _29	
DMA_CHANNEL _30	
DMA_CHANNEL _31	
DMA_CHANNEL _MAX	

enum DMA_CircularBufferRoleType

The eBufferRole of the circular buffer.

Enumerator:

DMA_CIRCULAR _BUFEER_ROL E_SOURCE	The circular buffer is used as the data source
DMA_CIRCULAR _BUFEER_ROL E_DESTINATION	The circular buffer is used as the data destination

enum DMA_CircularBufferSizeType

The size of the circular buffer.

Note

The start address of the circular buffer should be aligned by the circular buffer size

Enumerator:

DMA_CIRCULAR _BUFFER_SIZE_ 1B	
DMA_CIRCULAR _BUFFER_SIZE_ 2B	
DMA_CIRCULAR _BUFFER_SIZE_ 4B	
DMA_CIRCULAR _BUFFER_SIZE_ 8B	
DMA_CIRCULAR _BUFFER_SIZE_ 16B	
DMA_CIRCULAR _BUFFER_SIZE_ 32B	

DMA_CIRCULAR _BUFFER_SIZE_ 64B	
DMA_CIRCULAR _BUFFER_SIZE_ 128B	
DMA_CIRCULAR _BUFFER_SIZE_ 256B	
DMA_CIRCULAR _BUFFER_SIZE_ 512B	
DMA_CIRCULAR _BUFFER_SIZE_ 1KB	
DMA_CIRCULAR _BUFFER_SIZE_ 2KB	
DMA_CIRCULAR _BUFFER_SIZE_ 4KB	
DMA_CIRCULAR _BUFFER_SIZE_ 8KB	
DMA_CIRCULAR _BUFFER_SIZE_ 16KB	
DMA_CIRCULAR _BUFFER_SIZE_ 32KB	
DMA_CIRCULAR _BUFFER_SIZE_ 64KB	
DMA_CIRCULAR _BUFFER_SIZE_ 128KB	

DMA_CIRCULAR _BUFFER_SIZE_ 256KB	
DMA_CIRCULAR _BUFFER_SIZE_ 512KB	
DMA_CIRCULAR _BUFFER_SIZE_ 1MB	
DMA_CIRCULAR _BUFFER_SIZE_ 2MB	
DMA_CIRCULAR _BUFFER_SIZE_ 4MB	
DMA_CIRCULAR _BUFFER_SIZE_ 8MB	
DMA_CIRCULAR _BUFFER_SIZE_ 16MB	
DMA_CIRCULAR _BUFFER_SIZE_ 32MB	
DMA_CIRCULAR _BUFFER_SIZE_ 64MB	
DMA_CIRCULAR _BUFFER_SIZE_ 128MB	
DMA_CIRCULAR _BUFFER_SIZE_ 256MB	
DMA_CIRCULAR _BUFFER_SIZE_ 512MB	

DMA_CIRCULAR _BUFFER_SIZE_ 1GB	
DMA_CIRCULAR _BUFFER_SIZE_ 2GB	

enum DMA_IncrementModeType

DMA data increment size.

Specify the data increment size after DMA engine transferred a datum

Enumerator:

DMA_INCREMEN T_DISABLE	The data address not increase
DMA_INCREMEN T_DATA_SIZE	The data address increase by the data size
DMA_INCREMEN T_DATA_SIZE_4 BYTE_ALIGNED	The data address increase by the data size, and is 4 byte aligned

enum DMA_InstanceType

Available DMA Instances.

Enumerator:

DMA_INSTANCE _0	
DMA_INSTANCE _1	
DMA_INSTANCE _MAX	

enum DMA_StatusType

DMA operation return values.

Enumerator:

DMA_STATUS_SUCCESS	The DMA operation is succeeded
DMA_STATUS_ERROR	The DMA operation is failed
DMA_STATUS_BUSY	The DMA operation is failed because DMA engine is busy
DMA_STATUS_TIMEOUT	The DMA operation is failed because operation time out
DMA_STATUS_UNSUPPORTED	The DMA configuration parameter is unsupported
DMA_STATUS_INVALID_ADDRESS	The DMA source/destination address is invalid
DMA_STATUS_NO_RESOURCE	The DMA operation is failed because there is no resource

Function Documentation

DMA_StatusType DMA_CancelTransfer (DMA_InstanceHandleType * *pDmaInstanceHandle*, bool *bGenerateErr*)

Cancel the ongoing DMA transfer.

Parameters

<i>pDmaInstanceHandle</i>	DMA handle
<i>bGenerateErr</i>	whether an error will be generated on the transferring channel

Returns

DMA_StatusType whether the transfer is stopped successfully

void DMA_ConfigChainedTransfer (DMA_HandleType * *pDmaHandle*, const DMA_ChainTransferType *const *pChainTransferCfg*)

Config the DMA channel chained transfer.

Parameters

<i>pDmaHandle</i>	the DMA channel to use
<i>pChainTransferCfg</i>	the parameters of the chained channel

void DMA_DeInit (DMA_InstanceHandleType * *pDmaInstanceHandle*)

De-initialize the DMA instance.

Disable the DMA and restore its configure values to default

Parameters

<i>pDmaInstanceHandle</i>	DMA handle
---------------------------	------------

void DMA_DeinitChannel (DMA_HandleType * *pDmaHandle*)

De-initialize the DMA channel.

Parameters

<i>pDmaHandle</i>	the DMA channel to de-initialize
-------------------	----------------------------------

DMA_RequestSourceType DMA_GetChannelRequestSrc (DMA_HandleType * *pDmaHandle*)

Get the request source of the selected channel.

Parameters

<i>pDmaHandle</i>	the DMA channel to get the request source
-------------------	---

Returns

DMA_RequestSourceType the request source of the selected channel

DMA_RunningStatusType DMA_GetChannelStatus (DMA_HandleType * *pDmaHandle*)

Get the status of the DMA channel.

Parameters

<i>pDmaHandle</i>	the DMA channel to get the status
-------------------	-----------------------------------

Returns

DMA_RunningStatusType the status of the DMA channel

DMA_RunningStatusType DMA_GetStatus (DMA_InstanceHandleType * *pDmaInstanceHandle*)

Get the status of the DMA engine.

Returns

DMA_RunningStatusType the status of the DMA engine

void DMA_Init (DMA_InstanceHandleType * *pDmaInstanceHandle*, const DMA_InitType *const *pInitCfg*)

Initialize the DMA instance.

Parameters

<i>pDmaInstanceHandle</i>	DMA handle
<i>pInitCfg</i>	the configuration of the DMA instance

DMA_StatusType DMA_InitChannel (DMA_HandleType * *pDmaHandle*, const DMA_ChannelCfgType *const *pChnCfg*)

Initialize the DMA channel.

Parameters

<i>pDmaHandle</i>	the DMA channel to initialize
<i>pChnCfg</i>	the configuration of the channel

Returns

DMA_StatusType whether the channel is initialized successfully

void DMA_InitChannelInterrupt (DMA_HandleType * *pDmaHandle*, const DMA_ChannelCfgType *const *pChnCfg*)

Initialize the DMA channel interrupt.

Parameters

<i>pDmaHandle</i>	the DMA channel to use
<i>pChnCfg</i>	the configuration of the DMA channel

DMA_StatusType DMA_ModifyAddress (DMA_HandleType * *pDmaHandle*, const volatile void * *pSrcBuffer*, const volatile void * *pDestBuffer*)

Modify the source and destination address of the DMA channel.

Note

This function should be called when the DMA channel is not actively transferring

Parameters

<i>pDmaHandle</i>	the channel to modify
<i>pSrcBuffer</i>	the new source address of data, set NULL to remain unmodified
<i>pDestBuffer</i>	the new destination address of data, set NULL to remain unmodified

Returns

DMA_StatusType whether the address is successfully modified

void DMA_ProcessErrorInterrupt (DMA_InstanceHandleType * *pDmaInstanceHandle*)

The internal interrupt handler for DMA error complete.

Parameters

<i>pDmaInstanceHandle</i>	the handle of the DMA complete interrupt
---------------------------	--

void DMA_StartChannel (DMA_HandleType * *pDmaHandle*)

Start the DMA channel transfer.

If the trigger source for the DMA channel is DMA_REQ_DISABLED, the channel will start immediately after called this function. Otherwise, the channel will start when the trigger source is generated on the channel.

Parameters

<i>pDmaHandle</i>	the DMA channel to start
-------------------	--------------------------

void DMA_StopChannel (DMA_HandleType * *pDmaHandle*)

Stop the DMA channel transfer.

If the trigger source for the DMA channel is DMA_REQ_DISABLED, the channel will stop automatically after the transfer completed. Otherwise, you should call this function manually if you do not want the DMA to be triggered when hardware request is generated.

Note

If there is ongoing transfer on the channel, the channel will be stopped after the transfer completed.

Parameters

<i>pDmaHandle</i>	the DMA channel to stop
-------------------	-------------------------

void DMA_Transfer_Complete_IRQHandler (DMA_HandleType * *pDmaHandle*)

The internal interrupt handler for DMA transfer complete.

Parameters

<i>pDmaHandle</i>	the handle of the DMA complete interrupt
-------------------	--

Module_driver_eim

Collaboration diagram for Module_driver_eim:Modules

HwA_eim

Classes

struct **_EIM_HandleType**

The structure of the EIM processing handle.

struct **EIM_ChCfgType**

Define the max EIM channels.

Typedefs

typedef struct **_EIM_HandleType** **EIM_HandleType**

The structure of the EIM processing handle.

Enumerations

enum **EIM_InstanceType** { **EIM_INSTANCE_0** = 0U }

EIM Instance type.

enum **EIM_StatusType** { **EIM_STATUS_SUCCESS** = 0U, **EIM_STATUS_FAIL** = 1U }

EIM operation return values.

Functions

EIM_StatusType **EIM_Init** (**EIM_HandleType** *const pHandle, const **EIM_ChCfgType** *const pInitCfg)

Initializes the Error Injection Module(EIM) with provided configuration settings.

void **EIM_Deinit** (**EIM_HandleType** *const pHandle)

Deinitializes the Error Injection Module(EIM).

void **EIM_DisableInjection** (**EIM_HandleType** *const pHandle)

Disables error injection in the EIM module.

EIM API Service IDs

```
#define EIM_INIT_ID 0x00U
#define EIM_DEINIT_ID 0x01U
#define EIM_DISABLEINJECTION_ID 0x02U
```

EIM Dev Error Code

```
#define EIM_E_PARAM_NULLPTR 0x01U
#define EIM_E_PARAM_INSTANCE 0x02U
```

Define EIM monitor configuration type

```
#define EIM_LOCKSTEP_MONITOR0 0U
#define EIM_LOCKSTEP_MONITOR1 1U
```

Error injection bus select

```
#define EIM_BUSREG_0 0U
#define EIM_BUSREG_1 1U
#define EIM_BUSREG_2 2U
#define EIM_BUSREG_3 3U
```

Detailed Description

Macro Definition Documentation

```
#define EIM_BUSREG_0 0U
    Sel bus REG0 to injection
```

```
#define EIM_BUSREG_1 1U
    Sel bus REG1 to injection
```

```
#define EIM_BUSREG_2 2U
    Sel bus REG2 to injection
```

```
#define EIM_BUSREG_3 3U
    Sel bus REG3 to injection
```

```
#define EIM_DEINIT_ID 0x01U

#define EIM_DISABLEINJECTION_ID 0x02U

#define EIM_E_PARAM_INSTANCE 0x02U

#define EIM_E_PARAM_NULLPTR 0x01U

#define EIM_INIT_ID 0x00U

#define EIM_LOCKSTEP_MONITOR0 0U
    Lockstep Monitor0 Error Injection

#define EIM_LOCKSTEP_MONITOR1 1U
    Lockstep Monitor1 Error Injection
```

Typedef Documentation

```
typedef struct _EIM_HandleType EIM_HandleType
```

The structure of the EIM processing handle.

Enumeration Type Documentation

```
enum EIM_InstanceType
```

EIM Instance type.

Enumerator:

EIM_INSTANCE_ 0	
--------------------	--

enum EIM_StatusType

EIM operation return values.

Enumerator:

EIM_STATUS_SUCCESS	The EIM status success
EIM_STATUS_FAIL	The EIM status fail

Function Documentation

void EIM_Deinit (EIM_HandleType *const *pHandle*)

Deinitializes the Error Injection Module(EIM).

This function disables the EIM module by clearing all channel configurations, disabling global error injection, and resetting the word channel registers to their default state. Error checking is performed to ensure a valid EIM instance.

Parameters

<i>pHandle</i>	A pointer to the EIM handle structure identifying the EIM instance to deinitialize.
----------------	---

Note

The error reporting macros are conditionally compiled based on EIM_DEV_ERROR_REPORT definition.

void EIM_DisableInjection (EIM_HandleType *const *pHandle*)

Disables error injection in the EIM module.

This function disables error injection in the specified EIM module instance. For lockstep channels, it sets the lockstep configuration.

Parameters

<i>pHandle</i>	Pointer to the EIM handle structure.
----------------	--------------------------------------

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported.

EIM_StatusType EIM_Init (EIM_HandleType *const *pHandle*, const EIM_ChCfgType *const *pInitCfg*)

Initializes the Error Injection Module(EIM) with provided configuration settings.

This function sets up the EIM module according to the configuration specified in *pInitCfg* . It configures the word channels (WORD0 and WORD1), enables the selected EIM channel, and activates global error injection if supported. Error checking is performed to ensure valid pointers and EIM instance.

Parameters

<i>pHandle</i>	A pointer to the EIM handle structure which identifies the EIM instance to initialize.
<i>pInitCfg</i>	A pointer to the initialization configuration structure (EIM_ChCfgType) defining setup parameters for the EIM module.

Returns

EIM_STATUS_SUCCESS if the initialization was successful, otherwise EIM_STATUS_FAIL .

Note

The error reporting macros are conditionally compiled based on EIM_DEV_ERROR_REPORT definition.

Module_driver_enet

Collaboration diagram for Module_driver_enet:Modules

HwA_enet

Classes

struct **ENET_DMADescType**

ENET DMA descriptor type.

struct **ENET_MiiConfigType**

Mdio configuration type.

struct **ENET_TimestampType**

IEEE1588 time stamp type.

struct **ENET_FlexPPSConfigType**

IEEE1588 flexible PPS configuration type.

struct **Enet_PTPConfig**

IEEE1588 time stamp configuration type.

struct **ENET_MMConfig**

Defines MMC counter configuration.

struct **ENET_EstConfig**

ENET EST configuration type.

struct **ENET_GateCtrl**

The ENET EST Gate control type.

struct **ENET_GateCtrlList**

The ENET EST Gate control List type.

struct **ENET_PreemptionConfig**

The ENET EST Frame Preemption configuration type.

struct **ENET_Buffer**

Defines the buffer type for ENET driver.

struct **ENET_TxVlanConfig**

ENET transmit VLAN configuration type.

struct **ENET_OuterVlanCBTI**

ENET Channel based VLAN insertion type.

struct **ENET_TxOption**

Defines the options for the Tx packet.

struct **ENET_RxInfo**

Defines the received buffer information.

struct **ENET_DescList**

DMA descriptor list type for internal use by ENET driver.

struct **ENET_DMACHannelConfig**

DMA channel configuration type.

struct **ENET_DMAConfig**

ENET DMA configuration type.

struct **ENET_MACFilter**

ENET MAC filter configuration type.

struct **ENET_L3L4Filter**

ENET Layer3 Layer4 filter configuration type.

struct **ENET_VlanFilterConfig**

ENET VLAN filter configuration type.

struct **ENET_VlanPerfectFilter**

ENET VLAN perfect filter type.

struct **ENET_FlowControlRxQ**

ENET flow control receive queue configuration type.

struct **ENET_FlowControl**

ENET flow control configuration type.

struct **ENET_MACConfig**

ENET MAC configuration type.

struct **ENET_MTLQueueConfig**

ENET MTL Queue configuration type.

struct **ENET_MTLConfig**

ENET MTL configuration type.

struct **ENET_DescConfig**

ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA.It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache.

struct **ENET_Config**

ENET configuration type.

struct **ENET_DrvHandle**

ENET handle is internal used by ENET driver.

Macros

#define **ENET_DATA_BUS_WIDTH_BYTES** (ENET_DATA_BUS_WIDTH_BITS / 8u)

ENET DMA descriptor length.

#define **ENET_DMA_DESC_LEN_BYTES** (16u)

ENET DMA descriptor length.

#define **ENET_MAX_FRAME_LENGTH** 1536u

Max Ethernet frame length.

#define **ENET_MAC_ADDRESS_LENGTH** 6u

Ethernet MAC address length.

#define **ENET_TRUE** (bool)1u

ENET Boolean true.

#define **ENET_FALSE** (bool)0u

ENET Boolean false.

#define **ENET_DATA_BARRIER()** DMB()

ENET data barrier.

#define **ENET_TDES2_IOC_MASK** (0x80000000u)

TDESC2 Interrupt Enabled on Completion in transmit descriptor 2 read format.

#define **ENET_TDES2_TTSE_MASK** (0x40000000u)

TDESC2 Transmit Timestamp Enable in transmit descriptor 2 read format.

#define **ENET_TDES2_VTIR(x)** (((uint32_t)(((uint32_t)(x)) << 14u)) & 0x0000C000u)

TDESC2 VLAN Tag Insertion or Replacement in transmit descriptor 2 read format.

#define **ENET_TDES2_BUFFER1_LEN(x)** (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x00003FFFu)

TDESC2 Header Length or Buffer 1 Length in transmit descriptor 2 read format.

#define **ENET_TDES2_BUFFER2_LEN(x)** (((uint32_t)(((uint32_t)(x)) << 16u)) & 0x3FFF0000u)

TDESC2 Buffer 2 Length in transmit descriptor 2 read format.

#define **ENET_TDES3_CPC(x)** (((uint32_t)(((uint32_t)(x)) << 26u)) & 0x0C000000u)

CRC Pad Control in transmit descriptor 3 read format.

#define **ENET_TDES3_CIC(x)** (((uint32_t)(((uint32_t)(x)) << 16u)) & 0x00030000u)

Checksum Insertion Control in transmit descriptor 3 read format.

#define **ENET_TDES3_FD_MASK** (0x20000000u)

First Descriptor in transmit descriptor 3 read format.

#define **ENET_TDES3_LD_MASK** (0x10000000u)

Last Descriptor in transmit descriptor 3 read format.

#define **ENET_TDES3_THL(x)** (((uint32_t)(((uint32_t)(x)) << 19u)) & 0x00780000u)

TCP/UDP Header Length in transmit descriptor 3 read format.

#define **ENET_TDES3_TSE_MASK** (0x00040000u)

TCP Segmentation Enable in transmit descriptor 3 read format.

#define **ENET_TDES3_TPL_MASK** (0x0003FFFFu)

TCP Payload Length mask in transmit descriptor 3 read format.

#define **ENET_TDES3_TPL(x)** (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x0003FFFFu)

TCP Payload Length in transmit descriptor 3 read format.

#define **ENET_TDES3_FL_MASK** (0x00007FFFu)

Frame Length mask in transmit descriptor 3 read format.

#define **ENET_TDES3_FL(x)** (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x00007FFFu)

Frame Length in transmit descriptor 3 read format.

#define **ENET_TDES3_TTSS_MASK** (0x00020000u)

Tx Timestamp Status in transmit descriptor 3 write back format.

#define **ENET_TDES2_IVT(x)** (((uint32_t)(((uint32_t)(x)) << 16u)) & 0xFFFF0000u)

Inner VLAN Tag in transmit descriptor 2 context format.

#define **ENET_TDES2_MSS(x)** (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x00003FFFFu)

Maximum Segment Size in transmit descriptor 2 context format.

#define **ENET_TDES3_TCMSSV_MASK** (0x04000000u)

One-Step Timestamp Correction Input or MSS Valid in transmit descriptor 3 context format.

#define **ENET_TDES3_IVTIR**(x) (((uint32_t)(((uint32_t)(x)) << 18u)) & 0x000C0000u)

Inner VLAN Tag Insert or Replace in transmit descriptor 3 context format.

#define **ENET_TDES3_IVLTV_MASK** (0x00020000u)

Inner VLAN Tag Valid in transmit descriptor 3 context format.

#define **ENET_TDES3_VLTV_MASK** (0x00010000u)

VLAN Tag Valid in transmit descriptor 3 context format.

#define **ENET_TDES3_VT**(x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x0000FFFFu)

VLAN Tag in transmit descriptor 3 context format.

#define **ENET_RDES3_IOC_MASK** (0x40000000u)

Interrupt Enabled on Completion in receive descriptor 3 read format.

#define **ENET_RDES3_BUFFER1_VALID_MASK** (0x01000000u)

Buffer 1 Address Valid in receive descriptor 3 read format.

#define **ENET_RDES3_BUFFER2_VALID_MASK** (0x02000000u)

Buffer 2 Address Valid in receive descriptor 3 read format.

#define **ENET_RDES0_INNER_VLAN_TAG_MASK** (0xFFFF0000u)

Inner VLAN Tag mask in receive descriptor 0 write back format.

#define **ENET_RDES0_INNER_VLAN_TAG_SHIFT** (16u)

Inner VLAN Tag shift in receive descriptor 0 write back format.

#define **ENET_RDES0_OUTER_VLAN_TAG_MASK** (0x0000FFFFu)

Outer VLAN Tag mask in receive descriptor 0 write back format.

#define **ENET_RDES1_TIMESTAMP_AVAILABLE_MASK** (0x00004000u)

Timestamp Available in receive descriptor 1 write back format.

#define ENET_RDES1_IP_PAYLOAD_ERROR_MASK (0x00000080u)

IP Payload Error in receive descriptor 1 write back format.

#define ENET_RDES1_IPV6_HEADER_MASK (0x00000020u)

IPv6 header Present in receive descriptor 1 write back format.

#define ENET_RDES1_IPV4_HEADER_MASK (0x00000010u)

IPv4 Header Present in receive descriptor 1 write back format.

#define ENET_RDES1_IP_HEADER_ERROR_MASK (0x00000008u)

IP Header Error in receive descriptor 1 write back format.

#define ENET_RDES1_IP_PAYLOAD_TYPE_MASK (0x00000007u)

Payload Type in receive descriptor 1 write back format.

#define ENET_RDES3_FD_MASK (0x20000000u)

First Descriptor in receive descriptor 3 write back format.

#define ENET_RDES3_LD_MASK (0x10000000u)

Last Descriptor in receive descriptor 3 write back format.

#define ENET_RDES3_PACKET_LENGTH_MASK (0x00007FFFu)

Packet Length in receive descriptor 3 write back format.

#define ENET_RDES3_RDES0_VALID_MASK (0x02000000u)

Receive Status RDES0 Valid in receive descriptor 3 write back format.

#define ENET_RDES3_RDES1_VALID_MASK (0x04000000u)

Receive Status RDES1 Valid in receive descriptor 3 write back format.

#define ENET_RDES3_RDES2_VALID_MASK (0x08000000u)

Receive Status RDES2 Valid in receive descriptor 3 write back format.

```
#define ENET_DES3_OWN_MASK (0x80000000u)
```

Own Bit for all format.

```
#define ENET_DES3_CTXT_MASK (0x40000000u)
```

Context Descriptor bit for all format.

Typedefs

```
typedef struct ENET_DrvHandle ENET_HandleType
```

```
typedef ENET_StatusType(* ENET_EventCallbackType) (const ENET_HandleType *pEnetHandle,  
ENET_EventType eEvent)
```

Callback function invoked when a general event is encountered.

```
typedef ENET_StatusType(* ENET_ChannelCallbackType) (const ENET_HandleType *pEnetHandle, const  
uint8 u8Channel)
```

Callback function invoked when a channel event is encountered.

```
typedef struct Enet_PTPConfig Enet_PTPConfigType
```

IEEE1588 time stamp configuration type.

```
typedef struct ENET_MMCCConfig ENET_MMCCConfigType
```

Defines MMC counter configuration.

```
typedef struct ENET_EstConfig ENET_EstConfigType
```

ENET EST configuration type.

```
typedef struct ENET_GateCtrl ENET_GateCtrlType
```

The ENET EST Gate control type.

```
typedef struct ENET_GateCtrlList ENET_GateCtrlListType
```

The ENET EST Gate control List type.

```
typedef struct ENET_PreemptionConfig ENET_PreemptionConfigType
```

The ENET EST Frame Preemption configuration type.

typedef struct **ENET_Buffer** ENET_BufferType

Defines the buffer type for ENET driver.

typedef struct **ENET_TxVlanConfig** ENET_TxVlanConfigType

ENET transmit VLAN configuration type.

typedef struct **ENET_OuterVlanCBTI** ENET_OuterVlanCBITType

ENET Channel based VLAN insertion type.

typedef struct **ENET_TxOption** ENET_TxOptionType

Defines the options for the Tx packet.

typedef struct **ENET_RxInfo** ENET_RxInfoType

Defines the received buffer information.

typedef struct **ENET_DescList** ENET_DescListType

DMA descriptor list type for internal use by ENET driver.

typedef struct **ENET_DMACHannelConfig** ENET_DMACHannelConfigType

DMA channel configuration type.

typedef struct **ENET_DMAConfig** ENET_DMAConfigType

ENET DMA configuration type.

typedef struct **ENET_MACFilter** ENET_MACFilterType

ENET MAC filter configuration type.

typedef struct **ENET_L3L4Filter** ENET_L3L4FilterType

ENET Layer3 Layer4 filter configuration type.

typedef struct **ENET_VlanFilterConfig** ENET_VlanFilterConfigType

ENET VLAN filter configuration type.

typedef struct **ENET_VlanPerfectFilter** ENET_VlanPerfectFilterType

ENET VLAN perfect filter type.

typedef struct **ENET_FlowControlRxQ** ENET_FlowControlRxQType

ENET flow control receive queue configuration type.

typedef struct **ENET_FlowControl** ENET_FlowControlType

ENET flow control configuration type.

typedef struct **ENET_MACConfig** ENET_MACConfigType

ENET MAC configuration type.

typedef struct **ENET_MTLQueueConfig** ENET_MTLQueueConfigType

ENET MTL Queue configuration type.

typedef struct **ENET_MTLConfig** ENET_MTLConfigType

ENET MTL configuration type.

typedef struct **ENET_DescConfig** ENET_DescConfigType

ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA.It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache.

typedef struct **ENET_Config** ENET_ConfigType

ENET configuration type.

Enumerations

enum **ENET_InstanceType** { **ENET_INSTANCE_0** = 0x0u }

ENET instances.

enum **ENET_ChannelType** { **ENET_CHANNEL_0** = ENET_DMA_CHANNEL_0, **ENET_CHANNEL_1** = ENET_DMA_CHANNEL_1 }

ENET channel index type.

```
enum ENET_StatusType { ENET_STATUS_SUCCESS = 0x00u, ENET_STATUS_ERROR = 0x01u,  
ENET_STATUS_BUSY = 0x02u, ENET_STATUS_TIMEOUT = 0x03u, ENET_STATUS_INVALID_PARAM =  
0x04u, ENET_STATUS_UNSUPPORTED = 0x05u, ENET_STATUS_NOT_READY = 0x06u }
```

ENET operation return values type.

```
enum ENET_StateType { ENET_STATE_UNINIT = 0u, ENET_STATE_INITIALIZED, ENET_STATE_READY,  
ENET_STATE_BUSY, ENET_STATE_STOPPED, ENET_STATE_ERROR, ENET_STATE_WAIT_RELEASE }
```

ENET driver state values type.

```
enum ENET_DMA_SysbusModeType { ENET_DMA_SYSBUS_MODE_FIXED = 0x0u,  
ENET_DMA_SYSBUS_MODE_MIXED = 0x1u }
```

ENET DMA system bus mode type.

```
enum ENET_DMABurstLengthType { ENET_DMA_BURST_LENGTH_1_BEAT = 1u,  
ENET_DMA_BURST_LENGTH_2_BEATS = 2u, ENET_DMA_BURST_LENGTH_4_BEATS = 4u,  
ENET_DMA_BURST_LENGTH_8_BEATS = 8u, ENET_DMA_BURST_LENGTH_16_BEATS = 16u,  
ENET_DMA_BURST_LENGTH_32_BEATS = 32u, ENET_DMA_BURST_LENGTH_64_BEATS = 64u,  
ENET_DMA_BURST_LENGTH_128_BEATS = 128u, ENET_DMA_BURST_LENGTH_256_BEATS = 256u }
```

The burst length for DMA TX and RX.

```
enum ENET_EventType { ENET_EVENT_NONE = 0u, ENET_EVENT_TARGET_TIME0_REACHED,  
ENET_EVENT_TARGET_TIME0_ERROR, ENET_EVENT_TARGET_TIME1_REACHED,  
ENET_EVENT_TARGET_TIME1_ERROR, ENET_EVENT_TARGET_TIME2_REACHED,  
ENET_EVENT_TARGET_TIME2_ERROR, ENET_EVENT_TARGET_TIME3_REACHED,  
ENET_EVENT_TARGET_TIME3_ERROR, ENET_EVENT_MMC_TX_COUNTER,  
ENET_EVENT_MMC_RX_COUNTER }
```

ENET general event type.

```
enum ENET_MiiOptType { ENET_MII_WRITE = ENET_MAC_MDIO_ADDRESS_GOC_0_MASK,  
ENET_MII_READ_INCREMENT = ENET_MAC_MDIO_ADDRESS_GOC_1_MASK, ENET_MII_READ =  
ENET_MAC_MDIO_ADDRESS_GOC_1_MASK | ENET_MAC_MDIO_ADDRESS_GOC_0_MASK }
```

Mdio operation type.

```
enum ENET_AttributeType { ENET_TX_ATTRIBUTES_CRC_PAD_CTRL = 0x00000001u,  
ENET_TX_ATTRIBUTES_SA_INSERTION = 0x00000002u, ENET_TX_ATTRIBUTES_TCP_SEGMENTATION =  
0x00000004u, ENET_TX_ATTRIBUTES_CHECKSUM_INSERTION = 0x00000008u,  
ENET_TX_ATTRIBUTES_VLAN_TAG_CTRL = 0x00000010u,  
ENET_TX_ATTRIBUTES_INNER_VLAN_TAG_CTRL = 0x00000020u,  
ENET_TX_ATTRIBUTES_TIMESTAMP_SNAPSHOT = 0x80000000u,  
ENET_TX_ATTRIBUTES_TIMESTAMP_VALID = 0x80000000u, ENET_RX_ATTRIBUTES_TIMESTAMP_VALID  
= 0x80000000u }
```

Descriptor TX and RX attributes type.

enum **ENET_TxDescCrcPadType** { **ENET_CRC_AND_PAD_INSERTION** = 0u, **ENET_CRC_INSERTION** = 1u, **ENET_CRC_INSERTION_DISABLE** = 2u, **ENET_CRC_REPLACEMENT** = 3u }

TX descriptor CRC and Pad insertion control type.

enum **ENET_TxDescChecksumType** { **ENET_CHECKSUM_INSERTION_DISABLE** = 0u, **ENET_CHECKSUM_INSERTION_IP** = 1u, **ENET_CHECKSUM_INSERTION_PROTO_NO_PSEUDOH** = 2u, **ENET_CHECKSUM_INSERTION_PROTO_PSEUDOH** = 3u }

TX descriptor Checksum insertion control type.

enum **ENET_FilterEnableType** { **ENET_FILTER_DISABLE** = 0u, **ENET_FILTER_ENABLE** = 1u, **ENET_FILTER_INVERSE_ENABLE** = 2u }

ENET filter enable type.

enum **ENET_VLANFilterModeType** { **ENET_VLAN_FILTER_OUTER_CVLAN** = 0u, **ENET_VLAN_FILTER_OUTER_SVLAN** = 1u, **ENET_VLAN_FILTER_INNER_CVLAN** = 2u, **ENET_VLAN_FILTER_INNER_SVLAN** = 3u, **ENET_VLAN_FILTER_OUTER_ANYTYPE** = 4u, **ENET_VLAN_FILTER_INNER_ANYTYPE** = 5u }

ENET filter mode type.

enum **ENET_QueueClassificType** { **ENET_CLISSIFICATION_EXPRESS** = 0u, **ENET_CLISSIFICATION_PREEMPTABLE** = 1u }

ENET Frame Preemption Classification type.

Functions

void **Enet_DefaultConfig** (**ENET_ConfigType** *pConfig)

Get ENET default configuration.

void **Enet_DeInit** (**ENET_HandleType** *pEnetHandle)

Reset ENET driver.

ENET_StatusType **Enet_Init** (**ENET_HandleType** *pEnetHandle, const **ENET_ConfigType** *pConfig)

Initialize ENET driver handle and peripheral.

ENET_StatusType **Enet_Start** (**ENET_HandleType** *pEnetHandle)

Start ENET transmission and reception process in polling mode.

ENET_StatusType **Enet_Stop** (**ENET_HandleType** *pEnetHandle)

Stop ENET transmission and reception process in polling mode.

ENET_StatusType Enet_StartIT (ENET_HandleType *pEnetHandle)

Start ENET transmission and reception in interrupt mode.

ENET_StatusType Enet_StopIT (ENET_HandleType *pEnetHandle)

Stop ENET transmission and reception in interrupt mode.

ENET_StatusType Enet_SendFrame (ENET_HandleType *pEnetHandle, ENET_ChannelType eChannel, ENET_BufferType *pBuffers, ENET_TxOptionType *pOption)

ENET transmit frame.

ENET_StatusType Enet_ReadFrame (ENET_HandleType *pEnetHandle, ENET_ChannelType eChannel, ENET_BufferType *pBuffers, ENET_RxInfoType *pInfo)

Read frame from ENET.

ENET_StatusType Enet_BuildRxDescriptors (ENET_HandleType *pEnetHandle, ENET_ChannelType eChannel)

Build RX Descriptors to DMA.

void Enet_SetMacAddress (ENET_HandleType *pEnetHandle, const uint8_t *pMacAddr, uint8_t u8MacLen)

Set MAC0 address.

void Enet_CheckSumErrorFilter (ENET_HandleType *pEnetHandle, ENET_ChannelType eChannel, bool bEnable)

Enable TCPIP checksum error filter.

void Enet_SetMacHashTable (ENET_HandleType *pEnetHandle, bool bAddOrRemove, const uint8_t *pMacAddr, uint8_t u8MacLen)

Set hash table for MAC hash filter.

void Enet_DefaultMACFilter (ENET_MACFilterType *pMACFilter)

Get default MAC filter configurations.

void Enet_GetMACFilter (const ENET_HandleType *pEnetHandle, ENET_MACFilterType *pMACFilter)

Get current MAC filter configurations.

void **Enet_SetMACFilter** (ENET_HandleType *pEnetHandle, const ENET_MACFilterType *pMACFilter)
Set MAC filter configurations.

void **Enet_DefaultL3L4Filter** (ENET_L3L4FilterType *pL3L4Filter)
Get default layer3 layer4 filter configurations.

void **Enet_GetL3L4Filter** (const ENET_HandleType *pEnetHandle, ENET_MAC_L3L4FilterGroupType
pFilterGroup, ENET_L3L4FilterType *pL3L4Filter)
Get current layer3 layer4 filter configurations.

void **Enet_SetL3L4Filter** (ENET_HandleType *pEnetHandle, ENET_MAC_L3L4FilterGroupType
pFilterGroup, const ENET_L3L4FilterType *pL3L4Filter)
Set layer3 layer4 filter configurations.

void **Enet_EnableL3L4Filter** (ENET_HandleType *pEnetHandle, bool bEnable)
Enable MAC layer3 layer4 filter.

void **Enet_GetVlanFilterConfig** (const ENET_HandleType *pEnetHandle, ENET_VlanFilterConfigType
*pVlanFilterConfig)
Get MAC VLAN filter configurations.

void **Enet_EnableVlanFilter** (ENET_HandleType *pEnetHandle, const ENET_VlanFilterConfigType
*pVlanFilterConfig)
Enable MAC VLAN filter.

void **Enet_SetVlanHashTable** (ENET_HandleType *pEnetHandle, bool bAddOrRemove, uint16_t
u16VlanTag)
Set vlan hash table.

ENET_StatusType **Enet_GetVlanPerfectFilter** (ENET_HandleType *pEnetHandle,
ENET_MAC_VLANFilterGroupType pFilterGroup, ENET_VlanPerfectFilterType *pPerfectFilter)
Get current VLAN perfect filter configurations.

ENET_StatusType Enet_SetVlanPerfectFilter (ENET_HandleType *pEnetHandle,
ENET_MAC_VLANFilterGroupType pFilterGroup, const ENET_VlanPerfectFilterType *pPerfectFilter)
Set VLAN perfect filter configurations.

void **Enet_SetTxOuterVlanConfig** (ENET_HandleType *pEnetHandle, const ENET_TxVlanConfigType
*pVlanConfig)
Set outer VLAN insertion replacement or deletion.

ENET_StatusType Enet_SetTxOuterVlanChannelBasedInsertion (ENET_HandleType *pEnetHandle,
const ENET_OuterVlanCBTIType *pVlanConfig)
Set outer VLAN channel based insertion.

void **Enet_SetTxInnerVlanConfig** (ENET_HandleType *pEnetHandle, const ENET_TxVlanConfigType
*pVlanConfig)
Set inner VLAN insertion replacement or deletion.

void **Enet_RegisterSystemEventCallback** (ENET_HandleType *pEnetHandle, ENET_EventCallbackType
eCallback)
Register ENET system event callback function.

void **Enet_RegisterRxCompleteCallback** (ENET_HandleType *pEnetHandle, ENET_ChannelCallbackType
eCallback)
Register receive frame callback function.

void **Enet_RegisterTxCompleteCallback** (ENET_HandleType *pEnetHandle, ENET_ChannelCallbackType
eCallback)
Register send frame complete callback function.

void **Enet_RegisterWaitTxCompleteCallback** (ENET_HandleType *pEnetHandle,
ENET_ChannelCallbackType eCallback)
Register wait send frame complete callback function.

void **Enet_MDIODefaultConfig** (ENET_MiiConfigType *pMiiConfig)
Get MDIO bus default configuration.

ENET_StatusType Enet_MDIOInit (ENET_HandleType *pEnetHandle, const ENET_MiiConfigType *pMiiConfig)

Initialize the MDIO bus.

ENET_StatusType Enet_MDIORead (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8PhyReg, uint16_t *pData, uint32_t u32TimeoutUs)

Read the PHY register using clause 22.

ENET_StatusType Enet_MDIOWrite (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8PhyReg, uint16_t u16Data, uint32_t u32TimeoutUs)

Write the PHY register using clause 22.

ENET_StatusType Enet_MMDRead (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8Mmd, uint16_t u16PhyReg, uint16_t *pData, uint32_t u32TimeoutUs)

Read the PHY register using clause 45.

ENET_StatusType Enet_MMDWrite (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8Mmd, uint16_t u16PhyReg, uint16_t u16Data, uint32_t u32TimeoutUs)

Write the PHY register using clause 45.

void **Enet_PTPDefaultConfig** (Enet_PTPConfigType *pPTPConfig)

Get PTP default configuration.

ENET_StatusType Enet_PTPStart (ENET_HandleType *pEnetHandle, const Enet_PTPConfigType *pPTPConfig)

Start PTP counter process.

void **Enet_PTPStop** (ENET_HandleType *pEnetHandle)

Stop PTP counter process.

ENET_StatusType Enet_PTPGetTime (const ENET_HandleType *pEnetHandle, ENET_TimestampType *pTime)

Get PTP current system time.

ENET_StatusType Enet_PTPSetTime (ENET_HandleType *pEnetHandle, const ENET_TimestampType *pTime)

Set PTP system time.

ENET_StatusType Enet_PTPUpdateTime (ENET_HandleType *pEnetHandle, const ENET_TimestampType *pTime, bool bSubtraction)

Update PTP system time offset.

ENET_StatusType Enet_PTPAdjustFrequency (ENET_HandleType *pEnetHandle, int PPb)

Adjust the PTP counter frequency.

ENET_StatusType Enet_PTPFixedPPSOut (ENET_HandleType *pEnetHandle, ENET_MAC_PPSSChannelType ePPSChannel, ENET_MAC_PPSPFreqCtrlType eFreqCtrl)

Enable PTP channel PPS out in fixed mode.

ENET_StatusType Enet_PTPFlexiblePPSOut (ENET_HandleType *pEnetHandle, ENET_MAC_PPSSChannelType ePPSChannel, const ENET_FlexPPSConfigType *PPSConfig)

Enable PTP channel PPS out in flexible mode.

ENET_StatusType Enet_TSN_GetEstConfig (ENET_HandleType *pEnetHandle, ENET_EstConfigType *pEstConfig)

Get EST configurations.

ENET_StatusType Enet_TSN_EstConfig (ENET_HandleType *pEnetHandle, const ENET_EstConfigType *pEstConfig)

Set EST configurations.

ENET_StatusType Enet_TSN_EstEnable (ENET_HandleType *pEnetHandle, bool bEnable)

Enable EST processing.

ENET_StatusType Enet_TSN_GetEstGclConfig (ENET_HandleType *pEnetHandle, ENET_GateCtrlListType *pGatelist)

Read the current gate control list.

ENET_StatusType Enet_TSN_EstGclConfig (**ENET_HandleType** *pEnetHandle, const **ENET_GateCtrlListType** *pGatelist)

Install new gate control list.

ENET_StatusType Enet_TSN_PreemptionConfig (**ENET_HandleType** *pEnetHandle, const **ENET_PreemptionConfigType** *pConfig)

Enable or disable the frame preemption function.

void **Enet_GetMMCCConfig** (const **ENET_HandleType** *pEnetHandle, **ENET_MMCCConfigType** *pMMCCConfig)

ENET get MAC management counters configuration.

void **Enet_SetMMCCConfig** (**ENET_HandleType** *pEnetHandle, const **ENET_MMCCConfigType** *pMMCCConfig)

ENET set MAC management counters configuration.

void **Enet_MMCCounterPreset** (**ENET_HandleType** *pEnetHandle)

Preset MAC management counters.

void **Enet_MMCCounterFreeze** (**ENET_HandleType** *pEnetHandle, bool bFreeze)

Freeze MMC counters to their current value.

void **Enet_MMCCounterReset** (**ENET_HandleType** *pEnetHandle)

Reset MAC management counters to zero.

uint32_t **Enet_GetMMCTxInterrupts** (const **ENET_HandleType** *pEnetHandle)

Get MMC transmit counters interrupt status.

uint32_t **Enet_GetMMCRxInterrupts** (const **ENET_HandleType** *pEnetHandle)

Get MMC receive counters interrupt status.

uint32_t **Enet_GetMMCTxCOUNTER** (const **ENET_HandleType** *pEnetHandle, **Enet_MMCTxCOUNTERType** eCounter)

Get ENET MAC management transmit counters.

uint32_t **Enet_GetMMCRxCounter** (const **ENET_HandleType** *pEnetHandle, **Enet_MMCRxCounterType** eCounter)

Get ENET MAC management receive counters.

void **Enet_GetFlowControlConfig** (const **ENET_HandleType** *pEnetHandle, **ENET_FlowControlType** *pFlowControl)

Get flow control configuration.

void **Enet_FlowControlConfig** (**ENET_HandleType** *pEnetHandle, const **ENET_FlowControlType** *pFlowControl)

Configure and enable flow control.

void **Enet_FlowControlSoftwareTrigger** (**ENET_HandleType** *pEnetHandle)

Generate and transmits a single pause packet.

void **Enet_SystemIrqHandler** (**ENET_HandleType** *pEnetHandle)

ENET driver system interrupt handler.

void **Enet_TxIrqHandler** (**ENET_HandleType** *pEnetHandle, **ENET_ChannelType** eChannel)

ENET driver DMA TX channel interrupt handler.

void **Enet_RxIrqHandler** (**ENET_HandleType** *pEnetHandle, **ENET_ChannelType** eChannel)

ENET driver DMA RX channel interrupt handler.

ENET API Service IDs

ENET API Service IDs

```
#define ENET_DEFAULT_CONFIG_SERVICE_ID 0u
```

```
#define ENET_DEINIT_SERVICE_ID 1u
```

```
#define ENET_INIT_SERVICE_ID 2u
```

```
#define ENET_PREPARE_DESC_SERVICE_ID 3u
```

```
#define ENET_START_SERVICE_ID 4u
```

```
#define ENET_STOP_SERVICE_ID 5u
```

```
#define ENET_STARTIT_SERVICE_ID 6u
```

```
#define ENET_STOPIT_SERVICE_ID 7u
```

```

#define ENET_SEND_FRAME_SERVICE_ID 8u
#define ENET_READ_FRAME_SERVICE_ID 9u
#define ENET_BUILD_RXDESC_SERVICE_ID 10u
#define ENET_SET_MACADDR_SERVICE_ID 11u
#define ENET_CKSUM_ERROR_FILTER_SERVICE_ID 12u
#define ENET_SET_MACHASH_SERVICE_ID 13u
#define ENET_DEFAULT_MACFILTER_SERVICE_ID 14u
#define ENET_GET_MACFILTER_SERVICE_ID 15u
#define ENET_SET_MACFILTER_SERVICE_ID 16u
#define ENET_DEFAULT_L3L4FILTER_SERVICE_ID 17u
#define ENET_GET_L3L4FILTER_SERVICE_ID 18u
#define ENET_SET_L3L4FILTER_SERVICE_ID 19u
#define ENET_ENABLE_L3L4FILTER_SERVICE_ID 20u
#define ENET_GET_VLANFILTER_CFG_SERVICE_ID 21u
#define ENET_ENABLE_VLANFILTER_SERVICE_ID 22u
#define ENET_SET_VLANHASH_SERVICE_ID 23u
#define ENET_GET_VLAN_PERFECTFILTER_SERVICE_ID 24u
#define ENET_SET_VLAN_PERFECTFILTER_SERVICE_ID 25u
#define ENET_SET_TXOUTERVLAN_SERVICE_ID 26u
#define ENET_SET_TXOUTVLAN_CBI_SERVICE_ID 27u
#define ENET_SET_TXINNERVLAN_SERVICE_ID 28u
#define ENET_REGISTER_SYSEVENT_SERVICE_ID 29u
#define ENET_REGISTER_RXCOMP_SERVICE_ID 30u
#define ENET_REGISTER_TXCOMP_SERVICE_ID 31u
#define ENET_REGISTER_TXWAIT_SERVICE_ID 32u
#define ENET_MDIO_DEFCONFIG_SERVICE_ID 33u
#define ENET_MDIO_INIT_SERVICE_ID 34u
#define ENET_MDIO_READ_SERVICE_ID 35u
#define ENET_MDIO_WRITE_SERVICE_ID 36u
#define ENET_MMD_READ_SERVICE_ID 37u
#define ENET_MMD_WRITE_SERVICE_ID 38u
#define ENET_PTP_DEFCONFIG_SERVICE_ID 39u
#define ENET_PTP_START_SERVICE_ID 40u
#define ENET_PTP_STOP_SERVICE_ID 41u
#define ENET_PTP_GETTIME_SERVICE_ID 42u
#define ENET_PTP_SETTIME_SERVICE_ID 43u
#define ENET_PTP_UPDATETIME_SERVICE_ID 44u
#define ENET_PTP_ADJUST_FREQ_SERVICE_ID 45u

```

```

#define ENET_PTP_FIXED_PPSOUT_SERVICE_ID 46u
#define ENET_PTP_FLEXIBLE_PPSOUT_SERVICE_ID 47u
#define ENET_GET_MMCCONFIG_SERVICE_ID 48u
#define ENET_SET_MMCCONFIG_SERVICE_ID 49u
#define ENET_MMC_COUNTER_PRESET_SERVICE_ID 50u
#define ENET_MMC_COUNTER_FREEZE_SERVICE_ID 51u
#define ENET_MMC_COUNTER_RESET_SERVICE_ID 52u
#define ENET_MMC_GET_TXINT_SERVICE_ID 53u
#define ENET_MMC_GET_RXINT_SERVICE_ID 54u
#define ENET_MMC_GET_TXCOUNTER_SERVICE_ID 55u
#define ENET_MMC_GET_RXCOUNTER_SERVICE_ID 56u
#define ENET_GET_FLOWCTRL_SERVICE_ID 57u
#define ENET_SET_FLOWCTRL_SERVICE_ID 58u
#define ENET_FLOWCTRL_SOFTTRIG_SERVICE_ID 59u
#define ENET_SYSIRQ_HANDLER_SERVICE_ID 60u
#define ENET_RXIRQ_HANDLER_SERVICE_ID 61u
#define ENET_TXIRQ_HANDLER_SERVICE_ID 62u
#define ENET_GET_ESTCONFIG_SERVICE_ID 63u
#define ENET_EST_CONFIG_SERVICE_ID 64u
#define ENET_EST_ENABLE_SERVICE_ID 65u
#define ENET_EST_GCLCONFIG_SERVICE_ID 66u
#define ENET_EST_GETGCLCONFIG_SERVICE_ID 67u
#define ENET_PREEMPTION_CONFIG_SERVICE_ID 68u

```

ENET DEV Error Code

ENET DEV Error Code

```

#define ENET_E_DRIVER_UNINIT 0u
#define ENET_E_DRIVER_REPEAT_INIT 1u
#define ENET_E_CONTROLLER_MODE 2u
#define ENET_E_PARAM_HANDLE 3u
#define ENET_E_PARAM_INSTANCE 4u
#define ENET_E_PARAM_POINTER 5u
#define ENET_E_PARAM_TXCHANNEL 6u
#define ENET_E_PARAM_RXCHANNEL 7u
#define ENET_E_PARAM_GROUP 8u
#define ENET_E_PARAM_ASSERTION 9u
#define ENET_E_PARAM_MACLEN 10u
#define ENET_E_PARAM_PPSCHANNEL 11u

```

Detailed Description

Macro Definition Documentation

#define ENET_BUILD_RXDESC_SERVICE_ID 10u

#define ENET_CKSUM_ERROR_FILTER_SERVICE_ID 12u

#define ENET_DATA_BARRIER() DMB()

ENET data barrier.

#define ENET_DATA_BUS_WIDTH_BYTES (ENET_DATA_BUS_WIDTH_BITS / 8u)

ENET DMA descriptor length.

#define ENET_DEFAULT_CONFIG_SERVICE_ID 0u

#define ENET_DEFAULT_L3L4FILTER_SERVICE_ID 17u

#define ENET_DEFAULT_MACFILTER_SERVICE_ID 14u

#define ENET_DEINIT_SERVICE_ID 1u

#define ENET_DES3_CTXT_MASK (0x40000000u)

Context Descriptor bit for all format.

#define ENET_DES3_OWN_MASK (0x80000000u)

Own Bit for all format.

#define ENET_DMA_DESC_LEN_BYTES (16u)

ENET DMA descriptor length.

```
#define ENET_E_CONTROLLER_MODE 2u

#define ENET_E_DRIVER_REPEAT_INIT 1u

#define ENET_E_DRIVER_UNINIT 0u

#define ENET_E_PARAM_ASSERTION 9u

#define ENET_E_PARAM_GROUP 8u

#define ENET_E_PARAM_HANDLE 3u

#define ENET_E_PARAM_INSTANCE 4u

#define ENET_E_PARAM_MACLEN 10u

#define ENET_E_PARAM_POINTER 5u

#define ENET_E_PARAM_PPSCHANNEL 11u

#define ENET_E_PARAM_RXCHANNEL 7u

#define ENET_E_PARAM_TXCHANNEL 6u

#define ENET_ENABLE_L3L4FILTER_SERVICE_ID 20u

#define ENET_ENABLE_VLANFILTER_SERVICE_ID 22u

#define ENET_EST_CONFIG_SERVICE_ID 64u

#define ENET_EST_ENABLE_SERVICE_ID 65u

#define ENET_EST_GCLCONFIG_SERVICE_ID 66u

#define ENET_EST_GETGCLCONFIG_SERVICE_ID 67u

#define ENET_FALSE (bool)0u
```

ENET Boolean false.

#define ENET_FLOWCTRL_SOFTTRIG_SERVICE_ID 59u

#define ENET_GET_ESTCONFIG_SERVICE_ID 63u

#define ENET_GET_FLOWCTRL_SERVICE_ID 57u

#define ENET_GET_L3L4FILTER_SERVICE_ID 18u

#define ENET_GET_MACFILTER_SERVICE_ID 15u

#define ENET_GET_MMCCONFIG_SERVICE_ID 48u

#define ENET_GET_VLAN_PERFECTFILTER_SERVICE_ID 24u

#define ENET_GET_VLANFILTER_CFG_SERVICE_ID 21u

#define ENET_INIT_SERVICE_ID 2u

#define ENET_MAC_ADDRESS_LENGTH 6u

Ethernet MAC address length.

#define ENET_MAX_FRAME_LENGTH 1536u

Max Ethernet frame length.

```
#define ENET_MDIO_DEFCONFIG_SERVICE_ID 33u

#define ENET_MDIO_INIT_SERVICE_ID 34u

#define ENET_MDIO_READ_SERVICE_ID 35u

#define ENET_MDIO_WRITE_SERVICE_ID 36u

#define ENET_MMC_COUNTER_FREEZE_SERVICE_ID 51u

#define ENET_MMC_COUNTER_PRESET_SERVICE_ID 50u

#define ENET_MMC_COUNTER_RESET_SERVICE_ID 52u

#define ENET_MMC_GET_RXCOUNTER_SERVICE_ID 56u

#define ENET_MMC_GET_RXINT_SERVICE_ID 54u

#define ENET_MMC_GET_TXCOUNTER_SERVICE_ID 55u

#define ENET_MMC_GET_TXINT_SERVICE_ID 53u

#define ENET_MMD_READ_SERVICE_ID 37u

#define ENET_MMD_WRITE_SERVICE_ID 38u

#define ENET_PREEMPTION_CONFIG_SERVICE_ID 68u

#define ENET_PREPARE_DESC_SERVICE_ID 3u

#define ENET_PTP_ADJUST_FREQ_SERVICE_ID 45u

#define ENET_PTP_DEFCONFIG_SERVICE_ID 39u

#define ENET_PTP_FIXED_PPSOUT_SERVICE_ID 46u

#define ENET_PTP_FLEXIBLE_PPSOUT_SERVICE_ID 47u

#define ENET_PTP_GETTIME_SERVICE_ID 42u

#define ENET_PTP_SETTIME_SERVICE_ID 43u

#define ENET_PTP_START_SERVICE_ID 40u
```

#define ENET_PTP_STOP_SERVICE_ID 41u

#define ENET_PTP_UPDATETIME_SERVICE_ID 44u

#define ENET_RDES0_INNER_VLAN_TAG_MASK (0xFFFF0000u)

Inner VLAN Tag mask in receive descriptor 0 write back format.

#define ENET_RDES0_INNER_VLAN_TAG_SHIFT (16u)

Inner VLAN Tag shift in receive descriptor 0 write back format.

#define ENET_RDES0_OUTER_VLAN_TAG_MASK (0x0000FFFFu)

Outer VLAN Tag mask in receive descriptor 0 write back format.

#define ENET_RDES1_IP_HEADER_ERROR_MASK (0x00000008u)

IP Header Error in receive descriptor 1 write back format.

#define ENET_RDES1_IP_PAYLOAD_ERROR_MASK (0x00000080u)

IP Payload Error in receive descriptor 1 write back format.

#define ENET_RDES1_IP_PAYLOAD_TYPE_MASK (0x00000007u)

Payload Type in receive descriptor 1 write back format.

#define ENET_RDES1_IPV4_HEADER_MASK (0x00000010u)

IPv4 Header Present in receive descriptor 1 write back format.

#define ENET_RDES1_IPV6_HEADER_MASK (0x00000020u)

IPv6 header Present in receive descriptor 1 write back format.

#define ENET_RDES1_TIMESTAMP_AVAILABLE_MASK (0x00004000u)

Timestamp Available in receive descriptor 1 write back format.

#define ENET_RDES3_BUFFER1_VALID_MASK (0x01000000u)

Buffer 1 Address Valid in receive descriptor 3 read format.

#define ENET_RDES3_BUFFER2_VALID_MASK (0x02000000u)

Buffer 2 Address Valid in receive descriptor 3 read format.

#define ENET_RDES3_FD_MASK (0x20000000u)

First Descriptor in receive descriptor 3 write back format.

#define ENET_RDES3_IOC_MASK (0x40000000u)

Interrupt Enabled on Completion in receive descriptor 3 read format.

#define ENET_RDES3_LD_MASK (0x10000000u)

Last Descriptor in receive descriptor 3 write back format.

#define ENET_RDES3_PACKET_LENGTH_MASK (0x00007FFFu)

Packet Length in receive descriptor 3 write back format.

#define ENET_RDES3_RDES0_VALID_MASK (0x02000000u)

Receive Status RDES0 Valid in receive descriptor 3 write back format.

#define ENET_RDES3_RDES1_VALID_MASK (0x04000000u)

Receive Status RDES1 Valid in receive descriptor 3 write back format.

```
#define ENET_RDES3_RDES2_VALID_MASK (0x08000000u)
```

Receive Status RDES2 Valid in receive descriptor 3 write back format.

```
#define ENET_READ_FRAME_SERVICE_ID 9u

#define ENET_REGISTER_RXCOMP_SERVICE_ID 30u

#define ENET_REGISTER_SYSEVENT_SERVICE_ID 29u

#define ENET_REGISTER_TXCOMP_SERVICE_ID 31u

#define ENET_REGISTER_TXWAIT_SERVICE_ID 32u

#define ENET_RXIRQ_HANDLER_SERVICE_ID 61u

#define ENET_SEND_FRAME_SERVICE_ID 8u

#define ENET_SET_FLOWCTRL_SERVICE_ID 58u

#define ENET_SET_L3L4FILTER_SERVICE_ID 19u

#define ENET_SET_MACADDR_SERVICE_ID 11u

#define ENET_SET_MACFILTER_SERVICE_ID 16u

#define ENET_SET_MACHASH_SERVICE_ID 13u

#define ENET_SET_MMCCONFIG_SERVICE_ID 49u

#define ENET_SET_TXINNERVLAN_SERVICE_ID 28u

#define ENET_SET_TXOUTERVLAN_SERVICE_ID 26u

#define ENET_SET_TXOUTVLAN_CBI_SERVICE_ID 27u

#define ENET_SET_VLAN_PERFECTFILTER_SERVICE_ID 25u

#define ENET_SET_VLANHASH_SERVICE_ID 23u

#define ENET_START_SERVICE_ID 4u

#define ENET_STARTIT_SERVICE_ID 6u

#define ENET_STOP_SERVICE_ID 5u

#define ENET_STOPIT_SERVICE_ID 7u
```


#define ENET_SYSIRQ_HANDLER_SERVICE_ID 60u

#define ENET_TDES2_BUFFER1_LEN(x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x00003FFFu)

TDESC2 Header Length or Buffer 1 Length in transmit descriptor 2 read format.

#define ENET_TDES2_BUFFER2_LEN(x) (((uint32_t)(((uint32_t)(x)) << 16u)) & 0x3FFF0000u)

TDESC2 Buffer 2 Length in transmit descriptor 2 read format.

#define ENET_TDES2_IOC_MASK (0x80000000u)

TDESC2 Interrupt Enabled on Completion in transmit descriptor 2 read format.

#define ENET_TDES2_IVT(x) (((uint32_t)(((uint32_t)(x)) << 16u)) & 0xFFFF0000u)

Inner VLAN Tag in transmit descriptor 2 context format.

#define ENET_TDES2_MSS(x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x00003FFFu)

Maximum Segment Size in transmit descriptor 2 context format.

#define ENET_TDES2_TTSE_MASK (0x40000000u)

TDESC2 Transmit Timestamp Enable in transmit descriptor 2 read format.

#define ENET_TDES2_VTIR(x) (((uint32_t)(((uint32_t)(x)) << 14u)) & 0x0000C000u)

TDESC2 VLAN Tag Insertion or Replacement in transmit descriptor 2 read format.

#define ENET_TDES3_CIC(x) (((uint32_t)(((uint32_t)(x)) << 16u)) & 0x00030000u)

Checksum Insertion Control in transmit descriptor 3 read format.

#define ENET_TDES3_CPC(x) (((uint32_t)(((uint32_t)(x)) << 26u)) & 0x0C000000u)

CRC Pad Control in transmit descriptor 3 read format.

```
#define ENET_TDES3_FD_MASK (0x20000000u)
```

First Descriptor in transmit descriptor 3 read format.

```
#define ENET_TDES3_FL( x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x00007FFFu)
```

Frame Length in transmit descriptor 3 read format.

```
#define ENET_TDES3_FL_MASK (0x00007FFFu)
```

Frame Length mask in transmit descriptor 3 read format.

```
#define ENET_TDES3_IVLTV_MASK (0x00020000u)
```

Inner VLAN Tag Valid in transmit descriptor 3 context format.

```
#define ENET_TDES3_IVTIR( x) (((uint32_t)(((uint32_t)(x)) << 18u)) & 0x000C0000u)
```

Inner VLAN Tag Insert or Replace in transmit descriptor 3 context format.

```
#define ENET_TDES3_LD_MASK (0x10000000u)
```

Last Descriptor in transmit descriptor 3 read format.

```
#define ENET_TDES3_TCMSSV_MASK (0x04000000u)
```

One-Step Timestamp Correction Input or MSS Valid in transmit descriptor 3 context format.

```
#define ENET_TDES3_THL( x) (((uint32_t)(((uint32_t)(x)) << 19u)) & 0x00780000u)
```

TCP/UDP Header Length in transmit descriptor 3 read format.

```
#define ENET_TDES3_TPL( x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x0003FFFFu)
```

TCP Payload Length in transmit descriptor 3 read format.

```
#define ENET_TDES3_TPL_MASK (0x0003FFFFu)
```

TCP Payload Length mask in transmit descriptor 3 read format.

```
#define ENET_TDES3_TSE_MASK (0x00040000u)
```

TCP Segmentation Enable in transmit descriptor 3 read format.

```
#define ENET_TDES3_TTSS_MASK (0x00020000u)
```

Tx Timestamp Status in transmit descriptor 3 write back format.

```
#define ENET_TDES3_VLTV_MASK (0x00010000u)
```

VLAN Tag Valid in transmit descriptor 3 context format.

```
#define ENET_TDES3_VT( x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x0000FFFFu)
```

VLAN Tag in transmit descriptor 3 context format.

```
#define ENET_TRUE (bool)1u
```

ENET Boolean true.

```
#define ENET_TXIRQ_HANDLER_SERVICE_ID 62u
```

Typedef Documentation

```
typedef struct ENET_Buffer ENET_BufferType
```

Defines the buffer type for ENET driver.

typedef ENET_StatusType(* ENET_ChannelCallbackType) (const ENET_HandleType *pEnetHandle, const uint8 u8Channel)

Callback function invoked when a channel event is encountered.

typedef struct ENET_Config ENET_ConfigType

ENET configuration type.

typedef struct ENET_DescConfig ENET_DescConfigType

ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA.It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache.

For RX DMA descriptor: Buffer is necessary. DescList Buffer will be accessed by MCU and DMA.It is best to place these buffers in a non-cacheable area, or users need to manually refresh the cache.

typedef struct ENET_DescList ENET_DescListType

DMA descriptor list type for internal use by ENET driver.

typedef struct ENET_DMACHannelConfig ENET_DMACHannelConfigType

DMA channel configuration type.

typedef struct ENET_DMAConfig ENET_DMAConfigType

ENET DMA configuration type.

typedef struct ENET_EstConfig ENET_EstConfigType

ENET EST configuration type.

```
typedef ENET_StatusType(* ENET_EventCallbackType) (const ENET_HandleType *pEnetHandle,  
ENET_EventType eEvent)
```

Callback function invoked when a general event is encountered.

```
typedef struct ENET_FlowControlRxQ  ENET_FlowControlRxQType
```

ENET flow control receive queue configuration type.

```
typedef struct ENET_FlowControl  ENET_FlowControlType
```

ENET flow control configuration type.

```
typedef struct ENET_GateCtrlList  ENET_GateCtrlListType
```

The ENET EST Gate control List type.

```
typedef struct ENET_GateCtrl  ENET_GateCtrlType
```

The ENET EST Gate control type.

```
typedef struct ENET_DrvHandle ENET_HandleType
```

```
typedef struct ENET_L3L4Filter  ENET_L3L4FilterType
```

ENET Layer3 Layer4 filter configuration type.

typedef struct ENET_MACConfig ENET_MACConfigType

ENET MAC configuration type.

typedef struct ENET_MACFilter ENET_MACFilterType

ENET MAC filter configuration type.

typedef struct ENET_MMConfig ENET_MMConfigType

Defines MMC counter configuration.

typedef struct ENET_MTLConfig ENET_MTLConfigType

ENET MTL configuration type.

typedef struct ENET_MTLQueueConfig ENET_MTLQueueConfigType

ENET MTL Queue configuration type.

typedef struct ENET_OuterVlanCBTI ENET_OuterVlanCBTIType

ENET Channel based VLAN insertion type.

typedef struct ENET_PreemptionConfig ENET_PreemptionConfigType

The ENET EST Frame Preemption configuration type.

typedef struct Enet_PTPConfig Enet_PTPConfigType

IEEE1588 time stamp configuration type.

typedef struct ENET_RxInfo ENET_RxInfoType

Defines the received buffer information.

typedef struct ENET_TxOption ENET_TxOptionType

Defines the options for the Tx packet.

typedef struct ENET_TxVlanConfig ENET_TxVlanConfigType

ENET transmit VLAN configuration type.

typedef struct ENET_VlanFilterConfig ENET_VlanFilterConfigType

ENET VLAN filter configuration type.

typedef struct ENET_VlanPerfectFilter ENET_VlanPerfectFilterType

ENET VLAN perfect filter type.

Enumeration Type Documentation

enum ENET_AttributeType

Descriptor TX and RX attributes type.

Enumerator:

ENET_TX_ATTRIBUTES_CRC_PAD_CTRL	TX Option Attributes CRC and PAD insertion.
ENET_TX_ATTRIBUTES_SA_INSERTION	Source MAC address insertion.
ENET_TX_ATTRIBUTES_TCP_SEGMENTATION	TCP segmentation not supported by hardware.
ENET_TX_ATTRIBUTES_CHECKSUM_INSERTION	Checksum insertion control.
ENET_TX_ATTRIBUTES_VLAN_TAG_CTRL	Add outer VLAN tag.
ENET_TX_ATTRIBUTES_INNER_VLAN_TAG_CTRL	Add inner VLAN tag.
ENET_TX_ATTRIBUTES_TIMESTAMP_SNAPSHOT	Enable TX PTP packet snap shoot.
ENET_TX_ATTRIBUTES_TIMESTAMP_VALID	TX Result Attributes Transmitted PTP packet time stamp valid.
ENET_RX_ATTRIBUTES_TIMESTAMP_VALID	RX Attributes Received PTP packet time stamp valid.

enum ENET_ChannelType

ENET channel index type.

Enumerator:

ENET_CHANNEL _0	
ENET_CHANNEL _1	

enum ENET_DMA_SysbusModeType

ENET DMA system bus mode type.

Enumerator:

ENET_DMA_SYS BUS_MODE_FIX ED	
ENET_DMA_SYS BUS_MODE_MIX ED	

enum ENET_DMABurstLengthType

The burst length for DMA TX and RX.

Enumerator:

ENET_DMA_BU RST_LENGTH_1 _BEAT	
--------------------------------------	--

ENET_DMA_BUFFER_LENGTH_2_BEATS	
ENET_DMA_BUFFER_LENGTH_4_BEATS	
ENET_DMA_BUFFER_LENGTH_8_BEATS	
ENET_DMA_BUFFER_LENGTH_16_BEATS	
ENET_DMA_BUFFER_LENGTH_32_BEATS	
ENET_DMA_BUFFER_LENGTH_64_BEATS	
ENET_DMA_BUFFER_LENGTH_128_BEATS	
ENET_DMA_BUFFER_LENGTH_256_BEATS	

enum ENET_EventType

ENET general event type.

Enumerator:

ENET_EVENT_NONE	
ENET_EVENT_TARGET_TIME0_REACHED	PTP PPS target time register0 reached

ENET_EVENT_T ARGET_TIME0_E RROR	PTP PPS target time register0 elapsed
ENET_EVENT_T ARGET_TIME1_R EACHED	PTP PPS target time register1 reached
ENET_EVENT_T ARGET_TIME1_E RROR	PTP PPS target time register1 elapsed
ENET_EVENT_T ARGET_TIME2_R EACHED	PTP PPS target time register2 reached
ENET_EVENT_T ARGET_TIME2_E RROR	PTP PPS target time register2 elapsed
ENET_EVENT_T ARGET_TIME3_R EACHED	PTP PPS target time register3 reached
ENET_EVENT_T ARGET_TIME3_E RROR	PTP PPS target time register3 elapsed
ENET_EVENT_M MC_TX_COUNT ER	MMC TX counters interrupt is triggered, user must call Enet_GetMMCTxInterrupts to get the interrupt source, and call Enet_GetMMCTxCounter to clear the MMC interrupt bit
ENET_EVENT_M MC_RX_COUNT ER	MMC TX counters interrupt is triggered, user must call Enet_GetMMCTxInterrupts to get the interrupt source, and call Enet_GetMMCRxCounter to clear the MMC interrupt bit

enum ENET_FilterEnableType

ENET filter enable type.

Enumerator:

ENET_FILTER_DISABLE	Disable filter,all packets are forwarded to the application
ENET_FILTER_ENABLE	Enable filter,the matched packets are forwarded to the application
ENET_FILTER_INVERSE_ENABLE	Enable inverse filter,the packets match failed are forwarded to the application

enum ENET_InstanceType

ENET instances.

Enumerator:

ENET_INSTANCE_0	
-----------------	--

enum ENET_MiiOptType

Mdio operation type.

Enumerator:

ENET_MII_WRITE	
ENET_MII_READ_INCREMENT	
ENET_MII_READ	

enum ENET_QueueClassificType

ENET Frame Preemption Classification type.

Enumerator:

ENET_CLISSIFICATION_EXPRESS	MTL Queue is classified as express
ENET_CLISSIFICATION_PREEMPTABLE	MTL Queue is classified as preemptable

enum ENET_StateType

ENET driver state values type.

Enumerator:

ENET_STATE_UNINIT	The ENET driver not initialized
ENET_STATE_INITIALIZED	The ENET driver initialized
ENET_STATE_READY	The ENET driver is ready for transmission and reception
ENET_STATE_BUSY	The ENET driver is busy for transmission for reception
ENET_STATE_STOPPED	The ENET driver is stopped
ENET_STATE_ERROR	The ENET driver error occurred
ENET_STATE_WAIT_RELEASE	The ENET driver wait Enet_BuildRxDescriptors

enum ENET_StatusType

ENET operation return values type.

Enumerator:

ENET_STATUS_SUCCESS	The ENET operation is succeed
ENET_STATUS_ERROR	The ENET operation is failed
ENET_STATUS_BUSY	The ENET operation is failed because ENET is busy
ENET_STATUS_TIMEOUT	The ENET operation is failed because of time out
ENET_STATUS_INVALID_PARAM	The ENET operation is failed because of the param is invalid
ENET_STATUS_UNSUPPORTED	The ENET configuration parameter is unsupported
ENET_STATUS_NOT_READY	The ENET operation is not ready

enum ENET_TxDescChecksumType

TX descriptor Checksum insertion control type.

Enumerator:

ENET_CHECKSUM_INSERTION_DISABLE	Checksum insertion is disabled.
ENET_CHECKSUM_INSERTION_IP	IP header checksum calculation and insertion are enabled.
ENET_CHECKSUM_INSERTION_PROTO_NO_PSEUDOHEADER	Protocol checksum calculation and insertion are enabled, but pseudo-header checksum is not calculated in hardware.

ENET_CHECKSUM_INSERTION_PROTO_PSEUDO_HEADER	Protocol checksum calculation and insertion are enabled, and pseudo-header checksum is calculated in hardware.
---	--

enum ENET_TxDescCrcPadType

TX descriptor CRC and Pad insertion control type.

Enumerator:

ENET_CRC_AND_PAD_INSERTION	Appends the CRC at the end of the transmitted packet of length greater than or equal to 60 bytes. Appends padding and CRC to a packet with length less than 60 bytes.
ENET_CRC_INSERTION	Appends the CRC at the end of the transmitted packet but it does not append padding. The application should ensure that the padding bytes are present in the packet being transferred.
ENET_CRC_INSERTION_DISABLE	Does not append the CRC at the end of the transmitted packet. The application should ensure that the padding and CRC bytes are present in the packet being transferred.
ENET_CRC_REPLACEMENT	Replaces the last four bytes of the transmitted packet with recalculated CRC bytes. The application should ensure that the padding and CRC bytes are present in the packet being transferred.

enum ENET_VLANFilterModeType

ENET filter mode type.

Enumerator:

ENET_VLAN_FILTER_OUTER_CVLAN	Outer VLAN Tag Comparison,only filter packets with C-VLAN tag
ENET_VLAN_FILTER_OUTER_SVLAN	Outer VLAN Tag Comparison,only filter packets with S-VLAN tag
ENET_VLAN_FILTER_INNER_CVLAN	Inner VLAN Tag Comparison,only filter packets with C-VLAN tag
ENET_VLAN_FILTER_INNER_SVLAN	Inner VLAN Tag Comparison,only filter packets with S-VLAN tag
ENET_VLAN_FILTER_OUTER_ANYTYPE	Outer VLAN Tag Comparison, do not check VLAN type
ENET_VLAN_FILTER_INNER_ANYTYPE	Inner VLAN Tag Comparison, do not check VLAN type

Function Documentation

ENET_StatusType Enet_BuildRxDescriptors (ENET_HandleType * *pEnetHandle*, ENET_ChannelType *eChannel*)

Build RX Descriptors to DMA.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eChannel</i>	DMA channel to use

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Descriptors are released to DMA successfully
<i>other</i>	values Descriptors are released to DMA failed

void Enet_CheckSumErrorFilter (ENET_HandleType * *pEnetHandle*, ENET_ChannelType *eChannel*, bool *bEnable*)

Enable TCPIP checksum error filter.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eChannel</i>	ENET channel
in	<i>bEnable</i>	Enable MAC to drop packets with checksum error

void Enet_DefaultConfig (ENET_ConfigType * *pConfig*)

Get ENET default configuration.

Parameters

out	<i>pConfig</i>	ENET default configurations
-----	----------------	-----------------------------

void Enet_DefaultL3L4Filter (ENET_L3L4FilterType * *pL3L4Filter*)

Get default layer3 layer4 filter configurations.

Parameters

out	<i>pL3L4Filter</i>	Default layer3 layer4 filter configurations
-----	--------------------	---

void Enet_DefaultMACFilter (ENET_MACFilterType * *pMACFilter*)

Get default MAC filter configurations.

Parameters

in	<i>pMACFilter</i>	MAC filter Default configurations
----	-------------------	-----------------------------------

void Enet_DeInit (ENET_HandleType * *pEnetHandle*)

Reset ENET driver.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

void Enet_EnableL3L4Filter (ENET_HandleType * *pEnetHandle*, bool *bEnable*)

Enable MAC layer3 layer4 filter.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>bEnable</i>	true: enable layer3 layer4 filter, false: disable

void Enet_EnableVlanFilter (ENET_HandleType * *pEnetHandle*, const ENET_VlanFilterConfigType * *pVlanFilterConfig*)

Enable MAC VLAN filter.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pVlanFilterConfig</i>	MAC VLAN filter configurations

void Enet_FlowControlConfig (ENET_HandleType * *pEnetHandle*, const ENET_FlowControlType * *pFlowControl*)

Configure and enable flow control.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pFlowControl</i>	MAC flow control configuration

void Enet_FlowControlSoftwareTrigger (ENET_HandleType * *pEnetHandle*)

Generate and transmits a single pause packet.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

void Enet_GetFlowControlConfig (const ENET_HandleType * *pEnetHandle*, ENET_FlowControlType * *pFlowControl*)

Get flow control configuration.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pFlowControl</i>	MAC flow control configuration

void Enet_GetL3L4Filter (const ENET_HandleType * *pEnetHandle*, ENET_MAC_L3L4FilterGroupType *pFilterGroup*, ENET_L3L4FilterType * *pL3L4Filter*)

Get current layer3 layer4 filter configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pFilterGroup</i>	Layer3 layer4 filter group
in	<i>pL3L4Filter</i>	Current layer3 layer4 filter configurations

void Enet_GetMACFilter (const ENET_HandleType * *pEnetHandle*, ENET_MACFilterType * *pMACFilter*)

Get current MAC filter configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pMACFilter</i>	MAC filter Current configurations

void Enet_GetMMCConfig (const ENET_HandleType * *pEnetHandle*, ENET_MMCConfigType * *pMMCConfig*)

ENET get MAC management counters configuration.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pMMCConfig</i>	MAC management configuration type

uint32_t Enet_GetMMCRxCounter (const ENET_HandleType * *pEnetHandle*, Enet_MMCRxCounterType *eCounter*)

Get ENET MAC management receive counters.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eCounter</i>	MAC management receive counter type

uint32_t Enet_GetMMCRxInterrupts (const ENET_HandleType * *pEnetHandle*)

Get MMC receive counters interrupt status.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

Note

User must read the respective counter to clear the MMC interrupt bit.

uint32_t Enet_GetMMCTxCounter (const ENET_HandleType * *pEnetHandle*, Enet_MMCTxCounterType *eCounter*)

Get ENET MAC management transmit counters.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eCounter</i>	MAC management transmit counter type

uint32_t Enet_GetMMCTxInterrupts (const ENET_HandleType * *pEnetHandle*)

Get MMC transmit counters interrupt status.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

Note

User must read the respective counter to clear the MMC interrupt bit.

void Enet_GetVlanFilterConfig (const ENET_HandleType * *pEnetHandle*, ENET_VlanFilterConfigType * *pVlanFilterConfig*)

Get MAC VLAN filter configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pVlanFilterConfig</i>	MAC VLAN filter configurations

**ENET_StatusType Enet_GetVlanPerfectFilter (ENET_HandleType * *pEnetHandle*,
ENET_MAC_VLANFilterGroupType *pFilterGroup*, ENET_VlanPerfectFilterType * *pPerfectFilter*)**

Get current VLAN perfect filter configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pFilterGroup</i>	VLAN perfect filter group
out	<i>pPerfectFilter</i>	VLAN perfect filter configurations

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Get VLAN perfect filter configurations successfully
<i>other</i>	values Get VLAN perfect filter configurations failed

ENET_StatusType Enet_Init (ENET_HandleType * *pEnetHandle*, const ENET_ConfigType * *pConfig*)

Initialize ENET driver handle and peripheral.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pConfig</i>	ENET configure parameters

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	ENET initialize succeeded
<i>other</i>	values ENET initialize failed

void Enet_MDIODefaultConfig (ENET_MiiConfigType * *pMiiConfig*)

Get MDIO bus default configuration.

Parameters

out	<i>pMiiConfig</i>	MDIO configure parameters
-----	-------------------	---------------------------

ENET_StatusType Enet_MDIOInit (ENET_HandleType * *pEnetHandle*, const ENET_MiiConfigType * *pMiiConfig*)

Initialize the MDIO bus.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pMiiConfig</i>	MDIO configure parameters

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Initialize the MDIO bus successfully
<i>other</i>	values Initialize the MDIO bus failed

ENET_StatusType Enet_MDIORead (ENET_HandleType * *pEnetHandle*, uint8_t *u8PhyAddr*, uint8_t *u8PhyReg*, uint16_t * *pData*, uint32_t *u32TimeoutUs*)

Read the PHY register using clause 22.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>u8PhyAddr</i>	PHY device address
in	<i>u8PhyReg</i>	PHY register address
in	<i>pData</i>	Data buffer
in	<i>u32TimeoutUs</i>	Wait time out value in microsecond

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	DMIO read successfully
<i>other</i>	values DMIO read failed

ENET_StatusType Enet_MDIOWrite (ENET_HandleType * *pEnetHandle*, uint8_t *u8PhyAddr*, uint8_t *u8PhyReg*, uint16_t *u16Data*, uint32_t *u32TimeoutUs*)

Write the PHY register using clause 22.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>u8PhyAddr</i>	PHY device address
in	<i>u8PhyReg</i>	PHY register address
in	<i>u16Data</i>	Data to write

in	<i>u32TimeoutUs</i>	Wait time out value in microsecond
----	---------------------	------------------------------------

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	DMIO write successfully
<i>other</i>	values DMIO write failed

void Enet_MMCCounterFreeze (ENET_HandleType * *pEnetHandle*, bool *bFreeze*)

Freeze MMC counters to their current value.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>bFreeze</i>	Enable or disable Freeze

void Enet_MMCCounterPreset (ENET_HandleType * *pEnetHandle*)

Preset MAC management counters.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

void Enet_MMCCounterReset (ENET_HandleType * *pEnetHandle*)

Reset MAC management counters to zero.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

ENET_StatusType Enet_MMDRead (ENET_HandleType * *pEnetHandle*, uint8_t *u8PhyAddr*, uint8_t *u8Mmd*, uint16_t *u16PhyReg*, uint16_t * *pData*, uint32_t *u32TimeoutUs*)

Read the PHY register using clause 45.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>u8PhyAddr</i>	PHY device address
in	<i>u8Mmd</i>	MMD device address
in	<i>u16PhyReg</i>	PHY register address
in	<i>pData</i>	Data buffer
in	<i>u32TimeoutUs</i>	Wait time out value in microsecond

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	DMIO read successfully
<i>other</i>	values DMIO read failed

ENET_StatusType Enet_MMDWrite (ENET_HandleType * *pEnetHandle*, uint8_t *u8PhyAddr*, uint8_t *u8Mmd*, uint16_t *u16PhyReg*, uint16_t *u16Data*, uint32_t *u32TimeoutUs*)

Write the PHY register using clause 45.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>u8PhyAddr</i>	PHY device address
in	<i>u8Mmd</i>	MMD device address
in	<i>u16PhyReg</i>	PHY register address
in	<i>u16Data</i>	Data to write
in	<i>u32TimeoutUs</i>	Wait time out value in microsecond

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	DMIO write successfully
<i>other</i>	values DMIO write failed

ENET_StatusType Enet_PTPAdjustFrequency (ENET_HandleType * *pEnetHandle*, int *PPb*)

Adjust the PTP counter frequency.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>PPb</i>	PTP frequency bias in PPB

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Adjust PTP frequency successfully
<i>ENET_STATUS_INVALID_PARAM</i>	Parameter invalid
<i>ENET_STATUS_UNSUPPORTED</i>	Time stamp update mode unsupported
<i>ENET_STATUS_NOT_READY</i>	Time stamp not ready

void Enet_PTPDefaultConfig (Enet_PTPConfigType * *pPTPConfig*)

Get PTP default configuration.

Parameters

out	<i>pPTPConfig</i>	PTP default configurations
-----	-------------------	----------------------------

**ENET_StatusType Enet_PTPFixedPPSOut (ENET_HandleType * *pEnetHandle*,
ENET_MAC_PPSChannelType *ePPSChannel*, ENET_MAC_PPSFreqCtrlType *eFreqCtrl*)**

Enable PTP channel PPS out in fixed mode.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>ePPSChannel</i>	PTP PPS channel

in	<i>eFreqCtrl</i>	PTP fixed PPS out frequency control In binary rollover mode: PPS frequency = 2 ^{eFreqCtrl} In digital rollover mode: PPS frequency = (eFreqCtrl == 0) ? 1 : 2 ^(eFreqCtrl - 1)
----	------------------	---

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Enable PTP channel PPS out successfully
<i>other</i>	values Enable PTP channel PPS out failed

**ENET_StatusType Enet_PTPTFlexiblePPSOut (ENET_HandleType * *pEnetHandle*,
ENET_MAC_PPSTChannelType *ePPSTChannel*, const ENET_FlexPPSTConfigType * *PPSTConfig*)**

Enable PTP channel PPS out in flexible mode.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>ePPSTChannel</i>	PTP PPS channel
in	<i>PPSTConfig</i>	PTP flexible PPS out configuration

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Enable PTP flexible PPS out successfully
<i>other</i>	values Enable PTP flexible PPS out failed

Note

The Enet_PTPTSetTime API may be called by application before target time reached, in this case, the target time reached interrupt will not be triggered, so users need to use some method to handle this event, such as a timeout mechanism.

ENET_StatusType Enet_PTPTGetTime (const ENET_HandleType * *pEnetHandle*, ENET_TimestampType * *pTime*)

Get PTP current system time.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
out	<i>pTime</i>	PTP system time

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Get PTP system time successfully
<i>other</i>	values Get PTP system time failed

ENET_StatusType Enet_PTPSetTime (ENET_HandleType * *pEnetHandle*, const ENET_TimestampType * *pTime*)

Set PTP system time.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pTime</i>	Time to set

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Set PTP time successfully
<i>other</i>	values Set PTP time failed

ENET_StatusType Enet_PTPStart (ENET_HandleType * *pEnetHandle*, const Enet_PTPConfigType * *pPTPConfig*)

Start PTP counter process.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pPTPConfig</i>	PTP initialize configurations

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Start PTP successfully
<i>other</i>	values Start PTP failed

void Enet_PTPStop (ENET_HandleType * *pEnetHandle*)

Stop PTP counter process.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

ENET_StatusType Enet_PTPUpdateTime (ENET_HandleType * *pEnetHandle*, const ENET_TimestampType * *pTime*, bool *bSubtraction*)

Update PTP system time offset.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pTime</i>	Time offset
in	<i>bSubtraction</i>	Subtract from PTP system time,or add to PTP system time

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Update PTP time successfully
<i>other</i>	values Update PTP time failed

ENET_StatusType Enet_ReadFrame (ENET_HandleType * *pEnetHandle*, ENET_ChannelType *eChannel*, ENET_BufferType * *pBuffers*, ENET_RxInfoType * *pInfo*)

Read frame from ENET.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eChannel</i>	DMA channel to use
out	<i>pBuffers</i>	Buffer chain to receive data,The valid buffers must be ended with NULL pointer. The member TotalLength of the first buffer will be assigned by this driver. The member Length of available buffers will be assigned by this driver. The member Data of available buffers will be filled by DMA.
out	<i>pInfo</i>	Received frame information,set NULL if not needed

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Receive frame successfully
<i>other</i>	values Receive frame failed

void Enet_RegisterRxCompleteCallback (ENET_HandleType * *pEnetHandle*, ENET_ChannelCallbackType *eCallback*)

Register receive frame callback function.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eCallback</i>	Callback function

void Enet_RegisterSystemEventCallback (ENET_HandleType * *pEnetHandle*, ENET_EventCallbackType *eCallback*)

Register ENET system event callback function.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eCallback</i>	Callback function

void Enet_RegisterTxCompleteCallback (ENET_HandleType * *pEnetHandle*, ENET_ChannelCallbackType *eCallback*)

Register send frame complete callback function.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eCallback</i>	Callback function

void Enet_RegisterWaitTxCompleteCallback (ENET_HandleType * *pEnetHandle*, ENET_ChannelCallbackType *eCallback*)

Register wait send frame complete callback function.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eCallback</i>	Callback function

void Enet_RxIrqHandler (ENET_HandleType * *pEnetHandle*, ENET_ChannelType *eChannel*)

ENET driver DMA RX channel interrupt handler.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eChannel</i>	ENET DMA channel

ENET_StatusType Enet_SendFrame (ENET_HandleType * *pEnetHandle*, ENET_ChannelType *eChannel*, ENET_BufferType * *pBuffers*, ENET_TxOptionType * *pOption*)

ENET transmit frame.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eChannel</i>	DMA channel to use

in	<i>pBuffers</i>	Data buffer chain to transmit,The valid buffers must be ended with NULL pointer. The member TotalLength of the first buffer will be used by this driver. The member Length of all buffers will be used by this driver. The member Data of all buffers will be accessed by DMA. It is recommended that the Data buffer be placed in a non-cacheable area, otherwise users need to manually refresh the cache.
in,out	<i>pOption</i>	Transmit options and results,set NULL if not needed

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Send frame successfully
<i>other</i>	values Send frame failed

void Enet_SetL3L4Filter (ENET_HandleType * *pEnetHandle*, ENET_MAC_L3L4FilterGroupType *pFilterGroup*, const ENET_L3L4FilterType * *pL3L4Filter*)

Set layer3 layer4 filter configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pFilterGroup</i>	Layer3 layer4 filter group
in	<i>pL3L4Filter</i>	Layer3 layer4 filter configurations

void Enet_SetMacAddress (ENET_HandleType * *pEnetHandle*, const uint8_t * *pMacAddr*, uint8_t *u8MacLen*)

Set MAC0 address.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pMacAddr</i>	MAC address
in	<i>u8MacLen</i>	MAC address length

void Enet_SetMACFilter (ENET_HandleType * *pEnetHandle*, const ENET_MACFilterType * *pMACFilter*)

Set MAC filter configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pMACFilter</i>	MAC filter configurations

void Enet_SetMacHashTable (ENET_HandleType * *pEnetHandle*, bool *bAddOrRemove*, const uint8_t * *pMacAddr*, uint8_t *u8MacLen*)

Set hash table for MAC hash filter.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>bAddOrRemove</i>	true: add,false: remove
in	<i>pMacAddr</i>	MAC address
in	<i>u8MacLen</i>	MAC address length

void Enet_SetMMCCConfig (ENET_HandleType * *pEnetHandle*, const ENET_MMCCConfigType * *pMMCCConfig*)

ENET set MAC management counters configuration.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pMMCCConfig</i>	MAC management configuration type

void Enet_SetTxInnerVlanConfig (ENET_HandleType * *pEnetHandle*, const ENET_TxVlanConfigType * *pVlanConfig*)

Set inner VLAN insertion replacement or deletion.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pVlanConfig</i>	Inner VLAN operation parameters

**ENET_StatusType Enet_SetTxOuterVlanChannelBasedInsertion (ENET_HandleType * *pEnetHandle*,
const ENET_OuterVlanCBTIType * *pVlanConfig*)**

Set outer VLAN channel based insertion.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pVlanConfig</i>	Outer VLAN channel based configurations

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	successful
<i>other</i>	values failed

**void Enet_SetTxOuterVlanConfig (ENET_HandleType * *pEnetHandle*, const ENET_TxVlanConfigType *
pVlanConfig)**

Set outer VLAN insertion replacement or deletion.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pVlanConfig</i>	Outer VLAN operation parameters

**void Enet_SetVlanHashTable (ENET_HandleType * *pEnetHandle*, bool *bAddOrRemove*, uint16_t
u16VlanTag)**

Set vlan hash table.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>bAddOrRemove</i>	true add to hash table,false remove from hash table
in	<i>u16VlanTag</i>	VLAN tag ID

**ENET_StatusType Enet_SetVlanPerfectFilter (ENET_HandleType * *pEnetHandle*,
ENET_MAC_VLANFilterGroupType *pFilterGroup*, const ENET_VlanPerfectFilterType * *pPerfectFilter*)**

Set VLAN perfect filter configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pFilterGroup</i>	VLAN perfect filter group
in	<i>pPerfectFilter</i>	VLAN perfect filter configurations

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Set VLAN perfect filter configurations successfully
<i>other</i>	values Set VLAN perfect filter configurations failed

ENET_StatusType Enet_Start (ENET_HandleType * *pEnetHandle*)

Start ENET transmission and reception process in polling mode.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	ENET start succeeded
<i>other</i>	values ENET start failed

ENET_StatusType Enet_StartIT (ENET_HandleType * *pEnetHandle*)

Start ENET transmission and reception in interrupt mode.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	ENET start succeeded
<i>other</i>	values ENET start failed

ENET_StatusType Enet_Stop (ENET_HandleType * *pEnetHandle*)

Stop ENET transmission and reception process in polling mode.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	ENET Stop succeeded
<i>other</i>	values ENET Stop failed

ENET_StatusType Enet_StopIT (ENET_HandleType * *pEnetHandle*)

Stop ENET transmission and reception in interrupt mode.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	ENET Stop succeeded
<i>other</i>	values ENET Stop failed

void Enet_SystemIrqHandler (ENET_HandleType * *pEnetHandle*)

ENET driver system interrupt handler.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
----	--------------------	--------------------

ENET_StatusType Enet_TSN_EstConfig (ENET_HandleType * *pEnetHandle*, const ENET_EstConfigType * *pEstConfig*)

Set EST configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pEstConfig</i>	EST configurations

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Set EST configurations successfully
<i>other</i>	values Set EST configurations failed

ENET_StatusType Enet_TSN_EstEnable (ENET_HandleType * *pEnetHandle*, bool *bEnable*)

Enable EST processing.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>bEnable</i>	Enable or disable

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Enable EST function successfully
<i>other</i>	values Enable EST function failed

ENET_StatusType Enet_TSN_EstGclConfig (ENET_HandleType * *pEnetHandle*, const ENET_GateCtrlListType * *pGatelist*)

Install new gate control list.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pGatelist</i>	Gate control list

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Install new GCL successfully
<i>other</i>	values Install new GCL failed

ENET_StatusType Enet_TSN_GetEstConfig (ENET_HandleType * *pEnetHandle*, ENET_EstConfigType * *pEstConfig*)

Get EST configurations.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
out	<i>pEstConfig</i>	Structure to save configurations

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Get EST configurations successfully
<i>other</i>	values Get EST configurations failed

**ENET_StatusType Enet_TSN_GetEstGclConfig (ENET_HandleType * *pEnetHandle*,
ENET_GateCtrlListType * *pGatelist*)**

Read the current gate control list.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
out	<i>pGatelist</i>	Gate control list

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Read GCL successfully
<i>other</i>	values Read GCL failed

**ENET_StatusType Enet_TSN_PreemptionConfig (ENET_HandleType * *pEnetHandle*, const
ENET_PreemptionConfigType * *pConfig*)**

Enable or disable the frame preemption function.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>pConfig</i>	Preemption configuration structure

Returns

ENET_StatusType

Return values

<i>ENET_STATUS_SUCCESS</i>	Enable or disable frame preemption successfully
<i>other</i>	values Enable or disable frame preemption failed

void Enet_TxIrqHandler (ENET_HandleType * *pEnetHandle*, ENET_ChannelType *eChannel*)

ENET driver DMA TX channel interrupt handler.

Parameters

in	<i>pEnetHandle</i>	ENET driver handle
in	<i>eChannel</i>	ENET DMA channel

Module_driver_erm

Collaboration diagram for Module_driver_erm:Modules

HwA_erm

Classes

struct **_ERM_HandleType**

The structure of the ERM processing handle.

struct **ERM_CfgType**

The structure of the ERM Init configuration.

Typedefs

typedef struct **_ERM_HandleType** **ERM_HandleType**

The structure of the ERM processing handle.

Enumerations

enum **ERM_InstanceType** { **ERM_INSTANCE_0** = 0U }

ERM Instance type.

enum **ERM_StatusType** { **ERM_STATUS_SUCCESS** = 0U, **ERM_STATUS_FAIL** = 1U }

ERM operation return values.

Functions

ERM_StatusType **ERM_Init** (**ERM_HandleType** *const pHandle, const **ERM_CfgType** *const pInitCfg)

Initializes the Error Reporting Module (ERM) for the specified instance.

ERM_StatusType **ERM_DeInit** (**ERM_HandleType** *const pHandle)

Deinitializes the ERM module.

void **ERM_CommonIRQHandler** (**ERM_HandleType** *const pHandle)

Handles common Error Reporting Module (ERM) interrupts and invokes error callbacks.

ERM API Service IDs

#define **ERM_INIT_ID** 0x00U

#define **ERM_DEINIT_ID** 0x01U

#define **ERM_COMMONIRQHANDLER_ID** 0x02U

ERM Dev Error Code

Error Code of calling ERM apis

#define **ERM_E_PARAM_NULLPTR** 0x01U

#define **ERM_E_PARAM_INSTANCE** 0x02U

#define **ERM_NON_CORRECTABLE_IRQ** 1U

ERM error interrupt define.

#define **ERM_SINGLE_CORRECTION_IRQ** 2U

Detailed Description

Macro Definition Documentation

#define ERM_COMMONIRQHANDLER_ID 0x02U

#define ERM_DEINIT_ID 0x01U

#define ERM_E_PARAM_INSTANCE 0x02U

#define ERM_E_PARAM_NULLPTR 0x01U

#define ERM_INIT_ID 0x00U

#define ERM_NON_CORRECTABLE_IRQ 1U

ERM error interrupt define.

select Non-correctable interrupt report

#define ERM_SINGLE_CORRECTION_IRQ 2U

select single correction interrupt report

Typedef Documentation

typedef struct _ERM_HandleType ERM_HandleType

The structure of the ERM processing handle.

Enumeration Type Documentation

enum ERM_InstanceType

ERM Instance type.

Enumerator:

ERM_INSTANCE _0	
--------------------	--

enum ERM_StatusType

ERM operation return values.

Enumerator:

ERM_STATUS_S UCCESS	The ERM status success
ERM_STATUS_F AIL	The ERM status fail

Function Documentation**void ERM_CommonIRQHandler (ERM_HandleType *const *pHandle*)**

Handles common Error Reporting Module (ERM) interrupts and invokes error callbacks.

This function serves as the common interrupt handler for the ERM, checking for errors in various channels (CTCM, DTCM, PFLASH, optionally DFLASH, and DMA) and invoking the error callback function when an error is detected. It also clears the status flags after processing the interrupts.

Parameters

<i>pHandle</i>	Pointer to an ERM handle structure containing the instance information and error callback
----------------	---

Note

This function is designed to be used within an interrupt context.

Support for DFLASH depends on the compile-time configuration ERM_DFLASH_SUPPORT .

ERM_StatusType ERM_DeInit (ERM_HandleType *const *pHandle*)

Deinitializes the ERM module.

This function deinitializes the specified ERM module instance by clearing its control and status registers.

Parameters

<i>pHandle</i>	Pointer to the ERM handle structure.
----------------	--------------------------------------

Returns

ERM_StatusType Indicates the status of the operation (ERM_STATUS_SUCCESS or ERM_STATUS_FAIL).

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported, and the function returns ERM_STATUS_FAIL.

ERM_StatusType ERM_Init (ERM_HandleType *const *pHandle*, const ERM_CfgType *const *pInitCfg*)

Initializes the Error Reporting Module (ERM) for the specified instance.

This function initializes the ERM module with the provided configuration structure. It performs parameter validation, sets up the control register based on the configuration, and clears the status. If an error occurs or the handle is invalid, an error report will be generated, and the function will return a failure status.

Parameters

<i>pHandle</i>	Pointer to an ERM handle structure containing the instance information
<i>pInitCfg</i>	Pointer to an ERM configuration structure with desired settings

Returns

- Status of the initialization operation
- ERM_STATUS_SUCCESS: Initialization successful
 - ERM_STATUS_FAIL: Initialization failed due to an error

Note

Error reporting is enabled when ERM_DEV_ERROR_REPORT is set to STD_ON .

Module_driver_fciic

Collaboration diagram for Module_driver_fciic:Modules

HwA_fciic

Classes

struct **FCIIC_TxDataType**

Structure for holding transmit data for FCIIC operations.

struct **FCIIC_RxDataType**

Structure for holding received data from FCIIC operations.

struct **FCIIC_MasterConfigType**

Structure for configuring the FCIIC Master.

struct **FCIIC_SlaveConfigType**

Configuration structure for initializing the FCIIC (Fast Communication Interface IC) Slave module.

struct **FCIIC_MasterTransferType**

Structure for defining a master transfer in the FCIIC.

struct **FCIIC_SlaveTransferType**

Structure for defining a slave transfer in the FCIIC.

struct **_FCIIC_DmaConfig**

Structure for configuring DMA for I2C Master and Slave operations.

struct **_FCIIC_MasterHandle**

*Structure for handling the FCIIC Master operations. Implements : **_FCIIC_MasterHandle**.*

struct **_FCIIC_SlaveHandle**

*Structure for handling the FCIIC Slave operations. Implements : **_FCIIC_SlaveHandle**.*

struct **FCIIC_InterruptType**

Structure for configuring FCIIC interrupts.

Macros

#define **FCIIC_E_PARAM_NULLPTR** 0x01U

Error code indicating a null pointer parameter.

#define **FCIIC_E_PARAM_INSTANCE** 0x02U

Error code indicating an invalid instance parameter.

#define **FCIIC_E_PARAM_CHANNEL** 0x03U

Error code indicating an invalid channel parameter.

#define **FCIIC_DIR_MASK** (0x1U)

Mask to extract the direction bit from the status register.

#define **FCIIC_MASTER_CMD_SIZE** (0x8U)

Defines the size of the master command in bytes.

Typedefs

typedef struct **_FCIIC_DmaConfig** **FCIIC_DmaConfigType**

Structure for configuring DMA for I2C Master and Slave operations.

typedef struct **_FCIIC_MasterHandle** **FCIIC_MasterHandleType**

*Structure for handling the FCIIC Master operations. Implements : **_FCIIC_MasterHandle**.*

typedef struct **_FCIIC_SlaveHandle** **FCIIC_SlaveHandleType**

*Structure for handling the FCIIC Slave operations. Implements : **_FCIIC_SlaveHandle**.*

typedef void(* **FCIIC_MasterTransfer_CallbackType**) (struct **_FCIIC_MasterHandle** *pFciicMasterHandle,
FCIIC_StatusType Status)

Callback function type for master receive operations.

typedef void(* **FCIIC_MasterError_CallBackType**) (struct **_FCIIC_MasterHandle** *pFciicMasterHandle,
FCIIC_StatusType Status)

Callback function type for master error operations.

typedef void(* **FCIIC_SlaveTransfer_CallbackType**) (struct **_FCIIC_SlaveHandle** *pFciicSlaveHandle)

Callback function type for slave receive operations.

typedef void(* **FCIIC_SlaveError_CallBackType**) (struct **_FCIIC_SlaveHandle** *pFciicSlaveHandle)

Callback function type for slave error operations.

typedef void(* **FCIIC_RxInterrupt_CallBackType**) (struct **_FCIIC_SlaveHandle** *pFciicSlaveHandle,
FCIIC_StatusType status)

Callback function type for receive interrupt operations.

Enumerations

enum **FCIIC_InstanceType** { **FCIIC_INSTANCE_0** = 0U, **FCIIC_INSTANCE_1** = 1U }

Enumeration for selecting the FCIIC instance.

enum **FCIIC_StatusType** { **FCIIC_SUCCESS** = 0, **FCIIC_BUSY**, **FCIIC_IDLE**, **FCIIC_NAK**, **FCIIC_FIFOERROR**, **FCIIC_BITERROR**, **FCIIC_TRANSMITORRECEIVEERROR**, **FCIIC_REPEATEDSTARTERROR**, **FCIIC_ADDRESSVALID**, **FCIIC_ARBITRATIONLOST**, **FCIIC_PINLOWTIMEOUT**, **FCIIC_NOTTRANSFERINPROGRESS**, **FCIIC_DMAREQUESTFAIL**, **FCIIC_TIMEOUT**, **FCIIC_DIRECTIONERROR**, **FCIIC_STATEMACHINEERROR**, **FCIIC_OVERFLOWERROR**, **FCIIC_SLAVEDMANUMBERERROR**, **FCIIC_FAIL** }

Enumerates the possible return status codes for FCIIC (Fast Communication Interface IC) operations.

enum **FCIIC_MasterPinConfigType** { **FCIIC_OPENDRAIN** = 0x0U, **FCIIC_PUSHPULL** = 0x1U }

Specifies the pin configuration options for the FCIIC Master.

enum **FCIIC_DirectionType** { **FCIIC_WRITE** = 0U, **FCIIC_READ** = 1U, **FCIIC_DIRNOCONFIG** = 2U }

Specifies the direction of data transfer for the FCIIC Master.

enum **FCIIC_DmaStatusType** { **FCIIC_DMA_NO_CONFIG** = 0U, **FCIIC_DMA_NEXT_STAGE_IDLE** = 1U, **FCIIC_DMA_NEXT_STAGE_DATA** = 2U, **FCIIC_DMA_NEXT_STAGE_STOP** = 3U, **FCIIC_DMA_NEXT_STAGE_NOTIFICATION** = 4U }

Specifies the status of the DMA (Direct Memory Access) for FCIIC operations.

enum **FCIIC_ChannelStatusType** { **FCIIC_CH_IDLE** = 0U, **FCIIC_CH_LISTENING_READ**, **FCIIC_CH_LISTENING**, **FCIIC_CH_START_READ**, **FCIIC_CH_START_WRITE**, **FCIIC_CH_SENDING**, **FCIIC_CH_RECEIVING**, **FCIIC_CH_TRANSFER_FINISHED**, **FCIIC_CH_ERROR_PRESENT**, **FCIIC_CH_CLOSED** }

Specifies the status of the FCIIC channel.

Functions

FCIIC_StatusType **FCIIC_MasterCheckForBusyBus** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Checks if the I2C bus is busy for the master.

void **FCIIC_MasterGetDefaultConfig** (**FCIIC_MasterConfigType** *const pMasterConfig)

Initializes a default configuration for the I2C master.

void **FCIIC_MasterSetBaudRate** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint32_t sourceClock_Hz, uint32_t baudRate_Hz)

Sets the baud rate for the I2C master.

void **FCIIC_MasterInit** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, const **FCIIC_MasterConfigType** *const pMasterConfig, uint32_t sourceClock_Hz)

Initializes the I2C master with the specified configuration.

void **FCIIC_MasterDeinit** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Deinitializes the I2C master.

void **FCIIC_MasterInitHandle** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, **FCIIC_InstanceType** eInstance)

Initializes the I2C master handle.

void **FCIIC_MasterEnableInterrupt** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, **FCIIC_MasterTransfer_CallbackType** pCallback, **FCIIC_MasterError_CallbackType** pErrorCallback, void *pUserData)

Enables interrupt handling for the I2C master.

FCIIC_StatusType **FCIIC_MasterCheckAndClearError** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint32_t status)

Checks and clears errors in the I2C master.

FCIIC_StatusType **FCIIC_MasterWaitFlag** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint32_t u32Flag)

Waits for a specific flag to be set in the I2C master.

FCIIC_StatusType **FCIIC_MasterStart** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint8_t address, **FCIIC_DirectionType** dir)

Initiates an I2C start condition.

FCIIC_StatusType **FCIIC_MasterRepeatedStart** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint8_t address, **FCIIC_DirectionType** dir)

Initiates an I2C repeated start condition.

FCIIC_StatusType **FCIIC_MasterSendStartCommand** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Sends a sequence of commands in the I2C master.

FCIIC_StatusType FCIIC_MasterSend (FCIIC_MasterHandleType *const pFciicMasterHandle, uint8 *txBuff, uint16 txSize)

Sends data over the I2C bus.

FCIIC_StatusType FCIIC_MasterReceive (FCIIC_MasterHandleType *const pFciicMasterHandle)

Receives data over the I2C bus.

FCIIC_StatusType FCIIC_MasterStop (FCIIC_MasterHandleType *const pFciicMasterHandle)

Sends a stop condition over the I2C bus.

void FCIIC_MasterTransferAbort (FCIIC_MasterHandleType *const pFciicMasterHandle)

Aborts an ongoing I2C transfer.

uint8_t FCIIC_MasterSetReceiveLen (FCIIC_MasterHandleType *const pFciicMasterHandle, uint8 u8Len)

Sets the receive length for an ongoing I2C transfer.

uint8_t FCIIC_MasterTransmit (FCIIC_MasterHandleType *const pFciicMasterHandle, FCIIC_TxDataType *pTxData)

Transmits data over the I2C bus.

uint8_t FCIIC_MasterGetStatus (FCIIC_MasterHandleType *const pFciicMasterHandle, FCIIC_MasterStatusType eStatus)

Gets the status of the I2C master.

void FCIIC_MasterClrStatus (FCIIC_MasterHandleType *const pFciicMasterHandle)

Clears the status flags of the I2C master.

FCIIC_StatusType FCIIC_MasterTransferBlocking (FCIIC_MasterHandleType *const pFciicMasterHandle, FCIIC_MasterTransferType *const transfer)

Performs a blocking I2C transfer.

void FCIIC_MasterDmaPrepare (FCIIC_MasterHandleType *const pFciicMasterHandle)

Prepares the DMA channel for I2C master transfer.

void **FCIIC_MasterDmaStop** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Stops the DMA transfer for the I2C master.

void **FCIIC_MasterDmaTransmit** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Initiates a DMA transmit operation for the I2C master.

void **FCIIC_MasterDmaReceive** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Initiates a DMA receive operation for the I2C master.

void **FCIIC_MasterDmaStart** (**FCIIC_MasterHandleType** *const pFciicMasterHandle,
FCIIC_MasterTransferType *const transfer)

Starts the DMA transfer for the FCIIC master.

void **FCIIC_MasterDmaStageNotification** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Handles the notification stage after a DMA transmission.

void **FCIIC_MasterDmaStageData** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Handles the data stage of a DMA transmission.

void **FCIIC_LL_MasterDmaIRQnHandler** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

DMA interrupt handler for the FCIIC master.

FCIIC_StatusType **FCIIC_MasterDmaTransfer** (**FCIIC_MasterHandleType** *const pFciicMasterHandle,
FCIIC_MasterTransferType *const transfer)

Initiates a DMA transfer for the FCIIC master.

void **FCIIC_MasterInterruptProcessStop** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Stops the interrupt process for the FCIIC master.

void **FCIIC_MasterInterruptProcessTransmit** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Processes the transmit phase in interrupt mode for the FCIIC master.

void **FCIIC_MasterInterruptProcessReceive** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Processes the receive phase in interrupt mode for the FCIIC master.

void **FCIIC_MasterInterruptProcessStart** (FCIIC_MasterHandleType *const pFciicMasterHandle)

Processes the start phase in interrupt mode for the FCIIC master.

void **FCIIC_LL_MasterIRQnHandler** (FCIIC_MasterHandleType *const pFciicMasterHandle)

Handles the interrupt for the FCIIC master.

void **FCIIC_MasterGenerateStartCommand** (FCIIC_MasterHandleType *const pFciicMasterHandle,
FCIIC_MasterTransferType *const transfer)

Generates a start command for the FCIIC Master.

FCIIC_StatusType **FCIIC_MasterTransferNonBlocking** (FCIIC_MasterHandleType *const
pFciicMasterHandle, FCIIC_MasterTransferType *const transfer)

Initiates a non-blocking I2C Master transfer.

void **FCIIC_SlaveInit** (FCIIC_SlaveHandleType *const pFciicSlaveHandle, const FCIIC_SlaveConfigType
*pInitReg)

Initializes the I2C Slave mode.

FCIIC_StatusType **FCIIC_Slavelistening** (FCIIC_SlaveHandleType *const pFciicSlaveHandle,
FCIIC_SlaveTransferType *const transfer)

De-initializes the IIC Slave mode.

void **FCIIC_SlaveDeInit** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Deinitializes the FCIIC slave configuration.

void **FCIIC_SlaveDeinitHandle** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Deinitializes the FCIIC slave handle.

void **FCIIC_SlaveAck** (FCIIC_SlaveHandleType *const pFciicSlaveHandle, boolean bAck)

Sends an ACK/NACK signal in IIC Slave mode.

void **FCIIC_SlaveClrStatus** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Clears the I2C master status flag state.

uint32_t FCIIC_SlaveGetStatus (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Gets the slave SSR status.

**void FCIIC_SlaveEnableInterrupt (FCIIC_SlaveHandleType *const pFciicSlaveHandle,
FCIIC_SlaveTransfer_CallbackType pCallback, FCIIC_SlaveError_CallBackType pErrorCallBack, void
*pUserData)**

Enables the interrupt for the specified FCIIC Slave instance.

FCIIC_StatusType FCIIC_SlaveCheckForBusyBus (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Checks if the FCIIC bus is busy.

**FCIIC_StatusType FCIIC_SlaveCheckAndClearError (FCIIC_SlaveHandleType *const pFciicSlaveHandle,
uint32_t status)**

Checks for and clears I2C Slave errors.

void FCIIC_SlaveGetDefaultConfig (FCIIC_SlaveConfigType *const slaveConfig, boolean bUsedDMA)

Sets the default configuration for the I2C Slave.

uint8_t FCIIC_SlaveReceive (FCIIC_SlaveHandleType *const pFciicSlaveHandle, uint8_t *pData)

Receives data in IIC Slave mode.

uint32_t FCIIC_SlaveGetError (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Gets the slave error value.

void FCIIC_SlaveClrError (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Clears the slave error value.

FCIIC_DirectionType FCIIC_SlaveGetDir (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Gets the current direction of the slave.

**void FCIIC_SlaveInterruptProcessAddressMatch (FCIIC_SlaveHandleType *const pFciicSlaveHandle,
uint32_t u32IsrStatus)**

Processes the IIC Slave AM0F interrupt.

void FCIIC_SlaveInterruptProcessReceive (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Processes the IIC Slave RDF interrupt.

void FCIIC_SlaveInterruptProcessTransmit (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Processes IIC Slave TDF interrupt.

void FCIIC_SlaveInterruptProcessStop (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Processes IIC Slave SDF interrupt.

void FCIIC_LL_SlaveIRQnHandler (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

I2C Slave Interrupt Service Routine (ISR);.

FCIIC_StatusType FCIIC_SlaveTransferNonBlocking (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Starts accepting slave transfers in non-blocking mode.

void FCIIC_SlaveDmaPrepare (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Prepares DMA configuration for IIC Slave.

FCIIC_StatusType FCIIC_SlaveTransferDMA (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Initiates an IIC Slave Asynchronous DMA transfer.

void FCIIC_SlaveDMAStageNotification (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Starts DMA for IIC Slave in asynchronous mode.

void FCIIC_SlaveStartDMA (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Starts DMA for IIC Slave in asynchronous mode.

void FCIIC_LL_SlaveDmaIRQnHandler (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

DMA handler for IIC Slave.

FCIIC API Service IDs

```
#define FCIIC_MASTERCHECKFORBUSYBUS_ID 0x00
#define FCIIC_MASTERGETDEFAULTCONFIG_ID 0x01
#define FCIIC_MASTERSETBAUDRATE_ID 0x02
#define FCIIC_MASTERINIT_ID 0x03
#define FCIIC_MASTERDEINIT_ID 0x04
#define FCIIC_MASTERINITHANDLE_ID 0x05
#define FCIIC_MASTERENABLEINTERRUPT_ID 0x06
#define FCIIC_MASTERCHECKKANDCLEARERROR_ID 0x07
#define FCIIC_MASTERWAITFLAG_ID 0x08
#define FCIIC_MASTERSTART_ID 0x09
#define FCIIC_MASTERREPEATEDSTART_ID 0x0a
#define FCIIC_MASTERSENDSTARTCOMMAND_ID 0x0b
#define FCIIC_MASTERSEND_ID 0x0c
#define FCIIC_MASTERRECEIVE_ID 0x0d
#define FCIIC_MASTERSTOP_ID 0x0e
#define FCIIC_MASTERTRANSFERABORT_ID 0x0f
#define FCIIC_MASTERSETRECEIVELEN_ID 0x10
#define FCIIC_MASTERTRANSMIT_ID 0x11
#define FCIIC_MASTERGETSTATUS_ID 0x12
#define FCIIC_MASTERCLRSTATUS_ID 0x13
#define FCIIC_MASTERTRANSFERBLOCKING_ID 0x14
#define FCIIC_MASTERDMPREPREARE_ID 0x15
#define FCIIC_MASTERDMASTOP_ID 0x16
#define FCIIC_MASTERDMATRANSMIT_ID 0x17
#define FCIIC_MASTERDMARECEIVE_ID 0x18
#define FCIIC_MASTERDMASTART_ID 0x19
#define FCIIC_MASTERDMASTAGENOTIFICATION_ID 0x1a
#define FCIIC_MASTERDMASTAGEDATA_ID 0x1b
#define FCIIC_LL_MASTERDMAIRQNHANDLER_ID 0x1c
#define FCIIC_MASTERDMATRANSFER_ID 0x1d
#define FCIIC_MASTERINTERRUPTPROCESSSTOP_ID 0x1e
#define FCIIC_MASTERINTERRUPTPROCESSTRANSMIT_ID 0x1f
#define FCIIC_MASTERINTERRUPTPROCESSRECEIVE_ID 0x20
#define FCIIC_MASTERINTERRUPTPROCESSSTART_ID 0x21
#define FCIIC_LL_MASTERIRQNHANDLER_ID 0x22
#define FCIIC_MASTERGENERATESTARTCOMMAND_ID 0x23
#define FCIIC_MASTERTRANSFERNONBLOCKING_ID 0x24
```

```

#define FCIIC_SLAVEINIT_ID 0x25
#define FCIIC_SLAVELISTENING_ID 0x26
#define FCIIC_SLAVEDEINIT_ID 0x27
#define FCIIC_SLAVEDEINITHANDLE_ID 0x28
#define FCIIC_SLAVEACK_ID 0x29
#define FCIIC_SLAVECLRSTATUS_ID 0x2a
#define FCIIC_SLAVEGETSTATUS_ID 0x2b
#define FCIIC_SLAVEENABLEINTERRUPT_ID 0x2c
#define FCIIC_SLAVECHECKFORBUSYBUS_ID 0x2d
#define FCIIC_SLAVECHECKANDCLEARERROR_ID 0x2e
#define FCIIC_SLAVEGETDEFAULTCONFIG_ID 0x2f
#define FCIIC_SLAVERECEIVE_ID 0x30
#define FCIIC_SLAVEGETERROR_ID 0x31
#define FCIIC_SLAVECLRERROR_ID 0x32
#define FCIIC_SLAVEGETDIR_ID 0x33
#define FCIIC_SLAVEINTERRUPTPROCESSADDRESSMATCH_ID 0x34
#define FCIIC_SLAVEINTERRUPTPROCESSRECEIVE_ID 0x35
#define FCIIC_SLAVEINTERRUPTPROCESSTRANSMIT_ID 0x36
#define FCIIC_SLAVEINTERRUPTPROCESSTOP_ID 0x37
#define FCIIC_LL_SLAVEIRQNHANDLER_ID 0x38
#define FCIIC_SLAVETRANSFERNONBLOCKING_ID 0x39
#define FCIIC_SLAVEDMAPPREPARE_ID 0x3a
#define FCIIC_SLAVETRANSFERDMA_ID 0x3b
#define FCIIC_SLAVEDMASTAGENOTIFICATION_ID 0x3c
#define FCIIC_SLAVESTARTDMA_ID 0x3d
#define FCIIC_LL_SLAVEDMAIRQNHANDLER_ID 0x3e

```

Detailed Description

Macro Definition Documentation

#define FCIIC_DIR_MASK (0x1U)

Mask to extract the direction bit from the status register.

#define FCIIC_E_PARAM_CHANNEL 0x03U

Error code indicating an invalid channel parameter.

#define FCIIC_E_PARAM_INSTANCE 0x02U

Error code indicating an invalid instance parameter.

#define FCIIC_E_PARAM_NULLPTR 0x01U

Error code indicating a null pointer parameter.

#define FCIIC_LL_MASTERDMAIRQHANDLER_ID 0x1c

ID for the low-level master DMA IRQ handler.

#define FCIIC_LL_MASTERIRQHANDLER_ID 0x22

ID for the low-level master IRQ handler.

#define FCIIC_LL_SLAVEDMAIRQHANDLER_ID 0x3e

ID for the low-level slave DMA IRQ handler.

#define FCIIC_LL_SLAVEIRQHANDLER_ID 0x38

ID for the low-level slave IRQ handler.

#define FCIIC_MASTER_CMD_SIZE (0x8U)

Defines the size of the master command in bytes.

#define FCIIC_MASTERCHECKANDCLEARERROR_ID 0x07

ID for checking and clearing errors.

#define FCIIC_MASTERCHECKFORBUSYBUS_ID 0x00

ID for checking the bus busy status.

#define FCIIC_MASTERCLRSTATUS_ID 0x13

ID for clearing the status.

#define FCIIC_MASTERDEINIT_ID 0x04

ID for deinitializing the master.

#define FCIIC_MASTERDMAPREPARE_ID 0x15

ID for preparing DMA.

#define FCIIC_MASTERDMARECEIVE_ID 0x18

ID for DMA reception.

#define FCIIC_MASTERDMASTAGEDATA_ID 0x1b

ID for DMA stage data.

#define FCIIC_MASTERDMASTAGENOTIFICATION_ID 0x1a

ID for DMA stage notification.

#define FCIIC_MASTERDMASTART_ID 0x19

ID for starting DMA.

#define FCIIC_MASTERDMASTOP_ID 0x16

ID for stopping DMA.

#define FCIIC_MASTERDMATRANSFER_ID 0x1d

ID for DMA transfer.

#define FCIIC_MASTERDMATRANSMIT_ID 0x17

ID for DMA transmission.

#define FCIIC_MASTERENABLEINTERRUPT_ID 0x06

ID for enabling interrupts.

#define FCIIC_MASTERGENERATESTARTCOMMAND_ID 0x23

ID for generating a start command.

#define FCIIC_MASTERGETDEFAULTCONFIG_ID 0x01

ID for getting the default configuration.

#define FCIIC_MASTERGETSTATUS_ID 0x12

ID for getting the status.

#define FCIIC_MASTERINIT_ID 0x03

ID for initializing the master.

#define FCIIC_MASTERINITHANDLE_ID 0x05

ID for initializing the master handle.

#define FCIIC_MASTERINTERRUPTPROCESSRECEIVE_ID 0x20

ID for processing the receive interrupt.

#define FCIIC_MASTERINTERRUPTPROCESSSTART_ID 0x21

ID for processing the start interrupt.

#define FCIIC_MASTERINTERRUPTPROCESSSTOP_ID 0x1e

ID for processing the stop interrupt.

#define FCIIC_MASTERINTERRUPTPROCESSTRANSMIT_ID 0x1f

ID for processing the transmit interrupt.

#define FCIIC_MASTERRECEIVE_ID 0x0d

ID for receiving data.

#define FCIIC_MASTERREPEATEDSTART_ID 0x0a

ID for initiating a repeated start.

#define FCIIC_MASTERSEND_ID 0x0c

ID for sending data.

#define FCIIC_MASTERSENDSTARTCOMMAND_ID 0x0b

ID for sending a start command.

#define FCIIC_MASTERSETBAUDRATE_ID 0x02

ID for setting the baud rate.

#define FCIIC_MASTERSETRECEIVELEN_ID 0x10

ID for setting the receive length.

#define FCIIC_MASTERSTART_ID 0x09

ID for starting the transfer.

#define FCIIC_MASTERSTOP_ID 0x0e

ID for stopping the transfer.

#define FCIIC_MASTERTRANSFERABORT_ID 0x0f

ID for aborting the transfer.

#define FCIIC_MASTERTRANSFERBLOCKING_ID 0x14

ID for performing a blocking transfer.

#define FCIIC_MASTERTRANSFERNONBLOCKING_ID 0x24

ID for non-blocking transfer.

#define FCIIC_MASTERTRANSMIT_ID 0x11

ID for transmitting data.

#define FCIIC_MASTERWAITFLAG_ID 0x08

ID for waiting on a flag.

#define FCIIC_SLAVEACK_ID 0x29

ID for acknowledging.

#define FCIIC_SLAVECHECKANDCLEARERROR_ID 0x2e

ID for checking and clearing errors.

#define FCIIC_SLAVECHECKFORBUSYBUS_ID 0x2d

ID for checking the bus busy status.

#define FCIIC_SLAVECLRERROR_ID 0x32

ID for clearing the error.

#define FCIIC_SLAVECLRSTATUS_ID 0x2a

ID for clearing the status.

#define FCIIC_SLAVEDEINIT_ID 0x27

ID for deinitializing the slave.

#define FCIIC_SLAVEDEINITHANDLE_ID 0x28

ID for deinitializing the slave handle.

#define FCIIC_SLAVEDMAPREPREARE_ID 0x3a

ID for preparing DMA.

#define FCIIC_SLAVEDMASTAGENOTIFICATION_ID 0x3c

ID for DMA stage notification.

#define FCIIC_SLAVEENABLEINTERRUPT_ID 0x2c

ID for enabling interrupts.

#define FCIIC_SLAVEGETDEFAULTCONFIG_ID 0x2f

ID for getting the default configuration.

#define FCIIC_SLAVEGETDIR_ID 0x33

ID for getting the direction.

#define FCIIC_SLAVEGETERROR_ID 0x31

ID for getting the error.

#define FCIIC_SLAVEGETSTATUS_ID 0x2b

ID for getting the status.

#define FCIIC_SLAVEINIT_ID 0x25

ID for initializing the slave.

#define FCIIC_SLAVEINTERRUPTPROCESSADDRESSMATCH_ID 0x34

ID for processing address match interrupt.

#define FCIIC_SLAVEINTERRUPTPROCESSRECEIVE_ID 0x35

ID for processing the receive interrupt.

#define FCIIC_SLAVEINTERRUPTPROCESSSTOP_ID 0x37

ID for processing the stop interrupt.

#define FCIIC_SLAVEINTERRUPTPROCESSTRANSMIT_ID 0x36

ID for processing the transmit interrupt.

#define FCIIC_SLAVELISTENING_ID 0x26

ID for listening mode.

#define FCIIC_SLAVERECEIVE_ID 0x30

ID for receiving data.

#define FCIIC_SLAVESTARTDMA_ID 0x3d

ID for starting DMA.

#define FCIIC_SLAVETRANSFERDMA_ID 0x3b

ID for DMA transfer.

#define FCIIC_SLAVETRANSFERNONBLOCKING_ID 0x39

ID for non-blocking transfer.

Typedef Documentation

typedef struct _FCIIC_DmaConfig FCIIC_DmaConfigType

Structure for configuring DMA for I2C Master and Slave operations.

This structure holds the configuration for DMA operations in both I2C Master and Slave modes.

FCIIC_MasterError_CallBackType

Callback function type for master error operations.

This callback function is called when an error occurs during a master operation.

Parameters

<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
<i>Status</i>	The status of the error.

typedef struct _FCIIC_MasterHandle FCIIC_MasterHandleType

Structure for handling the FCIIC Master operations. Implements : **_FCIIC_MasterHandle**.

This structure represents the handle for managing FCIIC Master operations, including the current state and settings.

FCIIC_MasterTransfer_CallbackType

Callback function type for master receive operations.

This callback function is called when a master receive operation completes.

Parameters

<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
<i>Status</i>	The status of the operation.

FCIIC_RxInterrupt_CallBackType

Callback function type for receive interrupt operations.

This callback function is called when a receive interrupt occurs.

Parameters

<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle.
<i>status</i>	The status of the interrupt.

FCIIC_SlaveError_CallbackType

Callback function type for slave error operations.

This callback function is called when an error occurs during a slave operation.

Parameters

<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle.
--------------------------	------------------------------------

typedef struct _FCIIC_SlaveHandle FCIIC_SlaveHandleType

Structure for handling the FCIIC Slave operations. Implements : **_FCIIC_SlaveHandle**.

This structure represents the handle for managing FCIIC Slave operations, including the current state and settings.

FCIIC_SlaveTransfer_CallbackType

Callback function type for slave receive operations.

This callback function is called when a slave receive operation completes.

Parameters

<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle.
--------------------------	------------------------------------

Enumeration Type Documentation

enum FCIIC_ChannelStatusType

Specifies the status of the FCIIC channel.

This enumeration defines the different states of the FCIIC channel, indicating the current state of the channel during communication.

Enumerator:

FCIIC_CH_IDLE	I2C channel is idle.
FCIIC_CH_LISTENING_READ	I2C channel is listening for slave reading.
FCIIC_CH_LISTENING_WRITE	I2C channel is listening for slave writing.

FCIIC_CH_START_READ	I2C channel will be reading for master.
FCIIC_CH_START_WRITE	I2C channel will be writing for master.
FCIIC_CH_SENDING	I2C channel is sending.
FCIIC_CH_RECEIVING	I2C channel is receiving.
FCIIC_CH_TRANSFER_FINISHED	I2C channel has finished transferring data.
FCIIC_CH_ERROR_PRESENT	I2C channel has an error present.
FCIIC_CH_CLOSED	I2C channel is closed.

enum FCIIC_DirectionType

Specifies the direction of data transfer for the FCIIC Master.

This enumeration defines the direction of data transfer for the FCIIC Master, indicating whether the operation is a write (transmit) or read (receive).

Enumerator:

FCIIC_WRITE	Indicates a Master transmit operation, writing data to a slave.
FCIIC_READ	Indicates a Master receive operation, reading data from a slave.
FCIIC_DIRNOCONFIG	Indicates no configuration for direction.

enum FCIIC_DmaStatusType

Specifies the status of the DMA (Direct Memory Access) for FCIIC operations.

This enumeration defines the different states of the DMA for FCIIC, indicating the current or next stage of the DMA process.

Enumerator:

FCIIC_DMA_NO_CONFIG	DMA mode is not configured.
FCIIC_DMA_NEXT_STAGE_IDLE	DMA next stage is idle.
FCIIC_DMA_NEXT_STAGE_DATA	DMA next stage is transmit/receive data.
FCIIC_DMA_NEXT_STAGE_STOP	DMA next stage is transmit stop.
FCIIC_DMA_NEXT_STAGE_NOTIFICATION	DMA next stage is notification to the user.

enum FCIIC_InstanceType

Enumeration for selecting the FCIIC instance.

Enumerator:

FCIIC_INSTANCE_0	FCIIC instance 0 is selected. Corresponds to the first available FCIIC module.
FCIIC_INSTANCE_1	FCIIC instance 1 is selected. Corresponds to the second available FCIIC module.

enum FCIIC_MasterPinConfigType

Specifies the pin configuration options for the FCIIC Master.

This enumeration defines the pin configurations for the FCIIC Master interface, which can be set to either open-drain mode or push-pull mode.

Enumerator:

FCIIC_OPENDRAIN	FCIIC configured for 2-pin open-drain mode, suitable for standard and fast modes.
FCIIC_PUSH_PULL	FCIIC configured for 2-pin output-only (push-pull) mode, typically used for ultra-fast mode.

enum FCIIC_StatusType

Enumerates the possible return status codes for FCIIC (Fast Communication Interface IC) operations.

This enumeration defines the status values returned by FCIIC functions, indicating success or various errors that can occur during communication. These codes provide information about the outcome of a particular operation.

Note

Check the specific function documentation for the meaning of each status code in the context of the operation performed.

Enumerator:

FCIIC_SUCCESS	Generic status for successful completion of an operation.
FCIIC_BUSY	The FCIIC Master is already performing a transfer.
FCIIC_IDLE	The FCIIC Slave driver is currently idle, no transfer is in progress.
FCIIC_NAK	The slave device sent a Negative Acknowledge (NAK) in response to a byte.
FCIIC_FIFOERROR	FIFO underflow or overflow error occurred.

FCIIC_BITERR R	A transferred bit was not recognized on the bus.
FCIIC_TRANSMI TORRECEIVEER ROR	The slave device transmit or receive Error.
FCIIC_REPEATE DSTARTERROR	The FCIIC slave not detects a repeated START condition and if the FCIIC slave matched the last address byte, but should detect.
FCIIC_ADDRESS VALID	Indicates whether the Address Status Register is valid.
FCIIC_ARBITRAT IONLOST	Lost arbitration during a transfer.
FCIIC_PINLOWTI MEOUT	SCL or SDA line was held low longer than the timeout period.
FCIIC_NOTRANS FERINPROGRES S	Attempted to abort a transfer when none was in progress.
FCIIC_DMAREQ UESTFAIL	DMA request failed to initiate or complete.
FCIIC_TIMEOUT	Timeout while polling for a status flag.
FCIIC_DIRECTIO NERROR	Direction does not match the desired direction Error.
FCIIC_STATEMA CHINEERROR	State machine error due to unknown error.
FCIIC_OVERFLOW WERROR	The slave device transmit or receive byte number Error, Exceeding the maximum limit length.
FCIIC_SLAVEDM ANUMBERERRO R	The slave device DMA transmit or receive byte number Error, The number of bytes transferred is less or more than the number required to trigger a DMA completion interrupt.

FCIIC_FAIL	parameter error or others.
------------	----------------------------

Function Documentation

void FCIIC_LL_MasterDmaIRQnHandler (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

DMA interrupt handler for the FCIIC master.

This function handles DMA interrupts by determining the next stage of the DMA operation, updating the channel status, and managing error conditions.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

void FCIIC_LL_MasterIRQnHandler (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Handles the interrupt for the FCIIC master.

This function processes various interrupt sources and updates the state of the transfer based on the current channel status. It also handles errors and calls appropriate callbacks.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

void FCIIC_LL_SlaveDmaIRQnHandler (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

DMA handler for IIC Slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

void FCIIC_LL_SlaveIRQnHandler (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

I2C Slave Interrupt Service Routine (ISR);

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

FCIIC_StatusType FCIIC_MasterCheckAndClearError (FCIIC_MasterHandleType *const *pFciicMasterHandle*, uint32_t *status*)

Checks and clears errors in the I2C master.

This function checks for specific error conditions in the I2C master and clears them if present.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>status</i>	The status flags to check and clear.

Returns

FCIIC_StatusType The error status or FCIIC_SUCCESS if no errors were detected.

The function performs the following steps:

- Checks the status flags against known error conditions.

- Selects the appropriate error code based on the severity and type of error.

- Clears the error flags in the I2C hardware.

- Resets the FIFOs if necessary.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

FCIIC_StatusType FCIIC_MasterCheckForBusyBus (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Checks if the I2C bus is busy for the master.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC Master handle.
----	---------------------------	-------------------------------------

Returns

FCIIC_StatusType Indicates whether the bus is busy or not.

Return values

<i>FCIIC_SUCCESS</i>	Bus is not busy.
<i>FCIIC_BUSY</i>	Bus is busy.

void FCIIC_MasterClrStatus (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Clears the status flags of the I2C master.

This function clears specific status flags of the I2C master.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

The function performs the following steps:

Clears the data match flag, protocol error flag, FIFO error flag, address lost flag, not data flag, start detection flag, and end packet flag.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_MasterDeinit (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Deinitializes the I2C master.

This function deinitializes the I2C master by resetting it and clearing its configuration.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

The function performs the following steps:

Resets the master control register (MCR); to disable the master, reset, and clear DMA and buffer flags.

Disables DMA operations by setting the master DMA enable register (MDER);.

Clears the pin configuration and glitch filter settings in the master configuration registers (MCFGR1, MCFGR2, MCFGR3);.

Resets the I2C hardware.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly.

void FCIIC_MasterDmaPrepare (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Prepares the DMA channel for I2C master transfer.

This function prepares the DMA channel for data transfer by configuring the DMA attributes and starting the DMA transfer.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

The function performs the following steps:

Configures the DMA channel attributes based on the transfer direction (read/write);.

Sets up the DMA source and destination buffers, block size, and other settings.

Starts the DMA transfer.

Updates the channel status based on the transfer direction.

Calls the error callback function if an error occurs during preparation.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_MasterDmaReceive (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Initiates a DMA receive operation for the I2C master.

This function initiates the DMA receive operation by preparing the DMA for reception and setting the next DMA stage to stop.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

The function performs the following steps:

Prepares the DMA for the receive operation.

Sets the next DMA stage to stop after the reception.

Calls the error callback function if an error occurs during preparation.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_MasterDmaStageData (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Handles the data stage of a DMA transmission.

This function updates the channel status and triggers the data transfer phase based on the current state and direction of the transfer.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

void FCIIC_MasterDmaStageNotification (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Handles the notification stage after a DMA transmission.

This function updates the channel status and triggers a callback if one is registered. It also manages the next stage of DMA operations based on the direction of the transfer.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

**void FCIIC_MasterDmaStart (FCIIC_MasterHandleType *const *pFciicMasterHandle*,
FCIIC_MasterTransferType *const *transfer*)**

Starts the DMA transfer for the FCIIC master.

This function initializes the DMA command sequence based on the given transfer parameters. It checks for valid input parameters and then prepares the DMA commands to be sent.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle structure.
in	<i>transfer</i>	Pointer to the FCIIC master transfer structure containing transfer details.

void FCIIC_MasterDmaStop (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Stops the DMA transfer for the I2C master.

This function stops the DMA transfer by preparing a stop command and updating the DMA configuration.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

The function performs the following steps:

- Sets up the stop command in the DMA command buffer.
- Prepares the DMA for the stop command.
- Updates the channel status to indicate the transfer has been stopped.
- Adjusts the next DMA stage based on the transfer direction.
- Calls the error callback function if an error occurs during the preparation.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

FCIIC_StatusType FCIIC_MasterDmaTransfer (FCIIC_MasterHandleType *const *pFciicMasterHandle*, FCIIC_MasterTransferType *const *transfer*)

Initiates a DMA transfer for the FCIIC master.

This function starts a DMA transfer by configuring the DMA settings and initiating the transfer process. It checks for valid parameters, ensures the bus is not busy, and configures the DMA accordingly.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>transfer</i>	Pointer to the FCIIC master transfer structure containing transfer details.

Returns

FCIIC_StatusType The status of the DMA transfer initiation.

void FCIIC_MasterDmaTransmit (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Initiates a DMA transmit operation for the I2C master.

This function initiates the DMA transmit operation by preparing the DMA for transmission and setting the next DMA stage to stop.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

The function performs the following steps:

- Prepares the DMA for the transmit operation.

- Sets the next DMA stage to stop after the transmission.

- Calls the error callback function if an error occurs during preparation.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

```
void FCIIC_MasterEnableInterrupt (FCIIC_MasterHandleType *const pFciicMasterHandle,  
FCIIC_MasterTransfer_CallbackType pCallback, FCIIC_MasterError_CallbackType pErrorCallback, void  
* pUserData)
```

Enables interrupt handling for the I2C master.

This function sets the callback functions and user data for interrupt handling in the I2C master.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>pCallback</i>	Pointer to the transfer callback function.
in	<i>pErrorCallback</i>	Pointer to the error callback function.
in	<i>pUserData</i>	User-defined data to be passed to the callback functions.

The function performs the following steps:

- Sets the transfer callback, error callback, and user data pointers in the handle's settings.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly.

```
void FCIIC_MasterGenerateStartCommand (FCIIC_MasterHandleType *const pFciicMasterHandle,  
FCIIC_MasterTransferType *const transfer)
```

Generates a start command for the FCIIC Master.

This function initializes the master for transmission by generating the necessary commands for the start of a transfer.

Parameters

in	<i>pFciicMasterHandle</i>	The FCIIC Master Handle.
in	<i>transfer</i>	Pointer to the FCIIC_MasterTransferType structure containing the transfer details.

Returns

void

void FCIIC_MasterGetDefaultConfig (FCIIC_MasterConfigType *const pMasterConfig)

Initializes a default configuration for the I2C master.

This function sets up the default configuration parameters for the I2C master.

Parameters

out	<i>pMasterConfig</i>	Pointer to the structure where the default configuration will be stored.
-----	----------------------	--

The following configurations are set:

bEnableMaster: Enables the master functionality.

bDebugEnabled: Disables debug mode by default.

ePinConfig: Sets the pin configuration to open-drain.

u32BaudRate: Sets the baud rate to 100 kHz.

u32BusIdleTimeout: Disables the bus idle timeout.

u32PinLowTimeout: Disables the pin low timeout.

u8SdaGlitchFilterWidth: Disables the SDA glitch filter.

u8SclGlitchFilterWidth: Disables the SCL glitch filter.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly.

uint8_t FCIIC_MasterGetStatus (FCIIC_MasterHandleType *const pFciicMasterHandle, FCIIC_MasterStatusType eStatus)

Gets the status of the I2C master.

This function retrieves the status of the I2C master based on the specified status type.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>eStatus</i>	The status type to retrieve.

Returns

uint8_t Returns the status value as a bit mask.

The function performs the following steps:

Retrieves the status based on the specified status type.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_MasterInit (FCIIC_MasterHandleType *const *pFciicMasterHandle*, const FCIIC_MasterConfigType *const *pMasterConfig*, uint32_t *sourceClock_Hz*)

Initializes the I2C master with the specified configuration.

This function initializes the I2C master with the provided configuration and sets up the baud rate.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>pMasterConfig</i>	Pointer to the master configuration structure.
in	<i>sourceClock_Hz</i>	The source clock frequency in Hz.

The function performs the following steps:

Resets the I2C hardware.

Sets the baud rate using the FCIIC_MasterSetBaudRate function.

Configures the master control register (MCR); for debug enablement.

Configures the master configuration registers (MCFGR1, MCFGR2, MCFGR3); for pin configuration, glitch filters, and timeouts.

Enables the master operation.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointers are null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_MasterInitHandle (FCIIC_MasterHandleType *const *pFciicMasterHandle*, FCIIC_InstanceType *eInstance*)

Initializes the I2C master handle.

This function initializes the I2C master handle with the specified instance and clears its status and settings.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>eInstance</i>	The instance of the I2C master to initialize.

The function performs the following steps:

- Sets the instance of the handle.

- Clears the status fields including command data, direction, channel status, and offsets.

- Sets callback functions and user data pointers to null.

- Initializes DMA-related settings to null or zero.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_MasterInterruptProcessReceive (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Processes the receive phase in interrupt mode for the FCIIC master.

This function receives data and updates the data offset. It stops the transfer and updates the channel status when the data reception is complete.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

void FCIIC_MasterInterruptProcessStart (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Processes the start phase in interrupt mode for the FCIIC master.

This function manages the command transmission and updates the command offset. It transitions the channel status based on the current phase of the transfer.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

void FCIIC_MasterInterruptProcessStop (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Stops the interrupt process for the FCIIC master.

This function disables all interrupts, updates the channel status, and triggers a callback if the data transfer is complete. It also handles error conditions if the data offset does not match the expected size.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

void FCIIC_MasterInterruptProcessTransmit (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Processes the transmit phase in interrupt mode for the FCIIC master.

This function transmits data and updates the data offset. It stops the transfer and updates the channel status when the data transfer is complete.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

FCIIC_StatusType FCIIC_MasterReceive (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Receives data over the I2C bus.

This function receives a block of data over the I2C bus.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

Returns

FCIIC_StatusType The status indicating success or failure.

The function performs the following steps:

Iterates through each byte of the receive buffer.

Waits for the receive data flag to be set.
 Reads the received byte from the I2C bus.
 Stores the received byte in the receive buffer.
 Increments the data offset after each successful reception.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

**FCIIC_StatusType FCIIC_MasterRepeatedStart (FCIIC_MasterHandleType *const *pFciicMasterHandle*,
 uint8_t *address*, FCIIC_DirectionType *dir*)**

Initiates an I2C repeated start condition.
 This function initiates an I2C repeated start condition and sends the device address along with the read/write direction.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>address</i>	The 7-bit I2C address of the target device.
in	<i>dir</i>	The direction of the communication (read or write);.

Returns

FCIIC_StatusType The status indicating success or failure.

The function performs the following steps:

Sends a repeated start condition followed by the address and direction.
 Waits for the transmission done flag to be set.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

**FCIIC_StatusType FCIIC_MasterSend (FCIIC_MasterHandleType *const *pFciicMasterHandle*, uint8 *
txBuff, uint16 *txSize*)**

Sends data over the I2C bus.
 This function sends a block of data over the I2C bus.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>txBuff</i>	Pointer to the transmit buffer containing the data to send.
in	<i>txSize</i>	The size of the data to send in bytes.

Returns

FCIIC_StatusType The status indicating success or failure.

The function performs the following steps:

- Iterates through each byte in the transmit buffer.
- Sends the byte over the I2C bus.
- Waits for the transmission done flag to be set for each byte.
- Increments the data offset after each successful transmission.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

FCIIC_StatusType FCIIC_MasterSendStartCommand (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Sends a sequence of commands in the I2C master.

This function sends a sequence of commands and data to the I2C bus.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

Returns

FCIIC_StatusType The status indicating success or failure.

The function performs the following steps:

- Iterates through each command in the command buffer.
- Sends the command and corresponding data.
- Waits for the transmission done flag to be set for each command.
- Increments the command offset after each successful transmission.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.


```
void FCIIC_MasterSetBaudRate (FCIIC_MasterHandleType *const pFciicMasterHandle, uint32_t sourceClock_Hz, uint32_t baudRate_Hz)
```

Sets the baud rate for the I2C master.

This function calculates and sets the baud rate for the I2C master based on the given source clock frequency and desired baud rate.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>sourceClock_Hz</i>	The source clock frequency in Hz.
in	<i>baudRate_Hz</i>	The desired baud rate in Hz.

The function performs the following steps:

- Calculates the optimal values for CLKHI, CLKLO, and PRESCALE to achieve the closest possible baud rate to the desired one.

- Configures the MCCR and MCFGR1 registers with the calculated values.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

```
uint8_t FCIIC_MasterSetReceiveLen (FCIIC_MasterHandleType *const pFciicMasterHandle, uint8_t u8Len)
```

Sets the receive length for an ongoing I2C transfer.

This function sets the number of bytes to receive during an ongoing I2C transfer.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>u8Len</i>	The number of bytes to receive.

Returns

uint8_t Returns 1 if the receive length was successfully set, otherwise 0.

The function performs the following steps:

- Checks if the transmit count is less than 8.

- If the condition is met, sends a receive command with the specified length minus one.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly.

FCIIC_StatusType FCIIC_MasterStart (FCIIC_MasterHandleType *const *pFciicMasterHandle*, uint8_t *address*, FCIIC_DirectionType *dir*)

Initiates an I2C start condition.

This function initiates an I2C start condition and sends the device address along with the read/write direction.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
in	<i>address</i>	The 7-bit I2C address of the target device.
in	<i>dir</i>	The direction of the communication (read or write);.

Returns

FCIIC_StatusType The status indicating success or failure.

The function performs the following steps:

Checks if the bus is busy before starting.

Clears all relevant flags in the I2C hardware.

Sends a start condition followed by the address and direction.

Waits for the transmission done flag to be set.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

FCIIC_StatusType FCIIC_MasterStop (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Sends a stop condition over the I2C bus.

This function sends a stop condition over the I2C bus.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

Returns

FCIIC_StatusType The status indicating success or failure.

The function performs the following steps:

Sends a stop condition over the I2C bus.

Waits for the transmission done flag to be set.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_MasterTransferAbort (FCIIC_MasterHandleType *const *pFciicMasterHandle*)

Aborts an ongoing I2C transfer.

This function aborts an ongoing I2C transfer by disabling interrupts, resetting the FIFO, and marking the handle as unused.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

The function performs the following steps:

Disables interrupts for the I2C master.

Resets the FIFO.

Marks the handle as unused.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

FCIIC_StatusType FCIIC_MasterTransferBlocking (FCIIC_MasterHandleType *const *pFciicMasterHandle*, FCIIC_MasterTransferType *const *transfer*)

Performs a blocking I2C transfer.

This function performs a blocking I2C transfer, which includes sending commands and data, and handling the transfer completion or errors.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC master handle.
----	---------------------------	-------------------------------------

in	<i>transfer</i>	Pointer to the I2C transfer parameters.
----	-----------------	---

Returns

FCIIC_StatusType Returns the status of the transfer operation.

The function performs the following steps:

- Checks if the bus is busy or the handle is already in use.
- Clears the status flags.
- Initializes the handle's status fields.
- Generates and sends the start command.
- Sends or receives data based on the direction.
- Sends a stop signal after the data transfer.
- Calls the callback function upon successful transfer completion.
- Calls the error callback function if an error occurs during the transfer.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointers are null and report an error accordingly.

FCIIC_StatusType FCIIC_MasterTransferNonBlocking (FCIIC_MasterHandleType *const *pFciicMasterHandle*, FCIIC_MasterTransferType *const *transfer*)

Initiates a non-blocking I2C Master transfer.

Sets up and initiates a non-blocking I2C Master transfer, including handling of START, addressing, subaddress (if provided);, data transfer, and STOP conditions. Returns immediately after starting the transfer, and the transfer is managed asynchronously using interrupts.

Parameters

in	<i>pFciicMasterHandle</i>	Pointer to the FCIIC Master Handle.
in	<i>transfer</i>	Pointer to the FCIIC_MasterTransferType structure containing the transfer details.

Returns

A status code indicating the outcome of the initiation process:

- FCIIC_SUCCESS: The transfer was initiated successfully.
- FCIIC_BSY: Another non-blocking transfer is already in progress.
- Other error codes: Errors encountered during the transfer setup, such as a busy bus.

```
uint8_t FCIIC_MasterTransmit (FCIIC_MasterHandleType *const  pFciicMasterHandle, FCIIC_TxDataType
*  pTxData)
```

Transmits data over the I2C bus.

This function transmits data over the I2C bus if the transmission data flag is set.

Parameters

in	<i>pFciicMasterHan dle</i>	Pointer to the FCIIC master handle.
in	<i>pTxData</i>	Pointer to the transmit data structure containing the command and data to send.

Returns

uint8_t Returns 1 if the transmission was initiated, otherwise 0.

The function performs the following steps:

- Clears any error flags.
- Checks the transmission data flag.
- If the flag is set, sends the command and data.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointers are null and report an error accordingly. It also checks if the instance is valid.

```
FCIIC_StatusType FCIIC_MasterWaitFlag (FCIIC_MasterHandleType *const  pFciicMasterHandle, uint32_t
u32Flag)
```

Waits for a specific flag to be set in the I2C master.

This function waits for a given flag to be set in the I2C master status and returns the result.

Parameters

in	<i>pFciicMasterHan dle</i>	Pointer to the FCIIC master handle.
in	<i>u32Flag</i>	The flag to wait for.

Returns

FCIIC_StatusType The status indicating success or timeout.

The function performs the following steps:

- Waits for the specified flag to be set within a certain number of attempts.

Returns FCIIC_SUCCESS if the flag is set, or FCIIC_TIMEOUT if the flag is not set within the timeout period.

Checks and clears any errors that may have occurred during the wait.

Note

If FCIIC_DEV_ERROR_REPORT is defined as STD_ON, this function will check if the provided pointer is null and report an error accordingly. It also checks if the instance is valid.

void FCIIC_SlaveAck (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*, boolean *bAck*)

Sends an ACK/NACK signal in IIC Slave mode.

This function sends an ACK or NACK signal based on the input parameter.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
in	<i>bAck</i>	Boolean flag to indicate whether to send ACK (true); or NACK (false);.

FCIIC_StatusType FCIIC_SlaveCheckAndClearError (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*, uint32_t *status*)

Checks for and clears I2C Slave errors.

This function checks the provided status register for any I2C Slave errors and clears them.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
in	<i>status</i>	The current status register value.

Returns

FCIIC_StatusType indicating the encountered error or FCIIC_SUCCESS if no error is found.

FCIIC_StatusType FCIIC_SlaveCheckForBusyBus (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Checks if the FCIIC bus is busy.

This function determines if the FCIIC bus is currently busy.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

Returns

FCIIC_StatusType indicating the bus status.

void FCIIC_SlaveClrError (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Clears the slave error value.

This function clears the error value of the slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

void FCIIC_SlaveClrStatus (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Clears the I2C master status flag state.

This function clears the status flags of the I2C master.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

void FCIIC_SlaveDeInit (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Deinitializes the FCIIC slave configuration.

This function resets the FCIIC slave peripheral to its default state. It disables the FCIIC slave, clears all registers, and disables interrupts.

Parameters

<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
--------------------------	--

void FCIIC_SlaveDeinitHandle (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Deinitializes the FCIIC slave handle.

This function resets the FCIIC slave handle to its default state.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

void FCIIC_SlaveDmaPrepare (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Prepares DMA configuration for IIC Slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

void FCIIC_SlaveDMAStageNotification (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Starts DMA for IIC Slave in asynchronous mode.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

void FCIIC_SlaveEnableInterrupt (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*, FCIIC_SlaveTransfer_CallbackType *pCallback*, FCIIC_SlaveError_CallBackType *pErrorCallBack*, void **pUserData*)

Enables the interrupt for the specified FCIIC Slave instance.

This function configures the interrupt system to handle non-blocking transfers for the given FCIIC Slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
in	<i>pCallback</i>	Pointer to a callback function that will be invoked when a non-blocking transfer completes.
in	<i>pErrorCallBack</i>	Pointer to a callback function that will be invoked when an error occurs.

in	<i>pUserData</i>	Pointer to user-defined data that will be passed back to the callback function when it is called.
----	------------------	---

void FCIIC_SlaveGetDefaultConfig (FCIIC_SlaveConfigType *const *slaveConfig*, boolean *bUsedDMA*)

Sets the default configuration for the I2C Slave.

This function sets the default configuration parameters for the I2C Slave.

Parameters

in	<i>slaveConfig</i>	Pointer to the configuration structure.
in	<i>bUsedDMA</i>	Boolean flag indicating whether DMA is used.

FCIIC_DirectionType FCIIC_SlaveGetDir (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Gets the current direction of the slave.

This function retrieves the current direction (read/write); of the slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

Returns

The current direction as an FCIIC_DirectionType enum value.

uint32_t FCIIC_SlaveGetError (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Gets the slave error value.

This function retrieves the current error value of the slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

Returns

The current error value as a 32-bit unsigned integer.

uint32_t FCIIC_SlaveGetStatus (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Gets the slave SSR status.

This function retrieves the current status of the slave SSR register.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

Returns

The current SSR status as a 32-bit unsigned integer.

void FCIIC_SlaveInit (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*, const FCIIC_SlaveConfigType **pInitReg*)

Initializes the I2C Slave mode.

Configures the I2C interface for slave mode operation according to the parameters provided.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave Handle.
in	<i>pInitReg</i>	Pointer to the FCIIC_SlaveConfigType structure containing the initialization settings.

Returns

void

void FCIIC_SlaveInterruptProcessAddressMatch (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*, uint32 *u32IsrStatus*)

Processes the IIC Slave AMOF interrupt.

This function processes the Address Match 0 Flag interrupt for the IIC Slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
in	<i>u32IsrStatus</i>	The status of the SSR flags.

void FCIIC_SlaveInterruptProcessReceive (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Processes the IIC Slave RDF interrupt.

This function processes the Receive Data Flag interrupt for the IIC Slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
----	--------------------------	--

void FCIIC_SlaveInterruptProcessStop (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Processes IIC Slave SDF interrupt.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

void FCIIC_SlaveInterruptProcessTransmit (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Processes IIC Slave TDF interrupt.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

**FCIIC_StatusType FCIIC_Slavelistening (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*,
FCIIC_SlaveTransferType *const *transfer*)**

De-initializes the IIC Slave mode.

This function de-initializes the IIC Slave mode by resetting the handle and disabling the slave.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the slave handle structure.
in	<i>transfer</i>	Pointer to the I2C transfer parameters.

Returns

FCIIC_StatusType indicating the status of the operation.

uint8_t FCIIC_SlaveReceive (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*, uint8_t * *pData*)

Receives data in IIC Slave mode.

This function receives data when polling (not used when RX interrupt is enabled); in Slave mode.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC slave handle structure.
out	<i>pData</i>	Pointer to the data buffer where the received data will be stored.

Returns

0 if successful, non-zero otherwise.

void FCIIC_SlaveStartDMA (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Starts DMA for IIC Slave in asynchronous mode.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

FCIIC_StatusType FCIIC_SlaveTransferDMA (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Initiates an IIC Slave Asynchronous DMA transfer.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

Returns

FCIIC_StatusType Status indicating whether the DMA transfer was successful.

FCIIC_StatusType FCIIC_SlaveTransferNonBlocking (FCIIC_SlaveHandleType *const *pFciicSlaveHandle*)

Starts accepting slave transfers in non-blocking mode.

Parameters

in	<i>pFciicSlaveHandle</i>	Pointer to the FCIIC Slave handle.
----	--------------------------	------------------------------------

Returns

FCIIC_StatusType Status indicating whether the start was successful.

Module_driver_fcpit

Collaboration diagram for Module_driver_fcpit:Modules

HwA_fcpit

Classes

struct **FCPIT_TriggerType**

struct **FCPIT_CommonInitType**

struct **_FCPIT_HandleType**

struct **FCPIT_IntType**

Fcpit interrupt structure.

struct **FCPIT_ChannellInitType**

Fcpit initialization type.

Typedefs

typedef void(* **FCPIT_InterruptCallbackType**) (void)

callback function type

typedef struct **_FCPIT_HandleType** **FCPIT_HandleType**

Enumerations

enum **FCPIT_InstanceType** { **FCPIT_INSTANCE_0**, **FCPIT_INSTANCE_1**, **FCPIT_INSTANCE_MAX** }

FCPIT instance.

enum **FCPIT_TriggerSelectType** { **FCPIT_TRIGGER_INTERNAL_0** = 0, **FCPIT_TRIGGER_INTERNAL_1**,
FCPIT_TRIGGER_INTERNAL_2, **FCPIT_TRIGGER_INTERNAL_3**, **FCPIT_TRIGGER_EXTERNAL**,
FCPIT_NO_TRIGGER }

Fcpit trigger source.

Functions

void **FCPIT_CommonInit** (**FCPIT_HandleType** *pFcpitHandle, const **FCPIT_CommonInitType** *pInitStruct)

Initialize Fcpit instance common feature.

void **FCPIT_ChannelInit** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_ChannelInitType** *pInitStruct)
Initialize Fcpit instance.

void **FCPIT_InitTrigger** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_TriggerType** *pTrgStruct)
Initialize Fcpit trigger configuration.

void **FCPIT_Deinit** (FCPIT_HandleType *pFcpitHandle)
De-initialize Fcpit instance.

void **FCPIT_InitInterrupt** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_IntType** *pIntStruct)
Initialize Fcpit interrupt functionality.

void **FCPIT_EnableInterrupt** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_ChannelType** eChannel)
Enable Fcpit interrupt.

void **FCPIT_DisableInterrupt** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_ChannelType** eChannel)
Disable Fcpit interrupt.

void **FCPIT_Start** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_ChannelType** eChannel)
Fcpit start timer.

void **FCPIT_Stop** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_ChannelType** eChannel)
Fcpit stop.

void **FCPIT_UpdateChannelValue** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_ChannelType** eChannel, const uint32_t u32ChannelValue)
Update Fcpit channel value.

void **FCPIT_ImmediateUpdateChannelValue** (FCPIT_HandleType *pFcpitHandle, const **FCPIT_ChannelType** eChannel, const uint32_t u32ChannelValue)
Immediately update Fcpit channel value.

void **FCPITn_IRQHandler** (FCPIT_HandleType *pFcpitHandle)

fcpit interrupt function.

FCPIT API Service IDs

```
#define FCPIT_COMMON_INIT_ID 0U
#define FCPIT_CHANNEL_INIT_ID 1U
#define FCPIT_INIT_TRIGGER_ID 2U
#define FCPIT_DE_INIT_ID 3U
#define FCPIT_INIT_INTERRUPT_ID 4U
#define FCPIT_ENABLE_INTERRUPT_ID 5U
#define FCPIT_DISABLE_INTERRUPT_ID 6U
#define FCPIT_START_ID 7U
#define FCPIT_STOP_ID 8U
#define FCPIT_UPDATEVALUE_ID 9U
#define FCPIT_UPDATEVALUE_IMMEDIATE_ID 10U
```

FCPIT Dev Error Code

Error Code of calling FCPIT apis

```
#define FCPIT_STATUS_FUNCTION_ERROR 0x0U
#define FCPIT_E_PARAM_CHANNEL 0x02U
#define FCPIT_E_PARAM_POINTER 0x03U
#define FCPIT_E_PARAM_COUNT 0x04U
#define FCPIT_E_PARAM_INVALID 0x05U
```

Detailed Description

Macro Definition Documentation

#define FCPIT_CHANNEL_INIT_ID 1U

#define FCPIT_COMMON_INIT_ID 0U

#define FCPIT_DE_INIT_ID 3U

#define FCPIT_DISABLE_INTERRUPT_ID 6U

#define FCPIT_E_PARAM_CHANNEL 0x02U

#define FCPIT_E_PARAM_COUNT 0x04U

#define FCPIT_E_PARAM_INVALID 0x05U

#define FCPIT_E_PARAM_POINTER 0x03U

#define FCPIT_ENABLE_INTERRUPT_ID 5U

#define FCPIT_INIT_INTERRUPT_ID 4U

#define FCPIT_INIT_TRIGGER_ID 2U

#define FCPIT_START_ID 7U

#define FCPIT_STATUS_FUNCTION_ERROR 0x0U

#define FCPIT_STOP_ID 8U

#define FCPIT_UPDATEVALUE_ID 9U

#define FCPIT_UPDATEVALUE_IMMEDIATE_ID 10U

Typedef Documentation

typedef struct _FCPIT_HandleType FCPIT_HandleType

typedef void(* FCPIT_InterruptCallBackType) (void)

callback function type

Enumeration Type Documentation

enum FCPIT_InstanceType

FCPIT instance.

Enumerator:

FCPIT_INSTANC E_0	
FCPIT_INSTANC E_1	
FCPIT_INSTANC E_MAX	

enum FCPIT_TriggerSelectType

Fcpit trigger source.

Enumerator:

FCPIT_TRIGGER _INTERNAL_0	
FCPIT_TRIGGER _INTERNAL_1	
FCPIT_TRIGGER _INTERNAL_2	
FCPIT_TRIGGER _INTERNAL_3	
FCPIT_TRIGGER _EXTERNAL	
FCPIT_NO_TRIG GER	

Function Documentation

void FCPIT_ChannelInit (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_ChannelInitType * *pInitStruct*)

Initialize Fcpit instance.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>pInitStruct</i>	Fcpit initialization structure

void FCPIT_CommonInit (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_CommonInitType * *pInitStruct*)

Initialize Fcpit instance common feature.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>pInitStruct</i>	Fcpit initialization structure

void FCPIT_Deinit (FCPIT_HandleType * *pFcpitHandle*)

De-initialize Fcpit instance.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
---------------------	-------------------------

void FCPIT_DisableInterrupt (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_ChannelType *eChannel*)

Disable Fcpit interrupt.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>eChannel</i>	Fcpit channel

void FCPIT_EnableInterrupt (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_ChannelType *eChannel*)

Enable Fcpit interrupt.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>eChannel</i>	Fcpit channel

void FCPIT_ImmediateUpdateChannelValue (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_ChannelType *eChannel*, const uint32_t *u32ChannelValue*)

Immediately update Fcpit channel value.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>eChannel</i>	Fcpit channel
<i>u32ChannelValue</i>	in/Out value

void FCPIT_InitInterrupt (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_IntType * *pIntStruct*)

Initialize Fcpit interrupt functionality.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>pIntStruct</i>	Fcpit interrupt structure

Note

this function will stop timer

void FCPIT_InitTrigger (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_TriggerType * *pTrgStruct*)

Initialize Fcpit trigger configuration.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>pTrgStruct</i>	Fcpit trigger structure

void FCPIT_Start (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_ChannelType *eChannel*)

Fcpit start timer.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>eChannel</i>	Fcpit channel

void FCPIT_Stop (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_ChannelType *eChannel*)

Fcpit stop.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>eChannel</i>	Fcpit channel

void FCPIT_UpdateChannelValue (FCPIT_HandleType * *pFcpitHandle*, const FCPIT_ChannelType *eChannel*, const uint32_t *u32ChannelValue*)

Update Fcpit channel value.

Parameters

<i>pFcpitHandle</i>	Fcpit processing handle
<i>eChannel</i>	Fcpit channel
<i>u32ChannelValue</i>	in/Out value

void FCPITn_IRQHandler (FCPIT_HandleType * *pFcpitHandle*)

fcpit interrupt function.

Parameters

<i>pFcpiHandle</i>	Fcpit processing handle
--------------------	-------------------------

Module_driver_fcsmu

Collaboration diagram for Module_driver_fcsmu:Modules

HwA_fcsmu

Classes

struct **FCSMU_InitCfgType**

FCSMU init configure structure.

struct **FCSMU_StatusOutputCfgType**

FCSMU status output pin configure structure.

struct **FCSMU_CrcCfgType**

FCSMU CRC configure structure.

struct **_FCSMU_HandleType**

The structure of the FCSMU processing handle.

Typedefs

typedef struct **_FCSMU_HandleType** **FCSMU_HandleType**

The structure of the FCSMU processing handle.

Enumerations

enum **FCSMU_InstanceType** { **FCSMU_INSTANCE_0** = 0U }

FCSMU Instance type.

enum **FCSMU_StatusType** { **FCSMU_STATUS_SUCCESS** = 0U, **FCSMU_STATUS_FAIL** = 1U,
FCSMU_STATUS_TIMEOUT = 2U }

FCSMU operation return values.

Functions

FCSMU_StatusType **FCSMU_Init** (**FCSMU_HandleType** *const pHandle, const **FCSMU_InitCfgType** *const pInitCfg)

Initializes the FCSMU module.

FCSMU_StatusType FCSMU_Deinit (FCSMU_HandleType *const pHandle)

Deinitializes the FCSMU module.

FCSMU_StatusType FCSMU_StatusOutputInit (FCSMU_HandleType *const pHandle, const FCSMU_StatusOutputCfgType *const pInitCfg)

Initializes the status output configuration for the FCSMU module.

FCSMU_StatusType FCSMU_StatusOutputDeinit (FCSMU_HandleType *const pHandle)

Deinitializes the status output configuration for the FCSMU module.

uint32_t FCSMU_GetFaultChannel (FCSMU_HandleType *const pHandle)

Retrieves the fault channel for the specified FCSMU module.

FCSMU_StatusType FCSMU_ClearFaultFlag (FCSMU_HandleType *const pHandle, uint32_t u32FaultChannel)

Clears the fault flag for the specified fault channel in the FCSMU module.

FCSMU_StatusType FCSMU_InjectionFault (FCSMU_HandleType *const pHandle, uint32_t u32ChannelIndex)

Injects a fault into the specified channel in the FCSMU module.

FCSMU_StatusType FCSMU_CrcInit (FCSMU_HandleType *const pHandle, const FCSMU_CrcCfgType *const pInitCfg)

Initializes the CRC checker for the FCSMU module.

FCSMU_StatusType FCSMU_CrcDeinit (FCSMU_HandleType *const pHandle)

Deinitializes the CRC checker for the FCSMU module.

FCSMU_StatusType FCSMU_CrcGen (FCSMU_HandleType *const pHandle)

Generates a CRC for the specified FCSMU module.

bool FCSMU_CrcIsBusy (FCSMU_HandleType *const pHandle)

Checks if the CRC generation process is busy for the specified FCSMU module.

uint32_t **FCSMU_CrcGetResult** (**FCSMU_HandleType** *const pHandle)

Retrieves the result of the CRC calculation for the specified FCSMU module.

void **FCSMU_CommonIRQHandler** (**FCSMU_HandleType** *const pHandle)

Handles common interrupts for the FCSMU module.

FCSMU API Service IDs

#define **FCSMU_INIT_ID** 0x00U

#define **FCSMU_DEINIT_ID** 0x01U

#define **FCSMU_STATUSOUTPUTINIT_ID** 0x02U

#define **FCSMU_STATUSOUTPUTDEINIT_ID** 0x03U

#define **FCSMU_GETFAULTCHANNEL_ID** 0x04U

#define **FCSMU_CLEARFAULT_ID** 0x05U

#define **FCSMU_INJECTIONFAULT_ID** 0x06U

#define **FCSMU_CRCINIT_ID** 0x07U

#define **FCSMU_CRCDEINIT_ID** 0x08U

#define **FCSMU_CRCGEN_ID** 0x09U

#define **FCSMU_CRCISBUSY_ID** 0x0AU

#define **FCSMU_CRCCLREBUSY_ID** 0x0BU

#define **FCSMU_CRCGETRESULT_ID** 0x0CU

#define **FCSMU_COMMONIRQHANDLER_ID** 0x0DU

FCSMU Dev Error Code

Error Code of calling FCSMU apis

#define **FCSMU_E_PARAM_NULLPTR** 0x01U

#define **FCSMU_E_PARAM_INSTANCE** 0x02U

#define **FCSMU_OP_STATE_IDLE** 0U

FCSMU operation status define.

#define **FCSMU_OP_STATE_BUSY** 1U

#define **FCSMU_OP_STATE_FAILED** 2U

#define **FCSMU_OP_STATE_SUCCESSFUL** 3U

#define **FCSMU_OP_CODE_NONE** 0U

FCSMU operation code define.

```
#define FCSMU_OP_CODE_MOVE_TO_CONFIG 1U
#define FCSMU_OP_CODE_MOVE_TO_NORMAL 2U
#define FCSMU_OP_CODE_CLEAR_FAULT_INFO 13U
#define FCSMU_OP_CODE_CLEAR_OPS_TO_IDLE 15U
#define FCSMU_SOUT_CTRL_BY_FSM 0U
```

FCSMU SOUT Control define.

```
#define FCSMU_SOUT_CTRL_KEEP_LOW 1U
#define FCSMU_SOUT_CTRL_BY_FSM2 2U
#define FCSMU_SOUT_CTRL_KEEP_HIGH_THEN_FSM 3U
#define FCSMU_SOUT_DEFAULT_POLARITY 0U
```

FCSMU SOUT Polarity Switch define.

```
#define FCSMU_SOUT_SWITCH_POLARITY 1U
#define FCSMU_SOUT_PROTOCOL_DUAL_RAIL 0U
```

FCSMU SOUT Protocol define.

```
#define FCSMU_SOUT_PROTOCOL_TIME_SWITCH 1U
#define FCSMU_SOUT_PROTOCOL_BISTABLE 2U
#define FCSMU_SOUT_PROTOCOL_FAULT_TOGGLE 3U
#define FCSMU_SOUT_PROTOCOL_TIME_DUAL_RAIL 4U
#define FCSMU_SOUT_PROTOCOL_DIAG0 5U
#define FCSMU_SOUT_PROTOCOL_DIAG1 6U
#define FCSMU_SOUT_PROTOCOL_DIAG2 7U
#define FCSMU_STATE_NORMAL 0U
```

FCSMU state in state register define.

```
#define FCSMU_STATE_CONGIG 1U
#define FCSMU_STATE_WARN 2U
#define FCSMU_STATE_FAULT 3U
#define FCSMU_FAULT_TOLERANCE_TIME_1MS 0U
```

FCSMU fault tolerance time define.

```
#define FCSMU_FAULT_TOLERANCE_TIME_2MS 1U
#define FCSMU_FAULT_TOLERANCE_TIME_5MS 2U
```



```

#define FCSMU_FAULT_TOLERANCE_TIME_10MS 3U
#define FCSMU_CRC_STATE_IDLE 0U
FCSMU crc Status define.

#define FCSMU_CRC_STATE_BUSY 1U
#define FCSMU_CRC_SW_MODE 0U
FCSMU crc mode define.

#define FCSMU_CRC_TRIGGER_MODE 1U
#define FCSMU_CRC_ERROR_REQ_MODE_NONE 0U
FCSMU crc error request mode define.

#define FCSMU_CRC_ERROR_REQ_MODE_IRQ 1U
#define FCSMU_CRC_ERROR_REQ_MODE_RST 2U

```

Detailed Description

Macro Definition Documentation

```

#define FCSMU_CLEARFAULT_ID 0x05U

#define FCSMU_COMMONIRQHANDLER_ID 0x0DU

#define FCSMU_CRC_ERROR_REQ_MODE_IRQ 1U

#define FCSMU_CRC_ERROR_REQ_MODE_NONE 0U

FCSMU crc error request mode define.

#define FCSMU_CRC_ERROR_REQ_MODE_RST 2U

#define FCSMU_CRC_STATE_BUSY 1U

FCSMU crc status busy

```

#define FCSMU_CRC_STATE_IDLE 0U

FCSMU crc Status define.

FCSMU crc status idle

#define FCSMU_CRC_SW_MODE 0U

FCSMU crc mode define.

FCSMU crc software mode.

#define FCSMU_CRC_TRIGGER_MODE 1U

FCSMU crc trigger mode.

#define FCSMU_CRC_CLEARBUSY_ID 0x0BU

#define FCSMU_CRC_DEINIT_ID 0x08U

#define FCSMU_CRC_GEN_ID 0x09U

#define FCSMU_CRC_GETRESULT_ID 0x0CU

#define FCSMU_CRC_INIT_ID 0x07U

#define FCSMU_CRC_ISBUSY_ID 0x0AU

#define FCSMU_DEINIT_ID 0x01U

#define FCSMU_E_PARAM_INSTANCE 0x02U

#define FCSMU_E_PARAM_NULLPTR 0x01U

#define FCSMU_FAULT_TOLERANCE_TIME_10MS 3U

#define FCSMU_FAULT_TOLERANCE_TIME_1MS 0U

FCSMU fault tolerance time define.

```

#define FCSMU_FAULT_TOLERANCE_TIME_2MS 1U

#define FCSMU_FAULT_TOLERANCE_TIME_5MS 2U

#define FCSMU_GETFAULTCHANNEL_ID 0x04U

#define FCSMU_INIT_ID 0x00U

#define FCSMU_INJECTIONFAULT_ID 0x06U

#define FCSMU_OP_CODE_CLEAR_FAULT_INFO 13U
    FCSMU operation code clear fault channel information register

#define FCSMU_OP_CODE_CLEAR_OPS_TO_IDLE 15U
    FCSMU operation code clear ops to idle

#define FCSMU_OP_CODE_MOVE_TO_CONFIG 1U
    FCSMU operation code move to configuration state

#define FCSMU_OP_CODE_MOVE_TO_NORMAL 2U
    FCSMU operation code move to normal state

#define FCSMU_OP_CODE_NONE 0U

    FCSMU operation code define.
    FCSMU operation code none

#define FCSMU_OP_STATE_BUSY 1U
    FCSMU operation status busy

#define FCSMU_OP_STATE_FAILED 2U
    FCSMU operation status failed

#define FCSMU_OP_STATE_IDLE 0U

    FCSMU operation status define.
    FCSMU operation status idle

```

#define FCSMU_OP_STATE_SUCCESSFUL 3U

FCSMU operation status successful

#define FCSMU_SOUT_CTRL_BY_FSM 0U

FCSMU SOUT Control define.

SOUT is controlled by the FSM.

#define FCSMU_SOUT_CTRL_BY_FSM2 2U

SOUT is controlled by the FSM.

#define FCSMU_SOUT_CTRL_KEEP_HIGH_THEN_FSM 3U

SOUT keeps high until a fault occurs on a channel, then controlled by FSM.

#define FCSMU_SOUT_CTRL_KEEP_LOW 1U

SOUT keeps low.

#define FCSMU_SOUT_DEFAULT_POLARITY 0U

FCSMU SOUT Polarity Switch define.

Default polarity.

#define FCSMU_SOUT_PROTOCOL_BISTABLE 2U

#define FCSMU_SOUT_PROTOCOL_DIAG0 5U

#define FCSMU_SOUT_PROTOCOL_DIAG1 6U

#define FCSMU_SOUT_PROTOCOL_DIAG2 7U

#define FCSMU_SOUT_PROTOCOL_DUAL_RAIL 0U

FCSMU SOUT Protocol define.

#define FCSMU_SOUT_PROTOCOL_FAULT_TOGGLE 3U

#define FCSMU_SOUT_PROTOCOL_TIME_DUAL_RAIL 4U

#define FCSMU_SOUT_PROTOCOL_TIME_SWITCH 1U

#define FCSMU_SOUT_SWITCH_POLARITY 1U

Switch polarity.

#define FCSMU_STATE_CONGIG 1U

FCSMU state config

#define FCSMU_STATE_FAULT 3U

FCSMU state fault

#define FCSMU_STATE_NORMAL 0U

FCSMU state in state register define.

FCSMU state normal

#define FCSMU_STATE_WARN 2U

FCSMU state warn

#define FCSMU_STATUSOUTPUTDEINIT_ID 0x03U

#define FCSMU_STATUSOUTPUTINIT_ID 0x02U

Typedef Documentation

typedef struct _FCSMU_HandleType FCSMU_HandleType

The structure of the FCSMU processing handle.

Enumeration Type Documentation

enum FCSMU_InstanceType

FCSMU Instance type.

Enumerator:

FCSMU_INSTAN CE_0	
----------------------	--

enum FCSMU_StatusType

FCSMU operation return values.

Enumerator:

FCSMU_STATUS _SUCCESS	FCSMU operation is succeed.
FCSMU_STATUS _FAIL	FCSMU operation is failed.
FCSMU_STATUS _TIMEOUT	FCSMU operation is timeout.

Function Documentation

FCSMU_StatusType FCSMU_ClearFaultFlag (FCSMU_HandleType *const *pHandle*, uint32_t *u32FaultChannel*)

Clears the fault flag for the specified fault channel in the FCSMU module.

This function clears the fault flag for the specified fault channel in the specified FCSMU module instance by unlocking the fault clear mechanism, setting the fault channel to clear, and waiting for the operation to complete.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
<i>u32FaultChanne l</i>	The fault channel whose fault flag should be cleared.

Returns

FCSMU_StatusType Indicates the status of the fault flag clearing process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_TIMEOUT).

Note

If the handle pointer or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

void FCSMU_CommonIRQHandler (FCSMU_HandleType *const *pHandle*)

Handles common interrupts for the FCSMU module.

This static function handles common interrupts for the specified FCSMU module instance. It processes fault, warning, and configuration timeout interrupts by retrieving relevant information and invoking appropriate callbacks.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Note

This function is intended for internal use only and should not be called directly.

If the handle pointer or instance index is invalid, a device error will be reported.

FCSMU_StatusType FCSMU_CrcDeinit (FCSMU_HandleType *const *pHandle*)

Deinitializes the CRC checker for the FCSMU module.

This function deinitializes the CRC checker for the specified FCSMU module instance by clearing the CRC control register and the CRC error flag.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Returns

FCSMU_StatusType Indicates the status of the CRC deinitialization process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_FAIL).

Note

If the handle pointer or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

FCSMU_StatusType FCSMU_CrcGen (FCSMU_HandleType *const *pHandle*)

Generates a CRC for the specified FCSMU module.

This function generates a CRC for the specified FCSMU module instance by initiating the CRC generation process.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Returns

FCSMU_StatusType Indicates the status of the CRC generation process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_FAIL).

Note

If the handle pointer or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

uint32_t FCSMU_CrcGetResult (FCSMU_HandleType *const *pHandle*)

Retrieves the result of the CRC calculation for the specified FCSMU module.

This function retrieves the result of the CRC calculation for the specified FCSMU module instance.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Returns

uint32_t The result of the CRC calculation.

Note

If the handle pointer or instance index is invalid, a device error will be reported.

FCSMU_StatusType FCSMU_CrcInit (FCSMU_HandleType *const *pHandle*, const FCSMU_CrcCfgType *const *pInitCfg*)

Initializes the CRC checker for the FCSMU module.

This function initializes the CRC checker for the specified FCSMU module instance by generating a CRC, configuring the trigger mode and error request mode, and enabling the CRC checker and error output.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
<i>pInitCfg</i>	Pointer to the FCSMU CRC configuration structure.

Returns

FCSMU_StatusType Indicates the status of the CRC initialization process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_TIMEOUT).

Note

If the handle pointer, initialization configuration pointer, or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

bool FCSMU_CrcIsBusy (FCSMU_HandleType *const *pHandle*)

Checks if the CRC generation process is busy for the specified FCSMU module.

This function checks if the CRC generation process is currently busy for the specified FCSMU module instance.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Returns

bool Indicates whether the CRC generation process is busy (true) or not (false).

Note

If the handle pointer or instance index is invalid, a device error will be reported.

FCSMU_StatusType FCSMU_Deinit (FCSMU_HandleType *const *pHandle*)

Deinitializes the FCSMU module.

This function deinitializes the specified FCSMU module instance by transitioning the state to the configuration state, resetting the configuration settings, and then transitioning back to the normal state.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Returns

FCSMU_StatusType Indicates the status of the deinitialization process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_FAIL).

Note

If the handle pointer or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

uint32_t FCSMU_GetFaultChannel (FCSMU_HandleType *const *pHandle*)

Retrieves the fault channel for the specified FCSMU module.

This function retrieves the fault channel for the specified FCSMU module instance.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Returns

uint32_t The fault channel number.

Note

If the handle pointer or instance index is invalid, a device error will be reported.

FCSMU_StatusType FCSMU_Init (FCSMU_HandleType *const *pHandle*, const FCSMU_InitCfgType *const *pInitCfg*)

Initializes the FCSMU module.

This function initializes the specified FCSMU module instance by transitioning the state to the configuration state, configuring the module according to the provided initialization configuration, and then transitioning back to the normal state.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
<i>pInitCfg</i>	Pointer to the FCSMU initialization configuration structure.

Returns

FCSMU_StatusType Indicates the status of the initialization process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_FAIL).

Note

If the handle pointer, initialization configuration pointer, or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

FCSMU_StatusType FCSMU_InjectionFault (FCSMU_HandleType *const *pHandle*, uint32_t *u32ChannelIndex*)

Injects a fault into the specified channel in the FCSMU module.

This function injects a fault into the specified channel in the specified FCSMU module instance by setting the injection register.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
<i>u32ChannelIndex</i>	The channel index where the fault should be injected.

Returns

FCSMU_StatusType Indicates the status of the fault injection process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_FAIL).

Note

If the handle pointer or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

FCSMU_StatusType FCSMU_StatusOutputDeinit (FCSMU_HandleType *const *pHandle*)

Deinitializes the status output configuration for the FCSMU module.

This function deinitializes the status output configuration for the specified FCSMU module instance by transitioning the state to the configuration state, clearing the status output control register, and then transitioning back to the normal state.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
----------------	--

Returns

FCSMU_StatusType Indicates the status of the status output deinitialization process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_FAIL).

Note

If the handle pointer or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

FCSMU_StatusType FCSMU_StatusOutputInit (FCSMU_HandleType *const *pHandle*, const FCSMU_StatusOutputCfgType *const *pInitCfg*)

Initializes the status output configuration for the FCSMU module.

This function initializes the status output configuration for the specified FCSMU module instance by transitioning the state to the configuration state, setting the status output control register, and then transitioning back to the normal state.

Parameters

<i>pHandle</i>	Pointer to the FCSMU handle structure.
<i>pInitCfg</i>	Pointer to the FCSMU status output configuration structure.

Returns

FCSMU_StatusType Indicates the status of the status output initialization process (FCSMU_STATUS_SUCCESS or FCSMU_STATUS_FAIL).

Note

If the handle pointer, initialization configuration pointer, or instance index is invalid, a device error will be reported, and the function returns FCSMU_STATUS_FAIL.

Module_driver_fcspi

Classes

struct _FCSPI_HandleType

The structure of the FCSPI processing handle.

Macros

#define FCSPI_DEV_ERROR_REPORT STD_ON

Typedefs

typedef struct _FCSPI_HandleType FCSPI_HandleType

The structure of the FCSPI processing handle.

Enumerations

```
enum FCSPI_StateType { FCSPI_STATE_UNINIT = 0x00U, FCSPI_STATE_IDLE = 0x01U,  
FCSPI_STATE_BUSY = 0x02U, FCSPI_STATE_BUSY_TX = 0x03U, FCSPI_STATE_BUSY_RX = 0x04U,  
FCSPI_STATE_BUSY_TX_RX = 0x05U, FCSPI_STATE_ERROR = 0x06U, FCSPI_STATE_ABORT = 0x07U }
```

FCSPI states.

```
enum FCSPI_StatusType { FCSPI_STATUS_SUCCESS = 0x0U, FCSPI_STATUS_ERROR = 0x1U,  
FCSPI_STATUS_TIMEOUT = 0x2U }
```

Return value of FCSPi API services.

```
enum FCSPI_DataAccessWidthType { FCSPI_DATA_ACCESS_8_BIT = 0x0U, FCSPI_DATA_ACCESS_16_BIT  
= 0x1U, FCSPI_DATA_ACCESS_32_BIT = 0x2U }
```

Data access width according to differen frame size user configured.

```
enum FCSPI_PcsModeType { FCSPI_PCS_HARDWARE_SEL = 0x0U, FCSPI_PCS_GPIO_SEL = 0x1U,  
FCSPI_PCS_NONE = 0x2U }
```

Configuration item: The user chooses what strategy to use for the chip selection signal(PCS).

```
enum FCSPI_SckFeature { FCSPI_SCK_FEATURE_PRECISE_FREQ = 0x0,  
FCSPI_SCK_FEATURE_50_DUTY_CYCLE = 0x1 }
```

Functions

```
uint32_t FCSPI_LL_CaculateSckdivPrescale (uint32_t *Prescale, uint32_t *Sckdiv, uint32_t InputClkFreq,  
uint32_t TargetSckFreq)
```

```
FCSPI_StatusType FCSPI_Init (FCSPI_HandleType *pFcspiHandle)
```

Initialize FCSPi based on configurations in FCSPi_HandleType.

```
void FCSPI_DeInit (FCSPI_HandleType *pFcspiHandle)
```

De-init FCSPi peripheral and 'tStatus' of FCSPi_HandleType structure.

```
FCSPI_StatusType FCSPI_Transmit (FCSPI_HandleType *pFcspiHandle, uint8_t *pData, uint32_t Size)
```

Transmit data in blocking(polling) mode.

```
FCSPI_StatusType FCSPI_Receive (FCSPI_HandleType *pFcspiHandle, uint8_t *pRxData, uint32_t Size)
```

Receive data in blocking(polling) mode.

FCSPI_StatusType FCSPI_TransmitReceive (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint8_t *pRxData, uint32_t Size)

Transmit and receive data in blocking(polling) mode.

FCSPI_StatusType FCSPI_Transmit_IT (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint32_t Size)

Transmit data in interrupt mode(non-blocking).

FCSPI_StatusType FCSPI_Receive_IT (FCSPI_HandleType *pFcspiHandle, uint8_t *pRxData, uint32_t Size)

Receive data in interrupt mode(non-blocking).

FCSPI_StatusType FCSPI_TransmitReceive_IT (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint8_t *pRxData, uint32_t Size)

Transmit and receive data in interrupt mode(non-blocking).

FCSPI_StatusType FCSPI_Transmit_DMA (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint32_t Size)

Transmit data in DMA mode(non-blocking).

FCSPI_StatusType FCSPI_Receive_DMA (FCSPI_HandleType *pFcspiHandle, uint8_t *pRxData, uint32_t Size)

Receive data in DMA mode(non-blocking).

FCSPI_StatusType FCSPI_TransmitReceive_DMA (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint8_t *pRxData, uint32_t Size)

Transmit and receive data in DMA mode(non-blocking).

void FCSPIn_IRQHandler (FCSPI_HandleType *pFcspiHandle)

The internal interrupt handler function for FCSPI instances.

void FCSPIn_DmaDoneCallback_Tx (void *pFcspiHandle)

FCSPI callback of DMA channel triggered by Tx FIFO.

void FCSPIn_DmaDoneCallback_Rx (void *pFcspiHandle)

FCSPI callback of DMA channel triggered by Rx FIFO.

FCSPI API Service IDs

```
#define FCSPI_INIT_ID 0U
#define FCSPI_DEINIT_ID 1U
#define FCSPI_TRANSMIT_ID 3U
#define FCSPI_RECEIVE_ID 4U
#define FCSPI_TRANSMITRECEIVE_ID 5U
#define FCSPI_TRANSMIT_IT_ID 6U
#define FCSPI_RECEIVE_IT_ID 7U
#define FCSPI_TRANSMITRECEIVE_IT_ID 8U
#define FCSPI_TRANSMIT_DMA_ID 9U
#define FCSPI_RECEIVE_DMA_ID 10U
#define FCSPI_TRANSMITRECEIVE_DMA_ID 11U
```

FCSPI Dev Error Code

Error Code of calling FCSPI apis

```
#define FCSPI_E_PARAM_UNINIT 0x01U /* API service called without module initialization */
#define FCSPI_E_PARAM_POINTER 0x02U /* API service called with an unexpected value for the pointer */
#define FCSPI_E_PARAM_CONFIG 0x03U /* API service called with unreasonable configuration */
```

Detailed Description

Macro Definition Documentation

```
#define FCSPI_DEINIT_ID 1U
```

```
#define FCSPI_DEV_ERROR_REPORT STD_ON
```

```
#define FCSPI_E_PARAM_CONFIG 0x03U /* API service called with unreasonable configuration */
```

```
#define FCSPI_E_PARAM_POINTER 0x02U /* API service called with an unexpected value for the  
pointer */
```

```
#define FCSPI_E_PARAM_UNINIT 0x01U /* API service called without module initialization */
```

```
#define FCSPI_INIT_ID 0U
```

```
#define FCSPI_RECEIVE_DMA_ID 10U
```

```
#define FCSPI_RECEIVE_ID 4U
```

```
#define FCSPI_RECEIVE_IT_ID 7U
```

```
#define FCSPI_TRANSMIT_DMA_ID 9U
```

```
#define FCSPI_TRANSMIT_ID 3U
```

```
#define FCSPI_TRANSMIT_IT_ID 6U
```

```
#define FCSPI_TRANSMITRECEIVE_DMA_ID 11U
```

```
#define FCSPI_TRANSMITRECEIVE_ID 5U
```

```
#define FCSPI_TRANSMITRECEIVE_IT_ID 8U
```

Typedef Documentation

```
typedef struct _FCSPI_HandleType FCSPI_HandleType
```

The structure of the FCSPI processing handle.

Enumeration Type Documentation

enum FCSPI_DataAccessWidthType

Data access width according to differen frame size user configured.

Enumerator:

FCSPI_DATA_ACCESS_8_BIT	Access FCSPI DATA register and Tx/Rx buffer by byte
FCSPI_DATA_ACCESS_16_BIT	Access FCSPI DATA register and Tx/Rx buffer by half word
FCSPI_DATA_ACCESS_32_BIT	Access FCSPI DATA register and Tx/Rx buffer by word

enum FCSPI_PcsModeType

Configuration item: The user chooses what strategy to use for the chip selection signal(PCS).

Enumerator:

FCSPI_PCS_HARDWARE_SEL	PCS(CS,Chip Select) is automatically controlled by the FCSPI peripheral hardware.
FCSPI_PCS_GPIO_SEL	User will chose a GPIO pin and manually change its level high and low to simulate the PCS.
FCSPI_PCS_NONE	No need PCS.

enum FCSPI_SckFeature

Enumerator:

FCSPI_SCK_FEATURE_PRECISE_FREQ	
FCSPI_SCK_FEATURE_50_DUTY_CYCLE	

enum FCSPI_StateType

FCSPI states.

Enumerator:

FCSPI_STATE_UNINIT	Peripheral not Initialized
FCSPI_STATE_IDLE	Peripheral Initialized and idle for use
FCSPI_STATE_BUSY	an internal process is ongoing
FCSPI_STATE_BUSY_TX	Data Transmission process is ongoing
FCSPI_STATE_BUSY_RX	Data Reception process is ongoing
FCSPI_STATE_BUSY_TX_RX	Data Transmission and Reception process is ongoing
FCSPI_STATE_ERROR	SPI error state

FCSPI_STATE_ABORT	SPI abort is ongoing

enum FCSPI_StatusType

Return value of FCSPI API services.

Enumerator:

FCSPI_STATUS_SUCCESS	The FCSPI operation is succeed
FCSPI_STATUS_ERROR	The FCSPI operation is failed
FCSPI_STATUS_TIMEOUT	The FCSPI operation is failed because of time out

Function Documentation

void FCSPI_DeInit (FCSPI_HandleType * *pFcspiHandle*)

De-init FCSPI peripheral and 'tStatus' of FCSPI_HandleType structure.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPI_HandleType structure
---------------------	---

Returns

void

FCSPI_StatusType FCSPI_Init (FCSPI_HandleType * *pFcspiHandle*)

Initialize FCSPI based on configurations in FCSPI_HandleType.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPi_HandleType structure
---------------------	---

Returns

FCSPi_StatusType

uint32_t FCSPi_LL_CaculateSckdivPrescale (uint32_t * *Prescale*, uint32_t * *Sckdiv*, uint32_t *InputClkFreq*, uint32_t *TargetSckFreq*)

FCSPi_StatusType FCSPi_Receive (FCSPi_HandleType * *pFcspiHandle*, uint8_t * *pRxData*, uint32_t *Size*)

Receive data in blocking(polling) mode.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPi_HandleType structure
<i>pRxData</i>	pointer to Rx data buffer.
<i>Size</i>	Bytes count of data to be received.

Returns

FCSPi_StatusType

FCSPi_StatusType FCSPi_Receive_DMA (FCSPi_HandleType * *pFcspiHandle*, uint8_t * *pRxData*, uint32_t *Size*)

Receive data in DMA mode(non-blocking).

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPi_HandleType structure
<i>pRxData</i>	pointer to Rx data buffer.
<i>Size</i>	Bytes count of data to be received.

Returns

FCSPi_StatusType

FCSPI_StatusType FCSPI_Receive_IT (FCSPI_HandleType * *pFcspiHandle*, uint8_t * *pRxData*, uint32_t *Size*)

Receive data in interrupt mode(non-blocking).

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPI_HandleType structure
<i>pRxData</i>	pointer to Rx data buffer.
<i>Size</i>	Bytes count of data to be received.

Returns

FCSPI_StatusType

FCSPI_StatusType FCSPI_Transmit (FCSPI_HandleType * *pFcspiHandle*, uint8_t * *pData*, uint32_t *Size*)

Transmit data in blocking(polling) mode.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPI_HandleType structure
<i>pData</i>	pointer to Tx data buffer.
<i>Size</i>	Bytes count of data to be transmitted.

Returns

FCSPI_StatusType

FCSPI_StatusType FCSPI_Transmit_DMA (FCSPI_HandleType * *pFcspiHandle*, uint8_t * *pTxData*, uint32_t *Size*)

Transmit data in DMA mode(non-blocking).

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPI_HandleType structure
<i>pTxData</i>	pointer to Tx data buffer.
<i>Size</i>	Bytes count of data to be transmitted.

Returns

FCSPI_StatusType

FCSPI_StatusType FCSPI_Transmit_IT (FCSPI_HandleType * *pFcspiHandle*, uint8_t * *pTxData*, uint32_t *Size*)

Transmit data in interrupt mode(non-blocking).

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPI_HandleType structure
<i>pTxData</i>	pointer to Tx data buffer.
<i>Size</i>	Bytes count of data to be transmitted.

Returns

FCSPI_StatusType

FCSPI_StatusType FCSPI_TransmitReceive (FCSPI_HandleType * *pFcspiHandle*, uint8_t * *pTxData*, uint8_t * *pRxData*, uint32_t *Size*)

Transmit and receive data in blocking(polling) mode.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPI_HandleType structure
<i>pTxData</i>	pointer to Tx data buffer.
<i>pRxData</i>	pointer to Rx data buffer.
<i>Size</i>	Bytes count of data to be transmitted and received.

Returns

FCSPI_StatusType

FCSPI_StatusType FCSPI_TransmitReceive_DMA (FCSPI_HandleType * *pFcspiHandle*, uint8_t * *pTxData*, uint8_t * *pRxData*, uint32_t *Size*)

Transmit and receive data in DMA mode(non-blocking).

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPi_HandleType structure
<i>pTxData</i>	pointer to Tx data buffer.
<i>pRxData</i>	pointer to Rx data buffer.
<i>Size</i>	Bytes count of data to be transmitted and received.

Returns

FCSPI_StatusType

FCSPI_StatusType FCSPi_TransmitReceive_IT (FCSPi_HandleType * *pFcspiHandle*, uint8_t * *pTxData*, uint8_t * *pRxData*, uint32_t *Size*)

Transmit and receive data in interrupt mode(non-blocking).

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPi_HandleType structure
<i>pTxData</i>	pointer to Tx data buffer.
<i>pRxData</i>	pointer to Rx data buffer.
<i>Size</i>	Bytes count of data to be transmitted and received.

Returns

FCSPI_StatusType

void FCSPIn_DmaDoneCallback_Rx (void * *pFcspiHandle*)

FCSPI callback of DMA channel triggered by Rx FIFO.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPi_HandleType structure
---------------------	---

void FCSPIn_DmaDoneCallback_Tx (void * *pFcspiHandle*)

FCSPI callback of DMA channel triggered by Tx FIFO.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPi_HandleType structure
---------------------	---

```
void FCSPIn_IRQHandler (FCSPI_HandleType * pFcspiHandle)
```

The internal interrupt handler function for FCSPI instances.

Parameters

<i>pFcspiHandle</i>	pointer to a FCSPI_HandleType structure
---------------------	---

Module_driver_fcuart

Collaboration diagram for Module_driver_fcuart:Modules

HwA_fcuart

Classes

- struct **FCUART_DataType**
UART transmit and receive data define.
- struct **_FCUART_HandleType**
The structure of the UART processing handle.
- struct **FCUART_InterruptType**
UART callback functions data define.
- struct **FCUART_InitType**
UART initial data define.
- struct **FCUART_WakeupType**
UART WakeUp functions data define.

Macros

```
#define FCUART_FIFO_DEPTH 16U
```

Typedefs

- typedef struct **_FCUART_HandleType** **FCUART_HandleType**
The structure of the UART processing handle.
- typedef void(* **FCUART_ErrorInterrupt_CallBackType**) (**FCUART_HandleType** *pUartHandle, uint32_t u32Error)

FCUART error interrupt callback function type.

```
typedef void(* FCUART_TxRxInterrupt_CallBackType) (FCUART_HandleType *pUartHandle,  
FCUART_DataType *pTxRxCfg)
```

FCUART transmit/receive interrupt callback function type.

```
typedef void(* FCUART_IdleInterrupt_CallBackType) (FCUART_HandleType *pUartHandle)
```

FCUART idle interrupt callback function type.

```
typedef uint8_t FCUART_ErrorType
```

Error type Define as uint8.

Enumerations

```
enum UART_PrintIntType { UART_PRINT_RADIX_BIN = 2U, UART_PRINT_RADIX_OCT = 8U,  
UART_PRINT_RADIX_DEC = 10U, UART_PRINT_RADIX_HEX = 16U }
```

Define the type of integer representation for UART printing.

```
enum FCUART_SequenceType { FCUART_SEQUENCE_VAR_NOINIT, FCUART_SEQUENCE_DEINIT,  
FCUART_SEQUENCE_NOTSTART_RECEIVE, FCUART_SEQUENCE_START_RECEIVE }
```

FCUART Operation Sequence.

```
enum FCUART_ErrorTypeEnum { FCUART_ERROR_OK = 0x00U, FCUART_ERROR_INVALID_VERSION =  
0x01U, FCUART_ERROR_FAILED = 0x02U, FCUART_ERROR_INVALID_PARAM = 0x04U,  
FCUART_ERROR_INVALID_SIZE = 0x08U, FCUART_ERROR_INVALID_SEQUENCE = 0x10U,  
FCUART_ERROR_TIMEOUT = 0x20U }
```

FCUART error status.

```
enum FCUART_Fifo_RxIdleCharNumType { FCUART_FIFO_RX_IDLE_DISABLE = 0U,  
FCUART_FIFO_RX_IDLE_CHARACTER_1, FCUART_FIFO_RX_IDLE_CHARACTER_2,  
FCUART_FIFO_RX_IDLE_CHARACTER_4, FCUART_FIFO_RX_IDLE_CHARACTER_8,  
FCUART_FIFO_RX_IDLE_CHARACTER_16, FCUART_FIFO_RX_IDLE_CHARACTER_32,  
FCUART_FIFO_RX_IDLE_CHARACTER_64 }
```

UART fifo receive idle character number.

```
enum FCUART_InterruptSel { FCUART_INT_CTRL_ORIE = FCUART_CTRL_ORIE_MASK,  
FCUART_INT_CTRL_NEIE = FCUART_CTRL_NEIE_MASK, FCUART_INT_CTRL_FEIE =  
FCUART_CTRL_FEIE_MASK, FCUART_INT_CTRL_PEIE = FCUART_CTRL_PEIE_MASK,  
FCUART_INT_CTRL_TIE = FCUART_CTRL_TIE_MASK, FCUART_INT_CTRL_TCIE =  
FCUART_CTRL_TCIE_MASK, FCUART_INT_CTRL_RIE = FCUART_CTRL_RIE_MASK, FCUART_INT_CTRL_IIE =
```

FCUART_CTRL_IIE_MASK, FCUART_INT_CTRL_TE = FCUART_CTRL_TE_MASK, FCUART_INT_CTRL_RE = FCUART_CTRL_RE_MASK, FCUART_INT_CTRL_MOIE = FCUART_CTRL_MOIE_MASK, FCUART_INT_CTRL_M1IE = FCUART_CTRL_M1IE_MASK }

UART CTRL register interrupt.

enum FCUART_ErrorStatusType { FCUART_ERROR_NONE = 0x0000U, FCUART_ERROR_RORF = 0x0001U, FCUART_ERROR_NF = 0x0002U, FCUART_ERROR_FEF = 0x0004U, FCUART_ERROR_PEF = 0x0008U, FCUART_ERROR_RPEF = 0x0010U, FCUART_ERROR_TPEF = 0x0020U }

UART error status type.

enum FCUART_IdleCharNumType { FCUART_IDLE_CHARACTER_1 = 0U, FCUART_IDLE_CHARACTER_2, FCUART_IDLE_CHARACTER_4, FCUART_IDLE_CHARACTER_8, FCUART_IDLE_CHARACTER_16, FCUART_IDLE_CHARACTER_32, FCUART_IDLE_CHARACTER_64, FCUART_IDLE_CHARACTER_128 }

UART idle character number.

enum FCUART_IdleStartType { FCUART_START_AFTER_STARTBIT = 0U, FCUART_START_AFTER_STOPBIT }

UART idle type.

Functions

void FCUART_InitStructure (FCUART_HandleType *pUartHandle)

Initial UART variables Memory.

FCUART_ErrorType FCUART_Init (FCUART_HandleType *pUartHandle, FCUART_InitType *pInitCfg)

This Function is used to initial UART instance.

void FCUART_Delinit (FCUART_HandleType *pUartHandle)

This Function is used to de-initial UART instance.

void FCUART_SetWakeup (FCUART_HandleType *pUartHandle, FCUART_WakeupType *pWakeupCfg)

This Function is used to set UART WakeUp.

void FCUART_StartReceive (FCUART_HandleType *pUartHandle)

This Function is used to start receiving.

void FCUART_StopReceive (FCUART_HandleType *pUartHandle)

This Function is used to stop receiving.

void **FCUART_StartTransmit** (FCUART_HandleType *pUartHandle)

This Function is used to start transmit.

void **FCUART_StopTransmit** (FCUART_HandleType *pUartHandle)

This Function is used to stop transmitting.

FCUART_ErrorType **FCUART_Transmit** (FCUART_HandleType *pUartHandle, **FCUART_DataType** *pUartData)

This Function is used to transmit UART data.

FCUART_ErrorType **FCUART_Printf** (FCUART_HandleType *pUartHandle, char *fmt,...)

This Function is used to print ASCII char from UART.

uint32_t **FCUART_GetStatus** (FCUART_HandleType *pUartHandle, **FCUART_StatType** eStatusType)

Get Stat Flag.

FCUART_ErrorType **FCUART_Receive_Polling** (FCUART_HandleType *pUartHandle, **FCUART_DataType** *pRxMsg)

This Function is used to receive data when polling (not used when rx interrupt enabled)

void **FCUART_Error_Polling** (FCUART_HandleType *pUartHandle, uint32_t *pErrorValue)

This Function is used to get error when polling (not used when error interrupt enabled)

void **FCUART_SetLoopMode** (FCUART_HandleType *pUartHandle, bool bStatus)

This Function is used to enable fcuart loop mode.

void **FCUARTN_RxTx_IRQHandler** (FCUART_HandleType *pUartHandle)

This Function is used to deal with FCUART TxRx interrupt.

void **FCUART_SendData_9Bits** (FCUART_HandleType *pUartHandle, uint16_t u16Data)

This Function is used to send 9 bits data.

void **FCUART_GetData_9Bits** (FCUART_HandleType *pUartHandle, uint16_t *pData)

This Function is used to Get 9 bits data.

void **FCUART_SendData_10Bits** (FCUART_HandleType *pUartHandle, uint16_t u16Data)

This Function is used to send 10 bits data.

void **FCUART_GetData_10Bits** (FCUART_HandleType *pUartHandle, uint16_t *pData)

This Function is used to Get 10 bits data.

void **FCUART_SetBitModeAndParity** (FCUART_HandleType *pUartHandle, FCUART_BitModeType eBitMode, FCUART_StopBitNumType eStopBit, FCUART_ParityType eParityType, bool bParityEnable)

This Function is used to set bit mode,parity and stop bit.

bool **FCUART_GetInterruptMode** (FCUART_HandleType *pUartHandle, uint32_t u32Data)

This Function is used to Get current interrupt mode.

void **FCUART_AssignTxInterruptData** (FCUART_HandleType *pUartHandle, uint8_t *pData, uint32_t u32Length)

This Function is used to assign data to send through interrupt.

FCUART API Service IDs

Defines the service IDs for the FCUART module, which are used to identify different functions or services provided by the FCUART layer. These IDs are utilized when communicating with the FCUART module, enabling calls to or responses from various functions.

```
#define FCUART_INIT_ID 0U
#define FCUART_DEINIT_ID 1U
#define FCUART_SET_INTERRUPT_ID 2U
#define FCUART_SET_WAKEUP_ID 3U
#define FCUART_GET_INTERRUPT_ID 4U
#define FCUART_START_RECEIVE_ID 5U
#define FCUART_ASSIGN_TXINTRRUPT_ID 6U
#define FCUART_START_TRANSMIT_ID 7U
#define FCUART_TRANSMIT_ID 8U
#define FCUART_PRINTF_ID 9U
#define FCUART_GET_SATTUS_ID 10U
#define FCUART_RECEIVE_POLLING_ID 11U
```

```
#define FCUART_ERROR_POLLING_ID 12U
#define FCUART_SET_LOOPMODE_ID 13U
#define FCUART_STOP_RECEIVE_ID 14U
#define FCUART_STOP_TRANSMIT_ID 15U
#define FCUART_SENDDATA_9BITS 16U
#define FCUART_GETDATA_9BITS 17U
#define FCUART_SENDDATA_10BITS 18U
#define FCUART_GETDATA_10BITS 19U
#define FCUART_SETBITMODEANDPARITY 20U
```

FCUART Dev Error Code

Error Code of calling FCUART apis

```
#define FCUART_E_PARAM_INSTANCE 0x01U
#define FCUART_E_PARAM_ERROR_STATE 0x02U
#define FCUART_E_PARAM_POINTER 0x03U
```

Special_Characters Special Characters

Special characters used in UART communication.

```
#define ESCAPE_CHARACTER (char)0x5c
#define ENTER (uint8_t)0x0d
#define NEW_LINE (uint8_t)0x0a
#define SPACE (uint8_t)0x20
#define FLOAT_ZERO 0.00000001
```

Detailed Description

Macro Definition Documentation

```
#define ENTER (uint8_t)0x0d
```

Enter character

```
#define ESCAPE_CHARACTER (char)0x5c
```

Escape character

#define FCUART_ASSIGN_TXINTRRUPT_ID 6U

Transmit interrupt assignment ID, used to assign the interrupt for transmitting data.

#define FCUART_DEINIT_ID 1U

Deinitialization ID, used to deinitialize the FCUART module.

#define FCUART_E_PARAM_ERROR_STATE 0x02U

FCUART_E_PARAM_ERROR_STATE means error state parameter is invalid

#define FCUART_E_PARAM_INSTANCE 0x01U

FCUART_E_PARAM_INSTANCE means instance parameter is invalid

#define FCUART_E_PARAM_POINTER 0x03U

FCUART_E_PARAM_POINTER means pointer parameter is invalid

#define FCUART_ERROR_POLLING_ID 12U

Error polling ID, used to poll for errors in the FCUART module.

#define FCUART_FIFO_DEPTH 16U

#define FCUART_GET_INTERRUPT_ID 4U

Interrupt status ID, used to get the interrupt status of the FCUART module.

#define FCUART_GET_SATTUS_ID 10U

Status get ID, used to get the status of the FCUART module.

#define FCUART_GETDATA_10BITS 19U

Receive data with 10 bits ID, used to receive data with 10 bits from the FCUART module.

#define FCUART_GETDATA_9BITS 17U

Receive data with 9 bits ID, used to receive data with 9 bits from the FCUART module.

#define FCUART_INIT_ID 0U

Initialization ID, used to initialize the FCUART module.

#define FCUART_PRINTF_ID 9U

Printf ID, used to print formatted data to the FCUART module.

#define FCUART_RECEIVE_POLLING_ID 11U

Receive polling ID, used to poll for received data from the FCUART module.

#define FCUART_SENDDATA_10BITS 18U

Send data with 10 bits ID, used to send data with 10 bits to the FCUART module.

#define FCUART_SENDDATA_9BITS 16U

Send data with 9 bits ID, used to send data with 9 bits to the FCUART module.

#define FCUART_SET_INTERRUPT_ID 2U

Interrupt setting ID, used to set the interrupt status of the FCUART module.

#define FCUART_SET_LOOPMODE_ID 13U

Loopback mode setting ID, used to set the loopback mode of the FCUART module.

#define FCUART_SET_WAKEUP_ID 3U

Wakeup ID, used to wake up the FCUART module.

#define FCUART_SETBITMODEANDPARITY 20U

Set bit mode and parity ID, used to set the bit mode and parity of the FCUART module.

#define FCUART_START_RECEIVE_ID 5U

Receive start ID, used to start receiving data from the FCUART module.

#define FCUART_START_TRANSMIT_ID 7U

Transmit start ID, used to start transmitting data from the FCUART module.

#define FCUART_STOP_RECEIVE_ID 14U

Receive stop ID, used to stop receiving data from the FCUART module.

#define FCUART_STOP_TRANSMIT_ID 15U

Transmit stop ID, used to stop transmitting data from the FCUART module.

#define FCUART_TRANSMIT_ID 8U

Transmit data ID, used to transmit data to the FCUART module.

#define FLOAT_ZERO 0.00000001

Floating point zero

#define NEW_LINE (uint8_t)0x0a

New line character

#define SPACE (uint8_t)0x20

Space character

Typedef Documentation

typedef void(* FCUART_ErrorInterrupt_CallBackType) (FCUART_HandleType *pUartHandle, uint32_t u32Error)

FCUART error interrupt callback function type.

This function type is used to define the callback function that is invoked when an error occurs in the FCUART module. It combines the error status with the FCUART_ErrorStatusType to notify the user of specific error conditions. This design allows users to perform custom operations when errors occur, enhancing the flexibility and error handling capabilities of UART communication.

Parameters

<i>pUartHandle</i>	Pointer to the UART handle structure containing configuration and state information.
<i>u32Error</i>	The error code indicating the specific error that occurred.

typedef uint8_t FCUART_ErrorType

Error type Define as uint8.

typedef struct _FCUART_HandleType FCUART_HandleType

The structure of the UART processing handle.

typedef void(* FCUART_IdleInterrupt_CallBackType) (FCUART_HandleType *pUartHandle)

FCUART idle interrupt callback function type.

This function type is used to define the callback function that is invoked when the UART module enters an idle state. It can be used to perform any necessary actions when there is no ongoing transmission or reception.

Parameters

<i>pUartHandle</i>	Pointer to the UART handle structure containing configuration and state information.
--------------------	--

typedef void(* FCUART_TxRxInterrupt_CallBackType) (FCUART_HandleType *pUartHandle, FCUART_DataType *pTxRxCfg)

FCUART transmit/receive interrupt callback function type.

This function type is used to define the callback function that is invoked when a transmission or reception operation completes. It provides the UART handle and data configuration parameters to allow the user to process the transmitted or received data.

Parameters

<i>pUartHandle</i>	Pointer to the UART handle structure containing configuration and state information.
<i>pTxRxCfg</i>	Pointer to the configuration structure for the transmitted or received data.

Enumeration Type Documentation

enum FCUART_ErrorStatusType

UART error status type.

Enumerator:

FCUART_ERROR_NONE	FCUART_ERROR_NONE No Error,
-------------------	-----------------------------

FCUART_ERROR _RORF	FCUART_ERROR_RORF Receiver Overrun Flag,
FCUART_ERROR _NF	FCUART_ERROR_NF Noise Flag,
FCUART_ERROR _FEF	FCUART_ERROR_FEF Frame Error Flag,
FCUART_ERROR _PEF	FCUART_ERROR_PEF Parity Error Flag,
FCUART_ERROR _RPEF	FCUART_ERROR_RPEF Receive Data Parity Error Flag,
FCUART_ERROR _TPEF	FCUART_ERROR_TPEF Transmit Data Parity Error Flag,

enum FCUART_ErrorTypeEnum

FCUART error status.

Enumerator:

FCUART_ERROR _OK	FCUART_ERROR_OK means no error
FCUART_ERROR _INVALID_VERSION	FCUART_ERROR_INVALID_VERSION means version is not same

FCUART_ERROR_FAILED	FCUART_ERROR_FAILED means operation is failed
FCUART_ERROR_INVALID_PARAM	FCUART_ERROR_INVALID_PARAM means parameters are invalid
FCUART_ERROR_INVALID_SIZE	FCUART_ERROR_INVALID_SIZE means size is invalid
FCUART_ERROR_INVALID_SEQUENCE	FCUART_ERROR_INVALID_SEQUENCE means sequence is error
FCUART_ERROR_TIMEOUT	FCUART_ERROR_TIMEOUT means operation is timeout

enum FCUART_Fifo_RxIdleCharNumType

UART fifo receive idle character number.

Enumerator:

FCUART_FIFO_RX_IDLE_DISABLE	The UART receive FIFO is idle and character reception is disabled.
FCUART_FIFO_RX_IDLE_CHARACTER_1	The UART receive FIFO is idle with a 1 character reception setting.
FCUART_FIFO_RX_IDLE_CHARACTER_2	The UART receive FIFO is idle with a 2 character reception setting.

FCUART_FIFO_RX_IDLE_CHARACTER_4	The UART receive FIFO is idle with a 4 character reception setting.
FCUART_FIFO_RX_IDLE_CHARACTER_8	The UART receive FIFO is idle with a 8 character reception setting.
FCUART_FIFO_RX_IDLE_CHARACTER_16	The UART receive FIFO is idle with a 16 character reception setting.
FCUART_FIFO_RX_IDLE_CHARACTER_32	The UART receive FIFO is idle with a 32 character reception setting.
FCUART_FIFO_RX_IDLE_CHARACTER_64	The UART receive FIFO is idle with a 64 character reception setting.

enum FCUART_IdleCharNumType

UART idle character number.

Enumerator:

FCUART_IDLE_CHARACTER_1	
FCUART_IDLE_CHARACTER_2	
FCUART_IDLE_CHARACTER_4	
FCUART_IDLE_CHARACTER_8	
FCUART_IDLE_CHARACTER_16	
FCUART_IDLE_CHARACTER_32	
FCUART_IDLE_CHARACTER_64	

FCUART_IDLE_C HARCTER_128	
------------------------------	--

enum FCUART_IdleStartType

UART idle type.

Enumerator:

FCUART_START _AFTER_START BIT	
FCUART_START _AFTER_STOPBI T	

enum FCUART_InterruptSel

UART CTRL register interrupt.

Enumerator:

FCUART_INT_CT RL_ORIE	FCUART_INT_CTRL_ORIE means Overrun Interrupt Enable
FCUART_INT_CT RL_NEIE	FCUART_INT_CTRL_NEIE means Noise Error Interrupt Enable
FCUART_INT_CT RL_FEIE	FCUART_INT_CTRL_FEIE means Framing Error Interrupt Enable
FCUART_INT_CT RL_PEIE	FCUART_INT_CTRL_PEIE means Parity Error Interrupt Enable
FCUART_INT_CT RL_TIE	FCUART_INT_CTRL_TIE means Transmit Interrupt Enable
FCUART_INT_CT RL_TCIE	FCUART_INT_CTRL_TCIE means Transmit Complete Interrupt Enable

FCUART_INT_CTL RL_RIE	FCUART_INT_CTRL_RIE means Receive Interrupt Enable
FCUART_INT_CTL RL_IIE	FCUART_INT_CTRL_IIE means Idle Line Interrupt Enable
FCUART_INT_CTL RL_TE	FCUART_INT_CTRL_TE means Transmit Enable
FCUART_INT_CTL RL_RE	FCUART_INT_CTRL_RE means Receive Enable
FCUART_INT_CTL RL_M0IE	FCUART_INT_CTRL_M0IE means Multi-Processor Mode Enable
FCUART_INT_CTL RL_M1IE	FCUART_INT_CTRL_M1IE means Multi-Processor Mode Enable

enum FCUART_SequenceType

FCUART Operation Sequence.

Enumerator:

FCUART_SEQUENCE_VAR_NOINIT	FCUART_SEQUENCE_DEINIT means fcuart variables data is not initialed
FCUART_SEQUENCE_DEINIT	FCUART_SEQUENCE_DEINIT means fcuart driver is not initialed
FCUART_SEQUENCE_NOTSTART_RECEIVE	FCUART_SEQUENCE_NOTSTART_RECEIVE means fcuart driver initialed and write/erase is allowed

FCUART_SEQUENCE_START_RECEIVE	FCUART_SEQUENCE_START_RECEIVE means fcuart driver started received
-------------------------------	--

enum UART_PrintIntType

Define the type of integer representation for UART printing.

Enumerator:

UART_PRINT_RADIX_BIN	
UART_PRINT_RADIX_OCT	
UART_PRINT_RADIX_DEC	
UART_PRINT_RADIX_HEX	

Function Documentation

void FCUART_AssignTxInterruptData (FCUART_HandleType * *pUartHandle*, uint8_t * *pData*, uint32_t *u32Length*)

This Function is used to assign data to send through interrupt.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pData</i>	data pointer
<i>u32Length</i>	data length to send

void FCUART_DeInit (FCUART_HandleType * *pUartHandle*)

This Function is used to de-initial UART instance.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
--------------------	---

void FCUART_Error_Polling (FCUART_HandleType * *pUartHandle*, uint32_t * *pErrorValue*)

This Function is used to get error when polling (not used when error interrupt enabled)

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pErrorValue</i>	is error value

void FCUART_GetData_10Bits (FCUART_HandleType * *pUartHandle*, uint16_t * *pData*)

This Function is used to Get 10 bits data.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pData</i>	pointer to data

void FCUART_GetData_9Bits (FCUART_HandleType * *pUartHandle*, uint16_t * *pData*)

This Function is used to Get 9 bits data.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pData</i>	pointer to data

bool FCUART_GetInterruptMode (FCUART_HandleType * *pUartHandle*, uint32_t *u32Data*)

This Function is used to Get current interrupt mode.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>u32Data</i>	Interrupt type to get

Returns

true/false

uint32_t FCUART_GetStatus (FCUART_HandleType * *pUartHandle*, FCUART_StatType *eStatusType*)

Get Stat Flag.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>eStatusType</i>	stat type

Returns

FCUART_STAT status flag

FCUART_ErrorType FCUART_Init (FCUART_HandleType * *pUartHandle*, FCUART_InitType * *pInitCfg*)

This Function is used to initial UART instance.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pInitCfg</i>	contains clock, baud-rate, Bit Mode, parity and so on.

Returns

FCUART_ERROR_OK is ok, others are not ok

void FCUART_InitStructure (FCUART_HandleType * *pUartHandle*)

Initial UART variables Memory.

FCUART_ErrorType FCUART_Printf (FCUART_HandleType * *pUartHandle*, char * *fmt*, ...)

This Function is used to print ASCII char from UART.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>fmt</i>	is char format

Returns

FCUART_ERROR_OK is ok, others are not ok

FCUART_ErrorType FCUART_Receive_Polling (FCUART_HandleType * *pUartHandle*, FCUART_DataType * *pRxMsg*)

This Function is used to receive data when polling (not used when rx interrupt enabled)

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pRxMsg</i>	is data buffer address, and pDatas need to be initialed with external buffer

Returns

FCUART_ERROR_OK is ok, others are not ok

void FCUART_SendData_10Bits (FCUART_HandleType * *pUartHandle*, uint16_t *u16Data*)

This Function is used to send 10 bits data.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>u16Data</i>	data to send

void FCUART_SendData_9Bits (FCUART_HandleType * *pUartHandle*, uint16_t *u16Data*)

This Function is used to send 9 bits data.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>u16Data</i>	data to send

void FCUART_SetBitModeAndParity (FCUART_HandleType * *pUartHandle*, FCUART_BitModeType *eBitMode*, FCUART_StopBitNumType *eStopBit*, FCUART_ParityType *eParityType*, bool *bParityEnable*)

This Function is used to set bit mode,parity and stop bit.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>eBitMode</i>	bit mode
<i>eStopBit</i>	stop bit number
<i>eParityType</i>	parity type
<i>bParityEnable</i>	if enable parity

void FCUART_SetLoopMode (FCUART_HandleType * *pUartHandle*, bool *bStatus*)

This Function is used to enable fcuart loop mode.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>bStatus</i>	enable/disable status of loop mode

void FCUART_SetWakeup (FCUART_HandleType * *pUartHandle*, FCUART_WakeupType * *pWakeupCfg*)

This Function is used to set UART WakeUp.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pWakeupCfg</i>	contains UART wake-up parameters

void FCUART_StartReceive (FCUART_HandleType * *pUartHandle*)

This Function is used to start receiving.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
--------------------	---

void FCUART_StartTransmit (FCUART_HandleType * *pUartHandle*)

This Function is used to start transmit.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
--------------------	---

void FCUART_StopReceive (FCUART_HandleType * *pUartHandle*)

This Function is used to stop receiving.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
--------------------	---

void FCUART_StopTransmit (FCUART_HandleType * *pUartHandle*)

This Function is used to stop transmitting.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
--------------------	---

FCUART_ErrorType FCUART_Transmit (FCUART_HandleType * *pUartHandle*, FCUART_DataType * *pUartData*)

This Function is used to transmit UART data.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
<i>pUartData</i>	contains UART data and length

Returns

FCUART_ERROR_OK is ok, others are not ok

void FCUARTN_RxTx_IRQHandler (FCUART_HandleType * *pUartHandle*)

This Function is used to deal with FCUART TxRx interrupt.

Parameters

<i>pUartHandle</i>	The structure of the UART processing handle
--------------------	---

Module_driver_flash

Classes

struct **_FLASH_HandleType**

The structure of the FLASH processing handle.

struct **FLASH_DRIVER_ParamType**

Flash driver parameter define.

struct **FLASH_Lock_ParamType**

unlock parameter define

Typedefs

typedef enum **fls_Ret Fls_Status_t**

typedef enum **Async_Return Async_Return_t**

FLASH async prog/erase status.

typedef enum **Interrupt_Return Fls_ReturnType**

FLASH interrupt prog/erase status.

typedef struct **_FLASH_HandleType FLASH_HandleType**

The structure of the FLASH processing handle.

Enumerations

enum **FLASH_InstanceType** { **FLASH_INSTANCE_0** = 0U }

FLASH instance.

enum **fls_Ret** { **FLS_IDLE** = 0, **FLS_ERASING** = 1, **FLS_WRITING** = 2, **FLS_READING** = 3, **FLS_ERASE_ERROR** = 4, **FLS_WRITE_ERROR** = 5, **FLS_PARA_ERROR** = 6, **FLS_DMA_WRITING** = 7 }

```
enum FLASH_StatusType { FLASH_ERROR_OK = 0x00U, FLASH_ERROR_NO_INIT,  
FLASH_ERROR_INVALID_VERSION, FLASH_ERROR_FAILED, FLASH_ERROR_INVALID_PARAM,  
FLASH_ERROR_INVALID_ADDR, FLASH_ERROR_INVALID_SIZE, FLASH_ERROR_INVALID_SEQUENCE,  
FLASH_ERROR_TIMEOUT, FLASH_ERROR_END = 0xFFFFFFFFU }
```

FLASH API error status.

```
enum ECC_StatusType { FLASH_ECC_ERROR_NONE, FLASH_ECC_ERROR_SBC,  
FLASH_ECC_ERROR_DBC }
```

FLASH ECC error status.

```
enum Async_Return { FLASH_ASYNC_ERR_OK = 0, FLASH_ASYNC_ERR_NOT_OK = 1,  
FLASH_ASYNC_ERR_BUSY = 2 }
```

FLASH async prog/erase status.

```
enum Interrupt_Return { FLASH_INTERRUPT_ERR_OK = 0, FLASH_INTERRUPT_ERR_NOT_OK = 1,  
FLASH_INTERRUPT_ERR_BUSY = 2 }
```

FLASH interrupt prog/erase status.

```
enum FLASH_LOCK_TYPE { FLASH_UNLOCK = 0x000u, FLASH_LOCK = 0x001u }
```

FMC lock type.

```
enum FLASH_TYPE { PFlash = 0x000u, DFlash = 0x001u }
```

Flash type.

Functions

```
void FLASHDRIVER_Init (void)
```

Initial Flash api address.

```
FLASH_StatusType FLASHDRIVER_Lock (FLASH_Lock_ParamType *pUnlockParam)
```

Flash Driver Function for lock/unlock sector.

```
FLASH_StatusType FLASHDRIVER_DMAWrite (FLASH_HandleType *pFlashHandle, uint32_t  
u32DMACount)
```

Flash Driver Function for DMA Write.

```
uint32_t FLASHDRIVER_DMAAbort (FLASH_HandleType *pFlashHandle)
```

Flash Driver Function for DMA Abort.

```
FLASH_StatusType FLASHDRIVER_SyncErase (FLASH_HandleType *pFlashHandle,  
FLASH_DRIVER_ParamType *pFlashParam)
```

Flash Driver Function for Erasing.

**FLASH_StatusType FLASHDRIVER_SyncWrite (FLASH_HandleType *pFlashHandle,
FLASH_DRIVER_ParamType *pFlashParam)**

Flash Driver Function for Writing.

**void FLASHDRIVER_AsyncErase (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType
*pFlashParam)**

flash driver erase a sector

**void FLASHDRIVER_AsyncWrite (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType
*pFlashParam)**

flash driver write a fixed size

Async_Return_t FLASHDRIVER_AsyncCheckEraseStatus (FLASH_HandleType *pFlashHandle)

flash driver check current status when erasing

Async_Return_t FLASHDRIVER_AsyncCheckWriteStatus (FLASH_HandleType *pFlashHandle)

flash driver check current status when writing

**FLASH_StatusType NVRDRIVER_SyncWrite (FLASH_HandleType *pFlashHandle,
FLASH_DRIVER_ParamType *pFlashParam)**

NVR driver write function warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked.

.=====. */_____/ */.-" "-./*//\| */_//_/*
/(\|,.-.-,./)/ */> "= _)(_/ _)(/ _." <| */(_/"= _"= _//\| _."= _")/ */ "= _"(_ ^^ _)"= _"/ *
/ "= _|/|/|/|/|/_/=" | */ _."= " |/|/|/|/|/"= _ / * / _ _."= _" \ /"= _"= _ _ / * / (_."= _"= " ----- "= _"= _/) / * / > _."= "
"= _ <| */(_/_)/ */| *'-----' *.

**FLASH_StatusType NVRDRIVER_SyncErase (FLASH_HandleType *pFlashHandle,
FLASH_DRIVER_ParamType *pFlashParam)**

NVR driver erase function warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked.

```

become bricked. .=====, */_____/ */.-"      "./*//\|/*/_//_/*
/(\|,.-.-,|/)/ */> "= _    /)(_/  _)(/    _."<| */(/"= _"= _ //|/_."=_."= _)/ */ "= _"( ^ ^ _)"= _"/ *
/ "= _|/|/|/|/|/_="/ */ _."= _| |/|/|/|/"= _ / */ _."= _"/|/"= _"= _ / */ ( _."= _."= _ -----  "= _"= _/)/ */> _."= _
"= _ <| */(/_)/ */| *'===== '*,

```

```

void FLASHDRIVER_InterruptErase (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType
*pFlashParam)
flash driver interrupt erase

```

```

void FLASHDRIVER_InterruptProgram (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType
*pFlashParam)
flash driver interrupt write

```

```

void FLASHDRIVER_Close_Test_Mode (FLASH_HandleType *pFlashHandle)
Close teset mode.

```

```

void FLASHDRIVER_EnInterrupt (uint32_t u32CoreID)
Flash Driver Function for enable interrupt.

```

```

Fls_ReturnType Fls_CommonProcessInterrupt (FLASH_HandleType *pFlashHandle)
Process Flash interrupt.

```

FLASH API Service IDs

```

#define FLASH_SYNC_ERASE_ID  0U
#define FLASH_SYNC_WRITE_ID  1U
#define FLASH_ASYNC_ERASE_ID 2U
#define FLASH_ASYNC_WRITE_ID 3U
#define FLASH_INTERRUPT_ERASE_ID 4U
#define FLASH_INTERRUPT_WRITE_ID 5U
#define NVR_SYNC_ERASE_ID  6U
#define NVR_SYNC_WRITE_ID  7U

```

FLASH Dev Error Code

```

#define FLASH_E_PARAM_POINTER 0x01U

```

Detailed Description

Macro Definition Documentation

#define FLASH_ASYNC_ERASE_ID 2U

#define FLASH_ASYNC_WRITE_ID 3U

#define FLASH_E_PARAM_POINTER 0x01U

#define FLASH_INTERRUPT_ERASE_ID 4U

#define FLASH_INTERRUPT_WRITE_ID 5U

#define FLASH_SYNC_ERASE_ID 0U

#define FLASH_SYNC_WRITE_ID 1U

#define NVR_SYNC_ERASE_ID 6U

#define NVR_SYNC_WRITE_ID 7U

Typedef Documentation

typedef enum Async_Return Async_Return_t

FLASH async prog/erase status.

Some values from a called function.

typedef struct _FLASH_HandleType FLASH_HandleType

The structure of the FLASH processing handle.

typedef enum Interrupt_Return Fls_ReturnType

FLASH interrupt prog/erase status.

typedef enum fls_Ret Fls_Status_t

Enumeration Type Documentation

enum Async_Return

FLASH async prog/erase status.

Enumerator:

FLASH_ASYNC_ ERR_OK	Indicate that a function was successfully processed.
FLASH_ASYNC_ ERR_NOT_OK	Indicate that a function was NOT successfully processed.
FLASH_ASYNC_ ERR_BUSY	Indicate that a function was busy .

enum ECC_StatusType

FLASH ECC error status.

Enumerator:

FLASH_ECC_ER ROR_NONE	
FLASH_ECC_ER ROR_SBC	

FLASH_ECC_ER ROR_DBC	
-------------------------	--

enum FLASH_InstanceType

FLASH instance.

Enumerator:

FLASH_INSTAN CE_0	
----------------------	--

enum FLASH_LOCK_TYPE

FMC lock type.

Enumerator:

FLASH_UNLOCK	
FLASH_LOCK	

enum FLASH_StatusType

FLASH API error status.

Enumerator:

FLASH_ERROR_ OK	FLASH_ERROR_OK means no error
FLASH_ERROR_ NO_INIT	FLASH_ERROR_NO_INIT means rom code api address is not initialed

FLASH_ERROR_INVALID_VERSION	FLASH_ERROR_INVALID_VERSION means rom code api version is not same
FLASH_ERROR_FAILED	FLASH_ERROR_FAILED means operation is failed
FLASH_ERROR_INVALID_PARAM	FLASH_ERROR_INVALID_PARAM means parameters are invalid
FLASH_ERROR_INVALID_ADDR	FLASH_ERROR_INVALID_ADDR means address is invalid
FLASH_ERROR_INVALID_SIZE	FLASH_ERROR_INVALID_SIZE means size is invalid
FLASH_ERROR_INVALID_SEQUENCE	FLASH_ERROR_INVALID_SEQUENCE means sequence is error
FLASH_ERROR_TIMEOUT	FLASH_ERROR_TIMEOUT means operation is timeout
FLASH_ERROR_END	FLASH_ERROR_END make aligned 32bits

enum FLASH_TYPE

Flash type.

Enumerator:

PFlash	
DFlash	

enum fls_Ret

Enumerator:

FLS_IDLE	Indicate that a function was successfully processed.
----------	--

FLS_ERASING	Indicate that a function was NOT successfully processed.
FLS_WRITING	
FLS_READING	
FLS_ERASE_ERR OR	
FLS_WRITE_ERR OR	
FLS_PARA_ERR OR	
FLS_DMA_WRITI NG	

enum Interrupt_Return

FLASH interrupt prog/erase status.

Enumerator:

FLASH_INTERR UPT_ERR_OK	Indicate that a function was successfully processed.
FLASH_INTERR UPT_ERR_NOT_ OK	Indicate that a function was NOT successfully processed.
FLASH_INTERR UPT_ERR_BUSY	Indicate that a function was busy

Function Documentation

Async_Return_t FLASHDRIVER_AsyncCheckEraseStatus (FLASH_HandleType * *pFlashHandle*)

flash driver check current status when erasing

Parameters

<i>pFlashHandle</i>	flash handle
---------------------	--------------

Returns

Fls_Return_t return the erase status

Async_Return_t FLASHDRIVER_AsyncCheckWriteStatus (FLASH_HandleType * *pFlashHandle*)

flash driver check current status when writing

Parameters

<i>pFlashHandle</i>	flash handle
---------------------	--------------

Returns

Fls_Return_t return the write status

void FLASHDRIVER_AsyncErase (FLASH_HandleType * *pFlashHandle*, FLASH_DRIVER_ParamType * *pFlashParam*)

flash driver erase a sector

Parameters

<i>pFlashHandle</i>	flash handle
<i>pFlashParam</i>	flash driver write parameter

void FLASHDRIVER_AsyncWrite (FLASH_HandleType * *pFlashHandle*, FLASH_DRIVER_ParamType * *pFlashParam*)

flash driver write a fixed size

Parameters

<i>pFlashHandle</i>	flash handle
<i>pFlashParam</i>	flash driver write parameter

void FLASHDRIVER_Close_Test_Mode (FLASH_HandleType * *pFlashHandle*)

Close teset mode.

Parameters

<i>pFlashHandle</i>	the Flash instance to use
---------------------	---------------------------

Returns

none

uint32_t FLASHDRIVER_DMAAbort (FLASH_HandleType * *pFlashHandle*)

Flash Driver Function for DMA Abort.

Parameters

<i>u32DMACount</i>	flash DMA count
--------------------	-----------------

Returns

complete count

FLASH_StatusType FLASHDRIVER_DMAWrite (FLASH_HandleType * *pFlashHandle*, uint32_t *u32DMACount*)

Flash Driver Function for DMA Write.

Parameters

<i>pFlashHandle</i>	flash handle
<i>u32DMACount</i>	u32DMACount flash DMA count

Returns

ErrorType

void FLASHDRIVER_EnInterrupt (uint32_t *u32CoreID*)

Flash Driver Function for enable interrupt.

Parameters

<i>u32CoreID</i>	core id
------------------	---------

void FLASHDRIVER_Init (void)

Initial Flash api address.

void FLASHDRIVER_InterruptErase (FLASH_HandleType * *pFlashHandle*, FLASH_DRIVER_ParamType * *pFlashParam*)

flash driver interrupt erase

Parameters

<i>pFlashHandle</i>	flash handle
<i>pFlashParam</i>	flash driver write parameter

void FLASHDRIVER_InterruptProgram (FLASH_HandleType * *pFlashHandle*, FLASH_DRIVER_ParamType * *pFlashParam*)

flash driver interrupt write

Parameters

in	<i>pFlashHandle</i>	Flash handle
	<i>pFlashParam</i>	flash driver write parameter

FLASH_StatusType FLASHDRIVER_Lock (FLASH_Lock_ParamType * *pUnlockParam*)

Flash Driver Function for lock/unlock sector.

Parameters

<i>pUnlockParam</i>	flash driver lock/unlock parameter
---------------------	------------------------------------

Returns

ErrorType

**FLASH_StatusType FLASHDRIVER_SyncErase (FLASH_HandleType * *pFlashHandle*,
FLASH_DRIVER_ParamType * *pFlashParam*)**

Flash Driver Function for Erasing.

Parameters

<i>pFlashHandle</i>	flash handle
<i>pFlashParam</i>	flash driver erase parameter

Returns

ErrorType

**FLASH_StatusType FLASHDRIVER_SyncWrite (FLASH_HandleType * *pFlashHandle*,
FLASH_DRIVER_ParamType * *pFlashParam*)**

Flash Driver Function for Writing.

Parameters

<i>pFlashHandle</i>	flash handle
<i>pFlashParam</i>	flash driver write parameter

Returns

ErrorType

Fls_ReturnType Fls_CommonProcessInterrupt (FLASH_HandleType * *pFlashHandle*)

Process Flash interrupt.

Parameters

<i>pFlashHandle</i>	flash handle point
---------------------	--------------------

Returns

process result

**FLASH_StatusType NVRDRIVER_SyncErase (FLASH_HandleType * *pFlashHandle*,
FLASH_DRIVER_ParamType * *pFlashParam*)**

NVR driver erase function warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. .=-=-=-=-=-. * | _____ | * | .-" "-. | * | / \ | * | _ | | _ | * | (\ | , .- .- . , /) | * | > "= _ |) (_ / \ _) (| _ ." < | * | (_ / "= _ ." _ / \ | _ ." _ ." _) | * | "= _ ." (_ ^ ^ _) " _ ." | * | "= \ _ | | | | | | _ / =" | * | _ ." | \ | | | | | / | "= _ | * | _ _ ." _ ." \ / "= _ ." _ _ | * | (_ ." _ ." ----- "= _ ." _ /) | * | > _ ." "= _ < | * | (_ / _) | * | | * '=-=-=-=-=-' *.

Parameters

<i>pFlashHandle</i>	the Flash instance to use
<i>pFlashParam</i>	contains flash erase function parameter, address is align to sector, and length is align to sector

Returns

ErrorType

**FLASH_StatusType NVRDRIVER_SyncWrite (FLASH_HandleType * *pFlashHandle*,
FLASH_DRIVER_ParamType * *pFlashParam*)**

NVR driver write function warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. .=-=-=-=-=-. * | _____ | * | .-" "-. | * | / \ | * | _ | | _ | * | (\ | , .- .- . , /) | * | > "= _ |) (_ / \ _) (| _ ." < | * | (_ / "= _ ." _ / \ | _ ." _ ." _) | * | "= _ ." (_ ^ ^ _) " _ ." | * | "= \ _ | | | | | _ / =" | * | _ ." | \ | | | | | / | "= _ | * | _ _ ." _ ." \ / "= _ ." _ _ | * | (_ ." _ ." ----- "= _ ." _ /) | * | > _ ." "= _ < | * | (_ / _) | * | | * '=-=-=-=-=-' *.

Parameters

<i>pFlashHandle</i>	the Flash instance to use
<i>pFlashParam</i>	contains flash write function parameter, address is align to page, and length is align to page

Returns

ErrorType

Module_driver_flexcan

Collaboration diagram for Module_driver_flexcan:Modules

HwA_flexcan

Classes

struct **FLEXCAN_BaudRegType**

Baud-rate clock and divider.

struct **FLEXCAN_ErrorInfoType**

FLEXCAN Error Information.

struct **FLEXCAN_RxMsgType**

FLEXCAN receive message.

struct **FLEXCAN_FifoFilterType**

Receive Message Buffer Configuration.

struct **FLEXCAN_MbSettingType**

Message Buffer Configuration.

struct **FLEXCAN_LegacyFifoConfigType**

FLEXCAN legacy fifo configuration structure definition.

struct **FLEXCAN_EnhancedFifoConfigType**

FLEXCAN enhanced fifo configuration structure definition.

struct **FLEXCAN_FifoConfigType**

FLEXCAN fifo configuration structure definition.

struct **FLEXCAN_MbConfigType**

FLEXCAN mb configuration structure definition.

struct **FLEXCAN_TxMsgType**

FLEXCAN transmit message.

struct **FLEXCAN_RxMsgListType**

FLEXCAN polling receive message.

struct **FLEXCAN_FifoIssueType**

FIFO error response type.

struct **FLEXCAN_PNET_IdSettingType**

CAN ID filter setting.

struct **FLEXCAN_PNET_IDConfigType**

CAN Pretended network id filter setting.

struct **FLEXCAN_PNET_PayloadConfigType**

struct **FLEXCAN_PNET_ConfigType**

CAN Pretended network configuration.

struct **FLEXCAN_WakeupMsgType**

FLEXCAN wake-up receive message.

struct **_FLEXCAN_HandleType**

The structure of the CAN processing handle.

struct **FLEXCAN_InitType**

FLEXCAN init structure definition.

Typedefs

typedef uint32_t **FLEXCAN_ErrorType**

FLEXCAN error status.

CAN Detection Error Code

All can det code

#define **FLEXCAN_DET_ERROR_ID_FUNC_CHECKINSTANCE** 0x01U

#define **FLEXCAN_DET_ERROR_ID_FUNC_INIT** 0x01U

#define **FLEXCAN_DET_ERROR_ID_FUNC_PROCESS** 0x02U

#define **FLEXCAN_DET_ERROR_ID_FUNC_SETINTERRUPT** 0x03U

#define **FLEXCAN_DET_ERROR_ID_FUNC_START** 0x04U

#define **FLEXCAN_DET_ERROR_ID_FUNC_STOP** 0x05U

#define **FLEXCAN_DET_ERROR_ID_FUNC_TRANSMIT** 0x06U

#define **FLEXCAN_DET_ERROR_ID_FUNC_RECEIVE** 0x07U

CAN Return Error Code

All can error code

#define **FLEXCAN_E_OK** 0x00U

#define **FLEXCAN_E_INVALID_VERSION** 0x01U

#define **FLEXCAN_E_FAILED** 0x02U

#define **FLEXCAN_E_INVALID_PARAM** 0x04U

#define **FLEXCAN_E_INVALID_SIZE** 0x08U

#define **FLEXCAN_E_INVALID_SEQUENCE** 0x10U

#define **FLEXCAN_E_NULL_POINT** 0x20U

```
#define FLEXCAN_E_TIMEOUT 0x40U
#define FLEXCAN_E_BUSY 0x80U
#define FLEXCAN_E_MB_OUTOFRANGE 0x100U
#define FLEXCAN_E_UNUPPORT_PARAM 0x200U
```

CAN DET Function

CAN DET Function

```
#define FLEXCAN_DEV_ERROR_REPORT STD_OFF
#define FLEXCAN_ReportDevError(func, error) ReportDevError(FLEXCAN_MODULE_ID, func, error)
#define FLEXCAN_TX_FLAG_CLEAR_FIRST STD_OFF
#define FLEXCAN_RX_FLAG_CLEAR_FIRST STD_OFF
#define FLEXCAN_ERROR_FLAG_CLEAR_FIRST STD_ON
```

CAN enumeration definition

CAN enumeration definition

```
enum FLEXCAN_IdType { FLEXCAN_ID_STD = 0x00U, FLEXCAN_ID_EXT = 0x01U }
FLEXCAN ID Type.

enum FLEXCAN_DataType { FLEXCAN_FRAME_DATA = 0x00U, FLEXCAN_FRAME_REMOTE = 0x01U }
FLEXCAN Data Frame Type.

enum FLEXCAN_SequenceType { FLEXCAN_SEQUENCE_DEINIT, FLEXCAN_SEQUENCE_NOTSTART,
FLEXCAN_SEQUENCE_STARTED }
CAN Operation Sequence.

enum FLEXCAN_PNET_WakeupSrcType { FLEXCAN_PENT_WAKEUP_SRC_TIMEOUT,
FLEXCAN_PENT_WAKEUP_SRC_MATCH }
CAN wake-up source.

enum FLEXCAN_FifoType { FLEXCAN_LEGACY_FIFO, FLEXCAN_ENHANCED_FIFO }
CAN wake-up source.
```

CAN structure definition

CAN structure definition

```
typedef struct _FLEXCAN_HandleType FLEXCAN_HandleType
The structure of the CAN processing handle.
```

CAN api definition

CAN api definition

uint8_t **FLEXCAN_GetDataWidthArrayLen** (FLEXCAN_HandleType *pCanHandle)

Get CAN Data width number for current can MBDSR0..MBDSRn.

void **FLEXCAN_EccInjectionInit** (FLEXCAN_HandleType *pCanHandle, uint32_t erro_inject_addr, uint8_t data_pattern_bit)

Initializes Error Correction Code (ECC) injection for FlexCAN module.

void **FLEXCAN_EccInjectionDeInit** (FLEXCAN_HandleType *pCanHandle)

Deinitializes Error Correction Code (ECC) injection for FlexCAN module.

FLEXCAN_ErrorType **FLEXCAN_Init** (FLEXCAN_HandleType *pCanHandle, const **FLEXCAN_InitType** *const pInitCfg)

Initial CAN.

FLEXCAN_ErrorType **FLEXCAN_DeInit** (FLEXCAN_HandleType *pCanHandle)

De-initial can instance.

FLEXCAN_ErrorType **FLEXCAN_Start** (FLEXCAN_HandleType *pCanHandle)

Start can instance.

FLEXCAN_ErrorType **FLEXCAN_Stop** (FLEXCAN_HandleType *pCanHandle)

Stop can instance.

FLEXCAN_ErrorType **FLEXCAN_TransmitData** (FLEXCAN_HandleType *pCanHandle, const **FLEXCAN_TxMsgType** *const pTxMsg)

Transmit data, if tx disable, must call FLEXCAN_TransmitProcess after transmitting.

FLEXCAN_ErrorType **FLEXCAN_TransmitAbort** (FLEXCAN_HandleType *pCanHandle, uint8_t u8TxHandle)

Abort transmit with special transmit handler.

uint32_t **FLEXCAN_DataLenToDlc** (uint32_t u32DataLen)

Transfer Data length to DLC.

uint32_t **FLEXCAN_DlcToDataLen** (uint32_t u32Dlc)

Transfer DLC to Data length.

void **FLEXCAN_InterruptProcess** (FLEXCAN_HandleType *pCanHandle)

Can process for interrupt.

void **FLEXCAN_PollingProcess** (FLEXCAN_HandleType *pCanHandle)

Can process for polling.

void **FLEXCAN_InterruptErrorProcess** (FLEXCAN_HandleType *pCanHandle)

Can Error process for interrupt.

void **FLEXCAN_PollingErrorProcess** (FLEXCAN_HandleType *pCanHandle)

Can Error process for polling.

Detailed Description

Macro Definition Documentation

#define FLEXCAN_DET_ERROR_ID_FUNC_CHECKINSTANCE 0x01U

FLEXCAN_DET_ERROR_ID_FUNC_CHECKINSTANCE means error in check instance error function

#define FLEXCAN_DET_ERROR_ID_FUNC_INIT 0x01U

FLEXCAN_DET_ERROR_ID_FUNC_INIT means error in initial function

#define FLEXCAN_DET_ERROR_ID_FUNC_PROCESS 0x02U

FLEXCAN_DET_ERROR_ID_FUNC_PROCESS means error in process

#define FLEXCAN_DET_ERROR_ID_FUNC_RECEIVE 0x07U

FLEXCAN_DET_ERROR_ID_FUNC_RECEIVE means error in can receive

#define FLEXCAN_DET_ERROR_ID_FUNC_SETINTERRUPT 0x03U

FLEXCAN_DET_ERROR_ID_FUNC_SETINTERRUPT means error in set interrupt

#define FLEXCAN_DET_ERROR_ID_FUNC_START 0x04U

FLEXCAN_DET_ERROR_ID_FUNC_START means error in can start

#define FLEXCAN_DET_ERROR_ID_FUNC_STOP 0x05U

FLEXCAN_DET_ERROR_ID_FUNC_STOP means error in can stop

#define FLEXCAN_DET_ERROR_ID_FUNC_TRANSMIT 0x06U

FLEXCAN_DET_ERROR_ID_FUNC_TRANSMIT means error in can transmit

#define FLEXCAN_DEV_ERROR_REPORT STD_OFF

#define FLEXCAN_E_BUSY 0x80U

FLEXCAN_E_BUSY means current mb is busy

#define FLEXCAN_E_FAILED 0x02U

FLEXCAN_E_FAILED means operation is failed

#define FLEXCAN_E_INVALID_PARAM 0x04U

FLEXCAN_E_INVALID_PARAM means parameters are invalid

#define FLEXCAN_E_INVALID_SEQUENCE 0x10U

FLEXCAN_E_INVALID_SEQUENCE means sequence is error

#define FLEXCAN_E_INVALID_SIZE 0x08U

FLEXCAN_E_INVALID_SIZE means size is invalid

#define FLEXCAN_E_INVALID_VERSION 0x01U

FLEXCAN_E_INVALID_VERSION means version is not same

#define FLEXCAN_E_MB_OUTOFRANGE 0x100U

FLEXCAN_E_MB_OUTOFRANGE mb number is out of range

#define FLEXCAN_E_NULL_POINT 0x20U

FLEXCAN_E_INVALID_SEQUENCE means sequence is error

#define FLEXCAN_E_OK 0x00U

FLEXCAN_E_OK means no error

#define FLEXCAN_E_TIMEOUT 0x40U

FLEXCAN_E_NULL_POINT means buffer point is null

#define FLEXCAN_E_UNSUPPORT_PARAM 0x200U

FLEXCAN_E_UNSUPPORT_PARAM give unsupported parameter

#define FLEXCAN_ERROR_FLAG_CLEAR_FIRST STD_ON

#define FLEXCAN_ReportDevError(func, error) ReportDevError(FLEXCAN_MODULE_ID, func, error)

#define FLEXCAN_RX_FLAG_CLEAR_FIRST STD_OFF

#define FLEXCAN_TX_FLAG_CLEAR_FIRST STD_OFF

Typedef Documentation

typedef uint32_t FLEXCAN_ErrorType

FLEXCAN error status.

typedef struct _FLEXCAN_HandleType FLEXCAN_HandleType

The structure of the CAN processing handle.

Enumeration Type Documentation

enum FLEXCAN_DataType

FLEXCAN Data Frame Type.

Enumerator:

FLEXCAN_FRAME_DATA E_DATA	FLEXCAN_FRAME_DATA data frame
FLEXCAN_FRAME_REMOTE E_REMOTE	FLEXCAN_FRAME_REMOTE remote frame

enum FLEXCAN_FifoType

CAN wake-up source.

Enumerator:

FLEXCAN_LEGACY_FIFO	FLEXCAN_PENT_WAKEUP_SRC_TIMEOUT means wake-up source is timeout
---------------------	---

FLEXCAN_ENHANCED_FIFO	FLEXCAN_PENT_WAKEUP_SRC_MATCH means wake-up source is match
-----------------------	---

enum FLEXCAN_IdType

FLEXCAN ID Type.

Enumerator:

FLEXCAN_ID_STANDARD	FLEXCAN_ID_STD standard id
FLEXCAN_ID_EXTENDED	FLEXCAN_ID_EXT extended id

enum FLEXCAN_PNET_WakeupSrcType

CAN wake-up source.

Enumerator:

FLEXCAN_PENT_WAKEUP_SRC_TIMEOUT	FLEXCAN_PENT_WAKEUP_SRC_TIMEOUT means wake-up source is timeout
FLEXCAN_PENT_WAKEUP_SRC_MATCH	FLEXCAN_PENT_WAKEUP_SRC_MATCH means wake-up source is match

enum FLEXCAN_SequenceType

CAN Operation Sequence.

Enumerator:

FLEXCAN_SEQUENCE_DEINIT	FLEXCAN_SEQUENCE_DEINIT means CAN driver is not initialed
FLEXCAN_SEQUENCE_NOTSTART	FLEXCAN_SEQUENCE_NOTSTART means can driver initialed and not start
FLEXCAN_SEQUENCE_STARTED	FLEXCAN_SEQUENCE_STARTED means can driver started

Function Documentation

uint32_t FLEXCAN_DataLenToDlc (uint32_t *u32DataLen*)

Transfer Data length to DLC.

Parameters

<i>u32DataLen</i>	data length
-------------------	-------------

Returns

DLC

FLEXCAN_ErrorType FLEXCAN_DeInit (FLEXCAN_HandleType * *pCanHandle*)

De-initial can instance.

Parameters

<i>pCanHandle</i>	Can Handle point
-------------------	------------------

Returns

0 is ok, others are not ok

uint32_t FLEXCAN_DlcToDataLen (uint32_t *u32Dlc*)

Transfer DLC to Data length.

Parameters

<i>u32Dlc</i>	DLC Data Length
---------------	-----------------

Returns

void FLEXCAN_EccInjectionDeInit (FLEXCAN_HandleType * *pCanHandle*)

Deinitializes Error Correction Code (ECC) injection for FlexCAN module.

Disables ECC error injection by:

Clearing error injection interrupts (MECR[FAERRIE|HAERRIE])

Disabling ECC register write access (CTRL2[ECRWRE])

Parameters

<i>pCanHandle</i>	Pointer to FlexCAN instance handle
-------------------	------------------------------------

**void FLEXCAN_EccInjectionInit (FLEXCAN_HandleType * *pCanHandle*, uint32_t *erro_inject_addr*,
uint8_t *data_pattern_bit*)**

Initializes Error Correction Code (ECC) injection for FlexCAN module.

Configures the FlexCAN module for ECC error injection by:

Disabling module disable mode (MCR[MDIS])

Enabling ECC register write access (CTRL2[ECRWRE])

Configuring error injection address and data pattern

Enabling ECC error injection interrupts and mechanisms

Parameters

<i>pCanHandle</i>	Pointer to FlexCAN instance handle
<i>erro_inject_addr</i>	Error injection address (written to ERRIAR register)
<i>data_pattern_bit</i>	Data pattern for error injection (written to ERRIDPR register)

uint8_t FLEXCAN_GetDataWidthArrayLen (FLEXCAN_HandleType * *pCanHandle*)

Get CAN Data width number for current can MBDSR0..MBDSRn.

Parameters

<i>pCanHandle</i>	Can Handle point
-------------------	------------------

Returns

data width array number

FLEXCAN_ErrorType FLEXCAN_Init (FLEXCAN_HandleType * *pCanHandle*, const FLEXCAN_InitType *const *pInitCfg*)

Initial CAN.

Parameters

<i>pCanHandle</i>	Can Handle point
<i>pInitCfg</i>	clock, baud-rate, canfd, data length and so on.

Returns

0 is ok, others are not ok

void FLEXCAN_InterruptErrorProcess (FLEXCAN_HandleType * *pCanHandle*)

Can Error process for interrupt.

Parameters

<i>pCanHandle</i>	Can handle point
-------------------	------------------

void FLEXCAN_InterruptProcess (FLEXCAN_HandleType * *pCanHandle*)

Can process for interrupt.

Parameters

<i>pCanHandle</i>	Can Handle point
-------------------	------------------

void FLEXCAN_PollingErrorProcess (FLEXCAN_HandleType * *pCanHandle*)

Can Error process for polling.

Parameters

<i>pCanHandle</i>	Can handle point
-------------------	------------------

void FLEXCAN_PollingProcess (FLEXCAN_HandleType * *pCanHandle*)

Can process for polling.

Parameters

<i>pCanHandle</i>	Can Handle point
-------------------	------------------

FLEXCAN_ErrorType FLEXCAN_Start (FLEXCAN_HandleType * *pCanHandle*)

Start can instance.

Parameters

<i>pCanHandle</i>	Can Handle point
-------------------	------------------

Returns

0 is ok

FLEXCAN_ErrorType FLEXCAN_Stop (FLEXCAN_HandleType * *pCanHandle*)

Stop can instance.

Parameters

<i>pCanHandle</i>	Can Handle point
-------------------	------------------

FLEXCAN_ErrorType FLEXCAN_TransmitAbort (FLEXCAN_HandleType * *pCanHandle*, uint8_t *u8TxHandle*)

Abort transmit with special transmit handler.

Parameters

<i>pCanHandle</i>	Can Handle point
<i>u8TxHandle</i>	Transmit handler

Returns

0 is ok

FLEXCAN_ErrorType FLEXCAN_TransmitData (FLEXCAN_HandleType * *pCanHandle*, const FLEXCAN_TxMsgType *const *pTxMsg*)

Transmit data, if tx disable, must call FLEXCAN_TransmitProcess after transmitting.

Parameters

<i>pCanHandle</i>	Can Handle point
<i>pTxMsg</i>	contains CAN instance, CAN ID, CAN FD and CAN data.

Returns

0 is ok

Module_driver_freqm

Collaboration diagram for Module_driver_freqm:Modules

HwA_freqm

Classes

struct **_FREQM_HandleType**

The structure of the FREQM processing handle.

struct **FREQM_CfgType**

FREQM configure structure.

Typedefs

typedef struct **_FREQM_HandleType** **FREQM_HandleType**

The structure of the FREQM processing handle.

Enumerations

enum **FREQM_InstanceType** { **FREQM_INSTANCE_0** = 0U }

FREQM Instance type.

enum **FREQM_StatusType** { **FREQM_STATUS_SUCCESS** = 0x0U, **FREQM_STATUS_FAIL** = 0x1U,
FREQM_STATUS_TIMEOUT = 0x2U }

FREQM operation return values.

Functions

FREQM_StatusType **FREQM_Init** (**FREQM_HandleType** *const pHandle, const **FREQM_CfgType** *const
pInitStruct)

Initializes the FREQM module with the specified configuration.

FREQM_StatusType **FREQM_DeInit** (**FREQM_HandleType** *pHandle)

Deinitializes the FREQM module.

FREQM_StatusType **FREQM_StartMeasure** (**FREQM_HandleType** *pHandle)

Starts the frequency measurement process in the FREQM module.

FREQM_StatusType **FREQM_ClearStatus** (**FREQM_HandleType** *pHandle)

Clears the status of the FREQM module.

void **FREQM_CommonIRQHandler** (**FREQM_HandleType** *const pHandle)

Handles common interrupts for the FREQM module.

FREQM API Service IDs

#define **FREQM_INIT_ID** 0x00U

#define **FREQM_DEINIT_ID** 0x01U

#define **FREQM_STARTMEASURE_ID** 0x02U

#define **FREQM_CLEARSTATUS_ID** 0x03U

#define **FREQM_COMMONIRQHANDLER_ID** 0x04U

FREQM Dev Error Code

Error Code of calling FREQM apis

```
#define FREQM_E_PARAM_NULLPTR 0x01U
#define FREQM_E_PARAM_INSTANCE 0x02U
```

Detailed Description

Macro Definition Documentation

```
#define FREQM_CLEARSTATUS_ID 0x03U

#define FREQM_COMMONIRQHANDLER_ID 0x04U

#define FREQM_DEINIT_ID 0x01U

#define FREQM_E_PARAM_INSTANCE 0x02U

#define FREQM_E_PARAM_NULLPTR 0x01U

#define FREQM_INIT_ID 0x00U

#define FREQM_STARTMEASURE_ID 0x02U
```

Typedef Documentation

```
typedef struct _FREQM_HandleType FREQM_HandleType
```

The structure of the FREQM processing handle.

Enumeration Type Documentation

enum FREQM_InstanceType

FREQM Instance type.

Enumerator:

FREQM_INSTAN CE_0	
----------------------	--

enum FREQM_StatusType

FREQM operation return values.

Enumerator:

FREQM_STATUS _SUCCESS	The FREQM status success
FREQM_STATUS _FAIL	The FREQM status fail
FREQM_STATUS _TIMEOUT	The FREQM status timeout

Function Documentation

FREQM_StatusType FREQM_ClearStatus (FREQM_HandleType * *pHandle*)

Clears the status of the FREQM module.

This function clears the status of the specified FREQM module instance by resetting the reference counter and waiting for the counters to stop.

Parameters

<i>pHandle</i>	Pointer to the FREQM handle structure.
----------------	--

Returns

FREQM_StatusType Indicates the status of the clear status operation (FREQM_STATUS_SUCCESS or FREQM_STATUS_FAIL).

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported, and the function returns FREQM_STATUS_FAIL.

void FREQM_CommonIRQHandler (FREQM_HandleType *const *pHandle*)

Handles common interrupts for the FREQM module.

This function processes the common interrupts for the specified FREQM module instance. It checks the interrupt flags and calls the appropriate callback functions based on the interrupt status.

Parameters

<i>pHandle</i>	Pointer to the FREQM handle structure.
----------------	--

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported.

FREQM_StatusType FREQM_DeInit (FREQM_HandleType * *pHandle*)

Deinitializes the FREQM module.

This function deinitializes the specified FREQM module instance by resetting its configuration settings to default values and disabling the counter event interrupt.

Parameters

<i>pHandle</i>	Pointer to the FREQM handle structure.
----------------	--

Returns

FREQM_StatusType Indicates the status of the deinitialization process (FREQM_STATUS_SUCCESS or FREQM_STATUS_FAIL).

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported, and the function returns FREQM_STATUS_FAIL.

FREQM_StatusType FREQM_Init (FREQM_HandleType *const *pHandle*, const FREQM_CfgType *const *pInitStruct*)

Initializes the FREQM module with the specified configuration.

This function initializes the FREQM module with the given configuration settings, including the clock divider value, the clock source to be measured, the measurement length, and the reference timeout.

Parameters

<i>pHandle</i>	Pointer to the FREQM handle structure.
<i>pInitStruct</i>	Pointer to the initialization structure containing the configuration settings.

Returns

FREQM_StatusType Indicates the status of the initialization process (FREQM_STATUS_SUCCESS or FREQM_STATUS_FAIL).

Note

If any of the required pointers are NULL or the instance index is out of range, a device error will be reported, and the function returns FREQM_STATUS_FAIL.

FREQM_StatusType FREQM_StartMeasure (FREQM_HandleType * *pHandle*)

Starts the frequency measurement process in the FREQM module.

This function starts the frequency measurement process by resetting the measurement counter to zero.

Parameters

<i>pHandle</i>	Pointer to the FREQM handle structure.
----------------	--

Returns

FREQM_StatusType Indicates the status of the start measurement process (FREQM_STATUS_SUCCESS or FREQM_STATUS_FAIL).

Note

If the handle pointer is NULL or the instance index is out of range, a device error will be reported, and the function returns FREQM_STATUS_FAIL.

Module_driver_ftu

Collaboration diagram for Module_driver_ftu:Modules

HwA_ftu

Classes

struct **FTU_SignalMeasureType**

FTU signal measurement configuration type.

struct **FTU_SignalMeasureValueType**

FTU signal measurement result time.

struct **FTU_ExpectEdgeNumberResultType**

Expect edge number result time.

struct **FTU_EdgeNumberMeasureType**

FTU edge number measurement configuration type.

struct **FTU_ExpectEdgeNumberMeasureType**

FTU expect edge number measurement configuration type.

struct **_FTU_HandleType**

The structure of the FTU processing handle.

struct **FTU_PwmChannelType**

The configuration for the Pwm channel.

struct **FTU_PwmModeType**

The configuration option for the Pwm Mode.

struct **FTU_OutputCompareModeType**

The configuration option for the output compare mode.

struct **FTU_CounterModeType**

The configuration option for the time counter mode.

struct **FTU_PwmDutyUpdateType**

The configuration option for update PWM Duty.

struct **FTU_CommonType**

The basic configuration option for the FTU peripheral.

struct **FTU_FaultInitType**

The configuration option for initializing FTU fault.

struct **FTU_FaultControlType**

The configuration option for enabling a FTU fault.

struct **FTU_InputChannelType**

The configuration option for a input capture channel.

struct **FTU_QuadratureInitType**

The configuration option for the quadrature decoder mode.

Macros

#define **FTU_FAULT_CHANNEL_COUNT** 4u

Max number of Ftu fault channel.

#define **FTU_INPUT_FILTER_COUNT** 4u

Max number of Ftu input count.

#define **FTU_StopGlobalTimeBase** SMISC_HWA_EnabeFtuGTB

Stop the FTU global time base.

#define **FTU_StartGlobalTimeBase** SMISC_HWA_DisabeFtuGTB

Start the FTU global time base.

Typedefs

typedef struct **_FTU_HandleType** **FTU_HandleType**

The structure of the FTU processing handle.

Enumerations

enum **FTU_StatusType** { **FTU_STATUS_SUCCESS** = 0U, **FTU_STATUS_PARAM_INVALID**,
FTU_STATUS_NO_CLOCK_SOURCE }

FTU operation return values.

enum **FTU_InstanceType** { **FTU_INSTANCE_0** = 0U, **FTU_INSTANCE_1**, **FTU_INSTANCE_2**,
FTU_INSTANCE_3, **FTU_INSTANCE_4**, **FTU_INSTANCE_5**, **FTU_INSTANCE_6**, **FTU_INSTANCE_7**,
FTU_INSTANCE_8, **FTU_INSTANCE_9**, **FTU_INSTANCE_10**, **FTU_INSTANCE_11** }

The instance index of the FTU peripheral.

enum **FTU_ChannelType** { **FTU_CHANNEL_0** = 0U, **FTU_CHANNEL_1**, **FTU_CHANNEL_2**, **FTU_CHANNEL_3**,
FTU_CHANNEL_4, **FTU_CHANNEL_5**, **FTU_CHANNEL_6**, **FTU_CHANNEL_7** }

The channel index of the FTU peripheral.

```
enum FTU_ClkSrcType { FTU_NO_CLK = 0U, FTU_INTERNAL_CLK = 1U, FTU_PCC_CLK = 2U,  
FTU_EXTERNAL_CLK0 = 3U, FTU_EXTERNAL_CLK1 = 4U, FTU_EXTERNAL_CLK2 = 5U }
```

The clock source of FTU peripheral.

```
enum FTU_OutputComparePinModeType { FTU_OUTPUT_TOGGLE_PIN = FTU_CHANNEL_OC_TOGGLE,  
FTU_OUTPUT_CLEAR_PIN = FTU_CHANNEL_OC_CLEAR, FTU_OUTPUT_SET_PIN =  
FTU_CHANNEL_OC_SET }
```

The channel operation of output compare mode.

```
enum FTU_OutputCompareInitLevelType { FTU_OUTPUT_CMP_INIT_LOW = 0U,  
FTU_OUTPUT_CMP_INIT_HIGH }
```

The initial level of the output compare channel.

```
enum FTU_PwmPinModeType { FTU_PWM_HIGH_TRUE_PULSE = FTU_CHANNEL_PWM_HIGH_TRUE,  
FTU_PWM_LOW_TRUE_PULSE = FTU_CHANNEL_PWM_LOW_TRUE }
```

The channel operation of PWM mode.

```
enum FTU_PwmAlignedType { FTU_EDGE_ALIGNED_PWM = 0u, FTU_CENTER_ALIGNED_PWM }
```

PWM Aligned Mode.

```
enum FTU_MeasureContModeType { FTU_MEASURE_SINGLE_MODE = 0u,  
FTU_MEASURE_CONTINUOUS_MODE }
```

The measurement is single or continuous.

```
enum FTU_MeasureStartModeType { FTU_MEASURE_START_AFTER_EDGE = 0u,  
FTU_MEASURE_START_IMMEDIATELY }
```

Timing of start measurement.

```
enum FTU_SignalMeasureModeType { FTU_SIGNAL_MEASURE_HIGH_TIME = 0u,  
FTU_SIGNAL_MEASURE_LOW_TIME = 1u, FTU_SIGNAL_MEASURE_PERIOD_RISING_EDGE = 2u,  
FTU_SIGNAL_MEASURE_PERIOD_FALLING_EDGE = 3u }
```

FTU signal measurement mode type.

```
enum FTU_ReloadPointCfgType { FTU_RELOAD_POINT_CNTMAX = 1u, FTU_RELOAD_POINT_CNTMIN =  
(1u << 1), FTU_RELOAD_POINT_CHANNEL_0 = (1u << 2), FTU_RELOAD_POINT_CHANNEL_1 = (1u << 3),  
FTU_RELOAD_POINT_CHANNEL_2 = (1u << 4), FTU_RELOAD_POINT_CHANNEL_3 = (1u << 5),  
FTU_RELOAD_POINT_CHANNEL_4 = (1u << 6), FTU_RELOAD_POINT_CHANNEL_5 = (1u << 7),  
FTU_RELOAD_POINT_CHANNEL_6 = (1u << 8), FTU_RELOAD_POINT_CHANNEL_7 = (1u << 9) }
```

The reload point flag of FTU peripheral.

```
enum FTU_SyncFlagType { FTU_SYNC_FLAG_FTUEN = 1u, FTU_SYNC_FLAG_CNTINC = (1u << 1),  
FTU_SYNC_FLAG_PWMSYNC = (1u << 2), FTU_SYNC_FLAG_REINIT = (1u << 3),  
FTU_SYNC_FLAG_SYNCHOM = (1u << 4), FTU_SYNC_FLAG_SYNCEN01 = (1u << 5),  
FTU_SYNC_FLAG_SYNCEN23 = (1u << 6), FTU_SYNC_FLAG_SYNCEN45 = (1u << 7),
```


FTU_SYNC_FLAG_SYNCEN67 = (1u << 8), **FTU_SYNC_FLAG_HW_TRIG0** = (1u << 9),
FTU_SYNC_FLAG_HW_TRIG1 = (1u << 10), **FTU_SYNC_FLAG_HW_TRIG2** = (1u << 11) }

The synchronization flag of FTU peripheral.

enum **FTU_IntrMaskType** { **FTU_INTR_MASK_CHANNEL_0** = 1u, **FTU_INTR_MASK_CHANNEL_1** = (1u << 1),
FTU_INTR_MASK_CHANNEL_2 = (1u << 2), **FTU_INTR_MASK_CHANNEL_3** = (1u << 3),
FTU_INTR_MASK_CHANNEL_4 = (1u << 4), **FTU_INTR_MASK_CHANNEL_5** = (1u << 5),
FTU_INTR_MASK_CHANNEL_6 = (1u << 6), **FTU_INTR_MASK_CHANNEL_7** = (1u << 7),
FTU_INTR_MASK_OVERFLOW = (1u << 8), **FTU_INTR_MASK_FAULT** = (1u << 9),
FTU_INTR_MASK_RELOAD_POINT = (1u << 10) }

The interrupt enable/disable mask of FTU peripheral.

enum **FTU_FaultChannelEnableType** { **FTU_FAULT_FOR_CHANNEL01** = 1u,
FTU_FAULT_FOR_CHANNEL23 = (1u << 1), **FTU_FAULT_FOR_CHANNEL45** = (1u << 2),
FTU_FAULT_FOR_CHANNEL67 = (1u << 3) }

The fault control enable/disable mask for channels.

enum **FTU_FaultPolActiveType** { **FTU_FAULT_POL_ACTIVE_HIGH** = 0u, **FTU_FAULT_POL_ACTIVE_LOW** =
1u }

the fault input polarity

enum **FTU_TriggerOutputMaskType** { **FTU_TRIG_OUTPUT_MASK_CHANNEL_0_MATCH** = 1u,
FTU_TRIG_OUTPUT_MASK_CHANNEL_1_MATCH = (1u << 1),
FTU_TRIG_OUTPUT_MASK_CHANNEL_2_MATCH = (1u << 2),
FTU_TRIG_OUTPUT_MASK_CHANNEL_3_MATCH = (1u << 3),
FTU_TRIG_OUTPUT_MASK_CHANNEL_4_MATCH = (1u << 4),
FTU_TRIG_OUTPUT_MASK_CHANNEL_5_MATCH = (1u << 5),
FTU_TRIG_OUTPUT_MASK_CHANNEL_6_MATCH = (1u << 6),
FTU_TRIG_OUTPUT_MASK_CHANNEL_7_MATCH = (1u << 7),
FTU_TRIG_OUTPUT_MASK_ALL_CHANNEL_MATCH = (0xFFu), **FTU_TRIG_OUTPUT_MASK_RELOAD** = (1u
<< 8) }

the output trigger mask

enum **FTU_InputCapturePinModeType** { **FTU_INPUT_RISING_EDGE** = FTU_CHANNEL_EDGE_RISING,
FTU_INPUT_FALLING_EDGE = FTU_CHANNEL_EDGE_FALLING, **FTU_INPUT_BOTH_EDGE** =
FTU_CHANNEL_EDGE_BOTH }

Ftu input capture mode.

enum **FTU_GlobalTimeBaseStartInstanceType** { **FTU_GTB_INSTANCE_START_FTU0** = 1u,
FTU_GTB_INSTANCE_START_FTU1 = (1u << 1), **FTU_GTB_INSTANCE_START_FTU2** = (1u << 2),
FTU_GTB_INSTANCE_START_FTU3 = (1u << 3), **FTU_GTB_INSTANCE_START_FTU4** = (1u << 4),
FTU_GTB_INSTANCE_START_FTU5 = (1u << 5), **FTU_GTB_INSTANCE_START_FTU6** = (1u << 6),
FTU_GTB_INSTANCE_START_FTU7 = (1u << 7) }

Ftu Global Time Base instance start mask.

```
enum FTU_GlobalTimeBaseStartType { FTU_GTB_START_AT_ONCE = 1u,  
FTU_GTB_START_AT_TSTMP1_MOD0 = (1u << 1), FTU_GTB_START_AT_TSTMP1_MOD1 = (1u << 2),  
FTU_GTB_START_AT_TSTMP1_MOD2 = (1u << 3), FTU_GTB_START_AT_TSTMP1_MOD3 = (1u << 4) }
```

Ftu Global Time Base start mask.

```
enum FTU_PwmUpdateModeType { FTU_PWM_UPDATE_IMMEDIATELY,  
FTU_PWM_UPDATE_END_PERIOD, FTU_PWM_UPDATE_HALF_PERIOD,  
FTU_PWM_UPDATE_BOTH_HALF_AND_END_PERIOD }
```

Functions

```
void FTU_CommonInit (FTU_HandleType *pFtuHandle, const FTU_CommonType *const  
pCommonStruct)
```

Initialize FTU basic configuration.

```
void FTU_GetDefaultInitCfg (FTU_CommonType *pCommonStruct)
```

Fills in the FTU configuration structure with the default settings.

```
void FTU_CounterModeInit (FTU_HandleType *pFtuHandle, const FTU_CounterModeType *const  
pCounterStruct)
```

Configure FTU to counter mode.

```
void FTU_OutputCompareModeInit (FTU_HandleType *pFtuHandle, const  
FTU_OutputCompareModeType *const pOutputModeStruct)
```

Configure FTU to output compare mode.

```
void FTU_OutputCompareUpdate (FTU_HandleType *pFtuHandle, const  
FTU_OutputCompareModeType *const pOCConfig)
```

Update the behavior of Output compare channel.

```
uint32_t FTU_GetChannelValue (FTU_HandleType *pFtuHandle, uint32_t u32Channel)
```

Set the value of the FTU Channel.

```
uint32_t FTU_GetCounter (FTU_HandleType *pFtuHandle)
```

Get the value of the FTU counter.

void **FTU_SetChannelValue** (**FTU_HandleType** *pFtuHandle, uint32_t u32Channel, uint32_t u32Value)

Set the value of the FTU Channel.

void **FTU_PwmModelInit** (**FTU_HandleType** *pFtuHandle, const **FTU_PwmModeType** *const pPwmModeStruct)

Configure Configure FTU to PWM mode.

void **FTU_PwmUpdateDuty** (**FTU_HandleType** *pFtuHandle, **FTU_PwmDutyUpdateType** *pConfig)

Update the duty cycle of the FTU Channel.

void **FTU_PwmUpdatePeriod** (**FTU_HandleType** *pFtuHandle, uint32_t u32Period, bool bUpdate)

Update the Period cycle of the FTU instance.

void **FTU_PwmSyncUpdate** (**FTU_HandleType** *pFtuHandle)

Synchronous update cycle and period of the FTU instance.

void **FTU_EnableInterrupt** (**FTU_HandleType** *pFtuHandle, uint32_t u32InterruptMask)

Enable FTU interrupt.

void **FTU_DisableInterrupt** (**FTU_HandleType** *pFtuHandle, uint32_t u32InterruptMask)

Enable FTU interrupt.

void **FTU_StartTimer** (**FTU_HandleType** *pFtuHandle)

Start the FTU instance.

void **FTU_StopTimer** (**FTU_HandleType** *pFtuHandle)

Stop the FTU instance.

void **FTU_DeInit** (**FTU_HandleType** *pFtuHandle)

De-initialize the FTU instance.

void **FTU_FaultInit** (**FTU_HandleType** *pFtuHandle, const **FTU_FaultInitType** *const pFaultInit)

initialize fault input of the selected Ftu instance

void **FTU_FaultEnable** (FTU_HandleType *pFtuHandle, const FTU_FaultControlType *const pFaultCtrl)

Enable a fault input.

void **FTU_FaultSelectDelayValue** (FTU_HandleType *pFtuHandle, uint8_t u8Channel,
FTU_FaultDisableDelayType eDelaySelection)

Select fault disable channel output delay value.

uint8_t **FTU_GetEdgeNumberCount** (FTU_HandleType *pFtuHandle, uint8_t u8Channel)

Get Edge number counter.

void **FTU_EdgeNumberMeasureChannelInit** (FTU_HandleType *pFtuHandle,
FTU_EdgeNumberMeasureType *pEdgeNumMeasure)

Initialize a Edge Number Measurement channel.

void **FTU_SignalMeasureChannelInit** (FTU_HandleType *pFtuHandle, FTU_SignalMeasureType
*pMeasure)

Initialize a signal measure channel.

void **FTU_SignalMeasureChannelSingle** (FTU_HandleType *pFtuHandle, uint8_t u8Channel)

Re-start measurement when in single mode.

void **FTU_GetSignalMeasureResult** (const FTU_InstanceType eInstance, uint8_t u8Channel,
FTU_SignalMeasureValueType *pResult)

Get the measurement result of the channel.

void **FTU_FaultDisable** (FTU_HandleType *pFtuHandle, uint32_t u32FaultIndex)

Disable a fault input.

void **FTU_ClearFault** (FTU_HandleType *pFtuHandle, uint32_t u32FaultFlag)

Clear the fault flag of the FTU instance.

uint32_t **FTU_GetFaultFlag** (FTU_HandleType *pFtuHandle)

Get the fault flag of the FTU instance.

void **FTU_InputCaptureChannelInit** (FTU_HandleType *pFtuHandle, const FTU_InputChannelType *const pInputChannel)

initialize a input capture channel of the selected Ftu instance

void **FTU_EnableTriggerOutput** (FTU_HandleType *pFtuHandle, uint32_t u32TriggerOutputMask)

Enable the output trigger of the selected FTU instance.

void **FTU_DisableTriggerOutput** (FTU_HandleType *pFtuHandle, uint32_t u32TriggerOutputMask)

Disable the output trigger of the selected FTU instance.

void **FTU_QuadratureModelInit** (FTU_HandleType *pFtuHandle, const FTU_QuadratureInitType *const pQuadInit)

initialize the quadrature decoder mode

void **FTU_EnableChannelDma** (FTU_HandleType *pFtuHandle, uint32_t u32Channel)

Enable ftu channel DMA.

void **FTU_GetExpectEdgeNumberResult** (FTU_HandleType *pFtuHandle, uint8_t u8Channel, FTU_ExpectEdgeNumberResultType *pResult)

Get the expect edge number result of the channel.

void **FTU_ExpectEdgeNumberMeasureChannelInit** (FTU_HandleType *pFtuHandle, FTU_ExpectEdgeNumberMeasureType *pExpectEdgeNumMeasure)

Initialize a Expect Edge Number Measurement channel.

void **FTU_GetSignalMeasureResult** (FTU_HandleType *pFtuHandle, uint8_t u8Channel, FTU_SignalMeasureValueType *pResult)

Get the measurement result of the channel.

void **FTUn_IRQHandler** (FTU_HandleType *pFtuHandle)

Interrupt IRQ handle of FTU instance.

FTU Channel Bit Flag

Bit of channel indicate channel number

```
#define BIT_FTU_CHANNEL_0 0x01U
#define BIT_FTU_CHANNEL_1 0x02U
#define BIT_FTU_CHANNEL_2 0x04U
#define BIT_FTU_CHANNEL_3 0x08U
#define BIT_FTU_CHANNEL_4 0x10U
#define BIT_FTU_CHANNEL_5 0x20U
#define BIT_FTU_CHANNEL_6 0x40U
#define BIT_FTU_CHANNEL_7 0x80U
```

FTU API Service IDs

```
#define FTU_COMMON_INIT_ID 0U
#define FTU_DEINIT_ID 1U
#define FTU_GET_DEFAULT_INIT_CFG_ID 2U
#define FTU_COUNTER_INIT_ID 3U
#define FTU_PWM_UPDATE_DUTY_ID 4U
#define FTU_PWM_UPDATE_PERIOD_ID 5U
#define FTU_PWM_SYNC_UPDATE_ID 6U
#define FTU_PWM_MODE_INIT_ID 7U
#define FTU_OC_MODE_INIT_ID 8U
#define FTU_GET_CHANNEL_VALUE_ID 9U
#define FTU_GET_COUNTER_ID 10U
#define FTU_SET_CHANNEL_VALUE_ID 11U
#define FTU_ENABLE_TRIG_OUT_ID 12U
#define FTU_DISABLE_TRIG_OUT_ID 13U
#define FTU_START_TIMER_ID 14U
#define FTU_STOP_TIMER_ID 15U
#define FTU_INTR_INIT_ID 16U
#define FTU_FAULT_INIT_ID 17U
#define FTU_FAULT_SEL_DELAY_ID 18U
#define FTU_FAULT_ENABLE_ID 19U
#define FTU_FAULT_DISABLE_ID 20U
#define FTU_IC_CHANNEL_INIT_ID 21U
#define FTU_QD_MODE_INIT_ID 22U
#define FTU_ENABLE_INTR_ID 23U
#define FTU_DISABLE_INTR_ID 24U
#define FTU_CLEAR_FAULT_ID 25U
```

```
#define FTU_GET_FAULT_FLAG_ID 26U
#define FTU_ENABLE_CHANNEL_DMA_ID 27U
#define FTU_IC_EXPENM_INIT_ID 28U
#define FTU_IC_ENM_INIT_ID 29U
#define FTU_IC_EXPENM_GET_ID 30U
#define FTU_IC_ENM_GET_ID 31U
#define FTU_IC_SIG_MEAS_INIT_ID 32U
#define FTU_IC_SIG_MEAS_SINGLE_ID 33U
#define FTU_IC_SIG_MEAS_GET_ID 34U
#define FTU_OC_MODE_UPDATE_ID 35U
```

FTU Dev Error Code

Error Code of calling FTU apis

```
#define FTU_E_PARAM_INSTANCE 0x01U
#define FTU_E_PARAM_CHANNEL 0x02U
#define FTU_E_PARAM_POINTER 0x03U
#define FTU_E_PARAM_COUNT 0x04U
#define FTU_E_PARAM_INVALID 0x05U
#define FTU_E_PARAM_PHASE_SHIFT 0x06U
#define FTU_E_PARAM_INVALID_CHANNEL 0x07U
```

FTU Fault Bit Flag

Bit of fault flag

```
#define FTU_FAULT_INPUT_0 0x01U
#define FTU_FAULT_INPUT_1 0x02U
```

Detailed Description

Macro Definition Documentation

#define BIT_FTU_CHANNEL_0 0x01U

#define BIT_FTU_CHANNEL_1 0x02U

#define BIT_FTU_CHANNEL_2 0x04U

#define BIT_FTU_CHANNEL_3 0x08U

#define BIT_FTU_CHANNEL_4 0x10U

#define BIT_FTU_CHANNEL_5 0x20U

#define BIT_FTU_CHANNEL_6 0x40U

#define BIT_FTU_CHANNEL_7 0x80U

#define FTU_CLEAR_FAULT_ID 25U

#define FTU_COMMON_INIT_ID 0U

#define FTU_COUNTER_INIT_ID 3U

#define FTU_DEINIT_ID 1U

#define FTU_DISABLE_INTR_ID 24U

#define FTU_DISABLE_TRIG_OUT_ID 13U

#define FTU_E_PARAM_CHANNEL 0x02U

#define FTU_E_PARAM_COUNT 0x04U

#define FTU_E_PARAM_INSTANCE 0x01U

#define FTU_E_PARAM_INVALID 0x05U

#define FTU_E_PARAM_INVALID_CHANNEL 0x07U

#define FTU_E_PARAM_PHASE_SHIFT 0x06U

#define FTU_E_PARAM_POINTER 0x03U


```
#define FTU_ENABLE_CHANNEL_DMA_ID 27U
```

```
#define FTU_ENABLE_INTR_ID 23U
```

```
#define FTU_ENABLE_TRIG_OUT_ID 12U
```

```
#define FTU_FAULT_CHANNEL_COUNT 4u
```

Max number of Ftu fault channel.

```

#define FTU_FAULT_DISABLE_ID 20U

#define FTU_FAULT_ENABLE_ID 19U

#define FTU_FAULT_INIT_ID 17U

#define FTU_FAULT_INPUT_0 0x01U

#define FTU_FAULT_INPUT_1 0x02U

#define FTU_FAULT_SEL_DELAY_ID 18U

#define FTU_GET_CHANNEL_VALUE_ID 9U

#define FTU_GET_COUNTER_ID 10U

#define FTU_GET_DEFAULT_INIT_CFG_ID 2U

#define FTU_GET_FAULT_FLAG_ID 26U

#define FTU_IC_CHANNEL_INIT_ID 21U

#define FTU_IC_ENM_GET_ID 31U

#define FTU_IC_ENM_INIT_ID 29U

#define FTU_IC_EXPENM_GET_ID 30U

#define FTU_IC_EXPENM_INIT_ID 28U

#define FTU_IC_SIG_MEAS_GET_ID 34U

#define FTU_IC_SIG_MEAS_INIT_ID 32U

#define FTU_IC_SIG_MEAS_SINGLE_ID 33U

#define FTU_INPUT_FILTER_COUNT 4u

```

Max number of Ftu input count.

#define FTU_INTR_INIT_ID 16U

#define FTU_OC_MODE_INIT_ID 8U

#define FTU_OC_MODE_UPDATE_ID 35U

#define FTU_PWM_MODE_INIT_ID 7U

#define FTU_PWM_SYNC_UPDATE_ID 6U

#define FTU_PWM_UPDATE_DUTY_ID 4U

#define FTU_PWM_UPDATE_PERIOD_ID 5U

#define FTU_QD_MODE_INIT_ID 22U

#define FTU_SET_CHANNEL_VALUE_ID 11U

#define FTU_START_TIMER_ID 14U

#define FTU_StartGlobalTimeBase SMISC_HWA_DisabeFtuGTB

Start the FTU global time base.

#define FTU_STOP_TIMER_ID 15U

#define FTU_StopGlobalTimeBase SMISC_HWA_EnabeFtuGTB

Stop the FTU global time base.

Typedef Documentation

typedef struct _FTU_HandleType FTU_HandleType

The structure of the FTU processing handle.

Enumeration Type Documentation

enum FTU_ChannelType

The channel index of the FTU peripheral.

Enumerator:

FTU_CHANNEL_ 0	
FTU_CHANNEL_ 1	
FTU_CHANNEL_ 2	
FTU_CHANNEL_ 3	
FTU_CHANNEL_ 4	
FTU_CHANNEL_ 5	
FTU_CHANNEL_ 6	
FTU_CHANNEL_ 7	

enum FTU_ClkSrcType

The clock source of FTU peripheral.

Enumerator:

FTU_NO_CLK	No clock selected
------------	-------------------

FTU_INTERNAL_CLK	FTU input clock
FTU_PCC_CLK	FTU input PCC clock
FTU_EXTERNAL_CLK0	External pin 0 input clock
FTU_EXTERNAL_CLK1	External pin 1 input clock
FTU_EXTERNAL_CLK2	External pin 2 input clock

enum FTU_FaultChannelEnableType

The fault control enable/disable mask for channels.

Enumerator:

FTU_FAULT_FO_R_CHANNEL01	fault control for channel 0 and channel 1
FTU_FAULT_FO_R_CHANNEL23	fault control for channel 2 and channel 3
FTU_FAULT_FO_R_CHANNEL45	fault control for channel 4 and channel 5
FTU_FAULT_FO_R_CHANNEL67	fault control for channel 6 and channel 7

enum FTU_FaultPolActiveType

the fault input polarity

Enumerator:

FTU_FAULT_PO L_ACTIVE_HIGH	The fault input polarity is active high. 1 at the fault input indicates a fault
FTU_FAULT_PO L_ACTIVE_LOW	The fault input polarity is active low. 0 at the fault input indicates a fault

enum FTU_GlobalTimeBaseStartInstanceType

Ftu Global Time Base instance start mask.

Enumerator:

FTU_GTB_INST ANCE_START_F TU0	FTU0 GTB start
FTU_GTB_INST ANCE_START_F TU1	FTU1 GTB start
FTU_GTB_INST ANCE_START_F TU2	FTU2 GTB start
FTU_GTB_INST ANCE_START_F TU3	FTU3 GTB start
FTU_GTB_INST ANCE_START_F TU4	FTU4 GTB start
FTU_GTB_INST ANCE_START_F TU5	FTU5 GTB start
FTU_GTB_INST ANCE_START_F TU6	FTU6 GTB start

FTU_GTB_INST ANCE_START_F TU7	FTU7 GTB start
-------------------------------------	----------------

enum FTU_GlobalTimeBaseStartType

Ftu Global Time Base start mask.

Enumerator:

FTU_GTB_STAR T_AT_ONCE	GTB start at once
FTU_GTB_STAR T_AT_TSTMP1_ MOD0	GTB start at modulate timer 0 of TSTMP1
FTU_GTB_STAR T_AT_TSTMP1_ MOD1	GTB start at modulate timer 1 of TSTMP1
FTU_GTB_STAR T_AT_TSTMP1_ MOD2	GTB start at modulate timer 2 of TSTMP1
FTU_GTB_STAR T_AT_TSTMP1_ MOD3	GTB start at modulate timer 3 of TSTMP1

enum FTU_InputCapturePinModeType

Ftu input capture mode.

Enumerator:

FTU_INPUT_RIS ING_EDGE	capture on rising edge only
FTU_INPUT_FAL LING_EDGE	capture on falling edge only
FTU_INPUT_BO TH_EDGE	capture on rising or falling edge

enum FTU_InstanceType

The instance index of the FTU peripheral.

Enumerator:

FTU_INSTANCE _0	
FTU_INSTANCE _1	
FTU_INSTANCE _2	
FTU_INSTANCE _3	
FTU_INSTANCE _4	
FTU_INSTANCE _5	
FTU_INSTANCE _6	
FTU_INSTANCE _7	
FTU_INSTANCE _8	
FTU_INSTANCE _9	
FTU_INSTANCE _10	
FTU_INSTANCE _11	

enum FTU_IntrMaskType

The interrupt enable/disable mask of FTU peripheral.

Enumerator:

FTU_INTR_MAS K_CHANNEL_0	interrupt mask of channel 0
FTU_INTR_MAS K_CHANNEL_1	interrupt mask of channel 1
FTU_INTR_MAS K_CHANNEL_2	interrupt mask of channel 2
FTU_INTR_MAS K_CHANNEL_3	interrupt mask of channel 3
FTU_INTR_MAS K_CHANNEL_4	interrupt mask of channel 4
FTU_INTR_MAS K_CHANNEL_5	interrupt mask of channel 5
FTU_INTR_MAS K_CHANNEL_6	interrupt mask of channel 6
FTU_INTR_MAS K_CHANNEL_7	interrupt mask of channel 7
FTU_INTR_MAS K_OVERFLOW	interrupt mask of overflow
FTU_INTR_MAS K_FAULT	interrupt mask of fault
FTU_INTR_MAS K_RELOAD_POI NT	interrupt mask of reload point

enum FTU_MeasureContModeType

The measurement is single or continuous.

Enumerator:

FTU_MEASURE_ SINGLE_MODE	Single mode
-----------------------------	-------------

FTU_MEASURE_ CONTINUOUS_ MODE	Continuous mode
-------------------------------------	-----------------

enum FTU_MeasureStartModeType

Timing of start measurement.

Enumerator:

FTU_MEASURE_ START_AFTER_ EDGE	The channel starts measuring after the first edge is detected
FTU_MEASURE_ START_IMMEDIA TELY	The measurement starts immediately after activating the channel

enum FTU_OutputCompareInitLevelType

The initial level of the output compare channel.

Enumerator:

FTU_OUTPUT_C MP_INIT_LOW	The initial level is low
FTU_OUTPUT_C MP_INIT_HIGH	The initial level is high

enum FTU_OutputComparePinModeType

The channel operation of output compare mode.

Enumerator:

FTU_OUTPUT_TOGGLE_PIN	Toggle Output on match
FTU_OUTPUT_CLEAR_PIN	Clear Output on match
FTU_OUTPUT_SET_PIN	Set Output on match

enum FTU_PwmAlignedType

PWM Aligned Mode.

Enumerator:

FTU_EDGE_ALIGNED_PWM	Edge-Aligned PWM
FTU_CENTER_ALIGNED_PWM	Center-Aligned PWM

enum FTU_PwmPinModeType

The channel operation of PWM mode.

Enumerator:

FTU_PWM_HIGH_TRUE_PULSE	High-true pulses in PWM
FTU_PWM_LOW_TRUE_PULSE	Low-true pulses in PWM

enum FTU_PwmUpdateModeType

Enumerator:

FTU_PWM_UPD ATE_IMMEDIATE LY	Pwm duty and period update immediately
FTU_PWM_UPD ATE_END_PERIOD	Pwm duty and period update when instance count reload
FTU_PWM_UPD ATE_HALF_PERIOD	Pwm duty and period update when instance count overflow(only for CPWM)
FTU_PWM_UPD ATE_BOTH_HALF_AND_END_PERIOD	Pwm duty and period update both instance count overflow and reload(only for CPWM)

enum FTU_ReloadPointCfgType

The reload point flag of FTU peripheral.

Enumerator:

FTU_RELOAD_POINT_CNTMAX	Maximum Loading Point
FTU_RELOAD_POINT_CNTMIN	Minimum Loading Point
FTU_RELOAD_POINT_CHANNEL_0	Channel 0 match is included as a reload opportunity
FTU_RELOAD_POINT_CHANNEL_1	Channel 1 match is included as a reload opportunity
FTU_RELOAD_POINT_CHANNEL_2	Channel 2 match is included as a reload opportunity

FTU_RELOAD_P OINT_CHANNEL _3	Channel 3 match is included as a reload opportunity
FTU_RELOAD_P OINT_CHANNEL _4	Channel 4 match is included as a reload opportunity
FTU_RELOAD_P OINT_CHANNEL _5	Channel 5 match is included as a reload opportunity
FTU_RELOAD_P OINT_CHANNEL _6	Channel 6 match is included as a reload opportunity
FTU_RELOAD_P OINT_CHANNEL _7	Channel 7 match is included as a reload opportunity

enum FTU_SignalMeasureModeType

FTU signal measurement mode type.

Enumerator:

FTU_SIGNAL_M EASURE_HIGH_ TIME	Measurement high time
FTU_SIGNAL_M EASURE_LOW_T IME	Measurement low time
FTU_SIGNAL_M EASURE_PERIO D_RISING_EDGE	Measurement period of rising edge
FTU_SIGNAL_M EASURE_PERIO D_FALLING_EDG E	Measurement period of falling edge

enum FTU_StatusType

FTU operation return values.

Enumerator:

FTU_STATUS_SUCCESS	The FTU operation is success
FTU_STATUS_PARAMETER_INVALID	The FTU operation is failed because of parameter error
FTU_STATUS_INSTANCE_CLOCK_SOURCE_NOT_SET	The FTU operation is failed because of instance clock source is not set

enum FTU_SyncFlagType

The synchronization flag of FTU peripheral.

Enumerator:

FTU_SYNC_FLAG_FTUEN	FTU Enable
FTU_SYNC_FLAG_CNTINC	CNTIN Register Synchronization
FTU_SYNC_FLAG_PWM_SYNC	Synchronization Mode: Selects which triggers can be used by MOD, CV, OUTMASK, and FTU counter synchronization.
FTU_SYNC_FLAG_REINIT	FTU Counter re-initialization by Synchronization
FTU_SYNC_FLAG_SYNC_HOM	Selects when the OUTMASK register is updated with the value of its buffer
FTU_SYNC_FLAG_SYNC_CEN01	synchronization of registers C(0)V and C(1)V.

FTU_SYNC_FL G_SYNCEN23	synchronization of registers C(2)V and C(3)V.
FTU_SYNC_FL G_SYNCEN45	synchronization of registers C(4)V and C(5)V.
FTU_SYNC_FL G_SYNCEN67	synchronization of registers C(6)V and C(7)V.
FTU_SYNC_FL G_HW_TRIG0	hardware trigger 0 to the synchronization
FTU_SYNC_FL G_HW_TRIG1	hardware trigger 1 to the synchronization
FTU_SYNC_FL G_HW_TRIG2	hardware trigger 2 to the synchronization

enum FTU_TriggerOutputMaskType

the output trigger mask

Enumerator:

FTU_TRIG_OUT PUT_MASK_CHANNEL_0_MATCH	Enables/Disables the channel 0 match trigger
FTU_TRIG_OUT PUT_MASK_CHANNEL_1_MATCH	Enables/Disables the channel 1 match trigger
FTU_TRIG_OUT PUT_MASK_CHANNEL_2_MATCH	Enables/Disables the channel 2 match trigger
FTU_TRIG_OUT PUT_MASK_CHANNEL_3_MATCH	Enables/Disables the channel 3 match trigger
FTU_TRIG_OUT PUT_MASK_CHANNEL_4_MATCH	Enables/Disables the channel 4 match trigger

FTU_TRIG_OUT PUT_MASK_CHA NNEL_5_MATCH	Enables/Disables the channel 5 match trigger
FTU_TRIG_OUT PUT_MASK_CHA NNEL_6_MATCH	Enables/Disables the channel 6 match trigger
FTU_TRIG_OUT PUT_MASK_CHA NNEL_7_MATCH	Enables/Disables the channel 7 match trigger
FTU_TRIG_OUT PUT_MASK_ALL _CHANNEL_MAT CH	Enables/Disables all channel match trigger
FTU_TRIG_OUT PUT_MASK_REL OAD	Enables/Disables reload trigger

Function Documentation

void FTU_ClearFault (FTU_HandleType * *pFtuHandle*, uint32_t *u32FaultFlag*)

Clear the fault flag of the FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32FaultFlag</i>	flag to clear

void FTU_CommonInit (FTU_HandleType * *pFtuHandle*, const FTU_CommonType *const *pCommonStruct*)

Initialize FTU basic configuration.

Parameters

<i>pFtuHandle</i>	FTU processing handle
-------------------	-----------------------

<i>pCommonStruct</i>	the basic configurations of the FTU instance
----------------------	--

void FTU_CounterModeInit (FTU_HandleType * *pFtuHandle*, const FTU_CounterModeType *const *pCounterStruct*)

Configure FTU to counter mode.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pCounterStruct</i>	the configurations of the counter mode

Note

This function will stop timer

void FTU_DeInit (FTU_HandleType * *pFtuHandle*)

De-initialize the FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
-------------------	-----------------------

void FTU_DisableInterrupt (FTU_HandleType * *pFtuHandle*, uint32_t *u32InterruptMask*)

Enable FTU interrupt.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32InterruptMask</i>	interrupt disable mask

void FTU_DisableTriggerOutput (FTU_HandleType * *pFtuHandle*, uint32_t *u32TriggerOutputMask*)

Disable the output trigger of the selected FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32TriggerOutputMask</i>	the output trigger mask

**void FTU_EdgeNumberMeasureChannelInit (FTU_HandleType * *pFtuHandle*,
FTU_EdgeNumberMeasureType * *pEdgeNumMeasure*)**

Initialize a Edge Number Measurement channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pEdgeNumMeasure</i>	measurement configuration.

void FTU_EnableChannelDma (FTU_HandleType * *pFtuHandle*, uint32_t *u32Channel*)

Enable ftu channel DMA.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32Channel</i>	The FTU channel .

void FTU_EnableInterrupt (FTU_HandleType * *pFtuHandle*, uint32_t *u32InterruptMask*)

Enable FTU interrupt.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32InterruptMask</i>	interrupt enable mask

void FTU_EnableTriggerOutput (FTU_HandleType * *pFtuHandle*, uint32_t *u32TriggerOutputMask*)

Enable the output trigger of the selected FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32TriggerOutp utMask</i>	the output trigger mask

**void FTU_ExpectEdgeNumberMeasureChannelInit (FTU_HandleType * *pFtuHandle*,
FTU_ExpectEdgeNumberMeasureType * *pExpectEdgeNumMeasure*)**

Initialize a Expect Edge Number Measurement channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pExpectEdgeNu mMeasure</i>	measurement configuration.

void FTU_FaultDisable (FTU_HandleType * *pFtuHandle*, uint32_t *u32FaultIndex*)

Disable a fault input.

Parameters

<i>pFtuHandle</i>	FTU processing handlece
<i>u32FaultIndex</i>	index of the fault input

void FTU_FaultEnable (FTU_HandleType * *pFtuHandle*, const FTU_FaultControlType *const *pFaultCtrl*)

Enable a fault input.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pFaultCtrl</i>	configurations of the fault input

void FTU_FaultInit (FTU_HandleType * *pFtuHandle*, const FTU_FaultInitType *const *pFaultInit*)

initialize fault input of the selected Ftu instance

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pFaultInit</i>	the fault configurations of the FTU instance

void FTU_FaultSelectDelayValue (FTU_HandleType * *pFtuHandle*, uint8_t *u8Channel*, FTU_FaultDisableDelayType *eDelaySelection*)

Select fault disable channel output delay value.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u8Channel</i>	FTU channel number, range is 0-7.
<i>eDelaySelection</i>	Fault disable channel output delay value selection.

uint32_t FTU_GetChannelValue (FTU_HandleType * *pFtuHandle*, uint32_t *u32Channel*)

Set the value of the FTU Channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32Channel</i>	FTU Channel

Returns

the value of the FTU Channel

uint32_t FTU_GetCounter (FTU_HandleType * *pFtuHandle*)

Get the value of the FTU counter.

Parameters

<i>pFtuHandle</i>	FTU processing handle
-------------------	-----------------------

Returns

the counter of the FTU

void FTU_GetDefaultInitCfg (FTU_CommonType * *pCommonStruct*)

Fills in the FTU configuration structure with the default settings.

Parameters

<i>pCommonStruct</i>	Pointer to the user configuration structure
----------------------	---

uint8_t FTU_GetEdgeNumberCount (FTU_HandleType * *pFtuHandle*, uint8_t *u8Channel*)

Get Edge number counter.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u8Channel</i>	FTU channel number.

Returns

uint8_t Edge number counter

void FTU_GetExpectEdgeNumberResult (FTU_HandleType * *pFtuHandle*, uint8_t *u8Channel*, FTU_ExpectEdgeNumberResultType * *pResult*)

Get the expect edge number result of the channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u8Channel</i>	FTU channel number, range is 0-7.
<i>pResult</i>	point to the result buffer.

uint32_t FTU_GetFaultFlag (FTU_HandleType * *pFtuHandle*)

Get the fault flag of the FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
-------------------	-----------------------

Returns

uint32_t the fault flag of the selected Ftu instance

```
void FTU_GetSignalMeasureResult (FTU_HandleType * pFtuHandle, uint8_t u8Channel,  
FTU_SignalMeasureValueType * pResult)
```

Get the measurement result of the channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u8Channel</i>	FTU channel number, range is 0-7.
<i>pResult</i>	point to the result buffer.

```
void FTU_GetSignaMeasureResult (const FTU_InstanceType eInstance, uint8_t u8Channel,  
FTU_SignalMeasureValueType * pResult)
```

Get the measurement result of the channel.

Parameters

<i>eInstance</i>	the selected FTU instance
<i>u8Channel</i>	FTU channel number, range is 0-7.
<i>pResult</i>	point to the result buffer.

```
void FTU_InputCaptureChannelInit (FTU_HandleType * pFtuHandle, const FTU_InputChannelType  
*const pInputChannel)
```

initialize a input capture channel of the selected Ftu instance

Parameters

<i>pFtuHandle</i>	the selected FTU instance
<i>pInputChannel</i>	configurations of the input capture channel

```
void FTU_OutputCompareModelInit (FTU_HandleType * pFtuHandle, const  
FTU_OutputCompareModeType *const pOutputModeStruct)
```

Configure FTU to output compare mode.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pOutputModeStruct</i>	the configurations of the output compare mode

Note

This function will stop timer

```
void FTU_OutputCompareUpdate (FTU_HandleType * pFtuHandle, const  
FTU_OutputCompareModeType *const pOCCConfig)
```

Update the behavior of Output compare channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pOCCConfig</i>	the configurations of the output compare mode

```
void FTU_PwmModeInit (FTU_HandleType * pFtuHandle, const FTU_PwmModeType *const  
pPwmModeStruct)
```

Configure Configure FTU to PWM mode.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pPwmModeStruct</i>	the configurations of the PWM mode

Note

This function will stop timer

```
void FTU_PwmSyncUpdate (FTU_HandleType * pFtuHandle)
```

Synchronous update cycle and period of the FTU instance.

Parameters

<i>pFtuHandle</i>	the selected FTU instance
-------------------	---------------------------

void FTU_PwmUpdateDuty (FTU_HandleType * *pFtuHandle*, FTU_PwmDutyUpdateType * *pConfig*)

Update the duty cycle of the FTU Channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pConfig</i>	The configuration option for update PWM Duty

void FTU_PwmUpdatePeriod (FTU_HandleType * *pFtuHandle*, uint32_t *u32Period*, bool *bUpdate*)

Update the Period cycle of the FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32Period</i>	period of the PWM mode
<i>bUpdate</i>	if enable update

void FTU_QuadratureModelInit (FTU_HandleType * *pFtuHandle*, const FTU_QuadratureInitType *const *pQuadInit*)

initialize the quadrature decoder mode

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pQuadInit</i>	configurations of the quadrature decoder

void FTU_SetChannelValue (FTU_HandleType * *pFtuHandle*, uint32_t *u32Channel*, uint32_t *u32Value*)

Set the value of the FTU Channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u32Channel</i>	FTU Channel
<i>u32Value</i>	the value to set

void FTU_SignalMeasureChannelInit (FTU_HandleType * *pFtuHandle*, FTU_SignalMeasureType * *pMeasure*)

Initialize a signal measure channel.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>pMeasure</i>	measurement configuration.

void FTU_SignalMeasureChannelSingle (FTU_HandleType * *pFtuHandle*, uint8_t *u8Channel*)

Re-start measurement when in single mode.

Parameters

<i>pFtuHandle</i>	FTU processing handle
<i>u8Channel</i>	FTU channel number, range is 0-7.

void FTU_StartTimer (FTU_HandleType * *pFtuHandle*)

Start the FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
-------------------	-----------------------

void FTU_StopTimer (FTU_HandleType * *pFtuHandle*)

Stop the FTU instance.

Parameters

<i>pFtuHandle</i>	FTU processing handle
-------------------	-----------------------

void FTUn_IRQHandler (FTU_HandleType * *pFtuHandle*)

Interrupt IRQ handle of FTU instance.

Parameters

<i>pFtuHandle</i>	the selected FTU instance
-------------------	---------------------------

Module_driver_gpio

Collaboration diagram for Module_driver_gpio:Modules

HwA_gpio

Macros

```
#define GPIO_MAX_PORT_PINS 32
#define GPIO_PIN_0 ((uint32_t)0x00000001)
#define GPIO_PIN_1 ((uint32_t)0x00000002)
#define GPIO_PIN_2 ((uint32_t)0x00000004)
#define GPIO_PIN_3 ((uint32_t)0x00000008)
#define GPIO_PIN_4 ((uint32_t)0x00000010)
#define GPIO_PIN_5 ((uint32_t)0x00000020)
#define GPIO_PIN_6 ((uint32_t)0x00000040)
#define GPIO_PIN_7 ((uint32_t)0x00000080)
#define GPIO_PIN_8 ((uint32_t)0x00000100)
#define GPIO_PIN_9 ((uint32_t)0x00000200)
#define GPIO_PIN_10 ((uint32_t)0x00000400)
#define GPIO_PIN_11 ((uint32_t)0x00000800)
#define GPIO_PIN_12 ((uint32_t)0x00001000)
#define GPIO_PIN_13 ((uint32_t)0x00002000)
#define GPIO_PIN_14 ((uint32_t)0x00004000)
#define GPIO_PIN_15 ((uint32_t)0x00008000)
#define GPIO_PIN_16 ((uint32_t)0x00010000)
#define GPIO_PIN_17 ((uint32_t)0x00020000)
```

```

#define GPIO_PIN_18 ((uint32_t)0x00040000)
#define GPIO_PIN_19 ((uint32_t)0x00080000)
#define GPIO_PIN_20 ((uint32_t)0x00100000)
#define GPIO_PIN_21 ((uint32_t)0x00200000)
#define GPIO_PIN_22 ((uint32_t)0x00400000)
#define GPIO_PIN_23 ((uint32_t)0x00800000)
#define GPIO_PIN_24 ((uint32_t)0x01000000)
#define GPIO_PIN_25 ((uint32_t)0x02000000)
#define GPIO_PIN_26 ((uint32_t)0x04000000)
#define GPIO_PIN_27 ((uint32_t)0x08000000)
#define GPIO_PIN_28 ((uint32_t)0x10000000)
#define GPIO_PIN_29 ((uint32_t)0x20000000)
#define GPIO_PIN_30 ((uint32_t)0x40000000)
#define GPIO_PIN_31 ((uint32_t)0x80000000)

```

Enumerations

```

enum GPIO_InstanceType { GPIO_A = 0U, GPIO_B, GPIO_C, GPIO_D, GPIO_E, GPIO_F, GPIO_G, GPIO_H,
GPIO_I }

```

Port instance number.

```

enum Gpio_LevelType { GPIO_LOW = 0U, GPIO_HIGH }

```

Gpio level.

```

enum Gpio_Direction { GPIO_IN = 0U, GPIO_OUT }

```

Gpio Direction.

Functions

```

void GPIO_Toggle (const GPIO_InstanceType eGpio, const uint32_t u32Pins)

```

Toggle gpio api.

```

uint32_t GPIO_ReadPins (const GPIO_InstanceType eGpio, const uint32_t u32Pins)

```

Read level of input port pins.

```

void GPIO_WritePins (const GPIO_InstanceType eGpio, const uint32_t u32Pins, const Gpio_LevelType
eOutput)

```

Write gpio level to u8Pins.

```
void GPIO_SetPinsDir (const GPIO_InstanceType eGpio, const uint32_t u32Pins, const Gpio_Direction ePindir)
```

Set Pins Direction.

GPIO API Service IDs

```
#define GPIO_Toggle_ID 0U  
#define GPIO_ReadPins_ID 1U  
#define GPIO_WritePins_ID 2U  
#define GPIO_SetPinsDir_ID 3U
```

GPIO Dev Error Code

Error Code of calling GPIO apis

```
#define GPIO_E_PARAM_INSTANCE 0x01U  
#define GPIO_E_PARAM_LEVEL 0x02U  
#define GPIO_E_PARAM_PIN 0x03U  
#define GPIO_E_PARAM_DIRECTION 0x04U
```

Detailed Description

Macro Definition Documentation

```
#define GPIO_E_PARAM_DIRECTION 0x04U
```

```
#define GPIO_E_PARAM_INSTANCE 0x01U
```

```
#define GPIO_E_PARAM_LEVEL 0x02U
```

```
#define GPIO_E_PARAM_PIN 0x03U
```

```
#define GPIO_MAX_PORT_PINS 32
```

```
#define GPIO_PIN_0 ((uint32_t)0x00000001)
```

GPIO Pin 0 select

```
#define GPIO_PIN_1 ((uint32_t)0x00000002)
    GPIO Pin 1 select

#define GPIO_PIN_10 ((uint32_t)0x00000400)
    GPIO Pin 10 select

#define GPIO_PIN_11 ((uint32_t)0x00000800)
    GPIO Pin 11 select

#define GPIO_PIN_12 ((uint32_t)0x00001000)
    GPIO Pin 12 select

#define GPIO_PIN_13 ((uint32_t)0x00002000)
    GPIO Pin 13 select

#define GPIO_PIN_14 ((uint32_t)0x00004000)
    GPIO Pin 14 select

#define GPIO_PIN_15 ((uint32_t)0x00008000)
    GPIO Pin 15 select

#define GPIO_PIN_16 ((uint32_t)0x00010000)
    GPIO Pin 16 select

#define GPIO_PIN_17 ((uint32_t)0x00020000)
    GPIO Pin 17 select

#define GPIO_PIN_18 ((uint32_t)0x00040000)
    GPIO Pin 18 select

#define GPIO_PIN_19 ((uint32_t)0x00080000)
    GPIO Pin 19 select

#define GPIO_PIN_2 ((uint32_t)0x00000004)
    GPIO Pin 2 select
```

```
#define GPIO_PIN_20 ((uint32_t)0x00100000)
    GPIO Pin 20 select

#define GPIO_PIN_21 ((uint32_t)0x00200000)
    GPIO Pin 21 select

#define GPIO_PIN_22 ((uint32_t)0x00400000)
    GPIO Pin 22 select

#define GPIO_PIN_23 ((uint32_t)0x00800000)
    GPIO Pin 23 select

#define GPIO_PIN_24 ((uint32_t)0x01000000)
    GPIO Pin 24 select

#define GPIO_PIN_25 ((uint32_t)0x02000000)
    GPIO Pin 25 select

#define GPIO_PIN_26 ((uint32_t)0x04000000)
    GPIO Pin 26 select

#define GPIO_PIN_27 ((uint32_t)0x08000000)
    GPIO Pin 27 select

#define GPIO_PIN_28 ((uint32_t)0x10000000)
    GPIO Pin 28 select

#define GPIO_PIN_29 ((uint32_t)0x20000000)
    GPIO Pin 29 select

#define GPIO_PIN_3 ((uint32_t)0x00000008)
    GPIO Pin 3 select

#define GPIO_PIN_30 ((uint32_t)0x40000000)
    GPIO Pin 30 select
```

```
#define GPIO_PIN_31 ((uint32_t)0x80000000)
```

GPIO Pin 31 select

```
#define GPIO_PIN_4 ((uint32_t)0x00000010)
```

GPIO Pin 4 select

```
#define GPIO_PIN_5 ((uint32_t)0x00000020)
```

GPIO Pin 5 select

```
#define GPIO_PIN_6 ((uint32_t)0x00000040)
```

GPIO Pin 6 select

```
#define GPIO_PIN_7 ((uint32_t)0x00000080)
```

GPIO Pin 7 select

```
#define GPIO_PIN_8 ((uint32_t)0x00000100)
```

GPIO Pin 8 select

```
#define GPIO_PIN_9 ((uint32_t)0x00000200)
```

GPIO Pin 9 select

```
#define GPIO_ReadPins_ID 1U
```

```
#define GPIO_SetPinsDir_ID 3U
```

```
#define GPIO_Toggle_ID 0U
```

```
#define GPIO_WritePins_ID 2U
```

Enumeration Type Documentation

enum Gpio_Direction

Gpio Direction.

Enumerator:

GPIO_IN	
GPIO_OUT	

enum GPIO_InstanceType

Port instance number.

Enumerator:

GPIO_A	
GPIO_B	
GPIO_C	
GPIO_D	
GPIO_E	
GPIO_F	
GPIO_G	
GPIO_H	
GPIO_I	

enum Gpio_LevelType

Gpio level.

Enumerator:

GPIO_LOW	
GPIO_HIGH	

Function Documentation

uint32_t GPIO_ReadPins (const GPIO_InstanceType *eGpio*, const uint32_t *u32Pins*)

Read level of input port pins.

Parameters

<i>eGpio</i>	eGpio GPIO Instance
<i>u32Pins</i>	The bit of u8Pins indicate the pin number of this Port.

Returns

Pins level

void GPIO_SetPinsDir (const GPIO_InstanceType *eGpio*, const uint32_t *u32Pins*, const Gpio_Direction *ePindir*)

Set Pins Direction.

Parameters

<i>eGpio</i>	eGpio GPIO Instance
<i>u32Pins</i>	The bit of u8Pins indicate the pin number of this Port.
<i>ePindir</i>	Pin direction enumeration

void GPIO_Toggle (const GPIO_InstanceType *eGpio*, const uint32_t *u32Pins*)

Toggle gpio api.

Parameters

<i>eGpio</i>	eGpio GPIO Instance
<i>u32Pins</i>	The bit of u8Pins indicate the pin number of this Port.

void GPIO_WritePins (const GPIO_InstanceType *eGpio*, const uint32_t *u32Pins*, const Gpio_LevelType *eOutput*)

Write gpio level to u8Pins.

Parameters

<i>eGpio</i>	eGpio GPIO Instance
<i>u32Pins</i>	The bit of u8Pins indicate the pin number of this Port.
<i>eOutput</i>	Output level enumeration

Module_driver_hsadc

Collaboration diagram for Module_driver_hsadc:Modules

HwA_hsadc

Classes

struct **_HSADC_HandleType**

*The structure of the CMU processing handle Implements : **_HSADC_HandleType**.*

struct **HSADC_InitType**

Defines the converter configuration.

struct **HSADC_ChannelCfgType**

The configuration option for the HSADC channel.

struct **HSADC_SeqGroupType**

The configuration option for the HSADC sequence group.

struct **HSADC_CmpType**

Defines the hardware compare configuration.

struct **HSADC_FastCmpType**

Defines the hardware fast compare configuration.

Typedefs

typedef struct **_HSADC_HandleType** **HSADC_HandleType**

*The structure of the CMU processing handle Implements : **_HSADC_HandleType**.*

Enumerations

enum **HSADC_InstanceType** { **HSADC_INSTANCE_0** = 0U, **HSADC_INSTANCE_1** = 1U, **HSADC_INSTANCE_2** = 2U, **HSADC_INSTANCE_3** = 3U }

The instance index of the HSADC peripheral.

enum **HSADC_ChannelType** { **HSADC_CHANNEL_0** = 0U, **HSADC_CHANNEL_1** = 1U, **HSADC_CHANNEL_2** = 2U, **HSADC_CHANNEL_3** = 3U, **HSADC_CHANNEL_4** = 4U, **HSADC_CHANNEL_5** = 5U, **HSADC_CHANNEL_INTERNAL_0** = 6U, **HSADC_CHANNEL_INTERNAL_1** = 7U, **HSADC_CHANNEL_EXTEND_0** = 8U, **HSADC_CHANNEL_EXTEND_1** = 9U, **HSADC_CHANNEL_EXTEND_2** = 10U, **HSADC_CHANNEL_EXTEND_3** = 11U, **HSADC_CHANNEL_EXTEND_4** = 12U, **HSADC_CHANNEL_EXTEND_5** = 13U, **HSADC_CHANNEL_EXTEND_6** = 14U, **HSADC_CHANNEL_EXTEND_7**

= 15U, HSADC_CHANNEL_OFFSET_CALIBRATION = 27U, HSADC_CHANNEL_GAIN_LOW_CALIBRATION = 28U, HSADC_CHANNEL_GAIN_HIGH_CALIBRATION = 29U }

The channel selected for HSADC conversion.

enum HSADC_SampleTimeOptionType { HSADC_SAMPLE_TIME_OPTION_0 = 0U,
HSADC_SAMPLE_TIME_OPTION_1 = 1U, HSADC_SAMPLE_TIME_OPTION_2 = 2U,
HSADC_SAMPLE_TIME_OPTION_3 = 3U }

The HSADC sample time option for selection.

enum HSADC_StatusType { HSADC_STATUS_SUCCESS = 0x0U, HSADC_STATUS_ERROR = 0x1U,
HSADC_STATUS_TIMEOUT = 0x2U }

HSADC operation return values.

Functions

void HSADC_InitStructure (HSADC_InitType *const pInitCfg)

Provide the default values of HSADC_InitType.

void HSADC_InitCmpStructure (HSADC_CmpType *const pCmpCfg)

Provide the default values of HSADC_CmpType.

void HSADC_InitFastCmpStructure (HSADC_FastCmpType *const pFastCmpCfg)

Provide the default values of HSADC_FastCmpType.

void HSADC_Init (HSADC_HandleType *pAdcHandle, const HSADC_InitType *const pInitCfg)

Initialize the HSADC instance.

void HSADC_DeInit (HSADC_HandleType *pAdcHandle)

De-initialize the HSADC instance.

void HSADC_InitChannel (HSADC_HandleType *pAdcHandle, const HSADC_ChannelCfgType aChannels[],
const uint8_t u8ChannelCnt)

Configure the HSADC sample channels.

void HSADC_InitSeqGroup (HSADC_HandleType *pAdcHandle, const HSADC_SeqGroupType
aSeqGroup[], const uint8_t u8SeqGroupCnt)

Configure the Sequence groups.

void **HSADC_InitCmp** (**HSADC_HandleType** *pAdcHandle, const **HSADC_CmpType** *const pCmpCfg)
Configure the hardware compare feature of HSADC.

void **HSADC_InitFastCmp** (**HSADC_HandleType** *pAdcHandle, const **HSADC_FastCmpType** *const pFastCmpCfg)
Configure the hardware fast compare feature of HSADC.

HSADC_StatusType **HSADC_Enable** (const **HSADC_HandleType** *pAdcHandle)
Enable the HSADC instance.

HSADC_StatusType **HSADC_Disable** (**HSADC_HandleType** *pAdcHandle)
Disable the HSADC instance.

void **HSADC_Start** (const **HSADC_HandleType** *pAdcHandle)
Start the HSADC conversion.

HSADC_StatusType **HSADC_Stop** (const **HSADC_HandleType** *pAdcHandle)
Stop the HSADC conversion.

void **HSADC_Reset** (const **HSADC_HandleType** *pAdcHandle)
Reset the HSADC hardware.

void **HSADCn_IRQHandler** (**HSADC_HandleType** *pAdcHandle)
The internal interrupt handler function for HSADC instances.

bool **HSADC_GetFastCmpResult** (const **HSADC_HandleType** *pHsadcHandle)
Get fast compare result.

bool **HSADC_GetFastCmpBoundaryFlag** (const **HSADC_HandleType** *pHsadcHandle)
Get fast compare boundary flag.

HSADC API Service IDs

#define **HSADC_INIT_STRUCTURE_ID** 0U

```

#define HSADC_INIT_ID 1U
#define HSADC_DEINIT_ID 2U
#define HSADC_INIT_CHANNEL_ID 3U
#define HSADC_INIT_CMP_STRUCTURE_ID 4U
#define HSADC_INIT_CMP_ID 5U
#define HSADC_INIT_FAST_CMP_ID 6U
#define HSADC_INIT_FAST_CMP_STRUCTURE_ID 7U
#define HSADC_INIT_SEQ_GROUP_ID 8U
#define HSADC_ENABLE_ID 9U
#define HSADC_DISABLE_ID 10U
#define HSADC_START_ID 11U
#define HSADC_STOP_ID 12U
#define HSADC_RESET_ID 13U
#define HSADC_GET_FAST_CMP_RESULT_ID 14U
#define HSADC_GET_FAST_CMP_BOUNDARY_FLAG_ID 15U

```

HSADC Dev Error Code

Error Code of calling HSADC apis

```

#define HSADC_E_PARAM_INSTANCE 0x01U
#define HSADC_E_PARAM_CHANNEL 0x02U
#define HSADC_E_PARAM_POINTER 0x03U
#define HSADC_E_PARAM_COUNT 0x04U
#define HSADC_E_PARAM_INVALID 0x05U
#define HSADC_E_PARAM_PHASE_SHIFT 0x06U

```

HSADC0 Internal Channels

Available internal HSADC channels for HSADC Instance 0

```

#define HSADC0_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC0_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

```

HSADC1 Internal channels

Available internal HSADC channels for HSADC Instance 1

```

#define HSADC1_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC1_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

```

HSADC2 Internal channels

Available internal HSADC channels for HSADC instance 2

```
#define HSADC2_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC2_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1
```

HSADC3 Internal channels

Available internal HSADC channels for HSADC instance 3

```
#define HSADC3_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC3_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1
```

Detailed Description

Macro Definition Documentation

#define HSADC0_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

#define HSADC0_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0

#define HSADC1_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

#define HSADC1_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0

#define HSADC2_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

#define HSADC2_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0

#define HSADC3_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

#define HSADC3_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0

#define HSADC_DEINIT_ID 2U

#define HSADC_DISABLE_ID 10U

#define HSADC_E_PARAM_CHANNEL 0x02U

#define HSADC_E_PARAM_COUNT 0x04U

#define HSADC_E_PARAM_INSTANCE 0x01U

#define HSADC_E_PARAM_INVALID 0x05U

#define HSADC_E_PARAM_PHASE_SHIFT 0x06U

#define HSADC_E_PARAM_POINTER 0x03U

#define HSADC_ENABLE_ID 9U

#define HSADC_GET_FAST_CMP_BOUNDARY_FLAG_ID 15U

#define HSADC_GET_FAST_CMP_RESULT_ID 14U

#define HSADC_INIT_CHANNEL_ID 3U

#define HSADC_INIT_CMP_ID 5U

```

#define HSADC_INIT_CMP_STRUCTURE_ID  4U

#define HSADC_INIT_FAST_CMP_ID  6U

#define HSADC_INIT_FAST_CMP_STRUCTURE_ID  7U

#define HSADC_INIT_ID  1U

#define HSADC_INIT_SEQ_GROUP_ID  8U

#define HSADC_INIT_STRUCTURE_ID  0U

#define HSADC_RESET_ID  13U

#define HSADC_START_ID  11U

#define HSADC_STOP_ID  12U

```

Typedef Documentation

```
typedef struct _HSADC_HandleType  HSADC_HandleType
```

The structure of the CMU processing handle Implements : **_HSADC_HandleType**.

Enumeration Type Documentation

```
enum HSADC_ChannelType
```

The channel selected for HSADC conversion.

Enumerator:

HSADC_CHANN EL_0	
HSADC_CHANN EL_1	

HSADC_CHANN EL_2	
HSADC_CHANN EL_3	
HSADC_CHANN EL_4	
HSADC_CHANN EL_5	
HSADC_CHANN EL_INTERNAL_0	
HSADC_CHANN EL_INTERNAL_1	
HSADC_CHANN EL_EXTEND_0	
HSADC_CHANN EL_EXTEND_1	
HSADC_CHANN EL_EXTEND_2	
HSADC_CHANN EL_EXTEND_3	
HSADC_CHANN EL_EXTEND_4	
HSADC_CHANN EL_EXTEND_5	
HSADC_CHANN EL_EXTEND_6	
HSADC_CHANN EL_EXTEND_7	
HSADC_CHANN EL_OFFSET_CAL IBRATION	
HSADC_CHANN EL_GAIN_LOW_ CALIBRATION	
HSADC_CHANN EL_GAIN_HIGH_ CALIBRATION	

enum HSADC_InstanceType

The instance index of the HSADC peripheral.

Enumerator:

HSADC_INSTAN CE_0	HSADC instance 0 is selected
HSADC_INSTAN CE_1	HSADC instance 1 is selected
HSADC_INSTAN CE_2	HSADC instance 2 is selected
HSADC_INSTAN CE_3	HSADC instance 3 is selected

enum HSADC_SampleTimeOptionType

The HSADC sample time option for selection.

Enumerator:

HSADC_SAMPLE _TIME_OPTION_ 0	
HSADC_SAMPLE _TIME_OPTION_ 1	
HSADC_SAMPLE _TIME_OPTION_ 2	
HSADC_SAMPLE _TIME_OPTION_ 3	

enum HSADC_StatusType

HSADC operation return values.

Enumerator:

HSADC_STATUS_SUCCESS	The HSADC operation is succeed
HSADC_STATUS_ERROR	The HSADC operation is failed
HSADC_STATUS_TIMEOUT	The HSADC operation is failed because of time out

Function Documentation

void HSADC_DeInit (HSADC_HandleType * *pAdcHandle*)

De-initialize the HSADC instance.

Restore the HSADC instance to its reset state

Parameters

<i>pAdcHandle</i>	the HSADC instance to de-init
-------------------	-------------------------------

HSADC_StatusType HSADC_Disable (HSADC_HandleType * *pAdcHandle*)

Disable the HSADC instance.

Parameters

<i>pAdcHandle</i>	the HSADC instance to disable
-------------------	-------------------------------

Returns

HSADC_StatusType whether HSADC is disabled successfully

HSADC_StatusType HSADC_Enable (const HSADC_HandleType * *pAdcHandle*)

Enable the HSADC instance.

Parameters

<i>pAdcHandle</i>	the HSADC instance to enable
-------------------	------------------------------

Returns

HSADC_StatusType whether HSADC is enabled successfully

bool HSADC_GetFastCmpBoundaryFlag (const HSADC_HandleType * *pHsadCHandle*)

Get fast compare boundary flag.

Parameters

<i>pAdcHandle</i>	the HSADC process handler
-------------------	---------------------------

bool HSADC_GetFastCmpResult (const HSADC_HandleType * *pHsadCHandle*)

Get fast compare result.

Parameters

<i>pAdcHandle</i>	the HSADC process handler
-------------------	---------------------------

void HSADC_Init (HSADC_HandleType * *pAdcHandle*, const HSADC_InitType *const *pInitCfg*)

Initialize the HSADC instance.

Parameters

<i>pAdcHandle</i>	the HSADC instance to init
<i>pInitCfg</i>	the configurations of the HSADC instance

void HSADC_InitChannel (HSADC_HandleType * *pAdcHandle*, const HSADC_ChannelCfgType *aChannels*[], const uint8_t *u8ChannelCnt*)

Configure the HSADC sample channels.

Parameters

<i>pAdcHandle</i>	the HSADC instance to use
<i>aChannels</i>	the channels to use
<i>u8ChannelCnt</i>	the quantity of channels

void HSADC_InitCmp (HSADC_HandleType * *pAdcHandle*, const HSADC_CmpType *const *pCmpCfg*)

Configure the hardware compare feature of HSADC.

Parameters

<i>pAdcHandle</i>	the HSADC instance to use
<i>pCmpCfg</i>	the compare paremeters

void HSADC_InitCmpStructure (HSADC_CmpType *const *pCmpCfg*)

Provide the default values of **HSADC_CmpType**.

Parameters

<i>pCmpCfg</i>	the structure to initialize
----------------	-----------------------------

void HSADC_InitFastCmp (HSADC_HandleType * *pAdcHandle*, const HSADC_FastCmpType *const *pFastCmpCfg*)

Configure the hardware fast compare feature of HSADC.

Parameters

<i>pAdcHandle</i>	the HSADC instance to use
<i>pFastCompareCfg</i>	the fast compare paremeters

Note

Fast compare function can only work in continuous conversion mode or single conversion mode with periodic trigger and on a fixed channel to do continuous compare on a same channel. SEQ_LEN bits in CFG1 register must be configured as 0 when using fast compare function.

void HSADC_InitFastCmpStructure (HSADC_FastCmpType *const *pFastCmpCfg*)

Provide the default values of **HSADC_FastCmpType**.

Parameters

<i>pFastCmpCfg</i>	the structure to initialize
--------------------	-----------------------------

void HSADC_InitSeqGroup (HSADC_HandleType * *pAdcHandle*, const HSADC_SeqGroupType *aSeqGroup*[], const uint8_t *u8SeqGroupCnt*)

Configure the Sequence groups.

Parameters

<i>pAdcHandle</i>	the HSADC instance to use
<i>aSeqGroup</i>	the sequence groups to use
<i>u8SeqGroupCnt</i>	the quantity of sequence groups

void HSADC_InitStructure (HSADC_InitType *const *pInitCfg*)

Provide the default values of **HSADC_InitType**.

Parameters

<i>pInitCfg</i>	the structure to initialize
-----------------	-----------------------------

void HSADC_Reset (const HSADC_HandleType * *pAdcHandle*)

Reset the HSADC hardware.

Parameters

<i>pAdcHandle</i>	the HSADC instance to reset
-------------------	-----------------------------

void HSADC_Start (const HSADC_HandleType * *pAdcHandle*)

Start the HSADC conversion.

If the HSADC sequence mode is single or continuous, and the trigger mode is HSADC_TRIGMODE_SW, the adc conversion will start immediately. Otherwise, the HSADC will wait for the trigger signal to start the conversion

Parameters

<i>pAdcHandle</i>	the HSADC instance to start
-------------------	-----------------------------

HSADC_StatusType HSADC_Stop (const HSADC_HandleType * *pAdcHandle*)

Stop the HSADC conversion.

If the HSADC sequence mode is single, it will stop the ongoing conversion. If no ongoing conversion, it will have no effect. If the HSADC sequence mode is continuous or discontinuous, it will stop the ongoing conversion and meanwhile the further conversions.

Parameters

<i>pAdcHandle</i>	the HSADC instance to stop
-------------------	----------------------------

Returns

HSADC_StatusType whether HSADC is stopped successfully

void HSADCn_IRQHandler (HSADC_HandleType * *pAdcHandle*)

The internal interrupt handler function for HSADC instances.

Parameters

<i>pAdcHandle</i>	the HSADC process handler
-------------------	---------------------------

Fc7xxx_driver_hsm

Classes

struct **HSM_CmdType**

HSM cmd type definition.

struct **HSMCom_TrueRandType**

the true random information passed to HSM core by mailbox

struct **HSMCom_TrueRandExType**

the true random extend information passed to HSM core by mailbox

struct **HSM_TrngInitCfgType**

struct **HSMCom_FastTrueRandType**

struct **HSM_SmsCfgType**

SHA/MD5/SM3 context information used by driver when call specific API.

struct **HSM_ShaResultBufType**

type definition for driver to store information

struct **HSM_ShaType**

SHA information used by driver when call specific API.

struct **HSMCom_ShaType**

SHA information used by driver when call specific API.

struct **HSM_ShaExType**

SHA extend information used by driver when call specific API.

struct **HSMCom_ShaExType**

SHA extend information used by driver when call specific API.

struct **HSM_Sha2Type**

SHA information used by driver when call specific API.

struct **HSMCom_Sha2Type**

SHA information used by driver when call specific API.

struct **HSM_ScatterHashType**

Scatter Hash information used by driver when call specific API.

struct **HSMCom_ScatterHashType**

Scatter Hash information used by driver when call specific API.

struct **HSM_DrvEccCurveParamType**

ECC curve information used by driver when call specific API.

struct **HSM_EccSignType**

ECC sign information used by driver when call specific API.

struct **HSMCom_EccSignType**

ECC sign information used by driver when call specific API.

struct **HSM_EccVerifyType**

ECC verify information used by driver when call specific API.

struct **HSMCom_EccVerifyType**

ECC verify information used by driver when call specific API.

struct **HSM_Sm2GenKeyPairType**

SM2 key pair generation information used by driver when call specific API.

struct **HSMCom_Sm2GenKeyPairType**

SM2 key pair generation information used by driver when call specific API.

struct **HSM_Sm2EncryptType**

SM2 encrypt information used by driver when call specific API.

struct **HSMCom_Sm2EncryptType**

SM2 encrypt information used by driver when call specific API.

struct **HSM_Sm2DecryptType**

SM2 decrypt information used by driver when call specific API.

struct **HSMCom_Sm2DecryptType**

SM2 decrypt information used by driver when call specific API.

struct **HSM_Sm2SignType**

SM2 sign information used by driver when call specific API.

struct **HSMCom_Sm2SignType**

SM2 sign information used by driver when call specific API.

struct **HSM_Sm2VerifyType**

SM2 verify information used by driver when call specific API.

struct **HSMCom_Sm2VerifyType**

SM2 verify information used by driver when call specific API.

struct **HSM_Sm2GenZaType**

SM2 ZA generation information used by driver when call specific API.

struct **HSMCom_Sm2GenZaType**

SM2 ZA generation information used by driver when call specific API.

struct **HSM_Sm2GenHashType**

SM2 HASH generation information used by driver when call specific API.

struct **HSMCom_Sm2GenHashType**

SM2 HASH generation information used by driver when call specific API.

struct **HSM_EccEasyEncryType**

ECC encrypt information used by driver when call specific API.

struct **HSMCom_EccEasyEncryType**

ECC encrypt information used by driver when call specific API.

struct **HSM_EccEasyDecryType**

ECC decrypt information used by driver when call specific API.

struct **HSMCom_EccEasyDecryType**

ECC decrypt information used by driver when call specific API.

struct **HSM_EccCalcType**

ECC point calculate used by driver when call specific API.

struct **HSMCom_EccCalcType**

ECC point calculate used by driver when call specific API.

struct **HSMCom_RequestAuthType**

Request Auth used by driver when call specific API.

struct **HSMCom_LifeCycleChangeType**

Change lifecycle used by driver when call specific API.

struct **HSMCom_CancelJobType**

Cancel job used by driver when call specific API.

struct **HSMCom_SelfTestType**

struct **HSMCom_NvrOtpType**

struct **HSM_KeyManagerExportUserKeyType**

struct **HSM_KeyManagerImportUserKeyType**

struct **HSM_Ln_ImportUserKeyExtType**

struct **HSM_Ln_CopyUserKeyType**

struct **HSM_KeyManagerGenUserKeyType**

struct **HSMCom_GenUserKeyType**

struct **HSM_KeyManagerCheckUserKeyStatusExtType**

struct **HSMCom_UserKeyManageType**

struct **HSM_Ln_KeySpaceStatusType**

struct **HSMCom_KeySpaceStatusType**

struct **HSM_AesNewKeyInfType**

key information used by AES when use one key in software

struct **HSM_AesVendorKeyInfType**

key information used by AES when use one key in nvr flash

struct **HSM_AesEnDecryptType**

AES encrypt/decrypt information used by driver when call specific API.

struct **HSM_FlexAesEnDecryptType**

Flex AES encrypt/decrypt information used by driver when call specific API.

struct **HSMCom_AesEncryptType**

struct **HSMCom_AesDecryptType**

struct **HSMCom_FlexAesEncryptType**

struct **HSMCom_FlexAesDecryptType**

struct **HSM_Sm4EnDecryptType**

SM4 encrypt/decrypt information used by driver when call specific API.

struct **HSMCom_Sm4EnDecryptType**

struct **HSMWrap_Sm4EncryptType**

struct **HSMWrap_Sm4EncryptExType**

struct **HSM_XCMacType**

CMAC/XMAC information used by driver when call specific API.

struct **HSMCom_CMacType**

struct **HSM_XCMacExType**

struct **HSMCom_CMacExType**

struct **HSMCom_XMacType**

struct **HSM_CcmEncryptType**

CCM Encrypt information used by driver when call specific API.

struct **HSMCom_CcmEncryptType**

struct **HSM_CcmDecryptType**

CCM Decrypt information used by driver when call specific API.

struct **HSMCom_CcmDecryptType**

struct **HSM_GcmEncryptType**

GCM Encrypt information used by driver when call specific API.

struct **HSMCom_GcmEncryptType**

struct **HSM_GcmDecryptType**

GCM Decrypt information used by driver when call specific API.

struct **HSMCom_GcmDecryptType**

struct **HSM_Md5Type**

MD5 information used by driver when call specific API.

struct **HSMCom_Md5Type**

struct **HSM_Md5ExType**

MD5 information used by driver when call specific API.

struct **HSMCom_Md5ExType**

struct **HSM_Sm3Type**

SM3 information used by driver when call specific API.

struct **HSMCom_Sm3Type**

struct **HSM_Sm3ExType**

struct **HSMCom_Sm3ExType**

struct **HSMCom_MonCountType**

struct **HSM_RsaType**

RSA information used by driver when call specific API.

struct **HSMCom_RsaType**

struct **HSM_BigNumberCalcType**

RSA information used by driver when call specific API.

struct **HSMCom_BigNumberCalcType**

struct **HSM_BigNumberCalcExtType**

struct **HSMCom_BigNumberCalcExtType**

struct **HSMCom_LoadFirmwareType**

struct **HSMCom_AuthCheckType**

struct **HSM_AESM_CFG_KEY_CFG_T**

struct **HSM_AESM_ENTRY_CFG_T**

struct **HSM_AesmRawApiType**

struct **HSMCom_AesmRawApiType**

struct **HSM_RsaSsaPssVerifyType**

RSA PSS signature verify information used by driver when call specific API.

struct **HSMCom_RsaSsaPssVerifyType**

struct **HSM_RsaSsaPkcs1V15VerifyType**

RSA PKCS1V1.5 signature verify information used by driver when call specific API.

struct **HSMCom_RsaSsaPkcs1V15VerifyType**

struct **HSM_RsaSsaPssSignType**

struct **HSMCom_RsaSsaPssSignType**

struct **HSM_RsaSsaPkcs1V15SignType**

RSA PKCS1 V1.5 signature information used by driver when call specific API.

struct **HSMCom_RsaSsaPkcs1v15SignType**

struct **HSMCom_GetRomHashType**

Macros

#define **HSM_ECC_CALC_PADD** 0xFC73F801U

#define **HSM_ECC_CALC_PDBL** 0xFC73F802U

```

#define HSM_ECC_CALC_PMUL 0xFC73F803U
#define HSM_ECC_CALC_PCHK 0xFC73F804U
#define HSM_COM_REQUEST_AUTH_DATA_U32_CNT 8
#define HSMCOM_NVR_OTP_START_ADDR 0x04400400
#define HSMCOM_NVR_OTP_END_ADDR 0x04401fff
#define KEYMANAGER_GEN_KEYPAIR_SM2 (0u)
#define KEYMANAGER_GEN_KEYPAIR_ECC (1u)
#define KEYMANAGER_GEN_KEYPAIR_MAX (2u)
#define KEYMANAGER_USERKEY_STATUS_NOT_EXIST (0u)
#define KEYMANAGER_USERKEY_STATUS_EXIST_VALID (1u)
#define KEYMANAGER_USERKEY_STATUS_EXIST_INVALID (2u)
#define KEYMANAGER_USERKEY_STATUS_ERROR (3u)
#define HSM_FLEXAES_PAD_ZERO 0u

```

Flexible AES zero padding type for type "HSM_FlexAesPadType" if input data is not aligned with 16bytes, this configuration will make append byte data 0 to keep 16bytes align.

```

#define HSM_FLEXAES_PAD_PKCS7 1u

```

Flexible AES PKCS7 padding type for type "HSM_FlexAesPadType" if input data is not aligned with 16bytes, this configuration will make append bytes by PKCS7 method to keep 16bytes align.

```

#define HSM_XCMAC_ENGINE_SM4 1U
#define HSM_XCMAC_ENGINE_AES 0U
#define HSM_BIGNUMBER_CALC_A_ADD_B 0xFC730002
#define HSM_BIGNUMBER_CALC_A_SUB_B 0xFC730003
#define HSM_BIGNUMBER_CALC_B_SUB_A 0xFC730004
#define HSM_BIGNUMBER_CALC_A_MUL_B 0xFC730005
#define HSM_BIGNUMBER_CALC_A_EXP_E 0xFC730006
#define HSM_BIGNUMBER_CALC_A_RED_N 0xFC730007
#define HSM_BIGNUMBER_CALC_A_INV 0xFC730008
#define HSM_BN_VALID_FLAG_E 0x1u
#define HSM_BN_VALID_FLAG_B 0x2u
#define HSM_RSA_SSA_INPUT_RAW_MESSAGE 0u

```

RSA SSA input data is the raw message for HSM_RsaSsaInputType.

```

#define HSM_RSA_SSA_INPUT_HASH_DATA 1u

```

RSA SSA input data is the hash value of raw message for HSM_RsaSsaInputType.

#define HSM_HASH_MD5 (0U)

MD5 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_160 (1U)

SHA160 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_224 (2U)

SHA224 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_256 (3U)

SHA256 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_384 (4U)

SHA384 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512 (5U)

SHA512 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512_224 (6U)

SHA512/224 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512_256 (7U)

SHA512/256 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SM3 (8U)

SM3 algorithm value for type "HSM_HashAlgType".

Typedefs

typedef HSM_SmsCfgType HSM_ShaCtxType

SHA context information used by driver when call specific API.

typedef hsm_uint32_t HSM_KeyManagerGenKeyPairType

typedef hsm_uint32_t KeyManager_UserKeyStatusType

typedef hsm_uint32_t HSM_FlexAesPadType

Flexiable AES padding type definition if use zero padding, refer to HSM_FLEXAES_PAD_ZERO, after decrypt, user should remove the zero manually if use PKCS7 padding, refer to HSM_FLEXAES_PAD_PKCS7.

typedef HSM_AesEnDecryptType HSM_AesEncryptType

AES decrypt information used by driver when call specific API.

typedef HSM_AesEnDecryptType HSM_AesDecryptType

AES encrypt information used by driver when call specific API.

typedef HSM_FlexAesEnDecryptType HSM_FlexAesEncryptType

Flex AES decrypt information used by driver when call specific API.

typedef HSM_FlexAesEnDecryptType HSM_FlexAesDecryptType

AES encrypt information used by driver when call specific API.

typedef HSMCom_Sm4EnDecryptType HSMCom_Sm4DecryptType

typedef HSMCom_Sm4EnDecryptType HSMCom_Sm4EncryptType

typedef HSMCom_FlexAesEncryptType HSMCom_Sm4EncryptExType

typedef HSMCom_FlexAesDecryptType HSMCom_Sm4DecryptExType

typedef HSM_XCMacType HSM_CMacType

CMAC information used by driver when call specific API.

typedef hsm_uint32_t HSM_XCMacEngineType

typedef HSM_XCMacExType HSM_CMacExType

typedef HSM_XCMacType HSM_XMacType

XMAC information used by driver when call specific API.

typedef HSM_SmsCfgType HSM_Md5CtxType

MD5 context information used by driver when call specific API.

typedef HSM_SmsCfgType HSM_Sm3CtxType

SM3 context information used by driver when call specific API.

typedef **hsm_uint32_t** **HSM_RsaSsaInputType**

type definition for rsa ssa input data type refer to HSM_RSA_SSA_INPUT_RAW_MESSAGE, HSM_RSA_SSA_INPUT_HASH_DATA

typedef **hsm_uint32_t** **HSM_HashAlgType**

type definition for SHA algorithm refer to HSM_SHA_160, HSM_SHA_224, HSM_SHA_256, HSM_SHA_384, HSM_SHA_512, HSM_SHA_512_224, HSM_SHA_512_256, HSM_HASH_MD5, HSM_HASH_SM3

Enumerations

enum **HSM_StatusType** { **HSM_STATUS_SUCCESS** = 0U, **HSM_STATUS_BUSY**,
HSM_STATUS_PARAM_ERR, **HSM_STATUS_ERROR**, **HSM_STATUS_ECC_ERROR**, **HSM_STATUS_IDLE**,
HSM_STATUS_ERROR_STOP, **HSM_STATUS_FINISH**, **HSM_STATUS_PASS**, **HSM_STATUS_FAIL**,
HSM_STATUS_AGAIN, **HSM_STATUS_LOGIC_ERR**, **HSM_STATUS_HW_ERR**, **HSM_STATUS_HW_IDLE**,
HSM_STATUS_ALREADY_FINISHED, **HSM_STATUS_SEND_FAIL**, **HSM_STATUS_BUFFER_SIZE_ERR**,
HSM_STATUS_TIMEOUT_ERR, **HSM_STATUS_NOT_EXIST**, **HSM_STATUS_EXIST**, **HSM_STATUS_ZOMBIE**,
HSM_STATUS_SPACE_FULL }

Status returned by HSM SDK CM7 side APIs.

definitions for HSM_BoolType

typedef **hsm_uint32_t** **HSM_BoolType**

Boolean type for HSM.

#define **HSM_FALSE** (0u)

Boolean false value definition for type HSM_BoolType used by HSM.

#define **HSM_TRUE** (1u)

Boolean true value definition for type HSM_BoolType used by HSM.

definitions for HSM_MailboxApiRetType

typedef **hsm_uint32_t** **HSM_MailboxApiRetType**

status type definition returned by HSM subsystem APIs

#define MAILBOXAPI_RET_SUCCESS (0u)
success status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_ERROR (1u)
error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_FAIL (2u)
failure status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_PASS (3u)
verify pass status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_NOT_SUPPORT (4u)
function not support status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_HW_ERROR (5u)
hardware error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_NO_PRIVILEGES (6u)
have no permission status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_PARAMETER_ERROR (7u)
parameters error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_AUTHORITY_ERROR (8u)
authority error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_BUSY (9u)
hardware busy status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_ECC_ERR (10u)
flash ecc error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_TIMEOUT** (11u)

time expire error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_NONE** (0xFFFFFFFFFu)

Type definition for HSM Isr/Poll mode

typedef **hsm_uint32_t** **HSM_RndSrcType**

typedef **hsm_uint32_t** **HSM_FastTrngSrcType**

typedef **hsm_uint32_t** **HSM_TriggerSrcType**

type definition for hsm state machine trigger source used by cortex side hsm driver

typedef **hsm_uint32_t** **HSM_DataFormatType**

HSM data format refer to HSM_DATA_FORMAT_4B, HSM_DATA_FORMAT_1B.

#define **HSM_TRNG_RAND_U32_CNT** 16

the true random count return by the API

#define **HSM_TRNG_RAND_U32_CNT** 16

the true random count return by the API

#define **HSMCOM_TRNG_SRC_0** 0x40cau

#define **HSMCOM_TRNG_SRC_1** 0xc51cu

#define **HSMCOM_TRNG_SRC_ANY** 0x359bu

#define **HSMCOM_TRNG_SRC_XOR** 0x80a5u

#define **HSMCOM_TRNG_SRC_SHIFTXOR** 0x648Cu

#define **HSMCOM_FASTTRNG_SRC_NORMAL** 0x0u

#define **HSMCOM_FASTTRNG_SRC_XOR** 0x1u

#define **HSM_TRIGGER_SRC_USER_POLL** (0U)

Poll mode value for type "HSM_TriggerSrcType".

#define **HSM_TRIGGER_SRC_ISR** (1U)

Isr mode value for type "HSM_TriggerSrcType".

#define **HSM_DATA_FORMAT_4B** 0u

HSM 4bytes unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, use this format, should be divided as following. 0x00010203, 0x04050607, 0x08090a0b, 0x0c0d0e0f.

#define HSM_DATA_FORMAT_1B 1u

HSM 1byte unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, use this format, should be divided as following. 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f.

#define HSM_DATA_FORMAT_1B_PAD_0_LEFT_FIT_BUF 3u

HSM 1byte unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, the buffer size is 18bytes, use this format, should be divided as following. 0x00(pad), 0x00(pad), 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f.

definitions for HSM_ShaAlgType

typedef hsm_uint32_t HSM_ShaAlgType

<i>type</i>	<i>definition</i>	<i>for</i>	<i>SHA</i>	<i>algorithm</i>	<i>refer</i>	<i>to</i>
						<i>HSM_SHA_160, HSM_SHA_224, HSM_SHA_256, HSM_SHA_384, HSM_SHA_512, HSM_SHA_512_224, HSM_SHA_512_256</i>

#define HSM_SHA_160 (1U)

SHA160 algorithm value for type "HSM_ShaAlgType".

#define HSM_SHA_224 (2U)

SHA224 algorithm value for type "HSM_ShaAlgType".

#define HSM_SHA_256 (3U)

SHA256 algorithm value for type "HSM_ShaAlgType".

#define HSM_SHA_384 (4U)

SHA384 algorithm value for type "HSM_ShaAlgType".

#define HSM_SHA_512 (5U)

SHA512 algorithm value for type "HSM_ShaAlgType".

#define HSM_SHA_512_224 (6U)

SHA512/224 algorithm value for type "HSM_ShaAlgType".

#define HSM_SHA_512_256 (7U)

SHA512/256 algorithm value for type "HSM_ShaAlgType".

definitions for HSM_HfamMacType

typedef hsm_uint32_t HSM_HfamMacType

*SM3/MD5/SHA can generate MAC, use this to select the type of MAC SMAC refer to HSM_HFAM_MAC_SMAC
HMAC refer to HSM_HFAM_MAC_HMAC.*

#define HSM_HFAM_MAC_SMAC (0U)

SMAC mode value for type "HSM_HfamMacType".

#define HSM_HFAM_MAC_HMAC (1U)

HMAC mode value for type "HSM_HfamMacType".

definitions for HSM_DrvEccCurvePrmIndexType

typedef hsm_uint32_t HSM_DrvEccCurvePrmIndexType

type definition for ECC curve data that defined by NIST

#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP224R1 (4U)

ECC curve NIST SECP224R1 value for type "HSM_DrvEccCurvePrmIndexType".

#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP256R1 (1U)

ECC curve NIST SECP256R1 value for type "HSM_DrvEccCurvePrmIndexType".

#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP384R1 (2U)

ECC curve NIST SECP384R1 value for type "HSM_DrvEccCurvePrmIndexType".

```
#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP521R1 (3U)
ECC curve NIST SECP521R1 value for type "HSM_DrvEccCurvePrmIndexType".

#define HSM_DRV_ECC_CURVE_PRM_IDX_MAX (0xFFFFFFFFU)
max count of ECC curve supported for type "HSM_DrvEccCurvePrmIndexType"
```

definitions for HSM_Sm4KeyType

```
typedef hsm_uint32_t HSM_Sm4KeyType
#define HSM_SM4_KEY_CHIP_VENDOR_IFR (0U) /* key has existed in Flash IFR, it provided by chip, not
changeable */
vendor key value for type "HSM_Sm4KeyType"

#define HSM_SM4_KEY_NEW (1U) /* user should provide the new key */
driver user's new key value for type "HSM_Sm4KeyType"
```

definitions for HSM_SymmEnDecryptAlgType

```
typedef hsm_uint32_t HSM_SymmEnDecryptAlgType
type definition for SM4/AES encrypt/decrypt mode

#define HSM_SYMM_ENDECRYPT_MODE_CTR (0U)
CTR mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_CBC (1U)
CBC mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_ECB (2U)
ECB mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_CFB (3U)
CFB mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_OFB (4U)
```

OFB mode value for type "HSM_SymmEnDecryptAlgType".

definitions for HSMCom_FirmwareLoadStatus

typedef **hsm_uint32_t** HSMCom_FirmwareLoadStatus

type definition for firmware load status

#define **HSMCOM_FIRMWARE_LOAD_NONE** 0U

no firmware detected in flash status

#define **HSMCOM_FIRMWARE_LOADING** 1U

status value represents firmware is loading now

#define **HSMCOM_FIRMWARE_NEED_USER_LOAD** 2U

status value represents firmware waiting user to trigger load in CM7 side

#define **HSMCOM_FIRMWARE_LOAD_OK** 3U

status value represents firmware is loaded successfully

#define **HSMCOM_FIRMWARE_LOAD_FAIL** 4U

status value represents firmware load fail

definitions for HSM_KeyManagerBoolType

typedef **hsm_uint32_t** HSM_KeyManagerBoolType

#define **KEYMANAGER_FALSE** (0u)

#define **KEYMANAGER_TRUE** (1u)

definitions for HSM_KeyManagerKeyExportType

typedef **hsm_uint32_t** HSM_KeyManagerKeyExportType

#define **KEYMANAGER_KEY_EXPORT_FORBID** (0u)

#define **KEYMANAGER_KEY_EXPORT_PLAIN** (1u)

#define **KEYMANAGER_KEY_EXPORT_CIPHER** (2u)

#define **KEYMANAGER_KEY_EXPORT_MAX** (3u)

definitions for HSM_KeyManagerUserKeyEnDecryType

```
typedef hsm_uint32_t HSM_KeyManagerUserKeyEnDecryType
#define KEYMANAGER_ENDECRY_NONE (0u)/* For plain key, no need to encrypt or decrypt. */
#define KEYMANAGER_ENDECRY_AES256 (1u)
#define KEYMANAGER_ENDECRY_SM4 (2u)
#define KEYMANAGER_ENDECRY_MAX (3u)
```

definitions for HSM_KeyManagerUserKeyTypeType

```
typedef hsm_uint32_t HSM_KeyManagerUserKeyTypeType
#define KEYMANAGER_USER_KEY_TYPE_NONE (0u) /* not configure the key type, HSM shall not check if
the key type is matched with de/encrypt ALG. */
#define KEYMANAGER_USER_KEY_TYPE_AES (1u)
#define KEYMANAGER_USER_KEY_TYPE_SM4 (1u)
#define KEYMANAGER_USER_KEY_TYPE_ECC_PRIVATE (3u)
#define KEYMANAGER_USER_KEY_TYPE_ECC_PUBLIC (4u)
#define KEYMANAGER_USER_KEY_TYPE_SM2_PRIVATE (5u)
#define KEYMANAGER_USER_KEY_TYPE_SM2_PUBLIC (6u)
#define KEYMANAGER_USER_KEY_TYPE_RSA_E (7u)
#define KEYMANAGER_USER_KEY_TYPE_RSA_N (8u)
#define KEYMANAGER_USER_KEY_TYPE_XMAC (9u)
#define KEYMANAGER_USER_KEY_TYPE_CMAC_KEY (10u)
#define KEYMANAGER_USER_KEY_TYPE_CCM (11u)
#define KEYMANAGER_USER_KEY_TYPE_GCM (12u)
#define KEYMANAGER_USER_KEY_TYPE_ECC_SIGNATURE (13u)
#define KEYMANAGER_USER_KEY_TYPE_SM2_SIGNATURE (14u)
#define KEYMANAGER_USER_KEY_TYPE_IV (15u)
#define KEYMANAGER_USER_KEY_TYPE_CMAC_RESULT (16u)
#define KEYMANAGER_USER_KEY_TYPE_XMAC_RESULT (17u)
#define KEYMANAGER_USER_KEY_TYPE_RSA_SIGNATURE (18u)
#define KEYMANAGER_USER_KEY_TYPE_ED25519_PUBKEY (19u)
#define KEYMANAGER_USER_KEY_TYPE_ED25519_SIGNATURE (20u)
#define KEYMANAGER_USER_KEY_TYPE_ED25519_PRIVATE (21u)
#define KEYMANAGER_USER_KEY_TYPE_PBKDF2_PWD (22u)
#define KEYMANAGER_USER_KEY_TYPE_HASHMAC_KEY (23u)
#define KEYMANAGER_USER_KEY_TYPE_RSA (24u)
```

definitions for HSM_KeyManagerEnDecryAlgType

```
typedef hsm_uint32_t HSM_KeyManagerEnDecryAlgType
```

```
#define KEYMANAGER_ENDECRY_ALG_CTR (0u)
#define KEYMANAGER_ENDECRY_ALG_CBC (1u)
#define KEYMANAGER_ENDECRY_ALG_ECB (2u)
#define KEYMANAGER_ENDECRY_ALG_CFB (3u)
#define KEYMANAGER_ENDECRY_ALG_OFB (4u)
#define KEYMANAGER_ENDECRY_ALG_MAX (5u)
```

definitions for HSM_AesKeyType

```
typedef hsm_uint32_t HSM_AesKeyType
```

type definition for the key source

```
#define HSM_AES_KEY_CHIP_VENDOR_IFR (0U) /* key has existed in Flash IFR, it provided by chip
hardware, not changeable */
```

vendor key value for type "HSM_AesKeyType"

```
#define HSM_AES_KEY_NEW (1U) /* user should provide the new key */
```

driver user's new key value for type "HSM_AesKeyType"

definitions for HSM_AesKeyBitCntType

```
typedef hsm_uint32_t HSM_AesKeyBitCntType
```

type definition for AES key

```
#define HSM_AES_KEY128 (0U)
```

AES128 key type value for type "HSM_AesKeyBitCntType".

```
#define HSM_AES_KEY192 (1U)
```

AES192 key type value for type "HSM_AesKeyBitCntType".

```
#define HSM_AES_KEY256 (2U)
```

AES256 key type value for type "HSM_AesKeyBitCntType".

Detailed Description

Macro Definition Documentation

#define HSM_AES_KEY128 (0U)

AES128 key type value for type "HSM_AesKeyBitCntType".

#define HSM_AES_KEY192 (1U)

AES192 key type value for type "HSM_AesKeyBitCntType".

#define HSM_AES_KEY256 (2U)

AES256 key type value for type "HSM_AesKeyBitCntType".

#define HSM_AES_KEY_CHIP_VENDOR_IFR (0U) /* key has existed in Flash IFR, it provided by chip hardware, not changeable */

vendor key value for type "HSM_AesKeyType"

HSM hardware support accessing the aes key stored in nvr flash directly, chip user should program the key in flash already. This way don't need software read data and write it to some place, hardware will read the data itself automatically.

#define HSM_AES_KEY_NEW (1U) /* user should provide the new key */

driver user's new key value for type "HSM_AesKeyType"

HSM use the key passed by driver API user.

#define HSM_BIGNUMBER_CALC_A_ADD_B 0xFC730002

#define HSM_BIGNUMBER_CALC_A_EXP_E 0xFC730006

#define HSM_BIGNUMBER_CALC_A_INV 0xFC730008

#define HSM_BIGNUMBER_CALC_A_MUL_B 0xFC730005

#define HSM_BIGNUMBER_CALC_A_RED_N 0xFC730007

#define HSM_BIGNUMBER_CALC_A_SUB_B 0xFC730003

#define HSM_BIGNUMBER_CALC_B_SUB_A 0xFC730004

#define HSM_BN_VALID_FLAG_B 0x2u

#define HSM_BN_VALID_FLAG_E 0x1u

#define HSM_COM_REQUEST_AUTH_DATA_U32_CNT 8

#define HSM_DATA_FORMAT_1B 1u

HSM 1byte unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, use this format, should be divided as following. 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f.

#define HSM_DATA_FORMAT_1B_PAD_0_LEFT_FIT_BUF 3u

HSM 1byte unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, the buffer size is 18bytes, use this format, should be divided as following. 0x00(pad), 0x00(pad), 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f.

#define HSM_DATA_FORMAT_4B 0u

HSM 4bytes unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, use this format, should be divided as following. 0x00010203, 0x04050607, 0x08090a0b, 0x0c0d0e0f.

#define HSM_DRV_ECC_CURVE_PRM_IDX_MAX (0xFFFFFFFFU)

max count of ECC curve supported for type "HSM_DrvEccCurvePrmIndexType"

#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP224R1 (4U)

ECC curve NIST SECP224R1 value for type "HSM_DrvEccCurvePrmIndexType".

#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP256R1 (1U)

ECC curve NIST SECP256R1 value for type "HSM_DrvEccCurvePrmIndexType".

#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP384R1 (2U)

ECC curve NIST SECP384R1 value for type "HSM_DrvEccCurvePrmIndexType".

#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP521R1 (3U)

ECC curve NIST SECP521R1 value for type "HSM_DrvEccCurvePrmIndexType".

#define HSM_ECC_CALC_PADD 0xFC73F801U

#define HSM_ECC_CALC_PCHK 0xFC73F804U

#define HSM_ECC_CALC_PDBL 0xFC73F802U

#define HSM_ECC_CALC_PMUL 0xFC73F803U

#define HSM_FALSE (0u)

Boolean false value definition for type HSM_BoolType used by HSM.

#define HSM_FLEXAES_PAD_PKCS7 1u

Flexiable AES PKCS7 padding type for type "HSM_FlexAesPadType" if input data is not aligned with 16bytes, this configuration will make append bytes by PKCS7 method to keep 16bytes align.

#define HSM_FLEXAES_PAD_ZERO 0u

Flexiable AES zero padding type for type "HSM_FlexAesPadType" if input data is not aligned with 16bytes, this configuration will make append byte data 0 to keep 16bytes align.

#define HSM_HASH_MD5 (0U)

MD5 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_160 (1U)

SHA160 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_224 (2U)

SHA224 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_256 (3U)

SHA256 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_384 (4U)

SHA384 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512 (5U)

SHA512 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512_224 (6U)

SHA512/224 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512_256 (7U)

SHA512/256 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SM3 (8U)

SM3 algorithm value for type "HSM_HashAlgType".

#define HSM_HFAM_MAC_HMAC (1U)

HMAC mode value for type "HSM_HfamMacType".

#define HSM_HFAM_MAC_SMAC (0U)

SMAC mode value for type "HSM_HfamMacType".

#define HSM_RSA_SSA_INPUT_HASH_DATA 1u

RSA SSA input data is the hash value of raw message for HSM_RsaSsaInputType.

```
#define HSM_RSA_SSA_INPUT_RAW_MESSAGE 0u
```

RSA SSA input data is the raw message for HSM_RsaSsaInputType.

```
#define HSM_SHA_160 (1U)
```

SHA160 algorithm value for type "HSM_ShaAlgType".

Note

it's the so-called SHA1

```
#define HSM_SHA_224 (2U)
```

SHA224 algorithm value for type "HSM_ShaAlgType".

Note

it's the so-called SHA2-224

```
#define HSM_SHA_256 (3U)
```

SHA256 algorithm value for type "HSM_ShaAlgType".

Note

it's the so-called SHA2-256

```
#define HSM_SHA_384 (4U)
```

SHA384 algorithm value for type "HSM_ShaAlgType".

Note

it's the so-called SHA2-384

```
#define HSM_SHA_512 (5U)
```

SHA512 algorithm value for type "HSM_ShaAlgType".

Note

it's the so-called SHA2-512

```
#define HSM_SHA_512_224 (6U)
```

SHA512/224 algorithm value for type "HSM_ShaAlgType".

Note

it's the so-called SHA2-512/224

```
#define HSM_SHA_512_256 (7U)
```

SHA512/256 algorithm value for type "HSM_ShaAlgType".

Note

it's the so-called SHA2-512/256

```
#define HSM_SM4_KEY_CHIP_VENDOR_IFR (0U) /* key has existed in Flash IFR, it provided by chip, not  
changeable */
```

vendor key value for type "HSM_Sm4KeyType"

HSM hardware support accessing the SM4 key stored in nvr flash directly, chip user should program the key in flash already. This way don't need software read data and write it to some place, hardware will read the data itself automatically.

```
#define HSM_SM4_KEY_NEW (1U) /* user should provide the new key */
```

driver user's new key value for type "HSM_Sm4KeyType"

HSM use the key passed by driver API user.

#define HSM_SYMM_ENDECRYPT_MODE_CBC (1U)

CBC mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_CFB (3U)

CFB mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_CTR (0U)

CTR mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_ECB (2U)

ECB mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_SYMM_ENDECRYPT_MODE_OFB (4U)

OFB mode value for type "HSM_SymmEnDecryptAlgType".

#define HSM_TRIGGER_SRC_ISR (1U)

Isr mode value for type "HSM_TriggerSrcType".

HSM driver can work in two mode: Isr mode and poll mode. In Isr mode, Hsm interrupt is enabled, driver API user only call the specific API to start the process. If the algorithm need more than one

step in driver, the isr will trigger the next process step in driver internal. After all steps finished, and the result will be passed to the callback registered in the init API called by driver user.

#define HSM_TRIGGER_SRC_USER_POLL (0U)

Poll mode value for type "HSM_TriggerSrcType".

HSM driver can work in two mode:lsr mode and poll mode. In poll mode, Hsm interrupt is disabled, if the algorithm need more than one step in driver, driver API user should call the API to trigger the internal state machine go to next state. If don't need trigger the process any more, user should call the specific API to get the result state more than one time until driver API return it actually.

#define HSM_TRNG_RAND_U32_CNT 16

the true random count return by the API

#define HSM_TRNG_RAND_U32_CNT 16

the true random count return by the API

#define HSM_TRUE (1u)

Boolean true value definition for type HSM_BoolType used by HSM.

#define HSM_XCMAC_ENGINE_AES 0U

#define HSM_XCMAC_ENGINE_SM4 1U

#define HSMCOM_FASTTRNG_SRC_NORMAL 0x0u

#define HSMCOM_FASTTRNG_SRC_XOR 0x1u

#define HSMCOM_FIRMWARE_LOAD_FAIL 4U

status value represents firmware load fail

#define HSMCOM_FIRMWARE_LOAD_NONE 0U

no firmware detected in flash status

#define HSMCOM_FIRMWARE_LOAD_OK 3U

status value represents firmware is loaded successfully

#define HSMCOM_FIRMWARE_LOADING 1U

status value represents firmware is loading now

#define HSMCOM_FIRMWARE_NEED_USER_LOAD 2U

status value represents firmware waiting user to trigger load in CM7 side

```

#define HSMCOM_NVR_OTP_END_ADDR 0x04401fff

#define HSMCOM_NVR_OTP_START_ADDR 0x04400400

#define HSMCOM_TRNG_SRC_0 0x40cau

#define HSMCOM_TRNG_SRC_1 0xc51cu

#define HSMCOM_TRNG_SRC_ANY 0x359bu

#define HSMCOM_TRNG_SRC_SHIFTXOR 0x648Cu

#define HSMCOM_TRNG_SRC_XOR 0x80a5u

#define KEYMANAGER_ENDECRY_AES256 (1u)

#define KEYMANAGER_ENDECRY_ALG_CBC (1u)

#define KEYMANAGER_ENDECRY_ALG_CFB (3u)

#define KEYMANAGER_ENDECRY_ALG_CTR (0u)

#define KEYMANAGER_ENDECRY_ALG_ECB (2u)

#define KEYMANAGER_ENDECRY_ALG_MAX (5u)

#define KEYMANAGER_ENDECRY_ALG_OFB (4u)

#define KEYMANAGER_ENDECRY_MAX (3u)

#define KEYMANAGER_ENDECRY_NONE (0u)/* For plain key, no need to encrypt or decrypt. */

#define KEYMANAGER_ENDECRY_SM4 (2u)

#define KEYMANAGER_FALSE (0u)

#define KEYMANAGER_GEN_KEYPAIR_ECC (1u)

#define KEYMANAGER_GEN_KEYPAIR_MAX (2u)

#define KEYMANAGER_GEN_KEYPAIR_SM2 (0u)

#define KEYMANAGER_KEY_EXPORT_CIPHER (2u)

```

```

#define KEYMANAGER_KEY_EXPORT_FORBID (0u)

#define KEYMANAGER_KEY_EXPORT_MAX (3u)

#define KEYMANAGER_KEY_EXPORT_PLAIN (1u)

#define KEYMANAGER_TRUE (1u)

#define KEYMANAGER_USER_KEY_TYPE_AES (1u)

#define KEYMANAGER_USER_KEY_TYPE_CCM (11u)

#define KEYMANAGER_USER_KEY_TYPE_CMAC_KEY (10u)

#define KEYMANAGER_USER_KEY_TYPE_CMAC_RESULT (16u)

#define KEYMANAGER_USER_KEY_TYPE_ECC_PRIVATE (3u)

#define KEYMANAGER_USER_KEY_TYPE_ECC_PUBLIC (4u)

#define KEYMANAGER_USER_KEY_TYPE_ECC_SIGNATURE (13u)

#define KEYMANAGER_USER_KEY_TYPE_ED25519_PRIVATE (21u)

#define KEYMANAGER_USER_KEY_TYPE_ED25519_PUBKEY (19u)

#define KEYMANAGER_USER_KEY_TYPE_ED25519_SIGNATURE (20u)

#define KEYMANAGER_USER_KEY_TYPE_GCM (12u)

#define KEYMANAGER_USER_KEY_TYPE_HASHMAC_KEY (23u)

#define KEYMANAGER_USER_KEY_TYPE_IV (15u)

#define KEYMANAGER_USER_KEY_TYPE_NONE (0u) /* not configure the key type, HSM shall not check if
the key type is matched with de/encrypt ALG. */

#define KEYMANAGER_USER_KEY_TYPE_PBKDF2_PWD (22u)

#define KEYMANAGER_USER_KEY_TYPE_RSA (24u)

#define KEYMANAGER_USER_KEY_TYPE_RSA_E (7u)

```

#define KEYMANAGER_USER_KEY_TYPE_RSA_N (8u)

#define KEYMANAGER_USER_KEY_TYPE_RSA_SIGNATURE (18u)

#define KEYMANAGER_USER_KEY_TYPE_SM2_PRIVATE (5u)

#define KEYMANAGER_USER_KEY_TYPE_SM2_PUBLIC (6u)

#define KEYMANAGER_USER_KEY_TYPE_SM2_SIGNATURE (14u)

#define KEYMANAGER_USER_KEY_TYPE_SM4 (1u)

#define KEYMANAGER_USER_KEY_TYPE_XMAC (9u)

#define KEYMANAGER_USER_KEY_TYPE_XMAC_RESULT (17u)

#define KEYMANAGER_USERKEY_STATUS_ERROR (3u)

#define KEYMANAGER_USERKEY_STATUS_EXIST_INVALID (2u)

#define KEYMANAGER_USERKEY_STATUS_EXIST_VALID (1u)

#define KEYMANAGER_USERKEY_STATUS_NOT_EXIST (0u)

#define MAILBOXAPI_RET_AUTHORITY_ERROR (8u)

authority error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_BUSY (9u)

hardware busy status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_ECC_ERR (10u)

flash ecc error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_ERROR (1u)

error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_FAIL (2u)

failure status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_HW_ERROR (5u)

hardware error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_NO_PRIVILEGES (6u)

have no permission status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_NONE (0xFFFFFFFFu)

#define MAILBOXAPI_RET_NOT_SUPPORT (4u)

function not support status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_PARAMETER_ERROR (7u)

parameters error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_PASS (3u)

verify pass status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_SUCCESS (0u)

success status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define MAILBOXAPI_RET_TIMEOUT (11u)

time expire error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

Typedef Documentation

typedef HSM_AesEnDecryptType HSM_AesDecryptType

AES encrypt information used by driver when call specific API.

typedef HSM_AesEnDecryptType HSM_AesEncryptType

AES decrypt information used by driver when call specific API.

typedef hsm_uint32_t HSM_AesKeyBitCntType

type definition for AES key

HSM support AES128/192/256, driver api use this to select the algorithm.

typedef hsm_uint32_t HSM_AesKeyType

type definition for the key source

key from the nvr refer to HSM_AES_KEY_CHIP_VENDOR_IFR key from driver user's parameters refer to HSM_AES_KEY_NEW

typedef hsm_uint32_t HSM_BoolType

Boolean type for HSM.

typedef HSM_XCMacExType HSM_CMacExType

typedef HSM_XCMacType HSM_CMacType

CMAC information used by driver when call specific API.

typedef hsm_uint32_t HSM_DataFormatType

HSM data format refer to HSM_DATA_FORMAT_4B, HSM_DATA_FORMAT_1B.

typedef hsm_uint32_t HSM_DrvEccCurvePrmIndexType

type definition for ECC curve data that defined by NIST

typedef hsm_uint32_t HSM_FastTrngSrcType

typedef HSM_FlexAesEnDecryptType HSM_FlexAesDecryptType

AES encrypt information used by driver when call specific API.

typedef HSM_FlexAesEnDecryptType HSM_FlexAesEncryptType

Flex AES decrypt information used by driver when call specific API.

typedef hsm_uint32_t HSM_FlexAesPadType

Flexiable AES padding type definition if use zero padding, refer to HSM_FLEXAES_PAD_ZERO, after decrypt, user should remove the zero manually if use PKCS7 padding, refer to HSM_FLEXAES_PAD_PKCS7.

typedef hsm_uint32_t HSM_HashAlgType

type	definition	for	SHA	algorithm	refer	to
	HSM_SHA_160,HSM_SHA_224,HSM_SHA_256,HSM_SHA_384,HSM_SHA_512,HSM_SHA_512_224,HSM_SHA_512_256,HSM_HASH_MD5,HSM_HASH_SM3					

typedef hsm_uint32_t HSM_HfamMacType

SM3/MD5/SHA can generate MAC, use this to select the type of MAC SMAC refer to HSM_HFAM_MAC_SMAC
HMAC refer to HSM_HFAM_MAC_HMAC.

typedef hsm_uint32_t HSM_KeyManagerBoolType

typedef hsm_uint32_t HSM_KeyManagerEnDecryAlgType

typedef hsm_uint32_t HSM_KeyManagerGenKeyPairType

typedef hsm_uint32_t HSM_KeyManagerKeyExportType

typedef hsm_uint32_t HSM_KeyManagerUserKeyEnDecryType

typedef hsm_uint32_t HSM_KeyManagerUserKeyTypeType

typedef hsm_uint32_t HSM_MailboxApiRetType

status type definition returned by HSM subsystem APIs

typedef HSM_SmsCfgType HSM_Md5CtxType

MD5 context information used by driver when call specific API.

typedef hsm_uint32_t HSM_RndSrcType

typedef hsm_uint32_t HSM_RsaSsaInputType

type definition for rsa ssa input data type refer to HSM_RSA_SSA_INPUT_RAW_MESSAGE,
HSM_RSA_SSA_INPUT_HASH_DATA

typedef hsm_uint32_t HSM_ShaAlgType

type definition for SHA algorithm refer to
HSM_SHA_160,HSM_SHA_224,HSM_SHA_256,HSM_SHA_384,HSM_SHA_512,HSM_SHA_512_224,HSM_SHA_512_256

typedef HSM_SmsCfgType HSM_ShaCtxType

SHA context information used by driver when call specific API.

typedef HSM_SmsCfgType HSM_Sm3CtxType

SM3 context information used by driver when call specific API.

typedef hsm_uint32_t HSM_Sm4KeyType

typedef hsm_uint32_t HSM_SymmEnDecryptAlgType

type definition for SM4/AES encrypt/decrypt mode

HSM support CTR/CBC/ECB/CFB/OFB mode, driver api use this to select the encrypt/decrypt mode.

typedef hsm_uint32_t HSM_TriggerSrcType

type definition for hsm state machine trigger source used by cortex side hsm driver

HSM driver can work in two mode:lsr mode and poll mode. lsr mode's detail can refer to
HSM_TRIGGER_SRC_ISR. Poll mode's detail can refer to HSM_TRIGGER_SRC_USER_POLL

typedef hsm_uint32_t HSM_XCMacEngineType

typedef HSM_XCMacType HSM_XMacType

XMAC information used by driver when call specific API.

typedef hsm_uint32_t HSMCom_FirmwareLoadStatus

type definition for firmware load status

typedef HSMCom_FlexAesDecryptType HSMCom_Sm4DecryptExType

typedef HSMCom_Sm4EnDecryptType HSMCom_Sm4DecryptType

typedef HSMCom_FlexAesEncryptType HSMCom_Sm4EncryptExType

typedef HSMCom_Sm4EnDecryptType HSMCom_Sm4EncryptType

typedef hsm_uint32_t KeyManager_UserKeyStatusType

Enumeration Type Documentation

enum HSM_StatusType

Status returned by HSM SDK CM7 side APIs.

Enumerator:

HSM_STATUS_SUCCESS	return this when HSM API execute successfully
HSM_STATUS_BUSY	return this when hsm driver or the hardware is busy now

HSM_STATUS_PARAMETER_ERR	return this when parameter error
HSM_STATUS_ERROR	return this when some error happen in HSM API
HSM_STATUS_FLASH_ERROR	return this when flash ecc error
HSM_STATUS_IDLE	return this when driver has nothing to do
HSM_STATUS_ERROR_STOP	return this when the continuous operation in poll API or ISR is stopped due to some error happen
HSM_STATUS_FINISH	return this only when FC4150F512 CMAC/XMAC push data into hardware finish
HSM_STATUS_PASS	return this when ecc/sm2 verify pass, or check pass in CMAC/XMAC/CCM/GCM which icv check enabled
HSM_STATUS_FAIL	return this when ecc/sm2 verify fail, or check fail in CMAC/XMAC/CCM/GCM which icv check enabled, or get random data fail
HSM_STATUS_AGAIN	return this when the HSM API need call again
HSM_STATUS_LOGIC_ERR	return this when HSM API called in wrong logic
HSM_STATUS_HW_ERR	return this when the HSM hardware error happen
HSM_STATUS_HW_IDLE	return this when the HSM hardware has nothing to do
HSM_STATUS_ALREADY_FINISHED	return this only when FC4150F512 CMAC/XMAC has pushed all data into the hardware

HSM_STATUS_SEND_FAIL	return this when send the command to HSM core fail
HSM_STATUS_BUFFER_SIZE_ERROR	return this when the result buffer size not enough
HSM_STATUS_TIMEOUT_ERR	
HSM_STATUS_NOT_EXIST	
HSM_STATUS_EXIST	
HSM_STATUS_ZOMBIE	
HSM_STATUS_SPACE_FULL	

Module_driver_hsm_Level1_LevelN

Classes

struct **HSM_L1_EccCurveParamType**

Ecc curve type.

struct **HSM_L1_RsaType**

L1 Rsa config type.

struct **HSM_Ln_EccSignType**

Ecc sign config type.

struct **HSM_DrvRandPoolType**

Drive rand pool type.

struct **HSM_DrvRsaInfType**

Rsa information type.

struct **HSM_DrvMd5InfType**

Md5 information type.

struct **HSM_DrvSm3InfType**

Sm3 information type.

struct **HSM_DrvShaInfType**

Sha information type.

struct **HSM_DrvTrngInfType**

Trng information type.

struct **HSM_DrvEccVerifyInfType**

Ecc verify information type.

struct **HSM_DrvEccDecryptInfType**

Ecc decrypt information type.

struct **HSM_DrvCMacInfType**

Cmac information type.

struct **HSM_DrvAesEncryptInfType**

Aes encrypt information type.

struct **HSM_DrvAesDecryptInfType**

Aes decrypt information type.

struct **HSM_DrvSm4EncryptInfType**

Sm4 encrypt information type.

struct **HSM_DrvSm4DecryptInfType**

Sm4 decrypt information type.

struct **HSM_DrvBnCalcInfType**

Big number calculator information type.

struct **HSM_Ln_HashMacCfgType**

Ln hash mac config type.

struct **HSM_DrvScatterHashInfType**

Scatter hash information type.

struct **HSM_DrvScatterCMacInfType**

Scatter SM CMac information type.

struct **HSM_DrvEccSignInfType**

Ecc sign information type.

struct **HSM_L1_DrvRSASsaPssVerifyType**

Level 1 RSAssa Pss verify type.

struct **HSM_L1_DrvRSASsaPkcs1V15VerifyType**

Level 1 RSAssa Pkcs15 verify type.

struct **HSM_DrvEccCalcInfType**

ECC calculate info type.

struct **HSM_DrvBnCalcExtInfType**

struct **HSM_DrvKeyManagelnfType**

struct **HSM_DrvKeySpaceInfType**

struct **HSM_L1_DrvRSASsaPkcs1V15SignType**

struct **HSM_L1_DrvRSASsaPssSignType**

struct **HSM_DrvLoadFirmwareInfType**

struct **HSM_L1_Sm2VerifyType**

struct **HSM_DrvSm2VerifyInfType**

struct **HSM_DrvSm2GenZaInfType**

struct **HSM_LN_Sm2GenZaType**

struct **HSM_L1_RtCtxType_tag**

The structure of the HSM level 1 processing handle.

struct **HSM_Ln_AesEncryptType**

The structure of the aes encrypt type.

struct **HSM_L2_DrvCMacEmptyInfType**

The structure of the HSM level 2 processing handle.

struct **HSM_L2_DrvUpdateKeyInfType**

struct **HSM_L2_DrvRevokeKeyInfType**

struct **HSM_Ln_EccDecryptType**

Ecc decrypt type.

struct **HSM_L1_EccVerifyType**

struct **HSM_Ln_EccVerifyType**

Ecc verify type.

struct **HSM_Ln_ShaType**

Sha information used by driver when call specific API.

struct **HSM_L2_DrvEccVerifyInfType**

struct **HSM_Ln_ScatterHashType**

Scatter hash level n type.

struct **HSM_Ln_ResultBufInfoType**

HSM result buffer type.

struct **HSM_Ln_Sm2VerifyType**

struct **HSM_L2_DrvSm2VerifyInfType**

struct **HSM_L2_RtCtxType_tag**

The structure of the HSM level 2 processing handle.

struct **HSM_Ln_CMacCfgParmsType**

The structure of the level 3 cmac config type.

struct **HSM_L3_DrvScatterCMacInfType**

The structure of the level 3 scatter cmac information type.

struct **HSM_L3_DrvUpdateKeyInfType**

struct **HSM_L3_RtCtxType_tag**

The structure of the HSM level 3 processing handle.

struct **HSM_Ln_RtCtxType**

The structure of the HSM processing handle.

struct **HSM_Ln_InitParmType**

init information for HSM_Ln_Init

struct **HSM_Ln_RsaSsaPssVerifyType**

RSA PSS signature verify information used by driver when call specific API.

struct **HSM_Ln_RsaSsaPkcs1v15VerifyType**

RSA PKCS1 V1.5 signature verify information used by driver when call specific API.

struct **HSM_Ln_AesDecryptType**

The structure of the aes decrypt type.

struct **HSM_Ln_Md5Type**

MD5 information used by driver when call specific API.

struct **HSM_Ln_Sm3Type**

Sm3 information used by driver when call specific API.

struct **HSM_Ln_CMacType**

CMAC information used by driver when call specific API.

struct **HSM_Ln_RngType**

Rng information used by driver when call specific API.

struct **HSM_Ln_ScatterCMacType**

Scatter cmac information used by driver when call specific API.

struct **HSM_L1_InitParmType**

Ecc level 1 init type.

struct **HSM_L2_InitParmType**

Ecc level 2 init type.

struct **HSM_L3_InitParmType**

Ecc level 3 init type.

struct **HSM_Ln_PlainKeyImport**

struct **HSM_Ln_RsaSsaPkcs1V15SignDataType**

struct **HSM_Ln_RsaSsaPkcs1V15SignHashType**

struct **HSM_Ln_RsaSsaPssSignDataType**

struct **HSM_Ln_RsaSsaPssSignHashType**


```

struct HSM_L1_PKCS_RsaPrivateKeyInfType
struct HSM_L1_PKCS_EccPrivateKeyInfType
struct HSM_L1_PKCS8_PrivateKeyInfType
struct HSM_L1_PKCS_RsaPublicKeyInfType
struct HSM_L1_PKCS_EccPublicKeyInfType
struct HSM_L1_PublicKeyInfType
struct HSM_L1_BnCalcType
struct HSM_Ln_EccKPTType

```

Macros

```

#define hsm_hw_swap_u32(x)  ({hsm_uint32_t y; asm volatile ("rev %0, %1\n" : "=r"(y) : "r"(x)); y;})
#define HSM_KEY_LOCATION_NVR_RAM  (0x1u)
#define HSM_KEY_LOCATION_NVR_FLASH  (0x0u)
#define HSM_KEY_ID(id, location)  ((((hsm_uint32_t(id)) << 1u)) | (((hsm_uint32_t(location)) & 0x1u))
#define HSM_ADDRESS_UP_ALIGN(addr)  ((((hsm_uint32_t(addr)) + 0x3u) & ~(hsm_uint32_t0x3)))
#define HSM_4B_BYTE_INDEX_TO_OFFSET(idx)  (((idx) & 0xFFFFFFFcu) + (3u - ((idx) & 0x3u)))
#define HSM_HFAM_HW_PROCESS_ALIGN(x)  128u
#define HSM_AESM_SCATTER_CMAC_HW_PROCESS_ALIGN()  16u
#define CRYPTO_TEXT_SECTION
Section define.

#define HSM_AESM_BACKEND_AES  0x0
AESM BACKEND is aes.

#define HSM_AESM_BACKEND_SM4  0x1
AESM BACKEND is SM4.

#define HSM_L1_PKCS8_RSA_PRIVATE_KEY  0
Trng random number count.

#define HSM_L1_PKCS8_ECC_PRIVATE_KEY  1
#define HSM_L1_RSA_PUBLIC_KEY  0
#define HSM_L1_ECC_PUBLIC_KEY  1
#define HSM_ECC_VERIFY_INPUT_RAW_MESSAGE  0u
#define HSM_ECC_VERIFY_INPUT_HASH_DATA  1u
#define HSM_SM2_USERID_MAX_LENGTH  (128U)/*max length of user id, for mcal ,this parameter is
configed by user*/

```

```
#define HSM_SM2_VERIFY_INPUT_RAW_MESSAGE 0u  
HSM_ECC_VERIFY_INPUT_RAW_MESSAGE value for type "HSM_EccVerifyInputType".
```

```
#define HSM_SM2_VERIFY_INPUT_HASH_DATA 1u  
HSM_ECC_VERIFY_INPUT_HASH_DATA value for type "HSM_EccVerifyInputType".
```

Typedefs

```
typedef hsm_uint32_t HSM_AesmBackendType  
AESM BACKEND Type.
```

```
typedef hsm_uint32_t HSM_L1_PKCS8_PrivateKeyType  
typedef hsm_uint32_t HSM_L1_PublicKeyType  
typedef hsm_uint32_t HSM_EccVerifyInputType  
typedef void(* HSM_CleanDcacheCbType) (void)  
Type definition for algorithm stop callback registered in HSM_Ln_Init API.
```

```
typedef void(* HSM_CleanAndFlushDcacheCbType) (void)  
Type definition for algorithm stop callback registered in HSM_Ln_Init API.
```

```
typedef void(* HSM_StopCbType) (void *pLnCtx, HSM_BoolType bErrorHappen, HSM_StatusType eRet)  
Type definition for algorithm stop callback registered in HSM_Ln_Init API.
```

```
typedef HSM_StatusType(* HSM_SendCommandCbType) (HSM_CmdType *)  
Type definition for send command callback registered in HSM_Ln_Init API.
```

```
typedef HSM_StatusType(* HSM_GetAckStatCbType) (void)  
Type definition for get ack callback registered in HSM_Ln_Init API.
```

```
typedef HSM_StatusType(* HSM_CancelCmdCbType) (void)  
Type definition for cancel command callback registered in HSM_Init API.
```

```
typedef struct HSM_L1_RtCtxType_tag HSM_L1_RtCtxType  
The structure of the HSM level 1 processing handle.
```

typedef **HSM_Ln_AesEncryptType** **HSM_Ln_Sm4EncryptType**

SM4 Encrypt config type.

typedef **hsm_uint32_t** **HSM_Sm2VerifyInputType**

typedef struct **HSM_L2_RtCtxType_tag** **HSM_L2_RtCtxType**

The structure of the HSM level 2 processing handle.

typedef struct **HSM_L3_RtCtxType_tag** **HSM_L3_RtCtxType**

The structure of the HSM level 3 processing handle.

typedef **HSM_Ln_AesDecryptType** **HSM_Ln_Sm4DecryptType**

The structure of the sm4 decrypt type.

Enumerations

enum **HSM_L1_DrvSMTType** { **HSM_DRV_SM_READY** = 0, **HSM_DRV_SM_ARITH**, **HSM_DRV_SM_STOP** }

Level 1 drive SM type.

enum **Crypto_VerifyResultType** { **CRYPTO_E_VER_OK** = 0x00U, **CRYPTO_E_VER_NOT_OK** = 0x01U }

verify result type

enum **HSM_ScatterHashSMTType** { **HSM_L1_SCATTER_HASH_READY**,
HSM_L1_SCATTER_HASH_NEED_MORE_BEFORE_INIT, **HSM_L1_SCATTER_HASH_INIT**,
HSM_L1_SCATTER_HASH_INIT_OK, **HSM_L1_SCATTER_HASH_NEED_MORE_BEFORE_UPDATE_FINAL**,
HSM_L1_SCATTER_HASH_UPDATE, **HSM_L1_SCATTER_HASH_UPDATE_OK**,
HSM_L1_SCATTER_HASH_FINAL, **HSM_L1_SCATTER_HASH_STOP** }

scatter hash sm type

enum **HSM_ScatterCMacSMTType** { **HSM_L1_SCATTER_CMAC_READY**,
HSM_L1_SCATTER_CMAC_NEED_MORE_BEFORE_INIT, **HSM_L1_SCATTER_CMAC_INIT**,
HSM_L1_SCATTER_CMAC_INIT_OK, **HSM_L1_SCATTER_CMAC_NEED_MORE_BEFORE_UPDATE_FINAL**,
HSM_L1_SCATTER_CMAC_UPDATE, **HSM_L1_SCATTER_CMAC_UPDATE_OK**,
HSM_L1_SCATTER_CMAC_FINAL, **HSM_L1_SCATTER_CMAC_STOP** }

scatter hash cmac type

enum **HSM_L2_DrvOpType** { **HSM_LEVEL_2ND_OP_NONE** = 0, **HSM_LEVEL_2ND_OP_CMAC_EMPTY**,
HSM_LEVEL_2ND_OP_UPDATE_KEY, **HSM_LEVEL_2ND_OP_REVOKE_KEY**,
HSM_LEVEL_2ND_OP_ECC_VERIFY, **HSM_LEVEL_2ND_OP_SM2_VERIFY** }

Level 2 operation type.

```
enum HSM_L2_DrvCMacEmptySMType { HSM_L2_CMAC_EMPTY_SM_READY,  
HSM_L2_CMAC_EMPTY_SM_GENERATE_L, HSM_L2_CMAC_EMPTY_SM_GENERATE_T,  
HSM_L2_CMAC_EMPTY_SM_STOP }
```

Level 2 cmac empty sm type.

```
enum HSM_L3_DrvOpType { HSM_LEVEL_3RD_OP_NONE = 0, HSM_LEVEL_3RD_OP_SCATTER_CMAC,  
HSM_LEVEL_3RD_OP_UPDATE_KEY }
```

Level 3 operation type.

```
enum HSM_L3_ScatterCMacSMType { HSM_L3_SCATTER_CMAC_READY,  
HSM_L3_SCATTER_CMAC_EMPTY_INPUT, HSM_L3_SCATTER_CMAC_INIT,  
HSM_L3_SCATTER_CMAC_NEED_MORE_AFTER_INIT, HSM_L3_SCATTER_CMAC_INIT_OK,  
HSM_L3_SCATTER_CMAC_UPDATE, HSM_L3_SCATTER_CMAC_NEED_MORE_AFTER_UPDATE,  
HSM_L3_SCATTER_CMAC_UPDATE_OK, HSM_L3_SCATTER_CMAC_FINAL,  
HSM_L3_SCATTER_CMAC_STOP }
```

Level 3 scatter cmac sm type.

```
enum HSM_Ln_KeyDataStoreFmtType { HSM_LN_KEY_DATA_STORE_FMT_BN_LOADABLE = 0,  
HSM_LN_KEY_DATA_STORE_FMT_4B, HSM_LN_KEY_DATA_STORE_FMT_1B,  
HSM_LN_KEY_DATA_STORE_FMT_SPLIT_BN_LOADABLE }
```

```
enum HSM_L2_DrvUpdateKeySMType { HSM_L2_UPDATE_KEY_SM_READY,  
HSM_L2_UPDATE_KEY_SM_LOADFIRMWARE, HSM_L2_UPDATE_KEY_SM_CHECK_EXIST,  
HSM_L2_UPDATE_KEY_SM_REVOKE, HSM_L2_UPDATE_KEY_SM_TIDYUP,  
HSM_L2_UPDATE_KEY_SM_IMPORT, HSM_L2_UPDATE_KEY_SM_GET_STATUS,  
HSM_L2_UPDATE_KEY_SM_STOP }
```

```
enum HSM_L2_DrvRevokeKeySMType { HSM_L2_REVOKE_KEY_SM_READY,  
HSM_L2_REVOKE_KEY_SM_CHECK_EXIST, HSM_L2_REVOKE_KEY_SM_REVOKE,  
HSM_L2_REVOKE_KEY_SM_STOP }
```

```
enum HSM_L2_DrvEccVerifySMType { HSM_L2_ECC_VERIFY_SM_READY,  
HSM_L2_ECC_VERIFY_SM_HASH, HSM_L2_ECC_VERIFY_SM_VERIFY, HSM_L2_ECC_VERIFY_SM_STOP }
```

```
enum HSM_L2_DrvSm2VerifySMType { HSM_L2_SM2_VERIFY_SM_READY,  
HSM_L2_SM2_VERIFY_SM_GEN_ZA, HSM_L2_SM2_VERIFY_SM_HASH_UPDATE,  
HSM_L2_SM2_VERIFY_SM_HASH_FINALLY, HSM_L2_SM2_VERIFY_SM_VERIFY,  
HSM_L2_SM2_VERIFY_SM_STOP }
```

```
enum HSM_L3_UpdateKeySMType { HSM_L3_UPDATE_KEY_READY, HSM_L3_UPDATE_KEY_PROCESS,  
HSM_L3_UPDATE_KEY_STOP }
```

```
enum HSM_DrvOpCatagoryType { HSM_OP_NONE, HSM_OP_AES_ENCRYPT, HSM_OP_AES_DECRYPT,  
HSM_OP_SM4_ENCRYPT, HSM_OP_SM4_DECRYPT, HSM_OP_CMAC, HSM_OP_RNG, HSM_OP_SHA,  
HSM_OP_MD5, HSM_OP_SM3, HSM_OP_ECC_VERIFY, HSM_OP_ECC_EASYDECY, HSM_OP_ECC_CALC,  
HSM_OP_BN_CALC, HSM_OP_SCATTER_HASH, HSM_OP_SCATTER_CMAC, HSM_OP_ECC_SIGN,
```

```

HSM_OP_RSA_SSA_PSS_VERIFY, HSM_OP_RSA_SSA_PKCS1V15_VERIFY,
HSM_OP_KEY_MANAGE_IMPORT, HSM_OP_KEY_MANAGE_VALIDATE, HSM_OP_KEY_MANAGE_COPY,
HSM_OP_KEY_MANAGE_REVOKE, HSM_OP_KEY_MANAGE_TIDYUP,
HSM_OP_KEY_MANAGE_GETSTATUS, HSM_OP_RSA_SSA_PKCS1V15_SIGN,
HSM_OP_RSA_SSA_PSS_SIGN, HSM_OP_KEY_MANAGE_GET_ID_STATUS, HSM_OP_LOAD_FIRMWARE,
HSM_OP_SM2_VERIFY, HSM_OP_SM2_GEN_ZA }
enum HFAM_MODE_E { HFA_MD5 = 0, HFA_SHA160 = 1, HFA_SHA224 = 2, HFA_SHA256 = 3, HFA_SHA384 =
4, HFA_SHA512 = 5, HFA_SHA512_224 = 6, HFA_SHA512_256 = 7, HFA_SM3 = 8 }
enum HSM_L1_BnCalcTypeType { HSM_L1_BNCALC_A_ADD_B = 0, HSM_L1_BNCALC_A_SUB_B,
HSM_L1_BNCALC_B_SUB_A, HSM_L1_BNCALC_A_x_B, HSM_L1_BNCALC_A_Exp_E,
HSM_L1_BNCALC_A_MOD, HSM_L1_BNCALC_A_RECIPROCAL }

```

Functions

CRYPTO_TEXT_SECTION void HSM_Ln_Init (HSM_Ln_RtCtxType *pLnRt, HSM_Ln_InitParmType *pInitParm)

Init config of hsm.

CRYPTO_TEXT_SECTION void HSM_Ln_Ack_IrqHandler (HSM_Ln_RtCtxType *pLnCtx)

Interrupt handle function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Abort (HSM_Ln_RtCtxType *pCtx)

HSM abort function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifySetupEnv (HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32RsaBitCnt)

Rsassa pss verify setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerify (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_RsaSsaPssVerifyType *pCfg)

Rsassa pss verify function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifyPoll (HSM_Ln_RtCtxType *pLnCtx)

Rsassa pss verify poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Rsassa pss verify exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifySetupEnv
(HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32RsaBitCnt)

Rsassa pkcs1v15 verify setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15Verify (HSM_Ln_RtCtxType
*pLnCtx, const HSM_Ln_RsaSsaPkcs1v15VerifyType *pCfg)

Rsassa pkcs1v15 verify function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifyPoll (HSM_Ln_RtCtxType
*pLnCtx)

Rsassa pkcs1v15 verify poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifyExitEnv (HSM_Ln_RtCtxType
*pLnCtx)

Rsassa pkcs1v15 verify exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Sha verify setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Sha verify exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sha (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_ShaType *pSha)

Sha function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaPoll (HSM_Ln_RtCtxType *pLnCtx)

Sha poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CommonExitEnv (HSM_Ln_RtCtxType *pCtx)

Hsm common exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

SM4 encrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

SM4 encrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4Encrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Sm4EncryptType *pCfg)

SM4 encrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptPoll (HSM_Ln_RtCtxType *pLnCtx)

SM4 encrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

SM4 decrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

SM4 decrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4Decrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Sm4DecryptType *pCfg)

SM4 decrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptPoll (HSM_Ln_RtCtxType *pLnCtx)

SM4 decrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

AES encrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

AES encrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_AesEncryptType *pCfg)

AES encrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptPoll (HSM_Ln_RtCtxType *pLnCtx)

AES encrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

AES decrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

AES decrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_AesDecryptType *pCfg)

AES decrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptPoll (HSM_Ln_RtCtxType *pLnCtx)

AES decrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5SetupEnv (HSM_Ln_RtCtxType *pLnCtx)

MD5 setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5ExitEnv (HSM_Ln_RtCtxType *pLnCtx)

MD5 exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5 (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Md5Type *pCfg)

MD5 function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5Poll (HSM_Ln_RtCtxType *pLnCtx)

MD5 poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3SetupEnv (HSM_Ln_RtCtxType *pLnCtx)

SM3 setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3ExitEnv (HSM_Ln_RtCtxType *pLnCtx)

SM3 exit environment.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3 (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_Sm3Type *pCfg)**

SM3 function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3Poll (HSM_Ln_RtCtxType *pLnCtx)

SM3 poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

CMAC setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacExitEnv (HSM_Ln_RtCtxType *pLnCtx)

CMAC exit environment.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMac (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_CMacType *pCfg)**

CMAC function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacPoll (HSM_Ln_RtCtxType *pLnCtx)

CMAC poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Rng setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Rng exit environment.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Rng (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_RngType *pCfg)**

Rng function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngPoll (HSM_Ln_RtCtxType *pLnCtx)

Rng poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter cmac setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter cmac exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacPoll (HSM_Ln_RtCtxType *pLnCtx)

Scatter cmac poll function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacInit (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_CMacCfgParmsType *pParmCfg, const HSM_Ln_ScatterCMacType *pCfg)**

Scatter cmac init function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacUpdate (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_ScatterCMacType *pCfg)**

Scatter cmac update function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacFinal (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_ScatterCMacType *pCfg, const HSM_Ln_ResultBufInfoType *pResult)**

Scatter cmac final function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter hash setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter hash exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashPoll (HSM_Ln_RtCtxType *pLnCtx)

Scatter hash poll function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashInit (HSM_Ln_RtCtxType *pLnCtx,
const HSM_ShaAlgType eAlg, const HSM_Ln_ScatterHashType *pCfg)**

Scatter hash init function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashUpdate (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_ScatterHashType *pCfg)**

Scatter hash update function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashFinal (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_ScatterHashType *pCfg, const HSM_Ln_ResultBufInfoType *pResult)**

Scatter hash final function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashSetMacCfg (HSM_Ln_RtCtxType
*pLnCtx, HSM_Ln_HashMacCfgType *pCfg)**

Scatter hash set mac config function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignSetupEnv (HSM_Ln_RtCtxType *pLnCtx,
hsm_uint32_t u32BitCnt)**

Ecc sign setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Ecc sign exit environment.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSign (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_EccSignType *pCfg)**

Ecc sign function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignPoll (HSM_Ln_RtCtxType *pLnCtx)

Ecc sign poll function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx,
hsm_uint32_t u32BitCnt)**

Ecc decrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Ecc decrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_EccDecryptType *pCfg)

Ecc decrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptPoll (HSM_Ln_RtCtxType *pLnCtx)

Ecc decrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifySetupEnv (HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32BitCnt)

Ecc verify setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Ecc verify exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerify (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_EccVerifyType *pCfg)

Ecc verify function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifyPoll (HSM_Ln_RtCtxType *pLnCtx)

Ecc verify poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImportSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for import keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImport (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_ImportUserKeyExtType *pCfg)

Import key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImportExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for import keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidateSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for valid keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidate (HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32KeyId)

Valid key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidateExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for valid keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for copy keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopy (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_CopyUserKeyType *pCfg)

Copy keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for copy keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevokeSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for revoke keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevoke (HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32KeyId)

Revoke keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevokeExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for revoke keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUpSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for tidy up key space.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUp (HSM_Ln_RtCtxType *pLnCtx, HSM_Ln_KeySpaceStatusType *pCfg)

Tidy up key space.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUpExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for tidy up key space.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKeySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for update key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKey (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_PlainKeyImport *pCfg)

Update key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKeyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for update key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKeySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKey (HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32KeyId)

Revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKeyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignSetupEnv (HSM_Ln_RtCtxType *pCtx)

Setup an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignData (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPkcs1V15SignDataType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss-Pkcs-1v15 sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignHash (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPkcs1V15SignHashType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss-Pkcs-1v15 sign hash.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignExitEnv (HSM_Ln_RtCtxType *pCtx)

Exit an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignSetupEnv (HSM_Ln_RtCtxType *pCtx)

Setup an environment for RsaSsa-pss sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignData (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPssSignDataType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignHash (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPssSignHashType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss sign hash.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignExitEnv (HSM_Ln_RtCtxType *pCtx)

Exit an environment for RsaSsa-pss sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2Verify (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Sm2VerifyType *pCfg)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifyPoll (HSM_Ln_RtCtxType *pLnCtx)

Detailed Description

Macro Definition Documentation

#define CRYPTO_TEXT_SECTION

Section define.

#define HSM_4B_BYTE_INDEX_TO_OFFSET(idx) (((idx) & 0xFFFFFFFFCu) + (3u - ((idx) & 0x3u)))

#define HSM_ADDRESS_UP_ALIGN(addr) (((hsm_uint32_t)(addr) + 0x3u) & ~(hsm_uint32_t)0x3))

#define HSM_AESM_BACKEND_AES 0x0

AESM BACKEND is aes.

#define HSM_AESM_BACKEND_SM4 0x1

AESM BACKEND is SM4.


```

#define HSM_AESM_SCATTER_CMAC_HW_PROCESS_ALIGN() 16u

#define HSM_ECC_VERIFY_INPUT_HASH_DATA 1u

#define HSM_ECC_VERIFY_INPUT_RAW_MESSAGE 0u

#define HSM_HFAM_HW_PROCESS_ALIGN(x) 128u

#define hsm_hw_swap_u32(x) ({hsm_uint32_t y; asm volatile ("rev %0, %1\n" : "=r"(y) : "r"(x)); y;})

#define HSM_KEY_ID(id, location) ((((((hsm_uint32_t)(id)) << 1u)) | (((hsm_uint32_t)(location)) & 0x1u))

#define HSM_KEY_LOCATION_NVR_FLASH (0x0u)

#define HSM_KEY_LOCATION_NVR_RAM (0x1u)

#define HSM_L1_ECC_PUBLIC_KEY 1

#define HSM_L1_PKCS8_ECC_PRIVATE_KEY 1

#define HSM_L1_PKCS8_RSA_PRIVATE_KEY 0

```

Trng random number count.

```

#define HSM_L1_RSA_PUBLIC_KEY 0

#define HSM_SM2_USERID_MAX_LENGTH (128U)/*max length of user id, for mcal ,this parameter is
configd by user*/

#define HSM_SM2_VERIFY_INPUT_HASH_DATA 1u

```

HSM_ECC_VERIFY_INPUT_HASH_DATA value for type "HSM_EccVerifyInputType".

```

#define HSM_SM2_VERIFY_INPUT_RAW_MESSAGE 0u

```

HSM_ECC_VERIFY_INPUT_RAW_MESSAGE value for type "HSM_EccVerifyInputType".

Typedef Documentation

typedef hsm_uint32_t HSM_AesmBackendType

AESM BACKEND Type.

typedef HSM_StatusType(* HSM_CancelCmdCbType) (void)

Type definition for cancel command callback registered in HSM_Init API.

typedef void(* HSM_CleanAndFlushDcacheCbType) (void)

Type definition for algorithm stop callback registered in HSM_Ln_Init API.

typedef void(* HSM_CleanDcacheCbType) (void)

Type definition for algorithm stop callback registered in HSM_Ln_Init API.

typedef hsm_uint32_t HSM_EccVerifyInputType

typedef HSM_StatusType(* HSM_GetAckStatCbType) (void)

Type definition for get ack callback registered in HSM_Ln_Init API.

typedef hsm_uint32_t HSM_L1_PKCS8_PrivateKeyType

typedef hsm_uint32_t HSM_L1_PublicKeyType

typedef struct HSM_L1_RtCtxType_tag HSM_L1_RtCtxType

The structure of the HSM level 1 processing handle.

typedef struct HSM_L2_RtCtxType_tag HSM_L2_RtCtxType

The structure of the HSM level 2 processing handle.

typedef struct HSM_L3_RtCtxType_tag HSM_L3_RtCtxType

The structure of the HSM level 3 processing handle.

typedef HSM_Ln_AesDecryptType HSM_Ln_Sm4DecryptType

The structure of the sm4 decrypt type.

typedef HSM_Ln_AesEncryptType HSM_Ln_Sm4EncryptType

SM4 Encrypt config type.

typedef HSM_StatusType(* HSM_SendCommandCbType) (HSM_CmdType *)

Type definition for send command callback registered in HSM_Ln_Init API.

typedef hsm_uint32_t HSM_Sm2VerifyInputType

typedef void(* HSM_StopCbType) (void *pLnCtx, HSM_BoolType bErrorHappen, HSM_StatusType eRet)

Type definition for algorithm stop callback registered in HSM_Ln_Init API.

Enumeration Type Documentation

enum Crypto_VerifyResultType

verify result type

Enumerator:

CRYPTO_E_VER _OK	The result of the verification is "true", i.e. the two compared elements are identical. This return code shall be given as value "0"
CRYPTO_E_VER _NOT_OK	The result of the verification is "false", i.e. the two compared elements are not identical. This return code shall be given as value "1".

enum HFAM_MODE_E

Enumerator:

HFA_MD5	
HFA_SHA160	
HFA_SHA224	
HFA_SHA256	
HFA_SHA384	
HFA_SHA512	
HFA_SHA512_22 4	
HFA_SHA512_25 6	
HFA_SM3	

enum HSM_DrvOpCatagoryType

Enumerator:

HSM_OP_NONE	
HSM_OP_AES_E NCRYPT	
HSM_OP_AES_D ECRYPT	
HSM_OP_SM4_ ENCRYPT	
HSM_OP_SM4_ DECRYPT	
HSM_OP_CMAC	
HSM_OP_RNG	
HSM_OP_SHA	
HSM_OP_MD5	
HSM_OP_SM3	
HSM_OP_ECC_V ERIFY	
HSM_OP_ECC_E ASYDECRY	
HSM_OP_ECC_C ALC	
HSM_OP_BN_C ALC	
HSM_OP_SCATT ER_HASH	
HSM_OP_SCATT ER_CMAC	
HSM_OP_ECC_S IGN	
HSM_OP_RSA_S SA_PSS_VERIFY	
HSM_OP_RSA_S SA_PKCS1V15_V ERIFY	
HSM_OP_KEY_ MANAGE_IMPORT T	

HSM_OP_KEY_ MANAGE_VALID ATE	
HSM_OP_KEY_ MANAGE_COPY	
HSM_OP_KEY_ MANAGE_REVO KE	
HSM_OP_KEY_ MANAGE_TIDYU P	
HSM_OP_KEY_ MANAGE_GETST ATUS	
HSM_OP_RSA_S SA_PKCS1V15_S IGN	
HSM_OP_RSA_S SA_PSS_SIGN	
HSM_OP_KEY_ MANAGE_GET_I D_STATUS	
HSM_OP_LOAD _FIRMWARE	
HSM_OP_SM2_ VERIFY	
HSM_OP_SM2_ GEN_ZA	

enum HSM_L1_BnCalcTypeType

Enumerator:

HSM_L1_BNCAL C_A_ADD_B	
HSM_L1_BNCAL C_A_SUB_B	
HSM_L1_BNCAL C_B_SUB_A	

HSM_L1_BNCAL C_A_x_B	
HSM_L1_BNCAL C_A_Exp_E	
HSM_L1_BNCAL C_A_MOD	
HSM_L1_BNCAL C_A_RECIPROC AL	

enum HSM_L1_DrvSMType

Level 1 drive SM type.

Enumerator:

HSM_DRV_SM_ READY	
HSM_DRV_SM_ ARITH	
HSM_DRV_SM_S TOP	

enum HSM_L2_DrvCMacEmptySMType

Level 2 cmac empty sm type.

Enumerator:

HSM_L2_CMAC_ EMPTY_SM_REA DY	
HSM_L2_CMAC_ EMPTY_SM_GEN ERATE_L	

HSM_L2_CMAC_ EMPTY_SM_GEN ERATE_T	
HSM_L2_CMAC_ EMPTY_SM_STO P	

enum HSM_L2_DrvEccVerifySMType

Enumerator:

HSM_L2_ECC_V ERIFY_SM_READ Y	
HSM_L2_ECC_V ERIFY_SM_HAS H	
HSM_L2_ECC_V ERIFY_SM_VERI FY	
HSM_L2_ECC_V ERIFY_SM_STOP	

enum HSM_L2_DrvOpType

Level 2 operation type.

Enumerator:

HSM_LEVEL_2N D_OP_NONE	
HSM_LEVEL_2N D_OP_CMACE MPTY	
HSM_LEVEL_2N D_OP_UPDATE_ KEY	

HSM_LEVEL_2N D_OP_REVOKE_ KEY	
HSM_LEVEL_2N D_OP_ECC_VERI FY	
HSM_LEVEL_2N D_OP_SM2_VER IFY	

enum HSM_L2_DrvRevokeKeySMType

Enumerator:

HSM_L2_REVOK E_KEY_SM_REA DY	
HSM_L2_REVOK E_KEY_SM_CHE CK_EXIST	
HSM_L2_REVOK E_KEY_SM_REV OKE	
HSM_L2_REVOK E_KEY_SM_STO P	

enum HSM_L2_DrvSm2VerifySMType

Enumerator:

HSM_L2_SM2_V ERIFY_SM_READ Y	
HSM_L2_SM2_V ERIFY_SM_GEN_ ZA	

HSM_L2_SM2_V ERIFY_SM_HAS H_UPDATE	
HSM_L2_SM2_V ERIFY_SM_HAS H_FINALLY	
HSM_L2_SM2_V ERIFY_SM_VERI FY	
HSM_L2_SM2_V ERIFY_SM_STOP	

enum HSM_L2_DrvUpdateKeySMType

Enumerator:

HSM_L2_UPDAT E_KEY_SM_REA DY	
HSM_L2_UPDAT E_KEY_SM_LOA DFIRMWARE	
HSM_L2_UPDAT E_KEY_SM_CHE CK_EXIST	
HSM_L2_UPDAT E_KEY_SM_REV OKE	
HSM_L2_UPDAT E_KEY_SM_TIDY UP	
HSM_L2_UPDAT E_KEY_SM_IMP ORT	
HSM_L2_UPDAT E_KEY_SM_GET _STATUS	

HSM_L2_UPDAT E_KEY_SM_STO P	
-----------------------------------	--

enum HSM_L3_DrvOpType

Level 3 operation type.

Enumerator:

HSM_LEVEL_3R D_OP_NONE	
HSM_LEVEL_3R D_OP_SCATTER _CMAC	
HSM_LEVEL_3R D_OP_UPDATE_ KEY	

enum HSM_L3_ScatterCMacSMType

Level 3 scatter cmac sm type.

Enumerator:

HSM_L3_SCATT ER_CMAC_READ Y	
HSM_L3_SCATT ER_CMAC_EMPT Y_INPUT	
HSM_L3_SCATT ER_CMAC_INIT	
HSM_L3_SCATT ER_CMAC_NEED	

HSM_L3_SCATTER_CMACE_INIT_MORE_AFTER_INIT	
HSM_L3_SCATTER_CMACE_INIT_OK	
HSM_L3_SCATTER_CMACE_UPDATE	
HSM_L3_SCATTER_CMACE_NEED_MORE_AFTER_UPDATE	
HSM_L3_SCATTER_CMACE_UPDATE_OK	
HSM_L3_SCATTER_CMACE_FINAL	
HSM_L3_SCATTER_CMACE_STOP	

enum HSM_L3_UpdateKeySMType

Enumerator:

HSM_L3_UPDATE_KEY_READY	
HSM_L3_UPDATE_KEY_PROCESS	
HSM_L3_UPDATE_KEY_STOP	

enum HSM_Ln_KeyDataStoreFmtType

Enumerator:

HSM_LN_KEY_DATA_STORE_FMT	
---------------------------	--

T_BN_LOADABLE	
HSM_LN_KEY_DATA_STORE_FORMAT_4B	
HSM_LN_KEY_DATA_STORE_FORMAT_1B	
HSM_LN_KEY_DATA_STORE_FORMAT_SPLIT_BN_LOADABLE	

enum HSM_ScatterCMacSMType

scatter hash cmac type

Enumerator:

HSM_L1_SCATTER_CMAC_READ_Y	
HSM_L1_SCATTER_CMAC_NEED_MORE_BEFORE_INIT	
HSM_L1_SCATTER_CMAC_INIT	
HSM_L1_SCATTER_CMAC_INIT_OK	
HSM_L1_SCATTER_CMAC_NEED_MORE_BEFORE_UPDATE_FINAL	

HSM_L1_SCATT ER_CMACE_UPDA TE	
HSM_L1_SCATT ER_CMACE_UPDA TE_OK	
HSM_L1_SCATT ER_CMACE_FINA L	
HSM_L1_SCATT ER_CMACE_STOP	

enum HSM_ScatterHashSMType

scatter hash sm type

Enumerator:

HSM_L1_SCATT ER_HASH_READ Y	
HSM_L1_SCATT ER_HASH_NEED _MORE_BEFORE _INIT	
HSM_L1_SCATT ER_HASH_INIT	
HSM_L1_SCATT ER_HASH_INIT_ OK	
HSM_L1_SCATT ER_HASH_NEED _MORE_BEFORE _UPDATE_FINAL	
HSM_L1_SCATT ER_HASH_UPDA TE	

HSM_L1_SCATT ER_HASH_UPDA TE_OK	
HSM_L1_SCATT ER_HASH_FINAL	
HSM_L1_SCATT ER_HASH_STOP	

Function Documentation

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Abort (HSM_Ln_RtCtxType * *pCtx*)

HSM abort function.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
-------------	----------------------------------

Returns

hsm status

CRYPTO_TEXT_SECTION void HSM_Ln_Ack_IrqHandler (HSM_Ln_RtCtxType * *pLnCtx*)

Interrupt handle function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecrypt (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_AesDecryptType * *pCfg*)

AES decrypt function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	AES decrypt config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

AES decrypt exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptPoll (HSM_Ln_RtCtxType * *pLnCtx*)

AES decrypt poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

AES decrypt setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncrypt (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_AesEncryptType * *pCfg*)

AES encrypt function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	AES encrypt config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

AES encrypt exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptPoll (HSM_Ln_RtCtxType * *pLnCtx*)

AES encrypt poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

AES encrypt setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMac (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_CMacType * *pCfg*)

CMAC function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	CMAC config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

CMAC exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacPoll (HSM_Ln_RtCtxType * *pLnCtx*)

CMAC poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

CMAC setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CommonExitEnv (HSM_Ln_RtCtxType * *pCtx*)

Hsm common exit environment.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
-------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecrypt (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_EccDecryptType * *pCfg*)

Ecc decrypt function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Ecc decrypt config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Ecc decrypt exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Ecc decrypt poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*,
hsm_uint32_t *u32BitCnt*)**

Ecc decrypt setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>u32BitCnt</i>	Byte count

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSign (HSM_Ln_RtCtxType * *pLnCtx*, const
HSM_Ln_EccSignType * *pCfg*)**

Ecc sign function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Ecc sign config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Ecc sign exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Ecc sign poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Ecc sign config parameter
<i>pLnCtx</i>	Hsm handle for HSM functionality

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*,
hsm_uint32_t *u32BitCnt*)**

Ecc sign setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerify (HSM_Ln_RtCtxType * *pLnCtx*, const
HSM_Ln_EccVerifyType * *pCfg*)**

Ecc verify function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Ecc verify config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifyExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Ecc verify exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifyPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Ecc verify poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifySetupEnv (HSM_Ln_RtCtxType * *pLnCtx*,
hsm_uint32_t *u32BitCnt*)**

Ecc verify setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>u32BitCnt</i>	Byte count

**CRYPTO_TEXT_SECTION void HSM_Ln_Init (HSM_Ln_RtCtxType * *pLnRt*, HSM_Ln_InitParmType *
pInitParm)**

Init config of hsm.

Parameters

<i>pLnRt</i>	Hsm handle for HSM functionality
<i>pInitParm</i>	The init parameter of the Hsm.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopy (HSM_Ln_RtCtxType * *pLnCtx*, const
HSM_Ln_CopyUserKeyType * *pCfg*)**

Copy keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Copy key config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopyExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Exit an environment for copy keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopySetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Setup an environment for copy keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImport (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_ImportUserKeyExtType * *pCfg*)

Import key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Import key config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImportExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Exit an environment for import keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImportSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Setup an environment for import keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevoke (HSM_Ln_RtCtxType * *pLnCtx*,
hsm_uint32_t *u32KeyId*)**

Revoke keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>u32KeyId</i>	Key id

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevokeExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Exit an environment for revoke keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevokeSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Setup an environment for revoke keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUp (HSM_Ln_RtCtxType * *pLnCtx*,
HSM_Ln_KeySpaceStatusType * *pCfg*)**

Tidy up key space.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Tidy up key space config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUpExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Exit an environment for tidy up key space.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUpSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Setup an environment for tidy up key space.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidate (HSM_Ln_RtCtxType * *pLnCtx*,
hsm_uint32_t *u32KeyId*)**

Valid key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>u32KeyId</i>	Key id

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidateExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Exit an environment for valid keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidateSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Setup an environment for valid keys.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5 (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_Md5Type * *pCfg*)

MD5 function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	MD5 config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5ExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

MD5 exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5Poll (HSM_Ln_RtCtxType * *pLnCtx*)

MD5 poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5SetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

MD5 setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKey (HSM_Ln_RtCtxType * *pLnCtx*,
hsm_uint32_t *u32KeyId*)**

Revoke key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>u32KeyId</i>	Key id

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKeyExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Exit an environment for revoke key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKeySetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Setup an environment for revoke key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Rng (HSM_Ln_RtCtxType * *pLnCtx*, const
HSM_Ln_RngType * *pCfg*)**

Rng function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Rng config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Rng exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Rng poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Rng setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignData (HSM_Ln_RtCtxType * *pCtx*, const HSM_Ln_RsaSsaPkcs1V15SignDataType * *pCfg*, hsm_uint32_t *u32TimeoutMs*)

RsaSsa-pss-Pkcs-1v15 sign.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Sign date type
<i>u32TimeoutMs</i>	Timeout period

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignExitEnv (HSM_Ln_RtCtxType * *pCtx*)

Exit an environment for revoke key.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
-------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignHash (HSM_Ln_RtCtxType * *pCtx*, const HSM_Ln_RsaSsaPkcs1V15SignHashType * *pCfg*, hsm_uint32_t *u32TimeoutMs*)

RsaSsa-pss-Pkcs-1v15 sign hash.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Sign date hash type
<i>u32TimeoutMs</i>	Timeout period

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignSetupEnv (HSM_Ln_RtCtxType * *pCtx*)

Setup an environment for revoke key.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
-------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15Verify (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_RsaSsaPkcs1v15VerifyType * *pCfg*)

Rsassa pkcs1v15 verify function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Rsassa pkcs1v15 config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifyExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Rsassa pkcs1v15 verify exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifyPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Rsassa pkcs1v15 verify poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifySetupEnv (HSM_Ln_RtCtxType * *pLnCtx*, hsm_uint32_t *u32RsaBitCnt*)

Rsassa pkcs1v15 verify setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>u32RsaBitCnt</i>	RSA byte length

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignData (HSM_Ln_RtCtxType * *pCtx*, const HSM_Ln_RsaSsaPssSignDataType * *pCfg*, hsm_uint32_t *u32TimeoutMs*)

RsaSsa-pss sign.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Sign date type

<i>u32TimeoutMs</i>	Timeout period
---------------------	----------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignExitEnv (HSM_Ln_RtCtxType * *pCtx*)

Exit an environment for RsaSsa-pss sign.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Sign date type
<i>u32TimeoutMs</i>	Timeout period

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignHash (HSM_Ln_RtCtxType * *pCtx*,
const HSM_Ln_RsaSsaPssSignHashType * *pCfg*, hsm_uint32_t *u32TimeoutMs*)**

RsaSsa-pss sign hash.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Sign date type
<i>u32TimeoutMs</i>	Timeout period

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignSetupEnv (HSM_Ln_RtCtxType * *pCtx*)

Setup an environment for RsaSsa-pss sign.

Parameters

<i>pCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Sign date type
<i>u32TimeoutMs</i>	Timeout period

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerify (HSM_Ln_RtCtxType * *pLnCtx*,
const HSM_Ln_RsaSsaPssVerifyType * *pCfg*)**

Rsassa pss verify function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Rsassa pss config parameter

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifyExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Rsassa pss verify exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifyPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Rsassa pss verify poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifySetupEnv (HSM_Ln_RtCtxType * *pLnCtx*, hsm_uint32_t *u32RsaBitCnt*)

Rsassa pss verify setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>u32RsaBitCnt</i>	RSA byte length

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Scatter cmac exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacFinal (HSM_Ln_RtCtxType * *pLnCtx*,
const HSM_Ln_ScatterCMacType * *pCfg*, const HSM_Ln_ResultBuflInfoType * *pResult*)**

Scatter cmac final function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Scatter cmac config parameter
<i>pResult</i>	Scatter cmac result

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacInit (HSM_Ln_RtCtxType * *pLnCtx*, const
HSM_Ln_CMacCfgParmsType * *pParmCfg*, const HSM_Ln_ScatterCMacType * *pCfg*)**

Scatter cmac init function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Scatter cmac config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Scatter cmac poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Scatter cmac setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacUpdate (HSM_Ln_RtCtxType * *pLnCtx*,
const HSM_Ln_ScatterCMacType * *pCfg*)**

Scatter cmac update function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Scatter cmac config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Scatter hash exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashFinal (HSM_Ln_RtCtxType * *pLnCtx*,
const HSM_Ln_ScatterHashType * *pCfg*, const HSM_Ln_ResultBufInfoType * *pResult*)**

Scatter hash final function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

<i>pCfg</i>	Scatter hash config parameter
<i>pResult</i>	Scatter hash result

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashInit (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_ShaAlgType *eAlg*, const HSM_Ln_ScatterHashType * *pCfg*)

Scatter hash init function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Scatter hash config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Scatter hash poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashSetMacCfg (HSM_Ln_RtCtxType * *pLnCtx*, HSM_Ln_HashMacCfgType * *pCfg*)

Scatter hash set mac config function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Scatter hash config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Scatter hash setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashUpdate (HSM_Ln_RtCtxType * *pLnCtx*,
const HSM_Ln_ScatterHashType * *pCfg*)**

Scatter hash update function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Scatter hash config parameter

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sha (HSM_Ln_RtCtxType * *pLnCtx*, const
HSM_Ln_ShaType * *pSha*)**

Sha function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pSha</i>	sha config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Sha verify exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaPoll (HSM_Ln_RtCtxType * *pLnCtx*)

Sha poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Sha verify setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

Returns

hsm status

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2Verify (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_Sm2VerifyType * *pCfg*)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifyExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifyPoll (HSM_Ln_RtCtxType * *pLnCtx*)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifySetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3 (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_Sm3Type * *pCfg*)

SM3 function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	SM3 config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3ExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

SM3 exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3Poll (HSM_Ln_RtCtxType * *pLnCtx*)

SM3 poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3SetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

SM3 setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4Decrypt (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_Sm4DecryptType * *pCfg*)

SM4 decrypt function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	SM4 decrypt config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

SM4 decrypt exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptPoll (HSM_Ln_RtCtxType * *pLnCtx*)

SM4 decrypt poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

SM4 decrypt setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4Encrypt (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_Sm4EncryptType * *pCfg*)

SM4 encrypt function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	SM4 encrypt config parameter

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

SM4 encrypt exit environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptPoll (HSM_Ln_RtCtxType * *pLnCtx*)

SM4 encrypt poll function.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptSetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

SM4 encrypt setup environment.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKey (HSM_Ln_RtCtxType * *pLnCtx*, const HSM_Ln_PlainKeyImport * *pCfg*)

Update key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
<i>pCfg</i>	Update key

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKeyExitEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Exit an environment for update key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKeySetupEnv (HSM_Ln_RtCtxType * *pLnCtx*)

Setup an environment for update key.

Parameters

<i>pLnCtx</i>	Hsm handle for HSM functionality
---------------	----------------------------------

Module_driver_intm

Classes

struct _INTM_HandleType

The structure of the INTM handle.

struct INTM_InitType

The structure of the INTM init.

Typedefs

```
typedef struct _INTM_HandleType INTM_HandleType
```

The structure of the INTM handle.

```
typedef void(* INTM_ISRCallbackType) (INTM_HandleType *pHandle)
```

INTM Channel ISR callback function prototype.

Enumerations

```
enum INTM_IrqMonitorType { INTM_IRQ_MONITOR_0 = 0U, INTM_IRQ_MONITOR_1,  
INTM_IRQ_MONITOR_2, INTM_IRQ_MONITOR_3, INTM_IRQ_MONITOR_MAX }
```

The INTM monitor.

```
enum INTM_InterruptModeType { INTM_INTERRUPT_MODE_ACTIVE = 0x00U,  
INTM_INTERRUPT_MODE_INACTIVE }
```

Functions

```
void INTM_Init (INTM_HandleType *pIntmHandle, const INTM_InitType *const pInitCfg)
```

Init the INTM.

```
void INTM_enable (INTM_HandleType *pIntmHandle, const bool bEnable)
```

Enable INTM.

```
void INTM_StartInactiveMode (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

Start INTM inactive mode.

```
void INTM_StopInactiveMode (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

```
void INTM_SetAcknowledge (INTM_HandleType *pIntmHandle, const uint16_t u16IrqNumber)
```

Write the interrupt acknowledge.

```
uint32_t INTM_GetCounterValue (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

Return counter value.


```
void INTM_ClearIntFlag (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

Clear the interrupt flag.

```
bool INTM_GetTimeoutStatus (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

Get the timeout flag. Return true if INTM_TMR value has exceeded the INTM_LATR value.

```
void INTM_IRQProcess (INTM_HandleType *pIntmHandle)
```

intm process interrupt function

INTM API Service IDs

```
#define INTM_INIT_ID 0U  
#define INTM_ENABLE_ID 1U  
#define INTM_START_INACTIVE_ID 2U  
#define INTM_STOP_INACTIVE_ID 3U  
#define INTM_SET_ACKNOWLEDGE_ID 4U  
#define INTM_GET_COUNTER_ID 5U  
#define INTM_CLEAR_INT_FLAG_ID 6U  
#define INTM_GET_TIMEOUT_STAT_ID 7U  
#define INTM_IRQ_PRCE_ID 8U
```

INTM Dev Error Code

Error Code of calling INTM apis

```
#define INTM_E_PARAM_INSTANCE 0x01U  
#define INTM_E_PARAM_POINTER 0x02U  
#define INTM_E_PARAM_PARAM 0x03U  
#define INTM_E_PARAM_NO_SUPPORT 0x04U
```

Detailed Description

Macro Definition Documentation

#define INTM_CLEAR_INT_FLAG_ID 6U

#define INTM_E_PARAM_INSTANCE 0x01U

#define INTM_E_PARAM_NO_SUPPORT 0x04U

#define INTM_E_PARAM_PARAM 0x03U

#define INTM_E_PARAM_POINTER 0x02U

#define INTM_ENABLE_ID 1U

#define INTM_GET_COUNTER_ID 5U

#define INTM_GET_TIMEOUT_STAT_ID 7U

#define INTM_INIT_ID 0U

#define INTM_IRQ_PRCE_ID 8U

#define INTM_SET_ACKNOWLEDGE_ID 4U

#define INTM_START_INACTIVE_ID 2U

#define INTM_STOP_INACTIVE_ID 3U

Typedef Documentation

typedef struct _INTM_HandleType INTM_HandleType

The structure of the INTM handle.

typedef void(* INTM_ISRCallbackType) (INTM_HandleType *pHandle)

INTM Channel ISR callback function prototype.

Enumeration Type Documentation

enum INTM_InterruptModeType

Enumerator:

INTM_INTERRUPT_MODE_ACTIVE	INTM is configured as active mode
INTM_INTERRUPT_MODE_INACTIVE	INTM is configured as inactive mode

enum INTM_IrqMonitorType

The INTM monitor.

Enumerator:

INTM_IRQ_MONITOR_0	
INTM_IRQ_MONITOR_1	
INTM_IRQ_MONITOR_2	
INTM_IRQ_MONITOR_3	
INTM_IRQ_MONITOR_MAX	

Function Documentation

void INTM_ClearIntFlag (INTM_HandleType * *pIntmHandle*, const INTM_IrqMonitorType *elrqMonitorIndex*)

Clear the interrupt flag.

Parameters

<i>pIntmHandle</i>	INTM instance.
<i>elrqMonitorInde</i> <i>x</i>	Monitor index.

void INTM_enable (INTM_HandleType * *pIntmHandle*, const bool *bEnable*)

Enable INTM.

Parameters

<i>pIntmHandle</i>	INTM instance.
<i>bEnable</i>	Enable INTM.

uint32_t INTM_GetCounterValue (INTM_HandleType * *pIntmHandle*, const INTM_IrqMonitorType *elrqMonitorIndex*)

Return counter value.

Parameters

<i>pIntmHandle</i>	INTM instance.
<i>elrqMonitorInde</i> <i>x</i>	Monitor index.

Returns

Counter value.

bool INTM_GetTimeoutStatus (INTM_HandleType * *pIntmHandle*, const INTM_IrqMonitorType *elrqMonitorIndex*)

Get the timeout flag. Return true if INTM_TMR value has exceeded the INTM_LATR value.

Parameters

<i>pIntmHandle</i>	INTM instance.
--------------------	----------------

<i>elrqMonitorIndex</i>	Monitor index.
-------------------------	----------------

Returns

Timeout flag.

void INTM_Init (INTM_HandleType * *pIntmHandle*, const INTM_InitType *const *pInitCfg*)

Init the INTM.

Parameters

<i>pIntmHandle</i>	INTM instance.
<i>pInitCfg</i>	Init Configuration.

void INTM_IRQProcess (INTM_HandleType * *pIntmHandle*)

intm process interrupt function

Parameters

<i>pIntmHandle</i>	INTM instance.
--------------------	----------------

void INTM_SetAcknowledge (INTM_HandleType * *pIntmHandle*, const uint16_t *u16IrqNumber*)

Write the interrupt acknowledge.

Parameters

<i>pIntmHandle</i>	INTM instance.
<i>u16IrqNumber</i>	Interrupt number.

void INTM_StartInactiveMode (INTM_HandleType * *pIntmHandle*, const INTM_IrqMonitorType *elrqMonitorIndex*)

Start INTM inactive mode.

Parameters

<i>plntmHandle</i>	INTM instance.
<i>elrqMonitorInde</i> <i>x</i>	Monitor index.

void INTM_StopInactiveMode (INTM_HandleType * *plntmHandle*, const INTM_IrqMonitorType *elrqMonitorIndex*)

Parameters

<i>plntmHandle</i>	INTM instance.
<i>elrqMonitorInde</i> <i>x</i>	Monitor index.

Module_driver_ism

Collaboration diagram for Module_driver_ism:Modules

HwA_ism

Classes

```
struct _ISM_HandleType
struct ISM_FpcCfgType
struct ISM_LamCfgType
struct ISM_InitCfgType
```

Macros

```
#define ISM_ECM_CHANNEL(x) (1<<(x))
#define ISM_LAM_CHANNEL(x) (1<<(x))
#define ISM_FPC_CHANNEL(x) (1<<(x))
#define ECM_INSTANCE_COUNT (4U)
#define ISM_COMMON_INIT_ID 0U
#define ISM_FPC_CONFIG_ID 1U
#define ISM_FPC_ENABLE_ID 2U
#define ISM_LAM_CONFIG_ID 3U
#define ISM_LAM_ENABLE_ID 4U
#define ISM_ECM_CONFIG_ID 5U
#define ISM_STATUS_FUNCTION_ERROR 0x0U
```

```
#define ISM_E_PARAM_CHANNEL 0x02U
#define ISM_E_PARAM_POINTER 0x03U
#define ISM_E_PARAM_COUNT 0x04U
#define ISM_E_PARAM_INVALID 0x05U
```

Typedefs

```
typedef struct _ISM_HandleType ISM_HandleType
```

Enumerations

```
enum ISM_InstanceType { ISM_INSTANCE_0 = 0U }
enum FPC_GlitchDetectType { FPC_RISING_GLITCH_DETECTED = 0x01U,
FPC_FALLING_GLITCH_DETECTED = 0x02U }
```

Functions

```
void ISM_CommonInit (ISM_HandleType *pIsmHandle, const ISM_InitCfgType *pInitConfig)
```

Init the ISM.

```
void ISM_FpcConfig (ISM_HandleType *pIsmHandle, uint8_t u8FpcIndex, const ISM_FpcCfgType *pConfig)
```

Config the FPC channel.

```
void ISM_FpcEnable (ISM_HandleType *pIsmHandle, uint8_t u8FpcIndex, bool bEnable)
```

Enable FPC channel.

```
void ISM_LamConfig (ISM_HandleType *pIsmHandle, uint8_t u8LamIndex, const ISM_LamCfgType
*pConfig)
```

Config the LAM channel.

```
void ISM_LamEnable (ISM_HandleType *pIsmHandle, uint8_t u8LamIndex, bool bEnable)
```

Enable LAM Channel.

```
void ISM_EcmEventConfig (ISM_HandleType *pIsmHandle, uint8_t u8LamIndex, uint8_t u8EcmlIndex,
uint8_t u8EventCount)
```

Enable LAM system event.

```
uint8_t ISM_GetEcmEventHappenedChannels (void)
```

Get the channels mask of ECM that has event happened.

uint16_t **ISM_GetLamEventHappenedChannels** (void)

Get the channels mask of LAM that has event happened.

void **ISM_ClearEcmEventHappenedChannels** (uint8_t u8Channels)

Clear the channels status of ECM that has event happened.

void **ISM_ClearLamEventHappenedChannels** (uint16_t u16Channels)

Clear the channels status of LAM that has event happened.

void **ISM_EnableEcmSystemEvent** (uint32_t u32Channels, bool bEnable)

Enable ECM system event.

void **ISM_EnableLamSystemEvent** (uint32_t u32Channels, bool bEnable)

Enable LAM system event.

void **ISM_ClearLamStatusCounter** (uint8_t u8LamIndex)

Clears the value of the LAM counter.

uint32_t **ISM_GetLamStatusCounter** (uint8_t u8LamIndex)

Gets the value of the LAM counter when a LAM event is triggered.

void **ISM_ClearLamStatusOvfl** (uint8_t u8LamIndex)

Clears LAM Timer Overflow Flag.

bool **ISM_GetLamStatusOvfl** (uint8_t u8LamIndex)

Gets LAM Timer Overflow Flag.

void **ISMn_IRQHandler** (ISM_HandleType *plsmHandle)

ISM Interrupt Process Function.

Detailed Description

Macro Definition Documentation

#define ECM_INSTANCE_COUNT (4U)

#define ISM_COMMON_INIT_ID 0U

#define ISM_E_PARAM_CHANNEL 0x02U

#define ISM_E_PARAM_COUNT 0x04U

#define ISM_E_PARAM_INVALID 0x05U

#define ISM_E_PARAM_POINTER 0x03U

#define ISM_ECM_CHANNEL(x) (1<<(x))

#define ISM_ECM_CONFIG_ID 5U

#define ISM_FPC_CHANNEL(x) (1<<(x))

#define ISM_FPC_CONFIG_ID 1U

#define ISM_FPC_ENABLE_ID 2U

#define ISM_LAM_CHANNEL(x) (1<<(x))

#define ISM_LAM_CONFIG_ID 3U

#define ISM_LAM_ENABLE_ID 4U

#define ISM_STATUS_FUNCTION_ERROR 0x0U

Typedef Documentation

typedef struct _ISM_HandleType ISM_HandleType

Enumeration Type Documentation

enum FPC_GlitchDetectType

Enumerator:

FPC_RISING_GLITCH_DETECTED	PFC Rising Glitch Detected.
FPC_FALLING_GLITCH_DETECTED	PFC Falling Glitch Detected.

enum ISM_InstanceType

Enumerator:

ISM_INSTANCE_0	ISM instance 0 is selected.
----------------	-----------------------------

Function Documentation

void ISM_ClearEcmEventHappenedChannels (uint8_t *u8Channels*)

Clear the channels status of ECM that has event happened.

Parameters

<i>u8Channels</i>	Channels mask.
-------------------	----------------

void ISM_ClearLamEventHappenedChannels (uint16_t *u16Channels*)

Clear the channels status of LAM that has event happened.

Parameters

<i>u16Channels</i>	Channels mask.
--------------------	----------------

void ISM_ClearLamStatusCounter (uint8_t *u8LamIndex*)

Clears the value of the LAM counter.

Parameters

<i>u32LamIndex</i>	Lam channel index.
--------------------	--------------------

void ISM_ClearLamStatusOvfl (uint8_t *u8LamIndex*)

Clears LAM Timer Overflow Flag.

Parameters

<i>u32LamIndex</i>	Lam channel index.
--------------------	--------------------

void ISM_CommonInit (ISM_HandleType * *pIsmHandle*, const ISM_InitCfgType * *pInitConfig*)

Init the ISM.

Parameters

<i>pIsmHandle</i>	Ism processing handle.
<i>pInitConfig</i>	ISMInstance initial configuration.

void ISM_EcmEventConfig (ISM_HandleType * *pIsmHandle*, uint8_t *u8LamIndex*, uint8_t *u8EcmIndex*, uint8_t *u8EventCount*)

Enable LAM system event.

Parameters

<i>pIsmHandle</i>	Ism processing handle.
<i>u8LamIndex</i>	Lam channel index.
<i>u8EcmIndex</i>	Ecm channel index.
<i>u8EventCount</i>	Threshold of the ECM channel counter value.

void ISM_EnableEcmSystemEvent (uint32_t *u32Channels*, bool *bEnable*)

Enable ECM system event.

Parameters

<i>u32Channels</i>	Channel masks.
<i>bEnable</i>	Enable value.

void ISM_EnableLamSystemEvent (uint32_t *u32Channels*, bool *bEnable*)

Enable LAM system event.

Parameters

<i>u32Channels</i>	Channel masks.
<i>bEnable</i>	Enable value.

void ISM_FpcConfig (ISM_HandleType * *plsmHandle*, uint8_t *u8FpcIndex*, const ISM_FpcCfgType * *pConfig*)

Config the FPC channel.

Parameters

<i>plsmHandle</i>	IsM processing handle.
<i>u8FpcIndex</i>	FPC index.
<i>pConfig</i>	FPC configuration.

void ISM_FpcEnable (ISM_HandleType * *plsmHandle*, uint8_t *u8FpcIndex*, bool *bEnable*)

Enable FPC channel.

Parameters

<i>plsmHandle</i>	IsM processing handle.
<i>u8FpcIndex</i>	FPC index.
<i>bEnable</i>	Enable value.

uint8_t ISM_GetEcmEventHappenedChannels (void)

Get the channels mask of ECM that has event happened.

Returns

Channels mask.

uint16_t ISM_GetLamEventHappenedChannels (void)

Get the channels mask of LAM that has event happened.

Returns

Channels mask.

uint32_t ISM_GetLamStatusCounter (uint8_t *u8LamIndex*)

Gets the value of the LAM counter when a LAM event is triggered.

Parameters

<i>u32LamIndex</i>	Lam channel index.
--------------------	--------------------

Returns

Counter value.

bool ISM_GetLamStatusOvfl (uint8_t *u8LamIndex*)

Gets LAM Timer Overflow Flag.

Parameters

<i>u32LamIndex</i>	Lam channel index.
--------------------	--------------------

Returns

Overflow flag.

```
void ISM_LamConfig (ISM_HandleType * pIsmHandle, uint8_t u8LamIndex, const ISM_LamCfgType * pConfig)
```

Config the LAM channel.

Parameters

<i>pIsmHandle</i>	Ism processing handle.
<i>u8LamIndex</i>	LAM index.
<i>pConfig</i>	LAM configuration.

```
void ISM_LamEnable (ISM_HandleType * pIsmHandle, uint8_t u8LamIndex, bool bEnable)
```

Enable LAM Channel.

Parameters

<i>pIsmHandle</i>	Ism processing handle.
<i>u8LamIndex</i>	LAM index.
<i>bEnable</i>	Enable value.

```
void ISMn_IRQHandler (ISM_HandleType * pIsmHandle)
```

ISM Interrupt Process Function.

Parameters

<i>pIsmHandle</i>	Ism processing handle.
-------------------	------------------------

Module_driver_lin

Classes

struct **LIN_PduType**

LIN Protocol Data Unit (PDU) structure.

struct **LIN_BaudRateCfgType**

Baudrate configuration parameters.

struct **LIN_XferStateType**

Runtime state of the LIN driver (maintained during operation)

struct **LIN_ConfigType**

LIN controller hardware configuration parameters.

struct **_LIN_HandleType**

LIN driver handle structure (entry point for all API operations)

Typedefs

typedef void(* **LIN_TimestampCallbackType**) (uint64_t *microseconds)

Callback function type to get time interval in microseconds.

typedef uint8 **LIN_FrameDlType**

Data length of LIN frame (number of data bytes in SDU)

typedef uint8 **LIN_FramePidType**

LIN protected identifier (PID) type.

typedef void(* **LIN_CallbackType**) (uint8_t u8Lin, **LIN_EventIdType** event, **LIN_PduType** *pdu, uint8_t *checksum)

LIN driver callback function type.

typedef struct **_LIN_HandleType** **LIN_HandleType**

LIN driver handle structure (entry point for all API operations)

Enumerations

enum **LIN_ReturnType** { **LIN_RET_SUCCESS** = 0x00U, **LIN_RET_STATE_ERROR**, **LIN_RET_BUSY**, **LIN_RET_TIMEOUT**, **LIN_RET_PARAM_ERROR**, **LIN_RET_NOT_INIT**, **LIN_RET_USEED**, **LIN_RET_NOT_START**, **LIN_RET_UNSUPPORTED** }

LIN API return status type.

enum **LIN_NodeType** { **LIN_NODE_MASTER**, **LIN_NODE_SLAVE** }

LIN node type (master or slave)

enum **LIN_EventIdType** { **LIN_NO_EVENT** = 0x00U, **LIN_WAKEUP_SIGNAL**, **LIN_RECV_BREAK_FIELD_OK**, **LIN_SYNC_ERROR**, **LIN_PID_ERROR**, **LIN_HEADER_OK**, **LIN_HEADER_ERROR**, **LIN_FRAME_ERROR**,

LIN_READBACK_ERROR, LIN_REPONSE_ERROR, LIN_CHECKSUM_ERROR, LIN_TX_COMPLETED, LIN_RX_COMPLETED, LIN_RX_OVERRUN }

LIN event identifier types.

enum LIN_FrameStateType { LIN_FRAME_SLEEP_STATE = 0x00U, LIN_FRAME_IDLE_STATE, LIN_FRAME_BREAK_SEND_STATE, LIN_FRAME_BREAK_RECV_STATE, LIN_FRAME_HEADER_SEND_STATE, LIN_FRAME_HEADER_RECV_STATE, LIN_FRAME_RESPONSE_SEND_STATE, LIN_FRAME_RESPONSE_RECV_STATE, LIN_FRAME_RESPONSE_SEND_OK_STATE, LIN_FRAME_RESPONSE_RECV_OK_STATE }

LIN frame processing state definitions (describes current frame processing phase)

enum LIN_StatusType { LIN_NOT_OK = 0x00U, LIN_TX_HEADER_OK, LIN_TX_RESP_OK, LIN_TX_HEADER_BUSY, LIN_TX_RESP_BUSY, LIN_TX_HEADER_ERROR, LIN_TX_RESP_ERROR, LIN_RX_HEADER_OK, LIN_RX_RESP_OK, LIN_RX_HEADER_BUSY, LIN_RX_RESP_BUSY, LIN_RX_HEADER_ERROR, LIN_RX_RESP_ERROR, LIN_CALLBACK_ERROR, LIN_CH_OPERATIONAL, LIN_CH_SLEEP, LIN_CH_SLEEP_PENDING }

LIN node operational state definitions (describes overall node status)

enum LIN_FrameResponseType { LIN_FRAMERESPONSE_TX = 0, LIN_FRAMERESPONSE_RX, LIN_FRAMERESPONSE_IGNORE }

Frame response types (specifies responsibility for frame response)

enum LIN_FrameCsModelType { LIN_ENHANCED_CS = 0U, LIN_CLASSIC_CS }

Checksum models for LIN frames.

LIN API Service IDs

Defines the service IDs for the LIN API (used in development error reporting)

```
#define LIN_INIT_ID 0U
#define LIN_DEINIT_ID 1U
#define LIN_GET_DEFAULT_CFG_ID 2U
#define LIN_DRV_SEND_FRAME_ID 3U
#define LIN_DRV_GOTO_SLEEP_ID 4U
#define LIN_DRV_GOTO_SLEEP_INTERNAL_ID 5U
#define LIN_DRV_WAKEUP_ID 6U
#define LIN_DRV_WAKEUP_INTERNAL_ID 7U
#define LIN_DRV_GET_STATUS_ID 8U
#define LIN_DRV_ABORT_TRANSFER_ID 9U
#define LIN_DRV_IRQ_HANDLE 10U
```

LIN Dev Error Code

Error codes for LIN API parameter validation


```
#define LIN_E_PARAM_INSTANCE 0x01U
#define LIN_E_PARAM_POINTER 0x02U
#define LIN_E_PARAM_ERROR_STATE 0x03U
#define LIN_E_PARAM_ERROR_CALL 0x04U
```

API declaration for LIN driver.

LIN driver APIs

LIN_ReturnType LIN_DrvInit (LIN_HandleType *pLinHandle, LIN_ConfigType *pConfig)

Initialize LIN controller instance.

void LIN_DrvDeInit (LIN_HandleType *pLinHandle)

De-initialize LIN controller instance.

void LIN_DrvGetDefaultConfig (LIN_NodeType eNodeMode, LIN_ConfigType *pConfig)

Initialize default LIN node configuration parameters.

LIN_ReturnType LIN_DrvSendFrame (LIN_HandleType *pLinHandle, LIN_PduType *pLinPdu)

Send a LIN frame over the bus (Master/Slave mode).

LIN_ReturnType LIN_DrvGoToSleep (LIN_HandleType *pLinHandle)

Command LIN master node to initiate sleep mode sequence.

LIN_ReturnType LIN_DrvGoToSleepInternal (LIN_HandleType *pLinHandle)

Internal implementation to transition LIN node to sleep state.

LIN_ReturnType LIN_DrvWakeup (LIN_HandleType *pLinHandle)

Send wake-up signal to LIN bus to rouse all nodes from sleep mode.

LIN_ReturnType LIN_DrvWakeupInternal (LIN_HandleType *pLinHandle)

Internal implementation to wake up LIN node from sleep state.

LIN_StatusType LIN_DrvGetStatus (LIN_HandleType *pLinHandle)

Get current LIN node operational status.

LIN_ReturnType LIN_DrvAbortTransfer (LIN_HandleType *pLinHandle)

Abort ongoing LIN transfer and reset node to operational state.

void LIN_DrvIRQHandler (LIN_HandleType *pLinHandle)

LIN controller interrupt service routine (ISR)

uint8_t LIN_DrvParityMake (uint8_t PID)

Generate LIN PID parity bits according to LIN protocol specification.

uint8_t LIN_DrvParityCheck (uint8_t PID)

Validate LIN PID parity bits according to LIN protocol specification.

Detailed Description

Macro Definition Documentation

#define LIN_DEINIT_ID 1U

De-initialize LIN instance

#define LIN_DRV_ABORT_TRANSFER_ID 9U

Abort LIN node transfer

#define LIN_DRV_GET_STATUS_ID 8U

Get current LIN node status

#define LIN_DRV_GOTO_SLEEP_ID 4U

Command LIN node to enter sleep mode

#define LIN_DRV_GOTO_SLEEP_INTERNAL_ID 5U

Internal sleep mode transition

#define LIN_DRV_IRQ_HANDLE 10U

LIN interrupt service routine

#define LIN_DRV_SEND_FRAME_ID 3U

Send LIN frame

#define LIN_DRV_WAKEUP_ID 6U

Send wake-up signal to LIN bus

#define LIN_DRV_WAKEUP_INTERNAL_ID 7U

Internal wake-up mode transition

#define LIN_E_PARAM_ERROR_CALL 0x04U

Invalid API call (e.g., slave node calling master-only API)

#define LIN_E_PARAM_ERROR_STATE 0x03U

Node in invalid state for operation

#define LIN_E_PARAM_INSTANCE 0x01U

Invalid LIN hardware instance index

#define LIN_E_PARAM_POINTER 0x02U

NULL or invalid pointer parameter

#define LIN_GET_DEFAULT_CFG_ID 2U

Get default LIN configuration

#define LIN_INIT_ID 0U

Initialize LIN instance

Typedef Documentation

typedef void(* LIN_CallbackType) (uint8_t u8Lin, LIN_EventIdType event, LIN_PduType *pdu, uint8_t *checksum)

LIN driver callback function type.

Parameters

<i>u8Lin</i>	LIN controller channel ID
<i>event</i>	LIN event identifier (e.g., TX completed, RX error)
<i>pdu</i>	Pointer to the processed Protocol Data Unit
<i>checksum</i>	Pointer to the calculated/validated checksum value

typedef uint8 LIN_FrameDlType

Data length of LIN frame (number of data bytes in SDU)

Valid range: 0-8 bytes (per LIN protocol specification)

typedef uint8 LIN_FramePidType

LIN protected identifier (PID) type.

8-bit value where bits 0-5 are the raw ID (0x00-0x3F), bits 6-7 are parity bits

typedef struct _LIN_HandleType LIN_HandleType

LIN driver handle structure (entry point for all API operations)

Connects configuration, runtime state, and hardware instance.

typedef void(* LIN_TimestampCallbackType) (uint64_t *microseconds)

Callback function type to get time interval in microseconds.

Parameters

<i>microseconds</i>	Pointer to store the time interval value (unit: microseconds)
---------------------	---

Enumeration Type Documentation

enum LIN_EventIdType

LIN event identifier types.

Enumerator:

LIN_NO_EVENT	No event occurred
LIN_WAKEUP_SIGNAL	Wake-up signal detected
LIN_RECV_BREAK_FIELD_OK	Break field received successfully
LIN_SYNC_ERROR	Sync field validation failed
LIN_PID_ERROR	PID validation failed
LIN_HEADER_OK	Frame header received successfully
LIN_HEADER_ERROR	Frame header reception failed
LIN_FRAME_ERROR	Frame reception error
LIN_READBACK_ERROR	Data readback mismatch
LIN_RESPONSE_ERROR	Response reception error
LIN_CHECKSUM_ERROR	Checksum validation failed
LIN_TX_COMPLETED	Transmission completed successfully
LIN_RX_COMPLETED	Reception completed successfully
LIN_RX_OVERRUN	RX buffer overrun occurred

enum LIN_FrameCsModelType

Checksum models for LIN frames.

LIN protocol defines two checksum calculation methods:

Enhanced: Includes PID in checksum calculation

Classic: Excludes PID from checksum calculation

Enumerator:

LIN_ENHANCED_CS	Enhanced checksum model (includes PID)
LIN_CLASSIC_CS	Classic checksum model (excludes PID)

enum LIN_FrameResponseType

Frame response types (specifies responsibility for frame response)

Determines if the node generates, receives, or ignores the frame response

Enumerator:

LIN_FRAMERESPONSE_TX	Response is generated by this node (transmit responsibility)
LIN_FRAMERESPONSE_RX	Response is received from another node (relevant to this node)
LIN_FRAMERESPONSE_IGNORE	Response is received from another node (irrelevant to this node)

enum LIN_FrameStateType

LIN frame processing state definitions (describes current frame processing phase)

Enumerator:

LIN_FRAME_SLEEP_STATE	Frame processing in sleep state (no active communication)
-----------------------	---

LIN_FRAME_IDLE_STATE	Frame processing idle (ready to start new transfer)
LIN_FRAME_BREAK_SEND_STATE	Sending break field (13-bit low signal)
LIN_FRAME_BREAK_RECV_STATE	Receiving break field (13-bit low signal)
LIN_FRAME_HEADER_SEND_STATE	Sending frame header (break + sync + PID)
LIN_FRAME_HEADER_RECV_STATE	Receiving frame header (break + sync + PID)
LIN_FRAME_RESPONSE_SEND_STATE	Sending frame response (data + checksum)
LIN_FRAME_RESPONSE_RECV_STATE	Receiving frame response (data + checksum)
LIN_FRAME_RESPONSE_SEND_OK_STATE	Response sent successfully (no errors detected)
LIN_FRAME_RESPONSE_RECV_OK_STATE	Response received successfully (no errors detected)

enum LIN_NodeType

LIN node type (master or slave)

Enumerator:

LIN_NODE_MASTER	LIN Master node (initiates communication)
LIN_NODE_SLAVE	LIN Slave node (responds to master)

enum LIN_ReturnType

LIN API return status type.

Enumerator:

LIN_RET_SUCCESS	API executed successfully
LIN_RET_STATE_ERROR	API execution error (invalid node state)
LIN_RET_BUSY	Node is currently busy (e.g., transmitting)
LIN_RET_TIMEOUT	Operation timed out
LIN_RET_PARAMETER_ERROR	Invalid parameter provided
LIN_RET_NOT_INITIALIZED	LIN node has not been initialized
LIN_RET_USEED	LIN node is already in use
LIN_RET_NOT_STARTED	LIN node has not been started
LIN_RET_UNSUPPORTED	Unsupported operation

enum LIN_StatusType

LIN node operational state definitions (describes overall node status)

Enumerator:

LIN_NOT_OK	Node not initialized or in error state
LIN_TX_HEADER_OK	Transmit header data ready for operation

LIN_TX_RESP_OK	Transmit response data ready for operation
LIN_TX_HEADER_BUSY	Transmitting header
LIN_TX_RESP_BUSY	Transmitting response
LIN_TX_HEADER_ERROR	Header transmission error occurred
LIN_TX_RESP_ERROR	Response transmission error occurred
LIN_RX_HEADER_OK	Receive header data ready for operation
LIN_RX_RESP_OK	Receive response ready for operation
LIN_RX_HEADER_BUSY	Receiving header
LIN_RX_RESP_BUSY	Receiving response
LIN_RX_HEADER_ERROR	Header receive error occurred
LIN_RX_RESP_ERROR	Response receive error occurred
LIN_CALLBACK_ERROR	Expected callback not configured
LIN_CH_OPERATIONAL	Node in operational mode (active communication)
LIN_CH_SLEEP	Node in sleep mode (low power)
LIN_CH_SLEEP_PENDING	Sleep mode transition pending (waiting for RX completion)

--	--

Function Documentation

LIN_ReturnType LIN_DrvAbortTransfer (LIN_HandleType * *pLinHandle*)

Abort ongoing LIN transfer and reset node to operational state.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure
-------------------	---------------------------------

Returns

Operation status:

- LIN_RET_SUCCESS: Transfer aborted successfully
- LIN_RET_STATE_ERROR: Invalid state for abort operation

Performs hardware-level abort operations including: • Disabling receive interrupts • Clearing status flags • Flushing Tx/Rx FIFOs

For slave nodes, re-enables LIN break detection

Resets internal state machines to operational/idle state

Validates handle and hardware index via LIN_DEV_ERROR_REPORT

void LIN_DrvDeInit (LIN_HandleType * *pLinHandle*)

De-initialize LIN controller instance.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure
-------------------	---------------------------------

Resets FCUART hardware to default state

Clears internal pointers in LIN handle (pCfg, pXfer)

Disables interrupts and stops TX/RX transfer

- **Note**

This function does not return a status; error checking is done via LIN_DEV_ERROR_REPORT

void LIN_DrvGetDefaultConfig (LIN_NodeType *eNodeMode*, LIN_ConfigType * *pConfig*)

Initialize default LIN node configuration parameters.

Parameters

<i>eNodeMode</i>	LIN node mode (LIN_NODE_MASTER or LIN_NODE_SLAVE)
<i>pConfig</i>	Pointer to configuration structure to be initialized

Sets default values for:

nodeMode: Input node mode (master/slave)

baudRate: Default baudrate (19200 bps)

timeCallback: NULL (user must set if needed)

xferState: Initial state (LIN_NOT_OK, LIN_FRAME_SLEEP_STATE)

● Note

pConfig must be a valid pointer (checked via LIN_DEV_ERROR_REPORT)

LIN_StatusType LIN_DrvGetStatus (LIN_HandleType * *pLinHandle*)

Get current LIN node operational status.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure
-------------------	---------------------------------

Returns

Current node state:

- LIN_CH_SLEEP: Node in sleep mode
- LIN_CH_OPERATIONAL: Node in active mode
- LIN_TX_BUSY/LIN_RX_BUSY: Transmitting/receiving data

Converts internal transfer state (curChnState) to user-visible status

Handles state transition from sleep-pending to sleep mode

Validates handle and hardware index via LIN_DEV_ERROR_REPORT

LIN_ReturnType LIN_DrvGoToSleep (LIN_HandleType * *pLinHandle*)

Command LIN master node to initiate sleep mode sequence.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure (contains configuration and state)
-------------------	--

Returns

Operation status:

- LIN_RET_SUCCESS: Sleep sequence initiated successfully
- LIN_RET_STATE_ERROR: Node state is invalid for sleep
- LIN_RET_ERROR: Parameter validation failed (via LIN_DEV_ERROR_REPORT)

Validates input parameters and node state (must be operational)

Constructs a sleep frame (PID=0x3C, 8-byte payload: 0x00 followed by 0xFFs)

Sends header to start sleep sequence transmission

● Note

Only valid for master nodes (slave nodes cannot initiate sleep)

LIN_ReturnType LIN_DrvGoToSleepInternal (LIN_HandleType * *pLinHandle*)

Internal implementation to transition LIN node to sleep state.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure (contains configuration and state)
-------------------	--

Returns

Operation status:

- LIN_RET_SUCCESS: Sleep state transition completed
- LIN_RET_STATE_ERROR: Node state is invalid for sleep
- LIN_RET_ERROR: Parameter validation failed (via LIN_DEV_ERROR_REPORT)

Validates input parameters and node state (must be operational)

Configures control register to disable error interrupts

Flushes TX/RX FIFOs and clears data register

Updates frame/channel state to LIN_FRAME_SLEEP_STATE and LIN_CH_SLEEP

Enables receive active interrupt to detect wake-up signals

LIN_ReturnType LIN_DrvInit (LIN_HandleType * *pLinHandle*, LIN_ConfigType * *pConfig*)

Initialize LIN controller instance.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure (stores configuration and state)
<i>pConfig</i>	Pointer to LIN configuration structure (contains node mode, baudrate, etc.)

Returns

Operation status:

- LIN_RET_SUCCESS: Initialization succeeded
- LIN_RET_ERROR: Baudrate configuration failed

Configures hardware registers and initializes transfer state:

1. Validates input pointers and hardware index
2. Resets hardware to default state
3. Calculates and sets baudrate configuration (oversampling ratio and divider)
4. Configures FIFO (TX/RX enable, depth) and watermark levels
5. Sets control register (error interrupts, parity, mode)
6. Initializes transfer state (buffer pointer, frame/channel state)
7. Enables receive active interrupt and starts LIN controller

void LIN_DrvIRQHandler (LIN_HandleType * *pLinHandle*)

LIN controller interrupt service routine (ISR)

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure
-------------------	---------------------------------

Processes different types of interrupts based on hardware status:

1. Wake-up interrupt: Handles LIN bus wake-up signal detection
2. Transmission complete interrupt: Manages frame transmission completion
3. Break detection interrupt: Processes LIN bus break field reception
4. Data reception interrupt: Handles frame data reception
5. Error handling: Invokes error handler for unrecognized interrupts

Clears all interrupt flags after processing to prevent re-triggering

uint8_t LIN_DrvParityCheck (uint8_t *PID*)

Validate LIN PID parity bits according to LIN protocol specification.

Parameters

<i>PID</i>	Input 8-bit PID value (with parity bits in bits 6 and 7)
------------	--

Returns

Validated raw PID value (6-bit) or 0xFFU if parity check fails

Recalculates expected parity bits from input PID's data bits (0-5), compares with actual parity bits (6-7) in input PID:

If match: returns raw PID (bits 0-5)

If mismatch: returns 0xFFU (invalid PID)

uint8_t LIN_DrvParityMake (uint8_t *PID*)

Generate LIN PID parity bits according to LIN protocol specification.

Parameters

<i>PID</i>	Input raw PID value (6-bit, without parity bits)
------------	--

Returns

8-bit PID value with parity bits (bits 6 and 7)

Calculates two parity bits:

P0 (bit 6): XOR of PID bits 0,1,2,4

P1 (bit 7): XOR of PID bits 1,3,4,5 (inverted) Combines raw PID with parity bits to form final 8-bit identifier

LIN_ReturnType LIN_DrvSendFrame (LIN_HandleType * *pLinHandle*, LIN_PduType * *pLinPdu*)

Send a LIN frame over the bus (Master/Slave mode).

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure (contains configuration and state)
<i>pLinPdu</i>	Pointer to Protocol Data Unit (PDU) containing frame information (PID, data, etc.)

Returns

Operation status:

- LIN_RET_SUCCESS: Frame transmission initiated successfully
- LIN_RET_STATE_ERROR: Node state is invalid for transmission
- LIN_RET_ERROR: Parameter validation failed (via LIN_DEV_ERROR_REPORT)

Validates input parameters and node state

For master nodes: Prepares response buffer and sends header

Updates transfer state to LIN_TX_BUSY during transmission

Slave nodes: No action (only master initiates frame transmission)

LIN_ReturnType LIN_DrvWakeup (LIN_HandleType * *pLinHandle*)

Send wake-up signal to LIN bus to rouse all nodes from sleep mode.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure (contains configuration and state)
-------------------	--

Returns

Operation status:

- LIN_RET_SUCCESS: Wake-up signal sent successfully
- LIN_RET_NOT_INIT: LIN node is not initialized
- LIN_RET_ERROR: Parameter validation failed (via LIN_DEV_ERROR_REPORT)

Validates input parameters and node state (must be initialized)

Configures control register to enable error interrupts

Disables receive active interrupt to focus on wake-up signal transmission

Sends specific data (0x00 for baudrate >10000bps, 0xF8 otherwise) to generate 150us+ active level

Updates node state to LIN_CH_OPERATIONAL and frame state to LIN_FRAME_IDLE_STATE

- **Note**

Master node sends signal; slave nodes enable break detection interrupt after wake-up

LIN_ReturnType LIN_DrvWakeupInternal (LIN_HandleType * *pLinHandle*)

Internal implementation to wake up LIN node from sleep state.

Parameters

<i>pLinHandle</i>	Pointer to LIN handle structure (contains configuration and state)
-------------------	--

Returns

Operation status:

- LIN_RET_SUCCESS: Wake-up transition completed
- LIN_RET_NOT_INIT: LIN node is not initialized
- LIN_RET_ERROR: Parameter validation failed (via LIN_DEV_ERROR_REPORT)

Validates input parameters and node state (must be in sleep/sleep-pending state)

Configures control register to enable error interrupts (ORIE, FEIE)

Disables receive active interrupt to focus on operational mode

Clears status register flags and flushes TX/RX FIFOs

For slave nodes: Re-enables LIN break detection interrupt

Updates node state to LIN_CH_OPERATIONAL and frame state to LIN_FRAME_IDLE_STATE

Module_driver_lu

Collaboration diagram for Module_driver_lu:Modules

HwA_lu

Classes

struct **LU_InConfigType**

LU IN(n) configuration register type.

struct **LU_AoiConfigType**

LU AOI IN(n) configuration type.

struct **LU_InputsSyncCtrlType**

LU LG inputs synchronous control.

struct **LU_ConfigType**

LU initialization type.

Macros

```
#define LU_AOI_IN_N_CFG_N(AoiIn, AoiInN, InNCfgNType) (((uint32_t)(InNType) & 0x3U) << (((uint32_t)3U - (uint32_t)(AoiInN)) << 3U)) << (((uint32_t)3U - (uint32_t)(AoiIn)) << 3U))
```



```

#define LU_AOI_IN_N_CFG_N_MASK(AoInN, AoInN) (((uint32_t)0x3U << (((uint32_t)3U - (uint32_t)(AoInN)) << 3U)) << (((uint32_t)3U - (uint32_t)(AoInN)) << 3U))
#define LU_AOI_IN_CFG(AoInN, CfgNType) (((uint32_t)(CfgNType) & 0x3U) << (((uint32_t)3U - (uint32_t)(AoInN)) << 1U)))
#define LU_AOI_IN_CFG_MASK(AoInN) (((uint32_t)0x3U) << (((uint32_t)3U - (uint32_t)(AoInN)) << 1U)))
#define LU_AOI_IN_N_CFG(AoInN, InNType) (((uint32_t)(InNType) & 0xFFU) << (((uint32_t)3U - (uint32_t)(AoInN)) << 3U)))
#define LU_AOI_IN_N_CFG_MASK(AoInN) (((uint32_t)0xFFU) << (((uint32_t)3U - (uint32_t)(AoInN)) << 3U)))
#define LU_SYNC_CONTROL_INPUT_N(InputN, value) ((uint32_t)(value) << (InputN))

```

Enumerations

```
enum LU_InstanceType { LU_INSTANCE_0 = 0U, LU_INSTANCE_1 }
```

LU instance.

```
enum LU_StatusType { LU_STATUS_SUCCESS = 0U, LU_STATUS_PARAM_INVALID = 1U }
```

LU return structure.

```
enum LU_AoInType { LU_AOI_IN_0 = 0U, LU_AOI_IN_1, LU_AOI_IN_2, LU_AOI_IN_3 }
```

LU IN type.

```
enum LU_AoInNType { LU_AOI_IN_N_A = 0U, LU_AOI_IN_N_B, LU_AOI_IN_N_C, LU_AOI_IN_N_D }
```

LU IN(n) configuration type.

```
enum LU_InModeType { FORCE_ITEM_AS_ZERO = 0U, PASS_THROUGH_ITEM, COMPLEMENT_ITEM, FORCE_ITEM_AS_ONE }
```

LU IN(n) configuration.

```
enum LU_OutputInitValueType { LU_OUTPUT_INIT_ZERO = 0U, LU_OUTPUT_INIT_ONE, LU_OUTPUT_INIT_DISABLE }
```

LU output initialization value.

Functions

```
LU_StatusType LU_Init (const LU_InstanceType eLu, const LU_ConfigType *const pInitStruct)
```

Initialize LU instance.

```
void LU_Deinit (const LU_InstanceType eLu)
```

De-initialize LU instance.

CORDIC API Service IDs

```
#define LU_INIT_ID 0x00
```

```
#define LU_DEINIT_ID 0x01
```

LU Dev Error Code

```
#define LU_E_PARAM_INSTANCE 0x01U
```

```
#define LU_E_PARAM_NULLPTR 0x02U
```

Detailed Description

Macro Definition Documentation

```
#define LU_AOI_IN_CFG( AoilnN, CfgNType) (((uint32_t)(CfgNType) & 0x3U) << (((uint32_t)3U -  
(uint32_t)(AoilnN)) << 1U)))
```

```
#define LU_AOI_IN_CFG_MASK( AoilnN) (((uint32_t)0x3U) << (((uint32_t)3U - (uint32_t)(AoilnN)) << 1U)))
```

```
#define LU_AOI_IN_N_CFG( Aoiln, InNType) (((uint32_t)(InNType) & 0xFFU) << (((uint32_t)3U -  
(uint32_t)(Aoiln)) << 3U)))
```

```
#define LU_AOI_IN_N_CFG_MASK( Aoiln) (((uint32_t)0xFFU) << (((uint32_t)3U - (uint32_t)(Aoiln)) <<  
3U)))
```

```
#define LU_AOI_IN_N_CFG_N( Aoiln, AoilnN, InNCfgNType) (((uint32_t)(InNType) & 0x3U) <<  
(((uint32_t)3U - (uint32_t)(AoilnN)) << 3U)) << (((uint32_t)3U - (uint32_t)(Aoiln)) << 3U))
```

```
#define LU_AOI_IN_N_CFG_N_MASK( Aoiln, AoilnN) (((uint32_t)0x3U << (((uint32_t)3U -  
(uint32_t)(AoilnN)) << 3U)) << (((uint32_t)3U - (uint32_t)(Aoiln)) << 3U))
```

```
#define LU_DEINIT_ID 0x01
```

Service ID for LU deinitialization.

```
#define LU_E_PARAM_INSTANCE 0x01U
```

Error code for invalid LU instance parameter.

```
#define LU_E_PARAM_NULLPTR 0x02U
```

Error code for null pointer parameter.

```
#define LU_INIT_ID  0x00

Service ID for LU initialization.
```

```
#define LU_SYNC_CONTROL_INPUT_N( InputN,  value)  ((uint32_t)(value) << (InputN))
```

Enumeration Type Documentation

enum LU_AoiInNType

LU IN(n) configuration type.

Enumerator:

LU_AOI_IN_N_A	
LU_AOI_IN_N_B	
LU_AOI_IN_N_C	
LU_AOI_IN_N_D	

enum LU_AoiInType

LU IN type.

Enumerator:

LU_AOI_IN_0	
LU_AOI_IN_1	
LU_AOI_IN_2	
LU_AOI_IN_3	

enum LU_InModeType

LU IN(n) configuration.

Enumerator:

FORCE_ITEM_A S_ZERO	force item as logic zero
------------------------	--------------------------

PASS_THROUGH_ITEM	pass through item
COMPLEMENT_ITEM	complement item
FORCE_ITEM_AS_ONE	force item as logic one

enum LU_InstanceType

LU instance.

Enumerator:

LU_INSTANCE_0	
LU_INSTANCE_1	

enum LU_OutputInitValueType

LU output initialization value.

Enumerator:

LU_OUTPUT_INIT_ZERO	
LU_OUTPUT_INIT_ONE	
LU_OUTPUT_INIT_DISABLE	

enum LU_StatusType

LU return structure.

Enumerator:

LU_STATUS_SUCCESS	
-------------------	--

LU_STATUS_PA RAM_INVALID	
-----------------------------	--

Function Documentation

void LU_Deinit (const LU_InstanceType *eLu*)

De-initialize LU instance.

Parameters

<i>eLu</i>	LU instance
------------	-------------

LU_StatusType LU_Init (const LU_InstanceType *eLu*, const LU_ConfigType *const *plnitStruct*)

Initialize LU instance.

Parameters

<i>eLu</i>	LU instance
<i>plnitStruct</i>	LU initialization structure

Returns

LU return type

Module_driver_mam

Collaboration diagram for Module_driver_mam:Modules

HwA_mam

Classes

struct **MAM_Inf_Type**
the MAM slave information

Enumerations

```
enum MAM_Forbid_Access_Type { FORBID_READ_ACCESS = 1, FORBID_WRITE_ACCESS = 2,  
FORBID_EXECUTE_ACCESS = 4, FORBID_USER_ACCESS = 8 }
```

Functions

```
uint8_t MAM_Enable_Wdg (MAM_Master_Type Master, uint32_t u32Addr)
```

The function to enable MAM watch dog.

```
uint8_t MAM_Disable_Wdg (MAM_Master_Type Master, uint32_t u32Addr)
```

The function to disable MAM watch dog.

```
uint8_t MAM_Config (MAM_Master_Type Master, uint32_t u32Addr, uint32_t u32Val)
```

The function to config MAM.

Detailed Description

Enumeration Type Documentation

enum MAM_Forbid_Access_Type

Enumerator:

FORBID_READ_ ACCESS	
FORBID_WRITE_ ACCESS	
FORBID_EXECU TE_ACCESS	
FORBID_USER_ ACCESS	

Function Documentation

uint8_t MAM_Config (MAM_Master_Type *Master*, uint32_t *u32Addr*, uint32_t *u32Val*)

The function to config MAM.

Parameters

<i>1</i>	Master The master index
<i>2</i>	u32Addr The input address
<i>3</i>	u32Val The value to set

uint8_t MAM_Disable_Wdg (MAM_Master_Type *Master*, uint32_t *u32Addr*)

The function to disable MAM watch dog.

Parameters

<i>1</i>	Master The master index
<i>2</i>	u32Addr The input address

uint8_t MAM_Enable_Wdg (MAM_Master_Type *Master*, uint32_t *u32Addr*)

The function to enable MAM watch dog.

Parameters

<i>1</i>	Master The master index
<i>2</i>	u32Addr The input address

Module_driver_mb

Collaboration diagram for Module_driver_mb:Modules

HwA_mb

Classes

struct **MB_RequestType**

Configuration of a Mailbox request.

struct **MB_ReceiveType**

Configuration of a receiving request.

struct **_MB_HandleType**

struct **MB_InitType**

Configuration of the Mailbox.

Macros

#define **MB_CHANNEL_STATUS_SECURE** 2u

Mailbox channel bit flag of security information.

#define **MB_CHANNEL_STATUS_PRIVILEGED** 1u

Mailbox channel bit flag of processing information.

Typedefs

typedef struct **_MB_HandleType** **MB_HandleType**

Enumerations

enum **MB_StatusType** { **MB_STATUS_SUCCESS** = 0u, **MB_STATUS_FAILED**, **MB_STATUS_PARAM_ERROR**,
MB_STATUS_ALREADY_INITED, **MB_STATUS_UNINIT**, **MB_STATUS_LOCKED**,
MB_STATUS_NO_REQUEST }

Mailbox operation return values.

Functions

void **MB_Init** (**MB_HandleType** *pMbHandle, const **MB_InitType** *pInitConfig)

Initialize the Mailbox.

void **MB_DeInit** (**MB_HandleType** *pMbHandle)

De-initialize the Mailbox.

MB_StatusType **MB_AcquireChannel** (uint32_t u32Channel)

Attempt to acquire a Mailbox channel.

MB_StatusType MB_ReleaseChannel (MB_HandleType *pMbHandle, uint32_t u32Channel)

Release a Mailbox channel.

MB_StatusType MB_UnlockChannel (uint32_t u32Channel)

Software clears channel lock.

MB_StatusType MB_SendRequest (MB_HandleType *pMbHandle, MB_RequestType *pRequest)

Launching a Mailbox request.

MB_StatusType MB_ReceiveChannel (MB_HandleType *pMbHandle, MB_ReceiveType *pReceive)

Attempt to receive a request from the selected channel.

MB_StatusType MB_DoneChannel (MB_HandleType *pMbHandle, uint32_t u32Channel, uint8_t u8TargetCoreIndex)

Issue a done event to the selected channel.

uint32_t MB_PollDone (MB_HandleType *pMbHandle)

Polling the done event of all channels.

void MB_IRQProcess (MB_HandleType *pMbHandle)

Interrupt IRQ handle of Mailbox.

Mailbox channel bit mask

Bit of channel indicate channel number

```
#define MB_CHANNEL_MASK_NONE 0u
#define MB_CHANNEL_MASK_ALL 0xFFFFu
#define MB_CHANNEL_MASK(ch) ((uint32_t)(1ul << (ch)))
```

MB API Service IDs

```
#define MB_INIT_ID 0U
#define MB_DEINIT_ID 1U
#define MB_LOCK_CHANNEL_ID 2U
#define MB_RELEASE_CHANNEL_ID 3U
```

```
#define MB_SEND_REQUEST_ID 4U
#define MB_RECEIVE_CHANNEL_ID 5U
#define MB_UNLOCK_CHANNEL_ID 6U
#define MB_POLL_DONE_ID 7U
#define MB_DONE_CHANNEL_ID 8U
```

MB Dev Error Code

Error Code of calling MB apis

```
#define MB_E_PARAM_CHANNEL 0x02U
#define MB_E_PARAM_INVALID 0x03U
#define MB_E_PARAM_POINTER 0x04U
```

Detailed Description

Macro Definition Documentation

```
#define MB_CHANNEL_MASK( ch) ((uint32_t)(1ul << (ch)))
```

```
#define MB_CHANNEL_MASK_ALL 0xFFFFu
```

```
#define MB_CHANNEL_MASK_NONE 0u
```

```
#define MB_CHANNEL_STATUS_PRIVILEGED 1u
```

Mailbox channel bit flag of processing information.

```
#define MB_CHANNEL_STATUS_SECURE 2u
```

Mailbox channel bit flag of security information.

```
#define MB_DEINIT_ID 1U

#define MB_DONE_CHANNEL_ID 8U

#define MB_E_PARAM_CHANNEL 0x02U

#define MB_E_PARAM_INVALID 0x03U

#define MB_E_PARAM_POINTER 0x04U

#define MB_INIT_ID 0U

#define MB_LOCK_CHANNEL_ID 2U

#define MB_POLL_DONE_ID 7U

#define MB_RECEIVE_CHANNEL_ID 5U

#define MB_RELEASE_CHANNEL_ID 3U

#define MB_SEND_REQUEST_ID 4U

#define MB_UNLOCK_CHANNEL_ID 6U
```

Typedef Documentation

```
typedef struct _MB_HandleType MB_HandleType
```

Enumeration Type Documentation

```
enum MB_StatusType
```

Mailbox operation return values.

Enumerator:

MB_STATUS_SUCCESS	The Mailbox operation is success
MB_STATUS_FAILED	The Mailbox operation is failed
MB_STATUS_PARAMETER_ERROR	The Mailbox operation is failed because of parameter error
MB_STATUS_ALREADY_INITIALIZED	The Mailbox operation is failed because of Mailbox has already initialized
MB_STATUS_UNINITIALIZED	The Mailbox operation is failed because of Mailbox has not initialized
MB_STATUS_LOCKED	The Mailbox channel is locked
MB_STATUS_NO_REQUEST	No valid requests

Function Documentation

MB_StatusType MB_AcquireChannel (uint32_t *u32Channel*)

Attempt to acquire a Mailbox channel.

Parameters

<i>u32Channel</i>	the selected Mailbox channel
-------------------	------------------------------

Returns

MB_StatusType whether the operation is successfully

void MB_DeInit (MB_HandleType * *pMbHandle*)

De-initialize the Mailbox.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
------------------	---------------------------

MB_StatusType MB_DoneChannel (MB_HandleType * *pMbHandle*, uint32_t *u32Channel*, uint8_t *u8TargetCoreIndex*)

Issue a done event to the selected channel.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
<i>u32Channel</i>	the selected Mailbox channel
<i>u8TargetCoreIndex</i>	the target core index

Returns

MB_StatusType whether the operation is successfully

void MB_Init (MB_HandleType * *pMbHandle*, const MB_InitType * *pInitConfig*)

Initialize the Mailbox.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
<i>pInitConfig</i>	the configurations of the Mailbox

void MB_IRQProcess (MB_HandleType * *pMbHandle*)

Interrupt IRQ handle of Mailbox.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
------------------	---------------------------

uint32_t MB_PollDone (MB_HandleType * *pMbHandle*)

Polling the done event of all channels.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
------------------	---------------------------

Returns

uint32_t Done event from the communication channel

MB_StatusType MB_ReceiveChannel (MB_HandleType * *pMbHandle*, MB_ReceiveType * *pReceive*)

Attempt to receive a request from the selected channel.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
<i>pReceive</i>	Configuration of the receiving request

Returns

MB_StatusType whether the operation is successfully

MB_StatusType MB_ReleaseChannel (MB_HandleType * *pMbHandle*, uint32_t *u32Channel*)

Release a Mailbox channel.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
<i>u32Channel</i>	the selected Mailbox channel

Returns

MB_StatusType whether the operation is successfully

MB_StatusType MB_SendRequest (MB_HandleType * *pMbHandle*, MB_RequestType * *pRequest*)

Launching a Mailbox request.

Parameters

<i>pMbHandle</i>	Mailbox processing handle
<i>pRequest</i>	Configuration of the request

Returns

MB_StatusType whether the operation is successfully

MB_StatusType MB_UnlockChannel (uint32_t *u32Channel*)

Software clears channel lock.

Parameters

<i>u32Channel</i>	the selected Mailbox channel
-------------------	------------------------------

Returns

MB_StatusType whether the operation is successfully

Module_driver_msc

Collaboration diagram for Module_driver_msc:Modules

HwA_msc

Classes

struct **_MSC_HandleType**
struct **MSC_InterruptCfgType**
MSC interrupt configure structure.
struct **MSC_IOControlInitType**
struct **MSC_InitCfgType**

Macros

```
#define MSC_DEV_ERROR_REPORT STD_OFF  
#define MSC_ReportDevError(func, error) ReportDevError(MSC_MODULE_ID, func, error)  
#define MSC_INSTANCE_0 0U  
#define MSC_INSTANCE_1 1U
```

Typedefs

```
typedef uint8_t MSC_InstanceType  
typedef struct _MSC_HandleType MSC_HandleType  
typedef void(* MSC_ISRCallbackType) (const MSC_HandleType *pMscHandle, const uint32_t u32Status)
```

MSC Channel ISR callback function prototype.

Enumerations

```
enum MSC_ReturnType { MSC_RETURN_OK = 0x00U, MSC_RETURN_E_NOT_OK = 0x01U,  
MSC_RETURN_E_PARAM = 0x04U }  
enum MSC_RDRxIndexType { MSC_RDR0 = 0x0U, MSC_RDR1 = 0x1U, MSC_RDR2 = 0x2U, MSC_RDR3 =  
0x3U, MSC_RDRx_COUNT = 0x4U }  
enum MSC_RFIEModeType { MSC_RFIE_DISABLE = 0x0U, MSC_RFIE_MODE1 = 0x1U, MSC_RFIE_MODE2 =  
0x2U, MSC_RFIE_MODE3 = 0x3U }  
enum MSC_DFIEModeType { MSC_DFIE_DISABLE = 0x0U, MSC_DFIE_MODE1 = 0x1U, MSC_DFIE_MODE2 =  
0x2U }  
enum MSC_SDISelectionType { MSC_SDI_SEL_SDI0 = 0x0U, MSC_SDI_SEL_SDI1 = 0x1U,  
MSC_SDI_SEL_SDI2 = 0x2U, MSC_SDI_SEL_SDI3 = 0x3U, MSC_SDI_SEL_SDI4 = 0x4U, MSC_SDI_SEL_SDI5  
= 0x5U, MSC_SDI_SEL_SDI6 = 0x6U, MSC_SDI_SEL_SDI7 = 0x7U }  
enum MSC_ENCSelectionType { MSC_ENC_SEL_EN0 = 0x0U, MSC_ENC_SEL_EN1 = 0x1U,  
MSC_ENC_SEL_EN2 = 0x2U, MSC_ENC_SEL_EN3 = 0x3U }  
enum MSC_ENHSelectionType { MSC_ENH_SEL_EN0 = 0x0U, MSC_ENH_SEL_EN1 = 0x1U,  
MSC_ENH_SEL_EN2 = 0x2U, MSC_ENH_SEL_EN3 = 0x3U }  
enum MSC_ENLSelectionType { MSC_ENL_SEL_EN0 = 0x0U, MSC_ENL_SEL_EN1 = 0x1U,  
MSC_ENL_SEL_EN2 = 0x2U, MSC_ENL_SEL_EN3 = 0x3U }  
enum MSC_FCLControlType { MSC_FLC_ACTIVE_ON_FRAMES = 0x0U, MSC_FLC_ALWAYS_ACTIVE = 0x1U }  
enum MSC_InputPolarityType { MSC_INPUT_IDENTICAL = 0x0U, MSC_INPUT_INVERTED = 0x1U }  
enum MSC_ENxPolarityType { MSC_ENX_IDENTICAL = 0x0U, MSC_ENX_INVERTED = 0x1U }  
enum MSC_SOPolarityType { MSC_SO_IDENTICAL = 0x0U, MSC_SO_INVERTED = 0x1U }  
enum MSC_FCLpolarityType { MSC_FLC_IDENTICAL = 0x0U, MSC_FLC_INVERTED = 0x1U }  
enum MSC_TransSourceType { MSC_TRANS_SOURCE_DATA_REG = 0x0U, MSC_TRANS_SOURCE_ALTIN =  
0x2U, MSC_TRANS_SOURCE_ALTIN_INV = 0x3U }  
enum MSC_WorkModeType { MSC_TRIGGER_MODE = 0x00U, MSC_REPETITION_MODE = 0x01U }  
enum MSC_ParityType { MSC_EVEN_PARITY = 0x00U, MSC_ODD_PARITY = 0x01U }  
enum MSC_ReceiveBaudRateType { MSC_RECEIVE_DISABLED = 0x00U, MSC_BAUDRATE_FMSC_DIV4 =  
0x01U, MSC_BAUDRATE_FMSC_DIV8 = 0x02U, MSC_BAUDRATE_FMSC_DIV16 = 0x03U,  
MSC_BAUDRATE_FMSC_DIV32 = 0x04U, MSC_BAUDRATE_FMSC_DIV64 = 0x05U,  
MSC_BAUDRATE_FMSC_DIV128 = 0x06U, MSC_BAUDRATE_FMSC_DIV256 = 0x07U }  
enum MSC_ReceiveFrameType { MSC_FRAME_12_BIT = 0x00U, MSC_FRAME_16_BIT = 0x01U }
```

Functions

```
MSC_ReturnType MSC_InitStructure (MSC_InitCfgType *pInitConfig)
```


Initialize the MSC structure with the provided configuration.

MSC_ReturnType MSC_Init (MSC_HandleType *pMscHandle, const MSC_InitCfgType *pInitConfig)

Init the MSC.

MSC_ReturnType MSC_DeInit (MSC_HandleType *pMscHandle)

De-initialize the MSC.

void MSC_initInterrupt (MSC_HandleType *pMscHandle, const MSC_InterruptCfgType *pInterruptConfig)

Init the MSC interrupt.

**void MSC_SelTranmittingSource (const MSC_HandleType *pMscHandle, uint32_t u32SourceMask,
MSC_TransSourceType eSourceType)**

Select the transmitting sources.

void MSC_SetEmergencyLoad (const MSC_HandleType *pMscHandle, uint32_t u32Value)

Set Emergency load value.

**void MSC_SetIOControl (const MSC_HandleType *pMscHandle, const MSC_IOControlInitType
*pIOConfig)**

Configure the MSC IO control.

void MSC_Enable (const MSC_HandleType *pMscHandle)

Enable MSC.

void MSC_Disable (const MSC_HandleType *pMscHandle)

Disable MSC.

void MSC_SetDataFrame (const MSC_HandleType *pMscHandle, uint32_t u32Data)

MSC set data frame.

void MSC_SendDataFrame (const MSC_HandleType *pMscHandle)

MSC send data frame.

void **MSC_SendCommandFrame** (const **MSC_HandleType** *pMscHandle, uint32_t u32Command)
MSC send command frame.

uint8_t **MSC_GetReceivedFrameAddr** (const **MSC_HandleType** *pMscHandle, **MSC_RDRxIndexType** eIndex)
Get the msc received data address.

MSC_ReceiveStatusType **MSC_GetReceivedFrame** (const **MSC_HandleType** *pMscHandle, **MSC_RDRxIndexType** eIndex, uint8_t *pData)
Get the msc received data.

uint32_t **MSC_GetInterruptStatus** (const **MSC_HandleType** *pMscHandle)
Get the msc interrupt status.

uint32_t **MSC_GetToInterruptStatus** (const **MSC_HandleType** *pMscHandle)
Get the msc interrupt status.

void **MSC_EnableTrasmit** (const **MSC_HandleType** *pMscHandle, bool bEnable)
Get the msc interrupt status.

void **MSC_ClearDataFrameInterruptFlag** (const **MSC_HandleType** *pMscHandle)
Clear the msc data frame interrupt flag.

void **MSC_ClearCommandFrameInterruptFlag** (const **MSC_HandleType** *pMscHandle)
Clear the msc command frame interrupt flag.

void **MSC_ClearTimeFrameInterruptFlag** (const **MSC_HandleType** *pMscHandle)
Clear the msc time frame interrupt flag.

void **MSC_ClearReceiveInterruptFlag** (const **MSC_HandleType** *pMscHandle)
Clear the msc receive interrupt flag.

void **MSC_ClearTimeoutInterruptFlag** (const **MSC_HandleType** *pMscHandle)

Clear the msc timeout interrupt flag.

```
void MSCn_IRQHandler (const MSC_HandleType *pMscHandle)
```

MSC API Service IDs

```
#define MSC_INIT_ID 0x00U
#define MSC_DEINIT_ID 0x01U
#define MSC_INIT_STRUCTURE_ID 0x02U
#define MSC_INIT_INTERRUPT_ID 0x03U
#define MSC_SEL_TRANSMITTING_SOURCE_ID 0x04U
#define MSC_SET_EMERGENCY_LOAD_ID 0x05U
#define MSC_SET_IO_CONTROL_ID 0x06U
#define MSC_ENABLE_ID 0x07U
#define MSC_DISABLE_ID 0x08U
#define MSC_SET_DATA_FRAME_ID 0x09U
#define MSC_SEND_DATA_FRAME_ID 0x0AU
#define MSC_SEND_COMMAND_FRAME_ID 0x0BU
#define MSC_GET_RECEIVED_FRAME_ADDR_ID 0x0CU
#define MSC_GET_RECEIVED_FRAME_ID 0x0DU
#define MSC_GET_INTERRUPT_STATUS_ID 0x0EU
#define MSC_ENABLE_TRANSMIT_ID 0x0FU
#define MSC_CLEAR_DATA_FRAME_INTERRUPT_FLAG_ID 0x10U
#define MSC_CLEAR_COMMAND_FRAME_INTERRUPT_FLAG_ID 0x11U
#define MSC_CLEAR_TIME_FRAME_INTERRUPT_FLAG_ID 0x12U
#define MSC_CLEAR_RECEIVE_INTERRUPT_FLAG_ID 0x13U
#define MSC_CLEAR_TIMEOUT_INTERRUPT_FLAG_ID 0x14U
```

FTU Dev Error Code

Error Code of calling FTU apis

```
#define MSC_E_PARAM_INSTANCE 0x01U
#define MSC_E_PARAM_RXINDEX 0x02U
#define MSC_E_PARAM_POINTER 0x03U
#define MSC_E_PARAM_BITLENGTH 0x04U
```

Detailed Description

Macro Definition Documentation

#define MSC_CLEAR_COMMAND_FRAME_INTERRUPT_FLAG_ID 0x11U

#define MSC_CLEAR_DATA_FRAME_INTERRUPT_FLAG_ID 0x10U

#define MSC_CLEAR_RECEIVE_INTERRUPT_FLAG_ID 0x13U

#define MSC_CLEAR_TIME_FRAME_INTERRUPT_FLAG_ID 0x12U

#define MSC_CLEAR_TIMEOUT_INTERRUPT_FLAG_ID 0x14U

#define MSC_DEINIT_ID 0x01U

#define MSC_DEV_ERROR_REPORT STD_OFF

#define MSC_DISABLE_ID 0x08U

#define MSC_E_PARAM_BITLENGTH 0x04U

#define MSC_E_PARAM_INSTANCE 0x01U

#define MSC_E_PARAM_POINTER 0x03U

#define MSC_E_PARAM_RXINDEX 0x02U

#define MSC_ENABLE_ID 0x07U

#define MSC_ENABLE_TRANSMIT_ID 0x0FU

#define MSC_GET_INTERRUPT_STATUS_ID 0x0EU

#define MSC_GET_RECEIVED_FRAME_ADDR_ID 0x0CU

#define MSC_GET_RECEIVED_FRAME_ID 0x0DU

#define MSC_INIT_ID 0x00U

#define MSC_INIT_INTERRUPT_ID 0x03U

#define MSC_INIT_STRUCTURE_ID 0x02U

#define MSC_INSTANCE_0 0U

MSC instance 0 is selected.

```
#define MSC_INSTANCE_1 1U
```

MSC instance 1 is selected.

```
#define MSC_ReportDevError( func, error) ReportDevError(MSC_MODULE_ID, func, error)
```

```
#define MSC_SEL_TRANSMITTING_SOURCE_ID 0x04U
```

```
#define MSC_SEND_COMMAND_FRAME_ID 0x0BU
```

```
#define MSC_SEND_DATA_FRAME_ID 0x0AU
```

```
#define MSC_SET_DATA_FRAME_ID 0x09U
```

```
#define MSC_SET_EMERGENCY_LOAD_ID 0x05U
```

```
#define MSC_SET_IO_CONTROL_ID 0x06U
```

Typedef Documentation

```
typedef struct _MSC_HandleType MSC_HandleType
```

```
typedef uint8_t MSC_InstanceType
```

```
typedef void(* MSC_ISRCallbackType) (const MSC_HandleType *pMscHandle, const uint32_t u32Status)
```

MSC Channel ISR callback function prototype.

Enumeration Type Documentation

```
enum MSC_DFIEModeType
```

Enumerator:

MSC_DFIE_DISA BLE	Disable.
----------------------	----------

MSC_DFIE_MOD E1	Interrupt is generated when the last data bit has been shifted.
MSC_DFIE_MOD E2	Interrupt is generated when the first data bit has been shifted.

enum MSC_ENCSelectionType

Enumerator:

MSC_ENC_SEL_ EN0	EN0 is selected for ENC.
MSC_ENC_SEL_ EN1	EN1 is selected for ENC.
MSC_ENC_SEL_ EN2	EN2 is selected for ENC.
MSC_ENC_SEL_ EN3	EN3 is selected for ENC.

enum MSC_ENHSelectionType

Enumerator:

MSC_ENH_SEL_ EN0	EN0 is selected for ENH.
MSC_ENH_SEL_ EN1	EN1 is selected for ENH.
MSC_ENH_SEL_ EN2	EN2 is selected for ENH.
MSC_ENH_SEL_ EN3	EN3 is selected for ENH.

enum MSC_ENLSelectionType

Enumerator:

MSC_ENL_SEL_ EN0	EN0 is selected for ENL.
MSC_ENL_SEL_ EN1	EN1 is selected for ENL.
MSC_ENL_SEL_ EN2	EN2 is selected for ENL.
MSC_ENL_SEL_ EN3	EN3 is selected for ENL.

enum MSC_ENxPolarityType**Enumerator:**

MSC_ENX_IDEN TICAL	Enx and ENL, ENH, ENC signal polarities are identical (high active).
MSC_ENX_INVE RTED	ENx and ENL, ENH, ENC signal polarities are inverted (low active).

enum MSC_FCLControlType**Enumerator:**

MSC_FLC_ACTIV E_ON_FRAMES	FCL is actived only during the active phases of frames.
MSC_FLC_ALWA YS_ACTIVE	FCL is always active whether or not a downstream frame is transmitted currently..

enum MSC_FCLpolarityType**Enumerator:**

MSC_FLC_IDEN TICAL	FCLP and FCL polarity is identical.
-----------------------	-------------------------------------

MSC_FLC_INVERTED	FCLP and FCL polarity is inverted.
------------------	------------------------------------

enum MSC_InputPolarityType

Enumerator:

MSC_INPUT_IDENTICAL	SDI and SI signal polarities are identical.
MSC_INPUT_INVERTED	SDI and SI signal polarities are inverted.

enum MSC_ParityType

Enumerator:

MSC_EVEN_PARITY	Reception even parity.
MSC_ODD_PARITY	Reception odd parity.

enum MSC_RDRxIndexType

Enumerator:

MSC_RDR0	
MSC_RDR1	
MSC_RDR2	
MSC_RDR3	
MSC_RDRx_COUNT	

enum MSC_ReceiveBaudRateType

Enumerator:

MSC_RECEIVE_DISABLED	Receiving channel is disabled.
----------------------	--------------------------------

MSC_BAUDRATE _FMSC_DIV4	Baud rate=fmssc/4.
MSC_BAUDRATE _FMSC_DIV8	Baud rate=fmssc/8.
MSC_BAUDRATE _FMSC_DIV16	Baud rate=fmssc/16.
MSC_BAUDRATE _FMSC_DIV32	Baud rate=fmssc/32.
MSC_BAUDRATE _FMSC_DIV64	Baud rate=fmssc/64.
MSC_BAUDRATE _FMSC_DIV128	Baud rate=fmssc/128.
MSC_BAUDRATE _FMSC_DIV256	Baud rate=fmssc/256.

enum MSC_ReceiveFrameType

Enumerator:

MSC_FRAME_12 _BIT	12-bit frame selected.
MSC_FRAME_16 _BIT	16-bit frame selected.

enum MSC_ReturnType

Enumerator:

MSC_RETURN_ OK	The SENT operation is succeeded
MSC_RETURN_E _NOT_OK	The SENT operation is failed
MSC_RETURN_E _PARAM	The SENT parameter is incorrect or out of range.

enum MSC_RFIEModeType

Enumerator:

MSC_RFIE_DISABLE	Interrupt generation disabled.
MSC_RFIE_MODE1	The interrupt is generated when data is received and written into the RDRx.
MSC_RFIE_MODE2	The interrupt is generated when received data is not equal to 0.
MSC_RFIE_MODE3	The interrupt is generated when data is received and written into register RDR3.

enum MSC_SDISelectionType

Enumerator:

MSC_SDI_SEL_SDI0	SDI0 input is selected for SDI.
MSC_SDI_SEL_SDI1	SDI1 input is selected for SDI.
MSC_SDI_SEL_SDI2	SDI2 input is selected for SDI.
MSC_SDI_SEL_SDI3	SDI3 input is selected for SDI.
MSC_SDI_SEL_SDI4	SDI4 input is selected for SDI.
MSC_SDI_SEL_SDI5	SDI5 input is selected for SDI.
MSC_SDI_SEL_SDI6	SDI6 input is selected for SDI.

MSC_SDI_SEL_S DI7	SDI7 input is selected for SDI.
----------------------	---------------------------------

enum MSC_SOPolarityType

Enumerator:

MSC_SO_IDENTI CAL	SOP and SO signal polarity is identical.
MSC_SO_INVER TED	SOP and SO signal polarity is inverted.

enum MSC_TransSourceType

Enumerator:

MSC_TRANS_SO URCE_DATA_RE G	SSL[x] is taken from TCDAR.DL[x].
MSC_TRANS_SO URCE_ALTIN	SSL[x] is taken from the ALTINL input line x.
MSC_TRANS_SO URCE_ALTIN_IN V	SSL[x] is taken from the ALTINL input line x in inverted state.

enum MSC_WorkModeType

Enumerator:

MSC_TRIGGER_ MODE	Trigger work mode.
MSC_REPETITIO N_MODE	Repetition work mode.

Function Documentation

void MSC_ClearCommandFrameInterruptFlag (const MSC_HandleType * *pMscHandle*)

Clear the msc command frame interrupt flag.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

void MSC_ClearDataFrameInterruptFlag (const MSC_HandleType * *pMscHandle*)

Clear the msc data frame interrupt flag.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

void MSC_ClearReceiveInterruptFlag (const MSC_HandleType * *pMscHandle*)

Clear the msc receive interrupt flag.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

void MSC_ClearTimeFrameInterruptFlag (const MSC_HandleType * *pMscHandle*)

Clear the msc time frame interrupt flag.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

void MSC_ClearTimeoutInterruptFlag (const MSC_HandleType * *pMscHandle*)

Clear the msc timeout interrupt flag.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

MSC_ReturnType MSC_Deinit (MSC_HandleType * *pMscHandle*)

De-initialize the MSC.

This function de-initializes the MSC module, resetting it to its default state.

Parameters

<i>pMscHandle</i>	Pointer to the MSC handle structure.
-------------------	--------------------------------------

Returns

MSC_ReturnType Returns MSC_RETURN_OK if de-initialization is successful, otherwise returns an error code indicating the failure.

void MSC_Disable (const MSC_HandleType * *pMscHandle*)

Disable MSC.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

void MSC_Enable (const MSC_HandleType * *pMscHandle*)

Enable MSC.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

void MSC_EnableTrasmit (const MSC_HandleType * *pMscHandle*, bool *bEnable*)

Get the msc interrupt status.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>bEnable</i>	Enable transmit channel.

uint32_t MSC_GetInterruptStatus (const MSC_HandleType * *pMscHandle*)

Get the msc interrupt status.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

Returns

uint32_t Interrupt status.

**MSC_ReceiveStatusType MSC_GetReceivedFrame (const MSC_HandleType * *pMscHandle*,
MSC_RDRxIndexType *eIndex*, uint8_t * *pData*)**

Get the msc received data.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>eIndex</i>	Receive data register index.
<i>pData</i>	Received data value.

Returns

MSC_ReceiveStatusType Status of getting received data.

**uint8_t MSC_GetReceivedFrameAddr (const MSC_HandleType * *pMscHandle*, MSC_RDRxIndexType
eIndex)**

Get the msc received data address.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>eIndex</i>	Receive data register index.

Returns

uint8_t Receiverd data address.

uint32_t MSC_GetToInterruptStatus (const MSC_HandleType * *pMscHandle*)

Get the msc interrupt status.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

Returns

uint32_t Interrupt status.

MSC_ReturnType MSC_Init (MSC_HandleType * *pMscHandle*, const MSC_InitCfgType * *pInitConfig*)

Init the MSC.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>pInitConfig</i>	MSCInstance initial configuration.

void MSC_initInterrupt (MSC_HandleType * *pMscHandle*, const MSC_InterruptCfgType * *pInterruptConfig*)

Init the MSC interrupt.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>pInterruptConfig</i>	MSCInstance interrupt configuration.

MSC_ReturnType MSC_InitStructure (MSC_InitCfgType * *pInitConfig*)

Initialize the MSC structure with the provided configuration.

Parameters

<i>pInitConfig</i>	Pointer to the MSC initialization configuration structure.
--------------------	--

Returns

MSC_ReturnType Returns MSC_RETURN_OK if the initialization is successful, otherwise returns an error code indicating the failure.


```
void MSC_SelTranmittingSource (const MSC_HandleType *  pMscHandle, uint32_t  u32SourceMask,
MSC_TransSourceType  eSourceType)
```

Select the transmitting sources.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>u32SourceMask</i>	Transmitting sources.
<i>eSourceType</i>	Transmitting sources type.

```
void MSC_SendCommandFrame (const MSC_HandleType *  pMscHandle, uint32_t  u32Command)
```

MSC send command frame.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>u32Command</i>	command to be sent.

```
void MSC_SendDataFrame (const MSC_HandleType *  pMscHandle)
```

MSC send data frame.

Parameters

<i>pMscHandle</i>	InstanceHandle
-------------------	----------------

```
void MSC_SetDataFrame (const MSC_HandleType *  pMscHandle, uint32_t  u32Data)
```

MSC set data frame.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>u32Data</i>	Data to be sent.

```
void MSC_SetEmergencyLoad (const MSC_HandleType *  pMscHandle, uint32_t  u32Value)
```

Set Emergency load value.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>u32Value</i>	Emergency load value.

```
void MSC_SetIOControl (const MSC_HandleType *  pMscHandle, const MSC_IOControlInitType *  pIOConfig)
```

Configure the MSC IO control.

Parameters

<i>pMscHandle</i>	InstanceHandle
<i>pIOConfig</i>	MSC IO control configuration.

```
void MSCn_IRQHandler (const MSC_HandleType *  pMscHandle)
```

Module_driver_ospi

Collaboration diagram for Module_driver_ospi:Modules

HwA_ospi

Classes

- struct **OSPI_ConfigType**
The basic configuration option for the OSPI peripheral.
- struct **OSPI_DeviceConfigType**
Define the structure for OSPI device configuration.
- struct **_flexspi_transfer**
FlexSPI Transfer structure definition.

Macros

```
#define OSPI_FIFO_DEPTH_MAX 16
```

```
#define OSPI_ONE_TRANSFER_SIZE_MAX 64  
#define OSPI_LUT_SEQ(cmd0, pad0, op0, cmd1, pad1, op1)
```

Typedefs

```
typedef struct _flexspi_transfer OSPI_TransferType  
FlexSPI Transfer structure definition.
```

Enumerations

```
enum OSPI_StatusType { OSPI_STATUS_SUCCESS = 0, OSPI_STATUS_PARAM_ERR,  
OSPI_STATUS_TIMEOUT }  
Status returned by OSPI APIs.  
  
enum OSPI_OperationType { OSPI_OPERATION_COMMAND, OSPI_OPERATION_CONFIG,  
OSPI_OPERATION_READ, OSPI_OPERATION_WRITE }
```

Functions

```
void OSPI_Init (OSPI_Type *base, const OSPI_ConfigType *const pConfig)  
Initialize OSPI configuration.
```

```
void OSPI_GetDefaultConfig (OSPI_ConfigType *config)  
Get OSPI configuration.
```

```
void OSPI_DeInit (OSPI_Type *base)  
De-initialize the OSPI.
```

```
void OSPI_FlashConfig (OSPI_Type *base, OSPI_DeviceConfigType *pConfig)  
OSPI Config Flash Parameter.
```

```
void OSPI_UpdateLUT (OSPI_Type *base, uint32_t index, const uint32_t *cmd, uint32_t count)  
OSPI Config LUT.
```

```
OSPI_StatusType OSPI_Wait_Cmd_Done (OSPI_Type *base)  
OSPI Wait bus command transaction done.
```

```
OSPI_StatusType OSPI_Wait_Bus_Idle (OSPI_Type *base)
```

OSPI Wait bus command transaction done.

OSPI_StatusType OSPI_Write_Fifo (OSPI_Type *base, uint8_t u8SeqId, uint32_t *pBuf, uint16_t u16Size)

OSPI write fifo. The fifo size is 16 words.

OSPI_StatusType OSPI_Read_Fifo (OSPI_Type *base, uint8_t u8SeqId, uint32_t *pBuf, uint16_t u16Size)

OSPI read fifo. The fifo size is 16 words.

OSPI_StatusType OSPI_TransferBlocking (OSPI_Type *base, **OSPI_TransferType** *xfer)

OSPI blocking transfer data.

OSPI_StatusType OSPI_TransferNonBlocking (OSPI_Type *base, **OSPI_TransferType** *xfer)

OSPI non-blocking transfer data.

void **OSPI_DriverIRQHandler** (OSPI_Type *base)

OSPI interrupt driver handler.

OsPI API Service IDs

These macros define the command IDs used for communication with the on-chip OSPI interface. Each ID corresponds to a specific operation that can be performed via the OSPI interface.

```
#define OSPI_INIT_ID ((uint8) 0x00u)
#define OSPI_SET_FLASHCFG_ID ((uint8) 0x01u)
#define OSPI_UPDATE_LUT_ID ((uint8) 0x02u)
#define OSPI_WRITE_FIFO_ID ((uint8) 0x03u)
#define OSPI_READ_FIFO_ID ((uint8) 0x04u)
#define OSPI_TRANSFER_BLOCKING_ID ((uint8) 0x05u)
#define OSPI_TRANSFER_NONBLOCKING_ID ((uint8) 0x06u)
```

OsPI Dev Error Code

Error Code of calling OSPI apis

```
#define OSPI_E_PARAM_POINTER ((uint8)0x01u)
#define OSPI_E_PARAM_INVALID ((uint8)0x02u)
```

IO configurations IO configurations for the OSPI interface

IO configurations for the OSPI interface

```
#define SINGLE_IO 0x00  
#define DUAL_IO 0x01  
#define QUAD_IO 0x02  
#define OCTAL_IO 0x03
```

OSPI command_set OSPI command set definitions

OSPI command set definitions

```
#define OSPI_CMD_STOP 0x0  
#define OSPI_CFG_DRV 0x1  
#define OSPI_CFG_ADDR 0x2  
#define OSPI_CMD_DUMMY 0x3  
#define OSPI_CFG_MODE8 0x4  
#define OSPI_CFG_MODE2 0x5  
#define OSPI_CFG_MODE4 0x6  
#define OSPI_READ_DRV 0x7  
#define OSPI_WRITE_DRV 0x8  
#define OSPI_CMD_END 0x9  
#define OSPI_CFG_ADDR_DDR 0xA  
#define OSPI_CFG_MODE8_DDR 0xB  
#define OSPI_CFG_MODE2_DDR 0xC  
#define OSPI_CFG_MODE4_DDR 0xD  
#define OSPI_READ_DRV_DDR 0xE  
#define OSPI_WRITE_DRV_DDR 0xF  
#define OSPI_CFG_DRV_DDR 0x11  
#define OSPI_CFG_CADDR 0x12  
#define OSPI_CFG_CADDR_DDR 0x13
```

Detailed Description

Macro Definition Documentation

#define DUAL_IO 0x01

Dual I/O

#define OCTAL_IO 0x03

Octal I/O

#define OSPI_CFG_ADDR 0x2

Configure address

#define OSPI_CFG_ADDR_DDR 0xA

Configure address DDR

#define OSPI_CFG_CADDR 0x12

Configure CADDR

#define OSPI_CFG_CADDR_DDR 0x13

Configure CADDR DDR

#define OSPI_CFG_DRV 0x1

Configure drive strength

#define OSPI_CFG_DRV_DDR 0x11

Configure drive strength DDR

#define OSPI_CFG_MODE2 0x5

Configure mode 2

#define OSPI_CFG_MODE2_DDR 0xC

Configure mode 2 DDR

#define OSPI_CFG_MODE4 0x6

Configure mode 4

#define OSPI_CFG_MODE4_DDR 0xD

Configure mode 4 DDR

#define OSPI_CFG_MODE8 0x4

Configure mode 8

#define OSPI_CFG_MODE8_DDR 0xB

Configure mode 8 DDR

#define OSPI_CMD_DUMMY 0x3

Configure dummy cycles

#define OSPI_CMD_END 0x9

End command

#define OSPI_CMD_STOP 0x0

Stop command

#define OSPI_E_PARAM_INVALID ((uint8)0x02u)

#define OSPI_E_PARAM_POINTER ((uint8)0x01u)

#define OSPI_FIFO_DEPTH_MAX 16

OSPI FIFO depth

#define OSPI_INIT_ID ((uint8) 0x00u)

#define OSPI_LUT_SEQ(cmd0, pad0, op0, cmd1, pad1, op1)

Value:(OSPI_LUT_INSTR0_CMD_NAME(cmd0) | OSPI_LUT_INSTR0_PAD_NUM(pad0) | OSPI_LUT_INSTR0_DRV_CMD(op0) | \n OSPI_LUT_INSTR1_CMD_NAME(cmd1) | OSPI_LUT_INSTR1_PAD_NUM(pad1) | OSPI_LUT_INSTR1_DRV_CMD(op1))

#define OSPI_ONE_TRANSFER_SIZE_MAX 64

OSPI one transfer size

#define OSPI_READ_DRV 0x7

Configure read drive strength

#define OSPI_READ_DRV_DDR 0xE

Configure read drive strength DDR

```

#define OSPI_READ_FIFO_ID  ((uint8) 0x04u)

#define OSPI_SET_FLASHCFG_ID  ((uint8) 0x01u)

#define OSPI_TRANSFER_BLOCKING_ID  ((uint8) 0x05u)

#define OSPI_TRANSFER_NONBLOCKING_ID  ((uint8) 0x06u)

#define OSPI_UPDATE_LUT_ID  ((uint8) 0x02u)

#define OSPI_WRITE_DRV  0x8
    Configure write drive strength

#define OSPI_WRITE_DRV_DDR  0xF
    Configure write drive strength DDR

#define OSPI_WRITE_FIFO_ID  ((uint8) 0x03u)

#define QUAD_IO  0x02
    Quad I/O

#define SINGLE_IO  0x00
    Single I/O

```

Typedef Documentation

typedef struct _flexspi_transfer OSPI_TransferType

FlexSPI Transfer structure definition.

This structure defines the parameters required for a FlexSPI transfer operation.

Enumeration Type Documentation

enum OSPI_OperationType

Enumerator:

OSPI_OPERATION_COMMAND	OSPI operation: Only command, both TX and Rx buffer are ignored.
OSPI_OPERATION_CONFIG	OSPI operation: Configure device mode, the TX fifo size is fixed in LUT.
OSPI_OPERATION_READ	OSPI operation: Read, only Rx Buffer is effective.
OSPI_OPERATION_WRITE	OSPI operation: Read, only Tx Buffer is effective.

enum OSPI_StatusType

Status returned by OSPI APIs.

Enumerator:

OSPI_STATUS_SUCCESS	API execute successfully
OSPI_STATUS_PARAM_ERR	parameter error
OSPI_STATUS_TIMEOUT	

Function Documentation

void OSPI_DeInit (OSPI_Type * *base*)

De-initialize the OSPI.

Parameters

<i>base</i>	OSPI peripheral base address
-------------	------------------------------

void OSPI_DriverIRQHandler (OSPI_Type * *base*)

OSPI interrupt driver handler.

Parameters

<i>base</i>	OSPI peripheral base address
-------------	------------------------------

void OSPI_FlashConfig (OSPI_Type * *base*, OSPI_DeviceConfigType * *pConfig*)

OSPI Config Flash Parameter.

Parameters

<i>base</i>	OSPI peripheral base address
<i>pConfig</i>	the basic configurations of the OSPI

void OSPI_GetDefaultConfig (OSPI_ConfigType * *config*)

Get OSPI configuration.

Parameters

<i>config</i>	the basic configurations of the OSPI
---------------	--------------------------------------

void OSPI_Init (OSPI_Type * *base*, const OSPI_ConfigType *const *pConfig*)

Initialize OSPI configuration.

Parameters

<i>base</i>	OSPI peripheral base address
<i>pConfig</i>	the basic configurations of the OSPI

OSPI_StatusType OSPI_Read_Fifo (OSPI_Type * *base*, uint8_t *u8SeqId*, uint32_t * *pBuf*, uint16_t *u16Size*)

OSPI read fifo's size is 16 words.

Parameters

<i>base</i>	OSPI peripheral base address
<i>u8SeqId</i>	the cmd id location in lut.
<i>pBuf</i>	the read buffer start address.
<i>u8Size</i>	fifo size to be read.

Returns

OSPI_StatusType whether the operation is successfully

OSPI_StatusType OSPI_TransferBlocking (OSPI_Type * *base*, OSPI_TransferType * *xfer*)

OSPI blocking transfer data.

Parameters

<i>base</i>	OSPI peripheral base address
<i>xfer</i>	pointer to the transfer structure

Returns

OSPI_StatusType whether the operation is successfully

OSPI_StatusType OSPI_TransferNonBlocking (OSPI_Type * *base*, OSPI_TransferType * *xfer*)

OSPI non-blocking transfer data.

Parameters

<i>base</i>	OSPI peripheral base address
<i>xfer</i>	pointer to the transfer structure

Returns

OSPI_StatusType whether the operation is successfully

void OSPI_UpdateLUT (OSPI_Type * *base*, uint32_t *index*, const uint32_t * *cmd*, uint32_t *count*)

OSPI Config LUT.

Parameters

<i>base</i>	OSPI peripheral base address
<i>index</i>	index to be written
<i>cmd</i>	Command sequence array
<i>count</i>	Number of sequences

OSPI_StatusType OSPI_Wait_Bus_Idle (OSPI_Type * *base*)

OSPI Wait bus command transaction done.

Parameters

<i>base</i>	OSPI peripheral base address
-------------	------------------------------

Returns

OSPI_StatusType whether the operation is successfully

OSPI_StatusType OSPI_Wait_Cmd_Done (OSPI_Type * *base*)

OSPI Wait bus command transaction done.

Parameters

<i>base</i>	OSPI peripheral base address
-------------	------------------------------

Returns

OSPI_StatusType whether the operation is successfully

OSPI_StatusType OSPI_Write_Fifo (OSPI_Type * *base*, uint8_t *u8SeqId*, uint32_t * *pBuf*, uint16_t *u16Size*)

OSPI write fifo size is 16words.

Parameters

<i>base</i>	OSPI peripheral base address
<i>u8SeqId</i>	the cmd id location in lut.
<i>pBuf</i>	the write buffer start address.
<i>u8Size</i>	fifo size to be written.

Returns

OSPI_StatusType whether the operation is successfully

Module_driver_overlay

Collaboration diagram for Module_driver_overlay:Modules

HwA_overlay

Classes

struct **OVERLAY_OverlayRegionInitType**

Overlay Region parameters.

struct **OVERLAY_FARInitType**

FAR parameter.

struct **OVERLAY_ErrorInfoType**

Error Info.

struct **OVERLAY_InterruptType**

Interrupt Parameter.

Typedefs

typedef void(* **OVERLAY_ErrorCallback_Type**) (**OVERLAY_ErrorInfoType** u16ErrorFlag)

Overlay Setting Error callback type.

Enumerations

enum **OVERLAY_ErrorType** { **OVERLAY_ERROR_OK**, **OVERLAY_ERROR_ADDR**, **OVERLAY_ERROR_SIZE**,
OVERLAY_ERROR_FLAG }

Overlay Error Information.

Functions

OVERLAY_ErrorType **OVERLAY_RegionInit** (**OVERLAY_OverlayRegionInitType** *pOverlayInitCfg)

Overlay region initial function.

OVERLAY_ErrorType **OVERLAY_RegionDeInit** (void)

De-Init Overlay Region.

OVERLAY_ErrorType OVERLAY_FARInit (OVERLAY_FARInitType *pFarInitCfg)

Far initial function.

OVERLAY_ErrorType OVERLAY_FARDeInit (void)

De-Init FAR Region.

void OVERLAY_SetInterrupt (OVERLAY_InterruptType *pInterruptCfg)

Enable or Disable Interrupt.

void OVERLAY_GetErrorInfo (OVERLAY_ErrorInfoType *pErrorInfo)

Get Error Info.

void OVERLAY_ClrErrorInfo (OVERLAY_ErrorInfoType *pErrorInfo)

Clear Error Info.

void OVERLAY_ErrorInterruptRoutine (void)

Call Me in Overlay Error interrupt handler.

Detailed Description

Typedef Documentation

typedef void(* OVERLAY_ErrorCallback_Type) (OVERLAY_ErrorInfoType u16ErrorFlag)

Overlay Setting Error callback type.

Enumeration Type Documentation

enum OVERLAY_ErrorType

Overlay Error Information.

Enumerator:

OVERLAY_ERRO R_OK	no error
OVERLAY_ERRO R_ADDR	error address
OVERLAY_ERRO R_SIZE	error size
OVERLAY_ERRO R_FLAG	error flag

Function Documentation

void OVERLAY_ClrErrorInfo (OVERLAY_ErrorInfoType * *pErrorInfo*)

Clear Error Info.

Parameters

<i>pErrorInfo</i>	error info point
-------------------	------------------

void OVERLAY_ErrorInterruptRoutine (void)

Call Me in Overlay Error interrupt handler.

OVERLAY_ErrorType OVERLAY_FARDeInit (void)

De-Init FAR Region.

Returns

ERROR_OK is ok, others are not ok

OVERLAY_ErrorType OVERLAY_FARInit (OVERLAY_FARInitType * *pFarInitCfg*)

Far initial function.

Parameters

<i>pFarInitCfg</i>	initial parameters
--------------------	--------------------

Returns

ERROR_OK is ok, others are not ok

void OVERLAY_GetErrorInfo (OVERLAY_ErrorInfoType * *pErrorInfo*)

Get Error Info.

Parameters

<i>pErrorInfo</i>	error info point
-------------------	------------------

OVERLAY_ErrorType OVERLAY_RegionDeInit (void)

De-Init Overlay Region.

Returns

ERROR_OK is ok, others are not ok

OVERLAY_ErrorType OVERLAY_RegionInit (OVERLAY_OverlayRegionInitType * *pOverlayInitCfg*)

Overlay region initial function.

Parameters

<i>pOverlayInitCfg</i>	initial parameters
------------------------	--------------------

Returns

ERROR_OK is ok, others are not ok

void OVERLAY_SetInterrupt (OVERLAY_InterruptType * *pInterruptCfg*)

Enable or Disable Interrupt.

Parameters

<i>pInterruptCfg</i>	interrupt config parameter
----------------------	----------------------------

Module_driver_pcc

Classes

struct **PCC_CtrlType**

define the PCC module initialization structure.

Macros

#define PCC_DWPLK_MASK 0x80000000u

Marco for PCCn Bit Field definition.

#define PCC_DWPLK_SHIFT 31u

#define PCC_DWPLK_WIDTH 1u

#define PCC_DWP_MASK 0x70000000u

#define PCC_DWP_SHIFT 28u

#define PCC_DWP_WIDTH 3u

#define PCC_DWP(x) (((uint32_t)(((uint32_t)(x))<<PCC_DWP_SHIFT))&PCC_DWP_MASK)

#define PCC_CGC_MASK 0x800000U

```

#define PCC_CGC_SHIFT 23U
#define PCC_CGC_WIDTH 1U
#define PCC_CGC(x) (((uint32_t)(((uint32_t)(x))<<PCC_CGC_SHIFT))&PCC_CGC_MASK)
#define PCC_SEL_MASK 0x700000U
#define PCC_SEL_SHIFT 20U
#define PCC_SEL_WIDTH 3U
#define PCC_SEL(x) (((uint32_t)(((uint32_t)(x))<<PCC_SEL_SHIFT))&PCC_SEL_MASK)
#define PCC_GetSEL(x) (((uint32_t)(x))&PCC_SEL_MASK)>>PCC_SEL_SHIFT)
#define PCC_DIV_MASK 0x7U
#define PCC_DIV_SHIFT 0U
#define PCC_DIV_WIDTH 3U
#define PCC_DIV(x) (((uint32_t)(((uint32_t)(x))<<PCC_DIV_SHIFT))&PCC_DIV_MASK)
#define PCC_GetDIV(x) (((uint32_t)(x))&PCC_DIV_MASK)>>PCC_DIV_SHIFT)
#define PCC_SWR_MASK 0x10000u
#define PCC_MUX_MAX_NUMBER 8U

```

Enumerations

```

enum PCC_StatusType { PCC_STATUS_SUCCESS = 0U, PCC_STATUS_CLOCK_INVALID = 1U,
PCC_STATUS_CONFIGURED_NOT_SUPPORT = 1U }

```

Pcc clock status.

```

enum PCC_ClkGateSrcType { PCC_CLKGATE_SRC_OFF = 0U, PCC_CLKGATE_SRC_FOSCDIV = 1U,
PCC_CLKGATE_SRC_SIRCDIV = 2U, PCC_CLKGATE_SRC_FIRCDIV = 3U, PCC_CLKGATE_SRC_RESERVE0 =
4U, PCC_CLKGATE_SRC_PLL1DIV = 5U, PCC_CLKGATE_SRC_PLL0DIV = 6U,
PCC_CLKGATE_SRC_RESERVE1 = 7U }

```

defined the clock source for function clock,match with PCC_XXX[SEL] bit filed.

```

enum PCC_ClkDivType { PCC_CLK_DIV_BY1 = 0U, PCC_CLK_DIV_BY2 = 1U, PCC_CLK_DIV_BY3 = 2U,
PCC_CLK_DIV_BY4 = 3U, PCC_CLK_DIV_BY5 = 4U, PCC_CLK_DIV_BY6 = 5U, PCC_CLK_DIV_BY7 = 6U,
PCC_CLK_DIV_BY8 = 7U, PCC_CLK_UNINVOLVED = 8U }

```

define the clock divider,match with PCC_XXX[DIV] bit filed.

```

enum PCC_CtrlOwnerType { PCC_CTRL_BY_ALL = 0U, PCC_CTRL_BY_CPU0 = 1U, PCC_CTRL_BY_CPU1 =
2U, PCC_CTRL_BY_CPU2 = 3U, PCC_CTRL_BY_NONE = 7U }

```

Indicate which CPU can control PCC.

Functions

```

uint32_t PCC_GetPccFunctionClock (const PCC_ClkSrcType eClockName)

```

get PCC function clock status and value.

uint32_t **PCC_GetPccInterfaceClock** (const PCC_ClkSrcType eClockName)

get PCC interface clock status and value.

PCC_StatusType PCC_SetPcc (const **PCC_CtrlType** *const pConfig)

set PCC one peripheral clock configuration.

void **PCC_GenPeripheralReset** (const PCC_ClkSrcType eClockName)

Generate peripheral reset.

PCC API Service IDs

#define **PCC_SET_PCC_ID** 0x00U

#define **PCC_GET_PCC_INTERFACE_CLOCK_ID** 0x01U

#define **PCC_GET_PCC_FUNCTION_CLOCK_ID** 0x02U

PCC Dev Error Code

Error Code of calling PCC apis

#define **PCC_E_PARAM_POINTER** 0x01U

#define **PCC_E_PARAM_OUT_RANGE** 0x02U

Detailed Description

Macro Definition Documentation

```
#define PCC_CGC( x)  (((uint32_t)(((uint32_t)(x))<<PCC_CGC_SHIFT))&PCC_CGC_MASK)
```

```
#define PCC_CGC_MASK  0x8000000U
```

```
#define PCC_CGC_SHIFT  23U
```

```
#define PCC_CGC_WIDTH  1U
```

```
#define PCC_DIV( x)  (((uint32_t)(((uint32_t)(x))<<PCC_DIV_SHIFT))&PCC_DIV_MASK)
```

```
#define PCC_DIV_MASK  0x7U
```

```
#define PCC_DIV_SHIFT  0U
```

```
#define PCC_DIV_WIDTH  3U
```

```
#define PCC_DWP( x)  (((uint32_t)(((uint32_t)(x))<<PCC_DWP_SHIFT))&PCC_DWP_MASK)
```

```
#define PCC_DWP_MASK  0x70000000u
```

```
#define PCC_DWP_SHIFT  28u
```

```
#define PCC_DWP_WIDTH  3u
```

```
#define PCC_DWPLK_MASK  0x80000000u
```

Marco for PCCn Bit Field definition.

```

#define PCC_DWPLK_SHIFT  31u

#define PCC_DWPLK_WIDTH  1u

#define PCC_E_PARAM_OUT_RANGE  0x02U

#define PCC_E_PARAM_POINTER  0x01U

#define PCC_GET_PCC_FUNCTION_CLOCK_ID  0x02U

#define PCC_GET_PCC_INTERFACE_CLOCK_ID  0x01U

#define PCC_GetDIV( x)  (((uint32_t)(x))&PCC_DIV_MASK)>>PCC_DIV_SHIFT)

#define PCC_GetSEL( x)  (((uint32_t)(x))&PCC_SEL_MASK)>>PCC_SEL_SHIFT)

#define PCC_MUX_MAX_NUMBER  8U

#define PCC_SEL( x)  (((uint32_t)(((uint32_t)(x))<<PCC_SEL_SHIFT))&PCC_SEL_MASK)

#define PCC_SEL_MASK  0x700000U

#define PCC_SEL_SHIFT  20U

#define PCC_SEL_WIDTH  3U

#define PCC_SET_PCC_ID  0x00U

#define PCC_SWR_MASK  0x10000u

```

Enumeration Type Documentation

enum PCC_ClkDivType

define the clock divider,match with PCC_XXX[DIV] bit filed.

Enumerator:

PCC_CLK_DIV_B Y1	Divide by 1 (pass-through, no clock divide)
---------------------	---

PCC_CLK_DIV_B Y2	Divide by 2
PCC_CLK_DIV_B Y3	Divide by 3
PCC_CLK_DIV_B Y4	Divide by 4
PCC_CLK_DIV_B Y5	Divide by 5
PCC_CLK_DIV_B Y6	Divide by 6
PCC_CLK_DIV_B Y7	Divide by 7
PCC_CLK_DIV_B Y8	Divide by 8
PCC_CLK_UNIN VOLVED	Current peripheral dose not contain DIV configuration

enum PCC_ClkGateSrcType

defined the clock source for function clock,match with PCC_XXX[SEL] bit filed.

Enumerator:

PCC_CLKGATE_ SRC_OFF	
PCC_CLKGATE_ SRC_FOSCDIV	
PCC_CLKGATE_ SRC_SIRCDIV	
PCC_CLKGATE_ SRC_FIRCDIV	
PCC_CLKGATE_ SRC_RESERVE0	
PCC_CLKGATE_ SRC_PLL1DIV	

PCC_CLKGATE_ SRC_PLL0DIV	
PCC_CLKGATE_ SRC_RESERVE1	

enum PCC_CtrlOwnerType

Indicate which CPU can control PCC.

Enumerator:

PCC_CTRL_BY_ ALL	All CPUs are allowed to write this peripheral
PCC_CTRL_BY_ CPU0	Only CPU0 is allowed to control this peripheral
PCC_CTRL_BY_ CPU1	Only CPU1 is allowed to control this peripheral
PCC_CTRL_BY_ CPU2	Only CPU2 is allowed to control this peripheral
PCC_CTRL_BY_ NONE	None CPU is allowed to control this peripheral

enum PCC_StatusType

Pcc clock status.

Enumerator:

PCC_STATUS_S UCCESS	
PCC_STATUS_C LOCK_INVALID	
PCC_STATUS_C ONFIGURED_NO T_SUPPORT	

Function Documentation

void PCC_GenPeripheralReset (const PCC_ClkSrcType *eClockName*)

Generate peripheral reset.

uint32_t PCC_GetPccFunctionClock (const PCC_ClkSrcType *eClockName*)

get PCC function clock status and value.

Parameters

<i>pcc_ClkSrcType</i>	eClockName: used for choose PCC clock source to query.
-----------------------	--

Returns

uint32_t Pcc function clock frequency, if PCC is not enable or do not have clock mux configuration, the function will return 0

uint32_t PCC_GetPccInterfaceClock (const PCC_ClkSrcType *eClockName*)

get PCC interface clock status and value.

Parameters

<i>pcc_ClkSrcType</i>	eClockName: used for choose PCC clock source to query.
-----------------------	--

PCC_StatusType PCC_SetPcc (const PCC_CtrlType *const *pConfig*)

set PCC one peripheral clock configuration.

Parameters

<i>PCC_CtrlType*</i>	pConfig: the PCC initialize value point set by user.
----------------------	--

Returns

PCC_StatusType pcc function status

Module_driver_pmc

Collaboration diagram for Module_driver_pmc:Modules

HwA_pmc

Classes

struct **_PMC_HandleType**

The structure of the SCG processing handle.

struct **PMC_ConfigType**

Pmc control type.

Variables

void(* **_PMC_HandleType::pPmcFaultNotify**)(struct **_PMC_HandleType** *pHandle, **PMC_Fault_Type** eFault)

boolean **PMC_ConfigType::bV15CtrlEn**

boolean **PMC_ConfigType::bV15AutoswEn**

boolean **PMC_ConfigType::bRpmV25En**

uint32_t **PMC_ConfigType::u32InterruptMask**

void(* **PMC_ConfigType::pPmcFaultNotify**)(**PMC_HandleType** *pHandle, **PMC_Fault_Type** eFault)

PMC API Service IDs

#define **PMC_INIT_ID** 0x00U

PMC Dev Error Code

Error Code of calling PCC apis

#define **PMC_E_PARAM_INSTANCE** 0x01U

#define **PMC_E_PARAM_POINTER** 0x02U

PMC interrupt mask micros

enum **PMC_Fault_Type** { **PMC_NONE** = 0, **PMC_HVD5V**, **PMC_HVD2P5V**, **PMC_HVD1P1V**, **PMC_LVD5V** }

PMC fault type. This provides constants for the PMC fault type for use in the PMC interrupt callback functions.

enum **PMC_FlagType** { **PMC_HVD5V_FLAG** = **PMC_LVSCR_HVD5V_FLAG_MASK**, **PMC_HVD2P5V_FLAG** =

PMC_LVSCR_HVD2P5V_FLAG_MASK, **PMC_HVD1P1V_FLAG** = **PMC_LVSCR_HVD1P1V_FLAG_MASK**,

PMC_LVD5V_FLAG = **PMC_LVSCR_LVD5V_FLAG_MASK**, **PMC_LVR5V_FPM_FLAG** =

PMC_LVSCR_LVR5V_FPM_FLAG_MASK, **PMC_LVR5V_RPM_FLAG** = **PMC_LVSCR_LVR5V_RPM_FLAG_MASK**,

PMC_LVR2P5V_FPM_FLAG = PMC_LVSCR_LVR2P5V_FPM_FLAG_MASK, **PMC_LVR2P5V_RPM_FLAG** = PMC_LVSCR_LVR2P5V_RPM_FLAG_MASK, **PMC_LVR1P1V_FPM_FLAG** = PMC_LVSCR_LVR1P1V_FPM_FLAG_MASK, **PMC_LVR1P1V_RPM_FLAG** = PMC_LVSCR_LVR1P1V_RPM_FLAG_MASK, **PMC_POR_FLAG** = PMC_LVSCR_POR_FLAG_MASK }

PMC voltage flags.

enum **PMC_StatusType** { **PMC_HVD5V_STATUS** = PMC_LVSCR_HVD5V_STATUS_MASK, **PMC_HVD2P5V_STATUS** = PMC_LVSCR_HVD2P5V_STATUS_MASK, **PMC_HVD1P1V_STATUS** = PMC_LVSCR_HVD1P1V_STATUS_MASK, **PMC_LVD5V_STATUS** = PMC_LVSCR_LVD5V_STATUS_MASK }

PMC voltage status.

enum **PMC_IntrMaskType** { **PMC_INTR_MASK_HVD** = 1U, **PMC_INTR_MASK_LVD** = 2U }

The interrupt enable/disable mask of PMC.

typedef struct **_PMC_HandleType** **PMC_HandleType**

The structure of the SCG processing handle.

bool **PMC_GetFlag** (PMC_FlagType eFlag)

PMC get status flag.

void **PMC_ClearFlag** (PMC_FlagType eFlag)

PMC clears an individual status flag.

bool **PMC_GetStatus** (PMC_StatusType eStatus)

PMC get status.

void **Pmc_EnableInterrupt** (uint32 u32Val)

Enable PMC interrupt.

void **Pmc_DisableInterrupt** (uint32 u32Val)

Disable PMC interrupt.

void **Pmc_Init** (PMC_HandleType *pPmcHandle, PMC_ConfigType *pIntCfg)

PMC Init.

void **PMC_Deinit** (void)

Clear all PMC register.

```
void PMC_UserIRQHandler (PMC_HandleType *pPmcHandle)
```

*PMC_UserIRQHandler function Customer needs to write this function according by customer's requirement.
This function is only resolve compile conflict.*

```
#define LVD_INTERRUPT PMC_CONFIG_LVD_IE_MASK
```

```
#define HVD_INTERRUPT PMC_CONFIG_HVD_IE_MASK
```

Detailed Description

Macro Definition Documentation

```
#define HVD_INTERRUPT PMC_CONFIG_HVD_IE_MASK
```

```
#define LVD_INTERRUPT PMC_CONFIG_LVD_IE_MASK
```

```
#define PMC_E_PARAM_INSTANCE 0x01U
```

```
#define PMC_E_PARAM_POINTER 0x02U
```

```
#define PMC_INIT_ID 0x00U
```

Typedef Documentation

```
typedef struct _PMC_HandleType PMC_HandleType
```

The structure of the SCG processing handle.

Enumeration Type Documentation

enum PMC_Fault_Type

PMC fault type. This provides constants for the PMC fault type for use in the PMC interrupt callback functions.

Enumerator:

PMC_NONE	PMC no fault
PMC_HVD5V	PMC HVD5V fault
PMC_HVD2P5V	PMC HVD2P5V fault
PMC_HVD1P1V	PMC HVD1P1V fault
PMC_LVD5V	PMC LVD5V fault

enum PMC_FlagType

PMC voltage flags.

Enumerator:

PMC_HVD5V_FL AG	interrupt flag on V5 domain in FPM
PMC_HVD2P5V_ FLAG	interrupt flag on V25 domain in FPM
PMC_HVD1P1V_ FLAG	interrupt flag on V11 domain in FPM
PMC_LVD5V_FL AG	interrupt flag on V5 domain in FPM

PMC_LVR5V_FPM_FLAG	Low voltage reset flag of V5 domain in FPM
PMC_LVR5V_RPM_FLAG	Low voltage reset flag of V5 domain in RPM
PMC_LVR2P5V_FPM_FLAG	Low voltage reset flag of V25 domain in FPM
PMC_LVR2P5V_RPM_FLAG	Low voltage reset flag of V25 domain in RPM
PMC_LVR1P1V_FPM_FLAG	Low voltage reset flag of V11 domain in FPM
PMC_LVR1P1V_RPM_FLAG	Low voltage reset flag of V11 domain in RPM
PMC_POR_FLAG	POR flag

enum PMC_IntrMaskType

The interrupt enable/disable mask of PMC.

Enumerator:

PMC_INTR_MASK_HVD	interrupt mask of HVD
PMC_INTR_MASK_LVD	interrupt mask of LVD

enum PMC_StatusType

PMC voltage status.

Enumerator:

PMC_HVD5V_ST ATUS	HVD5V status on V5 domain in FPM
PMC_HVD2P5V_ STATUS	HVD25 status on V25 domain in FPM
PMC_HVD1P1V_ STATUS	HVD11 status on V11 domain in FPM
PMC_LVD5V_ST ATUS	LVD5V status on V5 domain in FPM

Function Documentation

void PMC_ClearFlag (PMC_FlagType *eFlag*)

PMC clears an individual status flag.

This function clears an individual status flag (see pmc_status_flag_t for list of status bits).

Parameters

<i>eFlag</i>	PMC flag type
--------------	---------------

void PMC_Deinit (void)

Clear all PMC register.

void Pmc_DisableInterrupt (uint32 *u32Val*)

Disable PMC interrupt.

Parameters

<i>the</i>	parameter of interrupt flag
------------	-----------------------------

void Pmc_EnableInterrupt (uint32 *u32Val*)

Enable PMC interrupt.

Parameters

<i>the</i>	parameter of interrupt flag
------------	-----------------------------

bool PMC_GetFlag (PMC_FlagType *eFlag*)

PMC get status flag.

This function returns the state of a status flag.

Parameters

<i>eFlag</i>	PMC flag type
--------------	---------------

Returns

true the flag bit is 1,false the flag bit is 0

bool PMC_GetStatus (PMC_StatusType *eStatus*)

PMC get status.

Parameters

<i>eStatus</i>	the status type
----------------	-----------------

Returns

true the status bit return 1

false the status bit return 0

void Pmc_Init (PMC_HandleType * *pPmcHandle*, PMC_ConfigType * *pIntCfg*)

PMC Init.

Parameters

<i>pPmcHandle</i>	Pmc handle
<i>pIntCfg</i>	Pmc initialize configuration

void PMC_UserIRQHandler (PMC_HandleType * *pPmcHandle*)

PMC_UserIRQHandler function Customer needs to write this function according by customer's requirement.
This function is only resolve compile conflict.

Parameters

<i>pPmcHandle</i>	Pmc handle
-------------------	------------

Variable Documentation

boolean PMC_ConfigType::bRpmV25En

bit3, V25 domain enable during RPM

boolean PMC_ConfigType::bV15AutoswEn

bit4, V15 auto switch enable

boolean PMC_ConfigType::bV15CtrlEn

bit5, V15 controller with on board NPN enable

void(* PMC_ConfigType::pPmcFaultNotify) (PMC_HandleType *pHandle, PMC_Fault_Type eFault)

PMC interrupt callback

void(* _PMC_HandleType::pPmcFaultNotify) (struct _PMC_HandleType *pHandle, PMC_Fault_Type eFault)

PMC interrupt callback

uint32_t PMC_ConfigType::u32InterruptMask

interrupt enable mask

Module_driver_port

Collaboration diagram for Module_driver_port:Modules

HwA_port

Classes

struct **PORT_DigitalFilterType**

Port digital filter configuration structure.

struct **_PORT_HandleType**

The structure of the PORT processing handle.

struct **PORT_InterruptType**

Port interrupt configuration structure.

struct **PORT_InitType**

Port initialization structure.

Macros

```
#define PORT_PIN_0 ((uint32_t)0x00000001)
#define PORT_PIN_1 ((uint32_t)0x00000002)
#define PORT_PIN_2 ((uint32_t)0x00000004)
#define PORT_PIN_3 ((uint32_t)0x00000008)
#define PORT_PIN_4 ((uint32_t)0x00000010)
#define PORT_PIN_5 ((uint32_t)0x00000020)
#define PORT_PIN_6 ((uint32_t)0x00000040)
#define PORT_PIN_7 ((uint32_t)0x00000080)
#define PORT_PIN_8 ((uint32_t)0x00000100)
#define PORT_PIN_9 ((uint32_t)0x00000200)
#define PORT_PIN_10 ((uint32_t)0x00000400)
#define PORT_PIN_11 ((uint32_t)0x00000800)
#define PORT_PIN_12 ((uint32_t)0x00001000)
#define PORT_PIN_13 ((uint32_t)0x00002000)
#define PORT_PIN_14 ((uint32_t)0x00004000)
#define PORT_PIN_15 ((uint32_t)0x00008000)
#define PORT_PIN_16 ((uint32_t)0x00010000)
#define PORT_PIN_17 ((uint32_t)0x00020000)
#define PORT_PIN_18 ((uint32_t)0x00040000)
```

```
#define PORT_PIN_19 ((uint32_t)0x00080000)
#define PORT_PIN_20 ((uint32_t)0x00100000)
#define PORT_PIN_21 ((uint32_t)0x00200000)
#define PORT_PIN_22 ((uint32_t)0x00400000)
#define PORT_PIN_23 ((uint32_t)0x00800000)
#define PORT_PIN_24 ((uint32_t)0x01000000)
#define PORT_PIN_25 ((uint32_t)0x02000000)
#define PORT_PIN_26 ((uint32_t)0x04000000)
#define PORT_PIN_27 ((uint32_t)0x08000000)
#define PORT_PIN_28 ((uint32_t)0x10000000)
#define PORT_PIN_29 ((uint32_t)0x20000000)
#define PORT_PIN_30 ((uint32_t)0x40000000)
#define PORT_PIN_31 ((uint32_t)0x80000000)
#define PORT_ALT0_FUNC_MODE ((Port_PinModeType)0)
```

Port Alternate 0 Mode.

```
#define PORT_GPIO_MODE ((Port_PinModeType)1)
```

Port GPIO Mode.

```
#define PORT_ALT2_FUNC_MODE ((Port_PinModeType)2)
```

Port Alternate 2 Mode.

```
#define PORT_ALT3_FUNC_MODE ((Port_PinModeType)3)
```

Port Alternate 3 Mode.

```
#define PORT_ALT4_FUNC_MODE ((Port_PinModeType)4)
```

Port Alternate 4 Mode.

```
#define PORT_ALT5_FUNC_MODE ((Port_PinModeType)5)
```

Port Alternate 5 Mode.

```
#define PORT_ALT6_FUNC_MODE ((Port_PinModeType)6)
```

Port Alternate 6 Mode.

```
#define PORT_ALT7_FUNC_MODE ((Port_PinModeType)7)
```

Port Alternate 7 Mode.

Typedefs

typedef struct **_PORT_HandleType** **PORT_HandleType**

The structure of the PORT processing handle.

Enumerations

enum **PORT_IrqcConfigurationType** { **PORT_IRQ_DISABLE** = 0U, **PORT_IRQ_DMA_RISING** = 1U, **PORT_IRQ_DMA_FALLING** = 2U, **PORT_IRQ_DMA_BOTH_EDGE** = 3U, **PORT_IRQ_LOGIC_0** = 8U, **PORT_IRQ_RISING** = 9U, **PORT_IRQ_FALLING** = 10U, **PORT_IRQ_BOTH_EDGE** = 11U, **PORT_IRQ_LOGIC_1** = 12U, **PORT_IRQ_TOOGLED_WHEN_LOGIC_0** = 13U, **PORT_IRQ_TOOGLED_WHEN_LOGIC_1** = 14U }

Port PCR[IRQC] configuration.

enum **PORT_PullStatusType** { **PORT_PULL_DOWN** = 0U, **PORT_PULL_UP** }

Port pull status.

enum **PORT_GpioDirType** { **PORT_GPIO_IN** = 0U, **PORT_GPIO_OUT**, **PORT_GPIO_ZERO** }

Gpio direction.

enum **PORT_GpioLevelType** { **PORT_GPIO_LOW** = 0U, **PORT_GPIO_HIGH** }

Gpio initialization level.

enum **PORT_DigitalFilterClkSrcType** { **PORT_FILTER_BUS_CLK** = 0U, **PORT_FILTER_AON32K_CLK** }

Port digital filter clock source.

enum **PORT_InstanceType** { **PORT_A** = 0U, **PORT_B**, **PORT_C**, **PORT_D**, **PORT_E**, **PORT_F**, **PORT_G**, **PORT_H**, **PORT_I** }

Port instance number.

Functions

void **PORT_InitPins** (**PORT_HandleType** *pPortHandle, const **PORT_InitType** *const pInitStruct)

Initialize port pins.

void **PORT_Deinit** (**PORT_HandleType** *pPortHandle, const uint32_t u32Pins)

De-initialize the Port instance.

void **PORT_EnableEmergencyStop** (**PORT_HandleType** *pPortHandle, const uint32_t u32Pins, bool bEnable)

set Emergency Stop of pin

```
void PORT_SetInterruptCfg (PORT_HandleType *pPortHandle, const uint32_t u32Pins,  
PORT_IrqcConfigurationType ePortPinIrq)
```

set interrupt config of port

```
void PORT_InitDigitalFilterPort (PORT_HandleType *pPortHandle, const PORT_DigitalFilterType *const  
pDFStruct)
```

Initialize digital filter for Port instance.

```
void PORT_DeinitDigitalFilterPort (PORT_HandleType *pPortHandle)
```

De-initialize digital filter for Port instance.

```
void PORT_EnableDigitalFilterPin (PORT_HandleType *pPortHandle, const uint32_t u32Pins)
```

Enable the digital filter function for the specific pin.

```
void PORT_DisableDigitalFilterPin (PORT_HandleType *pPortHandle, const uint32_t u32Pins)
```

Disable the digital filter function for the specific pin.

```
void PORT_IRQHandler (PORT_HandleType *pPortHandle)
```

PORT interrupt process function.

PORT API Service IDs

```
#define PORT_InitPins_ID 0U
```

```
#define PORT_Deinit_ID 1U
```

```
#define PORT_SetPinMode_ID 2U
```

```
#define PORT_SetInterruptCfg_ID 3U
```

```
#define PORT_InitDigitalFilterPort_ID 4U
```

```
#define PORT_DeinitDigitalFilterPort_ID 5U
```

```
#define PORT_EnableDigitalFilterPin_ID 6U
```

```
#define PORT_DisableDigitalFilterPin_ID 7U
```

```
#define PORT_IRQHandler_ID 8U
```

```
#define PORT_SetPinEmergencyStop_ID 9U
```

PORT Dev Error Code

Error Code of calling GPIO apis

```
#define PORT_E_PARAM_INSTANCE 0x01U
#define PORT_E_PARAM_INPUT 0x02U
#define PORT_E_PARAM_PIN 0x03U
```

Detailed Description

Macro Definition Documentation

```
#define PORT_ALT0_FUNC_MODE ((Port_PinModeType)0)
```

Port Alternate 0 Mode.

```
#define PORT_ALT2_FUNC_MODE ((Port_PinModeType)2)
```

Port Alternate 2 Mode.

```
#define PORT_ALT3_FUNC_MODE ((Port_PinModeType)3)
```

Port Alternate 3 Mode.

```
#define PORT_ALT4_FUNC_MODE ((Port_PinModeType)4)
```

Port Alternate 4 Mode.

```
#define PORT_ALT5_FUNC_MODE ((Port_PinModeType)5)
```

Port Alternate 5 Mode.

```
#define PORT_ALT6_FUNC_MODE ((Port_PinModeType)6)
```

Port Alternate 6 Mode.

```
#define PORT_ALT7_FUNC_MODE ((Port_PinModeType)7)
```

Port Alternate 7 Mode.

```
#define PORT_Deinit_ID 1U
```

```
#define PORT_DeinitDigitalFilterPort_ID 5U
```

```
#define PORT_DisableDigitalFilterPin_ID 7U
```

```
#define PORT_E_PARAM_INPUT 0x02U
```

```
#define PORT_E_PARAM_INSTANCE 0x01U
```

```
#define PORT_E_PARAM_PIN 0x03U
```

```
#define PORT_EnableDigitalFilterPin_ID 6U
```

```
#define PORT_GPIO_MODE ((Port_PinModeType)1)
```

Port GPIO Mode.

```
#define PORT_InitDigitalFilterPort_ID 4U
```

```
#define PORT_InitPins_ID 0U
```

```
#define PORT_IRQHandler_ID 8U
```

```
#define PORT_PIN_0 ((uint32_t)0x00000001)
```

PORT Pin 0 select

```
#define PORT_PIN_1 ((uint32_t)0x00000002)
```

PORT Pin 1 select

```
#define PORT_PIN_10 ((uint32_t)0x00000400)
```

PORT Pin 10 select

```
#define PORT_PIN_11 ((uint32_t)0x00000800)
    PORT Pin 11 select

#define PORT_PIN_12 ((uint32_t)0x00001000)
    PORT Pin 12 select

#define PORT_PIN_13 ((uint32_t)0x00002000)
    PORT Pin 13 select

#define PORT_PIN_14 ((uint32_t)0x00004000)
    PORT Pin 14 select

#define PORT_PIN_15 ((uint32_t)0x00008000)
    PORT Pin 15 select

#define PORT_PIN_16 ((uint32_t)0x00010000)
    PORT Pin 16 select

#define PORT_PIN_17 ((uint32_t)0x00020000)
    PORT Pin 17 select

#define PORT_PIN_18 ((uint32_t)0x00040000)
    PORT Pin 18 select

#define PORT_PIN_19 ((uint32_t)0x00080000)
    PORT Pin 19 select

#define PORT_PIN_2 ((uint32_t)0x00000004)
    PORT Pin 2 select

#define PORT_PIN_20 ((uint32_t)0x00100000)
    PORT Pin 20 select

#define PORT_PIN_21 ((uint32_t)0x00200000)
    PORT Pin 21 select
```

```
#define PORT_PIN_22 ((uint32_t)0x00400000)
    PORT Pin 22 select

#define PORT_PIN_23 ((uint32_t)0x00800000)
    PORT Pin 23 select

#define PORT_PIN_24 ((uint32_t)0x01000000)
    PORT Pin 24 select

#define PORT_PIN_25 ((uint32_t)0x02000000)
    PORT Pin 25 select

#define PORT_PIN_26 ((uint32_t)0x04000000)
    PORT Pin 26 select

#define PORT_PIN_27 ((uint32_t)0x08000000)
    PORT Pin 27 select

#define PORT_PIN_28 ((uint32_t)0x10000000)
    PORT Pin 28 select

#define PORT_PIN_29 ((uint32_t)0x20000000)
    PORT Pin 29 select

#define PORT_PIN_3 ((uint32_t)0x00000008)
    PORT Pin 3 select

#define PORT_PIN_30 ((uint32_t)0x40000000)
    PORT Pin 30 select

#define PORT_PIN_31 ((uint32_t)0x80000000)
    PORT Pin 31 select

#define PORT_PIN_4 ((uint32_t)0x00000010)
    PORT Pin 4 select
```



```
#define PORT_PIN_5 ((uint32_t)0x00000020)
```

PORT Pin 5 select

```
#define PORT_PIN_6 ((uint32_t)0x00000040)
```

PORT Pin 6 select

```
#define PORT_PIN_7 ((uint32_t)0x00000080)
```

PORT Pin 7 select

```
#define PORT_PIN_8 ((uint32_t)0x00000100)
```

PORT Pin 8 select

```
#define PORT_PIN_9 ((uint32_t)0x00000200)
```

PORT Pin 9 select

```
#define PORT_SetInterruptCfg_ID 3U
```

```
#define PORT_SetPinEmergencyStop_ID 9U
```

```
#define PORT_SetPinMode_ID 2U
```

Typedef Documentation

```
typedef struct _PORT_HandleType PORT_HandleType
```

The structure of the PORT processing handle.

Enumeration Type Documentation

```
enum PORT_DigitalFilterClkSrcType
```

Port digital filter clock source.

Enumerator:

PORT_FILTER_B US_CLK	
PORT_FILTER_A ON32K_CLK	

enum PORT_GpioDirType

Gpio direction.

Enumerator:

PORT_GPIO_IN	
PORT_GPIO_OU T	
PORT_GPIO_ZE RO	

enum PORT_GpioLevelType

Gpio initialization level.

Enumerator:

PORT_GPIO_LO W	
PORT_GPIO_HI GH	

enum PORT_InstanceType

Port instance number.

Enumerator:

PORT_A	
PORT_B	
PORT_C	

PORT_D	
PORT_E	
PORT_F	
PORT_G	
PORT_H	
PORT_I	

enum PORT_IrqConfigurationType

Port PCR[IRQC] configuration.

Enumerator:

PORT_IRQ_DISABLE	
PORT_IRQ_DMA_RISING	
PORT_IRQ_DMA_FALLING	
PORT_IRQ_DMA_BOTH_EDGE	
PORT_IRQ_LOGIC_0	
PORT_IRQ_RISING	
PORT_IRQ_FALLING	
PORT_IRQ_BOTH_EDGE	
PORT_IRQ_LOGIC_1	
PORT_IRQ_TOGGLED_WHEN_LOGIC_0	
PORT_IRQ_TOGGLED_WHEN_LOGIC_1	

enum PORT_PullStatusType

Port pull status.

Enumerator:

PORT_PULL_DO WN	
PORT_PULL_UP	

Function Documentation

void PORT_Deinit (PORT_HandleType * *pPortHandle*, const uint32_t *u32Pins*)

De-initialize the Port instance.

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
<i>u32Pins</i>	The bit of u32Pins indicate the Pin number of this Port.

void PORT_DeinitDigitalFilterPort (PORT_HandleType * *pPortHandle*)

De-initialize digital filter for Port instance.

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
--------------------	--

void PORT_DisableDigitalFilterPin (PORT_HandleType * *pPortHandle*, const uint32_t *u32Pins*)

Disable the digital filter function for the specific pin.

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
<i>u32Pins</i>	The bit of u32Pins indicate the Pin number of this Port.

void PORT_EnableDigitalFilterPin (PORT_HandleType * *pPortHandle*, const uint32_t *u32Pins*)

Enable the digital filter function for the specific pin.

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
<i>u32Pins</i>	The bit of u32Pins indicate the Pin number of this Port.

void PORT_EnableEmergencyStop (PORT_HandleType * *pPortHandle*, const uint32_t *u32Pins*, bool *bEnable*)

set Emergency Stop of pin

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
<i>u32Pins</i>	The bit of u32Pins indicate the Pin number of this Port.
<i>bEnable</i>	Enable or Disable.

void PORT_InitDigitalFilterPort (PORT_HandleType * *pPortHandle*, const PORT_DigitalFilterType *const *pDFStruct*)

Initialize digital filter for Port instance.

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
<i>pDFStruct</i>	Digital filter initialization structure of port

void PORT_InitPins (PORT_HandleType * *pPortHandle*, const PORT_InitType *const *pInitStruct*)

Initialize port pins.

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
<i>pInitStruct</i>	Initialization structure of port

void PORT_IRQHandler (PORT_HandleType * pPortHandle)

PORT interrupt process function.

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
--------------------	--

void PORT_SetInterruptCfg (PORT_HandleType * pPortHandle, const uint32_t u32Pins, PORT_IrqcConfigurationType ePortPinIrq)

set interrupt config of port

Parameters

<i>pPortHandle</i>	pPortHandle PORT handle for PORT functionality
<i>u32Pins</i>	The bit of u32Pins indicate the Pin number of this Port.
<i>ePortPinIrq</i>	The config of interrupt.

Module_driver_ptimer

Collaboration diagram for Module_driver_ptimer:Modules

HwA_ptimer

Classes

- struct **_PTIMER_HandleType**
Ptimer process handler.
- struct **PTIMER_InitType**
Ptimer instance initialization parameters.
- struct **PTIMER_ChannelCfgType**
Ptimer channel configuration parameters.
- struct **PTIMER_PulseOutType**
Ptimer pulse out configurations.

Typedefs

typedef struct **_PTIMER_HandleType** **PTIMER_HandleType**
Ptimer process handler.

Enumerations

enum **PTIMER_ChannelSelectionType** { **PTIMER0_USED_FOR_ADC0** = 0U, **PTIMER0_USED_FOR_ADC4**,
PTIMER3_USED_FOR_ADC3, **PTIMER3_USED_FOR_ADC5** }

PTIMER Chanenl1 Selection Type.

enum **PTIMER_InstanceType** { **PTIMER_INSTANCE_0** = 0U, **PTIMER_INSTANCE_1** = 1U,
PTIMER_INSTANCE_2 = 2U, **PTIMER_INSTANCE_3** = 3U }

The instance index of the PTIMER peripheral.

Functions

void **PTIMER_InitStructure** (**PTIMER_InitType** *const pInitCfg)
Provide the default values of PTIMER_InitType.

void **PTIMER_Init** (**PTIMER_HandleType** *PtimerHandle, const **PTIMER_InitType** *const pInitCfg)
Initialize the Ptimer instance.

void **PTIMER_DeInit** (**PTIMER_HandleType** *PtimerHandle)
De-initialize the Ptimer instance Disable the Ptimer instance and reset the configurations to its reset values.

void **PTIMER_InitChannel** (**PTIMER_HandleType** *PtimerHandle, const **PTIMER_ChannelCfgType**
aChannelCfg[], const uint8_t u8ChnNum)
Initialize the Ptimer channels.

void **PTIMER_SetPeriod** (const **PTIMER_HandleType** *PtimerHandle, uint16 u16PtimerPeriod)
Set the Ptimer max counter period in microsecond When the Ptimer counter reaches the period, it will return to zero.

void **PTIMER_SetPulseOut** (const **PTIMER_HandleType** *PtimerHandle, const **PTIMER_PulseOutType**
*pPulseOutCfg)
Set the Ptimer pulse-out function.

void **PTIMER_LoadValue** (const **PTIMER_HandleType** *PtimerHandle)

Load the buffered values into register.

void **PTIMER_Enable** (const **PTIMER_HandleType** *PtimerHandle)

Enable the Ptimer instance.

void **PTIMER_Disable** (const **PTIMER_HandleType** *PtimerHandle)

Disable the Ptimer instance.

void **PTIMER_EnablePulseOut** (const **PTIMER_HandleType** *PtimerHandle)

Enable the Ptimer pulse-out function.

void **PTIMER_DisablePulseOut** (const **PTIMER_HandleType** *PtimerHandle)

Disable the Ptimer pulse-out function.

void **PTIMER_SetInstance01BackToBackEnable** (bool bEnable)

void **PTIMER_SetInstance23BackToBackEnable** (bool bEnable)

void **PTIMERn_IRQHandler** (**PTIMER_HandleType** *PtimerHandle)

void **PTIMER_GenerateSWTrigger** (const **PTIMER_HandleType** *PtimerHandle)

Generate the software trigger signal for the Ptimer instance.

void **PTIMER_ChannelSelection** (**PTIMER_ChannelSelectionType** selection)

Set PTIMER channel used for specific ADC instance.

FTU API Service IDs

#define **PTIMER_INIT_ID** 0U

#define **PTIMER_INIT_CHANNEL_ID** 1U

#define **PTIMER_INIT_STRUCTURE_ID** 2U

#define **PTIMER_DEINIT_ID** 3U

#define **PTIMER_SET_PERIOD_ID** 4U

#define **PTIMER_SET_PULSE_OUT_ID** 5U

#define **PTIMER_LOAD_VALUE_ID** 6U

#define **PTIMER_ENABLE_ID** 7U


```
#define PTIMER_DISABLE_ID 8U
#define PTIMER_PULSE_OUT_ID 9U
#define PTIMER_DISABLE_PULSE_OUT_ID 10U
#define PTIMER_GENERATE_SW_TRIGGER_ID 11U
```

FTU Dev Error Code

Error Code of calling FTU apis

```
#define PTIMER_E_PARAM_INSTANCE 0x01U
#define PTIMER_E_PARAM_CHANNEL 0x02U
#define PTIMER_E_PARAM_POINTER 0x03U
#define PTIMER_E_PARAM_DELAY_VALUE 0x04U
```

Detailed Description

Macro Definition Documentation

#define PTIMER_DEINIT_ID 3U

#define PTIMER_DISABLE_ID 8U

#define PTIMER_DISABLE_PULSE_OUT_ID 10U

#define PTIMER_E_PARAM_CHANNEL 0x02U

#define PTIMER_E_PARAM_DELAY_VALUE 0x04U

#define PTIMER_E_PARAM_INSTANCE 0x01U

#define PTIMER_E_PARAM_POINTER 0x03U

#define PTIMER_ENABLE_ID 7U

#define PTIMER_GENERATE_SW_TRIGGER_ID 11U

#define PTIMER_INIT_CHANNEL_ID 1U

#define PTIMER_INIT_ID 0U

#define PTIMER_INIT_STRUCTURE_ID 2U

#define PTIMER_LOAD_VALUE_ID 6U

#define PTIMER_PULSE_OUT_ID 9U

#define PTIMER_SET_PERIOD_ID 4U

#define PTIMER_SET_PULSE_OUT_ID 5U

Typedef Documentation

typedef struct _PTIMER_HandleType PTIMER_HandleType

Ptimer process handler.

Enumeration Type Documentation

enum PTIMER_ChannelSelectionType

PTIMER Chanenl1 Selection Type.

Enumerator:

PTIMER0_USED _FOR_ADC0	
PTIMER0_USED _FOR_ADC4	
PTIMER3_USED _FOR_ADC3	
PTIMER3_USED _FOR_ADC5	

enum PTIMER_InstanceType

The instance index of the PTIMER peripheral.

Enumerator:

PTIMER_INSTAN CE_0	
PTIMER_INSTAN CE_1	
PTIMER_INSTAN CE_2	
PTIMER_INSTAN CE_3	

Function Documentation

void PTIMER_ChannelSelection (PTIMER_ChannelSelectionType *selection*)

Set PTIMER channel used for specific ADC instance.

Parameters

<i>selection</i>	the Ptimer channel selection type
------------------	-----------------------------------

void PTIMER_DeInit (PTIMER_HandleType * *PtimerHandle*)

De-initialize the Ptimer instance Disable the Ptimer instance and reset the configurations to its reset values.

Parameters

<i>elInstance</i>	the Ptimer instance to use
-------------------	----------------------------

void PTIMER_Disable (const PTIMER_HandleType * *PtimerHandle*)

Disable the Ptimer instance.

Parameters

<i>elInstance</i>	the Ptimer instance to use
-------------------	----------------------------

void PTIMER_DisablePulseOut (const PTIMER_HandleType * *PtimerHandle*)

Disable the Ptimer pulse-out function.

Parameters

<i>elInstance</i>	the Ptimer instance to use
-------------------	----------------------------

void PTIMER_Enable (const PTIMER_HandleType * *PtimerHandle*)

Enable the Ptimer instance.

Parameters

<i>elInstance</i>	the Ptimer instance to use
-------------------	----------------------------

void PTIMER_EnablePulseOut (const PTIMER_HandleType * *PtimerHandle*)

Enable the Ptimer pulse-out function.

Parameters

<i>elInstance</i>	the Ptimer instance to use
-------------------	----------------------------

void PTIMER_GenerateSWTrigger (const PTIMER_HandleType * *PtimerHandle*)

Generate the software trigger signal for the Ptimer instance.

If the Ptimer trigger source is selected as PTIMER_TRGSRC_SW, call this function will make the Ptimer instance to start.

Parameters

<i>elInstance</i>	the Ptimer instance to use
-------------------	----------------------------

void PTIMER_Init (PTIMER_HandleType * *PtimerHandle*, const PTIMER_InitType *const *pInitCfg*)

Initialize the Ptimer instance.

Parameters

<i>elInstance</i>	the Ptimer instance to use
<i>pInitCfg</i>	the Ptimer initialization parameters

void PTIMER_InitChannel (PTIMER_HandleType * *PtimerHandle*, const PTIMER_ChannelCfgType *aChannelCfg*[], const uint8_t *u8ChnNum*)

Initialize the Ptimer channels.

Note

the channel delay value is buffered and will take effect only after called **PTIMER_LoadValue()** function.

Parameters

<i>eInstance</i>	the Ptimer instance to use
<i>aChannelCfg</i>	the array of channels to config
<i>u8ChnNum</i>	the channel numbers in the channel array

void PTIMER_InitStructure (PTIMER_InitType *const *pInitCfg*)

Provide the default values of **PTIMER_InitType**.

Parameters

<i>pInitCfg</i>	the structure to initialize
-----------------	-----------------------------

void PTIMER_LoadValue (const PTIMER_HandleType * *PtimerHandle*)

Load the buffered values into register.

Some Ptimer registers are buffered and will only take effect after called this function

Parameters

<i>eInstance</i>	the Ptimer instance to use
------------------	----------------------------

void PTIMER_SetInstance01BackToBackEnable (bool *bEnable*)

void PTIMER_SetInstance23BackToBackEnable (bool *bEnable*)

void PTIMER_SetPeriod (const PTIMER_HandleType * *PtimerHandle*, uint16 *u16PtimerPeriod*)

Set the Ptimer max counter period in microsecond When the Ptimer counter reaches the period, it will return to zero.

Note

the period parameter is buffered and will take effect only after called **PTIMER_LoadValue()** function.

Parameters

<i>eInstance</i>	the Ptimer instance to use
<i>u32PeriodUs</i>	the max counter period in microsecond

```
void PTIMER_SetPulseOut (const PTIMER_HandleType * PtimerHandle, const PTIMER_PulseOutType * pPulseOutCfg)
```

Set the Ptimer pulse-out function.

Note

the pulse-out delay values are buffered and will take effect only after called **PTIMER_LoadValue()** function.

Parameters

<i>eInstance</i>	the Ptimer instance to use
<i>pPulseOutCfg</i>	the pulse-out configuration

```
void PTIMERn_IRQHandler (PTIMER_HandleType * PtimerHandle)
```

Module_driver_rgm

Collaboration diagram for Module_driver_rgm:Modules

HwA_rgm

Classes

struct **_RGM_HandleType**
The structure of the SCG processing handle.
struct **RGM_ConfigType**
Rgm configure type.

Typedefs

typedef struct **_RGM_HandleType** **RGM_HandleType**
The structure of the SCG processing handle.

Enumerations

enum **RGM_StatusType** { **RGM_STATUS_SUCCESS** = 0U, **RGM_STATUS_PARAM_INVALID** = 1U, **RGM_STATUS_COREID_INVALID** = 2U, **RGM_STATUS_CLOCK_INVALID** }
Rgm return type.

enum **RGM_FilterClkSrc** { **RGM_RESET_FILTER_DISABLE** = 0U, **RGM_RESET_FILTER_BUS_CLOCK** = 1U }

Rgm reset pin filter clock type.

Functions

uint32_t **RGM_GetSystemLastResetFlag** (void)

This api can get RGM_SRS register that indicate the source of the most recent reset.

uint32_t **RGM_GetSystemAllResetFlag** (void)

This api can get RGM_SSRS register that indicate all reset sources since the last POR or LVD that have not been cleared by software.

void **RGM_ClearSystemAllResetFlagAfterPOR** (void)

This api can clear all reset flag of RGM_SSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

RGM_StatusType **RGM_EnableResetFilter** (**RGM_FilterClkSrc** eClk, uint8_t u8BusClockFilterWidth, bool bLpClkEn)

Enable reset pin filter.

RGM_StatusType **RGM_DisableResetFilter** (**RGM_FilterClkSrc** eClk, bool bLpClkEn)

Disable reset pin filter.

RGM_StatusType **RGM_EnableSystemResetInt** (**RGM_ResetDelayType** eDelay, **RGM_SysResetIntMangerType** eResetInterrupt)

This api can enable interrupt before an system reset appear.

void **RGM_DisableSystemResetInt** (**RGM_SysResetIntMangerType** eResetInterrupt)

This api can disable interrupt before an system reset appear.

void **RGM_GenerateSystemCoreReset** (void)

Generate software reset through cortex-m register.

void **RGM_GenerateCpu0SwReset** (void)

Generate software reset through RGM C0_RST register.

RGM_StatusType RGM_DisableCPU0CoreResetInt (RGM_CoreResetIntMangerType eCPU0Interrupt)

This api can disable interrupt before an CPU0 core related reset appear.

RGM_CoreResetStatusType RGM_GetCPU0OutResetFlag (void)

Get the CPU0 exit reset flag.

uint32_t RGM_GetCPU0LastResetFlag (void)

This api can get RGM_C0_SRS register that indicate the source of the most recent CPU0 reset.

uint32_t RGM_GetCPU0AllResetFlag (void)

This api can get RGM_C0_SSRS register that indicate all CPU0 reset sources since the last POR or LVD that have not been cleared by software.

void RGM_ClearCPU0AllResetFlagAfterPOR (void)

This api can clear all reset flag of RGM_C0_SSRS register which indicate CPU0 all reset sources since the last POR or LVD that have not been cleared by software.

void RGM_Pre_UserIRQHandler (RGM_HandleType *pRgmHandle)

RGM Pre-interrupt entry.

void RGM_Init (RGM_HandleType *pRgmHandle, RGM_ConfigType *pInitCfg)

RGM initialize function.

RGM API Service IDs

#define RGM_INIT_ID 0x00U

RGM Dev Error Code

Error Code of calling RGM apis

#define RGM_E_PARAM_DELAY_TIME 0x01U

#define RGM_E_PARAM_POINTER 0x02U

Detailed Description

Macro Definition Documentation

#define RGM_E_PARAM_DELAY_TIME 0x01U

#define RGM_E_PARAM_POINTER 0x02U

#define RGM_INIT_ID 0x00U

Typedef Documentation

typedef struct _RGM_HandleType RGM_HandleType

The structure of the SCG processing handle.

Enumeration Type Documentation

enum RGM_FilterClkSrc

Rgm reset pin filter clock type.

Enumerator:

RGM_RESET_FILTER_DISABLE	Reset Pin Filter Disable
RGM_RESET_FILTER_BUS_CLOCK	Reset Pin Filter Bus Clock

enum RGM_StatusType

Rgm return type.

Enumerator:

RGM_STATUS_SUCCESS	Return status success
RGM_STATUS_PARAMETER_INVALID	Return status parameter invalid
RGM_STATUS_COREID_INVALID	Return status core id invalid
RGM_STATUS_CLOCK_INVALID	Return status clock invalid

Function Documentation

void RGM_ClearCPU0AllResetFlagAfterPOR (void)

This api can clear all reset flag of RGM_C0_SSRS register which indicate CPU0 all reset sources since the last POR or LVD that have not been cleared by software.

void RGM_ClearSystemAllResetFlagAfterPOR (void)

This api can clear all reset flag of RGM_SSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

RGM_StatusType RGM_DisableCPU0CoreResetInt (RGM_CoreResetIntMangerType *eCPU0Interrupt*)

This api can disable interrupt before an CPU0 core related reset appear.

Parameters

<i>eCPU0Interrupt</i>	Reset event flag, like: RGM_CPU_INT_SWRST RGM_CPU_INT_SYSRST
-----------------------	---

Returns

RGM return type

RGM_StatusType RGM_DisableResetFilter (RGM_FilterClkSrc *eClk*, bool *bLpClkEn*)

Disable reset pin filter.

Parameters

<i>eClk</i>	Reset pin filter clock source
<i>bLpClkEn</i>	select whether disable reset pin filter using AON32clock in low power mode

Returns

RGM return type

void RGM_DisableSystemResetInt (RGM_SysResetIntMangerType *eResetInterrupt*)

This api can disable interrupt before an system reset appear.

Parameters

<i>eResetInterrupt</i>	Reset event flag, like: RGM_INT_CLKERR0 RGM_INT_FCSMU
------------------------	---

Returns

RGM return type

RGM_StatusType RGM_EnableResetFilter (RGM_FilterClkSrc *eClk*, uint8_t *u8BusClockFilterWidth*, bool *bLpClkEn*)

Enable reset pin filter.

Parameters

<i>eClk</i>	Reset pin filter clock source
<i>u8BusClockFilterWidth</i>	Bus clock filter width
<i>bLpClkEn</i>	select whether enable reset pin filter using AON32clock in low power mode

Returns

RGM return type

Note

If use AON32K clock, A reset signal whose length is less than 2 AON32K clock periods will be filtered

**RGM_StatusType RGM_EnableSystemResetInt (RGM_ResetDelayType *eDelay*,
RGM_SysResetIntMangerType *eResetInterrupt*)**

This api can enable interrupt before an system reset appear.

Parameters

<i>eDelay</i>	Enumeration of delay cycles
<i>eResetInterrupt</i>	Reset event flag, like: RGM_INT_CLKERR0 RGM_INT_FCSMU

Returns

RGM return type

Note

Here is the interrupted master switch control

void RGM_GenerateCpu0SwReset (void)

Generate software reset through RGM C0_RST register.

void RGM_GenerateSystemCoreReset (void)

Generate software reset through cortex-m register.

uint32_t RGM_GetCPU0AllResetFlag (void)

This api can get RGM_C0_SSRS register that indicate all CPU0 reset sources since the last POR or LVD that have not been cleared by software.

@returnRGM->RGM_C0_SSRS register, bit 0-20,29-31 corresponding to RGM_CPUResetEventType, refer to reference manual for details.

uint32_t RGM_GetCPU0LastResetFlag (void)

This api can get RGM_C0_SRS register that indicate the source of the most recent CPU0 reset.

Returns

RGM->RGM_C0_SRS register, bit 0-20 corresponding to RGM_ResetEventType, refer to reference manual for details.

Note

Multiple flags can be set if multiple reset events occur at the same time

RGM_CoreResetStatusType RGM_GetCPU0OutResetFlag (void)

Get the CPU0 exit reset flag.

Returns

RGM_CPU_OUT_RST_UNDER CPU0 is under reset

RGM_CPU_OUT_RST_OUT CPU0 is out of reset

uint32_t RGM_GetSystemAllResetFlag (void)

This api can get RGM_SSRS register that indicate all reset sources since the last POR or LVD that have not been cleared by software.

Returns

RGM->RGM_SSRS register, bit 0-15,29-31 corresponding to RGM_ResetEventType, refer to reference manual for details.

uint32_t RGM_GetSystemLastResetFlag (void)

This api can get RGM_SRS register that indicate the source of the most recent reset.

Returns

RGM->RGM_SRS register, bit 0-15,29-31 corresponding to RGM_ResetEventType, refer to reference manual for details.

Note

Multiple flags can be set if multiple reset events occur at the same time

void RGM_Init (RGM_HandleType * *pRgmHandle*, RGM_ConfigType * *pInitCfg*)

RGM initialize function.

Parameters

<i>pRgmHandle</i>	RGM handle
<i>pInitCfg</i>	Init pointer

Note

1. RGM system related register configured by core 0
2. If core1/core2 use RGM, both core0 and core1/core2 need call RGM_Init and different pRgmHandle need be given

void RGM_Pre_UserIRQHandler (RGM_HandleType * *pRgmHandle*)

RGM Pre-interrupt entry.

Module_driver_rtc

Collaboration diagram for Module_driver_rtc:Modules

HwA_rtc

Classes

struct **_RTC_HandleType**

The structure of the RTC processing handle.

struct **RTC_InitType**

Rtc initialization type.

struct **RTC_TimeDate**

struct **RTC_Compensation**

Macros

#define **SECONDS_DAY** 86400UL

#define **SECONDS_HOUR** 3600UL

#define **SECONDS_MINUTE** 60UL

#define **MINUTES_HOUR** 60UL

#define **HOURS_DAY** 24UL

#define **DAYS_LEAP_YEAR** 366UL

#define **DAYS_COM_YEAR** 365UL

#define **START_YEAR** 1970UL

#define **END_YEAR** 2099UL

Typedefs

typedef struct **_RTC_HandleType** **RTC_HandleType**

The structure of the RTC processing handle.

Enumerations

enum **RTC_InstanceType** { **RTC_INSTANCE_0** = 0U }

The instance index of the RTC peripheral.

enum **RTC_IntEventType** { **RTC_ALARM_INT** = 0U, **RTC_SECOND_INT** = 1U, **RTC_OVERFLOW_INT** = 2U }

Rtc mode.

Functions

void **RTC_Init** (**RTC_HandleType** *pRtcHandle, const **RTC_InitType** *const pInitStruct)

Initialize Rtc instance.

void **RTC_Deinit** (**RTC_HandleType** *pRtcHandle)

De-initialize Rtc instance.

void **RTC_Start** (**RTC_HandleType** *pRtcHandle)

Rtc start.

void **RTC_Stop** (**RTC_HandleType** *pRtcHandle)

Rtc stop.

void **RTC_SetCountValue** (**RTC_HandleType** *pRtcHandle, uint32_t u32CounetValue)

set count value

void **RTC_UpdateAlarmValue** (**RTC_HandleType** *pRtcHandle, const uint32_t u32AlarmValue)

Rtc alarm value.

uint32_t **RTC_CheckOverflowFlag** (**RTC_HandleType** *pRtcHandle)

Check RTC overflow flag.

void **RTC_GetTimeDate** (**RTC_HandleType** *pRtcHandle, **RTC_TimeDate** *const pTimeDate)

Get time and date.

void **RTC_SetTimeDate** (**RTC_HandleType** *pRtcHandle, const **RTC_TimeDate** *const pTimeDate)

Set time and date.

void **RTC_GetCompensation** (**RTC_HandleType** *pRtcHandle, **RTC_Compensation** *const pCompensation)

Get time and date compensation information.

void **RTC_SetCompensation** (**RTC_HandleType** *pRtcHandle, const **RTC_Compensation** *const pCompensation)

Set time and date compensation value.

void **Rtc_CommonProcessInterrupt** (**RTC_HandleType** *pRtcHandle)

RTC common interrupt function.

RTC API Service IDs

#define **RTC_INIT_ID** 0U

#define **RTC_DEINIT_ID** 1U

#define **RTC_START_ID** 2U

#define **RTC_STOP_ID** 3U

```
#define RTC_SET_COUNT_ID 4U
#define RTC_UPDATE_ALARM_ID 5U
#define RTC_CHECK_OVERFLOW_FLAF_ID 6U
#define RTC_GET_TIMEDATE_ID 7U
#define RTC_SET_TIMEDATE_ID 8U
#define RTC_GET_COMPENSATION_ID 9U
#define RTC_SET_COMPENSATION_ID 10U
#define RTC_INTE_PROC_ID 11U
#define RTC_IRQ_HANDLER_ID 12U
```

FTU Dev Error Code

Error Code of calling RTC apis

```
#define RTC_E_PARAM_INSTANCE 0x01U
#define RTC_E_PARAM_CLOCK 0x02U
#define RTC_E_PARAM_POINTER 0x03U
#define RTC_E_PARAM_FORMAT 0x04U
```

Detailed Description

Macro Definition Documentation

#define DAYS_COM_YEAR 365UL

#define DAYS_LEAP_YEAR 366UL

#define END_YEAR 2099UL

#define HOURS_DAY 24UL

#define MINUTES_HOUR 60UL

#define RTC_CHECK_OVERFLOW_FLAF_ID 6U

#define RTC_DEINIT_ID 1U

#define RTC_E_PARAM_CLOCK 0x02U

#define RTC_E_PARAM_FORMAT 0x04U

#define RTC_E_PARAM_INSTANCE 0x01U

#define RTC_E_PARAM_POINTER 0x03U

#define RTC_GET_COMPENSATION_ID 9U

#define RTC_GET_TIMEDATE_ID 7U

#define RTC_INIT_ID 0U

#define RTC_INTE_PROC_ID 11U

#define RTC_IRQ_HANDER_ID 12U

#define RTC_SET_COMPENSATION_ID 10U

#define RTC_SET_COUNT_ID 4U

#define RTC_SET_TIMEDATE_ID 8U

#define RTC_START_ID 2U

#define RTC_STOP_ID 3U

```
#define RTC_UPDATE_ALARM_ID 5U
```

```
#define SECONDS_DAY 86400UL
```

```
#define SECONDS_HOUR 3600UL
```

```
#define SECONDS_MINUTE 60UL
```

```
#define START_YEAR 1970UL
```

Typedef Documentation

```
typedef struct _RTC_HandleType RTC_HandleType
```

The structure of the RTC processing handle.

Enumeration Type Documentation

```
enum RTC_InstanceType
```

The instance index of the RTC peripheral.

Enumerator:

RTC_INSTANCE _0	
--------------------	--

```
enum RTC_IntEventType
```

Rtc mode.

Enumerator:

RTC_ALARM_IN T	
RTC_SECOND_I NT	
RTC_OVERFLOW _INT	

Function Documentation

uint32_t RTC_CheckOverflowFlag (RTC_HandleType * *pRtcHandle*)

Check RTC overflow flag.

Parameters

<i>pRtcHandle</i>	the RTC instance
-------------------	------------------

Returns

Overflow flag

void Rtc_CommonProcessInterrupt (RTC_HandleType * *pRtcHandle*)

RTC common interrupt function.

Parameters

<i>pRtcHandle</i>	the RTC instance
-------------------	------------------

void RTC_Deinit (RTC_HandleType * *pRtcHandle*)

De-initialize Rtc instance.

Parameters

<i>pRtcHandle</i>	the RTC instance
-------------------	------------------

void RTC_GetCompensation (RTC_HandleType * *pRtcHandle*, RTC_Compensation *const *pCompensation*)

Get time and date compensation information.

Parameters

<i>pRtcHandle</i>	the RTC instance
<i>pCompensation</i>	the structure to compensate time accuracy

void RTC_GetTimeDate (RTC_HandleType * *pRtcHandle*, RTC_TimeDate *const *pTimeDate*)

Get time and date.

Parameters

<i>pRtcHandle</i>	the RTC instance
<i>pTimeDate</i>	the structure of time and date

void RTC_Init (RTC_HandleType * *pRtcHandle*, const RTC_InitType *const *pInitStruct*)

Initialize Rtc instance.

Parameters

<i>pRtcHandle</i>	the RTC instance
<i>pInitStruct</i>	Rtc initialization structure

Note

This function could only write once after POR.

void RTC_SetCompensation (RTC_HandleType * *pRtcHandle*, const RTC_Compensation *const *pCompensation*)

Set time and date compensation value.

Parameters

<i>pRtcHandle</i>	the RTC instance
-------------------	------------------

<i>pCompensation</i>	the structure to compensate time accuracy
----------------------	---

void RTC_SetCountValue (RTC_HandleType * *pRtcHandle*, uint32_t *u32CounetValue*)

set count value

Parameters

<i>pRtcHandle</i>	the RTC instance
<i>u32CounetValue</i>	The count value to set.

void RTC_SetTimeDate (RTC_HandleType * *pRtcHandle*, const RTC_TimeDate *const *pTimeDate*)

Set time and date.

Parameters

<i>pRtcHandle</i>	the RTC instance
<i>pTimeDate</i>	the structure of time and date

void RTC_Start (RTC_HandleType * *pRtcHandle*)

Rtc start.

Parameters

<i>pRtcHandle</i>	the RTC instance
-------------------	------------------

void RTC_Stop (RTC_HandleType * *pRtcHandle*)

Rtc stop.

Parameters

<i>pRtcHandle</i>	the RTC instance
-------------------	------------------

void RTC_UpdateAlarmValue (RTC_HandleType * *pRtcHandle*, const uint32_t *u32AlarmValue*)

Rtc alarm value.

Parameters

<i>pRtcHandle</i>	the RTC instance
<i>u32AlarmValue</i>	Input value

Module_driver_scg

Collaboration diagram for Module_driver_scg:Modules

HwA_scg

Classes

struct **SCG_FoscType**

FOSC initial definition, include register SCG_FOSCCSR/SCG_FOSCDIV/SCG_FOSCCFG.

struct **SCG_SircType**

SIRC initial definition, include register SCG_SIRCCSR/SCG_SIRCDIV/SCG_SIRCTCCFG.

struct **SCG_Sirc32kType**

struct **SCG_FircType**

FIRC initial definition, include register SCG_FIRCCSR/SCG_FIRCDIV/SCG_FIRCFG.

struct **SCG_SoscType**

SOSC definition, include register SCG_SOSCCFG.

struct **SCG_PllType**

PLL definition, include register SCG_PLLCSR/SCG_PLLDIV/SCG_PLL0CFG.

struct **SCG_ClockCtrlType**

Current system clock definition, include register SCG_CCR.

struct **_SCG_HandleType**

The structure of the SCG processing handle.

struct **SCG_InterruptType**

The configuration option for the SCG interrupt.

Macros

```
#define SCG_DEV_ERROR_REPORT STD_OFF
```

```
#define SCG_ReportDevError(func, error) ReportDevError(SCG_MODULE_ID, func, error)
```

```
#define FOSC_FREQUENCY 24000000U
```


Fast OSC frequency.

```
#define SOSC_FREQUENCY 32768U
```

Slow OSC frequency.

```
#define PCC_FTU_TCLK_FREQ 0U
```

FTU input TCLK frequency.

```
#define UNKNOWN_CLOCK 0xFFFFFFFFU
```

```
#define SCG_CORE_CLOCK_SYMBOL CORE
```

```
#define SCG_BUS_CLOCK_SYMBOL BUS
```

```
#define SCG_SLOW_CLOCK_SYMBOL SLOW
```

```
#define SCG_SIRC_CLOCK_SYMBOL SIRC
```

```
#define SCG_SIRC32K_CLOCK_SYMBOL SIRC32K
```

```
#define SCG_FIRC_CLOCK_SYMBOL FIRC
```

```
#define SCG_FOSC_CLOCK_SYMBOL FOSC
```

```
#define SCG_SOSC_CLOCK_SYMBOL SOSC
```

```
#define SCG_PLL0_CLOCK_SYMBOL PLL0
```

```
#define SCG_PLL1_CLOCK_SYMBOL PLL1
```

Typedefs

```
typedef uint16_t SCG_PllMultiplyType
```

SCG_PLLCFG [MULT] for PLL clock calculation. The mult value range is 96 ~ 512.

```
typedef struct _SCG_HandleType SCG_HandleType
```

The structure of the SCG processing handle.

Enumerations

```
enum SCG_ClkSrcType { SCG_CORE_CLK = 0U, SCG_BUS_CLK = 1U, SCG_SLOW_CLK = 2U,  
SCG_FOSC_CLK = 3U, SCG_FIRC_CLK = 4U, SCG_PLL0_CLK = 5U, SCG_PLL1_CLK = 6U, SCG_SIRC_CLK =  
7U, SCG_SIRC32K_CLK = 8U, SCG_SOSC_CLK = 9U, SCG_SCG_CLKOUT_CLK = 10U, SCG_SIRCDIVH_CLK =  
11U, SCG_SIRCDIVM_CLK = 12U, SCG_SIRCDIVL_CLK = 13U, SCG_FIRCDIVH_CLK = 14U,  
SCG_FIRCDIVM_CLK = 15U, SCG_FIRCDIVL_CLK = 16U, SCG_FOSCDIVH_CLK = 17U, SCG_FOSCDIVM_CLK =  
18U, SCG_FOSCDIVL_CLK = 19U, SCG_PLL0DIVH_CLK = 20U, SCG_PLL0DIVM_CLK = 21U,
```

SCG_PLL0DIVL_CLK = 22U, SCG_PLL1DIVH_CLK = 23U, SCG_PLL1DIVM_CLK = 24U, SCG_PLL1DIVL_CLK = 25U, SCG_NVMINIT_CLK = 26U, SCG_CMU4REF_CLK = 27U, SCG_END_OF_CLOCKS = 34U }

clock list for internal clock tree

enum SCG_PllClkType { SCG_PLL0 = 0U, SCG_PLL1 = 1U }

PLL clock type.

enum SCG_StatusType { SCG_STATUS_SUCCESS = 0U, SCG_STATUS_SEQUENCE_ERROR = 1U, SCG_STATUS_TIMEOUT = 2U, SCG_STATUS_IRC_ERROR = 3U, SCG_STATUS_PARAM_ERROR = 4U }

indicate the clock function status

enum SCG_CrcModeType { SCG_CRC_SOFTWARE_MODE = 0U, SCG_CRC_TRIGGER_MODE = 1U }

SCG registers CRC trigger mode.

enum SCG_CrcCheckResType { SCG_CRC_CHECK_SUCCESS = 0U, SCG_CRC_CHECK_FAILED = 1U }

indicate the CRC check result

enum SCG_IrcTrimSrcType { SCG_IRC_TRIMSRC_RESERVE0 = 0U, SCG_IRC_TRIMSRC_RESERVE1 = 1U, SCG_IRC_TRIMSRC_FOSC = 2U, SCG_IRC_TRIMSRC_SOSC = 3U }

SCG xxxIRC TRIMSRC type.

enum SCG_ClockSrcType { SCG_CLOCK_SRC_FOSC = 1U, SCG_CLOCK_SRC_FIRC = 3U, SCG_CLOCK_SRC_PLL0 = 6U, SCG_CLOCK_SRC_NONE = 255U }

SCG_CCR [SCS] to Select system clock source.

enum SCG_ClockDivType { SCG_CLOCK_DIV_BY1 = 0U, SCG_CLOCK_DIV_BY2 = 1U, SCG_CLOCK_DIV_BY3 = 2U, SCG_CLOCK_DIV_BY4 = 3U }

SCG_CCR [DIVCORE] / SCG_CCR [DIVBUS] / SCG_CCR [DIVSLOW] to Select system clock source.

enum SCG_ClockoutSrcType { SCG_CLOCKOUT_SRC_OFF = 0U, SCG_CLOCKOUT_SRC_FOSC = 1U, SCG_CLOCKOUT_SRC_SIRC = 2U, SCG_CLOCKOUT_SRC_FIRC = 3U, SCG_CLOCKOUT_SRC_SOSC = 4U, SCG_CLOCKOUT_SRC_SIRC32K = 7U }

SCG_CLKOUTCFG [CLKOUTSEL] to Select system clock source.

enum SCG_NvmClkSrcType { SCG_NVMCLK_SRC_SIRC = 30u, SCG_NVMCLK_SRC_FIRC = 31u }

NVM clock source enumeration.

enum SCG_Cmu4ClkSrcType { SCG_CMU4CLK_SRC_SIRC = 28u, SCG_CMU4CLK_SRC_FOSC = 29u }

CMU4 clock source enumeration.

enum SCG_AsyncClockDivType { SCG_ASYNC_CLOCK_DISABLE = 0U, SCG_ASYNC_CLOCKDIV_BY1 = 1U, SCG_ASYNC_CLOCKDIV_BY2 = 2U, SCG_ASYNC_CLOCKDIV_BY4 = 3U, SCG_ASYNC_CLOCKDIV_BY8 = 4U, SCG_ASYNC_CLOCKDIV_BY16 = 5U, SCG_ASYNC_CLOCKDIV_BY32 = 6U, SCG_ASYNC_CLOCKDIV_BY64 = 7U }

[DIVL] / [DIVM] / [DIVH] bit field definition for SCG_SIRC / SCG_FIRC / SCG_FOSC / SCG_PLL0 registers

enum SCG_PllPredivType { SCG_PLLPREDIV_BY1 = 0U, SCG_PLLPREDIV_BY2 = 1U, SCG_PLLPREDIV_BY3 = 2U, SCG_PLLPREDIV_BY4 = 3U, SCG_PLLPREDIV_BY5 = 4U, SCG_PLLPREDIV_BY6 = 5U, SCG_PLLPREDIV_BY7 = 6U, SCG_PLLPREDIV_BY8 = 7U, SCG_PLLPREDIV_BY9 = 8U,

SCG_PLLPREDIV_BY10 = 9U, SCG_PLLPREDIV_BY11 = 10U, SCG_PLLPREDIV_BY12 = 11U,
 SCG_PLLPREDIV_BY13 = 12U, SCG_PLLPREDIV_BY14 = 13U, SCG_PLLPREDIV_BY15 = 14U,
 SCG_PLLPREDIV_BY16 = 15U, SCG_PLLPREDIV_BY17 = 16U, SCG_PLLPREDIV_BY18 = 17U,
 SCG_PLLPREDIV_BY19 = 18U, SCG_PLLPREDIV_BY20 = 19U, SCG_PLLPREDIV_BY21 = 20U,
 SCG_PLLPREDIV_BY22 = 21U, SCG_PLLPREDIV_BY23 = 22U, SCG_PLLPREDIV_BY24 = 23U,
 SCG_PLLPREDIV_BY25 = 24U, SCG_PLLPREDIV_BY26 = 25U, SCG_PLLPREDIV_BY27 = 26U,
 SCG_PLLPREDIV_BY28 = 27U, SCG_PLLPREDIV_BY29 = 28U, SCG_PLLPREDIV_BY30 = 29U,
 SCG_PLLPREDIV_BY31 = 30U, SCG_PLLPREDIV_BY32 = 31U }

SCG_PLLCFG[PREDIV] for PLL clock calculation. The pre-div value range is 0 ~ 31.

enum SCG_PllPstdivType { SCG_PLLPSTDIV_BY2 = 1U, SCG_PLLPSTDIV_BY4 = 2U, SCG_PLLPSTDIV_BY8
 = 3U }

SCG_PLLCFG[PSTDIV] for PLL clock calculation.

enum SCG_PllSourceType { SCG_PLLSOURCE_FOSC = 0U, SCG_PLLSOURCE_FIRC = 1U }

SCG_PLLCFG[SOURCE] to Select PLL clock source.

enum SCG_IntrMaskType { SCG_INTR_MASK_FIRC_CLK_ERR = 1U, SCG_INTR_MASK_SIRC_CLK_ERR =
 (1U << 1), SCG_INTR_MASK_FOSC_CLK_ERR = (1U << 2), SCG_INTR_MASK_SOSC_CLK_ERR = (1U << 3),
 SCG_INTR_MASK_PLL0_CLK_ERR = (1U << 4), SCG_INTR_MASK_PLL1_CLK_ERR = (1U << 5) }

The interrupt enable/disable mask of SCG.

Functions

SCG_StatusType SCG_EnableSOSC (const **SCG_SoscType** *const pSoscConfig)

Enable SOSC.

SCG_StatusType SCG_DisableSOSC (void)

Disable SOSC.

SCG_StatusType SCG_GetSOSCValidStatus (void)

Get SOSC Valid status.

SCG_StatusType SCG_EnableFOSC (const **SCG_FoscType** *const pFoscConfig)

Enable FOSC clock with input configuration.

SCG_StatusType SCG_DisableFOSC (void)

Disable FOSC.

SCG_StatusType SCG_SetSIRC (const **SCG_SircType** *const pSircConfig)

Set SIRC configuration and configure SIRC DIV.

SCG_StatusType SCG_ClearSIRC (void)

Disable SIRC DIV and clear DIV configuration.

SCG_StatusType SCG_EnableSIRC32K (const **SCG_Sirc32kType** *const pSirc32kConfig)

Enable SIRC32K.

SCG_StatusType SCG_DisableSIRC32K (void)

Disable SIRC32K.

SCG_StatusType SCG_EnableFIRC (const **SCG_FircType** *const pFircConfig)

Enable FIRC.

SCG_StatusType SCG_DisableFIRC (void)

Disable FIRC.

SCG_StatusType SCG_EnablePLL (const **SCG_PllClkType** ePll, const **SCG_PllType** *const pPllConfig)

Enable PLL.

SCG_StatusType SCG_DisablePLL (const **SCG_PllClkType** ePll)

Disable PLL.

SCG_StatusType SCG_SwitchSystemClock (const **SCG_ClockSrcType** eClock)

Switch system clock source.

SCG_StatusType SCG_SetClkCtrl (const **SCG_ClockCtrlType** *const pSysClkConfig)

Set system run time clock and related CORE/BUS/SLOW clock.

uint32_t SCG_GetScgClockFreq (const **SCG_ClkSrcType** eScgClockName)

Get clock frequency.

void **SCG_SetClkOut** (const **SCG_ClockoutSrcType** eClkoutSel)

Select clock out source.

SCG_StatusType **SCG_SetNvmClk** (const **SCG_NvmClkSrcType** eNvmClkSrc)

Select NVM clock source.

SCG_StatusType **SCG_SetCmu4Clk** (const **SCG_Cmu4ClkSrcType** eCmu4ClkSrc)

Select CMU4 clock source.

SCG_StatusType **SCG_CrcInit** (**SCG_CrcModeType** eMode)

Initializes the CRC module based on the specified mode.

void **SCG_CrcGenerate** (void)

Generates a CRC value using the SCG hardware accelerator.

SCG_StatusType **SCG_Deinit** (void)

Clock source De-init.

void **SCG_InterruptInit** (**SCG_HandleType** *pScgHandle, const **SCG_InterruptType** *const pIntrStruct)

Set clock error interrupt.

void **SCG_UserIRQHandler** (**SCG_HandleType** *pScgHandle)

Clock error interrupt handler.

SCG API Service IDs

#define **SCG_SET_SIRC_ID** 0x00U

#define **SCG_ENABLE_SIRC32K_ID** 0x01U

#define **SCG_ENABLE_FIRC_ID** 0x02U

#define **SCG_ENABLE_FOSC_ID** 0x03U

#define **SCG_ENABLE_SOSC_ID** 0x04U

#define **SCG_ENABLE_PLL_ID** 0x05U

#define **SCG_SET_CLOCK_CTRL_ID** 0x06U

#define **SCG_SET_CLOCK_OUT_ID** 0x07U

#define **SCG_SWITCH_SYS_CLOCK_ID** 0x08U

```
#define SCG_GET_SCG_CLOCK_FREQ_ID 0x09U
#define SCG_SCG_DEINIT_ID 0x0AU
#define SCG_SCG_INTERRUPT_INIT_ID 0x0BU
```

SCG Dev Error Code

Error Code of calling SCG apis

```
#define SCG_E_PARAM_POINTER 0x01U
#define SCG_E_PARAM_OUT_RANGE 0x02U
```

Detailed Description

Macro Definition Documentation

```
#define FOSC_FREQUENCY 24000000U
```

Fast OSC frequency.

```
#define PCC_FTU_TCLK_FREQ 0U
```

FTU input TCLK frequency.

```

#define SCG_BUS_CLOCK_SYMBOL  BUS

#define SCG_CORE_CLOCK_SYMBOL  CORE

#define SCG_DEV_ERROR_REPORT  STD_OFF

#define SCG_E_PARAM_OUT_RANGE  0x02U

#define SCG_E_PARAM_POINTER  0x01U

#define SCG_ENABLE_FIRC_ID  0x02U

#define SCG_ENABLE_FOSC_ID  0x03U

#define SCG_ENABLE_PLL_ID  0x05U

#define SCG_ENABLE_SIRC32K_ID  0x01U

#define SCG_ENABLE_SOSC_ID  0x04U

#define SCG_FIRC_CLOCK_SYMBOL  FIRC

#define SCG_FOSC_CLOCK_SYMBOL  FOSC

#define SCG_GET_SCG_CLOCK_FREQ_ID  0x09U

#define SCG_PLL0_CLOCK_SYMBOL  PLL0

#define SCG_PLL1_CLOCK_SYMBOL  PLL1

#define SCG_ReportDevError( func,  error)  ReportDevError(SCG_MODULE_ID, func, error)

#define SCG_SCG_DEINIT_ID  0x0AU

#define SCG_SCG_INTERRUPT_INIT_ID  0x0BU

#define SCG_SET_CLOCK_CTRL_ID  0x06U

#define SCG_SET_CLOCK_OUT_ID  0x07U

#define SCG_SET_SIRC_ID  0x00U

#define SCG_SIRC32K_CLOCK_SYMBOL  SIRC32K

```

```
#define SCG_SIRC_CLOCK_SYMBOL  SIRC

#define SCG_SLOW_CLOCK_SYMBOL  SLOW

#define SCG_SOSC_CLOCK_SYMBOL  SOSC

#define SCG_SWITCH_SYS_CLOCK_ID  0x08U

#define SOSC_FREQUENCY  32768U
```

Slow OSC frequency.

```
#define UNKNOWN_CLOCK  0xFFFFFFFFU
```

Typedef Documentation

```
typedef struct _SCG_HandleType  SCG_HandleType
```

The structure of the SCG processing handle.

```
typedef uint16_t SCG_PllMultiplyType
```

SCG_PLLCFG [MULT] for PLL clock calculation. The mult value range is 96 ~ 512.

Enumeration Type Documentation

```
enum SCG_AsyncClockDivType
```

[DIVL]/[DIVM]/[DIVH] bit field definition for SCG_SIRC/ SCG_FIRC/ SCG_FOSC/ SCG_PLL0 registers

Enumerator:

SCG_ASYNC_CLOCK_DISABLE	Clock output is disabled.
-------------------------	---------------------------

SCG_ASYNC_CLO CKDIV_BY1	Divided by 1.
SCG_ASYNC_CLO CKDIV_BY2	Divided by 2.
SCG_ASYNC_CLO CKDIV_BY4	Divided by 4.
SCG_ASYNC_CLO CKDIV_BY8	Divided by 8.
SCG_ASYNC_CLO CKDIV_BY16	Divided by 16.
SCG_ASYNC_CLO CKDIV_BY32	Divided by 32.
SCG_ASYNC_CLO CKDIV_BY64	Divided by 64.

enum SCG_ClkSrcType

clock list for internal clock tree

Enumerator:

SCG_CORE_CLK	Core clock
SCG_BUS_CLK	Bus clock

SCG_SLOW_CLK	Slow clock
SCG_FOSC_CLK	FOSC clock
SCG_FIRC_CLK	FIRC clock
SCG_PLL0_CLK	PLL0 clock
SCG_PLL1_CLK	PLL1 clock
SCG_SIRC_CLK	SIRC clock
SCG_SIRC32K_CLK	SIRC32K clock
SCG_SOSC_CLK	SOSC clock
SCG_SCG_CLKOUT_CLK	SCG CLK_OUT clock
SCG_SIRCDIVH_CLK	SIRCDIVH functional clock

SCG_SIRCDIVM_ CLK	SIRCDIVM functional clock
SCG_SIRCDIVL_ CLK	SIRCDIVL functional clock
SCG_FIRCDIVH_ CLK	FIRCDIVH functional clock
SCG_FIRCDIVM_ CLK	FIRCDIVM functional clock
SCG_FIRCDIVL_ CLK	FIRCDIVL functional clock
SCG_FOSCDIVH_ _CLK	FOSCDIVH functional clock
SCG_FOSCDIVM_ _CLK	FOSCDIVM functional clock
SCG_FOSCDIVL_ CLK	FOSCDIVL functional clock
SCG_PLL0DIVH_ CLK	PLL0DIVH functional clock
SCG_PLL0DIVM_ CLK	PLL0DIVM functional clock

SCG_PLL0DIVL_ CLK	PLL0DIVL functional clock
SCG_PLL1DIVH_ CLK	PLL1DIVH functional clock
SCG_PLL1DIVM_ CLK	PLL1DIVM functional clock
SCG_PLL1DIVL_ CLK	PLL1DIVL functional clock
SCG_NVMINIT_C LK	NVM initial clock source
SCG_CMU4REF_ CLK	CMU4 reference clock
SCG_END_OF_C LOCKS	End of SCG clocks

enum SCG_ClockDivType

SCG_CCR [DIVCORE]/ SCG_CCR [DIVBUS]/ SCG_CCR [DIVSLOW] to Select system clock source.

Enumerator:

SCG_CLOCK_DI V_BY1	Divided by 1.
-----------------------	---------------

SCG_CLOCK_DI V_BY2	Divided by 2.
SCG_CLOCK_DI V_BY3	Divided by 3.
SCG_CLOCK_DI V_BY4	Divided by 4.

enum SCG_ClockoutSrcType

SCG_CLKOUTCFG [CLKOUTSEL] to Select system clock source.

Enumerator:

SCG_CLOCKOU T_SRC_OFF	SCG OFF.
SCG_CLOCKOU T_SRC_FOSC	Fast OSC.
SCG_CLOCKOU T_SRC_SIRC	Slow IRC.
SCG_CLOCKOU T_SRC_FIRC	Fast IRC.
SCG_CLOCKOU T_SRC_SOSC	Slow OSC.
SCG_CLOCKOU T_SRC_SIRC32K	SIRC32K_CLK.

enum SCG_ClockSrcType

SCG_CCR [SCS] to Select system clock source.

Enumerator:

SCG_CLOCK_SR C_FOSC	System OSC.
SCG_CLOCK_SR C_FIRC	Fast IRC.
SCG_CLOCK_SR C_PLL0	System PLL.
SCG_CLOCK_SR C_NONE	MAX value.

enum SCG_Cmu4ClkSrcType

CMU4 clock source enumeration.

Enumerator:

SCG_CMU4CLK_ SRC_SIRC	NVM source choose SIRC.
SCG_CMU4CLK_ SRC_FOSC	NVM source choose FORC.

enum SCG_CrcCheckResType

indicate the CRC check result

Enumerator:

SCG_CRC_CHEC K_SUCCESS	CRC check success SCG_CRC_CHECK_SUCCESS
SCG_CRC_CHEC K_FAILED	CRC check failed SCG_CRC_CHECK_FAILED

enum SCG_CrcModeType

SCG registers CRC trigger mode.

Enumerator:

SCG_CRC_SOFT WARE_MODE	SCG registers CRC software mode.
SCG_CRC_TRIG GER_MODE	SCG registers CRC trigger mode.

enum SCG_IntrMaskType

The interrupt enable/disable mask of SCG.

Enumerator:

SCG_INTR_MAS K_FIRC_CLK_ER R	interrupt mask of FIRC clock error
SCG_INTR_MAS K_SIRC_CLK_ER R	interrupt mask of SIRC clock error
SCG_INTR_MAS K_FOSC_CLK_E RR	interrupt mask of FOSC clock error

SCG_INTR_MAS K_SOSC_CLK_ER RR	interrupt mask of SOSC clock error
SCG_INTR_MAS K_PLL0_CLK_ER R	interrupt mask of PLL0 clock error
SCG_INTR_MAS K_PLL1_CLK_ER R	interrupt mask of PLL0 clock error

enum SCG_IrcTrimSrcType

SCG xxxIRC TRIMSRC type.

Enumerator:

SCG_IRC_TRIMS RC_RESERVE0	Clock Trim source reserve0
SCG_IRC_TRIMS RC_RESERVE1	Clock Trim source reserve1
SCG_IRC_TRIMS RC_FOSC	Clock Trim source FOSC
SCG_IRC_TRIMS RC_SOSC	Clock Trim source SOSC

enum SCG_NvmClkSrcType

NVM clock source enumeration.

Enumerator:

SCG_NVMCLK_S RC_SIRC	NVM source choose SIRC.
SCG_NVMCLK_S RC_FIRC	NVM source choose FIRC.

enum SCG_PllClkType

PLL clock type.

Enumerator:

SCG_PLL0	PLL0 clock
SCG_PLL1	PLL1 clock

enum SCG_PllPredivType

SCG_PLLCFG[PREDIV] for PLL clock calculation. The pre-div value range is 0 ~ 31.

Enumerator:

SCG_PLLPREDIV _BY1	Divided by 1.
SCG_PLLPREDIV _BY2	Divided by 2.
SCG_PLLPREDIV _BY3	Divided by 3.
SCG_PLLPREDIV _BY4	Divided by 4.

SCG_PLLPREDIV _BY5	Divided by 5.
SCG_PLLPREDIV _BY6	Divided by 6.
SCG_PLLPREDIV _BY7	Divided by 7.
SCG_PLLPREDIV _BY8	Divided by 8.
SCG_PLLPREDIV _BY9	Divided by 9.
SCG_PLLPREDIV _BY10	Divided by 10.
SCG_PLLPREDIV _BY11	Divided by 11.
SCG_PLLPREDIV _BY12	Divided by 12.
SCG_PLLPREDIV _BY13	Divided by 13.
SCG_PLLPREDIV _BY14	Divided by 14.
SCG_PLLPREDIV _BY15	Divided by 15.
SCG_PLLPREDIV _BY16	Divided by 16.
SCG_PLLPREDIV _BY17	Divided by 17.
SCG_PLLPREDIV _BY18	Divided by 18.
SCG_PLLPREDIV _BY19	Divided by 19.
SCG_PLLPREDIV _BY20	Divided by 20.

SCG_PLLPREDIV _BY21	Divided by 21.
SCG_PLLPREDIV _BY22	Divided by 22.
SCG_PLLPREDIV _BY23	Divided by 23.
SCG_PLLPREDIV _BY24	Divided by 24.
SCG_PLLPREDIV _BY25	Divided by 25.
SCG_PLLPREDIV _BY26	Divided by 26.
SCG_PLLPREDIV _BY27	Divided by 27.
SCG_PLLPREDIV _BY28	Divided by 28.
SCG_PLLPREDIV _BY29	Divided by 29.
SCG_PLLPREDIV _BY30	Divided by 30.
SCG_PLLPREDIV _BY31	Divided by 31.
SCG_PLLPREDIV _BY32	Divided by 32.

enum SCG_PllPstdivType

SCG_PLLCFG[PSTDIV] for PLL clock calculation.

Enumerator:

SCG_PLLPSTDIV _BY2	
-----------------------	--

SCG_PLLPSTDIV _BY4	
SCG_PLLPSTDIV _BY8	

enum SCG_PllSourceType

SCG_PLLCFG[SOURCE] to Select PLL clock source.

Enumerator:

SCG_PLLSOURC E_FOSC	
SCG_PLLSOURC E_FIRC	

enum SCG_StatusType

indicate the clock function status

Enumerator:

SCG_STATUS_S UCCESS	function called success
SCG_STATUS_S EQUENCE_ERR OR	function called report sequence error
SCG_STATUS_TI MEOUT	function called report timeout error
SCG_STATUS_IR C_ERROR	function called report internal clock error
SCG_STATUS_P ARAM_ERROR	function called report internal clock error

Function Documentation

SCG_StatusType SCG_ClearSIRC (void)

Disable SIRC DIV and clear DIV configuration.

Returns

SCG_StatusType function status

void SCG_CrcGenerate (void)

Generates a CRC value using the SCG hardware accelerator.

This function invokes the hardware accelerator to generate a CRC value. It calls the **SCG_HWA_GenCrcVal()** function, which is responsible for the actual CRC computation.

Note

This function assumes that the hardware accelerator is properly initialized and configured before being called.

Returns

void

SCG_StatusType SCG_CrcInit (SCG_CrcModeType *eMode*)

Initializes the CRC module based on the specified mode.

This function initializes the CRC module by first disabling and clearing any existing CRC-related configurations. It then generates the original CRC value and waits for the CRC calculation to complete. If the operation times out, the function returns a timeout status. If the initialization is successful, it enables the CRC trigger mode (if specified) and enables CRC error output and CRC check.

Parameters

<i>eMode</i>	
The CRC mode to be initialized.	
Possible values are:	

SCG_CRC_TRIGGER_MODE: Enables CRC trigger mode.



Returns

SCG_StatusType Returns the status of the CRC initialization:

- SCG_STATUS_SUCCESS: CRC initialization was successful.
- SCG_STATUS_TIMEOUT: CRC initialization timed out.

SCG_StatusType SCG_Deinit (void)

Clock source De-init.

Returns

SCG_StatusType function status

SCG_StatusType SCG_DisableFIRC (void)

Disable FIRC.

Returns

SCG_StatusType function status

SCG_StatusType SCG_DisableFOSC (void)

Disable FOOSC.

Returns

SCG_StatusType function status

SCG_StatusType SCG_DisablePLL (const SCG_PllClkType *ePll*)

Disable PLL.

Parameters

<i>ePlI</i>	PLL instance
-------------	--------------

Returns

SCG_StatusType function status

SCG_StatusType SCG_DisableSIRC32K (void)

Disable SIRC32K.

Returns

SCG_StatusType function status

SCG_StatusType SCG_DisableSOSC (void)

Disable SOSC.

Returns

SCG_StatusType Funtion status

SCG_StatusType SCG_EnableFIRC (const SCG_FircType *const *pFircConfig*)

Enable FIRC.

Parameters

<i>pFircConfig</i>	FIRC configuration
--------------------	--------------------

Returns

SCG_StatusType function status

SCG_StatusType SCG_EnableFOSC (const SCG_FoscType *const *pFoscConfig*)

Enable FOSC clock with input configuration.

Parameters

<i>pFoscConfig</i>	FOSC configuration
--------------------	--------------------

Returns

SCG_StatusType function status

SCG_StatusType SCG_EnablePLL (const SCG_PllClkType *ePll*, const SCG_PllType *const *pPllConfig*)

Enable PLL.

Parameters

<i>ePll</i>	PLL instance
<i>pPllConfig</i>	PLL configuration

Returns

SCG_StatusType function status

SCG_StatusType SCG_EnableSIRC32K (const SCG_Sirc32kType *const *pSirc32kConfig*)

Enable SIRC32K.

Parameters

<i>pSirc32kConfig</i>	SIRC32K configuration
-----------------------	-----------------------

Returns

SCG_StatusType function status

SCG_StatusType SCG_EnableSOSC (const SCG_SoscType *const *pSoscConfig*)

Enable SOSC.

Parameters

<i>pSoscConfig</i>	SOSC configuration
--------------------	--------------------

Returns

SCG_StatusType Function status

uint32_t SCG_GetScgClockFreq (const SCG_ClkSrcType *eScgClockName*)

Get clock frequency.

Parameters

<i>eScgClockName</i>	Clock source type
----------------------	-------------------

Returns

uint32_t frequency value

SCG_StatusType SCG_GetSOSCValidStatus (void)

Get SOSC Valid status.

Returns

SCG_StatusType Function status

void SCG_InterruptInit (SCG_HandleType * *pScgHandle*, const SCG_InterruptType *const *pIntrStruct*)

Set clock error interrupt.

Parameters

<i>pScgHandle</i>	Scg handle
<i>pIntrStruct</i>	pointer to interrupt configuration,

SCG_StatusType SCG_SetClkCtrl (const SCG_ClockCtrlType *const *pSysClkConfig*)

Set system run time clock and related CORE/BUS/SLOW clock.

Parameters

<i>pSysClkConfig</i>	pointer to the clockCtrlType structure data instance,which defined for system clock selection.
----------------------	--

Returns

SCG_StatusType function status

void SCG_SetClkOut (const SCG_ClockoutSrcType *eClkoutSel*)

Select clock out source.

Parameters

<i>eClkoutSel</i>	clock out source
-------------------	------------------

SCG_StatusType SCG_SetCmu4Clk (const SCG_Cmu4ClkSrcType *eCmu4ClkSrc*)

Select CMU4 clock source.

Parameters

<i>eCmu4ClkSrc</i>	CMU4 clock source
--------------------	-------------------

Returns

SCG_StatusType function status

SCG_StatusType SCG_SetNvmClk (const SCG_NvmClkSrcType *eNvmClkSrc*)

Select NVM clock source.

Parameters

<i>eNvmClkSrc</i>	NVM clock source
-------------------	------------------

Returns

uint32_t function status

SCG_StatusType SCG_SetSIRC (const SCG_SircType *const *pSircConfig*)

Set SIRC configuration and configure SIRC DIV.

Parameters

<i>pSircConfig</i>	SIRC configuration
--------------------	--------------------

Returns

SCG_StatusType function status

SCG_StatusType SCG_SwitchSystemClock (const SCG_ClockSrcType eClock)

Switch system clock source.

Parameters

eClock	selected clock source
--------	-----------------------

Returns

SCG_StatusType function status

void SCG_UserIRQHandler (SCG_HandleType * pScgHandle)

Clock error interrupt handler.

Parameters

pScgHandle	Scg handle
------------	------------

Module_driver_scm

Collaboration diagram for Module_driver_scm:Modules

HwA_scm

Enumerations

- enum **SCM_RetStatusType** { **SCM_E_OK** = 0U, **SCM_E_NOT_OK** }

SCM return status.
- enum **SCM_CrcModeType** { **SCM_CRC_SOFTWARE_MODE** = 0U, **SCM_CRC_TRIGGER_MODE** = 1U }

SCM CRC trigger mode.

Functions

- void **SCM_GetData_UID** (uint32 *pUid)

Get unique identification for the chip, loaded from NVR.
- void **SCM_PeformReset_SYSAP** (void)

Generate SYS-AP system reset.

SCM_RetStatusType SCM_CrcInit (SCM_CrcModeType eMode)

Initializes the CRC module based on the specified mode.

void SCM_CrcGenerate (void)

Generates the CRC (Cyclic Redundancy Check) by enabling the CRC generation in the hardware.

Detailed Description

Enumeration Type Documentation

enum SCM_CrcModeType

SCM CRC trigger mode.

Enumerator:

SCM_CRC_SOFT WARE_MODE	SCM CRC software mode.
SCM_CRC_TRIG GER_MODE	SCM CRC trigger mode.

enum SCM_RetStatusType

SCM return status.

Enumerator:

SCM_E_OK	Return ok
----------	-----------

SCM_E_NOT_OK	Return not ok
--------------	---------------

Function Documentation

void SCM_CrcGenerate (void)

Generates the CRC (Cyclic Redundancy Check) by enabling the CRC generation in the hardware.

This function sets the CRC generation enable bit in the hardware, which initiates the CRC generation process. The CRC is typically used for error detection in data transmission or storage.

Note

This function does not return any value and does not take any parameters.

See also

SCM_HWA_SetCrcGenerateEnable

SCM_RetStatusType SCM_CrcInit (SCM_CrcModeType *eMode*)

Initializes the CRC module based on the specified mode.

This function initializes the CRC module by first deinitializing the CRC registers, generating the original CRC result, and then configuring the CRC module according to the specified mode. If the CRC generation completes successfully, the function enables the CRC trigger, error output, and check features based on the mode.

Parameters

<i>eMode</i>	The CRC mode to be configured. It can be either SCM_CRC_TRIGGER_MODE or another mode depending on the implementation.
--------------	---

Returns

SCM_RetStatusType Returns SCM_E_OK if the CRC initialization is successful. Returns SCM_E_NOT_OK if a timeout occurs during CRC generation.

void SCM_GetData_UID (uint32 * *pUid*)

Get unique identification for the chip, loaded from NVR.

Parameters

<i>pUid</i>	Pointer to UID
-------------	----------------

void SCM_PeformReset_SYSAP (void)

Generate SYS-AP system reset.

Returns

void

Module_driver_scst

Enumerations

```
enum Type_M7ST_AtomicStatus { M7ST_FaultInjectError, M7ST_ErrorAluTest, M7ST_ErrorAluMLATest,
M7ST_ErrorAluSHIFTTTest, M7ST_ErrorAluTes1t, M7ST_ErrorAluTest2, M7ST_ErrorAluTest3,
M7ST_ErrorAluTest4, M7ST_ErrorAluTest5, M7ST_ErrorAluTest6, M7ST_ErrorRegbankTest1,
M7ST_ErrorRegbankTest2, M7ST_ErrorRegbankTest3, M7ST_ErrorRegbankTest4,
M7ST_ErrorRegbankTest5, M7ST_ErrorRegbankTest6, M7ST_ErrorLoadStoreTest1,
M7ST_ErrorLoadStoreTest2, M7ST_ErrorLoadStoreTest3, M7ST_ErrorLoadStoreTest4,
M7ST_ErrorLoadStoreTest5, M7ST_ErrorLoadStoreTest6, M7ST_ErrorSimdSatTest1,
M7ST_ErrorSimdSatTest2, M7ST_ErrorSimdSatTest3, M7ST_ErrorSimdSatTest4, M7ST_ErrorMacTest1,
M7ST_ErrorMacTest2, M7ST_ErrorFetchTest, M7ST_ErrorStatusTest1, M7ST_ErrorStatusTest2,
M7ST_ErrorBranchTest1, M7ST_ErrorBranchTest2, M7ST_ErrorIntSvcTest,
M7ST_ErrorIntBusFaultTest, M7ST_ErrorIntHardFaultTest1, M7ST_ErrorIntHardFaultTest2,
M7ST_ErrorIntUsageFaultTest, M7ST_ErrorIntSystickTest, M7ST_ErrorIntPendSvTest,
M7ST_ErrorIntMemFaultTest, M7ST_ErrorIntMaskingTest, M7ST_ErrorIntHandlerThreadsTest,
M7ST_ErrorIntNMIHfTest, M7ST_ErrorIntTailChainTest, M7ST_ErrorIntAluTest,
M7ST_ErrorIntBranchTest, M7ST_ErrorIntStatusTest, M7ST_ErrorM7ST_TestPass }
```

typed of test result returned

```
enum SCST_TestIndexType { M7ST_AluParamTest, M7ST_AluParamMLATest, M7ST_AluParamSHIFTTTest, M7ST_AluParamTes1t,
M7ST_AluParamTest2, M7ST_AluParamTest3, M7ST_AluParamTest4, M7ST_AluParamTest5, M7ST_AluParamTest6,
M7ST_RegbankTest1, M7ST_RegbankTest2, M7ST_RegbankTest3, M7ST_RegbankTest4,
M7ST_RegbankTest5, M7ST_RegbankTest6, M7ST_LoadStoreTest1, M7ST_LoadStoreTest2,
```

M7ST_LoadStoreTest3, M7ST_LoadStoreTest4, M7ST_LoadStoreTest5, M7ST_LoadStoreTest6,
M7ST_SimdSatTest1, M7ST_SimdSatTest2, M7ST_SimdSatTest3, M7ST_SimdSatTest4,
M7ST_MacTest1, M7ST_MacTest2, M7ST_FetchTest, M7ST_StatusTest1, M7ST_StatusTest2,
M7ST_BranchTest1, M7ST_BranchTest2, M7ST_IntSvcTest, M7ST_IntBusFaultTest,
M7ST_IntHardFaultTest1, M7ST_IntHardFaultTest2, M7ST_IntUsageFaultTest, M7ST_IntSystickTest,
M7ST_IntPendSvTest, M7ST_IntMemFaultTest, M7ST_IntMaskingTest, M7ST_IntHandlerThreadsTest,
M7ST_IntNMIHfTest, M7ST_IntTailChainTest, M7ST_IntAluTest, M7ST_IntBranchTest,
M7ST_IntStatusTest, M7ST_RegressionTest }

typed of test index

Functions

Type_M7ST_AtomicStatus SCST_ExecuteTest (SCST_TestIndexType test_index, uint32_t *s_u32RamBase)

This function is used to get the result of the executed test.

Detailed Description

Enumeration Type Documentation

enum SCST_TestIndexType

typed of test index

Enumerator:

M7ST_AluTest	
M7ST_AluMLATest	
M7ST_AluSHIFTTest	
M7ST_AluTes1t	
M7ST_AluTest2	

M7ST_AluTest3	
M7ST_AluTest4	
M7ST_AluTest5	
M7ST_AluTest6	
M7ST_Regbank Test1	
M7ST_Regbank Test2	
M7ST_Regbank Test3	
M7ST_Regbank Test4	
M7ST_Regbank Test5	
M7ST_Regbank Test6	
M7ST_LoadStor eTest1	
M7ST_LoadStor eTest2	
M7ST_LoadStor eTest3	
M7ST_LoadStor eTest4	
M7ST_LoadStor eTest5	
M7ST_LoadStor eTest6	
M7ST_SimdSatT est1	
M7ST_SimdSatT est2	
M7ST_SimdSatT est3	
M7ST_SimdSatT est4	
M7ST_MacTest1	
M7ST_MacTest2	

M7ST_FetchTest	
M7ST_StatusTest1	
M7ST_StatusTest2	
M7ST_BranchTest1	
M7ST_BranchTest2	
M7ST_IntSvcTest	
M7ST_IntBusFaultTest	
M7ST_IntHardFaultTest1	
M7ST_IntHardFaultTest2	
M7ST_IntUsageFaultTest	
M7ST_IntSysTickTest	
M7ST_IntPendSVTest	
M7ST_IntMemFaultTest	
M7ST_IntMaskingTest	
M7ST_IntHandlerThreadsTest	
M7ST_IntNMIFTest	
M7ST_IntTailChainTest	
M7ST_IntAluTest	
M7ST_IntBranchTest	

M7ST_IntStatus Test	
M7ST_Regressio nTest	

enum Type_M7ST_AtomicStatus

typed of test result returned

Enumerator:

M7ST_FaultInje ctError	
M7ST_ErrorAluT est	
M7ST_ErrorAlu MLATest	
M7ST_ErrorAluS HIFTTest	
M7ST_ErrorAluT es1t	
M7ST_ErrorAluT est2	
M7ST_ErrorAluT est3	
M7ST_ErrorAluT est4	
M7ST_ErrorAluT est5	
M7ST_ErrorAluT est6	
M7ST_ErrorReg bankTest1	
M7ST_ErrorReg bankTest2	
M7ST_ErrorReg bankTest3	

M7ST_ErrorReg bankTest4	
M7ST_ErrorReg bankTest5	
M7ST_ErrorReg bankTest6	
M7ST_ErrorLoa dStoreTest1	
M7ST_ErrorLoa dStoreTest2	
M7ST_ErrorLoa dStoreTest3	
M7ST_ErrorLoa dStoreTest4	
M7ST_ErrorLoa dStoreTest5	
M7ST_ErrorLoa dStoreTest6	
M7ST_ErrorSim dSatTest1	
M7ST_ErrorSim dSatTest2	
M7ST_ErrorSim dSatTest3	
M7ST_ErrorSim dSatTest4	
M7ST_ErrorMac Test1	
M7ST_ErrorMac Test2	
M7ST_ErrorFetc hTest	
M7ST_ErrorStat usTest1	
M7ST_ErrorStat usTest2	
M7ST_ErrorBran chTest1	

M7ST_ErrorBranchTest2	
M7ST_ErrorIntSvcTest	
M7ST_ErrorIntBusFaultTest	
M7ST_ErrorIntHardFaultTest1	
M7ST_ErrorIntHardFaultTest2	
M7ST_ErrorIntUsageFaultTest	
M7ST_ErrorIntSystickTest	
M7ST_ErrorIntPendSvTest	
M7ST_ErrorIntMemFaultTest	
M7ST_ErrorIntMaskingTest	
M7ST_ErrorIntHandlerThreadsTest	
M7ST_ErrorIntNMIHFTest	
M7ST_ErrorIntTailChainTest	
M7ST_ErrorIntAliuTest	
M7ST_ErrorIntBranchTest	
M7ST_ErrorIntStatusTest	
M7ST_ErrorM7ST_TestPass	

Function Documentation

Type_M7ST_AtomicStatus SCST_ExecuteTest (SCST_TestIndexType *test_index*, uint32_t *
s_u32RamBase)

This function is used to get the result of the executed test.

Parameters

<i>test_index</i>	is test number,0U..47U
<i>s_u32RamBase</i>	The first address of the 1k memory that the program needs to run

Returns

M7ST_ErrorM7ST_TestPass is ok, others are not ok

Module_driver_sdadc

Collaboration diagram for Module_driver_sdadc:Modules

HwA_sdadc

Classes

struct _SDADC_HandleType
struct SDADC_PWMInitType
struct SDADC_IntInitType
struct SDADC_ModulatorCfgType
struct SDADC_DetectCfgType
struct SDADC_RectCfgType
struct SDADC_IntegrationCfgType
struct SDADC_FilterChainCfgType
struct SDADC_ChannelInitType
SDADC Channel Initialization Structure.
struct SDADC_InitType
SDADC Initialization Structure.

Macros

#define SDADC_DEV_ERROR_REPORT STD_ON

```

#define SDADC_ReportDevError(func, error) ReportDevError(SDADC_MODULE_ID, func, error)
#define SDADC_CHANNEL_CNT 3U
#define SDADC_MFSHIFT_LEFT 0x10U
#define SDADC_MFSHIFT_RIGHT 0x0U
#define SDADC_MFSHIFT_DATALENGTH 24U
#define SDADC_E_OK (0U)
#define SDADC_E_NOT_OK (0xFFU)
#define SDADC_INIT_ID 0x00U
#define SDADC_DEINIT_ID 0x01U
#define SDADC_MAIN_ENABLE_ID 0x02U
#define SDADC_CHANNELSWTRIGGER_ID 0x03U
#define SDADC_CHANNELENABLE_ID 0x04U
#define SDADC_CLEARCHANNELSWTRIGGER_ID 0x05U
#define SDADC_INTEGRATORSWTRIGGER_ID 0x06U
#define SDADC_ENABLETSIGNTRIGGEROUT_ID 0x07U
#define SDADC_GETCONVERSIONRESULT_ID 0x08U
#define SDADC_GETCAPTURETIMESTAMP_ID 0x09U
#define SDADC_GETSIGNDELAYCAPTURE_ID 0x0AU
#define SDADC_GETCHANNELREADY_ID 0x0BU
#define SDADC_GETCHANNELCONVERTING_ID 0x0CU
#define SDADC_GETCHANNELCONVERSIONCOMPLETE_ID 0x0DU
#define SDADC_GETCHANNELFIFOEMPTY_ID 0x0EU
#define SDADC_GETCHANNELFIFOREADY_ID 0x0FU
#define SDADC_CLEARCHANNELFIFOREADY_ID 0x10U
#define SDADC_CLEARCHANNELCONVERSIONCOMPLETE_ID 0x11U
#define SDADC_STARTPWMGENERATION_ID 0x12U
#define SDADC_STOPPWMGENERATION_ID 0x13U
#define SDADC_ENABLEPWMSIGNTRIGGEROUT_ID 0x14U
#define SDADC_SETINTERRUPPTENABLE_ID 0x15U
#define SDADC_E_PARAM_POINTER 0x00U
#define SDADC_E_PARAM_INSTANCE 0x01U
#define SDADC_E_PARAM_CHANNEL 0x02U
#define SDADC_INSTANCE_0 0U,
#define SDADC_INSTANCE_1 1U

```

Typedefs

```

typedef uint8_t SDADC_RetStatusType
typedef uint8_t SDADC_ChannelMaskType

```

```
typedef uint8_t SDADC_InstanceType
typedef struct _SDADC_HandleType SDADC_HandleType
```

Enumerations

```
enum SDADC_TRGSignOENType { SDADC_TRGSIGN_OEN_DISABLED = 0x0,
SDADC_TRGSIGN_OEN_ENABLED = 0x1 }

enum SDADC_PWM_CLKSELType { SDADC_PWM_CLKSEL_DIV_CLK0 = 0x0,
SDADC_PWM_CLKSEL_DIV_CLK1 = 0x1, SDADC_PWM_CLKSEL_DIV_CLK2 = 0x2,
SDADC_PWM_CLKSEL_OFF = 0x3 }

enum SDADC_BitRevType { SDADC_BIT_REV_NORMAL = 0x0, SDADC_BIT_REV_REVERSE = 0x1 }

enum SDADC_PWMPolarityType { SDADC_PWM_POLARITY_POSITIVE = 0x0,
SDADC_PWM_POLARITY_NEGATIVE = 0x1 }

enum SDADC_PWMPatternType { SDADC_PWM_PATTERN_STOPPED = 0x0,
SDADC_PWM_PATTERN_SQUARE = 0x1, SDADC_PWM_PATTERN_TRIANGLE = 0x2,
SDADC_PWM_PATTERN_SINE = 0x3 }

enum SDADC_PrescaleType { SDADC_PRESCALE_NO_PRESCALE = 0x0, SDADC_PRESCALE_DIV_BY_2 =
0x1, SDADC_PRESCALE_DIV_BY_4 = 0x2, SDADC_PRESCALE_DIV_BY_8 = 0x3 }

PRE_DIV Divider Value.

enum SDADC_MENSELType { SDADC_MENSEL_INTERNAL = 0x0, SDADC_MENSEL_EXTERNAL = 0x1 }

Main Enable Selection.

enum SDADC_FIFOIsrStatusType { SDADC_FIFO_READY = 0x0, SDADC_FIFO_OVERFLOW = 0x1,
SDADC_FIFO_UNDERFLOW = 0x2 }

enum SDADC_DetectIsrStatusType { SDADC_DET_ZERO_CROSS = 0x0, SDADC_DET_HIGH_LMT_REACH =
0x1, SDADC_DET_LOW_LMT_REACH = 0x2, SDADC_DET_WIN_LMT_REACH = 0x3,
SDADC_DET_SHORT_CIRCUIT = 0x4, SDADC_DET_DATA SATURATION = 0x5, SDADC_DET_CLK_ABSENCE
= 0x6 }

enum SDADC_CICSelectionType { SDADC_SEL_AUXILIARY = 0x0, SDADC_SEL_MAIN = 0x1 }

enum SDADC_AFCMType { SDADC_AFCM_ALWAYS_ON = 0x0, SDADC_AFCM_FOLLOW_MAIN = 0x1 }

Auxiliary Filter Conversion Mode.

enum SDADC_AFORDType { SDADC_AFORD_RESERVED_00 = 0x0, SDADC_AFORD_FIRST_ORDER = 0x1,
SDADC_AFORD_SECOND_ORDER = 0x2, SDADC_AFORD_THIRD_ORDER = 0x3 }

Auxiliary Filter Order.

enum SDADC_MFCMType { SDADC_MFCM_SINGLE_MODE = 0x0, SDADC_MFCM_CONTINUOUS_MODE,
SDADC_MFCM_ALWAYS_MODE = 0x2, SDADC_MFCM_RESERVED_11 = 0x3 }

Main Filter Conversion Mode.

enum SDADC_MFORDType { SDADC_MFORD_RESERVED_000 = 0x0, SDADC_MFORD_FIRST_ORDER = 0x1,
SDADC_MFORD_SECOND_ORDER = 0x2, SDADC_MFORD_THIRD_ORDER = 0x3,
```

SDADC_MFORD_FOURTH_ORDER = 0x4, **SDADC_MFORD_FIFTH_ORDER** = 0x5,
SDADC_MFORD_RESERVED_110_111 = 0x6 }

Main Filter Order.

enum **SDADC_LMTAVTType** { **SDADC_LMTAVT_SET_ON_LARGER_CLEAR_ON_SMALLER**,
SDADC_LMTAVT_CLEAR_ON_LARGER_SET_ON_SMALLER }

Limitation Result Activation Select.

enum **SDADC_LMTOPTType** { **SDADC_LMTOPT_DETECT_BOTH_HIGH_AND_LOW** = 0x0,
SDADC_LMTOPT_DETECT_HIGH_THRESHOLD = 0x1, **SDADC_LMTOPT_DETECT_LOW_THRESHOLD** = 0x2,
SDADC_LMTOPT_DETECT_WINDOWED_VALUE = 0x3 }

Limitation Option.

enum **SDADC_SCDOPTType** { **SDADC_SCDOPT_COUNT_ON_BOTH_0_AND_1** = 0x0,
SDADC_SCDOPT_COUNT_ON_1 = 0x1, **SDADC_SCDOPT_COUNT_ON_0** = 0x2,
SDADC_SCDOPT_RESERVED = 0x3 }

Short Circuit Option.

enum **SDADC_SCDCMType** { **SDADC_SCDCM_ALWAYS_ON** = 0x0, **SDADC_SCDCM_FILTER_IN_PROGRESS**
= 0x1 }

Short Circuit Detect Conversion Mode.

enum **SDADC_SYNC_TRGENType** { **SDADC_SYNC_TRGEN_SELF** = 0x0, **SDADC_SYNC_TRGEN_DEFINED** =
0x1 }

Synced Trigger Select Enable.

enum **SDADC_TRGLVLType** { **SDADC_TRGLVL_EDGE** = 0x0, **SDADC_TRGLVL_LEVEL** = 0x1 }

Trigger Level type.

enum **SDADC_HTRPOLType** { **SDADC_HTRPOL_HIGH_RISING** = 0x0, **SDADC_HTRPOL_LOW_FALLING** =
0x1 }

Hardware Trigger Polarity.

enum **SDADC_TRGSELType** { **SDADC_TRGSEL_SOFTWARE** = 0x0, **SDADC_TRGSEL_HARDWARE** = 0x1 }

Trigger Select.

enum **SDADC_CLKEDGType** { **SDADC_CLKEDG_RESERVED** = 0x0, **SDADC_CLKEDG_POSEDGE** = 0x1,
SDADC_CLKEDG_NEGEDGE = 0x2, **SDADC_CLKEDG_BOTH_EDGE** = 0x3 }

Input Clock Edge Select.

enum **SDADC_CLKSELType** { **SDADC_CLKSEL_RESERVED** = 0x0, **SDADC_CLKSEL_DIV_CLK1** = 0x1,
SDADC_CLKSEL_DIV_CLK2 = 0x2, **SDADC_CLKSEL_EXTERNAL** = 0x3 }

Input Clock Select.

enum **SDADC_IBFMTType** { **SDADC_IBFMT_ONE_BIT_STREAM** = 0x0, **SDADC_IBFMT_MANCHESTER_CODE**
= 0x1 }

Input Bit Format.

enum **SDADC_RDFMTType** { **SDADC_RDFMT_LEFT_JUSTIFIED_SIGNED** = 0x0,
SDADC_RDFMT_LEFT_JUSTIFIED_UNSIGNED = 0x1 }

Result Data Format.

enum **SDADC_CTRGSIGN_OENType** { **SDADC_CTRGSIGN_OEN_DISABLED** = 0x0,
SDADC_CTRGSIGN_OEN_ENABLED = 0x1 }

Channel Sign Signal Related Trigger Out Enable.

enum **SDADC_SSCHType** { **SDADC_SSCH_CH0** = 0x0, **SDADC_SSCH_CH1** = 0x1, **SDADC_SSCH_CH2** = 0x2 }

Sign Signal From Channel Selection.

enum **SDADC_SSRCType** { **SDADC_SSRC_INTERNAL_PWM** = 0x0, **SDADC_SSRC_CHANNEL_SELECTED** =
0x1, **SDADC_SSRC_EXTERNAL_SIGNAL_0** = 0x2, **SDADC_SSRC_EXTERNAL_SIGNAL_1** = 0x3 }

Sign Source Selection.

enum **SDADC_EXTSTPType** { **SDADC_EXTSTP_INTERNAL** = 0x0, **SDADC_EXTSTP_EXTERNAL** = 0x1 }

Integration Stop Control.

enum **SDADC_IOGTType** { **SDADC_IOGT_ALWAYS_WORK** = 0x0,
SDADC_IOGT_ONLY_WHEN_INTEGRATION_ONGOING = 0x1 }

Integration Output Gating.

enum **SDADC_MASELType** { **SDADC_MASEL_EXTERNAL** = 0x0, **SDADC_MASEL_INTERNAL** = 0x1 }

Modulator Analog Selection.

enum **SDADC_MACLKSELType** { **SDADC_MACLKSEL_DIV_CLK1** = 0x0, **SDADC_MACLKSEL_DIV_CLK0** = 0x1,
SDADC_MACLKSEL_RESERVED = 0x2 }

Internal Modulator Analog Clock Selection.

enum **SDADC_DITHER_ENType** { **SDADC_DITHER_EN_DISABLED** = 0x0, **SDADC_DITHER_EN_ENABLED** =
0x1 }

Dither Enable.

enum **SDADC_INCFGPV11Type** { **SDADC_INCFGPV11_DISABLED** = 0x0, **SDADC_INCFGPV11_ENABLED** =
0x1 }

Configuration of Positive Input Line with V11 from System.

enum **SDADC_GAINSELType** { **SDADC_GAINSEL_1** = 0x0, **SDADC_GAINSEL_2** = 0x1, **SDADC_GAINSEL_4** =
0x2, **SDADC_GAINSEL_8** = 0x3 }

Gain Select of Analog Input Path.

enum **SDADC_CMENPType** { **SDADC_CMENP_NOT_CONNECTED** = 0x0, **SDADC_CMENP_CONNECTED** =
0x1 }

Common Voltage Configuration of Positive Inputs.

enum **SDADC_CMENMType** { **SDADC_CMENM_NOT_CONNECTED** = 0x0, **SDADC_CMENM_CONNECTED** =
0x1 }

Common Voltage Configuration of Negative Inputs.

enum **SDADC_INCFGPTType** { **SDADC_INCFGP_FROM_PIN** = 0x0, **SDADC_INCFGP_FROM_VREFL** = 0x1, **SDADC_INCFGP_FROM_VREFH** = 0x2, **SDADC_INCFGP_FROM_VREFH_DIV_2** = 0x3 }

Configuration of Positive Input Line.

enum **SDADC_INCFGMTType** { **SDADC_INCFGM_FROM_PIN** = 0x0, **SDADC_INCFGM_FROM_VREFL** = 0x1, **SDADC_INCFGM_FROM_VREFH** = 0x2, **SDADC_INCFGM_FROM_VREFH_DIV_2** = 0x3 }

Configuration of Negative Input Line.

enum **SDADC_INSELType** { **SDADC_INSEL_PIN_0** = 0x0, **SDADC_INSEL_PIN_1** = 0x1, **SDADC_INSEL_PIN_2** = 0x2, **SDADC_INSEL_PIN_3** = 0x3 }

Input Pin Selection.

enum **SDADC_INMODType** { **SDADC_INMOD_SOFTWARE_CONTROL** = 0x0, **SDADC_INMOD_PRESET_MODE** = 0x1, **SDADC_INMOD_SCAN_MODE** = 0x2, **SDADC_INMOD_RESERVED** = 0x3 }

Input MUX Control Mode.

enum **SDADC_TSRDMType** { **SDADC_TSRDM_DEFAULT_MODE** = 0x0, **SDADC_TSRDM_DIRECT_MODE** = 0x1, **SDADC_TSRDM_FIFO_GATING_MODE** = 0x2, **SDADC_TSRDM_RESERVED** = 0x3 }

Timestamp Information Read Mode.

enum **SDADC_TSTRGMTType** { **SDADC_TSTRGM_IGNORED** = 0x0, **SDADC_TSTRGM_RISING_EDGE** = 0x1, **SDADC_TSTRGM_FALLING_EDGE** = 0x2 }

Timestamp Counter Trigger Mode.

enum **SDADC_INMXCPTType** { **SDADC_INMXCP_DISABLED** = 0x0, **SDADC_INMXCP_ENABLED** = 0x1 }

Input Mux Setting Copy Enable.

enum **SDADC_TS_ENType** { **SDADC_TS_EN_DISABLED** = 0x0, **SDADC_TS_EN_ENABLED** = 0x1 }

Timestamp Counter Enable.

enum **SDADC_TSCLKSELType** { **SDADC_TSCLKSEL_MODULATOR_CLOCK** = 0x0, **SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_2** = 0x1, **SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_4** = 0x2, **SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_8** = 0x3 }

Timestamp Counter Clock Selection.

enum **SDADC_GainCompType** { **SDADC_GAIN_CMP_BYPASS** = 0x0, **SDADC_GAIN_CMP_INTERNAL_GAIN1** = 0x1, **SDADC_GAIN_CMP_INTERNAL_GAIN2** = 0x2, **SDADC_GAIN_CMP_EXTERNAL_GAIN** = 0x3 }

enum **SDADC_InterruptType** { **SDADC_COCI_INTERRUPT** = 0, **SDADC_FIFO_READY_INTERRUPT** = 1, **SDADC_FIFO_OVERFLOW_INTERRUPT** = 2, **SDADC_FIFO_UNDERFLOW_INTERRUPT** = 3, **SDADC_ZERO_CROSS_DETECT_INTERRUPT** = 4, **SDADC_HIGH_LIMIT_REACH_INTERRUPT** = 5, **SDADC_LOW_LIMIT_REACH_INTERRUPT** = 6, **SDADC_WINDOW_LIMIT_REACH_INTERRUPT** = 7, **SDADC_SHORT_CIRCUIT_DETECT_INTERRUPT** = 8, **SDADC_DATA_SATURATION_DETECT_INTERRUPT** = 9, **SDADC_CLOCK_ABSENCE_DETECT_INTERRUPT** = 10, **SDADC_SIGN_DELAY_CAPTURE_INTERRUPT** =

11, **SDADC_TSIGN_CHANGED_INTERRUPT** = 12, **SDADC_TS_VALID_INTERRUPT** = 13,
SDADC_INTERRUPT_MAX = 14 }

Functions

void **SDADC_InitCfg** (**SDADC_InitType** *pInitCfg)

Initialize the SDADC configuration.

void **SDADC_InitChannelCfg** (**SDADC_ChannelInitType** *pInitCfg)

Initialize the SDADC channel configuration.

void **SDADC_InitModulatorCfg** (**SDADC_ModulatorCfgType** *pInitCfg)

Initialize the SDADC modulator configuration.

void **SDADC_InitPwmCfg** (**SDADC_PWMInitType** *pInitCfg)

Initialize the SDADC PWM configuration.

void **SDADC_InitFilterChainCfg** (**SDADC_FilterChainCfgType** *pInitCfg)

Initialize the SDADC filter chain configuration.

void **SDADC_InitDectCfg** (**SDADC_DetectCfgType** *pInitCfg)

Initialize the SDADC detection configuration.

SDADC_RetStatusType **SDADC_Init** (**SDADC_HandleType** *pSdadCHandle, const **SDADC_InitType** *const pInitCfg)

Initialize the SDADC with the given configuration.

void **SDADC_Deinit** (**SDADC_HandleType** *pSdadCHandle, uint32_t RstHoldCount)

Deinitialize the SDADC.

void **SDADC_ChannelEnable** (**SDADC_HandleType** *pSdadCHandle, uint8_t u8ChannelIndex, uint8_t u8Enable)

Enable or disable a specific SDADC channel.

SDADC_RetStatusType **SDADC_MainEnable** (**SDADC_HandleType** *pSdadCHandle, uint8_t u8Enable)

Enable or disable the main functionality of the SDADC.

void SDADC_ChannelSWTrigger (SDADC_HandleType *pSdadCHandle, SDADC_ChannelMaskType u8ChannelMask)

Trigger the SDADC channels via software.

void SDADC_ClearChannelSWTrigger (SDADC_HandleType *pSdadCHandle, SDADC_ChannelMaskType u8ChannelMask)

Clear the software trigger status of specified SDADC channels.

void SDADC_IntegratorSWTrigger (SDADC_HandleType *pSdadCHandle, SDADC_ChannelMaskType u8ChannelMask)

Trigger the SDADC integrator via software.

void SDADC_EnablePWMSignTriggerOut (SDADC_HandleType *pSdadCHandle, uint8_t u8Enable)

Enable or disable the Pwm Sign trigger out for a specific SDADC channel.

void SDADC_EnableTSignTriggerOut (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint8_t u8Enable)

Enable or disable the TSign trigger out for a specific SDADC channel.

SDADC_RetStatusType SDADC_GetConversionResult (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint32_t *pData)

Get the conversion result of a specific SDADC channel.

SDADC_RetStatusType SDADC_GetCaptureTimeStamp (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint16_t *pTimestamp, uint16_t *pLastCov)

Get the capture timestamp of a specific SDADC channel.

SDADC_RetStatusType SDADC_GetSignDelayCapture (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint8_t *pCapture)

Get the sign delay capture of a specific SDADC channel.

uint8_t SDADC_GetChannelReady (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the ready status of a specific SDADC channel.

uint8_t SDADC_GetChannelConverting (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the converting status of a specific SDADC channel.

uint8_t SDADC_GetChannelConversionComplete (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the conversion complete status of a specific SDADC channel.

uint8_t SDADC_GetChannelFIFOEmpty (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the FIFO empty status of a specific SDADC channel.

uint8_t SDADC_GetChannelFIFOReady (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the FIFO ready status of a specific SDADC channel.

void SDADC_ClearChannelFIFOReady (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Clear the FIFO ready status of a specific SDADC channel.

void SDADC_ClearChannelConversionComplete (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Clear the conversion complete status of a specific SDADC channel.

SDADC_RetStatusType SDADC_StartPwmGeneration (SDADC_HandleType *pSdadCHandle, SDADC_PWMPatternType ePattern)

Start PWM generation with the specified pattern.

SDADC_RetStatusType SDADC_StopPwmGeneration (SDADC_HandleType *pSdadCHandle)

Stop PWM generation.

void SDADC_ClearIntegratorSWTrigger (SDADC_HandleType *pSdadCHandle, SDADC_ChannelMaskType u8ChannelMask)

Clear the integrator software trigger for the specified SDADC channels.

```
void SDADC_SetInterruptEnable (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex,  
SDADC_InterruptionType eIntType, uint8_t u8Enable)
```

Set the interrupt enable status for the specified SDADC interrupt type.

```
void SDADCn_IRQHandler (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)
```

Detailed Description

Macro Definition Documentation

```
#define SDADC_CHANNEL_CNT 3U
```

```
#define SDADC_CHANNELENABLE_ID 0x04U
```

```
#define SDADC_CHANNELSWTRIGGER_ID 0x03U
```

```
#define SDADC_CLEARCHANNELCONVERSIONCOMPLETE_ID 0x11U
```

```
#define SDADC_CLEARCHANNELFIFOREADY_ID 0x10U
```

```
#define SDADC_CLEARCHANNELSWTRIGGER_ID 0x05U
```

```
#define SDADC_DEINIT_ID 0x01U
```

```
#define SDADC_DEV_ERROR_REPORT STD_ON
```

```
#define SDADC_E_NOT_OK (0xFFU)
```

Return not ok

```
#define SDADC_E_OK (0U)
```

Return ok

```
#define SDADC_E_PARAM_CHANNEL 0x02U

#define SDADC_E_PARAM_INSTANCE 0x01U

#define SDADC_E_PARAM_POINTER 0x00U

#define SDADC_ENABLEPWMSIGNTRIGGEROUT_ID 0x14U

#define SDADC_ENABLETSIGNTRIGGEROUT_ID 0x07U

#define SDADC_GETCAPTURETIMESTAMP_ID 0x09U

#define SDADC_GETCHANNELCONVERSIONCOMPLETE_ID 0x0DU

#define SDADC_GETCHANNELCONVERTING_ID 0x0CU

#define SDADC_GETCHANNELFIFOEMPTY_ID 0x0EU

#define SDADC_GETCHANNELFIFOREADY_ID 0x0FU

#define SDADC_GETCHANNELREADY_ID 0x0BU

#define SDADC_GETCONVERSIONRESULT_ID 0x08U

#define SDADC_GETSIGNDELAYCAPTURE_ID 0x0AU

#define SDADC_INIT_ID 0x00U

#define SDADC_INSTANCE_0 0U,
    SDADC instance 0 is selected

#define SDADC_INSTANCE_1 1U
    SDADC instance 1 is selected
```

```

#define SDADC_INTEGRATORSWTRIGGER_ID 0x06U

#define SDADC_MAIN_ENABLE_ID 0x02U

#define SDADC_MFSHIFT_DATALENGTH 24U

#define SDADC_MFSHIFT_LEFT 0x10U

#define SDADC_MFSHIFT_RIGHT 0x0U

#define SDADC_ReportDevError( func, error) ReportDevError(SDADC_MODULE_ID, func, error)

#define SDADC_SETINTERRUPPTENABLE_ID 0x15U

#define SDADC_STARTPWMGENERATION_ID 0x12U

#define SDADC_STOPPWMGENERATION_ID 0x13U

```

Typedef Documentation

```

typedef uint8_t SDADC_ChannelMaskType

typedef struct _SDADC_HandleType SDADC_HandleType

typedef uint8_t SDADC_InstanceType

typedef uint8_t SDADC_RetStatusType

```

Enumeration Type Documentation

enum SDADC_AFCMType

Auxiliary Filter Conversion Mode.

Enumerator:

SDADC_AFCM_ALWAYS_ON	0b - Always on
----------------------	----------------

SDADC_AFCM_FOLLOW_MAIN	1b - Follow Main Filter mode (trigger)
------------------------	--

enum SDADC_AFORDType

Auxiliary Filter Order.

Enumerator:

SDADC_AFORD_RESERVED_00	00b - Reserved
SDADC_AFORD_FIRST_ORDER	01b - First order filter
SDADC_AFORD_SECOND_ORDER	10b - Second order filter
SDADC_AFORD_THIRD_ORDER	11b - Third order filter

enum SDADC_BitRevType

Enumerator:

SDADC_BIT_REV_NORMAL	0x0 - Normal mode, a triangle wave is used as PWM carrier
SDADC_BIT_REV_REVERSE	0x1 - Bit reverse mode. A bit reversed sawtooth wave is used as PWM carrier

enum SDADC_CICSelectionType

Enumerator:

SDADC_SEL_AUXILIARY	0b - Auxiliary Filter
SDADC_SEL_MAIN	1b - Main Filter

enum SDADC_CLKEDGType

Input Clock Edge Select.

Enumerator:

SDADC_CLKEDG_RESERVED	00b - Reserved
SDADC_CLKEDG_POSEDGE	01b - Posedge
SDADC_CLKEDG_NEGEDGE	10b - Negedge
SDADC_CLKEDG_BOTH_EDGE	11b - Both edge

enum SDADC_CLKSELType

Input Clock Select.

Enumerator:

SDADC_CLKSEL_RESERVED	00b - Reserved
SDADC_CLKSEL_DIV_CLK1	01b - Internal generated function clock div_clk1 selected
SDADC_CLKSEL_DIV_CLK2	10b - Internal generated function clock div_clk2 selected
SDADC_CLKSEL_EXTERNAL	11b - Modulator clock from external modulator selected

enum SDADC_CMENMType

Common Voltage Configuration of Negative Inputs.

Enumerator:

SDADC_CMENM _NOT_CONNEC TED	0b - Is NOT connected to the common mode voltage
SDADC_CMENM _CONNECTED	1b - Is connected to the common mode voltage

enum SDADC_CMENPType

Common Voltage Configuration of Positive Inputs.

Enumerator:

SDADC_CMENP _NOT_CONNEC TED	0b - Is NOT connected to the common mode voltage
SDADC_CMENP _CONNECTED	1b - Is connected to the common mode voltage

enum SDADC_CTRGSIGN_OENType

Channel Sign Signal Related Trigger Out Enable.

Enumerator:

SDADC_CTRGSI GN_OEN_DISAB LED	0b - disabled
SDADC_CTRGSI GN_OEN_ENABL ED	1b - Related trigger is enabled and will be set once TSIGN falling edge is got

enum SDADC_DetectIsrStatusType**Enumerator:**

SDADC_DET_ZE RO_CROSS	Zero crossing detected
--------------------------	------------------------

SDADC_DET_HIGH_LMT_REACH	High limit reached
SDADC_DET_LOW_LMT_REACH	Low limit reached
SDADC_DET_WINDOW_LMT_REACH	Window limit reached
SDADC_DET_SHORT_CIRCUIT	Short circuit detected
SDADC_DET_DATA_SATURATION	Data saturation detected
SDADC_DET_CLOCK_ABSENCE	Clock absence detected

enum SDADC_DITHER_ENType

Dither Enable.

Enumerator:

SDADC_DITHER_EN_DISABLED	0b - Dither is disabled
SDADC_DITHER_EN_ENABLED	1b - Dither is enabled

enum SDADC_EXTSTPType

Integration Stop Control.

Enumerator:

SDADC_EXTSTP_INTERNAL	0b - Internal control, stop integration after NVALREP+1 integration cycles, support both edge and level events
-----------------------	--

SDADC_EXTSTP _EXTERNAL	1b - External control, stop integration upon the inverse trigger event
---------------------------	--

enum SDADC_FIFOIsrStatusType

Enumerator:

SDADC_FIFO_RE ADY	FIFO is ready for data
SDADC_FIFO_O VERFLOW	FIFO has overflowed
SDADC_FIFO_U NDERFLOW	FIFO has underflowed

enum SDADC_GainCompType

Enumerator:

SDADC_GAIN_C MP_BYPASS	00b - Bypass gain compensation
SDADC_GAIN_C MP_INTERNAL_ GAIN1	01b - use internal gain factor of gain 1
SDADC_GAIN_C MP_INTERNAL_ GAIN2	02b - use internal gain factor of gain 2
SDADC_GAIN_C MP_EXTERNAL_ GAIN	03b - use external gain factor

enum SDADC_GAINSELType

Gain Select of Analog Input Path.

Enumerator:

SDADC_GAINSE L_1	00b - 1
SDADC_GAINSE L_2	01b - 2
SDADC_GAINSE L_4	10b - 4
SDADC_GAINSE L_8	11b - 8

enum SDADC_HTRPOLType

Hardware Trigger Polarity.

Enumerator:

SDADC_HTRPOL _HIGH_RISING	0b - High and rising edge effective
SDADC_HTRPOL _LOW_FALLING	1b - Low and falling edge effective

enum SDADC_IBFMTType

Input Bit Format.

Enumerator:

SDADC_IBFMT_ ONE_BIT_STRE AM	00b - External one bit stream
SDADC_IBFMT_ MANCHESTER_C ODE	01b - External Manchester code, CLKEDG will determine rise or fall decoder

enum SDADC_INCFGMTType

Configuration of Negative Input Line.

Enumerator:

SDADC_INCFG _FROM_PIN	00b - From input Pin
SDADC_INCFG _FROM_VREFL	01b - From VREFL
SDADC_INCFG _FROM_VREFH	10b - From VREFH
SDADC_INCFG _FROM_VREFH_ DIV_2	11b - From VREFH/2

enum SDADC_INCFGPTType

Configuration of Positive Input Line.

Enumerator:

SDADC_INCFG _FROM_PIN	00b - From input Pin
SDADC_INCFG _FROM_VREFL	01b - From VREFL
SDADC_INCFG _FROM_VREFH	10b - From VREFH
SDADC_INCFG _FROM_VREFH_ DIV_2	11b - From VREFH/2

enum SDADC_INCFGPV11Type

Configuration of Positive Input Line with V11 from System.

Enumerator:

SDADC_INCFGP V11_DISABLED	0b - Positive input from V11 disabled
SDADC_INCFGP V11_ENABLED	1b - Positive input from V11 enabled

enum SDADC_INMODType

Input MUX Control Mode.

Enumerator:

SDADC_INMOD_ SOFTWARE_CO NTROL	00b - Software control mode (INMUX follows INSEL)
SDADC_INMOD_ PRESET_MODE	01b - Preset mode (load INMUX upon a channel conversion trigger)
SDADC_INMOD_ SCAN_MODE	10b - Scan mode (decrement INMUX upon a channel conversion trigger, wrap around to INSEL)
SDADC_INMOD_ RESERVED	11b - Reserved

enum SDADC_INMXCPTType

Input Mux Setting Copy Enable.

Enumerator:

SDADC_INMXCP _DISABLED	0b - Disabled
SDADC_INMXCP _ENABLED	1b - Copy INMUX from CMACSR to TSTAMP[15:14]

enum SDADC_INSELType

Input Pin Selection.

Enumerator:

SDADC_INSEL_P IN_0	00b - Input pin 0 selected
SDADC_INSEL_P IN_1	01b - Input pin 1 selected
SDADC_INSEL_P IN_2	10b - Input pin 2 selected
SDADC_INSEL_P IN_3	11b - Input pin 3 selected

enum SDADC_InterruptType

Enumerator:

SDADC_COCl_IN TERRUPT	Conversion Complete Interrupt (COCl)
SDADC_FIFO_RE ADY_INTERRUPT	FIFO Ready
SDADC_FIFO_O VERFLOW_INTERRUPT	FIFO Overflow
SDADC_FIFO_U NDERFLOW_INTERRUPT	FIFO Underflow
SDADC_ZERO_C ROSS_DETECT_INTERRUPT	Zero Cross Detection
SDADC_HIGH_LI MIT_REACH_INTERRUPT	High Limit Reached

SDADC_LOW_LIMIT_REACH_INTERRUPT	Low Limit Reached
SDADC_WINDOW_LIMIT_REACH_INTERRUPT	Window Limit Reached
SDADC_SHORT_CIRCUIT_DETECTION_INTERRUPT	Short Circuit Detection
SDADC_DATA_SATURATION_DETECTION_INTERRUPT	Data Saturation Detection
SDADC_CLOCK_ABSENCE_DETECTION_INTERRUPT	Clock Absence Detection
SDADC_SIGN_DELAY_CAPTURE_INTERRUPT	Sign Delay Capture
SDADC_TSIGN_CHANGED_INTERRUPT	TSign Changed
SDADC_TS_VALID_INTERRUPT	TimeStamp Valid
SDADC_INTERRUPT_MAX	Maximum interrupt type

enum SDADC_IOGTType

Integration Output Gating.

Enumerator:

SDADC_IOGT_ALWAYS_WORK	0b - Conversion complete flag and FIFO push always works
SDADC_IOGT_ONLY_WHEN_INT	1b - Conversion complete flag and FIFO push can only work when Integration is on-going(INTON asserted)

EGRATION_ONG OING	
----------------------	--

enum SDADC_LMTAVTType

Limitation Result Activation Select.

Enumerator:

SDADC_LMTAVT _SET_ON_LARGE _CLEAR_ON_ SMALLER	0b - The compare result is set only when the conversion result is larger than HILMTn and clear only when the conversion result is smaller than LOLMTn
SDADC_LMTAVT _CLEAR_ON_LARGE _SET_ON_ SMALLER	1b - The compare result is clear only when the conversion result is larger than HILMTn and set only when the conversion result is smaller than LOLMTn

enum SDADC_LMTOPTType

Limitation Option.

Enumerator:

SDADC_LMTOPT _DETECT_BOTH _HIGH_AND_LO W	00b - Limitation detect on exceed both high and low value
SDADC_LMTOPT _DETECT_HIGH _THRESHOLD	01b - Limitation detect on exceed high threshold value
SDADC_LMTOPT _DETECT_LOW_ THRESHOLD	10b - Limitation detect on exceed low threshold value
SDADC_LMTOPT _DETECT_WINDOWED_VALUE	11b - Limitation detect on windowed value

enum SDADC_MACLKSELType

Internal Modulator Analog Clock Selection.

Enumerator:

SDADC_MACLKS EL_DIV_CLK1	00b - Div_clk1 is selected as the clock
SDADC_MACLKS EL_DIV_CLK0	01b - Div_clk0 is selected as the clock
SDADC_MACLKS EL_RESERVED	1*b - Reserved

enum SDADC_MASELType

Modulator Analog Selection.

Enumerator:

SDADC_MASEL_ EXTERNAL	0b - External modulator analog is selected as channel input
SDADC_MASEL_ INTERNAL	1b - Internal modulator analog is selected as channel input

enum SDADC_MENSELType

Main Enable Selection.

Enumerator:

SDADC_MENSEL_ _INTERNAL	0b - Use internal MEN
SDADC_MENSEL_ _EXTERNAL	1b - Use external MEN

enum SDADC_MFCMType

Main Filter Conversion Mode.

Enumerator:

SDADC_MFCM_SINGLE_MODE	00b - Single mode. One conversion following a triggering event (edge or level)
SDADC_MFCM_CONTINUOUS_MODE	01b - Continuous conversion mode. Multiple conversions following a triggering event, and the next trigger will cancel and restart conversion
SDADC_MFCM_ALWAYS_MODE	10b - Always conversion mode. Multiple conversions following the first trigger, and the next trigger will be ignored
SDADC_MFCM_RESERVED_11	11b - Reserved

enum SDADC_MFORDType

Main Filter Order.

Enumerator:

SDADC_MFORD_RESERVED_000	000b - Reserved
SDADC_MFORD_FIRST_ORDER	001b - First order filter
SDADC_MFORD_SECOND_ORDER	010b - Second order filter
SDADC_MFORD_THIRD_ORDER	011b - Third order filter

SDADC_MFORD _FOURTH_ORDER	100b - Fourth order filter
SDADC_MFORD _FIFTH_ORDER	101b - Fifth order filter
SDADC_MFORD _RESERVED_110 _111	110,111 - Reserved

enum SDADC_PrescaleType

PRE_DIV Divider Value.

Enumerator:

SDADC_PRESCALE_NO_PRESCALE	00b - No prescale
SDADC_PRESCALE_DIV_BY_2	01b - Divided by 2
SDADC_PRESCALE_DIV_BY_4	10b - Divided by 4
SDADC_PRESCALE_DIV_BY_8	11b - Divided by 8

enum SDADC_PWM_CLKSELType

Enumerator:

SDADC_PWM_CLKSEL_DIV_CLK0	0x0 - div_clk0
SDADC_PWM_CLKSEL_DIV_CLK1	0x1 - div_clk1
SDADC_PWM_CLKSEL_DIV_CLK2	0x2 - div_clk2

SDADC_PWM_C LKSEL_OFF	0x3 - off
--------------------------	-----------

enum SDADC_PWMPatternType

Enumerator:

SDADC_PWM_P ATERN_STOPPED	0x0 - Stopped
SDADC_PWM_P ATERN_SQUARE	0x1 - Square wave
SDADC_PWM_P ATERN_TRIANGLE	0x2 - Triangle
SDADC_PWM_P ATERN_SINE	0x3 - Sine wave

enum SDADC_PWMPolarityType

Enumerator:

SDADC_PWM_P OLARITY_POSITIVE	0x0 - PWM generator output begin with +1
SDADC_PWM_P OLARITY_NEGATIVE	0x1 - PWM generator output begin with -1

enum SDADC_RDFMTType

Result Data Format.

Enumerator:

SDADC_RDFMT_ LEFT_JUSTIFIED _SIGNED	0b - Left justified, signed, for the case where input bitstream is signed
---	---

SDADC_RDFMT_LEFT_JUSTIFIED_UNSIGNED	1b - Left justified, unsigned, for the case where input bitstream is unsigned

enum SDADC_SCDCMType

Short Circuit Detect Conversion Mode.

Enumerator:

SDADC_SCDCM_ALWAYS_ON	0b - Always on when CHEN and MEN are asserted
SDADC_SCDCM_FILTER_IN_PROGRESS	1b - Only works when the any filter is in progress

enum SDADC_SCDOPTType

Short Circuit Option.

Enumerator:

SDADC_SCDOPT_COUNT_ON_BOTH_0_AND_1	00b - Count on both 0 and 1. Detect both high and low value
SDADC_SCDOPT_COUNT_ON_1	01b - Count on 1. Detect high value
SDADC_SCDOPT_COUNT_ON_0	10b - Count on 0. Detect low value
SDADC_SCDOPT_RESERVED	11b - Reserved

enum SDADC_SSCHType

Sign Signal From Channel Selection.

Enumerator:

SDADC_SSCH_C H0	0000b - From channel 0
SDADC_SSCH_C H1	0001b - From channel 1
SDADC_SSCH_C H2	0010b - From channel 2

enum SDADC_SSRCType

Sign Source Selection.

Enumerator:

SDADC_SSRC_I NTERNAL_PWM	00b - SDADC internal PWM generation block
SDADC_SSRC_C HANNEL_SELEC TED	01b - Sign of result of channel selected by SSCH
SDADC_SSRC_E XTERNAL_SIGN AL_0	10b - External sign signal 0
SDADC_SSRC_E XTERNAL_SIGN AL_1	11b - External sign signal 1

enum SDADC_SYNC_TRGENType

Synced Trigger Select Enable.

Enumerator:

SDADC_SYNC_T RGEN_SELF	0b - Synced trigger is selected on the channel itself
---------------------------	---

SDADC_SYNC_T RGEN_DEFINED	1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL
------------------------------	---

enum SDADC_TRGLVLType

Trigger Level type.

Enumerator:

SDADC_TRGLVL _EDGE	0b - Trigger is edge event. Next edge event will cancel the current conversion, and re-start in single or continuous mode
SDADC_TRGLVL _LEVEL	1b - Trigger is level event. De-assert cancels the current conversion, and stop filter function in single or continuous mode

enum SDADC_TRGSELType

Trigger Select.

Enumerator:

SDADC_TRGSEL _SOFTWARE	0b - Software trigger is selected
SDADC_TRGSEL _HARDWARE	1b - Hardware trigger is selected

enum SDADC_TRGSignOENType

Enumerator:

SDADC_TRGSIG N_OEN_DISABL ED	0x0 - disabled
------------------------------------	----------------

SDADC_TRGSIGN_OEN_ENABLE_D	0x1 - Related trigger is enabled and will be set once SIGN falling edge is got
----------------------------	--

enum SDADC_TS_ENType

Timestamp Counter Enable.

Enumerator:

SDADC_TS_EN_DISABLED	0b - Disabled
SDADC_TS_EN_ENABLED	1b - Counting at the rate selected by TSCLKSEL

enum SDADC_TSCLKSELType

Timestamp Counter Clock Selection.

Enumerator:

SDADC_TSCLKSEL_MODULATOR_CLOCK	00b - Modulator clock
SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_2	01b - Modulator clock divided by 2
SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_4	10b - Modulator clock divided by 4
SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_8	11b - Modulator clock divided by 8

enum SDADC_TSRDMType

Timestamp Information Read Mode.

Enumerator:

SDADC_TSRDM_DEFAULT_MODE	00b - Default mode
SDADC_TSRDM_DIRECT_MODE	01b - Direct mode
SDADC_TSRDM_FIFO_GATING_MODE	10b - FIFO gating mode
SDADC_TSRDM_RESERVED	11b - Reserved

enum SDADC_TSTRGMTType

Timestamp Counter Trigger Mode.

Enumerator:

SDADC_TSTRGM_IGNORED	00/11b - Trigger is ignored
SDADC_TSTRGM_RISING_EDGE	01b - Rising edge effective
SDADC_TSTRGM_FALLING_EDGE	10b - Falling edge effective

Function Documentation

void SDADC_ChannelEnable (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*, uint8_t *u8Enable*)

Enable or disable a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
---------------------	---------------------------------------

<i>u8ChannelIndex</i>	Index of the channel to enable or disable
<i>u8Enable</i>	Enable (1) or disable (0) the channel

void SDADC_ChannelSWTrigger (SDADC_HandleType * *pSdadCHandle*, SDADC_ChannelMaskType *u8ChannelMask*)

Trigger the SDADC channels via software.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelMask</i>	Mask of the channels to trigger

void SDADC_ClearChannelConversionComplete (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*)

Clear the conversion complete status of a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to clear the conversion complete status for

void SDADC_ClearChannelFIFOReady (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*)

Clear the FIFO ready status of a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to clear the FIFO ready status for

void SDADC_ClearChannelSWTrigger (SDADC_HandleType * *pSdadCHandle*, SDADC_ChannelMaskType *u8ChannelMask*)

Clear the software trigger status of specified SDADC channels.

This function clears the software trigger status for the SDADC channels specified by the channel mask. This is useful to reset the trigger state of the channels after a software trigger has been issued.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelMask</i>	Mask of the channels to clear the software trigger status for

void SDADC_ClearIntegratorSWTrigger (SDADC_HandleType * *pSdadCHandle*, SDADC_ChannelMaskType *u8ChannelMask*)

Clear the integrator software trigger for the specified SDADC channels.

This function clears the integrator software trigger for the SDADC channels specified by the channel mask.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelMask</i>	Channel mask indicating which channels to clear the integrator trigger for

Returns

void

void SDADC_Deinit (SDADC_HandleType * *pSdadCHandle*, uint32_t *RstHoldCount*)

Deinitialize the SDADC.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>RstHoldCount</i>	Number of cycles to hold the reset signal

Returns

SDADC_RetStatusType Return status of the deinitialization operation

void SDADC_EnablePWMSignTriggerOut (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8Enable*)

Enable or disable the Pwm Sign trigger out for a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to enable or disable the TSign trigger out for

<i>u8Enable</i>	Enable (1) or disable (0) the TSign trigger out
-----------------	---

void SDADC_EnableTSignTriggerOut (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*, uint8_t *u8Enable*)

Enable or disable the TSign trigger out for a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to enable or disable the TSign trigger out for
<i>u8Enable</i>	Enable (1) or disable (0) the TSign trigger out

SDADC_RetStatusType SDADC_GetCaptureTimeStamp (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*, uint16_t * *pTimestamp*, uint16_t * *pLastCov*)

Get the capture timestamp of a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the capture timestamp for
<i>pTimestamp</i>	Pointer to store the capture timestamp
<i>pLastCov</i>	Pointer to store the last conversion information

Returns

SDADC_RetStatusType Return status of the get capture timestamp operation

uint8_t SDADC_GetChannelConversionComplete (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*)

Get the conversion complete status of a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the conversion complete status for

Returns

uint8_t Conversion complete status of the channel

uint8_t SDADC_GetChannelConverting (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*)

Get the converting status of a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the converting status for

Returns

uint8_t Converting status of the channel

uint8_t SDADC_GetChannelFIFOEmpty (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*)

Get the FIFO empty status of a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the FIFO empty status for

Returns

uint8_t FIFO empty status of the channel

uint8_t SDADC_GetChannelFIFOReady (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8ChannelIndex*)

Get the FIFO ready status of a specific SDADC channel.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the FIFO ready status for

Returns

uint8_t FIFO ready status of the channel

uint8_t SDADC_GetChannelReady (SDADC_HandleType * *pSdadchandle*, uint8_t *u8ChannelIndex*)

Get the ready status of a specific SDADC channel.

Parameters

<i>pSdadchandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the ready status for

Returns

uint8_t Ready status of the channel

SDADC_RetStatusType SDADC_GetConversionResult (SDADC_HandleType * *pSdadchandle*, uint8_t *u8ChannelIndex*, uint32_t * *pData*)

Get the conversion result of a specific SDADC channel.

Parameters

<i>pSdadchandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the conversion result for
<i>pData</i>	Pointer to store the conversion result

Returns

SDADC_RetStatusType Return status of the get conversion result operation

SDADC_RetStatusType SDADC_GetSignDelayCapture (SDADC_HandleType * *pSdadchandle*, uint8_t *u8ChannelIndex*, uint8_t * *pCapture*)

Get the sign delay capture of a specific SDADC channel.

Parameters

<i>pSdadchandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the channel to get the sign delay capture for
<i>pCapture</i>	Pointer to store the sign delay capture

Returns

SDADC_RetStatusType Return status of the get sign delay capture operation

SDADC_RetStatusType SDADC_Init (SDADC_HandleType * *pSdadCHandle*, const SDADC_InitType *const *pInitCfg*)

Initialize the SDADC with the given configuration.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>pInitCfg</i>	Pointer to the SDADC initialization configuration structure

Returns

SDADC_RetStatusType Return status of the initialization operation

void SDADC_InitCfg (SDADC_InitType * *pInitCfg*)

Initialize the SDADC configuration.

Parameters

<i>pInitCfg</i>	Pointer to the SDADC initialization configuration structure
-----------------	---

void SDADC_InitChannelCfg (SDADC_ChannelInitType * *pInitCfg*)

Initialize the SDADC channel configuration.

Parameters

<i>pInitCfg</i>	Pointer to the SDADC channel initialization configuration structure
-----------------	---

void SDADC_InitDectCfg (SDADC_DetectCfgType * *pInitCfg*)

Initialize the SDADC detection configuration.

Parameters

<i>pInitCfg</i>	Pointer to the SDADC detection initialization configuration structure
-----------------	---

void SDADC_InitFilterChainCfg (SDADC_FilterChainCfgType * *pInitCfg*)

Initialize the SDADC filter chain configuration.

Parameters

<i>pInitCfg</i>	Pointer to the SDADC filter chain initialization configuration structure
-----------------	--

void SDADC_InitModulatorCfg (SDADC_ModulatorCfgType * *pInitCfg*)

Initialize the SDADC modulator configuration.

Parameters

<i>pInitCfg</i>	Pointer to the SDADC modulator initialization configuration structure
-----------------	---

void SDADC_InitPwmCfg (SDADC_PWMInitType * *pInitCfg*)

Initialize the SDADC PWM configuration.

Parameters

<i>pInitCfg</i>	Pointer to the SDADC PWM initialization configuration structure
-----------------	---

void SDADC_IntegratorSWTrigger (SDADC_HandleType * *pSdadCHandle*, SDADC_ChannelMaskType *u8ChannelMask*)

Trigger the SDADC integrator via software.

Parameters

<i>pSdadCHandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelMask</i>	Mask of the channels to trigger the integrator for

SDADC_RetStatusType SDADC_MainEnable (SDADC_HandleType * *pSdadCHandle*, uint8_t *u8Enable*)

Enable or disable the main functionality of the SDADC.

Parameters

<i>pSdadchandle</i>	Pointer to the SDADC handle structure
<i>u8Enable</i>	Enable (1) or disable (0) the main functionality

Returns

SDADC_RetStatusType Return status of the main enable operation

void SDADC_SetInterruptEnable (SDADC_HandleType * *pSdadchandle*, uint8_t *u8ChannelIndex*, SDADC_InterruptType *eIntType*, uint8_t *u8Enable*)

Set the interrupt enable status for the specified SDADC interrupt type.

This function enables or disables the specified interrupt type for the given SDADC channel. Before setting the interrupt enable status, the function clears the corresponding interrupt status to ensure no pending interrupts are left unhandled.

Parameters

<i>pSdadchandle</i>	Pointer to the SDADC handle structure
<i>u8ChannelIndex</i>	Index of the SDADC channel
<i>eIntType</i>	Interrupt type to be configured (e.g., SDADC_COCI_INTERRUPT, SDADC_FIFO_READY_INTERRUPT, etc.)
<i>u8Enable</i>	Enable or disable the interrupt (1 to enable, 0 to disable)

Returns

void

SDADC_RetStatusType SDADC_StartPwmGeneration (SDADC_HandleType * *pSdadchandle*, SDADC_PWMPatternType *ePattern*)

Start PWM generation with the specified pattern.

Parameters

<i>pSdadchandle</i>	Pointer to the SDADC handle structure
<i>ePattern</i>	The PWM pattern to be generated

Returns

SDADC_RetStatusType Return status of the PWM generation start operation (SDADC_E_OK if successful, SDADC_E_NOT_OK otherwise)

SDADC_RetStatusType SDADC_StopPwmGeneration (SDADC_HandleType * *pSdadchandle*)

Stop PWM generation.

Parameters

<i>pSdadchandle</i>	Pointer to the SDADC handle structure
---------------------	---------------------------------------

Returns

SDADC_RetStatusType Return status of the PWM generation stop operation (SDADC_E_OK if successful, SDADC_E_NOT_OK otherwise)

void SDADCn_IRQHandler (SDADC_HandleType * *pSdadchandle*, uint8_t *u8ChannelIndex*)

Module_driver_sddf

Collaboration diagram for Module_driver_sddf:Modules

HwA_sddf

Classes

struct **SDDF_ChannelNotificationType**

struct **_SDDF_HandleType**

struct **SDDF_InitConfigType**

The init configuration for SDDF instance.

struct **SDDF_ClockOutConfigType**

The clock out configuration for SDDF.

struct **SDDF_MainFilterConfigType**

The configuration for main filter.

struct **SDDF_AuxFilterConfigType**

The configuration for auxiliary filter.

struct **SDDF_ChannelConfigType**

The configuration of the SDDF channel.

struct **SDDF_ClockAbsenceDetectConfigType**

The configuration of the clock absence detect.

struct **SDDF_ShortCircuitDetectConfigType**

The configuration of the short circuit detect.

struct **SDDF_LimitCheckConfigType**

The configuration of the limit detect.

struct **SDDF_ZeroCrossDetectConfigType**

The configuration of the zero cross detect.

struct **SDDF_ChannelInterruptConfigType**

The result related interrupt configuraion of the SDDF channel.

struct **SDDF_ChannelDMAConfigType**

The DMA configuration of the SDDF channel.

Typedefs

typedef void(* **SDDF_GeneralCallbackType**) (void)

The general callback function prototype for SDDF interrupt.

typedef void(* **SDDF_FIFOReadyCallbackType**) (const uint32_t *const pData)

Function prototype for SDDF FIFO ready interrupt.

typedef void(* **SDDF_EventDetectCallbackType**) (const **SDDF_DetectEventType** eEvent)

Function prototype for SDDF event detect interrupt.

typedef struct **_SDDF_HandleType** **SDDF_HandleType**

Enumerations

enum **SDDF_InstanceType** { **SDDF_INSTANCE_0** = 0U }

The instance index of the SDDF peripheral.

enum **SDDF_DetectEventType** { **SDDF_CLOCK_ABSENCE_EVENT** = 0U, **SDDF_SHORT_CIRCUIT_EVENT** = 1U, **SDDF_LOW_LIMIT_EVENT** = 2U, **SDDF_HIGH_LIMIT_EVENT** = 3U, **SDDF_WINDOW_LIMIT_EVENT** = 4U, **SDDF_ZERO_CROSS_EVENT** = 5U }

The event detected by SDDF detectors.

enum **SDDF_ClockOutType** { **SDDF_CLOCK_OUT_DISABLED** = 0U, **SDDF_CLOCK_OUT_0** = 1U, **SDDF_CLOCK_OUT_1** = 2U, **SDDF_CLOCK_OUT_2** = 3U }

Select the clock output source.

enum **SDDF_ClcokInputType** { **SDDF_CLOCK_INPUT_CLK0** = 0U, **SDDF_CLOCK_INPUT_CLK1** = 1U, **SDDF_CLOCK_INPUT_CLK2** = 2U, **SDDF_CLOCK_INPUT_CLK3** = 3U }

Select the clock channel for SDDF external clock input.

Functions

void **SDDF_DeInit** (**SDDF_HandleType** *pHandle)

De-initialize the SDDF instance.

void **SDDF_Init** (**SDDF_HandleType** *pHandle, const **SDDF_InitConfigType** *pInitCfg)

Initialize the SDDF instance.

void **SDDF_ConfigClockOut** (**SDDF_HandleType** *pHandle, const **SDDF_ClockOutConfigType** *pClockOutCfg)

Configure the SDDF clock out.

void **SDDF_Enable** (**SDDF_HandleType** *pHandle)

Enable the SDDF instance.

void **SDDF_Disable** (**SDDF_HandleType** *pHandle)

Disable the SDDF instance.

void **SDDF_EnableDebugMode** (**SDDF_HandleType** *pHandle, **SDDF_DebugModuleEnableType** eModuleEnable)

Enable the debug mode of the SDDF instance.

void **SDDF_DisableDebugMode** (**SDDF_HandleType** *pHandle)

Disable the debug mode of the SDDF instance.

void **SDDF_InitChannel** (**SDDF_HandleType** *pHandle, uint8_t u8Channel, const **SDDF_ChannelConfigType** *pChannelCfg)

Initialize the selected SDDF channel.

void **SDDF_ChannelEnable** (**SDDF_HandleType** *pHandle, uint8_t u8Channel)

Enable the conversion of the selected SDDF channel.

void **SDDF_ChannelDisable** (**SDDF_HandleType** *pHandle, uint8_t u8Channel)

Disable the conversion of the selected SDDF channel.

void **SDDF_ConfigChannelInterrupt** (SDDF_HandleType *pHandle, uint8_t u8Channel, const SDDF_ChannelInterruptConfigType *pIntCfg)

Configure the interrupt for the selected SDDF channel.

void **SDDF_ConfigChannelDMA** (SDDF_HandleType *pHandle, uint8_t u8Channel, const SDDF_ChannelDMAConfigType *pDMACfg, DMA_TransferCompleteCallbackType pDmaCallback)

Configure the DMA for the selected the SDDF channel.

void **SDDF_ConfigChannelClockAbsenceDetect** (SDDF_HandleType *pHandle, uint8_t u8Channel, SDDF_ClockAbsenceDetectConfigType *pConfig)

Configure the clock absence detect (CAD) for the selected SDDF channel.

void **SDDF_ConfigChannelShortCircuitDetect** (SDDF_HandleType *pHandle, uint8_t u8Channel, SDDF_ShortCircuitDetectConfigType *pConfig)

Configure the short circuit detect (SCD) for the selected SDDF channel.

void **SDDF_ConfigChannelLimitCheck** (SDDF_HandleType *pHandle, uint8_t u8Channel, SDDF_LimitCheckConfigType *pConfig)

Configure the limit check for the selected SDDF channel.

void **SDDF_ConfigChannelZeroCrossDetect** (SDDF_HandleType *pHandle, uint8_t u8Channel, SDDF_ZeroCrossDetectConfigType *pConfig)

Configure the zero cross detect (ZCD) for the selected SDDF channel.

void **SDDF_GenerateChannelSwTrigger** (SDDF_HandleType *pHandle, uint8_t u8Channel)

Generate software trigger for the selected SDDF channel.

uint32_t **SDDF_GetChannelFIFOData** (SDDF_HandleType *pHandle, uint8_t u8Channel)

Get the channel data for the SDDF channel.

void **SDDFn_IRQHandler** (SDDF_HandleType *pHandle)

void **SDDFn_CHnDMAHandler** (SDDF_HandleType *pHandle, uint8_t u8Channel)

SDDF Dev Error Code

Error Code of calling SDDF apis

```
#define SDDF_E_PARAM_INSTANCE 0x01U
#define SDDF_E_PARAM_CHANNEL 0x02U
#define SDDF_E_PARAM_POINTER 0x03U
#define SDDF_E_PARAM_MAIN_FILTER 0x04U
#define SDDF_E_PARAM_AUX_FILTER 0x05U
```

SDDF API Service IDs

```
#define SDDF_INIT_ID 0x00U
#define SDDF_DEINIT_ID 0x01U
#define SDDF_CFG_CLK_OUT_ID 0x02U
#define SDDF_ENABLE_ID 0x03U
#define SDDF_DISABLE_ID 0x04U
#define SDDF_EN_DBG_MODE_ID 0x05U
#define SDDF_DIS_DBG_MODE_ID 0x06U
#define SDDF_INIT_CH_ID 0x07U
#define SDDF_CH_ENABLE_ID 0x08U
#define SDDF_CH_DISABLE_ID 0x09U
#define SDDF_CFG_CH_INT_ID 0x0AU
#define SDDF_CFG_CH_DMA_ID 0x0BU
#define SDDF_CFG_CH_CLK_ABS_DET_ID 0x0CU
#define SDDF_CFG_CH_SHORT_CIR_DET_ID 0x0DU
#define SDDF_CFG_CH_LIM_CHK_ID 0x0EU
#define SDDF_CFG_CH_ZERO_CROSS_DET_ID 0x0FU
#define SDDF_GEN_CH_SW_TRIG_ID 0x10U
#define SDDF_GET_CH_FIFO_DATA_ID 0x11U
```

Detailed Description

Macro Definition Documentation

#define SDDF_CFG_CH_CLK_ABS_DET_ID 0x0CU

#define SDDF_CFG_CH_DMA_ID 0x0BU

#define SDDF_CFG_CH_INT_ID 0x0AU

#define SDDF_CFG_CH_LIM_CHK_ID 0x0EU

#define SDDF_CFG_CH_SHORT_CIR_DET_ID 0x0DU

#define SDDF_CFG_CH_ZERO_CROSS_DET_ID 0x0FU

#define SDDF_CFG_CLK_OUT_ID 0x02U

#define SDDF_CH_DISABLE_ID 0x09U

#define SDDF_CH_ENABLE_ID 0x08U

#define SDDF_DEINIT_ID 0x01U

#define SDDF_DIS_DBG_MODE_ID 0x06U

#define SDDF_DISABLE_ID 0x04U

#define SDDF_E_PARAM_AUX_FILTER 0x05U

#define SDDF_E_PARAM_CHANNEL 0x02U

#define SDDF_E_PARAM_INSTANCE 0x01U

#define SDDF_E_PARAM_MAIN_FILTER 0x04U

#define SDDF_E_PARAM_POINTER 0x03U

#define SDDF_EN_DBG_MODE_ID 0x05U

#define SDDF_ENABLE_ID 0x03U

#define SDDF_GEN_CH_SW_TRIG_ID 0x10U

#define SDDF_GET_CH_FIFO_DATA_ID 0x11U

```
#define SDDF_INIT_CH_ID 0x07U
```

```
#define SDDF_INIT_ID 0x00U
```

Typedef Documentation

```
typedef void(* SDDF_EventDetectCallbackType) (const SDDF_DetectEventType eEvent)
```

Function prototype for SDDF event detect interrupt.

```
typedef void(* SDDF_FIFOReadyCallbackType) (const uint32_t *const pData)
```

Function prototype for SDDF FIFO ready interrupt.

```
typedef void(* SDDF_GeneralCallbackType) (void)
```

The general callback function prototype for SDDF interrupt.

```
typedef struct _SDDF_HandleType SDDF_HandleType
```

Enumeration Type Documentation

```
enum SDDF_ClcokInputType
```

Select the clock channel for SDDF external clock input.

Enumerator:

SDDF_CLOCK_I NPUT_CLK0	Clock channel 0 is used as clock input
---------------------------	--

SDDF_CLOCK_I NPUT_CLK1	Clock channel 1 is used as clock input
SDDF_CLOCK_I NPUT_CLK2	Clock channel 2 is used as clock input
SDDF_CLOCK_I NPUT_CLK3	Clock channel 3 is used as clock input

enum SDDF_ClockOutType

Select the clock output source.

Enumerator:

SDDF_CLOCK_O UT_DISABLED	The clock output is disabled
SDDF_CLOCK_O UT_0	The clock output channel uses CLOCK_OUT_0
SDDF_CLOCK_O UT_1	The clock output channel uses CLOCK_OUT_1
SDDF_CLOCK_O UT_2	The clock output channel uses CLOCK_OUT_2

enum SDDF_DetectEventType

The event detected by SDDF detectors.

Enumerator:

SDDF_CLOCK_A BSENCE_EVENT	Clock absence event is detected
SDDF_SHORT_C IRCUIT_EVENT	Short circuit event is detected

SDDF_LOW_LIMIT_EVENT	The result exceeds low limit
SDDF_HIGH_LIMIT_EVENT	The result exceeds high limit
SDDF_WINDOW_LIMIT_EVENT	The result is out of the limit window
SDDF_ZERO_CROSS_EVENT	Zero cross event is detected

enum SDDF_InstanceType

The instance index of the SDDF peripheral.

Enumerator:

SDDF_INSTANCE_0	SDDF instance 0 is selected
-----------------	-----------------------------

Function Documentation

void SDDF_ChannelDisable (SDDF_HandleType * *pHandle*, uint8_t *u8Channel*)

Disable the conversion of the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to disable

void SDDF_ChannelEnable (SDDF_HandleType * *pHandle*, uint8_t *u8Channel*)

Enable the conversion of the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to enable

void SDDF_ConfigChannelClockAbsenceDetect (SDDF_HandleType * *pHandle*, uint8_t *u8Channel*, SDDF_ClockAbsenceDetectConfigType * *pConfig*)

Configure the clock absence detect (CAD) for the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to configure
<i>pConfig</i>	the clock absence detect configuration

void SDDF_ConfigChannelDMA (SDDF_HandleType * *pHandle*, uint8_t *u8Channel*, const SDDF_ChannelDMAConfigType * *pDMACfg*, DMA_TransferCompleteCallbackType *pDmaCallback*)

Configure the DMA for the selected the SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to configure
<i>pDMACfg</i>	the DMA configuration

void SDDF_ConfigChannelInterrupt (SDDF_HandleType * *pHandle*, uint8_t *u8Channel*, const SDDF_ChannelInterruptConfigType * *pIntCfg*)

Configure the interrupt for the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to configure
<i>pIntCfg</i>	the interrupt configuration

```
void SDDF_ConfigChannelLimitCheck (SDDF_HandleType * pHandle, uint8_t u8Channel,  
SDDF_LimitCheckConfigType * pConfig)
```

Configure the limit check for the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to configure
<i>pConfig</i>	the limit check configuration

```
void SDDF_ConfigChannelShortCircuitDetect (SDDF_HandleType * pHandle, uint8_t u8Channel,  
SDDF_ShortCircuitDetectConfigType * pConfig)
```

Configure the short circuit detect (SCD) for the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to configure
<i>pConfig</i>	the short circuit detect configuration

```
void SDDF_ConfigChannelZeroCrossDetect (SDDF_HandleType * pHandle, uint8_t u8Channel,  
SDDF_ZeroCrossDetectConfigType * pConfig)
```

Configure the zero cross detect (ZCD) for the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to configure
<i>pConfig</i>	the zero cross detect configuration

```
void SDDF_ConfigClockOut (SDDF_HandleType * pHandle, const SDDF_ClockOutConfigType *  
pClockOutCfg)
```

Configure the SDDF clock out.

Parameters

<i>elInstance</i>	the SDDF instance to configure
<i>pClockOutCfg</i>	the configuration of the SDDF clock out

void SDDF_DeInit (SDDF_HandleType * *pHandle*)

De-initialize the SDDF instance.

Parameters

<i>elInstance</i>	the SDDF instance to de-initialize
-------------------	------------------------------------

void SDDF_Disable (SDDF_HandleType * *pHandle*)

Disable the SDDF instance.

Parameters

<i>elInstance</i>	the SDDF instance to disable
-------------------	------------------------------

void SDDF_DisableDebugMode (SDDF_HandleType * *pHandle*)

Disable the debug mode of the SDDF instance.

Note

This function shall be called after the SDDF instance is enabled

Parameters

<i>elInstance</i>	the SDDF instance to disable debug mode
-------------------	---

void SDDF_Enable (SDDF_HandleType * *pHandle*)

Enable the SDDF instance.

Parameters

<i>elInstance</i>	the SDDF instance to enable
-------------------	-----------------------------

void SDDF_EnableDebugMode (SDDF_HandleType * *pHandle*, SDDF_DebugModuleEnableType *eModuleEnable*)

Enable the debug mode of the SDDF instance.

Note

This function shall be called after the SDDF instance is enabled

Parameters

<i>eInstance</i>	the SDDF instance to enable debug mode
<i>eModuleEnable</i>	whether the SDDF instance is enabled during debug mode

void SDDF_GenerateChannelSwTrigger (SDDF_HandleType * *pHandle*, uint8_t *u8Channel*)

Generate software trigger for the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to generate software trigger

uint32_t SDDF_GetChannelFIFOData (SDDF_HandleType * *pHandle*, uint8_t *u8Channel*)

Get the channel data for the SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to get result

Returns

uint32_t the conversion result of the SDDF channel

void SDDF_Init (SDDF_HandleType * *pHandle*, const SDDF_InitConfigType * *pInitCfg*)

Initialize the SDDF instance.

Parameters

<i>eInstance</i>	the SDDF instance to initialize
<i>pInitCfg</i>	the configuration of the SDDF instance

```
void SDDF_InitChannel (SDDF_HandleType * pHandle, uint8_t u8Channel, const  
SDDF_ChannelConfigType * pChannelCfg)
```

Initialize the selected SDDF channel.

Parameters

<i>eInstance</i>	the SDDF instance to use
<i>u8Channel</i>	the SDDF channel to configure
<i>pChannelCfg</i>	the configuration of the SDDF channel

```
void SDDFn_CHnDMAHandler (SDDF_HandleType * pHandle, uint8_t u8Channel)
```

```
void SDDFn_IRQHandler (SDDF_HandleType * pHandle)
```

Module_driver_sec

Collaboration diagram for Module_driver_sec:Modules

HwA_sec

Classes

struct **ReEnDebug_Keys**

Debug Re-Enable Key.

struct **NVR_Per**

NVR write and read Permissions. .

struct **SEC_IspInfo**

Indicate the ISP information.

struct **SEC_BootInfo**

Indicate the boot information.

struct **HostUKPermission**

Indicate the Host User Key Read/Write/Erase Protection.

struct **HostNVRPermission**

Indicate the Host NVR Read/Write/Erase Protection.

struct **Hsm_configuration**

Indicate the HSM configuration.

Enumerations

enum **SEC_RetType** { **SEC_STATUS_SUCCESS** = 0U, **SEC_STATUS_FAILED** = 1U }

enum **Systemstate** { **Securedstate** = 0u, **UnSecuredstate** = 1u }

Indicate the system state.

enum **Isp_Instance** { **ISP_FCUART1** = 0U, **ISP_FCUART3** = 1U, **ISP_FLEXCAN1** = 2U, **ISP_FLEXCAN5** = 3U }

Indicate the ISP instance.

enum **FCUART_ISP_BAUDRATE** { **ISP_FCUART_2MBPS** = 0u, **ISP_FCUART_1MBPS** = 1u,
ISP_FCUART_512KBPS = 2u, **ISP_FCUART_115200BPS** = 3u, **ISP_FCUART_INVALID** = 4u }

Indicate the FCUART Baud Rate for ISP.

enum **FLEXCAN_ISP_BAUDRATE** { **ISP_FLEXCANFD_4MBPS** = 0u, **ISP_FLEXCANFD_2MBPS** = 1u,
ISP_FLEXCAN_1MBPS = 2u, **ISP_FLEXCAN_500KBPS** = 3u, **ISP_FLEXCAN_INVALID** = 4u }

Indicate the FLEXCAN Baud Rate for ISP.

enum **FastBoot_Speed** { **FASTBOOT_300MHZ** = 0u, **FASTBOOT_96MHZ** = 1u }

Indicate the fast boot speed.

enum **Osc_FreqInfo** { **OSC_8MHZ** = 0u, **OSC_12MHZ** = 1u, **OSC_16MHZ** = 2u, **OSC_24MHZ** = 3u, **OSC_25MHZ**
= 4u, **OSC_32MHZ** = 5u, **OSC_48MHZ** = 6u, **OSC_40MHZ** = 7u }

Indicate the OSC Frequency.

enum **SC_LifeCycle** { **LIFECYCLE_INVALID** = 0U, **LIFECYCLE_OEM_DEV** = 0X1U, **LIFECYCLE_OEM_PDT** =
0X2U, **LIFECYCLE_INFIELD** = 0X4U, **LIFECYCLE_PREFA** = 0X8U, **LIFECYCLE_FA** = 0X10U }

Indicate the lifecycle of the chip.

enum **BL_VerifyAlgorithm** { **BL_ALGORITHM_CMACE** = 0U, **BL_ALGORITHM_ECC256_SHA256** = 1U,
BL_ALGORITHM_SM2_SM3 = 7U }

Indicate the Bootloader Verification Algorithm.

enum **Decryption_Algorithm** { **DECRYPTTION_ECC256** = 0U, **DECRYPTTION_SM2** = 1U }

Indicate the Debug/ISP/PREFA Authentication and USRK decryption algorithm.

Functions

bool **SEC_EnDebugMode** (void)

This function can be used to enable the Debug mode.

bool **SEC_ReEnDebugMode** (**ReEnDebug_Keys** keys)

This function can be used to re-enable the Debug mode by the re-enable keys.

Systemstate **SEC_SystemSecureState** (void)

This function can get the system state (secured or no secured).

bool **SEC_EnTestMode** (void)

This function can enable the test mode.

bool **SEC_ReEnTestMode** (uint32_t key)

This function can Re-enable the test mode by the re-enable key.

bool **SEC_SetNvrPer** (**NVR_Per** per)

This function can used to set the NVR write and read permission.

bool **SEC_GetSecureStatus** (void)

Get Secure Boot Enable Status.

bool **SEC_GetISPModeStatus** (void)

Get ISP Status.

uint32_t **SEC_GetBootAddress** (void)

Get User setting bootloader address.

uint32_t **SEC_GetHsmFwAddress** (void)

Get the HSM Firmware Address.

SC_LifeCycle **SEC_GetLifeCycle** (void)

Get the lifecycle.

bool **SEC_HostUKAccess** (void)

User Key Access Enable. Only valid under non-secure boot.

bool **SEC_GetHostFwDisAccess** (void)

Firmware Read Disable/Enable. (Value loaded from NVR sector)

void **SEC_DisbleFwReadAccess** (bool readaccess)

Set the Firmware Read access.

bool **SEC_GetFwValid** (void)

Get whether Flash Firmware is valid.

BL_VerifyAlgorithm SEC_GetBLVerifyAlgorithm (void)

get the Bootloader Verification Algorithm

Decryption_Algorithm SEC_GetDecryptAlgorithm (void)

get the Debug/ISP/PREFA Authentication and USRK decryption algorithm

void **SEC_GetHostUKAccess** (**HostUKPermission** *const PHostUKAccess)

Indicate the Host User Key Read/Write/Erase Protection.

void **SEC_GetHostNVRAccess** (**HostNVRPermission** *const PHostNvrAccess)

Indicate the Host NVR Read/Write/Erase Protection.

bool **SEC_GethsmUKEraseAccess** (void)

Get the HSM User Key Erase Protection.

bool **SEC_GethsmNVREraseAccess** (void)

Get the HSM NVR Erase Protection.

uint32_t **SEC_GetBLVerMask** (void)

Get Bootloader Verification Mask.

bool **SEC_GetDMBDkeyEn** (void)

Get whether Debug Mailbox Backdoor Key Enable.

bool **SEC_GetDebugAuthEn** (void)

Host Debug Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

SEC_RetType SEC_GetIspInfo (**SEC_IspInfo** *const pIspInfo)

Get the isp information.

SEC_RetType SEC_GetBootInfo (**SEC_BootInfo** *const pBootInfo)

Get the boot information.

SEC_RetType SEC_GetHSMInfo (**Hsm_configuration** *const pHsmInfo)

Get the HSM information.

Detailed Description

Enumeration Type Documentation

enum BL_VerifyAlgorithm

Indicate the Bootloader Verification Algorithm.

Enumerator:

BL_ALGORITHM _CMAC	the Bootloader Verification Algorithm is CMAC
BL_ALGORITHM _ECC256_SHA256	the Bootloader Verification Algorithm is ECC256/SHA256
BL_ALGORITHM _SM2_SM3	the Bootloader Verification Algorithm is SM2/SM3

enum Decryption_Algorithm

Indicate the Debug/ISP/PREFA Authentication and USRK decryption algorithm.

Enumerator:

DECRYPTTION_ ECC256	the Debug/ISP/PREFA Authentication and USRK decryption algorithm is ecc256
DECRYPTTION_ SM2	the Debug/ISP/PREFA Authentication and USRK decryption algorithm is sm2

enum FastBoot_Speed

Indicate the fast boot speed.

Enumerator:

FASTBOOT_300 MHZ	Select PLL0 as the core clock source; the core clock is 300MHz.
FASTBOOT_96M HZ	Select FIRC as the core clock source; the core clock is 96MHz.

enum FCUART_ISP_BAUDRATE

Indicate the FCUART Baud Rate for ISP.

Enumerator:

ISP_FCUART_2 MBPS	the FCUART Baud Rate is 2Mbps
----------------------	-------------------------------

ISP_FCUART_1 MBPS	the FCUART Baud Rate is 1Mbps
ISP_FCUART_51 2KBPS	the FCUART Baud Rate is 512kbps
ISP_FCUART_11 5200BPS	the FCUART Baud Rate is 115200bps
ISP_FCUART_IN VALID	the ISP instance is not FCUART

enum FLEXCAN_ISP_BAUDRATE

Indicate the FLEXCAN Baud Rate for ISP.

Enumerator:

ISP_FLEXCANFD _4MBPS	the CANFD Baud Rate is 4Mbps
ISP_FLEXCANFD _2MBPS	the CANFD Baud Rate is 2Mbps
ISP_FLEXCAN_1 MBPS	the CAN Baud Rate is 1Mbps
ISP_FLEXCAN_5 00KBPS	the FCUART Baud Rate is 500kbps
ISP_FLEXCAN_I NVALID	the ISP instance is not FLEXCAN

enum Isp_Instance

Indicate the ISP instance.

Enumerator:

ISP_FCUART1	The ISP instance is FCUART1
ISP_FCUART3	The ISP instance is FCUART3
ISP_FLEXCAN1	The ISP instance is FLEXCAN1
ISP_FLEXCAN5	The ISP instance is FLEXCAN3

enum Osc_FreqInfo

Indicate the OSC Frequency.

Enumerator:

OSC_8MHZ	OSC Frequency is 8 MHZ
OSC_12MHZ	OSC Frequency is 12 MHZ
OSC_16MHZ	OSC Frequency is 16 MHZ
OSC_24MHZ	OSC Frequency is 24 MHZ
OSC_25MHZ	OSC Frequency is 25 MHZ
OSC_32MHZ	OSC Frequency is 32 MHZ
OSC_48MHZ	OSC Frequency is 48 MHZ
OSC_40MHZ	OSC Frequency is 40 MHZ

enum SC_LifeCycle

Indicate the lifecycle of the chip.

Enumerator:

LIFECYCLE_INVALID	no Lifecycle
LIFECYCLE_OEM_DEV	Lifecycle OEM Development
LIFECYCLE_OEM_PDT	Lifecycle OEM Production
LIFECYCLE_INFIELD	Lifecycle In Field
LIFECYCLE_PREFA	Lifecycle Pre Fault Analysis
LIFECYCLE_FA	Lifecycle Analysis

enum SEC_RetType

Enumerator:

SEC_STATUS_SUCCESS	SEC status success
SEC_STATUS_FAILED	SEC status failed

enum Systemstate

Indicate the system state.

Enumerator:

Securedstate	The system is in secured state
UnSecuredstate	The system is in no secured state

Function Documentation

void SEC_DisbleFwReadAccess (bool *readaccess*)

Set the Firmware Read access.

Parameters

<i>readaccess</i>	true means disable the Host core read access to firmware read.
-------------------	--

bool SEC_EnDebugMode (void)

This function can be used to enable the Debug mode.

Returns

true means enable debug success ,false means enable debug failed.

bool SEC_EnTestMode (void)

This function can enable the test mode.

Returns

true means enable test mode success,false means enable test mode failed.

BL_VerifyAlgorithm SEC_GetBLVerifyAlgorithm (void)

get the Bootloader Verification Algorithm

Returns

the Bootloader Verification Algorithm

uint32_t SEC_GetBLVerMask (void)

Get Bootloader Verification Mask.

Returns

Bootloader Verification Mask.

uint32_t SEC_GetBootAddress (void)

Get User setting bootloader address.

Returns

bootloader address

SEC_RetType SEC_GetBootInfo (SEC_BootInfo *const *pBootInfo*)

Get the boot information.

Parameters

<i>pBootInfo</i>	the structure for information.
------------------	--------------------------------

Returns

Get whether operation is success.

bool SEC_GetDebugAuthEn (void)

Host Debug Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

Returns

true means Host debug authentication enable. false means Host debug authentication disable.

Decryption_Algorithm SEC_GetDecryptAlgorithm (void)

get the Debug/ISP/PREFA Authentication and USRK decryption algorithm

Returns

decryption algorithm

bool SEC_GetDMBDkeyEn (void)

Get whether Debug Mailbox Backdoor Key Enable.

Returns

true means user can enable debug by writing debug backdoor key, false means user can not enable debug

bool SEC_GetFwValid (void)

Get whether Flash Firmware is valid.

Returns

true means Flash Firmware is valid. false means Flash Firmware is invalid

bool SEC_GetHostFwDisAccess (void)

Firmware Read Disable/Enable. (Value loaded from NVR sector)

Returns

true means Host core read access to firmware read is disabled, false means - Host core read access to firmware read is enabled.

void SEC_GetHostNVRAccess (HostNVRPermission *const *PHostNvrAccess*)

Indicate the Host NVR Read/Write/Erase Protection.

Parameters

<i>PHostNvrAccess</i>	the structure for information
-----------------------	-------------------------------

void SEC_GetHostUKAccess (HostUKPermission *const *PHostUKAccess*)

Indicate the Host User Key Read/Write/Erase Protection.

Parameters

<i>PHostUKAccess</i>	the structure for information
----------------------	-------------------------------

uint32_t SEC_GetHsmFwAddress (void)

Get the HSM Firmware Address.

Returns

the HSM Firmware address

SEC_RetType SEC_GetHSMInfo (Hsm_configuration *const *pHsmInfo*)

Get the HSM information.

Parameters

<i>pHsmInfo</i>	the structure for information
-----------------	-------------------------------

Returns

Get whether operation is success.

bool SEC_GethsmNVREraseAccess (void)

Get the HSM NVR Erase Protection.

Returns

true - HSM erase access to NVR region is enabled false -HSM erase access to NVR region is disabled

bool SEC_GethsmUKEraseAccess (void)

Get the HSM User Key Erase Protection.

Returns

true - HSM erase access to User Key region is enabled false -HSM erase access to User Key region is disabled

SEC_RetType SEC_GetIspInfo (SEC_IspInfo *const *plspInfo*)

Get the isp information.

Parameters

<i>plspInfo</i>	the structure for information
-----------------	-------------------------------

Returns

Get whether operation is success.

bool SEC_GetISPModeStatus (void)

Get ISP Status.

Returns

true is active and false is inactive

SC_LifeCycle SEC_GetLifeCycle (void)

Get the lifecycle.

Returns

the lifecycle.

bool SEC_GetSecureStatus (void)

Get Secure Boot Enable Status.

Returns

true is enable and false is disable

bool SEC_HostUKAccess (void)

User Key Access Enable. Only valid under non-secure boot.

Returns

true means User key can be read/programmed/erased by host CPU

bool SEC_ReEnDebugMode (ReEnDebug_Keys *keys*)

This function can be used to re-enable the Debug mode by the re-enable keys.

Parameters

<i>keys</i>	The Debug mode re-enable keys.The key is up to the user to decide in advance.
-------------	---

Returns

true means re-enable success,false means re-enable failed.

bool SEC_ReEnTestMode (uint32_t *key*)

This function can Re-enable the test mode by the re-enable key.

Parameters

<i>key</i>	The test mode re-enable key.
------------	------------------------------

Returns

true means re-enable success,false means re-enable failed.

bool SEC_SetNvrPer (NVR_Per *per*)

This function can used to set the NVR write and read permission.

Returns

true means setPer success,false means setPer failed.

Systemstate SEC_SystemSecureState (void)

This function can get the system state (secured or no secured).

Module_driver_sent

Collaboration diagram for Module_driver_sent:Modules

HwA_sent

Classes

struct **Sent_ChannelInterruptType**

Structure to enable or disable the channel interrupts.

struct **Sent_FastMessageDataType**

Structure for fast message buffer.

struct **Sent_SlowMessageDataType**

Structure for slow message buffer.

struct **Sent_ConfigType**

structure to configure the SENT.

struct **Sent_ChannelConfigType**

structure to configure the SENT channel.

struct **Sent_DMAConfigType**

structure to configure the SENT channel DMA.

struct **Sent_SpcConfigType**

structure to configure the SENT channel SPC.

struct **_Sent_HandleType**

The structure of the SENT processing handle.

Typedefs

```
typedef void(* SENT_GBISRCallbackType) (Sent_ChannelType eChannel, const  
Sent_GlobalInterruptType eInterrupt)  
SENT Global ISR callback function prototype.
```

```
typedef void(* SENT_CHISRCallbackType) (Sent_ChannelType eChannel, const uint32_t  
u32ChannelStatus)  
SENT Channel ISR callback function prototype.
```

```
typedef struct _Sent_HandleType Sent_HandleType  
The structure of the SENT processing handle.
```

Enumerations

```
enum Sent_ReturnType { SENT_RETURN_OK = 0x00U, SENT_RETURN_E_NOT_OK = 0x01U,  
SENT_RETURN_E_ALREADY_INIT = 0x02U, SENT_RETURN_E_UNINIT = 0x03U, SENT_RETURN_E_PARAM  
= 0x04U }
```

SENT operation return values.

```
enum Sent_ChannelType { SENT_CHANNEL_0 = 0x00U, SENT_CHANNEL_1 = 0x01U, SENT_CHANNEL_2 =  
0x02U, SENT_CHANNEL_3 = 0x03U, SENT_CHANNEL_MAX = SENT_CHANNEL_COUNT }
```

SENT Channel index.

```
enum Sent_InstanceType { SENT_INSTANCE_0 = 0x00U, SENT_INSTANCE_1 = 0x01U,  
SENT_INSTANCE_MAX = SENT_INSTANCE_COUNT }
```

SENT instance index.

```
enum Sent_DataNibbleModeType { SENT_DATA_NIBBLE_MODE_A = 0x0U,  
SENT_DATA_NIBBLE_MODE_H1 = 0x1U, SENT_DATA_NIBBLE_MODE_H2 = 0x2U,  
SENT_DATA_NIBBLE_MODE_H3 = 0x3U, SENT_DATA_NIBBLE_MODE_H4 = 0x4U,  
SENT_DATA_NIBBLE_MODE_H5 = 0x5U, SENT_DATA_NIBBLE_MODE_H6 = 0x6U,  
SENT_DATA_NIBBLE_MODE_H7 = 0x7U }
```

SENT nibble data mode control. Refer to SENT SAE J2716 2010 for detail protocol definition.

```
enum Sent_CalibrationValidType { SENT_CALIBRATION_VALID_DISABLE = 0x0U,  
SENT_CALIBRATION_VALID_WITHIN_20 = 0x1U, SENT_CALIBRATION_VALID_FROM_20_TO_25 = 0x2U }
```

SENT calibration valid type.

```
enum Sent_CalDiagOptionType { SENT_CALIBRATION_PULSE_DIAG_OPTION2 = 0x0U,  
SENT_CALIBRATION_PULSE_DIAG_OPTION1 = 0x1U }
```

SENT Successive calibration pulses diagnostic option.

```
enum Sent_IdleCountType { SENT_IDLE_COUNT_FLAG_DISABLE = 0x0U,  
SENT_IDLE_COUNT_FLAG_254_TICKS = 0x1U, SENT_IDLE_COUNT_FLAG_508_TICKS = 0x2U,  
SENT_IDLE_COUNT_FLAG_1016_TICKS = 0x4U, SENT_IDLE_COUNT_FLAG_2032_TICKS = 0x8U }
```

Determines how long the bus idle flag will assert when SENT bus is idle.

```
enum Sent_SerialMessageType { SENT_SERIAL_MESSAGE_SHORT = 0x0U,  
SENT_SERIAL_MESSAGE_ENHANCE_12DATA_8ID = 0x1U,  
SENT_SERIAL_MESSAGE_ENHANCE_16DATA_4ID = 0x2U }
```

SENT Slow message type.

```
enum Sent_SpcTriggerType { SENT_SPC_SOFTWARE_TRIGGER = 0x0U, SENT_SPC_EXTERNAL_TRIGGER  
= 0x1U }
```

SENT trigger type in SPC mode.

```
enum Sent_SpcTickBaseType { SENT_SPC_TICK_BASE_PRE_MSG = 0x0U,  
SENT_SPC_TICK_BASE_CONFIGURED = 0x1U }
```

Select the tick base in SPC mode.

```
enum Sent_GlobalInterruptType { SENT_SLOW_MSG_DMA_UF_IT = 0U, SENT_FAST_MSG_DMA_DF_IT =  
1U, SENT_FAST_MSG_READY_IT = 2U, SENT_SLOW_MSG_READY_IT = 3U, SENT_FAST_MSG_FIFO_OF_IT  
= 4U }
```

Sent global interrupt type.

Functions

```
void SENT_InterruptProcess (const Sent_HandleType *pSentHandle)
```

Process sent interrupt.

```
void SENT_Init (Sent_HandleType *pSentHandle, const Sent_ConfigType *ConfigPtr)
```

Initialize SENT configuration.

```
void SENT_SetChannelConfig (const Sent_HandleType *pSentHandle, const Sent_ChannelType  
eChannel, const Sent_ChannelConfigType *pCfg)
```

Set SENT channel configuration.

```
void SENT_SetDMAConfig (const Sent_HandleType *pSentHandle, const Sent_ChannelType eChannel,  
Sent_DMAConfigType *pCfg)
```

Set SENT DMA configuration.

void **SENT_SetSPCConfig** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, const **Sent_SpcConfigType** *pCfg)

Set SENT SPC configuration.

void **SENT_GetFastMessageData** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, **Sent_FastMessageDataType** *pMsg)

Get Fast message data.

void **SENT_GetSlowMessageData** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, **Sent_SlowMessageDataType** *pMsg)

Get Slow message data.

void **SENT_DecomposeFastMessageWithDMABuffer** (uint32_t *pDmaBuffer, uint32_t u32BufferLength, **Sent_FastMessageDataType** *pMsg)

Decode the fast message with DMA buffer.

void **SENT_DecomposeSlowMessageWithDMABuffer** (uint32_t *pDmaBuffer, uint32_t u32BufferLength, **Sent_SlowMessageDataType** *pMsg)

Decode the slow message with DMA buffer.

void **SENT_EnableGlobalFastMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Enable the fast message ready interrupt.

void **SENT_DisableGlobalFastMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Disable the fast message ready interrupt.

void **SENT_EnableGlobalSlowMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Enable the slow message ready interrupt.

void **SENT_DisableGlobalSlowMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Disable the slow message ready interrupt.

void **SENT_EnableGlobalFastFifoMsgNotification** (const **Sent_HandleType** *pSentHandle)

Enable the fast message fifo interrupt.

void **SENT_DisableGlobalFastFifoMsgNotification** (const **Sent_HandleType** *pSentHandle)

Disable the fast message fifo interrupt.

void **SENT_ConfigChannelStatusNotification** (const **Sent_HandleType** *pSentHandle, const

Sent_ChannelType eChannel, const **Sent_ChannelInterruptType** *sInterruptEn)

configure the channel interrupt.

void **SENT_RequestSPCPulse** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType**

eChannel)

Request the SPC pulse.

void **SENT_GetFastMsgReadyFlag** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType**

eChannel, bool *pFlag)

Get the fast message ready flag.

void **SENT_GetSlowMsgReadyFlag** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType**

eChannel, bool *pFlag)

Get the slow message ready flag.

void **SENT_GetChannelStatus** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType**

eChannel, uint32_t *pStatus)

Get the channel status.

void **SENT_ClearChannelStatus** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType**

eChannel, uint32_t Status)

Clear the channel status.

void **SENT_GetChannelSPCBusyFlag** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType**

eChannel, bool *pflag)

Get the channel SPC busy flag.

SENT API Service IDs

Defines the service IDs for the FCUART module, which are used to identify different functions or services provided by the SENT layer. These IDs are utilized when communicating with the SENT module, enabling calls to or responses from various functions.

```
#define SENT_INIT_ID 0U
#define SENT_SET_CHANNEL_CONFIG_ID 1U
#define SENT_SET_DMA_CONFIG_ID 2U
#define SENT_SET_SPC_CONFIG_ID 3U
#define SENT_GET_FAST_MSG_ID 4U
#define SENT_GET_SLOW_MSG_ID 5U
#define SENT_DECODE_FAST_MSG_ID 6U
#define SENT_DECODE_SLOW_MSG_ID 7U
#define SENT_ENABLE_FAST_MSG_NOTIFY_ID 8U
#define SENT_DISABLE_FAST_MSG_NOTIFY_ID 9U
#define SENT_ENABLE_SLOW_MSG_NOTIFY_ID 10U
#define SENT_DISABLE_SLOW_MSG_NOTIFY_ID 11U
#define SENT_ENABLE_FAST_FIFO_MSG_NOTIFY_ID 12U
#define SENT_DISABLE_FAST_FIFO_MSG_NOTIFY_ID 13U
#define SENT_CONFIG_CHANNEL_STATUS_NOTIFY_ID 14U
#define SENT_REQUEST_SPC_PULSE_ID 15U
#define SENT_GET_FAST_MSG_READY_FLAG_ID 16U
#define SENT_GET_SLOW_MSG_READY_FLAG_ID 17U
#define SENT_GET_CHANNEL_STATUS_ID 18U
#define SENT_CLEAR_CHANNEL_STATUS_ID 29U
#define SENT_GET_SPC_BUSY_FLAG_ID 20U
```

SENT Dev Error Code

Error Code of calling SENT apis

```
#define SENT_E_PARAM_POINTER 0x01U
#define SENT_E_ALREADY_INIT 0x02U
#define SENT_E_UNINIT 0x03U
#define SENT_E_PARAM_INSTANCE 0x04U
#define SENT_E_PARAM_CHANNEL 0x05U
#define SENT_E_PARAM_GENERAL 0x06U
#define SENT_E_INVALID_BRANCH 0x07U
```

SENT buffer address and size in DMA mode

```
#define SENT0_GET_FAST_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DFD))
#define SENT1_GET_FAST_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DFD))
#define SENT_GET_FAST_MSG_DMA_BYTELENGTH (12U)
#define SENT0_GET_SLOW_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DSB3))
#define SENT1_GET_SLOW_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DSB3))
#define SENT_GET_SLOW_MSG_DMA_BYTELENGTH (12U)
```

Detailed Description

Macro Definition Documentation

#define SENT0_GET_FAST_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DFD))

#define SENT0_GET_SLOW_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DSB3))

#define SENT1_GET_FAST_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DFD))

#define SENT1_GET_SLOW_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DSB3))

#define SENT_CLEAR_CHANNEL_STATUS_ID 29U

link to "SENT_ClearChannelStatus" API.

#define SENT_CONFIG_CHANNEL_STATUS_NOITFY_ID 14U

link to "SENT_ConfigChannelStatusNotification" API.

#define SENT_DECODE_FAST_MSG_ID 6U

link to "SENT_DecodeFastMessageWithDMABuffer" API.

#define SENT_DECODE_SLOW_MSG_ID 7U

link to "SENT_DecodeSlowMessageWithDMABuffer" API.

#define SENT_DISABLE_FAST_FIFO_MSG_NOTIFY_ID 13U

link to "SENT_DisableGlobalFastFifoMsgNotification" API.

#define SENT_DISABLE_FAST_MSG_NOTIFY_ID 9U

link to "SENT_DisableGlobalFastMsgNotification" API.

#define SENT_DISABLE_SLOW_MSG_NOTIFY_ID 11U

link to "SENT_DisableGlobalSlowMsgNotification" API.

#define SENT_E_ALREADY_INIT 0x02U

The SENT has been initialized.

#define SENT_E_INVALID_BRANCH 0x07U

The SENT API enter the incorrect branch.

#define SENT_E_PARAM_CHANNEL 0x05U

The SENT channel number is out of range.

#define SENT_E_PARAM_GENERAL 0x06U

The SENT parameter is incorrect or out of range.

#define SENT_E_PARAM_INSTANCE 0x04U

The SENT instance number is out of range.

#define SENT_E_PARAM_POINTER 0x01U

The SENT pointer parameter is NULL.

#define SENT_E_UNINIT 0x03U

The SENT is not initialized.

#define SENT_ENABLE_FAST_FIFO_MSG_NOTIFY_ID 12U

link to "SENT_EnableGlobalFastFifoMsgNotification" API.

#define SENT_ENABLE_FAST_MSG_NOTIFY_ID 8U

link to "SENT_EnableGlobalFastMsgNotification" API.

#define SENT_ENABLE_SLOW_MSG_NOTIFY_ID 10U

link to "SENT_EnableGlobalSlowMsgNotification" API.

#define SENT_GET_CHANNEL_STATUS_ID 18U

link to "SENT_GetChannelStatus" API.

#define SENT_GET_FAST_MSG_DMA_BYTELENGTH (12U)

#define SENT_GET_FAST_MSG_ID 4U

link to "SENT_GetFastMessageData" API.

#define SENT_GET_FAST_MSG_READY_FLAG_ID 16U

link to "SENT_GetFastMsgReadyFlag" API.

#define SENT_GET_SLOW_MSG_DMA_BYTELENGTH (12U)

#define SENT_GET_SLOW_MSG_ID 5U

link to "SENT_GetSlowMessageData" API.

#define SENT_GET_SLOW_MSG_READY_FLAG_ID 17U

link to "SENT_GetSlowMsgReadyFlag" API.

#define SENT_GET_SPC_BUSY_FLAG_ID 20U

link to "SENT_GetChannelSPCBusyFlag" API.

#define SENT_INIT_ID 0U

link to "SENT_Init" API.

#define SENT_REQUEST_SPC_PULSE_ID 15U

link to "SENT_RequestSPCPulse" API.

#define SENT_SET_CHANNEL_CONFIG_ID 1U

link to "SENT_SetChannelConfig" API.

#define SENT_SET_DMA_CONFIG_ID 2U

link to "SENT_SetDMAConfig" API.

#define SENT_SET_SPC_CONFIG_ID 3U

link to "SENT_SetSPCConfig" API.

Typedef Documentation

typedef void(* SENT_CHISRCallbackType) (Sent_ChannelType eChannel, const uint32_t u32ChannelStatus)

SENT Channel ISR callback function prototype.

typedef void(* SENT_GBISRCallbackType) (Sent_ChannelType eChannel, const Sent_GlobalInterruptType eInterrupt)

SENT Global ISR callback function prototype.

typedef struct _Sent_HandleType Sent_HandleType

The structure of the SENT processing handle.

Enumeration Type Documentation

enum Sent_CalDiagOptionType

SENT Successive calibration pulses diagnostic option.

Enumerator:

SENT_CALIBRATION_PULSE_DIAG_OPTION2	Option 2 i.e. Low Latency Option as per SAE Specification
SENT_CALIBRATION_PULSE_DIAG_OPTION1	Option 1 i.e. Preferred but High Latency Option as per SAE Specification

enum Sent_CalibrationValidType

SENT calibration valid type.

Enumerator:

SENT_CALIBRATION_VALID_DISABLE	The successive calibration pulses diagnostic will be disabled
SENT_CALIBRATION_VALID_WITHIN_20	The difference of received calibration pulse and receiver configuration is within 20%, the message is deemed to be valid
SENT_CALIBRATION_VALID_FROM_20_TO_25	The difference of received calibration pulse and receiver configuration is more than 20% but less than 25%, the message is deemed to be valid

enum Sent_ChannelType

SENT Channel index.

Enumerator:

SENT_CHANNEL_0	The SENT channel 0
SENT_CHANNEL_1	The SENT channel 1
SENT_CHANNEL_2	The SENT channel 2
SENT_CHANNEL_3	The SENT channel 3
SENT_CHANNEL_MAX	

enum Sent_DataNibbleModeType

SENT nibble data mode control. Refer to SENT SAE J2716 2010 for detail protocol definition.

Enumerator:

SENT_DATA_NI BBLE_MODE_A	Frame format A
SENT_DATA_NI BBLE_MODE_H1	Frame format H1
SENT_DATA_NI BBLE_MODE_H2	Frame format H2
SENT_DATA_NI BBLE_MODE_H3	Frame format H3
SENT_DATA_NI BBLE_MODE_H4	Frame format H4
SENT_DATA_NI BBLE_MODE_H5	Frame format H5
SENT_DATA_NI BBLE_MODE_H6	Frame format H6
SENT_DATA_NI BBLE_MODE_H7	Frame format H7

enum Sent_GlobalInterruptType

Sent global interrupt type.

Enumerator:

SENT_SLOW_M SG_DMA_UF_IT	Slow message dma read underflow interrupt enable.
-----------------------------	---

SENT_FAST_MS G_DMA_DF_IT	Fast message dma read underflow interrupt enable.
SENT_FAST_MS G_READY_IT	Fast message ready interrupt enable.
SENT_SLOW_M SG_READY_IT	Slow message ready interrupt enable.
SENT_FAST_MS G_FIFO_OF_IT	Fast message FIFO overflow interrupt enable.

enum Sent_IdleCountType

Determines how long the bus idle flag will assert when SENT bus is idle.

Enumerator:

SENT_IDLE_CO UNT_FLAG_DIS ABLE	The bus idle flag will never assert
SENT_IDLE_CO UNT_FLAG_254 _TICKS	The bus is idle for more than 127*2 ticks, then the bus idle flag will assert
SENT_IDLE_CO UNT_FLAG_508 _TICKS	The bus is idle for more than 127*2 ticks, then the bus idle flag will assert
SENT_IDLE_CO UNT_FLAG_101 6_TICKS	The bus is idle for more than 127*2 ticks, then the bus idle flag will assert
SENT_IDLE_CO UNT_FLAG_203 2_TICKS	The bus is idle for more than 127*2 ticks, then the bus idle flag will assert

enum Sent_InstanceType

SENT instance index.

Enumerator:

SENT_INSTANC E_0	The SENT channel 0
SENT_INSTANC E_1	The SENT channel 1
SENT_INSTANC E_MAX	

enum Sent_ReturnType

SENT operation return values.

Enumerator:

SENT_RETURN_ OK	The SENT operation is succeeded
SENT_RETURN_ E_NOT_OK	The SENT operation is failed
SENT_RETURN_ E_ALREADY_INI T	The SENT has been initialized.
SENT_RETURN_ E_UNINIT	The SENT is not initialized
SENT_RETURN_ E_PARAM	The SENT parameter is incorrect or out of range.

enum Sent_SerialMessageType

SENT Slow message type.

Enumerator:

SENT_SERIAL_MESSAGE_SHORT	short serial data message
SENT_SERIAL_MESSAGE_ENHANCED_12DATA_8ID	enhanced serial data message with 12-bit data and 8-bit ID
SENT_SERIAL_MESSAGE_ENHANCED_16DATA_4ID	enhanced serial data message with 16-bit data and 4-bit ID

enum Sent_SpcTickBaseType

Select the tick base in SPC mode.

Enumerator:

SENT_SPC_TICK_BASE_PREMSG	Previous received message tick base
SENT_SPC_TICK_BASE_CONFIGURED	SENT configured tick base

enum Sent_SpcTriggerType

SENT trigger type in SPC mode.

Enumerator:

SENT_SPC_SOFTWARE_TRIGGER	SPC pulse triggered by software method
SENT_SPC_EXTERNAL_TRIGGER	SPC pulse triggered by external trigger

Function Documentation

void SENT_ClearChannelStatus (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*, uint32_t *Status*)

Clear the channel status.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>Status</i>	The channel status, the bits are used for W1c

Returns

void

void SENT_ConfigChannelStatusNotification (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*, const Sent_ChannelInterruptType * *sInterruptEn*)

configure the channel interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>sInterruptEn</i>	the structure of interrupt enable

Returns

void

void SENT_DecodeFastMessageWithDMABuffer (uint32_t * *pDmaBuffer*, uint32_t *u32BufferLength*, Sent_FastMessageDataType * *pMsg*)

Decode the fast message with DMA buffer.

Parameters

<i>pDmaBuffer</i>	The pointer for DMA buffer. the buffer length should be 12bytes.
<i>u32BufferLengt h</i>	The byte length of dma buffer.
<i>pMsg</i>	The fast message structure buffer.

Returns

void

**void SENT_DecodeSlowMessageWithDMABuffer (uint32_t * *pDmaBuffer*, uint32_t *u32BufferLength*,
Sent_SlowMessageDataType * *pMsg*)**

Decode the slow message with DMA buffer.

Parameters

<i>pDmaBuffer</i>	The pointer for DMA buffer. the buffer length should be 12bytes.
<i>u32BufferLengt h</i>	The byte length of dma buffer.
<i>pMsg</i>	The slow message structure buffer.

Returns

void

void SENT_DisableGlobalFastFifoMsgNotification (const Sent_HandleType * *pSentHandle*)

Disable the fast message fifo interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance

Returns

void

**void SENT_DisableGlobalFastMsgNotification (const Sent_HandleType * *pSentHandle*, const
Sent_ChannelType *eChannel*)**

Disable the fast message ready interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance

Returns

void

void SENT_DisableGlobalSlowMsgNotification (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*)

Disable the slow message ready interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance

Returns

void

void SENT_EnableGlobalFastFifoMsgNotification (const Sent_HandleType * *pSentHandle*)

Enable the fast message fifo interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance

Returns

void

void SENT_EnableGlobalFastMsgNotification (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*)

Enable the fast message ready interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance

Returns

void

void SENT_EnableGlobalSlowMsgNotification (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*)

Enable the slow message ready interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance

Returns

void

void SENT_GetChannelSPCBusyFlag (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*, bool * *pflag*)

Get the channel SPC busy flag.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>pflag</i>	The point of flag(true or false)

Returns

void

void SENT_GetChannelStatus (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*, uint32_t * *pStatus*)

Get the channel status.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>pStatus</i>	The point of channel status

Returns

void

void SENT_GetFastMessageData (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*, Sent_FastMessageType * *pMsg*)

Get Fast message data.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>pMsg</i>	The Fast message structure buffer.

Returns

void

void SENT_GetFastMsgReadyFlag (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*, bool * *pFlag*)

Get the fast message ready flag.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>pFlag</i>	The point of flag(true or false)

Returns

void

```
void SENT_GetSlowMessageData (const Sent_HandleType * pSentHandle, const Sent_ChannelType eChannel, Sent_SlowMessageType * pMsg)
```

Get Slow message data.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>pMsg</i>	The slow message structure buffer.

Returns

void

```
void SENT_GetSlowMsgReadyFlag (const Sent_HandleType * pSentHandle, const Sent_ChannelType eChannel, bool * pFlag)
```

Get the slow message ready flag.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>pFlag</i>	The point of flag(true or false)

Returns

void

```
void SENT_Init (Sent_HandleType * pSentHandle, const Sent_ConfigType * ConfigPtr)
```

Initialize SENT configuration.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>ConfigPtr</i>	The basic configurations of the SENT

Returns

void

void SENT_InterruptProcess (const Sent_HandleType * *pSentHandle*)

Process sent interrupt.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>ConfigPtr</i>	The basic configurations of the SENT

Returns

void

void SENT_RequestSPCPulse (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*)

Request the SPC pulse.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance

Returns

void

void SENT_SetChannelConfig (const Sent_HandleType * *pSentHandle*, const Sent_ChannelType *eChannel*, const Sent_ChannelConfigType * *pCfg*)

Set SENT channel configuration.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>ConfigPtr</i>	The channel configurations of the SENT

Returns

void

```
void SENT_SetDMAConfig (const Sent_HandleType * pSentHandle, const Sent_ChannelType eChannel,
Sent_DMAConfigType * pCfg)
```

Set SENT DMA configuration.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>ConfigPtr</i>	The channel configurations of the DMA

Returns

void

```
void SENT_SetSPCConfig (const Sent_HandleType * pSentHandle, const Sent_ChannelType eChannel,
const Sent_SpcConfigType * pCfg)
```

Set SENT SPC configuration.

Parameters

<i>pSentHandle</i>	The sent handle point
<i>eChannel</i>	The channel of the selected SENT instance
<i>ConfigPtr</i>	The channel configurations of the SPC

Returns

void

Module_driver_smc

Collaboration diagram for Module_driver_smc:Modules

HwA_smc

Macros

```
#define SMC_DEV_ERROR_REPORT STD_OFF
#define SMC_ReportDevError(func, error) ReportDevError(SMC_MODULE_ID, func, error)
#define SMC_AONCLK_128K 128000U
```

```
#define SMC_AONCLK_32K 32000U
#define SMC_AONCLK_SOSC_32K 32768U
#define SMC_AONCLK_1K 1000U
#define LP_WAKEUP_PAD_NUM 5U
```

Enumerations

```
enum SMC_StatusType { SMC_STATUS_SUCCESS = 0U, SMC_STATUS_PARAM_INVALID = 1U }
```

Smc return type.

```
enum SMC_ModeType { SMC_MODE_RUN = 0U, SMC_MODE_STOP, SMC_MODE_WAIT,
SMC_MODE_STANBY_0, SMC_MODE_STANBY_1, SMC_MODE_STANBY_2, SMC_MODE_STANBY_3 }
```

Smc mode.

Functions

```
void SMC_SetSystemMode (const SMC_ModeType eMode)
```

Set system mode.

SMC API Service IDs

```
#define SMC_SET_CLOCK_OUT_ID 0x00U
#define SMC_SET_AONCLKSRC_ID 0x01U
#define SMC_GET_CLOCK_FREQ_ID 0x02U
```

SMC Dev Error Code

Error Code of calling SMC apis

```
#define SMC_E_PARAM_POINTER 0x01U
#define SMC_E_PARAM_OUT_RANGE 0x02U
```

Detailed Description

Macro Definition Documentation

```
#define LP_WAKEUP_PAD_NUM 5U
```

```
#define SMC_AONCLK_128K 128000U
```

```
#define SMC_AONCLK_1K 1000U
```

```
#define SMC_AONCLK_32K 32000U
```

```
#define SMC_AONCLK_SOSC_32K 32768U
```

```
#define SMC_DEV_ERROR_REPORT STD_OFF
```

```
#define SMC_E_PARAM_OUT_RANGE 0x02U
```

```
#define SMC_E_PARAM_POINTER 0x01U
```

```
#define SMC_GET_CLOCK_FREQ_ID 0x02U
```

```
#define SMC_ReportDevError( func, error) ReportDevError(SMC_MODULE_ID, func, error)
```

```
#define SMC_SET_AONCLKSRC_ID 0x01U
```

```
#define SMC_SET_CLOCK_OUT_ID 0x00U
```

Enumeration Type Documentation

enum SMC_ModeType

Smc mode.

Enumerator:

SMC_MODE_RUN	The MCU can run at full speed and the internal supply is fully regulated, that is, in run regulation
SMC_MODE_SLEEP	Core enters sleep mode and no other clock is gated. Only the core clock is gated.

SMC_MODE_WAIT	Both core and bus clocks are gated off.
SMC_MODE_STANDBY_0	All clocks except some low-power clocks can keep on are gated off and the PD1 domain is powered off. please refer to reference manual for detail about standby mode 0.
SMC_MODE_STANDBY_1	All clocks except some low-power clocks can keep on are gated off and the PD1 domain is powered off. please refer to reference manual for detail about standby mode 1.
SMC_MODE_STANDBY_2	All clocks except some low-power clocks can keep on are gated off and the PD1 domain is powered off. please refer to reference manual for detail about standby mode 2.
SMC_MODE_STANDBY_3	All clocks except some low-power clocks can keep on are gated off and the PD1 domain is powered off. please refer to reference manual for detail about standby mode 3.

enum SMC_StatusType

Smc return type.

Enumerator:

SMC_STATUS_SUCCESS	
SMC_STATUS_PARAMETER_INVALID	

Function Documentation

void SMC_SetSystemMode (const SMC_ModeType *eMode*)

Set system mode.

Parameters

<i>eMode</i>	MCU low power mode
--------------	--------------------

Module_driver_tmu

Collaboration diagram for Module_driver_tmu:Modules

HwA_tmu

Classes

struct **_TMU_HandleType**

The structure of the TMU processing handle.

struct **TMU_InitType**

Defines the temperature sensor configuration.

Typedefs

typedef struct **_TMU_HandleType** **TMU_HandleType**

The structure of the TMU processing handle.

Enumerations

enum **TMU_InstanceType** { **TMU_INSTANCE_0** = 0U, **TMU_INSTANCE_MAX** }

TMU instance declaration.

enum **TMU_StatusType** { **TMU_STATUS_ERROR** = 0x0U, **TMU_STATUS_SUCCESS** = 0x1U,
TMU_STATUS_TIMEOUT = 0x2U }

TMU operation return values.

Functions

void **TMU_ProcessIRQHandler** (**TMU_HandleType** *pTmuHandle)

TMU interrupt process.

void **TMU_Init** (**TMU_HandleType** *pTmuHandle, const **TMU_InitType** *const plnitCfg)

Initialize the TMU instance.

TMU_StatusType TMU_FlagTempEnable (TMU_HandleType *pTmuHandle)

Enable the Flag-based temperature sensor.

TMU_StatusType TMU_FlagTempDisable (TMU_HandleType *pTmuHandle)

Disable the Flag-based temperature sensor.

TMU_StatusType TMU_VoltageTempEnable (TMU_HandleType *pTmuHandle)

Enable the Voltage-based temperature sensor.

TMU_StatusType TMU_VoltageTempDisable (TMU_HandleType *pTmuHandle)

Disable the Voltage-based temperature sensor.

void TMU_TempOverClear (TMU_HandleType *pTmuHandle)

Check if cleaning flag is required.

TMU API Service IDs

#define TMU_INIT_ID 0U

TMU Dev Error Code

Error Code of calling TMU apis

#define TMU_E_PARAM_INSTANCE 0x01U

#define TMU_E_PARAM_POINTER 0x02U

Detailed Description

Macro Definition Documentation

```
#define TMU_E_PARAM_INSTANCE 0x01U
```

```
#define TMU_E_PARAM_POINTER 0x02U
```

```
#define TMU_INIT_ID 0U
```

Typedef Documentation

```
typedef struct _TMU_HandleType TMU_HandleType
```

The structure of the TMU processing handle.

Enumeration Type Documentation

```
enum TMU_InstanceType
```

TMU instance declaration.

Enumerator:

TMU_INSTANCE _0	TMU instance 0 is selected
TMU_INSTANCE _MAX	

```
enum TMU_StatusType
```

TMU operation return values.

Enumerator:

TMU_STATUS_ERROR	The TMU operation is failed
TMU_STATUS_SUCCESS	The TMU operation is succeed
TMU_STATUS_TIMEOUT	The TMU operation is failed because of time out

Function Documentation

TMU_StatusType TMU_FlagTempDisable (TMU_HandleType * *pTmuHandle*)

Disable the Flag-based temperature sensor.

Parameters

<i>pTmuHandle</i>	TMU processing handle
-------------------	-----------------------

Returns

TMU_StatusType TMU_STATUS_SUCCESS when disable successfully, others fail

TMU_StatusType TMU_FlagTempEnable (TMU_HandleType * *pTmuHandle*)

Enable the Flag-based temperature sensor.

Parameters

<i>pTmuHandle</i>	TMU processing handle
-------------------	-----------------------

Returns

TMU_StatusType TMU_STATUS_SUCCESS when enable successfully, others fail

void TMU_Init (TMU_HandleType * *pTmuHandle*, const TMU_InitType *const *pInitCfg*)

Initialize the TMU instance.

Parameters

<i>pTmuHandle</i>	TMU processing handle
<i>pInitCfg</i>	the configurations of the TMU instance

void TMU_ProcessIRQHandler (TMU_HandleType * *pTmuHandle*)

TMU interrupt process.

void TMU_TempOverClear (TMU_HandleType * *pTmuHandle*)

Check if cleaning flag is required.

Parameters

<i>pTmuHandle</i>	TMU processing handle
-------------------	-----------------------

TMU_StatusType TMU_VoltageTempDisable (TMU_HandleType * *pTmuHandle*)

Disable the Voltage-based temperature sensor.

Parameters

<i>pTmuHandle</i>	TMU processing handle
-------------------	-----------------------

Returns

TMU_StatusType TMU_STATUS_SUCCESS when disable successfully, others fail

TMU_StatusType TMU_VoltageTempEnable (TMU_HandleType * *pTmuHandle*)

Enable the Voltage-based temperature sensor.

Parameters

<i>pTmuHandle</i>	TMU processing handle
-------------------	-----------------------

Returns

TMU_StatusType TMU_STATUS_SUCCESS when enable successfully, others fail

Module_driver_trgsel

Collaboration diagram for Module_driver_trgsel:Modules

HwA_trgsel

Typedefs

typedef uint32_t **TRGSEL_TargetType**

TrgSel trigger targets.

typedef uint32_t **TRGSEL_SourceType**

TrgSel trigger sources.

Enumerations

```
enum TRGSEL_SwTriggerChannelType { TRGSEL_SW_TRIGGER_CHANNEL_0 = 0U,  
TRGSEL_SW_TRIGGER_CHANNEL_1 = 1U, TRGSEL_SW_TRIGGER_CHANNEL_2 = 2U,  
TRGSEL_SW_TRIGGER_CHANNEL_3 = 3U, TRGSEL_SW_TRIGGER_CHANNEL_4 = 4U,  
TRGSEL_SW_TRIGGER_CHANNEL_5 = 5U, TRGSEL_SW_TRIGGER_CHANNEL_6 = 6U,  
TRGSEL_SW_TRIGGER_CHANNEL_7 = 7U }
```

The TrgSel eInstance to select.

Functions

TRGSEL_SourceType TRGSEL_GetTargetTriggerSource (const **TRGSEL_InstanceType** eInstance, const **TRGSEL_TargetType** eTarget)

Get the trigger source of the selected trigger eTarget.

void TRGSEL_SetTargetTriggerSource (const **TRGSEL_InstanceType** eInstance, const **TRGSEL_TargetType** eTarget, const **TRGSEL_SourceType** eSource)

Set the trigger source of the selected trigger target.

bool TRGSEL_GetTargetLockStatus (const **TRGSEL_InstanceType** eInstance, const **TRGSEL_TargetType** eTarget)

Get whether the trigger source of the selected target is locked.

void **TRGSEL_LockTargetTriggerSource** (const TRGSEL_InstanceType eInstance, const **TRGSEL_TargetType** eTarget)

Lock the trigger source of the selected target.

void **TRGSEL_GenerateSwTrigger** (const **TRGSEL_SwTriggerChannelType** eChannel)

Generate trigger signal for the selected software trigger source channel.

TRGSEL API Service IDs

```
#define TRGSEL_GET_TARGET_TRIGGER_SOURCE_ID 0U
#define TRGSEL_SET_TARGET_TRIGGER_SOURCE_ID 1U
#define TRGSEL_GET_TARGET_LOCK_STATUS_ID 2U
#define TRGSEL_LOCK_TARGET_TRIGGER_SOURCE_ID 3U
#define TRGSEL_GENERATE_SW_TRIGGER_ID 4U
```

TRGSEL Dev Error Code

Error Code of calling TRGSEL apis

```
#define TRGSEL_E_PARAM_INSTANCE_NUMBER 0x01U
#define TRGSEL_E_INVALID_TARGET 0x02U
#define TRGSEL_E_INVALID_SOURCE 0x03U
#define TRGSEL_E_TARGET_LOCKED 0x04U
#define TRGSEL_E_INVALID_SW_TRIGGIER 0x05U
```

Detailed Description

Macro Definition Documentation

#define TRGSEL_E_INVALID_SOURCE 0x03U

#define TRGSEL_E_INVALID_SW_TRIGGER 0x05U

#define TRGSEL_E_INVALID_TARGET 0x02U

#define TRGSEL_E_PARAM_INSTANCE_NUMBER 0x01U

#define TRGSEL_E_TARGET_LOCKED 0x04U

#define TRGSEL_GENERATE_SW_TRIGGER_ID 4U

#define TRGSEL_GET_TARGET_LOCK_STATUS_ID 2U

#define TRGSEL_GET_TARGET_TRIGGER_SOURCE_ID 0U

#define TRGSEL_LOCK_TARGET_TRIGGER_SOURCE_ID 3U

#define TRGSEL_SET_TARGET_TRIGGER_SOURCE_ID 1U

Typedef Documentation

typedef uint32_t TRGSEL_SourceType

TrgSel trigger sources.

typedef uint32_t TRGSEL_TargetType

TrgSel trigger targets.

Enumeration Type Documentation

enum TRGSEL_SwTriggerChannelType

The TrgSel eInstance to select.

Software trigger channels for SMISC software trigger channel 0~7

Enumerator:

TRGSEL_SW_TRIGGER_CHANNEL_0	
TRGSEL_SW_TRIGGER_CHANNEL_1	
TRGSEL_SW_TRIGGER_CHANNEL_2	
TRGSEL_SW_TRIGGER_CHANNEL_3	
TRGSEL_SW_TRIGGER_CHANNEL_4	
TRGSEL_SW_TRIGGER_CHANNEL_5	
TRGSEL_SW_TRIGGER_CHANNEL_6	
TRGSEL_SW_TRIGGER_CHANNEL_7	

Function Documentation

void TRGSEL_GenerateSwTrigger (const TRGSEL_SwTriggerChannelType *eChannel*)

Generate trigger signal for the selected software trigger source channel.

Parameters

<i>channel</i>	the selected software trigger source channel
----------------	--

bool TRGSEL_GetTargetLockStatus (const TRGSEL_InstanceType *eInstance*, const TRGSEL_TargetType *eTarget*)

Get whether the trigger source of the selected target is locked.

Parameters

<i>eInstance</i>	the TrgSel instance to which the trigger target belongs
<i>eTarget</i>	the trigger target to get the lock status

Returns

true the trigger source of the selected target cannot be modified

false the trigger source of the selected target can be modified

TRGSEL_SourceType TRGSEL_GetTargetTriggerSource (const TRGSEL_InstanceType *eInstance*, const TRGSEL_TargetType *eTarget*)

Get the trigger source of the selected trigger eTarget.

Parameters

<i>eInstance</i>	the TrgSel instance to which the trigger target belongs
<i>eTarget</i>	the trigger target to get the trigger source

Returns

TRGSEL_SourceType the trigger source of the selected trigger target

```
void TRGSEL_LockTargetTriggerSource (const TRGSEL_InstanceType  eInstance, const
TRGSEL_TargetType  eTarget)
```

Lock the trigger source of the selected target.

Note

The trigger target is grouped by four, so if you lock the trigger source of one target, the adjacent three trigger targets in the same register group are also be locked. So please ensure the trigger sources are not to be modified before lock the trigger target.

Parameters

<i>eInstance</i>	the TrgSel instance to which the trigger target belongs
<i>eTarget</i>	the trigger target to lock the trigger source

```
void TRGSEL_SetTargetTriggerSource (const TRGSEL_InstanceType  eInstance, const
TRGSEL_TargetType  eTarget, const TRGSEL_SourceType  eSource)
```

Set the trigger source of the selected trigger target.

Parameters

<i>eInstance</i>	the TrgSel instance to which the trigger target belongs
<i>eTarget</i>	the trigger target to set the trigger source
<i>eSource</i>	the selected trigger source to trig the target

Module_driver_tstmp

Collaboration diagram for Module_driver_tstmp:Modules

HwA_tstmp

Classes

```
struct _TSTMP_HandleType
```

The structure of the Tstmp processing handle.

```
struct TSTMP_IntType
```

TSTMP interrupt type.

struct **TSTMP_InitType**

TSTMP initialization type.

struct **TSTMP_UpdateType**

TSTMP update type.

Macros

#define **MAX_MOD_NUMBER** 4U

TSTMP max mod number.

Typedefs

typedef struct **_TSTMP_HandleType** **TSTMP_HandleType**

The structure of the Tstmp processing handle.

Enumerations

enum **TSTMP_StatusType** { **TSTMP_STATUS_SUCCESS** = 0U, **TSTMP_STATUS_PARAM_INVALID** = 1U }

TSTMP return type.

enum **TSTMP_InstanceType** { **TSTMP_INSTANCE_0** = 0U, **TSTMP_INSTANCE_1** = 1U,
TSTMP_INSTANCE_MAX }

TSTMP instance.

Functions

void **TSTMP_Init** (**TSTMP_HandleType** *pTstmpHandle, const **TSTMP_InitType** *const pInitStruct)

Initialize TSTMP instance.

void **TSTMP_Deinit** (**TSTMP_HandleType** *pTstmpHandle)

De-initialize TSTMP instance.

void **TSTMP_InitInterrupt** (**TSTMP_HandleType** *pTstmpHandle, const **TSTMP_IntType** *const pIntStruct)

Initialize TSTMP interrupt functionality.

void **TSTMP_EnableInterrupt** (**TSTMP_HandleType** *pTstmpHandle, const **TSTMP_ModChannelType**
eMod)

Enable TSTMP interrupt function.

```
void TSTMP_DisableInterrupt (TSTMP_HandleType *pTstmpHandle, const TSTMP_ModChannelType  
eMod)
```

Disable TSTMP interrupt function.

```
void TSTMP_GetTstmpValue (TSTMP_HandleType *pTstmpHandle, uint64_t *const u64TstmpValue)
```

Get TSTMP count value.

```
void TSTMP_UpdateMod (TSTMP_HandleType *pTstmpHandle, const TSTMP_UpdateType *const  
pUpdateStruct)
```

Update Modulate configuration.

```
void TSTMPn_IRQHandler (TSTMP_HandleType *pTstmpHandle)
```

TSTMP interrupt process function.

TSTMP API Service IDs

```
#define TSTMP_INIT_ID 0U  
#define TSTMP_DEINIT_ID 1U  
#define TSTMP_SET_COUNTER_MODE_ID 2U  
#define TSTMP_START_COUNTER_ID 3U  
#define TSTMP_STOP_COUNTER_ID 4U  
#define TSTMP_INIT_INTERRUPT_ID 5U  
#define TSTMP_EN_INTERRUPT_ID 6U  
#define TSTMP_DIS_INTERRUPT_ID 7U  
#define TSTMP_GET_TSTMP_VALUE_ID 8U  
#define TSTMP_UPDATE_MOD_ID 9U
```

TSTMP Dev Error Code

Error Code of calling TSTMP apis

```
#define TSTMP_E_PARAM_INSTANCE 0x01U  
#define TSTMP_E_PARAM_CHANNEL 0x02U  
#define TSTMP_E_PARAM_POINTER 0x03U  
#define TSTMP_E_PARAM_COUNT 0x04U  
#define TSTMP_E_PARAM_INVALID 0x05U
```

Detailed Description

Macro Definition Documentation

#define MAX_MOD_NUMBER 4U

TSTMP max mod number.

#define TSTMP_DEINIT_ID 1U

#define TSTMP_DIS_INTERRUPT_ID 7U

#define TSTMP_E_PARAM_CHANNEL 0x02U

#define TSTMP_E_PARAM_COUNT 0x04U

#define TSTMP_E_PARAM_INSTANCE 0x01U

#define TSTMP_E_PARAM_INVALID 0x05U

#define TSTMP_E_PARAM_POINTER 0x03U

#define TSTMP_EN_INTERRUPT_ID 6U

#define TSTMP_GET_TSTMP_VALUE_ID 8U

#define TSTMP_INIT_ID 0U

#define TSTMP_INIT_INTERRUPT_ID 5U

#define TSTMP_SET_COUNTER_MODE_ID 2U

#define TSTMP_START_COUNTER_ID 3U

#define TSTMP_STOP_COUNTER_ID 4U

#define TSTMP_UPDATE_MOD_ID 9U

Typedef Documentation

typedef struct _TSTMP_HandleType TSTMP_HandleType

The structure of the Tstmp processing handle.

Enumeration Type Documentation

enum TSTMP_InstanceType

TSTMP instance.

Enumerator:

TSTMP_INSTAN CE_0	
TSTMP_INSTAN CE_1	
TSTMP_INSTAN CE_MAX	

enum TSTMP_StatusType

TSTMP return type.

Enumerator:

TSTMP_STATUS _SUCCESS	
TSTMP_STATUS _PARAM_INVALID	

Function Documentation

void TSTMP_Deinit (TSTMP_HandleType * *pTstmpHandle*)

De-initialize TSTMP instance.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
---------------------	-------------------------

void TSTMP_DisableInterrupt (TSTMP_HandleType * *pTstmpHandle*, const TSTMP_ModChannelType *eMod*)

Disable TSTMP interrupt function.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
<i>eMod</i>	TSTMP modulate enumeration

void TSTMP_EnableInterrupt (TSTMP_HandleType * *pTstmpHandle*, const TSTMP_ModChannelType *eMod*)

Enable TSTMP interrupt function.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
<i>eMod</i>	TSTMP modulate enumeration

void TSTMP_GetTstmpValue (TSTMP_HandleType * *pTstmpHandle*, uint64_t *const *u64TstmpValue*)

Get TSTMP count value.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
<i>u64TstmpValue</i>	the timestamp value

void TSTMP_Init (TSTMP_HandleType * *pTstmpHandle*, const TSTMP_InitType *const *pInitStruct*)

Initialize TSTMP instance.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
<i>pInitStruct</i>	TSTMP initialization structure

Note

TSTMP will auto start the modulate when write MODn_SETVAL register TSTMP1 clock source is core clock

void TSTMP_InitInterrupt (TSTMP_HandleType * *pTstmpHandle*, const TSTMP_IntType *const *pIntStruct*)

Initialize TSTMP interrupt functionality.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
<i>pIntStruct</i>	TSTMP interrupt structure

void TSTMP_UpdateMod (TSTMP_HandleType * *pTstmpHandle*, const TSTMP_UpdateType *const *pUpdateStruct*)

Update Modulate configuration.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
<i>pUpdateStruct</i>	TSTMP update structure pointer

Returns

TSTMP_StatusType TSTMP return type

void TSTMPn_IRQHandler (TSTMP_HandleType * *pTstmpHandle*)

TSTMP interrupt process function.

Parameters

<i>pTstmpHandle</i>	TSTMP processing handle
---------------------	-------------------------

Module_driver_wdog

Collaboration diagram for Module_driver_wdog:Modules

HwA_wdog

Classes

struct **_WDOG_HandleType**

The structure of the WDOG processing handle.

struct **WDOG_CfgType**

WDOG init module type, include register setting for WDOG_CS/WDOG_TIMEOUT/WDOG_WINDOW.

Typedefs

typedef struct **_WDOG_HandleType** **WDOG_HandleType**

The structure of the WDOG processing handle.

Enumerations

enum **WDOG_StatusType** { **WDOG_STATUS_SUCCESS** = 0x0U, **WDOG_STATUS_ERROR** = 0x1U, **WDOG_STATUS_ERROR_UNLOCK** = 0x2U, **WDOG_STATUS_ERROR_RECFG** = 0x3U }

WDOG initialization return values.

enum **WDOG_InstanceType** { **WDOG_INSTANCE_0** = 0U, **WDOG_INSTANCE_1** = 1U, **WDOG_INSTANCE_2** = 2U, **WDOG_INSTANCE_3** = 3U, **WDOG_INSTANCE_MAX** }

WDOG clock instance declaration.

enum **WDOG_ClkSelType** { **WDOG_BUS_CLK** = 0x00U, **WDOG_AON_CLK** = 0x01U, **WDOG_FOSC_CLK** = 0x02U, **WDOG_SIRC_CLK** = 0x03U }

WDOG clock source select, mapping with WDOG_CS[CLK_SEL] bit filed definition.

enum **WDOG_TestType** { **WDOG_TST_DISABLED** = 0x00U, **WDOG_USER_MODE** = 0x01U, **WDOG_TST_MODE_LOW** = 0x02U, **WDOG_TST_MODE_HIGH** = 0x03U }

WDOG test mode, mapping with WDOG_CS[TST] bit filed definition.

```
enum WDOG_TimeoutReactionType { WDOG_REACTION_INT_128_CYCLE = 0x00U,  
WDOG_REACTION_INT_256_CYCLE = 0x01U, WDOG_REACTION_INT_512_CYCLE = 0x02U,  
WDOG_REACTION_INT_1024_CYCLE = 0x03U, WDOG_REACTION_NO_INT = 0x04U }
```

WDOG delay time to generate the reset.

Functions

```
void WDOG_Unlock (WDOG_HandleType *pWdogHandle)
```

unlock the wdog before Watch dog reconfigure set.

```
void WDOG_Refresh (WDOG_HandleType *pWdogHandle)
```

feed the watch dog by writing typical CMD to counter.

```
WDOG_StatusType WDOG_Init (WDOG_HandleType *pWdogHandle, const WDOG_CfgType *const  
pWdogCfg)
```

Initialize the WDOG configuration setting.

```
void WDOG_SelfTest (WDOG_HandleType *pWdogHandle, WDOG_TestType eMode)
```

Performs a self-test on the Watchdog Timer (WDOG) module.

```
void WDOG_DeInit (WDOG_HandleType *pWdogHandle)
```

De-initialize the WDOG instance.

```
void WDOG_IRQHandler (WDOG_HandleType *pWdogHandle)
```

Wdg n interrupt process.

WDOG API Service IDs

```
#define WDOG_INIT_ID 0U
```

WDOG Dev Error Code

Error Code of calling WDOG apis

```
#define WDOG_E_PARAM_INSTANCE 0x01U
```

```
#define WDOG_E_PARAM_TIMEOUT 0x02U
```

Detailed Description

Macro Definition Documentation

#define WDOG_E_PARAM_INSTANCE 0x01U

#define WDOG_E_PARAM_TIMEOUT 0x02U

#define WDOG_INIT_ID 0U

Typedef Documentation

typedef struct _WDOG_HandleType WDOG_HandleType

The structure of the WDOG processing handle.

Enumeration Type Documentation

enum WDOG_ClkSelType

WDOG clock source select, mapping with WDOG_CS[CLK_SEL] bit filed definition.

Enumerator:

WDOG_BUS_CLK	Bus clock
WDOG_AON_CLK	AON clock, if choose this clock need to configure AON clock in SMISC setting
WDOG_FOSC_CLK	FOSC clock

WDOG_SIRC_CLK	SIRC clock
---------------	------------

enum WDOG_InstanceType

WDOG clock instance declaration.

Enumerator:

WDOG_INSTANCE_0	WDOG instance 0 is selected
WDOG_INSTANCE_1	WDOG instance 1 is selected
WDOG_INSTANCE_2	WDOG instance 2 is selected
WDOG_INSTANCE_3	WDOG instance 3 is selected
WDOG_INSTANCE_MAX	WDOG instance MAX number

enum WDOG_StatusType

WDOG initialization return values.

Enumerator:

WDOG_STATUS_SUCCESS	The WDOG initialize is succeed
WDOG_STATUS_ERROR	The WDOG initialize is error
WDOG_STATUS_ERROR_UNLOCK	The WDOG initialize is error because unlock failed
WDOG_STATUS_ERROR_RECFG	The WDOG initialize is error because reconfigure failed

enum WDOG_TestType

WDOG test mode, mapping with WDOG_CS[TST] bit field definition.

Enumerator:

WDOG_TST_DISABLED	Test mode disabled
WDOG_USER_MODE	User mode enabled. (Test mode disabled.)
WDOG_TST_MODE_LOW	Test mode enabled, only the low byte is used.
WDOG_TST_MODE_HIGH	Test mode enabled, only the high byte is used.

enum WDOG_TimeoutReactionType

WDOG delay time to generate the reset.

Enumerator:

WDOG_REACTION_INT_128_CYCLE	Enable the timeout interrupt, and the delay to generate the reset is configured to 128 bus clock cycles
WDOG_REACTION_INT_256_CYCLE	Enable the timeout interrupt, and the delay to generate the reset is configured to 256 bus clock cycles
WDOG_REACTION_INT_512_CYCLE	Enable the timeout interrupt, and the delay to generate the reset is configured to 512 bus clock cycles
WDOG_REACTION_INT_1024_CYCLE	Enable the timeout interrupt, and the delay to generate the reset is configured to 1024 bus clock cycles
WDOG_REACTION_NO_INT	Disable the timeout interrupt, and generate the reset immediately

--	--

Function Documentation

void WDOG_DeInit (WDOG_HandleType * *pWdogHandle*)

De-initialize the WDOG instance.

Parameters

<i>pWdogHandle</i>	WDOG processing handle
--------------------	------------------------

WDOG_StatusType WDOG_Init (WDOG_HandleType * *pWdogHandle*, const WDOG_CfgType *const *pWdogCfg*)

Initialize the WDOG configuration setting.

Parameters

<i>pWdogHandle</i>	WDOG processing handle
<i>pWdogCfg</i>	the configurations of the WDOG module

Returns

WDOG_StatusType initialization status

void WDOG_IRQHandler (WDOG_HandleType * *pWdogHandle*)

Wdg n interrupt process.

Parameters

<i>pWdogHandle</i>	WDOG processing handle
--------------------	------------------------

void WDOG_Refresh (WDOG_HandleType * *pWdogHandle*)

feed the watch dog by writing typical CMD to counter.

Parameters

<i>pWdogHandle</i>	WDOG processing handle
--------------------	------------------------

void WDOG_SelfTest (WDOG_HandleType * *pWdogHandle*, WDOG_TestType *eMode*)

Performs a self-test on the Watchdog Timer (WDOG) module.

This function configures the WDOG module to perform a self-test based on the specified test mode. It unlocks the WDOG registers, sets a timeout value, and configures the control and status register to enable the WDOG, update its configuration, select the clock source, and set the test mode.

Parameters

<i>pWdogHandle</i>	Pointer to the WDOG handle structure that contains the instance information.
<i>eMode</i>	The test mode to be performed. This parameter determines the type of self-test to execute.

Returns

void

Note

The function waits for the WDOG registers to be unlocked before proceeding with the configuration. The timeout value is set to the maximum value (0xFFFF) during the self-test.

void WDOG_Unlock (WDOG_HandleType * *pWdogHandle*)

unlock the wdog before Watch dog reconfigure set.

Parameters

<i>pWdogHandle</i>	WDOG processing handle
--------------------	------------------------

Module_driver_wku

Collaboration diagram for Module_driver_wku:Modules

HwA_wku

Enumerations

enum **WKU_StatusType** { **WKU_STATUS_SUCCESS** = 0U, **WKU_STATUS_PARAM_INVALID** = 1U }

WKU return type.

Functions

WKU_StatusType WKU_EnableWakeupSource (const uint32_t u32Input)

WKU Enable wakeup source for single input.

WKU_StatusType WKU_DisableWakeupSource (const uint32_t u32Input)

WKU Disable wakeup source for single input.

uint32_t **WKU_GetWakeupSources** (void)

WKU get wakeup source.

Detailed Description

Enumeration Type Documentation

enum **WKU_StatusType**

WKU return type.

Enumerator:

WKU_STATUS_SUCCESS	
WKU_STATUS_PARAM_INVALID	

Function Documentation

WKU_StatusType WKU_DisableWakeupSource (const uint32_t *u32Input*)

WKU Disable wakeup source for single input.

Parameters

<i>u32Input</i>	Number of input, WKU_WakeupInputType type is wakeup source definition
-----------------	---

Returns

WKU return type

WKU_StatusType WKU_EnableWakeupSource (const uint32_t *u32Input*)

WKU Enable wakeup source for single input.

Parameters

<i>u32Input</i>	Number of input, WKU_WakeupInputType type is wakeup source definition
-----------------	---

Returns

WKU return type

uint32_t WKU_GetWakeupSources (void)

WKU get wakeup source.

Returns

output wakeup source

Class Documentation

_ADC_HandleType Struct Reference

The structure of the ADC processing handle.

```
#include <module_driver_adc.h>
```

Public Attributes

ADC_InstanceType eInstance

uint8_t **reserve0** [3]

struct {

uint8_t **u8ChnCnt**

ADC_SeqModeType eSeqMode

bool **bConvCompleteIntEn**

bool **bOverRunIntEn**

bool **bCompareIntEn**

void(* **pConvCompleteNotify**)(struct **_ADC_HandleType** *pHandle)

void(* **pOverRunNotify**)(struct **_ADC_HandleType** *pHandle)

void(* **pCompareNotify**)(struct **_ADC_HandleType** *pHandle)

uint32_t * **pResultBuffer**

} **tSettings**

Detailed Description

The structure of the ADC processing handle.

Member Data Documentation

bool _ADC_HandleType::bCompareIntEn

Enable interrupt when conversion result lays in the compare threshold

bool _ADC_HandleType::bConvCompleteIntEn

Enable interrupt when conversion completed

bool _ADC_HandleType::bOverRunIntEn

Enable interrupt when overrun occurred

ADC_InstanceType _ADC_HandleType::eInstance

ADC instance

ADC_SeqModeType _ADC_HandleType::eSeqMode

ADC sequence mode (single, continuous, discontinuous)

void(* _ADC_HandleType::pCompareNotify) (struct _ADC_HandleType *pHandle)

Compare interrupt callback

void(* _ADC_HandleType::pConvCompleteNotify) (struct _ADC_HandleType *pHandle)

Conversion complete interrupt callback

void(* _ADC_HandleType::pOverRunNotify) (struct _ADC_HandleType *pHandle)

Overrun interrupt callback

uint32_t* _ADC_HandleType::pResultBuffer

only used for mode 0~3

uint8_t _ADC_HandleType::reserve0[3]

struct { ... } _ADC_HandleType::tSettings

uint8_t _ADC_HandleType::u8ChnCnt

number of channels

The documentation for this struct was generated from the following file:

module_driver_adc.h

_AONTIMER_HandleType Struct Reference

The structure of the AONTIMER processing handle.

#include <module_driver_aontimer.h>

Public Attributes

AONTIMER_InstanceType eInstance

char **_aligned** [3]

struct {

void(* **pAontimerCallback**)(struct **_AONTIMER_HandleType**
*pHandle)

} **tSettings**

Detailed Description

The structure of the AONTIMER processing handle.

Member Data Documentation

char **_AONTIMER_HandleType::_aligned**[3]

AONTIMER_InstanceType _AONTIMER_HandleType::eInstance

AONTIMER instance

void(* _AONTIMER_HandleType::pAontimerCallback) (struct _AONTIMER_HandleType *pHandle)

fault interrupt callback

struct { ... } _AONTIMER_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_aontimer.h

_CMP_HandleType Struct Reference

The structure of the CMP processing handle.

#include <module_driver_cmp.h>

Public Attributes

```
CMP_InstanceType eInstance
struct {
    bool bLPModeEn
    bool bOutToPackagePinEn
    bool bRisingIntEn
    bool bFallingIntEn
    uint32_t u32InitDelay
    CMP_CompleteIntCallback pIsr
} tSettings
```

Detailed Description

The structure of the CMP processing handle.

Member Data Documentation

bool _CMP_HandleType::bFallingIntEn

Enable falling interrupt

bool _CMP_HandleType::bLPModeEn

CMP stop mode enable

bool _CMP_HandleType::bOutToPackagePinEn

CMP output to package pin enable

bool _CMP_HandleType::bRisingIntEn

Enable rising interrupt

CMP_InstanceType _CMP_HandleType::eInstance

CMP instance

CMP_CompleteIntCallback _CMP_HandleType::pIsr

struct { ... } _CMP_HandleType::tSettings

uint32_t _CMP_HandleType::u32InitDelay

CMP initialization delay, the max initialization delay is 30us

The documentation for this struct was generated from the following file:

module_driver_cmp.h

_CMU_HandleType Struct Reference

The structure of the CMU processing handle.

```
#include <module_driver_cmu.h>
```

Public Attributes

```
CMU_InstanceType eInstance  
uint8_t _aligned [3U]  
struct {  
    void(* pErrorCallback)(struct _CMU_HandleType  
        *pHandle, const  
        CMU_InterruptType eStatus)  
} tSettings
```

Detailed Description

The structure of the CMU processing handle.

Member Data Documentation

uint8_t _CMU_HandleType::_aligned[3U]

CMU_InstanceType _CMU_HandleType::eInstance

CMU instance

void(* _CMU_HandleType::pErrorCallback) (struct **_CMU_HandleType** *pHandle, const
CMU_InterruptType eStatus)

Error Callback function point

struct { ... } _CMU_HandleType::tSettings

The documentation for this struct was generated from the following file:

`module_driver_cmu.h`

_CPM_HandleType Struct Reference

The structure of the CPM processing handle.

`#include <module_driver_cpm.h>`

Public Attributes

CPM_InstanceType eInstance

`char _aligned [3]`

`struct {`

`void(* pCpmCallback )(struct _CPM_HandleType *pHandle)`

`} tSettings`

Detailed Description

The structure of the CPM processing handle.

Member Data Documentation

`char _CPM_HandleType::_aligned[3]`

`CPM_InstanceType _CPM_HandleType::eInstance`

CPM instance

`void(* _CPM_HandleType::pCpmCallback) (struct _CPM_HandleType *pHandle)`

fault interrupt callback

`struct { ... } _CPM_HandleType::tSettings`

The documentation for this struct was generated from the following file:

`module_driver_cpm.h`

_DMA_HandleType Struct Reference

#include <module_driver_dma.h>

Collaboration diagram for _DMA_HandleType:

IMAGE

Public Attributes

```
struct {  
    uint8_t u8Channel  
    DMA_InstanceHandleType * pInstance  
    struct {  
        void * completedata  
        void(* pTransferCompleteCallback)(void *arg)  
    } callback  
} tSettings
```

Member Data Documentation

struct { ... } _DMA_HandleType::callback

void* _DMA_HandleType::completedata

DMA_InstanceHandleType* _DMA_HandleType::pInstance

void(* _DMA_HandleType::pTransferCompleteCallback) (void *arg)

channel interrupt callback

struct { ... } _DMA_HandleType::tSettings

uint8_t _DMA_HandleType::u8Channel

DMA Channel

The documentation for this struct was generated from the following file:

module_driver_dma.h

_EIM_HandleType Struct Reference

The structure of the EIM processing handle.

#include <module_driver_eim.h>

Public Attributes

EIM_InstanceType eInstance

uint8_t **_aligned** [3U]

struct {

uint8_t **u8EimChn**

uint8_t **_aligned** [3U]

} **tStatus**

Detailed Description

The structure of the EIM processing handle.

Member Data Documentation

uint8_t **_EIM_HandleType::_aligned**[3U]

EIM_InstanceType _EIM_HandleType::eInstance

EIM instance

struct { ... } **_EIM_HandleType::tStatus**

uint8_t **_EIM_HandleType::u8EimChn**

The documentation for this struct was generated from the following file:

module_driver_eim.h

_ERM_HandleType Struct Reference

The structure of the ERM processing handle.

#include <module_driver_erm.h>

Public Attributes

```
ERM_InstanceType eInstance
uint8_t _aligned [3U]
struct {
    void(* pErrorCallback [ERM_EAR_REG_COUNT])(struct
        _ERM_HandleType *pHandle,
        uint8_t u8ErrorType, uint32_t
        u32Addr)
    } tSettings
```

Detailed Description

The structure of the ERM processing handle.

Member Data Documentation

uint8_t **_ERM_HandleType::_aligned**[3U]

ERM_InstanceType **_ERM_HandleType::eInstance**

CMU instance

```
void(* _ERM_HandleType::pErrorCallback[ERM_EAR_REG_COUNT])(struct _ERM_HandleType *pHandle,
uint8_t u8ErrorType, uint32_t u32Addr)
```

```
struct { ... } _ERM_HandleType::tSettings
```

The documentation for this struct was generated from the following file:

module_driver_erm.h

_FCIIC_DmaConfig Struct Reference

Structure for configuring DMA for I2C Master and Slave operations.

#include <module_driver_fciic.h>

Collaboration diagram for _FCIIC_DmaConfig:

IMAGE

Public Attributes

FCIIC_DmaStatusType u8I2cDmaNextStage

uint8_t _aligned [3]

DMA_HandleType * pDmaHandle

Detailed Description

Structure for configuring DMA for I2C Master and Slave operations.

This structure holds the configuration for DMA operations in both I2C Master and Slave modes.

Member Data Documentation

uint8_t _FCIIC_DmaConfig::_aligned[3]

Populate Fields.

DMA_HandleType* _FCIIC_DmaConfig::pDmaHandle

DMA_Handle which should be initialized before using.

FCIIC_DmaStatusType _FCIIC_DmaConfig::u8I2cDmaNextStage

Represents the next stage of the IIC DMA state machine.

The documentation for this struct was generated from the following file:

module_driver_fciic.h

_FCIIC_MasterHandle Struct Reference

Structure for handling the FCIIC Master operations. Implements : **_FCIIC_MasterHandle**.

#include <module_driver_fciic.h>

Collaboration diagram for **_FCIIC_MasterHandle**:

IMAGE

Public Attributes

FCIIC_InstanceType **eInstance**

uint8_t **_aligned** [3]

struct {

 bool **bUsed**

FCIIC_DirectionType **eDirection**

 uint8_t **u8CmdSize**

 uint8_t **u8CmdOffset**

 uint8_t **u8DmaCmdSize**

FCIIC_ChannelStatusType **eChannelStatus**

 uint8_t **u8DmaCmdOffset**

 uint8_t **_aligned** [1]

 uint16_t **u16DataSize**

 uint16_t **u16DataOffset**

 uint16_t **u16DmaCmdData** [**FCIIC_MASTER_CMD_SIZE**]

FCIIC_TX_CMDType **u8Cmd** [**FCIIC_MASTER_CMD_SIZE**]

 uint8_t **u8CmdData** [**FCIIC_MASTER_CMD_SIZE**]

} **tStatus**

struct {

 uint8_t * **pData**

FCIIC_DmaConfigType * **pFciicTxDmaConfig**

FCIIC_DmaConfigType * **pFciicRxDmaConfig**

 void(* **pCallback**)(struct **_FCIIC_MasterHandle**
 ***pFciicMasterHandle**,

FCIIC_StatusType status)

 void(* **pErrorCallback**)(struct **_FCIIC_MasterHandle**
 ***pFciicMasterHandle**,

FCIIC_StatusType status)

 void * **pUserData**


```
} tSettings
```

Detailed Description

Structure for handling the FCIIC Master operations. Implements : **_FCIIC_MasterHandle**.

This structure represents the handle for managing FCIIC Master operations, including the current state and settings.

Member Data Documentation

uint8_t _FCIIC_MasterHandle::_aligned[1]

Populate Fields.

bool _FCIIC_MasterHandle::bUsed

Whether it is being used.

FCIIC_ChannelStatusType _FCIIC_MasterHandle::eChannelStatus

Master channel current state.

FCIIC_DirectionType _FCIIC_MasterHandle::eDirection

Master transfer direction.

FCIIC_InstanceType _FCIIC_MasterHandle::eInstance

I2C instance.

void(* _FCIIC_MasterHandle::pCallback) (struct _FCIIC_MasterHandle *pFciicMasterHandle, FCIIC_StatusType status)

Success callback function, can be designed when successfully transferred.

uint8_t* _FCIIC_MasterHandle::pData

Pointer to transfer data.

void(* _FCIIC_MasterHandle::pErrorCallback) (struct _FCIIC_MasterHandle *pFciicMasterHandle, FCIIC_StatusType status)

Error callback function, would return the error state.

FCIIC_DmaConfigType* _FCIIC_MasterHandle::pFciicRxDmaConfig

DMA config instance used for receiving.

FCIIC_DmaConfigType* _FCIIC_MasterHandle::pFciicTxDmaConfig

DMA config instance used for sending.

void* _FCIIC_MasterHandle::pUserData

User data.

struct { ... } _FCIIC_MasterHandle::tSettings

The field of tSettings allows the user to set.

struct { ... } _FCIIC_MasterHandle::tStatus

The field of tStatus cannot be allowed modifying by the user.

uint16_t _FCIIC_MasterHandle::u16DataOffset

Master had sent the command data of size.

uint16_t _FCIIC_MasterHandle::u16DataSize

Master will transfer the size of data.

uint16_t _FCIIC_MasterHandle::u16DmaCmdData[FCIIC_MASTER_CMD_SIZE]

The Master command data, prepared for DMA transfer.

FCIIC_TX_CMDType _FCIIC_MasterHandle::u8Cmd[FCIIC_MASTER_CMD_SIZE]

The Master data, prepared for Interrupt transfer.

uint8_t _FCIIC_MasterHandle::u8CmdData[FCIIC_MASTER_CMD_SIZE]

The Master command data, prepared for Interrupt transfer.

uint8_t _FCIIC_MasterHandle::u8CmdOffset

Master had sent the data of size, prepared for transfer.

uint8_t _FCIIC_MasterHandle::u8CmdSize

Master command data size, prepared for transfer.

uint8_t _FCIIC_MasterHandle::u8DmaCmdOffset

Master had sent the command data of size, prepared for DMA transfer.

uint8_t _FCIIC_MasterHandle::u8DmaCmdSize

Master had sent the command data of size, prepared for DMA transfer.

The documentation for this struct was generated from the following file:

module_driver_fciic.h

_FCIIC_SlaveHandle Struct Reference

Structure for handling the FCIIC Slave operations. Implements : **_FCIIC_SlaveHandle**.

#include <module_driver_fciic.h>

Collaboration diagram for _FCIIC_SlaveHandle:

IMAGE

Public Attributes

FCIIC_InstanceType eInstance

uint8_t **_aligned** [3]

struct {

 bool **bUsed**

FCIIC_StatusType eRunStatus

FCIIC_ChannelStatusType eChannelStatus

 uint8_t **subaddressSize**

 uint8_t **curSubaddressSize**

 uint16_t **u16DataSize**

 uint16_t **u16DataOffset**

 uint32_t **u32CurSubaddress**

 uint32_t **u32Subaddress**

} **tStatus**

struct {

 uint8_t * **pData**

FCIIC_DmaConfigType * pFciicTxDmaConfig

FCIIC_DmaConfigType * pFciicRxDmaConfig

 void(* **pCallback**)(struct **_FCIIC_SlaveHandle**
 * pFciicSlaveHandle)

 void(* **pErrorCallback**)(struct **_FCIIC_SlaveHandle**
 * pFciicSlaveHandle)

 void(* **pSubAddressSendingCallback**)(struct **_FCIIC_SlaveHandle**
 * pFciicSlaveHandle)

 void(* **pSubAddressReceivingCallback**)(struct **_FCIIC_SlaveHandle**
 * pFciicSlaveHandle)

 void * **pUserData**

} **tSettings**

Detailed Description

Structure for handling the FCIIC Slave operations. Implements : **_FCIIC_SlaveHandle**.

This structure represents the handle for managing FCIIC Slave operations, including the current state and settings.

Member Data Documentation

uint8_t _FCIIC_SlaveHandle::_aligned[3]

Populate Fields.

bool _FCIIC_SlaveHandle::bUsed

Whether it is being used.

uint8_t _FCIIC_SlaveHandle::curSubaddressSize

Current received subaddress length. Maximum is 4 bytes.

FCIIC_ChannelStatusType _FCIIC_SlaveHandle::eChannelStatus

salve channel current state.

FCIIC_InstanceType _FCIIC_SlaveHandle::eInstance

I2C instance.

FCIIC_StatusType _FCIIC_SlaveHandle::eRunStatus

This field records the errors encountered in a communication.

void(* _FCIIC_SlaveHandle::pCallback) (struct _FCIIC_SlaveHandle *pFciicSlaveHandle)

Success callback function, can be designed when successfully transferred.

uint8_t* _FCIIC_SlaveHandle::pData

Pointer to transfer data.

void(* _FCIIC_SlaveHandle::pErrorCallback) (struct _FCIIC_SlaveHandle *pFciicSlaveHandle)

Error callback function, would return the error state.

FCIIC_DmaConfigType* _FCIIC_SlaveHandle::pFciicRxDmaConfig

DMA config instance used for receiving.

FCIIC_DmaConfigType* _FCIIC_SlaveHandle::pFciicTxDmaConfig

DMA config instance used for sending.

**void(* _FCIIC_SlaveHandle::pSubAddressReceivingCallback) (struct _FCIIC_SlaveHandle
*pFciicSlaveHandle)**

Subaddress receiving callback function.

**void(* _FCIIC_SlaveHandle::pSubAddressSendingCallback) (struct _FCIIC_SlaveHandle
*pFciicSlaveHandle)**

Subaddress sending callback function.

void* _FCIIC_SlaveHandle::pUserData

User data.

uint8_t _FCIIC_SlaveHandle::subaddressSize

Size of the sub-address in bytes. Maximum is 4 bytes.

struct { ... } _FCIIC_SlaveHandle::tSettings

The field of tSettings allows the user to set.

struct { ... } _FCIIC_SlaveHandle::tStatus

The field of tStatus cannot be allowed modifying by the user.

uint16_t _FCIIC_SlaveHandle::u16DataOffset

Slave had sent or received the data of size.

uint16_t _FCIIC_SlaveHandle::u16DataSize

The longest length of data received or sent by the slave in a single communication.

uint32_t _FCIIC_SlaveHandle::u32CurSubaddress

Sub-address for targeted buffer address received from master. Sent most significant byte first.

uint32_t _FCIIC_SlaveHandle::u32Subaddress

Sub-address for targeted buffer address. Sent most significant byte first.

The documentation for this struct was generated from the following file:

`module_driver_fciic.h`

_FCPIT_HandleType Struct Reference

#include <module_driver_fcpit.h>

Public Attributes

```
FCPIT_InstanceType eInstance
bool bDebugEn
bool bLowPowerModeEn
struct {
    FCPIT_TimerModeType eMode
    bool bChainModeEn
    uint32_t u32TimerValue
    bool bStartOnTrigger
    bool bStopOnInterrupt
    bool bReloadOnTrigger
    FCPIT_TriggerSelectType eTriggerSel
    bool bChannellsrEn
    void(* pChannelCallback )(struct _FCPIT_HandleType
        *pHandle, uint32_t u32channel)
    bool bChannelStarted
} tChannelStatus [MAX_FCPIT_CHANNEL_NUM]
```

Member Data Documentation

bool _FCPIT_HandleType::bChainModeEn

whether to use chain mode, if use this mode, channel must not be the channel 0

bool _FCPIT_HandleType::bChannellsrEn

whether to use interrupt

bool _FCPIT_HandleType::bChannelStarted

bool _FCPIT_HandleType::bDebugEn

whether to use debug mode ,if enable this mode, the counter will stop when debugging.

bool _FCPIT_HandleType::bLowPowerModeEn

Configure the timer channels to continue running or stop when the device enters the LPM mode

bool _FCPIT_HandleType::bReloadOnTrigger

Fcpit timer reload when triggered

bool _FCPIT_HandleType::bStartOnTrigger

Fcpit timer start when triggered

bool _FCPIT_HandleType::bStopOnInterrupt

Fcpit timer stop on interrupt

FCPIT_InstanceType _FCPIT_HandleType::eInstance

FCPIT_TimerModeType _FCPIT_HandleType::eMode

Fcpit counter mode

FCPIT_TriggerSelectType _FCPIT_HandleType::eTriggerSel

trigger source

void(* _FCPIT_HandleType::pChannelCallback) (struct _FCPIT_HandleType *pHandle, uint32_t u32channel)

struct { ... } _FCPIT_HandleType::tChannelStatus[MAX_FCPIT_CHANNEL_NUM]

uint32_t _FCPIT_HandleType::u32TimerValue

timer compare value, the range of value is related to the counter mode

The documentation for this struct was generated from the following file:

module_driver_fcpit.h

_FCSMU_HandleType Struct Reference

The structure of the FCSMU processing handle.

#include <module_driver_fcsmu.h>

Public Attributes

FCSMU_InstanceType eInstance

uint8_t **_aligned** [3U]

struct {

void(* **pFaultCallback**)(struct **_FCSMU_HandleType**
*pHandle, uint32_t
u32IrqChannel)

void(* **pWarningCallback**)(struct **_FCSMU_HandleType**
*pHandle, uint32_t
u32IrqChannel)

void(* **pTimeoutCallback**)(struct **_FCSMU_HandleType**
*pHandle)

} **tSettings**

Detailed Description

The structure of the FCSMU processing handle.

Member Data Documentation

`uint8_t _FCSMU_HandleType::_aligned[3U]`

`FCSMU_InstanceType _FCSMU_HandleType::eInstance`

`void(* _FCSMU_HandleType::pFaultCallback) (struct _FCSMU_HandleType *pHandle, uint32_t u32IrqChannel)`

`void(* _FCSMU_HandleType::pTimeoutCallback) (struct _FCSMU_HandleType *pHandle)`

`void(* _FCSMU_HandleType::pWarningCallback) (struct _FCSMU_HandleType *pHandle, uint32_t u32IrqChannel)`

`struct { ... } _FCSMU_HandleType::tSettings`

The documentation for this struct was generated from the following file:

`module_driver_fcsmu.h`

_FCSPI_HandleType Struct Reference

The structure of the FCSPI processing handle.

#include <module_driver_fcspi.h>

Public Attributes

FCSPI_InstanceType **eInstance**

struct {

FCSPI_MasterSlaveModeType Mode

uint32_t **SckFreq**

FCSPI_SckPolarityType SckPolarity

FCSPI_SckPhaseType SckPhase

FCSPI_SckFeature SckFeature

FCSPI_PcsModeType PcsMode

bool **PcsContinuousEnable**

struct {

FCSPI_PcsSelType PCSn

FCSPI_PcsPolarityType PcsPolarity

} **HardwareSelect**

uint32_t **FrameSize**

FCSPI_DataFirstBitType BitOrder

FCSPI_TransferWidthType TransferWidth

uint32_t **TxFifoWatermark**

uint32_t **RxFifoWatermark**

FCSPI_PinConfigType PinConfig

bool **SckLoopbackEnable**

bool **InternalPcsEnable**

bool **ByteSwapEnable**

uint32_t **DelayPCS2SCK**

uint32_t **DelayPCS2PCS**

uint32_t **DelaySCK2PCS**

void(* **TansferStartNotification**)(struct **_FCSPI_HandleType**
*pFcspiHandle)

void(* **TansferEndNotification**)(struct **_FCSPI_HandleType**
*pFcspiHandle)

} **Config**

struct {

```

volatile FCSPI_StateType state
uint32_t TR_CTRL
FCSPI_DataAccessWidthType eAccessWidth
uint8_t * TxBuff
uint32_t TxSize
uint32_t TxCnt
void(* TxIsr)(struct _FCSPI_HandleType
    *pFcspiHandle)
uint8_t * RxBuff
uint32_t RxSize
uint32_t RxCnt
void(* RxIsr)(struct _FCSPI_HandleType
    *pFcspiHandle)
} tStatus

```

FCSPI driver inner status tStatus must not be set by user.

Detailed Description

The structure of the FCSPi processing handle.

Member Data Documentation

FCSPI_DataFirstBitType **_FCSPI_HandleType::BitOrder**

MSB or LSB

bool **_FCSPI_HandleType::ByteSwapEnable**

Byte Swap enable or not

struct { ... } **_FCSPI_HandleType::Config**

uint32_t **_FCSPI_HandleType::DelayPCS2PCS**

This configuration item is a multiple: How many times of the time interval between the PCS signal de-assert to the next frame PCS assert is one cycle of SCK.

uint32_t _FCSPI_HandleType::DelayPCS2SCK

This configuration item is a multiple: How many times of the time interval between the PCS signal assert to the first edge of SCK is one cycle of SCK.

uint32_t _FCSPI_HandleType::DelaySCK2PCS

This configuration item is a multiple: How many times of the time interval between the last edge of SCK to the PCS de-assert is one cycle of SCK.

FCSPI_DataAccessWidthType _FCSPI_HandleType::eAccessWidth

FCSPI_InstanceType _FCSPI_HandleType::eInstance

FCSPIn base address

uint32_t _FCSPI_HandleType::FrameSize

Bit count of one frame

struct { ... } _FCSPI_HandleType::HardwareSelect

bool _FCSPI_HandleType::InternalPcsEnable

Internal PCS enable or not, only valid on slave mode

FCSPI_MasterSlaveModeType _FCSPI_HandleType::Mode

Master or Slave mode

bool _FCSPI_HandleType::PcsContinuousEnable

Whether 'PCS' is continuous in a transfer API service

FCSPI_PcsModeType _FCSPI_HandleType::PcsMode

PCS is controled by FCSPI hardware or user control GPIO pin to simulate PCS

FCSPI_PcsSelType _FCSPI_HandleType::PCSn

Select from FCSPIIn_PCS0 ~ FCSPIIn_PCS3

FCSPI_PcsPolarityType _FCSPI_HandleType::PcsPolarity

Pcs Polarity

FCSPI_PinConfigType _FCSPI_HandleType::PinConfig

Configuration about 'SIN' and 'SOUT' pin

uint8_t* _FCSPI_HandleType::RxBuff

uint32_t _FCSPI_HandleType::RxCnt

Bytes count of data that has been read from RX FIFO,It is a state value that changes as the transfer progresses

uint32_t _FCSPI_HandleType::RxFifoWatermark

Rx FIFO watermark

void(* _FCSPI_HandleType::RxIsr) (struct _FCSPI_HandleType *pFcspiHandle)

function pointer for Rx ISR

uint32_t _FCSPI_HandleType::RxSize

Bytes count of all receive data.It is a reference value that usually does not change during transmission

FCSPI_SckFeature _FCSPI_HandleType::SckFeature

Whether the duty cycle of 'SCK' is required to be 50% or not

uint32_t _FCSPI_HandleType::SckFreq

Desired Frequency of 'SCK'

bool _FCSPI_HandleType::SckLoopbackEnable

Sck loopback enable or not

FCSPI_SckPhaseType _FCSPI_HandleType::SckPhase

SCK phase

FCSPI_SckPolarityType _FCSPI_HandleType::SckPolarity

SCK polarity

volatile FCSPI_StateType _FCSPI_HandleType::state

void(* _FCSPI_HandleType::TansferEndNotification) (struct _FCSPI_HandleType *pFcspiHandle)

callback to notify that transfer completes,only valid in non-blocking API service

void(* _FCSPI_HandleType::TansferStartNotification) (struct _FCSPI_HandleType *pFcspiHandle)

callback to notify that transfer starts,only valid in non-blocking API service

uint32_t _FCSPI_HandleType::TR_CTRL

The current TR_CTRL value of SPI Instance

FCSPI_TransferWidthType _FCSPI_HandleType::TransferWidth

Transfer width

struct { ... } _FCSPI_HandleType::tStatus

FCSPI driver inner status tStatus must not be set by user.

uint8_t* _FCSPI_HandleType::TxBuff

uint32_t _FCSPI_HandleType::TxCnt

Bytes count of data that has been writen into TX FIFO.It is a state value that changes as the transfer progresses

uint32_t _FCSPI_HandleType::TxFifoWatermark

Tx FIFO watermark

void(* _FCSPI_HandleType::TxIsr) (struct _FCSPI_HandleType *pFcspiHandle)

function pointer for Tx ISR

uint32_t _FCSPI_HandleType::TxSize

Bytes count of all send data.It is a reference value that usually does not change during transmission

The documentation for this struct was generated from the following file:

`module_driver_fcspi.h`

_FCUART_HandleType Struct Reference

The structure of the UART processing handle.

#include <module_driver_fcuart.h>

Collaboration diagram for _FCUART_HandleType:

IMAGE

Public Attributes

```
FCUART_InstanceType eInstance
uint8_t _aligned [3]
struct {
    FCUART_DataType * tFCUART_RxMsg
    FCUART_DataType * tFCUART_TxMsg
    uint32_t u32FCUART_TransmitTimeout
    FCUART_SequenceType eCurrentSequence
    uint8_t u8FCUART_UartUsed
    void(* pFCUART_ErrorNotify )(struct _FCUART_HandleType
        *pUartHandle, uint32_t
        u32Error)
    void(* pFCUART_RxNotify )(struct _FCUART_HandleType
        *pUartHandle, FCUART_DataType
        *pTxRxCfg)
    void(* pFCUART_TxEmptyNotify )(struct _FCUART_HandleType
        *pUartHandle, FCUART_DataType
        *pTxRxCfg)
    void(* pFCUART_TxCompleteNotify )(struct _FCUART_HandleType
        *pUartHandle, FCUART_DataType
        *pTxRxCfg)
    void(* pFCUART_IdleNotify )(struct _FCUART_HandleType
        *pUartHandle)
} tStatus
```

Detailed Description

The structure of the UART processing handle.

Member Data Documentation

uint8_t _FCUART_HandleType::_aligned[3]

Alignment bytes

FCUART_SequenceType _FCUART_HandleType::eCurrentSequence

sequence table

FCUART_InstanceType _FCUART_HandleType::eInstance

UART instance

void(* _FCUART_HandleType::pFCUART_ErrorNotify) (struct _FCUART_HandleType *pUartHandle, uint32_t u32Error)

error notify callback function point

void(* _FCUART_HandleType::pFCUART_IdleNotify) (struct _FCUART_HandleType *pUartHandle)

Idle notify callback function point

void(* _FCUART_HandleType::pFCUART_RxNotify) (struct _FCUART_HandleType *pUartHandle, FCUART_DataType *pTxRxCfg)

receive notify callback function point

void(* _FCUART_HandleType::pFCUART_TxCompleteNotify) (struct _FCUART_HandleType *pUartHandle, FCUART_DataType *pTxRxCfg)

Transfer complete notify callback function point

void(* _FCUART_HandleType::pFCUART_TxEmptyNotify) (struct _FCUART_HandleType *pUartHandle, FCUART_DataType *pTxRxCfg)

Transfer empty notify callback function point

FCUART_DataType* _FCUART_HandleType::tFCUART_RxMsg

check every pUart instance whether is used

FCUART_DataType* _FCUART_HandleType::tFCUART_TxMsg

pUart instance transfer buffer

struct { ... } _FCUART_HandleType::tStatus

uint32_t _FCUART_HandleType::u32FCUART_TransmitTimeout

check every pUart instance transmit timeout

uint8_t _FCUART_HandleType::u8FCUART_UartUsed

check every pUart instance whether is used

The documentation for this struct was generated from the following file:

module_driver_fcuart.h

_FLASH_HandleType Struct Reference

The structure of the FLASH processing handle.

#include <module_driver_flash.h>

Public Attributes

FLASH_InstanceType eInstance

char **_aligned** [3]

struct {

uint32_t **u32Address**

uint32_t **u32Length**

uint32_t **u32CurrentAddress**

uint32_t **u32ProgramSize**

uint32_t **u32EraseSectorSize**

uint32_t **u32ErrorAdress**

uint8_t * **pDate**

Fls_Status_t eFlsStatus

char **_aligned1** [3]

} **tStatus**

Detailed Description

The structure of the FLASH processing handle.

Member Data Documentation

char **_FLASH_HandleType::_aligned**[3]

char **_FLASH_HandleType::_aligned1**[3]

Fls_Status_t _FLASH_HandleType::eFlsStatus

Flash operation status

FLASH_InstanceType _FLASH_HandleType::eInstance

Flash instance

uint8_t* _FLASH_HandleType::pDate

struct { ... } _FLASH_HandleType::tStatus

uint32_t _FLASH_HandleType::u32Address

uint32_t _FLASH_HandleType::u32CurrentAddress

uint32_t _FLASH_HandleType::u32EraseSectorSize

uint32_t _FLASH_HandleType::u32ErrorAddress

uint32_t _FLASH_HandleType::u32Length

uint32_t _FLASH_HandleType::u32ProgramSize

The documentation for this struct was generated from the following file:

module_driver_flash.h

_FLEXCAN_HandleType Struct Reference

The structure of the CAN processing handle.

#include <module_driver_flexcan.h>

Collaboration diagram for _FLEXCAN_HandleType:

IMAGE

Public Attributes

FLEXCAN_InstanceType **eInstance**

struct {

FLEXCAN_SequenceType eStatusSeq

uint8_t **u8MbStart**

FLEXCAN_DataWidthType aMbDataWidth [FLEXCAN_FD_DATALEN_RANGE_NUM_MAX]

uint32_t **aRxInterruptMask** [FLEXCAN_IFLAGMASK_NUM_MAX]

uint32_t **aTxInterruptMask** [FLEXCAN_IFLAGMASK_NUM_MAX]

uint32_t **aInterruptMask** [FLEXCAN_IFLAGMASK_NUM_MAX]

uint32_t **aRxPollingMask** [FLEXCAN_IFLAGMASK_NUM_MAX]

uint32_t **aTxPollingMask** [FLEXCAN_IFLAGMASK_NUM_MAX]

uint32_t **aPollingMask** [FLEXCAN_IFLAGMASK_NUM_MAX]

} **tStatus**

can inner status tStatus Can't be set by user

struct {

FLEXCAN_RxMsgType * pRxBuf

FLEXCAN_WakeupMsgType * pWakeMsg

void(* **pErrorNotify**)(struct **_FLEXCAN_HandleType**

*pHandle,

FLEXCAN_ErrorInfoType

*pErrorInfo)

void(* **pFifoIssueNotify**)(struct **_FLEXCAN_HandleType**

*pHandle,

FLEXCAN_FifoIssueType

*pFifoIssue)

void(* **pTxNotify**)(struct **_FLEXCAN_HandleType**

*pHandle, uint8_t u8TxHandle)

```

void(* pRxNotify )(struct _FLEXCAN_HandleType
    *pHandle, FLEXCAN_RxMsgType
    *pRxMsg)
void(* pWakeNotify )(struct _FLEXCAN_HandleType
    *pHandle,
    FLEXCAN_WakeupMsgType
    *pWakeMsg)
} tSettings

```

CAN global settings, include callback function and special value like buffer address.

Detailed Description

The structure of the CAN processing handle.

Member Data Documentation

uint32_t _FLEXCAN_HandleType::aInterruptMask[FLEXCAN_IFLAGMASK_NUM_MAX]

tx mask value for interrupt

FLEXCAN_DataWidthType

_FLEXCAN_HandleType::aMbDataWidth[FLEXCAN_FD_DATALEN_RANGE_NUM_MAX]

Message Buffer Data Width, only used in canfd

uint32_t _FLEXCAN_HandleType::aPollingMask[FLEXCAN_IFLAGMASK_NUM_MAX]

tx mask value for polling

uint32_t _FLEXCAN_HandleType::aRxInterruptMask[FLEXCAN_IFLAGMASK_NUM_MAX]

rx mask value for interrupt

uint32_t _FLEXCAN_HandleType::aRxPollingMask[FLEXCAN_IFLAGMASK_NUM_MAX]

rx mask value for polling

uint32_t _FLEXCAN_HandleType::aTxInterruptMask[FLEXCAN_IFLAGMASK_NUM_MAX]

tx mask value for interrupt

uint32_t _FLEXCAN_HandleType::aTxPollingMask[FLEXCAN_IFLAGMASK_NUM_MAX]

tx mask value for polling

FLEXCAN_InstanceType _FLEXCAN_HandleType::eInstance

CAN instance

FLEXCAN_SequenceType _FLEXCAN_HandleType::eStatusSeq

Can statue sequence

**void(* _FLEXCAN_HandleType::pErrorNotify) (struct _FLEXCAN_HandleType *pHandle,
FLEXCAN_ErrorInfoType *pErrorInfo)**

Error callback function type

**void(* _FLEXCAN_HandleType::pFifoIssueNotify) (struct _FLEXCAN_HandleType *pHandle,
FLEXCAN_FifoIssueType *pFifoIssue)**

FIFO issue callback function type

FLEXCAN_RxMsgType* _FLEXCAN_HandleType::pRxBuf

receive buffer address

**void(* _FLEXCAN_HandleType::pRxNotify) (struct _FLEXCAN_HandleType *pHandle, FLEXCAN_RxMsgType
*pRxMsg)**

Receive callback function type

void(* _FLEXCAN_HandleType::pTxNotify) (struct _FLEXCAN_HandleType *pHandle, uint8_t u8TxHandle)

transmit callback function type, u8TxHandle is the handle for message

FLEXCAN_WakeupMsgType* _FLEXCAN_HandleType::pWakeMsg

Wake-up message buffer address

**void(* _FLEXCAN_HandleType::pWakeNotify) (struct _FLEXCAN_HandleType *pHandle,
FLEXCAN_WakeupMsgType *pWakeMsg)**

Pretended Network wake-up callback function type

struct { ... } _FLEXCAN_HandleType::tSettings

CAN global settings, include callback function and special value like buffer address.

struct { ... } _FLEXCAN_HandleType::tStatus

can inner status tStatus Can't be set by user

uint8_t _FLEXCAN_HandleType::u8MbStart

if legacy rx fifo enabled, mb start after n/4+6

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

_flexspi_transfer Struct Reference

FlexSPI Transfer structure definition.

```
#include <module_driver_ospi.h>
```

Public Attributes

```
uint32_t deviceAddress  
OSPI_OperationType cmdType  
uint8_t seqIndex  
uint32_t * data  
uint16_t dataSize
```

Detailed Description

FlexSPI Transfer structure definition.

This structure defines the parameters required for a FlexSPI transfer operation.

Member Data Documentation

OSPI_OperationType _flexspi_transfer::cmdType

Execution command type.

uint32_t* _flexspi_transfer::data

Data buffer.

uint16_t _flexspi_transfer::dataSize

Data size in bytes.

uint32_t _flexspi_transfer::deviceAddress

Operation device address.

uint8_t _flexspi_transfer::seqIndex

Sequence ID for command.

The documentation for this struct was generated from the following file:

`module_driver_ospi.h`

_FREQM_HandleType Struct Reference

The structure of the FREQM processing handle.

```
#include <module_driver_freqm.h>
```

Public Attributes

FREQM_InstanceType eInstance

uint8_t **_aligned** [3U]

struct {

void(* **pMesCntStartCallback**)(struct **_FREQM_HandleType**
*pHandle, uint32_t u32Value)

void(* **pMesCntStopCallback**)(struct **_FREQM_HandleType**
*pHandle, uint32_t u32Value)

void(* **pRefCntStopCallback**)(struct **_FREQM_HandleType**
*pHandle)

void(* **pErrorCallback**)(struct **_FREQM_HandleType**
*pHandle)

} **tSettings**

Detailed Description

The structure of the FREQM processing handle.

Member Data Documentation

uint8_t **_FREQM_HandleType::_aligned**[3U]

FREQM_InstanceType _FREQM_HandleType::eInstance

FREQM instance

void(* _FREQM_HandleType::pErrorCallback) (struct _FREQM_HandleType *pHandle)

void(* _FREQM_HandleType::pMesCntStartCallback) (struct _FREQM_HandleType *pHandle, uint32_t u32Value)

void(* _FREQM_HandleType::pMesCntStopCallback) (struct _FREQM_HandleType *pHandle, uint32_t u32Value)

void(* _FREQM_HandleType::pRefCntStopCallback) (struct _FREQM_HandleType *pHandle)

struct { ... } _FREQM_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_freqm.h

_FTU_HandleType Struct Reference

The structure of the FTU processing handle.

```
#include <module_driver_ftu.h>
```

Public Attributes

FTU_InstanceType eInstance

struct {

FTU_ClkSrcType eFtuClkSrc

FTU_PwmUpdateModeType aPwmUpdateMode

FTU_PwmAlignedType aPwmAlignedMode

uint8_t **u8PhaseShiftEnable**

uint32_t **u32PwmPeriod**

void(* **pChannelCallback**)(struct **_FTU_HandleType**

*pHandle, uint32_t u32Channel,

uint32_t u32Timestamp)

void(* **pFaultCallback**)(struct **_FTU_HandleType**

*pHandle, uint32_t u32Fault)

void(* **pOverflowCallback**)(struct **_FTU_HandleType** *pHandle)

void(* **pReloadPointCallback**)(struct **_FTU_HandleType** *pHandle)

} **tStatus**

Detailed Description

The structure of the FTU processing handle.

Member Data Documentation

FTU_PwmAlignedType _FTU_HandleType::aPwmAlignedMode

pwm Aligned Mode

FTU_PwmUpdateModeType _FTU_HandleType::aPwmUpdateMode

pwm Update Mode of Period and duty

FTU_ClkSrcType _FTU_HandleType::eFtuClkSrc

Ftu clock source

FTU_InstanceType _FTU_HandleType::eInstance

FTU instance

void(* _FTU_HandleType::pChannelCallback) (struct _FTU_HandleType *pHandle, uint32_t u32Channel, uint32_t u32Timestamp)

channel interrupt callback

void(* _FTU_HandleType::pFaultCallback) (struct _FTU_HandleType *pHandle, uint32_t u32Fault)

fault interrupt callback

void(* _FTU_HandleType::pOverflowCallback) (struct _FTU_HandleType *pHandle)

overflow interrupt callback

void(* _FTU_HandleType::pReloadPointCallback) (struct _FTU_HandleType *pHandle)

reload point interrupt callback

struct { ... } _FTU_HandleType::tStatus

uint32_t _FTU_HandleType::u32PwmPeriod

pwm period (timer ticks)

uint8_t _FTU_HandleType::u8PhaseShiftEnable

phase shift enable flag of each channel

The documentation for this struct was generated from the following file:

module_driver_ftu.h

_HSADC_HandleType Struct Reference

The structure of the CMU processing handle Implements : **_HSADC_HandleType**.

#include <module_driver_hsadc.h>

Public Attributes

HSADC_InstanceType eInstance

struct {

HSADC_SeqModeType eSeqMode

uint32_t u32FunctionClockFreq

uint8_t u8SeqGroupCnt

uint8_t u8ChannelCnt

uint8_t u8LastCompleteSeqGroup

bool bSeqCompleteIntEn

bool bConvCompleteIntEn

bool bOverRunIntEn

bool bEndOfSeqGroupIntEn

void(* pSeqCompleteNotify)(struct **_HSADC_HandleType**
*pHandle)

void(* pOverRunNotify)(struct **_HSADC_HandleType**
*pHandle)

void(* pCmpNotify)(struct **_HSADC_HandleType**
*pHandle)

void(* pEndOfSeqGroupNotify)(struct **_HSADC_HandleType**
*pHandle)

void(* pConvCompleteNotify)(struct **_HSADC_HandleType**
*pHandle)

uint32_t * pResultBuffer

uint32_t * pSeqGroupResultBuffer [HSADC_SEQ_GROUP_COUNT]

} **tSettings**

Detailed Description

The structure of the CMU processing handle Implements : **_HSADC_HandleType**.

Member Data Documentation

bool _HSADC_HandleType::bConvCompleteIntEn

Enable interrupt when conversion completed

bool _HSADC_HandleType::bEndOfSeqGroupIntEn

Enable interrupt when sequence group conversion completed

bool _HSADC_HandleType::bOverRunIntEn

Enable interrupt when overrun occurred

bool _HSADC_HandleType::bSeqCompleteIntEn

Enable interrupt when sequence completed

HSADC_InstanceType _HSADC_HandleType::eInstance

HSADC instance

HSADC_SeqModeType _HSADC_HandleType::eSeqMode

HSADC sequence mode

void(* _HSADC_HandleType::pCmpNotify) (struct _HSADC_HandleType *pHandle)

Compare interrupt Notify

void(* _HSADC_HandleType::pConvCompleteNotify) (struct _HSADC_HandleType *pHandle)

Conversion complete interrupt Notify

void(* _HSADC_HandleType::pEndOfSeqGroupNotify) (struct _HSADC_HandleType *pHandle)

Sequence group complete interrupt Notify

void(* _HSADC_HandleType::pOverRunNotify) (struct _HSADC_HandleType *pHandle)

Overrun interrupt Notify

uint32_t* _HSADC_HandleType::pResultBuffer

Result buffer for eSeqMode != HSADC_SEQ_GROUP_MODE

void(* _HSADC_HandleType::pSeqCompleteNotify) (struct _HSADC_HandleType *pHandle)

Sequence complete interrupt Notify

uint32_t* _HSADC_HandleType::pSeqGroupResultBuffer[HSADC_SEQ_GROUP_COUNT]

Result buffer for eSeqMode == HSADC_SEQ_GROUP_MODE

struct { ... } _HSADC_HandleType::tSettings

uint32_t _HSADC_HandleType::u32FunctionClockFreq

HSADC function clock frequency

uint8_t _HSADC_HandleType::u8ChannelCnt

Number of channels

uint8_t _HSADC_HandleType::u8LastCompleteSeqGroup

Last completed sequence group index for eSeqMode == HSADC_SEQ_GROUP_MODE

uint8_t _HSADC_HandleType::u8SeqGroupCnt

Number of sequence groups, only valid for eSeqMode == HSADC_SEQ_GROUP_MODE

The documentation for this struct was generated from the following file:

module_driver_hsadc.h

_INTM_HandleType Struct Reference

The structure of the INTM handle.

```
#include <module_driver_intm.h>
```

Public Attributes

```
INTM_InstanceType eInstance  
char _aligned [3]  
struct {  
    void(* pIntmIsrCallback [INTM_IRQ_MONITOR_MAX])(struct  
        _INTM_HandleType *pHandle)  
} tSettings
```

Detailed Description

The structure of the INTM handle.

Member Data Documentation

char _INTM_HandleType::_aligned[3]

INTM_InstanceType _INTM_HandleType::eInstance

INTM instance

void(* _INTM_HandleType::pIntmIsrCallback[INTM_IRQ_MONITOR_MAX])(struct _INTM_HandleType *pHandle)

INTM interrupt callback

struct { ... } _INTM_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_intm.h

_ISM_HandleType Struct Reference

#include <module_driver_ism.h>

Public Attributes

boolean **bEnable**

boolean **bIntEnable**

void(* **pEventIsrCalllback**)(struct **_ISM_HandleType** *pHandle, uint16_t u16Lamflags)

void(* **pEcmEventIsrCalllback**)(struct **_ISM_HandleType** *pHandle, uint8_t u8EcmFlags)

struct {

uint32_t **u32Value**

uint32_t **u32LamChannelRouter**

} **tEcmEventStatus** [ECM_INSTANCE_COUNT]

struct {

bool **bFpcChannelEn**

bool **bGlitchIntEnable**

ISM_FPC_EdgeDetectModeType **eFallingDetectMode**

ISM_FPC_EdgeDelayModeType **eFallingDelayNode**

ISM_FPC_EdgeDetectModeType **eRisingDetectMode**

ISM_FPC_EdgeDelayModeType **eRisingDelayNode**

uint16_t **u32ThresholdValue**

void(* **pFpcChannelCalllback**)(struct **_ISM_HandleType**
*pHandle, uint32_t u32channel)

} **tIsmFpcChannelStatus** [FPC_INSTANCE_COUNT]

struct {

bool **bLamChannelEn**

bool **bOvflIntEnable**

uint8_t **u8RcsSel**

uint8_t **u8MonSel**

ISM_LAM_InvertEventWindowType **eInvWin**

ISM_LAM_EventWindowEdgeType **eWinEdgSel**

ISM_LAM_EventWindowSelectType **eEvtWinSel**

ISM_LAM_RunModeSelectType **eRunMode**

ISM_LAM_MonitorSourceType **eMonSrcSel**

ISM_LAM_InvertMonitorType **eInvMon**

ISM_LAM_InvertReferenceType **eInvRef**

uint32_t **u32EvtCntThreshold**

```
void(* pLamChannelOverCalllback )(struct _ISM_HandleType  
    *pHandle, uint32_t u32channel)  
} tlsmLamChannelStatus [LAM_INSTANCE_COUNT]
```

Member Data Documentation

boolean _ISM_HandleType::bEnable

bool _ISM_HandleType::bFpcChannelEn

bool _ISM_HandleType::bGlitchIntEnable

boolean _ISM_HandleType::bIntEnable

bool _ISM_HandleType::bLamChannelEn

bool _ISM_HandleType::bOvflIntEnable

ISM_LAM_EventWindowSelectType _ISM_HandleType::eEvtWinSel

ISM_FPC_EdgeDelayModeType _ISM_HandleType::eFallingDelayNode

ISM_FPC_EdgeDetectModeType _ISM_HandleType::eFallingDetectMode

ISM_LAM_InvertMonitorType _ISM_HandleType::eInvMon

ISM_LAM_InvertReferenceType _ISM_HandleType::eInvRef

ISM_LAM_InvertEventWindowType _ISM_HandleType::eInvWin

ISM_LAM_MonitorSourceType _ISM_HandleType::eMonSrcSel

ISM_FPC_EdgeDelayModeType _ISM_HandleType::eRisingDelayNode

ISM_FPC_EdgeDetectModeType _ISM_HandleType::eRisingDetectMode

ISM_LAM_RunModeSelectType _ISM_HandleType::eRunMode

ISM_LAM_EventWindowEdgeType _ISM_HandleType::eWinEdgSel

void(* _ISM_HandleType::pEcmEventIsrCalllback) (struct _ISM_HandleType *pHandle, uint8_t u8EcmFlags)

void(* _ISM_HandleType::pEventIsrCalllback) (struct _ISM_HandleType *pHandle, uint16_t u16Lamflags)

```
void(* _ISM_HandleType::pFpcChannelCalllback) (struct _ISM_HandleType *pHandle, uint32_t
u32channel)
```

```
void(* _ISM_HandleType::pLamChannelOverCalllback) (struct _ISM_HandleType *pHandle, uint32_t
u32channel)
```

```
struct { ... } _ISM_HandleType::tEcmEventStatus[ECM_INSTANCE_COUNT]
```

```
struct { ... } _ISM_HandleType::tIsmFpcChannelStatus[FPC_INSTANCE_COUNT]
```

```
struct { ... } _ISM_HandleType::tIsmLamChannelStatus[LAM_INSTANCE_COUNT]
```

```
uint32_t _ISM_HandleType::u32EvtCntThreshold
```

```
uint32_t _ISM_HandleType::u32LamChannelRouter
```

```
uint16_t _ISM_HandleType::u32ThresholdValue
```

```
uint32_t _ISM_HandleType::u32Value
```

```
uint8_t _ISM_HandleType::u8MonSel
```

```
uint8_t _ISM_HandleType::u8RcsSel
```

The documentation for this struct was generated from the following file:

module_driver_ism.h

_LIN_HandleType Struct Reference

LIN driver handle structure (entry point for all API operations)

#include <module_driver_lin.h>

Collaboration diagram for _LIN_HandleType:

IMAGE

Public Attributes

uint8_t u8LinChannel

LIN_XferStateType * pXfer

LIN_ConfigType * pCfg

Detailed Description

LIN driver handle structure (entry point for all API operations)

Connects configuration, runtime state, and hardware instance.

Member Data Documentation

LIN_ConfigType* _LIN_HandleType::pCfg

Pointer to LIN configuration structure

LIN_XferStateType* _LIN_HandleType::pXfer

Pointer to LIN transfer state structure

uint8_t _LIN_HandleType::u8LinChannel

LIN controller channel ID (matches u8LinHwIndex in **LIN_ConfigType**)

The documentation for this struct was generated from the following file:

module_driver_lin.h

_MB_HandleType Struct Reference

```
#include <module_driver_mb.h>
```

Public Attributes

```
struct {  
    uint8_t u8CoreIndex  
    uint8_t u8MasterId  
    void(* pRequestCallback)(struct _MB_HandleType  
        *pHandle, MB_ReceiveType  
        *pReceive)  
    void(* pDoneCallback)(struct _MB_HandleType  
        *pHandle, uint32_t u32Channel)  
} tStatus
```

Member Data Documentation

```
void(* _MB_HandleType::pDoneCallback) (struct _MB_HandleType *pHandle, uint32_t u32Channel)
```

```
void(* _MB_HandleType::pRequestCallback) (struct _MB_HandleType *pHandle, MB_ReceiveType  
*pReceive)
```

```
struct { ... }  _MB_HandleType::tStatus
```

```
uint8_t _MB_HandleType::u8CoreIndex
```

```
uint8_t _MB_HandleType::u8MasterId
```

The documentation for this struct was generated from the following file:

```
module_driver_mb.h
```

_MSC_HandleType Struct Reference

```
#include <module_driver_msc.h>
```

Public Attributes

```
MSC_InstanceType eInstance  
uint8_t reserve0 [3]  
struct {  
    void(* pISRCallback)(const struct _MSC_HandleType  
        *pMscHandle, const uint32_t  
        u32Status)  
    void(* pTOISRCallback)(const struct _MSC_HandleType  
        *pMscHandle)  
    bool bToEnableFlag  
} tSettings
```

Member Data Documentation

bool _MSC_HandleType::bToEnableFlag

MSC_InstanceType _MSC_HandleType::eInstance

void(* _MSC_HandleType::pISRCallback) (const struct **_MSC_HandleType** *pMscHandle, const uint32_t u32Status)

void(* _MSC_HandleType::pTOISRCallback) (const struct **_MSC_HandleType** *pMscHandle)

uint8_t _MSC_HandleType::reserve0[3]

struct { ... } _MSC_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_msc.h

_PMC_HandleType Struct Reference

The structure of the SCG processing handle.

```
#include <module_driver_pmc.h>
```

Public Attributes

```
void(* pPmcFaultNotify )(struct _PMC_HandleType *pHandle, PMC_Fault_Type eFault)
```

Detailed Description

The structure of the SCG processing handle.

The documentation for this struct was generated from the following file:

```
module_driver_pmc.h
```

_PORT_HandleType Struct Reference

The structure of the PORT processing handle.

```
#include <module_driver_port.h>
```

Public Attributes

PORT_InstanceType eInstance

uint8_t **RESERVED_0** [3]

struct {

void(* **PORT_PinInterruptCallbackType**)(struct **_PORT_HandleType**
*pHandle, uint32_t u32PIns)

} **tSettings**

Detailed Description

The structure of the PORT processing handle.

Member Data Documentation

PORT_InstanceType _PORT_HandleType::eInstance

PORT instance

**void(* _PORT_HandleType::PORT_PinInterruptCallbackType) (struct _PORT_HandleType *pHandle,
uint32_t u32PIns)**

PORT user defined interrupt function

uint8_t _PORT_HandleType::RESERVED_0[3]

struct { ... } _PORT_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_port.h

_PTIMER_HandleType Struct Reference

Ptimer process handler.

```
#include <module_driver_ptimer.h>
```

Public Attributes

PTIMER_InstanceType **eInstance**

uint8_t **reserve0** [3]

struct {

uint8_t **u8ChnNum**

uint8_t **reserve1** [3]

void(* **pSeqErrorNotify**)(struct **_PTIMER_HandleType**
*pHandler, uint8_t u8Channel)

void(* **pIntNotify**)(struct **_PTIMER_HandleType**
*pHandler)

bool **bDelayIntEnable**

bool **bSeqErrIntEnable**

uint16_t **u16IntDelayPeriod**

} **tSettings**

Detailed Description

Ptimer process handler.

Member Data Documentation

bool **_PTIMER_HandleType::bDelayIntEnable**

Enable the interrupt for timer

bool **_PTIMER_HandleType::bSeqErrIntEnable**

Enable PTIMER Sequence Error Interrupt

PTIMER_InstanceType _PTIMER_HandleType::eInstance

Ptimer instance

void(* _PTIMER_HandleType::pIntNotify) (struct _PTIMER_HandleType *pHandler)

Ptimer interrupt callback

void(* _PTIMER_HandleType::pSeqErrorNotify) (struct _PTIMER_HandleType *pHandler, uint8_t u8Channel)

Ptimer sequence error callback

uint8_t _PTIMER_HandleType::reserve0[3]

uint8_t _PTIMER_HandleType::reserve1[3]

struct { ... } _PTIMER_HandleType::tSettings

uint16_t _PTIMER_HandleType::u16IntDelayPeriod

The period cnt

uint8_t _PTIMER_HandleType::u8ChnNum

The documentation for this struct was generated from the following file:

module_driver_ptimer.h

_RGM_HandleType Struct Reference

The structure of the SCG processing handle.

```
#include <module_driver_rgm.h>
```

Public Attributes

```
void(* pRgmCpu0ResetInterruptNotify )(struct _RGM_HandleType *pHandle, uint32_t u32ResetReson)
```

Detailed Description

The structure of the SCG processing handle.

Member Data Documentation

```
void(* _RGM_HandleType::pRgmCpu0ResetInterruptNotify )(struct _RGM_HandleType *pHandle, uint32_t u32ResetReson)
```

Cpu0 reset interrupt callback

The documentation for this struct was generated from the following file:

```
module_driver_rgm.h
```


_RTC_HandleType Struct Reference

The structure of the RTC processing handle.

```
#include <module_driver_rtc.h>
```

Public Attributes

RTC_InstanceType eInstance

uint32_t **u32AlarmValue**

struct {

void(* **pAlarmCallback**)(struct _RTC_HandleType *pHandle)

void(* **pSecondCallback**)(struct _RTC_HandleType *pHandle)

void(* **pOverflowCallback**)(struct _RTC_HandleType *pHandle)

} **tSettings**

Detailed Description

The structure of the RTC processing handle.

Member Data Documentation

RTC_InstanceType _RTC_HandleType::eInstance

RTC instance

void(* _RTC_HandleType::pAlarmCallback) (struct _RTC_HandleType *pHandle)

Alarm interrupt callback

void(* _RTC_HandleType::pOverflowCallback) (struct _RTC_HandleType *pHandle)

Overflow interrupt callback

void(* _RTC_HandleType::pSecondCallback) (struct _RTC_HandleType *pHandle)

Second interrupt callback

```
struct { ... } _RTC_HandleType::tSettings
```

```
uint32_t _RTC_HandleType::u32AlarmValue
```

The documentation for this struct was generated from the following file:

```
module_driver_rtc.h
```

_SCG_HandleType Struct Reference

The structure of the SCG processing handle.

```
#include <module_driver_scg.h>
```

Public Attributes

```
void(* pFoscClkErrNotify )(struct _SCG_HandleType *pHandle)
void(* pSircClkErrNotify )(struct _SCG_HandleType *pHandle)
void(* pFircClkErrNotify )(struct _SCG_HandleType *pHandle)
void(* pSoscClkErrNotify )(struct _SCG_HandleType *pHandle)
void(* pPll0ClkErrNotify )(struct _SCG_HandleType *pHandle)
void(* pPll1ClkErrNotify )(struct _SCG_HandleType *pHandle)
```

Detailed Description

The structure of the SCG processing handle.

Member Data Documentation

```
void(* _SCG_HandleType::pFircClkErrNotify) (struct _SCG_HandleType *pHandle)
```

FIRC clock error interrupt callback

```
void(* _SCG_HandleType::pFoscClkErrNotify) (struct _SCG_HandleType *pHandle)
```

FOSC clock error interrupt callback

```
void(* _SCG_HandleType::pPll0ClkErrNotify) (struct _SCG_HandleType *pHandle)
```

Pll0 clock error interrupt callback

```
void(* _SCG_HandleType::pPll1ClkErrNotify) (struct _SCG_HandleType *pHandle)
```

Pll0 clock error interrupt callback

```
void(* _SCG_HandleType::pSircClkErrNotify) (struct _SCG_HandleType *pHandle)
```

SIRC clock error interrupt callback

void(* _SCG_HandleType::pSoscClkErrNotify) (struct _SCG_HandleType *pHandle)

SOSC clock error interrupt callback

The documentation for this struct was generated from the following file:

module_driver_scg.h

_SDADC_HandleType Struct Reference

#include <module_driver_sdadc.h>

Public Attributes

SDADC_InstanceType eInstance

uint8_t reserve0 [3]

struct {

void(* **pCOCISRCallback**)(const struct

_SDADC_HandleType

*pSdadchandle, uint8_t

u8Channel)

void(* **pFIFOISRcallback**)(const struct

_SDADC_HandleType

*pSdadchandle, uint8_t

u8Channel,

SDADC_FIFOIsrStatusType

eStatus)

void(* **pDetectISRcallback**)(const struct

_SDADC_HandleType

*pSdadchandle, uint8_t

u8Channel,

SDADC_DetectIsrStatusType

eStatus)

void(* **pSignDelayISRcallback**)(const struct

_SDADC_HandleType

*pSdadchandle, uint8_t

u8Channel)

void(* **pTSignChangeISRcallback**)(const struct

_SDADC_HandleType

*pSdadchandle, uint8_t

u8Channel)

void(* **pTSValidISRcallback**)(const struct

_SDADC_HandleType

*pSdadchandle, uint8_t

u8Channel)

} **tSettings**

Member Data Documentation

SDADC_InstanceType _SDADC_HandleType::eInstance

ADC instance

void(* _SDADC_HandleType::pCOCISRcallback) (const struct _SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for Conversion Complete Interrupt (COCI)

void(* _SDADC_HandleType::pDetectISRcallback) (const struct _SDADC_HandleType *pSdadCHandle, uint8_t u8Channel, SDADC_DetectIsrStatusType eStatus)

Callback function for Detection Interrupt

void(* _SDADC_HandleType::pFIFOISRcallback) (const struct _SDADC_HandleType *pSdadCHandle, uint8_t u8Channel, SDADC_FIFOIsrStatusType eStatus)

Callback function for FIFO Interrupt

void(* _SDADC_HandleType::pSignDelayISRcallback) (const struct _SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for Sign Delay Interrupt

void(* _SDADC_HandleType::pTSignChangeISRcallback) (const struct _SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for TSign Change Interrupt

void(* _SDADC_HandleType::pTSValidISRcallback) (const struct _SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for TSign Valid Interrupt

uint8_t _SDADC_HandleType::reserve0[3]

struct { ... } _SDADC_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

_SDDF_HandleType Struct Reference

#include <module_driver_sddf.h>

Collaboration diagram for _SDDF_HandleType:

IMAGE

Public Attributes

SDDF_InstanceType eInstance

uint8_t **reserve0** [3]

struct {

SDDF_ChannelNotificationType Notifications [SDDF_CH_CNT]

} **tSettings**

Member Data Documentation

SDDF_InstanceType _SDDF_HandleType::eInstance

SDDF_ChannelNotificationType _SDDF_HandleType::Notifications[SDDF_CH_CNT]

uint8_t _SDDF_HandleType::reserve0[3]

struct { ... } _SDDF_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_sddf.h

_Sent_HandleType Struct Reference

The structure of the SENT processing handle.

```
#include <module_driver_sent.h>
```

Public Attributes

Sent_InstanceType eInstance

```
struct {
```

```
    bool bInitStatus
```

```
} tStatus
```

```
struct {
```

```
    SENT_GBISRCallbackType pGBCallback
```

```
    SENT_CHISRCallbackType pCHCallback
```

```
} tSettings
```

Detailed Description

The structure of the SENT processing handle.

Member Data Documentation

bool _Sent_HandleType::bInitStatus

Sent instance initialize status

Sent_InstanceType _Sent_HandleType::eInstance

CAN instance

SENT_CHISRCallbackType _Sent_HandleType::pCHCallback

Channel interrupt callback function

SENT_GBISRCallbackType _Sent_HandleType::pGBCallback

Global interrupt callback function


```
struct { ... } _Sent_HandleType::tSettings
```

```
struct { ... } _Sent_HandleType::tStatus
```

The documentation for this struct was generated from the following file:

```
module_driver_sent.h
```

_TMU_HandleType Struct Reference

The structure of the TMU processing handle.

```
#include <module_driver_tmu.h>
```

Public Attributes

TMU_InstanceType eIntance

uint8_t **reserve** [3]

struct {

void(* **TMU_TempOver150InterruptCallback**)(struct **_TMU_HandleType** *pHandle)

void(* **TMU_TempOver125InterruptCallback**)(struct **_TMU_HandleType** *pHandle)

void(* **TMU_TempFlagReadyInterruptCallback**)(struct **_TMU_HandleType** *pHandle)

void(* **TMU_TempVoltageReadyInterruptCallback**)(struct **_TMU_HandleType** *pHandle)

} **tSettings**

struct {

bool **bTemperatureOver150IntEn**

bool **bTemperatureOver125IntEn**

bool **bFlagTempReadyIntEn**

bool **bVoltageTempReadyIntEn**

} **tStatus**

Detailed Description

The structure of the TMU processing handle.

Member Data Documentation

bool _TMU_HandleType::bFlagTempReadyIntEn

The enable status of interrupt when Flag-based temperature sensor ready

bool _TMU_HandleType::bTemperatureOver125IntEn

The enable status of interrupt when Temperature over 125 Celsius

bool _TMU_HandleType::bTemperatureOver150IntEn

The enable status of interrupt when Temperature over 150 Celsius

bool _TMU_HandleType::bVoltageTempReadyIntEn

The enable status of interrupt when Voltage-based temperature sensor ready

TMU_InstanceType _TMU_HandleType::eIntance

uint8_t _TMU_HandleType::reserve[3]

@ TMU instance

void(* _TMU_HandleType::TMU_TempFlagReadyInterruptCallback) (struct _TMU_HandleType *pHandle)

@ The Flag-based temperature sensor over 125 Celsius interrupt callback.

void(* _TMU_HandleType::TMU_TempOver125InterruptCallback) (struct _TMU_HandleType *pHandle)

@ The Flag-based temperature sensor over 150 Celsius interrupt callback.

void(* _TMU_HandleType::TMU_TempOver150InterruptCallback) (struct _TMU_HandleType *pHandle)

void(* _TMU_HandleType::TMU_TempVoltageReadyInterruptCallback) (struct _TMU_HandleType *pHandle)

@ The Flag-based temperature sensor ready interrupt callback.

struct { ... } _TMU_HandleType::tSettings

struct { ... } _TMU_HandleType::tStatus

The documentation for this struct was generated from the following file:

module_driver_tmu.h

_TSTMP_HandleType Struct Reference

The structure of the Tstmp processing handle.

```
#include <module_driver_tstmp.h>
```

Public Attributes

TSTMP_InstanceType eInstance

struct {

uint32_t **u32ModulateValue** [MAX_MOD_NUMBER]

bool **bModulateIntEn** [MAX_MOD_NUMBER]

void(* **pChannelCallback** [MAX_MOD_NUMBER])(struct

_TSTMP_HandleType *pHandle,

 uint32_t u32channel)

} **tStatus**

Detailed Description

The structure of the Tstmp processing handle.

Member Data Documentation

bool _TSTMP_HandleType::bModulateIntEn[MAX_MOD_NUMBER]

TSTMP_InstanceType _TSTMP_HandleType::eInstance

TSTMP instance

void(* _TSTMP_HandleType::pChannelCallback[MAX_MOD_NUMBER])(struct **_TSTMP_HandleType**
***pHandle**, uint32_t u32channel)

struct { ... } _TSTMP_HandleType::tStatus

uint32_t _TSTMP_HandleType::u32ModulateValue[MAX_MOD_NUMBER]

modulate value

The documentation for this struct was generated from the following file:

`module_driver_tstmp.h`

_WDOG_HandleType Struct Reference

The structure of the WDOG processing handle.

```
#include <module_driver_wdog.h>
```

Public Attributes

WDOG_InstanceType eIntance

uint8_t **reserve** [3]

struct {

void(* **plSRCallback**)(struct **_WDOG_HandleType** *pHandle)

} **tSettings**

Detailed Description

The structure of the WDOG processing handle.

Member Data Documentation

WDOG_InstanceType _WDOG_HandleType::eIntance

void(* _WDOG_HandleType::plSRCallback) (struct _WDOG_HandleType *pHandle)

uint8_t _WDOG_HandleType::reserve[3]

@ WDOG instance

struct { ... } _WDOG_HandleType::tSettings

The documentation for this struct was generated from the following file:

module_driver_wdog.h

ADC_ChannelCfgType Struct Reference

The configuration option for the ADC channel.

```
#include <module_driver_adc.h>
```

Public Attributes

ADC_ChannelType eChannel

ADC_SampleTimeOptionType eSampleTimeOption

Detailed Description

The configuration option for the ADC channel.

Member Data Documentation

ADC_ChannelType ADC_ChannelCfgType::eChannel

Selected ADC channel

ADC_SampleTimeOptionType ADC_ChannelCfgType::eSampleTimeOption

The sample time selection for the channel

The documentation for this struct was generated from the following file:

module_driver_adc.h

ADC_CompareType Struct Reference

Defines the hardware compare configuration.

#include <module_driver_adc.h>

Public Attributes

bool **bCmpEnable**

ADC_CmpChannelType **eCmpSingleChn**

uint8_t **u8CmpChnSel**

uint16_t **u16HighThres**

uint16_t **u16LowThres**

Detailed Description

Defines the hardware compare configuration.

This structure is used to configure the hardware compare feature for the ADC

Implements : **ADC_CompareType**

Member Data Documentation

bool ADC_CompareType::bCmpEnable

Enable hardware compare

ADC_CmpChannelType ADC_CompareType::eCmpSingleChn

0: ADC compare on all channels; 1: ADC compare on the selected channel

uint16_t ADC_CompareType::u16HighThres

Compare high threshold

uint16_t ADC_CompareType::u16LowThres

Compare low threshold

uint8_t ADC_CompareType::u8CmpChnSel

Compare channel selection

The documentation for this struct was generated from the following file:

`module_driver_adc.h`

ADC_InitType Struct Reference

Defines the converter configuration.

#include <module_driver_adc.h>

Public Attributes

ADC_ResolutionType eResolution
ADC_AlignType eAlign
ADC_TriggerModeType eTrgMode
bool bWaitEn
ADC_ClockDivideType eClockDivider
ADC_SeqModeType eSeqMode
bool bAutoDis
ADC_OvrModeType eOverRunMode
ADC_RefType eVoltageRef
bool bAverageEn
ADC_AverageLenType eAverageLen
uint8_t aSampleTimes [ADC_SAMPLE_TIME_OPTION_CNT]
bool bDmaEn
bool bConvCompleteIntEn
bool bOverRunIntEn
bool bCompareIntEn
void(* pConvCompleteNotify)(struct **_ADC_HandleType** *pHandle)
void(* pOverRunNotify)(struct **_ADC_HandleType** *pHandle)
void(* pCompareNotify)(struct **_ADC_HandleType** *pHandle)
uint32_t * pResultBuffer

Detailed Description

Defines the converter configuration.

This structure is used to configure the ADC converter

Implements : **ADC_InitType**

Member Data Documentation

uint8_t ADC_InitType::aSampleTimes[ADC_SAMPLE_TIME_OPTION_CNT]

ADC sample time options, range: 4 ~ 257

bool ADC_InitType::bAutoDis

Whether to enable auto disable mode, only set this when adc in off state

bool ADC_InitType::bAverageEn

Enable averaging functionality

bool ADC_InitType::bCompareIntEn

Enable interrupt when conversion result lays in the compare threshold

bool ADC_InitType::bConvCompleteIntEn

Enable interrupt when conversion completed

bool ADC_InitType::bDmaEn

Enable DMA for the HSADC

bool ADC_InitType::bOverRunIntEn

Enable interrupt when overrun occurred

bool ADC_InitType::bWaitEn

Whether to enable ADC wait conversion mode

ADC_AlignType ADC_InitType::eAlign

ADC alignment (left, right)

ADC_AverageLenType ADC_InitType::eAverageLen

Selection for number of samples used for averaging

ADC_ClockDivideType ADC_InitType::eClockDivider

ADC clock divider

ADC_OvrModeType ADC_InitType::eOverRunMode

Whether to preserve data when ADC overruns

ADC_ResolutionType ADC_InitType::eResolution

ADC eResolution (8,10,12 bit)

ADC_SeqModeType ADC_InitType::eSeqMode

ADC sequence mode (single, continuous, discontinuous)

ADC_TriggerModeType ADC_InitType::eTrgMode

ADC trigger type (software, hardware)

ADC_RefType ADC_InitType::eVoltageRef

Voltage reference used (external, internal)

void(* ADC_InitType::pCompareNotify) (struct _ADC_HandleType *pHandle)

Compare interrupt callback

void(* ADC_InitType::pConvCompleteNotify) (struct _ADC_HandleType *pHandle)

Conversion complete interrupt callback

void(* ADC_InitType::pOverRunNotify) (struct _ADC_HandleType *pHandle)

Overrun interrupt callback

uint32_t* ADC_InitType::pResultBuffer

only used for mode 0~3

The documentation for this struct was generated from the following file:

module_driver_adc.h

AONTIMER_InitType Struct Reference

Aontimer Initialization struct type.

```
#include <module_driver_aontimer.h>
```

Public Attributes

AONTIMER_ModeType eMode

bool bBypassEn

AONTIMER_PulseClkSrcType ePulseClkSrc

AONTIMER_PulsePolarityType ePulsePol

uint8_t u8PulseFilterWidth

uint8_t u8Prescaler

uint16_t u16StartValue

void(* pAontimerCallback)(AONTIMER_HandleType *pHandle)

bool bIntEn

AONTIMER_ClkSrcType eClkSrc

char _aligned0 [2]

Detailed Description

Aontimer Initialization struct type.

Member Data Documentation

char AONTIMER_InitType::_aligned0[2]

bool AONTIMER_InitType::bBypassEn

Whether to use prescaler in counter mode or use glitch filter in pulse mode, if set input value is true ,then not use the prescaler or glitch filter.

bool AONTIMER_InitType::bIntEn

Whether enable interrupt

AONTIMER_ClkSrcType AONTIMER_InitType::eClkSrc

clock source of counter mode

AONTIMER_ModeType AONTIMER_InitType::eMode

enumeration of aontier mode

AONTIMER_PulseClkSrcType AONTIMER_InitType::ePulseClkSrc

clock source of pulse mode

AONTIMER_PulsePolarityType AONTIMER_InitType::ePulsePol

polarity of pulse mode

void(* AONTIMER_InitType::pAontimerCallback) (AONTIMER_HandleType *pHandle)

the notification function of interrupt

uint16_t AONTIMER_InitType::u16StartValue

start value of counter mode ,the range is : 0 ~ 65535

uint8_t AONTIMER_InitType::u8Prescaler

the width of prescaler in counter mode, the range of the input value is :0~15, and the range of prescaler is : $2^1 \sim 2^{16}$.

uint8_t AONTIMER_InitType::u8PulseFilterWidth

the width of glitch filter in pulse mode , the range of the input value is :1~15, and the range of glitch filter is : $2^1 \sim 2^{15}$.

The documentation for this struct was generated from the following file:

module_driver_aontimer.h

CMP_ChannelScanType Struct Reference

Defines the comparator channel scan configuration.

```
#include <module_driver_cmp.h>
```

Public Attributes

uint8_t **u8CSInitModulus**

uint8_t **u8CSNSAM**

bool **bCSCompareResultAC**

Detailed Description

Defines the comparator channel scan configuration.

Note

This structure is used to configure CMP channel scan Implements : **CMP_ChannelScanType**

Member Data Documentation

bool CMP_ChannelScanType::bCSCompareResultAC

Enable/disable comparison result auto clear

uint8_t CMP_ChannelScanType::u8CSInitModulus

Compartor and DAC initialization delay modulus

uint8_t CMP_ChannelScanType::u8CSNSAM

Number of smaple clocks later the the sample operation takes place

The documentation for this struct was generated from the following file:

module_driver_cmp.h

CMP_ComparatorType Struct Reference

The instance index of the CMP Comparator parts.

```
#include <module_driver_cmp.h>
```

Public Attributes

- bool **bLPModeEn**
- bool **bDmaEn**
- bool **bOutToPackagePinEn**
- bool **bWinSampleInvertEn**
- bool **bEventCloseWinEn**
- uint8_t **u8FilterPeriod**
- uint32_t **u32InitDelay**
- CMP_ModeSelType** **eModeSel**
- CMP_InvertType** **eInvert**
- CMP_OutSelectType** **eOutSelect**
- CMP_OutWinLevelType** **eOutWinLevel**
- CMP_OutWinLevel_UserDefType** **eOutWin**
- CMP_EventType** **eEventSelect**
- CMP_FilterCntType** **eFilterCnt**
- CMP_SpeedModeSelType** **eSpeedMode**
- CMP_HystCtrlType** **eHystCtrl**

Detailed Description

The instance index of the CMP Comparator parts.

Member Data Documentation

bool CMP_ComparatorType::bDmaEn

CMP dma enable

bool CMP_ComparatorType::bEventCloseWinEn

CMP out event close window enable

bool CMP_ComparatorType::bLPModeEn

CMP low power mode enable

bool CMP_ComparatorType::bOutToPackagePinEn

CMP output to package pin enable

bool CMP_ComparatorType::bWinSampleInvertEn

CMP window/sample signal invert enable

CMP_EventType CMP_ComparatorType::eEventSelect

CMP output event cause window close(rising edge, falling edge)

CMP_FilterCntType CMP_ComparatorType::eFilterCnt

CMP filter count numbers(0,1,2,3,4)

CMP_HystCtrlType CMP_ComparatorType::eHystCtrl

CMP hysteresis internal control(0,1,2,3)

CMP_InvertType CMP_ComparatorType::eInvert

CMP output Invert or not(invert, non-invert)

CMP_ModeSelType CMP_ComparatorType::eModeSel

CMP set function mode

CMP_OutSelectType CMP_ComparatorType::eOutSelect

CMP output filter or not(CMPO = CMPOUT_FILTER/CMPOUT_WIN)

CMP_OutWinLevel_UserDefType CMP_ComparatorType::eOutWin

CMP output under userdefine (0, 1)

CMP_OutWinLevelType CMP_ComparatorType::eOutWinLevel

CMP output when window close(hold, userdefine)

CMP_SpeedModeSelType CMP_ComparatorType::eSpeedMode

CMP speed mode(low,high)

uint32_t CMP_ComparatorType::u32InitDelay

CMP initialization delay, the max initialization delay is 30us

uint8_t CMP_ComparatorType::u8FilterPeriod

CMP set Filter Sample Period

The documentation for this struct was generated from the following file:

module_driver_cmp.h

CMP_CSChannelCfgType Struct Reference

Defines the comparator channel selected in channel scan mode.

```
#include <module_driver_cmp.h>
```

Public Attributes

CMP_MuxSelType eChannel

bool **bPreSetState**

Detailed Description

Defines the comparator channel selected in channel scan mode.

Note

This structure is used to configure a CMP channel that want to be enabled in channel scan sequence

Implements : **CMP_CSChannelCfgType**

Member Data Documentation

bool CMP_CSChannelCfgType::bPreSetState

Preset state for the given channel

CMP_MuxSelType CMP_CSChannelCfgType::eChannel

Enabled channel

The documentation for this struct was generated from the following file:

module_driver_cmp.h

CMP_DacType Struct Reference

The instance index of the CMP Dac parts.

#include <module_driver_cmp.h>

Public Attributes

bool **bDacEn**

uint8_t **u8DacData**

CMP_DacEnableSrcType **eDacEnsrc**

CMP_DacVinRefSelType **eVinRefSel**

Detailed Description

The instance index of the CMP Dac parts.

Member Data Documentation

bool CMP_DacType::bDacEn

CMP DAC enable

CMP_DacEnableSrcType CMP_DacType::eDacEnsrc

CMP DAC enable selection(DCR,CCR0)

CMP_DacVinRefSelType CMP_DacType::eVinRefSel

CMP DAC high voltage reference (vrefh0,vrefh1)

uint8_t CMP_DacType::u8DacData

CMP DAC output voltage, output = $V_{in} / 256 * (aDacData + 1)$

The documentation for this struct was generated from the following file:

module_driver_cmp.h

CMP_InitType Struct Reference

Defines the comparator configuration.

```
#include <module_driver_cmp.h>
```

Collaboration diagram for CMP_InitType:

IMAGE

Public Attributes

CMP_DacType tDacConfig

CMP_MuxType tMuxConfig

CMP_ComparatorType tComparatorConfig

CMP_ChannelScanType tChannelScanConfig

CMP_InterruptType tInterruptConfig

Detailed Description

Defines the comparator configuration.

Note

This structure is used to configure CMP Implements : **CMP_InitType**

Member Data Documentation

CMP_ChannelScanType CMP_InitType::tChannelScanConfig

CMP Channel Scan configure

CMP_ComparatorType CMP_InitType::tComparatorConfig

CMP Comparator configure

CMP_DacType CMP_InitType::tDacConfig

CMP DAC configure

CMP_InterruptType CMP_InitType::tInterruptConfig

CMP Interrupt configure

CMP_MuxType CMP_InitType::tMuxConfig

CMP MUX configure

The documentation for this struct was generated from the following file:

module_driver_cmp.h

CMP_InterruptType Struct Reference

Defines the comparator interrupt configuration.

```
#include <module_driver_cmp.h>
```

Public Attributes

bool **bRisingIntEn**

bool **bFallingIntEn**

bool **bChannelScanFlagIntEn**

CMP_CompleteIntCallback **pInterrupterNotify**

Detailed Description

Defines the comparator interrupt configuration.

Note

This structure is used to configure CMP interrupt Implements : **CMP_InterruptType**

Member Data Documentation

bool CMP_InterruptType::bChannelScanFlagIntEn

Enable/disable channel scan flag interrupt

bool CMP_InterruptType::bFallingIntEn

Enable falling interrupt

bool CMP_InterruptType::bRisingIntEn

Enable rising interrupt

CMP_CompleteIntCallback CMP_InterruptType::pInterrupterNotify

CMP complete interrupt callback

The documentation for this struct was generated from the following file:

`module_driver_cmp.h`

CMP_MuxType Struct Reference

The instance index of the CMP Mux parts.

#include <module_driver_cmp.h>

Public Attributes

CMP_INSrcSelType eINPSel

CMP_INSrcSelType eINNSel

CMP_MuxSelType ePSelMux

CMP_MuxSelType eNSelMux

CMP_MuxSelType eCSFIXCMux

CMP_PortSelType eCSFIXPSel

Detailed Description

The instance index of the CMP Mux parts.

Member Data Documentation

CMP_MuxSelType CMP_MuxType::eCSFIXCMux

CMP channel scan fixed channel select(IN0...IN4), this is ignored if DAC is enabled

CMP_PortSelType CMP_MuxType::eCSFIXPSel

CMP channel scan fixed channel port select(positive or negative)

CMP_INSrcSelType CMP_MuxType::eINNSel

CMP input negative select(DAC analog mux)

CMP_INSrcSelType CMP_MuxType::eINPSel

CMP input positive select(DAC analog mux)

CMP_MuxSelType CMP_MuxType::eNSelMux

CMP input negative mux select(IN0...IN7)

CMP_MuxSelType CMP_MuxType::ePSelMux

CMP input positive mux select(IN0...IN7)

The documentation for this struct was generated from the following file:

module_driver_cmp.h

CMU_CfgType Struct Reference

CMU configure structure.

```
#include <module_driver_cmu.h>
```

Public Attributes

```
uint32_t u32RefClk  
uint32_t u32MonitorClk  
uint32_t u32RefWindow  
uint32_t u32MinMonitorCnts  
uint32_t u32MaxMonitorCnts  
uint32_t u32PerMonitorWindow  
bool bIntEnable  
bool bLpen  
bool bSten  
bool bEnable  
bool bPerMonitorEnable  
CMU_RefClockDivType eDiv  
uint8_t _aligned [2U]
```

Detailed Description

CMU configure structure.

Member Data Documentation

uint8_t **CMU_CfgType::_aligned**[2U]

bool **CMU_CfgType::bEnable**

CMU_CTRL[EN] bit, cmu enable.

bool **CMU_CfgType::bIntEnable**

CMU_CTRL[IRQ_EN] bit, interrupt enable, if this bit asserted, either LOC or MIS asserted can lead the interrupt.

bool CMU_CfgType::bLpen

CMU_CTRL[LP_EN] bit, standby mode enable, need STOP_EN asserted.

bool CMU_CfgType::bPerMonitorEnable

CMU_PERIOD[EN] bit, Period Monitor mode ENABLE.

bool CMU_CfgType::bSten

CMU_CTRL[STOP_EN] bit, stop mode enable.

CMU_RefClockDivType CMU_CfgType::eDiv

Divider of reference clock.

uint32_t CMU_CfgType::u32MaxMonitorCnts

CMU_MAX[MAX] bits, The maximum threshold for the monitor counter.

uint32_t CMU_CfgType::u32MinMonitorCnts

CMU_MIN[MIN] bits, The minimum threshold for the monitor counter.

uint32_t CMU_CfgType::u32MonitorClk

Monitor clock

uint32_t CMU_CfgType::u32PerMonitorWindow

CMU_PERIOD[WINDOW] bits, Period Monitor mode WINDOW.

uint32_t CMU_CfgType::u32RefClk

Reference clock

uint32_t CMU_CfgType::u32RefWindow

CMU_REF_WINDOW[REF_WINDOW] bits, The sample window for the reference clock.

The documentation for this struct was generated from the following file:

module_driver_cmu.h

CORDIC_SinCos_Type Struct Reference

Circular sine and cosine type.

#include <module_driver_cordic.h>

Public Attributes

float **sinx**

float **cosx**

Detailed Description

Circular sine and cosine type.

Member Data Documentation

float CORDIC_SinCos_Type::cosx

Cosine value.

float CORDIC_SinCos_Type::sinx

Sine value.

The documentation for this struct was generated from the following file:

module_driver_cordic.h

CORDIC_SinhCosh_Type Struct Reference

Circular hyperbolic sine and cosine type.

#include <module_driver_cordic.h>

Public Attributes

float **sinxh**

float **cosxh**

Detailed Description

Circular hyperbolic sine and cosine type.

Member Data Documentation

float **CORDIC_SinhCosh_Type::cosxh**

Hyperbolic cosine value.

float **CORDIC_SinhCosh_Type::sinxh**

Hyperbolic sine value.

The documentation for this struct was generated from the following file:

module_driver_cordic.h

CPM_InterruptType Struct Reference

Cpm interrupt configuration type.

```
#include <module_driver_cpm.h>
```

Public Attributes

uint8_t **u8CpmEnable**

FPU_IntType **eFPU_IntType**

void(* **pIsrNotify**)(CPM_HandleType *pHandle)

Detailed Description

Cpm interrupt configuration type.

Member Data Documentation

FPU_IntType CPM_InterruptType::eFPU_IntType

the type of interrupt

void(* CPM_InterruptType::pIsrNotify)(CPM_HandleType *pHandle)

Cpm interrupt notification function pointer

uint8_t CPM_InterruptType::u8CpmEnable

whether enable cpm interrupt

The documentation for this struct was generated from the following file:

module_driver_cpm.h

Crc_ConfigType Struct Reference

Defines the crc configuration.

```
#include <module_driver_crc.h>
```

Public Attributes

CRC_WriteDataSwapType eWriteDataSwap

CRC_ReadDataSwapType eReadDataSwap

CRC_ReadDataFXORType eReadDataFXOR

CRC_BitWidthType eBitWidth

eCrctype eType

uint8_t _aligned [2]

uint32_t u32Polynomial

uint32_t u32SeedValue

Detailed Description

Defines the crc configuration.

This structure is used to configure the crc

Implements : CRC_InitType

The documentation for this struct was generated from the following file:

module_driver_crc.h

Crc_HandleType Struct Reference

The structure of the SENT processing handle.

```
#include <module_driver_crc.h>
```

Public Attributes

```
Crc_InstanceType eInstance  
struct {  
    bool Crc_HandleType::bInitStatus  
} tStatus
```

Detailed Description

The structure of the SENT processing handle.

The documentation for this struct was generated from the following file:

```
module_driver_crc.h
```

CSC0_AONCLKSRType Struct Reference

CSC0_AONCLKSR clock source info.

```
#include <module_driver_csc.h>
```

Public Attributes

CSC0_AON32KClkSrcType eAon32KSel

CSC0_RTCClkSrcType eRtcSel

CSC0_AONClkSrcType eAonSel

Detailed Description

CSC0_AONCLKSR clock source info.

Member Data Documentation

CSC0_AON32KClkSrcType CSC0_AONCLKSRType::eAon32KSel

CSC0 32 KHz Always-on Clock Source Select

CSC0_AONClkSrcType CSC0_AONCLKSRType::eAonSel

CSC0 AON Clock Source Select

CSC0_RTCClkSrcType CSC0_AONCLKSRType::eRtcSel

CSC0 RTC Clock Source Select

The documentation for this struct was generated from the following file:

module_driver_csc.h

CSC0_ClkoutType Struct Reference

Data type for register CSC0_CLKOUT_CTRL,mainly focus on CLKOUT setting.

```
#include <module_driver_csc.h>
```

Public Attributes

```
bool bEnable  
CSC0_ClockOutSrcType eClkOutSrc  
CSC0_ClockOutDivType eDivider
```

Detailed Description

Data type for register CSC0_CLKOUT_CTRL,mainly focus on CLKOUT setting.

Member Data Documentation

bool CSC0_ClkoutType::bEnable

CSC0 ClockOut enable

CSC0_ClockOutSrcType CSC0_ClkoutType::eClkOutSrc

CSC0 ClockOut source select

CSC0_ClockOutDivType CSC0_ClkoutType::eDivider

CSC0 ClockOut divide ratio

The documentation for this struct was generated from the following file:

```
module_driver_csc.h
```

DMA_ChainTransferType Struct Reference

The paarameters to configure the DMA chained transfer.

```
#include <module_driver_dma.h>
```

Public Attributes

bool **bChanelChainEn**

uint8_t **u8ChainedChannel**

Detailed Description

The paarameters to configure the DMA chained transfer.

Member Data Documentation

bool DMA_ChainTransferType::bChanelChainEn

Whether to start the chained channel when the current completed

uint8_t DMA_ChainTransferType::u8ChainedChannel

The chained channel

The documentation for this struct was generated from the following file:

module_driver_dma.h

DMA_ChannelCfgType Struct Reference

The configuration parameters of the DMA channel.

```
#include <module_driver_dma.h>
```

Public Attributes

```
const volatile void * pSrcBuffer
volatile void * pDestBuffer
void * completedata
void * errdata
uint32_t u32BlockSize
uint16_t u16BlockCount
uint8_t u8ChannelPriority
DMA_TransferSizeType eSrcDataSize
DMA_TransferSizeType eDestDataSize
DMA_IncrementModeType eSrcIncMode
DMA_IncrementModeType eDestIncMode
bool bSrcBlockOffsetEn
bool bDestBlockOffsetEn
int32_t s32BlockOffset
bool bSrcAddrLoopbackEn
bool bDestAddrLoopbackEn
bool bAutoStop
bool bSrcCircularBufferEn
uint32_t u32SrcCircBufferSize
bool bDestCircularBufferEn
uint32_t u32DestCircBufferSize
bool bInnerChannelChain
DMA_RequestSourceType eTriggerSrc
bool bTransferCompleteIntEn
DMA_TransferCompleteCallbackType pTransferCompleteNotify
bool bTransferErrorIntEn
DMA_TransferErrorCallbackType pTransferErrorNotify
```

Detailed Description

The configuration parameters of the DMA channel.

Note

The `u8ChannelPriority` must be unique for different channels, the default is the channel number. The `u16BlockCount` must be greater than 0.

When Circular buffer is enabled, the data address should be power of 2 aligned to the buffer size. For example, if the circular buffer size is 32 byte, the data address should be 32 byte aligned. If the circular buffer size is 40 byte, then the data address should be 64 byte aligned.

Member Data Documentation

`bool DMA_ChannelCfgType::bAutoStop`

whether the DMA channel transfer automatically stops after one transfer finishes when the channel is triggered by hardware

`bool DMA_ChannelCfgType::bDestAddrLoopbackEn`

whether the destination address return to the set value after transfer

`bool DMA_ChannelCfgType::bDestBlockOffsetEn`

whether to add a block offset to the destination address after a block transfer

`bool DMA_ChannelCfgType::bDestCircularBufferEn`

the destination buffer of the DMA channel is a circular buffer

`bool DMA_ChannelCfgType::bInnerChannelChain`

`bool DMA_ChannelCfgType::bSrcAddrLoopbackEn`

whether the source address return to the set value after transfer

`bool DMA_ChannelCfgType::bSrcBlockOffsetEn`

whether to add a block offset to the source address after a block transfer

`bool DMA_ChannelCfgType::bSrcCircularBufferEn`

the source buffer of the DMA channel is a circular buffer

bool DMA_ChannelCfgType::bTransferCompleteIntEn

Enable interrupt after transfer complete

bool DMA_ChannelCfgType::bTransferErrorIntEn

Enable interrupt when transfer error occurred

void* DMA_ChannelCfgType::completedata

DMA_TransferSizeType DMA_ChannelCfgType::eDestDataSize

destination data size

DMA_IncrementModeType DMA_ChannelCfgType::eDestIncMode

destination data address increment mode

void* DMA_ChannelCfgType::errdata

DMA_TransferSizeType DMA_ChannelCfgType::eSrcDataSize

source data size

DMA_IncrementModeType DMA_ChannelCfgType::eSrcIncMode

source data address increment mode

DMA_RequestSourceType DMA_ChannelCfgType::eTriggerSrc

Select the DMA channel trigger source, if the trigger source is DMA_REQ_DISABLED, the channel is triggered by software

volatile void* DMA_ChannelCfgType::pDestBuffer

the destination address of the data

const volatile void* DMA_ChannelCfgType::pSrcBuffer

the source address of the data

DMA_TransferCompleteCallbackType DMA_ChannelCfgType::pTransferCompleteNotify

transfer complete notification

DMA_TransferErrorCallbackType DMA_ChannelCfgType::pTransferErrorNotify

transfer error notification

int32_t DMA_ChannelCfgType::s32BlockOffset

the signed address offset applied to the src/dest address after a block transfer

uint16_t DMA_ChannelCfgType::u16BlockCount

the number of data blocks in one transfer

uint32_t DMA_ChannelCfgType::u32BlockSize

the data size of one block in byte

uint32_t DMA_ChannelCfgType::u32DestCircBufferSize

the destination circular buffer size in byte, the buffer size is suggested to be power of 2 aligned

uint32_t DMA_ChannelCfgType::u32SrcCircBufferSize

the source circular buffer size in byte, the buffer size is suggested to be power of 2 aligned

uint8_t DMA_ChannelCfgType::u8ChannelPriority

channel priority, greater number means higher priority

The documentation for this struct was generated from the following file:

module_driver_dma.h

DMA_CircularBufferType Struct Reference

The parameters to configure the DMA channel as a circular buffer engine.

```
#include <module_driver_dma.h>
```

Public Attributes

DMA_CircularBufferRoleType eBufferRole

DMA_CircularBufferSizeType eBufferSize

bool bCircularBufferEn

Detailed Description

The parameters to configure the DMA channel as a circular buffer engine.

Member Data Documentation

bool DMA_CircularBufferType::bCircularBufferEn

enable the DMA channel as a circular buffer engine

DMA_CircularBufferRoleType DMA_CircularBufferType::eBufferRole

the source or the destination address is a circular buffer

DMA_CircularBufferSizeType DMA_CircularBufferType::eBufferSize

the circular buffer size

The documentation for this struct was generated from the following file:

module_driver_dma.h

DMA_InitType Struct Reference

The configuration parameters of the DMA engine.

#include <module_driver_dma.h>

Public Attributes

DMA_ArbitrationAlgorithmType eArbitrationAlgorithm

bool **bHaltOnError**

bool **bGRP0HighPriority**

Detailed Description

The configuration parameters of the DMA engine.

Member Data Documentation

bool DMA_InitType::bGRP0HighPriority

Whether the priority of DMA group 0 channel is higher than group 1 channel

bool DMA_InitType::bHaltOnError

Whether DMA halts when error occurred

DMA_ArbitrationAlgorithmType DMA_InitType::eArbitrationAlgorithm

Channel Arbitration Algorithm

The documentation for this struct was generated from the following file:

module_driver_dma.h

DMA_InstanceHandleType Struct Reference

The structure of the DMA processing handle.

#include <module_driver_dma.h>

Public Attributes

```
struct {  
    uint8_t u8DmaDumoUsedStatus [DMA_UMO_COUNT]  
} tStatus  
struct {  
    uint8_t u8Instance  
    void(* pTransferErrorCallback [DMA_CFG_COUNT])(void *arg)  
    void * errdata [DMA_CFG_COUNT]  
} tSettings
```

Detailed Description

The structure of the DMA processing handle.

Member Data Documentation

void* DMA_InstanceHandleType::errdata[DMA_CFG_COUNT]

void(* DMA_InstanceHandleType::pTransferErrorCallback[DMA_CFG_COUNT])(void *arg)

fault interrupt callback

struct { ... } DMA_InstanceHandleType::tSettings

struct { ... } DMA_InstanceHandleType::tStatus

uint8_t DMA_InstanceHandleType::u8DmaDumoUsedStatus[DMA_UMO_COUNT]

uint8_t DMA_InstanceHandleType::u8Instance

DMA Channel

The documentation for this struct was generated from the following file:

`module_driver_dma.h`

EIM_ChCfgType Struct Reference

Define the max EIM channels.

```
#include <module_driver_eim.h>
```

Public Attributes

```
uint8_t u8EimChn  
uint8_t u8BusSelIdx  
uint8_t u8Attreenable  
uint8_t u8AttrPosition  
uint8_t u8Addreenable  
uint8_t u8AddrePosition  
uint8_t u8Data0enable  
uint8_t u8Data0Val  
uint8_t u8Data1enable  
uint8_t u8Data1Val  
uint8_t u8MonNum  
uint8_t u8DwpMode
```

Detailed Description

Define the max EIM channels.

Member Data Documentation

uint8_t EIM_ChCfgType::u8Addreenable

EIM address error injection enable or not

uint8_t EIM_ChCfgType::u8AddrePosition

EIM address error injection position

uint8_t EIM_ChCfgType::u8Attreenable

EIM attribute error injection enable or not

uint8_t EIM_ChCfgType::u8AttrPosition

EIM attribute error injection position

uint8_t EIM_ChCfgType::u8BusSelIdx

EIM error bus No, See Error injection bus sel

uint8_t EIM_ChCfgType::u8Data0enable

EIM data0 error injection enable or not

uint8_t EIM_ChCfgType::u8Data0Val

EIM data0 error injection position

uint8_t EIM_ChCfgType::u8Data1enable

EIM data1 error injection enable or not

uint8_t EIM_ChCfgType::u8Data1Val

EIM data1 error injection position

uint8_t EIM_ChCfgType::u8DwpMode

EIM dwp mode, See EIM DWP mode

uint8_t EIM_ChCfgType::u8EimChn

EIM channel No, See EIM channel number define and EIM lockstep channel define

uint8_t EIM_ChCfgType::u8MonNum

EIM monitor No, See EIM monitor configuration type

The documentation for this struct was generated from the following file:

module_driver_eim.h

ENET_Buffer Struct Reference

Defines the buffer type for ENET driver.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_Buffer:

IMAGE

Public Attributes

```
struct ENET_Buffer * Next
uint8 * Data
uint16 Length
uint16 TotalLength
```

Detailed Description

Defines the buffer type for ENET driver.

Member Data Documentation

uint8* ENET_Buffer::Data

Pointer to the data buffer

uint16 ENET_Buffer::Length

Length of the current buffer

struct ENET_Buffer* ENET_Buffer::Next

Pointer to the next buffer,the buffer chain should be ended with NULL

uint16 ENET_Buffer::TotalLength

Total length of the buffer chain,TotalLength is valid for the first buffer

The documentation for this struct was generated from the following file:

`module_driver_enet.h`

ENET_Config Struct Reference

ENET configuration type.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_Config:

IMAGE

Public Attributes

- `uint8_t RxChannelCount`
- `uint8_t TxChannelCount`
- `uint8_t MACAddress [ENET_MAC_ADDRESS_LENGTH]`
- `ENET_MAC_MiiModeType MiiMode`
- `ENET_MAC_RGMIIClkSelType RgmiiClk`
- `ENET_MAC_SpeedType Speed`
- `ENET_MAC_DuplexModeType Duplex`
- `ENET_DescConfigType RxBD [ENET_CHANNEL_COUNT]`
- `ENET_DescConfigType TxBD [ENET_CHANNEL_COUNT]`
- `ENET_MiiConfigType MiiConfig`
- `ENET_MACConfigType MACConfig`
- `ENET_MTLConfigType MTLConfig`
- `ENET_DMAConfigType DMAConfig`
- `ENET_EventCallbackType EventCallback`
- `ENET_ChannelCallbackType RxReady`
- `ENET_ChannelCallbackType TxComplete`
- `ENET_ChannelCallbackType TxCompleteWait`

Detailed Description

ENET configuration type.

Member Data Documentation

ENET_DMAConfigType ENET_Config::DMAConfig

DMA control registers configuration

ENET_MAC_DuplexModeType ENET_Config::Duplex

MAC communication duplex mode

ENET_EventCallbackType ENET_Config::EventCallback

ENET general event callback

uint8_t ENET_Config::MACAddress[ENET_MAC_ADDRESS_LENGTH]

ENET_MACConfigType ENET_Config::MACConfig

MAC control registers configuration

ENET_MiiConfigType ENET_Config::MiiConfig

PHY MDIO access configuration parameters

ENET_MAC_MiiModeType ENET_Config::MiiMode

PHY interface mode select

ENET_MTLConfigType ENET_Config::MTLConfig

MTL control registers configuration

ENET_MAC_RGMIIClkSelType ENET_Config::RgmiiClk

RGMII clock source select, this clock must be 125/25/2.5 MHz

ENET_DescConfigType ENET_Config::RxBD[ENET_CHANNEL_COUNT]

Receive Buffer Descriptors configuration

uint8_t ENET_Config::RxChannelCount

The number of Receive Channel used by the driver.

ENET_ChannelCallbackType ENET_Config::RxReady

ENET receive data ready callback

ENET_MAC_SpeedType ENET_Config::Speed

MAC link speed

ENET_DescConfigType ENET_Config::TxBD[ENET_CHANNEL_COUNT]

Transmit Buffer Descriptors configuration

uint8_t ENET_Config::TxChannelCount

The number of Transmit Channel used by the driver.

ENET_ChannelCallbackType ENET_Config::TxComplete

ENET send packet complete callback

ENET_ChannelCallbackType ENET_Config::TxCompleteWait

ENET wait sending packet complete callback

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_DescConfig Struct Reference

ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA. It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_DescConfig:

IMAGE

Public Attributes

ENET_DMADescType * DescList

uint8_t * Buffer

uint16_t BufferLen

uint16_t RingSize

Detailed Description

ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA. It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache.

For RX DMA descriptor: Buffer is necessary. DescList Buffer will be accessed by MCU and DMA. It is best to place these buffers in a non-cacheable area, or users need to manually refresh the cache.

Member Data Documentation

uint8_t* ENET_DescConfig::Buffer

Buffer data pool start address, size = BufferLen * RingSize The buffer must be 4 bytes aligned

uint16_t ENET_DescConfig::BufferLen

Length of each individual buffer in the pool The buffer length must be 4 bytes aligned

ENET_DMADescType* ENET_DescConfig::DescList

Buffer descriptor ring start address.

uint16_t ENET_DescConfig::RingSize

The number of descriptors in DescList.

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_DescList Struct Reference

DMA descriptor list type for internal use by ENET driver.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_DescList:

IMAGE

Public Attributes

ENET_DMADescType * DescList

uint16_t BufferLen

uint16_t RingSize

ENET_DMADescType * FirstDesc

ENET_DMADescType * LastDesc

Detailed Description

DMA descriptor list type for internal use by ENET driver.

Member Data Documentation

uint16_t ENET_DescList::BufferLen

DMA descriptor buffer length

ENET_DMADescType* ENET_DescList::DescList

DMA descriptor list start address

ENET_DMADescType* ENET_DescList::FirstDesc

First descriptor used by ENET driver

ENET_DMADescType* ENET_DescList::LastDesc

Last descriptor used by ENET driver

uint16_t ENET_DescList::RingSize

Number of DMA descriptors in DescList

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_DMACHannelConfig Struct Reference

DMA channel configuration type.

```
#include <module_driver_enet.h>
```

Public Attributes

ENET_DMABurstLengthType BurstLength

bool **EnhancedDescriptor**

bool **EarlySendIntEnable**

bool **OperateSecondPacket**

uint8_t **TxChannelWeight**

bool **EarlyReceiveIntEnable**

Detailed Description

DMA channel configuration type.

Member Data Documentation

ENET_DMABurstLengthType ENET_DMACHannelConfig::BurstLength

Assumption transmit BurstLength = receive BurstLength

bool ENET_DMACHannelConfig::EarlyReceiveIntEnable

DMA_CHn_RX_CONTROL[22] Enable early receive interrupt

bool ENET_DMACHannelConfig::EarlySendIntEnable

DMA_CHn_TX_CONTROL[22] Enable early transmit interrupt

bool ENET_DMACHannelConfig::EnhancedDescriptor

DMA_CHn_TX_CONTROL[28] Enhanced Descriptor Enable, Not supported yet

bool ENET_DMACHannelConfig::OperateSecondPacket

DMA_CHn_TX_CONTROL[4] Enable operation on second packet for transmission

uint8_t ENET_DMACHannelConfig::TxChannelWeight

DMA_CHn_TX_CONTROL[3:1] Transmit Channel Weight,range 0 ~ 7

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_DMAConfig Struct Reference

ENET DMA configuration type.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_DMAConfig:



Public Attributes

- ENET_DMA_PriorityRatioType PriorityRatio
- ENET_DMA_PriorityModeType PriorityMode
- ENET_DMA_TxArbitrationAlgorithmType TxArbitration
- ENET_DMA_ArbitrationSchemeType TxRxArbitration
- ENET_DMA_BurstModeType BurstMode
- bool RebuildINCRxBurst
- bool AddressAlignedBeats
- ENET_DMAChannelConfigType ChannelConfigs [ENET_CHANNEL_COUNT]

Detailed Description

ENET DMA configuration type.

Member Data Documentation

- bool ENET_DMAConfig::AddressAlignedBeats
 - DMA_SYSBUS_MOD[12] Enable address-aligned beats
- ENET_DMA_BurstModeType ENET_DMAConfig::BurstMode
 - DMA_SYSBUS_MOD[14,0] Burst length mode

ENET_DMAChannelConfigType ENET_DMAConfig::ChannelConfigs[ENET_CHANNEL_COUNT]

ENET_DMA_PriorityModeType ENET_DMAConfig::PriorityMode

DMA_MOD[11] Priority mode between transmit and receive

ENET_DMA_PriorityRatioType ENET_DMAConfig::PriorityRatio

DMA_MOD[14:12] Priority Ratio between transmit and receive

bool ENET_DMAConfig::RebuildINCRxBurst

DMA_SYSBUS_MOD[15] Enable rebuild INCRx burst

ENET_DMA_TxArbitrationAlgorithmType ENET_DMAConfig::TxArbitration

DMA_MOD[4:2] Arbitration mode between the transmit channels

ENET_DMA_ArbitrationSchemeType ENET_DMAConfig::TxRxArbitration

DMA_MOD[1] Arbitration mode between the transmit and receive

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_DMADescType Struct Reference

ENET DMA descriptor type.

#include <module_driver_enet.h>

Public Attributes

__IO uint32_t **Des0**
__IO uint32_t **Des1**
__IO uint32_t **Des2**
__IO uint32_t **Des3**
uint32_t **BackupInfo0**
uint32_t **BackupInfo1**

Detailed Description

ENET DMA descriptor type.

Member Data Documentation

uint32_t ENET_DMADescType::BackupInfo0

uint32_t ENET_DMADescType::BackupInfo1

__IO uint32_t ENET_DMADescType::Des0

__IO uint32_t ENET_DMADescType::Des1

__IO uint32_t ENET_DMADescType::Des2

__IO uint32_t ENET_DMADescType::Des3

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_DrvHandle Struct Reference

ENET handle is internal used by ENET driver.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_DrvHandle:

IMAGE

Public Attributes

ENET_InstanceType **eInstance**

uint8_t **ItMode**

uint8_t **TxChannelCount**

uint8_t **RxChannelCount**

ENET_Type * **EnetBase**

ENET_DescListType **TxDescLists** [ENET_CHANNEL_COUNT]

ENET_DescListType **RxDescLists** [ENET_CHANNEL_COUNT]

ENET_EventCallbackType **EventCallback**

ENET_ChannelCallbackType **RxReady**

ENET_ChannelCallbackType **TxComplete**

ENET_ChannelCallbackType **TxCompleteWait**

ENET_StateType **State**

ENET_StateType **TxState**

ENET_StateType **RxState**

ENET_StateType **PTPState**

uint32_t **PtpClkFreq**

uint32_t **PtpAddend**

uint32_t **PpsInterval** [ENET_PPS_CHANNEL_COUNT]

uint32_t **PpsWidth** [ENET_PPS_CHANNEL_COUNT]

Detailed Description

ENET handle is internal used by ENET driver.

Member Data Documentation

ENET_InstanceType ENET_DrvHandle::eInstance

ENET_Type* ENET_DrvHandle::EnetBase

ENET_EventCallbackType ENET_DrvHandle::EventCallback

uint8_t ENET_DrvHandle::ItMode

uint32_t ENET_DrvHandle::PpsInterval[ENET_PPS_CHANNEL_COUNT]

uint32_t ENET_DrvHandle::PpsWidth[ENET_PPS_CHANNEL_COUNT]

uint32_t ENET_DrvHandle::PtpAddend

uint32_t ENET_DrvHandle::PtpClkFreq

ENET_StateType ENET_DrvHandle::PTPState

uint8_t ENET_DrvHandle::RxChannelCount

ENET_DescListType ENET_DrvHandle::RxDescLists[ENET_CHANNEL_COUNT]

ENET_ChannelCallbackType ENET_DrvHandle::RxReady

ENET_StateType ENET_DrvHandle::RxState

ENET_StateType ENET_DrvHandle::State

uint8_t ENET_DrvHandle::TxChannelCount

ENET_ChannelCallbackType ENET_DrvHandle::TxComplete

ENET_ChannelCallbackType ENET_DrvHandle::TxCompleteWait

ENET_DescListType ENET_DrvHandle::TxDescLists[ENET_CHANNEL_COUNT]

ENET_StateType ENET_DrvHandle::TxState

The documentation for this struct was generated from the following file:

`module_driver_enet.h`

ENET_EstConfig Struct Reference

ENET EST configuration type.

#include <module_driver_enet.h>

Public Attributes

uint8_t **Overhead**

uint8_t **PtpTimeOffset**

uint16_t **CurrentTimeOffset**

ENET_MTL_GCLIntervalMultipleType **IntervalLeftShift**

ENET_MTL_ScheduleErrorLoopCountType **SchedErrLoopCount**

bool **DropSchedErrFrame**

bool **NotDropFrameSizeErr**

Detailed Description

ENET EST configuration type.

Member Data Documentation

uint16_t ENET_EstConfig::CurrentTimeOffset

MTL_EST_CONTROL[23:12] Current Time Offset Value

bool ENET_EstConfig::DropSchedErrFrame

MTL_EST_CONTROL[5] Drop frames causing Scheduling error, due to the bug in 5.20a errata, this parameter shell always be false

ENET_MTL_GCLIntervalMultipleType ENET_EstConfig::IntervalLeftShift

MTL_EST_CONTROL[10:8] Time Interval Left Shift Amount

bool ENET_EstConfig::NotDropFrameSizeErr

MTL_EST_CONTROL[4] Do not Drop frames during Frame Size Error, due to the bug in 5.20a errata, this parameter shell always be true

uint8_t ENET_EstConfig::Overhead

MTL_EST_EXT_CONTROL[5:0] Overhead Bytes Value

uint8_t ENET_EstConfig::PtpTimeOffset

MTL_EST_CONTROL[31:24] PTP time offset value

ENET_MTL_ScheduleErrorLoopCountType ENET_EstConfig::SchedErrLoopCount

MTL_EST_CONTROL[7:6] Loop count to report scheduling error

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_FlexPPSConfigType Struct Reference

IEEE1588 flexible PPS configuration type.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_FlexPPSConfigType:

IMAGE

Public Attributes

ENET_MAC_PPSCmdType Cmd

ENET_TimestampType TargetTime

uint32_t **PPSInterval**

uint32_t **PPSWidth**

Detailed Description

IEEE1588 flexible PPS configuration type.

Member Data Documentation

ENET_MAC_PPSCmdType ENET_FlexPPSConfigType::Cmd

Flexible PPS CMD

uint32_t ENET_FlexPPSConfigType::PPSInterval

The interval between the rising edges of Flexible PPS in nanoseconds

uint32_t ENET_FlexPPSConfigType::PPSWidth

The width between the rising edge and falling edge of Flexible PPS,in nanoseconds

ENET_TimestampType ENET_FlexPPSConfigType::TargetTime

Flexible PPS target time,SecondsHi value is ignored

The documentation for this struct was generated from the following file:

`module_driver_enet.h`

ENET_FlowControl Struct Reference

ENET flow control configuration type.

#include <module_driver_enet.h>

Collaboration diagram for ENET_FlowControl:

IMAGE

Public Attributes

- bool MacTxFlowCtrlEnable
- bool ZeroQuantaPauseDisable
- uint16_t PauseTime
- ENET_MAC_PauseThresholdType PauseThreshold
- ENET_FlowControlRxQType RxQueueConfig [ENET_CHANNEL_COUNT]
- bool MacRxFlowCtrlEnable
- bool UnicastPauseEnable

Detailed Description

ENET flow control configuration type.

Member Data Documentation

bool ENET_FlowControl::MacRxFlowCtrlEnable

MAC_RX_FLOW_CTRL[0] Enable MAC to receive and handle flow control pause packets

bool ENET_FlowControl::MacTxFlowCtrlEnable

MAC_Q0_TX_FLOW_CTRL[1] Enable MAC to transmit flow control Pause packets

ENET_MAC_PauseThresholdType ENET_FlowControl::PauseThreshold

MAC_Q0_TX_FLOW_CTRL[6] Pause threshold for automatic retransmission Pause packets

uint16_t ENET_FlowControl::PauseTime

MAC_Q0_TX_FLOW_CTRL[31:16] Pause time field in the TX control packet

ENET_FlowControlRxQType ENET_FlowControl::RxQueueConfig[ENET_CHANNEL_COUNT]

MTL receive queue configuration for hardware trigger flow control

bool ENET_FlowControl::UnicastPauseEnable

MAC_RX_FLOW_CTRL[1] Enable MAC to handle unicast flow control Pause packets

bool ENET_FlowControl::ZeroQuantaPauseDisable

MAC_Q0_TX_FLOW_CTRL[7] Disable automatic generation of zero-quanta Pause packets

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_FlowControlRxQ Struct Reference

ENET flow control receive queue configuration type.

```
#include <module_driver_enet.h>
```

Public Attributes

bool **RxHwFlowCtrlEnable**

ENET_MTL_RxFlowCtrlThresholdType RxDeactiveThreshold

ENET_MTL_RxFlowCtrlThresholdType RxActiveThreshold

Detailed Description

ENET flow control receive queue configuration type.

Member Data Documentation

ENET_MTL_RxFlowCtrlThresholdType ENET_FlowControlRxQ::RxActiveThreshold

MTL_RXQn_OPERATION_MODE[11:8] Threshold for activating flow control

ENET_MTL_RxFlowCtrlThresholdType ENET_FlowControlRxQ::RxDeactiveThreshold

MTL_RXQn_OPERATION_MODE[17:14] Threshold for deactivating flow control

bool ENET_FlowControlRxQ::RxHwFlowCtrlEnable

MTL_RXQn_OPERATION_MODE[7] Enable hardware flow control to trigger generation of pause packet

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_GateCtrl Struct Reference

The ENET EST Gate control type.

```
#include <module_driver_enet.h>
```

Public Attributes

bool **GateOpen** [ENET_CHANNEL_COUNT]

uint32_t **TimeInterval**

Detailed Description

The ENET EST Gate control type.

Member Data Documentation

bool ENET_GateCtrl::GateOpen[ENET_CHANNEL_COUNT]

Gate state, bit 0 - channel 0, bit 1- channel 1

uint32_t ENET_GateCtrl::TimeInterval

Gate control time interval (24 bits value)

The documentation for this struct was generated from the following file:

`module_driver_enet.h`

ENET_GateCtrlList Struct Reference

The ENET EST Gate control List type.

#include <module_driver_enet.h>

Collaboration diagram for ENET_GateCtrlList:

IMAGE

Public Attributes

- uint32_t **BaseTimeS**
- uint32_t **BaseTimeNs**
- uint8_t **CycleTimeS**
- uint32_t **CycleTimeNs**
- uint32_t **TimeExtension**
- uint16_t **GclLength**
- ENET_GateCtrlType Gate** [ENET_TSN_GCL_DEPTH]

Detailed Description

The ENET EST Gate control List type.

Member Data Documentation

uint32_t ENET_GateCtrlList::BaseTimeNs

Base time nanoseconds

uint32_t ENET_GateCtrlList::BaseTimeS

Base time seconds

uint32_t ENET_GateCtrlList::CycleTimeNs

Cycle time nanoseconds

uint8_t ENET_GateCtrlList::CycleTimeS

Cycle time seconds

ENET_GateCtrlType ENET_GateCtrlList::Gate[ENET_TSN_GCL_DEPTH]

The gate control List

uint16_t ENET_GateCtrlList::GclLength

The length of gate control List

uint32_t ENET_GateCtrlList::TimeExtension

Time extension in nanoseconds

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_L3L4Filter Struct Reference

ENET Layer3 Layer4 filter configuration type.

#include <module_driver_enet.h>

Public Attributes

bool **DMACHannelSelect**
ENET_ChannelType **DMACHannel**
ENET_MAC_Layer4ProtocolType **L4Protocol**
ENET_FilterEnableType **DstPortFilter**
ENET_FilterEnableType **SrcPortFilter**
uint16_t **DstPort**
uint16_t **SrcPort**
ENET_MAC_Layer3ProtocolType **L3Protocol**
ENET_FilterEnableType **DstIPFilter**
ENET_FilterEnableType **SrcIPFilter**
uint8_t **Ipv4DAMaskBit**
uint8_t **Ipv4SAMaskBit**
uint8_t **Ipv6MaskBit**
uint32_t **IPv4DstAddr**
uint32_t **IPv4SrcAddr**
uint8_t **IPv6Addr** [16u]

Detailed Description

ENET Layer3 Layer4 filter configuration type.

Member Data Documentation

ENET_ChannelType **ENET_L3L4Filter::DMACHannel**

MAC_L3_L4_CONTROLn[24] The DMA channel number for the filter

bool ENET_L3L4Filter::DMAChannelSelect

MAC_L3_L4_CONTROLn[28] Enable DMA channel select for the filter

ENET_FilterEnableType ENET_L3L4Filter::DstIPFilter

MAC_L3_L4_CONTROLn[5:4] The destination IP address filter mode

uint16_t ENET_L3L4Filter::DstPort

MAC_LAYER4_ADDRESSn[31:16] The destination port in little-endian

ENET_FilterEnableType ENET_L3L4Filter::DstPortFilter

MAC_L3_L4_CONTROLn[21:20] The destination port filter mode

uint8_t ENET_L3L4Filter::Ipv4DAMaskBit

MAC_L3_L4_CONTROLn[15:11] The number of ipv4 destination address lower bits mask, the filter will match the higher bits that are not masked

uint32_t ENET_L3L4Filter::IPv4DstAddr

MAC_LAYER3_ADDRn_REG1[31:0] Ipv4 destination IP address in little-endian To match "224.0.0.1", ipv4 address must be 0xE0000001

uint8_t ENET_L3L4Filter::Ipv4SAMaskBit

MAC_L3_L4_CONTROLn[10:6] The number of ipv4 source address lower bits mask, like Ipv4DAMaskBit

uint32_t ENET_L3L4Filter::IPv4SrcAddr

MAC_LAYER3_ADDRn_REG0[31:0] Ipv4 source IP address in little-endian, like IPv4DstAddr

uint8_t ENET_L3L4Filter::IPv6Addr[16u]

MAC_LAYER3_ADDRn_REG0~4 Ipv6 IP address

uint8_t ENET_L3L4Filter::Ipv6MaskBit

MAC_L3_L4_CONTROLn[12:6] The number of ipv6 address lower bits mask, like Ipv4DAMaskBit

ENET_MAC_Layer3ProtocolType ENET_L3L4Filter::L3Protocol

MAC_L3_L4_CONTROLn[0] The IPV4/IPV6 protocol for the layer3 filter

ENET_MAC_Layer4ProtocolType ENET_L3L4Filter::L4Protocol

MAC_L3_L4_CONTROLn[16] The TCP/UDP protocol for the layer4 filter

ENET_FilterEnableType ENET_L3L4Filter::SrcIPFilter

MAC_L3_L4_CONTROLn[3:2] The source IP address filter mode

uint16_t ENET_L3L4Filter::SrcPort

MAC_LAYER4_ADDRESSn[15:0] The source port in little-endian

ENET_FilterEnableType ENET_L3L4Filter::SrcPortFilter

MAC_L3_L4_CONTROLn[19:18] The source port filter mode

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_MACConfig Struct Reference

ENET MAC configuration type.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_MACConfig:

IMAGE

Public Attributes

ENET_MAC_SrcAddrCtrlType SrcAddrCtrl

ENET_MAC_InterPacketGapType InterPacketGap

bool **ChecksumOffload**

bool **GiantPacketLimit**

bool **Support2KPacket**

bool **CrcStripForTypePkt**

bool **AutoPadOrCrCStrip**

bool **WatchDogDisable**

bool **JabberDisable**

bool **JumboEnable**

bool **LoopbackMode**

bool **CarrierSenseBeforeTx**

bool **DisableReceiveOwn**

bool **DisCarrierSenseDurTx**

bool **DisableRetry**

ENET_MAC_BackOffLimitType BackOffLimit

bool **DeferralCheck**

ENET_MAC_ExtInterPacketGapType ExtInterPktGap

bool **ExtInterPktGapEn**

bool **PacketDuplication**

bool **UnicastSlowProtoDetect**

bool **SlowProtoDetectEnable**

bool **DisableCrcCheck**

uint16_t **GiantPktSizeLimit**

ENET_MACFilterType MACFilter

bool **ProgramWatchdogEnable**

ENET_MAC_WDGTimerSizeType WatchdogTimeout

Detailed Description

ENET MAC configuration type.

Member Data Documentation

bool ENET_MACConfig::AutoPadOrCrCStrip

MAC_CONFIGURATION[20] Automatic pad or CRC stripping

ENET_MAC_BackOffLimitType ENET_MACConfig::BackOffLimit

MAC_CONFIGURATION[6:5] Back-Off Limit

bool ENET_MACConfig::CarrierSenseBeforeTx

MAC_CONFIGURATION[11] Enable Carrier Sense Before Transmission in Full-Duplex Mode

bool ENET_MACConfig::ChecksumOffload

MAC_CONFIGURATION[27] Checksum Offload

bool ENET_MACConfig::CrCStripForTypePkt

MAC_CONFIGURATION[21] Enable CRC stripping for type packets

bool ENET_MACConfig::DeferralCheck

MAC_CONFIGURATION[4] Deferral Check enable

bool ENET_MACConfig::DisableCrCCheck

MAC_EXT_CONFIGURATION[16] Disable CRC Checking for Received Packets

bool ENET_MACConfig::DisableReceiveOwn

MAC_CONFIGURATION[10] Disable Receive Own in half-duplex mode

bool ENET_MACConfig::DisableRetry

MAC_CONFIGURATION[8] Disable Retry when a collision occurs in half-duplex mode

bool ENET_MACConfig::DisCarrierSenseDurTx

MAC_CONFIGURATION[9] Disable Carrier Sense During Transmission

ENET_MAC_ExtInterPacketGapType ENET_MACConfig::ExtInterPktGap

MAC_EXT_CONFIGURATION[29:25] Extended inter-packet gap value

bool ENET_MACConfig::ExtInterPktGapEn

MAC_EXT_CONFIGURATION[24] Extended Inter-Packet Gap Enable

bool ENET_MACConfig::GiantPacketLimit

MAC_CONFIGURATION[23] Giant packet size limit control Enable

uint16_t ENET_MACConfig::GiantPktSizeLimit

MAC_EXT_CONFIGURATION[13:0] Giant Packet Size Limit

ENET_MAC_InterPacketGapType ENET_MACConfig::InterPacketGap

MAC_CONFIGURATION[26:24] The minimum inter packet gap between packets during transmission

bool ENET_MACConfig::JabberDisable

MAC_CONFIGURATION[17] Jabber Disable, if disabled 16383 bytes can be transmitted

bool ENET_MACConfig::JumboEnable

MAC_CONFIGURATION[16] Jumbo Packet Enable, if enabled 9018/9022 bytes can be received

bool ENET_MACConfig::LoopbackMode

MAC_CONFIGURATION[12] Enable MAC loopback mode

ENET_MACFilterType ENET_MACConfig::MACFilter

MAC_PACKET_FILTER

bool ENET_MACConfig::PacketDuplication

MAC_EXT_CONFIGURATION[19] Packet Duplication Control

bool ENET_MACConfig::ProgramWatchdogEnable

MAC_WATCHDOG_TIMEOUT[8] Programmable Watchdog Enable

bool ENET_MACConfig::SlowProtoDetectEnable

MAC_EXT_CONFIGURATION[17] Slow Protocol Detection Enable

ENET_MAC_SrcAddrCtrlType ENET_MACConfig::SrcAddrCtrl

MAC_CONFIGURATION[30:28] MAC source address insertion or replacement control

bool ENET_MACConfig::Support2KPacket

MAC_CONFIGURATION[22] Support for 2K packets

bool ENET_MACConfig::UnicastSlowProtoDetect

MAC_EXT_CONFIGURATION[18] Unicast Slow Protocol Packet Detect

bool ENET_MACConfig::WatchDogDisable

MAC_CONFIGURATION[19] Watchdog Disable

ENET_MAC_WDGTimerSizeType ENET_MACConfig::WatchdogTimeout

MAC_WATCHDOG_TIMEOUT[3:0] Programmable Watchdog Timeout

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_MACFilter Struct Reference

ENET MAC filter configuration type.

```
#include <module_driver_enet.h>
```

Public Attributes

- bool **ReceiveAll**
- bool **PromiscuousMode**
- bool **DropNonTcpUdpOverIp**
- bool **HashOrPerfectFilter**
- ENET_FilterEnableType** **SAFilter**
- ENET_FilterEnableType** **DAFilter**
- ENET_MAC_CtrlPacketFilterModeType** **PassCtrlPkts**
- bool **DisableBroadcast**
- bool **PassAllMulticast**
- bool **HashMulticast**
- bool **HashUnicast**

Detailed Description

ENET MAC filter configuration type.

Member Data Documentation

ENET_FilterEnableType **ENET_MACFilter::DAFilter**

MAC_PACKET_FILTER[3] Source Address Filter type

bool **ENET_MACFilter::DisableBroadcast**

MAC_PACKET_FILTER[5] Disable Broadcast Packets

bool **ENET_MACFilter::DropNonTcpUdpOverIp**

MAC_PACKET_FILTER[21] Drop Non-TCP/UDP Over IP Packets

bool ENET_MACFilter::HashMulticast

MAC_PACKET_FILTER[2] Enable Hash Table Filter for Multicast Packets

bool ENET_MACFilter::HashOrPerfectFilter

MAC_PACKET_FILTER[10] Hash Or Perfect Filter Enable

bool ENET_MACFilter::HashUnicast

MAC_PACKET_FILTER[1] Enable Hash Table Filter for Unicast Packets

bool ENET_MACFilter::PassAllMulticast

MAC_PACKET_FILTER[4] Pass All Multicast

ENET_MAC_CtrlPacketFilterModeType ENET_MACFilter::PassCtrlPkts

MAC_PACKET_FILTER[7:6] Control packets filter mode

bool ENET_MACFilter::PromiscuousMode

MAC_PACKET_FILTER[0] Enable promiscuous mode

bool ENET_MACFilter::ReceiveAll

MAC_PACKET_FILTER[31] Receive all packets to the application

ENET_FilterEnableType ENET_MACFilter::SAFilter

MAC_PACKET_FILTER[9:8] Source Address Filter type

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_MiiConfigType Struct Reference

Mdio configuration type.

```
#include <module_driver_enet.h>
```

Public Attributes

bool **PreambleSuppress**

bool **BackToBack**

uint8_t **TrailingClocks**

Detailed Description

Mdio configuration type.

Member Data Documentation

bool ENET_MiiConfigType::BackToBack

MAC_MDIO_ADDRESS[26] Back to Back transactions

bool ENET_MiiConfigType::PreambleSuppress

MAC_MDIO_ADDRESS[27] Preamble Suppression Enable

uint8_t ENET_MiiConfigType::TrailingClocks

MAC_MDIO_ADDRESS[14:12] Number of Training Clocks

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_MMConfig Struct Reference

Defines MMC counter configuration.

#include <module_driver_enet.h>

Public Attributes

bool **UpdateDroppedBroadcast**

bool **ResetOnRead**

bool **CounterRollover**

ENET_MAC_MmcCntPresetLevelType **PresetLevel**

uint32_t **MmcTxCntIntEn**

uint32_t **MmcRxCntIntEn**

Detailed Description

Defines MMC counter configuration.

Member Data Documentation

bool **ENET_MMConfig::CounterRollover**

MMC_CONTROL[1] MMC counters roll over to zero after reaching the maximum value

uint32_t **ENET_MMConfig::MmcRxCntIntEn**

MMC_RX_INTERRUPT_MASK[25:0] Enable MMC RX counters interrupts, each bit represents an event of Enet_MMCRxCounterType

uint32_t **ENET_MMConfig::MmcTxCntIntEn**

MMC_TX_INTERRUPT_MASK[25:0] Enable MMC TX counters interrupts, each bit represents an event of Enet_MMCTxCounterType

ENET_MAC_MmcCntPresetLevelType **ENET_MMConfig::PresetLevel**

MMC_CONTROL[5] Enable MMC counters preset to almost-full value

bool ENET_MMConfig::ResetOnRead

MMC_CONTROL[2] MMC counters are reset to zero after read

bool ENET_MMConfig::UpdateDroppedBroadcast

MMC_CONTROL[8] Update MMC counters for dropped broadcast packets

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_MTLConfig Struct Reference

ENET MTL configuration type.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_MTLConfig:

IMAGE

Public Attributes

ENET_MTL_TxScheduleAlgorithmType TxSchedAlgo

ENET_MTL_RxArbitrationAlgorithmType RxArbitrationAlgo

bool EnableTransmitStatus

ENET_MTLQueueConfigType QueueConfigs [ENET_CHANNEL_COUNT]

Detailed Description

ENET MTL configuration type.

Member Data Documentation

bool ENET_MTLConfig::EnableTransmitStatus

Enable MTL to forward transmit status to application

ENET_MTLQueueConfigType ENET_MTLConfig::QueueConfigs[ENET_CHANNEL_COUNT]

ENET_MTL_RxArbitrationAlgorithmType ENET_MTLConfig::RxArbitrationAlgo

RX arbitration algorithm

ENET_MTL_TxScheduleAlgorithmType ENET_MTLConfig::TxSchedAlgo

TX scheduling algorithm

The documentation for this struct was generated from the following file:

`module_driver_enet.h`

ENET_MTLQueueConfig Struct Reference

ENET MTL Queue configuration type.

#include <module_driver_enet.h>

Public Attributes

uint32_t **TxFifoSize**
uint32_t **RxFifoSize**
ENET_MTL_QueueEnableType **TxQueueEnable**
ENET_MTL_TxThresholdType **TxThreshold**
uint32_t **IdleSlopeQuantumWeight**
uint32_t **SendSlopeCredit**
uint32_t **HiCredit**
int32_t **LoCredit**
ENET_MTL_QueueEnableType **RxQueueEnable**
ENET_MTL_RxThresholdType **RxThreshold**
ENET_MTL_RxFlowCtrlThresholdType **RxDeactiveThreshold**
ENET_MTL_RxFlowCtrlThresholdType **RxActiveThreshold**
bool **RxHwFlowCtrlEnable**
bool **ReveiveTCPIPChecksumErrPkts**
bool **ReveiveErrorPkts**
bool **ReveiveUndersizeGoodPkts**
bool **RxQueueArbitration**
uint8_t **RxQueueWeight**
uint8_t **RxQueuePriority**

Detailed Description

ENET MTL Queue configuration type.

Member Data Documentation

uint32_t **ENET_MTLQueueConfig::HiCredit**

MTL_TXQn_HICREDIT[28:0] High credit limit for AVB queues

uint32_t ENET_MTLQueueConfig::IdleSlopeQuantumWeight

MTL_TXQn_QUANTUM_WEIGHT[20:0] idleSlopeCredit, Quantum or Weights

int32_t ENET_MTLQueueConfig::LoCredit

MTL_TXQn_LOCREDIT[28:0] Low credit limit for AVB queues

bool ENET_MTLQueueConfig::ReveiveErrorPkts

MTL_RXQn_OPERATION_MODE[4] Forward error packets to application

bool ENET_MTLQueueConfig::ReveiveTCPIPChecksumErrPkts

MTL_RXQn_OPERATION_MODE[6] Disable Dropping of TCP/IP Checksum Error Packets

bool ENET_MTLQueueConfig::ReveiveUndersizeGoodPkts

MTL_RXQn_OPERATION_MODE[3] Forward undersized good packets to application

ENET_MTL_RxFlowCtrlThresholdType ENET_MTLQueueConfig::RxActiveThreshold

MTL_RXQn_OPERATION_MODE[11:8] Threshold for activating flow control

ENET_MTL_RxFlowCtrlThresholdType ENET_MTLQueueConfig::RxDeactiveThreshold

MTL_RXQn_OPERATION_MODE[17:14] Threshold for deactivating flow control

uint32_t ENET_MTLQueueConfig::RxFifoSize

MTL Receive FIFO size in bytes

bool ENET_MTLQueueConfig::RxHwFlowCtrlEnable

MTL_RXQn_OPERATION_MODE[7] Enable Hardware Flow Control

bool ENET_MTLQueueConfig::RxQueueArbitration

Enable receive queue packet arbitration

ENET_MTL_QueueEnableType ENET_MTLQueueConfig::RxQueueEnable

Receive queue enable mode

uint8_t ENET_MTLQueueConfig::RxQueuePriority

Receive queue priority to match the priority field of tagged packets bit n is set to 1 ,represent priority n is matched to the receive queue

uint8_t ENET_MTLQueueConfig::RxQueueWeight

Receive queue weight

ENET_MTL_RxThresholdType ENET_MTLQueueConfig::RxThreshold

MTL_RXQn_OPERATION_MODE[1:0,5] Threshold level of the MTL receive queue

uint32_t ENET_MTLQueueConfig::SendSlopeCredit

MTL_TXQn_SENDSLOPECREDIT[13:0] Send slope credit for AVB queues

uint32_t ENET_MTLQueueConfig::TxFifoSize

MTL Transmit FIFO size in bytes

ENET_MTL_QueueEnableType ENET_MTLQueueConfig::TxQueueEnable

MTL_TXQn_OPERATION_MODE[3:2] Transmit Queue Enable mode

ENET_MTL_TxThresholdType ENET_MTLQueueConfig::TxThreshold

MTL_TXQn_OPERATION_MODE[6:4,1] TTransmit Threshold Control

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_OuterVlanCBTI Struct Reference

ENET Channel based VLAN insertion type.

```
#include <module_driver_enet.h>
```

Public Attributes

ENET_MAC_VLANType VlanType [ENET_CHANNEL_COUNT]

uint16_t **VlanTag** [ENET_CHANNEL_COUNT]

Detailed Description

ENET Channel based VLAN insertion type.

Member Data Documentation

uint16_t ENET_OuterVlanCBTI::VlanTag[ENET_CHANNEL_COUNT]

VLAN tag for ENET_CHANNEL_0 ~ ENET_CHANNEL_1

ENET_MAC_VLANType ENET_OuterVlanCBTI::VlanType[ENET_CHANNEL_COUNT]

VLAN type for ENET_CHANNEL_0 ~ ENET_CHANNEL_1 : C-VLAN or S-VLAN

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_PreemptionConfig Struct Reference

The ENET EST Frame Preemption configuration type.

```
#include <module_driver_enet.h>
```

Public Attributes

bool **PreemptionEnable**

ENET_QueueClassificType Classification [ENET_CHANNEL_COUNT]

uint16_t **ReleaseAdvance**

uint16_t **HoldAdvance**

Detailed Description

The ENET EST Frame Preemption configuration type.

Member Data Documentation

ENET_QueueClassificType ENET_PreemptionConfig::Classification[ENET_CHANNEL_COUNT]

Preemption Classification

uint16_t ENET_PreemptionConfig::HoldAdvance

The maximum time in nanoseconds that can elapse between issuing a HOLD to the MAC and the MAC ceasing to transmit any preemptable frame

bool ENET_PreemptionConfig::PreemptionEnable

Enable frame preemption

uint16_t ENET_PreemptionConfig::ReleaseAdvance

The maximum time in nanoseconds that can elapse between issuing a RELEASE to the MAC and the MAC being ready to resume transmission of preemptable frames

The documentation for this struct was generated from the following file:

`module_driver_enet.h`

Enet_PTPConfig Struct Reference

IEEE1588 time stamp configuration type.

#include <module_driver_enet.h>

Collaboration diagram for Enet_PTPConfig:

IMAGE

Public Attributes

- bool OneStepMode
- bool Av8021ASMode
- bool MacFilterForPtp
- bool TimeStampForAll
- bool PresentationTimeGen
- uint8_t IncrementStep
- ENET_MAC_TxTimestampStatusModeType TXTSSTSM
- ENET_MAC_PTPSnapshotMsgType SnapshotType
- ENET_MAC_PTPPacketLayerType PtpLayer
- ENET_MAC_PTPVersionType PtpVersion
- ENET_MAC_TimestampRolloverType RolloverMode
- ENET_MAC_TimestampUpdateMethodType UpdateMethod
- ENET_MAC_PTPClkSelType ClkSource
- ENET_TimestampType InitTime

Detailed Description

IEEE1588 time stamp configuration type.

Member Data Documentation

- bool Enet_PTPConfig::Av8021ASMode**
- MAC_TIMESTAMP_CONTROL[28] AV 802.1AS mode enable

ENET_MAC_PTPClkSelType Enet_PTPConfig::ClkSource

PTP reference clock source(PCC, TXCLK or reference clock pin input) If the PCC clock is selected, the PCC_ENET clock must be enabled, and if reference pin input is selected, the parameter RefClkPinInputFreq must be assigned a frequency value

uint8_t Enet_PTPConfig::IncrementStep

PTP system time increment step in nanoseconds

ENET_TimestampType Enet_PTPConfig::InitTime

The time stamp for Initialization

bool Enet_PTPConfig::MacFilterForPtp

MAC_TIMESTAMP_CONTROL[18] Enable MAC filter for PTP frames over ethernet

bool Enet_PTPConfig::OneStepMode

PTP run in one step mode,currently not supported.

bool Enet_PTPConfig::PresentationTimeGen

MAC_TIMESTAMP_CONTROL[6] Presentation Time Generation Enable

ENET_MAC_PTTPacketLayerType Enet_PTPConfig::PtpLayer

MAC_TIMESTAMP_CONTROL[13:11] Snapshot for ethernet packet layer

ENET_MAC_PTPVersionType Enet_PTPConfig::PtpVersion

MAC_TIMESTAMP_CONTROL[10] IEEE1588 version

ENET_MAC_TimestampRolloverType Enet_PTPConfig::RolloverMode

MAC_TIMESTAMP_CONTROL[9] Time counter roll over mode

ENET_MAC_PTPSnapshotMsgType Enet_PTPConfig::SnapshotType

MAC_TIMESTAMP_CONTROL[17:14] Enable the frame type for time snapshotting

bool Enet_PTPConfig::TimeStampForAll

MAC_TIMESTAMP_CONTROL[8] Enable Timestamp For All Packets

ENET_MAC_TxTimestampStatusModeType Enet_PTPConfig::TXTSSTSM

MAC_TIMESTAMP_CONTROL[24] Transmit Timestamp Status Mode

ENET_MAC_TimestampUpdateMethodType Enet_PTPConfig::UpdateMethod

MAC_TIMESTAMP_CONTROL[1] Time stamp update mode

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_RxInfo Struct Reference

Defines the received buffer information.

```
#include <module_driver_enet.h>
```

Collaboration diagram for ENET_RxInfo:

IMAGE

Public Attributes

uint32_t **Attributes**

ENET_TimestampType **TimeStamp**

Detailed Description

Defines the received buffer information.

Member Data Documentation

uint32_t **ENET_RxInfo::Attributes**

Receive attributes defined in ENET_AttributeType

ENET_TimestampType **ENET_RxInfo::TimeStamp**

Time stamp snapshot for the received packet

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_TimestampType Struct Reference

IEEE1588 time stamp type.

```
#include <module_driver_enet.h>
```

Public Attributes

uint32_t **Nanoseconds**

uint32_t **Seconds**

uint16_t **SecondsHi**

Detailed Description

IEEE1588 time stamp type.

Member Data Documentation

uint32_t **ENET_TimestampType::Nanoseconds**

Nanoseconds part of the time.

uint32_t **ENET_TimestampType::Seconds**

The 32 least significant bits of the 48 bits seconds part of the time.

uint16_t **ENET_TimestampType::SecondsHi**

The 16 most significant bits of the 48 bit seconds part of the time.

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_TxOption Struct Reference

Defines the options for the Tx packet.

#include <module_driver_enet.h>

Collaboration diagram for ENET_TxOption:

IMAGE

Public Attributes

uint32_t **Attributes**

ENET_TimestampType **TimeStamp**

ENET_TxDescCrcPadType **CrcPadCtrl**

ENET_TxDescChecksumType **CheckSumCtrl**

ENET_TxVlanConfigType **VlanConfig**

ENET_TxVlanConfigType **InnerVlanConfig**

Detailed Description

Defines the options for the Tx packet.

Member Data Documentation

uint32_t **ENET_TxOption::Attributes**

Send attributes defined in ENET_AttributeType

ENET_TxDescChecksumType **ENET_TxOption::CheckSumCtrl**

Checksum control,available for ENET_TX_ATTRIBUTES_CHECKSUM_INSERTION

ENET_TxDescCrcPadType **ENET_TxOption::CrcPadCtrl**

CRC pad control,available for ENET_TX_ATTRIBUTES_CRC_PAD_CTRL

ENET_TxVlanConfigType **ENET_TxOption::InnerVlanConfig**

Inner VLAN tag control,available for ENET_TX_ATTRIBUTES_INNER_VLAN_TAG_CTRL

ENET_TimestampType ENET_TxOption::TimeStamp

Transmitted frame time stamp,available for ENET_TX_ATTRIBUTES_TIMESTAMP_SNAPSHOT

ENET_TxVlanConfigType ENET_TxOption::VlanConfig

VLAN tag control,available for ENET_TX_ATTRIBUTES_VLAN_TAG_CTRL

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_TxVlanConfig Struct Reference

ENET transmit VLAN configuration type.

#include <module_driver_enet.h>

Public Attributes

ENET_MAC_VLANTagCtrlType VlanCtrl

ENET_MAC_VLANType VlanType

uint16_t **VlanTag**

Detailed Description

ENET transmit VLAN configuration type.

Member Data Documentation

ENET_MAC_VLANTagCtrlType ENET_TxVlanConfig::VlanCtrl

VLAN tag control : delete insert or replace

uint16_t ENET_TxVlanConfig::VlanTag

VLAN type tag value

ENET_MAC_VLANType ENET_TxVlanConfig::VlanType

VLAN type : C-VLAN or S-VLAN

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_VlanFilterConfig Struct Reference

ENET VLAN filter configuration type.

#include <module_driver_enet.h>

Public Attributes

bool **VlanFilterEnable**
bool **InnerVlanRxStatus**
bool **OuterVlanRxStatus**
bool **DoubleVlanEnable**
bool **SVlanEnable**
bool **VlanInverseMatch**
bool **HashFilterEnable**
ENET_MAC_VLANCompareType **HashCompareMode**
ENET_VLANFilterModeType **HashFilterMode**
ENET_MAC_VLANTagStripModeType **InnerVlanStrip**
ENET_MAC_VLANTagStripModeType **OuterVlanStrip**

Detailed Description

ENET VLAN filter configuration type.

Member Data Documentation

bool ENET_VlanFilterConfig::DoubleVlanEnable

MAC_VLAN_TAG_CTR[26] Enable double VLAN processing

ENET_MAC_VLANCompareType ENET_VlanFilterConfig::HashCompareMode

MAC_VLAN_TAG_CTR[16] VLAN tag hash compare mode, 16bits or 12 bits comparison

bool ENET_VlanFilterConfig::HashFilterEnable

MAC_VLAN_TAG_CTR[25] Enable VLAN hash table filtering

ENET_VLANFilterModeType ENET_VlanFilterConfig::HashFilterMode

MAC_VLAN_TAG_CTR[27,20,19] VLAN hash table filter mode

bool ENET_VlanFilterConfig::InnerVlanRxStatus

MAC_PACKET_FILTER[31] Enable inner VLAN tag in RX status

ENET_MAC_VLANTagStripModeType ENET_VlanFilterConfig::InnerVlanStrip

MAC_PACKET_FILTER[29:28] Stripping operation on the inner VLAN tag

bool ENET_VlanFilterConfig::OuterVlanRxStatus

MAC_VLAN_TAG_CTR[24] Enable outer VLAN tag in RX status

ENET_MAC_VLANTagStripModeType ENET_VlanFilterConfig::OuterVlanStrip

MAC_PACKET_FILTER[22:21] Stripping operation on the outer VLAN tag

bool ENET_VlanFilterConfig::SVlanEnable

MAC_VLAN_TAG_CTR[18] Enable S-VLAN processing

bool ENET_VlanFilterConfig::VlanFilterEnable

MAC_PACKET_FILTER[16] Enable inner VLAN tag in RX status

bool ENET_VlanFilterConfig::VlanInverseMatch

MAC_VLAN_TAG_CTR[17] Enable VLAN tag inverse match

The documentation for this struct was generated from the following file:

module_driver_enet.h

ENET_VlanPerfectFilter Struct Reference

ENET VLAN perfect filter type.

```
#include <module_driver_enet.h>
```

Public Attributes

bool **FilterGroupEnable**

bool **DMACHannelEnable**

uint16_t **VlanTag**

ENET_ChannelType **DMACHannel**

ENET_MAC_VLANCompareType **CompareMode**

ENET_VLANFilterModeType **FilterMode**

Detailed Description

ENET VLAN perfect filter type.

Member Data Documentation

ENET_MAC_VLANCompareType **ENET_VlanPerfectFilter::CompareMode**

MAC_VLAN_TAG_DATA[17] Compare mode, 12 bits or 16 bits comparison

ENET_ChannelType **ENET_VlanPerfectFilter::DMACHannel**

MAC_VLAN_TAG_DATA[25] The DMA channel number for the filter

bool **ENET_VlanPerfectFilter::DMACHannelEnable**

MAC_VLAN_TAG_DATA[24] Enable DMA channel select for VLAN tag filter

bool **ENET_VlanPerfectFilter::FilterGroupEnable**

MAC_VLAN_TAG_DATA[16] VLAN perfect filter group enable

ENET_VLANFilterModeType **ENET_VlanPerfectFilter::FilterMode**

MAC_VLAN_TAG_DATA[20:18] VLAN tag filter comparison mode

uint16_t ENET_VlanPerfectFilter::VlanTag

MAC_VLAN_TAG_DATA[15:0] VLAN tag value

The documentation for this struct was generated from the following file:

module_driver_enet.h

ERM_CfgType Struct Reference

The structure of the ERM Init configuration.

```
#include <module_driver_erm.h>
```

Public Attributes

```
uint8_t u8Channel  
uint8_t u8IntType  
uint8_t u8ErmEnable  
uint8_t _aligned [1U]
```

Detailed Description

The structure of the ERM Init configuration.

Member Data Documentation

uint8_t ERM_CfgType::_aligned[1U]

uint8_t ERM_CfgType::u8Channel

ERM channel No, See ERM Channel define

uint8_t ERM_CfgType::u8ErmEnable

Enable/Disable Erm channel interrupt

uint8_t ERM_CfgType::u8IntType

ERM Interrupt type, See ERM error interrupt define

The documentation for this struct was generated from the following file:

module_driver_erm.h

FCIIC_InterruptType Struct Reference

Structure for configuring FCIIC interrupts.

```
#include <module_driver_fciic.h>
```

Collaboration diagram for FCIIC_InterruptType:

IMAGE

Public Attributes

uint8_t **bEnErrorInterrupt**

uint8_t **bEnRxInterrupt**

uint8_t **_aligned** [6]

FCIIC_SlaveError_CallBackType pErrorNotify

FCIIC_RxInterrupt_CallBackType pRxNotify

Detailed Description

Structure for configuring FCIIC interrupts.

This structure holds the configuration for enabling and handling interrupts in the FCIIC.

Member Data Documentation

uint8_t FCIIC_InterruptType::_aligned[6]

Populate Fields.

uint8_t FCIIC_InterruptType::bEnErrorInterrupt

Enable error interrupt.

uint8_t FCIIC_InterruptType::bEnRxInterrupt

Enable receive interrupt.

FCIIC_SlaveError_CallBackType FCIIC_InterruptType::pErrorNotify

Error interrupt callback function address.

FCIIC_RxInterrupt_CallbackType FCIIC_InterruptType::pRxNotify

Receive interrupt callback function address.

The documentation for this struct was generated from the following file:

module_driver_fciic.h

FCIIC_MasterConfigType Struct Reference

Structure for configuring the FCIIC Master.

```
#include <module_driver_fciic.h>
```

Public Attributes

bool **bEnableMaster**

bool **bDebugEnabled**

uint8_t **u8SdaGlitchFilterWidth**

uint8_t **u8SclGlitchFilterWidth**

FCIIC_MasterPinConfigType **ePinConfig**

uint8_t **_aligned** [3]

uint32_t **u32BaudRate**

uint32_t **u32BusIdleTimeout**

uint32_t **u32PinLowTimeout**

Detailed Description

Structure for configuring the FCIIC Master.

This structure contains the configuration parameters for setting up the FCIIC Master.

Member Data Documentation

uint8_t FCIIC_MasterConfigType::_aligned[3]

Populate Fields.

bool FCIIC_MasterConfigType::bDebugEnabled

Allows transfers to continue even when the debugger halts execution.

bool FCIIC_MasterConfigType::bEnableMaster

Enables Master mode if set to true.

FCIIC_MasterPinConfigType FCIIC_MasterConfigType::ePinConfig

Selects the pin configuration for the FCIIC pins.

uint32_t FCIIC_MasterConfigType::u32BaudRate

Target baud rate in Hz for the communication.

uint32_t FCIIC_MasterConfigType::u32BusIdleTimeout

Bus idle timeout in nanoseconds. Set to 0 to disable timeout.

uint32_t FCIIC_MasterConfigType::u32PinLowTimeout

Pin low timeout in nanoseconds. Set to 0 to disable timeout.

uint8_t FCIIC_MasterConfigType::u8SclGlitchFilterWidth

Width of the SCL glitch filter in nanoseconds. Set to 0 to disable.

uint8_t FCIIC_MasterConfigType::u8SdaGlitchFilterWidth

Width of the SDA glitch filter in nanoseconds. Set to 0 to disable.

The documentation for this struct was generated from the following file:

module_driver_fciic.h

FCIIC_MasterTransferType Struct Reference

Structure for defining a master transfer in the FCIIC.

```
#include <module_driver_fciic.h>
```

Public Attributes

```
uint8_t u8SlaveAddress
FCIIC_DirectionType eDirection
uint16_t u16DataSize
uint8_t subaddressSize
uint32_t subaddress
uint8_t * pData
```

Detailed Description

Structure for defining a master transfer in the FCIIC.

This structure holds the configuration and data for a master transfer operation.

Member Data Documentation

FCIIC_DirectionType FCIIC_MasterTransferType::eDirection

Transfer direction: either **FCIIC_WRITE** or **FCIIC_READ**.

uint8_t* FCIIC_MasterTransferType::pData

Pointer to the buffer containing data to transfer or receive.

uint32_t FCIIC_MasterTransferType::subaddress

Sub-address for targeted register. Sent most significant byte first.

uint8_t FCIIC_MasterTransferType::subaddressSize

Size of the sub-address in bytes. Maximum is 4 bytes.

uint16_t FCIIC_MasterTransferType::u16DataSize

Total number of bytes to transfer.

uint8_t FCIIC_MasterTransferType::u8SlaveAddress

The 7-bit slave address of the target device.

The documentation for this struct was generated from the following file:

module_driver_fciic.h

FCIIC_RxDataType Struct Reference

Structure for holding received data from FCIIC operations.

```
#include <module_driver_fciic.h>
```

Public Attributes

uint8_t **u8Data**

Detailed Description

Structure for holding received data from FCIIC operations.

This structure contains the field to store the received 8-bit data.

Member Data Documentation

uint8_t FCIIC_RxDataType::u8Data

8-bit data.

The documentation for this struct was generated from the following file:

module_driver_fciic.h

FCIIC_SlaveConfigType Struct Reference

Configuration structure for initializing the FCIIC (Fast Communication Interface IC) Slave module.

```
#include <module_driver_fciic.h>
```

Public Attributes

- bool **bEnableSlave**
- bool **bI2cSlaveFilterEnable**
- bool **bUsedDMA**
- bool **bI2cSlaveAckStall**
- bool **bI2cSlaveTxStall**
- bool **bI2cSlaveRxStall**
- bool **bI2cSlaveAdrStall**
- uint8_t **u8AddrCfg**
- uint8_t **u8SdaGlitchFilterWidth**
- uint8_t **u8SclGlitchFilterWidth**
- uint16_t **slaveAddress**
- uint32_t **u32BusIdleTimeout**
- uint32_t **u32PinLowTimeout**

Detailed Description

Configuration structure for initializing the FCIIC (Fast Communication Interface IC) Slave module.

This structure contains various settings to configure the FCIIC Slave module according to the desired operation. Initialize this structure with default values by calling the **FCIIC_SlaveGetDefaultConfig()** function, then modify the fields as needed.

The structure can be declared const to store the configuration in flash memory to save RAM resources.

Member Data Documentation

bool FCIIC_SlaveConfigType::bEnableSlave

Enables Slave mode if set to true.

bool FCIIC_SlaveConfigType::bl2cSlaveAckStall

Enable SCL clock stretching when SSR[AVF] = 1.

bool FCIIC_SlaveConfigType::bl2cSlaveAdrStall

Enables Slave mode if set to true.

bool FCIIC_SlaveConfigType::bl2cSlaveFilterEnable

Enable or disable digital filter and output delay counter for slave mode.

bool FCIIC_SlaveConfigType::bl2cSlaveRxStall

Enable SCL clock stretching when SSR[RDF] = 1 during a slave-receive transfer.

bool FCIIC_SlaveConfigType::bl2cSlaveTxStall

Enable SCL clock stretching when the SSR[TDF] = 1 during a slave-transmit transfer.

bool FCIIC_SlaveConfigType::bUsedDMA

Enable DMA mode to transfer data.

uint16_t FCIIC_SlaveConfigType::slaveAddress

A slave address configuration.

uint32_t FCIIC_SlaveConfigType::u32BusIdleTimeout

Bus idle timeout in nanoseconds. Set to 0 to disable timeout.

uint32_t FCIIC_SlaveConfigType::u32PinLowTimeout

Pin low timeout in nanoseconds. Set to 0 to disable timeout.

uint8_t FCIIC_SlaveConfigType::u8AddrCfg

Configure different address-matching strategies.

uint8_t FCIIC_SlaveConfigType::u8SclGlitchFilterWidth

Width of the SCL glitch filter in nanoseconds. Set to 0 to disable.

uint8_t FCIIC_SlaveConfigType::u8SdaGlitchFilterWidth

Width of the SDA glitch filter in nanoseconds. Set to 0 to disable.

The documentation for this struct was generated from the following file:

`module_driver_fciic.h`

FCIIC_SlaveTransferType Struct Reference

Structure for defining a slave transfer in the FCIIC.

```
#include <module_driver_fciic.h>
```

Public Attributes

uint8_t **u8SlaveAddress**

uint8_t **subaddressSize**

uint16_t **u16DataSize**

uint32_t **subaddress**

uint8_t * **pData**

Detailed Description

Structure for defining a slave transfer in the FCIIC.

This structure holds the configuration and data for a slave transfer operation.

Member Data Documentation

uint8_t* FCIIC_SlaveTransferType::pData

Transfer buffer, which the buffer must be a legitimate address.

uint32_t FCIIC_SlaveTransferType::subaddress

Sub-address for targeted register. Sent most significant byte first.

uint8_t FCIIC_SlaveTransferType::subaddressSize

Size of the sub-address in bytes. Maximum is 4 bytes.

uint16_t FCIIC_SlaveTransferType::u16DataSize

Maximum length of bytes allowed to be transmitted, Avoiding array out-of-bounds

uint8_t FCIIC_SlaveTransferType::u8SlaveAddress

The 7-bit slave address of the target device.

The documentation for this struct was generated from the following file:

`module_driver_fciic.h`

FCIIC_TxDataType Struct Reference

Structure for holding transmit data for FCIIC operations.

```
#include <module_driver_fciic.h>
```

Public Attributes

FCIIC_TX_CMDType eCmd

uint8_t u8Data

Detailed Description

Structure for holding transmit data for FCIIC operations.

This structure contains the necessary fields to define a transmit command and its associated data.

Member Data Documentation

FCIIC_TX_CMDType FCIIC_TxDataType::eCmd

Command type.

uint8_t FCIIC_TxDataType::u8Data

8-bit data.

The documentation for this struct was generated from the following file:

module_driver_fciic.h

FCPIT_ChannelInitType Struct Reference

Fcpit initialization type.

```
#include <module_driver_fcpit.h>
```

Public Attributes

FCPIT_ChannelType eFcpitChannel

FCPIT_TimerModeType eMode

bool **bChainModeEn**

uint32_t **u32TimerValue**

Detailed Description

Fcpit initialization type.

Member Data Documentation

bool FCPIT_ChannelInitType::bChainModeEn

whether to use chain mode, if use this mode, channel must not be the channel 0

FCPIT_ChannelType FCPIT_ChannelInitType::eFcpitChannel

Fcpit channel number

FCPIT_TimerModeType FCPIT_ChannelInitType::eMode

Fcpit counter mode

uint32_t FCPIT_ChannelInitType::u32TimerValue

timer compare value, the range of value is related to the counter mode

The documentation for this struct was generated from the following file:

module_driver_fcpit.h

FCPIT_CommonInitType Struct Reference

```
#include <module_driver_fcpit.h>
```

Public Attributes

```
bool bDebugEn  
bool bLowPowerModeEn
```

Member Data Documentation

bool FCPIT_CommonInitType::bDebugEn

whether to use debug mode ,if enable this mode, the counter will stop when debugging.

bool FCPIT_CommonInitType::bLowPowerModeEn

Configure the timer channels to continue running or stop when the device enters the LPM mode

The documentation for this struct was generated from the following file:

```
module_driver_fcpit.h
```

FCPIT_IntType Struct Reference

Fcpit interrupt structure.

```
#include <module_driver_fcpit.h>
```

Public Attributes

FCPIT_ChannelType eFcpitChannel

bool bChannellsrEn

void(* pChannelCallback)(struct _FCPIT_HandleType *pHandle, uint32_t u32channel)

Detailed Description

Fcpit interrupt structure.

Member Data Documentation

bool FCPIT_IntType::bChannellsrEn

whether to use interrupt

FCPIT_ChannelType FCPIT_IntType::eFcpitChannel

Fcpit channel number

void(* FCPIT_IntType::pChannelCallback) (struct _FCPIT_HandleType *pHandle, uint32_t u32channel)

interrupt notification function

The documentation for this struct was generated from the following file:

module_driver_fcpit.h

FCPIT_TriggerType Struct Reference

```
#include <module_driver_fc pit.h>
```

Public Attributes

```
FCPIT_ChannelType eFcpitChannel  
FCPIT_TriggerSelectType eTriggerSel  
bool bStartOnTrigger  
bool bStopOnInterrupt  
bool bReloadOnTrigger
```

Member Data Documentation

bool FCPIT_TriggerType::bReloadOnTrigger

Fcpit timer reload when triggered

bool FCPIT_TriggerType::bStartOnTrigger

Fcpit timer start when triggered

bool FCPIT_TriggerType::bStopOnInterrupt

Fcpit timer stop on interrupt

FCPIT_ChannelType FCPIT_TriggerType::eFcpitChannel

Fcpit channel number

FCPIT_TriggerSelectType FCPIT_TriggerType::eTriggerSel

trigger source

The documentation for this struct was generated from the following file:

module_driver_fc pit.h

FCSMU_CrcCfgType Struct Reference

FCSMU CRC configure structure.

```
#include <module_driver_fcsmu.h>
```

Public Attributes

uint16_t **u16ErrorReqMode**

uint16_t **u16Mode**

Detailed Description

FCSMU CRC configure structure.

Member Data Documentation

uint16_t **FCSMU_CrcCfgType::u16ErrorReqMode**

uint16_t **FCSMU_CrcCfgType::u16Mode**

The documentation for this struct was generated from the following file:

module_driver_fcsmu.h

FCSMU_InitCfgType Struct Reference

FCSMU init configure structure.

```
#include <module_driver_fcsmu.h>
```

Public Attributes

```
uint32_t u32FaultChannel
uint32_t u32WarnTo
uint32_t u32WarnChannel
uint32_t u32WarnInterruptChannel
uint32_t u32FaultInterruptChannel
uint32_t u32FaultResetChannel
uint32_t u32SoftwareClearedChannel
```

Detailed Description

FCSMU init configure structure.

Member Data Documentation

uint32_t FCSMU_InitCfgType::u32FaultChannel

FCSMU fault channel.

uint32_t FCSMU_InitCfgType::u32FaultInterruptChannel

FCSMU fault interrupt channel.

uint32_t FCSMU_InitCfgType::u32FaultResetChannel

FCSMU fault reset channel.

uint32_t FCSMU_InitCfgType::u32SoftwareClearedChannel

FCSMU fault is Cleard by software.

uint32_t FCSMU_InitCfgType::u32WarnChannel

FCSMU warning channel.

uint32_t FCSMU_InitCfgType::u32WarnInterruptChannel

FCSMU warning interrupt channel.

uint32_t FCSMU_InitCfgType::u32WarnTo

FCSMU warning timeoout value.

The documentation for this struct was generated from the following file:

module_driver_fcsmu.h

FCSMU_StatusOutputCfgType Struct Reference

FCSMU status output pin configure structure.

```
#include <module_driver_fcsmu.h>
```

Public Attributes

- bool **bEnable**
- bool **bFastMode**
- uint8_t **u8SoutCtrl**
- uint8_t **u8Polarity**
- uint32_t **u32Protocal**
- uint32_t **u32Delaytimer**
- uint32_t **u32Divder**
- uint32_t **u32SoutChannel**
- uint32_t **u32SoutMinTmrInterval**
- bool **bDivex**
- bool **bSoutMinTmr**
- uint8_t **u8FttiTime**
- uint8_t **_aligned**

Detailed Description

FCSMU status output pin configure structure.

Member Data Documentation

uint8_t FCSMU_StatusOutputCfgType::_aligned

bool FCSMU_StatusOutputCfgType::bDivex

Enable or disable SOUT Divider Extend Control.

bool FCSMU_StatusOutputCfgType::bEnable

Enable or disable status output.

bool FCSMU_StatusOutputCfgType::bFastMode

Enable or disable fast mode.

bool FCSMU_StatusOutputCfgType::bSoutMinTmr

SOUT Minimum Timer Enable

uint32_t FCSMU_StatusOutputCfgType::u32Delaytimer

Configure the safe mode request delay in cycles of CLKSAFE.

uint32_t FCSMU_StatusOutputCfgType::u32Divder

Configure the status output divider ratio. $SOUT_freq = CLKSAFE_freq / ((SOUT_DIV + 1) * 2 * 256)$

uint32_t FCSMU_StatusOutputCfgType::u32Protocal

Status output protocal.

uint32_t FCSMU_StatusOutputCfgType::u32SoutChannel

Configure the status output channel.

uint32_t FCSMU_StatusOutputCfgType::u32SoutMinTmrInterval

Minimum SOUT Timer Interval

uint8_t FCSMU_StatusOutputCfgType::u8FttiTime

Fault Tolerance Time

uint8_t FCSMU_StatusOutputCfgType::u8Polarity

Status output polarity.

uint8_t FCSMU_StatusOutputCfgType::u8SoutCtrl

Configure Sout Control.

The documentation for this struct was generated from the following file:

module_driver_fcsmu.h

FCUART_DataType Struct Reference

UART transmit and receive data define.

```
#include <module_driver_fcuart.h>
```

Public Attributes

```
uint8_t * pDatas  
uint32_t u32DataLen
```

Detailed Description

UART transmit and receive data define.

Member Data Documentation

uint8_t* FCUART_DataType::pDatas

Data buffer point, Must be initial with a array address

uint32_t FCUART_DataType::u32DataLen

data length

The documentation for this struct was generated from the following file:

`module_driver_fcuart.h`

FCUART_InitType Struct Reference

UART initial data define.

```
#include <module_driver_fcuart.h>
```

Collaboration diagram for FCUART_InitType:

IMAGE

Public Attributes

FCUART_InterruptType tInterruptCfg
uint32_t **u32ClkSrcHz**
FCUART_BitModeType eBitMode
bool **bParityEnable**
FCUART_ParityType eParityType
FCUART_StopBitNumType eStopBit
bool **bEnTxFifo**
uint8_t **u8TxFifoWaterMark**
bool **bEnRxFifo**
uint8_t **u8RxFifoWaterMark**
FCUART_Fifo_RxIdleCharNumType eFifoRxIdleCharNum
bool **bEnTxEmptyDma**
bool **bEnRxFullDma**
bool **bEnRxIdleDma**
FCUART_IdleCharNumType eIdleCharNum
FCUART_IdleStartType eIdleStart
uint32_t **u32Baudrate**
uint32_t **u32TransmitTimeout**

Detailed Description

UART initial data define.

Member Data Documentation

bool FCUART_InitType::bEnRxFifo

Enable Rx FIFO, 16 data words depth

bool FCUART_InitType::bEnRxFullDma

Enable receiver full DMA

bool FCUART_InitType::bEnRxIdleDma

Enable Rx Idle DMA

bool FCUART_InitType::bEnTxEmptyDma

Enable transmit DMA

bool FCUART_InitType::bEnTxFifo

Enable Tx FIFO, 16 data words depth

bool FCUART_InitType::bParityEnable

Parity Enable=1

FCUART_BitModeType FCUART_InitType::eBitMode

8bits or 9bits

FCUART_Fifo_RxIdleCharNumType FCUART_InitType::eFifoRxIdleCharNum

Fifo Rx idle character number

FCUART_IdleCharNumType FCUART_InitType::eIdleCharNum

Idle character number

FCUART_IdleStartType FCUART_InitType::eIdleStart

Idle character start type

FCUART_ParityType FCUART_InitType::eParityType

parity type

FCUART_StopBitNumType FCUART_InitType::eStopBit

stop bit num;

FCUART_InterruptType FCUART_InitType::tInterruptCfg

interrupt configuration parameter

uint32_t FCUART_InitType::u32Baudrate

normal baud-rate

uint32_t FCUART_InitType::u32ClkSrcHz

module clock hz

uint32_t FCUART_InitType::u32TransmitTimeout

transmit timeout tick, default 3000U

uint8_t FCUART_InitType::u8RxFifoWaterMark

Rx FIFO, When FIFO/buffer number greater than this an interrupt or a DMA request will be generated

uint8_t FCUART_InitType::u8TxFifoWaterMark

Tx FIFO, When FIFO/buffer number equal or less than this an interrupt or a DMA request will be generated

The documentation for this struct was generated from the following file:

module_driver_fcuart.h

FCUART_InterruptType Struct Reference

UART callback functions data define.

```
#include <module_driver_fcuart.h>
```

Collaboration diagram for FCUART_InterruptType:

IMAGE

Public Attributes

```
bool bEnErrorInterrupt
FCUART_ErrorInterrupt_CallBackType pErrorNotify
bool bEnRxInterrupt
FCUART_DataType * pRxBuf
FCUART_TxRxInterrupt_CallBackType pRxNotify
bool bEnTxInterrupt
FCUART_DataType * pTxBuf
FCUART_TxRxInterrupt_CallBackType pTxEmptyNotify
FCUART_TxRxInterrupt_CallBackType pTxCompleteNotify
bool bEnIdleInterrupt
FCUART_IdleInterrupt_CallBackType pIdleNotify
```

Detailed Description

UART callback functions data define.

Member Data Documentation

bool FCUART_InterruptType::bEnErrorInterrupt

enable error interrupt

bool FCUART_InterruptType::bEnIdleInterrupt

enable idle interrupt

bool FCUART_InterruptType::bEnRxInterrupt

enable receive interrupt

bool FCUART_InterruptType::bEnTxInterrupt

enable receive interrupt

FCUART_ErrorInterrupt_CallBackType FCUART_InterruptType::pErrorNotify

error interrupt callback function address

FCUART_IdleInterrupt_CallBackType FCUART_InterruptType::pIdleNotify

idle interrupt callback function address

FCUART_DataType* FCUART_InterruptType::pRxBuf

receive interrupt message buffer

FCUART_TxRxInterrupt_CallBackType FCUART_InterruptType::pRxNotify

receive interrupt callback function address

FCUART_DataType* FCUART_InterruptType::pTxBuf

transfer interrupt message buffer

FCUART_TxRxInterrupt_CallBackType FCUART_InterruptType::pTxCompleteNotify

transfer complete interrupt callback function address

FCUART_TxRxInterrupt_CallBackType FCUART_InterruptType::pTxEmptyNotify

transfer empty interrupt callback function address

The documentation for this struct was generated from the following file:

module_driver_fcuart.h

FCUART_WakeupType Struct Reference

UART WakeUp functions data define.

```
#include <module_driver_fcuart.h>
```

Public Attributes

bool **bEnWakeup**

uint32_t **u32WakeUpData**

Detailed Description

UART WakeUp functions data define.

Member Data Documentation

bool FCUART_WakeupType::bEnWakeup

enable wake-up

uint32_t FCUART_WakeupType::u32WakeUpData

wake-up data

The documentation for this struct was generated from the following file:

module_driver_fcuart.h

FLASH_DRIVER_ParamType Struct Reference

Flash driver parameter define.

```
#include <module_driver_flash.h>
```

Public Attributes

uint32_t **u32Address**

uint32_t **u32Length**

uint8_t * **pData**

uint8_t(* **wdTriggerFct**)(void)

uint32_t **u32ErrorAddress**

Detailed Description

Flash driver parameter define.

Member Data Documentation

uint8_t* FLASH_DRIVER_ParamType::pData

Pointer to data buffer (read only)

uint32_t FLASH_DRIVER_ParamType::u32Address

Logical target address

uint32_t FLASH_DRIVER_ParamType::u32ErrorAddress

Error address

uint32_t FLASH_DRIVER_ParamType::u32Length

Length in logical sectors or bytes

uint8_t(* FLASH_DRIVER_ParamType::wdTriggerFct) (void)

Pointer to watchdog handling function

The documentation for this struct was generated from the following file:

module_driver_flash.h

FLASH_Lock_ParamType Struct Reference

unlock parameter define

```
#include <module_driver_flash.h>
```

Public Attributes

uint32_t **u32Address**

uint32_t **u32Length**

FLASH_LOCK_TYPE **bLock**

FLASH_TYPE **bFlash**

Detailed Description

unlock parameter define

Member Data Documentation

FLASH_TYPE **FLASH_Lock_ParamType::bFlash**

flash type

FLASH_LOCK_TYPE **FLASH_Lock_ParamType::bLock**

lock type

uint32_t **FLASH_Lock_ParamType::u32Address**

Logical target address

uint32_t **FLASH_Lock_ParamType::u32Length**

Length in logical sectors or pages

The documentation for this struct was generated from the following file:

`module_driver_flash.h`

FLEXCAN_BaudRegType Struct Reference

Baud-rate clock and divider.

```
#include <module_driver_flexcan.h>
```

Public Attributes

uint32_t **u32Presdiv**

uint32_t **u32Propseg**

uint32_t **u32Pseg1**

uint32_t **u32Pseg2**

uint32_t **u32Rjw**

Detailed Description

Baud-rate clock and divider.

Member Data Documentation

uint32_t FLEXCAN_BaudRegType::u32Presdiv

Presdiv for can

uint32_t FLEXCAN_BaudRegType::u32Propseg

Propseg for can

uint32_t FLEXCAN_BaudRegType::u32Pseg1

Pseg1 for can

uint32_t FLEXCAN_BaudRegType::u32Pseg2

Pseg2 for can

uint32_t FLEXCAN_BaudRegType::u32Rjw

RJW for can

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_EnhancedFifoConfigType Struct Reference

FLEXCAN enhanced fifo configuration structure definition.

```
#include <module_driver_flexcan.h>
```

Collaboration diagram for FLEXCAN_EnhancedFifoConfigType:

IMAGE

Public Attributes

```
uint8_t bEnFifoDMA
uint8_t bDataAvaliableInt
uint8_t u8FifoWaterMark
uint8_t bWaterMarkInt
uint8_t bOverflowInt
uint8_t bUnderflowInt
FLEXCAN_DataWidthType eFifoDataLength
uint8_t u8RxFilterFifoCnt
FLEXCAN_FifoFilterType * pRxFilterFifoList
```

Detailed Description

FLEXCAN enhanced fifo configuration structure definition.

Member Data Documentation

uint8_t FLEXCAN_EnhancedFifoConfigType::bDataAvaliableInt

if at least one message in fifo interrupt

uint8_t FLEXCAN_EnhancedFifoConfigType::bEnFifoDMA

if enable can fifo dma, set 1U, only fifo support

uint8_t FLEXCAN_EnhancedFifoConfigType::bOverflowInt

fifo full and data lost in fifo interrupt

uint8_t FLEXCAN_EnhancedFifoConfigType::bUnderflowInt

no data in fifo but read data from fifo cause interrupt

uint8_t FLEXCAN_EnhancedFifoConfigType::bWaterMarkInt

if fifo depth greater than or equal watermark will trigger interrupt

FLEXCAN_DataWidthType FLEXCAN_EnhancedFifoConfigType::eFifoDataLength

data length for enhanced rx fifo DMALW

FLEXCAN_FifoFilterType* FLEXCAN_EnhancedFifoConfigType::pRxFilterFifoList

rx id list in fifo

uint8_t FLEXCAN_EnhancedFifoConfigType::u8FifoWaterMark

The watermark for only in Enhanced FIFO mode, can trigger dma

uint8_t FLEXCAN_EnhancedFifoConfigType::u8RxFilterFifoCnt

rx id filter count, in fifo

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_ErrorInfoType Struct Reference

FLEXCAN Error Information.

```
#include <module_driver_flexcan.h>
```

Public Attributes

uint32_t **u32RxErrCnt_Fast**

uint32_t **u32RxErrCnt**

uint32_t **u32TxErrCnt_Fast**

uint32_t **u32TxErrCnt**

uint32_t **u32ESR1Value**

Detailed Description

FLEXCAN Error Information.

Member Data Documentation

uint32_t FLEXCAN_ErrorInfoType::u32ESR1Value

Error Register ESR1 Value

uint32_t FLEXCAN_ErrorInfoType::u32RxErrCnt

Receive Error Counter

uint32_t FLEXCAN_ErrorInfoType::u32RxErrCnt_Fast

Receive Error Counter for fast bits

uint32_t FLEXCAN_ErrorInfoType::u32TxErrCnt

Transmit Error Counter

uint32_t FLEXCAN_ErrorInfoType::u32TxErrCnt_Fast

Transmit Error Counter for fast bits

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_FifoConfigType Struct Reference

FLEXCAN fifo configuration structure definition.

```
#include <module_driver_flexcan.h>
```

Collaboration diagram for FLEXCAN_FifoConfigType:

IMAGE

Public Attributes

uint8_t **bEnFifo**

FLEXCAN_FifoType **eFifoType**

FLEXCAN_LegacyFifoConfigType **tLegacyFifoCfg**

FLEXCAN_EnhancedFifoConfigType **tEnhancedFifoCfg**

Detailed Description

FLEXCAN fifo configuration structure definition.

Member Data Documentation

uint8_t FLEXCAN_FifoConfigType::bEnFifo

0 is disable, 1 is enable

FLEXCAN_FifoType FLEXCAN_FifoConfigType::eFifoType

0 is legacy fifo, 1 is enhanced fifo, only one of them

FLEXCAN_EnhancedFifoConfigType FLEXCAN_FifoConfigType::tEnhancedFifoCfg

Enhanced Rx FIFO configuration

FLEXCAN_LegacyFifoConfigType FLEXCAN_FifoConfigType::tLegacyFifoCfg

Legacy Rx FIFO configuration

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_FifoFilterType Struct Reference

Receive Message Buffer Configuration.

```
#include <module_driver_flexcan.h>
```

Public Attributes

FLEXCAN_IdType eRxFrameType

uint32_t **u32RxCanId**

uint32_t **u32RxCanIdMask**

Detailed Description

Receive Message Buffer Configuration.

Member Data Documentation

FLEXCAN_IdType FLEXCAN_FifoFilterType::eRxFrameType

FLEXCAN ID type, 0 STD, 1 EXT

uint32_t FLEXCAN_FifoFilterType::u32RxCanId

FLEXCAN received ID

uint32_t FLEXCAN_FifoFilterType::u32RxCanIdMask

FLEXCAN received ID mask, if mask bit is 1, received FLEXCAN id bit must same to u32RxCanId, or mask bit is 0, id bit is ignored

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_FifoIssueType Struct Reference

FIFO error response type.

```
#include <module_driver_flexcan.h>
```

Public Attributes

FLEXCAN_FifoType eFifoType

uint8_t **u8FifoWarings**

uint8_t **u8FifoOverflow**

uint8_t **u8FifoUnderflow**

Detailed Description

FIFO error response type.

Member Data Documentation

FLEXCAN_FifoType FLEXCAN_FifoIssueType::eFifoType

0 means legacy fifo, 1 means enhanced fifo, only one of them

uint8_t FLEXCAN_FifoIssueType::u8FifoOverflow

0 means data lost

uint8_t FLEXCAN_FifoIssueType::u8FifoUnderflow

0 means no enhanced fifo underflow, 0 means contain enhanced fifo underflow

uint8_t FLEXCAN_FifoIssueType::u8FifoWarings

0 means no legacy fifo waring, 0 means contain legacy fifo waring

The documentation for this struct was generated from the following file:

`module_driver_flexcan.h`

FLEXCAN_InitType Struct Reference

FLEXCAN init structure definition.

```
#include <module_driver_flexcan.h>
```

Collaboration diagram for FLEXCAN_InitType:



Public Attributes

- FLEXCAN_ClockSrcType** eClkSrcSel
- uint8_t** bListenOnly
- FLEXCAN_BaudRegType** tNormalBaud
- uint8_t** bEnFd
- FLEXCAN_BaudRegType** tFdBaud
- uint8_t** bEnSsp
- uint8_t** bEnAutoRec
- FLEXCAN_FifoConfigType** tRxFifoFilterCfg
- FLEXCAN_MbConfigType** tMbCfg
- uint8_t** bEnErrInt
- uint8_t** bEnBusOffInt
- uint8_t** bEnBusOffDoneInt
- uint8_t** bEnWarningInt
- FLEXCAN_PNET_ConfigType** tPnetCfg

Detailed Description

FLEXCAN init structure definition.

Member Data Documentation

uint8_t FLEXCAN_InitType::bEnAutoRec

Enable auto recovery after bus-off

uint8_t FLEXCAN_InitType::bEnBusOffDoneInt

if enable can busoff leaved interrupt(only occurred autorecovery), set 1U

uint8_t FLEXCAN_InitType::bEnBusOffInt

if enable can busoff interrupt, set 1U

uint8_t FLEXCAN_InitType::bEnErrInt

if enable can error interrupt, set 1U

uint8_t FLEXCAN_InitType::bEnFd

enable FLEXCAN fd

uint8_t FLEXCAN_InitType::bEnSsp

Enable Second Sample Point for data baudrate

uint8_t FLEXCAN_InitType::bEnWarningInt

if enable can tx/rx warning interrupt, set 1U

uint8_t FLEXCAN_InitType::bListenOnly

Listen Only

FLEXCAN_ClockSrcType FLEXCAN_InitType::eClkSrcSel

Clock Source Select

FLEXCAN_BaudRegType FLEXCAN_InitType::tFdBaud

Presdiv, Propseg, Pseg1, Pseg2, rjw for can fd data baud

FLEXCAN_MbConfigType FLEXCAN_InitType::tMbCfg

Can Tx MB and Rx MB Filter configuration parameter

FLEXCAN_BaudRegType FLEXCAN_InitType::tNormalBaud

Presdiv,Propseg,Pseg1,Pseg2,rjw for can baud

FLEXCAN_PNET_ConfigType FLEXCAN_InitType::tPnetCfg

Pretended Network configuration for wake-up in STOP Mode

FLEXCAN_FifoConfigType FLEXCAN_InitType::tRxFifoFilterCfg

Can Rx fifo Filter configuration parameter

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_LegacyFifoConfigType Struct Reference

FLEXCAN legacy fifo configuration structure definition.

```
#include <module_driver_flexcan.h>
```

Collaboration diagram for FLEXCAN_LegacyFifoConfigType:

IMAGE

Public Attributes

uint8_t **bEnFifoDMA**

uint8_t **bDataAvaliableInt**

uint8_t **bWarningInt**

uint8_t **bOverflowInt**

uint8_t **u8RxFilterFifoCnt**

FLEXCAN_FifoFilterType * pRxFilterFifoList

Detailed Description

FLEXCAN legacy fifo configuration structure definition.

Member Data Documentation

uint8_t FLEXCAN_LegacyFifoConfigType::bDataAvaliableInt

if at least one message in fifo interrupt, can trigger dma

uint8_t FLEXCAN_LegacyFifoConfigType::bEnFifoDMA

if enable can fifo dma, set 1U, only fifo support

uint8_t FLEXCAN_LegacyFifoConfigType::bOverflowInt

more than 5 messages in fifo and data lost interrupt

uint8_t FLEXCAN_LegacyFifoConfigType::bWarningInt

increase from 4 to 5 messages in fifo interrupt

FLEXCAN_FifoFilterType* FLEXCAN_LegacyFifoConfigType::pRxFilterFifoList

rx id list in fifo

uint8_t FLEXCAN_LegacyFifoConfigType::u8RxFilterFifoCnt

rx id filter count, in fifo

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_MbConfigType Struct Reference

FLEXCAN mb configuration structure definition.

#include <module_driver_flexcan.h>

Collaboration diagram for FLEXCAN_MbConfigType:

IMAGE

Public Attributes

FLEXCAN_DataWidthType aDataWidthRange [FLEXCAN_FD_DATALEN_RANGE_NUM_MAX]

uint8_t u8MBCnt

FLEXCAN_MbSettingType * pMBList

Detailed Description

FLEXCAN mb configuration structure definition.

Member Data Documentation

FLEXCAN_DataWidthType

FLEXCAN_MbConfigType::aDataWidthRange[FLEXCAN_FD_DATALEN_RANGE_NUM_MAX]

data length range buffer, MBDSR0..MBDSRn, only used in canfd

FLEXCAN_MbSettingType* FLEXCAN_MbConfigType::pMBList

rx id list in MB

uint8_t FLEXCAN_MbConfigType::u8MBCnt

rx id filter count in MB

The documentation for this struct was generated from the following file:

`module_driver_flexcan.h`

FLEXCAN_MbSettingType Struct Reference

Message Buffer Configuration.

```
#include <module_driver_flexcan.h>
```

Public Attributes

uint8_t **bIsTxMB**

uint8_t **bEnMBInt**

FLEXCAN_IdType **eRxFrameType**

uint32_t **u32RxCanId**

uint32_t **u32RxCanIdMask**

Detailed Description

Message Buffer Configuration.

Member Data Documentation

uint8_t FLEXCAN_MbSettingType::bEnMBInt

if enable can current MB interrupt, set 1U

uint8_t FLEXCAN_MbSettingType::bIsTxMB

0 is rx, 1 is tx

FLEXCAN_IdType FLEXCAN_MbSettingType::eRxFrameType

FLEXCAN ID type, 0 STD, 1 EXT

uint32_t FLEXCAN_MbSettingType::u32RxCanId

FLEXCAN received ID

uint32_t FLEXCAN_MbSettingType::u32RxCanIdMask

FLEXCAN received ID mask, if mask bit is 1, received FLEXCAN id bit must same to u32RxCanId, or mask bit is 0, id bit is ignored

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_PNET_ConfigType Struct Reference

CAN Pretended network configuration.

#include <module_driver_flexcan.h>

Collaboration diagram for FLEXCAN_PNET_ConfigType:

IMAGE

Public Attributes

uint8_t **bPNetEnable**

uint8_t **bTimeoutEn**

uint16_t **u16Timeout**

uint8_t **bFilterEnable**

FLEXCAN_PNET_FilterType ePnetFilterType

uint8 **u8MatchTimes**

FLEXCAN_PNET_IDCfgType tIdCfg

FLEXCAN_PNET_PayloadCfgType tPayloadCfg

Detailed Description

CAN Pretended network configuration.

Member Data Documentation

uint8_t FLEXCAN_PNET_ConfigType::bFilterEnable

Enable filter message wake-up

uint8_t FLEXCAN_PNET_ConfigType::bPNetEnable

Enable Pretended network settings

uint8_t FLEXCAN_PNET_ConfigType::bTimeoutEn

Enable timeout for no message matched

FLEXCAN_PNET_FilterType FLEXCAN_PNET_ConfigType::ePnetFilterType

Pretended network filter type

FLEXCAN_PNET_IDCfgType FLEXCAN_PNET_ConfigType::tIdCfg

When id selection set, configure this structure

FLEXCAN_PNET_PayloadCfgType FLEXCAN_PNET_ConfigType::tPayloadCfg

When payload selection set, configure this structure

uint16_t FLEXCAN_PNET_ConfigType::u16Timeout

time start after MCR[PNET_EN] set, period is 64 times of can bit time

uint8 FLEXCAN_PNET_ConfigType::u8MatchTimes

when match times reached, interrupt will be trigger, 1-255

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_PNET_IDConfigType Struct Reference

CAN Pretended network id filter setting.

#include <module_driver_flexcan.h>

Collaboration diagram for FLEXCAN_PNET_IDConfigType:

IMAGE

Public Attributes

- FLEXCAN_PNET_IdFilterType eFilterType
- FLEXCAN_PNET_IdSettingType tID1
- FLEXCAN_PNET_IdSettingType tID2_Mask

Detailed Description

CAN Pretended network id filter setting.

Member Data Documentation

FLEXCAN_PNET_IdFilterType FLEXCAN_PNET_IDConfigType::eFilterType

ID filter type for Pretented Network

FLEXCAN_PNET_IdSettingType FLEXCAN_PNET_IDConfigType::tID1

ID1 for filter

FLEXCAN_PNET_IdSettingType FLEXCAN_PNET_IDConfigType::tID2_Mask

ID2 for filter range or mask

The documentation for this struct was generated from the following file:

`module_driver_flexcan.h`

FLEXCAN_PNET_IdSettingType Struct Reference

CAN ID filter setting.

```
#include <module_driver_flexcan.h>
```

Public Attributes

uint8_t **u8IDE**

uint8_t **u8RTR**

uint32_t **u32Id**

Detailed Description

CAN ID filter setting.

Member Data Documentation

uint32_t FLEXCAN_PNET_IdSettingType::u32Id

CAN ID for wake-up

uint8_t FLEXCAN_PNET_IdSettingType::u8IDE

0-Standard ID, 1-Extended ID

uint8_t FLEXCAN_PNET_IdSettingType::u8RTR

0-Data Frame, 1-Remote Frame

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_PNET_PayloadConfigType Struct Reference

```
#include <module_driver_flexcan.h>
```

Public Attributes

FLEXCAN_PNET_PayloadFilterType eFilterType

uint8_t u8DataLen1

uint8_t u8DataLen2

uint32_t aData1 [2]

uint32_t aData2 [2]

Member Data Documentation

uint32_t FLEXCAN_PNET_PayloadConfigType::aData1[2]

Data Payload 1, LO+HI

uint32_t FLEXCAN_PNET_PayloadConfigType::aData2[2]

Data Payload 2, LO+HI

FLEXCAN_PNET_PayloadFilterType FLEXCAN_PNET_PayloadConfigType::eFilterType

Payload filter type for Pretented Network

uint8_t FLEXCAN_PNET_PayloadConfigType::u8DataLen1

Data length 1 for filter

uint8_t FLEXCAN_PNET_PayloadConfigType::u8DataLen2

Data length 2 for filter

The documentation for this struct was generated from the following file:

`module_driver_flexcan.h`

FLEXCAN_RxMsgListType Struct Reference

FLEXCAN polling receive message.

```
#include <module_driver_flexcan.h>
```

Collaboration diagram for FLEXCAN_RxMsgListType:

IMAGE

Public Attributes

uint8_t u8RxMsgCnt

FLEXCAN_RxMsgType * pRxMsgBuf

Detailed Description

FLEXCAN polling receive message.

Member Data Documentation

FLEXCAN_RxMsgType* FLEXCAN_RxMsgListType::pRxMsgBuf

Receive Message Buffer Address

uint8_t FLEXCAN_RxMsgListType::u8RxMsgCnt

Receive Message Buffer Count

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_RxMsgType Struct Reference

FLEXCAN receive message.

```
#include <module_driver_flexcan.h>
```

Public Attributes

```
uint8_t u8CanIndex  
uint8_t u8RxHandle  
uint8_t bEnFd  
uint8_t bEnBrs  
FLEXCAN_IdType eFrameType  
FLEXCAN_DataType eDataType  
uint32_t u32CanId  
uint32_t u32DataLen  
uint8_t aData [80]
```

Detailed Description

FLEXCAN receive message.

Member Data Documentation

uint8_t FLEXCAN_RxMsgType::aData[80]

data buffer

uint8_t FLEXCAN_RxMsgType::bEnBrs

enable canfd data bit switch

uint8_t FLEXCAN_RxMsgType::bEnFd

enable FLEXCAN fd

FLEXCAN_DataType FLEXCAN_RxMsgType::eDataType

0 Data, 1 remote

FLEXCAN_IdType FLEXCAN_RxMsgType::eFrameType

0 STD, 1 EXT

uint32_t FLEXCAN_RxMsgType::u32CanId

FLEXCAN id

uint32_t FLEXCAN_RxMsgType::u32DataLen

data length

uint8_t FLEXCAN_RxMsgType::u8CanIndex

FLEXCAN Index, 0,1,2..NUM

uint8_t FLEXCAN_RxMsgType::u8RxHandle

Handle set by user, FIFO always FF

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_TxMsgType Struct Reference

FLEXCAN transmit message.

```
#include <module_driver_flexcan.h>
```

Public Attributes

uint8_t **u8TxHandle**

uint8_t **bEnFd**

uint8_t **bEnBrs**

uint32_t **u32CanId**

FLEXCAN_IdType **eFrameType**

FLEXCAN_DataType **eDataType**

uint32_t **u32DataLen**

uint8_t **aData** [64]

Detailed Description

FLEXCAN transmit message.

Member Data Documentation

uint8_t FLEXCAN_TxMsgType::aData[64]

data buffer

uint8_t FLEXCAN_TxMsgType::bEnBrs

enable canfd data bit switch

uint8_t FLEXCAN_TxMsgType::bEnFd

enable FLEXCAN fd

FLEXCAN_DataType FLEXCAN_TxMsgType::eDataType

0 Data, 1 remote

FLEXCAN_IdType FLEXCAN_TxMsgType::eFrameType

0 STD, 1 EXT

uint32_t FLEXCAN_TxMsgType::u32CanId

FLEXCAN id

uint32_t FLEXCAN_TxMsgType::u32DataLen

data length

uint8_t FLEXCAN_TxMsgType::u8TxHandle

one message buffer used one handler, range 0 ~ txMbCnt-1

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FLEXCAN_WakeupMsgType Struct Reference

FLEXCAN wake-up receive message.

```
#include <module_driver_flexcan.h>
```

Public Attributes

```
uint8_t u8CanIndex  
FLEXCAN_PNET_WakeupSrcType eWakeupSrc  
FLEXCAN_IdType eFrameType  
FLEXCAN_DataType eDataType  
uint32_t u32CanId  
uint32_t u32DataLen  
uint8_t aData [8]
```

Detailed Description

FLEXCAN wake-up receive message.

Member Data Documentation

uint8_t FLEXCAN_WakeupMsgType::aData[8]

data buffer, do not support canfd

FLEXCAN_DataType FLEXCAN_WakeupMsgType::eDataType

0 Data, 1 remote

FLEXCAN_IdType FLEXCAN_WakeupMsgType::eFrameType

0 STD, 1 EXT

FLEXCAN_PNET_WakeupSrcType FLEXCAN_WakeupMsgType::eWakeupSrc

Wake-up source

uint32_t FLEXCAN_WakeupMsgType::u32CanId

FLEXCAN id

uint32_t FLEXCAN_WakeupMsgType::u32DataLen

data length

uint8_t FLEXCAN_WakeupMsgType::u8CanIndex

FLEXCAN Index, 0,1,2..NUM

The documentation for this struct was generated from the following file:

module_driver_flexcan.h

FPC_Type Struct Reference

```
#include <HwA_ism.h>
```

Public Attributes

```
__IO uint32_t FPC_STATUS
```

```
__IO uint32_t FPC_CTRL
```

```
__IO uint32_t FPC_CONFIG
```

```
__IO uint32_t FPC_TIMER
```

Member Data Documentation

```
__IO uint32_t FPC_Type::FPC_CONFIG
```

```
__IO uint32_t FPC_Type::FPC_CTRL
```

```
__IO uint32_t FPC_Type::FPC_STATUS
```

```
__IO uint32_t FPC_Type::FPC_TIMER
```

The documentation for this struct was generated from the following file:

```
HwA_ism.h
```


FREQM_CfgType Struct Reference

FREQM configure structure.

```
#include <module_driver_freqm.h>
```

Public Attributes

uint32_t **u32MesLen**

uint32_t **u32RefTo**

uint8_t **u8ClkSel**

uint8_t **u8PredivVal**

uint8_t **_aligned** [2U]

Detailed Description

FREQM configure structure.

Member Data Documentation

uint8_t FREQM_CfgType::_aligned[2U]

uint32_t FREQM_CfgType::u32MesLen

uint32_t FREQM_CfgType::u32RefTo

uint8_t FREQM_CfgType::u8ClkSel

clock measure select, See freqm measured clock define

uint8_t FREQM_CfgType::u8PredivVal

Frequency of the Reload Opportunities. Range is 0-31,

The documentation for this struct was generated from the following file:

module_driver_freqm.h

FTU_CommonType Struct Reference

The basic configuration option for the FTU peripheral.

```
#include <module_driver_ftu.h>
```

Public Attributes

FTU_PrescalerType ePrescaler

FTU_ClkSrcType eClkSrc

FTU_FilterPrescalerType eFliterPrescaler

uint32_t u32OverflowValue

uint8_t u8ReloadFreq

FTU_TrigModeType eHwTrigMode

FTU_DebugModeType eDbgMode

FTU_UpDownDisableType eUpDownDisable

bool bGtbEnable

uint32_t u32InterruptMask

void(* pChannelCallback)(FTU_HandleType *pHandle, uint32_t u32Channel, uint32_t u32Timestamp)

void(* pFaultCallback)(FTU_HandleType *pHandle, uint32_t u32Fault)

void(* pOverflowCallback)(FTU_HandleType *pHandle)

void(* pReloadPointCallback)(FTU_HandleType *pHandle)

Detailed Description

The basic configuration option for the FTU peripheral.

Member Data Documentation

bool FTU_CommonType::bGtbEnable

Global Time Base Enable

FTU_ClkSrcType FTU_CommonType::eClkSrc

Ftu clock source

FTU_DebugModeType FTU_CommonType::eDbgMode

Ftu debug mode

FTU_FilterPrescalerType FTU_CommonType::eFliterPrescaler

Select the prescaler of the FTU filter

FTU_TrigModeType FTU_CommonType::eHwTrigMode

hardware trigger mode

FTU_PrescalerType FTU_CommonType::ePrescaler

Ftu prescaler

FTU_UpDownDisableType FTU_CommonType::eUpDownDisable

Disable channel match trigger/interrupt when count-up/down in CPWM/QUAD mode

void(* FTU_CommonType::pChannelCallback) (FTU_HandleType *pHandle, uint32_t u32Channel, uint32_t u32Timestamp)

channel interrupt callback

void(* FTU_CommonType::pFaultCallback) (FTU_HandleType *pHandle, uint32_t u32Fault)

fault interrupt callback

void(* FTU_CommonType::pOverflowCallback) (FTU_HandleType *pHandle)

overflow interrupt callback

void(* FTU_CommonType::pReloadPointCallback) (FTU_HandleType *pHandle)

reload point interrupt callback

uint32_t FTU_CommonType::u32InterruptMask

interrupt enable mask

uint32_t FTU_CommonType::u32OverflowValue

Ftu modulo value

uint8_t FTU_CommonType::u8ReloadFreq

Frequency of the Reload Opportunities. Range is 0-31,

The documentation for this struct was generated from the following file:

`module_driver_ftu.h`

FTU_CounterModeType Struct Reference

The configuration option for the time counter mode.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint32_t **u32CounterValue**

uint32_t **u32InitialValue**

Detailed Description

The configuration option for the time counter mode.

Member Data Documentation

uint32_t FTU_CounterModeType::u32CounterValue

counter overflow value (timer ticks)

uint32_t FTU_CounterModeType::u32InitialValue

counter initial value (timer ticks)

The documentation for this struct was generated from the following file:

`module_driver_ftu.h`

FTU_EdgeNumberMeasureType Struct Reference

FTU edge number measurement configuration type.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint8_t **u8Channel**

FTU_InputCapturePinModeType **eEdgeMode**

FTU_MeasureContModeType **eContinuouslyMode**

uint32_t **u32StartWindow**

uint32_t **u32EndWindow**

Detailed Description

FTU edge number measurement configuration type.

Member Data Documentation

FTU_MeasureContModeType FTU_EdgeNumberMeasureType::eContinuouslyMode

The measurement is single or continuous

FTU_InputCapturePinModeType FTU_EdgeNumberMeasureType::eEdgeMode

input capture mode

uint32_t FTU_EdgeNumberMeasureType::u32EndWindow

end-point window

uint32_t FTU_EdgeNumberMeasureType::u32StartWindow

start-point window

uint8_t FTU_EdgeNumberMeasureType::u8Channel

selected FTU channel

The documentation for this struct was generated from the following file:

`module_driver_ftu.h`

FTU_ExpectEdgeNumberMeasureType Struct Reference

FTU expect edge number measurement configuration type.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint8_t **u8Channel**

FTU_InputCapturePinModeType **eEdgeMode**

uint8_t **u8ExpectEdgeNumber**

FTU_MeasureContModeType **eContinuouslyMode**

Detailed Description

FTU expect edge number measurement configuration type.

Member Data Documentation

FTU_MeasureContModeType FTU_ExpectEdgeNumberMeasureType::eContinuouslyMode

The measurement is single or continuous

FTU_InputCapturePinModeType FTU_ExpectEdgeNumberMeasureType::eEdgeMode

input capture mode

uint8_t FTU_ExpectEdgeNumberMeasureType::u8Channel

selected FTU channel

uint8_t FTU_ExpectEdgeNumberMeasureType::u8ExpectEdgeNumber

Expected number of edges

The documentation for this struct was generated from the following file:

`module_driver_ftu.h`

FTU_ExpectEdgeNumberResultType Struct Reference

Expect edge number result time.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint32_t **u32FirstEdgeTime**

uint32_t **u32LastEdgeTime**

Detailed Description

Expect edge number result time.

Member Data Documentation

uint32_t FTU_ExpectEdgeNumberResultType::u32FirstEdgeTime

first edge time of measured signal

uint32_t FTU_ExpectEdgeNumberResultType::u32LastEdgeTime

last edge time of measured signal

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_FaultControlType Struct Reference

The configuration option for enabling a FTU fault.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint8_t **u8FaultIndex**

FTU_FaultPolActiveType **eFaultPol**

bool **bFaultFilterEnable**

Detailed Description

The configuration option for enabling a FTU fault.

Member Data Documentation

bool FTU_FaultControlType::bFaultFilterEnable

whether to enable fault input glitch filter

FTU_FaultPolActiveType FTU_FaultControlType::eFaultPol

the fault input active polarity

uint8_t FTU_FaultControlType::u8FaultIndex

fault index

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_FaultInitType Struct Reference

The configuration option for initializing FTU fault.

```
#include <module_driver_ftu.h>
```

Public Attributes

FTU_FaultModeType eFaultMode

uint8_t **u8FilterValue**

uint8_t **u8FaultChannelEnable**

uint8_t **u8FaultDisableDelay0**

uint8_t **u8FaultDisableDelay1**

Detailed Description

The configuration option for initializing FTU fault.

Member Data Documentation

FTU_FaultModeType FTU_FaultInitType::eFaultMode

fault control mode

uint8_t FTU_FaultInitType::u8FaultChannelEnable

fault control for channel

uint8_t FTU_FaultInitType::u8FaultDisableDelay0

Fault Disable Channel Output Delay Value 0 (timer ticks)

uint8_t FTU_FaultInitType::u8FaultDisableDelay1

Fault Disable Channel Output Delay Value 1 (timer ticks)

uint8_t FTU_FaultInitType::u8FilterValue

selects the filter value for the fault inputs

The documentation for this struct was generated from the following file:

`module_driver_ftu.h`

FTU_InputChannelType Struct Reference

The configuration option for a input capture channel.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint8_t **u8Channel**

FTU_InputCapturePinModeType **eInputMode**

uint8_t **u8FilterValue**

Detailed Description

The configuration option for a input capture channel.

Member Data Documentation

FTU_InputCapturePinModeType FTU_InputChannelType::eInputMode

input capture mode

uint8_t FTU_InputChannelType::u8Channel

selected FTU channel

uint8_t FTU_InputChannelType::u8FilterValue

selects the filter value for the channel input

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_OutputCompareModeType Struct Reference

The configuration option for the output compare mode.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint8_t **u8Channel**

FTU_OutputComparePinModeType **eOutputMode**

uint32_t **u32CompareValue**

Detailed Description

The configuration option for the output compare mode.

Member Data Documentation

FTU_OutputComparePinModeType FTU_OutputCompareModeType::eOutputMode

output compare mode

uint32_t FTU_OutputCompareModeType::u32CompareValue

output compare value (timer ticks)

uint8_t FTU_OutputCompareModeType::u8Channel

selected FTU channel

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_PwmChannelType Struct Reference

The configuration for the Pwm channel.

```
#include <module_driver_ftu.h>
```

Public Attributes

```
uint8_t u8Channel
FTU_PwmPinModeType ePinMode
uint32_t u32PwmDuty
uint32_t u32PhaseShift
bool bLinkMode
bool bLinkChannelComplement
bool bDeadtimeEnable
uint32_t u32ChannelDeadtime
```

Detailed Description

The configuration for the Pwm channel.

Member Data Documentation

bool FTU_PwmChannelType::bDeadtimeEnable

Deadtime Enable

bool FTU_PwmChannelType::bLinkChannelComplement

pwm link channel output complement

bool FTU_PwmChannelType::bLinkMode

pwm channel link mode enable, channel num must be even,and the linked channel is current_channel+1

FTU_PwmPinModeType FTU_PwmChannelType::ePinMode

pwm mode

uint32_t FTU_PwmChannelType::u32ChannelDeadtime

The separated deadtime (source clock ticks)

uint32_t FTU_PwmChannelType::u32PhaseShift

pwm phase shift (timer ticks)

uint32_t FTU_PwmChannelType::u32PwmDuty

pwm duty (timer ticks)

uint8_t FTU_PwmChannelType::u8Channel

selected FTU channel

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_PwmDutyUpdateType Struct Reference

The configuration option for update PWM Duty.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint8_t **u8Channel**

uint32_t **u32Duty**

uint32_t **u32PhaseShift**

bool **bUpdate**

Detailed Description

The configuration option for update PWM Duty.

Member Data Documentation

bool FTU_PwmDutyUpdateType::bUpdate

whether to allow the update to take effect(not available for FTU_PWM_UPDATE_IMMEDIATELY)

uint32_t FTU_PwmDutyUpdateType::u32Duty

duty cycle

uint32_t FTU_PwmDutyUpdateType::u32PhaseShift

phase shift of the PWM channel(only available for phase shift channel)

uint8_t FTU_PwmDutyUpdateType::u8Channel

the selected FTU channel

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_PwmModeType Struct Reference

The configuration option for the Pwm Mode.

```
#include <module_driver_ftu.h>
```

Collaboration diagram for FTU_PwmModeType:

IMAGE

Public Attributes

uint32_t **u32PwmPeriod**

FTU_PwmAlignedType **eAlignedMode**

FTU_PwmUpdateModeType **eUpdateMode**

uint32_t **u32ChannelCount**

FTU_PwmChannelType * **pPwmChannels**

Detailed Description

The configuration option for the Pwm Mode.

Member Data Documentation

FTU_PwmAlignedType FTU_PwmModeType::eAlignedMode

pwm Aligned Mode

FTU_PwmUpdateModeType FTU_PwmModeType::eUpdateMode

pwm Update Mode of Period and duty

FTU_PwmChannelType* FTU_PwmModeType::pPwmChannels

point to the pwm channel

uint32_t FTU_PwmModeType::u32ChannelCount

channel count of the Ftu instance

uint32_t FTU_PwmModeType::u32PwmPeriod

pwm period (timer ticks)

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_QuadratureInitType Struct Reference

The configuration option for the quadrature decoder mode.

```
#include <module_driver_ftu.h>
```

Public Attributes

FTU_QuadratureModeType eQuadMode

uint8_t u8PhaFilterVal

uint8_t u8PhbFilterVal

bool bPhaInverted

bool bPhbInverted

uint16_t u16TopValue

uint16_t u16BottomValue

Detailed Description

The configuration option for the quadrature decoder mode.

Member Data Documentation

bool FTU_QuadratureInitType::bPhaInverted

whether to inverted polarity of phase A input

bool FTU_QuadratureInitType::bPhbInverted

whether to inverted polarity of phase B input

FTU_QuadratureModeType FTU_QuadratureInitType::eQuadMode

selects the encoding mode used in the Quadrature Decoder mode

uint16_t FTU_QuadratureInitType::u16BottomValue

the bottom value of counting

uint16_t FTU_QuadratureInitType::u16TopValue

the top value of counting

uint8_t FTU_QuadratureInitType::u8PhaFilterVal

The filter value for the phase A input

uint8_t FTU_QuadratureInitType::u8PhbFilterVal

The filter value for the phase B input

The documentation for this struct was generated from the following file:

module_driver_ftu.h

FTU_SignalMeasureType Struct Reference

FTU signal measurement configuration type.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint8_t **u8Channel**

FTU_SignalMeasureModeType **eMeasureMode**

FTU_MeasureContModeType **eContinuouslyMode**

FTU_MeasureStartModeType **eStartMode**

Detailed Description

FTU signal measurement configuration type.

Member Data Documentation

FTU_MeasureContModeType FTU_SignalMeasureType::eContinuouslyMode

The measurement is single or continuous

FTU_SignalMeasureModeType FTU_SignalMeasureType::eMeasureMode

signal measurement mode type

FTU_MeasureStartModeType FTU_SignalMeasureType::eStartMode

Timing of start measurement

uint8_t FTU_SignalMeasureType::u8Channel

selected FTU channel

The documentation for this struct was generated from the following file:

`module_driver_ftu.h`

FTU_SignalMeasureValueType Struct Reference

FTU signal measurement result time.

```
#include <module_driver_ftu.h>
```

Public Attributes

uint32_t **u32StartTime**

uint32_t **u32EndTime**

Detailed Description

FTU signal measurement result time.

Member Data Documentation

uint32_t FTU_SignalMeasureValueType::u32EndTime

end time of measured signal

uint32_t FTU_SignalMeasureValueType::u32StartTime

start time of measured signal

The documentation for this struct was generated from the following file:

module_driver_ftu.h

HostNVRPermission Struct Reference

Indicate the Host NVR Read/Write/Erase Protection.

```
#include <module_driver_sec.h>
```

Public Attributes

bool **HostNVRReadEn**

bool **HostNVRWriteEn**

bool **HostNVERaseEn**

Detailed Description

Indicate the Host NVR Read/Write/Erase Protection.

Member Data Documentation

bool HostNVRPermission::HostNVERaseEn

true means - Host eraseaccess to NVR region is enabled.

bool HostNVRPermission::HostNVRReadEn

true means - Host read access to NVRregion is enabled.

bool HostNVRPermission::HostNVRWriteEn

true means - Host write access to NVR region is enabled.

The documentation for this struct was generated from the following file:

module_driver_sec.h

HostUKPermission Struct Reference

Indicate the Host User Key Read/Write/Erase Protection.

```
#include <module_driver_sec.h>
```

Public Attributes

bool **HostUKReadEn**

bool **HostUKWriteEn**

bool **HostUKEraseEn**

Detailed Description

Indicate the Host User Key Read/Write/Erase Protection.

Member Data Documentation

bool HostUKPermission::HostUKEraseEn

true means - Host eraseaccess to User Key region is enabled.

bool HostUKPermission::HostUKReadEn

true means - Host read access to User Key region is enabled.

bool HostUKPermission::HostUKWriteEn

true means - Host write access to User Key region is enabled.

The documentation for this struct was generated from the following file:

module_driver_sec.h

HSADC_ChannelCfgType Struct Reference

The configuration option for the HSADC channel.

#include <module_driver_hsadc.h>

Public Attributes

HSADC_ChannelType eChannel

HSADC_SampleTimeOptionType eSampleTimeOption

Detailed Description

The configuration option for the HSADC channel.

Member Data Documentation

HSADC_ChannelType HSADC_ChannelCfgType::eChannel

Selected HSADC channel

HSADC_SampleTimeOptionType HSADC_ChannelCfgType::eSampleTimeOption

The sample time selection for the channel

The documentation for this struct was generated from the following file:

module_driver_hsadc.h

HSADC_CmpType Struct Reference

Defines the hardware compare configuration.

```
#include <module_driver_hsadc.h>
```

Public Attributes

```
bool bCmpEn
HSADC_CmpModeType eCmpMode
HSADC_ChannelType u8CmpChannel
uint16_t u16CmpHighThres
uint16_t u16CmpLowThres
bool bCmpIntEn
void(* pCmpNotify)(struct _HSADC_HandleType *pHandle)
```

Detailed Description

Defines the hardware compare configuration.

This structure is used to configure the hardware compare feature for the HSADC

Implements : **HSADC_CmpType**

Member Data Documentation

bool HSADC_CmpType::bCmpEn

Enable hardware compare

bool HSADC_CmpType::bCmpIntEn

Enable interrupt when conversion result lays in the compare threshold

HSADC_CmpModeType HSADC_CmpType::eCmpMode

Compare channel mode

void(* HSADC_CmpType::pCmpNotify) (struct _HSADC_HandleType *pHandle)

Compare interrupt Notify

uint16_t HSADC_CmpType::u16CmpHighThres

Compare high threshold

uint16_t HSADC_CmpType::u16CmpLowThres

Compare low threshold

HSADC_ChannelType HSADC_CmpType::u8CmpChannel

Compare channel selection

The documentation for this struct was generated from the following file:

module_driver_hsadc.h

HSADC_FastCmpType Struct Reference

Defines the hardware fast compare configuration.

#include <module_driver_hsadc.h>

Public Attributes

bool **bFastCmpEn**
bool **bFastCmpPreSetResult**
bool **bFastCmpDigitalModeEn**
bool **bFastCmpSaveConvData**
uint16_t **u16FastCmpRefValue**
uint16_t **u16FastCmpUpperDelta**
uint16_t **u16FastCmpLowerDelta**
HSADC_FastCmpRefMode **eFastCmpRefMode**
HSADC_FastCmpRampDir **eFastCmpRampDir**
HSADC_FastCmpRampTrgMode **eFastCmpRampTrgMode**
HSADC_FastCmpRampTrgPolarity **eFastCmpRampTrgPol**
uint8_t **u8FastCmpRampStep**
uint16_t **u16FastCmpRampRefA**
uint16_t **u16FastCmpRampRefB**
bool **bFastCmpBFLen**
HSADC_FcmpBFLAction **eFastCmpBFLAct**
bool **bFastCmpBFLInvert**

Detailed Description

Defines the hardware fast compare configuration.

This structure is used to configure the hardware fast compare feature for the HSADC

Implements : HSADC_FastCompareType

Member Data Documentation

bool HSADC_FastCmpType::bFastCmpBFLEn

bool HSADC_FastCmpType::bFastCmpBFLInvert

bool HSADC_FastCmpType::bFastCmpDigitalModeEn

Enable digital comparator

bool HSADC_FastCmpType::bFastCmpEn

Enable fast compare

bool HSADC_FastCmpType::bFastCmpPreSetResult

bool HSADC_FastCmpType::bFastCmpSaveConvData

Enable conversion data saving. This option is valid only when fast compare work in digital mode.

HSADC_FcmpBFLAction HSADC_FastCmpType::eFastCmpBFLAct

HSADC_FastCmpRampDir HSADC_FastCmpType::eFastCmpRampDir

HSADC_FastCmpRampTrgMode HSADC_FastCmpType::eFastCmpRampTrgMode

HSADC_FastCmpRampTrgPolarity HSADC_FastCmpType::eFastCmpRampTrgPol

HSADC_FastCmpRefMode HSADC_FastCmpType::eFastCmpRefMode

Fast compare operation mode

uint16_t HSADC_FastCmpType::u16FastCmpLowerDelta

uint16_t HSADC_FastCmpType::u16FastCmpRampRefA

uint16_t HSADC_FastCmpType::u16FastCmpRampRefB

uint16_t HSADC_FastCmpType::u16FastCmpRefValue

Fast compare reference value

uint16_t HSADC_FastCmpType::u16FastCmpUpperDelta

uint8_t HSADC_FastCmpType::u8FastCmpRampStep

The documentation for this struct was generated from the following file:

module_driver_hsadc.h

HSADC_InitType Struct Reference

Defines the converter configuration.

```
#include <module_driver_hsadc.h>
```

Public Attributes

```
HSADC_AlignType eAlign
HSADC_TriggerModeType eTrgMode
uint16_t u16PeriodTrgInterval
bool bWaitEn
HSADC_TriggerLatchUnitPri eTrgLatchUnitPri
HSADC_SequenceModeType eSeqMode
bool bAutoDis
HSADC_OvrModeType eOverRunMode
bool bAverageEn
HSADC_AverageLenType eAverageLen
uint8_t aSampleTimes [HSADC_SAMPLE_TIME_OPTION_COUNT]
HSADC_FunctionClockType eFunctionClockSel
uint32_t u32ExtClkFreq
HSADC_ClockDivideType eClockDivider
bool bSampleSpareEn
bool bCalibrationEn
bool bExecuteCalibration
bool bDmaEn
bool bSeqGroupDmaEn
uint8_t u8SeqGroupDmaSel
bool bSeqCompleteIntEn
bool bConvCompleteIntEn
bool bOverRunIntEn
bool bEndOfSeqGroupIntEn
void(* pSeqCompleteNotify)(struct _HSADC_HandleType *pHandle)
void(* pConvCompleteNotify)(struct _HSADC_HandleType *pHandle)
void(* pOverRunNotify)(struct _HSADC_HandleType *pHandle)
void(* pEndOfSeqGroupNotify)(struct _HSADC_HandleType *pHandle)
uint32_t * pResultBuffer
uint32_t * pSeqGroupResultBuffer [HSADC_SEQ_GROUP_COUNT]
```


Detailed Description

Defines the converter configuration.

This structure is used to configure the HSADC converter

Implements : **HSADC_InitType**

Member Data Documentation

uint8_t HSADC_InitType::aSampleTimes[HSADC_SAMPLE_TIME_OPTION_COUNT]

HSADC sample time options, range: 6 ~ 255 Total cycles = aSampleTimes[i] HSADC clock cycles if aSampleTimes[i] < 128, the minimum value is 6 Total cycles = (128 + (aSampleTimes[i] - 128)*16) HSADC clock cycles if aSampleTimes[i] >= 128

bool HSADC_InitType::bAutoDis

Whether to enable auto disable mode, only set this when adc in off state

bool HSADC_InitType::bAverageEn

Whether to enable averaging functionality

bool HSADC_InitType::bCalibrationEn

Whether to enable calibration compensation

bool HSADC_InitType::bConvCompleteIntEn

Enable interrupt when conversion completed

bool HSADC_InitType::bDmaEn

Enable DMA for the HSADC

bool HSADC_InitType::bEndOfSeqGroupIntEn

Enable interrupt when sequence group conversion completed

bool HSADC_InitType::bExecuteCalibration

Whether to execute calibration at initialization

bool HSADC_InitType::bOverRunIntEn

Enable interrupt when overrun occurred

bool HSADC_InitType::bSampleSpareEn

Spare time at the end of sampling enable, if HSADC_FCLK > 100MHZ, this must be Enabled

bool HSADC_InitType::bSeqCompleteIntEn

Enable interrupt when sequence completed

bool HSADC_InitType::bSeqGroupDmaEn

Enable sequence group DMA, only valid when eSeqMode == HSADC_SEQ_GROUP_MODE

bool HSADC_InitType::bWaitEn

Whether to enable HSADC wait conversion mode

HSADC_AlignType HSADC_InitType::eAlign

HSADC alignment (left, right)

HSADC_AverageLenType HSADC_InitType::eAverageLen

Number of samples used for averaging

HSADC_ClockDivideType HSADC_InitType::eClockDivider

HSADC clock divider

HSADC_FunctionClockType HSADC_InitType::eFunctionClockSel

Function clock selection

HSADC_OvrModeType HSADC_InitType::eOverRunMode

Whether to preserve data when HSADC overruns

HSADC_SeqModeType HSADC_InitType::eSeqMode

HSADC sequence mode

HSADC_TrgLatchUnitPri HSADC_InitType::eTrgLatchUnitPri

Select priority of Trigger Latch Unit

HSADC_TrgModeType HSADC_InitType::eTrgMode

HSADC trigger type

void(* HSADC_InitType::pConvCompleteNotify) (struct _HSADC_HandleType *pHandle)

Conversion complete interrupt Notify

void(* HSADC_InitType::pEndOfSeqGroupNotify) (struct _HSADC_HandleType *pHandle)

Sequence group complete interrupt Notify

void(* HSADC_InitType::pOverRunNotify) (struct _HSADC_HandleType *pHandle)

Overrun interrupt Notify

uint32_t* HSADC_InitType::pResultBuffer

Result buffer for eSeqMode != HSADC_SEQ_GROUP_MODE

void(* HSADC_InitType::pSeqCompleteNotify) (struct _HSADC_HandleType *pHandle)

Sequence complete interrupt Notify

uint32_t* HSADC_InitType::pSeqGroupResultBuffer[HSADC_SEQ_GROUP_COUNT]

Result buffer for eSeqMode == HSADC_SEQ_GROUP_MODE

uint16_t HSADC_InitType::u16PeriodTrgInterval

The interval of periodic trigger

uint32_t HSADC_InitType::u32ExtClkFreq

External clock Frequency if FCLK_FROM_PAD

uint8_t HSADC_InitType::u8SeqGroupDmaSel

Sequence group DMA selection

The documentation for this struct was generated from the following file:

module_driver_hsadc.h

HSADC_SeqGroupType Struct Reference

The configuration option for the HSADC sequence group.

#include <module_driver_hsadc.h>

Public Attributes

uint8_t **u8Start**

uint8_t **u8Len**

Detailed Description

The configuration option for the HSADC sequence group.

Implements : **HSADC_SeqGroupType**

Member Data Documentation

uint8_t HSADC_SeqGroupType::u8Len

Sequence group length, is must be ≥ 0

uint8_t HSADC_SeqGroupType::u8Start

Sequence channel start

The documentation for this struct was generated from the following file:

module_driver_hsadc.h

HSM_AesEnDecryptType Struct Reference

AES encrypt/decrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_AesEnDecryptType:

IMAGE

Public Attributes

```
HSM_AesKeyType eUseKeyType
HSM_AesNewKeyInfType tNewKeyInf
HSM_AesVendorKeyInfType tVendorKeyInf
HSM_SymmEnDecryptAlgType eAesAlg
const hsm_uint32_t * pDataInput
hsm_uint32_t u32InputByteCnt
hsm_uint32_t * pDataOutput
hsm_uint32_t u32OutputMemSize
const hsm_uint32_t * plvData
```

Detailed Description

AES encrypt/decrypt information used by driver when call specific API.

Member Data Documentation

HSM_SymmEnDecryptAlgType HSM_AesEnDecryptType::eAesAlg

encrypt/decrypt mode

HSM_AesKeyType HSM_AesEnDecryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

const hsm_uint32_t* HSM_AesEnDecryptType::pDataInput

address should align with 4bytes

hsm_uint32_t* HSM_AesEnDecryptType::pDataOutput

address should align with 4bytes

const hsm_uint32_t* HSM_AesEnDecryptType::pIvData

address should align with 4bytes, 128bit(16Bytes) iv(initialization vector) data array, ECB not need configure this

HSM_AesNewKeyInfType HSM_AesEnDecryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_AesEnDecryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_AesEnDecryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_AesEnDecryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_AESM_CFG_KEY_CFG_T Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t key_type
```

```
hsm_uint32_t key_sel
```

```
void * key_addr
```

Member Data Documentation

```
void* HSM_AESM_CFG_KEY_CFG_T::key_addr
```

```
hsm_uint32_t HSM_AESM_CFG_KEY_CFG_T::key_sel
```

```
hsm_uint32_t HSM_AESM_CFG_KEY_CFG_T::key_type
```

The documentation for this struct was generated from the following file:

```
module_driver_hsm.h
```

HSM_AESM_ENTRY_CFG_T Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSM_AESM_ENTRY_CFG_T:

IMAGE

Public Attributes

HSM_AESM_CFG_KEY_CFG_T * key_cfg

hsm_uint32_t key_cfg_en

hsm_uint32_t aes_mode

hsm_uint32_t aes_as

hsm_uint32_t enc_dec

hsm_uint32_t eng_sel

hsm_uint32_t mst_en

hsm_uint32_t icv_en

hsm_uint32_t data_len

hsm_uint32_t icv_len

hsm_uint32_t aad_len

const void * in_addr

void * out_addr

const void * iv_addr

hsm_uint32_t dbg_wait

hsm_uint32_t iv_len

Member Data Documentation

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::aad_len`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::aes_as`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::aes_mode`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::data_len`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::dbg_wait`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::enc_dec`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::eng_sel`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::icv_en`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::icv_len`

`const void* HSM_AESM_ENTRY_CFG_T::in_addr`

`const void* HSM_AESM_ENTRY_CFG_T::iv_addr`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::iv_len`

`HSM_AESM_CFG_KEY_CFG_T* HSM_AESM_ENTRY_CFG_T::key_cfg`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::key_cfg_en`

`hsm_uint32_t HSM_AESM_ENTRY_CFG_T::mst_en`

`void* HSM_AESM_ENTRY_CFG_T::out_addr`

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_AesmRawApiType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSM_AesmRawApiType:

IMAGE

Public Attributes

HSM_AESM_ENTRY_CFG_T tCfg

HSM_AESM_CFG_KEY_CFG_T tKeyCfg

hsm_uint32_t(* pCtx1)[16]

Member Data Documentation

hsm_uint32_t(* HSM_AesmRawApiType::pCtx1)[16]

HSM_AESM_ENTRY_CFG_T HSM_AesmRawApiType::tCfg

HSM_AESM_CFG_KEY_CFG_T HSM_AesmRawApiType::tKeyCfg

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_AesNewKeyInfType Struct Reference

key information used by AES when use one key in software

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_AesKeyBitCntType eKeyType

const void * **pKeyAddr**

Detailed Description

key information used by AES when use one key in software

Member Data Documentation

HSM_AesKeyBitCntType HSM_AesNewKeyInfType::eKeyType

the key bit count, which determine the AES algorithm type

const void* HSM_AesNewKeyInfType::pKeyAddr

the key data pointer, the address should 4bytes align, the buffer array should declared by hsm_uint32_t

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_AesVendorKeyInfType Struct Reference

key information used by AES when use one key in nvr flash

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_AesKeyBitCntType eKeyType

Detailed Description

key information used by AES when use one key in nvr flash

Member Data Documentation

HSM_AesKeyBitCntType HSM_AesVendorKeyInfType::eKeyType

the key bit count, which determine the AES algorithm type

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_BigNumberCalcExtType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
const hsm_uint8_t * pA
hsm_uint32_t u32AByteCount
const hsm_uint8_t * pE
hsm_uint32_t u32EByteCount
const hsm_uint8_t * pN
hsm_uint32_t u32NByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t u32CalcType
const hsm_uint8_t * pB
hsm_uint32_t u32BByteCount
hsm_uint32_t u32DataValidFlag
HSM_DataFormatType eInputDataFmt
HSM_DataFormatType eKeyEFmt
HSM_DataFormatType eKeyNFmt
HSM_DataFormatType eOutputFmt
hsm_uint32_t * pResultByteCount
```

Member Data Documentation

HSM_DataFormatType HSM_BigNumberCalcExtType::eInputDataFmt

HSM_DataFormatType HSM_BigNumberCalcExtType::eKeyEFmt

HSM_DataFormatType HSM_BigNumberCalcExtType::eKeyNFmt

HSM_DataFormatType HSM_BigNumberCalcExtType::eOutputFmt

const hsm_uint8_t* HSM_BigNumberCalcExtType::pA

address is 4bytes aligned, input data, if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_BigNumberCalcExtType::pB

const hsm_uint8_t* HSM_BigNumberCalcExtType::pE

address is 4bytes aligned, input key E data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_BigNumberCalcExtType::pN

address is 4bytes aligned, input key N data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

hsm_uint8_t* HSM_BigNumberCalcExtType::pResult

address is 4bytes aligned, output buffer, the buffer size should $\geq$ "u32Key_N_ByteCount"

hsm_uint32_t* HSM_BigNumberCalcExtType::pResultByteCount

hsm_uint32_t HSM_BigNumberCalcExtType::u32AByteCount

the byte count of input data

hsm_uint32_t HSM_BigNumberCalcExtType::u32BByteCount

hsm_uint32_t HSM_BigNumberCalcExtType::u32CalcType

hsm_uint32_t HSM_BigNumberCalcExtType::u32DataValidFlag

hsm_uint32_t HSM_BigNumberCalcExtType::u32EByteCount

the byte count of key E

hsm_uint32_t HSM_BigNumberCalcExtType::u32NByteCount

hw will get the actual key data bit count according to the non-zero bit count, it means key not generate by multiply 2

hsm_uint32_t HSM_BigNumberCalcExtType::u32ResultBufByteCnt

the result buffer size

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_BigNumberCalcType Struct Reference

RSA information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
const hsm_uint8_t * pA
hsm_uint32_t u32AByteCount
const hsm_uint8_t * pE
hsm_uint32_t u32EByteCount
const hsm_uint8_t * pN
hsm_uint32_t u32NByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t u32CalcType
const hsm_uint8_t * pB
hsm_uint32_t u32BByteCount
```

Detailed Description

RSA information used by driver when call specific API.

Member Data Documentation

const hsm_uint8_t* HSM_BigNumberCalcType::pA

address is 4bytes aligned, input data, if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_BigNumberCalcType::pB

const hsm_uint8_t* HSM_BigNumberCalcType::pE

address is 4bytes aligned, input key E data for "A = (input data)^E mod N", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_BigNumberCalcType::pN

address is 4bytes aligned, input key N data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

hsm_uint8_t* HSM_BigNumberCalcType::pResult

address is 4bytes aligned, output buffer, the buffer size should $\geq$ "u32Key_N_ByteCount"

hsm_uint32_t HSM_BigNumberCalcType::u32AByteCount

the byte count of input data

hsm_uint32_t HSM_BigNumberCalcType::u32BByteCount

hsm_uint32_t HSM_BigNumberCalcType::u32CalcType

hsm_uint32_t HSM_BigNumberCalcType::u32EByteCount

the byte count of key E

hsm_uint32_t HSM_BigNumberCalcType::u32NByteCount

hw will get the actual key data bit count according to the non-zero bit count, it means key not generate by multiply 2

hsm_uint32_t HSM_BigNumberCalcType::u32ResultBufByteCnt

the result buffer size

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_CcmDecryptType Struct Reference

CCM Decrypt information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSM_CcmDecryptType:

IMAGE

Public Attributes

HSM_AesKeyType eUseKeyType
HSM_AesNewKeyInfType tNewKeyInf
HSM_AesVendorKeyInfType tVendorKeyInf
hsm_uint32_t * pAllData
hsm_uint32_t u32AadByteCnt
hsm_uint32_t u32InputByteCnt
HSM_BoolType bCheckMacEn
hsm_uint32_t u32MacByteCnt
hsm_uint32_t * pDataOutput
hsm_uint32_t u32OutputMemSize
const **hsm_uint32_t** * plvData

Detailed Description

CCM Decrypt information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_CcmDecryptType::bCheckMacEn

check the MAC data in input or not

HSM_AesKeyType HSM_CcmDecryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

hsm_uint32_t* HSM_CcmDecryptType::pAllData

address should align with 4bytes, place AAD data(if exist, 16bytes align) first, then input data, then MAC data(if exist)

hsm_uint32_t* HSM_CcmDecryptType::pDataOutput

address should align with 4bytes, point to the buffer that store the result

const hsm_uint32_t* HSM_CcmDecryptType::pIvData

address should align with 4bytes, all data 32bytes, iv data, consist of 16bytes B0 data, and 16bytes CTR, MUST configure

HSM_AesNewKeyInfType HSM_CcmDecryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_CcmDecryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_CcmDecryptType::u32AadByteCnt

128bit(16Bytes) align, if not, only the aligned data treat as AAD, the left AAD bytes will be treated as input data

hsm_uint32_t HSM_CcmDecryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_CcmDecryptType::u32MacByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_CcmDecryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_CcmEncryptType Struct Reference

CCM Encrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_CcmEncryptType:

IMAGE

Public Attributes

```
HSM_AesKeyType eUseKeyType
HSM_AesNewKeyInfType tNewKeyInf
HSM_AesVendorKeyInfType tVendorKeyInf
hsm_uint32_t * pAllData
hsm_uint32_t u32AadByteCnt
hsm_uint32_t u32InputByteCnt
hsm_uint32_t u32GenMacByteCnt
hsm_uint32_t * pDataOutput
hsm_uint32_t u32OutputMemSize
const hsm_uint32_t * plvData
hsm_uint32_t(* pMacOut )[4]
hsm_uint32_t(* pEmacOut )[4]
```

Detailed Description

CCM Encrypt information used by driver when call specific API.

Member Data Documentation

HSM_AesKeyType HSM_CcmEncryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

hsm_uint32_t* HSM_CcmEncryptType::pAllData

address should align with 4bytes, place AAD data(16bytes align) first, then input data

hsm_uint32_t* HSM_CcmEncryptType::pDataOutput

address should align with 4bytes

hsm_uint32_t(* HSM_CcmEncryptType::pEmacOut)[4]

address should align with 4bytes, HSM generated encrypted MAC data

const hsm_uint32_t* HSM_CcmEncryptType::plvData

address should align with 4bytes, all data 32bytes, iv data, consist of 16bytes B0 data, and 16bytes CTR, MUST configure

hsm_uint32_t(* HSM_CcmEncryptType::pMacOut)[4]

address should align with 4bytes, HSM generated MAC data

HSM_AesNewKeyInfType HSM_CcmEncryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_CcmEncryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_CcmEncryptType::u32AadByteCnt

the byte count contain the 2bytes in the head, it's the valid data size, not after aligned, 128bit(16Bytes) align, if not, only the aligned data treat as AAD, the left AAD bytes will be treated as input data

hsm_uint32_t HSM_CcmEncryptType::u32GenMacByteCnt

output mac size

hsm_uint32_t HSM_CcmEncryptType::u32InputByteCnt

128bit(16Bytes) align, ONLY represent data

hsm_uint32_t HSM_CcmEncryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_CmdType Struct Reference

HSM cmd type definition.

```
#include <module_driver_hsm.h>
```

Public Attributes

hsm_uint32_t u32Cmd

hsm_uint32_t u32Addr

Detailed Description

HSM cmd type definition.

Member Data Documentation

hsm_uint32_t HSM_CmdType::u32Addr

current parameter address to HSM core via mailbox

hsm_uint32_t HSM_CmdType::u32Cmd

current command id to HSM core via mailbox

The documentation for this struct was generated from the following file:

module_driver_hsm.h

Hsm_configuration Struct Reference

Indicate the HSM configuration.

```
#include <module_driver_sec.h>
```

Public Attributes

bool **SrkEncryptEn**

bool **FwkEncryptEn**

uint8_t **KeyMinTimeout**

uint8_t **UserKeySize**

uint8_t **FirmwareSize**

Detailed Description

Indicate the HSM configuration.

Member Data Documentation

uint8_t Hsm_configuration::FirmwareSize

Get the firmware size

bool Hsm_configuration::FwkEncryptEn

true means enable the SRK Encrypt.

uint8_t Hsm_configuration::KeyMinTimeout

The minimum timeout in seconds when the bus clock is 150MHz; if the bus clock is 75MHz, 1bit means 2 seconds.

bool Hsm_configuration::SrkJEncryptEn

true means enable the SRK Encrypt.

uint8_t Hsm_configuration::UserKeySize

Get the user key size

The documentation for this struct was generated from the following file:

`module_driver_sec.h`

HSM_DrvAesDecryptInfType Struct Reference

Aes decrypt information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvAesDecryptInfType:

IMAGE

Public Attributes

HSMCom_FlexAesDecryptType * pParm

hsm_uint32_t u32HsmRetResultSize

hsm_uint32_t aParmBuf [((sizeof(HSMCom_FlexAesDecryptType)+3)/4)+1]

hsm_uint32_t alvData [4+1]

Detailed Description

Aes decrypt information type.

Member Data Documentation

hsm_uint32_t HSM_DrvAesDecryptInfType::alvData[4+1]

hsm_uint32_t HSM_DrvAesDecryptInfType::aParmBuf [((sizeof(HSMCom_FlexAesDecryptType)+3)/4)+1]

HSMCom_FlexAesDecryptType* HSM_DrvAesDecryptInfType::pParm

hsm_uint32_t HSM_DrvAesDecryptInfType::u32HsmRetResultSize

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_DrvAesEncryptInfType Struct Reference

Aes encrypt information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvAesEncryptInfType:

IMAGE

Public Attributes

```
HSMCom_FlexAesEncryptType * pParm  
hsm_uint32_t aParmBuf [((sizeof(HSMCom_FlexAesEncryptType)+3)/4)+1]  
HSM_BoolType bSwapOut  
hsm_uint8_t * pResultBuf  
hsm_uint32_t u32ResultByteCnt  
hsm_uint32_t u32HsmRetResultSize  
hsm_uint32_t aResultCacheBuf [132+1]  
hsm_uint32_t aInputCacheBuf [128+1]  
hsm_uint32_t alvData [4+1]
```

Detailed Description

Aes encrypt information type.

Member Data Documentation

hsm_uint32_t HSM_DrvAesEncryptInfType::aInputCacheBuf[128+1]

hsm_uint32_t HSM_DrvAesEncryptInfType::alvData[4+1]

hsm_uint32_t HSM_DrvAesEncryptInfType::aParmBuf[(((sizeof(HSMCom_FlexAesEncryptType)+3)/4)+1)]

hsm_uint32_t HSM_DrvAesEncryptInfType::aResultCacheBuf[132+1]

pkcs7 result size has more 16bytes than input

HSM_BoolType HSM_DrvAesEncryptInfType::bSwapOut

HSMCom_FlexAesEncryptType* HSM_DrvAesEncryptInfType::pParm

hsm_uint8_t* HSM_DrvAesEncryptInfType::pResultBuf

hsm_uint32_t HSM_DrvAesEncryptInfType::u32HsmRetResultSize

hsm_uint32_t HSM_DrvAesEncryptInfType::u32ResultByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvBnCalcExtInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvBnCalcExtInfType:

IMAGE

Public Attributes

HSMCom_BigNumberCalcExtType * pBnCalcParm

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pResultByteCnt

HSM_DataFormatType eOutFmt

hsm_uint32_t aResultCache [128+1]

hsm_uint32_t aB [128+1]

hsm_uint32_t aBnCalcParmBuf [(((sizeof(HSMCom_BigNumberCalcExtType)+3)/4)+1]

hsm_uint32_t u32ResultByteCnt

Member Data Documentation

hsm_uint32_t HSM_DrvBnCalcExtInfType::aB[128+1]

hsm_uint32_t

HSM_DrvBnCalcExtInfType::aBnCalcParmBuf[((sizeof(HSMCom_BigNumberCalcExtType)+3)/4)+1]

hsm_uint32_t HSM_DrvBnCalcExtInfType::aResultCache[128+1]

HSM_DataFormatType HSM_DrvBnCalcExtInfType::eOutFmt

HSMCom_BigNumberCalcExtType* HSM_DrvBnCalcExtInfType::pBnCalcParm

hsm_uint8_t* HSM_DrvBnCalcExtInfType::pResultBuf

hsm_uint32_t* HSM_DrvBnCalcExtInfType::pResultByteCnt

hsm_uint32_t HSM_DrvBnCalcExtInfType::u32ResultBufSize

hsm_uint32_t HSM_DrvBnCalcExtInfType::u32ResultByteCnt

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_DrvBnCalcInfType Struct Reference

Big number calculator information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvBnCalcInfType:

IMAGE

Public Attributes

HSMCom_BigNumberCalcType * pBnCalcParm

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pResultByteCnt

HSM_DataFormatType eOutFmt

hsm_uint32_t aAAndResult [128+1]

hsm_uint32_t aB [128+1]

hsm_uint32_t aE [128+1]

hsm_uint32_t aN [128+1]

hsm_uint32_t aBnCalcParmBuf [(((sizeof(HSMCom_BigNumberCalcType)+3)/4)+1)]

Detailed Description

Big number calculator information type.

Member Data Documentation

hsm_uint32_t HSM_DrvBnCalcInfType::aAAndResult[128+1]

4096bit for max pkam big number calculate, 1 for address align

hsm_uint32_t HSM_DrvBnCalcInfType::aB[128+1]

hsm_uint32_t HSM_DrvBnCalcInfType::aBnCalcParmBuf[((sizeof(HSMCom_BigNumberCalcType)+3)/4)+1]

hsm_uint32_t HSM_DrvBnCalcInfType::aE[128+1]

4096bit for max pkam big number calculate, 1 for address align

hsm_uint32_t HSM_DrvBnCalcInfType::aN[128+1]

4096bit for max pkam big number calculate, 1 for address align

HSM_DataFormatType HSM_DrvBnCalcInfType::eOutFmt

HSMCom_BigNumberCalcType* HSM_DrvBnCalcInfType::pBnCalcParm

hsm_uint8_t* HSM_DrvBnCalcInfType::pResultBuf

hsm_uint32_t* HSM_DrvBnCalcInfType::pResultByteCnt

hsm_uint32_t HSM_DrvBnCalcInfType::u32ResultBufSize

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvCMacInfType Struct Reference

Cmac information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvCMacInfType:

IMAGE

Public Attributes

HSMCom_CMacExType * pParm

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultBufByteCnt

hsm_uint32_t * pOutResultSize

HSM_BoolType bCheckMac

const hsm_uint8_t * pMacToCheck

hsm_uint32_t u32MacToCheckByteCnt

Crypto_VerifyResultType * pVerifyResult

HSM_BoolType bSwapOut

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_CMacExType)+3)/4)+1]

hsm_uint32_t aResultCacheBuf [8+1]

hsm_uint32_t aInputCacheBuf [128+1]

Detailed Description

Cmac information type.

Member Data Documentation

hsm_uint32_t HSM_DrvCMacInfType::aInputCacheBuf[128+1]

hsm_uint32_t HSM_DrvCMacInfType::aParmBuf[((sizeof(HSMCom_CMacExType)+3)/4)+1]

hsm_uint32_t HSM_DrvCMacInfType::aResultCacheBuf[8+1]

max aes/sm4 key length is 256bit, so use 32bytes, cache buffer for result

HSM_BoolType HSM_DrvCMacInfType::bCheckMac

HSM_BoolType HSM_DrvCMacInfType::bSwapOut

const hsm_uint8_t* HSM_DrvCMacInfType::pMacToCheck

hsm_uint32_t* HSM_DrvCMacInfType::pOutResultSize

HSMCom_CMacExType* HSM_DrvCMacInfType::pParm

hsm_uint8_t* HSM_DrvCMacInfType::pResultBuf

Crypto_VerifyResultType* HSM_DrvCMacInfType::pVerifyResult

hsm_uint32_t HSM_DrvCMacInfType::u32MacToCheckByteCnt

hsm_uint32_t HSM_DrvCMacInfType::u32ResultBufByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvEccCalcInfType Struct Reference

ECC calculate info type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvEccCalcInfType:

IMAGE

Public Attributes

hsm_uint32_t u32NBitCnt
HSMCom_EccCalcType * pParm
HSM_DataFormatType eOutFmt
hsm_uint8_t * pResultBufX
hsm_uint32_t u32ResultBufSizeX
hsm_uint32_t * pResultByteCntX
hsm_uint8_t * pResultBufY
hsm_uint32_t u32ResultBufSizeY
hsm_uint32_t * pResultByteCntY
hsm_uint32_t aA [32+1]
hsm_uint32_t aB [32+1]
hsm_uint32_t aP [32+1]
hsm_uint32_t aN [32+1]
hsm_uint32_t aP1x [32+1]
hsm_uint32_t aP1y [32+1]
hsm_uint32_t aE [32+1]
hsm_uint32_t aP2x [32+1]
hsm_uint32_t aP2y [32+1]
hsm_uint32_t aResultX [32+1]
hsm_uint32_t aResultY [32+1]
hsm_uint32_t aParmBuf [(((sizeof(**HSMCom_EccCalcType**)+3)/4)+1)]

Detailed Description

ECC calculate info type.

Member Data Documentation

hsm_uint32_t HSM_DrvEccCalcInfType::aA[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aB[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aE[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aN[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aP[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aP1x[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aP1y[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aP2x[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aP2y[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aParmBuf[((sizeof(HSMCom_EccCalcType)+3)/4)+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aResultX[32+1]

hsm_uint32_t HSM_DrvEccCalcInfType::aResultY[32+1]

HSM_DataFormatType HSM_DrvEccCalcInfType::eOutFmt

HSMCom_EccCalcType* HSM_DrvEccCalcInfType::pParm

hsm_uint8_t* HSM_DrvEccCalcInfType::pResultBufX

hsm_uint8_t* HSM_DrvEccCalcInfType::pResultBufY

hsm_uint32_t* HSM_DrvEccCalcInfType::pResultByteCntX

hsm_uint32_t* HSM_DrvEccCalcInfType::pResultByteCntY

hsm_uint32_t HSM_DrvEccCalcInfType::u32NBitCnt

hsm_uint32_t HSM_DrvEccCalcInfType::u32ResultBufSizeX

hsm_uint32_t HSM_DrvEccCalcInfType::u32ResultBufSizeY

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_DrvEccCurveParamType Struct Reference

ECC curve information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
const hsm_uint32_t * pP  
const hsm_uint32_t * pN  
const hsm_uint32_t * pA  
const hsm_uint32_t * pB  
const hsm_uint32_t * pGx  
const hsm_uint32_t * pGy
```

Detailed Description

ECC curve information used by driver when call specific API.

Member Data Documentation

const hsm_uint32_t* HSM_DrvEccCurveParamType::pA

address is 4bytes aligned, the constant "a" in $y^2 = x^3 + a*x + b \pmod{p}$

const hsm_uint32_t* HSM_DrvEccCurveParamType::pB

address is 4bytes aligned, the constant "b" in $y^2 = x^3 + a*x + b \pmod{p}$

const hsm_uint32_t* HSM_DrvEccCurveParamType::pGx

address is 4bytes aligned, x of the curve generator point G {x, y}

const hsm_uint32_t* HSM_DrvEccCurveParamType::pGy

address is 4bytes aligned, y of the curve generator point G {x, y}

const hsm_uint32_t* HSM_DrvEccCurveParamType::pN

address is 4bytes aligned, ecc curve order; size; the count of all possible EC points

const hsm_uint32_t* HSM_DrvEccCurveParamType::pP

address is 4bytes aligned, ecc curve modulus

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_DrvEccDecryptInfType Struct Reference

Ecc decrypt information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvEccDecryptInfType:

IMAGE

Public Attributes

```
hsm_uint32_t u32NBitCnt
HSMCom_EccEasyDecryType * pParm
HSM_DataFormatType eOutFmt
hsm_uint8_t * pResultBuf
hsm_uint32_t u32ResultBufSize
hsm_uint32_t * pResultByteCnt
hsm_uint32_t aP [32+1]
hsm_uint32_t aN [32+1]
hsm_uint32_t aA [32+1]
hsm_uint32_t aB [32+1]
hsm_uint32_t aPrivate [32+1]
hsm_uint32_t aPlain [32+1]
hsm_uint32_t akGx [32+1]
hsm_uint32_t akGy [32+1]
hsm_uint32_t aEncrypted [32+1]
hsm_uint32_t aParmBuf [(((sizeof(HSMCom_EccEasyDecryType)+3)/4)+1)]
```

Detailed Description

Ecc decrypt information type.

Member Data Documentation

hsm_uint32_t HSM_DrvEccDecryptInfType::aA[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::aB[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::aEncrypted[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::akGx[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::akGy[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::aN[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::aP[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::aParmBuf[((sizeof(HSMCom_EccEasyDecryType)+3)/4)+1]

hsm_uint32_t HSM_DrvEccDecryptInfType::aPlain[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccDecryptInfType::aPrivate[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

HSM_DataFormatType HSM_DrvEccDecryptInfType::eOutFmt

HSMCom_EccEasyDecryType* HSM_DrvEccDecryptInfType::pParm

hsm_uint8_t* HSM_DrvEccDecryptInfType::pResultBuf

hsm_uint32_t* HSM_DrvEccDecryptInfType::pResultByteCnt

hsm_uint32_t HSM_DrvEccDecryptInfType::u32NBitCnt

hsm_uint32_t HSM_DrvEccDecryptInfType::u32ResultBufSize

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvEccSignInfType Struct Reference

Ecc sign information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvEccSignInfType:

IMAGE

Public Attributes

hsm_uint32_t u32NBitCnt

HSM_DataFormatType eOutFmt

hsm_uint8_t * pR

hsm_uint32_t u32RBufByteCnt

hsm_uint8_t * pS

hsm_uint32_t u32SBufByteCnt

HSMCom_EccSignType * pParm

hsm_uint32_t aP [32+1]

hsm_uint32_t aN [32+1]

hsm_uint32_t aA [32+1]

hsm_uint32_t aB [32+1]

hsm_uint32_t aGx [32+1]

hsm_uint32_t aGy [32+1]

hsm_uint32_t aPrivate [32+1]

hsm_uint32_t aR [32+1]

hsm_uint32_t aS [32+1]

hsm_uint32_t aHash [32+1]

hsm_uint32_t aParmBuf [(((sizeof(**HSMCom_EccSignType**)+3)/4)+1]

Detailed Description

Ecc sign information type.

Member Data Documentation

hsm_uint32_t HSM_DrvEccSignInfType::aA[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aB[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aGx[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aGy[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aHash[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aN[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aP[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aParmBuf[((sizeof(HSMCom_EccSignType)+3)/4)+1]

hsm_uint32_t HSM_DrvEccSignInfType::aPrivate[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aR[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccSignInfType::aS[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

HSM_DataFormatType HSM_DrvEccSignInfType::eOutFmt

HSMCom_EccSignType* HSM_DrvEccSignInfType::pParm

hsm_uint8_t* HSM_DrvEccSignInfType::pR

hsm_uint8_t* HSM_DrvEccSignInfType::pS

hsm_uint32_t HSM_DrvEccSignInfType::u32NBitCnt

hsm_uint32_t HSM_DrvEccSignInfType::u32RBufByteCnt

hsm_uint32_t HSM_DrvEccSignInfType::u32SBufByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvEccVerifyInfType Struct Reference

Ecc verify information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvEccVerifyInfType:

IMAGE

Public Attributes

hsm_uint32_t u32NBitCnt

HSMCom_EccVerifyType * pParm

hsm_uint32_t aP [32+1]

hsm_uint32_t aN [32+1]

hsm_uint32_t aA [32+1]

hsm_uint32_t aB [32+1]

hsm_uint32_t aGx [32+1]

hsm_uint32_t aGy [32+1]

hsm_uint32_t akGx [32+1]

hsm_uint32_t akGy [32+1]

hsm_uint32_t aR [32+1]

hsm_uint32_t aS [32+1]

hsm_uint32_t aHash [32+1]

hsm_uint32_t aParmBuf [(((sizeof(**HSMCom_EccVerifyType**)+3)/4)+1]

Detailed Description

Ecc verify information type.

Member Data Documentation

hsm_uint32_t HSM_DrvEccVerifyInfType::aA[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aB[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aGx[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aGy[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aHash[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::akGx[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::akGy[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aN[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aP[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aParmBuf[((sizeof(HSMCom_EccVerifyType)+3)/4)+1]

hsm_uint32_t HSM_DrvEccVerifyInfType::aR[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

hsm_uint32_t HSM_DrvEccVerifyInfType::aS[32+1]

1024bit for max pkam ECC number calculate, 1 for address align

HSMCom_EccVerifyType* HSM_DrvEccVerifyInfType::pParm

hsm_uint32_t HSM_DrvEccVerifyInfType::u32NBitCnt

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_DrvKeyManageInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvKeyManageInfType:

IMAGE

Public Attributes

HSMCom_UserKeyManageType * pParm

hsm_uint32_t aParmBuf [((sizeof(HSMCom_UserKeyManageType)+3)/4)+1]

union {

KeyManager_UserKeyStatusType tKeyIdStatus

hsm_uint32_t aAlignedDataBuffer [(128 *3)+1]

} **tKeyInfoShared**

Member Data Documentation

hsm_uint32_t HSM_DrvKeyManageInfType::aAlignedDataBuffer[(128 *3)+1]

hsm_uint32_t HSM_DrvKeyManageInfType::aParmBuf[((sizeof(HSMCom_UserKeyManageType)+3)/4)+1]

HSMCom_UserKeyManageType* HSM_DrvKeyManageInfType::pParm

KeyManager_UserKeyStatusType HSM_DrvKeyManageInfType::tKeyIdStatus

union { ... } HSM_DrvKeyManageInfType::tKeyInfoShared

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvKeySpaceInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvKeySpaceInfType:

IMAGE

Public Attributes

HSMCom_KeySpaceStatusType * pSpaceParm

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_KeySpaceStatusType)+3)/4)+1]

HSM_Ln_KeySpaceStatusType tKeySpaceStatus

Member Data Documentation

hsm_uint32_t HSM_DrvKeySpaceInfType::aParmBuf[(((sizeof(HSMCom_KeySpaceStatusType)+3)/4)+1]

HSMCom_KeySpaceStatusType* HSM_DrvKeySpaceInfType::pSpaceParm

HSM_Ln_KeySpaceStatusType HSM_DrvKeySpaceInfType::tKeySpaceStatus

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvLoadFirmwareInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvLoadFirmwareInfType:

IMAGE

Public Attributes

HSMCom_LoadFirmwareType * pParm

uint32 aParmBuf [(((sizeof(HSMCom_LoadFirmwareType)+3)/4)+1]

Member Data Documentation

uint32 HSM_DrvLoadFirmwareInfType::aParmBuf[(((sizeof(HSMCom_LoadFirmwareType)+3)/4)+1]

HSMCom_LoadFirmwareType* HSM_DrvLoadFirmwareInfType::pParm

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvMd5InfType Struct Reference

Md5 information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvMd5InfType:

IMAGE

Public Attributes

HSMCom_Md5ExType * pParm

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pOutResultSize

HSM_BoolType bSwapOut

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_Md5ExType)+3)/4)+1]

hsm_uint32_t aResultCacheBuf [4+1]

hsm_uint32_t aInputCacheBuf [128+1]

Detailed Description

Md5 information type.

Member Data Documentation

hsm_uint32_t HSM_DrvMd5InfType::aInputCacheBuf[128+1]

hsm_uint32_t HSM_DrvMd5InfType::aParmBuf[(((sizeof(HSMCom_Md5ExType)+3)/4)+1]

hsm_uint32_t HSM_DrvMd5InfType::aResultCacheBuf[4+1]

cache buffer for result

HSM_BoolType HSM_DrvMd5InfType::bSwapOut

hsm_uint32_t* HSM_DrvMd5InfType::pOutResultSize

HSMCom_Md5ExType* HSM_DrvMd5InfType::pParm

hsm_uint8_t* HSM_DrvMd5InfType::pResultBuf

hsm_uint32_t HSM_DrvMd5InfType::u32ResultBufSize

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvRandPoolType Struct Reference

Drive rand pool type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

hsm_uint32_t aTrngBuf0 [HSM_TRNG_RAND_U32_CNT]

hsm_uint32_t u32LFSR

hsm_uint32_t u32LFSRUsedIndex

hsm_uint32_t u32LFSRUsedCnt

hsm_uint32_t u32NoRepeatCnt

Detailed Description

Drive rand pool type.

Member Data Documentation

hsm_uint32_t HSM_DrvRandPoolType::aTrngBuf0[HSM_TRNG_RAND_U32_CNT]

hsm_uint32_t HSM_DrvRandPoolType::u32LFSR

hsm_uint32_t HSM_DrvRandPoolType::u32LFSRUsedCnt

hsm_uint32_t HSM_DrvRandPoolType::u32LFSRUsedIndex

hsm_uint32_t HSM_DrvRandPoolType::u32NoRepeatCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvRsaInfType Struct Reference

Rsa information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvRsaInfType:

IMAGE

Public Attributes

```
hsm_uint32_t u32NBitCnt
HSMCom_RsaType * pRsaParm
hsm_uint8_t * pResultBuf
hsm_uint32_t u32ResultBufSize
hsm_uint32_t * pResultByteCnt
HSM_DataFormatType eOutFmt
hsm_uint32_t aRsaInputAndResult [128+1]
hsm_uint32_t aRsaE [128+1]
hsm_uint32_t aRsaN [128+1]
hsm_uint32_t aRsaParmBuf [(((sizeof(HSMCom_RsaType)+3)/4)+1)]
```

Detailed Description

Rsa information type.

Member Data Documentation

hsm_uint32_t HSM_DrvRsaInfType::aRsaE[128+1]

4096bit for max pkam big number calculate, 1 for address align

hsm_uint32_t HSM_DrvRsaInfType::aRsaInputAndResult[128+1]

4096bit for max pkam big number calculate, 1 for address align

hsm_uint32_t HSM_DrvRsaInfType::aRsaN[128+1]

4096bit for max pkam big number calculate, 1 for address align

hsm_uint32_t HSM_DrvRsaInfType::aRsaParmBuf[((sizeof(HSMCom_RsaType)+3)/4)+1]

HSM_DataFormatType HSM_DrvRsaInfType::eOutFmt

hsm_uint8_t* HSM_DrvRsaInfType::pResultBuf

hsm_uint32_t* HSM_DrvRsaInfType::pResultByteCnt

HSMCom_RsaType* HSM_DrvRsaInfType::pRsaParm

hsm_uint32_t HSM_DrvRsaInfType::u32NBitCnt

hsm_uint32_t HSM_DrvRsaInfType::u32ResultBufSize

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvScatterCMacInfType Struct Reference

Scatter SM CMac information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvScatterCMacInfType:

IMAGE

Public Attributes

```
HSM_ScatterCMacSMTType eState
hsm_uint32_t u32TimeoutMs
hsm_uint8_t * pOut
hsm_uint32_t u32OutBufSize
HSM_DataFormatType eOutFmt
hsm_uint32_t * pOutSize
const hsm_uint8_t * pInputData
HSM_DataFormatType eInputFmt
hsm_uint32_t u32InputTotalSize
hsm_uint32_t u32ProcessedSize
hsm_uint32_t u32CacheDataByteCnt
hsm_uint32_t u32CacheProcByteCnt
HSM_AesKeyType eUseKeyType
HSM_AesmBackendType eBackend
hsm_uint32_t u32KeyId
HSM_AesKeyBitCntType eKeyType
hsm_uint32_t aKeyData [8]
HSMCom_AesmRawApiType * pScatterCMacParm
union {
    hsm_uint32_t aCMacResultBuf [16]
    hsm_uint32_t aCMacCtx [16]
} tScatterCMacResultShared
union {
    hsm_uint32_t aCMacParmBuf [(((sizeof(HSMCom_CMacType)+3)/4)+1]
    hsm_uint32_t aScatterCMacParmBuf [(((sizeof(HSMCom_AesmRawApiType)+3)/4)+1]
} tScatterCMacParamShared
hsm_uint32_t aDataBuf [128]
```

Detailed Description

Scatter SM CMac information type.

Member Data Documentation

hsm_uint32_t HSM_DrvScatterCMacInfType::aCMacCtx[16]

hsm_uint32_t HSM_DrvScatterCMacInfType::aCMacParmBuf[((sizeof(HSMCom_CMacType)+3)/4)+1]

hsm_uint32_t HSM_DrvScatterCMacInfType::aCMacResultBuf[16]

hsm_uint32_t HSM_DrvScatterCMacInfType::aDataBuf[128]

hsm_uint32_t HSM_DrvScatterCMacInfType::aKeyData[8]

for max 256bit key

hsm_uint32_t

HSM_DrvScatterCMacInfType::aScatterCMacParmBuf[((sizeof(HSMCom_AesmRawApiType)+3)/4)+1]

HSM_AesmBackendType HSM_DrvScatterCMacInfType::eBackend

HSM_DataFormatType HSM_DrvScatterCMacInfType::eInputFmt

HSM_AesKeyBitCntType HSM_DrvScatterCMacInfType::eKeyType

the key bit count, which determine the AES algorithm type

HSM_DataFormatType HSM_DrvScatterCMacInfType::eOutFmt
HSM_ScatterCMacSMTType HSM_DrvScatterCMacInfType::eState
HSM_AesKeyType HSM_DrvScatterCMacInfType::eUseKeyType
const hsm_uint8_t* HSM_DrvScatterCMacInfType::pInputData
hsm_uint8_t* HSM_DrvScatterCMacInfType::pOut
hsm_uint32_t* HSM_DrvScatterCMacInfType::pOutSize
HSMCom_AesmRawApiType* HSM_DrvScatterCMacInfType::pScatterCMacParm
union { ... } HSM_DrvScatterCMacInfType::tScatterCMacParamShared
union { ... } HSM_DrvScatterCMacInfType::tScatterCMacResultShared
hsm_uint32_t HSM_DrvScatterCMacInfType::u32CacheDataByteCnt
hsm_uint32_t HSM_DrvScatterCMacInfType::u32CacheProcByteCnt
hsm_uint32_t HSM_DrvScatterCMacInfType::u32InputTotalSize
hsm_uint32_t HSM_DrvScatterCMacInfType::u32KeyId
hsm_uint32_t HSM_DrvScatterCMacInfType::u32OutBufSize
hsm_uint32_t HSM_DrvScatterCMacInfType::u32ProcessedSize
hsm_uint32_t HSM_DrvScatterCMacInfType::u32TimeoutMs

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvScatterHashInfType Struct Reference

Scatter hash information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvScatterHashInfType:

IMAGE

Public Attributes

```
hsm_uint32_t u32TimeoutMs
HSM_ShaAlgType eAlg
hsm_uint8_t * pOut
hsm_uint32_t u32OutBufSize
HSM_DataFormatType eOutFmt
hsm_uint32_t * pOutSize
HSM_ScatterHashSMType eState
const hsm_uint8_t * pInputData
HSM_DataFormatType eInputFmt
hsm_uint32_t u32InputTotalSize
hsm_uint32_t u32ProcessedSize
hsm_uint32_t u32CacheDataByteCnt
HSMCom_ScatterHashType * pScatterHashParm
union {
    hsm_uint32_t aShaResultBuf [(((sizeof(HSM_ShaResultBufType)+3)/4)+1]
    hsm_uint32_t aHashCtx [18]
} tScatterHashResultShared
union {
    hsm_uint32_t aShaParmBuf [(((sizeof(HSMCom_ShaExType)+3)/4)+1]
    hsm_uint32_t aScatterHashParmBuf [(((sizeof(HSMCom_ScatterHashType)+3)/4)+1]
} tScatterHashParamShared
HSM_Ln_HashMacCfgType tMacCfg
hsm_uint32_t aMacKeyCache [0x21]
hsm_uint32_t aDataBuf [128]
```

Detailed Description

Scatter hash information type.

Member Data Documentation

hsm_uint32_t HSM_DrvScatterHashInfType::aDataBuf[128]

hsm_uint32_t HSM_DrvScatterHashInfType::aHashCtx[18]

hsm_uint32_t HSM_DrvScatterHashInfType::aMacKeyCache[0x21]

hsm_uint32_t

HSM_DrvScatterHashInfType::aScatterHashParmBuf[((sizeof(HSMCom_ScatterHashType)+3)/4)+1]

hsm_uint32_t HSM_DrvScatterHashInfType::aShaParmBuf[((sizeof(HSMCom_ShaExType)+3)/4)+1]

hsm_uint32_t HSM_DrvScatterHashInfType::aShaResultBuf[((sizeof(HSM_ShaResultBufType)+3)/4)+1]

HSM_ShaAlgType HSM_DrvScatterHashInfType::eAlg

HSM_DataFormatType HSM_DrvScatterHashInfType::eInputFmt

HSM_DataFormatType HSM_DrvScatterHashInfType::eOutFmt

HSM_ScatterHashSMTType HSM_DrvScatterHashInfType::eState

const hsm_uint8_t* HSM_DrvScatterHashInfType::pInputData

hsm_uint8_t* HSM_DrvScatterHashInfType::pOut

hsm_uint32_t* HSM_DrvScatterHashInfType::pOutSize

HSMCom_ScatterHashType* HSM_DrvScatterHashInfType::pScatterHashParm

HSM_Ln_HashMacCfgType HSM_DrvScatterHashInfType::tMacCfg

union { ... } HSM_DrvScatterHashInfType::tScatterHashParamShared

union { ... } HSM_DrvScatterHashInfType::tScatterHashResultShared

hsm_uint32_t HSM_DrvScatterHashInfType::u32CacheDataByteCnt

hsm_uint32_t HSM_DrvScatterHashInfType::u32InputTotalSize

hsm_uint32_t HSM_DrvScatterHashInfType::u32OutBufSize

hsm_uint32_t HSM_DrvScatterHashInfType::u32ProcessedSize

hsm_uint32_t HSM_DrvScatterHashInfType::u32TimeoutMs

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvShaInfType Struct Reference

Sha information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvShaInfType:

IMAGE

Public Attributes

HSMCom_ShaExType * pParm

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pOutResultSize

HSM_BoolType bSwapOut

hsm_uint32_t aParmBuf [(((sizeof(**HSMCom_ShaExType**)+3)/4)+1]

hsm_uint32_t aInputCacheBuf [128+1]

hsm_uint32_t aShaResultCache [(((sizeof(**HSM_ShaResultBufType**)+3)/4)+1]

Detailed Description

Sha information type.

Member Data Documentation

hsm_uint32_t HSM_DrvShaInfType::aInputCacheBuf[128+1]

hsm_uint32_t HSM_DrvShaInfType::aParmBuf[((sizeof(HSMCom_ShaExType)+3)/4)+1]

hsm_uint32_t HSM_DrvShaInfType::aShaResultCache[((sizeof(HSM_ShaResultBufType)+3)/4)+1]

HSM_BoolType HSM_DrvShaInfType::bSwapOut

hsm_uint32_t* HSM_DrvShaInfType::pOutResultSize

HSMCom_ShaExType* HSM_DrvShaInfType::pParm

hsm_uint8_t* HSM_DrvShaInfType::pResultBuf

hsm_uint32_t HSM_DrvShaInfType::u32ResultBufSize

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvSm2GenZaInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvSm2GenZaInfType:

IMAGE

Public Attributes

HSMCom_Sm2GenZaType * pParm

HSM_BoolType bSwapOut

hsm_uint32_t akGx [8]

hsm_uint32_t akGy [8]

hsm_uint32_t aUserID [(HSM_SM2_USERID_MAX_LENGTH+3)/4]

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_Sm2GenZaType)+3)/4)+1)]

Member Data Documentation

hsm_uint32_t HSM_DrvSm2GenZaInfType::akGx[8]

hsm_uint32_t HSM_DrvSm2GenZaInfType::akGy[8]

hsm_uint32_t HSM_DrvSm2GenZaInfType::aParmBuf[(((sizeof(HSMCom_Sm2GenZaType)+3)/4)+1)]

hsm_uint32_t HSM_DrvSm2GenZaInfType::aUserID[(HSM_SM2_USERID_MAX_LENGTH+3)/4]

HSM_BoolType HSM_DrvSm2GenZaInfType::bSwapOut

HSMCom_Sm2GenZaType* HSM_DrvSm2GenZaInfType::pParm

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvSm2VerifyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_DrvSm2VerifyInfType:

IMAGE

Public Attributes

HSMCom_Sm2VerifyType * pParm

hsm_uint32_t akGx [8]

hsm_uint32_t akGy [8]

hsm_uint32_t aR [8]

hsm_uint32_t aS [8]

hsm_uint32_t aHash [8]

hsm_uint32_t aUserID [(HSM_SM2_USERID_MAX_LENGTH+3)/4]

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_Sm2VerifyType)+3)/4)+1)]

Member Data Documentation

hsm_uint32_t HSM_DrvSm2VerifyInfType::aHash[8]

hsm_uint32_t HSM_DrvSm2VerifyInfType::akGx[8]

hsm_uint32_t HSM_DrvSm2VerifyInfType::akGy[8]

hsm_uint32_t HSM_DrvSm2VerifyInfType::aParmBuf[(((sizeof(HSMCom_Sm2VerifyType)+3)/4)+1)]

hsm_uint32_t HSM_DrvSm2VerifyInfType::aR[8]

hsm_uint32_t HSM_DrvSm2VerifyInfType::aS[8]

hsm_uint32_t HSM_DrvSm2VerifyInfType::aUserID[(HSM_SM2_USERID_MAX_LENGTH+3)/4]

HSMCom_Sm2VerifyType* HSM_DrvSm2VerifyInfType::pParm

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_DrvSm3InfType Struct Reference

Sm3 information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvSm3InfType:

IMAGE

Public Attributes

HSMCom_Sm3ExType * pParm

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pOutResultSize

HSM_BoolType bSwapOut

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_Sm3ExType)+3)/4)+1]

hsm_uint32_t aResultCacheBuf [8+1]

hsm_uint32_t aInputCacheBuf [128+1]

Detailed Description

Sm3 information type.

Member Data Documentation

hsm_uint32_t HSM_DrvSm3InfType::aInputCacheBuf[128+1]

hsm_uint32_t HSM_DrvSm3InfType::aParmBuf[(((sizeof(HSMCom_Sm3ExType)+3)/4)+1]

hsm_uint32_t HSM_DrvSm3InfType::aResultCacheBuf[8+1]

cache buffer for result

HSM_BoolType HSM_DrvSm3InfType::bSwapOut

hsm_uint32_t* HSM_DrvSm3InfType::pOutResultSize

HSMCom_Sm3ExType* HSM_DrvSm3InfType::pParm

hsm_uint8_t* HSM_DrvSm3InfType::pResultBuf

hsm_uint32_t HSM_DrvSm3InfType::u32ResultBufSize

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvSm4DecryptInfType Struct Reference

Sm4 decrypt information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvSm4DecryptInfType:

IMAGE

Public Attributes

HSMCom_Sm4DecryptExType * pParm

hsm_uint32_t u32HsmRetResultSize

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_Sm4DecryptExType)+3)/4)+1)]

hsm_uint32_t alvData [4+1]

Detailed Description

Sm4 decrypt information type.

Member Data Documentation

hsm_uint32_t HSM_DrvSm4DecryptInfType::alvData[4+1]

hsm_uint32_t HSM_DrvSm4DecryptInfType::aParmBuf [(((sizeof(HSMCom_Sm4DecryptExType)+3)/4)+1)]

HSMCom_Sm4DecryptExType* HSM_DrvSm4DecryptInfType::pParm

hsm_uint32_t HSM_DrvSm4DecryptInfType::u32HsmRetResultSize

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_DrvSm4EncryptInfType Struct Reference

Sm4 encrypt information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvSm4EncryptInfType:

IMAGE

Public Attributes

HSMWrap_Sm4EncryptExType tWrapper

HSMCom_Sm4EncryptExType * pParm

hsm_uint32_t aParmBuf [((sizeof(HSMCom_FlexAesEncryptType)+3)/4)+1]

HSM_BoolType bSwapOut

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultByteCnt

hsm_uint32_t u32HsmRetResultSize

hsm_uint32_t aResultCacheBuf [132+1]

hsm_uint32_t aInputCacheBuf [128+1]

hsm_uint32_t alvData [4+1]

Detailed Description

Sm4 encrypt information type.

Member Data Documentation

hsm_uint32_t HSM_DrvSm4EncryptInfType::aInputCacheBuf[128+1]

hsm_uint32_t HSM_DrvSm4EncryptInfType::alvData[4+1]

hsm_uint32_t HSM_DrvSm4EncryptInfType::aParmBuf[(((sizeof(HSMCom_FlexAesEncryptType)+3)/4)+1)]

hsm_uint32_t HSM_DrvSm4EncryptInfType::aResultCacheBuf[132+1]

pkcs7 result size has more 16bytes than input

HSM_BoolType HSM_DrvSm4EncryptInfType::bSwapOut

HSMCom_Sm4EncryptExType* HSM_DrvSm4EncryptInfType::pParm

hsm_uint8_t* HSM_DrvSm4EncryptInfType::pResultBuf

HSMWrap_Sm4EncryptExType HSM_DrvSm4EncryptInfType::tWrapper

hsm_uint32_t HSM_DrvSm4EncryptInfType::u32HsmRetResultSize

hsm_uint32_t HSM_DrvSm4EncryptInfType::u32ResultByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_DrvTrngInfType Struct Reference

Trng information type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_DrvTrngInfType:

IMAGE

Public Attributes

HSMCom_TrueRandExType * pParm

hsm_uint8_t * pResultBuf

hsm_uint32_t u32ResultBufSize

hsm_uint32_t u32GetSize

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_TrueRandExType)+3)/4)+1)]

hsm_uint32_t aRandom [HSM_TRNG_RAND_U32_CNT+1]

Detailed Description

Trng information type.

Member Data Documentation

hsm_uint32_t HSM_DrvTrngInfType::aParmBuf [(((sizeof(HSMCom_TrueRandExType)+3)/4)+1)]

hsm_uint32_t HSM_DrvTrngInfType::aRandom [HSM_TRNG_RAND_U32_CNT+1]

HSMCom_TrueRandExType* HSM_DrvTrngInfType::pParm

hsm_uint8_t* HSM_DrvTrngInfType::pResultBuf

hsm_uint32_t HSM_DrvTrngInfType::u32GetSize

hsm_uint32_t HSM_DrvTrngInfType::u32ResultBufSize

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_EccCalcType Struct Reference

ECC point calculate used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32ByteCount
const hsm_uint32_t * pCoeffi_A
const hsm_uint32_t * pCoeffi_B
const hsm_uint32_t * pP
const hsm_uint32_t * pN
const hsm_uint32_t * pP1x
const hsm_uint32_t * pP1y
const hsm_uint32_t * pE
const hsm_uint32_t * pP2x
hsm_uint32_t * pResultx
hsm_uint32_t u32CalcType
const hsm_uint32_t * pP2y
hsm_uint32_t * pResulty
```

Detailed Description

ECC point calculate used by driver when call specific API.

Member Data Documentation

const hsm_uint32_t* HSM_EccCalcType::pCoeffi_A

address should align with 4bytes, the constant "a" in $y^2 = x^3 + a*x + b \pmod{p}$, all data 8 bytes aligned

const hsm_uint32_t* HSM_EccCalcType::pCoeffi_B

address should align with 4bytes, the constant "b" in $y^2 = x^3 + a*x + b \pmod{p}$

const hsm_uint32_t* HSM_EccCalcType::pE

address should align with 4bytes, decrypt side private key

const hsm_uint32_t* HSM_EccCalcType::pN

address should align with 4bytes, ecc curve order; size; the count of all possible EC points

const hsm_uint32_t* HSM_EccCalcType::pP

address should align with 4bytes, ecc curve modulus

const hsm_uint32_t* HSM_EccCalcType::pP1x

address should align with 4bytes, encrypt side's public key axis x data

const hsm_uint32_t* HSM_EccCalcType::pP1y

address should align with 4bytes, encrypt side's public key axis y data

const hsm_uint32_t* HSM_EccCalcType::pP2x

address should align with 4bytes, data to be decrypted, its byte count should be same as u32ByteCount

const hsm_uint32_t* HSM_EccCalcType::pP2y

hsm_uint32_t* HSM_EccCalcType::pResultx

address should align with 4bytes, driver internal write the result to this buffer

hsm_uint32_t* HSM_EccCalcType::pResulty

hsm_uint32_t HSM_EccCalcType::u32ByteCount

all the data size, it should contains all N data

hsm_uint32_t HSM_EccCalcType::u32CalcType

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_EccEasyDecryType Struct Reference

ECC decrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32ByteCount
const hsm_uint32_t * pCoeffi_A
const hsm_uint32_t * pCoeffi_B
const hsm_uint32_t * pP
const hsm_uint32_t * pN
const hsm_uint32_t * pOtherSidePublicKey_x
const hsm_uint32_t * pOtherSidePublicKey_y
const hsm_uint32_t * pPrivateKey
const hsm_uint32_t * pEncryedData
hsm_uint32_t * pPlainData
```

Detailed Description

ECC decrypt information used by driver when call specific API.

Member Data Documentation

const hsm_uint32_t* HSM_EccEasyDecryType::pCoeffi_A

address should align with 4bytes, the constant "a" in $y^2 = x^3 + a*x + b \pmod{p}$, all data 8 bytes aligned

const hsm_uint32_t* HSM_EccEasyDecryType::pCoeffi_B

address should align with 4bytes, the constant "b" in $y^2 = x^3 + a*x + b \pmod{p}$

const hsm_uint32_t* HSM_EccEasyDecryType::pEncryedData

address should align with 4bytes, data to be decrypted, its byte count should be same as u32ByteCount

const hsm_uint32_t* HSM_EccEasyDecryType::pN

address should align with 4bytes, ecc curve order; size; the count of all possible EC points

const hsm_uint32_t* HSM_EccEasyDecryType::pOtherSidePublicKey_x

address should align with 4bytes, encrypt side's public key axis x data

const hsm_uint32_t* HSM_EccEasyDecryType::pOtherSidePublicKey_y

address should align with 4bytes, encrypt side's public key axis y data

const hsm_uint32_t* HSM_EccEasyDecryType::pP

address should align with 4bytes, ecc curve modulus

hsm_uint32_t* HSM_EccEasyDecryType::pPlainData

address should align with 4bytes, driver internal write the result to this buffer

const hsm_uint32_t* HSM_EccEasyDecryType::pPrivateKey

address should align with 4bytes, decrypt side private key

hsm_uint32_t HSM_EccEasyDecryType::u32ByteCount

all the data size, it should contains all N data

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_EccEasyEncryType Struct Reference

ECC encrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32ByteCount
const hsm_uint32_t * pCoeffi_A
const hsm_uint32_t * pCoeffi_B
const hsm_uint32_t * pP
const hsm_uint32_t * pN
const hsm_uint32_t * pOtherSidePublicKey_x
const hsm_uint32_t * pOtherSidePublicKey_y
const hsm_uint32_t * pPrivateKey
const hsm_uint32_t * pPlainData
hsm_uint32_t * pEncrydedData
```

Detailed Description

ECC encrypt information used by driver when call specific API.

Member Data Documentation

const hsm_uint32_t* HSM_EccEasyEncryType::pCoeffi_A

address should align with 4bytes, the constant "a" in $y^2 = x^3 + a*x + b \pmod{p}$, all data 8 bytes aligned

const hsm_uint32_t* HSM_EccEasyEncryType::pCoeffi_B

address should align with 4bytes, the constant "b" in $y^2 = x^3 + a*x + b \pmod{p}$

hsm_uint32_t* HSM_EccEasyEncryType::pEncrydedData

address should align with 4bytes, driver internal write the result to this buffer

const hsm_uint32_t* HSM_EccEasyEncryType::pN

address should align with 4bytes, ecc curve order; size; the count of all possible EC points

const hsm_uint32_t* HSM_EccEasyEncryType::pOtherSidePublicKey_x

address should align with 4bytes, decrypt side's public key axis x data

const hsm_uint32_t* HSM_EccEasyEncryType::pOtherSidePublicKey_y

address should align with 4bytes, decrypt side's public key axis y data

const hsm_uint32_t* HSM_EccEasyEncryType::pP

address should align with 4bytes, ecc curve modulus

const hsm_uint32_t* HSM_EccEasyEncryType::pPlainData

address should align with 4bytes, data to be encrypted, its byte count should be same as u32ByteCount

const hsm_uint32_t* HSM_EccEasyEncryType::pPrivateKey

address should align with 4bytes, encrypt side private key

hsm_uint32_t HSM_EccEasyEncryType::u32ByteCount

all the data size, it should contains all N data

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_EccSignType Struct Reference

ECC sign information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_EccSignType:

IMAGE

Public Attributes

hsm_uint32_t u32ByteCount

HSM_DrvEccCurveParamType tCurve

const **hsm_uint32_t** * pPrivateKey

const **hsm_uint32_t** * pHashData

hsm_uint32_t * pR

hsm_uint32_t * pS

Detailed Description

ECC sign information used by driver when call specific API.

Member Data Documentation

const hsm_uint32_t* HSM_EccSignType::pHashData

address is 4bytes aligned, hash of the data to verify

const hsm_uint32_t* HSM_EccSignType::pPrivateKey

address is 4bytes aligned, private key

hsm_uint32_t* HSM_EccSignType::pR

address is 4bytes aligned, driver internal use this buffer to store the sign result R of the data to sign

hsm_uint32_t* HSM_EccSignType::pS

address is 4bytes aligned, driver internal use this buffer to store the sign result S of the data to sign

HSM_DrvEccCurveParamType HSM_EccSignType::tCurve

ecc curve parameters

hsm_uint32_t HSM_EccSignType::u32ByteCount

all the data size, it should contains all N data

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_EccVerifyType Struct Reference

ECC verify information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSM_EccVerifyType:

IMAGE

Public Attributes

hsm_uint32_t u32DataByteCnt

HSM_DrvEccCurveParamType tCurve

const **hsm_uint32_t** * pkG_x

const **hsm_uint32_t** * pkG_y

const **hsm_uint32_t** * pHashData

const **hsm_uint32_t** * pR

const **hsm_uint32_t** * pS

Detailed Description

ECC verify information used by driver when call specific API.

Member Data Documentation

const hsm_uint32_t* HSM_EccVerifyType::pHashData

address is 4bytes aligned, hash of the data to verify

const hsm_uint32_t* HSM_EccVerifyType::pkG_x

address is 4bytes aligned, public key axis x

const hsm_uint32_t* HSM_EccVerifyType::pkG_y

address is 4bytes aligned, public key axis y

const hsm_uint32_t* HSM_EccVerifyType::pR

address is 4bytes aligned, the sign result R of the data to verify

const hsm_uint32_t* HSM_EccVerifyType::pS

address is 4bytes aligned, the sign result S of the data to verify

HSM_DrvEccCurveParamType HSM_EccVerifyType::tCurve

ecc curve parameters

hsm_uint32_t HSM_EccVerifyType::u32DataByteCnt

the byte count of all ecc curve parameter

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_FlexAesEnDecryptType Struct Reference

Flex AES encrypt/decrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_FlexAesEnDecryptType:

IMAGE

Public Attributes

```
HSM_AesKeyType eUseKeyType  
HSM_AesNewKeyInfType tNewKeyInf  
HSM_AesVendorKeyInfType tVendorKeyInf  
HSM_SymmEnDecryptAlgType eAesAlg  
const hsm_uint8_t * pDataInput  
hsm_uint32_t u32InputByteCnt  
hsm_uint8_t * pDataOutput  
hsm_uint32_t u32OutputMemSize  
const hsm_uint8_t * plvData  
hsm_uint32_t * pGenerateOutByteCnt  
HSM_FlexAesPadType ePad  
HSM_DataFormatType eInputFmt  
HSM_DataFormatType eOutputFmt
```

Detailed Description

Flex AES encrypt/decrypt information used by driver when call specific API.

Member Data Documentation

HSM_SymmEnDecryptAlgType HSM_FlexAesEnDecryptType::eAesAlg

encrypt/decrypt mode

HSM_DataFormatType HSM_FlexAesEnDecryptType::eInputFmt

key/iv/input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_DataFormatType HSM_FlexAesEnDecryptType::eOutputFmt

output result data format

HSM_FlexAesPadType HSM_FlexAesEnDecryptType::ePad

for encrypt, if data size not 16bytes aligned, the pad byte should refer to this configuration, for decrypt, driver to process the padding data according to this, however, if zero padding, driver will not remove the padding 0, user should delete the data manually

HSM_AesKeyType HSM_FlexAesEnDecryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

const hsm_uint8_t* HSM_FlexAesEnDecryptType::pDataInput

address should align with 4bytes

hsm_uint8_t* HSM_FlexAesEnDecryptType::pDataOutput

address should align with 4bytes

hsm_uint32_t* HSM_FlexAesEnDecryptType::pGenerateOutByteCnt

point to the hsm_uint32_t variable to store the result byte count, can't be NULL_PTR

const hsm_uint8_t* HSM_FlexAesEnDecryptType::pIvData

address should align with 4bytes, 128bit(16Bytes) iv(initialization vector) data array, ECB not need configure this

HSM_AesNewKeyInfType HSM_FlexAesEnDecryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_FlexAesEnDecryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_FlexAesEnDecryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_FlexAesEnDecryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_GcmDecryptType Struct Reference

GCM Decrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_GcmDecryptType:

IMAGE

Public Attributes

```
HSM_AesKeyType eUseKeyType  
HSM_AesNewKeyInfType tNewKeyInf  
HSM_AesVendorKeyInfType tVendorKeyInf  
hsm_uint32_t * pAllData  
hsm_uint32_t u32IvDataByteCnt  
hsm_uint32_t u32AadByteCnt  
hsm_uint32_t u32InputByteCnt  
HSM_BoolType bCheckMacEn  
hsm_uint32_t u32MacByteCnt  
hsm_uint32_t * pDataOutput  
hsm_uint32_t u32OutputMemSize
```

Detailed Description

GCM Decrypt information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_GcmDecryptType::bCheckMacEn

check the MAC data in input or not

HSM_AesKeyType HSM_GcmDecryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

hsm_uint32_t* HSM_GcmDecryptType::pAllData

address should align with 4bytes, place IV data first, then AAD data(if exist, 16bytes align), then input data, then MAC data(if exist)

hsm_uint32_t* HSM_GcmDecryptType::pDataOutput

address should align with 4bytes, point to the buffer that store the result

HSM_AesNewKeyInfType HSM_GcmDecryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_GcmDecryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_GcmDecryptType::u32AadByteCnt

64bit(8Bytes) align, if not, only the aligned data treat as AAD, the left AAD bytes will be treated as input data

hsm_uint32_t HSM_GcmDecryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_GcmDecryptType::u32IvDataByteCnt

64bit(8Bytes) align, if not, only the aligned data treat as IV, the left IV bytes will be treated as AAD data

hsm_uint32_t HSM_GcmDecryptType::u32MacByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_GcmDecryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_GcmEncryptType Struct Reference

GCM Encrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_GcmEncryptType:

IMAGE

Public Attributes

```
HSM_AesKeyType eUseKeyType  
HSM_AesNewKeyInfType tNewKeyInf  
HSM_AesVendorKeyInfType tVendorKeyInf  
hsm_uint32_t * pAllData  
hsm_uint32_t u32IvDataByteCnt  
hsm_uint32_t u32AadByteCnt  
hsm_uint32_t u32InputByteCnt  
hsm_uint32_t u32GenMacByteCnt  
hsm_uint32_t * pDataOutput  
hsm_uint32_t u32OutputMemSize  
hsm_uint32_t(* pMacOut )[4]
```

Detailed Description

GCM Encrypt information used by driver when call specific API.

Member Data Documentation

HSM_AesKeyType HSM_GcmEncryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

hsm_uint32_t* HSM_GcmEncryptType::pAllData

address should align with 4bytes, place IV data first, then AAD data(16bytes align), then input data

hsm_uint32_t* HSM_GcmEncryptType::pDataOutput

address should align with 4bytes

hsm_uint32_t(* HSM_GcmEncryptType::pMacOut)[4]

address should align with 4bytes, buffer address, this buffer store the result

HSM_AesNewKeyInfType HSM_GcmEncryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_GcmEncryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_GcmEncryptType::u32AadByteCnt

64bit(8Bytes) align, if not, only the aligned data treat as AAD, the left AAD bytes will be treated as input data

hsm_uint32_t HSM_GcmEncryptType::u32GenMacByteCnt

set the MAC result byte count

hsm_uint32_t HSM_GcmEncryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_GcmEncryptType::u32IvDataByteCnt

64bit(8Bytes) align, if not, only the aligned data treat as IV, the left IV bytes will be treated as AAD data

hsm_uint32_t HSM_GcmEncryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_KeyManagerCheckUserKeyStatusExtType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32KeyID  
KeyManager_UserKeyStatusType * pUserKeyStatus
```

Member Data Documentation

KeyManager_UserKeyStatusType* HSM_KeyManagerCheckUserKeyStatusExtType::pUserKeyStatus

hsm_uint32_t HSM_KeyManagerCheckUserKeyStatusExtType::u32KeyID

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_KeyManagerExportUserKeyType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t * pDataAddr  
hsm_uint32_t * pDataLength  
hsm_uint32_t u32KeyID  
HSM_KeyManagerUserKeyEnDecryType eEncryType  
hsm_uint32_t u32EncryKeyID  
HSM_KeyManagerEnDecryAlgType eAlgType  
hsm_uint32_t(* pIV)[4]
```

Member Data Documentation

HSM_KeyManagerEnDecryAlgType HSM_KeyManagerExportUserKeyType::eAlgType

HSM_KeyManagerUserKeyEnDecryType HSM_KeyManagerExportUserKeyType::eEncryType

indicate if the key exported is need to be encrypted

hsm_uint32_t* HSM_KeyManagerExportUserKeyType::pDataAddr

address should align with 4bytes

hsm_uint32_t* HSM_KeyManagerExportUserKeyType::pDataLength

address should align with 4bytes, a hsm_uint32_t variable to store the length

hsm_uint32_t(* HSM_KeyManagerExportUserKeyType::pIV)[4]

address should align with 4bytes

hsm_uint32_t HSM_KeyManagerExportUserKeyType::u32EncryKeyID

0 means use USRK as encrykey, otherwise load decrykey with KEYID from hsm key space

hsm_uint32_t HSM_KeyManagerExportUserKeyType::u32KeyID

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_KeyManagerGenUserKeyType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_KeyManagerGenKeyPairType eKeyPairType

HSM_KeyManagerKeyExportType eExportType

HSM_KeyManagerBoolType bStoreInFlash

hsm_uint32_t * pPrivateKeyID

hsm_uint32_t * pPublicKeyID

Member Data Documentation

HSM_KeyManagerBoolType HSM_KeyManagerGenUserKeyType::bStoreInFlash

HSM_KeyManagerKeyExportType HSM_KeyManagerGenUserKeyType::eExportType

HSM_KeyManagerGenKeyPairType HSM_KeyManagerGenUserKeyType::eKeyPairType

hsm_uint32_t* HSM_KeyManagerGenUserKeyType::pPrivateKeyID

address should align with 4bytes

hsm_uint32_t* HSM_KeyManagerGenUserKeyType::pPublicKeyID

address should align with 4bytes

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_KeyManagerImportUserKeyType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_KeyManagerBoolType bStoreInFlash
HSM_KeyManagerKeyExportType eExportType
hsm_uint32_t * pDataAddr
hsm_uint32_t u32DataLength
hsm_uint32_t * pKeyID
HSM_KeyManagerUserKeyEnDecryType eDecryType
hsm_uint32_t u32DecryKeyID
HSM_KeyManagerUserKeyTypeType eUserKeyType
HSM_KeyManagerEnDecryAlgType eAlgType
hsm_uint32_t(* pIV)[4]
```

Member Data Documentation

HSM_KeyManagerBoolType HSM_KeyManagerImportUserKeyType::bStoreInFlash

HSM_KeyManagerEnDecryAlgType HSM_KeyManagerImportUserKeyType::eAlgType

HSM_KeyManagerUserKeyEnDecryType HSM_KeyManagerImportUserKeyType::eDecryType
indicate if the key imported is plain

HSM_KeyManagerKeyExportType HSM_KeyManagerImportUserKeyType::eExportType

HSM_KeyManagerUserKeyTypeType HSM_KeyManagerImportUserKeyType::eUserKeyType

hsm_uint32_t* HSM_KeyManagerImportUserKeyType::pDataAddr
address should align with 4bytes, the address of key imported

hsm_uint32_t(* HSM_KeyManagerImportUserKeyType::pIV)[4]
address should align with 4bytes

hsm_uint32_t* HSM_KeyManagerImportUserKeyType::pKeyID

address should align with 4bytes

hsm_uint32_t HSM_KeyManagerImportUserKeyType::u32DataLength

the length of key imported

hsm_uint32_t HSM_KeyManagerImportUserKeyType::u32DecryKeyID

0 means use USRK as decrykey, otherwise load decrykey with KEYID from hsm key space

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_L1_BnCalcType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Public Attributes

hsm_uint32_t u32KeyEId
hsm_uint32_t u32KeyNId
hsm_uint32_t u32BitCnt
HSM_L1_BnCalcTypeType eCalcType
const hsm_uint8_t * pA
hsm_uint32_t u32AByteCount
HSM_DataFormatType eAFmt
const hsm_uint8_t * pB
hsm_uint32_t u32BByteCount
HSM_DataFormatType eBFmt
const hsm_uint8_t * pE
hsm_uint32_t u32EByteCount
HSM_DataFormatType eEFmt
const hsm_uint8_t * pN
hsm_uint32_t u32NByteCount
HSM_DataFormatType eNFmt
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt
HSM_DataFormatType eResultFmt

Member Data Documentation

HSM_DataFormatType HSM_L1_BnCalcType::eAFmt

HSM_DataFormatType HSM_L1_BnCalcType::eBFmt

HSM_L1_BnCalcTypeType HSM_L1_BnCalcType::eCalcType

HSM_DataFormatType HSM_L1_BnCalcType::eEFmt

HSM_DataFormatType HSM_L1_BnCalcType::eNFmt

HSM_DataFormatType HSM_L1_BnCalcType::eResultFmt

const hsm_uint8_t* HSM_L1_BnCalcType::pA

address is 4bytes aligned, input data, if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_L1_BnCalcType::pB

const hsm_uint8_t* HSM_L1_BnCalcType::pE

address is 4bytes aligned, input key E data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_L1_BnCalcType::pN

address is 4bytes aligned, input key N data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

hsm_uint8_t* HSM_L1_BnCalcType::pResult

address is 4bytes aligned, output buffer, the buffer size should $\geq$ "u32Key_N_ByteCount"

hsm_uint32_t* HSM_L1_BnCalcType::pResultByteCnt

hsm_uint32_t HSM_L1_BnCalcType::u32AByteCount

the byte count of input data

hsm_uint32_t HSM_L1_BnCalcType::u32BByteCount

hsm_uint32_t HSM_L1_BnCalcType::u32BitCnt

the bit count of numbers used of N

hsm_uint32_t HSM_L1_BnCalcType::u32EByteCount

the byte count of key E

hsm_uint32_t HSM_L1_BnCalcType::u32KeyEId

means don't load key from HSM, otherwise would load key with KEYID from HSM

hsm_uint32_t HSM_L1_BnCalcType::u32KeyNId

means don't load key from HSM, otherwise would load key with KEYID from HSM

hsm_uint32_t HSM_L1_BnCalcType::u32NByteCount

hw will get the actual key data bit count according to the non-zero bit count, it means key not generate by multiply 2

hsm_uint32_t HSM_L1_BnCalcType::u32ResultBufByteCnt

the result buffer size

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_DrvRSASsaPkcs1V15SignType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L1_DrvRSASsaPkcs1V15SignType:

IMAGE

Public Attributes

hsm_uint32_t u32BitCnt

HSMCom_RsaSsaPkcs1v15SignType * pParm

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_RsaSsaPkcs1v15SignType)+3)/4)+1)]

Member Data Documentation

hsm_uint32_t

HSM_L1_DrvRSASsaPkcs1V15SignType::aParmBuf[(((sizeof(HSMCom_RsaSsaPkcs1v15SignType)+3)/4)+1)]

HSMCom_RsaSsaPkcs1v15SignType* HSM_L1_DrvRSASsaPkcs1V15SignType::pParm

hsm_uint32_t HSM_L1_DrvRSASsaPkcs1V15SignType::u32BitCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_DrvRSASsaPkcs1V15VerifyType Struct Reference

Level 1 RSAssa Pkcs15 verify type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_L1_DrvRSASsaPkcs1V15VerifyType:

IMAGE

Public Attributes

hsm_uint32_t u32BitCnt

HSMCom_RsaSsaPkcs1V15VerifyType * pParm

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_RsaSsaPkcs1V15VerifyType)+3)/4)+1]

Crypto_VerifyResultType * pVerifyResult

Detailed Description

Level 1 RSAssa Pkcs15 verify type.

Member Data Documentation

hsm_uint32_t

HSM_L1_DrvRSASsaPkcs1V15VerifyType::aParmBuf [(((sizeof(HSMCom_RsaSsaPkcs1V15VerifyType)+3)/4)+1]

HSMCom_RsaSsaPkcs1V15VerifyType* **HSM_L1_DrvRSASsaPkcs1V15VerifyType::pParm**

Crypto_VerifyResultType* **HSM_L1_DrvRSASsaPkcs1V15VerifyType::pVerifyResult**

hsm_uint32_t **HSM_L1_DrvRSASsaPkcs1V15VerifyType::u32BitCnt**

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_DrvRSASsaPssSignType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L1_DrvRSASsaPssSignType:

IMAGE

Public Attributes

hsm_uint32_t u32BitCnt

HSMCom_RsaSsaPssSignType * pParm

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_RsaSsaPssSignType)+3)/4)+1)]

Member Data Documentation

hsm_uint32_t

HSM_L1_DrvRSASsaPssSignType::aParmBuf[(((sizeof(HSMCom_RsaSsaPssSignType)+3)/4)+1)]

HSMCom_RsaSsaPssSignType* HSM_L1_DrvRSASsaPssSignType::pParm

hsm_uint32_t HSM_L1_DrvRSASsaPssSignType::u32BitCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_DrvRSASsaPssVerifyType Struct Reference

Level 1 RSAssa Pss verify type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L1_DrvRSASsaPssVerifyType:

IMAGE

Public Attributes

hsm_uint32_t u32BitCnt

HSMCom_RsaSsaPssVerifyType * pParm

hsm_uint32_t aParmBuf [(((sizeof(HSMCom_RsaSsaPssVerifyType)+3)/4)+1]

Crypto_VerifyResultType * pVerifyResult

Detailed Description

Level 1 RSAssa Pss verify type.

Member Data Documentation

hsm_uint32_t

HSM_L1_DrvRSASsaPssVerifyType::aParmBuf [(((sizeof(HSMCom_RsaSsaPssVerifyType)+3)/4)+1]

HSMCom_RsaSsaPssVerifyType* **HSM_L1_DrvRSASsaPssVerifyType::pParm**

Crypto_VerifyResultType* **HSM_L1_DrvRSASsaPssVerifyType::pVerifyResult**

hsm_uint32_t **HSM_L1_DrvRSASsaPssVerifyType::u32BitCnt**

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_EccCurveParamType Struct Reference

Ecc curve type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pP
hsm_uint32_t u32PByteCnt
const hsm_uint8_t * pN
hsm_uint32_t u32NByteCnt
const hsm_uint8_t * pA
hsm_uint32_t u32AByteCnt
const hsm_uint8_t * pB
hsm_uint32_t u32BByteCnt
const hsm_uint8_t * pGx
hsm_uint32_t u32GxByteCnt
const hsm_uint8_t * pGy
hsm_uint32_t u32GyByteCnt
```

Detailed Description

Ecc curve type.

Member Data Documentation

const hsm_uint8_t* HSM_L1_EccCurveParamType::pA

address is 4bytes aligned, the constant "a" in $y^2 = x^3 + a \cdot x + b \pmod{p}$

const hsm_uint8_t* HSM_L1_EccCurveParamType::pB

address is 4bytes aligned, the constant "b" in $y^2 = x^3 + a \cdot x + b \pmod{p}$

const hsm_uint8_t* HSM_L1_EccCurveParamType::pGx

address is 4bytes aligned, x of the curve generator point G {x, y}

const hsm_uint8_t* HSM_L1_EccCurveParamType::pGy

address is 4bytes aligned, y of the curve generator point G {x, y}

const hsm_uint8_t* HSM_L1_EccCurveParamType::pN

address is 4bytes aligned, ecc curve order; size; the count of all possible EC points

const hsm_uint8_t* HSM_L1_EccCurveParamType::pP

address is 4bytes aligned, ecc curve modulus

hsm_uint32_t HSM_L1_EccCurveParamType::u32AByteCnt

hsm_uint32_t HSM_L1_EccCurveParamType::u32BByteCnt

hsm_uint32_t HSM_L1_EccCurveParamType::u32GxByteCnt

hsm_uint32_t HSM_L1_EccCurveParamType::u32GyByteCnt

hsm_uint32_t HSM_L1_EccCurveParamType::u32NByteCnt

hsm_uint32_t HSM_L1_EccCurveParamType::u32PByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_EccVerifyType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L1_EccVerifyType:

IMAGE

Public Attributes

HSM_DrvEccCurvePrmIndexType u32EccCurve

HSM_L1_EccCurveParamType tCurve

HSM_DataFormatType eCurveDataFmt

hsm_uint32_t u32PublicKeyId

const **hsm_uint8_t** * pkGx

hsm_uint32_t u32kGxByteCnt

const **hsm_uint8_t** * pkGy

hsm_uint32_t u32kGyByteCnt

HSM_DataFormatType ekGxyDataFmt

const **hsm_uint8_t** * pData

hsm_uint32_t u32DataByteCnt

HSM_DataFormatType eDataFmt

const **hsm_uint8_t** * pR

hsm_uint32_t u32RByteCnt

HSM_DataFormatType eRDataFmt

const **hsm_uint8_t** * pS

hsm_uint32_t u32SByteCnt

HSM_DataFormatType eSDataFmt

Member Data Documentation

HSM_DataFormatType HSM_L1_EccVerifyType::eCurveDataFmt

HSM_DataFormatType HSM_L1_EccVerifyType::eDataFmt

HSM_DataFormatType HSM_L1_EccVerifyType::ekGxyDataFmt

HSM_DataFormatType HSM_L1_EccVerifyType::eRDataFmt

HSM_DataFormatType HSM_L1_EccVerifyType::eSDataFmt

const hsm_uint8_t* HSM_L1_EccVerifyType::pData

address is 4bytes aligned, if input hash, hash of the data to verify, if raw data to hash, it point to raw data

const hsm_uint8_t* HSM_L1_EccVerifyType::pkGx

address is 4bytes aligned, public key axis x

const hsm_uint8_t* HSM_L1_EccVerifyType::pkGy

address is 4bytes aligned, public key axis y

const hsm_uint8_t* HSM_L1_EccVerifyType::pR

address is 4bytes aligned, the sign result R of the data to verify

const hsm_uint8_t* HSM_L1_EccVerifyType::pS

address is 4bytes aligned, the sign result S of the data to verify

HSM_L1_EccCurveParamType HSM_L1_EccVerifyType::tCurve

ecc curve parameters

hsm_uint32_t HSM_L1_EccVerifyType::u32DataByteCnt

HSM_DrvEccCurvePrmIndexType HSM_L1_EccVerifyType::u32EccCurve

hsm_uint32_t HSM_L1_EccVerifyType::u32kGxByteCnt

hsm_uint32_t HSM_L1_EccVerifyType::u32kGyByteCnt

hsm_uint32_t HSM_L1_EccVerifyType::u32PublicKeyId

hsm_uint32_t HSM_L1_EccVerifyType::u32RByteCnt

hsm_uint32_t HSM_L1_EccVerifyType::u32SByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_InitParmType Struct Reference

Ecc level 1 init type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_L1_InitParmType:

IMAGE

Public Attributes

HSM_TriggerSrcType eTriggerSrc

HSM_StopCbType pStopCb

HSM_SendCommandCbType pSendCmdCb

HSM_GetAckStatCbType pReadAckCb

HSM_CancelCmdCbType pCancelCmdCb

HSM_CleanDcacheCbType pCleanDCache

HSM_CleanAndFlushDcacheCbType pCleanAndFlushDcache

Detailed Description

Ecc level 1 init type.

Member Data Documentation

HSM_TriggerSrcType HSM_L1_InitParmType::eTriggerSrc

HSM_CancelCmdCbType HSM_L1_InitParmType::pCancelCmdCb

HSM_CleanAndFlushDcacheCbType HSM_L1_InitParmType::pCleanAndFlushDcache

HSM_CleanDcacheCbType HSM_L1_InitParmType::pCleanDCache

HSM_GetAckStatCbType HSM_L1_InitParmType::pReadAckCb

HSM_SendCommandCbType HSM_L1_InitParmType::pSendCmdCb

HSM_StopCbType HSM_L1_InitParmType::pStopCb

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_PKCS8_PrivateKeyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L1_PKCS8_PrivateKeyInfType:

IMAGE

Public Attributes

```
HSM_L1_PKCS8_PrivateKeyType eKeyType
union {
    HSM_L1_PKCS_RsaPrivateKeyInfType tRsaKeyInf
    HSM_L1_PKCS_EccPrivateKeyInfType tEccKeyInf
} tPrivateInfo
```

Member Data Documentation

HSM_L1_PKCS8_PrivateKeyType HSM_L1_PKCS8_PrivateKeyInfType::eKeyType

HSM_L1_PKCS_EccPrivateKeyInfType HSM_L1_PKCS8_PrivateKeyInfType::tEccKeyInf

union { ... } HSM_L1_PKCS8_PrivateKeyInfType::tPrivateInfo

HSM_L1_PKCS_RsaPrivateKeyInfType HSM_L1_PKCS8_PrivateKeyInfType::tRsaKeyInf

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_PKCS_EccPrivateKeyInfType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pPrivate  
hsm_uint32_t u32PrivateByteCnt
```

Member Data Documentation

```
const hsm_uint8_t* HSM_L1_PKCS_EccPrivateKeyInfType::pPrivate
```

```
hsm_uint32_t HSM_L1_PKCS_EccPrivateKeyInfType::u32PrivateByteCnt
```

The documentation for this struct was generated from the following file:

```
module_driver_hsm_level1_leveln.h
```

HSM_L1_PKCS_EccPublicKeyInfType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pKGx  
hsm_uint32_t u32KGxByteCnt  
const hsm_uint8_t * pKGy  
hsm_uint32_t u32KGyByteCnt
```

Member Data Documentation

```
const hsm_uint8_t* HSM_L1_PKCS_EccPublicKeyInfType::pKGx
```

```
const hsm_uint8_t* HSM_L1_PKCS_EccPublicKeyInfType::pKGy
```

```
hsm_uint32_t HSM_L1_PKCS_EccPublicKeyInfType::u32KGxByteCnt
```

```
hsm_uint32_t HSM_L1_PKCS_EccPublicKeyInfType::u32KGyByteCnt
```

The documentation for this struct was generated from the following file:

```
module_driver_hsm_level1_leveln.h
```

HSM_L1_PKCS_RsaPrivateKeyInfType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pN  
hsm_uint32_t u32NByteCnt  
const hsm_uint8_t * pE  
hsm_uint32_t u32EByteCnt  
const hsm_uint8_t * pD  
hsm_uint32_t u32DByteCnt
```

Member Data Documentation

```
const hsm_uint8_t* HSM_L1_PKCS_RsaPrivateKeyInfType::pD
```

```
const hsm_uint8_t* HSM_L1_PKCS_RsaPrivateKeyInfType::pE
```

```
const hsm_uint8_t* HSM_L1_PKCS_RsaPrivateKeyInfType::pN
```

```
hsm_uint32_t HSM_L1_PKCS_RsaPrivateKeyInfType::u32DByteCnt
```

```
hsm_uint32_t HSM_L1_PKCS_RsaPrivateKeyInfType::u32EByteCnt
```

```
hsm_uint32_t HSM_L1_PKCS_RsaPrivateKeyInfType::u32NByteCnt
```

The documentation for this struct was generated from the following file:

```
module_driver_hsm_level1_leveln.h
```

HSM_L1_PKCS_RsaPublicKeyInfType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pN  
hsm_uint32_t u32NByteCnt  
const hsm_uint8_t * pE  
hsm_uint32_t u32EByteCnt
```

Member Data Documentation

```
const hsm_uint8_t* HSM_L1_PKCS_RsaPublicKeyInfType::pE
```

```
const hsm_uint8_t* HSM_L1_PKCS_RsaPublicKeyInfType::pN
```

```
hsm_uint32_t HSM_L1_PKCS_RsaPublicKeyInfType::u32EByteCnt
```

```
hsm_uint32_t HSM_L1_PKCS_RsaPublicKeyInfType::u32NByteCnt
```

The documentation for this struct was generated from the following file:

```
module_driver_hsm_level1_leveln.h
```

HSM_L1_PublicKeyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L1_PublicKeyInfType:

IMAGE

Public Attributes

HSM_L1_PublicKeyType eKeyType

union {

HSM_L1_PKCS_RsaPublicKeyInfType tRsaKeyInf

HSM_L1_PKCS_EccPublicKeyInfType tEccKeyInf

} tPublicInf

Member Data Documentation

HSM_L1_PublicKeyType HSM_L1_PublicKeyInfType::eKeyType

HSM_L1_PKCS_EccPublicKeyInfType HSM_L1_PublicKeyInfType::tEccKeyInf

union { ... } HSM_L1_PublicKeyInfType::tPublicInf

HSM_L1_PKCS_RsaPublicKeyInfType HSM_L1_PublicKeyInfType::tRsaKeyInf

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_RsaType Struct Reference

L1 Rsa config type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
HSM_DataFormatType eInputDataFmt
hsm_uint32_t u32KeyEId
const hsm_uint8_t * pKeyE
hsm_uint32_t u32KeyEByteCount
HSM_DataFormatType eKeyEFmt
hsm_uint32_t u32KeyNId
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
HSM_DataFormatType eKeyNFmt
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt
HSM_DataFormatType eOutputFmt
```

Detailed Description

L1 Rsa config type.

Member Data Documentation

HSM_DataFormatType HSM_L1_RsaType::eInputDataFmt

HSM_DataFormatType HSM_L1_RsaType::eKeyEFmt

HSM_DataFormatType HSM_L1_RsaType::eKeyNFmt

HSM_DataFormatType HSM_L1_RsaType::eOutputFmt

const hsm_uint8_t* HSM_L1_RsaType::pInputData

address is 4bytes aligned, input data, if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_L1_RsaType::pKeyE

address is 4bytes aligned, input key E data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint8_t* HSM_L1_RsaType::pKeyN

address is 4bytes aligned, input key N data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

hsm_uint8_t* HSM_L1_RsaType::pResult

address is 4bytes aligned, output buffer, the buffer size should $\geq$ "u32Key_N_ByteCount"

hsm_uint32_t HSM_L1_RsaType::pResultByteCnt

the result size

hsm_uint32_t HSM_L1_RsaType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_L1_RsaType::u32KeyEByteCount

the byte count of key E

hsm_uint32_t HSM_L1_RsaType::u32KeyEId

hsm_uint32_t HSM_L1_RsaType::u32KeyNByteCount

hw will get the actual key data bit count according to the non-zero bit count, it means key not generate by multiply 2

hsm_uint32_t HSM_L1_RsaType::u32KeyNId

hsm_uint32_t HSM_L1_RsaType::u32ResultBufByteCnt

the result buffer size

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L1_RtCtxType_tag Struct Reference

The structure of the HSM level 1 processing handle.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L1_RtCtxType_tag:

IMAGE

Public Attributes

HSM_TriggerSrcType eTriggerSrc
HSM_StopCbType pStopCb
HSM_SendCommandCbType pSendCmdCb
HSM_GetAckStatCbType pReadAckCb
HSM_CancelCmdCbType pCancelCmdCb
HSM_CleanDcacheCbType pCleanDCache
HSM_CleanAndFlushDcacheCbType pCleanAndFlushDcache
HSM_BoolType bInProcess
hsm_uint32_t eOperationCata
HSM_L1_DrvSMTType eStat
hsm_uint32_t eLastestHsmRet
hsm_uint32_t u32CurStatInf
HSM_DrvRandPoolType tRandPool
HSM_CmdType tCmd
union {
 HSM_DrvMd5InfType tMd5Inf
 HSM_DrvSm3InfType tSm3Inf
 HSM_DrvShaInfType tShaInf
 HSM_DrvTrngInfType tRngInf
 HSM_DrvEccVerifyInfType tEccVerifyInf
 HSM_DrvEccDecryptInfType tEccDecryptInf
 HSM_DrvCMacInfType tCMacInf
 HSM_DrvAesEncryptInfType tAesEncryptInf
 HSM_DrvAesDecryptInfType tAesDecryptInf
 HSM_DrvSm4EncryptInfType tSm4EncryptInf
 HSM_DrvSm4DecryptInfType tSm4DecryptInf
 HSM_DrvBnCalcExtInfType tBnCalcInf

```
HSM_DrvScatterHashInfType tScatterHash
HSM_DrvScatterCMacInfType tScatterCMac
HSM_DrvEccSignInfType tEccSignInf
HSM_L1_DrvRSASsaPssVerifyType tRsaSsaPssVerify
HSM_L1_DrvRSASsaPkcs1V15VerifyType tRsaSsaPkcs1V15Verify
HSM_DrvEccCalcInfType tEccCalc
HSM_DrvKeyManageInfType tKeyInf
HSM_DrvKeySpaceInfType tKeySpaceInf
HSM_L1_DrvRSASsaPkcs1V15SignType tRsaSsaPkcs1v15Sign
HSM_L1_DrvRSASsaPssSignType tRsaSsaPssSign
HSM_DrvLoadFirmwareInfType tLoadFirmware
HSM_DrvSm2VerifyInfType tSm2VerifyInf
HSM_DrvSm2GenZaInfType tSm2GenZaInf
} tL1AlgCtx
```

Detailed Description

The structure of the HSM level 1 processing handle.

Member Data Documentation

HSM_BoolType HSM_L1_RtCtxType_tag::blnProcess

hsm_uint32_t HSM_L1_RtCtxType_tag::eLastestHsmRet

hsm_uint32_t HSM_L1_RtCtxType_tag::eOperationCata

HSM_L1_DrvSMTType HSM_L1_RtCtxType_tag::eStat

HSM_TriggerSrcType HSM_L1_RtCtxType_tag::eTriggerSrc

HSM_CancelCmdCbType HSM_L1_RtCtxType_tag::pCancelCmdCb

HSM_CleanAndFlushDcacheCbType HSM_L1_RtCtxType_tag::pCleanAndFlushDcache

HSM_CleanDcacheCbType HSM_L1_RtCtxType_tag::pCleanDCache

HSM_GetAckStatCbType HSM_L1_RtCtxType_tag::pReadAckCb

HSM_SendCommandCbType HSM_L1_RtCtxType_tag::pSendCmdCb

HSM_StopCbType HSM_L1_RtCtxType_tag::pStopCb

HSM_DrvAesDecryptInfType HSM_L1_RtCtxType_tag::tAesDecryptInf

HSM_DrvAesEncryptInfType HSM_L1_RtCtxType_tag::tAesEncryptInf

HSM_DrvBnCalcExtInfType HSM_L1_RtCtxType_tag::tBnCalcInf

HSM_DrvCMacInfType HSM_L1_RtCtxType_tag::tCMacInf

HSM_CmdType HSM_L1_RtCtxType_tag::tCmd

HSM_DrvEccCalcInfType HSM_L1_RtCtxType_tag::tEccCalc

HSM_DrvEccDecryptInfType HSM_L1_RtCtxType_tag::tEccDecryptInf

HSM_DrvEccSignInfType HSM_L1_RtCtxType_tag::tEccSignInf

HSM_DrvEccVerifyInfType HSM_L1_RtCtxType_tag::tEccVerifyInf

HSM_DrvKeyManagelnfType HSM_L1_RtCtxType_tag::tKeyInf

HSM_DrvKeySpaceInfType HSM_L1_RtCtxType_tag::tKeySpaceInf

union { ... } HSM_L1_RtCtxType_tag::tL1AlgCtx

HSM_DrvLoadFirmwareInfType HSM_L1_RtCtxType_tag::tLoadFirmware

HSM_DrvMd5InfType HSM_L1_RtCtxType_tag::tMd5Inf

HSM_DrvRandPoolType HSM_L1_RtCtxType_tag::tRandPool

HSM_DrvTrngInfType HSM_L1_RtCtxType_tag::tRngInf

HSM_L1_DrvRSASsaPkcs1V15SignType HSM_L1_RtCtxType_tag::tRsaSsaPkcs1v15Sign

HSM_L1_DrvRSASsaPkcs1V15VerifyType HSM_L1_RtCtxType_tag::tRsaSsaPkcs1V15Verify

HSM_L1_DrvRSASsaPssSignType HSM_L1_RtCtxType_tag::tRsaSsaPssSign

HSM_L1_DrvRSASsaPssVerifyType HSM_L1_RtCtxType_tag::tRsaSsaPssVerify

HSM_DrvScatterCMacInfType HSM_L1_RtCtxType_tag::tScatterCMac

HSM_DrvScatterHashInfType HSM_L1_RtCtxType_tag::tScatterHash

HSM_DrvShalnfType HSM_L1_RtCtxType_tag::tShalnf

HSM_DrvSm2GenZaInfType HSM_L1_RtCtxType_tag::tSm2GenZaInf

HSM_DrvSm2VerifyInfType HSM_L1_RtCtxType_tag::tSm2VerifyInf

HSM_DrvSm3InfType HSM_L1_RtCtxType_tag::tSm3Inf

HSM_DrvSm4DecryptInfType HSM_L1_RtCtxType_tag::tSm4DecryptInf

HSM_DrvSm4EncryptInfType HSM_L1_RtCtxType_tag::tSm4EncryptInf

hsm_uint32_t HSM_L1_RtCtxType_tag::u32CurStatInf

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_L1_Sm2VerifyType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
HSM_BoolType bUseDefaultID
hsm_uint32_t u32PublicKeyId
const hsm_uint8_t * pkGx
hsm_uint32_t u32kGxByteCnt
const hsm_uint8_t * pkGy
hsm_uint32_t u32kGyByteCnt
HSM_DataFormatType ekGxyDataFmt
const hsm_uint8_t * pInputData_ID
hsm_uint32_t u32SM2InputIDByteCnt
const hsm_uint8_t * pData
hsm_uint32_t u32DataByteCnt
HSM_DataFormatType eDataFmt
const hsm_uint8_t * pR
hsm_uint32_t u32RByteCnt
const hsm_uint8_t * pS
hsm_uint32_t u32SByteCnt
HSM_DataFormatType eRSDataFmt
```

Member Data Documentation

HSM_BoolType HSM_L1_Sm2VerifyType::bUseDefaultID

true means use default ID. false means use specific ID with "pInputData_ID" and "u32SM2InputIDByteCnt" params.

HSM_DataFormatType HSM_L1_Sm2VerifyType::eDataFmt

HSM_DataFormatType HSM_L1_Sm2VerifyType::ekGxyDataFmt

HSM_DataFormatType HSM_L1_Sm2VerifyType::eRSDataFmt

const hsm_uint8_t* HSM_L1_Sm2VerifyType::pData

address is 4bytes aligned, if input hash, hash of the data to verify, if raw data to hash, it point to raw data

const hsm_uint8_t* HSM_L1_Sm2VerifyType::pInputData_ID

const hsm_uint8_t* HSM_L1_Sm2VerifyType::pkGx

address is 4bytes aligned, public key axis x

const hsm_uint8_t* HSM_L1_Sm2VerifyType::pkGy

address is 4bytes aligned, public key axis y

const hsm_uint8_t* HSM_L1_Sm2VerifyType::pR

address is 4bytes aligned, the sign result R of the data to verify

const hsm_uint8_t* HSM_L1_Sm2VerifyType::pS

address is 4bytes aligned, the sign result S of the data to verify

hsm_uint32_t HSM_L1_Sm2VerifyType::u32DataByteCnt

hsm_uint32_t HSM_L1_Sm2VerifyType::u32kGxByteCnt

hsm_uint32_t HSM_L1_Sm2VerifyType::u32kGyByteCnt

hsm_uint32_t HSM_L1_Sm2VerifyType::u32PublicKeyId

hsm_uint32_t HSM_L1_Sm2VerifyType::u32RByteCnt

hsm_uint32_t HSM_L1_Sm2VerifyType::u32SByteCnt

hsm_uint32_t HSM_L1_Sm2VerifyType::u32SM2InputIDByteCnt

the length should be <= 32bytes note: If "bUseDefaultID" is false, would use this specific ID length with byte unit.

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_L2_DrvCMacEmptyInfType Struct Reference

The structure of the HSM level 2 processing handle.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L2_DrvCMacEmptyInfType:

IMAGE

Public Attributes

HSM_L2_DrvCMacEmptySMType eStat

hsm_uint8_t * pOut

hsm_uint32_t u32OutBufSize

HSM_DataFormatType eOutFmt

hsm_uint32_t * pOutSize

HSM_Ln_AesEncryptType tParm

HSM_AesmBackendType eBackend

hsm_uint32_t aResultCacheBuf [4+1]

hsm_uint32_t aInputCacheBuf [4+1]

hsm_uint32_t alvData [4+1]

hsm_uint32_t aKeyData [8+1]

Detailed Description

The structure of the HSM level 2 processing handle.

Member Data Documentation

hsm_uint32_t HSM_L2_DrvCMacEmptyInfType::aInputCacheBuf[4+1]

hsm_uint32_t HSM_L2_DrvCMacEmptyInfType::aIvData[4+1]

hsm_uint32_t HSM_L2_DrvCMacEmptyInfType::aKeyData[8+1]

hsm_uint32_t HSM_L2_DrvCMacEmptyInfType::aResultCacheBuf[4+1]

pkcs7 result size has more 16bytes than input

HSM_AesmBackendType HSM_L2_DrvCMacEmptyInfType::eBackend

HSM_DataFormatType HSM_L2_DrvCMacEmptyInfType::eOutFmt

HSM_L2_DrvCMacEmptySMTType HSM_L2_DrvCMacEmptyInfType::eStat

hsm_uint8_t* HSM_L2_DrvCMacEmptyInfType::pOut

hsm_uint32_t* HSM_L2_DrvCMacEmptyInfType::pOutSize

HSM_Ln_AesEncryptType HSM_L2_DrvCMacEmptyInfType::tParm

hsm_uint32_t HSM_L2_DrvCMacEmptyInfType::u32OutBufSize

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L2_DrvEccVerifyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L2_DrvEccVerifyInfType:

IMAGE

Public Attributes

hsm_uint32_t u32BitCnt
HSM_L2_DrvEccVerifySMType eStat
hsm_uint32_t aCache [17]
HSM_Ln_EccVerifyType tEccCfg
HSM_Ln_ShaType tShaParm

Member Data Documentation

hsm_uint32_t HSM_L2_DrvEccVerifyInfType::aCache[17]

HSM_L2_DrvEccVerifySMType HSM_L2_DrvEccVerifyInfType::eStat

HSM_Ln_EccVerifyType HSM_L2_DrvEccVerifyInfType::tEccCfg

HSM_Ln_ShaType HSM_L2_DrvEccVerifyInfType::tShaParm

hsm_uint32_t HSM_L2_DrvEccVerifyInfType::u32BitCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L2_DrvRevokeKeyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L2_DrvRevokeKeyInfType:

IMAGE

Public Attributes

```
HSM_L2_DrvRevokeKeySMType eStat
hsm_uint32_t u32Id
union {
    HSM_Ln_KeySpaceStatusType tKeySpaceStatus
};
```

Member Data Documentation

union { ... }

HSM_L2_DrvRevokeKeySMType HSM_L2_DrvRevokeKeyInfType::eStat

HSM_Ln_KeySpaceStatusType HSM_L2_DrvRevokeKeyInfType::tKeySpaceStatus

hsm_uint32_t HSM_L2_DrvRevokeKeyInfType::u32Id

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L2_DrvSm2VerifyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L2_DrvSm2VerifyInfType:

IMAGE

Public Attributes

HSM_L2_DrvSm2VerifySMType eStat
HSM_Ln_Sm2VerifyType tSm2Cfg
HSM_LN_Sm2GenZaType tSm2GenZaCfg
HSM_Ln_ScatterHashType tScatterSm3Cfg
hsm_uint32_t aCache [8]
HSM_Ln_ResultBufInfoType tScatterSm3Result

Member Data Documentation

hsm_uint32_t HSM_L2_DrvSm2VerifyInfType::aCache[8]

HSM_L2_DrvSm2VerifySMType HSM_L2_DrvSm2VerifyInfType::eStat

HSM_Ln_ScatterHashType HSM_L2_DrvSm2VerifyInfType::tScatterSm3Cfg

HSM_Ln_ResultBufInfoType HSM_L2_DrvSm2VerifyInfType::tScatterSm3Result

HSM_Ln_Sm2VerifyType HSM_L2_DrvSm2VerifyInfType::tSm2Cfg

HSM_LN_Sm2GenZaType HSM_L2_DrvSm2VerifyInfType::tSm2GenZaCfg

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L2_DrvUpdateKeyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L2_DrvUpdateKeyInfType:

IMAGE

Public Attributes

HSM_L2_DrvUpdateKeySMType eStat

hsm_uint32_t u32Id

hsm_uint32_t * pKeyIdOut

HSM_BoolType bTidyUpAlready

HSM_Ln_ImportUserKeyExtType tImportCfg

HSM_Ln_KeySpaceStatusType tKeySpaceStatus

Member Data Documentation

HSM_BoolType HSM_L2_DrvUpdateKeyInfType::bTidyUpAlready

HSM_L2_DrvUpdateKeySMType HSM_L2_DrvUpdateKeyInfType::eStat

hsm_uint32_t* HSM_L2_DrvUpdateKeyInfType::pKeyIdOut

HSM_Ln_ImportUserKeyExtType HSM_L2_DrvUpdateKeyInfType::tImportCfg

HSM_Ln_KeySpaceStatusType HSM_L2_DrvUpdateKeyInfType::tKeySpaceStatus

hsm_uint32_t HSM_L2_DrvUpdateKeyInfType::u32Id

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L2_InitParmType Struct Reference

Ecc level 2 init type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

HSM_StopCbType pStopCb

Detailed Description

Ecc level 2 init type.

Member Data Documentation

HSM_StopCbType HSM_L2_InitParmType::pStopCb

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_L2_RtCtxType_tag Struct Reference

The structure of the HSM level 2 processing handle.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L2_RtCtxType_tag:

IMAGE

Public Attributes

HSM_L2_DrvOpType eOperation

HSM_StopCbType pStopCb

union {

 HSM_L2_DrvCMacEmptyInfType tCMacEmpty

 HSM_L2_DrvUpdateKeyInfType tUpdateKey

 HSM_L2_DrvRevokeKeyInfType tRevokeKey

 HSM_L2_DrvEccVerifyInfType tEccVerify

 HSM_L2_DrvSm2VerifyInfType tSm2Verify

} tL2AlgCtx

Detailed Description

The structure of the HSM level 2 processing handle.

Member Data Documentation

HSM_L2_DrvOpType HSM_L2_RtCtxType_tag::eOperation

HSM_StopCbType HSM_L2_RtCtxType_tag::pStopCb

HSM_L2_DrvCMacEmptyInfType HSM_L2_RtCtxType_tag::tCMacEmpty

HSM_L2_DrvEccVerifyInfType HSM_L2_RtCtxType_tag::tEccVerify

union { ... } HSM_L2_RtCtxType_tag::tL2AlgCtx

HSM_L2_DrvRevokeKeyInfType HSM_L2_RtCtxType_tag::tRevokeKey

HSM_L2_DrvSm2VerifyInfType HSM_L2_RtCtxType_tag::tSm2Verify

HSM_L2_DrvUpdateKeyInfType HSM_L2_RtCtxType_tag::tUpdateKey

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L3_DrvScatterCMacInfType Struct Reference

The structure of the level 3 scatter cmac information type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L3_DrvScatterCMacInfType:

IMAGE

Public Attributes

hsm_uint32_t u32TimeoutMs

HSM_L3_ScatterCMacSMTType eState

HSM_Ln_CMacCfgParmsType tCfgParm

hsm_uint32_t u32InputTotalSize

Detailed Description

The structure of the level 3 scatter cmac information type.

Member Data Documentation

HSM_L3_ScatterCMacSMTType HSM_L3_DrvScatterCMacInfType::eState

HSM_Ln_CMacCfgParmsType HSM_L3_DrvScatterCMacInfType::tCfgParm

hsm_uint32_t HSM_L3_DrvScatterCMacInfType::u32InputTotalSize

hsm_uint32_t HSM_L3_DrvScatterCMacInfType::u32TimeoutMs

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L3_DrvUpdateKeyInfType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Public Attributes

hsm_uint32_t u32TimeoutMs
HSM_L3_UpdateKeySMType eState
hsm_uint32_t aKeyDataCache [129]

Member Data Documentation

hsm_uint32_t HSM_L3_DrvUpdateKeyInfType::aKeyDataCache[129]

HSM_L3_UpdateKeySMType HSM_L3_DrvUpdateKeyInfType::eState

hsm_uint32_t HSM_L3_DrvUpdateKeyInfType::u32TimeoutMs

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_L3_InitParmType Struct Reference

Ecc level 3 init type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

HSM_StopCbType pStopCb

Detailed Description

Ecc level 3 init type.

Member Data Documentation

HSM_StopCbType HSM_L3_InitParmType::pStopCb

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_L3_RtCtxType_tag Struct Reference

The structure of the HSM level 3 processing handle.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_L3_RtCtxType_tag:

IMAGE

Public Attributes

HSM_L3_DrvOpType eOperation

HSM_StopCbType pStopCb

union {

 HSM_L3_DrvScatterCMacInfType tScatterCMac

 HSM_L3_DrvUpdateKeyInfType tUpdateKey

} tL3AlgCtx

Detailed Description

The structure of the HSM level 3 processing handle.

Member Data Documentation

HSM_L3_DrvOpType HSM_L3_RtCtxType_tag::eOperation

HSM_StopCbType HSM_L3_RtCtxType_tag::pStopCb

union { ... } HSM_L3_RtCtxType_tag::tL3AlgCtx

HSM_L3_DrvScatterCMacInfType HSM_L3_RtCtxType_tag::tScatterCMac

HSM_L3_DrvUpdateKeyInfType HSM_L3_RtCtxType_tag::tUpdateKey

The documentation for this struct was generated from the following file:

`module_driver_hsm_level1_leveln.h`

HSM_Ln_AesDecryptType Struct Reference

The structure of the aes decrypt type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_AesDecryptType:

IMAGE

Public Attributes

```
hsm_uint32_t u32KeyId
HSM_AesKeyType eUseKeyType
HSM_AesNewKeyInfType tNewKeyInf
HSM_AesVendorKeyInfType tVendorKeyInf
HSM_SymmEnDecryptAlgType eAesAlg
const hsm_uint8_t * pDataInput
hsm_uint32_t u32InputByteCnt
hsm_uint8_t * pDataOutput
hsm_uint32_t u32OutputMemSize
const hsm_uint8_t * plvData
HSM_FlexAesPadType ePad
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
hsm_uint32_t * pGenerateOutByteCnt
```

Detailed Description

The structure of the aes decrypt type.

Member Data Documentation

HSM_SymmEnDecryptAlgType HSM_Ln_AesDecryptType::eAesAlg

encrypt/decrypt mode

HSM_DataFormatType HSM_Ln_AesDecryptType::eInputFmt

key/iv/input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_DataFormatType HSM_Ln_AesDecryptType::eOutputFmt

output result data format

HSM_FlexAesPadType HSM_Ln_AesDecryptType::ePad

for encrypt, if data size not 16bytes aligned, the pad byte should refer to this configuration, for decrypt, driver to process the padding data according to this, however, if zero padding, driver will not remove the padding 0, user should delete the data manually

HSM_AesKeyType HSM_Ln_AesDecryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

const hsm_uint8_t* HSM_Ln_AesDecryptType::pDataInput

address should align with 4bytes

hsm_uint8_t* HSM_Ln_AesDecryptType::pDataOutput

address should align with 4bytes

hsm_uint32_t* HSM_Ln_AesDecryptType::pGenerateOutByteCnt

point to the hsm_uint32_t variable to store the result byte count, can't be NULL_PTR

const hsm_uint8_t* HSM_Ln_AesDecryptType::pIvData

address should align with 4bytes, 128bit(16Bytes) iv(initialization vector) data array, ECB not need configure this

HSM_AesNewKeyInfType HSM_Ln_AesDecryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_Ln_AesDecryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_Ln_AesDecryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_Ln_AesDecryptType::u32KeyId

hsm_uint32_t HSM_Ln_AesDecryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_AesEncryptType Struct Reference

The structure of the aes encrypt type.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_AesEncryptType:

IMAGE

Public Attributes

```
hsm_uint32_t u32KeyId
HSM_AesKeyType eUseKeyType
HSM_AesNewKeyInfType tNewKeyInf
HSM_AesVendorKeyInfType tVendorKeyInf
HSM_SymmEnDecryptAlgType eAesAlg
const hsm_uint8_t * pDataInput
hsm_uint32_t u32InputByteCnt
hsm_uint8_t * pDataOutput
hsm_uint32_t u32OutputMemSize
const hsm_uint8_t * plvData
HSM_FlexAesPadType ePad
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
hsm_uint32_t * pGenerateOutByteCnt
```

Detailed Description

The structure of the aes encrypt type.

Member Data Documentation

HSM_SymmEnDecryptAlgType HSM_Ln_AesEncryptType::eAesAlg

encrypt/decrypt mode

HSM_DataFormatType HSM_Ln_AesEncryptType::eInputFmt

key/iv/input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_DataFormatType HSM_Ln_AesEncryptType::eOutputFmt

output result data format

HSM_FlexAesPadType HSM_Ln_AesEncryptType::ePad

for encrypt, if data size not 16bytes aligned, the pad byte should refer to this configuration, for decrypt, driver to process the padding data according to this, however, if zero padding, driver will not remove the padding 0, user should delete the data manually

HSM_AesKeyType HSM_Ln_AesEncryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

const hsm_uint8_t* HSM_Ln_AesEncryptType::pDataInput

address should align with 4bytes

hsm_uint8_t* HSM_Ln_AesEncryptType::pDataOutput

address should align with 4bytes

hsm_uint32_t* HSM_Ln_AesEncryptType::pGenerateOutByteCnt

point to the hsm_uint8_t variable to store the result byte count, can't be NULL

const hsm_uint8_t* HSM_Ln_AesEncryptType::pIvData

address should align with 4bytes, 128bit(16Bytes) iv(initialization vector) data array, ECB not need configure this

HSM_AesNewKeyInfType HSM_Ln_AesEncryptType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_Ln_AesEncryptType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_Ln_AesEncryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_Ln_AesEncryptType::u32KeyId

hsm_uint32_t HSM_Ln_AesEncryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_CMacCfgParmsType Struct Reference

The structure of the level 3 cmac config type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

HSM_AesKeyType eUseKeyType

hsm_uint32_t u32KeyId

HSM_AesKeyBitCntType eKeyType

const hsm_uint8_t * pKeyAddr

HSM_DataFormatType eKeyDataFmt

HSM_AesmBackendType eBackend

Detailed Description

The structure of the level 3 cmac config type.

Member Data Documentation

HSM_AesmBackendType HSM_Ln_CMacCfgParmsType::eBackend

HSM_DataFormatType HSM_Ln_CMacCfgParmsType::eKeyDataFmt

HSM_AesKeyBitCntType HSM_Ln_CMacCfgParmsType::eKeyType

the key bit count, which determine the AES algorithm type

HSM_AesKeyType HSM_Ln_CMacCfgParmsType::eUseKeyType

const hsm_uint8_t* HSM_Ln_CMacCfgParmsType::pKeyAddr

hsm_uint32_t HSM_Ln_CMacCfgParmsType::u32KeyId

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_CMacType Struct Reference

CMAC information used by driver when call specific API.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_CMacType:

IMAGE

Public Attributes

hsm_uint32_t u32NewKeyId
HSM_AesKeyType eUseKeyType
HSM_AesNewKeyInfType tNewKeyInf
HSM_AesVendorKeyInfType tVendorKeyInf
hsm_uint32_t u32GenMacByteCnt
const hsm_uint8_t * pDataInput
hsm_uint32_t u32InputByteCnt
hsm_uint8_t * pDataOutput
hsm_uint32_t u32ResultBufSize
hsm_uint32_t * pResultSize
HSM_BoolType bCheckMacEn
const hsm_uint8_t * pCheckExternalMac
hsm_uint32_t u32CheckByteCnt
Crypto_VerifyResultType * pVerifyResult
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
HSM_XCMacEngineType eEngine

Detailed Description

CMAC information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Ln_CMacType::bCheckMacEn

if enable this check, user should place the data after the input data, hsm will check the generated data and it, if fail, hsm generate a interrupt, and if user get hw status, will get a error status

HSM_XCMacEngineType HSM_Ln_CMacType::eEngine

HSM_DataFormatType HSM_Ln_CMacType::eInputFmt

HSM_DataFormatType HSM_Ln_CMacType::eOutputFmt

HSM_AesKeyType HSM_Ln_CMacType::eUseKeyType

fix to HSM_AES_KEY_NEW

const hsm_uint8_t* HSM_Ln_CMacType::pCheckExternalMac

const hsm_uint8_t* HSM_Ln_CMacType::pDataInput

address should align with 4bytes

hsm_uint8_t* HSM_Ln_CMacType::pDataOutput

address should align with 4bytes

hsm_uint32_t* HSM_Ln_CMacType::pResultSize

Crypto_VerifyResultType* HSM_Ln_CMacType::pVerifyResult

HSM_AesNewKeyInfType HSM_Ln_CMacType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_Ln_CMacType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_Ln_CMacType::u32CheckByteCnt

hsm_uint32_t HSM_Ln_CMacType::u32GenMacByteCnt

XCMAC API generate MAC data, it's "u32GenMacByteCnt" size ICV data

hsm_uint32_t HSM_Ln_CMacType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_Ln_CMacType::u32NewKeyId

hsm_uint32_t HSM_Ln_CMacType::u32ResultBufSize

4bytes align, the output data buffer "pDataOutput" size, should >= "u32GenMacByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_CopyUserKeyType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32SrcKeyID  
hsm_uint32_t u32DestKeyID  
hsm_uint32_t * pDestKeyID  
HSM_KeyManagerBoolType bStoreInFlash  
hsm_uint32_t eUserKeyType
```

Member Data Documentation

HSM_KeyManagerBoolType HSM_Ln_CopyUserKeyType::bStoreInFlash

hsm_uint32_t HSM_Ln_CopyUserKeyType::eUserKeyType

hsm_uint32_t* HSM_Ln_CopyUserKeyType::pDestKeyID

hsm_uint32_t HSM_Ln_CopyUserKeyType::u32DestKeyID

hsm_uint32_t HSM_Ln_CopyUserKeyType::u32SrcKeyID

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_Ln_EccDecryptType Struct Reference

Ecc decrypt type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_Ln_EccDecryptType:

IMAGE

Public Attributes

HSM_DataFormatType eInputDataFmt

HSM_DataFormatType eOutputDataFmt

HSM_DrvEccCurvePrmIndexType u32EccCurve

HSM_L1_EccCurveParamType tCurve

hsm_uint32_t u32PublicKeyId

const **hsm_uint8_t** * pkGx

hsm_uint32_t u32kGxByteCnt

const **hsm_uint8_t** * pkGy

hsm_uint32_t u32kGyByteCnt

hsm_uint32_t u32PrivateKeyId

const **hsm_uint8_t** * pPrivateKey

hsm_uint32_t u32PrivateKeyByteCnt

const **hsm_uint8_t** * pEncryptedData

hsm_uint32_t u32EncryptedDataByteCnt

hsm_uint8_t * pPlainData

hsm_uint32_t u32PlainDataBufferByteCnt

hsm_uint32_t * pPlainResultByteCnt

Detailed Description

Ecc decrypt type.

Member Data Documentation

HSM_DataFormatType HSM_Ln_EccDecryptType::eInputDataFmt

HSM_DataFormatType HSM_Ln_EccDecryptType::eOutputDataFmt

const hsm_uint8_t* HSM_Ln_EccDecryptType::pEncryptedData

address is 4bytes aligned, the sign result S of the data to verify

const hsm_uint8_t* HSM_Ln_EccDecryptType::pkGx

address is 4bytes aligned, public key axis x

const hsm_uint8_t* HSM_Ln_EccDecryptType::pkGy

address is 4bytes aligned, public key axis y

hsm_uint8_t* HSM_Ln_EccDecryptType::pPlainData

address is 4bytes aligned, the sign result R of the data to verify

hsm_uint32_t* HSM_Ln_EccDecryptType::pPlainResultByteCnt

const hsm_uint8_t* HSM_Ln_EccDecryptType::pPrivateKey

address is 4bytes aligned, hash of the data to verify

HSM_L1_EccCurveParamType HSM_Ln_EccDecryptType::tCurve

ecc curve parameters

HSM_DrvEccCurvePrmIndexType HSM_Ln_EccDecryptType::u32EccCurve

hsm_uint32_t HSM_Ln_EccDecryptType::u32EncryptedDataByteCnt

hsm_uint32_t HSM_Ln_EccDecryptType::u32kGxByteCnt

hsm_uint32_t HSM_Ln_EccDecryptType::u32kGyByteCnt

hsm_uint32_t HSM_Ln_EccDecryptType::u32PlainDataBufferByteCnt

hsm_uint32_t HSM_Ln_EccDecryptType::u32PrivateKeyByteCnt

hsm_uint32_t HSM_Ln_EccDecryptType::u32PrivateKeyId

hsm_uint32_t HSM_Ln_EccDecryptType::u32PublicKeyId

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_EccKPTType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_EccKPTType:

IMAGE

Public Attributes

HSM_DataFormatType eInputDataFmt
HSM_DataFormatType eOutputDataFmt
HSM_DrvEccCurvePrmIndexType u32EccCurve
HSM_L1_EccCurveParamType tCurve
hsm_uint32_t u32PointId
const hsm_uint8_t * pPointX
hsm_uint32_t u32PointXByteCnt
const hsm_uint8_t * pPointY
hsm_uint32_t u32PointYByteCnt
hsm_uint32_t u32KId
const hsm_uint8_t * pK
hsm_uint32_t u32KByteCnt
hsm_uint8_t * pResultX
hsm_uint32_t u32ResultXBufferByteCnt
hsm_uint32_t * pResultXByteCnt
hsm_uint8_t * pResultY
hsm_uint32_t u32ResultYBufferByteCnt
hsm_uint32_t * pResultYByteCnt

Member Data Documentation

HSM_DataFormatType HSM_Ln_EccKPTType::eInputDataFmt

HSM_DataFormatType HSM_Ln_EccKPTType::eOutputDataFmt

const hsm_uint8_t* HSM_Ln_EccKPTType::pK

address is 4bytes aligned, hash of the data to verify

const hsm_uint8_t* HSM_Ln_EccKPTType::pPointX

address is 4bytes aligned, public key axis x

const hsm_uint8_t* HSM_Ln_EccKPTType::pPointY

address is 4bytes aligned, public key axis y

hsm_uint8_t* HSM_Ln_EccKPTType::pResultX

address is 4bytes aligned, the sign result R of the data to verify

hsm_uint32_t* HSM_Ln_EccKPTType::pResultXByteCnt

hsm_uint8_t* HSM_Ln_EccKPTType::pResultY

address is 4bytes aligned, the sign result R of the data to verify

hsm_uint32_t* HSM_Ln_EccKPTType::pResultYByteCnt

HSM_L1_EccCurveParamType HSM_Ln_EccKPTType::tCurve

ecc curve parameters

HSM_DrvEccCurvePrmIndexType HSM_Ln_EccKPTType::u32EccCurve

hsm_uint32_t HSM_Ln_EccKPTType::u32KByteCnt

hsm_uint32_t HSM_Ln_EccKPTType::u32KId

hsm_uint32_t HSM_Ln_EccKPTType::u32PointId

hsm_uint32_t HSM_Ln_EccKPTType::u32PointXByteCnt

hsm_uint32_t HSM_Ln_EccKPTType::u32PointYByteCnt

hsm_uint32_t HSM_Ln_EccKPTType::u32ResultXBufferByteCnt

hsm_uint32_t HSM_Ln_EccKPTType::u32ResultYBufferByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_EccSignType Struct Reference

Ecc sign config type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_Ln_EccSignType:

IMAGE

Public Attributes

```
HSM_DataFormatType eDataFmt
HSM_DrvEccCurvePrmIndexType u32EccCurve
HSM_L1_EccCurveParamType tCurve
hsm_uint32_t u32PrivateKeyId
const hsm_uint8_t * pPrivate
hsm_uint32_t u32PrivateByteCnt
const hsm_uint8_t * pHashData
hsm_uint32_t u32HashDataByteCnt
hsm_uint8_t * pR
hsm_uint32_t u32RBufByteCnt
hsm_uint8_t * pS
hsm_uint32_t u32SBufByteCnt
```

Detailed Description

Ecc sign config type.

Member Data Documentation

HSM_DataFormatType HSM_Ln_EccSignType::eDataFmt

const hsm_uint8_t* HSM_Ln_EccSignType::pHashData

address is 4bytes aligned, hash of the data to verify

const hsm_uint8_t* HSM_Ln_EccSignType::pPrivate

address is 4bytes aligned, public key axis x

hsm_uint8_t* HSM_Ln_EccSignType::pR

address is 4bytes aligned, the sign result R of the data to verify

hsm_uint8_t* HSM_Ln_EccSignType::pS

address is 4bytes aligned, the sign result S of the data to verify

HSM_L1_EccCurveParamType HSM_Ln_EccSignType::tCurve

ecc curve parameters

HSM_DrvEccCurvePrmIndexType HSM_Ln_EccSignType::u32EccCurve

hsm_uint32_t HSM_Ln_EccSignType::u32HashDataByteCnt

hsm_uint32_t HSM_Ln_EccSignType::u32PrivateByteCnt

hsm_uint32_t HSM_Ln_EccSignType::u32PrivateKeyId

hsm_uint32_t HSM_Ln_EccSignType::u32RBufByteCnt

hsm_uint32_t HSM_Ln_EccSignType::u32SBufByteCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_EccVerifyType Struct Reference

Ecc verify type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Collaboration diagram for HSM_Ln_EccVerifyType:

IMAGE

Public Attributes

HSM_EccVerifyInputType eInputType

HSM_ShaAlgType eAlg

Crypto_VerifyResultType * pVerifyResult

HSM_L1_EccVerifyType tVerifyParm

Detailed Description

Ecc verify type.

Member Data Documentation

HSM_ShaAlgType HSM_Ln_EccVerifyType::eAlg

HSM_EccVerifyInputType HSM_Ln_EccVerifyType::eInputType

Crypto_VerifyResultType* HSM_Ln_EccVerifyType::pVerifyResult

HSM_L1_EccVerifyType HSM_Ln_EccVerifyType::tVerifyParm

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_HashMacCfgType Struct Reference

Ln hash mac config type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

HSM_BoolType bGenerateMacEn

HSM_HfamMacType eMacType

union {

 const **hsm_uint8_t** * pKeyData

hsm_uint32_t u32KeyId

} tHashMacKeyInfo

hsm_uint32_t u32KeyByteCnt

hsm_uint32_t u32GenerateMacByteCnt

HSM_DataFormatType eKeyDataFmt

HSM_BoolType bKeyUseld

Detailed Description

Ln hash mac config type.

Member Data Documentation

HSM_BoolType HSM_Ln_HashMacCfgType::bGenerateMacEn

whether generate MAC

HSM_BoolType HSM_Ln_HashMacCfgType::bKeyUseld

HSM_DataFormatType HSM_Ln_HashMacCfgType::eKeyDataFmt

HSM_HfamMacType HSM_Ln_HashMacCfgType::eMacType

if enable generating MAC, this should configure

const hsm_uint8_t* HSM_Ln_HashMacCfgType::pKeyData

address is 4bytes aligned, key to generate MAC, if enable generating MAC, this should configure

union { ... } HSM_Ln_HashMacCfgType::tHashMacKeyInfo

hsm_uint32_t HSM_Ln_HashMacCfgType::u32GenerateMacByteCnt

the byte count of MAC data output

hsm_uint32_t HSM_Ln_HashMacCfgType::u32KeyByteCnt

the byte count of key data

hsm_uint32_t HSM_Ln_HashMacCfgType::u32KeyId

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_ImportUserKeyExtType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_KeyManagerBoolType bStoreInFlash  
HSM_KeyManagerKeyExportType eExportType  
const hsm_uint32_t * pDataAddr  
hsm_uint32_t u32DataLength  
hsm_uint32_t * pKeyID  
HSM_KeyManagerUserKeyEnDecryType eDecryType  
hsm_uint32_t u32DecryKeyID  
HSM_KeyManagerUserKeyType eUserKeyType  
HSM_KeyManagerEnDecryAlgType eAlgType  
hsm_uint32_t(* pIV)[4]  
hsm_uint32_t u32KeyIdFix  
hsm_uint32_t u32Protect
```

Member Data Documentation

HSM_KeyManagerBoolType HSM_Ln_ImportUserKeyExtType::bStoreInFlash

HSM_KeyManagerEnDecryAlgType HSM_Ln_ImportUserKeyExtType::eAlgType

HSM_KeyManagerUserKeyEnDecryType HSM_Ln_ImportUserKeyExtType::eDecryType

HSM_KeyManagerKeyExportType HSM_Ln_ImportUserKeyExtType::eExportType

HSM_KeyManagerUserKeyTypeType HSM_Ln_ImportUserKeyExtType::eUserKeyType

const hsm_uint32_t* HSM_Ln_ImportUserKeyExtType::pDataAddr

hsm_uint32_t(* HSM_Ln_ImportUserKeyExtType::pIV)[4]

hsm_uint32_t* HSM_Ln_ImportUserKeyExtType::pKeyID

hsm_uint32_t HSM_Ln_ImportUserKeyExtType::u32DataLength

hsm_uint32_t HSM_Ln_ImportUserKeyExtType::u32DecryKeyID

hsm_uint32_t HSM_Ln_ImportUserKeyExtType::u32KeyIdFix

hsm_uint32_t HSM_Ln_ImportUserKeyExtType::u32Protect

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Ln_InitParmType Struct Reference

init information for HSM_Ln_Init

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_InitParmType:

IMAGE

Public Attributes

- HSM_TriggerSrcType eTriggerSrc
- HSM_StopCbType pStopCb
- HSM_SendCommandCbType pSendCmdCb
- HSM_GetAckStatCbType pReadAckCb
- HSM_CancelCmdCbType pCancelCmdCb
- HSM_CleanDcacheCbType pCleanDCache
- HSM_CleanAndFlushDcacheCbType pCleanAndFlushDcache
- hsm_uint32_t u32OpLimitMs

Detailed Description

init information for HSM_Ln_Init

Member Data Documentation

HSM_TriggerSrcType HSM_Ln_InitParmType::eTriggerSrc

HSM_CancelCmdCbType HSM_Ln_InitParmType::pCancelCmdCb

HSM_CleanAndFlushDcacheCbType HSM_Ln_InitParmType::pCleanAndFlushDcache

HSM_CleanDcacheCbType HSM_Ln_InitParmType::pCleanDCache

HSM_GetAckStatCbType HSM_Ln_InitParmType::pReadAckCb

HSM_SendCommandCbType HSM_Ln_InitParmType::pSendCmdCb

HSM_StopCbType HSM_Ln_InitParmType::pStopCb

hsm_uint32_t HSM_Ln_InitParmType::u32OpLimitMs

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_KeySpaceStatusType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_KeyManagerBoolType bCriticalError
HSM_KeyManagerBoolType bKMLackKeyHeadSpace
HSM_KeyManagerBoolType bKMLackKeyDataSpace
hsm_uint32_t u32KeyHeadRestSpace
hsm_uint32_t u32KeyDataRestSpace
hsm_uint32_t u32InvalidKeyCounter
hsm_uint32_t u32KeyRamSpaceUsed
```

Member Data Documentation

HSM_KeyManagerBoolType HSM_Ln_KeySpaceStatusType::bCriticalError

HSM_KeyManagerBoolType HSM_Ln_KeySpaceStatusType::bKMLackKeyDataSpace

HSM_KeyManagerBoolType HSM_Ln_KeySpaceStatusType::bKMLackKeyHeadSpace

hsm_uint32_t HSM_Ln_KeySpaceStatusType::u32InvalidKeyCounter

hsm_uint32_t HSM_Ln_KeySpaceStatusType::u32KeyDataRestSpace

The unit is page

hsm_uint32_t HSM_Ln_KeySpaceStatusType::u32KeyHeadRestSpace

The unit is number

hsm_uint32_t HSM_Ln_KeySpaceStatusType::u32KeyRamSpaceUsed

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_Ln_Md5Type Struct Reference

MD5 information used by driver when call specific API.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_Md5Type:

IMAGE

Public Attributes

HSM_Md5CtxType tCfg

hsm_uint8_t * pResult

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pResultSize

HSM_DataFormatType eInputFmt

HSM_DataFormatType eOutputFmt

HSM_BoolType bKeyUseld

Detailed Description

MD5 information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Ln_Md5Type::bKeyUseld

HSM_DataFormatType HSM_Ln_Md5Type::eInputFmt

input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_DataFormatType HSM_Ln_Md5Type::eOutputFmt

output data organized format, hsm_uint8_t array, or hsm_uint32_t array

hsm_uint8_t* HSM_Ln_Md5Type::pResult

MD5 result is 16bytes, point to the memory that driver to store result

hsm_uint32_t* HSM_Ln_Md5Type::pResultSize

HSM_Md5CtxType HSM_Ln_Md5Type::tCfg

MD5 algorithm parameter set by user

hsm_uint32_t HSM_Ln_Md5Type::u32ResultBufSize

the size of buffer 'pResult' to store result

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_PlainKeyImport Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pKeyData  
HSM_DataFormatType eKeyDataFmt  
hsm_uint32_t u32KeyDataByteCnt  
hsm_uint32_t u32KeyId  
HSM_Ln_KeyDataStoreFmtType eKeyStoreFmt  
HSM_KeyManagerUserKeyType eKeyUsage  
HSM_KeyManagerKeyExportType eExportType
```

Member Data Documentation

HSM_KeyManagerKeyExportType HSM_Ln_PlainKeyImport::eExportType

HSM_DataFormatType HSM_Ln_PlainKeyImport::eKeyDataFmt

HSM_Ln_KeyDataStoreFmtType HSM_Ln_PlainKeyImport::eKeyStoreFmt

HSM_KeyManagerUserKeyType HSM_Ln_PlainKeyImport::eKeyUsage

const hsm_uint8_t* HSM_Ln_PlainKeyImport::pKeyData

hsm_uint32_t HSM_Ln_PlainKeyImport::u32KeyDataByteCnt

hsm_uint32_t HSM_Ln_PlainKeyImport::u32KeyId

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_ResultBufInfoType Struct Reference

HSM result buffer type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
hsm_uint8_t * pData
hsm_uint32_t u32DataBufSize
HSM_DataFormatType eDataFmt
hsm_uint32_t * pDataSize
```

Detailed Description

HSM result buffer type.

Member Data Documentation

HSM_DataFormatType HSM_Ln_ResultBufInfoType::eDataFmt

hsm_uint8_t* HSM_Ln_ResultBufInfoType::pData

hsm_uint32_t* HSM_Ln_ResultBufInfoType::pDataSize

hsm_uint32_t HSM_Ln_ResultBufInfoType::u32DataBufSize

The documentation for this struct was generated from the following file:

```
module_driver_hsm_level1_leveln.h
```

HSM_Ln_RngType Struct Reference

Rng information used by driver when call specific API.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

hsm_uint8_t * pResult

hsm_uint32_t u32ResultBufSize

HSM_RndSrcType eSrc

Detailed Description

Rng information used by driver when call specific API.

Member Data Documentation

HSM_RndSrcType HSM_Ln_RngType::eSrc

hsm_uint8_t* HSM_Ln_RngType::pResult

MD5 result is 16bytes, point to the memory that driver to store result

hsm_uint32_t HSM_Ln_RngType::u32ResultBufSize

the size of buffer 'pResult' to store result

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_RsaSsaPkcs1V15SignDataType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
hsm_uint32_t u32RsaBitCnt
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
HSM_HashAlgType eHashType
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
hsm_uint32_t u32PrivateKeyId
const hsm_uint8_t * pPrivateKey
hsm_uint32_t u32PrivateKeyByteCnt
hsm_uint32_t u32KeyId
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt
```

Member Data Documentation

HSM_HashAlgType HSM_Ln_RsaSsaPkcs1V15SignDataType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1V15SignDataType::eInputFmt

input data organized format, uint8_t array, or uint32_t array

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1V15SignDataType::eOutputFmt

output data organized format, uint8_t array, or uint32_t array

const hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignDataType::pInputData

input data

hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignDataType::pKeyN

input key N data for "A = (input data)^E mod N"

hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignDataType::pPrivateKey

private key, input key E data for "A = (input data)^E mod N"

hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignDataType::pResult

output buffer, the buffer size should >= "u32KeyNByteCount"

hsm_uint32_t* HSM_Ln_RsaSsaPkcs1V15SignDataType::pResultByteCnt

the result data byte count

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignDataType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignDataType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignDataType::u32KeyId

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignDataType::u32PrivateKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignDataType::u32PrivateKeyId

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignDataType::u32ResultBufByteCnt

the result buffer size

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignDataType::u32RsaBitCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_RsaSsaPkcs1V15SignHashType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
hsm_uint32_t u32RsaBitCnt
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
HSM_HashAlgType eHashType
const hsm_uint8_t * pHashData
hsm_uint32_t u32HashDataByteCount
hsm_uint32_t u32PrivateKeyId
const hsm_uint8_t * pPrivateKey
hsm_uint32_t u32PrivateKeyByteCnt
hsm_uint32_t u32KeyId
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt
```

Member Data Documentation

HSM_HashAlgType HSM_Ln_RsaSsaPkcs1V15SignHashType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1V15SignHashType::eInputFmt

input data organized format, uint8_t array, or uint32_t array

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1V15SignHashType::eOutputFmt

output data organized format, uint8_t array, or uint32_t array

const hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignHashType::pHashData

input data

hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignHashType::pKeyN

input key N data for "A = (input data)^E mod N"

hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignHashType::pPrivateKey

private key, input key E data for "A = (input data)^E mod N"

hsm_uint8_t* HSM_Ln_RsaSsaPkcs1V15SignHashType::pResult

output buffer, the buffer size should >= "u32KeyNByteCount"

hsm_uint32_t* HSM_Ln_RsaSsaPkcs1V15SignHashType::pResultByteCnt

the result data byte count

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignHashType::u32HashDataByteCount

the byte count of input data

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignHashType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignHashType::u32KeyId

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignHashType::u32PrivateKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignHashType::u32PrivateKeyId

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignHashType::u32ResultBufByteCnt

the result buffer size

hsm_uint32_t HSM_Ln_RsaSsaPkcs1V15SignHashType::u32RsaBitCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_RsaSsaPkcs1v15VerifyType Struct Reference

RSA PKCS1 V1.5 signature verify information used by driver when call specific API.

#include <module_driver_hsm_level1_leveln.h>

Public Attributes

hsm_uint32_t u32EmByteCnt
HSM_HashAlgType eHashType
HSM_DataFormatType eInputDataFmt
const hsm_uint8_t *pInputData
hsm_uint32_t u32InputDataByteCount
HSM_DataFormatType eSignDataFmt
const hsm_uint8_t *pSignData
hsm_uint32_t u32SignDataByteCount
hsm_uint32_t u32KeyEId
HSM_DataFormatType ePublicKeyFmt
const hsm_uint8_t *pPublicKey
hsm_uint32_t u32PublicKeyByteCnt
HSM_BoolType bUseInputDer
const hsm_uint8_t *pDer
hsm_uint32_t u32DerByteCount
hsm_uint32_t u32KeyNId
HSM_DataFormatType eKeyNFmt
const hsm_uint8_t *pKeyN
hsm_uint32_t u32KeyNByteCount
Crypto_VerifyResultType *pVerifyResult

Detailed Description

RSA PKCS1 V1.5 signature verify information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Ln_RsaSsaPkcs1v15VerifyType::bUseInputDer

set to HSM_TRUE when RFC8017 not support some hash algorithm, for example SM3 Hash type, DER encoding of the DigestInfo value, if use hsm internal data, just set it to HSM_FALSE

HSM_HashAlgType HSM_Ln_RsaSsaPkcs1v15VerifyType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1v15VerifyType::eInputDataFmt

input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1v15VerifyType::eKeyNFmt

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1v15VerifyType::ePublicKeyFmt

HSM_DataFormatType HSM_Ln_RsaSsaPkcs1v15VerifyType::eSignDataFmt

const hsm_uint8_t* HSM_Ln_RsaSsaPkcs1v15VerifyType::pDer

when bUseInputDer is HSM_TRUE, this field point to the DER encoding of the DigestInfo value, others, ignore

const hsm_uint8_t* HSM_Ln_RsaSsaPkcs1v15VerifyType::pInputData

input data

const hsm_uint8_t* HSM_Ln_RsaSsaPkcs1v15VerifyType::pKeyN

input key N data for $A = (\text{input data})^E \bmod N$

const hsm_uint8_t* HSM_Ln_RsaSsaPkcs1v15VerifyType::pPublicKey

public key, input key E data for $A = (\text{input data})^E \bmod N$

const hsm_uint8_t* HSM_Ln_RsaSsaPkcs1v15VerifyType::pSignData

input signature data

Crypto_VerifyResultType* HSM_Ln_RsaSsaPkcs1v15VerifyType::pVerifyResult

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32DerByteCount

when bUseInputDer is HSM_TRUE, this field represent the byte count of DER data, if bUseInputDer is HSM_FALSE, ignore this field

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32EmByteCnt

set it 0 to use default value. RFC8017 require set it to N'byte count, equal to u32KeyNByteCount. the em data byte count, itx8 MUST < N's bit count, should keep same with the signature generate configuration

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32KeyEId

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32KeyNId

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32PublicKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_Ln_RsaSsaPkcs1v15VerifyType::u32SignDataByteCount

the byte count of signature data, it must > 0

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_RsaSsaPssSignDataType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
hsm_uint32_t u32RsaBitCnt
HSM_BoolType bUsePseudoRand
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
HSM_HashAlgType eHashType
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
hsm_uint32_t u32PrivateKeyId
const hsm_uint8_t * pPrivateKey
hsm_uint32_t u32PrivateKeyByteCnt
hsm_uint32_t u32KeyId
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt
```

Member Data Documentation

HSM_BoolType HSM_Ln_RsaSsaPssSignDataType::bUsePseudoRand

HSM_HashAlgType HSM_Ln_RsaSsaPssSignDataType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_Ln_RsaSsaPssSignDataType::eInputFmt

input data organized format, uint8_t array, or uint32_t array

HSM_DataFormatType HSM_Ln_RsaSsaPssSignDataType::eOutputFmt

output data organized format, uint8_t array, or uint32_t array

const hsm_uint8_t* HSM_Ln_RsaSsaPssSignDataType::pInputData

input data

const hsm_uint8_t* HSM_Ln_RsaSsaPssSignDataType::pKeyN

input key N data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_Ln_RsaSsaPssSignDataType::pPrivateKey

private key, input key E data for " $A = (\text{input data})^E \bmod N$ "

hsm_uint8_t* HSM_Ln_RsaSsaPssSignDataType::pResult

output buffer, the buffer size should $\geq$ "u32KeyNByteCount"

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::pResultByteCnt

the result data byte count

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::u32KeyId

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::u32PrivateKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::u32PrivateKeyId

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::u32ResultBufByteCnt

the result buffer size

hsm_uint32_t HSM_Ln_RsaSsaPssSignDataType::u32RsaBitCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_RsaSsaPssSignHashType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
hsm_uint32_t u32RsaBitCnt
HSM_BoolType bUsePseudoRand
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
HSM_HashAlgType eHashType
const hsm_uint8_t * pHashData
hsm_uint32_t u32HashDataByteCount
hsm_uint32_t u32PrivateKeyId
const hsm_uint8_t * pPrivateKey
hsm_uint32_t u32PrivateKeyByteCnt
hsm_uint32_t u32KeyId
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt
```

Member Data Documentation

HSM_BoolType HSM_Ln_RsaSsaPssSignHashType::bUsePseudoRand

HSM_HashAlgType HSM_Ln_RsaSsaPssSignHashType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_Ln_RsaSsaPssSignHashType::eInputFmt

input data organized format, uint8_t array, or uint32_t array

HSM_DataFormatType HSM_Ln_RsaSsaPssSignHashType::eOutputFmt

output data organized format, uint8_t array, or uint32_t array

const hsm_uint8_t* HSM_Ln_RsaSsaPssSignHashType::pHashData

input data

const hsm_uint8_t* HSM_Ln_RsaSsaPssSignHashType::pKeyN

input key N data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_Ln_RsaSsaPssSignHashType::pPrivateKey

private key, input key E data for " $A = (\text{input data})^E \bmod N$ "

hsm_uint8_t* HSM_Ln_RsaSsaPssSignHashType::pResult

output buffer, the buffer size should $\geq$ "u32KeyNByteCount"

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::pResultByteCnt

the result data byte count

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::u32HashDataByteCount

the byte count of input data

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::u32KeyId

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::u32PrivateKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::u32PrivateKeyId

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::u32ResultBufByteCnt

the result buffer size

hsm_uint32_t HSM_Ln_RsaSsaPssSignHashType::u32RsaBitCnt

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_RsaSsaPssVerifyType Struct Reference

RSA PSS signature verify information used by driver when call specific API.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
hsm_uint32_t u32EmBitCnt
HSM_HashAlgType eMgfHashType
HSM_HashAlgType eHashType
hsm_uint32_t u32SaltByteCount
HSM_DataFormatType eInputDataFmt
HSM_RsaSsaInputType eInputType
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
HSM_DataFormatType eSignDataFmt
const hsm_uint8_t * pSignData
hsm_uint32_t u32SignDataByteCount
hsm_uint32_t u32KeyEId
HSM_DataFormatType ePublicKeyDataFmt
const hsm_uint8_t * pPublicKey
hsm_uint32_t u32PublicKeyByteCnt
hsm_uint32_t u32KeyNId
HSM_DataFormatType eKeyNDataFmt
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
Crypto_VerifyResultType * pVerifyResult
```

Detailed Description

RSA PSS signature verify information used by driver when call specific API.

Member Data Documentation

HSM_HashAlgType HSM_Ln_RsaSsaPssVerifyType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_Ln_RsaSsaPssVerifyType::eInputDataFmt

input data(raw message or hash data) organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_RsaSsaInputType HSM_Ln_RsaSsaPssVerifyType::eInputType

input data type, may raw input message, or the hash data of raw message

HSM_DataFormatType HSM_Ln_RsaSsaPssVerifyType::eKeyNDataFmt

HSM_HashAlgType HSM_Ln_RsaSsaPssVerifyType::eMgfHashType

the hash type used by signature

HSM_DataFormatType HSM_Ln_RsaSsaPssVerifyType::ePublicKeyDataFmt

HSM_DataFormatType HSM_Ln_RsaSsaPssVerifyType::eSignDataFmt

Signature/Public key/N input data organized format, hsm_uint8_t array, or hsm_uint32_t array

const hsm_uint8_t* HSM_Ln_RsaSsaPssVerifyType::pInputData

input data

const hsm_uint8_t* HSM_Ln_RsaSsaPssVerifyType::pKeyN

input key N data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_Ln_RsaSsaPssVerifyType::pPublicKey

public key, input key E data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_Ln_RsaSsaPssVerifyType::pSignData

input signature data

Crypto_VerifyResultType* HSM_Ln_RsaSsaPssVerifyType::pVerifyResult

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32EmBitCnt

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32InputDataByteCount

the byte count of input data, when the eInputType is HSM_RSA_SSA_INPUT_RAW_MESSAGE

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32KeyEId

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32KeyNByteCount

key N data byte count

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32KeyNId

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32PublicKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32SaltByteCount

the byte count of salt data, it must > 0, suggest use hash length, for example, if eHashType is HSM_SHA_256, set this to 256/8=32bytes

hsm_uint32_t HSM_Ln_RsaSsaPssVerifyType::u32SignDataByteCount

the byte count of signature data, it must > 0

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_RtCtxType Struct Reference

The structure of the HSM processing handle.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_RtCtxType:

IMAGE

Public Attributes

```
hsm_uint32_t u32OpLimitMs
HSM_L1_RtCtxType tHsmL1Ctx
HSM_L2_RtCtxType tHsmL2Ctx
HSM_L3_RtCtxType tHsmL3Ctx
```

Detailed Description

The structure of the HSM processing handle.

Member Data Documentation

HSM_L1_RtCtxType HSM_Ln_RtCtxType::tHsmL1Ctx

HSM_L2_RtCtxType HSM_Ln_RtCtxType::tHsmL2Ctx

HSM_L3_RtCtxType HSM_Ln_RtCtxType::tHsmL3Ctx

hsm_uint32_t HSM_Ln_RtCtxType::u32OpLimitMs

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_ScatterCMacType Struct Reference

Scatter cmac information used by driver when call specific API.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pData  
hsm_uint32_t u32DataSize  
HSM_DataFormatType eDataFmt
```

Detailed Description

Scatter cmac information used by driver when call specific API.

Member Data Documentation

HSM_DataFormatType HSM_Ln_ScatterCMacType::eDataFmt

const hsm_uint8_t* HSM_Ln_ScatterCMacType::pData

hsm_uint32_t HSM_Ln_ScatterCMacType::u32DataSize

The documentation for this struct was generated from the following file:

```
module_driver_hsm_level1_leveln.h
```

HSM_Ln_ScatterHashType Struct Reference

Scatter hash level n type.

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
const hsm_uint8_t * pData  
hsm_uint32_t u32DataSize  
HSM_DataFormatType eDataFmt
```

Detailed Description

Scatter hash level n type.

Member Data Documentation

HSM_DataFormatType HSM_Ln_ScatterHashType::eDataFmt

const hsm_uint8_t* HSM_Ln_ScatterHashType::pData

hsm_uint32_t HSM_Ln_ScatterHashType::u32DataSize

The documentation for this struct was generated from the following file:

```
module_driver_hsm_level1_leveln.h
```

HSM_Ln_ShaType Struct Reference

Sha information used by driver when call specific API.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_ShaType:

IMAGE

Public Attributes

HSM_ShaAlgType eAlg

HSM_ShaCtxType tCfg

hsm_uint8_t * pResult

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pResultSize

HSM_DataFormatType eInputFmt

HSM_DataFormatType eOutputFmt

HSM_BoolType bKeyUseld

Detailed Description

Sha information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Ln_ShaType::bKeyUseld

HSM_ShaAlgType HSM_Ln_ShaType::eAlg

HSM_DataFormatType HSM_Ln_ShaType::eInputFmt

input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_DataFormatType HSM_Ln_ShaType::eOutputFmt

output data organized format, hsm_uint8_t array, or hsm_uint32_t array

hsm_uint8_t* HSM_Ln_ShaType::pResult

MD5 result is 16bytes, point to the memory that driver to store result

hsm_uint32_t* HSM_Ln_ShaType::pResultSize

HSM_ShaCtxType HSM_Ln_ShaType::tCfg

algorithm parameter set by user

hsm_uint32_t HSM_Ln_ShaType::u32ResultBufSize

the size of buffer 'pResult' to store result

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_LN_Sm2GenZaType Struct Reference

```
#include <module_driver_hsm_level1_leveln.h>
```

Public Attributes

```
HSM_BoolType bUseDefaultID
hsm_uint32_t u32PublicKeyId
const hsm_uint8_t * pkGx
hsm_uint32_t u32kGxByteCnt
const hsm_uint8_t * pkGy
hsm_uint32_t u32kGyByteCnt
HSM_DataFormatType ekGxyDataFmt
const hsm_uint8_t * pInputData_ID
hsm_uint32_t u32SM2InputIDByteCnt
HSM_DataFormatType eDataFmt
hsm_uint32_t(* pOutputData_Za )[8]
HSM_DataFormatType eOutputFmt
```

Member Data Documentation

HSM_BoolType HSM_LN_Sm2GenZaType::bUseDefaultID

true means use default ID. false means use specific ID with "pInputData_ID" and "u32SM2InputIDByteCnt" params.

HSM_DataFormatType HSM_LN_Sm2GenZaType::eDataFmt

HSM_DataFormatType HSM_LN_Sm2GenZaType::ekGxyDataFmt

HSM_DataFormatType HSM_LN_Sm2GenZaType::eOutputFmt

const hsm_uint8_t* HSM_LN_Sm2GenZaType::pInputData_ID

const hsm_uint8_t* HSM_LN_Sm2GenZaType::pkGx

address is 4bytes aligned, public key axis x

const hsm_uint8_t* HSM_LN_Sm2GenZaType::pkGy

address is 4bytes aligned, public key axis y

hsm_uint32_t(* HSM_LN_Sm2GenZaType::pOutputData_Za)[8]

hsm_uint32_t HSM_LN_Sm2GenZaType::u32kGxByteCnt

hsm_uint32_t HSM_LN_Sm2GenZaType::u32kGyByteCnt

hsm_uint32_t HSM_LN_Sm2GenZaType::u32PublicKeyId

hsm_uint32_t HSM_LN_Sm2GenZaType::u32SM2InputIDByteCnt

the length should be <= 32bytes note: If "bUseDefaultID" is false, would use this specific ID length with byte unit.

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_Sm2VerifyType Struct Reference

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_Sm2VerifyType:

IMAGE

Public Attributes

HSM_Sm2VerifyInputType eInputType

Crypto_VerifyResultType * pVerifyResult

HSM_L1_Sm2VerifyType tVerifyParm

Member Data Documentation

HSM_Sm2VerifyInputType HSM_Ln_Sm2VerifyType::eInputType

Crypto_VerifyResultType* HSM_Ln_Sm2VerifyType::pVerifyResult

HSM_L1_Sm2VerifyType HSM_Ln_Sm2VerifyType::tVerifyParm

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Ln_Sm3Type Struct Reference

Sm3 information used by driver when call specific API.

#include <module_driver_hsm_level1_leveln.h>

Collaboration diagram for HSM_Ln_Sm3Type:

IMAGE

Public Attributes

HSM_Sm3CtxType tCfg

hsm_uint8_t * pResult

hsm_uint32_t u32ResultBufSize

hsm_uint32_t * pResultSize

HSM_DataFormatType eInputFmt

HSM_DataFormatType eOutputFmt

HSM_BoolType bKeyUseld

Detailed Description

Sm3 information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Ln_Sm3Type::bKeyUseld

HSM_DataFormatType HSM_Ln_Sm3Type::eInputFmt

input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_DataFormatType HSM_Ln_Sm3Type::eOutputFmt

output data organized format, hsm_uint8_t array, or hsm_uint32_t array

hsm_uint8_t* HSM_Ln_Sm3Type::pResult

MD5 result is 16bytes, point to the memory that driver to store result

hsm_uint32_t* HSM_Ln_Sm3Type::pResultSize

HSM_Sm3CtxType HSM_Ln_Sm3Type::tCfg

algorithm parameter set by user

hsm_uint32_t HSM_Ln_Sm3Type::u32ResultBufSize

the size of buffer 'pResult' to store result

The documentation for this struct was generated from the following file:

module_driver_hsm_level1_leveln.h

HSM_Md5ExType Struct Reference

MD5 information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSM_Md5ExType:

IMAGE

Public Attributes

HSM_Md5CtxType tCfg

hsm_uint32_t(* pRet)[4]

HSM_DataFormatType eInputFmt

HSM_DataFormatType eOutputFmt

Detailed Description

MD5 information used by driver when call specific API.

Member Data Documentation

HSM_DataFormatType HSM_Md5ExType::eInputFmt

HSM_DataFormatType HSM_Md5ExType::eOutputFmt

hsm_uint32_t(* HSM_Md5ExType::pRet)[4]

address should align with 4bytes, point to the memory that driver to store result

HSM_Md5CtxType HSM_Md5ExType::tCfg

MD5 algorithm parameter set by user

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Md5Type Struct Reference

MD5 information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_Md5Type:

IMAGE

Public Attributes

```
HSM_Md5CtxType tCfg  
hsm_uint32_t(* pRet)[4]
```

Detailed Description

MD5 information used by driver when call specific API.

Member Data Documentation

hsm_uint32_t(* HSM_Md5Type::pRet)[4]

address should align with 4bytes, point to the memory that driver to store result

HSM_Md5CtxType HSM_Md5Type::tCfg

MD5 algorithm parameter set by user

The documentation for this struct was generated from the following file:

```
module_driver_hsm.h
```

HSM_RsaSsaPkcs1V15SignType Struct Reference

RSA PKCS1 V1.5 signature information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
hsm_uint32_t u32EmByteCnt
HSM_HashAlgType eHashType
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
const hsm_uint8_t * pPrivateKey
hsm_uint32_t u32PrivateKeyByteCnt
HSM_BoolType bUseInputDer
const hsm_uint8_t * pDer
hsm_uint32_t u32DerByteCount
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt
HSM_RsaSsaInputType eInputType
```

Detailed Description

RSA PKCS1 V1.5 signature information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_RsaSsaPkcs1V15SignType::bUseInputDer

set to HSM_TRUE when RFC8017 not support some hash algorithm, for example SM3 Hash type, DER encoding of the DigestInfo value, if use hsm internal data, just set it to HSM_FALSE

HSM_HashAlgType HSM_RsaSsaPkcs1V15SignType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_RsaSsaPkcs1V15SignType::eInputFmt

input data organized format, uint8_t array, or uint32_t array

HSM_RsaSsaInputType HSM_RsaSsaPkcs1V15SignType::eInputType

input data type, may raw input message, or the hash data of raw message

HSM_DataFormatType HSM_RsaSsaPkcs1V15SignType::eOutputFmt

output data organized format, uint8_t array, or uint32_t array

const hsm_uint8_t* HSM_RsaSsaPkcs1V15SignType::pDer

when bUseInputDer is HSM_TRUE, this field point to the DER encoding of the DigestInfo value, others, ignore

const hsm_uint8_t* HSM_RsaSsaPkcs1V15SignType::pInputData

input data

const hsm_uint8_t* HSM_RsaSsaPkcs1V15SignType::pKeyN

input key N data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_RsaSsaPkcs1V15SignType::pPrivateKey

private key, input key E data for " $A = (\text{input data})^E \bmod N$ "

hsm_uint8_t* HSM_RsaSsaPkcs1V15SignType::pResult

output buffer, the buffer size should $\geq$ "u32KeyNByteCount"

hsm_uint32_t* HSM_RsaSsaPkcs1V15SignType::pResultByteCnt

the result data byte count

hsm_uint32_t HSM_RsaSsaPkcs1V15SignType::u32DerByteCount

when bUseInputDer is HSM_TRUE, this field represent the byte count of DER data, if bUseInputDer is HSM_FALSE, ignore this field

hsm_uint32_t HSM_RsaSsaPkcs1V15SignType::u32EmByteCnt

hsm_uint32_t HSM_RsaSsaPkcs1V15SignType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_RsaSsaPkcs1V15SignType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_RsaSsaPkcs1V15SignType::u32PrivateKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_RsaSsaPkcs1V15SignType::u32ResultBufByteCnt

the result buffer size

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_RsaSsaPkcs1V15VerifyType Struct Reference

RSA PKCS1V1.5 signature verify information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_DataFormatType eInputFmt
hsm_uint32_t u32EmByteCnt
HSM_HashAlgType eHashType
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
const hsm_uint8_t * pSignData
hsm_uint32_t u32SignDataByteCount
const hsm_uint8_t * pPublicKey
hsm_uint32_t u32PublicKeyByteCnt
HSM_BoolType bUseInputDer
const hsm_uint8_t * pDer
hsm_uint32_t u32DerByteCount
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
HSM_RsaSsaInputType eInputType
HSM_DataFormatType eSignDataFmt
HSM_DataFormatType ePublicKeyFmt
HSM_DataFormatType eKeyNFmt
```

Detailed Description

RSA PKCS1V1.5 signature verify information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_RsaSsaPkcs1V15VerifyType::bUseInputDer

set to HSM_TRUE when RFC8017 not support some hash algorithm, for example SM3 Hash type, DER encoding of the DigestInfo value, if use hsm internal data, just set it to HSM_FALSE

HSM_HashAlgType HSM_RsaSsaPkcs1V15VerifyType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_RsaSsaPkcs1V15VerifyType::eInputFmt

input data organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_RsaSsaInputType HSM_RsaSsaPkcs1V15VerifyType::eInputType

HSM_DataFormatType HSM_RsaSsaPkcs1V15VerifyType::eKeyNFmt

HSM_DataFormatType HSM_RsaSsaPkcs1V15VerifyType::ePublicKeyFmt

HSM_DataFormatType HSM_RsaSsaPkcs1V15VerifyType::eSignDataFmt

const hsm_uint8_t* HSM_RsaSsaPkcs1V15VerifyType::pDer

when bUseInputDer is HSM_TRUE, this field point to the DER encoding of the DigestInfo value, others, ignore

const hsm_uint8_t* HSM_RsaSsaPkcs1V15VerifyType::pInputData

input data

const hsm_uint8_t* HSM_RsaSsaPkcs1V15VerifyType::pKeyN

input key N data for $A = (\text{input data})^E \bmod N$

const hsm_uint8_t* HSM_RsaSsaPkcs1V15VerifyType::pPublicKey

public key, input key E data for $A = (\text{input data})^E \bmod N$

const hsm_uint8_t* HSM_RsaSsaPkcs1V15VerifyType::pSignData

input signature data

hsm_uint32_t HSM_RsaSsaPkcs1V15VerifyType::u32DerByteCount

when bUseInputDer is HSM_TRUE, this field represent the byte count of DER data, if bUseInputDer is HSM_FALSE, ignore this field

hsm_uint32_t HSM_RsaSsaPkcs1V15VerifyType::u32EmByteCnt

set it 0 to use default value. RFC8017 require set it to N'byte count, equal to u32KeyNByteCount. the em data byte count, itx8 MUST < N's bit count, should keep same with the signature generate configuration

hsm_uint32_t HSM_RsaSsaPkcs1V15VerifyType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_RsaSsaPkcs1V15VerifyType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_RsaSsaPkcs1V15VerifyType::u32PublicKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_RsaSsaPkcs1V15VerifyType::u32SignDataByteCount

the byte count of signature data, it must > 0

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_RsaSsaPssSignType Struct Reference

#include <module_driver_hsm.h>

Public Attributes

HSM_BoolType bUsePseudoRand
HSM_HashAlgType eMgfHashType
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
hsm_uint32_t u32EmBitCnt
HSM_HashAlgType eHashType
HSM_RsaSsaInputType eInputType
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
const hsm_uint8_t * pPrivateKey
hsm_uint32_t u32PrivateKeyByteCnt
HSM_BoolType bUseInputSaltData
const hsm_uint8_t * pSalt
hsm_uint32_t u32SaltByteCount
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
hsm_uint8_t * pResult
hsm_uint32_t u32ResultBufByteCnt
hsm_uint32_t * pResultByteCnt

Member Data Documentation

HSM_BoolType HSM_RsaSsaPssSignType::bUseInputSaltData

use input salt data or not, , suggest set to HSM_FALSE to use internal random data

HSM_BoolType HSM_RsaSsaPssSignType::bUsePseudoRand

HSM_HashAlgType HSM_RsaSsaPssSignType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_RsaSsaPssSignType::eInputFmt

input data organized format, uint8_t array, or uint32_t array

HSM_RsaSsaInputType HSM_RsaSsaPssSignType::eInputType

input data type, may raw input message, or the hash data of raw message

HSM_HashAlgType HSM_RsaSsaPssSignType::eMgfHashType

the hash type used by MGF1

HSM_DataFormatType HSM_RsaSsaPssSignType::eOutputFmt

output data organized format, uint8_t array, or uint32_t array

const hsm_uint8_t* HSM_RsaSsaPssSignType::pInputData

input data

const hsm_uint8_t* HSM_RsaSsaPssSignType::pKeyN

input key N data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_RsaSsaPssSignType::pPrivateKey

private key, input key E data for " $A = (\text{input data})^E \bmod N$ "

hsm_uint8_t* HSM_RsaSsaPssSignType::pResult

output buffer, the buffer size should $\geq$ "u32KeyNByteCount"

hsm_uint32_t* HSM_RsaSsaPssSignType::pResultByteCnt

the result data byte count

const hsm_uint8_t* HSM_RsaSsaPssSignType::pSalt

input salt data, , suggest set to NULL. if use hsm internal random data, just set it to NULL

hsm_uint32_t HSM_RsaSsaPssSignType::u32EmBitCnt

hsm_uint32_t HSM_RsaSsaPssSignType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_RsaSsaPssSignType::u32KeyNByteCount

key N's data byte count

hsm_uint32_t HSM_RsaSsaPssSignType::u32PrivateKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_RsaSsaPssSignType::u32ResultBufByteCnt

the result buffer size

hsm_uint32_t HSM_RsaSsaPssSignType::u32SaltByteCount

the byte count of salt data, it must > 0 , suggest use hash length, for example, if eHashType is HSM_SHA_256, set this to $256/8=32$ bytes

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_RsaSsaPssVerifyType Struct Reference

RSA PSS signature verify information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32EmBitCnt
HSM_HashAlgType eMgfHashType
HSM_HashAlgType eHashType
hsm_uint32_t u32SaltByteCount
HSM_DataFormatType eInputDataFmt
HSM_RsaSsaInputType eInputType
const hsm_uint8_t * pInputData
hsm_uint32_t u32InputDataByteCount
HSM_DataFormatType eSignDataFmt
const hsm_uint8_t * pSignData
hsm_uint32_t u32SignDataByteCount
HSM_DataFormatType ePublicKeyDataFmt
const hsm_uint8_t * pPublicKey
hsm_uint32_t u32PublicKeyByteCnt
HSM_DataFormatType eKeyNDataFmt
const hsm_uint8_t * pKeyN
hsm_uint32_t u32KeyNByteCount
```

Detailed Description

RSA PSS signature verify information used by driver when call specific API.

Member Data Documentation

HSM_HashAlgType HSM_RsaSsaPssVerifyType::eHashType

the hash type used by signature

HSM_DataFormatType HSM_RsaSsaPssVerifyType::eInputDataFmt

input data(raw message or hash data) organized format, hsm_uint8_t array, or hsm_uint32_t array

HSM_RsaSsaInputType HSM_RsaSsaPssVerifyType::eInputType

input data type, may raw input message, or the hash data of raw message

HSM_DataFormatType HSM_RsaSsaPssVerifyType::eKeyNDataFmt

HSM_HashAlgType HSM_RsaSsaPssVerifyType::eMgfHashType

the hash type used by signature

HSM_DataFormatType HSM_RsaSsaPssVerifyType::ePublicKeyDataFmt

HSM_DataFormatType HSM_RsaSsaPssVerifyType::eSignDataFmt

Signature/Public key/N input data organized format, hsm_uint8_t array, or hsm_uint32_t array

const hsm_uint8_t* HSM_RsaSsaPssVerifyType::pInputData

input data

const hsm_uint8_t* HSM_RsaSsaPssVerifyType::pKeyN

input key N data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_RsaSsaPssVerifyType::pPublicKey

public key, input key E data for " $A = (\text{input data})^E \bmod N$ "

const hsm_uint8_t* HSM_RsaSsaPssVerifyType::pSignData

input signature data

hsm_uint32_t HSM_RsaSsaPssVerifyType::u32EmBitCnt

hsm_uint32_t HSM_RsaSsaPssVerifyType::u32InputDataByteCount

the byte count of input data, when the eInputType is HSM_RSA_SSA_INPUT_RAW_MESSAGE

hsm_uint32_t HSM_RsaSsaPssVerifyType::u32KeyNByteCount

key N data byte count

hsm_uint32_t HSM_RsaSsaPssVerifyType::u32PublicKeyByteCnt

the byte count of key E

hsm_uint32_t HSM_RsaSsaPssVerifyType::u32SaltByteCount

the byte count of salt data, it must > 0, suggest use hash length, for example, if eHashType is HSM_SHA_256, set this to 256/8=32bytes

hsm_uint32_t HSM_RsaSsaPssVerifyType::u32SignDataByteCount

the byte count of signature data, it must > 0

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_RsaType Struct Reference

RSA information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
const hsm_uint32_t * pInputData
hsm_uint32_t u32InputDataByteCount
const hsm_uint32_t * pKey_E
hsm_uint32_t u32Key_E_ByteCount
const hsm_uint32_t * pKey_N
hsm_uint32_t u32Key_N_ByteCount
hsm_uint32_t * pResult
hsm_uint32_t u32ResultBufByteCnt
```

Detailed Description

RSA information used by driver when call specific API.

Member Data Documentation

const hsm_uint32_t* HSM_RsaType::pInputData

address is 4bytes aligned, input data, if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint32_t* HSM_RsaType::pKey_E

address is 4bytes aligned, input key E data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

const hsm_uint32_t* HSM_RsaType::pKey_N

address is 4bytes aligned, input key N data for " $A = (\text{input data})^E \bmod N$ ", if not 64bytes align, the last uint64_t's high byte left to patch 0 to align

hsm_uint32_t* HSM_RsaType::pResult

address is 4bytes aligned, output buffer, the buffer size should >= "u32Key_N_ByteCount"

hsm_uint32_t HSM_RsaType::u32InputDataByteCount

the byte count of input data

hsm_uint32_t HSM_RsaType::u32Key_E_ByteCount

the byte count of key E

hsm_uint32_t HSM_RsaType::u32Key_N_ByteCount

hw will get the actual key data bit count according to the non-zero bit count, it means key not generate by multiply 2

hsm_uint32_t HSM_RsaType::u32ResultBufByteCnt

the result buffer size

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_ScatterHashType Struct Reference

Scatter Hash information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSM_ScatterHashType:

IMAGE

Public Attributes

HSM_ShaAlgType eAlg

HSM_ShaCtxType tCfg

hsm_uint32_t(* pResult)[18]

Detailed Description

Scatter Hash information used by driver when call specific API.

Member Data Documentation

HSM_ShaAlgType HSM_ScatterHashType::eAlg

select sha algorithm type

hsm_uint32_t(* HSM_ScatterHashType::pResult)[18]

point to the memory that HSM core to write result

HSM_ShaCtxType HSM_ScatterHashType::tCfg

sha algorithm parameter set by user

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sha2Type Struct Reference

SHA information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_Sha2Type:

IMAGE

Public Attributes

HSM_ShaAlgType eAlg

HSM_ShaCtxType tCfg

hsm_uint8_t * pResult

Detailed Description

SHA information used by driver when call specific API.

Member Data Documentation

HSM_ShaAlgType HSM_Sha2Type::eAlg

select sha algorithm type

hsm_uint8_t* HSM_Sha2Type::pResult

point to the memory that HSM core to write result

HSM_ShaCtxType HSM_Sha2Type::tCfg

sha algorithm parameter set by user

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_ShaExType Struct Reference

SHA extend information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_ShaExType:

IMAGE

Public Attributes

HSM_ShaAlgType eAlg

HSM_ShaCtxType tCfg

HSM_ShaResultBufType * pResult

HSM_DataFormatType eInputFmt

HSM_DataFormatType eOutputFmt

Detailed Description

SHA extend information used by driver when call specific API.

Member Data Documentation

HSM_ShaAlgType HSM_ShaExType::eAlg

select sha algorithm type

HSM_DataFormatType HSM_ShaExType::eInputFmt

HSM_DataFormatType HSM_ShaExType::eOutputFmt

HSM_ShaResultBufType* HSM_ShaExType::pResult

point to the memory that HSM core to write result

HSM_ShaCtxType HSM_ShaExType::tCfg

sha algorithm parameter set by user

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_ShaResultBufType Struct Reference

type definition for driver to store information

```
#include <module_driver_hsm.h>
```

Public Attributes

hsm_uint32_t aResult [0x10]

hsm_uint32_t u32ResultU32Cnt

Detailed Description

type definition for driver to store information

Member Data Documentation

hsm_uint32_t HSM_ShaResultBufType::aResult[0x10]

store the sha result, max size 0x10 is for SHA512

hsm_uint32_t HSM_ShaResultBufType::u32ResultU32Cnt

store the result hsm_uint32_t count, it's set by driver

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_ShaType Struct Reference

SHA information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_ShaType:

IMAGE

Public Attributes

HSM_ShaAlgType eAlg

HSM_ShaCtxType tCfg

HSM_ShaResultBufType * pResult

Detailed Description

SHA information used by driver when call specific API.

Member Data Documentation

HSM_ShaAlgType HSM_ShaType::eAlg

select sha algorithm type

HSM_ShaResultBufType* HSM_ShaType::pResult

point to the memory that HSM core to write result

HSM_ShaCtxType HSM_ShaType::tCfg

sha algorithm parameter set by user

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sm2DecryptType Struct Reference

SM2 decrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32SM2InputByteCnt
hsm_uint32_t * pSM2OutputByteCnt
hsm_uint32_t * pInputData
hsm_uint32_t(* pPrivateKey )[8]
hsm_uint32_t * pOutputData
```

Detailed Description

SM2 decrypt information used by driver when call specific API.

Member Data Documentation

hsm_uint32_t* HSM_Sm2DecryptType::pInputData

address is 4bytes aligned, length should be more than u32SM2InputByteCnt, and must be 4bytes align

hsm_uint32_t* HSM_Sm2DecryptType::pOutputData

address is 4bytes aligned, length should be more than u32SM2InputByteCnt-97, and must be 4bytes align

hsm_uint32_t(* HSM_Sm2DecryptType::pPrivateKey)[8]

address is 4bytes aligned, private key

hsm_uint32_t* HSM_Sm2DecryptType::pSM2OutputByteCnt

address is 4bytes aligned, driver internal will set output byte count in the memory this pointer point to

hsm_uint32_t HSM_Sm2DecryptType::u32SM2InputByteCnt

the length should be <= 129bytes

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sm2EncryptType Struct Reference

SM2 encrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32SM2InputByteCnt
hsm_uint32_t * pSM2OutputByteCnt
hsm_uint32_t * pInputData
hsm_uint32_t(* pPublicKey_x )[8]
hsm_uint32_t(* pPublicKey_y )[8]
hsm_uint32_t * pOutputData
```

Detailed Description

SM2 encrypt information used by driver when call specific API.

Member Data Documentation

hsm_uint32_t* HSM_Sm2EncryptType::pInputData

address is 4bytes aligned, length should be more than u32SM2InputByteCnt, and must be 4bytes align

hsm_uint32_t* HSM_Sm2EncryptType::pOutputData

address is 4bytes aligned, length should be more than u32SM2InputByteCnt+97, and must be 4bytes align

hsm_uint32_t(* HSM_Sm2EncryptType::pPublicKey_x)[8]

address is 4bytes aligned, public key axis x data

hsm_uint32_t(* HSM_Sm2EncryptType::pPublicKey_y)[8]

address is 4bytes aligned, public key axis y data

hsm_uint32_t* HSM_Sm2EncryptType::pSM2OutputByteCnt

address is 4bytes aligned, driver internal will set output byte count in the memory this pointer point to

hsm_uint32_t HSM_Sm2EncryptType::u32SM2InputByteCnt

the length should be <= 32bytes

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sm2GenHashType Struct Reference

SM2 HASH generation information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32InputDataByteCnt
hsm_uint32_t * pInputData_ZaAndM
hsm_uint32_t(* pOutputData_Hash )[8]
```

Detailed Description

SM2 HASH generation information used by driver when call specific API.

Member Data Documentation

hsm_uint32_t* HSM_Sm2GenHashType::pInputData_ZaAndM

address is 4bytes aligned, note: it should be ZA||M, the length should be more than u32InputDataByteCnt, and must be 4bytes align

hsm_uint32_t(* HSM_Sm2GenHashType::pOutputData_Hash)[8]

address is 4bytes aligned, Length is 32bytes, SM3 always output 256bit

hsm_uint32_t HSM_Sm2GenHashType::u32InputDataByteCnt

input data byte count

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_Sm2GenKeyPairType Struct Reference

SM2 key pair generation information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t(* pPrivateKey )[8]
```

```
hsm_uint32_t(* pPublicKey_X )[8]
```

```
hsm_uint32_t(* pPublicKey_Y )[8]
```

Detailed Description

SM2 key pair generation information used by driver when call specific API.

Member Data Documentation

```
hsm_uint32_t(* HSM_Sm2GenKeyPairType::pPrivateKey)[8]
```

address is 4bytes aligned, private key

```
hsm_uint32_t(* HSM_Sm2GenKeyPairType::pPublicKey_X)[8]
```

address is 4bytes aligned, the public key X

```
hsm_uint32_t(* HSM_Sm2GenKeyPairType::pPublicKey_Y)[8]
```

address is 4bytes aligned, the public key Y

The documentation for this struct was generated from the following file:

```
module_driver_hsm.h
```

HSM_Sm2GenZaType Struct Reference

SM2 ZA generation information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_BoolType bUseDefaultID

hsm_uint32_t u32SM2InputIDByteCnt

hsm_uint32_t(* pPublicKey_X)[8]

hsm_uint32_t(* pPublicKey_Y)[8]

hsm_uint32_t * pInputData_ID

hsm_uint32_t(* pOutputData_Za)[8]

Detailed Description

SM2 ZA generation information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Sm2GenZaType::bUseDefaultID

true means use default ID. false means use specific ID with "pInputData_ID" and "u32SM2InputIDByteCnt" params.

hsm_uint32_t* HSM_Sm2GenZaType::pInputData_ID

address is 4bytes aligned, note: If "bUseDefaultID" is false, would use this specific ID. length should be more than u32SM2InputIDByteCnt, and must be 4bytes align

hsm_uint32_t(* HSM_Sm2GenZaType::pOutputData_Za)[8]

address is 4bytes aligned, Length is 32bytes, SM3 always output 256bit

hsm_uint32_t(* HSM_Sm2GenZaType::pPublicKey_X)[8]

address is 4bytes aligned, public key x

hsm_uint32_t(* HSM_Sm2GenZaType::pPublicKey_Y)[8]

address is 4bytes aligned, public key y

hsm_uint32_t HSM_Sm2GenZaType::u32SM2InputIDByteCnt

the length should be <= 32bytes note: If "bUseDefaultID" is false, would use this specific ID length with byte unit.

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sm2SignType Struct Reference

SM2 sign information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_BoolType bUseDefaultID

HSM_BoolType bHashInput

hsm_uint32_t u32SM2InputMByteCnt

hsm_uint32_t u32SM2InputIDByteCnt

hsm_uint32_t * pInputData_ID

hsm_uint32_t * pInputData_MOrHash

hsm_uint32_t(* pPrivateKey)[8]

hsm_uint32_t(* pPublicKey_X)[8]

hsm_uint32_t(* pPublicKey_Y)[8]

hsm_uint32_t(* pOutputData_R)[8]

hsm_uint32_t(* pOutputData_S)[8]

Detailed Description

SM2 sign information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Sm2SignType::bHashInput

true means process hash message. false means process raw M message

HSM_BoolType HSM_Sm2SignType::bUseDefaultID

true means use default ID. false means use specific ID with "pInputData_ID" and "u32SM2InputIDByteCnt" params.

hsm_uint32_t* HSM_Sm2SignType::pInputData_ID

address is 4bytes aligned, note: If "bUseDefaultID" is false, would use this specific ID. length should be more than u32SM2InputIDByteCnt, and must be 4bytes align

hsm_uint32_t* HSM_Sm2SignType::pInputData_MOrHash

address is 4bytes aligned, length should be more than u32SM2InputMOrHashByteCnt, and must be 4bytes align

hsm_uint32_t(* HSM_Sm2SignType::pOutputData_R)[8]

address is 4bytes aligned, sign result R

hsm_uint32_t(* HSM_Sm2SignType::pOutputData_S)[8]

address is 4bytes aligned, sign result S

hsm_uint32_t(* HSM_Sm2SignType::pPrivateKey)[8]

address is 4bytes aligned, private key

hsm_uint32_t(* HSM_Sm2SignType::pPublicKey_X)[8]

address is 4bytes aligned, If bHashInput is false, the pointer must be not NULL_PTR

hsm_uint32_t(* HSM_Sm2SignType::pPublicKey_Y)[8]

address is 4bytes aligned, If bHashInput is false, the pointer must be not NULL_PTR

hsm_uint32_t HSM_Sm2SignType::u32SM2InputIDByteCnt

the length should be <= 32bytes note: If "bUseDefaultID" is false, would use this specific ID length with byte unit.

hsm_uint32_t HSM_Sm2SignType::u32SM2InputMByteCnt

the length should be <= 32bytes

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sm2VerifyType Struct Reference

SM2 verify information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_BoolType bUseDefaultID

HSM_BoolType bHashInput

hsm_uint32_t u32SM2InputMByteCnt

hsm_uint32_t u32SM2InputIDByteCnt

hsm_uint32_t * pInputData_MOrHash

hsm_uint32_t * pInputData_ID

hsm_uint32_t(* pInputData_R)[8]

hsm_uint32_t(* pInputData_S)[8]

hsm_uint32_t(* pPublicKey_X)[8]

hsm_uint32_t(* pPublicKey_Y)[8]

Detailed Description

SM2 verify information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_Sm2VerifyType::bHashInput

true means process hash message. false means process raw M message

HSM_BoolType HSM_Sm2VerifyType::bUseDefaultID

true means use default ID. false means use specific ID with "pInputData_ID" and "u32SM2InputIDByteCnt" params.

hsm_uint32_t* HSM_Sm2VerifyType::pInputData_ID

address is 4bytes aligned, note: If "bUseDefaultID" is false, would use this specific ID. length should be more than u32SM2InputIDByteCnt, and must be 4bytes align

hsm_uint32_t* HSM_Sm2VerifyType::pInputData_MOrHash

address is 4bytes aligned, length should be more than u32SM2InputMOrHashByteCnt, and must be 4bytes align

hsm_uint32_t(* HSM_Sm2VerifyType::pInputData_R)[8]

address is 4bytes aligned, the sign result R

hsm_uint32_t(* HSM_Sm2VerifyType::pInputData_S)[8]

address is 4bytes aligned, the sign result S

hsm_uint32_t(* HSM_Sm2VerifyType::pPublicKey_X)[8]

address is 4bytes aligned, the public key X

hsm_uint32_t(* HSM_Sm2VerifyType::pPublicKey_Y)[8]

address is 4bytes aligned, the public key Y

hsm_uint32_t HSM_Sm2VerifyType::u32SM2InputIDByteCnt

the length should be <= 32bytes note: If "bUseDefaultID" is false, would use this specific ID length with byte unit.

hsm_uint32_t HSM_Sm2VerifyType::u32SM2InputMByteCnt

the length should be <= 32bytes

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sm3ExType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSM_Sm3ExType:

IMAGE

Public Attributes

HSM_Sm3CtxType tCfg

hsm_uint32_t(* pRet)[8]

HSM_DataFormatType eInputFmt

HSM_DataFormatType eOutputFmt

Member Data Documentation

HSM_DataFormatType HSM_Sm3ExType::eInputFmt

HSM_DataFormatType HSM_Sm3ExType::eOutputFmt

hsm_uint32_t(* HSM_Sm3ExType::pRet)[8]

address should align with 4bytes, point to the memory that driver to store result

HSM_Sm3CtxType HSM_Sm3ExType::tCfg

sm3 algorithm parameter set by user

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_Sm3Type Struct Reference

SM3 information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_Sm3Type:

IMAGE

Public Attributes

```
HSM_Sm3CtxType tCfg  
hsm_uint32_t(* pRet)[8]
```

Detailed Description

SM3 information used by driver when call specific API.

Member Data Documentation

```
hsm_uint32_t(* HSM_Sm3Type::pRet)[8]
```

address should align with 4bytes, point to the memory that driver to store result

```
HSM_Sm3CtxType HSM_Sm3Type::tCfg
```

sm3 algorithm parameter set by user

The documentation for this struct was generated from the following file:

```
module_driver_hsm.h
```

HSM_Sm4EnDecryptType Struct Reference

SM4 encrypt/decrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_Sm4KeyType eUseKeyType

const **hsm_uint32_t**(* pKeyAddr)[4]

HSM_SymmEnDecryptAlgType eSm4Alg

const **hsm_uint32_t** * pDataInput

hsm_uint32_t u32InputByteCnt

hsm_uint32_t * pDataOutput

hsm_uint32_t u32OutputMemSize

const void * pIvData

Detailed Description

SM4 encrypt/decrypt information used by driver when call specific API.

Member Data Documentation

HSM_SymmEnDecryptAlgType HSM_Sm4EnDecryptType::eSm4Alg

SM4 encrypt/decrypt mode

HSM_Sm4KeyType HSM_Sm4EnDecryptType::eUseKeyType

the key source:vendor key programmed in nvr flash or new key in software

const hsm_uint32_t* HSM_Sm4EnDecryptType::pDataInput

address should align with 4bytes

hsm_uint32_t* HSM_Sm4EnDecryptType::pDataOutput

address should align with 4bytes

const void* HSM_Sm4EnDecryptType::pIvData

address is 4bytes aligned, 128bit(16Bytes) iv(initialization vector) data array, ECB not need configure this

const hsm_uint32_t(* HSM_Sm4EnDecryptType::pKeyAddr)[4]

address is 4bytes aligned, when eUseKeyType is HSM_SM4_KEY_NEW, this MUST configure

hsm_uint32_t HSM_Sm4EnDecryptType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_Sm4EnDecryptType::u32OutputMemSize

the output data buffer "pDataOutput" size, should >= "u32InputByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_SmsCfgType Struct Reference

SHA/MD5/SM3 context information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_BoolType bGenerateMacEn

HSM_HfamMacType eMacType

union {

 const void * **pKeyData**

hsm_uint32_t u32KeyId

} **tKeyInfo**

hsm_uint32_t u32KeyByteCnt

hsm_uint32_t u32GenerateMacByteCnt

hsm_uint32_t u32InputDataByteCnt

const void * **pInputData**

Detailed Description

SHA/MD5/SM3 context information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_SmsCfgType::bGenerateMacEn

whether generate MAC

HSM_HfamMacType HSM_SmsCfgType::eMacType

if enable generating MAC, this should configure

const void* HSM_SmsCfgType::pInputData

address is 4bytes aligned, point to the input data

const void* HSM_SmsCfgType::pKeyData

address is 4bytes aligned, key to generate MAC, if enable generating MAC, this should configure

union { ... } HSM_SmsCfgType::tKeyInfo

hsm_uint32_t HSM_SmsCfgType::u32GenerateMacByteCnt

the byte count of MAC data output

hsm_uint32_t HSM_SmsCfgType::u32InputDataByteCnt

the byte count of input data

hsm_uint32_t HSM_SmsCfgType::u32KeyByteCnt

the byte count of key data

hsm_uint32_t HSM_SmsCfgType::u32KeyId

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_TrngInitCfgType Struct Reference

#include <module_driver_hsm.h>

Public Attributes

- uint32_t **entropy_delay**
- uint32_t **samp_size**
- uint32_t **freq_min**
- uint32_t **freq_max**
- uint32_t **lrun_max**
- uint32_t **retry_max**
- uint32_t **mono_max**
- uint32_t **mono_range**
- uint32_t **run_range**
- uint32_t **run_max**
- uint32_t **poker_range**
- uint32_t **u32RngCtrlBitsOr**

Member Data Documentation

`uint32_t HSM_TrngInitCfgType::entropy_delay`

`uint32_t HSM_TrngInitCfgType::freq_max`

`uint32_t HSM_TrngInitCfgType::freq_min`

`uint32_t HSM_TrngInitCfgType::lrun_max`

`uint32_t HSM_TrngInitCfgType::mono_max`

`uint32_t HSM_TrngInitCfgType::mono_range`

`uint32_t HSM_TrngInitCfgType::poker_range`

`uint32_t HSM_TrngInitCfgType::retry_max`

`uint32_t HSM_TrngInitCfgType::run_max`

`uint32_t HSM_TrngInitCfgType::run_range`

`uint32_t HSM_TrngInitCfgType::samp_size`

`uint32_t HSM_TrngInitCfgType::u32RngCtrlBitsOr`

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSM_XCMacExType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSM_XCMacExType:

IMAGE

Public Attributes

HSM_AesKeyType eUseKeyType
HSM_AesNewKeyInfType tNewKeyInf
HSM_AesVendorKeyInfType tVendorKeyInf
hsm_uint32_t u32GenMacByteCnt
HSM_BoolType bCheckMacEn
const hsm_uint32_t * pDataInput
hsm_uint32_t u32InputByteCnt
hsm_uint32_t * pDataOutput
hsm_uint32_t u32OutputMemSize
HSM_DataFormatType eInputFmt
HSM_DataFormatType eOutputFmt
HSM_XCMacEngineType eEngine

Member Data Documentation

HSM_BoolType HSM_XCMacExType::bCheckMacEn

if enable this check, user should place the data after the input data, hsm will check the generated data and it, if fail, hsm generate a interrupt, and if user get hw status, will get a error status

HSM_XCMacEngineType HSM_XCMacExType::eEngine

HSM_DataFormatType HSM_XCMacExType::eInputFmt

HSM_DataFormatType HSM_XCMacExType::eOutputFmt

HSM_AesKeyType HSM_XCMacExType::eUseKeyType

fix to HSM_AES_KEY_NEW

const hsm_uint32_t* HSM_XCMacExType::pDataInput

address should align with 4bytes

hsm_uint32_t* HSM_XCMacExType::pDataOutput

address should align with 4bytes

HSM_AesNewKeyInfType HSM_XCMacExType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_XCMacExType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_XCMacExType::u32GenMacByteCnt

XCMAC API generate MAC data, it's "u32GenMacByteCnt" size ICV data

hsm_uint32_t HSM_XCMacExType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_XCMacExType::u32OutputMemSize

4bytes align, the output data buffer "pDataOutput" size, should >= "u32GenMacByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSM_XCMacType Struct Reference

CMAC/XMAC information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSM_XCMacType:

IMAGE

Public Attributes

```
HSM_AesKeyType eUseKeyType  
HSM_AesNewKeyInfType tNewKeyInf  
HSM_AesVendorKeyInfType tVendorKeyInf  
hsm_uint32_t u32GenMacByteCnt  
HSM_BoolType bCheckMacEn  
const hsm_uint32_t * pDataInput  
hsm_uint32_t u32InputByteCnt  
hsm_uint32_t * pDataOutput  
hsm_uint32_t u32OutputMemSize
```

Detailed Description

CMAC/XMAC information used by driver when call specific API.

Member Data Documentation

HSM_BoolType HSM_XCMacType::bCheckMacEn

if enable this check, user should place the data after the input data, hsm will check the generated data and it, if fail, hsm generate a interrupt, and if user get hw status, will get a error status

HSM_AesKeyType HSM_XCMacType::eUseKeyType

fix to HSM_AES_KEY_NEW

const hsm_uint32_t* HSM_XCMacType::pDataInput

address should align with 4bytes

hsm_uint32_t* HSM_XCMacType::pDataOutput

address should align with 4bytes

HSM_AesNewKeyInfType HSM_XCMacType::tNewKeyInf

when eUseKeyType is HSM_AES_KEY_NEW, this MUST configure

HSM_AesVendorKeyInfType HSM_XCMacType::tVendorKeyInf

when eUseKeyType is HSM_AES_KEY_CHIP_VENDOR_IFR, this MUST configure

hsm_uint32_t HSM_XCMacType::u32GenMacByteCnt

XCMAc API generate MAC data, it's "u32GenMacByteCnt" size ICV data

hsm_uint32_t HSM_XCMacType::u32InputByteCnt

128bit(16Bytes) align

hsm_uint32_t HSM_XCMacType::u32OutputMemSize

4bytes align, the output data buffer "pDataOutput" size, should >= "u32GenMacByteCnt"

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_AesDecryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_AesDecryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_AesDecryptType tCfg

Member Data Documentation

HSM_AesDecryptType HSMCom_AesDecryptType::tCfg

AES decrypt parameters

HSM_MailboxApiRetType HSMCom_AesDecryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_AesDecryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_AesDecryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_AesEncryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_AesEncryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_AesEncryptType tCfg

Member Data Documentation

HSM_AesEncryptType HSMCom_AesEncryptType::tCfg

aes parameters

HSM_MailboxApiRetType HSMCom_AesEncryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_AesEncryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_AesEncryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_AesmRawApiType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_AesmRawApiType:

IMAGE

Public Attributes

hsm_uint32_t u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_AesmRawApiType tCfg

Member Data Documentation

HSM_AesmRawApiType HSMCom_AesmRawApiType::tCfg

hsm_uint32_t HSMCom_AesmRawApiType::u32HSMStatusRet

hsm_uint32_t HSMCom_AesmRawApiType::u32Timeout

hsm_uint32_t HSMCom_AesmRawApiType::u32UserKeyID

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_AuthCheckType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
hsm_uint32_t u32HSMStatusRet  
hsm_uint32_t u32Timeout  
const hsm_uint32_t * pkG_x  
const hsm_uint32_t * pkG_y  
const hsm_uint32_t * pR  
const hsm_uint32_t * pS  
const hsm_uint32_t * pData  
hsm_uint32_t u32DataLength
```

Member Data Documentation

const hsm_uint32_t* HSMCom_AuthCheckType::pData

address should align with 4bytes

const hsm_uint32_t* HSMCom_AuthCheckType::pkG_x

address should align with 4bytes

const hsm_uint32_t* HSMCom_AuthCheckType::pkG_y

address should align with 4bytes

const hsm_uint32_t* HSMCom_AuthCheckType::pR

address should align with 4bytes

const hsm_uint32_t* HSMCom_AuthCheckType::pS

address should align with 4bytes

hsm_uint32_t HSMCom_AuthCheckType::u32DataLength

hsm_uint32_t HSMCom_AuthCheckType::u32HSMStatusRet

hsm_uint32_t HSMCom_AuthCheckType::u32Timeout

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_BigNumberCalcExtType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_BigNumberCalcExtType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_KEY_E

hsm_uint32_t u32UserKeyID_KEY_N

hsm_uint32_t u32BitCnt

HSM_BigNumberCalcExtType tCfg

Member Data Documentation

HSM_BigNumberCalcExtType HSMCom_BigNumberCalcExtType::tCfg

RSA parameters

hsm_uint32_t HSMCom_BigNumberCalcExtType::u32BitCnt

the bit count of numbers used of N

HSM_MailboxApiRetType HSMCom_BigNumberCalcExtType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_BigNumberCalcExtType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_BigNumberCalcExtType::u32UserKeyID_KEY_E

means don't load key from HSM, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_BigNumberCalcExtType::u32UserKeyID_KEY_N

means don't load key from HSM, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_BigNumberCalcType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_BigNumberCalcType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_KEY_E

hsm_uint32_t u32UserKeyID_KEY_N

hsm_uint32_t u32BitCnt

HSM_BigNumberCalcType tCfg

Member Data Documentation

HSM_BigNumberCalcType HSMCom_BigNumberCalcType::tCfg

RSA parameters

hsm_uint32_t HSMCom_BigNumberCalcType::u32BitCnt

the bit count of numbers used of N

HSM_MailboxApiRetType HSMCom_BigNumberCalcType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_BigNumberCalcType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_BigNumberCalcType::u32UserKeyID_KEY_E

means don't load key from HSM, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_BigNumberCalcType::u32UserKeyID_KEY_N

means don't load key from HSM, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_CancelJobType Struct Reference

Cancel job used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32MailboxChannel

Detailed Description

Cancel job used by driver when call specific API.

Member Data Documentation

HSM_MailboxApiRetType HSMCom_CancelJobType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_CancelJobType::u32MailboxChannel

the mailbox channel of the job to be canceled

hsm_uint32_t HSMCom_CancelJobType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_CcmDecryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_CcmDecryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_CcmDecryptType tCfg

Member Data Documentation

HSM_CcmDecryptType HSMCom_CcmDecryptType::tCfg

CCM decrypt parameter

HSM_MailboxApiRetType HSMCom_CcmDecryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_CcmDecryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_CcmDecryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_CcmEncryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_CcmEncryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_CcmEncryptType tCfg

Member Data Documentation

HSM_CcmEncryptType HSMCom_CcmEncryptType::tCfg

CCM encrypt parameters

HSM_MailboxApiRetType HSMCom_CcmEncryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_CcmEncryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_CcmEncryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_CMacExType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_CMacExType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_CMacExType tCfg

Member Data Documentation

HSM_CMacExType HSMCom_CMacExType::tCfg

CMAC parameters

HSM_MailboxApiRetType HSMCom_CMacExType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_CMacExType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_CMacExType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_CMacType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_CMacType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_CMacType tCfg

Member Data Documentation

HSM_CMacType HSMCom_CMacType::tCfg

CMAC parameters

HSM_MailboxApiRetType HSMCom_CMacType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_CMacType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_CMacType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_EccCalcType Struct Reference

ECC point calculate used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_EccCalcType:

IMAGE

Public Attributes

```
HSM_MailboxApiRetType u32HSMStatusRet  
hsm_uint32_t u32Timeout  
hsm_uint32_t u32UserKeyID_E  
hsm_uint32_t u32UserKeyID_P1  
hsm_uint32_t u32BitCnt  
HSM_DrvEccCurvePrmIndexType u32EccCurve  
HSM_EccCalcType tCfg
```

Detailed Description

ECC point calculate used by driver when call specific API.

Member Data Documentation

HSM_EccCalcType HSMCom_EccCalcType::tCfg

ecc decrypt parameters

hsm_uint32_t HSMCom_EccCalcType::u32BitCnt

ecc encrypt data bit count

HSM_DrvEccCurvePrmIndexType HSMCom_EccCalcType::u32EccCurve

ecc curve

HSM_MailboxApiRetType HSMCom_EccCalcType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_EccCalcType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_EccCalcType::u32UserKeyID_E

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_EccCalcType::u32UserKeyID_P1

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_EccEasyDecryType Struct Reference

ECC decrypt information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_EccEasyDecryType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PrivateKey

hsm_uint32_t u32UserKeyID_PublicKey

hsm_uint32_t u32BitCnt

HSM_DrvEccCurvePrmIndexType u32EccCurve

HSM_EccEasyDecryType tCfg

Detailed Description

ECC decrypt information used by driver when call specific API.

Member Data Documentation

HSM_EccEasyDecryType HSMCom_EccEasyDecryType::tCfg

ecc decrypt parameters

hsm_uint32_t HSMCom_EccEasyDecryType::u32BitCnt

ecc encrypt data bit count

HSM_DrvEccCurvePrmIndexType HSMCom_EccEasyDecryType::u32EccCurve

ecc curve

HSM_MailboxApiRetType HSMCom_EccEasyDecryType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_EccEasyDecryType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_EccEasyDecryType::u32UserKeyID_PrivateKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_EccEasyDecryType::u32UserKeyID_PublicKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_EccEasyEncryType Struct Reference

ECC encrypt information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_EccEasyEncryType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet
hsm_uint32_t u32Timeout
hsm_uint32_t u32UserKeyID_PrivateKey
hsm_uint32_t u32UserKeyID_PublicKey
hsm_uint32_t u32BitCnt
HSM_DrvEccCurvePrmIndexType u32EccCurve
HSM_EccEasyEncryType tCfg

Detailed Description

ECC encrypt information used by driver when call specific API.

Member Data Documentation

HSM_EccEasyEncryType HSMCom_EccEasyEncryType::tCfg

ecc encrypt parameters

hsm_uint32_t HSMCom_EccEasyEncryType::u32BitCnt

ecc encrypt data bit count

HSM_DrvEccCurvePrmIndexType HSMCom_EccEasyEncryType::u32EccCurve

ecc curve

HSM_MailboxApiRetType HSMCom_EccEasyEncryType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_EccEasyEncryType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_EccEasyEncryType::u32UserKeyID_PrivateKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_EccEasyEncryType::u32UserKeyID_PublicKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_EccSignType Struct Reference

ECC sign information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_EccSignType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PrivateKey

hsm_uint32_t u32BitCnt

HSM_DrvEccCurvePrmIndexType u32EccCurve

HSM_EccSignType tCfg

Detailed Description

ECC sign information used by driver when call specific API.

Member Data Documentation

HSM_EccSignType HSMCom_EccSignType::tCfg

ecc sign parameters

hsm_uint32_t HSMCom_EccSignType::u32BitCnt

ECC parameter N bit count

HSM_DrvEccCurvePrmIndexType HSMCom_EccSignType::u32EccCurve

select hsm rom internal ecc curve parameters, if use user's self curve, set it to
HSM_DRV_ECC_CURVE_PRM_IDX_MAX

HSM_MailboxApiRetType HSMCom_EccSignType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_EccSignType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_EccSignType::u32UserKeyID_PrivateKey

0 means use key from HSM internal key space, otherwise would load key with KEYID from key space

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_EccVerifyType Struct Reference

ECC verify information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_EccVerifyType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PublicKey

hsm_uint32_t u32BitCnt

HSM_DrvEccCurvePrmIndexType u32EccCurve

HSM_EccVerifyType tCfg

Detailed Description

ECC verify information used by driver when call specific API.

Member Data Documentation

HSM_EccVerifyType HSMCom_EccVerifyType::tCfg

ecc verify parameters

hsm_uint32_t HSMCom_EccVerifyType::u32BitCnt

ECC parameter N bit count

HSM_DrvEccCurvePrmIndexType HSMCom_EccVerifyType::u32EccCurve

select hsm rom internal ecc curve parameters, if use user's self curve, set it to
HSM_DRV_ECC_CURVE_PRM_IDX_MAX

HSM_MailboxApiRetType HSMCom_EccVerifyType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_EccVerifyType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_EccVerifyType::u32UserKeyID_PublicKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_FastTrueRandType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_FastTrueRandType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet
uint32_t u32Timeout
uint32_t(* aRandom)[HSM_TRNG_RAND_U32_CNT]
uint32_t u32CtrlBit
HSM_TrngInitCfgType tTrngCfg

Member Data Documentation

uint32_t(* HSMCom_FastTrueRandType::aRandom)[HSM_TRNG_RAND_U32_CNT]

the buffer address, HSM core will write random data to this buffer

HSM_TrngInitCfgType HSMCom_FastTrueRandType::tTrngCfg

uint32_t HSMCom_FastTrueRandType::u32CtrlBit

HSM_MailboxApiRetType HSMCom_FastTrueRandType::u32HSMStatusRet

HSM core write return status value in this

uint32_t HSMCom_FastTrueRandType::u32Timeout

current command timeout tick count, tick refer to the HSM bus clock

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_FlexAesDecryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_FlexAesDecryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_FlexAesDecryptType tCfg

Member Data Documentation

HSM_FlexAesDecryptType HSMCom_FlexAesDecryptType::tCfg

AES decrypt parameters

HSM_MailboxApiRetType HSMCom_FlexAesDecryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_FlexAesDecryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_FlexAesDecryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_FlexAesEncryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_FlexAesEncryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_FlexAesEncryptType tCfg

Member Data Documentation

HSM_FlexAesEncryptType HSMCom_FlexAesEncryptType::tCfg

aes parameters

HSM_MailboxApiRetType HSMCom_FlexAesEncryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_FlexAesEncryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_FlexAesEncryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_GcmDecryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_GcmDecryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_GcmDecryptType tCfg

Member Data Documentation

HSM_GcmDecryptType HSMCom_GcmDecryptType::tCfg

GCM decrypt parameter

HSM_MailboxApiRetType HSMCom_GcmDecryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_GcmDecryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_GcmDecryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_GcmEncryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_GcmEncryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_GcmEncryptType tCfg

Member Data Documentation

HSM_GcmEncryptType HSMCom_GcmEncryptType::tCfg

GCM parameters

HSM_MailboxApiRetType HSMCom_GcmEncryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_GcmEncryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_GcmEncryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_GenUserKeyType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_GenUserKeyType:

IMAGE

Public Attributes

HSM_KeyManagerGenUserKeyType tGenKeyPairCfg

HSM_DrvEccCurveParamType tEccParam

hsm_uint32_t u32EccBitLength

Member Data Documentation

HSM_DrvEccCurveParamType HSMCom_GenUserKeyType::tEccParam

only for generate ECC key pair

HSM_KeyManagerGenUserKeyType HSMCom_GenUserKeyType::tGenKeyPairCfg

hsm_uint32_t HSMCom_GenUserKeyType::u32EccBitLength

only for generate ECC key pair

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_GetRomHashType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_GetRomHashType:

IMAGE

Public Attributes

uint32_t u32HSMStatusRet

uint32_t u32Timeout

HSM_HashAlgType eAlg

HSM_ShaResultBufType * pResult

Member Data Documentation

HSM_HashAlgType HSMCom_GetRomHashType::eAlg

HSM_ShaResultBufType* HSMCom_GetRomHashType::pResult

uint32_t HSMCom_GetRomHashType::u32HSMStatusRet

uint32_t HSMCom_GetRomHashType::u32Timeout

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_KeySpaceStatusType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_KeySpaceStatusType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Ln_KeySpaceStatusType * pKMSpaceStatus

Member Data Documentation

HSM_Ln_KeySpaceStatusType* HSMCom_KeySpaceStatusType::pKMSpaceStatus

address should align with 4bytes

HSM_MailboxApiRetType HSMCom_KeySpaceStatusType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_KeySpaceStatusType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_LifeCycleChangeType Struct Reference

Change lifecycle used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_MailboxApiRetType u32HSMStatusRet  
hsm_uint32_t u32Timeout  
const hsm_uint32_t * pkG_x  
const hsm_uint32_t * pkG_y  
const hsm_uint32_t * pR  
const hsm_uint32_t * pS  
const hsm_uint32_t * pData  
hsm_uint32_t u32DataLength
```

Detailed Description

Change lifecycle used by driver when call specific API.

Member Data Documentation

```
const hsm_uint32_t* HSMCom_LifeCycleChangeType::pData
```

address should align with 4bytes

```
const hsm_uint32_t* HSMCom_LifeCycleChangeType::pkG_x
```

address should align with 4bytes

```
const hsm_uint32_t* HSMCom_LifeCycleChangeType::pkG_y
```

address should align with 4bytes

```
const hsm_uint32_t* HSMCom_LifeCycleChangeType::pR
```

address should align with 4bytes

const hsm_uint32_t* HSMCom_LifeCycleChangeType::pS

address should align with 4bytes

hsm_uint32_t HSMCom_LifeCycleChangeType::u32DataLength

the data byte count

HSM_MailboxApiRetType HSMCom_LifeCycleChangeType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_LifeCycleChangeType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_LoadFirmwareType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_MailboxApiRetType u32HSMStatusRet  
hsm_uint32_t u32Timeout
```

Member Data Documentation

HSM_MailboxApiRetType HSMCom_LoadFirmwareType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_LoadFirmwareType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

```
module_driver_hsm.h
```

HSMCom_Md5ExType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_Md5ExType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Md5ExType tCfg

Member Data Documentation

HSM_Md5ExType HSMCom_Md5ExType::tCfg

HSM_MailboxApiRetType HSMCom_Md5ExType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Md5ExType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Md5Type Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_Md5Type:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Md5Type tCfg

Member Data Documentation

HSM_Md5Type HSMCom_Md5Type::tCfg

HSM_MailboxApiRetType HSMCom_Md5Type::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Md5Type::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_MonCountType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32MonotonicIndex

hsm_uint32_t u32CurrentValue

Member Data Documentation

hsm_uint32_t HSMCom_MonCountType::u32CurrentValue

store the monotonic counter value

HSM_MailboxApiRetType HSMCom_MonCountType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_MonCountType::u32MonotonicIndex

range 0-13

hsm_uint32_t HSMCom_MonCountType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_NvrOtpType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_MailboxApiRetType u32HSMStatusRet  
hsm_uint32_t u32Timeout  
hsm_uint32_t u32Addr  
hsm_uint32_t aData [4]
```

Member Data Documentation

hsm_uint32_t HSMCom_NvrOtpType::aData[4]

hsm_uint32_t HSMCom_NvrOtpType::u32Addr

0x04400400 - 0x04401fff, 16bytes align

HSM_MailboxApiRetType HSMCom_NvrOtpType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_NvrOtpType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_RequestAuthType Struct Reference

Request Auth used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t(* pData)[HSM_COM_REQUEST_AUTH_DATA_U32_CNT]

Detailed Description

Request Auth used by driver when call specific API.

Member Data Documentation

hsm_uint32_t(* HSMCom_RequestAuthType::pData)[HSM_COM_REQUEST_AUTH_DATA_U32_CNT]

address should align with 4bytes, buffer to store the data

HSM_MailboxApiRetType HSMCom_RequestAuthType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_RequestAuthType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_RsaSsaPkcs1v15SignType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_RsaSsaPkcs1v15SignType:

IMAGE

Public Attributes

hsm_uint32_t u32HSMStatusRet
hsm_uint32_t u32Timeout
hsm_uint32_t u32UserKeyID_KEY_E
hsm_uint32_t u32UserKeyID_KEY_N
hsm_uint32_t u32RsaBitCnt
HSM_RsaSsaPkcs1V15SignType tCfg

Member Data Documentation

HSM_RsaSsaPkcs1V15SignType HSMCom_RsaSsaPkcs1v15SignType::tCfg

hsm_uint32_t HSMCom_RsaSsaPkcs1v15SignType::u32HSMStatusRet

hsm_uint32_t HSMCom_RsaSsaPkcs1v15SignType::u32RsaBitCnt

hsm_uint32_t HSMCom_RsaSsaPkcs1v15SignType::u32Timeout

hsm_uint32_t HSMCom_RsaSsaPkcs1v15SignType::u32UserKeyID_KEY_E

hsm_uint32_t HSMCom_RsaSsaPkcs1v15SignType::u32UserKeyID_KEY_N

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_RsaSsaPkcs1V15VerifyType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_RsaSsaPkcs1V15VerifyType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_KEY_E

hsm_uint32_t u32UserKeyID_KEY_N

hsm_uint32_t u32RsaBitCnt

HSM_RsaSsaPkcs1V15VerifyType tCfg

Member Data Documentation

HSM_RsaSsaPkcs1V15VerifyType HSMCom_RsaSsaPkcs1V15VerifyType::tCfg

RSA parameters

HSM_MailboxApiRetType HSMCom_RsaSsaPkcs1V15VerifyType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_RsaSsaPkcs1V15VerifyType::u32RsaBitCnt

the bit count of numbers used in RSA

hsm_uint32_t HSMCom_RsaSsaPkcs1V15VerifyType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_RsaSsaPkcs1V15VerifyType::u32UserKeyID_KEY_E

means don't load key from HSM, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_RsaSsaPkcs1V15VerifyType::u32UserKeyID_KEY_N

means don't load key from HSM, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_RsaSsaPssSignType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_RsaSsaPssSignType:

IMAGE

Public Attributes

hsm_uint32_t u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_KEY_E

hsm_uint32_t u32UserKeyID_KEY_N

hsm_uint32_t u32RsaBitCnt

HSM_RsaSsaPssSignType tCfg

Member Data Documentation

HSM_RsaSsaPssSignType HSMCom_RsaSsaPssSignType::tCfg

hsm_uint32_t HSMCom_RsaSsaPssSignType::u32HSMStatusRet

hsm_uint32_t HSMCom_RsaSsaPssSignType::u32RsaBitCnt

hsm_uint32_t HSMCom_RsaSsaPssSignType::u32Timeout

hsm_uint32_t HSMCom_RsaSsaPssSignType::u32UserKeyID_KEY_E

hsm_uint32_t HSMCom_RsaSsaPssSignType::u32UserKeyID_KEY_N

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_RsaSsaPssVerifyType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_RsaSsaPssVerifyType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_KEY_E

hsm_uint32_t u32UserKeyID_KEY_N

hsm_uint32_t u32RsaBitCnt

HSM_RsaSsaPssVerifyType tCfg

Member Data Documentation

HSM_RsaSsaPssVerifyType HSMCom_RsaSsaPssVerifyType::tCfg

RSA parameters

HSM_MailboxApiRetType HSMCom_RsaSsaPssVerifyType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_RsaSsaPssVerifyType::u32RsaBitCnt

the bit count of numbers used in RSA

hsm_uint32_t HSMCom_RsaSsaPssVerifyType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_RsaSsaPssVerifyType::u32UserKeyID_KEY_E

means don't load key from HSM, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_RsaSsaPssVerifyType::u32UserKeyID_KEY_N

means don't load key from HSM, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_RsaType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_RsaType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_KEY_E

hsm_uint32_t u32UserKeyID_KEY_N

hsm_uint32_t u32RsaBitCnt

HSM_RsaType tCfg

Member Data Documentation

HSM_RsaType HSMCom_RsaType::tCfg

RSA parameters

HSM_MailboxApiRetType HSMCom_RsaType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_RsaType::u32RsaBitCnt

the bit count of numbers used in RSA

hsm_uint32_t HSMCom_RsaType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_RsaType::u32UserKeyID_KEY_E

means don't load key from HSM, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_RsaType::u32UserKeyID_KEY_N

means don't load key from HSM, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_ScatterHashType Struct Reference

Scatter Hash information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_ScatterHashType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_ScatterHashType tCfg

Detailed Description

Scatter Hash information used by driver when call specific API.

Member Data Documentation

HSM_ScatterHashType HSMCom_ScatterHashType::tCfg

sha algorithm parameter set by user

HSM_MailboxApiRetType HSMCom_ScatterHashType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_ScatterHashType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_SelfTestType Struct Reference

```
#include <module_driver_hsm.h>
```

Public Attributes

```
HSM_MailboxApiRetType u32HSMStatusRet  
hsm_uint32_t u32Timeout  
hsm_uint32_t u32CommitId  
hsm_uint32_t u32BuildDate  
hsm_uint32_t u32FirmVersion  
hsm_uint32_t u32FixedValue  
hsm_uint32_t u32BusClk  
hsm_uint32_t u32Reserved0  
hsm_uint32_t u32Reserved1  
hsm_uint32_t u32Reserved2  
hsm_uint32_t u32Reserved3  
HSMCom_FirmwareLoadStatus u32FirmwareLoadStatus
```

Member Data Documentation

hsm_uint32_t HSMCom_SelfTestType::u32BuildDate

HSM repo build id

hsm_uint32_t HSMCom_SelfTestType::u32BusClk

the hsm bus clock

hsm_uint32_t HSMCom_SelfTestType::u32CommitId

HSM repo commit id

hsm_uint32_t HSMCom_SelfTestType::u32FirmVersion

firmware version

HSMCom_FirmwareLoadStatus HSMCom_SelfTestType::u32FirmwareLoadStatus

the status of loading firmware, IF >= 4 FAIL

hsm_uint32_t HSMCom_SelfTestType::u32FixedValue

FIX to 0x4d6f0000

HSM_MailboxApiRetType HSMCom_SelfTestType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_SelfTestType::u32Reserved0

hsm_uint32_t HSMCom_SelfTestType::u32Reserved1

hsm_uint32_t HSMCom_SelfTestType::u32Reserved2

hsm_uint32_t HSMCom_SelfTestType::u32Reserved3

hsm_uint32_t HSMCom_SelfTestType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Sha2Type Struct Reference

SHA information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_Sha2Type:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Sha2Type tCfg

Detailed Description

SHA information used by driver when call specific API.

Member Data Documentation

HSM_Sha2Type HSMCom_Sha2Type::tCfg

sha algorithm parameter set by user

HSM_MailboxApiRetType HSMCom_Sha2Type::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sha2Type::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_ShaExType Struct Reference

SHA extend information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_ShaExType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_ShaExType tCfg

Detailed Description

SHA extend information used by driver when call specific API.

Member Data Documentation

HSM_ShaExType HSMCom_ShaExType::tCfg

sha algorithm parameter set by user

HSM_MailboxApiRetType HSMCom_ShaExType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_ShaExType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_ShaType Struct Reference

SHA information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_ShaType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_ShaType tCfg

Detailed Description

SHA information used by driver when call specific API.

Member Data Documentation

HSM_ShaType HSMCom_ShaType::tCfg

sha algorithm parameter set by user

HSM_MailboxApiRetType HSMCom_ShaType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_ShaType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Sm2DecryptType Struct Reference

SM2 decrypt information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_Sm2DecryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PrivateKey

HSM_Sm2DecryptType tCfg

Detailed Description

SM2 decrypt information used by driver when call specific API.

Member Data Documentation

HSM_Sm2DecryptType HSMCom_Sm2DecryptType::tCfg

sm2 decrypt parameters

HSM_MailboxApiRetType HSMCom_Sm2DecryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm2DecryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_Sm2DecryptType::u32UserKeyID_PrivateKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_Sm2EncryptType Struct Reference

SM2 encrypt information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_Sm2EncryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PublicKey

HSM_Sm2EncryptType tCfg

Detailed Description

SM2 encrypt information used by driver when call specific API.

Member Data Documentation

HSM_Sm2EncryptType HSMCom_Sm2EncryptType::tCfg

sm2 encrypt parameters

HSM_MailboxApiRetType HSMCom_Sm2EncryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm2EncryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_Sm2EncryptType::u32UserKeyID_PublicKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_Sm2GenHashType Struct Reference

SM2 HASH generation information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_Sm2GenHashType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Sm2GenHashType tCfg

Detailed Description

SM2 HASH generation information used by driver when call specific API.

Member Data Documentation

HSM_Sm2GenHashType HSMCom_Sm2GenHashType::tCfg

sm2 generate hash parameters

HSM_MailboxApiRetType HSMCom_Sm2GenHashType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm2GenHashType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Sm2GenKeyPairType Struct Reference

SM2 key pair generation information used by driver when call specific API.

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_Sm2GenKeyPairType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Sm2GenKeyPairType tCfg

Detailed Description

SM2 key pair generation information used by driver when call specific API.

Member Data Documentation

HSM_Sm2GenKeyPairType HSMCom_Sm2GenKeyPairType::tCfg

sm2 generate key pair parameters

HSM_MailboxApiRetType HSMCom_Sm2GenKeyPairType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm2GenKeyPairType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Sm2GenZaType Struct Reference

SM2 ZA generation information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_Sm2GenZaType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PublicKey

HSM_Sm2GenZaType tCfg

Detailed Description

SM2 ZA generation information used by driver when call specific API.

Member Data Documentation

HSM_Sm2GenZaType HSMCom_Sm2GenZaType::tCfg

HSM_MailboxApiRetType HSMCom_Sm2GenZaType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm2GenZaType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_Sm2GenZaType::u32UserKeyID_PublicKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_Sm2SignType Struct Reference

SM2 sign information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_Sm2SignType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PublicKey

hsm_uint32_t u32UserKeyID_PrivateKey

HSM_Sm2SignType tCfg

Detailed Description

SM2 sign information used by driver when call specific API.

Member Data Documentation

HSM_Sm2SignType HSMCom_Sm2SignType::tCfg

sm2 sign parameters

HSM_MailboxApiRetType HSMCom_Sm2SignType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm2SignType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_Sm2SignType::u32UserKeyID_PrivateKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

hsm_uint32_t HSMCom_Sm2SignType::u32UserKeyID_PublicKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Sm2VerifyType Struct Reference

SM2 verify information used by driver when call specific API.

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_Sm2VerifyType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID_PublicKey

HSM_Sm2VerifyType tCfg

Detailed Description

SM2 verify information used by driver when call specific API.

Member Data Documentation

HSM_Sm2VerifyType HSMCom_Sm2VerifyType::tCfg

sm2 verify parameters

HSM_MailboxApiRetType HSMCom_Sm2VerifyType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm2VerifyType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_Sm2VerifyType::u32UserKeyID_PublicKey

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

`module_driver_hsm.h`

HSMCom_Sm3ExType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_Sm3ExType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Sm3ExType tCfg

Member Data Documentation

HSM_Sm3ExType HSMCom_Sm3ExType::tCfg

HSM_MailboxApiRetType HSMCom_Sm3ExType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm3ExType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Sm3Type Struct Reference

```
#include <module_driver_hsm.h>
```

Collaboration diagram for HSMCom_Sm3Type:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

HSM_Sm3Type tCfg

Member Data Documentation

HSM_Sm3Type HSMCom_Sm3Type::tCfg

HSM_MailboxApiRetType HSMCom_Sm3Type::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm3Type::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_Sm4EnDecryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_Sm4EnDecryptType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_Sm4EnDecryptType tCfg

Member Data Documentation

HSM_Sm4EnDecryptType HSMCom_Sm4EnDecryptType::tCfg

sm4 encrypt/decrypt parameters

HSM_MailboxApiRetType HSMCom_Sm4EnDecryptType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_Sm4EnDecryptType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_Sm4EnDecryptType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_TrueRandExType Struct Reference

the true random extend information passed to HSM core by mailbox

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t(* aRandom)[HSM_TRNG_RAND_U32_CNT]

HSM_RndSrcType u32RandSrc

Detailed Description

the true random extend information passed to HSM core by mailbox

Member Data Documentation

hsm_uint32_t(* HSMCom_TrueRandExType::aRandom)[HSM_TRNG_RAND_U32_CNT]

address is 4bytes aligned, the buffer address, HSM core will write random data to this buffer

HSM_MailboxApiRetType HSMCom_TrueRandExType::u32HSMStatusRet

HSM core write return status value in this

HSM_RndSrcType HSMCom_TrueRandExType::u32RandSrc

which random source will be selected

hsm_uint32_t HSMCom_TrueRandExType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_TrueRandType Struct Reference

the true random information passed to HSM core by mailbox

```
#include <module_driver_hsm.h>
```

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t(* aRandom)[HSM_TRNG_RAND_U32_CNT]

Detailed Description

the true random information passed to HSM core by mailbox

Member Data Documentation

hsm_uint32_t(* HSMCom_TrueRandType::aRandom)[HSM_TRNG_RAND_U32_CNT]

address is 4bytes aligned, the buffer address, HSM core will write random data to this buffer

HSM_MailboxApiRetType HSMCom_TrueRandType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_TrueRandType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_UserKeyManageType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_UserKeyManageType:

IMAGE

Public Attributes

```
HSM_MailboxApiRetType u32HSMStatusRet
hsm_uint32_t u32Timeout
union {
    hsm_uint32_t u32RevokeUserKeyID
    HSM_KeyManagerImportUserKeyType tImportUserKeyCfg
    HSM_KeyManagerExportUserKeyType tExportUserKeyCfg
    HSMCom_GenUserKeyType tGenUserKeyCfg
    HSM_Ln_ImportUserKeyExtType tImportUserKeyExtCfg
    hsm_uint32_t u32CheckUserKeyIDExt
    HSM_Ln_CopyUserKeyType tCopyUserKeyExtCfg
    HSM_KeyManagerCheckUserKeyStatusExtType tKeyIDStatusExtCfg
} tKMParam
```

Member Data Documentation

HSM_Ln_CopyUserKeyType HSMCom_UserKeyManageType::tCopyUserKeyExtCfg

HSM_KeyManagerExportUserKeyType HSMCom_UserKeyManageType::tExportUserKeyCfg

HSMCom_GenUserKeyType HSMCom_UserKeyManageType::tGenUserKeyCfg

HSM_KeyManagerImportUserKeyType HSMCom_UserKeyManageType::tImportUserKeyCfg

HSM_Ln_ImportUserKeyExtType HSMCom_UserKeyManageType::tImportUserKeyExtCfg

HSM_KeyManagerCheckUserKeyStatusExtType HSMCom_UserKeyManageType::tKeyIdStatusExtCfg

union { ... } HSMCom_UserKeyManageType::tKMPParam

hsm_uint32_t HSMCom_UserKeyManageType::u32CheckUserKeyIDExt

HSM_MailboxApiRetType HSMCom_UserKeyManageType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_UserKeyManageType::u32RevokeUserKeyID

hsm_uint32_t HSMCom_UserKeyManageType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMCom_XMacType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMCom_XMacType:

IMAGE

Public Attributes

HSM_MailboxApiRetType u32HSMStatusRet

hsm_uint32_t u32Timeout

hsm_uint32_t u32UserKeyID

HSM_XMacType tCfg

Member Data Documentation

HSM_XMacType HSMCom_XMacType::tCfg

XMAC parameter

HSM_MailboxApiRetType HSMCom_XMacType::u32HSMStatusRet

HSM core write return status value in this

hsm_uint32_t HSMCom_XMacType::u32Timeout

current command timeout Millisecond count, max 916000ms, if set to 0, means 1000ms

hsm_uint32_t HSMCom_XMacType::u32UserKeyID

0 means don't use key from HSM internal key space, otherwise would load key with KEYID from HSM

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMWrap_Sm4EncryptExType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMWrap_Sm4EncryptExType:

IMAGE

Public Attributes

```
HSMCom_Sm4EncryptExType * pParam
hsm_uint32_t u32Remain
HSM_FlexAesPadType ePad
hsm_uint32_t u32CurrentOutSize
hsm_uint32_t u32OutAlreadyByteCnt
hsm_uint32_t * pOutResultByteCnt
```

Member Data Documentation

HSM_FlexAesPadType HSMWrap_Sm4EncryptExType::ePad

hsm_uint32_t* HSMWrap_Sm4EncryptExType::pOutResultByteCnt

HSMCom_Sm4EncryptExType* HSMWrap_Sm4EncryptExType::pParam

hsm_uint32_t HSMWrap_Sm4EncryptExType::u32CurrentOutSize

hsm_uint32_t HSMWrap_Sm4EncryptExType::u32OutAlreadyByteCnt

hsm_uint32_t HSMWrap_Sm4EncryptExType::u32Remain

The documentation for this struct was generated from the following file:

module_driver_hsm.h

HSMWrap_Sm4EncryptType Struct Reference

#include <module_driver_hsm.h>

Collaboration diagram for HSMWrap_Sm4EncryptType:

IMAGE

Public Attributes

HSMCom_Sm4EncryptType * pParam

hsm_uint32_t u32Remain

Member Data Documentation

HSMCom_Sm4EncryptType* HSMWrap_Sm4EncryptType::pParam

hsm_uint32_t HSMWrap_Sm4EncryptType::u32Remain

The documentation for this struct was generated from the following file:

module_driver_hsm.h

INTM_InitType Struct Reference

The structure of the INTM init.

```
#include <module_driver_intm.h>
```

Public Attributes

INTM_IrqMonitorType **elrqMonitorIndex**

INTM_InterruptModeType **eMode**

bool **bEnReset**

bool **bEnInterrupt**

uint32_t **u32SrcDelayCnt**

uint16_t **u16IrqNumber**

char **aligned** [2]

Detailed Description

The structure of the INTM init.

Member Data Documentation

char INTM_InitType::aligned[2]

bool INTM_InitType::bEnInterrupt

Enable Interrupt

bool INTM_InitType::bEnReset

Enable Reset

INTM_IrqMonitorType INTM_InitType::elrqMonitorIndex

index of INTM channel

INTM_InterruptModeType INTM_InitType::eMode

INTM active mode

uint16_t INTM_InitType::u16IrqNumber

Monitored interrupt number

uint32_t INTM_InitType::u32SrcDelayCnt

Enable Interrupt

The documentation for this struct was generated from the following file:

module_driver_intm.h

INTM_MonitorType Struct Reference

```
#include <HwA_intm.h>
```

Public Attributes

```
__IO uint32_t IRQSELR
```

```
__IO uint32_t LATR
```

```
__IO uint32_t TMR
```

```
__I uint32_t SR
```

Member Data Documentation

```
__IO uint32_t INTM_MonitorType::IRQSELR
```

INTM Interrupt Request Select Register

```
__IO uint32_t INTM_MonitorType::LATR
```

INTM Latency Register

```
__I uint32_t INTM_MonitorType::SR
```

INTM Status Register

```
__IO uint32_t INTM_MonitorType::TMR
```

INTM Timer Register

The documentation for this struct was generated from the following file:

HwA_intm.h

ISM_FpcCfgType Struct Reference

```
#include <module_driver_ism.h>
```

Public Attributes

```
uint8_t u8FpcIndex  
bool bGlitchIntEnable  
ISM_FPC_EdgeDetectModeType eFallingDetectMode  
ISM_FPC_EdgeDelayModeType eFallingDelayNode  
ISM_FPC_EdgeDetectModeType eRisingDetectMode  
ISM_FPC_EdgeDelayModeType eRisingDelayNode  
uint16_t u32ThresholdValue  
void(* pFpcChannelCalllback)(ISM_HandleType *pHandle, uint32_t u32channel)
```

Member Data Documentation

bool ISM_FpcCfgType::bGlitchIntEnable

ISM_FPC_EdgeDelayModeType ISM_FpcCfgType::eFallingDelayNode

ISM_FPC_EdgeDetectModeType ISM_FpcCfgType::eFallingDetectMode

ISM_FPC_EdgeDelayModeType ISM_FpcCfgType::eRisingDelayNode

ISM_FPC_EdgeDetectModeType ISM_FpcCfgType::eRisingDetectMode

void(* ISM_FpcCfgType::pFpcChannelCalllback) (ISM_HandleType *pHandle, uint32_t u32channel)

uint16_t ISM_FpcCfgType::u32ThresholdValue

uint8_t ISM_FpcCfgType::u8FpcIndex

The documentation for this struct was generated from the following file:

module_driver_ism.h

ISM_InitCfgType Struct Reference

```
#include <module_driver_ism.h>
```

Public Attributes

```
bool bIsmEnable
```

```
bool bIntEnable
```

```
void(* pEcmEventIsrCallback)(ISM_HandleType *pHandle, uint8_t u8EcmFlags)
```

```
void(* pEventIsrCallback)(ISM_HandleType *pHandle, uint16_t u16Lamflags)
```

Member Data Documentation

```
bool ISM_InitCfgType::bIntEnable
```

```
bool ISM_InitCfgType::bIsmEnable
```

```
void(* ISM_InitCfgType::pEcmEventIsrCallback)(ISM_HandleType *pHandle, uint8_t u8EcmFlags)
```

```
void(* ISM_InitCfgType::pEventIsrCallback)(ISM_HandleType *pHandle, uint16_t u16Lamflags)
```

The documentation for this struct was generated from the following file:

```
module_driver_ism.h
```

ISM_LamCfgType Struct Reference

```
#include <module_driver_ism.h>
```

Public Attributes

uint8_t **u8LamIndex**

bool **bOvflIntEnable**

uint8_t **u8RcsSel**

uint8_t **u8MonSel**

ISM_LAM_InvertEventWindowType **eInvWin**

ISM_LAM_EventWindowEdgeType **eWinEdgSel**

ISM_LAM_EventWindowSelectType **eEvtWinSel**

ISM_LAM_RunModeSelectType **eRunMode**

ISM_LAM_MonitorSourceType **eMonSrcSel**

ISM_LAM_InvertMonitorType **eInvMon**

ISM_LAM_InvertReferenceType **eInvRef**

uint32_t **u32EvtCntThreshold**

void(* **pLamChannelOverCalllback**)(ISM_HandleType *pHandle, uint32_t u32channel)

Member Data Documentation

bool ISM_LamCfgType::bOvflIntEnable

ISM_LAM_EventWindowSelectType ISM_LamCfgType::eEvtWinSel

ISM_LAM_InvertMonitorType ISM_LamCfgType::eInvMon

ISM_LAM_InvertReferenceType ISM_LamCfgType::eInvRef

ISM_LAM_InvertEventWindowType ISM_LamCfgType::eInvWin

ISM_LAM_MonitorSourceType ISM_LamCfgType::eMonSrcSel

ISM_LAM_RunModeSelectType ISM_LamCfgType::eRunMode

ISM_LAM_EventWindowEdgeType ISM_LamCfgType::eWinEdgSel

void(* ISM_LamCfgType::pLamChannelOverCalllback) (ISM_HandleType *pHandle, uint32_t u32channel)

uint32_t ISM_LamCfgType::u32EvtCntThreshold

uint8_t ISM_LamCfgType::u8LamIndex

uint8_t ISM_LamCfgType::u8MonSel

uint8_t ISM_LamCfgType::u8RcsSel

The documentation for this struct was generated from the following file:

module_driver_ism.h

LAM_Type Struct Reference

```
#include <HwA_ism.h>
```

Public Attributes

```
__IO uint32_t LAM_STATUS
```

```
__IO uint32_t LAM_CTRL
```

```
__IO uint32_t LAM_CONFIG
```

```
__IO uint32_t LAM_CONTER
```

Member Data Documentation

```
__IO uint32_t LAM_Type::LAM_CONFIG
```

```
__IO uint32_t LAM_Type::LAM_CONTER
```

```
__IO uint32_t LAM_Type::LAM_CTRL
```

```
__IO uint32_t LAM_Type::LAM_STATUS
```

The documentation for this struct was generated from the following file:

```
HwA_ism.h
```

LIN_BaudRateCfgType Struct Reference

Baudrate configuration parameters.

```
#include <module_driver_lin.h>
```

Public Attributes

uint16_t **u16OvrSamp**

uint16_t **u16SBR**

Detailed Description

Baudrate configuration parameters.

Contains hardware-specific settings for baudrate calculation

Member Data Documentation

uint16_t LIN_BaudRateCfgType::u16OvrSamp

Oversampling ratio (number of samples per bit period)

uint16_t LIN_BaudRateCfgType::u16SBR

Baudrate scale factor (clock divider value)

The documentation for this struct was generated from the following file:

`module_driver_lin.h`

LIN_ConfigType Struct Reference

LIN controller hardware configuration parameters.

#include <module_driver_lin.h>

Collaboration diagram for LIN_ConfigType:

IMAGE

Public Attributes

LIN_XferStateType xferState

uint32_t baudRate

uint32_t funcClkFreq

uint32_t coreClkFreq

LIN_TimestampCallbackType timeCallback

LIN_NodeType nodeMode

uint8_t u8LinHwIndex

LIN_CallbackType callback

Detailed Description

LIN controller hardware configuration parameters.

Contains initialization settings and user-provided callbacks. Memory for this structure must be provided by the caller.

Member Data Documentation

uint32_t LIN_ConfigType::baudRate

LIN bus baudrate (default: 19200 bps)

LIN_CallbackType LIN_ConfigType::callback

Event callback (invoked on frame completion/error)

uint32_t LIN_ConfigType::coreClkFreq

LIN controller core clock frequency (Hz)

uint32_t LIN_ConfigType::funcClkFreq

LIN controller functional clock frequency (Hz)

LIN_NodeType LIN_ConfigType::nodeMode

Node role (LIN_NODE_MASTER or LIN_NODE_SLAVE)

LIN_TimestampCallbackType LIN_ConfigType::timeCallback

Callback to get time intervals (unit: microseconds)

uint8_t LIN_ConfigType::u8LinHwIndex

Hardware instance index (0 to FCUART_INSTANCE_COUNT-1)

LIN_XferStateType LIN_ConfigType::xferState

Runtime state instance (driver internal state)

The documentation for this struct was generated from the following file:

module_driver_lin.h

LIN_PduType Struct Reference

LIN Protocol Data Unit (PDU) structure.

```
#include <module_driver_lin.h>
```

Public Attributes

LIN_FramePidType Pid

LIN_FrameCsModelType Cs

LIN_FrameResponseType Drc

LIN_FrameDlType Dl

uint8 * **SduPtr**

Detailed Description

LIN Protocol Data Unit (PDU) structure.

Contains all necessary information to process a LIN frame

Member Data Documentation

LIN_FrameCsModelType LIN_PduType::Cs

Checksum model to use (enhanced/classic)

LIN_FrameDlType LIN_PduType::Dl

Data length (number of SDU bytes, 0-8)

LIN_FrameResponseType LIN_PduType::Drc

Response type (TX/RX/ignore)

LIN_FramePidType LIN_PduType::Pid

LIN frame protected identifier (0x00-0x3F with parity)

uint8* LIN_PduType::SduPtr

Pointer to SDU data buffer (must be >= Dl bytes)

The documentation for this struct was generated from the following file:

`module_driver_lin.h`

LIN_XferStateType Struct Reference

Runtime state of the LIN driver (maintained during operation)

#include <module_driver_lin.h>

Collaboration diagram for LIN_XferStateType:

IMAGE

Public Attributes

```
uint8_t u8LinChannel
FCUART_Type * base
__IO LIN_EventIdType curEventId
__IO LIN_FrameStateType curFrState
__IO LIN_StatusType curChnState
uint8_t sleepSignal
uint8_t cntByte
LIN_BaudRateCfgType xferBrCfg
uint16_t xferDlyCnt
uint8_t xferBuff [8]
LIN_PduType xferPdu
uint8_t checksum
uint64_t wakeupTime
```

Detailed Description

Runtime state of the LIN driver (maintained during operation)

Contains hardware context, current status, and temporary data buffers. Memory for this structure must be provided by the caller.

Member Data Documentation

FCUART_Type* LIN_XferStateType::base

Pointer to FCUART hardware instance (from **HwA_fcuart.h**)

uint8_t LIN_XferStateType::checksum

Calculated/validated checksum byte for current frame

uint8_t LIN_XferStateType::cntByte

Count of bytes transmitted/received in current frame

__IO LIN_StatusType LIN_XferStateType::curChnState

Current channel operational state (e.g., operational, sleep)

__IO LIN_EventIdType LIN_XferStateType::curEventId

Current LIN event ID (e.g., TX completed, RX error)

__IO LIN_FrameStateType LIN_XferStateType::curFrState

Current frame processing state (e.g., sending break field)

uint8_t LIN_XferStateType::sleepSignal

Transmit sleep signal flag

uint8_t LIN_XferStateType::u8LinChannel

LIN controller channel ID

uint64_t LIN_XferStateType::wakeupTime

Timestamp for wake-up signal detection (microseconds)

LIN_BaudRateCfgType LIN_XferStateType::xferBrCfg

Baudrate configuration parameters (oversampling ratio and divider)

uint8_t LIN_XferStateType::xferBuff[8]

Temporary buffer for frame data (max 8 bytes per LIN frame)

uint16_t LIN_XferStateType::xferDlyCnt

Clock frequency division ratio (coreClkFreq / funcClkFreq)

LIN_PduType LIN_XferStateType::xferPdu

Current Protocol Data Unit (PID, checksum model, data pointer)

The documentation for this struct was generated from the following file:

`module_driver_lin.h`

LU_AoiConfigType Struct Reference

LU AOI IN(n) configuration type.

#include <module_driver_lu.h>

Collaboration diagram for LU_AoiConfigType:

IMAGE

Public Attributes

LU_InConfigType tIn0Config

LU_InConfigType tIn1Config

LU_InConfigType tIn2Config

LU_InConfigType tIn3Config

Detailed Description

LU AOI IN(n) configuration type.

Member Data Documentation

LU_InConfigType LU_AoiConfigType::tIn0Config

AOI IN0 configuration

LU_InConfigType LU_AoiConfigType::tIn1Config

AOI IN1 configuration

LU_InConfigType LU_AoiConfigType::tIn2Config

AOI IN2 configuration

LU_InConfigType LU_AoiConfigType::tIn3Config

AOI IN3 configuration

The documentation for this struct was generated from the following file:

`module_driver_lu.h`

LU_ConfigType Struct Reference

LU initialization type.

```
#include <module_driver_lu.h>
```

Collaboration diagram for LU_ConfigType:

IMAGE

Public Attributes

LU_LgType eLgNum

LU_AoiConfigType tAoi0Config

LU_AoiConfigType tAoi1Config

LU_InputsSyncCtrlType tSyncCtrl

LU_BypassModeType eAoiMode

LU_ConfigModeType eFFMode

LU_OutputInitValueType eFFInitValue

LU_InputNType eFbMode

uint8_t u8Aoi0FiltCnt

uint8_t u8Aoi0Period

uint8_t u8Aoi1FiltCnt

uint8_t u8Aoi1Period

Detailed Description

LU initialization type.

Member Data Documentation

LU_BypassModeType LU_ConfigType::eAoiMode

AOI mode

LU_InputNType LU_ConfigType::eFbMode

feedback override control in JKFF mode

LU_OutputInitValueType LU_ConfigType::eFFInitValue

flip-flop initial value

LU_ConfigModeType LU_ConfigType::eFFMode

flip-flop mode

LU_LgType LU_ConfigType::eLgNum

LG number

LU_AoiConfigType LU_ConfigType::tAoi0Config

aoi0 configuration

LU_AoiConfigType LU_ConfigType::tAoi1Config

aoi1 configuration

LU_InputsSyncCtrlType LU_ConfigType::tSyncCtrl

inputs sync control, when set, would sync input product with bus clock

uint8_t LU_ConfigType::u8Aoi0FiltCnt

aoi0 input filter sample count

uint8_t LU_ConfigType::u8Aoi0Period

aoi0 input filter sample period

uint8_t LU_ConfigType::u8Aoi1FiltCnt

aoi1 input filter sample count

uint8_t LU_ConfigType::u8Aoi1Period

aoi1 input filter sample period

The documentation for this struct was generated from the following file:

module_driver_lu.h

LU_InConfigType Struct Reference

LU IN(n) configuration register type.

#include <module_driver_lu.h>

Public Attributes

LU_InModeType eInNACfg

LU_InModeType eInNBCfg

LU_InModeType eInNCCfg

LU_InModeType eInNDCfg

Detailed Description

LU IN(n) configuration register type.

Member Data Documentation

LU_InModeType LU_InConfigType::eInNACfg

AOI IN(n) A configuration

LU_InModeType LU_InConfigType::eInNBCfg

AOI IN(n) B configuration

LU_InModeType LU_InConfigType::eInNCCfg

AOI IN(n) C configuration

LU_InModeType LU_InConfigType::eInNDCfg

AOI IN(n) D configuration

The documentation for this struct was generated from the following file:

module_driver_lu.h

LU_InputsSyncCtrlType Struct Reference

LU LG inputs synchronous control.

```
#include <module_driver_lu.h>
```

Public Attributes

bool **bInputNA**

bool **bInputNB**

bool **bInputNC**

bool **bInputND**

Detailed Description

LU LG inputs synchronous control.

Member Data Documentation

bool LU_InputsSyncCtrlType::bInputNA

LU IN(n) A sync control

bool LU_InputsSyncCtrlType::bInputNB

LU IN(n) B sync control

bool LU_InputsSyncCtrlType::bInputNC

LU IN(n) C sync control

bool LU_InputsSyncCtrlType::bInputND

LU IN(n) D sync control

The documentation for this struct was generated from the following file:

module_driver_lu.h

MAM_Inf_Type Struct Reference

the MAM slave information

```
#include <module_driver_mam.h>
```

Public Attributes

uint8_t **error**

MAM_Index_Type **mam_num**

uint32_t **block_num**

Detailed Description

the MAM slave information

Member Data Documentation

uint32_t MAM_Inf_Type::block_num

uint8_t MAM_Inf_Type::error

MAM_Index_Type MAM_Inf_Type::mam_num

The documentation for this struct was generated from the following file:

module_driver_mam.h

MB_InitType Struct Reference

Configuration of the Mailbox.

```
#include <module_driver_mb.h>
```

Public Attributes

uint32_t **u32EventMask**

uint32_t **u32IntrMask**

void(* **pRequestCallback**)(MB_HandleType *pHandle, MB_ReceiveType *pReceive)

void(* **pDoneCallback**)(MB_HandleType *pHandle, uint32_t u32Channel)

Detailed Description

Configuration of the Mailbox.

Member Data Documentation

void(* MB_InitType::pDoneCallback) (MB_HandleType *pHandle, uint32_t u32Channel)

Callback of the done events

void(* MB_InitType::pRequestCallback) (MB_HandleType *pHandle, MB_ReceiveType *pReceive)

Callback of the request events

uint32_t MB_InitType::u32EventMask

Mask for the request events and done events

uint32_t MB_InitType::u32IntrMask

Mask for enable the interrupts

The documentation for this struct was generated from the following file:

module_driver_mb.h

MB_ReceiveType Struct Reference

Configuration of a receiving request.

```
#include <module_driver_mb.h>
```

Public Attributes

uint8_t **u8Channel**

uint8_t **u8MasterCoreIndex**

bool **bSecure**

bool **bSupervisor**

uint32_t **aData** [2]

Detailed Description

Configuration of a receiving request.

Member Data Documentation

uint32_t MB_ReceiveType::aData[2]

Buffer to store the receiving data

bool MB_ReceiveType::bSecure

The master security information of the core that currently obtains the channel

bool MB_ReceiveType::bSupervisor

The master processing mode of the core that currently obtains the channel.

uint8_t MB_ReceiveType::u8Channel

The selected Mailbox channel

uint8_t MB_ReceiveType::u8MasterCoreIndex

Buffer to store the master core index

The documentation for this struct was generated from the following file:

`module_driver_mb.h`

MB_RequestType Struct Reference

Configuration of a Mailbox request.

```
#include <module_driver_mb.h>
```

Public Attributes

uint8_t **u8Channel**

uint8_t **u8TargetCoreIndex**

bool **bAutoRelease**

uint32_t **aData** [2]

Detailed Description

Configuration of a Mailbox request.

Member Data Documentation

uint32_t MB_RequestType::aData[2]

Sending data

bool MB_RequestType::bAutoRelease

Automatically clear the channel

uint8_t MB_RequestType::u8Channel

The selected Mailbox channel

uint8_t MB_RequestType::u8TargetCoreIndex

Target Core Index

The documentation for this struct was generated from the following file:

module_driver_mb.h

MSC_InitCfgType Struct Reference

#include <module_driver_msc.h>

Public Attributes

uint8_t **u8PassiveLength**
uint8_t **u8PTFNumber**
MSC_WorkModeType **eWorkMode**
uint8_t **u8CommandBitLength**
bool **bSelSRH**
uint8_t **u8SRHDataBitLength**
bool **bSelSRL**
uint8_t **u8SRLDataBitLength**
bool **bDelayControl**
MSC_ParityType **eParity**
MSC_ReceiveBaudRateType **eBaudRate**
MSC_ReceiveFrameType **eRsvFrameType**

Member Data Documentation

bool **MSC_InitCfgType::bDelayControl**

Hardware Receive Interrupt Delay Control.

bool **MSC_InitCfgType::bSelSRH**

Select SRH.

bool **MSC_InitCfgType::bSelSRL**

Select SRL.

MSC_ReceiveBaudRateType **MSC_InitCfgType::eBaudRate**

Baudrate.

MSC_ParityType **MSC_InitCfgType::eParity**

Parity mode.

MSC_ReceiveFrameType MSC_InitCfgType::eRsvFrameType

Frame type.

MSC_WorkModeType MSC_InitCfgType::eWorkMode

MSC work mode.

uint8_t MSC_InitCfgType::u8CommandBitLength

Bit length of command frame.

uint8_t MSC_InitCfgType::u8PassiveLength

Frame Passive length.

uint8_t MSC_InitCfgType::u8PTFNumber

The number of passive time frames that are inserted in repetition mode between two data frames.

uint8_t MSC_InitCfgType::u8SRHDataBitLength

Bit length of SRH.

uint8_t MSC_InitCfgType::u8SRLDataBitLength

Bit length of SRL.

The documentation for this struct was generated from the following file:

module_driver_msc.h

MSC_InterruptCfgType Struct Reference

MSC interrupt configure structure.

```
#include <module_driver_msc.h>
```

Public Attributes

bool **bCFIntEnable**

bool **bTFIntEnable**

MSC_RFIEModeType **eRFIEMode**

MSC_DFIEModeType **eDFIEMode**

bool **bTOIntEnable**

uint16_t **u16TimeoutValue**

void(* **pISRCallback**)(const **MSC_HandleType** *pMscHandle, const uint32_t u32Status)

void(* **pTOISRCallback**)(const **MSC_HandleType** *pMscHandle)

Detailed Description

MSC interrupt configure structure.

Member Data Documentation

bool MSC_InterruptCfgType::bCFIntEnable

MSC_INCR[CFIE] bit, interrupt enable, if this bit asserted, command frame can lead the interrupt.

bool MSC_InterruptCfgType::bTFIntEnable

MSC_INCR[TFIE] bit, interrupt enable, if this bit asserted, time frame can lead the interrupt.

bool MSC_InterruptCfgType::bTOIntEnable

Enable the interrupt of timeout.

MSC_DFIEModeType MSC_InterruptCfgType::eDFIEMode

Interrupt mode of DFIE.

MSC_RFIEModeType MSC_InterruptCfgType::eRFIEMode

Interrupt mode of RFIE.

void(* MSC_InterruptCfgType::pISRCallback) (const MSC_HandleType *pMscHandle, const uint32_t u32Status)

void(* MSC_InterruptCfgType::pTOISRCallback) (const MSC_HandleType *pMscHandle)

uint16_t MSC_InterruptCfgType::u16TimeoutValue

Timeout value of timeout interrupt.

The documentation for this struct was generated from the following file:

module_driver_msc.h

MSC_IOControlInitType Struct Reference

#include <module_driver_msc.h>

Public Attributes

MSC_SDSelectionType eSDIsel
MSC_ENCSelectionType eENCSEL
MSC_ENHSelectionType eENHSEL
MSC_ENLSelectionType eENLSEL
MSC_FCLControlType eFclCtrl
MSC_InputPolarityType eSDIPol
MSC_ENxPolarityType eENXPol
MSC_SOPolarityType eSOPPol
MSC_FCLpolarityType eFCLPPol

Member Data Documentation

MSC_ENCSelectionType MSC_IOControlInitType::eENCSEL

ENC seletion.

MSC_ENHSelectionType MSC_IOControlInitType::eENHSEL

ENH seletion.

MSC_ENLSelectionType MSC_IOControlInitType::eENLSEL

ENL seletion.

MSC_ENxPolarityType MSC_IOControlInitType::eENXPol

ENX Polarity.

MSC_FCLControlType MSC_IOControlInitType::eFclCtrl

FCL Control.

MSC_FCLpolarityType MSC_IOControlInitType::eFCLPPol

FCL Polarity.

MSC_InputPolarityType MSC_IOControlInitType::eSDIPol

SDI Polarity.

MSC_SDISelectionType MSC_IOControlInitType::eSDIsel

SDI seletion.

MSC_SOPolarityType MSC_IOControlInitType::eSOPPol

SO Polarity.

The documentation for this struct was generated from the following file:

module_driver_msc.h

NVR_Per Struct Reference

NVR write and read Permissions. .

```
#include <module_driver_sec.h>
```

Public Attributes

bool **WritEn**

bool **ReadEn**

bool **EraseEn**

Detailed Description

NVR write and read Permissions. .

Member Data Documentation

bool NVR_Per::EraseEn

The NVR Erase permission

bool NVR_Per::ReadEn

The NVR read permission

bool NVR_Per::WritEn

The NVR write permission

The documentation for this struct was generated from the following file:

module_driver_sec.h

OSPI_ConfigType Struct Reference

The basic configuration option for the OSPI peripheral.

```
#include <module_driver_ospi.h>
```

Public Attributes

```
bool bDdrEn
OSPI_DqsSrcSelType eDqsSrcSel
OSPI_ClockDivideType eClkDivider
OSPI_ClockMuxType eClkMux
OSPI_EndianType eEndian
```

Detailed Description

The basic configuration option for the OSPI peripheral.

Member Data Documentation

```
bool OSPI_ConfigType::bDdrEn

OSPI_ClockDivideType OSPI_ConfigType::eClkDivider

OSPI_ClockMuxType OSPI_ConfigType::eClkMux

OSPI_DqsSrcSelType OSPI_ConfigType::eDqsSrcSel

OSPI_EndianType OSPI_ConfigType::eEndian
```

The documentation for this struct was generated from the following file:

```
module_driver_ospi.h
```

OSPI_DeviceConfigType Struct Reference

Define the structure for OSPI device configuration.

```
#include <module_driver_ospi.h>
```

Public Attributes

```
uint32_t u32FlashAddress  
uint32_t u32FlashTopAddress  
uint8_t u8FlashColAddressSpace  
uint8_t u8WordAddressable  
uint8_t u8CsHoldTime  
uint8_t u8CsSetupTime  
uint8_t u8DelayLine
```

Detailed Description

Define the structure for OSPI device configuration.

This structure is used to configure the parameters of the OSPI (OctoSPI) interface.

Member Data Documentation

uint32_t OSPI_DeviceConfigType::u32FlashAddress

uint32_t OSPI_DeviceConfigType::u32FlashTopAddress

uint8_t OSPI_DeviceConfigType::u8CsHoldTime

uint8_t OSPI_DeviceConfigType::u8CsSetupTime

uint8_t OSPI_DeviceConfigType::u8DelayLine

uint8_t OSPI_DeviceConfigType::u8FlashColAddressSpace

uint8_t OSPI_DeviceConfigType::u8WordAddressable

The documentation for this struct was generated from the following file:

`module_driver_ospi.h`

OVERLAY_ErrorInfoType Struct Reference

Error Info.

```
#include <module_driver_overlay.h>
```

Public Attributes

```
uint8_t bError_FAR_SIZE_INTR
uint8_t bError_FAR_DST_OVERFLOW_INTR
uint8_t bError_FAR_DST_NO_FLASH_INTR
uint8_t bError_REGION2_D_CROS_INTR
uint8_t bError_REGION2_S_CROS_INTR
uint8_t bError_REGION2_SIZE_INTR
uint8_t bError_REGION2_DST_INTR
uint8_t bError_REGION2_SRC_INTR
uint8_t bError_REGION1_D_CROS_INTR
uint8_t bError_REGION1_S_CROS_INTR
uint8_t bError_REGION1_SIZE_INTR
uint8_t bError_REGION1_DST_INTR
uint8_t bError_REGION1_SRC_INTR
uint8_t bError_REGION0_D_CROS_INTR
uint8_t bError_REGION0_S_CROS_INTR
uint8_t bError_REGION0_SIZE_INTR
uint8_t bError_REGION0_DST_INTR
uint8_t bError_REGION0_SRC_INTR
```

Detailed Description

Error Info.

Member Data Documentation

uint8_t OVERLAY_ErrorInfoType::bError_FAR_DST_NO_FLASH_INTR

bit 15, FAR of destination address is not flash

uint8_t OVERLAY_ErrorInfoType::bError_FAR_DST_OVERFLOW_INTR

bit 16, FAR of destination region is overflow with size

uint8_t OVERLAY_ErrorInfoType::bError_FAR_SIZE_INTR

bit 17, The FAR of size value is invalid

uint8_t OVERLAY_ErrorInfoType::bError_REGION0_D_CROS_INTR

bit 4, The range of region 0 destination address is crossed with other region

uint8_t OVERLAY_ErrorInfoType::bError_REGION0_DST_INTR

bit 1, The region 0 of destination address is not aligned with size

uint8_t OVERLAY_ErrorInfoType::bError_REGION0_S_CROS_INTR

bit 3, The range of region 0 source address is crossed with other region

uint8_t OVERLAY_ErrorInfoType::bError_REGION0_SIZE_INTR

bit 2, The region 0 of size value is invalid

uint8_t OVERLAY_ErrorInfoType::bError_REGION0_SRC_INTR

bit 0, The region 0 of source address is not aligned with size

uint8_t OVERLAY_ErrorInfoType::bError_REGION1_D_CROS_INTR

bit 9, The range of region 1 destination address is crossed with other region

uint8_t OVERLAY_ErrorInfoType::bError_REGION1_DST_INTR

bit 6, The region 1 of destination address is not aligned with size

uint8_t OVERLAY_ErrorInfoType::bError_REGION1_S_CROS_INTR

bit 8, The range of region 1 source address is crossed with other region

uint8_t OVERLAY_ErrorInfoType::bError_REGION1_SIZE_INTR

bit 7, The region 1 of size value is invalid

uint8_t OVERLAY_ErrorInfoType::bError_REGION1_SRC_INTR

bit 5, The region 1 of source address is not aligned with size

uint8_t OVERLAY_ErrorInfoType::bError_REGION2_D_CROS_INTR

bit 14, The range of region 2 destination address is crossed with other region

uint8_t OVERLAY_ErrorInfoType::bError_REGION2_DST_INTR

bit 11, The region 2 of destination address is not aligned with size

uint8_t OVERLAY_ErrorInfoType::bError_REGION2_S_CROS_INTR

bit 13, The range of region 2 source address is crossed with other region

uint8_t OVERLAY_ErrorInfoType::bError_REGION2_SIZE_INTR

bit 12, The region 2 of size value is invalid

uint8_t OVERLAY_ErrorInfoType::bError_REGION2_SRC_INTR

bit 10, The region 2 of source address is not aligned with size

The documentation for this struct was generated from the following file:

module_driver_overlay.h

OVERLAY_FARInitType Struct Reference

FAR parameter.

```
#include <module_driver_overlay.h>
```

Public Attributes

```
uint32_t u32FAREn  
uint32_t u32FARDst  
uint32_t u32FARSize
```

Detailed Description

FAR parameter.

Member Data Documentation

uint32_t OVERLAY_FARInitType::u32FARDst

Overlay region destination array, pflash, source always is 0x08000000

uint32_t OVERLAY_FARInitType::u32FAREn

Overlay region enable array

uint32_t OVERLAY_FARInitType::u32FARSize

Overlay region size array, must align to 64KB

The documentation for this struct was generated from the following file:

`module_driver_overlay.h`

OVERLAY_InterruptType Struct Reference

Interrupt Parameter.

```
#include <module_driver_overlay.h>
```

Collaboration diagram for OVERLAY_InterruptType:

IMAGE

Public Attributes

uint32_t **bEnableInterrupt**

OVERLAY_ErrorCallback_Type pCallBack

Detailed Description

Interrupt Parameter.

Member Data Documentation

uint32_t OVERLAY_InterruptType::bEnableInterrupt

Enable interrupt, 0 is disable, 1 is enable

OVERLAY_ErrorCallback_Type OVERLAY_InterruptType::pCallBack

Callback function for error interrupt

The documentation for this struct was generated from the following file:

module_driver_overlay.h

OVERLAY_OverlayRegionInitType Struct Reference

Overlay Region parameters.

```
#include <module_driver_overlay.h>
```

Public Attributes

```
uint32_t aOverlayRegionEn [OVERLYA_REGION_CNT]
uint32_t aOverlayRegionSrc [OVERLYA_REGION_CNT]
uint32_t aOverlayRegionDst [OVERLYA_REGION_CNT]
OVERLAY_OverlaySizeType aOverlayRegionSize [OVERLYA_REGION_CNT]
```

Detailed Description

Overlay Region parameters.

Member Data Documentation

uint32_t OVERLAY_OverlayRegionInitType::aOverlayRegionDst[OVERLYA_REGION_CNT]

Overlay region destination array, sram

uint32_t OVERLAY_OverlayRegionInitType::aOverlayRegionEn[OVERLYA_REGION_CNT]

Overlay region enable array

OVERLAY_OverlaySizeType

OVERLAY_OverlayRegionInitType::aOverlayRegionSize[OVERLYA_REGION_CNT]

Overlay region size array

uint32_t OVERLAY_OverlayRegionInitType::aOverlayRegionSrc[OVERLYA_REGION_CNT]

Overlay region source array, pflash

The documentation for this struct was generated from the following file:

`module_driver_overlay.h`

PCC_CtrlType Struct Reference

define the PCC module initialization structure.

```
#include <module_driver_pcc.h>
```

Public Attributes

PCC_ClkSrcType **eClockName**

bool **bEn**

PCC_ClkGateSrcType **eClkSrc**

PCC_ClkDivType **eDivider**

Detailed Description

define the PCC module initialization structure.

Member Data Documentation

bool PCC_CtrlType::bEn

Peripheral clock enable or disable

PCC_ClkGateSrcType PCC_CtrlType::eClkSrc

Peripheral function clock source select

PCC_ClkSrcType PCC_CtrlType::eClockName

Peripheral clock

PCC_ClkDivType PCC_CtrlType::eDivider

Peripheral clock divider value

The documentation for this struct was generated from the following file:

module_driver_pcc.h

PMC_ConfigType Struct Reference

Pmc control type.

```
#include <module_driver_pmc.h>
```

Public Attributes

boolean **bV15CtrlEn**

boolean **bV15AutoswEn**

boolean **bRpmV25En**

uint32_t **u32InterruptMask**

void(* **pPmcFaultNotify**)(PMC_HandleType *pHandle, PMC_Fault_Type eFault)

Detailed Description

Pmc control type.

The documentation for this struct was generated from the following file:

module_driver_pmc.h

PORT_DigitalFilterType Struct Reference

Port digital filter configuration structure.

```
#include <module_driver_port.h>
```

Public Attributes

uint32_t **u32PortPinsEn**

PORT_DigitalFilterClkSrcType eClkSrc

uint8_t **u8FilterLength**

uint8_t **RESERVED_0** [2]

Detailed Description

Port digital filter configuration structure.

Member Data Documentation

PORT_DigitalFilterClkSrcType PORT_DigitalFilterType::eClkSrc

Port clock source

uint8_t PORT_DigitalFilterType::RESERVED_0[2]

uint32_t PORT_DigitalFilterType::u32PortPinsEn

Port pin, 0~31 bit indicates the pin 0~31

uint8_t PORT_DigitalFilterType::u8FilterLength

digital filter length, the range is :0~31

The documentation for this struct was generated from the following file:

module_driver_port.h

PORT_InitType Struct Reference

Port initialization structure.

```
#include <module_driver_port.h>
```

Collaboration diagram for PORT_InitType:

IMAGE

Public Attributes

uint32_t **u32PortPins**

Port_PinMuxType **uPortPinMux**

PORT_GpioDirType **ePortGpioDir**

PORT_GpioLevelType **ePortGpioLevel**

bool **bPullEn**

PORT_PullStatusType **ePullSel**

bool **bDrvStrengthEn**

bool **bPassiveFilterEn**

uint8_t **RESERVED_0** [1]

PORT_InterruptType **tInterruptCfg**

Detailed Description

Port initialization structure.

Member Data Documentation

bool PORT_InitType::bDrvStrengthEn

whether to enable pad drive strength, only hs PAD used

bool PORT_InitType::bPassiveFilterEn

whether to use passive filter, please refer to reference manual for details

bool PORT_InitType::bPullEn

whether to pull the port pin

PORT_GpioDirType PORT_InitType::ePortGpioDir

Gpio direction

PORT_GpioLevelType PORT_InitType::ePortGpioLevel

Gpio level

PORT_PullStatusType PORT_InitType::ePullSel

pull status, pull up or pull down

uint8_t PORT_InitType::RESERVED_0[1]

PORT_InterruptType PORT_InitType::tInterruptCfg

Port interrupt config

uint32_t PORT_InitType::u32PortPins

Port pin, 0~31 bit indicates the pin 0~31

Port_PinMuxType PORT_InitType::uPortPinMux

Port pin mode

The documentation for this struct was generated from the following file:

module_driver_port.h

PORT_InterruptType Struct Reference

Port interrupt configuration structure.

```
#include <module_driver_port.h>
```

Public Attributes

PORT_IrqcConfigurationType ePortIsrMode

uint8_t **RESERVED_0** [3]

Detailed Description

Port interrupt configuration structure.

Member Data Documentation

PORT_IrqcConfigurationType PORT_InterruptType::ePortIsrMode

Port interrupt mode

uint8_t PORT_InterruptType::RESERVED_0[3]

The documentation for this struct was generated from the following file:

module_driver_port.h

PTIMER_ChannelCfgType Struct Reference

Ptimer channel configuration parameters.

```
#include <module_driver_ptimer.h>
```

Public Attributes

uint16_t **u16DelayCnt**

bool **bPreTriggerEnable**

bool **bPreTriggerOutputEnable**

bool **bPreTriggerBackToBackEnable**

Detailed Description

Ptimer channel configuration parameters.

Member Data Documentation

bool PTIMER_ChannelCfgType::bPreTriggerBackToBackEnable

Enable the back to back mode for ADC pre_trigger.

bool PTIMER_ChannelCfgType::bPreTriggerEnable

Enable the pre_trigger.

bool PTIMER_ChannelCfgType::bPreTriggerOutputEnable

Enable the pre_trigger output.

uint16_t PTIMER_ChannelCfgType::u16DelayCnt

Setting pre_trigger's delay cnt (delay time = u16DelayCnt* ptimer_clock_cnt).

The documentation for this struct was generated from the following file:

module_driver_ptimer.h

PTIMER_InitType Struct Reference

Ptimer instance initialization parameters.

```
#include <module_driver_ptimer.h>
```

Public Attributes

PTIMER_LoadValueModeType eLoadValueMode

PTIMER_ClockPreDividerType eClkPreDiv

PTIMER_ClockPreDivMultiplyFactorType eClkPreMultFactor

PTIMER_TriggerSrcType eTriggerInput

bool **bContinuousModeEnable**

bool **bDmaEnable**

bool **bInstanceBackToBackEnable**

bool **bDelayIntEnable**

bool **bSeqErrIntEnable**

uint16_t **u16IntDelayPeriod**

void(* **pSeqErrorNotify**)(struct **_PTIMER_HandleType** *pHandler, uint8_t u8Channel)

void(* **pIntNotify**)(struct **_PTIMER_HandleType** *pHandler)

Detailed Description

Ptimer instance initialization parameters.

Member Data Documentation

bool PTIMER_InitType::bContinuousModeEnable

Enable the continuous mode

bool PTIMER_InitType::bDelayIntEnable

Enable the interrupt for timer

bool PTIMER_InitType::bDmaEnable

Enable the dma for timer

bool PTIMER_InitType::bInstanceBackToBackEnable

Enable the back to back mode with all of the PTIMER units as a ring

bool PTIMER_InitType::bSeqErrIntEnable

Enable PTIMER Sequence Error Interrupt

PTIMER_ClockPreDividerType PTIMER_InitType::eClkPreDiv

Select the prescaler divider

PTIMER_ClockPreDivMultiplyFactorType PTIMER_InitType::eClkPreMultFactor

Select multiplication factor for prescaler

PTIMER_LoadValueModeType PTIMER_InitType::eLoadValueMode

Select the load mode

PTIMER_TrgSrcType PTIMER_InitType::eTriggerInput

Select the trigger input source

void(* PTIMER_InitType::pIntNotify) (struct _PTIMER_HandleType *pHandler)

Ptimer interrupt callback

void(* PTIMER_InitType::pSeqErrorNotify) (struct _PTIMER_HandleType *pHandler, uint8_t u8Channel)

Ptimer sequence error callback

uint16_t PTIMER_InitType::u16IntDelayPeriod

The period cnt

The documentation for this struct was generated from the following file:

module_driver_ptimer.h

PTIMER_PulseOutType Struct Reference

Ptimer pulse out configurations.

```
#include <module_driver_ptimer.h>
```

Public Attributes

uint32_t **u32PulseOutDlyHighCnt**

uint32_t **u32PulseOutDlyLowCnt**

Detailed Description

Ptimer pulse out configurations.

When the Ptimer counter time reaches u32PulseOutDlyHighUs, the pulse-out goes high; When the Ptimer counter time reaches u32PulseOutDlyLowUs, the pulse-out goes low.

Member Data Documentation

uint32_t **PTIMER_PulseOutType::u32PulseOutDlyHighCnt**

Delay cnt for pulse out to output high

uint32_t **PTIMER_PulseOutType::u32PulseOutDlyLowCnt**

Delay cnt for pulse out to output low

The documentation for this struct was generated from the following file:

module_driver_ptimer.h

ReEnDebug_Keys Struct Reference

Debug Re-Enable Key.

```
#include <module_driver_sec.h>
```

Public Attributes

uint32_t **Re_key0**

uint32_t **Re_key1**

uint32_t **Re_key2**

uint32_t **Re_key3**

Detailed Description

Debug Re-Enable Key.

Note

Only if four 32-bit writes match, it will set the DEN to 0x5 to enable debug and unlock SEC registers. The key is up to the user to decide in advance.

Member Data Documentation

uint32_t ReEnDebug_Keys::Re_key0

The Debug Re-enable key 0

uint32_t ReEnDebug_Keys::Re_key1

The Debug Re-enable key 1

uint32_t ReEnDebug_Keys::Re_key2

The Debug Re-enable key 2

uint32_t ReEnDebug_Keys::Re_key3

The Debug Re-enable key 3

The documentation for this struct was generated from the following file:

`module_driver_sec.h`

RGM_ConfigType Struct Reference

Rgm configure type.

```
#include <module_driver_rgm.h>
```

Public Attributes

bool **bResetfiltAonLp**

uint8_t **u8BusClockFilterWidth**

RGM_FilterClkSrc eFliterClkSrc

RGM_ResetDelayType eDelay

uint32_t **u32SystemInterruptMask**

uint32_t **u32Cpu0InterruptMask**

void(* **pRgmCpu0ResetInterruptNotify**)(RGM_HandleType *pHandle, uint32_t u32ResetReson)

Detailed Description

Rgm configure type.

Member Data Documentation

bool RGM_ConfigType::bResetfiltAonLp

Reset Pin Filter using AON32K Clock in Low Power Mode

RGM_ResetDelayType RGM_ConfigType::eDelay

Reset delay after interrupt assert

RGM_FilterClkSrc RGM_ConfigType::eFliterClkSrc

Reset Pin Filter Clock source

void(* RGM_ConfigType::pRgmCpu0ResetInterruptNotify) (RGM_HandleType *pHandle, uint32_t u32ResetReson)

Cpu0 reset interrupt callback

uint32_t RGM_ConfigType::u32Cpu0InterruptMask

Core interrupt enable mask

uint32_t RGM_ConfigType::u32SystemInterruptMask

System interrupt enable mask

uint8_t RGM_ConfigType::u8BusClockFilterWidth

Reset Pin Filter Bus Clock Width Configuration

The documentation for this struct was generated from the following file:

module_driver_rgm.h

RTC_Compensation Struct Reference

```
#include <module_driver_rtc.h>
```

Public Attributes

```
uint8_t u8Interval
```

```
uint8_t u8Value
```

Member Data Documentation

```
uint8_t RTC_Compensation::u8Interval
```

```
uint8_t RTC_Compensation::u8Value
```

The documentation for this struct was generated from the following file:

```
module_driver_rtc.h
```

RTC_InitType Struct Reference

Rtc initialization type.

```
#include <module_driver_rtc.h>
```

Public Attributes

```
uint32_t u32AlarmValue  
RTC_ClkoutSecIntFreqType eSecIntAndClkoutFreq  
bool bStableClkoutFreq  
bool bAlarmIntEn  
bool bSecondIntEn  
bool bOverflowIntEn
```

Detailed Description

Rtc initialization type.

Member Data Documentation

bool RTC_InitType::bAlarmIntEn

whether enable rtc alarm interrupt

bool RTC_InitType::bOverflowIntEn

overflow interrupt notification function pointer

bool RTC_InitType::bSecondIntEn

whether to use the second interrupt

bool RTC_InitType::bStableClkoutFreq

Select the RTC Seconds interrupt and the RTC_CLKOUT prescaler output frequency or RTC_CLKOUT is from the 32.768 kHz clock.

RTC_ClkoutSecIntFreqType RTC_InitType::eSecIntAndClkoutFreq

Select the RTC Seconds interrupt and the RTC_CLKOUT prescaler output frequency

uint32_t RTC_InitType::u32AlarmValue

rtc alarm value

The documentation for this struct was generated from the following file:

module_driver_rtc.h

RTC_TimeDate Struct Reference

```
#include <module_driver_rtc.h>
```

Public Attributes

uint16_t **u16Year**

uint8_t **u8Month**

uint8_t **u8Day**

uint8_t **u8Hour**

uint8_t **u8Minute**

uint8_t **u8Second**

Member Data Documentation

uint16_t RTC_TimeDate::u16Year

uint8_t RTC_TimeDate::u8Day

uint8_t RTC_TimeDate::u8Hour

uint8_t RTC_TimeDate::u8Minute

uint8_t RTC_TimeDate::u8Month

uint8_t RTC_TimeDate::u8Second

The documentation for this struct was generated from the following file:

`module_driver_rtc.h`

SCG_ClockCtrlType Struct Reference

Current system clock definition, include register SCG_CCR.

```
#include <module_driver_scg.h>
```

Public Attributes

bool **bSysClkMonitor**

SCG_ClockSrcType **eSrc**

SCG_ClockDivType **eDivSlow**

SCG_ClockDivType **eDivBus**

SCG_ClockDivType **eDivCore**

Detailed Description

Current system clock definition, include register SCG_CCR.

Member Data Documentation

bool SCG_ClockCtrlType::bSysClkMonitor

SCG_CCR[SYSCLK_CME], System Clock monitor bit this bit will be set 1 mandatory.

SCG_ClockDivType SCG_ClockCtrlType::eDivBus

SCG_CCR[DIVBUS], Bus Clock Divide Ratio.

SCG_ClockDivType SCG_ClockCtrlType::eDivCore

SCG_CCR[DIVCORE], Core Clock Divide Ratio.

SCG_ClockDivType SCG_ClockCtrlType::eDivSlow

SCG_CCR[DIVSLOW], Slow Clock Divide Ratio.

SCG_ClockSrcType SCG_ClockCtrlType::eSrc

SCG_CCR[SCS], System Clock Source.

The documentation for this struct was generated from the following file:

`module_driver_scg.h`

SCG_FircType Struct Reference

FIRC initial definition, include register SCG_FIRCCSR/ SCG_FIRCDIV/ SCG_FIRCFG.

```
#include <module_driver_scg.h>
```

Public Attributes

bool **bLock**

bool **bCm**

bool **bTrEn**

bool **bSten**

SCG_AsyncClockDivType **eDivH**

SCG_AsyncClockDivType **eDivM**

SCG_AsyncClockDivType **eDivL**

uint8_t **u8TrimSrc**

Detailed Description

FIRC initial definition, include register SCG_FIRCCSR/ SCG_FIRCDIV/ SCG_FIRCFG.

Member Data Documentation

bool SCG_FircType::bCm

SCG_FIRCCSR[CM] bit, FIRC Clock Monitor Enable

bool SCG_FircType::bLock

SCG_FIRCCSR[VLD] bit, Read only, Clock is enabled and output clock is valid

bool SCG_FircType::bSten

SCG_FIRCCSR[STEN] bit, Clock Standby Enable

bool SCG_FircType::bTrEn

SCG_FIRCCSR[TREN] bit, IRC software trim enable (auto trim)

SCG_AsyncClockDivType SCG_FirType::eDivH

SCG_FIRCDIV[DIVH] bit field definition.

SCG_AsyncClockDivType SCG_FirType::eDivL

SCG_FIRCDIV[DIVL] bit field definition.

SCG_AsyncClockDivType SCG_FirType::eDivM

SCG_FIRCDIV[DIVM] bit field definition.

uint8_t SCG_FirType::u8TrimSrc

SCG_FIRCTCFG[TRIMSRC] IRC clock auto trim reference clock source select.

The documentation for this struct was generated from the following file:

module_driver_scg.h

SCG_FoscType Struct Reference

FOSC initial definition, include register SCG_FOSCCSR/SCG_FOSCDIV/SCG_FOSCCFG.

```
#include <module_driver_scg.h>
```

Public Attributes

bool **bLock**

bool **bCm**

bool **bCmre**

bool **bSten**

bool **bBypass**

SCG_AsyncClockDivType **eDivH**

SCG_AsyncClockDivType **eDivM**

SCG_AsyncClockDivType **eDivL**

uint8_t **u8Eocv**

Detailed Description

FOSC initial definition, include register SCG_FOSCCSR/SCG_FOSCDIV/SCG_FOSCCFG.

Member Data Documentation

bool SCG_FoscType::bBypass

SCG_FOSCCFG[BYPASS] bit, Configures FOSC for bypassing the internal oscillator.

bool SCG_FoscType::bCm

SCG_FOSCCSR[CM] bit, Clock Monitor is enable

bool SCG_FoscType::bCmre

SCG_FOSCCSR[CMRE] bit, Clock Monitor Reset Enable 0 Generates interrupt, 1 Generates rese

bool SCG_FoscType::bLock

SCG_FOSCCSR[LK] bit, Write to set the register can be written or not

bool SCG_FoscType::bSten

SCG_FOSCCSR[STEN] bit, Clock Stop in Stop modes

SCG_AsyncClockDivType SCG_FoscType::eDivH

SCG_FOSCDIV[DIVH] bit field definition.

SCG_AsyncClockDivType SCG_FoscType::eDivL

SCG_FOSCDIV[DIVL] bit field definition.

SCG_AsyncClockDivType SCG_FoscType::eDivM

SCG_FOSCDIV[DIVM] bit field definition.

uint8_t SCG_FoscType::u8Eocv

SCG_FOSCCFG[EOCV] bit, end of count value

The documentation for this struct was generated from the following file:

module_driver_scg.h

SCG_InterruptType Struct Reference

The configuration option for the SCG interrupt.

```
#include <module_driver_scg.h>
```

Public Attributes

```
uint32_t u32InterruptMask  
void(* pFoscClkErrNotify)(struct _SCG_HandleType *pHandle)  
void(* pSircClkErrNotify)(struct _SCG_HandleType *pHandle)  
void(* pFircClkErrNotify)(struct _SCG_HandleType *pHandle)  
void(* pSoscClkErrNotify)(struct _SCG_HandleType *pHandle)  
void(* pPll0ClkErrNotify)(struct _SCG_HandleType *pHandle)  
void(* pPll1ClkErrNotify)(struct _SCG_HandleType *pHandle)
```

Detailed Description

The configuration option for the SCG interrupt.

Member Data Documentation

```
void(* SCG_InterruptType::pFircClkErrNotify)(struct _SCG_HandleType *pHandle)
```

FIRC clock error interrupt callback

```
void(* SCG_InterruptType::pFoscClkErrNotify)(struct _SCG_HandleType *pHandle)
```

FOSC clock error interrupt callback

```
void(* SCG_InterruptType::pPll0ClkErrNotify)(struct _SCG_HandleType *pHandle)
```

Pll0 clock error interrupt callback

```
void(* SCG_InterruptType::pPll1ClkErrNotify)(struct _SCG_HandleType *pHandle)
```

Pll0 clock error interrupt callback

void(* SCG_InterruptType::pSircClkErrNotify) (struct _SCG_HandleType *pHandle)

SIRC clock error interrupt callback

void(* SCG_InterruptType::pSoscClkErrNotify) (struct _SCG_HandleType *pHandle)

SOSC clock error interrupt callback

uint32_t SCG_InterruptType::u32InterruptMask

interrupt enable mask

The documentation for this struct was generated from the following file:

module_driver_scg.h

SCG_PllType Struct Reference

PLL definition, include register SCG_PLLCSR/ SCG_PLLDIV/SCG_PLL0CFG.

```
#include <module_driver_scg.h>
```

Public Attributes

bool **bLock**

bool **bCm**

bool **bCmre**

bool **bSten**

SCG_PllSourceType eSrc

SCG_AsyncClockDivType eDivH

SCG_AsyncClockDivType eDivM

SCG_AsyncClockDivType eDivL

SCG_PllPredivType ePrediv

SCG_PllPstdivType ePstDiv

SCG_PllMultiplyType u16Mult

Detailed Description

PLL definition, include register SCG_PLLCSR/ SCG_PLLDIV/SCG_PLL0CFG.

Member Data Documentation

bool SCG_PllType::bCm

SCG_PLLCSR[CM] bit, Clock Monitor enable.

bool SCG_PllType::bCmre

SCG_PLLCSR[CMRE] bit, 1: Clock Monitor Reset, 0: interrupt

bool SCG_PllType::bLock

SCG_PLLCSR[LK] bit, Write to set the register can be written or not.

bool SCG_PllType::bSten

SCG_PLLCSR[STEN] bit, Clock Standby Enable

SCG_AsyncClockDivType SCG_PllType::eDivH

SCG_PLLDIV[DIVH] bit field definition.

SCG_AsyncClockDivType SCG_PllType::eDivL

SCG_PLLDIV[DIVL] bit field definition.

SCG_AsyncClockDivType SCG_PllType::eDivM

SCG_PLLDIV[DIVM] bit field definition.

SCG_PllPredivType SCG_PllType::ePrediv

SCG_PLLCFG[PREDIV] bit field definition, the range is 0~31.

SCG_PllPstdivType SCG_PllType::ePstDiv

SCG_PLLCFG[PSTDIV] bit field definition.

SCG_PllSourceType SCG_PllType::eSrc

SCG_PLLCFG[SOURCE] bit field definition.

SCG_PllMultiplyType SCG_PllType::u16Mult

SCG_PLLCFG[MULT] bit field definition, this value need to be greater than 95.

The documentation for this struct was generated from the following file:

module_driver_scg.h

SCG_Sirc32kType Struct Reference

```
#include <module_driver_scg.h>
```

Public Attributes

bool **bLock**

Member Data Documentation

bool SCG_Sirc32kType::bLock

SIRC32KCSR[LK] bit, Write to set the register can be written or not

The documentation for this struct was generated from the following file:

module_driver_scg.h

SCG_SircType Struct Reference

SIRC initial definition, include register SCG_SIRCCSR/ SCG_SIRCDIV/ SCG_SIRCTCCFG.

```
#include <module_driver_scg.h>
```

Public Attributes

bool **bLock**

bool **bCm**

bool **bTrEn**

bool **bLpen**

bool **bSten**

SCG_AsyncClockDivType **eDivH**

SCG_AsyncClockDivType **eDivM**

SCG_AsyncClockDivType **eDivL**

uint8_t **u8TrimSrc**

Detailed Description

SIRC initial definition, include register SCG_SIRCCSR/ SCG_SIRCDIV/ SCG_SIRCTCCFG.

Member Data Documentation

bool SCG_SircType::bCm

SCG_SIRCCSR[CM] bit, SIRC Clock Monitor Enable

bool SCG_SircType::bLock

SCG_SIRCCSR[LK] bit, Write to set the register can be written or not

bool SCG_SircType::bLpen

SCG_SIRCCSR[LPEN] bit, Clock Stop Enable

bool SCG_SircType::bSten

SCG_SIRCCSR[STEN] bit, Clock Standby Enable

bool SCG_SircType::bTrEn

SCG_SIRCCSR[TREN] bit, IRC software trim enable (auto trim)

SCG_AsyncClockDivType SCG_SircType::eDivH

SCG_SIRCDIV[DIVH] bit field definition.

SCG_AsyncClockDivType SCG_SircType::eDivL

SCG_SIRCDIV[DIVL] bit field definition.

SCG_AsyncClockDivType SCG_SircType::eDivM

SCG_SIRCDIV[DIVM] bit field definition.

uint8_t SCG_SircType::u8TrimSrc

SCG_SIRCTCFG[TRIMSRC] IRC clock auto trim reference clock source select.

The documentation for this struct was generated from the following file:

module_driver_scg.h

SCG_SoscType Struct Reference

SOSC definition, include register SCG_SOSCCFG.

```
#include <module_driver_scg.h>
```

Public Attributes

bool **bLock**

bool **bBypass**

bool **bCm**

bool **bCmre**

bool **bNoWait**

uint8_t **u8Eocv**

Detailed Description

SOSC definition, include register SCG_SOSCCFG.

Member Data Documentation

bool SCG_SoscType::bBypass

SCG_SOSCCSR[BYPASS] bit, Configures SOSC for bypassing the internal oscillator.

bool SCG_SoscType::bCm

SCG_SOSCCSR[CM] bit, Clock Monitor enable.

bool SCG_SoscType::bCmre

SCG_SOSCCSR[CMRE] bit, 1: Clock Monitor Reset, 0: interrupt

bool SCG_SoscType::bLock

SCG_SOSCCSR[LK] bit, Write to set the register can be written or not.

bool SCG_SoscType::bNoWait

No wait SOSC valid when clock is enabled.

uint8_t SCG_SoscType::u8Eocv

SCG_SOSCCFG[EOCV] bit, end of count value

The documentation for this struct was generated from the following file:

module_driver_scg.h

SDADC_ChannelInitType Struct Reference

SDADC Channel Initialization Structure.

#include <module_driver_sdadc.h>

Collaboration diagram for SDADC_ChannelInitType:

IMAGE

Public Attributes

uint8_t u8SyncTrgSel
SDADC_SYNC_TRGENType eSyncSrc
SDADC_TRGLVLType eTrgLevel
SDADC_TRGSELType eTrgType
SDADC_CLKSELType eRcvClkSel
uint8_t u8HPFCutOff
SDADC_MASELType u8InternalModulator
SDADC_CLKEDGType eClkEdg
SDADC_IBFMTType eInputBitFormat
SDADC_RDFMTType eResultFmt
uint8_t u8FIFOEnable
uint8_t u8FIFOWaterMark
uint8_t u8DMAEnable
uint8_t u8BiasPositiveInputMask
uint8_t u8BiasNegativeInputMask
uint8_t u8TsSyncTrgSel
SDADC_SYNC_TRGENType eTsSyncSrc
SDADC_TSRDMType eTsReadMode
SDADC_TSTRGMTType eTsCounterTriggerMode
SDADC_INMXCPTType eTsInputMuxCopy
SDADC_TS_ENType eTsCounterEnable
SDADC_TSCLKSELType eTsCounterClkDiv
SDADC_IntlInitType tIntCfg
SDADC_ModulatorCfgType * pModulatorCfg
SDADC_FilterChainCfgType * pFilterChainCfg
SDADC_DetectCfgType * pDectectCfg

Detailed Description

SDADC Channel Initialization Structure.

Member Data Documentation

SDADC_CLKEDGType SDADC_ChannelInitType::eClkEdg

Clock edge type

SDADC_IBFMTType SDADC_ChannelInitType::eInputBitFormat

Input bit format type

SDADC_CLKSELType SDADC_ChannelInitType::eRcvClkSel

Receive clock selection

SDADC_RDFMTType SDADC_ChannelInitType::eResultFmt

Result data format type

SDADC_SYNC_TRGENType SDADC_ChannelInitType::eSyncSrc

Sync source type

SDADC_TRGLVLType SDADC_ChannelInitType::eTrgLevel

Trigger level type

SDADC_TRGSELType SDADC_ChannelInitType::eTrgType

Trigger type

SDADC_TSCLKSELType SDADC_ChannelInitType::eTsCounterClkDiv

Timestamp Counter Clock Selection

SDADC_TS_ENType SDADC_ChannelInitType::eTsCounterEnable

Timestamp Counter Enable

SDADC_TSTRGMTType SDADC_ChannelInitType::eTsCounterTriggerMode

Timestamp Counter Trigger Mode

SDADC_INMXCPTType SDADC_ChannelInitType::eTsInputMuxCopy

Input Mux Setting Copy Enable

SDADC_TSRDMType SDADC_ChannelInitType::eTsReadMode

Timestamp Information Read Mode

SDADC_SYNC_TRGENType SDADC_ChannelInitType::eTsSyncSrc

Timestamp Sync source type

SDADC_DetectCfgType* SDADC_ChannelInitType::pDectectCfg

Pointer to detection configuration

SDADC_FilterChainCfgType* SDADC_ChannelInitType::pFilterChainCfg

Pointer to filter chain configuration

SDADC_ModulatorCfgType* SDADC_ChannelInitType::pModulatorCfg

Pointer to modulator configuration

SDADC_IntInitType SDADC_ChannelInitType::tIntCfg

Interrupt configuration

uint8_t SDADC_ChannelInitType::u8BiasNegativeInputMask

Bias negative input mask

uint8_t SDADC_ChannelInitType::u8BiasPositiveInputMask

Bias positive input mask

uint8_t SDADC_ChannelInitType::u8DMAEnable

DMA enable

uint8_t SDADC_ChannelInitType::u8FIFOEnable

FIFO enable

uint8_t SDADC_ChannelInitType::u8FIFOWaterMark

FIFO watermark level

uint8_t SDADC_ChannelInitType::u8HPFCutOff

High-pass filter cut-off frequency

SDADC_MASELType SDADC_ChannelInitType::u8InternalModulator

Internal modulator selection

uint8_t SDADC_ChannelInitType::u8SyncTrgSel

Sync trigger selection

uint8_t SDADC_ChannelInitType::u8TsSyncTrgSel

Timestamp Sync trigger selection

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_ChannelRegType Struct Reference

```
#include <HwA_sdadc.h>
```

Public Attributes

```
__IO uint32_t CCR  
__IO uint32_t CDR  
__IO uint32_t CFR  
__IO uint32_t CPR  
__IO uint32_t CBR  
__IO uint32_t CLLMT  
__IO uint32_t CHLMT  
__I uint32_t CRDATA  
__I uint32_t CMPDATA
```

Member Data Documentation

__IO uint32_t SDADC_ChannelRegType::CBR

Channel Bias Register, offset: 0x10

__IO uint32_t SDADC_ChannelRegType::CCR

Channel Control Register, offset: 0x0

__IO uint32_t SDADC_ChannelRegType::CDR

Channel Data Rate Register, offset: 0x4

__IO uint32_t SDADC_ChannelRegType::CFR

Channel Configuration Register, offset: 0x8

__IO uint32_t SDADC_ChannelRegType::CHLMT

Channel High Limit Register, offset: 0x18

__IO uint32_t SDADC_ChannelRegType::CLLMT

Channel Low Limit Register, offset: 0x14

__I uint32_t SDADC_ChannelRegType::CMPDATA

Channel Multi Purpose Data Register, offset: 0x20

__IO uint32_t SDADC_ChannelRegType::CPR

Channel Protection Register, offset: 0xc

__I uint32_t SDADC_ChannelRegType::CRDATA

Channel Result Data Register, offset: 0x1c

The documentation for this struct was generated from the following file:

HwA_sdadc.h

SDADC_DetectCfgType Struct Reference

#include <module_driver_sdadc.h>

Public Attributes

uint8_t u8LmtCheckEnable
SDADC_CICSelectionType eLmtCheckSrc
uint8_t u8ZCDCheckEnable
SDADC_CICSelectionType eZCDCheckSrc
uint8_t u8ClkAbsenceCheckEnable
uint8_t u8ShortCircuitCheckEnable
uint8_t u8HighLimitReachBreakEnable
uint8_t u8LowLimitReachBreakEnable
uint8_t u8WindowLimitReachBreakEnable
uint8_t u8ShortCircuitDetectBreakEnable
uint8_t u8ClockAbsenceDetectBreakEnable
uint8_t u8LMTFlagProtectionEnable
uint8_t u8LMTResultProtectionEnable
uint8_t u8LMT_HysteresisProtectionEnable
uint8_t u8SCDFlagProtectionEnable
uint8_t u8CADFlagProtectionEnable
SDADC_LMTAVTType eLMTAvtType
uint8_t u8ClkAbsenceCnt
SDADC_LMTOPTType eLMTOptType
SDADC_SCDOPTType eSCDOptType
SDADC_SCDCMType eSCDCmtType
uint8_t u8SCDLmtCnt
uint32_t u32HighLmt
uint32_t u32LowLmt

Member Data Documentation

SDADC_LMTAVTType SDADC_DetectCfgType::eLMTAvtType

Limit activation type

SDADC_CICSelectionType SDADC_DetectCfgType::eLmtCheckSrc

Source for limit check

SDADC_LMTOPTType SDADC_DetectCfgType::eLMTOptType

Limit option type

SDADC_SCDCMType SDADC_DetectCfgType::eSCDCmtType

Short circuit detection conversion mode type

SDADC_SCDOPTType SDADC_DetectCfgType::eSCDOptType

Short circuit detection option type

SDADC_CICSelectionType SDADC_DetectCfgType::eZCDCheckSrc

Source for zero crossing detection check

uint32_t SDADC_DetectCfgType::u32HighLmt

High limit value

uint32_t SDADC_DetectCfgType::u32LowLmt

Low limit value

uint8_t SDADC_DetectCfgType::u8CADFlagProtectionEnable

Enable clock absence detection flag protection

uint8_t SDADC_DetectCfgType::u8ClkAbsenceCheckEnable

Enable clock absence detection check

uint8_t SDADC_DetectCfgType::u8ClkAbsenceCnt

Clock absence count

uint8_t SDADC_DetectCfgType::u8ClockAbsenceDetectBreakEnable

Enable break on clock absence detection

uint8_t SDADC_DetectCfgType::u8HighLimitReachBreakEnable

Enable break on high limit reached

uint8_t SDADC_DetectCfgType::u8LmtCheckEnable

Enable limit check

uint8_t SDADC_DetectCfgType::u8LMTFlagProtectionEnable

Enable limit flag protection

uint8_t SDADC_DetectCfgType::u8LMTHysteresisProtectionEnable

Enable limit hysteresis protection

uint8_t SDADC_DetectCfgType::u8LMTResultProtectionEnable

Enable limit result protection

uint8_t SDADC_DetectCfgType::u8LowLimitReachBreakEnable

Enable break on low limit reached

uint8_t SDADC_DetectCfgType::u8SCDFlagProtectionEnable

Enable short circuit detection flag protection

uint8_t SDADC_DetectCfgType::u8SCDLmtCnt

Short circuit detection limit count

uint8_t SDADC_DetectCfgType::u8ShortCircuitDetectBreakEnable

Enable break on short circuit detection

uint8_t SDADC_DetectCfgType::u8ShortCircuitCheckEnable

Enable short circuit detection check

uint8_t SDADC_DetectCfgType::u8WindowLimitReachBreakEnable

Enable break on window limit reached

uint8_t SDADC_DetectCfgType::u8ZCDCheckEnable

Enable zero crossing detection check

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_FilterChainCfgType Struct Reference

#include <module_driver_sdadc.h>

Collaboration diagram for SDADC_FilterChainCfgType:

IMAGE

Public Attributes

uint8_t **u8IntgraterEnable**
SDADC_IntegrationCfgType tIntgraterCfg
uint8_t **u8RectEnable**
SDADC_RectCfgType tRectCfg
uint8_t **u8MainFilterEnable**
SDADC_MFCMType eMainFilterMode
SDADC_MFORDType eMainOrder
uint16_t **u16MainOSR**
uint8_t **u8AuxFilterEnable**
SDADC_AFCMType eAuxFilterMode
SDADC_AFORDType eAuxOrder
uint16_t **u16AuxOSR**
SDADC_GainCompType eGainCompMode
uint32_t **u32GainFactor**
int32_t **s32Offset**
uint32_t **u32DebugSel**

Member Data Documentation

SDADC_AFCMType SDADC_FilterChainCfgType::eAuxFilterMode

Auxiliary filter mode

SDADC_AFORDType SDADC_FilterChainCfgType::eAuxOrder

Auxiliary filter order

SDADC_GainCompType SDADC_FilterChainCfgType::eGainCompMode

Gain compensation mode

SDADC_MFCMType SDADC_FilterChainCfgType::eMainFilterMode

Main filter mode

SDADC_MFORDType SDADC_FilterChainCfgType::eMainOrder

Main filter order

int32_t SDADC_FilterChainCfgType::s32Offset

Offset

SDADC_IntegrationCfgType SDADC_FilterChainCfgType::tIntgraterCfg

Integrator configuration

SDADC_RectCfgType SDADC_FilterChainCfgType::tRectCfg

Rectifier configuration

uint16_t SDADC_FilterChainCfgType::u16AuxOSR

Auxiliary filter oversampling ratio

uint16_t SDADC_FilterChainCfgType::u16MainOSR

Main filter oversampling ratio

uint32_t SDADC_FilterChainCfgType::u32DebugSel

Debug Selection

uint32_t SDADC_FilterChainCfgType::u32GainFactor

Gain factor

uint8_t SDADC_FilterChainCfgType::u8AuxFilterEnable

Auxiliary filter enable

uint8_t SDADC_FilterChainCfgType::u8IntgraterEnable

Integrator enable

uint8_t SDADC_FilterChainCfgType::u8MainFilterEnable

Main filter enable

uint8_t SDADC_FilterChainCfgType::u8RectEnable

Rectifier enable

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_InitType Struct Reference

SDADC Initialization Structure.

```
#include <module_driver_sdadc.h>
```

Collaboration diagram for SDADC_InitType:

IMAGE

Public Attributes

uint8_t **u8BiasEnable**

SDADC_PrescaleType **ePrescale**

SDADC_MENSELType **eMenSel**

uint8_t **u8CLKDiv**

uint8_t **u8Clk0Disable**

uint8_t **u8Clk1Disable**

uint8_t **u8Clk2Disable**

uint32_t **u32FunctionClk**

SDADC_ChannelInitType * **pChannelCfg** [**SDADC_CHANNEL_CNT**]

SDADC_PWMInitType * **pPWMCfg**

void(* **pCOCISRCallback**)(const **SDADC_HandleType** *pSdadCHandle, uint8_t u8Channel)

void(* **pFIFOISRCallback**)(const **SDADC_HandleType** *pSdadCHandle, uint8_t u8Channel,
SDADC_FIFOIsrStatusType eStatus)

void(* **pDetectISRCallback**)(const **SDADC_HandleType** *pSdadCHandle, uint8_t u8Channel,
SDADC_DetectIsrStatusType eStatus)

void(* **pSignDelayISRCallback**)(const **SDADC_HandleType** *pSdadCHandle, uint8_t u8Channel)

void(* **pTSignChangeISRCallback**)(const **SDADC_HandleType** *pSdadCHandle, uint8_t u8Channel)

void(* **pTSValidISRCallback**)(const **SDADC_HandleType** *pSdadCHandle, uint8_t u8Channel)

Detailed Description

SDADC Initialization Structure.

Member Data Documentation

SDADC_MENSELType SDADC_InitType::eMenSel

Main enable selection type

SDADC_PrescaleType SDADC_InitType::ePrescale

Prescale value for the clock

SDADC_ChannelInitType* SDADC_InitType::pChannelCfg[SDADC_CHANNEL_CNT]

Array of pointers to channel configurations

void(* SDADC_InitType::pCOCISRcallback) (const SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for Conversion Complete Interrupt (COCI)

void(* SDADC_InitType::pDetectISRcallback) (const SDADC_HandleType *pSdadCHandle, uint8_t u8Channel, SDADC_DetectIsrStatusType eStatus)

Callback function for Detection Interrupt

void(* SDADC_InitType::pFIFOISRcallback) (const SDADC_HandleType *pSdadCHandle, uint8_t u8Channel, SDADC_FIFOIsrStatusType eStatus)

Callback function for FIFO Interrupt

SDADC_PWMInitType* SDADC_InitType::pPWMCfg

Pointer to PWM configuration

void(* SDADC_InitType::pSignDelayISRcallback) (const SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for Sign Delay Interrupt

void(* SDADC_InitType::pTSignChangeISRcallback) (const SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for TSign Change Interrupt

void(* SDADC_InitType::pTSignValidISRcallback) (const SDADC_HandleType *pSdadCHandle, uint8_t u8Channel)

Callback function for TSign Valid Interrupt

uint32_t SDADC_InitType::u32FunctionClk

uint8_t SDADC_InitType::u8BiasEnable

Bias enable

uint8_t SDADC_InitType::u8Clk0Disable

Clock 0 out disable

uint8_t SDADC_InitType::u8Clk1Disable

Clock 1 out disable

uint8_t SDADC_InitType::u8Clk2Disable

Clock 2 out disable

uint8_t SDADC_InitType::u8CLKDiv

Clock divider value

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_IntegrationCfgType Struct Reference

#include <module_driver_sdadc.h>

Public Attributes

uint8_t u8SyncTrgSel
SDADC_SYNC_TRGENType eSyncSrc
SDADC_HTRPOLType eHWTrgPolarity
SDADC_TRGSELType eTrgType
uint8_t u8RestartFilterOnStart
uint8_t u8IntgratorCnt
uint8_t u8IntgratorDiscardCnt
uint8_t u8IntgratorRepeatCnt
uint8_t u8IntgratShiftValue
SDADC_EXTSTPType eStopControl
SDADC_IOGTTType eGateControl

Member Data Documentation

SDADC_IOGTTType SDADC_IntegrationCfgType::eGateControl

Integration output gating control

SDADC_HTRPOLType SDADC_IntegrationCfgType::eHWTrgPolarity

Hardware trigger polarity

SDADC_EXTSTPType SDADC_IntegrationCfgType::eStopControl

Integration stop control

SDADC_SYNC_TRGENType SDADC_IntegrationCfgType::eSyncSrc

Synchronized trigger source

SDADC_TRGSELType SDADC_IntegrationCfgType::eTrgType

Trigger type

uint8_t SDADC_IntegrationCfgType::u8IntegratorCnt

Integrator count

uint8_t SDADC_IntegrationCfgType::u8IntegratorDiscardCnt

Integrator discard count

uint8_t SDADC_IntegrationCfgType::u8IntegratorRepeatCnt

Integrator repeat count

uint8_t SDADC_IntegrationCfgType::u8IntegratShiftValue

Integrator shift value

uint8_t SDADC_IntegrationCfgType::u8RestartFilterOnStart

Restart filter on start

uint8_t SDADC_IntegrationCfgType::u8SyncTrgSel

Synchronized trigger selection

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_IntInitType Struct Reference

#include <module_driver_sdadc.h>

Public Attributes

uint8_t u8COCIrqEnable
uint8_t u8FIFOReadyIrqEnable
uint8_t u8FIFOOverFlowIrqEnable
uint8_t u8FIFOUnderFlowIrqEnable
uint8_t u8ZeroCrossDetectIrqEnable
uint8_t u8HighLimitReachIrqEnable
uint8_t u8LowLimitReachIrqEnable
uint8_t u8WindowLimitReachIrqEnable
uint8_t u8ShortCircuitDetectIrqEnable
uint8_t u8DataSaturationDetectIrqEnable
uint8_t u8ClockAbsenceDetectIrqEnable
uint8_t u8SignDelayCaptureIrqEnable
uint8_t u8TSignChangedIrqEnable
uint8_t u8TSValidIrqEnable

Member Data Documentation

uint8_t SDADC_IntInitType::u8ClockAbsenceDetectIrqEnable

Enable interrupt for Clock Absence Detection

uint8_t SDADC_IntInitType::u8COCIrqEnable

Enable interrupt for Conversion Complete Interrupt (COCI)

uint8_t SDADC_IntInitType::u8DataSaturationDetectIrqEnable

Enable interrupt for Data Saturation Detection

uint8_t SDADC_IntInitType::u8FIFOOverFlowIrqEnable

Enable interrupt for FIFO Overflow

uint8_t SDADC_IntInitType::u8FIFOReadyIrqEnable

Enable interrupt for FIFO Ready

uint8_t SDADC_IntInitType::u8FIFOUnderFlowIrqEnable

Enable interrupt for FIFO Underflow

uint8_t SDADC_IntInitType::u8HighLimitReachIrqEnable

Enable interrupt for High Limit Reached

uint8_t SDADC_IntInitType::u8LowLimitReachIrqEnable

Enable interrupt for Low Limit Reached

uint8_t SDADC_IntInitType::u8ShortCircuitDetectIrqEnable

Enable interrupt for Short Circuit Detection

uint8_t SDADC_IntInitType::u8SignDelayCaptureIrqEnable

Enable interrupt for Sign Delay Capture

uint8_t SDADC_IntInitType::u8TSignChangedIrqEnable

Enable interrupt for TSign Changed

uint8_t SDADC_IntInitType::u8TSValidIrqEnable

Enable interrupt for TimeStamp Valid

uint8_t SDADC_IntInitType::u8WindowLimitReachIrqEnable

Enable interrupt for Window Limit Reached

uint8_t SDADC_IntInitType::u8ZeroCrossDetectIrqEnable

Enable interrupt for Zero Cross Detection

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_ModulatorCfgType Struct Reference

```
#include <module_driver_sdadc.h>
```

Public Attributes

```
SDADC_MACLKSELType eMaClkSrc
SDADC_DITHER_ENType eDitherEnable
uint8_t u8ResetCycle
SDADC_INCFGPV11Type ePosV11
SDADC_GAINSELType eGain
uint8_t u8PosInputCommonVolModeMask
uint8_t u8NegInputCommonVolModeMask
SDADC_INCFGPTType ePosInputSrc
SDADC_INCFGMTType eNegInputSrc
SDADC_INSELType eInputSel
SDADC_INMODType eInputSelMode
```

Member Data Documentation

SDADC_DITHER_ENType SDADC_ModulatorCfgType::eDitherEnable

Dither enable control

SDADC_GAINSELType SDADC_ModulatorCfgType::eGain

Gain selection of analog input path

SDADC_INSELType SDADC_ModulatorCfgType::eInputSel

Input pin selection

SDADC_INMODType SDADC_ModulatorCfgType::eInputSelMode

Input MUX control mode

SDADC_MACLKSELType SDADC_ModulatorCfgType::eMaClkSrc

Internal modulator analog clock source selection

SDADC_INCFGMTType SDADC_ModulatorCfgType::eNegInputSrc

Negative input line source configuration

SDADC_INCFGPTType SDADC_ModulatorCfgType::ePosInputSrc

Positive input line source configuration

SDADC_INCFGPV11Type SDADC_ModulatorCfgType::ePosV11

Positive input line V11 configuration

uint8_t SDADC_ModulatorCfgType::u8NegInputCommonVolModeMask

Negative input common voltage mode mask

uint8_t SDADC_ModulatorCfgType::u8PosInputCommonVolModeMask

Positive input common voltage mode mask

uint8_t SDADC_ModulatorCfgType::u8ResetCycle

Reset cycle configuration

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_PWMInitType Struct Reference

```
#include <module_driver_sdadc.h>
```

Public Attributes

```
SDADC_TRGSignOENType eTrgSignOEN  
SDADC_PWM_CLKSELType eClkSel  
SDADC_BitRevType eBitRev  
SDADC_PWMPolarityType ePwmPolarity  
SDADC_PWMPatternType ePwmPattern
```

Member Data Documentation

SDADC_BitRevType SDADC_PWMInitType::eBitRev

Bit reverse mode configuration

SDADC_PWM_CLKSELType SDADC_PWMInitType::eClkSel

Clock selection for SDADC

SDADC_PWMPatternType SDADC_PWMInitType::ePwmPattern

PWM pattern configuration

SDADC_PWMPolarityType SDADC_PWMInitType::ePwmPolarity

PWM polarity configuration

SDADC_TRGSignOENType SDADC_PWMInitType::eTrgSignOEN

Trigger signal output enable configuration

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDADC_RectCfgType Struct Reference

```
#include <module_driver_sdadc.h>
```

Public Attributes

```
uint8_t u8SignDelayTsignPosEdge  
SDADC_CTRGSIGN_OENType eChannelTrgSignOutEnable  
SDADC_SSCHType eSignSignalChSel  
SDADC_SSRCType eSignSignalSrcSel  
uint8_t u8SignPosDelay  
uint8_t u8SignNegDelay
```

Member Data Documentation

SDADC_CTRGSIGN_OENType SDADC_RectCfgType::eChannelTrgSignOutEnable

Related trigger is enabled and will be set once TSIGN falling edge is got.

SDADC_SSCHType SDADC_RectCfgType::eSignSignalChSel

Sign signal channel selection

SDADC_SSRCType SDADC_RectCfgType::eSignSignalSrcSel

Sign signal source selection

uint8_t SDADC_RectCfgType::u8SignDelayTsignPosEdge

Delay for TSIGN positive edge

uint8_t SDADC_RectCfgType::u8SignNegDelay

Negative sign delay

uint8_t SDADC_RectCfgType::u8SignPosDelay

Positive sign delay

The documentation for this struct was generated from the following file:

module_driver_sdadc.h

SDDF_AuxFilterConfigType Struct Reference

The configuration for auxiliary filter.

```
#include <module_driver_sddf.h>
```

Public Attributes

SDDF_AuxFilterOrderType eFilterOrder

uint16_t u16OverSamplingRate

SDDF_AuxFilterConvModeType eConvMode

Detailed Description

The configuration for auxiliary filter.

Member Data Documentation

SDDF_AuxFilterConvModeType SDDF_AuxFilterConfigType::eConvMode

The conversion mode of the auxiliary filter

SDDF_AuxFilterOrderType SDDF_AuxFilterConfigType::eFilterOrder

The filter order of the auxiliary filter

uint16_t SDDF_AuxFilterConfigType::u16OverSamplingRate

The oversampling rate of auxiliary filter

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_ChannelConfigType Struct Reference

The configuration of the SDDF channel.

```
#include <module_driver_sddf.h>
```

Collaboration diagram for SDDF_ChannelConfigType:

IMAGE

Public Attributes

```
SDDF_TriggerEventType eTriggerEvent
SDDF_TriggerSrcType eTriggerSrc
SDDF_ClockEdgeType eInputClockEdge
SDDF_ClcokInputType eInputClockSrc
SDDF_BitFormatType eBitFormat
SDDF_ResultFormatType eResultFormat
bool bFIFOEnable
uint8_t u8FIFOWatermark
bool bMainFilterEnable
const SDDF_MainFilterConfigType * pMainFilterConfig
bool bAuxiliaryFilterEnable
const SDDF_AuxFilterConfigType * pAuxiliaryFilterConfig
```

Detailed Description

The configuration of the SDDF channel.

Member Data Documentation

bool SDDF_ChannelConfigType::bAuxiliaryFilterEnable

Whether to enable auxiliary filter of the channel

bool SDDF_ChannelConfigType::bFIFOEnable

Whether to enable result FIFO

bool SDDF_ChannelConfigType::bMainFilterEnable

Whether to enable main filter of the channel

SDDF_BitFormatType SDDF_ChannelConfigType::eBitFormat

The bit format of the channel

SDDF_ClockEdgeType SDDF_ChannelConfigType::eInputClockEdge

The input clock edge used by the channel

SDDF_ClcokInputType SDDF_ChannelConfigType::eInputClockSrc

The input clock source used by the channel

SDDF_ResultFormatType SDDF_ChannelConfigType::eResultFormat

The result format (signed or unsigned) of the channel

SDDF_TriggerEventType SDDF_ChannelConfigType::eTriggerEvent

The trigger event type of the channel

SDDF_TriggerSrcType SDDF_ChannelConfigType::eTriggerSrc

The trigger source of the channel

const SDDF_AuxFilterConfigType* SDDF_ChannelConfigType::pAuxiliaryFilterConfig

The pointer of the auxiliary filter configuraion

const SDDF_MainFilterConfigType* SDDF_ChannelConfigType::pMainFilterConfig

The pointer of the main filter configuration

uint8_t SDDF_ChannelConfigType::u8FIFOWatermark

The result FIFO watermark

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_ChannelDMAConfigType Struct Reference

The DMA configuration of the SDDF channel.

```
#include <module_driver_sddf.h>
```

Collaboration diagram for SDDF_ChannelDMAConfigType:

IMAGE

Public Attributes

bool **bDMAEnable**

DMA_HandleType * **pDmahandle**

DMA_ChannelType **eDmaChannel**

uint8_t **u8ChannelPriority**

uint32_t * **pResultBuffer**

SDDF_FIFOReadyCallbackType **pDMAReadyNotify**

Detailed Description

The DMA configuration of the SDDF channel.

Member Data Documentation

bool SDDF_ChannelDMAConfigType::bDMAEnable

Whether to enable DMA for the SDDF channel

DMA_ChannelType SDDF_ChannelDMAConfigType::eDmaChannel

The DMA channel used to transfer the SDDF conversion results

DMA_HandleType* SDDF_ChannelDMAConfigType::pDmahandle

The DMA instance used to transfer the SDDF conversion results

SDDF_FIFOReadyCallbackType SDDF_ChannelDMAConfigType::pDMAReadyNotify

The callback function when DMA transfer completed

uint32_t* SDDF_ChannelDMAConfigType::pResultBuffer

When channel DMA is enabled, you shall provide the result buffer to store the conversion results

uint8_t SDDF_ChannelDMAConfigType::u8ChannelPriority

The DMA channel priority, higher value means higher priority. The priority for different channels must be unique. Default priority value is same the channel number

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_ChannelInterruptConfigType Struct Reference

The result related interrupt configuraion of the SDDF channel.

```
#include <module_driver_sddf.h>
```

Public Attributes

```
bool bConversionCompleteIntEnable  
SDDF_GeneralCallbackType pConvCompleteNotify  
bool bFIFOReadyIntEnable  
uint32_t * pResultBuffer  
SDDF_FIFOReadyCallbackType pFIFOReadyNotify  
bool bDataSaturationIntEnable  
SDDF_GeneralCallbackType pDataSaturationNotify  
bool bUnderflowIntEnable  
SDDF_GeneralCallbackType pUnderflowNotify  
bool bOverflowIntEnable  
SDDF_GeneralCallbackType pOverflowNotify
```

Detailed Description

The result related interrupt configuraion of the SDDF channel.

Member Data Documentation

bool SDDF_ChannelInterruptConfigType::bConversionCompleteIntEnable

Whether to enable conversion complete interrupt

bool SDDF_ChannelInterruptConfigType::bDataSaturationIntEnable

Whether to enable interrupt for result data saturation

bool SDDF_ChannelInterruptConfigType::bFIFOReadyIntEnable

Whether to enable FIFO ready interrupt

bool SDDF_ChannelInterruptConfigType::bOverflowIntEnable

Whether to enable interrupt for FIFO overflow

bool SDDF_ChannelInterruptConfigType::bUnderflowIntEnable

Whether to enable interrupt for FIFO underflow

SDDF_GeneralCallbackType SDDF_ChannelInterruptConfigType::pConvCompleteNotify

The callback function of conversion complete interrupt

SDDF_GeneralCallbackType SDDF_ChannelInterruptConfigType::pDataSaturationNotify

The callback function of result data saturation interrupt

SDDF_FIFOReadyCallbackType SDDF_ChannelInterruptConfigType::pFIFOReadyNotify

The callback function of FIFO ready interrupt

SDDF_GeneralCallbackType SDDF_ChannelInterruptConfigType::pOverflowNotify

The callback function of FIFO overflow interrupt

uint32_t* SDDF_ChannelInterruptConfigType::pResultBuffer

When FIFO ready interrupt is enabled, you shall provide the result buffer to store the conversion results

SDDF_GeneralCallbackType SDDF_ChannelInterruptConfigType::pUnderflowNotify

The callback function of FIFO underflow interrupt

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_ChannelNotificationType Struct Reference

#include <module_driver_sddf.h>

Public Attributes

SDDF_GeneralCallbackType pConvCompleteNotify
SDDF_FIFOReadyCallbackType pFIFOReadyNotify
SDDF_GeneralCallbackType pDataSaturationNotify
SDDF_GeneralCallbackType pUnderflowNotify
SDDF_GeneralCallbackType pOverflowNotify
SDDF_FIFOReadyCallbackType pDMAReadyNotify
SDDF_EventDetectCallbackType pClockAbsenceDetectNotify
SDDF_EventDetectCallbackType pShortCircuitDetectNotify
SDDF_EventDetectCallbackType pLimitCheckNotify
SDDF_EventDetectCallbackType pZeroCrossDetectNotify

Member Data Documentation

SDDF_EventDetectCallbackType SDDF_ChannelNotificationType::pClockAbsenceDetectNotify

SDDF_GeneralCallbackType SDDF_ChannelNotificationType::pConvCompleteNotify

SDDF_GeneralCallbackType SDDF_ChannelNotificationType::pDataSaturationNotify

SDDF_FIFOReadyCallbackType SDDF_ChannelNotificationType::pDMAReadyNotify

SDDF_FIFOReadyCallbackType SDDF_ChannelNotificationType::pFIFOReadyNotify

SDDF_EventDetectCallbackType SDDF_ChannelNotificationType::pLimitCheckNotify

SDDF_GeneralCallbackType SDDF_ChannelNotificationType::pOverflowNotify

SDDF_EventDetectCallbackType SDDF_ChannelNotificationType::pShortCircuitDetectNotify

SDDF_GeneralCallbackType SDDF_ChannelNotificationType::pUnderflowNotify

SDDF_EventDetectCallbackType SDDF_ChannelNotificationType::pZeroCrossDetectNotify

The documentation for this struct was generated from the following file:

`module_driver_sddf.h`

SDDF_ClockAbsenceDetectConfigType Struct Reference

The configuration of the clock absence detect.

```
#include <module_driver_sddf.h>
```

Public Attributes

```
bool bClockAbsenceDetectEnable
uint8_t u8ClockAbsenceDetectThreshold
bool bClockAbsenceDetectIntEnable
SDDF_EventDetectCallbackType pClockAbsenceDetectNotify
bool bClockAbsenceDetectBreakEnable
```

Detailed Description

The configuration of the clock absence detect.

Member Data Documentation

bool SDDF_ClockAbsenceDetectConfigType::bClockAbsenceDetectBreakEnable

Whether to enable clock absence detect break signal

bool SDDF_ClockAbsenceDetectConfigType::bClockAbsenceDetectEnable

Whether to enable clock absence detect

bool SDDF_ClockAbsenceDetectConfigType::bClockAbsenceDetectIntEnable

Whether to enable clock absence detect interrupt

SDDF_EventDetectCallbackType SDDF_ClockAbsenceDetectConfigType::pClockAbsenceDetectNotify

The callback function for clock absence detect interrupt

uint8_t SDDF_ClockAbsenceDetectConfigType::u8ClockAbsenceDetectThreshold

The clock absence detector limitation threshold

The documentation for this struct was generated from the following file:

`module_driver_sddf.h`

SDDF_ClockOutConfigType Struct Reference

The clock out configuration for SDDF.

```
#include <module_driver_sddf.h>
```

Public Attributes

bool **bClockOut0Enable**

bool **bClockOut1Enable**

bool **bClockOut2Enable**

SDDF_ClockOutType **eClk0Src**

SDDF_ClockOutType **eClk1Src**

SDDF_ClockOutType **eClk2Src**

SDDF_ClockOutType **eClk3Src**

Detailed Description

The clock out configuration for SDDF.

Member Data Documentation

bool SDDF_ClockOutConfigType::bClockOut0Enable

Whether to enable clock out 0

bool SDDF_ClockOutConfigType::bClockOut1Enable

Whether to enable clock out 1

bool SDDF_ClockOutConfigType::bClockOut2Enable

Whether to enable clock out 2

SDDF_ClockOutType SDDF_ClockOutConfigType::eClk0Src

The clock source for clock channel 0

SDDF_ClockOutType SDDF_ClockOutConfigType::eClk1Src

The clock source for clock channel 1

SDDF_ClockOutType SDDF_ClockOutConfigType::eClk2Src

The clock source for clock channel 2

SDDF_ClockOutType SDDF_ClockOutConfigType::eClk3Src

The clock source for clock channel 3

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_InitConfigType Struct Reference

The init configuration for SDDF instance.

#include <module_driver_sddf.h>

Public Attributes

SDDF_PreDivType ePreDiv

uint16_t u16Div

Detailed Description

The init configuration for SDDF instance.

Member Data Documentation

SDDF_PreDivType SDDF_InitConfigType::ePreDiv

The pre-divider ratio for SDDF function clock

uint16_t SDDF_InitConfigType::u16Div

The clock divider for SDDF clock out

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_LimitCheckConfigType Struct Reference

The configuration of the limit detect.

```
#include <module_driver_sddf.h>
```

Public Attributes

bool bLimitCheckEnable
SDDF_ChannelLimitOptionType eLimitCheckOption
SDDF_InputSrcType eLimitCheckInputSrc
uint32_t u32LowLimit
uint32_t u32HighLimit
bool bLowLimitIntEnable
bool bHighLimitIntEnable
bool bWindowLimitIntEnable
SDDF_EventDetectCallbackType pLimitCheckNotify
bool bLowLimitBreakEnable
bool bHighLimitBreakEnable
bool bWindowLimitBreakEnable

Detailed Description

The configuration of the limit detect.

Member Data Documentation

bool SDDF_LimitCheckConfigType::bHighLimitBreakEnable

Whether to enable high limit detect break signal

bool SDDF_LimitCheckConfigType::bHighLimitIntEnable

Whether to enable high limit detect interrupt

bool SDDF_LimitCheckConfigType::bLimitCheckEnable

Whether to enable limit detect

bool SDDF_LimitCheckConfigType::bLowLimitBreakEnable

Whether to enable low limit detect break signal

bool SDDF_LimitCheckConfigType::bLowLimitIntEnable

Whether to enable low limit detect interrupt

bool SDDF_LimitCheckConfigType::bWindowLimitBreakEnable

Whether to enable window limit detect break signal

bool SDDF_LimitCheckConfigType::bWindowLimitIntEnable

Whether to enable window limit detect interrupt

SDDF_InputSrcType SDDF_LimitCheckConfigType::eLimitCheckInputSrc

The result source of limit detect

SDDF_ChannelLimitOptionType SDDF_LimitCheckConfigType::eLimitCheckOption

Limit detect option

SDDF_EventDetectCallbackType SDDF_LimitCheckConfigType::pLimitCheckNotify

The callback function for limit detect interrupt

uint32_t SDDF_LimitCheckConfigType::u32HighLimit

The higher threshold of limit detect

uint32_t SDDF_LimitCheckConfigType::u32LowLimit

The lower threshold of limit detect

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_MainFilterConfigType Struct Reference

The configuration for main filter.

```
#include <module_driver_sddf.h>
```

Public Attributes

SDDF_MainFilterOrderType eFilterOrder

uint16_t **u16OverSamplingRate**

SDDF_MainFilterConvModeType eConvMode

SDDF_MainFilterShiftType eShift

int32_t **s32Bias**

Detailed Description

The configuration for main filter.

Member Data Documentation

SDDF_MainFilterConvModeType SDDF_MainFilterConfigType::eConvMode

The conversion mode of the main filter

SDDF_MainFilterOrderType SDDF_MainFilterConfigType::eFilterOrder

The filter order of the main filter, the recommended filter order is often 1 order higher than the order of the delta-sigma ADC

SDDF_MainFilterShiftType SDDF_MainFilterConfigType::eShift

Main filter shift to get correct 24 bit precision the effective width of main filter raw data is $W_{MCIC} = \text{ceiling}[\log_2(\text{OversamplingRate} + 1)^{\text{FilterOrder}}] + 1 - \text{ResultFormat}$ And thus: $\text{Shift} = 24 - W_{MCIC}$, positive means shift left, negative means shift right

int32_t SDDF_MainFilterConfigType::s32Bias

Bias offset for dedicated channel main filter; used for sign adjustment and compensation. The main filter data after shift, will subtract this bias value

uint16_t SDDF_MainFilterConfigType::u16OverSamplingRate

The oversampling rate of main filter, lower range: 3, higher range depends on the filter order to keep the filter result not exceed 32bit. If filter order is 5th, higher range is 72; If filter order is 4th, higher range is 203; Otherwise higher range is 511

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_ShortCircuitDetectConfigType Struct Reference

The configuration of the short circuit detect.

```
#include <module_driver_sddf.h>
```

Public Attributes

bool **bShortCircuitDetectEnable**

SDDF_ShortCircuitOptionType **eShortCircuitDetectOption**

SDDF_ShortCircuitDetectModeType **eShortCircuitDetectMode**

uint8_t **u8ShortCircuitDetectThreshold**

bool **bShortCircuitDetectIntEnable**

SDDF_EventDetectCallbackType **pShortCircuitDetectNotify**

bool **bShortCircuitDetectBreakEnable**

Detailed Description

The configuration of the short circuit detect.

Member Data Documentation

bool SDDF_ShortCircuitDetectConfigType::bShortCircuitDetectBreakEnable

Whether to enable short circuit detect break signal

bool SDDF_ShortCircuitDetectConfigType::bShortCircuitDetectEnable

Whether to enable short circuit detect

bool SDDF_ShortCircuitDetectConfigType::bShortCircuitDetectIntEnable

Whether to enable short circuit detect interrupt

SDDF_ShortCircuitDetectModeType SDDF_ShortCircuitDetectConfigType::eShortCircuitDetectMode

Short circuit detect conversion mode

SDDF_ShortCircuitOptionType SDDF_ShortCircuitDetectConfigType::eShortCircuitDetectOption

Short circuit detect counting option

SDDF_EventDetectCallbackType SDDF_ShortCircuitDetectConfigType::pShortCircuitDetectNotify

The callback function for short circuit detect interrupt

uint8_t SDDF_ShortCircuitDetectConfigType::u8ShortCircuitDetectThreshold

Short circuit detect counter threshold

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SDDF_ZeroCrossDetectConfigType Struct Reference

The configuration of the zero cross detect.

```
#include <module_driver_sddf.h>
```

Public Attributes

bool **bZeroCrossDetectEnable**

SDDF_InputSrcType **eZeroCrossDetectInputSrc**

bool **bZeroCrossDetectIntEnable**

SDDF_EventDetectCallbackType **pZeroCrossDetectNotify**

Detailed Description

The configuration of the zero cross detect.

Member Data Documentation

bool SDDF_ZeroCrossDetectConfigType::bZeroCrossDetectEnable

Whether to enable zero cross detect

bool SDDF_ZeroCrossDetectConfigType::bZeroCrossDetectIntEnable

Whether to enable zero cross detect interrupt

SDDF_InputSrcType SDDF_ZeroCrossDetectConfigType::eZeroCrossDetectInputSrc

The result source of zero cross detect

SDDF_EventDetectCallbackType SDDF_ZeroCrossDetectConfigType::pZeroCrossDetectNotify

The callback function for zero cross detect interrupt

The documentation for this struct was generated from the following file:

module_driver_sddf.h

SEC_BootInfo Struct Reference

Indicate the boot information.

```
#include <module_driver_sec.h>
```

Public Attributes

bool **BootRom**

bool **NmiPinEn**

bool **SecBootEn**

bool **OscEn**

FastBoot_Speed FastBootSpeed

Osc_FreqInfo OscFreq

Detailed Description

Indicate the boot information.

Member Data Documentation

bool SEC_BootInfo::BootRom

true means - boot from rom.

FastBoot_Speed SEC_BootInfo::FastBootSpeed

the fast boot speed.

bool SEC_BootInfo::NmiPinEn

true means - nmi pin is active.The associated pin defaults to internal pull up enabled.

bool SEC_BootInfo::OscEn

true means - OSC is available.

Osc_FreqInfo SEC_BootInfo::OscFreq

Indicate the OSC Frequency

bool SEC_BootInfo::SecBootEn

true means - secure boot enable false means no secure boot mode..

The documentation for this struct was generated from the following file:

module_driver_sec.h

SEC_IspInfo Struct Reference

Indicate the ISP information.

```
#include <module_driver_sec.h>
```

Public Attributes

bool **IspModeEn**

bool **IspAuthEn**

Isp_Instance Ispinstance

FCUART_ISP_BAUDRATE Ispfcuartbaudrate

FLEXCAN_ISP_BAUDRATE Ispflexcanbaudrate

Detailed Description

Indicate the ISP information.

Member Data Documentation

bool SEC_IspInfo::IspAuthEn

*true means - ISP Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

FCUART_ISP_BAUDRATE SEC_IspInfo::Ispfcuartbaudrate

Indicate the FCUART Baud Rate for ISP

FLEXCAN_ISP_BAUDRATE SEC_IspInfo::Ispflexcanbaudrate

Indicate the FLEXCAN Baud Rate for ISP

Isp_Instance SEC_IspInfo::Ispinstance

the Isp instance

bool SEC_IspInfo::IspModeEn

true means - ISP mode is active.

The documentation for this struct was generated from the following file:

`module_driver_sec.h`

Sent_ChannelConfigType Struct Reference

structure to configure the SENT channel.

```
#include <module_driver_sent.h>
```

Public Attributes

```
uint8_t u8DataNibbleNumber
uint8_t u8DigitalFilterCount
uint16_t u16TickScaler
Sent_CalDiagOptionType eCalDiagOption
Sent_DataNibbleModeType eDataNibbleMode
Sent_CalibrationValidType eCalValid
Sent_IdleCountType eIdleCount
bool bFastMessageFifoEn
bool bTickCompensateEn
bool bUseAlternativeCrc
bool bFastMsgCRCCheckEn
bool bFastMsgDataChangeEn
bool bFastMsgCRCAugEn
bool bFastMsgCRCWithSCEn
bool bSlowMsgCRCAugEn
bool bPausePulseEn
bool bChannelEn
```

Detailed Description

structure to configure the SENT channel.

Member Data Documentation

bool Sent_ChannelConfigType::bChannelEn

Enable the SENT Channel.

bool Sent_ChannelConfigType::bFastMessageFifoEn

Enable Channel FIFO function.

bool Sent_ChannelConfigType::bFastMsgCRCAugEn

Augmentation is selected for Fast Message CRC calculation.

bool Sent_ChannelConfigType::bFastMsgCRCCheckEn

Enable Fast message CRC check.

bool Sent_ChannelConfigType::bFastMsgCRCWithSCEn

If enabled, the crc calculation will includes S&C nibble.

bool Sent_ChannelConfigType::bFastMsgDataChangeEn

If enabled, the SENT will only assert the fast message ready flag when received data has changed compared with previous data

bool Sent_ChannelConfigType::bPausePulseEn

If enabled, a pause pulse will be appended at the end of a frame.

bool Sent_ChannelConfigType::bSlowMsgCRCAugEn

Augmentation is selected for Short Serial Message CRC calculation.

bool Sent_ChannelConfigType::bTickCompensateEn

Enable the tick period compensate function.

bool Sent_ChannelConfigType::bUseAlternativeCrc

Use alternative 4-bit CRC algorithm instead of SAE CRC algorithm.

Sent_CalDiagOptionType Sent_ChannelConfigType::eCalDiagOption

Successive calibration pulses diagnostic option.

Sent_CalibrationValidType Sent_ChannelConfigType::eCalValid

Select valid Calibration pulse range

Sent_DataNibbleModeType Sent_ChannelConfigType::eDataNibbleMode

Refer to SENT SAE J2716 2010 for detail protocol definition.

Sent_IdleCountType Sent_ChannelConfigType::eldleCount

Select how long the bus idle flag will assert when bus is idle

uint16_t Sent_ChannelConfigType::u16TickScaler

The value plus 1 is used to divide the function clock to tick clock.

uint8_t Sent_ChannelConfigType::u8DataNibbleNumber

The received data number will be configured value plus 1, 0 and value bigger than 5 is forbidden.

uint8_t Sent_ChannelConfigType::u8DigitalFilterCount

The input signal needs keep stable for $(filt_cnt < 2) * function_clk$ cycles to pass through the digital filter.

The documentation for this struct was generated from the following file:

module_driver_sent.h

Sent_ChannelInterruptType Struct Reference

Structure to enable or disable the channel interrupts.

```
#include <module_driver_sent.h>
```

Public Attributes

- bool **bBusIdleITen**
- bool **bCalResyncErrITen**
- bool **bCalFailITen**
- bool **bSlowMsgOFITen**
- bool **bFastMsgOFITen**
- bool **bNibbleValueErrITen**
- bool **bPrePulseDiagErrITen**
- bool **bCalDiagErrITen**
- bool **bCalErrITen**
- bool **bSlowMsgCrcErrITen**
- bool **bFastMsgCrcErrITen**
- bool **bFallingEdgeNumErrITen**

Detailed Description

Structure to enable or disable the channel interrupts.

Member Data Documentation

bool Sent_ChannelInterruptType::bBusIdleITen

Bus idle interrupt enable.

bool Sent_ChannelInterruptType::bCalDiagErrITen

Calibration diagnosis over 25% error interrupt enable.

bool Sent_ChannelInterruptType::bCalErrITen

Sync/calibration pulse difference exceed 1.5625% error interrupt enable.

bool Sent_ChannelInterruptType::bCalFailITen

Calibration 20% pass 25% fail interrupt enable.

bool Sent_ChannelInterruptType::bCalResyncErrITen

Continuous 2 sync/calibration pulse difference exceed 1.5625% error interrupt enable.

bool Sent_ChannelInterruptType::bFallingEdgeNumErrITen

Falling edge number error interrupt enable.

bool Sent_ChannelInterruptType::bFastMsgCrcErrITen

Fast message CRC check error interrupt enable.

bool Sent_ChannelInterruptType::bFastMsgOFITen

Fast message overflow interrupt enable.

bool Sent_ChannelInterruptType::bNibbleValueErrITen

Nibble value more than 15 or less than 0 error interrupt enable.

bool Sent_ChannelInterruptType::bPrePulseDiagErrITen

Previous pulse diagnosis error interrupt enable.

bool Sent_ChannelInterruptType::bSlowMsgCrcErrITen

Slow message CRC check error interrupt enable.

bool Sent_ChannelInterruptType::bSlowMsgOFITen

Slow message overflow interrupt enable.

The documentation for this struct was generated from the following file:

module_driver_sent.h

Sent_ConfigType Struct Reference

structure to configure the SENT.

```
#include <module_driver_sent.h>
```

Public Attributes

uint8_t **u8PreScaler**

uint8_t **u8DmaWaterMark**

bool **bDebugModeEn**

bool **bAutoClearReadyFlag**

SENT_GBISRCallbackType **pGBCallback**

SENT_CHISRCallbackType **pCHCallback**

Detailed Description

structure to configure the SENT.

Member Data Documentation

bool Sent_ConfigType::bAutoClearReadyFlag

When enabled after data ready read of received data, crc and timestamp register will prevent the assert of data overflow flag.

bool Sent_ConfigType::bDebugModeEn

If enable, the SENT will keep running under debug mode.

SENT_CHISRCallbackType Sent_ConfigType::pCHCallback

Channel interrupt callback function

SENT_GBISRCallbackType Sent_ConfigType::pGBCallback

Global interrupt callback function

uint8_t Sent_ConfigType::u8DmaWaterMark

When Fast message DMA request is enabled, the request will asserts if data number in FIFO is more than watermark

uint8_t Sent_ConfigType::u8PreScaler

This value is used to divide the function clock to count the timestamp

The documentation for this struct was generated from the following file:

module_driver_sent.h

Sent_DMAConfigType Struct Reference

structure to configure the SENT channel DMA.

```
#include <module_driver_sent.h>
```

Public Attributes

bool **bFastMsgDmaEn**

bool **bSlowMsgDmaEn**

Detailed Description

structure to configure the SENT channel DMA.

Member Data Documentation

bool Sent_DMAConfigType::bFastMsgDmaEn

Fast message DMA request enable.

bool Sent_DMAConfigType::bSlowMsgDmaEn

Slow message DMA request enable.

The documentation for this struct was generated from the following file:

`module_driver_sent.h`

Sent_FastMessageDataType Struct Reference

Structure for fast message buffer.

```
#include <module_driver_sent.h>
```

Public Attributes

uint8_t **u8CRC**

uint8_t **u8SC**

uint32_t **u32Timestamp**

uint32_t **u32Data**

Detailed Description

Structure for fast message buffer.

Member Data Documentation

uint32_t Sent_FastMessageDataType::u32Data

Received fast message data nibble.

uint32_t Sent_FastMessageDataType::u32Timestamp

Fast message timestamp value.

uint8_t Sent_FastMessageDataType::u8CRC

Receied fast message CRC data.

uint8_t Sent_FastMessageDataType::u8SC

Fast message status communication nibble value.

The documentation for this struct was generated from the following file:

module_driver_sent.h

Sent_SlowMessageDataType Struct Reference

Structure for slow message buffer.

```
#include <module_driver_sent.h>
```

Public Attributes

Sent_SerialMessageType eMsgType

uint8_t **u8CRC**

uint8_t **u8ID**

uint16_t **u16Data**

uint32_t **u32Timestamp**

Detailed Description

Structure for slow message buffer.

Member Data Documentation

Sent_SerialMessageType Sent_SlowMessageDataType::eMsgType

Slow message type.

uint16_t Sent_SlowMessageDataType::u16Data

Slow message data.

uint32_t Sent_SlowMessageDataType::u32Timestamp

Slow message timestamp value.

uint8_t Sent_SlowMessageDataType::u8CRC

Slow message crc data.

uint8_t Sent_SlowMessageDataType::u8ID

Slow message ID.

The documentation for this struct was generated from the following file:

`module_driver_sent.h`

Sent_SpcConfigType Struct Reference

structure to configure the SENT channel SPC.

```
#include <module_driver_sent.h>
```

Public Attributes

uint8_t **u8PulseWidth**

uint8_t **u8PulseDelay**

Sent_SpcTickBaseType **eSpcTickBase**

Sent_SpcTriggerType **eSpcTrigger**

bool **bCalDiagEn**

bool **bSpcModeEn**

Detailed Description

structure to configure the SENT channel SPC.

Member Data Documentation

bool **Sent_SpcConfigType::bCalDiagEn**

Enable Calibration diagnosis.

bool **Sent_SpcConfigType::bSpcModeEn**

Enable SPC mode.

Sent_SpcTickBaseType **Sent_SpcConfigType::eSpcTickBase**

Select the tick base of SPC

Sent_SpcTriggerType **Sent_SpcConfigType::eSpcTrigger**

Select the SPC pulse trigger type.

uint8_t **Sent_SpcConfigType::u8PulseDelay**

Configure the delay between SPC-trigger assert and finally generate a SPC pulse.

uint8_t Sent_SpcConfigType::u8PulseWidth

Configure the width of the SPC pulse.

The documentation for this struct was generated from the following file:

module_driver_sent.h

STCU_ConfigType Struct Reference

```
#include <module_driver_stcu.h>
```

Public Attributes

```
bool bLbistEn  
bool bMbistLPC  
bool bMbistEn  
bool bMbistFullTest  
bool bMbistSramInit  
bool bInterruptEn  
STCU_PortPullModeType ePortPullMode  
STCU_ClkSourceType eClkSource  
STCU_LbistClkDivType eLbistClkDivider  
uint16_t u16MaxTime  
uint32_t u32MbistSel  
Stcu_IRQCallback plrCallback
```

Member Data Documentation

bool STCU_ConfigType::bInterruptEn

! STCU Interrupt Enable >

bool STCU_ConfigType::bLbistEn

! LBIST Enable >

bool STCU_ConfigType::bMbistEn

! MBIST Enable >

bool STCU_ConfigType::bMbistFullTest

! MBIST Full test Enable >

bool STCU_ConfigType::bMbistLPC

! MBIST Low Power Control Enable >

bool STCU_ConfigType::bMbistSramInit

! Enable SRAM initialization at the end of power on self-test. >

STCU_ClkSourceType STCU_ConfigType::eClkSource

! Select the clock source of self-test >

STCU_LbistClkDivType STCU_ConfigType::eLbistClkDivider

! LBIST Clock Divider Control. Note:A slower LBIST clock will cause less power consumption but a longer test time. >

STCU_PortPullModeType STCU_ConfigType::ePortPullMode

! Self-test Port Pull Selection >

Stcu_IRQCallback STCU_ConfigType::pIrqCallback

! Callback function for STCU IRQ >

uint16_t STCU_ConfigType::u16MaxTime

! The maximum time allowed for self-test execution.(unit : ms) >

uint32_t STCU_ConfigType::u32MbistSel

! Refer to "STCU_MbistSelType", use OR to select the MBIST range.

The documentation for this struct was generated from the following file:

module_driver_stcu.h

STCU_InitRamConfigType Struct Reference

```
#include <module_driver_stcu.h>
```

Public Attributes

uint8_t **initMode**

uint8_t **bLockAfterEn**

uint32_t **u32InitRamType**

Member Data Documentation

uint8_t STCU_InitRamConfigType::bLockAfterEn

Lock STCU after RAM Initial

uint8_t STCU_InitRamConfigType::initMode

RAM Initial Mode

uint32_t STCU_InitRamConfigType::u32InitRamType

RAM Type, DTCM, ITCM, SRAM, refer to "STCU_InitRamType" and use OR to combine them

The documentation for this struct was generated from the following file:

`module_driver_stcu.h`

TMU_InitType Struct Reference

Defines the temperature sensor configuration.

```
#include <module_driver_tmu.h>
```

Public Attributes

```
bool bTemperatureOver150IntEn  
bool bTemperatureOver125IntEn  
bool bFlagTempReadyIntEn  
TMU_HysteresisType eFlagTempHysteresisCon  
TMU_bypassType eFlagTempFilterBypassCon  
bool bVoltageTempReadyIntEn
```

Detailed Description

Defines the temperature sensor configuration.

This structure is used to configure for Flag-based temperature sensor and Voltage-based temperature sensor

Implements : **TMU_InitType**

Member Data Documentation

bool TMU_InitType::bFlagTempReadyIntEn

Enable interrupt when Flag-based temperature sensor ready

bool TMU_InitType::bTemperatureOver125IntEn

Enable interrupt when Temperature over 125 Celsius

bool TMU_InitType::bTemperatureOver150IntEn

Enable interrupt when Temperature over 150 Celsius

bool TMU_InitType::bVoltageTempReadyIntEn

Enable interrupt when Voltage-based temperature sensor ready

TMU_bypassType TMU_InitType::eFlagTempFilterBypassCon

Flag-based temperature sensor filter bypass control

TMU_HysteresisType TMU_InitType::eFlagTempHysteresisCon

Flag-based temperature sensor hysteresis control (on,off)

The documentation for this struct was generated from the following file:

module_driver_tmu.h

TSTMP_InitType Struct Reference

TSTMP initialization type.

```
#include <module_driver_tstmp.h>
```

Public Attributes

uint32_t **u32Modulate0Value**

uint32_t **u32Modulate1Value**

uint32_t **u32Modulate2Value**

uint32_t **u32Modulate3Value**

Detailed Description

TSTMP initialization type.

Member Data Documentation

uint32_t TSTMP_InitType::u32Modulate0Value

modulate 0 value

uint32_t TSTMP_InitType::u32Modulate1Value

modulate 1 value

uint32_t TSTMP_InitType::u32Modulate2Value

modulate 2 value

uint32_t TSTMP_InitType::u32Modulate3Value

modulate 3 value

The documentation for this struct was generated from the following file:

module_driver_tstmp.h

TSTMP_IntType Struct Reference

TSTMP interrupt type.

```
#include <module_driver_tstmp.h>
```

Public Attributes

```
bool bModulateIntEn [MAX_MOD_NUMBER]
```

```
void(* pChannelCallback [MAX_MOD_NUMBER])(TSTMP_HandleType *pHandle, uint32_t u32channel)
```

Detailed Description

TSTMP interrupt type.

Member Data Documentation

```
bool TSTMP_IntType::bModulateIntEn[MAX_MOD_NUMBER]
```

whether use the modulate n interrupt

```
void(* TSTMP_IntType::pChannelCallback[MAX_MOD_NUMBER])(TSTMP_HandleType *pHandle, uint32_t  
u32channel)
```

modulate n interrupt function pointer

The documentation for this struct was generated from the following file:

```
module_driver_tstmp.h
```

TSTMP_UpdateType Struct Reference

TSTMP update type.

```
#include <module_driver_tstmp.h>
```

Public Attributes

TSTMP_ModChannelType eMod

uint32_t u32ModValue

void(* pChannelCallback)(TSTMP_HandleType *pHandle, uint32_t u32channel)

bool bIntEn

Detailed Description

TSTMP update type.

Member Data Documentation

bool TSTMP_UpdateType::bIntEn

whether enable TSTMP modulate interrupt

TSTMP_ModChannelType TSTMP_UpdateType::eMod

TSTMP modulate

void(* TSTMP_UpdateType::pChannelCallback) (TSTMP_HandleType *pHandle, uint32_t u32channel)

modulate n interrupt function pointer

uint32_t TSTMP_UpdateType::u32ModValue

TSTMP modulate value

The documentation for this struct was generated from the following file:

module_driver_tstmp.h

WDOG_CfgType Struct Reference

WDOG init module type, include register setting for WDOG_CS/WDOG_TIMEOUT/WDOG_WINDOW.

#include <module_driver_wdog.h>

Public Attributes

bool **bWinEnable**

bool **bPrescalerEnable**

bool **bInterruptEnable**

WDOG_ClkSelType **eClkSource**

WDOG_TimeoutReactionType **eTimeoutReaction**

bool **bEnable**

WDOG_TestType **eTesttype**

bool **bEnableInDebug**

bool **bEnableInWait**

bool **bEnableInStop**

uint16_t **u16WindowValue**

uint16_t **u16TimeoutValue**

Detailed Description

WDOG init module type, include register setting for WDOG_CS/WDOG_TIMEOUT/WDOG_WINDOW.

Member Data Documentation

bool WDOG_CfgType::bEnable

@ WDOG timeout reaction.

bool WDOG_CfgType::bEnableInDebug

@ WDOG_CS[TST]

bool WDOG_CfgType::bEnableInStop

@ WDOG_CS[WAIT]

bool WDOG_CfgType::bEnableInWait

@ WDOG_CS[DBG]

bool WDOG_CfgType::bInterruptEnable

@ fixed 256 prescaler from clock input

bool WDOG_CfgType::bPrescalerEnable

@ windowed watch dog mode

bool WDOG_CfgType::bWinEnable

WDOG_ClkSelType WDOG_CfgType::eClkSource

@ WDOG Interrupt, W1C bit

WDOG_TestType WDOG_CfgType::eTesttype

@ WDOG enable bit

WDOG_TimeoutReactionType WDOG_CfgType::eTimeoutReaction

@ WDOG clock source

uint16_t WDOG_CfgType::u16TimeoutValue

@ The window value

uint16_t WDOG_CfgType::u16WindowValue

@ WDOG_CS[STOP]

The documentation for this struct was generated from the following file:

module_driver_wdog.h

File Documentation

HwA_adc.h File Reference

Hardware access layer for ADC.

```
#include "device_header.h"
```

Include dependency graph for HwA_adc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum ADC_ResolutionType { ADC_RESOLUTION_12_BIT = 0U, ADC_RESOLUTION_10_BIT = 1U,  
ADC_RESOLUTION_8_BIT = 2U }
```

Select the ADC Resolution.

```
enum ADC_AlignType { ADC_ALIGN_RIGHT = 0U, ADC_ALIGN_LEFT = 1U }
```

Select the ADC result alignment.

```
enum ADC_TriggerModeType { ADC_TRGMODE_SW = 0U, ADC_TRGMODE_RISING_EDGE = 1U,  
ADC_TRGMODE_FALLING_EDGE = 2U, ADC_TRGMODE_BOTH_EDGE = 3U,  
ADC_TRGMODE_HIGH_VOLTAGE = 4U, ADC_TRGMODE_LOW_VOLTAGE = 5U }
```

Select the ADC trigger mode.

```
enum ADC_SequenceModeType { ADC_SEQMODE_SINGLE = 0U, ADC_SEQMODE_CONTINUOUS = 1U,  
ADC_SEQMODE_DISCONTINUOUS_0 = 2U, ADC_SEQMODE_DISCONTINUOUS_1 = 3U }
```

Select the ADC sequence mode.

```
enum ADC_OvrModeType { ADC_OVERRUN_MODE_PRESERVE = 0U, ADC_OVERRUN_MODE_OVERWRITE  
= 1U }
```

Select the ADC overrun management mode.

```
enum ADC_RefType { ADC_REF_INTERNAL = 0U, ADC_REF_EXTERNAL = 1U }
```

Select the ADC voltage reference source.

```
enum ADC_AverageLenType { ADC_AVERAGE_LEN_4 = 0U, ADC_AVERAGE_LEN_8 = 1U,  
ADC_AVERAGE_LEN_16 = 2U, ADC_AVERAGE_LEN_32 = 3U }
```

Select the ADC hardware average samples.

enum **ADC_ClockDivideType** { **ADC_CLOCK_DIV_1** = 0U, **ADC_CLOCK_DIV_2** = 1U, **ADC_CLOCK_DIV_4** = 2U, **ADC_CLOCK_DIV_8** = 3U }

Set the ADC clock divider.

enum **ADC_TrgSrcType** { **ADC_TRGSRC_PTIMER** = 0U, **ADC_TRGSRC_TRGSEL** = 2U }

The trigger source of the ADC instance.

enum **ADC_CmpChannelType** { **ADC_CMP_CHANNEL_ALL** = 0U, **ADC_CMP_CHANNEL_SINGLE** = 1U }

Select the channel compare mode.

Functions

LOCAL_INLINE bool **ADC_HWA_GetCompareFlag** (const ADC_Type *const pAdc)

Check whether the conversion result is in the comparing range.

LOCAL_INLINE void **ADC_HWA_ClearCompareFlag** (ADC_Type *const pAdc)

Clear the compare flag.

LOCAL_INLINE bool **ADC_HWA_GetOverRunFlag** (const ADC_Type *const pAdc)

Get the overrun status of the ADC instance.

LOCAL_INLINE void **ADC_HWA_ClearOverRunFlag** (ADC_Type *const pAdc)

Clear the overrun flag of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSequenceFlag** (const ADC_Type *const pAdc)

Check whether the ADC conversion sequence is finished.

LOCAL_INLINE void **ADC_HWA_ClearEndOfSequenceFlag** (ADC_Type *const pAdc)

Clear the ADC conversion sequence complete flag.

LOCAL_INLINE bool **ADC_HWA_GetEndOfConversionFlag** (const ADC_Type *const pAdc)

Check whether the current ADC conversion is finished.

LOCAL_INLINE void **ADC_HWA_ClearEndOfConversionFlag** (ADC_Type *const pAdc)

Clear the ADC single conversion complete flag.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSampleFlag** (const ADC_Type *const pAdc)

Check whether the sampling phase of the current ADC conversion is finished.

LOCAL_INLINE void **ADC_HWA_ClearEndOfSampleFlag** (ADC_Type *const pAdc)

Clear the ADC sampling complete flag.

LOCAL_INLINE bool **ADC_HWA_GetFIFOReadyFlag** (const ADC_Type *const pAdc)

Check whether the data quantity in the FIFO is greater than watermark.

LOCAL_INLINE bool **ADC_HWA_GetFIFOFullFlag** (const ADC_Type *const pAdc)

Check whether the ADC FIFO is full.

LOCAL_INLINE bool **ADC_HWA_GetFIFOEmptyFlag** (const ADC_Type *const pAdc)

Check whether the ADC FIFO is empty.

LOCAL_INLINE bool **ADC_HWA_GetReady** (const ADC_Type *const pAdc)

Check whether the ADC instance is ready to operate.

LOCAL_INLINE void **ADC_HWA_ClearReady** (ADC_Type *const pAdc)

Clear the ready flag of the ADC instance.

LOCAL_INLINE void **ADC_HWA_ClearFlags** (ADC_Type *const pAdc, uint32_t u32Flags)

Clear the ready flag of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetFIFOReadyInterruptFlag** (const ADC_Type *const pAdc)

Get the FIFO Ready interrupt flag If enabled, ADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE void **ADC_HWA_SetFIFOReadyInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the FIFO Ready interrupt flag If enabled, ADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE bool **ADC_HWA_GetCompareInterruptFlag** (const ADC_Type *const pAdc)

Get the Compare interrupt flag If enabled, ADC interrupt is generated when the ADC conversion result is not within the compare threshold.

LOCAL_INLINE void **ADC_HWA_SetCompareInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the Compare interrupt flag If enabled, ADC interrupt is generated when the ADC conversion result is not within the compare threshold.

LOCAL_INLINE bool **ADC_HWA_GetOverrunInterruptFlag** (const ADC_Type *const pAdc)

Get the Overrun interrupt flag If enabled, ADC interrupt is generated when the ADC instance is overrun.

LOCAL_INLINE void **ADC_HWA_SetOverrunInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the Overrun interrupt flag If enabled, ADC interrupt is generated when the ADC instance is overrun.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSequenceInterruptFlag** (const ADC_Type *const pAdc)

Get the End of Sequence interrupt enable flag If enabled, ADC interrupt is generated when the ADC sequence conversion is completed.

LOCAL_INLINE void **ADC_HWA_SetEndOfSequenceInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the End of Sequence interrupt enable flag If enabled, ADC interrupt is generated when the ADC sequence conversion is completed.

LOCAL_INLINE bool **ADC_HWA_GetEndOfConversionInterruptFlag** (const ADC_Type *const pAdc)

Get the conversion complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion is completed.

LOCAL_INLINE void **ADC_HWA_SetEndOfConversionInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the conversion complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion is completed.

LOCAL_INLINE bool **ADC_HWA_GetEndOfSampleInterruptFlag** (const ADC_Type *const pAdc)

Get the sample complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion finished the sampling phase.

LOCAL_INLINE void **ADC_HWA_SetEndOfSampleInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the sample complete interrupt enable flag If enabled, ADC interrupt is generated when each ADC conversion finished the sampling phase.

LOCAL_INLINE bool **ADC_HWA_GetADCReadyInterruptFlag** (const ADC_Type *const pAdc)

Get the ADC ready interrupt enable flag If enabled, ADC interrupt is generated when the ADC module is ready for conversion.

LOCAL_INLINE void **ADC_HWA_SetADCReadyInterruptFlag** (ADC_Type *const pAdc, bool bEnable)

Set the ADC ready interrupt enable flag If enabled, ADC interrupt is generated when the ADC module is ready for conversion.

LOCAL_INLINE uint32_t **ADC_HWA_GetInterruptEnable** (const ADC_Type *const pAdc)

Get the interrupt enable config.

LOCAL_INLINE void **ADC_HWA_SetInterruptEnable** (ADC_Type *const pAdc, uint32_t u32InterruptCfg)

Set the interrupt enable.

LOCAL_INLINE void **ADC_HWA_Reset** (ADC_Type *const pAdc)

Reset the ADC hardware.

LOCAL_INLINE bool **ADC_HWA_GetEnable** (const ADC_Type *const pAdc)

Get the enable status of the ADC instance.

LOCAL_INLINE void **ADC_HWA_Enable** (ADC_Type *const pAdc)

Enable the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetDisable** (const ADC_Type *const pAdc)

Get whether the ADC instance is in disable status.

LOCAL_INLINE void **ADC_HWA_Disable** (ADC_Type *const pAdc)

Disable the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetStart** (const ADC_Type *const pAdc)

Get the conversion start status of the ADC instance.

LOCAL_INLINE void **ADC_HWA_Start** (ADC_Type *const pAdc)

Start the ADC conversion.

LOCAL_INLINE bool **ADC_HWA_GetStop** (const ADC_Type *const pAdc)

Get whether ADC is in stopping status.

LOCAL_INLINE void **ADC_HWA_Stop** (ADC_Type *const pAdc)

Stop the ADC conversion.

LOCAL_INLINE ADC_OvrModeType **ADC_HWA_GetOverrunManagementMode** (const ADC_Type *const pAdc)

Get the overrun management mode.

LOCAL_INLINE void **ADC_HWA_SetOverrunManagementMode** (ADC_Type *const pAdc, ADC_OvrModeType eOvrMode)

Set the overrun management mode.

LOCAL_INLINE uint8_t **ADC_HWA_GetSequenceLength** (const ADC_Type *const pAdc)

Get the sequence length of the ADC conversion sequence.

LOCAL_INLINE void **ADC_HWA_SetSequenceLength** (ADC_Type *const pAdc, uint8_t u8SequenceLength)

Set the sequence length of the ADC conversion sequence.

LOCAL_INLINE ADC_SeqModeType **ADC_HWA_GetSequenceMode** (const ADC_Type *const pAdc)

Get the ADC sequence mode.

LOCAL_INLINE void **ADC_HWA_SetSequenceMode** (ADC_Type *const pAdc, ADC_SeqModeType eSequenceMode)

Set the ADC sequence mode.

LOCAL_INLINE bool **ADC_HWA_GetAutoDisableModeEnableFlag** (const ADC_Type *const pAdc)

Get whether auto disable is enabled.

LOCAL_INLINE void **ADC_HWA_SetAutoDisableModeEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Set whether to enable auto disable mode.

LOCAL_INLINE bool **ADC_HWA_GetWaitConversionModeFlag** (const ADC_Type *const pAdc)

Get whether the wait conversion mode is enabled.

LOCAL_INLINE void **ADC_HWA_SetWaitConversionModeFlag** (ADC_Type *const pAdc, bool bEnable)

Set the wait conversion mode.

LOCAL_INLINE ADC_TriggerSrcType **ADC_HWA_GetTriggerSource** (const ADC_Type *const pAdc)

Get the trigger source the the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetTriggerSource** (ADC_Type *const pAdc, ADC_TriggerSrcType eTrgSource)

Set the trigger source the the ADC instance.

LOCAL_INLINE ADC_TriggerModeType **ADC_HWA_GetTriggerMode** (const ADC_Type *const pAdc)

Get the trigger mode of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetTriggerMode** (ADC_Type *const pAdc, ADC_TriggerModeType eTrgMode)

Set the trigger mode of the ADC instance.

LOCAL_INLINE ADC_AlignType **ADC_HWA_GetDataAlignment** (const ADC_Type *const pAdc)

Get the data align mode.

LOCAL_INLINE void **ADC_HWA_SetDataAlignment** (ADC_Type *const pAdc, ADC_AlignType eAlign)

Set the data align mode.

LOCAL_INLINE ADC_ResolutionType **ADC_HWA_GetDataResolution** (const ADC_Type *const pAdc)

Get the ADC resolution of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetDataResolution** (ADC_Type *const pAdc, ADC_ResolutionType eResolution)

Set the resolution of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetDMAEnableFlag** (const ADC_Type *const pAdc)

Get whether DMA for the ADC instance is enabled.

LOCAL_INLINE void **ADC_HWA_SetDMAEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Enable or disable the DMA for the ADC instance.

LOCAL_INLINE uint32_t **ADC_HWA_GetConfig1** (const ADC_Type *const pAdc)

Get the ADC_CFG1 config.

LOCAL_INLINE void **ADC_HWA_SetConfig1** (ADC_Type *const pAdc, uint32_t u32Config)

Set the ADC_CFG1 config.

LOCAL_INLINE uint8_t **ADC_HWA_GetFIFOWaterMark** (const ADC_Type *const pAdc)

Get the FIFO water mark settings for the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetFIFOWaterMark** (ADC_Type *const pAdc, uint8_t u8WaterMark)

Set the FIFO water mark for the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetAverageEnableFlag** (const ADC_Type *const pAdc)

Get whether hardware average is enabled.

LOCAL_INLINE void **ADC_HWA_SetAverageEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Enable or disable hardware average for the ADC instance.

LOCAL_INLINE ADC_AverageLenType **ADC_HWA_GetAverageNumber** (const ADC_Type *const pAdc)

Get the hardware average number.

LOCAL_INLINE void **ADC_HWA_SetAverageNumber** (ADC_Type *const pAdc, ADC_AverageLenType eAverageNumber)

Set the hardware average number.

LOCAL_INLINE bool **ADC_HWA_GetClockGatingAck** (const ADC_Type *const pAdc)

Whether clock gating is acknowledged.

LOCAL_INLINE bool **ADC_HWA_GetClockGatingEnableFlag** (const ADC_Type *const pAdc)

Get whether clock gating is enabled.

LOCAL_INLINE void **ADC_HWA_SetClockGatingEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Set whether to enable ADC clock gating.

LOCAL_INLINE ADC_RefType **ADC_HWA_GetVoltageReference** (const ADC_Type *const pAdc)

Get the voltage reference of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetVoltageReference** (ADC_Type *const pAdc, ADC_RefType eRefSel)

Set the voltage reference of the ADC instance.

LOCAL_INLINE ADC_ClockDivideType **ADC_HWA_GetClockDivider** (const ADC_Type *const pAdc)

Get the ADC clock divider.

LOCAL_INLINE void **ADC_HWA_SetClockDivider** (ADC_Type *const pAdc, ADC_ClockDivideType eDivider)

Set the ADC clock divider.

LOCAL_INLINE uint8_t **ADC_HWA_GetStartupCnt** (const ADC_Type *const pAdc)

Get the ADC start up count.

LOCAL_INLINE void **ADC_HWA_SetStartupCnt** (ADC_Type *const pAdc, uint8_t u8StartupCnt)

Set the ADC start up count.

LOCAL_INLINE uint32_t **ADC_HWA_GetConfig2** (const ADC_Type *const pAdc)

Get the ADC_CFG2 config.

LOCAL_INLINE void **ADC_HWA_SetConfig2** (ADC_Type *const pAdc, uint32_t u32Config)

Set the ADC_CFG2 config.

LOCAL_INLINE uint8_t **ADC_HWA_GetSampleTime** (ADC_Type *const pAdc, uint8_t u8Selection)

Get the sample time of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetSampleTime** (ADC_Type *const pAdc, uint8_t u8Selection, uint8_t u8SampleTime)

Set the sample time of the ADC instance.

LOCAL_INLINE void **ADC_HWA_SetHwCompareChannel** (ADC_Type *const pAdc, **ADC_CmpChannelType** eChannelType, uint8_t u8ChannalNum)

Set the ADC hardware compare channel.

LOCAL_INLINE bool **ADC_HWA_GetHwCompareEnableFlag** (const ADC_Type *const pAdc)

Get whether hardware compare is enabled.

LOCAL_INLINE void **ADC_HWA_SetHwCompareEnableFlag** (ADC_Type *const pAdc, bool bEnable)

Set to enable or disable the hardware compare.

LOCAL_INLINE void **ADC_HWA_SetHwCompareThreshold** (ADC_Type *const pAdc, uint16_t u16LowThres, uint16_t u16HighThres)

Set the ADC hardware compare threshold.

LOCAL_INLINE uint32_t **ADC_HWA_GetFIFOData** (const ADC_Type *const pAdc)

Get the conversion result FIFO data of the ADC instance.

LOCAL_INLINE uint8_t **ADC_HWA_GetChannelSampleTimeIndex** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Get the sample time index of the ADC channel.

LOCAL_INLINE void **ADC_HWA_SetChannelSampleTimeIndex** (ADC_Type *const pAdc, const uint8_t u8ChnIdx, uint8_t u8SmpSel)

Set the sample time index of the ADC channel.

LOCAL_INLINE bool **ADC_HWA_GetChannelConvertComplete** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Get the channel conversion complete status of the ADC instance.

LOCAL_INLINE bool **ADC_HWA_GetChannelInterruptEnable** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Check whether interrupt is enabled on the selected ADC channel.

LOCAL_INLINE void **ADC_HWA_SetChannelInterruptEnable** (ADC_Type *const pAdc, const uint8_t u8ChnIdx, bool bEnable)

Set to enable or disable interrupt on the selected ADC channel.

LOCAL_INLINE uint8_t **ADC_HWA_GetChannelInput** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Get the input channel of the selected ADC channel.

LOCAL_INLINE void **ADC_HWA_SetChannelInput** (ADC_Type *const pAdc, const uint8_t u8ChnIdx, uint8_t u8InputChann)

Set the input channel to the selected ADC channel.

LOCAL_INLINE uint32_t **ADC_HWA_GetChannelData** (const ADC_Type *const pAdc, const uint8_t u8ChnIdx)

Get the conversion result data of the ADC instance.

Detailed Description

Hardware access layer for ADC.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_aontimer.h File Reference

aontimer hardware access layer

```
#include "device_header.h"
```

Include dependency graph for HwA_aontimer.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum AONTIMER_PulseClkSrcType { AONTIMER_CLK0_PIN = 0, AONTIMER_CLK1_PIN,  
AONTIMER_CLK2_PIN, AONTIMER_TRGSEL_OUTPUT }
```

The clock source of the pulse mode.

```
enum AONTIMER_ClkSrcType { AONTIMER_SIRC_1MHZ = 0U, AONTIMER_RTC_CLK = 2U,  
AONTIMER_IRC_CLK = 3U }
```

Aontimer clock source, please refer to Reference Manual chapter8.Aontimer for details.

```
enum AONTIMER_PulsePolarityType { AONTIMER_PulsePolarityType_HIGH = 0,  
AONTIMER_PulsePolarityType_LOW }
```

The polarity of pulse mode.

Functions

```
LOCAL_INLINE void AONTIMER_HWA_ConfigModule (AONTIMER_Type *const pAontimer, uint32_t  
u32RegValue)
```

Configure AONTIMER module.

```
LOCAL_INLINE void AONTIMER_HWA_ConfigModulePrescale (AONTIMER_Type *const pAontimer, uint32_t  
u32RegValue)
```

Configure AONTIMER module prescale.

```
LOCAL_INLINE void AONTIMER_HWA_SetModuleCompareValue (AONTIMER_Type *const pAontimer,  
uint32_t u32RegValue)
```

Set AONTIMER compare value.

LOCAL_INLINE void **AONTIMER_HWA_SetModuleCounterValue** (AONTIMER_Type *const pAontimer, uint32_t u32RegValue)

Set AONTIMER current counter value.

LOCAL_INLINE void **AONTIMER_HWA_SetModuleRunOnDebug** (AONTIMER_Type *const pAontimer)

Set AONTIMER module running on debug mode.

LOCAL_INLINE void **AONTIMER_HWA_EnableModuleInterrupt** (AONTIMER_Type *const pAontimer)

Enable AONTIMER module interrupt.

LOCAL_INLINE void **AONTIMER_HWA_SelectClkSrcOnPulseMode** (AONTIMER_Type *const pAontimer, **AONTIMER_PulseClkSrcType** eClk)

Select AONTIMER module external clock source when timer configured to pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_ClearInterruptFlag** (AONTIMER_Type *const pAontimer)

Clear AONTIMER interrupt flags.

LOCAL_INLINE void **AONTIMER_HWA_SetModulePolarity** (AONTIMER_Type *const pAontimer)

Set AONTIMER module polarity. Pulse counter input source is active-low, and the CNR increments on falling-edge.

LOCAL_INLINE void **AONTIMER_HWA_ConfigModulePolarity** (AONTIMER_Type *const pAontimer, **AONTIMER_PulsePolarityType** ePol)

Configure AONTIMER module polarity. If ePol is 0:Pulse counter input source is active-high, and the CNR increments on rising-edge. If ePol is 1:Pulse counter input source is active-low, and the CNR increments on falling-edge.

LOCAL_INLINE void **AONTIMER_HWA_EnablePulseMode** (AONTIMER_Type *const pAontimer)

Enable AONTIMER module pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_EnableTimer** (AONTIMER_Type *const pAontimer)

Enable AONTIMER timer.

LOCAL_INLINE void **AONTIMER_HWA_SetPrescale** (AONTIMER_Type *const pAontimer, uint8_t u8PrescalerValue)

Set AONTIMER prescale.

LOCAL_INLINE void **AONTIMER_HWA_EnableBypassMode** (AONTIMER_Type *const pAontimer)

If enable bypass mode, the timer will bypass the prescaler in timer counter mode or glitch filter in pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_SelectModuleClkSrc** (AONTIMER_Type *const pAontimer, **AONTIMER_ClkSrcType** eClk)

Select AONTIMER module clock source.

LOCAL_INLINE void **AONTIMER_HWA_SetModuleStopOnDebug** (AONTIMER_Type *const pAontimer)

Set AONTIMER module stop on debug mode.

LOCAL_INLINE void **AONTIMER_HWA_DisableModuleInterrupt** (AONTIMER_Type *const pAontimer)

Disable AONTIMER module interrupt.

LOCAL_INLINE void **AONTIMER_HWA_ClearModuleMode** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module mode.

LOCAL_INLINE void **AONTIMER_HWA_ClearModulePolarity** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module polarity. Pulse counter input source is active-high, and the CNR increments on rising-edge.

LOCAL_INLINE void **AONTIMER_HWA_DisablePulseMode** (AONTIMER_Type *const pAontimer)

Disable AONTIMER module pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_DisableTimer** (AONTIMER_Type *const pAontimer)

Disable AONTIMER module timer.

LOCAL_INLINE void **AONTIMER_HWA_ClearPrescale** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module prescaler.

LOCAL_INLINE void **AONTIMER_HWA_DisableBypassMode** (AONTIMER_Type *const pAontimer)

If disable bypass mode, the timer will enable the prescaler in timer counter mode or glitch filter in pulse mode.

LOCAL_INLINE void **AONTIMER_HWA_ClearModuleClkSrc** (AONTIMER_Type *const pAontimer)

Clear AONTIMER module clock source.

Detailed Description

aontimer hardware access layer

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_cm7.h File Reference

cortex m4 hardware access layer

#include "device_header.h"

Include dependency graph for HwA_cm7.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **CM7_HWA_EnableDeepSleep** (void)
Enable deepsleep mode.

LOCAL_INLINE void **CM7_HWA_DisableDeepSleep** (void)
Disable deepsleep mode.

Detailed Description

cortex m4 hardware access layer

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

Function Documentation

LOCAL_INLINE void CM7_HWA_DisableDeepSleep (void)

Disable deepsleep mode.

LOCAL_INLINE void CM7_HWA_EnableDeepSleep (void)

Enable deepsleep mode.

HwA_cmp.h File Reference

CMP driver type definition and API.

```
#include "device_header.h"
```

Include dependency graph for HwA_cmp.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum CMP_ModeSelType { CMP_MODE_DISABLE = 0U, CMP_MODE_CONTINUOUS = 1U,  
CMP_MODE_SAMPLE_NONFILTER_EXTCLK = 2U, CMP_MODE_SAMPLE_NONFILTER_INTCLK = 3U,  
CMP_MODE_SAMPLE_FILTER_EXTCLK = 4U, CMP_MODE_SAMPLE_FILTER_INTCLK = 5U,  
CMP_MODE_WINDOW = 6U, CMP_MODE_WINDOW_RESAMPLE = 7U, CMP_MODE_WINDOW_FILTER = 8U,  
CMP_MODE_CHANNEL_SCAN = 9U }
```

The instance index of the CMP mode.

```
enum CMP_InstanceType { CMP_INSTANCE_0 = 0U, CMP_INSTANCE_1 = 1U, CMP_INSTANCE_2 = 2U }
```

The instance index of the CMP peripheral.

```
enum CMP_DacEnableSrcType { CMP_DACENABLE_DCR = 0U, CMP_DACENABLE_CCR0 = 1U }
```

The instance index of the CMP DAC enable select.

```
enum CMP_InvertType { CMP_NON_INVERT = 0U, CMP_INVERT = 1U }
```

The instance index of the CMP output invert.

```
enum CMP_OutSelectType { CMP_FILTEROUT = 0U, CMP_UNFILTEROUT = 1U }
```

The instance index of the CMP output select.

```
enum CMP_OutWinLevelType { CMP_HOLD = 0U, CMP_USERDEF = 1U }
```

The instance index of the CMP window level.

```
enum CMP_OutWinLevel_UserDefType { CMP_OUTWIN_0 = 0U, CMP_OUTWIN_1 = 1U }
```

The instance index of the window output under userdefine CMP window level.

```
enum CMP_EventType { CMP_RISINGEDGE = 0U, CMP_FALLINGEDGE = 1U, CMP_BOTHEDGES = 2U }
```

The instance index of the CMP Event caused window close.

```
enum CMP_FilterCntType { CMP_FILTERCNT_0 = 0U, CMP_FILTERCNT_1 = 1U, CMP_FILTERCNT_2 = 2U,  
CMP_FILTERCNT_3 = 3U, CMP_FILTERCNT_4 = 4U }
```


The instance index of the CMP filter count numbers.

```
enum CMP_HystCtrlType { CMP_HYSTCTRL_0 = 0U, CMP_HYSTCTRL_1 = 1U, CMP_HYSTCTRL_2 = 2U,  
CMP_HYSTCTRL_3 = 3U }
```

The instance index of the CMP hysteresis control.

```
enum CMP_PortSelType { CMP_PORTSEL_MUX_P = 0U, CMP_PORTSEL_MUX_N = 1U }
```

The fixed CMP port for reference in channel scan mode.

```
enum CMP_INSrcSelType { CMP_INSRCSEL_DAC = 0U, CMP_INSRCSEL_MUX = 1U }
```

The source of the CMP input.

```
enum CMP_MuxSelType { CMP_INSEL_MUX_IN0 = 0U, CMP_INSEL_MUX_IN1 = 1U, CMP_INSEL_MUX_IN2  
= 2U, CMP_INSEL_MUX_IN3 = 3U, CMP_INSEL_MUX_IN4 = 4U, CMP_INSEL_MUX_IN5 = 5U,  
CMP_INSEL_MUX_IN6 = 6U, CMP_INSEL_MUX_IN7 = 7U }
```

The instance index of the CMP input mux.

```
enum CMP_DacVinRefSelType { CMP_DACVINREF_H0 = 0U, CMP_DACVINREF_H1 = 1U }
```

The instance index of DAC high voltage reference select.

```
enum CMP_SpeedModeSelType { CMP_LOWSPEEDMODE = 0U, CMP_HIGHSPEEDMODE = 1U }
```

The instance index of high power mode select.

```
enum CMP_OutStatus { CMP_OUT_FALLING_EDGE = 0U, CMP_OUT_RISING_EDGE = 1U, CMP_OUT_NONE  
= 2U }
```

Defines CMP out status.

Functions

```
LOCAL_INLINE void CMP_HWA_SetEn (CMP_Type *const pCmp, bool bEnable)
```

set CMP enable

```
LOCAL_INLINE void CMP_HWA_SetEnLPMode (CMP_Type *const pCmp, bool bEnable)
```

STOP mode enable.

```
LOCAL_INLINE void CMP_HWA_SetDacEnableSrc (CMP_Type *const pCmp, CMP_DacEnableSrcType  
eType)
```

set DAC enable selection

```
LOCAL_INLINE void CMP_HWA_SetDma (CMP_Type *const pCmp, bool bEnable)
```

set CMP DMA enable

LOCAL_INLINE void **CMP_HWA_SetComparatorMode** (CMP_Type *const pCmp, **CMP_ModeSelType** eMode, uint8_t u8FilterPrd, **CMP_FilterCntType** eFilterCnt)

set CMP mode

LOCAL_INLINE void **CMP_HWA_SetCmpOutInvert** (CMP_Type *const pCmp, **CMP_InvertType** eType)

set CPM output invert

LOCAL_INLINE void **CMP_HWA_SetCmpOutSel** (CMP_Type *const pCmp, **CMP_OutSelectType** eType)

set CPM output filter/unfilter selection

LOCAL_INLINE void **CMP_HWA_SetEnCmpOutPack** (CMP_Type *const pCmp, bool bEnable)

set comparator output pin enable

LOCAL_INLINE void **CMP_HWA_SetCmpOutWinLevel** (CMP_Type *const pCmp, **CMP_OutWinLevelType** eType)

set CMPOUT_WIN level, when window is closed

LOCAL_INLINE void **CMP_HWA_SetCmpOutWin** (CMP_Type *const pCmp, **CMP_OutWinLevel_UserDefType** eType)

set CMPOUT_WIN level in user-define mode, when window is closed

LOCAL_INLINE void **CMP_HWA_SetEnWinSampleInvert** (CMP_Type *const pCmp, bool bEnable)

set invert the WINDOW/SAMPLE signal enable or not

LOCAL_INLINE void **CMP_HWA_SetEnEventCloseWin** (CMP_Type *const pCmp, bool bEnable)

WINDOW signal can or not be closed by CMPO event when window mode.

LOCAL_INLINE void **CMP_HWA_SetEventCloseWin** (CMP_Type *const pCmp, **CMP_EventType** eType)

set which CMPO event causes window close

LOCAL_INLINE void **CMP_HWA_SetCCR1** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CCR1 register

LOCAL_INLINE void **CMP_HWA_SetSpeedMode** (CMP_Type *const pCmp, **CMP_SpeedModeSelType** eMode)

set CMP power mode select

LOCAL_INLINE void **CMP_HWA_SetHystCtrl** (CMP_Type *const pCmp, **CMP_HystCtrlType** eType)

set Comparator hard block hysteresis control

LOCAL_INLINE void **CMP_HWA_SetPSelMux** (CMP_Type *const pCmp, **CMP_MuxSelType** eType)

set which input is selected for the positive mux

LOCAL_INLINE void **CMP_HWA_SetNSelMux** (CMP_Type *const pCmp, **CMP_MuxSelType** eType)

set which input is selected for the negative mux

LOCAL_INLINE void **CMP_HWA_SetINPSel** (CMP_Type *const pCmp, **CMP_INSrcSelType** eType)

set the input to the positive port of the comparator

LOCAL_INLINE void **CMP_HWA_SetINNSel** (CMP_Type *const pCmp, **CMP_INSrcSelType** eType)

set the input to the negative port of the comparator

LOCAL_INLINE void **CMP_HWA_SetCCR2** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CCR2 register

LOCAL_INLINE void **CMP_HWA_SetEnDac** (CMP_Type *const pCmp, bool bEnable)

set CMP DAC enable

LOCAL_INLINE void **CMP_HWA_SetVinRefSel** (CMP_Type *const pCmp, **CMP_DacVinRefSelType** eType)

set DAC reference voltage source

LOCAL_INLINE void **CMP_HWA_SetDacData** (CMP_Type *const pCmp, uint8_t u8Data)

set CMP Dac output

LOCAL_INLINE void **CMP_HWA_SetDCR** (CMP_Type *const pCmp, uint32_t u32RegVal)

set DCR register

LOCAL_INLINE void **CMP_HWA_SetIntEn_Rising** (CMP_Type *const pCmp, bool bEnable)

set comparator rising interrupt enable

LOCAL_INLINE bool **CMP_HWA_GetIntEn_Rising** (CMP_Type *const pCmp)

get comparator rising interrupt enable status

LOCAL_INLINE void **CMP_HWA_SetIntEn_Falling** (CMP_Type *const pCmp, bool bEnable)

set comparator falling interrupt enable

LOCAL_INLINE bool **CMP_HWA_GetIntEn_Falling** (CMP_Type *const pCmp)

get comparator falling interrupt enable status

LOCAL_INLINE void **CMP_HWA_SetIER** (CMP_Type *const pCmp, uint32_t u32RegVal)

set IER register

LOCAL_INLINE bool **CMP_HWA_GetIntFlag_Rising** (CMP_Type *const pCmp)

get CMP output rising edge status

LOCAL_INLINE bool **CMP_HWA_GetIntFlag_Falling** (CMP_Type *const pCmp)

get CMP output falling edge status

LOCAL_INLINE void **CMP_HWA_ClearIntFlag_Rising** (CMP_Type *const pCmp)

clear rising interrupt flag

LOCAL_INLINE void **CMP_HWA_ClearIntFlag_Falling** (CMP_Type *const pCmp)

clear falling interrupt flag

LOCAL_INLINE uint32_t **CMP_HWA_GetCmpOut** (CMP_Type *const pCmp)

get CMP filtered output

LOCAL_INLINE void **CMP_HWA_SetCSR** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSR register

LOCAL_INLINE void **CMP_HWA_SetIntEn_ChannelScan** (CMP_Type *const pCmp, bool bEnable)
set comparator channel scan interrupt enable

LOCAL_INLINE bool **CMP_HWA_GetIntEn_ChannelScan** (CMP_Type *const pCmp)
get comparator channel scan interrupt enable status

LOCAL_INLINE void **CMP_HWA_ClearIntFlag_ChannelScan** (CMP_Type *const pCmp)
clear channel scan interrupt flag

LOCAL_INLINE void **CMP_HWA_SetCSInitModulus** (CMP_Type *const pCmp, uint8_t u8Modulus)
set CMP and DAC initialization delay modulus

LOCAL_INLINE void **CMP_HWA_SetCSNSAM** (CMP_Type *const pCmp, uint8_t u8Nsam)
set number of clock cycles for sampling

LOCAL_INLINE void **CMP_HWA_SetCSEn** (CMP_Type *const pCmp, bool bEnable)
set CMP channel scan enable

LOCAL_INLINE void **CMP_HWA_SetCSCR0** (CMP_Type *const pCmp, uint32_t u32RegVal)
set CSCR0 register

LOCAL_INLINE void **CMP_HWA_SetCSFixedChannel** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel)
set channel scan fixed channel

LOCAL_INLINE void **CMP_HWA_SetCSFixedPort** (CMP_Type *const pCmp, **CMP_PortSelType** ePort)
set channel scan fixed port

LOCAL_INLINE void **CMP_HWA_SetCSChannelEn** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel, bool bEnable)
set channel scan channel enabled

LOCAL_INLINE void **CMP_HWA_SetCSCR1** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSCR1 register

LOCAL_INLINE bool **CMP_HWA_GetIntFlag_ChannelScan** (CMP_Type *const pCmp)

get CMP output channel scan status

LOCAL_INLINE void **CMP_HWA_SetCSChannelPresetstate** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel, bool bPresetstate)

set channel scan channel enabled

LOCAL_INLINE bool **CMP_HWA_GetCSChannelOut** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel)

get channel scan channel current state

LOCAL_INLINE void **CMP_HWA_ClearCSCompRes** (CMP_Type *const pCmp)

software clear channel scan comparison results

LOCAL_INLINE void **CMP_HWA_SetCSCompareResultACEn** (CMP_Type *const pCmp, bool bEnable)

software clear channel scan comparison results

LOCAL_INLINE void **CMP_HWA_SetCSCSR** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSCSR register

LOCAL_INLINE bool **CMP_HWA_GetCSActive** (CMP_Type *const pCmp)

get channel scan active status

LOCAL_INLINE bool **CMP_HWA_GetCSComparisonResultFlag** (CMP_Type *const pCmp, **CMP_MuxSelType** eChannel)

get channel scan comparison result changed flag

LOCAL_INLINE void **CMP_HWA_SetCSSR** (CMP_Type *const pCmp, uint32_t u32RegVal)

set CSSR register

Detailed Description

CMP driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_cmu.h File Reference

CMU Module Register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_cmu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **CMU_HWA_SetRefWindow** (CMU_Type *const pCmu, uint32_t u32Temp)
Set Reference Window value.

LOCAL_INLINE void **CMU_HWA_SetMinCnts** (CMU_Type *const pCmu, uint32_t u32Temp)
Set Minimun Counter value.

LOCAL_INLINE uint32_t **CMU_HWA_GetMinCnts** (const CMU_Type *const pCmu)
Get Minimun Counter value.

LOCAL_INLINE void **CMU_HWA_SetMaxCnts** (CMU_Type *const pCmu, uint32_t u32Temp)
Set Maximun Counter value.

LOCAL_INLINE uint32_t **CMU_HWA_GetMaxCnts** (const CMU_Type *const pCmu)
Get Maximun Counter value.

LOCAL_INLINE uint32_t **CMU_HWA_GetCount** (const CMU_Type *const pCmu)
Get Counter value.

LOCAL_INLINE void **CMU_HWA_SetPeriodWindow** (CMU_Type *const pCmu, uint32_t u32Temp)
Set period window Counter.

LOCAL_INLINE void **CMU_HWA_SetPeriodEnable** (CMU_Type *const pCmu, bool bEnable)

Set period enable bit.

LOCAL_INLINE bool **CMU_HWA_GetPeriodEnable** (const CMU_Type *const pCmu)

Get period enable bit.

LOCAL_INLINE void **CMU_HWA_SetCtrl** (CMU_Type *const pCmu, uint32_t u32Temp)

Set control register value.

LOCAL_INLINE uint32_t **CMU_HWA_GetCtrl** (const CMU_Type *const pCmu)

Get control register value.

LOCAL_INLINE uint32_t **CMU_HWA_GetST** (const CMU_Type *const pCmu)

Get status register value.

LOCAL_INLINE void **CMU_HWA_ClsST** (CMU_Type *const pCmu)

Clear clock monitor status.

LOCAL_INLINE void **CMU_HWA_StopModeEnable** (CMU_Type *const pCmu, bool bEnable)

Enable Stop mode.

LOCAL_INLINE void **CMU_HWA_SoftwareRST** (CMU_Type *const pCmu)

Enable Software Reset.

LOCAL_INLINE bool **CMU_HWA_GetSoftwareRST** (const CMU_Type *const pCmu)

Enable Software Reset.

LOCAL_INLINE void **CMU_HWA_LPRestartEnable** (CMU_Type *const pCmu, bool bEnable)

Enable low power restart mode.

Detailed Description

CMU Module Register API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_cordic.h File Reference

CORDIC hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_cordic.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define CORDIC_CTRL_VAL(s, a, b, c, d) (CORDIC_CTRL_SCALE(s) | CORDIC_CTRL_IE(a) |  
CORDIC_CTRL_ITER(b) | CORDIC_CTRL_OS(c) | CORDIC_CTRL_MODE(d))
```

Enumerations

```
enum CORDIC_IterationType { CORDIC_Iteration_8 = 0, CORDIC_Iteration_16, CORDIC_Iteration_24 }
```

CORDIC Iteration Type.

```
enum CORDIC_SystemType { CORDIC_Trigonometric = 0, CORDIC_Hyperbolic, CORDIC_Linear }
```

CORDIC System Type.

```
enum CORDIC_ModeType { CORDIC_Rotate = 0, CORDIC_Vector }
```

CORDIC Mode Type.

Functions

```
LOCAL_INLINE void Cordic_HWA_SetCtrl (CORDIC_Type *const pCordic, uint32_t u32Value)
```

Set CORDIC module Control register.

```
LOCAL_INLINE uint32_t Cordic_HWA_GetCtrl (CORDIC_Type *const pCordic)
```

Read CORDIC module Control register.

```
LOCAL_INLINE void Cordic_HWA_Set_XInput (CORDIC_Type *const pCordic, int32_t u32Value)
```

Set CORDIC module X Input register.

```
LOCAL_INLINE void Cordic_HWA_Set_YInput (CORDIC_Type *const pCordic, int32_t u32Value)
```

Set CORDIC module Y Input register.

LOCAL_INLINE void **Cordic_HWA_Set_ZInput** (CORDIC_Type *const pCordic, int32_t u32Value)

Set CORDIC module Z Input register.

LOCAL_INLINE int32_t **Cordic_HWA_Get_XOutput** (CORDIC_Type *const pCordic)

Read CORDIC module X Output register.

LOCAL_INLINE int32_t **Cordic_HWA_Get_YOutput** (CORDIC_Type *const pCordic)

Read CORDIC module Y Output register.

LOCAL_INLINE int32_t **Cordic_HWA_Get_ZOutput** (CORDIC_Type *const pCordic)

Read CORDIC module Z Output register.

LOCAL_INLINE bool **Cordic_HWA_Get_Stat** (CORDIC_Type *const pCordic)

Read CORDIC module State.

LOCAL_INLINE void **Cordic_HWA_Clear_Stat** (CORDIC_Type *const pCordic)

Clear CORDIC module done Flag.

Detailed Description

CORDIC hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_cpm.h File Reference

CPM register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_cpm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define FPSCR_IOC_MASK 0x00000001U
#define FPSCR_DZC_MASK 0x00000002U
#define FPSCR_OFC_MASK 0x00000004U
#define FPSCR_UFC_MASK 0x00000008U
#define FPSCR_IXC_MASK 0x00000010U
#define FPSCR_IDC_MASK 0x00000080U
#define CPM_FPU_INTFLAGMASK 0x0000003FU
```

Functions

```
LOCAL_INLINE uint32_t CPM_HWA_GetFiscr (CPM_Type *const pCpm)
```

Get the value of CPM FISCER.

```
LOCAL_INLINE bool CPM_HWA_GetFpuFiocFlag (CPM_Type *const pCpm)
```

Return CPM_FISCER FIOC value.

```
LOCAL_INLINE bool CPM_HWA_GetFpuFdzcFlag (CPM_Type *const pCpm)
```

Return CPM_FISCER FDZC value.

```
LOCAL_INLINE bool CPM_HWA_GetFpuFofcFlag (CPM_Type *const pCpm)
```

Return CPM_FISCER FOFC value.

```
LOCAL_INLINE bool CPM_HWA_GetFpuFufcFlag (CPM_Type *const pCpm)
```

Return CPM_FISCR FUFCE value.

LOCAL_INLINE bool **CPM_HWA_GetFpuFixcFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FIXC value.

LOCAL_INLINE bool **CPM_HWA_GetFpuFidcFlag** (CPM_Type *const pCpm)

Return CPM_FISCR FIDC value.

LOCAL_INLINE void **CPM_HWA_SetFiocelInt** (CPM_Type *const pCpm, bool bEnable)

Set FIOCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFdzcelInt** (CPM_Type *const pCpm, bool bEnable)

Set FDZCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFofcelInt** (CPM_Type *const pCpm, bool bEnable)

Set FOFCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFufcelInt** (CPM_Type *const pCpm, bool bEnable)

Set FUFCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFixcelInt** (CPM_Type *const pCpm, bool bEnable)

Set FIXCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFidcelInt** (CPM_Type *const pCpm, bool bEnable)

Set FIDCE interrupt.

LOCAL_INLINE void **CPM_HWA_SetFiscr** (CPM_Type *const pCpm, uint32_t u32Val)

Set FISCR value.

LOCAL_INLINE uint32_t **CPM_HWA_GetCoreIDValue** (CPM_Type *const pCpm)

Return CPM_CoreID value.

Detailed Description

CPM register API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

Macro Definition Documentation

#define CPM_FPU_INTFLAGMASK 0x0000003FU

#define FPSCR_DZC_MASK 0x00000002U

#define FPSCR_IDC_MASK 0x00000080U

#define FPSCR_IOC_MASK 0x00000001U

FPSCR Bit Fields

#define FPSCR_IXC_MASK 0x00000010U

#define FPSCR_OFC_MASK 0x00000004U

#define FPSCR_UFC_MASK 0x00000008U

HwA_crc.h File Reference

CRC hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_crc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum CRC_WriteDataSwapType { WRITE_DATASWAP_NONE = 0U, WRITE_DATASWAP_BIT = 1U,  
WRITE_DATASWAP_BIT_BYTE = 2U, WRITE_DATASWAP_BYTE = 3U }
```

The data swap for write

```
enum CRC_ReadDataSwapType { READ_DATASWAP_NONE = 0U, READ_DATASWAP_BIT = 1U,  
READ_DATASWAP_BIT_BYTE = 2U, READ_DATASWAP_BYTE = 3U }
```

The data swap for read.

```
enum CRC_ReadDataFXORType { READ_DATA_NORMAL = 0U, READ_DATA_FXOR = 1U }
```

The complement of reading crc data.

```
enum CRC_WriteCommandType { WRITE_COMMAND_DATA = 0U, WRITE_COMMAND_SEED = 1U }
```

The command type of write crc data or seed value.

```
enum CRC_BitWidthType { CRC_BIT_16 = 0U, CRC_BIT_32 = 1U, CRC_BIT_8 = 2U, CRC_BIT_INVALID = 3U }
```

The crc mode select.

Functions

```
LOCAL_INLINE void CRC_HWA_SetDataOrSeed (CRC_Type *const pCrc, CRC_WriteCommandType  
u32Mod)
```

set CRC CR register for writing data or seed

```
LOCAL_INLINE void CRC_HWA_SetPolyVal (CRC_Type *const pCrc, uint32_t u32Poly)
```

set polynomial value

LOCAL_INLINE void **CRC_HWA_SetData_U32** (CRC_Type *const pCrc, uint32_t u32Data)
set data register(32 bits)

LOCAL_INLINE void **CRC_HWA_SetData_U16** (CRC_Type *const pCrc, uint16_t u16Data)
set data register(16 bits)

LOCAL_INLINE void **CRC_HWA_SetData_U8** (CRC_Type *const pCrc, uint8_t u8Data)
set data register(low 8 bits)

LOCAL_INLINE uint32_t **CRC_HWA_GetData_U32** (CRC_Type *const pCrc)
get data register(32 bits)

LOCAL_INLINE uint16_t **CRC_HWA_GetData_U16_H** (CRC_Type *const pCrc)
get data register(high 16 bits)

LOCAL_INLINE uint16_t **CRC_HWA_GetData_U16_L** (CRC_Type *const pCrc)
get data register(low 16 bits)

LOCAL_INLINE void **CRC_HWA_SetWriteDataSwap** (CRC_Type *const pCrc, **CRC_WriteDataSwapType**
eWrDataSwap)
set data swap for writes

LOCAL_INLINE void **CRC_HWA_SetReadDataSwap** (CRC_Type *const pCrc, **CRC_ReadDataSwapType**
eRdDataSwap)
set data swap for read

LOCAL_INLINE void **CRC_HWA_SetReadDataFXOR** (CRC_Type *const pCrc, **CRC_ReadDataFXORType**
eRdDataFXOR)
set complement read Of CRC data

LOCAL_INLINE void **CRC_HWA_SetBitWidth** (CRC_Type *const pCrc, **CRC_BitWidthType** eWidth)
set width of CRC protocol

LOCAL_INLINE CRC_BitWidthType CRC_HWA_GetBitWidth (CRC_Type *const pCrc)
get width of CRC protocol

LOCAL_INLINE CRC_ReadDataSwapType CRC_HWA_GetReadDataSwap (CRC_Type *const pCrc)
get data swap type for read

Detailed Description

CRC hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_csc.h File Reference

Hardware access layer for CSC.

```
#include "device_header.h"
```

Include dependency graph for HwA_csc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum CSC0_ClockOutDivType { CSC0_CLKOUT_DIV_BY1 = 0U, CSC0_CLKOUT_DIV_BY2 = 1U,  
CSC0_CLKOUT_DIV_BY3 = 2U, CSC0_CLKOUT_DIV_BY4 = 3U, CSC0_CLKOUT_DIV_BY5 = 4U,  
CSC0_CLKOUT_DIV_BY6 = 5U, CSC0_CLKOUT_DIV_BY7 = 6U, CSC0_CLKOUT_DIV_BY8 = 7U }
```

CSC0 clock out divide ratio type.

```
enum CSC0_ClockOutSrcType { CSC0_CLKOUT_SCG_CLKOUT = 0U, CSC0_CLKOUT_FOSC_DIVM_CLK =  
2U, CSC0_CLKOUT_SLOW_CLK = 3U, CSC0_CLKOUT_SIRC_DIVM_CLK = 4U,  
CSC0_CLKOUT_PLL1_DIVM_CLK = 5U, CSC0_CLKOUT_FIRC_DIVM_CLK = 6U, CSC0_CLKOUT_CORE_CLK  
= 7U, CSC0_CLKOUT_PLL0_DIVM_CLK = 8U, CSC0_CLKOUT_BUS_CLK = 9U,  
CSC0_CLKOUT_SIRC_128K_CLK = 10U, CSC0_CLKOUT_AON_CLK = 12U, CSC0_CLKOUT_RTC_CLK = 14U }
```

CSC0 clock out source type.

```
enum CSC0_AON32KClkSrcType { CSC0_AON32K_SIRCDIV_32K_CLK = 1U, CSC0_AON32K_SOSC32K_CLK  
= 2U, CSC0_AON32K_SIRC32K_CLK = 3U }
```

Data type for CSC0_AONCLKSR[32KAONCLKSEL], set AON32KCLK source clock.

```
enum CSC0_RTCClkSrcType { CSC0_RTC_FOSCDIVL_CLK = 0U, CSC0_RTC_SIRCDIV_32K_CLK = 1U,  
CSC0_RTC_SOSC_CLK = 2U, CSC0_RTC_SIRC32K_CLK = 3U }
```

Data type for CSC0_RTCCLKSEL[RTCCLKSEL], set RTCCLK source clock.

```
enum CSC0_AONClkSrcType { CSC0_AON_SIRCDIV_128K_CLK = 0U, CSC0_AON_SIRC32K_CLK = 1U,  
CSC0_AON_SIRCDIV_32K_CLK = 2U, CSC0_AON_SIRC32_1K_CLK = 3U }
```

Data type for CSC0_AONCLKSEL[AONCLKSEL], set AONCLK source clock.

Functions

```
LOCAL_INLINE void CSC0_HWA_Set_STOP_MODER0 (uint32_t u32val)
```

Set stop mode register 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_MODER0** (void)

Get stop mode register 0.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_REQR0** (uint32_t u32val)

Set stop request register 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_REQR0** (void)

Get stop request register 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_ACKR0** (void)

Get stop ack status in register 0.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_MODER1** (uint32_t u32val)

Set stop mode register 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_MODER1** (void)

Get stop mode register 1.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_REQR1** (uint32_t u32val)

Set stop request register 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_REQR1** (void)

Get stop request register 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_ACKR1** (void)

Get stop ack status in register 1.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_MODER2** (uint32_t u32val)

Set stop mode register 2.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_MODER2** (void)

Get stop mode register 2.

LOCAL_INLINE void **CSC0_HWA_Set_STOP_REQR2** (uint32_t u32val)

Set stop request register 2.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_REQR2** (void)

Get stop request register 2.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_STOP_ACKR2** (void)

Get stop ack status in register 2.

LOCAL_INLINE void **CSC0_HWA_Set_CCM0_CFG** (uint32_t u32val)

Set CCM0 configure register.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_CCM0_CFG** (void)

Get CCM0 configure.

LOCAL_INLINE void **CSC0_HWA_Set_SCG_MAM_STALL** (uint32_t u32val)

Set SCG MAM stall request register.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SCG_MAM_STALL** (void)

Get SCG MAM stall request register.

LOCAL_INLINE void **CSC0_HWA_Set_CPU0_INT** (uint32_t u32val)

Set cpu0 software interrupt register.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_CPU0_INT** (void)

Get cpu0 software interrupt register.

LOCAL_INLINE void **CSC0_HWA_EnableCpu0SWInterrupt** (void)

Enable cpu0 software interrupt.

LOCAL_INLINE void **CSC0_HWA_DisableCpu0SWInterrupt** (void)

Disable cpu0 software interrupt.

LOCAL_INLINE void **CSC0_HWA_Set_SMU_CTRL0** (uint32_t u32Value)

Set multi CSC_SMU control group 0.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SMU_CTRL0** (void)

Get CSC0_SMU_CTRL0 register status.

LOCAL_INLINE void **CSC0_HWA_Set_SMU_CTRL1** (uint32_t u32Value)

Set multi CSC_SMU control group 1.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SMU_CTRL1** (void)

Get CSC0_SMU_CTRL1 register status.

LOCAL_INLINE void **CSC0_HWA_Set_SMU_CTRL4** (uint32_t u32Value)

Set multi CSC_SMU control group 4.

LOCAL_INLINE uint32_t **CSC0_HWA_Get_SMU_CTRL4** (void)

Get CSC0_SMU_CTRL4 register status.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_MODER0** (uint32_t u32val)

Set stop mode register 0.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_MODER0** (void)

Get stop mode register 0.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_REQR0** (uint32_t u32val)

Set stop request register 0.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_REQR0** (void)

Get stop request register 0.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_ACKR0** (void)

Get stop ack status in register 0.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_MODER1** (uint32_t u32val)

Set stop mode register 1.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_MODER1** (void)

Get stop mode register 1.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_REQR1** (uint32_t u32val)

Set stop request register 1.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_REQR1** (void)

Get stop request register 1.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_ACKR1** (void)

Get stop ack status in register 1.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_MODER2** (uint32_t u32val)

Set stop mode register 2.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_MODER2** (void)

Get stop mode register 2.

LOCAL_INLINE void **CSC1_HWA_Set_STOP_REQR2** (uint32_t u32val)

Set stop request register 2.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_REQR2** (void)

Get stop request register 2.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_STOP_ACKR2** (void)

Get stop ack status in register 2.

LOCAL_INLINE void **CSC1_HWA_Set_CCM1_CFG** (uint32_t u32val)

Set CCM0 configure register.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_CCM1_CFG** (void)

Get CCM0 configure.

LOCAL_INLINE void **CSC1_HWA_Set_CPU1_INT** (uint32_t u32val)

Set cpu0 software interrupt register.

LOCAL_INLINE uint32_t **CSC1_HWA_Get_CPU1_INT** (void)

Get cpu1 software interrupt register.

LOCAL_INLINE void **CSC1_HWA_EnableCpu1SWInterrupt** (void)

Enable cpu1 software interrupt.

LOCAL_INLINE void **CSC1_HWA_DisableCpu1SWInterrupt** (void)

Disable cpu1 software interrupt.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_MODER0** (uint32_t u32val)

Set stop mode register 0.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_MODER0** (void)

Get stop mode register 0.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_REQR0** (uint32_t u32val)

Set stop request register 0.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_REQR0** (void)

Get stop request register 0.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_ACKR0** (void)

Get stop ack status in register 0.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_MODER1** (uint32_t u32val)

Set stop mode register 1.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_MODER1** (void)

Get stop mode register 1.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_REQR1** (uint32_t u32val)

Set stop request register 1.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_REQR1** (void)

Get stop request register 1.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_ACKR1** (void)

Get stop ack status in register 1.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_MODER2** (uint32_t u32val)

Set stop mode register 2.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_MODER2** (void)

Get stop mode register 2.

LOCAL_INLINE void **CSC2_HWA_Set_STOP_REQR2** (uint32_t u32val)

Set stop request register 2.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_REQR2** (void)

Get stop request register 2.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_STOP_ACKR2** (void)

Get stop ack status in register 2.

LOCAL_INLINE void **CSC2_HWA_Set_CCM2_CFG** (uint32_t u32val)

Set CCM0 configure register.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_CCM2_CFG** (void)

Get CCM0 configure.

LOCAL_INLINE void **CSC2_HWA_Set_CPU2_INT** (uint32_t u32val)

Set cpu0 software interrupt register.

LOCAL_INLINE uint32_t **CSC2_HWA_Get_CPU2_INT** (void)

Get cpu0 software interrupt register.

LOCAL_INLINE void **CSC2_HWA_EnableCpu2SWInterrupt** (void)

Enable cpu0 software interrupt.

LOCAL_INLINE void **CSC2_HWA_DisableCpu2SWInterrupt** (void)

Disable cpu0 software interrupt.

Detailed Description

Hardware access layer for CSC.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_dma.h File Reference

Hardware access layer for DMA.

```
#include "device_header.h"
```

Include dependency graph for HwA_dma.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define DMA_CH_TO_DCHPRI(x) ((x) ^ 3U)
```

Enumerations

```
enum DMA_RunningStatusType { DMA_RUNNING_STATUS_IDLE = 0x0U,  
DMA_RUNNING_STATUS_ACTIVE = 0x1U }
```

DMA running status.

```
enum DMA_ArbitrationAlgorithmType { DMA_ARBITRATION_ALGORITHM_FIXED_PRIORITY = 0U,  
DMA_ARBITRATION_ALGORITHM_ROUND_ROBIN = 1U }
```

DMA channel arbitration algorithm used in the channel arbitration phase.

```
enum DMA_TransferSizeType { DMA_TRANSFER_SIZE_1B = 0x0U, DMA_TRANSFER_SIZE_2B = 0x1U,  
DMA_TRANSFER_SIZE_4B = 0x2U, DMA_TRANSFER_SIZE_8B = 0x3U, DMA_TRANSFER_SIZE_32B = 0x5U }
```

Defines the size of data in one transfer.

Functions

```
LOCAL_INLINE bool DMA_HWA_GetCancelTransferStatus (const DMA_Type *const pDma)
```

Get whether the DMA instance is cancelling.

```
LOCAL_INLINE void DMA_HWA_CancelTransfer (DMA_Type *const pDma)
```

Cancel the remaining transfer of the DMA.

```
LOCAL_INLINE bool DMA_HWA_GetErrorCancelTransferStatus (const DMA_Type *const pDma)
```

Get whether the DMA instance is cancelling with error.

LOCAL_INLINE void **DMA_HWA_ErrorCancelTransfer** (DMA_Type *const pDma)

Cancel the remaining transfer of the DMA and generate an error after finished cancelling.

LOCAL_INLINE bool **DMA_HWA_GetInnerLoopMappingEnableFlag** (const DMA_Type *const pDma)

Get whether inner loop mapping is enabled.

LOCAL_INLINE void **DMA_HWA_SetInnerLoopMappingEnableFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to enable inner loop mapping.

LOCAL_INLINE bool **DMA_HWA_GetContinuousTrigModeEnableFlag** (const DMA_Type *const pDma)

Get whether the continuous trig mode is enabled When continuous trig mode is enabled, channel arbitration is not used for a inner loop channel trig before being activated again. Upon inner loop completion, the channel activates again if that channel has a inner loop channel trig enabled and the trig channel is itself. This effectively applies the inner loop offsets and restarts the next inner loop.

LOCAL_INLINE void **DMA_HWA_SetContinuousTrigModeEnableFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to enable the continuous trig mode.

LOCAL_INLINE bool **DMA_HWA_GetHaltStatus** (const DMA_Type *const pDma)

Get whether the DMA is halted When the DMA is halted, it will ignore all service requests.

LOCAL_INLINE void **DMA_HWA_HaltOperations** (DMA_Type *const pDma)

Halt the DMA operations.

LOCAL_INLINE void **DMA_HWA_ClearHaltFlag** (DMA_Type *const pDma)

Clear the halt flag of the DMA instance After HALT is cleared, the DMA could continue to operate.

LOCAL_INLINE bool **DMA_HWA_GetHaltOnErrorFlag** (const DMA_Type *const pDma)

Get whether halt on error is enabled on the DMA instance.

LOCAL_INLINE void **DMA_HWA_SetHaltOnErrorFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to enable halt on error on the DMA instance.

LOCAL_INLINE **DMA_ArbitrationAlgorithmType** **DMA_HWA_GetArbitrationAlgorithm** (const DMA_Type *const pDma)

Get the DMA arbitration algorithm.

LOCAL_INLINE void **DMA_HWA_SetArbitrationAlgorithm** (DMA_Type *const pDma, **DMA_ArbitrationAlgorithmType** eAlgorithm)

Set the DMA arbitration algorithm.

LOCAL_INLINE bool **DMA_HWA_GetDebugModeStopFlag** (const DMA_Type *const pDma)

Get whether the DMA is configured to stop under debug mode.

LOCAL_INLINE uint32_t **DMA_HWA_GetControlRegister** (const DMA_Type *const pDma)

Get the control register of the DMA instance.

LOCAL_INLINE void **DMA_HWA_SetControlRegister** (DMA_Type *const pDma, uint32_t u32Settings)

Set the control register of the DMA instance.

LOCAL_INLINE void **DMA_HWA_SetDebugModeStopFlag** (DMA_Type *const pDma, bool bEnable)

Set whether to stop DMA under debug mode.

LOCAL_INLINE uint32_t **DMA_HWA_GetErrorStatus** (const DMA_Type *const pDma)

Get the DMA error status.

LOCAL_INLINE bool **DMA_HWA_GetChannelErrorFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether there is error occurred on the specified channel.

LOCAL_INLINE uint16_t **DMA_HWA_GetAllChannelErrorFlag** (const DMA_Type *const pDma)

Get the channel error status, each bit represents a channel.

LOCAL_INLINE bool **DMA_HWA_GetChannelErrorInterruptEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether error interrupt is enabled on the specified channel.

LOCAL_INLINE uint16_t **DMA_HWA_GetAllChannelErrorInterruptEnableFlag** (const DMA_Type *const pDma)

Get channel error interrupt enable status.

LOCAL_INLINE void **DMA_HWA_EnableChannelErrorInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Enable error interrupt for the specified channel.

LOCAL_INLINE void **DMA_HWA_EnableAllChannelErrorInterrupt** (DMA_Type *const pDma)

Enable error interrupt for all channels.

LOCAL_INLINE void **DMA_HWA_DisableChannelErrorInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Disable error interrupt for the specified channel.

LOCAL_INLINE void **DMA_HWA_DisableAllChannelErrorInterrupt** (DMA_Type *const pDma)

Disable error interrupt for all channels.

LOCAL_INLINE void **DMA_HWA_EnableChannelRequest** (DMA_Type *const pDma, uint8_t u8Channel)

Enable channel request for the specified channel.

LOCAL_INLINE void **DMA_HWA_EnableAllChannelRequest** (DMA_Type *const pDma)

Enable channel request for all channels.

LOCAL_INLINE void **DMA_HWA_DisableChannelRequest** (DMA_Type *const pDma, uint8_t u8Channel)

Disable channel request for the specified channel.

LOCAL_INLINE void **DMA_HWA_DisableAllChannelRequest** (DMA_Type *const pDma)

Disable channel request for all channels.

LOCAL_INLINE void **DMA_HWA_ClearChannelDoneStatus** (DMA_Type *const pDma, uint8_t u8Channel)

Clear DONE bit for the specified channel.

LOCAL_INLINE void **DMA_HWA_ClearAllChannelDoneStatus** (DMA_Type *const pDma)

Clear DONE bit for the all channels.

LOCAL_INLINE void **DMA_HWA_ClearChannelErrorFlag** (DMA_Type *const pDma, uint8_t u8Channel)

Clear ERR bit for the specified channel.

LOCAL_INLINE void **DMA_HWA_ClearAllChannelErrorFlag** (DMA_Type *const pDma)

Clear ERR bit for the all channels.

LOCAL_INLINE void **DMA_HWA_ClearChannelInterruptFlag** (DMA_Type *const pDma, uint8_t u8Channel)

Clear interrupt flag for the specified channel.

LOCAL_INLINE void **DMA_HWA_ClearAllChannelInterruptFlag** (DMA_Type *const pDma)

Clear interrupt flag for the all channels.

LOCAL_INLINE void **DMA_HWA_SetChannelStart** (DMA_Type *const pDma, uint8_t u8Channel)

Set start for the specified channel.

LOCAL_INLINE void **DMA_HWA_SetAllChannelStart** (DMA_Type *const pDma)

Set start for the all channels.

LOCAL_INLINE bool **DMA_HWA_GetSrcUnalignModuloEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether source unalign modulo is enabled for the specified channel.

LOCAL_INLINE bool **DMA_HWA_GetDestUnalignModuloEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether destination unalign modulo is enabled for the specified channel.

LOCAL_INLINE void **DMA_HWA_SetUnalignModuloEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnableSrcModulo, bool bEnableDestModulo)

Get whether to enable unalign modulo for the specified channel.

LOCAL_INLINE uint16_t **DMA_HWA_GetSrcUnalignModulo** (const DMA_Type *const pDma, uint8_t u8Selection)

Get the source unalign modulo of the index.

```
LOCAL_INLINE uint16_t DMA_HWA_GetDestUnalignModulo (const DMA_Type *const pDma, uint8_t u8Selection)
```

Get the destination unalign modulo of the index.

```
LOCAL_INLINE void DMA_HWA_SetUnalignModulo (DMA_Type *const pDma, uint8_t u8Selection, uint16_t u16SrcModulo, uint16_t u16DestModulo)
```

Set unalign modulo of the index.

```
LOCAL_INLINE uint8_t DMA_HWA_GetPriority (const DMA_Type *const pDma, uint8_t u8Channel)
```

Get priority of the channel.

```
LOCAL_INLINE void DMA_HWA_SetPriority (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8Priority)
```

Set priority of the channel.

```
LOCAL_INLINE uint32_t DMA_HWA_GetSrcAddr (const DMA_Type *const pDma, uint8_t u8Channel)
```

Get the source address of the DMA channel.

```
LOCAL_INLINE void DMA_HWA_SetSrcAddr (DMA_Type *const pDma, uint8_t u8Channel, uint32_t u32SrcAddr)
```

Set the source address of the DMA channel.

```
LOCAL_INLINE uint32_t DMA_HWA_GetDestAddr (const DMA_Type *const pDma, uint8_t u8Channel)
```

Get the destination address of the DMA channel.

```
LOCAL_INLINE void DMA_HWA_SetDestAddr (DMA_Type *const pDma, uint8_t u8Channel, uint32_t u32DestAddr)
```

Set the destination address of the DMA channel.

```
LOCAL_INLINE int16_t DMA_HWA_GetSrcOffset (const DMA_Type *const pDma, uint8_t u8Channel)
```

Get source data offset of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetSrcOffset** (DMA_Type *const pDma, uint8_t u8Channel, int16_t s16Offset)

Set source data offset of the DMA channel.

LOCAL_INLINE int16_t **DMA_HWA_GetDestOffset** (const DMA_Type *const pDma, uint8_t u8Channel)

Get destination data offset of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetDestOffset** (DMA_Type *const pDma, uint8_t u8Channel, int16_t s16Offset)

Set destination data offset of the DMA channel.

LOCAL_INLINE **DMA_TransferSizeType** **DMA_HWA_GetSrcDataSize** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the source data size of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetSrcDataSize** (DMA_Type *const pDma, uint8_t u8Channel, **DMA_TransferSizeType** eDataSize)

Set the source data size of the DMA channel.

LOCAL_INLINE **DMA_TransferSizeType** **DMA_HWA_GetDestDataSize** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the destination data size of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetDestDataSize** (DMA_Type *const pDma, uint8_t u8Channel, **DMA_TransferSizeType** eDataSize)

Set the destination data size of the DMA channel.

LOCAL_INLINE uint8_t **DMA_HWA_GetSrcModulo** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the source address aligned modulo of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetSrcModulo** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8SrcModulo)

Set the source address aligned modulo of the DMA channel.

LOCAL_INLINE uint8_t **DMA_HWA_GetDestModulo** (const DMA_Type *const pDma, uint8_t u8Channel)
Get the destination address aligned modulo of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetDestModulo** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8DestModulo)
Set the destination address aligned modulo of the DMA channel.

LOCAL_INLINE int32_t **DMA_HWA_GetSrcLastAddrAdjustment** (const DMA_Type *const pDma, uint8_t u8Channel)
Get the address adjustment applied to the source address after major loop finished.

LOCAL_INLINE void **DMA_HWA_SetSrcLastAddrAdjustment** (DMA_Type *const pDma, uint8_t u8Channel, int32_t s32LastAdj)
Set the address adjustment applied to the source address after major loop finished.

LOCAL_INLINE int32_t **DMA_HWA_GetDestLastAddrAdjustment** (const DMA_Type *const pDma, uint8_t u8Channel)
Get the address adjustment applied to the destination address after major loop finished.

LOCAL_INLINE void **DMA_HWA_SetDestLastAddrAdjustment** (DMA_Type *const pDma, uint8_t u8Channel, int32_t s32LastAdj)
Set the address adjustment applied to the destination address after major loop finished.

LOCAL_INLINE bool **DMA_HWA_GetInnerLoopSrcOffsetEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)
Get whether source address inner loop offset is enabled.

LOCAL_INLINE bool **DMA_HWA_GetInnerLoopDestOffsetEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)
Get whether destination address inner loop offset is enabled.

LOCAL_INLINE int32_t **DMA_HWA_GetInnerLoopOffset** (const DMA_Type *const pDma, uint8_t u8Channel)
Get the inner loop offset of the DMA channel.

LOCAL_INLINE uint32_t **DMA_HWA_GetInnerLoopSize** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the transfer data size of the inner loop.

LOCAL_INLINE void **DMA_HWA_SetInnerLoopOffset** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnableSrcOffset, bool bEnableDestOffset, int32_t s32Offset)

Set the inner loop offset of the DMA channel.

LOCAL_INLINE void **DMA_HWA_SetInnerLoopSize** (DMA_Type *const pDma, uint8_t u8Channel, uint32_t u32Size)

Set the transfer data size of the inner loop.

LOCAL_INLINE bool **DMA_HWA_GetCurrentChannelToChannelTrigEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether current channel to channel trig is enabled when inner loop complete.

LOCAL_INLINE uint8_t **DMA_HWA_GetCurrentChannelToChannelTrigChannel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the target channel when current channel to channel trig is enabled.

LOCAL_INLINE uint16_t **DMA_HWA_GetCurrentLoopCount** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the current loop count The current loop count is the same as the beginning loop count initially, and it will decrement each time a inner loop finishes.

LOCAL_INLINE bool **DMA_HWA_GetBeginningChannelToChannelTrigEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether beginning channel to channel trig is enabled when inner loop complete.

LOCAL_INLINE uint8_t **DMA_HWA_GetBeginningChannelToChannelTrigChannel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the target channel when beginning channel to channel trig is enabled.

LOCAL_INLINE uint16_t **DMA_HWA_GetBeginningLoopCount** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the beginning loop count This field specifies how many inner loops will be executed in a DMA transfer.

LOCAL_INLINE void **DMA_HWA_SetChannelToChannelTrig** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable, uint8_t u8TrigChannel)

Set channel to channel trig when inner loop complete.

LOCAL_INLINE void **DMA_HWA_SetLoopCount** (DMA_Type *const pDma, uint8_t u8Channel, uint16_t u16LoopCnt)

Set the loop count of the DMA transfer This field specifies how many inner loops will be executed in a DMA transfer.

LOCAL_INLINE uint8_t **DMA_HWA_GetUnalignModuloSel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the unalign modulo index selection.

LOCAL_INLINE void **DMA_HWA_SetUnalignModuloSel** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8UnalignModuloSel)

Set the unalign modulo index selection.

LOCAL_INLINE uint8_t **DMA_HWA_GetOuterLoopTrigChannel** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the target channel to trig when outer loop is completed.

LOCAL_INLINE void **DMA_HWA_SetOuterLoopTrigChannel** (DMA_Type *const pDma, uint8_t u8Channel, uint8_t u8TrigChannel)

Set the target channel to trig when outer loop is completed.

LOCAL_INLINE bool **DMA_HWA_GetChannelDoneStatus** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether the transfer is done on the selected channel.

LOCAL_INLINE DMA_RunningStatusType **DMA_HWA_GetChannelActiveStatus** (const DMA_Type *const pDma, uint8_t u8Channel)

Get the active status of the DMA channel This flag signals the channel is currently in execution. It is set when channel service begins, and is cleared by the DMA as the inner loop completes or when any error condition is detected.

LOCAL_INLINE bool **DMA_HWA_GetOuterLoopTrigEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get channel to channel trig is enabled when outer loop is completed.

LOCAL_INLINE void **DMA_HWA_SetOuterLoopTrigEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable)

Set whether to enable channel to channel trig when outer loop is completed.

LOCAL_INLINE bool **DMA_HWA_GetAutoDisableReuquestEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether DMA request will be disabled automatically when outer loop is completed.

LOCAL_INLINE void **DMA_HWA_SetAutoDisableReuquestEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable)

Set whether to disable DMA request automatically when outer loop is completed.

LOCAL_INLINE bool **DMA_HWA_GetHalfCompleteInterruptEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether DMA interrupt will be generated when the outer loop is half done.

LOCAL_INLINE void **DMA_HWA_SetHalfCompleteInterruptEnableFlag** (DMA_Type *const pDma, uint8_t u8Channel, bool bEnable)

Set whether to enable DMA interrupt when the outer loop is half done.

LOCAL_INLINE bool **DMA_HWA_GetTransferCompleteInterruptEnableFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether DMA interrupt is enabled when DMA transfer is completed.

LOCAL_INLINE void **DMA_HWA_EnableTransferCompleteInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Enable DMA transfer complete interrupt.

LOCAL_INLINE void **DMA_HWA_DisableTransferCompleteInterrupt** (DMA_Type *const pDma, uint8_t u8Channel)

Disable DMA transfer complete interrupt.

LOCAL_INLINE bool **DMA_HWA_GetWaitStartFlag** (const DMA_Type *const pDma, uint8_t u8Channel)

Get whether the DMA start is requested but has not been executed.

LOCAL_INLINE uint16_t **DMA_HWA_GetChannelControlStatus** (const DMA_Type *const pDma, uint8_t u8Channel)

Get DMA channel control and status.

LOCAL_INLINE void **DMA_HWA_SetChannelControlStatus** (DMA_Type *const pDma, uint8_t u8Channel, uint16_t u16Setting)

Set DMA channel control and status.

Detailed Description

Hardware access layer for DMA.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

Macro Definition Documentation

```
#define DMA_CH_TO_DCHPRI(x) ((x) ^ 3U)
```


HwA_dmamux.h File Reference

Hardware access layer for DMAMUX.

```
#include "device_header.h"
```

Include dependency graph for HwA_dmamux.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE bool **DMAMUX_HWA_GetEnableFlag** (const DMAMUX_Type *const pDmamux, uint8_t u8Channel)

Get whether DMAMUX is enabled for the specified DMA channel.

LOCAL_INLINE DMA_RequestSourceType **DMAMUX_HWA_GetRequestSource** (const DMAMUX_Type *const pDmamux, uint8_t u8Channel)

Get the request source for the specified DMA channel.

LOCAL_INLINE void **DMAMUX_HWA_SetRequestSource** (DMAMUX_Type *const pDmamux, uint8_t u8Channel, bool bEnable, DMA_RequestSourceType eReqSrc)

Set the request source for the specified DMA channel.

LOCAL_INLINE bool **DMAMUX_HWA_GetPeriodicTrigFlag** (const DMAMUX_Type *const pDmamux, uint8_t u8Channel)

Get whether periodic trig is enabled for the specified DMA channel.

LOCAL_INLINE void **DMAMUX_HWA_SetPeriodicTrigFlag** (DMAMUX_Type *const pDmamux, uint8_t u8Channel, bool bEnable)

Set whether to enable periodic trig for the specified DMA channel.

Detailed Description

Hardware access layer for DMAMUX.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_eftu_ccm.h File Reference

Hardware access layer for EFTU CCM.

```
#include "device_header.h"
```

Include dependency graph for HwA_eftu_ccm.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

Hardware access layer for EFTU CCM.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_eftu_cmu.h File Reference

Hardware access layer for EFTU CMU.

```
#include "device_header.h"
```

Include dependency graph for HwA_eftu_cmu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

Hardware access layer for EFTU CMU.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_eftu_dtm.h File Reference

Hardware access layer for EFTU DTM.

```
#include "device_header.h"
```

Include dependency graph for HwA_eftu_dtm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

Hardware access layer for EFTU DTM.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_eftu_tbu.h File Reference

Hardware access layer for EFTU TBU.

```
#include "device_header.h"
```

Include dependency graph for HwA_eftu_tbu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

Hardware access layer for EFTU TBU.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_eftu_tim.h File Reference

Hardware access layer for EFTU TIM.

```
#include "device_header.h"
```

Include dependency graph for HwA_eftu_tim.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

Hardware access layer for EFTU TIM.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_eftu_tom.h File Reference

Hardware access layer for EFTU TOM.

```
#include "device_header.h"
```

Include dependency graph for HwA_eftu_tom.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

Hardware access layer for EFTU TOM.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_eim.h File Reference

EIM Module Register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_eim.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **EIM_HWA_EnableGlobalErrorInjection** (EIM_Type *const pEim)

Enable EIM Global Error Injection.

LOCAL_INLINE void **EIM_HWA_DisableGlobalErrorInjection** (EIM_Type *const pEim)

Disable EIM Global Error Injection.

LOCAL_INLINE uint32_t **EIM_HWA_GetCtrlRegn** (const EIM_Type *const pEim, uint8_t u8Idx)

Get EIM channel N control register value.

LOCAL_INLINE void **EIM_HWA_SetCtrlRegn** (EIM_Type *const pEim, uint8_t u8Idx, uint32_t u32Value)

Set EIM channel N control register value.

LOCAL_INLINE bool **EIM_HWA_GetCtrlRegnDwpLockStatus** (const EIM_Type *const pEim, uint8_t u8Idx)

Get EIM_CTRL_REGn register DWP lock status.

LOCAL_INLINE void **EIM_HWA_SetCtrlRegnDwpLock** (EIM_Type *const pEim, uint8_t u8Idx, bool bEnable)

Set EIM_CTRL_REGn register DWP lock status.

LOCAL_INLINE void **EIM_HWA_SetCtrlRegnDwpMode** (EIM_Type *const pEim, uint8_t u8Idx, uint8_t u8DwpMode)

Set EIM_CTRL_REGn register DWP mode.

LOCAL_INLINE void **EIM_HWA_SetLockstep** (EIM_Type *const pEim, uint8_t u8Idx, uint32_t u32Value)
Set EIM LOCKSTEP error injection register.

LOCAL_INLINE uint32_t **EIM_HWA_GetBusRegn** (const EIM_Type *const pEim, uint8_t u8Idx)
Get EIM bus n register value.

LOCAL_INLINE void **EIM_HWA_SetBusRegn** (EIM_Type *const pEim, uint8_t u8Idx, uint32_t u32Value)
Set EIM bus n register value.

Detailed Description

EIM Module Register API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_elu.h File Reference

esLU hardware access layer

```
#include "device_header.h"
```

Include dependency graph for HwA_elu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

esLU hardware access layer

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_enet.h File Reference

ENET hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_enet.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum ENET_MAC_StatusType { ENET_MAC_STATUS_IDLE = 0u, ENET_MAC_STATUS_BUSY = 1u }
```

ENE MAC status type definition.

```
enum ENET_MAC_SrcAddrCtrlType { ENET_MAC_SRC_ADDR_CTRL_BY_INPUT_SIGNAL = 0u,  
ENET_MAC_SRC_ADDR_INSERT_BY_MAC_ADDR_IDX_0 = 2u,  
ENET_MAC_SRC_ADDR_REPLACE_BY_MAC_ADDR_IDX_0 = 3u,  
ENET_MAC_SRC_ADDR_INSERT_BY_MAC_ADDR_IDX_1 = 6u,  
ENET_MAC_SRC_ADDR_REPLACE_BY_MAC_ADDR_IDX_1 = 7u }
```

ENET MAC source address insertion control type definition.

```
enum ENET_MAC_InterPacketGapType { ENET_MAC_INTER_PACKET_GAP_96_BIT = 0u,  
ENET_MAC_INTER_PACKET_GAP_88_BIT = 1u, ENET_MAC_INTER_PACKET_GAP_80_BIT = 2u,  
ENET_MAC_INTER_PACKET_GAP_72_BIT = 3u, ENET_MAC_INTER_PACKET_GAP_64_BIT = 4u,  
ENET_MAC_INTER_PACKET_GAP_56_BIT = 5u, ENET_MAC_INTER_PACKET_GAP_48_BIT = 6u,  
ENET_MAC_INTER_PACKET_GAP_40_BIT = 7u }
```

MAC inter packet gap type definition.

```
enum ENET_MAC_ExtInterPacketGapType { ENET_MAC_EXT_INTER_PACKET_GAP_104_BIT = 0u,  
ENET_MAC_EXT_INTER_PACKET_GAP_112_BIT = 1u, ENET_MAC_EXT_INTER_PACKET_GAP_120_BIT =  
2u, ENET_MAC_EXT_INTER_PACKET_GAP_128_BIT = 3u, ENET_MAC_EXT_INTER_PACKET_GAP_136_BIT =  
4u, ENET_MAC_EXT_INTER_PACKET_GAP_144_BIT = 5u,  
ENET_MAC_EXT_INTER_PACKET_GAP_152_BIT = 6u, ENET_MAC_EXT_INTER_PACKET_GAP_160_BIT =  
7u, ENET_MAC_EXT_INTER_PACKET_GAP_168_BIT = 8u, ENET_MAC_EXT_INTER_PACKET_GAP_176_BIT =  
9u, ENET_MAC_EXT_INTER_PACKET_GAP_184_BIT = 10u,  
ENET_MAC_EXT_INTER_PACKET_GAP_192_BIT = 11u, ENET_MAC_EXT_INTER_PACKET_GAP_200_BIT =  
12u, ENET_MAC_EXT_INTER_PACKET_GAP_208_BIT = 13u,
```

ENET_MAC_EXT_INTER_PACKET_GAP_216_BIT = 14u, **ENET_MAC_EXT_INTER_PACKET_GAP_224_BIT** = 15u, **ENET_MAC_EXT_INTER_PACKET_GAP_232_BIT** = 16u,
ENET_MAC_EXT_INTER_PACKET_GAP_240_BIT = 17u, **ENET_MAC_EXT_INTER_PACKET_GAP_248_BIT** = 18u, **ENET_MAC_EXT_INTER_PACKET_GAP_256_BIT** = 19u,
ENET_MAC_EXT_INTER_PACKET_GAP_264_BIT = 20u, **ENET_MAC_EXT_INTER_PACKET_GAP_272_BIT** = 21u, **ENET_MAC_EXT_INTER_PACKET_GAP_280_BIT** = 22u,
ENET_MAC_EXT_INTER_PACKET_GAP_288_BIT = 23u, **ENET_MAC_EXT_INTER_PACKET_GAP_296_BIT** = 24u, **ENET_MAC_EXT_INTER_PACKET_GAP_304_BIT** = 25u,
ENET_MAC_EXT_INTER_PACKET_GAP_312_BIT = 26u, **ENET_MAC_EXT_INTER_PACKET_GAP_320_BIT** = 27u, **ENET_MAC_EXT_INTER_PACKET_GAP_328_BIT** = 28u,
ENET_MAC_EXT_INTER_PACKET_GAP_336_BIT = 29u, **ENET_MAC_EXT_INTER_PACKET_GAP_344_BIT** = 30u, **ENET_MAC_EXT_INTER_PACKET_GAP_352_BIT** = 31u }

MAC extend inner packet gap type definition.

enum **ENET_MAC_SpeedType** { **ENET_MAC_SPEED_1000MBPS** = 0u, **ENET_MAC_SPEED_2500MBPS** = 1u,
ENET_MAC_SPEED_10MBPS = 2u, **ENET_MAC_SPEED_100MBPS** = 3u }

MAC speed mode type definition.

enum **ENET_MAC_DuplexModeType** { **ENET_MAC_HALF_DUPLEX** = 0u, **ENET_MAC_FULL_DUPLEX** = 1u }

MAC duplex mode type definition.

enum **ENET_MAC_BackOffLimitType** { **ENET_MAC_BACK_OFF_LIMIT_10_SLOT** = 0u,
ENET_MAC_BACK_OFF_LIMIT_8_SLOT = 1u, **ENET_MAC_BACK_OFF_LIMIT_4_SLOT** = 2u,
ENET_MAC_BACK_OFF_LIMIT_1_SLOT = 3u }

MAC back off limit type definition.

enum **ENET_MAC_SlowProtocolDetectType** { **ENET_MAC_SLOW_PROTOCOL_DETECT_STANDARD** = 0u,
ENET_MAC_SLOW_PROTOCOL_DETECT_EXTENDED = 1u }

MAC slow protocol detect type definition.

enum **ENET_MAC_AddrFilterModeType** { **ENET_MAC_ADDR_FILTER_DIRECT** = 0u,
ENET_MAC_ADDR_FILTER_INVERSE = 1u }

MAC address filter mode type definition.

enum **ENET_MAC_CtrlPacketFilterModeType** { **ENET_MAC_CTRL_PACKET_FILTER_ALL** = 0u,
ENET_MAC_CTRL_PACKET_FILTER_PAUSE_PACKET = 1u, **ENET_MAC_CTRL_PACKET_FORWARD_ALL** = 2u,
ENET_MAC_CTRL_PACKET_ADDR_FILTER = 3u }

MAC control packets filter mode type definition.

enum **ENET_MAC_HashOrPerfectModeType** { **ENET_MAC_USE_PERFECT_FILTER** = 0u,
ENET_MAC_USE_HASH_FILTER = 1u }

MAC perfect or hash filter mode type definition.

```
enum ENET_MAC_WDGTimerSizeType { ENET_MAC_WDG_TIMER_SIZE_2KB = 0u,
ENET_MAC_WDG_TIMER_SIZE_3KB = 1u, ENET_MAC_WDG_TIMER_SIZE_4KB = 2u,
ENET_MAC_WDG_TIMER_SIZE_5KB = 3u, ENET_MAC_WDG_TIMER_SIZE_6KB = 4u,
ENET_MAC_WDG_TIMER_SIZE_7KB = 5u, ENET_MAC_WDG_TIMER_SIZE_8KB = 6u,
ENET_MAC_WDG_TIMER_SIZE_9KB = 7u, ENET_MAC_WDG_TIMER_SIZE_10KB = 8u,
ENET_MAC_WDG_TIMER_SIZE_11KB = 9u, ENET_MAC_WDG_TIMER_SIZE_12KB = 10u,
ENET_MAC_WDG_TIMER_SIZE_13KB = 11u, ENET_MAC_WDG_TIMER_SIZE_14KB = 12u,
ENET_MAC_WDG_TIMER_SIZE_15KB = 13u }
```

MAC watchdog timer type definition.

```
enum ENET_MAC_VLANTagStripModeType { ENET_MAC_VLAN_TAG_STRIP_NEVER = 0u,
ENET_MAC_VLAN_TAG_STRIP_FILTER_PASS = 1u, ENET_MAC_VLAN_TAG_STRIP_FILTER_FAIL = 2u,
ENET_MAC_VLAN_TAG_STRIP_ALWAYS = 3u }
```

MAC VLAN tag strip type.

```
enum ENET_MAC_VLANCommandType { ENET_MAC_VLAN_COMMAND_WRITE = 0u,
ENET_MAC_VLAN_COMMAND_READ = 1u }
```

MAC VLAN indirect registers access command type.

```
enum ENET_MAC_VLANInclCommandType { ENET_MAC_VLAN_INCL_COMMAND_READ = 0u,
ENET_MAC_VLAN_INCL_COMMAND_WRITE = 1u }
```

MAC VLAN incl indirect registers access command type.

```
enum ENET_MAC_VLANTagCtrlType { ENET_MAC_VLAN_TAG_NO_OPERATION = 0u,
ENET_MAC_VLAN_TAG_DELETION = 1u, ENET_MAC_VLAN_TAG_INSERTION = 2u,
ENET_MAC_VLAN_TAG_REPLACEMENT = 3u }
```

MAC VLAN tag control type.

```
enum ENET_MAC_VLANType { ENET_MAC_VLAN_TYPE_C_VLAN = 0u, ENET_MAC_VLAN_TYPE_S_VLAN =
1u }
```

MAC VLAN tag type S-VLAN IEEE802.1ad (Q-in-Q) C-VLAN IEEE802.1Q.

```
enum ENET_MAC_VLANCompareType { ENET_MAC_VLAN_16BITS_COMPARE = 0u,
ENET_MAC_VLAN_12BITS_COMPARE = 1u }
```

MAC VLAN compare mode type.

```
enum ENET_MAC_VLANFilterGroupType { ENET_MAC_VLAN_FILTER_GROUP_0 = 0u,
ENET_MAC_VLAN_FILTER_GROUP_1 = 1u, ENET_MAC_VLAN_FILTER_GROUP_2 = 2u,
ENET_MAC_VLAN_FILTER_GROUP_3 = 3u }
```

MAC VLAN perfect filter group index type.

```
enum ENET_MAC_PauseThresholdType { ENET_MAC_PAUSE_THRESHOLD_4SLOT_TIMES = 0u,
ENET_MAC_PAUSE_THRESHOLD_28SLOT_TIMES = 1u, ENET_MAC_PAUSE_THRESHOLD_36SLOT_TIMES
= 2u, ENET_MAC_PAUSE_THRESHOLD_144SLOT_TIMES = 3u,
```

ENET_MAC_PAUSE_THRESHOLD_256SLOT_TIMES = 4u,
ENET_MAC_PAUSE_THRESHOLD_512SLOT_TIMES = 5u }

ENET flow control pause threshold type.

enum ENET_MAC_TxPacketControllerStatusType { ENET_MAC_TX_PACKET_CONTROLLER_IDLE = 0u,
ENET_MAC_TX_PACKET_CONTROLLER_WAIT = 1u,
ENET_MAC_TX_PACKET_CONTROLLER_TRANSFER_PAUSE_PACKET = 2u,
ENET_MAC_TX_PACKET_CONTROLLER_TRANSFER_INPUT_PACKET = 3u }

MAC TX packet controller status type.

enum ENET_MTL_QueueEnableType { ENET_MTL_QUEUE_DISABLE = 0u,
ENET_MTL_QUEUE_ENABLE_AV_MODE = 1u, ENET_MTL_QUEUE_ENABLE_GENERIC = 2u }

MTL queue enable type.

enum ENET_MAC_PTPoEQueueCtrlType { ENET_MAC_VLAN_PTPOE_ROUTED_AS_GENERIC = 0u,
ENET_MAC_VLAN_PTPOE_ROUTED_TO_PTP_QUEUE = 1u,
ENET_MAC_VLAN_PTPOE_ROUTED_BY_PRIORITY = 2u }

Tagged PTP over Ethernet packets queuing control type.

enum ENET_MAC_RGMIIClkSelType { ENET_MAC_RGMII_CLK_SEL_DISABLE = 0u,
ENET_MAC_RGMII_CLK_SEL_ENET_REF_CLK = 1u, ENET_MAC_RGMII_CLK_SEL_PLL0DIVH = 2u,
ENET_MAC_RGMII_CLK_SEL_PLL1DIVH = 3u }

MAC RGMII clock select type.

enum ENET_MAC_TxCkSelType { ENET_MAC_TX_CLK_SEL_DISABLE = 0u,
ENET_MAC_TX_CLK_SEL_PLL0DIVL = 1u, ENET_MAC_TX_CLK_SEL_FOSC = 2u,
ENET_MAC_TX_CLK_SEL_ENET_TX_CLK = 3u }

MAC TX clock select type.

enum ENET_MAC_MiiModeType { ENET_MAC_PHY_MII = 0u, ENET_MAC_PHY_RGMII = 1u,
ENET_MAC_PHY_RMII = 4u }

MAC PHY interface mode type.

enum ENET_MAC_CSRClockRangeType { ENET_MAC_CSR_CLOCK_RANGE_60_100MHZ = 0u,
ENET_MAC_CSR_CLOCK_RANGE_100_150MHZ = 1u, ENET_MAC_CSR_CLOCK_RANGE_20_35MHZ = 2u,
ENET_MAC_CSR_CLOCK_RANGE_35_60MHZ = 3u, ENET_MAC_CSR_CLOCK_RANGE_150_250MHZ = 4u,
ENET_MAC_CSR_CLOCK_RANGE_250_300MHZ = 5u }

MAC MDIO access clock source range type.

enum ENET_MAC_PhyOperationCmdType { ENET_MAC_PHY_OPERATION_WRITE = 1u,
ENET_MAC_PHY_OPERATION_POST_READ_INCR_ADDR_FOR_CLAUSE_45 = 2u,
ENET_MAC_PHY_OPERATION_READ = 3u }

MAC MDIO bus operation command type.

enum **ENET_MAC_PhyClauseVersionType** { **ENET_MAC_PHY_CLAUSE_22** = 0u,
ENET_MAC_PHY_CLAUSE_45 = 1u }

MAC MDIO bus clause type.

enum **ENET_MAC_AddrType** { **ENET_MAC_ADDR_TYPE_DEST** = 0u, **ENET_MAC_ADDR_TYPE_SRC** = 1u }

MAC address destination or source type.

enum **ENET_MAC_MmcCntPresetLevelType** { **ENET_MAC_MMC_COUNTER_PRESET_LEVEL_HALF** = 0u,
ENET_MAC_MMC_COUNTER_PRESET_LEVEL_FULL = 1u }

MMC counter present level type.

enum **ENET_MAC_Layer4ProtocolType** { **ENET_MAC_LAYER_4_PROTOCOL_TCP** = 0u,
ENET_MAC_LAYER_4_PROTOCOL_UDP = 1u }

MAC layer 4 filter protocol type.

enum **ENET_MAC_Layer3ProtocolType** { **ENET_MAC_LAYER_3_PROTOCOL_IPV4** = 0u,
ENET_MAC_LAYER_3_PROTOCOL_IPV6 = 1u }

MAC layer 3 filter protocol type.

enum **ENET_MAC_L3L4FilterGroupType** { **ENET_MAC_L3_L4_FILTER_GROUP_0** = 0u,
ENET_MAC_L3_L4_FILTER_GROUP_1 = 1u, **ENET_MAC_L3_L4_FILTER_GROUP_2** = 2u,
ENET_MAC_L3_L4_FILTER_GROUP_3 = 3u }

MAC layer3 layer 4 filter group index type.

enum **Enet_MMCTxCounterType** { **ENET_CTR_TX_OCTET_COUNT_GOOD_BAD** = 0u,
ENET_CTR_TX_PACKET_COUNT_GOOD_BAD, **ENET_CTR_TX_BROADCAST_PACKETS_GOOD**,
ENET_CTR_TX_MULTICAST_PACKETS_GOOD, **ENET_CTR_TX_64OCTETS_PACKETS_GOOD_BAD**,
ENET_CTR_TX_65TO127OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_128TO255OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_256TO511OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_512TO1023OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_1024TOMAXOCTETS_PACKETS_GOOD_BAD,
ENET_CTR_TX_UNICAST_PACKETS_GOOD_BAD, **ENET_CTR_TX_MULTICAST_PACKETS_GOOD_BAD**,
ENET_CTR_TX_BROADCAST_PACKETS_GOOD_BAD, **ENET_CTR_TX_UNDERFLOW_ERROR_PACKETS**,
ENET_CTR_TX_SINGLE_COLLISION_GOOD_PACKETS,
ENET_CTR_TX_MULTIPLE_COLLISION_GOOD_PACKETS, **ENET_CTR_TX_DEFERRED_PACKETS**,
ENET_CTR_TX_LATE_COLLISION_PACKETS, **ENET_CTR_TX_EXCESSIVE_COLLISION_PACKETS**,
ENET_CTR_TX_CARRIER_ERROR_PACKETS, **ENET_CTR_TX_OCTET_COUNT_GOOD**,
ENET_CTR_TX_PACKET_COUNT_GOOD, **ENET_CTR_TX_EXCESSIVE_DEFERRAL_ERROR**,
ENET_CTR_TX_PAUSE_PACKETS, **ENET_CTR_TX_VLAN_PACKETS_GOOD**,
ENET_CTR_TX_OSIZE_PACKETS_GOOD }

MAC MMC transmit statistics counter type.


```
enum Enet_MMCRxCounterType { ENET_CTR_RX_PACKETS_COUNT_GOOD_BAD = 0u,
ENET_CTR_RX_OCTET_COUNT_GOOD_BAD, ENET_CTR_RX_OCTET_COUNT_GOOD,
ENET_CTR_RX_BROADCAST_PACKETS_GOOD, ENET_CTR_RX_MULTICAST_PACKETS_GOOD,
ENET_CTR_RX_CRC_ERROR_PACKETS, ENET_CTR_RX_ALIGNMENT_ERROR_PACKETS,
ENET_CTR_RX_RUNT_ERROR_PACKETS, ENET_CTR_RX_JABBER_ERROR_PACKETS,
ENET_CTR_RX_UNDERSIZE_PACKETS_GOOD, ENET_CTR_RX_OVERSIZE_PACKETS_GOOD,
ENET_CTR_RX_64OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_65TO127OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_128TO255OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_256TO511OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_512TO1023OCTETS_PACKETS_GOOD_BAD,
ENET_CTR_RX_1024TOMAXOCTETS_PACKETS_GOOD_BAD, ENET_CTR_RX_UNICAST_PACKETS_GOOD,
ENET_CTR_RX_LENGTH_ERROR_PACKETS, ENET_CTR_RX_OUT_OF_RANGE_TYPE_PACKETS,
ENET_CTR_RX_PAUSE_PACKETS, ENET_CTR_RX_FIFO_OVERFLOW_PACKETS,
ENET_CTR_RX_VLAN_PACKETS_GOOD_BAD, ENET_CTR_RX_WATCHDOG_ERROR_PACKETS,
ENET_CTR_RX_RECEIVE_ERROR_PACKETS, ENET_CTR_RX_CONTROL_PACKETS_GOOD }
```

MAC receive statistics counter type.

```
enum ENET_MAC_TxTimestampStatusModeType
{ ENET_MAC_TX_TIMESTAMP_STATUS_MODE_PRESERVE = 0u,
ENET_MAC_TX_TIMESTAMP_STATUS_MODE_OVERWRITE = 1u }
```

PTP TX status mode type.

```
enum ENET_MAC_SystemTimeInputType { ENET_MAC_SYSTEM_TIME_INPUT_INTERNAL = 0u,
ENET_MAC_SYSTEM_TIME_INPUT_EXTERNAL = 1u }
```

PTP system time input source type.

```
enum ENET_MAC_PTPVersionType { ENET_MAC_PTP_IEEE_1588_V1 = 0u,
ENET_MAC_PTP_IEEE_1588_V2 = 1u }
```

PTP IEEE1588 version type.

```
enum ENET_MAC_TimestampRolloverType { ENET_MAC_TIMESTAMP_ROLLOVER_BINARY = 0u,
ENET_MAC_TIMESTAMP_ROLLOVER_DIGITAL = 1u }
```

PTP counter rollover mode type.

```
enum ENET_MAC_TimestampUpdateMethodType { ENET_MAC_TIMESTAMP_UPDATE_COARSE = 0u,
ENET_MAC_TIMESTAMP_UPDATE_FINE = 1u }
```

PTP timestamp update mode type.

```
enum ENET_MAC_SystemTimeOperationType { ENET_MAC_SYSTEM_TIME_ADD_TIME = 0u,
ENET_MAC_SYSTEM_TIME_SUBTRACT_TIME = 1u }
```

PTP system time offset update operation type.

enum **ENET_MAC_PPSChannelType** { **ENET_MAC_PPS_CHANNEL_0** = 0u, **ENET_MAC_PPS_CHANNEL_1** = 1u, **ENET_MAC_PPS_CHANNEL_2** = 2u, **ENET_MAC_PPS_CHANNEL_3** = 3u }

PTP PPS channel type.

enum **ENET_MAC_PPSModeType** { **ENET_MAC_PPS_MODE_PPS** = 0u, **ENET_MAC_PPS_MODE_MCGR** = 1u }

PTP PPS channel operation mode type.

enum **ENET_MAC_PPSTargetTimeModeType** { **ENET_MAC_PPS_TARGET_TIME_ONLY_INT** = 0u, **ENET_MAC_PPS_TARGET_TIME_MCGR** = 1u, **ENET_MAC_PPS_TARGET_TIME_INT_ST** = 2u, **ENET_MAC_PPS_TARGET_TIME_ONLY_ST** = 3u }

PPS target time mode type.

enum **ENET_MAC_PPSFreqCtrlType** { **ENET_MAC_PPS_BINARY_1HZ_DIGITAL_NONE** = 0u, **ENET_MAC_PPS_BINARY_2HZ_DIGITAL_1HZ** = 1u, **ENET_MAC_PPS_BINARY_4HZ_DIGITAL_2HZ** = 2u, **ENET_MAC_PPS_BINARY_8HZ_DIGITAL_4HZ** = 3u, **ENET_MAC_PPS_BINARY_16HZ_DIGITAL_8HZ** = 4u, **ENET_MAC_PPS_BINARY_32HZ_DIGITAL_16HZ** = 5u, **ENET_MAC_PPS_BINARY_64HZ_DIGITAL_32HZ** = 6u, **ENET_MAC_PPS_BINARY_128HZ_DIGITAL_64HZ** = 7u, **ENET_MAC_PPS_BINARY_256HZ_DIGITAL_128HZ** = 8u, **ENET_MAC_PPS_BINARY_512HZ_DIGITAL_256HZ** = 9u, **ENET_MAC_PPS_BINARY_1024HZ_DIGITAL_512HZ** = 10u, **ENET_MAC_PPS_BINARY_2048HZ_DIGITAL_1024HZ** = 11u, **ENET_MAC_PPS_BINARY_4096HZ_DIGITAL_2048HZ** = 12u, **ENET_MAC_PPS_BINARY_8192HZ_DIGITAL_4096HZ** = 13u, **ENET_MAC_PPS_BINARY_16384HZ_DIGITAL_8192HZ** = 14u, **ENET_MAC_PPS_BINARY_32768HZ_DIGITAL_16384HZ** = 15u }

PTP fixed PPS output frequency control type.

enum **ENET_MAC_PPSCmdType** { **ENET_MAC_PPS_NO_COMMAND** = 0u, **ENET_MAC_PPS_START_SINGLE_PULSE** = 1u, **ENET_MAC_PPS_START_PULSE_TRAIN** = 2u, **ENET_MAC_PPS_CANCEL_START** = 3u, **ENET_MAC_PPS_STOP_PULSE_TRAIN_AT_TIME** = 4u, **ENET_MAC_PPS_STOP_PULSE_TRAIN_IMMEDIATELY** = 5u, **ENET_MAC_PPS_CANCEL_STOP_PULSE_TRAIN** = 6u }

PTP flexible PPS output command type.

enum **ENET_MAC_MCGRCmdType** { **ENET_MAC_MCGR_OPERATION_NOT_CARRIED_OUT** = 0u, **ENET_MAC_MCGR_CAPTURE_RISING_EDGE** = 1u, **ENET_MAC_MCGR_CAPTURE_FALLING_EDGE** = 2u, **ENET_MAC_MCGR_CAPTURE_BOTH_EDGE** = 3u, **ENET_MAC_MCGR_TOGGLE_OUTPUT_ON_COMPARE** = 9u, **ENET_MAC_MCGR_PULSE_OUTPUT_LOW_ON_COMPARE** = 10u, **ENET_MAC_MCGR_PULSE_OUTPUT_HIGH_ON_COMPARE** = 11u }

PTP flexible MCGR command type.

```
enum ENET_MAC_PTPSnapshotMsgType { ENET_MAC_PTP_SNAPSHOT_FOR_E2E = (0x0u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_SYNC = (0x1u << 14u), ENET_MAC_PTP_SNAPSHOT_FOR_DELAYREQ =
(0x3u << 14u), ENET_MAC_PTP_SNAPSHOT_FOR_ALL_PTP = (0x4u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_SLAVE_NODE_EVENT = (0x5u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_MASTER_NODE_EVENT = (0x7u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_E2E_EVENT = (0x8u << 14u),
ENET_MAC_PTP_SNAPSHOT_FOR_P2P_EVENT = (0xCu << 14u) }
```

PTP snapshot packet select type.

```
enum ENET_MAC_PTTPacketLayerType { ENET_MAC_PTP_OVER_ETHERNET =
ENET_MAC_TIMESTAMP_CONTROL_TSIPENA_MASK, ENET_MAC_PTP_OVER_IPV4 =
ENET_MAC_TIMESTAMP_CONTROL_TSIPV4ENA_MASK, ENET_MAC_PTP_OVER_IPV6 =
ENET_MAC_TIMESTAMP_CONTROL_TSIPV6ENA_MASK }
```

PTP snapshot network layer select type.

```
enum ENET_MAC_PTPClkSelType { ENET_MAC_PTP_CLK_SEL_PCC = 0u,
ENET_MAC_PTP_CLK_SEL_TX_CLK = 1u }
```

MAC PTP clock select type.

```
enum ENET_MAC_PPSPinDirType { ENET_PTP_PPS_DIR_INPUT = 0u, ENET_PTP_PPS_DIR_OUTPUT =
1u }
```

PTP PPS channel direction type.

```
enum ENET_MTL_QueueType { ENET_MTL_QUEUE_0 = 0u, ENET_MTL_QUEUE_1 = 1u }
```

ENET MTL queue type.

```
enum ENET_MTL_TxScheduleAlgorithmType { ENET_MTL_SCHED_ALGO_WRR = 0u,
ENET_MTL_SCHED_ALGO_SP = 3u }
```

ENET TX schedule algorithm type.

```
enum ENET_MTL_RxArbitrationAlgorithmType { ENET_MTL_RX_ARBITRATION_STRICT_POLICY = 0u,
ENET_MTL_RX_ARBITRATION_WEIGHTED_STRICT_POLICY = 1u }
```

ENET RX arbitration type.

```
enum ENET_MTL_DebugAccessFIFOType { ENET_MTL_DEBUG_ACCESS_TX_FIFO = 0u,
ENET_MTL_DEBUG_ACCESS_TX_STATUS_FIFO = 1u, ENET_MTL_DEBUG_ACCESS_TSO_FIFO = 2u,
ENET_MTL_DEBUG_ACCESS_RX_FIFO = 3u }
```

MTL debug access type.

```
enum ENET_MTL_FIFOPacketStatusType { ENET_MTL_PACKET_STATE_PACKET_DATA = 0u,
ENET_MTL_PACKET_STATE_TX_CTRL_WORD_RX_NORMAL_STATUS = 1u,
ENET_MTL_PACKET_STATE_TX_SOP_DATA_RX_LAST_STATUS = 2u,
ENET_MTL_PACKET_STATE_TX_EOP_DATA_RX_EOP = 3u }
```

MTL fifo packet status type.

enum **ENET_MTL_ByteEnableType** { **ENET_MTL_BYTE_0_VALID** = 0u, **ENET_MTL_BYTE_0_1_VALID** = 1u, **ENET_MTL_BYTE_0_1_2_VALID** = 2u, **ENET_MTL_BYTE_ALL_VALID** = 3u }

MTL byte enable type.

enum **ENET_MTL_FIFODebugAccessType** { **ENET_MTL_FIFO_DEBUG_ACCESS_RESTRICTED** = 0u, **ENET_MTL_FIFO_DEBUG_ACCESS_UNRESTRICTED** = 1u }

MTL fifo debug access type.

enum **ENET_MTL_TxThresholdType** { **ENET_MTL_TX_THRESHOLD_32B** = 0u, **ENET_MTL_TX_THRESHOLD_64B** = 1u, **ENET_MTL_TX_THRESHOLD_96B** = 2u, **ENET_MTL_TX_THRESHOLD_128B** = 3u, **ENET_MTL_TX_THRESHOLD_192B** = 4u, **ENET_MTL_TX_THRESHOLD_256B** = 5u, **ENET_MTL_TX_STORE_FORWARD** = (1u << **ENET_MTL_TXQn_OPERATION_MODE_TTC_WIDTH**) }

MTL TX threshold type.

enum **ENET_MTL_WriteControllerStatusType** { **ENET_MTL_WRITE_CONTROLLER_IDLE** = 0u, **ENET_MTL_WRITE_CONTROLLER_ACTIVE** = 1u }

MTL write controller status type.

enum **ENET_MTL_ReadControllerStatusType** { **ENET_MTL_READ_CONTROLLER_IDLE** = 0u, **ENET_MTL_READ_CONTROLLER_READ** = 1u, **ENET_MTL_READ_CONTROLLER_WAIT** = 2u, **ENET_MTL_READ_CONTROLLER_FLUSH** = 3u }

MTL read controller status type.

enum **ENET_MTL_RxFlowCtrlThresholdType** { **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_1_KB** = 0u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_1_5_KB** = 1u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_2_KB** = 2u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_2_5_KB** = 3u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_3_KB** = 4u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_3_5_KB** = 5u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_4_KB** = 6u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_4_5_KB** = 7u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_5_KB** = 8u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_5_5_KB** = 9u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_6_KB** = 10u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_6_5_KB** = 11u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_7_KB** = 12u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_7_5_KB** = 13u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_8_KB** = 14u, **ENET_MTL_RX_FLOWCTRL_THRESHOLD_FULL_8_5_KB** = 15u }

MTL RX flow control threshold type.

```
enum ENET_MTL_RxThresholdType { ENET_MTL_RX_THRESHOLD_64B = 0u,
ENET_MTL_RX_THRESHOLD_32B = 1u, ENET_MTL_RX_THRESHOLD_96B = 2u,
ENET_MTL_RX_THRESHOLD_128B = 3u, ENET_MTL_RX_STORE_FORWARD = (1u <<
ENET_MTL_RXQn_OPERATION_MODE_RTC_WIDTH) }
```

MTL RX threshold type.

```
enum ENET_MTL_RxQueueStatusType { ENET_MTL_RX_QUEUE_EMPTY = 0u,
ENET_MTL_RX_QUEUE_BELOW_FLOW_CTRL_DEACTIVATE_THRES = 1u,
ENET_MTL_RX_QUEUE_ABOVE_FLOW_CTRL_ACTIVATE_THRES = 2u, ENET_MTL_RX_QUEUE_FULL = 3u }
```

MTL RX queue status type.

```
enum ENET_MTL_GCLIntervalMultipleType { ENET_MTL_GCL_INTERVAL_X_1 = 0x0u,
ENET_MTL_GCL_INTERVAL_X_2 = 0x1u, ENET_MTL_GCL_INTERVAL_X_4 = 0x2u,
ENET_MTL_GCL_INTERVAL_X_8 = 0x3u, ENET_MTL_GCL_INTERVAL_X_16 = 0x4u,
ENET_MTL_GCL_INTERVAL_X_32 = 0x5u, ENET_MTL_GCL_INTERVAL_X_64 = 0x6u,
ENET_MTL_GCL_INTERVAL_X_128 = 0x7u }
```

Multiple of gate control list interval time.

```
enum ENET_MTL_ScheduleErrorLoopCountType { ENET_MTL_GCL_4_ITERATIONS = 0x0u,
ENET_MTL_GCL_8_ITERATIONS = 0x1u, ENET_MTL_GCL_16_ITERATIONS = 0x2u,
ENET_MTL_GCL_32_ITERATIONS = 0x3u }
```

Loop count to report Scheduling Error.

```
enum ENET_MTL_GclRegistersType { ENET_MTL_GCL_BTR_LOW = 0x0u, ENET_MTL_GCL_BTR_HIGH =
0x1u, ENET_MTL_GCL_CTR_LOW = 0x2u, ENET_MTL_GCL_CTR_HIGH = 0x3u, ENET_MTL_GCL_TER = 0x4u,
ENET_MTL_GCL_LLRR = 0x5u }
```

Gcl related registers.

```
enum ENET_MTL_GclOperationType { ENET_MTL_GCL_WRITE = 0x0u, ENET_MTL_GCL_READ = 0x1u }
```

Gcl operation type.

```
enum ENET_MTL_GateControlListType { ENET_MTL_GCL_NUMBER_0 = 0x0u,
ENET_MTL_GCL_NUMBER_1 = 0x1u }
```

MTL gate control list index type.

```
enum ENET_DMA_ChannelType { ENET_DMA_CHANNEL_0 = 0x0u, ENET_DMA_CHANNEL_1 = 0x1u }
```

ENET DMA channel index type.

```
enum ENET_DMA_StatusType { ENET_DMA_STATUS_IDLE = 0x0u, ENET_DMA_STATUS_ACTIVE = 0x1u }
```

ENET DMA running status.

```
enum ENET_DMA_IntModeType { ENET_DMA_INTM_SBD_PERCH_PULSE = 0u,
ENET_DMA_INTM_SBD_PERCH_LEVEL = 1u, ENET_DMA_INTM_SBD_PERCH_LEVEL_REENTRANT = 2u }
```

The interrupt mode of ENET DMA.

```
enum ENET_DMA_PriorityRatioType { ENET_DMA_PRIORITY_RATIO_1_1 = 0u,
ENET_DMA_PRIORITY_RATIO_2_1 = 1u, ENET_DMA_PRIORITY_RATIO_3_1 = 2u,
ENET_DMA_PRIORITY_RATIO_4_1 = 3u, ENET_DMA_PRIORITY_RATIO_5_1 = 4u,
ENET_DMA_PRIORITY_RATIO_6_1 = 5u, ENET_DMA_PRIORITY_RATIO_7_1 = 6u,
ENET_DMA_PRIORITY_RATIO_8_1 = 7u }
```

ENET DMA priority ratio type.

```
enum ENET_DMA_PriorityModeType { ENET_DMA_PRIORITY_MODE_RX_TX = 0u,
ENET_DMA_PRIORITY_MODE_TX_RX = 1u }
```

ENET DMA priority mode type.

```
enum ENET_DMA_TxArbitrationAlgorithmType
{ ENET_DMA_TX_ARBITRATION_ALGORITHM_FIXED_PRIORITY = 0u,
ENET_DMA_TX_ARBITRATION_ALGORITHM_WEIGHTED_FIXED_PRIORITY = 1u,
ENET_DMA_TX_ARBITRATION_ALGORITHM_WEIGHTED_ROUND_ROBIN = 2u }
```

ENET DMA arbitration algorithm between transmit channels.

```
enum ENET_DMA_ArbitrationSchemeType
{ ENET_DMA_TXRX_ARBITRATION_SCHEME_WEIGHTED_ROUND_ROBIN = 0u,
ENET_DMA_TXRX_ARBITRATION_SCHEME_FIXED_PRIORITY = 1u }
```

ENET DMA arbitration scheme between the transmit and receive paths.

```
enum ENET_DMA_BurstModeType { ENET_BURSTLENGTH_FIXED =
ENET_DMA_SYSBUS_MODE_FB_MASK, ENET_BURSTLENGTH_MIXED =
ENET_DMA_SYSBUS_MODE_MB_MASK }
```

ENET DMA burst mode type.

```
enum ENET_DMA_InterruptFlagType { ENET_DMA_TX_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_TIE_MASK, ENET_DMA_TX_STOPPED_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_TXSE_MASK, ENET_DMA_TX_BUFFER_UNAVAILABLE_INTERRUPT
= ENET_DMA_CHn_INTERRUPT_ENABLE_TBUE_MASK, ENET_DMA_RX_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_RIE_MASK, ENET_DMA_RX_BUFFER_UNAVAILABLE_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_RBUE_MASK, ENET_DMA_RX_STOPPED_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_RSE_MASK, ENET_DMA_RX_WATCHDOG_TIMEOUT_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_RWTE_MASK, ENET_DMA_EARLY_TX_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_ETIE_MASK, ENET_DMA_EARLY_RX_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_ERIE_MASK, ENET_DMA_FATAL_BUS_ERROR_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_FBEE_MASK,
ENET_DMA_CONTEXT_DESCRIPTOR_ERROR_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_CDEE_MASK, ENET_DMA_ABNORMAL_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_AIE_MASK, ENET_DMA_NORMAL_INTERRUPT =
ENET_DMA_CHn_INTERRUPT_ENABLE_NIE_MASK }
```

ENET DMA interrupt flag type.

```
enum ENET_DMA_WDGCounterUnitType { ENET_DMA_WDG_COUNTER_UNIT_256_CYCLE = 0x0u,  
ENET_DMA_WDG_COUNTER_UNIT_512_CYCLE = 0x1u, ENET_DMA_WDG_COUNTER_UNIT_1024_CYCLE  
= 0x2u, ENET_DMA_WDG_COUNTER_UNIT_2048_CYCLE = 0x3u }
```

ENET DMA watchdog counter unit type.

```
enum ENET_DMA_RxErrorType { ENET_DMA_RX_NO_ERROR = 0x0u,  
ENET_DMA_RX_READ_DESCRIPTOR_ERROR = 0x7u, ENET_DMA_RX_WRITE_DESCRIPTOR_ERROR = 0x6u,  
ENET_DMA_RX_READ_BUFFER_ERROR = 0x5u, ENET_DMA_RX_WRITE_BUFFER_ERROR = 0x4u }
```

ENET DMA RX error type.

```
enum ENET_DMA_TxErrorType { ENET_DMA_TX_NO_ERROR = 0x0u,  
ENET_DMA_TX_READ_DESCRIPTOR_ERROR = 0x7u, ENET_DMA_TX_WRITE_DESCRIPTOR_ERROR = 0x6u,  
ENET_DMA_TX_READ_BUFFER_ERROR = 0x5u, ENET_DMA_TX_WRITE_BUFFER_ERROR = 0x4u }
```

ENET DMA TX error type.

Functions

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetConfiguration (const ENET_Type *const pEnet)
```

Get the MAC Configuration Register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetConfiguration (ENET_Type *const pEnet, uint32_t u32Config)
```

Set the MAC Configuration Register value.

```
LOCAL_INLINE ENET_MAC_SrcAddrCtrlType ENET_HWA_MAC_GetSrcAddrInsertReplaceCtrl (const  
ENET_Type *const pEnet)
```

Get the source address insertion or replacement control type.

```
LOCAL_INLINE void ENET_HWA_MAC_SetSrcAddrInsertReplaceCtrl (ENET_Type *const pEnet,  
ENET_MAC_SrcAddrCtrlType eInsertReplaceCtrl)
```

Set the source address insertion or replacement control type.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetIPChecksumOffloadEnFlag (const ENET_Type *const pEnet)
```

Get checksum offload function enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetIPChecksumOffloadEnFlag (ENET_Type *const pEnet, bool  
bEnable)
```

Set checksum offload function enable flag.

LOCAL_INLINE **ENET_MAC_InterPacketGapType** **ENET_HWA_MAC_GetMinInterPacketGap** (const ENET_Type *const pEnet)

Get the minimum Inter-Packet Gap.

LOCAL_INLINE void **ENET_HWA_MAC_SetMinInterPacketGap** (ENET_Type *const pEnet, **ENET_MAC_InterPacketGapType** eGap)

Set the minimum Inter-Packet Gap value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetGiantPacketSizeLimitCtrlEnFlag** (const ENET_Type *const pEnet)

Get giant packet size Limit control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetGiantPacketSizeLimitCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set giant packet size Limit control enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_Get2KPacketSupportFlag** (const ENET_Type *const pEnet)

Get 2K packet support enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_Set2KPacketSupportFlag** (ENET_Type *const pEnet, bool bEnable)

Set 2K packet support enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetStripTypePacketCRCEnFlag** (const ENET_Type *const pEnet)

Get CRC stripping for type packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetStripTypePacketCRCEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set CRC stripping for type packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetAutoStripPadOrCRCEnFlag** (const ENET_Type *const pEnet)

Get automatic Pad or CRC stripping enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetAutoStripPadOrCRCEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set automatic Pad or CRC stripping enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetWatchdogDisableFlag** (const ENET_Type *const pEnet)

Get receiver watchdog timer disabled flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetWatchdogDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set receiver watchdog timer disabled flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetJabberTimerDisableFlag** (const ENET_Type *const pEnet)

Get transmitter jabber timer disabled flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetJabberTimerDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set transmitter jabber timer disabled flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetJumboPacketEnFlag** (const ENET_Type *const pEnet)

Get jumbo packet enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetJumboPacketEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set jumbo packet enable flag.

LOCAL_INLINE **ENET_MAC_SpeedType** **ENET_HWA_MAC_GetSpeed** (const ENET_Type *const pEnet)

Get ethernet line speed.

LOCAL_INLINE void **ENET_HWA_MAC_SetSpeed** (ENET_Type *const pEnet, **ENET_MAC_SpeedType** eSpeed)

Set ethernet line speed.

LOCAL_INLINE **ENET_MAC_DuplexModeType** **ENET_HWA_MAC_GetDuplexMode** (const ENET_Type *const pEnet)

Get ethernet duplex mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetDuplexMode** (ENET_Type *const pEnet,
ENET_MAC_DuplexModeType eMode)

Set ethernet duplex mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLoopbackModeEnFlag** (const ENET_Type *const pEnet)

Get loopback mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLoopbackModeEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set loopback mode enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetFullDuplexCRSEnFlag** (const ENET_Type *const pEnet)

Get carrier sense before transmission enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetFullDuplexCRSEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set carrier sense before transmission enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetReceiveOwnDisableFlag** (const ENET_Type *const pEnet)

Get receive own disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetReceiveOwnDisableFlag** (ENET_Type *const pEnet, bool
bDisable)

Set receive own disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetHalfDuplexCRSDisableFlag** (const ENET_Type *const pEnet)

Get carrier sense during transmission disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetHalfDuplexCRSDisableFlag** (ENET_Type *const pEnet, bool
bDisable)

Set carrier sense during transmission disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRetryDisableFlag** (const ENET_Type *const pEnet)

Get transmission retry disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRetryDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Set transmission retry disable flag.

LOCAL_INLINE **ENET_MAC_BackOffLimitType** **ENET_HWA_MAC_GetBackOffLimit** (const ENET_Type *const pEnet)

Get back-off limit.

LOCAL_INLINE void **ENET_HWA_MAC_SetBackOffLimit** (ENET_Type *const pEnet, **ENET_MAC_BackOffLimitType** eLimit)

Set back-off limit.

LOCAL_INLINE bool **ENET_HWA_MAC_GetDeferralCheckEnFlag** (const ENET_Type *const pEnet)

Get deferral check enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetDeferralCheckEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set deferral check enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxEnFlag** (const ENET_Type *const pEnet)

Get transmitter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set transmitter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxEnFlag** (const ENET_Type *const pEnet)

Get receiver enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set receiver enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetExtConfiguration** (const ENET_Type *const pEnet)

Get the MAC extended configuration register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetExtConfiguration** (ENET_Type *const pEnet, uint32_t u32Configuration)

Set the MAC extended configuration register value.

LOCAL_INLINE **ENET_MAC_ExtInterPacketGapType** **ENET_HWA_MAC_GetExtInterPacketGap** (const ENET_Type *const pEnet)

Get the MAC extended inter-packet gap.

LOCAL_INLINE void **ENET_HWA_MAC_SetExtInterPacketGap** (ENET_Type *const pEnet, **ENET_MAC_ExtInterPacketGapType** eGap)

Set the MAC extended inter-packet gap.

LOCAL_INLINE bool **ENET_HWA_MAC_GetExtInterPacketGapEnFlag** (const ENET_Type *const pEnet)

Get extended inter-packet gap enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetExtInterPacketGapEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set extended inter-packet gap enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPacketDuplicationCtrlEnFlag** (const ENET_Type *const pEnet)

Get packet duplication control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPacketDuplicationCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set packet duplication control enable flag.

LOCAL_INLINE **ENET_MAC_SlowProtocolDetectType**

ENET_HWA_MAC_GetUnicastSlowProtocolPacketDetectMethod (const ENET_Type *const pEnet)

Get unicast slow protocol packet detect method.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastSlowProtocolPacketDetectMethod** (ENET_Type *const pEnet, **ENET_MAC_SlowProtocolDetectType** eMethod)

Set unicast slow protocol packet detect method.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSlowDetectEnFlag** (const ENET_Type *const pEnet)

Get slow protocol detection enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSlowDetectEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set slow protocol detection enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxCRCCheckDisableFlag** (const ENET_Type *const pEnet)
Get CRC checking for Received packets disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxCRCCheckDisableFlag** (ENET_Type *const pEnet, bool bDisable)
Set CRC checking for rReceived packets disable flag.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetGiantPacketSizeLimit** (const ENET_Type *const pEnet)
Get giant packet size limit value.

LOCAL_INLINE void **ENET_HWA_MAC_SetGiantPacketSizeLimit** (ENET_Type *const pEnet, uint16_t u16SizeLimit)
Set giant packet size limit value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPacketFilter** (const ENET_Type *const pEnet)
Get the MAC packet filter register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetPacketFilter** (ENET_Type *const pEnet, uint32_t u32FilterCtrl)
Set the MAC packet filter register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetReceiveAllEnFlag** (const ENET_Type *const pEnet)
Get receive all packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetReceiveAllEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set receive all packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetDropNonTCPUDPOverIPEnFlag** (const ENET_Type *const pEnet)
Get drop non-TCP/UDP over IP packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetDropNonTCPUDPOverIPEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set drop non-TCP/UDP over IP packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3Layer4FilterEnFlag** (const ENET_Type *const pEnet)

Get layer 3 and layer 4 filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Layer4FilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set layer 3 and layer 4 filter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagFilterEnFlag** (const ENET_Type *const pEnet)

Get VLAN tag filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterFilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag filter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetHashOrPerfectFilterEnFlag** (const ENET_Type *const pEnet)

Get hash or perfect filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetHashOrPerfectFilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set hash or perfect filter enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSourceAddressFilterEnFlag** (const ENET_Type *const pEnet)

Get source address filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSourceAddressFilterEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set source address filter enable flag.

LOCAL_INLINE **ENET_MAC_AddrFilterModeType** **ENET_HWA_MAC_GetSrcAddrFilterMode** (const ENET_Type *const pEnet)

Get source address filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetSrcAddrFilterMode** (ENET_Type *const pEnet,
ENET_MAC_AddrFilterModeType eFilterMode)

Set source address filter mode.

LOCAL_INLINE **ENET_MAC_CtrlPacketFilterModeType** **ENET_HWA_MAC_GetCtrlPacketFilterMode** (const
ENET_Type *const pEnet)

Get control packets filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetCtrlPacketFilterMode** (ENET_Type *const pEnet,
ENET_MAC_CtrlPacketFilterModeType eFilterMode)

Set control packets filter mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetBroadcastPacketDisableFlag** (const ENET_Type *const pEnet)

Get broadcast packets disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetBroadcastPacketDisableFlag** (ENET_Type *const pEnet, bool
bDisable)

Set broadcast packets disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPassAllMulticastEnFlag** (const ENET_Type *const pEnet)

Get pass all the received multicast packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPassAllMulticastEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set pass all the received multicast packets enable flag.

LOCAL_INLINE **ENET_MAC_AddrFilterModeType** **ENET_HWA_MAC_GetDestAddrFilterMode** (const
ENET_Type *const pEnet)

Get destination address filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetDestAddrFilterMode** (ENET_Type *const pEnet,
ENET_MAC_AddrFilterModeType eFilterMode)

Set destination address filter mode.

LOCAL_INLINE **ENET_MAC_HashOrPerfectModeType**

ENET_HWA_MAC_GetHashOrPerfectMulticastMode (const ENET_Type *const pEnet)

Get hash or perfect multicast filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetHashOrPerfectMulticastMode** (ENET_Type *const pEnet,

ENET_MAC_HashOrPerfectModeType eMode)

Set hash or perfect multicast filter mode.

LOCAL_INLINE **ENET_MAC_HashOrPerfectModeType** **ENET_HWA_MAC_GetHashOrPerfectUnicastMode**

(const ENET_Type *const pEnet)

Get hash or perfect unicast filter mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetHashOrPerfectUnicastMode** (ENET_Type *const pEnet,

ENET_MAC_HashOrPerfectModeType eMode)

Set hash or perfect unicast filter mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPromiscuousModeEnFlag** (const ENET_Type *const pEnet)

Get promiscuous mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPromiscuousModeEnFlag** (ENET_Type *const pEnet, bool

bEnable)

Set promiscuous mode enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetWatchdogTimeout** (const ENET_Type *const pEnet)

Get programmable watchdog timeout register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetWatchdogTimeout** (ENET_Type *const pEnet, uint32_t

u32Settings)

Set programmable watchdog timeout register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetWatchdogEnFlag** (const ENET_Type *const pEnet)

Get programmable watchdog enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetWatchdogEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set programmable watchdog enable flag.

```
LOCAL_INLINE ENET_MAC_WDGTimerSizeType ENET_HWA_MAC_GetWatchdogTimerSize (const  
ENET_Type *const pEnet)
```

Get programmable watchdog timer size value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetWatchdogTimerSize (ENET_Type *const pEnet,  
ENET_MAC_WDGTimerSizeType eSize)
```

Set programmable watchdog timer size.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHashTableFirst32Bits (const ENET_Type *const pEnet)
```

Get the hash table first 32 bits value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetHashTableFirst32Bits (ENET_Type *const pEnet, uint32_t  
u32Hash0)
```

Set the hash table first 32 bits value.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetHashTableSecond32Bits (const ENET_Type *const pEnet)
```

Get the hash table second 32 bits value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetHashTableSecond32Bits (ENET_Type *const pEnet, uint32_t  
u32Hash1)
```

Set the hash table second 32 bits value.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetVLANTagCtrl (const ENET_Type *const pEnet)
```

Get VLAN tag control register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetVLANTagCtrl (ENET_Type *const pEnet, uint32_t u32VlanCtrl)
```

Set VLAN tag control register value.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetRxStatusInnerVLANTagEnFlag (const ENET_Type *const pEnet)
```

Get inner VLAN tag enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxStatusInnerVLANTagEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set inner VLAN tag enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagStripModeType** **ENET_HWA_MAC_GetInnerVLANTagStripMode** (const ENET_Type *const pEnet)

Get inner VLAN tag stripping mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagStripMode** (ENET_Type *const pEnet, **ENET_MAC_VLANTagStripModeType** eMode)

Set inner VLAN tag stripping mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetInnerVLANTagCmpEnFlag** (const ENET_Type *const pEnet)

Get inner VLAN tag comparison enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagCmpEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set inner VLAN tag comparison enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetDoubleVLANProcessEnFlag** (const ENET_Type *const pEnet)

Get double VLAN processing enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetDoubleVLANProcessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set double VLAN processing enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagHashTableMatchEnFlag** (const ENET_Type *const pEnet)

Get VLAN tag hash table match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagHashTableMatchEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag hash table match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxStatusVLANTagEnFlag** (const ENET_Type *const pEnet)
Get VLAN tag in rx status enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxStatusVLANTagEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set VLAN tag in rx status enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagStripModeType** **ENET_HWA_MAC_GetVLANTagStripMode** (const ENET_Type *const pEnet)
Get VLAN tag stripping mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagStripMode** (ENET_Type *const pEnet, **ENET_MAC_VLANTagStripModeType** eMode)
Set VLAN tag stripping mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTypeCheckDisableFlag** (const ENET_Type *const pEnet)
Get VLAN type check disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTypeCheckDisableFlag** (ENET_Type *const pEnet, bool bDisable)
Set VLAN type check disable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxSVLANMatchEnFlag** (const ENET_Type *const pEnet)
Get Receive S-VLAN match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxSVLANMatchEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set Receive S-VLAN match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSVLANEnFlag** (const ENET_Type *const pEnet)
Get S-VLAN enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSVLANEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set S-VLAN enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANInverseMatchEnFlag** (const ENET_Type *const pEnet)
Get VLAN tag inverse match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInverseMatchEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set VLAN tag inverse match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_Get12BitVLANTagEnFlag** (const ENET_Type *const pEnet)
Get 12-Bit VLAN tag comparison enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_Set12BitVLANTagEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set 12-Bit VLAN tag comparison enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetVLANTagFilterRegOffset** (const ENET_Type *const pEnet)
Get VLAN tag filter register address offset.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterRegOffset** (ENET_Type *const pEnet, uint8_t u8Offset)
Set VLAN tag filter register address offset.

LOCAL_INLINE **ENET_MAC_VLANCommandType** **ENET_HWA_MAC_GetVLANCommandType** (const ENET_Type *const pEnet)
Get VLAN tag filter register command type.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANCommandType** (ENET_Type *const pEnet, **ENET_MAC_VLANCommandType** eCommand)
Set VLAN tag filter register command type.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetVLANOperationStatus** (const ENET_Type *const pEnet)
Get VLAN tag filter register operation status.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANOperationBusy** (ENET_Type *const pEnet)
Set VLAN tag filter register operation busy status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetVLANTagData** (const ENET_Type *const pEnet)

Get VLAN tag data registers value.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagData** (ENET_Type *const pEnet, uint32_t u32VLANData)

Set VLAN tag data registers value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetVLANHashTable** (const ENET_Type *const pEnet)

Get VLAN hash table registers value.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANHashTable** (ENET_Type *const pEnet, uint16_t u16HashTable)

Set VLAN hash table registers value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetVLANInclusion** (const ENET_Type *const pEnet)

Get VLAN tag inclusion register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInclusion** (ENET_Type *const pEnet, uint32_t u32Inclusion)

Set VLAN tag inclusion register value.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetVLANInclusionOperationStatus** (const ENET_Type *const pEnet)

Get VLAN inclusion operation status.

LOCAL_INLINE **ENET_MAC_VLANInclCommandType** **ENET_HWA_MAC_GetVLANInclusionCommandType** (const ENET_Type *const pEnet)

Set VLAN inclusion operation status.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInclusionCommandType** (ENET_Type *const pEnet, **ENET_MAC_VLANInclCommandType** eCommand)

Set VLAN tag inclusion command.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetVLANInclusionAddr** (const ENET_Type *const pEnet)

Get VLAN inclusion address.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANInclusionAddr** (ENET_Type *const pEnet, uint8_t u8Addr)
Set VLAN inclusion address.

LOCAL_INLINE bool **ENET_HWA_MAC_GetChannelBasedVLANTagInsertEnFlag** (const ENET_Type *const pEnet)
Get channel based tag insertion enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetChannelBasedVLANTagInsertEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set channel based tag insertion enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagInputEnFlag** (const ENET_Type *const pEnet)
Get VLAN tag input enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagInputEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set VLAN tag input enable flag.

LOCAL_INLINE **ENET_MAC_VLANType** **ENET_HWA_MAC_GetVLANType** (const ENET_Type *const pEnet)
Get VLAN type.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANType** (ENET_Type *const pEnet, **ENET_MAC_VLANType** eType)
Set VLAN type.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANPriorityCtrlEnFlag** (const ENET_Type *const pEnet)
Get VLAN priority control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANPriorityCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set VLAN tag input enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagCtrlType** **ENET_HWA_MAC_GetVLANTagCtrlOperation** (const ENET_Type *const pEnet)
Get VLAN tag control mode in transmit packets.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagCtrlOperation** (ENET_Type *const pEnet,
ENET_MAC_VLANTagCtrlType eCtrl)

Set VLAN tag control mode in transmit packets.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetTxVLANTag** (const ENET_Type *const pEnet)

Get Tx VLAN tag value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxVLANTag** (ENET_Type *const pEnet, uint16_t u16VLANTag)

Set Tx VLAN tag value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetInnerVLANInclusion** (const ENET_Type *const pEnet)

Get inner VLAN tag inclusion or replacement register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANInclusion** (ENET_Type *const pEnet, uint32_t
u32Inclusion)

Set inner VLAN tag inclusion or replacement register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetInnerVLANTagInputEnFlag** (const ENET_Type *const pEnet)

Get inner VLAN tag input enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagInputEnFlag** (ENET_Type *const pEnet, bool
bEnable)

Set inner VLAN tag input enable flag.

LOCAL_INLINE **ENET_MAC_VLANType** **ENET_HWA_MAC_GetInnerVLANType** (const ENET_Type *const
pEnet)

Get inner VLAN type.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANType** (ENET_Type *const pEnet,
ENET_MAC_VLANType eType)

Set inner VLAN type.

LOCAL_INLINE bool **ENET_HWA_MAC_GetInnerVLANPriorityCtrlEnFlag** (const ENET_Type *const pEnet)

Get inner VLAN priority control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANPriorityCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set inner VLAN priority control enable flag.

LOCAL_INLINE **ENET_MAC_VLANTagCtrlType** **ENET_HWA_MAC_GetInnerVLANTagCtrl** (const ENET_Type *const pEnet)

Get inner VLAN tag control mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetInnerVLANTagCtrl** (ENET_Type *const pEnet, **ENET_MAC_VLANTagCtrlType** eCtrl)

Set inner VLAN tag control mode.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetTxInnerVLANTag** (const ENET_Type *const pEnet)

Get inner VLAN tag value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxInnerVLANTag** (ENET_Type *const pEnet, uint16_t u16VLANTag)

Set inner VLAN tag value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetQnTxFlowCtrl** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx flow control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxFlowCtrl** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint32_t u32FlowCtrl)

Set Queue n tx flow control register value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetQnTxPauseTime** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx pause time value.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxPauseTime** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint16_t u16PauseTime)

Set Queue n tx pause time value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetQnTxZeroQuantaPauseDisableFlag** (const ENET_Type *const
pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx zero quanta pause disable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxZeroQuantaPauseDisableFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bDisable)

Set Queue n tx zero quanta pause disable flag.

LOCAL_INLINE **ENET_MAC_PauseThresholdType** **ENET_HWA_MAC_GetQnTxPauseLowThreshold** (const
ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Queue n tx pause low threshold.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxPauseLowThreshold** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, **ENET_MAC_PauseThresholdType** ePauseThreshold)

Set Queue n tx pause low threshold.

LOCAL_INLINE bool **ENET_HWA_MAC_GetQnTxFlowCtrlEnableFlag** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get Queue n tx flow control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetQnTxFlowCtrlEnableFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)

Set Queue n tx flow control enable flag.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetFlowCtrlOrBackpressureStatus** (const
ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get flow control busy or backpressure status of Queue n.

LOCAL_INLINE void **ENET_HWA_MAC_InitiatePausePacketOrBackpressure** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Initiate flow control busy or backpressure of Queue n.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxFlowCtrl** (const ENET_Type *const pEnet)

Get receive flow control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxFlowCtrl** (ENET_Type *const pEnet, uint32_t u32FlowCtrl)

Set receive flow control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUnicastPausePacketDetectEnFlag** (const ENET_Type *const pEnet)

Get unicast pause packet detect enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastPausePacketDetectEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set unicast pause packet detect enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxFlowCtrlEnFlag** (const ENET_Type *const pEnet)

Get receive flow control enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxFlowCtrlEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set receive flow control enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl0** (const ENET_Type *const pEnet)

Get Receive queue control 0 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl0** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Receive queue control 0 register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl1** (const ENET_Type *const pEnet)

Get Receive queue control 1 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl1** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Receive queue control 1 register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl2** (const ENET_Type *const pEnet)

Get Receive queue control 2 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl2** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Receive queue control 2 register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxQueueCtrl4** (const ENET_Type *const pEnet)

Get Receive queue control 4 register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueCtrl4** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Receive queue control 4 register value.

LOCAL_INLINE **ENET_MTL_QueueEnableType** **ENET_HWA_MTL_GetRxQueueEnStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get Receive queue enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueueEnStatus** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_QueueEnableType** eEnable)

Set Receive queue enable flag.

LOCAL_INLINE **ENET_MAC_PTPoEQueueCtrlType** **ENET_HWA_MAC_GetTaggedPTPoEQueueCtrl** (const ENET_Type *const pEnet)

Get tagged PTP over ethernet packets queuing control.

LOCAL_INLINE void **ENET_HWA_MAC_SetTaggedPTPoEQueueCtrl** (ENET_Type *const pEnet, **ENET_MAC_PTPoEQueueCtrlType** eCtrl)

Set tagged PTP over ethernet packets queuing control.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTaggedAVCtrlPacketQueueEnFlag** (const ENET_Type *const pEnet)

Get tagged AV control packets queuing enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTaggedAVCtrlPacketQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set tagged AV control packets queuing enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMulticastAndBroadcastQueueEnFlag** (const ENET_Type *const pEnet)

Get multicast and broadcast queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAndBroadcastQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set multicast and broadcast queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetMulticastAndBroadcastQueue** (const ENET_Type *const pEnet)

Get multicast and broadcast queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAndBroadcastQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set multicast and broadcast queue id.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetUntaggedPacketQueue** (const ENET_Type *const pEnet)

Get untagged packet queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetUntaggedPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set untagged packet queue id.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetPTPoEQueue** (const ENET_Type *const pEnet)

Get PTP over ethernet packet queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetPTPoEQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set PTP over ethernet packet queue id.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetAVCtrlPacketQueue** (const ENET_Type *const pEnet)

Get AV untagged control packets queue id.

LOCAL_INLINE void **ENET_HWA_MAC_SetAVCtrlPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set AV untagged control packets queue id.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetRxQueueSelectedPriorities** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue priorities.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxQueueSelectedPriorities** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint8_t u8SelPrio)

Set receive queue priorities.

LOCAL_INLINE bool **ENET_HWA_MAC_GetVLANTagFilterFailPacketQueueEnFlag** (const ENET_Type *const pEnet)

Get VLAN tag filter fail packets queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterFailPacketQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set VLAN tag filter fail packets queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetVLANTagFilterFailPacketQueue** (const ENET_Type *const pEnet)

Get VLAN tag filter fail packets queue.

LOCAL_INLINE void **ENET_HWA_MAC_SetVLANTagFilterFailPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set VLAN tag filter fail packets queue.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMulticastAddrFilterFailPacketQueueEnFlag** (const ENET_Type *const pEnet)

Get multicast address filter fail packets queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAddrFilterFailPacketQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set multicast address filter fail packets queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetMulticastAddrFilterFailPacketQueue** (const ENET_Type *const pEnet)

Get multicast address filter fail packets queue.

LOCAL_INLINE void **ENET_HWA_MAC_SetMulticastAddrFilterFailPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set multicast address filter fail packets queue.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUnicastAddrFilterFailPacketQueueEnFlag** (const ENET_Type *const pEnet)

Get unicast address filter fail packets queue enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastAddrFilterFailPacketQueueEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set unicast address filter fail packets queue enable flag.

LOCAL_INLINE **ENET_MTL_QueueType** **ENET_HWA_MAC_GetUnicastAddrFilterFailPacketQueue** (const ENET_Type *const pEnet)

Get unicast address filter fail packets queue.

LOCAL_INLINE void **ENET_HWA_MAC_SetUnicastAddrFilterFailPacketQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Set unicast address filter fail packets queue.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetInterruptStatus** (const ENET_Type *const pEnet)

Get the interrupt status of ENET MAC.

LOCAL_INLINE void **ENET_HWA_MAC_ClearInterruptStatus** (ENET_Type *const pEnet, uint32_t u32Interrupt)

Clear the interrupt flags of MAC.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMDIOInterruptStatus** (const ENET_Type *const pEnet)

Get the MDIO interrupt status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearMDIOInterruptStatus** (ENET_Type *const pEnet)

Clear the MDIO interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxInterruptStatus** (const ENET_Type *const pEnet)

Get the receive interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxInterruptStatus** (const ENET_Type *const pEnet)

Get the transmit interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampInterruptStatus** (const ENET_Type *const pEnet)

Get the timestamp interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMMCTxInterruptStatus** (const ENET_Type *const pEnet)

Get the MMC transmit interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMMCRxInterruptStatus** (const ENET_Type *const pEnet)

Get the MMC receive interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMMCInterruptStatus** (const ENET_Type *const pEnet)

Get the MMC interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPhyInterruptStatus** (const ENET_Type *const pEnet)

Get the PHY interrupt status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearPhyInterruptStatus** (ENET_Type *const pEnet)

Clear the PHY interrupt status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetInterruptEnable** (const ENET_Type *const pEnet)

Get the MAC interrupts enable register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetInterruptEnable** (ENET_Type *const pEnet, uint32_t u32IntEnable)

Set the MAC interrupts enable register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMDIOInterruptEnFlag** (const ENET_Type *const pEnet)

Get MDIO interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMDIOInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MDIO interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxStatusInterruptEnFlag** (const ENET_Type *const pEnet)

Get receive status interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetRxStatusInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set receive status interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxStatusInterruptEnFlag** (const ENET_Type *const pEnet)

Get transmit status interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxStatusInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set transmit status interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampInterruptEnFlag** (const ENET_Type *const pEnet)

Get timestamp interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set timestamp interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPhyInterruptEnFlag** (const ENET_Type *const pEnet)

Get phy interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set phy interrupt enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxTxStatus** (const ENET_Type *const pEnet)

Get the MAC receive and transmit status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearRxTxStatus** (ENET_Type *const pEnet, uint32_t u32Status)

Clear the MAC receive and transmit status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxWDGTimeoutStatus** (const ENET_Type *const pEnet)

Get the receive watchdog timeout status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearRxWDGTimeoutStatus** (ENET_Type *const pEnet)

Clear the receive watchdog timeout status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxExcessiveCollisionStatus** (const ENET_Type *const pEnet)

Get the excessive collision status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxExcessiveCollisionStatus** (ENET_Type *const pEnet)

Clear the excessive collision status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxLateCollisionStatus** (const ENET_Type *const pEnet)

Get the late collision status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxLateCollisionStatus** (ENET_Type *const pEnet)

Clear the late collision status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxExcessiveDeferralStatus** (const ENET_Type *const pEnet)

Get the excessive deferral status.

LOCAL_INLINE void **ENET_HWA_MAC_ClearExcessiveDeferralStatus** (ENET_Type *const pEnet)

Clear the excessive deferral status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxLossOfCarrierStatus** (const ENET_Type *const pEnet)

Indicates the status of carrier signal.

LOCAL_INLINE void **ENET_HWA_MAC_ClearLossOfCarrierStatus** (ENET_Type *const pEnet)

Clear the loss of carrier status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxNoCarrierStatus** (const ENET_Type *const pEnet)

Indicates whether the carrier signal is present from the PHY at the end of preamble transmission.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxNoCarrierStatus** (ENET_Type *const pEnet)

Clear the no carrier status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxJabberTimeoutStatus** (const ENET_Type *const pEnet)

Indicates whether a transmit jabber timer timeout occurred.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxJabberTimeoutStatus** (ENET_Type *const pEnet)

Clear the transmit jabber timer timeout status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMACVersion** (const ENET_Type *const pEnet)

Get MAC version.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetDebugStatus** (const ENET_Type *const pEnet)

Get MAC debug status.

LOCAL_INLINE **ENET_MAC_TxPacketControllerStatusType**

ENET_HWA_MAC_GetTxPacketControllerStatus (const ENET_Type *const pEnet)

Get MAC transmit packet controller status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxPacketEngineStatus** (const ENET_Type *const pEnet)

Get MAC GMII or MII transmit protocol engine status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxPacketControllerFIFOStatus** (const ENET_Type *const pEnet)

Get MAC receive packet controller FIFO status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetRxProtocalEngineStatus** (const ENET_Type *const pEnet)

Get MAC GMII or MII receive protocol engine status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature0** (const ENET_Type *const pEnet)

Get MAC hardware features 0.

LOCAL_INLINE **ENET_MAC_MiiModeType** **ENET_HWA_MAC_GetPhyMiiMode** (const ENET_Type *const pEnet)

Get phy interface mii mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature1** (const ENET_Type *const pEnet)

Get MAC hardware features 1.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature2** (const ENET_Type *const pEnet)

Get MAC hardware features 2.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetHwFeature3** (const ENET_Type *const pEnet)

Get MAC hardware features 3.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMDIOAddress** (const ENET_Type *const pEnet)

Get MAC MDIO address register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMDIOAddress** (ENET_Type *const pEnet, uint32_t u32MDIOAddr)

Set MAC MDIO address register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPreambleSuppressionEnFlag** (const ENET_Type *const pEnet)

Get preamble suppression enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPreambleSuppressionEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MDIO preamble suppression enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetBackToBackTransactionEnFlag** (const ENET_Type *const pEnet)

Get MDIO back to back transactions enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetBackToBackTransactionEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MDIO back to back transactions enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetPhysicalLayerAddr** (const ENET_Type *const pEnet)

Get MDIO physical layer address.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhysicalLayerAddr** (ENET_Type *const pEnet, uint8_t u8Addr)

Set MDIO physical layer address.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetPhyRegisterOrDeviceAddr** (const ENET_Type *const pEnet)

Get MDIO phy register(clause22) or MMD device(clause 45)

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyRegisterOrDeviceAddr** (ENET_Type *const pEnet, uint8_t u8Addr)

Set MDIO phy register(clause22) or MMD device(clause 45)

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetTrailingClockCnt** (const ENET_Type *const pEnet)

Get the number of trailing clock cycles.

LOCAL_INLINE void **ENET_HWA_MAC_SetTrailingClockCnt** (ENET_Type *const pEnet, uint8_t u8Cnt)

Set MDIO trailing clock cycles.

LOCAL_INLINE **ENET_MAC_CSRClockRangeType** **ENET_HWA_MAC_GetCSRClockRange** (const ENET_Type *const pEnet)

Get the MDC clock range.

LOCAL_INLINE void **ENET_HWA_MAC_SetCSRClockRange** (ENET_Type *const pEnet,
ENET_MAC_CSRClockRangeType eRange)

Set the MDC clock range.

LOCAL_INLINE bool **ENET_HWA_MAC_GetSkipAddrPacketEnFlag** (const ENET_Type *const pEnet)

Get skip address packet enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetSkipAddrPacketEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set skip address packet enable flag.

LOCAL_INLINE **ENET_MAC_PhyOperationCmdType** **ENET_HWA_MAC_GetPhyOperationCommand** (const
ENET_Type *const pEnet)

Get phy operation command.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyOperationCommand** (ENET_Type *const pEnet,
ENET_MAC_PhyOperationCmdType eCmd)

Set phy operation command.

LOCAL_INLINE **ENET_MAC_PhyClauseVersionType** **ENET_HWA_MAC_GetPhyClauseVersion** (const
ENET_Type *const pEnet)

Get MDIO clause version.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyClauseVersion** (ENET_Type *const pEnet,
ENET_MAC_PhyClauseVersionType eVersion)

Set MDIO clause version.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetGMIIStatus** (const ENET_Type *const pEnet)

Get MDIO busy status.

LOCAL_INLINE void **ENET_HWA_MAC_SetGMIIBusy** (ENET_Type *const pEnet)

Set MDIO busy to initiate a read or write access.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMDIOData** (const ENET_Type *const pEnet)

Get MDIO data register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMDIOData** (ENET_Type *const pEnet, uint32_t u32Data)

Set MDIO data register value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetPhyRegisterAddr** (const ENET_Type *const pEnet)

Get phy register address.

LOCAL_INLINE void **ENET_HWA_MAC_SetPhyRegisterAddr** (ENET_Type *const pEnet, uint16_t u16Addr)

Set phy register address.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetGMIIData** (const ENET_Type *const pEnet)

Get MDIO data.

LOCAL_INLINE void **ENET_HWA_MAC_SetGMIIData** (ENET_Type *const pEnet, uint16_t u16Data)

Set MDIO data.

LOCAL_INLINE void **ENET_HWA_MAC_SetFramePreemptionEnFlag** (ENET_Type *const pEnet, uint8
bEnable)

Enable TX frame preemption.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPresentationTime** (const ENET_Type *const pEnet)

Get presentation time in ns.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPresentationTimeUdata** (const ENET_Type *const pEnet)

Get presentation time update.

LOCAL_INLINE void **ENET_HWA_MAC_SetPresentationTimeUdata** (ENET_Type *const pEnet, uint32_t
u32NanoSeconds)

Set presentation time update.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr0High** (const ENET_Type *const pEnet)

Get MAC address0 high register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetAddr0High** (ENET_Type *const pEnet, uint32_t u32AddrHigh)
Set MAC address0 high register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr0Low** (const ENET_Type *const pEnet)
Get MAC address0 low register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr0Low** (ENET_Type *const pEnet, uint32_t u32AddrLow)
Set MAC address0 low register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMacAddr0EnFlag** (const ENET_Type *const pEnet)
Get MAC address0 enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr0DMAChannelSelect** (const ENET_Type *const pEnet)
Get MAC address0 DMA channel select.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr0DMAChannelSelect** (ENET_Type *const pEnet, uint8_t u8DMAChannelSel)
Set MAC address0 DMA channel select.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr1High** (const ENET_Type *const pEnet)
Get MAC address1 high register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetAddr1High** (ENET_Type *const pEnet, uint32_t u32AddrHigh)
Set MAC address1 high register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr1Low** (const ENET_Type *const pEnet)
Get MAC address1 low register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1Low** (ENET_Type *const pEnet, uint32_t u32AddrLow)
Set MAC address1 low register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMacAddr1EnFlag** (const ENET_Type *const pEnet)

Get MAC address1 enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1EnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MAC address1 enable flag.

LOCAL_INLINE **ENET_MAC_AddrType** **ENET_HWA_MAC_GetMacAddr1Type** (const ENET_Type *const pEnet)

Get MAC address1 compare mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1Type** (ENET_Type *const pEnet, **ENET_MAC_AddrType** eAddrType)

Set MAC address1 compare mode.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr1MaskByteCtrl** (const ENET_Type *const pEnet)

Get MAC address1 mask byte control.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1MaskByteCtrl** (ENET_Type *const pEnet, uint8_t u8MaskCtrl)

Set MAC address1 mask byte control.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr1DMAChannelSelect** (const ENET_Type *const pEnet)

Get MAC address1 DMA channel select.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr1DMAChannelSelect** (ENET_Type *const pEnet, uint8_t u8DMAChannelSel)

Set MAC address1 DMA channel select.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr2High** (const ENET_Type *const pEnet)

Get MAC address2 high register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetAddr2High** (ENET_Type *const pEnet, uint32_t u32AddrHigh)

Set MAC address2 high register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMacAddr2Low** (const ENET_Type *const pEnet)

Get MAC address2 low register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2Low** (ENET_Type *const pEnet, uint32_t u32AddrLow)

Set MAC address2 low register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMacAddr2EnFlag** (const ENET_Type *const pEnet)

Get MAC address2 enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2EnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MAC address2 enable flag.

LOCAL_INLINE **ENET_MAC_AddrType** **ENET_HWA_MAC_GetMacAddr2Type** (const ENET_Type *const pEnet)

Get MAC address2 compare mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2Type** (ENET_Type *const pEnet, **ENET_MAC_AddrType** eAddrType)

Set MAC address2 compare mode.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr2MaskByteCtrl** (const ENET_Type *const pEnet)

Get MAC address2 mask byte control.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2MaskByteCtrl** (ENET_Type *const pEnet, uint8_t u8MaskCtrl)

Set MAC address2 mask byte control.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetMacAddr2DMAChannelSelect** (const ENET_Type *const pEnet)

Get MAC address2 DMA channel select.

LOCAL_INLINE void **ENET_HWA_MAC_SetMacAddr2DMAChannelSelect** (ENET_Type *const pEnet, uint8_t u8DMAChannelSel)

Set MAC address2 DMA channel select.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcControl** (const ENET_Type *const pEnet)

Get MMC control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcControl** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set MMC control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUpdateMmcCntForDroppedBroadcastPktEnFlag** (const ENET_Type *const pEnet)

Get update MMC counters for dropped broadcast packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUpdateMmcCntForDroppedBroadcastPktEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set update MMC counters for dropped broadcast packets enable flag.

LOCAL_INLINE **ENET_MAC_MmcCntPresetLevelType** **ENET_HWA_MAC_GetMmcCounterPresetLevel** (const ENET_Type *const pEnet)

Get MMC counters full-half preset level.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterPresetLevel** (ENET_Type *const pEnet, **ENET_MAC_MmcCntPresetLevelType** eLevel)

Set MMC counters full-half preset level.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterPresetStatus** (const ENET_Type *const pEnet)

Get MMC counters preset status.

LOCAL_INLINE void **ENET_HWA_MAC_PresetMmcCounters** (ENET_Type *const pEnet)

Set MMC counters preset.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterFreezeStatus** (const ENET_Type *const pEnet)

Get MMC counters freeze status.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterFreezeStatus** (ENET_Type *const pEnet, bool bFreeze)

Set MMC counters freeze.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterResetOnReadEnFlag** (const ENET_Type *const pEnet)

Get MMC counters reset on read enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterResetOnReadEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MMC counters reset on read enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterStopRolloverEnFlag** (const ENET_Type *const pEnet)

Get MMC counters stop rollover enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcCounterStopRolloverEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set MMC counters stop rollover enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMmcCounterResetStatus** (const ENET_Type *const pEnet)

Get MMC counters reset status.

LOCAL_INLINE void **ENET_HWA_MAC_ResetMmcCounters** (ENET_Type *const pEnet)

Reset all MMC counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcRxInterruptStatus** (const ENET_Type *const pEnet)

Get MMC interrupts of receive statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcTxInterruptStatus** (const ENET_Type *const pEnet)

Get MMC interrupts of transmit statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcRxInterruptMask** (const ENET_Type *const pEnet)

Get MMC interrupts mask of receive statistics counters.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcRxInterruptMask** (ENET_Type *const pEnet, uint32_t u32Mask)

Set MMC interrupts mask of receive statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetMmcTxInterruptMask** (const ENET_Type *const pEnet)

Get MMC interrupts mask of transmit statistics counters.

LOCAL_INLINE void **ENET_HWA_MAC_SetMmcTxInterruptMask** (ENET_Type *const pEnet, uint32_t u32Mask)

Set MMC interrupts mask of transmit statistics counters.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxOctetCntGoodBad** (const ENET_Type *const pEnet)

Get transmitted bytes of good and bad packets.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxBroadcastPacketGood** (const ENET_Type *const pEnet)

Get the number of good broadcast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxMulticastPacketGood** (const ENET_Type *const pEnet)

Get the number of good multicast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx64OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length 64 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx65To127OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 65 and 127 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx128To255OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 128 and 255 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx256To511OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 256 and 511 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx512To1023OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length between 512 and 1023 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTx1024ToMaxOctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets transmitted with length 1024 to maxsize bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxUnicastPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad unicast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxMulticastPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad multicast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxBroadcastPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad broadcast packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxUnderflowErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit underflow error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxSingleCollisionGoodPacketCnt** (const ENET_Type *const pEnet)

Get the number of successfully transmitted packets after a single collision in half-duplex mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxMultipleCollisionGoodPacketCnt** (const ENET_Type *const pEnet)

Get the number of successfully transmitted packets after a multiple collision in half-duplex mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxDeferredPacketCnt** (const ENET_Type *const pEnet)

Get the number of successfully transmitted packets after a deferral in half-duplex mode.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxLateCollisionPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit late collision error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxExcessiveCollisionPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit excessive collision errors.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxCarrierErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit carrier sense error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxOctetCntGood** (const ENET_Type *const pEnet)

Get the number of bytes of good packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxPacketCntGood** (const ENET_Type *const pEnet)

Get the number of good packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxExcessiveDeferralErrorCnt** (const ENET_Type *const pEnet)

Get the number of packets with transmit excessive deferral error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxPausePacketCnt** (const ENET_Type *const pEnet)

Get the number of good pause packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxVLANPacketGood** (const ENET_Type *const pEnet)

Get the number of good VLAN packets transmitted.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxOverSizePacketGood** (const ENET_Type *const pEnet)

Get the number of transmitted packets without errors and with length greater than the maxsize.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxPacketCntGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOctetCntGoodBad** (const ENET_Type *const pEnet)

Get the number of bytes received in good and bad packets.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOctetCntGood** (const ENET_Type *const pEnet)

Get the number of bytes received in good packets.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxBroadcastPacketGood** (const ENET_Type *const pEnet)

Get the number of good broadcast packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxMulticastPacketGood** (const ENET_Type *const pEnet)

Get the number of good multicast packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxCRCErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with CRC error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxAlignmentErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with alignment error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxRunErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with runt error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxJabberErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with jabber error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxUndersizePacketGood** (const ENET_Type *const pEnet)

Get the number of packets received with length less than 64 bytes, without any errors.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOverSizeGood** (const ENET_Type *const pEnet)

Get the number of packets received without errors and with length greater than maxsize.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx64OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 64 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx65To127OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 65 to 127 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx128To255OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 128 to 255 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx256To511OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 256 to 511 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx512To1023OctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 512 to 1023 bytes.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRx1024ToMaxOctetPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of packets received with length 1024 to maxsize.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetUnicastPacketGood** (const ENET_Type *const pEnet)

Get the number of good unicast packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxLengthErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with length error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxOutOfRangeTypePacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with length greater than 1500 but less than 1536.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxPausePacketCnt** (const ENET_Type *const pEnet)

Get the number of good and valid pause packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxFIFOOverflowPacketCnt** (const ENET_Type *const pEnet)

Get the number of missed received packets because of FIFO overflow.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxVLANPacketGoodBad** (const ENET_Type *const pEnet)

Get the number of good and bad VLAN packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxWatchdogErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with error because of watchdog timeout error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxReceiveErrorPacketCnt** (const ENET_Type *const pEnet)

Get the number of packets received with receive error or Packet Extension error.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetRxControlPacketGood** (const ENET_Type *const pEnet)

Get the number of good control packets received.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3Layer4Ctrl** (const ENET_Type *const pEnet, uint8_t u8Index)

Get the layer 3 and layer 4 control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Layer4Ctrl** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Ctrl)

Set the layer 3 and layer 4 control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer3Layer4DMASelectEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 and layer 4 control DMA channel select enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Layer4DMASelectEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 3 and layer 4 control DMA channel select enable flag.

```
LOCAL_INLINE ENET_DMA_ChannelType ENET_HWA_MAC_GetLayer3Layer4DMAChannel (const  
ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 and layer 4 control DMA channel number.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3Layer4DMAChannel (ENET_Type *const pEnet, uint8_t  
u8Index, ENET_DMA_ChannelType eChannel)
```

Set layer 3 and layer 4 control DMA channel number.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetLayer4DestPortInvMatchEnFlag (const ENET_Type *const  
pEnet, uint8_t u8Index)
```

Get layer 4 destination port inverse match enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer4DestPortInvMatchEnFlag (ENET_Type *const pEnet,  
uint8_t u8Index, bool bEnable)
```

Set layer 4 destination port inverse match enable flag.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetLayer4DestPortMatchEnFlag (const ENET_Type *const pEnet,  
uint8_t u8Index)
```

Get layer 4 destination port match enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer4DestPortMatchEnFlag (ENET_Type *const pEnet, uint8_t  
u8Index, bool bEnable)
```

Set layer 4 destination port match enable flag.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetLayer4SrcPortInvMatchEnFlag (const ENET_Type *const pEnet,  
uint8_t u8Index)
```

Get layer 4 source port inverse match enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer4SrcPortInvMatchEnFlag (ENET_Type *const pEnet,  
uint8_t u8Index, bool bEnable)
```

Set layer 4 source port inverse match enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetLayer4SrcPortMatchEnFlag** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 source port match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4SrcPortMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 4 source port match enable flag.

LOCAL_INLINE **ENET_MAC_Layer4ProtocolType** **ENET_HWA_MAC_GetLayer4ProtocolType** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 protocol type (UDP or TCP)

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4ProtocolType** (ENET_Type *const pEnet, uint8_t u8Index, **ENET_MAC_Layer4ProtocolType** eProtocol)

Set layer 4 protocol type (UDP or TCP)

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetLayer3IPv4DestAddrBitsMatch** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer3 IPv4 destination address bits match.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3IPv4DestAddrBitsMatch** (ENET_Type *const pEnet, uint8_t u8Index, uint8_t u8MaskBits)

Set layer3 IPv4 destination address bits match.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetLayer3IPv4SrcAddrBitsMatch** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer3 IPv4 source address bits match.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3IPv4SrcAddrBitsMatch** (ENET_Type *const pEnet, uint8_t u8Index, uint8_t u8MaskBits)

Set layer3 IPv4 source address bits match.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetLayer3IPv6AddrBitsMatch** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer3 IPv6 address bits match.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3IPv6AddrBitsMatch (ENET_Type *const pEnet, uint8_t u8Index, uint8_t u8MaskBits)
```

Set layer3 IPv6 address bits match.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3DestAddrInvMatchEnFlag (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 destination address inverse match enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3DestAddrInvMatchEnFlag (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)
```

Set layer 3 destination address inverse match enable flag.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3DestAddrMatchEnFlag (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 destination address match enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3DestAddrMatchEnFlag (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)
```

Set layer 3 destination address match enable flag.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3SrcAddrInvMatchEnFlag (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 source address inverse match enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetLayer3SrcAddrInvMatchEnFlag (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)
```

Set layer 3 source address inverse match enable flag.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetLayer3SrcAddrMatchEnFlag (const ENET_Type *const pEnet, uint8_t u8Index)
```

Get layer 3 source address match enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3SrcAddrMatchEnFlag** (ENET_Type *const pEnet, uint8_t u8Index, bool bEnable)

Set layer 3 source address match enable flag.

LOCAL_INLINE **ENET_MAC_Layer3ProtocolType** **ENET_HWA_MAC_GetLayer3ProtocolType** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 protocol type (IPv4 or IPv6)

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3ProtocolType** (ENET_Type *const pEnet, uint8_t u8Index, **ENET_MAC_Layer3ProtocolType** eProtocol)

Set layer 3 protocol type (IPv4 or IPv6)

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer4Addr** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 address.

LOCAL_INLINE void **ENET_MAC_SetLayer4Addr** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)

Set layer 4 address.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetLayer4DestPort** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 destination port number.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4DestPort** (ENET_Type *const pEnet, uint8_t u8Index, uint16_t u16DestPort)

Set layer 4 destination port number.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetLayer4SrcPort** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 4 source port number.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer4SrcPort** (ENET_Type *const pEnet, uint8_t u8Index, uint16_t u16SrcPort)

Set layer 4 source port number.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3Addr0** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 address0.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Addr0** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)

Set layer 3 address0.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3Addr1** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 address1.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Addr1** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)

Set layer 3 address1.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3Addr2** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 address2.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Addr2** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)

Set layer 3 address2.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3Addr3** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 address3.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3Addr3** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32Addr)

Set layer 3 address3.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3SrcIPv4Addr** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 source IPv4 address.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3SrcIPv4Addr** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32SrcAddr)

Set layer 3 source IPv4 address.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetLayer3DestIPv4Addr** (const ENET_Type *const pEnet, uint8_t u8Index)

Get layer 3 destination IPv4 address.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3DestIPv4Addr** (ENET_Type *const pEnet, uint8_t u8Index, uint32_t u32DestAddr)

Set layer 3 destination IPv4 address.

LOCAL_INLINE void **ENET_HWA_MAC_GetLayer3IPv6Addr** (const ENET_Type *const pEnet, uint8_t u8Index, uint8_t aIPv6Addr[16u])

Get layer 3 IPv6 address.

LOCAL_INLINE void **ENET_HWA_MAC_SetLayer3IPv6Addr** (ENET_Type *const pEnet, uint8_t u8Index, const uint8_t aIPv6Addr[16])

Set layer 3 IPv6 address.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampControl** (const ENET_Type *const pEnet)

Get time stamp control register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampControl** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set time stamp control register value.

LOCAL_INLINE bool **ENET_HWA_MAC_GetAV802_1ASModeEnFlag** (const ENET_Type *const pEnet)

Get time stamp AV802_1AS mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetAV802_1ASModeEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set time stamp AV802_1AS mode enable flag.

LOCAL_INLINE **ENET_MAC_TxTimestampStatusModeType**

ENET_HWA_MAC_GetTxTimestampStatusMode (const ENET_Type *const pEnet)

Get transmit timestamp status mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetTxTimestampStatusMode** (ENET_Type *const pEnet,
ENET_MAC_TxTimestampStatusModeType eMode)

Set transmit timestamp status mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetUseMACAddrForPTPPacketFilterEnFlag** (const ENET_Type
*const pEnet)

Get MAC address for PTP packet filter enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetUseMACAddrForPTPPacketFilterEnFlag** (ENET_Type *const
pEnet, bool bEnable)

Set MAC address for PTP packet filter enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetSnapshotPTPPacketType** (const ENET_Type *const pEnet)

Get PTP packets type for taking snapshots.

LOCAL_INLINE void **ENET_HWA_MAC_SetSnapshotPTPPacketType** (ENET_Type *const pEnet, uint8_t
u8Type)

Set PTP packets type for taking snapshots.

LOCAL_INLINE bool **ENET_HWA_MAC_GetMasterMsgSanpshotEnFlag** (const ENET_Type *const pEnet)

Get snapshot for messages relevant to master enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetMasterMsgSanpshotEnFlag** (ENET_Type *const pEnet, bool
bEnable)

Set snapshot for messages relevant to master enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetEventMsgSnapshotEnFlag** (const ENET_Type *const pEnet)

Get snapshot for event messages enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetEventMsgSanpshotEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set snapshot for event messages enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetIPv4UDPPTPPacketProcessEnFlag** (const ENET_Type *const pEnet)

Get processing of PTP packets over IPv4-UDP enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetIPv4UDPPTPPacketProcessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set processing of PTP packets over IPv4-UDP enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetIPv6UDPPTPPacketProcessEnFlag** (const ENET_Type *const pEnet)

Get processing of PTP packets over IPv6-UDP enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetIPv6UDPPTPPacketProcessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set processing of PTP packets over IPv6-UDP enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetEthernetPTPPacketProcessEnFlag** (const ENET_Type *const pEnet)

Get processing of PTP packets over ethernet enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetEthernetPTPPacketProcessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set processing of PTP packets over ethernet enable flag.

LOCAL_INLINE **ENET_MAC_PTPVersionType** **ENET_HWA_MAC_GetPTPPacketVersion** (const ENET_Type *const pEnet)

Get PTP packets processing format version.

LOCAL_INLINE void **ENET_HWA_MAC_SetPTPPacketVersion** (ENET_Type *const pEnet,
ENET_MAC_PTPVersionType eVersion)

Set PTP packets processing format version.

LOCAL_INLINE **ENET_MAC_TimestampRolloverType** **ENET_HWA_MAC_GetTimestampRolloverCtrl** (const
ENET_Type *const pEnet)

Get PTP timestamp rollover mode.

LOCAL_INLINE void **ENET_HWA_MAC_SetlimestamprolloverCtrl** (ENET_Type *const pEnet,
ENET_MAC_TimestampRolloverType eCtrl)

Set PTP timestamp rollover mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampForAllPacketEnFlag** (const ENET_Type *const pEnet)

Get timestamp for all packets enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampForAllPacketEnFlag** (ENET_Type *const pEnet, bool
bEnable)

Set timestamp for all packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MAC_GetPresentationTimeGenerationEnFlag** (const ENET_Type *const
pEnet)

Get presentation time generation enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetPresentationTimeGenerationEnFlag** (ENET_Type *const pEnet,
bool bEnable)

Set presentation time generation enable flag.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetAddendRegisterUpdateStatus** (const
ENET_Type *const pEnet)

Get PTP addend register update status.

LOCAL_INLINE void **ENET_HWA_MAC_UpdateAddendRegister** (ENET_Type *const pEnet)

Initiate the addend register update operation.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetTimestampUpdateStatus** (const ENET_Type *const pEnet)

Get timestamp update status.

LOCAL_INLINE void **ENET_HWA_MAC_UpdateTimestamp** (ENET_Type *const pEnet)

Initiate the timestamp update operation.

LOCAL_INLINE **ENET_MAC_StatusType** **ENET_HWA_MAC_GetTimestampInitStatus** (const ENET_Type *const pEnet)

Get timestamp initialize status.

LOCAL_INLINE void **ENET_HWA_MAC_InitTimestamp** (ENET_Type *const pEnet)

Initialize PTP timestamp.

LOCAL_INLINE **ENET_MAC_TimestampUpdateMethodType**

ENET_HWA_MAC_GetTimestampUpdateMethod (const ENET_Type *const pEnet)

Get timestamp update method (fine or coarse)

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampUpdateMethod** (ENET_Type *const pEnet, **ENET_MAC_TimestampUpdateMethodType** eMethod)

Set timestamp update method (fine or coarse)

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampEnFlag** (const ENET_Type *const pEnet)

Get timestamp enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set timestamp processing enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSubSecondIncrementReg** (const ENET_Type *const pEnet)

Get PTP sub-second increment register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSubSecondIncrementReg** (ENET_Type *const pEnet, uint32_t u32Increment)

Set PTP sub-second increment register value.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetSubSecondIncrementValue** (const ENET_Type *const pEnet)
Get PTP sub-second increment value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSubSecondIncrementValue** (ENET_Type *const pEnet, uint8_t u8Increment)
Set PTP sub-second increment value.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetSubNanoSecondIncrementValue** (const ENET_Type *const pEnet)
Get PTP sub-nanosecond increment value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSubNanoSecondIncrementValue** (ENET_Type *const pEnet, uint8_t u8Increment)
Set PTP sub-nanosecond increment value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeSeconds** (const ENET_Type *const pEnet)
Get PTP system time seconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeNanoSeconds** (const ENET_Type *const pEnet)
Get PTP system time nanoseconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeSecondsUpdateValue** (const ENET_Type *const pEnet)
Get PTP system time seconds update register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeSecondsUpdateValue** (ENET_Type *const pEnet, uint32_t u32Seconds)
Set PTP system time seconds update register value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeNanoSecondsUpdateReg** (const ENET_Type *const pEnet)
Get PTP system time nanoseconds update register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeNanoSecondsUpdateReg** (ENET_Type *const pEnet, uint32_t u32Reg)

Set PTP system time nanoseconds update register value.

LOCAL_INLINE **ENET_MAC_SystemTimeOperationType** **ENET_HWA_MAC_GetTimeUpdateOperation** (const ENET_Type *const pEnet)

Get PTP system time update operation, add or subtract time.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimeUpdateOperation** (ENET_Type *const pEnet, **ENET_MAC_SystemTimeOperationType** eOperation)

Set PTP system time update operation, add or subtract time.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetSystemTimeNanoSecondsUpdateValue** (const ENET_Type *const pEnet)

Get PTP system time nanoseconds update value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeNanoSecondsUpdateValue** (ENET_Type *const pEnet, uint32_t u32NanoSeconds)

Set PTP system time nanoseconds update value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampAddend** (const ENET_Type *const pEnet)

Get PTP timestamp addend register value.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampAddend** (ENET_Type *const pEnet, uint32_t u32Time)

Set PTP timestamp addend register value.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetSystemTimeHigherWordSeconds** (const ENET_Type *const pEnet)

Get PTP system time seconds bits[47:32] value.

LOCAL_INLINE void **ENET_HWA_MAC_SetSystemTimeHigherWordSeconds** (ENET_Type *const pEnet, uint16_t u16Seconds)

Set PTP system time seconds bits[47:32] value.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampStatus** (const ENET_Type *const pEnet)

Get the timestamp status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTxTimestampInterruptStatus** (const ENET_Type *const pEnet)

Indicates whether the transmit timestamp interrupt status is detected.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTxTimestampInterruptStatus** (ENET_Type *const pEnet)

Clear the transmit timestamp interrupt status.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampTargetTimeError** (const ENET_Type *const pEnet,
ENET_MAC_PPSSChannelType ePPSChann)

Indicates whether the timestamp target time error status of the PPS channel is detected.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTimestampTargetTimeError** (ENET_Type *const pEnet,
ENET_MAC_PPSSChannelType ePPSChann)

Clear the timestamp target time error status of the PPS channel.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampTargetTimeReachedStatus** (const ENET_Type
*const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Indicates whether the timestamp target time reached for target time of the PPS channel.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTimestampTargetTimeReachedStatus** (ENET_Type *const
pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Clear the status of the timestamp target time reached for target time of the PPS channel.

LOCAL_INLINE bool **ENET_HWA_MAC_GetTimestampSecondsOverflowStatus** (const ENET_Type *const
pEnet)

Indicates whether the timestamp seconds overflow status is detected.

LOCAL_INLINE void **ENET_HWA_MAC_ClearTimestampSecondsOverflowStatus** (ENET_Type *const
pEnet)

Clear the timestamp seconds overflow status.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxTimestampStatusInNanoseconds** (const ENET_Type *const pEnet)

Contains the nanosecond part of timestamp captured for transmit packets when tx status is disabled.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTxTimestampStatusInSeconds** (const ENET_Type *const pEnet)

Contains the second part of timestamp captured for transmit packets when tx status is disabled.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampIngressAsymmetryCorrection** (const ENET_Type *const pEnet)

Get timestamp ingress asymmetry correction, used for PDelay_Resp PTP messages.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampIngressAsymmetryCorrection** (ENET_Type *const pEnet, int32_t s32Correction)

Set timestamp ingress asymmetry correction, used for PDelay_Resp PTP messages.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampEgressAsymmetryCorrection** (const ENET_Type *const pEnet)

Get timestamp egress asymmetry correction, used for PDelay_Req PTP messages.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEgressAsymmetryCorrection** (ENET_Type *const pEnet, int32_t s32Correction)

Set timestamp egress asymmetry correction, used for PDelay_Req PTP messages.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampIngressCorrectionInNanoseconds** (const ENET_Type *const pEnet)

Get timestamp ingress correction in nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampIngressCorrectionInNanoseconds** (ENET_Type *const pEnet, int32_t s32NanoSec)

Set timestamp ingress correction in nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE int32_t **ENET_HWA_MAC_GetTimestampEgressCorrectionInNanoseconds** (const ENET_Type *const pEnet)

Get timestamp egress correction in nanoseconds, used for timestamp in egress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEgressCorrectionInNanoseconds** (ENET_Type *const pEnet, int32_t s32NanoSec)

Set timestamp egress correction in nanoseconds, used for timestamp in egress path.

LOCAL_INLINE int8_t **ENET_HWA_MAC_GetTimestampIngressCorrectionInSubNanoseconds** (const ENET_Type *const pEnet)

Get timestamp ingress correction in sub-nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampIngressCorrectionInSubNanoseconds** (ENET_Type *const pEnet, int8_t s8SubNanoSec)

Set timestamp ingress correction in sub-nanoseconds, used for timestamp in ingress path.

LOCAL_INLINE int8_t **ENET_HWA_MAC_GetTimestampEgressCorrectionInSubNanoseconds** (const ENET_Type *const pEnet)

Get timestamp egress correction in sub-nanoseconds, used for timestamp in egress path.

LOCAL_INLINE void **ENET_HWA_MAC_SetTimestampEgressCorrectionInSubNanoseconds** (ENET_Type *const pEnet, int8_t s8SubNanoSec)

Set timestamp egress correction in sub-nanoseconds, used for timestamp in egress path.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampIngressLatency** (const ENET_Type *const pEnet)

Get the ingress MAC latency.

LOCAL_INLINE uint16_t **ENET_HWA_MAC_GetTimestampIngressLatencyInNanoseconds** (const ENET_Type *const pEnet)

Get the ingress MAC latency in nanoseconds.

LOCAL_INLINE uint8_t **ENET_HWA_MAC_GetTimestampIngressLatencyInSubNanoseconds** (const ENET_Type *const pEnet)

Get the ingress MAC latency in sub-nanoseconds.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetTimestampEgressLatency** (const ENET_Type *const pEnet)

Get the egress MAC latency.

```
LOCAL_INLINE uint16_t ENET_HWA_MAC_GetTimestampEgressLatencyInNanoseconds (const  
ENET_Type *const pEnet)
```

Get the egress MAC latency in nanoseconds.

```
LOCAL_INLINE uint8_t ENET_HWA_MAC_GetTimestampEgressLatencyInSubNanoseconds (const  
ENET_Type *const pEnet)
```

Get the egress MAC latency in sub-nanoseconds.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSControl (const ENET_Type *const pEnet)
```

Get PPS control register value.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSControl (ENET_Type *const pEnet, uint32_t u32Ctrl)
```

Set PPS control register value.

```
LOCAL_INLINE bool ENET_HWA_MAC_GetPPSMCGRModeEnFlag (const ENET_Type *const pEnet,  
ENET_MAC_PPSTargetTimeModeType ePPSChann)
```

Get PPS MCGR Mode enable flag.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSMCGRModeEnFlag (ENET_Type *const pEnet,  
ENET_MAC_PPSTargetTimeModeType ePPSChann, bool bEnable)
```

Set PPS MCGR Mode enable flag.

```
LOCAL_INLINE ENET_MAC_PPSTargetTimeModeType  
ENET_HWA_MAC_GetChannelPPSTargetTimeModeSelect (const ENET_Type *const pEnet,  
ENET_MAC_PPSTargetTimeModeType ePPSChann)
```

Get PPS channel target time register operating mode.

```
LOCAL_INLINE void ENET_HWA_MAC_SetChannelPPSTargetTimeModeSelect (ENET_Type *const pEnet,  
ENET_MAC_PPSTargetTimeModeType ePPSChann, ENET_MAC_PPSTargetTimeModeType eTrgtMode)
```

Set PPS channel target time register operating mode.

LOCAL_INLINE bool **ENET_HWA_MAC_GetChannelFlexiblePPSOutputEnFlag** (const ENET_Type *const pEnet)

Get PPS flexible mode enable flag.

LOCAL_INLINE void **ENET_HWA_MAC_SetChannelFlexiblePPSOutputEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set PPS flexible mode enable flag.

LOCAL_INLINE **ENET_MAC_MCGRCmdType** **ENET_HWA_MAC_GetPPSMCGRCommand** (const ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Get PPS MCGR command.

LOCAL_INLINE void **ENET_HWA_MAC_SetPPSMCGRCommand** (ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann, **ENET_MAC_MCGRCmdType** eCmd)

Set PPS MCGR command.

LOCAL_INLINE **ENET_MAC_PPSCmdType** **ENET_HWA_MAC_GetFlexiblePPSCommand** (ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Get flexible PPS command.

LOCAL_INLINE void **ENET_HWA_MAC_SetFlexiblePPSCommand** (ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann, **ENET_MAC_PPSCmdType** eCmd)

Set flexible PPS command.

LOCAL_INLINE **ENET_MAC_PPSPFreqCtrlType** **ENET_HWA_MAC_GetFixedPPSOutputFrequency** (ENET_Type *const pEnet)

Get fixed PPS output frequency.

LOCAL_INLINE void **ENET_HWA_MAC_SetFixedPPSOutputFrequency** (ENET_Type *const pEnet, **ENET_MAC_PPSPFreqCtrlType** eFreq)

Set fixed PPS output frequency.

LOCAL_INLINE uint32_t **ENET_HWA_MAC_GetPPSTargetTimeInSeconds** (const ENET_Type *const pEnet, **ENET_MAC_PPSSChannelType** ePPSChann)

Get PPS target time seconds.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSTargetTimeInSeconds (ENET_Type *const pEnet,  
ENET_MAC_PPSChannelType ePPSChann, uint32_t u32Seconds)
```

Set PPS target time seconds.

```
LOCAL_INLINE ENET_MAC_StatusType ENET_HWA_MAC_GetPPSTargetTimeBusyStatus (const  
ENET_Type *const pEnet, ENET_MAC_PPSChannelType ePPSChann)
```

Get PPS target time busy status.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSTargetTimeInNanoseconds (const ENET_Type *const  
pEnet, ENET_MAC_PPSChannelType ePPSChann)
```

Get PPS target time nanoseconds.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSTargetTimeInNanoseconds (ENET_Type *const pEnet,  
ENET_MAC_PPSChannelType ePPSChann, uint32_t u32NanoSeconds)
```

Set PPS target time nanoseconds.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSInterval (const ENET_Type *const pEnet,  
ENET_MAC_PPSChannelType ePPSChann)
```

Get PPS pulse interval.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSInterval (ENET_Type *const pEnet,  
ENET_MAC_PPSChannelType ePPSChann, uint32_t u32Interval)
```

Set PPS pulse interval.

```
LOCAL_INLINE uint32_t ENET_HWA_MAC_GetPPSWidth (const ENET_Type *const pEnet,  
ENET_MAC_PPSChannelType ePPSChann)
```

Get PPS pulse width.

```
LOCAL_INLINE void ENET_HWA_MAC_SetPPSWidth (ENET_Type *const pEnet,  
ENET_MAC_PPSChannelType ePPSChann, uint32_t u32Width)
```

Set PPS pulse width.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetOperationMode** (const ENET_Type *const pEnet)

Get MTL operation mode register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetOperationMode** (ENET_Type *const pEnet, uint32_t u32OpMode)

Set MTL operation mode register value.

LOCAL_INLINE bool **ENET_HWA_MTL_GetCounterResetStatus** (const ENET_Type *const pEnet)

Get MTL counter reset status.

LOCAL_INLINE void **ENET_HWA_MTL_ResetCounters** (ENET_Type *const pEnet)

Reset MTL counters.

LOCAL_INLINE bool **ENET_HWA_MTL_GetPresetCounterStatus** (const ENET_Type *const pEnet)

Get MTL counter preset status.

LOCAL_INLINE void **ENET_HWA_MTL_InitPresetCounters** (ENET_Type *const pEnet)

Preset MTL counters.

LOCAL_INLINE **ENET_MTL_TxScheduleAlgorithmType** **ENET_HWA_MTL_GetTxSchedulingAlgorithm**

(const ENET_Type *const pEnet)

Get MTL algorithm for transmit scheduling.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxSchedulingAlgorithm** (ENET_Type *const pEnet,

ENET_MTL_TxScheduleAlgorithmType eScheduleAlgorithm)

Set MTL algorithm for transmit scheduling.

LOCAL_INLINE **ENET_MTL_RxArbitrationAlgorithmType** **ENET_HWA_MTL_GetRxArbitrationAlgorithm**

(const ENET_Type *const pEnet)

Get MTL algorithm for receive arbitration.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxArbitrationAlgorithm** (ENET_Type *const pEnet,

ENET_MTL_RxArbitrationAlgorithmType eArbitrationAlgorithm)

Set MTL algorithm for receive arbitration.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDropTxPacketStatusEnFlag** (const ENET_Type *const pEnet)
Get whether tx packet status received from the MAC is dropped or not.

LOCAL_INLINE void **ENET_HWA_MTL_SetDropTxPacketStatusEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set drop tx packet status enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetDebugCtrl** (const ENET_Type *const pEnet)
Get MTL debug control register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetDebugCtrl** (ENET_Type *const pEnet, uint32_t u32DebugCtrl)
Set MTL debug control register value.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxStatusAvailableIntEnFlag** (const ENET_Type *const pEnet)
Get transmit status available interrupt status enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxStatusAvailableIntEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set transmit status available interrupt status enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxPacketAvailableIntEnFlag** (const ENET_Type *const pEnet)
Get receive packet available interrupt status enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxPacketAvailableIntEnFlag** (ENET_Type *const pEnet, bool bEnable)
Set receive packet available interrupt status enable flag.

LOCAL_INLINE **ENET_MTL_DebugAccessFIFOType** **ENET_HWA_MTL_GetDebugAccessFIFO** (const ENET_Type *const pEnet)
Get the FIFO selected for debug access.

LOCAL_INLINE void **ENET_HWA_MTL_SetDebugAccessFIFO** (ENET_Type *const pEnet, **ENET_MTL_DebugAccessFIFOType** eFIFOType)
Set the FIFO selected for debug access.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFIFOWriteEnFlag** (const ENET_Type *const pEnet)

Get write operation on selected FIFO enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFOWriteEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set write operation on selected FIFO enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSelectedFIFOPointerResetStatus** (const ENET_Type *const pEnet)

Get pointers of selected FIFO reset status.

LOCAL_INLINE void **ENET_HWA_MTL_SetSelectedFIFOPointerResetFlag** (ENET_Type *const pEnet, bool bEnable)

Reset pointers of selected FIFO.

LOCAL_INLINE bool **ENET_HWA_MTL_GetAllFIFOPointersResetStatus** (const ENET_Type *const pEnet)

Get pointers of all FIFOs reset status.

LOCAL_INLINE void **ENET_HWA_MTL_SetAllFIFOPointerResetFlag** (ENET_Type *const pEnet, bool bEnable)

Reset pointers of all FIFOs.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFOPacketState** (ENET_Type *const pEnet, **ENET_MTL_FIFOPacketStatusType** eState)

Encoded packet state, the control information to the Tx FIFO or Rx FIFO.

LOCAL_INLINE void **ENET_HWA_MTL_SetByteEnable** (ENET_Type *const pEnet, **ENET_MTL_ByteEnableType** eByteValid)

Indicates the number of data bytes valid in the data register during Write operation.

LOCAL_INLINE **ENET_MTL_FIFODebugAccessType** **ENET_HWA_MTL_GetFIFODebugModeAccessType** (const ENET_Type *const pEnet)

Get pointers of all FIFOs reset status.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFODebugModeAccessType** (ENET_Type *const pEnet, **ENET_MTL_FIFODebugAccessType** eAccessType)

Set the debug mode of accessing to fifo.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFIFODebugAccessEnFlag** (const ENET_Type *const pEnet)

Get FIFO debug access enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFODebugAccessEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set FIFO debug access enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetDebugStatus** (const ENET_Type *const pEnet)

Get MTL debug status register value.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetFIFORemainingLocation** (const ENET_Type *const pEnet)

Get MTL remaining locations in the FIFO.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxStatusAvailableIntStatus** (const ENET_Type *const pEnet)

Get MTL transmit status available interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearTxStatusAvailableIntStatus** (ENET_Type *const pEnet)

Clear MTL transmit status available interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxPacketAvailableIntStatus** (const ENET_Type *const pEnet)

Get MTL receive packet available interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearRxPacketAvailableIntStatus** (ENET_Type *const pEnet)

Clear MTL receive packet available interrupt status.

LOCAL_INLINE **ENET_MTL_ByteEnableType** **ENET_HWA_MTL_GetByteEnable** (const ENET_Type *const pEnet)

Get the number of data bytes valid in the data register during Read operation.

LOCAL_INLINE **ENET_MTL_FIFOPacketStatusType** **ENET_HWA_MTL_GetFIFOPacketState** (const ENET_Type *const pEnet)

Get the control or status information of the selected FIFO.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFIFOBusy** (const ENET_Type *const pEnet)

Get MTL FIFO busy status.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetFIFODebugData** (const ENET_Type *const pEnet)

Get MTL FIFO debug data.

LOCAL_INLINE void **ENET_HWA_MTL_SetFIFODebugData** (ENET_Type *const pEnet, uint32_t u32Data)

Set MTL FIFO debug data.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetIntStatus** (const ENET_Type *const pEnet)

Get MTL interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDebugIntStatus** (const ENET_Type *const pEnet)

Get debug interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetQueue1IntStatus** (const ENET_Type *const pEnet)

Get Queue 1 interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetQueue0IntStatus** (const ENET_Type *const pEnet)

Get Queue 0 interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxDABasedDMAChannelSelectionEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get DA-based DMA channel selection enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxDABasedDMAChannelSelectionEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set DA-based DMA channel selection enable flag.

LOCAL_INLINE **ENET_DMA_ChannelType** **ENET_HWA_MTL_GetRxDMAChannelMap** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get DMA channel mapped to eQueue.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxDMAChannelMap** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_DMA_ChannelType** eDMAChannel)

Set DMA channel mapped to eQueue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTBSCtrl** (const ENET_Type *const pEnet)

Get Time Based Scheduling control register.

LOCAL_INLINE void **ENET_HWA_MTL_SetTBSCtrl** (ENET_Type *const pEnet, uint32_t u32Ctrl)

Set Time Based Scheduling control register.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetLaunchExpiryOffset** (const ENET_Type *const pEnet)

Get launch expiry offset.

LOCAL_INLINE void **ENET_HWA_MTL_SetLaunchExpiryOffset** (ENET_Type *const pEnet, uint32_t u32ExpiryOffset)

Set launch expiry offset.

LOCAL_INLINE bool **ENET_HWA_MTL_GetLaunchExpiryOffsetEnFlag** (const ENET_Type *const pEnet)

Get launch expiry offset enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetLaunchExpiryOffsetEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set launch expiry offset enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetEstControl** (const ENET_Type *const pEnet)

Get enhancements to scheduled transmission control register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstControl** (ENET_Type *const pEnet, uint32_t u32EstControl)

Set enhancements to scheduled transmission control register value.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetEstPtpTimeOffset** (const ENET_Type *const pEnet)

Get enhancements to scheduled transmission PTP time offset.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstPtpTimeOffset** (ENET_Type *const pEnet, uint8_t u8Value)

Set enhancements to scheduled transmission PTP time offset.

LOCAL_INLINE uint16_t **ENET_HWA_MTL_GetEstCurrentTimeOffset** (const ENET_Type *const pEnet)

Get enhancements to scheduled transmission current time offset.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstCurrentTimeOffset** (ENET_Type *const pEnet, uint16_t u16Value)

Set enhancements to scheduled transmission current time offset.

LOCAL_INLINE **ENET_MTL_GCLIntervalMultipleType** **ENET_HWA_MTL_GetEstTimeIntervalLeftShift** (const ENET_Type *const pEnet)

Get EST time interval left shift amount.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstTimeIntervalLeftShift** (ENET_Type *const pEnet, **ENET_MTL_GCLIntervalMultipleType** eValue)

Set EST time interval left shift amount.

LOCAL_INLINE **ENET_MTL_ScheduleErrorLoopCountType**

ENET_HWA_MTL_GetEstScheduleErrorLoopCount (const ENET_Type *const pEnet)

Get the loop count to report scheduling error.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstScheduleErrorLoopCount** (ENET_Type *const pEnet, **ENET_MTL_ScheduleErrorLoopCountType** eValue)

Set the loop count to report scheduling error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDropSchedulingErrorFramesEnFlag** (const ENET_Type *const pEnet)

Drop frames causing scheduling error.

LOCAL_INLINE void **ENET_HWA_MTL_SetDropSchedulingErrorFramesEnFlag** (ENET_Type *const pEnet, bool bEnable)

Enable dropping frames causing scheduling error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetDropFrameSizeErrorFramesDisableFlag** (const ENET_Type *const pEnet)

Drop frames during frame size error.

LOCAL_INLINE void **ENET_HWA_MTL_SetDropFrameSizeErrorFramesDisableFlag** (ENET_Type *const pEnet, bool bDisable)

Enable dropping frames causing scheduling error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSwitchGateControlListEnFlag** (const ENET_Type *const pEnet)

Get switch gate control list enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetSwitchGateControlListEnFlag** (ENET_Type *const pEnet, bool bEnable)

Switch gate control list.

LOCAL_INLINE bool **ENET_HWA_MTL_GetEstEnableFlag** (const ENET_Type *const pEnet)

Get EST enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstEnableFlag** (ENET_Type *const pEnet, bool bEnable)

Enable EST.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetEstOverheadBytes** (const ENET_Type *const pEnet)

Get EST overhead bytes value.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstOverheadBytes** (ENET_Type *const pEnet, uint8_t u8Value)

Set EST overhead bytes value.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetEstStatus** (const ENET_Type *const pEnet)

Get EST status register value.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetCurrentGCLSlotNumber** (const ENET_Type *const pEnet)
Get current GCL slot number.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetBTRErrorLoopCount** (const ENET_Type *const pEnet)
Get BTR error loop count.

LOCAL_INLINE **ENET_MTL_GateControllListType** **ENET_HWA_MTL_GetSoftwareOwnedList** (const ENET_Type *const pEnet)
Get the gate control list owned by software.

LOCAL_INLINE bool **ENET_HWA_MTL_GetConstantGateControlErrorFlag** (const ENET_Type *const pEnet)
Get constant gate control error.

LOCAL_INLINE void **ENET_HWA_MTL_ClearConstantGateControlErrorFlag** (ENET_Type *const pEnet)
Clear constant gate control error flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadOfLineBlockingDueToSchedulingFlag** (const ENET_Type *const pEnet)
Get Head-Of-Line blocking due to scheduling.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadOfLineBlockingDueToFrameSizeFlag** (const ENET_Type *const pEnet)
Get Head-Of-Line blocking due to frame size.

LOCAL_INLINE bool **ENET_HWA_MTL_GetBTRErrorFlag** (const ENET_Type *const pEnet)
Get BTR error flag.

LOCAL_INLINE void **ENET_HWA_MTL_ClearBTRErrorFlag** (ENET_Type *const pEnet)
Clear BTR error flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSwitchGateControllListCompleteFlag** (const ENET_Type *const pEnet)
Get switch gate control list complete status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearSwitchGateControlListCompleteFlag** (ENET_Type *const pEnet)

Clear switch gate control list complete status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetScheduleErrorQueueNumber** (const ENET_Type *const pEnet)

Get the One Hot encoded queue numbers that are having the scheduling related error.

LOCAL_INLINE void **ENET_HWA_MTL_ClearScheduleErrorQueueNumber** (ENET_Type *const pEnet)

Clear the value of schedule error queue number register.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFrameSizeErrorQueueNumber** (const ENET_Type *const pEnet)

Get the One Hot encoded queue numbers that are having the frame size error.

LOCAL_INLINE void **ENET_HWA_MTL_ClearFrameSizeErrorQueueNumber** (ENET_Type *const pEnet)

Clear the value of frame size error queue number register.

LOCAL_INLINE bool **ENET_HWA_MTL_GetFrameSizeErrorFirstQueueNumber** (const ENET_Type *const pEnet)

Get the queue numbers that are having the first frame size error.

LOCAL_INLINE uint16_t **ENET_HWA_MTL_GetFrameSizeErrorFirstFrameSize** (ENET_Type *const pEnet)

Get the frame size of the first frame size error.

LOCAL_INLINE bool **ENET_HWA_MTL_GetConstantGateControlErrorInterruptEnFlag** (ENET_Type *const pEnet)

Get enable flag of constant gate control error interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetConstantGateControlErrorInterruptEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set enable flag of constant gate control error interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadofLineBlockingDueToSchedulingInterruptEnFlag** (ENET_Type *const pEnet)

Get enable flag of head of line blocking due to scheduling interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetHeadofLineBlockingDueToSchedulingInterruptEnFlag**
(ENET_Type *const pEnet, bool bEnable)

Set enable flag of head of line blocking due to scheduling interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetHeadofLineBlockingDueToFrameSizeInterruptEnFlag**
(ENET_Type *const pEnet)

Get enable flag of head of line blocking due to frame size interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetHeadofLineBlockingDueToFrameSizeInterruptEnFlag**
(ENET_Type *const pEnet, bool bEnable)

Set enable flag of head of line blocking due to frame size interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetBaseTimeRegisterErrorInterruptEnFlag** (ENET_Type *const
pEnet)

Get enable flag of base time register error interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetBaseTimeRegisterErrorInterruptEnFlag** (ENET_Type *const
pEnet, bool bEnable)

Set enable flag of base time register error interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSwitchListInterruptEnFlag** (ENET_Type *const pEnet)

Get enable flag of switch GCL completion interrupt.

LOCAL_INLINE void **ENET_HWA_MTL_SetSwitchListInterruptEnFlag** (ENET_Type *const pEnet, bool
bEnable)

Set enable flag of base time switch GCL completion interrupt.

LOCAL_INLINE bool **ENET_HWA_MTL_GetEstEccInjectErrorControl** (ENET_Type *const pEnet)

Get the mode of EST ECC inject error control for EST memory.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstEccInjectErrorControl** (ENET_Type *const pEnet, bool
bEnable)

Set the mode of EST ECC inject error control for EST memory.

LOCAL_INLINE bool **ENET_HWA_MTL_GetEstEcInjectAddressErrorEnFlag** (ENET_Type *const pEnet)

Get enable flag of EST ECC address error injection.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstEcInjectAddressErrorEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set enable flag of EST ECC address error injection.

LOCAL_INLINE bool **ENET_HWA_MTL_GetEstEcInjectErrorEnFlag** (ENET_Type *const pEnet)

Get enable flag of EST ECC error injection.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstEcInjectErrorEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set enable flag of EST ECC error injection.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetEstGateControlListAddress** (ENET_Type *const pEnet)

Get GCL address or GCL related registers address depend on GCRR bit.

LOCAL_INLINE void **ENET_HWA_MTL_SetEstGateControlListAddress** (ENET_Type *const pEnet, uint8_t eAddress)

Set GCL address or GCL related registers address depend on GCRR bit.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetGclDebugModeBankSelect** (ENET_Type *const pEnet)

Get GCL debug mode bank selected.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclDebugModeBankSelect** (ENET_Type *const pEnet, uint8_t eBank)

Set GCL debug mode bank.

LOCAL_INLINE bool **ENET_HWA_MTL_GetGclDebugModeEnFlag** (ENET_Type *const pEnet)

Get GCL debug mode enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclDebugModeEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set GCL debug mode enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetGateControlReleatedRegistersEnFlag** (ENET_Type *const pEnet)

Get gate control releated registers enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetGateControlReleatedRegistersEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set gate control releated registers enable flag.

LOCAL_INLINE **ENET_MTL_GclOperationType** **ENET_HWA_MTL_GetGclOperation** (ENET_Type *const pEnet)

Get GCL operation mode (read or write)

LOCAL_INLINE void **ENET_HWA_MTL_SetGclOperation** (ENET_Type *const pEnet, **ENET_MTL_GclOperationType** eOperation)

Set GCL operation mode (read or write)

LOCAL_INLINE bool **ENET_HWA_MTL_GetGclOperationBusyState** (ENET_Type *const pEnet)

Get GCL operation busy status.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclOperationStartFlag** (ENET_Type *const pEnet, bool bEnable)

Start GCL Read/Write operation.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetGclData** (ENET_Type *const pEnet)

Get GCL data.

LOCAL_INLINE void **ENET_HWA_MTL_SetGclData** (ENET_Type *const pEnet, uint32_t eData)

Set GCL data register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetPreemptionClassification** (ENET_Type *const pEnet, uint8 u8value)

Set the corresponding Queue classified as express or preemptable.

LOCAL_INLINE void **ENET_HWA_MTL_SetPreemptionReleaseAdvance** (ENET_Type *const pEnet, uint16 u16value)

Set the maximum time in nanoseconds that can elapse between issuing a RELEASE to the MAC and the MAC being ready to resume transmission of preemptable frames.

LOCAL_INLINE void **ENET_HWA_MTL_SetPreemptionHoldAdvance** (ENET_Type *const pEnet, uint16 u16value)

Set the maximum time in nanoseconds that can elapse between issuing a HOLD to the MAC and the MAC ceasing to transmit any preemptable frame.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTxOperationMode** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get MTL queue Tx operation mode register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxOperationMode** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint32_t u32OpMode)

Set MTL queue Tx operation mode register value.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetTxQueueSize** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get MTL transmit queue size.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueSize** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint8_t u8QueueSize)

Set MTL transmit queue size.

LOCAL_INLINE **ENET_MTL_TxThresholdType** **ENET_HWA_MTL_GetTxThreshold** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the threshold level of the MTL Tx Queue.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxThreshold** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_TxThresholdType** eThreshold)

Set the threshold level of the MTL Tx Queue.

LOCAL_INLINE **ENET_MTL_QueueEnableType** **ENET_HWA_MTL_GetTxQueueEnStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the transmit queue enable mode.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueEnStatus** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_QueueEnableType** eEnable)

Set the transmit queue enable mode.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxStoreForwardModeEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get transmit queue store and forward enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxStoreForwardModeEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set transmit queue store and forward enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxFlushState** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get transmit queue flush state.

LOCAL_INLINE void **ENET_HWA_MTL_FlushTxQueue** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Flush transmit queue.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxUnderflowPacketCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, uint16_t *pPacketCnt)

Get the underflow counter register of transmit queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTxDebugStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get transmit queue debug status.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetTxStatusWordCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the current number of status in the transmit Status FIFO of this queue.

```
LOCAL_INLINE uint8_t ENET_HWA_MTL_GetTxPacketCount (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get the current number of packets in the transmit queue.

```
LOCAL_INLINE bool ENET_HWA_MTL_GetTxStatusFIFOFullStatus (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get the full flag of transmit Status FIFO.

```
LOCAL_INLINE bool ENET_HWA_MTL_GetTxQueueNotEmptyStatus (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get the not empty flag of transmit queue data.

```
LOCAL_INLINE ENET_MTL_WriteControllerStatusType  
ENET_HWA_MTL_GetTxQueueWriteControllerStatus (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get the transmit queue write controller status, indicates that the MTL Tx Queue Write controller is active, and it is transferring the data to the transmit queue.

```
LOCAL_INLINE ENET_MTL_ReadControllerStatusType  
ENET_HWA_MTL_GetTxQueueReadControllerStatus (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get the transmit queue read controller status.

```
LOCAL_INLINE bool ENET_HWA_MTL_GetTxQueuePausedStatus (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get the transmit queue paused status.

```
LOCAL_INLINE void ENET_HWA_MTL_SetAVAlgorithmCBSEnFlag (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue, uint8 bEnable)
```

Set AV algorithm for strict priority or credit based shaper.

```
LOCAL_INLINE uint32_t ENET_HWA_MTL_GetSlotAverageBits (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get the average bits per slot,when the DCB operation is enabled for the queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetTxQueueQuantumOrWeight** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the quantum or weights for transmit algorithm (DWRR WRR WFQ)

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueQuantumOrWeight** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32_t u32QuantumWeight)

Set the quantum or weights for transmit algorithm (DWRR WRR WFQ)

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueSendSlopeCredit** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32_t u32SendSlopeCredit)

Set the sendSlope credit value required for the credit-based shaper algorithm for the Queue.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueHiCredit** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32_t u32hiCredit)

Set hiCredit value required for the credit-based shaper algorithm for the Queue.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueLoCredit** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32_t u32LoCredit)

Set the loCredit value required for the credit-based shaper algorithm for the Queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetInterruptCtrlStatus** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the interrupt enable and status for the queue interrupts.

LOCAL_INLINE void **ENET_HWA_MTL_SetInterruptCtrlStatus** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint32_t u32CtrlStatus)

Set the interrupt enable and status for the queue interrupts.

LOCAL_INLINE void **ENET_HWA_MTL_ClearAllIntStatus** (ENET_Type *const pEnet, **ENET_MTL_QueueType**
eQueue)

Clear all the interrupts of the MTL queue.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxQueueOverflowIntEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue overflow interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueueOverflowIntEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set receive queue overflow interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxQueueOverflowIntStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue overflow interrupt status.

LOCAL_INLINE void **ENET_HWA_MTL_ClearRxQueueOverflowIntStatus** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Clear receive queue overflow interrupt status.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSlotAverageBitsIntEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get average bits per slot interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetSlotAverageBitsIntEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set average bits per slot interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetTxQueueUnderflowIntEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get transmit queue underflow interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetTxQueueUnderflowIntEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set transmit queue underflow interrupt enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetSlotAverageBitsIntStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get average bits per slot interrupt status.

```
LOCAL_INLINE void ENET_HWA_MTL_ClearSlotAverageBitsIntStatus (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Clear average bits per slot interrupt status.

```
LOCAL_INLINE bool ENET_HWA_MTL_GetTxQueueUnderflowIntStatus (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get transmit queue underflow interrupt status.

```
LOCAL_INLINE void ENET_HWA_MTL_ClearTxQueueUnderflowIntStatus (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Clear transmit queue underflow interrupt status.

```
LOCAL_INLINE uint32_t ENET_HWA_MTL_GetRxOperationMode (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get receive operation mode of the queue.

```
LOCAL_INLINE void ENET_HWA_MTL_SetRxOperationMode (ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue, uint32_t u32OpMode)
```

Set receive operation mode of the queue.

```
LOCAL_INLINE uint8_t ENET_HWA_MTL_GetRxQueueSize (const ENET_Type *const pEnet,  
ENET_MTL_QueueType eQueue)
```

Get receive queue size.

```
LOCAL_INLINE void ENET_HWA_MTL_SetRxQueueSize (ENET_Type *const pEnet, ENET_MTL_QueueType  
eQueue, uint8_t u8QueueSize)
```

Set receive queue size.

```
LOCAL_INLINE ENET_MTL_RxFlowCtrlThresholdType ENET_HWA_MTL_GetRxDeactivateFlowCtrlThres  
(const ENET_Type *const pEnet, ENET_MTL_QueueType eQueue)
```

Get the threshold of receive queue at which the flow control is deactivated.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxDeactivateFlowCtrlThres** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_RxFlowCtrlThresholdType** eThreshold)

Set the threshold of receive queue at which the flow control is deactivated.

LOCAL_INLINE **ENET_MTL_RxFlowCtrlThresholdType** **ENET_HWA_MTL_GetRxActivateFlowCtrlThres** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the threshold of receive queue at which the flow control is activated.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxActivateFlowCtrlThres** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_RxFlowCtrlThresholdType** eThreshold)

Set the threshold of receive queue at which the flow control is activated.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxHwFlowCtrlEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue hardware flow control enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxHwFlowCtrlEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set receive queue hardware flow control enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetForwardTCPChecksumErrorPacketEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get forwarding of TCP/IP checksum error packets enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetForwardTCPChecksumErrorPacketEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set forwarding of TCP/IP checksum error packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxStoreForwardModeEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue store and forward enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxStoreForwardModeEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set receive queue store and forward enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetForwardErrorPacketEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get forward error packets enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetForwardErrorPacketEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set forward error packets enable flag.

LOCAL_INLINE bool **ENET_HWA_MTL_GetForwardUndersizedPacketEnFlag** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get forward undersized good packets enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetForwardUndersizedPacketEnFlag** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool bEnable)

Set forward undersized good packets enable flag.

LOCAL_INLINE **ENET_MTL_RxThresholdType** **ENET_HWA_MTL_GetRxThreshold** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the receive queue threshold control.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxThreshold** (ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, **ENET_MTL_RxThresholdType** eThreshold)

Set the receive queue threshold.

LOCAL_INLINE void **ENET_HWA_MTL_GetRxMissedPacketOverflowCount** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue, bool *pMissedPacketCntOverflowFlag, uint16_t *pMissedPacketCnt, bool *pOverflowPacketCntOverflowFlag, uint16_t *pOverflowPacketCnt)

Get the missed packet and overflow counters of the queue.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetRxDebugStatus** (const ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get the receive queue debug status.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetRxPacketCount** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get the number of packets in receive queue.

LOCAL_INLINE **ENET_MTL_RxQueueStatusType** **ENET_HWA_MTL_GetRxFillLevelStatus** (const
ENET_Type *const pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue Fill-Level status.

LOCAL_INLINE **ENET_MTL_ReadControllerStatusType**
ENET_HWA_MTL_GetRxQueueReadControllerStatus (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue read controller state.

LOCAL_INLINE **ENET_MTL_WriteControllerStatusType**
ENET_HWA_MTL_GetRxQueueWriteControllerStatus (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue write controller state.

LOCAL_INLINE uint32_t **ENET_HWA_MTL_GetRxControl** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get MTL queue receive control register value.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxControl** (ENET_Type *const pEnet, **ENET_MTL_QueueType**
eQueue, uint32_t u32Ctrl)

Set MTL queue receive control register value.

LOCAL_INLINE bool **ENET_HWA_MTL_GetRxQueuePacketArbitrationEnFlag** (const ENET_Type *const
pEnet, **ENET_MTL_QueueType** eQueue)

Get receive queue packet arbitration enable flag.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueuePacketArbitrationEnFlag** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, bool bEnable)

Set receive queue packet arbitration enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_MTL_GetRxQueueWeight** (const ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue)

Get receive queue weight.

LOCAL_INLINE void **ENET_HWA_MTL_SetRxQueueWeight** (ENET_Type *const pEnet,
ENET_MTL_QueueType eQueue, uint8_t u8Weight)

Set receive queue weight.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetMode** (const ENET_Type *const pEnet)

Get DMA bus mode register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetMode** (ENET_Type *const pEnet, uint32_t u32Mode)

Set DMA bus mode register value.

LOCAL_INLINE **ENET_DMA_StatusType** **ENET_HWA_DMA_GetSwResetStatus** (const ENET_Type *const
pEnet)

Get DMA software reset status.

LOCAL_INLINE void **ENET_HWA_DMA_SwReset** (ENET_Type *const pEnet)

Initiate DMA software reset.

LOCAL_INLINE **ENET_DMA_IntModeType** **ENET_HWA_DMA_GetInterruptMode** (const ENET_Type *const
pEnet)

Get DMA interrupt mode.

LOCAL_INLINE void **ENET_HWA_DMA_SetInterruptMode** (ENET_Type *const pEnet,
ENET_DMA_IntModeType eIntMode)

Set DMA interrupt mode.

LOCAL_INLINE **ENET_DMA_PriorityRatioType** **ENET_HWA_DMA_GetPriorityRatio** (const ENET_Type
*const pEnet)

Get DMA Rx:Tx or Tx:Rx priority ratio.

LOCAL_INLINE void **ENET_HWA_DMA_SetPriorityRatio** (ENET_Type *const pEnet,
ENET_DMA_PriorityRatioType ePriorityRatio)
Set DMA Rx:Tx or Tx:Rx priority ratio.

LOCAL_INLINE **ENET_DMA_PriorityModeType** **ENET_HWA_DMA_GetPriorityMode** (const ENET_Type
*const pEnet)
Get DMA priority mode.

LOCAL_INLINE void **ENET_HWA_DMA_SetPriorityMode** (ENET_Type *const pEnet,
ENET_DMA_PriorityModeType ePriorityMode)
Set DMA priority mode, Rx:Tx or Tx:Rx.

LOCAL_INLINE **ENET_DMA_TxArbitrationAlgorithmType** **ENET_HWA_DMA_GetTxArbitrationAlgorithm**
(const ENET_Type *const pEnet)
Get DMA arbitration algorithm between transmit channels.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxArbitrationAlgorithm** (ENET_Type *const pEnet,
ENET_DMA_TxArbitrationAlgorithmType eArbitrationAlgorithm)
Set DMA arbitration algorithm between transmit channels.

LOCAL_INLINE **ENET_DMA_ArbitrationSchemeType** **ENET_HWA_DMA_GetTxRxArbitrationScheme** (const
ENET_Type *const pEnet)
Get DMA arbitration scheme between the transmit and receive paths.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxRxArbitrationScheme** (ENET_Type *const pEnet,
ENET_DMA_ArbitrationSchemeType eArbitrationScheme)
Set DMA arbitration scheme between the transmit and receive paths.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRebuildINCRxBurstEnFlag** (const ENET_Type *const pEnet)
Get DMA rebuild INCRx burst enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetRebuildINCRxBurstEnFlag** (ENET_Type *const pEnet, bool
bEnable)
Set DMA rebuild INCRx burst enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetMixedBurstEnFlag** (const ENET_Type *const pEnet)

Get DMA mixed burst length enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetMixedBurstEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set DMA mixed burst length enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetAddressAlignedBeatsEnFlag** (const ENET_Type *const pEnet)

Get DMA Address-Aligned Beats enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetAddressAlignedBeatsEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set DMA Address-Aligned Beats enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetFixedBurstLengthEnFlag** (const ENET_Type *const pEnet)

Get DMA fixed burst length enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetFixedBurstLengthEnFlag** (ENET_Type *const pEnet, bool bEnable)

Set DMA fixed burst length enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetDMAIntStatus** (const ENET_Type *const pEnet)

Get DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetMACIntStatus** (const ENET_Type *const pEnet)

Get MAC interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetMTLIntStatus** (const ENET_Type *const pEnet)

Get MTL interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetDMAChannel0IntStatus** (const ENET_Type *const pEnet)

Get DMA channel0 interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetDMAChannel1IntStatus** (const ENET_Type *const pEnet)
Get DMA channel1 interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetDMAChannelIntStatus** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eChannel)
Get DMA channel interrupt event in DMA interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetAHBMasterBusyStatus** (const ENET_Type *const pEnet)
Get AXI Master Read Channel Status.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetFetchTimeOffset** (const ENET_Type *const pEnet, uint8_t
eCtrlIndex)
Get DMA TBS fetch time offset.

LOCAL_INLINE void **ENET_HWA_DMA_SetFetchTimeOffset** (ENET_Type *const pEnet, uint8_t eCtrlIndex,
uint32_t u32FetchTimeOffset)
Set DMA TBS fetch time offset.

LOCAL_INLINE bool **ENET_HWA_DMA_GetFetchTimeOffsetValid** (const ENET_Type *const pEnet, uint8_t
eCtrlIndex)
Get DMA TBS fetch time offset valid.

LOCAL_INLINE void **ENET_HWA_DMA_SetFetchTimeOffsetValid** (ENET_Type *const pEnet, uint8_t
eCtrlIndex, bool bEnable)
Set DMA TBS fetch time offset valid.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetChannelCtrl** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)
Get DMA channel control register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetChannelCtrl** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32ChannelCtrl)
Set DMA channel control register value.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetDescriptorSkipLength** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)
Get DMA descriptor skip length.

LOCAL_INLINE void **ENET_HWA_DMA_SetDescriptorSkipLength** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8Length)
Set DMA descriptor skip length.

LOCAL_INLINE bool **ENET_HWA_DMA_Get8xPBLModeEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)
Get DMA burst length multiply by 8 mode.

LOCAL_INLINE void **ENET_HWA_DMA_Set8xPBLModeEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)
Set DMA burst length multiply by 8 mode.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetChannelTxCtrl** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)
Get DMA transmit channel control register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetChannelTxCtrl** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32TxCtrl)
Set DMA transmit channel control register value.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEnhancedDescriptorEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)
Get DMA enhanced descriptor enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetEnhancedDescriptorEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)
Set DMA enhanced descriptor enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEarlyTxIntEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA early transmit interrupt enable flag.

```
LOCAL_INLINE void ENET_HWA_DMA_SetEarlyTxIntEnFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, bool bEnable)
```

Set DMA early transmit interrupt enable flag.

```
LOCAL_INLINE uint8_t ENET_HWA_DMA_GetTxBurstLength (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA transmit programmable burst length.

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxBurstLength (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, uint8_t u8Length)
```

Set DMA transmit programmable burst length.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetOperateOnSecondPacketEnFlag (const ENET_Type *const  
pEnet, ENET_DMA_ChannelType eDMAChannel)
```

Get DMA operate on second packet enable flag.

```
LOCAL_INLINE void ENET_HWA_DMA_SetOperateOnSecondPacketEnFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, bool bEnable)
```

Set DMA operate on second packet enable flag.

```
LOCAL_INLINE uint8_t ENET_HWA_DMA_GetTxChannelWeight (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA transmit channel weight.

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxChannelWeight (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, uint8_t u8Weight)
```

Set DMA transmit channel weight.

```
LOCAL_INLINE ENET_DMA_StatusType ENET_HWA_DMA_GetTxActiveStatus (const ENET_Type *const  
pEnet, ENET_DMA_ChannelType eDMAChannel)
```

Get DMA transmit channel active status.

LOCAL_INLINE void **ENET_HWA_DMA_StartTx** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Start DMA transmission.

LOCAL_INLINE void **ENET_HWA_DMA_StopTx** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Stop DMA transmission.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetChannelRxCtrl** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA receive channel control register value.

LOCAL_INLINE void **ENET_HWA_DMA_SetChannelRxCtrl** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, uint32_t u32RxCtrl)

Set DMA receive channel control register value.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxPacketFlushEnFlag** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA receive packet flush enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxPacketFlushEnFlag** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, bool bEnable)

Set DMA receive packet flush enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetEarlyRxIntEnFlag** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA early receive interrupt enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetEarlyRxIntEnFlag** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel, bool bEnable)

Set DMA early receive interrupt enable flag.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetRxBurstLength** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA receive programmable burst length.

```
LOCAL_INLINE void ENET_HWA_DMA_SetRxBurstLength (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, uint8_t u8Length)
```

Set DMA receive programmable burst length.

```
LOCAL_INLINE uint16_t ENET_HWA_DMA_GetRxBufferSize (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA receive buffer size.

```
LOCAL_INLINE void ENET_HWA_DMA_SetRxBufferSize (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, uint16_t u16BufferSize)
```

Set DMA receive buffer size.

```
LOCAL_INLINE ENET_DMA_StatusType ENET_HWA_DMA_GetRxActiveStatus (const ENET_Type *const  
pEnet, ENET_DMA_ChannelType eDMAChannel)
```

Get DMA receive channel active status.

```
LOCAL_INLINE void ENET_HWA_DMA_StartRx (ENET_Type *const pEnet, ENET_DMA_ChannelType  
eDMAChannel)
```

Start DMA reception.

```
LOCAL_INLINE void ENET_HWA_DMA_StopRx (ENET_Type *const pEnet, ENET_DMA_ChannelType  
eDMAChannel)
```

Stop DMA reception.

```
LOCAL_INLINE uint32_t ENET_HWA_DMA_GetTxDescListHeadAddr (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA channel Tx descriptor list header address.

```
LOCAL_INLINE void ENET_HWA_DMA_SetTxDescListHeadAddr (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, uint32_t u32TxAddr)
```

Set DMA channel Tx descriptor list header address.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetRxDescListHeadAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Rx descriptor list header address.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxDescListHeadAddr** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32RxAddr)

Set DMA channel Rx descriptor list header address.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetTxDescListTailAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Tx descriptor list tail address.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxDescListTailAddr** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32TxAddr)

Set DMA channel Tx descriptor list tail address.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetRxDescListTailAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Rx descriptor list tail address.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxDescListTailAddr** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32RxAddr)

Set DMA channel Rx descriptor list tail address.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetTxDescRingLength** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Tx descriptor ring length.

LOCAL_INLINE void **ENET_HWA_DMA_SetTxDescRingLength** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t u16Length)

Set DMA channel Tx descriptor ring length.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetRxDescRingLength** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA channel Rx descriptor ring length.

```
LOCAL_INLINE void ENET_HWA_DMA_SetRxDescRingLength (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, uint16_t u16Length)
```

Set DMA channel Rx descriptor ring length.

```
LOCAL_INLINE uint32_t ENET_HWA_DMA_GetEnabledInterrupts (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA channel enabled interrupts.

```
LOCAL_INLINE void ENET_HWA_DMA_SetEnabledInterrupts (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, uint32_t u32IntEnable)
```

Set DMA channel interrupts enable state.

```
LOCAL_INLINE void ENET_HWA_DMA_EnableInterrupt (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, ENET_DMA_InterruptFlagType eIntFlag)
```

Enable one of the DMA channel interrupts.

```
LOCAL_INLINE void ENET_HWA_DMA_DisableInterrupt (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, ENET_DMA_InterruptFlagType eIntFlag)
```

Disable one of the DMA channel interrupts.

```
LOCAL_INLINE ENET_DMA_WDGCounterUnitType ENET_HWA_DMA_GetRxWDGCounterUnit (const  
ENET_Type *const pEnet, ENET_DMA_ChannelType eDMAChannel)
```

Get receive interrupt watchdog timer count unit.

```
LOCAL_INLINE void ENET_HWA_DMA_SetRxWDGCounterUnit (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel, ENET_DMA_WDGCounterUnitType eCounterUnit)
```

Set receive interrupt watchdog timer count unit.

```
LOCAL_INLINE uint8_t ENET_HWA_DMA_GetRxWDGCount (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get receive interrupt watchdog timer count.

LOCAL_INLINE void **ENET_HWA_DMA_SetRxWDGCount** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8WDGCnt)

Set receive interrupt watchdog timer count.

LOCAL_INLINE uint8_t **ENET_HWA_DMA_GetRefSlotNumber** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get reference slot number in the DMA.

LOCAL_INLINE void **ENET_HWA_DMA_SetRefSlotNumber** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint8_t u8SlotNumber)

Set reference slot number in the DMA.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetSlotInterval** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get slot interval value.

LOCAL_INLINE void **ENET_HWA_DMA_SetSlotInterval** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t u16SlotInterval)

Set slot interval value.

LOCAL_INLINE bool **ENET_HWA_DMA_GetAdvanceSlotCheckEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get advance slot check enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetAdvanceSlotCheckEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set advance slot check enable flag.

LOCAL_INLINE bool **ENET_HWA_DMA_GetSlotComparisonEnFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get slot comparison enable flag.

LOCAL_INLINE void **ENET_HWA_DMA_SetSlotComparisonEnFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, bool bEnable)

Set slot comparison enable flag.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentTxDescAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Tx descriptor address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentRxDescAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Rx descriptor address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentTxBufAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Tx buffer address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentRxBufAddr** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current Rx buffer address pointer.

LOCAL_INLINE uint32_t **ENET_HWA_DMA_GetCurrentStatus** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA current status.

LOCAL_INLINE void **ENET_HWA_DMA_SetCurrentStatus** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint32_t u32Status)

Set DMA current status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearAllAbnormalInterruptFlags** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear all DMA abnormal interrupts.

LOCAL_INLINE void **ENET_HWA_DMA_ClearAllNormalInterruptFlags** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear all DMA normal interrupts.

LOCAL_INLINE **ENET_DMA_RxErrorType** **ENET_HWA_DMA_GetRxError** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA Rx error status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxError** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Clear DMA Rx error status.

LOCAL_INLINE **ENET_DMA_TxErrorType** **ENET_HWA_DMA_GetTxError** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA Tx error status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxError** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Clear DMA Tx error status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetNormalInterruptFlag** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA normal interrupt summary.

LOCAL_INLINE void **ENET_HWA_DMA_ClearNormalInterruptFlag** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Clear DMA normal interrupt summary.

LOCAL_INLINE bool **ENET_HWA_DMA_GetAbnormalInterruptFlag** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA abnormal interrupt summary.

LOCAL_INLINE void **ENET_HWA_DMA_ClearAbnormalInterruptFlag** (ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Clear DMA abnormal interrupt summary.

LOCAL_INLINE bool **ENET_HWA_DMA_GetContextDescriptorErrorFlag** (const ENET_Type *const pEnet, **ENET_DMA_ChannelType** eDMAChannel)

Get DMA context descriptor error.

```
LOCAL_INLINE void ENET_HWA_DMA_ClearContextDescriptorErrorFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Clear DMA context descriptor error.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetFatalBusErrorFlag (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA fatal bus error.

```
LOCAL_INLINE void ENET_HWA_DMA_ClearFatalBusErrorFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Clear DMA fatal bus error.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetEarlyRxInterruptFlag (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA early receive interrupt.

```
LOCAL_INLINE void ENET_HWA_DMA_ClearEarlyRxInterruptFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Clear DMA early receive interrupt.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetEarlyTxInterruptFlag (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA early transmit interrupt.

```
LOCAL_INLINE void ENET_HWA_DMA_ClearEarlyTxInterruptFlag (ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Clear DMA early transmit interrupt.

```
LOCAL_INLINE bool ENET_HWA_DMA_GetRxWDGTimeoutFlag (const ENET_Type *const pEnet,  
ENET_DMA_ChannelType eDMAChannel)
```

Get DMA receive watchdog timeout status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxWDGTimeoutFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive watchdog timeout status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxProcessStoppedFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive process stopped status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxProcessStoppedFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive process stopped status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxBufferUnavailableFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive buffer unavailable status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxBufferUnavailableFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive buffer unavailable status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetRxInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA receive interrupt status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearRxInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA receive interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetTxBufferUnavailableFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit buffer unavailable status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxBufferUnavailableFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA transmit buffer unavailable status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetTxProcessStoppedFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit process stopped status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxProcessStoppedFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA transmit process stopped status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetTxInterruptFlag** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get DMA transmit interrupt status.

LOCAL_INLINE void **ENET_HWA_DMA_ClearTxInterruptFlag** (ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Clear DMA transmit interrupt status.

LOCAL_INLINE bool **ENET_HWA_DMA_GetMissFrameCount** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel, uint16_t *pMissPacketCnt)

Get the miss frame count and overflow status of the Ethernet DMA channel.

LOCAL_INLINE uint16_t **ENET_HWA_DMA_GetEarlyRxInterruptCount** (const ENET_Type *const pEnet,
ENET_DMA_ChannelType eDMAChannel)

Get the count of the number of times Early Receive Interrupt was asserted.

Detailed Description

ENET hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-10-12

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_erm.h File Reference

ERM Module Register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_erm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **ERM_HWA_SetCRn** (ERM_Type *const pErm, uint8_t u8Idx, uint32_t u32RegValue)
set erm configuration register.

LOCAL_INLINE uint32_t **ERM_HWA_GetCRn** (const ERM_Type *const pErm, uint8_t u8Idx)
get ERM configuration register.

LOCAL_INLINE void **ERM_HWA_SetSRn** (ERM_Type *const pErm, uint8_t u8Idx, uint32_t u32RegValue)
set erm status register.

LOCAL_INLINE uint32_t **ERM_HWA_GetSRn** (const ERM_Type *const pErm, uint8_t u8Idx)
get ERM status register.

LOCAL_INLINE uint32_t **ERM_HWA_GetEARN** (const ERM_Type *const pErm, uint8_t u8Idx)
get ERM error address register.

Detailed Description

ERM Module Register API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_fciic.h File Reference

FCIIC hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_fciic.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum FCIIC_MsrType { FCIIC_MSR_TDF = FCIIC_MSR_TDF_MASK, FCIIC_MSR_RDF =  
FCIIC_MSR_RDF_MASK, FCIIC_MSR_EPF = FCIIC_MSR_EPF_MASK, FCIIC_MSR_SDF =  
FCIIC_MSR_SDF_MASK, FCIIC_MSR_NDF = FCIIC_MSR_NDF_MASK, FCIIC_MSR_ALF =  
FCIIC_MSR_ALF_MASK, FCIIC_MSR_FEF = FCIIC_MSR_FEF_MASK, FCIIC_MSR_PLTF =  
FCIIC_MSR_PLTF_MASK, FCIIC_MSR_DMF = FCIIC_MSR_DMF_MASK, FCIIC_MSR_MBF =  
FCIIC_MSR_MBF_MASK, FCIIC_MSR_BBF = FCIIC_MSR_BBF_MASK, FCIIC_MasterClearFlags,  
FCIIC_MasterErrorFlags }
```

FCIIC MSR flags.

```
enum FCIIC_MierType { FCIIC_MIER_TDIE = FCIIC_MIER_TDIE_MASK, FCIIC_MIER_RDIE =  
FCIIC_MIER_RDIE_MASK, FCIIC_MIER_EPIE = FCIIC_MIER_EPIE_MASK, FCIIC_MIER_SDIE =  
FCIIC_MIER_SDIE_MASK, FCIIC_MIER_NDIE = FCIIC_MIER_NDIE_MASK, FCIIC_MIER_ALIE =  
FCIIC_MIER_ALIE_MASK, FCIIC_MIER_FEIE = FCIIC_MIER_FEIE_MASK, FCIIC_MIER_PLTIE =  
FCIIC_MIER_PLTIE_MASK, FCIIC_MIER_DMIE = FCIIC_MIER_DMIE_MASK, FCIIC_MasterIrqErrorFlags =  
FCIIC_MIER_ALIE | FCIIC_MIER_FEIE | FCIIC_MIER_PLTIE, FCIIC_MasterIrqFlags }
```

FCIIC MIER flags.

```
enum FCIIC_TX_CMDType { FCIIC_TX_CMD_TRANSMIT = 0, FCIIC_TX_CMD_RECEIVE,  
FCIIC_TX_CMD_STOP, FCIIC_TX_CMD_RECANDDISCARD, FCIIC_TX_CMD_STARTANDTRANSMIT,  
FCIIC_TX_CMD_STARTANDTRANSMIT_WITHNAK }
```

IIC Command Type Enumeration.

```
enum FCIIC_MasterStatusType { FCIIC_MSR_BBF_STATUS = 25U, FCIIC_MSR_MBF_STATUS = 24U,  
FCIIC_MSR_DMF_STATUS = 14U, FCIIC_MSR_PLTF_STATUS = 13U, FCIIC_MSR_FEF_STATUS = 12U,  
FCIIC_MSR_ALF_STATUS = 11U, FCIIC_MSR_NDF_STATUS = 10U, FCIIC_MSR_SDF_STATUS = 9U,  
FCIIC_MSR_EPF_STATUS = 8U, FCIIC_MSR_RDF_STATUS = 1U, FCIIC_MSR_TDF_STATUS = 0U }
```

IIC Master status type enumeration.

```
enum FCIIC_SsrType { FCIIC_SSR_TDF = FCIIC_SSR_TDF_MASK, FCIIC_SSR_RDF = FCIIC_SSR_RDF_MASK,  
FCIIC_SSR_AVF = FCIIC_SSR_AVF_MASK, FCIIC_SSR_TAF = FCIIC_SSR_TAF_MASK, FCIIC_SSR_RSF =  
FCIIC_SSR_RSF_MASK, FCIIC_SSR_SDF = FCIIC_SSR_SDF_MASK, FCIIC_SSR_BEF = FCIIC_SSR_BEF_MASK,  
FCIIC_SSR_TREF = FCIIC_SSR_TREF_MASK, FCIIC_SSR_AM0F = FCIIC_SSR_AM0F_MASK, FCIIC_SSR_AM1F  
= FCIIC_SSR_AM1F_MASK, FCIIC_SSR_GCF = FCIIC_SSR_GCF_MASK, FCIIC_SSR_SARF =  
FCIIC_SSR_SARF_MASK, FCIIC_SSR_SBF = FCIIC_SSR_SBF_MASK, FCIIC_SSR_BBF = FCIIC_SSR_BBF_MASK,  
FCIIC_SlaveClearFlags, FCIIC_SlaveErrorFlags = FCIIC_SSR_BEF | FCIIC_SSR_TREF }
```

FCIIC SSR flags.

```
enum FCIIC_SierType { FCIIC_SIER_TDIE = FCIIC_SIER_TDIE_MASK, FCIIC_SIER_RDIE =  
FCIIC_SIER_RDIE_MASK, FCIIC_SIER_AVIE = FCIIC_SIER_AVIE_MASK, FCIIC_SIER_TAIE =  
FCIIC_SIER_TAIE_MASK, FCIIC_SIER_RSIE = FCIIC_SIER_RSIE_MASK, FCIIC_SIER_SDIE =  
FCIIC_SIER_SDIE_MASK, FCIIC_SIER_BEIE = FCIIC_SIER_BEIE_MASK, FCIIC_SIER_TREIE =  
FCIIC_SIER_TREIE_MASK, FCIIC_SIER_AM0IE = FCIIC_SIER_AM0IE_MASK, FCIIC_SIER_AM1IE =  
FCIIC_SIER_AM1IE_MASK, FCIIC_SIER_GCIE = FCIIC_SIER_GCIE_MASK, FCIIC_SIER_SARIE =  
FCIIC_SIER_SARIE_MASK, FCIIC_SlaveIrqFlags }
```

FCIIC MIER flags.

```
enum FCIIC_SlaveStatusType { FCIIC_SSR_BBF_STATUS = 25U, FCIIC_SSR_SBF_STATUS = 24U,  
FCIIC_SSR_SARF_STATUS = 15U, FCIIC_SSR_GCF_STATUS = 14U, FCIIC_SSR_AM1F_STATUS = 13U,  
FCIIC_SSR_AM0F_STATUS = 12U, FCIIC_SSR_TREF_STATUS = 11U, FCIIC_SSR_BEF_STATUS = 10U,  
FCIIC_SSR_SDF_STATUS = 9U, FCIIC_SSR_RSF_STATUS = 8U, FCIIC_SSR_TAF_STATUS = 3U,  
FCIIC_SSR_AVF_STATUS = 2U, FCIIC_SSR_RDF_STATUS = 1U, FCIIC_SSR_TDF_STATUS = 0U }
```

IIC Slave status type enumeration.

Functions

LOCAL_INLINE void **FCIIC_HwA_MasterReset** (FCIIC_Type *const base)

Restores the FCIIC master peripheral to reset conditions.

LOCAL_INLINE void **FCIIC_HwA_MasterEnable** (FCIIC_Type *const base, bool enable)

Enables or disables the FCIIC module as master.

LOCAL_INLINE void **FCIIC_HWA_MasterResetFIFO** (FCIIC_Type *const base)

Resets the FCIIC module's transmit and receive FIFOs.

LOCAL_INLINE uint8_t **FCIIC_HWA_MasterGetEnable** (FCIIC_Type *const base)

Retrieves the current status of the FCIIC master enable bit.

LOCAL_INLINE uint32_t **FCIIC_HWA_MasterGetStatusFlags** (const FCIIC_Type *const base)

Gets the FCIIC master status flags.

LOCAL_INLINE void **FCIIC_HWA_MasterClearStatusFlags** (FCIIC_Type *const base, uint32_t statusMask)

Clears the FCIIC master status flag state.

LOCAL_INLINE void **FCIIC_HWA_MasterEnableInterrupts** (FCIIC_Type *const base, uint32_t interruptMask)

Enables the FCIIC master interrupt requests.

LOCAL_INLINE void **FCIIC_HWA_MasterDisableInterrupts** (FCIIC_Type *const base, uint32_t interruptMask)

Disables the FCIIC master interrupt requests.

LOCAL_INLINE uint32_t **FCIIC_HWA_MasterGetEnabledInterrupts** (const FCIIC_Type *const base)

Returns the set of currently enabled FCIIC master interrupt requests.

LOCAL_INLINE void **FCIIC_HWA_MasterEnableDMA** (FCIIC_Type *const base, bool enableTx, bool enableRx)

Enables or disables FCIIC master DMA requests.

LOCAL_INLINE uint32_t **FCIIC_HWA_MasterGetTxFifoAddress** (const FCIIC_Type *const base)

Gets FCIIC master transmit data register address for DMA transfer.

LOCAL_INLINE uint32_t **FCIIC_HWA_MasterGetRxFifoAddress** (const FCIIC_Type *const base)

Gets FCIIC master receive data register address for DMA transfer.

LOCAL_INLINE void **FCIIC_HWA_MasterSetWatermarks** (FCIIC_Type *const base, uint16_t txWords, uint16_t rxWords)

Sets the watermarks for FCIIC master FIFOs.

LOCAL_INLINE void **FCIIC_Master_HWA_Transmit** (FCIIC_Type *pFciic, **FCIIC_TX_CMDType** tCmdType, uint8_t u8Data)

Master Transmit Data.

LOCAL_INLINE uint32 **FCIIC_Master_HWA_GetTxCount** (FCIIC_Type *pFciic)

Get the number of bytes transmitted by the master.

LOCAL_INLINE uint8_t **FCIIC_Master_HWA_Receive** (FCIIC_Type *pFciic)

Master Receive Data.

LOCAL_INLINE void **FCIIC_Master_HWA_CheckClearStatus** (FCIIC_Type *pFciic, **FCIIC_MasterStatusType** eStatus)

Get and clear IIC master status.

LOCAL_INLINE uint8_t **FCIIC_Master_HWA_GetStatus** (FCIIC_Type *pFciic, **FCIIC_MasterStatusType** eStatus)

Get master status.

LOCAL_INLINE void **FCIIC_Master_HWA_ClrStatus** (FCIIC_Type *pFciic, uint32_t u32Value)

Clear master status.

LOCAL_INLINE void **FCIIC_Master_HWA_EnableReceiveInterrupt** (FCIIC_Type *pFciic)

Enable Receive Interrupt.

LOCAL_INLINE void **FCIIC_Master_HWA_DisableReceiveInterrupt** (FCIIC_Type *pFciic)

Disable Receive Interrupt.

LOCAL_INLINE void **FCIIC_Master_HWA_EnableErrorInterrupt** (FCIIC_Type *pFciic)

Enable Error Interrupt.

LOCAL_INLINE void **FCIIC_Master_HWA_DisableErrorInterrupt** (FCIIC_Type *pFciic)

Disable Error Interrupt.

LOCAL_INLINE uint32_t **FCIIC_Master_HWA_GetErrorFlag** (FCIIC_Type *pFciic)

Get Error Flag.

LOCAL_INLINE void **FCIIC_Master_HWA_EnableTransmitInterrupt** (FCIIC_Type *pFciic)

Enable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_Master_HWA_DisableTransmitInterrupt** (FCIIC_Type *pFciic)

Disable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableInterrupt** (FCIIC_Type *const pFciic, uint32 u32Mask)

Enable FCIIC SIER register bit.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableInterrupt** (FCIIC_Type *const pFciic, uint32 u32Mask)

Disable FCIIC SIER register bit.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableReceiveInterrupt** (FCIIC_Type *pFciic)

Enable Receive Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableReceiveInterrupt** (FCIIC_Type *pFciic)

Disable Receive Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableErrorInterrupt** (FCIIC_Type *pFciic)

Enable Error Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableErrorInterrupt** (FCIIC_Type *pFciic)

Disable Error Interrupt.

LOCAL_INLINE uint32_t **FCIIC_Slave_HWA_GetErrorFlag** (FCIIC_Type *pFciic)

Get Error Flag.

LOCAL_INLINE void **FCIIC_Slave_HWA_ClrErrorFlag** (FCIIC_Type *pFciic)

Clear Error Flag.

LOCAL_INLINE void **FCIIC_Slave_HWA_EnableTransmitInterrupt** (FCIIC_Type *pFciic)

Enable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_Slave_HWA_DisableTransmitInterrupt** (FCIIC_Type *pFciic)

Disable Transmit Interrupt.

LOCAL_INLINE void **FCIIC_HWA_SlaveEnableDMA** (FCIIC_Type *const pFciic, bool enableTx, bool enableRx)

Enables or disables FCIIC Slave DMA requests.

LOCAL_INLINE uint8_t **FCIIC_HWA_SlaveGetEnable** (FCIIC_Type *const pFciic)

Checks if the slave enable flag is set in the SCR register.

LOCAL_INLINE uint8_t **FCIIC_Slave_HWA_GetStatus** (FCIIC_Type *pFciic, **FCIIC_SlaveStatusType** eStatus)

This Function is used to get slave status.

LOCAL_INLINE void **FCIIC_Slave_HWA_ClrStatus** (FCIIC_Type *pFciic, uint32_t eStatus)

This Function is used to clear slave status.

LOCAL_INLINE uint32 **FCIIC_HWA_GetSIER** (const FCIIC_Type *const pFciic)

get FCIIC SIER register.

LOCAL_INLINE void **FCIIC_Slave_HWA_Transmit** (FCIIC_Type *pFciic, uint8_t u8Data)

Slave Transmit Data.

LOCAL_INLINE uint8_t **FCIIC_Slave_HWA_Receive** (FCIIC_Type *pFciic)

Slave Get Data.

LOCAL_INLINE void **FCIIC_Slave_HWA_NACK** (FCIIC_Type *const pFciic, uint32 u32Value)

Set FCIIC STAR register.

LOCAL_INLINE void **FCIIC_HWA_SetMcr** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCR register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetMSR** (FCIIC_Type *pFciic)

Set FCIIC SSR register.

LOCAL_INLINE void **FCIIC_HWA_AttachMcr** (FCIIC_Type *pFciic, uint32_t u32Value)

Attach FCIIC MCR register.

LOCAL_INLINE void **FCIIC_HWA_SetMder** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MDER register.

LOCAL_INLINE uint32 **FCIIC_HWA_GetMIER** (const FCIIC_Type *const pFciic)

get FCIIC MIER register.

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR0** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR0 register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetMCFGR1** (FCIIC_Type *pFciic)

get FCIIC MCFGR1 register

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR1** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR1 register.

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR2** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR2 register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetMCFGR2** (FCIIC_Type *pFciic)

Set FCIIC MCFGR2 register.

LOCAL_INLINE void **FCIIC_HWA_SetMCFGR3** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCFGR3 register.

LOCAL_INLINE void **FCIIC_HWA_SetMFCR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MFCR register.

LOCAL_INLINE void **FCIIC_HWA_SetMCCR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC MCCR register.

LOCAL_INLINE void **FCIIC_HWA_SetSDER** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SDER register.

LOCAL_INLINE void **FCIIC_HWA_SetSCFGR1** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SCFGR1 register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetSCFGR1** (FCIIC_Type *pFciic)

Get FCIIC SCFGR1 register.

LOCAL_INLINE void **FCIIC_HWA_SetSCFGR2** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SCFGR2 register.

LOCAL_INLINE void **FCIIC_HWA_SetSAMR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SAMR register.

LOCAL_INLINE void **FCIIC_HWA_SetSCR** (FCIIC_Type *pFciic, uint32_t u32Value)

Set FCIIC SCR register.

LOCAL_INLINE uint32_t **FCIIC_HWA_GetSSR** (FCIIC_Type *pFciic)

Set FCIIC SSR register.

LOCAL_INLINE uint32 **FCIIC_HWA_GetSDER** (FCIIC_Type *const pFciic)

Set FCIIC SDER register.

LOCAL_INLINE uint32 **FCIIC_HWA_GetSASR** (const FCIIC_Type *const pFciic)

Get FCIIC slave received address.

Detailed Description

FCIIC hardware access layer.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_fc pit.h File Reference

FCPIT hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_fc pit.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum FCPIT_TimerModeType { FCPIT_32PERIODIC_COUNTER = 0, FCPIT_DUAL_16PERIODIC_COUNTER,  
FCPIT_ACCUMULATOR, FCPIT_INPUT_CAPTURE, FCPIT_CHANNEL_NOMODE }
```

Fcpit counter mode, the default mode is 32bit periodic count mode.

```
enum FCPIT_ChannelType { FCPIT_CHANNEL_0 = 0U, FCPIT_CHANNEL_1, FCPIT_CHANNEL_2,  
FCPIT_CHANNEL_3, FCPIT_CHANNEL_MAX }
```

Fcpit channel number.

Functions

```
LOCAL_INLINE void FCPIT_HWA_SetChannelValue (FCPIT_Type *pFcpit, FCPIT_ChannelType eChannel,  
uint32_t u32RegValue)
```

Set FCPIT channel value.

```
LOCAL_INLINE void FCPIT_HWA_ConfigChannel (FCPIT_Type *pFcpit, FCPIT_ChannelType eChannel,  
uint32_t u32RegValue)
```

Configure FCPIT channel.

```
LOCAL_INLINE void FCPIT_HWA_ConfigModule (FCPIT_Type *pFcpit, uint32_t u32RegValue)
```

Configure FCPIT module.

```
LOCAL_INLINE uint32_t FCPIT_HWA_ReadModuleEnable (FCPIT_Type *pFcpit)
```

Read FCPIT module enable.

LOCAL_INLINE uint32_t **FCPIT_HWA_ReadChannelEnable** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Read FCPIT channel.

LOCAL_INLINE uint32_t **FCPIT_HWA_ReadInterruptFlag** (FCPIT_Type *pFcpit)

Read FCPIT active interrupt flag.

LOCAL_INLINE uint32_t **FCPIT_HWA_ReadEnableInterruptFlag** (FCPIT_Type *pFcpit)

Read FCPIT enable interrupt flag.

LOCAL_INLINE void **FCPIT_HWA_SetChannelRunOnDebug** (FCPIT_Type *pFcpit)

Set FCPIT channel running on debug mode.

LOCAL_INLINE void **FCPIT_HWA_SetChannelRunOnLpm** (FCPIT_Type *pFcpit)

Set FCPIT channel running on low power mode.

LOCAL_INLINE void **FCPIT_HWA_EnableModule** (FCPIT_Type *pFcpit)

Enable FCPIT module.

LOCAL_INLINE void **FCPIT_HWA_EnableChannelsInterrupt** (FCPIT_Type *pFcpit, uint32_t u32RegValue)

Enable FCPIT channel(n) interrupt.

LOCAL_INLINE void **FCPIT_HWA_EnableChannel** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Enable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_SetTimerEnable** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Enable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_ClearTimerEnable** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Disable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_EnableChannelChainMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Enable FCPIT channel(n) chain mode.

LOCAL_INLINE void **FCPIT_HWA_ConfigChannelMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel, **FCPIT_TimerModeType** eMode)

Configure FCPIT channel operation mode.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStartOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel start on trigger.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStopOnInterrupt** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel stop on interrupt.

LOCAL_INLINE void **FCPIT_HWA_SetChannelReloadOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel reload on trigger.

LOCAL_INLINE void **FCPIT_HWA_SetChannelTriggerSrc** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Set FCPIT channel trigger source.

LOCAL_INLINE void **FCPIT_HWA_SelectChannelTrigger** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel, uint8_t u8SelChannel)

Select FCPIT channel trigger.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStopOnDebug** (FCPIT_Type *pFcpit)

Set FCPIT channel stop on debug mode.

LOCAL_INLINE void **FCPIT_HWA_SetChannelStopOnLpm** (FCPIT_Type *pFcpit)

Set FCPIT channel stop on low power mode.

LOCAL_INLINE void **FCPIT_HWA_DisableModule** (FCPIT_Type *pFcpit)

Disable FCPIT module.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelsInterruptFlag** (FCPIT_Type *pFcpit, uint32_t u32RegValue)
Clear FCPIT channel(n) interrupt flag.

LOCAL_INLINE void **FCPIT_HWA_DisableChannelsInterrupt** (FCPIT_Type *pFcpit, uint32_t u32RegValue)
Disable FCPIT channel(n) interrupt.

LOCAL_INLINE void **FCPIT_HWA_DisableChannel** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)
Disable FCPIT channel.

LOCAL_INLINE void **FCPIT_HWA_DisableChannelChainMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)
Disable FCPIT channel chain mode.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelMode** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)
Clear FCPIT channel operation mode.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelStartOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)
Clear FCPIT channel start on trigger.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelStopOnInterrupt** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)
Clear FCPIT channel stop on interrupt.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelReloadOnTrig** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)
Clear FCPIT channel reload on trigger.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelTriggerSrc** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)
Clear FCPIT channel trigger source.

LOCAL_INLINE void **FCPIT_HWA_ClearChannelTriggerSelect** (FCPIT_Type *pFcpit, **FCPIT_ChannelType** eChannel)

Clear FCPIT channel trigger select.

Detailed Description

FCPIT hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_fcsmu.h File Reference

FCSMU module register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_fcsmu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **FCSMU_HWA_SetCtrl** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu CTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCtrl** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCtrlOps** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CTRL register OPS.

LOCAL_INLINE void **FCSMU_HWA_SetOprk** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu OPRK register.

LOCAL_INLINE void **FCSMU_HWA_SetSoctrl** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu SOCTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSoctrl** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SOCTRL register.

LOCAL_INLINE void **FCSMU_HWA_SetFccr0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FCCR0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFccr0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FCCR0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFrst0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FRST0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFrst0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FRST0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFst0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FST0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFst0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FST0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFstUnlk** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FST_UNLK register.

LOCAL_INLINE void **FCSMU_HWA_SetFe0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FE0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFe0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FE0 register.

LOCAL_INLINE void **FCSMU_HWA_SetWarningEn0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu WARNING_EN0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWarningEn0** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WARNING_EN0 register.

LOCAL_INLINE void **FCSMU_HWA_SetWarningTo** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu WARNING_TO register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWarningTo** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WARNING_TO register.

LOCAL_INLINE void **FCSMU_HWA_SetCfgTo** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu CFG_TO register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCfgTo** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CFG_TO register.

LOCAL_INLINE void **FCSMU_HWA_SetSoutDiag** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu SOUT_DIAG register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSoutDiag** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SOUT_DIAG register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetStatus** (const FCSMU_Type *const pFcsmu)

Get the fcsmu STATUS register.

LOCAL_INLINE bool **FCSMU_HWA_GetFaultState** (const FCSMU_Type *const pFcsmu)

Get the fcsmu STATUS register FIF value.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFcsmuState** (const FCSMU_Type *const pFcsmu)

Get the fcsmu STATUS register STAT value.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetNtw** (const FCSMU_Type *const pFcsmu)

Get the fcsmu NTW register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWtf** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WTF register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetNtf** (const FCSMU_Type *const pFcsmu)

Get the fcsmu NTF register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetFtw** (const FCSMU_Type *const pFcsmu)

Get the fcsmu FTW register.

LOCAL_INLINE void **FCSMU_HWA_SetInject** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu INJECT register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetIrqStat** (const FCSMU_Type *const pFcsmu)

Get the fcsmu IRQ_STAT register.

LOCAL_INLINE void **FCSMU_HWA_ClearCfgToIrq** (FCSMU_Type *const pFcsmu)

Clear the fcsmu IRQ_STAT register.

LOCAL_INLINE void **FCSMU_HWA_EnableCfgToIrq** (FCSMU_Type *const pFcsmu, bool bEnable)

Set the fcsmu IRQ_EN register.

LOCAL_INLINE void **FCSMU_HWA_SetTempUnlk** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu TEMP_UNLK register.

LOCAL_INLINE void **FCSMU_HWA_SetPermntLock** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu PERMNT_LOCK register.

LOCAL_INLINE void **FCSMU_HWA_SetStmr** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu STMR register.

LOCAL_INLINE void **FCSMU_HWA_SetWarningIen0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu WARNING_IEN0 register.

LOCAL_INLINE void **FCSMU_HWA_SetFaultIen0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu FAULT_IEN0 register.

LOCAL_INLINE void **FCSMU_HWA_SetSoutEn0** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu SOUT_EN0 register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetWarningTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu WARNING_TMR register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSafeModeTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SM_TMR register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCfgTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CFG_TMR register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetSoutTmr** (const FCSMU_Type *const pFcsmu)

Get the fcsmu SOUT_TMR register.

LOCAL_INLINE void **FCSMU_HWA_SetCrcCtrl** (FCSMU_Type *const pFcsmu, uint32_t u32Value)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcBusy** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register BUSY value.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcErrorFlag** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register EF value.

LOCAL_INLINE void **FCSMU_HWA_ClearCrcErrorFlag** (FCSMU_Type *const pFcsmu)

Clear the fcsmu CRC_CTRL register EF.

LOCAL_INLINE void **FCSMU_HWA_EnableErrorOutput** (FCSMU_Type *const pFcsmu, bool bEnable)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcErrorOutputEnable** (const FCSMU_Type *const pFcsmu)

Get the fcsmu CRC_CTRL register EOEN value.

LOCAL_INLINE void **FCSMU_HWA_EnableCrcChecker** (FCSMU_Type *const pFcsmu, bool bEnable)

Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetCrcCheckerEnable** (const FCSMU_Type *const pFcsmu)
Get the fcsmu CRC_CTRL register CHKEN value.

LOCAL_INLINE void **FCSMU_HWA_EnableTrigger** (FCSMU_Type *const pFcsmu, bool bEnable)
Set the fcsmu CRC_CTRL register.

LOCAL_INLINE bool **FCSMU_HWA_GetTriggerEnable** (const FCSMU_Type *const pFcsmu)
Get the fcsmu CRC_CTRL register TRGEN value.

LOCAL_INLINE void **FCSMU_HWA_GenerateCrc** (FCSMU_Type *const pFcsmu)
Set the fcsmu CRC_CTRL register.

LOCAL_INLINE uint32_t **FCSMU_HWA_GetCrcResult** (const FCSMU_Type *const pFcsmu)
Get the fcsmu CRC_RES register.

Detailed Description

FCSMU module register API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_fcspi.h File Reference

#include "device_header.h"

Include dependency graph for HwA_fcspi.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define FCSPI_SET_BIT(REG, BIT) ((REG) |= (BIT))
#define FCSPI_CLEAR_BIT(REG, BIT) ((REG) &= ~(BIT))
#define FCSPI_READ_BIT(REG, BIT) ((REG) & (BIT))
#define FCSPI_CLEAR_REG(REG) ((REG) = (0x0))
#define FCSPI_WRITE_REG(REG, VAL) ((REG) = (VAL))
#define FCSPI_READ_REG(REG) ((REG))
#define FCSPI_MODIFY_REG(REG, CLEARMASK, SETMASK) FCSPI_WRITE_REG((REG),
(((FCSPI_READ_REG(REG)) & ~(CLEARMASK))) | (SETMASK)))
```

Enumerations

```
enum FCSPI_MasterSlaveModeType { FCSPI_MODE_SLAVE = 0, FCSPI_MODE_MASTER = 1 }
FCSPI Mode type, master or slave.

enum FCSPI_PcsSelType { FCSPI_PCS_0 = 0, FCSPI_PCS_1 = 1, FCSPI_PCS_2 = 2, FCSPI_PCS_3 = 3 }
enum FCSPI_PcsPolarityType { FCSPI_PCS_ACTIVE_LOW = 0, FCSPI_PCS_ACTIVE_HIGH = 1 }
enum FCSPI_PinConfigType { FCSPI_SIN_INPUT_SOUT_OUTPUT = 0, FCSPI_SIN_INPUT_OUTPUT = 1,
FCSPI_SOUT_INPUT_OUTPUT = 2, FCSPI_SOUT_INPUT_SIN_OUTPUT = 3 }
enum FCSPI_OutputConfigType { FCSPI_OUTPUT_RETAIN_LAST = 0, FCSPI_OUTPUT_TRISTATE = 1 }
enum FCSPI_TransferWidthType { FCSPI_TRANSFER_WIDTH_1_BIT = 0,
FCSPI_TRANSFER_WIDTH_2_BIT = 1, FCSPI_TRANSFER_WIDTH_4_BIT = 2 }
enum FCSPI_DataFirstBitType { FCSPI_MSB_FIRST = 0, FCSPI_LSB_FIRST = 1 }
enum FCSPI_PrescaleType { FCSPI_PRESCALE_DIV_1 = 0, FCSPI_PRESCALE_DIV_2 = 1,
FCSPI_PRESCALE_DIV_4 = 2, FCSPI_PRESCALE_DIV_8 = 3, FCSPI_PRESCALE_DIV_16 = 4,
FCSPI_PRESCALE_DIV_32 = 5, FCSPI_PRESCALE_DIV_64 = 6, FCSPI_PRESCALE_DIV_128 = 7 }
enum FCSPI_SckPhaseType { FCSPI_SCK_SAMPLE_FIRST_EDGE = 0,
FCSPI_SCK_SAMPLE_SECOND_EDGE = 1 }
enum FCSPI_SckPolarityType { FCSPI_SCK_IDLE_LOW = 0, FCSPI_SCK_IDLE_HIGH = 1 }
```

Functions

LOCAL_INLINE void **FCSPI_HWA_EnableModule** (FCSPI_Type *pFCSPI, bool enable)

Manipulate CTRL.M_EN to enable/disable FCSPi.

LOCAL_INLINE void **FCSPI_HWA_EnableDebugMode** (FCSPI_Type *pFCSPI, bool enable)

LOCAL_INLINE bool **FCSPI_HWA_IsEnabled** (FCSPI_Type *pFCSPI)

LOCAL_INLINE void **FCSPI_HWA_ResetRxFIFO** (FCSPI_Type *pFCSPI)

LOCAL_INLINE void **FCSPI_HWA_ResetTxFIFO** (FCSPI_Type *pFCSPI)

LOCAL_INLINE void **FCSPI_HWA_EnableInterrupts** (FCSPI_Type *pFCSPI, uint32_t IntsMask, bool enable)

Set the FCSPi INT_EN register value for enable or disable some interrupts.

LOCAL_INLINE bool **FCSPI_HWA_IsEnabledInterrupt** (FCSPI_Type *pFCSPI, uint32_t IntMask)

LOCAL_INLINE void **FCSPI_HWA_EnableRxDMA** (FCSPI_Type *pFCSPI, bool enable)

Enable or disable the DMA feature by setting the FCSPi DMA_EN register value.

LOCAL_INLINE bool **FCSPI_HWA_IsEnabledRxDMA** (FCSPI_Type *pFCSPI)

LOCAL_INLINE void **FCSPI_HWA_EnableTxDMA** (FCSPI_Type *pFCSPI, bool enable)

LOCAL_INLINE bool **FCSPI_HWA_IsEnabledTxDMA** (FCSPI_Type *pFCSPI)

LOCAL_INLINE void **FCSPI_HWA_SoftwareReset** (FCSPI_Type *pFCSPI)

Software reset FCSPi. This operation can be done regardless of whether the module is enabled or not(CTRL.M_EN).

LOCAL_INLINE void **FCSPI_HWA_SetMasterMode** (FCSPI_Type *pFCSPI)

Set the FCSPi to master mode.

LOCAL_INLINE void **FCSPI_HWA_SetSlaveMode** (FCSPI_Type *pFCSPI)

Set FCSPi to slave mode.

LOCAL_INLINE **FCSPI_MasterSlaveModeType** **FCSPI_HWA_GetMode** (FCSPI_Type *pFCSPI)

Check the current mode status, master or slave.

LOCAL_INLINE void **FCSPI_HWA_ClearStatus** (FCSPI_Type *pFCSPI, uint32_t u32StatusMask)

LOCAL_INLINE uint32_t **FCSPI_HWA_GetStatus** (FCSPI_Type *pFCSPI, uint32_t u32FlagBitMask)

Clear FCSPi STATUS register for certain function.

LOCAL_INLINE void **FCSPI_HWA_EnableSckLoopback** (FCSPI_Type *pFCSPI, bool enable)
 LOCAL_INLINE void **FCSPI_HWA_EnableInternalPcs** (FCSPI_Type *pFCSPI, bool enable)
 LOCAL_INLINE void **FCSPI_HWA_SetPcsPolarity** (FCSPI_Type *pFCSPI, **FCSPI_PcsSelType** PCSn, **FCSPI_PcsPolarityType** PcsPolarity)
 LOCAL_INLINE void **FCSPI_HWA_SetPinConfig** (FCSPI_Type *pFCSPI, **FCSPI_PinConfigType** PinCfg)
 LOCAL_INLINE void **FCSPI_HWA_SetOutputConfig** (FCSPI_Type *pFCSPI, **FCSPI_OutputConfigType** OutputCfg)
 LOCAL_INLINE void **FCSPI_HWA_SetPcsConfig** (FCSPI_Type *pFCSPI, bool Is4bitMode)
 LOCAL_INLINE void **FCSPI_HWA_SetSCKDIV** (FCSPI_Type *pFCSPI, uint8 u8Value)
 LOCAL_INLINE void **FCSPI_HWA_SetDelayPCSPCS** (FCSPI_Type *pFCSPI, uint8 u8Value)
 LOCAL_INLINE void **FCSPI_HWA_SetDelayPCSSCK** (FCSPI_Type *pFCSPI, uint8 u8Value)
 LOCAL_INLINE void **FCSPI_HWA_SetDelaySCKPCS** (FCSPI_Type *pFCSPI, uint8 u8Value)
 LOCAL_INLINE void **FCSPI_HWA_SetWatermarkTx** (FCSPI_Type *pFCSPI, uint8 u8Value)
 LOCAL_INLINE void **FCSPI_HWA_SetWatermarkRx** (FCSPI_Type *pFCSPI, uint8 u8Value)
 LOCAL_INLINE uint32 **FCSPI_HWA_GetTxFIFOCnt** (FCSPI_Type *pFCSPI)
 LOCAL_INLINE uint32 **FCSPI_HWA_GetRxFIFOCnt** (FCSPI_Type *pFCSPI)
 LOCAL_INLINE void **FCSPI_HWA_SetFrameSize** (FCSPI_Type *pFCSPI, uint16 FrameSize)
 LOCAL_INLINE void **FCSPI_HWA_SetTransferWidth** (FCSPI_Type *pFCSPI, **FCSPI_TransferWidthType** Width)
 LOCAL_INLINE void **FCSPI_HWA_EnableTxMask** (FCSPI_Type *pFCSPI, bool enable)
 LOCAL_INLINE void **FCSPI_HWA_EnableRxMask** (FCSPI_Type *pFCSPI, bool enable)
 LOCAL_INLINE void **FCSPI_HWA_EnableCTGO** (FCSPI_Type *pFCSPI, bool enable)
 LOCAL_INLINE void **FCSPI_HWA_EnableCTEN** (FCSPI_Type *pFCSPI, bool enable)
 LOCAL_INLINE void **FCSPI_HWA_EnableByteSwap** (FCSPI_Type *pFCSPI, bool enable)
 LOCAL_INLINE void **FCSPI_HWA_SetDataFirstBit** (FCSPI_Type *pFCSPI, **FCSPI_DataFirstBitType** eFirstBit)
 LOCAL_INLINE void **FCSPI_HWA_SelectPCS** (FCSPI_Type *pFCSPI, **FCSPI_PcsSelType** PCSn)
 LOCAL_INLINE void **FCSPI_HWA_SetPrescale** (FCSPI_Type *pFCSPI, **FCSPI_PrescaleType** PrescaleValue)
 LOCAL_INLINE void **FCSPI_HWA_SetSckPhase** (FCSPI_Type *pFCSPI, **FCSPI_SckPhaseType** SckPhase)
 LOCAL_INLINE void **FCSPI_HWA_SetSckPolarity** (FCSPI_Type *pFCSPI, **FCSPI_SckPolarityType** SckPolarity)
 LOCAL_INLINE void **FCSPI_HWA_WriteTxData** (FCSPI_Type *pFCSPI, uint32 TxData)
 LOCAL_INLINE bool **FCSPI_HWA_GetFirstDataFlag** (FCSPI_Type *pFCSPI)
 LOCAL_INLINE bool **FCSPI_HWA_IsRxFIFOEmpty** (FCSPI_Type *pFCSPI)
 LOCAL_INLINE uint32 **FCSPI_HWA_ReadRxData** (FCSPI_Type *pFCSPI)

Macro Definition Documentation

```
#define FCSPI_CLEAR_BIT( REG, BIT) ((REG) &= ~(BIT))
```

```
#define FCSPI_CLEAR_REG( REG) ((REG) = (0x0))
```

```
#define FCSPI_MODIFY_REG( REG, CLEARMASK, SETMASK) FCSPI_WRITE_REG((REG),  
(((FCSPI_READ_REG(REG)) & ~(CLEARMASK))) | (SETMASK)))
```

```
#define FCSPI_READ_BIT( REG, BIT) ((REG) & (BIT))
```

```
#define FCSPI_READ_REG( REG) ((REG))
```

```
#define FCSPI_SET_BIT( REG, BIT) ((REG) |= (BIT))
```

```
#define FCSPI_WRITE_REG( REG, VAL) ((REG) = (VAL))
```

Enumeration Type Documentation

enum FCSPI_DataFirstBitType

Enumerator:

FCSPI_MSB_FIR ST	most significant bit first handle, from msb to lsb
FCSPI_LSB_FIRS T	least significant bit first handle, from lsb to msb

enum FCSPI_MasterSlaveModeType

FCSPI Mode type, master or slave.

Enumerator:

FCSPI_MODE_S LAVE	
FCSPI_MODE_M ASTER	

enum FCSPI_OutputConfigType

Enumerator:

FCSPI_OUTPUT _RETAIN_LAST	SIN is configured as input pin and SOUT is configured as output pin
FCSPI_OUTPUT _TRISTATE	

enum FCSPI_PcsPolarityType

Enumerator:

FCSPI_PCS_ACT IVE_LOW	pcs use low level to select external device
FCSPI_PCS_ACT IVE_HIGH	pcs use high level to select external device

enum FCSPI_PcsSelType

Enumerator:

FCSPI_PCS_0	Transfer using pcs_0
FCSPI_PCS_1	Transfer using pcs_1
FCSPI_PCS_2	Transfer using pcs_2
FCSPI_PCS_3	Transfer using pcs_3

enum FCSPI_PinConfigType

Enumerator:

FCSPI_SIN_INP UT_SOUT_OUT PUT	SIN is configured as input pin and SOUT is configured as output pin
FCSPI_SIN_INP UT_OUTPUT	SIN is configured as input and output pin
FCSPI_SOUT_IN PUT_OUTPUT	SOUT is configured as input and output pin
FCSPI_SOUT_IN PUT_SIN_OUTP UT	SOUT is configured as input pin and SIN is configured as output pin

enum FCSPi_PrescaleType**Enumerator:**

FCSPI_PRESCAL E_DIV_1	most significant bit first handle, from msb to lsb
FCSPI_PRESCAL E_DIV_2	
FCSPI_PRESCAL E_DIV_4	
FCSPI_PRESCAL E_DIV_8	
FCSPI_PRESCAL E_DIV_16	
FCSPI_PRESCAL E_DIV_32	
FCSPI_PRESCAL E_DIV_64	
FCSPI_PRESCAL E_DIV_128	

enum FCSPi_SckPhaseType

Enumerator:

FCSPI_SCK_SAMPLE_FIRST_EDGE	sample on first edge of sck active polarity, change on second
FCSPI_SCK_SAMPLE_SECOND_EDGE	sample on second edge of sck active polarity, change on first

enum FCSPISckPolarityType**Enumerator:**

FCSPI_SCK_IDLE_LOW	sck is high level when active (idles low).
FCSPI_SCK_IDLE_HIGH	sck is low level when active (idles high).

enum FCSPITransferWidthType**Enumerator:**

FCSPI_TRANSFER_WIDTH_1_BIT	1-bit shift at a time, data out on SDO, in on SDI (normal mode)
FCSPI_TRANSFER_WIDTH_2_BIT	2-bits shift out on SDO/SDI and in on SDO/SDI
FCSPI_TRANSFER_WIDTH_4_BIT	4-bits shift out on SDO/SDI/PCS[3:2] and in on SDO/SDI/PCS[3:2]

Function Documentation

LOCAL_INLINE void FCSPi_HWA_ClearStatus (FCSPi_Type * *pFCSPi*, uint32_t *u32StatusMask*)

LOCAL_INLINE void FCSPi_HWA_EnableByteSwap (FCSPi_Type * *pFCSPi*, bool *enable*)

LOCAL_INLINE void FCSPi_HWA_EnableCTEN (FCSPi_Type * *pFCSPi*, bool *enable*)

LOCAL_INLINE void FCSPi_HWA_EnableCTGO (FCSPi_Type * *pFCSPi*, bool *enable*)

LOCAL_INLINE void FCSPi_HWA_EnableDebugMode (FCSPi_Type * *pFCSPi*, bool *enable*)

LOCAL_INLINE void FCSPi_HWA_EnableInternalPcs (FCSPi_Type * *pFCSPi*, bool *enable*)

LOCAL_INLINE void FCSPi_HWA_EnableInterrupts (FCSPi_Type * *pFCSPi*, uint32_t *IntsMask*, bool *enable*)

Set the FCSPi INT_EN register value for enable or disable some interrupts.

Parameters

<i>pFCSPi</i>	FCSPi instance, e.g. FCSPi0, FCSPi1.
<i>u32Value</i>	the value write to the register.

LOCAL_INLINE void FCSPi_HWA_EnableModule (FCSPi_Type * *pFCSPi*, bool *enable*)

Manipulate CTRL.M_EN to enable/disable FCSPi.

Parameters

<i>pFCSPi</i>	FCSPi instance, e.g. FCSPi0, FCSPi1.
---------------	--------------------------------------

LOCAL_INLINE void FCSPi_HWA_EnableRxDMA (FCSPi_Type * *pFCSPi*, bool *enable*)

Enable or disable the DMA feature by setting the FCSPi DMA_EN register value.

Parameters

<i>pFCSPi</i>	FCSPi instance, e.g. FCSPi0, FCSPi1.
---------------	--------------------------------------

<i>u32Value</i>	the value write to the register.
-----------------	----------------------------------

LOCAL_INLINE void FCSPI_HWA_EnableRxMask (FCSPI_Type * *pFCSPI*, bool *enable*)

LOCAL_INLINE void FCSPI_HWA_EnableSckLoopback (FCSPI_Type * *pFCSPI*, bool *enable*)

LOCAL_INLINE void FCSPI_HWA_EnableTxDMA (FCSPI_Type * *pFCSPI*, bool *enable*)

LOCAL_INLINE void FCSPI_HWA_EnableTxMask (FCSPI_Type * *pFCSPI*, bool *enable*)

LOCAL_INLINE bool FCSPI_HWA_GetFirstDataFlag (FCSPI_Type * *pFCSPI*)

LOCAL_INLINE FCSPI_MasterSlaveModeType FCSPI_HWA_GetMode (FCSPI_Type * *pFCSPI*)

Check the current mode status, master or slave.

Parameters

<i>pFCSPI</i>	FCSPI instance, e.g. FCSPI0, FCSPI1.
---------------	--------------------------------------

Returns

the mode type of FCSPI, FCSPI_MODE_MASTER or FCSPI_MODE_SLAVE

LOCAL_INLINE uint32 FCSPI_HWA_GetRxFIFOCnt (FCSPI_Type * *pFCSPI*)

LOCAL_INLINE uint32_t FCSPI_HWA_GetStatus (FCSPI_Type * *pFCSPI*, uint32_t *u32FlagBitMask*)

Clear FCSPI STATUS register for certain function.

Parameters

<i>pFCSPI</i>	FCSPI instance, e.g. FCSPI0, FCSPI1.
<i>u32Value</i>	the value write to the register.

```

LOCAL_INLINE uint32 FCSPI_HWA_GetTxFIFOCnt (FCSPI_Type * pFCSPI)

LOCAL_INLINE bool FCSPI_HWA_IsEnabled (FCSPI_Type * pFCSPI)

LOCAL_INLINE bool FCSPI_HWA_IsEnabledInterrupt (FCSPI_Type * pFCSPI, uint32_t IntMask)

LOCAL_INLINE bool FCSPI_HWA_IsEnabledRxDMA (FCSPI_Type * pFCSPI)

LOCAL_INLINE bool FCSPI_HWA_IsEnabledTxDMA (FCSPI_Type * pFCSPI)

LOCAL_INLINE bool FCSPI_HWA_IsRxFIFOEmpty (FCSPI_Type * pFCSPI)

LOCAL_INLINE uint32 FCSPI_HWA_ReadRxData (FCSPI_Type * pFCSPI)

LOCAL_INLINE void FCSPI_HWA_ResetRxFIFO (FCSPI_Type * pFCSPI)

LOCAL_INLINE void FCSPI_HWA_ResetTxFIFO (FCSPI_Type * pFCSPI)

LOCAL_INLINE void FCSPI_HWA_SelectPCS (FCSPI_Type * pFCSPI, FCSPI_PcsSelType PCSn)

LOCAL_INLINE void FCSPI_HWA_SetDataFirstBit (FCSPI_Type * pFCSPI, FCSPI_DataFirstBitType eFirstBit)

LOCAL_INLINE void FCSPI_HWA_SetDelayPCSPCS (FCSPI_Type * pFCSPI, uint8 u8Value)

LOCAL_INLINE void FCSPI_HWA_SetDelayPCSSCK (FCSPI_Type * pFCSPI, uint8 u8Value)

LOCAL_INLINE void FCSPI_HWA_SetDelaySCKPCS (FCSPI_Type * pFCSPI, uint8 u8Value)

LOCAL_INLINE void FCSPI_HWA_SetFrameSize (FCSPI_Type * pFCSPI, uint16 FrameSize)

LOCAL_INLINE void FCSPI_HWA_SetMasterMode (FCSPI_Type * pFCSPI)

```

Set the FCSPI to master mode.

Parameters

<i>pFCSPI</i>	FCSPI instance, e.g. FCSPI0, FCSPI1.
---------------	--------------------------------------

LOCAL_INLINE void FCSPI_HWA_SetOutputConfig (FCSPI_Type * *pFCSPI*, FCSPI_OutputConfigType *OutputCfg*)

LOCAL_INLINE void FCSPI_HWA_SetPcsConfig (FCSPI_Type * *pFCSPI*, bool *Is4bitMode*)

LOCAL_INLINE void FCSPI_HWA_SetPcsPolarity (FCSPI_Type * *pFCSPI*, FCSPI_PcsSelType *PCSn*, FCSPI_PcsPolarityType *PcsPolarity*)

LOCAL_INLINE void FCSPI_HWA_SetPinConfig (FCSPI_Type * *pFCSPI*, FCSPI_PinConfigType *PinCfg*)

LOCAL_INLINE void FCSPI_HWA_SetPrescale (FCSPI_Type * *pFCSPI*, FCSPI_PrescaleType *PrescaleValue*)

LOCAL_INLINE void FCSPI_HWA_SetSCKDIV (FCSPI_Type * *pFCSPI*, uint8 *u8Value*)

LOCAL_INLINE void FCSPI_HWA_SetSckPhase (FCSPI_Type * *pFCSPI*, FCSPI_SckPhaseType *SckPhase*)

LOCAL_INLINE void FCSPI_HWA_SetSckPolarity (FCSPI_Type * *pFCSPI*, FCSPI_SckPolarityType *SckPolarity*)

LOCAL_INLINE void FCSPI_HWA_SetSlaveMode (FCSPI_Type * *pFCSPI*)

Set FCSPI to slave mode.

Parameters

<i>pFCSPI</i>	FCSPI instance, e.g. FCSPI0, FCSPI1.
---------------	--------------------------------------

LOCAL_INLINE void FCSPI_HWA_SetTransferWidth (FCSPI_Type * *pFCSPI*, FCSPI_TransferWidthType *Width*)

LOCAL_INLINE void FCSPI_HWA_SetWatermarkRx (FCSPI_Type * *pFCSPI*, uint8 *u8Value*)

LOCAL_INLINE void FCSPI_HWA_SetWatermarkTx (FCSPI_Type * *pFCSPI*, uint8 *u8Value*)

LOCAL_INLINE void FCSPI_HWA_SoftwareReset (FCSPI_Type * *pFCSPI*)

Software reset FCSPI. This operation can be done regardless of whether the module is enabled or not(CTRL.M_EN).

Parameters

<i>pFCSPI</i>	FCSPI instance, e.g. FCSPI0, FCSPI1.
---------------	--------------------------------------

Here is the call graph for this function:

IMAGE

```
LOCAL_INLINE void FCSPI_HWA_WriteTxData (FCSPI_Type * pFCSPI, uint32 TxData)
```

HwA_fcuart.h File Reference

FCUart register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_fcuart.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum FCUART_StatType { FCUART_STAT_LBKDIF = FCUART_STAT_LBKDIF_MASK, FCUART_STAT_RPAEIF = FCUART_STAT_RPAEIF_MASK, FCUART_STAT_MSBF = FCUART_STAT_MSBF_MASK, FCUART_STAT_RXINV = FCUART_STAT_RXINV_MASK, FCUART_STAT_RWUID = FCUART_STAT_RWUID_MASK, FCUART_STAT_BCGL = FCUART_STAT_BCGL_MASK, FCUART_STAT_LBKDE = FCUART_STAT_LBKDE_MASK, FCUART_STAT_RAF = FCUART_STAT_RAF_MASK, FCUART_STAT_TDREF = FCUART_STAT_TDREF_MASK, FCUART_STAT_TCF = FCUART_STAT_TCF_MASK, FCUART_STAT_RDRFF = FCUART_STAT_RDRFF_MASK, FCUART_STAT_IDLEF = FCUART_STAT_IDLEF_MASK, FCUART_STAT_RORF = FCUART_STAT_RORF_MASK, FCUART_STAT_NF = FCUART_STAT_NF_MASK, FCUART_STAT_FEF = FCUART_STAT_FEF_MASK, FCUART_STAT_PEF = FCUART_STAT_PEF_MASK, FCUART_STAT_M0F = FCUART_STAT_M0F_MASK, FCUART_STAT_M1F = FCUART_STAT_M1F_MASK, FCUART_STAT_RPEF = FCUART_STAT_RPEF_MASK, FCUART_STAT_TPEF = FCUART_STAT_TPEF_MASK }
```

UART STAT register flag.

```
enum FCUART_BitModeType { UART_BITMODE_8 = 0, UART_BITMODE_9 }
```

UART data bit length mode.

```
enum FCUART_StopBitNumType { UART_STOPBIT_NUM_1 = 0, UART_STOPBIT_NUM_2 }
```

UART stop bits number.

```
enum FCUART_ParityType { UART_PARITY_EVEN = 0, UART_PARITY_ODD }
```

UART parity check type.

Functions

```
LOCAL_INLINE void FCUART_HWA_SetSoftWareReset (FCUART_Type *pUart)
```

Get Stat Flag.

LOCAL_INLINE uint32_t **FCUART_HWA_GetStatus** (FCUART_Type *pUart, **FCUART_StatType** eStatusType)
Get Stat Flag.

LOCAL_INLINE void **FCUART_HWA_ClearStatus** (FCUART_Type *pUart, uint32_t u32StatusType)
Clear Stat Flag.

LOCAL_INLINE void **FCUART_HWA_EnableInterrupt** (FCUART_Type *pUart, uint32_t u32Value)
Enable Interrupt.

LOCAL_INLINE void **FCUART_HWA_DisableInterrupt** (FCUART_Type *pUart, uint32_t u32Value)
Disable Interrupt.

LOCAL_INLINE void **FCUART_HWA_EnableReceiveInterrupt** (FCUART_Type *pUart)
Enable Receive Interrupt.

LOCAL_INLINE void **FCUART_HWA_DisableReceiveInterrupt** (FCUART_Type *pUart)
Disable Receive Interrupt.

LOCAL_INLINE void **FCUART_HWA_EnableErrorInterrupt** (FCUART_Type *pUart)
Enable Error Interrupt.

LOCAL_INLINE void **FCUART_HWA_DisableErrorInterrupt** (FCUART_Type *pUart)
Disable Error Interrupt.

LOCAL_INLINE void **FCUART_HWA_SetCtrl** (FCUART_Type *pUart, uint32_t u32Value)
Set FCUART Ctrl register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetCtrl** (FCUART_Type *pUart)
Get FCUART Ctrl register.

LOCAL_INLINE void **FCUART_HWA_AttachCtrl** (FCUART_Type *pUart, uint32_t u32Value)
Attach FCUART Ctrl register.

LOCAL_INLINE void **FCUART_HWA_SetBaud** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART Baud register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetBaud** (FCUART_Type *pUart)

Get FCUART baud register.

LOCAL_INLINE void **FCUART_HWA_AttachBaud** (FCUART_Type *pUart, uint32_t u32Value)

Attach FCUART Baud register.

LOCAL_INLINE void **FCUART_HWA_SetFifo** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART Fifo register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetFifo** (FCUART_Type *pUart)

Get FCUART fifo register.

LOCAL_INLINE void **FCUART_HWA_FlushTxRxFifo** (FCUART_Type *pUart)

Flush FCUART Tx Rx Fifo register.

LOCAL_INLINE bool **FCUART_HWA_GetEnStatusRxFifo** (FCUART_Type *pUart)

Get FCUART fifo register.

LOCAL_INLINE bool **FCUART_HWA_GetEnStatusTxFifo** (FCUART_Type *pUart)

Get FCUART fifo register.

LOCAL_INLINE void **FCUART_HWA_AttachFifo** (FCUART_Type *pUart, uint32_t u32Value)

Attach FCUART Fifo register.

LOCAL_INLINE void **FCUART_HWA_SetWaterMark** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART WaterMark register.

LOCAL_INLINE uint8_t **FCUART_HWA_GetRxWaterMark** (FCUART_Type *pUart)

Get FCUART Rx WaterMark.

LOCAL_INLINE uint8_t **FCUART_HWA_GetTxWaterMark** (FCUART_Type *pUart)

Get FCUART Tx WaterMark.

LOCAL_INLINE uint8_t **FCUART_HWA_GetFifoRxCount** (FCUART_Type *pUart)

Get FCUART WaterMark Rxcount.

LOCAL_INLINE uint8_t **FCUART_HWA_GetFifoTxCount** (FCUART_Type *pUart)

Get FCUART WaterMark Txcount.

LOCAL_INLINE void **FCUART_HWA_AttachWaterMark** (FCUART_Type *pUart, uint32_t u32Value)

Attach FCUART WaterMark register.

LOCAL_INLINE void **FCUART_HWA_SetMatch** (FCUART_Type *pUart, uint32_t u32Value)

Set FCUART Match register.

LOCAL_INLINE void **FCUART_HWA_AttachMatch** (FCUART_Type *pUart, uint32_t u32Value)

Attach FCUART Match register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetMatch** (FCUART_Type *pUart)

Get FCUART Match register.

LOCAL_INLINE uint32_t **FCUART_HWA_GetSTAT** (FCUART_Type *pUart)

Read FCUART STAT register.

LOCAL_INLINE void **FCUART_HWA_WriteClearSTAT** (FCUART_Type *pUart, uint32_t u32Value)

Write 1 Clear FCUART STAT register.

LOCAL_INLINE void **FCUART_HWA_SetBitMode** (FCUART_Type *pUart, **FCUART_BitModeType** eBitMode)

Set Bit Mode and Parity.

LOCAL_INLINE void **FCUART_HWA_SetParity** (FCUART_Type *pUart, **FCUART_ParityType** eParityType, bool bParityEnable)

Set Bit Mode and Parity.

LOCAL_INLINE void **FCUART_HWA_SetStopBit** (FCUART_Type *pUart, **FCUART_StopBitNumType** eStopBit)

Set Bit Mode and Parity.

LOCAL_INLINE void **FCUART_HWA_EnableReceiveDMA** (FCUART_Type *pUart)

Enable Receive DMA.

LOCAL_INLINE void **FCUART_HWA_DisableReceiveDMA** (FCUART_Type *pUart)

Disable Receive DMA.

LOCAL_INLINE void **FCUART_HWA_EnableReceiveFIFO** (FCUART_Type *pUart)

Enable Receive FIFO.

LOCAL_INLINE void **FCUART_HWA_DisableReceiveFIFO** (FCUART_Type *pUart)

Disable Receive FIFO.

LOCAL_INLINE void **FCUART_HWA_ClearFIFOErrorFlag** (FCUART_Type *pUart)

Clear Fifo Overflow/Underflow flag.

LOCAL_INLINE void **FCUART_HWA_SetTxWaterMark** (FCUART_Type *pUart, uint8_t u8Value)

Set FCUART Tx WaterMark.

LOCAL_INLINE void **FCUART_HWA_SetRxWaterMark** (FCUART_Type *pUart, uint8_t u8Value)

Set FCUART Rx WaterMark.

LOCAL_INLINE void **FCUART_HWA_SetData** (FCUART_Type *pUart, uint32_t u32Data)

Set Data Value.

LOCAL_INLINE uint8_t **FCUART_HWA_GetData** (FCUART_Type *pUart)

Get Data Value.

LOCAL_INLINE void **FCUART_HWA_SetR8T9** (FCUART_Type *pUart, uint8_t u8Data)

Set R8T9 bit.

LOCAL_INLINE uint8_t **FCUART_HWA_GetR8T9** (FCUART_Type *pUart)

Get R8T9 bit.

LOCAL_INLINE void **FCUART_HWA_SetR9T8** (FCUART_Type *pUart, uint8_t u8Data)

Set R9T8 bit.

LOCAL_INLINE uint8_t **FCUART_HWA_GetR9T8** (FCUART_Type *pUart)

Get R9T8 bit.

LOCAL_INLINE void **FCUART_HWA_SoftwareReset** (FCUART_Type *pUart)

Reset the instance by software.

LOCAL_INLINE void **FCUART_HWA_SetTxTransfer** (FCUART_Type *pUart, bool bEnable)

Set fcuart TX Transfer enable or disable.

LOCAL_INLINE void **FCUART_HWA_StartTransmit** (FCUART_Type *pUart)

Start transmit.

LOCAL_INLINE void **FCUART_HWA_SetRxTransfer** (FCUART_Type *pUart, bool bEnable)

Set fcuart RX Transfer enable or disable.

LOCAL_INLINE void **FCUART_HWA_StartTransfer** (FCUART_Type *pUart)

Set fcuart start transfer.

LOCAL_INLINE void **FCUART_HWA_StopTransfer** (FCUART_Type *pUart)

Set fcuart stop transfer.

LOCAL_INLINE void **FCUART_HWA_SetLinBreakDetectInterrupt** (FCUART_Type *pUart, bool bEnable)

Set lin break detect interrupt.

LOCAL_INLINE void **FCUART_HWA_SetLinBreakDetectEnable** (FCUART_Type *pUart, bool bEnable)

Set lin break detect feature enable.

LOCAL_INLINE void **FCUART_HWA_SendBreakField** (FCUART_Type *pUart)

Send a lin break field.

LOCAL_INLINE void **FCUART_HWA_SetReceiveActiveInterrupt** (FCUART_Type *pUart, bool bEnable)

Set uart receive active interrupt.

LOCAL_INLINE bool **FCUART_HWA_GetReceiveActiveInterrupt** (FCUART_Type *pUart)

Set uart receive active interrupt.

LOCAL_INLINE void **FCUART_HWA_SetReceiveDataInverse** (FCUART_Type *pUart, bool bEnable)

Set FCUART inverse feature.

LOCAL_INLINE bool **FCUART_HWA_GetReceiveDataInverse** (FCUART_Type *pUart)

Get the FCUART inverse bit value.

LOCAL_INLINE void **FCUART_HWA_SetFrameErrorInterrupt** (FCUART_Type *pUart, bool bEnable)

Set the FCUART frame error interrupt.

LOCAL_INLINE void **FCUART_HWA_EnableLoopMode** (FCUART_Type *pUart)

Enable the FCUART loop mode.

LOCAL_INLINE void **FCUART_HWA_DisableLoopMode** (FCUART_Type *pUart)

Disable the FCUART loop mode.

LOCAL_INLINE void **FCUART_HWA_SetModir** (FCUART_Type *pUart, uint32_t u32Data)

Set FCUART MODIR value.

LOCAL_INLINE uint32_t **FCUART_HWA_GetModir** (FCUART_Type *pUart)

Get FCUART MODIR value.

Detailed Description

FCUart register API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_flexcan.h File Reference

FLEXCAN hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_flexcan.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define FLEXCAN_REGISTER_WITHOFFSET2(reg, offset1) ((uint32_t)(reg)+(uint32_t)(offset1))
```

```
#define FLEXCAN_REGISTER_WITHOFFSET(reg, offset1,  
offset2) ((uint32_t)(reg)+(uint32_t)(offset1)+(uint32_t)(offset2))
```

```
#define FLEXCAN_REG32_CONTENT(reg) (*((uint32_t*)(reg)))
```

```
#define FLEXCAN_U32MACRO(var) ((uint32_t)(var))
```

U32 value around.

```
#define FLEXCAN_U32SHIFT(var, mask, shift) ((uint32_t)((((uint32_t)(var))&(mask))<<(shift)))
```

U32 value shift.

```
#define FLEXCAN_U32NOTSHIFT(mask, shift) ((uint32_t)(~((((uint32_t)(mask))<<(shift))))
```

U32 value shift and revert.

```
#define FLEXCAN_MB_SRR_MASK_U32 0x00400000U
```

```
#define FLEXCAN_ONE_REGION_SIZE 0x200U
```

```
#define FLEXCAN_MB_HEADER_BYTELEN 8UL
```

Legacy/Enhanced Message Buffer Header.

```
#define FLEXCAN_MB_ADDR(mbOffset, mbIndex, dataLen)
```

Legacy Message Buffer Address.

```
#define FLEXCAN_MB_WORDN_ADDR(mbOffset, mbIndex, dataLen,  
offset) (FLEXCAN_MB_ADDR((mbOffset), (mbIndex), (dataLen))+FLEXCAN_U32MACRO(offset))  
Legacy Message Buffer word n Address.
```

```
#define FLEXCAN_MB_GET_DATALEN(mbindex, mbNum, datalenNum,  
datalenarray) ((uint8_t)(datalenarray[((mbindex)*(datalenNum)/(mbNum)) ]]))  
check mb data length
```

```
#define FLEXCAN_MB_EDL_GET(edlAddr) ((FLEXCAN_REG32_CONTENT(edlAddr)>>31)&0x01U)  
Legacy Message Buffer Data Address.
```

```
#define FLEXCAN_MB_EDL_CLR(edlAddr) (FLEXCAN_REG32_CONTENT(edlAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,31))  
Legacy Message Buffer EDL Clear.
```

```
#define FLEXCAN_MB_EDL_ATTACH(edlAddr, var) (FLEXCAN_REG32_CONTENT(edlAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,31))  
Legacy Message Buffer EDL Attach.
```

```
#define FLEXCAN_MB_EDL_SET(edlAddr, var) FLEXCAN_REG32_CONTENT(edlAddr) =  
((FLEXCAN_REG32_CONTENT(edlAddr) & FLEXCAN_U32NOTSHIFT(0x01U,31)) |  
FLEXCAN_U32SHIFT(var,0x01U,31))  
Legacy Message Buffer EDL Set.
```

```
#define FLEXCAN_MB_BRS_GET(brsAddr) (((FLEXCAN_REG32_CONTENT(brsAddr))>>30)&0x01U)  
Legacy Message Buffer BRS Get.
```

```
#define FLEXCAN_MB_BRS_CLR(brsAddr) (FLEXCAN_REG32_CONTENT(brsAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,30))  
Legacy Message Buffer BRS Clear.
```

```
#define FLEXCAN_MB_BRS_ATTACH(brsAddr, var) (FLEXCAN_REG32_CONTENT(brsAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,30))  
Legacy Message Buffer BRS Attach.
```

```
#define FLEXCAN_MB_BRS_SET(brsAddr, var) FLEXCAN_REG32_CONTENT(brsAddr) =  
((FLEXCAN_REG32_CONTENT(brsAddr) & FLEXCAN_U32NOTSHIFT(0x01U,30)) |  
FLEXCAN_U32SHIFT(var,0x01U,30))
```

Legacy Message Buffer BRS set.

```
#define FLEXCAN_MB_ESI_GET(esiAddr) (((FLEXCAN_REG32_CONTENT(esiAddr))>>29)&0x01U)
```

Legacy Message Buffer ESI Get.

```
#define FLEXCAN_MB_ESI_CLR(esiAddr) (FLEXCAN_REG32_CONTENT(esiAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,29))
```

Legacy Message Buffer ESI Clear.

```
#define FLEXCAN_MB_ESI_ATTACH(esiAddr, var) (FLEXCAN_REG32_CONTENT(esiAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,29))
```

Legacy Message Buffer ESI Attach.

```
#define FLEXCAN_MB_ESI_SET(esiAddr, var) FLEXCAN_REG32_CONTENT(esiAddr) =  
((FLEXCAN_REG32_CONTENT(esiAddr) & FLEXCAN_U32NOTSHIFT(0x01U,29)) |  
FLEXCAN_U32SHIFT(var,0x01U,29))
```

Legacy Message Buffer ESI Set.

```
#define FLEXCAN_MB_CODE_GET(codeAddr) (((FLEXCAN_REG32_CONTENT(codeAddr))>>24)&0x0EU)
```

Legacy Message Buffer CODE Get.

```
#define FLEXCAN_MB_CODE_CLR(codeAddr) (FLEXCAN_REG32_CONTENT(codeAddr) &=  
FLEXCAN_U32NOTSHIFT(0x0FU,24))
```

Legacy Message Buffer CODE Clear.

```
#define FLEXCAN_MB_CODE_ATTACH(codeAddr, var) (FLEXCAN_REG32_CONTENT(codeAddr) |=  
FLEXCAN_U32SHIFT(var,0x0FU,24))
```

Legacy Message Buffer CODE Attach.

```
#define FLEXCAN_MB_CODE_SET(codeAddr, var) FLEXCAN_REG32_CONTENT(codeAddr) =  
((FLEXCAN_REG32_CONTENT(codeAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,24)) |  
FLEXCAN_U32SHIFT(var,0x0FU,24))
```

Legacy Message Buffer CODE set.

```
#define FLEXCAN_MB_SRR_GET(srrAddr) (((FLEXCAN_REG32_CONTENT(srrAddr)) >>22)&0x01U)
```

Legacy Message Buffer SRR Get.

```
#define FLEXCAN_MB_SRR_CLR(srrAddr) (FLEXCAN_REG32_CONTENT(srrAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,22))
```

Legacy Message Buffer SRR Clear.

```
#define FLEXCAN_MB_SRR_ATTACH(srrAddr, var) (FLEXCAN_REG32_CONTENT(srrAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,22))
```

Legacy Message Buffer SRR Attach.

```
#define FLEXCAN_MB_SRR_SET(srrAddr, var) FLEXCAN_REG32_CONTENT(srrAddr) =  
((FLEXCAN_REG32_CONTENT(srrAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,22)) |  
FLEXCAN_U32SHIFT(var,0x0FU,22))
```

Legacy Message Buffer SRR set.

```
#define FLEXCAN_MB_IDE_GET(ideAddr) (((FLEXCAN_REG32_CONTENT(ideAddr))>>21)&0x01U)
```

Legacy Message Buffer IDE Get.

```
#define FLEXCAN_MB_IDE_CLR(ideAddr) (FLEXCAN_REG32_CONTENT(ideAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,21))
```

Legacy Message Buffer IDE Clear.

```
#define FLEXCAN_MB_IDE_ATTACH(ideAddr, var) (FLEXCAN_REG32_CONTENT(ideAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,21))
```

Legacy Message Buffer IDE Attach.

```
#define FLEXCAN_MB_IDE_SRR_SET(ideAddr, var) FLEXCAN_REG32_CONTENT(ideAddr) =  
((FLEXCAN_REG32_CONTENT(ideAddr) & FLEXCAN_U32NOTSHIFT(0x03U,21)) |  
FLEXCAN_U32SHIFT(var,0x03U,21))
```


Legacy Message Buffer IDE set with SRR.

```
#define FLEXCAN_MB_RTR_GET(rtrAddr) (((FLEXCAN_REG32_CONTENT(rtrAddr))>>20)&0x01U)
```

Legacy Message Buffer RTR Get.

```
#define FLEXCAN_MB_RTR_CLR(rtrAddr) (FLEXCAN_REG32_CONTENT(rtrAddr) &=  
FLEXCAN_U32NOTSHIFT(0x01U,20))
```

Legacy Message Buffer RTR Clear.

```
#define FLEXCAN_MB_RTR_ATTACH(rtrAddr, var) (FLEXCAN_REG32_CONTENT(rtrAddr) |=  
FLEXCAN_U32SHIFT(var,0x01U,20))
```

Legacy Message Buffer RTR Attach.

```
#define FLEXCAN_MB_RTR_SET(rtrAddr, var) FLEXCAN_REG32_CONTENT(rtrAddr) =  
((FLEXCAN_REG32_CONTENT(rtrAddr) & FLEXCAN_U32NOTSHIFT(0x01U,20)) |  
FLEXCAN_U32SHIFT(var,0x01U,20))
```

Legacy Message Buffer RTR set.

```
#define FLEXCAN_MB_DLC_GET(dlcAddr) (((FLEXCAN_REG32_CONTENT(dlcAddr))>>16)&0x0FU)
```

Legacy Message Buffer DLC Get.

```
#define FLEXCAN_MB_DLC_CLR(dlcAddr) (FLEXCAN_REG32_CONTENT(dlcAddr) &=  
FLEXCAN_U32NOTSHIFT(0x0FU,16))
```

Legacy Message Buffer DLC Clear.

```
#define FLEXCAN_MB_DLC_ATTACH(dlcAddr, var) (FLEXCAN_REG32_CONTENT(dlcAddr) |=  
FLEXCAN_U32SHIFT(var,0x0FU,16))
```

Legacy Message Buffer DLC Attach.

```
#define FLEXCAN_MB_DLC_SET(dlcAddr, var) FLEXCAN_REG32_CONTENT(dlcAddr) =  
((FLEXCAN_REG32_CONTENT(dlcAddr) & FLEXCAN_U32NOTSHIFT(0x0FU,16)) |  
FLEXCAN_U32SHIFT(var,0x0FU,16))
```

Legacy Message Buffer DLC Set.

#define

FLEXCAN_MB_TIMESTAMP_GET(timestampAddr) (**FLEXCAN_REG32_CONTENT**(timestampAddr)&0xFFFFU)

Legacy Message Buffer TIME STAMP Get.

#define **FLEXCAN_MB_TIMESTAMP_CLR**(timestampAddr) (**FLEXCAN_REG32_CONTENT**(timestampAddr) &= **FLEXCAN_U32NOTSHIFT**(0xFFFFU,0))

Legacy Message Buffer TIME STAMP Clear.

#define **FLEXCAN_MB_TIMESTAMP_ATTACH**(timestampAddr, var) (**FLEXCAN_REG32_CONTENT**(timestampAddr) |= **FLEXCAN_U32SHIFT**(var,0xFFFFU,0))

Legacy Message Buffer TIME STAMP Attach.

#define **FLEXCAN_MB_TIMESTAMP_SET**(timestampAddr, var) **FLEXCAN_REG32_CONTENT**(timestampAddr) = ((**FLEXCAN_REG32_CONTENT**(timestampAddr) & **FLEXCAN_U32NOTSHIFT**(0xFFFFU,0)) | **FLEXCAN_U32SHIFT**(var,0xFFFFU,0))

Legacy Message Buffer TIME STAMP Set.

#define **FLEXCAN_MB_PRIO_GET**(prioAddr) (((**FLEXCAN_REG32_CONTENT**(prioAddr))>>29)&0x07U)

Legacy Message Buffer PRIO Get.

#define **FLEXCAN_MB_PRIO_CLR**(prioAddr) (**FLEXCAN_REG32_CONTENT**(prioAddr) &= **FLEXCAN_U32NOTSHIFT**(0x07U,29))

Legacy Message Buffer PRIO Clear.

#define **FLEXCAN_MB_PRIO_ATTACH**(prioAddr, var) (**FLEXCAN_REG32_CONTENT**(prioAddr) |= **FLEXCAN_U32SHIFT**(var,0x07U,29))

Legacy Message Buffer PRIO Attach.

#define **FLEXCAN_MB_PRIO_SET**(prioAddr, var) **FLEXCAN_REG32_CONTENT**(prioAddr) = ((**FLEXCAN_REG32_CONTENT**(prioAddr) & **FLEXCAN_U32NOTSHIFT**(0x07U,29)) | **FLEXCAN_U32SHIFT**(var,0x07U,29))

Legacy Message Buffer PRIO Set.

#define **FLEXCAN_MB_STDID_GET**(stdidAddr) (((**FLEXCAN_REG32_CONTENT**(stdidAddr))>>18)&0x7FFU)

Legacy Message Buffer STD ID Get.

```
#define FLEXCAN_MB_STDID_CLR(stdidAddr) (FLEXCAN_REG32_CONTENT(stdidAddr) &=  
FLEXCAN_U32NOTSHIFT(0x7FFU,18))
```

Legacy Message Buffer STD ID CLR.

```
#define FLEXCAN_MB_STDID_ATTACH(stdidAddr, var) (FLEXCAN_REG32_CONTENT(stdidAddr) |=  
FLEXCAN_U32SHIFT(var,0x7FFU,18))
```

Legacy Message Buffer STD ID Attach.

```
#define FLEXCAN_MB_STDID_SET(stdidAddr, var) FLEXCAN_REG32_CONTENT(stdidAddr) =  
((FLEXCAN_REG32_CONTENT(stdidAddr) & FLEXCAN_U32NOTSHIFT(0x7FFU,18)) |  
FLEXCAN_U32SHIFT(var,0x7FFU,18))
```

Legacy Message Buffer STD ID set.

```
#define FLEXCAN_MB_EXTID_GET(extidAddr) ((FLEXCAN_REG32_CONTENT(extidAddr))&0x1FFFFFFF)
```

Legacy Message Buffer EXT ID Get.

```
#define FLEXCAN_MB_EXTID_CLR(extidAddr) (FLEXCAN_REG32_CONTENT(extidAddr) &=  
FLEXCAN_U32NOTSHIFT(0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID Clear.

```
#define FLEXCAN_MB_EXTID_ATTACH(extidAddr, var) (FLEXCAN_REG32_CONTENT(extidAddr) |=  
FLEXCAN_U32SHIFT(var,0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID Attach.

```
#define FLEXCAN_MB_EXTID_SET(extidAddr, var) FLEXCAN_REG32_CONTENT(extidAddr) =  
((FLEXCAN_REG32_CONTENT(extidAddr) & FLEXCAN_U32NOTSHIFT(0x1FFFFFFF,0)) |  
FLEXCAN_U32SHIFT(var,0x1FFFFFFF,0))
```

Legacy Message Buffer EXT ID set.

```
#define FLEXCAN_EFIFOMB_EDL_GET() ((pCan->ERX_FIFO[0U]>>31)&0x01U)
```

Enhanced FIFO Message Buffer EDL Get.

#define FLEXCAN_EFIFOMB_BRS_GET() ((pCan->ERX_FIFO[0U]>>30)&0x01U)

Enhanced FIFO Message Buffer BRS Get.

#define FLEXCAN_EFIFOMB_ESI_GET() ((pCan->ERX_FIFO[0U]>>29)&0x01U)

Enhanced FIFO Message Buffer ESI Get.

#define FLEXCAN_EFIFOMB_SRR_GET() ((pCan->ERX_FIFO[0U]>>22)&0x01U)

Enhanced FIFO Message Buffer SRR Get.

#define FLEXCAN_EFIFOMB_IDE_GET() ((pCan->ERX_FIFO[0U]>>21)&0x01U)

Enhanced FIFO Message Buffer IDE Get.

#define FLEXCAN_EFIFOMB_RTR_GET() ((pCan->ERX_FIFO[0U]>>20)&0x01U)

Enhanced FIFO Message Buffer RTR Get.

#define FLEXCAN_EFIFOMB_DLC_GET() ((pCan->ERX_FIFO[0U]>>16)&0x0FU)

Enhanced FIFO Message Buffer DLC Get.

#define FLEXCAN_EFIFOMB_TIMESTAMP_GET() (pCan->ERX_FIFO[0U]&0xFFU)

Enhanced FIFO Message Buffer TIME STAMP Get.

#define FLEXCAN_EFIFOMB_STDID_GET() ((pCan->ERX_FIFO[1U]>>18)&0x7FFU)

Enhanced FIFO Message Buffer STD ID Get.

#define FLEXCAN_EFIFOMB_EXTID_GET() (pCan->ERX_FIFO[1U]&0x1FFFFFFF)

Enhanced FIFO Message Buffer EXT ID Get.

#define FLEXCAN_EFIFOMB_DATAADDR_GET(wordIndex) (&(pCan->ERX_FIFO[2U+(wordIndex)]))

Enhanced FIFO Message Buffer Data Get.

#define FLEXCAN_EFIFOMB_IDHIT_GET(dataWordLen) ((pCan->ERX_FIFO[2U+(dataWordLen)]&0x1FFU)

Enhanced FIFO Message Buffer IDHIT Get.

```

#define FLEXCAN_EFIFOMB_HRTIMESTAMP_GET(dataWordLen) (pCan->ERX_FIFO[3U+(dataWordLen)])
Enhanced FIFO Message Buffer HR Time Stamp Get.

#define FLEXCAN_EFIFOMB_EDL_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>31)&0x01U)
Enhanced FIFO Message Buffer EDL Get.

#define FLEXCAN_EFIFOMB_BRS_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>30)&0x01U)
Enhanced FIFO Message Buffer BRS Get.

#define FLEXCAN_EFIFOMB_ESI_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>29)&0x01U)
Enhanced FIFO Message Buffer ESI Get.

#define FLEXCAN_EFIFOMB_SRR_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>22)&0x01U)
Enhanced FIFO Message Buffer SRR Get.

#define FLEXCAN_EFIFOMB_IDE_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>21)&0x01U)
Enhanced FIFO Message Buffer IDE Get.

#define FLEXCAN_EFIFOMB_RTR_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>20)&0x01U)
Enhanced FIFO Message Buffer RTR Get.

#define FLEXCAN_EFIFOMB_DLC_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]>>16)&0x0FU)
Enhanced FIFO Message Buffer DLC Get.

#define FLEXCAN_EFIFOMB_TIMESTAMP_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[0U]&0xFFU)
Enhanced FIFO Message Buffer TIME STAMP Get.

#define FLEXCAN_EFIFOMB_STDID_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[1U]>>18)&0x7FFU)
Enhanced FIFO Message Buffer STD ID Get.

#define FLEXCAN_EFIFOMB_EXTID_GET_FROM_BUFFER(Addr) (((uint32_t*)Addr)[1U]&0xFFFFFFFF)
Enhanced FIFO Message Buffer EXT ID Get.

```

```
#define FLEXCAN_EFIFOMB_DATAADDR_GET_FROM_BUFFER(Addr,  
wordIndex) (&(((uint32_t*)Addr)[2U+(wordIndex)]))
```

Enhanced FIFO Message Buffer Data Get.

```
#define FLEXCAN_EFIFOMB_IDHIT_GET_FROM_BUFFER(Addr,  
dataWordLen) (((uint32_t*)Addr)[2U+(dataWordLen)])&0x1FFU
```

Enhanced FIFO Message Buffer IDHIT Get.

```
#define FLEXCAN_EFIFOMB_HRTIMESTAMP_GET_FROM_BUFFER(Addr,  
dataWordLen) (((uint32_t*)Addr)[3U+(dataWordLen)])
```

Enhanced FIFO Message Buffer HR Time Stamp Get.

Enumerations

```
enum FLEXCAN_ClockSrcType { FLEXCAN_CLOCK_CANPE = 0x00U, FLEXCAN_CLOCK_INTERFACE =  
0x01U }
```

FLEXCAN Clock Source Select Enumeration.

```
enum FLEXCAN_DataWidthType { FLEXCAN_DATAWIDTH_8 = 8U, FLEXCAN_DATAWIDTH_16 = 16U,  
FLEXCAN_DATAWIDTH_32 = 32U, FLEXCAN_DATAWIDTH_64 = 64U }
```

FLEXCAN data length type.

```
enum FLEXCAN_PNET_FilterType { FLEXCAN_PNET_ID_FILTER = 0U,  
FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER = 1U, FLEXCAN_PNET_ID_FILTER_WITH_NCOUNT = 2U,  
FLEXCAN_PNET_ID_AND_PAYLOAD_FILTER_WITH_NCOUNT = 3U }
```

CAN Pretended Network Filter type.

```
enum FLEXCAN_PNET_IdFilterType { FLEXCAN_PNET_ID_FILTER_SAME = 0U,  
FLEXCAN_PNET_ID_FILTER_GREATER_EQUAL = 1U, FLEXCAN_PNET_ID_FILTER_SMALLER_EQUAL = 2U,  
FLEXCAN_PNET_ID_FILTER_RANGE = 3U }
```

CAN Pretended Network ID Filter type.

```
enum FLEXCAN_PNET_PayloadFilterType { FLEXCAN_PNET_PAYLOAD_FILTER_SAME = 0U,  
FLEXCAN_PNET_PAYLOAD_FILTER_GREATER_EQUAL = 1U,  
FLEXCAN_PNET_PAYLOAD_FILTER_SMALLER_EQUAL = 2U, FLEXCAN_PNET_PAYLOAD_FILTER_RANGE =  
3U }
```

CAN Pretended Network Payload Filter type.

Functions

```
LOCAL_INLINE void FLEXCAN_HWA_SetMcrDisable (FLEXCAN_Type *const pCan, uint8_t bDisable)
```

Set Module Disable.

LOCAL_INLINE void **FLEXCAN_HWA_SetMcrFreeze** (FLEXCAN_Type *const pCan, uint8_t bFreeze)

Set Module Freeze.

LOCAL_INLINE void **FLEXCAN_HWA_SetHaltFreeze** (FLEXCAN_Type *const pCan)

Set Module Halt and Freeze.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_WaitMcrFreezen** (FLEXCAN_Type *const pCan, uint32_t u32TimeoutTick)

Wait FLEXCAN Module Enter Freezen mode.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_WaitMcrExitFreezen** (FLEXCAN_Type *const pCan, uint32_t u32TimeoutTick)

Wait FLEXCAN Module Exit Freezen mode.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_WaitMcrReady** (FLEXCAN_Type *const pCan, uint32_t u32TimeoutTick)

Wait FLEXCAN Module Ready.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_WaitMcrNoReady** (FLEXCAN_Type *const pCan, uint32_t u32TimeoutTick)

Wait FLEXCAN Module Not Ready.

LOCAL_INLINE void **FLEXCAN_HWA_SetMCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FLEXCAN MCR value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetMCR** (FLEXCAN_Type *const pCan)

Get FLEXCAN MCR value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachMCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attach FLEXCAN MCR value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoEn** (FLEXCAN_Type *const pCan)

Check FLEXCAN LegacyFifo Enabled.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckFifoDma** (FLEXCAN_Type *const pCan)

Check if fifo dma enable.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckPNETAck** (FLEXCAN_Type *const pCan, uint8_t bEntryPNET,
uint32_t u32TimeoutTick)

Check if it is in pretended mode.

LOCAL_INLINE void **FLEXCAN_HWA_SetCtrl1BaudSrc** (FLEXCAN_Type *const pCan,
FLEXCAN_ClockSrcType tBaudClk)

Set FLEXCAN baudrate clock source.

LOCAL_INLINE void **FLEXCAN_HWA_SetCtrl1** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FLEXCAN ctrl1 value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachCtrl1** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attach FLEXCAN ctrl1 value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetCtrl1** (FLEXCAN_Type *const pCan)

Get FLEXCAN ctrl1 value.

LOCAL_INLINE void **FLEXCAN_HWA_SetCtrl2** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set CTRL2 value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachCtrl2** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attach FLEXCAN ctrl2 value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetCtrl2** (FLEXCAN_Type *const pCan)

Get CTRL2 value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckErrorInterrupt** (FLEXCAN_Type *const pCan)

Check Interrupt Mask.

LOCAL_INLINE void **FLEXCAN_HWA_SetFlag1** (FLEXCAN_Type *const pCan, uint32_t u32W1cFlag)
Clear IFLAG1 Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetFlag1** (FLEXCAN_Type *const pCan)
Get IFLAG1 Flag.

LOCAL_INLINE void **FLEXCAN_HWA_UnlockMbNoFifoFlag** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex)
Clear Rx Flag and unlock MB.

LOCAL_INLINE void **FLEXCAN_HWA_W1cFlag1NoFifoFlag** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex)
Clear MB Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetFlag1NoFifoFlag** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex)
Get MB Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoData** (FLEXCAN_Type *const pCan)
Check if legacy fifo contains at least one data.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoWarning** (FLEXCAN_Type *const pCan)
Check if legacy fifo contains 5 data and trigger warning.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoDataLost** (FLEXCAN_Type *const pCan)
Check if legacy fifo data lost.

LOCAL_INLINE void **FLEXCAN_HWA_ClearIFLAG1FifoFlag** (FLEXCAN_Type *const pCan)
Clear Legacy Fifo Rx data available Flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClearIFLAG1FifoWarningFlag** (FLEXCAN_Type *const pCan)
Clear Legacy Fifo Rx warning Flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClearIFLAG1FifoErrorFlag** (FLEXCAN_Type *const pCan)

Clear Legacy Fifo Rx error Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoDataPolling** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data Valid Polling Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoWarningPolling** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data warning Polling Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoErrorPolling** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data lost Polling Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoDataInterrupt** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data Valid Interrupt Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoWarningInterrupt** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data warning interrupt Flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckLegacyFifoErrorInterrupt** (FLEXCAN_Type *const pCan)

Check Legacy Fifo Data lost interrupt Flag.

LOCAL_INLINE void **FLEXCAN_HWA_SetMask1MbInterrupt** (FLEXCAN_Type *const pCan, uint32_t
u32Mask)

Set special message buffer interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_AttachMask1MbInterrupt** (FLEXCAN_Type *const pCan, uint32_t
u32Mask)

Attach special message buffer interrupt MASK1.

LOCAL_INLINE void **FLEXCAN_HWA_AttachMbIndexInterrupt** (FLEXCAN_Type *const pCan, uint32_t
u32MbIndex)

Attach special message buffer interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetMaskFifoInterrupt** (FLEXCAN_Type *const pCan)

Set Legacy Fifo Rx interrupt.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetMask1** (FLEXCAN_Type *const pCan)

Get special message buffer interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set CBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetFDCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FDCBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetFDCTRL** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set FDCTRL value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CalDataWidth** (uint8_t *pDataWidthArr)

Set MB data width in MBDSR.

LOCAL_INLINE void **FLEXCAN_HWA_SetEPRS** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set EPRS value.

LOCAL_INLINE void **FLEXCAN_HWA_SetENCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set ENCBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetEDCBT** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set EDCBT value.

LOCAL_INLINE void **FLEXCAN_HWA_SetETDC** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set ETDC value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetTimeStamp** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN rx timestamp.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetErrorInfo** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN error.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetErrorIntInfo** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN error with interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_ClrErrorInfo** (FLEXCAN_Type *const pCan, uint32_t u32ErrorEsr1)

Clear Error Info.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetErrorCount** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN error count.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetERFSR** (FLEXCAN_Type *const pCan)

This Function is used to get FLEXCAN enhanced fifo error count.

LOCAL_INLINE void **FLEXCAN_HWA_EnableLegacyFifo** (FLEXCAN_Type *const pCan)

Enable FLEXCAN Legacy Fifo.

LOCAL_INLINE void **FLEXCAN_HWA_DisableLegacyFifo** (FLEXCAN_Type *const pCan)

Disable FLEXCAN Legacy Fifo.

LOCAL_INLINE void **FLEXCAN_HWA_AttachERFCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attach FLEXCAN ERFCR value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFIFOEn** (FLEXCAN_Type *const pCan)

Check Enhanced FIFO Enable.

LOCAL_INLINE void **FLEXCAN_HWA_EnableFDFifo** (FLEXCAN_Type *const pCan)

Enable FLEXCAN FD Fifo.

LOCAL_INLINE void **FLEXCAN_HWA_DisableFDFifo** (FLEXCAN_Type *const pCan)

Disable FLEXCAN FD Fifo.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckLegacyFifoEnabled** (FLEXCAN_Type *const pCan)

Check FLEXCAN legacy fifo if enabled.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckFdFifoEnabled** (FLEXCAN_Type *const pCan)

Check FLEXCAN FD fifo if enabled.

LOCAL_INLINE void **FLEXCAN_HWA_SetLegacyFifoFilterNum** (FLEXCAN_Type *const pCan, uint32_t u32RFFN)

Set legacy fifo filter number number.

LOCAL_INLINE void **FLEXCAN_HWA_SetLegacyFifoFilterNum1** (FLEXCAN_Type *const pCan, uint32_t u32FilterNum)

Set legacy fifo filter number number.

LOCAL_INLINE uint8_t **FLEXCAN_HWA_CheckFd** (FLEXCAN_Type *const pCan)

Check FLEXCAN FD enable.

LOCAL_INLINE void **FLEXCAN_HWA_SetIndividualMask** (FLEXCAN_Type *const pCan, uint32_t u32MbIndex, uint32_t u32Mask)

Set RXIMR for FLEXCAN.

LOCAL_INLINE void **FLEXCAN_HWA_SetMBGlobalMask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)

Set MB global mask value.

LOCAL_INLINE void **FLEXCAN_HWA_SetFifoGlobalMask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)

Set Fifo global mask value.

LOCAL_INLINE void **FLEXCAN_HWA_SetRx14Mask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)

Set rx14 mask value.

LOCAL_INLINE void **FLEXCAN_HWA_SetRx15Mask** (FLEXCAN_Type *const pCan, uint32_t u32Mask)

Set rx15 mask value.

LOCAL_INLINE void **FLEXCAN_HWA_MbRam** (FLEXCAN_Type *const pCan, uint32_t u32RamIndex, uint32_t u32Value)

Set value to FLEXCAN RAM.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_CTRL1_PN** (FLEXCAN_Type *const pCan)

Clear PNET CTRL1_PN.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_TimeoutInterrupt** (FLEXCAN_Type *const pCan)

Set PNET timeout Interrupt.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ChkPNET_TimeoutInterrupt** (FLEXCAN_Type *const pCan)

Check PNET timeout Interrupt.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ChkPNET_MatchInterrupt** (FLEXCAN_Type *const pCan)

Check PNET match Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_TimeoutInterrupt** (FLEXCAN_Type *const pCan)

Attach PNET timeout Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_MatchInterrupt** (FLEXCAN_Type *const pCan)

Set PNET match Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_MatchInterrupt** (FLEXCAN_Type *const pCan)

Attach PNET match Interrupt.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_MatchCount** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Set PNET Match Count value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_MatchCount** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Attach PNET Match Count value.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_FilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)

Set PNET Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_FilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)
Attach PNET Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_IDFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)
Set PNET ID Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_IDFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)
Attach PNET ID Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_PayloadFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)
Set PNET Payload Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_AttachPNET_PayloadFilterType** (FLEXCAN_Type *const pCan, uint8_t u8Value)
Attach PNET Payload Filter Type.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_ID1** (FLEXCAN_Type *const pCan, uint8_t u8IDE, uint8_t u8RTR, uint32_t u32ID1)
Set PNET ID 1.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_ID2** (FLEXCAN_Type *const pCan, uint8_t u8IDE, uint8_t u8RTR, uint32_t u32ID2)
Set PNET ID 2.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_DlcRange** (FLEXCAN_Type *const pCan, uint8_t u8DLC1, uint8_t u8DLC2)
Set PNET DLC.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_Payload1Content** (FLEXCAN_Type *const pCan, uint32_t u32High, uint32_t u32Low)
Set PNET Payload 1 data content.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_Payload2Content** (FLEXCAN_Type *const pCan, uint32_t u32High, uint32_t u32Low)

Set PNET Payload 2 data content.

LOCAL_INLINE void **FLEXCAN_HWA_SetPNET_Timeout** (FLEXCAN_Type *const pCan, uint16_t u16Value)

Set PNET Timeout value.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_IntFlag** (FLEXCAN_Type *const pCan)

Clear PNET Interrupt flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_TimeoutFlag** (FLEXCAN_Type *const pCan)

Clear PNET timeout Interrupt flag.

LOCAL_INLINE void **FLEXCAN_HWA_ClrPNET_MatchFlag** (FLEXCAN_Type *const pCan)

Clear PNET match Interrupt flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_GetPNET_MatchCount** (FLEXCAN_Type *const pCan)

Get PNET Match Counter.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_PNETRamAddr** (FLEXCAN_Type *const pCan, uint32_t u32MsgIndex)

Get value from FLEXCAN Wake-up message index.

LOCAL_INLINE void **FLEXCAN_HWA_SetERFCR** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set ERFCR value.

LOCAL_INLINE void **FLEXCAN_HWA_SetERFIER** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Set ERFIER value.

LOCAL_INLINE void **FLEXCAN_HWA_AttachERFIER** (FLEXCAN_Type *const pCan, uint32_t u32Value)

Attache ERFIER value.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoPollingFlag** (FLEXCAN_Type *const pCan)

Check Enhanced FIFO Polling flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoDataPolling** (FLEXCAN_Type *const pCan)

Check Enhanced FIFO data polling flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoErrorPolling** (FLEXCAN_Type *const pCan)

Check Enhanced FIFO error polling flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoDataInterrupt** (FLEXCAN_Type *const pCan)

Check Enhanced FIFO data interrupt flag.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_CheckEnhancedFifoErrorInterrupt** (FLEXCAN_Type *const pCan)

Check Enhanced FIFO error interrupt flag.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRResetEnhancedFifo** (FLEXCAN_Type *const pCan)

Reset Enhanced FIFO Engine.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetEnhancedFifoFlag** (FLEXCAN_Type *const pCan,
uint32_t u32Shift, uint32_t u32Mask)

Get Enhanced Fifo Flag with mask.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetDataValid** (FLEXCAN_Type *const pCan)

Get Enhanced Fifo Data valid.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRClrUnderflow** (FLEXCAN_Type *const pCan)

Clear Enhanced Fifo underflow.

LOCAL_INLINE void **FLEXCAN_HWA_ERFSRClrOverflow** (FLEXCAN_Type *const pCan)

Clear Enhanced Fifo overflow.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetUnderflow** (FLEXCAN_Type *const pCan)

Get Enhanced Fifo underflow.

LOCAL_INLINE uint32_t **FLEXCAN_HWA_ERFSRGetOverflow** (FLEXCAN_Type *const pCan)

Get Enhanced Fifo overflow.

```
LOCAL_INLINE void FLEXCAN_HWA_ERFSRClearEnhancedFifoFlag (FLEXCAN_Type *const pCan, uint32_t  
u32ClearMask)
```

Clear Enhanced Fifo status.

```
LOCAL_INLINE void FLEXCAN_HWA_ERFSRClearError (FLEXCAN_Type *const pCan, uint32_t  
u32ClearMask)
```

Clear Enhanced Fifo Error.

```
LOCAL_INLINE void FLEXCAN_HWA_AttachERFIERDataInterrupt (FLEXCAN_Type *const pCan)
```

Set Enhance Fifo interrupt value.

```
LOCAL_INLINE void FLEXCAN_HWA_EnhancedFifoFilter (FLEXCAN_Type *const pCan, uint32_t  
u32FilterIndex, uint32_t u32Value)
```

Set Enhanced Fifo filter value.

Detailed Description

FLEXCAN hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_freqm.h File Reference

FREQM Module Register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_freqm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **FREQM_HWA_MesClkSel** (FREQM_Type *const pFreqm, uint8_t u8ClkSel)

Set the measured clock selection.

LOCAL_INLINE void **FREQM_HWA_MesClk_PreDiv** (FREQM_Type *const pFreqm, uint8_t u8PredivVal)

Set the clock selection.

LOCAL_INLINE void **FREQM_HWA_EnableCntEventInterrupt** (FREQM_Type *const pFreqm)

Enable count event interrupt.

LOCAL_INLINE void **FREQM_HWA_DisableCntEventInterrupt** (FREQM_Type *const pFreqm)

Disable count event interrupt.

LOCAL_INLINE void **FREQM_HWA_SetMesLength** (FREQM_Type *const pFreqm, uint32_t u32MesLen)

Set counting length of measure counter.

LOCAL_INLINE void **FREQM_HWA_SetRefTimeout** (FREQM_Type *const pFreqm, uint32_t u32RefTo)

Set timeout value of reference counter.

LOCAL_INLINE void **FREQM_HWA_SetRefCnt** (FREQM_Type *const pFreqm, uint32_t u32RefCnt)

Set value of reference counter.

LOCAL_INLINE void **FREQM_HWA_ClearInterruptFlag** (FREQM_Type *const pFreqm)
Clear counter event interrupt flag.

LOCAL_INLINE bool **FREQM_HWA_GetInterruptFlag** (const FREQM_Type *const pFreqm)
Get counter event interrupt flag.

LOCAL_INLINE uint32_t **FREQM_HWA_GetCntStatus** (const FREQM_Type *const pFreqm)
Get counter status.

LOCAL_INLINE void **FREQM_HWA_SetMesCnt** (FREQM_Type *const pFreqm, uint32_t u32MesCnt)
Set value of measure counter.

LOCAL_INLINE uint32_t **FREQM_HWA_GetRefCntSave** (const FREQM_Type *const pFreqm)
Get saved reference counter value.

Detailed Description

FREQM Module Register API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_ftu.h File Reference

FTU hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_ftu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define FTU_CHCTRL_DEADTIME_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 4U))
```

The definition of enable the deadtime of channel and channel+1.

```
#define FTU_CHCTRL_COMPLEMENT_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 1U))
```

The definition of enable the complement of channel.

```
#define FTU_CHCTRL_SYNCEN_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 5U))
```

The definition of enable the synchronization for channel and channel+1.

```
#define FTU_CHCTRL_PHASE_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U))
```

The definition of enable the Phase shift mode for channel and channel+1.

```
#define FTU_CHCTRL_EPHASE_CHANNEL(channel) (uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 7U))
```

The definition of enable the Enhanced Phase shift mode for channel and channel+1.

```
#define FTU_CHCTRL_FAULT_CHANNEL(channel) ((uint32_t)((uint32_t)1U << (((uint32_t)(channel) >> 1U) << 3U) + 6U))
```

The definition of enable the fault control in channel and channel+1.

Enumerations

```
enum FTU_DebugModeType { FTU_DBG_COUNTER_STOP_CHN_WORKS = 0U,  
FTU_DBG_COUNTER_STOP_CHN_FORCE_SAFE = 1U, FTU_DBG_COUNTER_STOP_CHN_FROZEN = 2U,  
FTU_DBG_COUNTER_WORKS_CHN_WORKS = 3U }
```

Select the FTU Debug Mode.

```
enum FTU_ChannelModeType { FTU_CHANNEL_MODE_INPUT = 0u,  
FTU_CHANNEL_MODE_OUTPUT_COMPARE = 1u, FTU_CHANNEL_MODE_EDGE_ALIGN_PWM = 2u,  
FTU_CHANNEL_MODE_DO_NOT_CARE = 3u }
```

```
enum FTU_ChannelEdgeLevelType { FTU_CHANNEL_EDGE_NOT_USED = 0u,  
FTU_CHANNEL_EDGE_RISING = 1u, FTU_CHANNEL_EDGE_FALLING = 2u, FTU_CHANNEL_EDGE_BOTH =  
3u, FTU_CHANNEL_OC_TOGGLE = 1u, FTU_CHANNEL_OC_CLEAR = 2u, FTU_CHANNEL_OC_SET = 3u,  
FTU_CHANNEL_PWM_HIGH_TRUE = 2u, FTU_CHANNEL_PWM_LOW_TRUE = 3u }
```

```
enum FTU_PrescalerType { FTU_DIV_1 = 0U, FTU_DIV_2, FTU_DIV_4, FTU_DIV_8, FTU_DIV_16,  
FTU_DIV_32, FTU_DIV_64, FTU_DIV_128 }
```

Select the prescaler of the FTU.

```
enum FTU_FilterPrescalerType { FTU_FLT_DIV_1 = 0U, FTU_FLT_DIV_2, FTU_FLT_DIV_3,  
FTU_FLT_DIV_4, FTU_FLT_DIV_5, FTU_FLT_DIV_6, FTU_FLT_DIV_7, FTU_FLT_DIV_8, FTU_FLT_DIV_9,  
FTU_FLT_DIV_10, FTU_FLT_DIV_11, FTU_FLT_DIV_12, FTU_FLT_DIV_13, FTU_FLT_DIV_14,  
FTU_FLT_DIV_15, FTU_FLT_DIV_16 }
```

Select the prescaler of the FTU filter.

```
enum FTU_QuadratureModeType { FTU_QUADMODE_PHA_PHB_ENCODING_MODE = 0U,  
FTU_QUADMODE_COUNT_DIR_ENCODING_MODE }
```

selects the encoding mode used in the Quadrature Decoder mode

```
enum FTU_QuadratureDirectionType { FTU_QUADIR_DECREMENT = 0U, FTU_QUADIR_INCREMENT }
```

FTU Counter Direction In Quadrature Decoder Mode.

```
enum FTU_BitStatusType { FTU_BIT_LOW = 0U, FTU_BIT_HIGH }
```

register bit status

```
enum FTU_ModuleClkSrcType { FTU_MODULE_NO_CLK = 0U, FTU_MODULE_INTERNAL_CLK = 1U,  
FTU_MODULE_EXTERNAL_CLK = 3U }
```

Ftu module clock source.

```
enum FTU_UpDownDisableType { FTU_DISABLE_TRIG_INTR_NONE = 0U,  
FTU_DISABLE_TRIG_INTR_CNT_DOWN = 1U, FTU_DISABLE_TRIG_INTR_CNT_UP = 2U,  
FTU_DISABLE_TRIG_INTR_CNT_UP_DOWN = 3U }
```

Disable channel match trigger/interrupt when count-up/down in CPWM/QUAD mode.

```
enum FTU_TrigModeType { FTU_CLEARS_TRIG_WHEN_DETECTED = 0x0U,  
FTU_NOT_CLEARS_TRIG_WHEN_DETECTED = 0x01U }
```

hardware trig(n) mode

```
enum FTU_FaultModeType { FTU_FAULT_MODE_DISABLED = 0x00U,  
FTU_FAULT_MODE_EVEN_CHANNEL = 0x01U, FTU_FAULT_MODE_CHANNEL_ALL = 0x02U,  
FTU_FAULT_MODE_AUTO = 0x03U }
```

fault mode enumeration

```
enum FTU_DeadTimePrescalerType { FTU_DEADTIME_PRESCALER_DIV_1 = 0U,  
FTU_DEADTIME_PRESCALER_DIV_4 = 2U, FTU_DEADTIME_PRESCALER_DIV_16 = 3U }
```

Deadtime Prescaler Value.

```
enum FTU_TrigOutputModeType { FTU_TRIG_OUTPUT_AS_CHANNEL_MODE = 0U,  
FTU_TRIG_OUTPUT_AS_TRIGGER = 1U }
```

Trigger output control.

```
enum FTU_FaultDisableDelayType { FTU_FAULT_DISABLE_DELAY_NO_DELAY = 0u,  
FTU_FAULT_DISABLE_DELAY_VALUE_0 = 1u, FTU_FAULT_DISABLE_DELAY_VALUE_1 = 2u }
```

Fault disable channel output delay value selection.

```
enum FTU_MeasurementModeType { FTU_MEASURE_MODE_OFF = 0u,  
FTU_MEASURE_MODE_DUTY_CYCLE = 1u, FTU_MEASURE_MODE_PERIOD = 2u,  
FTU_MEASURE_EDGE_NUMBER = 3u, FTU_MEASURE_EXPECT_EDGE_NUMBER = 4u,  
FTU_MEASURE_ICENM_WIND_WRITE = 7u }
```

Input Capture Measurement Mode.

Functions

```
LOCAL_INLINE void FTU_HWA_ClearModuleRegister (FTU_Type *pFtu)
```

Clear FTU module registers.

```
LOCAL_INLINE void FTU_HWA_DisableGlobalTimeBase (FTU_Type *pFtu)
```

Disable Global Time Base Enable.

```
LOCAL_INLINE void FTU_HWA_EnableGlobalTimeBase (FTU_Type *pFtu)
```

Enable Global Time Base Enable.

LOCAL_INLINE void **FTU_HWA_SetPwmLoadEnable** (FTU_Type *pFtu)

Set the LDOK flag.

LOCAL_INLINE void **FTU_HWA_ClearPwmLoadEnable** (FTU_Type *pFtu)

Clear the LDOK flag.

LOCAL_INLINE void **FTU_HWA_SetCntinSync** (FTU_Type *pFtu)

Set the CNTINC flag.

LOCAL_INLINE void **FTU_HWA_ClearCntinSync** (FTU_Type *pFtu)

Clear the CNTINC flag.

LOCAL_INLINE void **FTU_HWA_SetChannelSyncEnable** (FTU_Type *pFtu, uint32_t u32Index)

Set the SYNCEN flag.

LOCAL_INLINE void **FTU_HWA_ClearChannelSyncEnable** (FTU_Type *pFtu, uint32_t u32Index)

Clear the SYNCEN flag.

LOCAL_INLINE void **FTU_HWA_SetPwmSyncMode** (FTU_Type *pFtu)

Set the PWMSYNC flag.

LOCAL_INLINE void **FTU_HWA_ClearPwmSyncMode** (FTU_Type *pFtu)

Clear the PWMSYNC flag.

LOCAL_INLINE void **FTU_HWA_SetReinitBySync** (FTU_Type *pFtu)

Set the REINIT flag.

LOCAL_INLINE void **FTU_HWA_ClearReinitBySync** (FTU_Type *pFtu)

Clear the REINIT flag.

LOCAL_INLINE void **FTU_HWA_EnableMaxLoadPoint** (FTU_Type *pFtu)

Set the CNTMAX flag.

LOCAL_INLINE void **FTU_HWA_DisableMaxLoadPoint** (FTU_Type *pFtu)

Clear the CNTMAX flag.

LOCAL_INLINE void **FTU_HWA_EnableMinLoadPoint** (FTU_Type *pFtu)

Set the CNTMIN flag.

LOCAL_INLINE void **FTU_HWA_DisableMinLoadPoint** (FTU_Type *pFtu)

Clear the CNTMIN flag.

LOCAL_INLINE void **FTU_HWA_EnableChannelMatchReload** (FTU_Type *pFtu, uint32_t u32Channel)

Set the CHNSEL flag of a channel.

LOCAL_INLINE void **FTU_HWA_DisableChannelMatchReload** (FTU_Type *pFtu, uint32_t u32Channel)

Clear the CHNSEL flag of a channel.

LOCAL_INLINE void **FTU_HWA_ConfigFreqOfReloadOp** (FTU_Type *pFtu, uint8_t u8Freq)

configure the frequency of the reload opportunities

LOCAL_INLINE void **FTU_HWA_ConfigDebugMode** (FTU_Type *pFtu, uint32_t u32Mode)

Selects the FTU behavior in Debug mode.

LOCAL_INLINE void **FTU_HWA_ConfigUpDownDisable** (FTU_Type *pFtu, **FTU_UpDownDisableType** eUpDownDisable)

Disable channel match trigger/interrupt when count-up/down in CPWM/QUAD mode.

LOCAL_INLINE void **FTU_HWA_EnableReloadPointInterrupt** (FTU_Type *pFtu)

Enable reload point interrupt.

LOCAL_INLINE void **FTU_HWA_DisableReloadPointInterrupt** (FTU_Type *pFtu)

Disable reload point interrupt.

LOCAL_INLINE void **FTU_HWA_EnableSWOControl** (FTU_Type *pFtu, uint32_t u32Mask)

Enable software output control of channels.

LOCAL_INLINE void **FTU_HWA_DisalbeSWOControl** (FTU_Type *pFtu, uint32_t u32Mask)

Disable software output control of channels.

LOCAL_INLINE void **FTU_HWA_SetSWOValue** (FTU_Type *pFtu, uint32_t u32Mask)

Set the software output control value.

LOCAL_INLINE void **FTU_HWA_ClearSWOValue** (FTU_Type *pFtu, uint32_t u32Mask)

Clear the software output control value.

LOCAL_INLINE void **FTU_HWA_ConfigPairDeadtimePrescaler** (FTU_Type *pFtu, uint32_t u32Channel, **FTU_DeadTimePrescalerType** ePrescaler)

Configure the pair deadtime prescaler value.

LOCAL_INLINE void **FTU_HWA_ConfigPairDeadtimeValue** (FTU_Type *pFtu, uint32_t u32Channel, uint32_t u32Value)

Configure the deadtime value.

LOCAL_INLINE void **FTU_HWA_ConfigDeadtimePrescaler** (FTU_Type *pFtu, **FTU_DeadTimePrescalerType** ePrescaler)

Configure the deadtime prescaler value.

LOCAL_INLINE void **FTU_HWA_ConfigDeadtimeValue** (FTU_Type *pFtu, uint32_t u32Value)

Configure the deadtime value.

LOCAL_INLINE uint32_t **FTU_HWA_GetCounterValue** (FTU_Type *pFtu)

Get the counter value of the selected FTU module.

LOCAL_INLINE void **FTU_HWA_ClearModuleCounter** (FTU_Type *pFtu, uint32_t u32RegValue)

Clear FTU counter.

LOCAL_INLINE void **FTU_HWA_ConfigChannel** (FTU_Type *pFtu, uint8_t u8Channel, uint32_t u32RegValue)

Configure FTU channel.

LOCAL_INLINE uint32_t **FTU_HWA_GetChannelValue** (FTU_Type *pFtu, uint8_t u8Channel)

Get FTU channel value.

LOCAL_INLINE void **FTU_HWA_SetChannelValue** (FTU_Type *pFtu, uint8_t u8Channel, uint32_t u32RegValue)

Set FTU channel value.

LOCAL_INLINE void **FTU_HWA_SetModuleCompareValue** (FTU_Type *pFtu, uint32_t u32RegValue)

Set FTU counter compare val.

LOCAL_INLINE void **FTU_HWA_SetCounterInitialValue** (FTU_Type *pFtu, uint32_t u32RegValue)

Set FTU counter initial value.

LOCAL_INLINE void **FTU_HWA_ConfigModuleMode** (FTU_Type *pFtu, uint32_t u32RegValue)

Configure FTU mode.

LOCAL_INLINE void **FTU_HWA_ConfigDeadtime** (FTU_Type *pFtu, uint32_t u32RegValue)

Configure FTU deadtime.

LOCAL_INLINE void **FTU_HWA_ConfigInputCaptureFilter** (FTU_Type *pFtu, uint32_t u32Channel, uint32_t u32FilterValue)

Configure input capture filter.

LOCAL_INLINE void **FTU_HWA_EnableModuleFault** (FTU_Type *pFtu, uint8_t u8InputMask)

Enable FTU module fault.

LOCAL_INLINE uint32_t **FTU_HWA_GetFaultEnable** (FTU_Type *pFtu)

get the fault input enable status

LOCAL_INLINE void **FTU_HWA_DisableModuleFault** (FTU_Type *pFtu, uint8_t u8Input)

Disable FTU module fault.

LOCAL_INLINE void **FTU_HWA_EnableModuleFaultGlitchFilter** (FTU_Type *pFtu, uint8_t u8Input)
Enable FTU module fault glitch filter.

LOCAL_INLINE void **FTU_HWA_DisableModuleFaultGlitchFilter** (FTU_Type *pFtu, uint8_t u8Input)
Disable FTU module fault glitch filter.

LOCAL_INLINE void **FTU_HWA_SetModuleFaultFilterValue** (FTU_Type *pFtu, uint8_t u8Value)
Set FTU module fault filter value.

LOCAL_INLINE void **FTU_HWA_SetFaultDelay0** (FTU_Type *pFtu, uint8_t u8Value)
Set FTU module fault disable channel output delay value0.

LOCAL_INLINE void **FTU_HWA_SetFaultDelay1** (FTU_Type *pFtu, uint8_t u8Value)
Set FTU module fault disable channel output delay value.

LOCAL_INLINE void **FTU_HWA_ConfigFaultPolarity** (FTU_Type *pFtu, uint32_t u32FaultIndex, uint8_t u8InputPol)
Configure FTU module fault polarity.

LOCAL_INLINE void **FTU_HWA_SetModulePrescale** (FTU_Type *pFtu, **FTU_PrescalerType** ePs)
Set FTU module prescale.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleOverflowFlag** (FTU_Type *pFtu)
Read FTU module overflow flag.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleOverflowIntrEnable** (FTU_Type *pFtu)
Read FTU module overflow interrupt enable.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleReloadFlag** (FTU_Type *pFtu)
Read FTU module reload flag.

LOCAL_INLINE uint8_t **FTU_HWA_ReadReloadIntrEnable** (FTU_Type *pFtu)
Read FTU module reload interrupt enable.

LOCAL_INLINE bool **FTU_HWA_ReadChannelInterruptFlag** (FTU_Type *pFtu, uint8_t u8Channel)

Read FTU channel interrupt flag.

LOCAL_INLINE void **FTU_HWA_ConfigTrigOutputMode** (FTU_Type *pFtu, uint8_t u8Channel,
FTU_TrigOutputModeType eMode)

Configure the channel output mode.

LOCAL_INLINE void **FTU_HWA_SelectFaultDelay** (FTU_Type *pFtu, uint8_t u8Channel,
FTU_FaultDisableDelayType eSelection)

Select fault disable channel output delay value.

LOCAL_INLINE uint8_t **FTU_HWA_ReadChannelInterruptEnableFlag** (FTU_Type *pFtu, uint8_t u8Channel)

Read FTU channel interrupt enable flag.

LOCAL_INLINE uint32_t **FTU_HWA_ReadModuleFaultFlag** (FTU_Type *pFtu)

Read FTU module fault(n) flag.

LOCAL_INLINE uint8_t **FTU_HWA_ReadFaultIntrEnable** (FTU_Type *pFtu)

Read FTU module fault interrupt enable.

LOCAL_INLINE uint8_t **FTU_HWA_ReadModuleFaultDetectionFlag** (FTU_Type *pFtu)

Read FTU module fault detection flag.

LOCAL_INLINE void **FTU_HWA_EnableChannelsOutput** (FTU_Type *pFtu, uint8_t u8ChannelMask)

Enable FTU module channel(n) output.

LOCAL_INLINE uint8_t **FTU_HWA_GetModuleCpwmMode** (FTU_Type *pFtu)

Get cpwm mode of the FTU module.

LOCAL_INLINE void **FTU_HWA_EnableModuleCpwmMode** (FTU_Type *pFtu)

Enable FTU module cpwm mode.

LOCAL_INLINE void **FTU_HWA_DisableModuleCpwmMode** (FTU_Type *pFtu)

Disable FTU module cpwm mode.

LOCAL_INLINE void **FTU_HWA_ConfigModuleFilterPrescale** (FTU_Type *pFtu, **FTU_FilterPrescalerType** eFilterPs)

Set FTU module filter prescale.

LOCAL_INLINE void **FTU_HWA_ConfigChannelMode** (FTU_Type *pFtu, uint8_t u8Channel, **FTU_ChannelModeType** eMode)

LOCAL_INLINE void **FTU_HWA_ConfigChannelEdgeLevel** (FTU_Type *pFtu, uint8_t u8Channel, **FTU_ChannelEdgeLevelType** eEdgeLevel)

LOCAL_INLINE uint8_t **FTU_HWA_GetEdgeNumberCount** (FTU_Type *pFtu, uint8_t u8Channel)

LOCAL_INLINE void **FTU_HWA_ConfigEdgeNumber** (FTU_Type *pFtu, uint8_t u8Channel, uint8_t u8EdgeNumber)

LOCAL_INLINE void **FTU_HWA_SingleMeasurement** (FTU_Type *pFtu, uint8_t u8Channel)

Re-start measurement when in single mode.

LOCAL_INLINE void **FTU_HWA_EnableMeasureContinuous** (FTU_Type *pFtu, uint8_t u8Channel)

Set the measurement channel to continuous mode.

LOCAL_INLINE void **FTU_HWA_DisableMeasureContinuous** (FTU_Type *pFtu, uint8_t u8Channel)

Set the measurement channel to single mode.

LOCAL_INLINE void **FTU_HWA_EnableMeasureStartImmd** (FTU_Type *pFtu, uint8_t u8Channel)

Set the channel starts measuring after the first edge is detected.

LOCAL_INLINE void **FTU_HWA_ConfigInputCaptureMeasureMode** (FTU_Type *pFtu, uint8_t u8Channel, **FTU_MeasurementModeType** eMode)

LOCAL_INLINE void **FTU_HWA_DisableMeasureStartImmd** (FTU_Type *pFtu, uint8_t u8Channel)

Set the measurement starts immediately after activating the channel.

LOCAL_INLINE **FTU_MeasurementModeType** **FTU_HWA_GetMeasureMode** (FTU_Type *pFtu, uint8_t u8Channel)

Get channel measure mode.

LOCAL_INLINE void **FTU_HWA_SetChannelEpwmHighTrue** (FTU_Type *pFtu, uint8_t u8Channel)
Configure Edge-Aligned pwm mode channel to High-true pulses(clear output on match)

LOCAL_INLINE void **FTU_HWA_SetChannelEpwmLowTrue** (FTU_Type *pFtu, uint8_t u8Channel)
Configure Edge-Aligned pwm mode channel to Low-true pulses(set output on match)

LOCAL_INLINE void **FTU_HWA_ConfigChannelPwmLinkMode** (FTU_Type *pFtu, uint8_t u8Channel)
Configure pwm link mode channel.

LOCAL_INLINE void **FTU_HWA_EnableChannelInterrupt** (FTU_Type *pFtu, uint8_t u8Channel)
Enable FTU channel interrupt.

LOCAL_INLINE void **FTU_HWA_SetModuleUpdateRegBySync** (FTU_Type *pFtu)
Set MODE[FTUEN], this field define different free running counter and synchronization behavior.

LOCAL_INLINE void **FTU_HWA_ClearModuleUpdateRegBySync** (FTU_Type *pFtu)
Clear MODE[FTUEN], this field define different free running counter and synchronization behavior.

LOCAL_INLINE void **FTU_HWA_EnableModuleFaultInterrupt** (FTU_Type *pFtu)
Enable FTU module fault interrupt.

LOCAL_INLINE void **FTU_HWA_EnableChannelDma** (FTU_Type *pFtu, uint8_t u8Channel)
Enable FTU channel dma.

LOCAL_INLINE void **FTU_HWA_ConfigModuleFaultMode** (FTU_Type *pFtu, **FTU_FaultModeType** eMode)
Configure FTU module fault mode.

LOCAL_INLINE void **FTU_HWA_EnableTrigger0Sync** (FTU_Type *pFtu)
Enable synchronization hardware trigger 0.

LOCAL_INLINE void **FTU_HWA_EnableTrigger1Sync** (FTU_Type *pFtu)
Enable synchronization hardware trigger 1.

LOCAL_INLINE void **FTU_HWA_EnableTrigger2Sync** (FTU_Type *pFtu)

Enable synchronization hardware trigger 2.

LOCAL_INLINE void **FTU_HWA_EnableOutputMaskBySync** (FTU_Type *pFtu)

Enable FTU module output mask synchronization.

LOCAL_INLINE void **FTU_HWA_DisableOutputMaskBySync** (FTU_Type *pFtu)

Disable FTU module output mask synchronization.

LOCAL_INLINE void **FTU_HWA_GenerateSwSync** (FTU_Type *pFtu)

Generate FTU software trigger.

LOCAL_INLINE void **FTU_HWA_ConfigTrigMode** (FTU_Type *pFtu, **FTU_TrigModeType** eTrigMode)

Configure FTU trigger mode.

LOCAL_INLINE void **FTU_HWA_EnableChannelDeadtime** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) deadtime functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelComplement** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) complement functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelSync** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) synchronization functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelPhase** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelPhase** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel($n/n+1$) Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelEnhancedPhase** (FTU_Type *pFtu, uint8_t u8Channel)

Enable channel($n/n+1$) Enhanced Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelEnhancedPhase** (FTU_Type *pFtu, uint8_t u8Channel)
Disable channel(n/n+1) Enhanced Phase Shift Mode functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelFault** (FTU_Type *pFtu, uint8_t u8Channel)
Enable channel(n/n+1) fault functionality.

LOCAL_INLINE void **FTU_HWA_EnableChannelTriggerOut** (FTU_Type *pFtu, uint8_t u8ChannelMask)
Enable channel trigger out.

LOCAL_INLINE void **FTU_HWA_DisableChannelTriggerOut** (FTU_Type *pFtu, uint8_t u8ChannelMask)
Disable channel trigger out.

LOCAL_INLINE void **FTU_HWA_ClearChannelTriggerFlag** (FTU_Type *pFtu)
clear the channel trigger flag

LOCAL_INLINE void **FTU_HWA_EnableReloadTrigger** (FTU_Type *pFtu)
Enable reload trigger.

LOCAL_INLINE void **FTU_HWA_SetChannelPolarity** (FTU_Type *pFtu, uint8_t u8Mask)
Set channel polarity.

LOCAL_INLINE void **FTU_HWA_ConfigModuleOutmask** (FTU_Type *pFtu, uint8_t u8Channel)
Set FTU module outmask.

LOCAL_INLINE void **FTU_HWA_ConfigChannelFilterValue** (FTU_Type *pFtu, uint8_t u8Channel, uint8_t u8FilterVal)
Set channel filter value.

LOCAL_INLINE void **FTU_HWA_ConfigQuadratureMode** (FTU_Type *pFtu, **FTU_QuadratureModeType** eMode)
Configure quadrature mode.

LOCAL_INLINE void **FTU_HWA_EnablePhbInv** (FTU_Type *pFtu)
inverted polarity of phase B input signal

LOCAL_INLINE void **FTU_HWA_DisablePhbInv** (FTU_Type *pFtu)

do not inverted polarity of phase B input signal

LOCAL_INLINE void **FTU_HWA_EnablePhaInv** (FTU_Type *pFtu)

inverted polarity of phase A input signal

LOCAL_INLINE void **FTU_HWA_DisablePhaInv** (FTU_Type *pFtu)

do not inverted polarity of phase B input signal

LOCAL_INLINE void **FTU_HWA_EnablePhaGlitchFilter** (FTU_Type *pFtu)

Enable phase A glitch filter.

LOCAL_INLINE void **FTU_HWA_DisablePhaGlitchFilter** (FTU_Type *pFtu)

Disable phase A glitch filter.

LOCAL_INLINE void **FTU_HWA_EnablePhbGlitchFilter** (FTU_Type *pFtu)

Enable phase B glitch filter.

LOCAL_INLINE void **FTU_HWA_DisablePhbGlitchFilter** (FTU_Type *pFtu)

Disable phase B glitch filter.

LOCAL_INLINE void **FTU_HWA_EnableQuadratureMode** (FTU_Type *pFtu)

Enable FTU module quadrature mode.

LOCAL_INLINE void **FTU_HWA_DisableQuadratureMode** (FTU_Type *pFtu)

Disable FTU module quadrature mode.

LOCAL_INLINE void **FTU_HWA_ConfigModuleDebugMode** (FTU_Type *pFtu, **FTU_DebugModeType** eDbgMode)

Configure FTU module debug mode.

LOCAL_INLINE void **FTU_HWA_ConfigModuleClkSrc** (FTU_Type *pFtu, **FTU_ModuleClkSrcType** eClkSrc)

Set FTU module clock source.

LOCAL_INLINE void **FTU_HWA_ClearOverflowFlag** (FTU_Type *pFtu)

Clear FTU module overflow flag.

LOCAL_INLINE void **FTU_HWA_ClearReloadFlag** (FTU_Type *pFtu)

Clear FTU module reload flag.

LOCAL_INLINE void **FTU_HWA_EnableOverflowInterrupt** (FTU_Type *pFtu)

Enable FTU module interrupt.

LOCAL_INLINE void **FTU_HWA_DisableOverflowInterrupt** (FTU_Type *pFtu)

Disable FTU module overflow interrupt.

LOCAL_INLINE void **FTU_HWA_ClearChannelInterruptFlag** (FTU_Type *pFtu, uint8_t u8Channel)

Clear FTU channel interrupt flag.

LOCAL_INLINE void **FTU_HWA_DisableChannelInterrupt** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel interrupt.

LOCAL_INLINE void **FTU_HWA_DisableWriteProtection** (FTU_Type *pFtu)

Disable write protection.

LOCAL_INLINE void **FTU_HWA_DisableModuleFaultInterrupt** (FTU_Type *pFtu)

Disable FTU module fault interrupt.

LOCAL_INLINE void **FTU_HWA_DisableTrigger0Sync** (FTU_Type *pFtu)

Disable synchronization hardware trigger 0.

LOCAL_INLINE void **FTU_HWA_DisableTrigger1Sync** (FTU_Type *pFtu)

Disable synchronization hardware trigger 1.

LOCAL_INLINE void **FTU_HWA_DisableTrigger2Sync** (FTU_Type *pFtu)

Disable synchronization hardware trigger 2.

LOCAL_INLINE void **FTU_HWA_DisableGenerateSwSync** (FTU_Type *pFtu)

Disable generate FTU software trigger.

LOCAL_INLINE void **FTU_HWA_DisableReloadTrigger** (FTU_Type *pFtu)

Disable reload trigger.

LOCAL_INLINE void **FTU_HWA_DisableChannelDeadtime** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) deadtime functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelComplement** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) complement functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelSync** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) synchronization functionality.

LOCAL_INLINE void **FTU_HWA_DisableChannelFault** (FTU_Type *pFtu, uint8_t u8Channel)

Disable channel(n/n+1) fault functionality.

LOCAL_INLINE void **FTU_HWA_ClearChannelPolarity** (FTU_Type *pFtu, uint8_t u8Channel)

Clear channel polarity.

LOCAL_INLINE uint32_t **FTU_HWA_GetChannelPolarity** (FTU_Type *pFtu, uint8_t u8Channel)

Get channel polarity.

LOCAL_INLINE void **FTU_HWA_ClearModuleFaultFlag** (FTU_Type *pFtu, uint8_t u8Flag)

Clear FTU module fault flag.

LOCAL_INLINE **FTU_QuadratureDirectionType** **FTU_HWA_GetQuadDirection** (FTU_Type *pFtu)

Get FTU Counter Direction In Quadrature Decoder Mode.

LOCAL_INLINE void **FTU_HWA_ClearGlobalFaultFlag** (FTU_Type *pFtu)

Clear FTU global fault flag.

Detailed Description

FTU hardware access layer.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_gpio.h File Reference

GPIO hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_gpio.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **GPIO_HWA_SetPortOutput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port output to 1.

LOCAL_INLINE void **GPIO_HWA_ClearPortOutput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port output to 0.

LOCAL_INLINE void **GPIO_HWA_TogglePort** (GPIO_Type *pGpio, uint32_t u32Pins)

Toggle port output.

LOCAL_INLINE void **GPIO_HWA_SetPortDirectionInput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port direction Input.

LOCAL_INLINE void **GPIO_HWA_SetPortDirectionOutput** (GPIO_Type *pGpio, uint32_t u32Pins)

Set port direction output.

LOCAL_INLINE uint32_t **GPIO_HWA_ReadPortDataInput** (GPIO_Type *pGpio)

Read port data input, this register indicate data on pad.

LOCAL_INLINE void **GPIO_HWA_SetPinOutput** (GPIO_Type *pGpio, uint8_t u8Pin)

Set pin output to 1.

LOCAL_INLINE void **GPIO_HWA_ClearPinOutput** (GPIO_Type *pGpio, uint8_t u8Pin)

Set pin output to 0.

LOCAL_INLINE void **GPIO_HWA_SetPinDirection** (GPIO_Type *pGpio, uint8_t u8Pin)

Set pin direction.

LOCAL_INLINE void **GPIO_HWA_ClearPinDirection** (GPIO_Type *pGpio, uint8_t u8Pin)

Clear pin direction.

Detailed Description

GPIO hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_hrpwm.h File Reference

HRPWM hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_hrpwm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

HRPWM hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-11-06

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_hsadc.h File Reference

Hardware access layer for HSADC.

```
#include "device_header.h"
```

Include dependency graph for HwA_hsadc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum HSADC_AlignType { HSADC_ALIGN_RIGHT = 0U, HSADC_ALIGN_LEFT = 1U }
```

Select the HSADC result alignment.

```
enum HSADC_TriggerModeType { HSADC_TRGMODE_SW = 0U, HSADC_TRGMODE_RISING_EDGE = 1U,  
HSADC_TRGMODE_FALLING_EDGE = 2U, HSADC_TRGMODE_BOTH_EDGE = 3U,  
HSADC_TRGMODE_HIGH_VOLTAGE = 4U, HSADC_TRGMODE_LOW_VOLTAGE = 5U,  
HSADC_TRGMODE_INTERNAL_PERIODIC = 6U }
```

Select the HSADC trigger mode.

```
enum HSADC_SequenceModeType { HSADC_SEQMODE_SINGLE = 0U, HSADC_SEQMODE_CONTINUOUS = 1U,  
HSADC_SEQMODE_DISCONTINUOUS = 2U, HSADC_SEQMODE_GROUP = 4U }
```

Select the HSADC sequence mode.

```
enum HSADC_FastCmpRefMode { HSADC_FAST_CMP_REFMODE_SW = 0U,  
HSADC_FAST_CMP_REFMODE_RAMP = 1U, HSADC_FAST_CMP_REFMODE_ALT = 2U }
```

```
enum HSADC_FastCmpRampDir { HSADC_FAST_CMP_RAMPDIR_UP = 0U,  
HSADC_FAST_CMP_RAMPDIR_DOWN = 1U }
```

```
enum HSADC_FastCmpRampTrgMode { HSADC_FAST_CMP_RAMPTRGMODE_SW_AUTO = 0U,  
HSADC_FAST_CMP_RAMPTRGMODE_SW_EXT = 1U, HSADC_FAST_CMP_RAMPTRGMODE_EXT_AUTO =  
2U }
```

```
enum HSADC_FastCmpRampTrgPolarity { HSADC_FAST_CMP_RAMPTRGPOL_RISING_OR_HIGH = 0U,  
HSADC_FAST_CMP_RAMPTRGPOL_FALLING_OR_LOW = 1U }
```

```
enum HSADC_FcBFLAction { HSADC_FAST_CMP_BFA_A1B0 = 0U, HSADC_FAST_CMP_BFA_A0B1 =  
1U }
```

```
enum HSADC_OvrModeType { HSADC_OVERRUN_MODE_PRESERVE = 0U,  
HSADC_OVERRUN_MODE_OVERWRITE = 1U }
```

Select the HSADC overrun management mode.

```
enum HSADC_TrgLatchUnitPri { TRG_LATCH_UNIT_PRI_ROUND_ROBIN = 0U,  
TRG_LATCH_UNIT_PRI_FIX = 1U }
```

Select the priority of Trigger Latch Unit.

```
enum HSADC_AverageLenType { HSADC_AVERAGE_LEN_4 = 0U, HSADC_AVERAGE_LEN_8 = 1U,  
HSADC_AVERAGE_LEN_16 = 2U, HSADC_AVERAGE_LEN_32 = 3U }
```

Select the HSADC hardware average samples.

```
enum HSADC_ClockDivideType { HSADC_CLOCK_DIV_1 = 0U, HSADC_CLOCK_DIV_2 = 1U,  
HSADC_CLOCK_DIV_4 = 2U, HSADC_CLOCK_DIV_8 = 3U }
```

Set the HSADC clock divider.

```
enum HSADC_TrgSrcType { HSADC_TRGSRG_TRGSEL = 2U, HSADC_TRGSRG_TRIG_LATCH_UNIT = 3U }
```

The trigger source of the HSADC instance.

```
enum HSADC_CmpModeType { HSADC_CMP_ALL_CHANNEL = 0U, HSADC_CMP_SINGLE_CHANNEL = 1U }
```

Select the channel compare mode.

```
enum HSADC_FunctionClockType { FUNCTION_CLOCK_FROM_PCC = 0U, FUNCTION_CLOCK_FROM_PAD  
= 1U }
```

```
enum HSADC_SeqGroupIndex { HSADC_SEQ_GROUP_0 = 0U, HSADC_SEQ_GROUP_1 = 1U }
```

The HSADC Sequence Group Index.

Functions

```
LOCAL_INLINE bool HSADC_HWA_GetFIFOReady (const HSADC_Type *const pHsadc)
```

Check whether the data quantity in the FIFO is greater than watermark.

```
LOCAL_INLINE void HSADC_HWA_ClearFIFOReady (HSADC_Type *const pHsadc)
```

Clear the FIFO ready flag.

```
LOCAL_INLINE bool HSADC_HWA_GetCmpFlag (const HSADC_Type *const pHsadc)
```

Check whether the conversion result is in the comparing range.

```
LOCAL_INLINE void HSADC_HWA_ClearCmpFlag (HSADC_Type *const pHsadc)
```

Clear the compare flag.

```
LOCAL_INLINE bool HSADC_HWA_GetEmpty (const HSADC_Type *const pHsadc)
```

Check whether the HSADC FIFO is empty.

LOCAL_INLINE bool **HSADC_HWA_GetFull** (const HSADC_Type *const pHsadc)

Check whether the HSADC FIFO is full.

LOCAL_INLINE bool **HSADC_HWA_GetOverRun** (const HSADC_Type *const pHsadc)

Get the overrun status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_ClearOverRun** (HSADC_Type *const pHsadc)

Clear the overrun flag of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSequence** (const HSADC_Type *const pHsadc)

Check whether the HSADC conversion sequence is finished.

LOCAL_INLINE void **HSADC_HWA_ClearEndOfSequence** (HSADC_Type *const pHsadc)

Clear the HSADC conversion sequence complete flag.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfConversion** (const HSADC_Type *const pHsadc)

Check whether the current HSADC conversion is finished.

LOCAL_INLINE void **HSADC_HWA_ClearEndOfConversion** (HSADC_Type *const pHsadc)

Clear the HSADC single conversion complete flag.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSample** (const HSADC_Type *const pHsadc)

Check whether the sampling phase of the current HSADC conversion is finished.

LOCAL_INLINE void **HSADC_HWA_ClearEndOfSample** (HSADC_Type *const pHsadc)

Clear the HSADC sampling complete flag.

LOCAL_INLINE bool **HSADC_HWA_GetReady** (const HSADC_Type *const pHsadc)

Check whether the HSADC instance is ready to operate.

LOCAL_INLINE void **HSADC_HWA_ClearReady** (HSADC_Type *const pHsadc)

Clear the ready flag of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetFIFOReadyInterruptFlag** (const HSADC_Type *const pHsadc)

Get the FIFO Ready interrupt flag If enabled, HSADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE void **HSADC_HWA_SetFIFOReadyInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the FIFO Ready interrupt flag If enabled, HSADC interrupt is generated when the FIFO water mark is greater than FWMARK.

LOCAL_INLINE bool **HSADC_HWA_GetCmplInterruptFlag** (const HSADC_Type *const pHsadc)

Get the Compare interrupt flag If enabled, HSADC interrupt is generated when the HSADC conversion result is not within the compare threshold.

LOCAL_INLINE void **HSADC_HWA_SetCmplInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the Compare interrupt flag If enabled, HSADC interrupt is generated when the HSADC conversion result is not within the compare threshold.

LOCAL_INLINE bool **HSADC_HWA_GetOverrunInterruptFlag** (const HSADC_Type *const pHsadc)

Get the Overrun interrupt flag If enabled, HSADC interrupt is generated when the HSADC instance is overrun.

LOCAL_INLINE void **HSADC_HWA_SetOverrunInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the Overrun interrupt flag If enabled, HSADC interrupt is generated when the HSADC instance is overrun.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSequenceInterruptFlag** (const HSADC_Type *const pHsadc)

Get the End of Sequence interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC sequence conversion is completed.

LOCAL_INLINE void **HSADC_HWA_SetEndOfSequenceInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the End of Sequence interrupt enable flag If enabled, HSADC interrupt is generated when the HSADC sequence conversion is completed.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfConversionInterruptFlag** (const HSADC_Type *const pHsadc)

Get the conversion complete interrupt enable flag If enabled, HSADC interrupt is generated when each HSADC conversion is completed.

LOCAL_INLINE void **HSADC_HWA_SetEndOfConversionInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the conversion complete interrupt enable flag. If enabled, HSADC interrupt is generated when each HSADC conversion is completed.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSampleInterruptFlag** (const HSADC_Type *const pHsadc)

Get the sample complete interrupt enable flag. If enabled, HSADC interrupt is generated when each HSADC conversion finished the sampling phase.

LOCAL_INLINE void **HSADC_HWA_SetEndOfSampleInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the sample complete interrupt enable flag. If enabled, HSADC interrupt is generated when each HSADC conversion finished the sampling phase.

LOCAL_INLINE bool **HSADC_HWA_GetHSADCReadyInterruptFlag** (const HSADC_Type *const pHsadc)

Get the HSADC ready interrupt enable flag. If enabled, HSADC interrupt is generated when the HSADC module is ready for conversion.

LOCAL_INLINE void **HSADC_HWA_SetHSADCReadyInterruptFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set the HSADC ready interrupt enable flag. If enabled, HSADC interrupt is generated when the HSADC module is ready for conversion.

LOCAL_INLINE uint32_t **HSADC_HWA_GetInterruptEnable** (const HSADC_Type *const pHsadc)

Get the interrupt enable config.

LOCAL_INLINE void **HSADC_HWA_SetInterruptEnable** (HSADC_Type *const pHsadc, uint32_t u32InterruptCfg)

Set the interrupt enable.

LOCAL_INLINE void **HSADC_HWA_SetFCLKSel** (HSADC_Type *const pHsadc, const **HSADC_FunctionClockType** Type)

Select HSADC function clock.

LOCAL_INLINE void **HSADC_HWA_Reset** (HSADC_Type *const pHsadc)

Assert the HSADC hardware Reset bit.

LOCAL_INLINE bool **HSADC_HWA_GetStop** (const HSADC_Type *const pHsadc)

Get whether HSADC is in stopping status.

LOCAL_INLINE void **HSADC_HWA_Stop** (HSADC_Type *const pHsadc)

Stop the HSADC conversion.

LOCAL_INLINE bool **HSADC_HWA_GetStart** (const HSADC_Type *const pHsadc)

Get the conversion start status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_Start** (HSADC_Type *const pHsadc)

Start the HSADC conversion.

LOCAL_INLINE bool **HSADC_HWA_GetDisable** (const HSADC_Type *const pHsadc)

Get whether the HSADC instance is in disable status.

LOCAL_INLINE void **HSADC_HWA_Disable** (HSADC_Type *const pHsadc)

Disable the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetEnable** (const HSADC_Type *const pHsadc)

Get the enable status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_Enable** (HSADC_Type *const pHsadc)

Enable the HSADC instance.

LOCAL_INLINE **HSADC_OvrModeType** **HSADC_HWA_GetOverrunMode** (const HSADC_Type *const pHsadc)

Get the overrun mode.

LOCAL_INLINE void **HSADC_HWA_SetOverrunMode** (HSADC_Type *const pHsadc, **HSADC_OvrModeType** eOvrMode)

Set the overrun mode.

LOCAL_INLINE bool **HSADC_HWA_GetSeqGpEn** (const HSADC_Type *const pHsadc)

Get the sequence group mode state.

LOCAL_INLINE void **HSADC_HWA_SetSeqGpEn** (HSADC_Type *const pHsadc, bool bEnable)

Set the sequence group mode state.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSeqLength** (const HSADC_Type *const pHsadc)

Get the sequence length of the HSADC conversion sequence.

LOCAL_INLINE void **HSADC_HWA_SetSeqLength** (HSADC_Type *const pHsadc, uint8_t u8SequenceLength)

Set the sequence length of the HSADC conversion sequence.

LOCAL_INLINE **HSADC_SeqModeType** **HSADC_HWA_GetSeqMode** (const HSADC_Type *const pHsadc)

Get the HSADC sequence mode.

LOCAL_INLINE void **HSADC_HWA_SetSeqMode** (HSADC_Type *const pHsadc, **HSADC_SeqModeType** eSequenceMode)

Set the HSADC sequence mode.

LOCAL_INLINE bool **HSADC_HWA_GetAutoDisableMode** (const HSADC_Type *const pHsadc)

Get whether auto disable is enabled.

LOCAL_INLINE void **HSADC_HWA_SetAutoDisableMode** (HSADC_Type *const pHsadc, bool bEnable)

Set whether to enable auto disable mode.

LOCAL_INLINE bool **HSADC_HWA_GetWaitMode** (const HSADC_Type *const pHsadc)

Get whether the wait conversion mode is enabled.

LOCAL_INLINE void **HSADC_HWA_SetWaitMode** (HSADC_Type *const pHsadc, bool bEnable)

Set the wait conversion mode.

LOCAL_INLINE **HSADC_TrgSrcType** **HSADC_HWA_GetTriggerSource** (const HSADC_Type *const pHsadc)

Get the trigger source the the HSADC instance.

```
LOCAL_INLINE void HSADC_HWA_SetTriggerSource (HSADC_Type *const pHsadc, HSADC_TrgSrcType eTrgSource)
```

Set the trigger source the the HSADC instance.

```
LOCAL_INLINE HSADC_TrgModeType HSADC_HWA_GetTriggerMode (const HSADC_Type *const pHsadc)
```

Get the trigger mode of the HSADC instance.

```
LOCAL_INLINE void HSADC_HWA_SetTriggerMode (HSADC_Type *const pHsadc, HSADC_TrgModeType eTrgMode)
```

Set the trigger mode of the HSADC instance.

```
LOCAL_INLINE HSADC_AlignType HSADC_HWA_GetDataAlignment (const HSADC_Type *const pHsadc)
```

Get the data align mode.

```
LOCAL_INLINE void HSADC_HWA_SetDataAlignment (HSADC_Type *const pHsadc, HSADC_AlignType eAlign)
```

Set the data align mode.

```
LOCAL_INLINE uint8_t HSADC_HWA_GetSGDMASEL (const HSADC_Type *const pHsadc)
```

Get SGDMA SEL for the HSADC instance.

```
LOCAL_INLINE void HSADC_HWA_SetSGDMASEL (HSADC_Type *const pHsadc, uint8_t index)
```

Set the SGDMA SEL for the HSADC instance.

```
LOCAL_INLINE bool HSADC_HWA_GetSGDMAEnableFlag (const HSADC_Type *const pHsadc)
```

Get whether SGDMA for the HSADC instance is enabled.

```
LOCAL_INLINE void HSADC_HWA_SetSGDMAEnableFlag (HSADC_Type *const pHsadc, bool bEnable)
```

Enable or disable the SGDMA for the HSADC instance.

```
LOCAL_INLINE bool HSADC_HWA_GetDMAEnableFlag (const HSADC_Type *const pHsadc)
```

Get whether DMA for the HSADC instance is enabled.

LOCAL_INLINE void **HSADC_HWA_SetDMAEnableFlag** (HSADC_Type *const pHsadc, bool bEnable)

Enable or disable the DMA for the HSADC instance.

LOCAL_INLINE uint32_t **HSADC_HWA_GetConfig1** (const HSADC_Type *const pHsadc)

Get the HSADC_CFG1 config.

LOCAL_INLINE void **HSADC_HWA_SetConfig1** (HSADC_Type *const pHsadc, uint32_t u32Config)

Set the HSADC_CFG1 config.

LOCAL_INLINE bool **HSADC_HWA_GetExtChEnable** (const HSADC_Type *const pHsadc)

Get the extend channel enable settings for the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetExtChEnable** (HSADC_Type *const pHsadc, bool bEnable)

Set the extend channel feature for the HSADC instance.

LOCAL_INLINE uint8_t **HSADC_HWA_GetFIFOWaterMark** (const HSADC_Type *const pHsadc)

Get the FIFO water mark settings for the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFIFOWaterMark** (HSADC_Type *const pHsadc, uint8_t u8WaterMark)

Set the FIFO water mark for the HSADC instance.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSmpSpare** (const HSADC_Type *const pHsadc)

LOCAL_INLINE void **HSADC_HWA_SetSmpSpare** (HSADC_Type *const pHsadc, bool bEnable)

LOCAL_INLINE uint8_t **HSADC_HWA_GetTriggerLatchUnitPriority** (const HSADC_Type *const pHsadc)

Get the priority of Trigger Latch.

LOCAL_INLINE void **HSADC_HWA_SetTriggerLatchUnitPriority** (HSADC_Type *const pHsadc,

HSADC_TrgLatchUnitPri eTrgLatchUnitPri)

Set the priority of Trigger Latch.

LOCAL_INLINE void **HSADC_HWA_ClearLatchTrigger** (HSADC_Type *const pHsadc)

Clear Latch Trigger in Trigger Latch Unit.

LOCAL_INLINE bool **HSADC_HWA_GetAverageEnableFlag** (const HSADC_Type *const pHsadc)

Get whether hardware average is enabled.

LOCAL_INLINE void **HSADC_HWA_SetAverageEnableFlag** (HSADC_Type *const pHsadc, bool bEnable)

Enable or disable hardware average for the HSADC instance.

LOCAL_INLINE **HSADC_AverageLenType** **HSADC_HWA_GetAverageNumber** (const HSADC_Type *const pHsadc)

Get the hardware average number.

LOCAL_INLINE void **HSADC_HWA_SetAverageNumber** (HSADC_Type *const pHsadc, **HSADC_AverageLenType** eAverageNumber)

Set the hardware average number.

LOCAL_INLINE bool **HSADC_HWA_GetClockGatingAck** (const HSADC_Type *const pHsadc)

Whether clock gating is acknowledged.

LOCAL_INLINE bool **HSADC_HWA_GetClockGatingEnableFlag** (const HSADC_Type *const pHsadc)

Get whether clock gating is enabled.

LOCAL_INLINE void **HSADC_HWA_SetClockGatingEnableFlag** (HSADC_Type *const pHsadc, bool bEnable)

Set whether to enable HSADC clock gating.

LOCAL_INLINE **HSADC_ClockDivideType** **HSADC_HWA_GetClockDivider** (const HSADC_Type *const pHsadc)

Get the HSADC clock divider.

LOCAL_INLINE void **HSADC_HWA_SetClockDivider** (HSADC_Type *const pHsadc, **HSADC_ClockDivideType** eDivider)

Set the HSADC clock divider.

LOCAL_INLINE uint8_t **HSADC_HWA_GetStartupCnt** (const HSADC_Type *const pHsadc)

Get the HSADC start up count.

LOCAL_INLINE void **HSADC_HWA_SetStartupCnt** (HSADC_Type *const pHsadc, uint8_t u8StartupCnt)

Set the HSADC start up count.

LOCAL_INLINE uint32_t **HSADC_HWA_GetConfig2** (const HSADC_Type *const pHsadc)

Get the HSADC_CFG2 config.

LOCAL_INLINE void **HSADC_HWA_SetConfig2** (HSADC_Type *const pHsadc, uint32_t u32Config)

Set the HSADC_CFG2 config.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSampleTime** (HSADC_Type *const pHsadc, uint8_t u8Selection)

Get the sample time of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetSampleTime** (HSADC_Type *const pHsadc, uint8_t u8Selection,
uint8_t u8SampleTime)

Set the sample time of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetCmpEnable** (const HSADC_Type *const pHsadc)

Get whether hardware compare is enabled.

LOCAL_INLINE void **HSADC_HWA_SetCmpEnable** (HSADC_Type *const pHsadc, bool bEnable)

Set to enable or disable the hardware compare.

LOCAL_INLINE void **HSADC_HWA_SetCmpChannel** (HSADC_Type *const pHsadc, **HSADC_CmpModeType**
eMode, uint8_t u8ChannalNum)

Set the HSADC hardware compare channel.

LOCAL_INLINE void **HSADC_HWA_SetCmpThreshold** (HSADC_Type *const pHsadc, uint16_t u16LowThres,
uint16_t u16HighThres)

Set the HSADC hardware compare threshold.

LOCAL_INLINE uint32_t **HSADC_HWA_GetConfig3** (const HSADC_Type *const pHsadc)

Get the HSADC_CFG3 config.

LOCAL_INLINE void **HSADC_HWA_SetConfig3** (HSADC_Type *const pHsadc, uint32_t u32Config)

Set the HSADC_CFG3 config.

LOCAL_INLINE bool **HSADC_HWA_GetEndOfSeqGroupInterruptFlag** (HSADC_Type *const pHsadc, const uint8_t u8SeqGroupIndex)

Get the end of sequence group flag.

LOCAL_INLINE void **HSADC_HWA_ClearEndOfSeqGroupInterruptFlag** (HSADC_Type *const pHsadc, const uint8_t u8SeqGroupIndex)

Clear the end of sequence group flag.

LOCAL_INLINE void **HSADC_HWA_SetEndOfSeqGroupInterruptEnable** (HSADC_Type *const pHsadc, const uint8_t u8SeqGroupIndex, const bool bEnable)

Set the sequence group end of sequence interrupt enable.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSeqGroupStartPoint** (HSADC_Type *const pHsadc, const uint8_t u8SeqGroupIndex)

Get the sequence group start point.

LOCAL_INLINE uint8_t **HSADC_HWA_GetSeqGroupEndPoint** (HSADC_Type *const pHsadc, const uint8_t u8SeqGroupIndex)

Get the sequence group end point.

LOCAL_INLINE void **HSADC_HWA_SetSeqGroupStartEndPoint** (HSADC_Type *const pHsadc, const uint8_t u8SeqGroupIndex, const uint8_t u8Start, const uint8_t u8End)

Set the sequence group start and end point.

LOCAL_INLINE void **HSADC_HWA_SetOGCOffset** (HSADC_Type *const pHsadc, uint16_t val)

LOCAL_INLINE void **HSADC_HWA_SetOGCGain** (HSADC_Type *const pHsadc, uint16_t val)

LOCAL_INLINE void **HSADC_HWA_SetOGCEnable** (HSADC_Type *const pHsadc, bool bEnable)

LOCAL_INLINE void **HSADC_HWA_SetCCal** (HSADC_Type *const pHsadc, uint32_t u32CCal)

Set the HSADC_CCAL config.

LOCAL_INLINE uint32_t **HSADC_HWA_GetFIFOData** (const HSADC_Type *const pHsadc)

Get the conversion result FIFO data of the HSADC instance.

LOCAL_INLINE uint8_t **HSADC_HWA_GetChannelSampleTimeIndex** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Get the sample time index of the HSADC channel.

LOCAL_INLINE void **HSADC_HWA_SetChannelSampleTimeIndex** (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, uint8_t u8SmpSel)

Set the sample time index of the HSADC channel.

LOCAL_INLINE bool **HSADC_HWA_GetChannelConvComplete** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Get the channel conversion complete status of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_ClearChannelConvComplete** (HSADC_Type *const pHsadc, uint8_t u8ChnIdx)

Clear the channel conversion complete status of the HSADC instance.

LOCAL_INLINE bool **HSADC_HWA_GetChannelInterruptEnable** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Check whether interrupt is enabled on the selected HSADC channel.

LOCAL_INLINE void **HSADC_HWA_SetChannelInterruptEnable** (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, bool bEnable)

Set to enable or disable interrupt on the selected HSADC channel.

LOCAL_INLINE uint8_t **HSADC_HWA_GetChannelInput** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Get the input channel of the selected HSADC channel.

LOCAL_INLINE void **HSADC_HWA_SetChannelInput** (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, uint8_t u8InputChann)

Set the input channel to the selected HSADC channel.

LOCAL_INLINE void **HSADC_HWA_SetSCCfg** (HSADC_Type *const pHsadc, const uint8_t u8ChnIdx, uint32_t val)

LOCAL_INLINE uint32_t **HSADC_HWA_GetChannelData** (const HSADC_Type *const pHsadc, const uint8_t u8ChnIdx)

Get the conversion result data of the HSADC instance.

LOCAL_INLINE uint16_t **HSADC_HWA_GetResultCal** (HSADC_Type *const pHsadc)

LOCAL_INLINE bool **HSADC_HWA_GetFCEn** (const HSADC_Type *const pHsadc)

Get the fast compare enable.

LOCAL_INLINE void **HSADC_HWA_SetFCEn** (HSADC_Type *const pHsadc, bool enable)

Set the fast compare reference of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFCtrl** (HSADC_Type *const pHsadc, uint32_t value)

Set the fast compare control register.

LOCAL_INLINE int16_t **HSADC_HWA_GetFCRef** (const HSADC_Type *const pHsadc)

Get the fast compare reference of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFCRef** (HSADC_Type *const pHsadc, uint16_t ref)

Set the fast compare reference of the HSADC instance.

LOCAL_INLINE int16_t **HSADC_HWA_GetFCRampRefA** (const HSADC_Type *const pHsadc)

Get the fast compare ramp reference A of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFCRampRefA** (HSADC_Type *const pHsadc, uint16_t ref)

Set the fast compare ramp reference A of the HSADC instance.

LOCAL_INLINE uint8_t **HSADC_HWA_GetFCRampStep** (const HSADC_Type *const pHsadc)

Get the fast compare ramp step of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFCRampStep** (HSADC_Type *const pHsadc, uint16_t step)

Set the fast compare ramp step.

LOCAL_INLINE void **HSADC_HWA_SetFCRamp0** (HSADC_Type *const pHsadc, uint32_t value)

Set the fast compare ramp0 register.

LOCAL_INLINE int16_t **HSADC_HWA_GetFCRampRefB** (const HSADC_Type *const pHsadc)

Get the fast compare ramp reference B of the HSADC instance.

LOCAL_INLINE void **HSADC_HWA_SetFCRampRefB** (HSADC_Type *const pHsadc, uint16_t ref)

Set the fast compare ramp reference B of the HSADC instance.

LOCAL_INLINE **HSADC_FastCmpRampDir** **HSADC_HWA_GetFCRampDir** (const HSADC_Type *const pHsadc)

Get the fast compare ramp dir.

LOCAL_INLINE void **HSADC_HWA_SetFCRampDir** (HSADC_Type *const pHsadc, **HSADC_FastCmpRampDir** dir)

Set the fast compare ramp dir.

LOCAL_INLINE void **HSADC_HWA_SetFCRamp1** (HSADC_Type *const pHsadc, uint32_t value)

Set the fast compare ramp1 register.

LOCAL_INLINE uint16_t **HSADC_HWA_GetFCRefDeltaP** (const HSADC_Type *const pHsadc)

Get the fast compare reference upper delta.

LOCAL_INLINE void **HSADC_HWA_SetFCRefDeltaP** (HSADC_Type *const pHsadc, uint16_t delta)

Set the fast compare reference upper delta.

LOCAL_INLINE uint16_t **HSADC_HWA_GetFCRefDeltaN** (const HSADC_Type *const pHsadc)

Get the fast compare reference lower delta.

LOCAL_INLINE void **HSADC_HWA_SetFCRefDeltaN** (HSADC_Type *const pHsadc, uint16_t delta)

Set the fast compare reference lower delta.

LOCAL_INLINE void **HSADC_HWA_SetFCHysteresis** (HSADC_Type *const pHsadc, uint32_t u32Hyst)

Set the fast compare reference hysteresis.

LOCAL_INLINE bool **HSADC_HWA_GetFCResult** (HSADC_Type *const pHsadc)

Get the fast compare result.

LOCAL_INLINE bool **HSADC_HWA_GetFCBoundaryFlag** (HSADC_Type *const pHsadc)

Get the fast compare boundary flag.

Detailed Description

Hardware access layer for HSADC.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_intm.h File Reference

Interrupt monitor.

```
#include "device_header.h"
```

Include dependency graph for HwA_intm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Classes

struct **INTM_MonitorType**

Typedefs

typedef struct **INTM_MonitorType** * **INTM_MonitorMemMapPtr**

Functions

LOCAL_INLINE void **INTM_HWA_Enable** (INTM_Type *const plntm, bool bEnable)

Enable the INTM.

LOCAL_INLINE **INTM_MonitorType** * **INTM_HWA_GetIrqMonitor** (INTM_Type *const plntm, uint8_t u8IrqMonitorIndex)

Get the instance of interrupt monitor.

LOCAL_INLINE void **INTM_HWA_SetIACKR** (INTM_Type *const plntm, uint16_t u16IrqNum)

Set the interrupt acknowledge.

LOCAL_INLINE void **INTM_HWA_SetIRQReqNum** (**INTM_MonitorType** *const plrqMon, uint16_t u16IrqNum)

Set which interrupt to be monitored.

LOCAL_INLINE void **INTM_HWA_EnableReset** (**INTM_MonitorType** *const plrqMon, bool bEnable)

Enable reset when interrupt delays overtime.

LOCAL_INLINE void **INTM_HWA_EnableInterrupt** (INTM_MonitorType *const plrqMon, bool bEnable)

Enable interrupt when monitored interrupt delays overtime.

LOCAL_INLINE void **INTM_HWA_EnableInactiveMode** (INTM_MonitorType *const plrqMon, bool bEnable)

Enable inactive mode.

LOCAL_INLINE void **INTM_HWA_StartInactiveMode** (INTM_MonitorType *const plrqMon)

Start the inactive mode.

LOCAL_INLINE void **INTM_HWA_StopInactiveMode** (INTM_MonitorType *const plrqMon)

Stop the inactive mode.

LOCAL_INLINE void **INTM_HWA_SetLatency** (INTM_MonitorType *const plrqMon, uint32_t u32Latency)

Set the timeout value of interrupt.

LOCAL_INLINE uint32_t **INTM_HWA_GetTimerCounter** (INTM_MonitorType *const plrqMon)

Get the value of timer.

LOCAL_INLINE void **INTM_HWA_SetTimerCounter** (INTM_MonitorType *const plrqMon, uint32_t u32Value)

Set the value of timer.

LOCAL_INLINE bool **INTM_HWA_ReadStatus** (INTM_MonitorType *const plrqMon)

Read interrupt status.

Detailed Description

Interrupt monitor.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_ism.h File Reference

ISM hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_ism.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Classes

struct **FPC_Type**

struct **LAM_Type**

Typedefs

```
typedef struct FPC_Type * FPC_MemMapPtr
```

```
typedef struct LAM_Type * LAM_MemMapPtr
```

Enumerations

```
enum ISM_FPC_EdgeDetectModeType { ISM_FPC_DETECT_NO_FILTER = 0U,  
ISM_FPC_DETECT_IMMI_FILTER = 1U, ISM_FPC_DETECT_DELAY_MODE = 2U,  
ISM_FPC_DETECT_PRESCALER_MODE = 3U }
```

```
enum ISM_FPC_EdgeDelayModeType { ISM_FPC_DELAY_FIXED0 = 0U, ISM_FPC_DELAY_FIXED1 = 1U,  
ISM_FPC_DELAY_SMART_DELAY0 = 2U, ISM_FPC_DELAY_SMART_DELAY1 = 3U }
```

```
enum ISM_LAM_InvertEventWindowType { ISM_LAM_EVT_WIN_NON_INVERT = 0U,  
ISM_LAM_EVT_WIN_INVERT = 1U }
```

```
enum ISM_LAM_EventWindowEdgeType { ISM_LAM_NTR_CLEAR_NTR_GATE = 0U,  
ISM_LAM_POS_CLEAR_NTR_GATE = 1U, ISM_LAM_NEG_CLEAR_NTR_GATE = 2U,  
ISM_LAM_ETR_CLEAR_NTR_GATE = 3U, ISM_LAM_NTR_CLEAR_POS_GATE = 4U,  
ISM_LAM_POS_CLEAR_POS_GATE = 5U, ISM_LAM_NEG_CLEAR_POS_GATE = 6U,  
ISM_LAM_ETR_CLEAR_POS_GATE = 7U, ISM_LAM_NTR_CLEAR_NEG_GATE = 8U,  
ISM_LAM_POS_CLEAR_NEG_GATE = 9U, ISM_LAM_NEG_CLEAR_NEG_GATE = 10U,  
ISM_LAM_ETR_CLEAR_NEG_GATE = 11U, ISM_LAM_NTR_CLEAR_ETR_GATE = 12U,  
ISM_LAM_POS_CLEAR_ETR_GATE = 13U, ISM_LAM_NEG_CLEAR_ETR_GATE = 14U,  
ISM_LAM_ETR_CLEAR_ETR_GATE = 15U }
```

```

enum ISM_LAM_EventWindowSelectType { ISM_LAM_EVT_WIN_SEL_REF = 0U,
ISM_LAM_EVT_WIN_SEL_MON = 1U }
enum ISM_LAM_RunModeSelectType { ISM_LAM_RUN_FREE = 0U, ISM_LAM_RUN_GATED = 1U }
enum ISM_LAM_MonitorSourceType { ISM_LAM_SRC_FPC_MON = 0U, ISM_LAM_SRC_EXORD_FPC_REF =
1U }
enum ISM_LAM_InvertMonitorType { ISM_LAM_FPC_MON_NON_INVERT = 0U,
ISM_LAM_FPC_MON_INVERT = 1U }
enum ISM_LAM_InvertReferenceType { ISM_LAM_FPC_REF_NON_INVERT = 0U,
ISM_LAM_FPC_REF_INVERT = 1U }

```

Functions

```

LOCAL_INLINE uint8_t ISM_HWA_PARAM_ECMC (ISM_Type *const plsm)

```

Get ISM ECM count.

```

LOCAL_INLINE uint8_t ISM_HWA_PARAM_FPC (ISM_Type *const plsm)

```

Get ISM FPC count.

```

LOCAL_INLINE uint8_t ISM_HWA_PARAM_LAM (ISM_Type *const plsm)

```

Get ISM LAM count.

```

LOCAL_INLINE void ISM_HWA_Enable (ISM_Type *const plsm, bool bEnable)

```

Enable ISM.

```

LOCAL_INLINE void ISM_HWA_InterruptEnable (ISM_Type *const plsm, bool bEnable)

```

Enable ISM Interrupt.

```

LOCAL_INLINE uint8_t ISM_HWA_GetEcs (ISM_Type *const plsm)

```

Get ECS of ISM_E_STATUS register.

```

LOCAL_INLINE void ISM_HWA_ClearEcs (ISM_Type *const plsm, uint32_t u32Channels)

```

Clear ECS of ISM_E_STATUS register.

```

LOCAL_INLINE uint16_t ISM_HWA_GetEs (ISM_Type *const plsm)

```

Get ES of ISM_E_STATUS register.

LOCAL_INLINE void **ISM_HWA_ClearEs** (ISM_Type *const plsm, uint32_t u32Channels)

Clear ES of ISM_E_STATUS register.

LOCAL_INLINE void **ISM_HWA_EnableEcmSystemEvent** (ISM_Type *const plsm, uint32_t u32Channels, bool bEnable)

Enable ISM ECM channels system event.

LOCAL_INLINE uint8_t **ISM_HWA_GetEnabledEcmSystemEvent** (ISM_Type *const plsm)

Get Enabled ISM ECM channels system event.

LOCAL_INLINE void **ISM_HWA_EnableLamSystemEvent** (ISM_Type *const plsm, uint32_t u32Channels, bool bEnable)

Enable ISM LAM channels system event.

LOCAL_INLINE uint16_t **ISM_HWA_GetEnabledLamSystemEvent** (ISM_Type *const plsm)

Get Enabled ISM LAM channels system event.

LOCAL_INLINE void **ISM_HWA_SetEcCtrl** (ISM_Type *const plsm, uint32_t u32Value)

Set the EC_CTRL register.

LOCAL_INLINE void **ISM_HWA_SetEcm0EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM0.

LOCAL_INLINE void **ISM_HWA_SetEcm1EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM1.

LOCAL_INLINE void **ISM_HWA_SetEcm2EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM2.

LOCAL_INLINE void **ISM_HWA_SetEcm3EcCtrl** (ISM_Type *const plsm, uint32_t u32Value, uint32_t u32LamChannel)

Set THRL and LAM channel of ECM3.

LOCAL_INLINE bool **ISM_HWA_GetFpcRgd** (FPC_Type *const pFpc)

Get the RGD value of FPC_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearFpcRgd** (FPC_Type *const pFpc)

Clear the RGD value of FPC_STATUS.

LOCAL_INLINE bool **ISM_HWA_GetFpcFgd** (FPC_Type *const pFpc)

Get the FGD value of FPC_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearFpcFgd** (FPC_Type *const pFpc)

Clear the FGD value of FPC_STATUS.

LOCAL_INLINE void **ISM_HWA_SetFpcCtrlEn** (FPC_Type *const pFpc, bool bEnable)

Set the EN of FPC_CTRL.

LOCAL_INLINE void **ISM_HWA_SetFpcCtrlIen** (FPC_Type *const pFpc, bool bEnable)

Set the IEN of FPC_CTRL.

LOCAL_INLINE bool **ISM_HWA_GetFpcCtrlIen** (FPC_Type *const pFpc)

Get the IEN of FPC_CTRL.

LOCAL_INLINE void **ISM_HWA_SetFpcFallingDetectMode** (FPC_Type *const pFpc,

ISM_FPC_EdgeDetectModeType eMode)

Set the FEG of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcFallingDelayMode** (FPC_Type *const pFpc,

ISM_FPC_EdgeDelayModeType eMode)

Set the FED of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcRisingDetectMode** (FPC_Type *const pFpc,

ISM_FPC_EdgeDetectModeType eMode)

Set the REG of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcRisingDelayMode** (**FPC_Type** *const pFpc,
ISM_FPC_EdgeDelayModeType eMode)

Set the RED of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcConfigCmp** (**FPC_Type** *const pFpc, uint32_t u32Value)

Set the CMP of FPC_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetFpcConfig** (**FPC_Type** *const pFpc, uint32_t u32Value)

Set the value of FPC_CONFIG.

LOCAL_INLINE uint16_t **ISM_HWA_GetFpcTimer** (**FPC_Type** *const pFpc)

Get the TIM of FPC_TIMER.

LOCAL_INLINE void **ISM_HWA_ClearFpcTimer** (**FPC_Type** *const pFpc)

Clear the TIM of FPC_TIMER.

LOCAL_INLINE uint32_t **ISM_HWA_GetLamStatusCounter** (**LAM_Type** *const pLam)

Get the COUNT of LAM_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearLamStatusCounter** (**LAM_Type** *const pLam)

Clear the COUNT of LAM_STATUS.

LOCAL_INLINE bool **ISM_HWA_GetLamStatusOvfl** (**LAM_Type** *const pLam)

Get the OVFL of LAM_STATUS.

LOCAL_INLINE void **ISM_HWA_ClearLamStatusOvfl** (**LAM_Type** *const pLam)

Clear the OVFL of LAM_STATUS.

LOCAL_INLINE void **ISM_HWA_SetLamCtrlren** (**LAM_Type** *const pLam, bool bEnable)

Set the IEN of LAM_CTRL.

LOCAL_INLINE bool **ISM_HWA_GetLamCtrlren** (**LAM_Type** *const pLam)

Get the IEN of LAM_CTRL.

LOCAL_INLINE void **ISM_HWA_SetLamCtrEn** (LAM_Type *const pLam, bool bEnable)

Set the EN of LAM_CTRL.

LOCAL_INLINE void **ISM_HWA_SetLamConfig** (LAM_Type *const pLam, uint32_t u32Value)

Set the value of LAM_CONFIG.

LOCAL_INLINE void **ISM_HWA_SetLamCounter** (LAM_Type *const pLam, uint32_t u32Value)

Set the value of LAM_COUNTER.

LOCAL_INLINE FPC_Type * **ISM_HWA_GetFpc** (ISM_Type *const plsm, uint8_t u8FpcIndex)

Get the FPC instance.

LOCAL_INLINE LAM_Type * **ISM_HWA_GetLam** (ISM_Type *const plsm, uint8_t u8LamIndex)

Get the LAM instance.

Detailed Description

ISM hardware access layer.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_ldi.h File Reference

LDI hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_ldi.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

LDI hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2025-06-18

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_lu.h File Reference

LU hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_lu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum LU_LgType { LU_LG_0 = 0U, LU_LG_1, LU_LG_2, LU_LG_3 }
```

LU LG instance.

```
enum LU_BypassModeType { LU_NO_BYPASS = 0U, LU_AOI0_BYPASS, LU_AOI1_BYPASS,  
LU_AOI0_AOI1_BYPASS }
```

LU AOI mode.

```
enum LU_ConfigModeType { LU_BYPASS_MODE0 = 0U, LU_RS_MODE, LU_TFF_MODE, LU_DFF_MODE,  
LU_JKFF_MODE, LU_LATCH_MODE }
```

LU FF mode.

```
enum LU_InputNType { LU_INPUT_N_A = 0U, LU_INPUT_N_B, LU_INPUT_N_C, LU_INPUT_N_D }
```

LU Input(n) type.

Functions

```
LOCAL_INLINE void LU_HWA_ConfigAOI0 (LU_Type *pLu, LU_LgType eLg, uint32_t u32RegValue)
```

Configure LU LG(n) AOI0 configuration.

```
LOCAL_INLINE void LU_HWA_ConfigAOI1 (LU_Type *pLu, LU_LgType eLg, uint32_t u32RegValue)
```

Configure LU LG(n) AOI1 configuration.

```
LOCAL_INLINE void LU_HWA_ConfigCtrl (LU_Type *pLu, LU_LgType eLg, uint32_t u32RegValue)
```

Configure LU LG(n) ctrl configuration.

LOCAL_INLINE void **LU_HWA_ConfigFilter** (LU_Type *pLu, **LU_LgType** eLg, uint32_t u32RegValue)

Configure LU LG(n) filter configuration.

LOCAL_INLINE void **LU_HWA_SetLgBypassControl** (LU_Type *pLu, **LU_LgType** eLg, **LU_BypassModeType** eMode)

Set LG bypass control.

LOCAL_INLINE void **LU_HWA_SetLgFlipFlopMode** (LU_Type *pLu, **LU_LgType** eLg, **LU_ConfigModeType** eMode)

Set LG Flip-Flop mode.

LOCAL_INLINE void **LU_HWA_SetLgInputsSyncCtrl** (LU_Type *pLu, **LU_LgType** eLg, uint32_t u32Value)

Set LG inputs synchronous control.

LOCAL_INLINE void **LU_HWA_SetLgFeedbackOverrideCtrl** (LU_Type *pLu, **LU_LgType** eLg, **LU_InputNType** eInput)

Set LG output feedback override control.

LOCAL_INLINE void **LU_HWA_ConfigFlipFlopTo1** (LU_Type *pLu, **LU_LgType** eLg)

Configure the output of flip-flop as "1".

LOCAL_INLINE void **LU_HWA_EnableControlFlipFlopInitOutput** (LU_Type *pLu, **LU_LgType** eLg)

Generate enable pulse.

LOCAL_INLINE void **LU_HWA_SetAOI0InputFilterSampleCount** (LU_Type *pLu, **LU_LgType** eLg, uint32_t u32Value)

Input filter sample count for AOI0.

LOCAL_INLINE void **LU_HWA_SetAOI0InputFilterSamplePeriod** (LU_Type *pLu, **LU_LgType** eLg, uint32_t u32Value)

Input filter sample period for AOI0.

LOCAL_INLINE void **LU_HWA_SetAOI1InputFilterSampleCount** (LU_Type *pLu, **LU_LgType** eLg, uint32_t u32Value)

Input filter sample count for AOI1.

```
LOCAL_INLINE void LU_HWA_SetAOI1InputFilterSamplePeriod (LU_Type *pLu, LU_LgType eLg, uint32_t  
u32Value)
```

Input filter sample period for AOI1.

```
LOCAL_INLINE void LU_HWA_ConfigFlipFlopTo0 (LU_Type *pLu, LU_LgType eLg)
```

Configure the output of flip-flop as "0".

Detailed Description

LU hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_mam.h File Reference

Hardware access layer for MAM.

```
#include "device_header.h"
```

Include dependency graph for HwA_mam.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **Mam_HWA_SetMatrixCfg** (MAM_Type *MAM, uint32_t u32Value)

Set Mam module Matrix Configure register.

LOCAL_INLINE uint32_t **Mam_HWA_GetMatrixCfg** (MAM_Type *MAM)

Read Mam module Ctrl register.

LOCAL_INLINE void **Mam_HWA_Set_Wdgctr** (MAM_Type *MAM, uint32_t u32Value)

Set Mam module Wdgctr register.

LOCAL_INLINE uint32_t **Mam_HWA_Get_Wdgctr** (MAM_Type *MAM)

Read Mam module Wdgctr register.

LOCAL_INLINE void **Mam_HWA_Set_ACR** (MAM_Type *MAM, uint32_t idx, uint32_t u32Value)

Set Mam module ACR register.

LOCAL_INLINE uint32_t **Mam_HWA_Get_ACR** (MAM_Type *MAM, uint32_t idx)

Read Mam module ACR register.

LOCAL_INLINE void **Mam_HWA_Set_ACLR** (MAM_Type *MAM, uint32_t idx, uint32_t u32Value)

Set Mam module ACLR register.

LOCAL_INLINE uint32_t **Mam_HWA_Get_ACLR** (MAM_Type *MAM, uint32_t idx)
Read Mam module ACLR register.

Detailed Description

Hardware access layer for MAM.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_mb.h File Reference

Mailbox hardware access layer.

#include "device_header.h"

Include dependency graph for HwA_mb.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

#define **MB_EVENT_NONE** 0u

The definition of mask all events.

#define **MB_EVENT_ALL** 0xFFFFFFFFu

The definition of receiving all events.

#define **MB_EVENT_ALL_REQ** 0x0000FFFFu

The definition of receiving all request events.

#define **MB_EVENT_ALL_DONE** 0xFFFF0000u

The definition of receiving all done events.

#define **MB_EVENT_REQ**(ch) (uint32_t)((uint32_t)1u << (ch))

The definition of request events on ch.

#define **MB_EVENT_DONE**(ch) (uint32_t)((uint32_t)1u << ((ch) +
MB_INTn_FLG_MASK_DONE_FLAG_MASK_SHIFT))

The definition of done events on ch.

#define **MB_CORE_MASK_CORE_NONE** 0u

The definition of issue to no core.

#define **MB_CORE_MASK_CORE_0** 1u

The definition of issue to core 0.

#define **MB_CORE_MASK_CORE_1** 2u

The definition of issue to core 1.

#define **MB_CORE_MASK_CORE_2** 4u

The definition of issue to core 2.

#define **MB_CORE_MASK_HSM** 8u

The definition of issue to HSM.

#define **MB_CORE_MASK_ALL** 0xFu

The definition of issue to all cores.

#define **MB_CORE_MASK**(core) (uint32_t)(1ul << (core))

The definition of issue to core.

Functions

LOCAL_INLINE void **MB_HWA_LockIntrReg** (uint8_t u8CoreIndex, uint32_t u32Mask)

Lock the MB_INTn registers.

LOCAL_INLINE void **MB_HWA_UnlockIntrReg** (uint8_t u8CoreIndex, uint32_t u32Mask)

Unlock the MB_INTn registers.

LOCAL_INLINE void **MB_HWA_LockFlag** (uint8_t u8CoreIndex)

Lock the MB_INTn_FLG.

LOCAL_INLINE void **MB_HWA_UnlockFlag** (uint8_t u8CoreIndex)

Unlock the MB_INTn_FLG.

LOCAL_INLINE void **MB_HWA_LockFlagMask** (uint8_t u8CoreIndex)

Lock the MB_INTn_FLG_MASK.

LOCAL_INLINE void **MB_HWA_UnlockFlagMask** (uint8_t u8CoreIndex)

Unlock the MB_INTn_FLG_MASK.

LOCAL_INLINE void **MB_HWA_LockInten** (uint8_t u8CoreIndex)

Lock the MB_INTn_INTEN.

LOCAL_INLINE void **MB_HWA_UnlockInten** (uint8_t u8CoreIndex)

Unlock the MB_INTn_INTEN.

LOCAL_INLINE void **MB_HWA_ConfigFlagMask** (uint8_t u8CoreIndex, uint32_t u32Mask)

Configure receive events of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_EnableEvent** (uint8_t u8CoreIndex, uint32_t u32Mask)

Enable receive events of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_DisableEvent** (uint8_t u8CoreIndex, uint32_t u32Mask)

Disable receive events of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_ConfigIntrEnable** (uint8_t u8CoreIndex, uint32_t u32Mask)

Configure the interrupt of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_EnableIntr** (uint8_t u8CoreIndex, uint32_t u32Mask)

Enable the interrupt of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_DisableIntr** (uint8_t u8CoreIndex, uint32_t u32Mask)

Disable the interrupt of mailbox interrupt channel.

LOCAL_INLINE void **MB_HWA_ClearFlag** (uint8_t u8CoreIndex, uint32_t u32Mask)

Clear the flag of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetFlagMask** (uint8_t u8CoreIndex, uint32_t u32Mask)

Get the flag masks of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetFlag** (uint8_t u8CoreIndex, uint32_t u32Mask)

Get the flags of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetFlagStat** (uint8_t u8CoreIndex, uint32_t u32Mask)

Get the result of flag & inten of mailbox interrupt channel.

LOCAL_INLINE uint32_t **MB_HWA_GetMasterID** (uint32_t u32Channel)

Get the master ID of the currently obtained channel.

LOCAL_INLINE void **MB_HWA_WriteData** (uint32_t u32Channel, uint32_t *pData)

Send data to the mailbox channel.

LOCAL_INLINE void **MB_HWA_GetData** (uint32_t u32Channel, uint32_t *pData)

Receive data from the mailbox channel.

LOCAL_INLINE uint32_t **MB_HWA_GetAutoClear** (uint32_t u32Channel, uint8_t u8CoreIndex)

Get the automatically clear status of the mailbox channel.

LOCAL_INLINE void **MB_HWA_ReleaseChannel** (uint32_t u32Channel)

Release the mailbox channel.

LOCAL_INLINE void **MB_HWA_UnlockChannel** (uint32_t u32Channel)

Software clears channel lock.

LOCAL_INLINE void **MB_HWA_SetDone** (uint32_t u32Channel, uint32_t u32DoneMask)

Issue a done event.

LOCAL_INLINE uint32_t **MB_HWA_GetSemaphore** (uint32_t u32Channel, uint32_t u32Mask)

Get the Communication Channel Semaphore Register.

LOCAL_INLINE void **MB_HWA_ConfigDoneMasterId** (uint32_t u32Channel, uint32_t u32MasterId)
Configure the master ID of the core that generates a done event.

LOCAL_INLINE uint32_t **MB_HWA_GetDoneMasterId** (uint32_t u32Channel)
Get the master ID of the core that generates a done event.

LOCAL_INLINE void **MB_HWA_ConfigDoneMask** (uint32_t u32Channel, uint32_t u32DoneMask)
Configure the mask of the done events.

LOCAL_INLINE uint32_t **MB_HWA_GetDoneMask** (uint32_t u32Channel)
Get the mask of the done events.

LOCAL_INLINE void **MB_HWA_ConfigAutoUnlock** (uint32_t u32Channel, uint32_t u32AutoUnlockMask)
Configure the automatically clear of the lock enable bit.

LOCAL_INLINE void **MB_HWA_ConfigRequest** (uint32_t u32Channel, uint32_t u32RequestMask)
Issue request events.

Detailed Description

Mailbox hardware access layer.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_msc.h File Reference

MSC hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_msc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum MSC_ReceiveStatusType { MSC_RSV_SUCCESS = 0x0U, MSC_RSV_PARITY_ERROR = 0x1U,  
MSC_RSV_STOP_ERROR = 0x2U, MSC_RSV_ERROR = 0x4U }
```

Functions

```
LOCAL_INLINE void MSC_HWA_SetTcctr (MSC_Type *const pMsc, uint32_t u32Value)
```

Get the msc TCCTR register.

```
LOCAL_INLINE bool MSC_HWA_GetDataNeedSend (MSC_Type *const pMsc)
```

Get the msc TCCTR register DTS bit.

```
LOCAL_INLINE void MSC_HWA_SetDataNeedSend (MSC_Type *const pMsc)
```

Set the msc TCCTR register DTS bit.

```
LOCAL_INLINE bool MSC_HWA_GetCmdNeedSend (MSC_Type *const pMsc)
```

Get the msc TCCTR register CTS bit.

```
LOCAL_INLINE bool MSC_HWA_GetCfb (MSC_Type *const pMsc)
```

Get the CFB value of TCSTR register.

```
LOCAL_INLINE bool MSC_HWA_GetDfb (MSC_Type *const pMsc)
```

Get the DFB value of TCSTR register.

LOCAL_INLINE void **MSC_HWA_SetNp** (MSC_Type *const pMsc, uint8_t u8Np)

Set the NP value of TCSTR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetNp** (MSC_Type *const pMsc)

Get the NP value of TCSTR register.

LOCAL_INLINE void **MSC_HWA_SetTcdar** (MSC_Type *const pMsc, uint32_t u32Value)

Set the msc TCDAR register.

LOCAL_INLINE void **MSC_HWA_SetTcdarTdh** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TDH of TCDAR register.

LOCAL_INLINE void **MSC_HWA_SetTcdarTdl** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TDL of TCDAR register.

LOCAL_INLINE void **MSC_HWA_SetTccor** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCCOR register.

LOCAL_INLINE void **MSC_HWA_SetTccorTch** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TCH of TCCOR register.

LOCAL_INLINE void **MSC_HWA_SetTccorTcl** (MSC_Type *const pMsc, uint16_t u16Value)

Set the TCL of TCCOR register.

LOCAL_INLINE void **MSC_HWA_SetTcslr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCSLR register.

LOCAL_INLINE void **MSC_HWA_SetTcshr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCSHR register.

LOCAL_INLINE void **MSC_HWA_SetTcelr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the TCELR register.

LOCAL_INLINE void **MSC_HWA_SetIocr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the IOCR register.

LOCAL_INLINE void **MSC_HWA_SetIscr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the ISCR register.

LOCAL_INLINE void **MSC_HWA_SetTcdis** (MSC_Type *const pMsc)

Set the msc TCDIS.

LOCAL_INLINE void **MSC_HWA_ClearTcdis** (MSC_Type *const pMsc)

Clear the msc TCDIS.

LOCAL_INLINE void **MSC_HWA_ClearCrfi** (MSC_Type *const pMsc)

Clear the msc CRFI.

LOCAL_INLINE void **MSC_HWA_ClearCtfi** (MSC_Type *const pMsc)

Clear the msc CTFI.

LOCAL_INLINE void **MSC_HWA_ClearCcfi** (MSC_Type *const pMsc)

Clear the msc CCFI.

LOCAL_INLINE void **MSC_HWA_ClearCdfi** (MSC_Type *const pMsc)

Clear the msc CDFI.

LOCAL_INLINE uint32_t **MSC_HWA_GetInsr** (MSC_Type *const pMsc)

Get the msc INSR register.

LOCAL_INLINE void **MSC_HWA_SetIncr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the INCR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetIncr** (MSC_Type *const pMsc)

Get the msc INCR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetRccsr** (MSC_Type *const pMsc)

Get the msc RCCSR register.

LOCAL_INLINE void **MSC_HWA_SetRccsr** (MSC_Type *const pMsc, uint32_t u32Value)

Set the RCCSR register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr0Data** (MSC_Type *const pMsc)

Get the DATA of RDR0 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr0Addr** (MSC_Type *const pMsc)

Get the LAF of RDR0 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr0Vld** (MSC_Type *const pMsc)

Get the VLD of RDR0 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr0Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR0 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr0Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr1Data** (MSC_Type *const pMsc)

Get the DATA of RDR1 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr1Addr** (MSC_Type *const pMsc)

Get the LAF of RDR1 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr1Vld** (MSC_Type *const pMsc)

Get the VLD of RDR1 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr1Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR1 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr1Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr2Data** (MSC_Type *const pMsc)

Get the DATA of RDR2 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr2Addr** (MSC_Type *const pMsc)

Get the LAF of RDR2 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr2Vld** (MSC_Type *const pMsc)

Get the VLD of RDR2 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr2Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR2 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr2Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr3Data** (MSC_Type *const pMsc)

Get the DATA of RDR3 register.

LOCAL_INLINE uint8_t **MSC_HWA_GetRdr3Addr** (MSC_Type *const pMsc)

Get the LAF of RDR3 register.

LOCAL_INLINE bool **MSC_HWA_GetRdr3Vld** (MSC_Type *const pMsc)

Get the VLD of RDR3 register.

LOCAL_INLINE **MSC_ReceiveStatusType** **MSC_HWA_GetRdr3Rerr** (MSC_Type *const pMsc)

Get the RERR of RDR3 register.

LOCAL_INLINE void **MSC_HWA_ClearRdr3Vld** (MSC_Type *const pMsc)

Clear the VLD of register.

LOCAL_INLINE void **MSC_HWA_SetMsRst** (MSC_Type *const pMsc)

Reset the msc.

LOCAL_INLINE bool **MSC_HWA_GetMsRdone** (MSC_Type *const pMsc)

Get the reset status.

LOCAL_INLINE void **MSC_HWA_ClearMsRDone** (MSC_Type *const pMsc)

clear reset status

LOCAL_INLINE uint32_t **MSC_HWA_GetRtor** (MSC_Type *const pMsc)

Get the msc RTOR register.

LOCAL_INLINE void **MSC_HWA_SetRtor** (MSC_Type *const pMsc, uint32_t u32Value)

Set the msc RTOR register.

LOCAL_INLINE uint32_t **MSC_HWA_GetTcctr1** (MSC_Type *const pMsc)

Get the msc TCCTR1 register.

LOCAL_INLINE void **MSC_HWA_SetTcctr1** (MSC_Type *const pMsc, uint32_t u32Value)

Set the msc TCCTR1 register.

LOCAL_INLINE void **MSC_HWA_SetMscEnable** (MSC_Type *const pMsc, bool bEnable)

Set the EN msc GCR register.

LOCAL_INLINE void **MSC_HWA_SetMscWriteProtection** (MSC_Type *const pMsc, bool bEnable)

Set the WP_EN of msc GCR register.

LOCAL_INLINE void **MSC_HWA_Unlock** (MSC_Type *const pMsc)

Unlock the msc CCULR register.

Detailed Description

MSC hardware access layer.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_ospi.h File Reference

ospi hardware access layer

```
#include "device_header.h"
```

Include dependency graph for HwA_ospi.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define CTRL_RST_VALUE 0x34000
```

Enumerations

```
enum OSPI_DqsSrcSelType { DQS_INTER_LOOPBACK = 0, DQS_PAD_LOOPBACK,  
DQS_EXTERNAL_PADINPUT }
```

```
enum OSPI_ClockDivideType { OSPI_CLOCK_DIV_1 = 0U, OSPI_CLOCK_DIV_2 = 1U, OSPI_CLOCK_DIV_3 =  
2U, OSPI_CLOCK_DIV_4 = 3U, OSPI_CLOCK_DIV_5 = 4U, OSPI_CLOCK_DIV_6 = 5U, OSPI_CLOCK_DIV_7 =  
6U, OSPI_CLOCK_DIV_8 = 7U }
```

```
enum OSPI_ClockMuxType { OSPI_MUX_PLL0 = 0, OSPI_MUX_FIRC, OSPI_MUX_PLL1 }
```

```
enum OSPI_EndianType { OSPI_BIG_ENDIAN = 0, OSPI_LITTLE_ENDIAN }
```

Functions

```
LOCAL_INLINE void OSPI_HWA_Dio3DefHigh (OSPI_Type *pOSPI)
```

OSPI DIO3 Default High.

```
LOCAL_INLINE void OSPI_HWA_Dio3DefLow (OSPI_Type *pOSPI)
```

OSPI DIO3 Default Low.

```
LOCAL_INLINE void OSPI_HWA_Dio2DefHigh (OSPI_Type *pOSPI)
```

OSPI DIO2 Default High.

```
LOCAL_INLINE void OSPI_HWA_Dio2DefLow (OSPI_Type *pOSPI)
```

OSPI DIO2 Default Low.

LOCAL_INLINE void **OSPI_HWA_ModuleEnable** (OSPI_Type *pOSPI)

OSPI Module Enable.

LOCAL_INLINE void **OSPI_HWA_ModuleDisable** (OSPI_Type *pOSPI)

OSPI Module Disable.

LOCAL_INLINE void **OSPI_HWA_ClearTxFifo** (OSPI_Type *pOSPI)

OSPI Clear Tx fifo.

LOCAL_INLINE void **OSPI_HWA_ClearRxFifo** (OSPI_Type *pOSPI)

OSPI Clear Rx fifo.

LOCAL_INLINE void **OSPI_HWA_DdrModeEnable** (OSPI_Type *pOSPI)

OSPI DDR Mode Enable.

LOCAL_INLINE void **OSPI_HWA_DdrModeDisable** (OSPI_Type *pOSPI)

OSPI DDR Mode Disable.

LOCAL_INLINE void **OSPI_HWA_DqsModeEnable** (OSPI_Type *pOSPI)

OSPI DQS Mode Enable.

LOCAL_INLINE void **OSPI_HWA_DqsModeDisable** (OSPI_Type *pOSPI)

OSPI DQS Mode Disable.

LOCAL_INLINE void **OSPI_HWA_DqsLatEnable** (OSPI_Type *pOSPI)

OSPI DQS Lantency Enable.

LOCAL_INLINE void **OSPI_HWA_DqsLatDisable** (OSPI_Type *pOSPI)

OSPI DQS Lantency Disable.

LOCAL_INLINE void **OSPI_HWA_DqsOutEnable** (OSPI_Type *pOSPI)

OSPI DQS Out Enable.

LOCAL_INLINE void **OSPI_HWA_DqsOutDisable** (OSPI_Type *pOSPI)

OSPI DQS Out Disable.

LOCAL_INLINE void **OSPI_HWA_EndianSelect** (OSPI_Type *pOSPI, **OSPI_EndianType** eEndian)

Set OSPI Endian value.

LOCAL_INLINE void **OSPI_HWA_SetCtrlValue** (OSPI_Type *pOSPI, uint32_t u32Value)

Set OSPI CTRL value, users should write the whole value to this register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetCtrlValue** (OSPI_Type *pOSPI)

Read the OSPI CTRL register value for all.

LOCAL_INLINE void **OSPI_HWA_SetCmdId** (OSPI_Type *pOSPI, uint16_t u16Value)

Set CMD ID.

LOCAL_INLINE void **OSPI_HWA_SetCmdSize** (OSPI_Type *pOSPI, uint16_t u16Value)

Set CMD SIZE.

LOCAL_INLINE void **OSPI_HWA_SetCmdIdSize** (OSPI_Type *pOSPI, uint8_t u8SeqId, uint16_t u16Size)

Set CMD id and size.

LOCAL_INLINE uint32_t **OSPI_HWA_GetCmdcValue** (OSPI_Type *pOSPI)

Read the OSPI CMDC register value for all.

LOCAL_INLINE void **OSPI_HWA_SetDataHoldTime** (OSPI_Type *pOSPI, uint8_t u8Value)

Set Data Hold Time, This bit should only be valid in DDR mode.

LOCAL_INLINE void **OSPI_HWA_SetCsHoldTime** (OSPI_Type *pOSPI, uint8_t u8Value)

Set CS Hold Time.

LOCAL_INLINE void **OSPI_HWA_SetCsSetupTime** (OSPI_Type *pOSPI, uint8_t u8Value)

Set CS Setup Time.

LOCAL_INLINE uint32_t **OSPI_HWA_GetFlashCfgValue** (OSPI_Type *pOSPI)

Read the OSPI CMDC register value for all.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkEnable** (OSPI_Type *pOSPI)

OSPI Enable internal reference clock.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkDisable** (OSPI_Type *pOSPI)

OSPI Disable internal reference clock.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkDiv** (OSPI_Type *pOSPI, **OSPI_ClockDivideType** eClkDiv)

OSPI Set Internal Reference Clock Divider.

LOCAL_INLINE void **OSPI_HWA_PadIbeEnable** (OSPI_Type *pOSPI)

OSPI PAD IBE Enable.

LOCAL_INLINE void **OSPI_HWA_PadIbeDisable** (OSPI_Type *pOSPI)

OSPI PAD IBE Disable.

LOCAL_INLINE void **OSPI_HWA_InternalRefclkSource** (OSPI_Type *pOSPI, **OSPI_ClockMuxType** u8Value)

OSPI Select Internal Reference Clock Source.

LOCAL_INLINE void **OSPI_HWA_DqsInvertedEnable** (OSPI_Type *pOSPI)

OSPI Use inverted DQS.

LOCAL_INLINE void **OSPI_HWA_DqsInvertedDisable** (OSPI_Type *pOSPI)

OSPI Not use inverted DQS.

LOCAL_INLINE void **OSPI_HWA_DqsSource** (OSPI_Type *pOSPI, **OSPI_DqsSrcSelType** u8Value)

OSPI Select DQS Source.

LOCAL_INLINE void **OSPI_HWA_ObeTimRelaxEnable** (OSPI_Type *pOSPI)

OSPI OBE timing relax enable.

LOCAL_INLINE void **OSPI_HWA_ObeTimRelaxDisable** (OSPI_Type *pOSPI)

OSPI OBE timing relax disable.

LOCAL_INLINE void **OSPI_HWA_DdrClkEnable** (OSPI_Type *pOSPI)

OSPI DDR Clock Enable.

LOCAL_INLINE void **OSPI_HWA_DdrClkDisable** (OSPI_Type *pOSPI)

OSPI DDR Clock Disable.

LOCAL_INLINE void **OSPI_HWA_CfgDelayLine** (OSPI_Type *pOSPI, uint8_t u8Value)

*OSPI Switch Delay Line into DQS for timing. DQS will delay for DLLINE_CFG * Tcell.*

LOCAL_INLINE void **OSPI_HWA_SetSocCfgValue** (OSPI_Type *pOSPI, uint32_t u32Value)

Write the OSPI SOC CFG register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetSocCfgValue** (OSPI_Type *pOSPI)

Read the OSPI SOC CFG register value for all.

LOCAL_INLINE void **OSPI_HWA_SetFlashAddr** (OSPI_Type *pOSPI, uint32_t u32Value)

Set flash address, users should write the whole value to this register.

LOCAL_INLINE void **OSPI_HWA_SetFlashAddrMode** (OSPI_Type *pOSPI, uint8_t u8Value)

Set flash addressable mode.

LOCAL_INLINE void **OSPI_HWA_SetColAddrSpace** (OSPI_Type *pOSPI, uint8_t u8Value)

Set Column Address Space.

LOCAL_INLINE void **OSPI_HWA_SetSampleDlyClkCycle** (OSPI_Type *pOSPI, uint8_t u8Value)

Set Sample delay clock cycle.

LOCAL_INLINE void **OSPI_HWA_SelectRefClkEdge** (OSPI_Type *pOSPI, uint8_t u8Value)

Selects reference clock edge for valid sampling phase.

LOCAL_INLINE uint16_t **OSPI_HWA_GetRxFifoCnt** (OSPI_Type *pOSPI)

Read RX FIFO Counter, Indicates the current RX FIFO read pointer; Automatically increases when RX FIFO pops an event.

LOCAL_INLINE uint8_t **OSPI_HWA_GetRxFifoFillLevel** (OSPI_Type *pOSPI)

Read RX FIFO Fill Level, Indicates how many words are available in RX FIFO.

LOCAL_INLINE void **OSPI_HWA_SetRxFifoWaterMark** (OSPI_Type *pOSPI, uint8_t u8Value)

Set RX FIFO Watermark, Indicates how many valid entries will trigger a readout action.

LOCAL_INLINE uint16_t **OSPI_HWA_GetTxFifoCnt** (OSPI_Type *pOSPI)

Read TX FIFO Counter, Indicates the current TX FIFO stored data pointer; Automatically increases when TX FIFO pops an event.

LOCAL_INLINE uint8_t **OSPI_HWA_GetTxFifoFillLevel** (OSPI_Type *pOSPI)

Read TX FIFO Fill Level, Indicates how many words are available in TX FIFO.

LOCAL_INLINE uint32_t **OSPI_HWA_GetTFDRAddr** (OSPI_Type *pOSPI)

Get transmit fifo register address.

LOCAL_INLINE void **OSPI_HWA_WriteTxData** (OSPI_Type *pOSPI, uint32_t u32Value)

Writing this register will trigger data entry of TX FIFO.

LOCAL_INLINE void **OSPI_HWA_SetTxFifoWaterMark** (OSPI_Type *pOSPI, uint8_t u8Value)

Set TX FIFO Watermark, Indicates how many valid entries will trigger a transmit action.

LOCAL_INLINE uint32_t **OSPI_HWA_GetStatus** (OSPI_Type *pOSPI)

Read status register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetFlag** (OSPI_Type *pOSPI)

Read flag register.

LOCAL_INLINE void **OSPI_HWA_ClearFlag** (OSPI_Type *pOSPI, uint32_t u32Value)

Writing this register will clear ospi flag.

LOCAL_INLINE void **OSPI_HWA_SetIntDmasEnableReg** (OSPI_Type *pOSPI, uint32_t u32Value)

Set the OSPI IND_EN register value for enable or disable interrupts&DMA.

LOCAL_INLINE void **OSPI_HWA_SetRxDMA** (OSPI_Type *pOSPI, uint8_t u8Enable)

Set the OSPI Rx DMA mode.

LOCAL_INLINE void **OSPI_HWA_SetTxDMA** (OSPI_Type *pOSPI, uint8_t u8Enable)

Set the OSPI Tx DMA mode.

LOCAL_INLINE void **OSPI_HWA_SetFlashTopAddr** (OSPI_Type *pOSPI, uint32_t u32Value)

Set flash top address register.

LOCAL_INLINE uint32_t **OSPI_HWA_GetRFDRAddr** (OSPI_Type *pOSPI)

Get receive fifo register address.

LOCAL_INLINE uint32_t **OSPI_HWA_ReadRcvFifoReg** (OSPI_Type *pOSPI, uint8_t u8Idx)

Read receive fifo data register.

LOCAL_INLINE uint32_t **OSPI_HWA_ReadSecondaryRcvFifoReg** (OSPI_Type *pOSPI)

Read secondary receive fifo data register.

LOCAL_INLINE void **OSPI_HWA_WriteLutKey** (OSPI_Type *pOSPI, uint32_t u32Value)

Write LUT Key register.

LOCAL_INLINE uint32_t **OSPI_HWA_ReadLutKey** (OSPI_Type *pOSPI)

Read LUT Key register.

LOCAL_INLINE void **OSPI_HWA_LockLut** (OSPI_Type *pOSPI)

Lock the LUT.

LOCAL_INLINE void **OSPI_HWA_UnlockLut** (OSPI_Type *pOSPI)

Unlock the LUT.

LOCAL_INLINE void **OSPI_HWA_WriteLut** (OSPI_Type *pOSPI, uint8_t u8Idx, uint32_t u32Value)

Write lookup table register.

Detailed Description

ospi hardware access layer

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_overlay.h File Reference

FC4xxx Overlay hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_overlay.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define OVERLYA_REGION_CNT 3U
```

Overlay Region Count.

```
#define OVERLAY_FAR_SRC 0x08000000U
```

FAR source address.

```
#define OVERLAY_FAR_SIZE_MAX 0x2000000U
```

FAR max size.

```
#define OVERLAY_FAR_SIZE_ALIGN 0x10000U
```

FAR size align.

```
#define OVERLAY_FAR_SIZE_MASK 0xFFFFU
```

FAR size max mask.

Enumerations

```
enum OVERLAY_FARSizeType { OVERLAY_FARUINTSIZE_64KB = 0x10000 }
```

FAR Size.

Functions

LOCAL_INLINE void **OVERLAY_HWA_OverlayEnable** (void)

enable overlay global switch

LOCAL_INLINE void **OVERLAY_HWA_OverlayDisable** (void)

disable overlay global switch

LOCAL_INLINE void **OVERLAY_HWA_FAREnable** (void)

enable far global switch

LOCAL_INLINE void **OVERLAY_HWA_FARDisable** (void)

disable far global switch

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion0Enable** (void)

enable overlay region 0

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion0Disable** (void)

disable overlay region 0

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion0Src** (uint32_t u32Src)

Set Overlay Region 0 Source.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion0Dst** (uint32_t u32Dst)

Set Overlay Region 0 Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion0Size** (OVERLAY_OverlaySizeType eSize)

Set Overlay Region 0 Size.

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion1Enable** (void)

enable overlay region 1

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion1Disable** (void)

disable overlay region 1

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion1Src** (uint32_t u32Src)

Set Overlay Region 1 Source.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion1Dst** (uint32_t u32Dst)

Set Overlay Region 1 Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion1Size** (OVERLAY_OverlaySizeType eSize)

Set Overlay Region 1 Size.

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion2Enable** (void)

enable overlay region 2

LOCAL_INLINE void **OVERLAY_HWA_OverlayRegion2Disable** (void)

disable overlay region 2

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion2Src** (uint32_t u32Src)

Set Overlay Region 2 Source.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion2Dst** (uint32_t u32Dst)

Set Overlay Region 2 Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetOverlayRegion2Size** (OVERLAY_OverlaySizeType eSize)

Set Overlay Region 2 Size.

LOCAL_INLINE void **OVERLAY_HWA_SetFarDst** (uint32_t u32Dst)

Set Far Destination.

LOCAL_INLINE void **OVERLAY_HWA_SetFarSize** (uint32_t u32Size)

Set Far Size.

LOCAL_INLINE void **OVERLAY_HWA_ErrorInterruptEnable** (void)

enable error interrupt

LOCAL_INLINE void **OVERLAY_HWA_ErrorInterruptDisable** (void)
disable error interrupt

LOCAL_INLINE uint32 **OVERLAY_HWA_GetErrorFlag** (void)
get error flag

LOCAL_INLINE uint32 **OVERLAY_HWA_ClrErrorFlag** (uint32_t u32ClrBits)
clear error flag

Detailed Description

FC4xxx Overlay hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_pmc.h File Reference

PMC register API.

```
#include "device_header.h"
```

Include dependency graph for HwA_pmc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE uint32_t **PMC_HWA_GetLVSCR** (void)

get PMC LVCSR register

LOCAL_INLINE void **PMC_HWA_SetLVSCR** (uint32_t u32Val)

set PMC LVCSR register.

LOCAL_INLINE uint32_t **PMC_HWA_GetCONFIG** (void)

get PMC CONFIG register

LOCAL_INLINE void **PMC_HWA_SetCONFIG** (uint32_t u32Val)

set PMC CONFIG register.

LOCAL_INLINE void **PMC_HWA_DisableInterrupt** (uint32 u32Val)

HWA Disable PMC interrupt.

LOCAL_INLINE void **Pmc_HWA_EnableInterrupt** (uint32 u32Val)

HWA Enable PMC interrupt.

LOCAL_INLINE bool **PMC_ReadLVDInterruptFlag** (void)

Return Low Voltage Detect Interrupt Enable Flag.

LOCAL_INLINE bool **PMC_ReadHVDInterruptFlag** (void)

Return High Voltage Detect Interrupt Enable Flag.

Detailed Description

PMC register API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_port.h File Reference

PORT hardware access layer.

#include "device_header.h"

Include dependency graph for HwA_port.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **PORT_HWA_ConfigPin** (PORT_Type *pPort, uint8_t u8Pin, uint32_t u32PcrReg)
Configure pin.

LOCAL_INLINE uint32_t **PORT_HWA_ReadPinInterruptFlag** (PORT_Type *pPort, uint8_t u8Pin)
Read pin interrupt flag.

LOCAL_INLINE void **PORT_HWA_SetPinMux** (PORT_Type *pPort, uint8_t u8Pin, Port_PinMuxType u32PinMux)
Set pin multiplexing.

LOCAL_INLINE void **PORT_HWA_SetPinEmgcyStop** (PORT_Type *pPort, uint8_t u8Pin, bool bEnable)
Set Pin Emergency Stop.

LOCAL_INLINE void **PORT_HWA_SetPinInterruptMode** (PORT_Type *pPort, uint8_t u8Pin, uint32_t u32PinIrqc)
Set pin interrupt mode.

LOCAL_INLINE void **PORT_HWA_SetPinPullEnable** (PORT_Type *pPort, uint8_t u8Pin, uint8_t u8PeValue)
Set pin pull enable.

LOCAL_INLINE void **PORT_HWA_SetPinPullMode** (PORT_Type *pPort, uint8_t u8Pin, uint8_t u8PsValue)

Set pin pull mode.

LOCAL_INLINE void **PORT_HWA_SetPinDriveStrength** (PORT_Type *pPort, uint8_t u8Pin)

Set pin drive strength.

LOCAL_INLINE void **PORT_HWA_SetPinPassiveFilterEnable** (PORT_Type *pPort, uint8_t u8Pin, uint8_t u8PfeValue)

Set pin passive filter enable.

LOCAL_INLINE void **PORT_HWA_ConfigDigitalFilter** (PORT_Type *pPort, uint32_t u32RegValue)

Configure port digital filter.

LOCAL_INLINE void **PORT_HWA_SetDigitalFilterEnable** (PORT_Type *pPort, uint8_t u8RegBit)

Set port digital filter enable.

LOCAL_INLINE void **PORT_HWA_SetDigitalFilterClkSrc** (PORT_Type *pPort)

Set port digital filter clock source.

LOCAL_INLINE void **PORT_HWA_ConfigDigitalFilterWidth** (PORT_Type *pPort, uint32_t u32FilterWidth)

Configure digital filter width.

LOCAL_INLINE void **PORT_HWA_ClearPinInterruptMode** (PORT_Type *pPort, uint8_t u8Pin)

Clear pin interrupt mode.

LOCAL_INLINE void **PORT_HWA_ClearPinInterruptFlag** (PORT_Type *pPort, uint8_t u8Pin)

Clear pin interrupt flag.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterEnable** (PORT_Type *pPort)

Clear port digital filter enable.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterPin** (PORT_Type *pPort, uint8_t u8RegBit)

Clear port digital filter enable for specific pin.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterClkSrc** (PORT_Type *pPort)
Clear port digital filter clock source.

LOCAL_INLINE void **PORT_HWA_ClearDigitalFilterWidth** (PORT_Type *pPort)
Clear port digital filter width.

Detailed Description

PORT hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_ptimer.h File Reference

Hardware access layer for PTIMER.

#include "device_header.h"

Include dependency graph for HwA_ptimer.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum PTIMER_LoadValueModeType { PTIMER_LOAD_VAL_IMMEDIATELY = 0U,  
PTIMER_LOAD_VAL_AT_MODULO_COUNTER = 1U, PTIMER_LOAD_VAL_AT_NEXT_TRIGGER = 2U,  
PTIMER_LOAD_VAL_AT_MODULO_COUNTER_OR_NEXT_TRIGGER = 3U }
```

Ptimer value load mode.

```
enum PTIMER_ClockPreDividerType { PTIMER_PRE_DIVIDE_BY_1 = 0U, PTIMER_PRE_DIVIDE_BY_2 = 1U,  
PTIMER_PRE_DIVIDE_BY_4 = 2U, PTIMER_PRE_DIVIDE_BY_8 = 3U, PTIMER_PRE_DIVIDE_BY_16 = 4U,  
PTIMER_PRE_DIVIDE_BY_32 = 5U, PTIMER_PRE_DIVIDE_BY_64 = 6U, PTIMER_PRE_DIVIDE_BY_128 = 7U }
```

Ptimer clock pre-divider factor.

```
enum PTIMER_ClockPreDivMultiplyFactorType { PTIMER_PRE_DIVIDER_MULTIPLY_BY_1 = 0U,  
PTIMER_PRE_DIVIDER_MULTIPLY_BY_10 = 1U, PTIMER_PRE_DIVIDER_MULTIPLY_BY_20 = 2U,  
PTIMER_PRE_DIVIDER_MULTIPLY_BY_40 = 3U }
```

Ptimer clock divider multiplication factor.

```
enum PTIMER_TrgSrcType { PTIMER_TRGSRC_TRGSEL = 0x00U, PTIMER_TRGSRC_SW = 0x0FU }
```

Ptimer trigger source.

Functions

```
LOCAL_INLINE uint32_t PTIMER_HWA_GetStatusCtrl (const PTIMER_Type *const pPtimer)
```

Get the STATUS_CTRL register value.

```
LOCAL_INLINE void PTIMER_HWA_SetStatusCtrl (PTIMER_Type *const pPtimer, uint32_t u32CfgValue)
```

Set the STATUS_CTRL register value.

LOCAL_INLINE **PTIMER_LoadValueModeType** **PTIMER_HWA_GetLoadMode** (const PTIMER_Type *const pPtimer)

Get the load mode of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetLoadMode** (PTIMER_Type *const pPtimer, **PTIMER_LoadValueModeType** eLoadMode)

Set the load mode of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetSeqErrIntEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether sequence error interrupt is enabled.

LOCAL_INLINE void **PTIMER_HWA_SetSeqErrIntEnableFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable sequence error interrupt.

LOCAL_INLINE void **PTIMER_HWA_GenerateSwTrigger** (PTIMER_Type *const pPtimer)

Generate software trigger signal for Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetDMAEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether DMA is enabled for the Ptimer insstance.

LOCAL_INLINE void **PTIMER_HWA_SetDMAEnableFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable Ptimer DMA.

LOCAL_INLINE **PTIMER_ClockPreDividerType** **PTIMER_HWA_GetDivPrescaler** (const PTIMER_Type *const pPtimer)

Get the predivider value of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetDivPrescaler** (PTIMER_Type *const pPtimer, **PTIMER_ClockPreDividerType** eDivPrescaler)

Set the predivider value of the Ptimer instance.

LOCAL_INLINE **PTIMER_TriggerSrcType** **PTIMER_HWA_GetTriggerSource** (const PTIMER_Type *const pPtimer)

Get the trigger source of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetTriggerSource** (PTIMER_Type *const pPtimer, **PTIMER_TrgSrcType** eTriggerSource)

Set the trigger source of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether the Ptimer instance is enabled.

LOCAL_INLINE void **PTIMER_HWA_Enable** (PTIMER_Type *const pPtimer)

Enable the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_Disable** (PTIMER_Type *const pPtimer)

Disable the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetInterruptFlag** (const PTIMER_Type *const pPtimer)

Get the delay interrupt flag of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_ClearInterruptFlag** (PTIMER_Type *const pPtimer)

Clear the delay interrupt flag of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetInterruptEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether delay interrupt is enabled for the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetInterruptEnableFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable delay interrupt for the Ptimer instance.

LOCAL_INLINE **PTIMER_ClockPreDivMultiplyFactorType** **PTIMER_HWA_GetDivMultiply** (const PTIMER_Type *const pPtimer)

Get the multiply factor of the predivider of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetDivMultiply** (PTIMER_Type *const pPtimer, **PTIMER_ClockPreDivMultiplyFactorType** eMultFactor)

Set the multiply factor of the predivider of the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetContinuoiusModeFlag** (const PTIMER_Type *const pPtimer)

Get whether continuous mode is enabled for the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_SetContinuoiusModeFlag** (PTIMER_Type *const pPtimer, bool bEnable)

Set whether to enable continuous mode for the Ptimer instance.

LOCAL_INLINE bool **PTIMER_HWA_GetValueLoadStatus** (const PTIMER_Type *const pPtimer)

Get the config value loading status.

LOCAL_INLINE void **PTIMER_HWA_LoadValue** (PTIMER_Type *const pPtimer)

Load the buffered values into register.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetMaxCount** (const PTIMER_Type *const pPtimer)

Get the Ptimer max counter period When the Ptimer counter reaches the period, it will return to zero.

LOCAL_INLINE void **PTIMER_HWA_SetMaxCount** (PTIMER_Type *const pPtimer, uint16_t u16MaxCnt)

Set the Ptimer max counter period When the Ptimer counter reaches the period, it will return to zero.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetCounterValue** (const PTIMER_Type *const pPtimer)

Get the Ptimer current count value.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetInterruptDelay** (const PTIMER_Type *const pPtimer)

Get the ptimer interrupt period.

LOCAL_INLINE void **PTIMER_HWA_SetInterruptDelay** (PTIMER_Type *const pPtimer, uint16_t
u16InterruptDelay)

Set the ptimer interrupt period.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelBackToBackFlag** (const PTIMER_Type *const pPtimer,
uint8_t u8Channel)

Get whether back to back trigger is enbaled for the Ptimer channel.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelPretriggerOutputFlag** (const PTIMER_Type *const pPtimer,
uint8_t u8Channel)

Get whether pre-trigger output is enabled for the Ptimer channel.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelPretriggerEnableFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get whether pre-trigger is enabled for the Ptimer channel.

LOCAL_INLINE void **PTIMER_HWA_SetChannelControl** (PTIMER_Type *const pPtimer, uint8_t u8Channel, bool bEnablePretrigger, bool bEnablePretriggerOutput, bool bEnableBackToBack)

Set the Ptimer channel control flags.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelCounterFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get whether the Ptimer counter matches the channel counter.

LOCAL_INLINE void **PTIMER_HWA_ClearChannelCounterFlag** (PTIMER_Type *const pPtimer, uint8_t u8Channel)

Clear the Ptimer channel counter match flag.

LOCAL_INLINE bool **PTIMER_HWA_GetChannelSequenceErrorFlag** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get the sequence error status of the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_ClearChannelSequenceErrorFlag** (PTIMER_Type *const pPtimer, uint8_t u8Channel)

Clear the sequence error flag of the selected Ptimer channel.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetChannelDelay** (const PTIMER_Type *const pPtimer, uint8_t u8Channel)

Get the channel delay value.

LOCAL_INLINE void **PTIMER_HWA_SetChannelDelay** (PTIMER_Type *const pPtimer, uint8_t u8Channel, uint16_t u16Delay)

Set the channel delay value.

LOCAL_INLINE bool **PTIMER_HWA_GetPulseOutEnableFlag** (const PTIMER_Type *const pPtimer)

Get whether pulse-out is enabled.

LOCAL_INLINE void **PTIMER_HWA_EnablePulseOut** (PTIMER_Type *const pPtimer)

Enable pulse-out for the Ptimer instance.

LOCAL_INLINE void **PTIMER_HWA_DisablePulseOut** (PTIMER_Type *const pPtimer)

Disable pulse-out for the Ptimer instance.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetPulseOutDelayHigh** (const PTIMER_Type *const pPtimer)

Get the delay high value for the pulse-out function When the Ptimer counter reach the delay high value, the pulse output goes high.

LOCAL_INLINE uint16_t **PTIMER_HWA_GetPulseOutDelayLow** (const PTIMER_Type *const pPtimer)

Get the delay low value for the pulse-out function When the Ptimer counter reach the delay low value, the pulse output goes low.

LOCAL_INLINE void **PTIMER_HWA_SetPulseOutDelay** (PTIMER_Type *const pPtimer, uint16_t u16DelayHigh, uint16_t u16DelayLow)

Set the pulse out delay value When the Ptimer counter reach the delay high value, the pulse output goes high When the Ptimer counter reach the delay low value, the pulse output goes low The delay high value can be either greater or less than the delay low value.

Detailed Description

Hardware access layer for PTIMER.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_qdt.h File Reference

Hardware access layer for QDT.

```
#include "device_header.h"
```

Include dependency graph for HwA_qdt.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

Hardware access layer for QDT.

Author

Flagchip0084

Version

2.0.0

Date

2024-07-25

Copyright

Copyright (c) 2022 Flagchip Semiconductors Co., Ltd.

HwA_rgm.h File Reference

RGM hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_rgm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum RGM_ResetEventType { RGM_WAKEUP = RGM_SRS_WAKEUP_MASK, RGM_LVD =  
RGM_SRS_LVR_MASK, RGM_CLKERR1 = RGM_SRS_CLKERR1_MASK, RGM_CLKERR0 =  
RGM_SRS_CLKERR0_MASK, RGM_FSCMU = RGM_SRS_FSCMU_MASK, RGM_HSMWDOG =  
RGM_SRS_HSM_WDOG_MASK, RGM_PIN = RGM_SRS_PIN_MASK, RGM_POR = RGM_SRS_POR_MASK,  
RGM_JTAG = RGM_SRS_JTAG_MASK, RGM_SYSAP = RGM_SRS_SYSAP_MASK, RGM_SACKERR =  
RGM_SRS_SACKERR_MASK, RGM_CMU3 = RGM_SRS_CMU3_MASK, RGM_LBIST = RGM_SRS_LBIST_MASK,  
RGM_CPU_LOCKUP = RGM_C0_SRS_C0_LOCKUP_MASK, RGM_CPU_SYSRST =  
RGM_C0_SRS_C0_SYSRST_MASK, RGM_CPU_WDOG = RGM_C0_SRS_C0_WDOG_MASK, RGM_CPU_INTM =  
RGM_C0_SRS_C0_INTM_MASK, RGM_CPU_SWRST = RGM_C0_SRS_C0_SWRST_MASK, RGM_FSMERR =  
RGM_SRS_FSM_ERR_MASK, RGM_PINRSTTOUT = RGM_SRS_PINRST_TOUT_MASK, RGM_SYSRSTTOUT =  
RGM_SRS_SYSRST_TOUT_MASK }
```

Rgm reset event.

```
enum RGM_ResetDelayType { RGM_8_CLOCK_CYCLES = 0U, RGM_32_CLOCK_CYCLES,  
RGM_128_CLOCK_CYCLES, RGM_512_CLOCK_CYCLES }
```

Rgm reset interrupt delay cycles type.

```
enum RGM_SysResetIntMangerType { RGM_INT_CLKERR0 = RGM_SRIE_CLKERR0_RIE_MASK,  
RGM_INT_WDG = RGM_SRIE_WDG_RIE_MASK, RGM_INT_JTAG = RGM_SRIE_JTAG_RIE_MASK,  
RGM_INT_CPULOC = RGM_SRIE_CPULOC_RIE_MASK, RGM_INT_SW = RGM_SRIE_SW_RIE_MASK,  
RGM_INT_SYSAP = RGM_SRIE_SYSAP_RIE_MASK, RGM_INT_SACKERR = RGM_SRIE_SACKERR_RIE_MASK,  
RGM_INT_ALL = RGM_SRIE_ALL_RIE_MASK }
```

Rgm reset interrupt event manger.

```
enum RGM_CoreResetIntMangerType { RGM_CPU_INT_LOCKUP = RGM_C0_CFG_C0_LOCKUP_IE_MASK,  
RGM_CPU_INT_SYSRST = RGM_C0_CFG_C0_SYSRST_IE_MASK, RGM_CPU_INT_WDOG =
```

RGM_C0_CFG_C0_WDOG_IE_MASK, **RGM_CPU_INT_INTM** = RGM_C0_CFG_C0_INTM_IE_MASK,
RGM_CPU_INT_SWRST = RGM_C0_CFG_C0_SWRST_IE_MASK }

Rgm CPU0,1,2,3 related interrupt event manger.

enum **RGM_CoreResetStatusType** { **RGM_REEST_STATUS_UNDER_RESET** = 0U,
RGM_REEST_STATUS_OUT_RESET }

Rgm CPU0,1,2,3 software reset status.

enum **RGM_CoreRstTrigSysRstMangerType** { **RGM_CPU_EN_LOCKUP** =
RGM_C0_CFG_C0_LOCKUP_EN_MASK, **RGM_CPU_EN_SYSRST** = RGM_C0_CFG_C0_SYSRST_EN_MASK,
RGM_CPU_EN_WDOG = RGM_C0_CFG_C0_WDOG_EN_MASK, **RGM_CPU_EN_INTM** =
RGM_C0_CFG_C0_INTM_EN_MASK, **RGM_CPU_EN_SWRST** = RGM_C0_CFG_C0_SWRST_EN_MASK,
RGM_CPU_EN_ALL = 0x1F0000 }

Rgm CPU1,2,3 system reset enable flag.

Functions

LOCAL_INLINE uint32_t **RGM_HWA_ReadLastResetFlag** (void)

Read last reset flag.

LOCAL_INLINE uint32_t **RGM_HWA_ReadAllResetFlagBeforePOR** (void)

Read all reset flag before POR,SSRS register is reset on POR only.

LOCAL_INLINE void **RGM_HWA_SetSrsReg** (uint32_t u32Value)

Set SSRS register.

LOCAL_INLINE void **RGM_HWA_ClearAllResetFlagAfterPOR** (void)

This api can clear all reset flag of SSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

LOCAL_INLINE uint32_t **RGM_HWA_ReadResetPinFilterEnable** (void)

Read reset pin filter register.

LOCAL_INLINE void **RGM_HWA_SetRstfltReg** (uint32_t u32Value)

Set RSTFLT register.

LOCAL_INLINE void **RGM_HWA_SetBusClockFilterWidth** (uint8_t u8Value)

Set reset pin filter bus clock filter width.

LOCAL_INLINE void **RGM_HWA_EnableBusClockFilter** (void)

Enable reset pin filter bus clock.

LOCAL_INLINE void **RGM_HWA_EnableAon32kClockFilter** (void)

Enable reset pin filter AON32K clock.

LOCAL_INLINE void **RGM_HWA_EnableAon32kLPClockFilter** (void)

Enable reset pin filter AON32K low power clock.

LOCAL_INLINE void **RGM_HWA_ClearBusClockFilterWidth** (void)

Clear reset pin filter bus clock filter width.

LOCAL_INLINE void **RGM_HWA_DisableBusClockFilter** (void)

Disable reset pin filter bus clock.

LOCAL_INLINE void **RGM_HWA_DisableAon32kClockFilter** (void)

Disable reset pin filter AON32K clock.

LOCAL_INLINE void **RGM_HWA_DisableAon32kLPClockFilter** (void)

Disable reset pin filter AON32K low power clock.

LOCAL_INLINE void **RGM_HWA_SetSrieReg** (uint32_t u32Value)

Set SRIE register.

LOCAL_INLINE uint32_t **RGM_HWA_GetSrieReg** (void)

Get SRIE register.

LOCAL_INLINE void **RGM_HWA_SetResetDelay** (RGM_ResetDelayType eDelay)

Set Reset delay.

LOCAL_INLINE void **RGM_HWA_EnableGlobalResetInterrupt** (void)

Enable global reset interrupt.

LOCAL_INLINE void **RGM_HWA_EnableResetInterrupt** (RGM_SysResetIntMangerType eResetInterrupt)
Enable reset interrupt.

LOCAL_INLINE void **RGM_HWA_DisableResetInterrupt** (RGM_SysResetIntMangerType eResetInterrupt)
Disable reset interrupt.

LOCAL_INLINE uint32_t **RGM_HWA_GetResetInterrupt** (void)
Get enabling status of system reset.

LOCAL_INLINE void **CM7_HWA_SystemCoreReset** (void)
Perform system core reset.

LOCAL_INLINE void **RGM_HWA_SetC0CfgReg** (uint32_t u32Value)
Set C0_CFG register.

LOCAL_INLINE uint32_t **RGM_HWA_GetC0CfgReg** (void)
Get C0_CFG register.

LOCAL_INLINE void **RGM_HWA_EnableCPU0InterruptFlag** (uint32_t u32CPU0ResetInterrupt)
Enable CPU0 interrupt.

LOCAL_INLINE void **RGM_HWA_DisableCPU0InterruptFlag** (uint32_t u32CPU0ResetInterrupt)
Disable CPU0 interrupt.

LOCAL_INLINE uint32_t **RGM_HWA_GetCPU0ResetInterrupt** (void)
Get enabling status of CPU0 core related reset.

LOCAL_INLINE RGM_CoreResetStatusType **RGM_HWA_GetCPU0OutResetFlag** (void)
Get the CPU0 exit reset flag.

LOCAL_INLINE void **RGM_HWA_CPU0_SystemSoftWareReset** (void)
Issue a CPU0 system software reset.

LOCAL_INLINE uint32_t **RGM_HWA_ReadCPU0LastResetFlag** (void)

Read CPU0 last reset flag.

LOCAL_INLINE uint32_t **RGM_HWA_ReadCPU0AllResetFlagBeforePOR** (void)

Read CPU0 all reset flag before POR,SSRS register is reset on POR only.

LOCAL_INLINE void **RGM_HWA_SetC0SsrsReg** (uint32_t u32Value)

Set C0_SSRS register.

LOCAL_INLINE void **RGM_HWA_ClearC0AllResetFlagAfterPOR** (void)

This api can clear all reset flag of SSRGM_C0_SSRSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

Detailed Description

RGM hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_rtc.h File Reference

rtc hardware access layer

```
#include "device_header.h"
```

Include dependency graph for HwA_rtc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum RTC_ClkoutSecIntFreqType { RTC_FREQ_1HZ = 0, RTC_FREQ_2HZ, RTC_FREQ_4HZ,  
RTC_FREQ_8HZ, RTC_FREQ_16HZ, RTC_FREQ_32hZ, RTC_FREQ_64HZ, RTC_FREQ_128HZ }
```

in the second interrupt mode, this type indicates the interrupt frequency. in the clkout mode , this type indicates the clkout frequency.

Functions

```
LOCAL_INLINE uint32_t RTC_HWA_ReadSecondValue (RTC_Type *const pRtc)
```

Read second value.

```
LOCAL_INLINE bool RTC_HWA_ReadOverflowFlag (RTC_Type *const pRtc)
```

Read RTC SR overflow flag.

```
LOCAL_INLINE bool RTC_HWA_ReadOverflowEnable (RTC_Type *const pRtc)
```

Read RTC IER overflow enable bit.

```
LOCAL_INLINE bool RTC_HWA_ReadAlarmFlag (RTC_Type *const pRtc)
```

Read RTC alarm flag.

```
LOCAL_INLINE bool RTC_HWA_ReadAlarmEnable (RTC_Type *const pRtc)
```

Read RTC alarm Enable.

LOCAL_INLINE bool **RTC_HWA_ReadSecondEnable** (RTC_Type *const pRtc)

Read RTC second interrupt Enable.

LOCAL_INLINE uint8_t **RTC_HWA_ReadCompInterval** (RTC_Type *const pRtc)

Read RTC Compensation Interbal.

LOCAL_INLINE uint8_t **RTC_HWA_ReadCompValue** (RTC_Type *const pRtc)

Read RTC Compensation Value.

LOCAL_INLINE void **RTC_HWA_SetCompInterval** (RTC_Type *const pRtc, uint8_t u8Interval)

Sets the RTC compare interval.

LOCAL_INLINE void **RTC_HWA_SetCompValue** (RTC_Type *const pRtc, uint8_t u8Value)

Sets the RTC compare value.

LOCAL_INLINE void **RTC_HWA_SetPrescalerCounterValue** (RTC_Type *const pRtc, uint16_t u16Value)

Set RTC prescaler register.

LOCAL_INLINE void **RTC_HWA_SetSecondCounterValue** (RTC_Type *const pRtc, uint32_t u32Value)

Set RTC seconds register.

LOCAL_INLINE void **RTC_HWA_SetAlarmCounterValue** (RTC_Type *const pRtc, uint32_t u32Value)

Set RTC alarm value.

LOCAL_INLINE void **RTC_HWA_SetInterruptValue** (RTC_Type *const pRtc, uint32_t u32Value)

Set RTC interrupt value.

LOCAL_INLINE void **RTC_HWA_ConfigControl** (RTC_Type *const pRtc, uint32_t u32Value)

Configure control register.

LOCAL_INLINE void **RTC_HWA_EnableRtcCounter** (RTC_Type *const pRtc)

Enable RTC time counter.

LOCAL_INLINE void **RTC_HWA_SetClkoutFreqStable** (RTC_Type *const pRtc)

Set RTC_CLKOUT is from the 32.768 khz clock.

LOCAL_INLINE void **RTC_HWA_EnableAlarmInterrupt** (RTC_Type *const pRtc, bool bEnable)

Enable alarm interrupt.

LOCAL_INLINE void **RTC_HWA_EnableSecondInterrupt** (RTC_Type *const pRtc, bool bEnable)

Enable second interrupt.

LOCAL_INLINE void **RTC_HWA_EnableOverflowInterrupt** (RTC_Type *const pRtc, bool bEnable)

Enable overflow interrupt.

LOCAL_INLINE void **RTC_HWA_UnlockStatusControlCompensationReg** (RTC_Type *const pRtc)

Unlock lock/status/control/compensation register.

LOCAL_INLINE void **RTC_HWA_SetSecondAndClkoutFreq** (RTC_Type *const pRtc,

RTC_ClkoutSecIntFreqType eFreq)

Set second interrupt and RTC_CLKOUT frequency.

LOCAL_INLINE void **RTC_HWA_DisableRtcCounter** (RTC_Type *const pRtc)

Disable RTC time counter.

LOCAL_INLINE void **RTC_HWA_SetClkoutFromSelectFreq** (RTC_Type *const pRtc)

Set RTC_CLKOUT is from the prescaler output clock selected by IER[TSIC].

LOCAL_INLINE void **RTC_HWA_DisableAlarmInterrupt** (RTC_Type *const pRtc)

Disable alarm interrupt.

LOCAL_INLINE void **RTC_HWA_DisableSecondInterrupt** (RTC_Type *const pRtc)

Disable second interrupt.

LOCAL_INLINE void **RTC_HWA_DisableOverflowInterrupt** (RTC_Type *const pRtc)

Disable overflow interrupt.

Detailed Description

rtc hardware access layer

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_scg.h File Reference

SCG hardware access layer.

#include "device_header.h"

Include dependency graph for HwA_scg.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define SCG_PLLCSR_LK_MASK 0x800000u
```

PLL related.

```
#define SCG_PLLCSR_EN_MASK 0x1u
```

```
#define SCG_PLLCSR_CMRE_MASK 0x20000u
```

```
#define SCG_PLLCSR_CMRE_SHIFT 17u
```

```
#define
```

```
SCG_PLLCSR_CMRE(x) (((uint32_t)((uint32_t)(x))<<SCG_PLLCSR_CMRE_SHIFT)&SCG_PLLCSR_CMRE_MASK)
```

```
#define SCG_PLLCSR_CM_MASK 0x10000u
```

```
#define SCG_PLLCSR_CM_SHIFT 16u
```

```
#define
```

```
SCG_PLLCSR_CM(x) (((uint32_t)((uint32_t)(x))<<SCG_PLLCSR_CM_SHIFT)&SCG_PLLCSR_CM_MASK)
```

```
#define SCG_PLLCSR_STEN_MASK 0x2u
```

```
#define SCG_PLLCSR_STEN_SHIFT 1u
```

```
#define
```

```
SCG_PLLCSR_STEN(x) (((uint32_t)((uint32_t)(x))<<SCG_PLLCSR_STEN_SHIFT)&SCG_PLLCSR_STEN_MASK)
```

```
#define SCG_PLLDIV_DIVL_EN_MASK 0x4000000u
```

```
#define SCG_PLLDIV_DIVL_MASK 0x70000u
```

```
#define SCG_PLLDIV_DIVL_SHIFT 16u
```

```
#define
```

```
SCG_PLLDIV_DIVL(x) (((uint32_t)((uint32_t)(x))<<SCG_PLLDIV_DIVL_SHIFT)&SCG_PLLDIV_DIVL_MASK)
```

```

#define SCG_PLLDIV_DIVM_EN_MASK 0x2000000u
#define SCG_PLLDIV_DIVM_MASK 0x700u
#define SCG_PLLDIV_DIVM_SHIFT 8u
#define
SCG_PLLDIV_DIVM(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVM_SHIFT))&SCG_PLLDIV_DIVM_MASK)
#define SCG_PLLDIV_DIVH_EN_MASK 0x1000000u
#define SCG_PLLDIV_DIVH_MASK 0x7u
#define SCG_PLLDIV_DIVH_SHIFT 0u
#define
SCG_PLLDIV_DIVH(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLDIV_DIVH_SHIFT))&SCG_PLLDIV_DIVH_MASK)
#define SCG_PLLCFG_MULT_MASK 0x1FF0000u
#define SCG_PLLCFG_MULT_SHIFT 16u
#define
SCG_PLLCFG_MULT(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_MULT_SHIFT))&SCG_PLLCFG_MULT_MASK)
#define SCG_PLLCFG_PREDIV_MASK 0x1F00u
#define SCG_PLLCFG_PREDIV_SHIFT 8u
#define
SCG_PLLCFG_PREDIV(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PREDIV_SHIFT))&SCG_PLLCFG_PREDIV_MASK)
#define SCG_PLLCFG_PSTDIV_MASK 0x30u
#define SCG_PLLCFG_PSTDIV_SHIFT 4u
#define
SCG_PLLCFG_PSTDIV(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PSTDIV_SHIFT))&SCG_PLLCFG_PSTDIV_MASK)
#define SCG_PLLCFG_SOURCE_MASK 0x1u
#define SCG_PLLCFG_SOURCE_SHIFT 0u
#define
SCG_PLLCFG_SOURCE(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_SOURCE_SHIFT))&SCG_PLLCFG_SOURCE_MASK)
#define SCG_PLLCFG_PSTDIV1_MASK 0xEu
#define SCG_PLLCFG_PSTDIV1_SHIFT 1u
#define
SCG_PLLCFG_PSTDIV1(x) (((uint32_t)(((uint32_t)(x))<<SCG_PLLCFG_PSTDIV1_SHIFT))&SCG_PLLCFG_PSTDIV1_MASK)
#define _SCG_HWA_SetPllCsr(_pll_, val) SCG->_pll_##CSR = val

```

```

#define SCG_HWA_SetPllCsr(pll, val) _SCG_HWA_SetPllCsr(pll, val)
#define _SCG_HWA_UnlockPllCsr(_pll_) SCG->_pll_##CSR &= ~(uint32_t)SCG_PLLCSR_LK_MASK
#define SCG_HWA_UnlockPllCsr(pll) _SCG_HWA_UnlockPllCsr(pll)
#define _SCG_HWA_LockPllCsr(_pll_) SCG->_pll_##CSR |= (uint32_t)SCG_PLLCSR_LK_MASK
#define SCG_HWA_LockPllCsr(pll) _SCG_HWA_LockPllCsr(pll)
#define _SCG_HWA_EnablePllClockMonitor(_pll_) SCG->_pll_##CSR |=
(uint32_t)SCG_PLLCSR_CM_MASK
#define SCG_HWA_EnablePllClockMonitor(pll) _SCG_HWA_EnablePllClockMonitor(pll)
#define _SCG_HWA_EnablePllClockMonitorReset(_pll_) SCG->_pll_##CSR |=
(uint32_t)SCG_PLLCSR_CMRE_MASK
#define SCG_HWA_EnablePllClockMonitorReset(pll) _SCG_HWA_EnablePllClockMonitorReset(pll)
#define _SCG_HWA_SetPllCfg(_pll_, val) SCG->_pll_##CFG = val
#define SCG_HWA_SetPllCfg(pll, val) _SCG_HWA_SetPllCfg(pll, val)
#define _SCG_HWA_SetPllDiv(_pll_, val) SCG->_pll_##DIV = val
#define SCG_HWA_SetPllDiv(pll, val) _SCG_HWA_SetPllDiv(pll, val)
#define _SCG_HWA_EnablePll(_pll_) SCG->_pll_##CSR |= (uint32_t)(SCG_PLLCSR_EN_MASK)
#define SCG_HWA_EnablePll(pll) _SCG_HWA_EnablePll(pll)
#define _SCG_HWA_EnablePllDiv(_pll_) SCG->_pll_##DIV |= (uint32_t)(SCG_PLLDIV_DIVL_EN_MASK |
SCG_PLLDIV_DIVM_EN_MASK | SCG_PLLDIV_DIVH_EN_MASK)
#define SCG_HWA_EnablePllDiv(pll) _SCG_HWA_EnablePllDiv(pll)
#define _SCG_HWA_DisablePllDiv(_pll_) SCG->_pll_##DIV &= ~(uint32_t)(SCG_PLLDIV_DIVL_EN_MASK |
SCG_PLLDIV_DIVM_EN_MASK | SCG_PLLDIV_DIVH_EN_MASK)
#define SCG_HWA_DisablePllDiv(pll) _SCG_HWA_DisablePllDiv(pll)
#define _SCG_HWA_GetPllMult(_pll_) ((uint16_t)(((uint32_t)(SCG->_pll_##CFG) &
SCG_PLLCFG_MULT_MASK) >> SCG_PLLCFG_MULT_SHIFT))
#define SCG_HWA_GetPllMult(pll) _SCG_HWA_GetPllMult(pll)
#define _SCG_HWA_GetPllPrediv(_pll_) ((uint8_t)(((uint32_t)(SCG->_pll_##CFG) &
SCG_PLLCFG_PREDIV_MASK) >> SCG_PLLCFG_PREDIV_SHIFT))
#define SCG_HWA_GetPllPrediv(pll) _SCG_HWA_GetPllPrediv(pll)
#define _SCG_HWA_GetPllPstdiv(_pll_) ((uint8_t)(((uint32_t)(SCG->_pll_##CFG) &
SCG_PLLCFG_PSTDIV_MASK) >> SCG_PLLCFG_PSTDIV_SHIFT))
#define SCG_HWA_GetPllPstdiv(pll) _SCG_HWA_GetPllPstdiv(pll)
#define _SCG_HWA_GetPllPstdiv1(_pll_) ((uint8_t)(((uint32_t)(SCG->_pll_##CFG) &
SCG_PLLCFG_PSTDIV1_MASK) >> SCG_PLLCFG_PSTDIV1_SHIFT))
#define SCG_HWA_GetPllPstdiv1(pll) _SCG_HWA_GetPllPstdiv1(pll)
#define _SCG_HWA_GetPllSrc(_pll_) ((uint8_t)(((uint32_t)(SCG->_pll_##CFG) &
SCG_PLLCFG_SOURCE_MASK) >> SCG_PLLCFG_SOURCE_SHIFT))
#define SCG_HWA_GetPllSrc(pll) _SCG_HWA_GetPllSrc(pll)

```



```

#define _SCG_HWA_GetPllLocked(_clock_) ((bool)((((uint32_t)SCG->_clock_##CSR &
(uint32_t)SCG_##_clock_##CSR_PLLK_MASK) != 0U) ? true : false))
#define SCG_HWA_GetPllLocked(_clock_) _SCG_HWA_GetPllLocked(_clock_)
#define _SCG_HWA_GetClockValid(_clock_) ((bool)((((uint32_t)SCG->_clock_##CSR &
(uint32_t)SCG_##_clock_##CSR_VLD_MASK) != 0U) ? true : false))
#define SCG_HWA_GetClockValid(_clock_) _SCG_HWA_GetClockValid(_clock_)
#define _SCG_HWA_GetClockDiv(_clock_) (uint32_t)SCG->_clock_##DIV
#define SCG_HWA_GetClockDiv(_clock_) _SCG_HWA_GetClockDiv(_clock_)
#define _SCG_HWA_GetTrimLocked(_clock_) ((bool)((((uint32_t)SCG->_clock_##CSR &
(uint32_t)SCG_##_clock_##CSR_TRLK_MASK) != 0U) ? true : false))
#define SCG_HWA_GetTrimLocked(_clock_) _SCG_HWA_GetTrimLocked(_clock_)
#define SCG_CLOCKDIV_DIVL_ACK_MASK 0x40000000u
#define SCG_CLOCKDIV_DIVM_ACK_MASK 0x20000000u
#define SCG_CLOCKDIV_DIVH_ACK_MASK 0x10000000u
#define SCG_CLOCKDIV_DIV_ACK_MASK 0x70000000u
#define SCG_CLOCKDIV_DIV_ACK_MASK_SHIFT 0x7u
#define SCG_CLOCKDIV_DIVL_ACK_SHIFT 30u
#define SCG_CLOCKDIV_DIVM_ACK_SHIFT 29u
#define SCG_CLOCKDIV_DIVH_ACK_SHIFT 28u
#define SCG_CLOCKDIV_DIV_ACK_SHIFT 28u
#define SCG_CLOCKDIV_DIVL_EN_MASK 0x40000000u
#define SCG_CLOCKDIV_DIVM_EN_MASK 0x20000000u
#define SCG_CLOCKDIV_DIVH_EN_MASK 0x10000000u
#define SCG_CLOCKDIV_DIVL_EN_SHIFT 26u
#define SCG_CLOCKDIV_DIVM_EN_SHIFT 25u
#define SCG_CLOCKDIV_DIVH_EN_SHIFT 24u
#define SCG_CLOCKDIV_DIVL_VAL_MASK 0x70000u
#define SCG_CLOCKDIV_DIVM_VAL_MASK 0x700u
#define SCG_CLOCKDIV_DIVH_VAL_MASK 0x7u
#define SCG_CLOCKDIV_DIVL_VAL_SHIFT 16u
#define SCG_CLOCKDIV_DIVM_VAL_SHIFT 8u
#define SCG_CLOCKDIV_DIVH_VAL_SHIFT 0u
#define SCG_CHECK_DIVL_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIVL_ACK_MASK) >>
SCG_CLOCKDIV_DIVL_ACK_SHIFT)
#define SCG_CHECK_DIVM_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIVM_ACK_MASK) >>
SCG_CLOCKDIV_DIVM_ACK_SHIFT)
#define SCG_CHECK_DIVH_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIVH_ACK_MASK) >>
SCG_CLOCKDIV_DIVH_ACK_SHIFT)

```

```

#define SCG_CHECK_DIV_ACK(reg_val) ((reg_val & SCG_CLOCKDIV_DIV_ACK_MASK) >>
SCG_CLOCKDIV_DIV_ACK_SHIFT)
#define SCG_CHECK_DIVL_EN(reg_val) ((reg_val & SCG_CLOCKDIV_DIVL_EN_MASK) >>
SCG_CLOCKDIV_DIVL_EN_SHIFT)
#define SCG_CHECK_DIVM_EN(reg_val) ((reg_val & SCG_CLOCKDIV_DIVM_EN_MASK) >>
SCG_CLOCKDIV_DIVM_EN_SHIFT)
#define SCG_CHECK_DIVH_EN(reg_val) ((reg_val & SCG_CLOCKDIV_DIVH_EN_MASK) >>
SCG_CLOCKDIV_DIVH_EN_SHIFT)
#define SCG_GET_DIVL_VAL(reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVL_VAL_MASK) >>
SCG_CLOCKDIV_DIVL_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVL_VAL_MASK) >>
SCG_CLOCKDIV_DIVL_VAL_SHIFT))
#define SCG_GET_DIVM_VAL(reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVM_VAL_MASK) >>
SCG_CLOCKDIV_DIVM_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVM_VAL_MASK) >>
SCG_CLOCKDIV_DIVM_VAL_SHIFT))
#define SCG_GET_DIVH_VAL(reg_val) ((0U == ((reg_val & SCG_CLOCKDIV_DIVH_VAL_MASK) >>
SCG_CLOCKDIV_DIVH_VAL_SHIFT)) ? 1U : ((reg_val & SCG_CLOCKDIV_DIVH_VAL_MASK) >>
SCG_CLOCKDIV_DIVH_VAL_SHIFT))
#define SCG_CALCULATE_DIVL_FREQ(clk_freq, reg_val) (((1U == SCG_CHECK_DIVL_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVL_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)
#define SCG_CALCULATE_DIVM_FREQ(clk_freq, reg_val) (((1U == SCG_CHECK_DIVM_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVM_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)
#define SCG_CALCULATE_DIVH_FREQ(clk_freq, reg_val) (((1U == SCG_CHECK_DIVH_EN(reg_val)) &&
(clk_freq != UNKNOWN_CLOCK)) ? (uint32_t)clk_freq >> (uint8_t)(SCG_GET_DIVH_VAL(reg_val) - 1U) :
(uint32_t)UNKNOWN_CLOCK)

```

Functions

LOCAL_INLINE uint8_t **SCG_HWA_GetClkOutSel** (void)

Get clock out configure register CLKOUTSEL value.

LOCAL_INLINE uint32_t **SCG_HWA_GetClkOutCfg** (void)

Get clock out configure register value.

LOCAL_INLINE void **SCG_HWA_UnlockSoscCsr** (void)

Unlock SOSC CSR register.

LOCAL_INLINE void **SCG_HWA_LockSoscCsr** (void)

Lock SOSC CSR register.

LOCAL_INLINE void **SCG_HWA_EnableSoscClockMonitor** (void)

Enable SOSC clock monitor.

LOCAL_INLINE void **SCG_HWA_EnableSoscClockMonitorReset** (void)

Enable SOSC clock monitor Reset.

LOCAL_INLINE void **SCG_HWA_EnableSosc** (void)

Set SOSC enable.

LOCAL_INLINE void **SCG_HWA_DisableSosc** (void)

Disable SOSC.

LOCAL_INLINE void **SCG_HWA_SetSoscCfg** (uint32_t u32CfgValue)

Set SOSC CFG register value.

LOCAL_INLINE void **SCG_HWA_SetSoscCsr** (uint32_t u32CsrValue)

Set SOSC CSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetSoscCsr** (void)

Get SOSC CSR register value.

LOCAL_INLINE void **SCG_HWA_EnableFosc** (void)

Enable FOSC.

LOCAL_INLINE void **SCG_HWA_DisableFosc** (void)

Disable FOSC.

LOCAL_INLINE void **SCG_HWA_SetFoscCfg** (uint32_t u32CfgValue)

Set FOSCCFG register value.

LOCAL_INLINE void **SCG_HWA_SetFoscCsr** (uint32_t u32CsrValue)

Set FOSCCSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetFoscCsr** (void)

Get FOSCCSR register value.

LOCAL_INLINE void **SCG_HWA_UnlockFoscCsr** (void)

Unlock FOSC CSR register.

LOCAL_INLINE void **SCG_HWA_LockFoscCsr** (void)

lock FOSC CSR register

LOCAL_INLINE void **SCG_HWA_EnableFoscClockMonitor** (void)

Enable FOSC clock monitor.

LOCAL_INLINE void **SCG_HWA_EnableFoscClockMonitorReset** (void)

Enable FOSC clock monitor Reset.

LOCAL_INLINE void **SCG_HWA_EnableFoscDiv** (void)

enable FOSCDIV as user manual request sequence

LOCAL_INLINE void **SCG_HWA_DiableFoscDiv** (void)

Disable FOSCDIV as user manual request sequence.

LOCAL_INLINE void **SCG_HWA_SetFoscDiv** (uint32_t u32DivValue)

Set FOSCDIV register value.

LOCAL_INLINE void **SCG_HWA_SetSircCsr** (uint32_t u32CsrValue)

Set SIRCCSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetSircCsr** (void)

Get SIRCCSR register value.

LOCAL_INLINE void **SCG_HWA_UnlockSircCsr** (void)

Unlock SIRC CSR register.

LOCAL_INLINE void **SCG_HWA_LockSircCsr** (void)

Lock SIRC CSR register.

LOCAL_INLINE void **SCG_HWA_EnableSircClockMonitor** (void)

Enable SIRC clock monitor.

LOCAL_INLINE void **SCG_HWA_SetSircDiv** (uint32_t u32DivValue)

Set SIRC DIV register value.

LOCAL_INLINE void **SCG_HWA_DisableSircDiv** (void)

Disable SIRC DIV as user manual request sequence.

LOCAL_INLINE void **SCG_HWA_EnableSircDiv** (void)

enable SIRC DIV as user manual request sequence

LOCAL_INLINE void **SCG_HWA_SetSircTcfg** (uint32_t u32TcfgValue)

Set SIRC TCFG register value for SIRC Trim configure.

LOCAL_INLINE void **SCG_HWA_EnableSirc32kCsr** (void)

Enable SIRC32KCSR register.

LOCAL_INLINE void **SCG_HWA_DisableSirc32kCsr** (void)

Disable SIRC32KCSR register.

LOCAL_INLINE void **SCG_HWA_UnlockSirc32kCsr** (void)

Unlock SIRC32K CSR register.

LOCAL_INLINE void **SCG_HWA_LockSirc32kCsr** (void)

Lock SIRC32K CSR register.

LOCAL_INLINE uint32_t **SCG_HWA_GetFircCsr** (void)

Get FIRC CSR register value.

LOCAL_INLINE void **SCG_HWA_EnableFirc** (void)

Enable FIRC.

LOCAL_INLINE void **SCG_HWA_UnlockFircCsr** (void)

Unlock FIRC CSR register.

LOCAL_INLINE void **SCG_HWA_LockFircCsr** (void)

Lock FIRC CSR register.

LOCAL_INLINE void **SCG_HWA_EnableFircClockMonitor** (void)

Enable FIRC clock monitor.

LOCAL_INLINE void **SCG_HWA_DisableFirc** (void)

Disable FIRC.

LOCAL_INLINE void **SCG_HWA_SetFircCsr** (uint32_t u32CsrValue)

Set FIRCCSR register value.

LOCAL_INLINE void **SCG_HWA_SetFircCfg** (uint32_t u32CfgValue)

Set FIRCCFG register value.

LOCAL_INLINE void **SCG_HWA_SetFircDiv** (uint32_t u32DivValue)

Set FOSCDIV register value.

LOCAL_INLINE void **SCG_HWA_DiableFircDiv** (void)

Disable FIRCDIV as user manual request sequence.

LOCAL_INLINE void **SCG_HWA_EnableFircDiv** (void)

enable FIRCDIV as user manual request sequence

LOCAL_INLINE void **SCG_HWA_SetFircTcfg** (uint32_t u32TcfgValue)

Set FIRCTCFG register value for FIRC clock trim configure.

LOCAL_INLINE void **SCG_HWA_SetCCR** (uint32_t u32CcrValue)

Set system clock CCR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetCCR** (void)

Set system clock CCR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetPll0Csr** (void)

Get PLL0CSR register value.

LOCAL_INLINE uint32_t **SCG_HWA_GetPll1Csr** (void)

Get PLL1CSR register value.

LOCAL_INLINE void **SCG_HWA_SetSystemClock** (uint8_t u8SystemClock)

Set system clock.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkSrc** (void)

Get system clock source. used to calculate system clock frequency at startup. used to check if the target clock soure successfully switched.

LOCAL_INLINE bool **SCG_HWA_GetSysClkUPRD** (void)

Get system clock valid status, use this status to check system clock update finished or not.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkDivCore** (void)

Get system clock Divcore. used to calculate core clock frequency at startup.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkDivBus** (void)

Get system clock Divbus. used to calculate bus clock frequency at startup.

LOCAL_INLINE uint8_t **SCG_HWA_GetSysClkDivSlow** (void)

Get system clock Divslow. used to calculate slow clock frequency at startup.

LOCAL_INLINE void **SCG_HWA_SetClkOutSel** (uint8_t u8ClkOutSel)

Select clock out source.

LOCAL_INLINE void **SCG_HWA_SetNvmClk** (uint32_t u32NvmClkMask)

Set Nvm clock source.

LOCAL_INLINE void **SCG_HWA_SetCmu4Clk** (uint32_t u32Cmu4ClkMask)

Set CMU4 clock source.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearSircClkErr** (void)

Check and clear SIRC clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearSoscClkErr** (void)

Check and clear SOSC clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearFoscClkErr** (void)

Check and clear FOSC clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearPl10ClkErr** (void)

Check and clear PLL0 clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearPl11ClkErr** (void)

Check and clear PLL1 clock error.

LOCAL_INLINE bool **SCG_HWA_CheckAndClearFircClkErr** (void)

Check and clear FIRC clock error.

LOCAL_INLINE void **SCG_HWA_GenCrcVal** (void)

Generate SCG register CRC value.

LOCAL_INLINE void **SCG_HWA_EnableCrcCheck** (void)

Enable SCG register CRC check.

LOCAL_INLINE void **SCG_HWA_DisableCrcCheck** (void)

Disable SCG register CRC check.

LOCAL_INLINE void **SCG_HWA_EnableCrcTrigger** (void)

Enable SCG register CRC hardware trigger generate.

LOCAL_INLINE void **SCG_HWA_DisableCrcTrigger** (void)

Disable SCG register CRC hardware trigger generate.

LOCAL_INLINE void **SCG_HWA_EnableCrcErrorOutput** (void)

Enable SCG register CRC error output.

LOCAL_INLINE void **SCG_HWA_DisableCrcErrorOutput** (void)

Disable SCG register CRC error output.

LOCAL_INLINE bool **SCG_HWA_GetCrcBusyStatus** (void)

Get CRC busy status.

LOCAL_INLINE bool **SCG_HWA_GetCrcErrorStatus** (void)

Get CRC error status.

LOCAL_INLINE void **SCG_HWA_ClearCrcErrorFlag** (void)

Clear CRC error flag.

LOCAL_INLINE uint32_t **SCG_HWA_GetCrcResult** (void)

Get CRC result.

Detailed Description

SCG hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_scm.h File Reference

Hardware access layer for SCM.

```
#include "device_header.h"
```

Include dependency graph for HwA_scm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE uint32_t **SCM_HWA_GetData_UIDL** (void)

Get UIDL data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint32_t **SCM_HWA_GetData_UIDML** (void)

Get UIDML data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint32_t **SCM_HWA_GetData_UIDMH** (void)

Get UIDMH data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint32_t **SCM_HWA_GetData_UIDH** (void)

Get UIDH data(Unique identification for the chip. Loaded from NVR)

LOCAL_INLINE uint8_t **SCM_HWA_GetData_FamilyID** (void)

Get Family Identification.

LOCAL_INLINE uint8_t **SCM_HWA_GetData_RevID** (void)

Get Revision Identification.

LOCAL_INLINE uint32_t **SCM_HWA_Get_CHIPCFG0** (void)

Get CHIPCFG0 register status.

LOCAL_INLINE void **SCM_HWA_Set_SOCMISC** (uint32_t u32Val)

Set the SOCmisc register value.

LOCAL_INLINE uint32_t **SCM_HWA_Get_SOCMISC** (void)

Get the SOCmisc register status.

LOCAL_INLINE uint32_t **SCM_HWA_Get_SYSAP_CTRL** (void)

Get SYSAP_CTRL register status.

LOCAL_INLINE void **SCM_HWA_Set_FTU_GTBC** (uint32_t u32Val)

Set the FTU_GTBC register value.

LOCAL_INLINE void **SCM_HWA_SUBSYS_PCC_SetEnable_TPUclock** (bool bEnable)

Set TPU clock enable/disable.

LOCAL_INLINE void **SCM_HWA_SUBSYS_PCC_SetEnable_SubSystemClock** (bool bEnable)

Set SubSystem clock enable/disable.

Detailed Description

Hardware access layer for SCM.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_sdadc.h File Reference

Hardware access layer for SDADC.

#include "device_header.h"

Include dependency graph for HwA_sdadc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Classes

struct **SDADC_ChannelRegType**

Macros

#define **SDADC_MSR_CHON_MASK** 0x7u

#define **SDADC_MSR_CHON_SHIFT** 0u

#define **SDADC_MSR_CHON_WIDTH** 3u

#define

SDADC_MSR_CHON(x) (((uint32_t)(((uint32_t)(x))<<**SDADC_MSR_CHON_SHIFT**))&**SDADC_MSR_CHON_MASK**)

#define **SDADC_CHANNELCFGREG**(base) ((**pSDADC_ChannelRegType**)((int)&((SDADC_Type *)0)->CCR0) + (int)(base)))

Typedefs

typedef struct **SDADC_ChannelRegType** * **pSDADC_ChannelRegType**

Functions

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO2_DIS** (const SDADC_Type *const pSdadc)

Get the value of CLKO2_DIS. Modulator Clock Output 2 Disable 0b - CLKO2 is enabled when MEN is asserted. 1b - CLKO2 is disabled.

LOCAL_INLINE void **SDADC_HWA_SetCLKO2_DIS** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set CLKO2_DIS. Modulator Clock Output 2 Disable 0b - CLKO2 is enabled when MEN is asserted. 1b - CLKO2 is disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO1_DIS** (const SDADC_Type *const pSdadc)

Get the value of CLKO1_DIS. Modulator Clock Output 1 Disable 0b - CLKO1 is enabled when MEN is asserted. 1b - CLKO1 is disabled.

LOCAL_INLINE void **SDADC_HWA_SetCLKO1_DIS** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set CLKO1_DIS. Modulator Clock Output 1 Disable 0b - CLKO1 is enabled when MEN is asserted. 1b - CLKO1 is disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO0_DIS** (const SDADC_Type *const pSdadc)

Get the value of CLKO0_DIS. Modulator Clock Output 0 Disable 0b - CLKO0 is enabled when MEN is asserted. 1b - CLKO0 is disabled.

LOCAL_INLINE void **SDADC_HWA_SetCLKO0_DIS** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set CLKO0_DIS. Modulator Clock Output 0 Disable 0b - CLKO0 is enabled when MEN is asserted. 1b - CLKO0 is disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPRESCALE** (const SDADC_Type *const pSdadc)

Get the value of PRESCALE. PRE_DIV Divider Value 00b - No prescale. 01b - Divided by 2. 10b - Divided by 4. 11b - Divided by 8.

LOCAL_INLINE void **SDADC_HWA_SetPRESCALE** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set PRESCALE. PRE_DIV Divider Value 00b - No prescale. 01b - Divided by 2. 10b - Divided by 4. 11b - Divided by 8.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCLK_DIV** (const SDADC_Type *const pSdadc)

Get the value of MCLK_DIV. DIV Divider Value The (MCLK_DIV+1) sets the clock divider ratio for DIV block. The minimum value MCLK_DIV is 1.

LOCAL_INLINE void **SDADC_HWA_SetMCLK_DIV** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set MCLK_DIV. DIV Divider Value The (MCLK_DIV+1) sets the clock divider ratio for DIV block. The minimum value MCLK_DIV is 1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMEN** (const SDADC_Type *const pSdadc)

Get the value of MEN. Main Enable Setting this bit enables all Function block simultaneously if CHEN and corresponding enable bit are asserted. 0b - All function blocks are disabled. 1b - All function blocks are enabled simultaneously.

LOCAL_INLINE void **SDADC_HWA_SetMEN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set MEN. Main Enable Setting this bit enables all Function block simultaneously if CHEN and corresponding enable bit are asserted. 0b - All function blocks are disabled. 1b - All function blocks are enabled simultaneously.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMENSEL** (const SDADC_Type *const pSadc)

Get the value of MENSEL. Main Enable Selection Setting this bit to select main enable(MEN) from an enable from external of SDADC IP. By this way, all SDADC in a system can share one main enable and can be enable at the same time. 0b - Use internal MEN. 1b - Use external MEN.

LOCAL_INLINE void **SDADC_HWA_SetMENSEL** (SDADC_Type *const pSadc, uint32_t u32Value)

Set MENSEL. Main Enable Selection Setting this bit to select main enable(MEN) from an enable from external of SDADC IP. By this way, all SDADC in a system can share one main enable and can be enable at the same time. 0b - Use internal MEN. 1b - Use external MEN.

LOCAL_INLINE uint32_t **SDADC_HWA_GetRESET** (const SDADC_Type *const pSadc)

Get the value of RESET. Software Reset Used to reset all function blocks (filters, short circuit detect, etc) and all flags or status. Note, to reset all function domain, RESET must keep high at least 3 div_clk1 cycles. 0b - No reset. 1b - All function blocks are reset and disabled.

LOCAL_INLINE void **SDADC_HWA_SetRESET** (SDADC_Type *const pSadc, uint32_t u32Value)

Set RESET. Software Reset Used to reset all function blocks (filters, short circuit detect, etc) and all flags or status. Note, to reset all function domain, RESET must keep high at least 3 div_clk1 cycles. 0b - No reset. 1b - All function blocks are reset and disabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDBGM** (const SDADC_Type *const pSadc)

Get the value of DBGM. Software Debug Mode Request This bit is writable only when MEN is asserted. 0b - SDADC is in Normal Mode. 1b - SDADC is in Debug Mode.

LOCAL_INLINE void **SDADC_HWA_SetDBGM** (SDADC_Type *const pSadc, uint32_t u32Value)

Set DBGM. Software Debug Mode Request This bit is writable only when MEN is asserted. 0b - SDADC is in Normal Mode. 1b - SDADC is in Debug Mode.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDBGE** (const SDADC_Type *const pSdadc)

Get the value of DBGE. Module Enable in Debug Mode Enables/disables the SDADC operation in Debug mode. 0b - SDADC is enabled in debug mode. 1b - SDADC is disabled in debug mode. Need trigger again for new conversion.

LOCAL_INLINE void **SDADC_HWA_SetDBGE** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set DBGE. Module Enable in Debug Mode Enables/disables the SDADC operation in Debug mode. 0b - SDADC is enabled in debug mode. 1b - SDADC is disabled in debug mode. Need trigger again for new conversion.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSTRIG2** (const SDADC_Type *const pSdadc)

Get the value of STRIG2. Software Trigger for Filter Channel 2.

LOCAL_INLINE void **SDADC_HWA_SetSTRIG2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set STRIG2. Software Trigger for Filter Channel 2.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSTRIG1** (const SDADC_Type *const pSdadc)

Get the value of STRIG1. Software Trigger for Filter Channel 1.

LOCAL_INLINE void **SDADC_HWA_SetSTRIG1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set STRIG1. Software Trigger for Filter Channel 1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSTRIG0** (const SDADC_Type *const pSdadc)

Get the value of STRIG0. Software Trigger for Filter Channel 0.

LOCAL_INLINE void **SDADC_HWA_SetSTRIG0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set STRIG0. Software Trigger for Filter Channel 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCDIE2** (const SDADC_Type *const pSdadc)

Get the value of ZCDIE2. Filter Channel 2 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetZCDIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set ZCDIE2. Filter Channel 2 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCDIE1** (const SDADC_Type *const pSadc)

Get the value of ZCDIE1. Filter Channel 1 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetZCDIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set ZCDIE1. Filter Channel 1 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCDIE0** (const SDADC_Type *const pSadc)

Get the value of ZCDIE0. Filter Channel 0 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetZCDIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set ZCDIE0. Filter Channel 0 Zero Cross Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDYIE2** (const SDADC_Type *const pSadc)

Get the value of FRDYIE2. Filter Channel 2 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFRDYIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FRDYIE2. Filter Channel 2 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDYIE1** (const SDADC_Type *const pSadc)

Get the value of FRDYIE1. Filter Channel 1 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFRDYIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FRDYIE1. Filter Channel 1 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDYIE0** (const SDADC_Type *const pSadc)

Get the value of FRDYIE0. Filter Channel 0 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFRDYIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FRDYIE0. Filter Channel 0 FIFO Data Ready Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOCIE2** (const SDADC_Type *const pSadc)

Get the value of COCIE2. Filter Channel 2 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCOCIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set COCIE2. Filter Channel 2 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOCIE1** (const SDADC_Type *const pSadc)

Get the value of COCIE1. Filter Channel 1 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCOCIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set COCIE1. Filter Channel 1 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOCIE0** (const SDADC_Type *const pSadc)

Get the value of COCIE0. Filter Channel 0 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCOCIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set COCIE0. Filter Channel 0 Conversion Complete Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMTIE2** (const SDADC_Type *const pSdadc)

Get the value of HLMTIE2. Filter Channel 2 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetHLMTIE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set HLMTIE2. Filter Channel 2 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMTIE1** (const SDADC_Type *const pSdadc)

Get the value of HLMTIE1. Filter Channel 1 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetHLMTIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set HLMTIE1. Filter Channel 1 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMTIE0** (const SDADC_Type *const pSdadc)

Get the value of HLMTIE0. Filter Channel 0 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetHLMTIE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set HLMTIE0. Filter Channel 0 High Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMTIE2** (const SDADC_Type *const pSdadc)

Get the value of LLMTIE2. Filter Channel 2 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetLLMTIE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set LLMTIE2. Filter Channel 2 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMTIE1** (const SDADC_Type *const pSdadc)

Get the value of LLMTIE1. Filter Channel 1 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetLLMTIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set LLMTIE1. Filter Channel 1 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMTIE0** (const SDADC_Type *const pSdadc)

Get the value of LLMTIE0. Filter Channel 0 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetLLMTIE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set LLMTIE0. Filter Channel 0 Low Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMTIE2** (const SDADC_Type *const pSdadc)

Get the value of WLMTIE2. Filter Channel 2 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetWLMTIE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set WLMTIE2. Filter Channel 2 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMTIE1** (const SDADC_Type *const pSdadc)

Get the value of WLMTIE1. Filter Channel 1 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetWLMTIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set WLMTIE1. Filter Channel 1 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMTIE0** (const SDADC_Type *const pSdadc)

Get the value of WLMTIE0. Filter Channel 0 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetWLMTIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set WLMTIE0. Filter Channel 0 Window Limit Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDIE2** (const SDADC_Type *const pSadc)

Get the value of SCDIE2. Filter Channel 2 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSCDIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SCDIE2. Filter Channel 2 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDIE1** (const SDADC_Type *const pSadc)

Get the value of SCDIE1. Filter Channel 1 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSCDIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SCDIE1. Filter Channel 1 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDIE0** (const SDADC_Type *const pSadc)

Get the value of SCDIE0. Filter Channel 0 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSCDIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SCDIE0. Filter Channel 0 Short Circuit Detected Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSATIE2** (const SDADC_Type *const pSadc)

Get the value of SATIE2. Filter Channel 2 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSATIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set SATIE2. Filter Channel 2 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSATIE1** (const SDADC_Type *const pSdadc)

Get the value of SATIE1. Filter Channel 1 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSATIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set SATIE1. Filter Channel 1 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSATIE0** (const SDADC_Type *const pSdadc)

Get the value of SATIE0. Filter Channel 0 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSATIE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set SATIE0. Filter Channel 0 Saturation Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADIE2** (const SDADC_Type *const pSdadc)

Get the value of CADIE2. Filter Channel 2 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCADIE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set CADIE2. Filter Channel 2 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADIE1** (const SDADC_Type *const pSdadc)

Get the value of CADIE1. Filter Channel 1 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCADIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set CADIE1. Filter Channel 1 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADIE0** (const SDADC_Type *const pSadc)

Get the value of CADIE0. Filter Channel 0 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCADIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CADIE0. Filter Channel 0 Clock Absence Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVFIE2** (const SDADC_Type *const pSadc)

Get the value of FOVFIE2. Filter Channel 2 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFOVFIE2** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FOVFIE2. Filter Channel 2 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVFIE1** (const SDADC_Type *const pSadc)

Get the value of FOVFIE1. Filter Channel 1 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFOVFIE1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FOVFIE1. Filter Channel 1 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVFIE0** (const SDADC_Type *const pSadc)

Get the value of FOVFIE0. Filter Channel 0 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFOVFIE0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set FOVFIE0. Filter Channel 0 FIFO Overflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNFIE2** (const SDADC_Type *const pSadc)

Get the value of FUNFIE2. Filter Channel 2 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFUNFIE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set FUNFIE2. Filter Channel 2 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNFIE1** (const SDADC_Type *const pSdadc)

Get the value of FUNFIE1. Filter Channel 1 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFUNFIE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set FUNFIE1. Filter Channel 1 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNFIE0** (const SDADC_Type *const pSdadc)

Get the value of FUNFIE0. Filter Channel 0 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetFUNFIE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set FUNFIE0. Filter Channel 0 FIFO Underflow Interrupt Enable 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD2** (const SDADC_Type *const pSdadc)

Get the value of ZCD2. Filter Channel 2 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearZCD2** (SDADC_Type *const pSdadc)

Clear ZCD2. Filter Channel 2 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD1** (const SDADC_Type *const pSdadc)

Get the value of ZCD1. Filter Channel 1 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearZCD1** (SDADC_Type *const pSdac)

Clear ZCD1. Filter Channel 1 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD0** (const SDADC_Type *const pSdac)

Get the value of ZCD0. Filter Channel 0 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearZCD0** (SDADC_Type *const pSdac)

Clear ZCD0. Filter Channel 0 Zero Cross Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDY2** (const SDADC_Type *const pSdac)

Get the value of FRDY2. Filter Channel 2 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFRDY2** (SDADC_Type *const pSdac)

Clear FRDY2. Filter Channel 2 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDY1** (const SDADC_Type *const pSdac)

Get the value of FRDY1. Filter Channel 1 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFRDY1** (SDADC_Type *const pSdac)

Clear FRDY1. Filter Channel 1 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFRDY0** (const SDADC_Type *const pSdac)

Get the value of FRDY0. Filter Channel 0 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFRDY0** (SDADC_Type *const pSdac)

Clear FRDY0. Filter Channel 0 FIFO Data Ready Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOC2** (const SDADC_Type *const pSdac)

Get the value of COC2. Filter Channel 2 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCOC2** (SDADC_Type *const pSadc)

Clear COC2. Filter Channel 2 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOC1** (const SDADC_Type *const pSadc)

Get the value of COC1. Filter Channel 1 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCOC1** (SDADC_Type *const pSadc)

Clear COC1. Filter Channel 1 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCOC0** (const SDADC_Type *const pSadc)

Get the value of COC0. Filter Channel 0 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCOC0** (SDADC_Type *const pSadc)

Clear COC0. Filter Channel 0 Conversion Complete Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT2** (const SDADC_Type *const pSadc)

Get the value of HLMT2. Filter Channel 2 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearHLMT2** (SDADC_Type *const pSadc)

Clear HLMT2. Filter Channel 2 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT1** (const SDADC_Type *const pSadc)

Get the value of HLMT1. Filter Channel 1 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearHLMT1** (SDADC_Type *const pSadc)

Clear HLMT1. Filter Channel 1 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT0** (const SDADC_Type *const pSadc)

Get the value of HLMT0. Filter Channel 0 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearHLMT0** (SDADC_Type *const pSdadc)

Clear HLMT0. Filter Channel 0 High Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT2** (const SDADC_Type *const pSdadc)

Get the value of LLMT2. Filter Channel 2 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearLLMT2** (SDADC_Type *const pSdadc)

Clear LLMT2. Filter Channel 2 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT1** (const SDADC_Type *const pSdadc)

Get the value of LLMT1. Filter Channel 1 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearLLMT1** (SDADC_Type *const pSdadc)

Clear LLMT1. Filter Channel 1 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT0** (const SDADC_Type *const pSdadc)

Get the value of LLMT0. Filter Channel 0 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearLLMT0** (SDADC_Type *const pSdadc)

Clear LLMT0. Filter Channel 0 Low Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT2** (const SDADC_Type *const pSdadc)

Get the value of WLMT2. Filter Channel 2 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearWLMT2** (SDADC_Type *const pSdadc)

Clear WLMT2. Filter Channel 2 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT1** (const SDADC_Type *const pSdadc)

Get the value of WLMT1. Filter Channel 1 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearWLMT1** (SDADC_Type *const pSdadc)

Clear WLMT1. Filter Channel 1 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT0** (const SDADC_Type *const pSadc)

Get the value of WLMT0. Filter Channel 0 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearWLMT0** (SDADC_Type *const pSadc)

Clear WLMT0. Filter Channel 0 Window Limit Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD2** (const SDADC_Type *const pSadc)

Get the value of SCD2. Filter Channel 2 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSCD2** (SDADC_Type *const pSadc)

Clear SCD2. Filter Channel 2 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD1** (const SDADC_Type *const pSadc)

Get the value of SCD1. Filter Channel 1 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSCD1** (SDADC_Type *const pSadc)

Clear SCD1. Filter Channel 1 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD0** (const SDADC_Type *const pSadc)

Get the value of SCD0. Filter Channel 0 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSCD0** (SDADC_Type *const pSadc)

Clear SCD0. Filter Channel 0 Short Circuit Detected Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSAT2** (const SDADC_Type *const pSadc)

Get the value of SAT2. Filter Channel 2 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSAT2** (SDADC_Type *const pSadc)

Clear SAT2. Filter Channel 2 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSAT1** (const SDADC_Type *const pSadc)

Get the value of SAT1. Filter Channel 1 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSAT1** (SDADC_Type *const pSadc)

Clear SAT1. Filter Channel 1 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSAT0** (const SDADC_Type *const pSadc)

Get the value of SAT0. Filter Channel 0 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearSAT0** (SDADC_Type *const pSadc)

Clear SAT0. Filter Channel 0 Saturation Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD2** (const SDADC_Type *const pSadc)

Get the value of CAD2. Filter Channel 2 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCAD2** (SDADC_Type *const pSadc)

Clear CAD2. Filter Channel 2 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD1** (const SDADC_Type *const pSadc)

Get the value of CAD1. Filter Channel 1 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCAD1** (SDADC_Type *const pSadc)

Clear CAD1. Filter Channel 1 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD0** (const SDADC_Type *const pSadc)

Get the value of CAD0. Filter Channel 0 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearCAD0** (SDADC_Type *const pSadc)

Clear CAD0. Filter Channel 0 Clock Absence Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVF2** (const SDADC_Type *const pSadc)

Get the value of FOVF2. Filter Channel 2 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFOVF2** (SDADC_Type *const pSdadc)

Clear FOVF2. Filter Channel 2 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVF1** (const SDADC_Type *const pSdadc)

Get the value of FOVF1. Filter Channel 1 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFOVF1** (SDADC_Type *const pSdadc)

Clear FOVF1. Filter Channel 1 FIFO Overflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFOVF0** (const SDADC_Type *const pSdadc)

Get the value of FOVF0. Filter Channel 0 FIFO Overflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFOVF0** (SDADC_Type *const pSdadc)

Clear FOVF0. Filter Channel 0 FIFO Overflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNF2** (const SDADC_Type *const pSdadc)

Get the value of FUNF2. Filter Channel 2 FIFO underflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFUNF2** (SDADC_Type *const pSdadc)

Clear FUNF2. Filter Channel 2 FIFO underflow flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNF1** (const SDADC_Type *const pSdadc)

Get the value of FUNF1. Filter Channel 1 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFUNF1** (SDADC_Type *const pSdadc)

Clear FUNF1. Filter Channel 1 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFUNF0** (const SDADC_Type *const pSdadc)

Get the value of FUNF0. Filter Channel 0 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearFUNF0** (SDADC_Type *const pSadc)

Clear FUNF0. Filter Channel 0 FIFO Underflow Flag 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO2_RDY** (const SDADC_Type *const pSadc)

Get the value of CLKO2_RDY. Modulator Clock Output 2 Ready Flag Indicate the modulator clock 2 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO1_RDY** (const SDADC_Type *const pSadc)

Get the value of CLKO1_RDY. Modulator Clock Output 1 Ready Flag Indicate the modulator clock 1 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKO0_RDY** (const SDADC_Type *const pSadc)

Get the value of CLKO0_RDY. Modulator Clock Output 0 Ready Flag Indicate the modulator clock 0 output is ready. 0b - The clock is not ready. 1b - The clock is ready.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFIFO_EMPTY** (const SDADC_Type *const pSadc)

Get the value of FIFO_EMPTY. FIFO Empty Flag bit 18-16 are for filter channel 2-0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHRDY** (const SDADC_Type *const pSadc)

Get the value of CHRDY. Filter Channel Ready Flag bit 10-8 are for filter channel 2-0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON2** (const SDADC_Type *const pSadc)

Get the value of CHON2. Filter Channel 2 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON1** (const SDADC_Type *const pSadc)

Get the value of CHON1. Filter Channel 1 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON0** (const SDADC_Type *const pSadc)

Get the value of CHON0. Filter Channel 0 Conversion Ongoing Flag 0b - The conversion is not ongoing. 1b - The conversion is ongoing.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHON** (const SDADC_Type *const pSadc)

Get the value of CHON. Filter Channel ON Flag bit 10-8 are for filter channel 2-0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCCRN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of the Channel Configuration Register (CCR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCCRN** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Configuration Register (CCR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMT_SEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LMT_SEL. Limit Check Input Selection Select the input from Main Filter or Auxiliary Filter. 0b - Auxiliary Filter. 1b - Main Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD_SEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of ZCD_SEL. Zero Cross Check Input Selection Select the input from Main Filter or Auxiliary Filter. 0b - Auxiliary Filter. 1b - Main Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDBG_SEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DBG_SEL. Debug Output Selection Select which data is output when read debug register 0100b - Instantaneous Primary Filter shifted 24bit Data. 0110b - Instantaneous HPF Output Data. 1000b - Instantaneous GAIN Output Data. 1010b - Instantaneous Secondary Filter CIC Data. 1110b - Instantaneous Primary Filter offset removed 24bit Data. others - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INTG_EN. Integration Enable Enable the integration block. 0b - Disabled and bypassed. 1b - Enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetRECT_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of RECT_EN. Rectification Enable Enable the rectification block. 0b - Disabled and bypassed. 1b - Enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFIFO_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of FIFO_EN. FIFO Enable 0b - FIFO is disabled for the channel. 1b - FIFO is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMT_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of LMT_EN. Limit Enable 0b - Limit check is disabled for the Main or Auxiliary Filter. 1b - Limit check is enabled for the Main or Auxiliary Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetZCD_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of ZCD_EN. Zero Cross Detect Enable 0b - Zero cross detection is disabled for the Main or Auxiliary Filter. 1b - Zero cross detection is enabled for the Main or Auxiliary Filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of CAD_EN. Clock Absence Detect Enable 0b - Clock absence detection is disabled for the channel. 1b - Clock absence detection is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SCD_EN. Short Circuit Detect Enable 0b - Short current detection is disabled for the channel. 1b - Short current detection is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDMA_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of DMA_EN. DMA Enable 0b - DMA is disabled for the channel. 1b - DMA is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetAF_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AF_EN. Auxiliary Filter Enable 0b - The Auxiliary Filter is disabled for the channel. 1b - The Auxiliary Filter is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMF_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MF_EN. Main Filter Enable 0b - The Main Filter is disabled for the channel. 1b - The Main Filter is enabled for the channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCH_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of CH_EN. Channel Enable 0b - The channel is disabled. 0b - The channel is enabled.

LOCAL_INLINE void **SDADC_HWA_SetCH_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint8_t u8Enable)
Set the value of CH_EN. Channel Enable 0b - The channel is disabled. 0b - The channel is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetAFCM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AFCM. Auxiliary Filter Conversion Mode 0b - Always on. 1b - Follow Main Filter mode (trigger).

LOCAL_INLINE uint32_t **SDADC_HWA_GetAFORD** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AFORD. Auxiliary Filter Order 00b - Reserved. 01b - First order filter. 10b - Second order filter. 11b - Third order filter.

LOCAL_INLINE uint32_t **SDADC_HWA_GetAFOSR** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of AFOSR. Auxiliary Filter Oversampling Rate The oversampling rate = (AFOSR + 1). The lower valid value of AFOSR is 3. The higher is depended on AFORD, in order to keep the internal CIC raw result not exceeding 24bit. If AFORD = 3, the maximum value is 202, else the maximum value is 511.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFCM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MFCM. Main Filter Conversion Mode 00b - Single mode. One conversion following a triggering event (edge or level). 01b - Continuous conversion mode. Multiple conversions following a triggering event, and the next trigger will cancel and restart conversion. 10b - Always conversion mode. Multiple conversions following the first trigger, and the next trigger will be ignored. 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFORD** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MFORD. Main Filter Order 000b - Reserved. 001b - First order filter. 010b - Second order filter. 011b - Third order filter. 100b - Fourth order filter. 101b - Fifth order filter. 110,111 - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFOSR** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MFOSR. Main Filter Oversampling Rate The oversampling rate = (MFOSR + 1). The lower valid of MFOSR value is 3. The higher is depended on MFORD, in order to keep the internal CIC raw result not exceeding 32bit. If MFORD = 4, the maximum value is 203. If MFORD = 5, the maximum value is 72, else the maximum value is 511.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCFR_SYNC_TRGEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of SYNC_TRGEN. Synced Trigger Select Enable 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTRGLVL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TRGLVL. Trigger Level type 0b - Trigger is edge event. Next edge event will cancel the current conversion, and re-start in single or continuous mode. 1b - Trigger is level event. De-assert cancels the current conversion, and stop filter function in single or continuous mode.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSYNC_TRGSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGSEL. Synced Trigger Select Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - select the trigger from the channel 0. 001b - select the trigger from the channel 1. 010b - select the trigger from the channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetTRGSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TRGSEL. Trigger Select 0b - Software trigger is selected. 1b - Hardware trigger is selected.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLKEDG** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CLKEDG. Input Clock Edge Select This bits select the effective edge of the modulator clock recovery to generate mclk. When using on chip modulator, only posedge is supported. 01b - Posedge. 10b - Negedge. 11b - Both edge. 00b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCFRCLKSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CLKSEL. Input Clock Select This bits select the clock source for the clock recovery to generate mclk. 00b - Reserved. 01b - Internal generated function clock div_clk1 selected, used for the case where MA is using div_clk0 as its function clock. The clock recovery is worked in sync mode. 10b - Internal generated function clock div_clk2 selected, used for the case where MA is using div_clk1 as its function clock. The clock recovery is worked in sync mode. 11b - Modulator clock from external modulator selected. The clock recovery is worked in async mode.

LOCAL_INLINE uint32_t **SDADC_HWA_GetIBFMT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of IBFMT. Input Bit Format 00b - External one bit stream. 01b - External Manchester code, CLKEDG will determine rise or fall decoder. 10b - Reserved. 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFWMK** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of FWMK. FIFO Water Mark Controls the watermark of the FIFO used to set FRDYn. When the number of results in the FIFO is greater than FWMK value, the FRDYn flag will be set. A DMA request or interrupt can also be generated if related DMA_EN and FRDYIE bit-field is set.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCUTOFF** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of CUTOFF. Cutoff Frequency Setting for High Pass Filter(HPF) HPF can only be enabled when RDFMT=0. 000b - HPF is disabled 001b - HPF is enabled and setting the cutoff frequency to rate 1(lowest cutoff frequency) $i\frac{1}{2}$ $i\frac{1}{2}$ 111b - HPF is enabled and setting the cutoff frequency to rate 7.(highest cutoff frequency)

LOCAL_INLINE uint32_t **SDADC_HWA_GetRDFMT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of RDFMT. Result Data Format 0b - Left justified, signed, for the case where input bitstream is signed 1b - Left justified, unsigned, for the case where input bitstream is unsigned.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMFSHIFT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of MFSHIFT. Main Filter Shift to Get Correct 24 bit Precision Main Filter CIC raw result is always 32bit width. The user should program MFSHIFT for cut-off and shift left/right to get 24bit result. MFSHIFT[4] = 1 means left shift, and MFSHIFT[4] = 0 means right shift. MFSHIFT[3:0] is the shift length.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCPRn** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of the Channel Protection Register (CPR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCPRn** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)
Set the value of the Channel Protection Register (CPR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHLMT_BKEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of HLMT_BKEN. Break Signal Enable for High Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetWLMT_BKEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of WLMT_BKEN. Break Signal Enabled for Window Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLLMT_BKEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LLMT_BKEN. Break Signal Enabled for Low Limit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD_BKEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SCD_BKEN. Break Signal Enabled for Short Circuit Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD_BKEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of CAD_BKEN. Break Signal Enabled for Clock Absence Event 0b - The Break signal is not enabled. 1b - The Break signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTF_PROTEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LMTF_PROTEN. Protection Singal Enabled for Limit Flag The protection signal can be enabled and sent to TRGSEL for application protection function. LMTOPT can be used to select which limit flag(HLMT/LLMT/WLMT) is used as the protection source. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTR_PROTEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of LMTR_PROTEN. Protection Singal Enabled for Limit Result The protection signal can be enabled and sent to TRGSEL for application protection function. LMTOPT can be used to select which limit result(high/low/window limit result) is used as the protection source. This protection signal will keep on if related condition keeps(keep high/keep low/keep in window) until the condition is not satisfied. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHYSTR_PROTEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of HYSTR_PROTEN. Protection Singal Enabled for Limit Hysteresis Result The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal is from the result of hysteresis comparator in limit detector and will keep on if related condition keeps(keep high/keep low) until the condition is not satisfied. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCD_PROTEN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SCD_PROTEN. Protection Singal Enabled for SCD Flag The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCAD_PROTEN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CAD_PROTEN. Protection Singal Enabled for CAD Flag The protection signal can be enabled and sent to TRGSEL for application protection function. This protection signal can only be cleared by W1C of related flag reigsters. 0b - The Protection signal is not enabled. 1b - The Protection signal is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTAVT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of LMTAVT. Limitation Result Activation Select The conversion results are compared with HILMTn and LOLMTn in limit detector, and HILMTn and LOLMTn can define a hysteresis range , A compare result is set/clear and sent to system with configurable activation. Note, When this hysteresis compare function is used in system, LMTOPTn need to be 00b to enable detect on exceed both high and low value. 0b - The compare result is set only when the conversion result is larger than HILMTn and clear only when the conversion result is smaller than LOLMTn. 1b - The compare result is clear only when the conversion result is larger than HILMTn and set only when the conversion result is smaller than LOLMTn.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCADLMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CADLMT. Clock Absence Detector Limitation Threshold The threshold value for the clock absence counter. The counter indicates the number of cycles the modulator clock not detected. If this value is reached, a clock absence detector event will occur on the dedicated channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLMTOPT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of LMTOPT. Limitation Option 00b - Limitation detect on exceed both high and low value. 01b - Limitation detect on exceed high threshold value. 10b - Limitation detect on exceed low threshold value. 11b - Limitation detect on windowed value.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDOPT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SCDOPT. Short Circuit Option 00b - Count on both 0 and 1. Detect both high and low value. 01b - Count on 1. Detect high value. 10b - Count on 0. Detect low value. 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDCM** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SCDCM. Short Circuit Detect Conversion Mode 0b - Always on when CHEN and MEN are asserted. 1b - Only works when the any filter is in progress.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSCDLMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SCDLMT. Short Circuit Detector Limitation extension The threshold value for short circuit detector counter. The counter indicates the number of continuous received bit 0 or bit 1. If this value is reached, a short-circuit detector event will occur on the dedicated channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIAS** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of BIAS. Bias Value Bias offset for dedicated channel main filter; used for sign adjustment and compensation. It's a signed data. The main filter data after shift, will subtract this bias value.

LOCAL_INLINE void **SDADC_HWA_SetBIAS** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of BIAS. Bias Value Bias offset for dedicated channel main filter; used for sign adjustment and compensation. It's a signed data. The main filter data after shift, will subtract this bias value.

LOCAL_INLINE uint32_t **SDADC_HWA_GetLOLMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of LOLMT. Low Limit Threshold When the data value exceeds this value, a low limit event will occur. The limit value format (signed or unsigned) depends on RDFMT.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHILMT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of HILMT. High Limit Threshold When data value exceeds this value, high limit event will occur. The limit value format (signed or unsigned) is depended on RDFMT.

LOCAL_INLINE uint32_t **SDADC_HWA_GetRDATA** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of RDATA. Result Data Read this register to get the conversion results. Bit[31] is sign bit if RDFMT is signed and only support 32bit read. The data format may changed by TSRDM. When TSRDM=2'b00, read CRDATA will pop data from FIFO. RDATA[31:8] is a 24-bit conversion results and RDATA[7:0] is 8'h00. At this case, CRDATA can only be read when FIFO is enabled and not empty. When TSRDM=2'b01, read CRDATA will feedback latest conversion result and timestamp info, RDATA[31:16] is a cut and round 16-bit latest conversion result(update on last COC) and RDATA[15:0] is latest timestamp counter value. FIFO is not used. At this case, CRDATA can only be read at any time. When TSRDM=2'b10, read CRDATA will pop data from FIFO and a timestamp info captured by trigger will also be included. RDATA[31:16] is a cut and round 16-bit result from FIFO and RDATA[15:0] is captured timestamp counter value. FIFO is used and gated by timestamp trigger. At this case, CRDATA can only be read when FIFO is enabled and not empty.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMPDATA** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of MPDATA. Multi Purpose Data This register can be accessed by DBG_SEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSIGN** (const SDADC_Type *const pSadc)

Get the value of SIGN. Sign Signal for the PWM pattern 0b - Positive values 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPWM_CNT** (const SDADC_Type *const pSadc)

Get the value of PWM_CNT. PWM Counter Value Counts the 32 cycles for each PWM pattern step.(-16 to 15)

LOCAL_INLINE uint32_t **SDADC_HWA_GetPAT_CNT** (const SDADC_Type *const pSadc)

Get the value of PAT_CNT. PWM Pattern Counter Value Counts the 32 steps for each PWM pattern period.

LOCAL_INLINE uint32_t **SDADC_HWA_GetON** (const SDADC_Type *const pSadc)

Get the value of ON. ON Status 0b - PWM generator is stopped 1b - PWM generator is ON.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTRGSIGN_OEN** (const SDADC_Type *const pSadc)

Get the value of TRGSIGN_OEN. Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once SIGN falling edge is got.

LOCAL_INLINE void **SDADC_HWA_SetTRGSIGN_OEN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set TRGSIGN_OEN. Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once SIGN falling edge is got.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPWMCLKSEL** (const SDADC_Type *const pSadc)

Get the value of CLKSEL. PWM Generator Clock selection 00b - div_clk0 01b - div_clk1 10b - div_clk2 11b - off.

LOCAL_INLINE void **SDADC_HWA_SetCLKSEL** (SDADC_Type *const pSadc, uint32_t u32Value)

Set CLKSEL. PWM Generator Clock selection 00b - div_clk0 01b - div_clk1 10b - div_clk2 11b - off.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIT_REV** (const SDADC_Type *const pSadc)

Get the value of BIT_REV. Bit Reverse Mode 0b - Normal mode, a triangle wave is used as PWM carrier. 1b - Bit reverse mode. A bit reversed sawtooth wave is used as PWM carrier.

LOCAL_INLINE void **SDADC_HWA_SetBIT_REV** (SDADC_Type *const pSadc, uint32_t u32Value)

Set BIT_REV. Bit Reverse Mode 0b - Normal mode, a triangle wave is used as PWM carrier. 1b - Bit reverse mode. A bit reversed sawtooth wave is used as PWM carrier.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPOL** (const SDADC_Type *const pSadc)

Get the value of POL. PWM Generator Polarity 0b - PWM generator output begin with +1 1b - PWM generator output begin with -1.

LOCAL_INLINE void **SDADC_HWA_SetPOL** (SDADC_Type *const pSadc, uint32_t u32Value)

Set POL. PWM Generator Polarity 0b - PWM generator output begin with +1 1b - PWM generator output begin with -1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetPATTERN** (const SDADC_Type *const pSadc)

Get the value of PATTERN. PWM Generator Pattern Config this before set MEN. Clear the bits during PWM generator run will terminate the PWM generator outputs after completion of the current period(indicated by bit ON=0) 00b - Stopped 01b - Square wave 10b - Triangle 11b - Sine wave.

LOCAL_INLINE void **SDADC_HWA_SetPATTERN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set PATTERN. PWM Generator Pattern Config this before set MEN. Clear the bits during PWM generator run will terminate the PWM generator outputs after completion of the current period(indicated by bit ON=0) 00b - Stopped 01b - Square wave 10b - Triangle 11b - Sine wave.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD2** (const SDADC_Type *const pSadc)

Get the value of TSVLD2. Timestamp Valid Flag for Channel 2 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearTSVLD2** (SDADC_Type *const pSadc)

Clear TSVLD2. Timestamp Valid Flag for Channel 2 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD1** (const SDADC_Type *const pSadc)

Get the value of TSVLD1. Timestamp Valid Flag for Channel 1 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearTSVLD1** (SDADC_Type *const pSadc)

Clear TSVLD1. Timestamp Valid Flag for Channel 1 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD0** (const SDADC_Type *const pSadc)

Get the value of TSVLD0. Timestamp Valid Flag for Channel 0 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE void **SDADC_HWA_ClearTSVLD0** (SDADC_Type *const pSadc)

Clear TSVLD0. Timestamp Valid Flag for Channel 0 Indicates timestamp information is captured, it will be cleared if CTSINFO register is readed. 0b - The flag is not asserted. 1b - The flag is asserted.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N2** (const SDADC_Type *const pSadc)

Get the value of TSIGN_P2N2. Trigger Sign Changed from 1 to 0 for Channel 2 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE void **SDADC_HWA_ClearTSIGN_P2N2** (SDADC_Type *const pSadc)

Clear TSIGN_P2N2. Trigger Sign Changed from 1 to 0 for Channel 2 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N1** (const SDADC_Type *const pSadc)

Get the value of TSIGN_P2N1. Trigger Sign Changed from 1 to 0 for Channel 1 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE void **SDADC_HWA_ClearTSIGN_P2N1** (SDADC_Type *const pSadc)

Clear TSIGN_P2N1. Trigger Sign Changed from 1 to 0 for Channel 1 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N0** (const SDADC_Type *const pSadc)

Get the value of TSIGN_P2N0. Trigger Sign Changed from 1 to 0 for Channel 0 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE void **SDADC_HWA_ClearTSIGN_P2N0** (SDADC_Type *const pSdadc)

Clear TSIGN_P2N0. Trigger Sign Changed from 1 to 0 for Channel 0 Indicates TSIGN changed from 1 to 0 when RECT is on, this can be used as an INTG start trigger for a better angular evaluation. 0b - TSIGN is not changed. 1b - TSIGN is changed from 1 to 0.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCIC_COC2** (const SDADC_Type *const pSdadc)

Get the value of MCIC_COC2. Filter Channel 2 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE void **SDADC_HWA_ClearMCIC_COC2** (SDADC_Type *const pSdadc)

Clear MCIC_COC2. Filter Channel 2 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCIC_COC1** (const SDADC_Type *const pSdadc)

Get the value of MCIC_COC1. Filter Channel 1 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE void **SDADC_HWA_ClearMCIC_COC1** (SDADC_Type *const pSdadc)

Clear MCIC_COC1. Filter Channel 1 Conversion Complete Flag for MCIC Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMCIC_COC0** (const SDADC_Type *const pSdadc)

Get the value of MCIC_COC0. Filter channel 0 Conversion Complete Flag for MCIC. Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE void **SDADC_HWA_ClearMCIC_COC0** (SDADC_Type *const pSdadc)

Clear MCIC_COC0. Filter channel 0 Conversion Complete Flag for MCIC. Means MCIC data or rectification output or most recent accumulation data in Channel Integration Value Register is valid. When integration is not enabled, this is the same as COC. When integration is enabled and on, this is not the same as COC which means integration output valid.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD2** (const SDADC_Type *const pSdadc)

Get the value of SD_CAPVLD2. Capture Valid Flag for channel 2 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE void **SDADC_HWA_ClearSD_CAPVLD2** (SDADC_Type *const pSdadc)

Clear SD_CAPVLD2. Capture Valid Flag for channel 2 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD1** (const SDADC_Type *const pSdadc)

Get the value of SD_CAPVLD1. Capture Valid Flag for channel 1 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE void **SDADC_HWA_ClearSD_CAPVLD1** (SDADC_Type *const pSdadc)

Clear SD_CAPVLD1. Capture Valid Flag for channel 1 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD0** (const SDADC_Type *const pSdadc)

Get the value of SD_CAPVLD0. Capture Valid Flag for channel 0 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE void **SDADC_HWA_ClearSD_CAPVLD0** (SDADC_Type *const pSdadc)

Clear SD_CAPVLD0. Capture Valid Flag for channel 0 Indicates a new value in bitfield SD_CAP. This bit can be cleared by read related CSDRn register. 0b - no new result available. 1b - SD_CAP has been updated with a new captured value and has not been read.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD_IE2** (const SDADC_Type *const pSdadc)

Get the value of TSVLD_IE2. Timestamp Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSVLD_IE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSVLD_IE2. Timestamp Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD_IE1** (const SDADC_Type *const pSdadc)

Get the value of TSVLD_IE1. Timestamp Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSVLD_IE1** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSVLD_IE1. Timestamp Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSVLD_IE0** (const SDADC_Type *const pSdadc)

Get the value of TSVLD_IE0. Timestamp Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSVLD_IE0** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSVLD_IE0. Timestamp Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N_IE2** (const SDADC_Type *const pSdadc)

Get the value of TSIGN_P2N_IE2. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSIGN_P2N_IE2** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set TSIGN_P2N_IE2. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N_IE1** (const SDADC_Type *const pSdadc)

Get the value of TSIGN_P2N_IE1. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSIGN_P2N_IE1** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set TSIGN_P2N_IE1. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN_P2N_IE0** (const SDADC_Type *const pSdadc)
Get the value of TSIGN_P2N_IE0. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetTSIGN_P2N_IE0** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set TSIGN_P2N_IE0. Trigger Sign Changed from 1 to 0 Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD_IE2** (const SDADC_Type *const pSdadc)
Get the value of SD_CAPVLD_IE2. Capture Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSD_CAPVLD_IE2** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set SD_CAPVLD_IE2. Capture Valid Interrupt Enable for channel 2 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD_IE1** (const SDADC_Type *const pSdadc)
Get the value of SD_CAPVLD_IE1. Capture Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSD_CAPVLD_IE1** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set SD_CAPVLD_IE1. Capture Valid Interrupt Enable for channel 1 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAPVLD_IE0** (const SDADC_Type *const pSdadc)
Get the value of SD_CAPVLD_IE0. Capture Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE void **SDADC_HWA_SetSD_CAPVLD_IE0** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set SD_CAPVLD_IE0. Capture Valid Interrupt Enable for channel 0 0b - The interrupt is disabled. 1b - The interrupt is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_STRIG2** (const SDADC_Type *const pSdadc)
Get the value of INTG_STRIG2. Integration Software Trigger for channel 2 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE void **SDADC_HWA_SetINTG_STRIG2** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set INTG_STRIG2. Integration Software Trigger for channel 2 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_STRIG1** (const SDADC_Type *const pSdadc)
Get the value of INTG_STRIG1. Integration Software Trigger for channel 1 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE void **SDADC_HWA_SetINTG_STRIG1** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set INTG_STRIG1. Integration Software Trigger for channel 1 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTG_STRIG0** (const SDADC_Type *const pSdadc)
Get the value of INTG_STRIG0. Integration Software Trigger for channel 0 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE void **SDADC_HWA_SetINTG_STRIG0** (SDADC_Type *const pSdadc, uint32_t u32Value)
Set INTG_STRIG0. Integration Software Trigger for channel 0 Setting this bit will trigger to start of the integration when the integration is enabled. Clearing this bit will stop integration. So it works like a level event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIAS_ON** (const SDADC_Type *const pSdadc)
Get the value of BIAS_ON. Bias On Flag 0b - Bias analog is not ready for using. 1b - Bias analog is ready for using.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBSTC** (const SDADC_Type *const pSadc)

Get the value of BSTC. Bias Start Time Control Bias analog need a ~2us start time, this bits control number of div_clk1 cycles before BIAS_ON flag set after BIAS_EN set.

LOCAL_INLINE void **SDADC_HWA_SetBSTC** (SDADC_Type *const pSadc, uint32_t u32Value)

Set BSTC. Bias Start Time Control Bias analog need a ~2us start time, this bits control number of div_clk1 cycles before BIAS_ON flag set after BIAS_EN set.

LOCAL_INLINE uint32_t **SDADC_HWA_GetVCM_BEN** (const SDADC_Type *const pSadc)

Get the value of VCM_BEN. Common Mode Voltage Bias Enable Enable common mode voltage for AC input of internal modulator analog. 0b - Common mode voltage Bias is disabled. 1b - Common mode voltage Bias is enabled.

LOCAL_INLINE void **SDADC_HWA_SetVCM_BEN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set VCM_BEN. Common Mode Voltage Bias Enable Enable common mode voltage for AC input of internal modulator analog. 0b - Common mode voltage Bias is disabled. 1b - Common mode voltage Bias is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHFVREF_EN** (const SDADC_Type *const pSadc)

Get the value of HFVREF_EN. Half VREFH output Enable to Related Internal Modulator Analog. Bit 4 enable internal Half VREFH output to modulator analog for channel 0. Bit 5 enable internal Half VREFH output to modulator analog for channel 1. Bit 6 enable internal Half VREFH output to modulator analog for channel 2. Bit 7 enable internal Half VREFH output to modulator analog for channel 3. NOTE: Not all channels have internal modulator analog, if the channel has no internal modulator analog, related control bit is reserved.

LOCAL_INLINE void **SDADC_HWA_SetHFVREF_EN** (SDADC_Type *const pSadc, uint32_t u32Value)

Set HFVREF_EN. Half VREFH output Enable to Related Internal Modulator Analog. Bit 4 enable internal Half VREFH output to modulator analog for channel 0. Bit 5 enable internal Half VREFH output to modulator analog for channel 1. Bit 6 enable internal Half VREFH output to modulator analog for channel 2. Bit 7 enable internal Half VREFH output to modulator analog for channel 3. NOTE: Not all channels have internal modulator analog, if the channel has no internal modulator analog, related control bit is reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetBIAS_EN** (const SDADC_Type *const pSadc)

Get the value of BIAS_EN. Bias Analog Enable 0b - Bias analog is disabled. 1b - Bias analog is enabled.

LOCAL_INLINE void **SDADC_HWA_SetBIAS_EN** (SDADC_Type *const pSdadc, uint32_t u32Value)

Set BIAS_EN. Bias Analog Enable 0b - Bias analog is disabled. 1b - Bias analog is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDSIGN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DSIGN. Delayed Sign Signal 0b - Positive values. 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSSIGN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SSIGN. Selected Sign Signal 0b - Positive values. 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSDON** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SDON. Sign Delay Counter on Flag Indicates the activity of the sign delay counter. 0b - Sign delay counter stopped. 1b - Sign delay counter enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSIGN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSIGN. Sign Signal for Channel Trigger Output This sign can be configured to some cycles before DSIGN, and can be used to generate channel trigger output, typically can be used to trigger integration so that it can start at the ZCD point 0b - Positive values. 1b - Negative values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDLY_TRG** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DLY_TRG. Sign Delay Value for TSIGN Posedge TSIGN is clear at DLY_TRG and set at DLY_NEG in CSDRn.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCTRGSIGN_OEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint8_t u8Enable)

Get the value of CTRGSIGN_OEN. Channel Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once TSIGN falling edge is got.

LOCAL_INLINE void **SDADC_HWA_SetCTRGSIGN_OEN** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint8_t u8Enable)

Set the value of CTRGSIGN_OEN. Channel Sign Signal Related Trigger Out Enable 0b - disabled. 1b - Related trigger is enabled and will be set once TSIGN falling edge is got.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSSCH** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SSCH. Sign Signal From Channel Selection Select the channel providing the sign signal if SSRC = 01b 0000b - From channel 0. 0001b - From channel 1. 0010b - From channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetSSRC** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SSRC. Sign Source Selection Select the sign signal to be delayed. 00b - SDADC internal PWM generation block. 01b - Sign of result of channel selected by SSCH. 10b - External sign signal 0. 11b - External sign signal 1.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDLY_NEG** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DLY_NEG. Sign Delay Value for Sign Negedge Defines the content of SD_CNT to generate a positive delayed sign DSIGN.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDLY_POS** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DLY_POS. Sign Delay Value for Sign Posedge Defines the content of SD_CNT to generate a negative delayed sign DSIGN.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CAP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SD_CAP. Sign Delay Capture Value Indicates the results value counted between the negedge of SSIGN and the first received positive value.

LOCAL_INLINE uint32_t **SDADC_HWA_GetSD_CNT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SD_CNT. Sign Delay Counter Counts the results values from the filter chain to delay the sign signal, this field can only be checked after MCIC_COC assert.

LOCAL_INLINE uint32_t **SDADC_HWA_GetNVALINT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of NVALINT. Number of Values to be Accumulated NVALINT+1 defines a complete integration cycle.

LOCAL_INLINE uint32_t **SDADC_HWA_GetNVALDIS** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of NVALDIS. Number of Values discarded Start the integration cycle after NVALDIS values.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCICFR_SYNC_TRGEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGEN. Synced Trigger Select Enable for Integration 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCICFR_SYNC_TRGSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGSEL. Synced Trigger Select for Integration Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - select the trigger from the channel 0. 001b - select the trigger from the channel 1. 010b - select the trigger from the channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetNVALREP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of NVALREP. Number of Integration Cycles NVALREP+1 defines number of integration cycles to be counted by REPCNT if activated(EXTSTP=0)

LOCAL_INLINE uint32_t **SDADC_HWA_GetITRSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of ITRSEL. Integration Trigger Select 0b - Software trigger selected. 1b - Hardware trigger dedicate to channel selected.

LOCAL_INLINE uint32_t **SDADC_HWA_GetHTRPOL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of HTRPOL. Hardware Trigger Polarity 0b - High and rising edge effective. 1b - Low and falling edge effective.

LOCAL_INLINE uint32_t **SDADC_HWA_GetFCRC** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of FCRC. Filter Chain Restart Control 0b - Do not restart. 1b - Restart the filter chain when an integration window starts.

LOCAL_INLINE uint32_t **SDADC_HWA_GetEXTSTP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of EXTSTP. Integration Stop Control For Hardware trigger control only. 0b - Internal control, stop integration after NVALREP+1 integration cycles, support both edge and level events. 1b - External control, stop integration upon the inverse trigger event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetIOGT** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of IOGT. Integration Output Gating Setting this bit will gate conversion complete flag and FIFO push when Integration is not on-going. conversion complete flag and FIFO push can only be effective when Integration is not on-going(INTON asserted) 0b - Conversion complete flag and FIFO push always works. 1b - Conversion complete flag and FIFO push can only work when Integration is not on-going(INTON asserted)

LOCAL_INLINE uint32_t **SDADC_HWA_GetISVAL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of ISVAL. Integration Shift value 000b - Shift 0 bits 001b - Shift 1 bits, for 2 values integration 010b - Shift 2 bits, for 3-4 values integration 011b - Shift 3 bits, for 5-8 values integration 100b - Shift 4 bits, for 9-16 values integration 101b - Shift 5 bits, for 17-32 values integration 110b - Shift 6 bits, for 33-64 values integration.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTGON** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INTGON. Integration on Flag Indicates the activity of the integration. 0b - Integration stopped. 1b - Integration enabled and on-going.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINTVAL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INTVAL. Results of most recent accumulation, this field can only be checked after MCIC_COC assert.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMASEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of MASEL. Modulator Analog Selection 0b - External modulator analog is selected as channel input. 1b - Internal modulator analog is selected as channel input.

LOCAL_INLINE void **SDADC_HWA_SetMASEL** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set MASEL. Modulator Analog Selection 0b - External modulator analog is selected as channel input. 1b - Internal modulator analog is selected as channel input.

LOCAL_INLINE uint32_t **SDADC_HWA_GetMACLKSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

*Get the value of MACLKSEL. Internal Modulator Analog Clock Selection 00b - Div_clk1 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 01b - Div_clk0 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 1*b - Reserved.*

LOCAL_INLINE void **SDADC_HWA_SetMACLKSEL** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

*Set MACLKSEL. Internal Modulator Analog Clock Selection 00b - Div_clk1 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 01b - Div_clk0 is selected as the clock, output bitstream is sampled on a clock with frequency of $F_{div_clk1}/2$. 1*b - Reserved.*

LOCAL_INLINE uint32_t **SDADC_HWA_GetRSTSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of RSTSEL. Modulator Analog Reset Cycle Selection Select RSTSEL+1 mclk cycles for modulator analog reset after a channel conversion trigger. This is needed especially when the input mux(INMUX) is changed. 0d - Modulator analog is not reset after a channel conversion trigger 1d - Modulator analog is reset for 2 mclk cycles after a channel conversion trigger. Total 7 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 2d - Modulator analog is reset for 3 mclk cycles after a channel conversion trigger. Total 8 mclk cycle cycles needed for filters start to run after a channel conversion trigger. ... 8d - Modulator analog is reset for 9 mclk cycles after a channel conversion trigger. Total 14 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 9-15d - Modulator analog is reset for 10 mclk cycles after a channel conversion trigger. Total 15 mclk cycle cycles needed for filters start to run after a channel conversion trigger.

LOCAL_INLINE void **SDADC_HWA_SetRSTSEL** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set RSTSEL. Modulator Analog Reset Cycle Selection Select RSTSEL+1 mclk cycles for modulator analog reset after a channel conversion trigger. This is needed especially when the input mux(INMUX) is changed. 0d - Modulator analog is not reset after a channel conversion trigger 1d - Modulator analog is reset for 2 mclk cycles after a channel conversion trigger. Total 7 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 2d - Modulator analog is reset for 3 mclk cycles after a channel conversion trigger. Total 8 mclk cycle cycles needed for filters start to run after a channel conversion trigger. ... 8d - Modulator analog is reset for 9 mclk cycles after a channel conversion trigger. Total 14 mclk cycle cycles needed for filters start to run after a channel conversion trigger. 9-15d - Modulator analog is reset for 10 mclk cycles after a channel conversion trigger. Total 15 mclk cycle cycles needed for filters start to run after a channel conversion trigger.

LOCAL_INLINE uint32_t **SDADC_HWA_GetDITHER_EN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of DITHER_EN. Dither Enable This bit enable the dither to analog circuit to remove the idle tone. 0b - Dither is disabled. 1b - Dither is enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINCFGPV11** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of INCFGPV11. Configuration of Positive Input Line with V11 from System This bit have higher priority then INCFGP, this this bit is set, INCFGP is ignored. 0b - Positive input from V11 disabled. 1b - Positive input from V11 enabled.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGAINSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of GAINSEL. Gain Select of Analog Input Path 00b - 1. 01b - 2. 10b - 4. 11b - 8.

LOCAL_INLINE void **SDADC_HWA_SetGAINSEL** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set GAINSEL. Gain Select of Analog Input Path 00b - 1. 01b - 2. 10b - 4. 11b - 8.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCMENP** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CMENP. Common Voltage Configuration of Positive Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE void **SDADC_HWA_SetCMENP** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set CMENP. Common Voltage Configuration of Positive Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCMENM** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of CMENM. Common Voltage Configuration of Negative Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE void **SDADC_HWA_SetCMENM** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set CMENM. Common Voltage Configuration of Negative Inputs Defines the connection of respective negative input to the common mode voltage, bit n(0-3) controls if Pin n is connected to the common mode voltage. 0b - Is NOT connected to the common mode voltage. 1b - Is connected to the common mode voltage.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINCFGP** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of INCFGP. Configuration of Positive Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE void **SDADC_HWA_SetINCFGP** (SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set INCFGP. Configuration of Positive Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINCFGM** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INCFGM. Configuration of Negative Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE void **SDADC_HWA_SetINCFGM** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)
Set INCFGM. Configuration of Negative Input Line 00b - From input Pin. 01b - From VREFL. 10b - From VREFH. 11b - From VREFH/2.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINMUX** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INMUX. Active Input Pin Selection Indicates the active input pin selection for input MUX. 00b - Input pin 0 selected. 01b - Input pin 1 selected. 10b - Input pin 2 selected. 11b - Input pin 3 selected.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINSEL** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INSEL. Input Pin Selection Defines the initial or permanent setting for the input MUX(bits INMUX) depending on the selected operating mode (bits INMOD).

LOCAL_INLINE void **SDADC_HWA_SetINSEL** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)
Set INSEL. Input Pin Selection Defines the initial or permanent setting for the input MUX(bits INMUX) depending on the selected operating mode (bits INMOD).

LOCAL_INLINE uint32_t **SDADC_HWA_GetINMOD** (const SDADC_Type *const pSdadc, uint8_t u8Channel)
Get the value of INMOD. Input MUX Control Mode 00b - Software control mode (INMUX follows INSEL). 01b - Preset mode (load INMUX upon a channel conversion trigger). 10b - Scan mode (decrement INMUX upon a channel conversion trigger, wrap around to INSEL). 11b - Reserved.

LOCAL_INLINE void **SDADC_HWA_SetINMOD** (SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)
Set INMOD. Input MUX Control Mode 00b - Software control mode (INMUX follows INSEL). 01b - Preset mode (load INMUX upon a channel conversion trigger). 10b - Scan mode (decrement INMUX upon a channel conversion trigger, wrap around to INSEL). 11b - Reserved.

LOCAL_INLINE uint32_t **SDADC_HWA_GetOffsetOPT1** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT1. Offset Option 1 This is an offset value provided by factory test for modulator analog input path gain=2. User need read this field and left shift it by 8 bit to form a 24-bit offset and then write to related CBR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetOffsetOPT0** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT0. Offset Option 0 This is an offset value provided by factory test for modulator analog input path gain=1. User need read this field and left shift it by 8 bit to form a 24-bit offset and then write to related CBR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGainOPT1** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT1. Gain Option 1 This is a gain factor provided by factory test for modulator analog input path gain=2. User need read this field and multiply it with gain correction factor 2^m and then write to related CGNR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGainOPT0** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of OPT0. Gain Option 0 This is a gain factor provided by factory test for modulator analog input path gain=1. User need read this field and multiply it with gain correction factor 2^m and then write to related CGNR register.

LOCAL_INLINE uint32_t **SDADC_HWA_GetGAIN_FAC** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of GAIN_FAC. Gain Factor This bit field defines a gain factor($GAIN_FAC[17:0]/2^{16}$, range is from 0 to 4) which is multiplied with the data after HPF to compensate the gain error caused by analog circuits or the gain mismatch when decimation factor is not 2^N , after compensation, the total gain from analog input to data from FIFO should be 1. GAIN_FAC can be got from calibration, it should be in a suitable range, or the data output may be saturated. When $GAIN_FAC[17:0]=0$, GAIN block is bypassed.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCTSCNT_SYNC_TRGEN** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of SYNC_TRGEN. Synced Trigger Select Enable for Timestamp 0b - Synced trigger is selected on the channel itself. 1b - Synced trigger is selected on the channel defined by SYNC_TRGSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCTSCNT_SYNC_TRGSEL** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of SYNC_TRGSEL. Synced Trigger Select for Timestamp Each will detect and sync the trigger input as a synced trigger, this bits can select to use the synced trigger from which channel. By this bits, two or more channels can share the same synced trigger and trigger to start at the same time. 000b - Select the trigger from the channel 0. 001b - Select the trigger from the channel 1. 010b - Select the trigger from the channel 2. ...

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSRDM** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of TSRDM. Timestamp Information Read Mode 00b - Default mode, timestamp information can only be got by reading CTSINFO register captured by trigger event. FIFO is used for conversion results only and can be pop from FIFO by reading CRDATAn.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSTRGM** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of TSTRGM. Timestamp Counter Trigger Mode 00/11b - Trigger is ignored. 01b - Rising edge effective. Trigger level low will gating conversion results push to FIFO if TSRDM is 2'b10. Trigger level high will allow conversion results push to FIFO if TSRDM is 2'b10. 10b - Falling edge effective. Trigger level High will gating conversion results push to FIFO if TSRDM is 2'b10. Trigger level low will allow conversion results push to FIFO if TSRDM is 2'b10.

LOCAL_INLINE uint32_t **SDADC_HWA_GetINMXCP** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of INMXCP. Input Mux Setting Copy Enable 0b - Disabled. 1b - Copy INMUX from CMACSR to TSTAMP[15:14].

LOCAL_INLINE uint32_t **SDADC_HWA_GetTS_EN** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of TS_EN. Timestamp Counter Enable 0b - Disabled. 1b - Counting at the rate selected by TSCLKSEL.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSCLKSEL** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of TSCLKSEL. Timestamp Counter Clock Selection 00b - Modulator clock. 01b - Modulator clock divided by 2. 10b - Modulator clock divided by 4. 11b - Modulator clock divided by 8.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSCNT** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of TSCNT. Timestamp Counter Value TSCNT is clocked with the modulator clock and is cleared when a new result value has been generated.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSDATA** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSDATA. The Latest Captured Conversion Result Copied from saved conversion result.

LOCAL_INLINE uint32_t **SDADC_HWA_GetTSTAMP** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of TSTAMP. The Latest Captured Timestamp Value Copy from timestamp counter TSCNT at the trigger event.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCFRn** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of the Channel Configuration Register (CFR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCFRn** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Configuration Register (CFR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCBRn** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of the Channel Bias Register (CBR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCBRn** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Bias Register (CBR) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCLLMTn** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of the Channel Low Limit Register (CLLMT) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCLLMTn** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Low Limit Register (CLLMT) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCHLMTn** (const SDADC_Type *const pSdadc, uint8_t u8Channel)

Get the value of the Channel High Limit Register (CHLMT) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCHLMTn** (const SDADC_Type *const pSdadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel High Limit Register (CHLMT) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCRDATAn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Result Data Register (CRDATA) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCMPDATAn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Multi Purpose Data Register (CMPDATA) for a specified channel.

LOCAL_INLINE uint32_t **SDADC_HWA_GetCDRn** (const SDADC_Type *const pSadc, uint8_t u8Channel)

Get the value of the Channel Data Rate Register (CDR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetCDRn** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint32_t u32Value)

Set the value of the Channel Data Rate Register (CDR) for a specified channel.

LOCAL_INLINE void **SDADC_HWA_SetNISR** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of NISR.

LOCAL_INLINE void **SDADC_HWA_SetABNISR0** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of ABNISR0.

LOCAL_INLINE void **SDADC_HWA_SetABNISR1** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of ABNISR1.

LOCAL_INLINE void **SDADC_HWA_SetEXTIS** (SDADC_Type *const pSadc, uint32_t u32Value)

Set the value of EXTIS.

LOCAL_INLINE void **SDADC_HWA_SetDMA_EN** (const SDADC_Type *const pSadc, uint8_t u8Channel, uint8_t u8Enable)

Set the value of DMA_EN. Channel Enable 0b - The channel dma is disabled. 0b - The channel dma is enabled.

Detailed Description

Hardware access layer for SDADC.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

Macro Definition Documentation

#define

SDADC_MSR_CHON(x) (((uint32_t)(((uint32_t)(x))<<SDADC_MSR_CHON_SHIFT))&SDADC_MSR_CHON_MASK)

#define SDADC_MSR_CHON_MASK 0x7u

#define SDADC_MSR_CHON_SHIFT 0u

#define SDADC_MSR_CHON_WIDTH 3u

HwA_sddf.h File Reference

Hardware access layer for SDDF.

```
#include "device_header.h"
```

Include dependency graph for HwA_sddf.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum SDDF_PreDivType { SDDF_PREDIV_1 = 0U, SDDF_PREDIV_2 = 1U, SDDF_PREDIV_4 = 2U,  
SDDF_PREDIV_8 = 3U }
```

Select the SDDF function clock pre-divider ratio.

```
enum SDDF_ModeType { SDDF_MODE_NORMAL = 0U, SDDF_MODE_DEBUG = 1U }
```

Select the SDDF operation mode.

```
enum SDDF_DebugModuleEnableType { SDDF_MODULE_ENABLE_IN_DEBUG = 0U,  
SDDF_MODULE_DISABLE_IN_DEBUG = 1U }
```

Select whether the SDDF module enables in debug mode.

```
enum SDDF_InputSrcType { SDDF_INPUT_SRC_MAIN_FILTER = 0U,  
SDDF_INPUT_SRC_AUXILIARY_FILTER = 1U }
```

Select the data source for limit check and zero cross check.

```
enum SDDF_AuxFilterConvModeType { SDDF_AUX_FILTER_CONV_ALWAYS_ON = 0U,  
SDDF_AUX_FILTER_CONV_FOLLOW_MAIN = 1U }
```

Select the conversion mode for auxiliary filter.

```
enum SDDF_AuxFilterOrderType { SDDF_AUX_FILTER_FIRST_ORDER = 1U,  
SDDF_AUX_FILTER_SECOND_ORDER = 2U, SDDF_AUX_FILTER_THIRD_ORDER = 3U }
```

Selects the filter order for auxiliary filter.

```
enum SDDF_MainFilterConvModeType { SDDF_MAIN_FILTER_CONV_SINGLE = 0U,  
SDDF_MAIN_FILTER_CONV_CONTINUOUS = 1U, SDDF_MAIN_FILTER_CONV_ALWAYS = 2U }
```

Selects the conversion mode for main filter.

enum **SDDF_MainFilterOrderType** { **SDDF_MAIN_FILTER_FIRST_ORDER** = 1U,
SDDF_MAIN_FILTER_SECOND_ORDER = 2U, **SDDF_MAIN_FILTER_THIRD_ORDER** = 3U,
SDDF_MAIN_FILTER_FOURTH_ORDER = 4U, **SDDF_MAIN_FILTER_FIFTH_ORDER** = 5U }

Selects the filter order for main filter.

enum **SDDF_TriggerEventType** { **SDDF_TRIGGER_EVENT_EDGE** = 0U, **SDDF_TRIGGER_EVENT_LEVEL** = 1U }

Select the trigger level type.

enum **SDDF_TriggerSrcType** { **SDDF_TRIGGER_SRC_SOFTWARE** = 0U, **SDDF_TRIGGER_SRC_HARDWARE** = 1U }

Select the trigger source.

enum **SDDF_ClockEdgeType** { **SDDF_CLOCK_EDGE_RISING** = 1U, **SDDF_CLOCK_EDGE_FALLING** = 2U,
SDDF_CLOCK_EDGE_BOTH = 3U }

Select the input clock edge.

enum **SDDF_ClockSrcType** { **SDDF_CLOCK_SRC_EXTERNAL_MODULATOR_CLOCK** = 3U }

Select the input clock source.

enum **SDDF_BitFormatType** { **SDDF_BIT_FORMAT_EXTERNAL_ONE_BIT_STREAM** = 0U,
SDDF_BIT_FORMAT_EXTERNAL_MANCHSTER_CODE = 1U }

Select the input bit format.

enum **SDDF_ResultFormatType** { **SDDF_RESULT_DATA_LEFT_ALIGNED_SIGNED** = 0U,
SDDF_RESULT_DATA_LEFT_ALIGNED_UNSIGNED = 1U }

Select the result data format.

enum **SDDF_MainFilterShiftType** { **SDDF_MAIN_FILTER_NOT_SHIFT** = 0U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_1 = 1U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_2** = 2U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_3 = 3U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_4** = 4U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_5 = 5U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_6** = 6U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_7 = 7U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_8** = 8U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_9 = 9U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_10** = 10U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_11 = 11U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_12** = 12U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_13 = 13U, **SDDF_MAIN_FILTER_SHIFT_RIGHT_14** = 14U,
SDDF_MAIN_FILTER_SHIFT_RIGHT_15 = 15U, **SDDF_MAIN_FILTER_SHIFT_LEFT_1** = 17U,
SDDF_MAIN_FILTER_SHIFT_LEFT_2 = 18U, **SDDF_MAIN_FILTER_SHIFT_LEFT_3** = 19U,
SDDF_MAIN_FILTER_SHIFT_LEFT_4 = 20U, **SDDF_MAIN_FILTER_SHIFT_LEFT_5** = 21U,
SDDF_MAIN_FILTER_SHIFT_LEFT_6 = 22U, **SDDF_MAIN_FILTER_SHIFT_LEFT_7** = 23U,
SDDF_MAIN_FILTER_SHIFT_LEFT_8 = 24U, **SDDF_MAIN_FILTER_SHIFT_LEFT_9** = 25U,
SDDF_MAIN_FILTER_SHIFT_LEFT_10 = 26U, **SDDF_MAIN_FILTER_SHIFT_LEFT_11** = 27U,
SDDF_MAIN_FILTER_SHIFT_LEFT_12 = 28U, **SDDF_MAIN_FILTER_SHIFT_LEFT_13** = 29U,
SDDF_MAIN_FILTER_SHIFT_LEFT_14 = 30U, **SDDF_MAIN_FILTER_SHIFT_LEFT_15** = 31U }

Set the main filter shift to get correct 24 bit precision.

```
enum SDDF_ChannelLimitOptionType { SDDF_CHANNEL_LIMIT_BOTH_HITH_LOW_THRES = 0U,  
SDDF_CHANNEL_LIMIT_HITH_THRES = 1U, SDDF_CHANNEL_LIMIT_LOW_THRES = 2U,  
SDDF_CHANNEL_LIMIT_WINDOW = 3U }
```

Select the limit detection option.

```
enum SDDF_ShortCircuitOptionType { SDDF_SHORT_CIRCUIT_COUNT_ON_BOTH = 0U,  
SDDF_SHORT_CIRCUIT_COUNT_ON_1 = 1U, SDDF_SHORT_CIRCUIT_COUNT_ON_0 = 2U }
```

Select the short circuit detection option.

```
enum SDDF_ShortCircuitDetectModeType { SDDF_SHORT_DETECT_ALWAYS_ON = 0U,  
SDDF_SHORT_DETECT_AUX_FILTER_EN = 1U }
```

Select sht short circuit detect conversion mode.

Functions

```
LOCAL_INLINE uint32_t SDDF_HWA_GetCtrl (const SDDF_Type *const pSddf)
```

Get the global control register (SDDF_CTRL) value.

```
LOCAL_INLINE void SDDF_HWA_SetCtrl (SDDF_Type *const pSddf, uint32_t u32Ctrl)
```

Set the global control register (SDDF_CTRL) value.

```
LOCAL_INLINE bool SDDF_HWA_GetClockOut2DisableFlag (const SDDF_Type *const pSddf)
```

Get whether SDDF clock output 2 is disabled.

```
LOCAL_INLINE void SDDF_HWA_SetClockOut2DisableFlag (SDDF_Type *const pSddf, bool bDisable)
```

Set whether to disable clock output 2.

```
LOCAL_INLINE bool SDDF_HWA_GetClockOut1DisableFlag (const SDDF_Type *const pSddf)
```

Get whether SDDF clock output 1 is disabled.

```
LOCAL_INLINE void SDDF_HWA_SetClockOut1DisableFlag (SDDF_Type *const pSddf, bool bDisable)
```

Set whether to disable clock output 1.

```
LOCAL_INLINE bool SDDF_HWA_GetClockOut0DisableFlag (const SDDF_Type *const pSddf)
```

Get whether SDDF clock output 0 is disabled.

LOCAL_INLINE void **SDDF_HWA_SetClockOut0DisableFlag** (SDDF_Type *const pSddf, bool bDisable)

Set whether to disable clock output 0.

LOCAL_INLINE **SDDF_PreDivType** **SDDF_HWA_GetPreDivider** (const SDDF_Type *const pSddf)

Get the function clock pre-divider value.

LOCAL_INLINE void **SDDF_HWA_SetPreDivider** (SDDF_Type *const pSddf, **SDDF_PreDivType** eDiv)

Set the function clock pre-divider value.

LOCAL_INLINE uint32_t **SDDF_HWA_GetDivider** (const SDDF_Type *const pSddf)

Get the clock divider value.

LOCAL_INLINE void **SDDF_HWA_SetDivider** (SDDF_Type *const pSddf, uint32_t u32Div)

Set the clock divider value.

LOCAL_INLINE bool **SDDF_HWA_GetMainEnableFlag** (const SDDF_Type *const pSddf)

Get whether the SDDF function blocks are enabled.

LOCAL_INLINE void **SDDF_HWA_MainEnable** (SDDF_Type *const pSddf)

Enable the SDDF function blocks.

LOCAL_INLINE void **SDDF_HWA_MainDisable** (SDDF_Type *const pSddf)

Disable all SDDF function blocks.

LOCAL_INLINE void **SDDF_HWA_Reset** (SDDF_Type *const pSddf)

Reset all SDDF function blocks and clock configuration bits.

LOCAL_INLINE **SDDF_ModeType** **SDDF_HWA_GetOperationMode** (const SDDF_Type *const pSddf)

Get the SDDF operation mode (normal mode or debug mode)

LOCAL_INLINE void **SDDF_HWA_SetOperationMode** (SDDF_Type *const pSddf, **SDDF_ModeType** eMode)

Set the SDDF operation mode.

LOCAL_INLINE **SDDF_DebugModuleEnableType SDDF_HWA_GetModuleDebugEnableFlag** (const SDDF_Type *const pSddf)

Get whether SDDF module is enabled in debug mode.

LOCAL_INLINE void **SDDF_HWA_SetModuleDebugEnableFlag** (SDDF_Type *const pSddf, **SDDF_DebugModuleEnableType** eEnable)

Set whether SDDF module is enabled in debug mode.

LOCAL_INLINE void **SDDF_HWA_GenerateChannelSwTrigger** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Generate software trigger signal for the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetNormalInterruptEnable** (const SDDF_Type *const pSddf)

Get the normal interrupt (SDDF_NIER) register value.

LOCAL_INLINE void **SDDF_HWA_SetNormalInterruptEnable** (SDDF_Type *const pSddf, uint32_t u32IntEn)

Set the normal interrupt (SDDF_NIER) register value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelZeroCrossDetectIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether zero cross detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelZeroCrossDetectIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable zero cross detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFODataReadyIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel FIFO data ready interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFODataReadyIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel FIFO data ready interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelConvCompleteIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel conversion complete interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelConvCompleteIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel conversion complete interrupt.

LOCAL_INLINE uint32_t **SDDF_HWA_GetAbnormalInterruptEnable0** (const SDDF_Type *const pSddf)

Get the channel abnormal interrupt 0 (SDDF_ABNIER0) register value.

LOCAL_INLINE void **SDDF_HWA_SetAbnormalInterruptEnable0** (SDDF_Type *const pSddf, uint32_t u32IntEn)

Set the channel abnormal interrupt 0 (SDDF_ABNIER0) register value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelHighLimitIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel high limit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelHighLimitIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel high limit detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelLowLimitIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel low limit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelLowLimitIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel low limit detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelWindowLimitIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel window limit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelWindowLimitIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel window limit detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelShortCircuitDetectIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel short circuit detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelShortCircuitDetectIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel short circuit detect interrupt.

LOCAL_INLINE uint32_t **SDDF_HWA_GetAbnormalInterruptEnable1** (const SDDF_Type *const pSddf)

Get the channel abnormal interrupt 1 (SDDF_ABNIER1) register value.

LOCAL_INLINE void **SDDF_HWA_SetAbnormalInterruptEnable1** (SDDF_Type *const pSddf, uint32_t u32IntEn)

Set the channel abnormal interrupt 1 (SDDF_ABNIER1) register value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelSaturationIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel result saturation interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelSaturationIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel result saturation interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelClockAbsenceIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel clock absence detect interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelClockAbsenceIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel clock absence detect interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOOverflowIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel FIFO overflow interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFOOverflowIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel FIFO overflow interrupt.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOUnderflowIntEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether channel FIFO underflow interrupt is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFOUnderflowIntEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable channel FIFO underflow interrupt.

LOCAL_INLINE uint32_t **SDDF_HWA_GetNormalInterruptStatus** (const SDDF_Type *const pSddf)

Get the normal interrupt status register (SDDF_NISR) value.

LOCAL_INLINE void **SDDF_HWA_ClearNormalInterruptStatus** (SDDF_Type *const pSddf)

Clear the normal interrupt status register (SDDF_NISR) value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelZeroCrossDetectStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get zero cross detect status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelZeroCrossDetectStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear zero cross detect flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFODataReadyStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get FIFO data ready status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelFIFODataReadyStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear FIFO data ready flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelConvCompleteStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get channel conversion complete status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelConvCompleteStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear channel conversion complete flag of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetAbnormalInterruptStatus0** (const SDDF_Type *const pSddf)

Get the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

LOCAL_INLINE void **SDDF_HWA_ClearAbnormalInterruptStatus0** (SDDF_Type *const pSddf)

Clear the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelHighLimitStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get out of high limit range status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelHighLimitStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear out of high limit range flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelLowLimitStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get out of low limit range status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelLowLimitStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear out of low limit range flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelWindowLimitStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get out of window limit range status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelWindowLimitStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear out of window limit range flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelShortCircuitDetectStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get short circuit detect status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelShortCircuitDetectStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear short circuit flag of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetAbnormalInterruptStatus1** (const SDDF_Type *const pSddf)

Get the abnormal interrupt status 1 register (SDDF_ABNISR1) value.

LOCAL_INLINE void **SDDF_HWA_ClearAbnormalInterruptStatus1** (SDDF_Type *const pSddf)

Clear the abnormal interrupt status 0 register (SDDF_ABNISR0) value.

LOCAL_INLINE bool **SDDF_HWA_GetChannelSaturationStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get result saturation status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelSaturationStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear result saturation flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelClockAbsenceStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get clock absence detect status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelClockAbsenceStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear clock absence flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOOverflowStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get FIFO overflow status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelFIFOOverflowStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear FIFO overflow flag of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOUnderflowStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get FIFO underflow status of the selected channel.

LOCAL_INLINE void **SDDF_HWA_ClearChannelFIFOUnderflowStatus** (SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Clear FIFO underflow flag of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetMainStatus** (const SDDF_Type *const pSddf)

Get main status of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetClockOutput2ReadyFlag** (const SDDF_Type *const pSddf)

Get whether the modulator clock 2 output is ready.

LOCAL_INLINE bool **SDDF_HWA_GetClockOutput1ReadyFlag** (const SDDF_Type *const pSddf)

Get whether the modulator clock 1 output is ready.

LOCAL_INLINE bool **SDDF_HWA_GetClockOutput0ReadyFlag** (const SDDF_Type *const pSddf)

Get whether the modulator clock 0 output is ready.

LOCAL_INLINE bool **SDDF_HWA_GetChannelConvOngoingStatus** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether conversion of the selected channel is ongoing.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelControl** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get channel control register (SDDF_CCRn) value of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelControl** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32Ctrl)

Set channel control register (SDDF_CCRn) value of the selected channel.

LOCAL_INLINE SDDF_InputSrcType **SDDF_HWA_GetChannelLimitCheckInputSrc** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the result input source of channel limit check.

LOCAL_INLINE void **SDDF_HWA_SetChannelLimitCheckInputSrc** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, SDDF_InputSrcType eSrc)

Set the result input source of the channel limit check.

LOCAL_INLINE SDDF_InputSrcType **SDDF_HWA_GetChannelZeroCrossCheckInputSrc** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the result input source of channel zero cross check.

LOCAL_INLINE void **SDDF_HWA_SetChannelZeroCrossCheckInputSrc** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, SDDF_InputSrcType eSrc)

Set the result input source of the channel zero cross check.

LOCAL_INLINE bool **SDDF_HWA_GetChannelFIFOEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether FIFO is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFOEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable FIFO for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelLimitCheckEnableFlag (const SDDF_Type *const pSddf,  
uint8_t u8ChanIdx)
```

Get whether limit check is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelLimitCheckEnableFlag (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx, bool bEnable)
```

Set whether to enable limit check for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelZeroCrossDetectEnableFlag (const SDDF_Type *const pSddf,  
uint8_t u8ChanIdx)
```

Get whether zero cross detect is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelZeroCrossDetectEnableFlag (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable zero cross detect for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelClockAbsenceDetectEnableFlag (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get whether clock absence detect is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelClockAbsenceDetectEnableFlag (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable clock absence detect for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelShortCircuitDetectEnableFlag (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get whether short circuit detect is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelShortCircuitDetectEnableFlag (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable short circuit detect for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelDMAEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether DMA is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelDMAEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable DMA for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelAuxiliaryFilterEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether auxiliary filter is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelAuxiliaryFilterEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable auxiliary filter for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelMainFilterEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether main filter is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable main filter for the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether the selected channel is enabled.

LOCAL_INLINE void **SDDF_HWA_SetChannelEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelDataRate** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the data rate register (SDDF_CDRn) value of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelDataRate (SDDF_Type *const pSddf, uint8_t u8ChanIdx,  
uint32_t u32DataRate)
```

Set the data rate register (SDDF_CDRn) value of the selected channel.

```
LOCAL_INLINE SDDF_AuxFilterConvModeType SDDF_HWA_GetChannelAuxiliaryFilterConvMode (const  
SDDF_Type *const pSddf, uint8_t u8ChanIdx)
```

Get the auxiliary filter conversion mode of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelAuxiliaryFilterConvMode (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx, SDDF_AuxFilterConvModeType eConvMode)
```

Set the auxiliary filter conversion mode of the selected channel.

```
LOCAL_INLINE SDDF_AuxFilterOrderType SDDF_HWA_GetChannelAuxiliaryFilterOrder (const  
SDDF_Type *const pSddf, uint8_t u8ChanIdx)
```

Get the auxiliary filter order of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelAuxiliaryFilterOrder (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx, SDDF_AuxFilterOrderType eOrder)
```

Set the auxiliary filter order of the selected channel.

```
LOCAL_INLINE uint32_t SDDF_HWA_GetChannelAuxiliaryFilterOversampleRate (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get the auxiliary filter oversampling rate of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelAuxiliaryFilterOversampleRate (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx, uint32_t u32SampleRate)
```

Set the auxiliary filter oversampling rate of the selected channel.

```
LOCAL_INLINE SDDF_MainFilterConvModeType SDDF_HWA_GetChannelMainFilterConvMode (const  
SDDF_Type *const pSddf, uint8_t u8ChanIdx)
```

Get the main filter conversion mode of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterConvMode** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_MainFilterConvModeType** eConvMode)

Set the main filter conversion mode of the selected channel.

LOCAL_INLINE **SDDF_MainFilterOrderType** **SDDF_HWA_GetChannelMainFilterOrder** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the main filter order of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterOrder** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_MainFilterOrderType** eOrder)

Set the main filter order of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelMainFilterOversampleRate** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the main filter oversampling rate of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterOversampleRate** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32SampleRate)

Set the main filter oversampling rate of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelConfig** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the configuration register (SDDF_CFRn) value of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelConfig** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32Config)

Set the configuration register (SDDF_CFRn) value of the selected channel.

LOCAL_INLINE **SDDF_TriggerEventType** **SDDF_HWA_GetChannelTriggerEvent** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the trigger level type of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelTriggerEvent** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_TriggerEventType** eEvent)

Set the trigger level type of the selected channel.

```
LOCAL_INLINE SDDF_TriggerSrcType SDDF_HWA_GetChannelTriggerSource (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get the trigger source type of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelTriggerSource (SDDF_Type *const pSddf, uint8_t u8ChanIdx,  
SDDF_TriggerSrcType eSrc)
```

Set the trigger source type of the selected channel.

```
LOCAL_INLINE SDDF_ClockEdgeType SDDF_HWA_GetChannelInputClockEdge (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get the clock edge for conversion of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelInputClockEdge (SDDF_Type *const pSddf, uint8_t u8ChanIdx,  
SDDF_ClockEdgeType eClkEdge)
```

Set the clock edge for conversion of the selected channel.

```
LOCAL_INLINE SDDF_ClockSrcType SDDF_HWA_GetChannelInputClockSource (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get the input clock select of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelInputClockSource (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx, SDDF_ClockSrcType eSrc)
```

Set the input clock select of the selected channel.

```
LOCAL_INLINE SDDF_BitFormatType SDDF_HWA_GetChannelInputBitFormat (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get the input bit format of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelInputBitFormat (SDDF_Type *const pSddf, uint8_t u8ChanIdx,  
SDDF_BitFormatType eFormat)
```

Set the input bit format of the selected channel.

LOCAL_INLINE uint8_t **SDDF_HWA_GetChannelFIFOWatermark** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the FIFO watermark of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelFIFOWatermark** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint8_t u8Watermark)

Set the FIFO watermark of the selected channel.

LOCAL_INLINE **SDDF_ResultFormatType** **SDDF_HWA_GetChannelResultDataFormat** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the result data format of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelResultDataFormat** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ResultFormatType** eFormat)

Set the result data format of the selected channel.

LOCAL_INLINE **SDDF_MainFilterShiftType** **SDDF_HWA_GetChannelMainFilterShift** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the main filter shift of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelMainFilterShift** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_MainFilterShiftType** eShift)

Set the main filter shift to get correct 24 bit precision of the selected channel.

LOCAL_INLINE uint32_t **SDDF_HWA_GetChannelProtectionConfig** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the protection settings (SDDF_CPRn) value of the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelProtectionConfig** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint32_t u32Config)

Set the protection settings (SDDF_CPRn) value of the selected channel.

LOCAL_INLINE bool **SDDF_HWA_GetChannelHighLimitBreakSignalEnableFlag** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get whether the high limit break signal is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelHighLimitBreakSignalEnableFlag (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable high limit break signal for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelWindowLimitBreakSignalEnableFlag (const SDDF_Type  
*const pSddf, uint8_t u8ChanIdx)
```

Get whether the window limit break signal is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelWindowLimitBreakSignalEnableFlag (SDDF_Type *const  
pSddf, uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable window limit break signal for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelLowLimitBreakSignalEnableFlag (const SDDF_Type *const  
pSddf, uint8_t u8ChanIdx)
```

Get whether the low limit break signal is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelLowLimitBreakSignalEnableFlag (SDDF_Type *const pSddf,  
uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable low limit break signal for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelShortCircuitBreakSignalEnableFlag (const SDDF_Type  
*const pSddf, uint8_t u8ChanIdx)
```

Get whether short circuit detect break signal is enabled for the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelShortCircuitBreakSignalEnableFlag (SDDF_Type *const  
pSddf, uint8_t u8ChanIdx, bool bEnable)
```

Set whether to enable short circuit detect break signal for the selected channel.

```
LOCAL_INLINE bool SDDF_HWA_GetChannelClockAbsenceBreakSignalEnableFlag (const SDDF_Type  
*const pSddf, uint8_t u8ChanIdx)
```

Get whether clock absence detect break signal is enabled for the selected channel.

LOCAL_INLINE void **SDDF_HWA_SetChannelClockAbsenceBreakSignalEnableFlag** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, bool bEnable)

Set whether to enable clock absence detect break signal for the selected channel.

LOCAL_INLINE uint8_t **SDDF_HWA_GetChannelClockAbsenceLimitThreshold** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the clock absence detector limitation threshold.

LOCAL_INLINE void **SDDF_HWA_SetChannelClockAbsenceLimitThreshold** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, uint8_t u8Thres)

Set the clock absence detector limitation threshold.

LOCAL_INLINE **SDDF_ChannelLimitOptionType** **SDDF_HWA_GetChannelLimitOption** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the channel limit check option.

LOCAL_INLINE void **SDDF_HWA_SetChannelLimitOption** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ChannelLimitOptionType** eOption)

Set the channel limit check option.

LOCAL_INLINE **SDDF_ShortCircuitOptionType** **SDDF_HWA_GetChannelShortCircuitOption** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the channel short circuit detect option.

LOCAL_INLINE void **SDDF_HWA_SetChannelShortCircuitOption** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ShortCircuitOptionType** eOption)

Set the channel short circuit detect option.

LOCAL_INLINE **SDDF_ShortCircuitDetectModeType** **SDDF_HWA_GetShortCircuitDetectMode** (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)

Get the channel short circuit detect conversion mode.

LOCAL_INLINE void **SDDF_HWA_SetShortCircuitDetectMode** (SDDF_Type *const pSddf, uint8_t u8ChanIdx, **SDDF_ShortCircuitDetectModeType** eMode)

Set the channel short circuit detect conversion mode.

```
LOCAL_INLINE uint8_t SDDF_HWA_GetShortCircuitLimitThreshold (const SDDF_Type *const pSddf,  
uint8_t u8ChanIdx)
```

Get the threshold value for short circuit detector counter of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetShortCircuitLimitThreshold (SDDF_Type *const pSddf, uint8_t  
u8ChanIdx, uint8_t u8Thres)
```

Set the threshold value for short circuit detector counter of the selected channel.

```
LOCAL_INLINE int32_t SDDF_HWA_GetChannelBias (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)
```

Get the bias value for the selected channel main filter.

```
LOCAL_INLINE void SDDF_HWA_SetChannelBias (SDDF_Type *const pSddf, uint8_t u8ChanIdx, int32_t  
s32Bias)
```

Set the bias value for the selected channel main filter.

```
LOCAL_INLINE uint32_t SDDF_HWA_GetChannelLowLimit (const SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Get the low limit threshold for limit check of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelLowLimit (SDDF_Type *const pSddf, uint8_t u8ChanIdx,  
uint32_t u32LowLimit)
```

Set the low limit threshold for limit check of the selected channel.

```
LOCAL_INLINE uint32_t SDDF_HWA_GetChannelHighLimit (const SDDF_Type *const pSddf, uint8_t  
u8ChanIdx)
```

Get the high limit threshold for limit check of the selected channel.

```
LOCAL_INLINE void SDDF_HWA_SetChannelHighLimit (SDDF_Type *const pSddf, uint8_t u8ChanIdx,  
uint32_t u32HighLimit)
```

Set the high limit threshold for limit check of the selected channel.

```
LOCAL_INLINE uint32_t SDDF_HWA_GetChannelData (const SDDF_Type *const pSddf, uint8_t u8ChanIdx)
```

Get the result data of the selected channel.

Detailed Description

Hardware access layer for SDDF.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_sec.h File Reference

sec hardware access layer

```
#include "device_header.h"
```

Include dependency graph for HwA_sec.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **SEC_HWA_EnDebug** (SEC_Type *const pSec)

Enable the debug module.

LOCAL_INLINE uint16_t **SEC_HWA_GetSScontrol0** (SEC_Type *const pSec)

Get the system security KEY0.

LOCAL_INLINE uint16_t **SEC_HWA_GetSScontrol1** (SEC_Type *const pSec)

Get the system security KEY1.

LOCAL_INLINE void **SEC_HWA_ReEnDebug** (SEC_Type *const pSec)

Re-enable the Debug mode.

LOCAL_INLINE bool **SEC_HWA_GetReEnDebug** (SEC_Type *const pSec)

Get the Re-enable Debug permission.

LOCAL_INLINE void **SEC_HWA_WriteUnlock** (SEC_Type *const pSec)

Enable the write operation for SEC register.

LOCAL_INLINE bool **SEC_HWA_GetWritePer** (SEC_Type *const pSec)

Get the sec write permission.

LOCAL_INLINE void **SEC_HWA_WriteReEnKeyn** (SEC_Type *const pSec, uint8_t count, uint32_t key)

Enable the write operation for SEC register.

LOCAL_INLINE void **SEC_HWA_EnTest** (SEC_Type *const pSec)

Enable the Test module.

LOCAL_INLINE void **SEC_HWA_ReEnTest** (SEC_Type *const pSec)

Re-Enable Test mode.

LOCAL_INLINE bool **SEC_HWA_GetReEnTest** (SEC_Type *const pSec)

Get the Re-enable Test permission.

LOCAL_INLINE void **SEC_HWA_ReEnTestKey** (SEC_Type *const pSec, uint32_t testkey)

write Test re-enable mode key

LOCAL_INLINE void **SEC_HWA_EnME** (SEC_Type *const pSec)

Enable the Mass Erase.

LOCAL_INLINE void **SEC_HWA_EnWriteB0NVR** (SEC_Type *const pSec)

Enable the Block 0 NVR write .

LOCAL_INLINE void **SEC_HWA_DisWriteB0NVR** (SEC_Type *const pSec)

Disable the Block 0 NVR write .

LOCAL_INLINE void **SEC_HWA_EnEraseB0NVR** (SEC_Type *const pSec)

Enable the Block 0 NVR erase .

LOCAL_INLINE void **SEC_HWA_DisEraseB0NVR** (SEC_Type *const pSec)

Disable the Block 0 NVR erase .

LOCAL_INLINE uint8_t **SEC_HWA_GetFastBootClock** (SEC_Type *const pSec)

Get the Fast Boot Select.

LOCAL_INLINE bool **SEC_HWA_GetNmiPin** (SEC_Type *const pSec)

NMI Function Enable/Disable.

LOCAL_INLINE bool **SEC_HWA_GetBootRom** (SEC_Type *const pSec)

Boot Rom Configuration.

LOCAL_INLINE bool **SEC_HWA_GetIsSpStatus** (SEC_Type *const pSec)

ISP Mode Status.

LOCAL_INLINE bool **SEC_HWA_GetUKAS** (SEC_Type *const pSec)

User Key Access Enable. Only valid under non-secure boot.

LOCAL_INLINE bool **SEC_HWA_GetSB** (SEC_Type *const pSec)

Secure Boot Disable (Value loaded from NVR sector)

LOCAL_INLINE bool **SEC_HWA_GetDEAUEn** (SEC_Type *const pSec)

Host Debug Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

LOCAL_INLINE bool **SEC_HWA_GetISPAU** (SEC_Type *const pSec)

ISP Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

LOCAL_INLINE uint8_t **SEC_HWA_GetUartBR** (SEC_Type *const pSec)

Get the FCUART Baud Rate for ISP.

LOCAL_INLINE uint8_t **SEC_HWA_GetCanBR** (SEC_Type *const pSec)

Get CAN Baud Rate for ISP.

LOCAL_INLINE uint8_t **SEC_HWA_GetOSCFre** (SEC_Type *const pSec)

Get OSC Frequency.

LOCAL_INLINE bool **SEC_HWA_GetOSCAvail** (SEC_Type *const pSec)

Get whether OSC Available.

LOCAL_INLINE bool **SEC_HWA_GetMBBKEN** (SEC_Type *const pSec)

Get whether Debug Mailbox Backdoor Key Enable.

LOCAL_INLINE uint32_t **SEC_HWA_GetBLMask** (SEC_Type *const pSec)

Get Bootloader Verification Mask.

LOCAL_INLINE uint8_t **SEC_HWA_GetIsplns** (SEC_Type *const pSec)

Get the ISP Instance Select.

LOCAL_INLINE uint8_t **SEC_HWA_GetBLVer** (SEC_Type *const pSec)

Get Bootloader Verification Algorithm.

LOCAL_INLINE uint32_t **SEC_HWA_GetBLAddr** (SEC_Type *const pSec)

Get the bootloader address.

LOCAL_INLINE uint8_t **SEC_HWA_GetLCStaus** (SEC_Type *const pSec)

Get the life cycle status of the chip.

LOCAL_INLINE bool **SEC_HWA_GetHUKRead** (SEC_Type *const pSec)

Get the Host User Key Read Protection.

LOCAL_INLINE bool **SEC_HWA_GetHUKWrite** (SEC_Type *const pSec)

Get the Host User Key write Protection.

LOCAL_INLINE bool **SEC_HWA_GetHUKEraser** (SEC_Type *const pSec)

Get the Host User Key erase Protection.

LOCAL_INLINE bool **SEC_HWA_GetHostNvrRead** (SEC_Type *const pSec)

Get the Host NVR Read Protection.

LOCAL_INLINE bool **SEC_HWA_GetHostNvrWrite** (SEC_Type *const pSec)

Get the Host NVR write Protection.

LOCAL_INLINE bool **SEC_HWA_GetHostNvrErase** (SEC_Type *const pSec)

Get the Host NVR erase Protection.

LOCAL_INLINE bool **SEC_HWA_GetHsmUKERase** (SEC_Type *const pSec)

Get the HSM User Key Erase Protection.

LOCAL_INLINE bool **SEC_HWA_GetHsmNvrErase** (SEC_Type *const pSec)

Get the HSM NVR Erase Protection.

Detailed Description

sec hardware access layer

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_sent.h File Reference

Hardware access layer for SENT.

```
#include "device_header.h"
```

Include dependency graph for HwA_sent.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define DMA_CH_TO_DCHPRI(x) ((x) ^ 3U)
```

Enumerations

```
enum SENT_NibbleDataModeType { SENT_NIBBLE_DATA_MODE_A = 0x0U,  
SENT_NIBBLE_DATA_MODE_H1 = 0x1U, SENT_NIBBLE_DATA_MODE_H2 = 0x2U,  
SENT_NIBBLE_DATA_MODE_H3 = 0x3U, SENT_NIBBLE_DATA_MODE_H4 = 0x4U,  
SENT_NIBBLE_DATA_MODE_H5 = 0x5U, SENT_NIBBLE_DATA_MODE_H6 = 0x6U,  
SENT_NIBBLE_DATA_MODE_H7 = 0x7U }
```

sent running status

```
enum SENT_SlowMessageType { SENT_SLOW_MESSAGE_SHORT = 0x0U,  
SENT_SLOW_MESSAGE_ENHANCE_12DATA_8ID = 0x1U, SENT_SLOW_MESSAGE_ENHANCE_16DATA_4ID  
= 0x2U }
```

sent slow message type

```
enum SENT_SPCTriggerType { SENT_SPC_SW_TRIGGER = 0x0U, SENT_SPC_HW_TRIGGER = 0x1U }
```

sent SPC trigger type

```
enum SENT_SPCTickType { SENT_SPC_TICK_PRE_RECEIVED_MESSAGE = 0x0U,  
SENT_SPC_TICK_CONFIG = 0x1U }
```

sent SPC tick base

Functions

```
LOCAL_INLINE void SENT_HWA_SetGlobalPreScaler (SENT_Type *const pSent, uint8_t u8PreScaler)
```

Set SENT Global pre-scaler value.

LOCAL_INLINE void **SENT_HWA_SetDMAWaterMark** (SENT_Type *const pSent, uint8_t u8WaterMark)
Set SENT Global DMA watermark value.

LOCAL_INLINE void **SENT_HWA_EnableFastMessageFIFOOverflowInterrupt** (SENT_Type *const pSent)
Enable SENT Fast message FIFO overflow interrupt.

LOCAL_INLINE void **SENT_HWA_DisableFastMessageFIFOOverflowInterrupt** (SENT_Type *const pSent)
Disable SENT Fast message FIFO overflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableFastMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)
Enable SENT Fast message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_DisableFastMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)
Disable SENT Fast message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableSlowMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)
Enable SENT slow message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_DisableSlowMessagedmaReadUnderflowInterrupt** (SENT_Type *const pSent)
Disable SENT slow message read underflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableDebugMode** (SENT_Type *const pSent)
Enable SENT debug mode.

LOCAL_INLINE void **SENT_HWA_DisableDebugMode** (SENT_Type *const pSent)
Disable SENT debug mode.

LOCAL_INLINE void **SENT_HWA_EnableGlobal** (SENT_Type *const pSent)
Enable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_DisableGlobal** (SENT_Type *const pSent)

Disable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_EnableDataOverflowFlagFastClear** (SENT_Type *const pSent)

Enable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_DisableDataOverflowFlagFastClear** (SENT_Type *const pSent)

Disable Global SENT debug mode.

LOCAL_INLINE void **SENT_HWA_EnableChannelReceive** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel receive.

LOCAL_INLINE void **SENT_HWA_DisableChannelReceive** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel receive.

LOCAL_INLINE bool **SENT_HWA_GetChannelReceive** (const SENT_Type *const pSent, uint8_t u8Channel)

Get SENT channel receive.

LOCAL_INLINE bool **SENT_HWA_GetSlowMessageUnderflowFlag** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel slow message dma read underflow flag.

LOCAL_INLINE void **SENT_HWA_ClearSlowMessageUnderflowFlag** (SENT_Type *const pSent, uint8_t u8Channel)

Clear Channel slow message dma read underflow flag.

LOCAL_INLINE bool **SENT_HWA_GetFastMessageReadyFlag** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message ready flag.

LOCAL_INLINE void **SENT_HWA_ClearFastMessageReadyFlag** (SENT_Type *const pSent, uint8_t u8Channel)

Clear Channel fast message ready flag.

LOCAL_INLINE bool **SENT_HWA_GetSlowMessageReadyFlag** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel slow message ready flag.

LOCAL_INLINE void **SENT_HWA_ClearSlowMessageReadyFlag** (SENT_Type *const pSent, uint8_t u8Channel)

Clear Channel slow message ready flag.

LOCAL_INLINE void **SENT_HWA_EnableChannelFIFO** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel FIFO function.

LOCAL_INLINE void **SENT_HWA_DisableChannelFIFO** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel receive function.

LOCAL_INLINE void **SENT_HWA_SetChannelNibbleNumber** (SENT_Type *const pSent, uint8_t u8Channel, uint8_t u8Number)

Set SENT channel data nibble number.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelNibbleNumber** (const SENT_Type *const pSent, uint8_t u8Channel)

Get SENT channel data nibble number.

LOCAL_INLINE bool **SENT_HWA_GetFastMessageFIFOOverflowFlag** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message FIFO overflow flag.

LOCAL_INLINE void **SENT_HWA_ClearFastMessageFIFOOverflowFlag** (SENT_Type *const pSent, uint8_t u8Channel)

Clear Channel fast message FIFO overflow flag.

LOCAL_INLINE bool **SENT_HWA_GetFastMessageDMAUnderflowFlag** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message DMA underflow flag.

LOCAL_INLINE void **SENT_HWA_ClearFastMessageDMAUnderflowFlag** (SENT_Type *const pSent, uint8_t u8Channel)

Clear Channel fast message DMA underflow flag.

LOCAL_INLINE void **SENT_HWA_SetChannelNibbleDataMode** (SENT_Type *const pSent, uint8_t u8Channel, **SENT_NibbleDataModeType** eMode)

Set SENT channel nibble data mode.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel fast message DMA request.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel fast message DMA request.

LOCAL_INLINE void **SENT_HWA_EnableChannelSlowMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel slow message DMA request.

LOCAL_INLINE void **SENT_HWA_DisableChannelSlowMessageDmaRequest** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel slow message DMA request.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Fast message interrupt.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Fast message interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelSlowMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel slow message interrupt.

LOCAL_INLINE void **SENT_HWA_DisableChannelSlowMessageInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel slow message interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCompensate** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Compensate.

LOCAL_INLINE void **SENT_HWA_DisableChannelCompensate** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Compensate.

LOCAL_INLINE void **SENT_HWA_SetChannelPreScaler** (SENT_Type *const pSent, uint8_t u8Channel, uint16_t u16PreScaler)

Set SENT channel pre-scaler.

LOCAL_INLINE uint16_t **SENT_HWA_GetChannelCompensatePreScaler** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel Prescaler value after compensate.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelStatus** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel status.

LOCAL_INLINE void **SENT_HWA_ClearChannelStatus** (SENT_Type *const pSent, uint8_t u8Channel, uint32_t u32W1CBit)

Clear Channel status.

LOCAL_INLINE void **SENT_HWA_EnableChannelAllErrorInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel all error interrupt.

LOCAL_INLINE void **SENT_HWA_DisableChannelAllErrorInterrupt** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel all error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelBusIdleInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel idle flag interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalResyncInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel sync/calibration pulse error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalERRInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable Sync/calibration pulse difference exceed 1.5625% error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalFailInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Calibration 20% pass 25% fail interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelSOVFLInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Slow message overflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelFOVFLInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel fast message overflow interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelNibbleErrorInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Nibble value error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelPPDiagInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Previous pulse diagnosis error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelCALDiagInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Calibration diagnosis over 25% error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelFCRCInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Fast message CRC check error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelSCRCInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel slow message CRC check error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelEdgeERRInterrupt** (SENT_Type *const pSent, uint8_t u8Channel, bool bEn)

Enable/disable SENT channel Falling edge number error interrupt.

LOCAL_INLINE void **SENT_HWA_EnableChannelAltCRC** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Alternative 4-bit CRC algorithm.

LOCAL_INLINE void **SENT_HWA_DisableChannelAltCRC** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Alternative 4-bit CRC algorithm.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageDataChange** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Fast message data change.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageDataChange** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Fast message data change.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalValid20To25** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel calibration valid range from 20% tp 25%.

LOCAL_INLINE void **SENT_HWA_DisableChannelCalValid20To25** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel calibration valid range from 20% tp 25%.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalValidDiagnostic** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel calibration valid diagnostic.

LOCAL_INLINE void **SENT_HWA_DisableChannelCalValidDiagnostic** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel calibration valid diagnostic.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel fast message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel fast message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageCRCWithSC** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Fast message crc with S&C.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageCRCWithSC** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Fast message crc with S&C.

LOCAL_INLINE void **SENT_HWA_EnableChannelSlowMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel slow message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_DisableChannelSlowMessageAugmentation** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel slow message CRC with Augmentation.

LOCAL_INLINE void **SENT_HWA_EnableChannelPausePulse** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Pause pulse.

LOCAL_INLINE void **SENT_HWA_DisableChannelPausePulse** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Successive calibration pulses diagnostic option1.

LOCAL_INLINE void **SENT_HWA_EnableChannelSPCOption1** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Successive calibration pulses diagnostic option1.

LOCAL_INLINE void **SENT_HWA_DisableChannelSPCOption1** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Pause pulse.

LOCAL_INLINE void **SENT_HWA_EnableChannelFastMessageCRCCheck** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel Fast message crc check.

LOCAL_INLINE void **SENT_HWA_DisableChannelFastMessageCRCCheck** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel Fast message crc check.

LOCAL_INLINE void **SENT_HWA_SetChannelDigitalFilterCount** (SENT_Type *const pSent, uint8_t u8Channel, uint8_t u8Count)

Set SENT channel digital filter count.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelDataNibble** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message data nibble.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelFastMessageCRCNibble** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message crc nibble.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelFastMessageStatusNibble** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message status communication nibble.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelFastMessageTimeStamp** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel fast message status timestamp.

LOCAL_INLINE **SENT_SlowMessageType** **SENT_HWA_GetChannelSlowMessageType** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel slow message type.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelBit3EnhancedID7_4_OR_ID3_0** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel enhanced serial message id bit 7 to bit4 or id bit 3 to id bit 0.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelBit3EnhancedID3_0_OR_DATA15_12** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel enhanced serial message id bit 3 to bit0 or data bit 15 to id bit 12.

LOCAL_INLINE uint8_t **SENT_HWA_GetChannelBit2CRC** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel slow message bit2 with CRC.

LOCAL_INLINE uint16_t **SENT_HWA_GetChannelBit2DATA** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel slow message bit2 with DATA.

LOCAL_INLINE uint32_t **SENT_HWA_GetChannelSlowMessageTimeStamp** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel slow message status timestamp.

LOCAL_INLINE uint32_t **SENT_HWA_GetChanneReceivedFrameNumber** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel number of received frames.

LOCAL_INLINE uint32_t **SENT_HWA_GetChanneReceivedEdgeNumber** (const SENT_Type *const pSent, uint8_t u8Channel)

Get Channel number of received edges.

LOCAL_INLINE void **SENT_HWA_ClearChannelReceivedFrameNumber** (SENT_Type *const pSent, uint8_t u8Channel)

Clear the number of received frames.

LOCAL_INLINE void **SENT_HWA_ClearChannelReceivedEdgeNumber** (SENT_Type *const pSent, uint8_t u8Channel)

Clear the number of received edges.

LOCAL_INLINE void **SENT_HWA_EnableChannelSPCMode** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel SPC mode.

LOCAL_INLINE void **SENT_HWA_DisableChannelSPCMode** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel SPC mode.

LOCAL_INLINE void **SENT_HWA_EnableChannelCalibrationDiag** (SENT_Type *const pSent, uint8_t u8Channel)

Enable SENT channel calibration diagnosis in SPC mod.

LOCAL_INLINE void **SENT_HWA_DisableChannelCalibrationDiag** (SENT_Type *const pSent, uint8_t u8Channel)

Disable SENT channel calibration diagnosis in SPC mod.

LOCAL_INLINE void **SENT_HWA_SetChannelSPCTriggerMethod** (SENT_Type *const pSent, uint8_t u8Channel, **SENT_SPCTriggerType** eTrigger)

Set the SPC pulse trigger method.

```
LOCAL_INLINE void SENT_HWA_SetChannelSPCPulseDelay (SENT_Type *const pSent, uint8_t u8Channel,  
uint8_t u8Delay)
```

Set the delay between SPC-trigger assert and finally generate a SPC pulse.

```
LOCAL_INLINE void SENT_HWA_SetChannelSPCTickBase (SENT_Type *const pSent, uint8_t u8Channel,  
SENT_SPCTickType eTick)
```

Set the delay between SPC-trigger assert and finally generate a SPC pulse.

```
LOCAL_INLINE void SENT_HWA_StartChannelSPC (SENT_Type *const pSent, uint8_t u8Channel)
```

trigger the SENT to generate a SPC pulse by software

```
LOCAL_INLINE void SENT_HWA_SetChannelSPCPulseWidth (SENT_Type *const pSent, uint8_t u8Channel,  
uint8_t u8Width)
```

Set the width of the SPC pulse.

```
LOCAL_INLINE void SENT_HWA_SetChannelIdleCount (SENT_Type *const pSent, uint8_t u8Channel,  
uint8_t u8Count)
```

Set the idle count determines how long the bus idle flag will assert when bus is idle.

```
LOCAL_INLINE bool SENT_HWA_GetChanneSPCBusyFlag (const SENT_Type *const pSent, uint8_t  
u8Channel)
```

Get the flag indicating the spc is busy.

Detailed Description

Hardware access layer for SENT.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

Macro Definition Documentation

```
#define DMA_CH_TO_DCHPRI( x)  ((x) ^ 3U)
```

HwA_smc.h File Reference

SMC hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_smc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum SMC_LpwModeCtrlType { SMC_STOP_MODE = 0U, SMC_STANDBY_MODE = 4U }
```

SMC stop mode control.

```
enum SMC_StandbyModeType { SMC_CFG_STANDBY_0 = 0U, SMC_CFG_STANDBY_1 = 1U,  
SMC_CFG_STANDBY_2 = 2U, SMC_CFG_STANDBY_3 = 3U }
```

SMC standby mode.

```
enum SMC_ClockOutSrcType { SMC_CLKOUT_SCG_CLKOUT = 0U, SMC_CLKOUT_FOSC_DIVM_CLK = 2U,  
SMC_CLKOUT_SLOW_CLK = 3U, SMC_CLKOUT_SIRC_DIVM_CLK = 4U, SMC_CLKOUT_PLL1_DIVM_CLK =  
5U, SMC_CLKOUT_FIRC_DIVM_CLK = 6U, SMC_CLKOUT_CORE_CLK = 7U, SMC_CLKOUT_PLL0_DIVM_CLK =  
8U, SMC_CLKOUT_BUS_CLK = 9U, SMC_CLKOUT_SIRC_128K_CLK = 10U, SMC_CLKOUT_AON_CLK =  
12U, SMC_CLKOUT_RTC_CLK = 14U }
```

SMC clock out source type.

```
enum SMC_ClockOutDivType { SMC_CLKOUT_DIV_BY1 = 0U, SMC_CLKOUT_DIV_BY2 = 1U,  
SMC_CLKOUT_DIV_BY3 = 2U, SMC_CLKOUT_DIV_BY4 = 3U, SMC_CLKOUT_DIV_BY5 = 4U,  
SMC_CLKOUT_DIV_BY6 = 5U, SMC_CLKOUT_DIV_BY7 = 6U, SMC_CLKOUT_DIV_BY8 = 7U }
```

SMC clock out divide ratio type.

```
enum SMC_AON32KClkSrcType { SMC_AON32K_SIRCDIV_32K_CLK = 1U, SMC_AON32K_SOSC32K_CLK =  
2U, SMC_AON32K_SIRC32K_CLK = 3U }
```

Data type for SMC_AONCLKSR[32KAONCLKSR], set AON32KCLK source clock.

```
enum SMC_RTCClkSrcType { SMC_RTC_FOSCDIVL_CLK = 0U, SMC_RTC_SIRCDIV_32K_CLK = 1U,  
SMC_RTC_SOSC_CLK = 2U, SMC_RTC_SIRC32K_CLK = 3U }
```

Data type for SMC_RTCCLKSEL[RTCCLKSEL], set RTCCLK source clock.

enum **SMC_AONClkSrcType** { **SMC_AON_SIRCDIV_128K_CLK** = 0U, **SMC_AON_SIRC32K_CLK** = 1U, **SMC_AON_SIRCDIV_32K_CLK** = 2U, **SMC_AON_SIRC32_1K_CLK** = 3U }

Data type for SMC_AONCLKSR[AONCLKSR], set AONCLK source clock.

Functions

LOCAL_INLINE void **SMC_HWA_ClearStopModeCtrl** (void)

Clear stop mode control value.

LOCAL_INLINE void **SMC_HWA_SetStopModeCtrl** (SMC_LpwModeCtrlType eMode)

Set stop mode control.

LOCAL_INLINE void **SMC_HWA_ClearStandbyMode** (void)

Clear standby mode.

LOCAL_INLINE void **SMC_HWA_SetStandbyMode** (SMC_StandbyModeType eMode)

Set standby mode.

Detailed Description

SMC hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_ssi.h File Reference

ssi hardware access layer

```
#include "device_header.h"
```

Include dependency graph for HwA_ssi.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

ssi hardware access layer

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_stcu.h File Reference

Safety Test and Control Unit.

```
#include "device_header.h"
```

Include dependency graph for HwA_stcu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE uint32_t **STCU_HWA_GetSelfTestStatus** (void)

Get Self Test Status.

LOCAL_INLINE void **STCU_HWA_ClearSelfTestStatus** (void)

Clear all Self Test Status.

LOCAL_INLINE void **STCU_HWA_MbistSelect** (uint32_t u32MbistSel)

Mbist select for self-test.

LOCAL_INLINE uint32_t **STCU_GetMbistDone** (void)

Get Mbist Done Flag.

LOCAL_INLINE uint32_t **STCU_GetMbistFail** (void)

Get Mbist Fail Flag.

LOCAL_INLINE uint32_t **STCU_HWA_CheckMbistStatus** (void)

Check Mbist execution failed status.

LOCAL_INLINE uint32_t **STCU_HWA_GetHardwareRamInitStatus** (void)

Get STCU Hardware Ram Initial Status.

LOCAL_INLINE void **STCU_HWA_SetRamInitType** (uint32_t u32InitRamType)

STCU RAM Init Mode set, can select more than one, for example: STCU_INIT_RAM_TYPE_SRAM0 / STCU_INIT_RAM_TYPE_SRAM1.

LOCAL_INLINE void **STCU_HWA_SetInterrupt** (uint8_t bIntEn, uint8_t bSequenceErrorIntEn, uint8_t bSizeErrorIntEn)

Set STCU Interrupt.

LOCAL_INLINE void **STCU_HWA_StartRamInit** (uint8_t eInitRamMode, uint8_t bLock)

STCU Ram Initial Start.

LOCAL_INLINE uint32_t **STCU_HWA_GetRamInitDone** (void)

Get Ram Initial Done flag.

LOCAL_INLINE uint32_t **STCU_HWA_GetRamInitStatus** (void)

Get Ram Initial status.

LOCAL_INLINE void **STCU_HWA_SetLBISTPatternAmount** (uint32_t u32Amount)

Set the LBIST pattern amount control value N.

LOCAL_INLINE void **STCU_HWA_SetMBISTSWAlg** (bool bFullTest, bool bSRAMInit)

Set the MBIST algorithm for SW trigger self-test.

LOCAL_INLINE void **STCU_HWA_SetSafetyKey** (uint32_t u32Key)

Set safety key.

LOCAL_INLINE void **STCU_HWA_SwTriggerA** (void)

Enable A self-test software trigger.

LOCAL_INLINE void **STCU_HWA_SwTriggerB** (void)

Enable B self-test software trigger.

LOCAL_INLINE void **STCU_HWA_SetSelfTestCTRL** (uint32_t u32Reg)

Set self test ctrl register.

LOCAL_INLINE void **STCU_HWA_EnableInterrupt** (void)

Enable interrupt.

LOCAL_INLINE void **STCU_HWA_DisableInterrupt** (void)

Disable interrupt.

LOCAL_INLINE uint32_t **STCU_HWA_GetInterruptFlag** (void)

Get STCU interrupt flags.

LOCAL_INLINE void **STCU_HWA_ClearInterruptFlag** (void)

Clear STCU interrupt flags.

LOCAL_INLINE void **STCU_HWA_SetExpectedMisr** (uint32_t u32Misr)

Set STCU LBIST expected misr value.

LOCAL_INLINE uint32_t **STCU_HWA_GetExpectedMisr** (void)

Get STCU LBIST expected misr value.

LOCAL_INLINE uint32_t **STCU_HWA_GetActualMisr** (void)

Get STCU LBIST actual misr value.

Detailed Description

Safety Test and Control Unit.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

Function Documentation

LOCAL_INLINE uint32_t STCU_GetMbistDone (void)

Get Mbist Done Flag.

Returns

All Mbist Done flags, refer to "STCU_MbistDoneType"

LOCAL_INLINE uint32_t STCU_GetMbistFail (void)

Get Mbist Fail Flag.

Returns

All Mbist Fail flags "STCU_MbistFailedType"

LOCAL_INLINE uint32_t STCU_HWA_CheckLbistStatus (void)

Check Lbist execution failed status.

Returns

LBIST status, refer to "STCU_LbistStatusType"

LOCAL_INLINE void STCU_HWA_ClearInterruptFlag (void)

Clear STCU interrupt flags.

LOCAL_INLINE void STCU_HWA_ClearSelfTestStatus (void)

Clear all Self Test Status.

LOCAL_INLINE void STCU_HWA_DisableInterrupt (void)

Disable interrupt.

LOCAL_INLINE void STCU_HWA_EnableInterrupt (void)

Enable interrupt.

LOCAL_INLINE uint32_t STCU_HWA_GetActualMisr (void)

Get STCU LBIST actual misr value.

LOCAL_INLINE uint32_t STCU_HWA_GetExpectedMisr (void)

Get STCU LBIST expected misr value.

LOCAL_INLINE uint32_t STCU_HWA_GetHardwareRamInitStatus (void)

Get STCU Hardware Ram Initial Status.

Returns

Initial Status, refer to "STCU_HardwareInitRamStatusType"

LOCAL_INLINE uint32_t STCU_HWA_GetInterruptFlag (void)

Get STCU interrupt flags.

Returns

Status, maybe more than one status, refer to "STCU_InterruptFlagType"

LOCAL_INLINE uint32_t STCU_HWA_GetRamInitDone (void)

Get Ram Initial Done flag.

Returns

All Done flag, maybe more than one flag, refer to "STCU_InitRamDoneType"

LOCAL_INLINE uint32_t STCU_HWA_GetRamInitStatus (void)

Get Ram Initial status.

Returns

All Done flag, maybe more than one flag, refer to "STCU_HardwareInitRamStatusType"

LOCAL_INLINE uint32_t STCU_HWA_GetSelfTestStatus (void)

Get Self Test Status.

Returns

Status, maybe more than one status, refer to "STCU_SelfTestStatusType"

LOCAL_INLINE void STCU_HWA_MbistSelect (uint32_t *u32MbistSel*)

Mbist select for self-test.

Parameters

<i>u32MbistSel</i>	maybe more than one
--------------------	---------------------

LOCAL_INLINE void STCU_HWA_SetExpectedMisr (uint32_t *u32Misr*)

Set STCU LBIST expected misr value.

LOCAL_INLINE void STCU_HWA_SetInterrupt (uint8_t *bIntEn*, uint8_t *bSequenceErrorIntEn*, uint8_t *bSizeErrorIntEn*)

Set STCU Interrupt.

Parameters

<i>bIntEn</i>	Normal interrupt enable
<i>bSequenceErrorIntEn</i>	Sequence error interrupt enable
<i>bSizeErrorIntEn</i>	Size error interrupt enable

LOCAL_INLINE void STCU_HWA_SetLBISTPatternAmount (uint32_t *u32Amount*)

Set the LBIST pattern amount control value N.

Parameters

<i>u32Amount</i>	LBIST pattern amount control value N. (pattern amount = N * 256)
------------------	--

LOCAL_INLINE void STCU_HWA_SetMBISTSWAlg (bool *bFullTest*, bool *bSRAMInit*)

Set the MBIST algorithm for SW trigger self-test.

Parameters

<i>bFullTest</i>	Enable Full test for MBIST.
<i>bSRAMInit</i>	Enable SRAM initialization at the end of software trigger self-test.

LOCAL_INLINE void STCU_HWA_SetRamInitType (uint32_t *u32InitRamType*)

STCU RAM Init Mode set, can select more than one, for example: STCU_INIT_RAM_TYPE_SRAM0 | STCU_INIT_RAM_TYPE_SRAM1.

Parameters

<i>u32InitRamType</i>	Init Mode
-----------------------	-----------

LOCAL_INLINE void STCU_HWA_SetSafetyKey (uint32_t *u32Key*)

Set safety key.

Parameters

<i>u32Key</i>	Safety Key.
---------------	-------------

LOCAL_INLINE void STCU_HWA_SetSelfTestCTRL (uint32_t *u32Reg*)

Set self test ctrl register.

Parameters

<i>u32Reg</i>	Register value.
---------------	-----------------

LOCAL_INLINE void STCU_HWA_StartRamInit (uint8_t *eInitRamMode*, uint8_t *bLock*)

STCU Ram Initial Start.

Parameters

<i>eInitRamMode</i>	Initial mode
<i>bEnable</i>	Enable Initial
<i>bLock</i>	Lock after initial

LOCAL_INLINE void STCU_HWA_SwTriggerA (void)

Enable A self-test software trigger.

LOCAL_INLINE void STCU_HWA_SwTriggerB (void)

Enable B self-test software trigger.

HwA_tmu.h File Reference

Hardware access layer for TMU.

```
#include "device_header.h"
```

Include dependency graph for HwA_tmu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum TMU_LockType { TMU_TF_TV_LOCK = 0U, TMU_TF_TV_UNLOCK = 1U }
```

Select the TF_CTRL and TV_CTRL register is lock or not.

```
enum TMU_HysteresisType { TMU_TF_HYSOFF_ON = 0U, TMU_TF_HYSOFF_OFF = 1U }
```

Select the Flag-based temperature sensor hysteresis control.

```
enum TMU_bypassType { TMU_TF_FILT_BYP_BYPASSED = 0U, TMU_TF_FILT_BYP_ENABLED = 1U }
```

Select the Flag-based temperature sensor filter control.

Functions

```
LOCAL_INLINE TMU_LockType TMU_HWA_GetTemperatureLockStatus (const TMU_Type *const pTmu)
```

Get the status of whether the temperature sensor register is locked.

```
LOCAL_INLINE void TMU_HWA_SetTemperatureLockStatus (TMU_Type *const pTmu, TMU_LockType  
eLockStatus)
```

Set the status of whether the temperature sensor register is locked or not.

```
LOCAL_INLINE bool TMU_HWA_GetFlagTemperature150InterruptFlag (const TMU_Type *const pTmu)
```

Get the Flag-based temperature sensor over 150 Celsius interrupt flag.

```
LOCAL_INLINE void TMU_HWA_SetFlagTemperature150InterruptFlag (TMU_Type *const pTmu, bool  
bEnable)
```

Set the Flag-based temperature sensor over 150 Celsius interrupt flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperature125InterruptFlag** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor over 125 Celsius interrupt flag.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperature125InterruptFlag** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor over 125 Celsius interrupt flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureReadyInterruptFlag** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor ready interrupt flag.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureReadyInterruptFlag** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor over ready interrupt flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureHysteresisStatus** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor hysteresis control status.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureHysteresisStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor hysteresis control status.

LOCAL_INLINE uint8_t **TMU_HWA_GetFlagTemperatureCounter** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor startup counter.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureCounter** (TMU_Type *const pTmu, uint8_t u8Counter)

Set the Flag-based temperature sensor startup counter.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureFilterBypassStatus** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor filter bypass control status.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureFilterBypassStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor filter bypass control status.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureEnableStatus** (const TMU_Type *const pTmu)

Get the Flag-based temperature sensor enable status.

LOCAL_INLINE void **TMU_HWA_SetFlagTemperatureEnableStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Flag-based temperature sensor enable status.

LOCAL_INLINE bool **TMU_HWA_Get150Status** (TMU_Type *const pTmu)

Check whether the temperature is over 150 Celsius.

LOCAL_INLINE bool **TMU_HWA_Get125Status** (TMU_Type *const pTmu)

Check whether the temperature is over 125 Celsius.

LOCAL_INLINE bool **TMU_HWA_Get150Flag** (TMU_Type *const pTmu)

Check whether the flag for temperature over 150 Celsius is set.

LOCAL_INLINE void **TMU_HWA_Clear150Flag** (TMU_Type *const pTmu)

Clear the temperature over 150 Celsius flag.

LOCAL_INLINE bool **TMU_HWA_Get125Flag** (TMU_Type *const pTmu)

Check whether the flag for temperature over 125 Celsius is set.

LOCAL_INLINE void **TMU_HWA_Clear125Flag** (TMU_Type *const pTmu)

Clear the temperature over 125 Celsius flag.

LOCAL_INLINE bool **TMU_HWA_GetFlagTemperatureReady** (TMU_Type *const pTmu)

Check whether the Flag-based temperature sensor is ready.

LOCAL_INLINE void **TMU_HWA_ClearFlagTemperatureReady** (TMU_Type *const pTmu)

Clear the Flag-based temperature sensor ready flag.

LOCAL_INLINE bool **TMU_HWA_GetVoltageTemperatureReadyInterruptFlag** (const TMU_Type *const pTmu)

Get the Voltage-based temperature sensor ready interrupt flag.

LOCAL_INLINE void **TMU_HWA_SetVoltageTemperatureReadyInterruptFlag** (TMU_Type *const pTmu, bool bEnable)

Set the Voltage-based temperature sensor over ready interrupt flag.

LOCAL_INLINE uint8_t **TMU_HWA_GetVoltageTemperatureCounter** (const TMU_Type *const pTmu)

Get the Voltage-based temperature sensor startup counter.

LOCAL_INLINE void **TMU_HWA_SetVoltageTemperatureCounter** (TMU_Type *const pTmu, uint8_t u8Counter)

Set the Voltage-based temperature sensor startup counter.

LOCAL_INLINE bool **TMU_HWA_GetVoltageTemperatureEnableStatus** (const TMU_Type *const pTmu)

Get the Voltage-based temperature sensor enable status.

LOCAL_INLINE void **TMU_HWA_SetVoltageTemperatureEnableStatus** (TMU_Type *const pTmu, bool bEnable)

Set the Voltage-based temperature sensor enable status.

LOCAL_INLINE bool **TMU_HWA_GetVoltageTemperatureReady** (TMU_Type *const pTmu)

Check whether the Voltage-based temperature sensor is ready.

LOCAL_INLINE void **TMU_HWA_ClearVoltageTemperatureReady** (TMU_Type *const pTmu)

Clear the Voltage-based temperature sensor ready flag.

LOCAL_INLINE uint32_t **TMU_HWA_GetFlagTempCtrl** (const TMU_Type *const pTmu)

Get the TMU_TF_CTRL config.

LOCAL_INLINE void **TMU_HWA_SetFlagTempCtrl** (TMU_Type *const pTmu, uint32_t u32Config)

Set the TMU_TF_CTRL config.

LOCAL_INLINE uint32_t **TMU_HWA_GetVoltageTempCtrl** (const TMU_Type *const pTmu)

Get the TMU_TV_CTRL config.

LOCAL_INLINE void **TMU_HWA_SetVoltageTempCtrl** (TMU_Type *const pTmu, uint32_t u32Config)

Set the TMU_TV_CTRL config.

Detailed Description

Hardware access layer for TMU.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_tpue.h File Reference

FC7xxx TPUE hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_tpue.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

FC7xxx TPUE hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-1-12

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_tpuh.h File Reference

FC7xxx TPUH hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_tpuh.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

FC7xxx TPUH hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-1-12

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

HwA_trgsel.h File Reference

Hardware access layer for TrgSel.

```
#include "device_header.h"
```

Include dependency graph for HwA_trgsel.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define TRGSEL_REGSIZE 4U
```

Functions

```
LOCAL_INLINE uint32_t TRGSEL_HWA_GetTargetTriggerSource (const TRGSEL_Type *const pTrgsel,  
uint32_t u32TargetIndex)
```

Get the trigger source of the selected trigger target.

```
LOCAL_INLINE void TRGSEL_HWA_SetTargetTriggerSource (TRGSEL_Type *const pTrgsel, uint32_t  
u32TargetIndex, uint32_t u32SourceIndex)
```

Set the trigger source of the selected trigger target.

```
LOCAL_INLINE bool TRGSEL_HWA_GetTargetLockStatus (const TRGSEL_Type *const pTrgsel, uint32_t  
u32TargetIndex)
```

Get whether the trigger source of the selected target is locked.

```
LOCAL_INLINE void TRGSEL_HWA_LockTargetTriggerSource (TRGSEL_Type *const pTrgsel, uint32_t  
u32TargetIndex)
```

Lock the trigger source of the selected target.

Detailed Description

Hardware access layer for TrgSel.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

Macro Definition Documentation

```
#define TRGSEL_REGSIZE 4U
```

HwA_tstmp.h File Reference

TSTMP hardware access layer.

#include "device_header.h"

Include dependency graph for HwA_tstmp.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Enumerations

```
enum TSTMP_ModChannelType { TSTMP_MODChn0 = 0U, TSTMP_MODChn1, TSTMP_MODChn2,  
TSTMP_MODChn3 }
```

Tstmp modulate number.

Functions

```
LOCAL_INLINE uint64_t TSTMP_HWA_ReadTstmpValue (TSTMP_Type *pTstmp)
```

Read TSTMP value.

```
LOCAL_INLINE uint32_t TSTMP_HWA_ReadTstmpInterruptEnable (TSTMP_Type *pTstmp)
```

Read TSTMP interrupt enable bits.

```
LOCAL_INLINE uint32_t TSTMP_HWA_ReadModMatchFlag (TSTMP_Type *pTstmp)
```

Read TSTMP all MOD match flag.

```
LOCAL_INLINE void TSTMP_HWA_ClearModMatchFlag (TSTMP_Type *pTstmp, TSTMP_ModChannelType  
eMod)
```

Clear TSTMP MOD match flag.

```
LOCAL_INLINE void TSTMP_HWA_SetModMatchValue (TSTMP_Type *pTstmp, TSTMP_ModChannelType  
eMod, uint32_t u32ModValue)
```

Set MOD match value.

LOCAL_INLINE void **TSTMP_HWA_EnableModMatchInterrupt** (TSTMP_Type *pTstmp,
TSTMP_ModChannelType eMod)

Enable TSTMP MOD(n) match interrupt.

LOCAL_INLINE void **TSTMP_HWA_DisableModMatchInterrupt** (TSTMP_Type *pTstmp,
TSTMP_ModChannelType eMod)

Disable TSTMP MOD(n) match interrupt.

Detailed Description

TSTMP hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_wdog.h File Reference

Wdog hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_wdog.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Functions

LOCAL_INLINE void **WDOG_HWA_SetCs** (WDOG_Type *pWdog, uint32_t u32Cs)

Set CS register value, for Wdog working mode configuration.

LOCAL_INLINE uint32_t **WDOG_HWA_GetCs** (WDOG_Type *pWdog)

Get CS register value, for WDOG working mode configuration.

LOCAL_INLINE void **WDOG_HWA_ClearInterruptFlag** (WDOG_Type *pWdog)

Clear Wdog interrupt flag.

LOCAL_INLINE bool **WDOG_HWA_GetUnlockStatus** (WDOG_Type *pWdog)

Get WDOG unlock status, if locked, register can't be written.

LOCAL_INLINE void **WDOG_HWA_SetCounter** (WDOG_Type *pWdog, uint32_t u32Counter)

Set COUNTER register value. for Wdog unlock and refresh usage.

LOCAL_INLINE void **WDOG_HWA_SetTimeout** (WDOG_Type *pWdog, uint16_t u16Timeout)

Set TIMEOUT register value. for WDOG timeout value.

LOCAL_INLINE void **WDOG_HWA_SetWindow** (WDOG_Type *pWdog, uint16_t u16Window)

Set WINDOW register value. for windowed WDOG low threshold value.

Detailed Description

Wdog hardware access layer.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

HwA_wku.h File Reference

WKU hardware access layer.

```
#include "device_header.h"
```

Include dependency graph for HwA_wku.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Macros

```
#define WKU_MWR0 0x0000100
```

```
#define WKU_MWR1 0x0010000
```

```
#define WKU_MWR2 0x1000000
```

Enumerations

```
enum WKU_WakeupInputType { WKU_INPUT_MAX = 0xFFFFFU }
```

Wku input.

Functions

```
LOCAL_INLINE void WKU_HWA_EnableWakeupSource (const WKU_WakeupInputType eWakeup)
```

Enable wakeup source.

```
LOCAL_INLINE void WKU_HWA_DisableWakeupSource (const WKU_WakeupInputType eWakeup)
```

Disable wakeup source.

```
LOCAL_INLINE uint32_t WKU_HWA_ReadWakeupSource (void)
```

Read WKU wakeup source.

Detailed Description

WKU hardware access layer.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_adc.h File Reference

ADC driver type definition and API.

#include "HwA_adc.h"

Include dependency graph for module_driver_adc.h:

IMAGE

Classes

struct **_ADC_HandleType**

The structure of the ADC processing handle.

struct **ADC_InitType**

Defines the converter configuration.

struct **ADC_ChannelCfgType**

The configuration option for the ADC channel.

struct **ADC_CompareType**

Defines the hardware compare configuration.

Macros

ADC API Service IDs#define **ADC_INIT_STRUCTURE_ID** 0U

#define **ADC_INIT_ID** 1U

#define **ADC_DEINIT_ID** 2U

#define **ADC_INIT_CHANNEL_ID** 3U

#define **ADC_INIT_COMPARE_ID** 4U

#define **ADC_INIT_INTERRUPT_ID** 5U

#define **ADC_INIT_DMA_CHANNEL_ID** 6U

#define **ADC_SET_SW_PRETRIGGER_ID** 7U

#define **ADC_ENABLE_ID** 8U

#define **ADC_DISABLE_ID** 9U

#define **ADC_START_ID** 10U

#define **ADC_STOP_ID** 11U

#define **ADC_RESET_ID** 12U

#define **ADC_GET_TRIG_ERR_STATUS_ID** 13U

#define **ADC_CLEAR_TRIG_ERR_ID** 14U

#define **ADC_CLEAR_TRIG_ERRS_ID** 15U

#define **ADC_GET_TRIG_LATCH_STATUS_ID** 16U

#define **ADC_CLEAR_LATCH_TRIGGER_ID** 17U

ADC Dev Error Code *Error Code of calling ADC apis*

#define **ADC_E_PARAM_INSTANCE** 0x01U

#define **ADC_E_PARAM_CHANNEL** 0x02U

#define **ADC_E_PARAM_POINTER** 0x03U

#define **ADC_E_PARAM_COUNT** 0x04U

#define **ADC_E_PARAM_INVALID** 0x05U

#define **ADC_E_PARAM_PHASE_SHIFT** 0x06U

ADC0 Internal Channels *Available internal ADC channels for ADC Instance 0*

#define **ADC0_CHANNEL_VBG_BUFFER** **ADC_CHANNEL_INTERNAL_0**

#define **ADC0_CHANNEL_V25** **ADC_CHANNEL_INTERNAL_1**

#define **ADC0_CHANNEL_V11** **ADC_CHANNEL_INTERNAL_2**

#define **ADC0_CHANNEL_CMP0_DAC** **ADC_CHANNEL_INTERNAL_4**

#define **ADC0_CHANNEL_TEMPSENSOR_OUT** **ADC_CHANNEL_INTERNAL_5**

ADC1 Internal channels *Available internal ADC channels for ADC Instance 1*

#define **ADC1_CHANNEL_V11_PD0** **ADC_CHANNEL_INTERNAL_0**

#define **ADC1_CHANNEL_VREFH** **ADC_CHANNEL_INTERNAL_1**

#define **ADC1_CHANNEL_VDDA0** **ADC_CHANNEL_INTERNAL_2**

#define **ADC1_CHANNEL_V15** **ADC_CHANNEL_INTERNAL_3**

#define **ADC1_CHANNEL_CMP_DAC0** **ADC_CHANNEL_INTERNAL_4**

ADC2 Internal channels *Available internal ADC channels for ADC instance 2*

#define **ADC2_CHANNEL_V11_PD0** **ADC_CHANNEL_INTERNAL_0**

#define **ADC2_CHANNEL_VREFH0** **ADC_CHANNEL_INTERNAL_1**

#define **ADC2_CHANNEL_VDDA0** **ADC_CHANNEL_INTERNAL_2**

#define **ADC2_CHANNEL_CMP_DAC0** **ADC_CHANNEL_INTERNAL_4**

#define **ADC2_CHANNEL_TEMPSENSOR_OUT** **ADC_CHANNEL_INTERNAL_5**

#define **ADC4_CHANNEL_V11** **ADC_CHANNEL_INTERNAL_0**

#define **ADC4_CHANNEL_VREFH1** **ADC_CHANNEL_INTERNAL_1**

#define **ADC4_CHANNEL_VDDA1** **ADC_CHANNEL_INTERNAL_2**

#define **ADC5_CHANNEL_V11** **ADC_CHANNEL_INTERNAL_0**

#define **ADC5_CHANNEL_VREFH1** **ADC_CHANNEL_INTERNAL_1**

#define **ADC5_CHANNEL_VDDA1** **ADC_CHANNEL_INTERNAL_2**

ADC3 Internal channels *Available internal ADC channels for ADC instance 3*

#define **ADC3_CHANNEL_V11** **ADC_CHANNEL_INTERNAL_0**

#define **ADC3_CHANNEL_VREFH1** **ADC_CHANNEL_INTERNAL_1**

#define **ADC3_CHANNEL_VDDA1** **ADC_CHANNEL_INTERNAL_2**

Typedefs

typedef struct **_ADC_HandleType** **ADC_HandleType**

The structure of the ADC processing handle.

Enumerations

enum **ADC_InstanceType** { **ADC_INSTANCE_0** = 0U, **ADC_INSTANCE_1** = 1U, **ADC_INSTANCE_2** = 2U, **ADC_INSTANCE_3** = 3U, **ADC_INSTANCE_4** = 4U, **ADC_INSTANCE_5** = 5U }

The instance index of the ADC peripheral.

enum **ADC_ChannelType** { **ADC_CHANNEL_0** = 0U, **ADC_CHANNEL_1** = 1U, **ADC_CHANNEL_2** = 2U, **ADC_CHANNEL_3** = 3U, **ADC_CHANNEL_4** = 4U, **ADC_CHANNEL_5** = 5U, **ADC_CHANNEL_6** = 6U, **ADC_CHANNEL_7** = 7U, **ADC_CHANNEL_8** = 8U, **ADC_CHANNEL_9** = 9U, **ADC_CHANNEL_10** = 10U, **ADC_CHANNEL_11** = 11U, **ADC_CHANNEL_12** = 12U, **ADC_CHANNEL_13** = 13U, **ADC_CHANNEL_14** = 14U, **ADC_CHANNEL_15** = 15U, **ADC_CHANNEL_16** = 16U, **ADC_CHANNEL_17** = 17U, **ADC_CHANNEL_18** = 18U, **ADC_CHANNEL_19** = 19U, **ADC_CHANNEL_20** = 20U, **ADC_CHANNEL_21** = 21U, **ADC_CHANNEL_22** = 22U, **ADC_CHANNEL_23** = 23U, **ADC_CHANNEL_24** = 24U, **ADC_CHANNEL_25** = 25U, **ADC_CHANNEL_26** = 26U, **ADC_CHANNEL_27** = 27U, **ADC_CHANNEL_28** = 28U, **ADC_CHANNEL_29** = 29U, **ADC_CHANNEL_30** = 30U, **ADC_CHANNEL_31** = 31U, **ADC_CHANNEL_INTERNAL_0** = 32U, **ADC_CHANNEL_INTERNAL_1** = 33U, **ADC_CHANNEL_INTERNAL_2** = 34U, **ADC_CHANNEL_INTERNAL_3** = 35U, **ADC_CHANNEL_INTERNAL_4** = 36U, **ADC_CHANNEL_INTERNAL_5** = 37U }

The channel selected for ADC conversion.

enum **ADC_DifferentialChannelType** { **ADC_CHANNEL_0_4** = 0U, **ADC_CHANNEL_1_5** = 1U, **ADC_CHANNEL_2_6** = 2U, **ADC_CHANNEL_3_7** = 3U, **ADC_CHANNEL_TEMPSENSOR** = 37U }

The channel selected for ADC conversion in differential mode.

enum **ADC_SampleTimeOptionType** { **ADC_SAMPLE_TIME_OPTION_0** = 0U, **ADC_SAMPLE_TIME_OPTION_1** = 1U, **ADC_SAMPLE_TIME_OPTION_2** = 2U, **ADC_SAMPLE_TIME_OPTION_3** = 3U }

The ADC sample time option for selection.

enum **ADC_StatusType** { **ADC_STATUS_SUCCESS** = 0x0U, **ADC_STATUS_ERROR** = 0x1U, **ADC_STATUS_TIMEOUT** = 0x2U }

ADC operation return values.

Functions

Initial Structurevoid **ADC_InitStructure** (**ADC_InitType** *const pInitCfg)

*Provide the default values of **ADC_InitType**.*

Initialization Functions
void ADC_Init (ADC_HandleType *pAdcHandle, const ADC_InitType *const pInitCfg)

Initialize the ADC instance.

void ADC_DeInit (ADC_HandleType *pAdcHandle)

De-initialize the ADC instance.

void ADC_InitChannel (ADC_HandleType *pAdcHandle, const ADC_ChannelCfgType aChannels[], const uint8_t u8ChnCnt)

Configure the ADC sample channels.

void ADC_InitCompare (const ADC_HandleType *pAdcHandle, const ADC_CompareType *const pCmpCfg)

Configure the hardware compare feature of ADC.

Operation functions
ADC_StatusType ADC_Enable (const ADC_HandleType *pAdcHandle)

Enable the ADC instance.

ADC_StatusType ADC_Disable (ADC_HandleType *pAdcHandle)

Disable the ADC instance.

void ADC_Start (const ADC_HandleType *pAdcHandle)

Start the ADC conversion.

ADC_StatusType ADC_Stop (const ADC_HandleType *pAdcHandle)

Stop the ADC conversion.

void ADC_Reset (const ADC_HandleType *pAdcHandle)

Reset the ADC hardware.

void ADCn_IRQHandler (ADC_HandleType *pAdcHandle)

The internal interrupt handler function for ADC instances.

Detailed Description

ADC driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_aontimer.h File Reference

aontimer driver type definition and API

```
#include "HwA_aontimer.h"
```

```
#include <stddef.h>
```

Include dependency graph for module_driver_aontimer.h:

IMAGE

Classes

struct **_AONTIMER_HandleType**

The structure of the AONTIMER processing handle.

struct **AONTIMER_InitType**

Aontimer Initialization struct type.

Macros

AONTIMER API Service IDs#define **AONTIMER_INIT_ID** 0U

#define **AONTIMER_DEINIT_ID** 1U

#define **AONTIMER_START_ID** 2U

#define **AONTIMER_STOP_ID** 3U

#define **AONTIMER_UPDATE_COUNT_ID** 4U

#define **AONTIMER_INTE_PROC_ID** 5U

AONTIMER Dev Error Code*Error Code of calling AONTIMER apis*

#define **AONTIMER_E_PARAM_INSTANCE** 0x01U

#define **AONTIMER_E_PARAM_CLOCK** 0x02U

#define **AONTIMER_E_PARAM_POINTER** 0x03U

Typedefs

typedef struct **_AONTIMER_HandleType** **AONTIMER_HandleType**

The structure of the AONTIMER processing handle.

Enumerations

enum **AONTIMER_InstanceType** { **AONTIMER_INSTANCE_0** = 0U }

The instance index of the FTU peripheral.

enum **AONTIMER_ModeType** { **AONTIMER_COUNTER_MODE** = 0, **AONTIMER_PULSE_MODE** }

Aontimer mode.

Functions

void **AONTIMER_Init** (**AONTIMER_HandleType** *pAontimerHandle, const **AONTIMER_InitType** *const pInitStruct)

Initialize aontimer instance.

void **AONTIMER_Deinit** (**AONTIMER_HandleType** *pAontimerHandle)

De-initialize aontimer instance.

void **AONTIMER_StartTimer** (**AONTIMER_HandleType** *pAontimerHandle)

Start Aontimer.

void **AONTIMER_StopTimer** (**AONTIMER_HandleType** *pAontimerHandle)

Stop Aontimer.

void **AONTIMER_UpdateCounterValue** (**AONTIMER_HandleType** *pAontimerHandle, const uint16_t u16StartValue)

Update value of aontimer counter.

void **Aontimer_CommonProcessInterrupt** (**AONTIMER_HandleType** *pAontimerHandle)

Aontimer common interrupt handle function.

Detailed Description

aontimer driver type definition and API

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_cmp.h File Reference

CMP driver type definition and API.

#include "HwA_cmp.h"

Include dependency graph for module_driver_cmp.h:

IMAGE

Classes

struct **CMP_DacType**

The instance index of the CMP Dac parts.

struct **CMP_MuxType**

The instance index of the CMP Mux parts.

struct **CMP_ComparatorType**

The instance index of the CMP Comparator parts.

struct **CMP_InterruptType**

Defines the comparator interrupt configuration.

struct **_CMP_HandleType**

The structure of the CMP processing handle.

struct **CMP_CSChannelCfgType**

Defines the comparator channel selected in channel scan mode.

struct **CMP_ChannelScanType**

Defines the comparator channel scan configuration.

struct **CMP_InitType**

Defines the comparator configuration.

Macros

CMP API Service IDs#define **CMP_INIT_ID** 0U

#define **CMP_DEINIT_ID** 1U

#define **CMP_GETDEFAULTINITCFG_ID** 2U

#define **CMP_ENABLE_ID** 3U

#define **CMP_DISABLE_ID** 4U

#define **CMP_GETCMPOUT_ID** 5U

#define **CMP_SETDACDATA_ID** 6U


```

#define CMP_GETOUTFLAGSTATUS_ID 7U
#define CMP_GETCSFLAGSTATUS_ID 8U
#define CMP_ENABLECSCHANNEL_ID 9U
#define CMP_DISABLECSCHANNEL_ID 10U
#define CMP_GETCSCHANNELOUT_ID 11U
#define CMP_GETCMPCSACTIVE_ID 12U
#define CMP_ENTERLPMODE_ID 13U
#define CMP_EXITLPMODE_ID 14U

```

CMP Dev Error Code *Error Code of calling CMP apis*

```

#define CMP_E_PARAM_INSTANCE 0x01U
#define CMP_E_PARAM_CHANNEL 0x02U
#define CMP_E_PARAM_POINTER 0x03U

```

Typedefs

```
typedef void(* CMP_CompleteIntCallback) (void)
```

The CMP complete interrupt callback function prototype.

```
typedef struct _CMP_HandleType CMP_HandleType
```

The structure of the CMP processing handle.

Functions

```
void CMP_GetDefaultInitCfg (CMP_InitType *pInitCfg)
```

set CMP module configurations instructure to defalut value

```
void CMP_Enable (CMP_HandleType *pCmpHandle)
```

set CMP module enable

```
void CMP_Disable (CMP_HandleType *pCmpHandle)
```

set CMP module disable

```
void CMP_Init (CMP_HandleType *pCmpHandle, const CMP_InitType *const pInitCfg)
```

CMP init.

```
void CMP_Deinit (CMP_HandleType *pCmpHandle)
```

CMP Deinit.

void **CMP_Enter_LP_Mode** (CMP_HandleType *pCmpHandle)

CMP Deinit.

uint32_t **CMP_GetCmpOut** (CMP_HandleType *pCmpHandle)

get CMP filtered output

void **CMP_SetDacData** (CMP_HandleType *pCmpHandle, uint8_t u8Data)

set CMP Dac output

CMP_OutStatus **CMP_GetOutFlagStatus** (CMP_HandleType *pCmpHandle)

get CMP interrupter flag status

void **CMPn_IRQHandler** (CMP_HandleType *pCmpHandle)

CMP interrupt process function.

void **CMP_CSEnable** (CMP_HandleType *pCmpHandle)

set CMP module channel scan mode enable

void **CMP_CSDisable** (CMP_HandleType *pCmpHandle)

set CMP module channel scan mode disable

bool **CMP_GetCSFlagStatus** (CMP_HandleType *pCmpHandle)

get CMP channel scan flag status

void **CMP_EnableCSChannel** (CMP_HandleType *pCmpHandle, const **CMP_MuxSelType** eChannel, const bool bPreSetState)

enable CMP channel scan channels

void **CMP_DisableCSChannel** (CMP_HandleType *pCmpHandle, const **CMP_MuxSelType** eChannel)

disable CMP channel scan channels

bool **CMP_GetCSChannlOut** (CMP_HandleType *pCmpHandle, const **CMP_MuxSelType** e_Channel)
get CMP channel scan channel out

bool **CMP_GetCmpCSActive** (CMP_HandleType *pCmpHandle)
get CMP channel scan status

Detailed Description

CMP driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_cmu.h File Reference

CMU Module driver type definition and API.

#include "HwA_cmu.h"

Include dependency graph for module_driver_cmu.h:

IMAGE

Classes

struct **_CMU_HandleType**

The structure of the CMU processing handle.

struct **CMU_CfgType**

CMU configure structure.

Macros

CMU API Service IDs

```
#define CMU_INIT_ID 0x00U
#define CMU_GET_DEFAULT_CONFIG_ID 0x01U
#define CMU_ENABLE_ID 0x02U
#define CMU_DISABLE_ID 0x03U
#define CMU_ENABLE_IRQ_ID 0x04U
#define CMU_DISABLE_IRQ_ID 0x05U
#define CMU_COMMONIRQHANDLER_ID 0x06U
#define CMU_ENABLE_LOWPOWER_MODE_ID 0x07U
#define CMU_CAL_MONITOR_WINDOW_ID 0x08U
#define CMU_GET_CURRENTCOUNT_ID 0x09U
#define CMU_GET_MINCOUNT_ID 0x0AU
#define CMU_GET_MAXCOUNT_ID 0x0BU
#define CMU_GET_INQTYPE_ID 0x0CU
```

CMU Dev Error Code *Error Code of calling CMU apis*

```
#define CMU_E_PARAM_NULLPTR 0x01U
#define CMU_E_PARAM_INSTANCE 0x02U
```

Typedefs

typedef struct **_CMU_HandleType** **CMU_HandleType**

The structure of the CMU processing handle.

Enumerations

enum **CMU_InstanceType** { **CMU_INSTANCE_0** = 0U, **CMU_INSTANCE_1** = 1U, **CMU_INSTANCE_2** = 2U, **CMU_INSTANCE_3** = 3U, **CMU_INSTANCE_4** = 4U }

CMU Instance type.

enum **CMU_StatusType** { **CMU_STATUS_SUCCESS** = 0x0U, **CMU_STATUS_FAIL** = 0x1U }

CMU operation return values.

enum **CMU_LowpowerModeType** { **CMU_STOP_MODE** = 0U, **CMU_STANDBY_MODE** = 1U }

CMU Lowpower Mode type.

enum **CMU_Cmu4ClkSrcType** { **CMU_CMU4_REF_CLK_NONE** = 0U, **CMU_CMU4_REF_CLK_FOSC** = 1U, **CMU_CMU4_REF_CLK_SIRC** = 2U }

CMU4 Reference clock type.

enum **CMU_InterruptType** { **CMU_INTERRUPT_NONE** = 0U, **CMU_INTERRUPT_LOC** = 1U, **CMU_INTERRUPT_MIS** = 2U }

CMU interrupt mode type.

enum **CMU_RefClockDivType** { **CMU_REFCLK_NODIV** = 0U, **CMU_REFCLK_DIV_BY2** = 1U, **CMU_REFCLK_DIV_BY4** = 2U, **CMU_REFCLK_DIV_BY8** = 3U, **CMU_REFCLK_DIV_BY16** = 4U, **CMU_REFCLK_DIV_BY32** = 5U, **CMU_REFCLK_DIV_BY64** = 6U, **CMU_REFCLK_DIV_BY128** = 7U }

CMU Reference clock div type.

Functions

void **CMU_GetDefaultConfig** (**CMU_CfgType** *const pInitCfg)

Initializes a CMU configuration structure with default values.

void **CMU_CaculateMonitorWindowsValue** (**CMU_CfgType** *const pInitCfg)

Calculates monitor window values for the CMU based on the given configuration.

CMU_StatusType **CMU_Init** (**CMU_HandleType** *const pHandle, const **CMU_CfgType** *const pInitCfg)

Initializes the CMU module with the provided configuration.

void **CMU_Enable** (**CMU_HandleType** *const pHandle)

Enables the Clock Monitor Unit (CMU) for the specified instance.

void **CMU_Disable** (**CMU_HandleType** *const pHandle)

Disables the Clock Monitor Unit (CMU) for the specified instance.

void **CMU_EnableInterrupt** (CMU_HandleType *const pHandle)

Enables the Clock Monitor Unit (CMU) interrupt for the specified instance.

void **CMU_DisableInterrupt** (CMU_HandleType *const pHandle)

Disables the Clock Monitor Unit (CMU) interrupt for the specified instance.

void **CMU_LowPowerModeEnable** (CMU_HandleType *const pHandle, CMU_LowpowerModeType eMode, bool bModeEnable, bool bRestartEnable)

Enables or disables low power mode on the CMU module.

uint32_t **CMU_GetCount** (CMU_HandleType *const pHandle)

Retrieves the current count from the CMU module.

uint32_t **CMU_GetMinCount** (CMU_HandleType *const pHandle)

Retrieves the minimum count value from the CMU module.

uint32_t **CMU_GetMaxCount** (CMU_HandleType *const pHandle)

Retrieves the maximum count value from the CMU module.

void **CMU_SetCmu4RefSrc** (CMU_Cmu4ClkSrcType eSrc)

Sets the reference clock source for CMU4.

CMU_InterruptType **CMU_GetInterruptType** (CMU_HandleType *const pHandle)

Retrieves the interrupt type of the CMU (Clock Monitor Unit) module.

void **CMU_CommonIRQHandler** (CMU_HandleType *const pHandle)

Common Clock Monitor Unit (CMU) Interrupt Service Routine (ISR)

Detailed Description

CMU Module driver type definition and API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_cordic.h File Reference

Cordic driver type definition and API.

#include "HwA_cordic.h"

Include dependency graph for module_driver_cordic.h:

IMAGE

Classes

struct **CORDIC_SinCos_Type**

Circular sine and cosine type.

struct **CORDIC_SinhCosh_Type**

Circular hyperbolic sine and cosine type.

Macros

CORDIC API Service IDs#define **CORDIC_SIN_COS_ID** 0x00

#define **CORDIC_RADICAL_ADD_ID** 0x01

#define **CORDIC_ARCTAN_F_ID** 0x02

#define **CORDIC_ARCTAN_ID** 0x03

#define **CORDIC_SINH_COSH_ID** 0x04

#define **CORDIC_RADICAL_SUB_ID** 0x05

#define **CORDIC_ARCTANH_ID** 0x06

#define **CORDIC_EXTENDED_LN_ID** 0x07

#define **CORDIC_EXTENDED_SQRT_ID** 0x08

CORDIC Error Codes#define **CORDIC_E_PARAM_INSTANCE** 0x01U

Typedefs

typedef float **CORDIC_Radical_Type**

Circular radical type.

typedef float **CORDIC_Arctan_Type**

Circular arctangent type.

typedef float **CORDIC_Arctanh_Type**

Circular hyperbolic arctangent type.

typedef float **CORDIC_Ln_Type**

The arctan type.

typedef float **CORDIC_Sqrt_Type**

The arctanh type.

Enumerations

enum **CORDIC_InstanceType** { **CORDIC_INSTANCE_0** = 0U }

CORDIC instance type.

Functions

CORDIC_SinCos_Type **Cordic_Sin_Cos** (**CORDIC_InstanceType** eInstance, const float radian)

Computes sine and cosine values using the circular CORDIC algorithm.

CORDIC_Radical_Type **Cordic_Radical_Add** (**CORDIC_InstanceType** eInstance, const float x, const float y)

Computes the angle given the x and y components using the circular CORDIC algorithm.

CORDIC_Arctan_Type **Cordic_Arctan_F** (**CORDIC_InstanceType** eInstance, const float y)

Computes the arctangent of a value using the circular CORDIC algorithm.

CORDIC_Arctan_Type **Cordic_Arctan** (**CORDIC_InstanceType** eInstance, const int32_t x, const int32_t y)

Computes the arctangent of a point given its x and y coordinates using the circular CORDIC algorithm.

CORDIC_SinhCosh_Type **Cordic_Sinh_Cosh** (**CORDIC_InstanceType** eInstance, const float z)

Computes the hyperbolic sine and cosine of a value using the circular CORDIC algorithm.

CORDIC_Radical_Type **Cordic_Radical_Sub** (**CORDIC_InstanceType** eInstance, const float x, const float y)

Computes the circular radical subtraction using the CORDIC algorithm.

CORDIC_Arctanh_Type **Cordic_Arctanh** (**CORDIC_InstanceType** eInstance, const float y)

Computes the hyperbolic arctangent using the CORDIC algorithm.

CORDIC_Ln_Type Cordic_Extended_LN (CORDIC_InstanceType eInstance, const float y)
The function to calculate ln.

CORDIC_Sqrt_Type Cordic_Extended_Sqrt (CORDIC_InstanceType eInstance, const float y)
The function to calculate sqrt.

Detailed Description

Cordic driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_cpm.h File Reference

CPM driver type definition and API.

```
#include "device_header.h"
```

```
#include "HwA_cpm.h"
```

Include dependency graph for module_driver_cpm.h:

IMAGE

Classes

struct **_CPM_HandleType**

The structure of the CPM processing handle.

struct **CPM_InterruptType**

Cpm interrupt configuration type.

Typedefs

typedef struct **_CPM_HandleType** **CPM_HandleType**

The structure of the CPM processing handle.

Enumerations

enum **CPM_RetType** { **CPM_STATUS_SUCCESS** = 0U, **CPM_STATUS_PARAM_INVALID** = 1U }

Cpm return type.

enum **FPU_IntType** { **CPM_FPU_FIO** = 1U, **CPM_FPU_FDZ** = 2U, **CPM_FPU_FUF** = 4U, **CPM_FPU_FOF** = 8U, **CPM_FPU_FID** = 16U, **CPM_FPU_FIX** = 32U }

FPU INTERRUPT type.

enum **CPM_MISCRCoreActiveType** { **CPM_MISCR_CPUCORE0** = 1U, **CPM_MISCR_CPUCORE1** = 2U, **CPM_MISCR_CPUCORE2** = 4U, **CPM_MISCR_HSM** = 8U }

enum **CPM_InstanceType** { **CPM_INSTANCE_0** = 0U }

The instance index of the CPM peripheral.

Functions

void **CPM_FpuIntMode** (**CPM_HandleType** *pCpmHandle, **FPU_IntType** eIntSrc, bool bEnable)

Configures the CPM module interrupts.

bool **CPM_GetFpuIntStatus** (CPM_HandleType *pCpmHandle, FPU_IntType eIntSrc)

Get CPM Interrupt occurred flag.

uint32_t **CPM_MiscrGetCoreMode** (CPM_HandleType *pCpmHandle)

Get core mode.

uint32_t **CPM_GetFiscr** (CPM_HandleType *pCpmHandle)

Gets the value of the Fiscr register.

CPM_RetType CPM_InitInterrupt (CPM_HandleType *pCpmHandle, const CPM_InterruptType *pIntStruct)

Cpm set interrupt.

void **CPM_CommonProcessInterrupt** (CPM_HandleType *pCpmHandle)

CPM interrupt function.

Detailed Description

CPM driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_crc.h File Reference

CRC driver type definition and API.

```
#include "HwA_crc.h"
```

Include dependency graph for module_driver_crc.h:

IMAGE

Classes

struct **Crc_ConfigType**

Defines the crc configuration.

struct **Crc_HandleType**

The structure of the SENT processing handle.

Macros

```
SSI API Service IDs#define CRC_INIT_ID 1U
#define CRC_DEINIT_ID 2U
#define CRC_GETRST 3U
#define CRC_SETCALCDATA8 4U
#define CRC_SETCALCDATA16 5U
#define CRC_SETCALCDATA32 6U
#define CRC_SETINPUTDATA 7U
```

SSI Dev Error Code

Error Code of calling ADC apis

```
#define CRC_E_PARAM_INSTANCE 0x01U
#define CRC_E_PARAM_POINTER 0x02U
#define CRC_E_UNINIT 0x03U
#define CRC_E_ALREADY_INIT 0x04U
#define CRC_DEFAULT_POLY (0x1021U)
#define CRC_DEFAULT_INTVAL (0U)
enum eCrctype { CRC_NONE = 0, CRC_16_CCITT = 1, CRC_16_CCITT_FALSE = 2, CRC_16_MODBUS = 3,
CRC_16_USB = 4, CRC_32_USER = 5 }
typedef struct Crc_HandleType Crc_HandleType
The structure of the SENT processing handle.
```

void **CRC_Init** (Crc_HandleType *pCrcHandle, const Crc_ConfigType *const pInitCfg)

Initializes the CRC module with the specified configuration.

void **CRC_DeInit** (Crc_HandleType *pCrcHandle)

Deinitializes the CRC module and restores default configuration.

uint32_t **CRC_GetCrcResult** (Crc_HandleType *pCrcHandle)

get CRC result

void **CRC_SetCalcData_U8** (Crc_HandleType *pCrcHandle, uint8_t u8Data)

set CRC DATA register 8bits

void **CRC_SetCalcData_U16** (Crc_HandleType *pCrcHandle, uint16_t u16Data)

set CRC DATA register 16bits

void **CRC_SetCalcData_U32** (Crc_HandleType *pCrcHandle, uint32_t u32Data)

set CRC low lower register

void **CRC_SetSeed** (Crc_HandleType *pCrcHandle, uint32_t u32SeedVal)

set CRC seed value

void **CRC_SetInputData** (Crc_HandleType *pCrcHandle, const uint8_t *pData, uint32_t u32DataSize)

set CRC input data

Detailed Description

CRC driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_csc.h File Reference

csc driver type definition and API

```
#include "HwA_csc.h"
```

Include dependency graph for module_driver_csc.h:

IMAGE

Classes

struct **CSC0_ClkoutType**

Data type for register CSC0_CLKOUT_CTRL, mainly focus on CLKOUT setting.

struct **CSC0_AONCLKSRType**

CSC0_AONCLKSR clock source info.

Macros

```
#define CSC0_DEV_ERROR_REPORT STD_OFF
```

```
#define CSC0_ReportDevError(func, error) ReportDevError(CSC0_MODULE_ID, func, error)
```

```
#define CSC0_AONCLK_128K 128000U
```

```
#define CSC0_AONCLK_32K 32000U
```

```
#define CSC0_AONCLK_SOSC_32K 32768U
```

```
#define CSC0_AONCLK_1K 1000U
```

CSC0 API Service IDs#define **CSC0_SET_CLOCK_OUT_ID** 0x00U

```
#define CSC0_SET_AONCLKSRC_ID 0x01U
```

```
#define CSC0_GET_CLOCK_FREQ_ID 0x02U
```

CSC0 Dev Error Code*Error Code of calling CSC0 apis*

```
#define CSC0_E_PARAM_POINTER 0x01U
```

```
#define CSC0_E_PARAM_OUT_RANGE 0x02U
```

Enumerations

```
enum CSC0_ClockStatusDef { CSC0_CLOCK_UNDEFINE = 0U, CSC0_CLOCK_DISABLE = 1U,  
CSC0_CLOCK_VALID = 2U, CSC0_CLOCK_ERROR = 3U, CSC0_CLOCK_UNKNOWN = 4U }
```

Indicate the clock status for each clock source in clock tree list.

```
enum CSC0_ClkSrcType { CSC0_AON32K_CLK = 0U, CSC0_RTC_CLK = 1U, CSC0_AON_CLK = 2U,  
CSC0_CLKOUT_CLK = 3U, CSC0_END_OF_CLOCKS = 4U }
```

CSC0 clock source type for clock information query.


```
enum CSC_RetStatusType { CSC_E_OK = 0U, CSC_E_NOT_OK }
```

CSC return status.

Detailed Description

csc driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_dma.h File Reference

DMA driver type definition and API.

```
#include "HwA_dma.h"
```

```
#include "HwA_dmamux.h"
```

Include dependency graph for module_driver_dma.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Classes

struct **DMA_InitType**

The configuration parameters of the DMA engine.

struct **DMA_InstanceHandleType**

The structure of the DMA processing handle.

struct **_DMA_HandleType**

struct **DMA_ChannelCfgType**

The configuration parameters of the DMA channel.

struct **DMA_CircularBufferType**

The parameters to configure the DMA channel as a circular buffer engine.

struct **DMA_ChainTransferType**

The parameters to configure the DMA chained transfer.

Macros

```
#define DMA_CHANNEL_INVALID (0xFFU)
```

```
DMA API Service IDs#define DMA_INIT_ID 0U
```

```
#define DMA_INIT_CHANNEL_ID 1U
```

```
#define DMA_DEINIT_CHANNEL_ID 2U
```

```
#define DMA_INIT_CHANNEL_INT_ID 3U
```

```
#define DMA_CONFIG_CHAINED_TRANS_ID 4U
```

```
#define DMA_CONFIG_MODIFY_ADDR_ID 5U
```

```
#define DMA_CONFIG_START_CH_ID 6U
```

```
#define DMA_CONFIG_STOP_CH_ID 7U
```

```

#define DMA_CONFIG_GET_CH_REQ_SOURCE_ID 8U
#define DMA_CONFIG_GET_CH_STATUS_ID 9U
#define DMA_CONFIG_GET_MON_ID 10U
#define DMA_CONFIG_GET_STATUS_ID 11U
#define DMA_CONFIG_CLEAR_MON_ID 12U
#define DMA_CONFIG_DISABLE_MON_ID 13U
#define DMA_CONFIG_EN_MON_ID 14U
#define DMA_CONFIG_CANCEL_TRANS_ID 15U

```

DMA Dev Error Code *Error Code of calling DMA apis*

```

#define DMA_E_PARAM_CHCFG 0x01U
#define DMA_E_PARAM_BLOCKCNT 0x02U
#define DMA_E_PARAM_BLOCKSIZE 0x03U
#define DMA_E_PARAM_DESTCIRCULARBUFSIZE 0x04U
#define DMA_E_PARAM_SRCCIRCULARBUFSIZE 0x05U
#define DMA_E_PARAM_INTERRUPTCFG 0x06U
#define DMA_E_PARAM_CHANNEL 0x07U
#define DMA_E_PARAM_INSTANCE 0x08U
#define DMA_E_PARAM_INITCFG 0x09U
#define DMA_E_PARAM_BLOCKSIZE_DATASIZE 0x0AU

```

Typedefs

```
typedef void(* DMA_TransferCompleteCallbackType) (void *arg)
```

DMA transfer complete callback function prototype.

```
typedef void(* DMA_TransferErrorCallbackType) (void *arg)
```

DMA transfer error callback function prototype.

```
typedef struct _DMA_HandleType DMA_HandleType
```

Enumerations

```
enum DMA_InstanceType { DMA_INSTANCE_0 = 0U, DMA_INSTANCE_1 = 1U, DMA_INSTANCE_MAX =
DMA_INSTANCE_COUNT }
```

Available DMA Instances.

```
enum DMA_ChannelType { DMA_CHANNEL_0 = 0U, DMA_CHANNEL_1 = 1U, DMA_CHANNEL_2 = 2U,
DMA_CHANNEL_3 = 3U, DMA_CHANNEL_4 = 4U, DMA_CHANNEL_5 = 5U, DMA_CHANNEL_6 = 6U,
DMA_CHANNEL_7 = 7U, DMA_CHANNEL_8 = 8U, DMA_CHANNEL_9 = 9U, DMA_CHANNEL_10 = 10U,
DMA_CHANNEL_11 = 11U, DMA_CHANNEL_12 = 12U, DMA_CHANNEL_13 = 13U, DMA_CHANNEL_14 = 14U,

```

DMA_CHANNEL_15 = 15U, DMA_CHANNEL_16 = 16U, DMA_CHANNEL_17 = 17U, DMA_CHANNEL_18 = 18U, DMA_CHANNEL_19 = 19U, DMA_CHANNEL_20 = 20U, DMA_CHANNEL_21 = 21U, DMA_CHANNEL_22 = 22U, DMA_CHANNEL_23 = 23U, DMA_CHANNEL_24 = 24U, DMA_CHANNEL_25 = 25U, DMA_CHANNEL_26 = 26U, DMA_CHANNEL_27 = 27U, DMA_CHANNEL_28 = 28U, DMA_CHANNEL_29 = 29U, DMA_CHANNEL_30 = 30U, DMA_CHANNEL_31 = 31U, DMA_CHANNEL_MAX = DMA_CFG_COUNT }

Available DMA channels.

enum DMA_StatusType { DMA_STATUS_SUCCESS = 0x00U, DMA_STATUS_ERROR = 0x01U, DMA_STATUS_BUSY = 0x02U, DMA_STATUS_TIMEOUT = 0x03U, DMA_STATUS_UNSUPPORTED = 0x04U, DMA_STATUS_INVALID_ADDRESS = 0x05U, DMA_STATUS_NO_RESOURCE = 0x06U }

DMA operation return values.

enum DMA_IncrementModeType { DMA_INCREMENT_DISABLE = 0x0U, DMA_INCREMENT_DATA_SIZE = 0x1U, DMA_INCREMENT_DATA_SIZE_4BYTE_ALIGNED = 0x2U }

DMA data increment size.

enum DMA_CircularBufferSizeType { DMA_CIRCULAR_BUFFER_SIZE_1B = 0x1U, DMA_CIRCULAR_BUFFER_SIZE_2B = 0x2U, DMA_CIRCULAR_BUFFER_SIZE_4B = 0x4U, DMA_CIRCULAR_BUFFER_SIZE_8B = 0x8U, DMA_CIRCULAR_BUFFER_SIZE_16B = 0x10U, DMA_CIRCULAR_BUFFER_SIZE_32B = 0x20U, DMA_CIRCULAR_BUFFER_SIZE_64B = 0x40U, DMA_CIRCULAR_BUFFER_SIZE_128B = 0x80U, DMA_CIRCULAR_BUFFER_SIZE_256B = 0x100U, DMA_CIRCULAR_BUFFER_SIZE_512B = 0x200U, DMA_CIRCULAR_BUFFER_SIZE_1KB = 0x400U, DMA_CIRCULAR_BUFFER_SIZE_2KB = 0x800U, DMA_CIRCULAR_BUFFER_SIZE_4KB = 0x1000U, DMA_CIRCULAR_BUFFER_SIZE_8KB = 0x2000U, DMA_CIRCULAR_BUFFER_SIZE_16KB = 0x4000U, DMA_CIRCULAR_BUFFER_SIZE_32KB = 0x8000U, DMA_CIRCULAR_BUFFER_SIZE_64KB = 0x10000U, DMA_CIRCULAR_BUFFER_SIZE_128KB = 0x20000U, DMA_CIRCULAR_BUFFER_SIZE_256KB = 0x40000U, DMA_CIRCULAR_BUFFER_SIZE_512KB = 0x80000U, DMA_CIRCULAR_BUFFER_SIZE_1MB = 0x100000U, DMA_CIRCULAR_BUFFER_SIZE_2MB = 0x200000U, DMA_CIRCULAR_BUFFER_SIZE_4MB = 0x400000U, DMA_CIRCULAR_BUFFER_SIZE_8MB = 0x800000U, DMA_CIRCULAR_BUFFER_SIZE_16MB = 0x1000000U, DMA_CIRCULAR_BUFFER_SIZE_32MB = 0x2000000U, DMA_CIRCULAR_BUFFER_SIZE_64MB = 0x4000000U, DMA_CIRCULAR_BUFFER_SIZE_128MB = 0x8000000U, DMA_CIRCULAR_BUFFER_SIZE_256MB = 0x10000000U, DMA_CIRCULAR_BUFFER_SIZE_512MB = 0x20000000U, DMA_CIRCULAR_BUFFER_SIZE_1GB = 0x40000000U, DMA_CIRCULAR_BUFFER_SIZE_2GB = 0x80000000U }

The size of the circular buffer.

enum DMA_CircularBufferRoleType { DMA_CIRCULAR_BUFFER_ROLE_SOURCE = 0x0U, DMA_CIRCULAR_BUFFER_ROLE_DESTINATION = 0x1U }

The eBufferRole of the circular buffer.

Functions

void **DMA_Init** (**DMA_InstanceHandleType** *pDmaInstanceHandle, const **DMA_InitType** *const pInitCfg)
Initialize the DMA instance.

void **DMA_DeInit** (**DMA_InstanceHandleType** *pDmaInstanceHandle)
De-initialize the DMA instance.

DMA_StatusType **DMA_InitChannel** (**DMA_HandleType** *pDmaHandle, const **DMA_ChannelCfgType** *const pChnCfg)
Initialize the DMA channel.

void **DMA_DeinitChannel** (**DMA_HandleType** *pDmaHandle)
De-initialize the DMA channel.

void **DMA_InitChannelInterrupt** (**DMA_HandleType** *pDmaHandle, const **DMA_ChannelCfgType** *const pChnCfg)
Initialize the DMA channel interrupt.

void **DMA_ConfigChainedTransfer** (**DMA_HandleType** *pDmaHandle, const **DMA_ChainTransferType** *const pChainTransferCfg)
Config the DMA channel chained transfer.

DMA_StatusType **DMA_ModifyAddress** (**DMA_HandleType** *pDmaHandle, const volatile void *pSrcBuffer, const volatile void *pDestBuffer)
Modify the source and destination address of the DMA channel.

void **DMA_StartChannel** (**DMA_HandleType** *pDmaHandle)
Start the DMA channel transfer.

void **DMA_StopChannel** (**DMA_HandleType** *pDmaHandle)
Stop the DMA channel transfer.

DMA_StatusType **DMA_CancelTransfer** (**DMA_InstanceHandleType** *pDmaInstanceHandle, bool bGenerateErr)

Cancel the ongoing DMA transfer.

DMA_RequestSourceType **DMA_GetChannelRequestSrc** (DMA_HandleType *pDmaHandle)

Get the request source of the selected channel.

DMA_RunningStatusType **DMA_GetStatus** (DMA_InstanceHandleType *pDmaInstanceHandle)

Get the status of the DMA engine.

DMA_RunningStatusType **DMA_GetChannelStatus** (DMA_HandleType *pDmaHandle)

Get the status of the DMA channel.

void **DMA_Transfer_Complete_IRQHandler** (DMA_HandleType *pDmaHandle)

The internal interrupt handler for DMA transfer complete.

void **DMA_ProcessErrorInterrupt** (DMA_InstanceHandleType *pDmaInstanceHandle)

The internal interrupt handler for DMA error complete.

Detailed Description

DMA driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

module_driver_dsp.h File Reference

DSP driver type definition and API.

#include "device_header.h"

Include dependency graph for module_driver_dsp.h:

IMAGE

Detailed Description

DSP driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

```
If only want use FPU,
1) configure FCIDE to enable FPU compiler support, "Properties" -> C/C++ Build -> Settings -> Tool Settings -> Target Processor -> Float ABI ->
FP instructions(hard) -> FPU Type set to "fpv5-sp-d16"
2) configure FCIDE to enable FPU compiler support, "Properties" -> C/C++ Build -> Settings -> Tool Settings -> GNU Arm Cross C Compiler ->
Preprocessor -> Defined symbols(-D) -> Add "__FPU_PRESENT=1" (without ")
3) and call FPU_Enable to enable FPU at the beginning of program.

If want to use DSP,
1) configure FCIDE to enable FPU compiler support, Properties -> C/C++ Build -> Settings -> Tool Settings -> Target Processor -> Float ABI ->
FP instructions(hard)
2) add MACRO "DRIVER_CM7_DSP_ENABLE" in FCIDE Properties -> C/C++ General -> Paths and Symbols -> Symbols -> GNU C and GNU C++
and Assembly
3) add Include Path to project Properties -> C/C++ General -> Paths and Symbols -> Includes -> GNU C and GNU C++ and Assembly:
../../../../../Template/Device/CMSIS5_590/DSP/Include
../../../../../Template/Device/CMSIS5_590/Core/Include
```

```
../../../../Template/Device/CMSIS5_590/DSP/PrivateInclude
```

4) and call FPU_Enable to enable FPU at the beginning of program.

module_driver_eftu.h File Reference

eFTU driver type definition and API

```
#include <stdint.h>
```

```
#include "device_header.h"
```

```
#include <math.h>
```

Include dependency graph for module_driver_eftu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

eFTU driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_eftu_ccm.h File Reference

eFTU ccm driver type definition and API

```
#include "hwa_eftu_ccm.h"
```

```
#include "module_driver_eftu.h"
```

```
#include "module_driver_eftu_cmu.h"
```

Include dependency graph for module_driver_eftu_ccm.h:

IMAGE

Detailed Description

eFTU ccm driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_eftu_cmu.h File Reference

eFTU cmu driver type definition and API

```
#include "hwa_eftu_cmu.h"
```

```
#include "module_driver_eftu.h"
```

Include dependency graph for module_driver_eftu_cmu.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Detailed Description

eFTU cmu driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_eftu_dtm.h File Reference

eFTU dtm driver type definition and API

#include "HwA_eftu_dtm.h"

#include "module_driver_eftu.h"

#include "module_driver_eftu_cmu.h"

Include dependency graph for module_driver_eftu_dtm.h:

IMAGE

Detailed Description

eFTU dtm driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_eftu_tbu.h File Reference

eFTU tbu driver type definition and API

#include "hwa_eftu_tbu.h"

#include "module_driver_eftu.h"

Include dependency graph for module_driver_eftu_tbu.h:

IMAGE

Detailed Description

eFTU tbu driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_eftu_tim.h File Reference

eFTU tim driver type definition and API

#include "hwa_eftu_tim.h"

#include "module_driver_eftu.h"

Include dependency graph for module_driver_eftu_tim.h:

IMAGE

Detailed Description

eFTU tim driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_eftu_tom.h File Reference

eFTU tom driver type definition and API

#include "HwA_eftu_tom.h"

Include dependency graph for module_driver_eftu_tom.h:

IMAGE

Detailed Description

eFTU tom driver type definition and API

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_eim.h File Reference

FC7xxx EIM driver type definition and API.

#include "HwA_eim.h"

Include dependency graph for module_driver_eim.h:

IMAGE

Classes

struct **_EIM_HandleType**

The structure of the EIM processing handle.

struct **EIM_ChCfgType**

Define the max EIM channels.

Macros

EIM API Service IDs#define **EIM_INIT_ID** 0x00U

#define **EIM_DEINIT_ID** 0x01U

#define **EIM_DISABLEINJECTION_ID** 0x02U

EIM Dev Error Code#define **EIM_E_PARAM_NULLPTR** 0x01U

#define **EIM_E_PARAM_INSTANCE** 0x02U

Define EIM monitor configuration type#define **EIM_LOCKSTEP_MONITOR0** 0U

#define **EIM_LOCKSTEP_MONITOR1** 1U

Error injection bus select#define **EIM_BUSREG_0** 0U

#define **EIM_BUSREG_1** 1U

#define **EIM_BUSREG_2** 2U

#define **EIM_BUSREG_3** 3U

Typedefs

typedef struct **_EIM_HandleType** **EIM_HandleType**

The structure of the EIM processing handle.

Enumerations

enum **EIM_InstanceType** { **EIM_INSTANCE_0** = 0U }

EIM Instance type.

enum **EIM_StatusType** { **EIM_STATUS_SUCCESS** = 0U, **EIM_STATUS_FAIL** = 1U }

EIM operation return values.

Functions

EIM_StatusType EIM_Init (**EIM_HandleType** *const pHandle, const **EIM_ChCfgType** *const pInitCfg)

Initializes the Error Injection Module(EIM) with provided configuration settings.

void EIM_Deinit (**EIM_HandleType** *const pHandle)

Deinitializes the Error Injection Module(EIM).

void EIM_DisableInjection (**EIM_HandleType** *const pHandle)

Disables error injection in the EIM module.

Detailed Description

FC7xxx EIM driver type definition and API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_elu.h File Reference

#include "HwA_elu.h"

Include dependency graph for module_driver_elu.h:

IMAGE

module_driver_enet.h File Reference

ENET driver type definition and API.

```
#include "HwA_enet.h"
```

```
#include "module_driver_scg.h"
```

```
#include "module_driver_pcc.h"
```

Include dependency graph for module_driver_enet.h:

IMAGE

Classes

struct **ENET_DMADescType**

ENET DMA descriptor type.

struct **ENET_MiiConfigType**

Mdio configuration type.

struct **ENET_TimestampType**

IEEE1588 time stamp type.

struct **ENET_FlexPPSConfigType**

IEEE1588 flexible PPS configuration type.

struct **Enet_PTPConfig**

IEEE1588 time stamp configuration type.

struct **ENET_MMConfig**

Defines MMC counter configuration.

struct **ENET_EstConfig**

ENET EST configuration type.

struct **ENET_GateCtrl**

The ENET EST Gate control type.

struct **ENET_GateCtrlList**

The ENET EST Gate control List type.

struct **ENET_PreemptionConfig**

The ENET EST Frame Preemption configuration type.

struct **ENET_Buffer**

Defines the buffer type for ENET driver.

struct **ENET_TxVlanConfig**

ENET transmit VLAN configuration type.

struct **ENET_OuterVlanCBTI**

ENET Channel based VLAN insertion type.

struct **ENET_TxOption**

Defines the options for the Tx packet.

struct **ENET_RxInfo**

Defines the received buffer information.

struct **ENET_DescList**

DMA descriptor list type for internal use by ENET driver.

struct **ENET_DMACHannelConfig**

DMA channel configuration type.

struct **ENET_DMAConfig**

ENET DMA configuration type.

struct **ENET_MACFilter**

ENET MAC filter configuration type.

struct **ENET_L3L4Filter**

ENET Layer3 Layer4 filter configuration type.

struct **ENET_VlanFilterConfig**

ENET VLAN filter configuration type.

struct **ENET_VlanPerfectFilter**

ENET VLAN perfect filter type.

struct **ENET_FlowControlRxQ**

ENET flow control receive queue configuration type.

struct **ENET_FlowControl**

ENET flow control configuration type.

struct **ENET_MACConfig**

ENET MAC configuration type.

struct **ENET_MTLQueueConfig**

ENET MTL Queue configuration type.

struct **ENET_MTLConfig**

ENET MTL configuration type.

struct **ENET_DescConfig**

ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA.It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache.

struct **ENET_Config**

ENET configuration type.

struct **ENET_DrvHandle**

ENET handle is internal used by ENET driver.

Macros

#define **ENET_DATA_BUS_WIDTH_BYTES** (ENET_DATA_BUS_WIDTH_BITS / 8u)

ENET DMA descriptor length.

#define **ENET_DMA_DESC_LEN_BYTES** (16u)

ENET DMA descriptor length.

#define **ENET_MAX_FRAME_LENGTH** 1536u

Max Ethernet frame length.

#define **ENET_MAC_ADDRESS_LENGTH** 6u

Ethernet MAC address length.

#define **ENET_TRUE** (bool)1u

ENET Boolean true.

#define **ENET_FALSE** (bool)0u

ENET Boolean false.

#define **ENET_DATA_BARRIER()** DMB()

ENET data barrier.

#define **ENET_TDES2_IOC_MASK** (0x80000000u)

TDESC2 Interrupt Enabled on Completion in transmit descriptor 2 read format.

#define **ENET_TDES2_TTSE_MASK** (0x40000000u)

TDESC2 Transmit Timestamp Enable in transmit descriptor 2 read format.

#define **ENET_TDES2_VTIR(x)** (((uint32_t)(((uint32_t)(x)) << 14u)) & 0x0000C000u)

TDESC2 VLAN Tag Insertion or Replacement in transmit descriptor 2 read format.

```
#define ENET_TDES2_BUFFER1_LEN(x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x00003FFFu)
```

TDESC2 Header Length or Buffer 1 Length in transmit descriptor 2 read format.

```
#define ENET_TDES2_BUFFER2_LEN(x) (((uint32_t)(((uint32_t)(x)) << 16u)) & 0x3FFF0000u)
```

TDESC2 Buffer 2 Length in transmit descriptor 2 read format.

```
#define ENET_TDES3_CPC(x) (((uint32_t)(((uint32_t)(x)) << 26u)) & 0x0C000000u)
```

CRC Pad Control in transmit descriptor 3 read format.

```
#define ENET_TDES3_CIC(x) (((uint32_t)(((uint32_t)(x)) << 16u)) & 0x00030000u)
```

Checksum Insertion Control in transmit descriptor 3 read format.

```
#define ENET_TDES3_FD_MASK (0x20000000u)
```

First Descriptor in transmit descriptor 3 read format.

```
#define ENET_TDES3_LD_MASK (0x10000000u)
```

Last Descriptor in transmit descriptor 3 read format.

```
#define ENET_TDES3_THL(x) (((uint32_t)(((uint32_t)(x)) << 19u)) & 0x00780000u)
```

TCP/UDP Header Length in transmit descriptor 3 read format.

```
#define ENET_TDES3_TSE_MASK (0x00040000u)
```

TCP Segmentation Enable in transmit descriptor 3 read format.

```
#define ENET_TDES3_TPL_MASK (0x0003FFFFu)
```

TCP Payload Length mask in transmit descriptor 3 read format.

```
#define ENET_TDES3_TPL(x) (((uint32_t)(((uint32_t)(x)) << 0u)) & 0x0003FFFFu)
```

TCP Payload Length in transmit descriptor 3 read format.

```
#define ENET_TDES3_FL_MASK (0x00007FFFu)
```

Frame Length mask in transmit descriptor 3 read format.

```
#define ENET_TDES3_FL(x) (((uint32_t)((uint32_t)(x) << 0u)) & 0x00007FFFu)
```

Frame Length in transmit descriptor 3 read format.

```
#define ENET_TDES3_TTSS_MASK (0x00020000u)
```

Tx Timestamp Status in transmit descriptor 3 write back format.

```
#define ENET_TDES2_IVT(x) (((uint32_t)((uint32_t)(x) << 16u)) & 0xFFFF0000u)
```

Inner VLAN Tag in transmit descriptor 2 context format.

```
#define ENET_TDES2_MSS(x) (((uint32_t)((uint32_t)(x) << 0u)) & 0x00003FFFu)
```

Maximum Segment Size in transmit descriptor 2 context format.

```
#define ENET_TDES3_TCMSSV_MASK (0x04000000u)
```

One-Step Timestamp Correction Input or MSS Valid in transmit descriptor 3 context format.

```
#define ENET_TDES3_IVTIR(x) (((uint32_t)((uint32_t)(x) << 18u)) & 0x000C0000u)
```

Inner VLAN Tag Insert or Replace in transmit descriptor 3 context format.

```
#define ENET_TDES3_IVLTV_MASK (0x00020000u)
```

Inner VLAN Tag Valid in transmit descriptor 3 context format.

```
#define ENET_TDES3_VLTV_MASK (0x00010000u)
```

VLAN Tag Valid in transmit descriptor 3 context format.

```
#define ENET_TDES3_VT(x) (((uint32_t)((uint32_t)(x) << 0u)) & 0x0000FFFFu)
```

VLAN Tag in transmit descriptor 3 context format.

```
#define ENET_RDES3_IOC_MASK (0x40000000u)
```

Interrupt Enabled on Completion in receive descriptor 3 read format.

```
#define ENET_RDES3_BUFFER1_VALID_MASK (0x01000000u)
```

Buffer 1 Address Valid in receive descriptor 3 read format.

#define ENET_RDES3_BUFFER2_VALID_MASK (0x02000000u)

Buffer 2 Address Valid in receive descriptor 3 read format.

#define ENET_RDES0_INNER_VLAN_TAG_MASK (0xFFFF0000u)

Inner VLAN Tag mask in receive descriptor 0 write back format.

#define ENET_RDES0_INNER_VLAN_TAG_SHIFT (16u)

Inner VLAN Tag shift in receive descriptor 0 write back format.

#define ENET_RDES0_OUTER_VLAN_TAG_MASK (0x0000FFFFu)

Outer VLAN Tag mask in receive descriptor 0 write back format.

#define ENET_RDES1_TIMESTAMP_AVAILABLE_MASK (0x00004000u)

Timestamp Available in receive descriptor 1 write back format.

#define ENET_RDES1_IP_PAYLOAD_ERROR_MASK (0x00000080u)

IP Payload Error in receive descriptor 1 write back format.

#define ENET_RDES1_IPV6_HEADER_MASK (0x00000020u)

IPv6 header Present in receive descriptor 1 write back format.

#define ENET_RDES1_IPV4_HEADER_MASK (0x00000010u)

IPv4 Header Present in receive descriptor 1 write back format.

#define ENET_RDES1_IP_HEADER_ERROR_MASK (0x00000008u)

IP Header Error in receive descriptor 1 write back format.

#define ENET_RDES1_IP_PAYLOAD_TYPE_MASK (0x00000007u)

Payload Type in receive descriptor 1 write back format.

#define ENET_RDES3_FD_MASK (0x20000000u)

First Descriptor in receive descriptor 3 write back format.

#define ENET_RDES3_LD_MASK (0x10000000u)

Last Descriptor in receive descriptor 3 write back format.

#define ENET_RDES3_PACKET_LENGTH_MASK (0x00007FFFu)

Packet Length in receive descriptor 3 write back format.

#define ENET_RDES3_RDES0_VALID_MASK (0x02000000u)

Receive Status RDES0 Valid in receive descriptor 3 write back format.

#define ENET_RDES3_RDES1_VALID_MASK (0x04000000u)

Receive Status RDES1 Valid in receive descriptor 3 write back format.

#define ENET_RDES3_RDES2_VALID_MASK (0x08000000u)

Receive Status RDES2 Valid in receive descriptor 3 write back format.

#define ENET_DES3_OWN_MASK (0x80000000u)

Own Bit for all format.

#define ENET_DES3_CTXT_MASK (0x40000000u)

Context Descriptor bit for all format.

ENET API Service IDs *ENET API Service IDs*

#define ENET_DEFAULT_CONFIG_SERVICE_ID 0u

#define ENET_DEINIT_SERVICE_ID 1u

#define ENET_INIT_SERVICE_ID 2u

#define ENET_PREPARE_DESC_SERVICE_ID 3u

#define ENET_START_SERVICE_ID 4u

#define ENET_STOP_SERVICE_ID 5u

#define ENET_STARTIT_SERVICE_ID 6u

#define ENET_STOPIT_SERVICE_ID 7u

#define ENET_SEND_FRAME_SERVICE_ID 8u

#define ENET_READ_FRAME_SERVICE_ID 9u

```

#define ENET_BUILD_RXDESC_SERVICE_ID 10u
#define ENET_SET_MACADDR_SERVICE_ID 11u
#define ENET_CKSUM_ERROR_FILTER_SERVICE_ID 12u
#define ENET_SET_MACHASH_SERVICE_ID 13u
#define ENET_DEFAULT_MACFILTER_SERVICE_ID 14u
#define ENET_GET_MACFILTER_SERVICE_ID 15u
#define ENET_SET_MACFILTER_SERVICE_ID 16u
#define ENET_DEFAULT_L3L4FILTER_SERVICE_ID 17u
#define ENET_GET_L3L4FILTER_SERVICE_ID 18u
#define ENET_SET_L3L4FILTER_SERVICE_ID 19u
#define ENET_ENABLE_L3L4FILTER_SERVICE_ID 20u
#define ENET_GET_VLANFILTER_CFG_SERVICE_ID 21u
#define ENET_ENABLE_VLANFILTER_SERVICE_ID 22u
#define ENET_SET_VLANHASH_SERVICE_ID 23u
#define ENET_GET_VLAN_PERFECTFILTER_SERVICE_ID 24u
#define ENET_SET_VLAN_PERFECTFILTER_SERVICE_ID 25u
#define ENET_SET_TXOUTERVLAN_SERVICE_ID 26u
#define ENET_SET_TXOUTVLAN_CBI_SERVICE_ID 27u
#define ENET_SET_TXINNERVLAN_SERVICE_ID 28u
#define ENET_REGISTER_SYSEVENT_SERVICE_ID 29u
#define ENET_REGISTER_RXCOMP_SERVICE_ID 30u
#define ENET_REGISTER_TXCOMP_SERVICE_ID 31u
#define ENET_REGISTER_TXWAIT_SERVICE_ID 32u
#define ENET_MDIO_DEFCONFIG_SERVICE_ID 33u
#define ENET_MDIO_INIT_SERVICE_ID 34u
#define ENET_MDIO_READ_SERVICE_ID 35u
#define ENET_MDIO_WRITE_SERVICE_ID 36u
#define ENET_MMD_READ_SERVICE_ID 37u
#define ENET_MMD_WRITE_SERVICE_ID 38u
#define ENET_PTP_DEFCONFIG_SERVICE_ID 39u
#define ENET_PTP_START_SERVICE_ID 40u
#define ENET_PTP_STOP_SERVICE_ID 41u
#define ENET_PTP_GETTIME_SERVICE_ID 42u
#define ENET_PTP_SETTIME_SERVICE_ID 43u
#define ENET_PTP_UPDATETIME_SERVICE_ID 44u
#define ENET_PTP_ADJUST_FREQ_SERVICE_ID 45u
#define ENET_PTP_FIXED_PPSOUT_SERVICE_ID 46u
#define ENET_PTP_FLEXIBLE_PPSOUT_SERVICE_ID 47u

```

```

#define ENET_GET_MMCCONFIG_SERVICE_ID 48u
#define ENET_SET_MMCCONFIG_SERVICE_ID 49u
#define ENET_MMC_COUNTER_PRESET_SERVICE_ID 50u
#define ENET_MMC_COUNTER_FREEZE_SERVICE_ID 51u
#define ENET_MMC_COUNTER_RESET_SERVICE_ID 52u
#define ENET_MMC_GET_TXINT_SERVICE_ID 53u
#define ENET_MMC_GET_RXINT_SERVICE_ID 54u
#define ENET_MMC_GET_TXCOUNTER_SERVICE_ID 55u
#define ENET_MMC_GET_RXCOUNTER_SERVICE_ID 56u
#define ENET_GET_FLOWCTRL_SERVICE_ID 57u
#define ENET_SET_FLOWCTRL_SERVICE_ID 58u
#define ENET_FLOWCTRL_SOFTTRIG_SERVICE_ID 59u
#define ENET_SYSIRQ_HANDLER_SERVICE_ID 60u
#define ENET_RXIRQ_HANDLER_SERVICE_ID 61u
#define ENET_TXIRQ_HANDLER_SERVICE_ID 62u
#define ENET_GET_ESTCONFIG_SERVICE_ID 63u
#define ENET_EST_CONFIG_SERVICE_ID 64u
#define ENET_EST_ENABLE_SERVICE_ID 65u
#define ENET_EST_GCLCONFIG_SERVICE_ID 66u
#define ENET_EST_GETGCLCONFIG_SERVICE_ID 67u
#define ENET_PREEMPTION_CONFIG_SERVICE_ID 68u

```

ENET DEV Error Code *ENET DEV Error Code*

```

#define ENET_E_DRIVER_UNINIT 0u
#define ENET_E_DRIVER_REPEAT_INIT 1u
#define ENET_E_CONTROLLER_MODE 2u
#define ENET_E_PARAM_HANDLE 3u
#define ENET_E_PARAM_INSTANCE 4u
#define ENET_E_PARAM_POINTER 5u
#define ENET_E_PARAM_TXCHANNEL 6u
#define ENET_E_PARAM_RXCHANNEL 7u
#define ENET_E_PARAM_GROUP 8u
#define ENET_E_PARAM_ASSERTION 9u
#define ENET_E_PARAM_MACLEN 10u
#define ENET_E_PARAM_PPSCHANNEL 11u

```

Typedefs

```
typedef struct ENET_DrvHandle ENET_HandleType
```

typedef **ENET_StatusType**(* **ENET_EventCallbackType**) (const **ENET_HandleType** *pEnetHandle, **ENET_EventType** eEvent)

Callback function invoked when a general event is encountered.

typedef **ENET_StatusType**(* **ENET_ChannelCallbackType**) (const **ENET_HandleType** *pEnetHandle, const uint8 u8Channel)

Callback function invoked when a channel event is encountered.

typedef struct **Enet_PTPConfig** **Enet_PTPConfigType**

IEEE1588 time stamp configuration type.

typedef struct **ENET_MMCCConfig** **ENET_MMCCConfigType**

Defines MMC counter configuration.

typedef struct **ENET_EstConfig** **ENET_EstConfigType**

ENET EST configuration type.

typedef struct **ENET_GateCtrl** **ENET_GateCtrlType**

The ENET EST Gate control type.

typedef struct **ENET_GateCtrlList** **ENET_GateCtrlListType**

The ENET EST Gate control List type.

typedef struct **ENET_PreemptionConfig** **ENET_PreemptionConfigType**

The ENET EST Frame Preemption configuration type.

typedef struct **ENET_Buffer** **ENET_BufferType**

Defines the buffer type for ENET driver.

typedef struct **ENET_TxVlanConfig** **ENET_TxVlanConfigType**

ENET transmit VLAN configuration type.

typedef struct **ENET_OuterVlanCBTI** **ENET_OuterVlanCBTIType**

ENET Channel based VLAN insertion type.

typedef struct **ENET_TxOption** ENET_TxOptionType

Defines the options for the Tx packet.

typedef struct **ENET_RxInfo** ENET_RxInfoType

Defines the received buffer information.

typedef struct **ENET_DescList** ENET_DescListType

DMA descriptor list type for internal use by ENET driver.

typedef struct **ENET_DMACHannelConfig** ENET_DMACHannelConfigType

DMA channel configuration type.

typedef struct **ENET_DMAConfig** ENET_DMAConfigType

ENET DMA configuration type.

typedef struct **ENET_MACFilter** ENET_MACFilterType

ENET MAC filter configuration type.

typedef struct **ENET_L3L4Filter** ENET_L3L4FilterType

ENET Layer3 Layer4 filter configuration type.

typedef struct **ENET_VlanFilterConfig** ENET_VlanFilterConfigType

ENET VLAN filter configuration type.

typedef struct **ENET_VlanPerfectFilter** ENET_VlanPerfectFilterType

ENET VLAN perfect filter type.

typedef struct **ENET_FlowControlRxQ** ENET_FlowControlRxQType

ENET flow control receive queue configuration type.

typedef struct **ENET_FlowControl** ENET_FlowControlType

ENET flow control configuration type.

typedef struct **ENET_MACConfig** **ENET_MACConfigType**

ENET MAC configuration type.

typedef struct **ENET_MTLQueueConfig** **ENET_MTLQueueConfigType**

ENET MTL Queue configuration type.

typedef struct **ENET_MTLConfig** **ENET_MTLConfigType**

ENET MTL configuration type.

typedef struct **ENET_DescConfig** **ENET_DescConfigType**

ENET DMA descriptor configuration type Note: For TX DMA descriptor: Buffer is not necessary. DescList will be accessed by MCU and DMA. It is best to place DescList in a non-cacheable area, or users need to manually refresh the cache.

typedef struct **ENET_Config** **ENET_ConfigType**

ENET configuration type.

Enumerations

enum **ENET_InstanceType** { **ENET_INSTANCE_0** = 0x0u }

ENET instances.

enum **ENET_ChannelType** { **ENET_CHANNEL_0** = **ENET_DMA_CHANNEL_0**, **ENET_CHANNEL_1** = **ENET_DMA_CHANNEL_1** }

ENET channel index type.

enum **ENET_StatusType** { **ENET_STATUS_SUCCESS** = 0x00u, **ENET_STATUS_ERROR** = 0x01u, **ENET_STATUS_BUSY** = 0x02u, **ENET_STATUS_TIMEOUT** = 0x03u, **ENET_STATUS_INVALID_PARAM** = 0x04u, **ENET_STATUS_UNSUPPORTED** = 0x05u, **ENET_STATUS_NOT_READY** = 0x06u }

ENET operation return values type.

enum **ENET_StateType** { **ENET_STATE_UNINIT** = 0u, **ENET_STATE_INITIALIZED**, **ENET_STATE_READY**, **ENET_STATE_BUSY**, **ENET_STATE_STOPPED**, **ENET_STATE_ERROR**, **ENET_STATE_WAIT_RELEASE** }

ENET driver state values type.

enum **ENET_DMA_SysbusModeType** { **ENET_DMA_SYSBUS_MODE_FIXED** = 0x0u, **ENET_DMA_SYSBUS_MODE_MIXED** = 0x1u }

ENET DMA system bus mode type.

```
enum ENET_DMABurstLengthType { ENET_DMA_BURST_LENGTH_1_BEAT = 1u,
ENET_DMA_BURST_LENGTH_2_BEATS = 2u, ENET_DMA_BURST_LENGTH_4_BEATS = 4u,
ENET_DMA_BURST_LENGTH_8_BEATS = 8u, ENET_DMA_BURST_LENGTH_16_BEATS = 16u,
ENET_DMA_BURST_LENGTH_32_BEATS = 32u, ENET_DMA_BURST_LENGTH_64_BEATS = 64u,
ENET_DMA_BURST_LENGTH_128_BEATS = 128u, ENET_DMA_BURST_LENGTH_256_BEATS = 256u }
```

The burst length for DMA TX and RX.

```
enum ENET_EventType { ENET_EVENT_NONE = 0u, ENET_EVENT_TARGET_TIME0_REACHED,
ENET_EVENT_TARGET_TIME0_ERROR, ENET_EVENT_TARGET_TIME1_REACHED,
ENET_EVENT_TARGET_TIME1_ERROR, ENET_EVENT_TARGET_TIME2_REACHED,
ENET_EVENT_TARGET_TIME2_ERROR, ENET_EVENT_TARGET_TIME3_REACHED,
ENET_EVENT_TARGET_TIME3_ERROR, ENET_EVENT_MMC_TX_COUNTER,
ENET_EVENT_MMC_RX_COUNTER }
```

ENET general event type.

```
enum ENET_MiiOptType { ENET_MII_WRITE = ENET_MAC_MDIO_ADDRESS_GOC_0_MASK,
ENET_MII_READ_INCREMENT = ENET_MAC_MDIO_ADDRESS_GOC_1_MASK, ENET_MII_READ =
ENET_MAC_MDIO_ADDRESS_GOC_1_MASK | ENET_MAC_MDIO_ADDRESS_GOC_0_MASK }
```

Mdio operation type.

```
enum ENET_AttributeType { ENET_TX_ATTRIBUTES_CRC_PAD_CTRL = 0x00000001u,
ENET_TX_ATTRIBUTES_SA_INSERTION = 0x00000002u, ENET_TX_ATTRIBUTES_TCP_SEGMENTATION =
0x00000004u, ENET_TX_ATTRIBUTES_CHECKSUM_INSERTION = 0x00000008u,
ENET_TX_ATTRIBUTES_VLAN_TAG_CTRL = 0x00000010u,
ENET_TX_ATTRIBUTES_INNER_VLAN_TAG_CTRL = 0x00000020u,
ENET_TX_ATTRIBUTES_TIMESTAMP_SNAPSHOT = 0x80000000u,
ENET_TX_ATTRIBUTES_TIMESTAMP_VALID = 0x80000000u, ENET_RX_ATTRIBUTES_TIMESTAMP_VALID
= 0x80000000u }
```

Descriptor TX and RX attributes type.

```
enum ENET_TxDescCrcPadType { ENET_CRC_AND_PAD_INSERTION = 0u, ENET_CRC_INSERTION = 1u,
ENET_CRC_INSERTION_DISABLE = 2u, ENET_CRC_REPLACEMENT = 3u }
```

TX descriptor CRC and Pad insertion control type.

```
enum ENET_TxDescChecksumType { ENET_CHECKSUM_INSERTION_DISABLE = 0u,
ENET_CHECKSUM_INSERTION_IP = 1u, ENET_CHECKSUM_INSERTION_PROTO_NO_PSEUDOH = 2u,
ENET_CHECKSUM_INSERTION_PROTO_PSEUDOH = 3u }
```

TX descriptor Checksum insertion control type.

```
enum ENET_FilterEnableType { ENET_FILTER_DISABLE = 0u, ENET_FILTER_ENABLE = 1u,
ENET_FILTER_INVERSE_ENABLE = 2u }
```

ENET filter enable type.

```
enum ENET_VLANFilterModeType { ENET_VLAN_FILTER_OUTER_CVLAN = 0u,  
ENET_VLAN_FILTER_OUTER_SVLAN = 1u, ENET_VLAN_FILTER_INNER_CVLAN = 2u,  
ENET_VLAN_FILTER_INNER_SVLAN = 3u, ENET_VLAN_FILTER_OUTER_ANYTYPE = 4u,  
ENET_VLAN_FILTER_INNER_ANYTYPE = 5u }
```

ENET filter mode type.

```
enum ENET_QueueClassificType { ENET_CLISSIFICATION_EXPRESS = 0u,  
ENET_CLISSIFICATION_PREEMPTABLE = 1u }
```

ENET Frame Preemption Classification type.

Functions

```
void Enet_DefaultConfig (ENET_ConfigType *pConfig)
```

Get ENET default configuration.

```
void Enet_DeInit (ENET_HandleType *pEnetHandle)
```

Reset ENET driver.

```
ENET_StatusType Enet_Init (ENET_HandleType *pEnetHandle, const ENET_ConfigType *pConfig)
```

Initialize ENET driver handle and peripheral.

```
ENET_StatusType Enet_Start (ENET_HandleType *pEnetHandle)
```

Start ENET transmission and reception process in polling mode.

```
ENET_StatusType Enet_Stop (ENET_HandleType *pEnetHandle)
```

Stop ENET transmission and reception process in polling mode.

```
ENET_StatusType Enet_StartIT (ENET_HandleType *pEnetHandle)
```

Start ENET transmission and reception in interrupt mode.

```
ENET_StatusType Enet_StopIT (ENET_HandleType *pEnetHandle)
```

Stop ENET transmission and reception in interrupt mode.

```
ENET_StatusType Enet_SendFrame (ENET_HandleType *pEnetHandle, ENET_ChannelType eChannel,  
ENET_BufferType *pBuffers, ENET_TxOptionType *pOption)
```

ENET transmit frame.

ENET_StatusType Enet_ReadFrame (**ENET_HandleType** *pEnetHandle, **ENET_ChannelType** eChannel, **ENET_BufferType** *pBuffers, **ENET_RxInfoType** *pInfo)

Read frame from ENET.

ENET_StatusType Enet_BuildRxDescriptors (**ENET_HandleType** *pEnetHandle, **ENET_ChannelType** eChannel)

Build RX Descriptors to DMA.

void **Enet_SetMacAddress** (**ENET_HandleType** *pEnetHandle, const uint8_t *pMacAddr, uint8_t u8MacLen)

Set MAC0 address.

void **Enet_CheckSumErrorFilter** (**ENET_HandleType** *pEnetHandle, **ENET_ChannelType** eChannel, bool bEnable)

Enable TCPIP checksum error filter.

void **Enet_SetMacHashTable** (**ENET_HandleType** *pEnetHandle, bool bAddOrRemove, const uint8_t *pMacAddr, uint8_t u8MacLen)

Set hash table for MAC hash filter.

void **Enet_DefaultMACFilter** (**ENET_MACFilterType** *pMACFilter)

Get default MAC filter configurations.

void **Enet_GetMACFilter** (const **ENET_HandleType** *pEnetHandle, **ENET_MACFilterType** *pMACFilter)

Get current MAC filter configurations.

void **Enet_SetMACFilter** (**ENET_HandleType** *pEnetHandle, const **ENET_MACFilterType** *pMACFilter)

Set MAC filter configurations.

void **Enet_DefaultL3L4Filter** (**ENET_L3L4FilterType** *pL3L4Filter)

Get default layer3 layer4 filter configurations.

void **Enet_GetL3L4Filter** (const **ENET_HandleType** *pEnetHandle, **ENET_MAC_L3L4FilterGroupType** pFilterGroup, **ENET_L3L4FilterType** *pL3L4Filter)

Get current layer3 layer4 filter configurations.

```
void Enet_SetL3L4Filter (ENET_HandleType *pEnetHandle, ENET_MAC_L3L4FilterGroupType  
pFilterGroup, const ENET_L3L4FilterType *pL3L4Filter)
```

Set layer3 layer4 filter configurations.

```
void Enet_EnableL3L4Filter (ENET_HandleType *pEnetHandle, bool bEnable)
```

Enable MAC layer3 layer4 filter.

```
void Enet_GetVlanFilterConfig (const ENET_HandleType *pEnetHandle, ENET_VlanFilterConfigType  
*pVlanFilterConfig)
```

Get MAC VLAN filter configurations.

```
void Enet_EnableVlanFilter (ENET_HandleType *pEnetHandle, const ENET_VlanFilterConfigType  
*pVlanFilterConfig)
```

Enable MAC VLAN filter.

```
void Enet_SetVlanHashTable (ENET_HandleType *pEnetHandle, bool bAddOrRemove, uint16_t  
u16VlanTag)
```

Set vlan hash table.

```
ENET_StatusType Enet_GetVlanPerfectFilter (ENET_HandleType *pEnetHandle,  
ENET_MAC_VLANFilterGroupType pFilterGroup, ENET_VlanPerfectFilterType *pPerfectFilter)
```

Get current VLAN perfect filter configurations.

```
ENET_StatusType Enet_SetVlanPerfectFilter (ENET_HandleType *pEnetHandle,  
ENET_MAC_VLANFilterGroupType pFilterGroup, const ENET_VlanPerfectFilterType *pPerfectFilter)
```

Set VLAN perfect filter configurations.

```
void Enet_SetTxOuterVlanConfig (ENET_HandleType *pEnetHandle, const ENET_TxVlanConfigType  
*pVlanConfig)
```

Set outer VLAN insertion replacement or deletion.

ENET_StatusType Enet_SetTxOuterVlanChannelBasedInsertion (ENET_HandleType *pEnetHandle, const ENET_OuterVlanCBTIType *pVlanConfig)

Set outer VLAN channel based insertion.

void Enet_SetTxInnerVlanConfig (ENET_HandleType *pEnetHandle, const ENET_TxVlanConfigType *pVlanConfig)

Set inner VLAN insertion replacement or deletion.

void Enet_RegisterSystemEventCallback (ENET_HandleType *pEnetHandle, ENET_EventCallbackType eCallback)

Register ENET system event callback function.

void Enet_RegisterRxCompleteCallback (ENET_HandleType *pEnetHandle, ENET_ChannelCallbackType eCallback)

Register receive frame callback function.

void Enet_RegisterTxCompleteCallback (ENET_HandleType *pEnetHandle, ENET_ChannelCallbackType eCallback)

Register send frame complete callback function.

void Enet_RegisterWaitTxCompleteCallback (ENET_HandleType *pEnetHandle, ENET_ChannelCallbackType eCallback)

Register wait send frame complete callback function.

void Enet_MDIODefaultConfig (ENET_MiiConfigType *pMiiConfig)

Get MDIO bus default configuration.

ENET_StatusType Enet_MDIOInit (ENET_HandleType *pEnetHandle, const ENET_MiiConfigType *pMiiConfig)

Initialize the MDIO bus.

ENET_StatusType Enet_MDIORead (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8PhyReg, uint16_t *pData, uint32_t u32TimeoutUs)

Read the PHY register using clause 22.

ENET_StatusType Enet_MDIOWrite (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8PhyReg, uint16_t u16Data, uint32_t u32TimeoutUs)

Write the PHY register using clause 22.

ENET_StatusType Enet_MMRead (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8Mmd, uint16_t u16PhyReg, uint16_t *pData, uint32_t u32TimeoutUs)

Read the PHY register using clause 45.

ENET_StatusType Enet_MMWrite (ENET_HandleType *pEnetHandle, uint8_t u8PhyAddr, uint8_t u8Mmd, uint16_t u16PhyReg, uint16_t u16Data, uint32_t u32TimeoutUs)

Write the PHY register using clause 45.

void **Enet_PTPDefaultConfig** (Enet_PTPConfigType *pPTPConfig)

Get PTP default configuration.

ENET_StatusType Enet_PTPStart (ENET_HandleType *pEnetHandle, const Enet_PTPConfigType *pPTPConfig)

Start PTP counter process.

void **Enet_PTPStop** (ENET_HandleType *pEnetHandle)

Stop PTP counter process.

ENET_StatusType Enet_PTPGetTime (const ENET_HandleType *pEnetHandle, ENET_TimestampType *pTime)

Get PTP current system time.

ENET_StatusType Enet_PTPSetTime (ENET_HandleType *pEnetHandle, const ENET_TimestampType *pTime)

Set PTP system time.

ENET_StatusType Enet_PTPUpdateTime (ENET_HandleType *pEnetHandle, const ENET_TimestampType *pTime, bool bSubtraction)

Update PTP system time offset.

ENET_StatusType Enet_PTPAdjustFrequency (ENET_HandleType *pEnetHandle, int PPb)

Adjust the PTP counter frequency.

**ENET_StatusType Enet_PTPFixedPPSOut (ENET_HandleType *pEnetHandle,
ENET_MAC_PPSSChannelType ePPSChannel, ENET_MAC_PPSFreqCtrlType eFreqCtrl)**

Enable PTP channel PPS out in fixed mode.

**ENET_StatusType Enet_PTPFlexiblePPSOut (ENET_HandleType *pEnetHandle,
ENET_MAC_PPSSChannelType ePPSChannel, const ENET_FlexPPSConfigType *PPSConfig)**

Enable PTP channel PPS out in flexible mode.

**ENET_StatusType Enet_TSN_GetEstConfig (ENET_HandleType *pEnetHandle, ENET_EstConfigType
*pEstConfig)**

Get EST configurations.

**ENET_StatusType Enet_TSN_EstConfig (ENET_HandleType *pEnetHandle, const ENET_EstConfigType
*pEstConfig)**

Set EST configurations.

ENET_StatusType Enet_TSN_EstEnable (ENET_HandleType *pEnetHandle, bool bEnable)

Enable EST processing.

**ENET_StatusType Enet_TSN_GetEstGclConfig (ENET_HandleType *pEnetHandle,
ENET_GateCtrlListType *pGatelist)**

Read the current gate control list.

**ENET_StatusType Enet_TSN_EstGclConfig (ENET_HandleType *pEnetHandle, const
ENET_GateCtrlListType *pGatelist)**

Install new gate control list.

**ENET_StatusType Enet_TSN_PreemptionConfig (ENET_HandleType *pEnetHandle, const
ENET_PreemptionConfigType *pConfig)**

Enable or disable the frame preemption function.

void **Enet_GetMMCCConfig** (const **ENET_HandleType** *pEnetHandle, **ENET_MMCCConfigType** *pMMCCConfig)
ENET get MAC management counters configuration.

void **Enet_SetMMCCConfig** (**ENET_HandleType** *pEnetHandle, const **ENET_MMCCConfigType** *pMMCCConfig)
ENET set MAC management counters configuration.

void **Enet_MMCCounterPreset** (**ENET_HandleType** *pEnetHandle)
Preset MAC management counters.

void **Enet_MMCCounterFreeze** (**ENET_HandleType** *pEnetHandle, bool bFreeze)
Freeze MMC counters to their current value.

void **Enet_MMCCounterReset** (**ENET_HandleType** *pEnetHandle)
Reset MAC management counters to zero.

uint32_t **Enet_GetMMCTxInterrupts** (const **ENET_HandleType** *pEnetHandle)
Get MMC transmit counters interrupt status.

uint32_t **Enet_GetMMCRxInterrupts** (const **ENET_HandleType** *pEnetHandle)
Get MMC receive counters interrupt status.

uint32_t **Enet_GetMMCTxCounter** (const **ENET_HandleType** *pEnetHandle, **Enet_MMCTxCounterType** eCounter)
Get ENET MAC management transmit counters.

uint32_t **Enet_GetMMCRxCounter** (const **ENET_HandleType** *pEnetHandle, **Enet_MMCRxCounterType** eCounter)
Get ENET MAC management receive counters.

void **Enet_GetFlowControlConfig** (const **ENET_HandleType** *pEnetHandle, **ENET_FlowControlType** *pFlowControl)
Get flow control configuration.

void **Enet_FlowControlConfig** (**ENET_HandleType** *pEnetHandle, const **ENET_FlowControlType** *pFlowControl)

Configure and enable flow control.

void **Enet_FlowControlSoftwareTrigger** (**ENET_HandleType** *pEnetHandle)

Generate and transmits a single pause packet.

void **Enet_SystemIrqHandler** (**ENET_HandleType** *pEnetHandle)

ENET driver system interrupt handler.

void **Enet_TxIrqHandler** (**ENET_HandleType** *pEnetHandle, **ENET_ChannelType** eChannel)

ENET driver DMA TX channel interrupt handler.

void **Enet_RxIrqHandler** (**ENET_HandleType** *pEnetHandle, **ENET_ChannelType** eChannel)

ENET driver DMA RX channel interrupt handler.

Detailed Description

ENET driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-10-12

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_erm.h File Reference

ERM Module driver type definition and API.

#include "HwA_erm.h"

Include dependency graph for module_driver_erm.h:

IMAGE

Classes

struct **_ERM_HandleType**

The structure of the ERM processing handle.

struct **ERM_CfgType**

The structure of the ERM Init configuration.

Macros

ERM API Service IDs#define **ERM_INIT_ID** 0x00U

#define **ERM_DEINIT_ID** 0x01U

#define **ERM_COMMONIRQHANDLER_ID** 0x02U

ERM Dev Error Code*Error Code of calling ERM apis*

#define **ERM_E_PARAM_NULLPTR** 0x01U

#define **ERM_E_PARAM_INSTANCE** 0x02U

#define **ERM_NON_CORRECTABLE_IRQ** 1U

ERM error interrupt define.

#define **ERM_SINGLE_CORRECTION_IRQ** 2U

Typedefs

typedef struct **_ERM_HandleType** **ERM_HandleType**

The structure of the ERM processing handle.

Enumerations

enum **ERM_InstanceType** { **ERM_INSTANCE_0** = 0U }

ERM Instance type.

```
enum ERM_StatusType { ERM_STATUS_SUCCESS = 0U, ERM_STATUS_FAIL = 1U }
```

ERM operation return values.

Functions

```
ERM_StatusType ERM_Init (ERM_HandleType *const pHandle, const ERM_CfgType *const pInitCfg)
```

Initializes the Error Reporting Module (ERM) for the specified instance.

```
ERM_StatusType ERM_DeInit (ERM_HandleType *const pHandle)
```

Deinitializes the ERM module.

```
void ERM_CommonIRQHandler (ERM_HandleType *const pHandle)
```

Handles common Error Reporting Module (ERM) interrupts and invokes error callbacks.

Detailed Description

ERM Module driver type definition and API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_fciic.h File Reference

FCIIC driver type definition and API.

```
#include "HwA_fciic.h"
```

```
#include "module_driver_dma.h"
```

Include dependency graph for module_driver_fciic.h:

IMAGE

Classes

struct **FCIIC_TxDataType**

Structure for holding transmit data for FCIIC operations.

struct **FCIIC_RxDataType**

Structure for holding received data from FCIIC operations.

struct **FCIIC_MasterConfigType**

Structure for configuring the FCIIC Master.

struct **FCIIC_SlaveConfigType**

Configuration structure for initializing the FCIIC (Fast Communication Interface IC) Slave module.

struct **FCIIC_MasterTransferType**

Structure for defining a master transfer in the FCIIC.

struct **FCIIC_SlaveTransferType**

Structure for defining a slave transfer in the FCIIC.

struct **_FCIIC_DmaConfig**

Structure for configuring DMA for I2C Master and Slave operations.

struct **_FCIIC_MasterHandle**

*Structure for handling the FCIIC Master operations. Implements : **_FCIIC_MasterHandle**.*

struct **_FCIIC_SlaveHandle**

*Structure for handling the FCIIC Slave operations. Implements : **_FCIIC_SlaveHandle**.*

struct **FCIIC_InterruptType**

Structure for configuring FCIIC interrupts.

Macros

```
#define FCIIC_E_PARAM_NULLPTR 0x01U
```

Error code indicating a null pointer parameter.

#define **FCIIC_E_PARAM_INSTANCE** 0x02U

Error code indicating an invalid instance parameter.

#define **FCIIC_E_PARAM_CHANNEL** 0x03U

Error code indicating an invalid channel parameter.

#define **FCIIC_DIR_MASK** (0x1U)

Mask to extract the direction bit from the status register.

#define **FCIIC_MASTER_CMD_SIZE** (0x8U)

Defines the size of the master command in bytes.

FCIIC API Service IDs#define **FCIIC_MASTERCHECKFORBUSYBUS_ID** 0x00

#define **FCIIC_MASTERGETDEFAULTCONFIG_ID** 0x01

#define **FCIIC_MASTERSETBAUDRATE_ID** 0x02

#define **FCIIC_MASTERINIT_ID** 0x03

#define **FCIIC_MASTERDEINIT_ID** 0x04

#define **FCIIC_MASTERINITHANDLE_ID** 0x05

#define **FCIIC_MASTERENABLEINTERRUPT_ID** 0x06

#define **FCIIC_MASTERCHECKANDCLEARERROR_ID** 0x07

#define **FCIIC_MASTERWAITFLAG_ID** 0x08

#define **FCIIC_MASTERSTART_ID** 0x09

#define **FCIIC_MASTERREPEATEDSTART_ID** 0x0a

#define **FCIIC_MASTERSENDSTARTCOMMAND_ID** 0x0b

#define **FCIIC_MASTERSEND_ID** 0x0c

#define **FCIIC_MASTERRECEIVE_ID** 0x0d

#define **FCIIC_MASTERSTOP_ID** 0x0e

#define **FCIIC_MASTERTRANSFERABORT_ID** 0x0f

#define **FCIIC_MASTERSETRECEIVELEN_ID** 0x10

#define **FCIIC_MASTERTRANSMIT_ID** 0x11

#define **FCIIC_MASTERGETSTATUS_ID** 0x12

#define **FCIIC_MASTERCLRSTATUS_ID** 0x13

#define **FCIIC_MASTERTRANSFERBLOCKING_ID** 0x14

#define **FCIIC_MASTERDMAPREPARE_ID** 0x15

#define **FCIIC_MASTERDMASTOP_ID** 0x16

```

#define FCIIC_MASTERDMATRANSMIT_ID 0x17
#define FCIIC_MASTERDMARECEIVE_ID 0x18
#define FCIIC_MASTERDMASTART_ID 0x19
#define FCIIC_MASTERDMASTAGENOTIFICATION_ID 0x1a
#define FCIIC_MASTERDMASTAGEDATA_ID 0x1b
#define FCIIC_LL_MASTERDMAIRQNHANDLER_ID 0x1c
#define FCIIC_MASTERDMATRANSFER_ID 0x1d
#define FCIIC_MASTERINTERRUPTPROCESSSTOP_ID 0x1e
#define FCIIC_MASTERINTERRUPTPROCESSTRANSMIT_ID 0x1f
#define FCIIC_MASTERINTERRUPTPROCESSRECEIVE_ID 0x20
#define FCIIC_MASTERINTERRUPTPROCESSSTART_ID 0x21
#define FCIIC_LL_MASTERIRQNHANDLER_ID 0x22
#define FCIIC_MASTERGENERATESTARTCOMMAND_ID 0x23
#define FCIIC_MASTERTRANSFERNONBLOCKING_ID 0x24
#define FCIIC_SLAVEINIT_ID 0x25
#define FCIIC_SLAVELISTENING_ID 0x26
#define FCIIC_SLAVEDEINIT_ID 0x27
#define FCIIC_SLAVEDEINITHANDLE_ID 0x28
#define FCIIC_SLAVEACK_ID 0x29
#define FCIIC_SLAVECLRSTATUS_ID 0x2a
#define FCIIC_SLAVEGETSTATUS_ID 0x2b
#define FCIIC_SLAVEENABLEINTERRUPT_ID 0x2c
#define FCIIC_SLAVECHECKFORBUSYBUS_ID 0x2d
#define FCIIC_SLAVECHECKANDCLEARERROR_ID 0x2e
#define FCIIC_SLAVEGETDEFAULTCONFIG_ID 0x2f
#define FCIIC_SLAVERECEIVE_ID 0x30
#define FCIIC_SLAVEGETERROR_ID 0x31
#define FCIIC_SLAVECLRERROR_ID 0x32
#define FCIIC_SLAVEGETDIR_ID 0x33
#define FCIIC_SLAVEINTERRUPTPROCESSADDRESSMATCH_ID 0x34
#define FCIIC_SLAVEINTERRUPTPROCESSRECEIVE_ID 0x35
#define FCIIC_SLAVEINTERRUPTPROCESSTRANSMIT_ID 0x36
#define FCIIC_SLAVEINTERRUPTPROCESSSTOP_ID 0x37
#define FCIIC_LL_SLAVEIRQNHANDLER_ID 0x38
#define FCIIC_SLAVETRANSFERNONBLOCKING_ID 0x39
#define FCIIC_SLAVEDMAPREPREARE_ID 0x3a
#define FCIIC_SLAVETRANSFERDMA_ID 0x3b
#define FCIIC_SLAVEDMASTAGENOTIFICATION_ID 0x3c

```

```
#define FCIIC_SLAVESTARTDMA_ID 0x3d
#define FCIIC_LL_SLAVEDMAIRQHANDLER_ID 0x3e
```

Typedefs

```
typedef struct _FCIIC_DmaConfig FCIIC_DmaConfigType
```

Structure for configuring DMA for I2C Master and Slave operations.

```
typedef struct _FCIIC_MasterHandle FCIIC_MasterHandleType
```

*Structure for handling the FCIIC Master operations. Implements : **_FCIIC_MasterHandle**.*

```
typedef struct _FCIIC_SlaveHandle FCIIC_SlaveHandleType
```

*Structure for handling the FCIIC Slave operations. Implements : **_FCIIC_SlaveHandle**.*

```
typedef void(* FCIIC_MasterTransfer_CallbackType) (struct _FCIIC_MasterHandle *pFciicMasterHandle,
FCIIC_StatusType Status)
```

Callback function type for master receive operations.

```
typedef void(* FCIIC_MasterError_CallBackType) (struct _FCIIC_MasterHandle *pFciicMasterHandle,
FCIIC_StatusType Status)
```

Callback function type for master error operations.

```
typedef void(* FCIIC_SlaveTransfer_CallbackType) (struct _FCIIC_SlaveHandle *pFciicSlaveHandle)
```

Callback function type for slave receive operations.

```
typedef void(* FCIIC_SlaveError_CallBackType) (struct _FCIIC_SlaveHandle *pFciicSlaveHandle)
```

Callback function type for slave error operations.

```
typedef void(* FCIIC_RxInterrupt_CallBackType) (struct _FCIIC_SlaveHandle *pFciicSlaveHandle,
FCIIC_StatusType status)
```

Callback function type for receive interrupt operations.

Enumerations

```
enum FCIIC_InstanceType { FCIIC_INSTANCE_0 = 0U, FCIIC_INSTANCE_1 = 1U }
```

Enumeration for selecting the FCIIC instance.

```
enum FCIIC_StatusType { FCIIC_SUCCESS = 0, FCIIC_BUSY, FCIIC_IDLE, FCIIC_NAK, FCIIC_FIFOERROR,  
FCIIC_BITERROR, FCIIC_TRANSMITORRECEIVEERROR, FCIIC_REPEATEDSTARTERROR,  
FCIIC_ADDRESSVALID, FCIIC_ARBITRATIONLOST, FCIIC_PINLOWTIMEOUT,  
FCIIC_NOTTRANSFERINPROGRESS, FCIIC_DMAREQUESTFAIL, FCIIC_TIMEOUT, FCIIC_DIRECTIONERROR,  
FCIIC_STATEMACHINEERROR, FCIIC_OVERFLOWERROR, FCIIC_SLAVEDMANUMBERERROR, FCIIC_FAIL }
```

Enumerates the possible return status codes for FCIIC (Fast Communication Interface IC) operations.

```
enum FCIIC_MasterPinConfigType { FCIIC_OPENDRAIN = 0x0U, FCIIC_PUSHPULL = 0x1U }
```

Specifies the pin configuration options for the FCIIC Master.

```
enum FCIIC_DirectionType { FCIIC_WRITE = 0U, FCIIC_READ = 1U, FCIIC_DIRNOCONFIG = 2U }
```

Specifies the direction of data transfer for the FCIIC Master.

```
enum FCIIC_DmaStatusType { FCIIC_DMA_NO_CONFIG = 0U, FCIIC_DMA_NEXT_STAGE_IDLE = 1U,  
FCIIC_DMA_NEXT_STAGE_DATA = 2U, FCIIC_DMA_NEXT_STAGE_STOP = 3U,  
FCIIC_DMA_NEXT_STAGE_NOTIFICATION = 4U }
```

Specifies the status of the DMA (Direct Memory Access) for FCIIC operations.

```
enum FCIIC_ChannelStatusType { FCIIC_CH_IDLE = 0U, FCIIC_CH_LISTENING_READ,  
FCIIC_CH_LISTENING, FCIIC_CH_START_READ, FCIIC_CH_START_WRITE, FCIIC_CH_SENDING,  
FCIIC_CH_RECEIVING, FCIIC_CH_TRANSFER_FINISHED, FCIIC_CH_ERROR_PRESENT,  
FCIIC_CH_CLOSED }
```

Specifies the status of the FCIIC channel.

Functions

```
FCIIC_StatusType FCIIC_MasterCheckForBusyBus (FCIIC_MasterHandleType *const pFciicMasterHandle)
```

Checks if the I2C bus is busy for the master.

```
void FCIIC_MasterGetDefaultConfig (FCIIC_MasterConfigType *const pMasterConfig)
```

Initializes a default configuration for the I2C master.

```
void FCIIC_MasterSetBaudRate (FCIIC_MasterHandleType *const pFciicMasterHandle, uint32_t  
sourceClock_Hz, uint32_t baudRate_Hz)
```

Sets the baud rate for the I2C master.

```
void FCIIC_MasterInit (FCIIC_MasterHandleType *const pFciicMasterHandle, const  
FCIIC_MasterConfigType *const pMasterConfig, uint32_t sourceClock_Hz)
```

Initializes the I2C master with the specified configuration.

void **FCIIC_MasterDeinit** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Deinitializes the I2C master.

void **FCIIC_MasterInitHandle** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, **FCIIC_InstanceType** eInstance)

Initializes the I2C master handle.

void **FCIIC_MasterEnableInterrupt** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, **FCIIC_MasterTransfer_CallbackType** pCallback, **FCIIC_MasterError_CallBackType** pErrorCallback, void *pUserData)

Enables interrupt handling for the I2C master.

FCIIC_StatusType **FCIIC_MasterCheckAndClearError** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint32_t status)

Checks and clears errors in the I2C master.

FCIIC_StatusType **FCIIC_MasterWaitFlag** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint32_t u32Flag)

Waits for a specific flag to be set in the I2C master.

FCIIC_StatusType **FCIIC_MasterStart** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint8_t address, **FCIIC_DirectionType** dir)

Initiates an I2C start condition.

FCIIC_StatusType **FCIIC_MasterRepeatedStart** (**FCIIC_MasterHandleType** *const pFciicMasterHandle, uint8_t address, **FCIIC_DirectionType** dir)

Initiates an I2C repeated start condition.

FCIIC_StatusType **FCIIC_MasterSendStartCommand** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Sends a sequence of commands in the I2C master.

FCIIC_StatusType FCIIC_MasterSend (FCIIC_MasterHandleType *const pFciicMasterHandle, uint8 *txBuff, uint16 txSize)

Sends data over the I2C bus.

FCIIC_StatusType FCIIC_MasterReceive (FCIIC_MasterHandleType *const pFciicMasterHandle)

Receives data over the I2C bus.

FCIIC_StatusType FCIIC_MasterStop (FCIIC_MasterHandleType *const pFciicMasterHandle)

Sends a stop condition over the I2C bus.

void FCIIC_MasterTransferAbort (FCIIC_MasterHandleType *const pFciicMasterHandle)

Aborts an ongoing I2C transfer.

uint8_t FCIIC_MasterSetReceiveLen (FCIIC_MasterHandleType *const pFciicMasterHandle, uint8 u8Len)

Sets the receive length for an ongoing I2C transfer.

uint8_t FCIIC_MasterTransmit (FCIIC_MasterHandleType *const pFciicMasterHandle, FCIIC_TxDataType *pTxData)

Transmits data over the I2C bus.

uint8_t FCIIC_MasterGetStatus (FCIIC_MasterHandleType *const pFciicMasterHandle, FCIIC_MasterStatusType eStatus)

Gets the status of the I2C master.

void FCIIC_MasterClrStatus (FCIIC_MasterHandleType *const pFciicMasterHandle)

Clears the status flags of the I2C master.

FCIIC_StatusType FCIIC_MasterTransferBlocking (FCIIC_MasterHandleType *const pFciicMasterHandle, FCIIC_MasterTransferType *const transfer)

Performs a blocking I2C transfer.

void FCIIC_MasterDmaPrepare (FCIIC_MasterHandleType *const pFciicMasterHandle)

Prepares the DMA channel for I2C master transfer.

void **FCIIC_MasterDmaStop** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Stops the DMA transfer for the I2C master.

void **FCIIC_MasterDmaTransmit** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Initiates a DMA transmit operation for the I2C master.

void **FCIIC_MasterDmaReceive** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Initiates a DMA receive operation for the I2C master.

void **FCIIC_MasterDmaStart** (**FCIIC_MasterHandleType** *const pFciicMasterHandle,
FCIIC_MasterTransferType *const transfer)

Starts the DMA transfer for the FCIIC master.

void **FCIIC_MasterDmaStageNotification** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Handles the notification stage after a DMA transmission.

void **FCIIC_MasterDmaStageData** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Handles the data stage of a DMA transmission.

void **FCIIC_LL_MasterDmaIRQnHandler** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

DMA interrupt handler for the FCIIC master.

FCIIC_StatusType **FCIIC_MasterDmaTransfer** (**FCIIC_MasterHandleType** *const pFciicMasterHandle,
FCIIC_MasterTransferType *const transfer)

Initiates a DMA transfer for the FCIIC master.

void **FCIIC_MasterInterruptProcessStop** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Stops the interrupt process for the FCIIC master.

void **FCIIC_MasterInterruptProcessTransmit** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Processes the transmit phase in interrupt mode for the FCIIC master.

void **FCIIC_MasterInterruptProcessReceive** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Processes the receive phase in interrupt mode for the FCIIC master.

void **FCIIC_MasterInterruptProcessStart** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Processes the start phase in interrupt mode for the FCIIC master.

void **FCIIC_LL_MasterIRQnHandler** (**FCIIC_MasterHandleType** *const pFciicMasterHandle)

Handles the interrupt for the FCIIC master.

void **FCIIC_MasterGenerateStartCommand** (**FCIIC_MasterHandleType** *const pFciicMasterHandle,
FCIIC_MasterTransferType *const transfer)

Generates a start command for the FCIIC Master.

FCIIC_StatusType **FCIIC_MasterTransferNonBlocking** (**FCIIC_MasterHandleType** *const
pFciicMasterHandle, **FCIIC_MasterTransferType** *const transfer)

Initiates a non-blocking I2C Master transfer.

void **FCIIC_SlaveInit** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle, const **FCIIC_SlaveConfigType**
*pInitReg)

Initializes the I2C Slave mode.

FCIIC_StatusType **FCIIC_Slavelistening** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle,
FCIIC_SlaveTransferType *const transfer)

De-initializes the IIC Slave mode.

void **FCIIC_SlaveDeInit** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Deinitializes the FCIIC slave configuration.

void **FCIIC_SlaveDeinitHandle** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Deinitializes the FCIIC slave handle.

void **FCIIC_SlaveAck** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle, boolean bAck)

Sends an ACK/NACK signal in IIC Slave mode.

void **FCIIC_SlaveClrStatus** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Clears the I2C master status flag state.

uint32_t **FCIIC_SlaveGetStatus** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Gets the slave SSR status.

void **FCIIC_SlaveEnableInterrupt** (FCIIC_SlaveHandleType *const pFciicSlaveHandle,
FCIIC_SlaveTransfer_CallbackType pCallback, **FCIIC_SlaveError_CallBackType** pErrorCallBack, void
*pUserData)

Enables the interrupt for the specified FCIIC Slave instance.

FCIIC_StatusType **FCIIC_SlaveCheckForBusyBus** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Checks if the FCIIC bus is busy.

FCIIC_StatusType **FCIIC_SlaveCheckAndClearError** (FCIIC_SlaveHandleType *const pFciicSlaveHandle,
uint32_t status)

Checks for and clears I2C Slave errors.

void **FCIIC_SlaveGetDefaultConfig** (FCIIC_SlaveConfigType *const slaveConfig, boolean bUsedDMA)

Sets the default configuration for the I2C Slave.

uint8_t **FCIIC_SlaveReceive** (FCIIC_SlaveHandleType *const pFciicSlaveHandle, uint8_t *pData)

Receives data in IIC Slave mode.

uint32_t **FCIIC_SlaveGetError** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Gets the slave error value.

void **FCIIC_SlaveClrError** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Clears the slave error value.

FCIIC_DirectionType **FCIIC_SlaveGetDir** (FCIIC_SlaveHandleType *const pFciicSlaveHandle)

Gets the current direction of the slave.

void **FCIIC_SlaveInterruptProcessAddressMatch** (FCIIC_SlaveHandleType *const pFciicSlaveHandle,
uint32_t u32IsrStatus)

Processes the IIC Slave AMOF interrupt.

void **FCIIC_SlaveInterruptProcessReceive** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Processes the IIC Slave RDF interrupt.

void **FCIIC_SlaveInterruptProcessTransmit** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Processes IIC Slave TDF interrupt.

void **FCIIC_SlaveInterruptProcessStop** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Processes IIC Slave SDF interrupt.

void **FCIIC_LL_SlaveIRQnHandler** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

I2C Slave Interrupt Service Routine (ISR);.

FCIIC_StatusType **FCIIC_SlaveTransferNonBlocking** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Starts accepting slave transfers in non-blocking mode.

void **FCIIC_SlaveDmaPrepare** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Prepares DMA configuration for IIC Slave.

FCIIC_StatusType **FCIIC_SlaveTransferDMA** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Initiates an IIC Slave Asynchronous DMA transfer.

void **FCIIC_SlaveDMAStageNotification** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Starts DMA for IIC Slave in asynchronous mode.

void **FCIIC_SlaveStartDMA** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

Starts DMA for IIC Slave in asynchronous mode.

void **FCIIC_LL_SlaveDmaIRQnHandler** (**FCIIC_SlaveHandleType** *const pFciicSlaveHandle)

DMA handler for IIC Slave.

Detailed Description

FCIIC driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

module_driver_fcpit.h File Reference

FCPIT driver type definition and API.

```
#include "HwA_fcpit.h"
```

Include dependency graph for module_driver_fcpit.h:

IMAGE

Classes

struct **FCPIT_TriggerType**

struct **FCPIT_CommonInitType**

struct **_FCPIT_HandleType**

struct **FCPIT_IntType**

Fcpit interrupt structure.

struct **FCPIT_ChannelInitType**

Fcpit initialization type.

Macros

FCPIT API Service IDs#define **FCPIT_COMMON_INIT_ID** 0U

#define **FCPIT_CHANNEL_INIT_ID** 1U

#define **FCPIT_INIT_TRIGGER_ID** 2U

#define **FCPIT_DE_INIT_ID** 3U

#define **FCPIT_INIT_INTERRUPT_ID** 4U

#define **FCPIT_ENABLE_INTERRUPT_ID** 5U

#define **FCPIT_DISABLE_INTERRUPT_ID** 6U

#define **FCPIT_START_ID** 7U

#define **FCPIT_STOP_ID** 8U

#define **FCPIT_UPDATEVALUE_ID** 9U

#define **FCPIT_UPDATEVALUE_IMMEDIATE_ID** 10U

FCPIT Dev Error Code*Error Code of calling FCPIT apis*

#define **FCPIT_STATUS_FUNCTION_ERROR** 0x0U

#define **FCPIT_E_PARAM_CHANNEL** 0x02U

#define **FCPIT_E_PARAM_POINTER** 0x03U

#define **FCPIT_E_PARAM_COUNT** 0x04U

#define **FCPIT_E_PARAM_INVALID** 0x05U

Typedefs

```
typedef void(* FCPIT_InterruptCallbackType) (void)  
callback function type
```

```
typedef struct _FCPIT_HandleType FCPIT_HandleType
```

Enumerations

```
enum FCPIT_InstanceType { FCPIT_INSTANCE_0, FCPIT_INSTANCE_1, FCPIT_INSTANCE_MAX }  
FCPIT instance.
```

```
enum FCPIT_TriggerSelectType { FCPIT_TRIGGER_INTERNAL_0 = 0, FCPIT_TRIGGER_INTERNAL_1,  
FCPIT_TRIGGER_INTERNAL_2, FCPIT_TRIGGER_INTERNAL_3, FCPIT_TRIGGER_EXTERNAL,  
FCPIT_NO_TRIGGER }  
Fcpit trigger source.
```

Functions

```
void FCPIT_CommonInit (FCPIT_HandleType *pFcpitHandle, const FCPIT_CommonInitType *pInitStruct)  
Initialize Fcpit instance common feature.
```

```
void FCPIT_ChannelInit (FCPIT_HandleType *pFcpitHandle, const FCPIT_ChannelInitType *pInitStruct)  
Initialize Fcpit instance.
```

```
void FCPIT_InitTrigger (FCPIT_HandleType *pFcpitHandle, const FCPIT_TriggerType *pTrgStruct)  
Initialize Fcpit trigger configuration.
```

```
void FCPIT_Deinit (FCPIT_HandleType *pFcpitHandle)  
De-initialize Fcpit instance.
```

```
void FCPIT_InitInterrupt (FCPIT_HandleType *pFcpitHandle, const FCPIT_IntType *pIntStruct)  
Initialize Fcpit interrupt functionality.
```

```
void FCPIT_EnableInterrupt (FCPIT_HandleType *pFcpitHandle, const FCPIT_ChannelType eChannel)  
Enable Fcpit interrupt.
```

```
void FCPIT_DisableInterrupt (FCPIT_HandleType *pFcpitHandle, const FCPIT_ChannelType eChannel)
```

Disable Fcpit interrupt.

void **FCPIT_Start** (**FCPIT_HandleType** *pFcpitHandle, const **FCPIT_ChannelType** eChannel)

Fcpit start timer.

void **FCPIT_Stop** (**FCPIT_HandleType** *pFcpitHandle, const **FCPIT_ChannelType** eChannel)

Fcpit stop.

void **FCPIT_UpdateChannelValue** (**FCPIT_HandleType** *pFcpitHandle, const **FCPIT_ChannelType** eChannel, const uint32_t u32ChannelValue)

Update Fcpit channel value.

void **FCPIT_ImmediateUpdateChannelValue** (**FCPIT_HandleType** *pFcpitHandle, const **FCPIT_ChannelType** eChannel, const uint32_t u32ChannelValue)

Immediately update Fcpit channel value.

void **FCPITn_IRQHandler** (**FCPIT_HandleType** *pFcpitHandle)

fcpit interrupt function.

Detailed Description

FCPIT driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_fcsmu.h File Reference

FCSMU module driver type definition and API.

#include "HwA_fcsmu.h"

Include dependency graph for module_driver_fcsmu.h:

IMAGE

Classes

struct **FCSMU_InitCfgType**

FCSMU init configure structure.

struct **FCSMU_StatusOutputCfgType**

FCSMU status output pin configure structure.

struct **FCSMU_CrcCfgType**

FCSMU CRC configure structure.

struct **_FCSMU_HandleType**

The structure of the FCSMU processing handle.

Macros

FCSMU API Service IDs#define **FCSMU_INIT_ID** 0x00U

#define **FCSMU_DEINIT_ID** 0x01U

#define **FCSMU_STATUSOUTPUTINIT_ID** 0x02U

#define **FCSMU_STATUSOUTPUTDEINIT_ID** 0x03U

#define **FCSMU_GETFAULTCHANNEL_ID** 0x04U

#define **FCSMU_CLEARFAULT_ID** 0x05U

#define **FCSMU_INJECTIONFAULT_ID** 0x06U

#define **FCSMU_CRCINIT_ID** 0x07U

#define **FCSMU_CRCDEINIT_ID** 0x08U

#define **FCSMU_CRCGEN_ID** 0x09U

#define **FCSMU_CRCISBUSY_ID** 0x0AU

#define **FCSMU_CRCCLARBUSY_ID** 0x0BU

#define **FCSMU_CRCGETRESULT_ID** 0x0CU

#define **FCSMU_COMMONIRQHANDLER_ID** 0x0DU

FCSMU Dev Error Code*Error Code of calling FCSMU apis*

#define **FCSMU_E_PARAM_NULLPTR** 0x01U

#define FCSMU_E_PARAM_INSTANCE 0x02U

#define FCSMU_OP_STATE_IDLE 0U

FCSMU operation status define.

#define FCSMU_OP_STATE_BUSY 1U

#define FCSMU_OP_STATE_FAILED 2U

#define FCSMU_OP_STATE_SUCCESSFUL 3U

#define FCSMU_OP_CODE_NONE 0U

FCSMU operation code define.

#define FCSMU_OP_CODE_MOVE_TO_CONFIG 1U

#define FCSMU_OP_CODE_MOVE_TO_NORMAL 2U

#define FCSMU_OP_CODE_CLEAR_FAULT_INFO 13U

#define FCSMU_OP_CODE_CLEAR_OPS_TO_IDLE 15U

#define FCSMU_SOUT_CTRL_BY_FSM 0U

FCSMU SOUT Control define.

#define FCSMU_SOUT_CTRL_KEEP_LOW 1U

#define FCSMU_SOUT_CTRL_BY_FSM2 2U

#define FCSMU_SOUT_CTRL_KEEP_HIGH_THEN_FSM 3U

#define FCSMU_SOUT_DEFAULT_POLARITY 0U

FCSMU SOUT Polarity Switch define.

#define FCSMU_SOUT_SWITCH_POLARITY 1U

#define FCSMU_SOUT_PROTOCOL_DUAL_RAIL 0U

FCSMU SOUT Protocol define.

#define FCSMU_SOUT_PROTOCOL_TIME_SWITCH 1U

#define FCSMU_SOUT_PROTOCOL_BISTABLE 2U

```
#define FCSMU_SOUT_PROTOCOL_FAULT_TOGGLE 3U
#define FCSMU_SOUT_PROTOCOL_TIME_DUAL_RAIL 4U
#define FCSMU_SOUT_PROTOCOL_DIAG0 5U
#define FCSMU_SOUT_PROTOCOL_DIAG1 6U
#define FCSMU_SOUT_PROTOCOL_DIAG2 7U
```

```
#define FCSMU_STATE_NORMAL 0U
FCSMU state in state register define.
```

```
#define FCSMU_STATE_CONGIG 1U
#define FCSMU_STATE_WARN 2U
#define FCSMU_STATE_FAULT 3U
```

```
#define FCSMU_FAULT_TOLERANCE_TIME_1MS 0U
FCSMU fault tolerance time define.
```

```
#define FCSMU_FAULT_TOLERANCE_TIME_2MS 1U
#define FCSMU_FAULT_TOLERANCE_TIME_5MS 2U
#define FCSMU_FAULT_TOLERANCE_TIME_10MS 3U
```

```
#define FCSMU_CRC_STATE_IDLE 0U
FCSMU crc Status define.
```

```
#define FCSMU_CRC_STATE_BUSY 1U
```

```
#define FCSMU_CRC_SW_MODE 0U
FCSMU crc mode define.
```

```
#define FCSMU_CRC_TRIGGER_MODE 1U
```

```
#define FCSMU_CRC_ERROR_REQ_MODE_NONE 0U
FCSMU crc error request mode define.
```

```
#define FCSMU_CRC_ERROR_REQ_MODE_IRQ 1U
```

```
#define FCSMU_CRC_ERROR_REQ_MODE_RST 2U
```

Typedefs

```
typedef struct _FCSMU_HandleType FCSMU_HandleType
```

The structure of the FCSMU processing handle.

Enumerations

```
enum FCSMU_InstanceType { FCSMU_INSTANCE_0 = 0U }
```

FCSMU Instance type.

```
enum FCSMU_StatusType { FCSMU_STATUS_SUCCESS = 0U, FCSMU_STATUS_FAIL = 1U,  
FCSMU_STATUS_TIMEOUT = 2U }
```

FCSMU operation return values.

Functions

```
FCSMU_StatusType FCSMU_Init (FCSMU_HandleType *const pHandle, const FCSMU_InitCfgType *const  
pInitCfg)
```

Initializes the FCSMU module.

```
FCSMU_StatusType FCSMU_Deinit (FCSMU_HandleType *const pHandle)
```

Deinitializes the FCSMU module.

```
FCSMU_StatusType FCSMU_StatusOutputInit (FCSMU_HandleType *const pHandle, const  
FCSMU_StatusOutputCfgType *const pInitCfg)
```

Initializes the status output configuration for the FCSMU module.

```
FCSMU_StatusType FCSMU_StatusOutputDeinit (FCSMU_HandleType *const pHandle)
```

Deinitializes the status output configuration for the FCSMU module.

```
uint32_t FCSMU_GetFaultChannel (FCSMU_HandleType *const pHandle)
```

Retrieves the fault channel for the specified FCSMU module.

```
FCSMU_StatusType FCSMU_ClearFaultFlag (FCSMU_HandleType *const pHandle, uint32_t  
u32FaultChannel)
```

Clears the fault flag for the specified fault channel in the FCSMU module.

FCSMU_StatusType FCSMU_InjectionFault (FCSMU_HandleType *const pHandle, uint32_t u32ChannelIndex)

Injects a fault into the specified channel in the FCSMU module.

FCSMU_StatusType FCSMU_CrcInit (FCSMU_HandleType *const pHandle, const FCSMU_CrcCfgType *const pInitCfg)

Initializes the CRC checker for the FCSMU module.

FCSMU_StatusType FCSMU_CrcDeinit (FCSMU_HandleType *const pHandle)

Deinitializes the CRC checker for the FCSMU module.

FCSMU_StatusType FCSMU_CrcGen (FCSMU_HandleType *const pHandle)

Generates a CRC for the specified FCSMU module.

bool FCSMU_CrcIsBusy (FCSMU_HandleType *const pHandle)

Checks if the CRC generation process is busy for the specified FCSMU module.

uint32_t FCSMU_CrcGetResult (FCSMU_HandleType *const pHandle)

Retrieves the result of the CRC calculation for the specified FCSMU module.

void FCSMU_CommonIRQHandler (FCSMU_HandleType *const pHandle)

Handles common interrupts for the FCSMU module.

Detailed Description

FCSMU module driver type definition and API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_fcspi.h File Reference

ADC driver type definition and API.

```
#include "HwA_fcspi.h"
```

```
#include "module_driver_dma.h"
```

Include dependency graph for module_driver_fcspi.h:

IMAGE

Classes

struct **_FCSPI_HandleType**

The structure of the FCSPI processing handle.

Macros

```
#define FCSPI_DEV_ERROR_REPORT STD_ON  
FCSPI API Service IDs#define FCSPI_INIT_ID 0U  
#define FCSPI_DEINIT_ID 1U  
#define FCSPI_TRANSMIT_ID 3U  
#define FCSPI_RECEIVE_ID 4U  
#define FCSPI_TRANSMITRECEIVE_ID 5U  
#define FCSPI_TRANSMIT_IT_ID 6U  
#define FCSPI_RECEIVE_IT_ID 7U  
#define FCSPI_TRANSMITRECEIVE_IT_ID 8U  
#define FCSPI_TRANSMIT_DMA_ID 9U  
#define FCSPI_RECEIVE_DMA_ID 10U  
#define FCSPI_TRANSMITRECEIVE_DMA_ID 11U
```

FCSPI Dev Error Code*Error Code of calling FCSPI apis*

```
#define FCSPI_E_PARAM_UNINIT 0x01U /* API service called without module initialization */  
#define FCSPI_E_PARAM_POINTER 0x02U /* API service called with an unexpected value for the  
pointer */  
#define FCSPI_E_PARAM_CONFIG 0x03U /* API service called with unreasonable configuration */
```

Typedefs

typedef struct **_FCSPI_HandleType** **FCSPI_HandleType**

The structure of the FCSPI processing handle.

Enumerations

enum **FCSPI_StateType** { **FCSPI_STATE_UNINIT** = 0x00U, **FCSPI_STATE_IDLE** = 0x01U,
FCSPI_STATE_BUSY = 0x02U, **FCSPI_STATE_BUSY_TX** = 0x03U, **FCSPI_STATE_BUSY_RX** = 0x04U,
FCSPI_STATE_BUSY_TX_RX = 0x05U, **FCSPI_STATE_ERROR** = 0x06U, **FCSPI_STATE_ABORT** = 0x07U }

FCSPI states.

enum **FCSPI_StatusType** { **FCSPI_STATUS_SUCCESS** = 0x0U, **FCSPI_STATUS_ERROR** = 0x1U,
FCSPI_STATUS_TIMEOUT = 0x2U }

Return value of FCSPI API services.

enum **FCSPI_DataAccessWidthType** { **FCSPI_DATA_ACCESS_8_BIT** = 0x0U, **FCSPI_DATA_ACCESS_16_BIT**
= 0x1U, **FCSPI_DATA_ACCESS_32_BIT** = 0x2U }

Data access width according to differen frame size user configured.

enum **FCSPI_PcsModeType** { **FCSPI_PCS_HARDWARE_SEL** = 0x0U, **FCSPI_PCS_GPIO_SEL** = 0x1U,
FCSPI_PCS_NONE = 0x2U }

Configuration item: The user chooses what strategy to use for the chip selection signal(PCS).

enum **FCSPI_SckFeature** { **FCSPI_SCK_FEATURE_PRECISE_FREQ** = 0x0,
FCSPI_SCK_FEATURE_50_DUTY_CYCLE = 0x1 }

Functions

uint32_t **FCSPI_LL_CaculateSckdivPrescale** (uint32_t *Prescale, uint32_t *Sckdiv, uint32_t InputClkFreq,
uint32_t TargetSckFreq)

FCSPI_StatusType **FCSPI_Init** (**FCSPI_HandleType** *pFcspiHandle)

Initialize FCSPI based on configurations in FCSPI_HandleType.

void **FCSPI_DeInit** (**FCSPI_HandleType** *pFcspiHandle)

De-init FCSPI peripheral and 'tStatus' of FCSPI_HandleType structure.

FCSPI_StatusType **FCSPI_Transmit** (**FCSPI_HandleType** *pFcspiHandle, uint8_t *pData, uint32_t Size)

Transmit data in blocking(polling) mode.

FCSPI_StatusType **FCSPI_Receive** (**FCSPI_HandleType** *pFcspiHandle, uint8_t *pRxData, uint32_t Size)

Receive data in blocking(polling) mode.

FCSPI_StatusType FCSPI_TransmitReceive (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint8_t *pRxData, uint32_t Size)

Transmit and receive data in blocking(polling) mode.

FCSPI_StatusType FCSPI_Transmit_IT (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint32_t Size)

Transmit data in interrupt mode(non-blocking).

FCSPI_StatusType FCSPI_Receive_IT (FCSPI_HandleType *pFcspiHandle, uint8_t *pRxData, uint32_t Size)

Receive data in interrupt mode(non-blocking).

FCSPI_StatusType FCSPI_TransmitReceive_IT (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint8_t *pRxData, uint32_t Size)

Transmit and receive data in interrupt mode(non-blocking).

FCSPI_StatusType FCSPI_Transmit_DMA (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint32_t Size)

Transmit data in DMA mode(non-blocking).

FCSPI_StatusType FCSPI_Receive_DMA (FCSPI_HandleType *pFcspiHandle, uint8_t *pRxData, uint32_t Size)

Receive data in DMA mode(non-blocking).

FCSPI_StatusType FCSPI_TransmitReceive_DMA (FCSPI_HandleType *pFcspiHandle, uint8_t *pTxData, uint8_t *pRxData, uint32_t Size)

Transmit and receive data in DMA mode(non-blocking).

void FCSPIIn_IRQHandler (FCSPI_HandleType *pFcspiHandle)

The internal interrupt handler function for FCSPI instances.

void FCSPIIn_DmaDoneCallback_Tx (void *pFcspiHandle)

FCSPI callback of DMA channel triggered by Tx FIFO.

void FCSPIIn_DmaDoneCallback_Rx (void *pFcspiHandle)

FCSPI callback of DMA channel triggered by Rx FIFO.

Detailed Description

ADC driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-12-22

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_fcuart.h File Reference

FCUART driver type definition and API.

```
#include "HwA_fcuart.h"
```

Include dependency graph for module_driver_fcuart.h:

IMAGE

Classes

struct **FCUART_DataType**

UART transmit and receive data define.

struct **_FCUART_HandleType**

The structure of the UART processing handle.

struct **FCUART_InterruptType**

UART callback functions data define.

struct **FCUART_InitType**

UART initial data define.

struct **FCUART_WakeupType**

UART WakeUp functions data define.

Macros

```
#define FCUART_FIFO_DEPTH 16U
```

FCUART API Service IDs *Defines the service IDs for the FCUART module, which are used to identify different functions or services provided by the FCUART layer. These IDs are utilized when communicating with the FCUART module, enabling calls to or responses from various functions.*

```
#define FCUART_INIT_ID 0U
```

```
#define FCUART_DEINIT_ID 1U
```

```
#define FCUART_SET_INTERRUPT_ID 2U
```

```
#define FCUART_SET_WAKEUP_ID 3U
```

```
#define FCUART_GET_INTERRUPT_ID 4U
```

```
#define FCUART_START_RECEIVE_ID 5U
```

```
#define FCUART_ASSIGN_TXINTRRUPT_ID 6U
```

```
#define FCUART_START_TRANSMIT_ID 7U
```

```
#define FCUART_TRANSMIT_ID 8U
```

```
#define FCUART_PRINTF_ID 9U
```

```

#define FCUART_GET_SATTUS_ID 10U
#define FCUART_RECEIVE_POLLING_ID 11U
#define FCUART_ERROR_POLLING_ID 12U
#define FCUART_SET_LOOPMODE_ID 13U
#define FCUART_STOP_RECEIVE_ID 14U
#define FCUART_STOP_TRANSMIT_ID 15U
#define FCUART_SENDDATA_9BITS 16U
#define FCUART_GETDATA_9BITS 17U
#define FCUART_SENDDATA_10BITS 18U
#define FCUART_GETDATA_10BITS 19U
#define FCUART_SETBITMODEANDPARITY 20U

```

FCUART Dev Error Code *Error Code of calling FCUART apis*

```

#define FCUART_E_PARAM_INSTANCE 0x01U
#define FCUART_E_PARAM_ERROR_STATE 0x02U
#define FCUART_E_PARAM_POINTER 0x03U

```

Special_Characters Special Characters *Special characters used in UART communication.*

```

#define ESCAPE_CHARACTER (char)0x5c
#define ENTER (uint8_t)0x0d
#define NEW_LINE (uint8_t)0x0a
#define SPACE (uint8_t)0x20
#define FLOAT_ZERO 0.00000001

```

Typedefs

```
typedef struct _FCUART_HandleType FCUART_HandleType
```

The structure of the UART processing handle.

```
typedef void(* FCUART_ErrorInterrupt_CallbackType) (FCUART_HandleType *pUartHandle, uint32_t u32Error)
```

FCUART error interrupt callback function type.

```
typedef void(* FCUART_TxRxInterrupt_CallbackType) (FCUART_HandleType *pUartHandle, FCUART_DataType *pTxRxCfg)
```

FCUART transmit/receive interrupt callback function type.

```
typedef void(* FCUART_IdleInterrupt_CallbackType) (FCUART_HandleType *pUartHandle)
```

FCUART idle interrupt callback function type.

typedef uint8_t **FCUART_ErrorType**

Error type Define as uint8.

Enumerations

enum **UART_PrintfIntType** { **UART_PRINT_RADIX_BIN** = 2U, **UART_PRINT_RADIX_OCT** = 8U,
UART_PRINT_RADIX_DEC = 10U, **UART_PRINT_RADIX_HEX** = 16U }

Define the type of integer representation for UART printing.

enum **FCUART_SequenceType** { **FCUART_SEQUENCE_VAR_NOINIT**, **FCUART_SEQUENCE_DEINIT**,
FCUART_SEQUENCE_NOTSTART_RECEIVE, **FCUART_SEQUENCE_START_RECEIVE** }

FCUART Operation Sequence.

enum **FCUART_ErrorTypeEnum** { **FCUART_ERROR_OK** = 0x00U, **FCUART_ERROR_INVALID_VERSION** =
0x01U, **FCUART_ERROR_FAILED** = 0x02U, **FCUART_ERROR_INVALID_PARAM** = 0x04U,
FCUART_ERROR_INVALID_SIZE = 0x08U, **FCUART_ERROR_INVALID_SEQUENCE** = 0x10U,
FCUART_ERROR_TIMEOUT = 0x20U }

FCUART error status.

enum **FCUART_Fifo_RxIdleCharNumType** { **FCUART_FIFO_RX_IDLE_DISABLE** = 0U,
FCUART_FIFO_RX_IDLE_CHARACTER_1, **FCUART_FIFO_RX_IDLE_CHARACTER_2**,
FCUART_FIFO_RX_IDLE_CHARACTER_4, **FCUART_FIFO_RX_IDLE_CHARACTER_8**,
FCUART_FIFO_RX_IDLE_CHARACTER_16, **FCUART_FIFO_RX_IDLE_CHARACTER_32**,
FCUART_FIFO_RX_IDLE_CHARACTER_64 }

UART fifo receive idle character number.

enum **FCUART_InterruptSel** { **FCUART_INT_CTRL_ORIE** = **FCUART_CTRL_ORIE_MASK**,
FCUART_INT_CTRL_NEIE = **FCUART_CTRL_NEIE_MASK**, **FCUART_INT_CTRL_FEIE** =
FCUART_CTRL_FEIE_MASK, **FCUART_INT_CTRL_PEIE** = **FCUART_CTRL_PEIE_MASK**,
FCUART_INT_CTRL_TIE = **FCUART_CTRL_TIE_MASK**, **FCUART_INT_CTRL_TCIE** =
FCUART_CTRL_TCIE_MASK, **FCUART_INT_CTRL_RIE** = **FCUART_CTRL_RIE_MASK**, **FCUART_INT_CTRL_IIE** =
FCUART_CTRL_IIE_MASK, **FCUART_INT_CTRL_TE** = **FCUART_CTRL_TE_MASK**, **FCUART_INT_CTRL_RE** =
FCUART_CTRL_RE_MASK, **FCUART_INT_CTRL_MOIE** = **FCUART_CTRL_MOIE_MASK**,
FCUART_INT_CTRL_M1IE = **FCUART_CTRL_M1IE_MASK** }

UART CTRL register interrupt.

enum **FCUART_ErrorStatusType** { **FCUART_ERROR_NONE** = 0x0000U, **FCUART_ERROR_RORF** = 0x0001U,
FCUART_ERROR_NF = 0x0002U, **FCUART_ERROR_FEF** = 0x0004U, **FCUART_ERROR_PEF** = 0x0008U,
FCUART_ERROR_RPEF = 0x0010U, **FCUART_ERROR_TPEF** = 0x0020U }

UART error status type.

enum **FCUART_IdleCharNumType** { **FCUART_IDLE_CHARACTER_1** = 0U, **FCUART_IDLE_CHARACTER_2**,
FCUART_IDLE_CHARACTER_4, **FCUART_IDLE_CHARACTER_8**, **FCUART_IDLE_CHARACTER_16**,
FCUART_IDLE_CHARACTER_32, **FCUART_IDLE_CHARACTER_64**, **FCUART_IDLE_CHARACTER_128** }

UART idle character number.

enum **FCUART_IdleStartType** { **FCUART_START_AFTER_STARTBIT** = 0U,
FCUART_START_AFTER_STOPBIT }

UART idle type.

Functions

void **FCUART_InitStructure** (**FCUART_HandleType** *pUartHandle)

Initial UART variables Memory.

FCUART_ErrorType **FCUART_Init** (**FCUART_HandleType** *pUartHandle, **FCUART_InitType** *pInitCfg)

This Function is used to initial UART instance.

void **FCUART_DeInit** (**FCUART_HandleType** *pUartHandle)

This Function is used to de-initial UART instance.

void **FCUART_SetWakeup** (**FCUART_HandleType** *pUartHandle, **FCUART_WakeupType** *pWakeupCfg)

This Function is used to set UART WakeUp.

void **FCUART_StartReceive** (**FCUART_HandleType** *pUartHandle)

This Function is used to start receiving.

void **FCUART_StopReceive** (**FCUART_HandleType** *pUartHandle)

This Function is used to stop receiving.

void **FCUART_StartTransmit** (**FCUART_HandleType** *pUartHandle)

This Function is used to start transmit.

void **FCUART_StopTransmit** (**FCUART_HandleType** *pUartHandle)

This Function is used to stop transmitting.

FCUART_ErrorType FCUART_Transmit (FCUART_HandleType *pUartHandle, FCUART_DataType *pUartData)

This Function is used to transmit UART data.

FCUART_ErrorType FCUART_Printf (FCUART_HandleType *pUartHandle, char *fmt,...)

This Function is used to print ASCII char from UART.

uint32_t FCUART_GetStatus (FCUART_HandleType *pUartHandle, FCUART_StatType eStatusType)

Get Stat Flag.

FCUART_ErrorType FCUART_Receive_Polling (FCUART_HandleType *pUartHandle, FCUART_DataType *pRxMsg)

This Function is used to receive data when polling (not used when rx interrupt enabled)

void FCUART_Error_Polling (FCUART_HandleType *pUartHandle, uint32_t *pErrorValue)

This Function is used to get error when polling (not used when error interrupt enabled)

void FCUART_SetLoopMode (FCUART_HandleType *pUartHandle, bool bStatus)

This Function is used to enable fcuart loop mode.

void FCUARTN_RxTx_IRQHandler (FCUART_HandleType *pUartHandle)

This Function is used to deal with FCUART TxRx interrupt.

void FCUART_SendData_9Bits (FCUART_HandleType *pUartHandle, uint16_t u16Data)

This Function is used to send 9 bits data.

void FCUART_GetData_9Bits (FCUART_HandleType *pUartHandle, uint16_t *pData)

This Function is used to Get 9 bits data.

void FCUART_SendData_10Bits (FCUART_HandleType *pUartHandle, uint16_t u16Data)

This Function is used to send 10 bits data.

void FCUART_GetData_10Bits (FCUART_HandleType *pUartHandle, uint16_t *pData)

This Function is used to Get 10 bits data.

void **FCUART_SetBitModeAndParity** (FCUART_HandleType *pUartHandle, **FCUART_BitModeType** eBitMode, **FCUART_StopBitNumType** eStopBit, **FCUART_ParityType** eParityType, bool bParityEnable)
This Function is used to set bit mode,parity and stop bit.

bool **FCUART_GetInterruptMode** (FCUART_HandleType *pUartHandle, uint32_t u32Data)
This Function is used to Get current interrupt mode.

void **FCUART_AssignTxInterruptData** (FCUART_HandleType *pUartHandle, uint8_t *pData, uint32_t u32Length)
This Function is used to assign data to send through interrupt.

Detailed Description

FCUart driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_flash.h File Reference

Flash driver type definition and API.

#include "device_header.h"

Include dependency graph for module_driver_flash.h:

IMAGE

Classes

struct **_FLASH_HandleType**

The structure of the FLASH processing handle.

struct **FLASH_DRIVER_ParamType**

Flash driver parameter define.

struct **FLASH_Lock_ParamType**

unlock parameter define

Macros

FLASH API Service IDs#define **FLASH_SYNC_ERASE_ID** 0U

#define **FLASH_SYNC_WRITE_ID** 1U

#define **FLASH_ASYNC_ERASE_ID** 2U

#define **FLASH_ASYNC_WRITE_ID** 3U

#define **FLASH_INTERRUPT_ERASE_ID** 4U

#define **FLASH_INTERRUPT_WRITE_ID** 5U

#define **NVR_SYNC_ERASE_ID** 6U

#define **NVR_SYNC_WRITE_ID** 7U

FLASH Dev Error Code#define **FLASH_E_PARAM_POINTER** 0x01U

Typedefs

typedef enum **fls_Ret** **Fls_Status_t**

typedef enum **Async_Return** **Async_Return_t**

FLASH async prog/erase status.

typedef enum **Interrupt_Return** **Fls_ReturnType**

FLASH interrupt prog/erase status.

typedef struct **_FLASH_HandleType** **FLASH_HandleType**

The structure of the FLASH processing handle.

Enumerations

enum **FLASH_InstanceType** { **FLASH_INSTANCE_0** = 0U }

FLASH instance.

enum **fls_Ret** { **FLS_IDLE** = 0, **FLS_ERASING** = 1, **FLS_WRITING** = 2, **FLS_READING** = 3, **FLS_ERASE_ERROR** = 4, **FLS_WRITE_ERROR** = 5, **FLS_PARA_ERROR** = 6, **FLS_DMA_WRITING** = 7 }

enum **FLASH_StatusType** { **FLASH_ERROR_OK** = 0x00U, **FLASH_ERROR_NO_INIT**, **FLASH_ERROR_INVALID_VERSION**, **FLASH_ERROR_FAILED**, **FLASH_ERROR_INVALID_PARAM**, **FLASH_ERROR_INVALID_ADDR**, **FLASH_ERROR_INVALID_SIZE**, **FLASH_ERROR_INVALID_SEQUENCE**, **FLASH_ERROR_TIMEOUT**, **FLASH_ERROR_END** = 0xFFFFFFFFU }

FLASH API error status.

enum **ECC_StatusType** { **FLASH_ECC_ERROR_NONE**, **FLASH_ECC_ERROR_SBC**, **FLASH_ECC_ERROR_DBC** }

FLASH ECC error status.

enum **Async_Return** { **FLASH_ASYNC_ERR_OK** = 0, **FLASH_ASYNC_ERR_NOT_OK** = 1, **FLASH_ASYNC_ERR_BUSY** = 2 }

FLASH async prog/erase status.

enum **Interrupt_Return** { **FLASH_INTERRUPT_ERR_OK** = 0, **FLASH_INTERRUPT_ERR_NOT_OK** = 1, **FLASH_INTERRUPT_ERR_BUSY** = 2 }

FLASH interrupt prog/erase status.

enum **FLASH_LOCK_TYPE** { **FLASH_UNLOCK** = 0x000u, **FLASH_LOCK** = 0x001u }

FMC lock type.

enum **FLASH_TYPE** { **PFlash** = 0x000u, **DFlash** = 0x001u }

Flash type.

Functions

void **FLASHDRIVER_Init** (void)

Initial Flash api address.

FLASH_StatusType **FLASHDRIVER_Lock** (**FLASH_Lock_ParamType** *pUnlockParam)

Flash Driver Function for lock/unlock sector.

FLASH_StatusType FLASHDRIVER_DMAWrite (FLASH_HandleType *pFlashHandle, uint32_t u32DMACount)

Flash Driver Function for DMA Write.

uint32_t FLASHDRIVER_DMAAbort (FLASH_HandleType *pFlashHandle)

Flash Driver Function for DMA Abort.

FLASH_StatusType FLASHDRIVER_SyncErase (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType *pFlashParam)

Flash Driver Function for Erasing.

FLASH_StatusType FLASHDRIVER_SyncWrite (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType *pFlashParam)

Flash Driver Function for Writing.

void FLASHDRIVER_AsyncErase (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType *pFlashParam)

flash driver erase a sector

void FLASHDRIVER_AsyncWrite (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType *pFlashParam)

flash driver write a fixed size

Async_Return_t FLASHDRIVER_AsyncCheckEraseStatus (FLASH_HandleType *pFlashHandle)

flash driver check current status when erasing

Async_Return_t FLASHDRIVER_AsyncCheckWriteStatus (FLASH_HandleType *pFlashHandle)

flash driver check current status when writing

FLASH_StatusType NVRDRIVER_SyncWrite (FLASH_HandleType *pFlashHandle, FLASH_DRIVER_ParamType *pFlashParam)

NVR driver write function warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will become bricked. warning:Need to configure the NVR correctly, otherwise there is a risk that the chip will

Detailed Description

Flash driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_flexcan.h File Reference

FLEXCAN driver type definition and API.

#include "HwA_flexcan.h"

Include dependency graph for module_driver_flexcan.h:

IMAGE

Classes

struct **FLEXCAN_BaudRegType**

Baud-rate clock and divider.

struct **FLEXCAN_ErrorInfoType**

FLEXCAN Error Information.

struct **FLEXCAN_RxMsgType**

FLEXCAN receive message.

struct **FLEXCAN_FifoFilterType**

Receive Message Buffer Configuration.

struct **FLEXCAN_MbSettingType**

Message Buffer Configuration.

struct **FLEXCAN_LegacyFifoConfigType**

FLEXCAN legacy fifo configuration structure definition.

struct **FLEXCAN_EnhancedFifoConfigType**

FLEXCAN enhanced fifo configuration structure definition.

struct **FLEXCAN_FifoConfigType**

FLEXCAN fifo configuration structure definition.

struct **FLEXCAN_MbConfigType**

FLEXCAN mb configuration structure definition.

struct **FLEXCAN_TxMsgType**

FLEXCAN transmit message.

struct **FLEXCAN_RxMsgListType**

FLEXCAN polling receive message.

struct **FLEXCAN_FifoIssueType**

FIFO error response type.

struct **FLEXCAN_PNET_IdSettingType**
CAN ID filter setting.

struct **FLEXCAN_PNET_IDCfgType**
CAN Pretended network id filter setting.

struct **FLEXCAN_PNET_PayloadCfgType**

struct **FLEXCAN_PNET_ConfigType**
CAN Pretended network configuration.

struct **FLEXCAN_WakeupMsgType**
FLEXCAN wake-up receive message.

struct **_FLEXCAN_HandleType**
The structure of the CAN processing handle.

struct **FLEXCAN_InitType**
FLEXCAN init structure definition.

Macros

CAN Detection Error Code *All can det code*

```
#define FLEXCAN_DET_ERROR_ID_FUNC_CHECKINSTANCE 0x01U
#define FLEXCAN_DET_ERROR_ID_FUNC_INIT 0x01U
#define FLEXCAN_DET_ERROR_ID_FUNC_PROCESS 0x02U
#define FLEXCAN_DET_ERROR_ID_FUNC_SETINTERRUPT 0x03U
#define FLEXCAN_DET_ERROR_ID_FUNC_START 0x04U
#define FLEXCAN_DET_ERROR_ID_FUNC_STOP 0x05U
#define FLEXCAN_DET_ERROR_ID_FUNC_TRANSMIT 0x06U
#define FLEXCAN_DET_ERROR_ID_FUNC_RECEIVE 0x07U
```

CAN Return Error Code *All can error code*

```
#define FLEXCAN_E_OK 0x00U
#define FLEXCAN_E_INVALID_VERSION 0x01U
#define FLEXCAN_E_FAILED 0x02U
#define FLEXCAN_E_INVALID_PARAM 0x04U
#define FLEXCAN_E_INVALID_SIZE 0x08U
#define FLEXCAN_E_INVALID_SEQUENCE 0x10U
#define FLEXCAN_E_NULL_POINT 0x20U
#define FLEXCAN_E_TIMEOUT 0x40U
#define FLEXCAN_E_BUSY 0x80U
#define FLEXCAN_E_MB_OUTOFRANGE 0x100U
#define FLEXCAN_E_UNSUPPORT_PARAM 0x200U
```


CAN DET Function *CAN DET Function*

```
#define FLEXCAN_DEV_ERROR_REPORT STD_OFF
#define FLEXCAN_ReportDevError(func, error) ReportDevError(FLEXCAN_MODULE_ID, func, error)
#define FLEXCAN_TX_FLAG_CLEAR_FIRST STD_OFF
#define FLEXCAN_RX_FLAG_CLEAR_FIRST STD_OFF
#define FLEXCAN_ERROR_FLAG_CLEAR_FIRST STD_ON
```

Typedefs

```
typedef uint32_t FLEXCAN_ErrorType
```

FLEXCAN error status.

CAN structure definition *CAN structure definition*

```
typedef struct _FLEXCAN_HandleType FLEXCAN_HandleType
```

The structure of the CAN processing handle.

Enumerations

CAN enumeration definition *CAN enumeration definition*

```
enum FLEXCAN_IdType { FLEXCAN_ID_STD = 0x00U, FLEXCAN_ID_EXT = 0x01U }
```

FLEXCAN ID Type.

```
enum FLEXCAN_DataType { FLEXCAN_FRAME_DATA = 0x00U, FLEXCAN_FRAME_REMOTE = 0x01U }
```

FLEXCAN Data Frame Type.

```
enum FLEXCAN_SequenceType { FLEXCAN_SEQUENCE_DEINIT, FLEXCAN_SEQUENCE_NOTSTART,
FLEXCAN_SEQUENCE_STARTED }
```

CAN Operation Sequence.

```
enum FLEXCAN_PNET_WakeupSrcType { FLEXCAN_PENT_WAKEUP_SRC_TIMEOUT,
FLEXCAN_PENT_WAKEUP_SRC_MATCH }
```

CAN wake-up source.

```
enum FLEXCAN_FifoType { FLEXCAN_LEGACY_FIFO, FLEXCAN_ENHANCED_FIFO }
```

CAN wake-up source.

Functions

CAN api definition *CAN api definition*

```
uint8_t FLEXCAN_GetDataWidthArrayLen (FLEXCAN_HandleType *pCanHandle)
```

Get CAN Data width number for current can MBDSR0..MBDSRn.

void **FLEXCAN_EccInjectionInit** (**FLEXCAN_HandleType** *pCanHandle, uint32_t erro_inject_addr, uint8_t data_pattern_bit)

Initializes Error Correction Code (ECC) injection for FlexCAN module.

void **FLEXCAN_EccInjectionDeInit** (**FLEXCAN_HandleType** *pCanHandle)

Deinitializes Error Correction Code (ECC) injection for FlexCAN module.

FLEXCAN_ErrorType **FLEXCAN_Init** (**FLEXCAN_HandleType** *pCanHandle, const **FLEXCAN_InitType** *const pInitCfg)

Initial CAN.

FLEXCAN_ErrorType **FLEXCAN_DeInit** (**FLEXCAN_HandleType** *pCanHandle)

De-initial can instance.

FLEXCAN_ErrorType **FLEXCAN_Start** (**FLEXCAN_HandleType** *pCanHandle)

Start can instance.

FLEXCAN_ErrorType **FLEXCAN_Stop** (**FLEXCAN_HandleType** *pCanHandle)

Stop can instance.

FLEXCAN_ErrorType **FLEXCAN_TransmitData** (**FLEXCAN_HandleType** *pCanHandle, const **FLEXCAN_TxMsgType** *const pTxMsg)

Transmit data, if tx disable, must call FLEXCAN_TransmitProcess after transmitting.

FLEXCAN_ErrorType **FLEXCAN_TransmitAbort** (**FLEXCAN_HandleType** *pCanHandle, uint8_t u8TxHandle)

Abort transmit with special transmit handler.

uint32_t **FLEXCAN_DataLenToDlc** (uint32_t u32DataLen)

Transfer Data length to DLC.

uint32_t **FLEXCAN_DlcToDataLen** (uint32_t u32Dlc)

Transfer DLC to Data length.

void **FLEXCAN_InterruptProcess** (FLEXCAN_HandleType *pCanHandle)

Can process for interrupt.

void **FLEXCAN_PollingProcess** (FLEXCAN_HandleType *pCanHandle)

Can process for polling.

void **FLEXCAN_InterruptErrorProcess** (FLEXCAN_HandleType *pCanHandle)

Can Error process for interrupt.

void **FLEXCAN_PollingErrorProcess** (FLEXCAN_HandleType *pCanHandle)

Can Error process for polling.

Detailed Description

FLEXCAN driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_freqm.h File Reference

FREQM module driver type definition and API.

```
#include "HwA_freqm.h"
```

Include dependency graph for module_driver_freqm.h:

IMAGE

Classes

struct **_FREQM_HandleType**

The structure of the FREQM processing handle.

struct **FREQM_CfgType**

FREQM configure structure.

Macros

FREQM API Service IDs#define **FREQM_INIT_ID** 0x00U

#define **FREQM_DEINIT_ID** 0x01U

#define **FREQM_STARTMEASURE_ID** 0x02U

#define **FREQM_CLEARSTATUS_ID** 0x03U

#define **FREQM_COMMONIRQHANDLER_ID** 0x04U

FREQM Dev Error Code*Error Code of calling FREQM apis*

#define **FREQM_E_PARAM_NULLPTR** 0x01U

#define **FREQM_E_PARAM_INSTANCE** 0x02U

Typedefs

typedef struct **_FREQM_HandleType** **FREQM_HandleType**

The structure of the FREQM processing handle.

Enumerations

enum **FREQM_InstanceType** { **FREQM_INSTANCE_0** = 0U }

FREQM Instance type.

enum **FREQM_StatusType** { **FREQM_STATUS_SUCCESS** = 0x0U, **FREQM_STATUS_FAIL** = 0x1U, **FREQM_STATUS_TIMEOUT** = 0x2U }

FREQM operation return values.

Functions

FREQM_StatusType FREQM_Init (FREQM_HandleType *const pHandle, const FREQM_CfgType *const pInitStruct)

Initializes the FREQM module with the specified configuration.

FREQM_StatusType FREQM_DeInit (FREQM_HandleType *pHandle)

Deinitializes the FREQM module.

FREQM_StatusType FREQM_StartMeasure (FREQM_HandleType *pHandle)

Starts the frequency measurement process in the FREQM module.

FREQM_StatusType FREQM_ClearStatus (FREQM_HandleType *pHandle)

Clears the status of the FREQM module.

void FREQM_CommonIRQHandler (FREQM_HandleType *const pHandle)

Handles common interrupts for the FREQM module.

Detailed Description

FREQM module driver type definition and API.

Author

Flagchip0100

Version

2.0.0

Date

2024-08-20

Copyright

Copyright 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_ftu.h File Reference

FTU driver type definition and API.

```
#include "HwA_ftu.h"
```

```
#include "stddef.h"
```

```
#include "HwA_smisc.h"
```

Include dependency graph for module_driver_ftu.h:

IMAGE

Classes

struct **FTU_SignalMeasureType**

FTU signal measurement configuration type.

struct **FTU_SignalMeasureValueType**

FTU signal measurement result time.

struct **FTU_ExpectEdgeNumberResultType**

Expect edge number result time.

struct **FTU_EdgeNumberMeasureType**

FTU edge number measurement configuration type.

struct **FTU_ExpectEdgeNumberMeasureType**

FTU expect edge number measurement configuration type.

struct **_FTU_HandleType**

The structure of the FTU processing handle.

struct **FTU_PwmChannelType**

The configuration for the Pwm channel.

struct **FTU_PwmModeType**

The configuration option for the Pwm Mode.

struct **FTU_OutputCompareModeType**

The configuration option for the output compare mode.

struct **FTU_CounterModeType**

The configuration option for the time counter mode.

struct **FTU_PwmDutyUpdateType**

The configuration option for update PWM Duty.

struct **FTU_CommonType**

The basic configuration option for the FTU peripheral.

struct **FTU_FaultInitType**

The configuration option for initializing FTU fault.

struct **FTU_FaultControlType**

The configuration option for enabling a FTU fault.

struct **FTU_InputChannelType**

The configuration option for a input capture channel.

struct **FTU_QuadratureInitType**

The configuration option for the quadrature decoder mode.

Macros

#define **FTU_FAULT_CHANNEL_COUNT** 4u

Max number of Ftu fault channel.

#define **FTU_INPUT_FILTER_COUNT** 4u

Max number of Ftu input count.

#define **FTU_StopGlobalTimeBase** SMISC_HWA_EnabeFtuGTB

Stop the FTU global time base.

#define **FTU_StartGlobalTimeBase** SMISC_HWA_DisabeFtuGTB

Start the FTU global time base.

FTU Channel Bit Flag*Bit of channel indicate channel number*

#define **BIT_FTU_CHANNEL_0** 0x01U

#define **BIT_FTU_CHANNEL_1** 0x02U

#define **BIT_FTU_CHANNEL_2** 0x04U

#define **BIT_FTU_CHANNEL_3** 0x08U

#define **BIT_FTU_CHANNEL_4** 0x10U

#define **BIT_FTU_CHANNEL_5** 0x20U

#define **BIT_FTU_CHANNEL_6** 0x40U

#define **BIT_FTU_CHANNEL_7** 0x80U

FTU API Service IDs#define **FTU_COMMON_INIT_ID** 0U

#define **FTU_DEINIT_ID** 1U

#define **FTU_GET_DEFAULT_INIT_CFG_ID** 2U

```

#define FTU_COUNTER_INIT_ID 3U
#define FTU_PWM_UPDATE_DUTY_ID 4U
#define FTU_PWM_UPDATE_PERIOD_ID 5U
#define FTU_PWM_SYNC_UPDATE_ID 6U
#define FTU_PWM_MODE_INIT_ID 7U
#define FTU_OC_MODE_INIT_ID 8U
#define FTU_GET_CHANNEL_VALUE_ID 9U
#define FTU_GET_COUNTER_ID 10U
#define FTU_SET_CHANNEL_VALUE_ID 11U
#define FTU_ENABLE_TRIG_OUT_ID 12U
#define FTU_DISABLE_TRIG_OUT_ID 13U
#define FTU_START_TIMER_ID 14U
#define FTU_STOP_TIMER_ID 15U
#define FTU_INTR_INIT_ID 16U
#define FTU_FAULT_INIT_ID 17U
#define FTU_FAULT_SEL_DELAY_ID 18U
#define FTU_FAULT_ENABLE_ID 19U
#define FTU_FAULT_DISABLE_ID 20U
#define FTU_IC_CHANNEL_INIT_ID 21U
#define FTU_QD_MODE_INIT_ID 22U
#define FTU_ENABLE_INTR_ID 23U
#define FTU_DISABLE_INTR_ID 24U
#define FTU_CLEAR_FAULT_ID 25U
#define FTU_GET_FAULT_FLAG_ID 26U
#define FTU_ENABLE_CHANNEL_DMA_ID 27U
#define FTU_IC_EXPENM_INIT_ID 28U
#define FTU_IC_ENM_INIT_ID 29U
#define FTU_IC_EXPENM_GET_ID 30U
#define FTU_IC_ENM_GET_ID 31U
#define FTU_IC_SIG_MEAS_INIT_ID 32U
#define FTU_IC_SIG_MEAS_SINGLE_ID 33U
#define FTU_IC_SIG_MEAS_GET_ID 34U
#define FTU_OC_MODE_UPDATE_ID 35U

```

FTU Dev Error Code *Error Code of calling FTU apis*

```

#define FTU_E_PARAM_INSTANCE 0x01U
#define FTU_E_PARAM_CHANNEL 0x02U
#define FTU_E_PARAM_POINTER 0x03U
#define FTU_E_PARAM_COUNT 0x04U

```



```
#define FTU_E_PARAM_INVALID 0x05U
#define FTU_E_PARAM_PHASE_SHIFT 0x06U
#define FTU_E_PARAM_INVALID_CHANNEL 0x07U
```

FTU Fault Bit Flag *Bit of fault flag*

```
#define FTU_FAULT_INPUT_0 0x01U
#define FTU_FAULT_INPUT_1 0x02U
```

Typedefs

```
typedef struct _FTU_HandleType FTU_HandleType
```

The structure of the FTU processing handle.

Enumerations

```
enum FTU_StatusType { FTU_STATUS_SUCCESS = 0U, FTU_STATUS_PARAM_INVALID,
FTU_STATUS_NO_CLOCK_SOURCE }
```

FTU operation return values.

```
enum FTU_InstanceType { FTU_INSTANCE_0 = 0U, FTU_INSTANCE_1, FTU_INSTANCE_2,
FTU_INSTANCE_3, FTU_INSTANCE_4, FTU_INSTANCE_5, FTU_INSTANCE_6, FTU_INSTANCE_7,
FTU_INSTANCE_8, FTU_INSTANCE_9, FTU_INSTANCE_10, FTU_INSTANCE_11 }
```

The instance index of the FTU peripheral.

```
enum FTU_ChannelType { FTU_CHANNEL_0 = 0U, FTU_CHANNEL_1, FTU_CHANNEL_2, FTU_CHANNEL_3,
FTU_CHANNEL_4, FTU_CHANNEL_5, FTU_CHANNEL_6, FTU_CHANNEL_7 }
```

The channel index of the FTU peripheral.

```
enum FTU_ClkSrcType { FTU_NO_CLK = 0U, FTU_INTERNAL_CLK = 1U, FTU_PCC_CLK = 2U,
FTU_EXTERNAL_CLK0 = 3U, FTU_EXTERNAL_CLK1 = 4U, FTU_EXTERNAL_CLK2 = 5U }
```

The clock source of FTU peripheral.

```
enum FTU_OutputComparePinModeType { FTU_OUTPUT_TOGGLE_PIN = FTU_CHANNEL_OC_TOGGLE,
FTU_OUTPUT_CLEAR_PIN = FTU_CHANNEL_OC_CLEAR, FTU_OUTPUT_SET_PIN =
FTU_CHANNEL_OC_SET }
```

The channel operation of output compare mode.

```
enum FTU_OutputCompareInitLevelType { FTU_OUTPUT_CMP_INIT_LOW = 0U,
FTU_OUTPUT_CMP_INIT_HIGH }
```

The initial level of the output compare channel.

```
enum FTU_PwmPinModeType { FTU_PWM_HIGH_TRUE_PULSE = FTU_CHANNEL_PWM_HIGH_TRUE,
FTU_PWM_LOW_TRUE_PULSE = FTU_CHANNEL_PWM_LOW_TRUE }
```

The channel operation of PWM mode.

enum **FTU_PwmAlignedType** { **FTU_EDGE_ALIGNED_PWM** = 0u, **FTU_CENTER_ALIGNED_PWM** }

PWM Aligned Mode.

enum **FTU_MeasureContModeType** { **FTU_MEASURE_SINGLE_MODE** = 0u,
FTU_MEASURE_CONTINUOUS_MODE }

The measurement is single or continuous.

enum **FTU_MeasureStartModeType** { **FTU_MEASURE_START_AFTER_EDGE** = 0u,
FTU_MEASURE_START_IMMEDIATELY }

Timing of start measurement.

enum **FTU_SignalMeasureModeType** { **FTU_SIGNAL_MEASURE_HIGH_TIME** = 0u,
FTU_SIGNAL_MEASURE_LOW_TIME = 1u, **FTU_SIGNAL_MEASURE_PERIOD_RISING_EDGE** = 2u,
FTU_SIGNAL_MEASURE_PERIOD_FALLING_EDGE = 3u }

FTU signal measurement mode type.

enum **FTU_ReloadPointCfgType** { **FTU_RELOAD_POINT_CNTMAX** = 1u, **FTU_RELOAD_POINT_CNTMIN** =
(1u << 1), **FTU_RELOAD_POINT_CHANNEL_0** = (1u << 2), **FTU_RELOAD_POINT_CHANNEL_1** = (1u << 3),
FTU_RELOAD_POINT_CHANNEL_2 = (1u << 4), **FTU_RELOAD_POINT_CHANNEL_3** = (1u << 5),
FTU_RELOAD_POINT_CHANNEL_4 = (1u << 6), **FTU_RELOAD_POINT_CHANNEL_5** = (1u << 7),
FTU_RELOAD_POINT_CHANNEL_6 = (1u << 8), **FTU_RELOAD_POINT_CHANNEL_7** = (1u << 9) }

The reload point flag of FTU peripheral.

enum **FTU_SyncFlagType** { **FTU_SYNC_FLAG_FTUEN** = 1u, **FTU_SYNC_FLAG_CNTINC** = (1u << 1),
FTU_SYNC_FLAG_PWMSYNC = (1u << 2), **FTU_SYNC_FLAG_REINIT** = (1u << 3),
FTU_SYNC_FLAG_SYNCHOM = (1u << 4), **FTU_SYNC_FLAG_SYNCEN01** = (1u << 5),
FTU_SYNC_FLAG_SYNCEN23 = (1u << 6), **FTU_SYNC_FLAG_SYNCEN45** = (1u << 7),
FTU_SYNC_FLAG_SYNCEN67 = (1u << 8), **FTU_SYNC_FLAG_HW_TRIG0** = (1u << 9),
FTU_SYNC_FLAG_HW_TRIG1 = (1u << 10), **FTU_SYNC_FLAG_HW_TRIG2** = (1u << 11) }

The synchronization flag of FTU peripheral.

enum **FTU_IntrMaskType** { **FTU_INTR_MASK_CHANNEL_0** = 1u, **FTU_INTR_MASK_CHANNEL_1** = (1u << 1),
FTU_INTR_MASK_CHANNEL_2 = (1u << 2), **FTU_INTR_MASK_CHANNEL_3** = (1u << 3),
FTU_INTR_MASK_CHANNEL_4 = (1u << 4), **FTU_INTR_MASK_CHANNEL_5** = (1u << 5),
FTU_INTR_MASK_CHANNEL_6 = (1u << 6), **FTU_INTR_MASK_CHANNEL_7** = (1u << 7),
FTU_INTR_MASK_OVERFLOW = (1u << 8), **FTU_INTR_MASK_FAULT** = (1u << 9),
FTU_INTR_MASK_RELOAD_POINT = (1u << 10) }

The interrupt enable/disable mask of FTU peripheral.

enum **FTU_FaultChannelEnableType** { **FTU_FAULT_FOR_CHANNEL01** = 1u,
FTU_FAULT_FOR_CHANNEL23 = (1u << 1), **FTU_FAULT_FOR_CHANNEL45** = (1u << 2),
FTU_FAULT_FOR_CHANNEL67 = (1u << 3) }

The fault control enable/disable mask for channels.

```
enum FTU_FaultPolActiveType { FTU_FAULT_POL_ACTIVE_HIGH = 0u, FTU_FAULT_POL_ACTIVE_LOW = 1u }
```

the fault input polarity

```
enum FTU_TriggerOutputMaskType { FTU_TRIG_OUTPUT_MASK_CHANNEL_0_MATCH = 1u,
FTU_TRIG_OUTPUT_MASK_CHANNEL_1_MATCH = (1u << 1),
FTU_TRIG_OUTPUT_MASK_CHANNEL_2_MATCH = (1u << 2),
FTU_TRIG_OUTPUT_MASK_CHANNEL_3_MATCH = (1u << 3),
FTU_TRIG_OUTPUT_MASK_CHANNEL_4_MATCH = (1u << 4),
FTU_TRIG_OUTPUT_MASK_CHANNEL_5_MATCH = (1u << 5),
FTU_TRIG_OUTPUT_MASK_CHANNEL_6_MATCH = (1u << 6),
FTU_TRIG_OUTPUT_MASK_CHANNEL_7_MATCH = (1u << 7),
FTU_TRIG_OUTPUT_MASK_ALL_CHANNEL_MATCH = (0xFFu), FTU_TRIG_OUTPUT_MASK_RELOAD = (1u << 8) }
```

the output trigger mask

```
enum FTU_InputCapturePinModeType { FTU_INPUT_RISING_EDGE = FTU_CHANNEL_EDGE_RISING,
FTU_INPUT_FALLING_EDGE = FTU_CHANNEL_EDGE_FALLING, FTU_INPUT_BOTH_EDGE =
FTU_CHANNEL_EDGE_BOTH }
```

Ftu input capture mode.

```
enum FTU_GlobalTimeBaseStartInstanceType { FTU_GTB_INSTANCE_START_FTU0 = 1u,
FTU_GTB_INSTANCE_START_FTU1 = (1u << 1), FTU_GTB_INSTANCE_START_FTU2 = (1u << 2),
FTU_GTB_INSTANCE_START_FTU3 = (1u << 3), FTU_GTB_INSTANCE_START_FTU4 = (1u << 4),
FTU_GTB_INSTANCE_START_FTU5 = (1u << 5), FTU_GTB_INSTANCE_START_FTU6 = (1u << 6),
FTU_GTB_INSTANCE_START_FTU7 = (1u << 7) }
```

Ftu Global Time Base instance start mask.

```
enum FTU_GlobalTimeBaseStartType { FTU_GTB_START_AT_ONCE = 1u,
FTU_GTB_START_AT_TSTMP1_MOD0 = (1u << 1), FTU_GTB_START_AT_TSTMP1_MOD1 = (1u << 2),
FTU_GTB_START_AT_TSTMP1_MOD2 = (1u << 3), FTU_GTB_START_AT_TSTMP1_MOD3 = (1u << 4) }
```

Ftu Global Time Base start mask.

```
enum FTU_PwmUpdateModeType { FTU_PWM_UPDATE_IMMEDIATELY,
FTU_PWM_UPDATE_END_PERIOD, FTU_PWM_UPDATE_HALF_PERIOD,
FTU_PWM_UPDATE_BOTH_HALF_AND_END_PERIOD }
```

Functions

```
void FTU_CommonInit (FTU_HandleType *pFtuHandle, const FTU_CommonType *const
pCommonStruct)
```

Initialize FTU basic configuration.

void **FTU_GetDefaultInitCfg** (**FTU_CommonType** *pCommonStruct)

Fills in the FTU configuration structure with the default settings.

void **FTU_CounterModeInit** (**FTU_HandleType** *pFtuHandle, const **FTU_CounterModeType** *const pCounterStruct)

Configure FTU to counter mode.

void **FTU_OutputCompareModeInit** (**FTU_HandleType** *pFtuHandle, const **FTU_OutputCompareModeType** *const pOutputModeStruct)

Configure FTU to output compare mode.

void **FTU_OutputCompareUpdate** (**FTU_HandleType** *pFtuHandle, const **FTU_OutputCompareModeType** *const pOCCConfig)

Update the behavior of Output compare channel.

uint32_t **FTU_GetChannelValue** (**FTU_HandleType** *pFtuHandle, uint32_t u32Channel)

Set the value of the FTU Channel.

uint32_t **FTU_GetCounter** (**FTU_HandleType** *pFtuHandle)

Get the value of the FTU counter.

void **FTU_SetChannelValue** (**FTU_HandleType** *pFtuHandle, uint32_t u32Channel, uint32_t u32Value)

Set the value of the FTU Channel.

void **FTU_PwmModeInit** (**FTU_HandleType** *pFtuHandle, const **FTU_PwmModeType** *const pPwmModeStruct)

Configure Configure FTU to PWM mode.

void **FTU_PwmUpdateDuty** (**FTU_HandleType** *pFtuHandle, **FTU_PwmDutyUpdateType** *pConfig)

Update the duty cycle of the FTU Channel.

void **FTU_PwmUpdatePeriod** (**FTU_HandleType** *pFtuHandle, uint32_t u32Period, bool bUpdate)

Update the Period cycle of the FTU instance.

void **FTU_PwmSyncUpdate** (FTU_HandleType *pFtuHandle)

Synchronous update cycle and period of the FTU instance.

void **FTU_EnableInterrupt** (FTU_HandleType *pFtuHandle, uint32_t u32InterruptMask)

Enable FTU interrupt.

void **FTU_DisableInterrupt** (FTU_HandleType *pFtuHandle, uint32_t u32InterruptMask)

Enable FTU interrupt.

void **FTU_StartTimer** (FTU_HandleType *pFtuHandle)

Start the FTU instance.

void **FTU_StopTimer** (FTU_HandleType *pFtuHandle)

Stop the FTU instance.

void **FTU_DeInit** (FTU_HandleType *pFtuHandle)

De-initialize the FTU instance.

void **FTU_FaultInit** (FTU_HandleType *pFtuHandle, const FTU_FaultInitType *const pFaultInit)

initialize fault input of the selected Ftu instance

void **FTU_FaultEnable** (FTU_HandleType *pFtuHandle, const FTU_FaultControlType *const pFaultCtrl)

Enable a fault input.

void **FTU_FaultSelectDelayValue** (FTU_HandleType *pFtuHandle, uint8_t u8Channel,
FTU_FaultDisableDelayType eDelaySelection)

Select fault disable channel output delay value.

uint8_t **FTU_GetEdgeNumberCount** (FTU_HandleType *pFtuHandle, uint8_t u8Channel)

Get Edge number counter.

void **FTU_EdgeNumberMeasureChannelInit** (FTU_HandleType *pFtuHandle,
FTU_EdgeNumberMeasureType *pEdgeNumMeasure)

Initialize a Edge Number Measurement channel.

void **FTU_SignalMeasureChannelInit** (FTU_HandleType *pFtuHandle, FTU_SignalMeasureType *pMeasure)

Initialize a signal measure channel.

void **FTU_SignalMeasureChannelSingle** (FTU_HandleType *pFtuHandle, uint8_t u8Channel)

Re-start measurement when in single mode.

void **FTU_GetSignalMeasureResult** (const FTU_InstanceType eInstance, uint8_t u8Channel, FTU_SignalMeasureValueType *pResult)

Get the measurement result of the channel.

void **FTU_FaultDisable** (FTU_HandleType *pFtuHandle, uint32_t u32FaultIndex)

Disable a fault input.

void **FTU_ClearFault** (FTU_HandleType *pFtuHandle, uint32_t u32FaultFlag)

Clear the fault flag of the FTU instance.

uint32_t **FTU_GetFaultFlag** (FTU_HandleType *pFtuHandle)

Get the fault flag of the FTU instance.

void **FTU_InputCaptureChannelInit** (FTU_HandleType *pFtuHandle, const FTU_InputChannelType *const pInputChannel)

initialize a input capture channel of the selected Ftu instance

void **FTU_EnableTriggerOutput** (FTU_HandleType *pFtuHandle, uint32_t u32TriggerOutputMask)

Enable the output trigger of the selected FTU instance.

void **FTU_DisableTriggerOutput** (FTU_HandleType *pFtuHandle, uint32_t u32TriggerOutputMask)

Disable the output trigger of the selected FTU instance.

void **FTU_QuadratureModelInit** (FTU_HandleType *pFtuHandle, const FTU_QuadratureInitType *const pQuadInit)

initialize the quadrature decoder mode

void **FTU_EnableChannelDma** (FTU_HandleType *pFtuHandle, uint32_t u32Channel)

Enable ftu channel DMA.

void **FTU_GetExpectEdgeNumberResult** (FTU_HandleType *pFtuHandle, uint8_t u8Channel, FTU_ExpectEdgeNumberResultType *pResult)

Get the expect edge number result of the channel.

void **FTU_ExpectEdgeNumberMeasureChannelInit** (FTU_HandleType *pFtuHandle, FTU_ExpectEdgeNumberMeasureType *pExpectEdgeNumMeasure)

Initialize a Expect Edge Number Measurement channel.

void **FTU_GetSignalMeasureResult** (FTU_HandleType *pFtuHandle, uint8_t u8Channel, FTU_SignalMeasureValueType *pResult)

Get the measurement result of the channel.

void **FTUn_IRQHandler** (FTU_HandleType *pFtuHandle)

Interrupt IRQ handle of FTU instance.

Detailed Description

FTU driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_gpio.h File Reference

GPIO driver type definition and API.

```
#include "HwA_gpio.h"
```

```
#include <stddef.h>
```

Include dependency graph for module_driver_gpio.h:

IMAGE

Macros

```
#define GPIO_MAX_PORT_PINS 32
#define GPIO_PIN_0 ((uint32_t)0x00000001)
#define GPIO_PIN_1 ((uint32_t)0x00000002)
#define GPIO_PIN_2 ((uint32_t)0x00000004)
#define GPIO_PIN_3 ((uint32_t)0x00000008)
#define GPIO_PIN_4 ((uint32_t)0x00000010)
#define GPIO_PIN_5 ((uint32_t)0x00000020)
#define GPIO_PIN_6 ((uint32_t)0x00000040)
#define GPIO_PIN_7 ((uint32_t)0x00000080)
#define GPIO_PIN_8 ((uint32_t)0x00000100)
#define GPIO_PIN_9 ((uint32_t)0x00000200)
#define GPIO_PIN_10 ((uint32_t)0x00000400)
#define GPIO_PIN_11 ((uint32_t)0x00000800)
#define GPIO_PIN_12 ((uint32_t)0x00001000)
#define GPIO_PIN_13 ((uint32_t)0x00002000)
#define GPIO_PIN_14 ((uint32_t)0x00004000)
#define GPIO_PIN_15 ((uint32_t)0x00008000)
#define GPIO_PIN_16 ((uint32_t)0x00010000)
#define GPIO_PIN_17 ((uint32_t)0x00020000)
#define GPIO_PIN_18 ((uint32_t)0x00040000)
#define GPIO_PIN_19 ((uint32_t)0x00080000)
#define GPIO_PIN_20 ((uint32_t)0x00100000)
#define GPIO_PIN_21 ((uint32_t)0x00200000)
#define GPIO_PIN_22 ((uint32_t)0x00400000)
#define GPIO_PIN_23 ((uint32_t)0x00800000)
#define GPIO_PIN_24 ((uint32_t)0x01000000)
```



```

#define GPIO_PIN_25 ((uint32_t)0x02000000)
#define GPIO_PIN_26 ((uint32_t)0x04000000)
#define GPIO_PIN_27 ((uint32_t)0x08000000)
#define GPIO_PIN_28 ((uint32_t)0x10000000)
#define GPIO_PIN_29 ((uint32_t)0x20000000)
#define GPIO_PIN_30 ((uint32_t)0x40000000)
#define GPIO_PIN_31 ((uint32_t)0x80000000)

GPIO API Service IDs#define GPIO_Toggle_ID 0U
#define GPIO_ReadPins_ID 1U
#define GPIO_WritePins_ID 2U
#define GPIO_SetPinsDir_ID 3U

```

GPIO Dev Error Code*Error Code of calling GPIO apis*

```

#define GPIO_E_PARAM_INSTANCE 0x01U
#define GPIO_E_PARAM_LEVEL 0x02U
#define GPIO_E_PARAM_PIN 0x03U
#define GPIO_E_PARAM_DIRECTION 0x04U

```

Enumerations

```
enum GPIO_InstanceType { GPIO_A = 0U, GPIO_B, GPIO_C, GPIO_D, GPIO_E, GPIO_F, GPIO_G, GPIO_H, GPIO_I }
```

Port instance number.

```
enum Gpio_LevelType { GPIO_LOW = 0U, GPIO_HIGH }
```

Gpio level.

```
enum Gpio_Direction { GPIO_IN = 0U, GPIO_OUT }
```

Gpio Direction.

Functions

```
void GPIO_Toggle (const GPIO_InstanceType eGpio, const uint32_t u32Pins)
```

Toggle gpio api.

```
uint32_t GPIO_ReadPins (const GPIO_InstanceType eGpio, const uint32_t u32Pins)
```

Read level of input port pins.

```
void GPIO_WritePins (const GPIO_InstanceType eGpio, const uint32_t u32Pins, const Gpio_LevelType eOutput)
```

Write gpio level to u8Pins.

```
void GPIO_SetPinsDir (const GPIO_InstanceType eGpio, const uint32_t u32Pins, const Gpio_Direction
ePindir)
Set Pins Direction.
```

Detailed Description

GPIO driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_hrpwm.h File Reference

HRPWM driver type definition and API.

#include "HwA_hrpwm.h"

Include dependency graph for module_driver_hrpwm.h:

IMAGE

Detailed Description

HRPWM driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-11-07

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_hsadc.h File Reference

HSADC driver type definition and API.

#include "HwA_hsadc.h"

Include dependency graph for module_driver_hsadc.h:

IMAGE

Classes

struct **_HSADC_HandleType**

*The structure of the CMU processing handle Implements : **_HSADC_HandleType**.*

struct **HSADC_InitType**

Defines the converter configuration.

struct **HSADC_ChannelCfgType**

The configuration option for the HSADC channel.

struct **HSADC_SeqGroupType**

The configuration option for the HSADC sequence group.

struct **HSADC_CmpType**

Defines the hardware compare configuration.

struct **HSADC_FastCmpType**

Defines the hardware fast compare configuration.

Macros

HSADC API Service IDs#define **HSADC_INIT_STRUCTURE_ID** 0U

#define **HSADC_INIT_ID** 1U

#define **HSADC_DEINIT_ID** 2U

#define **HSADC_INIT_CHANNEL_ID** 3U

#define **HSADC_INIT_CMP_STRUCTURE_ID** 4U

#define **HSADC_INIT_CMP_ID** 5U

#define **HSADC_INIT_FAST_CMP_ID** 6U

#define **HSADC_INIT_FAST_CMP_STRUCTURE_ID** 7U

#define **HSADC_INIT_SEQ_GROUP_ID** 8U

#define **HSADC_ENABLE_ID** 9U

#define **HSADC_DISABLE_ID** 10U

#define **HSADC_START_ID** 11U

```

#define HSADC_STOP_ID 12U
#define HSADC_RESET_ID 13U
#define HSADC_GET_FAST_CMP_RESULT_ID 14U
#define HSADC_GET_FAST_CMP_BOUNDARY_FLAG_ID 15U

```

HSADC Dev Error Code *Error Code of calling HSADC apis*

```

#define HSADC_E_PARAM_INSTANCE 0x01U
#define HSADC_E_PARAM_CHANNEL 0x02U
#define HSADC_E_PARAM_POINTER 0x03U
#define HSADC_E_PARAM_COUNT 0x04U
#define HSADC_E_PARAM_INVALID 0x05U
#define HSADC_E_PARAM_PHASE_SHIFT 0x06U

```

HSADC0 Internal Channels *Available internal HSADC channels for HSADC Instance 0*

```

#define HSADC0_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC0_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

```

HSADC1 Internal channels *Available internal HSADC channels for HSADC Instance 1*

```

#define HSADC1_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC1_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

```

HSADC2 Internal channels *Available internal HSADC channels for HSADC instance 2*

```

#define HSADC2_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC2_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

```

HSADC3 Internal channels *Available internal HSADC channels for HSADC instance 3*

```

#define HSADC3_CHANNEL_VBG_BUFFER HSADC_CHANNEL_INTERNAL_0
#define HSADC3_CHANNEL_V25 HSADC_CHANNEL_INTERNAL_1

```

Typedefs

```
typedef struct _HSADC_HandleType HSADC_HandleType
```

The structure of the CMU processing handle Implements : _HSADC_HandleType.

Enumerations

```
enum HSADC_InstanceType { HSADC_INSTANCE_0 = 0U, HSADC_INSTANCE_1 = 1U, HSADC_INSTANCE_2 = 2U, HSADC_INSTANCE_3 = 3U }
```

The instance index of the HSADC peripheral.

```
enum HSADC_ChannelType { HSADC_CHANNEL_0 = 0U, HSADC_CHANNEL_1 = 1U, HSADC_CHANNEL_2 = 2U, HSADC_CHANNEL_3 = 3U, HSADC_CHANNEL_4 = 4U, HSADC_CHANNEL_5 = 5U, HSADC_CHANNEL_INTERNAL_0 = 6U, HSADC_CHANNEL_INTERNAL_1 = 7U,
```

HSADC_CHANNEL_EXTEND_0 = 8U, **HSADC_CHANNEL_EXTEND_1** = 9U, **HSADC_CHANNEL_EXTEND_2** = 10U, **HSADC_CHANNEL_EXTEND_3** = 11U, **HSADC_CHANNEL_EXTEND_4** = 12U, **HSADC_CHANNEL_EXTEND_5** = 13U, **HSADC_CHANNEL_EXTEND_6** = 14U, **HSADC_CHANNEL_EXTEND_7** = 15U, **HSADC_CHANNEL_OFFSET_CALIBRATION** = 27U, **HSADC_CHANNEL_GAIN_LOW_CALIBRATION** = 28U, **HSADC_CHANNEL_GAIN_HIGH_CALIBRATION** = 29U }

The channel selected for HSADC conversion.

enum **HSADC_SampleTimeOptionType** { **HSADC_SAMPLE_TIME_OPTION_0** = 0U, **HSADC_SAMPLE_TIME_OPTION_1** = 1U, **HSADC_SAMPLE_TIME_OPTION_2** = 2U, **HSADC_SAMPLE_TIME_OPTION_3** = 3U }

The HSADC sample time option for selection.

enum **HSADC_StatusType** { **HSADC_STATUS_SUCCESS** = 0x0U, **HSADC_STATUS_ERROR** = 0x1U, **HSADC_STATUS_TIMEOUT** = 0x2U }

HSADC operation return values.

Functions

void **HSADC_InitStructure** (**HSADC_InitType** *const pInitCfg)

*Provide the default values of **HSADC_InitType**.*

void **HSADC_InitCmpStructure** (**HSADC_CmpType** *const pCmpCfg)

*Provide the default values of **HSADC_CmpType**.*

void **HSADC_InitFastCmpStructure** (**HSADC_FastCmpType** *const pFastCmpCfg)

*Provide the default values of **HSADC_FastCmpType**.*

void **HSADC_Init** (**HSADC_HandleType** *pAdcHandle, const **HSADC_InitType** *const pInitCfg)

Initialize the HSADC instance.

void **HSADC_DeInit** (**HSADC_HandleType** *pAdcHandle)

De-initialize the HSADC instance.

void **HSADC_InitChannel** (**HSADC_HandleType** *pAdcHandle, const **HSADC_ChannelCfgType** aChannels[], const uint8_t u8ChannelCnt)

Configure the HSADC sample channels.

void **HSADC_InitSeqGroup** (HSADC_HandleType *pAdcHandle, const HSADC_SeqGroupType aSeqGroup[], const uint8_t u8SeqGroupCnt)

Configure the Sequence groups.

void **HSADC_InitCmp** (HSADC_HandleType *pAdcHandle, const HSADC_CmpType *const pCmpCfg)

Configure the hardware compare feature of HSADC.

void **HSADC_InitFastCmp** (HSADC_HandleType *pAdcHandle, const HSADC_FastCmpType *const pFastCmpCfg)

Configure the hardware fast compare feature of HSADC.

HSADC_StatusType **HSADC_Enable** (const HSADC_HandleType *pAdcHandle)

Enable the HSADC instance.

HSADC_StatusType **HSADC_Disable** (HSADC_HandleType *pAdcHandle)

Disable the HSADC instance.

void **HSADC_Start** (const HSADC_HandleType *pAdcHandle)

Start the HSADC conversion.

HSADC_StatusType **HSADC_Stop** (const HSADC_HandleType *pAdcHandle)

Stop the HSADC conversion.

void **HSADC_Reset** (const HSADC_HandleType *pAdcHandle)

Reset the HSADC hardware.

void **HSADCn_IRQHandler** (HSADC_HandleType *pAdcHandle)

The internal interrupt handler function for HSADC instances.

bool **HSADC_GetFastCmpResult** (const HSADC_HandleType *pHsadcHandle)

Get fast compare result.

bool **HSADC_GetFastCmpBoundaryFlag** (const HSADC_HandleType *pHsadcHandle)

Get fast compare boundary flag.

Detailed Description

HSADC driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_hsm.h File Reference

HSM driver type definition and API.

```
#include "device_header.h"
```

Include dependency graph for module_driver_hsm.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Classes

struct **HSM_CmdType**

HSM cmd type definition.

struct **HSMCom_TrueRandType**

the true random information passed to HSM core by mailbox

struct **HSMCom_TrueRandExType**

the true random extend information passed to HSM core by mailbox

struct **HSM_TrngInitCfgType**

struct **HSMCom_FastTrueRandType**

struct **HSM_SmsCfgType**

SHA/MD5/SM3 context information used by driver when call specific API.

struct **HSM_ShaResultBufType**

type definition for driver to store information

struct **HSM_ShaType**

SHA information used by driver when call specific API.

struct **HSMCom_ShaType**

SHA information used by driver when call specific API.

struct **HSM_ShaExType**

SHA extend information used by driver when call specific API.

struct **HSMCom_ShaExType**

SHA extend information used by driver when call specific API.

struct **HSM_Sha2Type**

SHA information used by driver when call specific API.

struct **HSMCom_Sha2Type**

SHA information used by driver when call specific API.

struct **HSM_ScatterHashType**

Scatter Hash information used by driver when call specific API.

struct **HSMCom_ScatterHashType**

Scatter Hash information used by driver when call specific API.

struct **HSM_DrvEccCurveParamType**

ECC curve information used by driver when call specific API.

struct **HSM_EccSignType**

ECC sign information used by driver when call specific API.

struct **HSMCom_EccSignType**

ECC sign information used by driver when call specific API.

struct **HSM_EccVerifyType**

ECC verify information used by driver when call specific API.

struct **HSMCom_EccVerifyType**

ECC verify information used by driver when call specific API.

struct **HSM_Sm2GenKeyPairType**

SM2 key pair generation information used by driver when call specific API.

struct **HSMCom_Sm2GenKeyPairType**

SM2 key pair generation information used by driver when call specific API.

struct **HSM_Sm2EncryptType**

SM2 encrypt information used by driver when call specific API.

struct **HSMCom_Sm2EncryptType**

SM2 encrypt information used by driver when call specific API.

struct **HSM_Sm2DecryptType**

SM2 decrypt information used by driver when call specific API.

struct **HSMCom_Sm2DecryptType**

SM2 decrypt information used by driver when call specific API.

struct **HSM_Sm2SignType**

SM2 sign information used by driver when call specific API.

struct **HSMCom_Sm2SignType**

SM2 sign information used by driver when call specific API.

struct **HSM_Sm2VerifyType**

SM2 verify information used by driver when call specific API.

struct **HSMCom_Sm2VerifyType**

SM2 verify information used by driver when call specific API.

struct **HSM_Sm2GenZaType**

SM2 ZA generation information used by driver when call specific API.

struct **HSMCom_Sm2GenZaType**

SM2 ZA generation information used by driver when call specific API.

struct **HSM_Sm2GenHashType**

SM2 HASH generation information used by driver when call specific API.

struct **HSMCom_Sm2GenHashType**

SM2 HASH generation information used by driver when call specific API.

struct **HSM_EccEasyEncryType**

ECC encrypt information used by driver when call specific API.

struct **HSMCom_EccEasyEncryType**

ECC encrypt information used by driver when call specific API.

struct **HSM_EccEasyDecryType**

ECC decrypt information used by driver when call specific API.

struct **HSMCom_EccEasyDecryType**

ECC decrypt information used by driver when call specific API.

struct **HSM_EccCalcType**

ECC point calculate used by driver when call specific API.

struct **HSMCom_EccCalcType**

ECC point calculate used by driver when call specific API.

struct **HSMCom_RequestAuthType**

Request Auth used by driver when call specific API.

struct **HSMCom_LifeCycleChangeType**

Change lifecycle used by driver when call specific API.

struct **HSMCom_CancelJobType**

Cancel job used by driver when call specific API.

struct **HSMCom_SelfTestType**

struct **HSMCom_NvrOtpType**

struct **HSM_KeyManagerExportUserKeyType**

struct **HSM_KeyManagerImportUserKeyType**

struct **HSM_Ln_ImportUserKeyExtType**

struct **HSM_Ln_CopyUserKeyType**

struct **HSM_KeyManagerGenUserKeyType**

```

struct HSMCom_GenUserKeyType
struct HSM_KeyManagerCheckUserKeyStatusExtType
struct HSMCom_UserKeyManageType
struct HSM_Ln_KeySpaceStatusType
struct HSMCom_KeySpaceStatusType
struct HSM_AesNewKeyInfType
key information used by AES when use one key in software
struct HSM_AesVendorKeyInfType
key information used by AES when use one key in nvr flash
struct HSM_AesEnDecryptType
AES encrypt/decrypt information used by driver when call specific API.
struct HSM_FlexAesEnDecryptType
Flex AES encrypt/decrypt information used by driver when call specific API.
struct HSMCom_AesEncryptType
struct HSMCom_AesDecryptType
struct HSMCom_FlexAesEncryptType
struct HSMCom_FlexAesDecryptType
struct HSM_Sm4EnDecryptType
SM4 encrypt/decrypt information used by driver when call specific API.
struct HSMCom_Sm4EnDecryptType
struct HSMWrap_Sm4EncryptType
struct HSMWrap_Sm4EncryptExType
struct HSM_XCMacType
CMAC/XMAC information used by driver when call specific API.
struct HSMCom_CMacType
struct HSM_XCMacExType
struct HSMCom_CMacExType
struct HSMCom_XMacType
struct HSM_CcmEncryptType
CCM Encrypt information used by driver when call specific API.
struct HSMCom_CcmEncryptType
struct HSM_CcmDecryptType
CCM Decrypt information used by driver when call specific API.
struct HSMCom_CcmDecryptType
struct HSM_GcmEncryptType
GCM Encrypt information used by driver when call specific API.

```

struct **HSMCom_GcmEncryptType**

struct **HSM_GcmDecryptType**

GCM Decrypt information used by driver when call specific API.

struct **HSMCom_GcmDecryptType**

struct **HSM_Md5Type**

MD5 information used by driver when call specific API.

struct **HSMCom_Md5Type**

struct **HSM_Md5ExType**

MD5 information used by driver when call specific API.

struct **HSMCom_Md5ExType**

struct **HSM_Sm3Type**

SM3 information used by driver when call specific API.

struct **HSMCom_Sm3Type**

struct **HSM_Sm3ExType**

struct **HSMCom_Sm3ExType**

struct **HSMCom_MonCountType**

struct **HSM_RsaType**

RSA information used by driver when call specific API.

struct **HSMCom_RsaType**

struct **HSM_BigNumberCalcType**

RSA information used by driver when call specific API.

struct **HSMCom_BigNumberCalcType**

struct **HSM_BigNumberCalcExtType**

struct **HSMCom_BigNumberCalcExtType**

struct **HSMCom_LoadFirmwareType**

struct **HSMCom_AuthCheckType**

struct **HSM_AESM_CFG_KEY_CFG_T**

struct **HSM_AESM_ENTRY_CFG_T**

struct **HSM_AesmRawApiType**

struct **HSMCom_AesmRawApiType**

struct **HSM_RsaSsaPssVerifyType**

RSA PSS signature verify information used by driver when call specific API.

struct **HSMCom_RsaSsaPssVerifyType**

struct **HSM_RsaSsaPkcs1V15VerifyType**

RSA PKCS1V1.5 signature verify information used by driver when call specific API.

struct **HSMCom_RsaSsaPkcs1V15VerifyType**

```

struct HSM_RsaSsaPssSignType
struct HSMCom_RsaSsaPssSignType
struct HSM_RsaSsaPkcs1V15SignType
RSA PKCS1 V1.5 signature information used by driver when call specific API.
struct HSMCom_RsaSsaPkcs1v15SignType
struct HSMCom_GetRomHashType

```

Macros

```

#define TYPE_CONVERT(from_type, to_type, data) ((to_type)(data))
#define HSM_ECC_CALC_PADD 0xFC73F801U
#define HSM_ECC_CALC_PDBL 0xFC73F802U
#define HSM_ECC_CALC_PMUL 0xFC73F803U
#define HSM_ECC_CALC_PCHK 0xFC73F804U
#define HSM_COM_REQUEST_AUTH_DATA_U32_CNT 8
#define HSMCOM_NVR_OTP_START_ADDR 0x04400400
#define HSMCOM_NVR_OTP_END_ADDR 0x04401fff
#define KEYMANAGER_GEN_KEYPAIR_SM2 (0u)
#define KEYMANAGER_GEN_KEYPAIR_ECC (1u)
#define KEYMANAGER_GEN_KEYPAIR_MAX (2u)
#define KEYMANAGER_USERKEY_STATUS_NOT_EXIST (0u)
#define KEYMANAGER_USERKEY_STATUS_EXIST_VALID (1u)
#define KEYMANAGER_USERKEY_STATUS_EXIST_INVALID (2u)
#define KEYMANAGER_USERKEY_STATUS_ERROR (3u)
#define HSM_FLEXAES_PAD_ZERO 0u
Flexiable AES zero padding type for type "HSM_FlexAesPadType" if input data is not aligned with 16bytes, this configuration will make append byte data 0 to keep 16bytes align.

#define HSM_FLEXAES_PAD_PKCS7 1u
Flexiable AES PKCS7 padding type for type "HSM_FlexAesPadType" if input data is not aligned with 16bytes, this configuration will make append bytes by PKCS7 method to keep 16bytes align.

#define HSM_XCMAC_ENGINE_SM4 1U
#define HSM_XCMAC_ENGINE_AES 0U
#define HSM_BIGNUMBER_CALC_A_ADD_B 0xFC730002
#define HSM_BIGNUMBER_CALC_A_SUB_B 0xFC730003
#define HSM_BIGNUMBER_CALC_B_SUB_A 0xFC730004
#define HSM_BIGNUMBER_CALC_A_MUL_B 0xFC730005

```

```

#define HSM_BIGNUMBER_CALC_A_EXP_E 0xFC730006
#define HSM_BIGNUMBER_CALC_A_RED_N 0xFC730007
#define HSM_BIGNUMBER_CALC_A_INV 0xFC730008
#define HSM_BN_VALID_FLAG_E 0x1u
#define HSM_BN_VALID_FLAG_B 0x2u
#define HSM_RSA_SSA_INPUT_RAW_MESSAGE 0u
RSA SSA input data is the raw message for HSM_RsaSsaInputType.

#define HSM_RSA_SSA_INPUT_HASH_DATA 1u
RSA SSA input data is the hash value of raw message for HSM_RsaSsaInputType.

#define HSM_HASH_MD5 (0U)
MD5 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_160 (1U)
SHA160 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_224 (2U)
SHA224 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_256 (3U)
SHA256 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_384 (4U)
SHA384 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512 (5U)
SHA512 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512_224 (6U)
SHA512/224 algorithm value for type "HSM_HashAlgType".

#define HSM_HASH_SHA_512_256 (7U)
SHA512/256 algorithm value for type "HSM_HashAlgType".

```

#define **HSM_HASH_SM3** (8U)

SM3 algorithm value for type "HSM_HashAlgType".

Typedefs

typedef uint32_t **hsm_uint32_t**

typedef uint16_t **hsm_uint16_t**

typedef uint8_t **hsm_uint8_t**

typedef int32_t **hsm_int32_t**

typedef int8_t **hsm_int8_t**

typedef **HSM_SmsCfgType** **HSM_ShaCtxType**

SHA context information used by driver when call specific API.

typedef **hsm_uint32_t** **HSM_KeyManagerGenKeyPairType**

typedef **hsm_uint32_t** **KeyManager_UserKeyStatusType**

typedef **hsm_uint32_t** **HSM_FlexAesPadType**

Flexiable AES padding type definition if use zero padding, refer to HSM_FLEXAES_PAD_ZERO, after decrypt, user should remove the zero manually if use PKCS7 padding, refer to HSM_FLEXAES_PAD_PKCS7.

typedef **HSM_AesEnDecryptType** **HSM_AesEncryptType**

AES decrypt information used by driver when call specific API.

typedef **HSM_AesEnDecryptType** **HSM_AesDecryptType**

AES encrypt information used by driver when call specific API.

typedef **HSM_FlexAesEnDecryptType** **HSM_FlexAesEncryptType**

Flex AES decrypt information used by driver when call specific API.

typedef **HSM_FlexAesEnDecryptType** **HSM_FlexAesDecryptType**

AES encrypt information used by driver when call specific API.

typedef **HSMCom_Sm4EnDecryptType** **HSMCom_Sm4DecryptType**

typedef **HSMCom_Sm4EnDecryptType** **HSMCom_Sm4EncryptType**

typedef **HSMCom_FlexAesEncryptType** **HSMCom_Sm4EncryptExType**

typedef HSMCom_FlexAesDecryptType HSMCom_Sm4DecryptExType

typedef HSM_XCMacType HSM_CMacType

CMAC information used by driver when call specific API.

typedef hsm_uint32_t HSM_XCMacEngineType

typedef HSM_XCMacExType HSM_CMacExType

typedef HSM_XCMacType HSM_XMacType

XMAC information used by driver when call specific API.

typedef HSM_SmsCfgType HSM_Md5CtxType

MD5 context information used by driver when call specific API.

typedef HSM_SmsCfgType HSM_Sm3CtxType

SM3 context information used by driver when call specific API.

typedef hsm_uint32_t HSM_RsaSsaInputType

type definition for rsa ssa input data type refer to HSM_RSA_SSA_INPUT_RAW_MESSAGE, HSM_RSA_SSA_INPUT_HASH_DATA

typedef hsm_uint32_t HSM_HashAlgType

type definition for SHA algorithm refer to HSM_SHA_160, HSM_SHA_224, HSM_SHA_256, HSM_SHA_384, HSM_SHA_512, HSM_SHA_512_224, HSM_SHA_512_256, HSM_HASH_MD5, HSM_HASH_SM3

Enumerations

enum HSM_StatusType { HSM_STATUS_SUCCESS = 0U, HSM_STATUS_BUSY, HSM_STATUS_PARAM_ERR, HSM_STATUS_ERROR, HSM_STATUS_ECC_ERROR, HSM_STATUS_IDLE, HSM_STATUS_ERROR_STOP, HSM_STATUS_FINISH, HSM_STATUS_PASS, HSM_STATUS_FAIL, HSM_STATUS_AGAIN, HSM_STATUS_LOGIC_ERR, HSM_STATUS_HW_ERR, HSM_STATUS_HW_IDLE, HSM_STATUS_ALREADY_FINISHED, HSM_STATUS_SEND_FAIL, HSM_STATUS_BUFFER_SIZE_ERR, HSM_STATUS_TIMEOUT_ERR, HSM_STATUS_NOT_EXIST, HSM_STATUS_EXIST, HSM_STATUS_ZOMBIE, HSM_STATUS_SPACE_FULL }

Status returned by HSM SDK CM7 side APIs.

Functions

API declaration for HSM **HSM_StatusType HSM_TrueRandGetSrc0** (**HSM_CmdType** *pCmd, const **HSMCom_TrueRandType** *pCfg)

Get the true random data from TRNG 0.

HSM_StatusType HSM_TrueRandGetSrc1 (**HSM_CmdType** *pCmd, const **HSMCom_TrueRandType** *pCfg)

Get the true random data from TRNG 1.

HSM_StatusType HSM_TrueRandGetSrcXor (**HSM_CmdType** *pCmd, const **HSMCom_TrueRandType** *pCfg)

Get the true random data from (TRNG 0 xor TRNG 1)

HSM_StatusType HSM_TrueRandGetSrcShiftXor (**HSM_CmdType** *pCmd, const **HSMCom_TrueRandType** *pCfg)

Get the random data from ((TRNG 0 << 1) xor TRNG 1)

HSM_StatusType HSM_TrueRandGet (**HSM_CmdType** *pCmd, const **HSMCom_TrueRandExType** *pCfg)

Get the random data from TRNG 0 or TRNG 1.

HSM_StatusType HSM_TrueRandGetAny (**HSM_CmdType** *pCmd, const **HSMCom_TrueRandType** *pCfg)

Get the random data from any TRNG.

HSM_StatusType HSM_FastTrueRandGet (**HSM_CmdType** *pCmd, const **HSMCom_FastTrueRandType** *pCfg)

Get the fast trng result.

HSM_StatusType HSM_Sha (**HSM_CmdType** *pCmd, const **HSMCom_ShaType** *pCfg)

SHA.

HSM_StatusType HSM_FW_ShaEx (**HSM_CmdType** *pCmd, const **HSMCom_ShaExType** *pCfg, **HSM_BoolType** bKeyUseld)

SHA Extend.

HSM_StatusType HSM_EccVerify (HSM_CmdType *pCmd, const HSMCom_EccVerifyType *pCfg)

ECC Verify.

HSM_StatusType HSM_EccCalculateEx (HSM_CmdType *pCmd, const HSMCom_EccCalcType *pCfg)

ECC calculate Externd.

HSM_StatusType HSM_Sm2Decry (HSM_CmdType *pCmd, const HSMCom_Sm2DecryptType *pCfg)

SM2 decrypt.

HSM_StatusType HSM_Sm2Verify (HSM_CmdType *pCmd, const HSMCom_Sm2VerifyType *pCfg)

SM2 verify.

HSM_StatusType HSM_Sm2GenZa (HSM_CmdType *pCmd, const HSMCom_Sm2GenZaType *pCfg)

SM2 ZA.

HSM_StatusType HSM_Sm2GenHash (HSM_CmdType *pCmd, const HSMCom_Sm2GenHashType *pCfg)

SM2 Hash generate.

HSM_StatusType HSM_Sm4EncryptStart (HSM_CmdType *pCmd, HSMWrap_Sm4EncryptType *pCtx, HSMCom_Sm4EncryptType *pDecrypt)

SM4 Encrypt start.

HSM_StatusType HSM_Sm4EncryptUpdate (HSM_CmdType *pCmd, HSMWrap_Sm4EncryptType *pCtx)

SM4 Encrypt update.

HSM_StatusType HSM_Sm4EncryptExStart (HSM_CmdType *pCmd, HSMWrap_Sm4EncryptExType *pCtx, HSMCom_Sm4EncryptExType *pCfg)

SM4 Encrypt extend start.

**HSM_StatusType HSM_Sm4EncryptExUpdate (HSM_CmdType *pCmd,
HSMWrap_Sm4EncryptExType *pCtx)**
SM4 Encrypt extend update.

**HSM_StatusType HSM_Sm4Decrypt (HSM_CmdType *pCmd, const HSMCom_Sm4DecryptType
*pCfg)**
SM4 Decrypt.

**HSM_StatusType HSM_Sm4DecryptEx (HSM_CmdType *pCmd, const HSMCom_Sm4DecryptExType
*pCfg)**
SM4 Decrypt extend.

**HSM_StatusType HSM_EccEasyDecry (HSM_CmdType *pCmd, const HSMCom_EccEasyDecryType
*pCfg)**
ECC decrypt.

**HSM_StatusType HSM_RequestAuthorization (HSM_CmdType *pCmd, const
HSMCom_RequestAuthType *pCfg)**
require challenge data for auth

HSM_StatusType HSM_LifeCycleTakeEffect (HSM_CmdType *pCmd)
Life cycle change take effect.

**HSM_StatusType HSM_OemDevEnter (HSM_CmdType *pCmd, const HSMCom_AuthCheckType
*pCfg)**
Life cycle change to OEM DEV(need call HSM_LifeCycleTakeEffect to take effect)

**HSM_StatusType HSM_OemPdtEnter (HSM_CmdType *pCmd, const HSMCom_AuthCheckType
*pCfg)**
Life cycle change to OEM PDT(need call HSM_LifeCycleTakeEffect take effect)

HSM_StatusType HSM_InFieldEnter (HSM_CmdType *pCmd, const HSMCom_AuthCheckType *pCfg)
Life cycle change to In Field(need call HSM_LifeCycleTakeEffect take effect)

HSM_StatusType HSM_CancelJob (HSM_CmdType *pCmd, const HSMCom_CancelJobType *pCfg)
cancel the current job of the channel

HSM_StatusType HSM_SelfTest (HSM_CmdType *pCmd, volatile HSMCom_SelfTestType *pCfg)
hsm self test

HSM_StatusType HSM_NvrOtpProgram (HSM_CmdType *pCmd, const HSMCom_NvrOtpType *pCfg)
NVR OTP program.

HSM_StatusType HSM_NvrOtpRead (HSM_CmdType *pCmd, const HSMCom_NvrOtpType *pCfg)
NVR OTP data read.

**HSM_StatusType HSM_UserKeyImport (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**
user key import

**HSM_StatusType HSM_UserKeyExport (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**
user key export

**HSM_StatusType HSM_GetKeySpaceStat (HSM_CmdType *pCmd, const
HSMCom_KeySpaceStatusType *pCfg)**
get the user key space status in HSM internal

**HSM_StatusType HSM_TidyUpKeySpace (HSM_CmdType *pCmd, const
HSMCom_KeySpaceStatusType *pCfg)**
tidy the user key space in HSM internal, if revoked key exist, it will make the space re-useable

**HSM_StatusType HSM_UserKeyRevoke (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**
Revoke user key.

HSM_StatusType HSM_AesEncrypt (HSM_CmdType *pCmd, const HSMCom_AesEncryptType *pCfg)
AES encrypt API.

HSM_StatusType HSM_AesEncryptEx (HSM_CmdType *pCmd, const HSMCom_FlexAesEncryptType *pCfg)

AES encrypt extend API.

HSM_StatusType HSM_AesDecrypt (HSM_CmdType *pCmd, const HSMCom_AesDecryptType *pCfg)

AES decrypt.

HSM_StatusType HSM_AesDecryptEx (HSM_CmdType *pCmd, const HSMCom_FlexAesDecryptType *pCfg)

AES decrypt extend.

HSM_StatusType HSM_CMac (HSM_CmdType *pCmd, const HSMCom_CMacType *pCfg)

CMAC.

HSM_StatusType HSM_CMacEx (HSM_CmdType *pCmd, const HSMCom_CMacExType *pCfg)

CMAC extend.

HSM_StatusType HSM_XMac (HSM_CmdType *pCmd, const HSMCom_XMacType *pCfg)

XMAC.

HSM_StatusType HSM_CcmEncry (HSM_CmdType *pCmd, const HSMCom_CcmEncryptType *pCfg)

CCM encrypt.

HSM_StatusType HSM_CcmDecry (HSM_CmdType *pCmd, const HSMCom_CcmDecryptType *pCfg)

CCM decrypt.

HSM_StatusType HSM_GcmEncry (HSM_CmdType *pCmd, const HSMCom_GcmEncryptType *pCfg)

GCM encrypt.

HSM_StatusType HSM_GcmDecry (HSM_CmdType *pCmd, const HSMCom_GcmDecryptType *pCfg)

GCM decrypt.

HSM_StatusType HSM_Md5 (HSM_CmdType *pCmd, const HSMCom_Md5Type *pCfg)

MD5.

**HSM_StatusType HSM_FW_Md5Ex (HSM_CmdType *pCmd, const HSMCom_Md5ExType *pCfg,
HSM_BoolType bKeyUseld)**

MD5 extend.

HSM_StatusType HSM_Sm3 (HSM_CmdType *pCmd, const HSMCom_Sm3Type *pCfg)

SM3.

**HSM_StatusType HSM_FW_Sm3Ex (HSM_CmdType *pCmd, const HSMCom_Sm3ExType *pCfg,
HSM_BoolType bKeyUseld)**

SM3 extend.

**HSM_StatusType HSM_MonotonicCounterIncrease (HSM_CmdType *pCmd, const
HSMCom_MonCountType *pCfg)**

monotonic counter increase 1

**HSM_StatusType HSM_MonotonicCounterRead (HSM_CmdType *pCmd, const
HSMCom_MonCountType *pCfg)**

read monotonic counter value

HSM_StatusType HSM_Rsa (HSM_CmdType *pCmd, const HSMCom_RsaType *pCfg)

RSA.

**HSM_StatusType HSM_BigNumberCalculateEx (HSM_CmdType *pCmd, const
HSMCom_BigNumberCalcType *pCfg)**

Big number calculate extend.

HSM_StatusType HSM_LoadFirmware (HSM_CmdType *pCmd, HSMCom_LoadFirmwareType *pCfg)

trigger firware take effect when the firmware is in delay load mode

**HSM_StatusType HSM_FW_ScatterHashInit (HSM_CmdType *pCmd, const
HSMCom_ScatterHashType *pCfg, HSM_BoolType bKeyUseld, HSM_DataFormatType elnputFmt)**

Scatter hash init.

**HSM_StatusType HSM_FW_ScatterHashUpdate (HSM_CmdType *pCmd, const
HSMCom_ScatterHashType *pCfg, HSM_BoolType bKeyUseld, HSM_DataFormatType elInputFmt)**

Scatter hash update.

**HSM_StatusType HSM_FW_ScatterHashFinal (HSM_CmdType *pCmd, const
HSMCom_ScatterHashType *pCfg, HSM_BoolType bKeyUseld, HSM_DataFormatType elInputFmt)**

Scatter hash final.

**HSM_StatusType HSM_FW_EccEncrypt (HSM_CmdType *pCmd, const HSMCom_EccEasyEncryType
*pCfg)**

ECC encrypt.

**HSM_StatusType HSM_FW_UserKeyGenPair (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**

Generate SM2/ECC key pair(private key and public key)

HSM_StatusType HSM_FW_EccSign (HSM_CmdType *pCmd, const HSMCom_EccSignType *pCfg)

ECC Sign.

**HSM_StatusType HSM_FW_Sm2GenKeyPair (HSM_CmdType *pCmd, const
HSMCom_Sm2GenKeyPairType *pCfg)**

SM2 key pair generate.

**HSM_StatusType HSM_FW_Sm2Encry (HSM_CmdType *pCmd, const HSMCom_Sm2EncryptType
*pCfg)**

SM2 encrypt.

HSM_StatusType HSM_FW_Sm2Sign (HSM_CmdType *pCmd, const HSMCom_Sm2SignType *pCfg)

SM2 sign.

**HSM_StatusType HSM_FW_UserKeyRevoke (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**

user key revoke(delete ram key or abandon key in flash)

**HSM_StatusType HSM_FW_ScatterCMacInit (HSM_CmdType *pCmd, const
HSMCom_AesmRawApiType *pCfg, HSM_DataFormatType eInputFmt, HSM_DataFormatType
eKeyFmt)**

Scatter cmac init.

**HSM_StatusType HSM_FW_ScatterCMacUpdate (HSM_CmdType *pCmd, const
HSMCom_AesmRawApiType *pCfg, HSM_DataFormatType eInputFmt, HSM_DataFormatType
eKeyFmt)**

Scatter cmac update.

**HSM_StatusType HSM_FW_ScatterCMacFinal (HSM_CmdType *pCmd, const
HSMCom_AesmRawApiType *pCfg, HSM_DataFormatType eInputFmt, HSM_DataFormatType
eKeyFmt)**

Scatter cmac final.

**HSM_StatusType HSM_FW_RsaSsaPssVerify (HSM_CmdType *pCmd, const
HSMCom_RsaSsaPssVerifyType *pCfg)**

RsaSsa-pss verify.

**HSM_StatusType HSM_FW_RsaSsaPkcs1v15Verify (HSM_CmdType *pCmd, const
HSMCom_RsaSsaPkcs1V15VerifyType *pCfg)**

RsaSsa-pkcs1-v1_5 verify.

**HSM_StatusType HSM_FW_UserKeyImportNew (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**

Userkey Import.

**HSM_StatusType HSM_FW_CheckKeyIdExt (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**

Check key ID.

**HSM_StatusType HSM_FW_CopyKeyExt (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**

Copy key.

**HSM_StatusType HSM_FW_BigNumberCalculateEx (HSM_CmdType *pCmd, const
HSMCom_BigNumberCalcExtType *pCfg)**

Big Number calculate.

**HSM_StatusType HSM_FW_RsaSsaPssSign (HSM_CmdType *pCmd, const
HSMCom_RsaSsaPssSignType *pCfg)**

RsaSsa-pss verify.

**HSM_StatusType HSM_FW_RsaSsaPkcs1v15Sign (HSM_CmdType *pCmd, const
HSMCom_RsaSsaPkcs1v15SignType *pCfg)**

RsaSsa-pss-Pkcs-1v15 verify.

**HSM_StatusType HSM_FW_GetKeyIdStatus (HSM_CmdType *pCmd, const
HSMCom_UserKeyManageType *pCfg)**

Get key Id status.

definitions for HSM_BoolType

#define HSM_FALSE (0u)

Boolean false value definition for type HSM_BoolType used by HSM.

#define HSM_TRUE (1u)

Boolean true value definition for type HSM_BoolType used by HSM.

typedef hsm_uint32_t HSM_BoolType

Boolean type for HSM.

definitions for HSM_MailboxApiRetType

#define MAILBOXAPI_RET_SUCCESS (0u)

success status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_ERROR** (1u)

error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_FAIL** (2u)

failure status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_PASS** (3u)

verify pass status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_NOT_SUPPORT** (4u)

function not support status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_HW_ERROR** (5u)

hardware error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_NO_PRIVILEGES** (6u)

have no permission status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_PARAMETER_ERROR** (7u)

parameters error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_AUTHORITY_ERROR** (8u)

authority error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_BUSY** (9u)

hardware busy status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_ECC_ERR** (10u)

flash ecc error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

#define **MAILBOXAPI_RET_TIMEOUT** (11u)

time expire error status (HSM_MailboxApiRetType) returned by HSM subsystem APIs

```
#define MAILBOXAPI_RET_NONE (0xFFFFFFFFu)
typedef hsm_uint32_t HSM_MailboxApiRetType
status type definition returned by HSM subsystem APIs
```

Type definition for HSM Isr/Poll mode

```
#define HSM_TRNG_RAND_U32_CNT 16
the true random count return by the API
```

```
#define HSM_TRNG_RAND_U32_CNT 16
the true random count return by the API
```

```
#define HSMCOM_TRNG_SRC_0 0x40cau
#define HSMCOM_TRNG_SRC_1 0xc51cu
#define HSMCOM_TRNG_SRC_ANY 0x359bu
#define HSMCOM_TRNG_SRC_XOR 0x80a5u
#define HSMCOM_TRNG_SRC_SHIFTXOR 0x648Cu
#define HSMCOM_FASTTRNG_SRC_NORMAL 0x0u
#define HSMCOM_FASTTRNG_SRC_XOR 0x1u
#define HSM_TRIGGER_SRC_USER_POLL (0U)
Poll mode value for type "HSM_TriggerSrcType".
```

```
#define HSM_TRIGGER_SRC_ISR (1U)
Isr mode value for type "HSM_TriggerSrcType".
```

```
#define HSM_DATA_FORMAT_4B 0u
HSM 4bytes unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, use this format, should be divided as following. 0x00010203, 0x04050607, 0x08090a0b, 0x0c0d0e0f.
```

```
#define HSM_DATA_FORMAT_1B 1u
HSM 1byte unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, use this format, should be divided as following. 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f.
```

```
#define HSM_DATA_FORMAT_1B_PAD_0_LEFT_FIT_BUF 3u
```

HSM 1byte unit data format for HSM_DataFormatType for example, if data is 0x000102030405060708090a0b0c0d0e0f, the buffer size is 18bytes, use this format, should be divided as following. 0x00(pad), 0x00(pad), 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f.

```
typedef hsm_uint32_t HSM_RndSrcType
```

```
typedef hsm_uint32_t HSM_FastTrngSrcType
```

```
typedef hsm_uint32_t HSM_TriggerSrcType
```

type definition for hsm state machine trigger source used by cortex side hsm driver

```
typedef hsm_uint32_t HSM_DataFormatType
```

HSM data format refer to HSM_DATA_FORMAT_4B, HSM_DATA_FORMAT_1B.

definitions for HSM_ShaAlgType

```
#define HSM_SHA_160 (1U)
```

SHA160 algorithm value for type "HSM_ShaAlgType".

```
#define HSM_SHA_224 (2U)
```

SHA224 algorithm value for type "HSM_ShaAlgType".

```
#define HSM_SHA_256 (3U)
```

SHA256 algorithm value for type "HSM_ShaAlgType".

```
#define HSM_SHA_384 (4U)
```

SHA384 algorithm value for type "HSM_ShaAlgType".

```
#define HSM_SHA_512 (5U)
```

SHA512 algorithm value for type "HSM_ShaAlgType".

```
#define HSM_SHA_512_224 (6U)
```

SHA512/224 algorithm value for type "HSM_ShaAlgType".

```
#define HSM_SHA_512_256 (7U)
SHA512/256 algorithm value for type "HSM_ShaAlgType".
```

```
typedef hsm_uint32_t HSM_ShaAlgType
type          definition          for          SHA          algorithm          refer          to
HSM_SHA_160,HSM_SHA_224,HSM_SHA_256,HSM_SHA_384,HSM_SHA_512,HSM_SHA_512_224,HSM_SHA_
512_256
```

definitions for HSM_HfamMacType

```
#define HSM_HFAM_MAC_SMAC (0U)
SMAC mode value for type "HSM_HfamMacType".
```

```
#define HSM_HFAM_MAC_HMAC (1U)
HMAC mode value for type "HSM_HfamMacType".
```

```
typedef hsm_uint32_t HSM_HfamMacType
SM3/MD5/SHA can generate MAC, use this to select the type of MAC SMAC refer to HSM_HFAM_MAC_SMAC
HMAC refer to HSM_HFAM_MAC_HMAC.
```

definitions for HSM_DrvEccCurvePrmIndexType

```
#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP224R1 (4U)
ECC curve NIST SECP224R1 value for type "HSM_DrvEccCurvePrmIndexType".
```

```
#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP256R1 (1U)
ECC curve NIST SECP256R1 value for type "HSM_DrvEccCurvePrmIndexType".
```

```
#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP384R1 (2U)
ECC curve NIST SECP384R1 value for type "HSM_DrvEccCurvePrmIndexType".
```

```
#define HSM_DRV_ECC_CURVE_PRM_IDX_SECP521R1 (3U)
ECC curve NIST SECP521R1 value for type "HSM_DrvEccCurvePrmIndexType".
```

```
#define HSM_DRV_ECC_CURVE_PRM_IDX_MAX (0xFFFFFFFFFU)  
max count of ECC curve supported for type "HSM_DrvEccCurvePrmIndexType"
```

```
typedef hsm_uint32_t HSM_DrvEccCurvePrmIndexType  
type definition for ECC curve data that defined by NIST
```

definitions for HSM_Sm4KeyType

```
#define HSM_SM4_KEY_CHIP_VENDOR_IFR (0U) /* key has existed in Flash IFR, it provided by chip, not  
changeable */  
vendor key value for type "HSM_Sm4KeyType"
```

```
#define HSM_SM4_KEY_NEW (1U) /* user should provide the new key */  
driver user's new key value for type "HSM_Sm4KeyType"
```

```
typedef hsm_uint32_t HSM_Sm4KeyType
```

definitions for HSM_SymmEnDecryptAlgType

```
#define HSM_SYMM_ENDECRYPT_MODE_CTR (0U)  
CTR mode value for type "HSM_SymmEnDecryptAlgType".
```

```
#define HSM_SYMM_ENDECRYPT_MODE_CBC (1U)  
CBC mode value for type "HSM_SymmEnDecryptAlgType".
```

```
#define HSM_SYMM_ENDECRYPT_MODE_ECB (2U)  
ECB mode value for type "HSM_SymmEnDecryptAlgType".
```

```
#define HSM_SYMM_ENDECRYPT_MODE_CFB (3U)  
CFB mode value for type "HSM_SymmEnDecryptAlgType".
```

```
#define HSM_SYMM_ENDECRYPT_MODE_OFB (4U)  
OFB mode value for type "HSM_SymmEnDecryptAlgType".
```

```
typedef hsm_uint32_t HSM_SymmEnDecryptAlgType
```

type definition for SM4/AES encrypt/decrypt mode

definitions for HSMCom_FirmwareLoadStatus

#define HSMCOM_FIRMWARE_LOAD_NONE 0U

no firmware detected in flash status

#define HSMCOM_FIRMWARE_LOADING 1U

status value represents firmware is loading now

#define HSMCOM_FIRMWARE_NEED_USER_LOAD 2U

status value represents firmware waiting user to trigger load in CM7 side

#define HSMCOM_FIRMWARE_LOAD_OK 3U

status value represents firmware is loaded successfully

#define HSMCOM_FIRMWARE_LOAD_FAIL 4U

status value represents firmware load fail

typedef hsm_uint32_t HSMCom_FirmwareLoadStatus

type definition for firmware load status

definitions for HSM_KeyManagerBoolType

#define KEYMANAGER_FALSE (0u)

#define KEYMANAGER_TRUE (1u)

typedef hsm_uint32_t HSM_KeyManagerBoolType

definitions for HSM_KeyManagerKeyExportType

#define KEYMANAGER_KEY_EXPORT_FORBID (0u)

#define KEYMANAGER_KEY_EXPORT_PLAIN (1u)

#define KEYMANAGER_KEY_EXPORT_CIPHER (2u)

#define KEYMANAGER_KEY_EXPORT_MAX (3u)

typedef hsm_uint32_t HSM_KeyManagerKeyExportType

definitions for HSM_KeyManagerUserKeyEnDecryType

```
#define KEYMANAGER_ENDECRY_NONE (0u)/* For plain key, no need to encrypt or decrypt. */
#define KEYMANAGER_ENDECRY_AES256 (1u)
#define KEYMANAGER_ENDECRY_SM4 (2u)
#define KEYMANAGER_ENDECRY_MAX (3u)
typedef hsm_uint32_t HSM_KeyManagerUserKeyEnDecryType
```

definitions for HSM_KeyManagerUserKeyTypeType

```
#define KEYMANAGER_USER_KEY_TYPE_NONE (0u) /* not configure the key type, HSM shall not check if
the key type is matched with de/encrypt ALG. */
#define KEYMANAGER_USER_KEY_TYPE_AES (1u)
#define KEYMANAGER_USER_KEY_TYPE_SM4 (1u)
#define KEYMANAGER_USER_KEY_TYPE_ECC_PRIVATE (3u)
#define KEYMANAGER_USER_KEY_TYPE_ECC_PUBLIC (4u)
#define KEYMANAGER_USER_KEY_TYPE_SM2_PRIVATE (5u)
#define KEYMANAGER_USER_KEY_TYPE_SM2_PUBLIC (6u)
#define KEYMANAGER_USER_KEY_TYPE_RSA_E (7u)
#define KEYMANAGER_USER_KEY_TYPE_RSA_N (8u)
#define KEYMANAGER_USER_KEY_TYPE_XMAC (9u)
#define KEYMANAGER_USER_KEY_TYPE_CMAC_KEY (10u)
#define KEYMANAGER_USER_KEY_TYPE_CCM (11u)
#define KEYMANAGER_USER_KEY_TYPE_GCM (12u)
#define KEYMANAGER_USER_KEY_TYPE_ECC_SIGNATURE (13u)
#define KEYMANAGER_USER_KEY_TYPE_SM2_SIGNATURE (14u)
#define KEYMANAGER_USER_KEY_TYPE_IV (15u)
#define KEYMANAGER_USER_KEY_TYPE_CMAC_RESULT (16u)
#define KEYMANAGER_USER_KEY_TYPE_XMAC_RESULT (17u)
#define KEYMANAGER_USER_KEY_TYPE_RSA_SIGNATURE (18u)
#define KEYMANAGER_USER_KEY_TYPE_ED25519_PUBKEY (19u)
#define KEYMANAGER_USER_KEY_TYPE_ED25519_SIGNATURE (20u)
#define KEYMANAGER_USER_KEY_TYPE_ED25519_PRIVATE (21u)
#define KEYMANAGER_USER_KEY_TYPE_PBKDF2_PWD (22u)
#define KEYMANAGER_USER_KEY_TYPE_HASHMAC_KEY (23u)
#define KEYMANAGER_USER_KEY_TYPE_RSA (24u)
typedef hsm_uint32_t HSM_KeyManagerUserKeyTypeType
```

definitions for HSM_KeyManagerEnDecryAlgType

```
#define KEYMANAGER_ENDECRY_ALG_CTR (0u)
```

```
#define KEYMANAGER_ENDECRY_ALG_CBC (1u)
#define KEYMANAGER_ENDECRY_ALG_ECB (2u)
#define KEYMANAGER_ENDECRY_ALG_CFB (3u)
#define KEYMANAGER_ENDECRY_ALG_OFB (4u)
#define KEYMANAGER_ENDECRY_ALG_MAX (5u)
typedef hsm_uint32_t HSM_KeyManagerEnDecryAlgType
```

definitions for HSM_AesKeyType

```
#define HSM_AES_KEY_CHIP_VENDOR_IFR (0U) /* key has existed in Flash IFR, it provided by chip
hardware, not changeable */
vendor key value for type "HSM_AesKeyType"
```

```
#define HSM_AES_KEY_NEW (1U) /* user should provide the new key */
driver user's new key value for type "HSM_AesKeyType"
```

```
typedef hsm_uint32_t HSM_AesKeyType
type definition for the key source
```

definitions for HSM_AesKeyBitCntType

```
#define HSM_AES_KEY128 (0U)
AES128 key type value for type "HSM_AesKeyBitCntType".
```

```
#define HSM_AES_KEY192 (1U)
AES192 key type value for type "HSM_AesKeyBitCntType".
```

```
#define HSM_AES_KEY256 (2U)
AES256 key type value for type "HSM_AesKeyBitCntType".
```

```
typedef hsm_uint32_t HSM_AesKeyBitCntType
type definition for AES key
```

Detailed Description

HSM driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

Macro Definition Documentation

```
#define TYPE_CONVERT( from_type, to_type, data) ((to_type)(data))
```

Typedef Documentation

```
typedef int32_t hsm_int32_t
```

```
typedef int8_t hsm_int8_t
```

```
typedef uint16_t hsm_uint16_t
```

```
typedef uint32_t hsm_uint32_t
```

```
typedef uint8_t hsm_uint8_t
```

Function Documentation

```
HSM_StatusType HSM_AesDecrypt (HSM_CmdType * pCmd, const HSMCom_AesDecryptType * pCfg)
```

AES decrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_AesDecryptEx (HSM_CmdType * *pCmd*, const HSMCom_FlexAesDecryptType * *pCfg*)

AES decrypt extend.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_AesEncrypt (HSM_CmdType * *pCmd*, const HSMCom_AesEncryptType * *pCfg*)

AES encrypt API.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_AesEncryptEx (HSM_CmdType * *pCmd*, const HSMCom_FlexAesEncryptType * *pCfg*)

AES encrypt extend API.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_BigNumberCalculateEx (HSM_CmdType * *pCmd*, const HSMCom_BigNumberCalcType * *pCfg*)

Big number calculate extend.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_CancelJob (HSM_CmdType * *pCmd*, const HSMCom_CancelJobType * *pCfg*)

cancel the current job of the channel

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_CcmDecry (HSM_CmdType * *pCmd*, const HSMCom_CcmDecryptType * *pCfg*)

CCM decrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_BUSY when driver is busy.
Others, some error occur.

HSM_StatusType HSM_CcmEncry (HSM_CmdType * *pCmd*, const HSMCom_CcmEncryptType * *pCfg*)

CCM encrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_BUSY when driver is busy.
Others, some error occur.

HSM_StatusType HSM_CMac (HSM_CmdType * *pCmd*, const HSMCom_CMacType * *pCfg*)

CMAC.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_BUSY when driver is busy.
Others, some error occur.

HSM_StatusType HSM_CMacEx (HSM_CmdType * *pCmd*, const HSMCom_CMacExType * *pCfg*)

CMAC extend.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_BUSY when driver is busy.
Others, some error occur.

HSM_StatusType HSM_EccCalculateEx (HSM_CmdType * *pCmd*, const HSMCom_EccCalcType * *pCfg*)

ECC calculate Externd.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_EccEasyDecry (HSM_CmdType * *pCmd*, const HSMCom_EccEasyDecryType * *pCfg*)

ECC decrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

HSM_StatusType HSM_EccVerify (HSM_CmdType * *pCmd*, const HSMCom_EccVerifyType * *pCfg*)

ECC Verify.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

HSM_StatusType HSM_FastTrueRandGet (HSM_CmdType * *pCmd*, const HSMCom_FastTrueRandType * *pCfg*)

Get the fast trng result.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

HSM_StatusType HSM_FW_BigNumberCalculateEx (HSM_CmdType * *pCmd*, const HSMCom_BigNumberCalcExtType * *pCfg*)

Big Number calculate.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_CheckKeyIdExt (HSM_CmdType * *pCmd*, const HSMCom_UserKeyManageType * *pCfg*)

Check key ID.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_CopyKeyExt (HSM_CmdType * *pCmd*, const HSMCom_UserKeyManageType * *pCfg*)

Copy key.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_EccEncrypt (HSM_CmdType * *pCmd*, const HSMCom_EccEasyEncryType * *pCfg*)

ECC encrypt.

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_FW_EccSign (HSM_CmdType * *pCmd*, const HSMCom_EccSignType * *pCfg*)

ECC Sign.

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_FW_GetKeyIdStatus (HSM_CmdType * *pCmd*, const HSMCom_UserKeyManageType * *pCfg*)

Get key Id status.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_Md5Ex (HSM_CmdType * *pCmd*, const HSMCom_Md5ExType * *pCfg*, HSM_BoolType *bKeyUsed*)

MD5 extend.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_FW_RsaSsaPkcs1v15Sign (HSM_CmdType * *pCmd*, const HSMCom_RsaSsaPkcs1v15SignType * *pCfg*)

RsaSsa-pss-Pkcs-1v15 verify.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_RsaSsaPkcs1v15Verify (HSM_CmdType * *pCmd*, const HSMCom_RsaSsaPkcs1v15VerifyType * *pCfg*)

RsaSsa-pkcs1-v1_5 verify.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_RsaSsaPssSign (HSM_CmdType * *pCmd*, const HSMCom_RsaSsaPssSignType * *pCfg*)

RsaSsa-pss verify.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

**HSM_StatusType HSM_FW_RsaSsaPssVerify (HSM_CmdType * *pCmd*, const
HSMCom_RsaSsaPssVerifyType * *pCfg*)**

RsaSsa-pss verify.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

**HSM_StatusType HSM_FW_ScatterCMacFinal (HSM_CmdType * *pCmd*, const HSMCom_AesmRawApiType
* *pCfg*, HSM_DataFormatType *eInputFmt*, HSM_DataFormatType *eKeyFmt*)**

Scatter cmac final.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

**HSM_StatusType HSM_FW_ScatterCMacInit (HSM_CmdType * *pCmd*, const HSMCom_AesmRawApiType *
pCfg, HSM_DataFormatType *eInputFmt*, HSM_DataFormatType *eKeyFmt*)**

Scatter cmac init.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_ScatterCMacUpdate (HSM_CmdType * *pCmd*, const HSMCom_AesmRawApiType * *pCfg*, HSM_DataFormatType *eInputFmt*, HSM_DataFormatType *eKeyFmt*)

Scatter cmac update.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_ScatterHashFinal (HSM_CmdType * *pCmd*, const HSMCom_ScatterHashType * *pCfg*, HSM_BoolType *bKeyUseId*, HSM_DataFormatType *eInputFmt*)

Scatter hash final.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_FW_ScatterHashInit (HSM_CmdType * *pCmd*, const HSMCom_ScatterHashType * *pCfg*, HSM_BoolType *bKeyUseId*, HSM_DataFormatType *eInputFmt*)

Scatter hash init.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

**HSM_StatusType HSM_FW_ScatterHashUpdate (HSM_CmdType * *pCmd*, const
HSMCom_ScatterHashType * *pCfg*, HSM_BoolType *bKeyUseld*, HSM_DataFormatType *eInputFmt*)**

Scatter hash update.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

**HSM_StatusType HSM_FW_ShaEx (HSM_CmdType * *pCmd*, const HSMCom_ShaExType * *pCfg*,
HSM_BoolType *bKeyUseld*)**

SHA Extend.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

HSM_StatusType HSM_FW_Sm2Encry (HSM_CmdType * *pCmd*, const HSMCom_Sm2EncryptType * *pCfg*)

SM2 encrypt.

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_FW_Sm2GenKeyPair (HSM_CmdType * *pCmd*, const HSMCom_Sm2GenKeyPairType * *pCfg*)

SM2 key pair generate.

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_FW_Sm2Sign (HSM_CmdType * *pCmd*, const HSMCom_Sm2SignType * *pCfg*)

SM2 sign.

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

**HSM_StatusType HSM_FW_Sm3Ex (HSM_CmdType * *pCmd*, const HSMCom_Sm3ExType * *pCfg*,
HSM_BoolType *bKeyUsed*)**

SM3 extend.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

**HSM_StatusType HSM_FW_UserKeyGenPair (HSM_CmdType * *pCmd*, const
HSMCom_UserKeyManageType * *pCfg*)**

Generate SM2/ECC key pair(private key and public key)

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
-------------	--

<i>pCfg</i>	user key pair generate parameters for trigger the process
-------------	---

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

**HSM_StatusType HSM_FW_UserKeyImportNew (HSM_CmdType * *pCmd*, const
HSMCom_UserKeyManageType * *pCfg*)**

Userkey Import.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

**HSM_StatusType HSM_FW_UserKeyRevoke (HSM_CmdType * *pCmd*, const
HSMCom_UserKeyManageType * *pCfg*)**

user key revoke(delete ram key or abandon key in flash)

Note

this function is implemented in firmware

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

HSM_StatusType HSM_GcmDecry (HSM_CmdType * *pCmd*, const HSMCom_GcmDecryptType * *pCfg*)

GCM decrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_BUSY when driver is busy.
Others, some error occur.

HSM_StatusType HSM_GcmEncry (HSM_CmdType * *pCmd*, const HSMCom_GcmEncryptType * *pCfg*)

GCM encrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_BUSY when driver is busy.
Others, some error occur.

HSM_StatusType HSM_GetKeySpaceStat (HSM_CmdType * *pCmd*, const HSMCom_KeySpaceStatusType * *pCfg*)

get the user key space status in HSM internal

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_InFieldEnter (HSM_CmdType * *pCmd*, const HSMCom_AuthCheckType * *pCfg*)

Life cycle change to In Field(need call HSM_LifeCycleTakeEffect take effect)

Note

You can't access the key data in this life cycle, before you enter this, you MUST program you key data. Or you will need to send the chip to us if you program has bugs.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
-------------	--

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_LifeCycleTakeEffect (HSM_CmdType * *pCmd*)

Life cycle change take effect.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
-------------	--

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_LoadFirmware (HSM_CmdType * *pCmd*, HSMCom_LoadFirmwareType * *pCfg*)

trigger firware take effect when the firmware is in delay load mode

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_Md5 (HSM_CmdType * *pCmd*, const HSMCom_Md5Type * *pCfg*)

MD5.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

**HSM_StatusType HSM_MonotonicCounterIncrease (HSM_CmdType * *pCmd*, const
HSMCom_MonCountType * *pCfg*)**

monotonic counter increase 1

Note

this counter keep value until chip power off

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

**HSM_StatusType HSM_MonotonicCounterRead (HSM_CmdType * *pCmd*, const HSMCom_MonCountType
* *pCfg*)**

read monotonic counter value

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_NvrOtpProgram (HSM_CmdType * *pCmd*, const HSMCom_NvrOtpType * *pCfg*)

NVR OTP program.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_NvrOtpRead (HSM_CmdType * *pCmd*, const HSMCom_NvrOtpType * *pCfg*)

NVR OTP data read.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_OemDevEnter (HSM_CmdType * *pCmd*, const HSMCom_AuthCheckType * *pCfg*)

Life cycle change to OEM DEV(need call HSM_LifeCycleTakeEffect to take effect)

Note

You can read/write/erase the USRK,UMRK,DebugAuth,IspAuth,PreFaAuth key data in the data flash in this life cycle

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
-------------	---

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_OemPdtEnter (HSM_CmdType * *pCmd*, const HSMCom_AuthCheckType * *pCfg*)

Life cycle change to OEM PDT(need call HSM_LifeCycleTakeEffect take effect)

Note

You can only write the USRK,UMRK,DebugAuth,IspAuth,PreFaAuth key data in the data flash in this life cycle

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
-------------	---

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_RequestAuthorization (HSM_CmdType * *pCmd*, const HSMCom_RequestAuthType * *pCfg*)

require challenge data for auth

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Rsa (HSM_CmdType * *pCmd*, const HSMCom_RsaType * *pCfg*)

RSA.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. Others, some error occur.

HSM_StatusType HSM_SelfTest (HSM_CmdType * *pCmd*, volatile HSMCom_SelfTestType * *pCfg*)

hsm self test

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sha (HSM_CmdType * *pCmd*, const HSMCom_ShaType * *pCfg*)

SHA.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm2Decry (HSM_CmdType * *pCmd*, const HSMCom_Sm2DecryptType * *pCfg*)

SM2 decrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm2GenHash (HSM_CmdType * *pCmd*, const HSMCom_Sm2GenHashType * *pCfg*)

SM2 Hash generate.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm2GenZa (HSM_CmdType * *pCmd*, const HSMCom_Sm2GenZaType * *pCfg*)

SM2 ZA.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm2Verify (HSM_CmdType * *pCmd*, const HSMCom_Sm2VerifyType * *pCfg*)

SM2 verify.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm3 (HSM_CmdType * *pCmd*, const HSMCom_Sm3Type * *pCfg*)

SM3.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm4Decrypt (HSM_CmdType * *pCmd*, const HSMCom_Sm4DecryptType * *pCfg*)

SM4 Decrypt.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm4DecryptEx (HSM_CmdType * *pCmd*, const HSMCom_Sm4DecryptExType * *pCfg*)

SM4 Decrypt extend.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm4EncryptExStart (HSM_CmdType * *pCmd*, HSMWrap_Sm4EncryptExType * *pCtx*, HSMCom_Sm4EncryptExType * *pCfg*)

SM4 Encrypt extend start.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm4EncryptExUpdate (HSM_CmdType * *pCmd*, HSMWrap_Sm4EncryptExType * *pCtx*)

SM4 Encrypt extend update.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm4EncryptStart (HSM_CmdType * *pCmd*, HSMWrap_Sm4EncryptType * *pCtx*, HSMCom_Sm4EncryptType * *pDecrypt*)

SM4 Encrypt start.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_Sm4EncryptUpdate (HSM_CmdType * *pCmd*, HSMWrap_Sm4EncryptType * *pCtx*)

SM4 Encrypt update.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_TidyUpKeySpace (HSM_CmdType * *pCmd*, const HSMCom_KeySpaceStatusType * *pCfg*)

tidy the user key space in HSM internal, if revoked key exist, it will make the space re-useable

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_TrueRandGet (HSM_CmdType * *pCmd*, const HSMCom_TrueRandExType * *pCfg*)

Get the random data from TRNG 0 or TRNG 1.

Note

suggest use HSM_TrueRandGetSrcShiftXor to get true random fast

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_TrueRandGetAny (HSM_CmdType * *pCmd*, const HSMCom_TrueRandType * *pCfg*)

Get the random data from any TRNG.

Note

suggest use HSM_TrueRandGetSrcShiftXor to get true random fast

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_TrueRandGetSrc0 (HSM_CmdType * *pCmd*, const HSMCom_TrueRandType * *pCfg*)

Get the true random data from TRNG 0.

Note

suggest use HSM_TrueRandGetSrcShiftXor to get true random fast

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_TrueRandGetSrc1 (HSM_CmdType * *pCmd*, const HSMCom_TrueRandType * *pCfg*)

Get the true random data from TRNG 1.

Note

suggest use HSM_TrueRandGetSrcShiftXor to get true random fast

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_TrueRandGetSrcShiftXor (HSM_CmdType * *pCmd*, const HSMCom_TrueRandType * *pCfg*)

Get the random data from ((TRNG 0 << 1) xor TRNG 1)

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_TrueRandGetSrcXor (HSM_CmdType * *pCmd*, const HSMCom_TrueRandType * *pCfg*)

Get the true random data from (TRNG 0 xor TRNG 1)

Note

suggest use HSM_TrueRandGetSrcShiftXor to get true random fast

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

**HSM_StatusType HSM_UserKeyExport (HSM_CmdType * *pCmd*, const HSMCom_UserKeyManageType *
pCfg)**

user key export

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR,
parameter error.

**HSM_StatusType HSM_UserKeyImport (HSM_CmdType * *pCmd*, const HSMCom_UserKeyManageType *
pCfg)**

user key import

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_UserKeyRevoke (HSM_CmdType * *pCmd*, const HSMCom_UserKeyManageType * *pCfg*)

Revoke user key.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_PARAM_ERR, parameter error.

HSM_StatusType HSM_XMac (HSM_CmdType * *pCmd*, const HSMCom_XMacType * *pCfg*)

XMAC.

Parameters

<i>pCmd</i>	point to the struct variable used to configure the mailbox information send to HSM core
<i>pCfg</i>	the struct variable address to store the result

Returns

HSM_StatusType HSM_STATUS_SUCCESS when succeed. HSM_STATUS_BUSY when driver is busy.
Others, some error occur.

module_driver_hsm_level1_leveln.h File Reference

HSM driver type definition and API.

#include "module_driver_hsm.h"

Include dependency graph for module_driver_hsm_level1_leveln.h:

IMAGE

Classes

struct **HSM_L1_EccCurveParamType**

Ecc curve type.

struct **HSM_L1_RsaType**

L1 Rsa config type.

struct **HSM_Ln_EccSignType**

Ecc sign config type.

struct **HSM_DrvRandPoolType**

Drive rand pool type.

struct **HSM_DrvRsaInfType**

Rsa information type.

struct **HSM_DrvMd5InfType**

Md5 information type.

struct **HSM_DrvSm3InfType**

Sm3 information type.

struct **HSM_DrvShaInfType**

Sha information type.

struct **HSM_DrvTrngInfType**

Trng information type.

struct **HSM_DrvEccVerifyInfType**

Ecc verify information type.

struct **HSM_DrvEccDecryptInfType**

Ecc decrypt information type.

struct **HSM_DrvCMacInfType**

Cmac information type.

struct **HSM_DrvAesEncryptInfType**
Aes encrypt information type.

struct **HSM_DrvAesDecryptInfType**
Aes decrypt information type.

struct **HSM_DrvSm4EncryptInfType**
Sm4 encrypt information type.

struct **HSM_DrvSm4DecryptInfType**
Sm4 decrypt information type.

struct **HSM_DrvBnCalcInfType**
Big number calculator information type.

struct **HSM_Ln_HashMacCfgType**
Ln hash mac config type.

struct **HSM_DrvScatterHashInfType**
Scatter hash information type.

struct **HSM_DrvScatterCMacInfType**
Scatter SM CMac information type.

struct **HSM_DrvEccSignInfType**
Ecc sign information type.

struct **HSM_L1_DrvRSASsaPssVerifyType**
Level 1 RSAssa Pss verify type.

struct **HSM_L1_DrvRSASsaPkcs1V15VerifyType**
Level 1 RSAssa Pkcs15 verify type.

struct **HSM_DrvEccCalcInfType**
ECC calculate info type.

struct **HSM_DrvBnCalcExtInfType**

struct **HSM_DrvKeyManagelInfType**

struct **HSM_DrvKeySpaceInfType**

struct **HSM_L1_DrvRSASsaPkcs1V15SignType**

struct **HSM_L1_DrvRSASsaPssSignType**

struct **HSM_DrvLoadFirmwareInfType**

struct **HSM_L1_Sm2VerifyType**

struct **HSM_DrvSm2VerifyInfType**

struct **HSM_DrvSm2GenZaInfType**

struct **HSM_LN_Sm2GenZaType**

struct **HSM_L1_RtCtxType_tag**

The structure of the HSM level 1 processing handle.

struct **HSM_Ln_AesEncryptType**

The structure of the aes encrypt type.

struct **HSM_L2_DrvCMacEmptyInfType**

The structure of the HSM level 2 processing handle.

struct **HSM_L2_DrvUpdateKeyInfType**

struct **HSM_L2_DrvRevokeKeyInfType**

struct **HSM_Ln_EccDecryptType**

Ecc decrypt type.

struct **HSM_L1_EccVerifyType**

struct **HSM_Ln_EccVerifyType**

Ecc verify type.

struct **HSM_Ln_ShaType**

Sha information used by driver when call specific API.

struct **HSM_L2_DrvEccVerifyInfType**

struct **HSM_Ln_ScatterHashType**

Scatter hash level n type.

struct **HSM_Ln_ResultBufInfoType**

HSM result buffer type.

struct **HSM_Ln_Sm2VerifyType**

struct **HSM_L2_DrvSm2VerifyInfType**

struct **HSM_L2_RtCtxType_tag**

The structure of the HSM level 2 processing handle.

struct **HSM_Ln_CMacCfgParmsType**

The structure of the level 3 cmac config type.

struct **HSM_L3_DrvScatterCMacInfType**

The structure of the level 3 scatter cmac information type.

struct **HSM_L3_DrvUpdateKeyInfType**

struct **HSM_L3_RtCtxType_tag**

The structure of the HSM level 3 processing handle.

struct **HSM_Ln_RtCtxType**

The structure of the HSM processing handle.

struct **HSM_Ln_InitParmType**

init information for HSM_Ln_Init

struct **HSM_Ln_RsaSsaPssVerifyType**

RSA PSS signature verify information used by driver when call specific API.

struct **HSM_Ln_RsaSsaPkcs1v15VerifyType**

RSA PKCS1 V1.5 signature verify information used by driver when call specific API.

struct **HSM_Ln_AesDecryptType**

The structure of the aes decrypt type.

struct **HSM_Ln_Md5Type**

MD5 information used by driver when call specific API.

struct **HSM_Ln_Sm3Type**

Sm3 information used by driver when call specific API.

struct **HSM_Ln_CMacType**

CMAC information used by driver when call specific API.

struct **HSM_Ln_RngType**

Rng information used by driver when call specific API.

struct **HSM_Ln_ScatterCMacType**

Scatter cmac information used by driver when call specific API.

struct **HSM_L1_InitParmType**

Ecc level 1 init type.

struct **HSM_L2_InitParmType**

Ecc level 2 init type.

struct **HSM_L3_InitParmType**

Ecc level 3 init type.

struct **HSM_Ln_PlainKeyImport**

struct **HSM_Ln_RsaSsaPkcs1V15SignDataType**

struct **HSM_Ln_RsaSsaPkcs1V15SignHashType**

struct **HSM_Ln_RsaSsaPssSignDataType**

struct **HSM_Ln_RsaSsaPssSignHashType**

struct **HSM_L1_PKCS_RsaPrivateKeyInfType**

struct **HSM_L1_PKCS_EccPrivateKeyInfType**

struct **HSM_L1_PKCS8_PrivateKeyInfType**

struct **HSM_L1_PKCS_RsaPublicKeyInfType**

struct **HSM_L1_PKCS_EccPublicKeyInfType**

struct **HSM_L1_PublicKeyInfType**

struct **HSM_L1_BnCalcType**

struct **HSM_Ln_EccKPTType**

Macros

```
#define hsm_hw_swap_u32(x)  ({hsm_uint32_t y; asm volatile ("rev %0, %1\n" : "=r"(y) : "r"(x)); y;})
#define HSM_KEY_LOCATION_NVR_RAM  (0x1u)
#define HSM_KEY_LOCATION_NVR_FLASH  (0x0u)
#define HSM_KEY_ID(id, location)  (((hsm_uint32_t)(id)) << 1u) | (((hsm_uint32_t)(location)) & 0x1u)
#define HSM_ADDRESS_UP_ALIGN(addr)  (((hsm_uint32_t)(addr)) + 0x3u) & ~(hsm_uint32_t)0x3u)
#define HSM_4B_BYTE_INDEX_TO_OFFSET(idx)  (((idx) & 0xFFFFFFFcu) + (3u - ((idx) & 0x3u)))
#define HSM_HFAM_HW_PROCESS_ALIGN(x)  128u
#define HSM_AESM_SCATTER_CMACHW_PROCESS_ALIGN()  16u
#define CRYPTO_TEXT_SECTION
    Section define.

#define HSM_AESM_BACKEND_AES  0x0
    AESM BACKEND is aes.

#define HSM_AESM_BACKEND_SM4  0x1
    AESM BACKEND is SM4.

#define HSM_L1_PKCS8_RSA_PRIVATE_KEY  0
    Trng random number count.

#define HSM_L1_PKCS8_ECC_PRIVATE_KEY  1
#define HSM_L1_RSA_PUBLIC_KEY  0
#define HSM_L1_ECC_PUBLIC_KEY  1
#define HSM_ECC_VERIFY_INPUT_RAW_MESSAGE  0u
#define HSM_ECC_VERIFY_INPUT_HASH_DATA  1u
#define HSM_SM2_USERID_MAX_LENGTH  (128U)/*max length of user id, for mcal ,this parameter is
    configed by user*/
#define HSM_SM2_VERIFY_INPUT_RAW_MESSAGE  0u
    HSM_ECC_VERIFY_INPUT_RAW_MESSAGE value for type "HSM_EccVerifyInputType".

#define HSM_SM2_VERIFY_INPUT_HASH_DATA  1u
    HSM_ECC_VERIFY_INPUT_HASH_DATA value for type "HSM_EccVerifyInputType".
```

Typedefs

typedef **hsm_uint32_t** **HSM_AesmBackendType**

AESM BACKEND Type.

typedef **hsm_uint32_t** **HSM_L1_PKCS8_PrivateKeyType**

typedef **hsm_uint32_t** **HSM_L1_PublicKeyType**

typedef **hsm_uint32_t** **HSM_EccVerifyInputType**

typedef void(* **HSM_CleanDcacheCbType**) (void)

Type definition for algorithm stop callback registered in HSM_Ln_Init API.

typedef void(* **HSM_CleanAndFlushDcacheCbType**) (void)

Type definition for algorithm stop callback registered in HSM_Ln_Init API.

typedef void(* **HSM_StopCbType**) (void *pLnCtx, **HSM_BoolType** bErrorHappen, **HSM_StatusType** eRet)

Type definition for algorithm stop callback registered in HSM_Ln_Init API.

typedef **HSM_StatusType**(* **HSM_SendCommandCbType**) (**HSM_CmdType** *)

Type definition for send command callback registered in HSM_Ln_Init API.

typedef **HSM_StatusType**(* **HSM_GetAckStatCbType**) (void)

Type definition for get ack callback registered in HSM_Ln_Init API.

typedef **HSM_StatusType**(* **HSM_CancelCmdCbType**) (void)

Type definition for cancel command callback registered in HSM_Init API.

typedef struct **HSM_L1_RtCtxType_tag** **HSM_L1_RtCtxType**

The structure of the HSM level 1 processing handle.

typedef **HSM_Ln_AesEncryptType** **HSM_Ln_Sm4EncryptType**

SM4 Encrypt config type.

typedef **hsm_uint32_t** **HSM_Sm2VerifyInputType**

typedef struct **HSM_L2_RtCtxType_tag** **HSM_L2_RtCtxType**

The structure of the HSM level 2 processing handle.

typedef struct **HSM_L3_RtCtxType_tag** **HSM_L3_RtCtxType**

The structure of the HSM level 3 processing handle.

typedef **HSM_Ln_AesDecryptType** **HSM_Ln_Sm4DecryptType**

The structure of the sm4 decrypt type.

Enumerations

enum **HSM_L1_DrvSMType** { **HSM_DRV_SM_READY** = 0, **HSM_DRV_SM_ARITH**, **HSM_DRV_SM_STOP** }

Level 1 drive SM type.

enum **Crypto_VerifyResultType** { **CRYPTO_E_VER_OK** = 0x00U, **CRYPTO_E_VER_NOT_OK** = 0x01U }

verify result type

enum **HSM_ScatterHashSMType** { **HSM_L1_SCATTER_HASH_READY**,
HSM_L1_SCATTER_HASH_NEED_MORE_BEFORE_INIT, **HSM_L1_SCATTER_HASH_INIT**,
HSM_L1_SCATTER_HASH_INIT_OK, **HSM_L1_SCATTER_HASH_NEED_MORE_BEFORE_UPDATE_FINAL**,
HSM_L1_SCATTER_HASH_UPDATE, **HSM_L1_SCATTER_HASH_UPDATE_OK**,
HSM_L1_SCATTER_HASH_FINAL, **HSM_L1_SCATTER_HASH_STOP** }

scatter hash sm type

enum **HSM_ScatterCMacSMType** { **HSM_L1_SCATTER_CMACE_READY**,
HSM_L1_SCATTER_CMACE_NEED_MORE_BEFORE_INIT, **HSM_L1_SCATTER_CMACE_INIT**,
HSM_L1_SCATTER_CMACE_INIT_OK, **HSM_L1_SCATTER_CMACE_NEED_MORE_BEFORE_UPDATE_FINAL**,
HSM_L1_SCATTER_CMACE_UPDATE, **HSM_L1_SCATTER_CMACE_UPDATE_OK**,
HSM_L1_SCATTER_CMACE_FINAL, **HSM_L1_SCATTER_CMACE_STOP** }

scatter hash cmac type

enum **HSM_L2_DrvOpType** { **HSM_LEVEL_2ND_OP_NONE** = 0, **HSM_LEVEL_2ND_OP_CMACE_EMPTY**,
HSM_LEVEL_2ND_OP_UPDATE_KEY, **HSM_LEVEL_2ND_OP_REVOKE_KEY**,
HSM_LEVEL_2ND_OP_ECC_VERIFY, **HSM_LEVEL_2ND_OP_SM2_VERIFY** }

Level 2 operation type.

enum **HSM_L2_DrvCMacEmptySMType** { **HSM_L2_CMACE_EMPTY_SM_READY**,
HSM_L2_CMACE_EMPTY_SM_GENERATE_L, **HSM_L2_CMACE_EMPTY_SM_GENERATE_T**,
HSM_L2_CMACE_EMPTY_SM_STOP }

Level 2 cmac empty sm type.

enum **HSM_L3_DrvOpType** { **HSM_LEVEL_3RD_OP_NONE** = 0, **HSM_LEVEL_3RD_OP_SCATTER_CMACE**,
HSM_LEVEL_3RD_OP_UPDATE_KEY }

Level 3 operation type.


```
enum HSM_L3_ScatterCMacSMType { HSM_L3_SCATTER_CMIC_READY,
HSM_L3_SCATTER_CMIC_EMPTY_INPUT, HSM_L3_SCATTER_CMIC_INIT,
HSM_L3_SCATTER_CMIC_NEED_MORE_AFTER_INIT, HSM_L3_SCATTER_CMIC_INIT_OK,
HSM_L3_SCATTER_CMIC_UPDATE, HSM_L3_SCATTER_CMIC_NEED_MORE_AFTER_UPDATE,
HSM_L3_SCATTER_CMIC_UPDATE_OK, HSM_L3_SCATTER_CMIC_FINAL,
HSM_L3_SCATTER_CMIC_STOP }
```

Level 3 scatter cmic sm type.

```
enum HSM_Ln_KeyDataStoreFmtType { HSM_LN_KEY_DATA_STORE_FMT_BN_LOADABLE = 0,
HSM_LN_KEY_DATA_STORE_FMT_4B, HSM_LN_KEY_DATA_STORE_FMT_1B,
HSM_LN_KEY_DATA_STORE_FMT_SPLIT_BN_LOADABLE }
```

```
enum HSM_L2_DrvUpdateKeySMType { HSM_L2_UPDATE_KEY_SM_READY,
HSM_L2_UPDATE_KEY_SM_LOADFIRMWARE, HSM_L2_UPDATE_KEY_SM_CHECK_EXIST,
HSM_L2_UPDATE_KEY_SM_REVOKE, HSM_L2_UPDATE_KEY_SM_TIDYUP,
HSM_L2_UPDATE_KEY_SM_IMPORT, HSM_L2_UPDATE_KEY_SM_GET_STATUS,
HSM_L2_UPDATE_KEY_SM_STOP }
```

```
enum HSM_L2_DrvRevokeKeySMType { HSM_L2_REVOKE_KEY_SM_READY,
HSM_L2_REVOKE_KEY_SM_CHECK_EXIST, HSM_L2_REVOKE_KEY_SM_REVOKE,
HSM_L2_REVOKE_KEY_SM_STOP }
```

```
enum HSM_L2_DrvEccVerifySMType { HSM_L2_ECC_VERIFY_SM_READY,
HSM_L2_ECC_VERIFY_SM_HASH, HSM_L2_ECC_VERIFY_SM_VERIFY, HSM_L2_ECC_VERIFY_SM_STOP }
```

```
enum HSM_L2_DrvSm2VerifySMType { HSM_L2_SM2_VERIFY_SM_READY,
HSM_L2_SM2_VERIFY_SM_GEN_ZA, HSM_L2_SM2_VERIFY_SM_HASH_UPDATE,
HSM_L2_SM2_VERIFY_SM_HASH_FINALLY, HSM_L2_SM2_VERIFY_SM_VERIFY,
HSM_L2_SM2_VERIFY_SM_STOP }
```

```
enum HSM_L3_UpdateKeySMType { HSM_L3_UPDATE_KEY_READY, HSM_L3_UPDATE_KEY_PROCESS,
HSM_L3_UPDATE_KEY_STOP }
```

```
enum HSM_DrvOpCatagoryType { HSM_OP_NONE, HSM_OP_AES_ENCRYPT, HSM_OP_AES_DECRYPT,
HSM_OP_SM4_ENCRYPT, HSM_OP_SM4_DECRYPT, HSM_OP_CMIC, HSM_OP_RNG, HSM_OP_SHA,
HSM_OP_MD5, HSM_OP_SM3, HSM_OP_ECC_VERIFY, HSM_OP_ECC_EASYDECY, HSM_OP_ECC_CALC,
HSM_OP_BN_CALC, HSM_OP_SCATTER_HASH, HSM_OP_SCATTER_CMIC, HSM_OP_ECC_SIGN,
HSM_OP_RSA_SSA_PSS_VERIFY, HSM_OP_RSA_SSA_PKCS1V15_VERIFY,
HSM_OP_KEY_MANAGE_IMPORT, HSM_OP_KEY_MANAGE_VALIDATE, HSM_OP_KEY_MANAGE_COPY,
HSM_OP_KEY_MANAGE_REVOKE, HSM_OP_KEY_MANAGE_TIDYUP,
HSM_OP_KEY_MANAGE_GETSTATUS, HSM_OP_RSA_SSA_PKCS1V15_SIGN,
HSM_OP_RSA_SSA_PSS_SIGN, HSM_OP_KEY_MANAGE_GET_ID_STATUS, HSM_OP_LOAD_FIRMWARE,
HSM_OP_SM2_VERIFY, HSM_OP_SM2_GEN_ZA }
```

```
enum HFAM_MODE_E { HFA_MD5 = 0, HFA_SHA160 = 1, HFA_SHA224 = 2, HFA_SHA256 = 3, HFA_SHA384 =
4, HFA_SHA512 = 5, HFA_SHA512_224 = 6, HFA_SHA512_256 = 7, HFA_SM3 = 8 }
```

```
enum HSM_L1_BnCalcTypeType { HSM_L1_BNCALC_A_ADD_B = 0, HSM_L1_BNCALC_A_SUB_B,
HSM_L1_BNCALC_B_SUB_A, HSM_L1_BNCALC_A_x_B, HSM_L1_BNCALC_A_Exp_E,
HSM_L1_BNCALC_A_MOD, HSM_L1_BNCALC_A_RECIPROCAL }
```

Functions

```
CRYPTO_TEXT_SECTION void HSM_Ln_Init (HSM_Ln_RtCtxType *pLnRt, HSM_Ln_InitParmType
*pInitParm)
```

Init config of hsm.

```
CRYPTO_TEXT_SECTION void HSM_Ln_Ack_IrqHandler (HSM_Ln_RtCtxType *pLnCtx)
```

Interrupt handle function.

```
CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Abort (HSM_Ln_RtCtxType *pCtx)
```

HSM abort function.

```
CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifySetupEnv (HSM_Ln_RtCtxType
*pLnCtx, hsm_uint32_t u32RsaBitCnt)
```

Rsassa pss verify setup environment.

```
CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerify (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_RsaSsaPssVerifyType *pCfg)
```

Rsassa pss verify function.

```
CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifyPoll (HSM_Ln_RtCtxType *pLnCtx)
```

Rsassa pss verify poll function.

```
CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssVerifyExitEnv (HSM_Ln_RtCtxType
*pLnCtx)
```

Rsassa pss verify exit environment.

```
CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifySetupEnv
(HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32RsaBitCnt)
```

Rsassa pkcs1v15 verify setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15Verify (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_RsaSsaPkcs1v15VerifyType *pCfg)
Rsassa pkcs1v15 verify function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifyPoll (HSM_Ln_RtCtxType *pLnCtx)
Rsassa pkcs1v15 verify poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1v15VerifyExitEnv (HSM_Ln_RtCtxType *pLnCtx)
Rsassa pkcs1v15 verify exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaSetupEnv (HSM_Ln_RtCtxType *pLnCtx)
Sha verify setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaExitEnv (HSM_Ln_RtCtxType *pLnCtx)
Sha verify exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sha (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_ShaType *pSha)
Sha function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ShaPoll (HSM_Ln_RtCtxType *pLnCtx)
Sha poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CommonExitEnv (HSM_Ln_RtCtxType *pCtx)
Hsm common exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)
SM4 encrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)
SM4 encrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4Encrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Sm4EncryptType *pCfg)

SM4 encrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4EncryptPoll (HSM_Ln_RtCtxType *pLnCtx)

SM4 encrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

SM4 decrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

SM4 decrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4Decrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Sm4DecryptType *pCfg)

SM4 decrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm4DecryptPoll (HSM_Ln_RtCtxType *pLnCtx)

SM4 decrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

AES encrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

AES encrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_AesEncryptType *pCfg)

AES encrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesEncryptPoll (HSM_Ln_RtCtxType *pLnCtx)

AES encrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

AES decrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

AES decrypt exit environment.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecrypt (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_AesDecryptType *pCfg)**

AES decrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_AesDecryptPoll (HSM_Ln_RtCtxType *pLnCtx)

AES decrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5SetupEnv (HSM_Ln_RtCtxType *pLnCtx)

MD5 setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5ExitEnv (HSM_Ln_RtCtxType *pLnCtx)

MD5 exit environment.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5 (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_Md5Type *pCfg)**

MD5 function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Md5Poll (HSM_Ln_RtCtxType *pLnCtx)

MD5 poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3SetupEnv (HSM_Ln_RtCtxType *pLnCtx)

SM3 setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3ExitEnv (HSM_Ln_RtCtxType *pLnCtx)

SM3 exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3 (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Sm3Type *pCfg)

SM3 function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm3Poll (HSM_Ln_RtCtxType *pLnCtx)

SM3 poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

CMAC setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacExitEnv (HSM_Ln_RtCtxType *pLnCtx)

CMAC exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMac (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_CMacType *pCfg)

CMAC function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_CMacPoll (HSM_Ln_RtCtxType *pLnCtx)

CMAC poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Rng setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Rng exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Rng (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_RngType *pCfg)

Rng function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RngPoll (HSM_Ln_RtCtxType *pLnCtx)

Rng poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter cmac setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter cmac exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacPoll (HSM_Ln_RtCtxType *pLnCtx)

Scatter cmac poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacInit (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_CMacCfgParmsType *pParmCfg, const HSM_Ln_ScatterCMacType *pCfg)

Scatter cmac init function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacUpdate (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_ScatterCMacType *pCfg)

Scatter cmac update function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterCMacFinal (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_ScatterCMacType *pCfg, const HSM_Ln_ResultBufInfoType *pResult)

Scatter cmac final function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter hash setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Scatter hash exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashPoll (HSM_Ln_RtCtxType *pLnCtx)

Scatter hash poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashInit (HSM_Ln_RtCtxType *pLnCtx, const HSM_ShaAlgType eAlg, const HSM_Ln_ScatterHashType *pCfg)

Scatter hash init function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashUpdate (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_ScatterHashType *pCfg)**

Scatter hash update function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashFinal (HSM_Ln_RtCtxType *pLnCtx,
const HSM_Ln_ScatterHashType *pCfg, const HSM_Ln_ResultBufInfoType *pResult)**

Scatter hash final function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_ScatterHashSetMacCfg (HSM_Ln_RtCtxType
*pLnCtx, HSM_Ln_HashMacCfgType *pCfg)**

Scatter hash set mac config function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignSetupEnv (HSM_Ln_RtCtxType *pLnCtx,
hsm_uint32_t u32BitCnt)**

Ecc sign setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Ecc sign exit environment.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSign (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_EccSignType *pCfg)**

Ecc sign function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccSignPoll (HSM_Ln_RtCtxType *pLnCtx)

Ecc sign poll function.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptSetupEnv (HSM_Ln_RtCtxType *pLnCtx,
hsm_uint32_t u32BitCnt)**

Ecc decrypt setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Ecc decrypt exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecrypt (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_EccDecryptType *pCfg)

Ecc decrypt function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccDecryptPoll (HSM_Ln_RtCtxType *pLnCtx)

Ecc decrypt poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifySetupEnv (HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32BitCnt)

Ecc verify setup environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Ecc verify exit environment.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerify (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_EccVerifyType *pCfg)

Ecc verify function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_EccVerifyPoll (HSM_Ln_RtCtxType *pLnCtx)

Ecc verify poll function.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImportSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for import keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImport (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_ImportUserKeyExtType *pCfg)

Import key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyImportExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for import keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidateSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for valid keys.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidate (HSM_Ln_RtCtxType *pLnCtx,
hsm_uint32_t u32KeyId)**

Valid key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyValidateExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for valid keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for copy keys.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopy (HSM_Ln_RtCtxType *pLnCtx, const
HSM_Ln_CopyUserKeyType *pCfg)**

Copy keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyCopyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for copy keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevokeSetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for revoke keys.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevoke (HSM_Ln_RtCtxType *pLnCtx,
hsm_uint32_t u32KeyId)**

Revoke keys.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeyRevokeExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for revoke keys.

**CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUpSetupEnv (HSM_Ln_RtCtxType
*pLnCtx)**

Setup an environment for tidy up key space.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUp (HSM_Ln_RtCtxType *pLnCtx, HSM_Ln_KeySpaceStatusType *pCfg)

Tidy up key space.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_KeySpaceTidyUpExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for tidy up key space.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKeySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for update key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKey (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_PlainKeyImport *pCfg)

Update key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_UpdateKeyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for update key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKeySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

Setup an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKey (HSM_Ln_RtCtxType *pLnCtx, hsm_uint32_t u32KeyId)

Revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RevokeKeyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

Exit an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignSetupEnv (HSM_Ln_RtCtxType *pCtx)

Setup an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignData (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPkcs1V15SignDataType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss-Pkcs-1v15 sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignHash (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPkcs1V15SignHashType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss-Pkcs-1v15 sign hash.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPkcs1V15SignExitEnv (HSM_Ln_RtCtxType *pCtx)

Exit an environment for revoke key.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignSetupEnv (HSM_Ln_RtCtxType *pCtx)

Setup an environment for RsaSsa-pss sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignData (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPssSignDataType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignHash (HSM_Ln_RtCtxType *pCtx, const HSM_Ln_RsaSsaPssSignHashType *pCfg, hsm_uint32_t u32TimeoutMs)

RsaSsa-pss sign hash.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_RsaSsaPssSignExitEnv (HSM_Ln_RtCtxType *pCtx)

Exit an environment for RsaSsa-pss sign.

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifySetupEnv (HSM_Ln_RtCtxType *pLnCtx)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifyExitEnv (HSM_Ln_RtCtxType *pLnCtx)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2Verify (HSM_Ln_RtCtxType *pLnCtx, const HSM_Ln_Sm2VerifyType *pCfg)

CRYPTO_TEXT_SECTION HSM_StatusType HSM_Ln_Sm2VerifyPoll (HSM_Ln_RtCtxType *pLnCtx)

Detailed Description

HSM driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_intm.h File Reference

INTM driver type definition and API.

```
#include "HwA_intm.h"
```

Include dependency graph for module_driver_intm.h:

IMAGE

Classes

struct **_INTM_HandleType**

The structure of the INTM handle.

struct **INTM_InitType**

The structure of the INTM init.

Macros

INTM API Service IDs#define **INTM_INIT_ID** 0U

#define **INTM_ENABLE_ID** 1U

#define **INTM_START_INACTIVE_ID** 2U

#define **INTM_STOP_INACTIVE_ID** 3U

#define **INTM_SET_ACKNOWLEDGE_ID** 4U

#define **INTM_GET_COUNTER_ID** 5U

#define **INTM_CLEAR_INT_FLAG_ID** 6U

#define **INTM_GET_TIMEOUT_STAT_ID** 7U

#define **INTM_IRQ_PRCE_ID** 8U

INTM Dev Error Code*Error Code of calling INTM apis*

#define **INTM_E_PARAM_INSTANCE** 0x01U

#define **INTM_E_PARAM_POINTER** 0x02U

#define **INTM_E_PARAM_PARAM** 0x03U

#define **INTM_E_PARAM_NO_SUPPORT** 0x04U

Typedefs

typedef struct **_INTM_HandleType** **INTM_HandleType**

The structure of the INTM handle.

typedef void(* **INTM_ISRCallbackType**) (**INTM_HandleType** *pHandle)

INTM Channel ISR callback function prototype.

Enumerations

```
enum INTM_IrqMonitorType { INTM_IRQ_MONITOR_0 = 0U, INTM_IRQ_MONITOR_1,  
INTM_IRQ_MONITOR_2, INTM_IRQ_MONITOR_3, INTM_IRQ_MONITOR_MAX }
```

The INTM monitor.

```
enum INTM_InterruptModeType { INTM_INTERRUPT_MODE_ACTIVE = 0x00U,  
INTM_INTERRUPT_MODE_INACTIVE }
```

Functions

```
void INTM_Init (INTM_HandleType *pIntmHandle, const INTM_InitType *const pInitCfg)
```

Init the INTM.

```
void INTM_enable (INTM_HandleType *pIntmHandle, const bool bEnable)
```

Enable INTM.

```
void INTM_StartInactiveMode (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

Start INTM inactive mode.

```
void INTM_StopInactiveMode (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

```
void INTM_SetAcknowledge (INTM_HandleType *pIntmHandle, const uint16_t u16IrqNumber)
```

Write the interrupt acknowledge.

```
uint32_t INTM_GetCounterValue (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

Return counter value.

```
void INTM_ClearIntFlag (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType  
elrqMonitorIndex)
```

Clear the interrupt flag.

bool **INTM_GetTimeoutStatus** (INTM_HandleType *pIntmHandle, const INTM_IrqMonitorType
eIrqMonitorIndex)

Get the timeout flag. Return true if INTM_TMR value has exceeded the INTM_LATR value.

void **INTM_IRQProcess** (INTM_HandleType *pIntmHandle)

intm process interrupt function

Detailed Description

INTM driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_ism.h File Reference

ISM driver type definition and API.

#include "HwA_ism.h"

Include dependency graph for module_driver_ism.h:

IMAGE

Classes

struct **_ISM_HandleType**

struct **ISM_FpcCfgType**

struct **ISM_LamCfgType**

struct **ISM_InitCfgType**

Macros

#define **ISM_ECM_CHANNEL**(x) (1<<(x))

#define **ISM_LAM_CHANNEL**(x) (1<<(x))

#define **ISM_FPC_CHANNEL**(x) (1<<(x))

#define **ECM_INSTANCE_COUNT** (4U)

#define **ISM_COMMON_INIT_ID** 0U

#define **ISM_FPC_CONFIG_ID** 1U

#define **ISM_FPC_ENABLE_ID** 2U

#define **ISM_LAM_CONFIG_ID** 3U

#define **ISM_LAM_ENABLE_ID** 4U

#define **ISM_ECM_CONFIG_ID** 5U

#define **ISM_STATUS_FUNCTION_ERROR** 0x0U

#define **ISM_E_PARAM_CHANNEL** 0x02U

#define **ISM_E_PARAM_POINTER** 0x03U

#define **ISM_E_PARAM_COUNT** 0x04U

#define **ISM_E_PARAM_INVALID** 0x05U

Typedefs

typedef struct **_ISM_HandleType** **ISM_HandleType**

Enumerations

enum **ISM_InstanceType** { **ISM_INSTANCE_0** = 0U }

```
enum FPC_GlitchDetectType { FPC_RISING_GLITCH_DETECTED = 0x01U,  
FPC_FALLING_GLITCH_DETECTED = 0x02U }
```

Functions

```
void ISM_CommonInit (ISM_HandleType *plsmHandle, const ISM_InitCfgType *pInitConfig)
```

Init the ISM.

```
void ISM_FpcConfig (ISM_HandleType *plsmHandle, uint8_t u8FpcIndex, const ISM_FpcCfgType *pConfig)
```

Config the FPC channel.

```
void ISM_FpcEnable (ISM_HandleType *plsmHandle, uint8_t u8FpcIndex, bool bEnable)
```

Enable FPC channel.

```
void ISM_LamConfig (ISM_HandleType *plsmHandle, uint8_t u8LamIndex, const ISM_LamCfgType  
*pConfig)
```

Config the LAM channel.

```
void ISM_LamEnable (ISM_HandleType *plsmHandle, uint8_t u8LamIndex, bool bEnable)
```

Enable LAM Channel.

```
void ISM_EcmEventConfig (ISM_HandleType *plsmHandle, uint8_t u8LamIndex, uint8_t u8EcmlIndex,  
uint8_t u8EventCount)
```

Enable LAM system event.

```
uint8_t ISM_GetEcmEventHappenedChannels (void)
```

Get the channels mask of ECM that has event happened.

```
uint16_t ISM_GetLamEventHappenedChannels (void)
```

Get the channels mask of LAM that has event happened.

```
void ISM_ClearEcmEventHappenedChannels (uint8_t u8Channels)
```

Clear the channels status of ECM that has event happened.

```
void ISM_ClearLamEventHappenedChannels (uint16_t u16Channels)
```

Clear the channels status of LAM that has event happened.

void **ISM_EnableEcmSystemEvent** (uint32_t u32Channels, bool bEnable)

Enable ECM system event.

void **ISM_EnableLamSystemEvent** (uint32_t u32Channels, bool bEnable)

Enable LAM system event.

void **ISM_ClearLamStatusCounter** (uint8_t u8LamIndex)

Clears the value of the LAM counter.

uint32_t **ISM_GetLamStatusCounter** (uint8_t u8LamIndex)

Gets the value of the LAM counter when a LAM event is triggered.

void **ISM_ClearLamStatusOvfl** (uint8_t u8LamIndex)

Clears LAM Timer Overflow Flag.

bool **ISM_GetLamStatusOvfl** (uint8_t u8LamIndex)

Gets LAM Timer Overflow Flag.

void **ISMn_IRQHandler** (ISM_HandleType *plsmHandle)

ISM Interrupt Process Function.

Detailed Description

ISM driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_ldi.h File Reference

LDI driver type definition and API.

```
#include "HwA_ldi.h"
```

Include dependency graph for module_driver_ldi.h:

IMAGE

Detailed Description

LDI driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2025-06-18

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_lin.h File Reference

LIN driver type definition and API.

#include "HwA_fcuart.h"

Include dependency graph for module_driver_lin.h:

IMAGE

Classes

struct **LIN_PduType**

LIN Protocol Data Unit (PDU) structure.

struct **LIN_BaudRateCfgType**

Baudrate configuration parameters.

struct **LIN_XferStateType**

Runtime state of the LIN driver (maintained during operation)

struct **LIN_ConfigType**

LIN controller hardware configuration parameters.

struct **_LIN_HandleType**

LIN driver handle structure (entry point for all API operations)

Macros

LIN API Service IDs*Defines the service IDs for the LIN API (used in development error reporting)*

#define **LIN_INIT_ID** 0U

#define **LIN_DEINIT_ID** 1U

#define **LIN_GET_DEFAULT_CFG_ID** 2U

#define **LIN_DRV_SEND_FRAME_ID** 3U

#define **LIN_DRV_GOTO_SLEEP_ID** 4U

#define **LIN_DRV_GOTO_SLEEP_INTERNAL_ID** 5U

#define **LIN_DRV_WAKEUP_ID** 6U

#define **LIN_DRV_WAKEUP_INTERNAL_ID** 7U

#define **LIN_DRV_GET_STATUS_ID** 8U

#define **LIN_DRV_ABORT_TRANSFER_ID** 9U

#define **LIN_DRV_IRQ_HANDLE** 10U

LIN Dev Error Code*Error codes for LIN API parameter validation*

```
#define LIN_E_PARAM_INSTANCE 0x01U
#define LIN_E_PARAM_POINTER 0x02U
#define LIN_E_PARAM_ERROR_STATE 0x03U
#define LIN_E_PARAM_ERROR_CALL 0x04U
```

Typedefs

```
typedef void(* LIN_TimestampCallbackType) (uint64_t *microseconds)
```

Callback function type to get time interval in microseconds.

```
typedef uint8 LIN_FrameDlType
```

Data length of LIN frame (number of data bytes in SDU)

```
typedef uint8 LIN_FramePidType
```

LIN protected identifier (PID) type.

```
typedef void(* LIN_CallbackType) (uint8_t u8Lin, LIN_EventIdType event, LIN_PduType *pdu, uint8_t *checksum)
```

LIN driver callback function type.

```
typedef struct _LIN_HandleType LIN_HandleType
```

LIN driver handle structure (entry point for all API operations)

Enumerations

```
enum LIN_ReturnType { LIN_RET_SUCCESS = 0x00U, LIN_RET_STATE_ERROR, LIN_RET_BUSY,
LIN_RET_TIMEOUT, LIN_RET_PARAM_ERROR, LIN_RET_NOT_INIT, LIN_RET_USEED,
LIN_RET_NOT_START, LIN_RET_UNSUPPORTED }
```

LIN API return status type.

```
enum LIN_NodeType { LIN_NODE_MASTER, LIN_NODE_SLAVE }
```

LIN node type (master or slave)

```
enum LIN_EventIdType { LIN_NO_EVENT = 0x00U, LIN_WAKEUP_SIGNAL, LIN_RECV_BREAK_FIELD_OK,
LIN_SYNC_ERROR, LIN_PID_ERROR, LIN_HEADER_OK, LIN_HEADER_ERROR, LIN_FRAME_ERROR,
LIN_READBACK_ERROR, LIN_REPONSE_ERROR, LIN_CHECKSUM_ERROR, LIN_TX_COMPLETED,
LIN_RX_COMPLETED, LIN_RX_OVERRUN }
```

LIN event identifier types.

```
enum LIN_FrameStateType { LIN_FRAME_SLEEP_STATE = 0x00U, LIN_FRAME_IDLE_STATE,
LIN_FRAME_BREAK_SEND_STATE, LIN_FRAME_BREAK_RECV_STATE,
LIN_FRAME_HEADER_SEND_STATE, LIN_FRAME_HEADER_RECV_STATE,
LIN_FRAME_RESPONSE_SEND_STATE, LIN_FRAME_RESPONSE_RECV_STATE,
LIN_FRAME_RESPONSE_SEND_OK_STATE, LIN_FRAME_RESPONSE_RECV_OK_STATE }
```

LIN frame processing state definitions (describes current frame processing phase)

```
enum LIN_StatusType { LIN_NOT_OK = 0x00U, LIN_TX_HEADER_OK, LIN_TX_RESP_OK,
LIN_TX_HEADER_BUSY, LIN_TX_RESP_BUSY, LIN_TX_HEADER_ERROR, LIN_TX_RESP_ERROR,
LIN_RX_HEADER_OK, LIN_RX_RESP_OK, LIN_RX_HEADER_BUSY, LIN_RX_RESP_BUSY,
LIN_RX_HEADER_ERROR, LIN_RX_RESP_ERROR, LIN_CALLBACK_ERROR, LIN_CH_OPERATIONAL,
LIN_CH_SLEEP, LIN_CH_SLEEP_PENDING }
```

LIN node operational state definitions (describes overall node status)

```
enum LIN_FrameResponseType { LIN_FRAMERESPONSE_TX = 0, LIN_FRAMERESPONSE_RX,
LIN_FRAMERESPONSE_IGNORE }
```

Frame response types (specifies responsibility for frame response)

```
enum LIN_FrameCsModelType { LIN_ENHANCED_CS = 0U, LIN_CLASSIC_CS }
```

Checksum models for LIN frames.

Functions

API declaration for LIN driver. *LIN driver APIs*

```
LIN_ReturnType LIN_DrvInit (LIN_HandleType *pLinHandle, LIN_ConfigType *pConfig)
```

Initialize LIN controller instance.

```
void LIN_DrvDeInit (LIN_HandleType *pLinHandle)
```

De-initialize LIN controller instance.

```
void LIN_DrvGetDefaultConfig (LIN_NodeType eNodeMode, LIN_ConfigType *pConfig)
```

Initialize default LIN node configuration parameters.

```
LIN_ReturnType LIN_DrvSendFrame (LIN_HandleType *pLinHandle, LIN_PduType *pLinPdu)
```

Send a LIN frame over the bus (Master/Slave mode).

```
LIN_ReturnType LIN_DrvGoToSleep (LIN_HandleType *pLinHandle)
```

Command LIN master node to initiate sleep mode sequence.

LIN_ReturnType LIN_DrvGoToSleepInternal (LIN_HandleType *pLinHandle)

Internal implementation to transition LIN node to sleep state.

LIN_ReturnType LIN_DrvWakeup (LIN_HandleType *pLinHandle)

Send wake-up signal to LIN bus to rouse all nodes from sleep mode.

LIN_ReturnType LIN_DrvWakeupInternal (LIN_HandleType *pLinHandle)

Internal implementation to wake up LIN node from sleep state.

LIN_StatusType LIN_DrvGetStatus (LIN_HandleType *pLinHandle)

Get current LIN node operational status.

LIN_ReturnType LIN_DrvAbortTransfer (LIN_HandleType *pLinHandle)

Abort ongoing LIN transfer and reset node to operational state.

void LIN_DrvIRQHandler (LIN_HandleType *pLinHandle)

LIN controller interrupt service routine (ISR)

uint8_t LIN_DrvParityMake (uint8_t PID)

Generate LIN PID parity bits according to LIN protocol specification.

uint8_t LIN_DrvParityCheck (uint8_t PID)

Validate LIN PID parity bits according to LIN protocol specification.

Detailed Description

LIN driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-23

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

module_driver_lu.h File Reference

LU driver type definition and API.

#include "HwA_lu.h"

Include dependency graph for module_driver_lu.h:

IMAGE

Classes

struct **LU_InConfigType**

LU IN(n) configuration register type.

struct **LU_AoiConfigType**

LU AOI IN(n) configuration type.

struct **LU_InputsSyncCtrlType**

LU LG inputs synchronous control.

struct **LU_ConfigType**

LU initialization type.

Macros

```
#define LU_AOI_IN_N_CFG_N(AoiIn, AoiInN, InNCfgNType) (((uint32_t)(InNType) & 0x3U) << (((uint32_t)3U - (uint32_t)(AoiInN)) << 3U)) << (((uint32_t)3U - (uint32_t)(AoiIn)) << 3U))
#define LU_AOI_IN_N_CFG_N_MASK(AoiIn, AoiInN) (((uint32_t)0x3U << (((uint32_t)3U - (uint32_t)(AoiInN)) << 3U)) << (((uint32_t)3U - (uint32_t)(AoiIn)) << 3U))
#define LU_AOI_IN_CFG(AoiInN, CfgNType) (((uint32_t)(CfgNType) & 0x3U) << (((uint32_t)3U - (uint32_t)(AoiInN)) << 1U)))
#define LU_AOI_IN_CFG_MASK(AoiInN) (((uint32_t)0x3U) << (((uint32_t)3U - (uint32_t)(AoiInN)) << 1U)))
#define LU_AOI_IN_N_CFG(AoiIn, InNType) (((uint32_t)(InNType) & 0xFFU) << (((uint32_t)3U - (uint32_t)(AoiIn)) << 3U)))
#define LU_AOI_IN_N_CFG_MASK(AoiIn) (((uint32_t)0xFFU) << (((uint32_t)3U - (uint32_t)(AoiIn)) << 3U)))
#define LU_SYNC_CONTROL_INPUT_N(InputN, value) ((uint32_t)(value) << (InputN))

CORDIC API Service IDs#define LU_INIT_ID 0x00
#define LU_DEINIT_ID 0x01
LU Dev Error Code#define LU_E_PARAM_INSTANCE 0x01U
#define LU_E_PARAM_NULLPTR 0x02U
```

Enumerations

enum **LU_InstanceType** { **LU_INSTANCE_0** = 0U, **LU_INSTANCE_1** }

LU instance.

enum **LU_StatusType** { **LU_STATUS_SUCCESS** = 0U, **LU_STATUS_PARAM_INVALID** = 1U }

LU return structure.

enum **LU_AoiInType** { **LU_AOI_IN_0** = 0U, **LU_AOI_IN_1**, **LU_AOI_IN_2**, **LU_AOI_IN_3** }

LU IN type.

enum **LU_AoiInNType** { **LU_AOI_IN_N_A** = 0U, **LU_AOI_IN_N_B**, **LU_AOI_IN_N_C**, **LU_AOI_IN_N_D** }

LU IN(n) configuration type.

enum **LU_InModeType** { **FORCE_ITEM_AS_ZERO** = 0U, **PASS_THROUGH_ITEM**, **COMPLEMENT_ITEM**, **FORCE_ITEM_AS_ONE** }

LU IN(n) configuration.

enum **LU_OutputInitValueType** { **LU_OUTPUT_INIT_ZERO** = 0U, **LU_OUTPUT_INIT_ONE**, **LU_OUTPUT_INIT_DISABLE** }

LU output initialization value.

Functions

LU_StatusType **LU_Init** (const **LU_InstanceType** eLu, const **LU_ConfigType** *const pInitStruct)

Initialize LU instance.

void **LU_Deinit** (const **LU_InstanceType** eLu)

De-initialize LU instance.

Detailed Description

LU driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_mam.h File Reference

MAM driver type definition and API.

```
#include "HwA_mam.h"
```

Include dependency graph for module_driver_mam.h:

IMAGE

Classes

struct **MAM_Inf_Type**

the MAM slave information

Enumerations

```
enum MAM_Forbid_Access_Type { FORBID_READ_ACCESS = 1, FORBID_WRITE_ACCESS = 2,  
FORBID_EXECUTE_ACCESS = 4, FORBID_USER_ACCESS = 8 }
```

Functions

uint8_t **MAM_Enable_Wdg** (MAM_Master_Type Master, uint32_t u32Addr)

The function to enable MAM watch dog.

uint8_t **MAM_Disable_Wdg** (MAM_Master_Type Master, uint32_t u32Addr)

The function to disable MAM watch dog.

uint8_t **MAM_Config** (MAM_Master_Type Master, uint32_t u32Addr, uint32_t u32Val)

The function to config MAM.

Detailed Description

MAM driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_mb.h File Reference

Mailbox driver type definition and API.

```
#include "HwA_mb.h"
```

Include dependency graph for module_driver_mb.h:

IMAGE

Classes

struct **MB_RequestType**

Configuration of a Mailbox request.

struct **MB_ReceiveType**

Configuration of a receiving request.

struct **_MB_HandleType**

struct **MB_InitType**

Configuration of the Mailbox.

Macros

```
#define MB_CHANNEL_STATUS_SECURE 2u
```

Mailbox channel bit flag of security information.

```
#define MB_CHANNEL_STATUS_PRIVILEGED 1u
```

Mailbox channel bit flag of processing information.

Mailbox channel bit mask*Bit of channel indicate channel number*

```
#define MB_CHANNEL_MASK_NONE 0u
```

```
#define MB_CHANNEL_MASK_ALL 0xFFFFu
```

```
#define MB_CHANNEL_MASK(ch) ((uint32_t)(1ul << (ch)))
```

MB API Service IDs#

```
define MB_INIT_ID 0U
```

```
define MB_DEINIT_ID 1U
```

```
define MB_LOCK_CHANNEL_ID 2U
```

```
define MB_RELEASE_CHANNEL_ID 3U
```

```
define MB_SEND_REQUEST_ID 4U
```

```
define MB_RECEIVE_CHANNEL_ID 5U
```



```
#define MB_UNLOCK_CHANNEL_ID 6U
#define MB_POLL_DONE_ID 7U
#define MB_DONE_CHANNEL_ID 8U
```

MB Dev Error Code *Error Code of calling MB apis*

```
#define MB_E_PARAM_CHANNEL 0x02U
#define MB_E_PARAM_INVALID 0x03U
#define MB_E_PARAM_POINTER 0x04U
```

Typedefs

```
typedef struct _MB_HandleType MB_HandleType
```

Enumerations

```
enum MB_StatusType { MB_STATUS_SUCCESS = 0u, MB_STATUS_FAILED, MB_STATUS_PARAM_ERROR,
MB_STATUS_ALREADY_INITED, MB_STATUS_UNINIT, MB_STATUS_LOCKED,
MB_STATUS_NO_REQUEST }
```

Mailbox operation return values.

Functions

```
void MB_Init (MB_HandleType *pMbHandle, const MB_InitType *pInitConfig)
```

Initialize the Mailbox.

```
void MB_DeInit (MB_HandleType *pMbHandle)
```

De-initialize the Mailbox.

```
MB_StatusType MB_AcquireChannel (uint32_t u32Channel)
```

Attempt to acquire a Mailbox channel.

```
MB_StatusType MB_ReleaseChannel (MB_HandleType *pMbHandle, uint32_t u32Channel)
```

Release a Mailbox channel.

```
MB_StatusType MB_UnlockChannel (uint32_t u32Channel)
```

Software clears channel lock.

```
MB_StatusType MB_SendRequest (MB_HandleType *pMbHandle, MB_RequestType *pRequest)
```

Launching a Mailbox request.

MB_StatusType MB_ReceiveChannel (MB_HandleType *pMbHandle, MB_ReceiveType *pReceive)

Attempt to receive a request from the selected channel.

MB_StatusType MB_DoneChannel (MB_HandleType *pMbHandle, uint32_t u32Channel, uint8_t u8TargetCoreIndex)

Issue a done event to the selected channel.

uint32_t MB_PollDone (MB_HandleType *pMbHandle)

Polling the done event of all channels.

void MB_IRQProcess (MB_HandleType *pMbHandle)

Interrupt IRQ handle of Mailbox.

Detailed Description

Mailbox driver type definition and API.

Mailbox-SEMA driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_msc.h File Reference

MSC driver type definition and API.

#include "HwA_msc.h"

Include dependency graph for module_driver_msc.h:

IMAGE

Classes

struct **_MSC_HandleType**

struct **MSC_InterruptCfgType**

MSC interrupt configure structure.

struct **MSC_IOControllInitType**

struct **MSC_InitCfgType**

Macros

#define **MSC_DEV_ERROR_REPORT** STD_OFF

#define **MSC_ReportDevError**(func, error) ReportDevError(MSC_MODULE_ID, func, error)

#define **MSC_INSTANCE_0** 0U

#define **MSC_INSTANCE_1** 1U

MSC API Service IDs#define **MSC_INIT_ID** 0x00U

#define **MSC_DEINIT_ID** 0x01U

#define **MSC_INIT_STRUCTURE_ID** 0x02U

#define **MSC_INIT_INTERRUPT_ID** 0x03U

#define **MSC_SEL_TRANSMITTING_SOURCE_ID** 0x04U

#define **MSC_SET_EMERGENCY_LOAD_ID** 0x05U

#define **MSC_SET_IO_CONTROL_ID** 0x06U

#define **MSC_ENABLE_ID** 0x07U

#define **MSC_DISABLE_ID** 0x08U

#define **MSC_SET_DATA_FRAME_ID** 0x09U

#define **MSC_SEND_DATA_FRAME_ID** 0x0AU

#define **MSC_SEND_COMMAND_FRAME_ID** 0x0BU

#define **MSC_GET_RECEIVED_FRAME_ADDR_ID** 0x0CU

#define **MSC_GET_RECEIVED_FRAME_ID** 0x0DU

#define **MSC_GET_INTERRUPT_STATUS_ID** 0x0EU

#define **MSC_ENABLE_TRANSMIT_ID** 0x0FU

```

#define MSC_CLEAR_DATA_FRAME_INTERRUPT_FLAG_ID 0x10U
#define MSC_CLEAR_COMMAND_FRAME_INTERRUPT_FLAG_ID 0x11U
#define MSC_CLEAR_TIME_FRAME_INTERRUPT_FLAG_ID 0x12U
#define MSC_CLEAR_RECEIVE_INTERRUPT_FLAG_ID 0x13U
#define MSC_CLEAR_TIMEOUT_INTERRUPT_FLAG_ID 0x14U

```

FTU Dev Error Code *Error Code of calling FTU apis*

```

#define MSC_E_PARAM_INSTANCE 0x01U
#define MSC_E_PARAM_RXINDEX 0x02U
#define MSC_E_PARAM_POINTER 0x03U
#define MSC_E_PARAM_BITLENGTH 0x04U

```

Typedefs

```

typedef uint8_t MSC_InstanceType
typedef struct _MSC_HandleType MSC_HandleType
typedef void(* MSC_ISRCallbackType) (const MSC_HandleType *pMscHandle, const uint32_t u32Status)

```

MSC Channel ISR callback function prototype.

Enumerations

```

enum MSC_ReturnType { MSC_RETURN_OK = 0x00U, MSC_RETURN_E_NOT_OK = 0x01U,
MSC_RETURN_E_PARAM = 0x04U }
enum MSC_RDRxIndexType { MSC_RDR0 = 0x0U, MSC_RDR1 = 0x1U, MSC_RDR2 = 0x2U, MSC_RDR3 =
0x3U, MSC_RDRx_COUNT = 0x4U }
enum MSC_RFIEModeType { MSC_RFIE_DISABLE = 0x0U, MSC_RFIE_MODE1 = 0x1U, MSC_RFIE_MODE2 =
0x2U, MSC_RFIE_MODE3 = 0x3U }
enum MSC_DFIEModeType { MSC_DFIE_DISABLE = 0x0U, MSC_DFIE_MODE1 = 0x1U, MSC_DFIE_MODE2 =
0x2U }
enum MSC_SDISelectionType { MSC_SDI_SEL_SDI0 = 0x0U, MSC_SDI_SEL_SDI1 = 0x1U,
MSC_SDI_SEL_SDI2 = 0x2U, MSC_SDI_SEL_SDI3 = 0x3U, MSC_SDI_SEL_SDI4 = 0x4U, MSC_SDI_SEL_SDI5
= 0x5U, MSC_SDI_SEL_SDI6 = 0x6U, MSC_SDI_SEL_SDI7 = 0x7U }
enum MSC_ENCSelectionType { MSC_ENC_SEL_EN0 = 0x0U, MSC_ENC_SEL_EN1 = 0x1U,
MSC_ENC_SEL_EN2 = 0x2U, MSC_ENC_SEL_EN3 = 0x3U }
enum MSC_ENHSelectionType { MSC_ENH_SEL_EN0 = 0x0U, MSC_ENH_SEL_EN1 = 0x1U,
MSC_ENH_SEL_EN2 = 0x2U, MSC_ENH_SEL_EN3 = 0x3U }
enum MSC_ENLSelectionType { MSC_ENL_SEL_EN0 = 0x0U, MSC_ENL_SEL_EN1 = 0x1U,
MSC_ENL_SEL_EN2 = 0x2U, MSC_ENL_SEL_EN3 = 0x3U }
enum MSC_FCLControlType { MSC_FLC_ACTIVE_ON_FRAMES = 0x0U, MSC_FLC_ALWAYS_ACTIVE = 0x1U }
enum MSC_InputPolarityType { MSC_INPUT_IDENTICAL = 0x0U, MSC_INPUT_INVERTED = 0x1U }

```

```

enum MSC_ENxPolarityType { MSC_ENX_IDENTICAL = 0x0U, MSC_ENX_INVERTED = 0x1U }
enum MSC_SOPolarityType { MSC_SO_IDENTICAL = 0x0U, MSC_SO_INVERTED = 0x1U }
enum MSC_FCLpolarityType { MSC_FLC_IDENTICAL = 0x0U, MSC_FLC_INVERTED = 0x1U }
enum MSC_TransSourceType { MSC_TRANS_SOURCE_DATA_REG = 0x0U, MSC_TRANS_SOURCE_ALTIN =
0x2U, MSC_TRANS_SOURCE_ALTIN_INV = 0x3U }
enum MSC_WorkModeType { MSC_TRIGGER_MODE = 0x00U, MSC_REPETITION_MODE = 0x01U }
enum MSC_ParityType { MSC_EVEN_PARITY = 0x00U, MSC_ODD_PARITY = 0x01U }
enum MSC_ReceiveBaudRateType { MSC_RECEIVE_DISABLED = 0x00U, MSC_BAUDRATE_FMSC_DIV4 =
0x01U, MSC_BAUDRATE_FMSC_DIV8 = 0x02U, MSC_BAUDRATE_FMSC_DIV16 = 0x03U,
MSC_BAUDRATE_FMSC_DIV32 = 0x04U, MSC_BAUDRATE_FMSC_DIV64 = 0x05U,
MSC_BAUDRATE_FMSC_DIV128 = 0x06U, MSC_BAUDRATE_FMSC_DIV256 = 0x07U }
enum MSC_ReceiveFrameType { MSC_FRAME_12_BIT = 0x00U, MSC_FRAME_16_BIT = 0x01U }

```

Functions

MSC_ReturnType **MSC_InitStructure** (**MSC_InitCfgType** *pInitConfig)

Initialize the MSC structure with the provided configuration.

MSC_ReturnType **MSC_Init** (**MSC_HandleType** *pMscHandle, const **MSC_InitCfgType** *pInitConfig)

Init the MSC.

MSC_ReturnType **MSC_DeInit** (**MSC_HandleType** *pMscHandle)

De-initialize the MSC.

void **MSC_initInterrupt** (**MSC_HandleType** *pMscHandle, const **MSC_InterruptCfgType** *pInterruptConfig)

Init the MSC interrupt.

void **MSC_SelTransmittingSource** (const **MSC_HandleType** *pMscHandle, uint32_t u32SourceMask, **MSC_TransSourceType** eSourceType)

Select the transmitting sources.

void **MSC_SetEmergencyLoad** (const **MSC_HandleType** *pMscHandle, uint32_t u32Value)

Set Emergency load value.

void **MSC_SetIOControl** (const **MSC_HandleType** *pMscHandle, const **MSC_IOControlInitType** *pIOConfig)

Configure the MSC IO control.

void **MSC_Enable** (const **MSC_HandleType** *pMscHandle)

Enable MSC.

void **MSC_Disable** (const **MSC_HandleType** *pMscHandle)

Disable MSC.

void **MSC_SetDataFrame** (const **MSC_HandleType** *pMscHandle, uint32_t u32Data)

MSC set data frame.

void **MSC_SendDataFrame** (const **MSC_HandleType** *pMscHandle)

MSC send data frame.

void **MSC_SendCommandFrame** (const **MSC_HandleType** *pMscHandle, uint32_t u32Command)

MSC send command frame.

uint8_t **MSC_GetReceivedFrameAddr** (const **MSC_HandleType** *pMscHandle, **MSC_RDRxIndexType** eIndex)

Get the msc received data address.

MSC_ReceiveStatusType **MSC_GetReceivedFrame** (const **MSC_HandleType** *pMscHandle, **MSC_RDRxIndexType** eIndex, uint8_t *pData)

Get the msc received data.

uint32_t **MSC_GetInterruptStatus** (const **MSC_HandleType** *pMscHandle)

Get the msc interrupt status.

uint32_t **MSC_GetToInterruptStatus** (const **MSC_HandleType** *pMscHandle)

Get the msc interrupt status.

void **MSC_EnableTrasmit** (const **MSC_HandleType** *pMscHandle, bool bEnable)

Get the msc interrupt status.

void **MSC_ClearDataFrameInterruptFlag** (const **MSC_HandleType** *pMscHandle)

Clear the msc data frame interrupt flag.

void **MSC_ClearCommandFrameInterruptFlag** (const **MSC_HandleType** *pMscHandle)

Clear the msc command frame interrupt flag.

void **MSC_ClearTimeFrameInterruptFlag** (const **MSC_HandleType** *pMscHandle)

Clear the msc time frame interrupt flag.

void **MSC_ClearReceiveInterruptFlag** (const **MSC_HandleType** *pMscHandle)

Clear the msc receive interrupt flag.

void **MSC_ClearTimeoutInterruptFlag** (const **MSC_HandleType** *pMscHandle)

Clear the msc timeout interrupt flag.

void **MSCn_IRQHandler** (const **MSC_HandleType** *pMscHandle)

Detailed Description

MSC driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_ospi.h File Reference

OSPI driver type definition and API.

#include "HwA_ospi.h"

Include dependency graph for module_driver_ospi.h:

IMAGE

Classes

struct **OSPI_ConfigType**

The basic configuration option for the OSPI peripheral.

struct **OSPI_DeviceConfigType**

Define the structure for OSPI device configuration.

struct **_flexspi_transfer**

FlexSPI Transfer structure definition.

Macros

#define **OSPI_FIFO_DEPTH_MAX** 16

#define **OSPI_ONE_TRANSFER_SIZE_MAX** 64

#define **OSPI_LUT_SEQ**(cmd0, pad0, op0, cmd1, pad1, op1)

Ospi API Service IDs *These macros define the command IDs used for communication with the on-chip OSPI interface. Each ID corresponds to a specific operation that can be performed via the OSPI interface.*

#define **OSPI_INIT_ID** ((uint8) 0x00u)

#define **OSPI_SET_FLASHCFG_ID** ((uint8) 0x01u)

#define **OSPI_UPDATE_LUT_ID** ((uint8) 0x02u)

#define **OSPI_WRITE_FIFO_ID** ((uint8) 0x03u)

#define **OSPI_READ_FIFO_ID** ((uint8) 0x04u)

#define **OSPI_TRANSFER_BLOCKING_ID** ((uint8) 0x05u)

#define **OSPI_TRANSFER_NONBLOCKING_ID** ((uint8) 0x06u)

Ospi Dev Error Code *Error Code of calling OSPI apis*

#define **OSPI_E_PARAM_POINTER** ((uint8)0x01u)

#define **OSPI_E_PARAM_INVALID** ((uint8)0x02u)

IO configurations *IO configurations for the OSPI interface*


```
#define SINGLE_IO 0x00
#define DUAL_IO 0x01
#define QUAD_IO 0x02
#define OCTAL_IO 0x03
```

OSPI command_set OSPI command set definitions *OSPI command set definitions*

```
#define OSPI_CMD_STOP 0x0
#define OSPI_CFG_DRV 0x1
#define OSPI_CFG_ADDR 0x2
#define OSPI_CMD_DUMMY 0x3
#define OSPI_CFG_MODE8 0x4
#define OSPI_CFG_MODE2 0x5
#define OSPI_CFG_MODE4 0x6
#define OSPI_READ_DRV 0x7
#define OSPI_WRITE_DRV 0x8
#define OSPI_CMD_END 0x9
#define OSPI_CFG_ADDR_DDR 0xA
#define OSPI_CFG_MODE8_DDR 0xB
#define OSPI_CFG_MODE2_DDR 0xC
#define OSPI_CFG_MODE4_DDR 0xD
#define OSPI_READ_DRV_DDR 0xE
#define OSPI_WRITE_DRV_DDR 0xF
#define OSPI_CFG_DRV_DDR 0x11
#define OSPI_CFG_CADDR 0x12
#define OSPI_CFG_CADDR_DDR 0x13
```

Typedefs

```
typedef struct _flexspi_transfer OSPI_TransferType
FlexSPI Transfer structure definition.
```

Enumerations

```
enum OSPI_StatusType { OSPI_STATUS_SUCCESS = 0, OSPI_STATUS_PARAM_ERR,  
OSPI_STATUS_TIMEOUT }
```

Status returned by OSPI APIs.

```
enum OSPI_OperationType { OSPI_OPERATION_COMMAND, OSPI_OPERATION_CONFIG,  
OSPI_OPERATION_READ, OSPI_OPERATION_WRITE }
```

Functions

void **OSPI_Init** (OSPI_Type *base, const **OSPI_ConfigType** *const pConfig)
Initialize OSPI configuration.

void **OSPI_GetDefaultConfig** (**OSPI_ConfigType** *config)
Get OSPI configuration.

void **OSPI_DeInit** (OSPI_Type *base)
De-initialize the OSPI.

void **OSPI_FlashConfig** (OSPI_Type *base, **OSPI_DeviceConfigType** *pConfig)
OSPI Config Flash Parameter.

void **OSPI_UpdateLUT** (OSPI_Type *base, uint32_t index, const uint32_t *cmd, uint32_t count)
OSPI Config LUT.

OSPI_StatusType **OSPI_Wait_Cmd_Done** (OSPI_Type *base)
OSPI Wait bus command transaction done.

OSPI_StatusType **OSPI_Wait_Bus_Idle** (OSPI_Type *base)
OSPI Wait bus command transaction done.

OSPI_StatusType **OSPI_Write_Fifo** (OSPI_Type *base, uint8_t u8SeqId, uint32_t *pBuf, uint16_t u16Size)
OSPI write fifo. The fifo size is 16 words.

OSPI_StatusType **OSPI_Read_Fifo** (OSPI_Type *base, uint8_t u8SeqId, uint32_t *pBuf, uint16_t u16Size)
OSPI read fifo. The fifo size is 16 words.

OSPI_StatusType **OSPI_TransferBlocking** (OSPI_Type *base, **OSPI_TransferType** *xfer)
OSPI blocking transfer data.

OSPI_StatusType **OSPI_TransferNonBlocking** (OSPI_Type *base, **OSPI_TransferType** *xfer)
OSPI non-blocking transfer data.

void **OSPI_DriverIRQHandler** (OSPI_Type *base)
OSPI interrupt driver handler.

Detailed Description

OSPI driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_overlay.h File Reference

FC7xxx overlay driver type definition and API.

```
#include "HwA_overlay.h"
```

Include dependency graph for module_driver_overlay.h:

IMAGE

Classes

struct **OVERLAY_OverlayRegionInitType**

Overlay Region parameters.

struct **OVERLAY_FARInitType**

FAR parameter.

struct **OVERLAY_ErrorInfoType**

Error Info.

struct **OVERLAY_InterruptType**

Interrupt Parameter.

Typedefs

typedef void(* **OVERLAY_ErrorCallback_Type**) (**OVERLAY_ErrorInfoType** u16ErrorFlag)

Overlay Setting Error callback type.

Enumerations

enum **OVERLAY_ErrorType** { **OVERLAY_ERROR_OK**, **OVERLAY_ERROR_ADDR**, **OVERLAY_ERROR_SIZE**,
OVERLAY_ERROR_FLAG }

Overlay Error Information.

Functions

OVERLAY_ErrorType **OVERLAY_RegionInit** (**OVERLAY_OverlayRegionInitType** *pOverlayInitCfg)

Overlay region initial function.

OVERLAY_ErrorType **OVERLAY_RegionDeInit** (void)

De-Init Overlay Region.

OVERLAY_ErrorType OVERLAY_FARInit (OVERLAY_FARInitType *pFarInitCfg)

Far initial function.

OVERLAY_ErrorType OVERLAY_FARDeInit (void)

De-Init FAR Region.

void OVERLAY_SetInterrupt (OVERLAY_InterruptType *pInterruptCfg)

Enable or Disable Interrupt.

void OVERLAY_GetErrorInfo (OVERLAY_ErrorInfoType *pErrorInfo)

Get Error Info.

void OVERLAY_ClrErrorInfo (OVERLAY_ErrorInfoType *pErrorInfo)

Clear Error Info.

void OVERLAY_ErrorInterruptRoutine (void)

Call Me in Overlay Error interrupt handler.

Detailed Description

FC7xxx overlay driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2023 Flagchip Semiconductors Co., Ltd.

module_driver_pcc.h File Reference

PCC driver type definition and API.

```
#include "device_header.h"
```

Include dependency graph for module_driver_pcc.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Classes

struct **PCC_CtrlType**

define the PCC module initialization structure.

Macros

```
#define PCC_DWPLK_MASK 0x80000000u
```

Marco for PCCn Bit Field definition.

```
#define PCC_DWPLK_SHIFT 31u
```

```
#define PCC_DWPLK_WIDTH 1u
```

```
#define PCC_DWP_MASK 0x70000000u
```

```
#define PCC_DWP_SHIFT 28u
```

```
#define PCC_DWP_WIDTH 3u
```

```
#define PCC_DWP(x) (((uint32_t)(((uint32_t)(x))<<PCC_DWP_SHIFT))&PCC_DWP_MASK)
```

```
#define PCC_CGC_MASK 0x8000000U
```

```
#define PCC_CGC_SHIFT 23U
```

```
#define PCC_CGC_WIDTH 1U
```

```
#define PCC_CGC(x) (((uint32_t)(((uint32_t)(x))<<PCC_CGC_SHIFT))&PCC_CGC_MASK)
```

```
#define PCC_SEL_MASK 0x7000000U
```

```
#define PCC_SEL_SHIFT 20U
```

```
#define PCC_SEL_WIDTH 3U
```

```
#define PCC_SEL(x) (((uint32_t)(((uint32_t)(x))<<PCC_SEL_SHIFT))&PCC_SEL_MASK)
```

```
#define PCC_GetSEL(x) (((uint32_t)(x))&PCC_SEL_MASK)>>PCC_SEL_SHIFT)
```

```
#define PCC_DIV_MASK 0x7U
```

```

#define PCC_DIV_SHIFT 0U
#define PCC_DIV_WIDTH 3U
#define PCC_DIV(x) (((uint32_t)(((uint32_t)(x))<<PCC_DIV_SHIFT))&PCC_DIV_MASK)
#define PCC_GetDIV(x) (((uint32_t)(x))&PCC_DIV_MASK)>>PCC_DIV_SHIFT)
#define PCC_SWR_MASK 0x10000u
#define PCC_MUX_MAX_NUMBER 8U
    PCC API Service IDs#define PCC_SET_PCC_ID 0x00U
    #define PCC_GET_PCC_INTERFACE_CLOCK_ID 0x01U
    #define PCC_GET_PCC_FUNCTION_CLOCK_ID 0x02U
PCC Dev Error CodeError Code of calling PCC apis
    #define PCC_E_PARAM_POINTER 0x01U
    #define PCC_E_PARAM_OUT_RANGE 0x02U

```

Enumerations

```

enum PCC_StatusType { PCC_STATUS_SUCCESS = 0U, PCC_STATUS_CLOCK_INVALID = 1U,
PCC_STATUS_CONFIGURED_NOT_SUPPORT = 1U }

```

Pcc clock status.

```

enum PCC_ClkGateSrcType { PCC_CLKGATE_SRC_OFF = 0U, PCC_CLKGATE_SRC_FOSCDIV = 1U,
PCC_CLKGATE_SRC_SIRCDIV = 2U, PCC_CLKGATE_SRC_FIRCDIV = 3U, PCC_CLKGATE_SRC_RESERVE0 =
4U, PCC_CLKGATE_SRC_PLL1DIV = 5U, PCC_CLKGATE_SRC_PLL0DIV = 6U,
PCC_CLKGATE_SRC_RESERVE1 = 7U }

```

defined the clock source for function clock,match with PCC_XXX[SEL] bit filed.

```

enum PCC_ClkDivType { PCC_CLK_DIV_BY1 = 0U, PCC_CLK_DIV_BY2 = 1U, PCC_CLK_DIV_BY3 = 2U,
PCC_CLK_DIV_BY4 = 3U, PCC_CLK_DIV_BY5 = 4U, PCC_CLK_DIV_BY6 = 5U, PCC_CLK_DIV_BY7 = 6U,
PCC_CLK_DIV_BY8 = 7U, PCC_CLK_UNINVOLVED = 8U }

```

define the clock divider,match with PCC_XXX[DIV] bit filed.

```

enum PCC_CtrlOwnerType { PCC_CTRL_BY_ALL = 0U, PCC_CTRL_BY_CPU0 = 1U, PCC_CTRL_BY_CPU1 =
2U, PCC_CTRL_BY_CPU2 = 3U, PCC_CTRL_BY_NONE = 7U }

```

Indicate which CPU can control PCC.

Functions

```

uint32_t PCC_GetPccFunctionClock (const PCC_ClkSrcType eClockName)

```

get PCC function clock status and value.

```

uint32_t PCC_GetPccInterfaceClock (const PCC_ClkSrcType eClockName)

```

get PCC interface clock status and value.

PCC_StatusType PCC_SetPcc (const **PCC_CtrlType** *const pConfig)
set PCC one peripheral clock configuration.

void **PCC_GenPeripheralReset** (const PCC_ClkSrcType eClockName)
Generate peripheral reset.

Detailed Description

PCC driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

module_driver_pmc.h File Reference

FC7xxx PMC driver type definition and API.

#include "HwA_pmc.h"

Include dependency graph for module_driver_pmc.h:

IMAGE

Classes

struct **_PMC_HandleType**

The structure of the SCG processing handle.

struct **PMC_ConfigType**

Pmc control type.

Macros

PMC API Service IDs#define **PMC_INIT_ID** 0x00U

PMC Dev Error Code*Error Code of calling PCC apis*

#define **PMC_E_PARAM_INSTANCE** 0x01U

#define **PMC_E_PARAM_POINTER** 0x02U

PMC interrupt mask micros

#define **LVD_INTERRUPT** PMC_CONFIG_LVD_IE_MASK

#define **HVD_INTERRUPT** PMC_CONFIG_HVD_IE_MASK

enum **PMC_Fault_Type** { **PMC_NONE** = 0, **PMC_HVD5V**, **PMC_HVD2P5V**, **PMC_HVD1P1V**, **PMC_LVD5V** }

PMC fault type. This provides constants for the PMC fault type for use in the PMC interrupt callback functions.

enum **PMC_FlagType** { **PMC_HVD5V_FLAG** = PMC_LVSCR_HVD5V_FLAG_MASK, **PMC_HVD2P5V_FLAG** = PMC_LVSCR_HVD2P5V_FLAG_MASK, **PMC_HVD1P1V_FLAG** = PMC_LVSCR_HVD1P1V_FLAG_MASK, **PMC_LVD5V_FLAG** = PMC_LVSCR_LVD5V_FLAG_MASK, **PMC_LVR5V_FPM_FLAG** = PMC_LVSCR_LVR5V_FPM_FLAG_MASK, **PMC_LVR5V_RPM_FLAG** = PMC_LVSCR_LVR5V_RPM_FLAG_MASK, **PMC_LVR2P5V_FPM_FLAG** = PMC_LVSCR_LVR2P5V_FPM_FLAG_MASK, **PMC_LVR2P5V_RPM_FLAG** = PMC_LVSCR_LVR2P5V_RPM_FLAG_MASK, **PMC_LVR1P1V_FPM_FLAG** = PMC_LVSCR_LVR1P1V_FPM_FLAG_MASK, **PMC_LVR1P1V_RPM_FLAG** = PMC_LVSCR_LVR1P1V_RPM_FLAG_MASK, **PMC_POR_FLAG** = PMC_LVSCR_POR_FLAG_MASK }

PMC voltage flags.

```
enum PMC_StatusType { PMC_HVD5V_STATUS = PMC_LVSCR_HVD5V_STATUS_MASK,  
PMC_HVD2P5V_STATUS = PMC_LVSCR_HVD2P5V_STATUS_MASK, PMC_HVD1P1V_STATUS =  
PMC_LVSCR_HVD1P1V_STATUS_MASK, PMC_LVD5V_STATUS = PMC_LVSCR_LVD5V_STATUS_MASK }
```

PMC voltage status.

```
enum PMC_IntrMaskType { PMC_INTR_MASK_HVD = 1U, PMC_INTR_MASK_LVD = 2U }
```

The interrupt enable/disable mask of PMC.

```
typedef struct _PMC_HandleType PMC_HandleType
```

The structure of the SCG processing handle.

```
bool PMC_GetFlag (PMC_FlagType eFlag)
```

PMC get status flag.

```
void PMC_ClearFlag (PMC_FlagType eFlag)
```

PMC clears an individual status flag.

```
bool PMC_GetStatus (PMC_StatusType eStatus)
```

PMC get status.

```
void Pmc_EnableInterrupt (uint32 u32Val)
```

Enable PMC interrupt.

```
void Pmc_DisableInterrupt (uint32 u32Val)
```

Disable PMC interrupt.

```
void Pmc_Init (PMC_HandleType *pPmcHandle, PMC_ConfigType *pIntCfg)
```

PMC Init.

```
void PMC_Deinit (void)
```

Clear all PMC register.

```
void PMC_UserIRQHandler (PMC_HandleType *pPmcHandle)
```

PMC_UserIRQHandler function Customer needs to write this function according by customer's requirement.

This function is only resolve compile conflict.

Detailed Description

FC7xxx PMC driver type definition and API.

Author

Flagchip032

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2022 Flagchip Semiconductors Co., Ltd.

module_driver_port.h File Reference

PORT driver type definition and API.

```
#include "HwA_port.h"
```

```
#include "HwA_gpio.h"
```

```
#include <stddef.h>
```

Include dependency graph for module_driver_port.h:

IMAGE

Classes

struct **PORT_DigitalFilterType**

Port digital filter configuration structure.

struct **_PORT_HandleType**

The structure of the PORT processing handle.

struct **PORT_InterruptType**

Port interrupt configuration structure.

struct **PORT_InitType**

Port initialization structure.

Macros

```
#define PORT_PIN_0 ((uint32_t)0x00000001)
```

```
#define PORT_PIN_1 ((uint32_t)0x00000002)
```

```
#define PORT_PIN_2 ((uint32_t)0x00000004)
```

```
#define PORT_PIN_3 ((uint32_t)0x00000008)
```

```
#define PORT_PIN_4 ((uint32_t)0x00000010)
```

```
#define PORT_PIN_5 ((uint32_t)0x00000020)
```

```
#define PORT_PIN_6 ((uint32_t)0x00000040)
```

```
#define PORT_PIN_7 ((uint32_t)0x00000080)
```

```
#define PORT_PIN_8 ((uint32_t)0x00000100)
```

```
#define PORT_PIN_9 ((uint32_t)0x00000200)
```

```
#define PORT_PIN_10 ((uint32_t)0x00000400)
```

```
#define PORT_PIN_11 ((uint32_t)0x00000800)
```

```
#define PORT_PIN_12 ((uint32_t)0x00001000)
```

```
#define PORT_PIN_13 ((uint32_t)0x00002000)
```

```

#define PORT_PIN_14 ((uint32_t)0x00004000)
#define PORT_PIN_15 ((uint32_t)0x00008000)
#define PORT_PIN_16 ((uint32_t)0x00010000)
#define PORT_PIN_17 ((uint32_t)0x00020000)
#define PORT_PIN_18 ((uint32_t)0x00040000)
#define PORT_PIN_19 ((uint32_t)0x00080000)
#define PORT_PIN_20 ((uint32_t)0x00100000)
#define PORT_PIN_21 ((uint32_t)0x00200000)
#define PORT_PIN_22 ((uint32_t)0x00400000)
#define PORT_PIN_23 ((uint32_t)0x00800000)
#define PORT_PIN_24 ((uint32_t)0x01000000)
#define PORT_PIN_25 ((uint32_t)0x02000000)
#define PORT_PIN_26 ((uint32_t)0x04000000)
#define PORT_PIN_27 ((uint32_t)0x08000000)
#define PORT_PIN_28 ((uint32_t)0x10000000)
#define PORT_PIN_29 ((uint32_t)0x20000000)
#define PORT_PIN_30 ((uint32_t)0x40000000)
#define PORT_PIN_31 ((uint32_t)0x80000000)
#define PORT_ALT0_FUNC_MODE ((Port_PinModeType)0)

```

Port Alternate 0 Mode.

```

#define PORT_GPIO_MODE ((Port_PinModeType)1)

```

Port GPIO Mode.

```

#define PORT_ALT2_FUNC_MODE ((Port_PinModeType)2)

```

Port Alternate 2 Mode.

```

#define PORT_ALT3_FUNC_MODE ((Port_PinModeType)3)

```

Port Alternate 3 Mode.

```

#define PORT_ALT4_FUNC_MODE ((Port_PinModeType)4)

```

Port Alternate 4 Mode.

```

#define PORT_ALT5_FUNC_MODE ((Port_PinModeType)5)

```

Port Alternate 5 Mode.

```
#define PORT_ALT6_FUNC_MODE ((Port_PinModeType)6)
```

Port Alternate 6 Mode.

```
#define PORT_ALT7_FUNC_MODE ((Port_PinModeType)7)
```

Port Alternate 7 Mode.

PORT API Service IDs#

```
define PORT_InitPins_ID 0U
```

```
define PORT_Deinit_ID 1U
```

```
define PORT_SetPinMode_ID 2U
```

```
define PORT_SetInterruptCfg_ID 3U
```

```
define PORT_InitDigitalFilterPort_ID 4U
```

```
define PORT_DeinitDigitalFilterPort_ID 5U
```

```
define PORT_EnableDigitalFilterPin_ID 6U
```

```
define PORT_DisableDigitalFilterPin_ID 7U
```

```
define PORT_IRQHandler_ID 8U
```

```
define PORT_SetPinEmergencyStop_ID 9U
```

PORT Dev Error Code*Error Code of calling GPIO apis*

```
define PORT_E_PARAM_INSTANCE 0x01U
```

```
define PORT_E_PARAM_INPUT 0x02U
```

```
define PORT_E_PARAM_PIN 0x03U
```

Typedefs

```
typedef struct _PORT_HandleType PORT_HandleType
```

The structure of the PORT processing handle.

Enumerations

```
enum PORT_IrqcConfigurationType { PORT_IRQ_DISABLE = 0U, PORT_IRQ_DMA_RISING = 1U,  
PORT_IRQ_DMA_FALLING = 2U, PORT_IRQ_DMA_BOTH_EDGE = 3U, PORT_IRQ_LOGIC_0 = 8U,  
PORT_IRQ_RISING = 9U, PORT_IRQ_FALLING = 10U, PORT_IRQ_BOTH_EDGE = 11U, PORT_IRQ_LOGIC_1  
= 12U, PORT_IRQ_TOOGLED_WHEN_LOGIC_0 = 13U, PORT_IRQ_TOOGLED_WHEN_LOGIC_1 = 14U }
```

Port PCR[IRQC] configuration.

```
enum PORT_PullStatusType { PORT_PULL_DOWN = 0U, PORT_PULL_UP }
```

Port pull status.

```
enum PORT_GpioDirType { PORT_GPIO_IN = 0U, PORT_GPIO_OUT, PORT_GPIO_ZERO }
```

Gpio direction.

enum **PORT_GpioLevelType** { **PORT_GPIO_LOW** = 0U, **PORT_GPIO_HIGH** }

Gpio initialization level.

enum **PORT_DigitalFilterClkSrcType** { **PORT_FILTER_BUS_CLK** = 0U, **PORT_FILTER_AON32K_CLK** }

Port digital filter clock source.

enum **PORT_InstanceType** { **PORT_A** = 0U, **PORT_B**, **PORT_C**, **PORT_D**, **PORT_E**, **PORT_F**, **PORT_G**, **PORT_H**, **PORT_I** }

Port instance number.

Functions

void **PORT_InitPins** (**PORT_HandleType** *pPortHandle, const **PORT_InitType** *const pInitStruct)

Initialize port pins.

void **PORT_Deinit** (**PORT_HandleType** *pPortHandle, const uint32_t u32Pins)

De-initialize the Port instance.

void **PORT_EnableEmergencyStop** (**PORT_HandleType** *pPortHandle, const uint32_t u32Pins, bool bEnable)

set Emergency Stop of pin

void **PORT_SetInterruptCfg** (**PORT_HandleType** *pPortHandle, const uint32_t u32Pins, **PORT_IrqcConfigurationType** ePortPinIrq)

set interrupt config of port

void **PORT_InitDigitalFilterPort** (**PORT_HandleType** *pPortHandle, const **PORT_DigitalFilterType** *const pDFStruct)

Initialize digital filter for Port instance.

void **PORT_DeinitDigitalFilterPort** (**PORT_HandleType** *pPortHandle)

De-initialize digital filter for Port instance.

void **PORT_EnableDigitalFilterPin** (**PORT_HandleType** *pPortHandle, const uint32_t u32Pins)

Enable the digital filter function for the specific pin.

void **PORT_DisableDigitalFilterPin** (**PORT_HandleType** *pPortHandle, const uint32_t u32Pins)

Disable the digital filter function for the specific pin.

void **PORT_IRQHandler** (**PORT_HandleType** *pPortHandle)

PORT interrupt process function.

Detailed Description

PORT driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_ptimer.h File Reference

PTIMER driver type definition and API.

```
#include "HwA_ptimer.h"
```

Include dependency graph for module_driver_ptimer.h:

IMAGE

Classes

struct **_PTIMER_HandleType**

Ptimer process handler.

struct **PTIMER_InitType**

Ptimer instance initialization parameters.

struct **PTIMER_ChannelCfgType**

Ptimer channel configuration parameters.

struct **PTIMER_PulseOutType**

Ptimer pulse out configurations.

Macros

```
FTU API Service IDs#define PTIMER_INIT_ID 0U
#define PTIMER_INIT_CHANNEL_ID 1U
#define PTIMER_INIT_STRUCTURE_ID 2U
#define PTIMER_DEINIT_ID 3U
#define PTIMER_SET_PERIOD_ID 4U
#define PTIMER_SET_PULSE_OUT_ID 5U
#define PTIMER_LOAD_VALUE_ID 6U
#define PTIMER_ENABLE_ID 7U
#define PTIMER_DISABLE_ID 8U
#define PTIMER_PULSE_OUT_ID 9U
#define PTIMER_DISABLE_PULSE_OUT_ID 10U
#define PTIMER_GENERATE_SW_TRIGGER_ID 11U
```

FTU Dev Error Code*Error Code of calling FTU apis*

```
#define PTIMER_E_PARAM_INSTANCE 0x01U
#define PTIMER_E_PARAM_CHANNEL 0x02U
#define PTIMER_E_PARAM_POINTER 0x03U
```

```
#define PTIMER_E_PARAM_DELAY_VALUE 0x04U
```

Typedefs

```
typedef struct _PTIMER_HandleType PTIMER_HandleType
```

Ptimer process handler.

Enumerations

```
enum PTIMER_ChannelSelectionType { PTIMER0_USED_FOR_ADC0 = 0U, PTIMER0_USED_FOR_ADC4,  
PTIMER3_USED_FOR_ADC3, PTIMER3_USED_FOR_ADC5 }
```

PTIMER Chanenl1 Selection Type.

```
enum PTIMER_InstanceType { PTIMER_INSTANCE_0 = 0U, PTIMER_INSTANCE_1 = 1U,  
PTIMER_INSTANCE_2 = 2U, PTIMER_INSTANCE_3 = 3U }
```

The instance index of the PTIMER peripheral.

Functions

```
void PTIMER_InitStructure (PTIMER_InitType *const pInitCfg)
```

Provide the default values of PTIMER_InitType.

```
void PTIMER_Init (PTIMER_HandleType *PtimerHandle, const PTIMER_InitType *const pInitCfg)
```

Initialize the Ptimer instance.

```
void PTIMER_DeInit (PTIMER_HandleType *PtimerHandle)
```

De-initialize the Ptimer instance Disable the Ptimer instance and reset the configurations to its reset values.

```
void PTIMER_InitChannel (PTIMER_HandleType *PtimerHandle, const PTIMER_ChannelCfgType  
aChannelCfg[], const uint8_t u8ChnNum)
```

Initialize the Ptimer channels.

```
void PTIMER_SetPeriod (const PTIMER_HandleType *PtimerHandle, uint16 u16PtimerPeriod)
```

Set the Ptimer max counter period in microsecond When the Ptimer counter reaches the period, it will return to zero.

```
void PTIMER_SetPulseOut (const PTIMER_HandleType *PtimerHandle, const PTIMER_PulseOutType  
*pPulseOutCfg)
```

Set the Ptimer pulse-out function.

void **PTIMER_LoadValue** (const **PTIMER_HandleType** *PtimerHandle)

Load the buffered values into register.

void **PTIMER_Enable** (const **PTIMER_HandleType** *PtimerHandle)

Enable the Ptimer instance.

void **PTIMER_Disable** (const **PTIMER_HandleType** *PtimerHandle)

Disable the Ptimer instance.

void **PTIMER_EnablePulseOut** (const **PTIMER_HandleType** *PtimerHandle)

Enable the Ptimer pulse-out function.

void **PTIMER_DisablePulseOut** (const **PTIMER_HandleType** *PtimerHandle)

Disable the Ptimer pulse-out function.

void **PTIMER_SetInstance01BackToBackEnable** (bool bEnable)

void **PTIMER_SetInstance23BackToBackEnable** (bool bEnable)

void **PTIMERn_IRQHandler** (**PTIMER_HandleType** *PtimerHandle)

void **PTIMER_GenerateSWTrigger** (const **PTIMER_HandleType** *PtimerHandle)

Generate the software trigger signal for the Ptimer instance.

void **PTIMER_ChannelSelection** (**PTIMER_ChannelSelectionType** selection)

Set PTIMER channel used for specific ADC instance.

Detailed Description

PTIMER driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_qdt.h File Reference

FC7xxx QDT driver type definition and API.

#include "HwA_qdt.h"

Include dependency graph for module_driver_qdt.h:

IMAGE

Detailed Description

FC7xxx QDT driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-28

Copyright

Copyright (c) 2024 Flagchip Semiconductors Co., Ltd.

module_driver_rgm.h File Reference

RGM driver type definition and API.

```
#include "HwA_rgm.h"
```

Include dependency graph for module_driver_rgm.h:

IMAGE

Classes

struct **_RGM_HandleType**

The structure of the SCG processing handle.

struct **RGM_ConfigType**

Rgm configure type.

Macros

RGM API Service IDs#define **RGM_INIT_ID** 0x00U

RGM Dev Error Code*Error Code of calling RGM apis*

#define **RGM_E_PARAM_DELAY_TIME** 0x01U

#define **RGM_E_PARAM_POINTER** 0x02U

Typedefs

typedef struct **_RGM_HandleType** **RGM_HandleType**

The structure of the SCG processing handle.

Enumerations

enum **RGM_StatusType** { **RGM_STATUS_SUCCESS** = 0U, **RGM_STATUS_PARAM_INVALID** = 1U, **RGM_STATUS_COREID_INVALID** = 2U, **RGM_STATUS_CLOCK_INVALID** }

Rgm return type.

enum **RGM_FilterClkSrc** { **RGM_RESET_FILTER_DISABLE** = 0U, **RGM_RESET_FILTER_BUS_CLOCK** = 1U }

Rgm reset pin filter clock type.

Functions

uint32_t **RGM_GetSystemLastResetFlag** (void)

This api can get RGM_SRS register that indicate the source of the most recent reset.

uint32_t **RGM_GetSystemAllResetFlag** (void)

This api can get RGM_SSRS register that indicate all reset sources since the last POR or LVD that have not been cleared by software.

void **RGM_ClearSystemAllResetFlagAfterPOR** (void)

This api can clear all reset flag of RGM_SSRS register which indicate all reset sources since the last POR or LVD that have not been cleared by software.

RGM_StatusType **RGM_EnableResetFilter** (**RGM_FilterClkSrc** eClk, uint8_t u8BusClockFilterWidth, bool bLpClkEn)

Enable reset pin filter.

RGM_StatusType **RGM_DisableResetFilter** (**RGM_FilterClkSrc** eClk, bool bLpClkEn)

Disable reset pin filter.

RGM_StatusType **RGM_EnableSystemResetInt** (**RGM_ResetDelayType** eDelay, **RGM_SysResetIntMangerType** eResetInterrupt)

This api can enable interrupt before an system reset appear.

void **RGM_DisableSystemResetInt** (**RGM_SysResetIntMangerType** eResetInterrupt)

This api can disable interrupt before an system reset appear.

void **RGM_GenerateSystemCoreReset** (void)

Generate software reset through cortex-m register.

void **RGM_GenerateCpu0SwReset** (void)

Generate software reset through RGM C0_RST register.

RGM_StatusType **RGM_DisableCPU0CoreResetInt** (**RGM_CoreResetIntMangerType** eCPU0Interrupt)

This api can disable interrupt before an CPU0 core related reset appear.

RGM_CoreResetStatusType **RGM_GetCPU0OutResetFlag** (void)

Get the CPU0 exit reset flag.

uint32_t **RGM_GetCPU0LastResetFlag** (void)

This api can get RGM_C0_SRS register that indicate the source of the most recent CPU0 reset.

uint32_t **RGM_GetCPU0AllResetFlag** (void)

This api can get RGM_C0_SSRS register that indicate all CPU0 reset sources since the last POR or LVD that have not been cleared by software.

void **RGM_ClearCPU0AllResetFlagAfterPOR** (void)

This api can clear all reset flag of RGM_C0_SSRS register which indicate CPU0 all reset sources since the last POR or LVD that have not been cleared by software.

void **RGM_Pre_UserIRQHandler** (RGM_HandleType *pRgmHandle)

RGM Pre-interrupt entry.

void **RGM_Init** (RGM_HandleType *pRgmHandle, RGM_ConfigType *pInitCfg)

RGM initialize function.

Detailed Description

RGM driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_rtc.h File Reference

rtc driver type definition and API

```
#include "HwA_rtc.h"
```

Include dependency graph for module_driver_rtc.h:

IMAGE

Classes

struct **_RTC_HandleType**

The structure of the RTC processing handle.

struct **RTC_InitType**

Rtc initialization type.

struct **RTC_TimeDate**

struct **RTC_Compensation**

Macros

```
#define SECONDS_DAY 86400UL
```

```
#define SECONDS_HOUR 3600UL
```

```
#define SECONDS_MINUTE 60UL
```

```
#define MINUTES_HOUR 60UL
```

```
#define HOURS_DAY 24UL
```

```
#define DAYS_LEAP_YEAR 366UL
```

```
#define DAYS_COM_YEAR 365UL
```

```
#define START_YEAR 1970UL
```

```
#define END_YEAR 2099UL
```

RTC API Service IDs#

```
#define RTC_INIT_ID 0U
```

```
#define RTC_DEINIT_ID 1U
```

```
#define RTC_START_ID 2U
```

```
#define RTC_STOP_ID 3U
```

```
#define RTC_SET_COUNT_ID 4U
```

```
#define RTC_UPDATE_ALARM_ID 5U
```

```
#define RTC_CHECK_OVERFLOW_FLAF_ID 6U
```

```
#define RTC_GET_TIMEDATE_ID 7U
```

```
#define RTC_SET_TIMEDATE_ID 8U
```

```
#define RTC_GET_COMPENSATION_ID 9U
```

```
#define RTC_SET_COMPENSATION_ID 10U
#define RTC_INTE_PROC_ID 11U
#define RTC_IRQ_HANDLER_ID 12U
```

FTU Dev Error Code *Error Code of calling RTC apis*

```
#define RTC_E_PARAM_INSTANCE 0x01U
#define RTC_E_PARAM_CLOCK 0x02U
#define RTC_E_PARAM_POINTER 0x03U
#define RTC_E_PARAM_FORMAT 0x04U
```

Typedefs

```
typedef struct _RTC_HandleType RTC_HandleType
```

The structure of the RTC processing handle.

Enumerations

```
enum RTC_InstanceType { RTC_INSTANCE_0 = 0U }
```

The instance index of the RTC peripheral.

```
enum RTC_IntEventType { RTC_ALARM_INT = 0U, RTC_SECOND_INT = 1U, RTC_OVERFLOW_INT = 2U }
```

Rtc mode.

Functions

```
void RTC_Init (RTC_HandleType *pRtcHandle, const RTC_InitType *const pInitStruct)
```

Initialize Rtc instance.

```
void RTC_Deinit (RTC_HandleType *pRtcHandle)
```

De-initialize Rtc instance.

```
void RTC_Start (RTC_HandleType *pRtcHandle)
```

Rtc start.

```
void RTC_Stop (RTC_HandleType *pRtcHandle)
```

Rtc stop.

```
void RTC_SetCountValue (RTC_HandleType *pRtcHandle, uint32_t u32CounetValue)
```

set count value

void **RTC_UpdateAlarmValue** (**RTC_HandleType** *pRtcHandle, const uint32_t u32AlarmValue)
Rtc alarm value.

uint32_t **RTC_CheckOverflowFlag** (**RTC_HandleType** *pRtcHandle)
Check RTC overflow flag.

void **RTC_GetTimeDate** (**RTC_HandleType** *pRtcHandle, **RTC_TimeDate** *const pTimeDate)
Get time and date.

void **RTC_SetTimeDate** (**RTC_HandleType** *pRtcHandle, const **RTC_TimeDate** *const pTimeDate)
Set time and date.

void **RTC_GetCompensation** (**RTC_HandleType** *pRtcHandle, **RTC_Compensation** *const pCompensation)
Get time and date compensation information.

void **RTC_SetCompensation** (**RTC_HandleType** *pRtcHandle, const **RTC_Compensation** *const pCompensation)
Set time and date compensation value.

void **Rtc_CommonProcessInterrupt** (**RTC_HandleType** *pRtcHandle)
RTC common interrupt function.

Detailed Description

rtc driver type definition and API

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_scg.h File Reference

SCG driver type definition and API.

```
#include "HWA_scg.h"
```

Include dependency graph for module_driver_scg.h:

IMAGE

This graph shows which files directly or indirectly include this file:

IMAGE

Classes

struct **SCG_FoscType**

FOSC initial definition, include register SCG_FOSCCSR/SCG_FOSCDIV/SCG_FOSCCFG.

struct **SCG_SircType**

SIRC initial definition, include register SCG_SIRCCSR/SCG_SIRCDIV/SCG_SIRCTCCFG.

struct **SCG_Sirc32kType**

struct **SCG_FircType**

FIRC initial definition, include register SCG_FIRCCSR/SCG_FIRCDIV/SCG_FIRCFG.

struct **SCG_SoscType**

SOSC definition, include register SCG_SOSCCFG.

struct **SCG_PllType**

PLL definition, include register SCG_PLLCSR/SCG_PLLDIV/SCG_PLL0CFG.

struct **SCG_ClockCtrlType**

Current system clock definition, include register SCG_CCR.

struct **_SCG_HandleType**

The structure of the SCG processing handle.

struct **SCG_InterruptType**

The configuration option for the SCG interrupt.

Macros

```
#define SCG_DEV_ERROR_REPORT STD_OFF
```

```
#define SCG_ReportDevError(func, error) ReportDevError(SCG_MODULE_ID, func, error)
```

```
#define FOSC_FREQUENCY 24000000U
```

Fast OSC frequency.

```
#define SOSC_FREQUENCY 32768U
```

Slow OSC frequency.

```
#define PCC_FTU_TCLK_FREQ 0U
```

FTU input TCLK frequency.

```
#define UNKNOWN_CLOCK 0xFFFFFFFFU
```

```
#define SCG_CORE_CLOCK_SYMBOL CORE
```

```
#define SCG_BUS_CLOCK_SYMBOL BUS
```

```
#define SCG_SLOW_CLOCK_SYMBOL SLOW
```

```
#define SCG_SIRC_CLOCK_SYMBOL SIRC
```

```
#define SCG_SIRC32K_CLOCK_SYMBOL SIRC32K
```

```
#define SCG_FIRC_CLOCK_SYMBOL FIRC
```

```
#define SCG_FOSC_CLOCK_SYMBOL FOSC
```

```
#define SCG_SOSC_CLOCK_SYMBOL SOSC
```

```
#define SCG_PLL0_CLOCK_SYMBOL PLL0
```

```
#define SCG_PLL1_CLOCK_SYMBOL PLL1
```

SCG API Service IDs#

```
#define SCG_SET_SIRC_ID 0x00U
```

```
#define SCG_ENABLE_SIRC32K_ID 0x01U
```

```
#define SCG_ENABLE_FIRC_ID 0x02U
```

```
#define SCG_ENABLE_FOSC_ID 0x03U
```

```
#define SCG_ENABLE_SOSC_ID 0x04U
```

```
#define SCG_ENABLE_PLL_ID 0x05U
```

```
#define SCG_SET_CLOCK_CTRL_ID 0x06U
```

```
#define SCG_SET_CLOCK_OUT_ID 0x07U
```

```
#define SCG_SWITCH_SYS_CLOCK_ID 0x08U
```

```
#define SCG_GET_SCG_CLOCK_FREQ_ID 0x09U
```

```
#define SCG_SCG_DEINIT_ID 0x0AU
```

```
#define SCG_SCG_INTERRUPT_INIT_ID 0x0BU
```

SCG Dev Error Code*Error Code of calling SCG apis*

```
#define SCG_E_PARAM_POINTER 0x01U
```

```
#define SCG_E_PARAM_OUT_RANGE 0x02U
```

Typedefs

```
typedef uint16_t SCG_PllMultiplyType
```

SCG_PLLCFG [MULT] for PLL clock calculation. The mult value range is 96 ~ 512.

typedef struct **_SCG_HandleType** **SCG_HandleType**

The structure of the SCG processing handle.

Enumerations

enum **SCG_ClkSrcType** { **SCG_CORE_CLK** = 0U, **SCG_BUS_CLK** = 1U, **SCG_SLOW_CLK** = 2U, **SCG_FOSC_CLK** = 3U, **SCG_FIRC_CLK** = 4U, **SCG_PLL0_CLK** = 5U, **SCG_PLL1_CLK** = 6U, **SCG_SIRC_CLK** = 7U, **SCG_SIRC32K_CLK** = 8U, **SCG_SOSC_CLK** = 9U, **SCG_SCG_CLKOUT_CLK** = 10U, **SCG_SIRCDIVH_CLK** = 11U, **SCG_SIRCDIVM_CLK** = 12U, **SCG_SIRCDIVL_CLK** = 13U, **SCG_FIRCDIVH_CLK** = 14U, **SCG_FIRCDIVM_CLK** = 15U, **SCG_FIRCDIVL_CLK** = 16U, **SCG_FOSCDIVH_CLK** = 17U, **SCG_FOSCDIVM_CLK** = 18U, **SCG_FOSCDIVL_CLK** = 19U, **SCG_PLL0DIVH_CLK** = 20U, **SCG_PLL0DIVM_CLK** = 21U, **SCG_PLL0DIVL_CLK** = 22U, **SCG_PLL1DIVH_CLK** = 23U, **SCG_PLL1DIVM_CLK** = 24U, **SCG_PLL1DIVL_CLK** = 25U, **SCG_NVMINIT_CLK** = 26U, **SCG_CMU4REF_CLK** = 27U, **SCG_END_OF_CLOCKS** = 34U }

clock list for internal clock tree

enum **SCG_PllClkType** { **SCG_PLL0** = 0U, **SCG_PLL1** = 1U }

PLL clock type.

enum **SCG_StatusType** { **SCG_STATUS_SUCCESS** = 0U, **SCG_STATUS_SEQUENCE_ERROR** = 1U, **SCG_STATUS_TIMEOUT** = 2U, **SCG_STATUS_IRC_ERROR** = 3U, **SCG_STATUS_PARAM_ERROR** = 4U }

indicate the clock function status

enum **SCG_CrcModeType** { **SCG_CRC_SOFTWARE_MODE** = 0U, **SCG_CRC_TRIGGER_MODE** = 1U }

SCG registers CRC trigger mode.

enum **SCG_CrcCheckResType** { **SCG_CRC_CHECK_SUCCESS** = 0U, **SCG_CRC_CHECK_FAILED** = 1U }

indicate the CRC check result

enum **SCG_IrcTrimSrcType** { **SCG_IRC_TRIMSRC_RESERVE0** = 0U, **SCG_IRC_TRIMSRC_RESERVE1** = 1U, **SCG_IRC_TRIMSRC_FOSC** = 2U, **SCG_IRC_TRIMSRC_SOSC** = 3U }

SCG xxxIRC TRIMSRC type.

enum **SCG_ClockSrcType** { **SCG_CLOCK_SRC_FOSC** = 1U, **SCG_CLOCK_SRC_FIRC** = 3U, **SCG_CLOCK_SRC_PLL0** = 6U, **SCG_CLOCK_SRC_NONE** = 255U }

SCG_CCR [SCS] to Select system clock source.

enum **SCG_ClockDivType** { **SCG_CLOCK_DIV_BY1** = 0U, **SCG_CLOCK_DIV_BY2** = 1U, **SCG_CLOCK_DIV_BY3** = 2U, **SCG_CLOCK_DIV_BY4** = 3U }

SCG_CCR [DIVCORE]//SCG_CCR [DIVBUS]//SCG_CCR [DIVSLOW] to Select system clock source.

```
enum SCG_ClockoutSrcType { SCG_CLOCKOUT_SRC_OFF = 0U, SCG_CLOCKOUT_SRC_FOSC = 1U,
SCG_CLOCKOUT_SRC_SIRC = 2U, SCG_CLOCKOUT_SRC_FIRC = 3U, SCG_CLOCKOUT_SRC_SOSC = 4U,
SCG_CLOCKOUT_SRC_SIRC32K = 7U }
```

SCG_CLKOUTCFG[CLKOUTSEL] to Select system clock source.

```
enum SCG_NvmClkSrcType { SCG_NVMCLK_SRC_SIRC = 30u, SCG_NVMCLK_SRC_FIRC = 31u }
```

NVM clock source enumeration.

```
enum SCG_Cmu4ClkSrcType { SCG_CMU4CLK_SRC_SIRC = 28u, SCG_CMU4CLK_SRC_FOSC = 29u }
```

CMU4 clock source enumeration.

```
enum SCG_AsyncClockDivType { SCG_ASYNC_CLOCK_DISABLE = 0U, SCG_ASYNC_CLOCKDIV_BY1 = 1U,
SCG_ASYNC_CLOCKDIV_BY2 = 2U, SCG_ASYNC_CLOCKDIV_BY4 = 3U, SCG_ASYNC_CLOCKDIV_BY8 = 4U,
SCG_ASYNC_CLOCKDIV_BY16 = 5U, SCG_ASYNC_CLOCKDIV_BY32 = 6U, SCG_ASYNC_CLOCKDIV_BY64 = 7U }
```

[DIVL]/[DIVM]/[DIVH] bit field definition for SCG_SIRC/SCG_FIRC/SCG_FOSC/SCG_PLL0 registers

```
enum SCG_PllPredivType { SCG_PLLPREDIV_BY1 = 0U, SCG_PLLPREDIV_BY2 = 1U, SCG_PLLPREDIV_BY3
= 2U, SCG_PLLPREDIV_BY4 = 3U, SCG_PLLPREDIV_BY5 = 4U, SCG_PLLPREDIV_BY6 = 5U,
SCG_PLLPREDIV_BY7 = 6U, SCG_PLLPREDIV_BY8 = 7U, SCG_PLLPREDIV_BY9 = 8U,
SCG_PLLPREDIV_BY10 = 9U, SCG_PLLPREDIV_BY11 = 10U, SCG_PLLPREDIV_BY12 = 11U,
SCG_PLLPREDIV_BY13 = 12U, SCG_PLLPREDIV_BY14 = 13U, SCG_PLLPREDIV_BY15 = 14U,
SCG_PLLPREDIV_BY16 = 15U, SCG_PLLPREDIV_BY17 = 16U, SCG_PLLPREDIV_BY18 = 17U,
SCG_PLLPREDIV_BY19 = 18U, SCG_PLLPREDIV_BY20 = 19U, SCG_PLLPREDIV_BY21 = 20U,
SCG_PLLPREDIV_BY22 = 21U, SCG_PLLPREDIV_BY23 = 22U, SCG_PLLPREDIV_BY24 = 23U,
SCG_PLLPREDIV_BY25 = 24U, SCG_PLLPREDIV_BY26 = 25U, SCG_PLLPREDIV_BY27 = 26U,
SCG_PLLPREDIV_BY28 = 27U, SCG_PLLPREDIV_BY29 = 28U, SCG_PLLPREDIV_BY30 = 29U,
SCG_PLLPREDIV_BY31 = 30U, SCG_PLLPREDIV_BY32 = 31U }
```

SCG_PLLCFG[PREDIV] for PLL clock calculation. The pre-div value range is 0 ~ 31.

```
enum SCG_PllPstdivType { SCG_PLLPSTDIV_BY2 = 1U, SCG_PLLPSTDIV_BY4 = 2U, SCG_PLLPSTDIV_BY8
= 3U }
```

SCG_PLLCFG[PSTDIV] for PLL clock calculation.

```
enum SCG_PllSourceType { SCG_PLLSOURCE_FOSC = 0U, SCG_PLLSOURCE_FIRC = 1U }
```

SCG_PLLCFG[SOURCE] to Select PLL clock source.

```
enum SCG_IntrMaskType { SCG_INTR_MASK_FIRC_CLK_ERR = 1U, SCG_INTR_MASK_SIRC_CLK_ERR =
(1U << 1), SCG_INTR_MASK_FOSC_CLK_ERR = (1U << 2), SCG_INTR_MASK_SOSC_CLK_ERR = (1U << 3),
SCG_INTR_MASK_PLL0_CLK_ERR = (1U << 4), SCG_INTR_MASK_PLL1_CLK_ERR = (1U << 5) }
```

The interrupt enable/disable mask of SCG.

Functions

```
SCG_StatusType SCG_EnableSOSC (const SCG_SoscType *const pSoscConfig)
```


Enable SOSC.

SCG_StatusType SCG_DisableSOSC (void)

Disable SOSC.

SCG_StatusType SCG_GetSOSCValidStatus (void)

Get SOSC Valid status.

SCG_StatusType SCG_EnableFOSC (const **SCG_FoscType** *const pFoscConfig)

Enable FOSC clock with input configuration.

SCG_StatusType SCG_DisableFOSC (void)

Disable FOSC.

SCG_StatusType SCG_SetSIRC (const **SCG_SircType** *const pSircConfig)

Set SIRC configuration and configure SIRC DIV.

SCG_StatusType SCG_ClearSIRC (void)

Disable SIRC DIV and clear DIV configuration.

SCG_StatusType SCG_EnableSIRC32K (const **SCG_Sirc32kType** *const pSirc32kConfig)

Enable SIRC32K.

SCG_StatusType SCG_DisableSIRC32K (void)

Disable SIRC32K.

SCG_StatusType SCG_EnableFIRC (const **SCG_FircType** *const pFircConfig)

Enable FIRC.

SCG_StatusType SCG_DisableFIRC (void)

Disable FIRC.

SCG_StatusType SCG_EnablePLL (const **SCG_PllClkType** ePll, const **SCG_PllType** *const pPllConfig)

Enable PLL.

SCG_StatusType SCG_DisablePLL (const **SCG_PllClkType** ePll)

Disable PLL.

SCG_StatusType SCG_SwitchSystemClock (const **SCG_ClockSrcType** eClock)

Switch system clock source.

SCG_StatusType SCG_SetClkCtrl (const **SCG_ClockCtrlType** *const pSysClkConfig)

Set system run time clock and related CORE/BUS/SLOW clock.

uint32_t SCG_GetScgClockFreq (const **SCG_ClkSrcType** eScgClockName)

Get clock frequency.

void SCG_SetClkOut (const **SCG_ClockoutSrcType** eClkoutSel)

Select clock out source.

SCG_StatusType SCG_SetNvmClk (const **SCG_NvmClkSrcType** eNvmClkSrc)

Select NVM clock source.

SCG_StatusType SCG_SetCmu4Clk (const **SCG_Cmu4ClkSrcType** eCmu4ClkSrc)

Select CMU4 clock source.

SCG_StatusType SCG_CrcInit (**SCG_CrcModeType** eMode)

Initializes the CRC module based on the specified mode.

void SCG_CrcGenerate (void)

Generates a CRC value using the SCG hardware accelerator.

SCG_StatusType SCG_Deinit (void)

Clock source De-init.

void SCG_InterruptInit (**SCG_HandleType** *pScgHandle, const **SCG_InterruptType** *const pIntrStruct)

Set clock error interrupt.

```
void SCG_UserIRQHandler (SCG_HandleType *pScgHandle)
```

Clock error interrupt handler.

Detailed Description

SCG driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_scm.h File Reference

SCM driver type definition and API.

```
#include "HwA_scm.h"
```

Include dependency graph for module_driver_scm.h:

IMAGE

Enumerations

```
enum SCM_RetStatusType { SCM_E_OK = 0U, SCM_E_NOT_OK }
```

SCM return status.

```
enum SCM_CrcModeType { SCM_CRC_SOFTWARE_MODE = 0U, SCM_CRC_TRIGGER_MODE = 1U }
```

SCM CRC trigger mode.

Functions

```
void SCM_GetData_UID (uint32 *pUid)
```

Get unique identification for the chip, loaded from NVR.

```
void SCM_PeformReset_SYSAP (void)
```

Generate SYS-AP system reset.

```
SCM_RetStatusType SCM_CrcInit (SCM_CrcModeType eMode)
```

Initializes the CRC module based on the specified mode.

```
void SCM_CrcGenerate (void)
```

Generates the CRC (Cyclic Redundancy Check) by enabling the CRC generation in the hardware.

Detailed Description

SCM driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_scst.h File Reference

scst driver type definition and API

#include "device_header.h"

Include dependency graph for module_driver_scst.h:

IMAGE

Enumerations

```
enum Type_M7ST_AtomicStatus { M7ST_FaultInjectError, M7ST_ErrorAluTest, M7ST_ErrorAluMLATest,
M7ST_ErrorAluSHIFTest, M7ST_ErrorAluTest1t, M7ST_ErrorAluTest2, M7ST_ErrorAluTest3,
M7ST_ErrorAluTest4, M7ST_ErrorAluTest5, M7ST_ErrorAluTest6, M7ST_ErrorRegbankTest1,
M7ST_ErrorRegbankTest2, M7ST_ErrorRegbankTest3, M7ST_ErrorRegbankTest4,
M7ST_ErrorRegbankTest5, M7ST_ErrorRegbankTest6, M7ST_ErrorLoadStoreTest1,
M7ST_ErrorLoadStoreTest2, M7ST_ErrorLoadStoreTest3, M7ST_ErrorLoadStoreTest4,
M7ST_ErrorLoadStoreTest5, M7ST_ErrorLoadStoreTest6, M7ST_ErrorSimdSatTest1,
M7ST_ErrorSimdSatTest2, M7ST_ErrorSimdSatTest3, M7ST_ErrorSimdSatTest4, M7ST_ErrorMacTest1,
M7ST_ErrorMacTest2, M7ST_ErrorFetchTest, M7ST_ErrorStatusTest1, M7ST_ErrorStatusTest2,
M7ST_ErrorBranchTest1, M7ST_ErrorBranchTest2, M7ST_ErrorIntSvcTest,
M7ST_ErrorIntBusFaultTest, M7ST_ErrorIntHardFaultTest1, M7ST_ErrorIntHardFaultTest2,
M7ST_ErrorIntUsageFaultTest, M7ST_ErrorIntSystickTest, M7ST_ErrorIntPendSvTest,
M7ST_ErrorIntMemFaultTest, M7ST_ErrorIntMaskingTest, M7ST_ErrorIntHandlerThreadsTest,
M7ST_ErrorIntNMIHfTest, M7ST_ErrorIntTailChainTest, M7ST_ErrorIntAluTest,
M7ST_ErrorIntBranchTest, M7ST_ErrorIntStatusTest, M7ST_ErrorM7ST_TestPass }
```

typed of test result returned

```
enum SCST_TestIndexType { M7ST_AluTest, M7ST_AluMLATest, M7ST_AluSHIFTest, M7ST_AluTest1t,
M7ST_AluTest2, M7ST_AluTest3, M7ST_AluTest4, M7ST_AluTest5, M7ST_AluTest6,
M7ST_RegbankTest1, M7ST_RegbankTest2, M7ST_RegbankTest3, M7ST_RegbankTest4,
M7ST_RegbankTest5, M7ST_RegbankTest6, M7ST_LoadStoreTest1, M7ST_LoadStoreTest2,
M7ST_LoadStoreTest3, M7ST_LoadStoreTest4, M7ST_LoadStoreTest5, M7ST_LoadStoreTest6,
M7ST_SimdSatTest1, M7ST_SimdSatTest2, M7ST_SimdSatTest3, M7ST_SimdSatTest4,
M7ST_MacTest1, M7ST_MacTest2, M7ST_FetchTest, M7ST_StatusTest1, M7ST_StatusTest2,
M7ST_BranchTest1, M7ST_BranchTest2, M7ST_IntSvcTest, M7ST_IntBusFaultTest,
M7ST_IntHardFaultTest1, M7ST_IntHardFaultTest2, M7ST_IntUsageFaultTest, M7ST_IntSystickTest,
M7ST_IntPendSvTest, M7ST_IntMemFaultTest, M7ST_IntMaskingTest, M7ST_IntHandlerThreadsTest,
```

M7ST_IntNMIHfTest, M7ST_IntTailChainTest, M7ST_IntAluTest, M7ST_IntBranchTest,
M7ST_IntStatusTest, M7ST_RegressionTest }

typed of test index

Functions

Type_M7ST_AtomicStatus SCST_ExecuteTest (SCST_TestIndexType test_index, uint32_t
*s_u32RamBase)

This function is used to get the result of the executed test.

Detailed Description

scst driver type definition and API

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_sdadc.h File Reference

SDADC driver source code.

```
#include "HwA_sdadc.h"
```

```
#include "module_driver_dma.h"
```

Include dependency graph for module_driver_sdadc.h:

IMAGE

Classes

struct **_SDADC_HandleType**

struct **SDADC_PWMInitType**

struct **SDADC_IntInitType**

struct **SDADC_ModulatorCfgType**

struct **SDADC_DetectCfgType**

struct **SDADC_RectCfgType**

struct **SDADC_IntegrationCfgType**

struct **SDADC_FilterChainCfgType**

struct **SDADC_ChannelInitType**

SDADC Channel Initialization Structure.

struct **SDADC_InitType**

SDADC Initialization Structure.

Macros

```
#define SDADC_DEV_ERROR_REPORT STD_ON
```

```
#define SDADC_ReportDevError(func, error) ReportDevError(SDADC_MODULE_ID, func, error)
```

```
#define SDADC_CHANNEL_CNT 3U
```

```
#define SDADC_MFSHIFT_LEFT 0x10U
```

```
#define SDADC_MFSHIFT_RIGHT 0x0U
```

```
#define SDADC_MFSHIFT_DATALENGTH 24U
```

```
#define SDADC_E_OK (0U)
```

```
#define SDADC_E_NOT_OK (0xFFU)
```

```
#define SDADC_INIT_ID 0x00U
```

```
#define SDADC_DEINIT_ID 0x01U
```

```
#define SDADC_MAIN_ENABLE_ID 0x02U
```

```
#define SDADC_CHANNELSWTRIGGER_ID 0x03U
```



```

#define SDADC_CHANNELENABLE_ID 0x04U
#define SDADC_CLEARCHANNELSWTRIGGER_ID 0x05U
#define SDADC_INTEGRATORSWTRIGGER_ID 0x06U
#define SDADC_ENABLETSIGNTRIGGEROUT_ID 0x07U
#define SDADC_GETCONVERSIONRESULT_ID 0x08U
#define SDADC_GETCAPTURETIMESTAMP_ID 0x09U
#define SDADC_GETSIGNDELAYCAPTURE_ID 0x0AU
#define SDADC_GETCHANNELREADY_ID 0x0BU
#define SDADC_GETCHANNELCONVERTING_ID 0x0CU
#define SDADC_GETCHANNELCONVERSIONCOMPLETE_ID 0x0DU
#define SDADC_GETCHANNELFIFOEMPTY_ID 0x0EU
#define SDADC_GETCHANNELFIFOREADY_ID 0x0FU
#define SDADC_CLEARCHANNELFIFOREADY_ID 0x10U
#define SDADC_CLEARCHANNELCONVERSIONCOMPLETE_ID 0x11U
#define SDADC_STARTPWMGENERATION_ID 0x12U
#define SDADC_STOPPWMGENERATION_ID 0x13U
#define SDADC_ENABLEPWMSIGNTRIGGEROUT_ID 0x14U
#define SDADC_SETINTERRUPPTENABLE_ID 0x15U
#define SDADC_E_PARAM_POINTER 0x00U
#define SDADC_E_PARAM_INSTANCE 0x01U
#define SDADC_E_PARAM_CHANNEL 0x02U
#define SDADC_INSTANCE_0 0U,
#define SDADC_INSTANCE_1 1U

```

Typedefs

```

typedef uint8_t SDADC_RetStatusType
typedef uint8_t SDADC_ChannelMaskType
typedef uint8_t SDADC_InstanceType
typedef struct _SDADC_HandleType SDADC_HandleType

```

Enumerations

```

enum SDADC_TRGSignOENType { SDADC_TRGSIGN_OEN_DISABLED = 0x0,
SDADC_TRGSIGN_OEN_ENABLED = 0x1 }
enum SDADC_PWM_CLKSELType { SDADC_PWM_CLKSEL_DIV_CLK0 = 0x0,
SDADC_PWM_CLKSEL_DIV_CLK1 = 0x1, SDADC_PWM_CLKSEL_DIV_CLK2 = 0x2,
SDADC_PWM_CLKSEL_OFF = 0x3 }
enum SDADC_BitRevType { SDADC_BIT_REV_NORMAL = 0x0, SDADC_BIT_REV_REVERSE = 0x1 }

```

```

enum SDADC_PWMPolarityType { SDADC_PWM_POLARITY_POSITIVE = 0x0,
SDADC_PWM_POLARITY_NEGATIVE = 0x1 }

enum SDADC_PWMPatternType { SDADC_PWM_PATTERN_STOPPED = 0x0,
SDADC_PWM_PATTERN_SQUARE = 0x1, SDADC_PWM_PATTERN_TRIANGLE = 0x2,
SDADC_PWM_PATTERN_SINE = 0x3 }

enum SDADC_PrescaleType { SDADC_PRESCALE_NO_PRESCALE = 0x0, SDADC_PRESCALE_DIV_BY_2 =
0x1, SDADC_PRESCALE_DIV_BY_4 = 0x2, SDADC_PRESCALE_DIV_BY_8 = 0x3 }

PRE_DIV Divider Value.

enum SDADC_MENSELType { SDADC_MENSEL_INTERNAL = 0x0, SDADC_MENSEL_EXTERNAL = 0x1 }

Main Enable Selection.

enum SDADC_FIFOIsrStatusType { SDADC_FIFO_READY = 0x0, SDADC_FIFO_OVERFLOW = 0x1,
SDADC_FIFO_UNDERFLOW = 0x2 }

enum SDADC_DetectIsrStatusType { SDADC_DET_ZERO_CROSS = 0x0, SDADC_DET_HIGH_LMT_REACH =
0x1, SDADC_DET_LOW_LMT_REACH = 0x2, SDADC_DET_WIN_LMT_REACH = 0x3,
SDADC_DET_SHORT_CIRCUIT = 0x4, SDADC_DET_DATA SATURATION = 0x5, SDADC_DET_CLK_ABSENCE
= 0x6 }

enum SDADC_CICSelectionType { SDADC_SEL_AUXILIARY = 0x0, SDADC_SEL_MAIN = 0x1 }

enum SDADC_AFCMType { SDADC_AFCM_ALWAYS_ON = 0x0, SDADC_AFCM_FOLLOW_MAIN = 0x1 }

Auxiliary Filter Conversion Mode.

enum SDADC_AFORDType { SDADC_AFORD_RESERVED_00 = 0x0, SDADC_AFORD_FIRST_ORDER = 0x1,
SDADC_AFORD_SECOND_ORDER = 0x2, SDADC_AFORD_THIRD_ORDER = 0x3 }

Auxiliary Filter Order.

enum SDADC_MFCMType { SDADC_MFCM_SINGLE_MODE = 0x0, SDADC_MFCM_CONTINUOUS_MODE,
SDADC_MFCM_ALWAYS_MODE = 0x2, SDADC_MFCM_RESERVED_11 = 0x3 }

Main Filter Conversion Mode.

enum SDADC_MFORDType { SDADC_MFORD_RESERVED_000 = 0x0, SDADC_MFORD_FIRST_ORDER = 0x1,
SDADC_MFORD_SECOND_ORDER = 0x2, SDADC_MFORD_THIRD_ORDER = 0x3,
SDADC_MFORD_FOURTH_ORDER = 0x4, SDADC_MFORD_FIFTH_ORDER = 0x5,
SDADC_MFORD_RESERVED_110_111 = 0x6 }

Main Filter Order.

enum SDADC_LMTAVTType { SDADC_LMTAVT_SET_ON_LARGER_CLEAR_ON_SMALLER,
SDADC_LMTAVT_CLEAR_ON_LARGER_SET_ON_SMALLER }

Limitation Result Activation Select.

enum SDADC_LMTOPTType { SDADC_LMTOPT_DETECT_BOTH_HIGH_AND_LOW = 0x0,
SDADC_LMTOPT_DETECT_HIGH_THRESHOLD = 0x1, SDADC_LMTOPT_DETECT_LOW_THRESHOLD = 0x2,
SDADC_LMTOPT_DETECT_WINDOWED_VALUE = 0x3 }

Limitation Option.

```

enum **SDADC_SCDOPTType** { **SDADC_SCDOPT_COUNT_ON_BOTH_0_AND_1** = 0x0,
SDADC_SCDOPT_COUNT_ON_1 = 0x1, **SDADC_SCDOPT_COUNT_ON_0** = 0x2,
SDADC_SCDOPT_RESERVED = 0x3 }

Short Circuit Option.

enum **SDADC_SCDCMType** { **SDADC_SCDCM_ALWAYS_ON** = 0x0, **SDADC_SCDCM_FILTER_IN_PROGRESS**
= 0x1 }

Short Circuit Detect Conversion Mode.

enum **SDADC_SYNC_TRGENType** { **SDADC_SYNC_TRGEN_SELF** = 0x0, **SDADC_SYNC_TRGEN_DEFINED** =
0x1 }

Synced Trigger Select Enable.

enum **SDADC_TRGLVLType** { **SDADC_TRGLVL_EDGE** = 0x0, **SDADC_TRGLVL_LEVEL** = 0x1 }

Trigger Level type.

enum **SDADC_HTRPOLType** { **SDADC_HTRPOL_HIGH_RISING** = 0x0, **SDADC_HTRPOL_LOW_FALLING** =
0x1 }

Hardware Trigger Polarity.

enum **SDADC_TRGSELType** { **SDADC_TRGSEL_SOFTWARE** = 0x0, **SDADC_TRGSEL_HARDWARE** = 0x1 }

Trigger Select.

enum **SDADC_CLKEDGType** { **SDADC_CLKEDG_RESERVED** = 0x0, **SDADC_CLKEDG_POSEDGE** = 0x1,
SDADC_CLKEDG_NEGEDGE = 0x2, **SDADC_CLKEDG_BOTH_EDGE** = 0x3 }

Input Clock Edge Select.

enum **SDADC_CLKSELType** { **SDADC_CLKSEL_RESERVED** = 0x0, **SDADC_CLKSEL_DIV_CLK1** = 0x1,
SDADC_CLKSEL_DIV_CLK2 = 0x2, **SDADC_CLKSEL_EXTERNAL** = 0x3 }

Input Clock Select.

enum **SDADC_IBFMTType** { **SDADC_IBFMT_ONE_BIT_STREAM** = 0x0, **SDADC_IBFMT_MANCHESTER_CODE**
= 0x1 }

Input Bit Format.

enum **SDADC_RDFMTType** { **SDADC_RDFMT_LEFT_JUSTIFIED_SIGNED** = 0x0,
SDADC_RDFMT_LEFT_JUSTIFIED_UNSIGNED = 0x1 }

Result Data Format.

enum **SDADC_CTRGSIGN_OENType** { **SDADC_CTRGSIGN_OEN_DISABLED** = 0x0,
SDADC_CTRGSIGN_OEN_ENABLED = 0x1 }

Channel Sign Signal Related Trigger Out Enable.

enum **SDADC_SSCHType** { **SDADC_SSCH_CH0** = 0x0, **SDADC_SSCH_CH1** = 0x1, **SDADC_SSCH_CH2** = 0x2 }

Sign Signal From Channel Selection.

enum **SDADC_SSRCType** { **SDADC_SSRC_INTERNAL_PWM** = 0x0, **SDADC_SSRC_CHANNEL_SELECTED** =
0x1, **SDADC_SSRC_EXTERNAL_SIGNAL_0** = 0x2, **SDADC_SSRC_EXTERNAL_SIGNAL_1** = 0x3 }

Sign Source Selection.

enum **SDADC_EXTSTPType** { **SDADC_EXTSTP_INTERNAL** = 0x0, **SDADC_EXTSTP_EXTERNAL** = 0x1 }

Integration Stop Control.

enum **SDADC_IOGTType** { **SDADC_IOGT_ALWAYS_WORK** = 0x0,
SDADC_IOGT_ONLY_WHEN_INTEGRATION_ONGOING = 0x1 }

Integration Output Gating.

enum **SDADC_MASELType** { **SDADC_MASEL_EXTERNAL** = 0x0, **SDADC_MASEL_INTERNAL** = 0x1 }

Modulator Analog Selection.

enum **SDADC_MACLKSELType** { **SDADC_MACLKSEL_DIV_CLK1** = 0x0, **SDADC_MACLKSEL_DIV_CLK0** = 0x1,
SDADC_MACLKSEL_RESERVED = 0x2 }

Internal Modulator Analog Clock Selection.

enum **SDADC_DITHER_ENType** { **SDADC_DITHER_EN_DISABLED** = 0x0, **SDADC_DITHER_EN_ENABLED** =
0x1 }

Dither Enable.

enum **SDADC_INCFGPV11Type** { **SDADC_INCFGPV11_DISABLED** = 0x0, **SDADC_INCFGPV11_ENABLED** =
0x1 }

Configuration of Positive Input Line with V11 from System.

enum **SDADC_GAINSELType** { **SDADC_GAINSEL_1** = 0x0, **SDADC_GAINSEL_2** = 0x1, **SDADC_GAINSEL_4** =
0x2, **SDADC_GAINSEL_8** = 0x3 }

Gain Select of Analog Input Path.

enum **SDADC_CMENPType** { **SDADC_CMENP_NOT_CONNECTED** = 0x0, **SDADC_CMENP_CONNECTED** =
0x1 }

Common Voltage Configuration of Positive Inputs.

enum **SDADC_CMENMType** { **SDADC_CMENM_NOT_CONNECTED** = 0x0, **SDADC_CMENM_CONNECTED** =
0x1 }

Common Voltage Configuration of Negative Inputs.

enum **SDADC_INCFGPTType** { **SDADC_INCFGP_FROM_PIN** = 0x0, **SDADC_INCFGP_FROM_VREFL** = 0x1,
SDADC_INCFGP_FROM_VREFH = 0x2, **SDADC_INCFGP_FROM_VREFH_DIV_2** = 0x3 }

Configuration of Positive Input Line.

enum **SDADC_INCFGMTType** { **SDADC_INCFGM_FROM_PIN** = 0x0, **SDADC_INCFGM_FROM_VREFL** = 0x1,
SDADC_INCFGM_FROM_VREFH = 0x2, **SDADC_INCFGM_FROM_VREFH_DIV_2** = 0x3 }

Configuration of Negative Input Line.

enum **SDADC_INSELType** { **SDADC_INSEL_PIN_0** = 0x0, **SDADC_INSEL_PIN_1** = 0x1, **SDADC_INSEL_PIN_2**
= 0x2, **SDADC_INSEL_PIN_3** = 0x3 }

Input Pin Selection.

```
enum SDADC_INMODType { SDADC_INMOD_SOFTWARE_CONTROL = 0x0,
SDADC_INMOD_PRESET_MODE = 0x1, SDADC_INMOD_SCAN_MODE = 0x2, SDADC_INMOD_RESERVED =
0x3 }
```

Input MUX Control Mode.

```
enum SDADC_TSRDMType { SDADC_TSRDM_DEFAULT_MODE = 0x0, SDADC_TSRDM_DIRECT_MODE =
0x1, SDADC_TSRDM_FIFO_GATING_MODE = 0x2, SDADC_TSRDM_RESERVED = 0x3 }
```

Timestamp Information Read Mode.

```
enum SDADC_TSTRGMType { SDADC_TSTRGM_IGNORED = 0x0, SDADC_TSTRGM_RISING_EDGE = 0x1,
SDADC_TSTRGM_FALLING_EDGE = 0x2 }
```

Timestamp Counter Trigger Mode.

```
enum SDADC_INMXCPTType { SDADC_INMXCP_DISABLED = 0x0, SDADC_INMXCP_ENABLED = 0x1 }
```

Input Mux Setting Copy Enable.

```
enum SDADC_TS_ENType { SDADC_TS_EN_DISABLED = 0x0, SDADC_TS_EN_ENABLED = 0x1 }
```

Timestamp Counter Enable.

```
enum SDADC_TSCLKSELType { SDADC_TSCLKSEL_MODULATOR_CLOCK = 0x0,
SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_2 = 0x1, SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_4 =
0x2, SDADC_TSCLKSEL_MODULATOR_CLOCK_DIV_8 = 0x3 }
```

Timestamp Counter Clock Selection.

```
enum SDADC_GainCompType { SDADC_GAIN_CMP_BYPASS = 0x0, SDADC_GAIN_CMP_INTERNAL_GAIN1
= 0x1, SDADC_GAIN_CMP_INTERNAL_GAIN2 = 0x2, SDADC_GAIN_CMP_EXTERNAL_GAIN = 0x3 }
```

```
enum SDADC_InterruptType { SDADC_COCI_INTERRUPT = 0, SDADC_FIFO_READY_INTERRUPT = 1,
SDADC_FIFO_OVERFLOW_INTERRUPT = 2, SDADC_FIFO_UNDERFLOW_INTERRUPT = 3,
SDADC_ZERO_CROSS_DETECT_INTERRUPT = 4, SDADC_HIGH_LIMIT_REACH_INTERRUPT = 5,
SDADC_LOW_LIMIT_REACH_INTERRUPT = 6, SDADC_WINDOW_LIMIT_REACH_INTERRUPT = 7,
SDADC_SHORT_CIRCUIT_DETECT_INTERRUPT = 8, SDADC_DATA_SATURATION_DETECT_INTERRUPT =
9, SDADC_CLOCK_ABSENCE_DETECT_INTERRUPT = 10, SDADC_SIGN_DELAY_CAPTURE_INTERRUPT =
11, SDADC_TSIGN_CHANGED_INTERRUPT = 12, SDADC_TS_VALID_INTERRUPT = 13,
SDADC_INTERRUPT_MAX = 14 }
```

Functions

```
void SDADC_InitCfg (SDADC_InitType *pInitCfg)
```

Initialize the SDADC configuration.

```
void SDADC_InitChannelCfg (SDADC_ChannelInitType *pInitCfg)
```

Initialize the SDADC channel configuration.

void **SDADC_InitModulatorCfg** (SDADC_ModulatorCfgType *pInitCfg)

Initialize the SDADC modulator configuration.

void **SDADC_InitPwmCfg** (SDADC_PWMInitType *pInitCfg)

Initialize the SDADC PWM configuration.

void **SDADC_InitFilterChainCfg** (SDADC_FilterChainCfgType *pInitCfg)

Initialize the SDADC filter chain configuration.

void **SDADC_InitDectCfg** (SDADC_DetectCfgType *pInitCfg)

Initialize the SDADC detection configuration.

SDADC_RetStatusType **SDADC_Init** (SDADC_HandleType *pSdadCHandle, const **SDADC_InitType** *const pInitCfg)

Initialize the SDADC with the given configuration.

void **SDADC_Deinit** (SDADC_HandleType *pSdadCHandle, uint32_t RstHoldCount)

Deinitialize the SDADC.

void **SDADC_ChannelEnable** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint8_t u8Enable)

Enable or disable a specific SDADC channel.

SDADC_RetStatusType **SDADC_MainEnable** (SDADC_HandleType *pSdadCHandle, uint8_t u8Enable)

Enable or disable the main functionality of the SDADC.

void **SDADC_ChannelSWTrigger** (SDADC_HandleType *pSdadCHandle, **SDADC_ChannelMaskType** u8ChannelMask)

Trigger the SDADC channels via software.

void **SDADC_ClearChannelSWTrigger** (SDADC_HandleType *pSdadCHandle, **SDADC_ChannelMaskType** u8ChannelMask)

Clear the software trigger status of specified SDADC channels.

void **SDADC_IntegratorSWTrigger** (SDADC_HandleType *pSdadCHandle, SDADC_ChannelMaskType u8ChannelMask)

Trigger the SDADC integrator via software.

void **SDADC_EnablePWMSignTriggerOut** (SDADC_HandleType *pSdadCHandle, uint8_t u8Enable)

Enable or disable the Pwm Sign trigger out for a specific SDADC channel.

void **SDADC_EnableTSignTriggerOut** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint8_t u8Enable)

Enable or disable the TSign trigger out for a specific SDADC channel.

SDADC_RetStatusType **SDADC_GetConversionResult** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint32_t *pData)

Get the conversion result of a specific SDADC channel.

SDADC_RetStatusType **SDADC_GetCaptureTimeStamp** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint16_t *pTimestamp, uint16_t *pLastCov)

Get the capture timestamp of a specific SDADC channel.

SDADC_RetStatusType **SDADC_GetSignDelayCapture** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, uint8_t *pCapture)

Get the sign delay capture of a specific SDADC channel.

uint8_t **SDADC_GetChannelReady** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the ready status of a specific SDADC channel.

uint8_t **SDADC_GetChannelConverting** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the converting status of a specific SDADC channel.

uint8_t **SDADC_GetChannelConversionComplete** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the conversion complete status of a specific SDADC channel.

uint8_t **SDADC_GetChannelFIFOEmpty** (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the FIFO empty status of a specific SDADC channel.

uint8_t SDADC_GetChannelFIFOReady (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Get the FIFO ready status of a specific SDADC channel.

void SDADC_ClearChannelFIFOReady (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Clear the FIFO ready status of a specific SDADC channel.

void SDADC_ClearChannelConversionComplete (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Clear the conversion complete status of a specific SDADC channel.

SDADC_RetStatusType SDADC_StartPwmGeneration (SDADC_HandleType *pSdadCHandle, SDADC_PWMPatternType ePattern)

Start PWM generation with the specified pattern.

SDADC_RetStatusType SDADC_StopPwmGeneration (SDADC_HandleType *pSdadCHandle)

Stop PWM generation.

void SDADC_ClearIntegratorSWTrigger (SDADC_HandleType *pSdadCHandle, SDADC_ChannelMaskType u8ChannelMask)

Clear the integrator software trigger for the specified SDADC channels.

void SDADC_SetInterruptEnable (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex, SDADC_InterruptType eIntType, uint8_t u8Enable)

Set the interrupt enable status for the specified SDADC interrupt type.

void SDADCn_IRQHandler (SDADC_HandleType *pSdadCHandle, uint8_t u8ChannelIndex)

Detailed Description

SDADC driver source code.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_sddf.h File Reference

Sigma Delta Digital Filter (SDDF) driver type definition and API.

```
#include "HwA_sddf.h"
```

```
#include "module_driver_dma.h"
```

Include dependency graph for module_driver_sddf.h:

IMAGE

Classes

struct **SDDF_ChannelNotificationType**

struct **_SDDF_HandleType**

struct **SDDF_InitConfigType**

The init configuration for SDDF instance.

struct **SDDF_ClockOutConfigType**

The clock out configuration for SDDF.

struct **SDDF_MainFilterConfigType**

The configuration for main filter.

struct **SDDF_AuxFilterConfigType**

The configuration for auxiliary filter.

struct **SDDF_ChannelConfigType**

The configuration of the SDDF channel.

struct **SDDF_ClockAbsenceDetectConfigType**

The configuration of the clock absence detect.

struct **SDDF_ShortCircuitDetectConfigType**

The configuration of the short circuit detect.

struct **SDDF_LimitCheckConfigType**

The configuration of the limit detect.

struct **SDDF_ZeroCrossDetectConfigType**

The configuration of the zero cross detect.

struct **SDDF_ChannelInterruptConfigType**

The result related interrupt configuraion of the SDDF channel.

struct **SDDF_ChannelDMAConfigType**

The DMA configuration of the SDDF channel.

Macros

SDDF Dev Error Code *Error Code of calling SDDF apis*

```
#define SDDF_E_PARAM_INSTANCE 0x01U
#define SDDF_E_PARAM_CHANNEL 0x02U
#define SDDF_E_PARAM_POINTER 0x03U
#define SDDF_E_PARAM_MAIN_FILTER 0x04U
#define SDDF_E_PARAM_AUX_FILTER 0x05U
SDDF API Service IDs#define SDDF_INIT_ID 0x00U
#define SDDF_DEINIT_ID 0x01U
#define SDDF_CFG_CLK_OUT_ID 0x02U
#define SDDF_ENABLE_ID 0x03U
#define SDDF_DISABLE_ID 0x04U
#define SDDF_EN_DBG_MODE_ID 0x05U
#define SDDF_DIS_DBG_MODE_ID 0x06U
#define SDDF_INIT_CH_ID 0x07U
#define SDDF_CH_ENABLE_ID 0x08U
#define SDDF_CH_DISABLE_ID 0x09U
#define SDDF_CFG_CH_INT_ID 0x0AU
#define SDDF_CFG_CH_DMA_ID 0x0BU
#define SDDF_CFG_CH_CLK_ABS_DET_ID 0x0CU
#define SDDF_CFG_CH_SHORT_CIR_DET_ID 0x0DU
#define SDDF_CFG_CH_LIM_CHK_ID 0x0EU
#define SDDF_CFG_CH_ZERO_CROSS_DET_ID 0x0FU
#define SDDF_GEN_CH_SW_TRIG_ID 0x10U
#define SDDF_GET_CH_FIFO_DATA_ID 0x11U
```

Typedefs

```
typedef void(* SDDF_GeneralCallbackType) (void)
```

The general callback function prototype for SDDF interrupt.

```
typedef void(* SDDF_FIFOReadyCallbackType) (const uint32_t *const pData)
```

Function prototype for SDDF FIFO ready interrupt.

```
typedef void(* SDDF_EventDetectCallbackType) (const SDDF_DetectEventType eEvent)
```

Function prototype for SDDF event detect interrupt.

typedef struct **_SDDF_HandleType** SDDF_HandleType

Enumerations

enum **SDDF_InstanceType** { **SDDF_INSTANCE_0** = 0U }

The instance index of the SDDF peripheral.

enum **SDDF_DetectEventType** { **SDDF_CLOCK_ABSENCE_EVENT** = 0U, **SDDF_SHORT_CIRCUIT_EVENT** = 1U, **SDDF_LOW_LIMIT_EVENT** = 2U, **SDDF_HIGH_LIMIT_EVENT** = 3U, **SDDF_WINDOW_LIMIT_EVENT** = 4U, **SDDF_ZERO_CROSS_EVENT** = 5U }

The event detected by SDDF detectors.

enum **SDDF_ClockOutType** { **SDDF_CLOCK_OUT_DISABLED** = 0U, **SDDF_CLOCK_OUT_0** = 1U, **SDDF_CLOCK_OUT_1** = 2U, **SDDF_CLOCK_OUT_2** = 3U }

Select the clock output source.

enum **SDDF_ClkokInputType** { **SDDF_CLOCK_INPUT_CLK0** = 0U, **SDDF_CLOCK_INPUT_CLK1** = 1U, **SDDF_CLOCK_INPUT_CLK2** = 2U, **SDDF_CLOCK_INPUT_CLK3** = 3U }

Select the clock channel for SDDF external clock input.

Functions

void **SDDF_DeInit** (SDDF_HandleType *pHandle)

De-initialize the SDDF instance.

void **SDDF_Init** (SDDF_HandleType *pHandle, const **SDDF_InitConfigType** *pInitCfg)

Initialize the SDDF instance.

void **SDDF_ConfigClockOut** (SDDF_HandleType *pHandle, const **SDDF_ClockOutConfigType** *pClockOutCfg)

Configure the SDDF clock out.

void **SDDF_Enable** (SDDF_HandleType *pHandle)

Enable the SDDF instance.

void **SDDF_Disable** (SDDF_HandleType *pHandle)

Disable the SDDF instance.

void **SDDF_EnableDebugMode** (SDDF_HandleType *pHandle, **SDDF_DebugModuleEnableType** eModuleEnable)

Enable the debug mode of the SDDF instance.

void SDDF_DisableDebugMode (SDDF_HandleType *pHandle)

Disable the debug mode of the SDDF instance.

void SDDF_InitChannel (SDDF_HandleType *pHandle, uint8_t u8Channel, const SDDF_ChannelConfigType *pChannelCfg)

Initialize the selected SDDF channel.

void SDDF_ChannelEnable (SDDF_HandleType *pHandle, uint8_t u8Channel)

Enable the conversion of the selected SDDF channel.

void SDDF_ChannelDisable (SDDF_HandleType *pHandle, uint8_t u8Channel)

Disable the conversion of the selected SDDF channel.

void SDDF_ConfigChannelInterrupt (SDDF_HandleType *pHandle, uint8_t u8Channel, const SDDF_ChannelInterruptConfigType *pIntCfg)

Configure the interrupt for the selected SDDF channel.

void SDDF_ConfigChannelDMA (SDDF_HandleType *pHandle, uint8_t u8Channel, const SDDF_ChannelDMAConfigType *pDMACfg, DMA_TransferCompleteCallbackType pDmaCallback)

Configure the DMA for the selected the SDDF channel.

void SDDF_ConfigChannelClockAbsenceDetect (SDDF_HandleType *pHandle, uint8_t u8Channel, SDDF_ClockAbsenceDetectConfigType *pConfig)

Configure the clock absence detect (CAD) for the selected SDDF channel.

void SDDF_ConfigChannelShortCircuitDetect (SDDF_HandleType *pHandle, uint8_t u8Channel, SDDF_ShortCircuitDetectConfigType *pConfig)

Configure the short circuit detect (SCD) for the selected SDDF channel.

void SDDF_ConfigChannelLimitCheck (SDDF_HandleType *pHandle, uint8_t u8Channel, SDDF_LimitCheckConfigType *pConfig)

Configure the limit check for the selected SDDF channel.

void **SDDF_ConfigChannelZeroCrossDetect** (**SDDF_HandleType** *pHandle, uint8_t u8Channel, **SDDF_ZeroCrossDetectConfigType** *pConfig)

Configure the zero cross detect (ZCD) for the selected SDDF channel.

void **SDDF_GenerateChannelSwTrigger** (**SDDF_HandleType** *pHandle, uint8_t u8Channel)

Generate software trigger for the selected SDDF channel.

uint32_t **SDDF_GetChannelFIFOData** (**SDDF_HandleType** *pHandle, uint8_t u8Channel)

Get the channel data for the SDDF channel.

void **SDDFn_IRQHandler** (**SDDF_HandleType** *pHandle)

void **SDDFn_CHnDMAHandler** (**SDDF_HandleType** *pHandle, uint8_t u8Channel)

Detailed Description

Sigma Delta Digital Filter (SDDF) driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_sec.h File Reference

sec driver type definition and API

#include "HwA_sec.h"

Include dependency graph for module_driver_sec.h:

IMAGE

Classes

struct **ReEnDebug_Keys**

Debug Re-Enable Key.

struct **NVR_Per**

NVR write and read Permissions. .

struct **SEC_IspInfo**

Indicate the ISP information.

struct **SEC_BootInfo**

Indicate the boot information.

struct **HostUKPermission**

Indicate the Host User Key Read/Write/Erase Protection.

struct **HostNVRPermission**

Indicate the Host NVR Read/Write/Erase Protection.

struct **Hsm_configuration**

Indicate the HSM configuration.

Enumerations

enum **SEC_RetType** { **SEC_STATUS_SUCCESS** = 0U, **SEC_STATUS_FAILED** = 1U }

enum **Systemstate** { **Securedstate** = 0u, **UnSecuredstate** = 1u }

Indicate the system state.

enum **Isp_Instance** { **ISP_FCUART1** = 0U, **ISP_FCUART3** = 1U, **ISP_FLEXCAN1** = 2U, **ISP_FLEXCAN5** = 3U }

Indicate the ISP instance.

enum **FCUART_ISP_BAUDRATE** { **ISP_FCUART_2MBPS** = 0u, **ISP_FCUART_1MBPS** = 1u,
ISP_FCUART_512KBPS = 2u, **ISP_FCUART_115200BPS** = 3u, **ISP_FCUART_INVALID** = 4u }

Indicate the FCUART Baud Rate for ISP.

enum **FLEXCAN_ISP_BAUDRATE** { **ISP_FLEXCANFD_4MBPS** = 0u, **ISP_FLEXCANFD_2MBPS** = 1u, **ISP_FLEXCAN_1MBPS** = 2u, **ISP_FLEXCAN_500kBPS** = 3u, **ISP_FLEXCAN_INVALID** = 4u }

Indicate the FLEXCAN Baud Rate for ISP.

enum **FastBoot_Speed** { **FASTBOOT_300MHZ** = 0u, **FASTBOOT_96MHZ** = 1u }

Indicate the fast boot speed.

enum **Osc_FreqInfo** { **OSC_8MHZ** = 0u, **OSC_12MHZ** = 1u, **OSC_16MHZ** = 2u, **OSC_24MHZ** = 3u, **OSC_25MHZ** = 4u, **OSC_32MHZ** = 5u, **OSC_48MHZ** = 6u, **OSC_40MHZ** = 7u }

Indicate the OSC Frequency.

enum **SC_LifeCycle** { **LIFECYCLE_INVALID** = 0U, **LIFECYCLE_OEM_DEV** = 0X1U, **LIFECYCLE_OEM_PDT** = 0X2U, **LIFECYCLE_INFIELD** = 0X4U, **LIFECYCLE_PREFA** = 0X8U, **LIFECYCLE_FA** = 0X10U }

Indicate the lifecycle of the chip.

enum **BL_VerifyAlgorithm** { **BL_ALGORITHM_CMACE** = 0U, **BL_ALGORITHM_ECC256_SHA256** = 1U, **BL_ALGORITHM_SM2_SM3** = 7U }

Indicate the Bootloader Verification Algorithm.

enum **Decryption_Algorithm** { **DECRYPTTION_ECC256** = 0U, **DECRYPTTION_SM2** = 1U }

Indicate the Debug/ISP/PREFA Authentication and USRK decryption algorithm.

Functions

bool **SEC_EnDebugMode** (void)

This function can be used to enable the Debug mode.

bool **SEC_ReEnDebugMode** (**ReEnDebug_Keys** keys)

This function can be used to re-enable the Debug mode by the re-enable keys.

Systemstate **SEC_SystemSecureState** (void)

This function can get the system state (secured or no secured).

bool **SEC_EnTestMode** (void)

This function can enable the test mode.

bool **SEC_ReEnTestMode** (uint32_t key)

This function can Re-enable the test mode by the re-enable key.

bool **SEC_SetNvrPer** (**NVR_Per** per)

This function can be used to set the NVR write and read permission.

bool **SEC_GetSecureStatus** (void)

Get Secure Boot Enable Status.

bool **SEC_GetISPModeStatus** (void)

Get ISP Status.

uint32_t **SEC_GetBootAddress** (void)

Get User setting bootloader address.

uint32_t **SEC_GetHsmFwAddress** (void)

Get the HSM Firmware Address.

SC_LifeCycle **SEC_GetLifeCycle** (void)

Get the lifecycle.

bool **SEC_HostUKAccess** (void)

User Key Access Enable. Only valid under non-secure boot.

bool **SEC_GetHostFwDisAccess** (void)

Firmware Read Disable/Enable. (Value loaded from NVR sector)

void **SEC_DisbleFwReadAccess** (bool readaccess)

Set the Firmware Read access.

bool **SEC_GetFwValid** (void)

Get whether Flash Firmware is valid.

BL_VerifyAlgorithm **SEC_GetBLVerifyAlgorithm** (void)

get the Bootloader Verification Algorithm

Decryption_Algorithm **SEC_GetDecryptAlgorithm** (void)

get the Debug/ISP/PREFA Authentication and USRK decryption algorithm

void **SEC_GetHostUKAccess** (**HostUKPermission** *const PHostUKAccess)

Indicate the Host User Key Read/Write/Erase Protection.

void **SEC_GetHostNVRAccess** (**HostNVRPermission** *const PHostNvrAccess)

Indicate the Host NVR Read/Write/Erase Protection.

bool **SEC_GethsmUKEraseAccess** (void)

Get the HSM User Key Erase Protection.

bool **SEC_GethsmNVREraseAccess** (void)

Get the HSM NVR Erase Protection.

uint32_t **SEC_GetBLVerMask** (void)

Get Bootloader Verification Mask.

bool **SEC_GetDMBDkeyEn** (void)

Get whether Debug Mailbox Backdoor Key Enable.

bool **SEC_GetDebugAuthEn** (void)

Host Debug Auth Enable. Only valid in secure boot. (Value loaded from NVR sector)

SEC_RetType **SEC_GetIsplInfo** (**SEC_IsplInfo** *const plsplInfo)

Get the isp information.

SEC_RetType **SEC_GetBootInfo** (**SEC_BootInfo** *const pBootInfo)

Get the boot information.

SEC_RetType **SEC_GetHSMInfo** (**Hsm_configuration** *const pHsmInfo)

Get the HSM information.

Detailed Description

sec driver type definition and API

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_sema.h File Reference

#include "HwA_mb.h"

Include dependency graph for module_driver_sema.h:

IMAGE

module_driver_sent.h File Reference

SENT driver type definition and API.

```
#include "HwA_sent.h"
```

Include dependency graph for module_driver_sent.h:

IMAGE

Classes

struct **Sent_ChannelInterruptType**

Structure to enable or disable the channel interrupts.

struct **Sent_FastMessageDataType**

Structure for fast message buffer.

struct **Sent_SlowMessageDataType**

Structure for slow message buffer.

struct **Sent_ConfigType**

structure to configure the SENT.

struct **Sent_ChannelConfigType**

structure to configure the SENT channel.

struct **Sent_DMAConfigType**

structure to configure the SENT channel DMA.

struct **Sent_SpcConfigType**

structure to configure the SENT channel SPC.

struct **_Sent_HandleType**

The structure of the SENT processing handle.

Macros

SENT API Service IDs *Defines the service IDs for the FCUART module, which are used to identify different functions or services provided by the SENT layer. These IDs are utilized when communicating with the SENT module, enabling calls to or responses from various functions.*

```
#define SENT_INIT_ID 0U
```

```
#define SENT_SET_CHANNEL_CONFIG_ID 1U
```

```
#define SENT_SET_DMA_CONFIG_ID 2U
```

```
#define SENT_SET_SPC_CONFIG_ID 3U
```

```

#define SENT_GET_FAST_MSG_ID 4U
#define SENT_GET_SLOW_MSG_ID 5U
#define SENT_DECODE_FAST_MSG_ID 6U
#define SENT_DECODE_SLOW_MSG_ID 7U
#define SENT_ENABLE_FAST_MSG_NOTIFY_ID 8U
#define SENT_DISABLE_FAST_MSG_NOTIFY_ID 9U
#define SENT_ENABLE_SLOW_MSG_NOTIFY_ID 10U
#define SENT_DISABLE_SLOW_MSG_NOTIFY_ID 11U
#define SENT_ENABLE_FAST_FIFO_MSG_NOTIFY_ID 12U
#define SENT_DISABLE_FAST_FIFO_MSG_NOTIFY_ID 13U
#define SENT_CONFIG_CHANNEL_STATUS_NOTIFY_ID 14U
#define SENT_REQUEST_SPC_PULSE_ID 15U
#define SENT_GET_FAST_MSG_READY_FLAG_ID 16U
#define SENT_GET_SLOW_MSG_READY_FLAG_ID 17U
#define SENT_GET_CHANNEL_STATUS_ID 18U
#define SENT_CLEAR_CHANNEL_STATUS_ID 29U
#define SENT_GET_SPC_BUSY_FLAG_ID 20U

```

SENT Dev Error Code *Error Code of calling SENT apis*

```

#define SENT_E_PARAM_POINTER 0x01U
#define SENT_E_ALREADY_INIT 0x02U
#define SENT_E_UNINIT 0x03U
#define SENT_E_PARAM_INSTANCE 0x04U
#define SENT_E_PARAM_CHANNEL 0x05U
#define SENT_E_PARAM_GENERAL 0x06U
#define SENT_E_INVALID_BRANCH 0x07U

```

SENT buffer address and size in DMA mode

```

#define SENT0_GET_FAST_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DFD))
#define SENT1_GET_FAST_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DFD))
#define SENT_GET_FAST_MSG_DMA_BYTELENGTH (12U)
#define
#define SENT0_GET_SLOW_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DSB3))
#define
#define SENT1_GET_SLOW_MSG_DMA_ADDR(echannel) (&(SENT0->SENT_CHN[echannel].CHN_DSB3))
#define SENT_GET_SLOW_MSG_DMA_BYTELENGTH (12U)

```

Typedefs

```

typedef void(* SENT_GBISRCallbackType) (Sent_ChannelType eChannel, const
Sent_GlobalInterruptType eInterrupt)

```

SENT Global ISR callback function prototype.

```
typedef void(* SENT_CHISRCallbackType) (Sent_ChannelType eChannel, const uint32_t  
u32ChannelStatus)
```

SENT Channel ISR callback function prototype.

```
typedef struct _Sent_HandleType Sent_HandleType
```

The structure of the SENT processing handle.

Enumerations

```
enum Sent_ReturnType { SENT_RETURN_OK = 0x00U, SENT_RETURN_E_NOT_OK = 0x01U,  
SENT_RETURN_E_ALREADY_INIT = 0x02U, SENT_RETURN_E_UNINIT = 0x03U, SENT_RETURN_E_PARAM  
= 0x04U }
```

SENT operation return values.

```
enum Sent_ChannelType { SENT_CHANNEL_0 = 0x00U, SENT_CHANNEL_1 = 0x01U, SENT_CHANNEL_2 =  
0x02U, SENT_CHANNEL_3 = 0x03U, SENT_CHANNEL_MAX = SENT_CHANNEL_COUNT }
```

SENT Channel index.

```
enum Sent_InstanceType { SENT_INSTANCE_0 = 0x00U, SENT_INSTANCE_1 = 0x01U,  
SENT_INSTANCE_MAX = SENT_INSTANCE_COUNT }
```

SENT instance index.

```
enum Sent_DataNibbleModeType { SENT_DATA_NIBBLE_MODE_A = 0x0U,  
SENT_DATA_NIBBLE_MODE_H1 = 0x1U, SENT_DATA_NIBBLE_MODE_H2 = 0x2U,  
SENT_DATA_NIBBLE_MODE_H3 = 0x3U, SENT_DATA_NIBBLE_MODE_H4 = 0x4U,  
SENT_DATA_NIBBLE_MODE_H5 = 0x5U, SENT_DATA_NIBBLE_MODE_H6 = 0x6U,  
SENT_DATA_NIBBLE_MODE_H7 = 0x7U }
```

SENT nibble data mode control. Refer to SENT SAE J2716 2010 for detail protocol definition.

```
enum Sent_CalibrationValidType { SENT_CALIBRATION_VALID_DISABLE = 0x0U,  
SENT_CALIBRATION_VALID_WITHIN_20 = 0x1U, SENT_CALIBRATION_VALID_FROM_20_TO_25 = 0x2U }
```

SENT calibration valid type.

```
enum Sent_CalDiagOptionType { SENT_CALIBRATION_PULSE_DIAG_OPTION2 = 0x0U,  
SENT_CALIBRATION_PULSE_DIAG_OPTION1 = 0x1U }
```

SENT Successive calibration pulses diagnostic option.

```
enum Sent_IdleCountType { SENT_IDLE_COUNT_FLAG_DISABLE = 0x0U,  
SENT_IDLE_COUNT_FLAG_254_TICKS = 0x1U, SENT_IDLE_COUNT_FLAG_508_TICKS = 0x2U,  
SENT_IDLE_COUNT_FLAG_1016_TICKS = 0x4U, SENT_IDLE_COUNT_FLAG_2032_TICKS = 0x8U }
```

Determines how long the bus idle flag will assert when SENT bus is idle.

```
enum Sent_SerialMessageType { SENT_SERIAL_MESSAGE_SHORT = 0x0U,  
SENT_SERIAL_MESSAGE_ENHANCE_12DATA_8ID = 0x1U,  
SENT_SERIAL_MESSAGE_ENHANCE_16DATA_4ID = 0x2U }
```

SENT Slow message type.

```
enum Sent_SpcTriggerType { SENT_SPC_SOFTWARE_TRIGGER = 0x0U, SENT_SPC_EXTERNAL_TRIGGER  
= 0x1U }
```

SENT trigger type in SPC mode.

```
enum Sent_SpcTickBaseType { SENT_SPC_TICK_BASE_PRE_MSG = 0x0U,  
SENT_SPC_TICK_BASE_CONFIGURED = 0x1U }
```

Select the tick base in SPC mode.

```
enum Sent_GlobalInterruptType { SENT_SLOW_MSG_DMA_UF_IT = 0U, SENT_FAST_MSG_DMA_DF_IT =  
1U, SENT_FAST_MSG_READY_IT = 2U, SENT_SLOW_MSG_READY_IT = 3U, SENT_FAST_MSG_FIFO_OF_IT =  
4U }
```

Sent global interrupt type.

Functions

```
void SENT_InterruptProcess (const Sent_HandleType *pSentHandle)
```

Process sent interrupt.

```
void SENT_Init (Sent_HandleType *pSentHandle, const Sent_ConfigType *ConfigPtr)
```

Initialize SENT configuration.

```
void SENT_SetChannelConfig (const Sent_HandleType *pSentHandle, const Sent_ChannelType  
eChannel, const Sent_ChannelConfigType *pCfg)
```

Set SENT channel configuration.

```
void SENT_SetDMAConfig (const Sent_HandleType *pSentHandle, const Sent_ChannelType eChannel,  
Sent_DMAConfigType *pCfg)
```

Set SENT DMA configuration.

```
void SENT_SetSPCConfig (const Sent_HandleType *pSentHandle, const Sent_ChannelType eChannel,  
const Sent_SpcConfigType *pCfg)
```

Set SENT SPC configuration.

void **SENT_GetFastMessageData** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, **Sent_FastMessageDataType** *pMsg)

Get Fast message data.

void **SENT_GetSlowMessageData** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, **Sent_SlowMessageDataType** *pMsg)

Get Slow message data.

void **SENT_DecomposeFastMessageWithDMABuffer** (uint32_t *pDmaBuffer, uint32_t u32BufferLength, **Sent_FastMessageDataType** *pMsg)

Decode the fast message with DMA buffer.

void **SENT_DecomposeSlowMessageWithDMABuffer** (uint32_t *pDmaBuffer, uint32_t u32BufferLength, **Sent_SlowMessageDataType** *pMsg)

Decode the slow message with DMA buffer.

void **SENT_EnableGlobalFastMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Enable the fast message ready interrupt.

void **SENT_DisableGlobalFastMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Disable the fast message ready interrupt.

void **SENT_EnableGlobalSlowMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Enable the slow message ready interrupt.

void **SENT_DisableGlobalSlowMsgNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Disable the slow message ready interrupt.

void **SENT_EnableGlobalFastFifoMsgNotification** (const **Sent_HandleType** *pSentHandle)

Enable the fast message fifo interrupt.

void **SENT_DisableGlobalFastFifoMsgNotification** (const **Sent_HandleType** *pSentHandle)

Disable the fast message fifo interrupt.

void **SENT_ConfigChannelStatusNotification** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, const **Sent_ChannelInterruptType** *sInterruptEn)

configure the channel interrupt.

void **SENT_RequestSPCPulse** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel)

Request the SPC pulse.

void **SENT_GetFastMsgReadyFlag** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, bool *pFlag)

Get the fast message ready flag.

void **SENT_GetSlowMsgReadyFlag** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, bool *pFlag)

Get the slow message ready flag.

void **SENT_GetChannelStatus** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, uint32_t *pStatus)

Get the channel status.

void **SENT_ClearChannelStatus** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, uint32_t Status)

Clear the channel status.

void **SENT_GetChannelSPCBusyFlag** (const **Sent_HandleType** *pSentHandle, const **Sent_ChannelType** eChannel, bool *pflag)

Get the channel SPC busy flag.

Detailed Description

SENT driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_smc.h File Reference

SMC driver type definition and API.

```
#include "HwA_smc.h"
```

```
#include "HwA_pmc.h"
```

```
#include "HwA_cm7.h"
```

Include dependency graph for module_driver_smc.h:

IMAGE

Macros

```
#define SMC_DEV_ERROR_REPORT STD_OFF
```

```
#define SMC_ReportDevError(func, error) ReportDevError(SMC_MODULE_ID, func, error)
```

```
#define SMC_AONCLK_128K 128000U
```

```
#define SMC_AONCLK_32K 32000U
```

```
#define SMC_AONCLK_SOSC_32K 32768U
```

```
#define SMC_AONCLK_1K 1000U
```

```
#define LP_WAKEUP_PAD_NUM 5U
```

```
SMC API Service IDs#define SMC_SET_CLOCK_OUT_ID 0x00U
```

```
#define SMC_SET_AONCLKSRC_ID 0x01U
```

```
#define SMC_GET_CLOCK_FREQ_ID 0x02U
```

SMC Dev Error Code *Error Code of calling SMC apis*

```
#define SMC_E_PARAM_POINTER 0x01U
```

```
#define SMC_E_PARAM_OUT_RANGE 0x02U
```

Enumerations

```
enum SMC_StatusType { SMC_STATUS_SUCCESS = 0U, SMC_STATUS_PARAM_INVALID = 1U }
```

Smc return type.

```
enum SMC_ModeType { SMC_MODE_RUN = 0U, SMC_MODE_STOP, SMC_MODE_WAIT,  
SMC_MODE_STANBY_0, SMC_MODE_STANBY_1, SMC_MODE_STANBY_2, SMC_MODE_STANBY_3 }
```

Smc mode.

Functions

```
void SMC_SetSystemMode (const SMC_ModeType eMode)
```

Set system mode.

Detailed Description

SMC driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_ssi.h File Reference

ssi driver type definition and API

#include "HwA_ssi.h"

Include dependency graph for module_driver_ssi.h:

IMAGE

Detailed Description

ssi driver type definition and API

Author

Flagchip

Version

2.1.0

Date

2024-12-26

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_stcu.h File Reference

#include "HwA_stcu.h"

Include dependency graph for module_driver_stcu.h:

IMAGE

Classes

struct **STCU_ConfigType**

struct **STCU_InitRamConfigType**

Typedefs

typedef void(* **Stcu_IRQCallback**) (uint32_t u32Status)

Stcu call back function type, the u32status should refer to "STCU_InterruptFlagType".

Enumerations

enum **STCU_SelfTestStatusType** { **STCU_SELFTEST_NONE** = 0x00U, **STCU_SELFTEST_DONE** = 0x01U, **STCU_SELFTEST_ABORT** = 0x02U, **STCU_SELFTEST_LBIST_ERROR** = 0x10U, **STCU_SELFTEST_MBIST_ERROR** = 0x20U, **STCU_SELFTEST_TIMEOUT_ERROR** = 0x40U, **STCU_SELFTEST_NVRLOAD_ERROR** = 0x100U, **STCU_SELFTEST_SELFCHECK_ERROR** = 0x200u }

STCU self test status flag.

enum **STCU_InterruptFlagType** { **STCU_INTERRUPT_FLAG_NONE** = 0x00U, **STCU_INTERRUPT_FLAG_SIZE_ERR** = 0x01U, **STCU_INTERRUPT_FLAG_SEQ_ERR** = 0x02U }

STCU interrupt flag.

enum **STCU_InitRamModeType** { **STCU_INIT_RAM_MODE_POR_OR_WAKEUP4STANDBY0** = 0U, **STCU_INIT_RAM_MODE_AFTER_WAKEUP4STANDBY1**, **STCU_INIT_RAM_MODE_AFTER_WAKEUP4STANDBY2**, **STCU_INIT_RAM_MODE_AFTER_WAKEUP4STANDBY3** }

Hardware SRAM initialization mode.

enum **STCU_HardwareInitRamStatusType** { **STCU_HARDWARE_INIT_RAM_STATUS_NONE** = 0U, **STCU_HARDWARE_INIT_RAM_STATUS_DONE** = 1U, **STCU_HARDWARE_INIT_RAM_STATUS_BUSY** = 2U, **STCU_HARDWARE_INIT_RAM_STATUS_ABORT** = 4U }

Init Ram status flag.

enum **STCU_LbistStatusType** { **STCU_LBIST_STATUS_NONE** = 0U, **STCU_LBIST_STATUS_DONE** = 1U, **STCU_LBIST_STATUS_FAILED** = 2U }

LBIST status flag.

```
enum STCU_LbistClkDivType { STCU_LBIST_CLK_DIVIDER_BY_2 = 1U, STCU_LBIST_CLK_DIVIDER_BY_3  
= 2U, STCU_LBIST_CLK_DIVIDER_BY_4 = 3U }
```

LBIST Clock Divider.

```
enum STCU_ClkSourceType { STCU_CLK_SOURCE_FIRC = 0U, STCU_CLK_SOURCE_PLL = 1U }
```

clock source of self-test.

```
enum STCU_PortPullModeType { STCU_PORT_PULL_DISABLE = 0U, STCU_PORT_PULL_DOWN,  
STCU_PORT_PULL_UP }
```

Self-test Port Pull Selection.

Functions

```
void STCU_Init (STCU_ConfigType *pConfig)
```

Init the STCU module.

```
void STCU_StartSelfTest (void)
```

Trigger to start Software self test.

```
uint32_t STCU_CheckTriggerResult (void)
```

Check Software Trigger Self-test result.

```
void STCU_StartRamInit (STCU_InitRamConfigType *pInitCfg)
```

Initial RAM with hardware.

```
uint32_t STCU_GetRamInitStatus (void)
```

Get status of RAM initialize action.

```
bool STCU_GetLbistFailResult (void)
```

Get if the LBIST test result is Fail.

```
uint32_t STCU_GetMbistFailResult (void)
```

Get each MBIST Fail result.

```
uint32_t STCU_GetRamInitDoneStatus (void)
```

Get each done status of SRAM initialize.

void **STCU_LBIST_Set_Pattern_Misr** (uint16_t u16Pattern, uint32_t u32ExpectedMisr)

Set the LBIST Pattern value and expected misr value. The two value should be load from NVR, but also can reconfigure by this API. The expected misr value is calculated from pattern, so please make sure this two value is right, or the LBST would fail.

uint32_t **STCU_GetLbistAcutalMisr** (void)

Get LBIST actual MISR value, the value should be read and use DFT tool to decode it, if LBIST test resault is fail.

void **STCU0_IRQHandler** (void)

STCU 0 interrupt handler.

Typedef Documentation

typedef void(* Stcu_IRQCallback) (uint32_t u32Status)

Stcu call back function type, the u32status should refer to "STCU_InterruptFlagType".

Enumeration Type Documentation

enum STCU_ClkSourceType

clock source of self-test.

Enumerator:

STCU_CLK_SOU RCE_FIRC	! FIRC clock source. >
STCU_CLK_SOU RCE_PLL	! PLL clock source. >

enum STCU_HardwareInitRamStatusType

Init Ram status flag.

Enumerator:

STCU_HARDWARE_INIT_RAM_STATUS_NONE	! No any SRAM Initialization flag. >
STCU_HARDWARE_INIT_RAM_STATUS_DONE	! Hardware SRAM Initialization Done Flag. >
STCU_HARDWARE_INIT_RAM_STATUS_BUSY	! Hardware SRAM Initialization Busy Flag. >
STCU_HARDWARE_INIT_RAM_STATUS_ABORT	! Hardware SRAM Initialization Abort Flag. >

enum STCU_InitRamModeType

Hardware SRAM initialization mode.

Enumerator:

STCU_INIT_RAM_MODE_POR_OR_WAKEUP4STANDBY0	! After POR or wakeup from STANDBY0, Hardware SRAM initialization. >
STCU_INIT_RAM_MODE_AFTER_WAKEUP4STANDBY1	! After wakeup from STANDBY1, Hardware SRAM initialization. >
STCU_INIT_RAM_MODE_AFTER_WAKEUP4STANDBY2	! After wakeup from STANDBY2, Hardware SRAM initialization. >
STCU_INIT_RAM_MODE_AFTER_WAKEUP4STANDBY3	! After wakeup from STANDBY3, Hardware SRAM initialization. >

enum STCU_InterruptFlagType

STCU interrupt flag.

Enumerator:

STCU_INTERRUPT_FLAG_NONE	! No any Interrupt flag. >
STCU_INTERRUPT_FLAG_SIZE_ERROR	! STCU Register Access Size Error IRQ Flag. >
STCU_INTERRUPT_FLAG_SEQUENCE_ERROR	! Illegal Register Access Sequence IRQ Flag. >

enum STCU_LbistClkDivType

LBIST Clock Divider.

Enumerator:

STCU_LBIST_CLK_DIVIDER_BY_2	! LBIST clock is Divided by 2. >
STCU_LBIST_CLK_DIVIDER_BY_3	! LBIST clock is Divided by 3. >
STCU_LBIST_CLK_DIVIDER_BY_4	! LBIST clock is Divided by 4. >

enum STCU_LbistStatusType

LBIST status flag.

Enumerator:

STCU_LBIST_ST ATUS_NONE	! No any LBIST flag. >
STCU_LBIST_ST ATUS_DONE	! LBIST Done Flag. >
STCU_LBIST_ST ATUS_FAILED	! LBIST Fail Flag. >

enum STCU_PortPullModeType

Self-test Port Pull Selection.

Enumerator:

STCU_PORT_PU LL_DISABLE	! Disable self test port pull function. >
STCU_PORT_PU LL_DOWN	! self test port pull down. >
STCU_PORT_PU LL_UP	! self test port pull up. >

enum STCU_SelfTestStatusType

STCU self test status flag.

Enumerator:

STCU_SELFTEST T_NONE	! No any status flag. >
STCU_SELFTEST T_DONE	! Self-test Done Flag. >
STCU_SELFTEST T_ABORT	! Self-test Abort Flag. >
STCU_SELFTEST T_LBIST_ERROR	! LBIST Error Flag. >

STCU_SELFTEST_MBIST_ERROR	! MBIST Error Flag. >
STCU_SELFTEST_TIMEOUT_ERROR	! Self-test Timeout Error Flag. >
STCU_SELFTEST_NVRLOAD_ERROR	! NVR Load Error Flag. >
STCU_SELFTEST_SELF_CHECK_ERROR	! STCU Self Check Error Flag. >

Function Documentation

void STCU0_IRQHandler (void)

STCU 0 interrupt handler.

Note

This function should be called as/in STCU 0 interrupt handler

uint32_t STCU_CheckTriggerResult (void)

Check Software Trigger Self-test result.

Returns

refer to "STCU_SelfTestStatusType" enum.

uint32_t STCU_GetLbistAcutalMisr (void)

Get LBIST actual MISR value, the value should be read and use DFT tool to decode it, if LBIST test result is fail.

Returns

the LBIST actual MISR value

bool STCU_GetLbistFailResult (void)

Get if the LBIST test result is Fail.

Returns

true means LBIST fail. false means LBIST pass.

uint32_t STCU_GetMbistFailResult (void)

Get each MBIST Fail result.

Returns

refer to "STCU_MbistFailedType" enum.

uint32_t STCU_GetRamInitDoneStatus (void)

Get each done status of SRAM initialize.

Returns

the done status of SRAM initialize action, refer to "STCU_InitRamDoneType"

uint32_t STCU_GetRamInitStatus (void)

Get status of RAM initialize action.

Returns

the status of ram initialize action, refer to "STCU_HardwareInitRamStatusType"

void STCU_Init (STCU_ConfigType * *pConfig*)

Init the STCU module.

Parameters

<i>pConfig</i>	the configuration structure
----------------	-----------------------------

void STCU_LBIST_Set_Pattern_Misr (uint16_t *u16Pattern*, uint32_t *u32ExpectedMisr*)

Set the LBIST Pattern value and expected misr value. The two value should be load from NVR, but also can reconfigure by this API. The expected misr value is calculated from pattern, so please make sure this two value is right, or the LBST would fail.

Parameters

<i>u16Pattern</i>	the LBIST Pattern value
<i>u16Pattern</i>	the LBIST expected misr value

void STCU_StartRamInit (STCU_InitRamConfigType * *pInitCfg*)

Initial RAM with hardware.

Parameters

<i>pConfig</i>	the configuration structure
----------------	-----------------------------

void STCU_StartSelfTest (void)

Trigger to start Software self test.

module_driver_tmu.h File Reference

TMU driver type definition and API.

#include "HwA_tmu.h"

Include dependency graph for module_driver_tmu.h:

IMAGE

Classes

struct **_TMU_HandleType**

The structure of the TMU processing handle.

struct **TMU_InitType**

Defines the temperature sensor configuration.

Macros

TMU API Service IDs#define **TMU_INIT_ID** 0U

TMU Dev Error Code*Error Code of calling TMU apis*

#define **TMU_E_PARAM_INSTANCE** 0x01U

#define **TMU_E_PARAM_POINTER** 0x02U

Typedefs

typedef struct **_TMU_HandleType** **TMU_HandleType**

The structure of the TMU processing handle.

Enumerations

enum **TMU_InstanceType** { **TMU_INSTANCE_0** = 0U, **TMU_INSTANCE_MAX** }

TMU instance declaration.

enum **TMU_StatusType** { **TMU_STATUS_ERROR** = 0x0U, **TMU_STATUS_SUCCESS** = 0x1U, **TMU_STATUS_TIMEOUT** = 0x2U }

TMU operation return values.

Functions

void **TMU_ProcessIRQHandler** (**TMU_HandleType** *pTmuHandle)

TMU interrupt process.

void **TMU_Init** (TMU_HandleType *pTmuHandle, const TMU_InitType *const pInitCfg)
Initialize the TMU instance.

TMU_StatusType **TMU_FlagTempEnable** (TMU_HandleType *pTmuHandle)
Enable the Flag-based temperature sensor.

TMU_StatusType **TMU_FlagTempDisable** (TMU_HandleType *pTmuHandle)
Disable the Flag-based temperature sensor.

TMU_StatusType **TMU_VoltageTempEnable** (TMU_HandleType *pTmuHandle)
Enable the Voltage-based temperature sensor.

TMU_StatusType **TMU_VoltageTempDisable** (TMU_HandleType *pTmuHandle)
Disable the Voltage-based temperature sensor.

void **TMU_TempOverClear** (TMU_HandleType *pTmuHandle)
Check if cleaning flag is required.

Detailed Description

TMU driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_tpu.h File Reference

#include "HwA_tpuh.h"

#include "HwA_tpue.h"

Include dependency graph for module_driver_tpu.h:

IMAGE

module_driver_trgsel.h File Reference

TRGSEL driver type definition and API.

```
#include "HwA_trgsel.h"
```

Include dependency graph for module_driver_trgsel.h:

IMAGE

Macros

```
TRGSEL API Service IDs#define TRGSEL_GET_TARGET_TRIGGER_SOURCE_ID 0U
#define TRGSEL_SET_TARGET_TRIGGER_SOURCE_ID 1U
#define TRGSEL_GET_TARGET_LOCK_STATUS_ID 2U
#define TRGSEL_LOCK_TARGET_TRIGGER_SOURCE_ID 3U
#define TRGSEL_GENERATE_SW_TRIGGER_ID 4U
```

TRGSEL Dev Error Code*Error Code of calling TRGSEL apis*

```
#define TRGSEL_E_PARAM_INSTANCE_NUMBER 0x01U
#define TRGSEL_E_INVALID_TARGET 0x02U
#define TRGSEL_E_INVALID_SOURCE 0x03U
#define TRGSEL_E_TARGET_LOCKED 0x04U
#define TRGSEL_E_INVALID_SW_TRIGGIER 0x05U
```

Typedefs

```
typedef uint32_t TRGSEL_TargetType
TrgSel trigger targets.
```

```
typedef uint32_t TRGSEL_SourceType
TrgSel trigger sources.
```

Enumerations

```
enum TRGSEL_SwTriggerChannelType { TRGSEL_SW_TRIGGER_CHANNEL_0 = 0U,
TRGSEL_SW_TRIGGER_CHANNEL_1 = 1U, TRGSEL_SW_TRIGGER_CHANNEL_2 = 2U,
TRGSEL_SW_TRIGGER_CHANNEL_3 = 3U, TRGSEL_SW_TRIGGER_CHANNEL_4 = 4U,
TRGSEL_SW_TRIGGER_CHANNEL_5 = 5U, TRGSEL_SW_TRIGGER_CHANNEL_6 = 6U,
TRGSEL_SW_TRIGGER_CHANNEL_7 = 7U }
```

The TrgSel eInstance to select.

Functions

TRGSEL_SourceType TRGSEL_GetTargetTriggerSource (const TRGSEL_InstanceType eInstance, const TRGSEL_TargetType eTarget)

Get the trigger source of the selected trigger eTarget.

void TRGSEL_SetTargetTriggerSource (const TRGSEL_InstanceType eInstance, const TRGSEL_TargetType eTarget, const TRGSEL_SourceType eSource)

Set the trigger source of the selected trigger target.

bool TRGSEL_GetTargetLockStatus (const TRGSEL_InstanceType eInstance, const TRGSEL_TargetType eTarget)

Get whether the trigger source of the selected target is locked.

void TRGSEL_LockTargetTriggerSource (const TRGSEL_InstanceType eInstance, const TRGSEL_TargetType eTarget)

Lock the trigger source of the selected target.

void TRGSEL_GenerateSwTrigger (const TRGSEL_SwTriggerChannelType eChannel)

Generate trigger signal for the selected software trigger source channel.

Detailed Description

TRGSEL driver type definition and API.

Author

flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_tstmp.h File Reference

TSTMP driver type definition and API.

```
#include "HwA_tstmp.h"
```

Include dependency graph for module_driver_tstmp.h:

IMAGE

Classes

struct **_TSTMP_HandleType**

The structure of the Tstmp processing handle.

struct **TSTMP_IntType**

TSTMP interrupt type.

struct **TSTMP_InitType**

TSTMP initialization type.

struct **TSTMP_UpdateType**

TSTMP update type.

Macros

```
#define MAX_MOD_NUMBER 4U
```

TSTMP max mod number.

TSTMP API Service IDs#define **TSTMP_INIT_ID** 0U

```
#define TSTMP_DEINIT_ID 1U
```

```
#define TSTMP_SET_COUNTER_MODE_ID 2U
```

```
#define TSTMP_START_COUNTER_ID 3U
```

```
#define TSTMP_STOP_COUNTER_ID 4U
```

```
#define TSTMP_INIT_INTERRUPT_ID 5U
```

```
#define TSTMP_EN_INTERRUPT_ID 6U
```

```
#define TSTMP_DIS_INTERRUPT_ID 7U
```

```
#define TSTMP_GET_TSTMP_VALUE_ID 8U
```

```
#define TSTMP_UPDATE_MOD_ID 9U
```

TSTMP Dev Error Code*Error Code of calling TSTMP apis*

```
#define TSTMP_E_PARAM_INSTANCE 0x01U
```

```
#define TSTMP_E_PARAM_CHANNEL 0x02U
```

```
#define TSTMP_E_PARAM_POINTER 0x03U
#define TSTMP_E_PARAM_COUNT 0x04U
#define TSTMP_E_PARAM_INVALID 0x05U
```

Typedefs

```
typedef struct _TSTMP_HandleType TSTMP_HandleType
```

The structure of the Tstmp processing handle.

Enumerations

```
enum TSTMP_StatusType { TSTMP_STATUS_SUCCESS = 0U, TSTMP_STATUS_PARAM_INVALID = 1U }
```

TSTMP return type.

```
enum TSTMP_InstanceType { TSTMP_INSTANCE_0 = 0U, TSTMP_INSTANCE_1 = 1U,
TSTMP_INSTANCE_MAX }
```

TSTMP instance.

Functions

```
void TSTMP_Init (TSTMP_HandleType *pTstmpHandle, const TSTMP_InitType *const pInitStruct)
```

Initialize TSTMP instance.

```
void TSTMP_Deinit (TSTMP_HandleType *pTstmpHandle)
```

De-initialize TSTMP instance.

```
void TSTMP_InitInterrupt (TSTMP_HandleType *pTstmpHandle, const TSTMP_IntType *const pIntStruct)
```

Initialize TSTMP interrupt functionality.

```
void TSTMP_EnableInterrupt (TSTMP_HandleType *pTstmpHandle, const TSTMP_ModChannelType
eMod)
```

Enable TSTMP interrupt function.

```
void TSTMP_DisableInterrupt (TSTMP_HandleType *pTstmpHandle, const TSTMP_ModChannelType
eMod)
```

Disable TSTMP interrupt function.

```
void TSTMP_GetTstmpValue (TSTMP_HandleType *pTstmpHandle, uint64_t *const u64TstmpValue)
```

Get TSTMP count value.

```
void TSTMP_UpdateMod (TSTMP_HandleType *pTstmpHandle, const TSTMP_UpdateType *const  
pUpdateStruct)
```

Update Modulate configuration.

```
void TSTMPn_IRQHandler (TSTMP_HandleType *pTstmpHandle)
```

TSTMP interrupt process function.

Detailed Description

TSTMP driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_wdog.h File Reference

WDOG driver type definition and API.

#include "HwA_wdog.h"

Include dependency graph for module_driver_wdog.h:

IMAGE

Classes

struct **_WDOG_HandleType**

The structure of the WDOG processing handle.

struct **WDOG_CfgType**

WDOG init module type, include register setting for WDOG_CS/WDOG_TIMEOUT/WDOG_WINDOW.

Macros

WDOG API Service IDs#define **WDOG_INIT_ID** 0U

WDOG Dev Error Code*Error Code of calling WDOG apis*

#define **WDOG_E_PARAM_INSTANCE** 0x01U

#define **WDOG_E_PARAM_TIMEOUT** 0x02U

Typedefs

typedef struct **_WDOG_HandleType** **WDOG_HandleType**

The structure of the WDOG processing handle.

Enumerations

enum **WDOG_StatusType** { **WDOG_STATUS_SUCCESS** = 0x0U, **WDOG_STATUS_ERROR** = 0x1U, **WDOG_STATUS_ERROR_UNLOCK** = 0x2U, **WDOG_STATUS_ERROR_RECFG** = 0x3U }

WDOG initialization return values.

enum **WDOG_InstanceType** { **WDOG_INSTANCE_0** = 0U, **WDOG_INSTANCE_1** = 1U, **WDOG_INSTANCE_2** = 2U, **WDOG_INSTANCE_3** = 3U, **WDOG_INSTANCE_MAX** }

WDOG clock instance declaration.

enum **WDOG_ClkSelType** { **WDOG_BUS_CLK** = 0x00U, **WDOG_AON_CLK** = 0x01U, **WDOG_FOSC_CLK** = 0x02U, **WDOG_SIRC_CLK** = 0x03U }

WDOG clock source select, mapping with WDOG_CS[CLK_SEL] bit filed definition.

```
enum WDOG_TestType { WDOG_TST_DISABLED = 0x00U, WDOG_USER_MODE = 0x01U,  
WDOG_TST_MODE_LOW = 0x02U, WDOG_TST_MODE_HIGH = 0x03U }
```

WDOG test mode, mapping with WDOG_CS[TST] bit field definition.

```
enum WDOG_TimeoutReactionType { WDOG_REACTION_INT_128_CYCLE = 0x00U,  
WDOG_REACTION_INT_256_CYCLE = 0x01U, WDOG_REACTION_INT_512_CYCLE = 0x02U,  
WDOG_REACTION_INT_1024_CYCLE = 0x03U, WDOG_REACTION_NO_INT = 0x04U }
```

WDOG delay time to generate the reset.

Functions

```
void WDOG_Unlock (WDOG_HandleType *pWdogHandle)
```

unlock the wdog before Watch dog reconfigure set.

```
void WDOG_Refresh (WDOG_HandleType *pWdogHandle)
```

feed the watch dog by writing typical CMD to counter.

```
WDOG_StatusType WDOG_Init (WDOG_HandleType *pWdogHandle, const WDOG_CfgType *const  
pWdogCfg)
```

Initialize the WDOG configuration setting.

```
void WDOG_SelfTest (WDOG_HandleType *pWdogHandle, WDOG_TestType eMode)
```

Performs a self-test on the Watchdog Timer (WDOG) module.

```
void WDOG_DeInit (WDOG_HandleType *pWdogHandle)
```

De-initialize the WDOG instance.

```
void WDOG_IRQHandler (WDOG_HandleType *pWdogHandle)
```

Wdg n interrupt process.

Detailed Description

WDOG driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.

module_driver_wku.h File Reference

WKU driver type definition and API.

```
#include "HwA_wku.h"
```

Include dependency graph for module_driver_wku.h:

IMAGE

Enumerations

```
enum WKU_StatusType { WKU_STATUS_SUCCESS = 0U, WKU_STATUS_PARAM_INVALID = 1U }
```

WKU return type.

Functions

```
WKU_StatusType WKU_EnableWakeupSource (const uint32_t u32Input)
```

WKU Enable wakeup source for single input.

```
WKU_StatusType WKU_DisableWakeupSource (const uint32_t u32Input)
```

WKU Disable wakeup source for single input.

```
uint32_t WKU_GetWakeupSources (void)
```

WKU get wakeup source.

Detailed Description

WKU driver type definition and API.

Author

Flagchip

Version

2.0.0

Date

2024-08-20

Copyright

Copyright (c) 2020-2024 Flagchip Semiconductors Co., Ltd.